Apache Doris V3.x 中文手册

20251128

目录

1 快速开始 15

1.1 Apache Doris 简介 . . . 15

1.1.1 Apache Doris 简介 . . . 15

1.1.2 使用场景 . . . 15

1.1.3 整体架构 . . . 17

1.1.4 Apache Doris 的核心特性 . . . 18

1.1.5 技术特点 . . . 19

1.2 技术对比 . . . 22

2 使用指南 22

2.1 安装部署 . . . 22

2.1.1 部署前准备 . . . 22

2.1.2 手动部署集群 . . . 28

2.2 第 5 步：启动 FE Master 节点 . . . 39

2.3 第 6 步：注册 FE Follower/Observer 节点 . . . 40

2.4 第 7 步：添加 BE 节点 . . . 40

2.5 第 8 步：添加 Storage Vault . . . 41

2.6 注意事项 . . . 42

2.6.1 在 Kubernetes 部署 . . . 42

2.6.2 云上部署集群 . . . 104

2.7 数据库连接 . . . 109

2.7.1 通过 MySQL 协议连接 . . . 109

2.7.2 基于 Arrow Flight SQL 的高速数据传输链路 . . . 114

2.8 数据表设计 · · · · · 131
2.8.1 概览 · · · · · 131
2.8.2 表类型 · · · · · 132
2.8.3 数据划分 · · · · · 147
2.8.4 数据类型 · · · · · 173
2.8.5 数据压缩 · · · · · 176
2.8.6 表索引 · · · · · 178
2.8.7 Schema 变更 · · · · · 204
2.8.8 自增列 · · · · · 211
2.8.9 冷热数据分层 · · · · · 220
2.8.10 行列混存 · · · · · 228
2.8.11 数据库建表最佳实践 · · · · · 230
2.9 数据导入 · · · · · 243
2.9.1 导入概览 · · · · · 243
2.9.2 数据源 · · · · · 244
2.9.3 导入方式 · · · · · 304
2.9.4 文件格式 · · · · · 418
2.9.5 复杂数据类型 · · · · · 431
2.9.6 异常数据处理 · · · · · 449
2.9.7 导入时实现数据转换 · · · · · 455
2.9.8 导入高可用性 · · · · · 481
2.9.9 高并发导入优化（Group Commit） · · · · · 483
2.9.10 导入最佳实践 · · · · · 499
2.10 数据更新与删除 · · · · · 500
2.10.1 数据更新 · · · · · 500
2.10.2 数据删除 · · · · · 526
2.10.3 事务 · · · · · 541
2.11 数据导出 · · · · · 551
2.11.1 数据导出概述 · · · · · 551
2.11.2 Export · · · · · 555
2.11.3 SELECT INTO OUTFILE · · · · · 563
2.11.4 MySQL Dump · · · · · 569
2.11.5 最佳实践 · · · · · 569

2.12 数据查询 · · · · · 572
2.12.1 MySQL 兼容性 · · · · · 572
2.12.2 连接（JOIN） · · · · · 578
2.12.3 子查询 · · · · · 590
2.12.4 聚合多维分析 · · · · · 596
2.12.5 分析函数（窗口函数） · · · · · 607
2.12.6 公用表表达式（CTE） · · · · · 624
2.12.7 自定义函数 · · · · · 625
2.12.8 复杂类型查询 · · · · · 637
2.12.9 列转行 (Lateral View) · · · · · 637
2.13 查询加速 · · · · · 639
2.13.1 查询调优概述 · · · · · 639
2.13.2 物化视图 · · · · · 648
2.13.3 查询缓存 · · · · · 726
2.13.4 高并发点查 · · · · · 731
2.13.5 高效去重 · · · · · 736
2.13.6 Colocation Join · · · · · 742
2.13.7 Hints · · · · · 751
2.13.8 查询优化实践 · · · · · 766
2.13.9 优化技术原理 · · · · · 793
2.14 湖仓一体 · · · · · 827
2.14.1 湖仓一体概述 · · · · · 827
2.14.2 数据目录概述 · · · · · 835
2.14.3 数据目录 · · · · · 841
2.14.4 分析 S3/HDFS 上的文件 · · · · · 894
2.14.5 元数据服务 · · · · · 899
2.14.6 存储服务 · · · · · 921
2.14.7 开放文件格式 · · · · · 946
2.14.8 数据缓存 · · · · · 950
2.14.9 元数据缓存 · · · · · 953
2.14.10 弹性计算节点 · · · · · 960
2.14.11 统计信息 · · · · · 962
2.14.12 SQL 方言转换 · · · · · 962
2.14.13 湖仓一体最佳实践 · · · · · 972

2.15 可观测性 · · · · · 1063
2.15.1 Log · · · · · 1063
2.15.2 Trace · · · · · 1075
2.15.3 集成 · · · · · 1080
2.16 存算分离 · · · · · 1128
2.16.1 概览 · · · · · 1128
2.16.2 Doris 存算分离模式部署准备 · · · · · 1131
2.16.3 编译部署 · · · · · 1135
2.16.4 管理 Storage Vault · · · · · 1141
2.16.5 计算组操作 · · · · · 1146
2.16.6 文件缓存 · · · · · 1149
2.16.7 数据回收 · · · · · 1159
2.16.8 升级 · · · · · 1169
2.17 安全合规 · · · · · 1173
2.17.1 安全概览 · · · · · 1173
2.17.2 认证与鉴权 · · · · · 1173
2.17.3 审计日志 · · · · · 1207
2.17.4 数据加密 · · · · · 1208
2.17.5 集成 · · · · · 1211

3 性能测试 1231
3.1 Star Schema Benchmark · · · · · 1231
3.1.1 1. 硬件环境 · · · · · 1231
3.1.2 2. 软件环境 · · · · · 1231
3.1.3 3. 测试数据量 · · · · · 1231
3.1.4 4. SSB 宽表测试结果 · · · · · 1232
3.1.5 5. 标准 SSB 测试结果 · · · · · 1232
3.1.6 6. 环境准备 · · · · · 1233
3.1.7 7. 数据准备 · · · · · 1233

3.2 TPC-H Benchmark · · · · · 1243
3.2.1 1. 硬件环境 · · · · · 1243
3.2.2 2. 软件环境 · · · · · 1244
3.2.3 3. 测试数据量 · · · · · 1244
3.2.4 4. 测试 SQL · · · · · 1244
3.2.5 5. 测试结果 · · · · · 1244
3.2.6 6. 环境准备 · · · · · 1245
3.2.7 7. 数据准备 · · · · · 1245
3.3 TPC-DS Benchmark · · · · · 1262
3.3.1 1. 硬件环境 · · · · · 1262
3.3.2 2. 软件环境 · · · · · 1263
3.3.3 3. 测试数据量 · · · · · 1263
3.3.4 4. 测试 SQL · · · · · 1263
3.3.5 5. 测试结果 · · · · · 1264
3.3.6 6. 环境准备 · · · · · 1266
3.3.7 7. 数据准备 · · · · · 1266
4 管理指南 1268
4.1 集群管理 · · · · · 1268
4.1.1 集群升级 · · · · · 1268
4.1.2 弹性扩缩容 · · · · · 1272
4.1.3 负载均衡 · · · · · 1275
4.1.4 时区管理 · · · · · 1286
4.1.5 FQDN · · · · · 1291
4.2 负载管理 · · · · · 1293
4.2.1 负载管理概述 · · · · · 1293
4.2.2 资源隔离 · · · · · 1294
4.2.3 工作负载分析与诊断 · · · · · 1322
4.2.4 并发控制与排队 · · · · · 1325
4.2.5 查询熔断 · · · · · 1327
4.2.6 终止查询 · · · · · 1337
4.2.7 调度管理 · · · · · 1340

4.3 容灾管理 · · · · · 1345
4.3.1 容灾管理概览 · · · · · 1345
4.3.2 备份与恢复 · · · · · 1346
4.3.3 跨集群复制 · · · · · 1354
4.3.4 从回收站恢复 · · · · · 1386
4.4 日志管理 · · · · · 1387
4.4.1 FE 日志管理 · · · · · 1387
4.4.2 BE 日志管理 · · · · · 1415
4.5 运维监控 · · · · · 1434
4.5.1 监控指标 · · · · · 1434
4.5.2 监控和报警 · · · · · 1456
4.5.3 磁盘空间管理 · · · · · 1464
4.5.4 数据副本管理 · · · · · 1468
4.5.5 服务自动拉起 · · · · · 1487
4.6 配置管理 · · · · · 1494
4.6.1 配置文件目录 · · · · · 1494
4.6.2 FE 配置项 · · · · · 1494
4.6.3 BE 配置项 · · · · · 1541
4.6.4 用户配置项 · · · · · 1577
4.7 系统表 · · · · · 1578
4.7.1 概述 · · · · · 1578
4.7.2 information_schema · · · · · 1578
4.7.3 mysql · · · · · 1578
4.7.4 __internal_schema · · · · · 1580
4.8 故障诊断处理 · · · · · 1580
4.8.1 内存管理 · · · · · 1580
4.8.2 Compaction 优化 · · · · · 1619
4.8.3 元数据运维 · · · · · 1621
4.8.4 FE 锁管理 · · · · · 1629
4.8.5 Tablet 本地调试 · · · · · 1631
4.8.6 Tablet 元数据管理工具 · · · · · 1635
4.8.7 数据修复 · · · · · 1638

4.9 OPEN API · · · · · 1638
4.9.1 总览 · · · · · 1638
4.9.2 FE HTTP API · · · · · 1641
4.9.3 BE HTTP API · · · · · 1745

5 生态扩展 1765

5.1 Spark Doris Connector · · · · · 1765
5.1.1 版本兼容 · · · · · 1765
5.1.2 使用 · · · · · 1766
5.1.3 使用示例 · · · · · 1767
5.1.4 配置 · · · · · 1773
5.1.5 Doris 和 Spark 列类型映射关系 · · · · · 1775
5.1.6 常见问题 · · · · · 1776
5.2 Flink Doris Connector · · · · · 1778
5.2.1 版本说明 · · · · · 1778
5.2.2 使用方式 · · · · · 1778
5.2.3 使用原理 · · · · · 1779
5.2.4 快速上手 · · · · · 1780
5.2.5 场景与操作 · · · · · 1782
5.2.6 使用说明 · · · · · 1794
5.2.7 最佳实践 · · · · · 1799
5.2.8 常见问题 · · · · · 1802
5.3 Doris Kafka Connector · · · · · 1803
5.3.1 版本说明 · · · · · 1803
5.3.2 使用方式 · · · · · 1804
5.3.3 配置项 · · · · · 1807
5.3.4 类型映射 · · · · · 1809
5.3.5 最佳实践 · · · · · 1810
5.3.6 常见问题 · · · · · 1815
5.4 Doris Operator · · · · · 1818
5.4.1 Doris Kubernetes Operator · · · · · 1818
5.4.2 在阿里云上的部署建议 · · · · · 1820
5.4.3 在 AWS 上的部署建议 · · · · · 1823

5.5 Doris Streamloader · · · · · 1825
5.5.1 概述 · · · · · 1825
5.5.2 获取与安装 · · · · · 1825
5.5.3 使用方法 · · · · · 1825
5.5.4 结果说明 · · · · · 1835
5.5.5 最佳实践 · · · · · 1836
5.6 BI · · · · · 1837
5.6.1 Apache Superset · · · · · 1837
5.6.2 FineBI · · · · · 1848
5.6.3 Power BI · · · · · 1856
5.6.4 Tableau · · · · · 1880
5.6.5 QuickSight · · · · · 1890
5.6.6 Quick BI · · · · · 1904
5.6.7 Smartbi · · · · · 1906
5.7 SQL Clients · · · · · 1910
5.7.1 CloudDM · · · · · 1910
5.7.2 DBeaver · · · · · 1913
5.7.3 DataGrip · · · · · 1921
5.8 可观测性 · · · · · 1926
5.8.1 Logstash · · · · · 1926
5.8.2 Filebeat · · · · · 1939
5.8.3 OpenTelemetry · · · · · 1950
5.9 More · · · · · 1975
5.9.1 CloudCanal · · · · · 1975
5.9.2 DataX Doriswriter · · · · · 1978
5.9.3 DBT Doris Adapter · · · · · 1987
5.9.4 Seatunnel Doris Sink · · · · · 1997
5.9.5 Kettle Doris Plugin · · · · · 2003
5.9.6 Kyuubi · · · · · 2005
5.9.7 AutoMQ Load · · · · · 2007
5.9.8 Hive Bitmap UDF · · · · · 2011
5.9.9 Hive HLL UDF · · · · · 2014

6 常见问题 2018
6.1 常见运维问题 2018
6.2 数据操作问题 2026
6.3 常见查询问题 2030
6.4 常见数据湖问题 2032
6.4.1 证书问题 2032
6.4.2 Kerberos 2032
6.4.3 JDBC Catalog 2034
6.4.4 Hive Catalog 2035
6.4.5 HDFS 2039
6.4.6 DLF Catalog 2040
6.4.7 其他问题 2040
6.5 常见 BI 问题 2041
6.5.1 Power BI 2041
6.5.2 Tableau 2041
6.6 数据正确性问题 2042
6.7 常见导入问题 2058
6.7.1 导入通用问题 2058
6.7.2 Stream Load 2059
6.7.3 Routine Load 2060
7 SQL 手册 2062
7.1 基础元素 2062
7.1.1 SQL 类型 2062
7.1.2 字面量 2149
7.1.3 NULL 2152
7.1.4 对象标识符 2153
7.1.5 保留关键字 2154
7.1.6 变量 2159
7.1.7 注释 2163
7.1.8 操作符 2163

7.2 SQL 函数 · · · · · 2169
7.2.1 标量函数 · · · · · 2169
7.2.2 聚合函数 · · · · · 2712
7.2.3 函数 Combinators · · · · · 2804
7.2.4 分析函数 (窗口函数) · · · · · 2807
7.2.5 表函数 · · · · · 2823
7.2.6 表值函数 · · · · · 2843
7.3 SQL 语句 · · · · · 2883
7.3.1 数据查询 · · · · · 2883
7.3.2 数据修改 · · · · · 2901
7.3.3 账户、角色和权限 · · · · · 3002
7.3.4 会话 · · · · · 3031
7.3.5 事务 · · · · · 3049
7.3.6 数据目录 · · · · · 3053
7.3.7 数据库 · · · · · 3065
7.3.8 表和视图 · · · · · 3073
7.3.9 回收站 · · · · · 3205
7.3.10 函数 · · · · · 3209
7.3.11 统计信息 · · · · · 3220
7.3.12 集群管理 · · · · · 3232
7.3.13 安全合规 · · · · · 3291
7.3.14 数据治理 · · · · · 3301
7.3.15 后台任务 · · · · · 3304
7.3.16 插件 · · · · · 3310
7.3.17 字符集 · · · · · 3313
7.3.18 类型 · · · · · 3315
7.3.19 系统信息和帮助 · · · · · 3317

8 版本发布 3328

8.1 最新发布 · · · · · 3328
8.2 v3.1 · · · · · 3330
8.2.1 Release 3.1.3 · · · · · 3330
8.2.2 Release 3.1.2 · · · · · 3333
8.2.3 Release 3.1.1 · · · · · 3334
8.2.4 Release 3.1.0 · · · · · 3340
8.3 v3.0 · · · · · 3355
8.3.1 Release 3.0.8 · · · · · 3355
8.3.2 Release 3.0.7 · · · · · 3357
8.3.3 Release 3.0.6 · · · · · 3361
8.3.4 Release 3.0.5 · · · · · 3365
8.3.5 Release 3.0.4 · · · · · 3370
8.3.6 Release 3.0.3 · · · · · 3375
8.3.7 Release 3.0.2 · · · · · 3380
8.3.8 Release 3.0.1 · · · · · 3387
8.3.9 Release 3.0.0 · · · · · 3399
8.4 v2.1 · · · · · 3414
8.4.1 Release 2.1.11 · · · · · 3414
8.4.2 Release 2.1.10 · · · · · 3416
8.4.3 Release 2.1.9 · · · · · 3419
8.4.4 Release 2.1.8 · · · · · 3421
8.4.5 Release 2.1.7 · · · · · 3425
8.4.6 Release 2.1.6 · · · · · 3429
8.4.7 Release 2.1.5 · · · · · 3439
8.4.8 Release 2.1.4 · · · · · 3447
8.4.9 Release 2.1.3 · · · · · 3453
8.4.10 Release 2.1.2 · · · · · 3456
8.4.11 Release 2.1.1 · · · · · 3458
8.4.12 Release 2.1.0 · · · · · 3464

8.5 v2.0 · · · · · 3493
8.5.1 Release 2.0.15 · · · · · 3493
8.5.2 Release 2.0.14 · · · · · 3495
8.5.3 Release 2.0.13 · · · · · 3495
8.5.4 Release 2.0.12 · · · · · 3496
8.5.5 Release 2.0.11 · · · · · 3497
8.5.6 Release 2.0.10 · · · · · 3498
8.5.7 Release 2.0.9 · · · · · 3498
8.5.8 Release 2.0.8 · · · · · 3499
8.5.9 Release 2.0.7 · · · · · 3500
8.5.10 Release 2.0.6 · · · · · 3501
8.5.11 Release 2.0.5 · · · · · 3502
8.5.12 Release 2.0.4 · · · · · 3503
8.5.13 Release 2.0.3 · · · · · 3504
8.5.14 Release 2.0.2 · · · · · 3511
8.5.15 Release 2.0.1 · · · · · 3516
8.5.16 功能改进 · · · · · 3516
8.5.17 问题修复 · · · · · 3517
8.5.18 致谢 · · · · · 3518
8.5.19 Release 2.0.0 · · · · · 3519
8.6 v1.2 · · · · · 3526
8.6.1 Release 1.2.8 · · · · · 3526
8.6.2 Release 1.2.7 · · · · · 3526
8.6.3 最新特性 · · · · · 3527
8.6.4 Release 1.2.6 · · · · · 3527
8.6.5 Bug Fixes · · · · · 3527
8.6.6 致谢 · · · · · 3528
8.6.7 Release 1.2.5 · · · · · 3529
8.6.8 Improvement · · · · · 3529
8.6.9 Bug Fixes · · · · · 3529
8.6.10 致谢 · · · · · 3530
8.6.11 Release 1.2.4 · · · · · 3532

8.6.12 Improvement · · · · · 3532
8.6.13 Bug Fixes · · · · · 3533
8.6.14 致谢 · · · · · 3533
8.6.15 Release 1.2.3 · · · · · 3535
8.6.16 Bug 修复 · · · · · 3536
8.6.17 Release 1.2.2 · · · · · 3536
8.6.18 Behavior Changes · · · · · 3537
8.6.19 Improvement · · · · · 3537
8.6.20 BugFix · · · · · 3538
8.6.21 其他 · · · · · 3539
8.6.22 致谢 · · · · · 3539
8.6.23 Release 1.2.1 · · · · · 3541
8.6.24 问题修复 · · · · · 3542
8.6.25 升级注意事项 · · · · · 3542
8.6.26 致谢 · · · · · 3542
8.6.27 Release 1.2.0 · · · · · 3544
8.6.28 重要更新 · · · · · 3545
8.6.29 更多功能 · · · · · 3553
8.6.30 升级注意事项 · · · · · 3556
8.6.31 其他 · · · · · 3559
8.6.32 致谢 · · · · · 3559
8.7 v1.1 · · · · · 3562
8.7.1 Release 1.1.5 · · · · · 3562
8.7.2 Features · · · · · 3562
8.7.3 Improvements · · · · · 3562
8.7.4 Bug Fix · · · · · 3563
8.7.5 Release 1.1.4 · · · · · 3563
8.7.6 优化改进 · · · · · 3563
8.7.7 Bug 修复 · · · · · 3563
8.7.8 Release 1.1.3 · · · · · 3564
8.7.9 优化改进 · · · · · 3564
8.7.10 Bug 修复 · · · · · 3564

8.7.11 升级说明 · · · · · 3565
8.7.12 Release 1.1.2 · · · · · 3565
8.7.13 优化改进 · · · · · 3566
8.7.14 Bug Fix · · · · · 3566
8.7.15 Release 1.1.1 · · · · · 3567
8.7.16 Release 1.1.0 · · · · · 3568

1 快速开始

1.1 Apache Doris 简介

1.1.1 Apache Doris 简介

Apache Doris 是一款基于 MPP 架构的高性能、实时分析型数据库。它以高效、简单和统一的特性著称，能够在亚秒级的时间内返回海量数据的查询结果。Doris 既能支持高并发的点查询场景，也能支持高吞吐的复杂分析场景。

基于这些优势，Apache Doris 非常适合用于报表分析、即席查询、统一数仓构建、数据湖联邦查询加速等场景。用户可以基于 Doris 构建大屏看板、用户行为分析、AB 实验平台、日志检索分析、用户画像分析、订单分析等应用。

1.1.1.1 发展历程

Apache Doris 最初是百度广告报表业务的 Palo 项目。2017 年正式对外开源，2018 年 7 月由百度捐赠给 Apache 基金会进行孵化。在 Apache 导师的指导下，由孵化器项目管理委员会成员进行孵化和运营。2022 年 6 月，Apache Doris 成功从 Apache 孵化器毕业，正式成为 Apache 顶级项目（Top-Level Project，TLP）。

目前，Apache Doris 社区已经聚集了来自不同行业数百家企业的 700 余位贡献者，并且每月活跃贡献者人数超过 120 位。

1.1.1.2 应用现状

Apache Doris 在中国乃至全球范围内拥有广泛的用户群体。截至目前，Apache Doris 已经在全球超过 5000 家中大型企业的生产环境中得到应用。在中国市值或估值排行前 50 的互联网公司中，有超过 80% 长期使用 Apache Doris，包括百度、美团、小米、京东、字节跳动、阿里巴巴、腾讯、网易、快手、微博等。同时，在金融、消费、电信、工业制造、能源、医疗、政务等传统行业也有着丰富的应用。

在中国，几乎所有的云厂商，如阿里云、华为云、天翼云、腾讯云、百度云、火山引擎等，都在提供托管的 Apache Doris 云服务。

1.1.2 使用场景

数据源经过各种数据集成和加工处理后，通常会进入实时数据仓库 Doris 和离线湖仓（如 Hive、Iceberg 和 Hudi），广泛应用于 OLAP 分析场景，如下图所示：

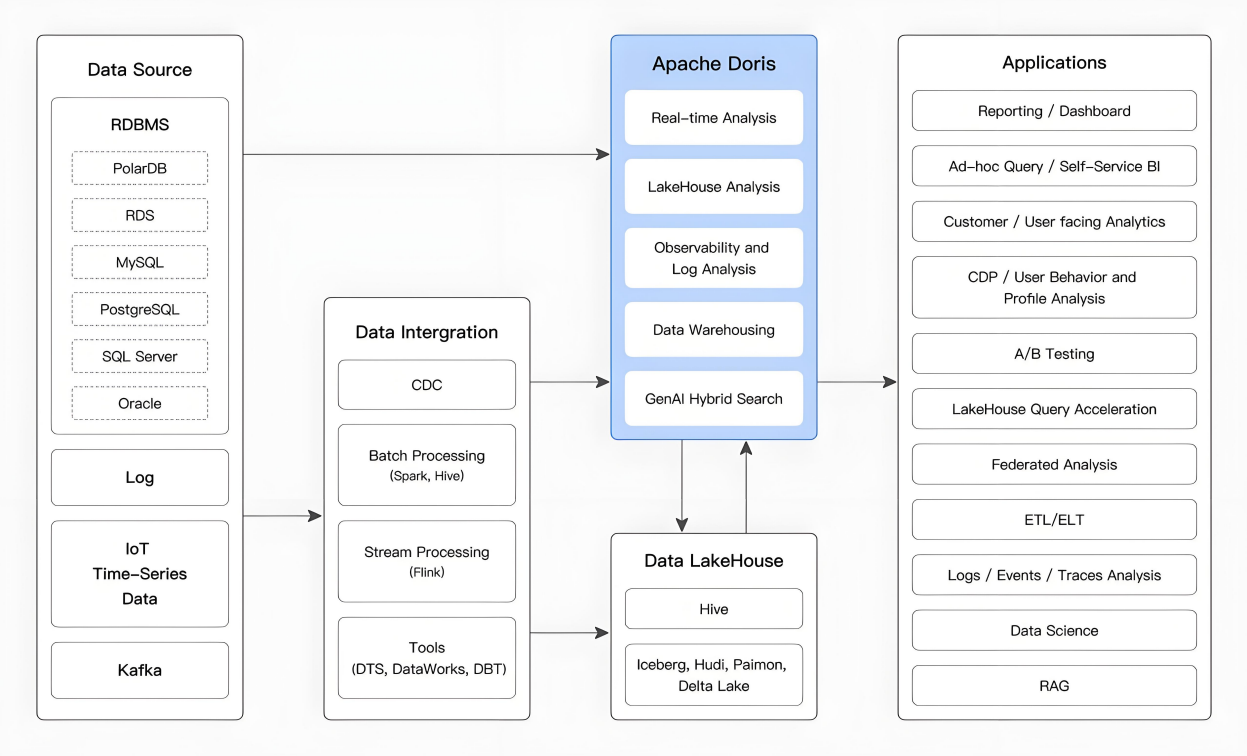


图 1: Apache Doris 的使用场景

Apache Doris 主要应用于以下场景：

- 实时数据分析：
  - 实时报表与实时决策： 为企业内外部提供实时更新的报表和仪表盘，支持自动化流程中的实时决策需求。
  - 交互式探索分析： 提供多维数据分析能力，支持对数据进行快速的商业智能分析和即席查询（Ad Hoc），帮助用户在复杂数据中快速发现洞察。
  - 用户行为与画像分析： 分析用户参与、留存、转化等行为，支持人群洞察和人群圈选等画像分析场景。
- 湖仓融合分析：
  - 湖仓查询加速： 通过高效的查询引擎加速湖仓数据的查询。
  - 多源联邦分析： 支持跨多个数据源的联邦查询，简化架构并消除数据孤岛。
  - 实时数据处理： 结合实时数据流和批量数据的处理能力，满足高并发和低延迟的复杂业务需求。
- 半结构化数据分析：
  - 日志与事件分析： 对分布式系统中的日志和事件数据进行实时或批量分析，帮助定位问题和优化性能。

1.1.3 整体架构

Apache Doris 采用 MySQL 协议，高度兼容 MySQL 语法，支持标准 SQL。用户可以通过各类客户端工具访问 Apache Doris，并支持与 BI 工具无缝集成。在部署 Apache Doris 时，可以根据硬件环境与业务需求选择存算一体架构或存算分离架构。

1.1.3.1 存算一体架构

Apache Doris 存算一体架构精简且易于维护。它包含以下两种类型的进程：

- Frontend (FE)： 主要负责接收用户请求、查询解析和规划、元数据管理以及节点管理。
- Backend (BE)： 主要负责数据存储和查询计划的执行。数据会被切分成数据分片（Shard），在 BE 中以多副本方式存储。

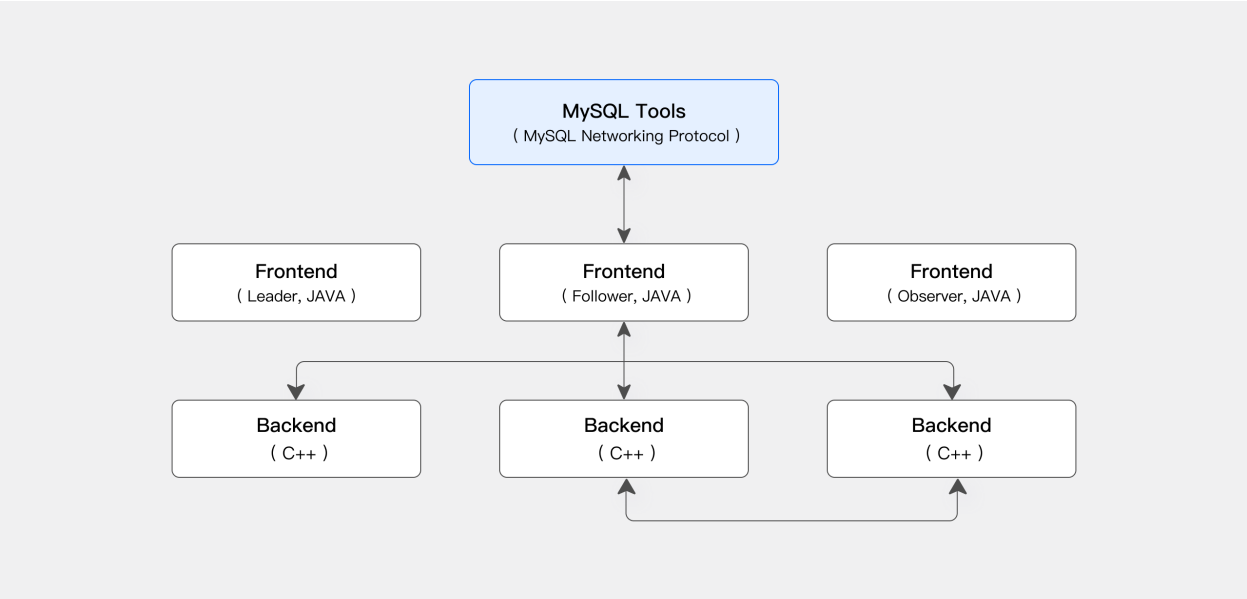


图 2: MPP 数据库整体架构和技术特点

在生产环境中，可以部署多个 FE 节点以实现容灾备份。每个 FE 节点都会维护完整的元数据副本。FE 节点分为以下三种角色：

角色	功能
Master	FE Master 节点负责元数据的读写。当 Master 节点的元数据发生变更后，会通过 BDB JE 协议同步给 Follower 或 Observer 节点。
Follower	Follower 节点负责读取元数据。当 Master 节点发生故障时，可以选取一个 Follower 节点作为新的 Master 节点。
Observer	Observer 节点负责读取元数据，主要目的是增加集群的查询并发能力。Observer 节点不参与集群的选主过程。

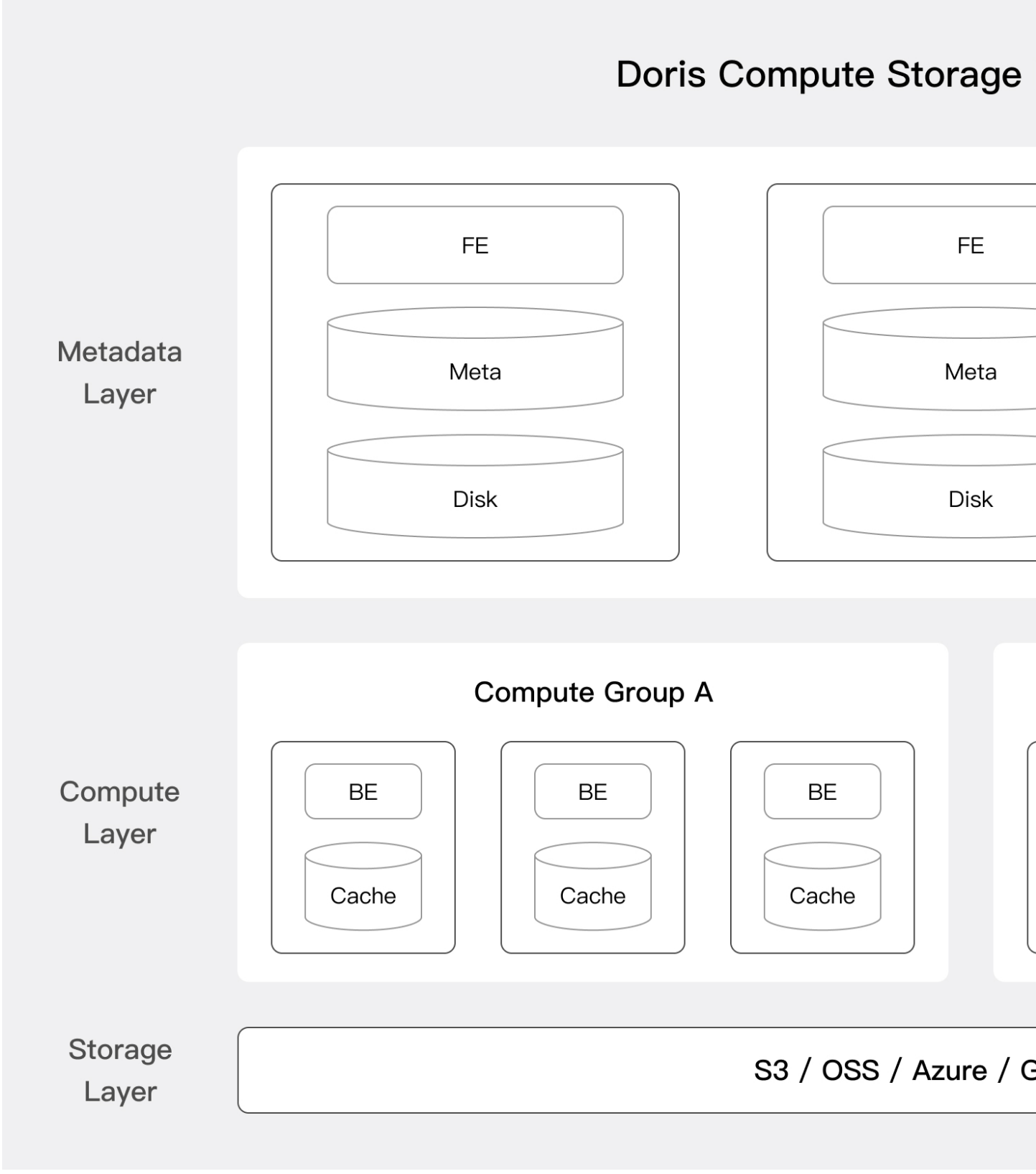


图 3: 存算分离整体架构和技术特点

1.1.4 Apache Doris 的核心特性

- 高可用： Apache Doris 的元数据和数据均采用多副本存储，并通过 Quorum 协议同步数据日志。当大多数副本完成写入后，即认为数据写入成功，从而确保即使少数节点发生故障，集群仍能保持可用性。

Apache Doris 支持同城和异地容灾，能够实现双集群主备模式。当部分节点发生异常时，集群可以自动隔离故障节点，避免影响整体集群的可用性。

- 高兼容： Apache Doris 高度兼容 MySQL 协议，支持标准 SQL 语法，涵盖绝大部分 MySQL 和 Hive 函数。通过这种高兼容性，用户可以无缝迁移和集成现有的应用和工具。Apache Doris 支持 MySQL 生态，用户可以通过 MySQL 客户端工具连接 Doris，使得操作和维护更加便捷。同时，可以使用 MySQL 协议对 BI 报表工具与数据传输工具进行兼容适配，确保数据分析和数据传输过程中的高效性和稳定性。
- 实时数仓： 基于 Apache Doris 可以构建实时数据仓库服务。Apache Doris 提供了秒级数据入库能力，上游在线联机事务库中的增量变更可以秒级捕获到 Doris 中。依靠向量化引擎、MPP 架构及 Pipeline 执行引擎等加速手段，可以提供亚秒级数据查询能力，从而构建高性能、低延迟的实时数仓平台。
- 湖仓一体： Apache Doris 可以基于外部数据源（如数据湖或关系型数据库）构建湖仓一体架构，从而解决数据在数据湖和数据仓库之间无缝集成和自由流动的问题，帮助用户直接利用数据仓库的能力来解决数据湖中的数据分析问题，同时充分利用数据湖的数据管理能力来提升数据的价值。
- 灵活建模： Apache Doris 提供多种建模方式，如宽表模型、预聚合模型、星型/雪花模型等。数据导入时，可以通过 Flink、 Spark 等计算引擎将数据打平成宽表写入到 Doris 中，也可以将数据直接导入到 Doris 中，通过视图、物化视图或实时多表关联等方式进行数据的建模操作。

1.1.5 技术特点

Doris 提供了高效的 SQL 接口，并完全兼容 MySQL 协议。其查询引擎基于 MPP（大规模并行处理）架构，能够高效执行复杂的分析查询，并实现低延迟的实时查询。通过列式存储技术对数据进行编码与压缩，显著优化了查询性能和存储压缩比。

1.1.5.1 使用接口

Apache Doris 采用 MySQL 协议，高度兼容 MySQL 语法，支持标准 SQL。用户可以通过各类客户端工具访问 Apache Doris，并支持与 BI 工具无缝集成。Apache Doris 当前支持多种主流的 BI 产品，包括 Smartbi、DataEase、FineBI、Tableau、Power BI、Apache Superset 等。只要支持 MySQL 协议的 BI 工具，Apache Doris 就可以作为数据源提供查询支持。

1.1.5.2 存储引擎

在存储引擎方面，Apache Doris 采用列式存储，按列进行数据的编码、压缩和读取，能够实现极高的压缩比，同时减少大量非相关数据的扫描，从而更有效地利用 IO 和 CPU 资源。

Apache Doris 也支持多种索引结构，以减少数据的扫描：

- Sorted Compound Key Index：最多可以指定三个列组成复合排序键。通过该索引，能够有效进行数据裁剪，从而更好地支持高并发的报表场景。
- Min/Max Index： 有效过滤数值类型的等值和范围查询。
- BloomFilter Index： 对高基数列的等值过滤裁剪非常有效。
- Inverted Index： 能够对任意字段实现快速检索。

在存储模型方面，Apache Doris 支持多种存储模型，针对不同的场景做了针对性的优化：

- 明细模型（Duplicate Key Model）：适用于事实表的明细数据存储。
- 主键模型（Unique Key Model）：保证 Key 的唯一性，相同 Key 的数据会被覆盖，从而实现行级别数据更新。
- 聚合模型（Aggregate Key Model）：相同 Key 的 Value 列会被合并，通过提前聚合大幅提升性能。

Apache Doris 也支持强一致的单表物化视图和异步刷新的多表物化视图。单表物化视图在系统中自动刷新和维护，无需用户手动选择。多表物化视图可以借助集群内的调度或集群外的调度工具定时刷新，从而降低数据建模的复杂性。

1.1.5.3 查询引擎

Apache Doris 采用大规模并行处理（MPP）架构，支持节点间和节点内并行执行，以及多个大型表的分布式 Shuffle Join，从而更好地应对复杂查询。

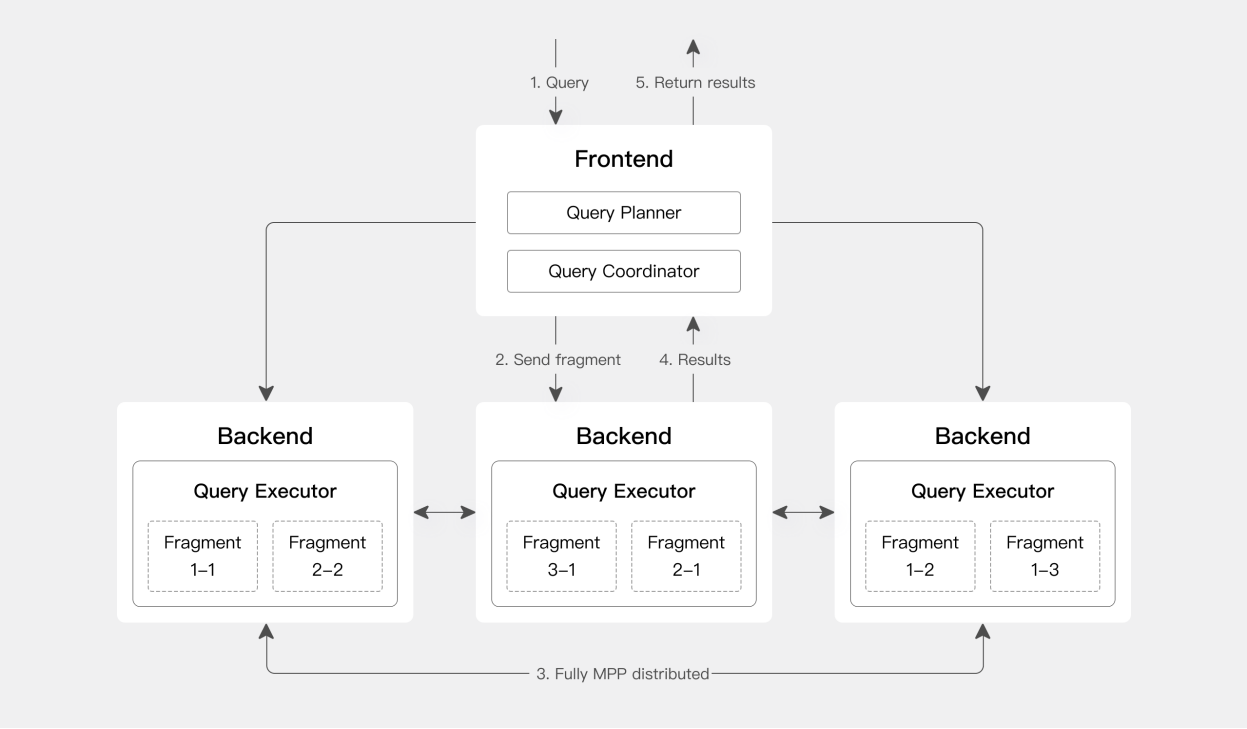


图 4: 查询引擎

Doris 查询引擎是向量化引擎，所有内存结构均按列式布局，可显著减少虚函数调用，提高缓存命中率，并有效利用 SIMD 指令。在宽表聚合场景下，性能是非向量化引擎的 5-10 倍。

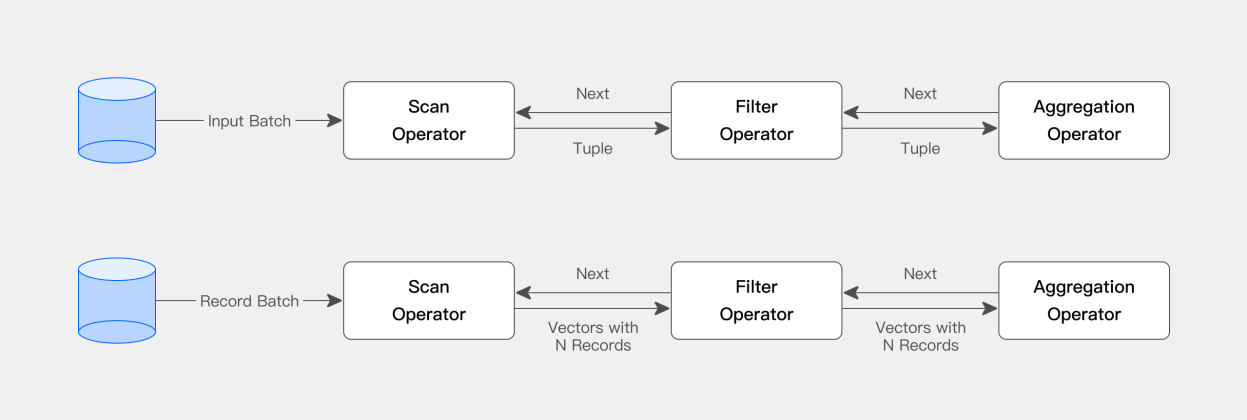


图 5: Doris 查询引擎是向量化

Doris 采用自适应查询执行（Adaptive Query Execution）技术，根据运行时统计信息动态调整执行计划。例如，通过运行时过滤（Runtime Filter）技术，可以在运行时生成过滤器并将其推送到 Probe 端，并自动将过滤器穿透到 Probe 端最底层的 Scan 节点，从而大幅减少 Probe 端的数据量，加速 Join 性能。Doris 的运行时过滤器支持 In/Min/Max/Bloom Filter。

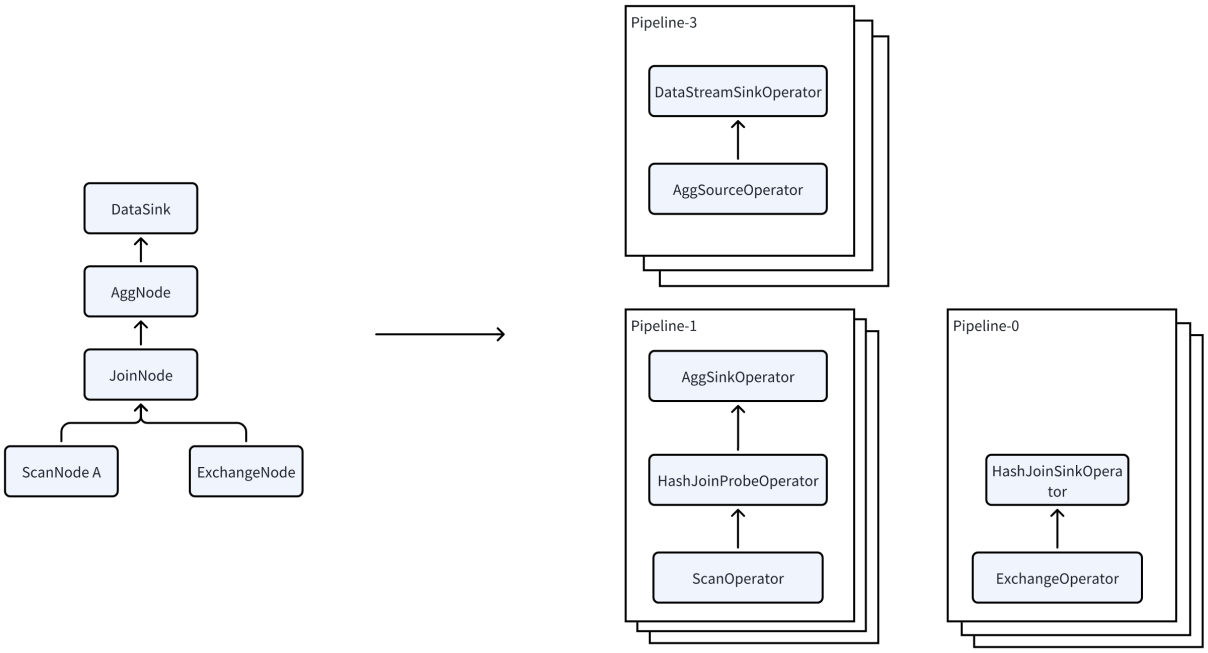


图 6: pip_exec_3

Doris 使用 Pipeline 执行引擎，将查询分解为多个子任务并行执行，充分利用多核 CPU 的能力，同时通过限制查询线程数来解决线程膨胀问题。Pipeline 执行引擎减少数据拷贝和共享，优化排序和聚合操作，从而显著提高查询效率和吞吐量。

在优化器方面，Doris 采用 CBO、RBO 和 HBO 相结合的优化策略。RBO 支持常量折叠、子查询重写和谓词下推等优化，CBO 支持 Join Reorder 等优化，HBO 能够基于历史查询信息推荐最优执行计划。多种优化措施确保 Doris 能够在各类查询中枚举出性能优异的查询计划。

1.2 技术对比

2 使用指南

2.1 安装部署

2.1.1 部署前准备

2.1.1.1 软硬件环境检查

部署 Doris 时，需要对软硬件环境进行以下检查：

- 硬件环境检查
- 服务器建议配置
- 硬盘空间计算
- Java 环境检查

2.1.1.1.1 硬件环境检查

在硬件环境检查中，要对以下硬件条件进行检查：

检查项	建议配置
CPU	支持 AVX2 指令集。
内存	建议至少 CPU 4 倍。
存储	推荐 SSD 硬盘。
文件系统	ext4 或 xfs 文件系统。
网卡	10GbE 网卡。

CPU 检查

当安装 Doris 时，建议选择支持 AVX2 指令集的机器，以利用 AVX2 的向量化能力实现查询向量化加速。

运行以下命令，有输出结果，及表示机器支持 AVX2 指令集。

```
cat /proc/cpuinfo | grep avx2
```

如果机器不支持 AVX2 指令集，可以使用 no AVX2 的 Doris 安装包进行部署。

内存检查

Doris 没有强制的内存限制。一般在生产环境中，可以根据以下建议选择内存大小：

组件	推荐内存配置
FE	建议至少 16GB 以上。
BE	建议内存至少是 CPU 核数的 4 倍（例如，16 核机器至少配置 64G 内存）。在内存是 CPU 核数 8 倍时，会得到更好的性能

存储检查

Doris 支持将数据存储在 SSD、HDD 或对象存储中。

在以下几种场景中建议使用 SSD 作为数据存储：

- 大规模数据量下的高并发点查场景
- 大规模数据量下的高频数据更新场景

文件系统检查

Doris 推荐使用 EXT4 或 XFS 文件系统：

- EXT4 文件系统：具有良好的稳定性、性能和较低的碎片化问题。
- XFS 文件系统：在处理大规模数据和高并发写操作时表现优越，适合高吞吐量应用。

网卡检查

Doris 的计算过程涉及数据分片和并行处理，可能产生网络资源开销。为了最大程度优化 Doris 性能并降低网络资源开销，强烈建议在部署时选用万兆网卡（10 Gigabit Ethernet，即 10GbE）或者更快网络。如果有多块网卡，建议使用链路聚合方式将多块网卡绑定成一块网卡，提高网络带宽、冗余性和复杂均衡的能力。

2.1.1.1.2 服务器建议配置

Doris 支持运行和部署在 x86-64 架构的服务器平台或 ARM64 架构的服务器上。

- 开发及测试环境

开发与测试环境中可以混合部署 FE 与 BE 实例，遵循以下规则：

- 验证测试环境中可以在一台服务器上混合部署一个 FE 与 BE，但不建议部署多个 FE 与 BE 实例；
- 如果需要 3 副本数据，至少需要 3 台服各部署一个 BE 实例。

服务器规格建议如下：

模块	CPU	内存	磁盘	网络	实例数量（最低要求）
Frontend	8 核 +	8 GB+	SSD 或 SATA，10 GB+	1GbE/10GbE 网卡	1
Backend	8 核 +	16 GB+	SSD 或 SATA，50 GB+	1GbE/10GbE 网卡	1

- 生产环境

生产环境中建议 FE 与 BE 实例独立部署，遵循以下规则：

- 如果环境资源紧张，将 FE 与 BE 混部在一台服务器上，建议 FE 与 BE 数据放在不同的硬盘；
- BE 节点可以配置多块硬盘存储，在一个 BE 实例上绑定多块 HDD 或 SSD 盘。

服务器规格建议如下：

模块	CPU	内存	磁盘	网络	实例数量（最低要求）
Frontend	16 核 +	64 GB+	SSD，100GB+	10GbE 网卡	1
Backend	16 核 +	64 GB+	SSD 或 SATA，100GB+	10GbE 网卡	3

2.1.1.1.3 硬盘空间计算

在 Doris 集群中，FE 主要用于元数据存储，包括元数据 edit log 和 image。BE 的磁盘空间主要用于存放数据，需要根据业务需求计算。

组件	磁盘空间说明
FE	建议预留 100GB 以上的存储空间，使用 SSD 硬盘。
BE	Doris 默认 LZ4 压缩方式进行存储，压缩比在 0.3 - 0.5 左右磁盘空间需要按照总数据量 * 3（3 副本）计算需要预留出 40% 空间用作后台 compaction 以及临时数据的存储

2.1.1.1.4 Java 环境检查

Doris 的所有进程都依赖 Java。

- 在 2.1（含）版本之前，请使用 Java 8，推荐版本：`jdk-8u352` 之后版本。
- 从 3.0（含）版本之后，请使用 Java 17，推荐版本：`jdk-17.0.10` 之后版本。

2.1.1.2 集群规划

2.1.1.2.1 架构规划

在部署 Doris 时，可以根据业务选择存算一体或存算分离架构：

- 存算一体：存算一体架构部署简单，性能优异，不依赖与外部的共享存储设备，适合不需要极致弹性扩缩容的业务场景；
- 存算分离：存算分离架构依赖于共享存储，实现了计算资源的弹性伸缩，适合需要动态调整计算资源的业务场景。

2.1.1.2.2 端口规划

Doris 的各个实例通过网络进行通信，其正常运行需要网络环境提供以下端口。管理员可以根据实际环境自行调整 Doris 的端口配置：

实例名称	端口名称	默认端口	通信方向	说明
BE	be_port	9060	FE -> BE	BE 上 Thrift Server 的端口，用于接收来自 FE 的请求
BE	webserver_port	8040	BE <-> BE	BE 上的 HTTP Server 端口
BE	heartbeat_service_port	9050	FE -> BE	BE 上的心跳服务端口（Thrift），用于接收来自 FE 的心跳
BE	brpc_port	8060	FE <-> BE，BE <-> BE	BE 上的 BRPC 端口，用于 BE 之间的通信
FE	http_port	8030	FE <-> FE，Client <-> FE	FE 上的 HTTP Server 端口
FE	rpc_port	9020	BE -> FE，FE <-> FE	FE 上的 Thrift Server 端口，每个 FE 的配置需保持一致
FE	query_port	9030	Client <-> FE	FE 上的 MySQL Server 端口
FE	edit_log_port	9010	FE <-> FE	FE 上的 bdbje 通信端口

2.1.1.2.3 节点数量规划

FE 节点数量

FE 节点主要负责用户请求的接入、查询解析规划、元数据管理及节点管理等工作。

对于生产集群，一般建议部署至少 3 个节点的 FE 以实现高可用环境。FE 节点分为以下两种角色：

- Follower 节点：参与选举操作，当 Master 节点宕机时，会选择一个可用的 Follower 节点成为新的 Master。
- Observer 节点：仅从 Leader 节点同步元数据，不参与选举，可用于横向扩展以提升元数据的读服务能力。

通常情况下，建议部署至少 3 个 Follower 节点。在高并发的场景中，可以通过增加 Observer 节点的数量来提高集群的连接数。

BE 节点数量

BE 节点负责数据的存储与计算。在生产环境中，为了数据的可靠性和容错性，通常会使用 3 副本存储数据，因此建议部署至少 3 个 BE 节点。

BE 节点支持横向扩容，通过增加 BE 节点的数量，可以有效提升查询的性能和并发处理能力。

2.1.1.3 操作系统检查

在部署 Doris 时，需要对以下操作系统项进行检查：

- 确保关闭 swap 分区
- 确保系统关闭透明大页
- 确保系统有足够大的虚拟内存区域
- 确保 CPU 不使用省电模式
- 确保网络连接溢出时自动重置新连接

- 确保 Doris 相关端口畅通或关闭系统防火墙
- 确保系统有足够大的打开文件句柄数
- 确定部署集群机器安装 NTP 服务

2.1.1.3.1 关闭 swap 分区

在部署 Doris 时，建议关闭 swap 分区。swap 分区是内核发现内存紧张时，会按照自己的策略将部分内存数据移动到配置的 swap 分区，由于内核策略不能充分了解应用的行为，会对 Doris 性能造成较大影响。所以建议关闭。

通过以下命令可以临时或者永久关闭。

临时关闭，下次机器启动时，swap 还会被打开。

```
swapoff -a
```

永久关闭，使用 Linux root 账户，注释掉 `/etc/fstab` 中的 swap 分区，重启即可彻底关闭 swap 分区。

```

#### <file system>        <dir>         <type>    <options>             <dump> <pass>
tmpfs                   /tmp          tmpfs     nodev,nosuid          0      0
/dev/sda1               /             ext4      defaults,noatime      0      1
#### /dev/sda2                 none          swap      defaults              0      0
/dev/sda3               /home         ext4      defaults,noatime      0      2
```

2.1.1.3.2 关闭系统透明大页

在高负载低延迟的场景中，建议关闭操作系统透明大页（Transparent Huge Pages, THP），避免其带来的性能波动和内存碎片问题，确保 Doris 能够稳定高效地使用内存。

使用以下命令临时关闭透明大页：

```
echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
echo madvise > /sys/kernel/mm/transparent_hugepage/defrag
```

如果需要永久关闭透明大页，可以使用以下命令，在下一次宿主机重启后生效：

```
cat >> /etc/rc.d/rc.local << EOF
   echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
   echo madvise > /sys/kernel/mm/transparent_hugepage/defrag
EOF
chmod +x /etc/rc.d/rc.local
```

2.1.1.3.3 增加虚拟内存区域

为了保证 Doris 有足够的内存映射区域来处理大量数据，需要修改 VMA（虚拟内存区域）。如果没有足够的内存映射区域，Doris 在启动或运行时可能会遇到 `Too many open files` 或类似的错误。

通过以下命令可以永久修改虚拟内存区域至少为 2000000，并立即生效：

```
cat >> /etc/sysctl.conf << EOF
vm.max_map_count = 2000000
EOF

#### Take effect immediately
sysctl -p
```

2.1.1.3.4 禁用 CPU 省电模式

在部署 Doris 时检修关闭 CPU 的省电模式，以确保 Doris 在高负载时提供稳定的高性能，避免由于 CPU 频率降低导致的性能波动、响应延迟和系统瓶颈，提高 Doris 的可靠性和吞吐量。如果您的 CPU 不支持 Scaling Governor，可以跳过此项配置。

通过以下命令可以关闭 CPU 省电模式：

```
echo 'performance' | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

2.1.1.3.5 网络连接溢出时自动重置新连接

在部署 Doris 时，需要确保在 TCP 连接的发送缓冲区溢出时，连接会被立即中断，以防止 Doris 在高负载或高并发情况下出现缓冲区阻塞，避免连接被长时间挂起，从而提高系统的响应性和稳定性。

通过以下命令可以永久设置系统自动重置新链接，并立即生效：

```
cat >> /etc/sysctl.conf << EOF
net.ipv4.tcp_abort_on_overflow=1
EOF

#### Take effect immediately
sysctl -p
```

2.1.1.3.6 相关端口畅通

如果发现端口不通，可以试着关闭防火墙，确认是否是本机防火墙造成。如果是防火墙造成，可以根据配置的 Doris 各组件端口打开相应的端口通信。

```
sudo systemctl stop firewalld.service
sudo systemctl disable firewalld.service
```

2.1.1.3.7 增加系统的最大文件句柄数

Doris 由于依赖大量文件来管理表数据，所以需要将系统对程序打开文件数的限制调高。

通过以下命令可以调整最大文件句柄数。在调整后，需要重启会话以生效配置：

```
vi /etc/security/limits.conf
* soft nofile 1000000
* hard nofile 1000000
```

2.1.1.3.8 安装并配置 NTP 服务

Doris 的元数据要求时间精度要小于 5000ms，所以所有集群所有机器要进行时钟同步，避免因为时钟问题引发的元数据不一致导致服务出现异常。

通常情况下，可以通过配置 NTP 服务保证各节点时钟同步。

```
sudo systemctl start ntpd.service
sudo systemctl enable ntpd.service
```

2.1.2 手动部署集群

2.1.2.1 手动部署存算一体集群

在完成前置检查及规划后，如环境检查、操作系统检查、集群规划，可以开始部署集存算一体集群。

存算一体集群架构如下，部署存算一体集群分为四步：

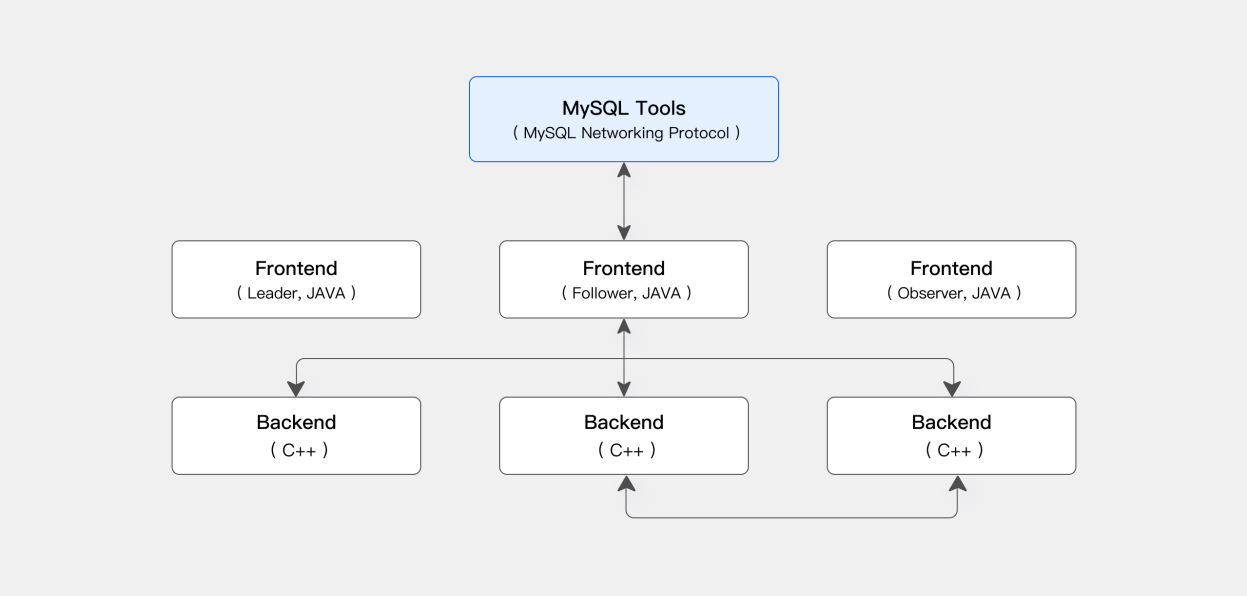


图 7: 存算一体架构

1. 部署 FE Master 节点：部署第一个 FE 节点作为 Master 节点；
2. 部署 FE 集群：部署 FE 集群，添加 Follower 或 Observer FE 节点；
3. 部署 BE 节点：向 FE 集群中注册 BE 节点；
4. 验证集群正确性：部署完成后连接并验证集群正确性。

在开始部署操作前，可以下载对应的 Doris 版本。

2.1.2.1.1 第 1 步：部署 FE Master 节点

1. 创建元数据路径

在部署 FE 时，建议与 BE 节点数据存储在不同的硬盘上。

在解压安装包时，会默认附带 doris-meta 目录，建议为元数据创建独立目录，并将其软连接到默认的 doris-meta 目录。生产环境应使用单独的 SSD 硬盘，不建议将其放在 Doris 安装目录下；开发和测试环境可以使用默认配置。

```
## Use a separate disk for FE metadata
mkdir -p <doris_meta_created>

## Create FE metadata directory symlink
ln -s <doris_meta_created> <doris_meta_original>
```

2. 修改 FE 配置文件

FE 的配置文件在 FE 部署路径下的 conf 目录中，启动 FE 节点前需要修改 conf/fe.conf。

在部署 FE 节点之前，建议调整以下配置：

```
## modify Java Heap
JAVA_OPTS="-Xmx16384m -XX:+UseMembar -XX:SurvivorRatio=8 -XX:MaxTenuringThreshold=7 -XX:+
    ↪ PrintGCDateStamps -XX:+PrintGCDetails -XX:+UseConcMarkSweepGC -XX:+UseParNewGC -XX:+
    ↪ CMSClassUnloadingEnabled -XX:-CMSParallelRemarkEnabled -XX:
    ↪ CMSInitiatingOccupancyFraction=80 -XX:SoftRefLRUPolicyMSPerMB=0 -Xloggc:$DORIS_HOME/
    ↪ log/fe.gc.log.$DATE"

## modify case sensitivity
lower_case_table_names = 1

## modify network CIDR
priority_networks = 10.1.3.0/24

## modify Java Home
JAVA_HOME = <your-java-home-path>
```

参数解释如下，更多详细配置项请参考FE 配置项：

| 参数 | 修改建议 | | ———————————————————— | ———————————————————— | | JAVA_OPTS | 指定参数 -Xmx 调整 Java Heap，生产环境建议 16G 以上。 | | lower_case_table_names | 设置大小写敏感，建议调整为 1，即大小写不敏感。 | | priority_networks | 网络 CIDR，更具网络 IP 地址指定。在 FQDN 环境中可以忽略。 | | JAVA_HOME | 建议 Doris 使用独立于操作系统的 JDK 环境。 |

3. 启动 FE 进程

通过以下命令可以启动 FE 进程

```
bin/start_fe.sh --daemon
```

FE 进程将在后台启动，日志默认保存在 log/ 目录。如果启动失败，可通过查看 log/fe.log 或 log/fe.out 文件获取错误信息。

4. 检查 FE 启动状态

通过 MySQL 客户端连接 Doris 集群，初始化用户为 root，默认密码为空。

```
mysql -uroot -P<fe_query_port> -h<fe_ip_address>
```

链接到 Doris 集群后，可以通过 show frontends 命令查看 FE 的状态，通常要确认以下几项

- Alive 为 true 表示节点存活；
- Join 为 true 表示节点加入到集群中，但不代表当前还在集群内（可能已失联）；
- IsMaster 为 true 表示当前节点为 Master 节点。

2.1.2.1.2 第 2 步：部署 FE 集群（可选）

生产环境建议至少部署 3 个节点。在部署过 FE Master 节点后，需要再部署两个 FE Follower 节点。

1. 创建元数据目录

参考部署 FE Master 节点，创建 doris-meta 目录

2. 修改 FE Follower 节点配置文件

参考部署 FE Master 节点，修改 FE Follower 节点配置文件。通常情况下，可以直接复制 FE Master 节点的配置文件。

3. 在 Doris 集群中注册新的 FE Follower 节点

在启动新的 FE 节点前，需要先在 FE 集群中注册新的 FE 节点。

```
## connect a alive FE node
mysql -uroot -P<fe_query_port> -h<fe_ip_address>

## registe a new FE follower node
ALTER SYSTEM ADD FOLLOWER "<fe_ip_address>:<fe_edit_log_port>"
```

如果要添加 observer 节点，可以使用 ADD OBSERVER 命令

```
## register a new FE observer node
ALTER SYSTEM ADD OBSERVER "<fe_ip_address>:<fe_edit_log_port>"
```

注意 - FE Follower（包括 Master）节点的数量建议为奇数，建议部署 3 个组成高可用模式。

- 当 FE 处于高可用部署时（1 个 Master，2 个 Follower），我们建议通过增加 Observer FE 来扩展 FE 的读服务能力

4. 启动 FE Follower 节点

通过以下命令，可以启动 FE Follower 节点，并自动同步元数据。

```
bin/start_fe.sh --helper <helper_fe_ip>:<fe_edit_log_port> --daemon
```

其中，helper_fe_ip 是 FE 集群中任何存活节点的 IP 地址。`--helper` 参数仅在第一次启动 FE 时需要，之后重启无需指定。

5. 判断 Follower 节点状态

与 FE Master 节点状态判断相同，添加 Follower 节点后，可通过 `show frontends` 命令查看节点状态，IsMaster 应为 false。

2.1.2.1.3 第 3 步：部署 BE 节点

1. 创建数据目录

BE 进程应用于数据的计算与存储。数据目录默认放在 `be/storage` 下。生产环境通常将 BE 数据与 BE 部署文件分别存储在不同的硬盘上。BE 支持数据分布在多盘上以更好的利用多块硬盘的 I/O 能力。

```
## Create a BE data storage directory on each data disk
mkdir -p <be_storage_root_path>
```

2. 修改 BE 配置文件

BE 的配置文件在 BE 部署路径下的 conf 目录中，启动 BE 节点前需要修改 `conf/be.conf`。

```
## modify storage path for BE node

storage_root_path=/home/disk1/doris,medium:HDD;/home/disk2/doris,medium:SSD

## modify network CIDR

priority_networks = 10.1.3.0/24

## modify Java Home in be/conf/be.conf

JAVA_HOME = <your-java-home-path>
```

参数解释如下：

参数	修改建议
priority_networks	网络 CIDR，根据网络 IP 地址指定。在 FQDN 环境中可以忽略。
JAVA_OPTS	指定参数 -Xmx 调整 Java Heap，生产环境建议 2G 以上。
JAVA_HOME	建议 Doris 使用独立于操作系统的 JDK 环境。

3. 在 Doris 中注册 BE 节点

在启动 BE 节点前，需要先在 FE 集群中注册该节点：

```
## connect a alive FE node
mysql -uroot -P<fe_query_port> -h<fe_ip_address>

## registe BE node
ALTER SYSTEM ADD BACKEND "<be_ip_address>:<be_heartbeat_service_port>"
```

4. 启动 BE 进程

通过以下命令可以启动 BE 进程：

```
bin/start_be.sh --daemon
```

BE 进程在后台启动，日志默认保存在 log/ 目录。如果启动失败，请检查 log/be.log 或 log/be.out 文件以获取错误信息。

5. 查看 BE 启动状态

连接 Doris 集群后，可通过 show backends 命令查看 BE 节点的状态。

```
## connect a alive FE node
mysql -uroot -P<fe_query_port> -h<fe_ip_address>

## check BE node status
show backends;
```

通常情况下需要注意以下几项状态：

- Alive 为 true 表示节点存活
- TabletNum 表示该节点上的分片数量，新加入的节点会进行数据均衡，TabletNum 逐渐趋于平均。

2.1.2.1.4 第 4 步：验证集群正确性

1. 登录数据库

使用 MySQL 客户端登录 Doris 集群。

```
## connect a alive fe node
mysql -uroot -P<fe_query_port> -h<fe_ip_address>
```

2. 检查 Doris 安装信息

通过 show frontends 与 show backends 可以查看数据库各实例的信息。

```
-- check fe status
show frontends `\G`

-- check be status
show backends `\G`
```

4. 修改 Doris 集群密码

在创建 Doris 集群时，系统会自动创建一个名为 root 的用户，并默认设置其密码为空。为了提高安全性，建议在集群创建后立即为 root 用户设置一个新密码。

```
-- check the current user
select user();
+-----------------------+
| user()                |
+-----------------------+
| 'root'@'192.168.88.30' |
+-----------------------+

-- modify the password for current user
SET PASSWORD = PASSWORD('doris_new_passwd');
```

5. 创建测试表并插入数据

为了验证集群的正确性，可以在新创建的集群中创建一个测试表，并插入测试数据。

```
-- create a test database
create database testdb;

-- create a test table
CREATE TABLE testdb.table_hash
(
    k1 TINYINT,
    k2 DECIMAL(10, 2) DEFAULT "10.5",
    k3 VARCHAR(10) COMMENT "string column",
    k4 INT NOT NULL DEFAULT "1" COMMENT "int column"
)
COMMENT "my first table"
DISTRIBUTED BY HASH(k1) BUCKETS 32;
```

Doris 兼容 MySQL 协议，可以使用 INSERT 语句插入数据。

```
-- insert data
INSERT INTO testdb.table_hash VALUES
(1, 10.1, 'AAA', 10),
(2, 10.2, 'BBB', 20),
(3, 10.3, 'CCC', 30),
(4, 10.4, 'DDD', 40),
(5, 10.5, 'EEE', 50);

-- check the data
SELECT * from testdb.table_hash;
+------+-------+------+------+
| k1   | k2    | k3   | k4   |
+------+-------+------+------+
|    3 | 10.30 | CCC  |   30 |
|    4 | 10.40 | DDD  |   40 |
|    5 | 10.50 | EEE  |   50 |
|    1 | 10.10 | AAA  |   10 |
|    2 | 10.20 | BBB  |   20 |
+------+-------+------+------+
```

2.1.2.2 手动部署存算分离集群

在完成前置检查及规划后，如环境检查、集群规划、操作系统检查后，可以开始部署集群。部署集群分为八步：

1. 准备 FoundationDB 集群：可以使用已有的 FoundationDB 集群，或新建 FoundationDB 集群；
2. 部署 S3 或 HDFS 服务：可以使用已有的共享存储，或新建共享存储；
3. 部署 Meta Service：为 Doris 集群部署 Meta Service 服务；
4. 部署数据回收进程：为 Doris 集群独立部署数据回收进程，可选操作；
5. 启动 FE Master 节点：启动第一个 FE 节点作为 Master FE 节点；
6. 创建 FE Master 集群：添加 FE Follower/Observer 节点组成 FE 集群；
7. 添加 BE 节点：向集群中添加并注册 BE 节点；
8. 添加 Storage Vault：使用共享存储创建一个或多个 Storage Vault。

在开始部署操作前，可以下载相应的 Doris 版本。

2.1.2.2.1 第 1 步：准备 FoundationDB

本节提供了脚本 `fdb_vars.sh` 和 `fdb_ctl.sh` 配置、部署和启动 FDB（FoundationDB）服务的分步指南。您可以下载 doris tools 并从 `fdb` 目录获取 `fdb_vars.sh` 和 `fdb_ctl.sh`。

> Doris 默认依赖的 FDB 版本为 7.1.x 系列。若已提前安装 FDB，请确认其版本属于 7.1.x 系列，否则 Meta Service 将启动失败。

1. 机器要求

通常，至少需要三台配备 SSD 的机器来组成具有双副本、单机故障容忍的 FoundationDB 集群。如果是测试/开发环境，单台机器也能搭建 FoundationDB。

2. 配置 `fdb_vars.sh` 脚本

在配置 `fdb_vars.sh` 脚本时，必须指定以下配置：

参数	描述	类型	示例	注意事项
DATA_DIRS	指定 FoundationDB 存储的数据目录	以逗号分隔的绝对路径列表	/mnt/foundationdb/data1,/mnt/foundationdb/data2,/mnt/foundationdb/data3	运行脚本前确保目录已创建，确保目录已创建，生产环境建议使用 SSD 和独立目录
FDB_CLUSTER_IP	定义集群 IP	字符串（以逗号分隔的 IP 地址）	172.200.0.2,172.200.0.3,172.200.0.4	生产集群至少应有 3 个 IP 地址，第一个 IP 地址将用作协调器 - 为高可用性，将机器放置在不同机架上
FDB_HOME	定义 FoundationDB 主目录	绝对路径	/fdbhome	默认路径为 /fdbhome，确保此路径是绝对路径
FDB_CLUSTER_ID	定义集群 ID	字符串	SAQESzbh	每个集群的 ID 必须唯一，可使用 mktemp -u XXXXXXXX 生成
FDB_CLUSTER_DESC	定义 FDB 集群的描述	字符串	dorisfdb	建议更改为对部署有意义的内容

可以选择指定以下自定义配置：

参数	描述	类型	示例	注意事项
MEMORY_LIMIT_GB	定义 FDB 进程的内存限制，单位为 GB	整数	MEMORY_LIMIT_GB=16	根据可用内存资源和 FDB 进程的要求调整此值
CPU_CORES_LIMIT	定义 FDB 进程的 CPU 核心限制	整数	CPU_CORES_LIMIT=8	根据可用的 CPU 核心数量和 FDB 进程的要求设置此值

参数	描述	类型	示例	注意事项

3. 部署 FDB 集群

使用 fdb_vars.sh 配置环境后，您可以在每个节点上使用 fdb_ctl.sh 脚本部署 FDB 集群。

```
./fdb_ctl.sh deploy
```

4. 启动 FDB 服务

FDB 集群部署完成后，您可以使用 fdb_ctl.sh 脚本启动 FDB 服务。

```
./fdb_ctl.sh start
```

以上命令启动 FDB 服务，使集群工作并获取 FDB 集群连接字符串，后续可以用于配置 MetaService。

> 注意 fdb_ctl.sh 脚本中的 clean 命令会清除所有 fdb 元数据信息，可能导致数据丢失，严禁在生产环境中使用！

2.1.2.2.2 第 2 步：安装 S3 或 HDFS 服务（可选）

Doris 的存算分离模式依赖于 S3 或 HDFS 服务来存储数据，如果您已经有相关服务，直接使用即可。如果没有，本文档提供 MinIO 的简单部署教程：

1. 在 MinIO 的下载页面选择合适的版本以及操作系统，下载对应的 Server 以及 Client 的二进制包或安装包。
2. 启动 MinIO Server

```
export MINIO_REGION_NAME=us-east-1
export MINIO_ROOT_USER=minio # 在较老版本中，该配置为 MINIO_ACCESS_KEY=minio
export MINIO_ROOT_PASSWORD=minioadmin # 在较老版本中，该配置为 MINIO_SECRET_KEY=minioadmin
nohup ./minio server /mnt/data 2>&1 &
```

3. 配置 MinIO Client

```
# 如果你使用的是安装包安装的客户端，那么客户端名为 mcli，直接下载客户端二进制包，则其名为 mc
./mc config host add myminio http://127.0.0.1:9000 minio minioadmin
```

4. 创建一个桶

```
./mc mb myminio/doris
```

5. 验证是否正常工作

```
# 上传一个文件
./mc mv test_file myminio/doris
# 查看这个文件
./mc ls myminio/doris
```

2.1.2.2.3 第 3 步：Meta Service 部署

1. 配置

在 `./conf/doris_cloud.conf` 文件中，主要需要修改以下两个参数：

- `brpc_listen_port`：Meta Service 的监听端口，默认为 5000。
- `fdb_cluster`：FoundationDB 集群的连接信息，部署 FoundationDB 时可以获取。（如果使用 Doris 提供的 fdb_ctl.sh 部署的话，可在 `$FDB_HOME/conf/fdb.cluster` 文件里获取该值）。

示例配置：

```
brpc_listen_port = 5000
fdb_cluster = xxx:yyy@127.0.0.1:4500
```

注意：`fdb_cluster` 的值应与 FoundationDB 部署机器上的 `/etc/foundationdb/fdb.cluster` 文件内容一致（如果使用 Doris 提供的 fdb_ctl.sh 部署的话，可在 `$FDB_HOME/conf/fdb.cluster` 文件里获取该值）。

示例，文件的最后一行就是要填到 doris_cloud.conf 里 fdb_cluster 字段的值：

```
cat /etc/foundationdb/fdb.cluster

DO NOT EDIT!
This file is auto-generated, it is not to be edited by hand.
cloud_ssb:A83c8Y1S3ZbqHLL4P4HHNTTw0A83CuHj@127.0.0.1:4500
```

2. 启动与停止

在启动前，需要确保已正确设置 `JAVA_HOME` 环境变量，指向 OpenJDK 17，进入 `ms` 目录。

启动命令如下：

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --daemon
```

启动脚本返回值为 0 表示启动成功，否则启动失败。启动成功同时标准输出的最后一行文本信息为“doris_cloud start successfully”。

停止命令如下：

```
bin/stop.sh
```

生产环境中请确保至少有 3 个 Meta Service 节点。

2.1.2.2.4 第 4 步：数据回收功能独立部署（可选）

::info 信息

Meta Service 本身具备了元数据管理和回收功能，这两个功能可以独立部署，如果需要独立部署数据回收功能，可参考以下步骤。

1. 创建新的工作目录（如 `recycler`），并复制 `ms` 目录内容到新目录：

```
cp -r ms recycler
```

2. 在新目录的配置文件中修改 BRPC 监听端口 `brpc_listen_port` 和 `fdb_cluster` 的值。

启动数据回收功能

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --recycler --daemon
```

启动仅元数据操作功能

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --meta-service --daemon
```

2.2 第 5 步：启动 FE Master 节点

1. 配置 fe.conf 文件

在 `fe.conf` 文件中，需要配置以下关键参数：

- `deploy_mode`
  - 描述：指定 doris 启动模式
  - 格式：cloud 表示存算分离模式，其它存算一体模式
  - 示例：`cloud`
- `cluster_id`
  - 描述：存算分离架构下集群的唯一标识符，不同的集群必须设置不同的 cluster_id。
  - 格式：int 类型

  - 示例：可以使用如下 shell 脚本 `echo $(($((RANDOM << 15))| $RANDOM))` 生成一个随机 id 使用。
  - 注意：不同的集群必须设置不同的 cluster_id
- `meta_service_endpoint`
  - 描述：Meta Service 的地址和端口
  - 格式：`IP地址:端口号`
  - 示例：`127.0.0.1:5000`，可以用逗号分割配置多个 meta service。

2. 启动 FE Master 节点

启动命令：

```
bin/start_fe.sh --daemon
```

第一个 FE 进程初始化集群并以 FOLLOWER 角色工作。使用 mysql 客户端连接 FE 使用 show frontends 确认刚才启动的 FE 是 master。

2.3 第 6 步：注册 FE Follower/Observer 节点

其他节点同样根据上述步骤修改配置文件并启动，使用 mysql 客户端连接 Master 角色的 FE，并用以下 SQL 命令添加额外的 FE 节点：

```
ALTER SYSTEM ADD FOLLOWER "host:port";
```

将 `host:port` 替换为 FE 节点的实际地址和编辑日志端口。更多信息请参见ADD FOLLOWER 和ADD OBSERVER。

生产环境中，请确保在 FOLLOWER 角色中的前端（FE）节点总数，包括第一个 FE，保持为奇数。一般来说，三个 FOLLOWER 就足够了。观察者角色的前端节点可以是任意数量。

2.4 第 7 步：添加 BE 节点

要向集群添加 Backend 节点，请对每个 Backend 执行以下步骤：

1. 配置 be.conf

在 `be.conf` 文件中，需要配置以下关键参数：- deploy_mode - 描述：指定 doris 启动模式 - 格式：cloud 表示存算分离模式，其它存算一体模式 - 示例：cloud - file_cache_path - 描述：用于文件缓存的磁盘路径和其他参数，以数组形式表示，每个磁盘一项。path 指定磁盘路径，total_size 限制缓存的大小；-1 或 0 将使用整个磁盘空间。- 格式：[{“path”：“/path/to/file_cache”，“total_size”:21474836480}，{“path”：“/path/to/file_cache2”，“total_size”:21474836480}] - 示例：[{“path”：“/path/to/file_cache”，“total_size”:21474836480}，{“path”：“/path/to/file_cache2”，“total_size”:21474836480}] - 默认：[{“path”：“${DORIS_HOME}/file_cache”}]

3. 启动 BE 进程

使用以下命令启动 Backend：

```
bin/start_be.sh --daemon
```

4. 将 BE 添加到集群：

使用 MySQL 客户端连接到任意 FE 节点：

```
ALTER SYSTEM ADD BACKEND "<ip>:<heartbeat_service_port>" [PROTERTIES propertires
    ↪ ];
```

将 <ip> 替换为新 Backend 的 IP 地址，将 <heartbeat_service_port> 替换为其配置的心跳服务端口（默认为 9050）。

可以通过 PROPERTIES 设置 BE 所在的计算组。

更详细的用法请参考ADD BACKEND 和REMOVE BACKEND。

5. 验证 BE 状态

检查 Backend 日志文件（be.log）以确保它已成功启动并加入集群。

您还可以使用以下 SQL 命令检查 Backend 状态：

```
SHOW BACKENDS;
```

这将显示集群中所有 Backend 及其当前状态。

2.5 第 8 步：添加 Storage Vault

Storage Vault 是 Doris 存算分离架构中的重要组件。它们代表了存储数据的共享存储层。您可以使用 HDFS 或兼容 S3 的对象存储创建一个或多个 Storage Vault。可以将一个 Storage Vault 设置为默认 Storage Vault，系统表和未指定 Storage Vault 的表都将存储在这个默认 Storage Vault 中。默认 Storage Vault 不能被删除。以下是为您的 Doris 集群创建 Storage Vault 的方法：

1. 创建 HDFS Storage Vault

要使用 SQL 创建 Storage Vault，请使用 MySQL 客户端连接到您的 Doris 集群

```
CREATE STORAGE VAULT IF_NOT_EXISTS hdfs_vault
    PROPERTIES (
    "type"="hdfs",
    "fs.defaultFS"="hdfs://127.0.0.1:8020"
);
```

2. 创建 S3 Storage Vault

要使用兼容 S3 的对象存储创建 Storage Vault，请按照以下步骤操作：

- 使用 MySQL 客户端连接到您的 Doris 集群。
- 执行以下 SQL 命令来创建 S3 Storage Vault：

```
CREATE STORAGE VAULT IF_NOT_EXISTS s3_vault
    PROPERTIES (
    "type"="S3",
    "s3.endpoint"="s3.us-east-1.amazonaws.com",
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk",
    "s3.region" = "us-east-1",
    "s3.root.path" = "ssb_sf1_p2_s3",
    "s3.bucket" = "doris-build-1308700295",
    "provider" = "S3"
);
```

要在其他对象存储上创建 Storage Vault，请参考创建 Storage Vault。

3. 设置默认 Storage Vault

使用如下 SQL 语句设置一个默认 Storage Vault。

```
SET <storage_vault_name> AS DEFAULT STORAGE VAULT
```

2.6 注意事项

- 仅元数据操作功能的 Meta Service 进程应作为 FE 和 BE 的 `meta_service_endpoint` 配置目标。
- 数据回收功能进程不应作为 `meta_service_endpoint` 配置目标。

2.6.1 在 Kubernetes 部署

2.6.1.1 部署存算一体集群

2.6.1.1.1 部署 Doris Operator

部署 Doris Operator 的过程分为安装 CRD、部署 Operator 服务以及检查部署状态三个步骤。

第 1 步：安装 Doris Operator CRD

通过以下命令添加 Doris Operator 的自定义资源（CRD）：

```
kubectl create -f https://raw.githubusercontent.com/apache/doris-operator/master/config/crd/bases
    ↪ /crds.yaml
```

第 2 步：部署 Doris Operator

通过以下命令安装 Doris Operator：

```
kubectl apply -f https://raw.githubusercontent.com/apache/doris-operator/master/config/operator/
    ↪ operator.yaml
```

期望输出结果：

```
namespace/doris created
role.rbac.authorization.k8s.io/leader-election-role created
rolebinding.rbac.authorization.k8s.io/leader-election-rolebinding created
clusterrole.rbac.authorization.k8s.io/doris-operator created
clusterrolebinding.rbac.authorization.k8s.io/doris-operator-rolebinding created
serviceaccount/doris-operator created
deployment.apps/doris-operator created
```

第 3 步：检查 Doris Operator 状态

通过以下命令检查 Doris Operator 的部署状态：

```
kubectl get pods -n doris
```

期望输出结果：

```
NAME                               READY   STATUS    RESTARTS   AGE
doris-operator-7f578c86cb-nz6jn    1/1     Running   0          19m
```

2.6.1.1.2 配置 Doris 集群

集群规划

默认部署的 DorisCluster 资源中，FE 和 BE 的镜像可能并非最新版本，且默认副本数均为 3。默认情况下，FE 使用的计算资源配置为 6c 12Gi，BE 使用的资源是 8c 16Gi。以下介绍如何根据需求调整这些默认配置。

Image 设置

Doris Operator 与 Doris 版本相互解耦，Doris Operator 支持 2.0 以上的 Doris 版本部署。

FE Image 设置

如需指定 FE 的镜像，可按以下方式进行配置：

```
spec:
  feSpec:
    image: ${image}
```

将 ${image} 替换想要部署的 image 名称后，将配置更新到需要部署的 DorisCluster 资源中。Doris 官方提供的 FE Image 可供使用。

BE Image 设置

如需指定 BE 的镜像，可按以下方式进行配置：

```
spec:
  beSpec:
    image: ${image}
```

将 ${image} 替换想要部署的 image 名称后，将配置更新到需要部署的 DorisCluster 资源中。Doris 官方提供的 BE Image 可供使用。

副本数设定

FE 副本数修改

将默认的 FE 副本数 3 改为 5，可按以下方式进行配置：

```
spec:
  feSpec:
    replicas: 5
```

将配置更新到需要部署的 DorisCluster 资源中。

BE 副本数修改

将默认的 BE 副本数 3 改为 5，可按以下方式进行配置：

```
spec:
  beSpec:
    replicas: 5
```

将配置更新到需要部署的 DorisCluster 资源中。

计算资源设定

FE 计算资源设定

默认部署的 FE 计算资源为 6c 12Gi，若需修改为 8c 16Gi，可按以下方式进行配置：

```
spec:
  feSpec:
    requests:
      cpu: 8
      memory: 16Gi
    limits:
      cpu: 8
      memory: 16Gi
```

将配置更新到需要部署的 DorisCluster 资源中。

BE 计算资源设定

默认部署的 BE 计算资源为 8c 16Gi, 如需修改为 16c 32Gi，可按以下方式进行配置：

```
spec:
  beSpec:
```

```
    requests:
      cpu: 16
      memory: 32Gi
    limits:
      cpu: 16
      memory: 32Gi
```

将配置更新到需要部署的 DorisCluster 资源中。

> 提示 FE 和 BE 所需要的最小启动资源为 4c 8Gi，如果需要进行正常能力测试，建议配置为 8c 8Gi。

定制化启动配置

在 Kubernetes 中，Doris 使用 ConfigMap 将配置文件和服务分离。默认情况下，服务使用镜像里默认配置作为启动参数。请根据FE 配置文档和BE 配置文档介绍，预先将定制好的启动参数配置到特定的 ConfigMap 中。配置完成后，将其部署到目标DorisCluster 资源所在的命名空间中。

FE 定制化启动配置

第 1 步：配置并部署 ConfigMap

以下示例定义了名为 fe-conf 的 ConfigMap，该配置可供 Doris FE 使用：

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fe-conf
  labels:
    app.kubernetes.io/component: fe
data:
  fe.conf: |
    CUR_DATE=`date +%Y%m%d-%H%M%S`
    # Log dir
    LOG_DIR = ${DORIS_HOME}/log
    # For jdk 8
    JAVA_OPTS="-Dfile.encoding=UTF-8 -Djavax.security.auth.useSubjectCredsOnly=false -Xss4m -
        ↪ Xmx8192m -XX:+UnlockExperimentalVMOptions -XX:+UseG1GC -XX:MaxGCPauseMillis=200 -XX:+
        ↪ PrintGCDateStamps -XX:+PrintGCDetails -XX:+PrintClassHistogramAfterFullGC -Xloggc:
        ↪ $LOG_DIR/log/fe.gc.log.$CUR_DATE -XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=10
        ↪ -XX:GCLogFileSize=50M -Dlog4j2.formatMsgNoLookups=true"

    # For jdk 17, this JAVA_OPTS will be used as default JVM options
    JAVA_OPTS_FOR_JDK_17="-Dfile.encoding=UTF-8 -Djavax.security.auth.useSubjectCredsOnly=false -
        ↪ Xmx8192m -Xms8192m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=$LOG_DIR -Xlog:gc
        ↪ *,classhisto*=trace:$LOG_DIR/fe.gc.log.$CUR_DATE:time,uptime:filecount=10,filesize=50
```

```
    ↪ M --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens java.base/jdk.internal.ref=
    ↪ ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED"
# Set your own JAVA_HOME
# JAVA_HOME=/path/to/jdk/
##
## the lowercase properties are read by main program.
##
# store metadata, must be created before start FE.
# Default value is ${DORIS_HOME}/doris-meta
# meta_dir = ${DORIS_HOME}/doris-meta
# Default dirs to put jdbc drivers,default value is ${DORIS_HOME}/jdbc_drivers
# jdbc_drivers_dir = ${DORIS_HOME}/jdbc_drivers

http_port = 8030
rpc_port = 9020
query_port = 9030
edit_log_port = 9010
arrow_flight_sql_port = -1

# Choose one if there are more than one ip except loopback address.
# Note that there should at most one ip match this list.
# If no ip match this rule, will choose one randomly.
# use CIDR format, e.g. 10.10.10.0/24 or IP format, e.g. 10.10.10.1
# Default value is empty.
# priority_networks = 10.10.10.0/24;192.168.0.0/16

# Advanced configurations
# log_roll_size_mb = 1024
# INFO, WARN, ERROR, FATAL
syg_level = INFO
# NORMAL, BRIEF, ASYNC
syg_mode = ASYNC
# audit_log_dir = $LOG_DIR
# audit_log_modules = slow_query, query
# audit_log_roll_num = 10
# meta_delay_toleration_second = 10
# qe_max_connection = 1024
# qe_query_timeout_second = 300
# qe_slow_log_ms = 5000
enable_fqdn_mode = true
```

使用 ConfigMap 挂载 FE 启动配置信息时，配置信息对应的 key 必须为 `fe.conf`。完成配置文件后，通过如下命令部署到 DorisCluster 资源将要部署的命名空间。

```
kubectl -n ${namespace} apply -f ${feConfigMapFile}.yaml
```

其中，${namespace} 为目标 DorisCluster 资源将要部署的命名空间，${feConfigMapFile} 为包含上述配置的文件名。

第 2 步：配置 DorisCluster 资源

以 fe-conf 对应的 ConfigMap 为例，需要在部署的 DorisCluster 资源中添加如下信息：

```
spec:
  feSpec:
    configMapInfo:
      configMapName: fe-conf
      resolveKey: fe.conf
```

> 提示 Kubernetes 部署中，建议使用 FQDN 模式，启动配置中应添加 enable_fqdn_mode=true。如果想用 IP 模式，且 Kubernetes 集群能够保证 pod 重启后 IP 不发生变化，请参照 issue #138 进行配置 IP 模式启动。

BE 定制化启动配置

第 1 步：配置并部署 ConfigMap

以下定义了名为 be-conf ConfigMap，该配置可供 Doris BE 使用：

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: be-conf
  labels:
    app.kubernetes.io/component: be
data:
  be.conf: |
    CUR_DATE=`date +%Y%m%d-%H%M%S`
    # Log dir
    LOG_DIR="${DORIS_HOME}/log/"
    # For jdk 8
    JAVA_OPTS="-Dfile.encoding=UTF-8 -Xmx2048m -DlogPath=$LOG_DIR/jni.log -Xloggc:$LOG_DIR/be.gc.
        ↪ log.$CUR_DATE -XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=10 -XX:GCLogFileSize
        ↪ =50M -Djavax.security.auth.useSubjectCredsOnly=false -Dsun.security.krb5.debug=true -
        ↪ Dsun.java.command=DorisBE -XX:-CriticalJNINatives"
    # For jdk 17, this JAVA_OPTS will be used as default JVM options
    JAVA_OPTS_FOR_JDK_17="-Dfile.encoding=UTF-8 -Djol.skipHotspotSAAttach=true -Xmx2048m -
        ↪ DlogPath=$LOG_DIR/jni.log -Xlog:gc*:$LOG_DIR/be.gc.log.$CUR_DATE:time,uptime:
        ↪ filecount=10,filesize=50M -Djavax.security.auth.useSubjectCredsOnly=false -Dsun.
        ↪ security.krb5.debug=true -Dsun.java.command=DorisBE -XX:-CriticalJNINatives -XX:+
        ↪ IgnoreUnrecognizedVMOptions --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=
```

```
    ↪ java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-
    ↪ UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-
    ↪ UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/java.util=
    ↪ ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED --add-opens=java.
    ↪ base/java.util.concurrent.atomic=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-
    ↪ UNNAMED --add-opens=java.base/sun.nio.cs=ALL-UNNAMED --add-opens=java.base/sun.
    ↪ security.action=ALL-UNNAMED --add-opens=java.base/sun.util.calendar=ALL-UNNAMED --add
    ↪ -opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED --add-opens=java.management/
    ↪ sun.management=ALL-UNNAMED -Darrow.enable_null_check_for_get=false"
# Set your own JAVA_HOME
# JAVA_HOME=/path/to/jdk/
# https://github.com/apache/doris/blob/master/docs/zh-CN/community/developer-guide/debug-tool
    ↪ .md#jemalloc-heap-profile
# https://jemalloc.net/jemalloc.3.html
JEMALLOC_CONF="percpu_arena:percpu,background_thread:true,metadata_thp:auto,muzzy_decay_ms
    ↪ :5000,dirty_decay_ms:5000,oversize_threshold:0,prof:true,prof_active:false,lg_prof_
    ↪ interval:-1"
JEMALLOC_PROF_PRFIX="jemalloc_heap_profile_"
# ports for admin, web, heartbeat service
be_port = 9060
webserver_port = 8040
heartbeat_service_port = 9050
brpc_port = 8060
arrow_flight_sql_port = -1
# HTTPS configures
enable_https = false
# path of certificate in PEM format.
ssl_certificate_path = "$DORIS_HOME/conf/cert.pem"
# path of private key in PEM format.
ssl_private_key_path = "$DORIS_HOME/conf/key.pem"

# Choose one if there are more than one ip except loopback address.
# Note that there should at most one ip match this list.
# If no ip match this rule, will choose one randomly.
# use CIDR format, e.g. 10.10.10.0/24 or IP format, e.g. 10.10.10.1
# Default value is empty.
# priority_networks = 10.10.10.0/24;192.168.0.0/16

# data root path, separate by ';'
# You can specify the storage type for each root path, HDD (cold data) or SSD (hot data)
# eg:
# storage_root_path = /home/disk1/doris;/home/disk2/doris;/home/disk2/doris
# storage_root_path = /home/disk1/doris,medium:SSD;/home/disk2/doris,medium:SSD;/home/disk2/
    ↪ doris,medium:HDD
# /home/disk2/doris,medium:HDD(default)
```

```
    #
    # you also can specify the properties by setting '<property>:<value>', separate by ','
    # property 'medium' has a higher priority than the extension of path
    #
    # Default value is ${DORIS_HOME}/storage, you should create it by hand.
    # storage_root_path = ${DORIS_HOME}/storage

    # Default dirs to put jdbc drivers,default value is ${DORIS_HOME}/jdbc_drivers
    # jdbc_drivers_dir = ${DORIS_HOME}/jdbc_drivers

    # Advanced configurations
    # INFO, WARNING, ERROR, FATAL
    sys_log_level = INFO
    # sys_log_roll_mode = SIZE-MB-1024
    # sys_log_roll_num = 10
    # sys_log_verbose_modules = *
    # log_buffer_level = -1

    # aws sdk log level
    #    Off = 0,
    #    Fatal = 1,
    #    Error = 2,
    #    Warn = 3,
    #    Info = 4,
    #    Debug = 5,
    #    Trace = 6
    # Default to turn off aws sdk log, because aws sdk errors that need to be cared will be
        ↪ output through Doris logs
    aws_log_level=0
    ## If you are not running in aws cloud, you can disable EC2 metadata
    AWS_EC2_METADATA_DISABLED=true
```

使用 ConfigMap 挂载 BE 启动配置信息时，配置信息对应的 key 必须为 be.conf 。完成配置文件后，将其部署到目标 DorisCluster 资源需要部署的命名空间。

```
kubectl -n ${namespace} apply -f ${beConfigMapFile}.yaml
```

其中，${namespace} 为 DorisCluster 资源需要部署到的 namespace，${beConfigMapFile} 为包含上述配置的文件名。

第 2 步：配置 DorisCluster 资源

以 be-conf 对应的 ConfigMap 为例，需要在部署的 DorisCluster 资源中添加如下信息：

```
spec:
  beSpec:
    configMapInfo:
```

```
      configMapName: be-conf
      resolveKey: be.conf
```

> 提示如果需要将文件挂载到和启动配置同一目录下，需要将配置信息配置到启动配置所在的 ConfigMap 中。ConfigMap 中的 key 为文件名称，value 为配置信息。

多 ConfigMap 挂载

Doris Operator 除了支持通过 ConfigMap 挂载配置文件，还提供了将多个 ConfigMap 挂载到容器不同目录的功能。

FE 挂载多 ConfigMap

以下示例展示将 `test-fe1` ，`test-fe2` 的 ConfigMap 分别挂载到 FE 容器 `/etc/fe/config1/` 和 `/etc/fe/config2` 目录下：

```
spec:
  feSpec:
    configMaps:
    - configMapName: test-fe1
      mountPath: /etc/fe/config1
    - configMapName: test-fe2
      mountPath: /etc/fe/config2
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示希望使用的存储大小，${storageSize} 的格式遵循 K8s 的 quantity 表达方式, 比如：100Gi。请在使用时按需替换。

BE 挂载多 ConfigMap

以下示例展示将 test-be1，test-be2 的 ConfigMap 分别挂载到 BE 容器 `/etc/be/config1/` 和 `/etc/be/config2` 目录下：

```
spec:
  beSpec:
    configMaps:
    - configMapName: test-be1
      mountPath: /etc/be/config1
    - configMapName: test-be2
      mountPath: /etc/be/config2
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示希望使用的存储大小，${storageSize} 的格式遵循 K8s 的 quantity 表达方式, 比如：100Gi。请在使用时按需替换。

配置持久化存储

在 Doris 集群中，FE、BE 组件需要将数据持久化。Kubernetes 提供了 Persistent Volume 机制，将数据持久化到物理存储中。在 Kubernetes 环境中，Doris Operator 使用 StorageClass 自动创建 PersistentVolumeClaim 关联合适的 PersistentVolume。

FE 持久化存储配置

在 Kubernetes 部署 Doris 集群时，建议默认持久化 /opt/apache-doris/fe/doris-meta 挂载点，该路径为 FE 元数据的默认存储路径。Doris 默认将所有的日志信息输出到标准输出（console），如集群缺乏日志收集能力，建议持久化 /opt/apache-doris/fe/log 挂载点以实现日志持久化。

FE 元数据持久化

使用默认配置文件时，需要在部署的 DorisCluster 资源中添加如下内容：

```
spec:
  feSpec:
    persistentVolumes:
    - mountPath: /opt/apache-doris/fe/doris-meta
      name: meta
      persistentVolumeClaimSpec:
        # when use specific storageclass, the storageClassName should reConfig, example as
            ↪ annotation.
        storageClassName: ${your_storageclass}
        accessModes:
          - ReadWriteOnce
        resources:
          # notice: if the storage size less 5G, fe will not start normal.
          requests:
            storage: ${storageSize}
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示指定的存储大小，格式遵循 Kubernetes 的 quantity 表达方式, 比如：100Gi。

FE 日志持久化

使用默认配置文件时，将如下配置添加到需要部署的 DorisCluster 资源中：

```
spec:
  feSpec:
    persistentVolumes:
    - mountPath: /opt/apache-doris/fe/log
      name: log
      persistentVolumeClaimSpec:
        # when use specific storageclass, the storageClassName should reConfig, example as
            ↪ annotation.
        storageClassName: ${your_storageclass}
        accessModes:
          - ReadWriteOnce
        resources:
          # notice: if the storage size less 5G, fe will not start normal.
```

```
      requests:
        storage: ${storageSize}
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示希望使用的存储大小，${storageSize} 的格式遵循 Kubernetes 的 quantity 表达方式, 比如：100Gi。

> 提示如果在定制化配置文件中，重新设置了 `meta_dir` 或者 `LOG_DIR`请重新设置 `mountPath` 。

BE 持久化存储配置

在 Kubernetes 部署 Doris 集群时，建议持久化 `/opt/apache-doris/be/storage` 挂载点，该路径为 BE 节点默认的数据存储路径。在 Kubernetes 部署时，Doris 默认将所有的日志信息输出标准输出（console）。如果集群缺少日志收集能力，建议持久化 `/opt/apache-doris/be/log` 挂载点。

BE 数据持久化

- 默认持久化存储路径

如果 BE 使用默认配置，需要在部署的 DorisCluster 资源中添加如下内容：

```
beSpec:
  persistentVolumes:
  - mountPath: /opt/apache-doris/be/storage
    name: be-storage
    persistentVolumeClaimSpec:
      storageClassName: ${your_storageclass}
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: ${storageSize}
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示希望使用的存储大小，格式遵循 Kubernetes 的 quantity 表达方式, 比如：100Gi。

- 多存储路径持久化

如果自定义配置中通过 `storage_root_path` 指定了多个存储目录（如：`storage_root_path=/home/disk1/doris.HDD;/home/disk2/doris.SSD` ），需要在部署 DorisCluster 资源中添加如下配置：

```
beSpec:
  persistentVolumes:
  - mountPath: /home/disk1/doris
```

```
    name: be-storage1
    persistentVolumeClaimSpec:
      storageClassName: ${your_storageclass}
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: ${storageSize}
  - mountPath: /home/disk2/doris
    name: be-storage2
    persistentVolumeClaimSpec:
      storageClassName: ${your_storageclass}
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: ${storageSize}
```

BE 日志持久化

使用默认配置文件时，在需要部署的 DorisCluster 资源中，添加以下内容：

```
beSpec:
  persistentVolumes:
  - mountPath: /opt/apache-doris/be/log
    name: belog
    persistentVolumeClaimSpec:
      storageClassName: ${your_storageclass}
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: ${storageSize}
```

上述配置中，$your_storageclass[StorageClass](https://kubernetes.io/docs/concepts/storage/storage-classes/)${storageSize} 表示希望使用的存储大小，格式遵循 Kubernetes 的 quantity 表达方式, 比如：100Gi。

访问配置

Kubernetes 通过 Service 作为 vip 和负载均衡器的能力，Service 有三种对外暴漏模式 `ClusterIP` 、 `NodePort` 、 `LoadBalancer` .

ClusterIP

Doris 在 Kubernetes 上默认使用 ClusterIP 访问模式。ClusterIP 访问模式在 Kubernetes 集群内提供了一个内部地址，该地址作为服务在 Kubernetes 内部的被访问地址。

第 1 步：配置使用 ClusterIP 作为 Service 类型

Doris 默认在 Kubernetes 上启用 ClusterIP 访问模式，用户无需额外修改即可使用该模式。

第 2 步：获取 Service 访问地址

部署集群后，通过以下命令可以查看 Doris Operator 暴露的 service：

```
kubectl -n doris get svc
```

返回结果如下：

```
NAME                              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
    ↪                                AGE
doriscluster-sample-be-internal   ClusterIP   None           <none>        9050/TCP
    ↪                                9m
doriscluster-sample-be-service    ClusterIP   10.1.68.128    <none>        9060/TCP,8040/TCP,9050/
    ↪ TCP,8060/TCP    9m
doriscluster-sample-fe-internal   ClusterIP   None           <none>        9030/TCP
    ↪                                14m
doriscluster-sample-fe-service    ClusterIP   10.1.118.16    <none>        8030/TCP,9020/TCP,9030/
    ↪ TCP,9010/TCP    14m
```

在上述结果中，FE 与 BE 各自有两类 Service，分别以 internal 与 service 作为后缀：

以 internal 后缀的 Service 仅供 Doris 内部通信使用，如心跳，数据交换等，不对外暴漏。

以 service 后缀的 Service 用于访问集群服务。

第 3 步：在容器内部访问 Doris

使用如下命令在当前的 Kubernetes 集群中创建一个包含 MySQL 客户端的 Pod：

```
kubectl run mysql-client --image=mysql:5.7 -it --rm --restart=Never --namespace=doris -- /bin/
    ↪ bash
```

在容器内部，可以通过访问带有 `service` 后缀的 Service 名称连接 Doris 集群：

```
mysql -uroot -P9030 -hdoriscluster-sample-fe-service
```

NodePort

若需从 Kubernetes 集群外部访问 Doris，可以选择 NodePort 的模式。NodePort 模式提供两种配置方式：静态宿主机端口映射和动态宿主机端口分配。

- 动态宿主机端口分配：如果未显示设置端口映射，Kubernetes 会在创建 pod 的时自动分配一个宿主机未被使用的端口（默认范围为 30000-32767）；
- 静态宿主机端口分配：如果显示指定了端口映射，当宿主机端口未被占用且无冲突的时，Kubernetes 会固定分配该端口。

静态分配需要规划端口映射，Doris 提供以下端口用于与外部交互：

端口名称	默认端口	端口描述
Query Port	9030	用于通过 MySQL 协议访问 Doris 集群
HTTP Port	8030	FE 上的 http server 端口，用于查看 FE 的信息
Web Server Port	8040	BE 上的 http server 端口，用于查看 BE 的信息

第 1 步：配置 FE 和 BE 的 NodePort

FE NodePort

- 动态分配配置：

```
spec:
  feSpec:
    service:
      type: NodePort
```

- 静态分配配置示例：

```
spec:
  feSpec:
    service:
      type: NodePort
      servicePorts:
      - nodePort: 31001
        targetPort: 8030
      - nodePort: 31002
        targetPort: 9030
```

BE NodePort

- 动态分配配置：

```
spec:
  beSpec:
    service:
      type: NodePort
```

- 静态分配配置示例：

```
beSpec:
  service:
    type: NodePort
    servicePorts:
    - nodePort: 31006
      targetPort: 8040
```

第 2 步：获取 Service

集群部署完成后，通过以下命令查看 Service：

```
kubectl get service
```

返回结果如下：

```
NAME                                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
    ↪                                                            AGE
kubernetes                          ClusterIP   10.152.183.1     <none>        443/TCP
    ↪                                                           169d
doriscluster-sample-fe-internal     ClusterIP   None             <none>        9030/TCP
    ↪                                                          2d
doriscluster-sample-fe-service      NodePort    10.152.183.58    <none>        8030:31041/TCP
    ↪ ,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
doriscluster-sample-be-internal     ClusterIP   None             <none>        9050/TCP
    ↪                                                          2d
doriscluster-sample-be-service      NodePort    10.152.183.244   <none>        9060:30940/TCP
    ↪ ,8040:32713/TCP,9050:30621/TCP,8060:30926/TCP   2d
```

第 3 步：使用 NodePort 访问服务

以 mysql 连接为例，Doris 的 Query Port 默认端口 9030，在上述示例中，端口 9030 被映射到本地端口 31545。要访问 Doris 集群，需要获取到集群的节点 IP 地址，可以使用以下命令查看：

```
kubectl get nodes -owide
```

返回结果如下：

```
NAME    STATUS    ROLES           AGE    VERSION    INTERNAL-IP     EXTERNAL-IP   OS-IMAGE
    ↪ KERNEL-VERSION          CONTAINER-RUNTIME
r60     Ready     control-plane   14d    v1.28.2    192.168.88.60   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r61     Ready     <none>          14d    v1.28.2    192.168.88.61   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r62     Ready     <none>          14d    v1.28.2    192.168.88.62   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r63     Ready     <none>          14d    v1.28.2    192.168.88.63   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
```

在 NodePort 模式下，可以通过任意 node 节点的 IP 地址与映射的宿主机端口方位 Kubernetes 集群内的服务。在本例中，可以使用的 node 节点 IP 包括 192.168.88.61、192.168.88.62、192.168.88.63。以下示例展示了如何使用节点 192.168.88.62 和 query port 映射的宿主机端口 31545 连接 Doris：

```
mysql -h 192.168.88.62 -P 31545 -uroot
```

LoadBalancer

LoadBalancer 是由云服务商提供的负载均衡器。此配置仅适用于云平台提供的 Kubernetes 环境。

第 1 步：配置 LoadBalancer 模式

FE 配置 LoadBalancer

```
spec:
  feSpec:
    service:
      type: LoadBalancer
```

BE 配置 LoadBalancer

```
  spec:
    beSpec:
      service:
        type: LoadBalancer
```

第 2 步：获取 Service

在部署集群后，通过以下命令可以查看可访问 Doris 的 Service：

```
kubectl get service
```

返回结果如下：

```
NAME                                TYPE           CLUSTER-IP       EXTERNAL-IP
    ↪                                                                    PORT(S)
    ↪                                                          AGE
kubernetes                          ClusterIP      10.152.183.1     <none>
    ↪                                                                              443/TCP
    ↪                                                          169d
doriscluster-sample-fe-internal     ClusterIP      None             <none>
    ↪                                                                              9030/TCP
    ↪                                                         2d
doriscluster-sample-fe-service      LoadBalancer   10.152.183.58
    ↪ ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com
    ↪ 8030:31041/TCP,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
doriscluster-sample-be-internal     ClusterIP      None             <none>
    ↪                                                                              9050/TCP
    ↪                                                         2d
doriscluster-sample-be-service      LoadBalancer   10.152.183.244
    ↪ ac4828493dgrftb884g67wg4tb68gyut-1137823345.us-east-1.elb.amazonaws.com
    ↪ 9060:30940/TCP,8040:32713/TCP,9050:30621/TCP,8060:30926/TCP   2d
```

第 3 步：使用 LoadBalancer 模式访问

以 MySQL 连接为例：

```
mysql -h ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com -P 31545 -uroot
```

配置管理用户名和密码

Doris 节点的管理需要通过用户名、密码以 MySQL 协议连接活着的 FE 节点进行操作。Doris 实现类似 RBAC 的权限管理机制，节点的管理需要用户拥有Node_priv 权限。Doris Operator 默认使用拥有所有权限的 root 用户无密

码模式对 DorisCluster 资源配置的集群进行部署和管理。root 用户添加密码后，需要在 DorisCluster 资源中显示配置拥有 Node_Priv 权限的用户名和密码，以便 Doris Operator 对集群进行自动化管理操作。

DorisCluster 资源提供两种方式来配置管理集群节点所需的用户名、密码，包括：环境变量配置的方式，以及使用 Secret 配置的方式。配置集群管理的用户名和密码分为 3 种情况：

- 集群部署需初始化 root 用户密码；
- root 无密码部署下，自动化设置拥有管理权限的非 root 用户；
- 集群 root 无密码模式部署后，设置 root 用户密码。

集群部署配置 root 用户密码

Doris 支持将 root 的用户以密文的形式配置在 `fe.conf` 中，在 Doris 首次部署时配置 root 用户的密码，以便让 Doris Operator 能够自动管理集群节点，请按照如下步骤操作：

第 1 步：构建 root 加密密码

Doris 支持密文的方式在FE 的配置文件中设置 root 用户的密码，密码的加密方式是采用两阶段 SHA-1 加密实现。代码实现示例如下：

Java 代码实现：

```
import org.apache.commons.codec.digest.DigestUtils;

public static void main( String[] args ) {
      //the original password
      String a = "123456";
      String b = DigestUtils.sha1Hex(DigestUtils.sha1(a.getBytes())).toUpperCase();
      //output the 2 stage encrypted password.
      System.out.println("*"+b);
  }
```

Golang 代码实现：

```
import (
"crypto/sha1"
"encoding/hex"
"fmt"
"strings"
)

func main() {
    //original password
    plan := "123456"
    //the first stage encryption.
    h := sha1.New()
    h.Write([]byte(plan))
    eb := h.Sum(nil)
```

```
    //the two stage encryption.
    h.Reset()
    h.Write(eb)
    teb := h.Sum(nil)
    dst := hex.EncodeToString(teb)
    tes := strings.ToUpper(fmt.Sprintf("%s", dst))
    //output the 2 stage encrypted password.
    fmt.Println("*"+tes)
}
```

将加密后的密码按照配置文件要求配置到 `fe.conf` 中，根据集群参数配置章节章节的说明，将配置文件以 ConfigMap 的形式下发到 Kubernetes 集群。

第 2 步：构建 DorisCluster 资源

配置文件设置了 root 初始化密码后，当 Doris FE 第一个节点启动后 root 的密码会立即生效，后续节点加入集群时，Doris Operator 将使用 root 用户名和密码来添加节点。因此，需要在部署的 DorisCluster 资源中指定用户名和密码，以便 Doris Operator 管理集群节点。

- 环境变量方式

将 root 用户名和密码配置到 DorisCluster 资源中的 “.spec.adminUser.name” 和 “.spec.adminUser.password” 字段，Doris Operator 会自动将这些配置转为容器的环境变量，容器内的辅助服务会使用环境变量来添加节点到集群。配置格式如下：

```
spec:
  adminUser:
    name: root
    password: ${password}
```

其中，`${password}` 为 root 的非加密密码。

- Secret 方式

Doris Operator 提供使用 Basic authentication Secret 来指定管理节点的用户名和密码，Doris Operator 会自动将 Secret 以文件形式挂载到容器指定位置，容器的辅助服务会解析出文件中的用户名和密码，用于自动将节点加入集群。basic-authentication-secret 的 stringData 只包含 2 个字段：username 和 password。使用 Secret 配置管理用户名和密码流程如下：

a. 配置需要使用的 Secret

按照如下格式配置需要使用的 Basic Authentication Secret：

```
stringData:
  username: root
  password: ${password}
```

其中，${password} 为 root 设置的非加密密码。
通过如下命令将更新后的 Secret 部署到 Kubernetes 集群中。

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，${namespace} 为 DorisCluster 资源需要部署的命名空间，${secretFileName} 为需要部署的 Secret 的文件名称。

b. 配置 DorisCluster 资源

在需要部署的 DorisCluster 资源中，指定使用的 Secret。配置如下：

```
spec:
  authSecret: ${secretName}
```

其中，${secretName} 为包含 root 用户名和密码的 Secret 名称。

部署时自动创建非 root 管理用户和密码（推荐）

在首次部署时，如果不设置 root 的初始化密码，通过环境变量或者 Secret 的方式配置非 root 用户和登录密码。Doris 容器的辅助服务会自动在 Doris 中创建该用户，设置密码和赋予 Node_priv 权限，Doris Operator 将使用自动创建的用户名和密码管理集群节点。

- 环境变量模式

按照如下格式配置需要部署的 DorisCluster 资源：

```
spec:
  adminUser:
    name: ${DB_ADMIN_USER}
    password: ${DB_ADMIN_PASSWD}
```

其中，${DB_ADMIN_USER} 为需要新建拥有管理权限的用户名，${DB_ADMIN_PASSWD} 为新建用户的密码。

- Secret 方式

a. 配置需要使用的 Secret

按照如下格式配置需要使用的 Basic authentication Secret：

```
stringData:
  username: ${DB_ADMIN_USER}
  password: ${DB_ADMIN_PASSWD}
```

其中，${DB_ADMIN_USER} 为新创建的用户名，${DB_ADMIN_PASSWD} 为新建用户名设置的密码。

使用以下命令将 Secret 部署到 Kubernetes 集群中：

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，${namespace} 为 DorisCluster 资源部署的命名空间，${secretFileName} 为需要部署的 Secret 的文件名称。

b. 更新 DorisCluster 资源

在 DorisCluster 资源中指定使用的 Secret，如下所示：

```
spec:
  authSecret: ${secretName}
```

其中，${secretName} 为部署的 Basic Authentication Secret 的名称。

> 提示 - 部署后请设置 root 的密码，Doris Operator 会切换为使用新用户和密码管理集群节点，请避免删除新建的用户。

集群部署后设置 root 用户密码

Doris 集群在部署后，若未设置 root 用户的密码。需要配置一个具有Node_priv 权限的用户，便于 Doris Operator 自动化的管理集群节点。建议不要使用 root 用户，请参考用户新建和权限赋值章节来创建新用户并赋予 Node_priv 权限。创建用户后，通过环境变量或者 Secret 配置新的管理用户和密码，并在 DorisCluster 资源中配置。

第 1 步：新建拥有 Node_priv 权限用户

通过 MySQL 协议连接数据库后，通过如下命令创建一个仅拥有 Node_priv 权限的用户并设置密码。

```
CREATE USER '${DB_ADMIN_USER}' IDENTIFIED BY '${DB_ADMIN_PASSWD}';
```

其中 DB_ADMIN_USER {DB_ADMIN_PASSWD} 为要设置的密码。

第 2 步：为新用户赋予 Node_priv 权限

使用 MySQL 协议连接数据库后，执行如下命令将 Node_priv 权限赋予新用户。

```
GRANT NODE_PRIV ON *.*.* TO ${DB_ADMIN_USER};
```

其中，${DB_ADMIN_USER} 为新创建的用户名。

新建用户名密码，以及赋予权限详细使用，请参考官方文档CREATE-USER 部分。

第 3 步：配置 DorisCluster 资源

- 环境变量方式

在 DorisCluster 资源中配置新建用户及其密码，格式如下：

```
spec:
  adminUser:
    name: ${DB_ADMIN_USER}
    password: ${DB_ADMIN_PASSWD}
```

其中，DB_ADMIN_USER{DB_ADMIN_PASSWD} 为新建用户设置的密码。

- Secret 方式

a. 配置 Secret

按照如下格式创建 Basic Authentication Secret：

```
stringData:
  username: ${DB_ADMIN_USER}
  password: ${DB_ADMIN_PASSWD}
```

其中 DB_ADMIN_USER{DB_ADMIN_PASSWD} 为新建用户名设置的密码。

使用以下命令将 Secret 部署到 Kubernetes 集群：

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，`${namespace}` 为 DorisCluster 资源部署的命名空间，`${secretFileName}` 为需要部署的 Secret 的文件名称。

b. 更新需要使用 Secret 的 DorisCluster 资源

在 DorisCluster 资源中指定使用的 Secret，如下所示：

```
spec:
  authSecret: ${secretName}
```

其中，`${secretName}` 为部署的 Basic authentication Secret 的名称。

> 提示 - 部署后设置 root 密码，并配置新的拥有管理节点的用户名和密码后，会引起存量服务滚动重启一次。

启动配置修改后自动重启服务生效参数

Doris 通过配置文件的方式指定启动参数。目前大部分参数可以通过相应的 web 接口进行修改并实时生效，一些不能通过 web 接口修改的参数需要重启服务生效。Doris Operator 从 25.1.0 版本后提供服务启动参数修改后自动重启生效的能力。
在 `DorisCluster` 资源中配置开启上述能力，配置如下：

```
spec:
  enableRestartWhenConfigChange: true
```

如果 DorisCluster 资源含有上述配置，Doris Operator 将会进行如下处理：1. 监测 `DorisCluster` 资源部署的集群依赖的启动配置 (通过 ConfigMap 挂载，详情请查看定制化启动配置章节) 是否发生变化。2. 启动配置变化后，自动重启相应服务来使配置生效。

使用范例

支持 FE、BE 节点类型的 configmap 监测重启，这里以 FE 为例。1. DorisCluster 部署规格如下：

```
  spec:
    enableRestartWhenConfigChange: true
    feSpec:
      image: apache/doris:fe-2.1.8
      replicas: 1
      configMapInfo:
        configMapName: fe-configmap
```

2. 更新 fe-configmap 里面指定的 FE 服务启动配置。
   当更新 fe-configmap 中 key 为 fe.conf 对应的值 (FE 服务的启动配置) 后，Doris Operator 自动滚动重启 FE 服务使配置生效。

配置启动配置修改后自动重启服务

Doris 的服务通过配置文件的方式指定启动参数。目前大部分参数可以通过相应的 web 接口进行修改并实时生效，一些不能通过 web 接口修改的参数需要重启服务生效。Doris Operator 的 25.1.0 版本后提供修改服务启动参数自动重启生效的能力。启动参数修改并自动重启配置如下：

```
spec:
  enableRestartWhenConfigChange: true
```

如果 DorisCluster 资源含有上述配置，Doris Operator 会监测 DorisCluster 资源部署的集群依赖的启动配置 (通过 ConfigMap 挂载，详情请查看定制化启动配置章节) 是否发生变化，如果集群服务的启动配置发生变化，Doris Operator 会自动重启相应服务来使配置生效。
FE 的使用范例如下：1. DorisCluster 部署规格如下：

```
spec:
  enableRestartWhenConfigChange: true
  feSpec:
    image: apache/doris:fe-2.1.8
    replicas: 1
    configMapInfo:
      configMapName: fe-configmap
```

2. 更新 fe-configmap 里面指定的 FE 服务启动配置。

当更新 fe-configmap 中 key 为 fe.conf 对应的值 (为 FE 服务的启动配置) 后，Doris Operator 自动滚动重启 FE 服务使配置生效。

使用 Kerberos 认证

Doris Operator 从 25.2.0 版本开始支持 Doris (2.1.9 和 3.0.4 及以后版本) 在 Kubernetes 使用 Kerberos 认证。Doris 使用 Kerberos 认证需要使用 krb5.conf 和 keytab 文件 。 Doris Operator 使用 ConfigMap 资源挂载 krb5.conf 文件，使用 Secret 资源挂载 keytab 文件。使用 Kerberos 认证流程如下：1. 构建包含 krb5.conf 文件的 ConfigMap：

```
  kubectl create -n ${namespace} create configmap ${name} --from-file=krb5.conf
```

$namespace`DorisCluster`$ {name} 为 ConfigMap 想要指定的名字。2. 构建包含 keytab 的 Secret:

```
kubectl create -n ${namespace} secret generic ${name} --from-file= ${xxx.keytab}
```

$namespace`DorisCluster`$ {name} 为 Secret 想要指定的名字，如果需要挂载多个 keytab 文件，请参考 kubectl 创建 Secret 文档将多个 keytab 文件放到一个 Secret 中。3. 配置 DorisCluster 资源，指定包含 krb5.conf 的 ConfigMap, 以及包含 keytab 文件的 Secret。

```
spec:
  kerberosInfo:
    krb5ConfigMap: ${krb5ConfigMapName}
    keytabSecretName: ${keytabSecretName}
    keytabPath: ${keytabPath}
```

$krb5ConfigMapName`krb5.conf`ConfigMap$ {keytabSecretName} 为包含 keytab 文件的 Secret 名称。${keytabPath} 为 Secret 希望挂载到容器中的路径，这个路径是创建 catalog 时，通过 hadoop.kerberos.keytab 指定 keytab 的文件所在目录。创建 catalog 请参考配置 Hive Catalog 文档。

配置共享存储

Doris Operator 从 25.4.0 版本开始支持为多个组件的所有 Pod 挂载一个 ReadWriteMany 的共享存储。使用前请提前创建好共享存储 PersistentVolume 和 PersistentVolumeClaim 资源，在部署 Doris 集群之前按照如下配置 DorisCluster 资源：

```
spec:
  sharedPersistentVolumeClaims:
  - mountPath: ${mountPath}
    persistentVolumeClaimName: ${sharedPVCName}
    supportComponents:
    - fe
    - be
```

- ${mountPath} 指定挂载到容器内的绝对路径。
- ${sharedPVCName} 表示被挂载的 PersistentVolumeClaim 的名称。
- supportComponents 指定需要挂载该共享存储的组件名称，上述实例中，指定 FE，BE 两种组件挂载该共享存储，如果 supportComponents 数组为空，表示所有组件部署的的组件都挂载该共享存储。

> 提示 mountPath 支持使用 ${DORIS_HOME} 作为路径前缀。当 mountPath 使用 ${DORIS_HOME} 作为前缀使用时，在 FE 容器中 ${DORIS_HOME} 指代 /opt/apache-doris/fe; 在 BE 容器中 ${DORIS_HOME} 指代 /opt/apache-doris/be。

配置探测超时

DorisCluster 为每种服务提供两种探测超时配置：启动探测超时配置，存活探测超时配置。当服务启动时间超过配置的启动探测超时时间时，则认定服务启动失败并重新启动服务。当服务超过存活探测时间没有响应时，Pod 会被自动重启。####### 启动探测超时配置 - FE 服务启动探测超时配置

```
  spec:
    feSpec:
      startTimeout: 3600
```

以上配置将 FE 的启动超时设置为 3600 秒。- BE 服务启动探测超时配置

```
  spec:
    beSpec:
      startTimeout: 3600
```

以上配置将 BE 的启动超时设置为 3600 秒。####### 存活探测超时配置 - FE 服务存活探测超时配置

```
  spec:
    feSpec:
      liveTimeout: 60
```

以上配置将 FE 的存活超时设置为 60 秒。- BE 服务存活探测超时配置

```
  spec:
    beSpec:
      liveTimeout: 60
```

以上配置将 BE 的存活超时设置为 60 秒。

2.6.1.1.3 部署 Doris 集群

在 Kubernetes 上部署 Doris 集群时，请提前部署 Doris Operator。

部署 Doris 集群的过程分为三个步骤：下载 Doris 部署模板、配置并安装自定义部署模板、检查集群状态。

第 1 步：下载 Doris 部署模板

```
curl -O https://raw.githubusercontent.com/apache/doris-operator/master/doc/examples/doriscluster-
    ↪ sample.yaml
```

第 2 步：安装自定义部署模板

根据集群配置章节按需进行定制化配置，配置完成后通过如下命令部署：

```
kubectl apply -f doriscluster-sample.yaml
```

第 3 步：检查集群部署状态

1. 查看 pods 的状态：

```
 kubectl get pods
```

期望结果：

```
NAME                        READY   STATUS    RESTARTS   AGE
doriscluster-sample-fe-0    1/1     Running   0          2m
doriscluster-sample-be-0    1/1     Running   0          3m
```

2. 查看部署资源的状态：

```
kubectl get dcr -n doris
```

期望结果：

```
NAME                  FESTATUS    BESTATUS    CNSTATUS   BROKERSTATUS
doriscluster-sample   available   available
```

2.6.1.1.4 访问 Doris 集群

Kubernetes 通过 Service 作为 vip 和负载均衡器的能力，Service 有三种对外暴漏模式 `ClusterIP` 、 `NodePort` 、 `LoadBalancer`。

ClusterIP 模式

Doris 在 Kubernetes 上默认使用 ClusterIP 访问模式。ClusterIP 访问模式在 Kubernetes 集群内提供了一个内部地址，该地址作为服务在 Kubernetes 内部的。

首次部署后，通过 MySQL 协议，使用 root 用户无密码的模式访问部署如下。

第 1 步：获取 Service

部署集群后，通过以下命令可以查看 Doris Operator 暴露的 service：

```
kubectl -n doris get svc
```

返回结果如下：

```
NAME                              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
    ↪                                AGE
doriscluster-sample-be-internal   ClusterIP   None           <none>        9050/TCP
    ↪                                9m
doriscluster-sample-be-service    ClusterIP   10.1.68.128    <none>        9060/TCP,8040/TCP,9050/
    ↪ TCP,8060/TCP    9m
doriscluster-sample-fe-internal   ClusterIP   None           <none>        9030/TCP
    ↪                                14m
doriscluster-sample-fe-service    ClusterIP   10.1.118.16    <none>        8030/TCP,9020/TCP,9030/
    ↪ TCP,9010/TCP    14m
```

在上述结果中，FE 与 BE 各自有两类 Service，分别以 internal 与 service 作为后缀：

- 以 internal 后缀的 Service 仅供 Doris 内部通信使用，如心跳，数据交换等，不对外暴漏。

- 以 service 后缀的 Service 用于访问集群服务。

第 2 步：访问 Doris

ClusterIP 模式只能在 Kubernetes 内部使用，使用如下命令在当前的 Kubernetes 集群中创建一个包含 MySQL 客户端的 Pod：

```
kubectl run mysql-client --image=mysql:5.7 -it --rm --restart=Never --namespace=doris -- /bin/
    ↪ bash
```

在容器内部，可以通过访问带有 service 后缀的 Service 名称连接 Doris 集群：

```
mysql -uroot -P9030 -hdoriscluster-sample-fe-service
```

NodePort 模式

按照 DorisCluster 访问配置章节，配置使用 NodePort 访问模式后，使用 MySQL 协议，通过 root 无密码模式访问 FE 步骤如下。

第 1 步：获取 Service

集群部署完成后，通过以下命令查看 Service：

```
kubectl get service
```

返回结果如下：

```
NAME                              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
    ↪                                                        AGE
kubernetes                        ClusterIP   10.152.183.1     <none>        443/TCP
    ↪                                                       169d
doriscluster-sample-fe-internal   ClusterIP   None             <none>        9030/TCP
    ↪                                                      2d
doriscluster-sample-fe-service    NodePort    10.152.183.58    <none>        8030:31041/TCP
    ↪ ,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
doriscluster-sample-be-internal   ClusterIP   None             <none>        9050/TCP
    ↪                                                      2d
doriscluster-sample-be-service    NodePort    10.152.183.244   <none>        9060:30940/TCP
    ↪ ,8040:32713/TCP,9050:30621/TCP,8060:30926/TCP   2d
```

第 2 步：访问 Doris

以 mysql 连接为例，Doris 的 Query Port 默认端口 9030，在上述示例中，端口 9030 被映射到本地端口 31545。要访问 Doris 集群，需要获取到集群的节点 IP 地址，可以使用以下命令查看：

```
kubectl get nodes -owide
```

返回结果如下：

```
NAME    STATUS   ROLES           AGE   VERSION   INTERNAL-IP     EXTERNAL-IP   OS-IMAGE
    ↪ KERNEL-VERSION          CONTAINER-RUNTIME
```

```
r60    Ready    control-plane   14d   v1.28.2   192.168.88.60   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r61    Ready    <none>          14d   v1.28.2   192.168.88.61   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r62    Ready    <none>          14d   v1.28.2   192.168.88.62   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r63    Ready    <none>          14d   v1.28.2   192.168.88.63   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
```

在 NodePort 模式下，可以通过任意 node 节点的 IP 地址与映射的宿主机端口方位 Kubernetes 集群内的服务。在本例中，可以使用的 node 节点 IP 包括 192.168.88.61、192.168.88.62、192.168.88.63。以下示例展示了如何使用节点 192.168.88.62 和 query port 映射的宿主机端口 31545 连接 Doris：

```
mysql -h 192.168.88.62 -P 31545 -uroot
```

LoadBalancer 模式

按照 DorisCluster 访问配置章节，在公有云上，配置使用 LoadBalancer 访问模式后，使用 MySQL 协议，通过 root 无密码模式访问 FE 步骤如下。

第 1 步：获取 Service

在部署集群后，通过以下命令可以查看可访问 Doris 的 Service：

```
kubectl get service
```

返回结果如下：

```
NAME                              TYPE           CLUSTER-IP       EXTERNAL-IP
    ↪                                                                  PORT(S)
    ↪                                                 AGE
kubernetes                        ClusterIP      10.152.183.1     <none>
    ↪                                                                       443/TCP
    ↪                                                 169d
doriscluster-sample-fe-internal   ClusterIP      None             <none>
    ↪                                                                       9030/TCP
    ↪                                                 2d
doriscluster-sample-fe-service    LoadBalancer   10.152.183.58
    ↪ ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com
    ↪ 8030:31041/TCP,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
doriscluster-sample-be-internal   ClusterIP      None             <none>
    ↪                                                                       9050/TCP
    ↪                                                 2d
doriscluster-sample-be-service    LoadBalancer   10.152.183.244
    ↪ ac4828493dgrftb884g67wg4tb68gyut-1137823345.us-east-1.elb.amazonaws.com
    ↪ 9060:30940/TCP,8040:32713/TCP,9050:30621/TCP,8060:30926/TCP   2d
```

第 2 步：访问 Doris

以 MySQL 连接为例：

```
mysql -h ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com -P 31545 -uroot
```

StreamLoad 访问部署在 Kubernetes 的 Doris

Doris 提供使用 StreamLoad 模式导入数据。客户端和 Doris 集群在同一个局域网内，客户端可直接使用 FE 的地址作为请求地址，FE 服务接收请求并返回 HTTP 301 的状态码以及 BE 的可访问地址，告诉客户端请求 BE 的地址导入数据。在 Kubernetes 上部署的 Doris 集群使用只在 Kubernetes 内部可访问的地址通信，当使用 StreamLoad 方式配置 FE 的可访问地址时，FE 通过 301 机制返回的是 BE 只在 Kubernetes 内部可访问的地址，导致在 Kubernetes 外部的客户端导入数据失败。

在 Kubernetes 外部的客户端使用 StreamLoad 模式向部署在 Kubernetes 上的 Doris 集群导入数据时，需要配置可从外部访问的 BE 地址作为 StreamLoad 的导入地址。####### 配置 BE Service 外部可访问按照 NodePort 或者 LoadBalancer 配置 BE 服务的 `Service` 可以从 Kubernetes 集群外部访问。更新部署 Doris 集群的 `DorisCluster` 资源。

配置 BE 代理地址

根据NodePort 或者LoadBalancer 获取访问地址的方式，获取可在 Kubernetes 外部访问的地址以及对应的可访问 web_server 服务的端口。将获取到的地址和访问端口配置到使用 StreamLoad 导入数据的请求地址中。

2.6.1.1.5 集群运维

服务 Crash 情况下如何进入容器

在 K8s 环境中服务因为一些预期之外的事情会进入 `CrashLoopBackOff` 状态，通过 `kubectl get pod --namespace ${namespace}` 命令可以查看指定 namespace 下的 pod 状态和 pod_name。

在这种状态下，单纯通过 describe 和 logs 命令无法判定服务出问题的原因。当服务进入 `CrashLoopBackOff` 状态时，需要有一种机制允许部署服务的 pod 进入 `running` 状态方便用户通过 exec 进入容器内进行 debug。

Doris Operator 提供了 Debug 的运行模式，下面描述了当服务进入 `CrashLoopBackOff` 时如何进入 Debug 模式进行人工 Debug，以及解决后如何恢复到正常启动状态。

启动 Debug 模式

当服务一个 pod 进入 CrashLoopBackOff 或者正常运行过程中无法再正常启动时，通过以下步骤让服务进入 Debug 模式，进行手动启动服务查找问题。

1. 通过以下命令给运行有问题的 pod 进行添加 annnotation

```
kubectl annotate pod ${pod_name} --namespace ${namespace} apache.com.doris/runmode=debug
```

当服务进行下一次重启时候，服务会检测到标识 Debug 模式启动的 annotation 就会进入 Debug 模式启动，pod 状态为 `running`。

2. 当服务进入 Debug 模式，此时服务的 pod 显示为正常状态，用户可以通过如下命令进入 pod 内部

```
kubectl --namespace ${namespace} exec -ti ${pod_name} bash
```

3. Debug 下手动启动服务，当用户进入 pod 内部，通过修改对应配置文件有关端口进行手动执行 start_xx.sh 脚本，脚本目录为 /opt/apache-doris/xx/bin 下。

FE 需要修改 query_port，BE 需要修改 heartbeat_service_port，避免 Debug 模式下还能通过 service 访问到 crash 的节点导致误导流。

退出 Debug 模式

当服务定位到问题后需要退出 Debug 运行，此时只需要按照如下命令删除对应的 pod，服务就会按照正常的模式启动。

```
kubectl delete pod ${pod_name} --namespace ${namespace}
```

> 提示 进入 pod 内部后，需要修改配置文件的端口信息，才能手动启动相应的 Doris 组件 - FE 需要修改默认路径为：/opt/apache-doris/fe/conf/fe.conf 的 query_port=9030 配置。- BE 需要修改默认路径为：/opt/apache-doris/be/conf/be.conf 的 heartbeat_service_port=9050 配置。

服务扩缩容

Doris 在 K8s 之上的扩缩容可通过修改 DorisCluster 资源对应组件的 replicas 字段来实现。修改可直接编辑对应的资源，也可通过命令的方式。

获取 DorisCluster 资源

使用命令 kubectl --namespace {namespace} get doriscluster 获取已部署 DorisCluster (简称 dcr) 资源的名称。本文中，我们以 doris 为 namespace.

```
kubectl --namespace doris get doriscluster
NAME                  FESTATUS    BESTATUS    CNSTATUS    BROKERSTATUS
doriscluster-sample   available   available
```

扩缩容资源

K8s 所有运维操作通过修改资源为最终状态，由 Operator 服务自动完成运维。扩缩容操作可通过 kubectl --namespace {namespace} edit doriscluster {dcr_name} 直接进入编辑模式修改对应 spec 的 replicas 值，保存退出后 Doris Operator 完成运维，也可以通过如下命令实现不同组件的扩缩容。

FE 扩容

1. 查看当前 FE 服务数量

```
kubectl --namespace doris get pods -l "app.kubernetes.io/component=fe"
NAME                        READY   STATUS    RESTARTS       AGE
doriscluster-sample-fe-0    1/1     Running   0              10d
```

2. 扩容 FE

```
kubectl --namespace doris patch doriscluster doriscluster-sample --type merge --patch '{"spec
    ↪ ":{"feSpec":{"replicas":3}}}'
```

3. 检测扩容结果

```
kubectl --namespace doris get pods -l "app.kubernetes.io/component=fe"
NAME                        READY   STATUS    RESTARTS   AGE
doriscluster-sample-fe-2    1/1     Running   0          9m37s
doriscluster-sample-fe-1    1/1     Running   0          9m37s
doriscluster-sample-fe-0    1/1     Running   0          8m49s
```

BE 扩容

1. 查看当前 BE 服务数量

```
kubectl --namespace doris get pods -l "app.kubernetes.io/component=be"
NAME                        READY   STATUS    RESTARTS      AGE
doriscluster-sample-be-0    1/1     Running   0             3d2h
```

2. 扩容 BE

```
kubectl --namespace doris patch doriscluster doriscluster-sample --type merge --patch '{"spec
    ↪ ":{"beSpec":{"replicas":3}}}'
```

3. 检测扩容结果

```
kubectl --namespace doris get pods -l "app.kubernetes.io/component=be"
NAME                        READY   STATUS    RESTARTS      AGE
doriscluster-sample-be-0    1/1     Running   0             3d2h
doriscluster-sample-be-2    1/1     Running   0             12m
doriscluster-sample-be-1    1/1     Running   0             12m
```

节点缩容

关于节点缩容问题，Doris-Operator 目前并不能很好的支持节点安全下线，在这里仍能够通过减少集群组件的 replicas 属性来实现减少 FE 或 BE 的目的，这里是直接 stop 节点来实现节点下线，当前版本的 Doris-Operator 并未能实现decommission 安全转移副本后下线。由此可能引发一些问题及其注意事项如下

- 表存在单副本情况下贸然下线 BE 节点，一定会有数据丢失，尽可能避免此操作。
- FE Follower 节点尽量避免随意下线，可能带来元数据损坏影响服务。
- FE Observer 类型节点可以随意下线，并无风险。

- CN 节点不持有数据副本，可以随意下线，但因此会损失存在于该 CN 节点的远端数据缓存，导致数据查询短时间内存在一定的性能回退。

升级 Doris 集群

Doris 集群整体升级需要先升级 BE，再升级 FE。 Doris Operator 基于 Kubernetes 的 滚动更新功能 实现每个组件的滚动平滑升级。

升级前注意事项

- 升级操作推荐在业务低峰期进行。
- 滚动升级过程中，会导致连接到被关闭节点的连接失效，造成请求失败，对于这类业务，推荐在客户端添加重试能力。
- 升级前可以阅读常规升级手册，便于理解升级中的一些原理和注意事项。
- 升级前无法对数据和元数据的兼容性进行验证，因此集群升级一定要避免数据存在单副本情况和集群单 FE FOLLOWER 节点。
- 升级过程中会有节点重启，所以可能会触发不必要的集群均衡和副本修复逻辑，先通过以下命令关闭

```
admin set frontend config("disable_balance" = "true");
admin set frontend config("disable_colocate_balance" = "true");
admin set frontend config("disable_tablet_scheduler" = "true");
```

- Doris 升级请遵守不要跨两个及以上关键节点版本升级的原则，若要跨多个关键节点版本升级，先升级到最近的关键节点版本，随后再依次往后升级，若是非关键节点版本，则可忽略跳过。具体参考升级版本说明

升级操作

升级过程节点类型顺序如下，如果某类型节点不存在则跳过：

```
  cn/be -> fe -> broker
```

建议依次修改对应集群组件的 image 然后应用该配置，待当前类型的组件完全升级成功状态恢复正常后，再进行下一个类型节点的滚动升级。

升级 BE

如果保留了集群的 CRD（Doris Operator 定义了 DorisCluster 类型资源名称的简写）文件，则可以通过修改该配置文件并且 kubectl apply 的命令来进行升级。

1. 修改 spec.beSpec.image

将 apache/doris:be-2.1.8 变为 apache/doris:be-2.1.9

2. 保存修改后应用本次修改进行 BE 升级：

```
kubectl apply -f doriscluster-sample.yaml -n doris
```

也可通过 kubectl edit dcr 的方式直接修改。

1. 查看 namespace 为 ‘doris’ 下的 dcr 列表，获取需要更新的 cluster_name

```
$ kubectl get dcr -n doris
NAME                  FESTATUS    BESTATUS    CNSTATUS
Doriscluster-sample   available   available
```

2. 修改、保存并生效

```
kubectl edit dcr doriscluster-sample -n doris
```

进入文本编辑器后，将找到 spec.beSpec.image ，将 apache/doris:be-2.1.8 修改为 apache/doris:be-2.1.9

3. 查看升级过程和结果：

```
kubectl get pod -n doris
```

当所有 Pod 都重建完毕进入 Running 状态后，升级完成。

升级 FE

如果保留了集群的 crd（Doris-Operator 定义了 DorisCluster 类型资源名称的简写）文件，则可以通过修改该配置文件并且 kubectl apply 的命令来进行升级。

1. 修改 spec.feSpec.image

将 apache/doris:fe-2.1.8 变为 apache/doris:fe-2.1.9

```
vim doriscluster-sample.yaml
```

2. 保存修改后应用本次修改进行 be 升级：

```
kubectl apply -f doriscluster-sample.yaml -n doris
```

也可通过 kubectl edit dcr 的方式直接修改。

1. 修改、保存并生效

```
kubectl edit dcr doriscluster-sample -n doris
```

进入文本编辑器后，将找到spec.feSpec.image，将 apache/doris:fe-2.1.8 修改为 apache/doris:fe-2.1.9

2. 查看升级过程和结果

```
kubectl get pod -n doris
```

当所有 Pod 都重建完毕进入 Running 状态后，升级完成。

升级完成处理

验证集群节点状态

通过访问 Doris 集群文档提供的方式，通过 mysql-client 访问 Doris。

使用 show frontends 和 show backends 等 SQL 查看各个组件的版本和状态。

```
show frontends`\G`;
*************************** 1. row ***************************
             Name: fe_13c132aa_3281_4f4f_97e8_655d01287425
             Host: doriscluster-sample-fe-0.doriscluster-sample-fe-internal.doris.svc.cluster.
                 ↪ local
       EditLogPort: 9010
          HttpPort: 8030
         QueryPort: 9030
           RpcPort: 9020
ArrowFlightSqlPort: -1
             Role: FOLLOWER
         IsMaster: false
        ClusterId: 1779160761
             Join: true
            Alive: true
 ReplayedJournalId: 2422
     LastStartTime: 2024-02-19 06:38:47
     LastHeartbeat: 2024-02-19 09:31:33
          IsHelper: true
            ErrMsg:
          Version: doris-2.1.0
  CurrentConnected: Yes
*************************** 2. row ***************************
             Name: fe_f1a9d008_d110_4780_8e60_13d392faa54e
             Host: doriscluster-sample-fe-2.doriscluster-sample-fe-internal.doris.svc.cluster.
                 ↪ local
       EditLogPort: 9010
          HttpPort: 8030
         QueryPort: 9030
           RpcPort: 9020
ArrowFlightSqlPort: -1
             Role: FOLLOWER
         IsMaster: true
        ClusterId: 1779160761
```

```
              Join: true
             Alive: true
 ReplayedJournalId: 2423
     LastStartTime: 2024-02-19 06:37:35
     LastHeartbeat: 2024-02-19 09:31:33
          IsHelper: true
            ErrMsg:
           Version: doris-2.1.0
  CurrentConnected: No
*************************** 3. row ***************************
              Name: fe_e42bf9da_006f_4302_b861_770d2c955a47
              Host: doriscluster-sample-fe-1.doriscluster-sample-fe-internal.doris.svc.cluster.
                  ↪ local
       EditLogPort: 9010
          HttpPort: 8030
         QueryPort: 9030
           RpcPort: 9020
ArrowFlightSqlPort: -1
              Role: FOLLOWER
          IsMaster: false
         ClusterId: 1779160761
              Join: true
             Alive: true
 ReplayedJournalId: 2422
     LastStartTime: 2024-02-19 06:38:17
     LastHeartbeat: 2024-02-19 09:31:33
          IsHelper: true
            ErrMsg:
           Version: doris-2.1.0
  CurrentConnected: No
3 rows in set (0.02 sec)
```

若 FE 节点 alive 状态为 true，且 Version 值为新版本，则该 FE 节点升级成功。

```
show backends`\G`;
*************************** 1. row ***************************
              BackendId: 10002
                   Host: doriscluster-sample-be-0.doriscluster-sample-be-internal.doris.svc.
                       ↪ cluster.local
          HeartbeatPort: 9050
                 BePort: 9060
               HttpPort: 8040
               BrpcPort: 8060
     ArrowFlightSqlPort: -1
          LastStartTime: 2024-02-19 06:37:56
          LastHeartbeat: 2024-02-19 09:32:43
```

```
                Alive: true
   SystemDecommissioned: false
              TabletNum: 14
       DataUsedCapacity: 0.000
     TrashUsedCapcacity: 0.000
          AvailCapacity: 12.719 GB
          TotalCapacity: 295.167 GB
                UsedPct: 95.69 %
         MaxDiskUsedPct: 95.69 %
     RemoteUsedCapacity: 0.000
                    Tag: {"location" : "default"}
                 ErrMsg:
                Version: doris-2.1.0
                 Status: {"lastSuccessReportTabletsTime":"2024-02-19 09:31:48","
                     ↪ lastStreamLoadTime":-1,"isQueryDisabled":false,"isLoadDisabled":false}
HeartbeatFailureCounter: 0
               NodeRole: mix
*************************** 2. row ***************************
             BackendId: 10003
                  Host: doriscluster-sample-be-1.doriscluster-sample-be-internal.doris.svc.
                      ↪ cluster.local
         HeartbeatPort: 9050
                BePort: 9060
              HttpPort: 8040
              BrpcPort: 8060
     ArrowFlightSqlPort: -1
          LastStartTime: 2024-02-19 06:37:35
          LastHeartbeat: 2024-02-19 09:32:43
                  Alive: true
   SystemDecommissioned: false
              TabletNum: 8
       DataUsedCapacity: 0.000
     TrashUsedCapcacity: 0.000
          AvailCapacity: 12.719 GB
          TotalCapacity: 295.167 GB
                UsedPct: 95.69 %
         MaxDiskUsedPct: 95.69 %
     RemoteUsedCapacity: 0.000
                    Tag: {"location" : "default"}
                 ErrMsg:
                Version: doris-2.1.0
                 Status: {"lastSuccessReportTabletsTime":"2024-02-19 09:31:43","
                     ↪ lastStreamLoadTime":-1,"isQueryDisabled":false,"isLoadDisabled":false}
HeartbeatFailureCounter: 0
               NodeRole: mix
```

```
*************************** 3. row ***************************
              BackendId: 11024
                   Host: doriscluster-sample-be-2.doriscluster-sample-be-internal.doris.svc.
                       ↪ cluster.local
          HeartbeatPort: 9050
                 BePort: 9060
               HttpPort: 8040
               BrpcPort: 8060
     ArrowFlightSqlPort: -1
          LastStartTime: 2024-02-19 08:50:36
          LastHeartbeat: 2024-02-19 09:32:43
                  Alive: true
   SystemDecommissioned: false
             TabletNum: 0
       DataUsedCapacity: 0.000
     TrashUsedCapcacity: 0.000
          AvailCapacity: 12.719 GB
          TotalCapacity: 295.167 GB
                UsedPct: 95.69 %
         MaxDiskUsedPct: 95.69 %
     RemoteUsedCapacity: 0.000
                    Tag: {"location" : "default"}
                 ErrMsg:
                Version: doris-2.1.0
                 Status: {"lastSuccessReportTabletsTime":"2024-02-19 09:32:04","
                     ↪ lastStreamLoadTime":-1,"isQueryDisabled":false,"isLoadDisabled":false}
HeartbeatFailureCounter: 0
               NodeRole: mix
3 rows in set (0.01 sec)
```

若 BE 节点 alive 状态为 true，且 Version 值为新版本，则该 BE 节点升级成功

恢复集群副本同步和均衡

在确认各个节点状态无误后，执行以下 SQL 恢复集群均衡和副本修复：

```
admin set frontend config("disable_balance" = "false");
admin set frontend config("disable_colocate_balance" = "false");
admin set frontend config("disable_tablet_scheduler" = "false");
```

FE 使用 metadata_failure_recovery 模式启动

当 FE 无法选主时，服务处于不可用状态时，可以通过选定一个拥有 VLSN 最大值的节点使用 metadata_failure
↪ _recovery 机制强制启动作为 master 节点，依此来恢复集群。

容器环境下使用 recovery 模式启动

1. 找到 VLSN 最大值所在的节点
   k8s 下，FE 的 Pod 每次启动时会输出本节点的上最近 10 条 VLSN 记录，如下图所示：

```
the annotations value:
the value not equal!  debug
/opt/apache-doris/fe/doris-meta/bdb/je.info.0:19:2025-08-05 03:42:47.650 UTC INFO [fe_
    ↪ f35530c4_3ff1_48fe_80d1_cc8e32dbc942] Replica-feeder fe_d8763579_92da_4d72_8c58_4
    ↪ e62b88bdff0 start stream at VLSN: 30
/opt/apache-doris/fe/doris-meta/bdb/je.info.0:21:2025-08-05 03:42:47.659 UTC INFO [fe_
    ↪ f35530c4_3ff1_48fe_80d1_cc8e32dbc942] Replica initialization completed. Replica VLSN
    ↪ : -1  Heartbeat master commit VLSN: 49  DTVLSN:0 Replica VLSN delta: 50
[Tue Aug  5 06:14:05 UTC 2025] start with meta run start_fe.sh with additional options: '--
    ↪ console'
```

以上是一个实例集群 FE 启动时输出的 VLSN 记录，当前节点最大 VLSN 为 30（日志输出前缀为 `start stream at VLSN:`）。

2. 选定最大值节点的 pod 作为使用 recovery 机制的节点。找到 VLSN 最大值所在节点的 pod 后，通过如下命令给 pod 添加需要使用 recovery 机制启动的注解。

```
kubectl annotate pod {podName} "selectdb.com.doris/recovery=true"
```

当 Pod 再次重新启动后，当前节点会自动在启动命令中添加 `--metadata_failure_recovery`，服务以 recovery 模式启动。

3. 服务正常后，必须删除第二步添加的 annotation，否则后面节点重启后会出现不可预期的行为。

> 提示 1. 添加注解后，不可以通过 delete pod 的模式重启，这样会导致注解丢失。等待 kubelet 自动重启拉起，或者进入容器手动 kill 进程。2. 使用 `metadata_failure_recovery` 模式启动，FE 回放日志耗时会很长，在使用该模式启动之前请先修改 FE 服务的启动超时时间，然后删除所有的 FE Pod 在进行 `metadata_failure_recovery` 启动。

2.6.1.2 部署存算分离集群

2.6.1.2.1 部署 FoundationDB

FoundationDB 是基于 Apache 2.0 开源协议的分布式强一致性存储结构化数据的数据库，Doris 存算分离模式使用 FoundationDB 作为元数据存储。Kubernetes 上部署存算分离集群需要提前部署 FoundationDB 服务，推荐两种部署方式：- 在机器（包括物理机）上直接部署。机器直接部署 FoundationDB 请参考 Doris 存算分离官方文档部署前准备部分搭建 FoundationDB 集群。部署前请确保 FoundationDB 部署的机器和 Doris 所在的 Kubernetes 在同一个局域网内。- 在 Kubernetes 上部署 FoundationDB。FoundationDB 官方提供 Kubernetes 上部署运维管理服务 fdb-kubernetes-operator。

在 Kubernetes 上部署 FoundationDB

在 Kubernetes 上部署 FoundationDB 分为 4 步：1. 部署 FoundationDB 相关资源定义。
2. 部署 fdb-kubernetes-operator 服务。
3. 部署 FoundationDB 集群。
4. 确认 FoundationDB 状态。

第 1 步：部署 FoundationDB 相关资源定义

通过以下命令下发 FoundationDB 资源定义：

```
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbclusters.yaml
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbbackups.yaml
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbrestores.yaml
```

预期结果：

```
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbclusters.yaml
customresourcedefinition.apiextensions.k8s.io/foundationdbclusters.apps.foundationdb.org created
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbbackups.yaml
customresourcedefinition.apiextensions.k8s.io/foundationdbbackups.apps.foundationdb.org created
kubectl apply -f https://raw.githubusercontent.com/FoundationDB/fdb-kubernetes-operator/main/
    ↪ config/crd/bases/apps.foundationdb.org_foundationdbrestores.yaml
customresourcedefinition.apiextensions.k8s.io/foundationdbrestores.apps.foundationdb.org created
```

第 2 步：部署 fdb-kubernetes-operator 服务

fdb-kubernetes-operator 仓库提供了以 IP 模式部署 FoundationDB 集群的部署样例。在 doris-operator 仓库中提供了以 FQDN 模式部署的 FoundationDB 集群样例，可以按需下载。1. 下载部署样例 - 从 fdb-kubernetes-operator 官方仓库下载

fdb-kubernetes-operator 默认情况下使用 IP 模式部署 FoundationDB Cluster，可以下载 YAML 文件 fdb-kubernetes-operator 默认部署。如果使用 FQDN 部署模式，请按照官方文档使用 DNS 部分进行定制化使用域名模式。

```
    wget -O fdb-operator.yaml https://raw.githubusercontent.com/foundationdb/fdb-kubernetes-
        ↪ operator/main/config/samples/deployment.yaml
```

```
- 从 doris-operator 仓库下载
doris-operator 仓库中制定化了以 fdb-kuberentes-operator 1.46.0 版本为基础的部署示例，
    ↪ 可直接使用部署 FoundationDB cluster。
```

```
    wget https://raw.githubusercontent.com/apache/doris-operator/master/config/operator/fdb-
        ↪ operator.yaml
```

2. 部署 fdb-kubernetes-operator 服务
   定制化 fdb-kubernetes-operator 的部署 yaml 后，使用如下命令部署 fdb-kubernetes-operator：

```
kubectl apply -f fdb-operator.yaml
```

预期结果：

```
serviceaccount/fdb-kubernetes-operator-controller-manager created
clusterrole.rbac.authorization.k8s.io/fdb-kubernetes-operator-manager-clusterrole created
clusterrole.rbac.authorization.k8s.io/fdb-kubernetes-operator-manager-role created
rolebinding.rbac.authorization.k8s.io/fdb-kubernetes-operator-manager-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/fdb-kubernetes-operator-manager-
    ↪ clusterrolebinding created
deployment.apps/fdb-kubernetes-operator-controller-manager created
```

第 3 步：部署 FoundationDB 集群

在 fdb-kubernetes-operator 仓库中提供了部署 FoundationDB 的部署样例，通过如下命令直接下载使用。

3. 下载部署样例
   从 FoundationDB 官方下载 IP 模式部署样例：

```
wget https://raw.githubusercontent.com/foundationdb/fdb-kubernetes-operator/main/config/
    ↪ samples/cluster.yaml
```

4. 定制化部署样例

   - 环境可访问 dockerhub
     根据官网提供的用户手册定制化部署终态。如果使用 FQDN 部署，请将 routing.useDNSInClusterFile
     ↪ 字段设置为 true，配置如下：
     doris-operator 的官方仓库中提供了使用 FQDN 部署 FoundationDB 的部署样例可直接下载使用。

```
  spec:
routing:
useDNSInClusterFile: true
```

   - 私网环境
     在私网环境下，如果不能直接访问 dockerhub 可从 FoundationDB 的官方仓库中将需要的镜像下载，并推到私有仓库中。fdb-kubernetes-operator 依赖 foundationdb/fdb-kubernetes-operator, foundationdb/foundationdb-kubernetes-sidecar 。部署 FoundationDB 依赖的镜像是 fdb-kubernetes-monitor。
     推到私有仓库后，按照 fdb-kubernetes-operator 官方文档定制化镜像配置说明进行配置。
     可参考如下配置添加私有仓库镜像配置：

```
  spec:
mainContainer:
  imageConfigs:
  - baseImage: foundationdb/fdb-kubernetes-monitor
    tag: 7.1.38
sidecarContainer:
  imageConfigs:
  - baseImage: foundationdb/fdb-kubernetes-monitor
```

```
    tag: 7.1.38
version: 7.1.38
```

在 doris operator 仓库中，总结了 4 种 FoundationDB 的部署形态，单副本模式最简部署，两副本模式最简部署，两副本生产部署，两副本生产使用私有仓库镜像部署。

> 提示 - 部署 FoundationDB 时，FoundationDBCluster 资源，`.spec.version` 必须配置，且为 FoundationDB 发布的版本号。 - FoundationDB 基于 fdb-kubernetes-operator 部署，要求 Kubernetes 集群至少有三台宿主机才可满足生产环境高可用要求。

第 4 步：确认 FoundationDB 状态

FoundationDB 基于 fdb-kubernetes-operator 部署，可以通过如下命令查看 FoundationDB 集群状态：

```
kubectl get fdb
```

预期结果如下，若 `AVAILABLE` 为 `true` 则代表集群可用：

```
NAME           GENERATION   RECONCILED   AVAILABLE   FULLREPLICATION   VERSION   AGE
test-cluster   1            1            true        true              7.1.26    13m
```

获取包含 FoundationDB 访问信息的 ConfigMap

使用 fdb-kubernetes-operator 部署 FoundationDB，会在部署的命名空间下生成一个特定的 ConfigMap 包含 FoundationDB 的访问信息。这个 ConfigMap 的名称为部署 FoundationDB 的资源名称加上 “-config”。使用如下命令查看 ConfigMap：

```
kubectl get configmap
```

预期结果：

```
test-cluster-config   5      15d
```

> 提示在 Kubernetes 上部署，清理 FoundationDBCluster 资源会导致元数据丢失，请慎重处理 FoundationDBCluster 资源。

2.6.1.2.2 集群级配置

存算分离集群存在集群级别的配置，比如管控账号的用户名密码用于管理集群中各个组件的节点等。

配置管理用户名和密码

Doris 节点的管理需要通过用户名、密码以 MySQL 协议连接活着的 FE 节点进行操作。Doris 实现类似 RBAC 的权限管理机制，节点的管理需要用户拥有Node_priv 权限。Doris Operator 默认使用拥有所有权限的 root 用

户无密码模式对 DorisDisaggregatedCluster 资源配置的集群进行部署和管理。root 用户添加密码后，需要在 DorisDisaggregatedCluster 资源中显示配置拥有 Node_Priv 权限的用户名和密码，以便 Doris Operator 对集群进行自动化管理操作。

DorisDisaggregatedCluster 资源提供两种方式来配置管理集群节点所需的用户名、密码，包括：环境变量配置的方式，以及使用 Secret 配置的方式。配置集群管理的用户名和密码分为 3 种情况：

- 集群部署需初始化 root 用户密码；
- root 无密码部署下，自动化设置拥有管理权限的非 root 用户；
- 集群 root 无密码模式部署后，设置 root 用户密码。

集群部署配置 root 用户密码

Doris 支持将 root 的用户以密文的形式配置在 `fe.conf` 中，在 Doris 首次部署时配置 root 用户的密码，以便让 Doris Operator 能够自动管理集群节点，请按照如下步骤操作：

第 1 步：构建 root 加密密码

Doris 支持密文的方式在FE 的配置文件中设置 root 用户的密码，密码的加密方式是采用两阶段 SHA-1 加密实现。代码实现示例如下：

Java 代码实现：

```
import org.apache.commons.codec.digest.DigestUtils;

public static void main( String[] args ) {
      //the original password
      String a = "123456";
      String b = DigestUtils.sha1Hex(DigestUtils.sha1(a.getBytes())).toUpperCase();
      //output the 2 stage encrypted password.
      System.out.println("*"+b);
  }
```

Golang 代码实现：

```
import (
"crypto/sha1"
"encoding/hex"
"fmt"
"strings"
)

func main() {
    //original password
    plan := "123456"
    //the first stage encryption.
    h := sha1.New()
    h.Write([]byte(plan))
```

```
    eb := h.Sum(nil)

    //the two stage encryption.
    h.Reset()
    h.Write(eb)
    teb := h.Sum(nil)
    dst := hex.EncodeToString(teb)
    tes := strings.ToUpper(fmt.Sprintf("%s", dst))
    //output the 2 stage encrypted password.
    fmt.Println("*"+tes)
}
```

将加密后的密码按照配置文件要求配置到 fe.conf 中，根据 FE 启动参数配置章节的说明，将配置文件以 ConfigMap 的形式下发到 Kubernetes 集群。

第 2 步：构建 DorisDisaggregatedCluster 资源

配置文件设置了 root 初始化密码后，当 Doris FE 第一个节点启动后 root 的密码会立即生效，后续节点加入集群时，Doris Operator 将使用 root 用户名和密码来添加节点。因此，需要在部署的 DorisDisaggregatedCluster 资源中指定用户名和密码，以便 Doris Operator 管理集群节点。

- 环境变量方式

将 root 用户名和密码配置到 DorisDisaggregatedCluster 资源中的“.spec.adminUser.name”和“.spec.adminUser.password”字段，Doris Operator 会自动将这些配置转为容器的环境变量，容器内的辅助服务会使用环境变量来添加节点到集群。配置格式如下：

```
  spec:
    adminUser:
      name: root
      password: ${password}
```

其中，${password} 为 root 的非加密密码。

- Secret 方式

Doris Operator 提供使用 Basic authentication Secret 来指定管理节点的用户名和密码，Doris Operator 会自动将 Secret 以文件形式挂载到容器指定位置，容器的辅助服务会解析出文件中的用户名和密码，用于自动将节点加入集群。basic-authentication-secret 的 stringData 只包含 2 个字段：username 和 password。使用 Secret 配置管理用户名和密码流程如下：

a. 配置需要使用的 Secret

按照如下格式配置需要使用的 Basic Authentication Secret：

```
  stringData:
    username: root
    password: ${password}
```

其中，${password} 为 root 设置的非加密密码。
通过如下命令将更新后的 Secret 部署到 Kubernetes 集群中。

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，${namespace} 为 DorisDisaggregatedCluster 资源需要部署的命名空间，${secretFileName} 为需要部署的 Secret 的文件名称。

b. 配置 DorisDisaggregatedCluster 资源

在需要部署的 DorisDisaggregatedCluster 资源中，指定使用的 Secret。配置如下：

```
spec:
  authSecret: ${secretName}
```

其中，${secretName} 为包含 root 用户名和密码的 Secret 名称。

部署时自动创建非 root 管理用户和密码（推荐）

在首次部署时，如果不设置 root 的初始化密码，通过环境变量或者 Secret 的方式配置非 root 用户和登录密码。Doris 容器的辅助服务会自动在 Doris 中创建该用户，设置密码和赋予 Node_priv 权限，Doris Operator 将使用自动创建的用户名和密码管理集群节点。

- 环境变量模式

按照如下格式配置需要部署的 DorisDisaggregatedCluster 资源：

```
spec:
  adminUser:
    name: ${DB_ADMIN_USER}
    password: ${DB_ADMIN_PASSWD}
```

其中，${DB_ADMIN_USER} 为需要新建拥有管理权限的用户名，${DB_ADMIN_PASSWD} 为新建用户的密码。

- Secret 方式

a. 配置需要使用的 Secret

按照如下格式配置需要使用的 Basic authentication Secret：

```
stringData:
  username: ${DB_ADMIN_USER}
  password: ${DB_ADMIN_PASSWD}
```

其中，${DB_ADMIN_USER} 为新创建的用户名，${DB_ADMIN_PASSWD} 为新建用户名设置的密码。

使用以下命令将 Secret 部署到 Kubernetes 集群中：

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，${namespace} 为 DorisDisaggregatedCluster 资源部署的命名空间，${secretFileName} 为需要部署的 Secret 的文件名称。

b. 更新 DorisDisaggregatedCluster 资源

在 DorisDisaggregatedCluster 资源中指定使用的 Secret，如下所示：

```
spec:
  authSecret: ${secretName}
```

其中，${secretName} 为部署的 Basic Authentication Secret 的名称。

> 提示 - 部署后请设置 root 的密码，Doris Operator 会切换为使用新用户和密码管理集群节点，请避免删除新建的用户。

集群部署后设置 root 用户密码

Doris 集群在部署后，若未设置 root 用户的密码。需要配置一个具有Node_priv 权限的用户，便于 Doris Operator 自动化的管理集群节点。建议不要使用 root 用户，请参考用户新建和权限赋值章节来创建新用户并赋予 Node_priv 权限。创建用户后，通过环境变量或者 Secret 配置新的管理用户和密码，并在 DorisDisaggregatedCluster 资源中配置。

第 1 步：新建拥有 Node_priv 权限用户

通过 MySQL 协议连接数据库后，通过如下命令创建一个仅拥有 Node_priv 权限的用户并设置密码。

```
CREATE USER '${DB_ADMIN_USER}' IDENTIFIED BY '${DB_ADMIN_PASSWD}';
```

其中 DB_ADMIN_USERDB_ADMIN_PASSWD} 为要设置的密码。

第 2 步：为新用户赋予 Node_priv 权限

使用 MySQL 协议连接数据库后，执行如下命令将 Node_priv 权限赋予新用户。

```
GRANT NODE_PRIV ON *.*.* TO ${DB_ADMIN_USER};
```

其中，${DB_ADMIN_USER} 为新创建的用户名。

新建用户名密码，以及赋予权限详细使用，请参考官方文档CREATE-USER 部分。

第 3 步：配置 DorisDisaggregatedCluster 资源

- 环境变量方式

在 DorisDisaggregatedCluster 资源中配置新建用户及其密码，格式如下：

```
spec:
  adminUser:
    name: ${DB_ADMIN_USER}
    password: ${DB_ADMIN_PASSWD}
```

其中，DB_ADMIN_USER{DB_ADMIN_PASSWD} 为新建用户设置的密码。

- Secret 方式

a. 配置 Secret

按照如下格式创建 Basic Authentication Secret：

```
stringData:
  username: ${DB_ADMIN_USER}
  password: ${DB_ADMIN_PASSWD}
```

其中 DB_ADMIN_USER{DB_ADMIN_PASSWD} 为新建用户名设置的密码。

使用以下命令将 Secret 部署到 Kubernetes 集群：

```
kubectl -n ${namespace} apply -f ${secretFileName}.yaml
```

其中，`${namespace}` 为 DorisDisaggregatedCluster 资源部署的命名空间，`${secretFileName}` 为需要部署的 Secret 的文件名称。

b. 更新需要使用 Secret 的 DorisDisaggregatedCluster 资源

在 DorisDisaggregatedCluster 资源中指定使用的 Secret，如下所示：

```
spec:
  authSecret: ${secretName}
```

其中，`${secretName}` 为部署的 Basic authentication Secret 的名称。

> 提示 - 部署后设置 root 密码，并配置新的拥有管理节点的用户名和密码后，会引起存量服务滚动重启一次。

使用 Kerberos 认证

Doris Operator 从 25.5.1 版本开始支持 Doris 存算分离集群 (2.1.10 和 3.0.6 及以后版本) 在 Kubernetes 使用 Kerberos 认证。 Doris 使用 Kerberos 认证需要使用 krb5.conf 和 keytab 文件 。 Doris Operator 使用 `ConfigMap` 资源挂载 krb5.conf 文件，使用 `Secret` 资源挂载 keytab 文件。使用 Kerberos 认证流程如下： 1. 构建包含 krb5.conf 文件的 ConfigMap：

```
   kubectl create -n ${namespace} create configmap ${name} --from-file=krb5.conf
```

$namespace‘DorisDisaggregatedCluster‘{name} 为 ConfigMap 想要指定的名字。 2. 构建包含 keytab 的 Secret:

```
   kubectl create -n ${namespace} secret generic ${name} --from-file= ${xxx.keytab}
```

$namespace`DorisDisaggregatedCluster`${name} 为 Secret 想要指定的名字，如果需要挂载多个 keytab 文件，请参考 kubectl 创建 Secret 文档将多个 keytab 文件放到一个 Secret 中。3. 配置 DorisDisaggregatedCluster 资源，指定包含 krb5.conf 的 ConfigMap, 以及包含 keytab 文件的 Secret。

```
spec:
  kerberosInfo:
    krb5ConfigMap: ${krb5ConfigMapName}
    keytabSecretName: ${keytabSecretName}
    keytabPath: ${keytabPath}
```

$krb5ConfigMapName`krb5.conf`ConfigMap${keytabSecretName} 为包含 keytab 文件的 Secret 名称。${keytabPath} 为 Secret 希望挂载到容器中的路径，这个路径是创建 catalog 时，通过 hadoop.kerberos.keytab 指定 keytab 的文件所在目录。创建 catalog 请参考配置 Hive Catalog 文档。

2.6.1.2.3 配置部署 MetaService

MetaService 是 Doris 存算分离集群元数据管理组件，不对外暴漏，仅用于内部使用。MetaService 属于无状态服务，通常采用主备模式部署。下面介绍如何在 DorisDisaggregatedCluster 资源中配置 MetaService。

配置 FoundationDB 访问

根据 FoundationDB 部署环境不同，配置方式也有所差异：- 使用 ConfigMap 配置 FoundationDB 访问
如果 FoundationDB 集群通过 fdb-kubernetes-operator 部署，可直接使用该 Operator 生成的包含 FoundationDB 访问地址的 ConfigMap，示例如下：

```
spec:
  metaService:
    fdb:
      configMapNamespaceName:
        name: ${foundationdbConfigMapName}
        namespace: ${namespace}
```

```
其中，`${foundationdbConfigMapName}` 为 ConfigMap 的名称。`${namespace}` 为 FoundationDB
    ↪ 部署的命名空间。查找 `fdb-kubernetes-operator` 生成的 ConfigMap，请参考部署 FoundationDB
    ↪ 章节的 [获取包含 FoundationDB 访问信息的 ConfigMap](\#)。
```

- 直接配置 FoundationDB 访问地址
  如果 FoundationDB 是直接在物理机上部署，则可以直接在 MetaService 配置中指定访问地址：

  ```
  spec:
    metaService:
      fdb:
        address: ${fdbEndpoint}
  ```

  ${fdbEndpoint} 为可访问 FoundationDB 的访问地址信息，物理机部署情况下查找请参考存算分离章节MetaService 部署获取 fdb_cluster 介绍。

配置镜像

在部署样例中，MetaService 配置的镜像可能不是最新版本镜像。自定义镜像时，请按照如下格式配置：

```
spec:
  metaService:
    image: ${msImage}
```

其中 `${msImage}` 为想要部署的 MetaService 的镜像。请使用 Doris 官方提供的 MetaService 镜像(镜像 tag 中包含 ms 前缀)。

配置资源

可以通过 Kubernetes 的资源限制为 MetaService 分配合适的计算资源，例如限制为 4 核 CPU 和 4Gi 内存，配置如下：

```
spec:
  metaService:
    requests:
      cpu: 4
      memory: 4Gi
    limits:
      cpu: 4
      memory: 4Gi
```

将配置更新到需要部署的 DorisDisaggregatedCluster 资源中。

定制化启动配置

Doris-Operator 通过 ConfigMap 挂载组件的启动配置文件。Doris-Operator 自动填充 MetaService 启动配置中有关 FoundationDB 的相关配置，因此定制化启动配置时无需填写这些信息。1. 创建自定义 ConfigMap
自定义一个包含启动配置信息的 ConfigMap。启动配置文件的名称必须为 `doris_cloud.conf`，示例如下：

```
    apiVersion: v1
    data:
      doris_cloud.conf: |
        # // meta_service
        brpc_listen_port = 5000
        brpc_num_threads = -1
        brpc_idle_timeout_sec = 30
        http_token = greedisgood9999

        # // doris txn config
        label_keep_max_second = 259200
        expired_txn_scan_key_nums = 1000

        # // logging
        log_dir = ./log/
        # info warn error
        log_level = info
```

```
        log_size_mb = 1024
        log_filenum_quota = 10
        log_immediate_flush = false
        # log_verbose_modules = *

        # //max stage num
        max_num_stages = 40
    kind: ConfigMap
    metadata:
      name: doris-metaservice
      namespace: default
```

2. 挂载自定义启动配置

   在 DorisDisaggregatedCluster 资源中，通过 metaService.configMaps 挂载上述 ConfigMap，示例如下：

```
spec:
  metaService:
    configMaps:
    - name: ${msConfigMapName}
      mountPath: /etc/doris
```

   ${msConfigMapName} 为包含 MetaService 启动配置的 ConfigMap 名称。更新到需要部署的 DorisDisaggregated-Cluster 资源。包含启动配置的 ConfigMap 的挂载点必须为 /etc/doris ，即 mountPath 为 /etc/doris。

> 提示在 Kubernetes 部署中，定制化 MetaService 启动配置时请不要填写 fdb_cluster 配置，Doris Operator 会自动处理相关信息。

配置服务探测超时

Doris Operator 为存算分离集群服务提供两种超时参数配置：存活探测超时和启动超时。####### 存活探测超时配置存活探测（ LivenessProbe ）用于监控服务运行状态，当探测失败超过设定阈值时，服务将被强制重启。默认超时时间为 180 秒，若需要配置为 30 秒，可按如下设置

```
spec:
  metaService:
    liveTimeout: 30
```

启动超时配置

启动超时用于应对服务启动时间过长的情况，当服务启动时间超过设定阈值时，服务将被强制重启。默认启动超时时间为 300 秒，若需要配置为 120 秒，可按如下设置：

```
spec:
  metaService:
    startTimeout: 120
```

2.6.1.2.4 配置部署 FE

FE 在存算分离模式下主要负责查询解析和规划等相关工作。

配置计算资源

Doris-Operator 仓库提供的部署样例中，FE 默认不限制资源使用。通过 Kubernetes 的 requests 和 Limits 配置服务的计算资源。例如，为 FE 分配 8c 8Gi 计算资源配置如下：

```
spec:
  feSpec:
    requests:
      cpu: 8
      memory: 8Gi
    limits:
      cpu: 8
      memory: 8Gi
```

将上述配置信息更新到需要部署的 DorisDisaggregatedCluster 资源中。

配置 Follower 数量

FE 服务有 Follower 和 Observer 两种角色，Follower 负责 sql 解析任务和元数据的管理和存储。Observer 主要负责 sql 解析任务，分担 Follower 的查询和写入负载任务。Doris 使用 bdbje 存储系统管理元数据，bdbje 底层实现类似 paxos 协议算法。分布式部署中，需要配置多个 Follower 节点参与分布式环境下元数据管理工作。使用 DorisDisaggregatedCluster 资源部署 Doris 存算分离集群，Follower 默认的数量为 1。可通过如下配置设置 Follower 节点的数量。设置 Follower 节点数量为 3 的配置示例如下：

```
spec:
  feSpec:
    electionNumber: 3
```

> 提示存算分离集群部署后，electionNumber 不允许修改。

自定义启动配置

Doris Operator 通过 Kubernetes 的 ConfigMap 挂载 FE 启动配置。配置步骤如下：1. 自定义一个包含 FE 启动配置的 ConfigMap，样例如下：

```
  apiVersion: v1
  kind: ConfigMap
  metadata:
    name: fe-configmap
    namespace: default
    labels:
      app.kubernetes.io/component: fe
  data:
```

```
  fe.conf: |
    CUR_DATE=`date +%Y%m%d-%H%M%S`
    # Log dir
    LOG_DIR = ${DORIS_HOME}/log
    # For jdk 17, this JAVA_OPTS will be used as default JVM options
    JAVA_OPTS_FOR_JDK_17="-Djavax.security.auth.useSubjectCredsOnly=false -Xmx8192m -Xms8192m
        ↪  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=$LOG_DIR -Xlog:gc*:$LOG_DIR/fe.
        ↪ gc.log.$CUR_DATE:time,uptime:filecount=10,filesize=50M --add-opens=java.base/java
        ↪ .nio=ALL-UNNAMED --add-opens java.base/jdk.internal.ref=ALL-UNNAMED"
    # INFO, WARN, ERROR, FATAL
    sys_log_level = INFO
    # NORMAL, BRIEF, ASYNC
    sys_log_mode = NORMAL
    # Default dirs to put jdbc drivers,default value is ${DORIS_HOME}/jdbc_drivers
    # jdbc_drivers_dir = ${DORIS_HOME}/jdbc_drivers
    http_port = 8030
    rpc_port = 9020
    query_port = 9030
    edit_log_port = 9010
    enable_fqdn_mode=true
    deploy_mode = true
```

2. 通过如下命令部署 ConfigMap 到 DorisDisaggregatedCluster 所在的命名空间：

```
kubectl apply -n ${namespace} -f ${feConfigMapName}.yaml
```

其中，${namespace} 为 DorisDisaggregatedCluster 所在的命名空间，${feConfigMapName} 为包含上述配置的文件名称。

3. 更新DorisDisaggregatedCluster 资源使用 ConfigMap。在 DorisDisaggregatedCluster 资源中，通过 feSpec ↪ .configMaps 数组挂载 ConfigMap，示例如下：

```
spec:
  feSpec:
    replicas: 2
    configMaps:
    - name: fe-configmap
```

提示 1. Kubernetes 部署中，启动配置中无需要添加 meta_service_endpoint 以及 cluster_id 配置，Doris-Operator 会自动添加相关信息。
2. Kubernetes 部署中，自定义启动配置时，必须设定 enable_fqdn_mode=true。

访问配置

Doris-Operator 使用 Kubernetes 的 Service 提供 VIP 和负载均衡器的能力，支持以下三种对外暴漏模式：ClusterIP 、NodePort 、LoadBalancer . ####### ClusterIP 模式在 Kubernetes 上默认使用 ClusterIP 访问模式。ClusterIP 访问模式在 Kubernetes 集群内提供了一个内部地址，该地址作为服务在 Kubernetes 内部的。

第 1 步：配置 ClusterIP

默情况下，Doris 在 Kubernetes 上启用 ClusterIP 访问模式，用户无需额外修改即可使用该模式。######## 第 2 步：获取 Service 访问地址部署集群后，通过以下命令可以查看 FE 暴露的 service：

```
kubectl -n doris get svc
```

示例返回结果如下：

```
NAME                              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
    ↪                                AGE
doriscluster-sample-fe-internal   ClusterIP   None           <none>        9030/TCP
    ↪                                14m
doriscluster-sample-fe            ClusterIP   10.1.118.16    <none>        8030/TCP,9020/TCP,9030/
    ↪ TCP,9010/TCP    14m
```

在上述结果中，以 internal 后缀的 Service 仅供 Doris 内部通信使用（如心跳和数据交换）不对外暴漏。不带 internal 后缀的 Service 用于外部访问 FE 服务。

第 3 步：在容器内部访问 Doris

使用如下命令在当前的 Kubernetes 集群中创建一个包含 MySQL 客户端的 Pod：

```
kubectl run mysql-client --image=mysql:5.7 -it --rm --restart=Never --namespace=doris -- /bin/
    ↪ bash
```

在容器内部，可以通过访问不带有 internal 后缀的 Service 名称连接 Doris 集群：

```
mysql -uroot -P9030 -hdoriscluster-sample-fe-service
```

NodePort

若需从 Kubernetes 集群外部访问 Doris，可使用 NodePort 的模式。NodePort 模式支持两种配置方式：静态宿主机端口分配和动态宿主机端口分配。- 动态宿主机端口分配：如果未显示设置端口映射，Kubernetes 会自动分配一个宿主机未被使用的端口（默认范围为 30000-32767 ）。- 静态宿主机端口分配：如果显示指定了端口映射，当宿主机端口未被占用且无冲突的时，Kubernetes 会固定分配该端口。静态分配需要规划端口映射，Doris 默认提供以下端口用于与外部交互：

端口名称	默认端口	端口描述
Query Port	9030	用于通过 MySQL 协议访问 Doris 集群
HTTP Port	8030	FE 上的 http server 端口，用于查看 FE 的信息

第 1 步：配置 FE NodePort

- 动态分配配置：

```
spec:
  feSpec:
    service:
      type: NodePort
```

- 静态分配配置示例：

```
spec:
  feSpec:
    service:
      type: NodePort
      portMaps:
      - nodePort: 31001
        targetPort: 8030
      - nodePort: 31002
        targetPort: 9030
```

第 2 步：获取 Service

集群部署完成后，通过以下命令查看 Service：

```
kubectl get service
```

返回结果如下：

```
NAME                              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
    ↪                                                          AGE
kubernetes                        ClusterIP   10.152.183.1     <none>        443/TCP
    ↪                                                          169d
doriscluster-sample-fe-internal   ClusterIP   None             <none>        9030/TCP
    ↪                                                         2d
doriscluster-sample-fe            NodePort    10.152.183.58    <none>        8030:31041/TCP
    ↪ ,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
```

第 3 步：使用 NodePort 访问 Doris

以 MySQL 连接为例，Doris 的 Query Port 映射到宿主机端口 31545。首先获取到 Kubernetes 集群任一 node 的 IP 地址，例如通过：

```
kubectl get nodes -owide
```

示例返回：

```
NAME   STATUS   ROLES           AGE   VERSION   INTERNAL-IP     EXTERNAL-IP   OS-IMAGE
    ↪ KERNEL-VERSION          CONTAINER-RUNTIME
r60    Ready    control-plane   14d   v1.28.2   192.168.88.60   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
```

```
r61    Ready    <none>          14d   v1.28.2   192.168.88.61   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r62    Ready    <none>          14d   v1.28.2   192.168.88.62   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
r63    Ready    <none>          14d   v1.28.2   192.168.88.63   <none>        CentOS Stream 8
    ↪ 4.18.0-294.el8.x86_64   containerd://1.6.22
```

使用其中任一节点的 IP（如 192.168.88.62），通过以下命令连接 Doris 集群：

```
mysql -h 192.168.88.62 -P 31545 -uroot
```

LoadBalancer

LoadBalancer 模式适用于云平台的 Kubernetes 环境，是由云服务商提供的负载均衡器。######## 第 1 步：配置 LoadBalancer 模式在 feSpec.service 中设置类型为 LoadBalancer，如下所示：

```
spec:
  feSpec:
    service:
      type: LoadBalancer
      annotations:
        service.beta.kubernetes.io/load-balancer-type: "external"
```

第 2 步：获取 Service

在部署集群后，通过以下命令查看 Service：

```
kubectl get service
```

示例返回结果：

```
NAME                              TYPE           CLUSTER-IP       EXTERNAL-IP
    ↪                                                                    PORT(S)
    ↪                                                 AGE
kubernetes                        ClusterIP      10.152.183.1     <none>
    ↪                                                                          443/TCP
    ↪                                                 169d
doriscluster-sample-fe-internal   ClusterIP      None             <none>
    ↪                                                                          9030/TCP
    ↪                                                2d
doriscluster-sample-fe            LoadBalancer   10.152.183.58
    ↪ ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com
    ↪ 8030:31041/TCP,9020:30783/TCP,9030:31545/TCP,9010:31610/TCP   2d
```

第 3 步：使用 LoadBalancer 访问

以 MySQL 连接为例，假设 Query Port 的监听端口为 9030，则可使用如下命令连接 Doris 集群：

```
mysql -h ac4828493dgrftb884g67wg4tb68gyut-1137856348.us-east-1.elb.amazonaws.com -P 9030 -uroot
```

持久化存储

默认部署中，FE 服务使用 Kubernetes 的 EmptyDir 作为元数据存储模式。由于 EmptyDir 模式是非持久化存储模式，服务重启后元数据会丢失。为了保证 FE 元数据在重启后不丢失，需要配置持久化存储。####### 使用存储模板自动生成使用存储模板对日志和元数据进行持久化配置，示例如下：

```
spec:
  feSpec:
    persistentVolumes:
    - persistentVolumeClaimSpec:
      # storageClassName: ${storageclass_name}
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 200Gi
```

使用如上配置部署集群后，Doris Operator 会自动为日志目录（默认为 /opt/apache-doris/fe/log ）以及元数据目录（默认为 /opt/apache-doris/fe/doris-meta ）挂载持久化存储。如果在自定义启动配置中显示指定了日志或元数据目录，Doris Operator 会自动解析并进行挂载。持久化存储采用 StorageClass 模式，可以通过 storageClassName 指定所需的 StorageClass。

自定义挂载点配置

Doris Operator 支持对挂载目录进行个性化存储配置。为日志目录使用自定义存储配置挂载 300Gi 的存储磁盘，为元数据目录使用存储模板挂载 200Gi 的存储磁盘：

```
spec:
  feSpec:
    persistentVolumes:
    - mountPaths:
      - /opt/apache-doris/fe/log
      persistentVolumeClaimSpec:
      # storageClassName: ${storageclass_name}
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 300Gi
    - persistentVolumeClaimSpec:
      # storageClassName: ${storageclass_name}
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 200Gi
```

提示若 mountPaths 数组为空，则表示当前存储配置为模板配置。

不持久化日志

如果不希望将日志持久化，而仅输出到标准输出，则可配置如下：

```
spec:
  feSpec:
    logNotStore: true
```

2.6.1.2.5 配置计算组

存算分离集群中，计算组（Compute Group）负责数据导入并缓存对象存储中的数据以提高查询效率，计算组之间相互隔离。

最简计算组配置

计算组为一组负责相同任务的 BE 集合。在配置 DorisDisaggregatedCluster 资源时，必须为每个计算组设置唯一标识符，唯一标识符也是计算组的名称，一旦设定便无法修改。一个最简单计算组配置包括 3 个组成部分，uniqueId，image，replicas，配置如下：

```
spec:
  computeGroups:
  - uniqueId: ${uniqueId}
    image: ${beImage}
    replicas: 1
```

${beImage} 为部署 BE 服务的镜像地址，请使用 apache doris 官方镜像仓库提供的镜像。${uniqueId} 为计算组的唯一标识也是计算组的名称，匹配规则为[a-zA-Z][0-9a-zA-Z_]+。replicas 为计算组内 BE 服务节点的数量。

配置多计算组

DorisDisaggregatedCluster 资源支持部署多套计算组，每套计算组之间相互独立。以下展示了部署名称为 cg1 和 cg2 两套计算组的配置示例：

```
spec:
  computeGroups:
  - uniqueId: cg1
    image: ${beImage}
    replicas: 3
  - uniqueId: cg2
    image: ${beImage}
    replicas: 2
```

其中，名称为 cg1 的计算组副本数为 3，名称为 cg2 的计算组副本数为 2。${beImage} 表示部署的 BE 服务镜像。尽管计算组之间相互独立，但建议同一存算分离集群中各计算组内 BE 服务所使用的镜像保持一致。

计算资源配置

存算分离默认部署样例中，没有对 BE 服务使用的计算资源做限制。DorisDisaggregatedCluster 使用 Kubernetes 的resources.requests 和 resources.limits 指定 CPU 和内存资源。例如，配置名称为 cg1 的计算组 BE 可使用 8c 8Gi 的资源，配置如下：

```
spec:
  computeGroups:
  - uniqueId: cg1
    requests:
      cpu: 8
      memory: 8Gi
    limits:
      cpu: 8
      memory: 8Gi
```

将上述配置更新到需要部署的DorisDisaggregatedCluster 资源中。

访问配置

默认情况下，计算组不会直接对外提供服务。Doris Operator 在 DorisDisaggregatedCluster 资源中为计算组提供 Service 作为被访问的代理。Service 有三种对外暴漏模式 ClusterIP、NodePort、LoadBalancer。####### ClusterIP 在 Kubernetes 上默认使用 ClusterIP 访问模式。ClusterIP 访问模式在 Kubernetes 集群内提供了一个内部地址，该地址作为服务在 Kubernetes 内部的。

第 1 步：配置使用 ClusterIP 作为 Service 类型

Doris 默认在 Kubernetes 上启用 ClusterIP 访问模式，用户无需额外配置即可使用。######## 第 2 步：获取 Service 访问地址部署集群后，通过以下命令可以查看计算组服务对外暴露的 Service：

```
kubectl -n doris get svc
```

返回结果如下：

```
NAME                                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
    ↪                                AGE
test-disaggregated-cluster-cg1          ClusterIP   10.152.183.154   <none>        9060/TCP
    ↪ ,8040/TCP,9050/TCP,8060/TCP   2d
```

在上述结果中，获取了 namespace 为 doris 下，uniqueId 为 cg1 的对外可使用的 Service 。

NodePort

若需从 Kubernetes 集群外部访问 Doris，可以选择 NodePort 的模式。NodePort 模式提供两种配置方式：静态宿主机端口映射和动态宿主机端口分配。- 动态宿主机端口分配：如果未显示设置端口映射，Kubernetes 会在创建 pod 的时自动分配一个宿主机未被使用的端口（默认范围为 30000-32767 ）。- 静态宿主机端口分配：如果显示指定了端口映射，当宿主机端口未被占用且无冲突的时，Kubernetes 会固定分配该端口。静态分配需要规划端口映射，Doris 提供以下端口用于与外部交互：

端口名称	默认端口	端口描述
Web Server Port	8040	BE 上的 http server 端口，用于查看 BE 的信息

静态分配配置

名称为 cg1 的计算组静态配置 NodePort 访问模式如下：

```
spec:
  computeGroups:
  - uniqueId: cg1
    service:
      type: NodePort
      portMaps:
      - nodePort: 31012
        targetPort: 8040
```

上述配置中，将计算组名称为 cg1 的 BE 监听端口 8040 映射到宿主机的 31012 端口。######## 动态配置名称为 cg1 的计算组动态配置 NodePort 访问模式如下：

```
spec:
  computeGroups:
  - uniqueId: cg1
    service:
      type: NodePort
```

LoadBalancer

LoadBalancer 模式适用于云平台的 Kubernetes 环境，是由云服务商提供的负载均衡器。在 computeGroup.service 中设置类型为 LoadBalancer，如下所示：

```
spec:
  computeGroups:
  - uniqueId: cg1
    service:
      type: LoadBalancer
      annotations:
        service.beta.kubernetes.io/load-balancer-type: "external"
```

自定义启动配置

1. 自定义包含启动信息的 ConfigMap
   默认部署中，每个计算组的 BE 服务均使用镜像内的默认配置文件启动。Doris Operator 使用 Kubernetes 的 ConfigMap 来挂载自定义启动配置文件。以下展示了一个 BE 服务可使用的 ConfigMap 示例：

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: be-configmap
     labels:
       app.kubernetes.io/component: be
   data:
   ```

```
  be.conf: |
    # For jdk 17, this JAVA_OPTS will be used as default JVM options
    JAVA_OPTS_FOR_JDK_17="-Xmx1024m -DlogPath=$LOG_DIR/jni.log -Xlog:gc*:$LOG_DIR/be.gc.log.
        ↪ $CUR_DATE:time,uptime:filecount=10,filesize=50M -Djavax.security.auth.
        ↪ useSubjectCredsOnly=false -Dsun.security.krb5.debug=true -Dsun.java.command=
        ↪ DorisBE -XX:-CriticalJNINatives -XX:+IgnoreUnrecognizedVMOptions --add-opens=
        ↪ java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-
        ↪ UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.
        ↪ base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-opens=
        ↪ java.base/java.nio=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add
        ↪ -opens=java.base/java.util.concurrent=ALL-UNNAMED --add-opens=java.base/java.
        ↪ util.concurrent.atomic=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED
        ↪ --add-opens=java.base/sun.nio.cs=ALL-UNNAMED --add-opens=java.base/sun.security.
        ↪ action=ALL-UNNAMED --add-opens=java.base/sun.util.calendar=ALL-UNNAMED --add-
        ↪ opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED --add-opens=java.
        ↪ management/sun.management=ALL-UNNAMED"
    file_cache_path = [{"path":"/mnt/apache-doris/be/file_cache","total_size":107374182400,"
        ↪ query_limit":107374182400}]
    deploy_mode = cloud
```

存算分离集群 BE 服务的启动配置必须设置 `file_cache_path`，格式请参考存算分离配置 `be.conf` 章节。

2. 部署 ConfigMap
   使用如下命令将自定义启动配置信息的 ConfigMap 部署到 Kubernetes 集群中：

```
kubectl -n ${namespace} -f ${beConfigMapFileName}.yaml
```

`${namespace}` 为 `DorisDisaggregatedCluster` 部署的命名空间，`${beConfigMapFileName}` 为包含自定义 ConfigMap 的文件名称。

3. 更新`DorisDisaggregatedCluster` 资源以 ConfigMap，配置如下：

```
spec:
  computeGroups:
  - uniqueId: cg1
    configMaps:
    - name: be-configmap
      mountPath: "/etc/doris"
```

> 提示启动配置必须挂载到 “/etc/doris” 目录下。

持久化存储配置

默认部署中，BE 服务使用 Kubernetes 的 EmptyDir 作为服务的缓存。`EmptyDir` 模式是非持久化存储模式，服务重启后缓存的数据会丢失相应查询效率会降低。为了保证 BE 服务在重启后缓存数据不丢失、查询效率不降

低，需要对缓存数据进行持久化存储。BE 服务的日志既会输出到标准输出，也会写入启动配置中 LOG_DIR 指定的目录。StreamLoad 模式导入会使用 /opt/apache-doris/be/storage 作为数据的暂存位置，避免服务异常重启后暂存的数据丢失，需要对对应写入位置挂载持久化存储。

持久化存储样例

以下为需要持久化数据挂载持久化存储的配置样例：

```
spec:
  computeGroups:
  - uniqueId: cg1
    persistentVolumes:
    - mountPaths:
      - /opt/apache-doris/be/log
      persistentVolumeClaimSpec:
        # storageClassName: ${storageclass_name}
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 300Gi
    - mountPaths:
      - /opt/apache-doris/be/storage
      persistentVolumeClaimSpec:
        # storageClassName: ${storageclass_name}
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 300Gi
    - persistentVolumeClaimSpec:
        # storageClassName: ${storageclass_name}
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 500Gi
```

上述配置中，日志目录使用自定义的存储配置挂载 300Gi 的存储磁盘，WAL 以及 StreamLoad 导入时使用的目录配置挂载 300Gi 的存储磁盘，而缓存目录则使用存储模板挂载 500Gi 的存储磁盘。

> 提示 - 若 mountPaths 数组为空，则表示当前存储配置为模板配置。当用户在启动配置中配置了 file_cache_path, operator 会自动解析出目录地址进行挂载。- 推荐配置 4 个目录，挂载 4 个持久化存储来发挥云盘的最大效应。

不持久化日志

如果不希望将日志持久化，而仅输出到标准输出，则可配置如下：

```
spec:
  computeGroups:
  - uniqueId: cg1
    logNotStore: true
```

2.6.1.2.6 部署存算分离 Doris 集群

在 Kubernetes 上部署可用存算分离集群分为 4 步：1. 部署前准备，主要包括安装 FoundationDB 集群。2. 部署 Doris Operator。3. 部署 Doris 存算分离集群。4. 创建存储后端。

第 1 步：部署前准备

在 Kubernetes 上部署存算分离集群需要提前部署好 FoundationDB。- （推荐）如果使用机器直接部署，需要确保该机器能够被 Kubernetes 集群上的服务访问。FoundationDB 在机器上直接部署请参考存算分离部署文档中部署前准备阶段的介绍。- 在 Kubernetes 上部署请参考在 Kubernetes 上部署 FoundationDB。

第 2 步：部署 Doris Operator

1. 下发资源定义

```
kubectl create -f https://raw.githubusercontent.com/apache/doris-operator/master/config/crd/bases
    ↪ /crds.yaml
```

如果已经部署过非存算分离集群请用如下命令下发资源定义：

```
kubectl create -f https://raw.githubusercontent.com/apache/doris-operator/master/config/crd/bases
    ↪ /disaggregated.cluster.doris.com_dorisdisaggregatedclusters.yaml
```

2. 部署 Operator 及 RBAC 规则

执行如下命令部署 Doris Operator 及其依赖的 RBAC 规则：

```
kubectl apply -f https://raw.githubusercontent.com/apache/doris-operator/master/config/operator/
    ↪ disaggregated-operator.yaml
```

部署后可通过以下命令检查 Operator Pod 状态：

```
kubectl -n doris get pods
NAME                               READY   STATUS    RESTARTS   AGE
doris-operator-6b97df65c4-xwvw8    1/1     Running   0          19s
```

第 3 步：部署存算分离集群

1. 下载部署样例

从 Doris Operator 仓库下载默认部署样例：

```
curl -O https://raw.githubusercontent.com/apache/doris-operator/master/doc/examples/disaggregated
    ↪ /cluster/ddc-sample.yaml
```

2. 配置 FoundationDB 访问信息

Doris 存算分离版本使用 FDB 存储元数据，在 DorisDisaggregatedCluster 的 spec.metaService.fdb 中提供两种配置方式：- 配置访问地址
若 FoundationDB 部署在 Kubernetes 外部，可直接配置 FoundationDB 的访问地址：

```
  spec:
    metaService:
      fdb:
        address: ${fdbAddress}
```

其中，${fdbAddress} 为 FoundationDB 使用客户端的访问地址。Linux 虚机默认部署的情况下存储在 `/etc/foundationdb/fdb.cluster`，可参考 FoundationDB 对于 [cluster file](https://apple.github.io/foundationdb/administration.html#foundationdb-cluster-file) 的介绍了解详细信息。

- 配置包含访问信息的 ConfigMap
  使用 fdb-kubernetes-operator 部署 FoundationDB，fdb-kubernetes-operator 会在部署的命名空间下生成一个特定的，包含 FoundationDB 访问信息的 ConfigMap。生成的 ConfigMap 名称为部署 FoundationDB 的资源名称加上 “-config”。如何获取 ConfigMap，请参考文档 FoundationDB 在 Kubernetes 上部署中的访问信息获取章节。获取 ConfigMap 的命名空间和名称后，请按照如下格式配置 DorisDisaggregatedCluster 资源：

  ```
  spec:
    metaService:
      fdb:
        configMapNamespaceName:
          name: {foundationdbConfigMapName}
          namespace: {namespace}
  ```

  其 中，$foundationdbConfigMapName`fdb - kubernetes - operator``ConfigMap`${namespace} 为 ConfigMap 所在的命名空间。

3. 配置 DorisDisaggregatedCluster 资源

根据存算分离 Kubernetes 部署文档中：- 元数据配置章节配置 metaService；
- FE 集群配置章节进行 FE 规格配置；
- 计算资源组配置章节进行相关资源组的配置。
配置完成后，使用如下命令部署：

```
kubectl apply -f ddc-sample.yaml
```

部署资源下发后，等待集群自动搭建完成，预期结果如下：

```
kubectl get ddc
NAME                          CLUSTERHEALTH   FEPHASE   CGCOUNT   CGAVAILABLECOUNT
    ↪ CGFULLAVAILABLECOUNT
test-disaggregated-cluster    green           Ready     2         2                 2
```

第 4 步：创建远程存储后端

集群启动成功后，需要通过相应的 SQL 配置，将可用的对象存储作为持久化存储后端（Doris 称之为 Vault）。
####### 1. 获取 FE Service 的访问地址部署集群后，通过以下命令查找可访问 FE 服务的 Service：

```
kubectl get svc
```

示例输出：

```
NAME                                    TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
    ↪                                AGE
test-disaggregated-cluster-fe           ClusterIP   10.96.147.97   <none>        8030/TCP,9020/
    ↪ TCP,9030/TCP,9010/TCP   15m
test-disaggregated-cluster-fe-internal   ClusterIP   None           <none>        9030/TCP
    ↪                                15m
test-disaggregated-cluster-ms           ClusterIP   10.96.169.8    <none>        5000/TCP
    ↪                                15m
test-disaggregated-cluster-cg1          ClusterIP   10.96.47.90    <none>        9060/TCP,8040/
    ↪ TCP,9050/TCP,8060/TCP   14m
test-disaggregated-cluster-cg2          ClusterIP   10.96.50.199   <none>        9060/TCP,8040/
    ↪ TCP,9050/TCP,8060/TCP   14m
```

其中不带 “-internal” 后缀的 Service 为外部访问使用的 Service 。

2. 通过 MySQL 客户端连接

在 Kubernetes 集群中创建一个包含 MySQL Client 的 Pod，并进入 Pod 内部：

```
kubectl run mysql-client --image=mysql:5.7 -it --rm --restart=Never -- /bin/bash
```

在 Pod 内部使用 Service 名称直接连接 Doris 集群：

```
mysql -uroot -P9030 -h test-disaggregated-cluster-fe
```

3. 存储后端创建 (Vault)

通过 SQL 命令创建支持 S3 协议的对象存储作为 Vault。示例如下：

```
 CREATE STORAGE VAULT IF NOT EXISTS s3_vault
     PROPERTIES (
         "type"="S3",
         "s3.endpoint" = "oss-cn-beijing.aliyuncs.com",
         "s3.region" = "bj",
         "s3.bucket" = "bucket",
         "s3.root.path" = "big/data/prefix",
         "s3.access_key" = "your-ak",
         "s3.secret_key" = "your-sk",
         "provider" = "OSS"
     );
```

有关其他存储后端的创建以及各字段详细说明，请参考存算分离文档中的管理 Storage Vault部分。设置默认存储后端，命令如下：

```
SET {vaultName} AS DEFAULT STORAGE VAULT;
```

其中，{vaultName} 为希望使用的 Vault 的名称，比如创建示例中的 `s3_vault`。

2.6.2 云上部署集群

2.6.2.1 Doris on AWS

为了方便大家在 AWS 上快速体验 Doris，提供了 CloudFormation 模版（CFT），允许快速启动和运行集群。使用模板，只需最少的配置，就可以自动配置 AWS 资源，并启动 Doris 集群。

当然，您也可以自行购买 AWS 资源，采用标准的手动方式进行集群部署。

> 目前还不支持存算分离模式编译部署

2.6.2.1.1 什么是 AWS CloudFormation？

CloudFormation 允许用户只用一个步骤就可以创建一个“资源堆栈”。资源是指用户所创建的东西（如 EC2 实例、VPC、子网等），一组这样的资源称为堆栈。用户可以编写一个模板，使用它可以很容易地按照用户的意愿通过一个步骤创建一个资源堆栈。这比手动创建并且配置更快，而且可重复，一致性更好。并且可以将模板放入源代码做版本控制，在任何时候根据需要把它用于任何目的。

2.6.2.1.2 什么是 Doris on AWS CloudFormation？

当前 Doris 提供了 Doris CloudFormation Template，方便用户直接使用这个模板可以在 AWS 上快速创建 Doris 相关版本的集群，以便体验最新的 Doris 功能。

> 注意：
>
> 基于 CloudFormation 构建 Doris 集群的模板，当前仅支持 us-east-1，us-west-1，us-west-2 区域。
>
> Doris on AWS CloudFormation 主要用于测试或者体验，请不要用于生产环境。

2.6.2.1.3 使用前注意

- 确定要部署的 VPC 和 Subnet
- 确定用来登录节点的 key pair
- 部署中会建立 S3 的 VPC Endpoint Interface

2.6.2.1.4 开始部署

1. AWS 控制台上，进入 CloudFormation，点击 Create stack

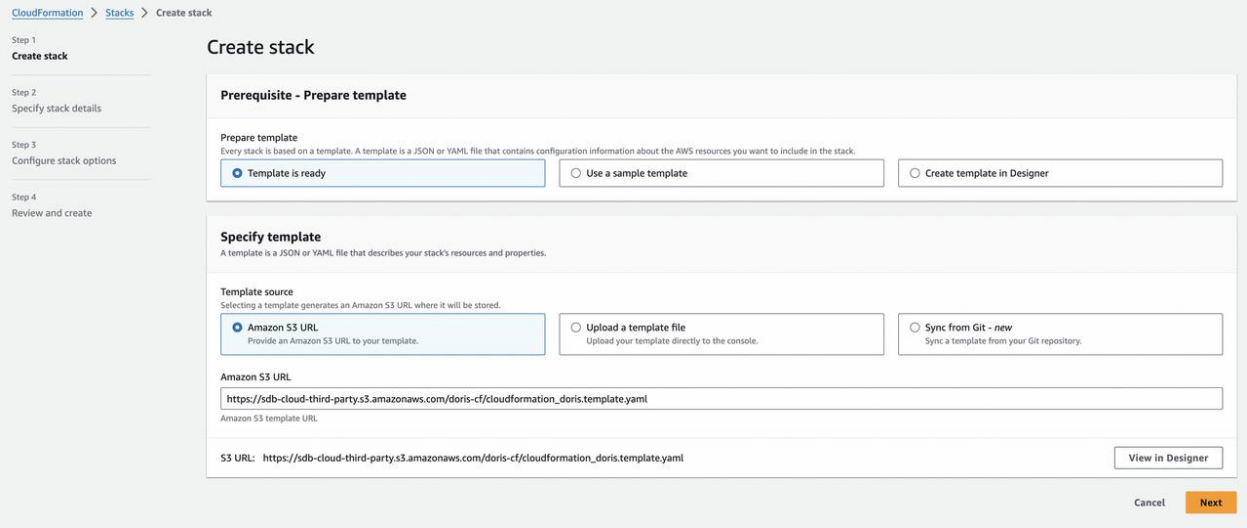


图 8: 开始部署 -AWS 控制台进入 CloudFormation

选用 Amazon S3 URL Template source，填写 Amazon S3 URL 为下面模板链接：

https://sdb-cloud-third-party.s3.amazonaws.com/doris-cf/cloudformation_doris.template.yaml

2. 配置模板的具体参数

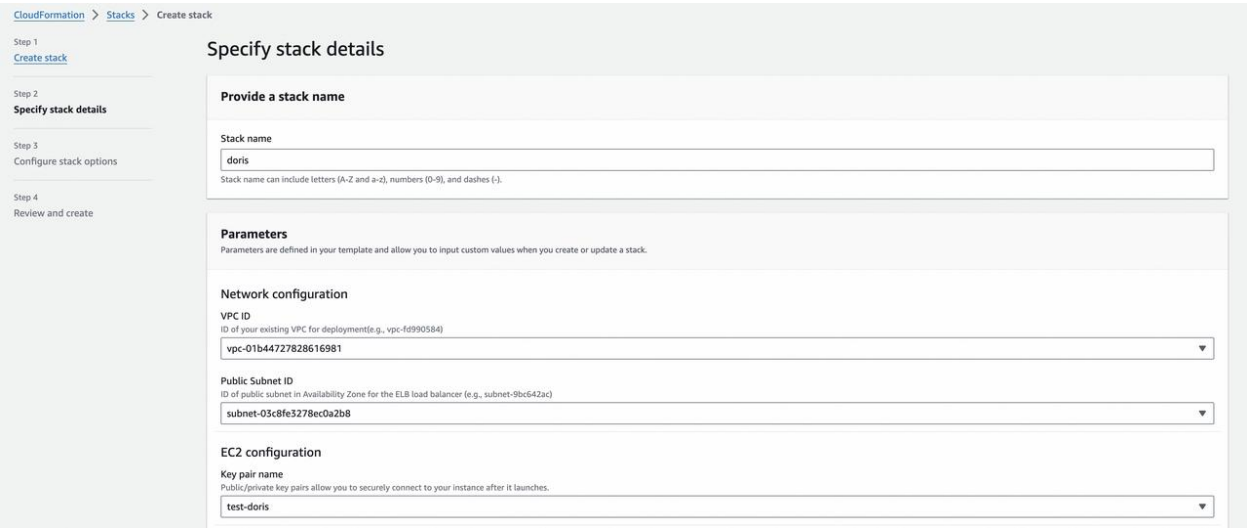


图 9: 配置模板的具体参数

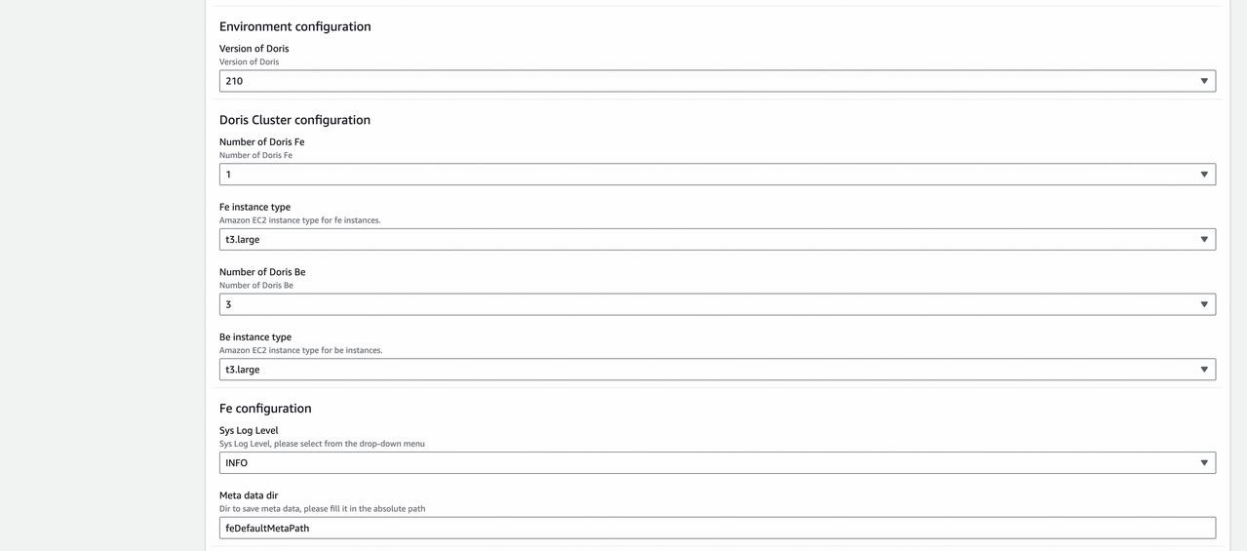


图 10: 配置模板的具体参数

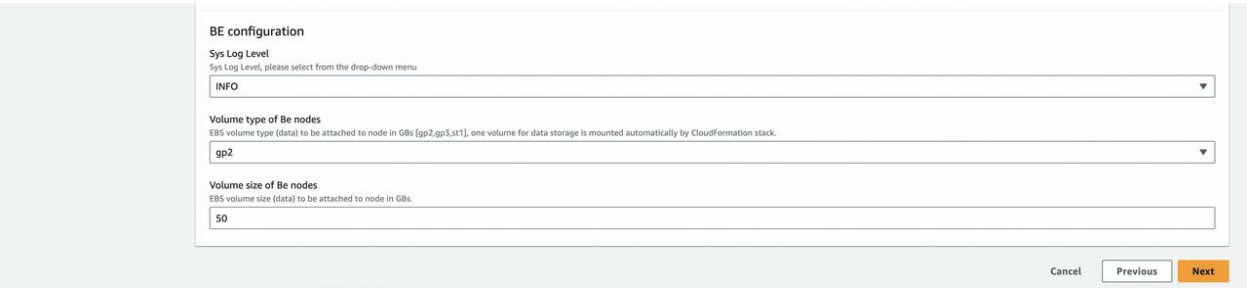


图 11: 配置模板的具体参数

主要参数说明如下：

- VPC ID：要部署到的 VPC
- Subnet ID：要部署的子网
- Key pair name：用来连接部署后的 BE 和 FE 节点的 public/private key pairs
- Version of Doris：选择部署的 Doris 版本，比如 2.1.0、2.0.6 等
- Number of Doris FE：FE 的个数，模板默认只能选择 1 个 FE
- Fe instance type: FE 的节点类型，可以采用默认值
- Number of Doris Be：BE 节点的个数，可以选择 1 个或者 3 个
- Be instance type：BE 的节点类型，可以采用默认值
- Meta data dir：FE 节点的元数据目录，可以采用默认值

- Sys log level：设置系统日志的等级，可以使用默认的 info
- Volume type of Be nodes：BE 节点挂载 EBS 的 volume type，每台节点默认挂载一块磁盘。可以使用默认值
- Volume size of Be nodes: BE 节点挂载 EBS 的大小，单位 GB，可以使用默认值。

2.6.2.1.5 部署后，如何连接数据库

1. 部署成功后的展示如下

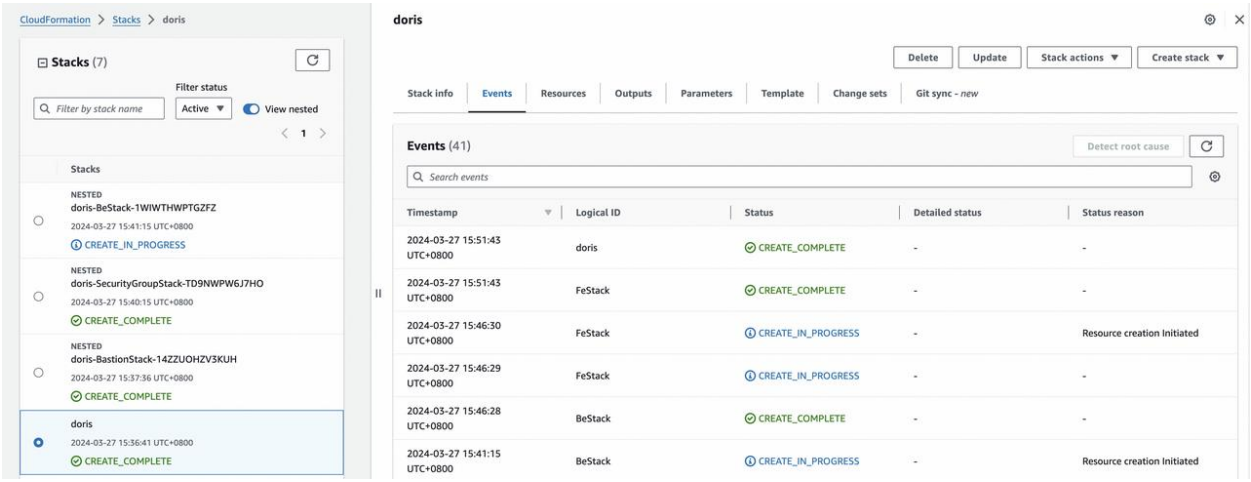


图 12: 如何连接数据库

2. 依次如下面，找到 FE 的连接地址。这个例子中，从 FE Outputs 里，可以查看到地址为 172.16.0.97。

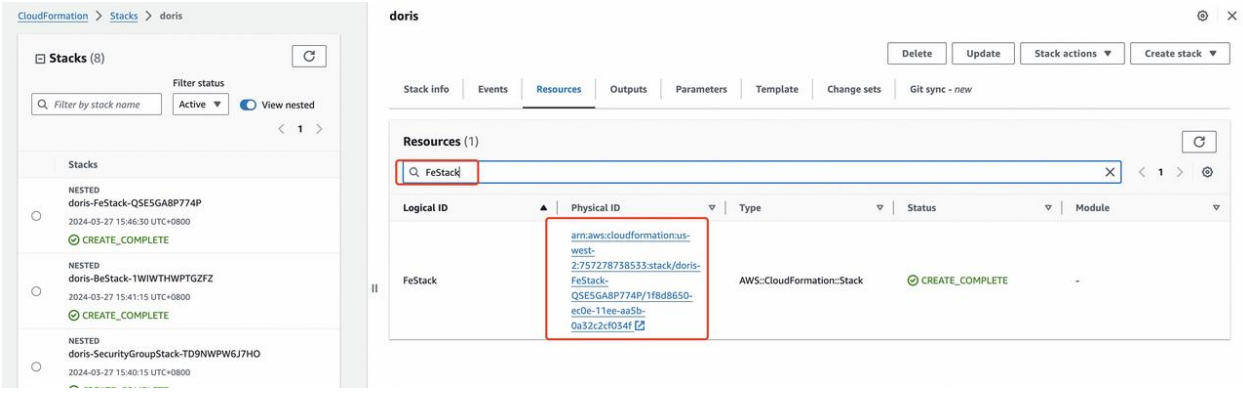


图 13: 找到 FE 的连接地址

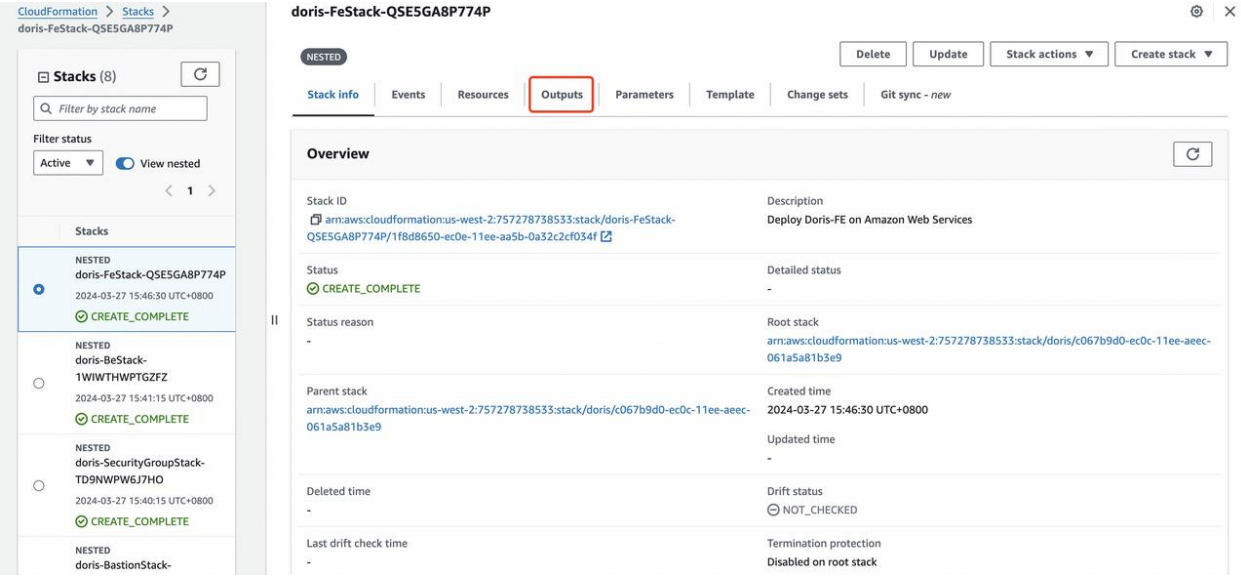


图 14: 找到 FE 的连接地址

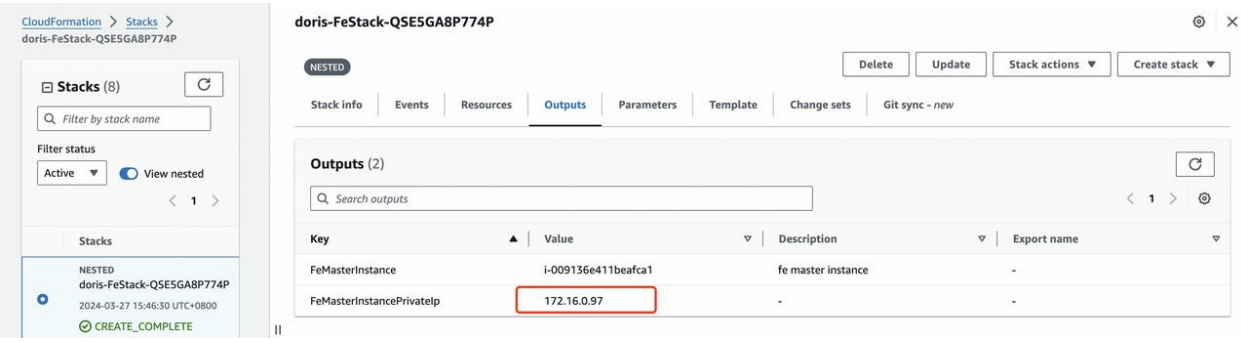


图 15: 找到 FE 的连接地址

3. 连接部署的 Doris Cluster，Doris 的 CloudFormation 部署后的一些默认值：

- FE 的 IP：按照上面步骤 2 获取 FE 的 IP 地址
- FE 的 MySQL 协议端口：9030
- FE 的 HTTP 协议端口：8030
- 默认的 root 密码：空
- 默认的 admin 密码：空

2.7 数据库连接

2.7.1 通过 MySQL 协议连接

Apache Doris 采用 MySQL 网络连接协议，兼容 MySQL 生态的命令行工具、JDBC/ODBC 和各种可视化工具。同时 Apache Doris 也内置了一个简单的 Web UI，方便使用。下面分别介绍如何通过 MySQL Client、MySQL JDBC Connector、DBeaver 和 Doris 内置的 Web UI 来连接 Doris。

2.7.1.1 MySQL Client

从 MySQL 官网 下载 Linux 版 MySQL 客户端。目前 Doris 主要兼容 MySQL 5.7 及以上版本的客户端。

解压下载的 MySQL 客户端，在 `bin/` 目录下可以找到 `mysql` 命令行工具。然后执行下面的命令连接 Doris。

```
mysql -h FE_IP -P FE_QUERY_PORT -u USER_NAME
```

登录后，显示如下。

```
Welcome to the MySQL monitor.  Commands end with ; or `\G`.
Your MySQL connection id is 236
Server version: 5.7.99 Doris version doris-2.0.3-rc06-37d31a5

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

2.7.1.2 MySQL JDBC Connector

请在 MySQL 官方网站下载相应的 JDBC Connector。

连接代码示例如下：

```
String user = "user_name";
String password = "user_password";
String newUrl = "jdbc:mysql://FE_IP:FE_PORT/demo?useUnicode=true&characterEncoding=utf8&
    ↪ useTimezone=true&serverTimezone=Asia/Shanghai&useSSL=false&allowPublicKeyRetrieval=true";
try {
    Connection myCon = DriverManager.getConnection(newUrl, user, password);
    Statement stmt = myCon.createStatement();
    ResultSet result = stmt.executeQuery("show databases");
    ResultSetMetaData metaData = result.getMetaData();
```

```
    int columnCount = metaData.getColumnCount();
    while (result.next()) {
        for (int i = 1; i <= columnCount; i++) {
            System.out.println(result.getObject(i));
        }
    }
} catch (SQLException e) {
    log.error("get JDBC connection exception.", e);
}
```

如果需要在连接时初始换会话变量（Session Variables），可以使用下列格式：

```
jdbc:mysql://FE_IP:FE_PORT/demo?sessionVariables=key1=val1,key2=val2
```

2.7.1.3 DBeaver

创建一个到 Apache Doris 的 MySQL 连接：

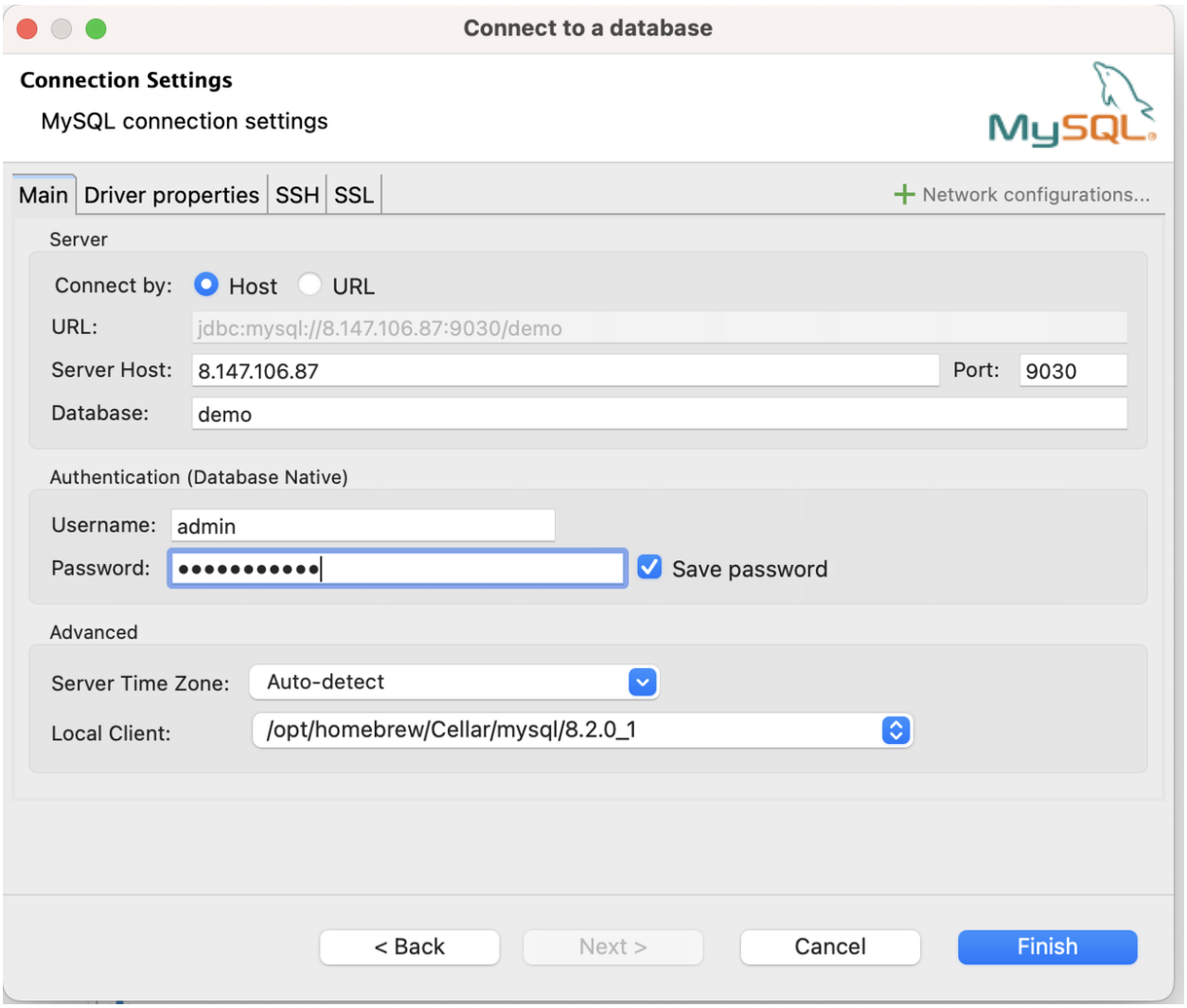


图 16: 创建到 Apache Doris 的 MySQL 连接

在 DBeaver 中进行查询：

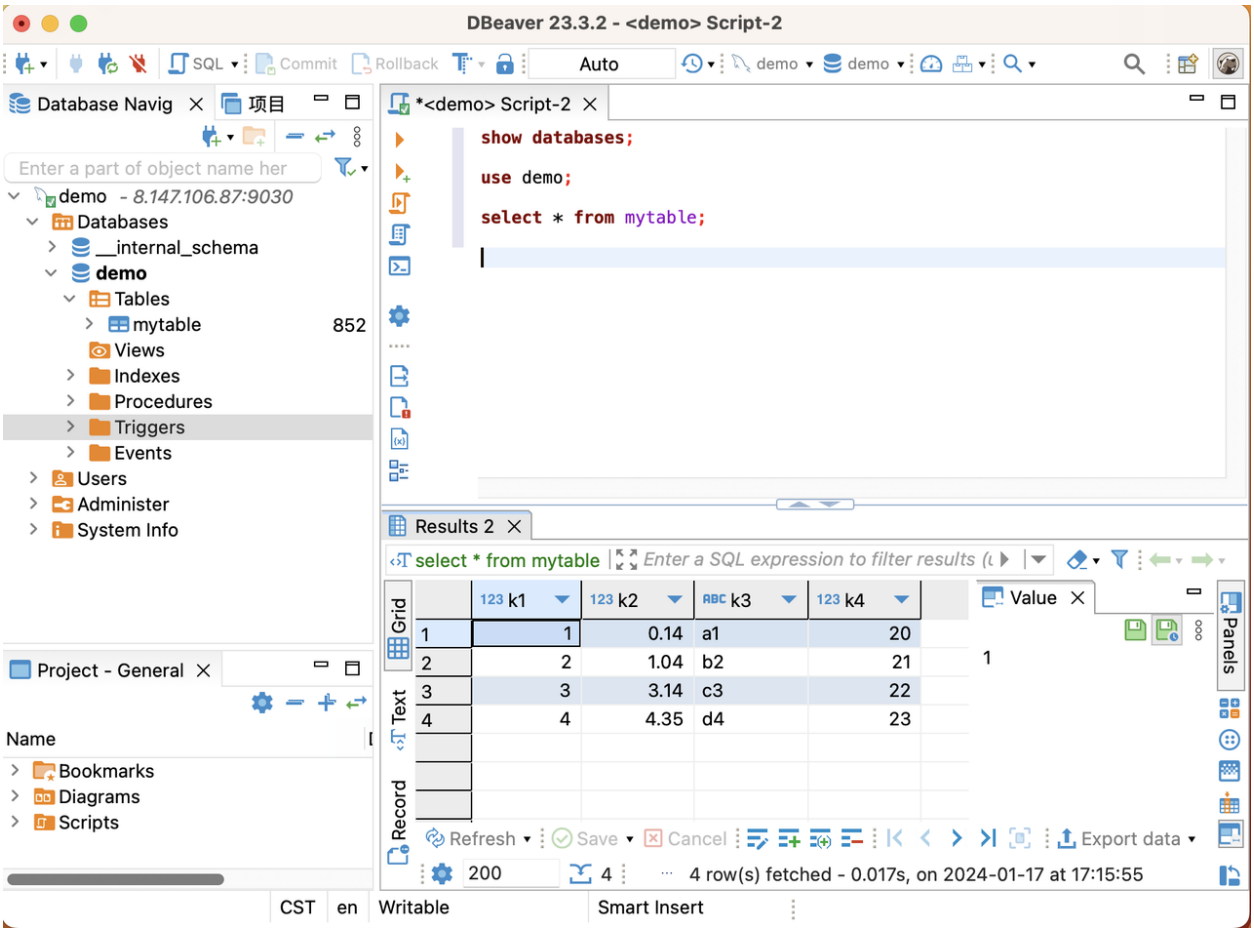


图 17: DBeaver Connect

2.7.1.4 Doris 内置的 Web UI

Doris FE 内置 Web UI。用户无须安装 MySQL 客户端，即可通过内置的 Web UI 进行 SQL 查询和其它相关信息的查看。

在浏览器中输入 http://fe_ip:fe_port, 比如 `http://172.20.63.118:8030`，打开 Doris 内置的 Web 控制台。

内置 Web 控制台，主要供集群 root 账户使用，默认安装后 root 账户密码为空。

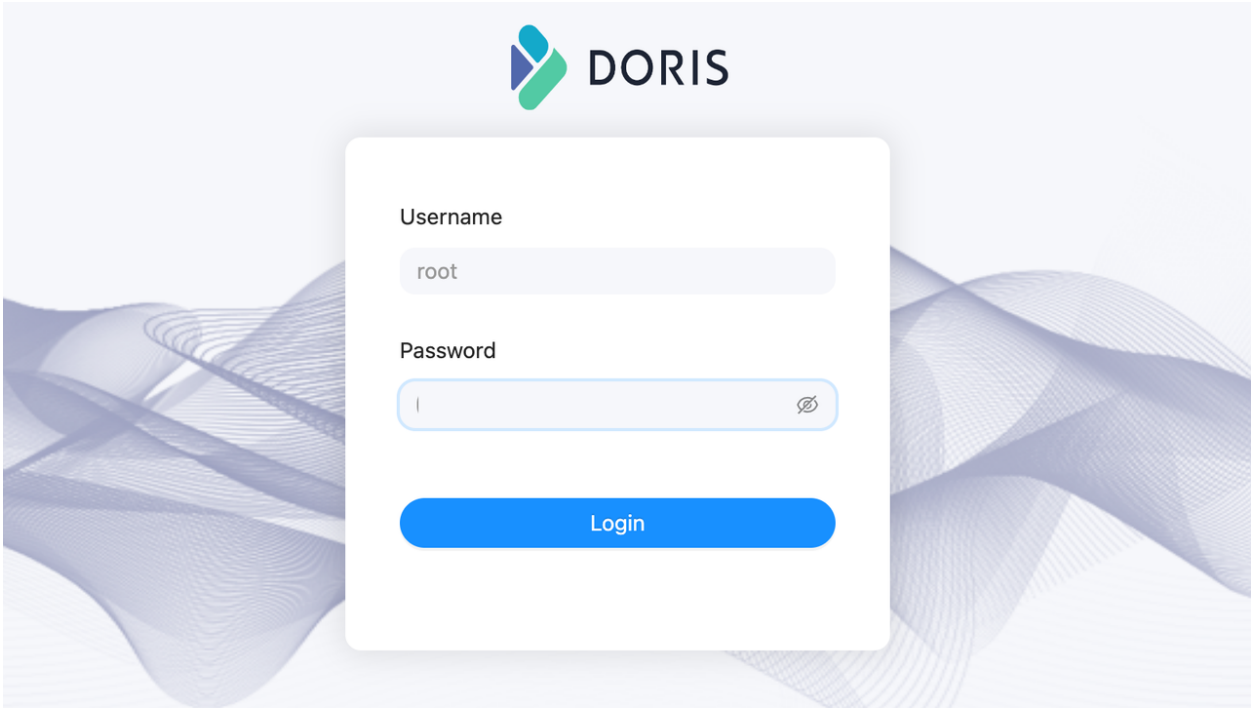


图 18: WebUI

比如，在 Playground 中，执行如下语句，可以完成对 BE 节点的添加。

```
ALTER SYSTEM ADD BACKEND "be_host_ip:heartbeat_service_port";
```

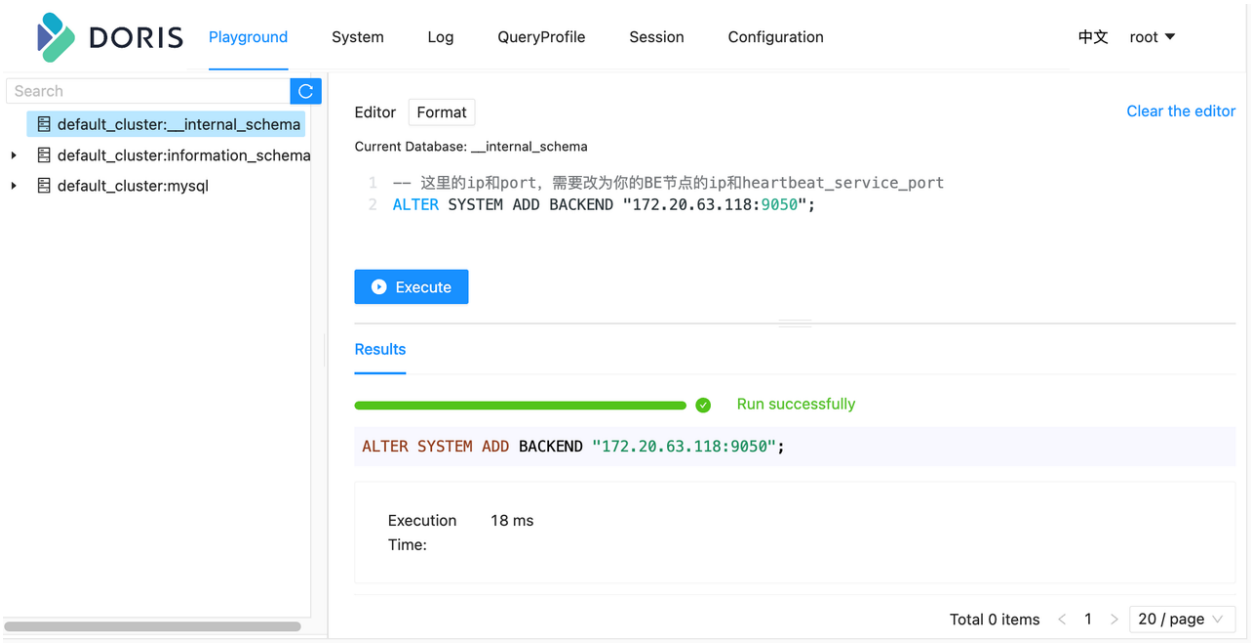


图 19: Playground

Playground 中执行这种和具体数据库/表没有关系的语句，务必在左侧库栏里随意选择一个数据库，才能执行成功，这个限制，稍后会去掉。

当前内置的 Web 控制台，还不能执行 SET 类型的 SQL 语句，所以，在 Web 控制台，当前还不能通过执行 SET PASSWORD FOR ‘user’ = PASSWORD(‘user_password’) ‘类似语句。

2.7.2 基于 Arrow Flight SQL 的高速数据传输链路

自 Doris 2.1 版本后，基于 Arrow Flight SQL 协议实现了高速数据链路，支持多种语言使用 SQL 从 Doris 高速读取大批量数据。Arrow Flight SQL 还提供了通用的 JDBC 驱动，支持与同样遵循 Arrow Flight SQL 协议的数据库无缝交互。部分场景相比 MySQL Client 或 JDBC/ODBC 驱动数据传输方案，性能提升百倍。

2.7.2.1 实现原理

在 Doris 中查询结果以列存格式的 Block 组织。在 2.1 以前版本，可以通过 MySQL Client 或 JDBC/ODBC 驱动传输至目标客户端，需要将行存格式的 Bytes 再反序列化为列存格式。基于 Arrow Flight SQL 构建高速数据传输链路，若目标客户端同样支持 Arrow 列存格式，整体传输过程将完全避免序列化/反序列化操作，彻底消除因此带来时间及性能损耗。

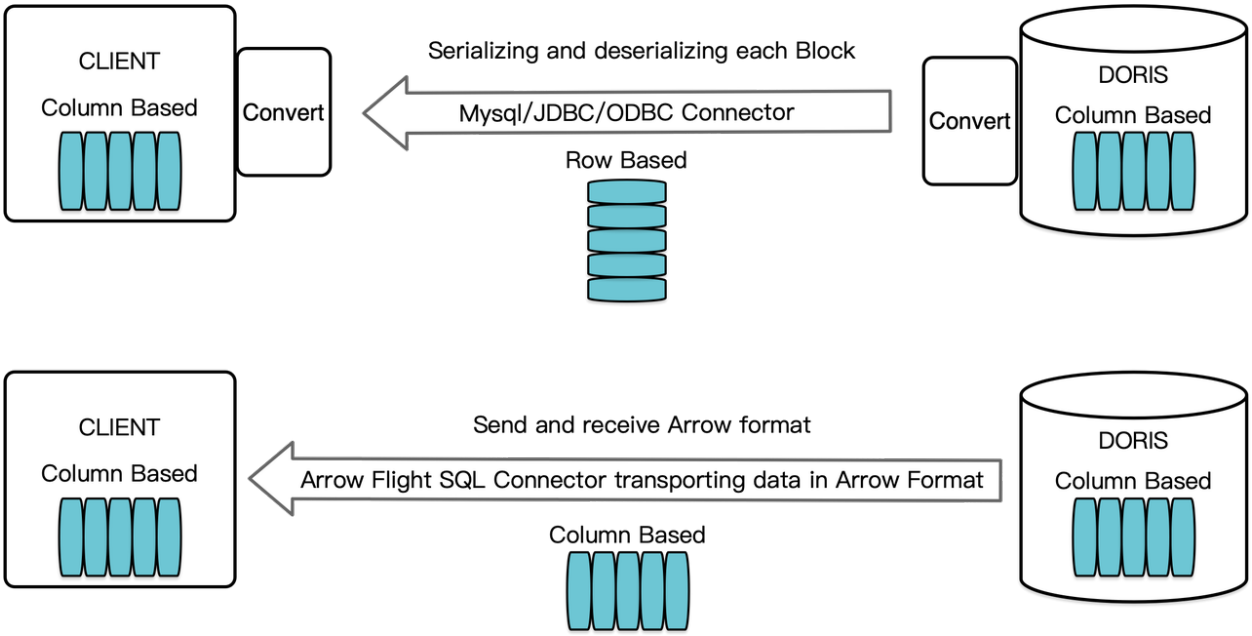


图 20: Arrow_Flight_SQL

安装 Apache Arrow 你可以去官方文档 Apache Arrow 找到详细的安装教程。更多关于 Doris 实现 Arrow Flight 协议的原理可以参考 Doris support Arrow Flight SQL protocol。

2.7.2.2 Python 使用方法

使用 Python 的 ADBC Driver 连接 Doris 实现数据的极速读取，下面的步骤使用 Python（版本 >= 3.9）的 ADBC Driver 执行一系列常见的数据库语法操作，包括 DDL、DML、设置 Session 变量以及 Show 语句等。

2.7.2.2.1 安装 Library

Library 被发布在 PyPI，可通过以下方式简单安装：

```
pip install adbc_driver_manager
pip install adbc_driver_flightsql
```

在代码中 import 以下模块/库来使用已安装的 Library：

```
import adbc_driver_manager
import adbc_driver_flightsql.dbapi as flight_sql

>>> print(adbc_driver_manager.__version__)
1.1.0
>>> print(adbc_driver_flightsql.__version__)
1.1.0
```

2.7.2.2.2 连接 Doris

创建与 Doris Arrow Flight SQL 服务交互的客户端。需提供 Doris FE 的 Host、Arrow Flight Port、登陆用户名以及密码，并进行以下配置。修改 Doris FE 和 BE 的配置参数：

- 修改 fe/conf/fe.conf 中 arrow_flight_sql_port 为一个可用端口，如 8070。
- 修改 be/conf/be.conf 中 arrow_flight_sql_port 为一个可用端口，如 8050。

注：`fe.conf` 与 `be.conf` 中配置的 `arrow_flight_sql_port` 端口号不相同

修改配置并重启集群后，在 fe/log/fe.log 文件中搜索到 `Arrow Flight SQL service is started` 表明 FE 的 Arrow Flight Server 启动成功；在 be/log/be.INFO 文件中搜索到 `Arrow Flight Service bind to host` 表明 BE 的 Arrow Flight Server 启动成功。

假设 Doris 实例中 FE 和 BE 的 Arrow Flight SQL 服务将分别在端口 8070 和 8050 上运行，且 Doris 用户名/密码为“user”/“pass”，那么连接过程如下所示：

```
conn = flight_sql.connect(uri="grpc://{FE_HOST}:{fe.conf:arrow_flight_sql_port}", db_kwargs={
            adbc_driver_manager.DatabaseOptions.USERNAME.value: "user",
            adbc_driver_manager.DatabaseOptions.PASSWORD.value: "pass",
        })
cursor = conn.cursor()
```

连接完成后，可以通过 SQL 使返回的 Cursor 与 Doris 交互，执行例如建表、获取元数据、导入数据、查询等操作。

2.7.2.2.3 建表与获取元数据

将 Query 传递给 cursor.execute () 函数，执行建表与获取元数据操作：

```
cursor.execute("DROP DATABASE IF EXISTS arrow_flight_sql FORCE;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("create database arrow_flight_sql;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("show databases;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("use arrow_flight_sql;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("""CREATE TABLE arrow_flight_sql_test
    (
         k0 INT,
         k1 DOUBLE,
         K2 varchar(32) NULL DEFAULT "" COMMENT "",
         k3 DECIMAL(27,9) DEFAULT "0",
         k4 BIGINT NULL DEFAULT '10',
         k5 DATE,
    )
    DISTRIBUTED BY HASH(k5) BUCKETS 5
    PROPERTIES("replication_num" = "1");""")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("show create table arrow_flight_sql_test;")
print(cursor.fetchallarrow().to_pandas())
```

如果 StatusResult 返回 0，则说明 Query 执行成功 (这样设计的原因是为了兼容 JDBC)。

```
  StatusResult
0            0

  StatusResult
0            0

                    Database
0         __internal_schema
1          arrow_flight_sql
..                      ...
507             udf_auth_db

[508 rows x 1 columns]
```

```
  StatusResult
0            0

  StatusResult
0            0
                   Table                                        Create Table
0  arrow_flight_sql_test  CREATE TABLE `arrow_flight_sql_test` (`\n`  `k0`...
```

2.7.2.2.4 导入数据

执行 INSERT INTO，向所创建表中导入少量测试数据：

```
cursor.execute("""INSERT INTO arrow_flight_sql_test VALUES
        ('0', 0.1, "ID", 0.0001, 9999999999, '2023-10-21'),
        ('1', 0.20, "ID_1", 1.00000001, 0, '2023-10-21'),
        ('2', 3.4, "ID_1", 3.1, 123456, '2023-10-22'),
        ('3', 4, "ID", 4, 4, '2023-10-22'),
        ('4', 122345.54321, "ID", 122345.54321, 5, '2023-10-22');""")
print(cursor.fetchallarrow().to_pandas())
```

如下所示则证明导入成功：

```
  StatusResult
0            0
```

如果需要导入大批量数据到 Doris，可以使用 pydoris 执行 Stream Load 来实现。

2.7.2.2.5 执行查询

接着对上面导入的表进行查询查询，包括聚合、排序、Set Session Variable 等操作。

```
cursor.execute("select * from arrow_flight_sql_test order by k0;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("set exec_mem_limit=2000;")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("show variables like \"%exec_mem_limit%\";")
print(cursor.fetchallarrow().to_pandas())

cursor.execute("select k5, sum(k1), count(1), avg(k3) from arrow_flight_sql_test group by k5;")
print(cursor.fetch_df())
```

结果如下所示：

```
   k0            k1    K2                k3          k4          k5
0   0       0.10000    ID       0.000100000  9999999999  2023-10-21
1   1       0.20000  ID_1       1.000000010           0  2023-10-21
2   2       3.40000  ID_1       3.100000000      123456  2023-10-22
3   3       4.00000    ID       4.000000000           4  2023-10-22
4   4  122345.54321    ID  122345.543210000           5  2023-10-22

[5 rows x 6 columns]

  StatusResult
0            0

    Variable_name Value Default_Value Changed
0  exec_mem_limit  2000    2147483648       1

           k5  Nullable(Float64)_1  Int64_2 Nullable(Decimal(38, 9))_3
0  2023-10-22         122352.94321        3             40784.214403333
1  2023-10-21              0.30000        2                 0.500050005

[2 rows x 5 columns]
```

注意：fetch 查询结果需要使用 `cursor.fetchallarrow()` 返回 arrow 格式，或使用 `cursor.fetch_df()` 直接返回 pandas dataframe，这将保持数据的列存格式。不能使用 `cursor.fetchall()`，否则会将列存格式的数据转回行存，这和使用 mysql-client 没有本质区别，甚至由于在 client 侧多了一次列转行的操作，可能比 mysql-client 还慢。

2.7.2.2.6 完整代码

```python

### step 1, library is released on PyPI and can be easily installed.
### pip install adbc_driver_manager
### pip install adbc_driver_flightsql
import adbc_driver_manager
import adbc_driver_flightsql.dbapi as flight_sql

### step 2, create a client that interacts with the Doris Arrow Flight SQL service.
### Modify arrow_flight_sql_port in fe/conf/fe.conf to an available port, such as 8070.
### Modify arrow_flight_sql_port in be/conf/be.conf to an available port, such as 8050.
conn = flight_sql.connect(uri="grpc://{FE_HOST}:{fe.conf:arrow_flight_sql_port}", db_kwargs={
            adbc_driver_manager.DatabaseOptions.USERNAME.value: "root",
            adbc_driver_manager.DatabaseOptions.PASSWORD.value: "",
        })
cursor = conn.cursor()

### interacting with Doris via SQL using Cursor
```

```
def execute(sql):
    print("`\n`### execute query: ###`\n` " + sql)
    cursor.execute(sql)
    print("### result: ###")
    print(cursor.fetchallarrow().to_pandas())

### step3, execute DDL statements, create database/table, show stmt.
execute("DROP DATABASE IF EXISTS arrow_flight_sql FORCE;")
execute("show databases;")
execute("create database arrow_flight_sql;")
execute("show databases;")
execute("use arrow_flight_sql;")
execute("""CREATE TABLE arrow_flight_sql_test
    (
         k0 INT,
         k1 DOUBLE,
         K2 varchar(32) NULL DEFAULT "" COMMENT "",
         k3 DECIMAL(27,9) DEFAULT "0",
         k4 BIGINT NULL DEFAULT '10',
         k5 DATE,
    )
    DISTRIBUTED BY HASH(k5) BUCKETS 5
    PROPERTIES("replication_num" = "1");""")
execute("show create table arrow_flight_sql_test;")


### step4, insert into
execute("""INSERT INTO arrow_flight_sql_test VALUES
        ('0', 0.1, "ID", 0.0001, 9999999999, '2023-10-21'),
        ('1', 0.20, "ID_1", 1.00000001, 0, '2023-10-21'),
        ('2', 3.4, "ID_1", 3.1, 123456, '2023-10-22'),
        ('3', 4, "ID", 4, 4, '2023-10-22'),
        ('4', 122345.54321, "ID", 122345.54321, 5, '2023-10-22');""")


### step5, execute queries, aggregation, sort, set session variable
execute("select * from arrow_flight_sql_test order by k0;")
execute("set exec_mem_limit=2000;")
execute("show variables like \"%exec_mem_limit%\";")
execute("select k5, sum(k1), count(1), avg(k3) from arrow_flight_sql_test group by k5;")

### step6, close cursor
cursor.close()
```

2.7.2.3 JDBC Connector with Arrow Flight SQL

Arrow Flight SQL 协议的开源 JDBC 驱动兼容标准的 JDBC API，可用于大多数 BI 工具通过 JDBC 访问 Doris，并支持高速传输 Apache Arrow 数据。使用方法与通过 MySQL 协议的 JDBC 驱动连接 Doris 类似，只需将链接 URL 中的 jdbc:mysql 协议换成 jdbc:arrow-flight-sql 协议，查询返回的结果依然是 JDBC 的 ResultSet 数据结构。

POM dependency:

```
<properties>
    <arrow.version>17.0.0</arrow.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.apache.arrow</groupId>
        <artifactId>flight-sql-jdbc-core</artifactId>
        <version>${arrow.version}</version>
    </dependency>
</dependencies>
```

注意：使用 Java 9 或更高版本时，必须通过在 Java 命令中添加 `--add-opens=java.base/java.nio=ALL-UNNAMED` 来暴露一些 JDK 内部结构，否则，您可能会看到一些错误，如 `module java.base does not "opens java.nio" to unnamed module` 或者 `module java.base does not "opens java.nio" to org.apache.arrow.memory.core` 或者 `java.lang.NoClassDefFoundError: Could not initialize class org.apache.arrow.memory.util.MemoryUtil (Internal; Prepare)`

```
### Directly on the command line
$ java --add-opens=java.base/java.nio=ALL-UNNAMED -jar ...
### Indirectly via environment variables
$ env _JAVA_OPTIONS="--add-opens=java.base/java.nio=ALL-UNNAMED" java -jar ...
```

如果在 IntelliJ IDEA 中调试，需要在 `Run/Debug Configurations` 的 `Build and run` 中增加 `--add-opens=java.base/java.nio=ALL-UNNAMED`，参照下面的图片：

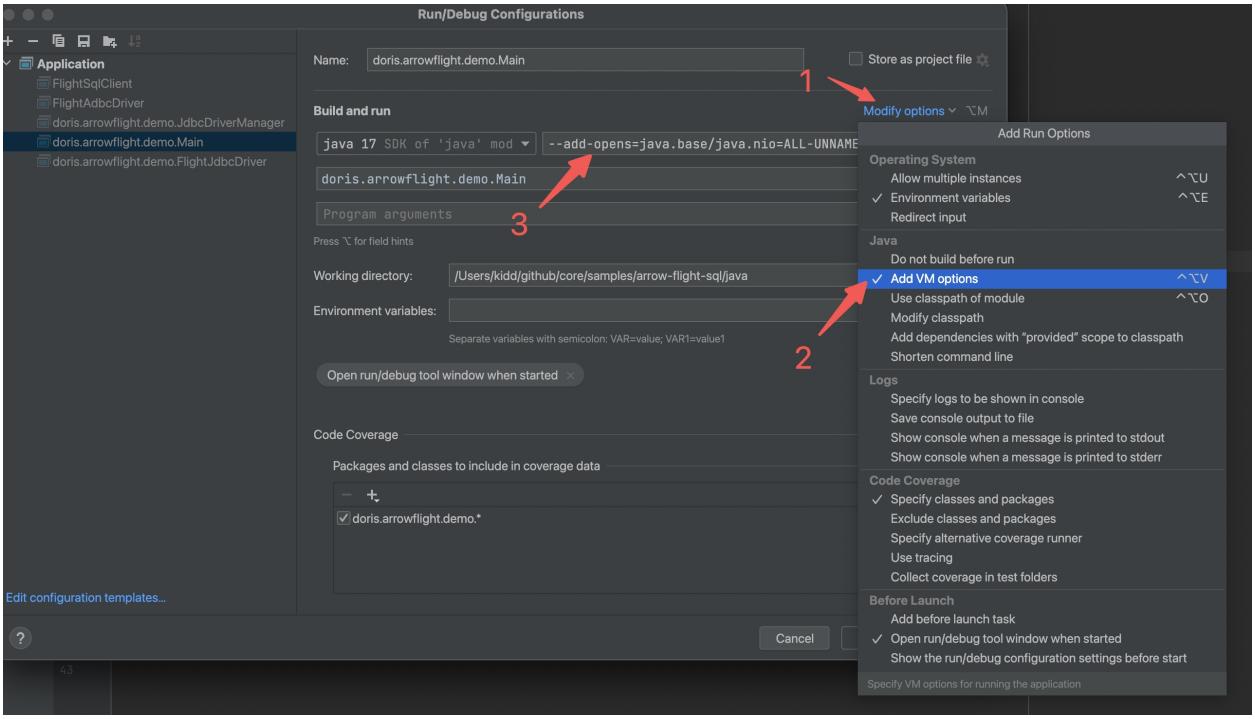


图 21: arrow-flight-sql-IntelliJ

连接代码示例如下：

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

Class.forName("org.apache.arrow.driver.jdbc.ArrowFlightJdbcDriver");
String DB_URL = "jdbc:arrow-flight-sql://{FE_HOST}:{fe.conf:arrow_flight_sql_port}?
    ↪ useServerPrepStmts=false"
        + "&cachePrepStmts=true&useSSL=false&useEncryption=false";
String USER = "root";
String PASS = "";

Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
Statement stmt = conn.createStatement();
ResultSet resultSet = stmt.executeQuery("select * from information_schema.tables;");
while (resultSet.next()) {
    System.out.println(resultSet.toString());
}

resultSet.close();
stmt.close();
```

```
conn.close();
```

2.7.2.4 Java 使用方法

除了使用 JDBC，与 Python 类似，Java 也可以创建 Driver 读取 Doris 并返回 Arrow 格式的数据，下面分别是使用 AdbcDriver 和 JdbcDriver 连接 Doris Arrow Flight Server。

POM dependency:

```
<properties>
    <adbc.version>0.15.0</adbc.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.apache.arrow.adbc</groupId>
        <artifactId>adbc-driver-jdbc</artifactId>
        <version>${adbc.version}</version>
    </dependency>
    <dependency>
        <groupId>org.apache.arrow.adbc</groupId>
        <artifactId>adbc-core</artifactId>
        <version>${adbc.version}</version>
    </dependency>
    <dependency>
        <groupId>org.apache.arrow.adbc</groupId>
        <artifactId>adbc-driver-manager</artifactId>
        <version>${adbc.version}</version>
    </dependency>
    <dependency>
        <groupId>org.apache.arrow.adbc</groupId>
        <artifactId>adbc-sql</artifactId>
        <version>${adbc.version}</version>
    </dependency>
    <dependency>
        <groupId>org.apache.arrow.adbc</groupId>
        <artifactId>adbc-driver-flight-sql</artifactId>
        <version>${adbc.version}</version>
    </dependency>
</dependencies>
```

2.7.2.4.1 ADBC Driver

连接代码示例如下：

```
// 1. new driver
```

```
final BufferAllocator allocator = new RootAllocator();
FlightSqlDriver driver = new FlightSqlDriver(allocator);
Map<String, Object> parameters = new HashMap<>();
AdbcDriver.PARAM_URI.set(parameters, Location.forGrpcInsecure("{FE_HOST}", {fe.conf:arrow_flight_
    ↪ sql_port}).getUri().toString());
AdbcDriver.PARAM_USERNAME.set(parameters, "root");
AdbcDriver.PARAM_PASSWORD.set(parameters, "");
AdbcDatabase adbcDatabase = driver.open(parameters);

// 2. new connection
AdbcConnection connection = adbcDatabase.connect();
AdbcStatement stmt = connection.createStatement();

// 3. execute query
stmt.setSqlQuery("select * from information_schema.tables;");
QueryResult queryResult = stmt.executeQuery();
ArrowReader reader = queryResult.getReader();

// 4. load result
List<String> result = new ArrayList<>();
while (reader.loadNextBatch()) {
    VectorSchemaRoot root = reader.getVectorSchemaRoot();
    String tsvString = root.contentToTSVString();
    result.add(tsvString);
}
System.out.printf("batchs %d`\n`", result.size());

// 5. close
reader.close();
queryResult.close();
stmt.close();
connection.close();
```

2.7.2.4.2 JDBC Driver

连接代码示例如下：

```
final Map<String, Object> parameters = new HashMap<>();
AdbcDriver.PARAM_URI.set(
        parameters,"jdbc:arrow-flight-sql://{FE_HOST}:{fe.conf:arrow_flight_sql_port}?
            ↪ useServerPrepStmts=false&cachePrepStmts=true&useSSL=false&useEncryption=false");
AdbcDriver.PARAM_USERNAME.set(parameters, "root");
AdbcDriver.PARAM_PASSWORD.set(parameters, "");
try (
        BufferAllocator allocator = new RootAllocator();
```

```
        AdbcDatabase db = new JdbcDriver(allocator).open(parameters);
        AdbcConnection connection = db.connect();
        AdbcStatement stmt = connection.createStatement()
) {
    stmt.setSqlQuery("select * from information_schema.tables;");
    AdbcStatement.QueryResult queryResult = stmt.executeQuery();
    ArrowReader reader = queryResult.getReader();
    List<String> result = new ArrayList<>();
    while (reader.loadNextBatch()) {
        VectorSchemaRoot root = reader.getVectorSchemaRoot();
        String tsvString = root.contentToTSVString();
        result.add(tsvString);
    }
    long etime = System.currentTimeMillis();
    System.out.printf("batchs %d`\n`", result.size());

    reader.close();
    queryResult.close();
    stmt.close();
}  catch (Exception e) {
    e.printStackTrace();
}
```

2.7.2.4.3 JDBC 和 Java 连接方式的选择

JDBC/Java Arrow Flight SQL Sample 是 JDBC/Java 使用 Arrow FLight SQL 的 demo，你可以使用它来测试向 Arrow Flight Server 发送查询的多种连接方法，帮助你了解如何使用 Arrow FLight SQL 并测试性能。预期的执行结果见 Add Arrow Flight Sql demo for Java。

对比传统的 `jdbc:mysql` 连接方式，Jdbc 和 Java 的 Arrow Flight SQL 连接方式的性能测试见 Section 6.2 of GitHub Issue，这里基于测试结论给出一些使用建议。

1. 上述三种 Java Arrow Flight SQL 连接方式的选择上，如果后续的数据分析将基于行存的数据格式，那么使用 jdbc:arrow-flight-sql，这将返回 JDBC ResultSet 格式的数据；如果后续的数据分析可以基于 Arrow 格式或其他列存数据格式，那么使用 Flight AdbcDriver 或 Flight JdbcDriver 直接返回 Arrow 格式的数据，这将避免行列转换，并可利用 Arrow 的特性加速数据解析。
2. 无论解析 JDBC ResultSet 还是 Arrow 格式的数据，所耗费的时间都大于读取数据的耗时，如果你那里使用 Arrow Flight SQL 的性能不符合预期，和 `jdbc:mysql://` 相比提升有限，不妨分析下是否解析数据耗时太长。
3. 对所有连接方式而言，JDK 17 都比 JDK 1.8 读取数据的速度更快。
4. 当读取数据量非常大时，使用 Arrow Flight SQL 将比 `jdbc:mysql://` 使用更少的内存，所以如果你受内存不足困扰，同样可以尝试下 Arrow Flight SQL。
5. 除了上述三种连接方式，还可以使用原生的 FlightClient 连接 Arrow Flight Server，可以更加灵活的并行读取多个 Endpoints。Flight AdbcDriver 也是基于 FlightClient 创建的链接，相较于直接使用 FlightClient 更简单。

2.7.2.5 与其他大数据组件交互

2.7.2.5.1 Spark & Flink

Arrow Flight 官方目前没有支持 Spark 和 Flink 的计划（见 GitHub Issue），Doris Spark Connector 和 Doris Flink Connector 自 24.0.0 开始支持通过 Arrow Flight SQL 访问 Doris，预期能提升数倍读取性能。

社区之前参考开源的 Spark-Flight-Connector，在 Spark 中使用 FlightClient 连接 Doris 测试，发现 Arrow 与 Doris Block 之间数据格式转换的速度更快，是 CSV 格式与 Doris Block 之间转换速度的 10 倍，而且对 Map，Array 等复杂类型的支持更好，这是因为 Arrow 数据格式的压缩率高，传输时网络开销小。不过目前 Doris Arrow Flight 还没有实现多节点并行读取，仍是将查询结果汇总到一台 BE 节点后返回，对简单的批量导出数据而言，性能可能没有 Doris Spark Connector 快，后者支持 Tablet 级别的并行读取。如果你希望在 Spark 使用 Arrow Flight SQL 连接 Doris，可以参考开源的 Spark-Flight-Connector 和 Dremio-Flight-Connector 自行实现。

2.7.2.5.2 支持 BI 工具

自 Doris v2.1.8 开始，支持 DBeaver 等 BI 工具使用 `arrow-flight-sql` 协议连接 Doris。DBeaver 使用 `arrow-flight-sql` Driver 连接 Doris 的方法参考：how-to-use-jdbc-driver-with-dbeaver-client，client-applications/clients/dbeaver/。

2.7.2.6 扩展应用

2.7.2.6.1 多 BE 并行返回结果

Doris 默认会将一个查询在所有 BE 节点上的结果汇总聚合到一个 BE 节点上，在 Mysql/JDBC 查询中 FE 会向这个汇总数据的节点请求查询结果，在 Arrow Flight SQL 查询中 FE 会将这个节点的 IP/Port 包装在 Endpoint 中返回给 ADBC Client，ADBC Client 会请求这个 Endpoint 对应的 BE 节点拉取数据。

如果查询只是简单的 Select 从 Doris 拉取数据，没有 Join、Sort、窗口函数等有数据 Shuffle 行为的算子，可以将查询按照 Tablet 粒度拆分，现在 Doris Spark/Flink Connector 就是用的这个方法实现并行读取数据，分为两个步骤：1. 执行 `explain sql`，FE 返回的查询计划中 ScanOperator 包含 Scan 的所有 Tablet ID List。2. 依据上面的 Tablet ID List 将原始 SQL 拆分为多个 SQL，每个 SQL 只读取部分 Tablet，用法类似 `SELECT * FROM t1 TABLET(10001,10002) limit 1000;`，拆分后的多个 SQL 可以并行执行。参考 Support select table sample。

如果查询最外层是聚合，SQL 类似 `select k1, sum(k2)from xxx group by k1`，Doris v3.0.4 版本后，执行 `set enable_parallel_result_sink=true;` 后允许一个查询的每个 BE 节点独立返回查询结果，ADBC Client 收到 FE 返回的 Endpoint 列表后并行从多个 BE 节点拉取结果。不过注意当聚合结果很小时，多 BE 返回会增加 RPC 的压力。具体实现参考 support parallel result sink。理论上除了最外层是排序的查询，其他查询都可以支持每个 BE 节点并行返回结果，不过暂时没有这方便的需求，没有更进一步实现。

2.7.2.6.2 多 BE 共享同一个可供集群外部访问的 IP

如果存在一个 Doris 集群，它的 FE 节点可以被集群外部访问，它的所有 BE 节点只可以被集群内部访问。这在使用 Mysql Client 和 JDBC 连接 Doris 执行查询是没问题的，查询结果将由 Doris FE 节点返回。但使用 Arrow Flight SQL 连接 Doris 无法执行查询，因为 ADBC Client 需要连接 Doris BE 节点拉取查询结果，但 Doris BE 节点不允许被集群外部访问。

在生产环境中，很多时候不方便将 Doris BE 节点暴露到集群外。但可以为所有 Doris BE 节点增加了一层反向代理（比如 Nginx），集群外部的 Client 连接 Nginx 时会随机路由到一台 Doris BE 节点上。默认情况下，Arrow Flight

SQL 查询结果会随机保存在一台 Doris BE 节点上，如果和 Nginx 随机路由的 Doris BE 节点不同，需要在 Doris BE 节点内部做一次数据转发。

自 Doris v2.1.8 开始，你可以在所有 Doris BE 节点的 be.conf 中将 public_host 和 arrow_flight_sql_proxy_port 配置成多 Doris BE 节点共享的可供集群外部访问的 IP 和端口，查询结果可以正确转发后返回 ADBC Client。

```
public_host={nginx ip}
arrow_flight_sql_proxy_port={nginx port}
```

2.7.2.7 常见问题

1. Q: 报错 connection error: desc = "transport: Error while dialing: dial tcp <ip:arrow_flight_port>: i/o timeout"。

   A: 如果报错信息中的 <ip:arrow_flight_port> 是 Doris FE 节点的 IP 和 arrow-flight-prot，

   首先检查 Doris FE 节点的 arrow-flight-server 是否正常启动，在 fe/log/fe.log 文件中搜索到 `Arrow Flight SQL service is started` 表明 FE 的 Arrow Flight Server 启动成功。

   若 Doris FE 节点的 arrow-flight-server 正常启动，，检查 Client 所在机器能否 `ping` 通报错信息 `<ip:arrow_flight_port>` 中的 IP，若无法 `ping` 通，需要为 Doris FE 节点开通一个可供外部访问的 IP，并重新部署集群。

   A: 如果报错信息中的 <ip:arrow_flight_port> 是 Doris BE 节点的 IP 和 arrow-flight-prot。

   首先检查 Doris BE 节点的 arrow-flight-server 是否正常启动，在 be/log/be.INFO 文件中搜索到 `Arrow Flight Service bind to host` 表明 BE 的 Arrow Flight Server 启动成功。

   若 Doris BE 节点的 arrow-flight-server 正常启动，检查 Client 所在机器能否 `ping` 通报错信息 `<ip:arrow_flight_port>` 中的 IP，若无法 `ping` 通，若已知 Doris BE 节点处于无法被外部访问的内网，下面两个方法：

   - 考虑为每个 Doris BE 节点开通一个可供外部访问的 IP，自 Doris v2.1.8 开始，你可以在这个 Doris BE 节点的 `be.conf` 中将 `public_host` 配置成这个 IP，同理将所有 Doris BE 节点的 `public_host` 配置成对应 BE 节点可被 Client 访问的 IP。

   - 参考上文 [多 BE 共享同一个可供集群外部访问的 IP] 章节，可以为所有 Doris BE 节点增加了一层反向代理。

   若不清楚 Doris BE 是否完全处于内网，检查 Client 所在机器与 Doris BE 节点所在机器的其他 IP 之间的连通性，在 Doris BE 节点所在机器执行 `ifconfig` 返回当前机器所有的 IP，其中一个 IP 应该和 `<ip:arrow_flight_port>` 中的 IP 相同，并且和 `show backends` 打印的这个 Doris BE 节点的 IP 相同，依次 `ping` `ifconfig` 返回的其他 IP，若 Doris BE 节点存在可以被 Client 访问的 IP，参考上文同样将这个 IP 配置为 `public_host`。若 Doris BE 节点所有的 IP 均无法被 Client 访问，那么 Doris BE 节点完全处于内网。

2. Q：使用 JDBC 或 JAVA 连接 Arrow Flight SQL 时报错 `module java.base does not "opens java.nio" to unnamed module` 或者 `module java.base does not "opens java.nio" to org.apache.arrow.memory.core` 或者 `java.lang.NoClassDefFoundError: Could not initialize class org.apache.arrow.memory.util.MemoryUtil (Internal; Prepare)`

   A：首先检查 fe/conf/fe.conf 中 JAVA_OPTS_FOR_JDK_17 是否包含 `--add-opens=java.base/java.nio=ALL-UNNAMED`，若没有则添加。然后参考上文JDBC Connector with Arrow Flight SQL 中的注意事项在 Java 命令中添加 `--add-opens=java.base/java.nio=ALL-UNNAMED`，如果在 IntelliJ IDEA 中调试，需要在 Run/Debug Configurations 的 Build and run 中增加 `--add-opens=java.base/java.nio=ALL-UNNAMED`。

3. Q: ARM 环境报错 `get flight info statement failed, arrow flight schema timeout, TimeoutException: Waited 5000 milliseconds for io.grpc.stub.Client`。

   A: 如果 Linux 内核版本 <= 4.19.90，需要升级到 4.19.279 及以上，或者在低版本 Linux 内核的环境中重新编译 Doris BE，具体编译方法参考文档

   问题原因：这是因为老版本 Linux 内核和 Arrow 存在兼容性问题，`cpp: arrow::RecordBatch::MakeEmpty()` 构造 Arrow Record Batch 时会卡住，导致 Doris BE 的 Arrow Flight Server 在 5000ms 内没有回应 Doris FE 的 Arrow Flight Server 的 RPC 请求，导致 FE 给 Client 返回 rpc timeout failed。Spark 和 Flink 读取 Doris 时也是将查询结果转换成 Arrow Record Batch 后返回，所以也存在同样的问题。

   kylinv10 SP2 和 SP3 的 Linux 内核版本最高只有 4.19.90-24.4.v2101.ky10.aarch64，无法继续升级内核版本，只能在 kylinv10 上重新编译 Doris BE，如果使用新版本 ldb_toolchain 编译 Doris BE 后问题依然存在，可以尝试使用低版本 ldb_toolchain v0.17 编译，如果你的 ARM 环境无法连外网，华为云提供 ARM + kylinv10，阿里云提供 x86 + kylinv10

4. Q: prepared statement 传递参数报错。

   A: 目前 `jdbc:arrow-flight-sql` 和 Java ADBC/JDBCDriver 不支持 prepared statement 传递参数，类似`select * from xxx where id=?`，将报错 `parameter ordinal 1 out of range`，这是 Arrow Flight SQL 的一个 BUG（ GitHub Issue ）。

5. Q: 如何修改 `jdbc:arrow-flight-sql` 每次读取的批次大小，在某些场景下提升性能。

   A: 通过修改`org.apache.arrow.adbc.driver.jdbc.JdbcArrowReader`文件中`makeJdbcConfig`方法中的`setTargetBatchSize`，默认是 1024，然后将修改后的文件保存到本地同名路径目录下，从而覆盖原文件生效。

6. Q: ADBC v0.10，JDBC 和 Java ADBC/JDBCDriver 不支持并行读取。

   A: 没有实现`stmt.executePartitioned()`这个方法，只能使用原生的 FlightClient 实现并行读取多个 Endpoints，使用方法`sqlClient=new FlightSqlClient, execute=sqlClient.execute(sql), endpoints=execute.getEndpoints(), for(FlightEndpoint endpoint: endpoints)`，此外，ADBC V0.10 默认的 AdbcStatement 实际是 JdbcStatement，executeQuery 后将行存格式的 JDBC ResultSet 又重新转成的 Arrow 列存格式，预期到 ADBC 1.0.0 时 Java ADBC 将功能完善 GitHub Issue。

7. Q: 在 URL 中指定 database name。

   A: 截止 Arrow v15.0，Arrow JDBC Connector 不支持在 URL 中指定 database name，比如 `jdbc:arrow-flight-sql://{FE_HOST}:{fe.conf:arrow_flight_sql_port}/test?useServerPrepStmts=false` 中指定连接`test` database 无效，只能手动执行 SQL `use database`。Arrow v18.0 支持了在 URL 中指定 database name，但实测仍有 BUG。

8. Q: Python ADBC print `Warning: Cannot disable autocommit; conn will not be DB-API 2.0 compliant`。

   A: 使用 Python 时忽略这个 Warning，这是 Python ADBC Client 的问题，不会影响查询。

9. Q: Python 报错 `grpc: received message larger than max (20748753 vs. 16777216)`。

   A: 参考 Python: grpc: received message larger than max (20748753 vs. 16777216) #2078 在 Database Option 中增加 `adbc_driver_flightsql.DatabaseOptions.WITH_MAX_MSG_SIZE.value`.

10. Q: 报错 `invalid bearer token`。

    A: 执行 `SET PROPERTY FOR 'root' 'max_user_connections' = '10000';` 修改当前用户的当前最大连接数到 10000；在 `fe.conf` 增加 qe_max_connection=30000 和 arrow_flight_token_cache_size=8000 并重启 FE。

    ADBC Client 和 Arrow Flight Server 端之间的连接本质上是个长链接，需要在 Server 缓存 Auth Token、Connection、Session，连接创建后不会在单个查询结束时立即断开，需要 Client 发送 close() 请求后清理，但实际上 Client 经常不会发送 close 请求，所以 Auth Token、 Connection、 Session 会长时间在 Arrow Flight Server 上保存，默认会在 3 天后超时断开，或者在连接数超过 `arrow_flight_token_cache_size` 的限制后依据 LRU 淘汰。

    截止 Doris v2.1.8，Arrow Flight 连接和 Mysql/JDBC 连接使用相同的连接数限制，包括 FE 所有用户的总连接数 `qe_max_connection` 和单个用户的连接数 `UserProperty` 中的 `max_user_connections`。但默认的 `qe_max_connection` 和 `max_user_connections` 分别是 1024 和 100。Arrow Flight SQL 常用来取代使用 JDBC 的场景，但 JDBC 连接会在查询结束后立即释放，所以使用 Arrow Flight SQL 时，Doris 默认的连接数限制太小，经常导致连接数超过 `arrow_flight_token_cache_size` 的限制后将仍在被使用的连接淘汰。

11. Q: 使用 JDBC 或 JAVA 连接 Arrow Flight SQL 读取 Datatime 类型返回时间戳，而不是格式化时间。

    A: JDBC 或 JAVA 连接 Arrow Flight SQL 读取 Datatime 类型需要自行转换时间戳，参考 Add java parsing datetime type in arrow flight sql sample #48578。 用 Python Arrow Flight SQL 读取 Datatime 类型返回结果为 `2025-03-03 17:23:28Z`，而 JDBC 或 JAVA 返回 1740993808。

12. Q: 使用 JDBC 或 Java JDBC Client 连接 Arrow Flight SQL 读取 Array 嵌套类型报错 `Configuration does not provide a mapping for array column 2`。

    A: 参考 `sample/arrow-flight-sql` 使用 JAVA ADBC Client。

    Python ADBC Client、JAVA ADBC Client、Java JDBC DriverManager 读取 Array 嵌套类型都没问题，只有使用 JDBC 或 Java JDBC Client 连接 Arrow Flight SQL 有问题，实际上 Arrow Flight JDBC 的兼容性不好保证，不是 Arrow 官方开发的，由一个第三方数据库公司 Dremio 开发，之前还发现过其他兼容性问题，所以建议优先用 JAVA ADBC Client。

2.7.2.8 2.1 Release Note

> v2.1.4 及之前的版本 Doris Arrow Flight 不够完善，建议升级后使用。

2.7.2.8.1 v2.1.9

1. 修复 Doris 数据序列化到 Arrow 的问题。Fix UT DataTypeSerDeArrowTest of Array/Map/Struct/Bitmap/HLL/Decimal256 types

- 读取 Decimal256 类型失败;
- 读取 DatetimeV2 类型微妙部分错误;
- 读取 DateV2 类型结果不正确;
- 读取 IPV4/IPV6 类型结果为 NULL 时报错;

2. 修复 Doris Arrow Flight SQL 查询失败返回空结果，没有返回真实的错误信息。 Fix query result is empty and not return query error message

2.7.2.8.2 v2.1.8

1. 支持 DBeaver 等 BI 工具使用 arrow-flight-sql 协议连接 Doris，支持正确显示元数据树。 Support arrow-flight-sql protocol getStreamCatalogs, getStreamSchemas, getStreamTables #46217。
2. 支持多 BE 共享同一个可供集群外部访问的 IP 时，查询结果可以正确转发后返回 ADBC Client。 Arrow flight server supports data forwarding when BE uses public vip
3. 支持多个 Endpoint 并行读取。 Arrow Flight support multiple endpoints
4. 修复查询报错 FE not found arrow flight schema。 Fix FE not found arrow flight schema
5. 修复读取允许 NULL 的列报错 BooleanBuilder::AppendValues。Fix Doris NULL column conversion to arrow batch
6. 修复 show processlist 显示重复的 Connection ID。 Fix arrow-flight-sql ConnectContext to use a unified ID #46284
7. 修复读取 Datetime 和 DatetimeV2 类型丢失时区，导致比真实数据的 datetime 少 8 小时的问题。 Fix time zone issues and accuracy issues #38215

2.7.2.8.3 v2.1.7

1. 修复频繁打印日志 Connection wait_timeout。 Fix kill timeout FlightSqlConnection and FlightSqlConnectProcessor close
2. 修复 Arrow Flight Bearer Token 过期后从 Cache 中淘汰。 Fix Arrow Flight bearer token cache evict after expired

2.7.2.8.4 v2.1.6

1. 修复查询报错 0.0.0.0:xxx, connection refused。 Fix return result from FE Arrow Flight server error 0.0.0.0:xxx, connection refused
2. 修复查询报错 Reach limit of connections。Fix exceed user property max connection cause Reach limit of connections #39127

   之前的版本执行 SET PROPERTY FOR 'root' 'max_user_connections' = '1024'; 修改当前用户的当前最大连接数到 1024，可临时规避。

   因为之前的版本只限制 Arrow Flight 连接数小于 qe_max_connection/2，qe_max_connection 是 fe 所有用户的总连接数，默认 1024，没有限制单个用户的 Arrow Flight 连接数小于 UserProperty 中的 max_user_connections，默认 100，所以当 Arrow Flight 连接数超过当前用户连接数上限时将报错 Reach limit of connections，所以需调大当前用户的 max_user_connections。

   问题详情见： Questions

3. 增加 Conf `arrow_flight_result_sink_buffer_size_rows`，支持修改单次返回的查询结果 ArrowBatch 大小，默认 4096 * 8。 Add config arrow_flight_result_sink_buffer_size_rows

2.7.2.8.5 v2.1.5

1. 修复 Arrow Flight SQL 查询结果为空。 Fix arrow flight result sink #36827

   Doris v2.1.4 读取大数据量时有几率报错，问题详情见： Questions

2.7.2.9 3.0 Release Note

2.7.2.9.1 v3.0.5

1. 修复 Doris 数据序列化到 Arrow 的问题。Fix UT DataTypeSerDeArrowTest of Array/Map/Struct/Bitmap/HLL/Decimal256 types

   - 读取 `Decimal256` 类型失败;
   - 读取 `DatetimeV2` 类型微妙部分错误;
   - 读取 `DateV2` 类型结果不正确;
   - 读取 `IPV4/IPV6` 类型结果为 NULL 时报错;

2.7.2.9.2 v3.0.4

1. 支持 DBeaver 等 BI 工具使用 `arrow-flight-sql` 协议连接 Doris，支持正确显示元数据树。Support arrow-flight-sql protocol getStreamCatalogs, getStreamSchemas, getStreamTables #46217。
2. 支持多个 Endpoint 并行读取。 Arrow Flight support multiple endpoints
3. 修复读取允许 NULL 的列报错 `BooleanBuilder::AppendValues`。Fix Doris NULL column conversion to arrow batch
4. 修复 `show processlist` 显示重复的 Connection ID。 Fix arrow-flight-sql ConnectContext to use a unified ID #46284
5. 修复 Doris Arrow Flight SQL 查询失败返回空结果，没有返回真实的错误信息。 Fix query result is empty and not return query error message

2.7.2.9.3 v3.0.3

1. 修复查询报错 `0.0.0.0:xxx, connection refused`。 Fix return result from FE Arrow Flight server error 0.0.0.0:xxx, connection refused
2. 修复查询报错 `Reach limit of connections`。Fix exceed user property max connection cause Reach limit of connections #39127

   之前的版本执行 `SET PROPERTY FOR 'root' 'max_user_connections' = '1024';` 修改当前用户的当前最大连接数到 1024，可临时规避。

因为之前的版本只限制 Arrow Flight 连接数小于 qe_max_connection/2，qe_max_connection 是 fe 所有用户的总连接数，默认 1024，没有限制单个用户的 Arrow Flight 连接数小于 UserProperty 中的 max_user_connections，默认 100，所以当 Arrow Flight 连接数超过当前用户连接数上限时将报错 Reach limit of connections，所以需调大当前用户的 max_user_connections。

问题详情见：Questions

3. 修复频繁打印日志 Connection wait_timeout。 Fix kill timeout FlightSqlConnection and FlightSqlConnectProcessor close
4. 修复 Arrow Flight Bearer Token 过期后从 Cache 中淘汰。 Fix Arrow Flight bearer token cache evict after expired
5. 支持多 BE 共享同一个可供集群外部访问的 IP 时，查询结果可以正确转发后返回 ADBC Client。 Arrow flight server supports data forwarding when BE uses public vip
6. 修复查询报错 FE not found arrow flight schema。 Fix FE not found arrow flight schema
7. 修复读取 Datetime 和 DatetimeV2 类型丢失时区，导致比真实数据的 datetime 少 8 小时的问题。 Fix time zone issues and accuracy issues #38215

2.7.2.9.4 v3.0.2

1. 增加 Conf arrow_flight_result_sink_buffer_size_rows，支持修改单次返回的查询结果 ArrowBatch 大小，默认 4096 * 8。 Add config arrow_flight_result_sink_buffer_size_rows

2.7.2.9.5 v3.0.1

1. 查询结果缺失，查询结果行数 = 实际行数 / BE 个数 Fix get Schema failed when enable_parallel_result_sink is false #37779

   在 Doris 3.0.0 版本，如果查询最外层是聚合，SQL 类似 select k1, sum(k2)from xxx group by k1，你可能会遇到（查询结果行数 = 实际行数 / BE 个数），这是 support parallel result sink 引入的问题，在 Fix get Schema failed when enable_parallel_result_sink is false 临时修复，在 Arrow Flight support multiple endpoints 支持多个 Endpoint 并行读取后正式修复。

2.8 数据表设计

2.8.1 概览

2.8.1.1 创建表

使用CREATE TABLE 语句在 Doris 中创建一个表，也可以使用CREATE TABKE LIKE 或CREATE TABLE AS 子句从另一个表派生表定义。

2.8.1.2 表名

Doris 中表名默认是大小写敏感的，可以在第一次初始化集群时配置lower_case_table_names为大小写不敏感的。默认的表名最大长度为 64 字节，可以通过配置table_name_length_limit更改，不建议配置过大。创建表的语法请参考CREATE TABLE。

2.8.1.3 表属性

Doris 的建表语句中可以指定建表属性，包括：

- 分桶数 (buckets)：决定数据在表中的分布；
- 存储介质 (storage_medium)：控制数据的存储方式，如使用 HDD、SSD 或远程共享存储；
- 副本数 (replication_num)：控制数据副本的数量，以保证数据的冗余和可靠性；
- 冷热分离存储策略 (storage_policy)：控制数据的冷热分离存储的迁移策略；

这些属性作用于分区，即分区创建之后，分区就会有自己的属性，修改表属性只对未来创建的分区生效，对已经创建好的分区不生效，关于属性更多的信息请参考修改表属性。动态分区可以单独设置这些属性。

2.8.1.4 注意事项

1. 选择合适的数据模型：数据模型不可更改，建表时需要选择一个合适的数据模型；
2. 选择合适的分桶数：已经创建的分区不能修改分桶数，可以通过替换分区来修改分桶数，可以修改动态分区未创建的分区分桶数；
3. 添加列操作：加减 VALUE 列是轻量级实现，秒级别可以完成，加减 KEY 列或者修改数据类型是重量级操作，完成时间取决于数据量，大规模数据下尽量避免加减 KEY 列或者修改数据类型；
4. 优化存储策略：可以使用层级存储将冷数据保存到 HDD 或者 S3 / HDFS。

2.8.2 表类型

2.8.2.1 表类型概述

在 Doris 中建表时需要指定表类型，以定义数据存储与管理方式。在 Doris 中提供了明细表、聚合表以及主键表三种表类型，可以应对不同的应用场景需求。不同的表类型具有相应的数据去重、聚合及更新机制。选择合适的表类型有助于实现业务目标，同时保证数据处理的灵活性和高效性。

2.8.2.1.1 表类型分类

在 Doris 中支持三种表类型：

- 明细表（Duplicate Key Table）：允许指定的 Key 列重复，Doirs 存储层保留所有写入的数据，适用于必须保留所有原始数据记录的情况；
- 主键表（Unique Key Table）：每一行的 Key 值唯一，可确保给定的 Key 列不会存在重复行，Doris 存储层对每个 key 只保留最新写入的数据，适用于数据更新的情况；
- 聚合表（Aggregate Key Table）：可根据 Key 列聚合数据，Doris 存储层保留聚合后的数据，从而可以减少存储空间和提升查询性能；通常用于需要汇总或聚合信息（如总数或平均值）的情况。

在建表后，表类型已经确认，无法修改。针对业务选择合适的类型至关重要：

- Duplicate Key Table：适合任意维度的 Ad-hoc 查询。虽然同样无法利用预聚合的特性，但是不受聚合表的约束，可以发挥列列存的优势（只读取相关列，而不需要读取所有 Key 列）。
- Unique Key Table：针对需要唯一主键约束的场景，可以保证主键唯一性约束。但是无法利用 ROLLUP 等预聚合带来的查询优势。
- Aggregate Key Table：可以通过预聚合，极大地降低聚合查询时所需扫描的数据量和查询的计算量，非常适合有固定模式的报表类查询场景。但是该类型表对 `count(*)` 查询很不友好。同时因为固定了 Value 列上的聚合方式，在进行其他类型的聚合查询时，需要考虑语意正确性。
- 部分列更新：请查阅文档主键表部分列更新与聚合表部份列更新获取相关使用建议。

2.8.2.1.2 排序键

在 Doris 中，数据以列的形式存储，一张表可以分为 key 列与 value 列。其中，key 列用于分组与排序，value 列用于参与聚合。Key 列可以是一个或多个字段，在建表时，按照各种表类型中，Aggregate Key、Unique Key 和 Duplicate Key 的列进行数据排序存储。

不同的表类型都需要在建表时指定 Key 列，分别有不同的意义：对于明细表，Key 列表示排序，没有唯一键的约束。在聚合表与主键表中，会基于 Key 列进行聚合，Key 列既有排序的能力，又有唯一键的约束。

合理使用排序键可以带来以下收益：

- 加速查询性能：排序键有助于减少数据扫描量。对于范围查询或过滤查询，可以利用排序键直接定位数据的位置。对于需要进行排序的查询，也可以利用排序键进行排序加速；
- 数据压缩优化：数据按排序键有序存储会提高压缩的效率，相似的数据会聚集在一起，压缩率会大幅度提高，从而减小数据的存储空间。
- 减少去重成本：当使用 Unique Key 表时，通过排序键，Doris 能更有效地进行去重操作，保证数据唯一性。

选择排序键时，可以遵循以下建议：

- Key 列必须在所有 Value 列之前。
- 尽量选择整型类型。因为整型类型的计算和查找效率远高于字符串。
- 对于不同长度的整型类型的选择原则，遵循够用即可。
- 对于 VARCHAR 和 STRING 类型的长度，遵循够用即可原则。

2.8.2.1.3 表类型能力对比

	明细表	主键表	聚合表
Key 列唯一约束	不支持，Key 列可以重复	支持	支持
同步物化视图	支持	支持	支持
异步物化视图	支持	支持	支持
UPDATE 语句	不支持	支持	不支持
DELETE 语句	部分支持	支持	不支持
导入时整行更新	不支持	支持	不支持
导入时部分列更新	不支持	支持	部分支持

2.8.2.2 明细表

明细表是 Doris 中的默认建表模型，用于保存每条原始数据记录。在建表时，通过 DUPLICATE KEY 指定数据存储的排序列，以优化常用查询。一般建议选择三列或更少的列作为排序键，具体选择方式参考排序键。明细表具有以下特点：

- 保留原始数据：明细表保留了全量的原始数据，适合于存储与查询原始数据。对于需要进行详细数据分析的应用场景，建议使用明细表，以避免数据丢失的风险；
- 不去重也不聚合：与聚合模型与主键模型不同，明细表不会对数据进行去重与聚合操作。即使两条相同的数据，每次插入时也会被完整保留；
- 灵活的数据查询：明细表保留了全量的原始数据，可以从完整数据中提取细节，基于全量数据做任意维度的聚合操作，从而进行元数数据的审计及细粒度的分析。

2.8.2.2.1 使用场景

一般明细表中的数据只进行追加，旧数据不会更新。明细表适用于需要存储全量原始数据的场景：

- 日志存储：用于存储各类的程序操作日志，如访问日志、错误日志等。每一条数据都需要被详细记录，方便后续的审计与分析；
- 用户行为数据：在分析用户行为时，如点击数据、用户访问轨迹等，需要保留用户的详细行为，方便后续构建用户画像及对行为路径进行详细分析；
- 交易数据：在某些存储交易行为或订单数据时，交易结束时一般不会发生数据变更。明细表适合保留这一类交易信息，不遗漏任意一笔记录，方便对交易进行精确的对账。

2.8.2.2.2 建表说明

在建表时，可以通过 DUPLICATE KEY 关键字指定明细表。明细表必须指定数据的 Key 列，用于在存储时对数据进行排序。下例的明细表中存储了日志信息，并针对于 log_time、log_type 及 error_code 三列进行了排序：

```
CREATE TABLE IF NOT EXISTS example_tbl_duplicate
(
    log_time        DATETIME       NOT NULL,
    log_type        INT            NOT NULL,
    error_code      INT,
    error_msg       VARCHAR(1024),
    op_id           BIGINT,
    op_time         DATETIME
)
DUPLICATE KEY(log_time, log_type, error_code)
DISTRIBUTED BY HASH(log_type) BUCKETS 10;
```

2.8.2.2.3 数据插入与存储

在明细表中，数据不进行去重与聚合，插入数据即存储数据。明细表中 Key 列指做为排序。

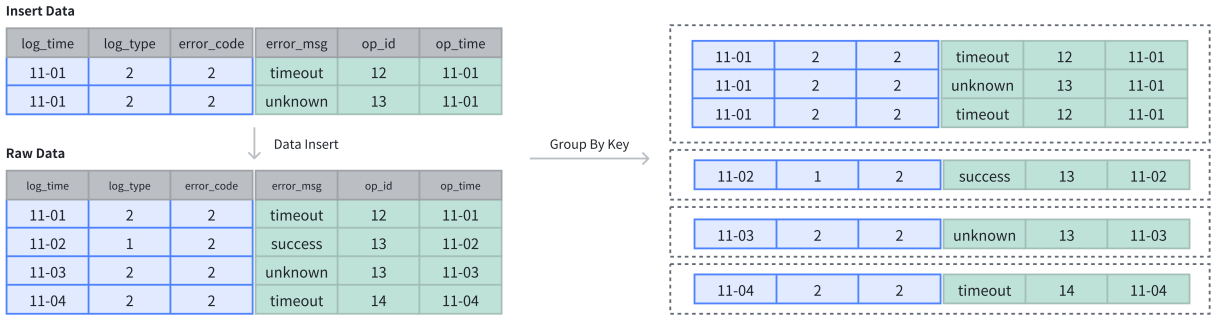


图 22: columnar_storage

在上例中，表中原有 4 行数据，插入 2 行数据后，采用追加（APPEND）方式存储，共计 6 行数据：

```
-- 4 rows raw data
INSERT INTO example_tbl_duplicate VALUES
('2024-11-01 00:00:00', 2, 2, 'timeout', 12, '2024-11-01 01:00:00'),
('2024-11-02 00:00:00', 1, 2, 'success', 13, '2024-11-02 01:00:00'),
('2024-11-03 00:00:00', 2, 2, 'unknown', 13, '2024-11-03 01:00:00'),
('2024-11-04 00:00:00', 2, 2, 'unknown', 12, '2024-11-04 01:00:00');

-- insert into 2 rows
INSERT INTO example_tbl_duplicate VALUES
('2024-11-01 00:00:00', 2, 2, 'timeout', 12, '2024-11-01 01:00:00'),
('2024-11-01 00:00:00', 2, 2, 'unknown', 13, '2024-11-01 01:00:00');

-- check the rows of table
SELECT * FROM example_tbl_duplicate;
+---------------------+----------+------------+-----------+-------+---------------------+
| log_time            | log_type | error_code | error_msg | op_id | op_time             |
+---------------------+----------+------------+-----------+-------+---------------------+
| 2024-11-02 00:00:00 |        1 |          2 | success   |    13 | 2024-11-02 01:00:00 |
| 2024-11-01 00:00:00 |        2 |          2 | timeout   |    12 | 2024-11-01 01:00:00 |
| 2024-11-03 00:00:00 |        2 |          2 | unknown   |    13 | 2024-11-03 01:00:00 |
| 2024-11-04 00:00:00 |        2 |          2 | unknown   |    12 | 2024-11-04 01:00:00 |
| 2024-11-01 00:00:00 |        2 |          2 | unknown   |    13 | 2024-11-01 01:00:00 |
| 2024-11-01 00:00:00 |        2 |          2 | timeout   |    12 | 2024-11-01 01:00:00 |
+---------------------+----------+------------+-----------+-------+---------------------+
```

2.8.2.3 主键表

当需要更新数据时，可以选择主键表（Unique Key Table）。该模型保证 Key 列的唯一性，插入或更新数据时，新数据会覆盖具有相同 Key 的旧数据，确保数据记录为最新。与其他数据模型相比，主键表适用于数据的更新场景，在插入过程中进行主键级别的更新覆盖。

主键表有以下特点：

- 基于主键进行 UPSERT：在插入数据时，主键重复的数据会更新，主键不存在的记录会插入；
- 基于主键进行去重：主键表中的 Key 列具有唯一性，会对根据主键列对数据进行去重操作；
- 高频数据更新：支持高频数据更新场景，同时平衡数据更新性能与查询性能。

2.8.2.3.1 使用场景

- 高频数据更新：适用于上游 OLTP 数据库中的维度表，实时同步更新记录，并高效执行 UPSERT 操作；
- 数据高效去重：如广告投放和客户关系管理系统中，使用主键表可以基于用户 ID 高效去重；
- 需要部分列更新：如画像标签场景需要变更频繁改动的动态标签，消费订单场景需要改变交易的状态。通过主键表部分列更新能力可以完成某几列的变更操作。

2.8.2.3.2 实现方式

在 Doris 中主键表有两种实现方式：

- 写时合并（merge-on-write）：自 1.2 版本起，Doris 默认使用写时合并模式，数据在写入时立即合并相同 Key 的记录，确保存储的始终是最新数据。写时合并兼顾查询和写入性能，避免多个版本的数据合并，并支持谓词下推到存储层。大多数场景推荐使用此模式；
- 读时合并（merge-on-read）：在 1.2 版本前，Doris 中的主键表默认使用读时合并模式，数据在写入时并不进行合并，以增量的方式被追加存储，在 Doris 内保留多个版本。查询或 Compaction 时，会对数据进行相同 Key 的版本合并。读时合并适合写多读少的场景，在查询是需要进行多个版本合并，谓词无法下推，可能会影响到查询速度。

在 Doris 中基于主键表更新有两种语义：

- 整行更新：Unique Key 模型默认的更新语义为整行UPSERT，即 UPDATE OR INSERT，该行数据的 Key 如果存在，则进行更新，如果不存在，则进行新数据插入。在整行 UPSERT 语义下，即使用户使用 Insert Into 指定部分列进行写入，Doris 也会在 Planner 中将未提供的列使用 NULL 值或者默认值进行填充。
- 部分列更新：如果用户希望更新部分字段，需要使用写时合并实现，并通过特定的参数来开启部分列更新的支持。请查阅文档部分列更新。

2.8.2.3.3 写时合并

在建表时，使用 UNIQUE KEY 关键字可以指定主键表。通过显示开启 enable_unique_key_merge_on_write 属性可以指定写时合并模式。自 Doris 2.1 版本以后，默认开启写时合并：

```
CREATE TABLE IF NOT EXISTS example_tbl_unique
(
    user_id         LARGEINT        NOT NULL,
    user_name       VARCHAR(50)     NOT NULL,
    city            VARCHAR(20),
    age             SMALLINT,
    sex             TINYINT
)
UNIQUE KEY(user_id, user_name)
DISTRIBUTED BY HASH(user_id) BUCKETS 10
PROPERTIES (
    "enable_unique_key_merge_on_write" = "true"
);
```

2.8.2.3.4 读时合并

在建表时，使用 UNIQUE KEY 关键字可以指定主键表。通过显示关闭 enable_unique_key_merge_on_write 属性可以指定读时合并模式。在 Doris 2.1 版本之前，默认开启读时合并：

```
CREATE TABLE IF NOT EXISTS example_tbl_unique
(
    user_id         LARGEINT        NOT NULL,
    username        VARCHAR(50)     NOT NULL,
    city            VARCHAR(20),
    age             SMALLINT,
    sex             TINYINT
)
UNIQUE KEY(user_id, username)
DISTRIBUTED BY HASH(user_id) BUCKETS 10
PROPERTIES (
    "enable_unique_key_merge_on_write" = "false"
);
```

2.8.2.3.5 数据插入与存储

在主键表中，Key 列不仅用于排序，还用于去重，插入数据时，相同 Key 的记录会被覆盖。

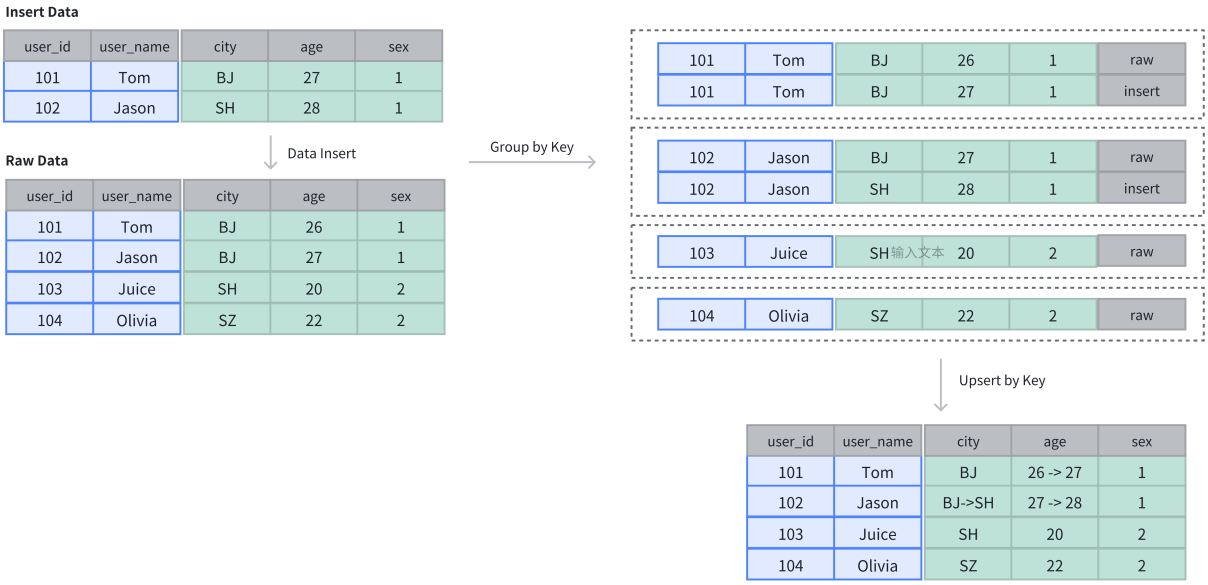


图 23: unique-key-model-insert

如上例所示，原表中有 4 行数据，插入 2 行后，新插入的数据基于主键进行了更新：

```
-- insert into raw data
INSERT INTO example_tbl_unique VALUES
(101, 'Tom', 'BJ', 26, 1),
(102, 'Jason', 'BJ', 27, 1),
(103, 'Juice', 'SH', 20, 2),
(104, 'Olivia', 'SZ', 22, 2);

-- insert into data to update by key
INSERT INTO example_tbl_unique VALUES
(101, 'Tom', 'BJ', 27, 1),
(102, 'Jason', 'SH', 28, 1);

-- check updated data
SELECT * FROM example_tbl_unique;
+---------+----------+------+------+------+
| user_id | username | city | age  | sex  |
+---------+----------+------+------+------+
| 101     | Tom      | BJ   |   27 |    1 |
| 102     | Jason    | SH   |   28 |    1 |
| 104     | Olivia   | SZ   |   22 |    2 |
| 103     | Juice    | SH   |   20 |    2 |
+---------+----------+------+------+------+
```

2.8.2.3.6 注意事项

- Unique 表的实现方式只能在建表时确定，无法通过 schema change 进行修改；
- 在整行 UPSERT 语义下，即使用户使用 insert into 指定部分列进行写入，Doris 也会在 Planner 中将未提供的列使用 NULL 值或者默认值进行填充；
- 部分列更新。如果用户希望更新部分字段，需要使用写时合并实现，并通过特定的参数来开启部分列更新的支持。请查阅文档部分列更新获取相关使用建议；
- 使用 Unique 表时，为了保证数据的唯一性，分区键必须包含在 Key 列内。

2.8.2.4 聚合表

Doris 的聚合表专为高效处理大规模数据查询中的聚合操作设计。它通过预聚合数据，减少重复计算，提升查询性能。聚合表只存储聚合后的数据，节省存储空间并加速查询。

2.8.2.4.1 使用场景

- 明细数据进行汇总：用于电商平台的月销售业绩、金融风控的客户交易总额、广告投放的点击量等业务场景中，进行多维度汇总；
- 不需要查询原始明细数据：如驾驶舱报表、用户交易行为分析等，原始数据存储在数据湖中，仅需存储汇总后的数据。

2.8.2.4.2 原理

每一次数据导入会在聚合表内形成一个版本，在 Compaction 阶段进行版本合并，在查询时会按照主键进行数据聚合：

1. 数据导入阶段：数据按批次导入，每批次生成一个版本，并对相同聚合键的数据进行初步聚合（如求和、计数）；
2. 后台文件合并阶段（Compaction）：多个版本文件会定期合并，减少冗余并优化存储；
3. 查询阶段：查询时，系统会聚合同一聚合键的数据，确保查询结果准确。

2.8.2.4.3 建表说明

使用 AGGREGATE KEY 关键字在建表时指定聚合表，并指定 Key 列用于聚合 Value 列。

```
CREATE TABLE IF NOT EXISTS example_tbl_agg
(
    user_id             LARGEINT    NOT NULL,
    load_dt             DATE        NOT NULL,
    city                VARCHAR(20),
    last_visit_dt       DATETIME    REPLACE DEFAULT "1970-01-01 00:00:00",
    cost                BIGINT      SUM DEFAULT "0",
    max_dwell           INT         MAX DEFAULT "0",
```

```
)
AGGREGATE KEY(user_id, load_dt, city)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

上例中定义了用户信息和访问的行为事实表，将 user_id、load_dt、city作为 Key 列进行聚合操作。数据导入时，Key 列会聚合成一行，Value 列会按照指定的聚合类型进行维度聚合。

在聚合表中支持以下类型的维度聚合：

聚合方式	描述
SUM	求和，多行的 Value 进行累加。
REPLACE	替代，下一批数据中的 Value 会替换之前导入过的行中的 Value。
MAX	保留最大值。
MIN	保留最小值。
REPLACE_IF_NOT_NULL	非空值替换。与 REPLACE 的区别在于对 null 值，不做替换。
HLL_UNION	HLL 类型的列的聚合方式，通过 HyperLogLog 算法聚合。
BITMAP_UNION	BITMAP 类型的列的聚合方式，进行位图的并集聚合。

> 提示：
> 如果以上的聚合方式无法满足业务需求，可以选择使用 agg_state 类型。

2.8.2.4.4 数据插入与存储

在聚合表中，数据基于主键进行聚合操作。数据插入后及完成聚合操作。

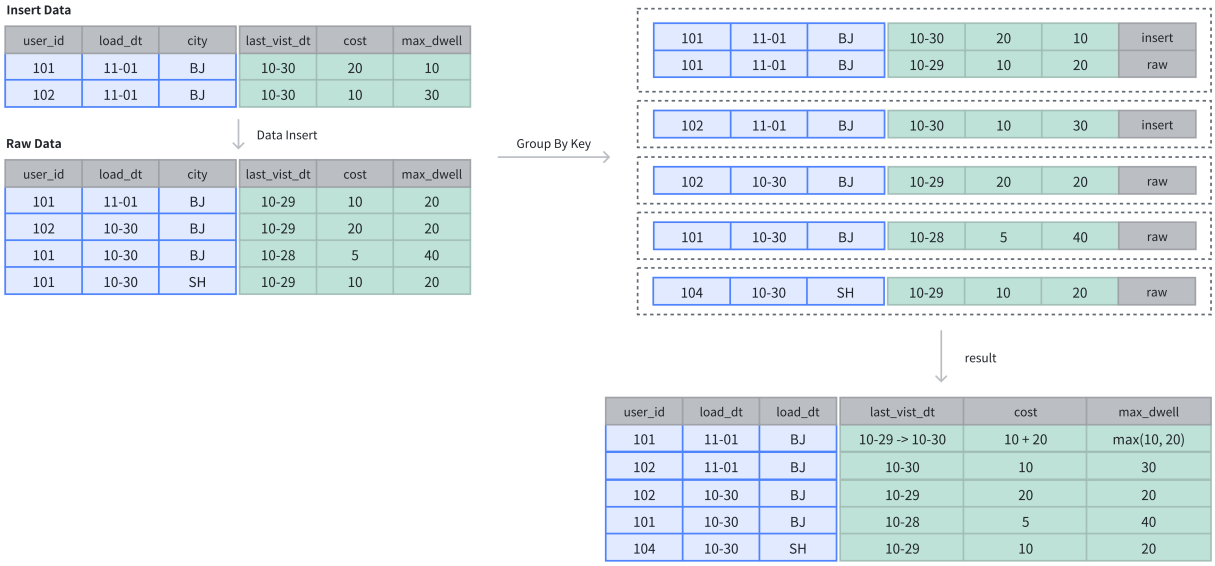


图 24: aggrate-key-model-insert

在上例中，表中原有 4 行数据，在插入 2 行数据后，基于 Key 列进行行维度列的聚合操作：

```
-- 4 rows raw data
INSERT INTO example_tbl_agg VALUES
(101, '2024-11-01', 'BJ', '2024-10-29', 10, 20),
(102, '2024-10-30', 'BJ', '2024-10-29', 20, 20),
(101, '2024-10-30', 'BJ', '2024-10-28', 5, 40),
(101, '2024-10-30', 'SH', '2024-10-29', 10, 20);

-- insert into 2 rows
INSERT INTO example_tbl_agg VALUES
(101, '2024-11-01', 'BJ', '2024-10-30', 20, 10),
(102, '2024-11-01', 'BJ', '2024-10-30', 10, 30);

-- check the rows of table
SELECT * FROM example_tbl_agg;
+---------+------------+------+---------------------+------+----------------+
| user_id | load_date  | city | last_visit_date     | cost | max_dwell_time |
+---------+------------+------+---------------------+------+----------------+
| 102     | 2024-10-30 | BJ   | 2024-10-29 00:00:00 |   20 |             20 |
| 102     | 2024-11-01 | BJ   | 2024-10-30 00:00:00 |   10 |             30 |
| 101     | 2024-10-30 | BJ   | 2024-10-28 00:00:00 |    5 |             40 |
| 101     | 2024-10-30 | SH   | 2024-10-29 00:00:00 |   10 |             20 |
| 101     | 2024-11-01 | BJ   | 2024-10-30 00:00:00 |   30 |             20 |
+---------+------------+------+---------------------+------+----------------+
```

2.8.2.4.5 AGG_STATE

> 提示：
>
> AGG_STATE 是实验特性，建议在开发与测试环境中使用。

AGG_STATE 不能作为 Key 列使用，建表时需要同时声明聚合函数的签名。不需要指定长度和默认值。实际存储的数据大小与函数实现有关。

```
set enable_agg_state = true;
CREATE TABLE aggstate(
    k1   int  NULL,
    v1   int  SUM,
    v2   agg_state<group_concat(string)> generic
)
AGGREGATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 3;
```

在此示例中，agg_state 用于声明数据类型，sum/group_concat 为聚合函数签名。agg_state 是一种数据类型，类似于 int、array、string。agg_state 只能与state、mergeunion 函数组合器配合使用。它表示聚合函数的中间结果，例如 group_concat 的中间状态，而非最终结果。

agg_state 类型需要使用 state 函数来生成，对于当前的这个表，需要使用 group_concat_state：

```
insert into aggstate values(1, 1, group_concat_state('a'));
insert into aggstate values(1, 2, group_concat_state('b'));
insert into aggstate values(1, 3, group_concat_state('c'));
insert into aggstate values(2, 4, group_concat_state('d'));
```

此时表内计算方式如下图所示：

K1	V1	V2
1	sum(1, 2, 3)	group_concat('a', 'b', 'c')
2	sum(4)	group_concat('d')

图 25: state-func-group-concat-state-result-1

在查询时，可以使用 merge 操作合并多个 state，并且返回最终聚合结果。因为 group_concat 对于顺序有要求，所以结果是不稳定的。

```
select group_concat_merge(v2) from aggstate;
+------------------------+
| group_concat_merge(v2) |
+------------------------+
| d,c,b,a                |
+------------------------+
```

如果不想要最终的聚合结果，而希望保留中间结果，可以使用 union 操作：

```
insert into aggstate select 3,sum_union(k2),group_concat_union(k3) from aggstate;
```

此时表中计算如下：

K1	V1	V2
1	sum(1, 2, 3)	group_concat('a', 'b', 'c')
2	sum(4)	group_concat('d')
3	sum(1, 2, 3, 4)	group_concat('a', 'b', 'c', 'd')

图 26: state-func-group-concat-state-result-2

查询结果如下：

```
mysql> select sum_merge(k2) , group_concat_merge(k3)from aggstate;
+---------------+------------------------+
| sum_merge(k2) | group_concat_merge(k3) |
+---------------+------------------------+
|            20 | c,b,a,d,c,b,a,d        |
+---------------+------------------------+

mysql> select sum_merge(k2) , group_concat_merge(k3)from aggstate where k1 != 2;
+---------------+------------------------+
| sum_merge(k2) | group_concat_merge(k3) |
+---------------+------------------------+
|            16 | c,b,a,d,c,b,a          |
+---------------+------------------------+
```

2.8.2.5 使用注意

2.8.2.5.1 建表时列类型建议

1. Key 列必须在所有 Value 列之前。
2. 尽量选择整型类型。因为整型类型的计算和查找效率远高于字符串。
3. 对于不同长度的整型类型的选择原则，遵循够用即可。
4. 对于 VARCHAR 和 STRING 类型的长度，遵循够用即可。

2.8.2.5.2 聚合模型的局限性

这里针对 Aggregate 模型，来介绍下聚合模型的局限性。

在聚合模型中，模型对外展现的，是最终聚合后的数据。也就是说，任何还未聚合的数据（比如说两个不同导入批次的数据），必须通过某种方式，以保证对外展示的一致性。举例说明。

假设表结构如下：

ColumnName	Type	AggregationType	Comment
user_id	LARGEINT		用户 id
date	DATE		数据灌入日期
cost	BIGINT	SUM	用户总消费

假设存储引擎中有如下两个已经导入完成的批次的数据：

batch 1

user_id	date	cost
10001	2017/11/20	50
10002	2017/11/21	39

batch 2

user_id	date	cost
10001	2017/11/20	1
10001	2017/11/21	5
10003	2017/11/22	22

可以看到，用户 10001 分属在两个导入批次中的数据还没有聚合。但是为了保证用户只能查询到如下最终聚合后的数据：

user_id	date	cost
10001	2017/11/20	51
10001	2017/11/21	5
10002	2017/11/21	39

user_id	date	cost
10003	2017/11/22	22

我们在查询引擎中加入了聚合算子，来保证数据对外的一致性。

另外，在聚合列（Value）上，执行与聚合类型不一致的聚合类查询时，要注意语意。比如在如上示例中执行如下查询：

```
SELECT MIN(cost) FROM table;
```

得到的结果是 5，而不是 1。

同时，这种一致性保证，在某些查询中，会极大地降低查询效率。

以最基本的 count(*) 查询为例：

```
SELECT COUNT(*) FROM table;
```

在其他数据库中，这类查询都会很快地返回结果。因为在实现上，我们可以通过如“导入时对行进行计数，保存 count 的统计信息”，或者在查询时“仅扫描某一列数据，获得 count 值”的方式，只需很小的开销，即可获得查询结果。但是在 Doris 的聚合模型中，这种查询的开销非常大。

以刚才的数据为例：

batch 1

user_id	date	cost
10001	2017/11/20	50
10002	2017/11/21	39

batch 2

user_id	date	cost
10001	2017/11/20	1
10001	2017/11/21	5
10003	2017/11/22	22

因为最终的聚合结果为：

user_id	date	cost
10001	2017/11/20	51
10001	2017/11/21	5
10002	2017/11/21	39
10003	2017/11/22	22

所以，`select count(*)from table;` 的正确结果应该为 4。但如果只扫描 `user_id` 这一列，如果加上查询时聚合，最终得到的结果是 3（10001, 10002, 10003）。而如果不加查询时聚合，则得到的结果是 5（两批次一共 5 行数据）。可见这两个结果都是不对的。

为了得到正确的结果，必须同时读取 `user_id` 和 `date` 这两列的数据，再加上查询时聚合，才能返回 4 这个正确的结果。也就是说，在 count(*) 查询中，Doris 必须扫描所有的 AGGREGATE KEY 列（这里就是`user_id date`），并且聚合后，才能得到语意正确的结果。 当聚合列非常多时，count(*) 查询需要扫描大量的数据。

因此，当业务上有频繁的 count(*) 查询时，建议用户通过增加一个值恒为 1 的，聚合类型为 SUM 的列来模拟 count(*)。如刚才的例子中的表结构，我们修改如下：

ColumnName	Type	AggregateType	Comment
user_id	BIGINT		用户 id
date	DATE		数据灌入日期
cost	BIGINT	SUM	用户总消费
count	BIGINT	SUM	用于计算 count

增加一个 count 列，并且导入数据中，该列值恒为 1。则 `select count(*)from table;` 的结果等价于 `select sum(count)from table;`。而后者的查询效率将远高于前者。不过这种方式也有使用限制，就是用户需要自行保证，不会重复导入 AGGREGATE KEY 列都相同地行。否则，`select sum(count)from table;` 只能表述原始导入的行数，而不是 `select count(*)from table;` 的语义。

另一种方式，就是将如上的 `count` 列的聚合类型改为 REPLACE，且依然值恒为 1。那么 `select sum(count) from table;` 和 `select count(*)from table;` 的结果将是一致的。并且这种方式，没有导入重复行的限制。

2.8.2.5.3 Unique 模型的写时合并实现

Unique 模型的写时合并实现没有聚合模型的局限性，还是以刚才的数据为例，写时合并为每次导入的 rowset 增加了对应的 delete bitmap，来标记哪些数据被覆盖。第一批数据导入后状态如下

batch 1

user_id	date	cost	delete bit
10001	2017/11/20	50	FALSE
10002	2017/11/21	39	FALSE

当第二批数据导入完成后，第一批数据中重复的行就会被标记为已删除，此时两批数据状态如下

batch 1

user_id	date	cost	delete bit
10001	2017/11/20	50	TRUE
10002	2017/11/21	39	FALSE

batch 2

user_id	date	cost	delete bit
10001	2017/11/20	1	FALSE
10001	2017/11/21	5	FALSE
10003	2017/11/22	22	FALSE

在查询时，所有在 delete bitmap 中被标记删除的数据都不会读出来，因此也无需进行做任何数据聚合，上述数据中有效地行数为 4 行，查询出的结果也应该是 4 行，也就可以采取开销最小的方式来获取结果，即前面提到的 “仅扫描某一列数据，获得 count 值” 的方式。

在测试环境中，count(*) 查询在 Unique 模型的写时合并实现上的性能，相比聚合模型有 10 倍以上的提升。

2.8.2.5.4 Duplicate 模型

Duplicate 模型没有聚合模型的这个局限性。因为该模型不涉及聚合语意，在做 count(*) 查询时，任意选择一列查询，即可得到语意正确的结果。

2.8.2.5.5 Key 列的不同意义

Duplicate、Aggregate、Unique 模型，都会在建表指定 Key 列，然而实际上是有所区别的：对于 Duplicate 模型，表的 Key 列，可以认为只是 “排序列”，并非起到唯一标识的作用。而 Aggregate、Unique 模型这种聚合类型的表，Key 列是兼顾 “排序列” 和 “唯一标识列”，是真正意义上的 “Key 列”。

2.8.2.5.6 模型选择建议

因为数据模型在建表时就已经确定，且无法修改。所以，选择一个合适的数据模型非常重要。

1. Aggregate 模型可以通过预聚合，极大地降低聚合查询时所需扫描的数据量和查询的计算量，非常适合有固定模式的报表类查询场景。但是该模型对 count(*) 查询很不友好。同时因为固定了 Value 列上的聚合方式，在进行其他类型的聚合查询时，需要考虑语意正确性。
2. Unique 模型针对需要唯一主键约束的场景，可以保证主键唯一性约束。但是无法利用 ROLLUP 等预聚合带来的查询优势。对于聚合查询有较高性能需求的用户，推荐使用自 1.2 版本加入的写时合并实现。
3. Duplicate 适合任意维度的 Ad-hoc 查询。虽然同样无法利用预聚合的特性，但是不受聚合模型的约束，可以发挥列存模型的优势（只读取相关列，而不需要读取所有 Key 列）。
4. 如果有部分列更新的需求，请查阅文档主键模型部分列更新 与聚合模型部份列更新 获取相关使用建议。

2.8.3 数据划分

2.8.3.1 数据分布概念

在 Doris 中，数据分布通过合理的分区和分桶策略，将数据高效地映射到各个数据分片（Tablet）上，从而充分利用多节点的存储和计算能力，支持大规模数据的高效存储和查询。

2.8.3.1.1 数据分布概览

数据写入

数据写入时，Doris 首先根据表的分区策略将数据行分配到对应的分区。接着，根据分桶策略将数据行进一步映射到分区内的具体分片，从而确定了数据行的存储位置。

查询执行

查询运行时，Doris 的优化器会根据分区和分桶策略裁剪数据，最大化减少扫描范围。在涉及 JOIN 或聚合查询时，可能会发生跨节点的数据传输（Shuffle）。合理的分区和分桶设计可以减少 Shuffle 并充分利用 Colocate Join 优化查询性能。

2.8.3.1.2 节点与存储架构

节点类型

Doris 集群由以下两种节点组成：

- FE 节点（Frontend）：管理集群元数据（如表、分片），负责 SQL 的解析与执行规划。
- BE 节点（Backend）：存储数据，负责计算任务的执行。BE 的结果汇总后返回至 FE，再返回给用户。

数据分片（Tablet）

BE 节点的存储数据分片的数据，每个分片是 Doris 中数据管理的最小单元，也是数据移动和复制的基本单位。

2.8.3.1.3 分区策略

分区是数据组织的第一层逻辑划分，用于将表中的数据划分为更小的子集。Doris 提供以下两种分区类型和三种分区模式：

分区类型

- Range 分区：根据分区列的值范围将数据行分配到对应分区。
- List 分区：根据分区列的具体值将数据行分配到对应分区。

分区模式

- 手动分区：用户手动创建分区（如建表时指定或通过 ALTER 语句增加）。
- 动态分区：系统根据时间调度规则自动创建分区，但写入数据时不会按需创建分区。
- 自动分区：数据写入时，系统根据需要自动创建相应的分区，使用时注意脏数据生成过多的分区。

2.8.3.1.4 分桶策略

分桶是数据组织的第二层逻辑划分，用于在分区内将数据行进一步划分到更小的单元。Doris 支持以下两种分桶方式：

- Hash 分桶：通过计算分桶列值的 `crc32` 哈希值，并对分桶数取模，将数据行均匀分布到分片中。
- Random 分桶：随机分配数据行到分片中。使用 Random 分桶时，可以使用 `load_to_single_tablet` 优化小规模数据的快速写入。

2.8.3.1.5 数据分布优化

Colocate Join

对于需要频繁进行 JOIN 或聚合查询的大表，可以启用 Colocate 策略，将相同分桶列值的数据放置在同一物理节点上，减少跨节点的数据传输，从而显著提升查询性能。

分区裁剪

查询时，Doris 可以通过过滤条件裁剪掉不相关的分区，从而减少数据扫描范围，降低 I/O 开销。

分桶并行

查询时，合理的分桶数可以充分利用机器的计算资源和 I/O 资源。

2.8.3.1.6 数据分布目标

1. 均匀数据分布确保数据均匀分布在各 BE 节点上，避免数据倾斜导致部分节点过载，从而提高系统整体性能。
2. 优化查询性能合理的分区裁剪可以大幅减少扫描的数据量，合理的分桶数可以提升计算并行度，合理利用 Colocate 可以降低 Shuffle 成本，提升 JOIN 和聚合查询效率。
3. 灵活数据管理

- 按时间分区保存冷数据（HDD）与热数据（SSD）。
- 定期删除历史分区释放存储空间。

4. 控制元数据规模每个分片的元数据存储在 FE 和 BE 中，因此需要合理控制分片数量。经验值建议：

- 每 1000 万分片，FE 至少需 100G 内存。
- 单个 BE 承载的分片数应小于 2 万。

5. 优化写入吞吐

- 分桶数应合理控制（建议 < 128），以避免写入性能下降。

- 每次写入的分区数量应适量（建议每次写入少量分区）。

通过精心设计和管理分区与分桶策略，Doris 能够高效地支持大规模数据的存储与查询处理，满足各种复杂业务需求。

2.8.3.2 手动分区

2.8.3.2.1 分区列

- 分区列可以指定一列或多列，分区列必须为 KEY 列。
- 不论分区列是什么类型，在写分区值时，都需要加双引号。
- 分区数量理论上没有上限。但默认限制每张表 4096 个分区，如果想突破这个限制，可以修改 FE 配置max_multi_partition_num和max_dynamic_partition_num。
- 当不使用分区建表时，系统会自动生成一个和表名同名的，全值范围的分区。该分区对用户不可见，并且不可删改。
- 创建分区时不可添加范围重叠的分区。

2.8.3.2.2 Range 分区

分区列通常为时间列，以方便的管理新旧数据。Range 分区支持的列类型 DATE, DATETIME, TINYINT, SMALLINT, INT, BIGINT, LARGEINT。

分区信息，支持四种写法：

1. FIXED RANGE：定义分区的左闭右开区间。

```
PARTITION BY RANGE(col1[, col2, ...])
(
    PARTITION partition_name1 VALUES [("k1-lower1", "k2-lower1", "k3-lower1",...), ("k1-upper1",
        ↪ "k2-upper1", "k3-upper1", ...)),
    PARTITION partition_name2 VALUES [("k1-lower1-2", "k2-lower1-2", ...), ("k1-upper1-2",
        ↪ MAXVALUE, ))
)
```

示例如下：

```
PARTITION BY RANGE(`date`)
(
    PARTITION `p201701` VALUES [("2017-01-01"),  ("2017-02-01")),
    PARTITION `p201702` VALUES [("2017-02-01"), ("2017-03-01")),
    PARTITION `p201703` VALUES [("2017-03-01"), ("2017-04-01"))
)
```

2. LESS THAN：仅定义分区上界。下界由上一个分区的上界决定。

```
PARTITION BY RANGE(col1[, col2, ...])
(
    PARTITION partition_name1 VALUES LESS THAN MAXVALUE | ("value1", "value2", ...),
    PARTITION partition_name2 VALUES LESS THAN MAXVALUE | ("value1", "value2", ...)
)
```

示例如下：

```
PARTITION BY RANGE(`date`)
(
    PARTITION `p201701` VALUES LESS THAN ("2017-02-01"),
    PARTITION `p201702` VALUES LESS THAN ("2017-03-01"),
    PARTITION `p201703` VALUES LESS THAN ("2017-04-01"),
    PARTITION `p2018` VALUES [("2018-01-01"), ("2019-01-01")),
    PARTITION `other` VALUES LESS THAN (MAXVALUE)
)
```

3. BATCH RANGE：批量创建数字类型和时间类型的 RANGE 分区，定义分区的左闭右开区间，设定步长。

```
PARTITION BY RANGE(int_col)
(
    FROM (start_num) TO (end_num) INTERVAL interval_value
)

PARTITION BY RANGE(date_col)
(
    FROM ("start_date") TO ("end_date") INTERVAL num YEAR | num MONTH | num WEEK | num DAY | 1
        ↪ HOUR
)
```

示例如下：

```
PARTITION BY RANGE(age)
(
    FROM (1) TO (100) INTERVAL 10
)

PARTITION BY RANGE(`date`)
(
    FROM ("2000-11-14") TO ("2021-11-14") INTERVAL 2 YEAR
)
```

4.MULTI RANGE：批量创建 RANGE 分区，定义分区的左闭右开区间。示例如下：

```
PARTITION BY RANGE(col)
(
   FROM ("2000-11-14") TO ("2021-11-14") INTERVAL 1 YEAR,
   FROM ("2021-11-14") TO ("2022-11-14") INTERVAL 1 MONTH,
   FROM ("2022-11-14") TO ("2023-01-03") INTERVAL 1 WEEK,
   FROM ("2023-01-03") TO ("2023-01-14") INTERVAL 1 DAY,
   PARTITION p_20230114 VALUES [('2023-01-14'), ('2023-01-15'))
)
```

2.8.3.2.3 List 分区

分区列支持 BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, DATE, DATETIME, CHAR, VARCHAR 数据类型，分区值为枚举值。只有当数据为目标分区枚举值其中之一时，才可以命中分区。

Partition 支持通过 VALUES IN (...) 来指定每个分区包含的枚举值。

举例如下：

```
PARTITION BY LIST(city)
(
    PARTITION `p_cn` VALUES IN ("Beijing", "Shanghai", "Hong Kong"),
    PARTITION `p_usa` VALUES IN ("New York", "San Francisco"),
    PARTITION `p_jp` VALUES IN ("Tokyo")
)
```

List 分区也支持多列分区，示例如下：

```
PARTITION BY LIST(id, city)
(
    PARTITION p1_city VALUES IN (("1", "Beijing"), ("1", "Shanghai")),
    PARTITION p2_city VALUES IN (("2", "Beijing"), ("2", "Shanghai")),
    PARTITION p3_city VALUES IN (("3", "Beijing"), ("3", "Shanghai"))
)
```

2.8.3.2.4 NULL 分区

PARTITION 列默认必须为 NOT NULL 列，如果需要使用 NULL 列，应设置 session variable allow_partition_column_nullable = true。对于 LIST PARTITION，我们支持真正的 NULL 分区。对于 RANGE PARTITION，NULL 值会被划归最小的 LESS THAN 分区。分列如下：

1. LIST 分区

```
mysql> create table null_list(
    -> k0 varchar null
    -> )
    -> partition by list (k0)
```

```
    -> (
    -> PARTITION pX values in ((NULL))
    -> )
    -> DISTRIBUTED BY HASH(`k0`) BUCKETS 1
    -> properties("replication_num" = "1");
Query OK, 0 rows affected (0.11 sec)

mysql> insert into null_list values (null);
Query OK, 1 row affected (0.19 sec)

mysql> select * from null_list;
+------+
| k0   |
+------+
| NULL |
+------+
1 row in set (0.18 sec)
```

2. RANGE 分区 —— 归属最小的 LESS THAN 分区

```
mysql> create table null_range(
    -> k0 int null
    -> )
    -> partition by range (k0)
    -> (
    -> PARTITION p10 values less than (10),
    -> PARTITION p100 values less than (100),
    -> PARTITION pMAX values less than (maxvalue)
    -> )
    -> DISTRIBUTED BY HASH(`k0`) BUCKETS 1
    -> properties("replication_num" = "1");
Query OK, 0 rows affected (0.12 sec)

mysql> insert into null_range values (null);
Query OK, 1 row affected (0.19 sec)

mysql> select * from null_range partition(p10);
+------+
| k0   |
+------+
| NULL |
+------+
1 row in set (0.18 sec)
```

3. RANGE 分区 —— 没有 LESS THAN 分区时，无法插入

```
mysql> create table null_range2(
    -> k0 int null
    -> )
    -> partition by range (k0)
    -> (
    -> PARTITION p200 values [("100"), ("200"))
    -> )
    -> DISTRIBUTED BY HASH(`k0`) BUCKETS 1
    -> properties("replication_num" = "1");
Query OK, 0 rows affected (0.13 sec)

mysql> insert into null_range2 values (null);
ERROR 5025 (HY000): Insert has filtered data in strict mode, tracking_url=......
```

2.8.3.3 动态分区

动态分区会按照设定的规则，滚动添加、删除分区，从而实现对表分区的生命周期管理（TTL），减少数据存储压力。在日志管理，时序数据管理等场景，通常可以使用动态分区能力滚动删除过期的数据。

下图中展示了使用动态分区进行生命周期管理，其中指定了以下规则：

- 动态分区调度单位 dynamic_partition.time_unit 为 DAY，按天组织分区；
- 动态分区起始偏移量 dynamic_partition.start 设置为 -1，保留一天前分区；
- 动态分区结束偏移量 dynamic_partition.end 设置为 2，保留未来两天分区

依据以上规则，随着时间推移，总会保留 4 个分区，即过去一天分区，当天分区与未来两天分区：

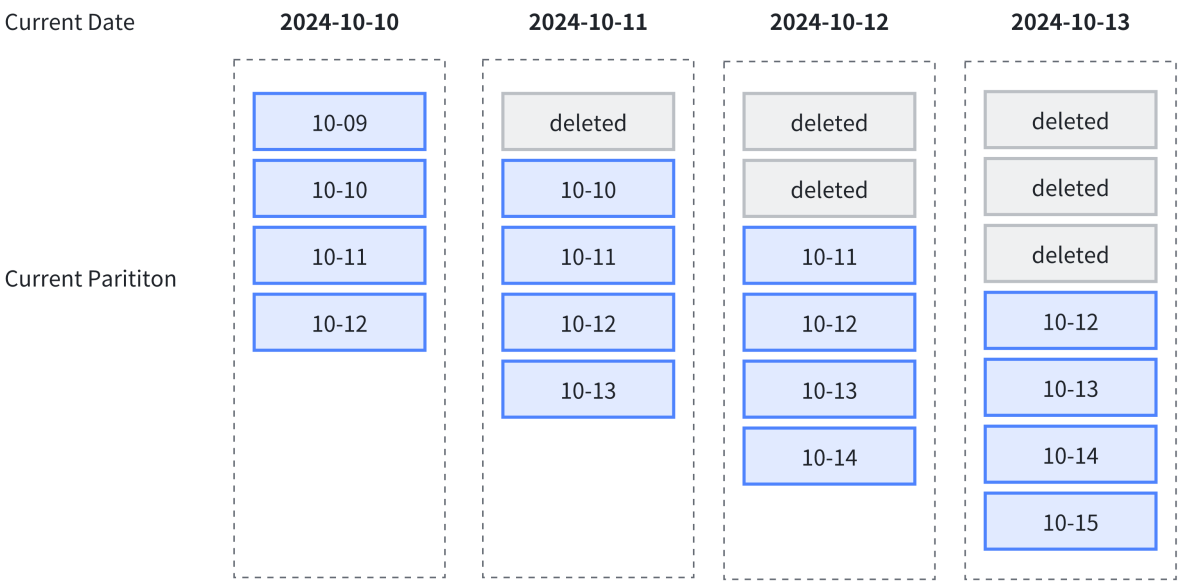


图 27: dynamic-partition

2.8.3.3.1 使用限制

在使用动态分区时，需要遵守以下规则：

- 动态分区与跨集群复制（CCR）同时使用时会失效；
- 动态分区只支持在 DATE/DATETIME 列上进行 Range 类型的分区；
- 动态分区只支持单一分区键。

2.8.3.3.2 创建动态分区

在建表时，通过指定 dynamic_partition 属性，可以创建动态分区表。

```
CREATE TABLE test_dynamic_partition(
    order_id    BIGINT,
    create_dt   DATE,
    username    VARCHAR(20)
)
DUPLICATE KEY(order_id)
PARTITION BY RANGE(create_dt) ()
DISTRIBUTED BY HASH(order_id) BUCKETS 10
PROPERTIES(
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-1",
```

```
    "dynamic_partition.end" = "2",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.create_history_partition" = "true"
);
```

上例中创建了动态分区表，指定以下

详细 dynamic_partition 参数可以参考动态分区参数说明。

2.8.3.3.3 管理动态分区

修改动态分区属性

> **提示：**
>
> 在使用 ALTER TABLE 语句修改动态分区时，不会立即生效。会以 dynamic_partition_check_interval_seconds 参数指定的时间间隔轮训检查 dynamic partition 分区，完成需要的分区创建与删除操作。

下例中通过 ALTER TABLE 语句，将非动态分区表修改为动态分区：

```
CREATE TABLE test_dynamic_partition(
    order_id    BIGINT,
    create_dt   DATE,
    username    VARCHAR(20)
)
DUPLICATE KEY(order_id)
DISTRIBUTED BY HASH(order_id) BUCKETS 10;

ALTER TABLE test_partition SET (
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-1",
    "dynamic_partition.end" = "2",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.create_history_partition" = "true"
);
```

查看动态分区调度情况

通过SHOW-DYNAMIC-PARTITION 可以查看当前数据库下，所有动态分区表的调度情况：

```
SHOW DYNAMIC PARTITION TABLES;
+--
    ↪ ---------+--------+----------+-------------+------+--------+---------+-----------+-----------------+------
    ↪
```

```
| TableName | Enable | TimeUnit | Start       | End  | Prefix | Buckets | StartOf   |
    ↪ LastUpdateTime | LastSchedulerTime   | State  | LastCreatePartitionMsg |
    ↪ LastDropPartitionMsg | ReservedHistoryPeriods  |
+--
    ↪ ---------+--------+----------+-------------+------+--------+---------+-----------+-----------------+------
    ↪
| d3        | true   | WEEK     | -3          | 3    | p      | 1       | MONDAY    | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | [2021-12-01,2021-12-31] |
| d5        | true   | DAY      | -7          | 3    | p      | 32      | N/A       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | NULL                    |
| d4        | true   | WEEK     | -3          | 3    | p      | 1       | WEDNESDAY | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | NULL                    |
| d6        | true   | MONTH    | -2147483648 | 2    | p      | 8       | 3rd       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | NULL                    |
| d2        | true   | DAY      | -3          | 3    | p      | 32      | N/A       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | NULL                    |
| d7        | true   | MONTH    | -2147483648 | 5    | p      | 8       | 24th      | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                    | N/A
    ↪                 | NULL                    |
+--
    ↪ ---------+--------+----------+-------------+------+--------+---------+-----------+-----------------+------
    ↪
7 rows in set (0.02 sec)
```

历史分区管理

在使用 start 与 end 属性指定动态分区数量时，为了避免一次性创建所有的分区造成等待时间过长，不会创建历史分区，只会创建当前时间以后得分区。如果需要一次性创建所有分区，需要开启 create_history_
↪ partition 参数。

例如当前日期为 2024-10-11，指定 start = -2，end = 2：

- 如果指定了 create_history_partition = true，立即创建所有分区，即 [10-09, 10-13] 五个分区；
- 如果指定了 create_history_partition = false，只创建包含 10-11 以后的分区，即 [10-11, 10-13] 三个分区。

2.8.3.3.4 动态分区参数说明

动态分区属性参数

动态分区的规则参数以 dynamic_partition 为前缀，可以设置以下规则参数：

参数	必选	说明
dynamic_partition.enable	否	是否开启动态分区特性。可以指定为 TRUE 或 FALSE。如果指定了动态分区其他必填参数，默认为 TRUE。
dynamic_partition.time_unit	是	动态分区调度的单位。可指定为 HOUR、DAY、WEEK、MONTH、YEAR。分别表示按小时、按天、按星期、按月、按年进行分区创建或删除：
dynamic_partition.start	否	动态分区的起始偏移，为负数。默认值为 -2147483648，即不删除历史分区。根据 time_unit 属性的不同，以当天（星期/月）为基准，分区范围在此偏移之前的分区将会被删除。此偏移之后至当前时间的历史分区如不存在，是否创建取决于 dynamic_partition.create_history_partition。
dynamic_partition.end	是	动态分区的结束偏移，为正数。根据 time_unit 属性的不同，以当天（星期/月）为基准，提前创建对应范围的分区。
dynamic_partition.prefix	是	动态创建的分区名前缀。
dynamic_partition.buckets	否	动态创建的分区所对应的分桶数。设置该参数后会覆盖 DISTRIBUTED 中指定的分桶数。量。
dynamic_partition.replication_num	否	动态创建的分区所对应的副本数量，如果不填写，则默认为该表创建时指定的副本数量。
dynamic_partition.create_history_partition	否	默认为 false。当置为 true 时，Doris 会自动创建所有分区，具体创建规则见下文。同时，FE 的参数 max_dynamic_partition_num 会限制总分区数量，以避免一次性创建过多分区。当期望创建的分区个数大于 max_dynamic_partition_num 值时，操作将被禁止。当不指定 start 属性时，该参数不生效。
dynamic_partition.history_partition_num	否	当create_history_partition 为 true 时，该参数用于指定创建历史分区数量。默认值为 -1，即未设置。该变量与 dynamic_partition.start 作用相同，建议同时只设置一个。
dynamic_partition.start_day_of_week	否	当 time_unit 为 WEEK 时，该参数用于指定每周的起始点。取值为 1 到 7。其中 1 表示周一，7 表示周日。默认为 1，即表示每周以周一为起始点。
dynamic_partition.start_day_of_month	否	当 time_unit 为 MONTH 时，该参数用于指定每月的起始日期。取值为 1 到 28。其中 1 表示每月 1 号，28 表示每月 28 号。默认为 1，即表示每月以 1 号为起始点。暂不支持以 29、30、31 号为起始日，以避免因闰年或闰月带来的歧义。

参数	必选	说明
dynamic_partition.reserved_history_periods	否	需要保留的历史分区的时间范围。当dynamic_partition.time_unit 设置为“DAY/WEEK/MONTH/YEAR”时，需要以 [yyyy-MM-dd,yyyy-MM-dd],[...,...] 格式进行设置。当dynamic_partition.time_unit 设置为“HOUR”时，需要以 [yyyy-MM-dd HH:mm:ss,yyyy-MM-dd HH:mm:ss],[...,...] 的格式来进行设置。如果不设置，默认为 "NULL"。
dynamic_partition.time_zone	否	动态分区时区，默认为当前服务器的系统时区，如 Asia/Shanghai。更多时区设置可以参考时区管理。

FE 配置参数

可以在 FE 配置文件或通过 ADMIN SET FRONTEND CONFIG 命令修改 FE 中的动态分区参数配置：

参数	默认值	说明
dynamic_partition_enable	false	是否开启 Doris 的动态分区功能。该参数只影响动态分区表的分区操
dynamic_partition_check_interval_seconds	600	动态分区线程的执行频率，单位为秒。
max_dynamic_partition_num	500	用于限制创建动态分区表时可以创建的最大分区数，避免一次创建过

2.8.3.3.5 动态分区最佳实践

示例 1：按天分区，只保留过去 7 天的及当天分区，并且预先创建未来 3 天的分区。

```
CREATE TABLE tbl1 (
    order_id    BIGINT,
    create_dt   DATE,
    username    VARCHAR(20)
)
PARTITION BY RANGE(create_dt) ()
DISTRIBUTED BY HASH(create_dt)
PROPERTIES (
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-7",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "32"
);
```

示例 2：按月分区，不删除历史分区，并且预先创建未来 2 个月的分区。同时设定以每月 3 号为起始日。

```
CREATE TABLE tbl1 (
    order_id    BIGINT,
    create_dt   DATE,
```

```
    username    VARCHAR(20)
)
PARTITION BY RANGE(create_dt) ()
DISTRIBUTED BY HASH(create_dt)
PROPERTIES (
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "MONTH",
    "dynamic_partition.end" = "2",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "8",
    "dynamic_partition.start_day_of_month" = "3"
);
```

示例 3：按天分区，保留过去 10 天及未来 10 天分区，并且保留 [2020-06-01,2020-06-20] 及 [2020-10-31,2020-11-15] 期间的历史数据。

```
CREATE TABLE tbl1 (
    order_id    BIGINT,
    create_dt   DATE,
    username    VARCHAR(20)
)
PARTITION BY RANGE(create_dt) ()
DISTRIBUTED BY HASH(create_dt)
PROPERTIES (
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-10",
    "dynamic_partition.end" = "10",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "8",
    "dynamic_partition.reserved_history_periods"="[2020-06-01,2020-06-20],[2020-10-31,2020-11-15]
        ↪ "
);
```

2.8.3.4 自动分区

2.8.3.4.1 使用场景

自动分区功能主要解决了用户预期基于某列对表进行分区操作，但该列的数据分布比较零散或者难以预测，在建表或调整表结构时难以准确创建所需分区，或者分区数量过多以至于手动创建过于繁琐的问题。

以时间类型分区列为例，在动态分区功能中，我们支持了按特定时间周期自动创建新分区以容纳实时数据。对于实时的用户行为日志等场景该功能基本能够满足需求。但在一些更复杂的场景下，例如处理非实时数据时，分区列与当前系统时间无关，且包含大量离散值。此时为提高效率我们希望依据此列对数据进行分区，但数据实际可能涉及的分区无法预先掌握，或者预期所需分区数量过大。这种情况下动态分区或者手动创建分区无法满足我们的需求，自动分区功能很好地覆盖了此类需求。

假设我们的表 DDL 如下：

```
CREATE TABLE `DAILY_TRADE_VALUE`
(
    `TRADE_DATE`              datev2 NOT NULL COMMENT '交易日期',
    `TRADE_ID`                varchar(40) NOT NULL COMMENT '交易编号',
    ......
)
UNIQUE KEY(`TRADE_DATE`, `TRADE_ID`)
PARTITION BY RANGE(`TRADE_DATE`)
(
    PARTITION p_2000 VALUES [('2000-01-01'), ('2001-01-01')),
    PARTITION p_2001 VALUES [('2001-01-01'), ('2002-01-01')),
    PARTITION p_2002 VALUES [('2002-01-01'), ('2003-01-01')),
    PARTITION p_2003 VALUES [('2003-01-01'), ('2004-01-01')),
    PARTITION p_2004 VALUES [('2004-01-01'), ('2005-01-01')),
    PARTITION p_2005 VALUES [('2005-01-01'), ('2006-01-01')),
    PARTITION p_2006 VALUES [('2006-01-01'), ('2007-01-01')),
    PARTITION p_2007 VALUES [('2007-01-01'), ('2008-01-01')),
    PARTITION p_2008 VALUES [('2008-01-01'), ('2009-01-01')),
    PARTITION p_2009 VALUES [('2009-01-01'), ('2010-01-01')),
    PARTITION p_2010 VALUES [('2010-01-01'), ('2011-01-01')),
    PARTITION p_2011 VALUES [('2011-01-01'), ('2012-01-01')),
    PARTITION p_2012 VALUES [('2012-01-01'), ('2013-01-01')),
    PARTITION p_2013 VALUES [('2013-01-01'), ('2014-01-01')),
    PARTITION p_2014 VALUES [('2014-01-01'), ('2015-01-01')),
    PARTITION p_2015 VALUES [('2015-01-01'), ('2016-01-01')),
    PARTITION p_2016 VALUES [('2016-01-01'), ('2017-01-01')),
    PARTITION p_2017 VALUES [('2017-01-01'), ('2018-01-01')),
    PARTITION p_2018 VALUES [('2018-01-01'), ('2019-01-01')),
    PARTITION p_2019 VALUES [('2019-01-01'), ('2020-01-01')),
    PARTITION p_2020 VALUES [('2020-01-01'), ('2021-01-01')),
    PARTITION p_2021 VALUES [('2021-01-01'), ('2022-01-01'))
)
DISTRIBUTED BY HASH(`TRADE_DATE`) BUCKETS 10
PROPERTIES (
  "replication_num" = "1"
);
```

该表内存储了大量业务历史数据，依据交易发生的日期进行分区。可以看到在建表时，我们需要预先手动创建分区。如果分区列的数据范围发生变化，例如上表中增加了 2022 年的数据，则我们需要通过ALTER-TABLE-PARTITION对表的分区进行更改。如果这种分区需要变更，或者进行更细粒度的细分，修改起来非常繁琐。此时我们就可以使用 AUTO PARTITION 改写该表 DDL。

2.8.3.4.2 语法

建表时，使用以下语法填充CREATE-TABLE 中的 partitions_definition 部分：

1. AUTO RANGE PARTITION:

```
AUTO PARTITION BY RANGE(<partition_expr>)
<origin_partitions_definition>
```

其中

```
partition_expr ::= date_trunc ( <partition_column>, '<interval>' )
```

2. AUTO LIST PARTITION:

```
  AUTO PARTITION BY LIST(`partition_col1` [, `partition_col2`, ...])
  <origin_partitions_definition>
```

用法示例

1. AUTO RANGE PARTITION

```
CREATE TABLE `date_table` (
    `TIME_STAMP` datev2 NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`TIME_STAMP`)
AUTO PARTITION BY RANGE (date_trunc(`TIME_STAMP`, 'month'))
()
DISTRIBUTED BY HASH(`TIME_STAMP`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

2. AUTO LIST PARTITION

```
CREATE TABLE `str_table` (
    `str` varchar not null
) ENGINE=OLAP
DUPLICATE KEY(`str`)
AUTO PARTITION BY LIST (`str`)
()
DISTRIBUTED BY HASH(`str`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

LIST 自动分区支持多个分区列，分区列写法同普通 LIST 分区一样：`AUTO PARTITION BY LIST (`col1`, `col2 ↪ `, ...)`

约束

1. 在 AUTO LIST PARTITION 中，分区名长度不得超过 50. 该长度来自于对应数据行上各分区列内容的拼接与转义，因此实际容许长度可能更短。
2. 在 AUTO RANGE PARTITION 中，分区函数仅支持 date_trunc，分区列仅支持 DATE 或者 DATETIME 类型；
3. 在 AUTO LIST PARTITION 中，不支持函数调用，分区列支持 BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, DATE, DATETIME, CHAR, VARCHAR 数据类型，分区值为枚举值。
4. 在 AUTO LIST PARTITION 中，分区列的每个当前不存在对应分区的取值，都会创建一个独立的新 PARTITION。

NULL 值分区

当开启 session variable `allow_partition_column_nullable` 后：

1. 对于 AUTO LIST PARTITION，可以使用 NULLABLE 列作为分区列，会正常创建对应的 NULL 值分区：

```
create table auto_null_list(
  k0 varchar null
)
auto partition by list (k0)
(
)
DISTRIBUTED BY HASH(`k0`) BUCKETS 1
properties("replication_num" = "1");

insert into auto_null_list values (null);

select * from auto_null_list;
+------+
| k0   |
+------+
| NULL |
+------+

select * from auto_null_list partition(pX);
+------+
| k0   |
+------+
| NULL |
+------+
```

2. 对于 AUTO RANGE PARTITION，不支持 NULLABLE 列作为分区列。

```
    CREATE TABLE `range_table_nullable` (
      `k1` INT,
      `k2` DATETIMEV2(3),
      `k3` DATETIMEV2(6)
    ) ENGINE=OLAP
    DUPLICATE KEY(`k1`)
    AUTO PARTITION BY RANGE (date_trunc(`k2`, 'day'))
    ()
    DISTRIBUTED BY HASH(`k1`) BUCKETS 16
    PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
    );

   ERROR 1105 (HY000): errCode = 2, detailMessage = AUTO RANGE PARTITION doesn't support NULL
       ↪ column
```

2.8.3.4.3 场景示例

在使用场景一节中的示例，在使用 AUTO PARTITION 后，该表 DDL 可以改写为：

```
CREATE TABLE `DAILY_TRADE_VALUE`
(
    `TRADE_DATE`              datev2 NOT NULL,
    `TRADE_ID`                varchar(40) NOT NULL,
    ......
)
UNIQUE KEY(`TRADE_DATE`, `TRADE_ID`)
AUTO PARTITION BY RANGE (date_trunc(`TRADE_DATE`, 'year'))
(
)
DISTRIBUTED BY HASH(`TRADE_DATE`) BUCKETS 10
PROPERTIES (
  "replication_num" = "1"
);
```

以此表只有两列为例，此时新表没有默认分区：

```
show partitions from `DAILY_TRADE_VALUE`;
Empty set (0.12 sec)
```

经过插入数据后再查看，发现该表已经创建了对应的分区：

```
insert into `DAILY_TRADE_VALUE` values ('2012-12-13', 1), ('2008-02-03', 2), ('2014-11-11', 3);

show partitions from `DAILY_TRADE_VALUE`;
```

```
+--
    ↪ -----------+-----------------+----------------+---------------------+--------+--------------+------------
    ↪
| PartitionId | PartitionName   | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range                                                                                      |
    ↪ DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable |
+--
    ↪ -----------+-----------------+----------------+---------------------+--------+--------------+------------
    ↪
| 180060      | p20080101000000 | 2              | 2023-09-18 21:49:29 | NORMAL | TRADE_DATE   |
    ↪ [types: [DATEV2]; keys: [2008-01-01]; ..types: [DATEV2]; keys: [2009-01-01]; ) | TRADE_
    ↪ DATE      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
    ↪                     | NULL                     | 0.000    | false      | tag.location.
    ↪ default: 1 | true      |
| 180039      | p20120101000000 | 2              | 2023-09-18 21:49:29 | NORMAL | TRADE_DATE   |
    ↪ [types: [DATEV2]; keys: [2012-01-01]; ..types: [DATEV2]; keys: [2013-01-01]; ) | TRADE_
    ↪ DATE      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
    ↪                     | NULL                     | 0.000    | false      | tag.location.
    ↪ default: 1 | true      |
| 180018      | p20140101000000 | 2              | 2023-09-18 21:49:29 | NORMAL | TRADE_DATE   |
    ↪ [types: [DATEV2]; keys: [2014-01-01]; ..types: [DATEV2]; keys: [2015-01-01]; ) | TRADE_
    ↪ DATE      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
    ↪                     | NULL                     | 0.000    | false      | tag.location.
    ↪ default: 1 | true      |
+--
    ↪ -----------+-----------------+----------------+---------------------+--------+--------------+------------
    ↪
```

经过自动分区功能所创建的 PARTITION，与手动创建的 PARTITION 具有完全一致的功能性质。

2.8.3.4.4 与动态分区联用

Doris 支持自动分区和动态分区同时使用。此时，二者的功能都生效：

1. 自动分区将会自动在数据导入过程中按需创建分区；
2. 动态分区将会自动创建、回收、转储分区。

二者语法功能不存在冲突，同时设置对应的子句/属性即可。请注意，当前时间所在的分区由自动分区还是动态分区创建，是不确定的。不同创建方式会导致分区的名称格式不同。

> 该功能自 Doris 3.0.3 起支持。

最佳实践

需要对分区生命周期设限的场景，可以将 Dynamic Partition 的创建功能关闭，创建分区完全交由 Auto Partition 完成，通过 Dynamic Partition 动态回收分区的功能完成分区生命周期的管理：

```
create table auto_dynamic(
    k0 datetime(6) NOT NULL
)
auto partition by range (date_trunc(k0, 'year'))
(
)
DISTRIBUTED BY HASH(`k0`) BUCKETS 2
properties(
    "dynamic_partition.enable" = "true",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.start" = "-50",
    "dynamic_partition.end" = "0", --- Dynamic Partition 不创建分区
    "dynamic_partition.time_unit" = "year",
    "replication_num" = "1"
);
```

这样我们同时具有了 Auto Partition 的灵活性，且分区名上保持了一致性。

2.8.3.4.5 与自动分桶联用

> 这个功能从 Doris 3.1.2 开始正常工作

只有 AUTO RANGE PARTITION 可以同时使用自动分桶功能。使用此功能时，Doris 将假设表的数据导入是按照时间顺序增量的，每次导入仅涉及一个分区。即是说，这种用法仅推荐用于逐批次增量导入的表。

> 注意！如果数据导入方式不符合上述范式，且同时使用了自动分区和自动分桶，存在新分区的分桶数极不合理的可能，较大影响查询性能。

2.8.3.4.6 分区管理

当启用自动分区后，分区名可以通过 auto_partition_name 函数映射到分区。partitions 表函数可以通过分区名产生详细的分区信息。仍然以 DAILY_TRADE_VALUE 表为例，在我们插入数据后，查看其当前分区：

```
select * from partitions("catalog"="internal","database"="optest","table"="DAILY_TRADE_VALUE")
    ↪ where PartitionName = auto_partition_name('range', 'year', '2008-02-03');
```

```
+--
    ↪ -----------+-----------------+---------------+--------------------+--------+--------------+-----------
    ↪
| PartitionId | PartitionName   | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range                                                                                      |
    ↪ DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize  | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+--
    ↪ -----------+-----------------+---------------+--------------------+--------+--------------+-----------
    ↪
|      127095 | p20080101000000 |              2 | 2024-11-14 17:29:02 | NORMAL | TRADE_DATE    |
    ↪ [types: [DATEV2]; keys: [2008-01-01]; ..types: [DATEV2]; keys: [2009-01-01]; ) | TRADE_
    ↪ DATE       |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                      | `\N`                        | 985.000 B |          0 | tag.location.
    ↪ default: 1 |         1 |                  1 | `\N`           |
+--
    ↪ -----------+-----------------+---------------+--------------------+--------+--------------+-----------
    ↪
```

这样每个分区的 ID 和取值就可以精准地被筛选出，用于后续针对分区的具体操作（例如 `insert overwrite partition`）。

详细语法说明请见：auto_partition_name 函数文档，partitions 表函数文档。

2.8.3.4.7 注意事项

- 如同普通分区表一样，AUTO LIST PARTITION 支持多列分区，语法并无区别。
- 在数据的插入或导入过程中如果创建了分区，而整个导入过程没有完成（失败或被取消），被创建的分区不会被自动删除。
- 使用 AUTO PARTITION 的表，只是分区创建方式上由手动转为了自动。表及其所创建分区的原本使用方法都与非 AUTO PARTITION 的表或分区相同。
- 为防止意外创建过多分区，我们通过FE 配置项中的`max_auto_partition_num`控制了一个 AUTO PARTITION 表最大容纳分区数。如有需要可以调整该值
- 向开启了 AUTO PARTITION 的表导入数据时，Coordinator 发送数据的轮询间隔与普通表有所不同。具体请见BE 配置项中的`olap_table_sink_send_interval_auto_partition_factor`。开启前移（`enable_memtable_on_sink_node = true`）后该变量不产生影响。
- 在使用insert-overwrite插入数据时 AUTO PARTITION 表的行为详见 INSERT OVERWRITE 文档。
- 如果导入创建分区时，该表涉及其他元数据操作（如 Schema Change、 Rebalance），则导入可能失败。

2.8.3.5 数据分桶

一个分区可以根据业务需求进一步划分为多个数据分桶（bucket）。每个分桶都作为一个物理数据分片（tablet）存储。合理的分桶策略可以有效降低查询时的数据扫描量，提升查询性能并增加并发处理能力。

2.8.3.5.1 分桶方式

Doris 支持两种分桶方式：Hash 分桶与 Random 分桶。

Hash 分桶

在创建表或新增分区时，用户需选择一列或多列作为分桶列，并明确指定分桶的数量。在同一分区内，系统会根据分桶键和分桶数量进行哈希计算。哈希值相同的数据会被分配到同一个分桶中。例如，在下图中，p250102 分区根据 region 列被划分为 3 个分桶，哈希值相同的行被归入同一个分桶。

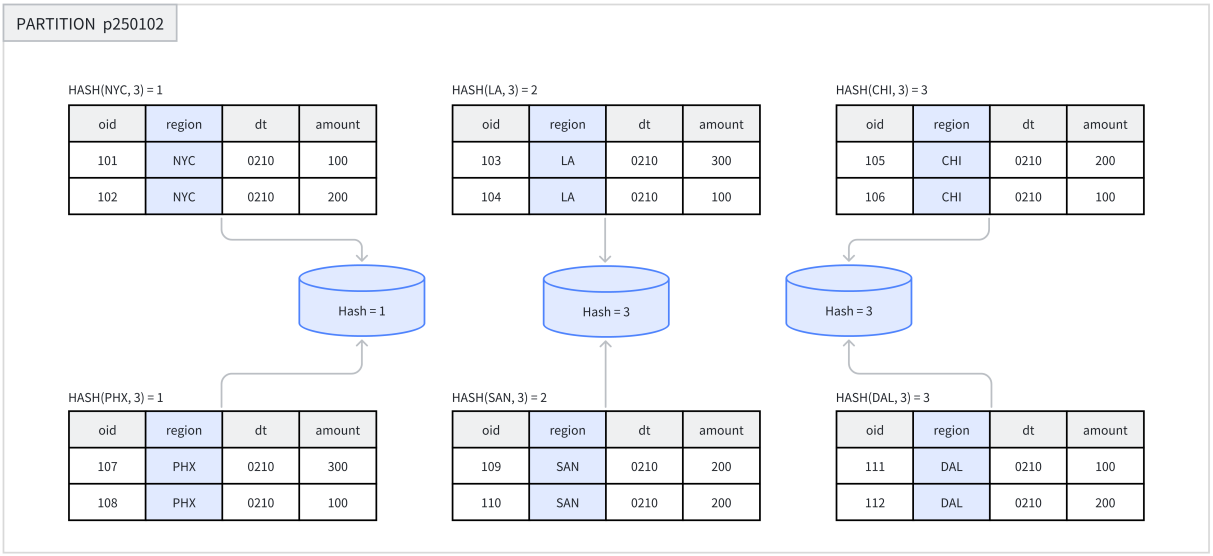


图 28: hash-bucket

推荐在以下场景中使用 Hash 分桶：

- 业务需求频繁基于某个字段进行过滤时，可将该字段作为分桶键，利用 Hash 分桶提高查询效率。
- 当表中的数据分布较为均匀时，Hash 分桶同样是一种有效的选择。

以下示例展示了如何创建带有 Hash 分桶的表。详细语法请参考 CREATE TABLE 语句。

```
CREATE TABLE demo.hash_bucket_tbl(
    oid         BIGINT,
    dt          DATE,
    region      VARCHAR(10),
    amount      INT
)
DUPLICATE KEY(oid)
PARTITION BY RANGE(dt) (
    PARTITION p250101 VALUES LESS THAN("2025-01-01"),
    PARTITION p250102 VALUES LESS THAN("2025-01-02")
)
```

```
DISTRIBUTED BY HASH(region) BUCKETS 8;
```

示例中，通过 DISTRIBUTED BY HASH(region) 指定了创建 Hash 分桶，并选择 region 列作为分桶键。同时，通过 BUCKETS 8 指定了创建 8 个分桶。

Random 分桶

在每个分区中，使用 Random 分桶会随机地将数据分散到各个分桶中，不依赖于某个字段的 Hash 值进行数据划分。Random 分桶能够确保数据均匀分散，从而避免由于分桶键选择不当而引发的数据倾斜问题。

在导入数据时，单次导入作业的每个批次会被随机写入到一个 tablet 中，以此保证数据的均匀分布。例如，在一次操作中，8 个批次的数据被随机分配到 p250102 分区下的 3 个分桶中。

图 29: random-bucket

在使用 Random 分桶时，可以启用单分片导入模式（通过设置 load_to_single_tablet 为 true）。这样，在大规模数据导入过程中，单个批次的数据仅写入一个数据分片，能够提高数据导入的并发度和吞吐量，减少因数据导入和压缩（Compaction）操作造成的写放大问题，从而确保集群稳定性。

在以下场景中，建议使用 Random 分桶：

- 在任意维度分析的场景中，业务没有特别针对某一列频繁进行过滤或关联查询时，可以选择 Random 分桶；
- 当经常查询的列或组合列数据分布极其不均匀时，使用 Random 分桶可以避免数据倾斜。
- Random 分桶无法根据分桶键进行剪裁，会扫描命中分区的所有数据，不建议在点查场景下使用；
- 只有 DUPLICATE 表可以使用 Random 分区，UNIQUE 与 AGGREGATE 表无法使用 Random 分桶；

以下示例展示了如何创建带有 Random 分桶的表。详细语法请参考 CREATE TABLE 语句：

```
CREATE TABLE demo.random_bucket_tbl(
    oid         BIGINT,
    dt          DATE,
    region      VARCHAR(10),
    amount      INT
)
DUPLICATE KEY(oid)
PARTITION BY RANGE(dt) (
    PARTITION p250101 VALUES LESS THAN("2025-01-01"),
    PARTITION p250102 VALUES LESS THAN("2025-01-02")
)
DISTRIBUTED BY RANDOM BUCKETS 8;
```

示例中，通过 DISTRIBUTED BY RANDOM 语句指定了使用 Random 分桶，创建 Random 分桶无需选择分桶键，通过 BUCKETS 8 语句指定创建 8 个分桶。

2.8.3.5.2 选择分桶键

> 提示
>
> 只有 Hash 分桶需要选择分桶键，Random 分桶不需要选择分桶键。

分桶键可以是一列或者多列。如果是 DUPLICATE 表，任何 Key 列与 Value 列都可以作为分桶键。如果是 AGGREGATE 或 UNIQUE 表，为了保证逐渐的聚合性，分桶列必须是 Key 列。

通常情况下，可以根据以下规则选择分桶键：

- 利用查询过滤条件：使用查询中的过滤条件进行 Hash 分桶，有助于数据的剪裁，减少数据扫描量；
- 利用高基数列：选择高基数（唯一值较多）的列进行 Hash 分桶，有助于数据均匀的分散在每一个分桶中；
- 高并发点查场景：建议选择单列或较少列进行分桶。点查可能仅触发一个分桶扫描，不同查询之间触发不同分桶扫描的概率较大，从而减小查询间的 IO 影响。
- 大吞吐查询场景：建议选择多列进行分桶，使数据更均匀分布。若查询条件不能包含所有分桶键的等值条件，将增加查询吞吐，降低单个查询延迟。

2.8.3.5.3 选择分桶数量

在 Doris 中，一个 bucket 会被存储为一个物理文件（tablet）。一个表的 Tablet 数量等于 partition_num（分区数）乘以 bucket_num（分桶数）。一旦指定 Partition 的数量，便不可更改。

在确定 bucket 数量时，需预先考虑机器扩容情况。自 2.0 版本起，Doris 支持根据机器资源和集群信息自动设置分区中的分桶数。

手动设置分桶数

通过 DISTRIBUTED 语句可以指定分桶数量：

```
-- Set hash bucket num to 8
DISTRIBUTED BY HASH(region) BUCKETS 8

-- Set random bucket num to 8
DISTRIBUTED BY RANDOM BUCKETS 8
```

在决定分桶数量时，通常遵循数量与大小两个原则，当发生冲突时，优先考虑大小原则：

- 大小原则：建议一个 tablet 的大小在 1-10G 范围内。过小的 tablet 可能导致聚合效果不佳，增加元数据管理压力；过大的 tablet 则不利于副本迁移、补齐，且会增加 Schema Change 操作的失败重试代价；
- 数量原则：在不考虑扩容的情况下，一个表的 tablet 数量建议略多于整个集群的磁盘数量。

例如，假设有 10 台 BE 机器，每个 BE 一块磁盘，可以按照以下建议进行数据分桶：

单表大小	建议分桶数量
500MB	4-8 个分桶
5GB	6-16 个分桶
50GB	32 个分桶
500GB	建议分区，每个分区 50GB，每个分区 16-32 个分桶
5TB	建议分区，每个分区 50GB，每个分区 16-32 个分桶

> 提示
>
> 表的数据量可以通过 SHOW DATA 命令查看。结果需要除以副本数，即表的数据量。

自动设置分桶数

自动推算分桶数功能会根据过去一段时间的分区大小，自动预测未来的分区大小，并据此确定分桶数量。

```
-- Set hash bucket auto
DISTRIBUTED BY HASH(region) BUCKETS AUTO
properties("estimate_partition_size" = "20G")

-- Set random bucket auto
DISTRIBUTED BY HASH(region) BUCKETS AUTO
properties("estimate_partition_size" = "20G")
```

在创建分桶时，可以通过 estimate_partition_size 属性来调整前期估算的分区大小。此参数为可选设置，若未给出，Doris 将默认取值为 10GB。请注意，该参数与后期系统通过历史分区数据推算出的未来分区大小无关。

2.8.3.5.4 维护数据分桶

> 提示
>
> 目前，Doris 仅支持修改新增分区的分桶数量，对于以下操作暂不支持：
>
> 1. 不支持修改分桶类型
> 2. 不支持修改分桶键
> 3. 不支持修改已创建的分桶的分桶数量

在建表时，已通过 DISTRIBUTED 语句统一指定了每个分区的数量。为了应对数据增长或减少的情况，在动态增加分区时，可以单独指定新分区的分桶数量。以下示例展示了如何通过 ALTER TABLE 命令来修改新增分区的分桶数：

```
-- Modify hash bucket table
ALTER TABLE demo.hash_bucket_tbl
ADD PARTITION p250103 VALUES LESS THAN("2025-01-03")
DISTRIBUTED BY HASH(region) BUCKETS 16;

-- Modify random bucket table
ALTER TABLE demo.random_bucket_tbl
ADD PARTITION p250103 VALUES LESS THAN("2025-01-03")
DISTRIBUTED BY RANDOM BUCKETS 16;

-- Modify dynamic partition table
ALTER TABLE demo.dynamic_partition_tbl
SET ("dynamic_partition.buckets"="16");
```

在修改分桶数量后，可以通过 SHOW PARTITION 命令查看修改后的分桶数量。

2.8.3.6 常见问题

1. 如果在较长的建表语句中出现语法错误，可能会出现语法错误提示不全的现象。这里罗列可能的语法错误供手动纠错：

- 语法结构错误。请仔细阅读 HELP CREATE TABLE;，检查相关语法结构。
- 保留字。当用户自定义名称遇到保留字时，需要用反引号 “ 引起来。建议所有自定义名称使用这个符号引起来。
- 中文字符或全角字符。非 utf8 编码的中文字符，或隐藏的全角字符（空格，标点等）会导致语法错误。建议使用带有显示不可见字符的文本编辑器进行检查。

2. Failed to create partition [xxx] . Timeout

Doris 建表是按照 Partition 粒度依次创建的。当一个 Partition 创建失败时，可能会报这个错误。即使不使用 Partition，当建表出现问题时，也会报 `Failed to create partition`，因为如前文所述，Doris 会为没有指定 Partition 的表创建一个不可更改的默认的 Partition。

当遇到这个错误是，通常是 BE 在创建数据分片时遇到了问题。可以参照以下步骤排查：

- 在 fe.log 中，查找对应时间点的 `Failed to create partition` 日志。在该日志中，会出现一系列类似 `{10001-10010}` 字样的数字对。数字对的第一个数字表示 Backend ID，第二个数字表示 Tablet ID。如上这个数字对，表示 ID 为 10001 的 Backend 上，创建 ID 为 10010 的 Tablet 失败了。
- 前往对应 Backend 的 be.INFO 日志，查找对应时间段内，tablet id 相关的日志，可以找到错误信息。
- 以下罗列一些常见的 tablet 创建失败错误，包括但不限于：
  - BE 没有收到相关 task，此时无法在 be.INFO 中找到 tablet id 相关日志或者 BE 创建成功，但汇报失败。以上问题，请参阅安装与部署检查 FE 和 BE 的连通性。
  - 预分配内存失败。可能是表中一行的字节长度超过了 100KB。
  - `Too many open files`。打开的文件句柄数超过了 Linux 系统限制。需修改 Linux 系统的句柄数限制。

如果创建数据分片时超时，也可以通过在 fe.conf 中设置 `tablet_create_timeout_second=xxx` 以及 `max_create_table_timeout_second=xxx` 来延长超时时间。其中 `tablet_create_timeout_second` 默认是 1 秒，`max_create_table_timeout_second` 默认是 60 秒，总体的超时时间为 min(tablet_create_timeout_second * replication_num, max_create_table_timeout_second)，具体参数设置可参阅FE 配置项 。

3. 建表命令长时间不返回结果。

- Doris 的建表命令是同步命令。该命令的超时时间目前设置的比较简单，即（tablet num * replication num）秒。如果创建较多的数据分片，并且其中有分片创建失败，则可能导致等待较长超时后，才会返回错误。
- 正常情况下，建表语句会在几秒或十几秒内返回。如果超过一分钟，建议直接取消掉这个操作，前往 FE 或 BE 的日志查看相关错误。

2.8.3.6.1 更多帮助

关于数据划分更多的详细说明，我们可以在CREATE TABLE 命令手册中查阅，也可以在 MySQL 客户端下输入 `HELP CREATE TABLE;` 获取更多的帮助信息。

2.8.4 数据类型

Apache Doris 已支持的数据类型列表如下：

2.8.4.0.1 数值类型

类型名	存储空间（字节）	描述
BOOLEAN	1	布尔值，0 代表 false，1 代表 true。
TINYINT	1	有符号整数，范围 [-128, 127]。

类型名	存储空间（字节）	描述
SMALLINT	2	有符号整数，范围 [-32768, 32767]。
INT	4	有符号整数，范围 [-2147483648, 2147483647]
BIGINT	8	有符号整数，范围 [-9223372036854775808, 9223372036854775807]。
LARGEINT	16	有符号整数，范围 [-2^127 + 1 ~ 2^127 - 1]。
FLOAT	4	浮点数，范围 [-3.410^38 ~ 3.410^38]。
DOUBLE	8	浮点数，范围 [-1.7910^308 ~ 1.7910^308]。

| DECIMAL | 4/8/16/32 | 高精度定点数，格式：DECIMAL(P[,S])。其中，P 代表一共有多少个有效数字（precision），S 代表小数位有多少数字（scale）。有效数字 P 的范围是 [1, MAX_P]，enable_decimal256=false 时，MAX_P=38，enable_decimal256=true 时，MAX_P=76。小数位数字数量 S 的范围是 [0, P]。enable_decimal256 的默认值是 false，设置为 true 可以获得更加精确的结果，但是会带来一些性能损失。存储空间：

0 < precision <= 9 时，占用 4 字节。

9 < precision <= 18 时，占用 8 字节。

16 < precision <= 38 时，占用 16 字节。

38 < precision <= 76 的场合，占用 32 字节。

2.8.4.0.2 日期类型

类型名	存储空间（字节）	描述
DATE	4	日期类型，目前的取值范围是 [‘0000-01-01’, ‘9999-12-31’]，默认的打印形式是 ‘yyyy-MM-dd’。
DATETIME	8	日期时间类型，格式：DATETIME([P])。可选参数 P 表示时间精度，取值范围是 [0, 6]，即最多支持 6 位小数（微秒）。不设置时为 0。取值范围是 [‘0000-01-01 00:00:00[.000000]’, ‘9999-12-31 23:59:59[.999999]’]。打印的形式是 ‘yyyy-MM-dd HH:mm:ss.SSSSSS’。

2.8.4.0.3 字符串类型

类型名	存储空间（字节）	描述
CHAR	M	定长字符串，M 代表的是定长字符串的字节长度。M 的范围是 1-255。
VARCHAR	不定长	变长字符串，M 代表的是变长字符串的字节长度。M 的范围是 1-65533。变长字符串是以 UTF-8 编码存储的，因此通常英文字符占 1 个字节，中文字符占 3 个字节。
STRING	不定长	变长字符串，默认支持 1048576 字节（1MB），可调大到 2147483643 字节（2GB）。可通过 BE 配置 string_type_length_soft_limit_bytes 调整。String 类型只能用在 Value 列，不能用在 Key 列和分区分桶列。

2.8.4.0.4 半结构类型

类型名	存储空间（字节）	描述
ARRAY	不定长	由 T 类型元素组成的数组，不能作为 Key 列使用。目前支持在 Duplicate 和 Unique 模型的表中使用。
MAP	不定长	由 K, V 类型元素组成的 map，不能作为 Key 列使用。目前支持在 Duplicate 和 Unique 模型的表中使用。
STRUCT	不定长	由多个 Field 组成的结构体，也可被理解为多个列的集合。不能作为 Key 使用，目前 STRUCT 仅支持在 Duplicate 模型的表中使用。一个 Struct 中的 Field 的名字和数量固定，总是为 Nullable。
JSON	不定长	二进制 JSON 类型，采用二进制 JSON 格式存储，通过 JSON 函数访问 JSON 内部字段。长度限制和配置方式与 String 相同
VARIANT	不定长	动态可变数据类型，专为半结构化数据如 JSON 设计，可以存入任意 JSON，自动将 JSON 中的字段拆分成子列存储，提升存储效率和查询分析性能。长度限制和配置方式与 String 相同。Variant 类型只能用在 Value 列，不能用在 Key 列和分区分桶列。

2.8.4.0.5 聚合类型

类型名	存储空间（字节）	描述
HLL	不定长	HLL 是模糊去重，在数据量大的情况性能优于 Count Distinct。HLL 的误差通常在 1% 左右，有时会达到 2%。HLL 不能作为 Key 列使用，建表时配合聚合类型为 HLL_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。HLL 列只能通过配套的 hll_union_agg、hll_raw_agg、hll_cardinality、hll_hash 进行查询或使用。
BITMAP	不定长	Bitmap 类型的列可以在 Aggregate 表、Unique 表或 Duplicate 表中使用。在 Unique 表或 Duplicate 表中使用时，其必须作为非 Key 列使用。在 Aggregate 表中使用时，其必须作为非 Key 列使用，且建表时配合的聚合类型为 BITMAP_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。BITMAP 列只能通过配套的 bitmap_union_count、bitmap_union、bitmap_hash、bitmap_hash64 等函数进行查询或使用。
QUANTILE_STATE	不定长	QUANTILE_STATE 是一种计算分位数近似值的类型，在导入时会对相同的 Key，不同 Value 进行预聚合，当 value 数量不超过 2048 时采用明细记录所有数据，当 Value 数量大于 2048 时采用 TDigest 算法，对数据进行聚合（聚类）保存聚类后的质心点。QUANTILE_STATE 不能作为 Key 列使用，建表时配合聚合类型为 QUANTILE_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。QUANTILE_STATE 列只能通过配套的 QUANTILE_PERCENT、QUANTILE_UNION、TO_QUANTILE_STATE 等函数进行查询或使用。
AGG_STATE	不定长	聚合函数，只能配合 state/merge/union 函数组合器使用。AGG_STATE 不能作为 Key 列使用，建表时需要同时声明聚合函数的签名。用户不需要指定长度和默认值。实际存储的数据大小与函数实现有关。

类型名	存储空间（字节）	描述

2.8.4.0.6 IP 类型

类型名	存储空间（字节）	描述
IPv4	4 字节	以 4 字节二进制存储 IPv4 地址，配合 ipv4_* 系列函数使用。
IPv6	16 字节	以 16 字节二进制存储 IPv6 地址，配合 ipv6_* 系列函数使用。

也可通过`SHOW DATA TYPES;`语句查看 Apache Doris 支持的所有数据类型。

2.8.5 数据压缩

Doris 采用 列式存储模型来组织和存储数据，这种存储模型特别适合分析型负载，能够显著提高查询效率。在列式存储中，表的每一列会独立存储，这为压缩技术的应用提供了便利，从而提高了存储效率。Doris 提供多种压缩算法，用户可以根据工作负载的需求，选择合适的压缩方式来优化存储和查询性能。

2.8.5.1 为什么需要压缩

在 Doris 中，数据压缩主要有以下两个核心目标：

1. 提升存储效率压缩可以显著减少数据存储所需的磁盘空间，支持在同样的物理资源上存储更多数据。
2. 优化性能压缩后的数据体积更小，查询时需要的 I/O 操作更少，从而加速查询响应时间。现代压缩算法的解压速度通常非常快，能够在减少存储空间的同时提升读取效率。

2.8.5.2 支持的压缩算法

Doris 支持多种压缩算法，每种算法在压缩率和解压速度之间有不同的权衡，可根据需求选择合适的算法：

压缩类型	特点	适用场景
无压缩	- 数据不进行压缩。	适用于不需要压缩的场景，例如数据已经被压缩或者存储空间不是问题的情况。
LZ4	- 压缩和解压速度非常快。- 压缩比适中。	适用于对解压速度要求高的场景，如实时查询或高并发负载。
LZ4F (LZ4 框架)	- LZ4 的扩展版本，支持更灵活的压缩配置。- 速度快，压缩比适中。	适用于需要快速压缩并对配置有细粒度控制的场景。
LZ4HC (LZ4 高压缩)	- 相比 LZ4 有更高的压缩比，但压缩速度较慢。- 解压速度与 LZ4 相当。	适用于需要更高压缩比的场景，同时仍然关注解压速度。
ZSTD (Zstandard)	- 高压缩比，支持灵活的压缩级别调整。- 即使在高压缩比下，解压速度仍然很快。	适用于对存储效率要求较高且需要平衡查询性能的场景。

压缩类型	特点	适用场景
Snappy	- 设计重点是快速解压。- 压缩比适中。	适用于对解压速度要求高且对 CPU 消耗低的场景。
Zlib	- 提供良好的压缩比与速度平衡。- 与其他算法相比，压缩和解压速度较慢，但压缩比更高。	适用于对存储效率要求较高且对解压速度不敏感的场景，如归档和冷数据存储。

2.8.5.3 压缩原理

按列压缩由于采用列式存储，Doris 能够对表中每一列独立压缩。这种方式提升了压缩效率，因为同一列的数据往往具有相似的分布特性。

压缩前的编码在压缩数据之前，Doris 会对列数据进行编码（例如字典编码、游程编码等），将数据转换为更适合压缩的形式，从而进一步提升压缩效率。

按页压缩 Doris 采用 页（Page）级别的压缩策略。每一列的数据会被分成多个页，每个页内的数据会独立进行压缩。通过按页压缩，Doris 能够高效地处理大规模数据集，同时保证高效的压缩率和解压性能。

可配置的压缩策略用户可以在创建表时指定需要使用的压缩算法。这种灵活性使用户可以根据具体工作负载，在压缩效率和性能之间做出最佳选择。

2.8.5.4 影响压缩效果的因素

虽然不同的压缩算法有不同的优缺点，但压缩的效果不仅仅依赖于选择的算法，还受以下因素的影响：

2.8.5.4.1 数据的序列性（Order of Data）

数据的顺序对于压缩效果有重要影响。对于具有高序列性的列（例如时间戳或连续数值列），压缩算法通常能够获得更好的效果。数据的顺序越有规律，压缩算法在压缩时可以识别出更多的重复模式，从而提升压缩比。

2.8.5.4.2 数据的重复度（Data Redundancy）

数据列中重复值越多，压缩效果越明显。例如，使用字典编码对重复值进行编码能够显著降低存储空间。而对于没有明显重复的数据列，压缩效果可能不如预期。

2.8.5.4.3 数据的类型（Data Type）

数据的类型也会影响压缩效果。通常，数值类型的数据（如整数和浮点数）比字符串类型的数据更容易压缩。对于浮动范围较大的数据类型，压缩算法的效果可能会受到影响。

2.8.5.4.4 列的长度（Column Length）

列中数据的长度也会影响压缩效果。较短的列通常比长列更容易压缩，因为压缩算法在较短数据块上能够更高效地找到重复模式。

2.8.5.4.5 空值（Nulls）

列中空值的比例较高时，压缩算法可能会更有效，因为压缩算法会将这些空值作为一种特殊的模式进行编码，减少存储空间。

2.8.5.5 如何选择合适的压缩算法

选择合适的压缩算法需根据工作负载特性：

- 对于 高性能实时分析场景，推荐使用 LZ4 或 Snappy。
- 对于 存储效率优先的场景，推荐使用 ZSTD 或 Zlib。
- 对于需要兼顾速度和压缩率的场景，可选择 LZ4F。
- 对于 归档或冷数据存储场景，建议使用 Zlib 或 LZ4HC。

2.8.5.6 在 Doris 中设置压缩

创建表时，可以通过设置压缩算法来指定存储数据的压缩方式：

“‘sql CREATE TABLE example_table (id INT, name STRING, age INT) DUPLICATE KEY(id) DISTRIBUTED BY HASH(id) BUCKETS 10 PROPERTIES (“compression” = “zstd”);

2.8.6 表索引

2.8.6.1 索引概述

数据库索引是用于查询加速的，为了加速不同的查询场景，Apache Doris 支持了多种丰富的索引。

2.8.6.1.1 索引分类和原理

从加速的查询和原理来看，Apache Doris 的索引分为点查索引和跳数索引两大类。- 点查索引：常用于加速点查，原理是通过索引定位到满足 WHERE 条件的有哪些行，直接读取那些行。点查索引在满足条件的行比较少时效果很好。Apache Doris 的点查索引包括前缀索引和倒排索引。- 前缀索引：Apache Doris 按照排序键以有序的方式存储数据，并每隔 1024 行数据创建一个稀疏前缀索引。索引中的 Key 是当前 1024 行中第一行中排序列的值。如果查询涉及已排序列，系统将找到相关 1024 行组的第一行并从那里开始扫描。- 倒排索引：对创建了倒排索引的列，建立每个值到对应行号集合的倒排表。对于等值查询，先从倒排表中查到行号集合，然后直接读取对应行的数据，而不用逐行扫描匹配数据，从而减少 I/O 加速查询。倒排索引还能加速范围过滤、文本关键词匹配，算法更加复杂但是基本原理类似。（备注：之前的 BITMAP 索引已经被更强的倒排索引取代）- 跳数索引：常用于加速分析，原理是通过索引确定不满足 WHERE 条件的数据块，跳过这些不满足条件的数据块，只读取可能满足条件的数据块并再进行一次逐行过滤，最终得到满足条件的行。跳数索引在满足条件的行比较多时效果较好。Apache Doris 的跳数索引包括 ZoneMap 索引、BloomFilter 索引、NGram BloomFilter 索引。- ZoneMap 索引：自动维护每一列的统计信息，为每一个数据文件（Segment）和数据块（Page）记录最大值、最小值、是否有 NULL。对于等值查询、范围查询、IS NULL，可以通过最大值、最小值、是否有 NULL 来判断数据文件和数据块是否可以包含满足条件的数据，如果没有则跳过不读对应的文件或数据块减少 I/O 加速查询。- BloomFilter 索引：将索引对应列的可能取值存入 BloomFilter 数据结构中，它可以快速判断一个值是否在 BloomFilter 里面，并且 BloomFilter 存储空间占用很低。对于等值查询，如果判断这个值不在 BloomFilter 里面，就可以跳过对应的数据文件或者数据块减少 I/O 加速查询。- NGram BloomFilter 索引：用于加速文本 LIKE 查询，基本原理与 BloomFilter 索引类似，只是存入 BloomFilter 的不是原始文本的值，而是对文本进行 NGram 分词，每个

词作为值存入 BloomFilter。对于 LIKE 查询，将 LIKE 的 pattern 也进行 NGram 分词，判断每个词是否在 BloomFilter 中，如果某个词不在则对应的数据文件或者数据块就不满足 LIKE 条件，可以跳过这部分数据减少 I/O 加速查询。

上述索引中，前缀索引和 ZoneMap 索引是 Apache Doris 自动维护的内建智能索引，无需用户管理，而倒排索引、BloomFilter 索引、 NGram BloomFilter 索引则需要用户自己根据场景选择，手动创建、删除。

- 各种类型索引特点对比

类型	索引	优点	局限
点查索引	前缀索引	内置索引，性能最好	一个表只有
点查索引	倒排索引	支持分词和关键词匹配，任意列可建索引，多条件组合，持续增加函数加速	索引存储空
跳数索引	ZoneMap 索引	内置索引，索引存储空间小	支持的查询
跳数索引	BloomFilter 索引	比 ZoneMap 更精细，索引空间中等	支持的查询
跳数索引	NGram BloomFilter 索引	支持 LIKE 加速，索引空间中等	支持的查询

- 索引加速的运算符和函数列表

运算符 / 函数	前缀索引	倒排索引	ZoneMap 索引	BloomFilter 索引	NGram BloomFilter 索引
=	YES	YES	YES	YES	NO
!=	YES	YES	NO	NO	NO
IN	YES	YES	YES	YES	NO
NOT IN	YES	YES	NO	NO	NO
>, >=, <, <=, BETWEEN	YES	YES	YES	NO	NO
IS NULL	YES	YES	YES	NO	NO
IS NOT NULL	YES	YES	NO	NO	NO
LIKE	NO	NO	NO	NO	YES
MATCH, MATCH_*	NO	YES	NO	NO	NO
array_contains	NO	YES	NO	NO	NO
array_overlaps	NO	YES	NO	NO	NO
is_ip_address_in_range	NO	YES	NO	NO	NO

2.8.6.1.2 索引设计指南

数据库表的索引设计和优化跟数据特点和查询很相关，需要根据实际场景测试和优化。虽然没有“银弹”，Apache Doris 仍然不断努力降低用户使用索引的难度，用户可以根据下面的简单建议原则进行索引选择和测试。

1. 最频繁使用的过滤条件指定为 Key 自动建前缀索引，因为它的过滤效果最好，但是一个表只能有一个前缀索引，因此要用在最频繁的过滤条件上
2. 对非 Key 字段如有过滤加速需求，首选建倒排索引，因为它的适用面广，可以多条件组合，次选下面两种索引：

- 有字符串 LIKE 匹配需求，再加一个 NGram BloomFilter 索引

- 对索引存储空间很敏感，将倒排索引换成 BloomFilter 索引

3. 如果性能不及预期，通过 QueryProfile 分析索引过滤掉的数据量和消耗的时间，具体参考各个索引的详细文档

2.8.6.2 前缀索引与排序键

2.8.6.2.1 索引原理

Doris 的数据存储在类似 SSTable（Sorted String Table）的数据结构中。该结构是一种有序的数据结构，可以按照指定的一个或多个列进行排序存储。在这种数据结构上，以排序列的全部或者前面几个作为条件进行查找，会非常的高效。

在 Aggregate、Unique 和 Duplicate 三种数据模型中。底层的数据存储，是按照各自建表语句中，Aggregate Key、Unique Key 和 Duplicate Key 中指定的列进行排序存储的。这些 Key，称为排序键（Sort Key）。借助排序键，在查询时，通过给排序列指定条件，Doris 不需要扫描全表即可快速找到需要处理的数据，降低搜索的复杂度，从而加速查询。

在排序键的基础上，又引入了前缀索引（Prefix Index）。前缀索引是一种稀疏索引。表中按照相应的行数的数据构成一个逻辑数据块 (Data Block)。每个逻辑数据块在前缀索引表中存储一个索引项，索引项的长度不超过 36 字节，其内容为数据块中第一行数据的排序列组成的前缀，在查找前缀索引表时可以帮助确定该行数据所在逻辑数据块的起始行号。由于前缀索引比较小，所以，可以全量在内存缓存，快速定位数据块，大大提升了查询效率。

> 数据块一行数据的前 36 个字节作为这行数据的前缀索引。当遇到 VARCHAR 类型时，前缀索引会直接截断。如果第一列即为 VARCHAR，那么即使没有达到 36 字节，也会直接截断，后面的列不再加入前缀索引。

2.8.6.2.2 使用场景

前缀索引可以加速等值查询和范围查询。

2.8.6.2.3 管理索引

前缀索引没有专门的语法去定义，建表时自动取表的 Key 的前 36 字节作为前缀索引。

前缀索引选择建议

> 因为一个表的 Key 定义是唯一的，所以一个表只有一组前缀索引，因此设计表结构时选择合适的前缀索引很重要，可以参考下面的建议：1. 选择查询中最常用于 WHERE 过滤条件的字段作为 Key。2. 约常用的字段越放在前面，因为前缀索引只对 WHERE 条件中字段在 Key 的前缀中才有效。

使用其他不能命中前缀索引的列作为条件进行的查询来说，效率上可能无法满足需求，有两种解决方案：1. 对需要加速查询的条件列创建倒排索引，由于一个表的倒排索引可以有很多个。2. 对于 Duplicate 表可以通过创建相应的调整了列顺序的单表强一致物化视图来间接实现多种前缀索引，详情可参考查询加速/物化视图。

2.8.6.2.4 使用索引

前缀索引用于加速 WHERE 条件中的等值和范围查询，能加速时自动生效，没有特殊语法。

可以通过 Query Profile 中的下面几个指标分析前缀索引的加速效果。- RowsKeyRangeFiltered 前缀索引过滤掉的行数，可以与其他几个 Rows 值对比分析索引过滤效果

2.8.6.2.5 使用示例

- 假如表的排序列为如下 5 列，那么前缀索引为：user_id(8 Bytes) + age(4 Bytes) + message(prefix 20 Bytes)。

ColumnName	Type
user_id	BIGINT
age	INT
message	VARCHAR(100)
max_dwell_time	DATETIME
min_dwell_time	DATETIME

- 假如表的排序列为如下 5 列，则前缀索引为 user_name(20 Bytes)。即使没有达到 36 个字节，因为遇到 VARCHAR，所以直接截断，不再往后继续。

ColumnName	Type
user_name	VARCHAR(20)
age	INT
message	VARCHAR(100)
max_dwell_time	DATETIME
min_dwell_time	DATETIME

- 当我们的查询条件，是前缀索引的前缀时，可以极大地加快查询速度。比如在第一个例子中，执行如下查询：

```
SELECT * FROM table WHERE user_id=1829239 and age=20;
```

该查询的效率会远高于如下查询：

```
SELECT * FROM table WHERE age=20;
```

所以在建表时，正确选择列顺序，能够极大地提高查询效率。

2.8.6.3 倒排索引

2.8.6.3.1 索引原理

倒排索引，是信息检索领域常用的索引技术，将文本分成一个个词，构建词 -> 文档编号的索引，可以快速查找一个词在哪些文档出现。

从 2.0.0 版本开始，Doris 支持倒排索引，可以用来进行文本类型的全文检索、普通数值日期类型的等值范围查询，快速从海量数据中过滤出满足条件的行。

在 Doris 的倒排索引实现中，Table 的一行对应一个文档、一列对应文档中的一个字段，因此利用倒排索引可以根据关键词快速定位包含它的行，达到 WHERE 子句加速的目的。

与 Doris 中其他索引不同的是，在存储层倒排索引使用独立的文件，跟数据文件一一对应、但物理存储上文件相互独立。这样的好处是可以做到创建、删除索引不用重写数据文件，大幅降低处理开销。

2.8.6.3.2 使用场景

倒排索引的使用范围很广泛，可以加速等值、范围、全文检索（关键词匹配、短语系列匹配等）。一个表可以有多个倒排索引，查询时多个倒排索引的条件可以任意组合。

倒排索引的功能简要介绍如下：

1. 加速字符串类型的全文检索

- 支持关键词检索，包括同时匹配多个关键字 MATCH_ALL、匹配任意一个关键字 MATCH_ANY
- 支持短语查询 MATCH_PHRASE
- 支持指定词距 slop
- 支持短语 + 前缀 MATCH_PHRASE_PREFIX
- 支持分词正则查询 MATCH_REGEXP
- 支持英文、中文以及 Unicode 多种分词

2. 加速普通等值、范围查询，覆盖原来 BITMAP 索引的功能，代替 BITMAP 索引

- 支持字符串、数值、日期时间类型的 =, !=, >, >=, <, <= 快速过滤
- 支持字符串、数字、日期时间数组类型的 =, !=, >, >=, <, <=

3. 支持完善的逻辑组合

- 不仅支持 AND 条件加速，还支持 OR NOT 条件加速
- 支持多个条件的任意 AND OR NOT 逻辑组合

4. 灵活高效的索引管理

- 支持在创建表上定义倒排索引
- 支持在已有的表上增加倒排索引，而且支持增量构建倒排索引，无需重写表中的已有数据
- 支持删除已有表上的倒排索引，无需重写表中的已有数据

> 倒排索引的使用有下面一些限制：
>
> 1. 存在精度问题的浮点数类型 FLOAT 和 DOUBLE 不支持倒排索引，原因是浮点数精度不准确。解决方案是使用精度准确的定点数类型 DECIMAL，DECIMAL 支持倒排索引。
> 2. 部分复杂数据类型还不支持倒排索引，包括：MAP、STRUCT、JSON、HLL、BITMAP、QUANTILE_STATE、AGG_STATE。其中 MAP、STRUCT 会逐步支持，JSON 类型可以换成 VARIANT 类型获得支持。其他几个类型因为其特殊用途暂不需要支持倒排索引。
> 3. DUPLICATE 和开启 Merge-on-Write 的 UNIQUE 表模型支持任意列建倒排索引。但是 AGGREGATE 和未开启 Merge-on-Write 的 UNIQUE 模型仅支持 Key 列建倒排索引，非 Key 列不能建倒排索引，这是因为这两个模型需要读取所有数据后做合并，因此不能利用索引做提前过滤。

2.8.6.3.3 管理索引

建表时定义倒排索引

在建表语句中 COLUMN 的定义之后是索引定义：

```
CREATE TABLE table_name
(
  column_name1 TYPE1,
  column_name2 TYPE2,
  column_name3 TYPE3,
  INDEX idx_name1(column_name1) USING INVERTED [PROPERTIES(...)] [COMMENT 'your comment'],
  INDEX idx_name2(column_name2) USING INVERTED [PROPERTIES(...)] [COMMENT 'your comment']
)
table_properties;
```

语法说明如下：

1. `idx_column_name(column_name)` 是必须的，`column_name` 是建索引的列名，必须是前面列定义中出现过的，`idx_column_name` 是索引名字，必须表级别唯一，建议命名规范：列名前面加前缀 `idx_`

2. `USING INVERTED` 是必须的，用于指定索引类型是倒排索引

3. `PROPERTIES` 是可选的，用于指定倒排索引的额外属性，目前支持的属性如下：

4. `COMMENT` 是可选的，用于指定索引注释

5. 表级属性 `inverted_index_storage_format` (该功能自 3.1.0 版本开始支持)

要使用新的 V3 存储格式，在建表时指定此属性：

```
CREATE TABLE table_name (
    column_name TEXT,
    INDEX idx_name(column_name) USING INVERTED PROPERTIES("parser" = "english", "dict_compression
        ↪ " = "true")
) PROPERTIES (
    "inverted_index_storage_format" = "V3"
);
```

已有表增加倒排索引

1. ADD INDEX

支持CREATE INDEX 和 ALTER TABLE ADD INDEX 两种语法，参数跟建表时索引定义相同

```
-- 语法 1
CREATE INDEX idx_name ON table_name(column_name) USING INVERTED [PROPERTIES(...)] [COMMENT 'your
    ↪ comment'];
-- 语法 2
ALTER TABLE table_name ADD INDEX idx_name(column_name) USING INVERTED [PROPERTIES(...)] [COMMENT
    ↪ 'your comment'];
```

2. BUILD INDEX

CREATE / ADD INDEX 操作只是新增了索引定义，这个操作之后的新写入数据会生成倒排索引，而存量数据需要使用 BUILD INDEX 触发：

```
-- 语法 1，默认给全表的所有分区 BUILD INDEX
BUILD INDEX index_name ON table_name;
-- 语法 2，可指定 Partition，可指定一个或多个
BUILD INDEX index_name ON table_name PARTITIONS(partition_name1, partition_name2);
```

通过 SHOW BUILD INDEX 查看 BUILD INDEX 进度：

```
SHOW BUILD INDEX [FROM db_name];
-- 示例 1，查看所有的 BUILD INDEX 任务进展
SHOW BUILD INDEX;
-- 示例 2，查看指定 table 的 BUILD INDEX 任务进展
SHOW BUILD INDEX where TableName = "table1";
```

通过 CANCEL BUILD INDEX 取消 BUILD INDEX：

```
CANCEL BUILD INDEX ON table_name;
CANCEL BUILD INDEX ON table_name (job_id1,jobid_2,...);
```

> BUILD INDEX 会生成一个异步任务执行，在每个 BE 上有多个线程执行索引构建任务，通过 BE 参数 alter_index_worker_count 可以设置，默认值是 3。

2.0.12 和 2.1.4 之前的版本 BUILD INDEX 会一直重试直到成功，从这两个版本开始通过失败和超时机制避免一直重试。3.0 存算分离模式暂不支持此命令。

1. 一个 tablet 的多数副本 BUILD INDEX 失败后，整个 BUILD INDEX 失败结束
2. 时间超过 alter_table_timeout_second()，BUILD INDEX 超时结束
3. 用户可以多次触发 BUILD INDEX，已经 BUILD 成功的索引不会重复 BUILD

已有表删除倒排索引

```
-- 语法 1
DROP INDEX idx_name ON table_name;
-- 语法 2
ALTER TABLE table_name DROP INDEX idx_name;
```

DROP INDEX 会删除索引定义，新写入数据不会再写索引，同时会生成一个异步任务执行索引删除操作，在每个 BE 上有多个线程执行索引删除任务，通过 BE 参数 alter_index_worker_count 可以设置，默认值是 3。

查看倒排索引

```
-- 语法 1，表的 schema 中 INDEX 部分 USING INVERTED 是倒排索引
SHOW CREATE TABLE table_name;

-- 语法 2，IndexType 为 INVERTED 的是倒排索引
SHOW INDEX FROM idx_name;
```

2.8.6.3.4 使用索引

利用倒排索引加速查询

```
-- 1. 全文检索关键词匹配，通过 MATCH_ANY MATCH_ALL 完成
SELECT * FROM table_name WHERE column_name MATCH_ANY | MATCH_ALL 'keyword1 ...';

-- 1.1 content 列中包含 keyword1 的行
SELECT * FROM table_name WHERE content MATCH_ANY 'keyword1';

-- 1.2 content 列中包含 keyword1 或者 keyword2 的行，后面还可以添加多个 keyword
SELECT * FROM table_name WHERE content MATCH_ANY 'keyword1 keyword2';

-- 1.3 content 列中同时包含 keyword1 和 keyword2 的行，后面还可以添加多个 keyword
SELECT * FROM table_name WHERE content MATCH_ALL 'keyword1 keyword2';
```

```
-- 2. 全文检索短语匹配，通过 MATCH_PHRASE 完成
-- 2.1 content 列中同时包含 keyword1 和 keyword2 的行，而且 keyword2 必须紧跟在 keyword1 后面
-- 'keyword1 keyword2', 'wordx keyword1 keyword2', 'wordx keyword1 keyword2 wordy' 能匹配，
    ↪ 因为他们都包含 keyword1 keyword2，而且 keyword2 紧跟在 keyword1 后面
-- 'keyword1 wordx keyword2' 不能匹配，因为 keyword1 keyword2 之间隔了一个词 wordx
-- 'keyword2 keyword1'，因为 keyword1 keyword2 的顺序反了
-- 使用 MATCH_PHRASE 需要再 PROPERTIES 中开启 "support_phrase" = "true"
SELECT * FROM table_name WHERE content MATCH_PHRASE 'keyword1 keyword2';

-- 2.2 content 列中同时包含 keyword1 和 keyword2 的行，而且 keyword1 keyword2 的 `词距`（slop）
    ↪ 不超过 3
-- 'keyword1 keyword2', 'keyword1 a keyword2', 'keyword1 a b c keyword2' 都能匹配，因为 keyword1
    ↪ keyword2 中间隔的词分别是 0 1 3 都不超过 3
-- 'keyword1 a b c d keyword2' 不能能匹配，因为 keyword1 keyword2 中间隔的词有 4 个，超过 3
-- 'keyword2 keyword1', 'keyword2 a keyword1', 'keyword2 a b c keyword1' 也能匹配，因为指定 slop
    ↪ > 0 时不再要求 keyword1 keyword2 的顺序。这个行为参考了 ES，与直觉的预期不一样，因此
    ↪ Doris 提供了在 slop 后面指定正数符号（+）表示需要保持 keyword1 keyword2 的先后顺序
SELECT * FROM table_name WHERE content MATCH_PHRASE 'keyword1 keyword2 ~3';
-- slop 指定正号，'keyword1 a b c keyword2' 能匹配，而 'keyword2 a b c keyword1' 不能匹配
SELECT * FROM table_name WHERE content MATCH_PHRASE 'keyword1 keyword2 ~3+';

-- 2.3 在保持词顺序的前提下，对最后一个词 keyword2 做前缀匹配，默认找 50 个前缀词（session 变量
    ↪ inverted_index_max_expansions 控制）
-- 需要保证 keyword1, keyword2 在原文分词后也是相邻的，不能中间有其他词
-- 'keyword1 keyword2abc' 能匹配，因为 keyword1 完全一样，最后一个 keyword2abc 是 keyword2 的前缀
-- 'keyword1 keyword2' 也能匹配，因为 keyword2 也是 keyword2 的前缀
-- 'keyword1 keyword3' 不能匹配，因为 keyword3 不是 keyword2 的前缀
-- 'keyword1 keyword3abc' 也不能匹配，因为 keyword3abc 也不是 keyword2 的前缀
SELECT * FROM table_name WHERE content MATCH_PHRASE_PREFIX 'keyword1 keyword2';

-- 2.4 如果只填一个词会退化为前缀查询，默认找 50 个前缀词（session 变量 inverted_index_max_
    ↪ expansions 控制）
SELECT * FROM table_name WHERE content MATCH_PHRASE_PREFIX 'keyword1';

-- 2.5 对分词后的词进行正则匹配，默认匹配 50 个（session 变量 inverted_index_max_expansions 控制
    ↪ ）
-- 类似 MATCH_PHRASE_PREFIX 的匹配规则，只是前缀变成了正则
SELECT * FROM table_name WHERE content MATCH_REGEXP 'key.*';

-- 3. 普通等值、范围、IN、NOT IN，正常的 SQL 语句即可，例如
SELECT * FROM table_name WHERE id = 123;
SELECT * FROM table_name WHERE ts > '2023-01-01 00:00:00';
SELECT * FROM table_name WHERE op_type IN ('add', 'delete');
```

```
-- 4. 多列全文检索匹配，通过 multi_match 函数完成
-- 参数说明:
--   前N个参数是要匹配的列名
--   倒数第二个参数指定匹配模式: 'any'/'all'/'phrase'/'phrase_prefix'
--   最后一个参数是要搜索的关键词或短语

-- 4.1 在col1,col2,col3任意一列中包含'keyword1'的行（OR逻辑）
SELECT * FROM table_name WHERE multi_match(col1, col2, col3, 'any', 'keyword1');

-- 4.2 在col1,col2,col3所有列中都包含'keyword1'的行（AND逻辑）
SELECT * FROM table_name WHERE multi_match(col1, col2, col3, 'all', 'keyword1');

-- 4.3 在col1,col2,col3任意一列中包含完整短语'keyword1'的行（精确短语匹配）
SELECT * FROM table_name WHERE multi_match(col1, col2, col3, 'phrase', 'keyword1');

-- 4.4 在col1,col2,col3任意一列中包含以'keyword1'开头的短语的行（短语前缀匹配）
-- 例如会匹配"keyword123"这样的内容
SELECT * FROM table_name WHERE multi_match(col1, col2, col3, 'phrase_prefix', 'keyword1');
```

通过 profile 分析索引加速效果

倒排查询加速可以通过 session 变量 enable_inverted_index_query 开关，默认是 true 打开，有时为了验证索引加速效果可以设置为 false 关闭。

可以通过 Query Profile 中的下面几个指标分析倒排索引的加速效果。- RowsInvertedIndexFiltered 倒排过滤掉的行数，可以与其他几个 Rows 值对比分析索引过滤效果 - InvertedIndexFilterTime 倒排索引消耗的时间 - InvertedIndexSearcherOpenTime 倒排索引打开索引的时间 - InvertedIndexSearcherSearchTime 倒排索引内部查询的时间

用分词函数验证分词效果

如果想检查分词实际效果或者对一段文本进行分词行为，可以使用 TOKENIZE 函数进行验证。

TOKENIZE 函数的第一个参数是待分词的文本，第二个参数是创建索引指定的分词参数。

```
SELECT TOKENIZE('武汉长江大桥','"parser"="chinese","parser_mode"="fine_grained"');
+-----------------------------------------------------------------------------------+
| tokenize('武汉长江大桥', '"parser"="chinese","parser_mode"="fine_grained"')       |
+-----------------------------------------------------------------------------------+
| ["武汉", "武汉长江大桥", "长江", "长江大桥", "大桥"]                                |
+-----------------------------------------------------------------------------------+

SELECT TOKENIZE('武汉市长江大桥','"parser"="chinese","parser_mode"="fine_grained"');
+-------------------------------------------------------------------------------------+
| tokenize('武汉市长江大桥', '"parser"="chinese","parser_mode"="fine_grained"')        |
+-------------------------------------------------------------------------------------+
| ["武汉", "武汉市", "市长", "长江", "长江大桥", "大桥"]                                |
+-------------------------------------------------------------------------------------+
```

```
SELECT TOKENIZE('武汉市长江大桥','"parser"="chinese","parser_mode"="coarse_grained"');
+-----------------------------------------------------------------------------------------+
| tokenize('武汉市长江大桥', '"parser"="chinese","parser_mode"="coarse_grained"')        |
+-----------------------------------------------------------------------------------------+
| ["武汉市", "长江大桥"]                                                                  |
+-----------------------------------------------------------------------------------------+

SELECT TOKENIZE('I love Doris','"parser"="english"');
+-----------------------------------------------+
| tokenize('I love Doris', '"parser"="english"') |
+-----------------------------------------------+
| ["i", "love", "doris"]                        |
+-----------------------------------------------+

SELECT TOKENIZE('I love CHINA 我爱我的祖国','"parser"="unicode"');
+-------------------------------------------------------------------+
| tokenize('I love CHINA 我爱我的祖国', '"parser"="unicode"')       |
+-------------------------------------------------------------------+
| ["i", "love", "china", "我", "爱", "我", "的", "祖", "国"]        |
+-------------------------------------------------------------------+

-- ICU 分词多语言文本（该功能自 3.1.0 版本开始支持）
SELECT TOKENIZE('□□□□□□ □□□□□□□□ Hello 世界', '"parser"="icu"');
+-----------------------------------------------------------+
| tokenize('□□□□□□ □□□□□□□□ Hello 世界', '"parser"="icu"') |
+-----------------------------------------------------------+
| ["□□□□□□", "□□□□□□□□", "Hello", "世界"]                   |
+-----------------------------------------------------------+

SELECT TOKENIZE('□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□', '"parser"="icu"');
+-----------------------------------------------------------------------+
| tokenize('□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□', '"parser"="icu"')      |
+-----------------------------------------------------------------------+
| ["□□", "□□□□□□", "□□", "□□□□", "□□□□□", "□□□□□□□□"]                   |
+-----------------------------------------------------------------------+

-- Basic 分词高性能场景（该功能自 3.1.0 版本开始支持）
SELECT TOKENIZE('Hello World! This is a test.', '"parser"="basic"');
+-----------------------------------------------------------+
| tokenize('Hello World! This is a test.', '"parser"="basic"') |
+-----------------------------------------------------------+
| ["hello", "world", "this", "is", "a", "test"]             |
+-----------------------------------------------------------+
```

```
SELECT TOKENIZE('你好世界', '"parser"="basic"');
+-----------------------------------------+
| tokenize('你好世界', '"parser"="basic"')   |
+-----------------------------------------+
| ["你", "好", "世", "界"]                    |
+-----------------------------------------+

SELECT TOKENIZE('Hello你好World世界', '"parser"="basic"');
+-----------------------------------------------------+
| tokenize('Hello你好World世界', '"parser"="basic"')    |
+-----------------------------------------------------+
| ["hello", "你", "好", "world", "世", "界"]             |
+-----------------------------------------------------+

SELECT TOKENIZE('GET /images/hm_bg.jpg HTTP/1.0', '"parser"="basic"');
+-------------------------------------------------------------------------+
| tokenize('GET /images/hm_bg.jpg HTTP/1.0', '"parser"="basic"')      |
+-------------------------------------------------------------------------+
| ["get", "images", "hm", "bg", "jpg", "http", "1", "0"]              |
+-------------------------------------------------------------------------+
```

2.8.6.3.5 使用示例

用 HackerNews 100 万条数据展示倒排索引的创建、全文检索、普通查询，包括跟无索引的查询性能进行简单对比。

建表

```
CREATE DATABASE test_inverted_index;

USE test_inverted_index;

-- 创建表的同时创建了 comment 的倒排索引 idx_comment
--   USING INVERTED 指定索引类型是倒排索引
--   PROPERTIES("parser" = "english") 指定采用 "english" 分词，还支持 "chinese" 中文分词和 "
    ↪ unicode" 中英文多语言混合分词，如果不指定 "parser" 参数表示不分词

CREATE TABLE hackernews_1m
(
    `id` BIGINT,
    `deleted` TINYINT,
    `type` String,
    `author` String,
    `timestamp` DateTimeV2,
    `comment` String,
```

```
    `dead` TINYINT,
    `parent` BIGINT,
    `poll` BIGINT,
    `children` Array<BIGINT>,
    `url` String,
    `score` INT,
    `title` String,
    `parts` Array<INT>,
    `descendants` INT,
    INDEX idx_comment (`comment`) USING INVERTED PROPERTIES("parser" = "english") COMMENT '
        ↪ inverted index for comment'
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES ("replication_num" = "1");
```

导入数据

通过 Stream Load 导入数据

```
wget https://qa-build.oss-cn-beijing.aliyuncs.com/regression/index/hacknernews_1m.csv.gz

curl --location-trusted -u root: -H "compress_type:gz" -T hacknernews_1m.csv.gz  http
    ↪ ://127.0.0.1:8030/api/test_inverted_index/hackernews_1m/_stream_load
{
    "TxnId": 2,
    "Label": "a8a3e802-2329-49e8-912b-04c800a461a6",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 1000000,
    "NumberLoadedRows": 1000000,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 130618406,
    "LoadTimeMs": 8988,
    "BeginTxnTimeMs": 23,
    "StreamLoadPutTimeMs": 113,
    "ReadDataTimeMs": 4788,
    "WriteDataTimeMs": 8811,
    "CommitAndPublishTimeMs": 38
}
```

SQL 运行 count() 确认导入数据成功

```
SELECT count() FROM hackernews_1m;
+---------+
```

```
| count() |
+---------+
| 1000000 |
+---------+
```

查询

01 全文检索

- 用 LIKE 匹配计算 comment 中含有 ‘OLAP’ 的行数，耗时 0.18s

```
SELECT count() FROM hackernews_1m WHERE comment LIKE '%OLAP%';
+---------+
| count() |
+---------+
|      34 |
+---------+
```

- 用基于倒排索引的全文检索 MATCH_ANY 计算 comment 中含有’OLAP’ 的行数，耗时 0.02s，加速 9 倍，在更大的数据集上效果会更加明显

这里结果条数的差异，是因为倒排索引对 comment 分词后，还会对词进行进行统一成小写等归一化处理，因此 MATCH_ANY 比 LIKE 的结果多一些

```
SELECT count() FROM hackernews_1m WHERE comment MATCH_ANY 'OLAP';
+---------+
| count() |
+---------+
|      35 |
+---------+
```

- 同样的对比统计 ‘OLTP’ 出现次数的性能，0.07s vs 0.01s，由于缓存的原因 LIKE 和 MATCH_ANY 都有提升，倒排索引仍然有 7 倍加速

```
SELECT count() FROM hackernews_1m WHERE comment LIKE '%OLTP%';
+---------+
| count() |
+---------+
|      48 |
+---------+


SELECT count() FROM hackernews_1m WHERE comment MATCH_ANY 'OLTP';
+---------+
```

```
| count() |
+---------+
|      51 |
+---------+
```

- 同时出现 ‘OLAP’ 和 ‘OLTP’ 两个词，0.13s vs 0.01s，13 倍加速

要求多个词同时出现时（AND 关系）使用 MATCH_ALL ‘keyword1 keyword2 …’

```
SELECT count() FROM hackernews_1m WHERE comment LIKE '%OLAP%' AND comment LIKE '%OLTP%';
+---------+
| count() |
+---------+
|      14 |
+---------+


SELECT count() FROM hackernews_1m WHERE comment MATCH_ALL 'OLAP OLTP';
+---------+
| count() |
+---------+
|      15 |
+---------+
```

- 任意出现 ‘OLAP’ 和 ‘OLTP’ 其中一个词，0.12s vs 0.01s，12 倍加速

只要求多个词任意一个或多个出现时（OR 关系）使用 MATCH_ANY ‘keyword1 keyword2 …’

```
SELECT count() FROM hackernews_1m WHERE comment LIKE '%OLAP%' OR comment LIKE '%OLTP%';
+---------+
| count() |
+---------+
|      68 |
+---------+

SELECT count() FROM hackernews_1m WHERE comment MATCH_ANY 'OLAP OLTP';
+---------+
| count() |
+---------+
|      71 |
+---------+
```

02 普通等值、范围查询

- DataTime 类型的列范围查询

```
SELECT count() FROM hackernews_1m WHERE timestamp > '2007-08-23 04:17:00';
+---------+
| count() |
+---------+
|  999081 |
+---------+
```

- 为 timestamp 列增加一个倒排索引

```
-- 对于日期时间类型 USING INVERTED，不用指定分词
-- CREATE INDEX 是第一种建索引的语法，另外一种在后面展示
CREATE INDEX idx_timestamp ON hackernews_1m(timestamp) USING INVERTED;
```

```
BUILD INDEX idx_timestamp ON hackernews_1m;
```

- 查看索引创建进度，通过 FinishTime 和 CreateTime 的差值，可以看到 100 万条数据对 timestamp 列建倒排索引只用了 1s

```
SHOW ALTER TABLE COLUMN;
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪
| JobId | TableName     | CreateTime              | FinishTime              | IndexName     |
    ↪ IndexId | OriginIndexId | SchemaVersion | TransactionId | State    | Msg  | Progress |
    ↪ Timeout |
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪
| 10030 | hackernews_1m | 2023-02-10 19:44:12.929 | 2023-02-10 19:44:13.938 | hackernews_1m |
    ↪ 10031   | 10008         | 1:1994690496  | 3             | FINISHED |      | NULL     |
    ↪ 2592000 |
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪
```

```
-- 若 table 没有分区，PartitionName 默认就是 TableName
SHOW BUILD INDEX;
+--
    ↪ -----+---------------+---------------+-------------------------------------------------------------+------
    ↪
| JobId | TableName     | PartitionName | AlterInvertedIndexes
    ↪                                     | CreateTime              | FinishTime
    ↪              | TransactionId | State    | Msg  | Progress |
```

```
+--
    ↪ -----+---------------+---------------+-------------------------------------------------------------------+------
    ↪
| 10191 | hackernews_1m | hackernews_1m | [ADD INDEX idx_timestamp (`timestamp`) USING INVERTED
    ↪ ],  | 2023-06-26 15:32:33.894 | 2023-06-26 15:32:34.847 | 3              | FINISHED |
    ↪       | NULL      |
+--
    ↪ -----+---------------+---------------+-------------------------------------------------------------------+------
    ↪
```

- 索引创建后，范围查询用同样的查询方式，Doris 会自动识别索引进行优化，但是这里由于数据量小性能差别不大

```
SELECT count() FROM hackernews_1m WHERE timestamp > '2007-08-23 04:17:00';
+---------+
| count() |
+---------+
|  999081 |
+---------+
```

- 在数值类型的列 Parent 进行类似 timestamp 的操作，这里查询使用等值匹配

```
SELECT count() FROM hackernews_1m WHERE parent = 11189;
+---------+
| count() |
+---------+
|       2 |
+---------+


-- 对于数值类型 USING INVERTED，不用指定分词
-- ALTER TABLE t ADD INDEX 是第二种建索引的语法
ALTER TABLE hackernews_1m ADD INDEX idx_parent(parent) USING INVERTED;


-- 执行 BUILD INDEX 给存量数据构建倒排索引
BUILD INDEX idx_parent ON hackernews_1m;


SHOW ALTER TABLE COLUMN;
+--
    ↪ -----+---------------+-----------------------+-----------------------+---------------+----------+---
    ↪
```

```
| JobId | TableName     | CreateTime              | FinishTime              | IndexName     |
    ↪ IndexId | OriginIndexId | SchemaVersion | TransactionId | State    | Msg  | Progress |
    ↪ Timeout |
+--
    ↪ -----+--------------+-------------------------+-------------------------+--------------+---------+---
    ↪
| 10030 | hackernews_1m | 2023-02-10 19:44:12.929 | 2023-02-10 19:44:13.938 | hackernews_1m |
    ↪ 10031   | 10008         | 1:1994690496  | 3             | FINISHED |      | NULL     |
    ↪ 2592000 |
| 10053 | hackernews_1m | 2023-02-10 19:49:32.893 | 2023-02-10 19:49:33.982 | hackernews_1m |
    ↪ 10054   | 10008         | 1:378856428   | 4             | FINISHED |      | NULL     |
    ↪ 2592000 |
+--
    ↪ -----+--------------+-------------------------+-------------------------+--------------+---------+---
    ↪

SHOW BUILD INDEX;
+--
    ↪ -----+--------------+--------------+-----------------------------------------------------------+-----------
    ↪
| JobId | TableName     | PartitionName | AlterInvertedIndexes                                      |
    ↪ CreateTime              | FinishTime              | TransactionId | State    | Msg  |
    ↪ Progress |
+--
    ↪ -----+--------------+--------------+-----------------------------------------------------------+-----------
    ↪
| 11005 | hackernews_1m | hackernews_1m | [ADD INDEX idx_parent (`parent`) USING INVERTED],  |
    ↪ 2023-06-26 16:25:10.167 | 2023-06-26 16:25:10.838 | 1002          | FINISHED |      |
    ↪ NULL     |
+--
    ↪ -----+--------------+--------------+-----------------------------------------------------------+-----------
    ↪


SELECT count() FROM hackernews_1m WHERE parent = 11189;
+---------+
| count() |
+---------+
|       2 |
+---------+
```

- 对字符串类型的 author 建立不分词的倒排索引，等值查询也可以利用索引加速

```
SELECT count() FROM hackernews_1m WHERE author = 'faster';
+---------+
```

```
| count() |
+---------+
|      20 |
+---------+

-- 这里只用了 USING INVERTED，不对 author 分词，整个当做一个词处理
ALTER TABLE hackernews_1m ADD INDEX idx_author(author) USING INVERTED;


-- 执行 BUILD INDEX 给存量数据加上倒排索引：
BUILD INDEX idx_author ON hackernews_1m;


-- 100 万条 author 数据增量建索引仅消耗 1.5s
SHOW ALTER TABLE COLUMN;
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪
| JobId | TableName     | CreateTime              | FinishTime              | IndexName     |
    ↪ IndexId | OriginIndexId | SchemaVersion | TransactionId | State    | Msg  | Progress |
    ↪ Timeout |
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪
| 10030 | hackernews_1m | 2023-02-10 19:44:12.929 | 2023-02-10 19:44:13.938 | hackernews_1m |
    ↪ 10031   | 10008         | 1:1994690496  | 3             | FINISHED |      | NULL     |
    ↪ 2592000 |
| 10053 | hackernews_1m | 2023-02-10 19:49:32.893 | 2023-02-10 19:49:33.982 | hackernews_1m |
    ↪ 10054   | 10008         | 1:378856428   | 4             | FINISHED |      | NULL     |
    ↪ 2592000 |
| 10076 | hackernews_1m | 2023-02-10 19:54:20.046 | 2023-02-10 19:54:21.521 | hackernews_1m |
    ↪ 10077   | 10008         | 1:1335127701  | 5             | FINISHED |      | NULL     |
    ↪ 2592000 |
+--
    ↪ -----+---------------+-------------------------+-------------------------+---------------+---------+---
    ↪

SHOW BUILD INDEX order by CreateTime desc limit 1;
+--
    ↪ -----+---------------+---------------+-----------------------------------------------------+------------
    ↪
| JobId | TableName     | PartitionName | AlterInvertedIndexes                                |
    ↪ CreateTime              | FinishTime              | TransactionId | State    | Msg  |
    ↪ Progress |
+--
```

```
    ↪ -----+--------------+--------------+-------------------------------------------------------+-----------
    ↪
| 13006 | hackernews_1m | hackernews_1m | [ADD INDEX idx_author (`author`) USING INVERTED],  |
    ↪ 2023-06-26 17:23:02.610 | 2023-06-26 17:23:03.755 | 3004          | FINISHED |      |
    ↪ NULL     |
+--
    ↪ -----+--------------+--------------+-------------------------------------------------------+-----------
    ↪

-- 创建索引后，字符串等值匹配也有明显加速
SELECT count() FROM hackernews_1m WHERE author = 'faster';
+---------+
| count() |
+---------+
|      20 |
+---------+
```

2.8.6.4 BloomFilter 索引

2.8.6.4.1 索引原理

BloomFilter 索引是基于 BloomFilter 的一种跳数索引。它的原理是利用 BloomFilter 跳过等值查询指定条件不满足的数据块，达到减少 I/O 查询加速的目的。

BloomFilter 是由 Bloom 在 1970 年提出的一种多哈希函数映射的快速查找算法。通常应用在一些需要快速判断某个元素是否属于集合，但是并不严格要求 100% 正确的场合，BloomFilter 有以下特点：

- 空间效率高的概率型数据结构，用来检查一个元素是否在一个集合中。
- 对于一个元素检测是否存在的调用，BloomFilter 会告诉调用者两个结果之一：可能存在或者一定不存在。

BloomFilter 是由一个超长的二进制位数组和一系列的哈希函数组成。二进制位数组初始全部为 0，当给定一个待查询的元素时，这个元素会被一系列哈希函数计算映射出一系列的值，所有的值在位数组的偏移量处置为 1。

下图所示出一个 m=18, k=3（m 是该 Bit 数组的大小，k 是 Hash 函数的个数）的 BloomFilter 示例。集合中的 x、y、z 三个元素通过 3 个不同的哈希函数散列到位数组中。当查询元素 w 时，通过 Hash 函数计算之后只要有一个位为 0，因此 w 不在该集合中。但是反过来全部都是 1 只能说明可能在集合中、不能肯定一定在集合中，因为 Hash 函数可能出现 Hash 碰撞。

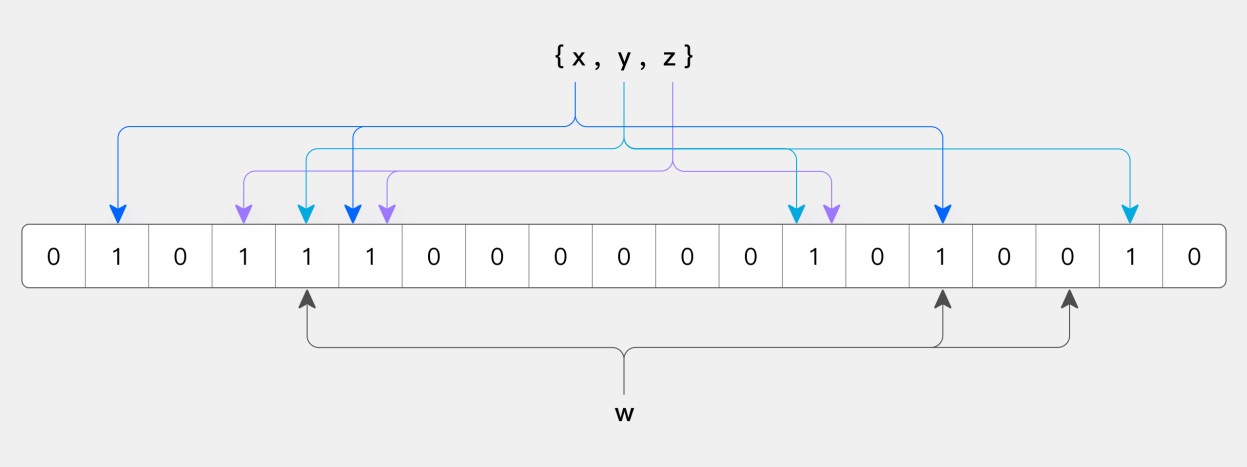


图 30: Bloom_filter.svg

反过来如果某个元素经过哈希函数计算后得到所有的偏移位置，若这些位置全都为 1，只能说明可能在集合中、不能肯定一定在集合中，因为 Hash 函数可能出现 Hash 碰撞。这就是 BloomFilter “假阳性”，因此基于 BloomFilter 的索引只能跳过不满足条件的数据，不能精确定位满足条件的数据。

Doris BloomFilter 索引以数据块（page）为单位构建，每个数据块存储一个 BloomFilter。写入时，对于数据块中的每个值，经过 Hash 存入数据块对应的 BloomFilter。查询时，根据等值条件的值，判断每个数据块对应的 BloomFilter 是否包含这个值，不包含则跳过对应的数据块不读取，达到减少 I/O 查询加速的目的。

2.8.6.4.2 使用场景

BloomFilter 索引能够对等值查询（包括 = 和 IN）加速，对高基数字段效果较好，比如 `userid` 等唯一 ID 字段。

BloomFilter 的使用有下面一些限制：

- 对 IN 和 = 之外的查询没有效果，比如!=, NOT IN, >, < 等
- 不支持对 Tinyint、 Float、 Double 类型的列建 BloomFilter 索引。
- 对低基数字段的加速效果很有限，比如 “性别” 字段仅有两种值，几乎每个数据块都会包含所有取值，导致 BloomFilter 索引失去意义。

如果要查看某个查询 BloomFilter 索引效果，可以通过 Query Profile 中的相关指标进行分析。

- BlockConditionsFilteredBloomFilterTime 是 BloomFilter 索引消耗的时间
- RowsBloomFilterFiltered 是 BloomFilter 过滤掉的行数，可以与其他几个 Rows 值对比分析 BloomFilter 索引过滤效果

2.8.6.4.3 管理索引

建表时创建 BloomFilter 索引

由于历史原因，BloomFilter 索引定义的语法与倒排索引等通用 INDEX 语法不一样。BloomFilter 索引通过表的 PROPERTIES “bloom_filter_columns” 指定哪些字段建 BloomFilter 索引，可以指定一个或者多个字段。

```
PROPERTIES (
"bloom_filter_columns" = "column_name1,column_name2"
);
```

查看 BloomFilter 索引

```
SHOW CREATE TABLE table_name;
```

已有表增加、删除 BloomFilter 索引

通过 ALTER TABLE 修改表的 bloom_filter_columns 属性来完成。

为 column_name3 增加 BloomFilter 索引

```
ALTER TABLE table_name SET ("bloom_filter_columns" = "column_name1,column_name2,column_name3");
```

删除 column_name1 的 BloomFilter 索引

```
ALTER TABLE table_name SET ("bloom_filter_columns" = "column_name2,column_name3");
```

2.8.6.4.4 使用索引

BloomFilter 索引用于加速 WHERE 条件中的等值查询，能加速时自动生效，没有特殊语法。

可以通过 Query Profile 中的下面几个指标分析 BloomFilter 索引的加速效果。- RowsBloomFilterFiltered BloomFilter 索引过滤掉的行数，可以与其他几个 Rows 值对比分析索引过滤效果 - BlockConditionsFilteredBloomFilterTime BloomFilter 索引消耗的时间

2.8.6.4.5 使用示例

下面通过实例来看看 Doris 怎么创建 BloomFilter 索引。

Doris BloomFilter 索引的创建是通过在建表语句的 PROPERTIES 里加上 “bloom_filter_columns”=“k1,k2,k3”, 这个属性，k1,k2,k3 是要创建的 BloomFilter 索引的 Key 列名称，例如下面对表里的 saler_id,category_id 创建了 BloomFilter 索引。

```
CREATE TABLE IF NOT EXISTS sale_detail_bloom  (
    sale_date date NOT NULL COMMENT "销售时间",
    customer_id int NOT NULL COMMENT "客户编号",
    saler_id int NOT NULL COMMENT "销售员",
    sku_id int NOT NULL COMMENT "商品编号",
    category_id int NOT NULL COMMENT "商品分类",
    sale_count int NOT NULL COMMENT "销售数量",
    sale_price DECIMAL(12,2) NOT NULL COMMENT "单价",
    sale_amt DECIMAL(20,2)  COMMENT "销售总金额"
```

```
)
Duplicate  KEY(sale_date, customer_id,saler_id,sku_id,category_id)
DISTRIBUTED BY HASH(saler_id) BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"bloom_filter_columns"="saler_id,category_id"
);
```

2.8.6.5 N-Gram 索引

2.8.6.5.1 索引原理

n-gram 分词是将一句话或一段文字拆分成多个相邻的词组的分词方法。NGram BloomFilter 索引和 BloomFilter 索引类似，也是基于 BloomFilter 的跳数索引。

与 BloomFilter 索引不同的是，NGram BloomFilter 索引用于加速文本 LIKE 查询，它存入 BloomFilter 的不是原始文本的值，而是对文本进行 NGram 分词，每个词作为值存入 BloomFilter。对于 LIKE 查询，将 LIKE '%pattern%' 的 pattern 也进行 NGram 分词，判断每个词是否在 BloomFilter 中，如果某个词不在则对应的数据块就不满足 LIKE 条件，可以跳过这部分数据减少 IO 加速查询。

2.8.6.5.2 使用场景

NGram BloomFilter 索引只能加速字符串 LIKE 查询，而且 LIKE pattern 中的连续字符个数要大于等于索引定义的 NGram 中的 N。

- NGram BloomFilter 只支持字符串列，只能加速 LIKE 查询。
- NGram BloomFilter 索引和 BloomFilter 索引为互斥关系，即同一个列只能设置两者中的一个。
- NGram BloomFilter 索引的效果分析，跟 BloomFilter 索引类似。

2.8.6.5.3 管理索引

创建 NGram BloomFilter 索引

在建表语句中 COLUMN 的定义之后是索引定义：

```
  INDEX `idx_column_name` (`column_name`) USING NGRAM_BF PROPERTIES("gram_size"="3", "bf_size="
      ↪ 1024") COMMENT 'username ngram_bf index'
```

语法说明如下：

1. `idx_column_name(column_name)` 是必须的，`column_name` 是建索引的列名，必须是前面列定义中出现过的，`idx_column_name` 是索引名字，必须表级别唯一，建议命名规范：列名前面加前缀 `idx_`

2. `USING NGRAM_BF` 是必须的，用于指定索引类型是 NGram BloomFilter 索引

3. `PROPERTIES` 是可选的，用于指定 NGram BloomFilter 索引的额外属性，目前支持的属性如下：

- gram_size：NGram 中的 N，指定 N 个连续字符分词一个词，比如 ‘This is a simple ngram example’ 在 N = 3 的时候分成 ‘This is a’，‘is a simple’，‘a simple ngram’，‘simple ngram example’ 4 个词。
- bf_size：BloomFilter 的大小，单位是 Bit。bf_size 决定每个数据块对应的索引大小，这个值越大占用存储空间越大，同时 Hash 碰撞的概率也越低。
- gram_size 建议取 LIKE 查询的字符串最小长度，但是不建议低于 2。一般建议设置 “gram_size” = “3”，“bf_size” = “1024”，然后根据 Query Profile 调优。

4. COMMENT 是可选的，用于指定索引注释

查看 NGram BloomFilter 索引

```
-- 语法 1，表的 schema 中 INDEX 部分 USING NGRAM_BF 是倒排索引
SHOW CREATE TABLE table_name;

-- 语法 2，IndexType 为 NGRAM_BF 的是倒排索引
SHOW INDEX FROM idx_name;
```

删除 NGram BloomFilter 索引

```
ALTER TABLE table_ngrambf DROP INDEX idx_ngrambf;
```

修改 NGram BloomFilter 索引

```
CREATE INDEX idx_column_name2(column_name2) ON table_ngrambf USING NGRAM_BF PROPERTIES("gram_size
    ↪ "="3", "bf_size"="1024") COMMENT 'username ngram_bf index';

ALTER TABLE table_ngrambf ADD INDEX idx_column_name2(column_name2) USING NGRAM_BF PROPERTIES("
    ↪ gram_size"="3", "bf_size"="1024") COMMENT 'username ngram_bf index';
```

2.8.6.5.4 使用索引

使用 NGram BloomFilter 索引需设置如下参数（enable_function_pushdown 默认为 false）：

```
SET enable_function_pushdown = true;
```

NGram BloomFilter 索引用于加速 LIKE 查询，比如：

```
SELECT count() FROM table1 WHERE message LIKE '%error%';
```

可以通过 Query Profile 中的下面几个指标分析 BloomFilter 索引（包括 NGram）的加速效果。- RowsBloomFilterFiltered BloomFilter 索引过滤掉的行数，可以与其他几个 Rows 值对比分析索引过滤效果 - BlockConditionsFilteredBloomFilterTime BloomFilter 索引消耗的时间

2.8.6.5.5 使用示例

以亚马逊产品的用户评论信息的数据集 amazon_reviews 为例展示 NGram BloomFilter 索引的使用和效果。

建表

```
CREATE TABLE `amazon_reviews` (
  `review_date` int(11) NULL,
  `marketplace` varchar(20) NULL,
  `customer_id` bigint(20) NULL,
  `review_id` varchar(40) NULL,
  `product_id` varchar(10) NULL,
  `product_parent` bigint(20) NULL,
  `product_title` varchar(500) NULL,
  `product_category` varchar(50) NULL,
  `star_rating` smallint(6) NULL,
  `helpful_votes` int(11) NULL,
  `total_votes` int(11) NULL,
  `vine` boolean NULL,
  `verified_purchase` boolean NULL,
  `review_headline` varchar(500) NULL,
  `review_body` string NULL
) ENGINE=OLAP
DUPLICATE KEY(`review_date`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`review_date`) BUCKETS 16
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"compression" = "ZSTD"
);
```

导入数据

用 wget 或者其他工具从下面的地址下载数据集

```
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2010.
    ↪ snappy.parquet
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2011.
    ↪ snappy.parquet
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2012.
    ↪ snappy.parquet
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2013.
    ↪ snappy.parquet
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2014.
    ↪ snappy.parquet
https://datasets-documentation.s3.eu-west-3.amazonaws.com/amazon_reviews/amazon_reviews_2015.
    ↪ snappy.parquet
```

用 stream load 导入数据

```
curl --location-trusted -u root: -T amazon_reviews_2010.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
curl --location-trusted -u root: -T amazon_reviews_2011.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
curl --location-trusted -u root: -T amazon_reviews_2012.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
curl --location-trusted -u root: -T amazon_reviews_2013.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
curl --location-trusted -u root: -T amazon_reviews_2014.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
curl --location-trusted -u root: -T amazon_reviews_2015.snappy.parquet -H "format:parquet" http
    ↪ ://127.0.0.1:8030/api/${DB}/amazon_reviews/_stream_load
```

上面的文件可能超过 10 GB，您可能需要调整 be.conf 的 streaming_load_max_mb 防止超过 stream load 文件上传大小的限制，可以通过下面方式动态调整

```
curl -X POST http://{be_ip}:{be_http_port}/api/update_config?streaming_load_max_mb
    ↪ =32768
```

需要每台 be 都执行上述命令。

SQL 运行 count() 确认导入数据成功

```
mysql> SELECT COUNT() FROM amazon_reviews;
+-----------+
| count(*)  |
+-----------+
| 135589433 |
+-----------+
```

查询

首先在没有索引的时候运行查询，WHERE 条件中有 LIKE，耗时 7.60s

```
SELECT
    product_id,
    any(product_title),
    AVG(star_rating) AS rating,
    COUNT() AS count
FROM
    amazon_reviews
WHERE
    review_body LIKE '%is super awesome%'
```

```
GROUP BY
    product_id
ORDER BY
    count DESC,
    rating DESC,
    product_id
LIMIT 5;


+------------+-----------------------------------------+--------------------+-------+
| product_id | any_value(product_title)                | rating             | count |
+------------+-----------------------------------------+--------------------+-------+
| B00992CF6W | Minecraft                               | 4.8235294117647056 |    17 |
| B009UX2YAC | Subway Surfers                          | 4.7777777777777777 |     9 |
| B00DJFIMW6 | Minion Rush: Despicable Me Official Game |              4.875 |     8 |
| B00867O0CM | Temple Run                              |                  5 |     6 |
| B00KWVZ75O | Angry Birds Epic RPG                    |                  5 |     6 |
+------------+-----------------------------------------+--------------------+-------+
5 rows in set (7.60 sec)
```

然后添加 NGram BloomFilter 索引，再次运行相同的查询耗时 0.93s，性能提升了 8 倍

```
ALTER TABLE amazon_reviews ADD INDEX review_body_ngram_idx(review_body) USING NGRAM_BF PROPERTIES
    ↪ ("gram_size"="10", "bf_size"="10240");
```

```
+------------+-----------------------------------------+--------------------+-------+
| product_id | any_value(product_title)                | rating             | count |
+------------+-----------------------------------------+--------------------+-------+
| B00992CF6W | Minecraft                               | 4.8235294117647056 |    17 |
| B009UX2YAC | Subway Surfers                          | 4.7777777777777777 |     9 |
| B00DJFIMW6 | Minion Rush: Despicable Me Official Game |              4.875 |     8 |
| B00867O0CM | Temple Run                              |                  5 |     6 |
| B00KWVZ75O | Angry Birds Epic RPG                    |                  5 |     6 |
+------------+-----------------------------------------+--------------------+-------+
5 rows in set (0.93 sec)
```

2.8.7 Schema 变更

用户可以通过Alter Table 操作来修改 Doris 表的 Schema。Schema 变更主要涉及列的修改和索引的变化。本文主要介绍列相关的 Schema 变更，关于索引相关的变更，请参考表索引了解不同索引的变更方法。

2.8.7.1 原理介绍

Doris 支持两种类型的 Schema Change 操作：轻量级 Schema Change 和重量级 Schema Change。它们的区别主要体现在执行过程的复杂性、执行速度和资源消耗上。

特性	轻量级 Schema Change	重量级 Schema Change
执行速度	秒级（几乎实时）	分钟级、小时级、天级（依赖表的数据量，数据量
是否需要数据重写	不需要	需要，涉及数据文件的重写
系统性能影响	影响较小	可能影响系统性能，尤其是在数据转换过程中
资源消耗	较低	较高，会占用计算资源重新组织数据，过程中涉及
操作类型	增加、删除 Value 列，修改列名，修改 VARCHAR 长度	修改列的数据类型、更改主键、修改列的顺序等

2.8.7.1.1 轻量级 Schema Change

轻量级 Schema Change 是指不涉及数据重写的简单模式更改操作。这些操作通常在元数据级别进行，仅需要修改表的元数据，而不涉及数据文件的物理修改。轻量级 Schema Change 操作通常能够在秒级别完成，不会对系统性能造成显著影响。轻量级 Schema Change 包括：

- 增加或删除 value 列
- 更改列名
- 修改 VARCHAR 列的长度（UNIQUE 和 DUP 表 Key 列除外）。

2.8.7.1.2 重量级 Schema Change

重量级 Schema Change 涉及到数据文件的重写或转换，这些操作相对复杂，通常需要借助 Doris 的 Backend（BE）进行数据的实际修改或重新组织。重量级 Schema Change 操作通常涉及对表数据结构的深度变更，可能会影响到存储的物理布局。所有不支持轻量级 Schema Change 的操作，均属于重量级 Schema Change，比如：

- 更改列的数据类型
- 修改列的排序顺序

重量级操作会在后台启动一个任务进行数据转换。后台任务会对表的每个 tablet 进行转换，按 tablet 为单位，将原始数据重写到新的数据文件中。数据转换过程中，可能会出现数据“双写”现象，即在转换期间，新数据同时写入新 tablet 旧 tablet 中。完成数据转换后，旧 tablet 会被删除，新 tablet 将取而代之。

2.8.7.2 作业管理

2.8.7.2.1 查看作业

用户可以通过SHOW ALTER TABLE COLUMN 命令查看 Schema Change 作业进度。可以查看当前正在执行或已经完成的 Schema Change 作业。当一次 Schema Change 作业涉及到物化视图时，该命令会显示多行，每行对应一个物化视图。举例如下：

```
mysql > SHOW ALTER TABLE COLUMN`\G`;
*************************** 1. row ***************************
        JobId: 20021
    TableName: tbl1
   CreateTime: 2019-08-05 23:03:13
   FinishTime: 2019-08-05 23:03:42
    IndexName: tbl1
```

```
      IndexId: 20022
OriginIndexId: 20017
SchemaVersion: 2:792557838
TransactionId: 10023
        State: FINISHED
          Msg:
     Progress: NULL
      Timeout: 86400
1 row in set (0.00 sec)
```

2.8.7.2.2 取消作业

在作业状态不为 FINISHED 或 CANCELLED 的情况下，可以通过以下命令取消 Schema Change 作业：

```
CANCEL ALTER TABLE COLUMN FROM tbl_name;
```

2.8.7.3 使用举例

2.8.7.3.1 修改列名称

```
ALTER TABLE [database.]table RENAME COLUMN old_column_name new_column_name;
```

具体语法参考ALTER TABLE RENAME。

2.8.7.3.2 添加一列

- 聚合模型如果增加 Value 列，需要指定 agg_type。
- 非聚合模型（如 DUPLICATE KEY）如果增加 Key 列，需要指定 KEY 关键字。

往非聚合表添加列

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
    col1 int,
    col2 int,
    col3 int,
    col4 int,
    col5 int
) DUPLICATE KEY(col1, col2, col3)
DISTRIBUTED BY RANDOM BUCKETS 10;
```

2. 向 example_db.my_table 的 col1 后添加一个 Key 列 key_col

```
ALTER TABLE example_db.my_table ADD COLUMN key_col INT KEY DEFAULT "0" AFTER col1;
```

3. 向 example_db.my_table 的 col4 后添加一个 Value 列 value_col

```
ALTER TABLE example_db.my_table ADD COLUMN value_col INT DEFAULT "0" AFTER col4;
```

往聚合表添加列

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
    col1 int,
    col2 int,
    col3 int,
    col4 int SUM,
    col5 varchar(32) REPLACE DEFAULT "abc"
) AGGREGATE KEY(col1, col2, col3)
DISTRIBUTED BY HASH(col1) BUCKETS 10;
```

2. 向 example_db.my_table 的 col1 后添加一个 Key 列 key_col

```
ALTER TABLE example_db.my_table ADD COLUMN key_col INT DEFAULT "0" AFTER col1;
```

3. 向 example_db.my_table 的 col4 后添加一个 Value 列 value_col SUM 聚合类型

```
ALTER TABLE example_db.my_table ADD COLUMN value_col INT SUM DEFAULT "0" AFTER col4;
```

2.8.7.3.3 添加多列

- 聚合模型如果增加 Value 列，需要指定 agg_type
- 聚合模型如果增加 Key 列，需要指定 KEY 关键字

向聚合表添加多列

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
    col1 int,
    col2 int,
    col3 int,
    col4 int SUM,
    col5 varchar(32) REPLACE DEFAULT "abc"
) AGGREGATE KEY(col1, col2, col3)
DISTRIBUTED BY HASH(col1) BUCKETS 10;
```

2. 向 example_db.my_table添加多列 (聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN (c1 INT DEFAULT "1", c2 FLOAT SUM DEFAULT "0");
```

2.8.7.3.4 删除列

- 不能删除分区列
- 不能删除 UNIQUE 的 KEY 列。

从 example_db.my_table 删除一列

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
    col1 int,
    col2 int,
    col3 int,
    col4 int SUM,
    col5 varchar(32) REPLACE DEFAULT "abc"
) AGGREGATE KEY(col1, col2, col3)
DISTRIBUTED BY HASH(col1) BUCKETS 10;
```

2. 从 example_db.my_table 删除col3列

```
ALTER TABLE example_db.my_table DROP COLUMN col4;
```

2.8.7.3.5 修改列类型和列位置

- 聚合模型如果修改 Value 列，需要指定 agg_type
- 非聚合类型如果修改 Key 列，需要指定 KEY 关键字
- 只能修改列的类型，列的其他属性维持原样
- 分区列和分桶列不能做任何修改
- 目前支持以下类型的转换（用户需要注意精度损失）
- TINYINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE 类型向范围更大的数字类型转换
- TINTINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE/DECIMAL 转换成 VARCHAR
- VARCHAR 支持修改最大长度
- VARCHAR/CHAR 转换成 TINTINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE
- VARCHAR/CHAR 转换成 DATE (目前支持“%Y-%m-%d”，“%y-%m-%d”，“%Y%m%d”，“%y%m%d”，“%Y/%m/%d,”%y/%m/%d”六种格式化格式)
- DATETIME 转换成 DATE (仅保留年 - 月 - 日信息，例如：2019-12-09 21:47:05 <--> 2019-12-09)
- DATE 转换成 DATETIME (时分秒自动补零，例如：2019-12-09 <--> 2019-12-09 00:00:00)
- FLOAT 转换成 DOUBLE
- INT 转换成 DATE (如果 INT 类型数据不合法则转换失败，原始数据不变)
- 除 DATE 与 DATETIME 以外都可以转换成 STRING，但是 STRING 不能转换任何其他类型

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
    col0 int,
    col1 int DEFAULT "1",
    col2 int,
    col3 varchar(32),
    col4 int SUM,
    col5 varchar(32) REPLACE DEFAULT "abc"
) AGGREGATE KEY(col0, col1, col2, col3)
DISTRIBUTED BY HASH(col0) BUCKETS 10;
```

2. 修改 Key 列 col1 的类型为 BIGINT，并移动到 col2 列后面

```
ALTER TABLE example_db.my_table
MODIFY COLUMN col1 BIGINT KEY DEFAULT "1" AFTER col2;
```

注意：无论是修改 Key 列还是 Value 列都需要声明完整的 Column 信息

2. 修改 Base Table 的 val1 列最大长度。原 val1 为 (val1 VARCHAR(32) REPLACE DEFAULT “abc”)

```
ALTER TABLE example_db.my_table
MODIFY COLUMN col5 VARCHAR(64) REPLACE DEFAULT "abc";
```

注意：只能修改列的类型，列的其他属性需要维持原样

3. 修改 Key 列的某个字段的长度

```
ALTER TABLE example_db.my_table
MODIFY COLUMN col3 varchar(50) KEY NULL comment 'to 50';
```

2.8.7.3.6 重新排序

- 所有列都要写出来
- Value 列在 Key 列之后

1. 建表语句

```
CREATE TABLE IF NOT EXISTS example_db.my_table(
k1 int DEFAULT "1",
k2 int,
k3 varchar(32),
k4 date,
v1 int SUM,
v2 int MAX,
) AGGREGATE KEY(k1, k2, k3, k4)
DISTRIBUTED BY HASH(k1) BUCKETS 10;
```

2. 重新排序 example_db.my_table 中的列

```
ALTER TABLE example_db.my_table
ORDER BY (k3,k1,k2,k4,v2,v1);
```

2.8.7.4 限制

- 一张表在同一时间只能有一个 Schema Change 作业在运行。
- 分区列和分桶列不能修改。
- 如果聚合表中有 REPLACE 方式聚合的 Value 列，则不允许删除 Key 列。
- Unique 表不允许删除 Key 列。
- 在新增聚合类型为 SUM 或者 REPLACE 的 Value 列时，该列的默认值对历史数据没有含义。

- 因为历史数据已经失去明细信息，所以默认值的取值并不能实际反映聚合后的取值。
- 当修改列类型时，除 Type 以外的字段都需要按原列上的信息补全。
- 注意，除新的列类型外，如聚合方式，Nullable 属性，以及默认值都要按照原信息补全。
- 不支持修改聚合类型、Nullable 属性和默认值。

2.8.7.5 相关配置

2.8.7.5.1 FE 配置

- `alter_table_timeout_second`：作业默认超时时间，86400 秒。

2.8.7.5.2 BE 配置

- `alter_tablet_worker_count`：在 BE 端用于执行历史数据转换的线程数。默认为 3。如果希望加快 Schema Change 作业的速度，可以适当调大这个参数后重启 BE。但过多的转换线程可能会导致 IO 压力增加，影响其他操作。
- `alter_index_worker_count`：在 BE 端用于执行历史数据构建索引的线程数（注：当前只支持倒排索引）。默认为 3。如果希望加快 Index Change 作业的速度，可以适当调大这个参数后重启 BE。但过多的线程可能会导致 IO 压力增加，影响其他操作。

2.8.8 自增列

在 Doris 中，自增列（Auto Increment Column）是一种自动生成唯一数字值的功能，常用于为每一行数据生成唯一的标识符，如主键。每当插入新记录时，自增列会自动分配一个递增的值，避免了手动指定数字的繁琐操作。使用 Doris 自增列，可以确保数据的唯一性和一致性，简化数据插入过程，减少人为错误，并提高数据管理的效率。这使得自增列成为处理需要唯一标识的场景（如用户 ID 等）时的理想选择。

2.8.8.1 功能

对于具有自增列的表，Doris 处理数据写入的方式如下：

- 自动填充（列排除）：如果写入的数据不包括自增列，Doris 会生成并填充该列的唯一值。
- 部分指定（列包含）：
- 空值：Doris 会用系统生成的唯一值替换写入数据中的空值。
- 非空值：用户提供的值保持不变。

> 重要用户提供的非空值可能会破坏自增列的唯一性。

2.8.8.1.1 唯一性

Doris 保证自增列中生成的值具有表级唯一性。但是：

- 保证唯一性：这仅适用于系统生成的值。
- 用户提供的值：Doris 不会验证或强制执行用户在自增列中指定的值的唯一性。这可能导致重复条目。

2.8.8.1.2 聚集性

Doris 生成的自增值通常是密集的，但有一些考虑：

- 潜在的间隙：由于性能优化，可能会出现间隙。每个后端节点（BE）会预分配一块唯一值以提高效率，这些块在节点之间不重叠。
- 非时间顺序值：Doris 不保证后续写入生成的值大于早期写入的值。

> 注意自增值不能用于推断写入的时间顺序。

2.8.8.2 语法

要使用自增列，需要在建表CREATE-TABLE时为对应的列添加AUTO_INCREMENT属性。若要手动指定自增列起始值，可以通过建表时AUTO_INCREMENT(start_value)语句指定，如果未指定，则默认起始值为 1。

2.8.8.2.1 示例

1. 创建一个 Dupliciate 模型表，其中一个 key 列是自增列

```
CREATE TABLE `demo`.`tbl` (
      `id` BIGINT NOT NULL AUTO_INCREMENT,
      `value` BIGINT NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

2. 创建一个 Dupliciate 模型表，其中一个 key 列是自增列，并设置起始值为 100

```
CREATE TABLE `demo`.`tbl` (
      `id` BIGINT NOT NULL AUTO_INCREMENT(100),
      `value` BIGINT NOT NULL
) ENGINE=OLAP
```

```
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

3. 创建一个 Dupliciate 模型表，其中一个 value 列是自增列

```
CREATE TABLE `demo`.`tbl` (
      `uid` BIGINT NOT NULL,
      `name` BIGINT NOT NULL,
      `id` BIGINT NOT NULL AUTO_INCREMENT,
      `value` BIGINT NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`uid`, `name`)
DISTRIBUTED BY HASH(`uid`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

4. 创建一个 Unique 模型表，其中一个 key 列是自增列

```
CREATE TABLE `demo`.`tbl` (
      `id` BIGINT NOT NULL AUTO_INCREMENT,
      `name` varchar(65533) NOT NULL,
      `value` int(11) NOT NULL
) ENGINE=OLAP
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

5. 创建一个 Unique 模型表，其中一个 value 列是自增列

```
CREATE TABLE `demo`.`tbl` (
      `text` varchar(65533) NOT NULL,
      `id` BIGINT NOT NULL AUTO_INCREMENT,
) ENGINE=OLAP
UNIQUE KEY(`text`)
DISTRIBUTED BY HASH(`text`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

2.8.8.2.2 约束和限制

1. 仅 Duplicate 模型表和 Unique 模型表可以包含自增列。
2. 一张表最多只能包含一个自增列。
3. 自增列的类型必须是 BIGINT 类型，且必须为 NOT NULL。
4. 自增列手动指定的起始值必须大于等于 0。

2.8.8.3 使用方式

2.8.8.3.1 普通导入

以下表为例：

```
CREATE TABLE `demo`.`tbl` (
    `id` BIGINT NOT NULL AUTO_INCREMENT,
    `name` varchar(65533) NOT NULL,
    `value` int(11) NOT NULL
) ENGINE=OLAP
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

使用 insert into 语句导入并且不指定自增列id时，id列会被自动填充生成的值。

```
insert into tbl(name, value) values("Bob", 10), ("Alice", 20), ("Jack", 30);

select * from tbl order by id;
+------+-------+-------+
| id   | name  | value |
+------+-------+-------+
|    1 | Bob   |    10 |
|    2 | Alice |    20 |
|    3 | Jack  |    30 |
+------+-------+-------+
```

类似地，使用 stream load 导入文件 test.csv 且不指定自增列id，id列会被自动填充生成的值。

test.csv:

```
Tom,40
John,50
```

```
curl --location-trusted -u user:passwd -H "columns:name,value" -H "column_separator:," -T ./test.
    ↪ csv http://{host}:{port}/api/{db}/tbl/_stream_load
```

```
select * from tbl order by id;
+------+-------+-------+
| id   | name  | value |
+------+-------+-------+
|    1 | Bob   |    10 |
|    2 | Alice |    20 |
|    3 | Jack  |    30 |
|    4 | Tom   |    40 |
|    5 | John  |    50 |
+------+-------+-------+
```

使用 insert into 导入时指定自增列id，则该列数据中的 null 值会被生成的值替换。

```
insert into tbl(id, name, value) values(null, "Doris", 60), (null, "Nereids", 70);

select * from tbl order by id;
+------+---------+-------+
| id   | name    | value |
+------+---------+-------+
|    1 | Bob     |    10 |
|    2 | Alice   |    20 |
|    3 | Jack    |    30 |
|    4 | Tom     |    40 |
|    5 | John    |    50 |
|    6 | Doris   |    60 |
|    7 | Nereids |    70 |
+------+---------+-------+
```

2.8.8.3.2 部分列更新

在对一张包含自增列的 merge-on-write Unique 表进行部分列更新时，如果自增列是 key 列，由于部分列更新时用户必须显示指定 key 列，部分列更新的目标列必须包含自增列。此时的导入行为和普通的部分列更新相同。

```
CREATE TABLE `demo`.`tbl2` (
  `id` BIGINT NOT NULL AUTO_INCREMENT,
  `name` varchar(65533) NOT NULL,
  `value` int(11) NOT NULL DEFAULT "0"
  ) ENGINE=OLAP
  UNIQUE KEY(`id`)
  DISTRIBUTED BY HASH(`id`) BUCKETS 10
  PROPERTIES (
  "replication_allocation" = "tag.location.default: 3",
  "enable_unique_key_merge_on_write" = "true"
  );
```

```
insert into tbl2(id, name, value) values(1, "Bob", 10), (2, "Alice", 20), (3, "Jack", 30);

select * from tbl2 order by id;
+------+-------+-------+
| id   | name  | value |
+------+-------+-------+
|    1 | Bob   |    10 |
|    2 | Alice |    20 |
|    3 | Jack  |    30 |
+------+-------+-------+


set enable_unique_key_partial_update=true;
set enable_insert_strict=false;
insert into tbl2(id, name) values(1, "modified"), (4, "added");

select * from tbl2 order by id;
+------+----------+-------+
| id   | name     | value |
+------+----------+-------+
|    1 | modified |    10 |
|    2 | Alice    |    20 |
|    3 | Jack     |    30 |
|    4 | added    |     0 |
+------+----------+-------+
```

当自增列是非 key 列时，如果用户没有指定自增列的值，其值会从表中原有的数据行中进行补齐。如果用户指定了自增列，则该列数据中的 null 值会被替换为生成出的值，非 null 值则保持不变，然后以部分列更新的语义插入该表。

```
CREATE TABLE `demo`.`tbl3` (
  `id` BIGINT NOT NULL,
  `name` varchar(100) NOT NULL,
  `score` BIGINT NOT NULL,
  `aid` BIGINT NOT NULL AUTO_INCREMENT
  ) ENGINE=OLAP
  UNIQUE KEY(`id`)
  DISTRIBUTED BY HASH(`id`) BUCKETS 1
  PROPERTIES (
  "replication_allocation" = "tag.location.default: 3",
  "enable_unique_key_merge_on_write" = "true"
  );


insert into tbl3(id, name, score) values(1, "Doris", 100), (2, "Nereids", 200), (3, "Bob", 300);
```

```
select * from tbl3 order by id;
+------+---------+-------+------+
| id   | name    | score | aid  |
+------+---------+-------+------+
|    1 | Doris   |   100 |    0 |
|    2 | Nereids |   200 |    1 |
|    3 | Bob     |   300 |    2 |
+------+---------+-------+------+

set enable_unique_key_partial_update=true;
set enable_insert_strict=false;
insert into tbl3(id, score) values(1, 999), (2, 888);

select * from tbl3 order by id;
+------+---------+-------+------+
| id   | name    | score | aid  |
+------+---------+-------+------+
|    1 | Doris   |   999 |    0 |
|    2 | Nereids |   888 |    1 |
|    3 | Bob     |   300 |    2 |
+------+---------+-------+------+

insert into tbl3(id, aid) values(1, 1000), (3, 500);

select * from tbl3 order by id;
+------+---------+-------+------+
| id   | name    | score | aid  |
+------+---------+-------+------+
|    1 | Doris   |   999 | 1000 |
|    2 | Nereids |   888 |    1 |
|    3 | Bob     |   300 |  500 |
+------+---------+-------+------+
```

2.8.8.4 使用场景

2.8.8.4.1 字典编码

在用户画像场景中使用 bitmap 做人群分析时需要构建用户字典，每个用户对应一个唯一的整数字典值，聚集的字典值可以获得更好的 bitmap 性能。

以离线 uv，pv 分析场景为例，假设有如下用户行为表存放明细数据：

```
CREATE TABLE `demo`.`dwd_dup_tbl` (
    `user_id` varchar(50) NOT NULL,
    `dim1` varchar(50) NOT NULL,
    `dim2` varchar(50) NOT NULL,
```

```
    `dim3` varchar(50) NOT NULL,
    `dim4` varchar(50) NOT NULL,
    `dim5` varchar(50) NOT NULL,
    `visit_time` DATE NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`user_id`)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 32
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

利用自增列创建如下字典表

```
CREATE TABLE `demo`.`dictionary_tbl` (
    `user_id` varchar(50) NOT NULL,
    `aid` BIGINT NOT NULL AUTO_INCREMENT
) ENGINE=OLAP
UNIQUE KEY(`user_id`)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 32
PROPERTIES (
"replication_allocation" = "tag.location.default: 3",
"enable_unique_key_merge_on_write" = "true"
);
```

将存量数据中的user_id导入字典表，建立user_id到整数值的编码映射

```
insert into dictionary_tbl(user_id)
select user_id from dwd_dup_tbl group by user_id;
```

或者使用如下方式仅将增量数据中的user_id导入到字典表

```
insert into dictionary_tbl(user_id)
select dwd_dup_tbl.user_id from dwd_dup_tbl left join dictionary_tbl
on dwd_dup_tbl.user_id = dictionary_tbl.user_id where dwd_dup_tbl.visit_time > '2023-12-10' and
    ↪ dictionary_tbl.user_id is NULL;
```

实际场景中也可以使用 flink connector 把数据写入到 doris。

假设dim1, dim3, dim5是我们关心的统计维度，建立如下聚合表存放聚合结果

```
CREATE TABLE `demo`.`dws_agg_tbl` (
    `dim1` varchar(50) NOT NULL,
    `dim3` varchar(50) NOT NULL,
    `dim5` varchar(50) NOT NULL,
    `user_id_bitmap` BITMAP BITMAP_UNION NOT NULL,
    `pv` BIGINT SUM NOT NULL
) ENGINE=OLAP
AGGREGATE KEY(`dim1`,`dim3`,`dim5`)
```

```
DISTRIBUTED BY HASH(`dim1`) BUCKETS 32
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

将数据聚合运算后存放至聚合结果表

```
insert into dws_agg_tbl
select dwd_dup_tbl.dim1, dwd_dup_tbl.dim3, dwd_dup_tbl.dim5, BITMAP_UNION(TO_BITMAP(dictionary_
    ↪ tbl.aid)), COUNT(1)
from dwd_dup_tbl INNER JOIN dictionary_tbl on dwd_dup_tbl.user_id = dictionary_tbl.user_id
group by dwd_dup_tbl.dim1, dwd_dup_tbl.dim3, dwd_dup_tbl.dim5;
```

用如下语句进行 uv, pv 查询

```
select dim1, dim3, dim5, bitmap_count(user_id_bitmap) as uv, pv from dws_agg_tbl;
```

2.8.8.4.2 高效分页

在页面展示数据时，往往需要做分页展示。传统的分页通常使用 SQL 中的 `limit, offset + order by` 进行查询。例如有如下业务表需要进行展示：

```
CREATE TABLE `demo`.`records_tbl` (
    `user_id` int(11) NOT NULL COMMENT "",
    `name` varchar(26) NOT NULL COMMENT "",
    `address` varchar(41) NOT NULL COMMENT "",
    `city` varchar(11) NOT NULL COMMENT "",
    `nation` varchar(16) NOT NULL COMMENT "",
    `region` varchar(13) NOT NULL COMMENT "",
    `phone` varchar(16) NOT NULL COMMENT "",
    `mktsegment` varchar(11) NOT NULL COMMENT ""
) DUPLICATE KEY (`user_id`, `name`)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

假设在分页展示中，每页展示 100 条数据。那么获取第 1 页的数据可以使用如下 sql 进行查询：

```
select * from records_tbl order by user_id, name limit 100;
```

获取第 2 页的数据可以使用如下 sql 进行查询：

```
select * from records_tbl order by user_id, name limit 100 offset 100;
```

然而，当进行深分页查询时 (offset 很大时)，即使实际需要需要的数据行很少，该方法依然会将全部数据读取到内存中进行全量排序后再进行后续处理，这种方法比较低效。可以通过自增列给每行数据一个唯一

值，在查询时就可以通过记录之前页面unique_value列的最大值max_value，然后使用 where unique_value > max_value limit rows_per_page 的方式通过提下推谓词提前过滤大量数据，从而更高效地实现分页。

仍然以上述业务表为例，通过在表中添加一个自增列从而赋予每一行一个唯一标识：

```
CREATE TABLE `demo`.`records_tbl2` (
    `user_id` int(11) NOT NULL COMMENT "",
    `name` varchar(26) NOT NULL COMMENT "",
    `address` varchar(41) NOT NULL COMMENT "",
    `city` varchar(11) NOT NULL COMMENT "",
    `nation` varchar(16) NOT NULL COMMENT "",
    `region` varchar(13) NOT NULL COMMENT "",
    `phone` varchar(16) NOT NULL COMMENT "",
    `mktsegment` varchar(11) NOT NULL COMMENT "",
    `unique_value` BIGINT NOT NULL AUTO_INCREMENT
) DUPLICATE KEY (`user_id`, `name`)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

在分页展示中，每页展示 100 条数据，使用如下方式获取第一页的数据：

```
select * from records_tbl2 order by unique_value limit 100;
```

通过程序记录下返回结果中unique_value中的最大值，假设为 99，则可用如下方式查询第 2 页的数据：

```
select * from records_tbl2 where unique_value > 99 order by unique_value limit 100;
```

如果要直接查询一个靠后页面的内容，此时不方便直接获取之前页面数据中unique_value的最大值时，例如要直接获取第 101 页的内容，则可以使用如下方式进行查询

```
select user_id, name, address, city, nation, region, phone, mktsegment
from records_tbl2, (select unique_value as max_value from records_tbl2 order by unique_value
    ↪ limit 1 offset 9999) as previous_data
where records_tbl2.unique_value > previous_data.max_value
order by unique_value limit 100;
```

2.8.9 冷热数据分层

2.8.9.1 冷热数据分层概述

为了帮助用户节省存储成本，Doris 针对冷数据提供了灵活的选择。

冷数据选择	适用条件	特性
存算分离	用户具备部署存算分离的条件	- 数据以单副本完全存储在对象存储中 - 通过本地缓存加速热数据访问 - 存储与计算资源独立扩展，显著降低存储成本
本地分层	存算一体模式下，用户希望进一步优化本地存储资源	- 支持将冷数据从 SSD 冷却到 HDD- 充分利用本地存储层级特性，节省高性能存储成本
远程分层	存算一体模式下，使用廉价的对象存储或者 HDFS 进一步降低成本	- 冷数据以单副本形式保存到对象存储或者 HDFS 中 - 热数据继续使用本地存储 - 不能对一个表和本地分层混合使用

通过上述模式，Doris 能够灵活适配用户的部署条件，实现查询效率与存储成本的平衡。

2.8.9.2 SSD 和 HDD 层级存储

Doris 支持在不同磁盘类型（SSD 和 HDD）之间进行分层存储，结合动态分区功能，根据冷热数据的特性将数据从 SSD 动态迁移到 HDD。这种方式既降低了存储成本，又在热数据的读写上保持了高性能。

2.8.9.2.1 动态分区与层级存储

通过配置动态分区参数，用户可以设置哪些分区存储在 SSD 上，以及冷却后自动迁移到 HDD 上。

- 热分区：最近活跃的分区，优先存储在 SSD 上，保证高性能。
- 冷分区：较少访问的分区，会逐步迁移到 HDD，以降低存储开销。

有关动态分区的更多信息，请参考：数据划分 - 动态分区。

2.8.9.2.2 参数说明

`dynamic_partition.hot_partition_num`

- 功能：
- 指定最近的多少个分区为热分区，这些分区存储在 SSD 上，其余分区存储在 HDD 上。
- 注意：
- 必须同时设置 `dynamic_partition.storage_medium = HDD`，否则此参数不会生效。
- 如果存储路径下没有 SSD 设备，则该配置会导致分区创建失败。

示例说明：

假设当前日期为 2021-05-20，按天分区，动态分区配置如下：

```
dynamic_partition.hot_partition_num = 2
dynamic_partition.start = -3
dynamic_partition.end = 3
```

系统会自动创建以下分区，并配置其存储介质和冷却时间：

```
p20210517: ["2021-05-17", "2021-05-18") storage_medium=HDD storage_cooldown_time=9999-12-31
    ↪ 23:59:59
p20210518: ["2021-05-18", "2021-05-19") storage_medium=HDD storage_cooldown_time=9999-12-31
    ↪ 23:59:59
p20210519: ["2021-05-19", "2021-05-20") storage_medium=SSD storage_cooldown_time=2021-05-21
    ↪ 00:00:00
p20210520: ["2021-05-20", "2021-05-21") storage_medium=SSD storage_cooldown_time=2021-05-22
    ↪ 00:00:00
p20210521: ["2021-05-21", "2021-05-22") storage_medium=SSD storage_cooldown_time=2021-05-23
    ↪ 00:00:00
p20210522: ["2021-05-22", "2021-05-23") storage_medium=SSD storage_cooldown_time=2021-05-24
    ↪ 00:00:00
p20210523: ["2021-05-23", "2021-05-24") storage_medium=SSD storage_cooldown_time=2021-05-25
    ↪ 00:00:00
```

dynamic_partition.storage_medium

- 功能：
- 指定动态分区的最终存储介质。默认是 HDD，可选择 SSD。
- 注意：
- 当设置为 SSD 时，hot_partition_num 属性将不再生效，所有分区将默认为 SSD 存储介质并且冷却时间为 9999-12-31 23:59:59。

2.8.9.2.3 示例

1. 创建一个分层存储表

```
CREATE TABLE tiered_table (k DATE)
PARTITION BY RANGE(k)()
DISTRIBUTED BY HASH (k) BUCKETS 5
PROPERTIES
(
    "dynamic_partition.storage_medium" = "hdd",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.hot_partition_num" = "2",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "5",
    "dynamic_partition.create_history_partition"= "true",
    "dynamic_partition.start" = "-3"
);
```

2. 检查分区存储介质

```
    SHOW PARTITIONS FROM tiered_table;
```

可以看见 7 个分区，5 个使用 SSD, 其它的 2 个使用 HDD。

```
  p20210517: ["2021-05-17", "2021-05-18") storage_medium=HDD storage_cooldown_time=9999-12-31
      ↪ 23:59:59
  p20210518: ["2021-05-18", "2021-05-19") storage_medium=HDD storage_cooldown_time=9999-12-31
      ↪ 23:59:59
  p20210519: ["2021-05-19", "2021-05-20") storage_medium=SSD storage_cooldown_time=2021-05-21
      ↪ 00:00:00
  p20210520: ["2021-05-20", "2021-05-21") storage_medium=SSD storage_cooldown_time=2021-05-22
      ↪ 00:00:00
  p20210521: ["2021-05-21", "2021-05-22") storage_medium=SSD storage_cooldown_time=2021-05-23
      ↪ 00:00:00
  p20210522: ["2021-05-22", "2021-05-23") storage_medium=SSD storage_cooldown_time=2021-05-24
      ↪ 00:00:00
  p20210523: ["2021-05-23", "2021-05-24") storage_medium=SSD storage_cooldown_time=2021-05-25
      ↪ 00:00:00
```

2.8.9.3 远程存储

2.8.9.3.1 概述

远程存储支持将冷数据放到外部存储（例如对象存储，HDFS）上。

> 注意远程存储的数据只有一个副本，数据可靠性依赖远程存储的数据可靠性，您需要保证远程存储有 ec（擦除码）或者多副本技术确保数据可靠性。

2.8.9.3.2 使用方法

冷数据保存到 S3 兼容存储

第一步： 创建 S3 Resource。

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
    "type" = "s3",
    "s3.endpoint" = "bj.s3.com",
    "s3.region" = "bj",
    "s3.bucket" = "test-bucket",
    "s3.root.path" = "path/to/root",
```

```
    "s3.access_key" = "bbb",
    "s3.secret_key" = "aaaa",
    "s3.connection.maximum" = "50",
    "s3.connection.request.timeout" = "3000",
    "s3.connection.timeout" = "1000"
);
```

> 创建 S3 RESOURCE 的时候，会进行 S3 远端的链接校验，以保证 RESOURCE 创建的正确。

第二步： 创建 STORAGE POLICY。

之后创建 STORAGE POLICY，关联上文创建的 RESOURCE：

```
CREATE STORAGE POLICY test_policy
PROPERTIES(
    "storage_resource" = "remote_s3",
    "cooldown_ttl" = "1d"
);
```

第三步： 建表时使用 STORAGE POLICY。

```
CREATE TABLE IF NOT EXISTS create_table_use_created_policy
(
    k1 BIGINT,
    k2 LARGEINT,
    v1 VARCHAR(2048)
)
UNIQUE KEY(k1)
DISTRIBUTED BY HASH (k1) BUCKETS 3
PROPERTIES(
    "enable_unique_key_merge_on_write" = "false",
    "storage_policy" = "test_policy"
);
```

> 注意 UNIQUE 表如果设置了 `"enable_unique_key_merge_on_write" = "true"` 的话，无法使用此功能。

冷数据保存到 HDFS

第一步： 创建 HDFS RESOURCE：

```
CREATE RESOURCE "remote_hdfs" PROPERTIES (
        "type"="hdfs",
        "fs.defaultFS"="fs_host:default_fs_port",
        "hadoop.username"="hive",
        "hadoop.password"="hive",
        "root_path"="/my/root/path",
        "dfs.nameservices" = "my_ha",
        "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
        "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
        "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
        "dfs.client.failover.proxy.provider.my_ha" = "org.apache.hadoop.hdfs.server.namenode.ha.
            ↪ ConfiguredFailoverProxyProvider"
    );
```

第二步：创建 STORAGE POLICY。

```
CREATE STORAGE POLICY test_policy PROPERTIES (
    "storage_resource" = "remote_hdfs",
    "cooldown_ttl" = "300"
)
```

第三步：使用 STORAGE POLICY 创建表。

```
CREATE TABLE IF NOT EXISTS create_table_use_created_policy (
    k1 BIGINT,
    k2 LARGEINT,
    v1 VARCHAR(2048)
)
UNIQUE KEY(k1)
DISTRIBUTED BY HASH (k1) BUCKETS 3
PROPERTIES(
"enable_unique_key_merge_on_write" = "false",
"storage_policy" = "test_policy"
);
```

> 注意 UNIQUE 表如果设置了 `"enable_unique_key_merge_on_write" = "true"` 的话，无法使用此功能。

存量表冷却到远程存储

除了新建表支持设置远程存储外，Doris 还支持对一个已存在的表或者 PARTITION，设置远程存储。

对一个已存在的表，设置远程存储，将创建好的 STORAGE POLICY 与表关联：

```
ALTER TABLE create_table_not_have_policy set ("storage_policy" = "test_policy");
```

对一个已存在的 PARTITION，设置远程存储，将创建好的 STORAGE POLICY 与 PARTITON 关联：

```
ALTER TABLE create_table_partition MODIFY PARTITION (*) SET("storage_policy"="test_policy");
```

> 注意，如果用户在建表时给整张 Table 和部分 Partition 指定了不同的 Storage Policy，Partition 设置的 Storage policy 会被无视，整张表的所有 Partition 都会使用 table 的 Policy. 如果您需要让某个 Partition 的 Policy 和别的不同，则可以使用上文中对一个已存在的 Partition，关联 Storage policy 的方式修改。
>
> 具体可以参考 Docs 目录下RESOURCE、 POLICY、 CREATE TABLE、 ALTER TABLE等文档。

配置 compaction

- BE 参数`cold_data_compaction_thread_num`可以设置执行远程存储的 Compaction 的并发，默认是 2。
- BE 参数`cold_data_compaction_interval_sec`可以设置执行远程存储的 Compaction 的时间间隔，默认是 1800，单位：秒，即半个小时。

2.8.9.3.3 限制

- 使用了远程存储的表不支持备份。
- 不支持修改远程存储的位置信息，比如 endpoint、 bucket、 path。
- Unique 模型表在开启 Merge-on-Write 特性时，不支持设置远程存储。
- Storage policy 支持创建、修改和删除，删除前需要先保证没有表引用此 Storage policy。
- 一旦设置了 Storage policy 之后，不能取消设置。

2.8.9.3.4 冷数据空间

查看

方式一：通过 show proc ’/backends’ 可以查看到每个 BE 上传到对象的大小，RemoteUsedCapacity 项，此方式略有延迟。

方式二：通过 show tablets from tableName 可以查看到表的每个 tablet 占用的对象大小，RemoteDataSize 项。

垃圾回收

远程存储上可能会有如下情况产生垃圾数据：

1. 上传 rowset 失败但是有部分 segment 上传成功。

2. 上传的 rowset 没有在多副本达成一致。

3. Compaction 完成后，参与 compaction 的 rowset。

垃圾数据并不会立即清理掉。BE 参数`remove_unused_remote_files_interval_sec`可以设置远程存储的垃圾回收的时间间隔，默认是 21600，单位：秒，即 6 个小时。

2.8.9.3.5 查询与性能优化

为了优化查询的性能和对象存储资源节省，引入了本地 Cache。在第一次查询远程存储的数据时，Doris 会将远程存储的数据加载到 BE 的本地磁盘做缓存，Cache 有以下特性：

- Cache 实际存储于 BE 本地磁盘，不占用内存空间。
- Cache 是通过 LRU 管理的，不支持 TTL。

具体配置请参考 (../../lakehouse/data-cache)。

2.8.9.3.6 常见问题

1. `ERROR 1105 (HY000): errCode = 2, detailMessage = Failed to create repository: connect to s3 failed: Unable to marshall request to JSON: host must not be null.`

S3 SDK 默认使用 virtual-hosted style 方式。但某些对象存储系统 (如：minio) 可能没开启或没支持 virtual-hosted style 方式的访问，此时我们可以添加 use_path_style 参数来强制使用 path style 方式：

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
    "type" = "s3",
    "s3.endpoint" = "bj.s3.com",
    "s3.region" = "bj",
    "s3.bucket" = "test-bucket",
    "s3.root.path" = "path/to/root",
    "s3.access_key" = "bbb",
    "s3.secret_key" = "aaaa",
    "s3.connection.maximum" = "50",
    "s3.connection.request.timeout" = "3000",
    "s3.connection.timeout" = "1000",
    "use_path_style" = "true"
);
```

2. 修改冷却时间相关参数之后的行为表现是怎么样的？

冷却时间相关的参数修改之后只对还未冷却到远程存储的数据生效，对于已经冷却到远程存储的数据不生效。比如将 `cooldown_ttl` 从 21 天修改为 7 天，已经在远程存储的数据不会回到本地；

2.8.10 行列混存

2.8.10.1 行列混存介绍

Doris 默认采用列式存储，每个列连续存储，在分析场景（如聚合，过滤，排序等）有很好的性能，因为只需要读取所需要的列减少不必要的 IO。但是在点查场景（比如 SELECT *），需要读取所有列，每个列都需要一次 IO 导致 IOPS 成为瓶颈，特别对列多的宽表（比如上百列）尤为明显。

为了解决点查场景 IOPS 的瓶颈问题，Doris 2.0.0 版本开始支持行列混存，用户建表时指定开启行存后，点查（比如 SELECT *）每一行只需要一次 IO，在宽表列很多的情况下性能有数量级提升。

行存的原理是在存储时增加了一个额外的列，这个列将对应行的所有列拼接起来采用特殊的二进制格式存储。

2.8.10.2 使用语法

建表时在表的 PROPERTIES 中指定是否开启行存，哪些列开启行存，行存的存储压缩单元大小 page_size。

1. 是否开启行存：默认为 false 不开启

```
"store_row_column" = "true"
```

2. 哪些列开启行存：如果 "store_row_column" = "true"，默认所有列开启行存，若需要指定部分列开启行存，设置 row_store_columns 参数（3.0 之后的版本），格式为逗号分割的列名

```
"row_store_columns" = "column1,column2,column3"
```

3. 行存 page_size：默认为 16KB。

```
"row_store_page_size" = "16384"
```

page 是存储读写的最小单元，page_size 是行存 page 的大小，也就是说读一行也需要产生一个 page 的 IO。这个值越大压缩效果越好存储空间占用越低，但是点查时 IO 开销越大性能越低（因为一次 IO 至少读一个 page），反过来值越小存储空间极高，点查性能越好。默认值 16KB 是大多数情况下比较均衡的选择，如果更偏向查询性能可以配置较小的值比如 4KB 甚至更低，如果更偏向存储空间可以配置较大的值比如 64KB 甚至更高。

2.8.10.3 行存命中条件

行存命中条件分成两种情况，一种是高并发主键点查需要依赖表的属性以及查询满足点查条件，另一种是单表 SELECT * 查询，下面针对这两种查询进行说明。

- 对于主键高并发点查，建表属性需要开启 "enable_unique_key_merge_on_write" = "true"（MOW 表）以及 "store_row_column" = "true"（所有列都会在行存中单独额外存一份，存储代价相对较高）或者 "row_store_columns" = "key,v1,v3,v5,v7"（只会存储询部分列到行存中）。查询的时候注意 where 条件中需要有所有的主键等值并且是 AND，例如SELECT * FROM tbl WHERE k1 = 1 AND k2 = 2 或者查询部分列 SELECT v1, v2 FROM tbl WHERE k1 = 1 AND k2 = 2，如果行存只包含了部分列（v1），但是查询的列不在行存中（例如 v2），那么将会从列存中查询剩余的列，该例子中 v1 将会从行存查询，而 v2 会从列存中查询（列存的 page size 更大，会有更多的读放大），通过 EXPLAIN 可以确认是否命中主键高并发点查优化，更多点查的使用请参考高并发点查。

- 对于一般的非主键点查，如果想要走行存那么表模型 DUPLICATE 或者开启"enable_unique_key_merge_on_write" = "true"（MOW 表），以及及 "store_row_column" = "true"（所有列都会在行存中单独额外存一份，存储代价相对较高）。查询满足这种模式将可以命中行存SELECT * FROM tble [WHERE XXXXX] ORDER BY XXX LIMIT N 方括号中的是可选查询条件，注意目前只能是SELECT *，且需要命中 TOPN 的延迟物化优化，具体参考TOPN 查询优化，即命中OPT TWO PHASE。最后通过 EXPLAIN 查看是否有有FETCH ROW STORE相应的标记即可确认命中行存

2.8.10.4 使用示例

下面的例子创建一个 8 列的表，其中 “key,v1,v3,v5,v7” 这 5 列开启行存，为了高并发点查性能配置 page_size 为 4KB。

```
CREATE TABLE `tbl_point_query` (
    `k` int(11) NULL,
    `v1` decimal(27, 9) NULL,
    `v2` varchar(30) NULL,
    `v3` varchar(30) NULL,
    `v4` date NULL,
    `v5` datetime NULL,
    `v6` float NULL,
    `v7` datev2 NULL
) ENGINE=OLAP
UNIQUE KEY(`k`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`k`) BUCKETS 1
PROPERTIES (
    "enable_unique_key_merge_on_write" = "true",
    "light_schema_change" = "true",
    "row_store_columns" = "k,v1,v3,v5,v7",
    "row_store_page_size" = "4096"
);
```

查询 1

```
SELECT k, v1, v3, v5, v7 FROM tbl_point_query WHERE k = 100
```

explain 上述语句应该包含 SHORT-CIRCUIT 相应的标记。更多点查的使用请参考高并发点查 。

下面这个例子展示了 DUPLICATE 表怎么命中行存查询条件

```
CREATE TABLE `tbl_duplicate` (
    `k` int(11) NULL,
    `v1` string NULL
) ENGINE=OLAP
DUPLICATE KEY(`k`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`k`) BUCKETS 1
PROPERTIES (
```

```
    "light_schema_change" = "true",
    "store_row_column" = "true",
    "row_store_page_size" = "4096"
);
```

"store_row_column" = "true", 是必须的

查询 2 (注意命中 TOPN 查询优化以及需要是SELECT *)

```
SELECT * FROM tbl_duplicate WHERE k < 10 ORDER BY k LIMIT 10
```

explain 上述语句应该包含FETCH ROW STORE 相应的标记，以及OPT TWO PHASE标记

2.8.10.5 注意事项

1. 开启行存后占用的存储空间会增加，存储空间的增加和数据特点有关，一般是原来表的 2 到 10 倍，具体空间占用需要使用实际数据测试。
2. 行存的 page_size 对存储空间的也有影响，可以根据前面的表属性参数 row_store_page_size 说明进行调整。

2.8.11 数据库建表最佳实践

2.8.11.1 1 数据表模型

Doris 数据表模型上目前分为三类：DUPLICATE KEY, UNIQUE KEY, AGGREGATE KEY。

推荐规约

因为数据模型在建表时就已经确定，且无法修改。所以，选择一个合适的数据模型非常重要。

1. Duplicate 适合任意维度的 Ad-hoc 查询。虽然同样无法利用预聚合的特性，但是不受聚合模型的约束，可以发挥列存模型的优势 (只读取相关列，而不需要读取所有 Key 列)。
2. Aggregate 模型可以通过预聚合，极大地降低聚合查询时所需扫描的数据量和查询的计算量，非常适合有固定模式的报表类查询场景。但是该模型对 count(*) 查询很不友好。同时因为固定了 Value 列上的聚合方式，在进行其他类型的聚合查询时，需要考虑语意正确性。
3. Unique 模型针对需要唯一主键约束的场景，可以保证主键唯一性约束。但是无法利用物化等预聚合带来的查询优势。对于聚合查询有较高性能需求的用户，推荐使用自 1.2 版本加入的写时合并实现。
4. 如果有部分列更新的需求，可以选择：
    a. Unique 模型的 Merge-on-Write 模式
    b. Aggregate 模型的 REPLACE_IF_NOT_NULL 聚合方式

2.8.11.1.1 01 DUPLICATE KEY 表模型

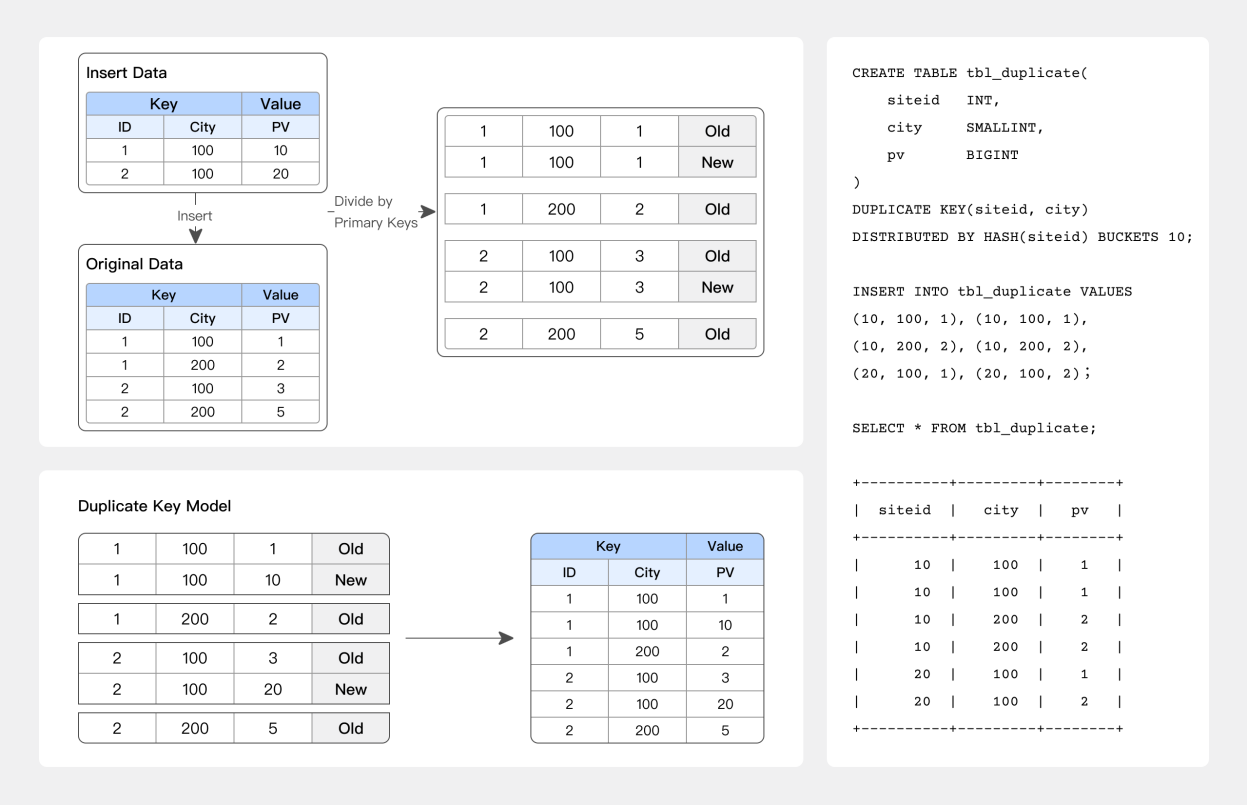


图 31: DUPLICATE KEY 表模型

只指定排序列，相同的 KEY 行不会合并。

适用于数据无需提前聚合的分析业务：

- 原始数据分析
- 仅追加新数据的日志或时序数据分析

最佳实践

```
-- 例如 允许 KEY 重复仅追加新数据的日志数据分析
CREATE TABLE session_data
(
    visitorid   SMALLINT,
    sessionid   BIGINT,
    visittime   DATETIME,
    city        CHAR(20),
    province    CHAR(20),
    ip          varchar(32),
    brower      CHAR(20),
```

```
    url          VARCHAR(1024)
)
DUPLICATE KEY(visitorid, sessionid) -- 只用于指定排序列，相同的 KEY 行不会合并
DISTRIBUTED BY HASH(sessionid, visitorid) BUCKETS 10;
```

2.8.11.1.2 02 AGGREGATE KEY 表模型

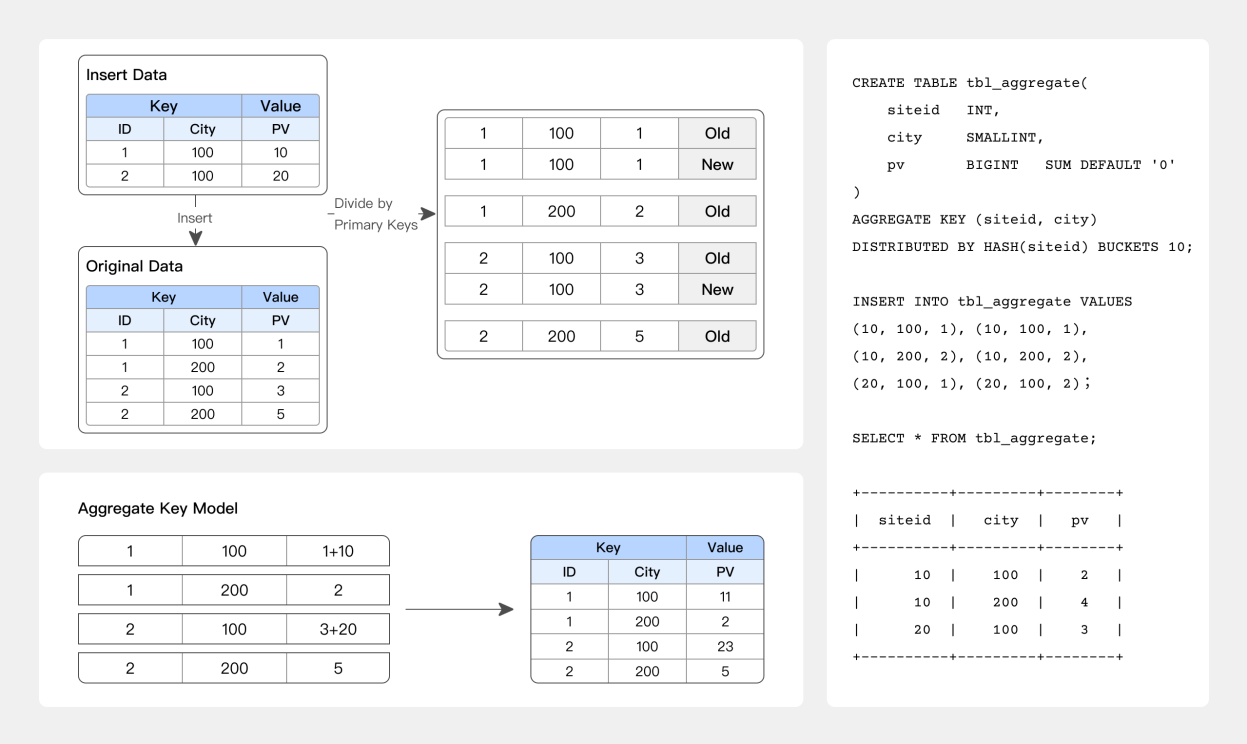


图 32: AGGREGATE KEY 表模型

AGGREGATE KEY 相同时，新旧记录进行聚合，目前支持的聚合方式：

1. SUM：求和，多行的 Value 进行累加。
2. REPLACE：替代，下一批数据中的 Value 会替换之前导入过的行中的 Value。
3. MAX：保留最大值。
4. MIN：保留最小值。
5. REPLACE_IF_NOT_NULL：非空值替换。和 REPLACE 的区别在于对于 null 值，不做替换。
6. HLL_UNION：HLL 类型的列的聚合方式，通过 HyperLogLog 算法聚合。
7. BITMAP_UNION：BIMTAP 类型的列的聚合方式，进行位图的并集聚合。

适合报表和多维分析业务：

- 网站流量分析
- 数据报表多维分析

最佳实践

```
-- 例如 网站流量分析
CREATE TABLE site_visit
(
    siteid      INT,
    city        SMALLINT,
    username    VARCHAR(32),
    pv BIGINT   SUM DEFAULT '0' -- PV 浏览量计算
)
AGGREGATE KEY(siteid, city, username) -- 相同的 KEY 行会合并，非 KEY
    ↪ 列会根据指定的聚合函数进行聚合
DISTRIBUTED BY HASH(siteid) BUCKETS 10;
```

2.8.11.1.3 03 UNIQUE KEY 表模型

UNIQUE KEY 相同时，新记录覆盖旧记录。在 1.2 版本之前，UNIQUE KEY 实现上和 AGGREGATE KEY 的 REPLACE 聚合方法一样，二者本质上相同，自 1.2 版本我们给 UNIQUE KEY 引入了 merge on write 实现，该实现有更好的聚合查询性能。

适用于有更新需求的分析业务：

- 订单去重分析
- 实时增删改同步

最佳实践

```
-- 例如 订单去重分析
CREATE TABLE sales_order
(
    orderid     BIGINT,
    status      TINYINT,
    username    VARCHAR(32),
    amount      BIGINT DEFAULT '0'
)
UNIQUE KEY(orderid) -- 相同的 KEY 行会合并
DISTRIBUTED BY HASH(orderid) BUCKETS 10;
```

2.8.11.2 2 索引

索引用于帮助快速过滤或查找数据。目前主要支持两类索引：

1. 内建自动创建的智能索引，包括前缀索引和 ZoneMap 索引。
2. 用户手动创建的二级索引，包括倒排索引、bloomfilter 索引、ngram bloomfilter 索引和 bitmap 索引。

2.8.11.2.1 01 前缀索引

在 Aggregate、 Unique 和 Duplicate 三种数据模型中。底层的数据存储，是按照各自建表语句中，AGGREGATE KEY、UNIQUE KEY 和 DUPLICATE KEY 中指定的列进行排序存储的。而前缀索引，即在排序的基础上，实现的一种根据给定前缀列，快速查询数据的索引方式。

前缀索引是稀疏索引，不能精确定位到 Key 所在的行，只能粗粒度地定位出 Key 可能存在的范围，然后使用二分查找算法精确地定位 Key 的位置。

推荐规约

1. 建表时，正确的选择列顺序，能够极大地提高查询效率。
   因为建表时已经指定了列顺序，所以一个表只有一种前缀索引。这对于使用其他不能命中前缀索引的列作为条件进行的查询来说，效率上可能无法满足需求，这种情况，我们可以通过创建物化视图来人为的调整列顺序。
2. 前缀索引的第一个字段一定是最常查询的字段，并且需要是高基数字段：
   a. 分桶字段注意事项：这个一般是数据分布比较均衡的，也是经常使用的字段，最好是高基数字段
   b. Int（4）+ Int（4）+ varchar(50)，前缀索引长度只有 28
   c. Int（4）+ varchar(50) + Int（4），前缀索引长度只有 24
   d. varchar(10) + varchar(50)，前缀索引长度只有 30
   e. 前缀索引（36 位）：第一个字段查询性能最好，前缀索引碰见 varchar 类型的字段，会自动截断前 20 个字符
   f. 最常用的查询字段如果能放到前缀索引里尽可能放到前前缀索引里，如果不能，可以放到分桶字段里
3. 前缀索引中的字段长度尽可能明确，因为 Doris 只有前 36 个字节能走前缀索引。
4. 如果某个范围数据在分区分桶和前缀索引中都不好设计，可以考虑引入倒排索引加速。

2.8.11.2.2 02 ZoneMap 索引

ZoneMap 索引是在列存格式上，对每一列自动维护的索引信息，包括 Min/Max，null 值个数等等。在数据查询时，会根据范围条件过滤的字段按照 ZoneMap 统计信息选取扫描的数据范围。

例如对 age 字段进行过滤，查询语句如下：

```
SELECT * FROM table WHERE age > 0 and age < 51;
```

在没有命中 Short Key Index 的情况下，会根据条件语句中 age 的查询条件，利用 ZoneMap 索引找到应该扫描的数据 ordinary 范围，减少要扫描的 page 数量。

2.8.11.2.3 03 倒排索引

从 2.0.0 版本开始，Doris 支持倒排索引，可以用来进行文本类型的全文检索、普通数值日期类型的等值范围查询，快速从海量数据中过滤出满足条件的行。

最佳实践

```
-- 创建示例：可以表创建时指定或者创建后新增，如下创建表时指定
CREATE TABLE table_name
(
  columns_difinition,
  INDEX idx_name1(column_name1) USING INVERTED [PROPERTIES("parser" = "english|unicode|chinese")]
      ↪  [COMMENT 'your comment']
  INDEX idx_name2(column_name2) USING INVERTED [PROPERTIES("parser" = "english|unicode|chinese")]
      ↪  [COMMENT 'your comment']
  INDEX idx_name3(column_name3) USING INVERTED [PROPERTIES("parser" = "chinese", "parser_mode" =
      ↪ "fine_grained|coarse_grained")] [COMMENT 'your comment']
  INDEX idx_name4(column_name4) USING INVERTED [PROPERTIES("parser" = "english|unicode|chinese",
      ↪ "support_phrase" = "true|false")] [COMMENT 'your comment']
  INDEX idx_name5(column_name4) USING INVERTED [PROPERTIES("char_filter_type" = "char_replace", "
      ↪ char_filter_pattern" = "._"), "char_filter_replacement" = " "] [COMMENT 'your comment']
  INDEX idx_name5(column_name4) USING INVERTED [PROPERTIES("char_filter_type" = "char_replace", "
      ↪ char_filter_pattern" = "._")] [COMMENT 'your comment']
)
table_properties;

-- 使用示例：全文检索关键词匹配，通过 MATCH_ANY MATCH_ALL 完成
SELECT * FROM table_name WHERE column_name MATCH_ANY | MATCH_ALL 'keyword1 ...';
```

推荐规约

1. 如果某个范围数据在分区分桶和前缀索引中都不好设计，可以考虑引入倒排索引加速。

强制规约 1. 倒排索引在不同数据模型中有不同的使用限制：

a. Aggregate KEY 表模型：只能为 Key 列建立倒排索引。

```
b. Unique KEY 表模型：需要开启 merge on write 特性，开启后，
    ↪ 可以为任意列建立倒排索引。

c. Duplicate KEY 表模型：可以为任意列建立倒排索引。
```

2.8.11.2.4 04 BloomFilter 索引

Doris 支持用户对取值区分度比较大的字段添加 BloomFilter 索引，适合在基数较高的列上进行等值查询的场景。

最佳实践

```
-- 创建示例：通过在建表语句的 PROPERTIES 里加上"bloom_filter_columns"="k1,k2,k3"
-- 例如下面我们对表里的 saler_id,category_id 创建了 BloomFilter 索引。
CREATE TABLE IF NOT EXISTS sale_detail_bloom  (
    sale_date date NOT NULL COMMENT "销售时间",
    customer_id int NOT NULL COMMENT "客户编号",
    saler_id int NOT NULL COMMENT "销售员",
    sku_id int NOT NULL COMMENT "商品编号",
    category_id int NOT NULL COMMENT "商品分类",
    sale_count int NOT NULL COMMENT "销售数量",
    sale_price DECIMAL(12,2) NOT NULL COMMENT "单价",
    sale_amt DECIMAL(20,2)  COMMENT "销售总金额"
)
Duplicate  KEY(sale_date, customer_id,saler_id,sku_id,category_id)
DISTRIBUTED BY HASH(saler_id) BUCKETS 10
PROPERTIES (
"bloom_filter_columns"="saler_id,category_id"
);
```

强制规约

1. 不支持对 Tinyint、 Float、 Double 类型的列建 BloomFilter 索引。
2. BloomFilter 索引只对 in 和 = 过滤查询有加速效果。
3. BloomFilter 索引必须在查询条件是 in 或者 =，并且是高基数（5000 以上）列上构建。
    a. 首先 BloomFilter 适用于非前缀过滤
    b. 查询会根据该列高频过滤，而且查询条件大多是 in 和 = 过滤
    c. 不同于 Bitmap, BloomFilter 适用于高基数列。比如 UserID。因为如果创建在低基数的列上，比如“性别”列，则每个 Block 几乎都会包含所有取值，导致 BloomFilter 索引失去意义

d. 数据基数在一半左右
e. 类似身份证号这种基数特别高并且查询是等值（=）查询，使用 BloomFilter 索引能极大加速

2.8.11.2.5 05 NGram BloomFilter 索引

从 2.0.0 版本开始，Doris 为了提升 LIKE 的查询性能，增加了 NGram BloomFilter 索引。

最佳实践

```
-- 创建示例：表创建时指定
CREATE TABLE `nb_table` (
  `siteid` int(11) NULL DEFAULT "10" COMMENT "",
  `citycode` smallint(6) NULL COMMENT "",
  `username` varchar(32) NULL DEFAULT "" COMMENT "",
  INDEX idx_ngrambf (`username`) USING NGRAM_BF PROPERTIES("gram_size"="3", "bf_size"="256")
      ↪ COMMENT 'username ngram_bf index'
) ENGINE=OLAP
AGGREGATE KEY(`siteid`, `citycode`, `username`) COMMENT "OLAP"
DISTRIBUTED BY HASH(`siteid`) BUCKETS 10;

-- PROPERTIES("gram_size"="3", "bf_size"="256")，分别表示 gram 的个数和 bloom filter 的字节数。
-- gram 的个数跟实际查询场景相关，通常设置为大部分查询字符串的长度，bloom filter 字节数，
    ↪ 可以通过测试得出，通常越大过滤效果越好，可以从 256 开始进行验证测试看看效果。
    ↪ 当然字节数越大也会带来索引存储、内存 cost 上升。
-- 如果数据基数比较高，字节数可以不用设置过大，如果基数不是很高，可以通过增加字节数来提升过滤效果
    ↪ 。
```

强制规约

1. NGram BloomFilter 只支持字符串列
2. NGram BloomFilter 索引和 BloomFilter 索引为互斥关系，即同一个列只能设置两者中的一个
3. NGram 大小和 BloomFilter 的字节数，可以根据实际情况调优，如果 NGram 比较小，可以适当增加 BloomFilter 大小
4. 亿级别以上数据，如果有模糊匹配，使用倒排索引或者是 NGram Bloomfilter

2.8.11.2.6 2.6 Bitmap 索引

为了加速数据查询，Doris 支持用户为某些字段添加 Bitmap 索引，适合在基数较低的列上进行等值查询或范围查询的场景。

最佳实践

```
-- 创建示例：在 bitmap_table 上为 siteid 创建 Bitmap 索引
CREATE INDEX [IF NOT EXISTS] bitmap_index_name ON
bitmap_table (siteid)
USING BITMAP COMMENT 'bitmap_siteid';
```

强制规约

1. Bitmap 索引仅在单列上创建。
2. Bitmap 索引能够应用在 Duplicate、Uniq 数据模型的所有列和 Aggregate模型的 key 列上。
3. Bitmap 索引支持的数据类型如下：

- TINYINT
- SMALLINT
- INT
- BIGINT
- CHAR
- VARCHAR
- DATE
- DATETIME
- LARGEINT
- DECIMAL
- BOOL

4. Bitmap 索引仅在 Segment V2 下生效。当创建 Index 时，表的存储格式将默认转换为 V2 格式。
5. Bitmap 索引必须在一定基数范围内构建，太高或者太低的基数都不合适
    a. 适用于低基数的列上，建议在 100 到 100,000 之间，如：职业、地市等。重复度过高则对比其他类型索引没有明显优势；重复度过低，则空间效率和性能会大大降低。特定类型的查询例如 COUNT, OR, AND 等逻辑操作因为只需要进行位运算
    b. 该索引更多的适合正交查询

2.8.11.3 3 字段类型

Doris 支持多种字段类型，例如精确去重 BITMAP、模糊去重 HLL、半结构化 ARRAY/MAP/JSON 和常见的数字、字符串和时间类型等。

推荐规约

1. VARCHAR
    a. 变长字符串，长度范围为：1-65533 字节长度，以 UTF-8 编码存储的，因此通常英文字符占 1 个字节，中文字符占 3 个字节。

b. 这里存在一个误区，即 varchar(255) 和 varchar(65533) 的性能问题，这二者如果存的数据是一样的，性能也是一样的，建表时如果不确定这个字段最大有多长，建议直接使用 65533 即可，防止由于字符串过长导致的导入问题。

2. STRING
   a. 变长字符串，默认支持 1048576 字节（1MB），可调大到 2147483643 字节（2G），以 UTF-8 编码存储的，因此通常英文字符占 1 个字节，中文字符占 3 个字节。
   b. 只能用在 Value 列，不能用在 Key 列和分区分桶列。
   c. 适用于一些比较大的文本存储，一般如果没有这种需求的话，建议使用 VARCHAR，STRING 列无法用在 Key 列和分桶列，局限性比较大。
3. 数值型字段：按照精度选择对应的数据类型即可，没有过于特殊的注意。
4. 时间字段：这里需要注意的是，如果有高精度（毫秒值时间戳）需求，需要指明使用 datetime(6)，否则默认是不支持毫秒值时间戳的。
5. 建议使用 JSON 数据类型代替字符串类型存放 JSON 数据的使用方式。

2.8.11.4 4 数据表创建

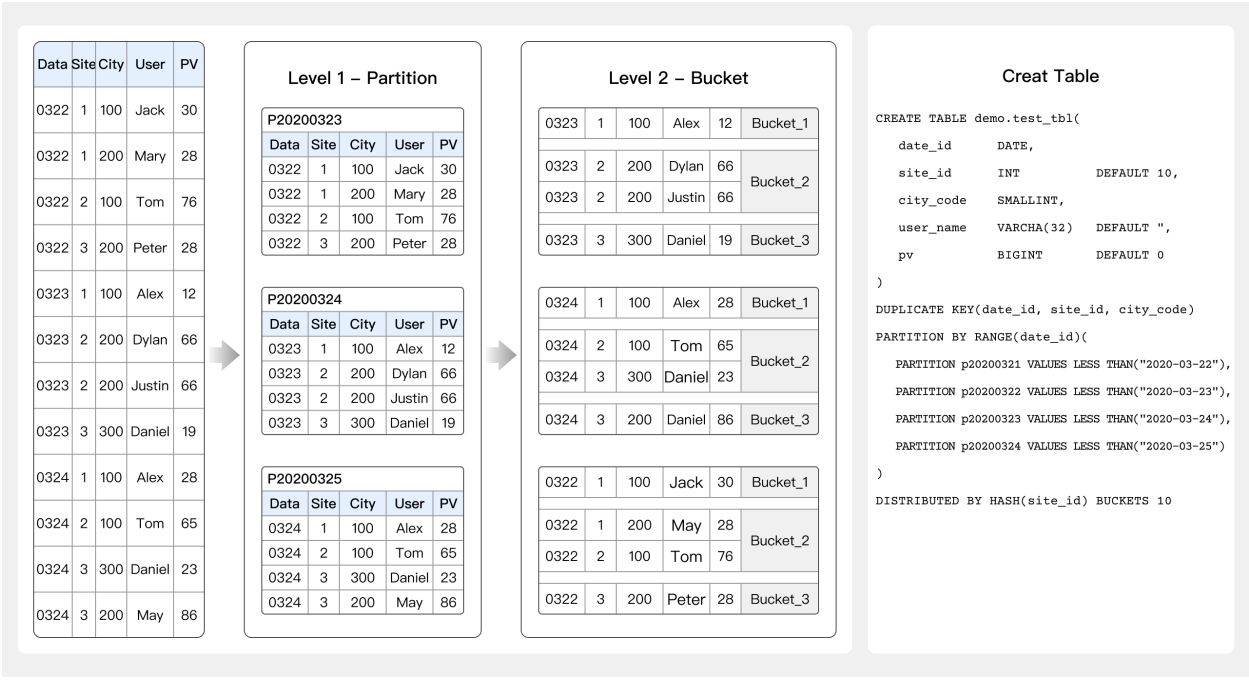


图 33: 数据表创建

建表时除了要注意数据表模型、索引和字段类型的选择还需要注意分区分桶的设置。

最佳实践

```
-- 以 Unique 模型的 Merge-on-Write 表为例
-- Unique 模型的写时合并实现，与聚合模型就是完全不同的两种模型了，查询性能更接近于 duplicate 模型
    ↪ ，
-- 在有主键约束需求的场景上相比聚合模型有较大的查询性能优势，
    ↪ 尤其是在聚合查询以及需要用索引过滤大量数据的查询中。

-- 非分区表
CREATE TABLE IF NOT EXISTS tbl_unique_merge_on_write
(
    `user_id` LARGEINT NOT NULL COMMENT "用户id",
    `username` VARCHAR(50) NOT NULL COMMENT "用户昵称",
    `register_time` DATE COMMENT "用户注册时间",
    `city` VARCHAR(20) COMMENT "用户所在城市",
    `age` SMALLINT COMMENT "用户年龄",
    `sex` TINYINT COMMENT "用户性别",
    `phone` LARGEINT COMMENT "用户电话",
    `address` VARCHAR(500) COMMENT "用户地址"
)
UNIQUE KEY(`user_id`, `username`)
-- 3-5G 的数据量
DISTRIBUTED BY HASH(`user_id`) BUCKETS 10
PROPERTIES (
-- 在 1.2.0 版本中，作为一个新的 feature，写时合并默认关闭，用户可以通过添加下面的 property
    ↪ 来开启
"enable_unique_key_merge_on_write" = "true"
);

-- 分区表
CREATE TABLE IF NOT EXISTS tbl_unique_merge_on_write_p
(
    `user_id` LARGEINT NOT NULL COMMENT "用户id",
    `username` VARCHAR(50) NOT NULL COMMENT "用户昵称",
    `register_time` DATE COMMENT "用户注册时间",
    `city` VARCHAR(20) COMMENT "用户所在城市",
    `age` SMALLINT COMMENT "用户年龄",
    `sex` TINYINT COMMENT "用户性别",
    `phone` LARGEINT COMMENT "用户电话",
    `address` VARCHAR(500) COMMENT "用户地址"
)
UNIQUE KEY(`user_id`, `username`, `register_time`)
PARTITION BY RANGE(`register_time`) (
    PARTITION p00010101_1899 VALUES [('0001-01-01'), ('1900-01-01')),
    PARTITION p19000101 VALUES [('1900-01-01'), ('1900-01-02')),
    PARTITION p19000102 VALUES [('1900-01-02'), ('1900-01-03')),
    PARTITION p19000103 VALUES [('1900-01-03'), ('1900-01-04')),
```

```
    PARTITION p19000104_1999 VALUES [('1900-01-04'), ('2000-01-01')),
    FROM ("2000-01-01") TO ("2022-01-01") INTERVAL 1 YEAR,
    PARTITION p30001231 VALUES [('3000-12-31'), ('3001-01-01')),
    PARTITION p99991231 VALUES [('9999-12-31'), (MAXVALUE))
)
-- 默认 3-5G 的数据量
DISTRIBUTED BY HASH(`user_id`) BUCKETS 10
PROPERTIES (
-- 在 1.2.0 版本中，作为一个新的 feature，写时合并默认关闭，用户可以通过添加下面的 property
    ↪ 来开启
"enable_unique_key_merge_on_write" = "true",
-- 动态分区调度的单位。可指定为 HOUR、DAY、WEEK、MONTH、YEAR。分别表示按小时、按天、按星期、按月
    ↪ 、按年进行分区创建或删除。
"dynamic_partition.time_unit" = "MONTH",
-- 动态分区的起始偏移，为负数。根据 time_unit 属性的不同，以当天（星期/月）为基准，
    ↪ 分区范围在此偏移之前的分区将会被删除（TTL）。如果不填写，则默认为 -2147483648，
    ↪ 即不删除历史分区。
"dynamic_partition.start" = "-3000",
-- 动态分区的结束偏移，为正数。根据 time_unit 属性的不同，以当天（星期/月）为基准，
    ↪ 提前创建对应范围的分区。
"dynamic_partition.end" = "10",
-- 动态创建的分区名前缀（必选）。
"dynamic_partition.prefix" = "p",
-- 动态创建的分区所对应的分桶数量。
"dynamic_partition.buckets" = "10",
"dynamic_partition.enable" = "true",
-- 动态创建的分区所对应的副本数量，如果不填写，则默认为该表创建时指定的副本数量 3。
"dynamic_partition.replication_num" = "3",
"replication_num" = "3"
);

-- 分区创建查看
-- 实际创建的分区数需要结合 dynamic_partition.start、end 以及 PARTITION BY RANGE 的设置共同决定
show partitions from tbl_unique_merge_on_write_p;
```

强制规约

1. 数据库字符集指定 UTF-8，并且只支持 UTF-8。
2. 表的副本数必须为 3（未指定副本数时，默认为 3）。
3. 单个 Tablet（Tablet 数 = 分区数 * 桶数 * 副本数）的数据量理论上没有上下界，除小表（百兆维表）外需确保在 1G - 10G 的范围内：
    a. 如果单个 Tablet 数据量过小，则数据的聚合效果不佳，且元数据管理压力大。

b. 如果数据量过大，则不利于副本的迁移、补齐，且会增加 Schema Change 或者物化操作失败重试的代价（这些操作失败重试的粒度是 Tablet）。

4. 5 亿以上的数据必须设置分区分桶策略：

   a. bucket 设置建议：

      i. 大表的单个 Tablet 存储数据大小在 1G-10G 区间，可防止过多的小文件产生。

      ii. 百兆左右的维表 Tablet 数量控制在 3-5 个，保证一定的并发数也不会产生过多的小文件。

   b. 没有办法分区的，数据又较快增长的，没办法按照时间动态分区，可以适当放大一下你的 Bucket 数量，按照你的数据保存周期（180 天）数据总量，来估算你的 Bucket 数量应该是多少，建议还是单个 Bucket 大小在 1-10G。

   c. 对分桶字段进行加盐处理，业务上查询的时候也是要同样的加盐策略，这样能利用到分桶数据剪裁能力。

   d. 数据随机分桶：

      i. 如果 OLAP 表没有更新类型的字段，将表的数据分桶模式设置为 RANDOM，则可以避免严重的数据倾斜 (数据在导入表对应的分区的时候，单次导入作业每个 Batch 的数据将随机选择一个 Tablet 进行写入)。

      ii. 当表的分桶模式被设置为 RANDOM 时，因为没有分桶列，无法根据分桶列的值仅对几个分桶查询，对表进行查询的时候将对命中分区的全部分桶同时扫描，该设置适合对表数据整体的聚合查询分析而不适合高并发的点查询。

      iii. 如果 OLAP 表的是 Random Distribution 的数据分布，那么在数据导入的时候可以设置单分片导入模式（将 `load_to_single_tablet` 设置为 true），那么在大数据量的导入的时候，一个任务在将数据写入对应的分区时将只写入一个分片，这样将能提高数据导入的并发度和吞吐量，减少数据导入和 Compaction 导致的写放大问题，保障集群的稳定性。

   e. 维度表：缓慢增长的，可以使用单分区，在分桶策略上使用常用查询条件（这个字段数据分布相对均衡）分桶。

   f. 事实表

5. 对于有大量历史分区数据，但是历史数据比较少，或者不均衡，或者查询概率的情况，使用如下方式将数据放在特殊分区。

   对于历史数据，如果数据量比较小我们可以创建历史分区（比如年分区，月分区），将所有历史数据放到对应分区里创建历史分区方式例如：`FROM ("2000-01-01")TO ("2022-01-01")INTERVAL 1 YEAR`，具体参考：

```
(
    PARTITION p00010101_1899 VALUES [('0001-01-01'), ('1900-01-01')),

    PARTITION p19000101 VALUES [('1900-01-01'), ('1900-01-02')),

    ...

    PARTITION p19000104_1999 VALUES [('1900-01-04'), ('2000-01-01')),
```

```
    FROM ("2000-01-01") TO ("2022-01-01") INTERVAL 1 YEAR,
    PARTITION p30001231 VALUES [('3000-12-31'), ('3001-01-01')),
    PARTITION p99991231 VALUES [('9999-12-31'), (MAXVALUE))
)
```

2.9 数据导入

2.9.1 导入概览

Apache Doris 提供了多种导入和集成数据的方法，您可以使用合适的导入方式从各种源将数据导入到数据库中。Apache Doris 提供的数据导入方式可以分为四类：

- 实时写入：应用程序通过 HTTP 或者 JDBC 实时写入数据到 Doris 表中，适用于需要实时分析和查询的场景。
  - 极少量数据（5 分钟一次）时可以使用 JDBC INSERT 写入数据。
  - 并发较高或者频次较高（大于 20 并发或者 1 分钟写入多次）时建议打开 Group Commit，使用 JDBC INSERT 或者 Stream Load 写入数据。
  - 吞吐较高时推荐使用 Stream Load 通过 HTTP 写入数据。
- 流式同步：通过实时数据流（如 Flink、Kafka、事务数据库）将数据实时导入到 Doris 表中，适用于需要实时分析和查询的场景。
  - 可以使用Flink Doris Connector 将 Flink 的实时数据流写入到 Doris 表中。
  - 可以使用 Routine Load 或者Doris Kafka Connector 将 Kafka 的实时数据流写入到 Doris 表中。Routine Load 方式下，Doris 会调度任务将 Kafka 中的数据拉取并写入 Doris 中，目前支持 csv 和 json 格式的数据。Kafka Connector 方式下，由 Kafka 将数据写入到 Doris 中，支持 avro、json、csv、protobuf 格式的数据。
  - 可以使用Flink CDC 或Datax 将事务数据库的 CDC 数据流写入到 Doris 中。
- 批量导入：将数据从外部存储系统（如对象存储、HDFS、本地文件、NAS）批量加载到 Doris 表中，适用于非实时数据导入的需求。
  - 可以使用 Broker Load 将对象存储和 HDFS 中的文件写入到 Doris 中。
  - 可以使用 INSERT INTO SELECT 将对象存储、HDFS 和 NAS 中的文件同步写入到 Doris 中，配合JOB 可以异步写入。
  - 可以使用 Stream Load 或者Doris Streamloader 将本地文件写入 Doris 中。
- 外部数据源集成：通过与外部数据源（如 Hive、JDBC、Iceberg 等）的集成，实现对外部数据的查询和部分数据导入到 Doris 表中。

- 可以创建Catalog 读取外部数据源中的数据，使用 INSERT INTO SELECT 将外部数据源中的数据同步写入到 Doris 中，配合JOB 可以异步写入。Doris 的每个导入默认都是一个隐式事务，事务相关的更多信息请参考事务。

2.9.1.1 导入方式快速浏览

Doris 的导入主要涉及数据源、数据格式、导入方式、错误数据处理、数据转换、事务多个方面。您可以在如下表格中快速浏览各导入方式适合的场景和支持的文件格式。

导入方式	使用场景	支持的文件格式	导入模式
Stream Load	导入本地文件或者应用程序写入	csv、json、parquet、orc	同步
Broker Load	从对象存储、HDFS 等导入	csv、json、parquet、orc	异步
INSERT INTO VALUES	通过 JDBC 等接口导入	SQL	同步
INSERT INTO SELECT	可以导入外部表或者对象存储、HDFS 中的文件	SQL	同步
Routine Load	从 kakfa 实时导入	csv、json	异步
MySQL Load	从本地数据导入	csv	同步
Group Commit	高频小批量导入	根据使用的导入方式而定	-

2.9.2 数据源

2.9.2.1 本地文件

Doris 提供多种方式从本地数据导入：

- Stream Load

Stream Load 是通过 HTTP 协议将本地文件或数据流导入到 Doris 中。Stream Load 是一个同步导入方式，执行导入后返回导入结果，可以通过请求的返回判断导入是否成功。支持导入 CSV、JSON、Parquet 与 ORC 格式的数据。更多文档参考 stream load。

- streamloader

Streamloader 工具是一款用于将数据导入 Doris 数据库的专用客户端工具，底层基于 Stream Load 实现，可以提供多文件，多并发导入的功能，降低大数据量导入的耗时。更多文档参考Streamloader。

- MySQL Load

Doris 兼容 MySQL 协议，可以使用 MySQL 标准的 LOAD DATA 语法导入本地文件。MySQL Load 是一种同步导入方式，执行导入后即返回导入结果，主要适用于导入客户端本地 CSV 文件。更多文档参考 MySQL Load。

2.9.2.1.1 使用 Stream Load 导入

第 1 步：准备数据

创建 CSV 文件 `streamload_example.csv`，内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在库中创建表

在 Doris 中创建表，语法如下：

```
CREATE TABLE testdb.test_streamload(
    user_id            BIGINT       NOT NULL COMMENT "用户 ID",
    name               VARCHAR(20)           COMMENT "用户姓名",
    age                INT                   COMMENT "用户年龄"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 Stream Load 导入数据

使用 curl 提交 Stream Load 导入作业：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

Stream Load 是一种同步导入方式，导入结果会直接返回给用户。

```
{
    "TxnId": 3,
    "Label": "123",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 10,
    "NumberLoadedRows": 10,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 118,
```

```
    "LoadTimeMs": 173,
    "BeginTxnTimeMs": 1,
    "StreamLoadPutTimeMs": 70,
    "ReadDataTimeMs": 2,
    "WriteDataTimeMs": 48,
    "CommitAndPublishTimeMs": 52
}
```

第 4 步：检查导入数据

```
select count(*) from testdb.test_streamload;
+----------+
| count(*) |
+----------+
|       10 |
+----------+
```

2.9.2.1.2 使用 Streamloader 工具导入

第 1 步：准备数据

创建 csv 文件 streamloader_example.csv 文件。具体内容如下

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在库中创建表

在 Doris 中创建被导入的表，具体语法如下：

```
CREATE TABLE testdb.test_streamloader(
    user_id            BIGINT       NOT NULL COMMENT "用户 ID",
    name               VARCHAR(20)           COMMENT "用户姓名",
    age                INT                   COMMENT "用户年龄"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 stream loader 工具导入数据

```
doris-streamloader --source_file="streamloader_example.csv" --url="http://localhost:8330" --
    ↪ header="column_separator:," --db="testdb" --table="test_streamloader"
```

这是一种同步导入方式，导入结果会直接返回给用户：

```
Load Result: {
        "Status": "Success",
        "TotalRows": 10,
        "FailLoadRows": 0,
        "LoadedRows": 10,
        "FilteredRows": 0,
        "UnselectedRows": 0,
        "LoadBytes": 118,
        "LoadTimeMs": 623,
        "LoadFiles": [
                "streamloader_example.csv"
        ]
}
```

第 4 步：检查导入数据

```
select count(*) from testdb.test_streamloader;
+----------+
| count(*) |
+----------+
|       10 |
+----------+
```

2.9.2.1.3 使用 MySQL Load 从本地数据导入

第 1 步：准备数据

创建名为 client_local.csv 的文件，样例数据如下：

```
1,10
2,20
3,30
4,40
5,50
6,60
```

第 2 步：在库中创建表

在执行 LOAD DATA 命令前，需要先链接 mysql 客户端。

```
mysql --local-infile  -h <fe_ip> -P <fe_query_port> -u root -D testdb
```

执行 MySQL Load，在连接时需要使用指定参数选项：

1. 在链接 mysql 客户端时，必须使用 `--local-infile` 选项，否则可能会报错。
2. 通过 JDBC 链接，需要在 URL 中指定配置 `allowLoadLocalInfile=true`

在 Doris 中创建以下表：

```
CREATE TABLE testdb.t1 (
    pk      INT,
    v1      INT SUM
) AGGREGATE KEY (pk)
DISTRIBUTED BY hash (pk);
```

第 3 步：使用 Mysql Load 导入数据

链接 MySQL Client 后，创建导入作业，命令如下：

```
LOAD DATA LOCAL
INFILE 'client_local.csv'
INTO TABLE testdb.t1
COLUMNS TERMINATED BY ','
LINES TERMINATED BY '`\n`';
```

第 4 步：检查导入数据

MySQL Load 是一种同步的导入方式，导入后结果会在命令行中返回给用户。如果导入执行失败，会展示具体的报错信息。

如下是导入成功的结果显示，会返回导入的行数：

```
Query OK, 6 row affected (0.17 sec)
Records: 6  Deleted: 0  Skipped: 0  Warnings: 0
```

2.9.2.2 Kafka

Doris 提供以下方式从 Kafka 导入数据：

- 使用 Routine Load 消费 Kafka 数据

Doris 通过 Routine Load 持续消费 Kafka Topic 中的数据。提交 Routine Load 作业后，Doris 会实时生成导入任务，消费 Kafka 集群中指定 Topic 的消息。Routine Load 支持 CSV 和 JSON 格式，具备 Exactly-Once 语义，确保数据不丢失且不重复。更多信息请参考 Routine Load。

- Doris Kafka Connector 消费 Kafka 数据

Doris Kafka Connector 是将 Kafka 数据流导入 Doris 数据库的工具。用户可通过 Kafka Connect 插件轻松导入多种序列化格式（如 JSON、Avro、Protobuf），并支持解析 Debezium 组件的数据格式。更多信息请参考Doris Kafka Connector。

在大多数情况下，可以直接选择 Routine Load 进行数据导入，无需集成外部组件即可消费 Kafka 数据。当需要加载 Avro、 Protobuf 格式的数据，或通过 Debezium 采集的上游数据库数据时，可以使用 Doris Kafka Connector。

2.9.2.2.1 使用 Routine Load 消费 Kafka 数据

使用限制

1. 支持的消息格式为 CSV 和 JSON。CSV 每个消息为一行，且行尾不包含换行符；
2. 默认支持 Kafka 0.10.0.0 及以上版本。若需使用旧版本（如 0.9.0，0.8.2，0.8.1，0.8.0），需修改 BE 配置，将 kafka_broker_version_fallback 设置为兼容的旧版本，或在创建 Routine Load 时设置 property.broker.version.fallback。使用旧版本可能导致部分新特性无法使用，如根据时间设置 Kafka 分区的 offset。

操作示例

在 Doris 中通过 CREATE ROUTINE LOAD 命令创建常驻 Routine Load 导入任务，分为单表导入和多表导入。详细语法请参考CREATE ROUTINE LOAD。

单表导入

第 1 步：准备数据

在 Kafka 中，样本数据如下：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-routine-load-csv --from-beginning
1,Emily,25
```

第 2 步：在库中创建表

在 Doris 中创建被导入的表，具体语法如下：

```
CREATE TABLE testdb.test_routineload_tbl(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：创建 Routine Load job 导入数据至单表

在 Doris 中，使用 CREATE ROUTINE LOAD 命令创建导入作业：

```
CREATE ROUTINE LOAD testdb.example_routine_load_csv ON test_routineload_tbl
COLUMNS TERMINATED BY ",",
COLUMNS(user_id, name, age)
FROM KAFKA(
    "kafka_broker_list" = "192.168.88.62:9092",
    "kafka_topic" = "test-routine-load-csv",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

第 4 步：检查导入数据

```
select * from test_routineload_tbl;
+-----------+----------------+------+
| user_id   | name           | age  |
+-----------+----------------+------+
|  1        | Emily          | 25   |
+-----------+----------------+------+
```

多表导入

对于需要同时导入多张表的场景，Kafka 中的数据必须包含表名信息，格式为：table_name|data。例如，导入 CSV 数据时，格式应为：table_name|val1,val2,val3。请注意，表名必须与 Doris 中的表名完全一致，否则导入将失败，并且不支持后面介绍的 column_mapping 配置。

第 1 步：准备数据

在 Kafka 中，样本数据如下：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-multi-table-load --from-
    ↪ beginning
test_multi_table_load1|1,Emily,25
test_multi_table_load2|2,Benjamin,35
```

第 2 步：在库中创建表

在 Doris 中创建被导入的表，具体语法如下：

表 1：

```
CREATE TABLE test_multi_table_load1(
    user_id            BIGINT       NOT NULL COMMENT "用户 ID",
    name               VARCHAR(20)           COMMENT "用户姓名",
    age                INT                   COMMENT "用户年龄"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

表 2：

```
CREATE TABLE test_multi_table_load2(
    user_id            BIGINT       NOT NULL COMMENT "用户 ID",
    name               VARCHAR(20)           COMMENT "用户姓名",
    age                INT                   COMMENT "用户年龄"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：创建 Routine Load job 导入数据至多表

在 Doris 中，使用 CREATE ROUTINE LOAD 命令创建导入作业：

```
CREATE ROUTINE LOAD example_multi_table_load
COLUMNS TERMINATED BY ","
FROM KAFKA(
    "kafka_broker_list" = "192.168.88.62:9092",
    "kafka_topic" = "test-multi-table-load",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

第 4 步：检查导入数据

```
select * from test_multi_table_load1;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|  1   | Emily          | 25   |
+------+----------------+------+

select * from test_multi_table_load2;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|  2   | Benjamin       | 35   |
+------+----------------+------+
```

配置安全认证

有关带有认证的 Kafka 配置方法，请参见 Kafka 安全认证。

2.9.2.2.2 使用 Doris Kafka Connector 消费 Kafka 数据

Doris Kafka Connector 是将 Kafka 数据流导入 Doris 数据库的工具。用户可通过 Kafka Connect 插件轻松导入多种序列化格式（如 JSON、Avro、Protobuf），并支持解析 Debezium 组件的数据格式。

以 Distributed 模式启动

Distributed 模式为 Kafka Connect 提供可扩展性和自动容错功能。在此模式下，可以使用相同的 `group.id` 启动多个工作进程，它们会协调在所有可用工作进程中安排连接器和任务的执行。

1. 在 `$KAFKA_HOME` 下创建 plugins 目录，将下载好的 doris-kafka-connector jar 包放入其中。
2. 配置 `config/connect-distributed.properties`：

```
bootstrap.servers=127.0.0.1:9092

#### 修改 group.id，同一集群的需要一致
group.id=connect-cluster
```

```
#### 修改为创建的 plugins 目录
#### 注意：此处请填写 Kafka 的直接路径。例如：plugin.path=/opt/kafka/plugins
plugin.path=$KAFKA_HOME/plugins

#### 建议将 Kafka 的 max.poll.interval.ms 时间调大到 30 分钟以上，默认 5 分钟
#### 避免 Stream Load 导入数据消费超时，消费者被踢出消费群组
max.poll.interval.ms=1800000
consumer.max.poll.interval.ms=1800000
```

3. 启动：

```
$KAFKA_HOME/bin/connect-distributed.sh -daemon $KAFKA_HOME/config/connect-distributed.properties
```

4. 消费 Kafka 数据：

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"test-doris-sink-cluster",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"topic_test",
    "doris.topic2table.map": "topic_test:test_kafka_tbl",
    "buffer.count.records":"10000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"5000000",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test_db",
    "key.converter":"org.apache.kafka.connect.storage.StringConverter",
    "value.converter":"org.apache.kafka.connect.storage.StringConverter"
  }
}'
```

操作 Kafka Connect

```
#### 查看 connector 状态
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/status -X GET
#### 删除当前 connector
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster -X DELETE
#### 暂停当前 connector
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/pause -X PUT
#### 重启当前 connector
```

```
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/resume -X PUT
#### 重启 connector 内的 tasks
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/tasks/0/restart -X POST
```

关于 Distributed 模式的介绍请参见 Distributed Workers。

消费普通数据

1. 导入数据样本：

在 Kafka 中，样本数据如下：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-data-topic --from-
    ↪ beginning
{"user_id":1,"name":"Emily","age":25}
{"user_id":2,"name":"Benjamin","age":35}
{"user_id":3,"name":"Olivia","age":28}
{"user_id":4,"name":"Alexander","age":60}
{"user_id":5,"name":"Ava","age":17}
{"user_id":6,"name":"William","age":69}
{"user_id":7,"name":"Sophia","age":32}
{"user_id":8,"name":"James","age":64}
{"user_id":9,"name":"Emma","age":37}
{"user_id":10,"name":"Liam","age":64}
```

2. 创建需要导入的表：

在 Doris 中创建被导入的表，具体语法如下：

```
CREATE TABLE test_db.test_kafka_connector_tbl(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 12;
```

3. 创建导入任务：

在部署 Kafka Connect 的机器上，通过 curl 命令提交如下导入任务：

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"test-doris-sink-cluster",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "tasks.max":"10",
```

```
    "topics":"test-data-topic",
    "doris.topic2table.map": "test-data-topic:test_kafka_connector_tbl",
    "buffer.count.records":"10000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"5000000",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test_db",
    "key.converter":"org.apache.kafka.connect.storage.StringConverter",
    "value.converter":"org.apache.kafka.connect.storage.StringConverter"
  }
}'
```

消费 Debezium 组件采集的数据

1. MySQL 数据库中有如下表：

```
CREATE TABLE test.test_user (
  user_id int NOT NULL ,
  name varchar(20),
  age int,
  PRIMARY KEY (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

insert into test.test_user values(1,'zhangsan',20);
insert into test.test_user values(2,'lisi',21);
insert into test.test_user values(3,'wangwu',22);
```

2. 在 Doris 创建被导入的表：

```
CREATE TABLE test_db.test_user(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
UNIQUE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 12;
```

3. 部署 Debezium connector for MySQL 组件，参考：Debezium connector for MySQL。

4. 创建 doris-kafka-connector 导入任务：

假设通过 Debezium 采集到的 MySQL 表数据在 `mysql_debezium.test.test_user` Topic 中：

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"test-debezium-doris-sink",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "tasks.max":"10",
    "topics":"mysql_debezium.test.test_user",
    "doris.topic2table.map": "mysql_debezium.test.test_user:test_user",
    "buffer.count.records":"10000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"5000000",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test_db",
    "converter.mode":"debezium_ingestion",
    "enable.delete":"true",
    "key.converter":"org.apache.kafka.connect.json.JsonConverter",
    "value.converter":"org.apache.kafka.connect.json.JsonConverter"
  }
}'
```

消费 AVRO 序列化格式数据

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"doris-avro-test",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"avro_topic",
    "tasks.max":"10",
    "doris.topic2table.map": "avro_topic:avro_tab",
    "buffer.count.records":"100000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"10000000",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test",
    "load.model":"stream_load",
    "key.converter":"io.confluent.connect.avro.AvroConverter",
    "key.converter.schema.registry.url":"http://127.0.0.1:8081",
```

```
    "value.converter":"io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url":"http://127.0.0.1:8081"
  }
}'
```

消费 Protobuf 序列化格式数据

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"doris-protobuf-test",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"proto_topic",
    "tasks.max":"10",
    "doris.topic2table.map": "proto_topic:proto_tab",
    "buffer.count.records":"100000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"10000000",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test",
    "load.model":"stream_load",
    "key.converter":"io.confluent.connect.protobuf.ProtobufConverter",
    "key.converter.schema.registry.url":"http://127.0.0.1:8081",
    "value.converter":"io.confluent.connect.protobuf.ProtobufConverter",
    "value.converter.schema.registry.url":"http://127.0.0.1:8081"
  }
}'
```

2.9.2.3 Flink

使用 Flink Doris Connector 可以实时的将 Flink 产生的数据（如：Flink 读取 Kafka，MySQL 中的数据）导入到 Doris 中。

2.9.2.3.1 使用限制

需要依赖用户部署的 Flink 集群。

2.9.2.3.2 使用 Flink 导入数据

使用 Flink 导入数据，详细步骤可以参考Flink-Doris-Connector。在以下步骤中，演示如何通过 Flink 快速导入数据。

第 1 步：创建表

```
CREATE TABLE `students` (
  `id` INT NULL,
```

```
  `name` VARCHAR(256) NULL,
  `age` INT NULL
) ENGINE=OLAP
UNIQUE KEY(`id`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

第 2 步：使用 Flink 导入数据

运行 bin/sql-client.sh 打开 FlinkSQL 的控制台

```
CREATE TABLE student_sink (
    id INT,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '10.16.10.6:28737',
      'table.identifier' = 'test.students',
      'username' = 'root',
      'password' = '',
      'sink.label-prefix' = 'doris_label'
);

INSERT INTO student_sink values(1,'zhangsan',123)
```

第 3 步：检查导入数据

```
select * from test.students;
+------+----------+------+
| id   | name     | age  |
+------+----------+------+
|  1   | zhangsan |  123 |
+------+----------+------+
```

2.9.2.4 HDFS

Doris 提供两种方式从 HDFS 导入文件：- 使用 HDFS Load 将 HDFS 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 HDFS 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.4.1 使用 HDFS Load 导入

使用 HDFS Load 导入 HDFS 上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 hdfsload_example.csv 文件存储在 HDFS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_hdfsload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 HDFS Load 导入数据

```
LOAD LABEL hdfs_load_2022_04_01
(
    DATA INFILE("hdfs://127.0.0.1:8020/tmp/hdfsload_example.csv")
    INTO TABLE test_hdfsload
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
with HDFS
(
    "fs.defaultFS" = "hdfs://127.0.0.1:8020",
    "hadoop.username" = "user"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_hdfsload;
```

结果：

```
mysql> select * from test_hdfsload;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.4.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 hdfsload_example.csv 文件存储在 HDFS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_hdfsload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_hdfsload
SELECT * FROM hdfs (
    "uri" = "hdfs://127.0.0.1:8020/tmp/hdfsload_example.csv",
    "fs.defaultFS" = "hdfs://127.0.0.1:8020",
    "hadoop.username" = "doris",
    "format" = "csv",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_hdfsload;
```

结果：

```
mysql> select * from test_hdfsload;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.5 Amazon S3

Doris 提供两种方式从 AWS S3 导入文件：- 使用 S3 Load 将 S3 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 S3 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.5.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 S3 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
```

```
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "s3.us-west-2.amazonaws.com",
    "s3.region" = "us-west-2",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.5.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 S3 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "s3.endpoint" = "s3.us-west-2.amazonaws.com",
    "s3.region" = "us-west-2",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

Doris 也支持AWS Assume Role的方式使用 S3 Load 和 TVF 请参考AWS 集成.

2.9.2.6 Google Cloud Storage

Doris 提供两种方式从 Google Cloud Storage 导入文件：- 使用 S3 Load 将 Google Cloud Storage 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 Google Cloud Storage 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.6.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 Google Cloud Storage 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "GCP",
    "s3.endpoint" = "storage.us-west2.rep.googleapis.com",
    "s3.region" = "US-WEST2",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.6.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 Google Cloud Storage 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
```

```
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "GCP",
    "s3.endpoint" = "storage.us-west2.rep.googleapis.com",
    "s3.region" = "US-WEST2",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.7 Azure Storage

Doris 提供两种方式从 Azure Storage 导入文件：- 使用 S3 Load 将 Azure Storage 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 Azure Storage 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.7.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 Azure Storage 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

> Caution Azure Storage 默认要求 HTTPS 传输，对应的存储账户配置是 需要安全传输：已启用。必须在 Doris be.conf 中设置 `s3_client_http_scheme = https` 才能正常访问 Azure Storage。
>
> Azure S3 properties 中的 `s3.region` 可以省略。

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
```

```
    "provider" = "AZURE",
    "s3.endpoint" = "StorageAccountA.blob.core.windows.net",
    "s3.region" = "westus3",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.7.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 Azure Storage 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
```

```
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

> Caution Azure Storage 默认要求 HTTPs 传输，对应的存储账户配置是 需要安全传输：已启用。必须在 Doris be.conf 中设置 s3_client_http_scheme = https 才能正常访问 Azure Storage。
>
> Azure S3 properties 中的 s3.region 可以省略。

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "AZURE",
    "s3.endpoint" = "StorageAccountA.blob.core.windows.net",
    "s3.region" = "westus3",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
```

```
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.8 阿里云 OSS

Doris 提供两种方式从阿里云 OSS 导入文件：- 使用 S3 Load 将阿里云 OSS 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将阿里云 OSS 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.8.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在阿里云 OSS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

> 注意阿里云 OSS 公网和内网的 endpoint 是不同的。如果服务器与 OSS 在同一个 region 下，建议使用内网的 endpoint 链接。
>
> - 内网 endpoint：oss-cn-hangzhou-internal.aliyuncs.com
> - 公网 endpoint：oss-cn-hangzhou.aliyuncs.com

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "OSS",
    "s3.endpoint" = "oss-cn-hangzhou.aliyuncs.com",
    "s3.region" = "oss-cn-hangzhou",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
```

```
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.8.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在阿里云 OSS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

> 注意阿里云 OSS 公网和内网的 endpoint 是不同的。如果服务器与 OSS 在同一个 region 下，建议使用内网的 endpoint 链接。
>
> - 内网 endpoint：`oss-cn-hangzhou-internal.aliyuncs.com`
> - 公网 endpoint：`oss-cn-hangzhou.aliyuncs.com`

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "OSS",
    "s3.endpoint" = "oss-cn-hangzhou.aliyuncs.com",
    "s3.region" = "oss-cn-hangzhou",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.9 华为云 OBS

Doris 提供两种方式从华为云 OBS 导入文件：- 使用 S3 Load 将华为云 OBS 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将华为云 OBS 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.9.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在华为云 OBS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "OBS",
    "s3.endpoint" = "obs.cn-north-1.myhuaweicloud.com",
    "s3.region" = "cn-north-1",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.9.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在华为云 OBS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
```

```
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "OBS",
    "s3.endpoint" = "obs.cn-north-1.myhuaweicloud.com",
    "s3.region" = "cn-north-1",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.10 腾讯云 COS

Doris 提供两种方式从腾讯云 COS 导入文件：- 使用 S3 Load 将腾讯云 COS 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将腾讯云 COS 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.10.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在腾讯云 COS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "COS",
    "s3.endpoint" = "cos.ap-beijing.myqcloud.com",
    "s3.region" = "ap-beijing",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>"
)
PROPERTIES
(
```

```
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.10.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在腾讯云 COS 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
```

```
    name                VARCHAR(20)           COMMENT "name",
    age                 INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "COS",
    "s3.endpoint" = "cos.ap-beijing.myqcloud.com",
    "s3.region" = "ap-beijing",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.11 MinIO

Doris 提供两种方式从 MinIO 导入文件：- 使用 S3 Load 将 MinIO 文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 MinIO 文件导入到 Doris 中，这是一个同步的导入方式。

2.9.2.11.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 MinIO 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

> 注意如果您在本地网络中部署了 MinIO 并且未启用 TLS，则需要在 endpoint 字符串中明确添加 `http://`。
>
> - `"s3.endpoint" = "http://localhost:9000"`
>
> S3 SDK 默认使用 virtual-hosted style 方式。但 MinIO 默认没开启 virtual-hosted style 方式的访问，此时我们可以添加 `use_path_style` 参数来强制使用 path style 方式。
>
> - `"use_path_style" = "true"`

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "play.min.io:9000",
    "s3.region" = "us-east-1",
    "s3.access_key" = "myminioadmin",
    "s3.secret_key" = "minio-secret-key-change-me",
    "use_path_style" = "true"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.11.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 MinIO 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

> 注意如果您在本地网络中部署了 MinIO 并且未启用 TLS，则需要在 endpoint 字符串中明确添加 http://。
>
> - "s3.endpoint" = "http://localhost:9000"
>
> S3 SDK 默认使用 virtual-hosted style 方式。但 MinIO 默认没开启 virtual-hosted style 方式的访问，此时我们可以添加 use_path_style 参数来强制使用 path style 方式。
>
> - "use_path_style" = "true"

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "S3",
    "s3.endpoint" = "play.min.io:9000",
```

```
    "s3.region" = "us-east-1",
    "s3.access_key" = "myminioadmin",
    "s3.secret_key" = "minio-secret-key-change-me",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int",
    "use_path_style" = "true"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.12 S3 兼容存储

Doris 提供两种方式从 S3 兼容存储导入文件：- 使用 S3 Load 将 S3 兼容存储文件导入到 Doris 中，这是一个异步的导入方式。- 使用 TVF 将 S3 兼容存储文件导入到 Doris 中，这是一个同步的导入方式。

> 注意 S3 SDK 默认使用 virtual-hosted style 方式。但某些对象存储系统可能没开启或没支持 virtual-hosted style 方式的访问，此时我们可以添加 `use_path_style` 参数来强制使用 path style 方式。

2.9.2.12.1 使用 S3 Load 导入

使用 S3 Load 导入对象存储上的文件，详细步骤可以参考 Broker Load 手册

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 S3 兼容存储上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 S3 Load 导入数据

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "play.min.io:9000",
    "s3.region" = "us-east-1",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "use_path_style" = "true"
)
PROPERTIES
(
    "timeout" = "3600"
```

```
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.12.2 使用 TVF 导入

第 1 步：准备数据

创建 CSV 文件 s3load_example.csv 文件存储在 S3 兼容存储上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

第 2 步：在 Doris 中创建表

```
CREATE TABLE test_s3load(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
```

```
    age                 INT                     COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

第 3 步：使用 TVF 导入数据

```
INSERT INTO test_s3load
SELECT * FROM S3
(
    "uri" = "s3://your_bucket_name/s3load_example.csv",
    "format" = "csv",
    "provider" = "S3",
    "s3.endpoint" = "play.min.io:9000",
    "s3.region" = "us-east-1",
    "s3.access_key" = "<your-ak>",
    "s3.secret_key" = "<your-sk>",
    "column_separator" = ",",
    "csv_schema" = "user_id:int;name:string;age:int",
    "use_path_style" = "true"
);
```

第 4 步：检查导入数据

```
SELECT * FROM test_s3load;
```

结果：

```
mysql> select * from test_s3load;
+---------+-----------+------+
| user_id | name      | age  |
+---------+-----------+------+
|       5 | Ava       |   17 |
|      10 | Liam      |   64 |
|       7 | Sophia    |   32 |
|       9 | Emma      |   37 |
|       1 | Emily     |   25 |
|       4 | Alexander |   60 |
|       2 | Benjamin  |   35 |
|       3 | Olivia    |   28 |
|       6 | William   |   69 |
|       8 | James     |   64 |
+---------+-----------+------+
10 rows in set (0.04 sec)
```

2.9.2.13 Snowflake

在迁移 Snowflake 的过程中，通常需要借助对象存储作为中间媒介。核心流程如下：首先通过 Snowflake 的 COPY INTO 语句将数据导出到对象存储；再利用 Doris 的 S3 Load 功能从对象存储中读取数据并导入到 Doris 中，具体可参考 S3 导入。

2.9.2.13.1 注意事项

在迁移之前，需要根据 Snowflake 的表结构选择 Doris 的数据模型，以及分区和分桶的策略，更多创建表策略可参考导入最佳实践。

2.9.2.13.2 数据类型映射

Snowflake	Doris	备注
NUMBER(p, s)/DECIMAL(p, s)/NUMERIC(p,s)	DECIMAL(p, s)	
INT/INTEGER	INT	
TINYINT/BYTEINT	TINYINT	
SMALLINT	SMALLINT	
BIGINT	BIGINT	
FLOAT/FLOAT4/FLOAT8/DOUBLE/DOUBLE PRECISION/REAL	DOUBLE	
VARCHAR/STRING/TEXT	VARCHAR/STRING	VARCHAR 长度最大 65535
CHAR/CHARACTER/NCHAR	CHAR	
BINARY/VARBINARY	STRING	
BOOLEAN	BOOLEAN	
DATE	DATE	
DATETIME/TIMESTAMP/TIMESTAMP_NTZ	DATETIME	TIMESTAMP 是用户可配置的别名，默认为 TIMESTAMP_NTZ
TIME	STRING	Snowflake 导出时需要 Cast 成 String 类型
VARIANT	VARIANT	
ARRAY	ARRAY	
OBJECT	JSON	
GEOGRAPHY/GEOMETRY	STRING	

2.9.2.13.3 1. 创建表

在迁移 Snowflake 表到 Doris 中的时候，需要先创建 Doris 表。

假设我们在 Snowflake 中已存在如下表和数据

```
CREATE OR REPLACE TABLE sales_data (
    order_id      INT PRIMARY KEY,
    customer_name VARCHAR(128),
    order_date    DATE,
    amount        DECIMAL(10,2),
    country       VARCHAR(48)
)
CLUSTER BY (order_date);
```

```
INSERT INTO sales_data VALUES
(1, 'Alice', '2025-04-08', 99.99, 'USA'),
(2, 'Bob', '2025-04-08', 149.50, 'Canada'),
(3, 'Charlie', '2025-04-09', 75.00, 'UK'),
(4, 'Diana', '2025-04-10', 200.00, 'Australia');
```

根据这个表结构，可以创建 Doris 主键分区表，分区字段和 Snowflake 的 Clustering Key 一致，同时按天分区

```
CREATE TABLE `sales_data` (
  order_id      INT,
  order_date    DATE NOT NULL,
  customer_name VARCHAR(128),
  amount        DECIMAL(10,2),
  country       VARCHAR(48)
) ENGINE=OLAP
UNIQUE KEY(`order_id`,`order_date`)
PARTITION BY RANGE(`order_date`) (
PARTITION p20250408 VALUES [('2025-04-08'), ('2025-04-09')),
PARTITION p20250409 VALUES [('2025-04-09'), ('2025-04-10')),
PARTITION p20250410 VALUES [('2025-04-10'), ('2025-04-11'))
)
DISTRIBUTED BY HASH(`order_id`) BUCKETS 16
PROPERTIES (
 "dynamic_partition.enable" = "true",
 "dynamic_partition.time_unit" = "DAY",
 "dynamic_partition.end" = "5",
 "dynamic_partition.prefix" = "p",
 "dynamic_partition.buckets" = "16",
 "replication_num" = "1"
);
```

2.9.2.13.4 2. 导出 Snowflake 数据

2.1. 通过 COPY INFO 方式导出到 S3 Parquet 格式的文件

Snowflake 支持导出到 AWS S3，GCS，AZURE，导出时，建议按照 Doris 的分区字段进行导出。以下为导出到 AWS S3 的示例：

```
   CREATE FILE FORMAT my_parquet_format TYPE = parquet;
   CREATE OR REPLACE STAGE external_stage
   URL='s3://mybucket/sales_data'
   CREDENTIALS=(AWS_KEY_ID='<ak>' AWS_SECRET_KEY='<sk>')
   FILE_FORMAT = my_parquet_format;
   COPY INTO @external_stage from sales_data PARTITION BY (CAST(order_date AS VARCHAR)) header=
       ↪ true;
```

2.2. 查看 S3 上的导出文件

导出后，在 S3 上会按照分区划分成具体的子目录，每一个目录是对应的如下

2.9.2.13.5 3. 导入数据到 Doris

导入使用 S3 Load 进行导入，S3 Load 是一种异步的数据导入方式，执行后 Doris 会主动从数据源拉取数据，数据源支持兼容 S3 协议的对象存储，包括 (AWS S3，GCS，AZURE 等)。

该方式适用于数据量大、需要后台异步处理的场景。对于需要同步处理的数据导入，可以参考 TVF 导入。

注意：对于含有复杂类型（Struct/Array/Map）的 Parquet/ORC 格式文件导入，目前必须使用 TVF 导入

3.1. 导入一个分区的数据

```
LOAD LABEL sales_data_2025_04_08
(
    DATA INFILE("s3://mybucket/sales_data/2025_04_08/*")
    INTO TABLE sales_data
    FORMAT AS "parquet"
    (order_id, order_date, customer_name, amount, country)
)
```

```
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "s3.ap-southeast-1.amazonaws.com",
    "s3.access_key" = "<ak>",
    "s3.secret_key" = "<sk>",
    "s3.region" = "ap-southeast-1"
);
```

3.2. 通过 Show Load 查看任务运行情况

由于 S3Load 导入是异步提交的，所以需要通过 show load 可以查看指定 label 的导入情况：

```
mysql> show load where label = "label_sales_data_2025_04_08"`\G`
*************************** 1. row ***************************
        JobId: 17956078
        Label: label_sales_data_2025_04_08
        State: FINISHED
    Progress: 100.00% (1/1)
        Type: BROKER
    EtlInfo: unselected.rows=0; dpp.abnorm.ALL=0; dpp.norm.ALL=2
    TaskInfo: cluster:s3.ap-southeast-1.amazonaws.com; timeout(s):3600; max_filter_ratio:0.0;
        ↪ priority:NORMAL
    ErrorMsg: NULL
    CreateTime: 2025-04-10 17:50:53
EtlStartTime: 2025-04-10 17:50:54
EtlFinishTime: 2025-04-10 17:50:54
LoadStartTime: 2025-04-10 17:50:54
LoadFinishTime: 2025-04-10 17:50:54
        URL: NULL
    JobDetails: {"Unfinished backends":{"5eec1be8612d4872-91040ff1e7208a4f":[]},"ScannedRows
        ↪ ":2,"TaskNumber":1,"LoadBytes":91,"All   backends":{"5eec1be8612d4872-91040
        ↪ ff1e7208a4f":[10022]},"FileNumber":1,"FileSize":1620}
TransactionId: 766228
ErrorTablets: {}
        User: root
    Comment:
1 row in set (0.00 sec)
```

3.3. 处理导入过程中的错误

当有多个导入任务时，可以通过以下语句，查询数据导入失败的日期和原因。

```
mysql> show load where state='CANCELLED' and label like "label_test%"`\G`
*************************** 1. row ***************************
        JobId: 18312384
        Label: label_test123
```

```
         State: CANCELLED
      Progress: 100.00% (3/3)
          Type: BROKER
      EtlInfo: unselected.rows=0; dpp.abnorm.ALL=4; dpp.norm.ALL=0
      TaskInfo: cluster:s3.ap-southeast-1.amazonaws.com; timeout(s):14400; max_filter_ratio:0.0;
          ↪  priority:NORMAL
      ErrorMsg: type:ETL_QUALITY_UNSATISFIED; msg:quality not good enough to cancel
      CreateTime: 2025-04-15 17:32:59
    EtlStartTime: 2025-04-15 17:33:02
    EtlFinishTime: 2025-04-15 17:33:02
    LoadStartTime: 2025-04-15 17:33:02
    LoadFinishTime: 2025-04-15 17:33:02
          URL: http://10.16.10.6:28747/api/_load_error_log?file=__shard_2   error_log_insert_
              ↪ stmt_7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307efca7bfe342
      JobDetails: {"Unfinished backends":{"7602ccd7c3a4854-95307efca7bfe341":[]},"ScannedRows"
          ↪ :4,"TaskNumber":1,"LoadBytes":188,"All   backends":{"7602ccd7c3a4854-95307
          ↪ efca7bfe341":[10022]},"FileNumber":3,"FileSize":4839}
    TransactionId: 769213
    ErrorTablets: {}
          User: root
      Comment:
```

如上面的例子是数据质量错误 (ETL_QUALITY_UNSATISFIED)，具体错误需要通过访问返回的 URL 的链接进行查看，如下是数据超过了表中的 Schema 中 country 列的实际长度：

```
[root@VM-10-6-centos ~]$ curl "http://10.16.10.6:28747/api/_load_error_log?file=__shard_2
    ↪ error_log_insert_stmt_7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307
    ↪ efca7bfe342"
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [USA] schema length: 1; actual   length: 3; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Canada] schema length: 1; actual   length: 6; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [UK] schema length: 1; actual   length: 2; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Australia] schema length: 1;   actual length: 9; . src line [];
```

同时对于数据质量的错误，如果可以允许错误数据跳过的，可以通过在 S3 Load 任务中 Properties 设置容错率，具体可参考导入配置参数。

3.4. 导入多个分区的数据

当需要迁移大数据量的存量数据时，建议使用分批导入的策略。每批数据对应 Doris 的一个分区或少量几个分区，数据量建议不超过 100GB，以减轻系统压力并降低导入失败后的重试成本。

可参考脚本 s3_load_demo.sh，该脚本可以实现了轮询 S3 上的分区目录，同时提交 S3 Load 任务到 Doris 中，实现批量导入的效果。

2.9.2.14 BigQuery

在迁移 BigQuery 的过程中，通常需要借助对象存储作为中间媒介。核心流程如下：首先通过 BigQuery 的 Export 语句将数据导出到 GCS（Google Cloud Storage）；再利用 Doris 的 S3 Load 功能从对象存储中读取数据并导入到 Doris 中，具体可参考 S3 导入。

2.9.2.14.1 注意事项

1. 在迁移之前，需要根据 BigQuery 的表结构选择 Doris 的数据模型，以及分区和分桶的策略，更多创建表策略可参考导入最佳实践。
2. BigQuery 导出 JSON 类型时，不支持 Parquet 格式导出，可使用 JSON 格式导出。
3. BigQuery 导出 Time 类型时，需要 Cast String 类型导出。

2.9.2.14.2 数据类型映射

BigQuery	Doris	备注
Array	Array	
BOOLEAN	BOOLEAN	
DATE	DATE	
DATETIME/TIMESTAMP	DATETIME	
JSON	JSON	
INT64	BIGINT	
NUMERIC	DECIMAL	
FLOAT64	DOUBLE	
STRING	VARCHAR/STRING	VARCHAR 长度最大 65535
STRUCT	STRUCT	
TIME	STRING	
OTHER	UNSUPPORTED	

2.9.2.14.3 1. 创建表

在迁移 BigQuery 表到 Doris 中的时候，需要先创建 Doris 表。

假设我们在 BigQuery 中已存在如下表和数据

```
CREATE OR REPLACE TABLE test.sales_data (
    order_id      INT64,
    customer_name STRING,
    order_date    DATE,
    amount        NUMERIC(10,2),
    country       STRING
)
PARTITION BY  order_date
```

```
INSERT INTO test.sales_data (order_id, customer_name, order_date, amount, country) VALUES
(1, 'Alice', '2025-04-08', 99.99, 'USA'),
(2, 'Bob', '2025-04-08', 149.50, 'Canada'),
(3, 'Charlie', '2025-04-09', 75.00, 'UK'),
(4, 'Diana', '2025-04-10', 200.00, 'Australia');
```

根据这个表结构，可以创建 Doris 主键分区表，分区字段和 Bigquery 的分区字段一致，同时按天分区

```
CREATE TABLE `sales_data` (
  order_id      INT,
  order_date    DATE NOT NULL,
  customer_name VARCHAR(128),
  amount        DECIMAL(10,2),
  country       VARCHAR(48)
) ENGINE=OLAP
UNIQUE KEY(`order_id`,`order_date`)
PARTITION BY RANGE(`order_date`) (
PARTITION p20250408 VALUES [('2025-04-08'), ('2025-04-09')),
PARTITION p20250409 VALUES [('2025-04-09'), ('2025-04-10')),
PARTITION p20250410 VALUES [('2025-04-10'), ('2025-04-11'))
)
DISTRIBUTED BY HASH(`order_id`) BUCKETS 16
PROPERTIES (
 "dynamic_partition.enable" = "true",
 "dynamic_partition.time_unit" = "DAY",
 "dynamic_partition.end" = "5",
 "dynamic_partition.prefix" = "p",
 "dynamic_partition.buckets" = "16",
 "replication_num" = "1"
);
```

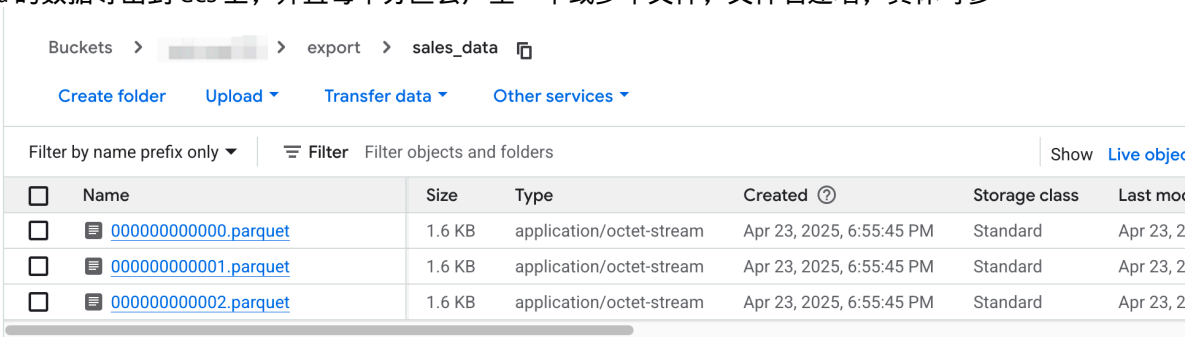
2.9.2.14.4 2. 导出 BigQuery 数据

2.1. 通过 Export 方式导出到 GCS Parquet 格式的文件

```
  EXPORT DATA
    OPTIONS (
      uri = 'gs://mybucket/export/sales_data/*.parquet',
      format = 'PARQUET')
  AS (
    SELECT *
    FROM test.sales_data
  );
```

2.2. 查看 GCS 上的导出文件

以上命令会将 sales_data 的数据导出到 GCS 上，并且每个分区会产生一个或多个文件，文件名递增，具体可参考exporting-data，如下



2.9.2.14.5 3. 导入数据到 Doris

导入使用 S3 Load 进行导入，S3 Load 是一种异步的数据导入方式，执行后 Doris 会主动从数据源拉取数据，数据源支持兼容 S3 协议的对象存储，包括 (AWS S3，GCS，AZURE 等)。

该方式适用于数据量大、需要后台异步处理的场景。对于需要同步处理的数据导入，可以参考 TVF 导入。

注意：对于含有复杂类型（Struct/Array/Map）的 Parquet/ORC 格式文件导入，目前必须使用 TVF 导入

3.1. 导入单个文件的数据

```
LOAD LABEL sales_data_2025_04_08
(
    DATA INFILE("s3://mybucket/export/sales_data/000000000000.parquet")
    INTO TABLE sales_data
    FORMAT AS "parquet"
    (order_id, order_date, customer_name, amount, country)
)
WITH S3
(
    "provider" = "GCP",
    "s3.endpoint" = "storage.asia-southeast1.rep.googleapis.com",
    "s3.region" = "asia-southeast1",
    "s3.access_key" = "<ak>",
    "s3.secret_key" = "<sk>"
);
```

3.2. 通过 Show Load 查看任务运行情况

由于 S3Load 导入是异步提交的，所以需要通过 show load 可以查看指定 label 的导入情况：

```
mysql> show load where label = "label_sales_data_2025_04_08"`\G`
*************************** 1. row ***************************
        JobId: 17956078
        Label: label_sales_data_2025_04_08
        State: FINISHED
     Progress: 100.00% (1/1)
         Type: BROKER
```

```
       EtlInfo: unselected.rows=0; dpp.abnorm.ALL=0; dpp.norm.ALL=2
       TaskInfo: cluster:storage.asia-southeast1.rep.googleapis.com; timeout(s):3600; max_
           ↪ filter_ratio:0.0; priority:NORMAL
       ErrorMsg: NULL
     CreateTime: 2025-04-10 17:50:53
   EtlStartTime: 2025-04-10 17:50:54
  EtlFinishTime: 2025-04-10 17:50:54
  LoadStartTime: 2025-04-10 17:50:54
  LoadFinishTime: 2025-04-10 17:50:54
           URL: NULL
     JobDetails: {"Unfinished backends":{"5eec1be8612d4872-91040ff1e7208a4f":[]},"ScannedRows
         ↪ ":2,"TaskNumber":1,"LoadBytes":91,"All backends":{"5eec1be8612d4872-91040
         ↪ ff1e7208a4f":[10022]},"FileNumber":1,"FileSize":1620}
  TransactionId: 766228
   ErrorTablets: {}
           User: root
       Comment:
1 row in set (0.00 sec)
```

3.3. 处理导入过程中的错误

当有多个导入任务时，可以通过以下语句，查询数据导入失败的日期和原因。

```
mysql> show load where state='CANCELLED' and label like "label_test%"`\G`
*************************** 1. row ***************************
        JobId: 18312384
        Label: label_test123
        State: CANCELLED
     Progress: 100.00% (3/3)
         Type: BROKER
     EtlInfo: unselected.rows=0; dpp.abnorm.ALL=4; dpp.norm.ALL=0
     TaskInfo: cluster:storage.asia-southeast1.rep.googleapis.com; timeout(s):14400; max_
         ↪ filter_ratio:0.0; priority:NORMAL
     ErrorMsg: type:ETL_QUALITY_UNSATISFIED; msg:quality not good enough to cancel
   CreateTime: 2025-04-15 17:32:59
 EtlStartTime: 2025-04-15 17:33:02
EtlFinishTime: 2025-04-15 17:33:02
LoadStartTime: 2025-04-15 17:33:02
LoadFinishTime: 2025-04-15 17:33:02
         URL: http://10.16.10.6:28747/api/_load_error_log?file=__shard_2/error_log_insert_
             ↪ stmt_7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307efca7bfe342
   JobDetails: {"Unfinished backends":{"7602ccd7c3a4854-95307efca7bfe341":[]},"ScannedRows
       ↪ ":4,"TaskNumber":1,"LoadBytes":188,"All backends":{"7602ccd7c3a4854-95307
       ↪ efca7bfe341":[10022]},"FileNumber":3,"FileSize":4839}
TransactionId: 769213
 ErrorTablets: {}
```

```
            User: root
        Comment:
```

如上面的例子是数据质量错误 (ETL_QUALITY_UNSATISFIED)，具体错误需要通过访问返回的 URL 的链接进行查看，如下是数据超过了表中的 Schema 中 country 列的实际度：

```
[root@VM-10-6-centos ~]$ curl "http://10.16.10.6:28747/api/_load_error_log?file=__shard_2/error
    ↪ _log_insert_stmt_7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307efca7bfe342"
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [USA] schema length: 1; actual length: 3; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Canada] schema length: 1; actual length: 6; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [UK] schema length: 1; actual length: 2; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Australia] schema length: 1; actual length: 9; . src line [];
```

同时对于数据质量的错误，如果可以允许错误数据跳过的，可以通过在 S3 Load 任务中 Properties 设置容错率，具体可参考导入配置参数。

3.4. 导入多个文件的数据

当需要迁移大数据量的存量数据时，建议使用分批导入的策略。每批数据对应 Doris 的一个分区或少量几个分区，数据量建议不超过 100GB，以减轻系统压力并降低导入失败后的重试成本。

可参考脚本 s3_load_file_demo.sh，该脚本可以对对象存储上指定目录下的文件列表进行拆分，分批提交多个 S3 Load 任务到 Doris 中，实现批量导入的效果。

2.9.2.15 Redshift

在迁移 Redshift 的过程中，通常需要借助对象存储作为中间媒介。核心流程如下：首先通过 Redshift 的 UNLOAD 语句将数据导出到对象存储；再利用 Doris 的 S3 Load 功能从对象存储中读取数据并导入到 Doris 中，具体可参考 S3 导入。

2.9.2.15.1 注意事项

1. 在迁移之前，需要根据 Redshift 的表结构选择 Doris 的数据模型，以及分区和分桶的策略，更多创建表策略可参考导入最佳实践。
2. Redshift 导出 Time 类型时，需要 Cast 成 Varchar 类型导出。

2.9.2.15.2 数据类型映射

Redshift	Doris	备注
SMALLINT	SMALLINT	
INTEGER	INT	
BIGINT	BIGINT	

Redshift	Doris	备注
DECIMAL	DECIMAL	
REAL	FLOAT	
DOUBLE PRECISION	DOUBLE	
BOOLEAN	BOOLEAN	
CHAR	CHAR	
VARCHAR	VARCHAR/STRING	VARCHAR 长度最大 65535
DATE	DATE	
TIMESTAMP	DATETIME	
TIME/TIMEZ	STRING	
SUPER	VARIANT	
OTHER	UNSUPPORTED	

2.9.2.15.3 1. 创建表

在迁移 Redshift 表到 Doris 中的时候，需要先创建 Doris 表。

假设我们在 Redshift 中已存在如下表和数据

```
CREATE TABLE sales_data (
    order_id      INTEGER,
    customer_name VARCHAR(128),
    order_date    DATE,
    amount        DECIMAL(10,2),
    country       VARCHAR(48)
)
DISTSTYLE AUTO

INSERT INTO sales_data VALUES
(1, 'Alice', '2025-04-08', 99.99, 'USA'),
(2, 'Bob', '2025-04-08', 149.50, 'Canada'),
(3, 'Charlie', '2025-04-09', 75.00, 'UK'),
(4, 'Diana', '2025-04-10', 200.00, 'Australia');
```

根据这个表结构，可以创建 Doris 主键分区表，分区字段根据业务场景选择，这里分区为 order_date，同时按天分区

```
CREATE TABLE `sales_data` (
  order_id      INT,
  order_date    DATE NOT NULL,
  customer_name VARCHAR(128),
  amount        DECIMAL(10,2),
  country       VARCHAR(48)
) ENGINE=OLAP
UNIQUE KEY(`order_id`,`order_date`)
PARTITION BY RANGE(`order_date`) (
```

```
PARTITION p20250408 VALUES [('2025-04-08'), ('2025-04-09')),
PARTITION p20250409 VALUES [('2025-04-09'), ('2025-04-10')),
PARTITION p20250410 VALUES [('2025-04-10'), ('2025-04-11'))
)
DISTRIBUTED BY HASH(`order_id`) BUCKETS 16
PROPERTIES (
 "dynamic_partition.enable" = "true",
 "dynamic_partition.time_unit" = "DAY",
 "dynamic_partition.end" = "5",
 "dynamic_partition.prefix" = "p",
 "dynamic_partition.buckets" = "16",
 "replication_num" = "1"
);
```

2.9.2.15.4 2. 导出 Redshift 数据

2.1 通过 UNLOAD 方式导出到 S3 Parquet 格式的文件

导出到 S3 时，按照 Doris 的 Partition 字段进行导出，如下：

```
unload ('select * from sales_data')
to 's3://mybucket/redshift/sales_data/'
iam_role 'arn:aws:iam::0123456789012:role/MyRedshiftRole'
PARQUET
PARTITION BY (order_date) INCLUDE
```

2.2 查看 S3 上的导出文件

导出后，在 S3 上会按照分区划分成具体的子目录，每一个目录是对应的分区数据。如下：

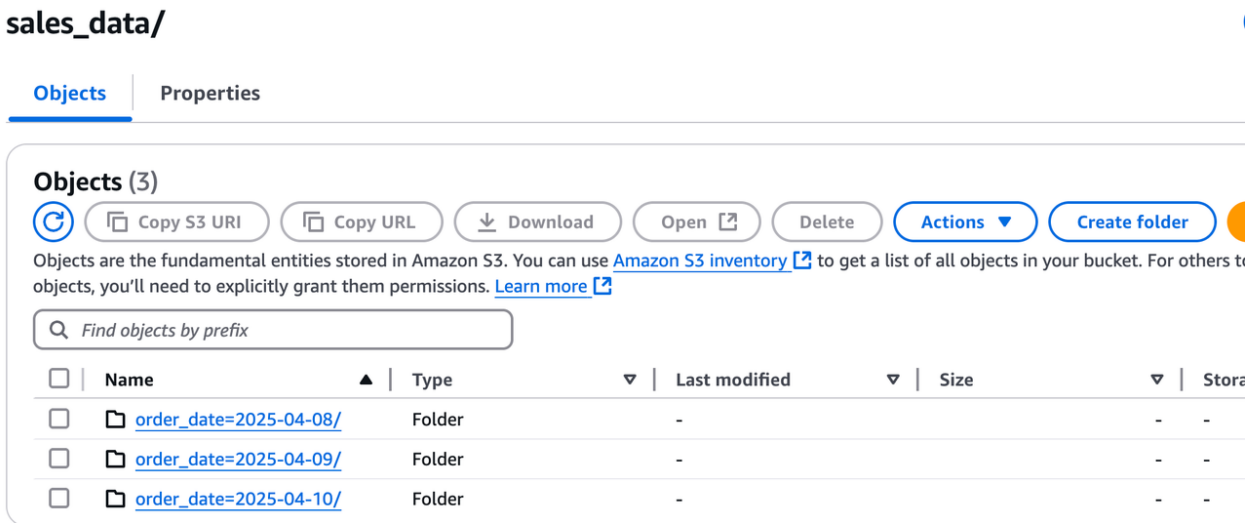


图 34: redshift_out

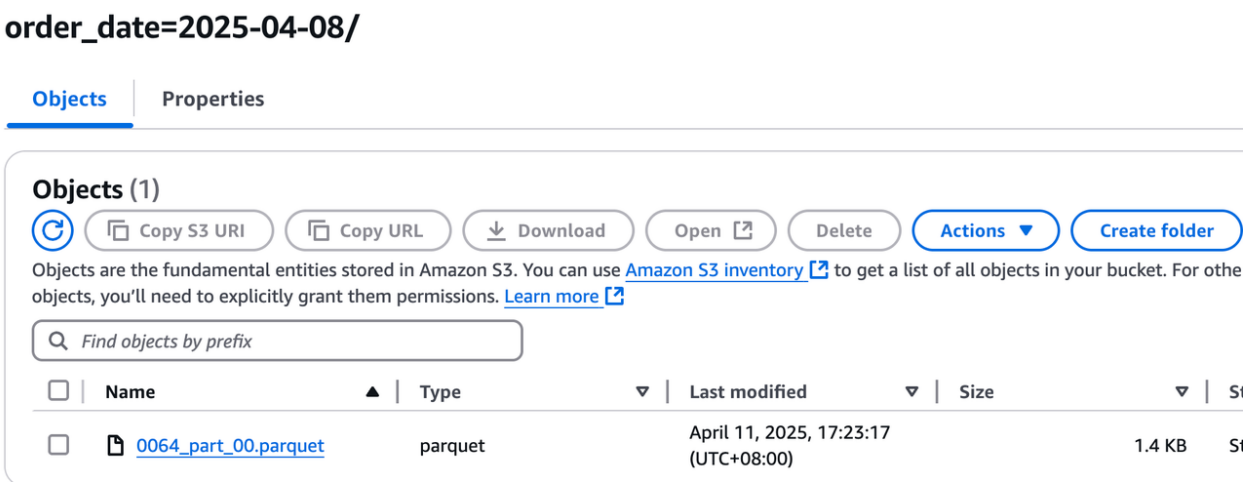


图 35: redshift_out2

2.9.2.15.5 3. 导入数据到 Doris

导入使用 S3 Load 进行导入，S3 Load 是一种异步的数据导入方式，执行后 Doris 会主动从数据源拉取数据，数据源支持兼容 S3 协议的对象存储，包括 (AWS S3，GCS，AZURE 等)。

该方式适用于数据量大、需要后台异步处理的场景。对于需要同步处理的数据导入，可以参考 TVF 导入。

注意：对于含有复杂类型（Struct/Array/Map）的 Parquet/ORC 格式文件导入，目前必须使用 TVF 导入

3.1 导入一个分区的数据

```
LOAD LABEL sales_data_2025_04_08
(
    DATA INFILE("s3://mybucket/redshift/sales_data/order_date=2025-04-08/*")
    INTO TABLE sales_data
    FORMAT AS "parquet"
    (order_id, order_date, customer_name, amount, country)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "s3.ap-southeast-1.amazonaws.com",
    "s3.access_key" = "<ak>",
    "s3.secret_key"="<sk>",
    "s3.region" = "ap-southeast-1"
);
```

3.2 通过 Show Load 查看任务运行情况

由于 S3Load 导入是异步提交的，所以需要通过 show load 可以查看指定 label 的导入情况：

```
mysql> show load where label = "label_sales_data_2025_04_08"`\G`
*************************** 1. row ***************************
```

```
         JobId: 17956078
         Label: label_sales_data_2025_04_08
         State: FINISHED
      Progress: 100.00% (1/1)
          Type: BROKER
       EtlInfo: unselected.rows=0; dpp.abnorm.ALL=0; dpp.norm.ALL=2
      TaskInfo: cluster:s3.ap-southeast-1.amazonaws.com; timeout(s):3600; max_filter_ratio:0.0;
          ↪ priority:NORMAL
      ErrorMsg: NULL
    CreateTime: 2025-04-10 17:50:53
  EtlStartTime: 2025-04-10 17:50:54
 EtlFinishTime: 2025-04-10 17:50:54
 LoadStartTime: 2025-04-10 17:50:54
LoadFinishTime: 2025-04-10 17:50:54
           URL: NULL
    JobDetails: {"Unfinished backends":{"5eec1be8612d4872-91040ff1e7208a4f":[]},"ScannedRows":2,"
        ↪ TaskNumber":1,"LoadBytes":91,"All backends":{"5eec1be8612d4872-91040ff1e7208a4f
        ↪ ":[10022]},"FileNumber":1,"FileSize":1620}
 TransactionId: 766228
  ErrorTablets: {}
          User: root
       Comment:
1 row in set (0.00 sec)
```

3.3 处理导入过程中的错误

当有多个导入任务时，可以通过以下语句，查询数据导入失败的日期和原因。

```
mysql> show load where state='CANCELLED' and label like "label_test%"`\G`
*************************** 1. row ***************************
         JobId: 18312384
         Label: label_test123
         State: CANCELLED
      Progress: 100.00% (3/3)
          Type: BROKER
       EtlInfo: unselected.rows=0; dpp.abnorm.ALL=4; dpp.norm.ALL=0
      TaskInfo: cluster:s3.ap-southeast-1.amazonaws.com; timeout(s):14400; max_filter_ratio:0.0;
          ↪ priority:NORMAL
      ErrorMsg: type:ETL_QUALITY_UNSATISFIED; msg:quality not good enough to cancel
    CreateTime: 2025-04-15 17:32:59
  EtlStartTime: 2025-04-15 17:33:02
 EtlFinishTime: 2025-04-15 17:33:02
 LoadStartTime: 2025-04-15 17:33:02
LoadFinishTime: 2025-04-15 17:33:02
           URL: http://10.16.10.6:28747/api/_load_error_log?file=__shard_2/error_log_insert_stmt_
              ↪ 7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307efca7bfe342
```

```
    JobDetails: {"Unfinished backends":{"7602ccd7c3a4854-95307efca7bfe341":[]},"ScannedRows":4,"
        ↪ TaskNumber":1,"LoadBytes":188,"All backends":{"7602ccd7c3a4854-95307efca7bfe341"
        ↪ :[10022]},"FileNumber":3,"FileSize":4839}
 TransactionId: 769213
  ErrorTablets: {}
          User: root
       Comment:
```

如上面的例子是数据质量错误 (ETL_QUALITY_UNSATISFIED)，具体错误需要通过访问返回的 URL 的链接进行查看，如下是数据超过了表中的 Schema 中 country 列的实际长度：

```
[root@VM-10-6-centos ~]$ curl "http://10.16.10.6:28747/api/_load_error_log?file=__shard_2/error_
    ↪ log_insert_stmt_7602ccd7c3a4854-95307efca7bfe342_7602ccd7c3a4854_95307efca7bfe342"
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [USA] schema length: 1; actual length: 3; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Canada] schema length: 1; actual length: 6; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [UK] schema length: 1; actual length: 2; . src line [];
Reason: column_name[country], the length of input is too long than schema. first 32 bytes of
    ↪ input str: [Australia] schema length: 1; actual length: 9; . src line [];
```

同时对于数据质量的错误，如果可以允许错误数据跳过的，可以通过在 S3 Load 任务中 Properties 设置容错率，具体可参考导入配置参数。

3.4 导入多个分区的数据

当需要迁移大数据量的存量数据时，建议使用分批导入的策略。每批数据对应 Doris 的一个分区或少量几个分区，数据量建议不超过 100GB，以减轻系统压力并降低导入失败后的重试成本。

可参考脚本 s3_load_demo.sh，该脚本可以实现了轮询 S3 上的分区目录，同时提交 S3 Load 任务到 Doris 中，实现批量导入的效果。

2.9.2.16 从其他 AP 系统迁移数据

从其他 AP 系统迁移数据到 Doris，可以有多种方式：

- Hive/Iceberg/Hudi 等，可以使用 Multi-Catalog 来映射为外表，然后使用 Insert Into，来将数据导入
- 也可以从原来 AP 系统中导出数据为 CSV 等数据格式，然后再将导出的数据导入到 Doris
- 可以使用 Spark / Flink 系统，利用 AP 系统的 Connector 来读取数据，然后调用 Doris Connector 写入 Doris

除此之外，还有以下第三方提供的迁移工具可供选择：

> NOTE 如果有其他迁移工具可以加入此列表，可以联系 dev@doris.apache.org

2.9.2.17 从其他 TP 系统迁移数据

从其他 TP 系统，如 MySQL/SqlServer/Oracle 等，迁移数据到 Doris，可以有多种方式。

2.9.2.17.1 Multi-Catalog

使用 Catalog 映射为外表，然后使用 INSERT INTO 或者 CREATE-TABLE-AS-SELECT 语句，完成数据导入。

以 MySQL 为例：

```
CREATE CATALOG mysql_catalog properties(
    'type' = 'jdbc',
    'user' = 'root',
    'password' = '123456',
    'jdbc_url' = 'jdbc:mysql://host:3306/mysql_db',
    'driver_url' = 'mysql-connector-java-8.0.25.jar',
    'driver_class' = 'com.mysql.cj.jdbc.Driver'
);

-- 通过 insert 导入
INSERT INTO internal.doris_db.tbl1
SELECT * FROM iceberg_catalog.iceberg_db.table1;

-- 通过 ctas 导入
CREATE TABLE internal.doris_db.tbl1
PROPERTIES('replication_num' = '1')
AS
SELECT * FROM iceberg_catalog.iceberg_db.table1;
```

具体可参考Catalog 数据导入。

2.9.2.17.2 Flink Doris Connector

可以借助于 Flink 完成 TP 系统的离线和实时同步。

- 离线同步可以使用 Flink 的 JDBC Source 和 Doris Sink 完成数据的导入，以 FlinkSQL 为例：“‘sql CREATE TABLE student_source (id INT, name STRING, age INT PRIMARY KEY (id) NOT ENFORCED) WITH (‘connector’ = ‘jdbc’, ‘url’ = ‘jdbc:mysql://localhost:3306/mydatabase’, ‘table-name’ = ‘students’, ‘username’ = ‘username’, ‘password’ = ‘password’,);

CREATE TABLE student_sink (id INT, name STRING, age INT) WITH (‘connector’ = ‘doris’, ‘fenodes’ = ‘127.0.0.1:8030’, ‘table.identifier’ = ‘test.students’, ‘username’ = ‘root’, ‘password’ = ‘password’, ‘sink.label-prefix’ = ‘doris_label’);

INSERT into student_sink select * from student_source; “‘ 具体可参考 Flink JDBC。

- 实时同步可以借助 FlinkCDC，完成全量和增量数据的读取，以 FlinkSQL 为例：“‘sql SET ‘execution.checkpointing.interval’ = ‘10s’;

CREATE TABLE cdc_mysql_source (id int ,name VARCHAR ,PRIMARY KEY (id) NOT ENFORCED) WITH (‘connector’ = ‘mysql-cdc’, ‘hostname’ = ‘127.0.0.1’, ‘port’ = ‘3306’, ‘username’ = ‘root’, ‘password’ = ‘password’, ‘database-name’ = ‘database’, ‘table-name’ = ‘table’);

– 支持同步 insert/update/delete 事件 CREATE TABLE doris_sink (id INT, name STRING) WITH (‘connector’ = ‘doris’, ‘fenodes’ = ‘127.0.0.1:8030’, ‘table.identifier’ = ‘database.table’, ‘username’ = ‘root’, ‘password’ = ‘’, ’sink.properties.format’ = ‘json’, ‘sink.properties.read_json_by_line’ = ‘true’, ‘sink.enable-delete’ = ‘true’, – 同步删除事件 ‘sink.label-prefix’ = ‘doris_label’);

insert into doris_sink select id,name from cdc_mysql_source; “ ‘

同时对于 TP 数据库中整库或者多表的同步操作，可以使用 Flink Doris Connector 提供的整库同步功能，一键完成 TP 数据库的写入，如：

```
<FLINK_HOME>bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.16-24.0.1.jar \
    mysql-sync-database \
    --database test_db \
    --mysql-conf hostname=127.0.0.1 \
    --mysql-conf port=3306 \
    --mysql-conf username=root \
    --mysql-conf password=123456 \
    --mysql-conf database-name=mysql_db \
    --including-tables "tbl1|test.*" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password=123456 \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

具体可参考：整库同步

2.9.2.17.3 Spark Connector

可以通过 Spark Connector 的 JDBC Source 和 Doris Sink 完成数据的写入。

```
val jdbcDF = spark.read
  .format("jdbc")
  .option("url", "jdbc:postgresql:dbserver")
  .option("dbtable", "schema.tablename")
  .option("user", "username")
  .option("password", "password")
  .load()
```

```
jdbcDF.write.format("doris")
 .option("doris.table.identifier", "db.table")
 .option("doris.fenodes", "127.0.0.1:8030")
 .option("user", "root")
 .option("password", "")
 .save()
```

具体可参考：JDBC To Other Databases，Spark Doris Connector

2.9.2.17.4 DataX / Seatunnel / CloudCanal 等三方工具

除此之外，也可以使用第三方同步工具来进行数据同步，更多可参考：- DataX - Seatunnel - CloudCanal

2.9.3 导入方式

2.9.3.1 Stream Load

Stream Load 支持通过 HTTP 协议将本地文件或数据流导入到 Doris 中。Stream Load 是一个同步导入方式，执行导入后返回导入结果，可以通过请求的返回判断导入是否成功。一般来说，可以使用 Stream Load 导入 10GB 以下的文件，如果文件过大，建议将文件进行切分后使用 Stream Load 进行导入。Stream Load 可以保证一批导入任务的原子性，要么全部导入成功，要么全部导入失败。

> 相比于直接使用 curl 的单并发导入，更推荐使用专用导入工具 Doris Streamloader。该工具是一款用于将数据导入 Doris 数据库的专用客户端工具，可以提供多并发导入的功能，降低大数据量导入的耗时。点击Doris Streamloader 文档 了解使用方法与实践详情。

2.9.3.1.1 使用场景

Stream Load 支持从本地或远程通过 HTTP 的方式导入 CSV、JSON、 Parquet 与 ORC 格式的数据。

在导入 CSV 文件时，需要明确区分空值（null）与空字符串：

- 空值（null）：使用 \N 表示。例如 a,\N,b 表示中间列的值为 null。
- 空字符串：当两个分隔符之间没有任何字符时表示空字符串。例如 a,,b 中，两个逗号之间没有字符，表示中间列的值为空字符串。

2.9.3.1.2 基本原理

在使用 Stream Load 时，需要通过 HTTP 协议发起导入作业给 FE 节点，FE 会以轮询方式，重定向（redirect）请求给一个 BE 节点以达到负载均衡的效果。也可以直接发送 HTTP 请求作业给指定的 BE 节点。在 Stream Load 中，Doris 会选定一个节点作为 Coordinator 节点。Coordinator 节点负责接受数据并分发数据到其他节点上。

下图展示了 Stream Load 的主要流程：

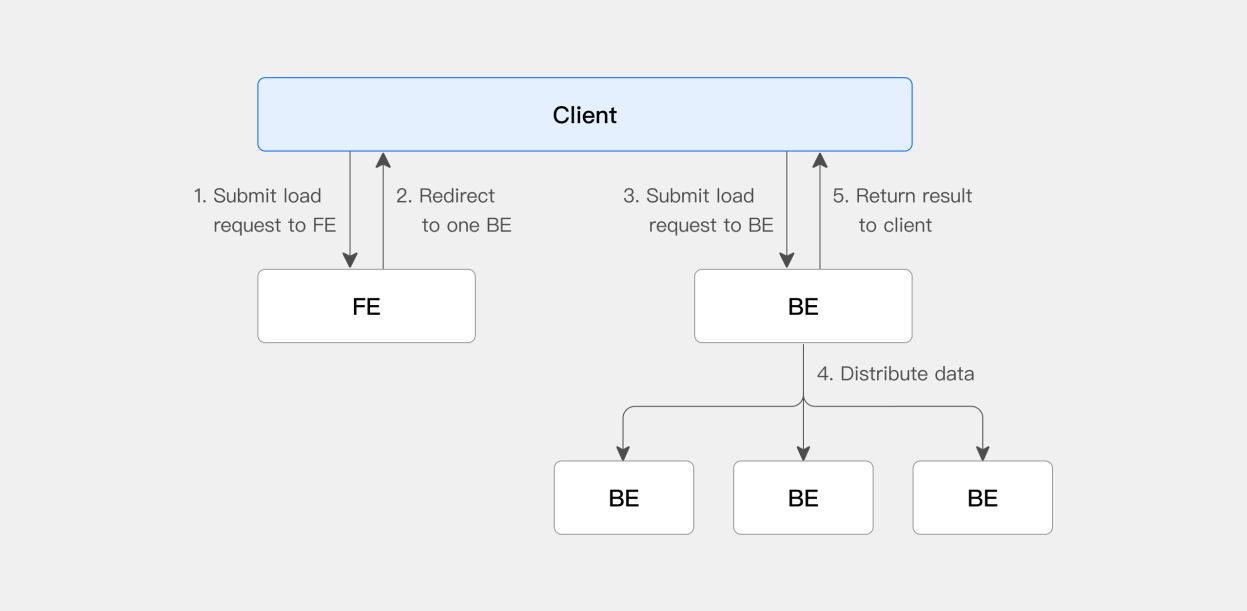


图 36: Stream Load 基本原理

1. Client 向 FE 提交 Stream Load 导入作业请求
2. FE 会轮询选择一台 BE 作为 Coordinator 节点，负责导入作业调度，然后返回给 Client 一个 HTTP 重定向
3. Client 连接 Coordinator BE 节点，提交导入请求
4. Coordinator BE 会分发数据给相应 BE 节点，导入完成后会返回导入结果给 Client
5. Client 也可以直接通过指定 BE 节点作为 Coordinator，直接分发导入作业

2.9.3.1.3 快速上手

Stream Load 通过 HTTP 协议提交和传输。下例以 curl 工具为例，演示通过 Stream Load 提交导入作业。

前置检查

Stream Load 需要对目标表的 INSERT 权限。如果没有 INSERT 权限，可以通过GRANT 命令给用户授权。

创建导入作业

导入 CSV 数据

1. 创建导入数据

   创建 CSV 文件 streamload_example.csv 文件。具体内容如下

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
```

```
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

2. 创建导入 Doris 表

   在 Doris 中创建被导入的表，具体语法如下

```
CREATE TABLE testdb.test_streamload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 启用导入作业

   通过 curl 命令可以提交 Stream Load 导入作业。

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

Stream Load 是一种同步导入方式，导入结果会直接返回给用户。

```
{
    "TxnId": 3,
    "Label": "123",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 10,
    "NumberLoadedRows": 10,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 118,
    "LoadTimeMs": 173,
```

```
    "BeginTxnTimeMs": 1,
    "StreamLoadPutTimeMs": 70,
    "ReadDataTimeMs": 2,
    "WriteDataTimeMs": 48,
    "CommitAndPublishTimeMs": 52
}
```

4. 查看导入数据

```
mysql> select count(*) from testdb.test_streamload;
+----------+
| count(*) |
+----------+
|       10 |
+----------+
```

导入 JSON 数据

1. 创建导入数据

   创建 JSON 文件 streamload_example.json。具体内容如下

```
[
{"userid":1,"username":"Emily","userage":25},
{"userid":2,"username":"Benjamin","userage":35},
{"userid":3,"username":"Olivia","userage":28},
{"userid":4,"username":"Alexander","userage":60},
{"userid":5,"username":"Ava","userage":17},
{"userid":6,"username":"William","userage":69},
{"userid":7,"username":"Sophia","userage":32},
{"userid":8,"username":"James","userage":64},
{"userid":9,"username":"Emma","userage":37},
{"userid":10,"username":"Liam","userage":64}
]
```

2. 创建导入 Doris 表

   在 Doris 中创建被导入的表，具体语法如下

```
CREATE TABLE testdb.test_streamload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 启用导入作业

   通过 curl 命令可以提交 Stream Load 导入作业。

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "label:124" \
    -H "Expect:100-continue" \
    -H "format:json" -H "strip_outer_array:true" \
    -H "jsonpaths:[\"$.userid\", \"$.username\", \"$.userage\"]" \
    -H "columns:user_id,name,age" \
    -T streamload_example.json \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

备注若 JSON 文件内容不是 JSON Array，而是每行一个 JSON 对象，添加 Header -H "strip_outer_array:false" -H "read_json_by_line:true"。如果需要将 JSON 文件中根节点的 JSON 对象导入，jsonpaths 需要指定为$.，如：-H "jsonpaths:[\"$.\"]"

Stream Load 是一种同步导入方式，导入结果会直接返回给用户。

```
{
    "TxnId": 7,
    "Label": "125",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 10,
    "NumberLoadedRows": 10,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 471,
    "LoadTimeMs": 52,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 11,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 23,
    "CommitAndPublishTimeMs": 16
}
```

查看导入作业

默认情况下，Stream Load 是同步返回给 Client，所以系统模式是不记录 Stream Load 历史作业的。如果需要记录，则在 be.conf 中添加配置 enable_stream_load_record=true。具体配置可以参考BE 配置项。

配置后，可以通过 show stream load 命令查看已完成的 Stream Load 任务。

```
mysql> show stream load from testdb;
+--
    ↪ -----+--------+-----------------+--------------+---------+---------+------+-----------+------------+----
    ↪
| Label | Db     | Table           | ClientIp      | Status  | Message | Url  | TotalRows |
    ↪ LoadedRows | FilteredRows | UnselectedRows | LoadBytes | StartTime               |
    ↪ FinishTime              | User | Comment |
+--
    ↪ -----+--------+-----------------+--------------+---------+---------+------+-----------+------------+----
    ↪
| 12356 | testdb | test_streamload | 192.168.88.31 | Success | OK      | N/A  | 10        | 10
    ↪         | 0            | 0              | 118       | 2023-11-29 08:53:00.594 |
    ↪ 2023-11-29 08:53:00.650 | root |         |
+--
    ↪ -----+--------+-----------------+--------------+---------+---------+------+-----------+------------+----
    ↪
1 row in set (0.00 sec)
```

取消导入作业

用户无法手动取消 Stream Load，Stream Load 在超时或者导入错误后会被系统自动取消。

2.9.3.1.4 参考手册

导入命令

Stream Load 导入语法如下：

```
curl --location-trusted -u <doris_user>:<doris_password> \
  -H "Expect:100-continue" [-H ""...] \
  -T <file_path> \
  -XPUT http://fe_host:http_port/api/{db}/{table}/_stream_load
```

Stream Load 操作支持 HTTP 分块导入（HTTP chunked）与 HTTP 非分块导入方式。对于非分块导入方式，必须要有 Content-Length 来标示上传内容的长度，这样能保证数据的完整性。

导入配置参数

FE 配置

1. stream_load_default_timeout_second

- 默认值：259200（s）
- 动态配置：是
- FE Master 独有配置：是

参数描述：Stream Load 默认的超时时间。导入任务的超时时间（以秒为单位），导入任务在设定的 timeout 时间内未完成则会被系统取消，变成 CANCELLED。如果导入的源文件无法在规定时间内完成导入，用户可以在 Stream Load 请求中设置单独的超时时间。或者调整 FE 的参数`stream_load_default_timeout_second` 来设置全局的默认超时时间。

BE 配置

1. streaming_load_max_mb

- 默认值：10240（MB）
- 动态配置：是
- 参数描述：Stream load 的最大导入大小。如果用户的原始文件超过这个值，则需要调整 BE 的参数 `streaming_load_max_mb`。

2. Header 参数

可以通过 HTTP 的 Header 部分来传入导入参数。具体参数介绍如下：

标签	参数说明
label	用于指定 Doris 该次导入的标签，标签相同的数据无法多次导入。如果不指定 label，Doris 会自动生成一个标签。用户可以通过指定 label 的方式来避免一份数据重复导入的问题。Doris 默认保留三天内的导入作业标签，可以 `label_keep_max_second` 调整保留时长。例如，指定本次导入 label 为 123，需要指定命令 `-H "label:123"`。label 的使用，可以防止用户重复导入相同的数据。强烈推荐用户同一批次数据使用相同的 label。这样同一批次数据的重复请求只会被接受一次，保证了 At-Most-Once 当 label 对应的导入作业状态为 CANCELLED 时，该 label 可以再次被使用。
column_separator	用于指定导入文件中的列分隔符，默认为`\t`。如果是不可见字符，则需要加`\x`作为前缀，使用十六进制来表示分隔符。可以使用多个字符的组合作为列分隔符。例如，hive 文件的分隔符 `\x01`，需要指定命令 `-H "column_separator:\x01"`。
line_delimiter	用于指定导入文件中的换行符，默认为 `\n`。可以使用做多个字符的组合作为换行符。例如，指定换行符为 `\n`，需要指定命令 `-H "line_delimiter:\\n"`。
columns	用于指定导入文件中的列和 table 中的列的对应关系。如果源文件中的列正好对应表中的内容，那么是不需要指定这个字段的内容的。如果源文件与表 schema 不对应，那么需要这个字段进行一些数据转换。有两种形式 column：直接对应导入文件中的字段，直接使用字段名表示衍生列，语法为 `column_name = expression` 详细案例参考导入过程中数据转换。
where	用于抽取部分数据。用户如果有需要将不需要的数据过滤掉，那么可以通过设定这个选项来达到。例如，只导入 k1 列等于 20180601 的数据，那么可以在导入时候指定 `-H "where: k1 = 20180601"`。

标签	参数说明
max_filter_ratio	最大容忍可过滤（数据不规范等原因）的数据比例，默认零容忍。取值范围是 0~1。当导入的错误率超过该值，则导入失败。数据不规范不包括通过 where 条件过滤掉的行。例如，最大程度保证所有正确的数据都可以导入（容忍度 100% ），需要指定命令 `-H "max_filter_ratio:1"`。
partitions	用于指定这次导入所涉及的 partition。如果用户能够确定数据对应的 partition，推荐指定该项。不满足这些分区的数据将被过滤掉。例如，指定导入到 p1, p2 分区，需要指定命令 `-H "partitions: p1, p2"`。
timeout	指定导入的超时时间。单位秒。默认是 600 秒。可设置范围为 1 秒 ~ 259200 秒。例如，指定导入超时时间为 1200s，需要指定命令 `-H "timeout:1200"`。
strict_mode	用户指定此次导入是否开启严格模式，默认为关闭。例如，指定开启严格模式，需要指定命令 `-H "strict_mode:true"`。
timezone	指定本次导入所使用的时区。默认为东八区。该参数会影响所有导入涉及的和时区有关的函数结果。例如，指定导入时区为 Africa/Abidjan，需要指定命令 `-H "timezone:Africa/Abidjan"`。
exec_mem_limit	导入内存限制。默认为 2GB。单位为字节。
format	指定导入数据格式，默认是 CSV 格式。目前支持以下格式：CSV, JSON, arrow, csv_with_names（支持 csv 文件行首过滤）csv_with_names_and_types（支持 CSV 文件前两行过滤）Parquet, ORC 例如，指定导入数据格式为 JSON，需要指定命令 `-H "format:json"`。
jsonpaths	导入 JSON 数据格式有两种方式：简单模式：没有指定 jsonpaths 为简单模式，这种模式要求 JSON 数据是对象类型匹配模式：用于 JSON 数据相对复杂，需要通过 jsonpaths 参数匹配对应的 value 在简单模式下，要求 JSON 中的 key 列与表中的列名是一一对应的，如 JSON 数据 {“k1”:1, “k2”:2, “k3”:“hello”}，其中 k1、k2 及 k3 分别对应表中的列。
strip_outer_array	指定 strip_outer_array 为 true 时表示 JSON 数据以数组对象开始且将数组对象中进行展平，默认为 false。在 JSON 数据的最外层是 [] 表示的数组时，需要设置 strip_outer_array 为 true。如以下示例数据，在设置 strip_outer_array 为 true 后，导入 Doris 中生成两行数据`[{"k1" : 1, "v1" : 2},{"k1" : 3, "v1" : 4}]`
json_root	json_root 为合法的 jsonpath 字符串，用于指定 json document 的根节点，默认值为 “”。
merge_type	数据的合并类型，支持三种类型：- APPEND（默认值）：表示这批数据全部追加到现有数据中 - DELETE：表示删除与这批数据 Key 相同的所有行 - MERGE：需要与 DELETE 条件联合使用，表示满足 DELETE 条件的数据按照 DELETE 语义处理，其余的按照 APPEND 语义处理例如，指定合并模式为 MERGE：`-H "merge_type: MERGE" -H "delete: flag=1"`
delete	仅在 MERGE 下有意义，表示数据的删除条件
function_column.sequence_col	只适用于 UNIQUE KEYS 模型，相同 Key 列下，保证 Value 列按照 source_sequence 列进行 REPLACE。source_sequence 可以是数据源中的列，也可以是表结构中的一列。

标签	参数说明
fuzzy_parse	布尔类型，为 true 表示 JSON 将以第一行为 schema 进行解析。开启这个选项可以提高 json 导入效率，但是要求所有 json 对象的 key 的顺序和第一行一致，默认为 false，仅用于 JSON 格式
num_as_string	布尔类型，为 true 表示在解析 JSON 数据时会将数字类型转为字符串，确保不会出现精度丢失的情况下进行导入。
read_json_by_line	布尔类型，为 true 表示支持每行读取一个 JSON 对象，默认值为 false。
send_batch_parallelism	整型，用于设置发送批处理数据的并行度，如果并行度的值超过 BE 配置中的 `max_send_batch_parallelism_per_job`，那么作为协调点的 BE 将使用 `max_send_batch_parallelism_per_job` 的值。
hidden_columns	用于指定导入数据中包含的隐藏列，在 Header 中不包含 Columns 时生效，多个 hidden column 用逗号分割。系统会使用用户指定的数据导入数据。在下例中，导入数据中最后一列数据为 `__DORIS_SEQUENCE_COL__`。 `hidden_columns: __DORIS_DELETE_SIGN__,__DORIS_SEQUENCE_COL__`
load_to_single_tablet	布尔类型，为 true 表示支持一个任务只导入数据到对应分区的一个 Tablet，默认值为 false。该参数只允许在对带有 random 分桶的 OLAP 表导数的时候设置。
compress_type	指定文件的压缩格式。目前只支持 CSV 文件的压缩。支持 gz, lzo, bz2, lz4, lzop, deflate 压缩格式。
trim_double_quotes	布尔类型，默认值为 false，为 true 时表示裁剪掉 CSV 文件每个字段最外层的双引号。
skip_lines	整数类型，默认值为 0，含义为跳过 CSV 文件的前几行。当设置 format 设置为 `csv_with_names`或`csv_with_names_and_types`时，该参数会失效。
comment	字符串类型，默认值为空。给任务增加额外的信息。
enclose	指定包围符。当 CSV 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为 “,”，包围符为 “ ‘ “，数据为”a,’b,c’ ”，则 “b,c” 会被解析为一个字段。注意：当 enclose 设置为"时，trim_double_quotes 一定要设置为 true。
escape	指定转义符。用于转义在字段中出现的与包围符相同的字符。例如数据为 “a, ‘b,’c’ ”，包围符为 “ ‘ “，希望”b,’c 被作为一个字段解析，则需要指定单字节转义符，例如 “”，将数据修改为”a,’b,’c’ ”。
memtable_on_sink_node	导入数据的时候是否开启 MemTable 前移，默认为 false。
unique_key_update_mode	Unique 表上的更新模式，目前仅对 Merge-On-Write Unique 表有效，一共支持三种类型 `UPSERT`, `UPDATE_FIXED_COLUMNS`, `UPDATE_FLEXIBLE_COLUMNS`。`UPSERT`: 表示以 upsert 语义导入数据; `UPDATE_FIXED_COLUMNS`: 表示以部分列更新的方式导入数据; `UPDATE_FLEXIBLE_COLUMNS`: 表示以灵活部分列更新的方式导入数据

导入返回值

Stream Load 是一种同步的导入方式，导入结果会通过创建导入的返回值直接给用户，如下所示：

```
{
    "TxnId": 1003,
    "Label": "b6f3bc78-0d2c-45d9-9e4c-faa0a0149bee",
    "Status": "Success",
    "ExistingJobStatus": "FINISHED", // optional
    "Message": "OK",
    "NumberTotalRows": 1000000,
    "NumberLoadedRows": 1000000,
    "NumberFilteredRows": 1,
    "NumberUnselectedRows": 0,
    "LoadBytes": 40888898,
    "LoadTimeMs": 2144,
    "BeginTxnTimeMs": 1,
    "StreamLoadPutTimeMs": 2,
    "ReadDataTimeMs": 325,
    "WriteDataTimeMs": 1933,
    "CommitAndPublishTimeMs": 106,
    "ErrorURL": "http://192.168.1.1:8042/api/_load_error_log?file=__shard_0/error_log_insert_stmt
        ↪ _db18266d4d9b4ee5-abb00ddd64bdf005_db18266d4d9b4ee5_abb00ddd64bdf005"
}
```

其中，返回结果参数如下表说明：

参数名称	说明
TxnId	导入事务的 ID
Label	导入作业的 label，通过 -H “label:” 指定
Status	导入的最终状态 - Success：表示导入成功 - Publish Timeout：该状态也表示导入已经完成，但数据可能会延迟可见，无需重试 - Label Already Exists：Label 重复，需要更换 label - Fail：导入失败
ExistingJobStatus	已存在的 Label 对应的导入作业的状态。这个字段只有在当 Status 为 “Label Already Exists” 时才会显示。用户可以通过这个状态，知晓已存在 Label 对应的导入作业的状态。“RUNNING” 表示作业还在执行，“FINISHED” 表示作业成功。
Message	导入错误信息
NumberTotalRows	导入总处理的行数
NumberLoadedRows	成功导入的行数
NumberFilteredRows	数据质量不合格的行数
NumberUnselectedRows	被 where 条件过滤的行数
LoadBytes	导入的字节数
LoadTimeMs	导入完成时间。单位毫秒
BeginTxnTimeMs	向 FE 请求开始一个事务所花费的时间，单位毫秒

参数名称	说明
StreamLoadPutTimeMs	向 FE 请求获取导入数据执行计划所花费的时间，单位毫秒
ReadDataTimeMs	读取数据所花费的时间，单位毫秒
WriteDataTimeMs	执行写入数据操作所花费的时间，单位毫秒
CommitAndPublishTimeMs	向 FE 请求提交并且发布事务所花费的时间，单位毫秒
ErrorURL	如果有数据质量问题，通过访问这个 URL 查看具体错误行

通过 ErrorURL 可以查看因为数据质量不佳导致的导入失败数据。使用命令 `curl "<ErrorURL>"` 命令直接查看错误数据的信息。

2.9.3.1.5 导入举例

设置导入超时时间与最大导入

导入任务的超时时间（以秒为单位），导入任务在设定的 timeout 时间内未完成则会被系统取消，变成 CANCELLED。通过指定参数 timeout 或者在 fe.conf 中添加参数 stream_load_default_timeout_second，可以调整 Stream Load 的导入超时时间。

在导入前需要根据文件大小计算导入的超时时间，如 100GB 的文件，预估 50MB/s 的性能导入：

```
导入时间 ≈ 100GB / 50MB/s ≈ 2048s
```

通过以下命令可以指定 timeout 3000s 创建 stream load 导入任务：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "timeout:3000" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

设置导入最大容错率

Doris 的导入任务可以容忍一部分格式错误的数据。容忍率通过 `max_filter_ratio` 设置。默认为 0，即表示当有一条错误数据时，整个导入任务将会失败。如果用户希望忽略部分有问题的数据行，可以将次参数设置为 0~1 之间的数值，Doris 会自动跳过哪些数据格式不正确的行。关于容忍率的一些计算方式，可以参阅数据转换 文档。

通过以下命令可以指定 max_filter_ratio 容忍度为 0.4 创建 stream load 导入任务：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "max_filter_ratio:0.4" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

设置导入过滤条件

导入过程中可以通过 WHERE 参数对导入的数据进行条件过滤。被过滤的数据不会参与到 filter ratio 的计算中，不影响 max_filter_ratio 的设置。在导入结束后，可以通过查看 num_rows_unselected 获取过滤的行数。

通过以下命令可以指定 WHERE 过滤条件创建 Stream Load 导入任务：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "where:age>=35" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入指定分区数据

将本地文件中的数据导入到表中的 p1, p2 分区，允许 20% 的错误率。

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "label:123" \
    -H "Expect:100-continue" \
    -H "max_filter_ratio:0.2" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -H "partitions: p1, p2" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

指定导入时区

由于 Doris 目前没有内置时区的时间类型，所有 DATETIME 相关类型均只表示绝对的时间点，而不包含时区信息，不因 Doris 系统时区变化而发生变化。因此，对于带时区数据的导入，我们统一的处理方式为将其转换为特定目标时区下的数据。在 Doris 系统中，即 session variable time_zone 所代表的时区。

而在导入中，我们的目标时区通过参数 timezone 指定，该变量在发生时区转换、运算时区敏感函数时将会替代 session variable time_zone。因此，如果没有特殊情况，在导入事务中应当设定 timezone 与当前 Doris 集群的 time_zone 一致。此时意味着所有带时区的时间数据，均会发生向该时区的转换。

例如，Doris 系统时区为 “+08:00”，导入数据中的时间列包含两条数据，分别为 “2012-01-01 01:00:00+00:00” 和 “2015-12-12 12:12:12-08:00”，则我们在导入时通过 -H "timezone: +08:00" 指定导入事务的时区后，这两条数据都会向该时区发生转换，从而得到结果 “2012-01-01 09:00:00” 和 “2015-12-13 04:12:12”。

更多关于时区解读可参考文档时区。

使用 Streaming 方式导入

Stream Load 是基于 HTTP 的协议进行导入，所以是支持使用程序，比如 Java、Go 或者 Python 等程序来流式写入，这也是为什么起名叫 Stream Load 的原因。

下面通过 bash 的命令管道来举例这种使用方式，这种导入的数据就是程序流式生成的，而不是本地文件。

```
seq 1 10 | awk '{OFS="`\t`"}{print $1, $1 * 10}' | curl --location-trusted -u root -T - http://
    ↪ host:port/api/testDb/testTbl/_stream_load
```

设置 CSV 首行过滤导入

文件数据：

```
 id,name,age
 1,doris,20
 2,flink,10
```

通过指定format=csv_with_names过滤首行导入

```
curl --location-trusted -u root -T test.csv  -H "label:1" -H "format:csv_with_names" -H "column_
    ↪ separator:," http://host:port/api/testDb/testTbl/_stream_load
```

指定 merge_type 进行 Delete 操作

在 Stream Load 中有三种导入类型：APPEND、 DELETE 与 MERGE。可以通过指定参数 merge_type 进行调整。如想指定将与导入数据 Key 相同的数据全部删除，可以使用以下命令：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "merge_type: DELETE" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

如导入数据前表中数据为：

```
+--------+----------+----------+------+
| siteid | citycode | username | pv   |
+--------+----------+----------+------+
|      3 |        2 | tom      |    2 |
|      4 |        3 | bush     |    3 |
|      5 |        3 | helen    |    3 |
+--------+----------+----------+------+
```

导入数据为：

```
3,2,tom,0
```

导入后会删除原表数据，变成以下结果集

```
+--------+----------+----------+------+
| siteid | citycode | username | pv   |
+--------+----------+----------+------+
|      4 |        3 | bush     |    3 |
```

```
|      5 |        3 | helen    |    3 |
+--------+----------+----------+------+
```

指定 merge_type 进行 Merge 操作

指定 merge_type 为 MERGE，可以将导入的数据 MERGE 到表中。MERGE 语义需要结合 DELETE 条件联合使用，表示满足 DELETE 条件的数据按照 DELETE 语义处理，其余按照 APPEND 语义添加到表中，如下面操作表示删除 siteid 为 1 的行，其余数据添加到表中：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "merge_type: MERGE" \
    -H "delete: siteid=1" \
    -H "column_separator:," \
    -H "columns:user_id,name,age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

如导入前的数据为：

```
+--------+----------+----------+------+
| siteid | citycode | username | pv   |
+--------+----------+----------+------+
|      4 |        3 | bush     |    3 |
|      5 |        3 | helen    |    3 |
|      1 |        1 | jim      |    2 |
+--------+----------+----------+------+
```

导入的数据为：

```
2,1,grace,2
3,2,tom,2
1,1,jim,2
```

导入后，将按照条件删除 siteid = 1 的行，siteid 为 2 与 3 的行会添加到表中：

```
+--------+----------+----------+------+
| siteid | citycode | username | pv   |
+--------+----------+----------+------+
|      4 |        3 | bush     |    3 |
|      2 |        1 | grace    |    2 |
|      3 |        2 | tom      |    2 |
|      5 |        3 | helen    |    3 |
+--------+----------+----------+------+
```

指定导入需要 Merge 的 Sequence 列

当 Unique Key 表设置了 Sequence 列时，在相同 Key 列下，Sequence 列的值会作为 REPLACE 聚合函数替换顺序的依据，较大值可以替换较小值。当对这种表基于 DORIS_DELETE_SIGN 进行删除标记时，需要保证 Key 相同和

Sequence 列值要大于等于当前值。通指定 function_column.sequence_col 参数可以结合 merge_type: DELETE 进行删除操作：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "merge_type: DELETE" \
    -H "function_column.sequence_col: age" \
    -H "column_separator:," \
    -H "columns: name, gender, age" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

如有以下表结构：

```
mysql> SET show_hidden_columns=true;
Query OK, 0 rows affected (0.00 sec)

mysql> DESC table1;
+-----------------------+--------------+------+-------+---------+---------+
| Field                 | Type         | Null | Key   | Default | Extra   |
+-----------------------+--------------+------+-------+---------+---------+
| name                  | VARCHAR(100) | No   | true  | NULL    |         |
| gender                | VARCHAR(10)  | Yes  | false | NULL    | REPLACE |
| age                   | INT          | Yes  | false | NULL    | REPLACE |
| __DORIS_DELETE_SIGN__ | TINYINT      | No   | false | 0       | REPLACE |
| __DORIS_SEQUENCE_COL__ | INT          | Yes  | false | NULL    | REPLACE |
+-----------------------+--------------+------+-------+---------+---------+
4 rows in set (0.00 sec)
```

假设原表中数据为：

```
+-------+--------+------+
| name  | gender | age  |
+-------+--------+------+
| li    | male   |   10 |
| wang  | male   |   14 |
| zhang | male   |   12 |
+-------+--------+------+
```

1. Sequence 参数生效，导入 Sequence 列大于等于表中原有数据

导入数据为：

```
li,male,10
```

由于指定了 function_column.sequence_col: age，并且 age 大于等于表中原有的列，原表数据被删除，表中数据变为：

```
+-------+--------+------+
| name  | gender | age  |
+-------+--------+------+
| wang  | male   |   14 |
| zhang | male   |   12 |
+-------+--------+------+
```

2. Sequence 参数未生效，导入 Sequence 列小于等于表中原有数据：

导入数据为：

```
li,male,9
```

由于指定了 function_column.sequence_col: age，但 age 小于表中原有的列，DELETE 操作并未生效，表中数据不变，依然会看到主键为 li 的列：

```
+-------+--------+------+
| name  | gender | age  |
+-------+--------+------+
| li    | male   |   10 |
| wang  | male   |   14 |
| zhang | male   |   12 |
+-------+--------+------+
```

并没有被删除，这是因为在底层的依赖关系上，会先判断 Key 相同的情况，对外展示 Sequence 列的值大的行数据，然后在看该行的 DORIS_DELETE_SIGN 值是否为 1，如果为 1 则不会对外展示，如果为 0，则仍会读出来。

导入包含包围符的数据

当 CSV 中的数据包含了分隔符或者分列符，为了防止截断，可以指定单字节字符作为包围符起到保护的作用。

如下列数据中，列中包含了分隔符 ,：

```
张三,30,'上海市，黄浦区，大沽路'
```

通过制定包围符'，可以将“上海市，黄浦区，大沽路”指定为一个字段：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "column_separator:," \
    -H "enclose:'" \
    -H "columns:username,age,address" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

如果包围字符也出现在字段中，如希望将“上海市，黄浦区，’ 大沽路”作为一个字段，需要先在列中进行字符串转义：

```
张三,30,'上海市，黄浦区，\'大沽路'
```

可以通过 escape 参数可以指定单字节转义字符，如下例中 \:

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "column_separator:," \
    -H "enclose:'" \
    -H "escape:\\" \
    -H "columns:username,age,address" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入包含 DEFAULT CURRENT_TIMESTAMP 类型的字段

下面给出导入数据到表字段含有 DEFAULT CURRENT_TIMESTAMP 的表中的例子：

表结构：

```
CREATE TABLE testDb.testTbl (
    `id` BIGINT(30) NOT NULL,
    `order_code` VARCHAR(30) DEFAULT NULL COMMENT '',
    `create_time` DATETIMEv2(3) DEFAULT CURRENT_TIMESTAMP
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 10;
```

JSON 数据格式：

```
{"id":1,"order_Code":"avc"}
```

导入命令：

```
curl --location-trusted -u root -T test.json -H "label:1" -H "format:json" -H 'columns: id, order
    ↪ _code, create_time=CURRENT_TIMESTAMP()' http://host:port/api/testDb/testTbl/_stream_load
```

简单模式导入 JSON 格式数据

在 JSON 字段和表中的列名一一对应时，可以通过指定参数 "strip_outer_array:true" 与 "format:json" 将 JSON 数据格式导入到表中。

如表定义如下：

```
CREATE TABLE testdb.test_streamload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

导入数据字段名与表中的字段名一一对应：

```
[
{"user_id":1,"name":"Emily","age":25},
{"user_id":2,"name":"Benjamin","age":35},
{"user_id":3,"name":"Olivia","age":28},
{"user_id":4,"name":"Alexander","age":60},
{"user_id":5,"name":"Ava","age":17},
{"user_id":6,"name":"William","age":69},
{"user_id":7,"name":"Sophia","age":32},
{"user_id":8,"name":"James","age":64},
{"user_id":9,"name":"Emma","age":37},
{"user_id":10,"name":"Liam","age":64}
]
```

通过以下命令，可以将 JSON 数据导入到表中：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "format:json" \
    -H "strip_outer_array:true" \
    -T streamload_example.json \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

匹配模式导入复杂的 JSON 格式数据

在 JSON 数据较为复杂，无法与表中的列名一一对应，或者有多余的列时，可以通过指定参数 jsonpaths 完成列名映射，进行数据匹配导入。如下列数据：

```
[
{"userid":1,"hudi":"lala","username":"Emily","userage":25,"userhp":101},
{"userid":2,"hudi":"kpkp","username":"Benjamin","userage":35,"userhp":102},
{"userid":3,"hudi":"ji","username":"Olivia","userage":28,"userhp":103},
{"userid":4,"hudi":"popo","username":"Alexander","userage":60,"userhp":103},
{"userid":5,"hudi":"uio","username":"Ava","userage":17,"userhp":104},
{"userid":6,"hudi":"lkj","username":"William","userage":69,"userhp":105},
{"userid":7,"hudi":"komf","username":"Sophia","userage":32,"userhp":106},
{"userid":8,"hudi":"mki","username":"James","userage":64,"userhp":107},
{"userid":9,"hudi":"hjk","username":"Emma","userage":37,"userhp":108},
{"userid":10,"hudi":"hua","username":"Liam","userage":64,"userhp":109}
]
```

通过指定 jsonpaths 参数可以匹配指定的列：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "format:json" \
    -H "strip_outer_array:true" \
    -H "jsonpaths:[\"$.userid\", \"$.username\", \"$.userage\"]" \
```

```
    -H "columns:user_id,name,age" \
    -T streamload_example.json \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

指定 JSON 根节点导入数据

如果 JSON 数据包含了嵌套 JSON 字段，需要指定导入 JSON 的根节点。默认值为 “”。

如下列数据，期望将 comment 列中的数据导入到表中：

```
[
    {"user":1,"comment":{"userid":101,"username":"Emily","userage":25}},
    {"user":2,"comment":{"userid":102,"username":"Benjamin","userage":35}},
    {"user":3,"comment":{"userid":103,"username":"Olivia","userage":28}},
    {"user":4,"comment":{"userid":104,"username":"Alexander","userage":60}},
    {"user":5,"comment":{"userid":105,"username":"Ava","userage":17}},
    {"user":6,"comment":{"userid":106,"username":"William","userage":69}},
    {"user":7,"comment":{"userid":107,"username":"Sophia","userage":32}},
    {"user":8,"comment":{"userid":108,"username":"James","userage":64}},
    {"user":9,"comment":{"userid":109,"username":"Emma","userage":37}},
    {"user":10,"comment":{"userid":110,"username":"Liam","userage":64}}
    ]
```

首先需要通过 json_root 参数指定根节点为 comment，然后根据 jsonpaths 参数完成列名映射：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "format:json" \
    -H "strip_outer_array:true" \
    -H "json_root: $.comment" \
    -H "jsonpaths:[\"$.userid\", \"$.username\", \"$.userage\"]" \
    -H "columns:user_id,name,age" \
    -T streamload_example.json \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入 Array 数据类型

如下列数据中包含了数组类型：

```
1|Emily|[1,2,3,4]
2|Benjamin|[22,45,90,12]
3|Olivia|[23,16,19,16]
4|Alexander|[123,234,456]
5|Ava|[12,15,789]
6|William|[57,68,97]
7|Sophia|[46,47,49]
8|James|[110,127,128]
9|Emma|[19,18,123,446]
10|Liam|[89,87,96,12]
```

将数据导入以下的表结构中：

```
CREATE TABLE testdb.test_streamload(
    typ_id      BIGINT          NOT NULL COMMENT "ID",
    name        VARCHAR(20)     NULL     COMMENT "name",
    arr         ARRAY<int(10)>  NULL     COMMENT "array"
)
DUPLICATE KEY(typ_id)
DISTRIBUTED BY HASH(typ_id) BUCKETS 10;
```

通过 Stream Load 任务作业，可以直接将文本文件中的 ARRAY 类型导入到表中：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "column_separator:|" \
    -H "columns:typ_id,name,arr" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入 map 数据类型

当导入数据中包含 map 类型，如以下的例子中：

```
[
{"user_id":1,"namemap":{"Emily":101,"age":25}},
{"user_id":2,"namemap":{"Benjamin":102,"age":35}},
{"user_id":3,"namemap":{"Olivia":103,"age":28}},
{"user_id":4,"namemap":{"Alexander":104,"age":60}},
{"user_id":5,"namemap":{"Ava":105,"age":17}},
{"user_id":6,"namemap":{"William":106,"age":69}},
{"user_id":7,"namemap":{"Sophia":107,"age":32}},
{"user_id":8,"namemap":{"James":108,"age":64}},
{"user_id":9,"namemap":{"Emma":109,"age":37}},
{"user_id":10,"namemap":{"Liam":110,"age":64}}
]
```

将数据导入以下表结构中：

```
CREATE TABLE testdb.test_streamload(
    user_id             BIGINT       NOT NULL COMMENT "ID",
    namemap             Map<STRING, INT>  NULL     COMMENT "namemap"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

通过 Stream Load 任务作业，可以直接将文本文件中的 map 类型导入到表中：

```
curl --location-trusted -u <doris_user>:<doris_password> \
```

```
    -H "Expect:100-continue" \
    -H "format: json" \
    -H "strip_outer_array:true" \
    -T streamload_example.json \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入 Bitmap 类型数据

在导入过程中，遇到 Bitmap 类型的数据，可以通过 to_bitmap 将数据转换成 Bitmap，或者通过 bitmap_empty 函数填充 Bitmap。

如导入数据如下：

```
1|koga|17723
2|nijg|146285
3|lojn|347890
4|lofn|489871
5|jfin|545679
6|kon|676724
7|nhga|767689
8|nfubg|879878
9|huang|969798
10|buag|97997
```

将数据导入到以下包含 Bitmap 类型的表中：

```
CREATE TABLE testdb.test_streamload(
    typ_id     BIGINT                NULL    COMMENT "ID",
    hou        VARCHAR(10)           NULL    COMMENT "one",
    arr        BITMAP  BITMAP_UNION  NOT NULL    COMMENT "two"
)
AGGREGATE KEY(typ_id,hou)
DISTRIBUTED BY HASH(typ_id,hou) BUCKETS 10;
```

通过以 to_bitmap 可以将数据转换成 Bitmap 类型：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "Expect:100-continue" \
    -H "columns:typ_id,hou,arr,arr=to_bitmap(arr)" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

导入 HLL 数据类型

通过 hll_hash 函数可以将数据转换成 hll 类型，如下数据：

```
1001|koga
1002|nijg
1003|lojn
```

```
1004|lofn
1005|jfin
1006|kon
1007|nhga
1008|nfubg
1009|huang
1010|buag
```

导入下例表中：

```
CREATE TABLE testdb.test_streamload(
    typ_id            BIGINT          NULL    COMMENT "ID",
    typ_name          VARCHAR(10)     NULL    COMMENT "NAME",
    pv                hll hll_union   NOT NULL    COMMENT "hll"
)
AGGREGATE KEY(typ_id,typ_name)
DISTRIBUTED BY HASH(typ_id) BUCKETS 10;
```

通过 hll_hash 命令进行导入：

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:|" \
    -H "columns:typ_id,typ_name,pv=hll_hash(typ_id)" \
    -T streamload_example.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_streamload/_stream_load
```

列映射、衍生列和过滤

Doris 可以在导入语句中支持非常丰富的列转换和过滤操作。支持绝大多数内置函数。关于如何正确的使用这个功能，可参阅数据转换 文档。

启用严格模式导入

strict_mode 属性用于设置导入任务是否运行在严格模式下。该属性会对列映射、转换和过滤的结果产生影响，它同时也将控制部分列更新的行为。关于严格模式的具体说明，可参阅严格模式文档。

导入时进行部分列更新/灵活部分列更新

关于导入时，如何表达部分列更新，可以参考数据更新/主键模型的导入更新 文档

2.9.3.2 Broker Load

Broker Load 通过 MySQL API 发起，Doris 会根据 LOAD 语句中的信息，主动从数据源拉取数据。Broker Load 是一个异步导入方式，需要通过 SHOW LOAD 语句查看导入进度和导入结果。

Broker Load 适合源数据存储在远程存储系统，比如对象存储或 HDFS，且数据量比较大的场景。从 HDFS 或者 S3 直接读取，也可以通过湖仓一体/TVF 中的 HDFS TVF 或者 S3 TVF 进行导入。基于 TVF 的 Insert Into 当前为同步导入，Broker Load 是一个异步的导入方式。

在 Doris 早期版本中，S3 Load 和 HDFS Load 都是通过 WITH BROKER 连接到具体的 Broker 进程实现的。随着版本的更新，S3 Load 和 HDFS Load 作为最常用的导入方式得到了优化，现在它们不再依赖额外的 Broker 进程，但仍然

使用与 Broker Load 类似的语法。由于历史原因以及语法上的相似，S3 Load、 HDFS Load 和 Broker Load 这三种导入方式被统称为 Broker Load。

2.9.3.2.1 使用限制

支持的存储后端：

- S3 协议
- HDFS 协议
- 其他协议（需要相应的 Broker 进程）

支持的数据类型：

- CSV
- JSON
- PARQUET
- ORC

支持的压缩类型：

- PLAIN
- GZ
- LZO
- BZ2
- LZ4FRAME
- DEFLATE
- LZOP
- LZ4BLOCK
- SNAPPYBLOCK
- ZLIB
- ZSTD

2.9.3.2.2 基本原理

用户在提交导入任务后，FE 会生成对应的 Plan 并根据目前 BE 的个数和文件的大小，将 Plan 分给多个 BE 执行，每个 BE 执行一部分导入数据。

BE 在执行的过程中会从 Broker 拉取数据，在对数据转换之后将数据导入系统。所有 BE 均完成导入，由 FE 最终决定导入是否成功。

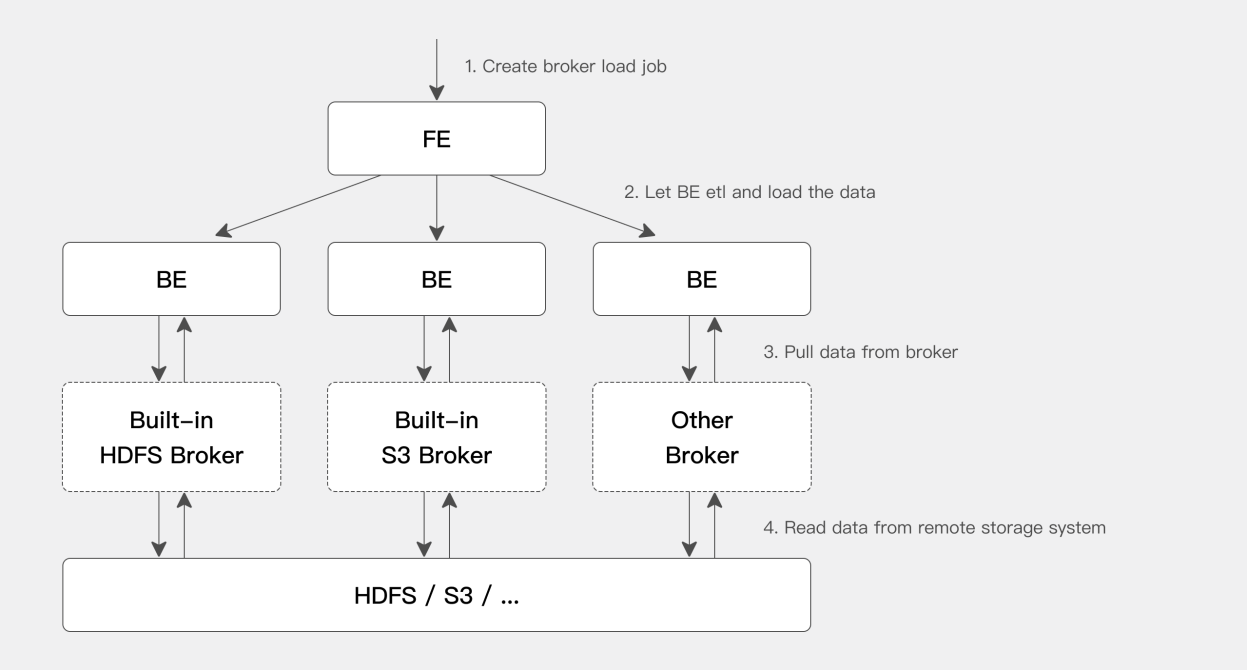


图 37: Broker Load 基本原理

从上图中可以看到，BE 会依赖 Broker 进程来读取相应远程存储系统的数据。之所以引入 Broker 进程，主要是用来针对不同的远程存储系统，用户可以按照 Broker 进程的标准开发其相应的 Broker 进程，Broker 进程可以使用 Java 程序开发，更好的兼容大数据生态中的各类存储系统。由于 broker 进程和 BE 进程的分离，也确保了两个进程的错误隔离，提升 BE 的稳定性。

当前 BE 内置了对 HDFS 和 S3 两个 Broker 的支持，所以如果从 HDFS 和 S3 中导入数据，则不需要额外启动 Broker 进程。如果有自己定制的 Broker 实现，则需要部署相应的 Broker 进程。

2.9.3.2.3 快速上手

本节演示了一个 S3 Load 示例。具体的使用语法，请参考 SQL 手册中的Broker Load。

前置检查

1. Doris 表权限

Broker Load 需要对目标表的 INSERT 权限。如果没有 INSERT 权限，可以通过GRANT 命令给用户授权。

2. S3 认证和连接信息

这里以 AWS S3 为例，从其他对象存储系统导入也可以作为参考。

- AK 和 SK：首先需要找到或者重新生成 AWS Access keys，可以在 AWS console 的 My Security Credentials 找到生成方式。

- REGION 和 ENDPOINT：REGION 可以在创建桶的时候选择也可以在桶列表中查看到。每个 REGION 的 S3 ENDPOINT 可以通过如下页面查到 AWS 文档。

创建导入作业

1. 创建 CSV 文件 brokerload_example.csv 文件存储在 S3 上，其内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

2. 创建导入 Doris 表

在 Doris 中创建被导入的表，具体语法如下：

```
CREATE TABLE testdb.test_brokerload(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 使用 Broker Load 从 S3 导入数据。其中 bucket 名称和 S3 认证信息要根据实际填写：

```
    LOAD LABEL broker_load_2022_04_01
    (
        DATA INFILE("s3://your_bucket_name/brokerload_example.csv")
        INTO TABLE test_brokerload
        COLUMNS TERMINATED BY ","
        FORMAT AS "CSV"
        (user_id, name, age)
    )
    WITH S3
    (
        "provider" = "S3",
        "AWS_ENDPOINT" = "s3.us-west-2.amazonaws.com",
```

```
        "AWS_ACCESS_KEY" = "<your-ak>",
        "AWS_SECRET_KEY"="<your-sk>",
        "AWS_REGION" = "us-west-2",
        "compress_type" = "PLAIN"
    )
    PROPERTIES
    (
        "timeout" = "3600"
    );
```

其中 provider 字段需要根据实际的对象存储服务商填写。Doris 支持的 provider 列表：

- “S3”（亚马逊 AWS）
- “AZURE”（微软 Azure）
- “GCP”（谷歌 GCP）
- “OSS”（阿里云）
- “COS”（腾讯云）
- “OBS”（华为云）
- “BOS”（百度云）

如不在列表中（例如 MinIO），可以尝试使用 “S3”（兼容 AWS 模式）

查看导入作业

Broker Load 是一个异步的导入方式，具体导入结果可以通过SHOW LOAD 命令查看

```
mysql> show load order by createtime desc limit 1`\G`;
*************************** 1. row ***************************
         JobId: 41326624
         Label: broker_load_2022_04_01
         State: FINISHED
      Progress: ETL:100%; LOAD:100%
          Type: BROKER
       EtlInfo: unselected.rows=0; dpp.abnorm.ALL=0; dpp.norm.ALL=27
      TaskInfo: cluster:N/A; timeout(s):1200; max_filter_ratio:0.1
      ErrorMsg: NULL
    CreateTime: 2022-04-01 18:59:06
  EtlStartTime: 2022-04-01 18:59:11
 EtlFinishTime: 2022-04-01 18:59:11
 LoadStartTime: 2022-04-01 18:59:11
LoadFinishTime: 2022-04-01 18:59:11
           URL: NULL
    JobDetails: {"Unfinished backends":{"5072bde59b74b65-8d2c0ee5b029adc0":[]},"ScannedRows":27,"
        ↪ TaskNumber":1,"All backends":{"5072bde59b74b65-8d2c0ee5b029adc0":[36728051]},"
        ↪ FileNumber":1,"FileSize":5540}
1 row in set (0.01 sec)
```

取消导入作业

当 Broker load 作业状态不为 CANCELLED 或 FINISHED 时，可以被用户手动取消。取消时需要指定待取消导入任务的 Label。取消导入命令语法可参考CANCEL LOAD 查看。

例如：取消数据库 demo 上，label 为 broker_load_2022_04_01 的导入作业

```
CANCEL LOAD FROM demo WHERE LABEL = "broker_load_2022_04_01";
```

2.9.3.2.4 参考手册

导入命令

```
LOAD LABEL load_label
(
data_desc1[, data_desc2, ...]
[format_properties]
)
WITH [S3|HDFS|BROKER broker_name]
[broker_properties]
[load_properties]
[COMMENT "comments"];
```

其中 WITH 子句指定了如何访问存储系统，broker_properties 是该访问方式的配置参数：

- S3: 使用 S3 协议的存储系统
- HDFS: 使用 HDFS 协议的存储系统
- BROKER broker_name: 其他协议的存储系统。可以通过 SHOW BROKER 查看目前可选的 broker_name 列表。更多信息见常见问题中的“其他 Broker 导入”

导入配置参数

导入参数（Load Properties）

Property 名称	类型	默认值	说明
“timeout”	Long	14400	导入的超时时间，单位秒。范围是1秒~259200秒。

Property 名称	类型	默认值	说明
“max_filter_ratio”	Float	0.0	最大容忍可过滤（数据不规范等原因）的数据比例，默认零容忍。取值范围是0~1。当导入的错误率超过该值，则导入失败

Property 名称	类型	默认值	说明
“strict_mode”	Boolean	false	是否开启严格模式。
“partial_columns”	Boolean	false	是否使用部分列更新，只在表模型为 Unique Key 且采用 Merge on Write 时有效。

Property 名称	类型	默认值	说明
“timezone”	String	“Asia/Shanghai”	本次导入所使用的时区。该参数会影响所有导入涉及的和时区有关的函数结果。

Property 名称	类型	默认值	说明
“load_parallelism”	Integer	8	每个 BE 上并发 instance 数量的上限。
“send_batch_parallelism”	Integer	1	sink 节点发送数据的并发度，仅在关闭 memtable 前移时生效。

Property 名称	类型	默认值	说明
“load_to_single_tablet”	Boolean	“false”	是否每个分区只导入一个tablet，默认值为false。该参数只允许在对带有random分桶的OLAP表导数的时候设置。

Property 名称	类型	默认值	说明
“priority”	“HIGH” 或 “NORMAL” 或 “LOW”	“NORMAL”	导入任务的优先级。

格式参数（Format Properties）

参数名	类型	默认值	描述
skip_lines	Integer	0	跳过 CSV 文件开头的若干行。当格式为 csv_with_names 或 csv_with_names ↪ _and_types 时，此参数无效。
trim_double_quotes	Boolean	false	是否去除字段外层的双引号。
enclose	String	""	字段包含换行符或分隔符时的包裹字符。例如，分隔符为 ,，包裹字符为 ' 时，'b,c' 会被解析为一个字段。
escape	String	""	用于转义包裹字符的转义字符。例如转义字符为 \，包裹字符为 '，字段 'b,\'c' 会被正确解析为 'b,'c'。

注意：格式参数用于定义如何解析源文件（如分隔符、引号处理），应在 LOAD 语句内部的 PROPERTIES 中设置。导入参数用于控制导入行为（如超时、重试），应在 LOAD 语句外部的最外层 PROPERTIES 块中设置。

```
LOAD LABEL s3_load_example (
    DATA INFILE("s3://bucket/path/file.csv")
    INTO TABLE users
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
    PROPERTIES (
        "trim_double_quotes" = "true"  -- 格式参数
    )
)
WITH S3 (
    ...
)
PROPERTIES (
    "timeout" = "3600"  -- 导入参数
);
```

fe.conf

下面几个配置属于 Broker load 的系统级别配置，也就是作用于所有 Broker load 导入任务的配置。主要通过修改 `fe.conf`来调整配置值。

Session Variable	类型	默认值	说明
min_bytes_per_broker_scanner	Long	67108864 (64 MB)	一个 Broker Load 作业中单 BE 处理的数据量的最小值，单位：字节。

Session Variable	类型	默认值	说明
max_bytes_per_broker_scanner	Long	536870912000 (500 GB)	一个 Broker Load 作业中单 BE 处理的数据量的最大值，单位：字节。通常一个导入作业支持的最大数据量为 max_bytes_per_broker_scanner * BE 节点数。当前的默认值是基于3个BE和每个BE 500G内存进行计算的。

Session Variable	类型	默认值	说明
max_broker_concurrency	Integer	10	限制了一个作业的最大的导入并发数。
default_load_parallelism	Integer	8	每个 BE 节点最大并发 instance 数

Session Variable	类型	默认值	说明
broker_load_default_timeout_second	14400	Broker Load 导入的默认超时时间，单位：秒。	

注：最小处理的数据量，最大并发数，源文件的大小和当前集群 BE 的数量共同决定了本次导入的并发数。

```
本次导入并发数 = Math.min(源文件大小/min_bytes_per_broker_scanner，max_broker_concurrency，当前BE
    ↪ 节点个数 * load_parallelism)
本次导入单个BE的处理量 = 源文件大小/本次导入的并发数
```

session variable

Session Variable	类型	默认值	说明
time_zone	String	“Asia/Shanghai”	默认时区，会影响导入中时区相关的函数结果。
send_batch_parallelism	Integer	1	sink 节点发送数据的并发度，仅在关闭 memtable 前移时生效。

2.9.3.2.5 常见问题

常见报错

1. 导入报错：`Scan bytes per broker scanner exceed limit:xxx`

请参考文档中最佳实践部分，修改 FE 配置项 `max_bytes_per_broker_scanner` 和 `max_broker_concurrency`

2. 导入报错：`failed to send batch` 或 `TabletWriter add batch with unknown id`

适当修改 `query_timeout` 和 `streaming_load_rpc_max_alive_time_sec`。

3. 导入报错：`LOAD_RUN_FAIL; msg:Invalid Column Name:xxx`

如果是 PARQUET 或者 ORC 格式的数据，则文件头的列名需要与 Doris 表中的列名保持一致，如：

```
(tmp_c1,tmp_c2)
SET
```

```
(
    id=tmp_c2,
    name=tmp_c1
)
```

代表获取在 parquet 或 orc 中以 (tmp_c1, tmp_c2) 为列名的列，映射到 doris 表中的 (id, name) 列。如果没有设置 set, 则以 column 中的列作为映射。

注：如果使用某些 hive 版本直接生成的 orc 文件，orc 文件中的表头并非 hive meta 数据，而是 (_col0, _col1, _col2, ···) , 可能导致 Invalid Column Name 错误，那么则需要使用 set 进行映射

4. 导入报错：`Failed to get S3 FileSystem for bucket is null/empty`

Bucket 信息填写不正确或者不存在。或者 bucket 的格式不受支持。使用 GCS 创建带_的桶名时，比如：`s3://gs_bucket/load_tbl`，S3 Client 访问 GCS 会报错，建议创建 bucket 路径时不使用_。

5. 导入超时

导入的 timeout 默认超时时间为 4 小时。如果超时，不推荐用户将导入最大超时时间直接改大来解决问题。单个导入时间如果超过默认的导入超时时间 4 小时，最好是通过切分待导入文件并且分多次导入来解决问题。因为超时时间设置过大，那么单次导入失败后重试的时间成本很高。

可以通过如下公式计算出 Doris 集群期望最大导入文件数据量：

```
期望最大导入文件数据量 = 14400s * 10M/s * BE 个数
比如：集群的 BE 个数为 10个
期望最大导入文件数据量 = 14400s * 10M/s * 10 = 1440000M ≈ 1440G

注意：一般用户的环境可能达不到 10M/s 的速度，所以建议超过 500G 的文件都进行文件切分，再导入。
```

S3 Load URL 访问方式

- S3 SDK 默认使用 virtual-hosted-style 方式。但某些对象存储系统可能没开启或没支持 virtual-hosted-style 方式的访问，此时我们可以添加 `use_path_style` 参数来强制使用 path-style 方式：

```
WITH S3
(
      "AWS_ENDPOINT" = "AWS_ENDPOINT",
      "AWS_ACCESS_KEY" = "AWS_ACCESS_KEY",
      "AWS_SECRET_KEY"="AWS_SECRET_KEY",
      "AWS_REGION" = "AWS_REGION",
      "use_path_style" = "true"
)
```

S3 Load 临时密钥

- 支持使用临时密钥 (TOKEN) 访问所有支持 S3 协议的对象存储，用法如下：

```
    WITH S3
    (
        "AWS_ENDPOINT" = "AWS_ENDPOINT",
        "AWS_ACCESS_KEY" = "AWS_TEMP_ACCESS_KEY",
        "AWS_SECRET_KEY" = "AWS_TEMP_SECRET_KEY",
        "AWS_TOKEN" = "AWS_TEMP_TOKEN",
        "AWS_REGION" = "AWS_REGION"
    )
```

HDFS 认证方式

1. 简单认证

简单认证即 Hadoop 配置 hadoop.security.authentication 为 simple。

```
(
    "username" = "user",
    "password" = ""
);
```

username 配置为要访问的用户，密码置空即可。

2. Kerberos 认证

该认证方式需提供以下信息：

- hadoop.security.authentication：指定认证方式为 Kerberos。
- hadoop.kerberos.principal：指定 Kerberos 的 principal。
- hadoop.kerberos.keytab：指定 Kerberos 的 keytab 文件路径。该文件必须为 Broker 进程所在服务器上的文件的绝对路径。并且可以被 Broker 进程访问。
- kerberos_keytab_content：指定 Kerberos 中 keytab 文件内容经过 base64 编码之后的内容。这个跟 kerberos_keytab 配置二选一即可。

示例如下：

```
(
    "hadoop.security.authentication" = "kerberos",
    "hadoop.kerberos.principal" = "doris@YOUR.COM",
    "hadoop.kerberos.keytab" = "/home/doris/my.keytab"
)
(
    "hadoop.security.authentication" = "kerberos",
    "hadoop.kerberos.principal" = "doris@YOUR.COM",
    "kerberos_keytab_content" = "ASDOWHDLAWIDJHWLDKSALDJSDIWALD"
)
```

采用 Kerberos 认证方式，需要 krb5.conf (opens new window) 文件，krb5.conf 文件包含 Kerberos 的配置信息，通常，应该将 krb5.conf 文件安装在目录/etc 中。可以通过设置环境变量 KRB5_CONFIG 覆盖默认位置。krb5.conf 文件的内容示例如下：

```
[libdefaults]
    default_realm = DORIS.HADOOP
    default_tkt_enctypes = des3-hmac-sha1 des-cbc-crc
    default_tgs_enctypes = des3-hmac-sha1 des-cbc-crc
    dns_lookup_kdc = true
    dns_lookup_realm = false

[realms]
    DORIS.HADOOP = {
        kdc = kerberos-doris.hadoop.service:7005
    }
```

HDFS HA 模式

这个配置用于访问以 HA 模式部署的 HDFS 集群。

- `dfs.nameservices`：指定 HDFS 服务的名字，自定义，如：“dfs.nameservices” = “my_ha”。
- `dfs.ha.namenodes.xxx`：自定义 namenode 的名字，多个名字以逗号分隔。其中 xxx 为 `dfs.nameservices` 中自定义的名字，如：“dfs.ha.namenodes.my_ha” = “my_nn”。
- `dfs.namenode.rpc-address.xxx.nn`：指定 namenode 的 rpc 地址信息。其中 nn 表示 `dfs.ha.namenodes.xxx` 中配置的 namenode 的名字，如：“dfs.namenode.rpc-address.my_ha.my_nn” = “host:port”。
- `dfs.client.failover.proxy.provider.[nameservice ID]`：指定 client 连接 namenode 的 provider，默认为：org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider。

示例如下：

```
(
    "fs.defaultFS" = "hdfs://my_ha",
    "dfs.nameservices" = "my_ha",
    "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
    "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
    "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
    "dfs.client.failover.proxy.provider.my_ha" = "org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider"
)
```

HA 模式可以和前面两种认证方式组合，进行集群访问。如通过简单认证访问 HA HDFS：

```
(
    "username"="user",
    "password"="passwd",
    "fs.defaultFS" = "hdfs://my_ha",
```

```
    "dfs.nameservices" = "my_ha",
    "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
    "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
    "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
    "dfs.client.failover.proxy.provider.my_ha" = "org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider"
)
```

其他 Broker 导入

其他远端存储系统的 Broker 是 Doris 集群中的可选进程，主要用于支持 Doris 对远端存储中文件和目录的读写。目前，Doris 提供了多种远端存储系统的 Broker 实现。历史版本中，Doris 还支持过各种对象存储的 Broker，但现在更推荐使用 `WITH S3` 方式来导入对象存储中的数据，而不再推荐使用 `WITH BROKER`。

- 腾讯云 CHDFS
- 腾讯云 GFS
- JuiceFS

Broker 通过提供一个 RPC 服务端口来提供服务，是一个无状态的 Java 进程，负责为远端存储的读写操作封装一些类 POSIX 的文件操作，如 open，pread，pwrite 等等。除此之外，Broker 不记录任何其他信息，所以包括远端存储的连接信息、文件信息、权限信息等等，都需要通过参数在 RPC 调用中传递给 Broker 进程，才能使得 Broker 能够正确读写文件。

Broker 仅作为一个数据通路，并不参与任何计算，因此仅需占用较少的内存。通常一个 Doris 系统中会部署一个或多个 Broker 进程。并且相同类型的 Broker 会组成一个组，并设定一个名称（Broker name）。

这里主要介绍 Broker 在访问不同远端存储时需要的参数，如连接信息、权限认证信息等等。

Broker 信息

Broker 的信息包括名称（Broker name）和认证信息两部分。通常的语法格式如下：

```
WITH BROKER "broker_name"
(
    "username" = "xxx",
    "password" = "yyy",
    "other_prop" = "prop_value",
    ...
);
```

- 名称

通常用户需要通过操作命令中的 `WITH BROKER "broker_name"` 子句来指定一个已经存在的 Broker Name。Broker Name 是用户在通过 `ALTER SYSTEM ADD BROKER` 命令添加 Broker 进程时指定的一个名称。一个名称通常对应一个或多个 Broker 进程。Doris 会根据名称选择可用的 Broker 进程。用户可以通过 `SHOW BROKER` 命令查看当前集群中已经存在的 Broker。

备注 Broker Name 只是一个用户自定义名称，不代表 Broker 的类型。

- 认证信息

不同的 Broker 类型，以及不同的访问方式需要提供不同的认证信息。认证信息通常在 `WITH BROKER "broker_name"` 之后的 Property Map 中以 Key-Value 的方式提供。

2.9.3.2.6 导入示例

导入 HDFS 上的 TXT 文件

```
LOAD LABEL demo.label_20220402
(
    DATA INFILE("hdfs://host:port/tmp/test_hdfs.txt")
    INTO TABLE `load_hdfs_file_test`
    COLUMNS TERMINATED BY "`\t`"
    (id,age,name)
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
)
PROPERTIES
(
    "timeout"="1200",
    "max_filter_ratio"="0.1"
);
```

HDFS 需要配置 NameNode HA 的情况

```
LOAD LABEL demo.label_20220402
(
    DATA INFILE("hdfs://hafs/tmp/test_hdfs.txt")
    INTO TABLE `load_hdfs_file_test`
    COLUMNS TERMINATED BY "`\t`"
    (id,age,name)
)
with HDFS
(
    "hadoop.username" = "user",
    "fs.defaultFS"="hdfs://hafs",
    "dfs.nameservices" = "hafs",
```

```
    "dfs.ha.namenodes.hafs" = "my_namenode1, my_namenode2",
    "dfs.namenode.rpc-address.hafs.my_namenode1" = "nn1_host:rpc_port",
    "dfs.namenode.rpc-address.hafs.my_namenode2" = "nn2_host:rpc_port",
    "dfs.client.failover.proxy.provider.hafs" = "org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider"
)
PROPERTIES
(
    "timeout"="1200",
    "max_filter_ratio"="0.1"
);
```

从 HDFS 导入数据，使用通配符匹配两批文件，分别导入到两个表中

```
LOAD LABEL example_db.label2
(
    DATA INFILE("hdfs://host:port/input/file-10*")
    INTO TABLE `my_table1`
    PARTITION (p1)
    COLUMNS TERMINATED BY ","
    (k1, tmp_k2, tmp_k3)
    SET (
        k2 = tmp_k2 + 1,
        k3 = tmp_k3 + 1
    ),
    DATA INFILE("hdfs://host:port/input/file-20*")
    INTO TABLE `my_table2`
    COLUMNS TERMINATED BY ","
    (k1, k2, k3)
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

使用通配符匹配导入两批文件 `file-10*` 和 `file-20*`。分别导入到 `my_table1` 和 `my_table2` 两张表中。其中 `my_table1` 指定导入到分区 `p1` 中，并且将导入源文件中第二列和第三列的值 `+1` 后导入。

使用通配符从 HDFS 导入一批数据

```
LOAD LABEL example_db.label3
(
    DATA INFILE("hdfs://host:port/user/doris/data/*/*")
    INTO TABLE `my_table`
    COLUMNS TERMINATED BY "\\x01"
)
```

```
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

指定分隔符为 Hive 经常用的默认分隔符 \\x01，并使用通配符 * 指定 data 目录下所有目录的所有文件。

导入 Parquet 格式数据，指定 FORMAT 为 parquet

```
  LOAD LABEL example_db.label4
  (
      DATA INFILE("hdfs://host:port/input/file")
      INTO TABLE `my_table`
      FORMAT AS "parquet"
      (k1, k2, k3)
  )
  with HDFS
  (
    "fs.defaultFS"="hdfs://host:port",
    "hadoop.username" = "user"
  );
```

默认是通过文件后缀判断。

导入数据，并提取文件路径中的分区字段

```
LOAD LABEL example_db.label5
(
    DATA INFILE("hdfs://host:port/input/city=beijing/*/*")
    INTO TABLE `my_table`
    FORMAT AS "csv"
    (k1, k2, k3)
    COLUMNS FROM PATH AS (city, utc_date)
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

my_table 表中的列为 k1, k2, k3, city, utc_date。

其中 hdfs://hdfs_host:hdfs_port/user/doris/data/input/dir/city=beijing 目录下包括如下文件：

```
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-01/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-02/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-03/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-04/0000.csv
```

文件中只包含 k1，k2，k3 三列数据，city，utc_date 这两列数据会从文件路径中提取。

对导入数据进行过滤

```
LOAD LABEL example_db.label6
(
    DATA INFILE("hdfs://host:port/input/file")
    INTO TABLE `my_table`
    (k1, k2, k3)
    SET (
        k2 = k2 + 1
    )
    PRECEDING FILTER k1 = 1
    WHERE k1 > k2
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

只有原始数据中，k1 = 1，并且转换后，k1 > k2 的行才会被导入。

导入数据，提取文件路径中的时间分区字段

```
LOAD LABEL example_db.label7
(
    DATA INFILE("hdfs://host:port/user/data/*/test.txt")
    INTO TABLE `tbl12`
    COLUMNS TERMINATED BY ","
    (k2,k3)
    COLUMNS FROM PATH AS (data_time)
    SET (
        data_time=str_to_date(data_time, '%Y-%m-%d %H%%3A%i%%3A%s')
    )
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

> 时间包含%3A。在 hdfs 路径中，不允许有 ':' ，所有 ':' 会由%3A 替换。

路径下有如下文件：

```
/user/data/data_time=2020-02-17 00%3A00%3A00/test.txt
/user/data/data_time=2020-02-18 00%3A00%3A00/test.txt
```

表结构为：

```
CREATE TABLE IF NOT EXISTS tbl12 (
    data_time DATETIME,
    k2        INT,
    k3        INT
) DISTRIBUTED BY HASH(data_time) BUCKETS 10
PROPERTIES (
    "replication_num" = "3"
);
```

使用 Merge 方式导入

```
LOAD LABEL example_db.label8
(
    MERGE DATA INFILE("hdfs://host:port/input/file")
    INTO TABLE `my_table`
    (k1, k2, k3, v2, v1)
    DELETE ON v2 > 100
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
)
PROPERTIES
(
    "timeout" = "3600",
    "max_filter_ratio" = "0.1"
);
```

使用 Merge 方式导入。my_table 必须是一张 Unique Key 的表。当导入数据中的 v2 列的值大于 100 时，该行会被认为是一个删除行。导入任务的超时时间是 3600 秒，并且允许错误率在 10% 以内。

导入时指定 source_sequence 列，保证替换顺序

```
LOAD LABEL example_db.label9
(
    DATA INFILE("hdfs://host:port/input/file")
    INTO TABLE `my_table`
    COLUMNS TERMINATED BY ","
    (k1,k2,source_sequence,v1,v2)
    ORDER BY source_sequence
)
```

```
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

my_table 必须是 Unique Key 模型表，并且指定了 Sequence 列。数据会按照源数据中 source_sequence 列的值来保证顺序性。

- 导入指定文件格式为 json，并指定 json_root、jsonpaths 属性：

```
LOAD LABEL example_db.label10
(
    DATA INFILE("hdfs://host:port/input/file.json")
    INTO TABLE `my_table`
    FORMAT AS "json"
    PROPERTIES(
      "json_root" = "$.item",
      "jsonpaths" = "[\"$.id\", \"$.city\", \"$.code\"]"
    )
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

jsonpaths 也可以与 column list 及 SET (column_mapping)配合使用：

```
LOAD LABEL example_db.label10
(
    DATA INFILE("hdfs://host:port/input/file.json")
    INTO TABLE `my_table`
    FORMAT AS "json"
    (id, code, city)
    SET (id = id * 10)
    PROPERTIES(
      "json_root" = "$.item",
      "jsonpaths" = "[\"$.id\", \"$.city\", \"$.code\"]"
    )
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

备注如果需要将 JSON 文件中根节点的 JSON 对象导入，jsonpaths 需要指定为 $.，如：PROPERTIES ("jsonpaths"="$.")

从其他 Broker 导入

- 阿里云 OSS

```
(
    "fs.oss.accessKeyId" = "",
    "fs.oss.accessKeySecret" = "",
    "fs.oss.endpoint" = ""
)
```

- 百度云 BOS

当前使用 BOS 时需要下载相应的 SDK 包，具体配置与使用，可以参考 BOS HDFS 官方文档。在下载完成并解压后将 jar 包放到 broker 的 lib 目录下。

```
(
    "fs.bos.access.key" = "xx",
    "fs.bos.secret.access.key" = "xx",
    "fs.bos.endpoint" = "xx"
)
```

- 华为云 OBS

```
(
    "fs.obs.access.key" = "xx",
    "fs.obs.secret.key" = "xx",
    "fs.obs.endpoint" = "xx"
)
```

- JuiceFS

```
(
    "fs.defaultFS" = "jfs://xxx/",
    "fs.jfs.impl" = "io.juicefs.JuiceFileSystem",
    "fs.AbstractFileSystem.jfs.impl" = "io.juicefs.JuiceFS",
    "juicefs.meta" = "xxx",
    "juicefs.access-log" = "xxx"
)
```

- GCS

在使用 Broker 访问 GCS 时，Project ID 是必须的，其他参数可选，所有参数配置请参考 GCS Config

```
(
    "fs.gs.project.id" = "Your Project ID",
    "fs.AbstractFileSystem.gs.impl" = "com.google.cloud.hadoop.fs.gcs.GoogleHadoopFS",
    "fs.gs.impl" = "com.google.cloud.hadoop.fs.gcs.GoogleHadoopFileSystem",
)
```

2.9.3.2.7 更多帮助

关于 Broker Load 使用的更多详细语法及最佳实践，请参阅Broker Load 命令手册，你也可以在 MySQL 客户端命令行中输入 HELP BROKER LOAD 获取更多帮助信息。

2.9.3.3 Routine Load

Doris 可以通过 Routine Load 导入方式持续消费 Kafka Topic 中的数据。在提交 Routine Load 作业后，Doris 会持续运行该导入作业，实时生成导入任务不断消费 Kafka 集群中指定 Topic 的消息。

Routine Load 是一个流式导入作业，支持 Exactly-Once 语义，保证数据不丢不重。

2.9.3.3.1 使用场景

支持数据源

Routine Load 支持从 Kafka 集群中消费数据。

支持数据文件格式

Routine Load 支持 CSV 及 JSON 格式的数据。

在导入 CSV 格式时，需要明确区分空值（null）与空字符串（”）：

- 空值（null）需要用 \n 表示，a,\n,b 数据表示中间列是一个空值（null）
- 空字符串（ ‘’ ）直接将数据置空，a,,b 数据表示中间列是一个空字符串（”）

使用限制

在使用 Routine Load 消费 Kafka 中数据时，有以下限制：

- 支持的消息格式为 CSV 及 JSON 文本格式。CSV 每一个 message 为一行，且行尾不包含换行符；
- 默认支持 Kafka 0.10.0.0（含）以上版本。如果要使用 Kafka 0.10.0.0 以下版本（0.9.0, 0.8.2, 0.8.1, 0.8.0），需要修改 BE 的配置，将 kafka_broker_version_fallback 的值设置为要兼容的旧版本，或者在创建 Routine Load 的时候直接设置 property.broker.version.fallback 的值为要兼容的旧版本，使用旧版本的代价是 Routine Load 的部分新特性可能无法使用，如根据时间设置 Kafka 分区的 offset。

2.9.3.3.2 基本原理

Routine Load 会持续消费 Kafka Topic 中的数据，写入 Doris 中。

在 Doris 中，创建 Routine Load 作业后会生成一个常驻的导入作业，包括若干个导入任务：

- 导入作业（load job）：一个 Routine Load Job 是一个常驻的导入作业，会持续不断地消费数据源中的数据。
- 导入任务（load task）：一个导入作业会被拆解成若干个导入任务进行实际消费，每个任务都是一个独立的事务。

Routine Load 的导入具体流程如下图所示：

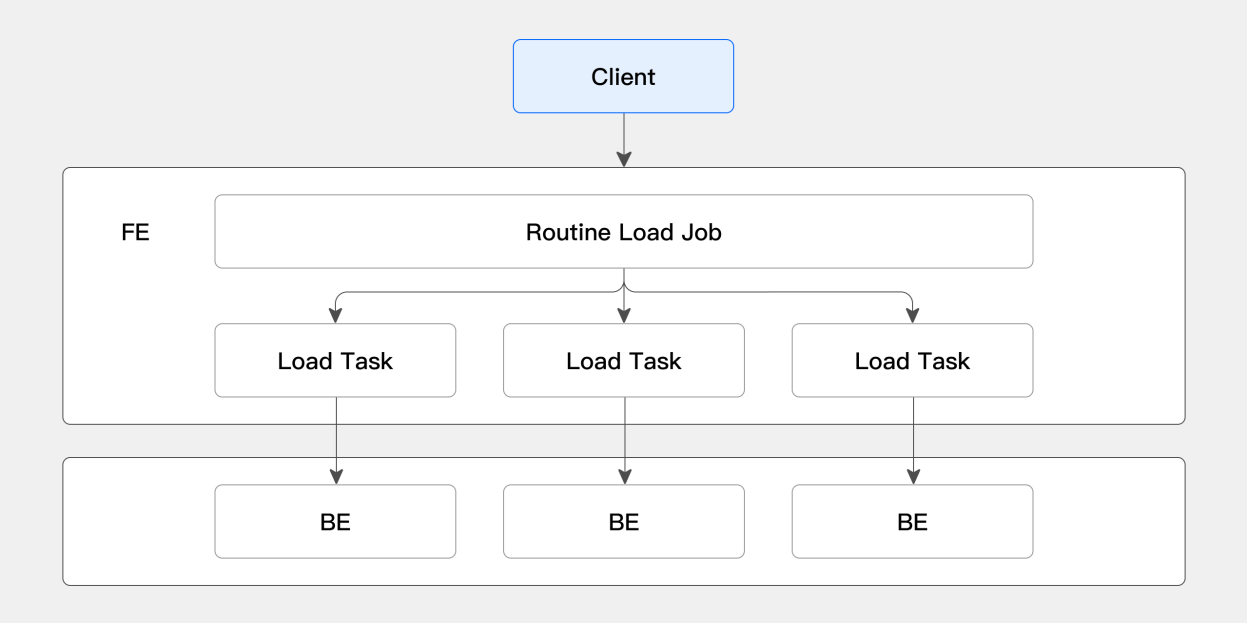


图 38: Routine Load

1. Client 向 FE 提交创建 Routine Load 作业请求，FE 通过 Routine Load Manager 生成一个常驻的导入作业（Routine Load Job）。
2. FE 通过 Job Scheduler 将 Routine Load Job 拆分成若干个 Routine Load Task，由 Task Scheduler 进行调度，下发到 BE 节点。
3. 在 BE 上，一个 Routine Load Task 导入完成后向 FE 提交事务，并更新 Job 的元数据。
4. 一个 Routine Load Task 提交后，会继续生成新的 Task，或对超时的 Task 进行重试。
5. 新生成的 Routine Load Task 由 Task Scheduler 继续调度，不断循环。

自动恢复

为了确保作业的高可用性，引入了自动恢复机制。在非预期暂停的情况下，Routine Load Scheduler 调度线程会尝试自动恢复作业。对于 Kafka 侧的意外宕机或其他无法工作的情况，自动恢复机制可以确保在 Kafka 恢复后，无需人工干预，导入作业能够继续正常运行。

不会自动恢复的情况：

- 用户手动执行 PAUSE ROUTINE LOAD 命令。
- 数据质量存在问题。
- 无法自动恢复的情况，例如库表被删除。

除了上述三种情况，其他暂停状态的作业都会尝试自动恢复。

FAQ

1. 对于集群重启或升级，自动恢复可能会遇到一些问题。在 3.0.2 版本之前，集群重启或升级同样有较高的概率导致任务暂停后无法自动恢复。自 3.0.2 版本起，即使集群重启或升级导致任务暂停，预期也能实现自动恢复。

2.9.3.3.3 快速上手

创建导入作业

在 Doris 内可以通过 CREATE ROUTINE LOAD 命令创建常驻 Routine Load 导入任务。详细语法可以参考CREATE ROUTINE LOAD。 Routine Load 可以消费 CSV 和 JSON 的数据。

导入 CSV 数据

1. 导入数据样本

在 Kafka 中，有以下样本数据

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-routine-load-csv --from-
    ↪ beginning
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

2. 创建需要导入的表

在 Doris 中，创建被导入的表，具体语法如下

```
CREATE TABLE testdb.test_routineload_tbl(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
```

```
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 创建 Routine Load 导入作业

在 Doris 中，使用 CREATE ROUTINE LOAD 命令，创建导入作业

```
CREATE ROUTINE LOAD testdb.example_routine_load_csv ON test_routineload_tbl
COLUMNS TERMINATED BY ",",
COLUMNS(user_id, name, age)
FROM KAFKA(
    "kafka_broker_list" = "192.168.88.62:9092",
    "kafka_topic" = "test-routine-load-csv",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

导入 JSON 数据

1. 导入样本数据

在 Kafka 中，有以下样本数据

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-routine-load-json --from
    ↪ -beginning
{"user_id":1,"name":"Emily","age":25}
{"user_id":2,"name":"Benjamin","age":35}
{"user_id":3,"name":"Olivia","age":28}
{"user_id":4,"name":"Alexander","age":60}
{"user_id":5,"name":"Ava","age":17}
{"user_id":6,"name":"William","age":69}
{"user_id":7,"name":"Sophia","age":32}
{"user_id":8,"name":"James","age":64}
{"user_id":9,"name":"Emma","age":37}
{"user_id":10,"name":"Liam","age":64}
```

2. 创建需要导入的表

在 Doris 中，创建被导入的表，具体语法如下

```
CREATE TABLE testdb.test_routineload_tbl(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 创建 Routine Load 导入作业

在 Doris 中，使用 CREATE ROUTINE LOAD 命令，创建导入作业

```
CREATE ROUTINE LOAD testdb.example_routine_load_json ON test_routineload_tbl
COLUMNS(user_id,name,age)
PROPERTIES(
    "format"="json",
    "jsonpaths"="[\"$.user_id\",\"$.name\",\"$.age\"]"
)
FROM KAFKA(
    "kafka_broker_list" = "192.168.88.62:9092",
    "kafka_topic" = "test-routine-load-json",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

> 备注如果需要将 JSON 文件中根节点的 JSON 对象导入，jsonpaths 需要指定为$.，如：PROPERTIES
> ↪ ("jsonpaths"="$.")

查看导入状态

在 Doris 中，Routine Load 的导入作业情况和导入任务状态：

- 导入作业：主要用于查看导入任务目标表、子任务数量、导入延迟状态、导入配置与导入结果等信息；
- 导入任务：主要用于查看导入的子任务状态、消费进度以及下发的 BE 节点。

01 查看导入运行任务

可以通过SHOW ROUTINE LOAD 命令查看导入作业情况。SHOW ROUTINE LOAD 描述了当前作业的基本情况，如导入目标表、导入延迟状态、导入配置信息、导入错误信息等。

如通过以下命令可以查看 testdb.example_routine_load_csv 的任务情况：

```
mysql>` SHOW ROUTINE LOAD FOR testdb.example_routine_load`\G`
*************************** 1. row ***************************
                  Id: 12025
                Name: example_routine_load
          CreateTime: 2024-01-15 08:12:42
           PauseTime: NULL
             EndTime: NULL
              DbName: default_cluster:testdb
           TableName: test_routineload_tbl
        IsMultiTable: false
               State: RUNNING
```

```
      DataSourceType: KAFKA
      CurrentTaskNum: 1
       JobProperties: {"max_batch_rows":"200000","timezone":"America/New_York","send_batch_
           ↪ parallelism":"1","load_to_single_tablet":"false","column_separator":"','","line_
           ↪ delimiter":"`\n`","current_concurrent_number":"1","delete":"*","partial_columns":"
           ↪ false","merge_type":"APPEND","exec_mem_limit":"2147483648","strict_mode":"false","
           ↪ jsonpaths":"","max_batch_interval":"10","max_batch_size":"104857600","fuzzy_parse"
           ↪ :"false","partitions":"*","columnToColumnExpr":"user_id,name,age","whereExpr":"*",
           ↪ "desired_concurrent_number":"5","precedingFilter":"*","format":"csv","max_error_
           ↪ number":"0","max_filter_ratio":"1.0","json_root":"","strip_outer_array":"false","
           ↪ num_as_string":"false"}
DataSourceProperties: {"topic":"test-topic","currentKafkaPartitions":"0","brokerList":"
    ↪ 192.168.88.62:9092"}
    CustomProperties: {"kafka_default_offsets":"OFFSET_BEGINNING","group.id":"example_routine_
        ↪ load_73daf600-884e-46c0-a02b-4e49fdf3b4dc"}
           Statistic: {"receivedBytes":28,"runningTxns":[],"errorRows":0,"committedTaskNum":3,"
               ↪ loadedRows":3,"loadRowsRate":0,"abortedTaskNum":0,"errorRowsAfterResumed":0,"
               ↪ totalRows":3,"unselectedRows":0,"receivedBytesRate":0,"taskExecuteTimeMs"
               ↪ :30069}
            Progress: {"0":"2"}
                 Lag: {"0":0}
ReasonOfStateChanged:
        ErrorLogUrls:
            OtherMsg:
                User: root
             Comment:
1 row in set (0.00 sec)
```

02 查看导入运行作业

可以通过SHOW ROUTINE LOAD TASK 命令查看导入子任务情况。SHOW ROUTINE LOAD TASK 描述了当前作业下的子任务信息，如子任务状态，下发 BE id 等信息。

如通过以下命令可以查看 testdb.example_routine_load_csv 的任务情况：

```
mysql>` SHOW ROUTINE LOAD TASK WHERE jobname = 'example_routine_load_csv';
+--
    ↪ ---------------------------------+-------+-----------+-------+---------------------+----------------------
    ↪
| TaskId                           | TxnId | TxnStatus | JobId | CreateTime          |
    ↪ ExecuteStartTime    | Timeout | BeId  | DataSourceProperties |
+--
    ↪ ---------------------------------+-------+-----------+-------+---------------------+----------------------
    ↪
| 8cf47e6a68ed4da3-8f45b431db50e466 | 195   | PREPARE   | 12177 | 2024-01-15 12:20:41 |
    ↪ 2024-01-15 12:21:01 | 20      | 10429 | {"4":1231,"9":2603}  |
| f2d4525c54074aa2-b6478cf8daaeb393 | 196   | PREPARE   | 12177 | 2024-01-15 12:20:41 |
```

```
    ↪ 2024-01-15 12:21:01 | 20      | 12109 | {"1":1225,"6":1216}  |
| cb870f1553864250-975279875a25fab6 | -1    | NULL      | 12177 | 2024-01-15 12:20:52 | NULL
    ↪                | 20      | -1    | {"2":7234,"7":4865}  |
| 68771fd8a1824637-90a9dac2a7a0075e | -1    | NULL      | 12177 | 2024-01-15 12:20:52 | NULL
    ↪                | 20      | -1    | {"3":1769,"8":2982}  |
| 77112dfea5e54b0a-a10eab3d5b19e565 | 197   | PREPARE   | 12177 | 2024-01-15 12:21:02 |
    ↪ 2024-01-15 12:21:02 | 20      | 12098 | {"0":3000,"5":2622}  |
+--
    ↪ ---------------------------------+-------+-----------+-------+---------------------+----------------------
    ↪
```

暂停导入作业

可以通过PAUSE ROUTINE LOAD 命令暂停导入作业。暂停导入作业后，会进入 PAUSED 状态，但导入作业并未终止，可以通过 RESUME ROUTINE LOAD 命令重启导入作业。

如通过以下命令可以暂停 testdb.example_routine_load_csv 导入作业：

```
PAUSE ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

恢复导入作业

可以通过RESUME ROUTINE LOAD 命令恢复导入作业。

如通过以下命令可以恢复 testdb.example_routine_load_csv 导入作业：

```
RESUME ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

修改导入作业

可以通过ALTER ROUTINE LOAD 命令修改已创建的导入作业。在修改导入作业前，需要使用 PAUSE ROUTINE LOAD 暂停导入作业，修改后需要使用 RESUME ROUTINE LOAD 恢复导入作业。

如通过以下命令可以修改期望导入任务并行度参数 desired_concurrent_number，并修改 Kafka Topic 信息：

```
ALTER ROUTINE LOAD FOR testdb.example_routine_load_csv
PROPERTIES(
    "desired_concurrent_number" = "3"
)
FROM KAFKA(
    "kafka_broker_list" = "192.168.88.60:9092",
    "kafka_topic" = "test-topic"
);
```

取消导入作业

可以通过STOP ROUTINE LOAD 命令停止并删除 Routine Load 导入作业。删除后的导入作业无法被恢复，也无法通过 SHOW ROUTINE LOAD 命令查看。

可以通过以下命令停止并删除导入作业 testdb.example_routine_load_csv：

```
STOP ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

2.9.3.3.4 参考手册

导入命令

创建一个 Routine Load 常驻导入作业语法如下：

```
CREATE ROUTINE LOAD [`<db_name>`.]`<job_name>` [ON `<tbl_name>`]
[merge_type]
[load_properties]
[job_properties]
FROM KAFKA [data_source_properties]
[COMMENT "`<comment>`"]
```

创建导入作业的模块说明如下：

模块	说明
db_name	指定创建导入任务的数据库。
job_name	指定创建的导入任务名称，同一个 database 不能有名字相同的任务。
tbl_name	指定需要导入的表的名称，可选参数，如果不指定，则采用动态表的方式，这个时候需要 Kafka 中的数据包含表名的信息。
merge_type	数据合并类型。默认值为 APPEND。 merge_type 有三种选项： - APPEND：追加导入方式； - MERGE：合并导入方式； - DELETE：导入的数据皆为需要删除的数据。
load_properties	导入描述模块，包括以下组成部分： - column_spearator 子句 - columns_mapping 子句 - preceding_filter 子句 - where_predicates 子句 - partitions 子句 - delete_on 子句 - order_by 子句
job_properties	用于指定 Routine Load 的通用导入参数。
data_source_properties	用于描述 Kafka 数据源属性。
comment	用于描述导入作业的备注信息。

导入参数说明

01 FE 配置参数

参数名称	默认值	动态配置	FE Master 独有配置	参数描述
max_routine_load_task_concurrent_num	256	是	是	限制 Routine Load 的导入作业最大子并发数量。建议维持在默认值。如果设置过大，可能导致并发任务数过多，占用集群资源。
max_routine_load_task_num_per_be	1024	是	是	每个 BE 限制的最大并发 Routine Load 任务数。max_routine_load_task_num_per_be 应该小于 routine_load_thread_pool_size。
max_routine_load_job_num	100	是	是	限制最大 Routine Load 作业数，包括 NEED_SCHEDULED，RUNNING，PAUSE。

参数名称	默认值	动态配置	FE Master 独有配置	参数描述
max_tolerable_backend_down_num	0	是	是	只要有一个 BE 宕机，Routine Load 就无法自动恢复。在满足某些条件时，Doris 可以将 PAUSED 的任务重新调度，转换为 RUNNING 状态。该参数为 0 表示只有所有 BE 节点都处于 alive 状态时允许重新调度。
period_of_auto_resume_min	5（分钟）	是	是	自动恢复 Routine Load 的周期。

02 BE 配置参数

参数名称	默认值	动态配置	描述
max_consumer_num_per_group	3	是	一个子任务最多生成几个 consumer 消费

03 导入配置参数

在创建 Routine Load 作业时，可以通过 CREATE ROUTINE LOAD 命令指定不同模块的导入配置参数。

tbl_name 子句

指定需要导入的表的名称，可选参数。

如果不指定，则采用动态表的方式，这个时候需要 Kafka 中的数据包含表名的信息。目前仅支持从 Kafka 的 Value 中获取动态表名，且需要符合这种格式：以 json 为例：`table_name|{"col1": "val1", "col2": "val2"}`，其中 `tbl_name` 为表名，以 `|` 作为表名和表数据的分隔符。CSV 格式的数据也是类似的，如：`table_name|val1,val2,val3`。注意，这里的 `table_name` 必须和 Doris 中的表名一致，否则会导致导入失败。注意，动态表不支持后面介绍的 column_mapping 配置。

merge_type 子句

可以通过 merge_type 模块指定数据合并的类型。merge_type 有三种选项：

- APPEND：追加导入方式
- MERGE：合并导入方式。仅适用于 Unique Key 模型。需要配合 [DELETE ON] 模块，以标注 Delete Flag 列
- DELETE：导入的数据皆为需要删除的数据

load_properties 子句

可以通过 load_properties 模块描述导入数据的属性，具体语法如下

```
[COLUMNS TERMINATED BY `<column_separator>`,]
[COLUMNS (`<column1_name>`[, `<column2_name>`, `<column_mapping>`, ...]),]
[WHERE `<where_expr>`,]
[PARTITION(`<partition1_name>`, [`<partition2_name>`, `<partition3_name>`, ...]),]
```

```
[DELETE ON `<delete_expr>`,]
[ORDER BY `<order_by_column1>`[, `<order_by_column2>`, `<order_by_column3>`, ...]]
```

具体模块对应参数如下：

子模块	参数	说明
COLUMNS TERMINATED BY	<column_separator>	用于指定列分隔符，默认为 \t。例如需要指定逗号为分隔符，可以使用以下命令：COLUMN TERMINATED BY ","对于空值处理，需要注意以下事项： - 空值（null）需要用 \n 表示，a,\n,b 数据表示中间列是一个空值（null） - 空字符串（‘’）直接将数据置空，a,,b 数据表示中间列是一个空字符串（”）
COLUMNS	<column_name>	用于指定对应的列名例如需要指定导入列 (k1, k2, k3)，可以使用以下命令：COLUMNS(k1, k2, k3)在以下情况下可以缺省 COLUMNS 子句： - CSV 中的列与表中的列一一对应 - JSON 中的 key 列与表中的列名相同
	<column_mapping>	在导入过程中，可以通过列映射进行列的过滤和转换。如在导入的过程中，目标列需要基于数据源的某一列进行衍生计算，目标列 k4 基于 k3 列使用公式 k3+1 计算得出，需要可以使用以下命令：COLUMNS(k1, k2, k3, k4 = k3 + 1)详细内容可以参考数据转换
WHERE	<where_expr>	指定 where_expr 可以根据条件过滤导入的数据源。如只希望导入 age >30 的数据源 ↪ ，可以使用以下命令：WHERE age >30
PARTITION	<partition_name>	指定导入目标表中的哪些 partition。如果不指定，会自动导入对应的 partition 中。如希望导入目标表 p1 与 p2 分区，可以使用以下命令：PARTITION(p1, p2)
DELETE ON	<delete_expr>	在 MERGE 导入模式下，使用 delete_expr 标记哪些列需要被删除。如需要在 MERGE 时删除 age >30 的列，可以使用以下命令：DELETE ON age >30

子模块	参数	说明
ORDER BY	`<order_by_column>`	仅针对 Unique Key 模型生效。用于指定导入数据中的 Sequence Column 列，以保证数据的顺序。如在 Unique Key 表导入时，需要指定导入的 Sequence Column 为 create_time，可以使用以下命令： `ORDER BY create_time`针对与 Unique Key 模型 Sequence Column 列的描述，可以参考文档数据更新/Sequence 列

job_properties 子句

在创建 Routine Load 导入作业时，可以指定 job_properties 子句以指定导入作业的属性。语法如下：

```
PROPERTIES ("`<key1>`" = "`<value1>`"[, "`<key2>`" = "`<value2>`" ...])
```

job_properties 子句具体参数选项如下：

参数	说明
desired_concurrent_number	默认值：256 参数描述：单个导入子任务（load task）期望的并发度，修改 Routine Load 导入作业切分的期望导入子任务数量。在导入过程中，期望的子任务并发度可能不等于实际并发度。实际的并发度会根据集群的节点数、负载情况，以及数据源的情况综合考虑，使用公式以下可以计算出实际的导入子任务数： `` min(topic_partition_num, desired_concurrent_number, max_routine_load_task_concurrent_num)，其中： - topic_partition_num 表示 Kafka Topic 的 paritition 数量 - desired_concurrent_number 表示设置的参数大小 - max_routine_load_task_concurrent_num 为 FE 中设置 Routine Load 最大任务并行度的参数
max_batch_interval	每个子任务的最大运行时间，单位是秒，必须大于 0，默认值为 60(s)。max_batch_interval/max_batch_rows/max_batch_size 共同形成子任务执行阈值。任一参数达到阈值，导入子任务结束，并生成新的导入子任务。
max_batch_rows	每个子任务最多读取的行数。必须大于等于 200000。默认是 20000000。max_batch_interval/max_batch_rows/max_batch_size 共同形成子任务执行阈值。任一参数达到阈值，导入子任务结束，并生成新的导入子任务。
max_batch_size	每个子任务最多读取的字节数。单位是字节，范围是 100MB 到 1GB。默认是 1G。max_batch_interval/max_batch_rows/max_batch_size 共同形成子任务执行阈值。任一参数达到阈值，导入子任务结束，并生成新的导入子任务。
max_error_number	采样窗口内，允许的最大错误行数。必须大于等于 0。默认是 0，即不允许有错误行。采样窗口为 `max_batch_rows * 10`。即如果在采样窗口内，错误行数大于 `max_error_number`，则会导致例行作业被暂停，需要人工介入检查数据质量问题，通过SHOW ROUTINE LOAD 命令中 `ErrorLogUrls` 检查数据的质量问题。被 where 条件过滤掉的行不算错误行。

参数	说明
strict_mode	是否开启严格模式，默认为关闭。严格模式表示对于导入过程中的列类型转换进行严格过滤。如果开启后，非空原始数据的列类型变换如果结果为 NULL，则会被过滤。 严格模式过滤策略如下： - 某衍生列（由函数转换生成而来），Strict Mode 对其不产生影响 - 当列类型需要转换，错误的数据类型将被过滤掉，在SHOW ROUTINE LOAD 的 ErrorLogUrls 中查看因为数据类型错误而被过滤掉的列 - 对于导入的某列类型包含范围限制的，如果原始数据能正常通过类型转换，但无法通过范围限制的，strict mode 对其也不产生影响。例如：如果类型是 decimal(1,0), 原始数据为 10，则属于可以通过类型转换但不在列声明的范围内。这种数据 strict 对其不产生影响。详细内容参考严格模式。
timezone	指定导入作业所使用的时区。默认为使用 Session 的 timezone 参数。该参数会影响所有导入涉及的和时区有关的函数结果。
format	指定导入数据格式，默认是 CSV，支持 JSON 格式。
jsonpaths	当导入数据格式为 JSON 时，可以通过 jsonpaths 指定抽取 JSON 数据中的字段。例如通过以下命令指定导入 jsonpaths：`"jsonpaths" = "[\"$.userid\",\"$.username\",\"$.age\",\"$.city\"]"`
json_root	当导入数据格式为 JSON 时，可以通过 json_root 指定 JSON 数据的根节点。Doris 将通过 json_root 抽取根节点的元素进行解析。默认为空。例如通过一下命令指定导入 JSON 根节点：`"json_root" = "$.RECORDS"`
strip_outer_array	当导入数据格式为 json 时，strip_outer_array 为 true 表示 JSON 数据以数组的形式展现，数据中的每一个元素将被视为一行数据。默认值是 false。通常情况下，Kafka 中的 JSON 数据可能以数组形式表示，即在最外层中包含中括号[]，此时，可以指定 `"strip_outer_array" = "true"`，以数组模式消费 Topic 中的数据。如以下数据会被解析成两行：`[{"user_id":1,"name":"Emily","age":25},{"user_id":2,"name":"Benjamin","age":35}]`
send_batch_parallelism	用于设置发送批量数据的并行度。如果并行度的值超过 BE 配置中的 `max_send_batch_parallelism_per_job`，那么作为协调点的 BE 将使用 `max_send_batch_parallelism_per_job` 的值。
load_to_single_tablet	支持一个任务只导入数据到对应分区的一个 tablet，默认值为 false，该参数只允许在对带有 random 分桶的 olap 表导数的时候设置。
partial_columns	指定是否开启部分列更新功能。默认值为 false。该参数只允许在表模型为 Unique 且采用 Merge on Write 时设置。一流多表不支持此参数。具体参考文档部分列更新
max_filter_ratio	采样窗口内，允许的最大过滤率。必须在大于等于 0 到小于等于 1 之间。默认值是 1.0，表示可以容忍任何错误行。采样窗口为 `max_batch_rows * 10`。即如果在采样窗口内，错误行数/总行数大于 `max_filter_ratio`，则会导致例行作业被暂停，需要人工介入检查数据质量问题。被 where 条件过滤掉的行不算错误行。
enclose	指定包围符。当 CSV 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为“,”，包围符为“‘“，数据为”a,’b,c’”，则“b,c”会被解析为一个字段。

参数	说明
escape	指定转义符。用于转义在字段中出现的与包围符相同的字符。例如数据为“a,‘b,’c’ ”，包围符为“ ‘ “，希望”b,’c 被作为一个字段解析，则需要指定单字节转义符，例如 “”，将数据修改为”a,’b,’c’ ”。

04 data_source_properties 子句

在创建 Routine Load 导入作业时，可以指定 data_source_properties 子句以指定 Kafka 数据源的属性。语法如下：

```
FROM KAFKA ("`<key1>`" = "`<value1>`"[, "`<key2>`" = "`<value2>`" ...])
```

data_source_properties 子句具体参数选项如下：

参数	说明
kafka_broker_list	指定 Kafka 的 broker 连接信息。格式为 <kafka_broker_ip>`:`<kafka port>。多个 broker 之间以逗号分隔。例如在 Kafka Broker 中默认端口号为 9092，可以使用以下命令指定 Broker List：`"kafka_broker_list" = ":9092,:9092"`
kafka_topic	指定要订阅的 Kafka 的 topic。一个导入作业仅能消费一个 Kafka Topic。
kafka_partitions	指定需要订阅的 Kafka Partition。如果不指定，则默认消费所有分区。
kafka_offsets	待消费的 Kakfa Partition 中起始消费点（offset）。如果指定时间，则会从大于等于该时间的最近一个 offset 处开始消费。offset 可以指定从大于等于 0 的具体 offset，也可以使用以下格式： - OFFSET_BEGINNING: 从有数据的位置开始订阅。 - OFFSET_END: 从末尾开始订阅。 - 时间格式，如：“2021-05-22 11:00:00” 如果没有指定，则默认从 OFFSET_END 开始订阅 topic 下的所有 partition。 可以指定多个其实消费点，使用逗号分隔，如：`"kafka_offsets" = "101,0,OFFSET_BEGINNING,OFFSET_END"`或者`"kafka_offsets" = "2021-05-22 11:00:00,2021-05-22 11:00:00"` 注意，时间格式不能和 OFFSET 格式混用。
property	指定自定义 kafka 参数。功能等同于 kafka shell 中“–property”参数。当参数的 Value 为一个文件时，需要在 Value 前加上关键词：“FILE:”。创建文件可以参考CREATE FILE 命令文档。更多支持的自定义参数，可以参考 librdkafka 的官方 CONFIGURATION 文档中，client 端的配置项。如：`"property.client.id" = "12345"`,`"property.group.id" = "group_id_0"`,`"property.ssl.ca.location" = "FILE:ca.pem"`

通过配置 data_source_properties 中的 kafka property 参数，可以配置安全访问选项。目前 Doris 支持多种 Kafka 安全协议，如 plaintext（默认）、SSL、PLAIN、Kerberos 等。

导入状态

通过 SHOW ROUTINE LOAD 命令可以查看导入作业的状态，具体语法如下：

```
SHOW [ALL] ROUTINE LOAD [FOR jobName];
```

如通过 SHOW ROUTINE LOAD 会返回以下结果集示例：

```
mysql>` SHOW ROUTINE LOAD FOR testdb.example_routine_load`\G`
*************************** 1. row ***************************
                  Id: 12025
                Name: example_routine_load
          CreateTime: 2024-01-15 08:12:42
           PauseTime: NULL
             EndTime: NULL
              DbName: default_cluster:testdb
           TableName: test_routineload_tbl
        IsMultiTable: false
               State: RUNNING
      DataSourceType: KAFKA
      CurrentTaskNum: 1
       JobProperties: {"max_batch_rows":"200000","timezone":"America/New_York","send_batch_
           ↪ parallelism":"1","load_to_single_tablet":"false","column_separator":"',","line_
           ↪ delimiter":"`\n`","current_concurrent_number":"1","delete":"*","partial_columns":"
           ↪ false","merge_type":"APPEND","exec_mem_limit":"2147483648","strict_mode":"false","
           ↪ jsonpaths":"","max_batch_interval":"10","max_batch_size":"104857600","fuzzy_parse"
           ↪ :"false","partitions":"*","columnToColumnExpr":"user_id,name,age","whereExpr":"*",
           ↪ "desired_concurrent_number":"5","precedingFilter":"*","format":"csv","max_error_
           ↪ number":"0","max_filter_ratio":"1.0","json_root":"","strip_outer_array":"false","
           ↪ num_as_string":"false"}
DataSourceProperties: {"topic":"test-topic","currentKafkaPartitions":"0","brokerList":"
    ↪ 192.168.88.62:9092"}
    CustomProperties: {"kafka_default_offsets":"OFFSET_BEGINNING","group.id":"example_routine_
        ↪ load_73daf600-884e-46c0-a02b-4e49fdf3b4dc"}
           Statistic: {"receivedBytes":28,"runningTxns":[],"errorRows":0,"committedTaskNum":3,"
               ↪ loadedRows":3,"loadRowsRate":0,"abortedTaskNum":0,"errorRowsAfterResumed":0,"
               ↪ totalRows":3,"unselectedRows":0,"receivedBytesRate":0,"taskExecuteTimeMs"
               ↪ :30069}
            Progress: {"0":"2"}
                 Lag: {"0":0}
ReasonOfStateChanged:
        ErrorLogUrls:
            OtherMsg:
                User: root
             Comment:
1 row in set (0.00 sec)
```

具体显示结果说明如下：

结果列	列说明
Id	作业 ID。由 Doris 自动生成。
Name	作业名称。

结果列	列说明
CreateTime	作业创建时间。
PauseTime	最近一次作业暂停时间。
EndTime	作业结束时间。
DbName	对应数据库名称
TableName	对应表名称。多表的情况下由于是动态表，因此不显示具体表名，会显示 multi-table。
IsMultiTbl	是是否为多表
State	作业运行状态，有 5 种状态： - NEED_SCHEDULE：作业等待被调度。在 CREATE ROUTINE LOAD 或 RESUME ROUTINE LOAD 后，作业会先进入到 NEED_SCHEDULE 状态； - RUNNING：作业运行中； - PAUSED：作业被暂停，可以通过 RESUME ROUTINE LOAD 恢复导入作业； - STOPPED：作业已结束，无法被重启； - CANCELLED：作业已取消。
DataSourceType	数据源类型：KAFKA。
CurrentTaskNum	当前子任务数量。
JobProperties	作业配置详情。
DataSourceProperties	数据源配置详情。
CustomProperties	自定义配置。
Statistic	作业运行状态统计信息。
Progress	作业运行进度。对于 Kafka 数据源，显示每个分区当前已消费的 offset。如 {"0":"2"} 表示 Kafka 分区 0 的消费进度为 2。
Lag	作业延迟状态。对于 Kafka 数据源，显示每个分区的消费延迟。如 {"0":10} 表示 Kafka 分区 0 的消费延迟为 10。
ReasonOfStateChanged	作业状态变更的原因
ErrorLogUrls	被过滤的质量不合格的数据的查看地址
OtherMsg	其他错误信息

2.9.3.3.5 导入示例

设置导入最大容错率

1. 导入数据样例

```
1,Benjamin,18
2,Emily,20
3,Alexander,dirty_data
```

2. 建表结构

```
CREATE TABLE demo.routine_test01 (
    id       INT             NOT NULL    COMMENT "User ID",
    name     VARCHAR(30)     NOT NULL    COMMENT "Name",
    age      INT                         COMMENT "Age"
```

```
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job01 ON routine_test01
        COLUMNS TERMINATED BY ","
        PROPERTIES
        (
            "max_filter_ratio"="0.5",
            "max_error_number" = "100",
            "strict_mode" = "true"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad01",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test01;
+------+------------+------+
| id   | name       | age  |
+------+------------+------+
|    1 | Benjamin   |   18 |
|    2 | Emily      |   20 |
+------+------------+------+
2 rows in set (0.01 sec)
```

从指定消费点消费数据

1. 导入数据样例

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
4,Sophia,24
5,William,26
6,Charlotte,28
```

2. 建表结构

```
CREATE TABLE demo.routine_test02 (
    id       INT             NOT NULL    COMMENT "User ID",
    name     VARCHAR(30)     NOT NULL    COMMENT "Name",
    age      INT                         COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job02 ON routine_test02
        COLUMNS TERMINATED BY ","
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad02",
            "kafka_partitions" = "0",
            "kafka_offsets" = "3"
        );
```

4. 导入结果

```
mysql>` select * from routine_test02;
+------+--------------+------+
| id   | name         | age  |
+------+--------------+------+
|    4 | Sophia       |   24 |
|    5 | William      |   26 |
|    6 | Charlotte    |   28 |
+------+--------------+------+
3 rows in set (0.01 sec)
```

指定 Consumer Group 的 group.id 与 client.id

1. 导入数据样例

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
```

2. 建表结构

```
CREATE TABLE demo.routine_test03 (
    id       INT             NOT NULL    COMMENT "User ID",
    name     VARCHAR(30)     NOT NULL    COMMENT "Name",
    age      INT                         COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job03 ON routine_test03
        COLUMNS TERMINATED BY ","
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad01",
            "property.group.id" = "kafka_job03",
            "property.client.id" = "kafka_client_03",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test03;
+------+------------+------+
| id   | name       | age  |
+------+------------+------+
|    1 | Benjamin   |   18 |
|    2 | Emily      |   20 |
|    3 | Alexander  |   22 |
+------+------------+------+
3 rows in set (0.01 sec)
```

设置导入过滤条件

1. 导入数据样例

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
4,Sophia,24
5,William,26
6,Charlotte,28
```

2. 建表结构

```
CREATE TABLE demo.routine_test04 (
    id       INT             NOT NULL    COMMENT "User ID",
    name     VARCHAR(30)     NOT NULL    COMMENT "Name",
    age      INT                         COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job04 ON routine_test04
        COLUMNS TERMINATED BY ",",
        WHERE id >`= 3
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad04",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test04;
+------+--------------+------+
| id   | name         | age  |
+------+--------------+------+
|    4 | Sophia       |   24 |
|    5 | William      |   26 |
|    6 | Charlotte    |   28 |
+------+--------------+------+
3 rows in set (0.01 sec)
```

导入指定分区数据

1. 导入数据样例

```
1,Benjamin,18,2024-02-04 10:00:00
2,Emily,20,2024-02-05 11:00:00
3,Alexander,22,2024-02-06 12:00:00
```

2. 建表结构

```
CREATE TABLE demo.routine_test05 (
    id      INT             NOT NULL  COMMENT "ID",
    name    VARCHAR(30)     NOT NULL  COMMENT "Name",
    age     INT                       COMMENT "Age",
    date    DATETIME                  COMMENT "Date"
)
DUPLICATE KEY(`id`)
PARTITION BY RANGE(`id`)
(PARTITION partition_a VALUES [("0"), ("1")),
PARTITION partition_b VALUES [("1"), ("2")),
PARTITION partition_c VALUES [("2"), ("3")))
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job05 ON routine_test05
        COLUMNS TERMINATED BY ",",
        PARTITION(partition_b)
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad05",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test05;
+------+----------+------+---------------------+
| id   | name     | age  | date                |
+------+----------+------+---------------------+
|    1 | Benjamin |   18 | 2024-02-04 10:00:00 |
+------+----------+------+---------------------+
1 rows in set (0.01 sec)
```

设置导入时区

1. 导入数据样例

```
1,Benjamin,18,2024-02-04 10:00:00
2,Emily,20,2024-02-05 11:00:00
3,Alexander,22,2024-02-06 12:00:00
```

2. 建表结构

```
CREATE TABLE demo.routine_test06 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age",
    date    DATETIME                 COMMENT "date"
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job06 ON routine_test06
        COLUMNS TERMINATED BY ","
        PROPERTIES
        (
            "timezone" = "Asia/Shanghai"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad06",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test06;
+------+-------------+------+---------------------+
| id   | name        | age  | date                |
+------+-------------+------+---------------------+
|    1 | Benjamin    |   18 | 2024-02-04 10:00:00 |
|    2 | Emily       |   20 | 2024-02-05 11:00:00 |
|    3 | Alexander   |   22 | 2024-02-06 12:00:00 |
+------+-------------+------+---------------------+
3 rows in set (0.00 sec)
```

设置 merge_type

指定 merge_type 进行 delete 操作

1. 导入数据样例

```
3,Alexander,22
5,William,26
```

导入前表中数据如下

```
mysql>` SELECT * FROM routine_test07;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
|    2 | Emily          |   20 |
|    3 | Alexander      |   22 |
|    4 | Sophia         |   24 |
|    5 | William        |   26 |
|    6 | Charlotte      |   28 |
+------+----------------+------+
```

2. 建表结构

```
CREATE TABLE demo.routine_test07 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age"
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job07 ON routine_test07
        WITH DELETE
        COLUMNS TERMINATED BY ","
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad07",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` SELECT * FROM routine_test07;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
|    2 | Emily          |   20 |
|    4 | Sophia         |   24 |
|    6 | Charlotte      |   28 |
+------+----------------+------+
```

指定 merge_typpe 进行 merge 操作

1. 导入数据样例

```
1,xiaoxiaoli,28
2,xiaoxiaowang,30
3,xiaoxiaoliu,32
4,dadali,34
5,dadawang,36
6,dadaliu,38
```

导入前表中数据如下：

```
mysql>` SELECT * FROM routine_test08;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
|    2 | Emily          |   20 |
|    3 | Alexander      |   22 |
|    4 | Sophia         |   24 |
|    5 | William        |   26 |
|    6 | Charlotte      |   28 |
+------+----------------+------+
6 rows in set (0.01 sec)
```

2. 建表结构

```
CREATE TABLE demo.routine_test08 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age"
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job08 ON routine_test08
        WITH MERGE
        COLUMNS TERMINATED BY ",",
        DELETE ON id = 2
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad08",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` SELECT * FROM routine_test08;
+------+-------------+------+
| id   | name        | age  |
+------+-------------+------+
|    1 | xiaoxiaoli  |   28 |
|    3 | xiaoxiaoliu |   32 |
|    4 | dadali      |   34 |
|    5 | dadawang    |   36 |
|    6 | dadaliu     |   38 |
+------+-------------+------+
5 rows in set (0.00 sec)
```

指定导入需要 merge 的 sequence 列

1. 导入数据样例

```
1,xiaoxiaoli,28
2,xiaoxiaowang,30
3,xiaoxiaoliu,32
4,dadali,34
5,dadawang,36
6,dadaliu,38
```

导入前表中数据如下：

```
mysql>` SELECT * FROM routine_test09;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
```

```
|    2 | Emily          |   20 |
|    3 | Alexander      |   22 |
|    4 | Sophia         |   24 |
|    5 | William        |   26 |
|    6 | Charlotte      |   28 |
+------+----------------+------+
6 rows in set (0.01 sec)
```

2. 建表结构

```
CREATE TABLE demo.routine_test08 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age",
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "function_column.sequence_col" = "age"
);
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job09 ON routine_test09
        WITH MERGE
        COLUMNS TERMINATED BY ",",
        COLUMNS(id, name, age),
        DELETE ON id = 2,
        ORDER BY age
        PROPERTIES
        (
            "desired_concurrent_number"="1",
            "strict_mode" = "false"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad09",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` SELECT * FROM routine_test09;
+------+-------------+------+
| id   | name        | age  |
+------+-------------+------+
|    1 | xiaoxiaoli  |   28 |
|    3 | xiaoxiaoliu |   32 |
|    4 | dadali      |   34 |
|    5 | dadawang    |   36 |
|    6 | dadaliu     |   38 |
+------+-------------+------+
5 rows in set (0.00 sec)
```

导入完成列影射与衍生列计算

1. 导入数据样例

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
```

2. 建表结构

```
CREATE TABLE demo.routine_test10 (
    id      INT             NOT NULL  COMMENT "id",
    name    VARCHAR(30)     NOT NULL  COMMENT "name",
    age     INT                       COMMENT "age",
    num     INT                       COMMENT "number"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job10 ON routine_test10
        COLUMNS TERMINATED BY ",",
        COLUMNS(id, name, age, num=age*10)
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad10",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` SELECT * FROM routine_test10;
+------+----------------+------+------+
| id   | name           | age  | num  |
+------+----------------+------+------+
|    1 | Benjamin       |   18 |  180 |
|    2 | Emily          |   20 |  200 |
|    3 | Alexander      |   22 |  220 |
+------+----------------+------+------+
3 rows in set (0.01 sec)
```

导入包含包围符的数据

1. 导入数据样例

```
1,"Benjamin",18
2,"Emily",20
3,"Alexander",22
```

2. 建表结构

```
CREATE TABLE demo.routine_test11 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age",
    num     INT                      COMMENT "number"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job11 ON routine_test11
        COLUMNS TERMINATED BY ","
        PROPERTIES
        (
            "desired_concurrent_number"="1",
            "enclose" = "\""
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
```

```
            "kafka_topic" = "routineLoad12",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` SELECT * FROM routine_test11;
+------+----------------+------+------+
| id   | name           | age  | num  |
+------+----------------+------+------+
|    1 | Benjamin       |   18 |  180 |
|    2 | Emily          |   20 |  200 |
|    3 | Alexander      |   22 |  220 |
+------+----------------+------+------+
3 rows in set (0.02 sec)
```

JSON 格式导入

以简单模式导入 JSON 格式数据

1. 导入数据样例

```
{ "id" : 1, "name" : "Benjamin", "age":18 }
{ "id" : 2, "name" : "Emily", "age":20 }
{ "id" : 3, "name" : "Alexander", "age":22 }
```

2. 建表结构

```
CREATE TABLE demo.routine_test12 (
    id      INT             NOT NULL  COMMENT "id",
    name    VARCHAR(30)     NOT NULL  COMMENT "name",
    age     INT                       COMMENT "age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job12 ON routine_test12
        PROPERTIES
        (
            "format" = "json"
        )
```

```
FROM KAFKA
(
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad12",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果

```
mysql>` select * from routine_test12;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
|    2 | Emily          |   20 |
|    3 | Alexander      |   22 |
+------+----------------+------+
3 rows in set (0.02 sec)
```

匹配模式导入复杂的JSON 格式数据

1. 导入数据样例

```
{ "name" : "Benjamin", "id" : 1, "num":180 , "age":18 }
{ "name" : "Emily", "id" : 2, "num":200 , "age":20 }
{ "name" : "Alexander", "id" : 3, "num":220 , "age":22 }
```

2. 建表结构

```
CREATE TABLE demo.routine_test13 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age",
    num     INT                      COMMENT "num"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job13 ON routine_test13
        COLUMNS(name, id, num, age)
        PROPERTIES
        (
            "format" = "json",
            "jsonpaths" = "[\"$.name\",\"$.id\",\"$.num\",\"$.age\"]"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad13",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test13;
+------+----------------+------+------+
| id   | name           | age  | num  |
+------+----------------+------+------+
|    1 | Benjamin       |   18 |  180 |
|    2 | Emily          |   20 |  200 |
|    3 | Alexander      |   22 |  220 |
+------+----------------+------+------+
3 rows in set (0.01 sec)
```

指定 JSON 根节点导入数据

1. 导入数据样例

```
{"id": 1231, "source" :{ "id" : 1, "name" : "Benjamin", "age":18 }}
{"id": 1232, "source" :{ "id" : 2, "name" : "Emily", "age":20 }}
{"id": 1233, "source" :{ "id" : 3, "name" : "Alexander", "age":22 }}
```

2. 建表结构

```
CREATE TABLE demo.routine_test14 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job14 ON routine_test14
        PROPERTIES
        (
            "format" = "json",
            "json_root" = "$.source"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad14",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test14;
+------+----------------+------+
| id   | name           | age  |
+------+----------------+------+
|    1 | Benjamin       |   18 |
|    2 | Emily          |   20 |
|    3 | Alexander      |   22 |
+------+----------------+------+
3 rows in set (0.01 sec)
```

导入完成列影射与衍生列计算

1. 导入数据样例

```
{ "id" : 1, "name" : "Benjamin", "age":18 }
{ "id" : 2, "name" : "Emily", "age":20 }
{ "id" : 3, "name" : "Alexander", "age":22 }
```

2. 建表结构

```
CREATE TABLE demo.routine_test15 (
    id      INT            NOT NULL  COMMENT "id",
    name    VARCHAR(30)    NOT NULL  COMMENT "name",
    age     INT                      COMMENT "age",
    num     INT                      COMMENT "num"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job15 ON routine_test15
        COLUMNS(id, name, age, num=age*10)
        PROPERTIES
        (
            "format" = "json",
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad15",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test15;
+------+----------------+------+------+
| id   | name           | age  | num  |
+------+----------------+------+------+
|    1 | Benjamin       |   18 |  180 |
|    2 | Emily          |   20 |  200 |
|    3 | Alexander      |   22 |  220 |
+------+----------------+------+------+
3 rows in set (0.01 sec)
```

导入复杂类型

导入 Array 数据类型

1. 导入数据样例

```
{ "id" : 1, "name" : "Benjamin", "age":18, "array":[1,2,3,4,5]}
{ "id" : 2, "name" : "Emily", "age":20, "array":[6,7,8,9,10]}
{ "id" : 3, "name" : "Alexander", "age":22, "array":[11,12,13,14,15]}
```

2. 建表结构

```
CREATE TABLE demo.routine_test16
(
    id      INT             NOT NULL  COMMENT "id",
    name    VARCHAR(30)     NOT NULL  COMMENT "name",
    age     INT                       COMMENT "age",
```

```
    array   ARRAY`<int(11)>`  NULL      COMMENT "test array column"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job16 ON routine_test16
        PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad16",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test16;
+------+----------------+------+----------------------+
| id   | name           | age  | array                |
+------+----------------+------+----------------------+
|    1 | Benjamin       |   18 | [1, 2, 3, 4, 5]      |
|    2 | Emily          |   20 | [6, 7, 8, 9, 10]     |
|    3 | Alexander      |   22 | [11, 12, 13, 14, 15] |
+------+----------------+------+----------------------+
3 rows in set (0.00 sec)
```

导入 Map 数据类型

1. 导入数据样例

```
{ "id" : 1, "name" : "Benjamin", "age":18, "map":{"a": 100, "b": 200}}
{ "id" : 2, "name" : "Emily", "age":20, "map":{"c": 300, "d": 400}}
{ "id" : 3, "name" : "Alexander", "age":22, "map":{"e": 500, "f": 600}}
```

2. 建表结构

```
CREATE TABLE demo.routine_test17 (
    id      INT                 NOT NULL  COMMENT "id",
    name    VARCHAR(30)         NOT NULL  COMMENT "name",
    age     INT                           COMMENT "age",
    map     Map`<STRING, INT>`    NULL      COMMENT "test column"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job17 ON routine_test17
    PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad17",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test17;
+------+----------------+------+--------------------+
| id   | name           | age  | map                |
+------+----------------+------+--------------------+
|    1 | Benjamin       |   18 | {"a":100, "b":200} |
|    2 | Emily          |   20 | {"c":300, "d":400} |
|    3 | Alexander      |   22 | {"e":500, "f":600} |
+------+----------------+------+--------------------+
3 rows in set (0.01 sec)
```

导入 Bitmap 数据类型

1. 导入数据样例

```
{ "id" : 1, "name" : "Benjamin", "age":18, "bitmap_id":243}
{ "id" : 2, "name" : "Emily", "age":20, "bitmap_id":28574}
{ "id" : 3, "name" : "Alexander", "age":22, "bitmap_id":8573}
```

2. 建表结构

```
CREATE TABLE demo.routine_test18 (
    id        INT            NOT NULL      COMMENT "id",
    name      VARCHAR(30)    NOT NULL      COMMENT "name",
    age       INT                          COMMENT "age",
    bitmap_id INT                          COMMENT "test",
    device_id BITMAP         BITMAP_UNION  COMMENT "test column"
)
AGGREGATE KEY (`id`,`name`,`age`,`bitmap_id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job18 ON routine_test18
        COLUMNS(id, name, age, bitmap_id, device_id=to_bitmap(bitmap_id))
        PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad18",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select id, BITMAP_UNION_COUNT(pv) over(order by id) uv from(
    ->`    select id, BITMAP_UNION(device_id) as pv
    ->`    from routine_test18
    ->` group by id
    ->` ) final;
+------+------+
| id   | uv   |
+------+------+
|    1 |    1 |
|    2 |    2 |
|    3 |    3 |
+------+------+
3 rows in set (0.00 sec)
```

导入 HLL 数据类型

1. 导入数据样例

```
2022-05-05,10001,Test01,Beijing,windows
2022-05-05,10002,Test01,Beijing,linux
2022-05-05,10003,Test01,Beijing,macos
2022-05-05,10004,Test01,Hebei,windows
2022-05-06,10001,Test01,Shanghai,windows
2022-05-06,10002,Test01,Shanghai,linux
2022-05-06,10003,Test01,Jiangsu,macos
2022-05-06,10004,Test01,Shaanxi,windows
```

2. 建表结构

```
create table demo.routine_test19 (
    dt        DATE,
    id        INT,
    name      VARCHAR(10),
    province  VARCHAR(10),
    os        VARCHAR(10),
    pv        hll hll_union
)
Aggregate KEY (dt,id,name,province,os)
distributed by hash(id) buckets 10;
```

3. 导入命令

```
CREATE ROUTINE LOAD demo.kafka_job19 ON routine_test19
        COLUMNS TERMINATED BY ",",
        COLUMNS(dt, id, name, province, os, pv=hll_hash(id))
        FROM KAFKA
        (
            "kafka_broker_list" = "10.16.10.6:9092",
            "kafka_topic" = "routineLoad19",
            "property.kafka_default_offsets" = "OFFSET_BEGINNING"
        );
```

4. 导入结果

```
mysql>` select * from routine_test19;
+------------+-------+----------+----------+---------+------+
| dt         | id    | name     | province | os      | pv   |
+------------+-------+----------+----------+---------+------+
| 2022-05-05 | 10001 | Test01   | Beijing     | windows | NULL |
```

```
| 2022-05-06 | 10001 | Test01   | Shanghai    | windows | NULL |
| 2022-05-05 | 10002 | Test01   | Beijing     | linux   | NULL |
| 2022-05-06 | 10002 | Test01   | Shanghai    | linux   | NULL |
| 2022-05-05 | 10004 | Test01   | Heibei      | windows | NULL |
| 2022-05-06 | 10004 | Test01   | Shanxi      | windows | NULL |
| 2022-05-05 | 10003 | Test01   | Beijing     | macos   | NULL |
| 2022-05-06 | 10003 | Test01   | Jiangsu     | macos   | NULL |
+------------+-------+----------+-------------+---------+------+
8 rows in set (0.01 sec)

mysql>` SELECT HLL_UNION_AGG(pv) FROM routine_test19;
+--------------------+
| hll_union_agg(pv) |
+--------------------+
|                  4 |
+--------------------+
1 row in set (0.01 sec)
```

Kafka 安全认证

导入 SSL 认证的 Kafka 数据

导入命令样例：

```
CREATE ROUTINE LOAD demo.kafka_job20 ON routine_test20
        PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "192.168.100.129:9092",
            "kafka_topic" = "routineLoad21",
            "property.security.protocol" = "ssl",
            "property.ssl.ca.location" = "FILE:ca.pem",
            "property.ssl.certificate.location" = "FILE:client.pem",
            "property.ssl.key.location" = "FILE:client.key",
            "property.ssl.key.password" = "ssl_passwd"
        );
```

参数说明：

参数	介绍
property.security.protocol	使用的安全协议，如上述的例子使用的是 SSL
property.ssl.ca.location	CA（Certificate Authority）证书的位置
property.ssl.certificate.location	（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 public key 的位置

参数	介绍
property.ssl.key.location	（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 private key 的位置
property.ssl.key.password	（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 private key 的密码

导入 Kerberos 认证的 Kafka 数据

导入命令样例：

```
CREATE ROUTINE LOAD demo.kafka_job21 ON routine_test21
        PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
            "kafka_broker_list" = "192.168.100.129:9092",
            "kafka_topic" = "routineLoad21",
            "property.security.protocol" = "SASL_PLAINTEXT",
            "property.sasl.kerberos.service.name" = "kafka",
            "property.sasl.kerberos.keytab"="/opt/third/kafka/kerberos/kafka_client.keytab",
            "property.sasl.kerberos.principal" = "clients/stream.dt.local@EXAMPLE.COM"
        );
```

参数说明：

参数	介绍
property.security.protocol	使用的安全协议，如上述的例子使用的是 SASL_PLAINTEXT
property.sasl.kerberos.service.name	指定 broker service name，默认是 Kafka
property.sasl.kerberos.keytab	keytab 文件的位置
property.sasl.kerberos.principal	指定 kerberos principal

导入 PLAIN 认证的 Kafka 集群

1. 导入命令样例：

```
CREATE ROUTINE LOAD demo.kafka_job22 ON routine_test22
        PROPERTIES
        (
            "format" = "json"
        )
        FROM KAFKA
        (
```

```
            "kafka_broker_list" = "192.168.100.129:9092",
            "kafka_topic" = "routineLoad22",
            "property.security.protocol"="SASL_PLAINTEXT",
            "property.sasl.mechanism"="PLAIN",
            "property.sasl.username"="admin",
            "property.sasl.password"="admin"
        );
```

参数说明：

参数	介绍
property.security.protocol	使用的安全协议，如上述的例子使用的是 SASL_PLAINTEXT
property.sasl.mechanism	指定 SASL 认证机制为 PLAIN
property.sasl.username	SASL 的用户名
property.sasl.password	SASL 的密码

一流多表导入

为 example_db 创建一个名为 test1 的 Kafka 例行动态多表导入任务。指定列分隔符和 group.id 和 client.id，并且自动默认消费所有分区，且从有数据的位置（OFFSET_BEGINNING）开始订阅。

这里假设需要将 Kafka 中的数据导入到 example_db 中的 tbl1 以及 tbl2 表中，我们创建了一个名为 test1 的例行导入任务，同时将名为 my_topic 的 Kafka 的 Topic 数据同时导入到 tbl1 和 tbl2 中的数据中，这样就可以通过一个例行导入任务将 Kafka 中的数据导入到两个表中。

```
CREATE ROUTINE LOAD example_db.test1
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

这个时候需要 Kafka 中的数据包含表名的信息。目前仅支持从 Kafka 的 Value 中获取动态表名，且需要符合这种格式：以 JSON 为例：`table_name|{"col1": "val1", "col2": "val2"}`, 其中 `tbl_name` 为表名，以 `|` 作为表名和表数据的分隔符。CSV 格式的数据也是类似的，如：`table_name|val1,val2,val3`。注意，这里的 `table_name` 必须和 Doris 中的表名一致，否则会导致导入失败。注意，动态表不支持后面介绍的 column_mapping 配置。

严格模式导入

为 example_db 的 example_tbl 创建一个名为 test1 的 Kafka 例行导入任务。导入任务为严格模式。

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100),
PRECEDING FILTER k1 = 1,
WHERE k1 `< 100 and k2 like "%doris%"
PROPERTIES
(
```

```
    "strict_mode" = "true"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic"
);
```

2.9.3.3.6 连接加密认证的 Kafka 服务

这里我们以访问 StreamNative 消息服务为例说明：

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(user_id, name, age)
FROM KAFKA (
    "kafka_broker_list" = "pc-xxxx.aws-mec1-test-xwiqv.aws.snio.cloud:9093",
    "kafka_topic" = "my_topic",
    "property.security.protocol" = "SASL_SSL",
    "property.sasl.mechanism" = "PLAIN",
    "property.sasl.username" = "user",
    "property.sasl.password" = "token:eyJhbxxx",
    "property.group.id" = "my_group_id_1",
    "property.client.id" = "my_client_id_1",
    "property.enable.ssl.certificate.verification" = "false"
);
```

注意，如果没有在 BE 端配置信任的 CA 证书路径，需设置 "property.enable.ssl.certificate.verification" = "false"，不验证服务器证书是否可信。

否则，需配置信任的 CA 证书路径：`"property.ssl.ca.location" = "/path/to/ca-cert.pem"`。

2.9.3.3.7 更多帮助

参考 SQL 手册Routine Load。也可以在客户端命令行下输入 HELP ROUTINE LOAD 获取更多帮助信息。

2.9.3.4 Insert Into Select

INSERT INTO 支持将 Doris 查询的结果导入到另一个表中。INSERT INTO 是一个同步导入方式，执行导入后返回导入结果。可以通过请求的返回判断导入是否成功。INSERT INTO 可以保证导入任务的原子性，要么全部导入成功，要么全部导入失败。

2.9.3.4.1 使用场景

1. 用户希望将已经在 Doris 表中的数据进行 ETL 转换并导入到一个新的 Doris 表中，此时适合使用 INSERT INTO SELECT 语法。

2. 与 Multi-Catalog 外部表机制进行结合，如通过 Multi-Catalog 映射 MySQL 或者 Hive 系统中的表，然后通过 INSERT INTO SELECT 语法将外部表中的数据导入到 Doris 表中存储。
3. 通过 Table Value Function（TVF）功能，可以直接将对象存储或 HDFS 上的文件作为 Table 进行查询，并且支持自动的列类型推断。然后，通过 INSERT INTO SELECT 语法将外部表中的数据导入到 Doris 表中存储。

2.9.3.4.2 基本原理

在使用 INSERT INTO 时，需要通过 MySQL 协议发起导入作业给 FE 节点，FE 会生成执行计划，执行计划中前部是查询相关的算子，最后一个是 OlapTableSink 算子，用于将查询结果写到目标表中。执行计划会被发送给 BE 节点执行，Doris 会选定一个节点作为 Coordinator 节点，Coordinator 节点负责接受数据并分发数据到其他节点上。

2.9.3.4.3 快速上手

INSERT INTO 通过 MySQL 协议提交和传输。下例以 MySQL 命令行为例，演示通过 INSERT INTO 提交导入作业。

详细语法可以参见INSERT INTO。

前置检查

INSERT INTO 需要对目标表的 INSERT 权限。如果没有 INSERT 权限，可以通过GRANT 命令给用户授权。

创建导入作业

1. 创建源表

```
CREATE TABLE testdb.test_table(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

2. 使用任何方式向源表导入数据（这里以 INSERT INTO VALUES 为例）

```
INSERT INTO testdb.test_table (user_id, name, age)
VALUES (1, "Emily", 25),
       (2, "Benjamin", 35),
       (3, "Olivia", 28),
       (4, "Alexander", 60),
       (5, "Ava", 17);
```

3. 在上述操作的基础上，创建一个新表作为目标表（其 schema 与源表相同）

```
CREATE TABLE testdb.test_table2 LIKE testdb.test_table;
```

4. 使用 INSERT INTO SELECT 导入到新表

```
INSERT INTO testdb.test_table2
SELECT * FROM testdb.test_table WHERE age < 30;
Query OK, 3 rows affected (0.544 sec)
{'label':'label_9c2bae970023407d_b2c5b78b368e78a7', 'status':'VISIBLE', 'txnId':'9084'}
```

5. 查看导入数据

```
MySQL> SELECT * FROM testdb.test_table2 ORDER BY age;
+---------+--------+------+
| user_id | name   | age  |
+---------+--------+------+
|       5 | Ava    |   17 |
|       1 | Emily  |   25 |
|       3 | Olivia |   28 |
+---------+--------+------+
3 rows in set (0.02 sec)
```

6. 可以使用JOB 异步执行 INSERT。

7. 数据源可以是tvf 或者catalog 中的表。

查看导入作业

可以通过 show load 命令查看已完成的 INSERT INTO 任务。

```
MySQL> SHOW LOAD FROM testdb;
+--
    ↪ ------+----------------------------------------+----------+--------------------+--------+---------+-----
    ↪
| JobId  | Label                                  | State    | Progress           | Type   |
    ↪ EtlInfo | TaskInfo                                                       | ErrorMsg
    ↪  | CreateTime          | EtlStartTime        | EtlFinishTime       | LoadStartTime
    ↪ | LoadFinishTime      | URL  | JobDetails
    ↪
    ↪ | TransactionId | ErrorTablets | User | Comment |
+--
    ↪ ------+----------------------------------------+----------+--------------------+--------+---------+-----
    ↪
| 376416 | label_3e52da787aab4222_9126d2fce8f6d1e5 | FINISHED | Unknown id: 376416 | INSERT |
    ↪ NULL    | cluster:N/A; timeout(s):26200; max_filter_ratio:0.0; priority:NORMAL | NULL
    ↪     | 2024-02-27 01:22:17 | 2024-02-27 01:22:17 | 2024-02-27 01:22:17 | 2024-02-27
    ↪ 01:22:17 | 2024-02-27 01:22:18 |      | {"Unfinished backends":{},"ScannedRows":0,"
    ↪ TaskNumber":0,"LoadBytes":0,"All backends":{},"FileNumber":0,"FileSize":0} | 9081
    ↪          | {}           | root |         |
```

```
| 376664 | label_9c2bae970023407d_b2c5b78b368e78a7 | FINISHED | Unknown id: 376664 | INSERT |
    ↪ NULL    | cluster:N/A; timeout(s):26200; max_filter_ratio:0.0; priority:NORMAL | NULL
    ↪     | 2024-02-27 01:39:37 | 2024-02-27 01:39:37 | 2024-02-27 01:39:37 | 2024-02-27
    ↪ 01:39:37 | 2024-02-27 01:39:38 |      | {"Unfinished backends":{},"ScannedRows":0,"
    ↪ TaskNumber":0,"LoadBytes":0,"All backends":{},"FileNumber":0,"FileSize":0} | 9084
    ↪          | {}           | root |         |
+--
    ↪ ------+-----------------------------------------+----------+--------------------+--------+---------+-----
    ↪
```

取消导入作业

用户可以通过 Ctrl-C 取消当前正在执行的 INSERT INTO 作业。

2.9.3.4.4 参考手册

导入命令

INSERT INTO SELECT 一般用于将查询结果保存到目标表中。

```
INSERT INTO target_table SELECT ... FROM source_table;
```

其中 SELECT 语句同一般的 SELECT 查询语句，可以包含 WHERE JOIN 等操作。

导入配置参数

FE 配置

参数	默认值	描述
insert_load_default_timeout_second	14400（4 小时）	导入任务的超时时间，单位：秒。导入任务在该超时时间内未完成则会被系统取消，变成 CANCELLED ↪ 。

环境变量

参数	默认值	描述
insert_timeout	14400（4 小时）	INSERT INTO 作为 SQL 语句的的超时时间，单位：秒。

参数	默认值	描述
enable_insert_strict	true	如果设置为 true，当 INSERT INTO 遇到不合格数据时导入会失败。如果设置为 false，INSERT INTO 会忽略不合格的行，只要有一条数据被正确导入，导入就会成功。在 2.1.4 及以前的版本中。INSERT INTO 无法控制错误率，只能

参数	默认值	描述
insert_max_filter_ratio	1.0	自 2.1.5 版本，仅当 enable_insert_strict 值为 false 时生效。用于控制当使用 INSERT INTO FROM S3/HDFS/LOCAL() 时，设定错误容忍率的。默认为 1.0 表示容忍所有错误。可以取值 0 ~ 1 之间的小数

参数	默认值	描述

导入返回值

INSERT INTO 是一个 SQL 语句，其返回结果会根据查询结果的不同，分为以下几种：

结果集为空

如果 INSERT INTO 中的 SELECT 语句的查询结果集为空，则返回如下：

```
mysql> INSERT INTO tbl1 SELECT * FROM empty_tbl;
Query OK, 0 rows affected (0.02 sec)
```

`Query OK` 表示执行成功。`0 rows affected` 表示没有数据被导入。

结果集不为空且 INSERT 执行成功

```
mysql> INSERT INTO tbl1 SELECT * FROM tbl2;
Query OK, 4 rows affected (0.38 sec)
{'label':'INSERT_8510c568-9eda-4173-9e36-6adc7d35291c', 'status':'visible', 'txnId':'4005'}

mysql> INSERT INTO tbl1 WITH LABEL my_label1 SELECT * FROM tbl2;
Query OK, 4 rows affected (0.38 sec)
{'label':'my_label1', 'status':'visible', 'txnId':'4005'}

mysql> INSERT INTO tbl1 SELECT * FROM tbl2;
Query OK, 2 rows affected, 2 warnings (0.31 sec)
{'label':'INSERT_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'visible', 'txnId':'4005'}

mysql> INSERT INTO tbl1 SELECT * FROM tbl2;
Query OK, 2 rows affected, 2 warnings (0.31 sec)
{'label':'INSERT_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'committed', 'txnId':'4005'}
```

`Query OK` 表示执行成功。`4 rows affected` 表示总共有 4 行数据被导入。`2 warnings` 表示被过滤的行数。

同时会返回一个 JSON 串：

```
{'label':'my_label1', 'status':'visible', 'txnId':'4005'}
{'label':'INSERT_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'committed', 'txnId':'4005'}
{'label':'my_label1', 'status':'visible', 'txnId':'4005', 'err':'some other error'}
```

其中，返回结果参数如下表说明：

参数名称	说明
TxnId	导入事务的 ID
Label	导入作业的 label，通过 INSERT INTO tbl WITH LABEL label … 指定

| Status | 表示导入数据是否可见。如果可见，显示 visible，如果不可见，显示 committed

- visible：表示导入成功，数据可见
  - committed：该状态也表示导入已经完成，只是数据可能会延迟可见，无需重试
    * Label Already Exists：Label 重复，需要更换 label
      · Fail：导入失败
        Err | 导入错误信息 |

当需要查看被过滤的行时，用户可以通过SHOW LOAD语句

```
SHOW LOAD WHERE label="xxx";
```

返回结果中的 URL 可以用于查询错误的数据，具体见后面查看错误行小结。数据不可见是一个临时状态，这批数据最终是一定可见的。可以通过SHOW TRANSACTION语句查看这批数据的可见状态：

```
SHOW TRANSACTION WHERE id=4005;
```

返回结果中的 TransactionStatus 列如果为 visible，则表述数据可见。

```
{'label':'my_label1', 'status':'visible', 'txnId':'4005'}
{'label':'INSERT_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'committed', 'txnId':'4005'}
{'label':'my_label1', 'status':'visible', 'txnId':'4005', 'err':'some other error'}
```

结果集不为空但 INSERT 执行失败

执行失败表示没有任何数据被成功导入，并返回如下：

```
mysql> INSERT INTO tbl1 SELECT * FROM tbl2 WHERE k1 = "a";
ERROR 1064 (HY000): all partitions have no load data. url: http://10.74.167.16:8042/api/_load_
    ↪ error_log?file=_shard_2/error_loginsert_stmt_ba8bb9e158e4879-ae8de8507c0bf8a2_
    ↪ ba8bb9e158e4879_ae8de8507c0bf8a2
```

其中 ERROR 1064 (HY000): all partitions have no load data 显示失败原因。后面的 url 可以用于查询错误的数据，具体见后面查看错误行小结。

2.9.3.4.5 导入最佳实践

数据量

INSERT INTO 对数据量没有限制，大数据量导入也可以支持。但如果导入数据量过大，就需要通过以下配置修改系统的 INSERT INTO 导入超时时间，确保导入超时 >= 数据量 ``/`` 预估导入速度。

1. FE 配置参数insert_load_default_timeout_second。
2. 环境变量 insert_timeout。

查看错误行

当 INSERT INTO 返回结果中提供了 url 字段时，可以通过以下命令查看错误行：

```
SHOW LOAD WARNINGS ON "url";
```

示例：

```
SHOW LOAD WARNINGS ON "http://ip:port/api/_load_error_log?file=_shard_13/error_loginsert_stmt_
    ↪ d2cac0a0a16d482d-9041c949a4b71605_d2cac0a0a16d482d_9041c949a4b71605";
```

常见的错误的原因有：源数据列长度超过目的数据列长度、列类型不匹配、分区不匹配、列顺序不匹配等。

可以通过环境变量 enable_insert_strict来控制 INSERT INTO 是否忽略错误行。

2.9.3.4.6 通过外部表 Multi-Catalog 导入数据

Doris 可以创建外部表。创建完成后，可以通过 INSERT INTO SELECT 的方式导入外部表的数据，当然也可以通过 SELECT 语句直接查询外部表的数据，

Doris 通过多源数据目录（Multi-Catalog）功能，支持了包括 Apache Hive、Apache Iceberg、Apache Hudi、Apache Paimon(Incubating)、Elasticsearch、MySQL、Oracle、SQL Server 等主流数据湖、数据库的连接访问。

Multi-Catalog 相关功能，请查看湖仓一体文档。

这里以通过构建 Hive 外部表，导入其数据到 Doris 内部表来举例说明。

创建 Hive Catalog

```
CREATE CATALOG hive PROPERTIES (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://172.0.0.1:9083',
    'hadoop.username' = 'hive',
    'dfs.nameservices'='your-nameservice',
    'dfs.ha.namenodes.your-nameservice'='nn1,nn2',
    'dfs.namenode.rpc-address.your-nameservice.nn1'='172.21.0.2:8088',
    'dfs.namenode.rpc-address.your-nameservice.nn2'='172.21.0.3:8088',
    'dfs.client.failover.proxy.provider.your-nameservice'='org.apache.hadoop.hdfs.server.namenode
        ↪ .ha.ConfiguredFailoverProxyProvider'
);
```

导入数据

1. 创建一张 Doris 的导入目标表

```
CREATE TABLE `target_tbl` (
  `k1` decimal(9, 3) NOT NULL COMMENT "",
  `k2` char(10) NOT NULL COMMENT "",
  `k3` datetime NOT NULL COMMENT "",
  `k5` varchar(20) NOT NULL COMMENT "",
  `k6` double NOT NULL COMMENT ""
)
```

```
COMMENT "Doris Table"
DISTRIBUTED BY HASH(k1) BUCKETS 2
PROPERTIES (
    "replication_num" = "1"
);
```

2. 关于创建 Doris 表的详细说明，请参阅CREATE-TABLE 语法帮助。

3. 导入数据 (从 hive.db1.source_tbl 表导入到 target_tbl 表)

```
INSERT INTO target_tbl SELECT k1,k2,k3 FROM  hive.db1.source_tbl limit 100;
```

INSERT 命令是同步命令，返回成功，即表示导入成功。

注意事项

- 必须保证外部数据源与 Doris 集群是可以互通，包括 BE 节点和外部数据源的网络是互通的。

2.9.3.4.7 通过 TVF 导入数据

通过 Table Value Function 功能，Doris 可以直接将对象存储或 HDFS 上的文件作为 Table 进行查询分析、并且支持自动的列类型推断。详细介绍，请参考湖仓一体/TVF 文档。

自动推断文件列类型

```
DESC FUNCTION s3 (
    "URI" = "http://127.0.0.1:9312/test2/test.snappy.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "parquet",
    "use_path_style"="true"
);
+---------------+--------------+------+-------+---------+-------+
| Field         | Type         | Null | Key   | Default | Extra |
+---------------+--------------+------+-------+---------+-------+
| p_partkey     | INT          | Yes  | false | NULL    | NONE  |
| p_name        | TEXT         | Yes  | false | NULL    | NONE  |
| p_mfgr        | TEXT         | Yes  | false | NULL    | NONE  |
| p_brand       | TEXT         | Yes  | false | NULL    | NONE  |
| p_type        | TEXT         | Yes  | false | NULL    | NONE  |
| p_size        | INT          | Yes  | false | NULL    | NONE  |
| p_container   | TEXT         | Yes  | false | NULL    | NONE  |
| p_retailprice | DECIMAL(9,0) | Yes  | false | NULL    | NONE  |
| p_comment     | TEXT         | Yes  | false | NULL    | NONE  |
+---------------+--------------+------+-------+---------+-------+
```

这里给出了一个 S3 TVF 的例子。这个例子中指定了文件的路径、连接信息、认证信息等。

之后，通过 DESC FUNCTION 语法可以查看这个文件的 Schema。

可以看到，对于 Parquet 文件，Doris 会根据文件内的元信息自动推断列类型。

目前支持对 Parquet、ORC、CSV、Json 格式进行分析和列类型推断。

配合 INSERT INTO SELECT 语法，可以方便将文件导入到 Doris 表中进行更快速的分析：

```
// 1. 创建doris内部表
CREATE TABLE IF NOT EXISTS test_table
(
    id int,
    name varchar(50),
    age int
)
DISTRIBUTED BY HASH(id) BUCKETS 4
PROPERTIES("replication_num" = "1");

// 2. 使用 S3 Table Value Function 插入数据
INSERT INTO test_table (id,name,age)
SELECT cast(id as INT) as id, name, cast (age as INT) as age
FROM s3(
    "uri" = "http://127.0.0.1:9312/test2/test.snappy.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "parquet",
    "use_path_style" = "true");
```

注意事项

- 如果 S3 / hdfs TVF 指定的 uri 匹配不到文件，或者匹配到的所有文件都是空文件，那么 S3 / hdfs TVF 将会返回空结果集。在这种情况下使用DESC FUNCTION查看这个文件的 Schema，会得到一列虚假的列__dummy_col，可忽略这一列。
- 如果指定 TVF 的 format 为 CSV The first line is empty, can not parse column numbers, 因为无法通过该文件的第一行解析出 Schema。

2.9.3.4.8 更多帮助

关于 Insert Into 使用的更多详细语法，请参阅INSERT INTO 命令手册，也可以在 MySQL 客户端命令行下输入 HELP INSERT 获取更多帮助信息。

2.9.3.5 Insert Into Values

INSERT INTO VALUES 语句支持将 SQL 中的值导入到 Doris 的表中。INSERT INTO VALUES 是一个同步导入方式，执行导入后返回导入结果。可以通过请求的返回判断导入是否成功。INSERT INTO VALUES 可以保证导入任务的原子性，要么全部导入成功，要么全部导入失败。

2.9.3.5.1 使用场景

1. 用户希望仅导入几条假数据，验证一下 Doris 系统的功能。此时适合使用 INSERT INTO VALUES 的语法，语法和 MySQL 一样。
2. 并发的 INSERT INTO VALUES 的性能会受到 commit 阶段的瓶颈限制。导入数据量较大时，可以打开 group commit 达到更高的性能。

2.9.3.5.2 基本原理

在使用 INSERT INTO VALUES 时，需要通过 MySQL 协议发起导入作业给 FE 节点，FE 会生成执行计划，执行计划中前部是查询相关的算子，最后一个是 OlapTableSink 算子，用于将查询结果写到目标表中。执行计划会被发送给 BE 节点执行，Doris 会选定一个节点做为 Coordinator 节点，Coordinator 节点负责接受数据并分发数据到其他节点上。

2.9.3.5.3 快速上手

INSERT INTO VALUES 通过 MySQL 协议提交和传输。下例以 MySQL 命令行为例，演示通过 INSERT INTO VALUES 提交导入作业。

详细语法可以参见INSERT INTO。

前置检查

INSERT INTO VALUES 需要对目标表的 INSERT 权限。如果没有 INSERT 权限，可以通过GRANT 命令给用户授权。

创建导入作业

INSERT INTO VALUES

1. 创建源表

```
CREATE TABLE testdb.test_table(
    user_id            BIGINT       NOT NULL COMMENT "user id",
    name               VARCHAR(20)           COMMENT "name",
    age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

2. 使用 INSERT INTO VALUES 向源表导入数据（不推荐在生产环境中使用）

```
INSERT INTO testdb.test_table (user_id, name, age)
VALUES (1, "Emily", 25),
       (2, "Benjamin", 35),
       (3, "Olivia", 28),
       (4, "Alexander", 60),
       (5, "Ava", 17);
```

INSERT INTO VALUES 是一种同步导入方式，导入结果会直接返回给用户。

```
Query OK, 5 rows affected (0.308 sec)
{'label':'label_26eebc33411f441c_b2b286730d495e2c', 'status':'VISIBLE', 'txnId':'61071'}
```

3. 查看导入数据

```
MySQL> SELECT COUNT(*) FROM testdb.test_table;
+----------+
| count(*) |
+----------+
|        5 |
+----------+
1 row in set (0.179 sec)
```

查看导入作业

可以通过 show load 命令查看已完成的 INSERT INTO VALUES 任务。

```
mysql> SHOW LOAD FROM testdb`\G`
*************************** 1. row ***************************
         JobId: 77172
         Label: label_26eebc33411f441c_b2b286730d495e2c
         State: FINISHED
      Progress: Unknown id: 77172
          Type: INSERT
       EtlInfo: NULL
      TaskInfo: cluster:N/A; timeout(s):14400; max_filter_ratio:0.0
      ErrorMsg: NULL
    CreateTime: 2024-11-20 16:44:08
  EtlStartTime: 2024-11-20 16:44:08
 EtlFinishTime: 2024-11-20 16:44:08
 LoadStartTime: 2024-11-20 16:44:08
LoadFinishTime: 2024-11-20 16:44:08
           URL:
    JobDetails: {"Unfinished backends":{},"ScannedRows":0,"TaskNumber":0,"LoadBytes":0,"All
        ↪ backends":{},"FileNumber":0,"FileSize":0}
 TransactionId: 61071
  ErrorTablets: {}
          User: root
       Comment:
1 row in set (0.00 sec)
```

取消导入作业

用户可以通过 Ctrl-C 取消当前正在执行的 INSERT INTO VALUES 作业。

2.9.3.5.4 参考手册

导入命令

INSERT INTO VALUES 一般仅用于 Demo，不建议在生产环境使用。

```
INSERT INTO target_table (col1, col2, ...)
VALUES (val1, val2, ...), (val3, val4, ...), ...;
```

导入配置参数

FE 配置

参数	默认值	描述
insert_load_default_timeout_second	14400（4 小时）	导入任务的超时时间，单位：秒。导入任务在该超时时间内未完成则会被系统取消，变成 CANCELLED ↪ 。

环境变量

参数	默认值	描述
insert_timeout	14400（4 小时）	INSERT INTO 作为 SQL 语句的的超时时间，单位：秒。

参数	默认值	描述
enable_insert_strict	true	如果设置为 true，当 INSERT INTO 遇到不合格数据时导入会失败。如果设置为 false，INSERT INTO 会忽略不合格的行，只要有一条数据被正确导入，导入就会成功。在 2.1.4 及以前的版本中。INSERT INTO 无法控制错误率，只能

参数	默认值	描述
insert_max_filter_ratio	1.0	自 2.1.5 版本，仅当 enable_insert_strict 值为 false 时生效。用于控制当使用 INSERT INTO FROM S3/HDFS/LOCAL() 时，设定错误容忍率的。默认为 1.0 表示容忍所有错误。可以取值 0 ~ 1 之间的小数

参数	默认值	描述

导入返回值

INSERT INTO VALUES 是一个 SQL 语句，其返回结果会包含一个 JSON 字符串。

其中的参数如下表说明：

参数名称	说明
Label	导入作业的 label，通过 INSERT INTO tbl WITH LABEL label … 指定

| Status | 表示导入数据是否可见。如果可见，显示 `visible`，如果不可见，显示 `committed`

- `visible`：表示导入成功，数据可见
  - `committed`：该状态也表示导入已经完成，只是数据可能会延迟可见，无需重试
    * Label Already Exists：Label 重复，需要更换 label
      · Fail：导入失败
      Err | 导入错误信息 |
      TxnId | 导入事务的 ID |

INSERT 执行成功

```
mysql> INSERT INTO test_table (user_id, name, age) VALUES (1, "Emily", 25), (2, "Benjamin", 35),
    ↪ (3, "Olivia", 28), (NULL, "Alexander", 60), (5, "Ava", 17);
Query OK, 5 rows affected (0.05 sec)
{'label':'label_26eebc33411f441c_b2b286730d495e2c', 'status':'VISIBLE', 'txnId':'61071'}
```

其中 `Query OK` 表示执行成功。`5 rows affected` 表示总共有 5 行数据被导入。

INSERT 执行成功但是有 warning

```
mysql> INSERT INTO test_table (user_id, name, age) VALUES (1, "Emily", 25), (2, "Benjamin", 35),
    ↪ (3, "Olivia", 28), (NULL, "Alexander", 60), (5, "Ava", 17);
Query OK, 4 rows affected, 1 warning (0.04 sec)
{'label':'label_a8d99ae931194d2b_93357aac59981a18', 'status':'VISIBLE', 'txnId':'61068'}
```

其中 `Query OK` 表示执行成功。`4 rows affected` 表示总共有 4 行数据被导入。`1 warnings` 表示被过滤了 1 行。

当需要查看被过滤的行时，用户可以通过SHOW LOAD语句。返回结果中的 URL 可以用于查询错误的数据，具体见后面查看错误行小结。

```
mysql> SHOW LOAD WHERE label="label_a8d99ae931194d2b_93357aac59981a18"`\G`
*************************** 1. row ***************************
         JobId: 77158
         Label: label_a8d99ae931194d2b_93357aac59981a18
```

```
         State: FINISHED
      Progress: Unknown id: 77158
          Type: INSERT
       EtlInfo: NULL
      TaskInfo: cluster:N/A; timeout(s):14400; max_filter_ratio:0.0
      ErrorMsg: NULL
    CreateTime: 2024-11-20 16:35:40
  EtlStartTime: 2024-11-20 16:35:40
 EtlFinishTime: 2024-11-20 16:35:40
 LoadStartTime: 2024-11-20 16:35:40
LoadFinishTime: 2024-11-20 16:35:40
           URL: http://10.16.10.7:8743/api/_load_error_log?file=__shard_18/error_log_insert_stmt_
               ↪ a8d99ae931194d2b-93357aac59981a19_a8d99ae931194d2b_93357aac59981a19
    JobDetails: {"Unfinished backends":{},"ScannedRows":0,"TaskNumber":0,"LoadBytes":0,"All
        ↪ backends":{},"FileNumber":0,"FileSize":0}
 TransactionId: 61068
  ErrorTablets: {}
          User: root
       Comment:
1 row in set (0.00 sec)
```

INSERT 执行成功但是 status 是 committed

```
mysql> INSERT INTO test_table (user_id, name, age) VALUES (1, "Emily", 25), (2, "Benjamin", 35),
    ↪ (3, "Olivia", 28), (4, "Alexander", 60), (5, "Ava", 17);
Query OK, 5 rows affected (0.04 sec)
{'label':'label_78bf5396d9594d4d_a8d9a914af40f73d', 'status':'COMMITTED', 'txnId':'61074'}
```

数据不可见是一个临时状态，这批数据最终是一定可见的

可以通过SHOW TRANSACTION 语句查看这批数据的可见状态。当返回结果中的 TransactionStatus 列变成 VISIBLE 时代表数据可见。

```
mysql> SHOW TRANSACTION WHERE id=61074`\G`
*************************** 1. row ***************************
     TransactionId: 61074
             Label: label_78bf5396d9594d4d_a8d9a914af40f73d
       Coordinator: FE: 10.16.10.7
 TransactionStatus: VISIBLE
 LoadJobSourceType: INSERT_STREAMING
       PrepareTime: 2024-11-20 16:51:54
     PreCommitTime: NULL
        CommitTime: 2024-11-20 16:51:54
       PublishTime: 2024-11-20 16:51:54
        FinishTime: 2024-11-20 16:51:54
            Reason:
ErrorReplicasCount: 0
```

```
        ListenerId: -1
         TimeoutMs: 14400000
            ErrMsg:
1 row in set (0.00 sec)
```

INSERT 执行失败

执行失败表示没有任何数据被成功导入，并返回如下：

```
mysql> INSERT INTO test_table (user_id, name, age) VALUES (1, "Emily", 25), (2, "Benjamin", 35),
    ↪ (3, "Olivia", 28), (NULL, "Alexander", 60), (5, "Ava", 17);
ERROR 1105 (HY000): errCode = 2, detailMessage = Insert has too many filtered data 1/5 insert_max
    ↪ _filter_ratio is 0.100000. url: http://10.16.10.7:8747/api/_load_error_log?file=__shard_
    ↪ 22/error_log_insert_stmt_5fafe6663e1a45e0-a666c1722ffc8c55_5fafe6663e1a45e0_
    ↪ a666c1722ffc8c55
```

其 中 `ERROR 1105 (HY000): errCode = 2, detailMessage = Insert has too many filtered data 1/5 ↪ insert_max_filter_ratio is 0.100000.` 显示失败原因。后面的 url 可以用于查询错误的数据，具体见后面查看错误行小结。

2.9.3.5.5 导入最佳实践

数据量

INSERT INTO VALUES 通常用于测试和演示，不建议用于导入大量数据的场景。

查看错误行

当 INSERT INTO 返回结果中提供了 url 字段时，可以通过以下命令查看错误行：

```
SHOW LOAD WARNINGS ON "url";
```

示例：

```
mysql> SHOW LOAD WARNINGS ON "http://10.16.10.7:8743/api/_load_error_log?file=__shard_18/error_
    ↪ log_insert_stmt_a8d99ae931194d2b-93357aac59981a19_a8d99ae931194d2b_93357aac59981a19"\G
*************************** 1. row ***************************
         JobId: -1
         Label: NULL
ErrorMsgDetail: Reason: column_name[user_id], null value for not null column, type=BIGINT. src
    ↪ line [];
1 row in set (0.00 sec)
```

常见的错误的原因有：源数据列长度超过目的数据列长度、列类型不匹配、分区不匹配、列顺序不匹配等。

可以通过环境变量 enable_insert_strict来控制 INSERT INTO 是否忽略错误行。

2.9.3.5.6 更多帮助

关于 Insert Into 使用的更多详细语法，请参阅INSERT INTO 命令手册，也可以在 MySQL 客户端命令行下输入 HELP INSERT 获取更多帮助信息。

2.9.3.6 MySQL Load

Doris 兼容 MySQL 协议，可以使用 MySQL 标准的 LOAD DATA 语法导入本地文件。MySQL Load 是一种同步导入方式，执行导入后即返回导入结果。可以通过 LOAD DATA 语句的返回结果判断导入是否成功。一般来说，可以使用 MySQL Load 导入 10GB 以下的文件，如果文件过大，建议将文件进行切分后使用 MySQL Load 进行导入。MySQL Load 可以保证一批导入任务的原子性，要么全部导入成功，要么全部导入失败。

2.9.3.6.1 使用场景

支持格式

MySQL Load 主要适用于导入客户端本地 CSV 文件，或通过程序导入数据流中的数据。

使用限制

在导入 CSV 文件时，需要明确区分空值（null）与空字符串（”）：

- 空值（null）需要用 \N 表示，a,\N,b 数据表示中间列是一个空值（null）
- 空字符串直接将数据置空，a,,b 数据表示中间列是一个空字符串

2.9.3.6.2 基本原理

MySQL Load 与 Stream Load 功能相似，都是导入本地文件到 Doris 集群中。因此 MySQL Load 的实现复用了 Stream Load 的基本导入能力。

下图展示了 MySQL Load 的主要流程：

1. 用户向 FE 提交 LOAD DATA 请求，FE 完成解析工作，并将请求封装成 Stream Load；
2. FE 会选择一个 BE 节点发送 Stream Load 请求；
3. 发送请求的同时，FE 会异步且流式的从 MySQL 客户端读取本地文件数据，并实时的发送到 Stream Load 的 HTTP 请求中；
4. MySQL 客户端数据传输完毕，FE 等待 Stream Load 完成，并展示导入成功或者失败的信息给客户端。

2.9.3.6.3 快速上手

前置检查

MySQL Load 需要对目标表的 INSERT 权限。如果没有 INSERT 权限，可以通过 GRANT 命令给用户授权。

创建导入作业

1. 准备测试数据

创建名为 client_local.csv 的文件，样例数据如下：

```
1,10
2,20
3,30
4,40
5,50
6,60
```

2. 链接客户端

在执行 LOAD DATA 命令前，需要先链接 MySQL 客户端。

```
mysql --local-infile  -h <fe_ip> -P <fe_query_port> -u root -D testdb
```

> 执行 MySQL Load，在连接时需要使用指定参数选项：
>
> 1. 在链接 mysql 客户端时，必须使用 `--local-infile` 选项，否则可能会报错。
> 2. 通过 JDBC 链接，需要在 URL 中指定配置 `allowLoadLocalInfile=true`

3. 创建测试用表

在 Doris 中创建以下表：

```
CREATE TABLE testdb.t1 (
    pk      INT,
    v1      INT SUM
) AGGREGATE KEY (pk)
DISTRIBUTED BY hash (pk);
```

4. 运行 LOAD DATA 导入命令

链接 MySQL Client 后，创建导入作业，命令如下：

```
LOAD DATA LOCAL
INFILE 'client_local.csv'
INTO TABLE testdb.t1
COLUMNS TERMINATED BY ','
LINES TERMINATED BY '``\n``';
```

查看导入作业结果

MySQL Load 是一种同步的导入方式，导入后结果会在命令行中返回给用户。如果导入执行失败，会展示具体的报错信息。

如下是导入成功的结果显示，会返回导入的行数：

```
Query OK, 6 row affected (0.17 sec)
Records: 6  Deleted: 0  Skipped: 0  Warnings: 0
```

当导入有异常时，会在客户端显示相应异常：

```
ERROR 1105 (HY000): errCode = 2, detailMessage = [DATA_QUALITY_ERROR]too many filtered rows with
    ↪ load id b612907c-ccf4-4ac2-82fe-107ece655f0f
```

在异常信息中，可以捕捉到导入的 loadId，通过 show load warnings 命令可以查看到具体信息：

```
show load warnings where label='b612907c-ccf4-4ac2-82fe-107ece655f0f';
```

取消导入作业

用户无法手动取消 MySQL Load，MySQL Load 在超时或者导入错误后会被系统自动取消。

2.9.3.6.4 参考手册

导入语法

LOAD DATA 语法如下：

```
LOAD DATA LOCAL
INFILE '<load_data_file>'
INTO TABLE [<db_name>.]<table_name>
[PARTITION (partition_name [, partition_name] ...)]
[COLUMNS TERMINATED BY '<column_terminated_operator>']
[LINES TERMINATED BY '<line_terminated_operator>']
[IGNORE <ignore_lines> LINES]
[(col_name_or_user_var[, col_name_or_user_var] ...)]
[SET col_name={expr | DEFAULT}[, col_name={expr | DEFAULT}] ...]
[PROPERTIES (key1 = value1 [, key2=value2]) ]
```

创建导入作业的模块说明如下：

模块	说明
INFILE	指定本地文件路径，可以是相对路径，也可以是绝对路径。目前 load_data_file 只支持单个文件导入。
INTO TABLE	指定数据库名与表名，可以省略数据库名。
PARTITION	指定导入的分区。如果用户能够确定数据对应的 partition，推荐指定该项。不满足这些分区的数据将
COLUMNS TERMINATED BY	指定导入的列分隔符。
LINE TERMINATED BY	指定导入的行分隔符。
IGNORE num LINES	指定导入的 CSV 跳过行数，通常指定 1 来跳过表头。
col_name_or_user_var	指定列映射语法，数据转换详见列映射章节。
PROPERTIES	导入参数。

导入参数

通过 PROPERTIES (key1 = value1 [, key2=value2]) 语法可以指定导入的参数配置：

参数	说明
max_filter_ratio	允许的最大过滤率。必须在大于等于 0 到小于等于 1 之间。默认值是 0，表示不容忍任何错误行。
timeout	指定导入的超时时间，单位秒。默认是 600 秒。可设置范围为 1s ~ 259200s。
strict_mode	用户指定此次导入是否开启严格模式，默认为关闭。
timezone	指定本次导入所使用的时区。默认为东八区。该参数会影响所有导入涉及的和时区有关的函数结果。
exec_mem_limit	导入内存限制。默认为 2GB。单位为字节。
trim_double_quotes	布尔类型，默认值为 false，为 true 时表示裁剪掉导入文件每个字段最外层的双引号。
enclose	指定包围符。当 CSV 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为 “,”，包围符为 “ ‘ “，数据为”a,’b,c’ ”，则 “b,c” 会被解析为一个字段。
escape	指定转义符。用于转义在字段中出现的与包围符相同的字符。例如数据为 “a, ‘b,’ c’ ”，包围符为 “ ‘ “，希望”b,’ c 被作为一个字段解析，则需要指定单字节转义符，例如 “”，将数据修改为”a,’b,’c’ ”。

2.9.3.6.5 导入举例

指定导入超时时间

通过指定 PROPERTIES 参数 timeout 可以调整导入超时时间。在以下案例中将超时时间设置为 100s：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("timeout"="100");
```

指定导入允许误差率

通过指定 PROPERTIES 参数 max_filter_ratio 可以调整导入容错率。在以下案例中将错误容忍率设置为 20%：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("max_filter_ratio"="0.2");
```

映射导入列

在以下案例中调整了 CSV 中列的顺序：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
(k2, k1, v1);
```

指定导入列分隔符与行分隔符

通过 COLUMNS TERMINATED BY 与 LINES TERMINATED BY 子句可以指定导入的列与行分隔符。在以下案例中使用逗号（,）与换行符（\n）作为列与行分隔符：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
COLUMNS TERMINATED BY ','
LINES TERMINATED BY ``\n``;
```

指定导入分区

通过 PARTITION 子句可以指定导入分区。在以下案例中将数据导入指定分区 p1 与 p2，如果数据不属于 p1 与 p2 分区，会被过滤掉：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PARTITION (p1, p2);
```

指定导入时区

通过 PROPERTIES 参数 timezone 可以指定时区。在以下案例中设置时区为 Africa/Abidjan：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("timezone"="Africa/Abidjan");
```

限制导入内存

通过 PROPERTIES 参数 exec_mem_limit 可以指定导入的内存限制。在以下案例中设置导入的内存限制为 10G：

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("exec_mem_limit"="10737418240");
```

2.9.3.6.6 更多帮助

关于 MySQL Load 使用的更多详细语法及最佳实践，请参阅MySQL Load 命令手册。

2.9.4 文件格式

2.9.4.1 CSV

本文介绍如何在 Doris 中导入 CSV 格式的数据文件。Doris 支持灵活的 CSV 格式配置，包括自定义分隔符、字段包围符等，并提供多种导入方式以满足不同场景的数据导入需求。

2.9.4.1.1 导入方式

Doris 支持以下方式导入 CSV 格式数据：

- Stream Load
- Broker Load
- Routine Load
- MySQL Load
- INSERT INTO FROM S3 TVF
- INSERT INTO FROM HDFS TVF

2.9.4.1.2 参数配置

参数支持情况

下表列出了各种导入方式支持的 CSV 格式参数：

参数	默认值	Stream Load	Broker Load	Routine Load	MySQL Load	TVF
行分隔符	\n	line_delimiter	LINES TERMINATED BY	不支持	LINES TERMINATED BY	line_delimiter
列分隔符	\t	column_separator	COLUMNS TERMINATED BY	COLUMNS TERMINATED BY	COLUMNS TERMINATED BY	column_separator
包围符	无	enclose	PROPERTIES.enclose	PROPERTIES.enclose	PROPERTIES.enclose	enclose
转义符	\	escape	PROPERTIES.escape	PROPERTIES.escape	PROPERTIES.escape	escape
跳过行数	0	skip_lines	PROPERTIES.skip_lines	不支持	IGNORE LINES	skip_lines
裁剪双引号	false	trim_double_quotes	不支持	PROPERTIES.trim_double_quotes	不支持	trim_double_quotes
压缩格式	plain	compress_type	PROPERTIES.compress_type	不支持	不支持	compress_type

> 注意 1. Stream Load：参数直接通过 HTTP Header 指定，如：`-H "line_delimiter:\\n"` 2. Broker Load：参数通过 SQL 语句指定，其中：- 分隔符通过 `COLUMNS TERMINATED BY`, `LINES TERMINATED BY` 指定 - 其他参数通过 `PROPERTIES` 指定，如：`PROPERTIES("compress_type"="gz")` 3. Routine Load：参数通过 SQL 语句指定，其中：- 分隔符通过 `COLUMNS TERMINATED BY` 指定 - 其他参数通过 `PROPERTIES` 指定，如：`PROPERTIES("enclose"="\"")` 4. MySQL Load：参数通过 SQL 语句指定，其中：- 分隔符通过 `LINES TERMINATED BY`, `COLUMNS TERMINATED BY` 指定 - 其他参数通过 `PROPERTIES` 指定，如：`PROPERTIES("escape"="\\")` 5. TVF：参数通过 TVF 语句指定，如：`S3("line_delimiter"="\\n")`

参数说明

行分隔符

- 作用：指定导入文件中的换行符
- 默认值：\n
- 特点：支持多个字符组合作为换行符
- 使用场景和示例：
- Linux/Unix 系统文件：

数据文件：
张三，25`\n`
李四，30`\n`

参数设置：
line_delimiter：`\n`（默认值，可不设置）

- Windows 系统文件：

数据文件：
张三，25\r`\n`
李四，30\r`\n`

参数设置：
line_delimiter：\r`\n`

- 特殊程序生成文件：

数据文件：
张三，25\r
李四，30\r

参数设置：
line_delimiter：\r

- 自定义多字符分隔符：

数据文件：
张三，25||
李四，30||

参数设置：
line_delimiter：||

列分隔符

- 作用：指定导入文件中的列分隔符
- 默认值：\t（制表符）

- 特点：
- 支持可见和不可见字符
- 支持多字符组合
- 不可见字符需要使用 \x 前缀的十六进制表示
- MySQL 协议特殊处理：
- 不可见字符需要额外增加反斜线
- 如 Hive 的 \x01 在 Broker Load 中需要写成 \\x01
- 使用场景和示例：
- 常见可见字符：

```
数据文件：
张三，25，北京
李四，30，上海

参数设置：
column_separator：，
```

- 制表符（默认）：

```
数据文件：
张三    25    北京
李四    30    上海

参数设置：
column_separator：`\t`（默认值，可不设置）
```

- Hive 文件（Stream Load）：

```
数据文件：
张三\x0125\x01 北京
李四\x0130\x01 上海

参数设置：
column_separator：\x01
```

- Hive 文件（Broker Load）：

```
数据文件：
张三\x0125\x01 北京
李四\x0130\x01 上海

参数设置：
PROPERTIES("column_separator"="\\x01")
```

- 多字符分隔符：

```
数据文件：
张三||25||北京
李四||30||上海

参数设置：
column_separator: ||
```

包围符

- 作用：保护包含特殊字符的字段，防止被错误解析
- 限制：仅支持单字节字符
- 常用字符：
- 单引号：'
- 双引号："
- 使用场景和示例：
- 字段包含列分隔符：

```
数据：a,'b,c',d
列分隔符：,
包围符：'
解析结果：3 个字段 [a] [b,c] [d]
```

- 字段包含行分隔符：

```
数据：a,'b\nc',d
列分隔符：,
包围符：'
解析结果：3 个字段 [a] [b\nc] [d]
```

- 字段既包含列分隔符又包含行分隔符：

```
数据：a,'b,c\nd,e',f
列分隔符：,
包围符：'
解析结果：3 个字段 [a] [b,c\nd,e] [f]
```

转义符

- 作用：转义字段中与包围符相同的字符
- 限制：仅支持单字节字符，默认为 \

- 使用场景和示例：
- 字段包含包围符：

```
数据：a,'b,\'c',d
列分隔符：,
包围符：'
转义符：\
解析结果：3 个字段 [a] [b,'c] [d]
```

- 字段包含多个包围符：

```
数据：a,"b,\"c\"d",e
列分隔符：,
包围符："
转义符：\
解析结果：3 个字段 [a] [b,"c"d] [e]
```

- 字段包含转义符本身：

```
数据：a,'b\\c',d
列分隔符：,
包围符：'
转义符：\
解析结果：3 个字段 [a] [b\c] [d]
```

跳过行数

- 作用：跳过 CSV 文件的前几行
- 类型：整数类型
- 默认值：0
- 特殊说明：
- 当 format 为 csv_with_names 时，系统会自动跳过首行（列名），忽略 skip_lines 参数
- 当 format 为 csv_with_names_and_types 时，系统会自动跳过前两行（列名和类型），忽略 skip_lines 参数
- 使用场景和示例：
- 跳过标题行：

```
数据文件：
姓名，年龄，城市
张三，25，北京
李四，30，上海

参数设置：
skip_lines：1
结果：跳过标题行，导入后续数据
```

- 跳过注释行：

数据文件：
用户信息表
创建时间：2024-01-01
张三，25，北京
李四，30，上海

参数设置：
skip_lines：2
结果：跳过注释行，导入后续数据

- 使用 csv_with_names 格式：

数据文件：
name,age,city
张三，25，北京
李四，30，上海

参数设置：
format：csv_with_names
结果：系统自动跳过首行列名

- 使用 csv_with_names_and_types 格式：

数据文件：
name,age,city
string,int,string
张三，25，北京
李四，30，上海

参数设置：
format：csv_with_names_and_types
结果：系统自动跳过前两行的列名和类型信息

裁剪双引号

- 作用：裁剪掉 CSV 文件每个字段最外层的双引号
- 类型：布尔类型
- 默认值：false
- 使用场景和示例：
- 裁剪双引号：

数据文件：
"张三","25","北京"
"李四","30","上海"

参数设置：
trim_double_quotes: true
结果：
张三，25，北京
李四，30，上海

压缩格式

- 作用：指定导入文件的压缩格式
- 类型：字符串，忽略大小写
- 默认值：plain
- 支持的压缩格式：
- plain：不压缩（默认）
- bz2：BZIP2 压缩
- deflate：DEFLATE 压缩
- gz：GZIP 压缩
- lz4：LZ4 Frame 格式压缩
- lz4_block：LZ4 Block 格式压缩
- lzo：LZO 压缩
- lzop：LZOP 压缩
- snappy_block：SNAPPY Block 格式压缩
- 注意事项：
- tar 是一种文件打包格式，不属于压缩格式，因此不支持.tar 文件
- 如需使用 tar 打包的文件，请先解包后再导入

2.9.4.1.3 使用示例

本节展示了不同导入方式下的 CSV 格式使用方法。

Stream Load 导入

```
curl --location-trusted -u root: \
    -H "column_separator:," \
    -H "line_delimiter:`\n`" \
    -T example.csv \
    http://<fe_host>:<fe_http_port>/api/test_db/test_table/_stream_load

#### 处理带引号的数据
curl --location-trusted -u root: \
    -H "column_separator:," \
    -H "enclose:\"" \
    -H "escape:\\" \
    -T example.csv \
    http://<fe_host>:<fe_http_port>/api/test_db/test_table/_stream_load

#### 导入压缩文件
curl --location-trusted -u root: \
    -H "compress_type:gz" \
    -T example.csv.gz \
    http://<fe_host>:<fe_http_port>/api/test_db/test_table/_stream_load
```

Broker Load 导入

```
-- 指定分隔符
LOAD LABEL test_db.test_label
(
    DATA INFILE("s3://bucket/example.csv")
    INTO TABLE test_table
    COLUMNS TERMINATED BY ","
    LINES TERMINATED BY "`\n`"
)
WITH S3
(
    ...
);

-- 处理带引号的数据
LOAD LABEL test_db.test_label
(
    DATA INFILE("s3://bucket/example.csv")
    INTO TABLE test_table
    PROPERTIES
    (
        "enclose" = "\"",
        "escape" = "\\"
```

```
    )
)
WITH S3
(
    ...
);

-- 导入压缩文件
LOAD LABEL test_db.test_label
(
    DATA INFILE("s3://bucket/example.csv.gz")
    INTO TABLE test_table
    PROPERTIES
    (
        "compress_type" = "gz"
    )
)
WITH S3
(
    ...
);
```

Routine Load 导入

```
-- 指定分隔符
CREATE ROUTINE LOAD test_db.test_job ON test_table
COLUMNS TERMINATED BY ","
FROM KAFKA
(
     ...
);

-- 处理带引号的数据
CREATE ROUTINE LOAD test_db.test_job ON test_table
COLUMNS TERMINATED BY ","
PROPERTIES
(
    "enclose" = "\"",
    "escape" = "\\"
)
FROM KAFKA
(
    ...
);
```

MySQL Load 导入

```
-- 指定分隔符
LOAD DATA LOCAL INFILE 'example.csv'
INTO TABLE test_table
COLUMNS TERMINATED BY ','
LINES TERMINATED BY ``\n``;

-- 处理带引号的数据
LOAD DATA LOCAL INFILE 'example.csv'
INTO TABLE test_table
COLUMNS TERMINATED BY ','
LINES TERMINATED BY ``\n``
PROPERTIES
(
    "enclose" = "\"",
    "escape" = "\\"
);

-- 跳过表头
LOAD DATA LOCAL INFILE 'example.csv'
INTO TABLE test_table
COLUMNS TERMINATED BY ','
LINES TERMINATED BY ``\n``
IGNORE 1 LINES;
```

TVF 导入

```
-- 指定分隔符
INSERT INTO test_table
SELECT *
FROM S3
(
    "uri" = "s3://bucket/example.csv",
    "format" = "csv",
    "column_separator" = ",",
    "line_delimiter" = "`\n`"
    ...
);

-- 处理带引号的数据
INSERT INTO test_table
SELECT *
FROM S3
(
    "uri" = "s3://bucket/example.csv",
    "format" = "csv",
```

```
    "column_separator" = ",",
    "enclose" = "\"",
    "escape" = "\\"
    ...
);

-- 导入压缩文件
INSERT INTO test_table
SELECT *
FROM S3
(
    "uri" = "s3://bucket/example.csv.gz",
    "format" = "csv",
    "compress_type" = "gz"
    ...
);
```

2.9.4.2 Parquet

本文介绍如何在 Doris 中导入 Parquet 格式的数据文件。

2.9.4.2.1 支持的导入方式

以下导入方式支持 Parquet 格式的数据导入：

- Stream Load
- Broker Load
- INSERT INTO FROM S3 TVF
- INSERT INTO FROM HDFS TVF

2.9.4.2.2 使用示例

本节展示了不同导入方式下的 Parquet 格式使用方法。

Stream Load 导入

```
curl --location-trusted -u <user>:<passwd> \
    -H "format: parquet" \
    -T example.parquet \
    http://<fe_host>:<fe_http_port>/api/example_db/example_table/_stream_load
```

Broker Load 导入

```
LOAD LABEL example_db.example_label
(
    DATA INFILE("s3://bucket/example.parquet")
    INTO TABLE example_table
```

```
    FORMAT AS "parquet"
)
WITH S3
(
    ...
);
```

TVF 导入

```
INSERT INTO example_table
SELECT *
FROM S3
(
    "uri" = "s3://bucket/example.parquet",
    "format" = "parquet",
    ...
);
```

2.9.4.3 ORC

本文介绍如何在 Doris 中导入 ORC 格式的数据文件。

2.9.4.3.1 支持的导入方式

以下导入方式支持 ORC 格式的数据导入：

- Stream Load
- Broker Load
- INSERT INTO FROM S3 TVF
- INSERT INTO FROM HDFS TVF

2.9.4.3.2 使用示例

本节展示了不同导入方式下的 ORC 格式使用方法。

Stream Load 导入

```
curl --location-trusted -u <user>:<passwd> \
    -H "format: orc" \
    -T example.orc \
    http://<fe_host>:<fe_http_port>/api/example_db/example_table/_stream_load
```

Broker Load 导入

```
LOAD LABEL example_db.example_label
(
    DATA INFILE("s3://bucket/example.orc")
```

```
    INTO TABLE example_table
    FORMAT AS "orc"
)
WITH S3
(
    ...
);
```

TVF 导入

```
INSERT INTO example_table
SELECT *
FROM S3
(
    "uri" = "s3://bucket/example.orc",
    "format" = "orc",
    ...
);
```

2.9.5 复杂数据类型

2.9.5.1 ARRAY

ARRAY<T> 表示由 T 类型元素组成的数组，不能作为 key 列使用。

- 2.0 之前仅支持在 Duplicate 模型的表中使用。
- 从 2.0 版本开始支持在 Unique 模型的表中的非 key 列使用。

T 支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL,
DATE, DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

2.9.5.1.1 CSV 格式导入

第 1 步：准备数据

创建如下的 csv 文件：`test_array.csv` 其中分隔符使用 | 而不是逗号，以便和 array 中的逗号区分。

```
1|[1,2,3,4,5]
2|[6,7,8]
3|[]
4|null
```

第 2 步：在数据库中建表

```
CREATE TABLE `array_test` (
    `id`          INT           NOT NULL,
    `c_array`     ARRAY<INT>    NULL
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "column_separator:|" \
        -H "columns: id, c_array" \
        -T "test_array.csv" \
        http://localhost:8040/api/testdb/array_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM array_test;
+------+-----------------+
| id   | c_array         |
+------+-----------------+
|    1 | [1, 2, 3, 4, 5] |
|    2 | [6, 7, 8]       |
|    3 | []              |
|    4 | NULL            |
+------+-----------------+
4 rows in set (0.01 sec)
```

2.9.5.1.2 JSON 格式导入

第 1 步：准备数据

创建如下的 JSON 文件， `test_array.json`

```
[
    {"id":1, "c_array":[1,2,3,4,5]},
    {"id":2, "c_array":[6,7,8]},
    {"id":3, "c_array":[]},
    {"id":4, "c_array":null}
]
```

第 2 步：在数据库中建表

```
CREATE TABLE `array_test` (
    `id`         INT           NOT NULL,
    `c_array`    ARRAY<INT>    NULL
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "format:json" \
        -H "columns: id, c_array" \
        -H "strip_outer_array:true" \
        -T "test_array.json" \
        http://localhost:8040/api/testdb/array_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM array_test;
+------+-----------------+
| id   | c_array         |
+------+-----------------+
|    1 | [1, 2, 3, 4, 5] |
|    2 | [6, 7, 8]       |
|    3 | []              |
|    4 | NULL            |
+------+-----------------+
4 rows in set (0.01 sec)
```

2.9.5.2 MAP

MAP<K，V> 表示由 K, V 类型元素组成的 map，不能作为 key 列使用。

- 目前支持在 Duplicate，Unique 模型的表中使用。

K, V 支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL, DECIMALV3,
DATE, DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

2.9.5.2.1 CSV 格式导入

第 1 步：准备数据

创建如下的 csv 文件：test_map.csv 其中分隔符使用 | 而不是逗号，以便和 map 中的逗号区分。

```
1|{"Emily":101,"age":25}
2|{"Benjamin":102}
3|{}
4|null
```

第 2 步：在数据库中建表

```
CREATE TABLE map_test (
    id       INT                 NOT NULL,
    c_map    MAP<STRING, INT>    NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "column_separator:|" \
        -H "columns: id, c_map" \
        -T "test_map.csv" \
        http://localhost:8040/api/testdb/map_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM map_test;
+------+-------------------------+
| id   | c_map                   |
+------+-------------------------+
|    1 | {"Emily":101, "age":25} |
|    2 | {"Benjamin":102}        |
|    3 | {}                      |
|    4 | NULL                    |
+------+-------------------------+
4 rows in set (0.01 sec)
```

2.9.5.2.2 JSON 格式导入

第 1 步：准备数据

创建如下的 JSON 文件，test_map.json

```
[
    {"id":1, "c_map":{"Emily":101, "age":25}},
    {"id":2, "c_map":{"Benjamin":102}},
    {"id":3, "c_map":{}},
    {"id":4, "c_map":null}
]
```

第 2 步：在数据库中建表

```
CREATE TABLE map_test (
    id       INT                 NOT NULL,
    c_map    MAP<STRING, INT>    NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "format:json" \
        -H "columns: id, c_map" \
        -H "strip_outer_array:true" \
        -T "test_map.json" \
        http://localhost:8040/api/testdb/map_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM map_test;
+------+-------------------------+
| id   | c_map                   |
+------+-------------------------+
|    1 | {"Emily":101, "age":25} |
|    2 | {"Benjamin":102}        |
|    3 | {}                      |
|    4 | NULL                    |
+------+-------------------------+
4 rows in set (0.01 sec)
```

2.9.5.3 STRUCT

STRUCT<field_name:field_type [COMMENT 'comment_string'], ... > 表示由多个 Field 组成的结构体，也可被理解为多个列的集合。

- 不能作为 Key 使用，目前 STRUCT 仅支持在 Duplicate 模型的表中使用。
- 一个 Struct 中的 Field 的名字和数量固定，总是为 Nullable，一个 Field 通常由下面部分组成。
- field_name: Field 的标识符，不可重复
- field_type: Field 的类型
- COMMENT: Field 的注释，可选 (暂不支持)

当前可支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL, DECIMALV3,
DATE, DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

2.9.5.3.1 CSV 格式导入

第 1 步：准备数据

创建如下的 csv 文件：test_struct.csv 其中分隔符使用 | 而不是逗号，以便和 struct 中的逗号区分。

```
1|{10, 3.14, "Emily"}
2|{4, 1.5, null}
3|{7, null, "Benjamin"}
4|{}
5|null
```

第 2 步：在数据库中建表

```
CREATE TABLE struct_test (
    id          INT                                 NOT NULL,
    c_struct    STRUCT<f1:INT,f2:FLOAT,f3:STRING>   NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "column_separator:|" \
        -H "columns: id, c_struct" \
        -T "test_struct.csv" \
        http://localhost:8040/api/testdb/struct_test/_stream_load
```

也可以使用 INSERT INTO VALUES 语句导入：

```
INSERT INTO struct_test VALUES(1, named_struct('f1', '1', 'f2', '2.0', 'f3', 'abc'));
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM struct_test;
+------+---------------------------------------+
| id   | c_struct                              |
+------+---------------------------------------+
|    1 | {"f1":10, "f2":3.14, "f3":"Emily"}    |
|    2 | {"f1":4, "f2":1.5, "f3":null}         |
|    3 | {"f1":7, "f2":null, "f3":"Benjamin"} |
|    4 | {"f1":null, "f2":null, "f3":null}     |
|    5 | NULL                                  |
+------+---------------------------------------+
5 rows in set (0.01 sec)
```

2.9.5.3.2 JSON 格式导入

第 1 步：准备数据

创建如下的 JSON 文件，test_struct.json

```
[
    {"id":1, "c_struct":{"f1":10, "f2":3.14, "f3":"Emily"}},
    {"id":2, "c_struct":{"f1":4, "f2":1.5, "f3":null}},
    {"id":3, "c_struct":{"f1":7, "f2":null, "f3":"Benjamin"}},
    {"id":4, "c_struct":{}},
    {"id":5, "c_struct":null}
]
```

第 2 步：在数据库中建表

```
CREATE TABLE struct_test (
    id          INT                                 NOT NULL,
    c_struct    STRUCT<f1:INT,f2:FLOAT,f3:STRING>    NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "format:json" \
        -H "columns: id, c_struct" \
        -H "strip_outer_array:true" \
        -T "test_struct.json" \
```

```
        http://localhost:8040/api/testdb/struct_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM struct_test;
+------+---------------------------------------+
| id   | c_struct                              |
+------+---------------------------------------+
|    1 | {"f1":10, "f2":3.14, "f3":"Emily"}    |
|    2 | {"f1":4, "f2":1.5, "f3":null}         |
|    3 | {"f1":7, "f2":null, "f3":"Benjamin"} |
|    4 | {"f1":null, "f2":null, "f3":null}     |
|    5 | NULL                                  |
+------+---------------------------------------+
5 rows in set (0.00 sec)
```

2.9.5.4 JSON

JSON 数据类型，用二进制格式高效存储 JSON 数据，通过 JSON 函数访问其内部字段。

默认支持 1048576 字节（1 MB），可调大到 2147483643 字节（2 GB），可通过 BE 配置`string_type_length_soft_limit_bytes` 调整。

与普通 String 类型存储的 JSON 字符串相比，JSON 类型有两点优势

1. 数据写入时进行 JSON 格式校验
2. 二进制存储格式更加高效，通过 json_extract 等函数可以高效访问 JSON 内部字段，比 get_json_xx 函数快几倍

> 在 1.2.x 版本中，JSON 类型的名字是 JSONB，为了尽量跟 MySQL 兼容，从 2.0.0 版本开始改名为 JSON，老的表仍然可以使用。

2.9.5.4.1 CSV 格式导入

第 1 步：准备数据

创建如下的 csv 文件：`test_json.csv` 其中分隔符使用 | 而不是逗号，以便和 json 中的逗号区分。

```
1|{"name": "tom", "age": 35}
2|{"name": null, "age": 28}
3|{"name": "micheal", "age": null}
4|{"name": null, "age": null}
5|null
```

第 2 步：在数据库中建表

```
CREATE TABLE json_test (
    id          INT     NOT NULL,
    c_json      JSON    NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "column_separator:|" \
        -H "columns: id, c_json" \
        -T "test_json.csv" \
        http://localhost:8040/api/testdb/json_test/_stream_load
```

第 4 步：检查导入数据

```
SELECT * FROM json_test;
+------+------------------------------+
| id   | c_json                       |
+------+------------------------------+
|    1 | {"name":"tom","age":35}      |
|    2 | {"name":null,"age":28}       |
|    3 | {"name":"micheal","age":null} |
|    4 | {"name":null,"age":null}     |
|    5 | null                         |
+------+------------------------------+
5 rows in set (0.01 sec)
```

2.9.5.4.2 JSON 格式导入

第 1 步：准备数据

创建如下的 JSON 文件， `test_json.json`

```
[
    {"id": 1, "c_json": {"name": "tom", "age": 35}},
    {"id": 2, "c_json": {"name": null, "age": 28}},
    {"id": 3, "c_json": {"name": "micheal", "age": null}},
    {"id": 4, "c_json": {"name": null, "age": null}},
    {"id": 5, "c_json": null}
]
```

第 2 步：在数据库中建表

```
CREATE TABLE json_test (
    id          INT     NOT NULL,
    c_json      JSON    NULL
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

第 3 步：导入数据

```
curl --location-trusted \
        -u "root":"" \
        -H "format:json" \
        -H "columns: id, c_json" \
        -H "strip_outer_array:true" \
        -T "test_json.json" \
        http://localhost:8040/api/testdb/json_test/_stream_load
```

第 4 步：检查导入数据

```
mysql> SELECT * FROM json_test;
+------+--------------------------------+
| id   | c_json                         |
+------+--------------------------------+
|    1 | {"name":"tom","age":35}        |
|    2 | {"name":null,"age":28}         |
|    3 | {"name":"micheal","age":null}  |
|    4 | {"name":null,"age":null}       |
|    5 | NULL                           |
+------+--------------------------------+
5 rows in set (0.01 sec)
```

2.9.5.5 Bitmap

BITMAP 类型可以在 Duplicate 表、Unique 表、Aggregate 表中使用，只能作为 Value 列，无法作为 Key 列使用。在 Aggregate 表中使用 BITMAP 类型，其建表时必须使用聚合类型 BITMAP_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。更多文档参考 Bitmap。

2.9.5.5.1 使用示例

第 1 步：准备数据

创建如下的 csv 文件：test_bitmap.csv

```
1|koga|17723
2|nijg|146285
3|lojn|347890
4|lofn|489871
5|jfin|545679
6|kon|676724
7|nhga|767689
8|nfubg|879878
9|huang|969798
10|buag|97997
```

第 2 步：在库中创建表

```
CREATE TABLE testdb.test_bitmap(
    typ_id     BIGINT                NULL    COMMENT "ID",
    hou        VARCHAR(10)           NULL    COMMENT "one",
    arr        BITMAP  BITMAP_UNION  NOT NULL    COMMENT "two"
)
AGGREGATE KEY(typ_id,hou)
DISTRIBUTED BY HASH(typ_id,hou) BUCKETS 10;
```

第 3 步：导入数据

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:|" \
    -H "columns:typ_id,hou,arr,arr=to_bitmap(arr)" \
    -T test_bitmap.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_bitmap/_stream_load
```

第 4 步：检查导入数据

```
mysql> select typ_id,hou,bitmap_to_string(arr) from testdb.test_bitmap;
+--------+-------+-----------------------+
| typ_id | hou   | bitmap_to_string(arr) |
+--------+-------+-----------------------+
|      4 | lofn  | 489871                |
|      6 | kon   | 676724                |
|      9 | huang | 969798                |
|      3 | lojn  | 347890                |
|      8 | nfubg | 879878                |
|      7 | nhga  | 767689                |
|      1 | koga  | 17723                 |
|      2 | nijg  | 146285                |
|      5 | jfin  | 545679                |
|     10 | buag  | 97997                 |
+--------+-------+-----------------------+
```

```
10 rows in set (0.07 sec)
```

2.9.5.5.2 导入含有多个元素的 bitmap

以下展示了 stream load 导入含有多个元素的 bitmap 列的两种方法，用户可以根据自己源文件格式选择合适的方法。

bitmap_from_string

使用 `bitmap_from_string` 导入不允许源文件中 arr 列存在方括号，否则会认为是数据质量错误。

```
1|koga|17,723
2|nijg|146,285
3|lojn|347,890
4|lofn|489,871
5|jfin|545,679
6|kon|676,724
7|nhga|767,689
8|nfubg|879,878
9|huang|969,798
10|buag|97,997
```

stream load 命令

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:|" \
    -H "columns:typ_id,hou,arr,arr=bitmap_from_string(arr)" \
    -T test_bitmap.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_bitmap/_stream_load
```

bitmap_from_array

使用 `bitmap_from_array` 导入允许源文件中 arr 列存在方括号，但是在 stream load 中必须先将 string 类型 cast 成 array 类型使用。如果不加 cast 转换参数类型，会因为找不到正确的函数签名报错 `[ANALYSIS_ERROR]TStatus ↪ : errCode = 2, detailMessage = Does not support non-builtin functions, or function does not ↪ exist: bitmap_from_array(<slot 8>)`。

```
1|koga|[17,723]
2|nijg|[146,285]
3|lojn|[347,890]
4|lofn|[489,871]
5|jfin|[545,679]
6|kon|[676,724]
7|nhga|[767,689]
8|nfubg|[879,878]
9|huang|[969,798]
10|buag|[97,997]
```

stream load 命令

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:|" \
    -H "columns:typ_id,hou,arr_str,arr=bitmap_from_array(cast(arr_str as array<int>))" \
    -T test_bitmap.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_bitmap/_stream_load
```

2.9.5.6 HLL

HLL 是用作模糊去重，在数据量大的情况性能优于 Count Distinct。 HLL 的导入需要结合 hll_hash 等函数来使用。更多文档参考HLL。

2.9.5.6.1 使用示例

第 1 步：准备数据

创建如下的 csv 文件： test_hll.csv

```
1001|koga
1002|nijg
1003|lojn
1004|lofn
1005|jfin
1006|kon
1007|nhga
1008|nfubg
1009|huang
1010|buag
```

第 2 步：在库中创建表

```
CREATE TABLE testdb.test_hll(
    typ_id           BIGINT          NULL    COMMENT "ID",
    typ_name         VARCHAR(10)     NULL    COMMENT "NAME",
    pv               hll hll_union   NOT NULL    COMMENT "hll"
)
AGGREGATE KEY(typ_id,typ_name)
DISTRIBUTED BY HASH(typ_id) BUCKETS 10;
```

第 3 步：导入数据

```
curl --location-trusted -u <doris_user>:<doris_password> \
    -H "column_separator:|" \
    -H "columns:typ_id,typ_name,pv=hll_hash(typ_id)" \
    -T test_hll.csv \
    -XPUT http://<fe_ip>:<fe_http_port>/api/testdb/test_hll/_stream_load
```

第 4 步：检查导入数据

使用 hll_cardinality 进行查询：

```
mysql> select typ_id,typ_name,hll_cardinality(pv) from testdb.test_hll;
+--------+----------+---------------------+
| typ_id | typ_name | hll_cardinality(pv) |
+--------+----------+---------------------+
|   1010 | buag     |                   1 |
|   1002 | nijg     |                   1 |
|   1001 | koga     |                   1 |
|   1008 | nfubg    |                   1 |
|   1005 | jfin     |                   1 |
|   1009 | huang    |                   1 |
|   1004 | lofn     |                   1 |
|   1007 | nhga     |                   1 |
|   1003 | lojn     |                   1 |
|   1006 | kon      |                   1 |
+--------+----------+---------------------+
10 rows in set (0.06 sec)
```

2.9.5.7 Variant

VARIANT 类型可以存储半结构化的 JSON 数据，允许存储包含不同数据类型（如整数、字符串、布尔值等）的复杂数据结构，而无需在表结构中预先定义具体的列。该类型特别适合处理复杂的嵌套结构，这些结构可能会随时发生变化。在写入过程中，VARIANT 类型能够自动推断列的结构和类型，动态合并写入的 schema，并通过将 JSON 键及其对应的值存储为列和动态子列。更多文档请参考VARIANT。

2.9.5.7.1 使用限制

支持 CSV 和 JSON 格式。

2.9.5.7.2 CSV 格式导入

第 1 步：准备数据

创建名为 `test_variant.csv` 的 CSV 文件，内容如下：

```
14186154924|PushEvent|{"avatar_url":"https://avatars.githubusercontent.com/u/282080?","display_
    ↪ login":"brianchandotcom","gravatar_id":"","id":282080,"login":"brianchandotcom","url":"
    ↪ https://api.github.com/users/brianchandotcom"}|{"id":1920851,"name":"brianchandotcom/
    ↪ liferay-portal","url":"https://api.github.com/repos/brianchandotcom/liferay-portal"}|{"
    ↪ before":"abb58cc0db673a0bd5190000d2ff9c53bb51d04d","commits":[""],"distinct_size":4,"head
    ↪ ":"91edd3c8c98c214155191feb852831ec535580ba","push_id":6027092734,"ref":"refs/heads/
    ↪ master","size":4}|1|2020-11-14 02:00:00
```

第 2 步：在库中创建表

执行以下 SQL 语句创建表：

```
CREATE TABLE IF NOT EXISTS testdb.test_variant (
    id BIGINT NOT NULL,
    type VARCHAR(30) NULL,
    actor VARIANT NULL,
    repo VARIANT NULL,
    payload VARIANT NULL,
    public BOOLEAN NULL,
    created_at DATETIME NULL,
    INDEX idx_payload (`payload`) USING INVERTED PROPERTIES("parser" = "english") COMMENT '
        ↪ inverted index for payload'
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(id) BUCKETS 10
properties("replication_num" = "1");
```

第 3 步：导入数据

以 stream load 为例，使用以下命令导入数据：

```
curl --location-trusted -u root:  -T test_variant.csv -H "column_separator:|" http
    ↪ ://127.0.0.1:8030/api/testdb/test_variant/_stream_load
```

导入结果示例：


```
{
    "TxnId": 12,
    "Label": "96cd6250-9c78-4a9f-b8b3-2b7cef0dd606",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 1,
    "NumberLoadedRows": 1,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 660,
    "LoadTimeMs": 213,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 6,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 183,
    "ReceiveDataTimeMs": 14,
    "CommitAndPublishTimeMs": 20
}
```


第 4 步：检查导入数据

使用以下 SQL 查询检查导入的数据：

```
mysql> select * from testdb.test_variant`\G`
*************************** 1. row ***************************
        id: 14186154924
      type: PushEvent
     actor: {"avatar_url":"https://avatars.githubusercontent.com/u/282080?","display_login":"
         ↪ brianchandotcom","gravatar_id":"","id":282080,"login":"brianchandotcom","url":"https
         ↪ ://api.github.com/users/brianchandotcom"}
      repo: {"id":1920851,"name":"brianchandotcom/liferay-portal","url":"https://api.github.com/
          ↪ repos/brianchandotcom/liferay-portal"}
   payload: {"before":"abb58cc0db673a0bd5190000d2ff9c53bb51d04d","commits":[""],"distinct_size"
       ↪ :4,"head":"91edd3c8c98c214155191feb852831ec535580ba","push_id":6027092734,"ref":"refs/
       ↪ heads/master","size":4}
    public: 1
created_at: 2020-11-14 02:00:00
```

2.9.5.7.3 JSON 格式导入

第 1 步：准备数据

创建名为 `test_variant.json` 的 JSON 文件，内容如下：

```
{"id": "14186154924","type": "PushEvent","actor": {"id": 282080,"login":"brianchandotcom","
    ↪ display_login": "brianchandotcom","gravatar_id": "","url": "https://api.github.com/users/
    ↪ brianchandotcom","avatar_url": "https://avatars.githubusercontent.com/u/282080?"},"repo":
    ↪  {"id": 1920851,"name": "brianchandotcom/liferay-portal","url": "https://api.github.com/
    ↪ repos/brianchandotcom/liferay-portal"},"payload": {"push_id": 6027092734,"size": 4,"
    ↪ distinct_size": 4,"ref": "refs/heads/master","head": "91
    ↪ edd3c8c98c214155191feb852831ec535580ba","before": "
    ↪ abb58cc0db673a0bd5190000d2ff9c53bb51d04d","commits": [""]},"public": true,"created_at": "
    ↪ 2020-11-13T18:00:00Z"}
```

第 2 步：在库中创建表

执行以下 SQL 语句创建表：

```
CREATE TABLE IF NOT EXISTS testdb.test_variant (
    id BIGINT NOT NULL,
    type VARCHAR(30) NULL,
    actor VARIANT NULL,
    repo VARIANT NULL,
    payload VARIANT NULL,
    public BOOLEAN NULL,
    created_at DATETIME NULL,
    INDEX idx_payload (`payload`) USING INVERTED PROPERTIES("parser" = "english") COMMENT '
        ↪ inverted index for payload'
)
```

```
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(id) BUCKETS 10;
```

第 3 步：导入数据

以 stream load 为例，使用以下命令导入数据：

```
curl --location-trusted -u root:  -T test_variant.json -H "format:json"  http://127.0.0.1:8030/
    ↪ api/testdb/test_variant/_stream_load
```

导入结果示例：

```
{
    "TxnId": 12,
    "Label": "96cd6250-9c78-4a9f-b8b3-2b7cef0dd606",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 1,
    "NumberLoadedRows": 1,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 660,
    "LoadTimeMs": 213,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 6,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 183,
    "ReceiveDataTimeMs": 14,
    "CommitAndPublishTimeMs": 20
}
```

第 4 步：检查导入数据

使用以下 SQL 查询检查导入的数据：

```
mysql> select * from testdb.test_variant`\G`
*************************** 1. row ***************************
        id: 14186154924
      type: PushEvent
     actor: {"avatar_url":"https://avatars.githubusercontent.com/u/282080?","display_login":"
        ↪ brianchandotcom","gravatar_id":"","id":282080,"login":"brianchandotcom","url":"https
        ↪ ://api.github.com/users/brianchandotcom"}
      repo: {"id":1920851,"name":"brianchandotcom/liferay-portal","url":"https://api.github.com/
         ↪ repos/brianchandotcom/liferay-portal"}
   payload: {"before":"abb58cc0db673a0bd5190000d2ff9c53bb51d04d","commits":[""],"distinct_size"
      ↪ :4,"head":"91edd3c8c98c214155191feb852831ec535580ba","push_id":6027092734,"ref":"refs/
      ↪ heads/master","size":4}
```

```
    public: 1
created_at: 2020-11-14 02:00:00
```

第 5 步：检查类型推导

```
mysql> desc github_events;
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
| Field                                                      | Type       | Null | Key   |
    ↪ Default | Extra |
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
| id                                                         | BIGINT     | No   | true  | NULL
    ↪    |       |
| type                                                       | VARCHAR(*) | Yes  | false | NULL
    ↪    | NONE  |
| actor                                                      | VARIANT    | Yes  | false | NULL
    ↪    | NONE  |
| created_at                                                 | DATETIME   | Yes  | false | NULL
    ↪    | NONE  |
| payload                                                    | VARIANT    | Yes  | false | NULL
    ↪    | NONE  |
| public                                                     | BOOLEAN    | Yes  | false | NULL
    ↪    | NONE  |
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
6 rows in set (0.07 sec)

mysql> set describe_extend_variant_column = true;
Query OK, 0 rows affected (0.01 sec)

mysql> desc github_events;
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
| Field                                                      | Type       | Null | Key   |
    ↪ Default | Extra |
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
| id                                                         | BIGINT     | No   | true  | NULL
    ↪    |       |
| type                                                       | VARCHAR(*) | Yes  | false | NULL
```

```
    ↪     | NONE  |
| actor                                                         | VARIANT    | Yes  | false | NULL
    ↪     | NONE  |
| actor.avatar_url                                              | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| actor.display_login                                           | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| actor.id                                                      | INT        | Yes  | false | NULL
    ↪     | NONE  |
| actor.login                                                   | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| actor.url                                                     | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| created_at                                                    | DATETIME   | Yes  | false | NULL
    ↪     | NONE  |
| payload                                                       | VARIANT    | Yes  | false | NULL
    ↪     | NONE  |
| payload.action                                                | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| payload.before                                                | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| payload.comment.author_association                            | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
| payload.comment.body                                          | TEXT       | Yes  | false | NULL
    ↪     | NONE  |
....
+--
    ↪ ------------------------------------------------------------+------------+------+-------+---------+-------+
    ↪
406 rows in set (0.07 sec)
```

可以按照 Partition 来展示

```
DESCRIBE ${table_name} PARTITION ($partition_name);
```

2.9.6 异常数据处理

在导入过程中，源数据列与目标列的数据类型可能存在不一致的情况。导入过程会对这些类型不一致的数据进行转换，但在转换过程中可能会出现字段类型不匹配、字段超长、精度不匹配等问题，从而导致转换失败。

为了处理这些异常情况，Doris 提供了两个重要的控制参数：

- 严格模式 (strict_mode)：用于控制是否过滤转换失败的数据行。
- 最大过滤比例 (max_filter_ratio)：用于设置可容忍的异常数据所占总数据的最大比例。

2.9.6.1 严格模式

严格模式具有两个主要功能：1. 对导入过程中发生列类型转换失败的数据行进行过滤。2. 在部分列更新场景中，限制只能更新已存在的列。

2.9.6.1.1 列类型转换失败的过滤策略

根据严格模式的设置，系统会采取不同的处理策略：

- 关闭严格模式时：转换失败的字段将被设置为 NULL 值，包含这些 NULL 值的异常数据行会与正确的数据行一起导入。
- 开启严格模式时：系统会过滤掉转换失败的数据行，仅导入正确的数据行。这里的“转换失败”特指：原始数据非 NULL，但在列类型转换后结果为 NULL 的情况。需要注意的是，这里的列类型转换不包括使用函数计算得到的 NULL 值。
- 对于 NULL 值的处理：无论是正确的数据行还是异常的数据行都可能包含 NULL 值。如果目标列被定义为不允许 NULL 值，则包含 NULL 值的数据行都会被过滤掉。

1. 以列类型为 TinyInt 来举例：

原始数据类型	原始数据举例	转换为 TinyInt 后的值	严格模式	结果
空值	\N	NULL	开启或关闭	NULL
非空值	“abc” or 2000	NULL	开启	非法值（被过滤）
非空值	“abc”	NULL	关闭	NULL
非空值	1	1	开启或关闭	正确导入

1. 表中的列允许导入空值
2. abc 及 2000 在转换为 TinyInt 后，会因类型或精度问题变为 NULL。在严格模式开启的情况下，这类数据将会被过滤。而如果是关闭状态，则会导入 null。

2. 以列类型为 Decimal(1,0) 举例

原始数据类型	原始数据举例	转换为 Decimal 后的值	严格模式	结果
空值	\N	null	开启或关闭	NULL
非空值	aaa	NULL	开启	非法值（被过滤）
非空值	aaa	NULL	关闭	NULL
非空值	1 or 10	1 or 10	开启或关闭	正确导入

1. 表中的列允许导入空值

2. abc 在转换为 Decimal 后，会因类型问题变为 NULL。在严格模式开启的情况下，这类数据将会被过滤。而如果是关闭状态，则会导入 null。
3. 10 虽然是一个超过范围的值，但是因为其类型符合 decimal 的要求，所以严格模式对其不产生影响。

2.9.6.1.2 限定部分列更新只能更新已有的列

在严格模式下，部分列更新插入的每一行数据必须满足该行数据的 Key 在表中已经存在。而在非严格模式下，进行部分列更新时可以更新 Key 已经存在的行，也可以插入 Key 不存在的新行。

例如有表结构如下：

```
CREATE TABLE user_profile
(
    id               INT,
    name             VARCHAR(10),
    age              INT,
    city             VARCHAR(10),
    balance          DECIMAL(9, 0),
    last_access_time DATETIME
) ENGINE=OLAP
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "enable_unique_key_merge_on_write" = "true"
);
```

表中有一条数据如下：

```
mysql> select * from user_profile;
+------+-------+------+----------+---------+---------------------+
| id   | name  | age  | city     | balance | last_access_time    |
+------+-------+------+----------+---------+---------------------+
|    1 | kevin |   18 | shenzhen |     400 | 2023-07-01 12:00:00|
+------+-------+------+----------+---------+---------------------+
```

当用户使用严格模式的 Insert Into 部分列更新向表中插入上述数据时，由于开启了严格模式且第二、三行的数据的 key((3), (18)) 不在原表中，所以本次插入会失败：

```
SET enable_unique_key_partial_update=true;
SET enable_insert_strict = true;
INSERT INTO user_profile (id, balance, last_access_time) VALUES
(1, 500, '2023-07-03 12:00:01'),
(3, 23, '2023-07-03 12:00:02'),
(18, 9999999, '2023-07-03 12:00:03');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = Insert has filtered data in strict mode
```

当用户使用非严格模式的 Insert Into 部分列更新向表中插入如下数据时：

```
SET enable_unique_key_partial_update=true;
SET enable_insert_strict = false;
INSERT INTO user_profile (id, balance, last_access_time) VALUES
(1, 500, '2023-07-03 12:00:01'),
(3, 23, '2023-07-03 12:00:02'),
(18, 9999999, '2023-07-03 12:00:03');
```

表中原有的一条数据将会被更新，此外还向表中插入了两条新数据。对于插入的数据中用户没有指定的列，如果该列有默认值，则会以默认值填充；否则，如果该列可以为 NULL，则将以 NULL 值填充；否则本次插入不成功。

查询结果如下：

```
mysql> select * from user_profile;
+------+-------+------+----------+---------+---------------------+
| id   | name  | age  | city     | balance | last_access_time    |
+------+-------+------+----------+---------+---------------------+
|    1 | kevin |   18 | shenzhen |     500 | 2023-07-03 12:00:01 |
|    3 | NULL  | NULL | NULL     |      23 | 2023-07-03 12:00:02 |
|   18 | NULL  | NULL | NULL     | 9999999 | 2023-07-03 12:00:03 |
+------+-------+------+----------+---------+---------------------+
```

2.9.6.1.3 开启严格模式

严格模式（strict_mode）默认为 False，以下是各种导入方式的设置方法：

Stream Load

```
 curl --location-trusted -u user:passwd \
 -H "strict_mode: true" \
 -T data.txt \
 http://host:port/api/example_db/test_table/_stream_load
```

Broker Load

```
LOAD LABEL example_db.label_1
(
    DATA INFILE("s3://bucket/data.txt")
    INTO TABLE test_table
)
WITH S3 (...)
PROPERTIES
(
    "strict_mode" = "true"
);
```

Routine Load

```
CREATE ROUTINE LOAD example_db.job1 ON test_table
PROPERTIES
(
    "strict_mode" = "true"
)
FROM KAFKA (...);
```

MySQL Load

```
LOAD DATA LOCAL INFILE 'data.txt'
INTO TABLE test_table
PROPERTIES
(
    "strict_mode" = "true"
);
```

Insert Into

```
SET enable_insert_strict = true;
INSERT INTO test_table ...;
```

2.9.6.2 最大过滤比例

最大过滤比例（max_filter_ratio）是一个重要的导入控制参数，它定义了导入过程中可以容忍的异常数据所占总数据的最大比例。如果实际过滤比例低于设定的最大过滤比例，导入任务将继续执行，异常数据会被忽略；如果超过这个比例，导入任务将失败。

2.9.6.2.1 过滤比例计算方法

- Filtered Rows 因数据质量不合格而被过滤掉的行。数据质量不合格包括类型错误、精度错误、字符串长度超长、文件列数不匹配等数据格式问题，以及因没有对应的分区而被过滤掉的数据行。
- Unselected Rows 这部分为因前置过滤或后置过滤条件而被过滤掉的数据行。
- Loaded Rows 被正确导入的数据行。

过滤比例的计算公式为：

```
###Filtered Rows / (#Filtered Rows + #Loaded Rows)
```

也就是说 Unselected Rows 不会参与过滤比例的计算。

2.9.6.2.2 最大过滤比例设置

最大过滤比例（max_filter_ratio）默认为 0，表示不允许任何异常数据。以下是各种导入方式的设置方法：

Stream Load

```
curl --location-trusted -u user:passwd \
-H "max_filter_ratio: 0.1" \
-T data.txt \
http://host:port/api/example_db/my_table/_stream_load
```

Broker Load

```
LOAD LABEL example_db.label_1
(
    DATA INFILE("s3://bucket/data.txt")
    INTO TABLE test_table
)
WITH S3 (...)
PROPERTIES
(
    "max_filter_ratio" = "0.1"
);
```

Routine Load

```
CREATE ROUTINE LOAD example_db.job1 ON test_table
PROPERTIES
(
    "max_filter_ratio" = "0.1"
)
FROM KAFKA (...);
```

MySQL Load

```
LOAD DATA LOCAL INFILE 'data.txt'
INTO TABLE test_table
PROPERTIES
(
    "max_filter_ratio" = "0.1"
);
```

Insert Into

```
SET insert_max_filter_ratio = 0.1;
INSERT INTO test_table FROM S3/HDFS/LOCAL();
```

> 对于 Insert Into 语句，`insert_max_filter_ratio` 仅在 `enable_insert_strict = false` 时生效，且只适用于 `INSERT INTO FROM S3/HDFS/LOCAL()` 语法。默认值为 1.0，表示允许所有异常数据被过滤。

2.9.7 导入时实现数据转换

Doris 在数据导入时提供了强大的数据转换能力，可以简化部分数据处理流程，减少对额外 ETL 工具的依赖。主要支持以下四种转换方式：

- 列映射：将源数据列映射到目标表的不同列。
- 列变换：使用函数和表达式对源数据进行实时转换。
- 前置过滤：在列映射和列变换前过滤掉不需要的原始数据。
- 后置过滤：在列映射和列变换后对数据最终结果进行过滤。

通过这些内置的数据转换功能，可以提高导入效率，并确保数据处理逻辑的一致性。

2.9.7.1 导入语法

2.9.7.1.1 Stream Load

通过在 HTTP header 中设置以下参数实现数据转换：

参数	说明
`columns`	指定列映射和列变换
`where`	指定后置过滤

> 注意: Stream Load 不支持前置过滤。

示例：

```
curl --location-trusted -u user:passwd \
    -H "columns: k1, k2, tmp_k3, k3 = tmp_k3 + 1" \
    -H "where: k1 > 1" \
    -T data.csv \
    http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

2.9.7.1.2 Broker Load

在 SQL 语句中通过以下子句实现数据转换：

子句	说明
column list	指定列映射，格式为 (k1, k2, tmp_k3)
SET	指定列变换
PRECEDING FILTER	指定前置过滤
WHERE	指定后置过滤

示例：

```
LOAD LABEL test_db.label1
(
    DATA INFILE("s3://bucket_name/data.csv")
    INTO TABLE `test_tbl`
    (k1, k2, tmp_k3)
    PRECEDING FILTER k1 = 1
    SET (
        k3 = tmp_k3 + 1
    )
    WHERE k1 > 1
)
WITH S3 (...);
```

2.9.7.1.3 Routine Load

在 SQL 语句中通过以下子句实现数据转换：

子句	说明
COLUMNS	指定列映射和列变换
PRECEDING FILTER	指定前置过滤
WHERE	指定后置过滤

示例：

```
CREATE ROUTINE LOAD test_db.label1 ON test_tbl
    COLUMNS(k1, k2, tmp_k3, k3 = tmp_k3 + 1),
    PRECEDING FILTER k1 = 1,
    WHERE k1 > 1
    ...
```

2.9.7.1.4 Insert Into

Insert Into 可以直接在 SELECT 语句中完成数据转换，使用 WHERE 子句实现数据过滤。

2.9.7.2 列映射

列映射用于定义源数据列与目标表列之间的对应关系，能够处理以下场景：- 源数据与目标表的列顺序不一致 - 源数据与目标表的列数量不匹配

2.9.7.2.1 实现原理

列映射的实现可以分为两个核心步骤：

- 步骤 1：数据源解析 - 根据数据格式将原始数据解析为中间变量
- 步骤 2：通过列映射进行赋值 - 将中间变量按列名映射到目标表字段

以下是三种不同数据格式的处理流程：

导入 CSV 格式数据

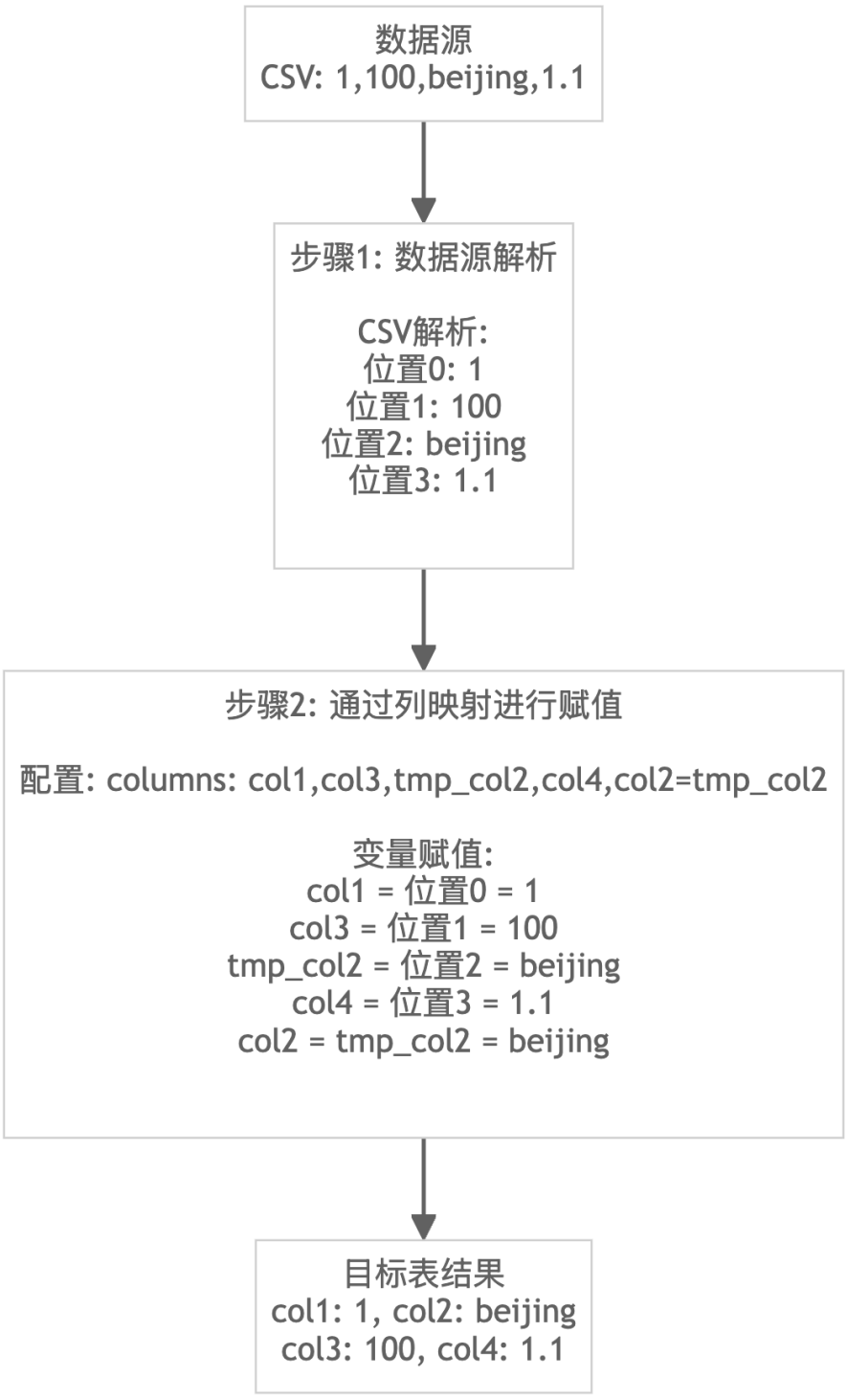


图 39:

指定 jsonpaths 导入 JSON 格式数据

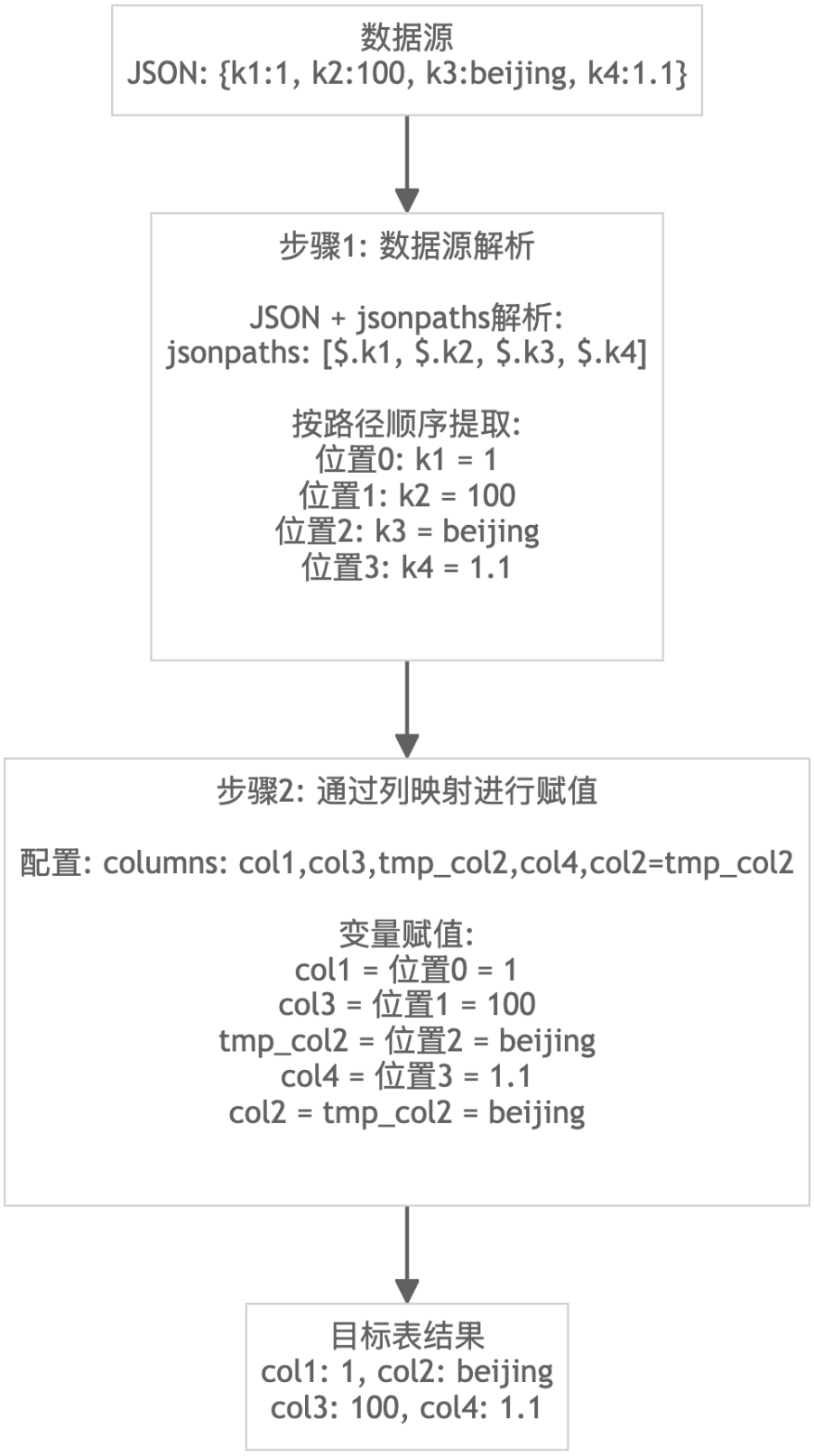

数据源
JSON: {k1:1, k2:100, k3:beijing, k4:1.1}
步骤1: 数据源解析
JSON + jsonpaths解析:
jsonpaths: [$.k1, $.k2, $.k3, $.k4]
按路径顺序提取:
位置0: k1 = 1
位置1: k2 = 100
位置2: k3 = beijing
位置3: k4 = 1.1
步骤2: 通过列映射进行赋值
配置: columns: col1,col3,tmp_col2,col4,col2=tmp_col2
变量赋值:
col1 = 位置0 = 1
col3 = 位置1 = 100
tmp_col2 = 位置2 = beijing
col4 = 位置3 = 1.1
col2 = tmp_col2 = beijing
目标表结果
col1: 1, col2: beijing
col3: 100, col4: 1.1

图 40:

不指定 jsonpaths 导入 JSON 格式数据

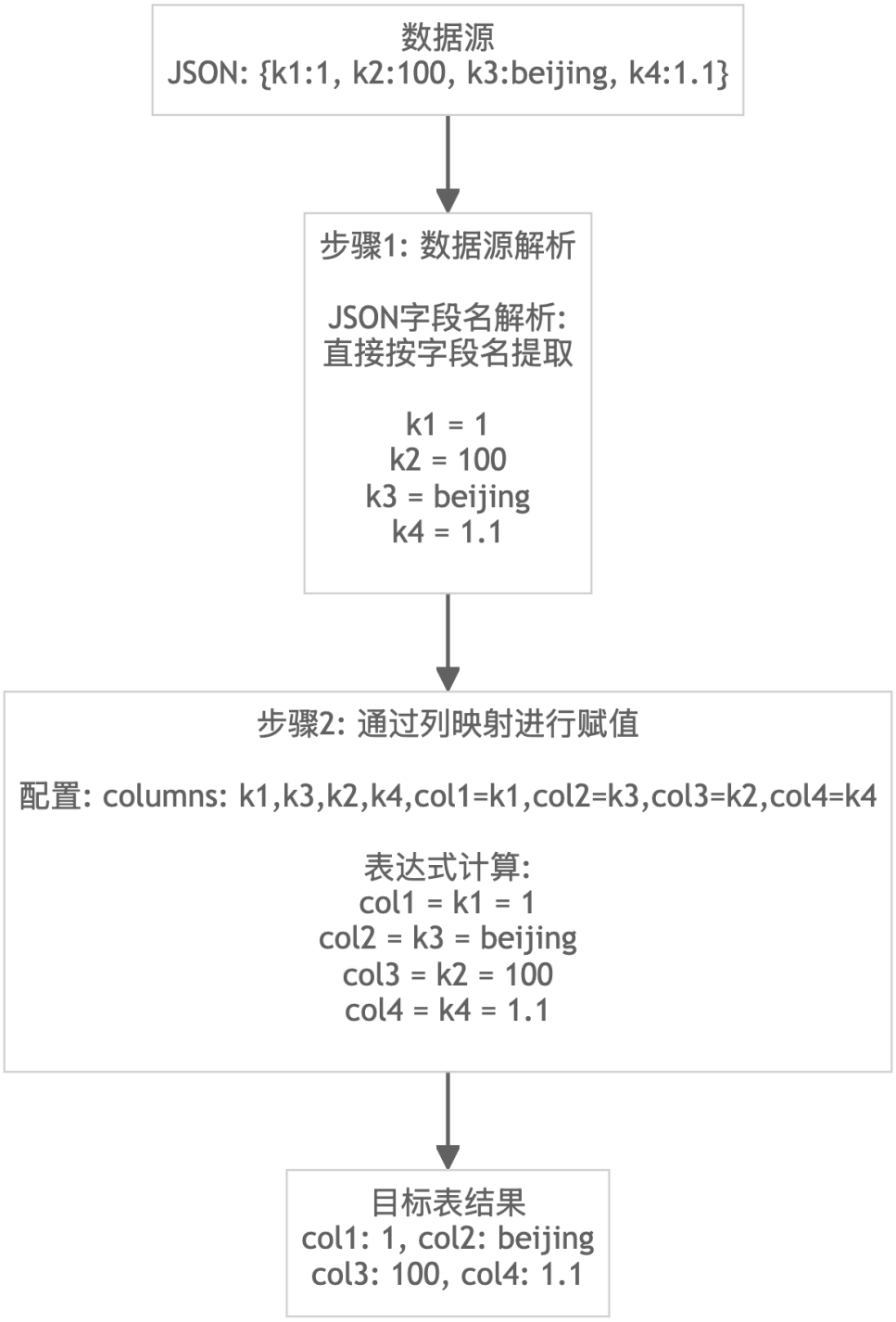


图 41:

2.9.7.2.2 指定 jsonpaths 导入 JSON 数据

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
{"k1":1,"k2":"100","k3":"beijing","k4":1.1}
{"k1":2,"k2":"200","k3":"shanghai","k4":1.2}
{"k1":3,"k2":"300","k3":"guangzhou","k4":1.3}
{"k1":4,"k2":"\`\N`","k3":"chongqing","k4":1.4}
```

创建目标表

```
CREATE TABLE example_table
(
    col1 INT,
    col2 STRING,
    col3 INT,
    col4 DOUBLE
) ENGINE = OLAP
DUPLICATE KEY(col1)
DISTRIBUTED BY HASH(col1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "columns:col1, col3, col2, col4" \
-H "jsonpaths:[\"$.k1\", \"$.k2\", \"$.k3\", \"$.k4\"]" \
-H "format:json" \
-H "read_json_by_line:true" \
-T data.json \
-X PUT \
http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label_broker
(
DATA INFILE("s3://bucket_name/data.json")
INTO TABLE example_table
FORMAT AS "json"
(col1, col3, col2, col4)
PROPERTIES
(
    "jsonpaths" = "[\"$.k1\", \"$.k2\", \"$.k3\", \"$.k4\"]"
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(col1, col3, col2, col4)
PROPERTIES
(
"format" = "json",
"jsonpaths" = "[\"$.k1\", \"$.k2\", \"$.k3\", \"$.k4\"]",
"read_json_by_line" = "true"
)
FROM KAFKA (...);
```

查询结果

```
mysql> SELECT * FROM example_table;
+------+-----------+------+------+
| col1 | col2      | col3 | col4 |
+------+-----------+------+------+
|    1 | beijing   |  100 |  1.1 |
|    2 | shanghai  |  200 |  1.2 |
|    3 | guangzhou |  300 |  1.3 |
|    4 | chongqing | NULL |  1.4 |
+------+-----------+------+------+
```

2.9.7.2.3 不指定 jsonpaths 导入 JSON 数据

假设有以下源数据（表头列名仅为方便表述，实际并无表头）:

```
{"k1":1,"k2":"100","k3":"beijing","k4":1.1}
{"k1":2,"k2":"200","k3":"shanghai","k4":1.2}
{"k1":3,"k2":"300","k3":"guangzhou","k4":1.3}
{"k1":4,"k2":"\`\N`","k3":"chongqing","k4":1.4}
```

创建目标表

```
CREATE TABLE example_table
(
    col1 INT,
    col2 STRING,
    col3 INT,
    col4 DOUBLE
) ENGINE = OLAP
DUPLICATE KEY(col1)
DISTRIBUTED BY HASH(col1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "columns:k1, k3, k2, k4,col1 = k1, col2 = k3, col3 = k2, col4 = k4" \
-H "format:json" \
-H "read_json_by_line:true" \
-T data.json \
-X PUT \
http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label_broker
(
DATA INFILE("s3://bucket_name/data.json")
INTO TABLE example_table
FORMAT AS "json"
(k1, k3, k2, k4)
SET (
    col1 = k1,
    col2 = k3,
    col3 = k2,
    col4 = k4
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k3, k2, k4, col1 = k1, col2 = k3, col3 = k2, col4 = k4),
PROPERTIES
(
"format" = "json",
"read_json_by_line" = "true"
)
FROM KAFKA (...);
```

查询结果

```
mysql> SELECT * FROM example_table;
+------+-----------+------+------+
| col1 | col2      | col3 | col4 |
+------+-----------+------+------+
|    1 | beijing   |  100 |  1.1 |
|    2 | shanghai  |  200 |  1.2 |
```

```
|    3 | guangzhou |  300 |  1.3 |
|    4 | chongqing | NULL |  1.4 |
+------+-----------+------+------+
```

2.9.7.2.4 调整列顺序

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

目标表有 k1, k2, k3, k4 四列，要实现如下映射：

```
列 1 -> k1
列 2 -> k3
列 3 -> k2
列 4 -> k4
```

创建目标表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 STRING,
    k3 INT,
    k4 DOUBLE
) ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1,k3,k2,k4" \
-T data.csv \
-X PUT \
http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label_broker
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k3, k2, k4)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k3, k2, k4),
COLUMNS TERMINATED BY ","
FROM KAFKA (...);
```

查询结果

```
mysql> select * from example_table;
+------+-----------+------+------+
| k1   | k2        | k3   | k4   |
+------+-----------+------+------+
|    2 | shanghai  |  200 |  1.2 |
|    4 | chongqing | NULL |  1.4 |
|    3 | guangzhou |  300 |  1.3 |
|    1 | beijing   |  100 |  1.1 |
+------+-----------+------+------+
```

2.9.7.2.5 源文件列数量多于表列数

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,``\N``,chongqing,1.4
```

目标表有 k1, k2, k3 三列，而源文件包含四列数据。我们只需要源文件的第 1、第 2、第 4 列，映射关系如下：

```
列 1 -> k1
列 2 -> k2
列 4 -> k3
```

要跳过源文件中的某些列，只需在列映射时使用任意不存在于目标表的列名。这些列名可以自定义，不受限制，导入时会自动忽略这些列的数据。

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 STRING,
    k3 DOUBLE
) ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:password \
-H "column_separator:," \
-H "columns: k1,k2,tmp_skip,k3" \
-T data.csv \
http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label_broker
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(tmp_k1, tmp_k2, tmp_skip, tmp_k3)
SET (
    k1 = tmp_k1,
    k2 = tmp_k2,
    k3 = tmp_k3
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, tmp_skip, k3),
PROPERTIES
(
"format" = "csv",
"column_separator" = ","
)
FROM KAFKA (...);
```

注意：示例中的 tmp_skip 可以替换为任意名称，只要这些名称不在目标表的列定义中即可。

查询结果

```
mysql> select * from example_table;
+------+------+------+
| k1   | k2   | k3   |
+------+------+------+
|    1 | 100  |  1.1 |
|    2 | 200  |  1.2 |
|    3 | 300  |  1.3 |
|    4 | NULL |  1.4 |
+------+------+------+
```

2.9.7.2.6 源文件列数量少于表列数

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

目标表有 k1, k2, k3, k4, k5 五列，而源文件包含四列数据。我们只需要源文件的第 1、第 2、第 3、第 4 列，映射关系如下：

```
列 1 -> k1
列 2 -> k3
列 3 -> k2
列 4 -> k4
k5 使用默认值
```

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 STRING,
    k3 INT,
    k4 DOUBLE,
    k5 INT DEFAULT 2
) ENGINE = OLAP
DUPLICATE KEY(k1)
```

```
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1,k3,k2,k4" \
-T data.csv \
http://<fe_ip>:<fe_http_port>/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label_broker
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(tmp_k1, tmp_k3, tmp_k2, tmp_k4)
SET (
    k1 = tmp_k1,
    k3 = tmp_k3,
    k2 = tmp_k2,
    k4 = tmp_k4
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k3, k2, k4),
COLUMNS TERMINATED BY ","
FROM KAFKA (...);
```

说明：- 如果 k5 列有默认值，将使用默认值填充 - 如果 k5 列是可空列（nullable）但没有默认值，将填充 NULL 值 - 如果 k5 列是非空列且没有默认值，导入会失败

查询结果

```
mysql> select * from example_table;
+------+-----------+------+------+------+
| k1   | k2        | k3   | k4   | k5   |
+------+-----------+------+------+------+
|    1 | beijing   |  100 |  1.1 |    2 |
|    2 | shanghai  |  200 |  1.2 |    2 |
```

```
|    3 | guangzhou |  300 |  1.3 |    2 |
|    4 | chongqing | NULL |  1.4 |    2 |
+------+-----------+------+------+------+
```

2.9.7.3 列变换

列变换功能允许用户对源文件中列值进行变换，支持使用绝大部分内置函数。列变换操作通常是和列映射一起定义的，即先对列进行映射，再进行变换。

2.9.7.3.1 将源文件中的列值经变换后导入表中

假设有以下源数据 (表头列名仅为方便表述，实际并无表头):

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列，导入映射和变换关系如下:

```
列 1       -> k1
列 2 * 100 -> k3
列 3       -> k2
列 4       -> k4
```

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 STRING,
    k3 INT,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, tmp_k3, k2, k4, k3 = tmp_k3 * 100" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, tmp_k3, k2, k4)
SET (
    k3 = tmp_k3 * 100
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, tmp_k3, k2, k4, k3 = tmp_k3 * 100),
COLUMNS TERMINATED BY ","
FROM KAFKA (...);
```

查询结果

```
mysql> select * from example_table;
+------+-----------+-------+------+
| k1   | k2        | k3    | k4   |
+------+-----------+-------+------+
|    1 | beijing   | 10000 |  1.1 |
|    2 | shanghai  | 20000 |  1.2 |
|    3 | guangzhou | 30000 |  1.3 |
|    4 | chongqing |  NULL |  1.4 |
+------+-----------+-------+------+
```

2.9.7.3.2 通过 case when 函数，有条件的进行列变换

假设有以下源数据（表头列名仅为方便表述，实际并无表头）:

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列。对于源数据中 beijing, shanghai, guangzhou, chongqing 分别转换为对应的地区 id 后导入：

```
列 1                    -> k1
列 2                    -> k2
```

```
列 3 进行地区 id 转换后     -> k3
列 4                    -> k4
```

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 INT,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, k2, tmp_k3, k4, k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN
    ↪  2 WHEN 'guangzhou' THEN 3 WHEN 'chongqing' THEN 4 ELSE NULL END" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k2, tmp_k3, k4)
SET (
    k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3
        ↪ WHEN 'chongqing' THEN 4 ELSE NULL END
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, tmp_k3, k4, k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN 2
    ↪ WHEN 'guangzhou' THEN 3 WHEN 'chongqing' THEN 4 ELSE NULL END),
```

```
COLUMNS TERMINATED BY ","
FROM KAFKA (...);
```

查询结果

```
mysql> select * from example_table;
+------+------+------+------+
| k1   | k2   | k3   | k4   |
+------+------+------+------+
|    1 |  100 |    1 |  1.1 |
|    2 |  200 |    2 |  1.2 |
|    3 |  300 |    3 |  1.3 |
|    4 | NULL |    4 |  1.4 |
+------+------+------+------+
```

2.9.7.3.3 源文件中的 NULL 值处理

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列。在对地区 id 转换的同时，对于源数据中 k1 列的 null 值转换成 0 导入：

```
列1                      -> k1
列2 如果为null 则转换成0   -> k2
列3                      -> k3
列4                      -> k4
```

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 INT,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, tmp_k2, tmp_k3, k4, k2 = ifnull(tmp_k2, 0), k3 = CASE tmp_k3 WHEN 'beijing'
    ↪  THEN 1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3 WHEN 'chongqing' THEN 4 ELSE
    ↪ NULL END" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, tmp_k2, tmp_k3, k4)
SET (
    k2 = ifnull(tmp_k2, 0),
    k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3
        ↪ WHEN 'chongqing' THEN 4 ELSE NULL END
)
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, tmp_k2, tmp_k3, k4, k2 = ifnull(tmp_k2, 0), k3 = CASE tmp_k3 WHEN 'beijing' THEN
    ↪  1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3 WHEN 'chongqing' THEN 4 ELSE NULL
    ↪ END),
COLUMNS TERMINATED BY ","
FROM KAFKA (...);
```

查询结果

```
mysql> select * from example_table;
+------+------+------+------+
| k1   | k2   | k3   | k4   |
+------+------+------+------+
|    1 |  100 |    1 |  1.1 |
|    2 |  200 |    2 |  1.2 |
|    3 |  300 |    3 |  1.3 |
|    4 |    0 |    4 |  1.4 |
+------+------+------+------+
```

2.9.7.4 前置过滤

前置过滤是在数据转换前对原始数据进行过滤的功能，可以提前过滤掉不需要处理的数据，减少后续处理的数据量，提高导入效率。该功能仅支持 Broker Load 和 Routine Load 两种导入方式。前置过滤有以下应用场景：

- 转换前做过滤

希望在列映射和转换前做过滤的场景，能够先行过滤掉部分不需要的数据。

- 过滤列不存在于表中，仅作为过滤标识

比如源数据中存储了多张表的数据（或者多张表的数据写入了同一个 Kafka 消息队列）。数据中每行有一列表名来标识该行数据属于哪个表。用户可以通过前置过滤条件来筛选对应的表数据进行导入。

前置过滤有以下限制：- 过滤列限制。

前置过滤只能对列表中的独立简单列进行过滤，无法对带有表达式的列进行过滤。如：在列映射为（a, tmp, b = tmp + 1）时，b 列无法作为过滤条件。

- 数据处理限制

前置过滤发生在数据转换之前，使用原始数据值进行比较，原始数据会视为字符串类型。如：对于 \N 这样的数据，会直接用 \N 字符串进行比较，而不会转换为 NULL 后再比较。

2.9.7.4.1 示例一：使用数值条件进行前置过滤

本示例展示如何使用简单的数值比较条件来过滤源数据。通过设置 k1 > 1 的过滤条件，实现在数据转换前过滤掉不需要的记录。

假设有以下源数据（表头列名仅为方便表述，实际并无表头）：

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

前置过滤条件为：

```
列1>1，即只导入 列1>1 的数据，其他数据过滤掉。
```

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 STRING,
    k4 DOUBLE
```

```
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k2, k3, k4)
PRECEDING FILTER k1 > 1
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, k3, k4),
COLUMNS TERMINATED BY ","
PRECEDING FILTER k1 > 1
FROM KAFKA (...)
```

查询结果

```
mysql> select * from example_table;
+------+------+-----------+------+
| k1   | k2   | k3        | k4   |
+------+------+-----------+------+
|    2 |  200 | shanghai  |  1.2 |
|    3 |  300 | guangzhou |  1.3 |
|    4 | NULL | chongqing |  1.4 |
+------+------+-----------+------+
```

2.9.7.4.2 示例二：使用中间列过滤无效数据

本示例展示如何处理包含无效数据的导入场景。

源数据为：`plain text 1,1 2,abc 3,3`

建表语句

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT NOT NULL
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

对于 k2 列，类型为 int，abc是不合法的脏数据，想要过滤该数据，可以引入中间列来过滤。

导入语句

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, tmp, k2 = tmp)
PRECEDING FILTER tmp != "abc"
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, tmp, k2 = tmp),
COLUMNS TERMINATED BY ","
PRECEDING FILTER tmp != "abc"
FROM KAFKA (...);
```

导入结果

```
mysql> select * from example_table;
+------+----+
| k1   | k2 |
+------+----+
|    1 |  1 |
|    3 |  3 |
+------+----+
```

2.9.7.5 后置过滤

后置过滤在数据转换后执行，可以根据转换后的结果进行过滤。

2.9.7.5.1 在列映射和转换缺省的情况下，直接过滤

假设有以下源数据 (表头列名仅为方便表述，实际并无表头):

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列，在缺省列映射和转换的情况下，只导入源文件中第 4 列为大于 1.2 的数据行。

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 STRING,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, k2, k3, k4" \
-H "where: k4 > 1.2" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k2, k3, k4)
where k4 > 1.2
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, k3, k4),
COLUMNS TERMINATED BY ","
WHERE k4 > 1.2;
FROM KAFKA (...)
```

查询结果

```
mysql> select * from example_table;
+------+------+-----------+------+
| k1   | k2   | k3        | k4   |
+------+------+-----------+------+
|    3 |  300 | guangzhou |  1.3 |
|    4 | NULL | chongqing |  1.4 |
+------+------+-----------+------+
```

2.9.7.5.2 对经过列变换的数据进行过滤

假设有以下源数据（表头列名仅为方便表述，实际并无表头）:

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列。在列变换示例中，我们将省份名称转换成了 id。这里我们希望过滤掉 id 为 3 的数据

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 INT,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, k2, tmp_k3, k4, k3 = case tmp_k3 when 'beijing' then 1 when 'shanghai' then
    ↪  2 when 'guangzhou' then 3 when 'chongqing' then 4 else null end" \
-H "where: k3 != 3" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k2, tmp_k3, k4)
SET (
    k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3
        ↪ WHEN 'chongqing' THEN 4 ELSE NULL END
)
WHERE k3 != 3
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, tmp_k3, k4),
COLUMNS TERMINATED BY ","
SET (
k3 = CASE tmp_k3 WHEN 'beijing' THEN 1 WHEN 'shanghai' THEN 2 WHEN 'guangzhou' THEN 3 WHEN '
    ↪ chongqing' THEN 4 ELSE NULL END
)
WHERE k3 != 3;
FROM KAFKA (...)
```

查询结果

```
mysql> select * from example_table;
+------+------+------+------+
| k1   | k2   | k3   | k4   |
+------+------+------+------+
|    1 |  100 |    1 |  1.1 |
|    2 |  200 |    2 |  1.2 |
|    4 | NULL |    4 |  1.4 |
+------+------+------+------+
```

2.9.7.5.3 多条件过滤

假设有以下源数据 (表头列名仅为方便表述，实际并无表头):

```
列 1，列 2，列 3，列 4
1,100,beijing,1.1
2,200,shanghai,1.2
3,300,guangzhou,1.3
4,`\N`,chongqing,1.4
```

表中有 k1,k2,k3,k4 4 列。过滤掉 k1 列为 null 的数据，同时过滤掉 k4 列小于 1.2 的数据

创建示例表

```
CREATE TABLE example_table
(
    k1 INT,
    k2 INT,
    k3 STRING,
    k4 DOUBLE
)
ENGINE = OLAP
DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS 1;
```

导入数据

- Stream Load

```
curl --location-trusted -u user:passwd \
-H "column_separator:," \
-H "columns: k1, k2, k3, k4" \
-H "where: k1 is not null and k4 > 1.2" \
-T data.csv \
http://host:port/api/example_db/example_table/_stream_load
```

- Broker Load

```
LOAD LABEL example_db.label1
(
DATA INFILE("s3://bucket_name/data.csv")
INTO TABLE example_table
COLUMNS TERMINATED BY ","
(k1, k2, k3, k4)
where k1 is not null and k4 > 1.2
)
WITH s3 (...);
```

- Routine Load

```
CREATE ROUTINE LOAD example_db.example_routine_load ON example_table
COLUMNS(k1, k2, k3, k4),
COLUMNS TERMINATED BY ","
WHERE k1 is not null and k4 > 1.2
FROM KAFKA (...);
```

查询结果

```
mysql> select * from example_table;
+------+------+-----------+------+
| k1   | k2   | k3        | k4   |
+------+------+-----------+------+
|    3 |  300 | guangzhou |  1.3 |
|    4 | NULL | chongqing |  1.4 |
+------+------+-----------+------+
```

2.9.8 导入高可用性

2.9.8.1 概述

Doris 在数据导入过程中提供了多种机制来确保高可用性。本文将详细介绍 Doris 的默认导入行为以及为提高导入可用性而提供的额外选项，特别是最小写入副本数功能。

2.9.8.2 多数派写入

默认情况下，Doris 采用多数派写入策略来确保数据的可靠性和一致性：

- 当成功写入的副本数超过总副本数的一半时，导入被视为成功。
- 例如，对于三副本的表，至少需要两个副本写入成功才算导入成功。

2.9.8.2.1 工作原理

1. 数据分发：导入任务首先将数据分发到所有相关的 BE 节点。
2. 并行写入：各个 BE 节点并行处理数据写入操作。
3. 写入确认：每个 BE 节点在完成数据写入后，会向 FE 发送确认信息。
4. 多数派判断：FE 统计成功写入的副本数，当达到多数派时，认为导入成功。
5. 事务提交：FE 提交导入事务，使数据对外可见。
6. 异步复制：对于未成功写入的副本，系统会在后台异步进行数据复制，以确保最终所有副本的数据一致性。

多数派写入策略是 Doris 在数据可靠性和系统可用性之间的一个平衡。对于有特殊需求的场景，Doris 提供了最小写入副本数等其他选项来进一步提高系统的灵活性。

2.9.8.3 最小写入副本数

多数派写入策略在保证数据可靠性的同时，也可能在某些场景下影响系统的可用性。例如，在两副本的情况下，必须两个副本都写入成功才能完成导入，这意味着在导入过程中不允许任何一个副本不可用。

为了解决上述问题并提高导入的可用性，Doris 提供了最小写入副本数（Min Load Replica Num）选项。

2.9.8.3.1 功能说明

最小写入副本数允许用户指定导入数据时需要成功写入的最少副本数。当成功写入的副本数大于或等于这个值时，导入即视为成功。

2.9.8.3.2 使用场景

- 在部分节点不可用时，仍需要保证数据能够成功导入。
- 对数据导入速度有较高要求，愿意在一定程度上牺牲一致性来换取更高的可用性。

2.9.8.3.3 配置方法

1. 单表配置

a. 创建表时设置：

```
CREATE TABLE example_table
(
id INT,
name STRING
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 10
PROPERTIES
(
'replication_num' = '3',
'min_load_replica_num' = '2'
);
```

b. 修改现有表：

```
ALTER TABLE example_table
SET ( 'min_load_replica_num' = '2' );
```

2. 全局配置

通过 FE 配置项 `min_load_replica_num` 设置。

- 有效值：大于 0

- 默认值：-1（表示不开启全局最小写入副本数）

优先级：表属性 > 全局配置 > 默认多数派规则

如果表属性未设置或无效，且全局配置有效，则表的最小写入副本数为：`min(FE 配置的 min_load_replica_num，表的副本数 / 2 + 1)`

关于 FE 配置项的查看和修改，请参考FE 配置项文档。

2.9.8.4 其他高可用性机制

除了最小写入副本数选项，Doris 还采用了以下机制来提高导入的可用性：

1. 导入重试：自动重试因临时故障导致的失败导入任务。
2. 负载均衡：将导入任务分散到不同的 BE 节点，避免单点压力过大。
3. 事务机制：确保数据的一致性，失败时自动回滚。

2.9.9 高并发导入优化（Group Commit）

在高频小批量写入场景下，传统的导入方式存在以下问题：

- 每个导入都会创建一个独立的事务，都需要经过 FE 解析 SQL 和生成执行计划，影响整体性能
- 每个导入都会生成一个新的版本，导致版本数快速增长，增加了后台 compaction 的压力

为了解决这些问题，Doris 引入了 Group Commit 机制。Group Commit 不是一种新的导入方式，而是对现有导入方式的优化扩展，主要针对：

- `INSERT INTO tbl VALUES(...)` 语句
- Stream Load 导入

通过将多个小批量导入在后台合并成一个大的事务提交，显著提升了高并发小批量写入的性能。同时，Group Commit 与 PreparedStatement 结合使用可以获得更高的性能提升。

2.9.9.1 Group Commit 模式

Group Commit 写入有三种模式，分别是：

- 关闭模式（`off_mode`）

  不开启 Group Commit。

- 同步模式（`sync_mode`）

  Doris 根据负载和表的 `group_commit_interval`属性将多个导入在一个事务提交，事务提交后导入返回。这适用于高并发写入场景，且在导入完成后要求数据立即可见。

- 异步模式（`async_mode`）

  Doris 首先将数据写入 WAL（`Write Ahead Log`），然后导入立即返回。Doris 会根据负载和表的`group_commit_interval`属性异步提交数据，提交之后数据可见。为了防止 WAL 占用较大的磁盘空间，单次导入数据量较大时，会自动切换为`sync_mode`。这适用于写入延迟敏感以及高频写入的场景。

  WAL 的数量可以通过 FE http 接口查看，具体可见这里，也可以在 BE 的 metrics 中搜索关键词`wal`查看。

2.9.9.2 Group Commit 使用方式

假如表的结构为：

```
CREATE TABLE `dt` (
    `id` int(11) NOT NULL,
    `name` varchar(50) NULL,
    `score` int(11) NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
    "replication_num" = "1"
);
```

2.9.9.2.1 使用 JDBC

当用户使用 JDBC `insert into values`方式写入时，为了减少 SQL 解析和生成规划的开销，我们在 FE 端支持了 MySQL 协议的 `PreparedStatement` 特性。当使用 `PreparedStatement` 时，SQL 和其导入规划将被缓存到 Session 级别的内存缓存中，后续的导入直接使用缓存对象，降低了 FE 的 CPU 压力。下面是在 JDBC 中使用 `PreparedStatement` 的例子：

1. 设置 JDBC URL 并在 Server 端开启 Prepared Statement

```
url = jdbc:mysql://127.0.0.1:9030/db?useServerPrepStmts=true&useLocalSessionState=true&
    ↪ rewriteBatchedStatements=true&cachePrepStmts=true&prepStmtCacheSqlLimit=99999&
    ↪ prepStmtCacheSize=500
```

2. 配置 `group_commit` session 变量，有如下两种方式：

   - 通过 JDBC url 设置，增加`sessionVariables=group_commit=async_mode`

```
url = jdbc:mysql://127.0.0.1:9030/db?useServerPrepStmts=true&useLocalSessionState=true&
    ↪ rewriteBatchedStatements=true&cachePrepStmts=true&prepStmtCacheSqlLimit=99999&
    ↪ prepStmtCacheSize=500&sessionVariables=group_commit=async_mode,enable_nereids_planner=
    ↪ false
```

   - 通过执行 SQL 设置

```
try (Statement statement = conn.createStatement()) {
    statement.execute("SET group_commit = async_mode;");
}
```

3. 使用 `PreparedStatement`

```
private static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
private static final String URL_PATTERN = "jdbc:mysql://%s:%d/%s?useServerPrepStmts=true&
    ↪ useLocalSessionState=true&rewriteBatchedStatements=true&cachePrepStmts=true&
    ↪ prepStmtCacheSqlLimit=99999&prepStmtCacheSize=50&sessionVariables=group_commit=async_mode
    ↪ ";
private static final String HOST = "127.0.0.1";
private static final int PORT = 9087;
private static final String DB = "db";
private static final String TBL = "dt";
private static final String USER = "root";
private static final String PASSWD = "";
private static final int INSERT_BATCH_SIZE = 10;

private static void groupCommitInsertBatch() throws Exception {
    Class.forName(JDBC_DRIVER);
    // add rewriteBatchedStatements=true and cachePrepStmts=true in JDBC url
    // set session variables by sessionVariables=group_commit=async_mode in JDBC url
    try (Connection conn = DriverManager.getConnection(
            String.format(URL_PATTERN, HOST, PORT, DB), USER, PASSWD)) {

        String query = "insert into " + TBL + " values(?, ?, ?)";
        try (PreparedStatement stmt = conn.prepareStatement(query)) {
            for (int j = 0; j < 5; j++) {
                // 10 rows per insert
                for (int i = 0; i < INSERT_BATCH_SIZE; i++) {
                    stmt.setInt(1, i);
                    stmt.setString(2, "name" + i);
                    stmt.setInt(3, i + 10);
                    stmt.addBatch();
                }
                int[] result = stmt.executeBatch();
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

注意：由于高频的 insert into 语句会打印大量的 audit log，对最终性能有一定影响，默认关闭了打印 prepared 语句的 audit log。可以通过设置 session variable 的方式控制是否打印 prepared 语句的 audit log。

```
set enable_prepared_stmt_audit_log=true;
```

关于 JDBC 的更多用法，参考使用 Insert 方式同步数据。

2.9.9.2.2 使用 Golang 进行 Group Commit

Golang 的 prepared 语句支持有限，所以我们可以通过手动客户端攒批的方式提高 Group Commit 的性能，以下为一个示例程序。

```
package main

import (
    "database/sql"
    "fmt"
    "math/rand"
    "strings"
    "sync"
    "sync/atomic"
    "time"

    _ "github.com/go-sql-driver/mysql"
)

const (
    host     = "127.0.0.1"
    port     = 9038
    db       = "test"
    user     = "root"
    password = ""
    table    = "async_lineitem"
)

var (
    threadCount = 20
    batchSize   = 100
)

var totalInsertedRows int64
var rowsInsertedLastSecond int64

func main() {
    dbDSN := fmt.Sprintf("%s:%s@tcp(%s:%d)/%s?parseTime=true", user, password, host, port, db)
    db, err := sql.Open("mysql", dbDSN)
    if err != nil {
        fmt.Printf("Error opening database: %s`\n`", err)
        return
    }
    defer db.Close()

    var wg sync.WaitGroup
```

```
    for i := 0; i < threadCount; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            groupCommitInsertBatch(db)
        }()
    }

    go logInsertStatistics()

    wg.Wait()
}

func groupCommitInsertBatch(db *sql.DB) {
    for {
        valueStrings := make([]string, 0, batchSize)
        valueArgs := make([]interface{}, 0, batchSize*16)
        for i := 0; i < batchSize; i++ {
            valueStrings = append(valueStrings, "(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ↪ ")
            valueArgs = append(valueArgs, rand.Intn(1000))
            valueArgs = append(valueArgs, rand.Intn(1000))
            valueArgs = append(valueArgs, rand.Intn(1000))
            valueArgs = append(valueArgs, rand.Intn(1000))
            valueArgs = append(valueArgs, sql.NullFloat64{Float64: 1.0, Valid: true})
            valueArgs = append(valueArgs, sql.NullFloat64{Float64: 1.0, Valid: true})
            valueArgs = append(valueArgs, sql.NullFloat64{Float64: 1.0, Valid: true})
            valueArgs = append(valueArgs, sql.NullFloat64{Float64: 1.0, Valid: true})
            valueArgs = append(valueArgs, "N")
            valueArgs = append(valueArgs, "O")
            valueArgs = append(valueArgs, time.Now())
            valueArgs = append(valueArgs, time.Now())
            valueArgs = append(valueArgs, time.Now())
            valueArgs = append(valueArgs, "DELIVER IN PERSON")
            valueArgs = append(valueArgs, "SHIP")
            valueArgs = append(valueArgs, "N/A")
        }
        stmt := fmt.Sprintf("INSERT INTO %s VALUES %s",
            table, strings.Join(valueStrings, ","))
        _, err := db.Exec(stmt, valueArgs...)
        if err != nil {
            fmt.Printf("Error executing batch: %s`\n`", err)
            return
        }
        atomic.AddInt64(&rowsInsertedLastSecond, int64(batchSize))
```

```
        atomic.AddInt64(&totalInsertedRows, int64(batchSize))
    }
}

func logInsertStatistics() {
    for {
        time.Sleep(1 * time.Second)
        fmt.Printf("Total inserted rows: %d`\n`", totalInsertedRows)
        fmt.Printf("Rows inserted in the last second: %d`\n`", rowsInsertedLastSecond)
        rowsInsertedLastSecond = 0
    }
}
```

2.9.9.2.3 INSERT INTO VALUES

- 异步模式

```
### 配置 session 变量开启 group commit (默认为 off_mode),开启异步模式
mysql> set group_commit = async_mode;

### 这里返回的 label 是 group_commit 开头的，可以区分出是否使用了 group commit
mysql> insert into dt values(1, 'Bob', 90), (2, 'Alice', 99);
Query OK, 2 rows affected (0.05 sec)
{'label':'group_commit_a145ce07f1c972fc-bd2c54597052a9ad', 'status':'PREPARE', 'txnId':'181508'}

### 可以看出这个 label, txn_id 和上一个相同，说明是攒到了同一个导入任务中
mysql> insert into dt(id, name) values(3, 'John');
Query OK, 1 row affected (0.01 sec)
{'label':'group_commit_a145ce07f1c972fc-bd2c54597052a9ad', 'status':'PREPARE', 'txnId':'181508'}

### 不能立刻查询到
mysql> select * from dt;
Empty set (0.01 sec)

### 10 秒后可以查询到，可以通过表属性 group_commit_interval 控制数据可见延迟。
mysql> select * from dt;
+------+-------+-------+
| id   | name  | score |
+------+-------+-------+
|    1 | Bob   |    90 |
|    2 | Alice |    99 |
|    3 | John  |  NULL |
+------+-------+-------+
3 rows in set (0.02 sec)
```

- 同步模式

```
### 配置 session 变量开启 group commit (默认为 off_mode),开启同步模式
mysql> set group_commit = sync_mode;

### 这里返回的 label 是 group_commit 开头的，可以区分出是否使用了 group commit，
    ↪ 导入耗时至少是表属性 group_commit_interval。
mysql> insert into dt values(4, 'Bob', 90), (5, 'Alice', 99);
Query OK, 2 rows affected (10.06 sec)
{'label':'group_commit_d84ab96c09b60587_ec455a33cb0e9e87', 'status':'PREPARE', 'txnId':'3007', '
    ↪ query_id':'fc6b94085d704a94-a69bfc9a202e66e2'}

### 数据可以立刻读出
mysql> select * from dt;
+------+-------+-------+
| id   | name  | score |
+------+-------+-------+
|    1 | Bob   |    90 |
|    2 | Alice |    99 |
|    3 | John  |  NULL |
|    4 | Bob   |    90 |
|    5 | Alice |    99 |
+------+-------+-------+
5 rows in set (0.03 sec)
```

- 关闭模式

```
mysql> set group_commit = off_mode;
```

2.9.9.2.4 Stream Load

假如data.csv的内容为：

```
6,Amy,60
7,Ross,98
```

- 异步模式

```
### 导入时在 header 中增加"group_commit:async_mode"配置

curl --location-trusted -u {user}:{passwd} -T data.csv -H "group_commit:async_mode"  -H "column_
    ↪ separator:,"  http://{fe_host}:{http_port}/api/db/dt/_stream_load
{
    "TxnId": 7009,
```

```
    "Label": "group_commit_c84d2099208436ab_96e33fda01eddba8",
    "Comment": "",
    "GroupCommit": true,
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 2,
    "NumberLoadedRows": 2,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 19,
    "LoadTimeMs": 35,
    "StreamLoadPutTimeMs": 5,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 26
}

### 返回的 GroupCommit 为 true，说明进入了 group commit 的流程
### 返回的 Label 是 group_commit 开头的，是真正消费数据的导入关联的 label
```

- 同步模式

```
### 导入时在 header 中增加"group_commit:sync_mode"配置

curl --location-trusted -u {user}:{passwd} -T data.csv -H "group_commit:sync_mode"  -H "column_
    ↪ separator:,"  http://{fe_host}:{http_port}/api/db/dt/_stream_load
{
    "TxnId": 3009,
    "Label": "group_commit_d941bf17f6efcc80_ccf4afdde9881293",
    "Comment": "",
    "GroupCommit": true,
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 2,
    "NumberLoadedRows": 2,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 19,
    "LoadTimeMs": 10044,
    "StreamLoadPutTimeMs": 4,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 10038
}

### 返回的 GroupCommit 为 true，说明进入了 group commit 的流程
### 返回的 Label 是 group_commit 开头的，是真正消费数据的导入关联的 label
```

关于 Stream Load 使用的更多详细语法及最佳实践，请参阅 Stream Load。

2.9.9.3 自动提交条件

当满足时间间隔 (默认为 10 秒) 或数据量 (默认为 64 MB) 其中一个条件时，会自动提交数据。这两个参数需要配合使用，建议根据实际场景进行调优。

2.9.9.3.1 修改提交间隔

默认提交间隔为 10 秒，用户可以通过修改表的配置调整：

```
### 修改提交间隔为 2 秒
ALTER TABLE dt SET ("group_commit_interval_ms" = "2000");
```

参数调整建议: - 较短的间隔 (如 2 秒): - 优点：数据可见性延迟更低，适合对实时性要求较高的场景 - 缺点：提交次数增多，版本数增长更快，后台 compaction 压力更大

- 较长的间隔 (如 30 秒):
- 优点：提交批次更大，版本数增长更慢，系统开销更小
- 缺点：数据可见性延迟更高

建议根据业务对数据可见性延迟的容忍度来设置，如果系统压力大，可以适当增加间隔。

2.9.9.3.2 修改提交数据量

Group Commit 的默认提交数据量为 64 MB，用户可以通过修改表的配置调整：

```
### 修改提交数据量为 128MB
ALTER TABLE dt SET ("group_commit_data_bytes" = "134217728");
```

参数调整建议: - 较小的阈值 (如 32MB): - 优点：内存占用更少，适合资源受限的环境 - 缺点：提交批次较小，吞吐量可能受限

- 较大的阈值 (如 256MB):
- 优点：批量提交效率更高，系统吞吐量更大
- 缺点：占用更多内存

建议根据系统内存资源和数据可靠性要求来权衡。如果内存充足且追求更高吞吐，可以适当增加到 128MB 或更大。

2.9.9.4 相关系统配置

2.9.9.4.1 BE 配置

1. group_commit_wal_path

- 描述：group commit 存放 WAL 文件的目录
- 默认值：默认在用户配置的storage_root_path的各个目录下创建一个名为wal的目录。配置示例：

```
group_commit_wal_path=/data1/storage/wal;/data2/storage/wal;/data3/storage/wal
```

2.9.9.5 使用限制

- Group Commit 限制条件
- INSERT INTO VALUES 语句在以下情况下会退化为非 Group Commit 方式：
  - 使用事务写入 (Begin; INSERT INTO VALUES; COMMIT)
  - 指定 Label (INSERT INTO dt WITH LABEL {label} VALUES)
  - VALUES 中包含表达式 (INSERT INTO dt VALUES (1 + 100))
  - 列更新写入
  - 表不支持轻量级模式更改
- Stream Load 在以下情况下会退化为非 Group Commit 方式：
  - 使用两阶段提交
  - 指定 Label (-H "label:my_label")
  - 列更新写入
  - 表不支持轻量级模式更改
- Unique 模型
- Group Commit 不保证提交顺序，建议使用 Sequence 列来保证数据一致性。
- WAL 限制
- async_mode 写入会将数据写入 WAL，成功后删除，失败时通过 WAL 恢复。
- WAL 文件是单副本存储的，如果对应磁盘损坏或文件误删可能导致数据丢失。
- 下线 BE 节点时，使用 DECOMMISSION 命令以防数据丢失。
- async_mode 在以下情况下切换为 sync_mode：
  - 导入数据量过大（超过 WAL 单目录 80% 空间）
  - 不知道数据量的 chunked stream load
  - 磁盘可用空间不足
- 重量级 Schema Change 时，拒绝 Group Commit 写入，客户端需重试。

2.9.9.6 性能

我们分别测试了使用Stream Load和JDBC在高并发小数据量场景下group commit(使用async mode)的写入性能。

2.9.9.6.1 Stream Load 日志场景测试

机器配置

- 1 台 FE：阿里云 8 核 CPU、16GB 内存、1 块 100GB ESSD PL1 云磁盘
- 3 台 BE：阿里云 16 核 CPU、64GB 内存、1 块 1TB ESSD PL1 云磁盘
- 1 台测试客户端：阿里云 16 核 CPU、64GB 内存、1 块 100GB ESSD PL1 云磁盘
- 测试版本为 Doris-3.0.1

数据集

- httplogs 数据集，总共 31GB、2.47 亿条

测试工具

- doris-streamloader

测试方法

- 对比 非 group_commit 和 group_commit的 async_mode 模式下，设置不同的单并发数据量和并发数，导入 247249096 行数据

测试结果

导入方式	单并发数据量	并发数	耗时 (秒)	导入速率 (行/秒)	导入吞吐 (MB/秒)
group_commit	10 KB	10	2204	112,181	14.8
group_commit	10 KB	30	2176	113,625	15.0
group_commit	100 KB	10	283	873,671	115.1
group_commit	100 KB	30	244	1,013,315	133.5
group_commit	500 KB	10	125	1,977,992	260.6
group_commit	500 KB	30	122	2,026,631	267.1
group_commit	1 MB	10	119	2,077,723	273.8
group_commit	1 MB	30	119	2,077,723	273.8
group_commit	10 MB	10	118	2,095,331	276.1
非group_commit	1 MB	10	1883	131,305	17.3
非group_commit	10 MB	10	294	840,983	105.4
非group_commit	10 MB	30	118	2,095,331	276.1

在上面的group_commit测试中，BE 的 CPU 使用率在 10-40% 之间。

可以看出，group_commit 模式在小数据量并发导入的场景下，能有效的提升导入性能，同时减少版本数，降低系统合并数据的压力。

2.9.9.6.2 JDBC

机器配置

- 1 台 FE：阿里云 8 核 CPU、16GB 内存、1 块 100GB ESSD PL1 云磁盘
- 1 台 BE：阿里云 16 核 CPU、64GB 内存、1 块 500GB ESSD PL1 云磁盘
- 1 台测试客户端：阿里云 16 核 CPU、64GB 内存、1 块 100GB ESSD PL1 云磁盘
- 测试版本为 Doris-3.0.1
- 关闭打印 parpared 语句的 audit log 以提高性能

数据集

- tpch sf10 lineitem 表数据集，30 个文件，总共约 22 GB，1.8 亿行

测试工具

- DataX

测试方法

- 通过 txtfilereader 向 mysqlwriter 写入数据，配置不同并发数和单个 INSERT 的行数

测试结果

单个 insert 的行数	并发数	导入速率 (行/秒)	导入吞吐 (MB/秒)
100	10	160,758	17.21
100	20	210,476	22.19
100	30	214,323	22.92

在上面的测试中，FE 的 CPU 使用率在 60-70% 左右，BE 的 CPU 使用率在 10-20% 左右。

2.9.9.6.3 Insert into sync 模式小批量数据

机器配置

- 1 台 FE：阿里云 16 核 CPU、64GB 内存、1 块 500GB ESSD PL1 云磁盘
- 5 台 BE：阿里云 16 核 CPU、64GB 内存、1 块 1TB ESSD PL1 云磁盘。
- 1 台测试客户端：阿里云 16 核 CPU、64GB 内存、1 块 100GB ESSD PL1 云磁盘

- 测试版本为 Doris-3.0.1

数据集

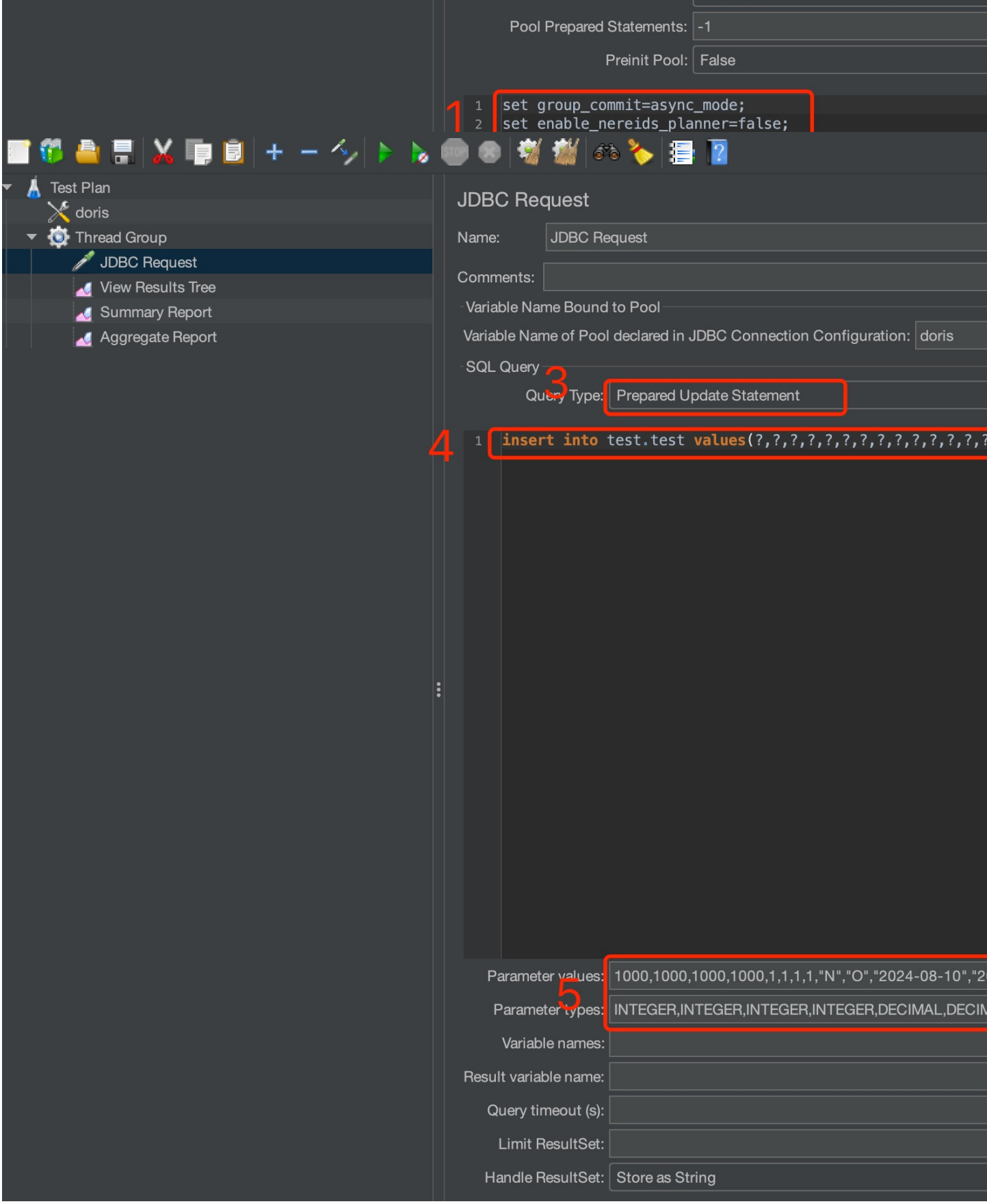
- tpch sf10 `lineitem` 表数据集。
- 建表语句为

```
CREATE TABLE IF NOT EXISTS lineitem (
  L_ORDERKEY    INTEGER NOT NULL,
  L_PARTKEY     INTEGER NOT NULL,
  L_SUPPKEY     INTEGER NOT NULL,
  L_LINENUMBER  INTEGER NOT NULL,
  L_QUANTITY    DECIMAL(15,2) NOT NULL,
  L_EXTENDEDPRICE  DECIMAL(15,2) NOT NULL,
  L_DISCOUNT    DECIMAL(15,2) NOT NULL,
  L_TAX         DECIMAL(15,2) NOT NULL,
  L_RETURNFLAG  CHAR(1) NOT NULL,
  L_LINESTATUS  CHAR(1) NOT NULL,
  L_SHIPDATE    DATE NOT NULL,
  L_COMMITDATE  DATE NOT NULL,
  L_RECEIPTDATE DATE NOT NULL,
  L_SHIPINSTRUCT CHAR(25) NOT NULL,
  L_SHIPMODE     CHAR(10) NOT NULL,
  L_COMMENT      VARCHAR(44) NOT NULL
)
DUPLICATE KEY(L_ORDERKEY, L_PARTKEY, L_SUPPKEY, L_LINENUMBER)
DISTRIBUTED BY HASH(L_ORDERKEY) BUCKETS 32
PROPERTIES (
  "replication_num" = "3"
);
```

测试工具

- Jmeter

需要设置的 jmeter 参数如下图所示



1. 设置测试前的 init 语句，`set group_commit=async_mode`以及`set enable_nereids_planner=false`。
2. 开启 jdbc 的 prepared statement，完整的 url 为:

```
jdbc:mysql://127.0.0.1:9030?useServerPrepStmts=true&useLocalSessionState=true&
    ↪ rewriteBatchedStatements=true&cachePrepStmts=true&prepStmtCacheSqlLimit=99999&
    ↪ prepStmtCacheSize=50&sessionVariables=group_commit=async_mode,enable_nereids_planner=
    ↪ false`。
```

3. 设置导入类型为 prepared update statement。
4. 设置导入语句。
5. 设置每次需要导入的值，注意，导入的值与导入值的类型要一一匹配。

测试方法

- 通过 Jmeter 向Doris写数据。 每个并发每次通过 insert into 写入 1 行数据。

测试结果

- 数据单位为行每秒。
- 以下测试分为 30，100，500 并发。

30 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	891.8	701.1	400.0	237.5
enable_nereids_planner=false	885.8	688.1	398.7	232.9

100 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	2427.8	2068.9	1259.4	764.9
enable_nereids_planner=false	2320.4	1899.3	1206.2	749.7

500 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	5567.5	5713.2	4681.0	3131.2
enable_nereids_planner=false	4471.6	5042.5	4932.2	3641.1

2.9.9.6.4 Insert into sync 模式大批量数据

机器配置

- 1 台 FE：阿里云 16 核 CPU、64GB 内存、1 块 500GB ESSD PL1 云磁盘
- 5 台 BE：阿里云 16 核 CPU、64GB 内存、1 块 1TB ESSD PL1 云磁盘。注：测试中分别用了 1 台，3 台，5 台 BE 进行测试。
- 1 台测试客户端：阿里云 16 核 CPU、64GB 内存、1 块 100GB ESSD PL1 云磁盘
- 测试版本为 Doris-3.0.1

数据集

- tpch sf10 `lineitem` 表数据集。
- 建表语句为

```
CREATE TABLE IF NOT EXISTS lineitem (
  L_ORDERKEY    INTEGER NOT NULL,
  L_PARTKEY     INTEGER NOT NULL,
  L_SUPPKEY     INTEGER NOT NULL,
  L_LINENUMBER  INTEGER NOT NULL,
  L_QUANTITY    DECIMAL(15,2) NOT NULL,
  L_EXTENDEDPRICE  DECIMAL(15,2) NOT NULL,
  L_DISCOUNT    DECIMAL(15,2) NOT NULL,
  L_TAX         DECIMAL(15,2) NOT NULL,
  L_RETURNFLAG  CHAR(1) NOT NULL,
  L_LINESTATUS  CHAR(1) NOT NULL,
  L_SHIPDATE    DATE NOT NULL,
  L_COMMITDATE  DATE NOT NULL,
  L_RECEIPTDATE DATE NOT NULL,
  L_SHIPINSTRUCT CHAR(25) NOT NULL,
  L_SHIPMODE     CHAR(10) NOT NULL,
  L_COMMENT      VARCHAR(44) NOT NULL
)
DUPLICATE KEY(L_ORDERKEY, L_PARTKEY, L_SUPPKEY, L_LINENUMBER)
DISTRIBUTED BY HASH(L_ORDERKEY) BUCKETS 32
PROPERTIES (
  "replication_num" = "3"
);
```

测试工具

- Jmeter

测试方法

- 通过 `Jmeter` 向`Doris`写数据。每个并发每次通过 insert into 写入 1000 行数据。

测试结果

- 数据单位为行每秒。
- 以下测试分为 30，100，500 并发。

30 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	9.1K	11.1K	11.4K	11.1K
enable_nereids_planner=false	157.8K	159.9K	154.1K	120.4K

100 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	10.0K	9.2K	8.9K	8.9K
enable_nereids_planner=false	130.4k	131.0K	130.4K	124.1K

500 并发 sync 模式 5 个 BE3 副本性能测试

Group commit interval	10ms	20ms	50ms	100ms
enable_nereids_planner=true	2.5K	2.5K	2.3K	2.1K
enable_nereids_planner=false	94.2K	95.1K	94.4K	94.8K

2.9.10 导入最佳实践

2.9.10.1 表模型选择

建议优先考虑使用明细模型, 明细模型在数据导入和查询性能方面相比其他模型都具有优势。如需了解更多信息，请参考：数据模型

2.9.10.2 分区分桶配置

建议一个 tablet 的大小在 1-10G 范围内。过小的 tablet 可能导致聚合效果不佳，增加元数据管理压力；过大的 tablet 不利于副本迁移、补齐。详细请参考：数据分布。

2.9.10.3 Random 分桶

在使用 Random 分桶时，可以通过设置 load_to_single_tablet 为 true 来启用单分片导入模式。这种模式在大规模数据导入过程中，能够提升数据导入的并发度和吞吐量，减少写放大问题。详细参考：Random 分桶

2.9.10.4 攒批导入

客户端攒批：建议将数据在客户端进行攒批（数 MB 到数 GB 大小）后再进行导入，高频小导入会频繁做 compaction，导致严重的写放大问题。服务端攒批：对于高并发小数据量导入，建议打开 Group Commit，在服务端实现攒批导入。

2.9.10.5 分区导入

每次导入建议只导入少量分区的数据。过多的分区同时导入会增加内存占用，并可能导致性能问题。Doris 每个 tablet 在内存中有一个活跃的 Memtable，每个 Memtable 达到一定大小时才会下刷到磁盘。为了避免进程 OOM，当活跃的 Memtable 占用内存过高时，会提前触发 Memtable 下刷，导致产生大量小文件，同时会影响导入的性能。

2.9.10.6 大规模数据分批导入

需要导入的文件数较多、数据量很大时，建议分批进行导入，避免导入出错后重试代价太大，同时减少对系统资源的冲击。对 Broker Load 每批次导入的数据量建议不超过 100G。对于本地的大数据量文件，可以使用 Doris 提供的 streamloader 工具进行导入，该工具会自动进行分批导入。

2.9.10.7 Broker Load 导入并发数

压缩文件/Parquet/ORC 文件：建议将文件分割成多个小文件进行导入，以实现多并发导入。

非压缩的 CSV 和 JSON 文件：Doris 内部会自动切分文件并并发导入。

并发数策略请参考：Broker Load 导入配置参数

2.9.10.8 Stream load 并发导入

Stream load 单 BE 上的并发数建议不超过 128（由 BE 的 webserver_num_workers 参数控制）。过高的并发数可能导致 webserver 线程数不够用，影响导入性能。特别是当单个 BE 的并发数超过 512（doris_max_remote_scanner_thread_pool_thread_num 参数）时，可能会导致 BE 进程卡住。

2.10 数据更新与删除

2.10.1 数据更新

2.10.1.1 数据更新概述

在数据驱动决策的今天，数据的“新鲜度”已成为企业在激烈市场竞争中脱颖而出的核心竞争力。传统的 T+1 数据处理模式，由于其固有的延迟，已无法满足现代商业对实时性的苛刻要求。无论是为了实现毫秒级的业务库与数据仓库同步、动态调整运营策略，还是为了在秒级内修正错误数据以保障决策的准确性，强大的实时数据更新能力都显得至关重要。

Apache Doris 作为一个现代化的实时分析型数据库，其设计的核心目标之一便是提供极致的数据新鲜度。它通过强大的数据模型和灵活的更新机制，将数据分析的延迟从天级、小时级成功压缩至秒级，为用户构建实时、敏捷的商业决策闭环提供了坚实的基础。

本文档将作为一份官方指南，系统性地阐述 Apache Doris 的数据更新能力，内容涵盖其核心原理、多样的更新与删除方式、典型的应用场景，以及在不同部署模式下的性能最佳实践，旨在帮助您全面掌握并高效利用 Doris 的数据更新功能。

2.10.1.1.1 1. 核心概念：表模型与更新机制

在 Doris 中，数据表的表模型（Data Model）决定了其数据组织方式和更新行为。为了支持不同的业务场景，Doris 提供了三种表模型：主键模型（Unique Key）、聚合模型（Aggregate Key）和明细模型（Duplicate Key）。其中，主键模型是实现复杂、高频数据更新的核心。

1.1. 表模型概览

表模型	主要特点	更新能力	适用场景
主键模型 (Unique Key)	为实时更新而生。每个数据行由唯一的键（Primary Key）标识，支持行级别的 UPSERT（Update/Insert）和部分列更新。	最强，支持所有更新和删除方式。	订单状态更新、用户标签实时计算、CDC 数据同步等需要频繁、实时变更的场景。
聚合模型 (Aggregate Key)	根据指定的 Key 列对数据进行预聚合。对于 Key 相同的行，其 Value 列会按照定义的聚合函数（如 SUM, MAX, MIN, REPLACE）进行合并。	有限，支持基于 Key 列的 REPLACE 式更新和删除。	需要实时汇总统计的场景，如实时报表、广告点击量统计等。
明细模型 (Duplicate Key)	数据仅支持追加写入（Append-only），不进行任何去重或聚合操作，即使数据行完全相同也会被保留。	有限，仅支持通过 DELETE 语句进行条件删除。	日志采集、用户行为埋点等只需追加、无需更新的场景。

1.2. 数据更新方式

Doris 提供了两大类数据更新方法：通过数据导入进行更新和通过 DML 语句进行更新。

1.2.1. 通过导入进行更新 (UPSERT)

这是 Doris 推荐的高性能、高并发的更新方式，主要针对主键模型。所有的导入方式（Stream Load, Broker Load, Routine Load, INSERT INTO）都天然支持 UPSERT 语义。当新数据导入时，如果其主键已存在，Doris 会用新行数据覆盖旧行数据；如果主键不存在，则插入新行。

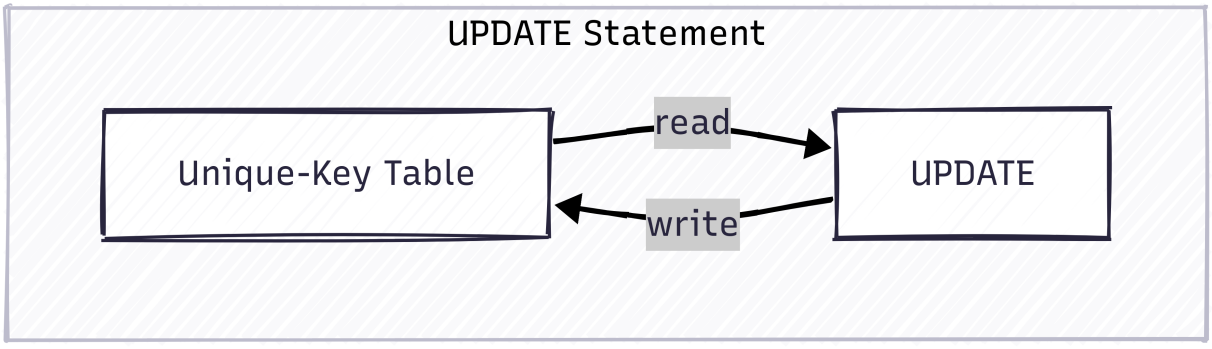


图 42: img

1.2.2. 通过 UPDATE DML 语句更新

Doris 支持标准的 SQL UPDATE 语句，允许用户根据 WHERE 子句指定的条件对数据进行更新。这种方式非常灵活，支持复杂的更新逻辑，例如跨表关联更新。

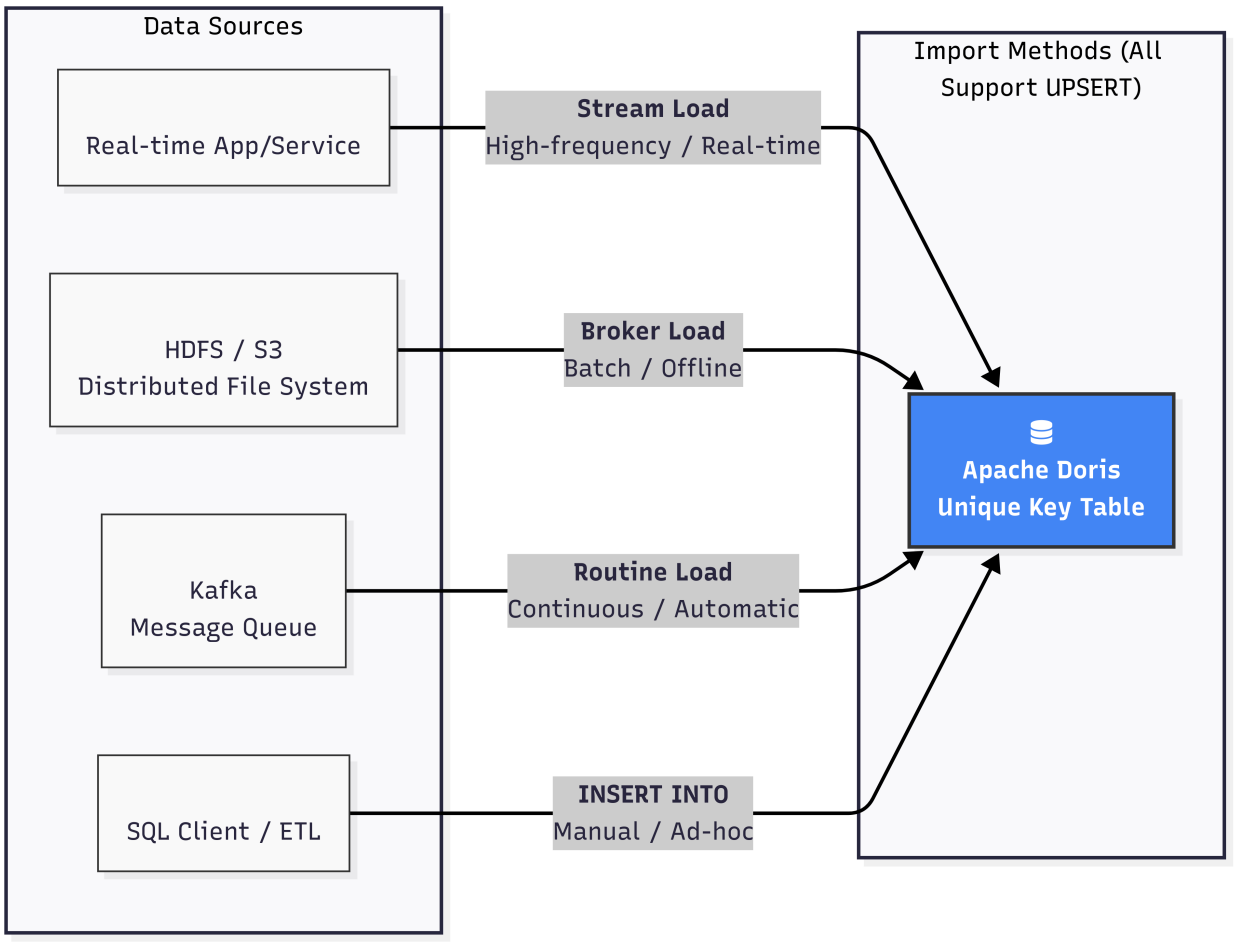


图 43: img

```
-- 简单更新
UPDATE user_profiles SET age = age + 1 WHERE user_id = 1;

-- 跨表关联更新
UPDATE sales_records t1
SET t1.user_name = t2.name
FROM user_profiles t2
WHERE t1.user_id = t2.user_id;
```

注意：UPDATE 语句的执行过程是先扫描满足条件的数据，然后将更新后的数据重新写回表中。它适合低频、批量的更新任务。不建议对 UPDATE 语句进行高并发操作，因为并发的 UPDATE 在涉及相同主键时，无法保证数据的隔离性。

1.2.2. 通过 INSERT INTO SELECT DML 语句更新

由于 Doris 默认提供了 UPSERT 的语义，因此使用INSERT INTO SELECT也可以实现类似于UPDATE的更新效果。

1.3. 数据删除方式

与更新类似，Doris 也支持通过导入和 DML 语句两种方式删除数据。

1.3.1. 通过导入进行标记删除

这是一种高效的批量删除方法，主要用于主键模型。用户可以在导入数据时，增加一个特殊的隐藏列 DORIS_DELETE_SIGN。当某行的该列值为 1 或 true 时，Doris 会将该主键对应的数据行标记为删除（关于 delete sign 的原理，后文会有详细的介绍）。

```
// Stream Load 导入数据，删除 user_id 为 2 的行
// curl --location-trusted -u user:passwd -H "columns:user_id, __DORIS_DELETE_SIGN__" -T delete.
    ↪ json http://fe_host:8030/api/db_name/table_name/_stream_load

// delete.json 内容
[
    {"user_id": 2, "__DORIS_DELETE_SIGN__": "1"}
]
```

1.3.2. 通过 DELETE DML 语句删除

Doris 支持标准的 SQL DELETE 语句，可以根据 WHERE 条件删除数据。

- 主键模型：DELETE 语句会将满足条件的行的主键重新写入，并附带删除标记。因此，其性能与需要删除的数据量成正比。主键模型上的DELETE语句执行原理与UPDATE语句非常相似，先通过查询把要删除的数据读取出来，然后再附加删除标记进行一次写入。相比UPDATE语句，DELETE 语句只需要写入 Key 列和删除标记列，相对轻量一些。
- 明细/聚合模型：DELETE 语句的实现方式是记录一个删除谓词（Delete Predicate）。在查询时，这个谓词会作为一个运行时过滤器（Runtime Filter）来过滤掉被删除的数据。因此，DELETE 操作本身非常快，几乎与删除的数据量无关。但需要注意，在明细/聚合模型上进行高频的 DELETE 操作会累积大量的运行时过滤器，严重影响后续的查询性能。

```
DELETE FROM user_profiles WHERE last_login < '2022-01-01';
```

下表是对使用 DML 语句进行删除的一个简要总结

	主键模型	聚合模型	明细模型
实现方式	Delete Sign	Delete Predicate	Delete Predicate
限制	无	删除条件只能用于 Key 列	无
删除性能	一般	快	快

2.10.1.1.2 2. 深入主键模型：原理与实现

主键模型是 Doris 实现高性能实时更新的基石。理解其内部工作原理，对于充分发挥其性能至关重要。

2.1. Merge-on-Write (MoW) vs. Merge-on-Read (MoR)

主键模型有两种数据合并策略：写时合并（MoW）和读时合并（MoR）。自 Doris 2.1 版本起，MoW 已成为默认且推荐的实现方式。

特性	Merge-on-Write (MoW)	Merge-on-Read (MoR) - (旧
核心思想	在数据写入时即完成数据去重和合并，保证存储中的每个主键只有一条最新记录。	数据写入时保留多个版
查询性能	极高。查询时无需额外合并操作，性能接近无更新的明细表。	较差。查询时需要进行
写入性能	写入时有合并开销，相比 MoR 有一定的性能损失（小批量约 10-20%，大批量约 30-50%）。	写入速度快，接近明细
资源消耗	写入和后台 Compaction 消耗更多 CPU 和内存。	查询时消耗更多 CPU 和
适用场景	绝大多数实时更新场景。尤其适合读多写少的业务，能提供极致的查询分析性能。	适用于写多读少的场景

MoW 机制通过在写入阶段付出少量代价，换取了查询性能的巨大提升，完美契合了 OLAP 系统“重读轻写”的特点。

2.2. 条件更新 (Sequence Column)

在分布式系统中，数据乱序到达是一个常见问题。例如，一个订单状态先后变更为“已支付”和“已发货”，但由于网络延迟，代表“已发货”的数据可能先于“已支付”的数据到达 Doris。

为了解决这个问题，Doris 引入了 Sequence 列机制。用户可以在建表时指定一个列（通常是时间戳或版本号）作为 Sequence 列。当处理具有相同主键的数据时，Doris 会比较它们的 Sequence 列的值，并始终保留 Sequence 值最大的那一行数据，从而保证了数据的最终一致性，即使数据乱序到达。

```
CREATE TABLE order_status (
    order_id BIGINT,
    status_name STRING,
    update_time DATETIME
)
UNIQUE KEY(order_id)
DISTRIBUTED BY HASH(order_id)
PROPERTIES (
    "function_column.sequence_col" = "update_time" -- 指定 update_time 为 Sequence 列
);

-- 1. 写入 "已发货" 记录 (update_time 较大)
-- {"order_id": 1001, "status_name": "Shipped", "update_time": "2023-10-26 12:00:00"}

-- 2. 写入 "已支付" 记录 (update_time 较小，后到达)
-- {"order_id": 1001, "status_name": "Paid", "update_time": "2023-10-26 11:00:00"}

-- 最终查询结果，保留了 update_time 最大的记录
-- order_id: 1001, status_name: "Shipped", update_time: "2023-10-26 12:00:00"
```

2.3. 删除机制 (DORIS_DELETE_SIGN) 工作流程

DORIS_DELETE_SIGN 的工作原理可以概括为“逻辑标记，后台清理”。

1. 执行删除：当用户通过导入或DELETE语句删除数据时，Doris 不会立即从物理文件中移除数据。相反，它会为要删除的主键写入一条新记录，该记录的 DORIS_DELETE_SIGN 列被标记为 1。
2. 查询过滤：当用户查询数据时，Doris 会在查询计划中自动添加一个过滤条件 WHERE DORIS_DELETE_SIGN ↪ = 0，从而在查询结果中隐藏所有被标记为删除的数据。
3. 后台 Compaction：Doris 的后台 Compaction 进程会定期扫描数据。当它发现一个主键同时存在正常记录和删除标记记录时，它会在合并过程中将这两条记录都物理地移除，最终释放存储空间。

这种机制确保了删除操作的快速响应，同时通过后台任务异步完成物理清理，避免了对在线业务的性能冲击。

下图展示了DORIS_DELETE_SIGN 的工作原理

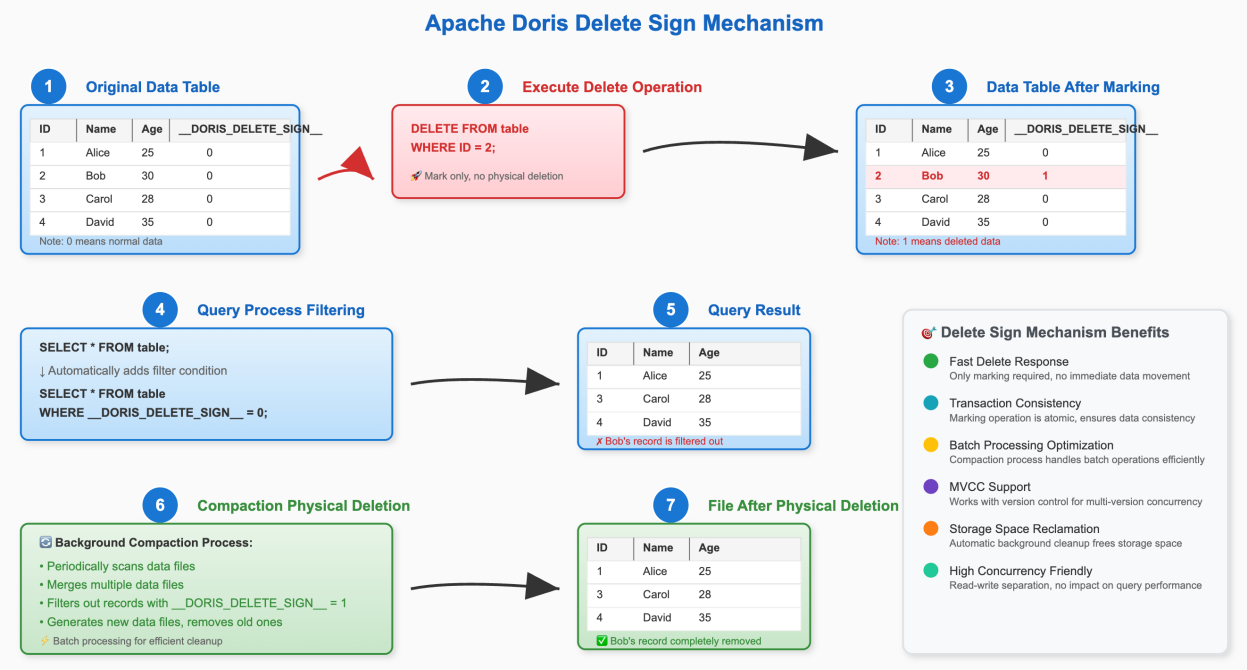


图 44: img

2.4 部分列更新 (Partial Column Update)

从 2.0 版本开始，Doris 在主键模型（MoW）上支持了强大的部分列更新能力。用户在导入数据时，只需提供主键和待更新的列，未提供的列将保持其原值不变。这极大地简化了宽表拼接、实时标签更新等场景的 ETL 流程。

要启用此功能，需在创建主键模型表时，开启 Merge-on-Write (MoW) 模式，并设置 enable_unique_key_partial ↪ _update 属性为 true。或者在数据导入时配置"partial_columns"参数

```
CREATE TABLE user_profiles (
    user_id BIGINT,
    name STRING,
    age INT,
    last_login DATETIME
)
```

```
UNIQUE KEY(user_id)
DISTRIBUTED BY HASH(user_id)
PROPERTIES (
    "enable_unique_key_partial_update" = "true"
);

-- 初始数据
-- user_id: 1, name: 'Alice', age: 30, last_login: '2023-10-01 10:00:00'

-- 通过 Stream Load 导入部分更新数据，只更新 age 和 last_login
-- {"user_id": 1, "age": 31, "last_login": "2023-10-26 18:00:00"}

-- 更新后数据
-- user_id: 1, name: 'Alice', age: 31, last_login: '2023-10-26 18:00:00'
```

部分列更新原理概要

不同于传统的 OLTP 数据库，Doris 的部分列更新并非是原地的数据更新，为了让 Doris 有更好的写入吞吐以及查询性能，主键模型的部分列更新采取了 “导入时将缺失字段补齐后再整行写入” 的实现方案。

因此使用 Doris 的部分列更新存在 “读放大” 和 “写放大” 的影响。例如给一个 100 列的宽表更新 10 个字段，Doris 在写入过程中需要补齐缺失的 90 个字段，假设每个字段的大小接近，则 1MB 的 10 字段更新，会在 Doris 系统中产生大约 9MB 的数据读取（补齐缺失的字段），以及 10MB 的数据写入（补齐整行后写入到新的文件），也就是有大约 9 倍的读放大和 10 倍的写放大。

部分列更新性能建议

由于部分列更新存在读放大和写放大，同时 Doris 还是列存系统，在数据读取的过程中可能会产生大量 d 随机 IO，因此对硬盘的随机读 IOPS 有较高的要求。由于传统的机械磁盘在随机 IO 上存在显著瓶颈，因此如果要使用部分列更新功能进行高频的写入，建议使用 SSD 硬盘，最好是 nvme 接口，能够提供最好的随机 IO 支撑

同时，如果表很宽，也建议开启行存来减少随机 IO。开启行存后，Doris 会在列存之外额外的存储一份行存数据，由于行存数据每一行都是连续存储的，因此可以一次 IO 就读取到整行数据（列存则需要 N 次 IO 才能读取到所有缺失的字段，例如前面的 100 列宽表更新 10 列的例子，每一行需要 90 次 IO 才能读取到所有的字段）

2.10.1.1.3 3. 典型应用场景

Doris 强大的数据更新能力使其能够胜任多种要求严苛的实时分析场景。

3.1. CDC 数据实时同步

通过 Flink CDC 等工具捕获上游业务数据库（如 MySQL, PostgreSQL, Oracle）的变更数据（Binlog），并实时写入 Doris 的主键模型表，是构建实时数仓最经典的场景。

- 整库同步：Flink Doris Connector 内部集成了 Flink CDC，可以实现从上游数据库到 Doris 的自动化、端到端的整库同步，无需手动建表和配置字段映射。
- 保证一致性：利用主键模型的 UPSERT 能力处理上游的 INSERT 和 UPDATE 操作，利用 DORIS_DELETE_SIGN 处理 DELETE 操作，并结合 Sequence 列（如 Binlog 中的时间戳）处理乱序数据，完美复刻上游数据库的状态，实现毫秒级延迟的数据同步。

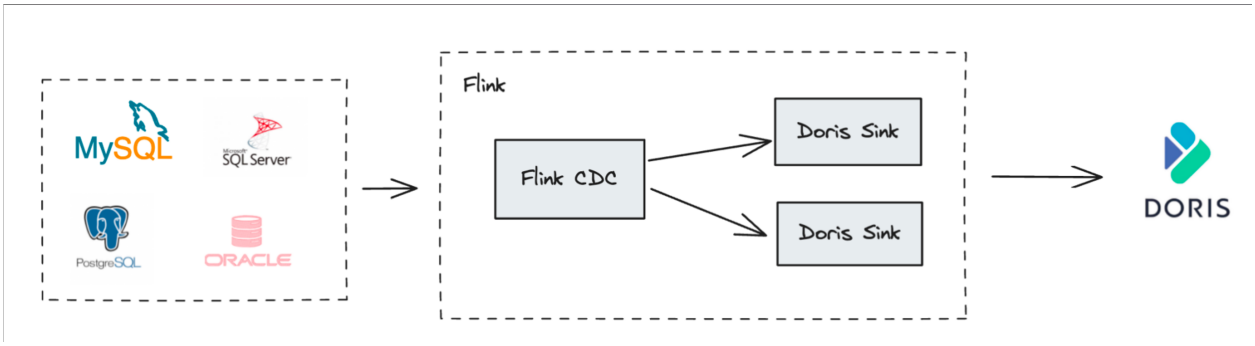


图 45: img

3.2. 实时宽表拼接

在很多分析场景中，需要将来自不同业务系统的数据拼接成一张用户宽表或商品宽表。传统的方式是使用离线的 ETL 任务（如 Spark 或 Hive）定期（T+1）进行拼接，实时性差，且维护成本高。或者使用 Flink 进行实时的宽表 join 计算，将拼接后的数据写入数据库，这通常需要消耗大量的计算资源。

利用 Doris 的部分列更新能力，可以极大地简化这一流程：

1. 在 Doris 中创建一张主键模型的宽表。
2. 将来自不同数据源（如用户基础信息、用户行为数据、交易数据等）的数据流通过 Stream Load 或 Routine Load 实时写入这张宽表。
3. 每个数据流只负责更新自己相关的字段。例如，用户行为数据流只更新 `page_view_count`, `last_login_time` 等字段；交易数据流只更新 `total_orders`, `total_amount` 等字段。

这种方式不仅将宽表的构建从离线 ETL 转变为实时流式处理，大大提升了数据新鲜度，还因为只写入变化的列而减少了 I/O 开销，提升了写入性能。

2.10.1.1.4 4. 最佳实践

遵循以下最佳实践，可以帮助您更稳定、更高效地使用 Doris 的数据更新功能。

4.1. 通用性能实践

1. 优先使用导入更新：对于高频、大量的更新操作，应优先选择 Stream Load, Routine Load 等导入方式，而非 UPDATE DML 语句。
2. 攒批写入：避免使用 `INSERT INTO` 语句进行逐条的高频写入（如 > 100 TPS），因为每条 `INSERT` 都会产生一次事务开销。如果必须使用，应考虑开启 Group Commit 功能，将多个小批量提交合并成一个大事务。
3. 谨慎使用高频 `DELETE`：在明细模型和聚合模型上，避免高频的 `DELETE` 操作，以防查询性能下降。
4. 删除分区数据时使用`TRUNCATE PARTITION`：如果需要删除整个分区的数据，应使用 `TRUNCATE PARTITION`，其效率远高于 `DELETE`。
5. 串行执行 `UPDATE`：避免并发执行可能作用于相同数据行的 `UPDATE` 任务。

4.2. 存算分离架构下的主键模型实践

Doris 3.0 引入了先进的存算分离架构，带来了极致的弹性和更低的成本。在该架构下，由于 BE 无状态，因此在 Merge-on-Write 过程中，需要通过 MetaService 来维护一个全局状态以解决导入/compaction/schema change 之间的写写冲突。主键模型的 MoW 实现依赖于一个基于 Meta Service 的分布式表锁来保证写操作的一致性，如下图所示：

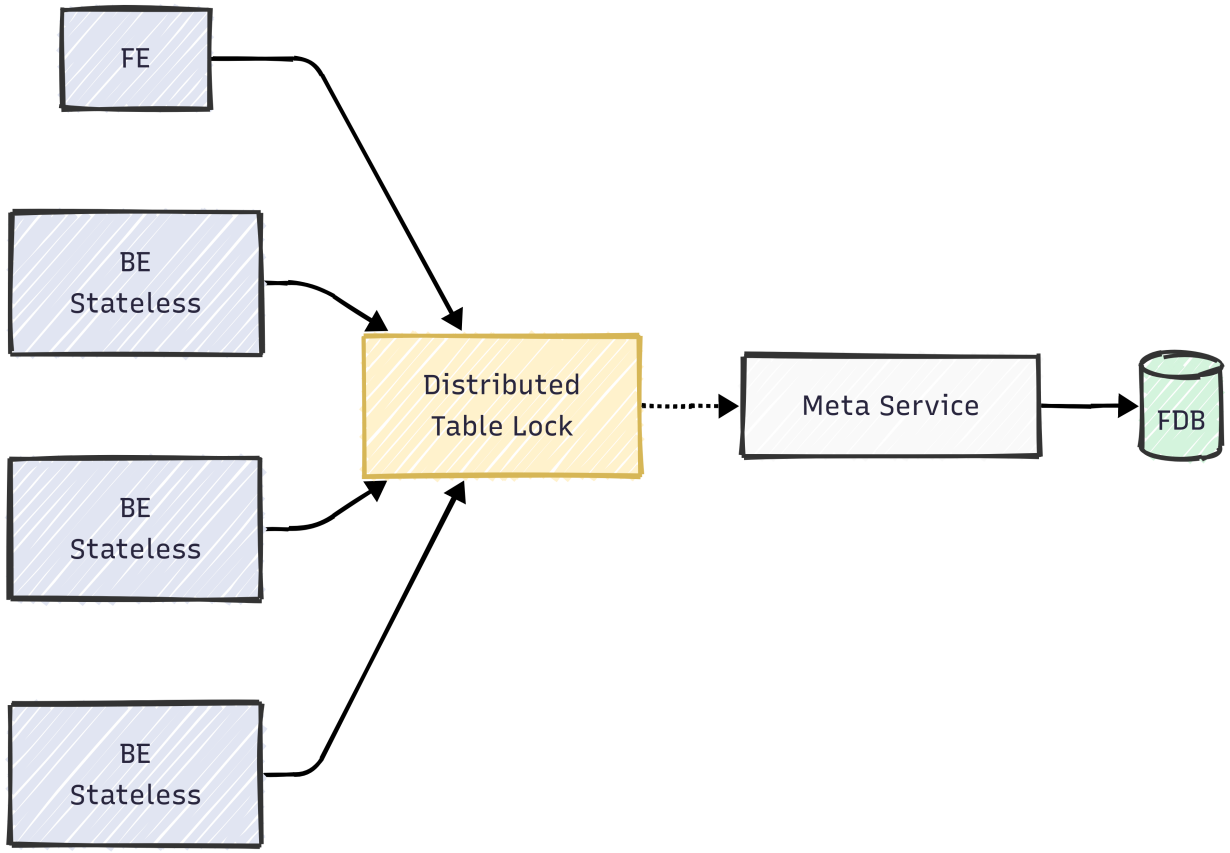


图 46: img

高频的导入和 Compaction 会导致对表锁的频繁竞争，因此需要特别注意以下几点：

1. 控制单表导入频率：建议将单张主键表的导入频率控制在 60 次/秒以内。可以通过攒批、调整导入并发等方式来降低频率。
2. 合理设计分区分桶：
3. 分区：利用时间分区（如按天或按小时）可以确保单次导入只更新少量分区，减少锁竞争的范围。
4. 分桶：分桶数（Tablet 数量）应根据数据量合理设置，通常在 8-64 之间。过多的 Tablet 会加剧锁竞争。
5. 调整 Compaction 策略：在写入压力非常大的场景下，可以适当调整 Compaction 策略，降低 Compaction 的频率，从而减少其与导入任务之间的锁冲突。
6. 升级到最新版本：Doris 社区正在持续优化存算分离架构下的主键模型性能。例如，即将发布的 3.1 版本对分布式表锁的实现进行了大幅优化。始终建议使用最新的稳定版本以获得最佳性能。

2.10.1.1.5 结论

Apache Doris 凭借其以主键模型为核心的强大、灵活且高效的数据更新能力，真正打破了传统 OLAP 系统在数据新鲜度上的瓶颈。无论是通过高性能的导入实现 UPSERT 和部分列更新，还是利用 Sequence 列保证乱序数据的一致性，Doris 都为构建端到端的实时分析应用提供了完整的解决方案。

通过深入理解其核心原理，掌握不同更新方式的适用场景，并遵循本文档提供的最佳实践，您将能够充分释放 Doris 的潜力，让实时数据真正成为驱动业务增长的强大引擎

2.10.1.2 主键模型的 Update 更新

主要介绍如何使用 Update 命令来更新 Doris 中的数据。Update 命令仅适用于 Unique 数据模型的表。

2.10.1.2.1 适用场景

- 小范围数据更新：适用于更新少量数据的场景，例如修复某些记录中的错误字段，或更新某些字段的状态（如订单状态更新等）。
- ETL 批量加工部分字段：适用于大批量更新某个字段，常见于 ETL 加工场景。注意：大范围数据更新仅适合低频调用。

2.10.1.2.2 基本原理

利用查询引擎自身的 where 过滤逻辑，从待更新表中筛选出需要被更新的行。再利用 Unique 模型自带的 Value 列新数据替换旧数据的逻辑，将待更新的行变更后，再重新插入到表中，从而实现行级别更新。

同步

Update 语法在 Doris 中是一个同步语法，即 Update 语句执行成功，更新操作也就完成了，数据是可见的。

性能

Update 语句的性能和待更新的行数以及查询条件的检索效率密切相关。

- 待更新的行数：待更新的行数越多，Update 语句的速度就会越慢。对于小范围更新，Doris 支持的频率与`INSERT INTO`语句类似，对于大范围更新，由于单个 update 执行的时间较长，仅适用于低频调用。
- 查询条件的检索效率：Update 实现原理是先将满足查询条件的行做读取处理，所以如果查询条件的检索效率高，则 Update 的速度也会快。条件列最好能命中索引或者分区分桶裁剪，这样 Doris 就不需要全表扫描，可以快速定位到需要更新的行，从而提升更新效率。强烈不推荐条件列中包含 value 列。

2.10.1.2.3 使用示例

假设在金融风控场景中，存在如下结构的交易明细表：

```
CREATE TABLE transaction_details (
    transaction_id BIGINT NOT NULL,        -- 唯一交易编号
    user_id BIGINT NOT NULL,               -- 用户编号
    transaction_date DATE NOT NULL,        -- 交易日期
    transaction_time DATETIME NOT NULL,    -- 交易时间
    transaction_amount DECIMAL(18, 2),     -- 交易金额
    transaction_device STRING,             -- 交易设备
```

```
    transaction_region STRING,             -- 交易地区
    average_daily_amount DECIMAL(18, 2),   -- 最近 3 个月日均交易金额
    recent_transaction_count INT,          -- 最近 7 天交易次数
    has_dispute_history BOOLEAN,           -- 是否有拒付记录
    risk_level STRING                      -- 风险等级
)
UNIQUE KEY(transaction_id)
DISTRIBUTED BY HASH(transaction_id) BUCKETS 16
PROPERTIES (
    "replication_num" = "3",               -- 副本数量，默认 3
    "enable_unique_key_merge_on_write" = "true"  -- 启用 MOW 模式，支持合并更新
);
```

存在如下交易数据：

```
+--
    ↪ --------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
| transaction_id | user_id | transaction_date | transaction_time    | transaction_amount |
    ↪ transaction_device | transaction_region | average_daily_amount | recent_transaction_count
    ↪  | has_dispute_history | risk_level |
+--
    ↪ --------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
|           1001 |    5001 | 2024-11-24       | 2024-11-24 14:30:00 |             100.00 | iPhone
    ↪  12          | New York           |               100.00 |                       10 |
    ↪                    0 | NULL       |
|           1002 |    5002 | 2024-11-24       | 2024-11-24 03:30:00 |             120.00 | iPhone
    ↪  12          | New York           |               100.00 |                       15 |
    ↪                    0 | NULL       |
|           1003 |    5003 | 2024-11-24       | 2024-11-24 10:00:00 |             150.00 |
    ↪ Samsung S21        | Los Angeles        |               100.00 |                       30
    ↪  |                  0 | NULL       |
|           1004 |    5004 | 2024-11-24       | 2024-11-24 16:00:00 |             300.00 |
    ↪ MacBook Pro        | high_risk_region1  |               200.00 |                        5
    ↪  |                  0 | NULL       |
|           1005 |    5005 | 2024-11-24       | 2024-11-24 11:00:00 |            1100.00 | iPad
    ↪ Pro           | Chicago            |               200.00 |                       10 |
    ↪                    0 | NULL       |
+--
    ↪ --------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
```

按照如下风控规则来更新每日所有交易记录的风险等级：1. 有拒付记录，风险为 high。2. 在高风险地区，风险为 high。3. 交易金额异常（超过日均 5 倍），风险为 high。4. 最近 7 天交易频繁：a. 交易次数 > 50，风险为 high。b. 交易次数在 20-50 之间，风险为 medium。5. 非工作时间交易（凌晨 2 点到 4 点），风险为 medium。6. 默

认风险为 low。

```
UPDATE transaction_details
SET risk_level = CASE
    -- 有拒付记录或在高风险地区的交易
    WHEN has_dispute_history = TRUE THEN 'high'
    WHEN transaction_region IN ('high_risk_region1', 'high_risk_region2') THEN 'high'

    -- 突然异常交易金额
    WHEN transaction_amount > 5 * average_daily_amount THEN 'high'

    -- 最近 7 天交易频率很高
    WHEN recent_transaction_count > 50 THEN 'high'
    WHEN recent_transaction_count BETWEEN 20 AND 50 THEN 'medium'

    -- 非工作时间交易
    WHEN HOUR(transaction_time) BETWEEN 2 AND 4 THEN 'medium'

    -- 默认风险
    ELSE 'low'
END
WHERE transaction_date = '2024-11-24';
```

更新之后的数据为

```
+--
    ↪ --------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
| transaction_id | user_id | transaction_date | transaction_time    | transaction_amount |
    ↪ transaction_device | transaction_region | average_daily_amount | recent_transaction_count
    ↪  | has_dispute_history | risk_level |
+--
    ↪ --------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
|           1001 |    5001 | 2024-11-24       | 2024-11-24 14:30:00 |             100.00 | iPhone
    ↪  12          | New York           |               100.00 |                       10 |
    ↪                    0 | low        |
|           1002 |    5002 | 2024-11-24       | 2024-11-24 03:30:00 |             120.00 | iPhone
    ↪  12          | New York           |               100.00 |                       15 |
    ↪                    0 | medium     |
|           1003 |    5003 | 2024-11-24       | 2024-11-24 10:00:00 |             150.00 |
    ↪ Samsung S21        | Los Angeles        |               100.00 |                       30
    ↪  |                   0 | medium     |
|           1004 |    5004 | 2024-11-24       | 2024-11-24 16:00:00 |             300.00 |
    ↪ MacBook Pro        | high_risk_region1  |               200.00 |                        5
    ↪  |                   0 | high       |
```

```
|          1005 |    5005 | 2024-11-24       | 2024-11-24 11:00:00 |            1100.00 | iPad
    ↪ Pro           | Chicago            |               200.00 |                      10 |
    ↪                   0 | high       |
+--
    ↪ -------------+---------+------------------+---------------------+--------------------+-----------------
    ↪
```

2.10.1.2.4 更多帮助

关于数据更新使用的更多详细语法，请参阅UPDATE 命令手册，也可以在 MySQL 客户端命令行下输入 HELP ↪ UPDATE 获取更多帮助信息。

2.10.1.3 主键模型的导入更新

这篇文档主要介绍 Doris 主键模型基于导入的更新。

2.10.1.3.1 整行更新

使用 Doris 支持的 Stream Load、 Broker Load、 Routine Load、 Insert Into 等导入方式，向主键模型（ Unique 模型）导入数据时，如果没有相应主键的数据行，则插入新数据；如果有相应主键的数据行，则进行更新。也就是说，Doris 主键模型的导入是一种 “upsert” 模式。基于导入，对已有记录的更新，默认和导入一个新记录是完全一样的，因此可以参考数据导入的文档部分。

2.10.1.3.2 部分列更新

部分列更新是指直接更新表中某些字段值，而不是全部字段值。可以使用 Update 语句进行更新，这种 Update 语句通常先读取整行数据，然后更新部分字段值，再写回。这种读写事务非常耗时，不适合大批量数据写入。Doris 在主键模型的导入更新中，提供了直接插入或更新部分列数据的功能，不需要先读取整行数据，从而大幅提升更新效率。

> 注意
>
> 1. 2.0 版本仅在 Unique Key 的 Merge-on-Write 实现中支持部分列更新能力。
> 2. 从 2.0.2 版本开始，支持使用 INSERT INTO 进行部分列更新。
> 3. 不支持在有同步物化视图的表上进行部分列更新。
> 4. 不支持在进行 Schema Change 的表上进行部分列更新。

适用场景

- 实时动态列更新，需要在表中实时高频更新某些字段值。例如用户标签表中有一些关于用户最新行为信息的字段需要实时更新，以便广告/推荐系统能够据此进行实时分析和决策。
- 将多张源表拼接成一张大宽表。
- 数据修正。

使用示例

假设 Doris 中存在一张订单表 order_tbl，其中订单 id 是 Key 列，订单状态和订单金额是 Value 列。数据状态如下：

订单 id	订单金额	订单状态
1	100	待付款

```
+----------+--------------+--------------+
| order_id | order_amount | order_status |
+----------+--------------+--------------+
| 1        |          100 | 待付款       |
+----------+--------------+--------------+
1 row in set (0.01 sec)
```

此时，用户点击付款后，Doris 系统需要将订单 id 为 ‘1’ 的订单状态变更为 ‘待发货’。

可以使用以下导入方式进行部分列更新

StreamLoad/BrokerLoad/RoutineLoad

准备如下 csv 文件：

```
1,待发货
```

在导入时添加如下 header：

```
partial_columns:true
```

同时在 columns 中指定要导入的列（必须包含所有 key 列，否则无法更新）。下面是一个 Stream Load 的例子：

```
curl --location-trusted -u root: -H "partial_columns:true" -H "column_separator:," -H "columns:
    ↪ order_id,order_status" -T /tmp/update.csv http://127.0.0.1:8030/api/db1/order_tbl/_stream
    ↪ _load
```

INSERT INTO

在所有数据模型中，INSERT INTO 给定部分列时默认行为是整行写入。为了防止误用，在 Merge-on-Write 实现中，INSERT INTO 默认仍然保持整行 UPSERT 的语义。如果需要开启部分列更新的语义，需要设置如下 session variable：

```
SET enable_unique_key_partial_update=true;
INSERT INTO order_tbl (order_id, order_status) VALUES (1, '待发货');
```

需要注意的是，控制 insert 语句是否开启严格模式的会话变量 enable_insert_strict 的默认值为 true，即 insert 语句默认开启严格模式。在严格模式下进行部分列更新不允许更新不存在的 key。所以，在使用 insert 语句进行部分列更新时，如果希望能插入不存在的 key，需要在 enable_unique_key_partial_update 设置为 true 的基础上，同时将 enable_insert_strict 设置为 false。

Flink Connector

如果使用 Flink Connector，需要添加如下配置：

```
'sink.properties.partial_columns' = 'true',
```

同时在 sink.properties.column 中指定要导入的列（必须包含所有 key 列，否则无法更新）。

更新结果

更新后结果如下：

```
+----------+--------------+--------------+
| order_id | order_amount | order_status |
+----------+--------------+--------------+
| 1        |          100 | 待发货       |
+----------+--------------+--------------+
1 row in set (0.01 sec)
```

使用注意

由于 Merge-on-Write 实现需要在数据写入时进行整行数据的补齐，以保证最优的查询性能，因此使用 Merge-on-Write 实现进行部分列更新会导致部分导入性能下降。

写入性能优化建议：

- 使用配备 NVMe 的 SSD，或者极速 SSD 云盘。因为补齐数据时会大量读取历史数据，产生较高的读 IOPS 以及读吞吐。
- 开启行存能够大大减少补齐数据时产生的 IOPS，导入性能提升明显。用户可以在建表时通过如下 property 来开启行存：

```
"store_row_column" = "true"
```

目前，同一批次数据写入任务（无论是导入任务还是 INSERT INTO）的所有行只能更新相同的列。如果需要更新不同列的数据，则需要分不同批次进行写入。

在未来版本中，将支持灵活的列更新，用户可以在同一批导入中，每一行更新不同的列。

2.10.1.3.3 灵活部分列更新

> 该功能自 3.1.0 版本支持

此前，doris 支持的部分列更新功能限制了一次导入中每一行必须更新相同的列。现在，doris 支持一种更加灵活的更新方式，它使得一次导入中的每一行可以更新不同的列。

> 注意：

1. 目前只有 stream load 这一种导入方式以及使用 stream load 作为其导入方式的工具 (如 doris-flink-connector) 支持灵活列更新功能
2. 在使用灵活列更新时导入文件必须为 json 格式的数据

适用场景

在使用 CDC 的方式将某个数据系统的数据实时同步到 Doris 中时，源端系统输出的记录可能并不是完整的行数据，而是只有主键和被更新的列的数据。在这种情况下，某个时间窗口内的一批数据中每一行更新的列可能都是不同的。此时，可以使用灵活列更新的方式来将数据导入到 Doris 中。

使用方式

存量表开启灵活列更新功能

对于在旧版本 Doris 中已经建好的存量 Merge-On-Write 表，在升级 Doris 之后如果想要使用灵活列更新的功能，可以使用 `ALTER TALBE db1.tbl1 ENABLE FEATURE "UPDATE_FLEXIBLE_COLUMNS";` 来开启这一功能。执行完上述语句后使用 `show create table db1.tbl1` 的结果中如果包含 `"enable_unique_key_skip_bitmap_column" = "true"` 则表示功能开启成功。注意，使用这一方式之前需要确保目标表已经开启了 light-schema-change 的功能。

新建表使用灵活列更新功能

对于新建的表，如果需要使用灵活列更新功能，建表时需要指定如下表属性，以开启 Merge-on-Write 实现，同时使得表具有灵活列更新所需要的 `bitmap` 隐藏列。

```
"enable_unique_key_merge_on_write" = "true"
"enable_unique_key_skip_bitmap_column" = "true"
```

StreamLoad

在使用 Stream Load 导入时添加如下 header

```
unique_key_update_mode:UPDATE_FLEXIBLE_COLUMNS
```

Flink Doris Connector

如果使用 Flink Doris Connector, 需要添加如下配置：

```
'sink.properties.unique_key_update_mode' = 'UPDATE_FLEXIBLE_COLUMNS'
```

示例

假设有如下表

```
CREATE TABLE t1 (
  `k` int(11) NULL,
  `v1` BIGINT NULL,
  `v2` BIGINT NULL DEFAULT "9876",
  `v3` BIGINT NOT NULL,
  `v4` BIGINT NOT NULL DEFAULT "1234",
```

```
  `v5` BIGINT NULL
) UNIQUE KEY(`k`) DISTRIBUTED BY HASH(`k`) BUCKETS 1
PROPERTIES(
"replication_num" = "3",
"enable_unique_key_merge_on_write" = "true",
"enable_unique_key_skip_bitmap_column" = "true");
```

表中有如下原始数据

```
MySQL root@127.1:d1> select * from t1;
+---+----+----+----+----+----+
| k | v1 | v2 | v3 | v4 | v5 |
+---+----+----+----+----+----+
| 0 | 0  | 0  | 0  | 0  | 0  |
| 1 | 1  | 1  | 1  | 1  | 1  |
| 2 | 2  | 2  | 2  | 2  | 2  |
| 3 | 3  | 3  | 3  | 3  | 3  |
| 4 | 4  | 4  | 4  | 4  | 4  |
| 5 | 5  | 5  | 5  | 5  | 5  |
+---+----+----+----+----+----+
```

现在通过灵活列更新导入来更新其中的一些行的字段

```
$ cat test1.json
```

```
{"k": 0, "__DORIS_DELETE_SIGN__": 1}
{"k": 1, "v1": 10}
{"k": 2, "v2": 20, "v5": 25}
{"k": 3, "v3": 30}
{"k": 4, "v4": 20, "v1": 43, "v3": 99}
{"k": 5, "v5": null}
{"k": 6, "v1": 999, "v3": 777}
{"k": 2, "v4": 222}
{"k": 1, "v2": 111, "v3": 111}
```

```
curl --location-trusted -u root: \
-H "strict_mode:false" \
-H "format:json" \
-H "read_json_by_line:true" \
-H "unique_key_update_mode:UPDATE_FLEXIBLE_COLUMNS" \
-T test1.json \
-XPUT http://<host>:<http_port>/api/d1/t1/_stream_load
```

更新后表中的数据如下：

```
MySQL root@127.1:d1> select * from t1;
```

```
+---+-----+------+-----+------+--------+
| k | v1  | v2   | v3  | v4   | v5     |
+---+-----+------+-----+------+--------+
| 1 | 10  | 111  | 111 | 1    | 1      |
| 2 | 2   | 20   | 2   | 222  | 25     |
| 3 | 3   | 3    | 30  | 3    | 3      |
| 4 | 43  | 4    | 99  | 20   | 4      |
| 5 | 5   | 5    | 5   | 5    | <null> |
| 6 | 999 | 9876 | 777 | 1234 | <null> |
+---+-----+------+-----+------+--------+
```

限制与注意事项

1. 和之前的部分列更新相同，灵活列更新要求导入的每一行数据需要包括所有的 Key 列，不满足这一要求的行数据将被过滤掉，同时计入 `filter rows` 的计数中，如果 `filtered rows` 的数量超过了本次导入 `max_filter_ratio` 所能容忍的上限，则整个导入将会失败。同时，被过滤的数据会在 error log 留下一条日志。
2. 灵活列更新导入中每一个 json 对象中的键值对只有当它的 Key 和目标表中某一列的列名一致时才是有效的，不满足这一要求的键值对将被忽略。同时，Key 为 `__DORIS_VERSION_COL__/__DORIS_ROW_STORE_COL__/__DORIS_SKIP_BITMAP_COL__` 的键值对也将被忽略。
3. 当目标表的表属性中设置了 `function_column.sequence_type` 这一属性时，灵活列更新的导入可以通过在 json 对象中包括 Key 为 `__DORIS_SEQUENCE_COL__` 的键值对来指定目标表中 `__DORIS_SEQUENCE_COL__` 列的值。对于不指定 `__DORIS_SEQUENCE_COL__` 列的值的行，如果这一行的 Key 在原表中存在，则这一行 `__DORIS_SEQUENCE_COL__` 列的值将被填充为旧行中对应的值，否则该列的值将被填充为 `null` 值

例如，对于下表：

```
CREATE TABLE t2 (
  `k` int(11) NULL,
  `v1` BIGINT NULL,
  `v2` BIGINT NULL DEFAULT "9876",
  `v3` BIGINT NOT NULL,
  `v4` BIGINT NOT NULL DEFAULT "1234",
  `v5` BIGINT NULL
) UNIQUE KEY(`k`) DISTRIBUTED BY HASH(`k`) BUCKETS 1
PROPERTIES(
"replication_num" = "3",
"enable_unique_key_merge_on_write" = "true",
"enable_unique_key_skip_bitmap_column" = "true",
"function_column.sequence_type" = "int");
```

表中有如下原始数据：

```
+---+----+----+----+----+----+----------------------+
| k | v1 | v2 | v3 | v4 | v5 |__DORIS_SEQUENCE_COL__|
```

```
+---+----+----+----+----+----+----------------------+
| 0 | 0  | 0  | 0  | 0  | 0  | 0                    |
| 1 | 1  | 1  | 1  | 1  | 10 | 10                   |
| 2 | 2  | 2  | 2  | 2  | 20 | 20                   |
| 3 | 3  | 3  | 3  | 3  | 30 | 30                   |
| 4 | 4  | 4  | 4  | 4  | 40 | 40                   |
| 5 | 5  | 5  | 5  | 5  | 50 | 50                   |
+---+----+----+----+----+----+----------------------+
```

通过灵活列更新导入如下数据：

```
{"k": 1, "v1": 111, "v5": 9, "__DORIS_SEQUENCE_COL__": 9}
{"k": 2, "v2": 222, "v5": 25, "__DORIS_SEQUENCE_COL__": 25}
{"k": 3, "v3": 333}
{"k": 4, "v4": 444, "v5": 50, "v1": 411, "v3": 433, "v2": null, "__DORIS_SEQUENCE_COL__": 50}
{"k": 5, "v5": null, "__DORIS_SEQUENCE_COL__": null}
{"k": 6, "v1": 611, "v3": 633}
{"k": 7, "v3": 733, "v5": 300, "__DORIS_SEQUENCE_COL__": 300}
```

最终表中的数据如下：

```
+---+--------+--------+-----+------+--------+
| k | v1     | v2     | v3  | v4   | v5     |
+---+--------+--------+-----+------+--------+
| 0 | 0      | 0      | 0   | 0    | 0      |
| 1 | 1      | 1      | 1   | 1    | 1      |
| 5 | 5      | 5      | 5   | 5    | 5      |
| 2 | 2      | 222    | 2   | 2    | 25     |
| 3 | 3      | 3      | 333 | 3    | 3      |
| 4 | 411    | <null> | 433 | 444  | 50     |
| 6 | 611    | 9876   | 633 | 1234 | <null> |
| 7 | <null> | 9876   | 733 | 1234 | 300    |
+---+--------+--------+-----+------+--------+
```

4. 当目标表的表属性中设置了 function_column.sequence_col 这一属性时，灵活列更新导入数据的 json 对象中 Key 为 __DORIS_SEQUENCE_COL__ 的键值对将被忽略，导入中某一行 __DORIS_SEQUENCE_COL__ 列的值将与这一行中表属性 function_column.sequence_col 所指定的列最终的值完全一致。

例如，对于下表：

```
CREATE TABLE t3 (
  `k` int(11) NULL,
  `v1` BIGINT NULL,
  `v2` BIGINT NULL DEFAULT "9876",
  `v3` BIGINT NOT NULL,
  `v4` BIGINT NOT NULL DEFAULT "1234",
```

```
  `v5` BIGINT NULL DEFAULT "31"
) UNIQUE KEY(`k`) DISTRIBUTED BY HASH(`k`) BUCKETS 1
PROPERTIES(
"replication_num" = "3",
"enable_unique_key_merge_on_write" = "true",
"enable_unique_key_skip_bitmap_column" = "true",
"function_column.sequence_col" = "v5");
```

表中有如下原始数据：

```
+---+----+----+----+----+----+----------------------+
| k | v1 | v2 | v3 | v4 | v5 |__DORIS_SEQUENCE_COL__|
+---+----+----+----+----+----+----------------------+
| 0 | 0  | 0  | 0  | 0  | 0  | 0                    |
| 1 | 1  | 1  | 1  | 1  | 10 | 10                   |
| 2 | 2  | 2  | 2  | 2  | 20 | 20                   |
| 3 | 3  | 3  | 3  | 3  | 30 | 30                   |
| 4 | 4  | 4  | 4  | 4  | 40 | 40                   |
| 5 | 5  | 5  | 5  | 5  | 50 | 50                   |
+---+----+----+----+----+----+----------------------+
```

通过灵活列更新导入如下数据：

```
{"k": 1, "v1": 111, "v5": 9}
{"k": 2, "v2": 222, "v5": 25}
{"k": 3, "v3": 333}
{"k": 4, "v4": 444, "v5": 50, "v1": 411, "v3": 433, "v2": null}
{"k": 5, "v5": null}
{"k": 6, "v1": 611, "v3": 633}
{"k": 7, "v3": 733, "v5": 300}
```

最终表中的数据如下：

```
+---+--------+--------+-----+------+-----+
| k | v1     | v2     | v3  | v4   | v5  |
+---+--------+--------+-----+------+-----+
| 0 | 0      | 0      | 0   | 0    | 0   |
| 1 | 1      | 1      | 1   | 1    | 10  |
| 5 | 5      | 5      | 5   | 5    | 50  |
| 2 | 2      | 222    | 2   | 2    | 25  |
| 3 | 3      | 3      | 333 | 3    | 30  |
| 4 | 411    | <null> | 433 | 444  | 50  |
| 6 | 611    | 9876   | 633 | 1234 | 31  |
| 7 | <null> | 9876   | 733 | 1234 | 300 |
+---+--------+--------+-----+------+-----+
```

5. 使用灵活列更新时不能指定或开启如下一些导入属参数：

    - 不能指定 merge_type 参数
    - 不能指定 delete 参数
    - 不能开启 fuzzy_parse 参数
    - 不能指定 columns 参数
    - 不能指定 jsonpaths 参数
    - 不能指定 hidden_columns 参数
    - 不能指定 function_column.sequence_col 参数
    - 不能指定 sql 参数
    - 不能开启 memtable_on_sink_node 前移
    - 不能指定 group_commit 参数
    - 不能指定 where 参数

6. 不支持在有 Variant 列的表上进行灵活列更新。

7. 不支持在有同步物化视图的表上进行灵活列更新。

2.10.1.4 聚合模型的导入更新

这篇文档主要介绍 Doris 聚合模型上基于导入的更新。

2.10.1.4.1 整行更新

使用 Doris 支持的 Stream Load，Broker Load，Routine Load，Insert Into 等导入方式，往聚合模型（Agg 模型）中进行数据导入时，都会将新的值与旧的聚合值，根据列的聚合函数产出新的聚合值，这个值可能是插入时产出，也可能是异步 Compaction 时产出，但是用户查询时，都会得到一样的返回值。

2.10.1.4.2 聚合模型的部分列更新

Aggregate 表主要在预聚合场景使用而非数据更新的场景使用，但也可以通过将聚合函数设置为 REPLACE_IF_NOT_NULL 来实现部分列更新效果。

建表

将需要进行列更新的字段对应的聚合函数设置为REPLACE_IF_NOT_NULL

```
CREATE TABLE order_tbl (
  order_id int(11) NULL,
  order_amount int(11) REPLACE_IF_NOT_NULL NULL,
  order_status varchar(100) REPLACE_IF_NOT_NULL NULL
) ENGINE=OLAP
AGGREGATE KEY(order_id)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(order_id) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

数据写入

无论是 Stream Load、 Broker Load、 Routine Load 还是INSERT INTO, 直接写入要更新的字段的数据即可

示例

与前面例子相同，对应的 Stream Load 命令为（不需要额外的 header ）:

```
$ cat update.csv

1,To be shipped

curl  --location-trusted -u root: -H "column_separator:," -H "columns:order_id,order_status" -T /
    ↪ tmp/update.csv http://127.0.0.1:8030/api/db1/order_tbl/_stream_load
```

对应的INSERT INTO语句为（不需要额外设置 session variable ）:

```
INSERT INTO order_tbl (order_id, order_status) values (1,'待发货');
```

2.10.1.4.3 部分列更新使用注意

Aggregate Key 模型在写入过程中不做任何额外处理，所以写入性能不受影响，与普通的数据导入相同。但是在查询时进行聚合的代价较大，典型的聚合查询性能相比 Unique Key 模型的 Merge-on-Write 实现会有 5-10 倍的下降。

由于 REPLACE_IF_NOT_NULL 聚合函数仅在非 NULL 值时才会生效，因此用户无法将某个字段值修改为 NULL 值。

2.10.1.5 主键模型的更新并发控制

2.10.1.5.1 概览

Doris 采用多版本并发控制机制（MVCC - Multi-Version Concurrency Control）来管理并发更新。每次数据写入操作均会分配一个写入事务，该事务确保数据写入的原子性（即写入操作要么完全成功，要么完全失败）。在写入事务提交时，系统会为其分配一个版本号。当用户使用 Unique Key 模型并多次导入数据时，如果存在重复主键，Doris 会根据版本号确定覆盖顺序：版本号较高的数据会覆盖版本号较低的数据。

在某些场景中，用户可能需要通过在建表语句中指定 sequence 列来灵活调整数据的生效顺序。例如，当通过多线程并发同步数据到 Doris 时，不同线程的数据可能会乱序到达。这种情况下，可能出现旧数据因较晚到达而错误覆盖新数据的情况。为解决这一问题，用户可以为旧数据指定较低的 sequence 值，为新数据指定较高的 sequence 值，从而让 Doris 根据用户提供的 sequence 值来正确确定数据的更新顺序。

此外，UPDATE 语句与通过导入实现更新在底层机制上存在较大差异。UPDATE 操作涉及两个步骤：从数据库中读取待更新的数据，以及写入更新后的数据。默认情况下，UPDATE 语句通过表级锁提供了 Serializable 隔离级别的事务能力，即多个 UPDATE 操作只能串行执行。用户也可以通过调整配置绕过这一限制，具体方法请参阅以下章节的详细说明。

2.10.1.5.2 UPDATE 并发控制

默认情况下，并不允许同一时间对同一张表并发进行多个 UPDATE 操作。

主要原因是，Doris 目前支持的是行更新，这意味着，即使用户声明的是 `SET v2 = 1`，实际上，其他所有的 Value 列也会被覆盖一遍（尽管值没有变化）。

这就会存在一个问题，如果同时有两个 UPDATE 操作对同一行进行更新，那么其行为可能是不确定的，也就是可能存在脏数据。

但在实际应用中，如果用户自己可以保证即使并发更新，也不会同时对同一行进行操作的话，就可以手动打开并发限制。通过修改 FE 配置 `enable_concurrent_update`，当该配置值设置为 `true` 时，更新命令将不再提供事务保证。

2.10.1.5.3 Sequence 列

Unique 模型主要针对需要唯一主键的场景，可以保证主键唯一性约束，在同一批次中导入或者不同批次中导入的数据，替换顺序不做保证。替换顺序无法保证则无法确定最终导入到表中的具体数据，存在了不确定性。

为了解决这个问题，Doris 支持了 sequence 列，通过用户在导入时指定 sequence 列，相同 key 列下，按照 sequence 列的值进行替换，较大值可以替换较小值，反之则无法替换。该方法将顺序的确定交给了用户，由用户控制替换顺序。

在实现层面，Doris 增加了一个隐藏列 DORIS_SEQUENCE_COL，该列的类型由用户在建表时指定，在导入时确定该列具体值，并依据该值决定相同 Key 列下，哪一行生效。

> 注意 sequence 列目前只支持 Unique 模型。

启用 sequence column 支持

在新建表时如果设置了 `function_column.sequence_col` 或者 `function_column.sequence_type`，则新建表将支持 sequence column。

对于一个不支持 sequence column 的表，如果想要使用该功能，可以使用如下语句：`ALTER TABLE example_db .my_table ENABLE FEATURE "SEQUENCE_LOAD" WITH PROPERTIES ("function_column.sequence_type" = " Date")` 来启用。

如果不确定一个表是否支持 sequence column，可以通过设置一个 session variable 来显示隐藏列 `SET show_hidden _columns=true`，之后使用 `desc tablename`，如果输出中有 `__DORIS_SEQUENCE_COL__` 列则支持，如果没有则不支持。

使用示例

下面以 Stream Load 为例展示使用方式：

1. 创建支持 sequence col 的表

创建 unique 模型的 test_table 数据表，并指定 sequence 列映射到表中的 modify_date 列。

```
CREATE TABLE test.test_table
(
```

```
    user_id bigint,
    date date,
    group_id bigint,
    modify_date date,
    keyword VARCHAR(128)
)
UNIQUE KEY(user_id, date, group_id)
DISTRIBUTED BY HASH (user_id) BUCKETS 32
PROPERTIES(
    "function_column.sequence_col" = 'modify_date',
    "replication_num" = "1",
    "in_memory" = "false"
);
```

sequence_col 用来指定 sequence 列到表中某一列的映射，该列可以为整型和时间类型（DATE、DATETIME），创建后不能更改该列的类型。

创建好的表结构如下：

```
MySQL>  desc test_table;
+-------------+--------------+------+-------+---------+---------+
| Field       | Type         | Null | Key   | Default | Extra   |
+-------------+--------------+------+-------+---------+---------+
| user_id     | BIGINT       | No   | true  | NULL    |         |
| date        | DATE         | No   | true  | NULL    |         |
| group_id    | BIGINT       | No   | true  | NULL    |         |
| modify_date | DATE         | No   | false | NULL    | REPLACE |
| keyword     | VARCHAR(128) | No   | false | NULL    | REPLACE |
+-------------+--------------+------+-------+---------+---------+
```

除了上述按照列映射的方式来指定 sequence 之外，Doris 还支持根据指定类型创建 sequence 列的语法，这种方式不要求建表时 schema 中必须有一列来做映射，下面是对应的语法：

```
PROPERTIES (
    "function_column.sequence_type" = 'Date',
);
```

sequence_type 用来指定 sequence 列的类型，可以为整型和时间类型（DATE、DATETIME）。

2. 导入数据：

使用列映射的方式 (function_column.sequence_col) 来指定 sequence 列，不需要修改任何参数。下面我们用 Stream Load 导入如下数据：

```
1   2020-02-22  1   2020-02-21  a
1   2020-02-22  1   2020-02-22  b
1   2020-02-22  1   2020-03-05  c
1   2020-02-22  1   2020-02-26  d
```

```
1   2020-02-22  1   2020-02-23  e
1   2020-02-22  1   2020-02-24  b
```

stream load 命令：

```
curl --location-trusted -u root: -T testData http://host:port/api/test/test_table/_stream_load
```

结果为

```
MySQL> select * from test_table;
+---------+------------+----------+-------------+---------+
| user_id | date       | group_id | modify_date | keyword |
+---------+------------+----------+-------------+---------+
|       1 | 2020-02-22 |        1 | 2020-03-05  | c       |
+---------+------------+----------+-------------+---------+
```

在这次导入中，因 sequence column 的值（也就是 modify_date 中的值）中 '2020-03-05' 为最大值，所以 keyword 列中最终保留了 c。

如果建表时使用了 function_column.sequence_col 方式来指定 sequence 列，在导入时需要指定 sequence 列到其他列的映射。

1. Stream Load

Stream Load 的写法是在 header 中的 function_column.sequence_col 字段添加隐藏列对应的 source_sequence 的映射，示例如下：

```
curl --location-trusted -u root -H "columns: k1,k2,source_sequence,v1,v2" -H "function_column.
    ↪ sequence_col: source_sequence" -T testData http://host:port/api/testDb/testTbl/_stream_
    ↪ load
```

2. Broker Load

在 ORDER BY 处设置隐藏列映射的 source_sequence 字段

```
LOAD LABEL db1.label1
(
    DATA INFILE("hdfs://host:port/user/data/*/test.txt")
    INTO TABLE `tbl1`
    COLUMNS TERMINATED BY ","
    (k1,k2,source_sequence,v1,v2)
    ORDER BY source_sequence
)
WITH BROKER 'broker'
(
    "username"="user",
    "password"="pass"
)
PROPERTIES
(
```

```
    "timeout" = "3600"
);
```

3. Routine Load

映射方式同上，示例如下

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
    [WITH MERGE|APPEND|DELETE]
    COLUMNS(k1, k2, source_sequence, v1, v2),
    WHERE k1  100 and k2 like "%doris%"
    [ORDER BY source_sequence]
    PROPERTIES
    (
        "desired_concurrent_number"="3",
        "max_batch_interval" = "20",
        "max_batch_rows" = "300000",
        "max_batch_size" = "209715200",
        "strict_mode" = "false"
    )
    FROM KAFKA
    (
        "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
        "kafka_topic" = "my_topic",
        "kafka_partitions" = "0,1,2,3",
        "kafka_offsets" = "101,0,0,200"
    );
```

3. 替换顺序的保证

上述步骤完成后，接着导入如下数据

```
1   2020-02-22  1   2020-02-22  a
1   2020-02-22  1   2020-02-23  b
```

查询数据

```
MySQL [test]> select * from test_table;
+---------+------------+----------+-------------+---------+
| user_id | date       | group_id | modify_date | keyword |
+---------+------------+----------+-------------+---------+
|       1 | 2020-02-22 |        1 | 2020-03-05  | c       |
+---------+------------+----------+-------------+---------+
```

在这次导入的数据中，会比较所有已导入数据的 sequence column (也就是 modify_date)，其中 ‘2020-03-05’ 为最大值，所以 keyword 列中最终保留了 c。

4. 再尝试导入如下数据

```
1   2020-02-22  1   2020-02-22  a
1   2020-02-22  1   2020-03-23  w
```

查询数据

```
MySQL [test]> select * from test_table;
+---------+------------+----------+-------------+---------+
| user_id | date       | group_id | modify_date | keyword |
+---------+------------+----------+-------------+---------+
|       1 | 2020-02-22 |        1 | 2020-03-23  | w       |
+---------+------------+----------+-------------+---------+
```

此时就可以替换表中原有的数据。综上，在导入过程中，会比较所有批次的 sequence 列值，选择值最大的记录导入 Doris 表中。

注意

1. 为防止误用，在 StreamLoad/BrokerLoad 等导入任务以及行更新 insert 语句中，用户必须显示指定 sequence 列 (除非 sequence 列的默认值为 CURRENT_TIMESTAMP)，不然会收到以下报错信息：

```
Table test_tbl has sequence column, need to specify the sequence column
```

2. 在使用 Insert 语句插入数据时，由于用户必须显示指定 sequence 列，否则会报如上异常。为了方便用户在一些场景下（表复制，内部数据迁移等场景）使用，Doris 可以通过 session 参数控制，来关闭 sequence 列的检查约束：

```
set require_sequence_in_insert = false;
```

3. 自版本 2.0 起，Doris 对 Unique Key 表的 Merge-on-Write 实现支持了部分列更新能力，在部分列更新导入中，用户每次可以只更新一部分列，因此并不是必须要包含 sequence 列。若用户提交的导入任务中，包含 sequence 列，则行为无影响；若用户提交的导入任务不包含 sequence 列，Doris 会使用匹配的历史数据中的 sequence 列作为更新后该行的 sequence 列的值。如果历史数据中不存在相同 key 的列，则会自动用 null 或默认值填充。
4. 当出现并发导入时，Doris 会利用 MVCC 机制来保证数据的正确性。如果两批数据导入都更新了一个相同 key 的不同列，则其中系统版本较高的导入任务会在版本较低的导入任务成功后，使用版本较低的导入任务写入的相同 key 的数据行重新进行补齐。

2.10.2 数据删除

2.10.2.1 删除操作概述

在 Apache Doris 中，删除操作（Delete）是一项关键功能，用于管理和清理数据，以满足用户在大规模数据分析场景中的灵活性需求。

Doris 提供了丰富多样的删除功能支持，包括：DELETE 语句、删除标记（delete sign）、分区删除、全表删除以及使用临时分区实现原子覆盖写等功能。下面将详细介绍每一项功能：

DELETE 语句

删除数据时最常用的是 DELETE 语句，该功能支持所有表模型，用户可以使用它删除符合条件的数据。

DELETE 语句的语法如下：

```
DELETE FROM table_name WHERE condition;
```

DELETE 语句基本能满足大部分用户在使用 Doris 过程中的删除需求，但在某些场景下它并不是最高效的。为了灵活高效地满足用户在各类场景的删除需求，Doris 还提供了如下几种删除方式。

分区删除

在 Doris 中，通过日期分区等方式来管理数据是很常见的实践。很多用户只需要保留最近一段时间的数据（例如 7 天），对于过期的数据分区，可以采用分区删除（truncate partition）功能来进行高效的删除。

相比 DELETE 语句，分区删除只需要修改一些分区元数据即可完成删除，是这种场景下最佳的删除方式。

分区删除的语法如下：

```
TRUNCATE TABLE tbl PARTITION(p1, p2);
```

整表删除

整表删除适用于快速清空表且保留表结构的场景，例如在离线分析场景中需要重做数据时。

整表删除的语法如下：

```
TRUNCATE TABLE table_name;
```

删除标记（Delete Sign）

数据删除可以视作数据更新的一种情况。因此，在具有更新能力的主键模型（Unique Key）上，用户可以通过删除标记功能，使用数据更新的方式实现删除操作。

例如在 CDC 数据同步场景中，CDC 程序可以将一条 DELETE 操作的 binlog 打上删除标记，当这条数据写入 Doris 时，就会删除掉对应的主键。

这种方式相对于 DELETE 语句来说，可以批量进行大量主键的删除操作，效率较高。

删除标记属于高级功能，使用起来相比前几种要更复杂一些，详细的用法请参考文档批量删除。

使用临时分区实现原子覆盖写

某些情况下，用户希望能够重写某一分区的数据，但如果采用先删除再导入的方式进行，在中间会有一段时间无法查看数据。这时，用户可以先创建一个对应的临时分区，将新的数据导入到临时分区后，通过替换操作，原子性地替换原有分区，以达到目的。详细用法请参考文档表原子替换。

2.10.2.1.1 注意事项

1. 删除操作会生成新的数据版本，因此频繁执行删除可能会导致版本数量增加，从而影响查询性能。
2. 删除后的数据在合并压缩完成之前仍会占用存储，因此删除操作本身不会立即降低存储使用。

2.10.2.2 Delete 操作

删除操作语句通过 MySQL 协议，按条件删除指定表或分区中的数据。支持通过简单的谓词组合条件来指定要删除的数据，也支持在主键表上使用 USING 子句关联多表进行删除。

2.10.2.2.1 通过指定过滤谓词删除

```
DELETE FROM table_name [table_alias]
  [PARTITION partition_name | PARTITIONS (partition_name [, partition_name])]
  WHERE column_name op { value | value_list } [ AND column_name op { value | value_list } ...];
```

必须参数

- table_name: 指定需要删除数据的表
- column_name: 属于 table_name 的列
- op: 逻辑比较操作符，包括：=, >, <, >=, <=, !=, in, not in
- value | value_list: 进行逻辑比较的值或值列表

可选参数

- PARTITION partition_name | PARTITIONS (partition_name [, partition_name]): 指定执行删除数据的分区名，如果表不存在此分区，则报错
- table_alias: 表的别名

使用限制

- 使用表模型 Aggregate 时，只能指定 Key 列上的条件。当选定的 Key 列不存在于某个 Rollup 中时，无法进行删除。
- 对于分区表，需要指定分区。如果不指定，Doris 会从条件中推断分区。
- 两种情况下，Doris 无法从条件中推断分区：
  1. 条件中不包含分区列
  2. 分区列的 op 为 not in
- 如果分区表不是 Unique 表，当分区表未指定分区，或无法从条件中推断分区时，需要设置会话变量 delete_without_partition 为 true，此时删除操作会应用到所有分区。

示例

1. 删除 my_table 分区 p1 中 k1 列值为 3 的数据行

```
DELETE FROM my_table PARTITION p1
  WHERE k1 = 3;
```

2. 删除 my_table 分区 p1 中 k1 列值大于等于 3 且 status 列值为 “outdated” 的数据行

```
DELETE FROM my_table PARTITION p1
WHERE k1 >= 3 AND status = "outdated";
```

3. 删除 my_table 分区 p1, p2 中 k1 列值大于等于 3 且 dt 列值位于 “2024-10-01” 和 “2024-10-31” 之间的数据行

```
DELETE FROM my_table PARTITIONS (p1, p2)
WHERE k1 >= 3 AND dt >= "2024-10-01" AND dt <= "2024-10-31";
```

2.10.2.2.2 通过使用 USING 子句删除

在某些场景下，用户需要关联多张表才能精确确定要删除的数据，这种情况下 USING 子句非常有用，语法如下：

```
DELETE FROM table_name [table_alias]
  [PARTITION partition_name | PARTITIONS (partition_name [, partition_name])]
  [USING additional_tables]
  WHERE condition
```

必须参数

- table_name: 指定需要删除数据的表
- WHERE condition: 指定用于选择删除行的条件

可选参数

- PARTITION partition_name | PARTITIONS (partition_name [, partition_name]): 指定执行删除数据的分区名，如果表不存在此分区，则报错
- table_alias: 表的别名

注意事项

- 此形式只能在 UNIQUE KEY 模型表上使用

示例

使用 t2 和 t3 表连接的结果，删除 t1 中的数据，删除的表只支持 unique 模型。

```
-- 创建 t1, t2, t3 三张表
CREATE TABLE t1
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
UNIQUE KEY (id)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1', "function_column.sequence_col" = "c4");

CREATE TABLE t2
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

CREATE TABLE t3
```

```
  (id INT)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

-- 插入数据
INSERT INTO t1 VALUES
  (1, 1, '1', 1.0, '2000-01-01'),
  (2, 2, '2', 2.0, '2000-01-02'),
  (3, 3, '3', 3.0, '2000-01-03');

INSERT INTO t2 VALUES
  (1, 10, '10', 10.0, '2000-01-10'),
  (2, 20, '20', 20.0, '2000-01-20'),
  (3, 30, '30', 30.0, '2000-01-30'),
  (4, 4, '4', 4.0, '2000-01-04'),
  (5, 5, '5', 5.0, '2000-01-05');

INSERT INTO t3 VALUES
  (1),
  (4),
  (5);

-- 删除 t1 中的数据
DELETE FROM t1
  USING t2 INNER JOIN t3 ON t2.id = t3.id
  WHERE t1.id = t2.id;
```

预期结果为，删除 t1 表中 id 为 1 的行。

```
+----+----+----+--------+------------+
| id | c1 | c2 | c3     | c4         |
+----+----+----+--------+------------+
| 2  | 2  | 2  |    2.0 | 2000-01-02 |
| 3  | 3  | 3  |    3.0 | 2000-01-03 |
+----+----+----+--------+------------+
```

2.10.2.2.3 相关配置

超时配置

- insert_timeout: 因为删除操作是一个 SQL 命令且被视为一种特殊的导入，因此删除语句会受 Session 中的 insert_timeout 值影响，可以通过 SET insert_timeout = xxx 来增加超时时间，单位为秒。

IN 谓词配置

- max_allowed_in_element_num_of_delete: 如果用户在使用 in 谓词时需要占用的元素较多，可以通过此项调整允许携带的元素上限，默认值为 1024。

2.10.2.2.4 查看历史记录

用户可以通过 SHOW DELETE 语句查看历史上已执行完成的删除记录。

语法如下：

```
SHOW DELETE [FROM db_name]
```

示例：

```
mysql> show delete from test_db;
+-----------+---------------+---------------------+-----------------+----------+
| TableName | PartitionName | CreateTime          | DeleteCondition | State    |
+-----------+---------------+---------------------+-----------------+----------+
| empty_tbl | p3            | 2020-04-15 23:09:35 | k1 EQ "1"       | FINISHED |
| test_tbl  | p4            | 2020-04-15 23:09:53 | k1 GT "80"      | FINISHED |
+-----------+---------------+---------------------+-----------------+----------+
2 rows in set (0.00 sec)
```

2.10.2.2.5 性能建议

1. 在明细表（Duplicate Key）和聚合表（Aggregate Key）上，删除操作执行速度较快，但短时间内大量删除操作会影响查询性能。
2. 在主键表（Unique Key）上，删除操作被转换成 INSERT INTO 语句，涉及大范围删除时执行速度较慢，但短时间内大量删除不会对查询性能有较大影响。

2.10.2.2.6 语法

删除语法详见DELETE 语法手册。

2.10.2.3 基于导入的批量删除

2.10.2.3.1 基于导入的批量删除

删除操作可以视为数据更新的一种特殊形式。在主键模型（Unique Key）表上，Doris 支持通过导入数据时添加删除标记来实现删除操作。

相比 DELETE 语句，使用删除标记在以下场景中具有更好的易用性和性能优势：

1. CDC 场景：在从 OLTP 数据库同步数据到 Doris 时，binlog 中的 Insert 和 Delete 操作通常交替出现。使用 DELETE 语句无法高效处理这些删除操作。通过使用删除标记，可以统一处理 Insert 和 Delete 操作，简化 CDC 写入 Doris 的代码，同时提高数据导入和查询性能。
2. 批量删除指定主键：如果需要删除大量主键，使用 DELETE 语句的效率较低。每次执行 DELETE 都会生成一个空的 rowset 来记录删除条件，并产生一个新的数据版本。频繁删除或删除条件过多时，会严重影响查询性能。

2.10.2.3.2 删除标记的工作原理

原理说明

- 表结构：删除标记在主键表上存储为一个隐藏列 __DORIS_DELETE_SIGN__，该列值为 1 时表示删除标记生效。
- 数据导入：用户在导入任务中可以指定删除标记列的映射条件，不同导入任务的用法不同，详见下文语法说明。
- 查询：在查询时，Doris FE 会在查询规划中自动添加 __DORIS_DELETE_SIGN__ != true 的过滤条件，将删除标记为 1 的数据过滤掉。
- 数据合并（compaction）：Doris 的后台数据合并会定期清理删除标记为 1 的数据。

数据示例

表结构

创建一个示例表：

```
CREATE TABLE example_table (
    id BIGINT NOT NULL,
    value STRING
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 10
PROPERTIES (
    "replication_num" = "3"
);
```

使用 session 变量 show_hidden_columns 查看隐藏列：

```
mysql> set show_hidden_columns=true;

mysql> desc example_table;
+-----------------------+---------+------+-------+---------+-------+
| Field                 | Type    | Null | Key   | Default | Extra |
+-----------------------+---------+------+-------+---------+-------+
| id                    | bigint  | No   | true  | NULL    |       |
| value                 | text    | Yes  | false | NULL    | NONE  |
| __DORIS_DELETE_SIGN__ | tinyint | No   | false | 0       | NONE  |
| __DORIS_VERSION_COL__ | bigint  | No   | false | 0       | NONE  |
+-----------------------+---------+------+-------+---------+-------+
```

数据导入

表中有如下存量数据：

```
+------+-------+
| id   | value |
+------+-------+
```

```
|    1 | foo   |
|    2 | bar   |
+------+-------+
```

通过 INSERT INTO 写入 id 为 1 的删除标记（此处仅做原理展示，不介绍各种导入使用删除标记的方法）：

```
mysql> insert into example_table (id, __DORIS_DELETE_SIGN__) values (1, 1);
```

查询

直接查看数据，可以发现 id 为 1 的记录已被删除：

```
mysql> select * from example_table;
+------+-------+
| id   | value |
+------+-------+
|    2 | bar   |
+------+-------+
```

使用 session 变量 show_hidden_columns 查看隐藏列，可以看到 id 为 1 的行并未被实际删除，其隐藏列 __DORIS_DELETE_SIGN__ 值为 1，在查询时被过滤掉：

```
mysql> set show_hidden_columns=true;
mysql> select * from example_table;
+------+-------+-----------------------+-----------------------+
| id   | value | __DORIS_DELETE_SIGN__ | __DORIS_VERSION_COL__ |
+------+-------+-----------------------+-----------------------+
|    1 | NULL  |                     1 |                     3 |
|    2 | bar   |                     0 |                     2 |
+------+-------+-----------------------+-----------------------+
```

2.10.2.3.3 语法说明

不同导入类型在设置删除标记的语法上有所不同，以下是各种导入类型的删除标记使用语法。

导入合并方式选择

导入数据时有几种合并方式：

1. APPEND：数据全部追加到现有数据中。
2. DELETE：删除所有与导入数据 key 列值相同的行。
3. MERGE：根据 DELETE ON 的条件决定 APPEND 还是 DELETE。

Stream Load

Stream Load 的写法是在 header 中的 columns 字段增加一个设置删除标记列的字段，示例：-H "columns: k1, k2, label_c3" -H "merge_type: [MERGE|APPEND|DELETE]" -H "delete: label_c3=1"。

关于 Stream Load 的使用示例，请查阅 Stream Load 使用手册中“指定 merge_type 进行 Delete 操作”和“指定 merge_type 进行 Merge 操作”章节的内容。

Broker Load

Broker Load 的写法是在 PROPERTIES 处设置删除标记列的字段，语法如下：

```
LOAD LABEL db1.label1
(
    [MERGE|APPEND|DELETE] DATA INFILE("hdfs://abc.com:8888/user/palo/test/ml/file1")
    INTO TABLE tbl1
    COLUMNS TERMINATED BY ","
    (tmp_c1,tmp_c2, label_c3)
    SET
    (
        id=tmp_c2,
        name=tmp_c1,
    )
    [DELETE ON label_c3=true]
)
WITH BROKER 'broker'
(
    "username"="user",
    "password"="pass"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

Routine Load

Routine Load 的写法是在 columns 字段增加映射，映射方式同上，语法如下：

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
 [WITH MERGE|APPEND|DELETE]
 COLUMNS(k1, k2, k3, v1, v2, label),
 WHERE k1  100 and k2 like "%doris%"
 [DELETE ON label=true]
 PROPERTIES
 (
     "desired_concurrent_number"="3",
     "max_batch_interval" = "20",
     "max_batch_rows" = "300000",
     "max_batch_size" = "209715200",
     "strict_mode" = "false"
 )
 FROM KAFKA
 (
     "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
     "kafka_topic" = "my_topic",
```

```
    "kafka_partitions" = "0,1,2,3",
    "kafka_offsets" = "101,0,0,200"
);
```

2.10.2.4 Truncate 操作

该语句用于清空指定表和分区的数据。

2.10.2.4.1 语法

```
TRUNCATE TABLE [db.]tbl [PARTITION(p1, p2, ...)];
```

- 该语句清空数据，但保留表或分区结构。
- 与 DELETE 不同，TRUNCATE 仅进行元数据操作，速度快且不会影响查询性能。
- 该操作删除的数据不可恢复。
- 表状态需为 NORMAL，不能有正在进行的 SCHEMA CHANGE 等操作。
- 该命令可能会导致正在进行的导入任务失败。

2.10.2.4.2 示例

1. 清空 example_db 下的表 tbl

```
TRUNCATE TABLE example_db.tbl;
```

2. 清空表 tbl 的 p1 和 p2 分区

```
TRUNCATE TABLE tbl PARTITION(p1, p2);
```

2.10.2.5 表原子替换

Doris 支持对两个表进行原子替换操作，仅适用于 OLAP 表。

2.10.2.5.1 适用场景

在某些情况下，用户希望重写表数据，但如果先删除再导入，会有一段时间无法查看数据。此时，用户可以先使用 CREATE TABLE LIKE 语句创建一个相同结构的新表，将新数据导入新表后，通过替换操作原子替换旧表。分区级别的原子覆盖写操作，请参阅临时分区文档。

2.10.2.5.2 语法说明

```
ALTER TABLE [db.]tbl1 REPLACE WITH TABLE tbl2
[PROPERTIES('swap' = 'true')];
```

将表 tbl1 替换为表 tbl2。

如果 swap 参数为 `true`，替换后，tbl1 表中的数据为原 tbl2 表中的数据，tbl2 表中的数据为原 tbl1 表中的数据，即两张表数据互换。

如果 swap 参数为 `false`，替换后，tbl1 表中的数据为原 tbl2 表中的数据，tbl2 表被删除。

2.10.2.5.3 原理

替换表功能将以下操作集合变成一个原子操作。

假设将表 A 替换为表 B，且 swap 为 `true`，操作如下：

1. 将表 B 重命名为表 A。
2. 将表 A 重命名为表 B。

如果 swap 为 `false`，操作如下：

1. 删除表 A。
2. 将表 B 重命名为表 A。

2.10.2.5.4 注意事项

- 如果 swap 参数为 `false`，被替换的表（表 A）将被删除，且无法恢复。
- 替换操作仅能发生在两张 OLAP 表之间，不检查两张表的表结构是否一致。
- 替换操作不会改变原有的权限设置，因为权限检查以表名称为准。

2.10.2.6 临时分区

Doris 支持在分区表中添加临时分区。临时分区与正式分区不同，临时分区不会被常规查询检索到，只有通过特殊查询语句才能查询。

- 临时分区的分区列与正式分区相同且不可修改。
- 所有临时分区之间的分区范围不可重叠，但临时分区与正式分区的范围可以重叠。
- 临时分区的名称不能与正式分区或其他临时分区重复。

临时分区的主要应用场景：

- 原子覆盖写操作：用户希望重写某一分区的数据，但不希望在删除旧数据和导入新数据之间有数据缺失。此时，可以创建一个临时分区，将新数据导入临时分区后，通过替换操作原子性地替换原有分区。对于非分区表的原子覆盖写操作，请参阅替换表文档。

- 修改分桶数：用户在创建分区时使用了不合适的分桶数，可以创建一个新的临时分区并指定新的分桶数，然后通过 INSERT INTO 命令将正式分区的数据导入临时分区，再通过替换操作原子性地替换原有分区。
- 合并或分割分区：用户希望修改分区范围，如合并两个分区或将一个大分区分割成多个小分区。可以先建立新的临时分区，然后通过 INSERT INTO 命令将正式分区的数据导入临时分区，再通过替换操作原子性地替换原有分区。

2.10.2.6.1 添加临时分区

使用 ALTER TABLE ADD TEMPORARY PARTITION 语句添加临时分区：

```
ALTER TABLE tbl1 ADD TEMPORARY PARTITION tp1 VALUES LESS THAN("2020-02-01");

ALTER TABLE tbl2 ADD TEMPORARY PARTITION tp1 VALUES [("2020-01-01"), ("2020-02-01"));

ALTER TABLE tbl1 ADD TEMPORARY PARTITION tp1 VALUES LESS THAN("2020-02-01")
("replication_num" = "1")
DISTRIBUTED BY HASH(k1) BUCKETS 5;

ALTER TABLE tbl3 ADD TEMPORARY PARTITION tp1 VALUES IN ("Beijing", "Shanghai");

ALTER TABLE tbl4 ADD TEMPORARY PARTITION tp1 VALUES IN ((1, "Beijing"), (1, "Shanghai"));

ALTER TABLE tbl3 ADD TEMPORARY PARTITION tp1 VALUES IN ("Beijing", "Shanghai")
("replication_num" = "1")
DISTRIBUTED BY HASH(k1) BUCKETS 5;
```

2.10.2.6.2 删除临时分区

使用 ALTER TABLE DROP TEMPORARY PARTITION 语句删除临时分区：

```
ALTER TABLE tbl1 DROP TEMPORARY PARTITION tp1;
```

2.10.2.6.3 替换正式分区

使用 ALTER TABLE REPLACE PARTITION 语句将正式分区替换为临时分区：

```
ALTER TABLE tbl1 REPLACE PARTITION (p1) WITH TEMPORARY PARTITION (tp1);

ALTER TABLE tbl1 REPLACE PARTITION (p1, p2) WITH TEMPORARY PARTITION (tp1, tp2, tp3);

ALTER TABLE tbl1 REPLACE PARTITION (p1, p2) WITH TEMPORARY PARTITION (tp1, tp2)
PROPERTIES (
  "strict_range" = "false",
  "use_temp_partition_name" = "true"
);
```

替换操作有两个特殊的可选参数：

1. strict_range

默认为 true。

对于 Range 分区，当该参数为 true 时，所有被替换的正式分区的范围并集需要与替换的临时分区的范围并集完全相同。当置为 false 时，只需保证替换后新的正式分区间的范围不重叠即可。

对于 List 分区，该参数恒为 true。所有被替换的正式分区的枚举值必须与替换的临时分区枚举值完全相同。

示例 1

```
-- 待替换的分区 p1, p2, p3 的范围 (=> 并集):
[10, 20), [20, 30), [40, 50) => [10, 30), [40, 50)

-- 替换分区 tp1, tp2 的范围 (=> 并集):
[10, 30), [40, 45), [45, 50) => [10, 30), [40, 50)

--范围并集相同，则可以使用 tp1 和 tp2 替换 p1, p2, p3。
```

示例 2

```
-- 待替换的分区 p1 的范围 (=> 并集):
[10, 50) => [10, 50)

-- 替换分区 tp1, tp2 的范围 (=> 并集):
[10, 30), [40, 50) => [10, 30), [40, 50)

-- 范围并集不相同，如果 strict_range 为 true，则不可以使用 tp1 和 tp2 替换 p1。如果为 false,
    ↪ 且替换后的两个分区范围 [10, 30), [40, 50) 和其他正式分区不重叠，则可以替换。
```

示例 3

```
-- 待替换的分区 p1, p2 的枚举值 (=> 并集):
(1, 2, 3), (4, 5, 6) => (1, 2, 3, 4, 5, 6)

-- 替换分区 tp1, tp2, tp3 的枚举值 (=> 并集):
(1, 2, 3), (4), (5, 6) => (1, 2, 3, 4, 5, 6)

-- 枚举值并集相同，可以使用 tp1, tp2, tp3 替换 p1, p2
```

示例 4

```
-- 待替换的分区 p1, p2, p3 的枚举值 (=> 并集):
(("1","beijing"), ("1", "shanghai")), (("2","beijing"), ("2", "shanghai")), (("3","beijing"), ("3
    ↪ ", "shanghai")) => (("1","beijing"), ("1", "shanghai"), ("2","beijing"), ("2", "shanghai"
    ↪ ), ("3","beijing"), ("3", "shanghai"))

-- 替换分区 tp1, tp2 的枚举值 (=> 并集):
```

```
(("1","beijing"), ("1", "shanghai")), (("2","beijing"), ("2", "shanghai"), ("3","beijing"), ("3",
    ↪  "shanghai")) => (("1","beijing"), ("1", "shanghai"), ("2","beijing"), ("2", "shanghai"),
    ↪  ("3","beijing"), ("3", "shanghai"))

-- 枚举值并集相同，可以使用 tp1，tp2 替换 p1，p2，p3
```

2. use_temp_partition_name

默认为 false。

当该参数为 false，并且待替换的分区和替换分区的个数相同时，替换后的正式分区名称维持不变。

如果为 true，替换后正式分区的名称为替换分区的名称。示例如下：

示例 1

```
ALTER TABLE tbl1 REPLACE PARTITION (p1) WITH TEMPORARY PARTITION (tp1);
```

- use_temp_partition_name 默认为 false，则在替换后，分区的名称依然为 p1，但相关的数据和属性都替换为 tp1 的。
- 如果 use_temp_partition_name 为 true，则在替换后，分区的名称为 tp1，p1 分区不再存在。

示例 2

```
ALTER TABLE tbl1 REPLACE PARTITION (p1, p2) WITH TEMPORARY PARTITION (tp1);
```

- use_temp_partition_name 默认为 false，但因待替换分区的个数与替换分区的个数不同，该参数无效。替换后，分区名称为 tp1，p1 和 p2 不再存在。

> 替换操作说明：
>
> 分区替换成功后，被替换的分区将被删除且不可恢复。

2.10.2.6.4 导入临时分区

根据导入方式的不同，指定导入临时分区的语法稍有差别。示例如下：

```
INSERT INTO tbl TEMPORARY PARTITION(tp1, tp2, ...) SELECT ....
curl --location-trusted -u root: -H "label:123" -H "temporary_partitions: tp1, tp2, ..." -T
    ↪ testData http://host:port/api/testDb/testTbl/_stream_load
LOAD LABEL example_db.label1
(
DATA INFILE("hdfs://hdfs_host:hdfs_port/user/palo/data/input/file")
INTO TABLE my_table
TEMPORARY PARTITION (tp1, tp2, ...)
...
```

```
)
WITH BROKER hdfs ("username"="hdfs_user", "password"="hdfs_password");
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100),
TEMPORARY PARTITIONS(tp1, tp2, ...),
WHERE k1 > 100
PROPERTIES
(...)
FROM KAFKA
(...);
```

2.10.2.6.5 查询临时分区

```
SELECT ... FROM
tbl1 TEMPORARY PARTITION(tp1, tp2, ...)
JOIN
tbl2 TEMPORARY PARTITION(tp1, tp2, ...)
ON ...
WHERE ...;
```

2.10.2.6.6 与其他操作的关系

DROP

- 使用 Drop 操作直接删除数据库或表后，可以通过 Recover 命令恢复数据库或表（限定时间内），但临时分区不会被恢复。
- 使用 Alter 命令删除正式分区后，可以通过 Recover 命令恢复分区（限定时间内）。操作正式分区和临时分区无关。
- 使用 Alter 命令删除临时分区后，无法通过 Recover 命令恢复临时分区。

TRUNCATE

- 使用 Truncate 命令清空表，表的临时分区会被删除，且不可恢复。
- 使用 Truncate 命令清空正式分区时，不影响临时分区。
- 不可使用 Truncate 命令清空临时分区。

ALTER

- 当表存在临时分区时，无法使用 Alter 命令对表进行 Schema Change、 Rollup 等变更操作。
- 当表在进行变更操作时，无法对表添加临时分区。

2.10.3 事务

事务是指一个操作，包含一个或多个 SQL 语句，这些语句的执行要么完全成功，要么完全失败，是一个不可分割的工作单位。

2.10.3.1 概览

查询和 DDL 单个语句是一个隐式事务，不支持多语句事务中包含查询和 DDL。每个单独的写入默认是一个隐式的事务，多个写入可以组成一个显式事务。目前 Doris 不支持嵌套事务。

2.10.3.1.1 显式事务

显式事务需要用户主动开启、提交或回滚事务，目前不支持 DDL 和查询语句。

```
BEGIN;
[INSERT, UPDATE, DELETE statement]
COMMIT; / ROLLBACK;
```

2.10.3.1.2 隐式事务

隐式事务是指用户在所执行的一条或多条 SQL 语句的前后，没有显式添加开启事务和提交事务的语句。

在 Doris 中，除Group Commit外，每个导入语句在开始执行时都会开启一个事务，并且在该语句执行完成之后，自动提交该事务；或执行失败后，自动回滚该事务。每个查询或者 DDL 也是一个隐藏事务。

2.10.3.1.3 隔离级别

Doris 当前支持的唯一隔离级别是 READ COMMITTED。在 READ COMMITTED 隔离级别下，语句只能看到在该语句开始执行之前已经提交的数据，它不会看到未提交的数据。

单个语句执行时，会在语句的开始捕获涉及到表的快照，即单个语句只能看见开始执行前其它事务的提交，单个语句执行期间不可见其它事务的提交。

当一个语句在多语句事务中执行时：

- 只能看到在该语句开始执行之前已经提交的数据。如果在执行第一个和第二个语句之间有另一个事务提交，那么同一事务中的两个连续语句可能会看到不同的数据。
- 目前看不到在同一事务中之前语句所做的更改。

2.10.3.1.4 不重不丢

Doris 有两个机制支持写入的不重不丢，使用 Label 机制提供了单个事务的不重，使用两阶段提交提供了协调多事务不重的能力。

Label 机制

Doris 的事务或者写入可以设置一个 Label。这个 Label 通常是用户自定义的、具有一定业务逻辑属性的字符串，不设置时内部会生成一个 UUID 字符串。Label 的主要作用是唯一标识一个事务或者导入任务，并且能够保证相同 Label 的事务或者导入仅会成功执行一次。Label 机制可以保证导入数据的不丢不重，如果上游数据源能

够保证 At-Least-Once 语义，则配合 Doris 的 Label 机制，能够保证 Exactly-Once 语义。Label 在一个数据库下具有唯一性。

Doris 会根据时间和数目清理 Label，默认 Label 数目超过 2000 个就会触发淘汰，默认超过 3 天的 Label 也会被淘汰。Label 被淘汰后相同名称的 Label 可以再次执行成功，即不再具有去重语义。

Label 通常被设置为 业务逻辑+时间 的格式。如 my_business1_20220330_125000。这个 Label 通常用于表示：业务 my_business1 这个业务在 2022-03-30 12:50:00 产生的一批数据。通过这种 Label 设定，业务上可以通过 Label 查询导入任务状态，来明确的获知该时间点批次的数据是否已经导入成功。如果没有成功，则可以使用这个 Label 继续重试导入。

StreamLoad 2PC

StreamLoad 2PC，主要用于支持 Flink 写入 Doris 时的 EOS 语义。

2.10.3.2 显式事务操作

2.10.3.2.1 开启事务

```
BEGIN;

BEGIN WITH LABEL {user_label};
```

如果执行该语句时，当前 Session 正处于一个事务的中间过程，那么 Doris 会忽略该语句，也可以理解为事务是不能嵌套的。

2.10.3.2.2 提交事务

```
COMMIT;
```

用于提交在当前事务中进行的所有修改。

2.10.3.2.3 回滚事务

```
ROLLBACK;
```

用于撤销当前事务的所有修改。

事务是 Session 级别的，如果 Session 中止或关闭，也会自动回滚该事务。

2.10.3.3 多条 SQL 语句写入

目前 Doris 中支持 2 种方式的事务写入。

2.10.3.3.1 单表多次 INSERT INTO VALUES 写入

假如表的结构为：

```
CREATE TABLE `dt` (
    `id` INT(11) NOT NULL,
    `name` VARCHAR(50) NULL,
    `score` INT(11) NULL
) ENGINE=OLAP
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
    "replication_num" = "1"
);
```

写入：

```
mysql> BEGIN;
Query OK, 0 rows affected (0.01 sec)
{'label':'txn_insert_b55db21aad7451b-b5b6c339704920c5', 'status':'PREPARE', 'txnId':''}

mysql> INSERT INTO dt (id, name, score) VALUES (1, "Emily", 25), (2, "Benjamin", 35), (3, "Olivia
    ↪ ", 28), (4, "Alexander", 60), (5, "Ava", 17);
Query OK, 5 rows affected (0.08 sec)
{'label':'txn_insert_b55db21aad7451b-b5b6c339704920c5', 'status':'PREPARE', 'txnId':'10013'}

mysql> INSERT INTO dt VALUES (6, "William", 69), (7, "Sophia", 32), (8, "James", 64), (9, "Emma",
    ↪  37), (10, "Liam", 64);
Query OK, 5 rows affected (0.00 sec)
{'label':'txn_insert_b55db21aad7451b-b5b6c339704920c5', 'status':'PREPARE', 'txnId':'10013'}

mysql> COMMIT;
Query OK, 0 rows affected (1.02 sec)
{'label':'txn_insert_b55db21aad7451b-b5b6c339704920c5', 'status':'VISIBLE', 'txnId':'10013'}
```

这种写入方式不仅可以实现写入的原子性，而且在 Doris 中，能提升 INSERT INTO VALUES 的写入性能。

如果用户同时开启了 Group Commit 和事务写，事务写生效。

2.10.3.3.2 多表多次 INSERT INTO SELECT, UPDATE, DELETE 写入

假设有dt1, dt2, dt3 3 张表，表结构同上，表中数据为：

```
mysql> SELECT * FROM dt1;
+------+-----------+-------+
| id   | name      | score |
+------+-----------+-------+
|    1 | Emily     |    25 |
```

```
|    2 | Benjamin  |    35 |
|    3 | Olivia    |    28 |
|    4 | Alexander |    60 |
|    5 | Ava       |    17 |
+------+-----------+-------+
5 rows in set (0.04 sec)

mysql> SELECT * FROM dt2;
+------+---------+-------+
| id   | name    | score |
+------+---------+-------+
|    6 | William |    69 |
|    7 | Sophia  |    32 |
|    8 | James   |    64 |
|    9 | Emma    |    37 |
|   10 | Liam    |    64 |
+------+---------+-------+
5 rows in set (0.03 sec)

mysql> SELECT * FROM dt3;
Empty set (0.03 sec)
```

做事务写入，把dt1和dt2的数据写入到dt3中，同时，对dt1表中的分数进行更新，dt2表中的数据进行删除：

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'PREPARE', 'txnId':''}


mysql> INSERT INTO dt3 SELECT * FROM dt1;
Query OK, 5 rows affected (0.07 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'PREPARE', 'txnId':'11024'}

mysql> INSERT INTO dt3 SELECT * FROM dt2;
Query OK, 5 rows affected (0.08 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'PREPARE', 'txnId':'11025'}

mysql> UPDATE dt1 SET score = score + 10 WHERE id >= 4;
Query OK, 2 rows affected (0.07 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'PREPARE', 'txnId':'11026'}

mysql> DELETE FROM dt2 WHERE id >= 9;
Query OK, 0 rows affected (0.01 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'PREPARE', 'txnId':'11027'}

mysql> COMMIT;
```

```
Query OK, 0 rows affected (0.03 sec)
{'label':'txn_insert_442a6311f6c541ae-b57d7f00fa5db028', 'status':'VISIBLE', 'txnId':'11024'}
```

查询数据：

```
### id >= 4 的分数加 10
mysql> SELECT * FROM dt1;
+------+-----------+-------+
| id   | name      | score |
+------+-----------+-------+
|    1 | Emily     |    25 |
|    2 | Benjamin  |    35 |
|    3 | Olivia    |    28 |
|    4 | Alexander |    70 |
|    5 | Ava       |    27 |
+------+-----------+-------+
5 rows in set (0.01 sec)

### id >= 9 的数据被删除
mysql> SELECT * FROM dt2;
+------+---------+-------+
| id   | name    | score |
+------+---------+-------+
|    6 | William |    69 |
|    7 | Sophia  |    32 |
|    8 | James   |    64 |
+------+---------+-------+
3 rows in set (0.02 sec)

### dt1 和 dt2 中已提交的数据被写入到 dt3 中
mysql> SELECT * FROM dt3;
+------+-----------+-------+
| id   | name      | score |
+------+-----------+-------+
|    1 | Emily     |    25 |
|    2 | Benjamin  |    35 |
|    3 | Olivia    |    28 |
|    4 | Alexander |    60 |
|    5 | Ava       |    17 |
|    6 | William   |    69 |
|    7 | Sophia    |    32 |
|    8 | James     |    64 |
|    9 | Emma      |    37 |
|   10 | Liam      |    64 |
+------+-----------+-------+
10 rows in set (0.01 sec)
```

隔离级别

目前 Doris 事务写提供的隔离级别为 READ COMMITTED。需要注意以下两点：

- 事务中的多个语句，每个语句会读取到本语句开始执行时已提交的数据，如：

```
 timestamp | ------------ Session 1 ------------  |  ------------ Session 2 ------------
   t1      | BEGIN;                               |
   t2      | # read n rows from dt1 table         |
           | INSERT INTO dt3 SELECT * FROM dt1;   |
   t3      |                                      | # write 2 rows to dt1 table
           |                                      | INSERT INTO dt1 VALUES(...), (...);
   t4      | # read n + 2 rows from dt1 table     |
           | INSERT INTO dt3 SELECT * FROM dt1;   |
   t5      | COMMIT;                              |
```

- 事务中的多个语句，每个语句不能读到本事务内其它语句做出的修改，如：

  假如事务开启前，表 dt1 有 5 行，表 dt2 有 5 行，表 dt3 为空，执行以下语句：

```
BEGIN;
# dt2 中写入 5 行，事务提交后共 10 行
INSERT INTO dt2 SELECT * FROM dt1;
# dt3 中写入 5 行，不能读出上一步中 dt2 中新写入的数据
INSERT INTO dt3 SELECT * FROM dt2;
COMMIT;
```

具体的例子为：

```
# 建表并写入数据
CREATE TABLE `dt1` (
    `id` INT(11) NOT NULL,
    `name` VARCHAR(50) NULL,
    `score` INT(11) NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
    "replication_num" = "1"
);
CREATE TABLE dt2 LIKE dt1;
CREATE TABLE dt3 LIKE dt1;
INSERT INTO dt1 VALUES (1, "Emily", 25), (2, "Benjamin", 35), (3, "Olivia", 28), (4, "
    ↪ Alexander", 60), (5, "Ava", 17);
```

```
INSERT INTO dt2 VALUES (6, "William", 69), (7, "Sophia", 32), (8, "James", 64), (9, "Emma",
    ↪ 37), (10, "Liam", 64);

# 事务写
BEGIN;
INSERT INTO dt2 SELECT * FROM dt1;
INSERT INTO dt3 SELECT * FROM dt2;
COMMIT;

# 查询
mysql> SELECT * FROM dt2;
+------+-----------+-------+
| id   | name      | score |
+------+-----------+-------+
|    6 | William   |    69 |
|    7 | Sophia    |    32 |
|    8 | James     |    64 |
|    9 | Emma      |    37 |
|   10 | Liam      |    64 |
|    1 | Emily     |    25 |
|    2 | Benjamin  |    35 |
|    3 | Olivia    |    28 |
|    4 | Alexander |    60 |
|    5 | Ava       |    17 |
+------+-----------+-------+
10 rows in set (0.01 sec)

mysql> SELECT * FROM dt3;
+------+---------+-------+
| id   | name    | score |
+------+---------+-------+
|    6 | William |    69 |
|    7 | Sophia  |    32 |
|    8 | James   |    64 |
|    9 | Emma    |    37 |
|   10 | Liam    |    64 |
+------+---------+-------+
5 rows in set (0.01 sec)
```

事务中执行失败的语句

当事务中的某个语句执行失败时，这个操作已经自动回滚。然而，事务中其它执行成功的语句，仍然是可提交或回滚的。当事务被成功提交后，事务中执行成功的语句的修改被应用。

比如：

```
mysql> BEGIN;
```

```
Query OK, 0 rows affected (0.00 sec)
{'label':'txn_insert_c5940d31bf364f57-a48b628886415442', 'status':'PREPARE', 'txnId':''}

mysql> INSERT INTO dt3 SELECT * FROM dt1;
Query OK, 5 rows affected (0.07 sec)
{'label':'txn_insert_c5940d31bf364f57-a48b628886415442', 'status':'PREPARE', 'txnId':'11058'}

### 失败的写入自动回滚
mysql> INSERT INTO dt3 SELECT * FROM dt2;
ERROR 5025 (HY000): Insert has filtered data in strict mode, tracking_url=http
    ↪ ://172.21.16.12:9082/api/_load_error_log?file=__shard_3/error_log_insert_stmt_3
    ↪ d1fed266ce443f2-b54d2609c2ea6b11_3d1fed266ce443f2_b54d2609c2ea6b11

mysql> INSERT INTO dt3 SELECT * FROM dt2 WHERE id = 7;
Query OK, 0 rows affected (0.07 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)
{'label':'txn_insert_c5940d31bf364f57-a48b628886415442', 'status':'VISIBLE', 'txnId':'11058'}
```

查询：

```
### dt1 的数据被写入到 dt3 中，dt2 中 id = 7的数据写入成功，其它写入失败
mysql> SELECT * FROM dt3;
+------+----------+-------+
| id   | name     | score |
+------+----------+-------+
|    1 | Emily    |    25 |
|    2 | Benjamin |    35 |
|    3 | Olivia   |    28 |
|    4 | Alexande |    60 |
|    5 | Ava      |    17 |
|    7 | Sophia   |    32 |
+------+----------+-------+
6 rows in set (0.01 sec)
```

常见问题

- 写入的多表必须属于同一个 Database，否则会遇到错误 `Transaction insert must be in the same database`
- 两种事务写入`INSERT INTO SELECT`, `UPDATE`, `DELETE` 和 `INSET INTO VALUES` 不能混用，否则会遇到错误 `Transaction insert can not insert into values and insert into select at the same time`
- Delete 操作提供了通过谓词和 Using 子句两种方式，为了保证隔离级别，在一个事务中，对相同表的删除必须在写入前，否则会遇到报错 `Can not delete because there is a insert operation for the same table`

- 当从 BEGIN 开始的导入耗时超出 Doris 配置的 timeout 时，会导致事务回滚，导入失败。目前 timeout 使用的是 Session 变量 `insert_timeout` 和 `query_timeout` 的最大值
- 当使用 JDBC 连接 Doris 进行事务操作时，请在 JDBC URL 中添加 `useLocalSessionState=true`，否则可能会遇到错误 `This is in a transaction, only insert, update, delete, commit, rollback is acceptable` ↪ .
- 存算分离模式下，事务写不支持 Merge-on-Write 表，否则会遇到报错 `Transaction load is not supported` ↪ `for merge on write unique keys table in cloud mode`

2.10.3.4 Stream Load 2PC

1. 在 HTTP Header 中设置 `two_phase_commit:true` 启用两阶段提交。

```
curl  --location-trusted -u user:passwd -H "two_phase_commit:true" -T test.txt http://fe_host:
    ↪ http_port/api/{db}/{table}/_stream_load
{
    "TxnId": 18036,
    "Label": "55c8ffc9-1c40-4d51-b75e-f2265b3602ef",
    "TwoPhaseCommit": "true",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 100,
    "NumberLoadedRows": 100,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 1031,
    "LoadTimeMs": 77,
    "BeginTxnTimeMs": 1,
    "StreamLoadPutTimeMs": 1,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 58,
    "CommitAndPublishTimeMs": 0
}
```

2. 对事务触发 commit 操作（请求发往 FE 或 BE 均可）

- 可以使用事务 id 指定事务

```
  curl -X PUT --location-trusted -u user:passwd -H "txn_id:18036" -H "txn_operation:commit" http
      ↪ ://fe_host:http_port/api/{db}/{table}/_stream_load_2pc
  {
      "status": "Success",
      "msg": "transaction [18036] commit successfully."
  }
```

- 也可以使用 label 指定事务

```
curl -X PUT --location-trusted -u user:passwd  -H "label:55c8ffc9-1c40-4d51-b75e-f2265b3602ef"
    ↪ -H "txn_operation:commit"  http://fe_host:http_port/api/{db}/{table}/_stream_load_2pc
{
    "status": "Success",
    "msg": "label [55c8ffc9-1c40-4d51-b75e-f2265b3602ef] commit successfully."
}
```

3. 对事务触发 abort 操作（请求发往 FE 或 BE 均可）

- 可以使用事务 id 指定事务

```
curl -X PUT --location-trusted -u user:passwd  -H "txn_id:18037" -H "txn_operation:abort"  http
    ↪ ://fe_host:http_port/api/{db}/{table}/_stream_load_2pc
{
    "status": "Success",
    "msg": "transaction [18037] abort successfully."
}
```

- 也可以使用 label 指定事务

```
curl -X PUT --location-trusted -u user:passwd  -H "label:55c8ffc9-1c40-4d51-b75e-f2265b3602ef"
    ↪ -H "txn_operation:abort"  http://fe_host:http_port/api/{db}/{table}/_stream_load_2pc
{
    "status": "Success",
    "msg": "label [55c8ffc9-1c40-4d51-b75e-f2265b3602ef] abort successfully."
}
```

2.10.3.5 Broker Load 多表事务

所有 Broker Load 导入任务都是原子生效的。并且在同一个导入任务中对多张表的导入也能够保证原子性。还可以通过 Label 的机制来保证数据导入的不丢不重。

下面例子是从 HDFS 导入数据，使用通配符匹配两批文件，分别导入到两个表中。

```
LOAD LABEL example_db.label2
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file-10*")
    INTO TABLE `my_table1`
    PARTITION (p1)
    COLUMNS TERMINATED BY ","
    (k1, tmp_k2, tmp_k3)
    SET (
        k2 = tmp_k2 + 1,
```

```
        k3 = tmp_k3 + 1
    )
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file-20*")
    INTO TABLE `my_table2`
    COLUMNS TERMINATED BY ","
    (k1, k2, k3)
)
WITH BROKER hdfs
(
    "username"="hdfs_user",
    "password"="hdfs_password"
);
```

使用通配符匹配导入两批文件 `file-10*` 和 `file-20*`。分别导入到 `my_table1` 和 `my_table2` 两张表中。其中 `my_table1` 指定导入到分区 `p1` 中，并且将导入源文件中第二列和第三列的值 `+1` 后导入。

2.11 数据导出

2.11.1 数据导出概述

数据导出功能，用于将查询结果集或者 Apache Doris 的表数据，使用指定的文件格式，写入指定的存储系统中的。

导出功能和数据备份功能有以下区别：

	数据导出	数据备份
数据最终存储位置	HDFS、对象存储、本地文件系统	HDFS、对象存储
数据格式	Parquet、ORC、CSV 等开放格式	Apache Doris 内部存储格式
执行速度	中等（需要读取数据并转换成目标数据格式）	快速（无需解析和转换，直接上传 Apache Doris 数据文件
灵活度	可以通过 SQL 语句灵活定义要导出的数据	仅支持表级别全量备份
使用场景	结果集下载、不同系统之间的数据交换	数据备份、Apache Doris 集群间的数据迁移

2.11.1.1 选择导出方式

Apache Doris 提供以下三种不同的数据导出方式：

- SELECT INTO OUTFILE：支持任意 SQL 结果集的导出。
- EXPORT：支持表级别的部分或全部数据导出。
- MySQL DUMP：兼容 MySQL Dump 指令的数据导出。

三种导出方式的异同点如下：

	SELECT INTO OUTFILE	EXPORT	MySQL DUMP
同步/异步	同步	异步（提交 EXPORT 任务后通过 SHOW EXPORT 命令查看任务进度）	同步
支持任意 SQL	支持	不支持	不支持
导出指定分区	支持	支持	不支持
导出指定 Tablets	支持	不支持	不支持
并发导出	支持且并发高（但取决于 SQL 语句是否有 ORDER BY 等需要单机处理的算子）	支持且并发高（支持 Tablet 粒度的并发导出）	不支持，只能单线程导出
支持导出的数据格式	Parquet、ORC、CSV	Parquet、ORC、CSV	MySQL Dump 专有格式
是否支持导出外表	支持	部分支持	不支持
是否支持导出 View	支持	支持	支持

	SELECT INTO OUTFILE	EXPORT	MySQL DUMP
支持的导出位置	S3、HDFS	S3、HDFS	LOCAL

2.11.1.1.1 SELECT INTO OUTFILE

适用于以下场景：

- 导出数据需要经过复杂计算逻辑的，如过滤、聚合、关联等。
- 适合执行同步任务的场景。

2.11.1.1.2 EXPORT

适用于以下场景：

- 大数据量的单表导出、仅需简单的过滤条件。
- 需要异步提交任务的场景。

2.11.1.1.3 MySQL Dump

适用于以下场景：

- 兼容 MySQL 生态，需要同时导出表结构和数据。
- 仅用于开发测试或者数据量很小的情况。

2.11.1.2 导出文件列类型映射

Parquet、 ORC 文件格式拥有自己的数据类型。Apache Doris 的导出功能能够自动将 Apache Doris 的数据类型导出为 Parquet、 ORC 文件格式的对应数据类型。CSV 格式没有类型，所有数据都以文本形式输出。

以下是 Apache Doris 数据类型和 Parquet、 ORC 文件格式的数据类型映射关系表：

- ORC

Doris Type	Orc Type
boolean	boolean
tinyint	tinyint
smallint	smallint
int	int
bigint	bigint
largeInt	string
date	string

Doris Type	Orc Type
datev2	string
datetime	string
datetimev2	timestamp
float	float
double	double
char / varchar / string	string
decimal	decimal
struct	struct
map	map
array	array
json	string
variant	string
bitmap	binary
quantile_state	binary
hll	binary

- Parquet

  Doris 导出到 Parquet 文件格式时，会先将 Doris 内存数据转换为 Arrow 内存数据格式，然后由 Arrow 写出到 Parquet 文件格式。

Doris Type	Arrow Type	Parquet Physical Type	Parquet Logical Type
boolean	boolean	BOOLEAN	
tinyint	int8	INT32	INT_8
smallint	int16	INT32	INT_16
int	int32	INT32	INT_32
bigint	int64	INT64	INT_64
largeInt	utf8	BYTE_ARRAY	UTF8
date	utf8	BYTE_ARRAY	UTF8
datev2	date32	INT32	DATE
datetime	utf8	BYTE_ARRAY	UTF8
datetimev2	timestamp	INT96/INT64	TIMESTAMP(MICROS/MILLIS/SECONDS)
float	float32	FLOAT	
double	float64	DOUBLE	
char / varchar / string	utf8	BYTE_ARRAY	UTF8
decimal	decimal128	FIXED_LEN_BYTE_ARRAY	DECIMAL(scale, precision)
struct	struct		Parquet Group
map	map		Parquet Map
array	list		Parquet List
json	utf8	BYTE_ARRAY	UTF8
variant	utf8	BYTE_ARRAY	UTF8
bitmap	binary	BYTE_ARRAY	
quantile_state	binary	BYTE_ARRAY	
hll	binary	BYTE_ARRAY	

> 注：在 2.1.11 和 3.0.7 版本中，支持指定 parquet.enable_int96_timestamps 属性，来决定 Doris 的 datetimev2 类型，是使用 Parquet 的 INT96 存储还是 INT64。默认使用 INT96。但 INT96 在 Parquet 标准中已经废弃，仅用于兼容一些旧系统（如 Hive 4.0 之前的版本）。

2.11.2 Export

本文档将介绍如何使用EXPORT命令导出 Doris 中存储的数据。

Export 是 Doris 提供的一种将数据异步导出的功能。该功能可以将用户指定的表或分区的数据，以指定的文件格式，导出到目标存储系统中，包括对象存储、HDFS 或本地文件系统。

Export 是一个异步执行的命令，命令执行成功后，立即返回结果，用户可以通过Show Export 命令查看该 Export 任务的详细信息。

有关EXPORT命令的详细介绍，请参考：EXPORT

关于如何选择 SELECT INTO OUTFILE 和 EXPORT，请参阅导出综述。

2.11.2.1 适用场景

Export 适用于以下场景：

- 大数据量的单表导出、仅需简单的过滤条件。
- 需要异步提交任务的场景。

使用 Export 时需要注意以下限制：

- 当前不支持文本文件压缩格式的导出。
- 不支持 Select 结果集导出。若需要导出 Select 结果集，请使用OUTFILE 导出

2.11.2.2 快速上手

2.11.2.2.1 建表与导入数据

```
CREATE TABLE IF NOT EXISTS tbl (
    `c1` int(11) NULL,
    `c2` string NULL,
    `c3` bigint NULL
)
DISTRIBUTED BY HASH(c1) BUCKETS 20
PROPERTIES("replication_num" = "1");


insert into tbl values
    (1, 'doris', 18),
```

```
    (2, 'nereids', 20),
    (3, 'pipelibe', 99999),
    (4, 'Apache', 122123455),
    (5, null, null);
```

2.11.2.2.2 创建导出作业

导出到 HDFS

将 tbl 表的所有数据导出到 HDFS 上，设置导出作业的文件格式为 csv（默认格式），并设置列分割符为 ,。

```
EXPORT TABLE tbl
TO "hdfs://host/path/to/export_"
PROPERTIES
(
    "line_delimiter" = ","
)
with HDFS (
    "fs.defaultFS"="hdfs://hdfs_host:port",
    "hadoop.username" = "hadoop"
);
```

导出到对象存储

将 tbl 表中的所有数据导出到对象存储上，设置导出作业的文件格式为 csv（默认格式），并设置列分割符为 ,。

```
EXPORT TABLE tbl TO "s3://bucket/export/export_"
PROPERTIES (
    "line_delimiter" = ","
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

2.11.2.2.3 查看导出作业

提交作业后，可以通过SHOW EXPORT 命令查询导出作业状态，结果举例如下：

```
mysql> show export`\G`
*************************** 1. row ***************************
      JobId: 143265
      Label: export_0aa6c944-5a09-4d0b-80e1-cb09ea223f65
      State: FINISHED
   Progress: 100%
```

```
   TaskInfo: {"partitions":[],"parallelism":5,"data_consistency":"partition","format":"csv","
       ↪ broker":"S3","column_separator":"`\t`","line_delimiter":"`\n`","max_file_size":"2048MB
       ↪ ","delete_existing_files":"","with_bom":"false","db":"tpch1","tbl":"lineitem"}
       Path: s3://bucket/export/export_
 CreateTime: 2024-06-11 18:01:18
  StartTime: 2024-06-11 18:01:18
 FinishTime: 2024-06-11 18:01:31
    Timeout: 7200
   ErrorMsg: NULL
OutfileInfo: [
  [
    {
      "fileNumber": "1",
      "totalRows": "6001215",
      "fileSize": "747503989",
      "url": "s3://bucket/export/export_6555cd33e7447c1-baa9568b5c4eb0ac_*"
    }
  ]
]
1 row in set (0.00 sec)
```

有关 show export 命令的详细用法及其返回结果的各个列的含义可以参看SHOW EXPORT：

2.11.2.2.4 取消导出作业

提交 Export 作业后，在 Export 任务成功或失败之前可以通过CANCEL EXPORT 命令取消导出作业。取消命令举例如下：

```
CANCEL EXPORT FROM dbName WHERE LABEL like "%export_%";
```

2.11.2.3 导出说明

2.11.2.3.1 导出数据源

EXPORT 当前支持导出以下类型的表或视图

- Doris 内表
- Doris 逻辑视图
- External Catalog 中的表

2.11.2.3.2 导出数据存储位置

Export 目前支持导出到以下存储位置：

- 对象存储：Amazon S3、 COS、 OSS、 OBS、 Google GCS
- HDFS

2.11.2.3.3 导出文件类型

EXPORT 目前支持导出为以下文件格式：

- Parquet
- ORC
- csv
- csv_with_names
- csv_with_names_and_types

2.11.2.4 导出示例

2.11.2.4.1 导出到开启了高可用的 HDFS 集群

如果 HDFS 开启了高可用，则需要提供 HA 信息，如：

```
EXPORT TABLE tbl
TO "hdfs://HDFS8000871/path/to/export_"
PROPERTIES
(
    "line_delimiter" = ","
)
with HDFS (
    "fs.defaultFS" = "hdfs://HDFS8000871",
    "hadoop.username" = "hadoop",
    "dfs.nameservices" = "your-nameservices",
    "dfs.ha.namenodes.your-nameservices" = "nn1,nn2",
    "dfs.namenode.rpc-address.HDFS8000871.nn1" = "ip:port",
    "dfs.namenode.rpc-address.HDFS8000871.nn2" = "ip:port",
    "dfs.client.failover.proxy.provider.HDFS8000871" = "org.apache.hadoop.hdfs.server.namenode.ha
        ↪ .ConfiguredFailoverProxyProvider"
);
```

2.11.2.4.2 导出到开启了高可用及 kerberos 认证的 HDFS 集群

如果 Hadoop 集群开启了高可用并且启用了 Kerberos 认证，可以参考如下 SQL 语句：

```
EXPORT TABLE tbl
TO "hdfs://HDFS8000871/path/to/export_"
PROPERTIES
(
    "line_delimiter" = ","
)
with HDFS (
    "fs.defaultFS"="hdfs://hacluster/",
    "hadoop.username" = "hadoop",
```

```
    "dfs.nameservices"="hacluster",
    "dfs.ha.namenodes.hacluster"="n1,n2",
    "dfs.namenode.rpc-address.hacluster.n1"="192.168.0.1:8020",
    "dfs.namenode.rpc-address.hacluster.n2"="192.168.0.2:8020",
    "dfs.client.failover.proxy.provider.hacluster"="org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider",
    "dfs.namenode.kerberos.principal"="hadoop/_HOST@REALM.COM"
    "hadoop.security.authentication"="kerberos",
    "hadoop.kerberos.principal"="doris_test@REALM.COM",
    "hadoop.kerberos.keytab"="/path/to/doris_test.keytab"
);
```

2.11.2.4.3 指定分区导出

导出作业支持仅导出 Doris 内表的部分分区，如仅导出 test 表的 p1 和 p2 分区

```
EXPORT TABLE test
PARTITION (p1,p2)
TO "s3://bucket/export/export_"
PROPERTIES (
    "columns" = "k1,k2"
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

2.11.2.4.4 导出时过滤数据

导出作业支持导出时根据谓词条件过滤数据，仅导出符合条件的数据，如仅导出满足 k1 < 50 条件的数据

```
EXPORT TABLE test
WHERE k1 < 50
TO "s3://bucket/export/export_"
PROPERTIES (
    "columns" = "k1,k2",
    "column_separator"=","
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

2.11.2.4.5 导出外表数据

导出作业支持 Export Catalog 外表数据的导出：

```
-- Create a hive catalog
CREATE CATALOG hive_catalog PROPERTIES (
    'type' = 'hms',
    'hive.metastore.uris' = 'thrift://172.0.0.1:9083'
);

-- Export hive table
EXPORT TABLE hive_catalog.sf1.lineitem TO "s3://bucket/export/export_"
PROPERTIES(
    "format" = "csv",
    "max_file_size" = "1024MB"
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

2.11.2.4.6 导出前清空导出目录

```
EXPORT TABLE test TO "s3://bucket/export/export_"
PROPERTIES (
    "format" = "parquet",
    "max_file_size" = "512MB",
    "delete_existing_files" = "true"
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

如果设置了 `"delete_existing_files" = "true"`，导出作业会先将 `s3://bucket/export/` 目录下所有文件及目录删除，然后导出数据到该目录下。

若要使用 `delete_existing_files` 参数，还需要在 `fe.conf` 中添加配置 `enable_delete_existing_files = true` 并重启 fe，此时 `delete_existing_files` 才会生效。该操作会删除外部系统的数据，属于高危操作，请自行确保外部系统的权限和数据安全性。

2.11.2.4.7 设置导出文件的大小

导出作业支持设置导出文件的大小，如果单个文件大小超过设定值，则会对导出文件进行切分。

```
EXPORT TABLE test TO "s3://bucket/export/export_"
PROPERTIES (
    "format" = "parquet",
    "max_file_size" = "512MB"
) WITH s3 (
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

通过设置 "max_file_size" = "512MB"，则单个导出文件的最大大小为 512MB。

max_file_size 不能小于 5MB 且不能大于 2GB。

在 2.1.11 和 3.0.7 版本中，取消了 2GB 的最大限制，仅保留最小 5MB 的限制。

2.11.2.5 注意事项

- 导出数据量

不建议一次性导出大量数据。一个 Export 作业建议的导出数据量最大在几十 GB。过大的导出会导致更多的垃圾文件和更高的重试成本。如果表数据量过大，建议按照分区导出。

另外，Export 作业会扫描数据，占用 IO 资源，可能会影响系统的查询延迟。

- 导出文件的管理

如果 Export 作业运行失败，已经生成的文件不会被删除，需要用户手动删除。

- 导出超时

若导出的数据量很大，超过导出的超时时间，则 Export 任务会失败。此时可以在 Export 命令中指定 timeout 参数来增加超时时间并重试 Export 命令。

- 导出失败

在 Export 作业运行过程中，如果 FE 发生重启或切主，则 Export 作业会失败，需要用户重新提交。可以通过show export 命令查看 Export 任务状态。

- 导出分区数量

一个 Export Job 允许导出的分区数量最大为 2000，可以在 fe.conf 中添加参数maximum_number_of_export_
↪ partitions并重启 FE 来修改该设置。

- 数据完整性

导出操作完成后，建议验证导出的数据是否完整和正确，以确保数据的质量和完整性。

2.11.2.6 附录

2.11.2.6.1 基本原理

Export 任务的底层是执行SELECT INTO OUTFILE SQL 语句。用户发起一个 Export 任务后，Doris 会根据 Export 要导出的表构造出一个或多个 SELECT INTO OUTFILE 执行计划，随后将这些SELECT INTO OUTFILE 执行计划提交给 Doris 的 Job Schedule 任务调度器，Job Schedule 任务调度器会自动调度这些任务并执行。

2.11.2.6.2 导出到本地文件系统

导出到本地文件系统功能默认是关闭的。这个功能仅用于本地调试和开发，请勿用于生产环境。

如要开启这个功能请在 fe.conf 中添加 enable_outfile_to_local=true 并且重启 FE。

示例：将 tbl 表中的所有数据导出到本地文件系统，设置导出作业的文件格式为 csv（默认格式），并设置列分割符为 ,。

```
EXPORT TABLE db.tbl TO "file:///path/to/result_"
PROPERTIES (
  "format" = "csv",
  "line_delimiter" = ","
);
```

此功能会将数据导出并写入到 BE 所在节点的磁盘上，如果有多个 BE 节点，则数据会根据导出任务的并发度分散在不同 BE 节点上，每个节点有一部分数据。

如在这个示例中，最终会在 BE 节点的 /path/to/ 下生产一组类似 result_7052bac522d840f5-972079771289
↪ e392_0.csv 的文件。

具体的 BE 节点 IP 可以在 SHOW EXPORT 结果中的 OutfileInfo 列查看，如：

```
[
    [
        {
            "fileNumber": "1",
            "totalRows": "0",
            "fileSize": "8388608",
            "url": "file:///172.20.32.136/path/to/result_7052bac522d840f5-972079771289e392_*"
        }
    ],
    [
        {
            "fileNumber": "1",
            "totalRows": "0",
            "fileSize": "8388608",
            "url": "file:///172.20.32.137/path/to/result_22aba7ec933b4922-ba81e5eca12bf0c2_*"
        }
    ]
]
```

此功能不适用于生产环境，并且请自行确保导出目录的权限和数据安全性。

2.11.3 SELECT INTO OUTFILE

本文档将介绍如何使用 SELECT INTO OUTFILE 命令进行查询结果的导出操作。

SELECT INTO OUTFILE 命令将 SELECT 部分的结果数据，以指定的文件格式导出到目标存储系统中，包括对象存储或 HDFS。

SELECT INTO OUTFILE 是一个同步命令，命令返回即表示导出结束。若导出成功，会返回导出的文件数量、大小、路径等信息。若导出失败，会返回错误信息。

关于如何选择 SELECT INTO OUTFILE 和 EXPORT，请参阅导出综述。

有关SELECT INTO OUTFILE命令的详细介绍，请参考：SELECT INTO OUTFILE

2.11.3.1 适用场景

SELECT INTO OUTFILE 适用于以下场景：

- 导出数据需要经过复杂计算逻辑的，如过滤、聚合、关联等。
- 适合需要执行同步任务的场景。

在使用 SELECT INTO OUTFILE 时需要注意以下限制：

- 不支持文本压缩格式的导出。
- 2.1 版本 pipeline 引擎不支持并发导出。

2.11.3.2 快速上手

2.11.3.2.1 建表与导入数据

```
CREATE TABLE IF NOT EXISTS tbl (
    `c1` int(11) NULL,
    `c2` string NULL,
    `c3` bigint NULL
)
DISTRIBUTED BY HASH(c1) BUCKETS 20
PROPERTIES("replication_num" = "1");

insert into tbl values
    (1, 'doris', 18),
    (2, 'nereids', 20),
    (3, 'pipelibe', 99999),
```

```
    (4, 'Apache', 122123455),
    (5, null, null);
```

2.11.3.2.2 导出到 HDFS

将查询结果导出到文件 `hdfs://path/to/` 目录下，指定导出格式为 Parquet：

```
SELECT c1, c2, c3 FROM tbl
INTO OUTFILE "hdfs://ip:port/path/to/result_"
FORMAT AS PARQUET
PROPERTIES
(
    "fs.defaultFS" = "hdfs://ip:port",
    "hadoop.username" = "hadoop"
);
```

2.11.3.2.3 导出到对象存储

将查询结果导出到 s3 存储的 `s3://bucket/export/` 目录下，指定导出格式为 ORC，需要提供sk ak等信息

```
SELECT * FROM tbl
INTO OUTFILE "s3://bucket/export/result_"
FORMAT AS ORC
PROPERTIES(
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx"
);
```

2.11.3.3 导出说明

2.11.3.3.1 导出数据存储位置

`SELECT INTO OUTFILE` 目前支持导出到以下存储位置：

- 对象存储：Amazon S3、 COS、 OSS、 OBS、 Google GCS
- HDFS

2.11.3.3.2 导出文件类型

`SELECT INTO OUTFILE` 目前支持导出以下文件格式

- Parquet
- ORC

- csv
- csv_with_names
- csv_with_names_and_types

2.11.3.3.3 导出并发度

可以通过会话参数 enable_parallel_outfile 开启并发导出。

```
SET enable_parallel_outfile=true;
```

并发导出会利用多节点、多线程导出结果数据，以提升整体的导出效率。但并发导出可能会产生更多的文件。

注意，某些查询即使打开此参数，也无法执行并发导出，如包含全局排序的查询。如果导出命令返回的行数大于 1 行，则表示开启了并发导出。

2.11.3.4 导出示例

2.11.3.4.1 导出到开启了高可用的 HDFS 集群

如果 HDFS 开启了高可用，则需要提供 HA 信息，如：

```
SELECT c1, c2, c3 FROM tbl
INTO OUTFILE "hdfs://HDFS8000871/path/to/result_"
FORMAT AS PARQUET
PROPERTIES
(
    "fs.defaultFS" = "hdfs://HDFS8000871",
    "hadoop.username" = "hadoop",
    "dfs.nameservices" = "your-nameservices",
    "dfs.ha.namenodes.your-nameservices" = "nn1,nn2",
    "dfs.namenode.rpc-address.HDFS8000871.nn1" = "ip:port",
    "dfs.namenode.rpc-address.HDFS8000871.nn2" = "ip:port",
    "dfs.client.failover.proxy.provider.HDFS8000871" = "org.apache.hadoop.hdfs.server.namenode.ha
        ↪ .ConfiguredFailoverProxyProvider"
);
```

2.11.3.4.2 导出到开启了高可用及 kerberos 认证的 HDFS 集群

如果 Hdfs 集群开启了高可用并且启用了 Kerberos 认证，可以参考如下 SQL 语句：

```
SELECT * FROM tbl
INTO OUTFILE "hdfs://path/to/result_"
FORMAT AS PARQUET
PROPERTIES
(
    "fs.defaultFS"="hdfs://hacluster/",
    "hadoop.username" = "hadoop",
    "dfs.nameservices"="hacluster",
```

```
    "dfs.ha.namenodes.hacluster"="n1,n2",
    "dfs.namenode.rpc-address.hacluster.n1"="192.168.0.1:8020",
    "dfs.namenode.rpc-address.hacluster.n2"="192.168.0.2:8020",
    "dfs.client.failover.proxy.provider.hacluster"="org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider",
    "dfs.namenode.kerberos.principal"="hadoop/_HOST@REALM.COM"
    "hadoop.security.authentication"="kerberos",
    "hadoop.kerberos.principal"="doris_test@REALM.COM",
    "hadoop.kerberos.keytab"="/path/to/doris_test.keytab"
);
```

2.11.3.4.3 生成导出成功标识文件示例

SELECT INTO OUTFILE 命令是一个同步命令，因此有可能在 SQL 执行过程中任务连接断开了，从而无法获悉导出的数据是否正常结束或是否完整。此时可以使用 success_file_name 参数要求导出成功后，在目录下生成一个文件标识。

类似 Hive，用户可以通过判断导出目录中是否有success_file_name 参数指定的文件，来判断导出是否正常结束以及导出目录中的文件是否完整。

例如：将 select 语句的查询结果导出到对象存储：s3://bucket/export/。指定导出格式为 csv。指定导出成功标识文件名为 SUCCESS。导出完成后，生成一个标识文件。

```
SELECT k1,k2,v1 FROM tbl1 LIMIT 100000
INTO OUTFILE "s3://bucket/export/result_"
FORMAT AS CSV
PROPERTIES
(
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx",
    "column_separator" = ",",
    "line_delimiter" = "`\n`",
    "success_file_name" = "SUCCESS"
);
```

在导出完成后，会多写出一个文件，该文件的文件名为 SUCCESS。

2.11.3.4.4 导出前清空导出目录

```
SELECT * FROM tbl1
INTO OUTFILE "s3://bucket/export/result_"
FORMAT AS CSV
PROPERTIES
(
    "s3.endpoint" = "xxxxx",
```

```
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx",
    "column_separator" = ",",
    "line_delimiter" = "`\n`",
    "delete_existing_files" = "true"
);
```

如果设置了 "delete_existing_files" = "true"，导出作业会先将 s3://bucket/export/ 目录下所有文件及目录删除，然后导出数据到该目录下。

若要使用 delete_existing_files 参数，还需要在 fe.conf 中添加配置 enable_delete_existing_files = true 并重启 fe，此时 delete_existing_files 才会生效。该操作会删除外部系统的数据，属于高危操作，请自行确保外部系统的权限和数据安全性。

2.11.3.4.5 设置导出文件的大小

```
SELECT * FROM tbl
INTO OUTFILE "s3://path/to/result_"
FORMAT AS ORC
PROPERTIES(
    "s3.endpoint" = "xxxxx",
    "s3.region" = "xxxxx",
    "s3.secret_key"="xxxx",
    "s3.access_key" = "xxxxx",
    "max_file_size" = "2048MB"
);
```

由于指定了 "max_file_size" = "2048MB" 最终生成文件如如果不大于 2GB，则只有一个文件。如果大于 2GB，则有多个文件。

2.11.3.5 注意事项

- 导出数据量和导出效率

  SELECT INTO OUTFILE功能本质上是执行一个 SQL 查询命令。如果不开启并发导出，查询结果是由单个 BE 节点，单线程导出的，因此整个导出的耗时包括查询本身的耗时和最终结果集写出的耗时。开启并发导出可以降低导出的时间。

- 导出超时

  导出命令的超时时间与查询的超时时间相同，如果数据量较大导致导出数据超时，可以设置会话变量 query_timeout 适当的延长查询超时时间。

- 导出文件的管理

  Doris 不会管理导出的文件，无论是导出成功的还是导出失败后残留的文件，都需要用户自行处理。

  另外，SELECT INTO OUTFILE 命令不会检查文件及文件路径是否存在。SELECT INTO OUTFILE 是否会自动创建路径、或是否会覆盖已存在文件，完全由远端存储系统的语义决定。

- 如果查询的结果集为空

  对于结果集为空的导出，依然会产生一个空文件。

- 文件切分

  文件切分会保证一行数据完整的存储在单一文件中。因此文件的大小并不严格等于 `max_file_size`。

- 非可见字符的函数

  对于部分输出为非可见字符的函数，如 BITMAP、HLL 类型，导出到 CSV 文件格式时输出为 `\N`。

2.11.3.6 附录

2.11.3.6.1 导出到本地文件系统

导出到本地文件系统功能默认是关闭的。这个功能仅用于本地调试和开发，请勿用于生产环境。

如要开启这个功能请在 `fe.conf` 中添加 `enable_outfile_to_local=true` 并且重启 FE。

示例：将 tbl 表中的所有数据导出到本地文件系统，设置导出作业的文件格式为 csv（默认格式），并设置列分割符为`,`。

```
SELECT c1, c2 FROM db.tbl
INTO OUTFILE "file:///path/to/result_"
FORMAT AS CSV
PROPERTIES(
    "column_separator" = ","
);
```

此功能会将数据导出并写入到 BE 所在节点的磁盘上，如果有多个 BE 节点，则数据会根据导出任务的并发度分散在不同 BE 节点上，每个节点有一部分数据。

如在这个示例中，最终会在 BE 节点的 `/path/to/` 下生产一组类似 `result_c6df5f01bd664dde-a2168b019b6c2b3f_0.csv` 的文件。

具体的 BE 节点 IP 会在返回的结果中显示，如：

```
+------------+-----------+----------+--------------------------------------------------------------------+
| FileNumber | TotalRows | FileSize | URL                                                                |
+------------+-----------+----------+--------------------------------------------------------------------+
|          1 |   1195072 |  4780288 | file:///172.20.32.136/path/to/result_c6df5f01bd664dde-a2168b019b6c2b3f_* |
|          1 |   1202944 |  4811776 | file:///172.20.32.136/path/to/result_c6df5f01bd664dde-a2168b019b6c2b40_* |
|          1 |   1198880 |  4795520 | file:///172.20.32.137/path/to/result_c6df5f01bd664dde-a2168b019b6c2b43_* |
|          1 |   1198880 |  4795520 | file:///172.20.32.137/path/to/result_c6df5f01bd664dde-a2168b019b6c2b45_* |
```

```
+------------+-----------+----------+-------------------------------------------------------------------------+-------------+
    ↪
```

此功能不适用于生产环境，并且请自行确保导出目录的权限和数据安全性。

2.11.4 MySQL Dump

Doris 在 0.15 之后的版本已经支持通过 `mysqldump` 工具导出数据或者表结构

2.11.4.1 使用示例

2.11.4.1.1 导出

1. 导出 test 数据库中的 table1 表：`mysqldump -h127.0.0.1 -P9030 -uroot --no-tablespaces --databases ↪ test --tables table1`
2. 导出 test 数据库中的 table1 表结构：`mysqldump -h127.0.0.1 -P9030 -uroot --no-tablespaces --databases test --tables table1 --no-data`
3. 导出 test1, test2 数据库中所有表：`mysqldump -h127.0.0.1 -P9030 -uroot --no-tablespaces --databases test1 test2`
4. 导出所有数据库和表 `mysqldump -h127.0.0.1 -P9030 -uroot --no-tablespaces --all-databases`

更多的使用参数可以参考`mysqldump` 的使用手册

2.11.4.1.2 导入

`mysqldump`导出的结果可以重定向到文件中，之后可以通过 source 命令导入到 Doris 中 `source filenamme.sql`

2.11.4.2 注意

1. 由于 Doris 中没有 MySQL 里的 tablespace 概念，因此在使用 MySQL Dump 时要加上 `--no-tablespaces` 参数
2. 使用 MySQL Dump 导出数据和表结构仅用于开发测试或者数据量很小的情况，请勿用于大数据量的生产环境

2.11.5 最佳实践

本文档主要用于介绍在进行数据导出操作中，如何判断资源利用是否合理，以及如何调整资源利用率已达到更好的数据导出效率。

2.11.5.1 SELECT INTO OUTFILE

1. 开启并行导出

   SELECT INTO OUTFILE 返回的行数即代表并行的 Writer 数量。Writer 的数量越多，则导出的并发度越高，但写出的文件数量也会越多。如果发现只有一个 Writer，则可以尝试开启并行导出功能。

   ```
   SET enable_parallel_outfile=true
   ```

   开启后，SELECT INTO OUTFILE 操作为根据查询的并行度来生成对应数量的 Writer。查询并行度由会话变量 parallel_pipeline_task_num 控制。默认为单 BE CPU 核数的一半。

   比如在一个 3 BE 节点的集群中，每个节点的 CPU 核数为 8。则开启并行导出情况下，会产生（4*3=）12 个 Writer。

   注意，即使开启了并行导出功能，也不是所有查询都能够并行导出。比如查询中包含全局排序、聚合语义时，则是无法并行导出的。如：

```
SELECT * FROM table ORDER BY id;

SELECT SUM(cost) FROM table;
```

2. 判断导出速度

   SELECT INTO OUTFILE 返回的每一行结果中，都带有对应的 Writer 的写出时的时间（单位：秒）和速度（单位：KB/s）。

   将同一个节点的多个 Writer 的速度相加，即为单个节点的写出速度。可以用这个速度，和磁盘带宽（比如导出到本地）或网络带宽（比如导出到对象存储）进行比较，看是否已经达到带宽瓶颈。

2.11.5.2 Export

1. 根据返回结果判断导出执行情况

   Export 命令本质上是将任务拆分成多个 SELECT INTO OUTFILE 子句进行执行。

   通过 SHOW EXPORT 命令返回的结果中包含一个 Json 字符串，是一个二维数组。第一维代表 Export 并发的线程数，并发多少个线程代表并发发起了多少个 Outfile 语句。第二维代表单个 Outfile 语句的返回结果。示例：

```
[
    [
        {
            "fileNumber": "1",
            "totalRows": "640321",
            "fileSize": "350758307",
            "url": "file:///127.0.0.1/mnt/disk2/ftw/tmp/export/exp_59fd917c43874adc-9
                ↪ b1c3e9cd6e655be_*",
            "writeTime": "17.989",
            "writeSpeed": "19041.66"
```

```
            },
            {...},
            {...},
            {...}
        ],
        [
            {
                "fileNumber": "1",
                "totalRows": "646609",
                "fileSize": "354228704",
                "url": "file:///127.0.0.1/mnt/disk2/ftw/tmp/export/exp_c75b9d4b59bf4943-92
                    ↪ eb94a7b97e46cb_*",
                "writeTime": "17.249",
                "writeSpeed": "20054.64"
            },
            {...},
            {...},
            {...}
        ]
    ]
```

上面的示例中，发起了 2 个 Outfile 命令。每个命令有 4 个 Writer 并发写出。

通过调整 Export 命令属性中的 `parallelism` 参数，可以控制并发 Outfile 的个数，从而控制并发度。

2. 影响并行度的参数

   Export 作业的并行度取决于两个参数：

   - parallelism

     用于设置最多拆分成几个 Outfile 命令。

   - data_consistency

     是否在分区内部进行 Outfile 命令的拆分。该参数默认为 partition，即不对分区进一步拆分。即 Outfile 命令的数量，只会小于等于涉及到的分区数量。如果设置为 none，则会对一个分区进一步拆分，这样可以提高并发，但如果分区在写入数据，则可能会牺牲导出的一致性（即同一个分区的不同 Outfile 命令，可能导出的是这个分区的不同版本的数据）。

     具体可参阅Export 命令手册

   - async_task_consumer_thread_num

     FE 配置参数，表示当前集群能够同时运行的 Export Task 的数量，默认是是 64。一个 Export Job 会根据并发度拆分成多个 Export Task。所有 Export Task 共享这个阈值。如果希望提升集群整体的可并发执行导出任务的数量，可以调大这个参数，并重启 FE 节点。

2.12 数据查询

2.12.1 MySQL 兼容性

Doris 高度兼容 MySQL 语法，支持标准 SQL。但是 Doris 与 MySQL 还是有很多不同的地方，下面给出了它们的差异点介绍。

2.12.1.1 数据类型

2.12.1.1.1 数字类型

类型	MySQL	Doris
Boolean	- 支持 - 范围：0 代表 false，1 代表 true	- 支持 - 关键字：Boolean - 范围：0 代表 false，1 代表 true
Bit	- 支持 - 范围：1 ~ 64	不支持
Tinyint	- 支持 - 支持 signed,unsigned - 范围：signed 的范围是 -128 ~ 127，unsigned 的范围是 0 ~ 255	- 支持 - 只支持 signed - 范围：-128 ~ 127
Smallint	- 支持 - 支持 signed,unsigned - 范围：signed 的范围是 -2^15 ~ 2^15-1，unsigned 的范围是 0 ~ 2^16-1	- 支持 - 只支持 signed - 范围：-32768 ~ 32767
Mediumint	- 支持 - 支持 signed,unsigned - 范围：signed 的范围是 -2^23 ~ 2^23-1，unsigned 的范围是 0 ~ 2^24-1	- 不支持
int	- 支持 - 支持 signed,unsigned - 范围：signed 的范围是 -2^31 ~ 2^31-1，unsigned 的范围是 0 ~ 2^32-1	- 支持 - 只支持 signed - 范围：-2147483648~2147483647
Bigint	- 支持 - 支持 signed,unsigned - 范围：signed 的范围是 -2^63 ~ 2^63-1，unsigned 的范围是 0 ~ 2^64-1	- 支持 - 只支持 signed - 范围：-2^63 ~ 2^63-1
Largeint	- 不支持	- 支持 - 只支持 signed - 范围：-2^127 ~ 2^127-1
Decimal	- 支持 - 支持 signed,unsigned（8.0.17 以前支持，该版本以上标记为 deprecated）- 默认值：Decimal(10, 0)	- 支持 - 只支持 signed - 默认值：Decimal(9, 0)
Float/Double	- 支持 - 支持 signed,unsigned（8.0.17 以前支持，该版本以上标记为 deprecated）	- 支持 - 只支持 signed

2.12.1.1.2 日期类型

类型	MySQL	Doris
Date	- 支持 - 范围：[‘1000-01-01’，‘9999-12-31’] - 格式：YYYY-MM-DD	- 支持 - 范围：[‘0000-01-01’，‘9999-12-31’] - 格式：YYYY-MM-DD
DateTime	- 支持 - DATETIME([P])，可选参数 P 表示精度 - 范围：‘1000-01-01 00:00:00.000000’，‘9999-12-31 23:59:59.999999’ - 格式：YYYY-MM-DD hh:mm:ss[.fraction]	- 支持 - DATETIME([P])，可选参数 P 表示精度 - 范围：[‘0000-01-01 00:00:00[.000000]’，‘9999-12-31 23:59:59[.999999]’] - 格式：YYYY-MM-DD hh:mm:ss[.fraction]

类型	MySQL	Doris
Timestamp	- 支持 - Timestamp[(p)]，可选参数 P 表示精度 - 范围：[‘1970-01-01 00:00:01.000000’ UTC , ‘2038-01-19 03:14:07.999999’ UTC] - 格式：YYYY-MM-DD hh:mm:ss[.fraction]	- 不支持
Time	- 支持 - Time[(p)] - 范围：[‘-838:59:59.000000’ to ‘838:59:59.000000’] - 格式：hh:mm:ss[.fraction]	- 不支持
Year	- 支持 - 范围：1901 to 2155, or 0000 - 格式：yyyy	- 不支持

2.12.1.1.3 字符串类型

类型	MySQL	Doris
Char	- 支持 - CHAR(M)，M 为字符长度，缺省表示长度为 1 - 定长 - 范围：[0,255]，字节大小	- 支持 - CHAR(M)，M 为字节长度 - 可变 - 范围：[1,255]
Varchar	- 支持 - VARCHAR(M)，M 为字符长度 - 范围：[0,65535]，字节大小	- 支持 - VARCHAR(M)，M 为字节长度。- 范围：[1,65533]
String	- 不支持	- 支持 - 1048576 字节（1MB），可调大到 2147483643 字节（2G）
Binary	- 支持 - 类似于 Char	- 不支持
Varbinary	- 支持 - 类似于 Varchar	- 不支持
Blob	- 支持 - TinyBlob、 Blob、 MediumBlob、 LongBlob	- 不支持
Text	- 支持 - TinyText、 Text、 MediumText、 LongText	- 不支持
Enum	- 支持 - 最多支持 65535 个 elements	- 不支持
Set	- 支持 - 最多支持 64 个 elements	- 不支持

2.12.1.1.4 JSON 数据类型

类型	MySQL	Doris
JSON	支持	支持

2.12.1.1.5 Doris 特有的数据类型

- HyperLogLog

HLL 类型不能作为 Key 列使用。在 Aggregate 模型表中使用时，建表时配合的聚合类型为 HLL_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。并且 HLL 列只能通过配套的 HLL_UNION_AGG、HLL_RAW_AGG、 HLL_CARDINALITY、 HLL_HASH 进行查询或使用。

HLL 是模糊去重，在处理大数据量时，其性能优于 Count Distinct。HLL 的误差率通常在 1% 左右，有时可能会达到 2%。

- BITMAP

BITMAP 类型不能作为 Key 列使用。在 Aggregate 表中使用时，还需配合 BITMAP_UNION 聚合定义。用户无需指定长度和默认值，长度会根据数据的聚合程度由系统内部控制。并且，BITMAP 列只能通过配套的 BITMAP_UNION_COUNT、BITMAP_UNION、 BITMAP_HASH、 BITMAP_HASH64 等函数进行查询或使用。

离线场景下使用 BITMAP 可能会影响导入速度，在数据量大的情况下，其查询速度会慢于 HLL，但优于 Count Distinct。注意：在实时场景下，如果 BITMAP 不使用全局字典，而使用了 BITMAP_HASH()，可能会导致约千分之一的误差。如果此误差不可接受，可以使用 BITMAP_HASH64。

- QUANTILE_PERCENT（QUANTILE_STATE）

QUANTILE_STATE 类型不能作为 Key 列使用。在 Aggregate 模型表中使用时，建表时配合的聚合类型为 QUANTILE_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。并且 QUANTILE_STATE 列只能通过配套的 QUANTILE_PERCENT、 QUANTILE_UNION、 TO_QUANTILE_STATE 等函数进行查询或使用。

QUANTILE_STATE 是一种计算分位数近似值的类型，在导入时会对相同的 Key，不同 Value 进行预聚合，当 Value 数量不超过 2048 时，会采用明细记录所有数据，当 Value 数量大于 2048 时采用 TDigest 算法，对数据进行聚合（聚类），并保存聚类后的质心点。

- Array

Array 由 T 类型元素组成的数组，不能作为 Key 列使用。

- MAP

Map 是由 K, V 类型元素组成的映射表，不能作为 Key 列使用。

- STRUCT

Struct 由多个 Field 组成的结构体，也可被理解为多个列的集合。不能作为 Key 使用。

一个 Struct 中的 Field 的名字和数量固定，且总是为 Nullable，一个 Field 通常由下面部分组成：

- field_name: Field 的标识符，不可重复
- field_type: Field 的类型
- Agg_State

AGG_STATE 不能作为 Key 列使用，建表时需要同时声明聚合函数的签名。

用户不需要指定长度和默认值。实际存储的数据大小与函数实现有关。

AGG_STATE 只能配合STATE / MERGE / UNION函数组合器使用。

2.12.1.2 语法区别

2.12.1.2.1 DDL

1 CREATE TABLE

Doris 建表语法：

```
CREATE TABLE [IF NOT EXISTS] [database.]table
(
    column_definition_list
    [, index_definition_list]
)
[engine_type]
[keys_type]
[table_comment]
[partition_info]
distribution_desc
[rollup_list]
[properties]
[extra_properties]
```

与 MySQL 的不同之处：

参数	与 MySQL 不同之处
column_definition_list	- 字段列表定义，其基本语法与 MySQL 类似。- Doris 额外包含一个聚合类型的操作，主要支持的数据模型为 Aggregate Key。- MySQL 允许在字段列表定义后添加 Index 等约束，如 Primary Key、 Unique Key 等；而 Doris 则是通过定义数据模型来实现对这些约束和计算的支持。
index_definition_list	- 索引列表定义，基本语法与 MySQL 类似 - MySQL 支持位图索引、倒排索引和 N-Gram 索引。另外可以通过属性设置布隆过滤器索引。- MySQL 支持 B+Tree 索引和 Hash 索引。
engine_type	- 表引擎类型，可选。- 目前支持的表引擎主要是 OLAP 原生引擎。- MySQL 支持的存储引擎有：Innodb，MyISAM 等
keys_type	- 数据模型，可选。- 支持的类型包括：1）DUPLICATE KEY（默认）：其后指定的列为排序列。2）AGGREGATE KEY：其后指定的列为维度列。3）UNIQUE KEY：其后指定的列为主键列。- MySQL 则没有数据模型的概念。
table_comment	表注释
partition_info	分区算法，可选。Doris 支持的分区算法，包括：- LESS THAN：仅定义分区上界。下界由上一个分区的上界决定。- FIXED RANGE：定义分区的左闭右开区间。- MULTI RANGE：批量创建 RANGE 分区，定义分区的左闭右开区间，设定时间单位和步长，时间单位支持年、月、日、周和小时。MySQL 支持的算法：Hash，Range，List Key，并且还支持子分区，子分区支持的算法有 Hash 和 Key。
distribution_desc	- 分桶算法，必选，包括：1）Hash 分桶语法：DISTRIBUTED BY HASH (k1[,k2 …]) [BUCKETS num|auto] 说明：使用指定的 key 列进行哈希分桶。2）Random 分桶语法：DISTRIBUTED BY RANDOM [BUCKETS num|auto] 说明：使用随机数进行分桶。- MySQL 没有分桶算法。

参数	与 MySQL 不同之处
rollup_list	- 建表的同时可以创建多个同步物化视图。- 语法：`rollup_name (col1[, col2, ...])[DUPLICATE KEY(col1[, col2, ...])][PROPERTIES("key" = "value")]` - MySQL 不支持
properties	表属性，与 MySQL 的表属性不一致，定义表属性的语法也与 MySQL 不一致

2 CREATE INDEX

```
CREATE INDEX [IF NOT EXISTS] index_name ON table_name (column [, ...],) [USING BITMAP];
```

- 目前支持：位图索引、倒排索引和 N-Gram 索引，布隆过滤器索引（单独的语法设置）
- MySQL 支持的索引算法有：B+Tree，Hash

3 CREATE VIEW

```
CREATE VIEW [IF NOT EXISTS]
 [db_name.]view_name
 (column1[ COMMENT "col comment"][, column2, ...])
AS query_stmt

CREATE MATERIALIZED VIEW [IF NOT EXISTS] mvName=multipartIdentifier
        (LEFT_PAREN cols=simpleColumnDefs RIGHT_PAREN)? buildMode?
        (REFRESH refreshMethod? refreshTrigger?)?
        (KEY keys=identifierList)?
        (COMMENT STRING_LITERAL)?
        (PARTITION BY LEFT_PAREN partitionKey = identifier RIGHT_PAREN)?
        (DISTRIBUTED BY (HASH hashKeys=identifierList | RANDOM) (BUCKETS (INTEGER_VALUE | AUTO))
            ↪ ?)?
        propertyClause?
        AS query
```

- 基本语法与 MySQL 一致
- Doris 除了支持逻辑视图外，还支持两种物化视图，同步物化视图和异步物化视图
- MySQL 不支持物化视图

4 ALTER TABLE / ALTER INDEX

Doris Alter 的语法与 MySQL 的基本一致。

2.12.1.2.2 DROP TABLE / DROP INDEX

Doris Drop 的语法与 MySQL 的基本一致

2.12.1.2.3 DML

1 INSERT

```
INSERT INTO table_name
    [ PARTITION (p1, ...) ]
    [ WITH LABEL label]
    [ (column [, ...]) ]
    [ [ hint [, ...] ] ]
    { VALUES ( { expression | DEFAULT } [, ...] ) [, ...] | query }
```

Doris Insert 语法与 MySQL 的基本一致。

2 UPDATE

```
UPDATE target_table [table_alias]
    SET assignment_list
    WHERE condition

assignment_list:
    assignment [, assignment] ...

assignment:
    col_name = value

value:
    {expr | DEFAULT}
```

Doris Update 语法与 MySQL 基本一致，但需要注意的是必须加上 WHERE 条件。

3 DELETE

```
DELETE FROM table_name [table_alias]
    [PARTITION partition_name | PARTITIONS (partition_name [, partition_name])]
    WHERE column_name op { value | value_list } [ AND column_name op { value | value_list } ...];
```

Doris 该语法只能指定过滤谓词

```
DELETE FROM table_name [table_alias]
    [PARTITION partition_name | PARTITIONS (partition_name [, partition_name])]
    [USING additional_tables]
    WHERE condition
```

Doris 该语法只能在 Unique Key 模型表上使用。

Doris Delete 语法与 MySQL 基本一致。但是由于 Doris 是一个分析数据库，所以删除不能过于频繁。

4 SELECT

```
SELECT
    [hint_statement, ...]
```

```
    [ALL | DISTINCT]
    select_expr [, select_expr ...]
    [EXCEPT ( col_name1 [, col_name2, col_name3, ...] )]
    [FROM table_references
      [PARTITION partition_list]
      [TABLET tabletid_list]
      [TABLESAMPLE sample_value [ROWS | PERCENT]
        [REPEATABLE pos_seek]]
    [WHERE where_condition]
    [GROUP BY [GROUPING SETS | ROLLUP | CUBE] {col_name | expr | position}]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
    [LIMIT {[offset_count,] row_count | row_count OFFSET offset_count}]
    [INTO OUTFILE 'file_name']
```

Doris Select 语法与 MySQL 基本一致

2.12.1.3 SQL Function

Doris Function 基本覆盖绝大部分 MySQL Function。

2.12.1.4 SQL Mode

名称	设置的行为	未设置的行为	备注
PIPES_AS_CONCAT	将 || 符号解析为 concat 函数	将 || 符号解析为逻辑与操作符	-
NO_BACKSLASH_ESCAPES	将字符串中的反斜杠当做正常字符解析	将字符串中的反斜杠当做转义起始字符	-
ONLY_FULL_GROUP_BY	只允许标准的聚合	允许聚合结果输出中包含不在聚合 KEY 中的标量值	自 3.1.0

2.12.2 连接（JOIN）

2.12.2.1 什么是 JOIN

在关系型数据库中，数据被分布在多个表中，这些表之间通过特定关系相互关联。SQL JOIN 操作允许我们根据这些关联条件将不同的表合并成一个更完整的结果集。

2.12.2.2 Doris 支持的 JOIN 类型

- INNER JOIN（内连接）：对左表每一行和右表所有行进行 JOIN 条件比较，返回两个表中满足 JOIN 条件的匹配行。详细信息请参考SELECT 中有关于联接查询的语法定义
- LEFT JOIN（左连接）：在 INNER JOIN 的结果集基础上。如果左表的行在右表中没有匹配，则返回左表的所有行，同时右表对应的列显示为 NULL。
- RIGHT JOIN（右连接）：与 LEFT JOIN 相反，如果右表的行在左表中没有匹配，则返回右表的所有行，同时左表对应的列显示为 NULL。

- FULL JOIN（全连接）：在 INNER JOIN 的结果集基础上。返回两个表中所有的行，如果某行在另一侧表中没有匹配，则另一侧表的相应列显示为 NULL。
- CROSS JOIN（交叉连接）：没有 JOIN 条件，返回两个表的笛卡尔积，即左表的每一行与右表的每一行都进行组合。
- LEFT SEMI JOIN（左半连接）：对左表每一行和右表所有行进行 JOIN 条件比较，如果存在匹配，就返回左表的对应行。
- RIGHT SEMI JOIN（右半连接）：与 LEFT SEMI JOIN 相反，对右表每一行和左表所有行进行 JOIN 条件比较，如果存在匹配，就返回右表的对应行。
- LEFT ANTI JOIN（左反半连接）：对左表每一行和右表所有行进行 JOIN 条件比较，如果没有匹配，则返回左表的对应行。
- RIGHT ANTI JOIN（右反半连接）：与 LEFT ANTI JOIN 相反，对右表每一行和左表所有行进行 JOIN 条件比较，如果没有匹配，则返回这些行。
- NULL AWARE LEFT ANTI JOIN (对 NULL 值特殊处理的左反半连接)：与 LEFT ANTI JOIN 类似，但忽略左表中匹配列为 NULL 的行。

2.12.2.3 Doris 中的 JOIN 物理实现

Doris 支持两种 JOIN 的物理实现方式：Hash Join 和 Nest Loop Join。

- Hash Join：在右表上根据等值 JOIN 列构建一个哈希表，左表的数据以流式方式通过该哈希表进行 JOIN 计算。这种方法的局限性在于它仅适用于等值 JOIN 条件的情况。
- Nest Loop Join： 通过两层循环，以左表驱动，对左表的每一行逐一遍历右表的每一行，进行 join 条件判断。适用于所有 JOIN 场景，包括处理 Hash Join 无法胜任的情况，比如涉及大于或小于比较条件的查询，或是需要执行笛卡尔积运算的场景。但相比 Hash Join，Nest Loop Join 在性能上可能会有所不及。

2.12.2.4 Doris Hash Join 的实现方式

作为分布式 MPP 数据库，Apache Doris 在 Hash Join 过程中需要进行数据的 Shuffle，进行拆分调度，以确保 JOIN 结果的正确性。以下是几种数据 Shuffle 方式：

2.12.2.4.1 Broadcast Join

如图所示，Broadcast Join 的过程涉及将右表的所有数据发送到所有参与 Join 计算的节点，包括左表数据的扫描节点，而左表数据则保持不动。这一过程中，每个节点都会接收到右表的完整数据副本（总量为 T(R) 的数据），以确保所有节点都具备执行 Join 操作所需的数据。

该方法适用于多种通用场景，但不适用于 RIGHT OUTER, RIGHT ANTI, 和 RIGHT SEMI 类型的 Hash Join。其网络开销为 Join 的节点数 N 乘以右表数据量 T(R)。

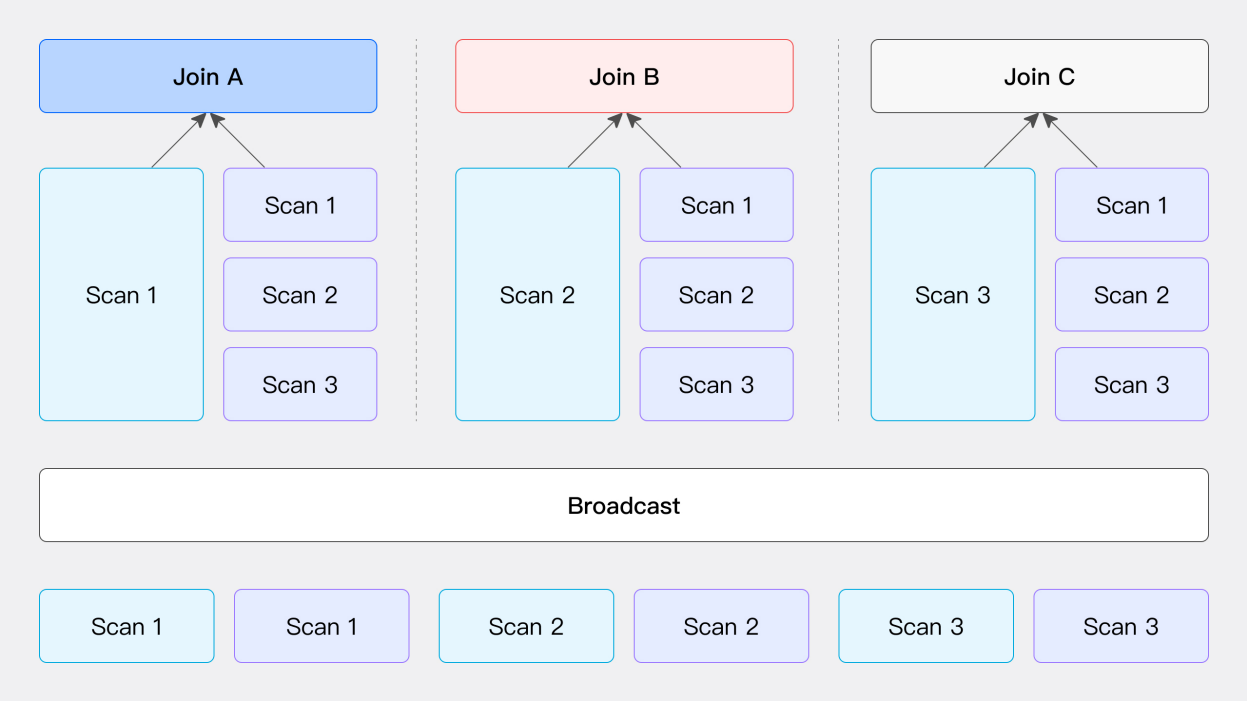


图 47: Implementation of Hash Join in Doris

2.12.2.4.2 Partition Shuffle Join

此方式通过 JOIN 条件计算 Hash 值并进行分桶。具体来说，左右表的数据会根据 JOIN 条件计算得到的 Hash 值进行分区，然后这些分区数据被发送到相应的分区节点上（如图所示）。

该方法的网络开销主要包括两个部分：传输左表数据 T(S) 所需的开销和传输右表数据 T(R) 所需的开销。该方法的仅支持 Hash Join 操作，因为它依赖于 JOIN 条件来执行数据的分桶操作。

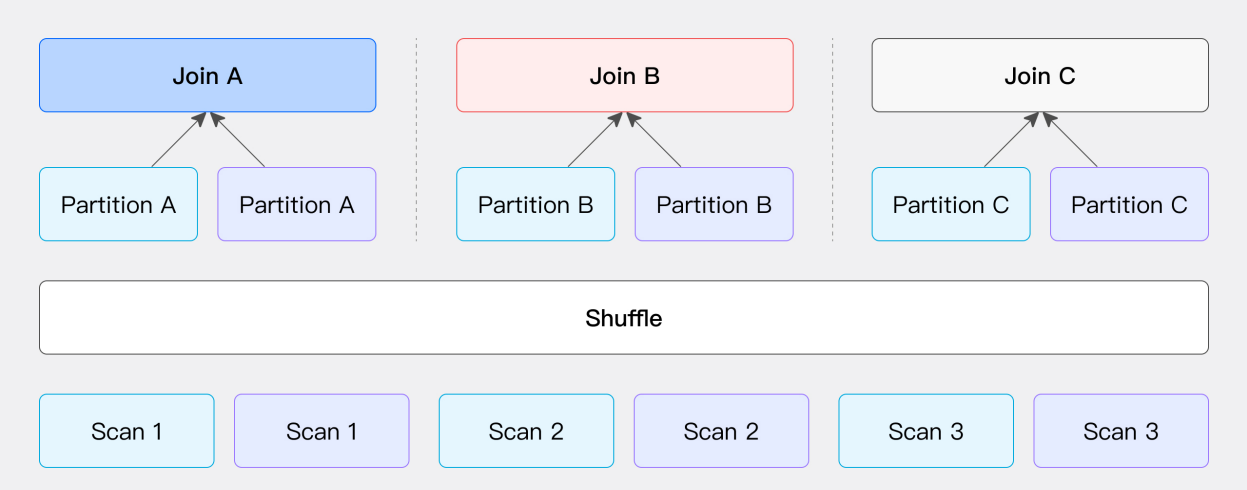


图 48: Partition Shuffle Join

2.12.2.4.3 Bucket Shuffle Join

当 JOIN 条件包含左表的分桶列时，保持左表数据不动，将右表数据分发到左表节点进行 JOIN，减少网络开销。

当参与 Join 操作的某一侧表的数据已经按照 Join 条件列进行了 Hash 分布时，我们可以选择保持这一侧的数据位置不变，而将另一侧的数据依据相同的 Join 条件列，相同的 Hash 分布计算进行数据分发。(这里提到的“表”不仅限于物理存储的表，还可以是 SQL 查询中任意算子的输出结果，并且可以灵活选择保持左表或右表的数据位置不变，而只移动并分发另一侧的表。)

以 Doris 的物理表为例，由于其表数据本身就是通过 Hash 计算进行分桶存储，因此可以直接利用这一特性来优化 Join 操作的数据 Shuffle 过程。假设我们有两张表需要进行 Join，且 Join 列是左表的分桶列，那么在这种情况下，我们无需移动左表的数据，只需根据左表的分桶信息将右表的数据分发到相应的位置，即可完成 Join 计算 (如图所示)。

此过程的网络开销主要来自于右表数据的移动，即 T(R)。

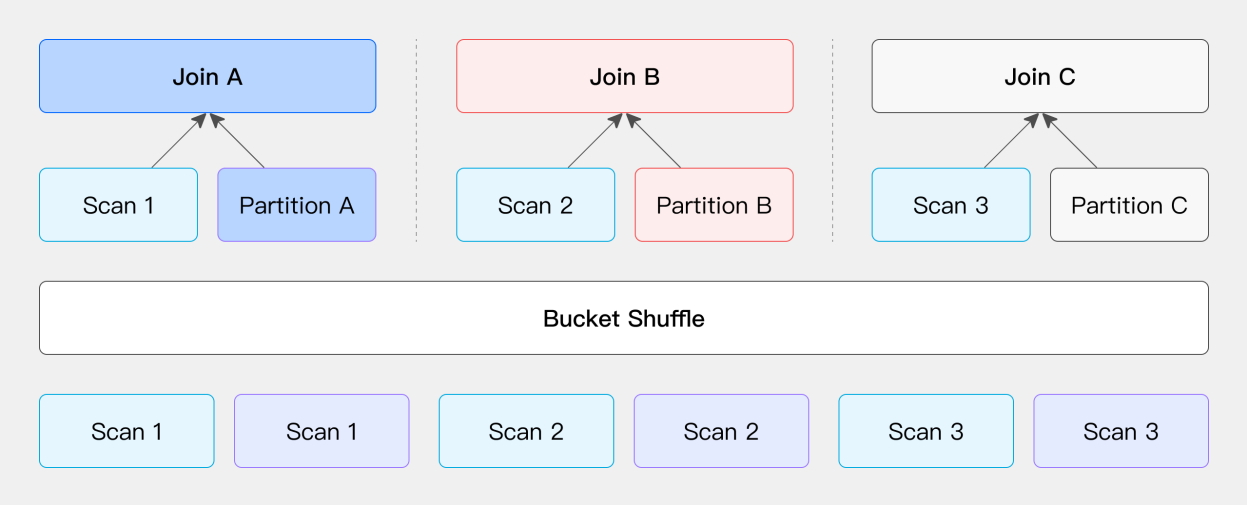


图 49: Bucket Shuffle Join

2.12.2.4.4 Colocate Join

与 Bucket Shuffle Join 相似，如果参与 Join 的两侧的表，刚好是按照 Join 条件列进行计算的 Hash 分布，那么可以跳过 Shuffle 过程，直接在本地进行 Join 计算。以下通过物理表进行简单说明：

当 Doris 在建表时指定为 DISTRIBUTED BY HASH，那么在数据导入时，系统会根据 Hash 分布键进行数据分发。如果两张表的 Hash 分布键恰好与 Join 条件列一致，那么可以认为这两张表的数据已经按照 Join 的需求进行了预分布，即无需额外的 Shuffle 操作。因此，在实际查询时，可以直接在这两张表上执行 Join 计算。

> 注意对于直接 Scan 数据后执行 Join 的场景，建表时需要满足一定的条件，具体请参考后续关于两张物理表进行Colocate Join 的限制说明。

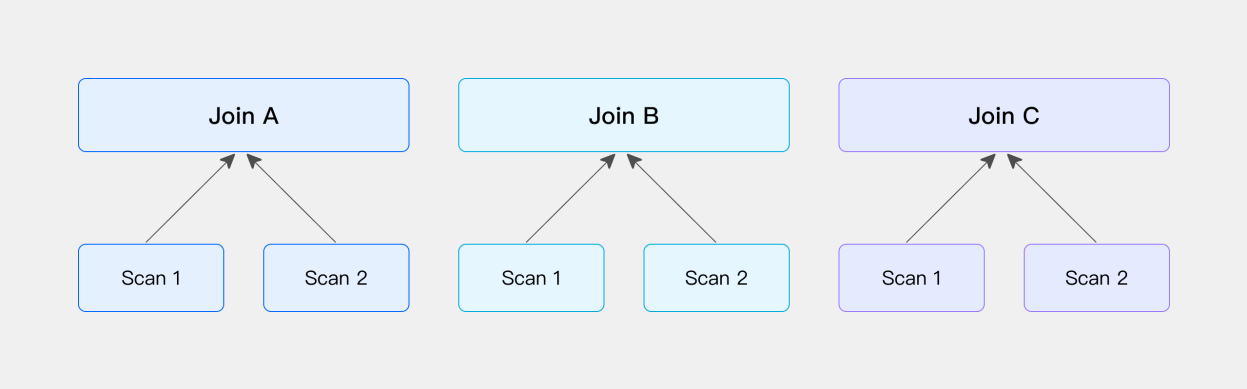


图 50: Colocate Join

2.12.2.5 对比 Bucket Shuffle Join 与 Colocate Join

上文我们提到过，对于 Bucket Shuffle Join 和 Colocate Join 只要参与 Join 操作的两侧的表分布满足特定条件，就可以执行相应的 join 操作（这里的表指的是更广义的表，即 SQL 查询中任意算子的输出都可以视为一张“表”）。

接下来，我们将分别通过 t1 和 t2 两张表以及相关的 SQL 示例，来更详细地介绍广义上的 Bucket Shuffle Join 和 Colocate Join。首先，给出这两张表的建表语句如下：

```
create table t1
(
    c1 bigint,
    c2 bigint
)
DISTRIBUTED BY HASH(c1) BUCKETS 3
PROPERTIES ("replication_num" = "1");

create table t2
(
    c1 bigint,
    c2 bigint
)
DISTRIBUTED BY HASH(c1) BUCKETS 3
PROPERTIES ("replication_num" = "1");
```

2.12.2.5.1 Bucket Shuffle Join 示例

在下面的例子中，t1 和 t2 表都经过了 GROUP BY 算子处理，并输出了新的表（此时 tx 表按照 c1 进行 hash 分布，而 ty 表则按照 c2 进行 Hash 分布）。随后的 JOIN 条件是 tx.c1 = ty.c2，这恰好满足了 Bucket Shuffle Join 的条件。

```
explain select *
from
    (
        -- t1 表按照 c1 做了 hash 分布，经过 group by 算子后，仍然保持按照 c1 做的 hash 分布
```

```
        select c1 as c1, sum(c2) as c2
        from t1
        group by c1
    ) tx
join
    (
        -- t2 表按照 c1 做了 hash 分布，经过 group by 算子后，数据分布变成按照 c2 进行的 hash
            ↪ 分布
        select c2 as c2, sum(c1) as c1
        from t2
        group by c2
    ) ty
on tx.c1 = ty.c2;
```

从下面的 Explain 执行计划中，我们可以观察到，7 号 Hash Join 节点的左侧子节点是 6 号聚合节点，而右侧子节点是 4 号 Exchange 节点。这表示左侧子节点聚合后的数据位置保持不变，而右侧子节点的数据则会根据 Bucket Shuffle 的方式被分发到左侧子节点所在的节点上，以便进行后续的 Hash Join 操作。

```
+------------------------------------------------------------+
| Explain String(Nereids Planner)                            |
+------------------------------------------------------------+
| PLAN FRAGMENT 0                                            |
|   OUTPUT EXPRS:                                            |
|     c1[#18]                                                |
|     c2[#19]                                                |
|     c2[#20]                                                |
|     c1[#21]                                                |
|   PARTITION: HASH_PARTITIONED: c1[#8]                      |
|                                                            |
|   HAS_COLO_PLAN_NODE: true                                 |
|                                                            |
|   VRESULT SINK                                             |
|      MYSQL_PROTOCAL                                        |
|                                                            |
|   7:VHASH JOIN(364)                                        |
|   |  join op: INNER JOIN(BUCKET_SHUFFLE)[]                 |
|   |  equal join conjunct: (c1[#12] = c2[#6])               |
|   |  cardinality=10                                        |
|   |  vec output tuple id: 8                                |
|   |  output tuple id: 8                                    |
|   |  vIntermediate tuple ids: 7                            |
|   |  hash output slot ids: 6 7 12 13                       |
|   |  final projections: c1[#14], c2[#15], c2[#16], c1[#17] |
|   |  final project output tuple id: 8                      |
|   |  distribute expr lists: c1[#12]                        |
|   |  distribute expr lists: c2[#6]                         |
```

```
|   |                                                    |
|   |----4:VEXCHANGE                                     |
|   |       offset: 0                                    |
|   |       distribute expr lists: c2[#6]                |
|   |                                                    |
|   6:VAGGREGATE (update finalize)(342)                  |
|   |  output: sum(c2[#9])[#11]                          |
|   |  group by: c1[#8]                                  |
|   |  sortByGroupKey:false                              |
|   |  cardinality=10                                    |
|   |  final projections: c1[#10], c2[#11]               |
|   |  final project output tuple id: 6                  |
|   |  distribute expr lists: c1[#8]                     |
|   |                                                    |
|   5:VOlapScanNode(339)                                 |
|      TABLE: tt.t1(t1), PREAGGREGATION: ON              |
|      partitions=1/1 (t1)                               |
|      tablets=1/1, tabletList=491188                    |
|      cardinality=21, avgRowSize=0.0, numNodes=1        |
|      pushAggOp=NONE                                    |
|                                                        |
| PLAN FRAGMENT 1                                        |
|                                                        |
|   PARTITION: HASH_PARTITIONED: c2[#2]                  |
|                                                        |
|   HAS_COLO_PLAN_NODE: true                             |
|                                                        |
|   STREAM DATA SINK                                     |
|     EXCHANGE ID: 04                                    |
|     BUCKET_SHFFULE_HASH_PARTITIONED: c2[#6]            |
|                                                        |
|   3:VAGGREGATE (merge finalize)(355)                   |
|   |  output: sum(partial_sum(c1)[#3])[#5]              |
|   |  group by: c2[#2]                                  |
|   |  sortByGroupKey:false                              |
|   |  cardinality=5                                     |
|   |  final projections: c2[#4], c1[#5]                 |
|   |  final project output tuple id: 3                  |
|   |  distribute expr lists: c2[#2]                     |
|   |                                                    |
|   2:VEXCHANGE                                          |
|      offset: 0                                         |
|      distribute expr lists:                            |
|                                                        |
| PLAN FRAGMENT 2                                        |
```

```
|                                                                 |
|   PARTITION: HASH_PARTITIONED: c1[#0]                           |
|                                                                 |
|   HAS_COLO_PLAN_NODE: false                                     |
|                                                                 |
|   STREAM DATA SINK                                              |
|     EXCHANGE ID: 02                                             |
|     HASH_PARTITIONED: c2[#2]                                    |
|                                                                 |
|   1:VAGGREGATE (update serialize)(349)                          |
|   |  STREAMING                                                  |
|   |  output: partial_sum(c1[#0])[#3]                            |
|   |  group by: c2[#1]                                           |
|   |  sortByGroupKey:false                                       |
|   |  cardinality=5                                              |
|   |  distribute expr lists: c1[#0]                              |
|   |                                                             |
|   0:VOlapScanNode(346)                                          |
|      TABLE: tt.t2(t2), PREAGGREGATION: ON                       |
|      partitions=1/1 (t2)                                        |
|      tablets=1/1, tabletList=491198                             |
|      cardinality=10, avgRowSize=0.0, numNodes=1                 |
|      pushAggOp=NONE                                             |
|                                                                 |
|                                                                 |
| Statistics                                                      |
|  planed with unknown column statistics                          |
+-----------------------------------------------------------------+
97 rows in set (0.01 sec)
```

2.12.2.5.2 Colocate Join 示例

在下面的例子中，t1 和 t2 表都通过 GROUP BY 算子进行了处理，并输出了新的表（此时 tx 和 ty 均按照 c2 进行了 Hash 分布）。随后的 JOIN 条件是 tx.c2 = ty.c2，这恰好满足了 Colocate Join 的条件。

```
explain select *
from
    (
        -- t1 表按照 c1 做了 hash 分布，经过 group by 算子后，数据分布变成按照 c2 进行的 hash
            ↪ 分布
        select c2 as c2, sum(c1) as c1
        from t1
        group by c2
    ) tx
join
```

```
    (
        -- t2 表按照 c1 做了 hash 分布，经过 group by 算子后，数据分布变成按照 c2 进行的 hash
            ↪ 分布
        select c2 as c2, sum(c1) as c1
        from t2
        group by c2
    ) ty
on tx.c2 = ty.c2;
```

从下面的 Explain 执行计划结果中可以看出，8 号 Hash Join 节点的左侧子节点是 7 号聚合节点，右侧子节点是 3 号聚合节点，并且没有出现 Exchange 节点。这表明左侧和右侧子节点聚合后的数据都保持在其原始位置不动，无需进行数据移动，可以直接在本地进行后续的 Hash Join 操作。

```
+------------------------------------------------------------+
| Explain String(Nereids Planner)                            |
+------------------------------------------------------------+
| PLAN FRAGMENT 0                                            |
|   OUTPUT EXPRS:                                            |
|     c2[#20]                                                |
|     c1[#21]                                                |
|     c2[#22]                                                |
|     c1[#23]                                                |
|   PARTITION: HASH_PARTITIONED: c2[#10]                     |
|                                                            |
|   HAS_COLO_PLAN_NODE: true                                 |
|                                                            |
|   VRESULT SINK                                             |
|      MYSQL_PROTOCAL                                        |
|                                                            |
|   8:VHASH JOIN(373)                                        |
|   |  join op: INNER JOIN(PARTITIONED)[]                    |
|   |  equal join conjunct: (c2[#14] = c2[#6])               |
|   |  cardinality=10                                        |
|   |  vec output tuple id: 9                                |
|   |  output tuple id: 9                                    |
|   |  vIntermediate tuple ids: 8                            |
|   |  hash output slot ids: 6 7 14 15                       |
|   |  final projections: c2[#16], c1[#17], c2[#18], c1[#19] |
|   |  final project output tuple id: 9                      |
|   |  distribute expr lists: c2[#14]                        |
|   |  distribute expr lists: c2[#6]                         |
|   |                                                        |
|   |----3:VAGGREGATE (merge finalize)(367)                  |
|   |    |  output: sum(partial_sum(c1)[#3])[#5]             |
|   |    |  group by: c2[#2]                                 |
|   |    |  sortByGroupKey:false                             |
```

```
|   |    |  cardinality=5                                  |
|   |    |  final projections: c2[#4], c1[#5]              |
|   |    |  final project output tuple id: 3               |
|   |    |  distribute expr lists: c2[#2]                  |
|   |    |                                                 |
|   |    2:VEXCHANGE                                       |
|   |       offset: 0                                      |
|   |       distribute expr lists:                         |
|   |                                                      |
|   7:VAGGREGATE (merge finalize)(354)                     |
|   |  output: sum(partial_sum(c1)[#11])[#13]              |
|   |  group by: c2[#10]                                   |
|   |  sortByGroupKey:false                                |
|   |  cardinality=10                                      |
|   |  final projections: c2[#12], c1[#13]                 |
|   |  final project output tuple id: 7                    |
|   |  distribute expr lists: c2[#10]                      |
|   |                                                      |
|   6:VEXCHANGE                                            |
|      offset: 0                                           |
|      distribute expr lists:                              |
|                                                          |
| PLAN FRAGMENT 1                                          |
|                                                          |
|   PARTITION: HASH_PARTITIONED: c1[#8]                    |
|                                                          |
|   HAS_COLO_PLAN_NODE: false                              |
|                                                          |
|   STREAM DATA SINK                                       |
|     EXCHANGE ID: 06                                      |
|     HASH_PARTITIONED: c2[#10]                            |
|                                                          |
|   5:VAGGREGATE (update serialize)(348)                   |
|   |  STREAMING                                           |
|   |  output: partial_sum(c1[#8])[#11]                    |
|   |  group by: c2[#9]                                    |
|   |  sortByGroupKey:false                                |
|   |  cardinality=10                                      |
|   |  distribute expr lists: c1[#8]                       |
|   |                                                      |
|   4:VOlapScanNode(345)                                   |
|      TABLE: tt.t1(t1), PREAGGREGATION: ON                |
|      partitions=1/1 (t1)                                 |
|      tablets=1/1, tabletList=491188                      |
|      cardinality=21, avgRowSize=0.0, numNodes=1          |
```

```
|      pushAggOp=NONE                                                |
|                                                                    |
| PLAN FRAGMENT 2                                                    |
|                                                                    |
|   PARTITION: HASH_PARTITIONED: c1[#0]                              |
|                                                                    |
|   HAS_COLO_PLAN_NODE: false                                        |
|                                                                    |
|   STREAM DATA SINK                                                 |
|     EXCHANGE ID: 02                                                |
|     HASH_PARTITIONED: c2[#2]                                       |
|                                                                    |
|   1:VAGGREGATE (update serialize)(361)                             |
|   |  STREAMING                                                     |
|   |  output: partial_sum(c1[#0])[#3]                               |
|   |  group by: c2[#1]                                              |
|   |  sortByGroupKey:false                                          |
|   |  cardinality=5                                                 |
|   |  distribute expr lists: c1[#0]                                 |
|   |                                                                |
|   0:VOlapScanNode(358)                                             |
|      TABLE: tt.t2(t2), PREAGGREGATION: ON                          |
|      partitions=1/1 (t2)                                           |
|      tablets=1/1, tabletList=491198                                |
|      cardinality=10, avgRowSize=0.0, numNodes=1                    |
|      pushAggOp=NONE                                                |
|                                                                    |
|                                                                    |
| Statistics                                                         |
|  planed with unknown column statistics                             |
+--------------------------------------------------------------------+
105 rows in set (0.06 sec)
```

2.12.2.6 四种 Shuffle 方式对比

Shuffle 方式	网络开销	物理算子	适用场景
Broadcast	N * T(R)	Hash Join /Nest Loop Join	通用
Shuffle	T(S) + T(R)	Hash Join	通用
Bucket Shuffle	T(R)	Hash Join	JOIN 条件含左表分桶列，左表单分区
Colocate	0	Hash Join	JOIN 条件含左表分桶列，且两表属同一 Colocate Group

备注

N：参与 Join 计算的 Instance 个数

T(关系) : 关系的 Tuple 数目

上述四种 Shuffle 方式的灵活性依次递减，它们对数据分布的要求也愈发严格。在多数场景下，随着对数据分布要求的提高，Join 计算的性能往往也会逐步提升。值得注意的是，如果表的 Bucket 数量较少，Bucket Shuffle 或 Colocate Join 可能会因为并行度较低而导致性能下降，甚至可能慢于 Shuffle Join。这是因为 Shuffle 操作能更有效地均衡数据的分布，从而在后续处理中提供更高的并行度。

2.12.2.7 常见问题

Bucket Shuffle Join 和 Colocate Join 在应用时对数据分布和 JOIN 条件具有一定限制条件。下面，我们将详细阐述这两种 JOIN 方式各自的具体限制。

2.12.2.7.1 Bucket Shuffle Join 的限制

在直接扫描两张物理表以进行 Bucket Shuffle Join 时，需要满足以下条件：

1. 等值 Join 条件：Bucket Shuffle Join 仅适用于 Join 条件为等值的场景，因为它依赖于 Hash 计算来确定数据分布。
2. 包含分桶列的等值条件：等值 Join 条件中须包含两张表的分桶列，当左表的分桶列作为等值 Join 条件时，更有可能被规划为 Bucket Shuffle Join。
3. 表类型限制：Bucket Shuffle Join 仅适用于 Doris 原生的 OLAP 表。对于 ODBC、 MySQL、 ES 等外部表，当它们作为左表时，Bucket Shuffle Join 无法生效。
4. 单分区要求：对于分区表，由于每个分区的数据分布可能不同，Bucket Shuffle Join 仅在左表为单分区时保证有效。因此在执行 SQL 时，应尽可能使用 WHERE 条件来启用分区裁剪策略。

2.12.2.7.2 Colocate Join 的限制

在直接扫描两张物理表时，Colocate Join 相较于 Bucket Shuffle Join 具有更严格的限制条件，除了满足 Bucket Shuffle Join 的所有条件外，还需满足以下要求：

1. 分桶列的类型和分桶数量必须一致，以确保数据分布的一致性。
2. 需要显式指定 Colocation Group，只有处于相同 Colocation Group 的表才能进行 Colocate Join。
3. 在进行副本修复或副本均衡等操作时，Colocation Group 可能处于 Unstable 状态，此时 Colocate Join 将退化为普通的 Join 操作。

2.12.3 子查询

子查询（Subquery）是嵌套在另一个查询（通常是 SELECT 语句）中的 SQL 查询。它可以用在 SELECT、FROM、WHERE 或 HAVING 子句中，为外部查询提供数据或条件。子查询的使用使得 SQL 查询变得更加灵活和强大，因为它们允许我们在单个查询中解决更复杂的问题。

子查询的一些重要特征如下：

1. 子查询的位置：子查询可以放在多个 SQL 子句中，如 SELECT、WHERE、HAVING 和 FROM 子句。它们可以与 SELECT、UPDATE、INSERT、DELETE 语句以及表达式运算符（如比较运算符 =、>、<、<=，以及 IN、EXISTS 等）一起使用。
2. 主查询与子查询的关系：子查询是嵌套在另一个查询内部的查询。外部查询被称为主查询，而内部查询则被称为子查询。
3. 执行顺序：当子查询是独立的（即不依赖于外部查询的结果）时，它通常首先执行。当存在相关性时，解析器会根据需要实时决定先执行哪个查询，并相应地使用子查询的输出。
4. 括号的使用：子查询必须用括号括起来，以区分它们是嵌套在另一个查询中。

下面我们分别用 t1 和 t2 表以及相关 SQL，介绍子查询的基本特性和用法。建表语句如下：

```
create table t1
(
    c1 bigint,
    c2 bigint
)
DISTRIBUTED BY HASH(c1) BUCKETS 3
PROPERTIES ("replication_num" = "1");

create table t2
(
    c1 bigint,
    c2 bigint
)
DISTRIBUTED BY HASH(c1) BUCKETS 3
PROPERTIES ("replication_num" = "1");
```

2.12.3.1 子查询的分类

2.12.3.1.1 按照子查询返回数据的特性分类

按照子查询返回数据的特性，可分为标量和非标量子查询：

1. 标量子查询

子查询一定返回一个单一的值（本质上等价于一个一行一列的 Relation）。如果子查询没有任何数据返回，则返回 NULL 值。标量子查询理论上可以出现在任何允许单值表达式出现的地方。

2. 非标量子查询

子查询返回一个 Relation（不同于标量子查询的返回值，该 Relation 可包含多行多列）。如果子查询没有任何数据返回，则返回空集（0 行）。非标量子查询理论上可以出现在任何允许关系（集合）出现的地方。

以下分别对标量和非标量子查询举例说明（对于两个括号内的子查询，当 t2 是空表时，两个子查询返回结果不同）。

```
-- 标量子查询，当 t2 是空表时，子查询返回标量值 null
select * from t1 where t1.c1 > (select sum(t2.c1) from t2);

-- 非标量子查询，当 t2 是空表时，子查询返回 empty set (0 rows)
select * from t1 where t1.c1 in (select t2.c1 from t2);
```

2.12.3.1.2 按照子查询是否引用了外部查询的列分类

按照子查询是否引用了外部查询的列，可分为关联子查询和非关联子查询

1. 非关联子查询

子查询没有引用外部查询的任何列。非关联子查询常常可以独立运算，并一次性返回相应结果供外部查询使用。

2. 关联子查询

子查询引用了主查询（又称为外部查询）的一个或多个列（引用的外部列常常在子查询的 WHERE 条件中）。关联子查询常常可以看做是对外部关联的表的一个过滤操作，因为对于外部表的每一行数据，都会对子查询进行运算，并返回相应结果。

以下分别对关联和非关联子查询举例说明：

```
-- 关联子查询，子查询内部使用了外部表的列 t1.c2
select * from t1 where t1.c1 in (select t2.c1 from t2 where t2.c2 = t1.c2);

-- 非关联子查询，子查询内部没有使用任何外部表 t1 的列
select * from t1 where t1.c1 in (select t2.c1 from t2);
```

2.12.3.2 Doris 支持的子查询

Doris 支持所有的非关联子查询，对关联子查询（有部分限制）的支持如下：

- 支持在 WHERE和 HAVING子句中的关联标量子查询。
- 支持在 WHERE和 HAVING子句中的关联的 IN、 NOT IN、 EXISTS、 NOT EXISTS 非标量子查询。
- 支持在SELECT列表中的关联标量子查询。
- 对于嵌套子查询，Doris 只支持子查询关联到自己的直接父查询，不支持跨层级关联到父查询的更外层查询。

2.12.3.3 关联子查询的限制

2.12.3.3.1 关联的标量子查询的限制

- 关联条件必须是等值条件。
- 子查询的输出必须是单个聚合函数的结果，且没有 group by 子句。

```
-- 单个聚合函数，且无 group by，支持
select * from t1 where t1.c1 < (select max(t2.c1) from t2 where t1.c2 = t2.c2);

-- 等价改写的 SQL 如下：
select t1.* from t1 inner join (select t2.c2 as c2, max(t2.c1) as c1 from t2 group by t2.c2) tx
    ↪  on t1.c1 < tx.c1 and t1.c2 = tx.c2;

-- 非等值条件，不支持
select * from t1 where t1.c1 = (select max(t2.c1) from t2 where t1.c2 > t2.c2);

-- 没有聚合函数，不支持
select * from t1 where t1.c1 = (select t2.c1 from t2 where t1.c2 = t2.c2);

-- 有聚合函数，但包含 group by，不支持
select * from t1 where t1.c1 = (select max(t2.c1) from t2 where t1.c2 = t2.c2 group by t2.c2);
```

2.12.3.3.2 关联的 (not) exists 子查询的限制

- 子查询不能同时有offset和 limit。

```
-- 带 limit 但无 offset，支持
select * from t1 where exists (select t2.c1 from t2 where t1.c2 = t2.c2 limit 2);

-- 等价改写 SQL 如下：
select * from t1 left semi join t2 on t1.c2 = t2.c2;

-- 带 offset 和 limit，不支持
select * from t1 where exists (select t2.c1 from t2 where t1.c2 = t2.c2 limit 2, 3);
```

2.12.3.3.3 关联的 (not) in 子查询的限制

- 子查询的输出必须是单个列。
- 子查询不能带有limit。
- 子查询不能带有聚合函数或group by子句。

```
-- 支持的子查询
select * from t1 where t1.c1 in (select t2.c1 from t2 where t1.c2 = t2.c2);

-- 改写的等价 SQL 如下:
select * from t1 left semi join t2 on t1.c1 = t2.c1 and t1.c2 = t2.c2;

-- 子查询输出为多列，不支持
select * from t1 where (t1.a, t1.c) in (select t2.c1, t2.c from t2 where t1.c2 = t2.c2);

-- 子查询带 limit，不支持
select * from t1 where t1.c1 in (select t2.c1 from t2 where t1.c2 = t2.c2 limit 3);

-- 带有 group by 子句，不支持
select * from t1 where t1.c1 in (select t2.c1 from t2 where t1.c2 = t2.c2 group by t2.c1);

-- 带有聚合函数，不支持
select * from t1 where t1.c1 in (select sum(t2.c1) from t2 where t1.c2 = t2.c2);
```

2.12.3.3.4 嵌套子查询的限制

目前只支持子查询关联到自己直接的父查询，不支持关联到父查询的更外层查询。

假设还有一个t3表，其建表语句如下：

```
create table t3
(
    c1 bigint,
    c2 bigint
)
DISTRIBUTED BY HASH(c1) BUCKETS 3
PROPERTIES ("replication_num" = "1");
```

- 可以支持当子查询只使用了自己直接父查询的列

```
select
    t1.c1
from
    t1
where not exists (
    select
        t2.c1
    from
        t2
    where not exists (
```

```
        select
            t3.c1
        from
            t3
        where
            t3.c2 = t2.c2
    ) and t2.c2 = t1.c2
);
```

- 不支持当最内层的子查询使用了直接父查询的列`t2.c2`，并使用了最外层查询的列`t1.c1`。

```
select
    t1.c1
from
    t1
where not exists (
    select
        t2.c1
    from
        t2
    where not exists (
        select
            t3.c1
        from
            t3
        where
            t3.c2 = t2.c2 and t3.c1 = t1.c1
    )
);
```

2.12.3.4 Mark Join

在 `where` 条件中，一些由 `(not)in` 或 `(not)exists` 的子查询和其他过滤条件组成的 `or` 关系子句，需要特殊处理才能生成正常结果。举例如下：

```
select
    t1.c1,
    t1.c2
from t1
where exists (
    select
        t2.c1
    from t2
    where
        t1.c2 = t2.c2
```

```
    ) or t1.c1 > 0;
```

这个 SQL 中的 exists 子句部分如果直接使用 left semi join，根据 left semi join 的语义，将会只输出 t1 中满足 t1.c2 = t2.c2 的行。然而，实际满足 t1.c1 > 0 这个条件的行也应该输出。为了达到这个目的，引入了 Mark Join 的机制。

> 备注 right semi join 类似，只是左右表不同。在这里，我们用 left semi join 作为示例。

示例 SQL 如下：

```
-- 此 SQL 不能实际执行，只作为演示使用
select
    tx.c1,
    tx.c2
from
    (
        select
            t1.c1,
            t1.c2,
            mark_join_flag
        from
            t1 left (mark) semi join t2 on t1.c2 = t2.c2
    ) tx
where
    tx.mark_join_flag or tx.c1 > 0;
```

Mark Join 相较于普通的 left semi join，区别在于普通的 left semi join 会直接输出左表满足条件的行，而 Mark Join 则输出原始的左表加上一个值为 true、false或 null 的标志位列（示例中的 mark_join_flag 标志）。标志位的值通过 join 条件表达式 t1.c2 = t2.c2 决定，每一行都对应一个标志位值。标志位值的计算参见下表：

t1.c2	t2.c2	mark_join_flag
1	1	TRUE
1	2	FALSE
1	NULL	NULL
NULL	1	NULL
NULL	NULL	NULL

有了这个标志位之后，where 过滤条件就可以改写为 where mark_join_flag or t1.c1 > 0，从而得到正确结果。

2.12.3.5 常见问题

由于标量子查询的输出必须是一个单值，如果子查询返回的数据量超过一条记录，将会报告运行时错误。

2.12.3.5.1 对于关联的标量子查询

在使用关联标量子查询时，如果满足关联条件的子查询返回的数据量超过一条记录，将会报告运行时错误。

请参考以下 SQL 示例：

```
-- 关联的标量子查询，如果 t2 表中满足 t1.c2 = t2.c2 的数据多于 1 条，则会报运行时错误
select t1.*, (select t2.c1 from t2 where t1.c2 = t2.c2) from t1;

-- 报错信息样例如下
ERROR 1105 (HY000): errCode = 2, detailMessage = (127.0.0.1)[INVALID_ARGUMENT][E33] correlate
    ↪ scalar subquery must return only 1 row
```

2.12.3.5.2 对于非关联的标量子查询

Doris 会在运行时添加一个assert num rows算子，如果子查询返回的数据量超过一条记录，将会报告运行时错误。

请参考以下 SQL 示例：

```
-- 非关联的标量子查询，如果 t2 表有多于 1 条的数据，则可能报运行时错误
select t1.*, (select t2.c1 from t2) from t1;

-- 报错信息样例如下
ERROR 1105 (HY000): errCode = 2, detailMessage = (127.0.0.1)[CANCELLED]Expected EQ 1 to be
    ↪ returned by expression
```

2.12.4 聚合多维分析

在数据库中，ROLLUP、CUBE 和 GROUPING SETS 是用于多维数据聚合的高级 SQL 语句。这些功能显著增强了 GROUP BY 子句的能力，使得用户可以在单一查询中获得多种层次的汇总结果，这在语义上等价于使用 UNION ALL 连接多个聚合语句。

- ROLLUP：ROLLUP 是一种用于生成层次化汇总的操作。它按照指定的列顺序进行汇总，从最细粒度的数据逐步汇总到最高层次。例如，在销售数据中，可以使用 ROLLUP 按地区、时间进行汇总，得到每个地区每个月的销售额、每个地区的总销售额以及整体总销售额。ROLLUP 适用于需要逐级汇总的场景。
- CUBE：CUBE 是一种更为强大的聚合操作，它生成所有可能的汇总组合。与 ROLLUP 不同，CUBE 会计算所有维度的子集。例如，对于按产品和地区进行统计的销售数据，CUBE 会计算每个产品在每个地区的销售额、每个产品的总销售额、每个地区的总销售额以及整体总销售额。CUBE 适用于需要全面多维分析的场景，如业务分析和市场调查。
- GROUPING SETS：GROUPING SETS 提供了对特定分组集进行聚合的灵活性。它允许用户指定一组列的组合进行独立聚合，而不是像 ROLLUP 和 CUBE 那样生成所有可能的组合。例如，可以定义按地区和时间的特定组合进行汇总，而不需要每个维度的所有组合。GROUPING SETS 适用于需要定制化汇总的场景，提供了灵活的聚合控制。

ROLLUP、CUBE 和 GROUPING SETS 提供了强大的多维数据汇总功能，适用于各种数据分析和报告需求，使得复杂的聚合计算变得更加简便和高效。接下来将详细介绍以上功能使用场景、语法与示例。

2.12.4.1 ROLLUP

2.12.4.1.1 使用场景

ROLLUP 对于按照时间、地理、类别等层次维度进行汇总时非常有用。例如，查询可以指定 `ROLLUP(year, month, day)` 或者 `(country, Province, city)`。

2.12.4.1.2 语法和示例

ROLLUP 的语法如下：

```
SELECT ··· GROUP BY ROLLUP(grouping_column_reference_list)
```

下面这个查询对销售额按照年月进行汇总分析：

```
SELECT
        YEAR(d_date),
        MONTH(d_date),
        SUM(ss_net_paid) AS total_sum
FROM
        store_sales,
        date_dim d1
WHERE
        d1.d_date_sk = ss_sold_date_sk
        AND YEAR(d_date) IN (2001, 2002)
        AND MONTH(d_date) IN (1, 2, 3)
GROUP BY
        ROLLUP(YEAR(d_date), MONTH(d_date))
ORDER BY
        YEAR(d_date), MONTH(d_date);
```

这个查询按照时间进行汇总，分别计算了每年的销售额小计、每年中每月的销售额小计，以及总体的销售额总计。查询结果如下：

```
+--------------+---------------+-------------+
| YEAR(d_date) | MONTH(d_date) | total_sum   |
+--------------+---------------+-------------+
|         NULL |          NULL | 54262669.17 |
|         2001 |          NULL | 26640320.46 |
|         2001 |             1 |  9982165.83 |
|         2001 |             2 |  8454915.34 |
|         2001 |             3 |  8203239.29 |
|         2002 |          NULL | 27622348.71 |
|         2002 |             1 | 11260654.35 |
```

```
|         2002 |              2 |  7722750.61 |
|         2002 |              3 |  8638943.75 |
+--------------+----------------+-------------+
9 rows in set (0.08 sec)
```

2.12.4.2 CUBE

2.12.4.2.1 使用场景

CUBE 最适合用于查询涉及多个独立维度的列，而不是表示单个维度的不同级别的列。例如，常见的使用场景是对月份、地区和产品的所有组合进行汇总。这是三个独立的维度，分析所有可能的小计组合是很常见的。相比之下，显示年、月、日所有可能组合的交叉制表将包含几个不必要的值，因为时间维度中存在自然的层次结构。在大多数分析中，诸如按月日计算的利润之类的小计都是不必要的。相对较少的用户需要询问“全年每月 16 日的总销售额是多少”。

2.12.4.2.2 语法和示例

CUBE 的语法如下：

```
SELECT ··· GROUP BY CUBE(grouping_column_reference_list)
```

使用示例：

```
SELECT
        YEAR(d_date),
        i_category,
        ca_state,
        SUM(ss_net_paid) AS total_sum
FROM
        store_sales,
        date_dim d1,
        item,
        customer_address ca
WHERE
        d1.d_date_sk = ss_sold_date_sk
        AND i_item_sk = ss_item_sk
        AND ss_addr_sk = ca_address_sk
        AND i_category IN ("Books", "Electronics")
        AND YEAR(d_date) IN (1998, 1999)
        AND ca_state IN ("LA", "AK")
GROUP BY CUBE(YEAR(d_date), i_category, ca_state)
ORDER BY YEAR(d_date), i_category, ca_state;
```

查询结果如下，它分别计算了：

- 总计的销售额；

- 各年度的销售额小计、各类别下商品的销售额小计、各州的销售额小计；
- 每年每类产品的销售额小计、每个州每个产品的销售额小计、每年每个州的销售额小计和每年每个州各类别的产品的销售额小计。

```
+--------------+-------------+----------+------------+
| YEAR(d_date) | i_category  | ca_state | total_sum  |
+--------------+-------------+----------+------------+
|         NULL | NULL        | NULL     | 8690374.60 |
|         NULL | NULL        | AK       | 2675198.33 |
|         NULL | NULL        | LA       | 6015176.27 |
|         NULL | Books       | NULL     | 4238177.69 |
|         NULL | Books       | AK       | 1310791.36 |
|         NULL | Books       | LA       | 2927386.33 |
|         NULL | Electronics | NULL     | 4452196.91 |
|         NULL | Electronics | AK       | 1364406.97 |
|         NULL | Electronics | LA       | 3087789.94 |
|         1998 | NULL        | NULL     | 4369656.14 |
|         1998 | NULL        | AK       | 1402539.19 |
|         1998 | NULL        | LA       | 2967116.95 |
|         1998 | Books       | NULL     | 2213703.82 |
|         1998 | Books       | AK       |  719911.29 |
|         1998 | Books       | LA       | 1493792.53 |
|         1998 | Electronics | NULL     | 2155952.32 |
|         1998 | Electronics | AK       |  682627.90 |
|         1998 | Electronics | LA       | 1473324.42 |
|         1999 | NULL        | NULL     | 4320718.46 |
|         1999 | NULL        | AK       | 1272659.14 |
|         1999 | NULL        | LA       | 3048059.32 |
|         1999 | Books       | NULL     | 2024473.87 |
|         1999 | Books       | AK       |  590880.07 |
|         1999 | Books       | LA       | 1433593.80 |
|         1999 | Electronics | NULL     | 2296244.59 |
|         1999 | Electronics | AK       |  681779.07 |
|         1999 | Electronics | LA       | 1614465.52 |
+--------------+-------------+----------+------------+
27 rows in set (0.21 sec)
```

2.12.4.3 GROUPING FUNCTION

本节将介绍如何解决使用 ROLLUP 和 CUBE 时出现的两个挑战：

1. 如何以编程方式识别出哪些结果集行代表小计，以及如何准确找到给定小计对应的聚合层级。由于在计算（如总计百分比）时经常需要使用小计，因此，我们需要一种简便的方法来识别这些小计行。

2. 当查询结果同时包含实际存储的 NULL 值和由 ROLLUP 或 CUBE 操作生成的“NULL”值时，会引发另一个问题：如何区分这两种 NULL 值？

通过 GROUPING、 GROUPING_ID、 GROUPING SETS 能够有效解决上述的两个挑战。

2.12.4.3.1 GROUPING

1. 原理介绍

GROUPING 使用单个列作为参数，在遇到由 ROLLUP 或 CUBE 操作创建的 NULL 值时返回 1，即 NULL 表示该行是小计，则 GROUPING 返回 1。任何其他类型的值（包括表数据中本身的 NULL 值）都返回 0。

示例如下：

```
select
        year(d_date),
        month(d_date),
        sum(ss_net_paid) as total_sum,
        grouping(year(d_date)),
        grouping(month(d_date))
from
        store_sales,
        date_dim d1
where
        d1.d_date_sk = ss_sold_date_sk
        and year(d_date) in (2001, 2002)
        and month(d_date) in (1, 2, 3)
group by
        rollup(year(d_date), month(d_date))
order by
        year(d_date), month(d_date);
```

- (year(d_date), month(d_date)) 组的 GROUPING 函数结果为 (0,0) 为按照年月聚合的结果
- (year(d_date)) 组的 GROUPING 函数结果为 (0,1)，为按年聚合的结果；
- () 组的 GROUPING 函数结果为 (1,1)，为总计结果。

查询结果如下：

```
+--------------+---------------+-------------+-----------------------+-------------------------+
| year(d_date) | month(d_date) | total_sum   | Grouping(year(d_date)) | Grouping(month(d_date)) |
+--------------+---------------+-------------+-----------------------+-------------------------+
|         NULL |          NULL | 54262669.17 |                     1 |                       1 |
|         2001 |          NULL | 26640320.46 |                     0 |                       1 |
|         2001 |             1 |  9982165.83 |                     0 |                       0 |
|         2001 |             2 |  8454915.34 |                     0 |                       0 |
|         2001 |             3 |  8203239.29 |                     0 |                       0 |
```

```
|         2002 |          NULL | 27622348.71 |                    0 |                        1 |
|         2002 |             1 | 11260654.35 |                    0 |                        0 |
|         2002 |             2 |  7722750.61 |                    0 |                        0 |
|         2002 |             3 |  8638943.75 |                    0 |                        0 |
+--------------+---------------+-------------+----------------------+--------------------------+
9 rows in set (0.06 sec)
```

2. 使用场景、语法与示例

GROUPING 函数可以用来过滤结果。示例如下：

```
select
        year(d_date),
        i_category,
        ca_state,
        sum(ss_net_paid) as total_sum
from
        store_sales,
        date_dim d1,
        item,
        customer_address ca
where
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
        and ss_addr_sk=ca_address_sk
        and i_category in ("Books", "Electronics")
        and year(d_date) in(1998, 1999)
        and ca_state in ("LA", "AK")
group by cube(year(d_date), i_category, ca_state)
having grouping(year(d_date))=1 and grouping(i_category)=1 and grouping(ca_state)=1
or grouping(year(d_date))=0 and grouping(i_category)=1 and grouping(ca_state)=1
or grouping(year(d_date))=1 and grouping(i_category)=1 and grouping(ca_state)=0
order by year(d_date), i_category, ca_state;
```

在 HAVING 过滤条件中使用 GROUPING 函数，仅保留总计销售额，按年度汇总的销售额和按地区汇总的销售额。查询结果如下：

```
+---------------------+------------+----------+------------+
| year(`d1`.`d_date`) | i_category | ca_state | total_sum  |
+---------------------+------------+----------+------------+
|                NULL | NULL       | NULL     | 8690374.60 |
|                NULL | NULL       | AK       | 2675198.33 |
|                NULL | NULL       | LA       | 6015176.27 |
|                1998 | NULL       | NULL     | 4369656.14 |
|                1999 | NULL       | NULL     | 4320718.46 |
+---------------------+------------+----------+------------+
5 rows in set (0.13 sec)
```

你也可以使用 GROUPING 函数和 IF 函数提高查询的可读性，示例如下：

```
select
        if(grouping(year(d_date)) = 1, "Multi-year sum", year(d_date)) as year,
        if(grouping(i_category) = 1, "Multi-category sum", i_category) as category,
        sum(ss_net_paid) as total_sum
from
        store_sales,
        date_dim d1,
        item,
        customer_address ca
where
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
        and ss_addr_sk = ca_address_sk
        and i_category in ("Books", "Electronics")
        and year(d_date) in (1998, 1999)
        and ca_state in ("LA", "AK")
group by cube(year(d_date), i_category)
```

查询结果如下：

```
+----------------+--------------------+------------+
| year           | category           | total_sum  |
+----------------+--------------------+------------+
| 1998           | Books              | 2213703.82 |
| 1998           | Electronics        | 2155952.32 |
| 1999           | Electronics        | 2296244.59 |
| 1999           | Books              | 2024473.87 |
| 1998           | Multi-category sum | 4369656.14 |
| 1999           | Multi-category sum | 4320718.46 |
| Multi-year sum | Books              | 4238177.69 |
| Multi-year sum | Electronics        | 4452196.91 |
| Multi-year sum | Multi-category sum | 8690374.60 |
+----------------+--------------------+------------+
9 rows in set (0.09 sec)
```

2.12.4.3.2 GROUPING_ID

1. 使用场景

在数据库中，GROUPING_ID 和 GROUPING 函数都是用于处理多维数据聚合查询（如 ROLLUP 和 CUBE）时的辅助函数，它们帮助用户区分不同层级的聚合结果。如果你想确定某一行的聚合层级，你需要使用 GROUPING 函数对所有的 GROUP BY 列进行计算，因为单列的计算结果无法满足需求。

GROUPING_ID 函数比 GROUPING 更强大，因为它可以同时对多列进行检测。GROUPING_ID 函数接受多个列作为参数，并返回一个整数，该整数通过二进制位表示多个列的聚合状态。当使用表或物化视图保存计算结果时，

使用 GROUPING 函数表示聚合的不同层级会占用较多的存储空间，在这种场景下，使用 GROUPING_ID 更加合适。

以 CUBE(a, b) 为例，其 GROUPING_ID 可以表示为：

聚合层级	Bit Vector	GROUPING_ID	GROUPING(a)	GROUPING(b)
a,b	0 0	0	0	0
a	0 1	1	0	1
b	1 0	2	1	0
Grand Total	1 1	3	1	1

2. 语法和示例

示例 SQL 查询如下：

```
SELECT
    year(d_date),
    i_category,
    SUM(ss_net_paid) AS total_sum,
    GROUPING(year(d_date)),
    GROUPING(i_category),
    GROUPING_ID(year(d_date), i_category)
FROM
    store_sales,
    date_dim d1,
    item,
    customer_address ca
WHERE
    d1.d_date_sk = ss_sold_date_sk
    AND i_item_sk = ss_item_sk
    AND ss_addr_sk = ca_address_sk
    AND i_category IN ('Books', 'Electronics')
    AND year(d_date) IN (1998, 1999)
    AND ca_state IN ('LA', 'AK')
GROUP BY CUBE(year(d_date), i_category);
```

查询结果如下：

```
+--
    ↪ ------------+-------------+-----------+-----------------------+---------------------+-----------------
    ↪
| year(d_date) | i_category  | total_sum  | GROUPING(year(d_date)) | GROUPING(i_category) |
    ↪ GROUPING_ID(year(d_date), i_category) |
+--
    ↪ ------------+-------------+-----------+-----------------------+---------------------+-----------------
    ↪
| 1998         | Electronics | 2155952.32 | 0                     | 0                   | 0
    ↪                                      |
```

```
| 1998         | Books       | 2213703.82 | 0                    | 0                    | 0
    ↪                                    |
| 1999         | Electronics | 2296244.59 | 0                    | 0                    | 0
    ↪                                    |
| 1999         | Books       | 2024473.87 | 0                    | 0                    | 0
    ↪                                    |
| 1998         | NULL        | 4369656.14 | 0                    | 1                    | 1
    ↪                                    |
| 1999         | NULL        | 4320718.46 | 0                    | 1                    | 1
    ↪                                    |
| NULL         | Electronics | 4452196.91 | 1                    | 0                    | 2
    ↪                                    |
| NULL         | Books       | 4238177.69 | 1                    | 0                    | 2
    ↪                                    |
| NULL         | NULL        | 8690374.60 | 1                    | 1                    | 3
    ↪                                    |
+--
    ↪ ------------+-------------+------------+----------------------+----------------------+----------------
    ↪
9 rows in set (0.12 sec)
```

2.12.4.3.3 GROUPING SETS

1. 使用场景

当需要有选择地指定要创建的组集，可以在 GROUP BY 子句中使用 GROUPING SETS 表达式。通过这种方法，允许用户跨多个维度进行精确指定，而无需计算整个 CUBE。

由于 CUBE 查询通常消耗较多资源，当仅对少数几个维度感兴趣时，使用 GROUPING SETS 可以提升查询的执行效率。

2. 语法和示例

GROUPING SETS 的语法如下：

```
SELECT ··· GROUP BY GROUPING SETS(grouping_column_reference_list)
```

如果你需要：

- 每年度每类产品的销售额小计
- 每年度在每个州的销售额小计
- 每年度每个州每个产品的销售额小计

那么你可以使用 GROUPING SETS 来指定这些维度并进行汇总。以下是一个示例：

```
SELECT
    YEAR(d_date),
```

```
    i_category,
    ca_state,
    SUM(ss_net_paid) AS total_sum
FROM
    store_sales,
    date_dim d1,
    item,
    customer_address ca
WHERE
    d1.d_date_sk = ss_sold_date_sk
    AND i_item_sk = ss_item_sk
    AND ss_addr_sk = ca_address_sk
    AND i_category IN ('Books', 'Electronics')
    AND YEAR(d_date) IN (1998, 1999)
    AND ca_state IN ('LA', 'AK')
GROUP BY GROUPING SETS(
    (YEAR(d_date), i_category),
    (YEAR(d_date), ca_state),
    (YEAR(d_date), ca_state, i_category)
)
ORDER BY YEAR(d_date), i_category, ca_state;
```

查询结果：

```
+--------------+-------------+----------+------------+
| YEAR(d_date) | i_category  | ca_state | total_sum  |
+--------------+-------------+----------+------------+
| 1998         | NULL        | AK       | 1402539.19 |
| 1998         | NULL        | LA       | 2967116.95 |
| 1998         | Books       | NULL     | 2213703.82 |
| 1998         | Books       | AK       |  719911.29 |
| 1998         | Books       | LA       | 1493792.53 |
| 1998         | Electronics | NULL     | 2155952.32 |
| 1998         | Electronics | AK       |  682627.90 |
| 1998         | Electronics | LA       | 1473324.42 |
| 1999         | NULL        | AK       | 1272659.14 |
| 1999         | NULL        | LA       | 3048059.32 |
| 1999         | Books       | NULL     | 2024473.87 |
| 1999         | Books       | AK       |  590880.07 |
| 1999         | Books       | LA       | 1433593.80 |
| 1999         | Electronics | NULL     | 2296244.59 |
| 1999         | Electronics | AK       |  681779.07 |
| 1999         | Electronics | LA       | 1614465.52 |
+--------------+-------------+----------+------------+
16 rows in set (0.11 sec)
```

上面的写法等价于使用 CUBE，但通过 grouping_id 指定了具体的聚合组合，从而减少了不必要的计算：

```
SELECT
    SUM(ss_net_paid) AS total_sum,
    YEAR(d_date),
    i_category,
    ca_state
FROM
    store_sales,
    date_dim d1,
    item,
    customer_address ca
WHERE
    d1.d_date_sk = ss_sold_date_sk
    AND i_item_sk = ss_item_sk
    AND ss_addr_sk = ca_address_sk
    AND i_category IN ('Books', 'Electronics')
    AND YEAR(d_date) IN (1998, 1999)
    AND ca_state IN ('LA', 'AK')
GROUP BY CUBE(YEAR(d_date), ca_state, i_category)
HAVING grouping_id(YEAR(d_date), ca_state, i_category) = 0
    OR grouping_id(YEAR(d_date), ca_state, i_category) = 2
    OR grouping_id(YEAR(d_date), ca_state, i_category) = 1;
```

> 备注使用 CUBE 会计算所有可能的聚合层级（在这个例子中是八种），但实际上你可能只对其中的几种感兴趣。

3. 语义等价

- GROUPING SETS 与 GROUP BY UNION ALL

  GROUPING SETS 语句：

```
    SELECT k1, k2, SUM(k3) FROM t GROUP BY GROUPING SETS ((k1, k2), (k1), (k2), ());
```

其查询结果等价于使用 `UNION ALL` 连接的多个 `GROUP BY` 查询：

```
    SELECT k1, k2, SUM(k3) FROM t GROUP BY k1, k2
    UNION ALL
    SELECT k1, NULL, SUM(k3) FROM t GROUP BY k1
    UNION ALL
    SELECT NULL, k2, SUM(k3) FROM t GROUP BY k2
    UNION ALL
    SELECT NULL, NULL, SUM(k3) FROM t;
```

使用 `UNION ALL` 连接的查询较长，同时需要多次扫描基表，因此在书写和执行上的效率都较低。

- GROUPING SETS 与 ROLLUP

  ROLLUP 是对 GROUPING SETS 的扩展。例如：

```
SELECT a, b, c, SUM(d) FROM tab1 GROUP BY ROLLUP(a, b, c);
```

这个 `ROLLUP` 等价于下面的 `GROUPING SETS`：

```
GROUPING SETS (
    (a, b, c),
    (a, b),
    (a),
    ()
);
```

- GROUPING SETS 与 CUBE

  CUBE(a, b, c) 等价于下面的 GROUPING SETS：

```
GROUPING SETS (
    (a, b, c),
    (a, b),
    (a, c),
    (a),
    (b, c),
    (b),
    (c),
    ()
);
```

2.12.4.4 附录

建表语句和数据文件见分析函数 (窗口函数) 附录。

2.12.5 分析函数 (窗口函数)

分析函数，也称为窗口函数，是一种在 SQL 查询中对数据集中的行进行复杂计算的函数。窗口函数的特点在于，它们不会减少查询结果的行数，而是为每一行增加一个新的计算结果。窗口函数适用于多种分析场景，如计算滚动合计、排名以及移动平均等。

下面是一个使用窗口函数计算每个商店的前后三天的销售移动平均值的例子：

```
CREATE TABLE daily_sales
(store_id INT, sales_date DATE, sales_amount DECIMAL(10, 2))
PROPERTIES (
  "replication_num" = "1"
);
INSERT INTO daily_sales (store_id, sales_date, sales_amount) VALUES (1, '2023-01-01', 100.00),
    ↪ (1, '2023-01-02', 150.00), (1, '2023-01-03', 200.00), (1, '2023-01-04', 250.00), (1, '
    ↪ 2023-01-05', 300.00), (1, '2023-01-06', 350.00), (1, '2023-01-07', 400.00), (1, '
    ↪ 2023-01-08', 450.00), (1, '2023-01-09', 500.00), (2, '2023-01-01', 110.00), (2, '
    ↪ 2023-01-02', 160.00), (2, '2023-01-03', 210.00), (2, '2023-01-04', 260.00), (2, '
    ↪ 2023-01-05', 310.00), (2, '2023-01-06', 360.00), (2, '2023-01-07', 410.00), (2, '
    ↪ 2023-01-08', 460.00), (2, '2023-01-09', 510.00);

SELECT
        store_id,
        sales_date,
        sales_amount,
        AVG(sales_amount) OVER ( PARTITION BY store_id ORDER BY sales_date
        ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING ) AS moving_avg_sales
FROM
        daily_sales;
```

查询结果为如下：

```
+----------+------------+--------------+------------------+
| store_id | sales_date | sales_amount | moving_avg_sales |
+----------+------------+--------------+------------------+
|        1 | 2023-01-01 |       100.00 |         175.0000 |
|        1 | 2023-01-02 |       150.00 |         200.0000 |
|        1 | 2023-01-03 |       200.00 |         225.0000 |
|        1 | 2023-01-04 |       250.00 |         250.0000 |
|        1 | 2023-01-05 |       300.00 |         300.0000 |
|        1 | 2023-01-06 |       350.00 |         350.0000 |
|        1 | 2023-01-07 |       400.00 |         375.0000 |
|        1 | 2023-01-08 |       450.00 |         400.0000 |
|        1 | 2023-01-09 |       500.00 |         425.0000 |
|        2 | 2023-01-01 |       110.00 |         185.0000 |
|        2 | 2023-01-02 |       160.00 |         210.0000 |
|        2 | 2023-01-03 |       210.00 |         235.0000 |
|        2 | 2023-01-04 |       260.00 |         260.0000 |
|        2 | 2023-01-05 |       310.00 |         310.0000 |
|        2 | 2023-01-06 |       360.00 |         360.0000 |
|        2 | 2023-01-07 |       410.00 |         385.0000 |
|        2 | 2023-01-08 |       460.00 |         410.0000 |
|        2 | 2023-01-09 |       510.00 |         435.0000 |
```

```
+----------+------------+--------------+------------------+
18 rows in set (0.09 sec)
```

2.12.5.1 基本概念介绍

2.12.5.1.1 处理顺序

使用分析函数的查询处理可以分为三个阶段。

1. 执行所有的连接、WHERE、GROUP BY 和 HAVING 子句。
2. 将结果集提供给分析函数，并进行所有必要的计算。
3. 如果查询的末尾包含 ORDER BY 子句，则处理该子句以实现精确的输出排序。

查询的处理顺序如图所示：

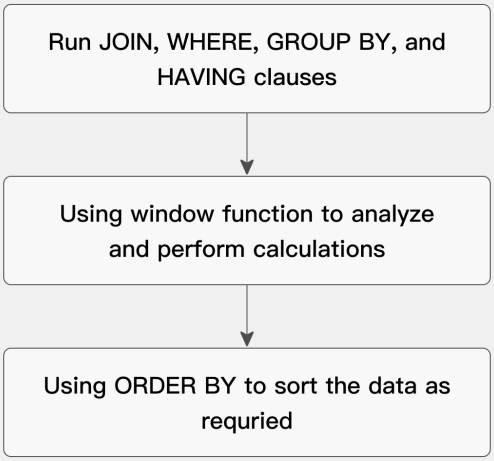


图 51: 基本概念介绍

2.12.5.1.2 结果集分区

分区是在使用 PARTITION BY 子句定义的组之后创建的。分析函数允许用户将查询结果集划分为称为分区的行组。

> 注意分析函数中使用的术语“分区”与表分区功能无关。在本章中，术语“分区”仅指与分析函数相关的含义。

2.12.5.1.3 窗口

对于分区中的每一行，你可以定义一个滑动数据窗口。此窗口确定了用于执行当前行计算所涉及的行范围。窗口具有一个起始行和一个结束行，根据其定义，窗口可以在一端或两端进行滑动。例如，为累积和函数定义的窗口，其起始行固定在其分区的第一行，而其结束行则从起点一直滑动到分区的最后一行。相反，为移动平均值定义的窗口，其起点和终点都会进行滑动。

窗口的大小可以设置为与分区中的所有行一样大，也可以设置为在分区内仅包含一行的滑动窗口。需要注意的是，当窗口靠近分区的边界时，由于边界的限制，计算的范围可能会缩减行数，此时函数仅返回可用行的计算结果。

在使用窗口函数时，当前行会被包含在计算之中。因此，在处理 n 个项目时，应指定为 (n-1)。例如，如果您需要计算五天的平均值，窗口应指定为“rows between 4 preceding and current row”，这也可以简写为“rows 4 preceding”。

2.12.5.1.4 当前行

使用分析函数执行的每个计算都是基于分区内的当前行。当前行作为确定窗口开始和结束的参考点，具体如图所示。

例如，可以使用一个窗口来定义中心移动平均值计算，该窗口包含当前行、当前行之前的 6 行以及当前行之后的 6 行。这样就创建了一个包含 13 行的滑动窗口。

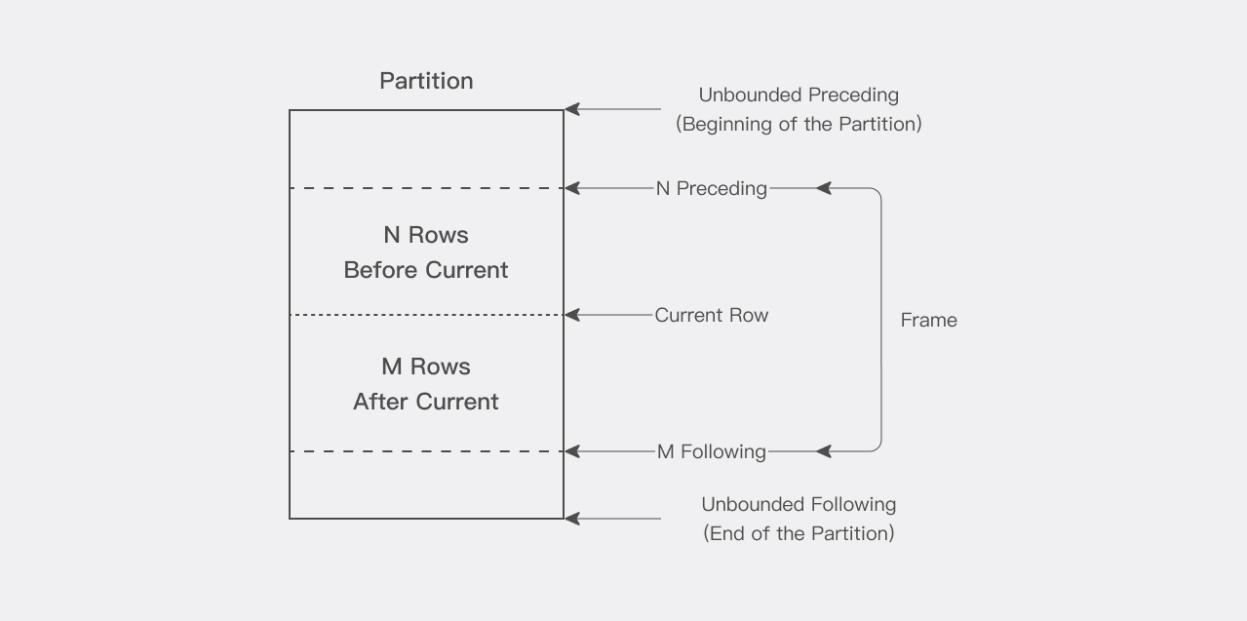


图 52: 当前行

2.12.5.2 排序函数

排序函数中，只有当指定的排序列是唯一值列时，查询结果才是确定的；如果排序列包含重复值，则每次的查询结果可能不同。

2.12.5.2.1 NTILE 函数

NTILE 是 SQL 中的一种窗口函数，用于将查询结果集分成指定数量的桶（组），并为每一行分配一个桶号。这在数据分析和报告中非常有用，特别是在需要对数据进行分组和排序时。

1. 函数语法

```
NTILE(num_buckets) OVER ([PARTITION BY partition_expression] ORDER BY order_expression)
```

- num_buckets：要将行划分成的桶的数量。
- PARTITION BY partition_expression（可选）：定义如何分区数据。
- ORDER BY order_expression：定义如何排序数据。

2. 使用 NTILE 函数

假设有一个包含学生考试成绩的表class_student_scores，希望将学生按成绩分成 4 个组，每组中的学生数量尽可能均匀。

首先，创建并插入数据到class_student_scores表中：

```
CREATE TABLE class_student_scores (
    class_id INT,
    student_id INT,
    student_name VARCHAR(50),
    score INT
)distributed by hash(student_id) properties('replication_num'=1);

INSERT INTO class_student_scores VALUES
(1, 1, 'Alice', 85),
(1, 2, 'Bob', 92),
(1, 3, 'Charlie', 87),
(2, 4, 'David', 78),
(2, 5, 'Eve', 95),
(2, 6, 'Frank', 80),
(2, 7, 'Grace', 90),
(2, 8, 'Hannah', 84);
```

然后，使用 NTILE 函数将学生按成绩分成 4 个组：

```
SELECT
    student_id,
    student_name,
    score,
    NTILE(4) OVER (ORDER BY score DESC) AS bucket
FROM
    class_student_scores;
```

结果如下：

```
+------------+--------------+-------+--------+
| student_id | student_name | score | bucket |
+------------+--------------+-------+--------+
|          5 | Eve          |    95 |      1 |
|          2 | Bob          |    92 |      1 |
|          7 | Grace        |    90 |      2 |
|          3 | Charlie      |    87 |      2 |
|          1 | Alice        |    85 |      3 |
|          8 | Hannah       |    84 |      3 |
|          6 | Frank        |    80 |      4 |
|          4 | David        |    78 |      4 |
+------------+--------------+-------+--------+
8 rows in set (0.12 sec)
```

在这个例子中，NTILE(4)函数根据成绩将学生分成了 4 个组（桶），每个组的学生数量尽可能均匀。

> 注意事项 - 如果不能均匀地将行分配到桶中，某些桶可能会多一行。
>
> - NTILE函数在每个分区内工作，如果使用PARTITION BY子句，则每个分区内的数据将分别进行桶分配。

3. 使用 NTILE 和 PARTITION BY

假设按班级对学生进行分组，然后在每个班级内将学生按成绩分成 3 个组，可以使用PARTITION BY和NTILE函数：

```
SELECT
    class_id,
    student_id,
    student_name,
    score,
    NTILE(3) OVER (PARTITION BY class_id ORDER BY score DESC) AS bucket
FROM
    class_student_scores;
```

结果如下：

```
+----------+------------+--------------+-------+--------+
| class_id | student_id | student_name | score | bucket |
+----------+------------+--------------+-------+--------+
|        1 |          2 | Bob          |    92 |      1 |
|        1 |          3 | Charlie      |    87 |      2 |
|        1 |          1 | Alice        |    85 |      3 |
|        2 |          5 | Eve          |    95 |      1 |
```

```
|        2 |          7 | Grace        |    90 |      1 |
|        2 |          8 | Hannah       |    84 |      2 |
|        2 |          6 | Frank        |    80 |      2 |
|        2 |          4 | David        |    78 |      3 |
+----------+------------+--------------+-------+--------+
8 rows in set (0.05 sec)
```

在这个例子中，学生按班级进行分区，然后在每个班级内按成绩分成 3 个组。每个组的学生数量尽可能均匀。

2.12.5.3 聚合函数

2.12.5.3.1 使用聚合函数 SUM 计算累计值

示例如下：

```
SELECT
        i_category,
        year(d_date),
        month(d_date),
        sum(ss_net_paid) as total_sales,
        sum(sum(ss_net_paid)) over (partition by i_category order by year(d_date),month(d_date)
            ↪ ROWS UNBOUNDED PRECEDING) as cum_sales
FROM
        store_sales,
        date_dim d1,
        item
WHERE
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
        and year(d_date) =2000
        and i_category in ('Books','Electronics')
GROUP BY
        i_category,
        year(d_date),
        month(d_date)
```

查询结果如下：

```
+-------------+--------------+---------------+-------------+-------------+
| i_category  | year(d_date) | month(d_date) | total_sales | cum_sales   |
+-------------+--------------+---------------+-------------+-------------+
| Books       |         2000 |             1 |  5348482.88 |  5348482.88 |
| Books       |         2000 |             2 |  4353162.03 |  9701644.91 |
| Books       |         2000 |             3 |  4466958.01 | 14168602.92 |
| Books       |         2000 |             4 |  4495802.19 | 18664405.11 |
| Books       |         2000 |             5 |  4589913.47 | 23254318.58 |
```

```
| Books       |         2000 |              6 |  4384384.00 | 27638702.58 |
| Books       |         2000 |              7 |  4488018.76 | 32126721.34 |
| Books       |         2000 |              8 |  9909227.94 | 42035949.28 |
| Books       |         2000 |              9 | 10366110.30 | 52402059.58 |
| Books       |         2000 |             10 | 10445320.76 | 62847380.34 |
| Books       |         2000 |             11 | 15246901.52 | 78094281.86 |
| Books       |         2000 |             12 | 15526630.11 | 93620911.97 |
| Electronics |         2000 |              1 |  5534568.17 |  5534568.17 |
| Electronics |         2000 |              2 |  4472655.10 | 10007223.27 |
| Electronics |         2000 |              3 |  4316942.60 | 14324165.87 |
| Electronics |         2000 |              4 |  4211523.06 | 18535688.93 |
| Electronics |         2000 |              5 |  4723661.00 | 23259349.93 |
| Electronics |         2000 |              6 |  4127773.06 | 27387122.99 |
| Electronics |         2000 |              7 |  4286523.05 | 31673646.04 |
| Electronics |         2000 |              8 | 10004890.96 | 41678537.00 |
| Electronics |         2000 |              9 | 10143665.77 | 51822202.77 |
| Electronics |         2000 |             10 | 10312020.35 | 62134223.12 |
| Electronics |         2000 |             11 | 14696000.54 | 76830223.66 |
| Electronics |         2000 |             12 | 15344441.52 | 92174665.18 |
+-------------+--------------+----------------+-------------+-------------+
24 rows in set (0.13 sec)
```

在此示例中，聚合函数 SUM 为每一行定义一个窗口，该窗口从分区的开头（UNBOUNDED PRECEDING）开始，默认在当前行结束。在此示例中，需要嵌套使用 SUM，因为需要对本身就是 SUM 的结果执行 SUM。嵌套聚合在分析聚合函数中高频使用。

2.12.5.3.2 使用聚合函数 AVG 计算移动平均值

示例如下：

```
SELECT
        i_category,
        year(d_date),
        month(d_date),
        sum(ss_net_paid) as total_sales,
        avg(sum(ss_net_paid)) over (order by year(d_date),month(d_date) ROWS 2 PRECEDING) as avg
FROM
        store_sales,
        date_dim d1,
        item
WHERE
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
        and year(d_date) =2000
        and i_category='Books'
GROUP BY
```

```
        i_category,
        year(d_date),
        month(d_date)
```

查询结果如下：

```
+------------+--------------+---------------+-------------+---------------+
| i_category | year(d_date) | month(d_date) | total_sales | avg           |
+------------+--------------+---------------+-------------+---------------+
| Books      |         2000 |             1 |  5348482.88 |  5348482.8800 |
| Books      |         2000 |             2 |  4353162.03 |  4850822.4550 |
| Books      |         2000 |             3 |  4466958.01 |  4722867.6400 |
| Books      |         2000 |             4 |  4495802.19 |  4438640.7433 |
| Books      |         2000 |             5 |  4589913.47 |  4517557.8900 |
| Books      |         2000 |             6 |  4384384.00 |  4490033.2200 |
| Books      |         2000 |             7 |  4488018.76 |  4487438.7433 |
| Books      |         2000 |             8 |  9909227.94 |  6260543.5666 |
| Books      |         2000 |             9 | 10366110.30 |  8254452.3333 |
| Books      |         2000 |            10 | 10445320.76 | 10240219.6666 |
| Books      |         2000 |            11 | 15246901.52 | 12019444.1933 |
| Books      |         2000 |            12 | 15526630.11 | 13739617.4633 |
+------------+--------------+---------------+-------------+---------------+
12 rows in set (0.13 sec)
```

> 注意输出数据中 AVG 列的前两行没有计算三天的移动平均值，因为边界数据前面没有足够的行数（在 SQL 中指定的行数为 3）。

同时，还可以计算以当前行为中心的窗口聚合函数。例如，此示例计算了 Books 类别的产品在 2000 年各月销售额的中心移动平均值，具体计算的是当前行前一个月、当前行、以及当前行后一个月的销售总额平均值。

```
SELECT
        i_category,
        year(d_date),
        month(d_date),
        sum(ss_net_paid) as total_sales,
        avg(sum(ss_net_paid)) over (order by year(d_date),month(d_date) ROWS between 1 PRECEDING
            ↪ and 1 following) as avg_sales
FROM
        store_sales,
        date_dim d1,
        item
WHERE
        d1.d_date_sk = ss_sold_date_sk
```

```
        and i_item_sk = ss_item_sk
        and year(d_date) =2000
        and i_category='Books'
GROUP BY
        i_category,
        year(d_date),
        month(d_date)
```

> 注意输出数据中起始行和结束行的中心移动平均值计算仅基于两天，因为边界数据前后没有足够的行数。

2.12.5.4 报告函数

报告函数是指每一行的窗口范围都是整个 partition。报告函数的主要优点是能够在单个查询块中多次传递数据，从而提高查询性能。例如，“对于每一年，找出其销售额最高的商品类别”之类的查询，使用报告函数则不需要进行 JOIN 操作。示例如下：

```
select year,category,total_sum from (
select
        year(d_date) as year,
        i_category as category,
        sum(ss_net_paid) as total_sum,
        max(sum(ss_net_paid)) over (partition by year(d_date)) as max_sales
from
        store_sales,
        date_dim d1,
        item
where
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
        and year(d_date) in(1998, 1999)
group by
        year(d_date), i_category
) t
where total_sum=max_sales;
```

报告MAX(SUM(ss_net_paid))的内层查询结果如下：

```
+------+-------------+-------------+-------------+
| year | category    | total_sum   | max_sales   |
+------+-------------+-------------+-------------+
| 1998 | Electronics | 91723676.27 | 91723676.27 |
| 1998 | Books       | 91307909.84 | 91723676.27 |
```

```
| 1999 | Electronics | 90310850.54 | 90310850.54 |
| 1999 | Books       | 88993351.11 | 90310850.54 |
+------+-------------+-------------+-------------+
4 rows in set (0.11 sec)
```

完整的查询结果如下：

```
+------+-------------+-------------+
| year | category    | total_sum   |
+------+-------------+-------------+
| 1998 | Electronics | 91723676.27 |
| 1999 | Electronics | 90310850.54 |
+------+-------------+-------------+
2 rows in set (0.12 sec)
```

你可以将报告聚合与嵌套查询结合使用，以解决一些复杂的问题，比如查找重要商品子类别中销量最好的产品。以“查找产品销售额占其产品类别总销售额 20% 以上的子类别，并从中选出其中销量最高的五种商品”为例，查询语句如下：

```
select i_category as categ, i_class as sub_categ, i_item_id
from
    (
    select
        i_item_id,i_class, i_category, sum(ss_net_paid) as sales,
        sum(sum(ss_net_paid)) over(partition by i_category) as cat_sales,
        sum(sum(ss_net_paid)) over(partition by i_class) as sub_cat_sales,
        rank() over (partition by i_class order by sum(ss_net_paid) desc) rank_in_line
    from
        store_sales,
        item
    where
        i_item_sk = ss_item_sk
    group by i_class, i_category, i_item_id) t
where sub_cat_sales>0.2*cat_sales and rank_in_line<=5;
```

2.12.5.5 LAG / LEAD 函数

LAG 和 LEAD 函数适用于值之间的比较。两个函数无需进行自连接，均可以同时访问表中的多个行，从而可以提高查询处理的速度。具体来说，LAG 函数能够提供对当前行之前给定偏移处的行的访问，而 LEAD 函数则提供对当前行之后给定偏移处的行的访问。

以下是一个使用 LAG 函数的 SQL 查询示例，该查询希望选取特定年份（1999, 2000, 2001, 2002）中，每个商品类别的总销售额、前一年的总销售额以及两者之间的差异：

```
select year, category, total_sales, before_year_sales, total_sales - before_year_sales from
(
select
```

```
        sum(ss_net_paid) as total_sales,
        year(d_date) year,
        i_category category,
        lag(sum(ss_net_paid), 1,0) over(PARTITION BY i_category ORDER BY YEAR(d_date)) AS before_
            ↪ year_sales
from
        store_sales,
        date_dim d1,
        item
where
        d1.d_date_sk = ss_sold_date_sk
        and i_item_sk = ss_item_sk
GROUP BY
        YEAR(d_date), i_category
) t
where year in (1999, 2000, 2001, 2002)
```

查询结果如下：

```
+------+-------------+-------------+-------------------+-----------------------------------+
| year | category    | total_sales | before_year_sales | (total_sales - before_year_sales) |
+------+-------------+-------------+-------------------+-----------------------------------+
| 1999 | Books       | 88993351.11 |       91307909.84 |                       -2314558.73 |
| 2000 | Books       | 93620911.97 |       88993351.11 |                        4627560.86 |
| 2001 | Books       | 90640097.99 |       93620911.97 |                       -2980813.98 |
| 2002 | Books       | 89585515.90 |       90640097.99 |                       -1054582.09 |
| 1999 | Electronics | 90310850.54 |       91723676.27 |                       -1412825.73 |
| 2000 | Electronics | 92174665.18 |       90310850.54 |                        1863814.64 |
| 2001 | Electronics | 92598527.85 |       92174665.18 |                         423862.67 |
| 2002 | Electronics | 94303831.84 |       92598527.85 |                        1705303.99 |
+------+-------------+-------------+-------------------+-----------------------------------+
8 rows in set (0.16 sec)
```

2.12.5.6 分析函数数据的唯一排序

1. 存在返回结果不一致的问题

当使用窗口函数的 ORDER BY 子句未能产生数据的唯一排序时，例如当 ORDER BY 表达式导致重复值时，行的顺序会变得不确定。这意味着在多次执行查询时，这些行的返回顺序可能会有所不同，进而导致窗口函数返回不一致的结果。

通过以下示例可以看出，该查询在多次运行时返回了不同的结果。出现不一致性的情况主要由于 ORDER BY ↪ dateid 没有为 SUM 窗口函数提供产生数据的唯一排序。

```
CREATE TABLE test_window_order
    (item_id int,
    date_time date,
```

```
    sales double)
distributed BY hash(item_id)
properties("replication_num" = 1);

INSERT INTO test_window_order VALUES
(1, '2024-07-01', 100),
(2, '2024-07-01', 100),
(3, '2024-07-01', 140);

SELECT
    item_id, date_time, sales,
    sum(sales) OVER (ORDER BY date_time ROWS BETWEEN
        UNBOUNDED PRECEDING AND CURRENT ROW) sum
FROM
    test_window_order;
```

由于排序列 date_time存在重复值，可能呈现以下两种查询结果：

```
+---------+------------+-------+------+
| item_id | date_time  | sales | sum  |
+---------+------------+-------+------+
|       1 | 2024-07-01 |   100 |  100 |
|       3 | 2024-07-01 |   140 |  240 |
|       2 | 2024-07-01 |   100 |  340 |
+---------+------------+-------+------+
3 rows in set (0.03 sec)
+---------+------------+-------+------+
| item_id | date_time  | sales | sum  |
+---------+------------+-------+------+
|       2 | 2024-07-01 |   100 |  100 |
|       1 | 2024-07-01 |   100 |  200 |
|       3 | 2024-07-01 |   140 |  340 |
+---------+------------+-------+------+
3 rows in set (0.02 sec)
```

2. 解决方法

为了解决这个问题，可以在 ORDER BY 子句中添加一个唯一值列，如 item_id，以确保排序的唯一性。

```
SELECT
        item_id,
        date_time,
        sales,
        sum(sales) OVER (
        ORDER BY item_id,
        date_time ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) sum
FROM
```

```
        test_window_order;
```

则查询结果固定为：

```
+---------+------------+-------+------+
| item_id | date_time  | sales | sum  |
+---------+------------+-------+------+
|       1 | 2024-07-01 |   100 |  100 |
|       2 | 2024-07-01 |   100 |  200 |
|       3 | 2024-07-01 |   140 |  340 |
+---------+------------+-------+------+
3 rows in set (0.03 sec)
```

了解更多有关分析函数信息，可以参考 Oracle 官网文档 SQL for Analysis and Reporting

2.12.5.7 附录

示例中使用到的表的建表语句如下：

```
CREATE DATABASE IF NOT EXISTS doc_tpcds;
USE doc_tpcds;

CREATE TABLE IF NOT EXISTS item (
    i_item_sk bigint not null,
    i_item_id char(16) not null,
    i_rec_start_date date,
    i_rec_end_date date,
    i_item_desc varchar(200),
    i_current_price decimal(7,2),
    i_wholesale_cost decimal(7,2),
    i_brand_id integer,
    i_brand char(50),
    i_class_id integer,
    i_class char(50),
    i_category_id integer,
    i_category char(50),
    i_manufact_id integer,
    i_manufact char(50),
    i_size char(20),
    i_formulation char(20),
    i_color char(20),
    i_units char(10),
    i_container char(10),
    i_manager_id integer,
    i_product_name char(50)
)
```

```
DUPLICATE KEY(i_item_sk)
DISTRIBUTED BY HASH(i_item_sk) BUCKETS 12
PROPERTIES (
  "replication_num" = "1"
);

CREATE TABLE IF NOT EXISTS store_sales (
    ss_item_sk bigint not null,
    ss_ticket_number bigint not null,
    ss_sold_date_sk bigint,
    ss_sold_time_sk bigint,
    ss_customer_sk bigint,
    ss_cdemo_sk bigint,
    ss_hdemo_sk bigint,
    ss_addr_sk bigint,
    ss_store_sk bigint,
    ss_promo_sk bigint,
    ss_quantity integer,
    ss_wholesale_cost decimal(7,2),
    ss_list_price decimal(7,2),
    ss_sales_price decimal(7,2),
    ss_ext_discount_amt decimal(7,2),
    ss_ext_sales_price decimal(7,2),
    ss_ext_wholesale_cost decimal(7,2),
    ss_ext_list_price decimal(7,2),
    ss_ext_tax decimal(7,2),
    ss_coupon_amt decimal(7,2),
    ss_net_paid decimal(7,2),
    ss_net_paid_inc_tax decimal(7,2),
    ss_net_profit decimal(7,2)
)
DUPLICATE KEY(ss_item_sk, ss_ticket_number)
DISTRIBUTED BY HASH(ss_item_sk, ss_ticket_number) BUCKETS 32
PROPERTIES (
  "replication_num" = "1"
);

CREATE TABLE IF NOT EXISTS date_dim (
    d_date_sk bigint not null,
    d_date_id char(16) not null,
    d_date date,
    d_month_seq integer,
    d_week_seq integer,
    d_quarter_seq integer,
    d_year integer,
```

```
    d_dow integer,
    d_moy integer,
    d_dom integer,
    d_qoy integer,
    d_fy_year integer,
    d_fy_quarter_seq integer,
    d_fy_week_seq integer,
    d_day_name char(9),
    d_quarter_name char(6),
    d_holiday char(1),
    d_weekend char(1),
    d_following_holiday char(1),
    d_first_dom integer,
    d_last_dom integer,
    d_same_day_ly integer,
    d_same_day_lq integer,
    d_current_day char(1),
    d_current_week char(1),
    d_current_month char(1),
    d_current_quarter char(1),
    d_current_year char(1)
)
DUPLICATE KEY(d_date_sk)
DISTRIBUTED BY HASH(d_date_sk) BUCKETS 12
PROPERTIES (
  "replication_num" = "1"
);

CREATE TABLE IF NOT EXISTS customer_address (
    ca_address_sk bigint not null,
    ca_address_id char(16) not null,
    ca_street_number char(10),
    ca_street_name varchar(60),
    ca_street_type char(15),
    ca_suite_number char(10),
    ca_city varchar(60),
    ca_county varchar(30),
    ca_state char(2),
    ca_zip char(10),
    ca_country varchar(20),
    ca_gmt_offset decimal(5,2),
    ca_location_type char(20)
)
DUPLICATE KEY(ca_address_sk)
DISTRIBUTED BY HASH(ca_address_sk) BUCKETS 12
```

```
PROPERTIES (
  "replication_num" = "1"
);
```

在终端执行如下命令，下载数据到本地，并使用 Stream Load 的方式加载数据：

```
curl -L https://cdn.selectdb.com/static/doc_ddl_dir_d27a752a7b.tar -o - | tar -Jxf -

curl --location-trusted \
-u "root:" \
-H "column_separator:|" \
-H "columns: i_item_sk, i_item_id, i_rec_start_date, i_rec_end_date, i_item_desc, i_current_price
    ↪ , i_wholesale_cost, i_brand_id, i_brand, i_class_id, i_class, i_category_id, i_category,
    ↪ i_manufact_id, i_manufact, i_size, i_formulation, i_color, i_units, i_container, i_
    ↪ manager_id, i_product_name" \
-T "doc_ddl_dir/item_1_10.dat" \
http://127.0.0.1:8030/api/doc_tpcds/item/_stream_load

curl --location-trusted \
-u "root:" \
-H "column_separator:|" \
-H "columns: d_date_sk, d_date_id, d_date, d_month_seq, d_week_seq, d_quarter_seq, d_year, d_dow,
    ↪  d_moy, d_dom, d_qoy, d_fy_year, d_fy_quarter_seq, d_fy_week_seq, d_day_name, d_quarter_
    ↪ name, d_holiday, d_weekend, d_following_holiday, d_first_dom, d_last_dom, d_same_day_ly,
    ↪ d_same_day_lq, d_current_day, d_current_week, d_current_month, d_current_quarter, d_
    ↪ current_year" \
-T "doc_ddl_dir/date_dim_1_10.dat" \
http://127.0.0.1:8030/api/doc_tpcds/date_dim/_stream_load

curl --location-trusted \
-u "root:" \
-H "column_separator:|" \
-H "columns: ss_sold_date_sk, ss_sold_time_sk, ss_item_sk, ss_customer_sk, ss_cdemo_sk, ss_hdemo_
    ↪ sk, ss_addr_sk, ss_store_sk, ss_promo_sk, ss_ticket_number, ss_quantity, ss_wholesale_
    ↪ cost, ss_list_price, ss_sales_price, ss_ext_discount_amt, ss_ext_sales_price, ss_ext_
    ↪ wholesale_cost, ss_ext_list_price, ss_ext_tax, ss_coupon_amt, ss_net_paid, ss_net_paid_
    ↪ inc_tax, ss_net_profit" \
-T "doc_ddl_dir/store_sales.csv" \
http://127.0.0.1:8030/api/doc_tpcds/store_sales/_stream_load

curl --location-trusted \
-u "root:" \
-H "column_separator:|" \
-H "ca_address_sk, ca_address_id, ca_street_number, ca_street_name, ca_street_type, ca_suite_
    ↪ number, ca_city, ca_county, ca_state, ca_zip, ca_country, ca_gmt_offset, ca_location_type
    ↪ " \
```

```
-T "doc_ddl_dir/customer_address_1_10.dat" \
http://127.0.0.1:8030/api/doc_tpcds/customer_address/_stream_load
```

数据文件item_1_10.dat，date_dim_1_10.dat，store_sales.csv，customer_address_1_10.dat可以点击链接下载。

2.12.6 公用表表达式（CTE）

2.12.6.1 描述

公用表表达式（Common Table Expression）定义一个临时结果集，你可以在 SQL 语句的范围内多次引用。CTE 主要用于 SELECT 语句中。

要指定公用表表达式，请使用 WITH 具有一个或多个逗号分隔子句的子句。每个子句都提供一个子查询，用于生成结果集，并将名称与子查询相关联。

Doris 支持嵌套 CTE。在包含该 WITH子句的语句中，可以引用每个 CTE 名称以访问相应的 CTE 结果集。CTE 名称可以在其他 CTE 中引用，从而可以基于其他 CTE 定义 CTE。

Doris 不支持递归 CTE。有关递归 CTE 的详细解释，可以参考 MySQL 递归 CTE 手册

2.12.6.2 示例

2.12.6.2.1 简单示例

下面的示例定义名为的 CTE cte1 和 cte2 中 WITH 子句，并且是指在它们的顶层 SELECT 下面的 WITH 子句：

```
WITH
  cte1 AS (SELECT a, b FROM table1),
  cte2 AS (SELECT c, d FROM table2)
SELECT b, d FROM cte1 JOIN cte2
WHERE cte1.a = cte2.c;
```

2.12.6.2.2 嵌套 CTE

```
WITH
  cte1 AS (SELECT a, b FROM table1),
  cte2 AS (SELECT c, d FROM cte1)
SELECT b, d FROM cte1 JOIN cte2
WHERE cte1.a = cte2.c;
```

2.12.6.2.3 递归 CTE（Doris 不支持）

“‘sql WITH r_cte AS (SELECT 1 AS user_id, 2 as manager_id UNION ALL SELECT user_id, manager_id FROM r_cte INNER JOIN (SELECT 1 AS user_id, 2 as manager_id) t ON r_cte.manager_id = t.user_id) SELECT * FROM r_cte

2.12.7 自定义函数

2.12.7.1 别名函数

2.12.7.1.1 概念介绍

别名函数，是指为函数起一个别名。通过在系统中为一个函数或表达式片段注册一个新的签名，可以达到提升兼容性或增加便利性的目的。

别名函数和其他自定义函数一样，支持两个作用域：LOCAL 和 GLOBAL。

- LOCAL：别名函数注册在当前数据库作用域下。如果需要在其他数据库下使用此别名函数，需要使用它的全限定名称，即 <所属数据库名>.<函数名>。
- GLOBAL：别名函数注册在全局作用域下。它可以在任意数据库下通过函数名直接访问。

2.12.7.1.2 使用场景

为函数起别名

该场景常见于系统迁移，当用户侧存在已有的、目标为其他数据库系统的查询时，可能在查询中存在一些与 Doris 中某个函数功能一致但名称不同的函数。这时，通过为这个函数定义一个新的别名函数，可以在用户侧无感的情况下完成迁移。

简化查询语句

该场景常见于复杂的分析，当书写复杂的查询语句时，可能在一个语句或不同语句中存在大量的重复性表达式片段。这时，通过为这一段复杂的表达式创建一个别名函数，可以简化查询语句，提升书写便利性和可维护性。

2.12.7.1.3 支持范围

表达式要求

当前，别名函数要求指向的真实表达式的根节点必须为函数表达式。

合法的例子：

```
-- 创建一个名为 func，参数为 INT, INT 的别名函数，实际指向的表达式为 abs(foo + bar);
CREATE ALIAS FUNCTION func(INT, INT) WITH PARAMETER(foo, bar) AS abs(foo + bar);

-- 创建一个名为 func，参数为 DATETIMEV2(3), INT 的别名函数，实际指向的表达式为 date_trunc(days_
    ↪ sub(foo, bar), 'day')
CREATE ALIAS FUNCTION func(DATETIMEV2(3), INT) WITH PARAMETER (foo, bar) as date_trunc(days_sub(
    ↪ foo, bar), 'day')
```

不合法的例子：

```
-- 根表达式不是函数
CREATE ALIAS FUNCTION func(INT, INT) WITH PARAMETER(foo, bar) AS foo + bar;
```

参数要求

当前别名函数不支持变长参数，且至少有一个参数。

2.12.7.2 Java UDF, UDAF, UDWF, UDTF

2.12.7.2.1 概述

Java UDF 为用户提供使用 Java 编写 UDF 的接口，以方便用户使用 Java 语言进行自定义函数的执行。Doris 支持使用 JAVA 编写 UDF、UDAF 和 UDTF。下文如无特殊说明，使用 UDF 统称所有用户自定义函数。1. Java UDF 是较为常见的自定义标量函数 (Scalar Function)，即每输入一行数据，就会有一行对应的结果输出，较为常见的有 ABS，LENGTH 等。值得一提的是对于用户来讲，Hive UDF 是可以直接迁移至 Doris 的。2. Java UDAF 即为自定义的聚合函数 (Aggregate Function)，即在输入多行数据进行聚合后，仅输出一行对应的结果，较为常见的有 MIN，MAX，COUNT 等。3. Java UDWF 即为自定义的窗口函数 (Window Function)，它为每行返回的结果是在一个窗口内 (一行或多行) 计算的值，较为常见的有 ROW_NUMBER，RANK，DENSE_RANK 等。4. JAVA UDTF 即为自定义的表函数 (Table Function)，即每输一行数据，可以产生一行或多行的结果，在 Doris 中需要结合 Lateral View 使用可以达到行转列的效果，较为常见的有 EXPLODE，EXPLODE_SPLIT 等。该功能自 Doris 3.0 版本起开始支持。

2.12.7.2.2 类型对应关系

Doris 数据类型	Java UDF 参数类型
Bool	Boolean
TinyInt	Byte
SmallInt	Short
Int	Integer
BigInt	Long
LargeInt	BigInteger
Float	Float
Double	Double
Date	LocalDate
Datetime	LocalDateTime
IPV4/IPV6	InetAddress
String	String
Decimal	BigDecimal
array<Type>	ArrayList<Type> List<Type> （支持嵌套）
map<Type1,Type2>	HashMap<Type1,Type2> Map<Type1,Type2> （支持嵌套）
struct<Type...>	ArrayList<Object> （从 3.0.0 版本开始支持） List<Type>

提示 array、map、struct 类型可以嵌套其它类型。例如，Doris 中的 array<array<int>> 对应 Java UDF 参数类型为 ArrayList<ArrayList<Integer>>，其他类型依此类推。List<Type> 和 Map<Type1,Type2> 类的支持从 3.1.0 版本开始

注意在创建函数时，请务必使用 `string` 类型而不是 `varchar`，否则可能会导致函数执行失败。

2.12.7.2.3 使用限制

1. 不支持复杂数据类型（HLL，Bitmap）。
2. 当前允许用户自己指定 JVM 最大堆大小，配置项是 be.conf 中的 JAVA_OPTS 的 -Xmx 部分。默认 1024m，如果需要聚合数据，建议调大一些，增加性能，减少内存溢出风险。
3. 由于 jvm 加载同名类的问题，不要同时使用多个同名类作为 udf 实现，如果想更新某个同名类的 udf，需要重启 be 重新加载 classpath。
4. 同名函数

   用户可以创建和内置函数签名完全相同的自定义函数。默认情况下，系统会优先匹配内置函数。但如果使用函数时，指定了 `database`（即 `db.function()`），则会被强制认为是用户自定义函数。

   在 3.0.7 版本中，新增了会话变量 `prefer_udf_over_builtin`。当设置为 `true` 时，会优先匹配用户自定义函数，以便于用户从其他系统迁移到 Doris 时，在不改变函数名称的情况下，通过自定义函数保持原有系统的函数行为。

2.12.7.2.4 快速上手

本节主要介绍如何开发 Java UDF。在 `samples/doris-demo/java-udf-demo/` 目录下提供了示例代码，供您参考。您也可以查看 demo。

UDF 的使用与普通的函数方式一致，唯一的区别在于，内置函数的作用域是全局的，而 UDF 的作用域是 DB 内部。所以如果当前链接 session 位于数据库 DB 内部时，直接使用 UDF 名字会在当前 DB 内部查找对应的 UDF。否则用户需要显示的指定 UDF 的数据库名字，例如 `dbName.funcName`。

接下来的章节介绍实例，均会在`test_table` 上做测试，对应建表如下：

```
CREATE TABLE `test_table` (
    id int NULL,
    d1 double NULL,
    str string NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_num" = "1");

insert into test_table values (1, 111.11, "a,b,c");
insert into test_table values (6, 666.66, "d,e");
```

Java-UDF 实例介绍

使用 Java 代码编写 UDF，UDF 的主入口必须为 evaluate 函数。这一点与 Hive 等其他引擎保持一致。在本示例中，我们编写了 AddOne UDF 来完成对整型输入进行加一的操作。

1. 首先编写对应的 Java 代码，打包生成 JAR 包。

```
public class AddOne extends UDF {
    public Integer evaluate(Integer value) {
        return value == null ? null : value + 1;
    }
}
```

2. 在 Doris 中注册创建 Java-UDF 函数。更多语法帮助可参阅CREATE FUNCTION.

```
CREATE FUNCTION java_udf_add_one(int) RETURNS int PROPERTIES (
    "file"="file:///path/to/java-udf-demo-jar-with-dependencies.jar",
    "symbol"="org.apache.doris.udf.AddOne",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

3. 用户使用 UDF 必须拥有对应数据库的 SELECT 权限。如果想查看注册成功的对应 UDF 函数，可以使用SHOW FUNCTIONS 命令。

```
select id,java_udf_add_one(id) from test_table;
+------+-----------------------+
| id   | java_udf_add_one(id) |
+------+-----------------------+
|    1 |                     2 |
|    6 |                     7 |
+------+-----------------------+
```

4. 当不再需要 UDF 函数时，可以通过下述命令来删除一个 UDF 函数，可以参考DROP FUNCTION

另外，如果定义的 UDF 中需要加载很大的资源文件，或者希望可以定义全局的 static 变量，可以参照文档下方的 static 变量加载方式。

Java-UDAF 实例介绍

在使用 Java 代码编写 UDAF 时，有一些必须实现的函数 (标记 required) 和一个内部类 State，下面将以具体的实例来说明。

1. 首先编写对应的 Java UDAF 代码，打包生成 JAR 包。

示例 1: SimpleDemo 将实现一个类似的 sum 的简单函数，输入参数 INT，输出参数是 INT

```
package org.apache.doris.udf;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.util.logging.Logger;

public class SimpleDemo  {

Logger log = Logger.getLogger("SimpleDemo");

//Need an inner class to store data
/*required*/
public static class State {
    /*some variables if you need */
    public int sum = 0;
}

/*required*/
public State create() {
    /* here could do some init work if needed */
    return new State();
}

/*required*/
public void destroy(State state) {
    /* here could do some destroy work if needed */
}

/*Not Required*/
public void reset(State state) {
    /*if you want this udaf function can work with window function.*/
    /*Must impl this, it will be reset to init state after calculate every window frame*/
    state.sum = 0;
}

/*required*/
//first argument is State, then other types your input
public void add(State state, Integer val) throws Exception {
    /* here doing update work when input data*/
    if (val != null) {
        state.sum += val;
    }
}
```

```
/*required*/
public void serialize(State state, DataOutputStream out) throws IOException {
    /* serialize some data into buffer */
    out.writeInt(state.sum);
}

/*required*/
public void deserialize(State state, DataInputStream in) throws IOException {
    /* deserialize get data from buffer before you put */
    int val = in.readInt();
    state.sum = val;
}

/*required*/
public void merge(State state, State rhs) throws Exception {
    /* merge data from state */
    state.sum += rhs.sum;
}

/*required*/
//return Type you defined
public Integer getValue(State state) throws Exception {
    /* return finally result */
    return state.sum;
}
}
```

示例 2: MedianUDAF 是一个计算中位数的功能，输入类型为 (DOUBLE, INT), 输出为 DOUBLE

```
package org.apache.doris.udf.demo;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.math.BigDecimal;
import java.util.Arrays;
import java.util.logging.Logger;

/*UDAF 计算中位数*/
public class MedianUDAF {
Logger log = Logger.getLogger("MedianUDAF");

//状态存储
public static class State {
    //返回结果的精度
    int scale = 0;
```

```
    //是否是某一个 tablet 下的某个聚合条件下的数据第一次执行 add 方法
    boolean isFirst = true;
    //数据存储
    public StringBuilder stringBuilder;
}

//状态初始化
public State create() {
    State state = new State();
    //根据每个 tablet 下的聚合条件需要聚合的数据量大小，预先初始化，增加性能
    state.stringBuilder = new StringBuilder(1000);
    return state;
}


//处理执行单位处理各自 tablet 下的各自聚合条件下的每个数据
public void add(State state, Double val, int scale) throws IOException {
    if (val != null && state.isFirst) {
        state.stringBuilder.append(scale).append(",").append(val).append(",");
        state.isFirst = false;
    } else if (val != null) {
        state.stringBuilder.append(val).append(",");
    }
}

//处理数据完需要输出等待聚合
public void serialize(State state, DataOutputStream out) throws IOException {
    //目前暂时只提供 DataOutputStream，如果需要序列化对象可以考虑拼接字符串，转换 json，
        ↪ 序列化成字节数组等方式
    //如果要序列化 State 对象，可能需要自己将 State 内部类实现序列化接口
    //最终都是要通过 DataOutputStream 传输
    out.writeUTF(state.stringBuilder.toString());
}

//获取处理数据执行单位输出的数据
public void deserialize(State state, DataInputStream in) throws IOException {
    String string = in.readUTF();
    state.scale = Integer.parseInt(String.valueOf(string.charAt(0)));
    StringBuilder stringBuilder = new StringBuilder(string.substring(2));
    state.stringBuilder = stringBuilder;
}

//聚合执行单位按照聚合条件合并某一个键下数据的处理结果 ,每个键第一次合并时，state1
    ↪ 参数是初始化的实例
public void merge(State state1, State state2) throws IOException {
```

```
    state1.scale = state2.scale;
    state1.stringBuilder.append(state2.stringBuilder.toString());
}

//对每个键合并后的数据进行并输出最终结果
public Double getValue(State state) throws IOException {
    String[] strings = state.stringBuilder.toString().split(",");
    double[] doubles = new double[strings.length + 1];
    doubles = Arrays.stream(strings).mapToDouble(Double::parseDouble).toArray();

    Arrays.sort(doubles);
    double n = doubles.length - 1;
    double index = n * 0.5;

    int low = (int) Math.floor(index);
    int high = (int) Math.ceil(index);

    double value = low == high ? (doubles[low] + doubles[high]) * 0.5 : doubles[high];

    BigDecimal decimal = new BigDecimal(value);
    return decimal.setScale(state.scale, BigDecimal.ROUND_HALF_UP).doubleValue();
}

//每个执行单位执行完都会执行
public void destroy(State state) {
}

}
```

2. 在 Doris 中注册创建 Java-UDAF 函数。更多语法帮助可参阅CREATE FUNCTION.

```
    CREATE AGGREGATE FUNCTION simple_demo(INT) RETURNS INT PROPERTIES (
        "file"="file:///pathTo/java-udaf.jar",
        "symbol"="org.apache.doris.udf.SimpleDemo",
        "always_nullable"="true",
        "type"="JAVA_UDF"
    );
```

3. 使用 Java-UDAF, 可以分组聚合或者聚合全部结果：

```
    select simple_demo(id) from test_table group by id;
    +-----------------+
    | simple_demo(id) |
    +-----------------+
```

```
|                1 |
|                6 |
+------------------+
```

```
select simple_demo(id) from test_table;
+------------------+
| simple_demo(id) |
+------------------+
|                7 |
+------------------+
```

Java-UDWF 实例介绍

1. 首先编写对应的 Java UDWF 代码，打包生成 JAR 包，它与 Java UDAF 的代码编写是一致的，仅需要实现额外 reset 的接口，将所有的 state 状态置为初始值:

```
void reset(State state)
```

2. 在 Doris 中注册创建 Java-UDWF 函数，与注册 Java-UDAF 一样。更多语法帮助可参阅CREATE FUNCTION.

```
CREATE AGGREGATE FUNCTION simple_demo_window(INT) RETURNS INT PROPERTIES (
    "file"="file:///pathTo/java-udaf.jar",
    "symbol"="org.apache.doris.udf.SimpleDemo",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

3. 使用 Java-UDWF, 可以查询在特定窗口内的计算结果，更多语法可以参考窗口函数:

```
select id, simple_demo_window(id) over(partition by id order by d1 rows between 1 preceding
    ↪ and 1 following) as res from test_table;
    +------+------+
    | id   | res  |
    +------+------+
    |    1 |    1 |
    |    6 |    6 |
    +------+------+
```

Java-UDTF 实例介绍

> UDTF 自 Doris 3.0 版本开始支持，

1. 首先编写对应的Java UDTF代码，打包生成JAR包。UDTF和UDF函数一样，需要用户自主实现一个evaluate方法，但是UDTF函数的返回值必须是Array类型。

```
public class UDTFStringTest {
    public ArrayList<String> evaluate(String value, String separator) {
        if (value == null || separator == null) {
            return null;
        } else {
            return new ArrayList<>(Arrays.asList(value.split(separator)));
        }
    }
}
```

2. 在Doris中注册创建Java-UDTF函数。此时会注册两个UTDF函数，另外一个是在函数名后面加上_outer后缀，其中带后缀_outer的是针对结果为0行时的特殊处理，具体可查看OUTER组合器。更多语法帮助可参阅CREATE FUNCTION.

```
CREATE TABLES FUNCTION java-utdf(string, string) RETURNS array<string> PROPERTIES (
    "file"="file:///pathTo/java-udtf.jar",
    "symbol"="org.apache.doris.udf.demo.UDTFStringTest",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

3. 使用Java-UDTF, 在Doris中使用UDTF需要结合Lateral View, 实现行转列的效果:

```
select id, str, e1 from test_table lateral view java_utdf(str,',') tmp as e1;
+------+-------+------+
| id   | str   | e1   |
+------+-------+------+
|    1 | a,b,c | a    |
|    1 | a,b,c | b    |
|    1 | a,b,c | c    |
|    6 | d,e   | d    |
|    6 | d,e   | e    |
+------+-------+------+
```

2.12.7.2.5 最佳实践

Static变量加载

当前在Doris中，执行一个UDF函数，例如`select udf(col)from table`, 每一个并发Instance会加载一次udf.jar包，在该Instance结束时卸载掉udf.jar包。

所以当 udf.jar 文件中需要加载一个几百 MB 的文件时，会因为并发的原因，使得占据的内存急剧增大，容易 OOM。或者想使用一个连接池时，这样无法做到仅在 static 区域初始化一次。

这里提供两个解决方案，其中方案二需要 Doris 版本在 branch-3.0 以上才行。

解决方案 1:

可以将资源加载代码拆分开，单独生成一个 JAR 包文件，然后其他包直接引用该资源 JAR 包。

假设已经将代码拆分为了 DictLibrary 和 FunctionUdf 两个文件。

```
public class DictLibrary {
    private static HashMap<String, String> res = new HashMap<>();

    static {
        // suppose we built this dictionary from a certain local file.
        res.put("key1", "value1");
        res.put("key2", "value2");
        res.put("key3", "value3");
        res.put("0", "value4");
        res.put("1", "value5");
        res.put("2", "value6");
    }

    public static String evaluate(String key) {
        if (key == null) {
            return null;
        }
        return res.get(key);
    }
}
```

```
public class FunctionUdf {
    public String evaluate(String key) {
        String value = DictLibrary.evaluate(key);
        return value;
    }
}
```

1. 单独编译 DictLibrary 文件，使其生成一个独立的 JAR 包，这样可以得到一个资源文件包 DictLibrary.jar:

```
    javac   ./DictLibrary.java
    jar -cf ./DictLibrary.jar ./DictLibrary.class
```

2. 编译 FunctionUdf 文件，需要引用上一步得到的资源包作为库使用，这样打包后可以得到 UDF 的 FunctionUdf.jar 包。

```
    javac -cp ./DictLibrary.jar  ./FunctionUdf.java
    jar  -cvf ./FunctionUdf.jar  ./FunctionUdf.class
```

3. 由于想让资源 JAR 包被所有的并发引用，所以想让它被 JVM 直接加载，可以将它放到指定路径 be/custom_lib 下面，BE 服务重启之后就可以随着 JVM 的启动加载进来，因此都会随着服务启动而加载，停止而释放。

4. 最后利用 CREATE FUNCTION 语句创建一个 UDF 函数，这样每次卸载仅是 FunctionUdf.jar。

```
   CREATE FUNCTION java_udf_dict(string) RETURNS string PROPERTIES (
    "file"="file:///pathTo/FunctionUdf.jar",
    "symbol"="org.apache.doris.udf.FunctionUdf",
    "always_nullable"="true",
    "type"="JAVA_UDF"
   );
```

解决方案 2:

BE 全局缓存 JAR 包，自定义过期淘汰时间，在 create function 时增加两个属性字段，其中 static_load: 用于定义是否使用静态 cache 加载的方式。

expiration_time: 用于定义 JAR 包的过期时间，单位为分钟。

若使用静态 cache 加载方式，则在第一次调用该 UDF 函数时，在初始化之后会将该 UDF 的实例缓存起来，在下次调用该 UDF 时，首先会在 cache 中进行查找，如果没有找到，则会进行相关初始化操作。

并且后台有线程定期检查，如果在配置的过期淘汰时间内，一直没有被调用过，则会从缓存 cache 中清理掉。如果被调用时，则会自动更新缓存时间点。

```
public class Print extends UDF {
    static Integer val = 0;
    public Integer evaluate() {
        val = val + 1;
        return val;
    }
}
```

```
CREATE FUNCTION print_12() RETURNS int
PROPERTIES (
    "file" = "file:///path/to/java-udf-demo-jar-with-dependencies.jar",
    "symbol" = "org.apache.doris.udf.Print",
    "always_nullable"="true",
    "type" = "JAVA_UDF",
    "static_load" = "true", // default value is false
    "expiration_time" = "60" // default value is 360 minutes
);
```

可以看到结果是一直在递增的，证明加载的 JAR 包没有被卸载后又加载，导致重新初始化变量为 0.

```
mysql [test_query_qa]>select print_12();
+------------+
| print_12() |
+------------+
|          1 |
+------------+
1 row in set (0.40 sec)

mysql [test_query_qa]>select print_12();
+------------+
| print_12() |
+------------+
|          2 |
+------------+
1 row in set (0.03 sec)

mysql [test_query_qa]>select print_12();
+------------+
| print_12() |
+------------+
|          3 |
+------------+
1 row in set (0.04 sec)
```

2.12.8 复杂类型查询

Doris 支持 Array，Map，Struct，JSON 等复杂类型。

Doris 提供了针对以上复杂类型的各类函数。

详细的函数支持，请查看 SQL 手册 / SQL 函数下的Array 函数、 Map 函数 、 Struct 函数 和JSON 函数。

2.12.9 列转行 (Lateral View)

与生成器函数（例如 EXPLODE）结合使用，LATERAL VIEW 可以生成一个包含一个或多个行的虚拟表，并将这些行应用于每个原始输出行。

2.12.9.1 语法

```
LATERAL VIEW generator_function ( expression [, ...] ) table_identifier AS column_identifier [,
    ↪ ...]
```

2.12.9.2 参数

- generator_function：生成器函数（如 EXPLODE、EXPLODE_SPLIT 等）。
- table_identifier：generator_function 的别名。
- column_identifier：列别名，用于输出行。列标识符的数量必须与生成器函数返回的列数匹配。

2.12.9.3 示例

假设有一个名为 person 的表，结构如下：

```
CREATE TABLE `person` (
  `id` int(11) NULL,
  `name` text NULL,
  `age` int(11) NULL,
  `class` int(11) NULL,
  `address` text NULL
) ENGINE=OLAP
UNIQUE KEY(`id`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"in_memory" = "false",
"storage_format" = "V2",
"disable_auto_compaction" = "false"
);

INSERT INTO person VALUES
    (100, 'John', 30, 1, 'Street 1'),
    (200, 'Mary', NULL, 1, 'Street 2'),
    (300, 'Mike', 80, 3, 'Street 3'),
    (400, 'Dan', 50, 4, 'Street 4');
```

使用 LATERAL VIEW 和 EXPLODE 函数查询 person 表：

```
SELECT * FROM person
LATERAL VIEW EXPLODE(ARRAY(30, 60)) tableName AS c_age;
```

查询结果将包含原始行的每个组合，以及 EXPLODE 函数生成的行：

```
+------+------+------+-------+----------+-------+
| id   | name | age  | class | address  | c_age |
+------+------+------+-------+----------+-------+
|  100 | John |   30 |     1 | Street 1 |    30 |
|  100 | John |   30 |     1 | Street 1 |    60 |
|  200 | Mary | NULL |     1 | Street 2 |    30 |
```

```
|  200 | Mary | NULL |     1 | Street 2 |    60 |
|  300 | Mike |   80 |     3 | Street 3 |    30 |
|  300 | Mike |   80 |     3 | Street 3 |    60 |
|  400 | Dan  |   50 |     4 | Street 4 |    30 |
|  400 | Dan  |   50 |     4 | Street 4 |    60 |
+------+------+------+-------+----------+-------+
8 rows in set (0.12 sec)
```

2.13 查询加速

2.13.1 查询调优概述

2.13.1.1 调优概述

查询性能调优是一个系统工程，需要从多层次、多维度对数据库系统进行调优。以下是调优流程和方法论概述：

1. 首先，业务人员和数据库管理员（DBA）需要对所使用的数据库系统有全面的了解，包括业务系统使用的硬件、集群的规模、使用的数据库软件版本，以及具体软件版本所提供的特性等。
2. 其次，一个好用的性能诊断工具是定位性能问题的必要前提。只有高效快速地定位到问题 SQL 或者慢 SQL，才能进行后续的具体性能调优流程。
3. 在进入性能调优环节之后，一些常用的性能分析工具是必不可少的。这其中包括当前运行数据库系统自带的特有工具，以及操作系统层面的通用工具。
4. 有了上述工具之后，使用特有工具可以获取 SQL 运行在当前数据库系统上的详细信息，帮助定位性能瓶颈，同时，通用工具也可以作为辅助分析手段帮助定位问题。

综上所述，性能调优需要从全局视角来评估当前系统的性能状况。首先需要定位存在性能问题的业务 SQL，然后运用分析工具发现性能瓶颈，最后实施具体的调优操作。

基于上述调优流程和方法论，Apache Doris 在上述各个层面都提供了相应的工具。下文将分别对性能诊断工具、分析工具、调优流程三个方面进行介绍。

2.13.1.2 诊断工具

2.13.1.2.1 概述

高效好用的性能诊断工具对于数据库系统的调优至关重要，因为这取决于是否能快速定位到存在性能问题的业务 SQL，继而快速定位和解决性能瓶颈，保证数据库系统服务的 SLA。

当前，Doris 系统默认将执行时间超过 5 秒的 SQL 认定为慢 SQL，此阈值可通过 `config.qe_slow_log_ms` 进行配置。目前 Doris 提供了以下三种诊断渠道，能够帮助快速定位存在性能问题的慢 SQL，分别如下：

2.13.1.2.2 Doris Manager 日志

Doris Manager 的日志模块提供了慢 SQL 筛选功能。用户可以通过选择特定 FE 节点上的 `fe.audit.log` 来查看慢 SQL。只需在搜索框中输入“slow_query”，即可在页面上展示当前系统的历史慢 SQL 信息，如下图所示：

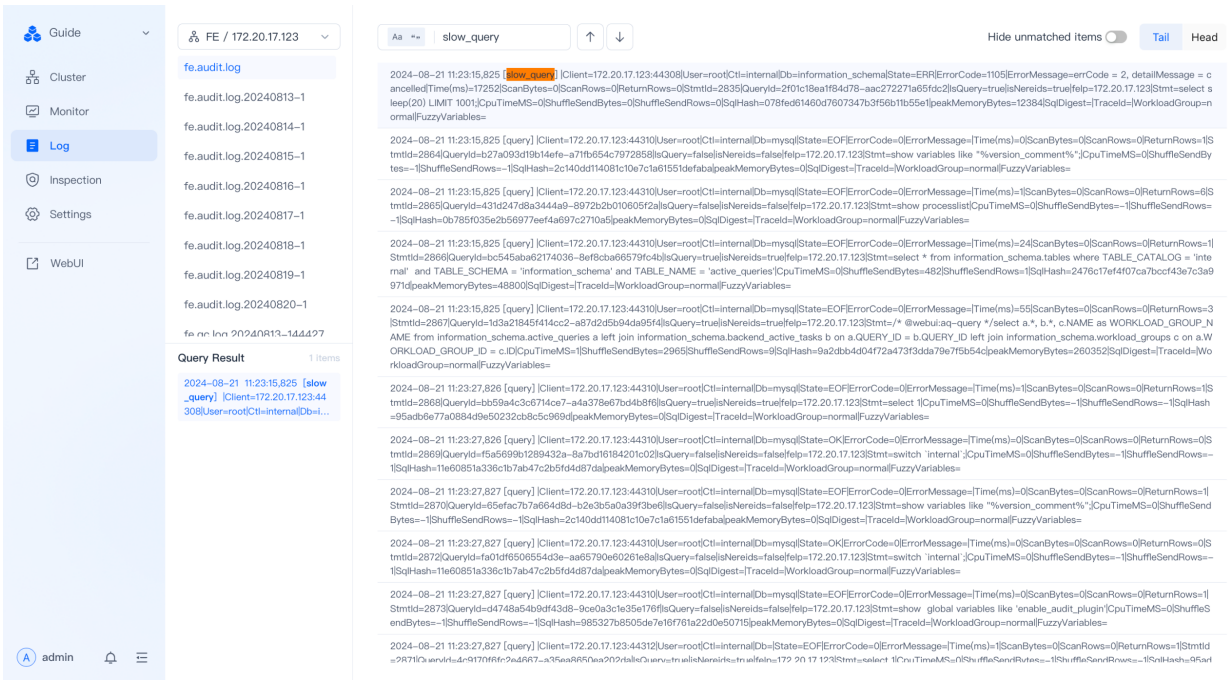


图 53: Doris Manager 监控与日志

2.13.1.2.3 Audit Log

当前 Doris FE 提供了四种类型的 Audit Log，包括 `slow_query`、`query`、`load` 和 `stream_load`。Audit Log 除了在安装部署 Manager 服务的集群上通过日志页面访问获取之外，也可以直接访问 FE 所在节点的 `fe/log/fe.audit.log` 文件获取信息。

通过直查 `fe.audit.log` 中的 `slow_query` 标签，可以快速筛选出执行缓慢的查询 SQL，如下所示：

```
2024-07-18 11:23:13,042 [slow_query] |Client=127.0.0.1:63510|User=root|Ctl=internal|Db=tpch_
    ↪ sf1000|State=EOF|ErrorCode=0|ErrorMessage=|Time(ms)=11603|ScanBytes=236667379712|ScanRows
    ↪ =13649979418|ReturnRows=100|StmtId=1689|QueryId=91ff336304f14182-9ca537eee75b3856|IsQuery
    ↪ =true|isNereids=true|feIp=172.21.0.10|Stmt=select     c_name,     c_custkey,     o_
    ↪ orderkey,     o_orderdate,     o_totalprice,     sum(l_quantity) from     customer,
    ↪ orders,     lineitem where     o_orderkey  in  (         select             l_orderkey
    ↪         from             lineitem         group  by             l_orderkey  having
    ↪                 sum(l_quantity)  >  300     )     and  c_custkey  =  o_custkey     and  o
    ↪ _orderkey  =  l_orderkey group  by     c_name,     c_custkey,     o_orderkey,     o_
    ↪ orderdate,     o_totalprice order  by     o_totalprice  desc,     o_orderdate limit  100|
    ↪ CpuTimeMS=918556|ShuffleSendBytes=3267419|ShuffleSendRows=89668|SqlHash=
    ↪ b4e1de9f251214a30188180f37907f7d|peakMemoryBytes=38720935552|SqlDigest=
    ↪ d41d8cd98f00b204e9800998ecf8427e|cloudClusterName=UNKNOWN|TraceId=|WorkloadGroup=normal|
    ↪ FuzzyVariables=|scanBytesFromLocalStorage=0|scanBytesFromRemoteStorage=0
2024-07-18 11:23:33,043 [slow_query] |Client=127.0.0.1:26672|User=root|Ctl=internal|Db=tpch_
    ↪ sf1000|State=EOF|ErrorCode=0|ErrorMessage=|Time(ms)=8978|ScanBytes=334985555968|ScanRows
    ↪ =10717654374|ReturnRows=100|StmtId=1815|QueryId=6e1fae453cb04d9a-b1e5f94d9cea1885|IsQuery
```

```
    ↪ =true|isNereids=true|feIp=172.21.0.10|Stmt=select     s_name,     count(*) as numwait
    ↪ from     supplier,     lineitem l1,     orders,     nation where     s_suppkey = l1.l_
    ↪ suppkey     and o_orderkey = l1.l_orderkey     and o_orderstatus = 'F'     and l1.l_
    ↪ receiptdate > l1.l_commitdate     and exists (         select             *         from
    ↪             lineitem l2         where                 l2.l_orderkey = l1.l_orderkey
    ↪           and l2.l_suppkey <> l1.l_suppkey     )     and not exists (         select
    ↪             *         from             lineitem l3         where                 l3.l_
    ↪ orderkey = l1.l_orderkey           and l3.l_suppkey <> l1.l_suppkey           and l3.l_
    ↪ receiptdate > l3.l_commitdate     )     and s_nationkey = n_nationkey     and n_name = '
    ↪ SAUDI ARABIA' group by     s_name order by     numwait desc,     s_name limit 100|
    ↪ CpuTimeMS=990127|ShuffleSendBytes=59208164|ShuffleSendRows=3651504|SqlHash=
    ↪ f8a30e4182d72cce3eff6cb385005b1f|peakMemoryBytes=10495660672|SqlDigest=
    ↪ d41d8cd98f00b204e9800998ecf8427e|cloudClusterName=UNKNOWN|TraceId=|WorkloadGroup=normal|
    ↪ FuzzyVariables=|scanBytesFromLocalStorage=0|scanBytesFromRemoteStorage=0
2024-07-18 11:23:41,044 [slow_query] |Client=127.0.0.1:26684|User=root|Ctl=internal|Db=tpch_
    ↪ sf1000|State=EOF|ErrorCode=0|ErrorMessage=|Time(ms)=8514|ScanBytes=334986551296|ScanRows
    ↪ =10717654374|ReturnRows=100|StmtId=1833|QueryId=4f91483464ce4aa8-beeed7dcb8675bc8|IsQuery
    ↪ =true|isNereids=true|feIp=172.21.0.10|Stmt=select     s_name,     count(*) as numwait
    ↪ from     supplier,     lineitem l1,     orders,     nation where     s_suppkey = l1.l_
    ↪ suppkey     and o_orderkey = l1.l_orderkey     and o_orderstatus = 'F'     and l1.l_
    ↪ receiptdate > l1.l_commitdate     and exists (         select             *         from
    ↪             lineitem l2         where                 l2.l_orderkey = l1.l_orderkey
    ↪           and l2.l_suppkey <> l1.l_suppkey     )     and not exists (         select
    ↪             *         from             lineitem l3         where                 l3.l_
    ↪ orderkey = l1.l_orderkey           and l3.l_suppkey <> l1.l_suppkey           and l3.l_
    ↪ receiptdate > l3.l_commitdate     )     and s_nationkey = n_nationkey     and n_name = '
    ↪ SAUDI ARABIA' group by     s_name order by     numwait desc,     s_name limit 100|
    ↪ CpuTimeMS=925841|ShuffleSendBytes=59223190|ShuffleSendRows=3651602|SqlHash=
    ↪ f8a30e4182d72cce3eff6cb385005b1f|peakMemoryBytes=10505123104|SqlDigest=
    ↪ d41d8cd98f00b204e9800998ecf8427e|cloudClusterName=UNKNOWN|TraceId=|WorkloadGroup=normal|
    ↪ FuzzyVariables=|scanBytesFromLocalStorage=0|scanBytesFromRemoteStorage=0
2024-07-18 11:23:49,044 [slow_query] |Client=127.0.0.1:10748|User=root|Ctl=internal|Db=tpch_
    ↪ sf1000|State=EOF|ErrorCode=0|ErrorMessage=|Time(ms)=8660|ScanBytes=334987673600|ScanRows
    ↪ =10717654374|ReturnRows=100|StmtId=1851|QueryId=4599cb1bab204f80-ac430dd78b45e3da|IsQuery
    ↪ =true|isNereids=true|feIp=172.21.0.10|Stmt=select     s_name,     count(*) as numwait
    ↪ from     supplier,     lineitem l1,     orders,     nation where     s_suppkey = l1.l_
    ↪ suppkey     and o_orderkey = l1.l_orderkey     and o_orderstatus = 'F'     and l1.l_
    ↪ receiptdate > l1.l_commitdate     and exists (         select             *         from
    ↪             lineitem l2         where                 l2.l_orderkey = l1.l_orderkey
    ↪           and l2.l_suppkey <> l1.l_suppkey     )     and not exists (         select
    ↪             *         from             lineitem l3         where                 l3.l_
    ↪ orderkey = l1.l_orderkey           and l3.l_suppkey <> l1.l_suppkey           and l3.l_
    ↪ receiptdate > l3.l_commitdate     )     and s_nationkey = n_nationkey     and n_name = '
    ↪ SAUDI ARABIA' group by     s_name order by     numwait desc,     s_name limit 100|
    ↪ CpuTimeMS=932664|ShuffleSendBytes=59223178|ShuffleSendRows=3651991|SqlHash=
```

```
    ↪ f8a30e4182d72cce3eff6cb385005b1f|peakMemoryBytes=10532849344|SqlDigest=
    ↪ d41d8cd98f00b204e9800998ecf8427e|cloudClusterName=UNKNOWN|TraceId=|WorkloadGroup=normal|
    ↪ FuzzyVariables=|scanBytesFromLocalStorage=0|scanBytesFromRemoteStorage=0
```

通过 fe.audit.log 获取的慢 SQL，使用者可以方便地获取执行时间、扫描行数、返回行数、SQL 语句等详细信息，为进一步重现和定位性能问题奠定了基础。

2.13.1.2.4 audit_log 系统表

Doris 2.1 以后的版本在 __internal_schema 数据库下提供了 audit_log 系统表，供用户查看 SQL 运行的情况。使用前需要打开全局配置 set global enable_audit_plugin=true;（此开关默认关闭）

```
mysql> use __internal_schema;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----------------------------+
| Tables_in___internal_schema |
+-----------------------------+
| audit_log                   |
| column_statistics           |
| histogram_statistics        |
| partition_statistics        |
+-----------------------------+
4 rows in set (0.00 sec)

mysql> desc audit_log;
+-------------------+--------------+------+-------+---------+-------+
| Field             | Type         | Null | Key   | Default | Extra |
+-------------------+--------------+------+-------+---------+-------+
| query_id          | varchar(48)  | Yes  | true  | NULL    |       |
| time              | datetime     | Yes  | true  | NULL    |       |
| client_ip         | varchar(128) | Yes  | true  | NULL    |       |
| user              | varchar(128) | Yes  | false | NULL    | NONE  |
| catalog           | varchar(128) | Yes  | false | NULL    | NONE  |
| db                | varchar(128) | Yes  | false | NULL    | NONE  |
| state             | varchar(128) | Yes  | false | NULL    | NONE  |
| error_code        | int          | Yes  | false | NULL    | NONE  |
| error_message     | text         | Yes  | false | NULL    | NONE  |
| query_time        | bigint       | Yes  | false | NULL    | NONE  |
| scan_bytes        | bigint       | Yes  | false | NULL    | NONE  |
| scan_rows         | bigint       | Yes  | false | NULL    | NONE  |
| return_rows       | bigint       | Yes  | false | NULL    | NONE  |
| stmt_id           | bigint       | Yes  | false | NULL    | NONE  |
```

```
| is_query          | tinyint      | Yes  | false | NULL    | NONE  |
| frontend_ip       | varchar(128) | Yes  | false | NULL    | NONE  |
| cpu_time_ms       | bigint       | Yes  | false | NULL    | NONE  |
| sql_hash          | varchar(128) | Yes  | false | NULL    | NONE  |
| sql_digest        | varchar(128) | Yes  | false | NULL    | NONE  |
| peak_memory_bytes | bigint       | Yes  | false | NULL    | NONE  |
| stmt              | text         | Yes  | false | NULL    | NONE  |
+-------------------+--------------+------+-------+---------+-------+
```

通过 audit_log 内部表，用户可以查询详细的 SQL 执行信息，进行如慢查询筛选等详细统计分析。

2.13.1.2.5 总结

Doris Manager 日志，audit log 以及 audit_log 系统表等工具，可以提供慢 SQL 自动或手动筛选过滤，以及细粒度 SQL 执行信息统计分析等能力。这些工具为系统性的性能诊断和调优提供了强大支撑。

2.13.1.3 分析工具

2.13.1.3.1 概述

上节诊断工具已经帮助业务和运维人员定位到具体的慢 SQL，本章节开始介绍如何对慢 SQL 的性能瓶颈进行分析，以确定具体慢在 SQL 执行的哪个环节。

一条 SQL 的执行过程大致可以分为计划生成和计划执行两个阶段，前一部分负责生成执行计划，后一部分负责具体计划的执行。这两个部分出现问题都可能导致性能瓶颈的发生。比如生成了差计划，那么即使再优秀的执行器也不可能获得很好的性能。同样一个正确的计划，如果相应的执行手段不合适，也容易产生性能瓶颈。此外，执行器的性能和当前运行的硬件和系统架构有紧密的关系，一些基础设施的缺陷或者配置不正确也会导致性能问题。

上述三类问题都需要良好的分析工具的支持。基于此，Doris 系统提供了两个性能分析工具来分别分析计划以及执行的性能瓶颈。另外系统级别也提供了相应的性能检测工具，辅助定位性能瓶颈。下面分别就这三个方面进行介绍：

2.13.1.3.2 Doris Explain

执行计划是对一条 SQL 具体的执行方式和执行过程的描述。例如，对于一个两表连接的 SQL，执行计划会展示这两张表的访问方式信息、连接方式信息，以及连接的顺序等。

Doris 提供了 Explain 工具，可以方便的展示一个 SQL 的执行计划的详细信息。通过对 Explain 输出的计划进行分析，可以帮助使用者快速定位计划层面的瓶颈，从而针对不同的情况进行计划层面的调优。

Doris 提供了多种不同粒度的 Explain 工具，如 Explain Verbose、Explain All Plan、Explain Memo Plan、Explain Shape Plan，分别用于展示最终物理计划、各阶段逻辑计划、基于成本优化过程的计划、计划形态等。详细信息请参考执行计划 Explain，了解各种 Explain 的使用方法和输出信息的解释。

通过分析 Explain 的输出，业务人员和 DBA 就可以快速定位当前计划的性能瓶颈。例如，通过分析执行计划发现 Filter 没有下推到基表，导致没有提前过滤数据，使得参与计算的数据量过多，从而导致性能问题。又如，两表的 Inner 等值连接中，连接条件一侧的过滤条件没有推导到另外一侧，导致没有对另一侧的表数据进行提前过滤，也可能导致性能不优。此类性能瓶颈都可以通过分析 Explain 的输出来定位和解决。

使用 Doris Explain 输出进行计划层调优的案例详见计划调优章节。

2.13.1.3.3 Doris Profile

上述 Explain 工具描述了一条 SQL 的执行的规划，比如一个 t1 和 t2 表的连接操作被规划成了 Hash Join 的执行方式，并且 t1 表被规划在 build 侧，t2 表被规划在 probe 侧。当 SQL 具体执行时，如何了解每个具体的执行分别耗费多少时间，比如 build 耗费多少时间，probe 耗费多少时间，profile 工具提供了详细的执行信息供性能分析和调优使用。下面部分先整体介绍 Profile 的文件结构，然后分别介绍 Merged Profile，Execution Profile 以及 PipelineTask 的执行时间含义：

Profile 文件结构

Profile 文件中包含几个主要的部分：

1. 查询基本信息：包括 ID，时间，数据库等。
2. SQL 语句以及执行计划。
3. FE 的耗时（Plan Time，Schedule Time 等）。
4. BE 在执行过程中各个 operator 的执行耗时（包括 Merged Profile 和 Execution Profile）。

执行侧的详细信息主要包含在最后一部分，接下来主要介绍 Profile 能够提供哪些信息供性能分析使用。

Merged Profile

为了帮助用户更准确的分析性能瓶颈，Doris 提供了各个 operator 聚合后的 profile 结果。以 EXCHANGE_OPERATOR 为例：

```
EXCHANGE_OPERATOR  (id=4):
    -  BlocksProduced:  sum  0,  avg  0,  max  0,  min  0
    -  CloseTime:  avg  34.133us,  max  38.287us,  min  29.979us
    -  ExecTime:  avg  700.357us,  max  706.351us,  min  694.364us
    -  InitTime:  avg  648.104us,  max  648.604us,  min  647.605us
    -  MemoryUsage:  sum  ,  avg  ,  max  ,  min
    -  PeakMemoryUsage:  sum  0.00  ,  avg  0.00  ,  max  0.00  ,  min  0.00
    -  OpenTime:  avg  4.541us,  max  5.943us,  min  3.139us
    -  ProjectionTime:  avg  0ns,  max  0ns,  min  0ns
    -  RowsProduced:  sum  0,  avg  0,  max  0,  min  0
    -  WaitForDependencyTime:  avg  0ns,  max  0ns,  min  0ns
    -  WaitForData0:  avg  9.434ms,  max  9.476ms,  min  9.391ms
```

Merged Profile 对每个 operator 的核心指标做了合并，核心指标和含义包括：

指标名称	指标含义
BlocksProduced	产生的 Data Block 数量
CloseTime	Operator 在 close 阶段的耗时
ExecTime	Operator 在各个阶段执行的总耗时
InitTime	Operator 在 Init 阶段的耗时
MemoryUsage	Operator 在执行阶段的内存用量
OpenTime	Operator 在 Open 阶段的耗时

指标名称	指标含义
ProjectionTime	Operator 在做 projection 的耗时
RowsProduced	Operator 返回的行数
WaitForDependencyTime	Operator 等待自身执行的条件依赖的时间

Doris 中，每个 operator 根据用户设置的并发数并发执行，所以 Merged Profile 对每个执行并发又计算出了每个指标的 Max，Avg 和 Min 的值。

其中 WaitForDependencyTime 是每个 Operator 不同的，因为每个 operator 执行的条件依赖不同，例如在这个例子的 EXCHANGE_OPERATOR 中，条件依赖是有数据被上游的算子通过 rpc 发送过来，所以这里的 WaitForDependencyTime 其实就是在等待上游算子发数据。

Execution Profile

区别于 Merged Profile，Execution Profile 展示的是具体的某个并发中的详细指标，以 id=4 的这个 exchange operator 为例：

```
EXCHANGE_OPERATOR  (id=4):(ExecTime:  706.351us)
      -  BlocksProduced:  0
      -  CloseTime:  38.287us
      -  DataArrivalWaitTime:  0ns
      -  DecompressBytes:  0.00
      -  DecompressTime:  0ns
      -  DeserializeRowBatchTimer:  0ns
      -  ExecTime:  706.351us
      -  FirstBatchArrivalWaitTime:  0ns
      -  InitTime:  647.605us
      -  LocalBytesReceived:  0.00
      -  MemoryUsage:
      -  PeakMemoryUsage:  0.00
      -  OpenTime:  5.943us
      -  ProjectionTime:  0ns
      -  RemoteBytesReceived:  0.00
      -  RowsProduced:  0
      -  SendersBlockedTotalTimer(*):  0ns
      -  WaitForDependencyTime:  0ns
      -  WaitForData0:  9.476ms
```

在这个 profile 中，例如 LocalBytesReceived 是 exchange operator 特化的一个指标，其他的 operator 中没有，所以没在 Merged Profile 中包含。

PipelineTask 执行时间

在 Doris 中，一个 PipelineTask 由多个 operator 组成。分析一个 PipelineTask 的执行耗时的时候，需要重点关注几个方面。

1. ExecuteTime：整个 PipelineTask 的实际执行时间，约等于这个 task 中所有 operator 的 ExecTime 相加

2. WaitWorkerTime：task 等待执行 worker 的时间。当 task 处于 runnable 状态时，他要等待一个空闲 worker 来执行，这个耗时主要取决于集群负载。
3. 等待执行依赖的时间：一个 task 可以执行的依赖条件是每个 operator 的 dependency 全部满足执行条件，而 task 等待执行依赖的时间就是将这些依赖的等待时间相加。例如简化这个例子中的其中一个 task：

```
PipelineTask  (index=1):(ExecTime:  4.773ms)
  -  ExecuteTime:  1.656ms
      -  CloseTime:  90.402us
      -  GetBlockTime:  11.235us
      -  OpenTime:  1.448ms
      -  PrepareTime:  1.555ms
      -  SinkTime:  14.228us
  -  WaitWorkerTime:  63.868us
    DATA_STREAM_SINK_OPERATOR  (id=8,dst_id=8):(ExecTime:  1.688ms)
      -  WaitForDependencyTime:  0ns
          -  WaitForBroadcastBuffer:  0ns
          -  WaitForRpcBufferQueue:  0ns
    AGGREGATION_OPERATOR  (id=7  ,  nereids_id=648):(ExecTime:  398.12us)
      -  WaitForDependency[AGGREGATION_OPERATOR_DEPENDENCY]Time:  10.495ms
```

这个 task 包含了（DATA_STREAM_SINK_OPERATOR - AGGREGATION_OPERATOR）两个 operator，其中 DATA_STREAM_SINK_OPERATOR 有两个依赖（WaitForBroadcastBuffer 和 WaitForRpcBufferQueue），AGGREGATION_OPERATOR 有一个依赖（AGGREGATION_OPERATOR_DEPENDENCY），所以当前 task 的耗时分布如下：

```
1. 执行总时间：1.656ms（约等于两个 operator 的 ExecTime 总和）
2. 等待 Worker 的时间：63.868us（说明当前集群负载不高，task 就绪以后立即就有 worker 来执行）
3. 等待执行依赖的时间（WaitForBroadcastBuffer + WaitForRpcBufferQueue + WaitForDependency[
    ↪ AGGREGATION_OPERATOR_DEPENDENCY]Time）：10.495ms。当前 task 的所有 dependency
    ↪ 相加得到总的等待时间。
```

使用 Profile 进行执行层调优的案例详见执行调优章节。

2.13.1.3.4 系统级性能工具

常用的系统工具，可以用来辅助定位执行期的性能瓶颈，比如常用的 Linux 下 top / free/ perf/ sar/ iostats 等，都可以用来观察 SQL 运行时系统 CPU/ MEM / IO / NETWORK 状态，以辅助定位性能瓶颈。

2.13.1.3.5 总结

好用的性能分析工具是快速定位性能瓶颈的重要前提。Doris 提供了 Explain 和 Profile，为分析执行计划问题和执行期哪个操作耗时高的问题，提供了强大的工具支撑。同时，熟练使用系统级别的分析工具也会对性能瓶颈的定位起到很好的辅助作用。

2.13.1.4 调优流程

2.13.1.4.1 概述

性能调优是一个系统工程，需要一个完善的方法论和实施体系，来进行系统化的诊断和调优。Doris 系统有了诊断工具和分析工具的强大支持，可以高效的进行性能问题的诊断，分析定位和调优解决。完整的调优四步流程如下所示：

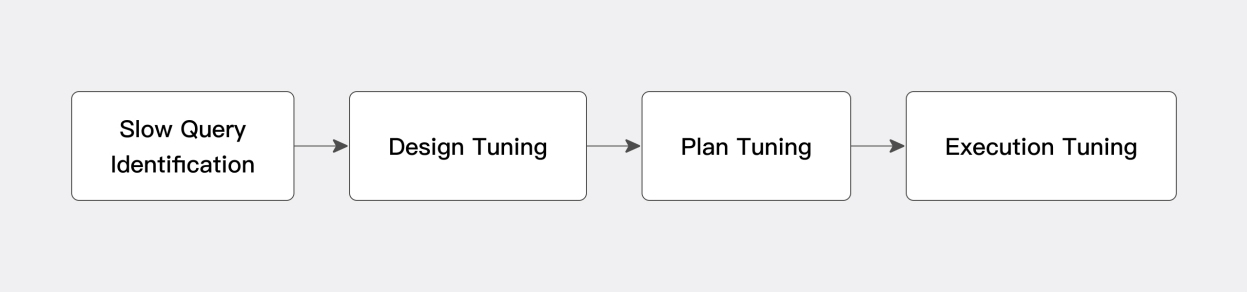


图 54: 性能调优流程

2.13.1.4.2 第 1 步：使用性能诊断工具进行慢查询定位

针对运行在 Doris 上的业务系统，使用上述性能诊断工具进行慢 SQL 的定位。

- 如果已经安装了 Doris Manager，推荐使用 Manager 日志页面，方便的进行可视化的慢查询定位。
- 如果没有安装 Manager，可以直查 FE 节点上的 `fe.audit.log` 或者 audit_log 系统表来获取慢 SQL 列表，按优先级进行调优。

2.13.1.4.3 第 2 步：Schema 设计与调优

定位到具体的慢 SQL 之后，优先需要对业务 Schema 设计进行检查与调优，排除因为 Schema 设计不合理导致的性能问题。

Schema 设计调优基本可分为三个方面：

- 表级别 Schema 设计调优，如分区分桶个数和字段调优；
- 索引的设计和调优；
- 特定优化手段的使用，如使用 Colocate Group 优化 Join 等。主要目的是排除因为 Schema 设计不合理或者没有充分利用 Doris 现有优化能力导致的性能问题。

详细调优案例请参考文档计划调优。

2.13.1.4.4 第 3 步：计划调优

检查和优化完业务 Schema 后，将进入调优的主体工作，即计划调优与执行调优。如上所述，在性能调优工具中，这个阶段的主要工作是充分利用 Doris 所提供的各种层级的 Explain 工具，对慢 SQL 的执行计划进行系统分析，以找到关键优化点进行针对性优化。

- 针对单表查询和分析场景，可以通过分析执行计划，查看分区裁剪是否正常，使用单表物化视图进行查询加速等。

- 针对复杂多表分析场景，可以分析 Join Order 是否合理等定位具体的性能瓶颈，也可以使用多表物化视图进行透明改写，以加速查询。如果出现非预期的情况，比如 Join Order 不合理，通过观察 Explain 的结果，手工指定 Join Hint 进行执行计划的绑定，如使用 Leading hint 控制 Join Order，使用 Shuffle Hint 调整 Join shuffle 方式，使用 Hint 控制代价改写行为等，以达到调优执行计划的目的。
- 针对部分特定场景，还可以通过使用 Doris 提供的高级功能，比如使用 SQL Cache 加速查询。

详细调优案例请参考文档计划调优。

2.13.1.4.5 第 4 步：执行调优

进入执行调优阶段后，需要根据 SQL 的实际运行情况，一方面验证计划调优的效果，另外一方面在现有计划的前提下，继续分析执行侧的瓶颈点，定位哪个执行阶段慢，或者其他普遍性的原因，如并行度不优等。

以多表分析的查询为例，我们可以通过分析 Profile，来检查计划规划的 Join 顺序是否合理，Runtime Filter 是否生效，并行度是否符合预期等。此外 Profile 还能反馈出一些机器负载的情况，例如 io 慢，网络传输性能不符合预期等。在对这类问题进行确认和定位时，需要使用系统级别的工具来辅助诊断和调优。

详细调优案例请参考文档执行调优。

> 提示在分析具体性能问题的时候，推荐先检查计划，后调优执行的顺序。首先利用 Explain 工具进行执行计划的确认，然后再利用 Profile 工具进行执行性能的定位和调优。如果使用顺序颠倒，有可能会导致效率低下，不利于性能问题的快速定位。

2.13.1.4.6 总结

查询调优是一个系统工程，Doris 为用户提供了各个维度的工具，方便从不同层面进行性能问题的诊断、定位、分析与解决。业务人员和 DBA 熟悉了这些诊断和分析工具后，使用合理的调优方法，能够快速有效的解决性能瓶颈，更好的释放 Doris 强大的性能优势，更好的适配业务场景进行业务赋能。

2.13.2 物化视图

2.13.2.1 物化视图概览

物化视图是既包含计算逻辑也包含数据的实体。它不同于视图，因为视图仅包含计算逻辑，本身不存储数据。

2.13.2.1.1 物化视图的使用场景

物化视图根据 SQL 定义计算并存储数据，且根据策略进行周期性或实时性更新。物化视图可直接查询，也可以将查询透明改写。它可用于以下几个场景：

查询加速

在决策支持系统中，如 BI 报表、Ad-Hoc 查询等，这类分析型查询通常包含聚合操作，可能还涉及多表连接。由于计算此类查询结果较为消耗资源、响应时间可能长达分钟级，且业务场景往往要求秒级响应，可以构建物化视图，对常见查询进行加速。

轻量化 ETL（数据建模）

在数据分层场景中，可以使用物化视图的嵌套来构建 DWD 和 DWM 层，利用物化视图的调度刷新能力。

湖仓一体

针对多种外部数据源，可以将这些数据源所使用的表进行物化视图构建，以此来节省从外部表导入数据到内部表的成本，并且加速查询过程。

2.13.2.1.2 物化视图的分类

按照数据时效性分类：同步 vs 异步

- 同步物化视图需要与基表的数据保持强一致性。
- 异步物化视图与基表的数据保持最终一致性，可能会有一定的延迟。它通常用于对数据时效性要求不高的场景，一般使用 T+1 或小时级别的数据来构建物化视图。如果时效性要求高，则考虑使用同步物化视图。

目前，同步物化视图不支持直接查询，而异步物化视图支持直接查询。

按照支持透明改写的 SQL 模式分类：单表 vs 多表

物化视图的定义 SQL 可以包含单表查询，也可以包含多表查询。从使用表的数量角度出发，可以划分物化视图为单表物化视图或多表物化视图。

- 对于异步物化视图，可以使用单表或多表。
- 对于同步物化视图，只能使用单表。

按照物化视图刷新分类：全量刷新 vs 分区增量刷新 vs 实时刷新

对于异步物化视图

- 全量刷新：计算物化视图定义 SQL 的所有数据。
- 分区增量刷新：当物化视图基表的分区数据发生变化时，识别出对应变化的分区并刷新这些分区，从而实现分区增量刷新，而无需刷新整个物化视图。

对于同步物化视图

- 可以理解为实时刷新，保持与基表的数据一致。

2.13.2.2 同步物化视图

2.13.2.2.1 什么是同步物化视图

同步物化视图是将预先计算（根据定义好的 SELECT 语句）的数据集，存储在 Doris 中的一个特殊的表。Doris 会自动维护同步物化视图的数据，无论是新增数据还是删除数据，都能保证基表（Base Table）和物化视图表的数据同步更新并保持一致，只有同步完成后，相关命令才会结束，无需任何额外的人工维护成本。查询时，Doris 会自动匹配到最优的物化视图，并直接从物化视图中读取数据。

2.13.2.2.2 适用场景

- 加速耗时的聚合运算
- 查询需要匹配不同的前缀索引
- 通过预先过滤减少需要扫描的数据量
- 通过预先完成复杂的表达式计算来加速查询

2.13.2.2.3 局限性

- 同步物化视图只支持针对单个表的 SELECT 语句，支持 WHERE、 GROUP BY、 ORDER BY 等子句，但不支持 JOIN、 HAVING、 LIMIT 子句和 LATERAL VIEW。
- 与异步物化视图不同，不能直接查询同步物化视图。
- SELECT 列表中，不能包含自增列，不能包含常量，不能有重复表达式，也不支持窗口函数。
- 如果 SELECT 列表包含聚合函数，则聚合函数必须是根表达式（不支持 `sum(a)+ 1`，支持 `sum(a + 1)`），且聚合函数之后不能有其他非聚合函数表达式（例如，`SELECT x, sum(a)` 可以，而 `SELECT sum(a), x` 不行）。
- 如果删除语句的条件列在物化视图中存在，则不能进行删除操作。如果确实需要删除数据，则需要先将物化视图删除，然后才能删除数据。
- 单表上过多的物化视图会影响导入的效率：导入数据时，物化视图和 Base 表的数据是同步更新的。如果一张表的物化视图表过多，可能会导致导入速度变慢，这就像单次导入需要同时导入多张表的数据一样。
- 物化视图针对 Unique Key 数据模型时，只能改变列的顺序，不能起到聚合的作用。因此，在 Unique Key 模型上不能通过创建物化视图的方式对数据进行粗粒度的聚合操作。

2.13.2.2.4 使用物化视图

Doris 系统提供了一整套针对物化视图的 DDL 语法，包括创建、查看和删除。下面通过一个示例来展示如何使用物化视图加速聚合计算。假设用户有一张销售记录明细表，该表存储了每个交易的交易 ID、销售员、售卖门店、销售时间以及金额。建表语句和插入数据语句如下：

```
-- 创建一个 test_db
create database test_db;
use test_db;

-- 创建表
create table sales_records
(
    record_id int,
    seller_id int,
    store_id int,
    sale_date date,
```

```
    sale_amt bigint
)
distributed by hash(record_id)
properties("replication_num" = "1");

-- 插入数据
insert into sales_records values(1,1,1,"2020-02-02",1), (1,1,1,"2020-02-02",2);
```

创建物化视图

如果用户经常需要分析不同门店的销售量，则可以为 sales_records 表创建一个物化视图，该视图以售卖门店分组，并对相同售卖门店的销售额进行求和。创建语句如下：

```
create materialized view store_amt as
select store_id, sum(sale_amt) from sales_records group by store_id;
```

检查物化视图是否创建完成

由于创建物化视图是一个异步操作，用户在提交创建物化视图任务后，可以通过命令异步地检查物化视图是否构建完成。命令如下：

```
show alter table materialized view from test_db;
```

该命令的结果将显示该数据库的所有创建物化视图的任务。结果示例如下：

```
+--
    ↪ ------+---------------+---------------------+---------------------+---------------+------------------+----
    ↪
| JobId  | TableName     | CreateTime          | FinishTime          | BaseIndexName |
    ↪ RollupIndexName | RollupId | TransactionId | State    | Msg  | Progress | Timeout |
+--
    ↪ ------+---------------+---------------------+---------------------+---------------+------------------+----
    ↪
| 494349 | sales_records | 2020-07-30 20:04:56 | 2020-07-30 20:04:57 | sales_records | store_amt
    ↪       | 494350   | 133107        | FINISHED |      | NULL     | 2592000 |
+--
    ↪ ------+---------------+---------------------+---------------------+---------------+------------------+----
    ↪
```

其中，TableName 指的是物化视图的数据来源表，RollupIndexName 指的是物化视图的名称。比较重要的指标是 State。当创建物化视图任务的 State 变为 FINISHED 时，就说明这个物化视图已经创建成功了。这意味着，在执行查询时有可能自动匹配到这张物化视图。

取消创建物化视图

如果创建物化视图的后台异步任务还未结束，可以通过以下命令取消任务：

```
cancel alter table materialized view from test_db.sales_records;
```

如果物化视图已经创建完毕，则无法通过该命令取消创建，但可以通过删除命令来删除物化视图。

查看物化视图的表结构

可以通过以下命令查看目标表上创建的所有物化视图及其表结构：

```
desc sales_records all;
```

该命令的结果如下：

```
+--
    ↪ -------------+---------------+---------------------+--------+-------------+------+-------+---------+----
    ↪
| IndexName     | IndexKeysType | Field               | Type   | InternalType | Null | Key   |
    ↪ Default | Extra | Visible | DefineExpr | WhereClause |
+--
    ↪ -------------+---------------+---------------------+--------+-------------+------+-------+---------+----
    ↪
| sales_records | DUP_KEYS      | record_id           | INT    | INT          | Yes  | true  |
    ↪ NULL    |       | true    |            |             |
|               |               | seller_id           | INT    | INT          | Yes  | true  |
    ↪ NULL    |       | true    |            |             |
|               |               | store_id            | INT    | INT          | Yes  | true  |
    ↪ NULL    |       | true    |            |             |
|               |               | sale_date           | DATE   | DATEV2       | Yes  | false |
    ↪ NULL    | NONE  | true    |            |             |
|               |               | sale_amt            | BIGINT | BIGINT       | Yes  | false |
    ↪ NULL    | NONE  | true    |            |             |
|               |               |                     |        |              |      |       |
    ↪         |       |         |            |             |
| store_amt     | AGG_KEYS      | mv_store_id         | INT    | INT          | Yes  | true  |
    ↪ NULL    |       | true    | `store_id` |             |
|               |               | mva_SUM__`sale_amt` | BIGINT | BIGINT       | Yes  | false |
    ↪ NULL    | SUM   | true    | `sale_amt` |             |
+--
    ↪ -------------+---------------+---------------------+--------+-------------+------+-------+---------+----
    ↪
```

可以看到，sales_records有一个名叫store_amt的物化视图，这个物化视图就是前面步骤创建的。

查看物化视图的创建语句

可以通过以下命令查看物化视图的创建语句：

```
show create materialized view store_amt on sales_records;
```

输出如下：

```
+--
    ↪ -------------+-----------+-----------------------------------------------------------------------------
    ↪
| TableName     | ViewName  | CreateStmt   |
+--
    ↪ -------------+-----------+-----------------------------------------------------------------------------
    ↪
| sales_records | store_amt | create materialized view store_amt as select store_id, sum(sale_amt
    ↪ ) from sales_records group by store_id |
+--
    ↪ -------------+-----------+-----------------------------------------------------------------------------
    ↪
```

查询物化视图

当物化视图创建完成后，用户在查询不同门店的销售量时，Doris 会直接从刚才创建的物化视图store_amt中读取聚合好的数据，从而提升查询效率。用户的查询依旧指定查询sales_records表，比如：

```
select store_id, sum(sale_amt) from sales_records group by store_id;
```

上面的查询就能自动匹配到store_amt。用户可以通过下面的命令，检验当前查询是否匹配到了合适的物化视图。

```
explain select store_id, sum(sale_amt) from sales_records group by store_id;
```

结果如下：

```
+------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                        |
+------------------------------------------------------------------------+
| PLAN FRAGMENT 0                                                        |
|   OUTPUT EXPRS:                                                        |
|     store_id[#11]                                                      |
|     sum(sale_amt)[#12]                                                 |
|   PARTITION: HASH_PARTITIONED: mv_store_id[#7]                         |
|                                                                        |
|   HAS_COLO_PLAN_NODE: true                                             |
|                                                                        |
|   VRESULT SINK                                                         |
|      MYSQL_PROTOCAL                                                    |
|                                                                        |
|   3:VAGGREGATE (merge finalize)(384)                                   |
|   |  output: sum(partial_sum(mva_SUM__`sale_amt`)[#8])[#10]            |
|   |  group by: mv_store_id[#7]                                         |
|   |  sortByGroupKey:false                                              |
|   |  cardinality=1                                                     |
|   |  final projections: mv_store_id[#9], sum(mva_SUM__`sale_amt`)[#10] |
```

```
|   |  final project output tuple id: 4                                                  |
|   |  distribute expr lists: mv_store_id[#7]                                            |
|   |                                                                                    |
|   2:VEXCHANGE                                                                        |
|      offset: 0                                                                       |
|      distribute expr lists:                                                          |
|                                                                                      |
| PLAN FRAGMENT 1                                                                      |
|                                                                                      |
|   PARTITION: HASH_PARTITIONED: record_id[#2]                                         |
|                                                                                      |
|   HAS_COLO_PLAN_NODE: false                                                          |
|                                                                                      |
|   STREAM DATA SINK                                                                   |
|     EXCHANGE ID: 02                                                                  |
|     HASH_PARTITIONED: mv_store_id[#7]                                                |
|                                                                                      |
|   1:VAGGREGATE (update serialize)(374)                                               |
|   |  STREAMING                                                                       |
|   |  output: partial_sum(mva_SUM__`sale_amt`[#1])[#8]                                |
|   |  group by: mv_store_id[#0]                                                       |
|   |  sortByGroupKey:false                                                            |
|   |  cardinality=1                                                                   |
|   |  distribute expr lists:                                                          |
|   |                                                                                  |
|   0:VOlapScanNode(369)                                                               |
|      TABLE: test_db.sales_records(store_amt), PREAGGREGATION: ON                     |
|      partitions=1/1 (sales_records)                                                  |
|      tablets=10/10, tabletList=266568,266570,266572 ...                              |
|      cardinality=1, avgRowSize=1805.0, numNodes=1                                    |
|      pushAggOp=NONE                                                                  |
|                                                                                      |
|                                                                                      |
| ========== MATERIALIZATIONS ==========                                              |
|                                                                                      |
| MaterializedView                                                                     |
| MaterializedViewRewriteSuccessAndChose:                                              |
|   internal.test_db.sales_records.store_amt chose,                                    |
|                                                                                      |
| MaterializedViewRewriteSuccessButNotChose:                                           |
|   not chose: none,                                                                   |
|                                                                                      |
| MaterializedViewRewriteFail:                                                         |
|                                                                                      |
|                                                                                      |
```

```
| ========== STATISTICS ==========                                          |
| planed with unknown column statistics                                     |
+---------------------------------------------------------------------------+
```

MaterializedViewRewriteSuccessAndChose 会展示被成功命中的物化视图，具体示例如下：

```
+---------------------------------------------------------------------------+
| MaterializedViewRewriteSuccessAndChose:                                   |
|   internal.test_db.sales_records.store_amt chose,                         |
+---------------------------------------------------------------------------+
```

上述内容表明，查询成功命中了名为 store_amt 的物化视图。值得注意的是，若目标表中无任何数据，则可能不会触发对物化视图的命中。

关于 MATERIALIZATIONS 的详细说明：

- MaterializedViewRewriteSuccessAndChose：展示被成功选中并用于查询优化的物化视图。
- MaterializedViewRewriteSuccessButNotChose：展示匹配成功但未被选中的物化视图（优化器会基于物化视图的成本进行最优选择，这些匹配但未被选中的物化视图，表示它们并非最优选择）。
- MaterializedViewRewriteFail：展示未能匹配的物化视图，即原始 SQL 查询与现有物化视图无法匹配，因此无法使用物化视图进行优化。

删除物化视图

```
drop materialized view store_amt on sales_records;
```

2.13.2.2.5 使用示例

示例一：加速聚合查询

业务场景： 计算广告的 UV（独立访客数）和 PV（页面访问量）。

1. 假设用户的原始广告点击数据存储在 Doris 中，那么针对广告 PV 和 UV 的查询就可以通过创建带有 bitmap_union 的物化视图来提升查询速度。首先，创建一个存储广告点击数据明细的表，包含每条点击的点击时间、点击的广告、点击的渠道以及点击的用户。原始表创建语句如下：

```
create table advertiser_view_record
(
    click_time datetime,
    advertiser varchar(10),
    channel varchar(10),
    user_id int
) distributed by hash(user_id) properties("replication_num" = "1");
insert into advertiser_view_record values("2020-02-02 02:02:02",'a','a',1), ("2020-02-02
    ↪ 02:02:02",'a','a',2);
```

2. 用户想要查询的是广告的 UV 值，也就是需要对相同广告的用户进行精确去重，查询语句一般为：

```
select
    advertiser,
    channel,
    count(distinct user_id)
from
    advertiser_view_record
group by
    advertiser, channel;
```

3. 针对这种求 UV 的场景，可以创建一个带有 bitmap_union 的物化视图，以达到预先精确去重的效果。在 Doris 中，count(distinct) 聚合的结果和 bitmap_union_count 聚合的结果是完全一致的。因此，如果查询中涉及到 count(distinct)，则通过创建带有 bitmap_union 聚合的物化视图可以加快查询。根据当前的使用场景，可以创建一个根据广告和渠道分组，对 user_id 进行精确去重的物化视图。

```
create materialized view advertiser_uv as
select
    advertiser,
    channel,
    bitmap_union(to_bitmap(user_id))
from
    advertiser_view_record
group by
    advertiser, channel;
```

4. 当物化视图表创建完成后，查询广告 UV 时，Doris 就会自动从刚才创建好的物化视图 advertiser_uv 中查询数据。如果执行之前的 SQL 查询：

```
select
    advertiser,
    channel,
    count(distinct user_id)
from
    advertiser_view_record
group by
    advertiser, channel;
```

5. 在选中物化视图后，实际的查询会转化为：

```
select
    advertiser,
    channel,
```

```
    bitmap_union_count(to_bitmap(user_id))
from
    advertiser_uv
group by
    advertiser, channel;
```

6. 通过 explain 命令检查查询是否匹配到了物化视图：

```
explain select
    advertiser,
    channel,
    count(distinct user_id)
from
    advertiser_view_record
group by
    advertiser, channel;
```

8. 在 explain 的结果中，可以看到 internal.test_db.advertiser_view_record.advertiser_uv chose。也就是说，查询会直接扫描物化视图的数据，说明匹配成功。其次，对于 user_id 字段求 count(distinct) 被改写为求 bitmap_union_count(to_bitmap)，也就是通过 Bitmap 的方式来达到精确去重的效果。

示例二：匹配不同前缀索引

业务场景： 匹配前缀索引

用户的原始表包含三列（k1、k2、k3），其中 k1 和 k2 被设置为前缀索引列。当用户查询条件中包含where k1=1 and k2=2时，查询可以通过索引进行加速。然而，在某些情况下，用户的过滤条件可能无法匹配到前缀索引，例如 where k3=3，此时无法通过索引来提升查询速度。为了解决这个问题，可以创建一个以 k3 作为第一列的物化视图。

1. 建表语句和插入数据语句如下：

```
create table test_table
(
    k1 int,
    k2 int,
    k3 int,
    kx date
)
distributed by hash(k1)
properties("replication_num" = "1");

insert into test_table values(1,1,1,1),(3,3,3,3);
```

2. 创建 k3 为前缀索引的物化视图：

```
create materialized view mv_1 as SELECT k3, k2, k1 FROM test_table;
```

3. 使用 EXPLAIN 检查查询是否匹配物化视图：

```
explain select k1, k2, k3 from test_table where k3=3;
```

4. 输出结果如下：

```
+------------------------------------------------------------+
| Explain String(Nereids Planner)                            |
+------------------------------------------------------------+
| PLAN FRAGMENT 0                                            |
|   OUTPUT EXPRS:                                            |
|     k1[#7]                                                 |
|     k2[#8]                                                 |
|     k3[#9]                                                 |
|   PARTITION: HASH_PARTITIONED: mv_k1[#2]                   |
|                                                            |
|   HAS_COLO_PLAN_NODE: false                                |
|                                                            |
|   VRESULT SINK                                             |
|      MYSQL_PROTOCAL                                        |
|                                                            |
|   0:VOlapScanNode(256)                                     |
|      TABLE: test_db.test_table(mv_1), PREAGGREGATION: ON   |
|      PREDICATES: (mv_k3[#0] = 3)                           |
|      partitions=1/1 (test_table)                           |
|      tablets=10/10, tabletList=271177,271179,271181 ...    |
|      cardinality=1, avgRowSize=0.0, numNodes=1             |
|      pushAggOp=NONE                                        |
|      final projections: mv_k1[#2], mv_k2[#1], mv_k3[#0]    |
|      final project output tuple id: 2                      |
|                                                            |
|                                                            |
| ========== MATERIALIZATIONS ==========                     |
|                                                            |
| MaterializedView                                           |
| MaterializedViewRewriteSuccessAndChose:                    |
|   internal.test_db.test_table.mv_1 chose,                  |
|                                                            |
| MaterializedViewRewriteSuccessButNotChose:                 |
|   not chose: none,                                         |
```

```
|                                                            |
| MaterializedViewRewriteFail:                               |
|                                                            |
|                                                            |
| ========== STATISTICS ==========                          |
| planed with unknown column statistics                      |
+------------------------------------------------------------+
```

5. 在 EXPLAIN 的结果中，可以看到 `internal.test_db.test_table.mv_1 chose`，这表明查询成功命中了物化视图。

示例三：预先过滤和表达式计算加速查询

业务场景： 需要提前过滤数据或加速表达式计算。

1. 建表和插入数据语句如下：

```
create table d_table (
   k1 int null,
   k2 int not null,
   k3 bigint null,
   k4 date null
)
duplicate key (k1,k2,k3)
distributed BY hash(k1) buckets 3
properties("replication_num" = "1");

insert into d_table select 1,1,1,'2020-02-20';
insert into d_table select 2,2,2,'2021-02-20';
insert into d_table select 3,-3,null,'2022-02-20';
```

2. 创建物化视图：

```
-- mv1 提前进行表达式计算
create materialized view mv1 as
select
    abs(k1)+k2+1,
    sum(abs(k2+2)+k3+3)
from
    d_table
group by
    abs(k1)+k2+1;

-- mv2 提前用 where 表达式过滤以减少物化视图中的数据量
```

```
create materialized view mv2 as
select
    year(k4),
    month(k4)
from
    d_table
where
    year(k4) = 2020;
```

3. 通过以下查询测试检测是否成功命中物化视图：

```
-- 命中 mv1
select
    abs(k1)+k2+1,
    sum(abs(k2+2)+k3+3)
from
    d_table
group by
    abs(k1)+k2+1;

-- 命中 mv1
select
    bin(abs(k1)+k2+1),
    sum(abs(k2+2)+k3+3)
from
    d_table
group by
    bin(abs(k1)+k2+1);

-- 命中 mv2
select
    year(k4) + month(k4)
from
    d_table
where
    year(k4) = 2020;

-- 命中原始表 d_table 不会命中 mv2，因为 where 条件不匹配
select
    year(k4),
    month(k4)
from
    d_table;
```

2.13.2.2.6 常见问题

1. 当创建好物化视图后，为什么没有改写成功？

如果发现没有匹配的数据，可能是因为物化视图还处于构建过程中。此时，可以使用以下命令来查看物化视图的构建状态：

```
show alter table materialized view from test_db;
```

如果查询结果显示status字段不是FINISHED，那么需要等待，直到状态变为FINISHED后，物化视图才会变得可用。

2. 从 2.x 升级到 3.0.0，为什么之前同步物化视图不能命中了？

自 3.0.0 版本起，默认使用基于 plan 结构信息的方式对同步物化视图进行透明改写，控制开关如下，如果发现之前 2.x 能命中但是 3.0.0 不能命中。把如下开关关闭，如下开关默认开启。

```
`SET enable_sync_mv_cost_based_rewrite = true;`
```

2.13.2.3 异步物化视图

2.13.2.3.1 异步物化视图概述

物化视图作为一种高效的解决方案，兼具了视图的灵活性和物理表的高性能优势。它能够预先计算并存储查询的结果集，从而在查询请求到达时，直接从已存储的物化视图中快速获取结果，避免了重新执行复杂的查询语句所带来的开销。

使用场景

- 查询加速与并发提升：物化视图能够显著提高查询速度，同时增强系统的并发处理能力，有效减少资源消耗。
- 简化 ETL 流程：在数据抽取、转换和加载（ETL）过程中，物化视图能够简化流程，提升开发效率，使数据处理更加顺畅。
- 加速湖仓一体架构中的外表查询：在湖仓一体架构中，物化视图能够显著提升对外部数据源的查询速度，提高数据访问效率。
- 提升写入效率：通过减少资源竞争，物化视图能够优化数据写入过程，提高写入效率，确保数据的一致性和完整性。

使用限制

- 异步物化视图与基表数据一致性：异步物化视图与基表的数据最终会保持一致，但无法实时同步，即无法保持实时一致性。
- 窗口函数查询支持：当前，如果查询中包含了窗口函数，暂不支持将该查询透明地改写为利用物化视图的形式。

- 物化视图连接表多于查询表：如果物化视图所连接的表数量多于查询所涉及的表（例如，查询仅涉及 t1 和 t2，而物化视图则包含了 t1、t2 以及额外的 t3），则系统目前不支持将该查询透明地改写为利用该物化视图的形式。
- 如果物化视图包含 UNION ALL 等集合操作，LIMIT，ORDER BY，CROSS JOIN，物化视图可以正常构建，但是不能用于透明改写。

原理介绍

物化视图，作为数据库中的一种高级特性，其实质为类型 MTMV 的内表。在创建物化视图时，系统会同时注册一个刷新任务。此任务会在需要时运行，执行 INSERT OVERWRITE 语句，以将最新的数据写入物化视图中。

刷新机制与同步物化视图所采用的实时增量刷新不同，异步物化视图提供了更为灵活的刷新选项

- 全量刷新：在此模式下，系统会重新计算物化视图定义 SQL 所涉及的所有数据，并将结果完整地写入物化视图。此过程确保了物化视图中的数据与基表数据保持一致，但可能会消耗更多的计算资源和时间。
- 分区增量刷新： 当物化视图的基表分区数据发生变化时，系统能够智能地识别出这些变化，并仅针对受影响的分区进行刷新。这种机制显著降低了刷新物化视图所需的计算资源和时间，同时保证了数据的最终一致性。

透明改写： 透明改写是数据库优化查询性能的一种重要手段。在处理用户查询时，系统能够自动对 SQL 进行优化和改写，以提高查询的执行效率和降低计算成本。这一改写过程对用户而言是透明的，无需用户进行任何干预。

Doris 异步物化视图采用了基于 SPJG（SELECT-PROJECT-JOIN-GROUP-BY）模式的透明改写算法。该算法能够深入分析 SQL 的结构信息，自动寻找并选择合适的物化视图进行透明改写。在多个物化视图可供选择时，算法还会根据一定的策略（如成本模型）选择最优的物化视图来响应查询 SQL，从而进一步提升查询性能。

基于数据湖创建异步物化视图

基于数据湖创建异步物化视图的语法和基于内表创建异步物化视图的语法完全一样，但有一些注意事项：- 物化视图刷新需要使用到数据湖的元数据，例如分区版本信息，这些信息是从数据湖中的元数据缓存获取的，并不是直接从外部环境中获取，因此物化视图刷新完成后，数据和通过 Doris 查询数据湖的结果保持一致，但有可能和通过其它引擎查询数据湖的结果不一致，取决于缓存的刷新情况 - 如果 Hive 底层数据通过非 Doris 控制的外部流程（如 Spark、Hive 或 Flink 作业）发生了变更，但是却没有改变元数据，例如执行了 `insert overwrite`，会导致物化视图认为和基表数据一致，但是查询出的数据和通过 Doris 查询数据湖的结果不一致，可以手动强制刷新物化视图来解决此问题 - 基于 iceberg 创建分区物化视图，仅支持 iceberg 表分区列仅有一列的情况，并且有限度的支持分区演进功能，例如时间类型的分区时间范围发生了变化是支持的，如果分区字段发生了变化是不支持的，物化视图刷新会失败 - 基于 hudi 创建物化视图，无法感知到基表数据是否发生了变化，所以只要刷新过物化视图（或物化视图的部分分区），就会认为物化视图（或这些分区）和基表是同步的，所以基于 hudi 创建物化视图仅适用于手动按需刷新的场景

物化刷新数据湖支持情况

物化刷新数据湖的支持情况，不同类型的表和 Catalog 有不同的支持程度

```
<tr>
    <th rowspan="2">表类型</th>
    <th rowspan="2">Catalog 类型</th>
    <th colspan="2">刷新方式</th>
```

```
    <th >刷新时机</th>
</tr>
<tr>
    <th>全量刷新</th>
    <th>分区刷新</th>
    <th>自动触发</th>
</tr>
<tr>
    <td>内表</td>
    <td>Internal</td>
    <td>2.1 支持</td>
    <td>2.1 支持</td>
    <td>2.1.4 支持</td>
</tr>
<tr>
    <td>Hive</td>
    <td>Hive</td>
    <td>2.1 支持</td>
    <td>2.1 支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>Iceberg</td>
    <td>Iceberg</td>
    <td>2.1 支持</td>
    <td>3.1 支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>Paimon</td>
    <td>Paimon</td>
    <td>2.1 支持</td>
    <td>3.1 支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>Hudi</td>
    <td>Hudi</td>
    <td>2.1 支持</td>
    <td>3.1 支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>JDBC</td>
    <td>JDBC</td>
```

```
    <td>2.1 支持</td>
    <td>不支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>ES</td>
    <td>ES</td>
    <td>2.1 支持</td>
    <td>不支持</td>
    <td>不支持</td>
</tr>
```

透明改写数据湖支持情况

目前，异步物化视图的透明改写功能支持以下类型的表和 Catalog。

实时感知基表数据：指的是物化视图使用的表数据发生变化，物化视图能够实时感知到基表数据的变化，并在查询时使用最新的数据。

```
<tr>
    <th>表类型</th>
    <th>Catalog 类型</th>
    <th>透明改写支持</th>
    <th>实时感知基表数据</th>
</tr>
<tr>
    <td>内表</td>
    <td>Internal</td>
    <td>支持</td>
    <td>支持</td>
</tr>
<tr>
    <td>Hive</td>
    <td>Hive</td>
    <td>支持</td>
    <td>3.1 支持</td>
</tr>
<tr>
    <td>Iceberg</td>
    <td>Iceberg</td>
    <td>支持</td>
    <td>3.1 支持</td>
</tr>
<tr>
    <td>Paimon</td>
    <td>Paimon</td>
    <td>支持</td>
```

```
    <td>3.1 支持</td>
</tr>
<tr>
    <td>Hudi</td>
    <td>Hudi</td>
    <td>支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>JDBC</td>
    <td>JDBC</td>
    <td>支持</td>
    <td>不支持</td>
</tr>
<tr>
    <td>ES</td>
    <td>ES</td>
    <td>支持</td>
    <td>不支持</td>
</tr>
```

物化视图使用外表，此物化视图默认是不参与透明改写的。如果想要使用外表的物化视图参与透明改写，可以通过设置 `SET materialized_view_rewrite_enable_contain_external_table = true` 来开启。

自 2.1.11 起，Doris 优化了外表的透明改写性能，主要优化了获取包含外表可用物化的性能。

如果是包含外表的分区物化视图，透明改写很慢，需要在 fe.conf 中配置 `max_hive_partition_cache_num = 20000`，Hive Metastore 表级别分区缓存的最大数量，这个默认值是 10000，如果 hive 外表的分区很多，可以设置更大一些。

`external_cache_expire_time_minutes_after_access`，缓存对象自最后一次访问后经过多长时间缓存失效。默认是 10 分钟，可以适当调长。（适用于外表 schema 缓存和 Hive 元数据缓存）

`external_cache_refresh_time_minutes = 60`，外部表元数据缓存对象的自动刷新时间，默认 10 分钟，可以适当调长, 此配置 3.1 才开始支持。外表的元数据缓存配置详情请看元数据缓存。

物化视图和 OLAP 内表关系

异步物化视图定义 SQL 使用基表的表模型没有限制，可以是明细模型，主键模型（merge-on-write 和 merge-on-read），聚合模型等。

物化视图自身的底层实现依托于 Duplicate 模型的 OLAP 表，这一设计使其理论上能够支持 Duplicate 模型的所有核心功能。然而，为了保障物化视图能够稳定且高效地执行数据刷新任务，我们对其功能进行了一系列必要的限制。以下是具体的限制内容：

- 物化视图的分区是基于其基表自动创建和维护的，因此用户不能对物化视图进行分区操作
- 由于物化视图背后有相关的作业（JOB）需要处理，所以不能使用删除表（DELETE TABLE）或重命名表（RENAME TABLE）的命令来操作物化视图。相反，需要使用物化视图自身的命令来进行这些操作。
- 物化视图的列数据类型是根据创建时指定的查询语句自动推导得出的，因此这些数据类型不能被修改。否则，可能会导致物化视图的刷新任务失败。

- 物化视图具有一些 Duplicate 表没有的属性（property），这些属性需要通过物化视图的命令进行修改。而其他公用的属性则需要使用 ALTER TABLE 命令进行修改。

更多参考

创建、查询与维护异步物化视图，可以参考创建、查询与维护异步物化视图

最佳实践，可以参考最佳实践

常见问题，可以参考常见问题

2.13.2.3.2 创建、查询与维护异步物化视图

本文将详细说明物化视图创建、物化视图直查、查询改写和物化视图常见运维。

物化视图创建

权限说明

- 创建物化视图：需要具有物化视图的创建权限（与建表权限相同）以及创建物化视图查询语句的查询权限（与 SELECT 权限相同）。

创建语法

```
CREATE MATERIALIZED VIEW
[ IF NOT EXISTS ] <materialized_view_name>
    [ (<columns_definition>) ]
    [ BUILD <build_mode> ]
    [ REFRESH <refresh_method> [refresh_trigger]]
    [ [DUPLICATE] KEY (<key_cols>) ]
    [ COMMENT '<table_comment>' ]
    [ PARTITION BY (
        { <partition_col>
            | DATE_TRUNC(<partition_col>, <partition_unit>) }
                    )]
    [ DISTRIBUTED BY { HASH (<distribute_cols>) | RANDOM }
        [ BUCKETS { <bucket_count> | AUTO } ]
    ]
    [ PROPERTIES (
          -- Table property
          <table_property>
          -- Additional table properties
          [ , ... ])
    ]
    AS <query>
```

刷新配置

build_mode 刷新时机

物化视图创建完成是否立即刷新。- IMMEDIATE：立即刷新，默认方式。- DEFERRED：延迟刷新。

refresh_method 刷新方式

- COMPLETE：刷新所有分区。
- AUTO：尽量增量刷新，只刷新自上次物化刷新后数据变化的分区，如果不能感知数据变化的分区，只能退化成全量刷新，刷新所有分区。

refresh_trigger 触发方式

- ON MANUAL 手动触发

用户通过 SQL 语句触发物化视图的刷新，策略如下

检测基表的分区数据自上次刷新后是否有变化，刷新数据变化的分区。

```
REFRESH MATERIALIZED VIEW mvName AUTO;
```

> 提示如果物化视图定义 SQL 使用的基表是 JDBC 表，Doris 无法感知表数据变化，刷新物化视图时需要指定 COMPLETE。如果指定了 AUTO，会导致基表有数据，但是刷新后物化视图没数据。刷新物化视图时，目前 Doris 只能感知内表和 Hive 数据源表数据变化，其他数据源逐步支持中。

不校验基表的分区数据自上次刷新后是否有变化，直接刷新物化视图的所有分区。

```
REFRESH MATERIALIZED VIEW mvName COMPLETE;
```

只刷新指定的分区。

```
REFRESH MATERIALIZED VIEW mvName partitions(partitionName1,partitionName2);
```

> 提示 partitionName 可以通过 SHOW PARTITIONS FROM mvName 获取。从 2.1.3 版本开始支持 Hive 检测基表的分区数据自上次刷新后是否有变化，其他外表暂时还不支持。内表一直支持。

- ON SCHEDULE 定时触发

通过物化视图的创建语句指定间隔多久刷新一次数据，refreshUnit(刷新时间间隔单位) 可以是 minute，hour，day，week 等。

如下，要求全量刷新 (REFRESH COMPLETE)，物化视图每 10 小时刷新一次，并且刷新物化视图的所有分区。

```
CREATE MATERIALIZED VIEW mv_6
REFRESH COMPLETE ON SCHEDULE EVERY 10 hour
AS
SELECT * FROM lineitem;
```

如下，尽量增量刷新 (REFRESH AUTO)，只刷新自上次物化刷新后数据变化的分区，如果不能增量刷新，就刷新所有分区，物化视图每 10 小时刷新一次（从 2.1.3 版本开始能自动计算 Hive 需要刷新的分区）。

```
  CREATE MATERIALIZED VIEW mv_7
  REFRESH AUTO ON SCHEDULE EVERY 10 hour
  PARTITION by(l_shipdate)
  AS
SELECT * FROM lineitem;
```

- ON COMMIT 自动触发

> 提示自 Apache Doris 2.1.4 版本起支持此功能。

基表数据发生变更后，自动触发相关物化视图刷新，刷新的分区范围与“定时触发”一致。

如果物化视图的创建语句如下，那么当基表 lineitem 的 t1 分区数据发生变化时，会自动触发物化视图的对应分区刷新。

```
CREATE MATERIALIZED VIEW mv_8
REFRESH AUTO ON COMMIT
PARTITION by(l_shipdate)
AS
SELECT * FROM lineitem;
```

> 注意如果基表的数据频繁变更，不太适合使用此种触发方式，因为会频繁构建物化刷新任务，消耗过多资源。

详情参考REFRESH MATERIALIZED VIEW

示例如下

建表语句

```
CREATE TABLE IF NOT EXISTS lineitem (
    l_orderkey    integer not null,
    l_partkey     integer not null,
```

```
    l_suppkey     integer not null,
    l_linenumber  integer not null,
    l_quantity    decimalv3(15,2) not null,
    l_extendedprice  decimalv3(15,2) not null,
    l_discount    decimalv3(15,2) not null,
    l_tax         decimalv3(15,2) not null,
    l_returnflag  char(1) not null,
    l_linestatus  char(1) not null,
    l_shipdate    date not null,
    l_commitdate  date not null,
    l_receiptdate date not null,
    l_shipinstruct char(25) not null,
    l_shipmode     char(10) not null,
    l_comment      varchar(44) not null
    )
    DUPLICATE KEY(l_orderkey, l_partkey, l_suppkey, l_linenumber)
    PARTITION BY RANGE(l_shipdate)
    (FROM ('2023-10-17') TO ('2023-11-01') INTERVAL 1 DAY)
    DISTRIBUTED BY HASH(l_orderkey) BUCKETS 3;

INSERT INTO lineitem VALUES
(1, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-17', '2023-10-17', '2023-10-17', 'a', 'b', '
    ↪ yyyyyyyyy'),
(2, 4, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-18', '2023-10-18', '2023-10-18', 'a', 'b', '
    ↪ yyyyyyyyy'),
(3, 2, 4, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-19', '2023-10-19', '2023-10-19', 'a', 'b', '
    ↪ yyyyyyyyy');

CREATE TABLE IF NOT EXISTS orders  (
    o_orderkey       integer not null,
    o_custkey        integer not null,
    o_orderstatus    char(1) not null,
    o_totalprice     decimalv3(15,2) not null,
    o_orderdate      date not null,
    o_orderpriority  char(15) not null,
    o_clerk          char(15) not null,
    o_shippriority   integer not null,
    o_comment        varchar(79) not null
    )
DUPLICATE KEY(o_orderkey, o_custkey)
PARTITION BY RANGE(o_orderdate)(
FROM ('2023-10-17') TO ('2023-11-01') INTERVAL 1 DAY)
DISTRIBUTED BY HASH(o_orderkey) BUCKETS 3;

INSERT INTO orders VALUES
```

```
(1, 1, 'o', 9.5, '2023-10-17', 'a', 'b', 1, 'yy'),
(1, 1, 'o', 10.5, '2023-10-18', 'a', 'b', 1, 'yy'),
(2, 1, 'o', 11.5, '2023-10-19', 'a', 'b', 1, 'yy'),
(3, 1, 'o', 12.5, '2023-10-19', 'a', 'b', 1, 'yy');

CREATE TABLE IF NOT EXISTS partsupp (
      ps_partkey     INTEGER NOT NULL,
      ps_suppkey     INTEGER NOT NULL,
      ps_availqty    INTEGER NOT NULL,
      ps_supplycost  DECIMALV3(15,2)  NOT NULL,
      ps_comment     VARCHAR(199) NOT NULL
    )
DUPLICATE KEY(ps_partkey, ps_suppkey)
DISTRIBUTED BY HASH(ps_partkey) BUCKETS 3;

INSERT INTO partsupp VALUES
(2, 3, 9, 10.01, 'supply1'),
(4, 3, 10, 11.01, 'supply2'),
(2, 3, 10, 11.01, 'supply3');
```

刷新机制示例一

如下，刷新时机是创建完立即刷新 BUILD IMMEDIATE，刷新方式尽量增量刷新 REFRESH AUTO，只刷新自上次物化刷新后数据变化的分区，如果不能增量刷新，就刷新所有分区。触发方式是手动 ON MANUAL。对于非分区全量物化视图，只有一个分区，如果基表数据发生变化，意味着要全量刷新。

```
CREATE MATERIALIZED VIEW mv_1_0
BUILD IMMEDIATE
REFRESH AUTO
ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  to_date(o_orderdate) as date_alias,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

刷新机制示例二

如下，刷新时机是延迟刷新 BUILD DEFERRED，刷新方式是全量刷新 REFRESH COMPLETE，触发时机是定时刷新 ON SCHEDULE，首次刷新时间是 2024-12-01 20:30:00, 并且每隔一天刷新一次。如果 BUILD DEFERRED 指定为 BUILD IMMEDIATE，创建完物化视图会立即刷新一次。之后从 2024-12-01 20:30:00 每隔一天刷新一次。

提示 STARTS 的时间要晚于当前的时间

```
CREATE MATERIALIZED VIEW mv_1_1
BUILD DEFERRED
REFRESH COMPLETE
ON SCHEDULE EVERY 1 DAY STARTS '2024-12-01 20:30:00'
PROPERTIES ('replication_num' = '1')
AS
SELECT
l_linestatus,
to_date(o_orderdate) as date_alias,
o_shippriority
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

刷新机制示例三

如下，刷新时机是创建完立即刷新 BUILD IMMEDIATE，刷新方式是全量刷新 REFRESH COMPLETE，触发方式是触发刷新 ON COMMIT，当 orders 或者 lineitem 表数据发生变化的时候，会自动触发物化视图的刷新。

```
CREATE MATERIALIZED VIEW mv_1_1
BUILD IMMEDIATE
REFRESH COMPLETE
ON COMMIT
PROPERTIES ('replication_num' = '1')
AS
SELECT
l_linestatus,
to_date(o_orderdate) as date_alias,
o_shippriority
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

分区配置

如下，创建分区物化视图时，需要指定 PARTITION BY，对于分区字段引用的表达式，仅允许使用 date_trunc 函数和标识符。以下语句是符合要求的：分区字段引用的列仅使用了 date_trunc 函数。分区物化视图的刷新方式一般是 AUTO，即尽量增量刷新，只刷新自上次物化刷新后数据变化的分区，如果不能增量刷新，就刷新所有分区。

```
CREATE MATERIALIZED VIEW mv_2_0
BUILD IMMEDIATE
```

```
REFRESH AUTO
ON MANUAL
PARTITION BY (order_date_month)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  date_trunc(o_orderdate, 'month') as order_date_month,
  o_shippriority
FROM
  orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

如下语句创建分区物化视图会失败，因为分区字段 order_date_month 使用了 date_add() 函数，报错 because column to check use invalid implicit expression, invalid expression is days_add(o_orderdate#4, 2)。

```
CREATE MATERIALIZED VIEW mv_2_1 BUILD IMMEDIATE REFRESH AUTO ON MANUAL
PARTITION BY (order_date_month)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  date_trunc(date_add(o_orderdate, INTERVAL 2 DAY), 'month') as order_date_month,
  o_shippriority
FROM
  orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

基表有多列分区

目前仅支持 Hive 外表有多列分区。Hive 外表有很多多级分区的情况，例如一级分区按照日期，二级分区按照区域。物化视图可以选择 Hive 的某一级分区列作为物化视图的分区列。

例如，Hive 的建表语句如下：

```
CREATE TABLE hive1 (
`k1` int)
PARTITIONED BY (
`year` int,
`region` string)
STORED AS ORC;

alter table hive1 add if not exists
partition(year=2020,region="bj")
partition(year=2020,region="sh")
partition(year=2021,region="bj")
```

```
partition(year=2021,region="sh")
partition(year=2022,region="bj")
partition(year=2022,region="sh")
```

当物化视图的创建语句如下时，物化视图mv_hive将有三个分区：('2020')，('2021')，('2022')

```
CREATE MATERIALIZED VIEW mv_hive
BUILD DEFERRED REFRESH AUTO ON MANUAL
partition by(`year`)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT k1,year,region FROM hive1;
```

当物化视图的建表语句如下时，那么物化视图mv_hive2将有如下两个分区：('bj')，('sh')：

```
CREATE MATERIALIZED VIEW mv_hive2
BUILD DEFERRED REFRESH AUTO ON MANUAL
partition by(`region`)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT k1,year,region FROM hive1;
```

使用基表部分分区

有些基表有很多分区，但是物化视图只关注最近一段时间的“热”数据，那么可以使用此功能。

基表的建表语句如下：

```
CREATE TABLE t1 (
    `k1` INT,
    `k2` DATE NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`k1`)
COMMENT 'OLAP'
PARTITION BY range(`k2`)
(
PARTITION p26 VALUES [("2024-03-26"),("2024-03-27")),
PARTITION p27 VALUES [("2024-03-27"),("2024-03-28")),
PARTITION p28 VALUES [("2024-03-28"),("2024-03-29"))
)
DISTRIBUTED BY HASH(`k1`) BUCKETS 2;
```

物化视图的创建语句如以下，代表物化视图只关注最近一天的数据。若当前时间为 2024-03-28 xx:xx:xx，这样物化视图会仅有一个分区 [("2024-03-28"),("2024-03-29")]：

```
CREATE MATERIALIZED VIEW mv1
BUILD DEFERRED REFRESH AUTO ON MANUAL
partition by(`k2`)
```

```
DISTRIBUTED BY RANDOM BUCKETS 2
PROPERTIES (
'partition_sync_limit'='1',
'partition_sync_time_unit'='DAY'
)
AS
SELECT * FROM t1;
```

若时间又过了一天，当前时间为2024-03-29 xx:xx:xx，t1则会新增一个分区 [("2024-03-29"),("2024-03-30")] ↪ ，若此时刷新物化视图，刷新完成后，物化视图会仅有一个分区 [("2024-03-29"),("2024-03-30")]。

此外，分区字段是字符串类型时，可以设置物化视图属性 partition_date_format，例如 %Y-%m-%d 。

分区上卷

> 提示自 Doris 2.1.5 版本起支持 Range 分区

当基表数据经过聚合处理后，各分区的数据量可能会显著减少。在这种情况下，可以采用分区上卷策略，以降低物化视图的分区数量。

假设基表的建表语句如下：

```
CREATE TABLE `t1` (
`k1` LARGEINT NOT NULL,
`k2` DATE NOT NULL
) ENGINE=OLAP
DUPLICATE KEY(`k1`)
COMMENT 'OLAP'
PARTITION BY range(`k2`)
(
PARTITION p_20200101 VALUES [("2020-01-01"),("2020-01-02")),
PARTITION p_20200102 VALUES [("2020-01-02"),("2020-01-03")),
PARTITION p_20200201 VALUES [("2020-02-01"),("2020-02-02"))
)
DISTRIBUTED BY HASH(`k1`) BUCKETS 2;
```

若物化视图的创建语句如下，则该物化视图将包含两个分区：[("2020-01-01","2020-02-01")] ↪ 和[("2020-02-01","2020-03-01")]

```
CREATE MATERIALIZED VIEW mv_3
BUILD DEFERRED REFRESH AUTO ON MANUAL
partition by (date_trunc(`k2`,'month'))
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT * FROM t1;
```

若物化视图的创建语句如下，则该物化视图将只包含一个分区：[("2020-01-01","2021-01-01")]

```
    CREATE MATERIALIZED VIEW mv_4
    BUILD DEFERRED REFRESH AUTO ON MANUAL
    partition by (date_trunc(`k2`,'year'))
    DISTRIBUTED BY RANDOM BUCKETS 2
    AS
    SELECT * FROM t1;
```

此外，如果分区字段为字符串类型，可以通过设置物化视图的 partition_date_format 属性来指定日期格式，例如 '%Y-%m-%d'。

详情参考CREATE ASYNC MATERIALIZED VIEW

SQL 定义

异步物化视图 SQL 定义没有限制。

直查物化视图

物化视图可以看作是表，可以对物化视图添加过滤条件和聚合等，进行直接查询。

物化视图的定义：

```
CREATE MATERIALIZED VIEW mv_5
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT t1.l_linenumber,
       o_custkey,
       o_orderdate
FROM (SELECT * FROM lineitem WHERE l_linenumber > 1) t1
LEFT OUTER JOIN orders
ON l_orderkey = o_orderkey;
```

原查询如下

```
SELECT t1.l_linenumber,
       o_custkey,
       o_orderdate
FROM (SELECT * FROM lineitem WHERE l_linenumber > 1) t1
LEFT OUTER JOIN orders
ON l_orderkey = o_orderkey
WHERE o_orderdate = '2023-10-18';
```

等价的直查物化语句如下，用户需要手动修改查询

```
SELECT
l_linenumber,
o_custkey
FROM mv_5
```

```
WHERE l_linenumber > 1 and o_orderdate = '2023-10-18';
```

查询透明改写

透明改写指在处理查询时，用户无需手动修改查询，系统会自动优化并改写查询。Doris 异步物化视图采用基于 SPJG（SELECT-PROJECT-JOIN-GROUP-BY）模式的透明改写算法。该算法能够分析 SQL 的结构信息，自动寻找合适的物化视图进行透明改写，并选择最优的物化视图来响应查询 SQL。Doris 提供了丰富且全面的透明改写能力。例如下面这些能力：

条件补偿

查询和物化视图的条件不必完全相同，通过在物化视图上补偿条件来表达查询，可以最大限度地复用物化视图，不用重复构建物化视图。

当物化视图和查询的 where 条件是通过 and 连接的表达式时：

1. 当查询的表达式包含物化视图的表达式时：

可以进行条件补偿。

例如，查询是 `a > 5 and b > 10 and c = 7`，物化的条件是 `a > 5 and b > 10`，物化视图的条件是查询条件的子集，那么只需补偿 `c = 7` 条件即可。

2. 当查询的表达式不完全包含物化视图的表达式时：

查询的条件可以推导出物化视图的条件时（常见的是比较和范围表达式，如 >、<、=、in 等），也可以进行条件补偿。补偿结果就是查询条件本身。

例如，查询是 `a > 5 and b = 10`，物化视图是 `a > 1 and b > 8`，可见物化的条件包含了查询的条件，查询的条件可以推导出物化视图的条件，这样也可以进行补偿，补偿结果就是 `a > 5 and b = 10`。

条件补偿使用限制：

1. 对于通过 `or` 连接的表达式，不能进行条件补偿，必须一样才可以改写成功。

2. 对于 `like` 这种非比较和范围表达式，不能进行条件补偿，必须一样才可以改写成功。

例如

物化视图定义：

```
CREATE MATERIALIZED VIEW mv1
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT t1.l_linenumber,
       o_custkey,
       o_orderdate
FROM (SELECT * FROM lineitem WHERE l_linenumber > 1) t1
LEFT OUTER JOIN orders
ON l_orderkey = o_orderkey;
```

如下查询都可以命中物化视图，多个查询通过透明改写可以复用一个物化视图，减少查询改写时间，节省物化视图构建成本。

```
    SELECT l_linenumber,
           o_custkey,
           o_orderdate
    FROM lineitem
    LEFT OUTER JOIN orders
    ON l_orderkey = o_orderkey
    WHERE l_linenumber > 2;
```

```
    SELECT l_linenumber,
           o_custkey,
           o_orderdate
    FROM lineitem
    LEFT OUTER JOIN orders
    ON l_orderkey = o_orderkey
    WHERE l_linenumber > 2 and o_orderdate = '2023-10-19';
```

JOIN 改写

JOIN 改写指的是查询和物化使用的表相同，可以在物化视图和查询的 JOIN 输入或者 JOIN 的外层写 where，优化器对此模式的查询会尝试进行透明改写。

支持多表 JOIN，支持的 JOIN 类型为：

- INNER JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN
- LEFT SEMI JOIN
- RIGHT SEMI JOIN
- LEFT ANTI JOIN
- RIGHT ANTI JOIN

例如：

物化视图定义：

```
CREATE MATERIALIZED VIEW mv2
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT t1.l_linenumber,
```

```
       o_custkey,
       o_orderdate
FROM (SELECT * FROM lineitem WHERE l_linenumber > 1) t1
LEFT OUTER JOIN orders
ON l_orderkey = o_orderkey;
```

如下查询可进行透明改写，条件 `l_linenumber > 1` 可以上拉，从而进行透明改写，使用物化视图的预计算结果来表达查询。命中物化视图后，可以节省 join 计算。

查询语句：

```
SELECT l_linenumber,
       o_custkey
FROM lineitem
LEFT OUTER JOIN orders
ON l_orderkey = o_orderkey
WHERE l_linenumber > 1 and o_orderdate = '2023-10-18';
```

JOIN 衍生

当查询和物化视图的 JOIN 类型不一致时，如果物化视图能够提供查询所需的所有数据，那么通过在 JOIN 的外部补偿谓词，也可以进行透明改写。

例如

物化视图定义：

```
CREATE MATERIALIZED VIEW mv3
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT
    l_shipdate, l_suppkey, o_orderdate,
    sum(o_totalprice) AS sum_total,
    max(o_totalprice) AS max_total,
    min(o_totalprice) AS min_total,
    count(*) AS count_all,
    count(distinct CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey ELSE null
        ↪  END) AS bitmap_union_basic
FROM lineitem
LEFT OUTER JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
GROUP BY
l_shipdate,
l_suppkey,
o_orderdate;
```

查询语句：

```
SELECT
```

```
    l_shipdate, l_suppkey, o_orderdate,
    sum(o_totalprice) AS sum_total,
    max(o_totalprice) AS max_total,
    min(o_totalprice) AS min_total,
    count(*) AS count_all,
    count(distinct CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey ELSE null
        ↪  END) AS bitmap_union_basic
FROM lineitem
INNER JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
WHERE o_orderdate = '2023-10-18' AND l_suppkey = 3
GROUP BY
l_shipdate,
l_suppkey,
o_orderdate;
```

聚合改写

当查询和物化视图定义中的 group 维度一致时，如果物化视图使用的 group by 维度和查询的 group by 维度相同，并且查询使用的聚合函数可以使用物化视图的聚合函数来表示，那么可以进行透明改写。

例如

物化视图定义

```
CREATE MATERIALIZED VIEW mv4
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT
    o_shippriority, o_comment,
    count(distinct CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey ELSE null
        ↪  END) AS cnt_1,
    count(distinct CASE WHEN O_SHIPPRIORITY > 2 AND o_orderkey IN (2) THEN o_custkey ELSE null
        ↪ END) AS cnt_2,
    sum(o_totalprice),
    max(o_totalprice),
    min(o_totalprice),
    count(*)
FROM orders
GROUP BY
o_shippriority,
o_comment;
```

如下查询可以进行透明改写，因为查询和物化视图使用的聚合维度一致，可以使用物化视图 o_shippriority 字段进行过滤结果。查询中的 group by 维度和聚合函数可以使用物化视图的 group by 维度和聚合函数来改写。命中聚合物化视图后，可以减少聚合计算。

查询语句：

```
SELECT
    o_shippriority, o_comment,
    count(distinct CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey ELSE null
        ↪  END) AS cnt_1,
    count(distinct CASE WHEN O_SHIPPRIORITY > 2 AND o_orderkey IN (2) THEN o_custkey ELSE null
        ↪ END) AS cnt_2,
    sum(o_totalprice),
    max(o_totalprice),
    min(o_totalprice),
    count(*)
FROM orders
WHERE o_shippriority in (1, 2)
GROUP BY
o_shippriority,
o_comment;
```

聚合改写（上卷）

在查询和物化视图定义中，即使聚合的维度不一致，也可以进行改写。物化视图使用的 group by 维度需要包含查询的 group by 维度，而查询可以没有 group by。并且，查询使用的聚合函数可以用物化视图的聚合函数来表示。

例如

物化视图定义：

```
CREATE MATERIALIZED VIEW mv5
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
SELECT
    l_shipdate, o_orderdate, l_partkey, l_suppkey,
    sum(o_totalprice) AS sum_total,
    max(o_totalprice) AS max_total,
    min(o_totalprice) AS min_total,
    count(*) AS count_all,
    bitmap_union(to_bitmap(CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey
        ↪ ELSE null END)) AS bitmap_union_basic
FROM lineitem
LEFT OUTER JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
GROUP BY
l_shipdate,
o_orderdate,
l_partkey,
l_suppkey;
```

以下查询可以进行透明改写。查询和物化视图使用的聚合维度不一致，但物化视图使用的维度包含了查询的维度。查询可以使用维度中的字段对结果进行过滤。查询会尝试使用物化视图 SELECT 后的函数进行上卷，例如，物化视图的 bitmap_union 最后会上卷成 bitmap_union_count，这和查询中的 count(distinct) 的语义保持一致。

通过聚合上卷，同一个物化视图可以被多个查询复用，节省物化视图构建成本。

查询语句：

```
SELECT
    l_shipdate, l_suppkey,
    sum(o_totalprice) AS sum_total,
    max(o_totalprice) AS max_total,
    min(o_totalprice) AS min_total,
    count(*) AS count_all,
    count(distinct CASE WHEN o_shippriority > 1 AND o_orderkey IN (1, 3) THEN o_custkey ELSE null
        ↪  END) AS bitmap_union_basic
FROM lineitem
LEFT OUTER JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
WHERE o_orderdate = '2023-10-18' AND l_partkey = 3
GROUP BY
l_shipdate,
l_suppkey;
```

目前支持的聚合上卷函数列表如下：

查询中函数	物化视图中函数	函数上卷后
max	max	max
min	min	min
sum	sum	sum
count	count	sum
count(distinct)	bitmap_union	bitmap_union_count
bitmap_union	bitmap_union	bitmap_union
bitmap_union_count	bitmap_union	bitmap_union_count
hll_union_agg, approx_count_distinct, hll_cardinality	hll_union 或者 hll_raw_agg	hll_union_agg
any_value	any_value 或者 select 后有 any_value 使用的列	any_value

多维聚合改写

支持多维聚合的透明改写，即如果物化视图中没有使用 GROUPING SETS, CUBE, ROLLUP，而查询中有多维聚合，并且物化视图 group by 后的字段包含查询中多维聚合的所有字段，那么也可以进行透明改写。

例如

物化视图定义：

```
CREATE MATERIALIZED VIEW mv5_1
```

```
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 hour
DISTRIBUTED BY RANDOM BUCKETS 3
AS
select o_orderstatus, o_orderdate, o_orderpriority,
       sum(o_totalprice) as sum_total,
       max(o_totalprice) as max_total,
       min(o_totalprice) as min_total,
       count(*) as count_all
from orders
group by
o_orderstatus, o_orderdate, o_orderpriority;
```

如下查询可以命中物化视图，复用了物化视图的聚合结果，节省计算

查询语句：

```
select o_orderstatus, o_orderdate, o_orderpriority,
       sum(o_totalprice),
       max(o_totalprice),
       min(o_totalprice),
       count(*)
from orders
group by
GROUPING SETS ((o_orderstatus, o_orderdate), (o_orderpriority), (o_orderstatus), ());
```

分区补偿改写

当分区物化视图不足以提供查询的所有数据时，可以使用 `union all` 的方式，将查询原表和物化视图的数据 `union all` 作为最终返回结果。

例如

物化视图定义：

```
CREATE MATERIALIZED VIEW mv7
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(l_shipdate)
DISTRIBUTED BY RANDOM BUCKETS 2
as
select l_shipdate, o_orderdate, l_partkey,
       l_suppkey, sum(o_totalprice) as sum_total
from lineitem
left join orders on lineitem.l_orderkey = orders.o_orderkey and l_shipdate = o_orderdate
group by
    l_shipdate,
    o_orderdate,
    l_partkey,
    l_suppkey;
```

当基表新增分区 2023-10-21 时，并且物化视图还未刷新时，可以通过物化视图 union all 原表的方式返回结果。

```
insert into lineitem values
(1, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-21', '2023-10-21', '2023-10-21', 'a', 'b', '
    ↪ yyyyyyyyy');
```

查询语句：

```
select l_shipdate, o_orderdate, l_partkey, l_suppkey, sum(o_totalprice) as sum_total
from lineitem
left join orders on lineitem.l_orderkey = orders.o_orderkey and l_shipdate = o_orderdate
group by
    l_shipdate,
    o_orderdate,
    l_partkey,
    l_suppkey;
```

查询可以部分使用物化预计算的结果，节省了这部分的计算。

改写结果示意：

```
SELECT *
FROM mv7
union all
select t1.l_shipdate, o_orderdate, t1.l_partkey, t1.l_suppkey, sum(o_totalprice) as sum_total
from (select * from lineitem where l_shipdate = '2023-10-21') t1
left join orders on t1.l_orderkey = orders.o_orderkey and t1.l_shipdate = o_orderdate
group by
    t1.l_shipdate,
    o_orderdate,
    t1.l_partkey,
    t1.l_suppkey;
```

注意目前支持分区补偿，暂时不支持带条件的 UNION ALL 补偿。

比如，如果物化视图带有 where 条件，以上述为例，如果构建物化的过滤条件加上 WHERE l_shipdate > '2023-10-19'，而查询是 WHERE l_shipdate > '2023-10-18'，目前这种还无法通过 UNION ALL 补偿，待支持。

备注从 3.1.0 版本开始分区补偿改写功能支持以下类型的分区表：内表、Hive、Iceberg 与 Paimon。这意味着，仅当分区物化视图是基于上述类型的分区表构建时，才能触发分区补偿改写机制。

嵌套物化视图改写

物化视图的定义 SQL 可以使用物化视图，此物化视图称为嵌套物化视图。嵌套的层数理论上没有限制，此物化视图既可以直查，也可以进行透明改写。嵌套物化视图同样可以参与透明改写。

例如

创建内层物化视图 mv8_0_inner_mv：

```
CREATE MATERIALIZED VIEW mv8_0_inner_mv
BUILD IMMEDIATE REFRESH COMPLETE ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
select
l_linenumber,
o_custkey,
o_orderkey,
o_orderstatus,
l_partkey,
l_suppkey,
l_orderkey
from lineitem
inner join orders on lineitem.l_orderkey = orders.o_orderkey;
```

创建外层物化视图 mv8_0：

```
CREATE MATERIALIZED VIEW mv8_0
BUILD IMMEDIATE REFRESH COMPLETE ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
select
l_linenumber,
o_custkey,
o_orderkey,
o_orderstatus,
l_partkey,
l_suppkey,
l_orderkey,
ps_availqty
from mv8_0_inner_mv
inner join partsupp on l_partkey = ps_partkey AND l_suppkey = ps_suppkey;
```

对于以下查询，mv8_0_inner_mv 和 mv8_0 都会成功进行改写，最终代价模型会选择 mv8_0。

嵌套物化视图常用于数据建模和特别复杂的查询，如果单独构建一个物化视图无法透明改写，可以将复杂的查询拆分，构建嵌套物化视图，透明改写会尝试使用嵌套物化视图改写，如果改写成功，会节省计算，提高查询性能。

```
select lineitem.l_linenumber
```

```
from lineitem
inner join orders on l_orderkey = o_orderkey
inner join partsupp on  l_partkey = ps_partkey AND l_suppkey = ps_suppkey
where o_orderstatus = 'o'
```

注意：

1. 嵌套物化视图的层数越多，透明改写的耗时会相应增加。建议嵌套物化视图层数不要超过 3 层。
2. 嵌套物化视图透明改写默认关闭，开启方式见下面的相关设置。

聚合查询使用非聚合物化视图改写

如果查询是聚合查询，物化视图不包含聚合，但是物化视图可以提供查询使用的所有列，那么也可以改写，比如查询先是 join 连接，之后是 group by 聚合，命中包含 join 连接的物化视图，那么也是有收益的。

```
CREATE MATERIALIZED VIEW mv10_0
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
as
select l_shipdate, o_orderdate, l_partkey,
       l_suppkey, o_totalprice
from lineitem
left join orders on lineitem.l_orderkey = orders.o_orderkey and l_shipdate = o_orderdate;
```

如下查询可以命中 mv10_0 的物化视图，节省了 `lineitem join orders` 连接的计算

```
select l_shipdate, o_orderdate, l_partkey,
       l_suppkey, sum(o_totalprice) as sum_total
from lineitem
left join orders on lineitem.l_orderkey = orders.o_orderkey and l_shipdate = o_orderdate
group by
    l_shipdate,
    o_orderdate,
    l_partkey,
    l_suppkey;
```

Explain 查询透明改写情况

查询透明改写命中情况，用于查看和调试。

1. 如果需要查看物化视图的透明改写命中情况，该语句会展示查询透明改写的简要过程信息。

```
   explain <query_sql>
```

返回的信息如下，此处截取了与物化视图相关的信息：

```
| MaterializedView
    ↪
    ↪ |
| MaterializedViewRewriteSuccessAndChose:
    ↪
    ↪ |
|   Names: mv5
    ↪
    ↪ |
| MaterializedViewRewriteSuccessButNotChose:
    ↪
    ↪ |
|
    ↪
    ↪ |
| MaterializedViewRewriteFail:
    ↪
    ↪ |
|   Name: mv4
    ↪
    ↪ |
|   FailSummary: Match mode is invalid, View struct info is invalid
    ↪
    ↪ |
|   Name: mv3
    ↪
    ↪ |
|   FailSummary: Match mode is invalid, Rewrite compensate predicate by view fail, View
    ↪ struct info is invalid
    ↪
    ↪ |
|   Name: mv1
    ↪
    ↪ |
|   FailSummary: The columns used by query are not in view, View struct info is invalid
    ↪
    ↪ |
|   Name: mv2
    ↪
    ↪ |
|   FailSummary: The columns used by query are not in view, View struct info is invalid
```

- MaterializedViewRewriteSuccessAndChose：表示透明改写成功，并且 CBO（Cost-Based Optimizer）选择的物化视图名称列表。

  - MaterializedViewRewriteSuccessButNotChose：表示透明改写成功，但是最终 CBO 没有选择的物化视图名称列表。

- MaterializedViewRewriteFail：列举透明改写失败的情况及原因摘要。

2. 如果想了解物化视图的候选、改写以及最终选择情况的详细过程信息，可以执行如下语句：

```
explain memo plan <query_sql>
```

维护物化视图

权限说明

- 删除物化视图：需要具有物化视图的删除权限（与删除表权限相同）。
- 修改物化视图：需要具有物化视图的修改权限（与修改表权限相同）。
- 暂停/恢复/取消/刷新物化视图：需要具有物化视图的创建权限。

物化视图修改

修改物化视图属性

```
ALTER MATERIALIZED VIEW mv_1
SET(
  "grace_period" = "10"
);
```

物化视图重命名，即物化视图原子替换

```

CREATE MATERIALIZED VIEW mv9_0
BUILD IMMEDIATE REFRESH COMPLETE ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
PROPERTIES ('replication_num' = '1')
AS
select
  l_linenumber,
  o_custkey,
  o_orderkey,
  o_orderstatus,
  l_partkey,
  l_suppkey,
  l_orderkey
from lineitem
inner join orders on lineitem.l_orderkey = orders.o_orderkey;
```

使用 mv9_0 的物化视图替换 mv7，并且删除 mv7。

```
ALTER MATERIALIZED VIEW mv7
REPLACE WITH MATERIALIZED VIEW mv9_0
PROPERTIES('swap' = 'false');
```

物化视图删除

```
DROP MATERIALIZED VIEW mv_1;
```

详情参考DROP ASYNC MATERIALIZED VIEW

查看物化视图创建语句

```
SHOW CREATE MATERIALIZED VIEW mv_1;
```

详情参考SHOW CREATE MATERIALIZED VIEW

暂停物化视图

详情参考PAUSE MATERIALIZED VIEW

启用物化视图

详情参考RESUME MATERIALIZED VIEW

取消物化视图刷新任务

详情参考CANCEL MATERIALIZED VIEW TASK

查询物化视图信息

```
SELECT *
FROM mv_infos('database'='db_name')
WHERE Name = 'mv_name' `\G`
```

返回结果如下：

```
*************************** 1. row ***************************
                Id: 139570
              Name: mv11
           JobName: inner_mtmv_139570
             State: NORMAL
SchemaChangeDetail:
      RefreshState: SUCCESS
       RefreshInfo: BUILD IMMEDIATE REFRESH AUTO ON MANUAL
          QuerySql: SELECT l_shipdate, l_orderkey, O_ORDERDATE, count(*)
FROM lineitem
LEFT OUTER JOIN orders on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_orderkey, O_ORDERDATE
           EnvInfo: EnvInfo{ctlId='0', dbId='16813'}
      MvProperties: {}
   MvPartitionInfo: MTMVPartitionInfo{partitionType=FOLLOW_BASE_TABLE, relatedTable=lineitem,
       ↪ relatedCol='l_shipdate', partitionCol='l_shipdate'}
```

```
SyncWithBaseTables: 1
```

- SyncWithBaseTables： 表示物化视图和基表的数据是否一致。
  - 对于全量构建的物化视图，此字段为 1，表明此物化视图可用于透明改写。
  - 对于分区增量的物化视图，分区物化视图是否可用，是以分区粒度去看的。也就是说，即使物化视图的部分分区不可用，但只要查询的是有效分区，那么此物化视图依旧可用于透明改写。是否能透明改写，主要看查询所用分区的 SyncWithBaseTables 字段是否一致。如果 SyncWithBaseTables 是 1，此分区可用于透明改写；如果是 0，则不能用于透明改写。
- JobName： 物化视图构建 Job 的名称，每个物化视图有一个 Job，每次刷新会有一个新的 Task，Job 和 Task 是 1:n 的关系
- State： 如果变为 SCHEMA_CHANGE，代表基表的 Schema 发生了变化，此时物化视图将不能用来透明改写(但是不影响直接查询物化视图)，下次刷新任务如果执行成功，将恢复为 NORMAL。
- SchemaChangeDetail： 表示 SCHEMA_CHANGE 发生的原因。
- RefreshState： 物化视图最后一次任务刷新的状态。如果为 FAIL，代表执行失败，可以通过 tasks()命令进一步定位失败原因。Task 命令见本文查询刷新任务 TASK 信息。
- SyncWithBaseTables：是否和基表数据同步。1 为同步，0 为不同步。如果不同步，可通过 show partitions 进一步判断哪个分区不同步。 show partitions 见下文分区物化视图查看 SyncWithBaseTables 状态方法。

对于透明改写，通常物化视图会出现两种状态：

- 状态正常： 指的是当前物化视图是否可用于透明改写。
- 不可用、状态不正常： 指的是物化视图不能用于透明改写的简称。尽管如此，该物化视图还是可以直查的。

详情参考 MV_INFOS

查询刷新任务 TASK 信息

每个物化视图有一个 Job，每次刷新会有一个新的 Task，Job 和 Task 是 1:n 的关系。根据物化视图名称查看物化视图的 Task 状态，运行如下语句，可以查看刷新任务的状态和进度：

```
SELECT *
FROM tasks("type"="mv")
WHERE
MvDatabaseName = 'mv_db_name' and
mvName = 'mv_name'
ORDER BY  CreateTime DESC `\G`
```

返回结果如下：

```
*************************** 1. row ***************************
               TaskId: 167019363907545
                JobId: 139872
              JobName: inner_mtmv_139570
                MvId: 139570
              MvName: mv11
        MvDatabaseId: 16813
      MvDatabaseName: regression_test_nereids_rules_p0_mv
              Status: SUCCESS
            ErrorMsg:
          CreateTime: 2024-06-21 10:31:43
           StartTime: 2024-06-21 10:31:43
          FinishTime: 2024-06-21 10:31:45
          DurationMs: 2466
         TaskContext: {"triggerMode":"SYSTEM","isComplete":false}
         RefreshMode: COMPLETE
NeedRefreshPartitions: ["p_20231023_20231024","p_20231019_20231020","p_20231020_20231021","p_
    ↪ 20231027_20231028","p_20231030_20231031","p_20231018_20231019","p_20231024_20231025","p_
    ↪ 20231021_20231022","p_20231029_20231030","p_20231028_20231029","p_20231025_20231026","p_
    ↪ 20231022_20231023","p_20231031_20231101","p_20231016_20231017","p_20231026_20231027"]
  CompletedPartitions: ["p_20231023_20231024","p_20231019_20231020","p_20231020_20231021","p_
      ↪ 20231027_20231028","p_20231030_20231031","p_20231018_20231019","p_20231024_20231025","p
      ↪ _20231021_20231022","p_20231029_20231030","p_20231028_20231029","p_20231025_20231026","
      ↪ p_20231022_20231023","p_20231031_20231101","p_20231016_20231017","p_20231026_20231027"]
            Progress: 100.00% (15/15)
         LastQueryId: fe700ca3d6504521-bb522fc9ccf615e3
```

- NeedRefreshPartitions，CompletedPartitions 记录的是此次 Task 刷新的分区。
- Status：如果为 FAILED，代表运行失败，可通过 ErrorMsg 查看失败原因，也可通过 LastQueryId 来搜索 Doris 的日志，获取更详细的错误信息。目前任务失败会导致已有物化视图不可用，后面会改成尽管任务失败，但是已存在的物化视图可用于透明改写。
- ErrorMsg：失败原因。
- RefreshMode：COMPLETE 代表刷新了全部分区，PARTIAL 代表刷新了部分分区，NOT_REFRESH 代表不需要刷新任何分区。

备注 - 目前 task 存储和展示的数量默认是 100 个，可以通过在 fe.conf 文件中配置 max_persistence_task_count 修改数量，超过这个数量将会丢弃旧的 task 记录, 如果值 < 1, 将不会持久化。修改完配置后需要重启 FE 才能生效。

- 如果物化视图创建的时候设置了 grace_period 属性，那么即使 SyncWithBaseTables 是 false 或者 0，有些情况下它依然可用于透明改写。

- grace_period 的单位是秒，指的是容许物化视图和所用基表数据不一致的时间。
- 如果设置成 0，意味着要求物化视图和基表数据保持一致，此物化视图才可用于透明改写。
- 如果设置成 10，意味着物化视图和基表数据允许 10 秒的延迟，如果物化视图的数据和基表的数据有延迟，在 10 秒内，此物化视图都可以用于透明改写。

详情参考TASKS

查询物化视图对应的 JOB

```
SELECT *
FROM jobs("type"="mv")
WHERE Name="inner_mtmv_75043";
```

详情参考JOBS

查询物化视图的分区信息：

分区物化视图查看 SyncWithBaseTables 状态方法

运行 show partitions from mv_name查看查询使用的分区是否有效，返回结果如下：

```
show partitions from mv11;
+-------------+--------------------+---------------+--------------------+--------+-------------+-------------
    ↪
| PartitionId | PartitionName       | VisibleVersion | VisibleVersionTime  | State  |
    ↪ PartitionKey | Range
    ↪                                                                                  |
    ↪ DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize  | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+-------------+--------------------+---------------+--------------------+--------+-------------+-------------
    ↪
| 140189      | p_20231016_20231017 | 1              | 2024-06-21 10:31:45 | NORMAL | l_shipdate
    ↪   | [types: [DATEV2]; keys: [2023-10-16]; ..types: [DATEV2]; keys: [2023-10-17]; ) | l_
    ↪ orderkey      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
    ↪                     | NULL                    | 0.000     | false      | tag.location.
    ↪ default: 1 | true      | true               | []           |
| 139995      | p_20231018_20231019 | 2              | 2024-06-21 10:31:44 | NORMAL | l_shipdate
    ↪   | [types: [DATEV2]; keys: [2023-10-18]; ..types: [DATEV2]; keys: [2023-10-19]; ) | l_
    ↪ orderkey      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
    ↪                     | NULL                    | 880.000 B | false      | tag.location.
    ↪ default: 1 | true      | true               | []           |
| 139898      | p_20231019_20231020 | 2              | 2024-06-21 10:31:43 | NORMAL | l_shipdate
    ↪   | [types: [DATEV2]; keys: [2023-10-19]; ..types: [DATEV2]; keys: [2023-10-20]; ) | l_
    ↪ orderkey      | 10      | 1              | HDD           | 9999-12-31 23:59:59 |
```

```
    ↪                     | NULL                      | 878.000 B | false      | tag.location.
    ↪ default: 1 | true      | true               | []           |
+------------+--------------------+---------------+--------------------+--------+-------------+------------
    ↪
```

主要查看 SyncWithBaseTables 字段是否为 true。false 表示此分区不可用于透明改写。

详情参考SHOW PARTITIONS

查看物化视图表结构

详情参考DESCRIBE

相关配置

Session Variables 开关

开关	说明
SET enable_nereids_planner = true;	异步物化视图只有在新优化器下才支持，所以物化视图透明改写没有生效时，需要开启新优化器
SET enable_materialized_view_rewrite = true;	开启或者关闭查询透明改写，从 2.1.5 版本开始默认开启
SET materialized_view_rewrite_enable_contain_external_table = true;	参与透明改写的物化视图是否允许包含外表，默认不允许，如果物化视图的定义 SQL 中包含外表，也想参与到透明改写，可以打开此开关。
SET materialized_view_rewrite_success_candidate_num = 3;	透明改写成功的结果集合，允许参与到 CBO 候选的最大数量，默认是 3。如果发现透明改写的性能很慢，可以考虑把这个值调小。
SET enable_materialized_view_union_rewrite = true;	当分区物化视图不足以提供查询的全部数据时，是否允许基表和物化视图 union all 来响应查询，默认允许。如果发现命中物化视图时数据错误，可以把此开关关闭。
SET enable_materialized_view_nest_rewrite = true;	是否允许嵌套改写，默认不允许。如果查询 SQL 很复杂，需要构建嵌套物化视图才可以命中，那么需要打开此开关。
SET materialized_view_relation_mapping_max_count = 8;	透明改写过程中，relation mapping 最大允许数量，如果超过，进行截取。relation mapping 通常由表自关联产生，数量一般会是笛卡尔积，比如 3 张表，可能会产生 8 种组合。默认是 8。如果发现透明改写时间很长，可以把这个值调低
SET enable_dml_materialized_view_rewrite = true;	DML 时，是否开启基于结构信息的物化视图透明改写，默认开启
SET enable_dml_materialized_view_rewrite_when_base_table_unawareness = true;	DML 时，当物化视图存在无法实时感知数据的外表时，是否开启基于结构信息的物化视图透明改写，默认关闭

fe.conf 配置

- job_mtmv_task_consumer_thread_num： 此参数控制同时运行的物化视图刷新任务数量，默认是 10，超过这个数量的任务将处于 pending 状态修改这个参数需要重启 FE 才可以生效。

2.13.2.3.3 最佳实践

异步物化视图使用原则

- 时效性考虑： 异步物化视图通常用于对数据时效性要求不高的场景，一般是 T+1 的数据。如果时效性要求高，应考虑使用同步物化视图。
- 加速效果与一致性考虑： 在查询加速场景，创建物化视图时，DBA 应将常见查询 SQL 模式分组，尽量使组之间无重合。SQL 模式组划分越清晰，物化视图构建的质量越高。一个查询可能使用多个物化视图，同时一个物化视图也可能被多个查询使用。构建物化视图需要综合考虑命中物化视图的响应时间（加速效果）、构建成本、数据一致性要求等。
- 物化视图定义与构建成本考虑：
  - 物化视图定义和原查询越接近，查询加速效果越好，但物化的通用性和复用性越差，意味着构建成本越高。
  - 物化视图定义越通用（例如没有 WHERE 条件和更多聚合维度），查询加速效果较低，但物化的通用性和复用性越好，意味着构建成本越低。

> 注意 - 物化视图数量控制： 物化视图并非越多越好。物化视图构建和刷新需要资源。物化视图参与透明改写，CBO 代价模型选择最优物化视图需要时间。理论上，物化视图越多，透明改写的时间越长。
>
> - 定期检查物化视图使用状态： 如果未使用，应及时删除。
> - 基表数据更新频率： 如果物化视图的基表数据频繁更新，可能不太适合使用物化视图，因为这会导致物化视图频繁失效，不能用于透明改写（可直查）。如果需要使用此类物化视图进行透明改写，需要允许查询的数据有一定的时效延迟，并可以设定grace_period。具体见grace_period的适用介绍。

物化视图刷新方式选择原则

当满足以下条件时，建议创建分区物化视图：

- 物化视图的基表数据量很大，并且基表是分区表。
- 物化视图使用的表除了分区表外，其他表不经常变化。
- 物化视图的定义 SQL 和分区字段满足分区推导的要求，即符合分区增量更新的要求。详细要求可参考：CREATE-ASYNC-MATERIALIZED-VIEW
- 物化视图分区数不多，分区过多会导致分区多物化视图构建时间会过长。

当物化视图的部分分区失效时，透明改写可以使用物化视图的有效分区 UNION ALL 基表返回数据。

如果不能构建分区物化视图，可以考虑选择全量刷新的物化视图。

分区物化视图常见使用方式

当物化视图的基表数据量很大，且基表是分区表时，如果物化视图的定义 SQL 和分区字段满足分区推导的要求，此种场景比较适合构建分区物化视图。分区推导的详细要求可参考CREATE-ASYNC-MATERIALIZED-VIEW和异步物化视图 FAQ 构建问题 12。

物化视图的分区是跟随基表的分区映射创建的，一般和基表的分区是 1:1 或者 1:n 的关系。

- 如果基表的分区发生数据变更，如新增分区、删除分区等情况，物化视图对应的分区也会失效。失效的分区不能用于透明改写，但可以直查。透明改写时发现物化视图的分区数据失效，失效的分区会通过联合基表来响应查询。

确认物化视图分区状态的命令详见查看物化视图状态，主要是`show partitions from mv_name`命令。

- 如果物化视图引用的非分区表发生数据变更，会触发物化视图所有分区失效，导致此物化视图不能用于透明改写。需要刷新物化视图所有分区的数据，命令为`REFRESH MATERIALIZED VIEW mv1 AUTO;`。此命令会尝试刷新物化视图所有数据变化的分区。

因此，一般将数据频繁变化的表放在分区物化视图引用的分区表，将不经常变化的维表放在非引用分区表的位置。- 如果物化视图引用的非分区表发生数据变更，非分区表数据只是新增，不涉及修改，创建物化视图的时候可以指定属性 `excluded_trigger_tables = '非分区表名1,非分区表名2'`，这样非分区表的数据变化就不会使物化视图的所有分区失效，下次刷新时，只刷新分区表对应的物化视图失效分区。

分区物化视图的透明改写是分区粒度的，即使物化视图的部分分区失效，此物化视图仍然可用于透明改写。但如果只查询了一个分区，并且物化视图这个分区数据失效了，那么此物化视图不能用于透明改写。

例如：

```
CREATE TABLE IF NOT EXISTS lineitem (
    l_orderkey INTEGER NOT NULL,
    l_partkey INTEGER NOT NULL,
    l_suppkey INTEGER NOT NULL,
    l_linenumber INTEGER NOT NULL,
    l_ordertime DATETIME NOT NULL,
    l_quantity DECIMALV3(15, 2) NOT NULL,
    l_extendedprice DECIMALV3(15, 2) NOT NULL,
    l_discount DECIMALV3(15, 2) NOT NULL,
    l_tax DECIMALV3(15, 2) NOT NULL,
    l_returnflag CHAR(1) NOT NULL,
    l_linestatus CHAR(1) NOT NULL,
    l_shipdate DATE NOT NULL,
    l_commitdate DATE NOT NULL,
    l_receiptdate DATE NOT NULL,
    l_shipinstruct CHAR(25) NOT NULL,
    l_shipmode CHAR(10) NOT NULL,
    l_comment VARCHAR(44) NOT NULL
  ) DUPLICATE KEY(
    l_orderkey, l_partkey, l_suppkey,
    l_linenumber
```

```
  ) PARTITION BY RANGE(l_ordertime) (
    FROM
      ('2024-05-01') TO ('2024-06-30') INTERVAL 1 DAY
  )
DISTRIBUTED BY HASH(l_orderkey) BUCKETS 3;

INSERT INTO lineitem VALUES
(1, 2, 3, 4, '2024-05-01 01:45:05', 5.5, 6.5, 0.1, 8.5, 'o', 'k', '2024-05-01', '2024-05-01', '
    ↪ 2024-05-01', 'a', 'b', 'yyyyyyyyy'),
(1, 2, 3, 4, '2024-05-15 02:35:05', 5.5, 6.5, 0.15, 8.5, 'o', 'k', '2024-05-15', '2024-05-15', '
    ↪ 2024-05-15', 'a', 'b', 'yyyyyyyyy'),
(2, 2, 3, 5, '2024-05-25 08:30:06', 5.5, 6.5, 0.2, 8.5, 'o', 'k', '2024-05-25', '2024-05-25', '
    ↪ 2024-05-25', 'a', 'b', 'yyyyyyyyy'),
(3, 4, 3, 6, '2024-06-02 09:25:07', 5.5, 6.5, 0.3, 8.5, 'o', 'k', '2024-06-02', '2024-06-02', '
    ↪ 2024-06-02', 'a', 'b', 'yyyyyyyyy'),
(4, 4, 3, 7, '2024-06-15 13:20:09', 5.5, 6.5, 0, 8.5, 'o', 'k', '2024-06-15', '2024-06-15', '
    ↪ 2024-06-15', 'a', 'b', 'yyyyyyyyy'),
(5, 5, 6, 8, '2024-06-25 15:15:36', 5.5, 6.5, 0.12, 8.5, 'o', 'k', '2024-06-25', '2024-06-25', '
    ↪ 2024-06-25', 'a', 'b', 'yyyyyyyyy'),
(5, 5, 6, 9, '2024-06-29 21:10:52', 5.5, 6.5, 0.1, 8.5, 'o', 'k', '2024-06-30', '2024-06-30', '
    ↪ 2024-06-30', 'a', 'b', 'yyyyyyyyy'),
(5, 6, 5, 10, '2024-06-03 22:05:50', 7.5, 8.5, 0.1, 10.5, 'k', 'o', '2024-06-03', '2024-06-03', '
    ↪ 2024-06-03', 'c', 'd', 'xxxxxxxxx');

CREATE TABLE IF NOT EXISTS partsupp (
    ps_partkey INTEGER NOT NULL,
    ps_suppkey INTEGER NOT NULL,
    ps_availqty INTEGER NOT NULL,
    ps_supplycost DECIMALV3(15, 2) NOT NULL,
    ps_comment VARCHAR(199) NOT NULL
  )
DUPLICATE KEY(ps_partkey, ps_suppkey)
DISTRIBUTED BY HASH(ps_partkey) BUCKETS 3;


INSERT INTO partsupp VALUES
(2, 3, 9, 10.01, 'supply1'),
(4, 3, 9, 10.01, 'supply2'),
(5, 6, 9, 10.01, 'supply3'),
(6, 5, 10, 11.01, 'supply4');
```

在这个例子中，orders表的o_ordertime字段是分区字段，类型是DATETIME，按照天分区。

查询主要是按照“天”的粒度

```
SELECT
```

```
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  ps_partkey
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey
WHERE
  date_trunc(l_ordertime, 'day') <= DATE '2024-05-25'
  AND date_trunc(l_ordertime, 'day') >= DATE '2024-05-05'
GROUP BY
  l_linestatus,
  ps_partkey;
```

为了不让物化视图每次刷新的分区数量过多，物化视图的分区粒度可以和基表orders一致，也按“天”分区。

物化视图的定义 SQL 的粒度可以按照“天”，并且按照“天”来聚合数据，

```
CREATE MATERIALIZED VIEW rollup_partition_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(order_date)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey
GROUP BY
  l_linestatus,
  ps_partkey,
  date_trunc(l_ordertime, 'day');
```

创建包含 UNION ALL 的分区物化视图

目前 Doris 有限制，分区物化定义中不能包含 UNION ALL 子句。如果想要创建包含 UNION ALL 的物化视图，可以使用采用如下的方式创建。对于 UNION ALL 的每部分输入尝试创建分区物化视图，之后针对整个 UNION ALL 结果集创建一个普通视图。

例如：物化视图定义如下，要使用如下 sql 语句构建分区物化视图，可以看到 sql 语句包含 UNION ALL 子句。

```
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey
GROUP BY
  l_linestatus,
  ps_partkey,
  date_trunc(l_ordertime, 'day')
UNION ALL
SELECT
  l_linestatus,
  l_extendedprice,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey;
```

可以将上述 SQL 语句拆分成两个部分，分别创建两个分区物化视图。

```
CREATE MATERIALIZED VIEW union_sub_mv1
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(order_date)
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey
GROUP BY
```

```
  l_linestatus,
  ps_partkey,
  date_trunc(l_ordertime, 'day');
```

```
CREATE MATERIALIZED VIEW union_sub_mv2
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(order_date)
DISTRIBUTED BY RANDOM BUCKETS 2

AS
SELECT
  l_linestatus,
  l_extendedprice,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey;
```

然后创建一个普通视图，将两个分区物化视图的结果集进行 UNION ALL。对外可以提供这个视图 union_all_view

```
CREATE VIEW union_all_view
AS
SELECT *
FROM
union_sub_mv1
UNION ALL
SELECT *
FROM
union_sub_mv2;
```

分区物化视图只保留最近分区数据

> 提示自 Apache Doris 2.1.1 版本起支持此功能。

物化视图可以只保留最近几个分区的数据，每次刷新时，自动删除过期的分区数据。可以通过设置物化视图的属性 `partition_sync_limit`，`partition_sync_time_unit`，`partition_sync_date_format` 来实现。

partition_sync_limit：基表的分区字段为时间时，可以用此属性配置同步基表的分区范围，配合 partition_sync_time_unit 一起使用。例如设置为 3，partition_sync_time_unit 设置为 DAY，代表仅同步基表近 3 天的分区和数据。

partition_sync_time_unit：分区刷新的时间单位，支持 DAY/MONTH/YEAR（默认 DAY）。

partition_date_format：当基表的分区字段为字符串时，如果想使用 partition_sync_limit 的能力，可以设置日期的格式。

例如：物化视图定义如下，物化视图只保留最近 3 天的数据，如果最近 3 天没有数据，直查如下物化视图，就不会返回数据。

```
CREATE MATERIALIZED VIEW latest_partition_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(order_date)
DISTRIBUTED BY RANDOM BUCKETS 2
PROPERTIES (
  "partition_sync_limit" = "3",
  "partition_sync_time_unit" = "DAY",
  "partition_date_format" = "yyyy-MM-dd"
)
AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  ps_partkey,
  date_trunc(l_ordertime, 'day') as order_date
FROM
  lineitem
  LEFT JOIN partsupp ON l_partkey = ps_partkey
  and l_suppkey = ps_suppkey
GROUP BY
  l_linestatus,
  ps_partkey,
  date_trunc(l_ordertime, 'day');
```

如何使用物化视图加速查询

使用物化视图查询加速，首先需要查看 profile 文件，找到一个查询消耗时间最多的操作，一般出现在连接（Join）、聚合（Aggregate）、过滤（Filter）或者表达式计算（Calculated Expressions）。

对于 Join、Aggregate、Filters、Calculated Expressions，构建物化视图都能起到加速查询的作用。如果一个查询中 Join 占用了大量的计算资源，而 Aggregate 相对而言占用较小的资源，则可以针对 Join 构建物化视图。

接下来，将详细说明如何针对上述四种操作构建物化视图：

1. 对于 Join

可以提取查询中使用的公共的表连接模式来构建物化视图。透明改写如果使用了此物化视图，可以节省 Join 连接的计算。将查询中的 Filters 去除，这样就是一个比较通用的 Join 物化视图。

2. 对于 Aggregate

建议尽量使用低基数的字段作为维度来构建物化视图。如果维度相关，那么聚合后的数量可以尽量减少。

比如表 t1，原表的数据量是 1000000，查询语句 SQL 中有 `group by a, b, c`。如果 a，b，c 的基数分别是 100，50，15，那么聚合后的数据大概在 75000 左右，说明此物化视图是有效的。如果 a，b，c 具有相关性，那么聚合后的数据量会进一步减少。

如果 a, b, c 的基数很高，会导致聚合后的数据急速膨胀。如果聚合后的数据比原表的数据还多，可能这样的场景不太适合构建物化视图。比如 c 的基数是 3500，那么聚合后的数据量在 17000000 左右，比原表数据量大的多，构建这样的物化视图性能加速收益低。

物化视图的聚合粒度要比查询细，即物化视图的聚合维度包含查询的聚合维度，这样才能提供查询所需的数据。查询可以不写 Group By，同理，物化视图的聚合函数应该包含查询的聚合函数。

3. 对于 Filter

如果查询中经常出现对相同字段的过滤，那么通过在物化视图中添加相应的 Filter，可以减少物化视图中的数据量，从而提高查询时命中物化视图的性能。

要注意的是，物化视图应该比查询中出现的 Filter 少，查询的 Filter 要包含物化的 Filter。比如查询是 `a > 10 and b > 5`，物化视图可以没有 Filter，如果有 Filter 的话应对 a 和 b 过滤，并且数据范围要求比查询大，例如物化视图可以是 `a > 5 and b > 5`，也可以是 `a > 5` 等。

4. 对于 Calculated Expressions

以 case when、处理字符串等函数为例，这部分表达式计算非常消耗性能，如果在物化视图中能够提前计算好，透明改写使用计算好的物化视图则可以提高查询的性能。

建议物化视图的列数量尽量不要过多。如果查询使用了多个字段，应该根据最开始的查询 SQL 模式分组，分别构建对应列的物化视图，避免单个物化视图的列过多。

以聚合查询加速为例：

查询 1：

```
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
WHERE
  o_orderdate <= DATE '2024-06-30'
  AND o_orderdate >= DATE '2024-05-01'
GROUP BY
  l_linestatus,
  o_shippriority,
  l_partkey;
```

查询 2：

```
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
WHERE
  o_orderdate <= DATE '2024-06-30'
  AND o_orderdate >= DATE '2024-05-01'
GROUP BY
  l_linestatus,
  o_shippriority,
  l_suppkey;
```

根据以上两个 SQL 查询，我们可以构建一个更为通用的包含 Aggregate 的物化视图。在这个物化视图中，我们将 l_partkey 和 l_suppkey 都作为聚合的 group by 维度，并将 o_orderdate 作为过滤条件。值得注意的是，o_orderdate 不仅在物化视图的条件补偿中使用，同时也需要被包含在物化视图的聚合 group by 维度中。

通过这种方式构建的物化视图后，查询 1 和查询 2 都可以命中该物化视图，物化视图定义如下：

```
CREATE MATERIALIZED VIEW common_agg_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_shippriority,
  l_suppkey,
  l_partkey,
  o_orderdate
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
GROUP BY
  l_linestatus,
  o_shippriority,
  l_suppkey,
  l_partkey,
  o_orderdate;
```

使用场景

场景一：查询加速

在 BI 报表场景或其他加速场景中，用户对于查询响应时间较为敏感，通常要求能够秒级别返回结果。而查询通常涉及多张表先进行 Join 计算、再聚合计算，该过程会消耗大量计算资源，并且有时难以保证时效性。对此，异步物化视图能够很好应对，它不仅支持直接查询，也支持透明改写，优化器会依据改写算法和代价模型，自动选择最优的物化视图来响应请求。

用例 1 多表连接聚合查询加速

通过构建更通用的物化视图能够加速多表连接聚合查询。

以下面三个查询 SQL 为例：

查询 1：

```
SELECT
  l_linestatus,
  l_extendedprice * (1 - l_discount)
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
WHERE
  o_orderdate <= DATE '2024-06-30'
  AND o_orderdate >= DATE '2024-05-01';
```

查询 2：

```
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_orderdate,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
WHERE
  o_orderdate <= DATE '2024-06-30'
  AND o_orderdate >= DATE '2024-05-01'
GROUP BY
  l_linestatus,
  o_orderdate,
  o_shippriority;
```

查询 3：

```
SELECT
  l_linestatus,
  l_extendedprice * (1 - l_discount),
  o_orderdate,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

对于如上查询，可以构建如下物化视图来满足上述所有查询。

物化视图的定义中去除了查询 1 和查询 2 的过滤条件，得到了一个更通用的 Join，并提前计算了表达式`l_extendedprice * (1 - l_discount)`，这样当查询命中物化视图时，可以节省表达式的计算。

```
CREATE MATERIALIZED VIEW common_join_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  l_extendedprice * (1 - l_discount),
  o_orderdate,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

如果上述物化视图不能满足查询 2 的加速性能要求，可以构建聚合物化视图。为了保持通用性，可以去除对`o_orderdate`字段的过滤条件。

```
CREATE MATERIALIZED VIEW target_agg_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 2
AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_orderdate,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
GROUP BY
  l_linestatus,
```

```
  o_orderdate,
  o_shippriority;
```

用例 2 日志查询加速

在日志查询加速场景中，建议不局限于单独使用异步物化视图，可以结合同步物化视图。

一般基表是分区表，按照小时分区居多，单表聚合查询，一般过滤条件是按照时间，还有一些标识位。有时查询的响应速度无法达到要求，一般可以构建异步物化视图进行加速。

例如，基表的定义可能如下：

```
CREATE TABLE IF NOT EXISTS test (
`app_name` VARCHAR(64) NULL COMMENT '标识',
`event_id` VARCHAR(128) NULL COMMENT '标识',
`decision` VARCHAR(32) NULL COMMENT '枚举值',
`time` DATETIME NULL COMMENT '查询时间',
`id` VARCHAR(35) NOT NULL COMMENT 'od',
`code` VARCHAR(64) NULL COMMENT '标识',
`event_type` VARCHAR(32) NULL COMMENT '事件类型'
)
DUPLICATE KEY(app_name, event_id)
PARTITION BY RANGE(time)
(
    FROM ("2024-07-01 00:00:00") TO ("2024-07-15 00:00:00") INTERVAL 1 HOUR
)
DISTRIBUTED BY HASH(event_id)
BUCKETS 3;
```

物化视图可以按照分钟聚合数据，这样也能达到一定的聚合效果。例如：

```
CREATE MATERIALIZED VIEW sync_mv
    AS
    SELECT
      decision,
      code,
      app_name,
      event_id,
      event_type,
      date_trunc(time, 'minute'),
      DATE_FORMAT(
        `time`, '%Y-%m-%d'
      ),
      cast(FLOOR(MINUTE(time) / 15) as decimal(9, 0)),
      count(id) as cnt
    from
      test
    group by
```

```
    code,
    app_name,
    event_id,
    event_type,
    date_trunc(time, 'minute'),
    decision,
    DATE_FORMAT(time, '%Y-%m-%d'),
    cast(FLOOR(MINUTE(`time`) / 15) as decimal(9, 0));
```

查询语句可能如下：

```
SELECT
    decision,
    CONCAT(
        CONCAT(
          DATE_FORMAT(
            `time`, '%Y-%m-%d'
          ),
          '',
          LPAD(
            cast(FLOOR(MINUTE(`time`) / 15) as decimal(9, 0)) * 15,
            5,
            '00'
          ),
          ':00'
        )
      ) as time,
      count(id) as cnt
    from
      test
    where
    date_trunc(time, 'minute') BETWEEN '2024-07-02 18:00:00'
      AND '2024-07-03 20:00:00'
    group by
      decision,
      DATE_FORMAT(
        `time`, "%Y-%m-%d"
      ),
      cast(FLOOR(MINUTE(`time`) / 15) as decimal(9, 0));
```

场景二：数据建模（ETL）

数据分析工作往往需要对多表进行连接和聚合，这一过程通常涉及复杂且频繁重复的查询。这类查询可能引发查询延迟高或资源消耗大的问题。然而，如果采用异步物化视图构建数据分层模型，则可以很好避免该问题，利用创建好的物化视图创建更高层级的物化视图（2.1.3 支持），灵活满足不同的需求。

不同层级的物化视图可以设置各自的触发方式，例如：

- 第一层的物化视图可以设置为定时刷新，第二层的设置为触发刷新。这样，第一层的物化视图刷新完成后，会自动触发第二层物化视图的刷新。
- 如果每层的物化视图都设置为定时刷新，那么第二层物化视图刷新的时候，不会考虑第一层的物化视图数据是否和基表同步，只会把第一层物化视图的数据加工后同步到第二层。

接下来，通过 TPC-H 数据集说明异步物化视图在数据建模中的应用，以分析每月各地区和国家的订单数量和利润为例：

原始查询（未使用物化视图）：

```
SELECT
n_name,
date_trunc(o.o_orderdate, 'month') as month,
count(distinct o.o_orderkey) as order_count,
sum(l.l_extendedprice * (1 - l.l_discount)) as revenue
FROM orders o
JOIN lineitem l ON o.o_orderkey = l.l_orderkey
JOIN customer c ON o.o_custkey = c.c_custkey
JOIN nation n ON c.c_nationkey = n.n_nationkey
JOIN region r ON n.n_regionkey = r.r_regionkey
GROUP BY n_name, month;
```

使用异步物化视图分层建模：

构建 DWD 层（明细数据），处理订单明细宽表

```
CREATE MATERIALIZED VIEW dwd_order_detail
BUILD IMMEDIATE REFRESH AUTO ON COMMIT
DISTRIBUTED BY RANDOM BUCKETS 16
AS
select
o.o_orderkey,
o.o_custkey,
o.o_orderstatus,
o.o_totalprice,
o.o_orderdate,
c.c_name,
c.c_nationkey,
n.n_name as nation_name,
r.r_name as region_name,
l.l_partkey,
l.l_quantity,
l.l_extendedprice,
l.l_discount,
l.l_tax
from orders o
join customer c on o.o_custkey = c.c_custkey
join nation n on c.c_nationkey = n.n_nationkey
```

```
join region r on n.n_regionkey = r.r_regionkey
join lineitem l on o.o_orderkey = l.l_orderkey;
```

构建 DWS 层（汇总数据），进行每日订单汇总

```
CREATE MATERIALIZED VIEW dws_daily_sales
BUILD IMMEDIATE REFRESH AUTO ON COMMIT
DISTRIBUTED BY RANDOM BUCKETS 16
AS
select
date_trunc(o_orderdate, 'month') as month,
nation_name,
region_name,
bitmap_union(to_bitmap(o_orderkey)) as order_count,
sum(l_extendedprice * (1 - l_discount)) as net_revenue
from dwd_order_detail
group by
date_trunc(o_orderdate, 'month'),
nation_name,
region_name;
```

使用物化视图优化查询如下：

```
SELECT
nation_name,
month,
bitmap_union_count(order_count),
sum(net_revenue) as revenue
FROM dws_daily_sales
GROUP BY nation_name, month;
```

场景三：湖仓一体联邦数据查询

在现代化的数据架构中，企业通常会采用湖仓一体设计，以平衡数据的存储成本与查询性能。在这种架构下，经常会遇到两个关键挑战：- 查询性能受限：频繁查询数据湖中的数据时，可能会受到网络延迟和第三方服务的影响，从而导致查询延迟，进而影响用户体验。- 数据分层建模的复杂性：在数据湖到实时数仓的数据流转和转换过程中，通常需要复杂的 ETL 流程，这增加了维护成本和开发难度。

使用 Doris 异步物化视图，可以很好的应对上述挑战：- 透明改写加速查询：将常用的数据湖查询结果物化到 Doris 内部存储，采用透明改写可有效提升查询性能。- 简化分层建模：支持基于数据湖中的表创建物化视图，实现从数据湖到实时数仓的便捷转换，极大简化了数据建模流程。

如下，以 Hive 示例说明：

基于 Hive 创建 Catalog，使用 TPC-H 数据集

```
CREATE CATALOG hive_catalog PROPERTIES (
'type'='hms', -- hive meta store 地址
'hive.metastore.uris' = 'thrift://172.21.0.1:7004'
```

```
);
```

基于 Hive Catalog 创建物化视图

```
-- 物化视图只能在 internal 的 catalog 上创建，切换到内部 catalog
switch internal;
create database hive_mv_db;
use hive_mv_db;

CREATE MATERIALIZED VIEW external_hive_mv
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
DISTRIBUTED BY RANDOM BUCKETS 12
AS
SELECT
n_name,
o_orderdate,
sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM
customer,
orders,
lineitem,
supplier,
nation,
region
WHERE
c_custkey = o_custkey
AND l_orderkey = o_orderkey
AND l_suppkey = s_suppkey
AND c_nationkey = s_nationkey
AND s_nationkey = n_nationkey
AND n_regionkey = r_regionkey
AND r_name = 'ASIA'
GROUP BY
n_name,
o_orderdate;
```

运行如下的查询，通过透明改写自动使用物化视图加速查询。

```
SELECT
n_name,
sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM
customer,
orders,
lineitem,
supplier,
```

```
nation,
region
WHERE
c_custkey = o_custkey
AND l_orderkey = o_orderkey
AND l_suppkey = s_suppkey
AND c_nationkey = s_nationkey
AND s_nationkey = n_nationkey
AND n_regionkey = r_regionkey
AND r_name = 'ASIA'
AND o_orderdate >= DATE '1994-01-01'
AND o_orderdate < DATE '1994-01-01' + INTERVAL '1' YEAR
GROUP BY
n_name
ORDER BY
revenue DESC;
```

> 提示 Doris 暂无法感知除 Hive 外的其他外表数据变更。当外表数据不一致时，使用物化视图可能出现数据不一致的情况。以下开关表示：参与透明改写的物化视图是否允许包含外表，默认 false。如接受数据不一致或者通过定时刷新来保证外表数据一致性，可以将此开关设置成 true。设置包含外表的物化视图是否可用于透明改写，默认不允许，如果可以接受数据不一致或者可以自行保证数据一致，可以开启
>
> ```
> SET materialized_view_rewrite_enable_contain_external_table = true;
> ```
>
> 如果物化视图在 MaterializedViewRewriteSuccessButNotChose 状态，说明改写成功但 plan 未被 CBO 选择，可能是因为外表的统计信息不完整。启用统计信息从文件中获取行数
>
> ```
> SET enable_get_row_count_from_file_list = true;
> ```
>
> 查看外表统计信息，确认是否已收集完整
>
> ```
> SHOW TABLE STATS external_table_name;
> ```

场景四：提升写入效率，减少资源竞争

在高吞吐的数据写入的场景中，系统性能的稳定性与数据处理的高效性同样重要。通过异步物化视图灵活的刷新策略，用户可以根据具体场景选择合适的刷新方式，从而降低写入压力，避免资源争抢。

相比之下，异步物化视图提供了手动触发、触发式、周期性触发三种灵活的刷新策略。用户可以根据场景需求差异，选择合适的刷新策略。当基表数据变更时，不会立即触发物化视图刷新，延迟刷新有利于降低资源压力，有效避免写入资源争抢。

如下所示，选择的刷新方式为定时刷新，每 2 小时刷新一次。当 orders 和 lineitem 导入数据时，不会立即触发物化视图刷新。

```
CREATE MATERIALIZED VIEW common_schedule_join_mv
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 2 HOUR
```

```
DISTRIBUTED BY RANDOM BUCKETS 16
AS
SELECT
l_linestatus,
l_extendedprice * (1 - l_discount),
o_orderdate,
o_shippriority
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

透明改写能够对查询 SQL 的改写，实现了查询加速，同时也能对导入 SQL 进行改写，从而提升导入效率。从 2.1.6 版本开始，当物化视图和基表数据强一致时，可对 DML 操作如 Insert Into 或者 Insert Overwrite 进行透明改写，这对于数据导入场景的性能提升有显著效果。

1. 创建 Insert Into 数据的目标表

```
CREATE TABLE IF NOT EXISTS target_table  (
orderdate      DATE NOT NULL,
shippriority   INTEGER NOT NULL,
linestatus     CHAR(1) NOT NULL,
sale           DECIMALV3(15,2) NOT NULL
)
DUPLICATE KEY(orderdate, shippriority)
DISTRIBUTED BY HASH(shippriority) BUCKETS 3;
```

2. common_schedule_join_mv

```
CREATE MATERIALIZED VIEW common_schedule_join_mv
BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 2 HOUR
DISTRIBUTED BY RANDOM BUCKETS 16
AS
SELECT
l_linestatus,
l_extendedprice * (1 - l_discount),
o_orderdate,
o_shippriority
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

未经改写的导入语句如下：

```
INSERT INTO target_table
SELECT
o_orderdate,
```

```
o_shippriority,
l_linestatus,
l_extendedprice * (1 - l_discount)
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey;
```

经过透明改写后，语句如下：

```
INSERT INTO target_table
SELECT *
FROM common_schedule_join_mv;
```

需要注意的是：如果 DML 操作的是无法感知数据变更的外表，透明改写可能导致基表最新数据无法实时导入目标表。如果用户可以接受数据不一致或能够自行保证数据一致性，可以打开如下开关

DML 时，当物化视图存在无法实时感知数据的外表时，是否开启基于结构信息的物化视图透明改写，默认关闭

```
SET enable_dml_materialized_view_rewrite_when_base_table_unawareness = true;
```

2.13.2.3.4 使用建议

概述

异步物化视图通过预先计算和存储查询结果来提高查询性能，但每次刷新可能带来较大开销。本文档提供异步物化视图的使用建议。物化视图的刷新原理参考：刷新原理

使用场景建议

推荐使用场景

复杂聚合查询

- 场景描述：包含多表连接、复杂聚合函数 (如 SUM、 AVG、 COUNT) 或窗口函数的查询
- 优势：避免每次执行时重新计算复杂逻辑

报表

- 场景描述：需要按固定时间点 (如每日午夜) 生成一致性快照的报表
- 优势：确保所有用户看到相同时间点的数据

计算密集型分析

- 场景描述：包含复杂数学计算或数据转换的分析查询，如客户生命周期价值计算、预测分析模型
- 优势：预先计算结果，减少运行时资源消耗

数据仓库中的星型/雪花模式

- 场景描述：事实表与多个维度表连接的场景，如销售事实表与产品、时间、地区等维度表的连接
- 优势：预先物化连接结果，加速分析查询

湖仓加速

- 场景描述：查询数据湖可能由于网络延迟和对象存储的吞吐限制而变慢
- 优势：利用 Doris 本地存储加速优势，加速数据湖分析

数仓分层

- 场景描述：基表中包含大量原始数据，查询需要进行复杂的 ETL 操作
- 优势：对数据建立多层异步物化视图实现数仓分层

不推荐使用场景

基础表频繁更新

- 场景描述：源表数据变更非常频繁 (如每分钟多次更新)
- 问题：异步物化视图难以保持同步，刷新成本过高，需要考虑定期刷新。

简单查询

- 场景描述：仅涉及单表扫描或简单过滤的查询
- 问题：异步物化视图带来的收益无法抵消刷新成本

需要实时（1~5 分钟内）数据的场景

- 场景描述：业务要求数据必须是最新版本
- 问题：异步物化视图存在数据延迟

源表数据量很小

- 场景描述：基础表只有少量记录 (如几百行)
- 问题：异步物化视图优化效果不明显

刷新策略选择建议

异步物化视图提供三种主要刷新策略，每种策略适用于不同的业务场景和数据特征。合理选择刷新策略对于平衡数据新鲜度和系统性能至关重要。

刷新策略详解

手动刷新

工作方式：

- 由用户通过显式命令或外部系统调度触发

适用场景： - 对数据实时性要求不高的报表系统 - 数据仓库中的历史数据分析 - 需要与特定业务流程同步刷新的场景 - 大规模数据刷新需要协调系统资源时

优缺点：

优点：完全控制刷新时机，可避开业务高峰期

缺点：需要额外管理刷新调度，需要做好容错，方式外部循环不断的刷新。

定时刷新

工作方式：

- 按固定时间间隔自动刷新
- 最小时间单位为分钟级
- 可指定第一次运行任务的开始时间

适用场景： - 周期性业务指标监控 - 阶梯式数据管道 - 时间敏感度分级的报表体系 - 有规律波动的源数据

优缺点：

优点：定时数据处理，确定性的数据延迟

缺点：数据新鲜度局限，相关视图的刷新时序需要人工编排

配置约束：

不建议将所有物化视图设置为高频定时刷新，达到类实时的目的，这会导致：

- 系统资源持续被占用
- 刷新作业相互竞争资源
- 频繁增删 partition/tablet 等，对 be 造成较大压力

触发式刷新

工作方式：

- 当基表数据变更时自动触发刷新

适用场景：

- 多层物化视图架构的上层视图
- 基表变更频率较低的场景

优缺点：

优点：数据新鲜度高，自动化程度高

缺点：可能造成刷新风暴，难以预测系统负载

配置约束：

不建议对基础层物化视图使用触发式刷新，除非：

- 能明确知道基表刷新频率不高（如：几十分钟变更一次）

刷新策略组合建议

分层策略

- 基础层：定时刷新 (如每小时)
- 中间层：定时或触发式刷新
- 展示层：触发式刷新或手动刷新 ######## 业务关键性分级
- 关键实时业务数据：不建议使用异步物化视图
- 常规分析数据：定时刷新 (每日/每小时)
- 历史/归档数据：手动刷新 ######## 数据变更频率适配
- 高频变更：定时刷新 (较长间隔) 或手动刷新
- 低频变更：触发式刷新或短间隔定时刷新
- 批量变更：变更后手动刷新

刷新频率建议

此建议仅为通用建议，实际还需根据系统资源，异步物化视图数量，其它业务资源占用等情况综合评估

实际刷新耗时	刷新频率
小于 15s	大于等于 5 分钟
小于 10 分钟	大于等于 1 小时
小于 1 小时	大于等于 1 天

异步物化视图注意点

1. 监控：物化视图运行后要通过metrics及时监控系统运行情况，后续异步物化视图自身也会暴露更多的监控指标，目前可通过tasks查看任务数量，执行状态，任务耗时等信息
2. 规划：要规划要物化视图的运行个数，运行频率，集群的最大计算量这些。切记不要“只管建物化视图，不维护物化视图”，物化视图本质上是增强的 ETL 计算，和传统 ETL 一样需要维护。
3. 资源隔离：物化视图是数据计算任务，需要按需做好资源隔离。

2.13.2.3.5 异步物化视图常见问题

构建和刷新

Q1：物化视图是如何判断需要刷新哪些分区的？

Doris 内部会计算物化视图和基表的分区对应关系，并且记录上次刷新成功后物化视图使用的基表分区版本。例如，物化视图 mv1 由基表 t1 和 t2 创建，并且依赖 t1 进行分区。

假设 mv1 的分区 p202003 对应基表 t1 的分区 p20200301 和 p20200302，那么刷新 p202003 之后，会记录分区 p20200301、p20200302，以及表 t2 的当前版本。

下次刷新时，会判断 p20200301、p20200302 以及 t2 的版本是否发生变化。如果其中之一发生了变化，就代表 p202003 需要刷新。

当然，如果业务上能接受 t2 的变化而不触发 mv1 的刷新，可以通过物化视图的属性excluded_trigger_tables ↪ 来设置。

Q2：物化视图占用资源过多，影响其他业务怎么办？

可以通过物化视图的属性指定workload_group 来控制物化视图刷新任务的资源。

使用时需要注意，如果内存设置的太小，单个分区刷新又需要的内存较多，任务会刷新失败。需要根据业务情况进行权衡。

Q3：能基于物化视图创建新的物化视图吗？

能。从 Doris 2.1.3 版本开始支持。但是，在刷新数据时，每个物化视图都是采用单独的刷新逻辑。例如，如果 mv2 是基于 mv1 创建的，而 mv1 又是基于 t1 创建的，那么在刷新 mv2 时，不会考虑 mv1 与 t1 之间的数据是否同步。

Q4：Doris 都支持哪些外表？

Doris 支持的所有外表都能用于创建物化视图，但是目前仅有 Hive 支持分区刷新，后续会陆续支持其他类型。

Q5：物化视图显示和 Hive 数据一致，但是实际上不一致

物化视图仅能保证其数据与通过 Catalog 查询的结果一致。由于 Catalog 包含一定的元数据和数据缓存，因此，如果想让物化视图与 Hive 中的数据保持一致，需要通过 Refresh Catalog 等方式，确保 Catalog 中的数据与 Hive 中的数据一致。

Q6：物化视图支持 Schema Change 吗？

不支持修改，因为物化视图的列属性是根据物化视图定义的 SQL 推导出来的。目前不支持显式地自定义修改。

Q7：物化视图使用的基表允许 Schema Change 吗？

允许。但是变更之后，使用到该基表的物化视图的状态会从 NORMAL 变为 SCHEMA_CHANGE，此时物化视图将不能被用来透明改写，但是不影响直接查询物化视图。如果物化视图下次刷新任务成功，那么状态会由 SCHEMA_CHANGE 变回 NORMAL。

Q8：主键模型的表能用来创建物化视图吗？

物化视图对基表的数据模型没有要求。但是物化视图本身只能是明细模型。

Q9：物化视图上还能建索引吗？

能。

Q10：物化视图刷新的时候会锁表吗？

在很小的阶段会锁表，但不会持续的占用表锁（几乎等同于导入数据的锁表时间）。

Q11：物化视图适合近实时场景吗？

不太适合。物化视图刷新的最小单位是分区，如果数据量较大会占用较多的资源，并且实时性不够。可以考虑使用同步物化视图或其他手段。

Q12：构建分区物化视图报错

报错信息如下：

```
Unable to find a suitable base table for partitioning
```

出现该报错通常指的是物化视图的 SQL 定义和物化视图分区字段的选择，导致不能分区增量更新，所以创建分区物化视图会报错。

- 物化视图想要分区增量更新，需要满足以下要求，详情见物化视图刷新模式
- 最新的代码可以提示分区构建失败的原因，原因摘要和说明见附录 2

例如：

```
CREATE TABLE IF NOT EXISTS orders (
  o_orderkey INTEGER NOT NULL,
  o_custkey INTEGER NOT NULL,
  o_orderstatus CHAR(1) NOT NULL,
  o_totalprice DECIMALV3(15, 2) NOT NULL,
  o_orderdate DATE NOT NULL,
  o_orderpriority CHAR(15) NOT NULL,
  o_clerk CHAR(15) NOT NULL,
  o_shippriority INTEGER NOT NULL,
  O_COMMENT VARCHAR(79) NOT NULL
) DUPLICATE KEY(o_orderkey, o_custkey) PARTITION BY RANGE(o_orderdate) (
  FROM
    ('2024-05-01') TO ('2024-06-30') INTERVAL 1 DAY
) DISTRIBUTED BY HASH(o_orderkey) BUCKETS 3;


CREATE TABLE IF NOT EXISTS lineitem (
  l_orderkey INTEGER NOT NULL,
  l_partkey INTEGER NOT NULL,
  l_suppkey INTEGER NOT NULL,
  l_linenumber INTEGER NOT NULL,
  l_quantity DECIMALV3(15, 2) NOT NULL,
  l_extendedprice DECIMALV3(15, 2) NOT NULL,
  l_discount DECIMALV3(15, 2) NOT NULL,
  l_tax DECIMALV3(15, 2) NOT NULL,
  l_returnflag CHAR(1) NOT NULL,
  l_linestatus CHAR(1) NOT NULL,
  l_shipdate DATE NOT NULL,
  l_commitdate DATE NOT NULL,
  l_receiptdate DATE NOT NULL,
  l_shipinstruct CHAR(25) NOT NULL,
  l_shipmode CHAR(10) NOT NULL,
  l_comment VARCHAR(44) NOT NULL
) DUPLICATE KEY(
  l_orderkey, l_partkey, l_suppkey,
  l_linenumber
) DISTRIBUTED BY HASH(l_orderkey) BUCKETS 3;
```

物化视图定义如下，可以进行分区增量更新。如果选择orders.o_orderdate作为物化视图的分区字段，那么它是可以支持增量分区更新的。相反，如果使用了lineitem.l_shipdate，则不能实现增量更新。

原因为：

1. lineitem.l_shipdate不是基表的分区列，实际上lineitem表并没有设置分区列。
2. lineitem.l_shipdate是outer join操作中产生null值那一端的列。

```
CREATE MATERIALIZED VIEW mv_1
       BUILD IMMEDIATE
       REFRESH AUTO ON MANUAL
       partition by(o_orderdate)
       DISTRIBUTED BY RANDOM BUCKETS 2
       AS
SELECT
  l_linestatus,
  sum(
    l_extendedprice * (1 - l_discount)
  ) AS revenue,
  o_orderdate,
  o_shippriority
FROM
  orders
  LEFT JOIN lineitem ON l_orderkey = o_orderkey
WHERE
  o_orderdate <= DATE '2024-06-30'
  AND o_orderdate >= DATE '2024-05-01'
GROUP BY
  l_linestatus,
  o_orderdate,
  o_shippriority;
```

Q13：创建物化视图时报错

报错信息如下：

```
ERROR 1105 (HY000): errCode = 2, detailMessage = Syntax error in line 1:
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
```

可能原因如下：

1. 异步物化视图的语句，在新优化器下才支持，确保使用的是新优化器。

```
    SET enable_nereids_planner = true;
```

2. 可能是构建物化的语句使用的 关键词写错或者物化定义 SQL 语法有问题，可以检查下物化定义 SQL 和创建物化语句是否正确。

Q14：物化视图刷新成功后，还是没有数据

物化视图判断数据是否需要更新依赖于能够获取到基表或基表分区的版本信息。

遇到目前不支持获取版本信息的数据湖，例如 jdbc catalog，那么刷新的时候会认为物化视图是不需要更新的，因此创建或者刷新物化视图的时候应该指定 complete 而不是 auto

物化视图支持数据湖的进度参考数据湖支持情况

Q15：创建的是分区物化视图，为什么每次都是全量刷新？

物化视图的分区增量刷新依赖于基表分区的版本信息。如果物化视图的分区自上次刷新后，基表分区的数据发生变化，那么物化视图就会刷新此分区。如果物化视图是分区物化视图，刷新的时候刷新了所有分区，那么可能是以下原因：- 物化视图定义 SQL 中非分区追踪表数据发生了变化，导致物化视图刷新时无法判断哪些分区需要更新，因此只能全量刷新。

例如：此物化视图追踪 orders 表的 o_orderdate 分区，但是 lineitem 或者 partsupp 数据发生了变化，导致物化视图无法判断哪些分区需要更新，因此只能全量刷新。

```
CREATE MATERIALIZED VIEW partition_mv
BUILD IMMEDIATE
REFRESH AUTO
ON SCHEDULE EVERY 1 DAY STARTS '2024-12-01 20:30:00'
PARTITION BY (DATE_TRUNC(o_orderdate, 'MONTH'))
DISTRIBUTED BY HASH (l_orderkey) BUCKETS 2
PROPERTIES
("replication_num" = "3")
AS
SELECT
o_orderdate,
l_orderkey,
l_partkey
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey
LEFT JOIN partsupp ON ps_partkey = l_partkey
and l_suppkey = ps_suppkey;
```

可以运行如下查看物化视图追踪的基表

```
SELECT *
FROM mv_infos('database'='db_name')
WHERE Name = 'partition_mv' `\G`
```

返回结果如下，MvPartitionInfo 中的 partitionType 为 FOLLOW_BASE_TABLE，表示物化视图分区跟随基表分区。relatedCol 为 o_orderdate，表示物化视图分区是基于 o_orderdate 分区的。

```
                Id: 1752809156450
              Name: partition_mv
           JobName: inner_mtmv_1752809156450
             State: NORMAL
SchemaChangeDetail:
      RefreshState: SUCCESS
       RefreshInfo: BUILD IMMEDIATE REFRESH AUTO ON SCHEDULE EVERY 1 DAY STARTS "2025-12-01
          ↪ 20:30:00"
          QuerySql: SELECT
                    `internal`.`doc_db`.`orders`.`o_orderdate`,
                    `internal`.`doc_db`.`lineitem`.`l_orderkey`,
                    `internal`.`doc_db`.`lineitem`.`l_partkey`
                    FROM
                    `internal`.`doc_db`.`orders`
                    LEFT JOIN `internal`.`doc_db`.`lineitem` ON `internal`.`doc_db`.`lineitem`.`l
                        ↪ _orderkey` = `internal`.`doc_db`.`orders`.`o_orderkey`
                    LEFT JOIN `internal`.`doc_db`.`partsupp` ON `internal`.`doc_db`.`partsupp`.`
                        ↪ ps_partkey` = `internal`.`doc_db`.`lineitem`.`l_partkey`
                    and `internal`.`doc_db`.`lineitem`.`l_suppkey` = `internal`.`doc_db`.`
                        ↪ partsupp`.`ps_suppkey`
   MvPartitionInfo: MTMVPartitionInfo{partitionType=EXPR, relatedTable=orders, relatedCol='o_
       ↪ orderdate', partitionCol='o_orderdate'}
SyncWithBaseTables: 1
```

解决办法：- 如果物化视图中 lineitem 或者 partsupp 表数据变化，对物化视图没有影响，可以通过设置物化视图的属性 `excluded_trigger_tables` 来排除 lineitem 或 partsupp 表的变化引起物化视图全量刷。命令为 `ALTER MATERIALIZED VIEW partition_mv set("excluded_trigger_tables"="lineitem,partsupp");`

查询和透明改写

Q1：如何确认是否命中，如果不命中如何查看原因？

可以通过 `explain query_sql` 的方式查看是物化视图命中情况摘要信息，例如物化视图如下：

```
CREATE MATERIALIZED VIEW mv11
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(l_shipdate)
DISTRIBUTED BY HASH(l_orderkey) BUCKETS 10
AS
SELECT l_shipdate, l_orderkey, O_ORDERDATE, count(*)
FROM lineitem
LEFT OUTER JOIN orders on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_orderkey, O_ORDERDATE;
```

查询如下：

```
explain
```

```
SELECT l_shipdate, l_orderkey, O_ORDERDATE, count(*)
FROM lineitem
LEFT OUTER JOIN orders on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_orderkey, O_ORDERDATE;
```

- 物化视图的命中信息在 plan 最后。
- MaterializedViewRewriteSuccessAndChose： 表示透明改写成功，并且 CBO 选择的物化视图名称列表。-
- MaterializedViewRewriteSuccessButNotChose： 表示透明改写成功，但是最终 CBO 没有选择的物化视图名称列表，没有选择意味着执行计划不会使用物化视图。”
- MaterializedViewRewriteFail：表示列举透明改写失败及原因摘要。
- 如果 explain 最后没有出现 MaterializedView 等信息，那么意味着此物化视图状态不可用，因此不能参与透明改写。(关于什么情况下会导致物化视图状态不可用，可详细参考使用与实践 - 查看物化视图状态)。

例如：

```
| MaterializedView                                                                          |
| MaterializedViewRewriteSuccessAndChose:                                                   |
| internal#regression_test_nereids_rules_p0_mv#mv11,                                        |
|                                                                                           |
| MaterializedViewRewriteSuccessButNotChose:                                                |
|                                                                                           |
| MaterializedViewRewriteFail:                                                              |
+-------------------------------------------------------------------------------------------+
```

Q2：物化视图没有命中的原因是什么？

首先，需要确认物化视图是否命中，需要执行如下 SQL，详细见查询和透明改写 - 问题 1

```
explain
your_query_sql;
```

如果未命中，可能是存在以下几个问题：

- 在 Doris 2.1.3 之前的版本中，物化视图透明改写功能是默认关闭的。需要打开对应的开关，才能实现透明改写。具体的开关值，请参见异步物化视图相关开关。
- 物化视图可能处于不可用状态，从而导致透明改写无法命中。要查看物化视图的构建状态，请参见查看物化视图状态。
- 若经过前两步的检查后，物化视图仍然无法命中，那么可能是物化视图的定义 SQL 和查询 SQL 不在当前物化视图改写能力的范围内。详情请参考物化视图透明改写能力。
- 对于失败命中的详细信息和说明，请查阅附录 1。

以下是物化视图透明改写失败的示例：

用例 1：

创建物化视图的 SQL 如下：

```
CREATE MATERIALIZED VIEW mv11
BUILD IMMEDIATE REFRESH AUTO ON MANUAL
partition by(l_shipdate)
DISTRIBUTED BY HASH(l_orderkey) BUCKETS 10
PROPERTIES ('replication_num' = '1')
AS
SELECT l_shipdate, l_orderkey, O_ORDERDATE, count(*)
FROM lineitem
LEFT OUTER JOIN orders on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_orderkey, O_ORDERDATE;
```

执行查询如下：

```
explain
SELECT l_shipdate, l_linestatus, O_ORDERDATE, count(*)
FROM orders
LEFT OUTER JOIN lineitem on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_linestatus, O_ORDERDATE;
```

Explain 显示的信息如下：

```
| MaterializedView
    ↪
    ↪ |
| MaterializedViewRewriteSuccessAndChose:
    ↪                                                                    |
|
    ↪
    ↪ |
| MaterializedViewRewriteSuccessButNotChose:
    ↪                                                                |
|
    ↪
    ↪ |
| MaterializedViewRewriteFail:
    ↪                                                                          |
|   Name: internal#doc_test#mv11
    ↪                                                                        |
|   FailSummary: View struct info is invalid, The graph logic between query and view is not
    ↪ consistent      |
```

在执行结果中，可以看到MaterializedViewRewriteFail有失败的摘要信息，其中The graph logic between query and view is not consistent表示查询和物化视图的 Join 逻辑不一致，即查询和物化视图的 Join 类型或 Join 的表不同。

在上述示例中，查询和物化视图 Join 的表顺序不一致，因此会报告此错误。对于透明改写失败的摘要信息和说明，请参见附录 1。

用例 2：

执行查询如下：

```
explain
SELECT l_shipdate, l_linestatus, O_ORDERDATE, count(*)
FROM lineitem
LEFT OUTER JOIN orders on l_orderkey = o_orderkey
GROUP BY l_shipdate, l_linestatus, O_ORDERDATE;
```

Explain 显示的信息如下：

```
| MaterializedView
    ↪
    ↪ |
| MaterializedViewRewriteSuccessAndChose:
    ↪                                                                                |
|
    ↪
    ↪ |
| MaterializedViewRewriteSuccessButNotChose:
    ↪                                                                            |
|
    ↪
    ↪ |
| MaterializedViewRewriteFail:
    ↪
    ↪ |
|   Name: internal#doc_test#mv11
    ↪
    ↪ |
|   FailSummary: View struct info is invalid, View dimensions doesn't not cover the query
    ↪ dimensions                        |
```

失败的摘要信息为View dimensions doesn't not cover the query dimensions，表示查询中group by的字段无法从物化视图的group by字段中获取，因此会报告此错误。

Q3：什么情况会导致物化视图的状态变更并且不可用？

不可用，指代的是“物化视图不能用于透明改写”的简称，而物化视图仍然可以直接查询。

- 对于全量物化视图，如果使用的基表数据发生变更或者发生 Schema Change，会导致物化视图不可用。

- 对于分区物化视图，基表数据变更会导致对应的分区不可用。而基表的 Schema Change 则会导致整个物化视图不可用。

目前，物化视图刷新失败也会导致其不可用。但后续会进行优化，即使刷新失败，已存在的物化视图仍然可用于透明改写。

Q4：出现直查物化视图没有数据的情况

可能物化视图正在构建中，也有可能物化视图构建已经失败。

可以查询物化视图的状态来确认，具体方法请参见查看物化视图状态。

Q5：物化视图使用的基表数据变了，但是此时物化视图还没有刷新，透明改写的行为是什么？

异步物化视图的数据与基表之间存在一定的时延。

1. 对于内表以及能够感知数据变化的外表（如 Hive）：当基表数据发生变更时，物化视图是否可用取决于 grace_period 的阈值。

grace_period 是指允许物化视图与所用基表数据不一致的时间段。例如：

- 如果 grace_period 设置为 0，则意味着要求物化视图与基表数据保持一致，此时物化视图才可用于透明改写；对于外表（除 Hive 外），由于无法感知数据变更，因此无论外表的数据是否为最新，使用了外表的物化视图都可以用于透明改写（但数据可能会不一致）。
- 如果 grace_period 设置为 10 秒，则意味着允许物化视图与基表数据有最多 10 秒的延迟。如果物化视图的数据与基表数据的延迟在 10 秒内，那么此物化视图仍然可以用于透明改写。

2. 对于分区物化视图，如果部分分区失效，存在以下两种情况：

- 如果查询没有使用失效的分区数据，那么此物化视图仍然可用。
- 如果查询使用了失效分区的数据，并且数据时效在 grace_period 范围内，那么此物化视图仍然可用。如果物化视图数据时效超出 grace_period 范围，可以通过联合原表和物化视图来响应查询。此时需要开启允许联合改写开关 enable_materialized_view_union_rewrite（自 2.1.5 版本起，该开关默认开启。）

附录

1 透明改写失败摘要信息和说明

摘要信息	说明
View struct info is invalid	物化视图的结构信息不合法，目前支持改写的 SQL pattern 如下查询是 join，物化也是 join，查询是 agg，物化可以没有 join 透明改写过程中，多数会显示这个问题，因为每个透明改写的规则负责一定 SQL pattern 的改写，如果命中了不符合要求规则，就会有这个错误，这个错误一般不是决定透明改写失败的主要原因。
Materialized view rule exec fail	这个一般是透明改写规则执行抛异常，这种情况需要 Explain memo plan query_sql 看下具体异常栈
Match mode is invalid	查询和物化视图表的数量不一致，暂不支持改写
Query to view table mapping is null	查询和物化视图表映射生成失败

摘要信息	说明
queryToViewTableMappings are over the limit and be intercepted	查询自关联的表太多了，导致透明改写空间膨胀过大，停止透明改写
Query to view slot mapping is null	查询和物化的表 slot 映射失败
The graph logic between query and view is not consistent	查询和物化的 Join 类型不同或者 Join 的表不同
Predicate compensate fail	一般是查询的条件范围在物化的范围外，比如查询是 a > 10，但是物化是 a > 15
Rewrite compensate predicate by view fail	条件补偿失败，通常是查询比物化多的条件要进行补偿，但是条件用的列没有出现在物化视图 select 后
Calc invalid partitions fail	如果是分区物化视图，会尝试计算查询使用的物化视图分区是否有效，计算查询可能用到的失效分区失败
mv can not offer any partition for query	查询使用的都是物化视图的失效分区，也就是说物化视图不能为查询提供有效的数据，可能是物化视图对应分区自上次刷新后，基表对应分区数据发生变更，可以使用 `show partitions from mv_name` 查看分区的 `SyncWithBaseTables` 字段是否为 true。如果为 false, 可以手动刷新下对应分区，如果允许物化和查询的数据有一定延迟，可以设置物化视图的 `grace_peroid`属性，单位是秒
Add filter to base table fail when union rewrite	查询使用了物化视图失效的分区，尝试将物化视图和原表 union all 失败
RewrittenPlan output logical properties is different with target group	改写完成，物化视图的 output 和原查询不一致
Rewrite expressions by view in join fail	join 改写中，查询使用的字段或者表达式不在物化视图中
Rewrite expressions by view in scan fail	单表改写中，查询使用的字段或者表达式不在物化视图中
Split view to top plan and agg fail, view doesn’t not contain aggregate	改写聚合时，物化视图中不含有聚合
Split query to top plan and agg fail	改写聚合时，查询中不含有聚合
rewritten expression contains aggregate functions when group equals aggregate rewrite	在查询和物化 group by 相等的时候，改写后的表达式含有聚合函数
Can not rewrite expression when no roll up	在查询和物化 group by 相等的时候，表达式改写失败
Query function roll up fail	聚合改写时，聚合函数上卷失败
View dimensions do not cover the query dimensions	查询中 group by 使用了一些维度，这些维度不在物化视图的 group by 后

摘要信息	说明
View dimensions don’t not cover the query dimensions in bottom agg	查询中 group by 使用了一些维度，这些维度不在物化视图的 group by 后
View dimensions do not cover the query group set dimensions	查询中 group sets 使用了一些维度，这些维度不在物化视图的 group by 后
The only one of query or view is scalar aggregate and can not rewrite expression meanwhile	查询中有 group by，但是物化视图中没有 group by
Both query and view have group sets, or query doesn’t have but view has, not supported	查询和物化视图都有 group sets 查询没有 group sets，但是物化视图有，这种不支持透明改写

2 异步物化视图分区构建物化视图失败原因和说明

分区物化视图的刷新原理是分区增量更新：

- 第一步需要计算物化视图的分区字段是否可以和基表的分区映射。
- 第二步是计算具体的映射关系，看分区是 1:1 还是 1:n。

摘要信息	说明
partition column can not be found in the SQL select column	物化视图定义中 partition by 后用的列需要出现在物化定义 SQL 的 select 后
can’t not find valid partition track column, because %s	找不到合适的分区列，具体原因在 because 后
partition track doesn’t support mark join	物化视图分区字段引用的列是 mark join 输入表的分区列，暂不支持
partition column is in un supported join null generate side	物化视图分区字段引用列在 join 的 null 产生端，比如 left join 的右侧
relation should be LogicalCatalogRelation	物化化视图引用的分区基表 scan 类型应该是 LogicalCatalogRelation，其他暂不支持
self join doesn’t support partition update	自关联的 SQL，暂不支持构建物化视图
partition track already has a related base table column	物化化视图引用的分区列，目前只支持引用一张基表的分区列
relation base table is not MTMVRelatedTableIf	物化视图引用的分区基表没有继承 MTMVRelatedTableIf，MTMVRelatedTableIf 标识了是不是可以分区的表
The related base table is not partition table	物化视图使用的基表不是分区表
The related base table partition column doesn’t contain the mv partition	物化视图 partition by 后引用的列在分区基表中不存在

摘要信息	说明
group by sets is empty, doesn’t contain the target partition	物化视图定义 SQL，使用了聚合，但是 group by 为空
window partition sets don’t contain the target partition	使用了 window 函数，但是物化视图引用分区列不在 partition by 中
Unsupported plan operate in track partition	物化视图定义 SQL 中使用了不支持的操作，比如 order by 等
context partition column should be slot from column	使用了 window 函数，partition by 中物化视图引用分区列不是单纯的列，而是表达式
partition expressions use more than one slot reference	group by 或者 partition by 后分区列是包含了多列的表达式，而不是单纯的列。比如 group by partition_col + other_col
column to check using invalid implicit expression	物化视图分区列仅仅可以使用 date_trunc 中，使用了分区列的表达式只能是 date_trunc 等
partition column time unit level should be greater than SQL select column	物化视图中 partition by 后的 date_trunc 中的时间单位粒度小于物化视图定义 SQL 中 select 后出现的时间单位粒度比如物化视图 `partition by(date_trunc(col, 'day'))`，但是物化视图定义 SQL select 后是 `date_trunc(col, 'month')`

2.13.3 查询缓存

2.13.3.1 概念介绍

SQL Cache 是 Doris 提供的一种查询优化机制，可以显著提升查询性能。它通过缓存查询结果来减少重复计算，适用于数据更新频率较低的场景。

SQL Cache 基于以下关键因素来存储和获取缓存：

- SQL 文本
- 视图定义
- 表和分区的版本
- 用户变量和结果值
- 非确定函数和结果值
- 行策略定义
- 数据脱敏定义

以上因素的组合唯一确定一个缓存数据集。如果其中任何一个发生变化，例如 SQL 变化、查询字段或条件不同或者数据更新后版本变化，缓存将不会命中。

对于涉及多表 Join 的查询，如果其中一个表更新了，分区 ID 或版本号就会不同，导致缓存无法命中。

SQL Cache 非常适合 T+1 更新场景。数据在凌晨更新，第一次查询从 BE 获取结果并放入缓存，后续相同相似的查询则直接从缓存获取结果。实时更新数据也可以使用 SQL Cache，但可能会面临较低的缓存命中率。

目前，SQL Cache 支持 OlapTable 内部表和 Hive 外部表。

2.13.3.2 使用限制

2.13.3.2.1 非确定函数

1. 非确定函数是指其运算结果与输入参数之间无法形成固定关系的函数。
2. 以常见函数 `select now()` 为例，它返回当前的日期与时间。由于该函数在不同时间执行时会返回不同的结果，因此其返回值是动态变化。now 函数返回的是秒级别的时间，所以在同一秒内可以复用之前的 SQL Cache；但下一秒之后，就需要重新创建 SQL Cache。
3. 为了优化缓存利用率，建议将这种细粒度的时间转为粗粒度的时间，例如使用 `select * from tbl where dt=date(now())`。在这种情况下，同一天的查询都可以利用到 SQL Cache。
4. 相比之下，`random()` 函数则很难利用到 Cache，因为它每次运算的结果都是不同的。因此，应尽量避免在查询中使用这类非确定函数。

2.13.3.3 实现原理

2.13.3.3.1 BE 实现原理

在大多数情况下，SQL Cache 的结果会通过一致性哈希方法选择一个 BE，并将其存放在该 BE 的内存中。这些结果以 HashMap 的结构进行存储。当读写 Cache 的请求到来时，系统会使用 SQL 字符串等元数据信息的摘要作为 Key，从 HashMap 中快速检索结果数据进行操作。

2.13.3.3.2 FE 实现原理

当 FE 接收到查询请求时，它首先会在内存中利用 SQL 字符串进行查找，判断之前是否执行过相同的查询，并尝试获取该查询的元数据信息，这些信息包括查询所涉及表的版本以及分区的版本。

若这些元数据信息保持不变，则说明相应表的数据未发生变更，因此可以重复利用之前的 SQL Cache。在这种情况下，FE 能够跳过 SQL 解析优化流程，直接依据一致性哈希算法定位到对应的 BE，并尝试从中检索查询结果。

- 若目标 BE 中存有该查询结果的缓存，FE 便能迅速将结果返回给客户端
- 反之，若 BE 中未找到对应的结果缓存，FE 则需执行完整的 SQL 解析与优化流程，随后将查询计划传送至 BE 进行计算处理。

当 BE 将计算结果返回给 FE 后，FE 会负责将这些结果存储至对应的 BE 中，并在其内存中记录此次查询的元数据信息。这样做是为了在后续接收到相同查询时，FE 能够直接从 BE 中获取结果，从而提高查询效率。

此外，如果 SQL 优化阶段判断出查询结果仅包含 0 行或 1 行数据，FE 会选择将这些结果保存在其内存中，以便更快速地响应未来可能的相同查询。

2.13.3.4 快速上手

2.13.3.4.1 开启和关闭 SQL Cache

```
-- 在当前 Session 打开 SQL Cache，默认是关闭状态
set enable_sql_cache=true;
-- 在当前 Session 关闭 SQL Cache
set enable_sql_cache=false;

-- 全局打开 SQL Cache，默认是关闭状态
set global enable_sql_cache=true;
-- 全局关闭 SQL Cache
set global enable_sql_cache=false;
```

2.13.3.4.2 检查查询是否命中 SQL Cache

在 Doris 2.1.3 版本及其后续版本中，用户能够通过执行`explain plan`语句检查当前查询是否能够成功命中 SQL Cache。

如示例所示，当查询计划树中出现`LogicalSqlCache`或`PhysicalSqlCache`节点时，即表明查询已命中 SQL Cache。

```
> explain plan select * from t2;

+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪                                                                                                              |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ========== PARSED PLAN (time: 28ms) ==========
    ↪                                                                                                              |
| LogicalSqlCache[1] ( queryId=711dea740e4746e6-8bc11afe08f6542c )
    ↪                                                |
| +--PhysicalResultSink[39] ( outputExprs=[id#0, name#1] )
    ↪                                                                |
|    +--PhysicalOlapScan[t2]@0 ( stats=12 )
    ↪                                                                                                              |
|
    ↪
    ↪ |
| ========== ANALYZED PLAN  ==========
    ↪                                                                                                                  |
| LogicalSqlCache[1] ( queryId=711dea740e4746e6-8bc11afe08f6542c )
    ↪                                                |
```

```
| +--PhysicalResultSink[39] ( outputExprs=[id#0, name#1] )
    ↪                                                    |
|    +--PhysicalOlapScan[t2]@0 ( stats=12 )
    ↪                                                                |
|
    ↪
    ↪ |
| ========== REWRITTEN PLAN  ==========
    ↪                                                                    |
| LogicalSqlCache[1] ( queryId=711dea740e4746e6-8bc11afe08f6542c )
    ↪                                              |
| +--PhysicalResultSink[39] ( outputExprs=[id#0, name#1] )
    ↪                                                    |
|    +--PhysicalOlapScan[t2]@0 ( stats=12 )
    ↪                                                                |
|
    ↪
    ↪ |
| ========== OPTIMIZED PLAN  ==========
    ↪                                                                    |
| PhysicalSqlCache[3] ( queryId=711dea740e4746e6-8bc11afe08f6542c, backend=192.168.126.3:9051,
    ↪ rowCount=12 ) |
| +--PhysicalResultSink[39] ( outputExprs=[id#0, name#1] )
    ↪                                                    |
|    +--PhysicalOlapScan[t2]@0 ( stats=12 )
    ↪                                                                |
+--
    ↪ ------------------------------------------------------------------------------------------------------------------------
    ↪
```

对于 Doris 2.1.3 之前的版本，用户则需要通过查看 Profile 信息来确认查询是否命中了 SQL Cache。在 Profile 信息中，若 `Is Cached:` 这一字段显示为 `Yes`，则代表该查询已成功命中 SQL Cache。

```
Execution Summary:
      - Parse SQL Time: 18ms
      - Nereids Analysis Time: N/A
      - Nereids Rewrite Time: N/A
      - Nereids Optimize Time: N/A
      - Nereids Translate Time: N/A
      - Workload Group: normal
      - Analysis Time: N/A
      - Wait and Fetch Result Time: N/A
      - Fetch Result Time: 0ms
      - Write Result Time: 0ms
      - Doris Version: 915138e801
      - Is Nereids: Yes
```

```
    - Is Cached: Yes
    - Total Instances Num: 0
    - Instances Num Per BE:
    - Parallel Fragment Exec Instance Num: 1
    - Trace ID:
    - Transaction Commit Time: N/A
    - Nereids Distribute Time: N/A
```

这两种方法均为用户提供了有效的手段来验证查询是否利用了 SQL Cache，从而帮助用户更好地评估查询性能并优化查询策略。

2.13.3.5 指标监控

1. 在 FE 的 HTTP 接口 `http://${FE_IP}:${FE_HTTP_PORT}/metrics` 会返回两个相关指标： 该指标统计的是命中次数，只增不减，当 FE 重启后从 0 开始统计。

```
doris_fe_cache_added{type="sql"} 1

### 代表命中了两次 SQL Cache
doris_fe_cache_hit{type="sql"} 2
```

2. 在 BE 的 HTTP 接口 `http://${BE_IP}:${BE_HTTP_PORT}/metrics` 会返回相关信息： 由于不同的 Cache 可能会存放到不同的 BE 中，因此需收集所有 BE 的 Metrics 才能得到完整信息。

```
### 代表当前 BE 的内存中存在 1205 个 Cache
doris_be_query_cache_sql_total_count 1205

### 当前所有 Cache 占用 BE 内存 44k
doris_be_query_cache_memory_total_byte 44101
```

2.13.3.6 内存控制

2.13.3.6.1 FE 内存控制

在 FE 中，Cache 的元数据信息被设置为弱引用。当 FE 内存不足时，系统会自动释放最近最久未使用的 Cache 元数据。此外，用户还可以通过执行以下 SQL 语句，进一步限制 FE 内存的使用量。此配置实时生效，且每个 FE 都需要进行配置。若需持久化配置，则需将其保存在 fe.conf 文件中。

```
-- 最多存放 100 个 Cache 元数据，超过时自动释放最近最久未使用的元数据。默认值为 100。
ADMIN SET FRONTEND CONFIG ('sql_cache_manage_num'='100');

-- 当 300 秒未访问该 Cache 元数据后，自动进行释放。默认值为 300。
ADMIN SET FRONTEND CONFIG ('expire_sql_cache_in_fe_second'='300');
```

2.13.3.6.2 BE 内存控制

在 be.conf 文件中进行以下配置更改，重启 BE 后生效：

```
-- 当 Cache 的内存空间超过 query_cache_max_size_mb + query_cache_elasticity_size_mb 时，
-- 释放最近最久未使用的 Cache，直至占用内存低于 query_cache_max_size_mb。
query_cache_max_size_mb = 256
query_cache_elasticity_size_mb = 128
```

另外还可以在 FE 中配置，当结果行数或大小超过某个阈值时，不创建 SQL Cache：

```
-- 默认超过 3000 行结果时，不创建 SQL Cache。
ADMIN SET FRONTEND CONFIG ('cache_result_max_row_count'='3000');

-- 默认超过 30M 时，不创建 SQL Cache。
ADMIN SET FRONTEND CONFIG ('cache_result_max_data_size'='31457280');
```

2.13.3.7 排查缓存失效原因

缓存失效原因一般包括以下几点：

1. 表/视图的结构发生了变化，例如执行了 `drop table`、`replace table`、`alter table` 或 `alter view` 等操作。
2. 表数据发生了变化，例如执行了 `insert`、`delete`、`update` 或 `truncate` 等操作。
3. 用户权限被移除，例如执行了 `revoke` 操作。
4. 使用了非确定函数，并且函数的评估值发生了变化，例如执行了 `select random()`。
5. 使用了变量，并且变量的值发生了变化，例如执行了 `select * from tbl where dt = @dt_var`。
6. Row Policy 或 Data Masking 发生了变化，例如设置了用户对某些表的部分数据不可见。
7. 结果行数超过了 FE 配置的 `cache_result_max_row_count`，默认值为 3000 行。
8. 结果大小超过了 FE 配置的 `cache_result_max_data_size`，默认值为 30MB。

2.13.4 高并发点查

> 高并发点查功能自 Doris 2.0 版本起具有重大性能提升

2.13.4.1 描述

Doris 基于列存格式引擎构建，在高并发服务场景中，用户总是希望从系统中获取整行数据。但是，当表宽时，列存格式将大大放大随机读取 IO。Doris 查询引擎和计划对于某些简单的查询（如点查询）来说太重了。需要一个在 FE 的查询规划中规划短路径来处理这样的查询。FE 是 SQL 查询的访问层服务，使用 Java 编写，分析和解析 SQL 也会导致高并发查询的高 CPU 开销。为了解决上述问题，我们在 Doris 中引入了行存、短查询路径、PreparedStatement 来解决上述问题，下面是开启这些优化的指南。

2.13.4.2 行存

用户可以在 Olap 表中开启行存模式，但是需要额外的空间来存储行存。目前的行存实现是将行存编码后存在单独的一列中，这样做是为了简化行存的实现。行存模式仅支持在建表的时候开启，需要在建表语句的 property 中指定如下属性：

```
"store_row_column" = "true"
```

2.13.4.3 在 Unique 模型下的点查优化

上述的行存用于在 Unique 模型下开启 Merge-On-Write 策略是减少点查时的 IO 开销。当enable_unique_key_merge_on_write与store_row_column在创建 Unique 表开启时，对于主键的点查会走短路径来对 SQL 执行进行优化，仅需要执行一次 RPC 即可执行完成查询。下面是在 Unique 模型下，点查结合行存开启 Merge-On-Write 策略的例子：

```
CREATE TABLE `tbl_point_query` (
    `k1` int(11) NULL,
    `v1` decimal(27, 9) NULL,
    `v2` varchar(30) NULL,
    `v3` varchar(30) NULL,
    `v4` date NULL,
    `v5` datetime NULL,
    `v6` float NULL,
    `v7` datev2 NULL
) ENGINE=OLAP
UNIQUE KEY(`k1`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`k1`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1",
    "enable_unique_key_merge_on_write" = "true",
    "light_schema_change" = "true",
    "store_row_column" = "true"
);
```

注意：

1. enable_unique_key_merge_on_write应该被开启，存储引擎需要根据主键来快速点查
2. 当条件只包含主键时，如select * from tbl_point_query where k1 = 123，类似的查询会走短路径来优化查询
3. light_schema_change应该被开启，因为主键点查的优化依赖了轻量级 Schema Change 中的column unique id来定位列
4. 只支持单表 key 列等值查询不支持 join、嵌套子查询，where 条件里需要有且仅有 key 列的等值，可以认为是一种 key value 查询

5. 开启行存会导致空间膨胀，占用更多的磁盘空间，如果只需要查询部分列，在 Doris 3.0 后建议使用"row_store_columns"="key,v1,v2" 类似的方式指定部份列作为行存，查询的时候只查询这部份列，例如

```
SELECT k1, v1, v2 FROM tbl_point_query WHERE k1 = 1
```

2.13.4.4 使用 PreparedStatement

为了减少 SQL 解析和表达式计算的开销，我们在 FE 端提供了与 MySQL 协议完全兼容的PreparedStatement特性 (目前只支持主键点查)。当PreparedStatement在 FE 开启，SQL 和其表达式将被提前计算并缓存到 Session 级别的内存缓存中，后续的查询直接使用缓存对象即可。当 CPU 成为主键点查的瓶颈，在开启 PreparedStatement 后，将会有 4 倍 + 的性能提升。下面是在 JDBC 中使用 PreparedStatement 的例子

1. 设置 JDBC url 并在 Server 端开启 prepared statement

```
url = jdbc:mysql://127.0.0.1:9030/ycsb?useServerPrepStmts=true
```

2. 使用 PreparedStatement

```
// use `?` for placement holders, readStatement should be reused
PreparedStatement readStatement = conn.prepareStatement("select * from tbl_point_query where
    ↪ k1 = ?");
...
readStatement.setInt(1,1234);
ResultSet resultSet = readStatement.executeQuery();
...
readStatement.setInt(1,1235);
resultSet = readStatement.executeQuery();
...
```

2.13.4.5 开启行缓存

Doris 中有针对 Page 级别的 Cache，每个 Page 中存的是某一列的数据，所以 Page cache 是针对列的缓存，对于前面提到的行存，一行里包括了多列数据，缓存可能被大查询给刷掉，为了增加行缓存命中率，单独引入了行存缓存，行缓存复用了 Doris 中的 LRU Cache 机制来保障内存的使用，通过指定下面的的 BE 配置来开启

- disable_storage_row_cache 是否开启行缓存，默认不开启
- row_cache_mem_limit 指定 Row cache 占用内存的百分比，默认 20% 内存

2.13.4.6 性能优化

1. 通常，通过增加 Observer 数量来提升处理 query 能力是有效的
2. query 负载均衡：点查中如果发现接受点查请求的 fe cpu 使用过高，或请求响应变慢，可使用 jdbc load balance 进行负载均衡，将请求分散到多个节点，分担压力（同时也可以使用其他方式进行 query 负载均衡配置，如 Nginx，proxySQL）
3. 通过将点查请求定向发送至 Observer 角色来分担高并发点查的请求压力，减少向 fe master 发送点查请求，通常可以解决 Fe Master 节点查询耗时上下浮动问题，以获得更好性能与稳定性

2.13.4.7 常见问题

Q1. 如何确定配置无误使用了并发点查的短路径优化

A：explain sql，当执行计划中出现 SHORT-CIRCUIT，证明使用了短路径优化

```
mysql> explain select * from tbl_point_query where k1 = -2147481418 ;
+------------------------------------------------------------------------------------------------+
| Explain String(Old Planner)                                                                    |
+------------------------------------------------------------------------------------------------+
| PLAN FRAGMENT 0                                                                                |
|   OUTPUT EXPRS:                                                                                |
|     `test`.`tbl_point_query`.`k1`                                                             |
|     `test`.`tbl_point_query`.`v1`                                                              |
|     `test`.`tbl_point_query`.`v2`                                                              |
|     `test`.`tbl_point_query`.`v3`                                                              |
|     `test`.`tbl_point_query`.`v4`                                                              |
|     `test`.`tbl_point_query`.`v5`                                                              |
|     `test`.`tbl_point_query`.`v6`                                                              |
|     `test`.`tbl_point_query`.`v7`                                                              |
|   PARTITION: UNPARTITIONED                                                                     |
|                                                                                                |
|   HAS_COLO_PLAN_NODE: false                                                                    |
|                                                                                                |
|   VRESULT SINK                                                                                 |
|      MYSQL_PROTOCAL                                                                            |
|                                                                                                |
|   0:VOlapScanNode                                                                              |
|      TABLE: test.tbl_point_query(tbl_point_query), PREAGGREGATION: ON                          |
|      PREDICATES: `k1` = -2147481418 AND `test`.`tbl_point_query`.`__DORIS_DELETE_SIGN__` = 0 |
|      partitions=1/1 (tbl_point_query), tablets=1/1, tabletList=360065                          |
|      cardinality=9452868, avgRowSize=833.31323, numNodes=1                                     |
|      pushAggOp=NONE                                                                            |
|      SHORT-CIRCUIT                                                                             |
+------------------------------------------------------------------------------------------------+
```

Q2. 如何确定 prepared statement 生效

A：当发送请求到 Doris 之后，在 fe.audit.log 中找到相应的 query 请求，发现 Stmt=EXECUTE()，说明 prepared statement 生效

```
2024-01-02 11:15:51,248 [query] |Client=192.168.1.82:53450|User=root|Db=test|State=EOF|ErrorCode
    ↪ =0|ErrorMessage=|Time(ms)=49|ScanBytes=0|ScanRows=0|ReturnRows=1|StmtId=51|QueryId=
    ↪ b63d30b908f04dad-ab4a
   3ba21d2c776b|IsQuery=true|isNereids=false|feIp=10.16.10.6|Stmt=EXECUTE(-2147481418)|CpuTimeMS
       ↪ =0|SqlHash=eee20fa2ac13a4f93bd4503a87921024|peakMemoryBytes=0|SqlDigest=|TraceId=|
       ↪ WorkloadGroup=|FuzzyVaria
   bles=
```

Q3. 非主键查询能否使用到高并发点查的特殊优化

A：不能，高并发点查只针对于 key 列的等值查询，且查询中不能包含 join，嵌套子查询

Q4. useServerPrepStmts 在普通查询中是否有用

A：Prepared Statement 目前只在主键点查的情况下生效

Q5. 优化器选择需要进行全局设置吗

A：在使用 prepared statement 进行查询时，Doris 会选择性能最好的查询方式，不需要手动设置优化器

Q6. FE 成为瓶颈怎么处理？

A：如果占用过高 cpu，%CPU 过高，则在 jdbc url 开启以下配置

```
jdbc:mysql:loadbalance://[host1][:port],[host2][:port][,[host3][:port]]/${tbl_name}?
    ↪ useServerPrepStmts=true&cachePrepStmts=true&prepStmtCacheSize=500&prepStmtCacheSqlLimit
    ↪ =1024
```

- 打开 loadbalance 确保多个 FE 都能提供服务，并且 FE 数量越多越好 (每个实例都部署一个)
- 打开 useServerPrepStmts，减少 fe 解析、规划开销
- 打开 cachePrepStmts 客户端缓存 prepared statement，不用频繁向 FE 发送 prepared 请求
- 调整 prepStmtCacheSize，设置最大可缓存的查询模版数量 ()
- 调整 prepStmtCacheSqlLimit，设置单条缓存的 SQL 模版的最大长度

Q7. 存算分离下怎么优化查询性能？

A：

- `set global enable_snapshot_point_query = false`, 点查从 meta service 获取 version 会多一次额外的 RPC，并且 meta service 在高 QPS 场景下容易成为瓶颈，设置成 false 可以加速查询但是会降低数据可见性（需权衡性能和可见性）
- 配置 BE 参数 enable_file_cache_keep_base_compaction_output=1，使得 Base Compact 后的结果数据放入缓存，避免远程访问导致的查询抖动。

2.13.5 高效去重

2.13.5.1 BITMAP 精准去重

本文介绍如何通过 Bitmap 类型实现精确去重。

Bitmap 是一种高效的位图索引技术，它通过 bit 位来表示对应的数据是否存在。Bitmap 特别适用于需要高效执行集合操作（如并集、交集等）的场景，并且在内存使用上非常节约。使用 Bitmap 进行精确去重相比 Count distinct 去重：

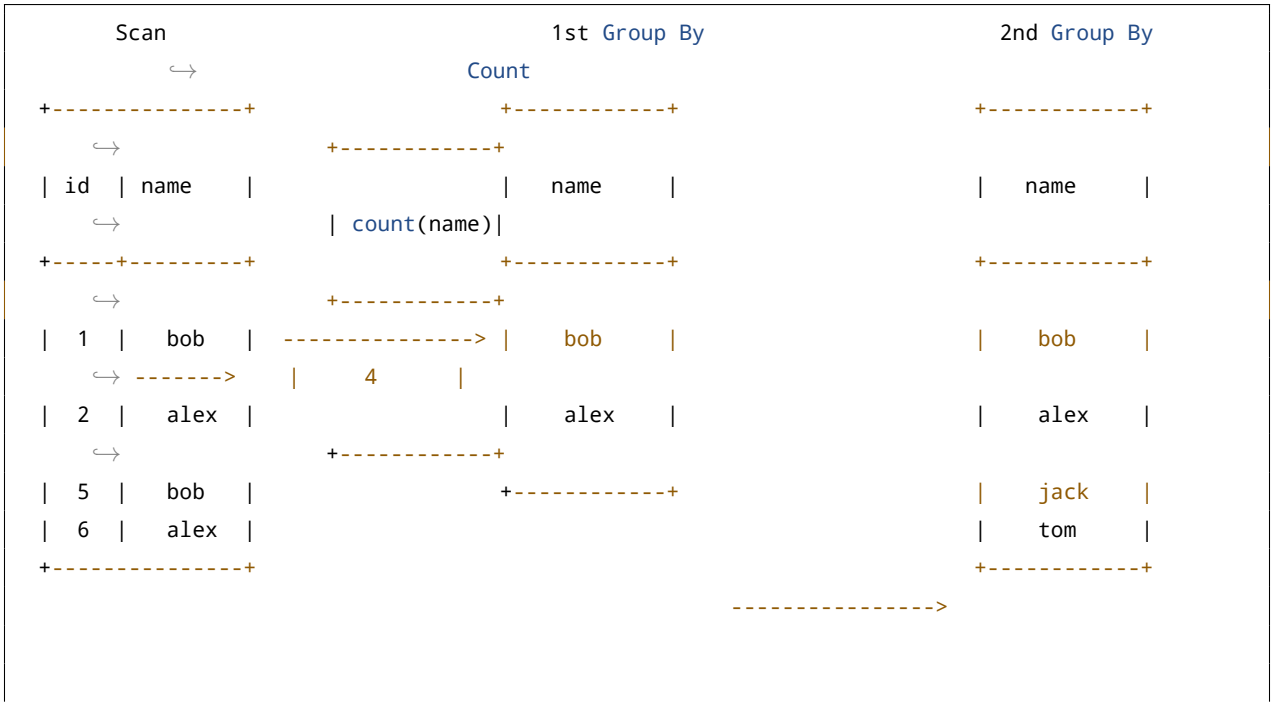
- 提高查询速度
- 减少内存/磁盘占用

2.13.5.1.1 Count Distinct 的实现

传统的精确去重依赖`count distinct`实现，表原始数据如下，假设要 name 列进行精确去重

id	name
1	bob
2	alex
3	jack
4	tom
5	bob
6	alex

Doris 在计算时`select count(distinct name)from t`。会按照下图进行计算，先根据 name 列 group by，计算一阶段去重，shuffle 之后二阶段进行去重，最终计算 count



```
+---------------+                  +------------+
| id  | name    |                  |   name     |
+-----+---------+  ---------------> +------------+
|  3  |  jack   |                  |    jack    |
|  4  |   tom   |                  |    tom     |
+-----+---------+                  +------------+
```

由于 Count Distinct 需要保存计算明细数据，并且需要进行 shuffle，当数据量增大时，查询也会越来越慢。用 Bitmap 来精确去重，可以解决 count distinct 在大量数据场景下的性能问题。

使用场景

在实际的业务场景中，当数据达到一定规模之后，通过 count distinct 去重的成本也越来越高。查询也会越来越慢。而使用 Bitmap 精确去重，就是为了解决 count distinct 在大量数据场景下的性能问题。Bitmap 将对应明细数据映射为 bit 位，放弃了明细数据的灵活性下，大幅度提升计算效率。所以在如下场景可以考虑利用 Bitmap 进行精确去重：

- 查询加速：Bitmap 利用位运算进行查询计算，性能表现良好
- 压缩存储：由于将明细数据压缩为了一个 bit 位，Bitmap 类型无论在磁盘还是内存上，资源消耗都远远低于明细数据

但 Bitmap 只能对 TINYINT，SMALLINT，INT 和 BIGINT 类型的数据进行精确去重。如想要使用 Bitmap 对其他类型的数据精确去重，则需要额外构建全局字典。Doris 使用了 RoaringBitmap 实现了 Bitmap 的精确去重，原理和细节可以参考RoaringBitmap。

2.13.5.1.2 使用 BITMAP 进行精确去重

创建表

1. 使用 Bitmap 去重的时候，需要在建表语句中将目标列类型设置成 Bitmap，聚合函数设置成 BITMAP_UNION
2. Bitmap 类型的列不能作为 Key 列使用

创建一张聚合表 `test_bitmap`。其中`id`列表示访问用户的 ID，这里添加了`uv`列类型为 BITMAP，表示使用聚合函数 BITMAP_UNION 来聚合数据，

```
create table test_bitmap(
        dt date,
        id int,
        name char(10),
        province char(10),
        os char(10),
        uv bitmap bitmap_union
)
Aggregate KEY (dt,id,name,province,os)
distributed by hash(id) buckets 10;
```

导入数据

示例数据如下（test_bitmap.csv），可以通过 Stream Load 导入。

```
2022-05-05,10001,测试 01,北京,windows
2022-05-05,10002,测试 01,北京,linux
2022-05-05,10003,测试 01,北京,macos
2022-05-05,10004,测试 01,河北,windows
2022-05-06,10001,测试 01,上海,windows
2022-05-06,10002,测试 01,上海,linux
2022-05-06,10003,测试 01,江苏,macos
2022-05-06,10004,测试 01,陕西,windows
```

Stream load 导入

```
curl --location-trusted -u root: -H "label:label_test_bitmap_load" \
    -H "column_separator:," \
    -H "columns:dt,id,name,province,os, uv=to_bitmap(id)" -T test_bitmap.csv http://fe_IP:8030/
        ↪ api/demo/test_bitmap/_stream_load
```

2.13.5.1.3 查询数据

Bitmap 列不允许直接查询原始值，只能通过 bitmap_union_count 的聚合函数进行查询。

求总的 UV

```
mysql> select bitmap_union_count(uv) from test_bitmap;
+----------------------+
| bitmap_union_count(`uv`) |
+----------------------+
|                    4 |
+----------------------+
1 row in set (0.00 sec)
```

等价于：

```
mysql> SELECT COUNT(DISTINCT pv) FROM test_bitmap;
+-----------------------+
| count(DISTINCT `uv`) |
+-----------------------+
|                     4 |
+-----------------------+
1 row in set (0.01 sec)
```

求每一天的 UV

```
mysql> select bitmap_union_count(uv) from test_bitmap group by dt;
+----------------------+
| bitmap_union_count(`uv`) |
```

```
+----------------------+
|                    4 |
|                    4 |
+----------------------+
2 rows in set (0.01 sec)
```

2.13.5.2 HLL 近似去重

2.13.5.2.1 使用场景

在实际的业务场景中，随着业务数据量越来越大，数据去重的压力也随之增大，当数据达到一定规模之后，使用精准去重的成本也越来越高。HLL 的特点是具有非常优异的空间复杂度 O(mloglogn)，时间复杂度为 O(n)，并且计算结果的误差可控制在 1%—2% 左右，误差与数据集大小以及所采用的哈希函数有关。

在业务可以接受的情况下，通过近似算法来实现快速去重降低计算压力是一个非常好的方式。

2.13.5.2.2 什么是 HyperLogLog

它是 LogLog 算法的升级版，作用是能够提供不精确的去重计数。其数学基础为伯努利试验。

假设硬币拥有正反两面，一次的上抛至落下，最终出现正反面的概率都是 50%。一直抛硬币，直到它出现正面为止，我们记录为一次完整的实验。

那么对于多次的伯努利试验，假设这个多次为 n 次。就意味着出现了 n 次的正面。假设每次伯努利试验所经历了的抛掷次数为 k。第一次伯努利试验，次数设为 k1，以此类推，第 n 次对应的是 kn。

其中，对于这 n 次伯努利试验中，必然会有一个最大的抛掷次数 k，例如抛了 12 次才出现正面，那么称这个为 k_max，代表抛了最多的次数。

伯努利试验容易得出有以下结论：

- n 次伯努利过程的投掷次数都不大于 k_max。
- n 次伯努利过程，至少有一次投掷次数等于 k_max

最终结合极大似然估算的方法，发现在 n 和 k_max 中存在估算关联：n = 2 ^ k_max。当我们只记录了 k_max 时，即可估算总共有多少条数据，也就是基数。

2.13.5.2.3 使用 HLL 进行近似去重

创建表

1. 使用 HLL 去重的时候，需要在建表语句中将目标列类型设置成 HLL，聚合函数设置成 HLL_UNION
2. HLL 类型的列不能作为 Key 列使用
3. 用户不需要指定长度及默认值，长度根据数据聚合程度系统内控制

```
create table test_hll(
        dt date,
        id int,
        name char(10),
        province char(10),
        os char(10),
        uv hll hll_union
)
Aggregate KEY (dt,id,name,province,os)
distributed by hash(id) buckets 10
PROPERTIES(
        "replication_num" = "1",
        "in_memory"="false"
);
```

导入数据

示例数据如下（test_hll.csv），这里通过 Stream Load 导入

```
2022-05-05,10001,测试 01,北京,windows
2022-05-05,10002,测试 01,北京,linux
2022-05-05,10003,测试 01,北京,macos
2022-05-05,10004,测试 01,河北,windows
2022-05-06,10001,测试 01,上海,windows
2022-05-06,10002,测试 01,上海,linux
2022-05-06,10003,测试 01,江苏,macos
2022-05-06,10004,测试 01,陕西,windows
```

Stream load 导入

```
curl --location-trusted -u root: -H "label:label_test_hll_load" \
    -H "column_separator:," \
    -H "columns:dt,id,name,province,os,uv=hll_hash(id)" -T test_hll.csv http://fe_IP:8030/api/
        ↪ demo/test_hll/_stream_load
```

导入结果如下

```
#### curl --location-trusted -u root: -H "label:label_test_hll_load"     -H "column_separator:,"
    ↪     -H "columns:dt,id,name,province,os, pv=hll_hash(id)" -T test_hll.csv http
    ↪ ://127.0.0.1:8030/api/demo/test_hll/_stream_load


{
    "TxnId": 693,
    "Label": "label_test_hll_load",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
```

```
    "NumberTotalRows": 8,
    "NumberLoadedRows": 8,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 320,
    "LoadTimeMs": 23,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 1,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 9,
    "CommitAndPublishTimeMs": 11
}
```

查询数据

HLL 列不允许直接查询原始值，只能通过 HLL 的聚合函数进行查询。

求总的 UV

```
mysql> select HLL_UNION_AGG(uv) from test_hll;
+----------------------+
| hll_union_agg(`uv`) |
+----------------------+
|                    4 |
+----------------------+
1 row in set (0.00 sec)
```

等价于：

```
mysql> SELECT COUNT(DISTINCT uv) FROM test_hll;
+-----------------------+
| count(DISTINCT `uv`) |
+-----------------------+
|                     4 |
+-----------------------+
1 row in set (0.01 sec)
```

求每一天的 UV

```
mysql> select HLL_UNION_AGG(uv) from test_hll group by dt;
+----------------------+
| hll_union_agg(`uv`) |
+----------------------+
|                    4 |
|                    4 |
+----------------------+
2 rows in set (0.01 sec)
```

2.13.5.2.4 相关函数

HLL_UNION_AGG(hll)：此函数为聚合函数，用于计算满足条件的所有数据的基数估算

HLL_CARDINALITY(hll)：此函数用于计算单条 HLL 列的基数估算

HLL_HASH(column_name)：生成 HLL 列类型，用于 Insert 或导入的时候，导入的使用见上文

HLL_EMPTY()：生成空 HLL 列，用于 `insert` 或导入数据时补充默认值

2.13.6 Colocation Join

Colocation Join 旨在为某些 Join 查询提供本地性优化，来减少数据在节点间的传输耗时，加速查询。本文档主要介绍 Colocation Join 的原理、实现、使用方式和注意事项。

注意：这个属性不会被 CCR 同步，如果这个表是被 CCR 复制而来的，即 PROPERTIES 中包含`is_being_synced = true`时，这个属性将会在这个表中被擦除。

2.13.6.1 名词解释

- Colocation Group（CG）：一个 CG 中会包含一张及以上的 Table。在同一个 Group 内的 Table 有着相同的 Colocation Group Schema，并且有着相同的数据分片分布。
- Colocation Group Schema（CGS）：用于描述一个 CG 中的 Table，和 Colocation 相关的通用 Schema 信息。包括分桶列类型，分桶数以及副本数等。

2.13.6.2 原理

Colocation Join 功能，是将一组拥有相同 CGS 的 Table 组成一个 CG。并保证这些 Table 对应的数据分片会落在同一个 BE 节点上。使得当 CG 内的表进行分桶列上的 Join 操作时，可以通过直接进行本地数据 Join，减少数据在节点间的传输耗时。

一个表的数据，最终会根据分桶列值 Hash、对桶数取模的后落在某一个分桶内。假设一个 Table 的分桶数为 8，则共有 `[0, 1, 2, 3, 4, 5, 6, 7]` 8 个分桶（Bucket），我们称这样一个序列为一个 BucketsSequence。每个 Bucket 内会有一个或多个数据分片（Tablet）。当表为单分区表时，一个 Bucket 内仅有一个 Tablet。如果是多分区表，则会有多个。

为了使得 Table 能够有相同的数据分布，同一 CG 内的 Table 必须保证以下属性相同：

1. 分桶列和分桶数

分桶列，即在建表语句中 `DISTRIBUTED BY HASH(col1, col2, ...)` 中指定的列。分桶列决定了一张表的数据通过哪些列的值进行 Hash 划分到不同的 Tablet 中。同一 CG 内的 Table 必须保证分桶列的类型和数量完全一致，并且桶数一致，才能保证多张表的数据分片能够一一对应的进行分布控制。

2. 副本数

同一个 CG 内所有表的所有分区（Partition）的副本数必须一致。如果不一致，可能出现某一个 Tablet 的某一个副本，在同一个 BE 上没有其他的表分片的副本对应。

同一个 CG 内的表，分区的个数、范围以及分区列的类型不要求一致。

在固定了分桶列和分桶数后，同一个 CG 内的表会拥有相同的 BucketsSequence。而副本数决定了每个分桶内的 Tablet 的多个副本，存放在哪些 BE 上。假设 BucketsSequence 为 `[0, 1, 2, 3, 4, 5, 6, 7]`，BE 节点有 `[A, B, C, D]` 4 个。则一个可能的数据分布如下：

```
+---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
| 0 | | 1 | | 2 | | 3 | | 4 | | 5 | | 6 | | 7 |
+---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
| A | | B | | C | | D | | A | | B | | C | | D |
|   | |   | |   | |   | |   | |   | |   | |   |
| B | | C | | D | | A | | B | | C | | D | | A |
|   | |   | |   | |   | |   | |   | |   | |   |
| C | | D | | A | | B | | C | | D | | A | | B |
+---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
```

CG 内所有表的数据都会按照上面的规则进行统一分布，这样就保证了，分桶列值相同的数据都在同一个 BE 节点上，可以进行本地数据 Join。

2.13.6.3 使用方式

2.13.6.3.1 建表

建表时，可以在 PROPERTIES 中指定属性 `"colocate_with" = "group_name"`，表示这个表是一个 Colocation Join 表，并且归属于一个指定的 Colocation Group。

示例：

```
CREATE TABLE tbl (k1 int, v1 int sum)
DISTRIBUTED BY HASH(k1)
BUCKETS 8
PROPERTIES(
    "colocate_with" = "group1"
);
```

如果指定的 Group 不存在，则 Doris 会自动创建一个只包含当前这张表的 Group。如果 Group 已存在，则 Doris 会检查当前表是否满足 Colocation Group Schema。如果满足，则会创建该表，并将该表加入 Group。同时，表会根据已存在的 Group 中的数据分布规则创建分片和副本。Group 归属于一个 Database，Group 的名字在一个 Database 内唯一。在内部存储是 Group 的全名为 `dbId_groupName`，但用户只感知 groupName。

> 提示 2.0 版本中，Doris 支持了跨 Database 的 Group。

在建表时，需使用关键词 `__global__` 作为 Group 名称的前缀。如：

```
CREATE TABLE tbl (k1 int, v1 int sum)
DISTRIBUTED BY HASH(k1)
BUCKETS 8
PROPERTIES(
    "colocate_with" = "__global__group1"
);
```

__global__ 前缀的 Group 不再归属于一个 Database，其名称也是全局唯一的。

通过创建 Global Group，可以实现跨 Database 的 Colocate Join。

2.13.6.3.2 删表

当 Group 中最后一张表彻底删除后（彻底删除是指从回收站中删除。通常，一张表通过 DROP TABLE 命令删除后，会在回收站默认停留一天的时间后，再删除），该 Group 也会被自动删除。

2.13.6.3.3 查看 Group

以下命令可以查看集群内已存在的 Group 信息。

```
SHOW PROC '/colocation_group';

+-------------+--------------+--------------+------------+----------------+----------+----------+
| GroupId     | GroupName    | TableIds     | BucketsNum | ReplicationNum | DistCols | IsStable |
+-------------+--------------+--------------+------------+----------------+----------+----------+
| 10005.10008 | 10005_group1 | 10007, 10040 | 10         | 3              | int(11)  | true     |
+-------------+--------------+--------------+------------+----------------+----------+----------+
```

- GroupId：一个 Group 的全集群唯一标识，前半部分为 db id，后半部分为 group id。
- GroupName：Group 的全名。
- TabletIds：该 Group 包含的 Table 的 id 列表。
- BucketsNum：分桶数。
- ReplicationNum：副本数。
- DistCols：Distribution columns，即分桶列类型。
- IsStable：该 Group 是否稳定（稳定的定义，见 Colocation 副本均衡和修复 一节）。

通过以下命令可以进一步查看一个 Group 的数据分布情况：

```
SHOW PROC '/colocation_group/10005.10008';

+-------------+---------------------+
| BucketIndex | BackendIds          |
```

```
+-------------+---------------------+
| 0           | 10004, 10002, 10001 |
| 1           | 10003, 10002, 10004 |
| 2           | 10002, 10004, 10001 |
| 3           | 10003, 10002, 10004 |
| 4           | 10002, 10004, 10003 |
| 5           | 10003, 10002, 10001 |
| 6           | 10003, 10004, 10001 |
| 7           | 10003, 10004, 10002 |
+-------------+---------------------+
```

- BucketIndex：分桶序列的下标。
- BackendIds：分桶中数据分片所在的 BE 节点 id 列表。

> 备注以上命令需要 ADMIN 权限。暂不支持普通用户查看。

2.13.6.3.4 修改表 Colocate Group 属性

可以对一个已经创建的表，修改其 Colocation Group 属性。示例：

```
ALTER TABLE tbl SET ("colocate_with" = "group2");
```

- 如果该表之前没有指定过 Group，则该命令检查 Schema，并将该表加入到该 Group（Group 不存在则会创建）。
- 如果该表之前有指定其他 Group，则该命令会先将该表从原有 Group 中移除，并加入新 Group（Group 不存在则会创建）。

也可以通过以下命令，删除一个表的 Colocation 属性：

```
ALTER TABLE tbl SET ("colocate_with" = "");
```

2.13.6.3.5 其他相关操作

当对一个具有 Colocation 属性的表进行增加分区（ADD PARTITION）、修改副本数时，Doris 会检查修改是否会违反 Colocation Group Schema，如果违反则会拒绝。

2.13.6.4 Colocation 副本均衡和修复

Colocation 表的副本分布需要遵循 Group 中指定的分布，所以在副本修复和均衡方面和普通分片有所区别。

Group 自身有一个 Stable 属性，当 Stable 为 true 时，表示当前 Group 内的表的所有分片没有正在进行变动，Colocation 特性可以正常使用。当 Stable 为 false 时（Unstable），表示当前 Group 内有部分表的分片正在做修复或迁移，此时，相关表的 Colocation Join 将退化为普通 Join。

2.13.6.4.1 副本修复

副本只能存储在指定的 BE 节点上。所以当某个 BE 不可用时（宕机、Decommission 等），需要寻找一个新的 BE 进行替换。Doris 会优先寻找负载最低的 BE 进行替换。替换后，该 Bucket 内的所有在旧 BE 上的数据分片都要做修复。迁移过程中，Group 被标记为 Unstable。

2.13.6.4.2 副本均衡

Doris 会尽力将 Colocation 表的分片均匀分布在所有 BE 节点上。对于普通表的副本均衡，是以单副本为粒度的，即单独为每一个副本寻找负载较低的 BE 节点即可。而 Colocation 表的均衡是 Bucket 级别的，即一个 Bucket 内的所有副本都会一起迁移。我们采用一个简单的均衡算法，即在不考虑副本实际大小，而只根据副本数量，将 BucketsSequence 均匀的分布在所有 BE 上。具体算法可以参阅 `ColocateTableBalancer.java` 中的代码注释。

> 注意 - 注 1：当前的 Colocation 副本均衡和修复算法，对于异构部署的 Doris 集群效果可能不佳。所谓异构部署，即 BE 节点的磁盘容量、数量、磁盘类型（SSD 和 HDD）不一致。在异构部署情况下，可能出现小容量的 BE 节点和大容量的 BE 节点存储了相同的副本数量。
>
> - 注 2：当一个 Group 处于 Unstable 状态时，其中的表的 Join 将退化为普通 Join。此时可能会极大降低集群的查询性能。如果不希望系统自动均衡，可以设置 FE 的配置项 `disable_colocate_balance` 来禁止自动均衡。然后在合适的时间打开即可。（具体参阅 高级操作 一节）

2.13.6.5 查询

对 Colocation 表的查询方式和普通表一样，用户无需感知 Colocation 属性。如果 Colocation 表所在的 Group 处于 Unstable 状态，将自动退化为普通 Join。

举例说明：

表 1：

```
CREATE TABLE `tbl1` (
    `k1` date NOT NULL COMMENT "",
    `k2` int(11) NOT NULL COMMENT "",
    `v1` int(11) SUM NOT NULL COMMENT ""
) ENGINE=OLAP
AGGREGATE KEY(`k1`, `k2`)
PARTITION BY RANGE(`k1`)
(
    PARTITION p1 VALUES LESS THAN ('2019-05-31'),
    PARTITION p2 VALUES LESS THAN ('2019-06-30')
)
DISTRIBUTED BY HASH(`k2`) BUCKETS 8
PROPERTIES (
    "colocate_with" = "group1"
);
```

表 2：

```
CREATE TABLE `tbl2` (
    `k1` datetime NOT NULL COMMENT "",
    `k2` int(11) NOT NULL COMMENT "",
    `v1` double SUM NOT NULL COMMENT ""
) ENGINE=OLAP
AGGREGATE KEY(`k1`, `k2`)
DISTRIBUTED BY HASH(`k2`) BUCKETS 8
PROPERTIES (
    "colocate_with" = "group1"
);
```

查看查询计划：

```
DESC SELECT * FROM tbl1 INNER JOIN tbl2 ON (tbl1.k2 = tbl2.k2);


+----------------------------------------------------+
| Explain String                                     |
+----------------------------------------------------+
| PLAN FRAGMENT 0                                    |
|  OUTPUT EXPRS:`tbl1`.`k1` |                        |
|   PARTITION: RANDOM                                |
|                                                    |
|   RESULT SINK                                      |
|                                                    |
|   2:HASH JOIN                                      |
|   |  join op: INNER JOIN                           |
|   |  hash predicates:                              |
|   |  colocate: true                                |
|   |    `tbl1`.`k2` = `tbl2`.`k2`                   |
|   |  tuple ids: 0 1                                |
|   |                                                |
|   |----1:OlapScanNode                              |
|   |       TABLE: tbl2                              |
|   |       PREAGGREGATION: OFF. Reason: null        |
|   |       partitions=0/1                           |
|   |       rollup: null                             |
|   |       buckets=0/0                              |
|   |       cardinality=-1                           |
|   |       avgRowSize=0.0                           |
|   |       numNodes=0                               |
|   |       tuple ids: 1                             |
|   |                                                |
|   0:OlapScanNode                                   |
|      TABLE: tbl1                                   |
```

```
|      PREAGGREGATION: OFF. Reason: No AggregateInfo |
|      partitions=0/2                                |
|      rollup: null                                  |
|      buckets=0/0                                   |
|      cardinality=-1                                |
|      avgRowSize=0.0                                |
|      numNodes=0                                    |
|      tuple ids: 0                                  |
+----------------------------------------------------+
```

如果 Colocation Join 生效，则 Hash Join 节点会显示 `colocate: true`。

如果没有生效，则查询计划如下：

```
+----------------------------------------------------+
| Explain String                                     |
+----------------------------------------------------+
| PLAN FRAGMENT 0                                    |
|  OUTPUT EXPRS:`tbl1`.`k1` |                        |
|   PARTITION: RANDOM                                |
|                                                    |
|   RESULT SINK                                      |
|                                                    |
|   2:HASH JOIN                                      |
|   |  join op: INNER JOIN (BROADCAST)               |
|   |  hash predicates:                              |
|   |  colocate: false, reason: group is not stable  |
|   |    `tbl1`.`k2` = `tbl2`.`k2`                   |
|   |  tuple ids: 0 1                                |
|   |                                                |
|   |----3:EXCHANGE                                  |
|   |       tuple ids: 1                             |
|   |                                                |
|   0:OlapScanNode                                   |
|      TABLE: tbl1                                   |
|      PREAGGREGATION: OFF. Reason: No AggregateInfo |
|      partitions=0/2                                |
|      rollup: null                                  |
|      buckets=0/0                                   |
|      cardinality=-1                                |
|      avgRowSize=0.0                                |
|      numNodes=0                                    |
|      tuple ids: 0                                  |
|                                                    |
| PLAN FRAGMENT 1                                    |
|  OUTPUT EXPRS:                                     |
```

```
|   PARTITION: RANDOM                                  |
|                                                      |
|   STREAM DATA SINK                                   |
|     EXCHANGE ID: 03                                  |
|     UNPARTITIONED                                    |
|                                                      |
|   1:OlapScanNode                                     |
|      TABLE: tbl2                                     |
|      PREAGGREGATION: OFF. Reason: null               |
|      partitions=0/1                                  |
|      rollup: null                                    |
|      buckets=0/0                                     |
|      cardinality=-1                                  |
|      avgRowSize=0.0                                  |
|      numNodes=0                                      |
|      tuple ids: 1                                    |
+------------------------------------------------------+
```

HASH JOIN 节点会显示对应原因：`colocate: false, reason: group is not stable`。同时会有一个 EXCHANGE 节点生成。

2.13.6.6 高级操作

2.13.6.6.1 FE 配置项

- disable_colocate_relocate

是否关闭 Doris 的自动 Colocation 副本修复。默认为 false，即不关闭。该参数只影响 Colocation 表的副本修复，不影响普通表。

- disable_colocate_balance

是否关闭 Doris 的自动 Colocation 副本均衡。默认为 false，即不关闭。该参数只影响 Colocation 表的副本均衡，不影响普通表。

以上参数可以动态修改，设置方式请参阅 `HELP SHOW CONFIG;` 和 `HELP SET CONFIG;`。

- disable_colocate_join

是否关闭 Colocation Join 功能。在 0.10 及之前的版本，默认为 true，即关闭。在之后的某个版本中将默认为 false，即开启。

- use_new_tablet_scheduler

在 0.10 及之前的版本中，新的副本调度逻辑与 Colocation Join 功能不兼容，所以在 0.10 及之前版本，如果 `disable_colocate_join = false`，则需设置 `use_new_tablet_scheduler = false`，即关闭新的副本调度器。之后的版本中，`use_new_tablet_scheduler` 将衡为 true。

2.13.6.6.2 HTTP Restful API

Doris 提供了几个和 Colocation Join 有关的 HTTP Restful API，用于查看和修改 Colocation Group。

该 API 实现在 FE 端，使用 `fe_host:fe_http_port` 进行访问。需要 ADMIN 权限。

1. 查看集群的全部 Colocation 信息

```
GET /api/colocate

返回以 Json 格式表示内部 Colocation 信息。

{
    "msg": "success",
 "code": 0,
 "data": {
     "infos": [
         ["10003.12002", "10003_group1", "10037, 10043", "1", "1", "int(11)", "true"]
     ],
     "unstableGroupIds": [],
     "allGroupIds": [{
         "dbId": 10003,
         "grpId": 12002
     }]
 },
 "count": 0
}
```

2. 将 Group 标记为 Stable 或 Unstable

- 标记为 Stable

```
  DELETE /api/colocate/group_stable?db_id=10005&group_id=10008

  返回：200
```

- 标记为 Unstable

```
  POST /api/colocate/group_stable?db_id=10005&group_id=10008

  返回：200
```

3. 设置 Group 的数据分布

该接口可以强制设置某一 Group 的数分布。

```
POST /api/colocate/bucketseq?db_id=10005&group_id=10008

Body:
[[10004,10002],[10003,10002],[10002,10004],[10003,10002],[10002,10004],[10003,10002],[10003,10004],[10003,1000
    ↪

返回 200
```

其中 Body 是以嵌套数组表示的 BucketsSequence 以及每个 Bucket 中分片分布所在 BE 的 id。

注意，使用该命令，可能需要将 FE 的配置 `disable_colocate_relocate` 和 `disable_colocate_balance` 设为 true。即关闭系统自动的 Colocation 副本修复和均衡。否则可能在修改后，会被系统自动重置。

2.13.7 Hints

2.13.7.1 Hint 概述

数据库 Hint 是一种查询优化技术，用于指导数据库查询优化器如何生成指定的计划。通过提供 Hint，用户可以对查询优化器的默认行为进行微调，以期望获得更好的性能或满足特定需求。> 注意 > 当前 Doris 已经具备良好的开箱即用的能力，在绝大多数场景下，Doris 会自适应的优化各种场景下的性能，无需用户来手工控制 hint 来进行业务调优。本章介绍的内容主要面向专业调优人员，业务人员仅做简单了解即可。

2.13.7.1.1 Hint 分类

Doris 目前支持以下几种 hint 类型，包括 leading hint，ordered hint，distribute hint 等：

- Leading Hint：用于指定 join order 为 leading 中提供的 order 顺序；
- Ordered Hint：一种特定的 leading hint, 用于指定 join order 为原始文本序；
- Distribute Hint：用于指定 join 的数据分发方式为 shuffle 还是 broadcast。

2.13.7.1.2 Hint 示例

假设有一个包含大量数据的表，而在某些特定情况下，你了解到在一个查询中，表的连接顺序可能会影响查询性能。此时，Leading Hint 允许你指定希望优化器遵循的表连接顺序。

以下面 SQL 查询为例，若执行效率不理想，我们希望调整 join 顺序，同时不改变原始 SQL，以免影响用户原始查询逻辑，并达到调优目的。

```
mysql> explain shape plan select * from t1 join t2 on t1.c1 = c2;
+----------------------------------------+
| Explain String                         |
+----------------------------------------+
| PhysicalResultSink                     |
| --PhysicalDistribute                   |
| ----PhysicalProject                    |
| ------hashJoin[INNER_JOIN](\#) |
```

```
| --------PhysicalOlapScan[t2]              |
| --------PhysicalDistribute                |
| ----------PhysicalOlapScan[t1]            |
+-------------------------------------------+
```

此时，我们可以使用 Leading Hint 来任意改变 t1 和 t2 的 Join 顺序。例如：

```
mysql> explain shape plan select  /*+ leading(t2 t1) */ * from t1 join t2 on t1.c1 = c2;
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| Explain String(Nereids Planner)
    ↪                                                                                                    |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| PhysicalResultSink
    ↪                                                                                                     |
| --PhysicalDistribute
    ↪                                                                                                   |
| ----PhysicalProject
    ↪                                                                                                    |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() build RFs:RF0 c1
    ↪ ->[c2] |
| --------PhysicalOlapScan[t2] apply RFs: RF0
    ↪                                                                |
| --------PhysicalDistribute
    ↪                                                                          |
| ----------PhysicalOlapScan[t1]
    ↪                                                                         |
|
    ↪
    ↪ |
| Hint log:
    ↪
    ↪ |
| Used: leading(t2 t1)
    ↪                                                                                                   |
| UnUsed:
    ↪
    ↪ |
| SyntaxError:
    ↪                                                                                                       |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
```

在此示例中，使用了 `/*+ leading(t2 t1)*/` 的 Leading Hint。Leading Hint 会告知优化器在执行计划中使用指定表（t2）作为驱动表，并将其置于 (t1) 之前。

2.13.7.1.3 Hint Log

Hint Log 主要用于在执行 EXPLAIN 时显示提示是否生效。其显示位置通常位于 EXPLAIN 输出的最下方。

Hint Log 分为三个状态：

```
+---------------------------------+
| Hint log:                       |
| Used:                           |
| UnUsed:                         |
| SyntaxError:                    |
+---------------------------------+
```

- Used：表明该提示生效了。
- UnUsed 和 SyntaxError：都表明该提示未生效。SyntaxError 表示提示语法使用错误或该语法不支持，同时会附加不支持的原因信息。

用户可以通过 Hint Log 查看生效情况以及未生效原因，便于调整和验证。

2.13.7.1.4 总结

Hint 是手动管理执行计划的强大工具。当前 Doris 支持的 leading hint, ordered hint, distribute hint 等，可以支撑用户手动管理 join order, shuffle 方式以及其他变量配置，给用户提供更方便有效的运维能力。

2.13.7.2 Leading Hint

Leading Hint 是一种强大的查询优化技术，允许用户指导 Doris 优化器确定查询计划中的表连接顺序。正确使用 Leading Hint 可以显著提高复杂查询的性能。本文将详细介绍如何在 Doris 中使用 Leading Hint 来控制 join 顺序。

2.13.7.2.1 常规 Leading Hint

语法

Leading Hint 允许指定希望优化器遵循的表连接顺序。在 Doris 里面，Leading Hint 的基本语法如下：

```
SELECT /*+ LEADING(tablespec [tablespec]...) */ ...
```

其中需要注意的是：

- Leading Hint 由 `/*+` 和 `*/` 包围，并置于 SQL 语句中 SELECT 关键字之后。
- `tablespec` 是表名或表别名，至少需要指定两个表。
- 多个表之间用空格或’,’ 分隔。
- 可以使用大括号 `{}` 来显式地指定 Join Tree 的形状。

举例说明：

```
mysql> explain shape plan select /*+ leading(t2 t1) */ * from t1 join t2 on c1 = c2;
+----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                  |
+----------------------------------------------------------------------------------+
| PhysicalResultSink                                                               |
| --PhysicalDistribute[DistributionSpecGather]                                     |
| ----PhysicalProject                                                              |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()      |
| --------PhysicalOlapScan[t2]                                                     |
| --------PhysicalDistribute[DistributionSpecHash]                                |
| ----------PhysicalOlapScan[t1]                                                   |
|                                                                                  |
| Hint log:                                                                        |
| Used: leading(t2 t1)                                                             |
| UnUsed:                                                                          |
| SyntaxError:                                                                     |
+----------------------------------------------------------------------------------+
```

当 Leading Hint 不生效的时候会走正常的流程生成计划，EXPLAIN 会显示使用的 Hint 是否生效，主要分三种来显示：

状态	描述
Used	Leading Hint 正常生效
Unused	这里不支持的情况包含 Leading Hint 指定的 join order 与原 SQL 不等价或本版本暂不支持特性（详见限制）
SyntaxError	指 Leading Hint 语法错误，如找不到对应的表等

1. **Leading Hint 语法默认构造出左深树：**

```
mysql> explain shape plan select /*+ leading(t1 t2 t3) */ * from t1 join t2 on c1 = c2 join
    ↪ t3 on c2=c3;
+----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                  |
+----------------------------------------------------------------------------------+
| PhysicalResultSink                                                               |
| --PhysicalDistribute[DistributionSpecGather]                                     |
| ----PhysicalProject                                                              |
| ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()      |
| --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()    |
| ----------PhysicalOlapScan[t1]                                                   |
| ----------PhysicalDistribute[DistributionSpecHash]                               |
| ------------PhysicalOlapScan[t2]                                                 |
| --------PhysicalDistribute[DistributionSpecHash]                                |
| ----------PhysicalOlapScan[t3]                                                   |
|                                                                                  |
```

```
| Hint log:                                                                    |
| Used: leading(t1 t2 t3)                                                      |
| UnUsed:                                                                      |
| SyntaxError:                                                                 |
+------------------------------------------------------------------------------+
```

2. 同时允许使用大括号指定 Join 树形状：

```
mysql> explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 join t2 on c1 = c2
    -> join t3 on c2=c3;
+-----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                   |
+-----------------------------------------------------------------------------------+
| PhysicalResultSink                                                                |
| --PhysicalDistribute[DistributionSpecGather]                                      |
| ----PhysicalProject                                                               |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()       |
| --------PhysicalOlapScan[t1]                                                      |
| --------PhysicalDistribute[DistributionSpecHash]                                 |
| ----------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()  |
| ------------PhysicalOlapScan[t2]                                                  |
| ------------PhysicalDistribute[DistributionSpecHash]                             |
| --------------PhysicalOlapScan[t3]                                               |
|                                                                                   |
| Hint log:                                                                         |
| Used: leading(t1 { t2 t3 })                                                       |
| UnUsed:                                                                           |
| SyntaxError:                                                                      |
+-----------------------------------------------------------------------------------+
```

3. 当有 View 作为别名参与 JoinReorder 的时候可以指定对应的 View 作为 Leading Hint 的参数。例：

```
mysql> explain shape plan select /*+ leading(alias t1) */ count(*) from t1 join (select c2
    -> from t2 join t3 on t2.c2 = t3.c3) as alias on t1.c1 = alias.c2;
  +-----------------------------------------------------------------------------------------+
  | Explain String(Nereids Planner)                                                         |
  +-----------------------------------------------------------------------------------------+
  | PhysicalResultSink                                                                      |
  | --hashAgg[GLOBAL]                                                                       |
  | ----PhysicalDistribute[DistributionSpecGather]                                          |
  | ------hashAgg[LOCAL]                                                                    |
  | --------PhysicalProject                                                                 |
  | ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = alias.c2)) otherCondition=()     |
  | ------------PhysicalProject                                                             |
  | --------------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()    |
  | ----------------PhysicalProject                                                        |
```

```
| ------------------PhysicalOlapScan[t2]                                                 |
| ----------------PhysicalDistribute[DistributionSpecHash]                             |
| ------------------PhysicalProject                                                    |
| --------------------PhysicalOlapScan[t3]                                            |
| ------------PhysicalDistribute[DistributionSpecHash]                                |
| --------------PhysicalProject                                                        |
| ----------------PhysicalOlapScan[t1]                                                |
|                                                                                      |
| Hint log:                                                                            |
| Used: leading(alias t1)                                                              |
| UnUsed:                                                                              |
| SyntaxError:                                                                         |
+--------------------------------------------------------------------------------------+
```

案例

基础场景

1. 建表语句如下：

```
CREATE DATABASE testleading;
USE testleading;

create table t1 (c1 int, c11 int) distributed by hash(c1) buckets 3 properties('replication_
    ↪ num' = '1');
create table t2 (c2 int, c22 int) distributed by hash(c2) buckets 3 properties('replication_
    ↪ num' = '1');
create table t3 (c3 int, c33 int) distributed by hash(c3) buckets 3 properties('replication_
    ↪ num' = '1');
create table t4 (c4 int, c44 int) distributed by hash(c4) buckets 3 properties('replication_
    ↪ num' = '1');
```

2. 原始 plan：

```
mysql> explain shape plan select * from t1 join t2 on t1.c1 = c2;
+-------------------------------------------+
| Explain String                            |
+-------------------------------------------+
| PhysicalResultSink                        |
| --PhysicalDistribute                      |
| ----PhysicalProject                       |
| ------hashJoin[INNER_JOIN](\#) |
| --------PhysicalOlapScan[t2]              |
| --------PhysicalDistribute                |
```

```
| ----------PhysicalOlapScan[t1]           |
+-------------------------------------------+
```

3. 当我们需要交换 t1 和 t2 的 join 顺序时，只需在前面加上 `leading(t2 t1)` 即可。在执行 `explain` 时，会显示是否使用了这个 hint。如下 Leading plan：Used 表示 Hint 正常生效

```
mysql> explain shape plan select /*+ leading(t2 t1) */ * from t1 join t2 on c1 = c2;
+-----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                   |
+-----------------------------------------------------------------------------------+
| PhysicalResultSink                                                                |
| --PhysicalDistribute[DistributionSpecGather]                                      |
| ----PhysicalProject                                                               |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()       |
| --------PhysicalOlapScan[t2]                                                      |
| --------PhysicalDistribute[DistributionSpecHash]                                  |
| ----------PhysicalOlapScan[t1]                                                    |
|                                                                                   |
| Hint log:                                                                         |
| Used: leading(t2 t1)                                                              |
| UnUsed:                                                                           |
| SyntaxError:                                                                      |
+-----------------------------------------------------------------------------------+
```

4. 如果 Leading Hint 存在语法错误，`explain` 时会在 `SyntaxError` 里显示相应信息，但计划仍能照常生成，只是不会使用 Leading 而已。例如：

```
mysql> explain shape plan select /*+ leading(t2 t3) */ * from t1 join t2 on t1.c1 = c2;
+-----------------------------------------------------------+
| Explain String                                            |
+-----------------------------------------------------------+
| PhysicalResultSink                                        |
| --PhysicalDistribute                                      |
| ----PhysicalProject                                       |
| ------hashJoin[INNER_JOIN](\#)                |
| --------PhysicalOlapScan[t1]                              |
| --------PhysicalDistribute                                |
| ----------PhysicalOlapScan[t2]                            |
|                                                           |
| Used:                                                     |
| UnUsed:                                                   |
| SyntaxError: leading(t2 t3) Msg:can not find table: t3 |
+-----------------------------------------------------------+
```

扩展场景

1. 左深树

上文我们提及，Doris 在查询语句不使用任何括号的情况下，Leading 会默认生成左深树。

```
mysql> explain shape plan select /*+ leading(t1 t2 t3) */ * from t1 join t2 on t1.c1 = c2
    ↪ join t3 on c2 = c3;
+-----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                   |
+-----------------------------------------------------------------------------------+
| PhysicalResultSink                                                                |
| --PhysicalDistribute[DistributionSpecGather]                                      |
| ----PhysicalProject                                                               |
| ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()      |
| --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()    |
| ----------PhysicalOlapScan[t1]                                                    |
| ----------PhysicalDistribute[DistributionSpecHash]                                |
| ------------PhysicalOlapScan[t2]                                                  |
| --------PhysicalDistribute[DistributionSpecHash]                                  |
| ----------PhysicalOlapScan[t3]                                                    |
|                                                                                   |
| Hint log:                                                                         |
| Used: leading(t1 t2 t3)                                                           |
| UnUsed:                                                                           |
| SyntaxError:                                                                      |
+-----------------------------------------------------------------------------------+
```

2. 右深树

当需要将计划的形状做成右深树、Bushy 树或者 zig-zag 树时，只需加上大括号来限制 plan 的形状即可，无需像 Oracle 使用 swap 从左深树一步步调整。

```
mysql> explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 join t2 on t1.c1 = c2
    ↪ join t3 on c2 = c3;
+-----------------------------------------------+
| Explain String                                |
+-----------------------------------------------+
| PhysicalResultSink                            |
| --PhysicalDistribute                          |
| ----PhysicalProject                           |
| ------hashJoin[INNER_JOIN](\#)     |
| --------PhysicalOlapScan[t1]                  |
| --------PhysicalDistribute                    |
| ----------hashJoin[INNER_JOIN](\#) |
| ------------PhysicalOlapScan[t2]              |
```

```
| ------------PhysicalDistribute                |
| --------------PhysicalOlapScan[t3]            |
|                                               |
| Used: leading(t1 { t2 t3 })                   |
| UnUsed:                                       |
| SyntaxError:                                  |
+-----------------------------------------------+
```

3. Bushy 树

```
mysql> explain shape plan select /*+ leading({t1 t2} {t3 t4}) */ * from t1 join t2 on t1.c1 =
    ↪  c2 join t3 on c2 = c3 join t4 on c3 = c4;
+-----------------------------------------------+
| Explain String                                |
+-----------------------------------------------+
| PhysicalResultSink                            |
| --PhysicalDistribute                          |
| ----PhysicalProject                           |
| ------hashJoin[INNER_JOIN](\#)     |
| --------hashJoin[INNER_JOIN](\#)   |
| ----------PhysicalOlapScan[t1]                |
| ----------PhysicalDistribute                  |
| ------------PhysicalOlapScan[t2]              |
| --------PhysicalDistribute                    |
| ----------hashJoin[INNER_JOIN](\#) |
| ------------PhysicalOlapScan[t3]              |
| ------------PhysicalDistribute                |
| --------------PhysicalOlapScan[t4]            |
|                                               |
| Used: leading({ t1 t2 } { t3 t4 })            |
| UnUsed:                                       |
| SyntaxError:                                  |
+-----------------------------------------------+
```

4. zig-zag 树

```
mysql> explain shape plan select /*+ leading(t1 {t2 t3} t4) */ * from t1 join t2 on t1.c1 =
    ↪ c2 join t3 on c2 = c3 join t4 on c3 = c4;
+-----------------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                         |
+-----------------------------------------------------------------------------------------+
| PhysicalResultSink                                                                      |
| --PhysicalDistribute[DistributionSpecGather]                                            |
| ----PhysicalProject                                                                     |
```

```
| ------hashJoin[INNER_JOIN] hashCondition=((t3.c3 = t4.c4)) otherCondition=()         |
| --------PhysicalDistribute[DistributionSpecHash]                                    |
| ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()     |
| ------------PhysicalOlapScan[t1]                                                    |
| ------------PhysicalDistribute[DistributionSpecHash]                                |
| --------------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=() |
| ----------------PhysicalOlapScan[t2]                                               |
| ----------------PhysicalDistribute[DistributionSpecHash]                           |
| ------------------PhysicalOlapScan[t3]                                            |
| --------PhysicalDistribute[DistributionSpecHash]                                    |
| ----------PhysicalOlapScan[t4]                                                      |
|                                                                                     |
| Hint log:                                                                           |
| Used: leading(t1 { t2 t3 } t4)                                                      |
| UnUsed:                                                                             |
| SyntaxError:                                                                        |
+-------------------------------------------------------------------------------------+
```

5. Non-inner Join

当遇到非 inner-join（如 Outer Join 或 Semi/Anti Join）时，Leading Hint 会根据原始 SQL 语义自动推导各个 Join 的方式。若 Leading Hint 与原始 SQL 语义不同或无法生成，则会将其放入 UnUsed 中，但这并不影响计划正常流程的生成。

以下是一个不能交换的例子：

```
-------- test outer join which can not swap
-- t1 leftjoin (t2 join t3 on (P23)) on (P12) != (t1 leftjoin t2 on (P12)) join t3 on (P23)
mysql> explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 left join t2 on c1 = c2
    ↪  join t3 on c2 = c3;
+-------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                               |
+-------------------------------------------------------------------------------+
| PhysicalResultSink                                                            |
| --PhysicalDistribute[DistributionSpecGather]                                  |
| ----PhysicalProject                                                           |
| ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()   |
| --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
| ----------PhysicalOlapScan[t1]                                                |
| ----------PhysicalDistribute[DistributionSpecHash]                            |
| ------------PhysicalOlapScan[t2]                                              |
| --------PhysicalDistribute[DistributionSpecHash]                              |
| ----------PhysicalOlapScan[t3]                                                |
|                                                                               |
| Hint log:                                                                     |
| Used:                                                                         |
```

```
| UnUsed: leading(t1 { t2 t3 })                                                  |
| SyntaxError:                                                                   |
+--------------------------------------------------------------------------------+
```

下面是一些可以交换的例子和不能交换的例子，读者可自行验证。

```
-------- test outer join which can swap
-- (t1 leftjoin t2  on (P12)) innerjoin t3 on (P13) = (t1 innerjoin t3 on (P13)) leftjoin t2
    ↪  on (P12)
explain shape plan select * from t1 left join t2 on c1 = c2 join t3 on c1 = c3;
explain shape plan select /*+ leading(t1 t3 t2) */ * from t1 left join t2 on c1 = c2 join t3
    ↪ on c1 = c3;

-- (t1 leftjoin t2  on (P12)) leftjoin t3 on (P13) = (t1 leftjoin t3 on (P13)) leftjoin t2
    ↪ on (P12)
explain shape plan select * from t1 left join t2 on c1 = c2 left join t3 on c1 = c3;
explain shape plan select /*+ leading(t1 t3 t2) */ * from t1 left join t2 on c1 = c2 left
    ↪ join t3 on c1 = c3;

-- (t1 leftjoin t2  on (P12)) leftjoin t3 on (P23) = t1 leftjoin (t2  leftjoin t3 on (P23))
    ↪ on (P12)
select /*+ leading(t2 t3 t1) SWAP_INPUT(t1) */ * from t1 left join t2 on c1 = c2 left join t3
    ↪  on c2 = c3;
explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 left join t2 on c1 = c2 left
    ↪ join t3 on c2 = c3;
explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 left join t2 on c1 = c2 left
    ↪ join t3 on c2 = c3;

-------- test outer join which can not swap
--  t1 leftjoin (t2  join t3 on (P23)) on (P12) != (t1 leftjoin t2  on (P12)) join t3 on (P23
    ↪ )
-- eliminated to inner join
explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 left join t2 on c1 = c2 join
    ↪ t3 on c2 = c3;
explain graph select /*+ leading(t1 t2 t3) */ * from t1 left join (select * from t2 join t3
    ↪ on c2 = c3) on c1 = c2;

-- test semi join
explain shape plan select * from t1 where c1 in (select c2 from t2);
explain shape plan select /*+ leading(t2 t1) */ * from t1 where c1 in (select c2 from t2);

-- test anti join
explain shape plan select * from t1 where exists (select c2 from t2);
```

6. View

在涉及别名（Alias）的情况下，可以将别名作为一个完整独立的子树进行指定，并在这些子树内部根据文本序生成 Join 顺序

```
mysql>  explain shape plan select /*+ leading(alias t1) */ count(*) from t1 join (select c2
    ↪ from t2 join t3 on t2.c2 = t3.c3) as alias on t1.c1 = alias.c2;
+----------------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                        |
+----------------------------------------------------------------------------------------+
| PhysicalResultSink                                                                     |
| --hashAgg[GLOBAL]                                                                      |
| ----PhysicalDistribute[DistributionSpecGather]                                         |
| ------hashAgg[LOCAL]                                                                   |
| --------PhysicalProject                                                                |
| ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = alias.c2)) otherCondition=()    |
| ------------PhysicalProject                                                            |
| --------------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()   |
| ----------------PhysicalProject                                                        |
| ------------------PhysicalOlapScan[t2]                                                |
| ----------------PhysicalDistribute[DistributionSpecHash]                              |
| ------------------PhysicalProject                                                     |
| --------------------PhysicalOlapScan[t3]                                              |
| ------------PhysicalDistribute[DistributionSpecHash]                                  |
| --------------PhysicalProject                                                          |
| ----------------PhysicalOlapScan[t1]                                                  |
|                                                                                        |
| Hint log:                                                                              |
| Used: leading(alias t1)                                                                |
| UnUsed:                                                                                |
| SyntaxError:                                                                           |
+----------------------------------------------------------------------------------------+
```

2.13.7.2.2 Ordered Hint

Ordered hint 可以看做 leading hint 的一种特例，用于控制 join order 为文本序。

语法

Ordered Hint 的语法为 `/*+ ORDERED */`，放置在 SELECT 语句中的 SELECT 关键字之后，紧接着查询的其余部分。

案例

以下是一个使用 Ordered Hint 的示例：

```
mysql> explain shape plan select /*+ ORDERED */ t1.c1 from t2 join t1 on t1.c1 = t2.c2 join t3 on
    ↪  c2 = c3;
+------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                              |
+------------------------------------------------------------------------------+
```

```
| PhysicalResultSink                                                                |
| --PhysicalDistribute[DistributionSpecGather]                                      |
| ----PhysicalProject                                                               |
| ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()      |
| --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()    |
| ----------PhysicalProject                                                         |
| ------------PhysicalOlapScan[t2]                                                  |
| ----------PhysicalDistribute[DistributionSpecHash]                               |
| ------------PhysicalProject                                                       |
| --------------PhysicalOlapScan[t1]                                                |
| --------PhysicalDistribute[DistributionSpecHash]                                 |
| ----------PhysicalProject                                                         |
| ------------PhysicalOlapScan[t3]                                                  |
|                                                                                   |
| Hint log:                                                                         |
| Used: ORDERED                                                                     |
| UnUsed:                                                                           |
| SyntaxError:                                                                      |
+-----------------------------------------------------------------------------------+
```

与 Leading Hint 的关系：

当 Ordered Hint 和 Leading Hint 同时使用时，Ordered Hint 将优先于 Leading Hint。这意味着，即使指定了 Leading Hint，如果同时存在 Ordered Hint，查询计划将按照 Ordered Hint 的规则来执行，而 Leading Hint 将被忽略。以下是一个示例，展示了当两者同时使用时的情况：

```
mysql> explain shape plan select /*+ ORDERED LEADING(t1 t2 t3) */ t1.c1 from t2 join t1 on t1.c1
    ↪ = t2.c2 join t3 on c2 = c3;
  +-----------------------------------------------------------------------------------+
  | Explain String(Nereids Planner)                                                   |
  +-----------------------------------------------------------------------------------+
  | PhysicalResultSink                                                                |
  | --PhysicalDistribute[DistributionSpecGather]                                      |
  | ----PhysicalProject                                                               |
  | ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()      |
  | --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()    |
  | ----------PhysicalProject                                                         |
  | ------------PhysicalOlapScan[t2]                                                  |
  | ----------PhysicalDistribute[DistributionSpecHash]                               |
  | ------------PhysicalProject                                                       |
  | --------------PhysicalOlapScan[t1]                                                |
  | --------PhysicalDistribute[DistributionSpecHash]                                 |
  | ----------PhysicalProject                                                         |
  | ------------PhysicalOlapScan[t3]                                                  |
  |                                                                                   |
  | Hint log:                                                                         |
```

```
  | Used: ORDERED                                                                  |
  | UnUsed: leading(t1 t2 t3)                                                      |
  | SyntaxError:                                                                   |
  +--------------------------------------------------------------------------------+
```

2.13.7.2.3 总结

Leading Hint 是一个强大的手工控制 join order 的特性，在生产业务调优中应用广泛。使用好 leading hint 能够满足现场针对 join order 的调优需求，增加系统控制的灵活性。Ordered hint 是一种特殊的 leading hint，用于固定当前业务的 join order 为文本序，使用时需要注意和其他 Hint 之间的优先级关系。

2.13.7.3 Distribute Hint

2.13.7.3.1 概述

Distribute hint 用来控制 join 的 shuffle 方式。

2.13.7.3.2 语法

- 支持指定右表的 Distribute Type，分为 `[shuffle]` 和 `[broadcast]` 两种，需写在 Join 右表前面。
- 支持任意个 Distribute Hint。
- 当遇到无法正确生成计划的 Distribute Hint 时，系统不会显示错误，会按最大努力原则生效，最终以 EXPLAIN 显示的 Distribute 方式为准。

2.13.7.3.3 案例

与 Ordered Hint 混用

把 Join 顺序固定为文本序，然后再指定相应的 Join 预期使用的 Distribute 方式。例如：

使用前：

```
mysql> explain shape plan select count(*) from t1 join t2 on t1.c1 = t2.c2;
  +------------------------------------------------------------------------------------+
  | Explain String(Nereids Planner)                                                    |
  +------------------------------------------------------------------------------------+
  | PhysicalResultSink                                                                 |
  | --hashAgg[GLOBAL]                                                                  |
  | ----PhysicalDistribute[DistributionSpecGather]                                     |
  | ------hashAgg[LOCAL]                                                               |
  | --------PhysicalProject                                                            |
  | ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
  | ------------PhysicalProject                                                        |
  | --------------PhysicalOlapScan[t1]                                                 |
  | ------------PhysicalDistribute[DistributionSpecHash]                              |
  | --------------PhysicalProject                                                      |
```

```
  | ----------------PhysicalOlapScan[t2]                                                 |
  +-----------------------------------------------------------------------------------+
```

使用后：

```
mysql> explain shape plan select /*+ ordered */ count(*) from t2 join[broadcast] t1 on t1.c1 = t2
    ↪ .c2;
+-----------------------------------------------------------------------------------+
| Explain String(Nereids Planner)                                                   |
+-----------------------------------------------------------------------------------+
| PhysicalResultSink                                                                |
| --hashAgg[GLOBAL]                                                                 |
| ----PhysicalDistribute[DistributionSpecGather]                                    |
| ------hashAgg[LOCAL]                                                              |
| --------PhysicalProject                                                           |
| ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
| ------------PhysicalProject                                                       |
| --------------PhysicalOlapScan[t2]                                               |
| ------------PhysicalDistribute[DistributionSpecReplicated]                       |
| --------------PhysicalProject                                                     |
| ----------------PhysicalOlapScan[t1]                                             |
|                                                                                   |
| Hint log:                                                                         |
| Used: ORDERED                                                                     |
| UnUsed:                                                                           |
| SyntaxError:                                                                      |
+-----------------------------------------------------------------------------------+
```

Explain Shape Plan 里面会显示 Distribute 算子相关的信息。其中：

- DistributionSpecReplicated 表示该算子将对应的数据复制到所有 BE 节点；
- DistributionSpecGather 表示将数据 Gather 到 FE 节点；
- DistributionSpecHash 表示将数据按照特定的 hashKey 以及算法打散到不同的 BE 节点。

与 Leading Hint 混用

在编写 SQL 查询时，可以在使用 LEADING 提示的同时，为每个 JOIN 操作指定相应的 DISTRIBUTE 方式。以下是一个具体的例子，展示了如何在 SQL 查询中混合使用 Distribute Hint 和 Leading Hint。

```
explain shape plan
    select
        nation,
        o_year,
        sum(amount) as sum_profit
    from
        (
            select
```

```
            /*+ leading(orders shuffle {lineitem shuffle part} shuffle {supplier broadcast
                ↪ nation} shuffle partsupp) */
            n_name as nation,
            extract(year from o_orderdate) as o_year,
            l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity as amount
        from
            part,
            supplier,
            lineitem,
            partsupp,
            orders,
            nation
        where
            s_suppkey = l_suppkey
            and ps_suppkey = l_suppkey
            and ps_partkey = l_partkey
            and p_partkey = l_partkey
            and o_orderkey = l_orderkey
            and s_nationkey = n_nationkey
            and p_name like '%green%'
    ) as profit
group by
    nation,
    o_year
order by
    nation,
    o_year desc;
```

2.13.7.3.4 总结

Distribute hint 是常用的控制 join shuffle 方式的 hint，用于手工指定 shuffle 或者 broadcast 分发方式。使用好 Distribute hint 能够满足现场针对 join shuffle 方式的调优需求，增加系统控制的灵活性。

2.13.8 查询优化实践

2.13.8.1 计划调优

2.13.8.1.1 优化表 Schema 设计

概述

Schema 设计和调优中，表设计是其中重要的一部分，包括表引擎选择、分区分桶列选择、分区分桶大小设置、key 列和字段类型优化等。缺乏 Schema 设计的系统，有可能会导致数据倾斜等问题，不能充分利用系统并行和排序特性，从而影响 Doris 在业务系统中发挥真实的性能优势。

详细的设计原则可以参考数据表设计章节了解详细信息。本章将从实际案例的角度，展示几种典型场景下因 Schema 设计问题导致的性能瓶颈，并给出优化建议，供业务调优参考。

案例 1：表引擎选择

Doris 支持 Duplicate、Unique、Aggregate 三种表模型。其中，Unique 又可以进一步分为 Merge-On-Read（MOR）和 Merge-On-Write（MOW）两种。

这几种表模型的查询性能，由好到差依次为：Duplicate > MOW > MOR == Aggregate。因此，通常情况下，如果没有特殊需求，推荐使用 Duplicate 表，以获得更好的查询性能。

> 优化建议
>
> 当业务无数据更新需求，但对查询性能有较高要求时，推荐使用Duplicate 表。

案例 2：分桶列选择

Doris 支持对数据进行分桶操作，即依据 Schema 中预设的分桶键来分布数据，进而形成数据 Bucket。

选取恰当的分桶列，对于原始数据的合理分布至关重要，它能有效防止数据倾斜所引发的性能问题。同时，这也能最大化地利用 Doris 提供的 Colocate Join 和 Bucket Shuffle Join 特性，从而显著提升 Join 操作的性能。

以下面 t1 表的建表语句为例，当前分桶列选定为 c2。然而，在实际数据导入过程中，若 c2 列的值全部默认为 null，那么即便设定了 64 个分桶，实际上也只有一个分桶会包含所有数据。这种极端情况会导致严重的数据倾斜，进而产生性能瓶颈。

```
CREATE TABLE `t1` (
  `c1` INT NULL,
  `c2` INT NULL
) ENGINE=OLAP
DUPLICATE KEY(`c1`)
DISTRIBUTED BY HASH(`c2`) BUCKETS 64
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
insert into t1 select number, null from numbers ('number'='10000000');
```

针对上述情况，我们可以将分桶列从 c2 改为 c1，以实现数据的充分散列，并最大化地利用系统的并行处理能力，从而达到调优的目的。

因此，在 Schema 设计阶段，业务人员需要根据业务特性，提前进行合理的分桶列设计。例如，如果预先了解到 c2 列的业务含义中可能包含大量倾斜的值，如 null 或某些特定的值，那么就应该避免选择这些字段作为分桶列。相反，应该选择那些在业务含义上具有充分散列特性的字段，如用户 ID，作为分桶列。在性能问题排查阶段，可以使用以下 SQL 语句来确认分桶字段是否存在数据倾斜，并据此进行后续的优化调整。

```
select c2, count(*) cnt from t1 group by c2 order by cnt desc limit 10;
```

> 优化建议检查分桶列是否存在数据倾斜问题，如果存在，则更换为在业务含义上具有充分散列特性的字段作为分桶列。

可以明确的是，良好的事前设计能够显著降低事后问题发生时的定位和修正成本。因此，强烈推荐业务人员在 Schema 设计阶段进行严格的设计和检查，以避免引入不必要的成本。

案例 3：Key 列优化

在三种表模型中，若建表 Schema 明确指定了 Duplicate Key、Unique Key 或 Aggregate Key，Doris 将在存储层面确保数据依据 Key 列进行排序。这一特性为数据查询的性能优化提供了新的思路。具体来说，在 Schema 设计阶段，若能将业务查询中频繁使用的等值或范围查询列定义为 Key 列，将会显著提升这类查询的执行速度，进而提升整体性能。

以下是一组业务查询需求的示例：

```
select * from t1 where t1.c1 = 1;
select * from t1 where t1.c1 > 1 and t1.c1 < 10;
select * from t1 where t1.c1 in (1, 2, 3);
```

针对上述业务需求和 t1 表的 Schema 设计与后期优化，可以考虑将 c1 列作为 Key 列，以加速查询过程。以下是一个示例：

```
CREATE TABLE `t1` (
  `c1` INT NULL,
  `c2` INT NULL
) ENGINE=OLAP
DUPLICATE KEY(`c1`)
DISTRIBUTED BY HASH(`c2`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

> 优化建议将业务查询中频繁使用的列设定为 Key 列，以加速查询过程。

案例 4：字段类型优化

在数据库系统中，不同类型的数据其处理复杂程度可能存在显著差异。例如，变长类型的数据处理相较于定长类型而言，其复杂性要高得多；同样，高精类型的数据处理也比低精类型更为复杂。

这一特性对业务系统 Schema 的设计及后期优化提供了重要启示：

1. 在满足业务系统表达和计算需求的前提下，应优先选择定长类型，避免使用变长类型；

2. 尽量采用低精类型，避免高精类型。具体实践包括：使用 BIGINT 替代 VARCHAR 或 STRING 类型的字段，以及用 FLOAT / INT / BIGINT 替换 DECIMAL 类型的字段等。此类字段类型的合理设计和优化，将极大地提升业务的计算效率，从而增强系统性能。

> 优化建议在定义 Schema 类型时，应遵循定长和低精优先的原则。

总结

综上所述，一个精心设计的 Schema 能够最大化地利用 Doris 的特性，进而显著提升业务性能。反观未经过调优的 Schema 设计则可能对业务造成全局性的负面影响，例如数据倾斜等问题。因此，前期的 Schema 设计优化工作显得尤为重要。

2.13.8.1.2 优化索引设计和使用

概述

Doris 目前支持两类索引：

1. 内置索引：包括前缀索引和 ZoneMap 索引等；
2. 二级索引：包括倒排索引、Bloomfilter 索引、N-Gram Bloomfilter 索引和 Bitmap 索引等

在业务优化过程中，充分分析业务特征并有效利用索引，会大大提升查询和分析的效果，从而达到性能调优的目的。

各类索引的详细介绍可以参考表索引章节进行了解。本章将从实际案例的角度出发，展示几种典型场景下的索引使用技巧，并总结优化建议，以供业务调优时参考。

案例 1：优化 Key 列顺序利用前缀索引加速查询

在优化表 Schema 设计中，我们已介绍了如何选择合适的字段作为 Key 字段，并利用 Doris 的 Key 列排序特性来加速查询。本案例将进一步扩展该场景。

由于 Doris 内置了前缀索引功能，它会在建表时自动取表 Key 的前 36 字节作为前缀索引。当查询条件与前缀索引的前缀相匹配时，可以显著加快查询速度。以下是一个表定义的示例：

```
CREATE TABLE `t1` (
  `c1` VARCHAR(10) NULL,
  `c2` VARCHAR(10) NULL
) ENGINE=OLAP
DUPLICATE KEY(`c1`)
DISTRIBUTED BY HASH(`c2`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

相应的业务 SQL 模式如下：

```
select * from t1 where t1.c2 = '1';
select * from t1 where t1.c2 in ('1', '2', '3');
```

在上述 Schema 定义中，c1 在前，c2 在后。然而，查询却是使用 c2 字段进行过滤。在这种情况下，无法利用前缀索引的加速功能。为了进行优化，我们可以调整 c1 和 c2 列的定义顺序，将 c2 列置于第一个字段位置，从而利用前缀索引的加速功能。

调整后的 Schema 如下：

```
CREATE TABLE `t1` (
  `c2` VARCHAR(10) NULL,
  `c1` VARCHAR(10) NULL
) ENGINE=OLAP
DUPLICATE KEY(`c2`)
DISTRIBUTED BY HASH(`c1`) BUCKETS 10
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

> 优化提示
>
> 在定义 schema 列顺序时，应参考业务查询过滤中的高频高优列，以便充分利用 Doris 的前缀索引加速功能。

案例 2：使用倒排索引加速查询

Doris 支持倒排索引作为二级索引，以加速等值、范围及文本类型的全文检索等业务场景。倒排索引的创建和管理是独立的，它能够在不影响原始表 Schema 和无需重新导入表数据的情况下，便捷地进行业务性能优化。

关于典型的使用场景、语法及案例，可参考倒排索引，查看详细介绍，本章节不再重复阐述。

> 优化建议
>
> 对于文本类型的全文检索，以及字符串、数值、日期时间类型字段上的等值或范围查询，均可利用倒排索引来加速查询。特别是在某些情况下，如原始表结构和 Key 定义不便优化，或重新导入表数据的成本较高时，倒排索引提供了一种灵活的加速方案，以优化业务执行性能。

总结

在 Schema 调优中，除了表级 Schema 优化外，索引优化同样占据重要地位。Doris 提供了多种索引类型，包括前缀等内置索引，以及倒排等二级索引。这些索引为性能加速提供了强大的支持，通过合理利用这些索引，我们可以显著提升多场景下的业务查询和分析速度，这对于多场景业务查询和分析具有重要意义。

2.13.8.1.3 使用分区裁剪优化扫表

概述

Doris 作为一款高性能实时分析数据库，提供了强大的分区裁剪（Partition Pruning）功能，可以显著提升查询性能。分区裁剪是一种查询优化技术，它通过分析查询条件，智能识别与查询相关的分区，并仅扫描这些分区的数据，从而避免了对无关分区的不必要扫描。这种优化方式能够大幅减少 I/O 操作和计算量，进而加速查询执行。

案例

下面通过一个实际案例来演示 Doris 的分区裁剪功能。

假设有一个销售数据表 `sales`，该表按照日期进行分区，每天的数据存储在一个独立的分区中。表结构定义如下：

```
CREATE TABLE sales (
    date DATE,
    product VARCHAR(50),
    amount DECIMAL(10, 2)
)
PARTITION BY RANGE(date) (
    PARTITION p1 VALUES LESS THAN ('2023-01-01'),
    PARTITION p2 VALUES LESS THAN ('2023-02-01'),
    PARTITION p3 VALUES LESS THAN ('2023-03-01'),
    PARTITION p4 VALUES LESS THAN ('2023-04-01')
)
DISTRIBUTED BY HASH(date) BUCKETS 16
PROPERTIES
(
    "replication_num" = "1"
);
```

现在，我们需要查询 2023 年 1 月 15 日到 2023 年 2 月 15 日之间的销售总额。查询语句如下：

```
SELECT SUM(amount) AS total_amount
FROM sales
WHERE date BETWEEN '2023-01-15' AND '2023-02-15';
```

对于上述查询，Doris 的分区裁剪优化过程如下：

1. Doris 智能分析查询条件中的分区列 `date`，识别出查询的日期范围在 ‘2023-01-15’ 到 ‘2023-02-15’ 之间。
2. 通过比较查询条件与分区定义，Doris 精确定位需要扫描的分区范围。在本例中，只需要扫描分区 p2 和 p3，因为这两个分区的日期范围完全覆盖了查询条件。
3. Doris 自动跳过与查询条件无关的分区，如 p1 和 p4，避免了不必要的数据扫描，从而减少了 I/O 开销。
4. 最后，Doris 仅在分区 p2 和 p3 中执行数据扫描和聚合计算，快速获取查询结果。

通过 `EXPLAIN` 命令，我们可以查看查询执行计划，确认 Doris 的分区裁剪优化已生效。在执行计划中，`OlapScanNode` 节点的 `partition` 属性将显示实际扫描的分区为 p2 和 p3。

```
|   0:VOlapScanNode(212)                                                                 |
|      TABLE: cir.sales(sales), PREAGGREGATION: ON                                       |
|      PREDICATES: (date[#0] >= '2023-01-15') AND (date[#0] <= '2023-02-15') |
|      partitions=2/4 (p2,p3)                                                            |
    ↪                                    |
```

总结

综上所述，Doris 的分区裁剪功能可以智能识别查询条件与分区之间的关联性，自动裁剪无关分区，仅扫描必要的数据，显著提升查询性能。合理利用分区裁剪特性，可以帮助用户构建高效的实时分析系统，轻松应对海量数据的查询需求。

2.13.8.1.4 使用同步物化视图透明改写

概述

同步物化视图（Sync-Materialized View）是一种特殊的表，它预先根据定义好的 SELECT 语句计算并存储数据。其主要目的是满足用户对原始明细数据的任意维度分析需求，同时也能快速地进行固定维度的分析查询。

同步物化视图的适用场景为：

1. 分析需求同时涵盖明细数据查询和固定维度查询。
2. 查询仅涉及表中的少部分列或行。
3. 查询包含耗时的处理操作，例如长时间的聚合操作等。
4. 查询需要匹配不同的前缀索引。

对于频繁重复使用相同子查询结果的查询，同步物化视图能显著提升性能。Doris 会自动维护物化视图的数据，确保基础表（Base Table）和物化视图表的数据一致性，无需额外的人工维护成本。在查询时，系统会自动匹配到最优的物化视图，并直接从物化视图中读取数据。

> 注意事项 - 在 Doris 2.0 及后续版本中，物化视图具备了一些增强功能。建议用户在正式的生产环境中使用物化视图之前，先在测试环境中确认预期中的查询能否命中想要创建的物化视图。- 不建议在同一张表上创建多个形态类似的物化视图，因为这可能会导致多个物化视图之间的冲突，从而使查询命中失败。

案例

下面通过一个具体例子来展示使用同步物化视图进行查询加速的流程：

假设我们拥有一张销售记录明细表 `sales_records`，该表详细记录了每笔交易的各项信息，包括交易 ID、销售员 ID、售卖门店 ID、销售日期以及交易金额。现在，我们经常需要针对不同门店的销售量进行分析查询。

为了优化这些查询的性能，我们可以创建一个物化视图 `store_amt`，该视图按售卖门店进行分组，并对同一门店的销售额进行求和。具体步骤如下：

创建同步物化视图

首先，我们使用以下 SQL 语句来创建物化视图 `store_amt`：

```
CREATE MATERIALIZED VIEW store_amt AS
SELECT store_id, SUM(sale_amt)
FROM sales_records
GROUP BY store_id;
```

提交创建任务后，Doris 会在后台异步构建这个物化视图。我们可以通过以下命令来查看物化视图的创建进度：

```
SHOW ALTER TABLE MATERIALIZED VIEW FROM db_name;
```

当 State 字段变为 FINISHED 时，就表示 store_amt 物化视图已经成功创建。

透明改写

物化视图创建完成后，当我们查询不同门店的销售量时，Doris 会自动匹配到 store_amt 物化视图，并直接从中读取预先聚合好的数据，从而显著提升查询效率。查询语句如下：

```
SELECT store_id, SUM(sale_amt) FROM sales_records GROUP BY store_id;
```

我们还可以通过 EXPLAIN 命令来检查查询是否成功命中了物化视图：

```
EXPLAIN SELECT store_id, SUM(sale_amt) FROM sales_records GROUP BY store_id;
```

在执行计划的最末尾，如果显示类似以下内容，则表示查询成功命中了 store_amt 物化视图：

```
TABLE: default_cluster:test.sales_records(store_amt), PREAGGREGATION: ON
```

通过以上步骤，我们可以利用同步物化视图来优化查询性能，提高数据分析的效率。

总结

通过创建同步物化视图，我们能够显著提升相关聚合分析的查询速度。同步物化视图不仅使我们能够快速进行统计分析，而且还灵活地支持了明细数据的查询需求，是 Doris 中一项非常强大的功能。

2.13.8.1.5 使用异步物化视图透明改写

概述

异步物化视图采用的是基于 SPJG（SELECT-PROJECT-JOIN-GROUP-BY）模式的透明改写算法。该算法能够分析查询 SQL 的结构信息，自动寻找合适的物化视图，并尝试进行透明改写，以利用最优的物化视图来表达查询 SQL。通过使用预计算的物化视图结果，可以显著提高查询性能，并降低计算成本。

案例

接下来将会通过示例，详细展示如何利用异步物化视图来进行查询加速。

创建基础表

首先，创建 tpch 数据库并在其中创建 orders 和 lineitem 两张表，并插入相应的数据。

```
CREATE DATABASE IF NOT EXISTS tpch;
USE tpch;

CREATE TABLE IF NOT EXISTS orders (
    o_orderkey       integer not null,
    o_custkey        integer not null,
    o_orderstatus    char(1) not null,
    o_totalprice     decimalv3(15,2) not null,
    o_orderdate      date not null,
    o_orderpriority  char(15) not null,
    o_clerk          char(15) not null,
    o_shippriority   integer not null,
    o_comment        varchar(79) not null
)
DUPLICATE KEY(o_orderkey, o_custkey)
PARTITION BY RANGE(o_orderdate)(
    FROM ('2023-10-17') TO ('2023-10-20') INTERVAL 1 DAY
)
DISTRIBUTED BY HASH(o_orderkey) BUCKETS 3
PROPERTIES ("replication_num" = "1");

INSERT INTO orders VALUES
    (1, 1, 'o', 99.5, '2023-10-17', 'a', 'b', 1, 'yy'),
    (2, 2, 'o', 109.2, '2023-10-18', 'c','d',2, 'mm'),
    (3, 3, 'o', 99.5, '2023-10-19', 'a', 'b', 1, 'yy');

CREATE TABLE IF NOT EXISTS lineitem (
    l_orderkey    integer not null,
    l_partkey     integer not null,
    l_suppkey     integer not null,
    l_linenumber  integer not null,
    l_quantity    decimalv3(15,2) not null,
    l_extendedprice  decimalv3(15,2) not null,
    l_discount    decimalv3(15,2) not null,
    l_tax         decimalv3(15,2) not null,
    l_returnflag  char(1) not null,
    l_linestatus  char(1) not null,
    l_shipdate    date not null,
    l_commitdate  date not null,
    l_receiptdate date not null,
    l_shipinstruct char(25) not null,
    l_shipmode     char(10) not null,
    l_comment      varchar(44) not null
)
```

```
DUPLICATE KEY(l_orderkey, l_partkey, l_suppkey, l_linenumber)
PARTITION BY RANGE(l_shipdate)
(FROM ('2023-10-17') TO ('2023-10-20') INTERVAL 1 DAY)
DISTRIBUTED BY HASH(l_orderkey) BUCKETS 3
PROPERTIES ("replication_num" = "1");

INSERT INTO lineitem VALUES
    (1, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-17', '2023-10-17', '2023-10-17', 'a', 'b'
        ↪ , 'yyyyyyyyy'),
    (2, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-18', '2023-10-18', '2023-10-18', 'a', 'b'
        ↪ , 'yyyyyyyyy'),
    (3, 2, 3, 6, 7.5, 8.5, 9.5, 10.5, 'k', 'o', '2023-10-19', '2023-10-19', '2023-10-19', 'c', 'd
        ↪ ', 'xxxxxxxxx');
```

创建异步物化视图

基于 tpch benchmark 中的若干原始表，创建一个异步物化视图 mv1。

```
CREATE MATERIALIZED VIEW mv1
BUILD IMMEDIATE REFRESH COMPLETE ON MANUAL
PARTITION BY(l_shipdate)
DISTRIBUTED BY RANDOM BUCKETS 2
PROPERTIES ('replication_num' = '1')
AS
SELECT l_shipdate, o_orderdate, l_partkey, l_suppkey, SUM(o_totalprice) AS sum_total
FROM lineitem
LEFT JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
GROUP BY
l_shipdate,
o_orderdate,
l_partkey,
l_suppkey;
```

使用物化视图进行透明改写

```
mysql> explain shape plan SELECT l_shipdate, SUM(o_totalprice) AS total_price
    -> FROM lineitem
    -> LEFT JOIN orders ON lineitem.l_orderkey = orders.o_orderkey AND l_shipdate = o_orderdate
    -> WHERE l_partkey = 2 AND l_suppkey = 3
    -> GROUP BY l_shipdate;
+----------------------------------------------------------------------+
| Explain String(Nereids Planner)                                      |
+----------------------------------------------------------------------+
| PhysicalResultSink                                                   |
| --PhysicalDistribute[DistributionSpecGather]                         |
| ----PhysicalProject                                                  |
| ------hashAgg[GLOBAL]                                                |
```

```
| --------PhysicalDistribute[DistributionSpecHash]                  |
| ----------hashAgg[LOCAL]                                          |
| ------------PhysicalProject                                       |
| --------------filter((mv1.l_partkey = 2) and (mv1.l_suppkey = 3)) |
| ----------------PhysicalOlapScan[mv1]                            |
+-------------------------------------------------------------------+
```

通过 explain shape plan 可见经过 mv1 透明改写后的计划已经命中 mv1。通过 explain 也可以查看当前计划经过 mv 改写的状态，包括是否命中以及命中的 mv 等信息，如下所示：

```
| ========== MATERIALIZATIONS ==========                                                  |
|                                                                                         |
| MaterializedView                                                                        |
| MaterializedViewRewriteSuccessAndChose:                                                 |
|   internal.tpch.mv1 chose,                                                              |
|                                                                                         |
| MaterializedViewRewriteSuccessButNotChose:                                              |
|   not chose: none,                                                                      |
|                                                                                         |
| MaterializedViewRewriteFail:                                                            |
```

总结

通过使用异步物化视图，可以显著提高查询性能，特别是对于复杂的连接和聚合查询。在使用的时候需要注意：

> 使用建议 - 预计算结果：物化视图将查询结果预先计算并存储，避免了每次查询时重复计算的开销。这对于需要频繁执行的复杂查询尤其有效。- 减少联接操作：物化视图可以将多个表的数据合并到一个视图中，减少了查询时的联接操作，从而提高查询效率。- 自动更新：当基表数据发生变化时，物化视图可以自动更新，以保持数据的一致性。这确保了查询结果始终反映最新的数据状态。- 空间开销：物化视图需要额外的存储空间来保存预计算的结果。在创建物化视图时，需要权衡查询性能提升和存储空间消耗。- 维护成本：物化视图的维护需要一定的系统资源和时间。频繁更新的基表可能导致物化视图的更新开销较大。因此，需要根据实际情况选择合适的刷新策略。- 适用场景：物化视图适用于数据变化频率较低、查询频率较高的场景。对于经常变化的数据，实时计算可能更为合适。

合理利用异步物化视图，可以显著改善数据库的查询性能，特别是在复杂查询和大数据量的情况下。同时，也需要综合考虑存储、维护等因素，以实现性能和成本的平衡。

2.13.8.1.6 使用 Colocate Group 优化 Join

Colocate Group 是一种高效的 Join 方式，使得执行引擎能有效地规避 Join 操作中数据的 shuffle 开销。相关原理介绍和案例参考详见 Colocation Join。

注意 - 在某些场景下，即使已经成功建立了 Colocate Group，执行计划（plan）仍然可能会显示为 Shuffle Join 或 Bucket Shuffle Join。这种情况通常发生在 Doris 正在进行数据整理的过程中，比如，它可能在 BE 间迁移 tablet，以确保数据在多个 BE 之间的分布达到更加均衡的状态。- 通过命令show proc "/colocation_group"; 可以查看 Colocate Group 状态，如下图所示：IsStable 出现 false，表示有 Colocate Group 不可用的情况。

```
mysql> show proc "/colocation_group";
+------------+----------------------+--------------------------+--------------+--------------------------+-----------+----------+----------+
| GroupId    | GroupName            | TableIds                 | BucketsNum   | ReplicaAllocation        | DistCols  | IsStable | ErrorMsg |
+------------+----------------------+--------------------------+--------------+--------------------------+-----------+----------+----------+
| 36023.36619| 36023_part_partsupp  | 36425,36621              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 32022.34353| 32022_lineitem_orders| 32815,34355              | 768          | tag.location.default: 1 | bigint(20) | true     |          |
| 10644.11087| 10644_part_partsupp  | 11037,11089              | 24           | tag.location.default: 1 | int(11)    | true     |          |
| 83282.83478| 83282_lineitem_orders| 8328483480               | 96           | tag.location.default: 1 | higint(20) | true     |          |
| 47947.50278| 47947_lineitem_orders| 48740,50280              | 768          | tag.location.default: 1 | bigint(20) | false    |          |
| 29082.30133| 29082_part_partsupp  | 30083,30135              | 24           | tag.location.default: 1 | int(11)    | false    |          |
| 85801.85997| 85801_tineitem_oraers| 85803,85999              | 96           | tag.location.default: 1 | bigint(20) | true     |          |
| 10644.10840| 10644_lineitem_orders| 10646,10842              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 47947.48543| 47947_part_partsupp  | 48349,48545              | 96           | tag.location.default: 1 | bigint(20) | true     |          |
| 29082.29866| 29082_lineitem_orders| 29672,29888              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 32022.32618| 32022_part_partsupp  | 32424,32620              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 44074.46405| 44074_lineitem_orders| 44867,46407              | 768          | tag.location.default: 1 | bigint(20) | true     |          |
| 83282.83725| 83282_part_partsupp  | 83675,83727              | 24           | tag.location.default: 1 | int(11)    | true     |          |
| 85801.86244| 85801_part_partsupp  | 86194,86246              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 40021.40617| 40021_part_partsupp  | 40423,40619              | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 36023.38354| 36023_lineitem_orders| 36816, 38356             | 768          | tag.location.default: 1 | bigint(20) | true     |          |
| 44074.44670| 44074_part_partsupp  | 44476, 44672             | 96           | tag.location.default: 1 | int(11)    | true     |          |
| 40021.42352| 40021_lineitem_orders| 40814, 42354             | 768          | tag.location.default: 1 | bigint(20) | true     |          |
+------------+----------------------+--------------------------+--------------+--------------------------+-----------+----------+----------+
18 rows in set (0.00 sec)
```

图 55: 使用 Colocate Group 优化 Join

2.13.8.1.7 使用 Hint 调整 Join Shuffle 方式

概述

Doris 支持使用 Hint 来调整 Join 操作中数据 Shuffle 的类型，从而优化查询性能。本节将详细介绍如何在 Doris 中利用 Hint 来指定 Join Shuffle 的类型。

注意当前 Doris 已经具备良好的开箱即用的能力，也就意味着在绝大多数场景下，Doris 会自适应的优化各种场景下的性能，无需用户来手工控制 hint 来进行业务调优。本章介绍的内容主要面向专业调优人员，业务人员仅做简单了解即可。

目前，Doris 支持两种独立的Distribute Hint，[shuffle] 和 [broadcast]，用来指定 Join 右表的 Distribute Type。Distribute Type 需置于 Join 右表之前，采用中括号 [] 的方式。同时，Doris 也可以通过 Leading Hint 配合 Distribute Hint 的方式，指定 shuffle 方式（详见使用 Leading Hint 控制 Join 顺序章节相关介绍）。

示例如下：

```
SELECT COUNT(*) FROM t2 JOIN [broadcast] t1 ON t1.c1 = t2.c2;
SELECT COUNT(*) FROM t2 JOIN [shuffle] t1 ON t1.c1 = t2.c2;
```

案例

接下来将通过同一个例子来展示 Distribute Hint 的使用方法：

```
EXPLAIN SHAPE PLAN SELECT COUNT(*) FROM t1 JOIN t2 ON t1.c1 = t2.c2;
```

原始 SQL 的计划如下，可见 t1 连接 t2 使用了 hash distribute 即DistributionSpecHash的方式。

```
+----------------------------------------------------------------------------------+
| Explain String (Nereids Planner)                                                 |
+----------------------------------------------------------------------------------+
| PhysicalResultSink                                                               |
| --hashAgg [GLOBAL]                                                               |
| ----PhysicalDistribute [DistributionSpecGather]                                  |
| ------hashAgg [LOCAL]                                                            |
| --------PhysicalProject                                                          |
| ----------hashJoin [INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()|
| ------------PhysicalProject                                                      |
| --------------PhysicalOlapScan [t1]                                             |
| ------------PhysicalDistribute [DistributionSpecHash]                           |
| --------------PhysicalProject                                                    |
| ----------------PhysicalOlapScan [t2]                                           |
+----------------------------------------------------------------------------------+
```

加入 [broadcast] hint 后：

```
EXPLAIN SHAPE PLAN SELECT COUNT(*) FROM t1 JOIN [broadcast] t2 ON t1.c1 = t2.c2;
```

可见 t1 连接 t2 的分发方式改为了 broadcast 即DistributionSpecReplicated的方式。

```
+----------------------------------------------------------------------------------+
| Explain String (Nereids Planner)                                                 |
+----------------------------------------------------------------------------------+
| PhysicalResultSink                                                               |
| --hashAgg [GLOBAL]                                                               |
| ----PhysicalDistribute [DistributionSpecGather]                                  |
| ------hashAgg [LOCAL]                                                            |
| --------PhysicalProject                                                          |
| ----------hashJoin [INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()|
| ------------PhysicalProject                                                      |
| --------------PhysicalOlapScan [t1]                                             |
| ------------PhysicalDistribute [DistributionSpecReplicated]                     |
| --------------PhysicalProject                                                    |
| ----------------PhysicalOlapScan [t2]                                           |
```

```
+------------------------------------------------------------------------------------+
```

总结

通过合理使用 Distribute Hint，可以优化 Join 操作的 Shuffle 方式，提升查询性能。在实践中，建议先通过 EXPLAIN 分析查询执行计划，再根据实际情况指定合适的 Shuffle 类型。

2.13.8.1.8 使用 Hint 控制代价改写

概述

查询优化器在生成执行计划的过程中，会应用一系列规则。这些规则主要分为两类：基于规则的优化（Rule-Based Optimizer 即 RBO）和基于代价的优化（Cost-Based Optimizer 即 CBO）。

- RBO：此类优化通过应用一系列预定义的启发式规则来改进查询计划，而不考虑具体的数据统计信息。例如，谓词下推、投影下推等策略均属于此类。
- CBO：此类优化则利用数据统计信息来估算不同执行计划的代价，并选择代价最小的计划进行执行。这包括访问路径的选择、连接算法的选择等。

在某些情况下，数据库管理员或开发人员可能需要对查询优化过程进行更为精细的控制。基于此，本文档将介绍如何使用查询 Hint 来管理 CBO 规则。

> 注意当前 Doris 已经具备良好的开箱即用的能力，也就意味着在绝大多数场景下，Doris 会自适应的优化各种场景下的性能，无需用户来手工控制 hint 来进行业务调优。本章介绍的内容主要面向专业调优人员，业务人员仅做简单了解即可。

CBO 规则控制 Hint 的基本语法如下所示：

```
SELECT /*+ USE_CBO_RULE(rule1, rule2, ...) */ ...
```

此 Hint 紧跟在 SELECT 关键字之后，并在括号内指定要启用的规则名称（规则名称不区分大小写）。

当前 Doris 优化器支持若干种代价改写，可以通过 USE_CBO_RULE hint 来显式启用，例如：

- PUSH_DOWN_AGG_THROUGH_JOIN
- PUSH_DOWN_AGG_THROUGH_JOIN_ONE_SIDE
- PUSH_DOWN_DISTINCT_THROUGH_JOIN

案例

查询示例如下：

```
explain shape plan
    select /*+ USE_CBO_RULE(push_down_agg_through_join_one_side) */
            a.event_id,
            b.group_id,
```

```
        COUNT(a.event_id)
    from a
    join b on
        a.device_id = b.device_id
    group by
        a.event_id,
        b.group_id
    ;
```

在此示例中启用了一个聚合下推 CBO 规则。这一操作可以使表 a 能够在连接操作之前进行提前聚合，减少连接的开销，加速查询。下推后的计划如下：

```
PhysicalResultSink
--hashAgg[GLOBAL]
----hashAgg[LOCAL]
------hashJoin[INNER_JOIN] hashCondition=((a.device_id = b.device_id)) otherCondition=()
--------hashAgg[LOCAL]
----------PhysicalOlapScan[a]
--------filter((cast(experiment_id as DOUBLE) = 73.0))
----------PhysicalOlapScan[b]
```

总结

合理使用 USE_CBO_RULE hint，可以帮助手动启用部分高级 CBO 优化规则，在特定场景下优化性能。但使用 CBO 优化规则需要对查询优化过程和数据特性有深入的理解，在大多数情况下，依赖 Doris 优化器的自动决策仍然是最佳的选择。

2.13.8.1.9 使用 Leading Hint 控制 Join 顺序

概述

Leading Hint 特性允许用户手工指定查询中的表的连接顺序，在特定场景优化复杂查询性能。本文将详细介绍如何在 Doris 中使用 Leading Hint 来控制 join 的顺序。详细使用说明，可参考leading hint文档。

> 注意当前 Doris 已经具备良好的开箱即用的能力，也就意味着在绝大多数场景下，Doris 会自适应的优化各种场景下的性能，无需用户来手工控制 hint 来进行业务调优。本章介绍的内容主要面向专业调优人员，业务人员仅做简单了解即可。

案例 1：调整左右表顺序

对于如下查询：

```
mysql> explain shape plan select from t1 join t2 on t1.c1 = t2.c2;
+------------------------------------------------------------------------------+
| _Explain_ String(Nereids Planner)                                            |
```

```
+----------------------------------------------------------------------------+
| PhysicalResultSink                                                         |
| --PhysicalDistribute[DistributionSpecGather]                               |
| ----PhysicalProject                                                        |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
| --------PhysicalOlapScan[t1]                                              |
| --------PhysicalDistribute[DistributionSpecHash]                          |
| ----------PhysicalOlapScan[t2]                                            |
+----------------------------------------------------------------------------+
```

可以使用 Leading Hint，强制指定 join order 为 t2 join t1，调整原始连接顺序。

```
mysql> explain shape plan select /*+ leading(t2 t1) */ * from t1 join t2 on t1.c1 = t2.c2;
+----------------------------------------------------------------------------+
| _Explain_ String(Nereids Planner)                                          |
+----------------------------------------------------------------------------+
| PhysicalResultSink                                                         |
| --PhysicalDistribute[DistributionSpecGather]                               |
| ----PhysicalProject                                                        |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
| --------PhysicalOlapScan[t2]                                              |
| --------PhysicalDistribute[DistributionSpecHash]                          |
| ----------PhysicalOlapScan[t1]                                            |
|                                                                            |
| Hint log:                                                                  |
| Used: leading(t2 t1)                                                       |
| UnUsed:                                                                    |
| SyntaxError:                                                               |
+----------------------------------------------------------------------------+
```

Hint log 展示了应用成功的 hint: `Used: leading(t2 t1)`。

案例 2：强制生成左深树

```
mysql> explain shape plan select /*+ leading(t1 t2 t3) */ * from t1 join t2 on t1.c1 = t2.c2 join
    ↪  t3 on t2.c2 = t3.c3;
+----------------------------------------------------------------------------+
| _Explain_ String(Nereids Planner)                                          |
+----------------------------------------------------------------------------+
| PhysicalResultSink                                                         |
| --PhysicalDistribute[DistributionSpecGather]                               |
| ----PhysicalProject                                                        |
| ------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()   |
| --------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=() |
| ----------PhysicalOlapScan[t1]                                             |
| ----------PhysicalDistribute[DistributionSpecHash]                         |
| ------------PhysicalOlapScan[t2]                                           |
```

```
| --------PhysicalDistribute[DistributionSpecHash]                                    |
| ----------PhysicalOlapScan[t3]                                                      |
|                                                                                     |
| Hint log:                                                                           |
| Used: leading(t1 t2 t3)                                                             |
| UnUsed:                                                                             |
| SyntaxError:                                                                        |
+-------------------------------------------------------------------------------------+
```

同样，Hint log 展示了应用成功的 hint: Used: leading(t1 t2 t3)。

案例 3：强制生成右深树

```
mysql> explain shape plan select /*+ leading(t1 {t2 t3}) */ * from t1 join t2 on t1.c1 = t2.c2
    ↪ join t3 on t2.c2 = t3.c3;
+-------------------------------------------------------------------------------------+
| _Explain_ String(Nereids Planner)                                                     |
+-------------------------------------------------------------------------------------+
| PhysicalResultSink                                                                  |
| --PhysicalDistribute[DistributionSpecGather]                                        |
| ----PhysicalProject                                                                 |
| ------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = t2.c2)) otherCondition=()         |
| --------PhysicalOlapScan[t1]                                                       |
| --------PhysicalDistribute[DistributionSpecHash]                                   |
| ----------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()    |
| ------------PhysicalOlapScan[t2]                                                   |
| ------------PhysicalDistribute[DistributionSpecHash]                               |
| --------------PhysicalOlapScan[t3]                                                 |
|                                                                                     |
| Hint log:                                                                           |
| Used: leading(t1 { t2 t3 })                                                         |
| UnUsed:                                                                             |
| SyntaxError:                                                                        |
+-------------------------------------------------------------------------------------+
```

同样，Hint log 展示了应用成功的 hint: Used: leading(t1 { t2 t3 })。

案例 4：强制生成 bushy 树

```
mysql> explain shape plan select /*+ leading({t1 t2} {t3 t4}) */ * from t1 join t2 on t1.c1 = t2.
    ↪ c2 join t3 on t2.c2 = t3.c3 join t4 on t3.c3 = t4.c4;
+-----------------------------------------------+
| _Explain_ String                              |
+-----------------------------------------------+
| PhysicalResultSink                            |
| --PhysicalDistribute                          |
| ----PhysicalProject                           |
```

```
| ------hashJoin[INNER_JOIN](\#)     |
| --------hashJoin[INNER_JOIN](\#)   |
| ----------PhysicalOlapScan[t1]               |
| ----------PhysicalDistribute                 |
| ------------PhysicalOlapScan[t2]             |
| --------PhysicalDistribute                   |
| ----------hashJoin[INNER_JOIN](\#) |
| ------------PhysicalOlapScan[t3]             |
| ------------PhysicalDistribute               |
| --------------PhysicalOlapScan[t4]           |
|                                              |
| Used: leading({ t1 t2 } { t3 t4 })           |
| UnUsed:                                      |
| SyntaxError:                                 |
+----------------------------------------------+
```

同样，Hint log 展示了应用成功的 hint: Used: `leading({ t1 t2 } { t3 t4 })`。

案例 5：view 作为整体参与连接

```
mysql>  explain shape plan select /*+ leading(alias t1) */ count(*) from t1 join (select c2 from
    ↪ t2 join t3 on t2.c2 = t3.c3) as alias on t1.c1 = alias.c2;
+-----------------------------------------------------------------------------------------+
| _Explain_ String(Nereids Planner)                                                       |
+-----------------------------------------------------------------------------------------+
| PhysicalResultSink                                                                      |
| --hashAgg[GLOBAL]                                                                       |
| ----PhysicalDistribute[DistributionSpecGather]                                          |
| ------hashAgg[LOCAL]                                                                    |
| --------PhysicalProject                                                                 |
| ----------hashJoin[INNER_JOIN] hashCondition=((t1.c1 = alias.c2)) otherCondition=()     |
| ------------PhysicalProject                                                             |
| --------------hashJoin[INNER_JOIN] hashCondition=((t2.c2 = t3.c3)) otherCondition=()    |
| ----------------PhysicalProject                                                         |
| ------------------PhysicalOlapScan[t2]                                                 |
| ----------------PhysicalDistribute[DistributionSpecHash]                               |
| ------------------PhysicalProject                                                      |
| --------------------PhysicalOlapScan[t3]                                               |
| ------------PhysicalDistribute[DistributionSpecHash]                                   |
| --------------PhysicalProject                                                          |
| ----------------PhysicalOlapScan[t1]                                                   |
|                                                                                         |
| Hint log:                                                                               |
| Used: leading(alias t1)                                                                 |
| UnUsed:                                                                                 |
| SyntaxError:                                                                            |
```

```
+----------------------------------------------------------------------------------------+
```

同样，Hint log 展示了应用成功的 hint: Used: leading(alias t1)。

案例 6：DistributeHint 与 LeadingHint 混用

```
explain shape plan
    select
        nation,
        o_year,
        sum(amount) as sum_profit
    from
        (
            select
                /*+ leading(orders shuffle {lineitem shuffle part} shuffle {supplier broadcast
                    ↪ nation} shuffle partsupp) */
                n_name as nation,
                extract(year from o_orderdate) as o_year,
                l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity as amount
            from
                part,
                supplier,
                lineitem,
                partsupp,
                orders,
                nation
            where
                s_suppkey = l_suppkey
                and ps_suppkey = l_suppkey
                and ps_partkey = l_partkey
                and p_partkey = l_partkey
                and o_orderkey = l_orderkey
                and s_nationkey = n_nationkey
                and p_name like '%green%'
        ) as profit
    group by
        nation,
        o_year
    order by
        nation,
        o_year desc;
```

上述 /*+ leading(orders shuffle {lineitem shuffle part} shuffle {supplier broadcast nation} ↪ shuffle partsupp)*/ hint 指定方式，混用了 leading 和 distribute hint 两种格式。leading 用于控制总体的表之间的相对 join 顺序，而 shuffle 和 broadcast 分别用于指定特定 join 使用何种 shuffle 方式。通过两种结合使用，可以灵活的控制连接顺序和连接方式，便于手工控制用户期望的计划行为。

使用建议 - 建议使用 EXPLAIN 来仔细分析执行计划，以确保 Leading Hint 能达到预期的效果。- Doris 版本升级或者业务数据变更时，应重新评估 Leading Hint 的效果，做到及时记录和调整。

总结

Leading Hint 是一种强大的可以手工控制连接顺序的功能，于此同时，也可以和 shuffle hint 结合使用，同时控制 join 分发方式，进而优化查询性能。注意这种高级特性，应当在充分理解查询特性及数据分布的基础上谨慎使用。

2.13.8.1.10 使用 SQL Cache 加速查询

概述

关于 SQL Cache 详细实现原理，请参考查询缓存（SQL Cache）章节

案例

详细案例请参考查询缓存（SQL Cache）章节

总结

SQL Cache 是 Doris 提供的一种查询优化机制，可以显著提升查询性能。在使用的时候需要注意：

提示 - SQL Cache 不适用于包含生成随机值的函数 (如 `random()`) 的查询，因为这会导致查询结果失去随机性。- 目前不支持使用部分指标的缓存结果来满足查询更多指标的需求。例如，之前查询了 2 个指标的缓存不能用于查询 3 个指标的情况。- 通过合理使用 SQL Cache，可以显著提升 Doris 的查询性能，特别是在数据更新频率较低的场景中。在实际应用中，需要根据具体的数据特征和查询模式来调整缓存参数，以获得最佳的性能提升。

2.13.8.1.11 DML 计划调优

DML 计划调优首先需要定位是导入引起的性能瓶颈，还是查询部分引起的性能瓶颈。查询部分的性能瓶颈的排查和调优详见计划调优其他小节。

Doris 支持从多种数据源导入数据，灵活运用 Doris 提供的多种导入功能，可以高效地将各种来源的数据导入到 Doris 中进行分析。最佳实践详情请参考导入概览。

2.13.8.2 执行调优

2.13.8.2.1 RuntimeFilter 的等待时间调整

概述

实际生产场景会遇到因为 RuntimeFilter 等待时间不合理，引起的性能问题的情况。RuntimeFilter 是一种查询优化技术，它通过运行时生成过滤条件，从而避免了对无关数据的扫描。这种优化方式能够大幅减少 I/O 操作和计算量，进而加速查询执行。下面介绍几种常见的案例，帮助在数据倾斜场景下进行调优。。

案例：RuntimeFilter 等待时间过短

参考下面 Profile 的信息：

```
OLAP_SCAN_OPERATOR (id=22. nereids_id=1764. table name = test_doris(test_doris)):(ExecTime:
    ↪ 62.870ms)
              - RuntimeFilters: : RuntimeFilter: (id = 6, type = minmax, need_local_merge: true,
                  ↪  is_broadcast: false, build_bf_cardinality: false, RuntimeFilter: (id = 7,
                  ↪  type = in_or_bloomfilter, need_local_merge: true, is_broadcast: false,
                  ↪ build_bf_cardinality: false,
              - PushDownPredicates: []
              - KeyRanges: ScanKeys:ScanKey=[null(-9223372036854775808) : 9223372036854775807]
              - TabletIds: [1732763414173, 1732763414187, 1732763414201, 1732763414215]
              - UseSpecificThreadToken: False
              - AcquireRuntimeFilterTime: 969ns
              - BlocksProduced: 1.829K (1829)
              - CloseTime: 0ns
              - ExecTime: 62.870ms
              - InitTime: 75.703us
              - KeyRangesNum: 0
              - MaxScannerThreadNum: 32
              - MemoryUsage:
                - PeakMemoryUsage: 0.00
              - NumScanners: 32
              - OpenTime: 19.276ms
              - ProcessConjunctTime: 30.360us
              - ProjectionTime: 0ns
              - RowsProduced: 7.433056M (7433056)
              - RowsRead: 0
              - RuntimeFilterInfo:
              - ScannerWorkerWaitTime: 0ns
              - TabletNum: 4
              - TotalReadThroughput: 0
              - WaitForDependency[OLAP_SCAN_OPERATOR_DEPENDENCY]Time: 0ns
              - WaitForRuntimeFilter: 1000ms
            RuntimeFilter: (id = 6, type = minmax):
               - Info: [IsPushDown = false, RuntimeFilterState = NOT_READY, HasRemoteTarget =
                   ↪ true, HasLocalTarget = false, Ignored = false]
            RuntimeFilter: (id = 7, type = in_or_bloomfilter):
               - Info: [IsPushDown = false, RuntimeFilterState = NOT_READY, HasRemoteTarget =
                   ↪ true, HasLocalTarget = false, Ignored = false]
```

从 Profile 中可以看到：`WaitForRuntimeFilter: 1000ms`。这里 RuntimeFilter 等待了 1000ms，但是这个 ScanOperator

并没有等到对应的 RuntimeFilter，RuntimeFilterState = NOT_READY。

```
RuntimeFilter: (id = 6, type = minmax):
                - Info: [IsPushDown = false, RuntimeFilterState = NOT_READY, HasRemoteTarget =
                    ↪ true, HasLocalTarget = false, Ignored = false]
RuntimeFilter: (id = 7, type = in_or_bloomfilter):
                - Info: [IsPushDown = false, RuntimeFilterState = NOT_READY, HasRemoteTarget =
                    ↪ true, HasLocalTarget = false, Ignored = false]
```

所以这里对应的 RuntimeFilter 的 id 6 和 7 都没有等到。通过 Profile 定位到生成 RuntimeFilter 的 Join，发现 Join 耗时

```
 HASH_JOIN_OPERATOR (id=26 , nereids_id=37948):
              - PlanInfo
                 - join op: RIGHT OUTER JOIN(PARTITIONED)[]
                 - equal join conjunct: (id = ID)
                 - runtime filters: RF006[min_max] <- ID(6418/8192/1048576), RF007[in_or_bloom]
                     ↪  <- ID(6418/8192/1048576)
                 - cardinality=6,418
                 - vec output tuple id: 27
                 - output tuple id: 27
                 - vIntermediate tuple ids: 25
                 - hash output slot ids: 396 398 399 400 401 402 403 404 405 406 407 408 409
                     ↪ 410 411 412 413 447
                 - projections: USER_ID
                 - project output tuple id: 27
              - BlocksProduced: sum 1, avg 1, max 1, min 1
              - CloseTime: avg 10.111us, max 10.111us, min 10.111us
              - ExecTime: avg 364.497us, max 364.497us, min 364.497us
              - InitTime: avg 26.653us, max 26.653us, min 26.653us
              - MemoryUsage: sum , avg , max , min
                - PeakMemoryUsage: sum 0.00 , avg 0.00 , max 0.00 , min 0.00
                - ProbeKeyArena: sum 0.00 , avg 0.00 , max 0.00 , min 0.00
              - OpenTime: avg 45.985us, max 45.985us, min 45.985us
              - ProbeRows: sum 0, avg 0, max 0, min 0
              - ProjectionTime: avg 211.930us, max 211.930us, min 211.930us
              - RowsProduced: sum 1, avg 1, max 1, min 1
              - WaitForDependency[HASH_JOIN_OPERATOR_DEPENDENCY]Time: avg 1sec780ms, max 1
                  ↪ sec780ms, min 1sec780ms
```

可以看到这个 Join 耗时大概是1sec780ms，所以 RuntimeFilter 在 1s 内并没有等到。于是调整 RuntimeFilter 的等待时间：

```
set runtime_filter_wait_time_ms = 3000;
```

调整之后，查询耗时从 5s 降低到 2s。

总结

RuntimeFilter 的等待时间需要根据场景定义，Doris 正在进行一些自适应的优化改造。通过 EXPLAIN 和 PROFILE 工具观察查询执行瓶颈，定位对应问题，通过 SQL HINT 修改 RuntimeFilter 等待时间，规避对应问题对性能的影响。

2.13.8.2.2 数据倾斜处理

概述

Doris 是一个 MPP 数据库，依赖数据 shuffle 进行并行的计算加速。但是实际生产场景经常会遇到因为数据倾斜导致查询并行的单线程的执行瓶颈。下节介绍如何发现这类问题，并提供一些通用的解决方法。

案例 1：Bucket 数据倾斜导致 shuffle 方式不优

当 Table 在 Join Key 上出现数据倾斜时，数据会在不同的 BE instance 间会分布不均，导致单点的执行瓶颈，进而拖慢整个查询的执行时间。

```
HASH_JOIN_OPERATOR  (id=27):
      -  PlanInfo
            -  join  op: INNER  JOIN(PARTITIONED)[]
            -  equal  join  conjunct:  (customer_number  =  customer_number)
            -  runtime  filters:  RF001[bloom]  <-  customer_number(200/256/2048)
            -  cardinality=200
            -  vec  output  tuple  id:  28
            -  output  tuple  id:  28
            -  vIntermediate  tuple  ids:  27
            -  hash  output  slot  ids:  192  193  194  195  196  197  198  199  200  201  174
                ↪ 175  240  176  177  178  179  180  181  182  183  184  185  186  187  188
                ↪ 189  190  191
            -  project  output  tuple  id:  28
      -  BlocksProduced:  sum  4.883K  (4883),  avg  33,  max  39,  min  29
      -  CloseTime:  avg  37.28us,  max  132.653us,  min  13.945us
      -  ExecTime:  avg  166.206ms,  max  10s947.344ms,  min  8.845ms
      -  InitTime:  avg  0ns,  max  0ns,  min  0ns
      -  MemoryUsage:  sum  ,  avg  ,  max  ,  min
          -  PeakMemoryUsage:  sum  11.81  MB,  avg  84.00  KB,  max  84.00  KB,  min  84.00  KB
          -  ProbeKeyArena:  sum  11.81  MB,  avg  84.00  KB,  max  84.00  KB,  min  84.00  KB
      -  OpenTime:  avg  194.970us,  max  497.685us,  min  93.738us
      -  ProbeRows:  sum  23.884018M  (23884018),  avg  165.861K  (165861),  max  219.346276M
          ↪ (219346276),  min  1984  (1984)
      -  ProjectionTime:  avg  7.336ms,  max  33.540ms,  min  3.760ms
      -  RowsProduced:  sum  28.8K  (28800),  avg  200,  max  200,  min  200
```

从上面的 Join 的 Profile 上 max 指标上来看，执行时间和 ProbeRows 的有明显的倾斜情况。

```
ExecTime:  avg  166.206ms,  max  10s947.344ms,  min  8.845ms
ProbeRows:  sum  23.884018M  (23884018),  avg  165.861K  (165861),  max  219.346276M  (219346276)
    ↪ ,  min  1984  (1984)
```

然而，由于数据基于 join key shuffle 之后分布不均，会导致其中一个线程处理了 2 亿行数据，而另一个线程只处理了几千行数据。

上述 case 在理想情况下，每个线程各处理的数据是接近的。但因为 Join 列数据倾斜的问题，可能会导致大量的计算工作由一个线程完成的。为了解决这个性能瓶颈，可以参考“使用 Hint 控制 Join Shuffle 方式”章节中提到的调优技巧，指定 broadcast join hint 如下，让左表不进行数据的 shuffle，这样就可以有效避免因为 Join 列数据倾斜导致的性能瓶颈。

```
SELECT COUNT(*) FROM orders o JOIN [broadcast] customer c ON o.customer_number = c.customer_
    ↪ number;
```

案例 2：列数据倾斜导致 join 左右边颠倒

当前 Doris 优化器基于数据均匀假设估算选择率，过滤估行偏差大会影响算子的计划选择。以如下 SQL 为例：

```
select count(*)
from orders, customer
where o_custkey = c_custkey
and o_orderdate < '1920-01-02';
```

在均匀分布的假设下，优化器可能会认为经过o_orderdate < '1920-01-02'过滤后输出的行数会少于 customer 表的行数，因此可能选择customerjoin orders 的连接顺序。

但是如果实际数据存在倾斜，导致满足条件的 orders 表的条数多于 customer ，那么更合理的连接顺序应该是ordersjoincustomer 。为了解决这个性能问题，可以参考“使用 Leading Hint 控制 Join 顺序”章节中提到的调优技巧，指定 leading hint 如下，强制生成customerjoin orders 的连接顺序。

改写 SQL 如下：

```
select /*+leading(orders customer)*/ count(*)
from orders, customer
where o_custkey = c_custkey
and o_orderdate < '1920-01-02'
```

总结

数据倾斜是常见的生产场景性能问题。通过 EXPLAIN 和 PROFILE 工具输出观察计划和执行瓶颈，定位倾斜原因，然后就可以使用 Hint 工具进行相应的计划调整，规避数据倾斜对性能的影响了。

概述

Doris 的查询是一个 MPP 的执行框架，每一条查询都会在多个 BE 上并行执行；同时，在单个 BE 内部也会采用多线程并行的方式来加速查询的执行效率，目前所有的语句（包括 Query，DML，DDL）均支持并行执行。

单个 BE 内并行度的控制参数是：parallel_pipeline_task_num，是指单个 Fragment 在执行时所使用的工作任务数。在实际生产场景会遇到并行度设置不合理，引起的性能问题。在以下的案例中，列举了调整并行度优化的案例。

并行度调优的原则

parallel_pipeline_task_num 设定目的是为了充分利用多核资源，降低查询的延迟；但是，为了多核并行执行，通常会引入一些数据 Shuffle 算子，以及多线程之间同步的逻辑，这也会带来一些不必要的资源浪费。

Doris 中默认值为 0，即 BE 的 CPU 核数目的一半，这个值考虑了单查询和并发的资源利用的情况，通常不需要用户介入调整。当存在性能瓶颈时可以参考下面示例进行必要的调整。Doris 在持续完善自适应的策略，通常建议在特定场景或 SQL 级别进行必要的调整。

假设 BE 的 CPU 核数为 16：

1. 对于单表的简单操作（如单表点查、where 扫描获取少量数据，limit 少量数据，命中物化视图) 并行度可设置为 1

说明：单表的简单操作，只有一个 Fragment，查询的瓶颈通常在数据扫描处理上，数据扫描线程和查询执行的线程是分开的，数据扫描线程会自适应的做并行的扫描，这里的瓶颈不是查询线程，并行度可以直接设置为 1。

2. 对于两表 JOIN 的查询/聚合查询，如果数据量很大，确认是 CPU 瓶颈型查询，并行度可设置为 16。

说明：对于两表 JOIN/聚合查询，这类数据计算密集型的查询，如果观察 CPU 没有打满，可以考虑在默认值的基础上，继续调大并行度，利用 Pipeline 执行引擎的并行能力，充分利用 CPU 资源参与计算。并不能保证每个 PipelineTask 都能将分配给它的 CPU 资源使用到极限。因此，可以适当调整并行度，比如设为 16，以更充分地利用 CPU。然而，不应无限制地增加并行度，设置为 48 根本不会带来实质性的收益，反而会增加线程调度开销和框架调度开销。

3. 对于压力测试场景，压测的多个查询的任务本身就能够充分利用 CPU，可以考虑并行度设置为 1。

说明：对于压力测试场景，压测的查询的任务足够多。过大的并行度同样带来了线程调度开销和框架调度开销，这里需要设置为 1 是比较合理的。

4. 复杂查询的情况要根据 Profile 和机器负载，灵活调整，这里建议使用默认值，如果不合适可以尝试 4-2-1 的阶梯方式调整，观察查询表现和机器负载。

查询并行度调优

Doris 可以手动指定查询的并行度，以调整查询执行时并行执行的效率。

SQL 级别调整

通过 SQL HINT 来指定单个 SQL 的并行度，这样可以灵活控制不同 SQL 的并行度来取得最佳的执行效果

```
select /*+SET_VAR("parallel_pipeline_task_num=8")*/ * from nation, lineitem where lineitem.l_
    ↪ suppkey = nation.n_nationkey
select /*+SET_VAR("parallel_pipeline_task_num=8,runtime_filter_mode=global")*/ * from nation,
    ↪ lineitem where lineitem.l_suppkey = nation.n_nationkey
```

会话级别调整

通过 session variables 来调整会话级别的并行度，session 中的所有查询语句都将以指定的并行度执行。请注意，即使是单行查询的 SQL，也会使用该并行度，可能导致性能下降。

```
set parallel_pipeline_task_num = 8;
```

全局调整

如果需要全局调整，通常涉及 cpu 利用率的调整，可以 global 设置并行度

```
set global parallel_pipeline_task_num = 8;
```

数据分片和并行度

从 2.1 版本开始，Doris 支持并行度和数据分片数量的解耦。

在之前的版本中，并行度不能大于查询涉及到的数据分片数量。比如一个查询涉及到 5 个分片（Tablet），则最大的 Scan 并发度只有 5。这会导致一些较大的数据分片无法进行并发读取。

新版本中，Doris 支持分片内部的并发读取。该功能自动开启，无需用户设置。

但需注意，该功能仅支持 Duplicate 和 Unique Key Merge-On-Write 表模型。对于 Aggregate 和 Unique Key Merge-On-Read 模型不适用。这两种模型下，查询并行度依然受限于分片数量。

最佳实践

案例 1：并行度过高导致高并发压力场景，CPU 使用率过高

当线上观察到 CPU 使用率过高，影响到部分低时延查询的性能时，可以考虑通过调整查询并行度来降低 CPU 使用率。由于 Doris 的设计理念是优先使用更多资源以最快速度获取查询结果，在某些线上资源紧张的场景下，可能会导致性能表现不佳。因此，适当调整并行度可以在资源有限的情况下提升查询的整体稳定性和效率。

设置并行度从默认的 0（CPU 核数的一半）到 4：

```
set global parallel_pipeline_task_num = 4;
```

global 设置后，对于当前链接和新建链接全局生效，已有的其他链接不生效。如果需要即时全部生效，可以重启 fe。调整之后，CPU 使用率降低到原先高峰值的 60%，降低了部分时延较低的查询的影响。

案例 2：调高并行度，进一步利用 CPU 加速查询

当前 Doris 默认的并行度为 CPU 核数的一半，部分计算密集型的场景并不能充分利用满 CPU 进行查询加速，

```
select sum(if(t2.value is null, 0, 1)) exist_value, sum(if(t2.value is null, 1, 0)) no_exist_
    ↪ value
from  t1 left join  t2 on t1.key = t2.key;
```

在左表 20 亿，右表 500 万的场景上，上述 SQL 需要执行 28s。观察 Profile：

```
HASH_JOIN_OPERATOR (id=3 , nereids_id=448):
  - PlanInfo
      - join op: LEFT OUTER JOIN(BROADCAST)[]
      - equal join conjunct: (value = value)
      - cardinality=2,462,330,332
      - vec output tuple id: 5
      - output tuple id: 5
      - vIntermediate tuple ids: 4
      - hash output slot ids: 16
      - projections: value
      - project output tuple id: 5
```

```
- BlocksProduced: sum 360.099K (360099), avg 45.012K (45012), max 45.014K (45014), min 45.011K
    ↪ (45011)
- CloseTime: avg 8.44us, max 13.327us, min 5.574us
- ExecTime: avg 26sec153ms, max 26sec261ms, min 26sec33ms
- InitTime: avg 7.122us, max 13.395us, min 4.541us
- MemoryUsage: sum , avg , max , min
  - PeakMemoryUsage: sum 1.16 MB, avg 148.00 KB, max 148.00 KB, min 148.00 KB
  - ProbeKeyArena: sum 1.16 MB, avg 148.00 KB, max 148.00 KB, min 148.00 KB
- OpenTime: avg 2.967us, max 4.120us, min 1.562us
- ProbeRows: sum 1.4662330332B (1462330332), avg 182.791291M (182791291), max 182.811875M
    ↪ (182811875), min 182.782658M (182782658)
- ProjectionTime: avg 165.392ms, max 169.762ms, min 161.727ms
- RowsProduced: sum 1.462330332B (1462330332), avg 182.791291M (182791291), max 182.811875M
    ↪ (182811875), min 182.782658M (182782658)
```

这里主要的时间耗时：ExecTime: avg 26sec153ms, max 26sec261ms, min 26sec33ms都发生在 Join 算子上，同时处理的数据总量：ProbeRows: sum 1.4662330332B有 14 亿，这是一个典型的 CPU 密集的运算情况。观察机器监控，发现 CPU 资源没有打满，CPU 利用率为 60%，此时可以考虑调高并行度来进一步利用空闲的 CPU 资源进行加速。

设置并行度如下：

```
set parallel_pipeline_task_num = 16;
```

查询耗时从 28s 降低到 19s，cpu 利用率从 60% 上升到 90%。

总结

通常用户不需要介入调整查询并行度，如需要调整，需要注意以下事项：

1. 建议从 CPU 利用率出发。通过 PROFILE 工具输出观察是否是 CPU 瓶颈，尝试进行并行度的合理修改
2. 单 SQL 调整比较安全，尽量不要全局做过于激进的修改

2.13.8.3 常见调优参数

参数	说明	默认值	使用场景
enable_nereids_planner	是否打开新优化器	TRUE	低版本升级等场景，此开关初始为 false；升级后，可设置为 true
enable_nereids_dml	是否启用新优化器的 DML 支持	TRUE	低版本升级等场景，此开关初始为 false；升级后，可设置为 true
parallel_pipeline_task_num	Pipeline 并行度	0	低版本升级等场景，此值为之前设置的固定值；升级后，可设置为 0，表示由系统自适应策略决定并行度
runtime_filter_mode	Runtime Filter 类型	GLOBAL	低版本升级等场景，此值为 NONE，表示不启用 Runtime Filter；升级后，可设置为 GLOBAL，表示默认启用 Runtime Filter

2.13.9 优化技术原理

2.13.9.1 查询优化器介绍

2.13.9.1.1 研发背景

在当前的信息技术环境中，查询优化器面临着多重挑战：一方面，它们需要处理用户日益复杂的查询语句和多样化的查询场景；另一方面，用户对查询实时性的要求愈发严格，渴望能够即时获取所需结果。此外，为了应对不断出现的新需求，查询优化器必须具备快速迭代与灵活适应的能力。

基于这样的背景，Doris 开始着手研发了一款全新的查询优化器。该优化器依托现代优化器架构，旨在更高效地应对当前 Doris 场景的查询请求，同时提供卓越的扩展性，为未来可能出现的更复杂需求奠定坚实基础。

2.13.9.1.2 Doris 查询优化器优势

更聪明

优化器将每个 RBO（基于规则的优化）和 CBO（基于成本的优化）的优化点，以规则的形式清晰地呈现出来。针对每一个规则，优化器都提供了一组描述查询计划形状的模式，这些模式能够精确地匹配可优化的查询计划。因此，优化器能够更好地支持诸如多层子查询嵌套等更为复杂的查询语句。

同时，优化器的 CBO 基于先进的 Cascades 框架，充分利用了丰富的数据统计信息、数据特征信息以及精心调优的代价模型。这使得优化器在处理多表 Join 等复杂查询时，能够游刃有余，轻松应对。

更稳定

优化器的所有优化规则均在逻辑执行计划树上完成。查询语法语义解析完成后，查询会被转换为树状结构。相比旧优化器，新优化器的内部数据结构更为合理、统一。

以子查询处理为例，新优化器基于新的数据结构，避免了旧优化器中众多规则对子查询的单独处理，从而降低了优化规则出现逻辑错误的可能性。

更灵活

优化器的架构设计合理且现代，使得扩展优化规则和处理阶段变得非常方便。因此，我们能够迅速增加新的功能，以满足不断变化的新需求。

2.13.9.1.3 优化器工作原理

整体流程

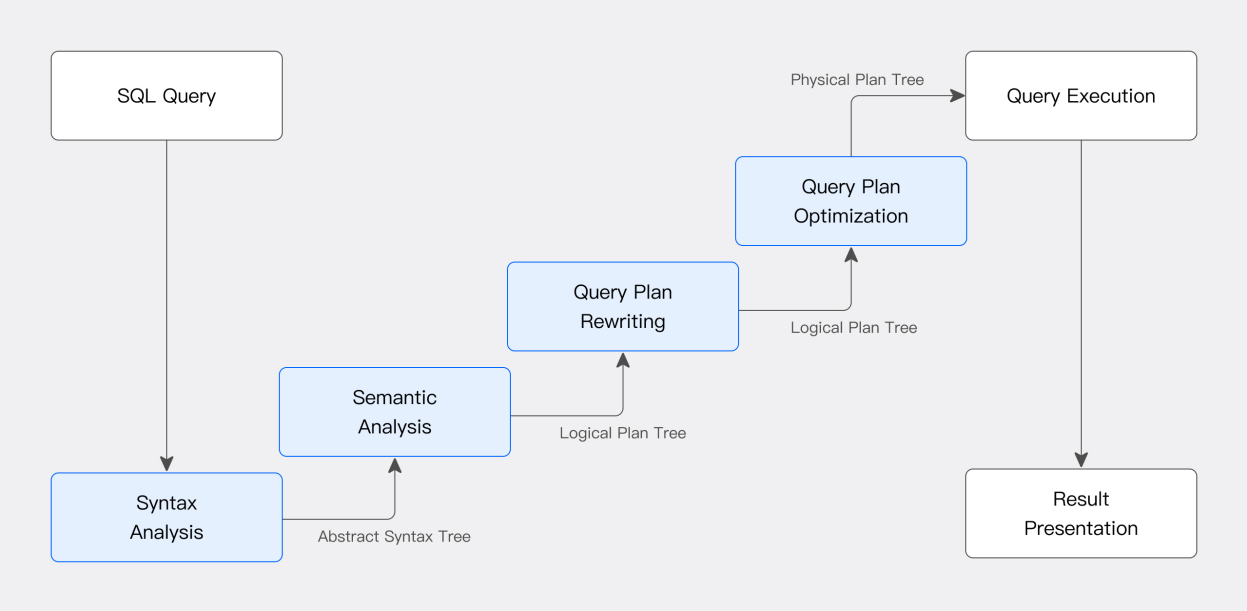


图 56: 优化器工作原理

优化器的执行流程大致分为以下几个步骤：

1. 语法分析： 优化器会尝试将 SQL 文本转换为抽象语法树（AST）。如果 SQL 文本合法，则继续进行后续步骤；如果非法，则会报错并终止执行。
2. 语义分析： 优化器会对 AST 中的元素进行语义分析。这一步骤会检查 SQL 查询中的表、列、函数等是否存在，以及它们的使用是否符合语法和语义规则。如果语义合法，则继续执行；如果语义非法，则会报错并终止执行。
3. 改写查询计划（RBO）： 在语法和语义分析之后，优化器会进行基于规则的优化（RBO）。这一步骤会通过一系列预定义的规则对查询计划进行改写，以确定性地优化执行速度。常见的优化手段包括列裁剪、谓词下推、分区裁剪等。
4. 优化查询计划（CBO）： 最后，优化器会进行基于代价的优化（CBO）。在这一步骤中，优化器会在搜索空间中枚举等价的计划集合，并评估它们的执行代价。通过比较不同计划的执行代价，优化器会选择代价最小的计划作为最终的执行计划。这一步骤旨在确保查询能够以最高效的方式执行，从而提供最佳的性能。

2.13.9.1.4 常用会话变量

1. 设置规划超时时间 `nereids_timeout_second`

- 此变量用于设置查询规划的最大允许时间。当规划时间超出该设定值时，查询规划将被终止，并返回错误信息。在规划查询语句的过程中，系统会获取 SQL 中涉及的所有表的读锁，这一机制的主要目的是维护集群的稳定性，防止因规划时间过长而造成的资源过度占用以及锁冲突问题。
- 默认值： 30s

- 适用场景：当查询涉及大量外部表或查询语句特别复杂时，可以适当增加此值，以确保查询能够正常进行。

2.13.9.2 Pipeline 执行引擎

Doris 的并行执行模型是一种 Pipeline 执行模型，主要参考了Hyper论文中 Pipeline 的实现方式，Pipeline 执行模型能够充分释放多核 CPU 的计算能力，并对 Doris 的查询线程的数目进行限制，从而解决 Doris 的执行线程膨胀的问题。它的具体设计、实现和效果可以参阅 DSIP-027 以及 DSIP-035。Doris 3.0 之后，Pipeline 执行模型彻底替换了原有的火山模型，基于 Pipeline 执行模型，Doris 实现了 Query、 DDL、 DML 语句的并行处理。

2.13.9.2.1 物理计划

为了更好的理解 Pipeline 执行模型，首先需要介绍一下物理查询计划中两个重要的概念：PlanFragment 和 PlanNode。我们使用下面这条 SQL 作为例子：

```
SELECT k1, SUM(v1) FROM A,B WHERE A.k2 = B.k2 GROUP BY k1 ORDER BY SUM(v1);
```

FE 首先会把它翻译成下面这种逻辑计划，计划中每个节点就是一个 PlanNode，每种 Node 的具体含义，可以参考查看物理计划的介绍。

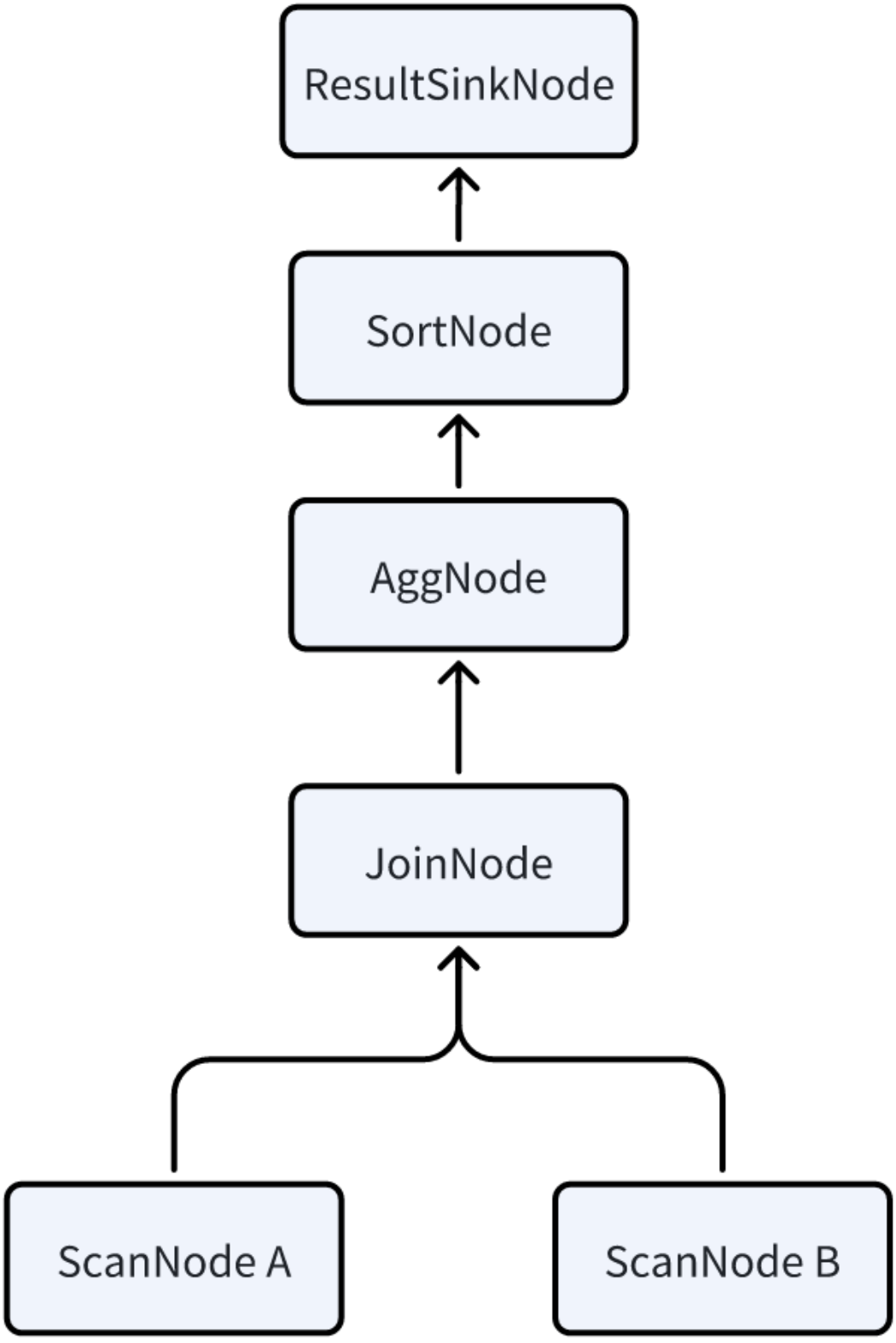


图 57: pip_exec_1

由于 Doris 是一个 MPP 的架构，每个查询都会尽可能的让所有的 BE 都参与进来并行执行，来降低查询的延时。所以还需要将上述逻辑计划拆分为一个物理计划，拆分物理计划基本上就是在逻辑计划中插入了 DataSink 和 ExchangeNode，通过这两个 Node 完成了数据在多个 BE 之间的 Shuffle。拆分完成后，每个 PlanFragment 相当于包含了一部分 PlanNode，可以作为一个独立的任务发送给 BE，每个 BE 完成了 PlanFragment 内包含的 PlanNode 的计算后，通过 DataSink 和 ExchangeNode 这两个算子把数据 shuffle 到其他 BE 上来进行接下来的计算。

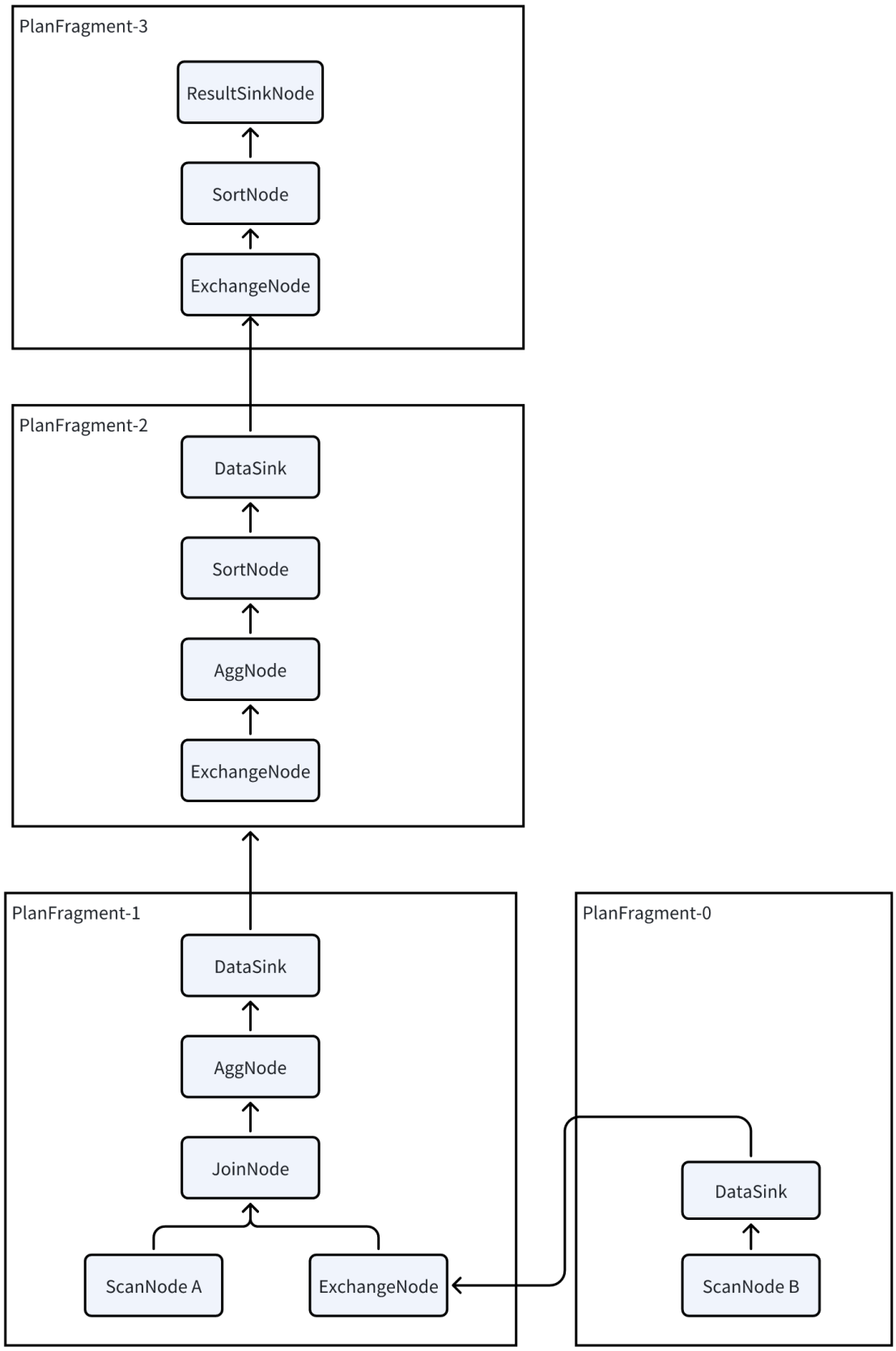


图 58: pip_exec_2

所以 Doris 的规划分为 3 层：PLAN：执行计划，一个 SQL 会被执行规划器翻译成一个执行计划，之后执行计划会提供给执行引擎执行。

FRAGMENT：由于 DORIS 是一个分布式执行引擎。一个完整的执行计划会被切分为多个单机的执行片段。一个 FRAGMENT 代表一个完整的单机执行片段。多个 FRAGMENT 组合在一起，构成一个完整的 PLAN。

PLAN NODE：算子，是执行计划的最小单位。一个 FRAGMENT 由多个算子构成。每一个算子负责一个实际的执行逻辑，比如聚合，连接等

2.13.9.2.2 Pipeline 执行

PlanFragment 是 FE 发往 BE 执行任务的最小单位。BE 可能会收到同一个 Query 的多个不同的 PlanFragment，每个 PlanFragment 都会被单独的处理。在收到 PlanFragment 之后，BE 会把 PlanFragment 拆分为多个 Pipeline，进而启动多个 PipelineTask 来实现并行执行，提升查询效率。

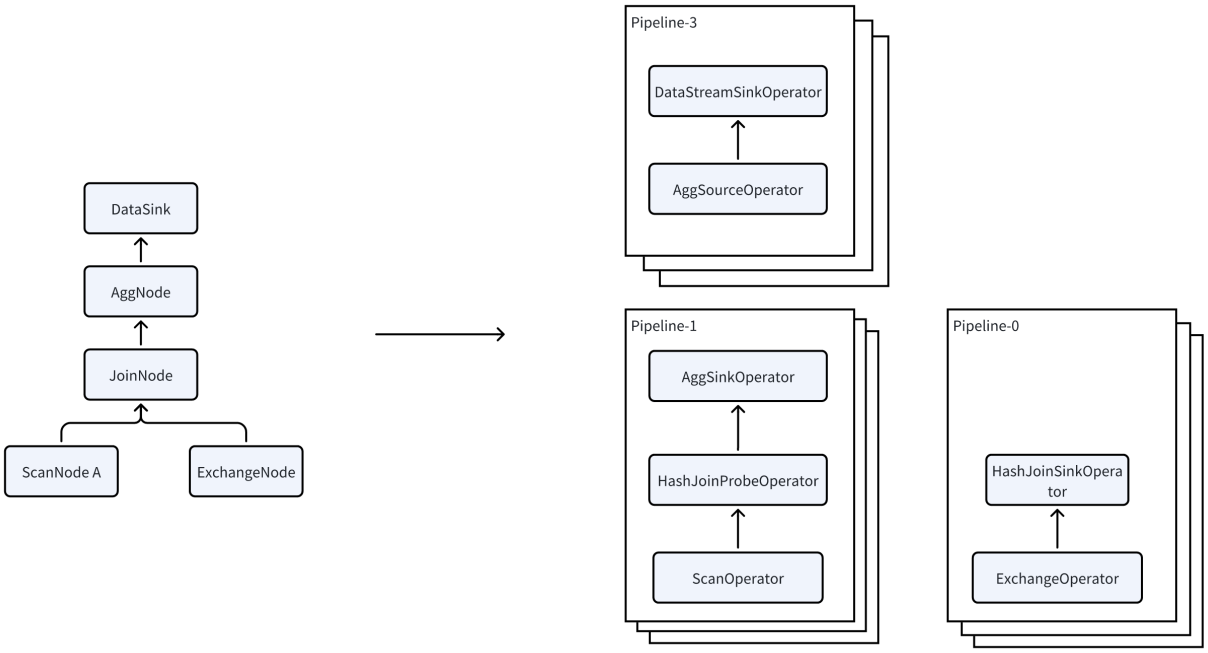


图 59: pip_exec_3

Pipeline

一个 Pipeline 有一个 SourceOperator 和一个 SinkOperator 以及中间的多个其他 Operator 组成。SourceOperator 代表从外部读取数据，可以是一个表（OlapTable），也可以是一个 Buffer（Exchange）。SinkOperator 表示数据的输出，输出可以是通过网络 shuffle 到别的节点，比如 DataStreamSinkOperator，也可以是输出到 HashTable，比如 Agg 算子，JoinBuildHashTable 等。

图 60: pip_exec_4

多个 Pipeline 之间实际是有依赖关系的，以 JoinNode 为例，实际被拆分到了 2 个 Pipeline 里。其中 Pipeline-0 是读取 Exchange 的数据，来构建 HashTable；Pipeline-1 是从表里读取数据，来进行 Probe。这 2 个 Pipeline 之间是有关联关系的，只有 Pipeline-0 运行完毕之后才能执行 Pipeline-1。这两者之间的依赖关系，称为 Dependency。当 Pipeline-0 运行完毕后，会调用 Dependency 的 set_ready 方法通知 Pipeline-1 可执行。

PipelineTask

Pipeline 实际还是一个逻辑概念，他并不是一个可执行的实体。在有了 Pipeline 之后，需要进一步的把 Pipeline 实例化为多个 PipelineTask。将需要读取的数据分配给不同的 PipelineTask 最终实现并行处理。同一个 Pipeline 的多个 PipelineTask 之间的 Operator 完全相同，他们的区别在于 Operator 的状态不一样，比如读取的数据不一样，构建出的 HashTable 不一样，这些不一样的状态，我们称之为 LocalState。每个 PipelineTask 最终都会被提交到一个线程池中作为独立的任务执行。在 Dependency 这种触发机制下，可以更好的利用多核 CPU，实现充分的并行。

Operator

在大多数时候，Pipeline 中的每个 Operator 都对应了一个 PlanNode，但是有一些特殊的算子除外：- JoinNode，被拆分为 JoinBuildOperator 和 JoinProbeOperator - AggNode 被拆分为 AggSinkOperator 和 AggSourceOperator - SortNode 被拆分为 SortSinkOperator 和 SortSourceOperator 基本原理是，对于一些 breaking 算子（指需要把所有的数据都收集齐之后才能运算的算子），把灌入数据的部分拆分为 Sink，然后把从这个算子里获取数据的部分称为 Source。

2.13.9.2.3 Scan 并行化

扫描数据是一个非常重的 IO 操作，它需要从本地磁盘读取大量的数据（如果是数据湖的场景，就需要从 HDFS 或者 S3 中读取，延时更长），需要比较多的时间。所以我们在 ScanOperator 中引入了并行扫描的技术，ScanOperator 会动态的生成多个 Scanner，每个 Scanner 扫描 100 万 -200 万行左右的数据，每个 Scanner 在做数据扫描时，完成相应的数据解压、过滤等计算任务，然后把数据发送给一个 DataQueue，供 ScanOperator 读取。

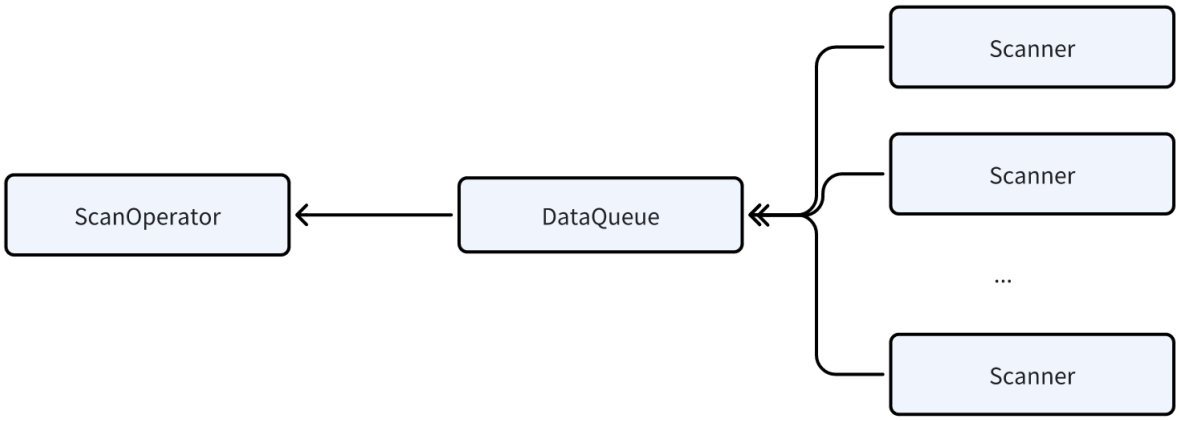


图 61: pip_exec_5

通过并行扫描的技术可以有效的避免由于分桶不合理或者数据倾斜导致某些 ScanOperator 执行时间特别久，把整个查询的延时都拖慢的问题。

2.13.9.2.4 Local Shuffle

在 Pipeline 执行模型中，Local Exchange 作为一个 Pipeline Breaker 出现，是在本地将数据重新分发至各个执行任务的技术。它把上游 Pipeline 输出的全部数据以某种方式（HASH / Round Robin）均匀分发到下游 Pipeline 的全部 Task 中。解决执行过程中的数据倾斜的问题，使执行模型不再受数据存储以及 plan 的限制。接下来我们举例来说明 Local Exchange 的工作逻辑。我们用上述例子中的 Pipeline-1 为例子进一步阐述 Local Exchange 如何可以避免数据倾斜。

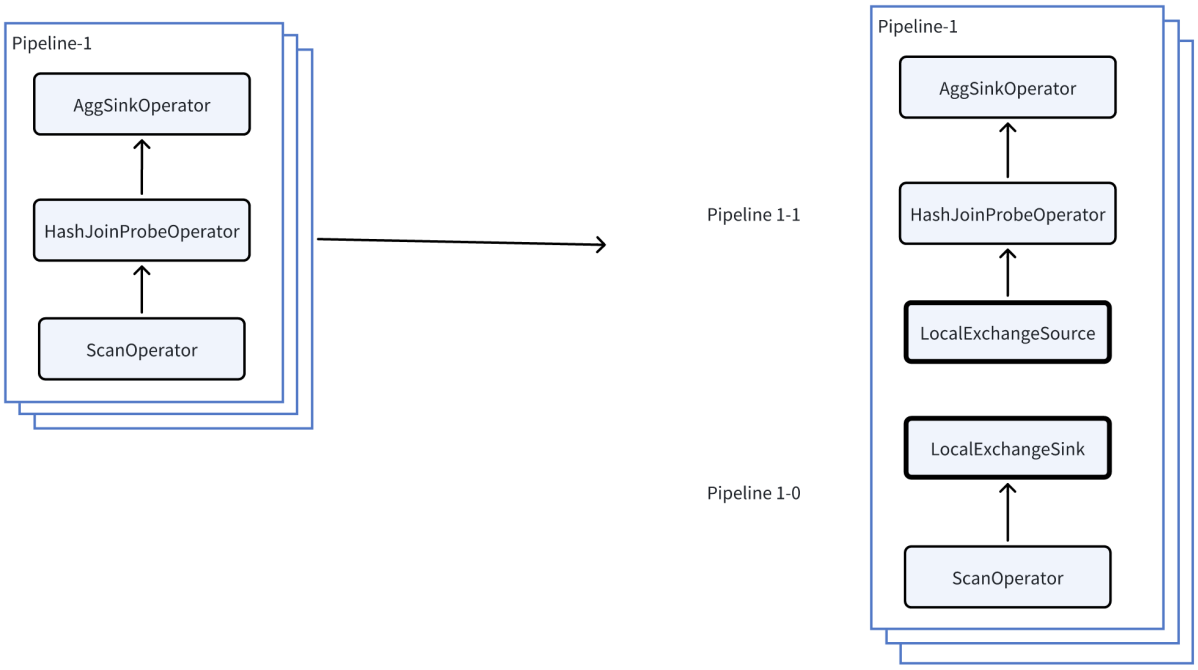


图 62: pip_exec_6

如上图所示，首先，通过在 Pipeline 1 中插入 Local Exchange，我们把 Pipeline 1 进一步拆分成 Pipeline 1-0 和 Pipeline 1-1。此时，我们不妨假设当前并发等于 3（每个 Pipeline 有 3 个 task），每个 task 读取存储层的一个 bucket，而 3 个 bucket 中数据行数分别是 1，1，7。则插入 Local Exchange 前后的执行变化如下：

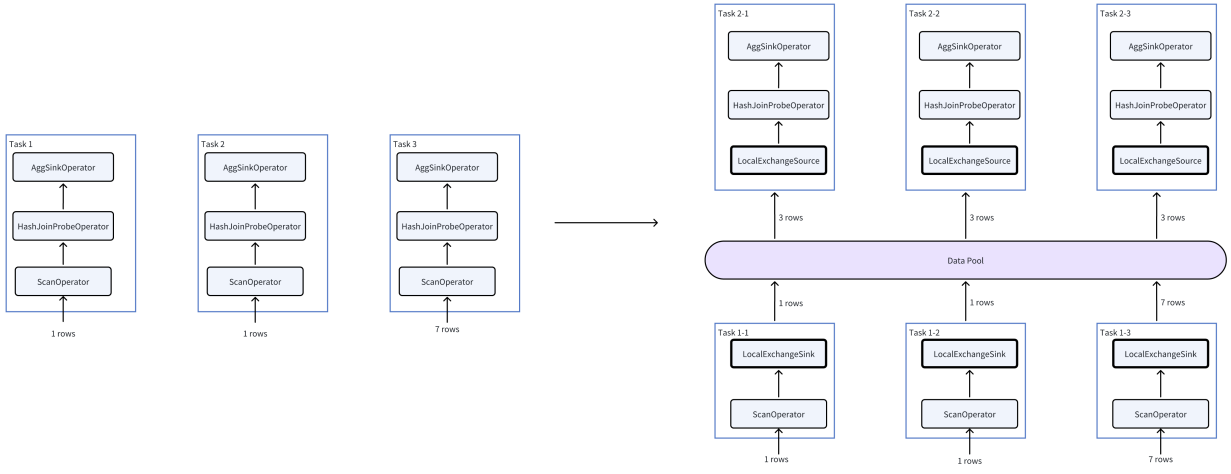


图 63: pip_exec_7

从图右可以看出，HashJoin 和 Agg 算子需要处理的数据量从 (1, 1, 7) 变成了 (3, 3, 3)，从而避免了数据倾斜。在 Doris 中，Local Exchange 根据一系列规则来决定是否被规划，例如当查询耗时比较大的 Join、聚合、窗口函数等

算子需要被执行时，我们就需要使用 Local Exchange 来尽可能避免数据倾斜。

2.13.9.3 Runtime Filter

Runtime Filter 主要分为两种，Join Runtime Filter 与 TopN Runtime Filter。本文将详细介绍两类 Runtime Filter 的工作原理、使用指南与优化方法。

2.13.9.3.1 Join Runtime Filter

Join Runtime Filter (以下简称 JRF) 是一种优化技术，它根据运行时数据在 Join 节点通过 Join 条件动态生成 Filter。此技术不仅能降低 Join Probe 的规模，还能有效减少数据 IO 和网络传输。

工作原理

我们以一个类似 TPC-H Schema 上的 Join 为例，来说明 JRF 的工作原理。

假设数据库中有两张表：

- 订单表（orders），包含 1 亿行数据，记录订单号 (o_orderkey)、客户编号 (o_custkey) 以及订单的其它信息。
- 客户表（customer），包含 10 万行数据，记录客户编号 (c_custkey)、客户国籍 (c_nation) 以及客户的其它信息。该表共记录了 25 个国家的客户，每个国家约有 4 千客户。

统计客户来自中国的订单数量，查询语句如下：

```
select count(*)
from orders join customer on o_custkey = c_custkey
where c_nation = "china"
```

此查询的执行计划主体是一个 Join，如下图所示：

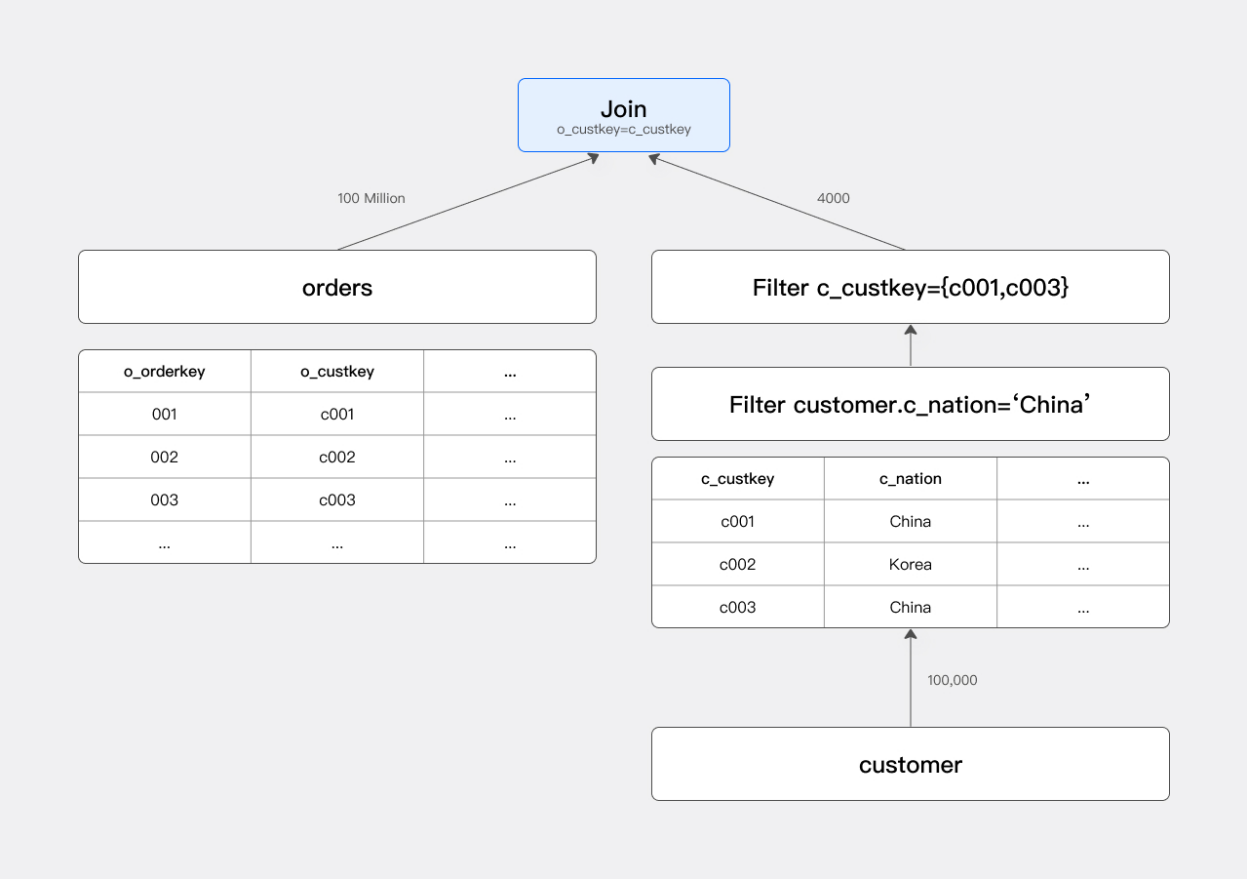


图 64: Join Runtime Filter

在没有 JRF 的情况下，Scan 节点会扫描 orders 表，读入 1 亿行数据，Join 节点则对这 1 亿行数据进行 Hash Probe，最后生成 Join 结果。

1. 优化思路

过滤条件 `c_nation = "china"` 会过滤掉所有非中国的客户，因此参与 Join 的 customer 只是 customer 表的一部分（约 1/25）。后续的 Join 条件为 `o_custkey = c_custkey`，所以我们需要关注过滤结果中 `c_custkey` 列有哪些被选中的 custkey。将过滤后的 `c_custkey` 记为集合 A。在下文中，我们用集合 A 专门指代参与 Join 的 `c_custkey` 集合。

如果将集合 A 作为一个 in 条件推给 orders 表，那么 orders 表的 Scan 节点就可以对 orders 进行过滤。这就类似增加了一个过滤条件 `c_custkey in (c001, c003)`。

基于以上的优化思路，SQL 可以优化为：

```
select count(*)
from orders join customer on o_custkey = c_custkey
where c_nation = "china" and o_custkey in (c001, c003)
```

优化后的执行计划如下图所示：

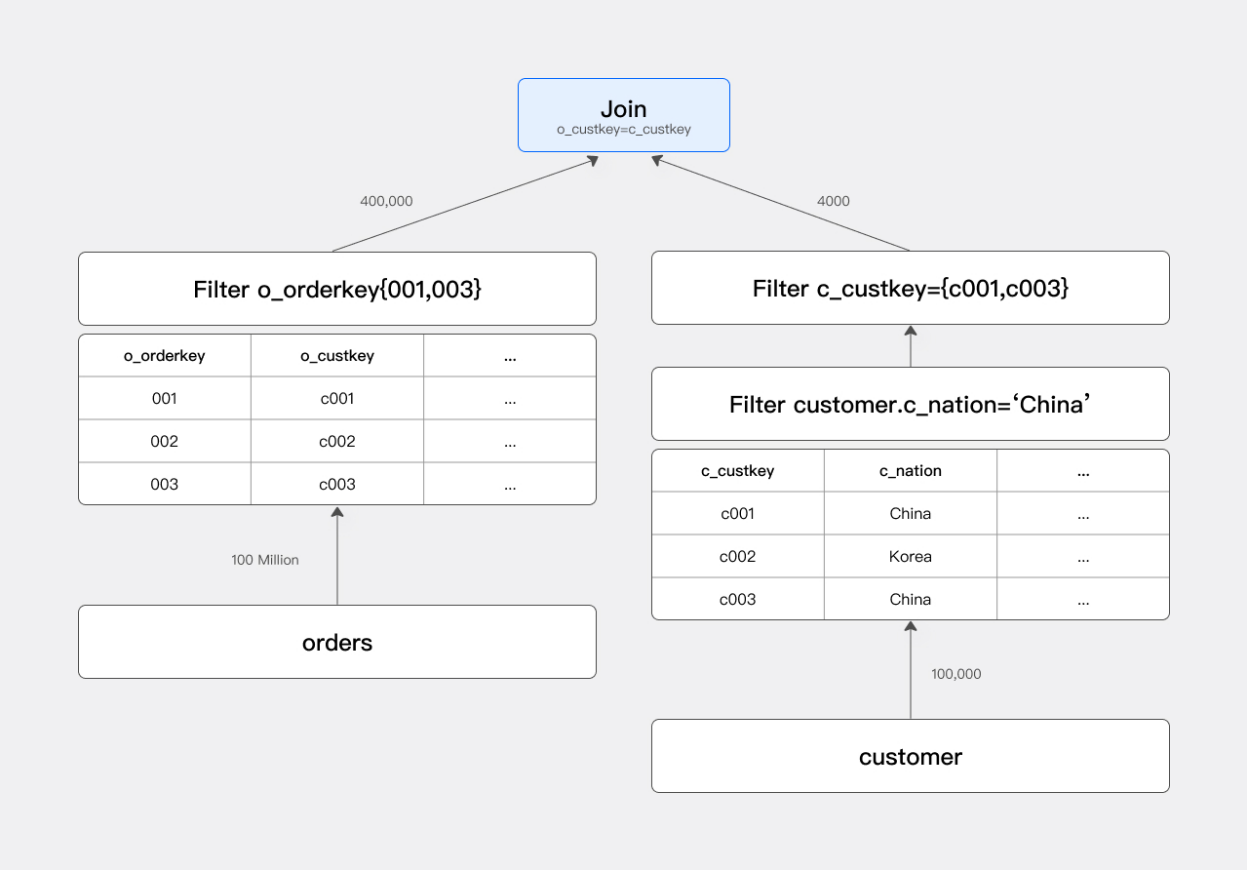


图 65: join-runtime-filter-2

可以看到，通过增加 Orders 表上的过滤条件，实际参与 Join 的 Orders 行数从 1 亿下降到 40 万，查询速度得到大幅提升。

2. 实现方法

上述优化效果显著，但优化器并不知道实际被选中的 `c_custkey`，即集合 A。因此，优化器无法在优化阶段静态分析生成一个固定的 in-predicate 过滤算子。

在实际应用中，我们会在 Join 节点收集右侧数据后，运行时生成集合 A，并将集合 A 下推给 orders 表的 scan 节点。我们通常将这个 JRF 记为：`RF(c_custkey -> [o_custkey])`。

Doris 是一个分布式数据库，为了满足分布式场景的需求，JRF 还需要进行一次合并。假设上述例子中的 Join 是一个 Shuffle Join，那么这个 Join 有多个 Instance，每个 Join 只处理 orders 和 customer 表的一个分片。因此，每个 Join Instance 都只得到了集合 A 的一部分。

在当前 Doris 的版本中，我们会选出一个节点作为 Runtime Filter Manager。每个 Join Instance 根据各自分片中的 `c_custkey` 生成 Partial JRF，并发送给 Manager。Manager 收集所有 Partial JRF 后，合并生成 Global JRF，再将 Global JRF 发送给 orders 表的相关 Scan Instance。

生成 Global JRF 的流程如下图所示：

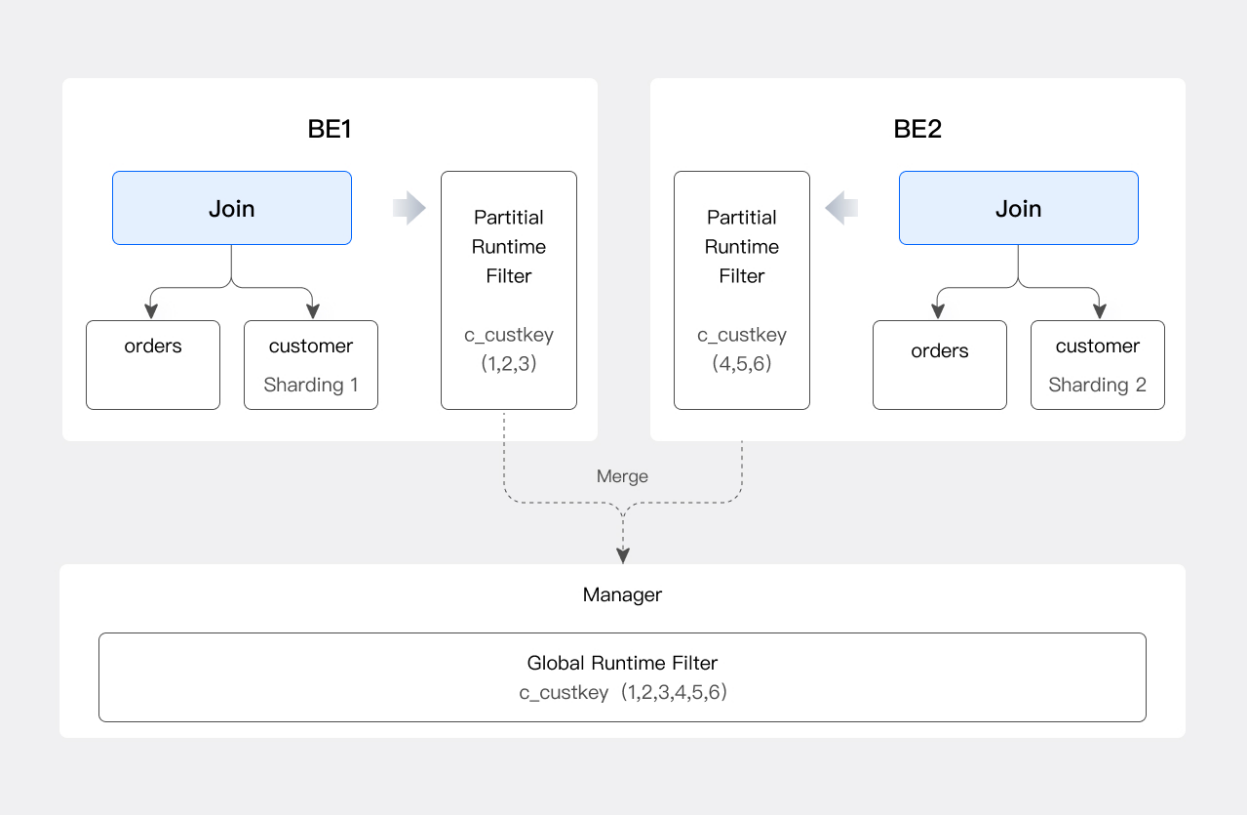


图 66: Global JRF

Filter 类型

有多种数据结构均可用于实现 JRF，但它们在生成、合并、传输、应用等方面效率各异，因此各自适用于不同的场景。

1. In Filter

这是实现 JRF 的最简单方式。以之前的例子为例，使用 In Filter 时，执行引擎会在左表上生成谓词 `o_custkey in (...A 中元素列表...)`。通过这个 In 过滤条件，可以对 orders 表进行过滤。当集合 A 中元素数量较少时，In Filter 的效率较高。

然而，当集合 A 中元素数量过大时，使用 In Filter 会带来性能问题：

1. 首先，生成 In Filter 的成本较高，尤其是在需要进行 JRF 合并的情况下。因为从不同数据分片对应的 Join 节点中收集的值可能会有重复，例如，如果 `c_custkey` 不是表的主键，那么 `c001`、`c003` 这样的 `c_custkey` 可能出现多次，这时就需要进行去重操作，而这个过程比较耗时。
2. 其次，当集合 A 元素较多时，Join 节点与 orders 表的 Scan 节点之间传输数据的代价也较高。
3. 最后，orders 表的 Scan 节点执行 In 谓词也会消耗时间。

基于上述考虑，我们引入了 Bloom Filter。

2. Bloom Filter

如果对 Bloom Filter 不太了解，可以将其理解为一个哈希表。简单来说，Bloom Filter 就是一组叠加的哈希表。使用 Bloom Filter（或哈希表）进行过滤，利用了以下性质：

- 基于集合 A 生成哈希表 T，如果一个元素不在哈希表 T 中，那么可以断定这个元素也不在集合 A 中。反之，则不成立。

因此，如果一个 o_orderkey 被 Bloom Filter 过滤掉，那么可以断定在 Join 的右侧没有相等的 c_custkey。但由于哈希碰撞，一些 o_custkey 即使没有相等的 c_custkey，也可能通过 Bloom Filter。

所以，虽然 Bloom Filter 不能实现精准过滤，但仍然能达到一定的过滤效果。

- 哈希表的桶数量决定了过滤的准确率。桶数量越大，Filter 的大小越大，准确性越高，但生成、传输、使用的计算代价也越大。

因此，Bloom Filter 的大小也需要在过滤效果和使用代价之间取得平衡。基于此，我们设置了一组可配参数来约束 Bloom Filter 的最大和最小值，分别是 RUNTIME_BLOOM_FILTER_MIN_SIZE 和 RUNTIME_BLOOM_FILTER_MAX_SIZE。

3. Min/Max Filter

除了 Bloom Filter 外，还有 Min-Max Filter 可用于进行模糊过滤。如果数据列是有序的，那么 Min-Max Filter 会有很好的过滤效果。此外，生成、合并、使用 Min-Max Filter 的代价也远低于 In Filter 和 Bloom Filter。

对于非等值的 Join，In Filter 和 Bloom Filter 都无法工作，但 Min-Max Filter 仍然可以继续发挥作用。假设我们将上例中的查询修改为：

```
select count(*)
from orders join customer on o_custkey > c_custkey
where c_name = "China"
```

那么可以选出过滤后最大的 c_custkey，记为 n，并将 n 传给 orders 表的 scan 节点。scan 节点则会只输出 o_custkey > n 的行。

查看 Join Runtime Filter

查看一个 Query 上生成了哪些 JRF，可以通过 explain / explain shape plan / explain physical plan 命令来查看。

我们以 TPC-H Schema 为例，详细说明通过这三个命令如何查看 JRF。

```
select count(*) from orders join customer on o_custkey=c_custkey;
```

1. Explain

在传统 Explain 文本中，JRF（Join Reference File）的信息分布通常出现在 Join 节点和 Scan 节点中，具体展示如下图所示：

```
4: VHASH JOIN(258)
| join op: INNER JOIN(PARTITIONED)[]
|  equal join conjunct: (o_custkey[#10] = c_custkey[#0])
|  runtime filters: RF000[bloom] <- c_custkey[#0] (150000000/134217728/16777216)
|  cardinality=1,500,000,000
```

```
|  vec output tuple id: 3
|  output tuple id: 3
|  vIntermediate tuple ids: 2
|  hash output slot ids: 10
|  final projections: o_custkey[#17]
|  final project output tuple id: 3
|  distribute expr lists: o_custkey[#10]
|  distribute expr lists: c_custkey[#0]
|
|---1: VEXCHANGE
|      offset: 0
|      distribute expr lists: c_custkey[#0]
3: VEXCHANGE
|  offset: 0
|  distribute expr lists:

PLAN FRAGMENT 2
| PARTITION: HASH_PARTITIONED: o_orderkey[#8]
| HAS_COLO_PLAN_NODE: false
| STREAM DATA SINK
|   EXCHANGE ID: 03
|   HASH_PARTITIONED: o_custkey[#10]

2: VOlapScanNode(242)
|  TABLE: regression_test_nereids_tpch_shape_sf1000_p0.orders(orders)
|  PREAGGREGATION: ON
|  runtime filters: RF000[bloom] -> o_custkey[#10]
|  partitions=1/1 (orders)
|  tablets=96/96, tabletList=54990,54992,54994 ...
|  cardinality=0, avgRowSize=0.0, numNodes=1
|  pushAggOp=NONE
```

- Join 端：`runtime filters: RF000[bloom] <- c_custkey[#0] (150000000/134217728/16777216)`

这表示生成了一个 Bloom Filter，编号 000，它以 c_custkey 字段作为输入生成 JRF。后面的三个数字和 Bloom Filter Size 计算相关，我们可以暂时忽略。

- Scan 端：`runtime filters: RF000[bloom] -> o_custkey[#10]`

这表示 000 号 JRF 将作用在 orders 表的 Scan 节点上，我们用 JRF 对 o_custkey 字段进行过滤。

2. Explain Shape Plan

在 Explain Plan 系列中，我们以 Shape Plan 为例说明如何查看 JRF。

```
mysql> explain shape plan select count(*) from orders join customer on o_custkey=c_custkey where
    ↪ c_nationkey=5;
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
PhysicalResultSink
    ↪
    ↪ |
--hashAgg[GLOBAL]
    ↪
    ↪ |
----PhysicalDistribute[DistributionSpecGather]
    ↪                                                                                 |
------hashAgg[LOCAL]
    ↪
    ↪ |
--------PhysicalProject
    ↪
    ↪ |
----------hashJoin[INNER_JOIN shuffle]
    ↪                                                                                     |
------------hashCondition=((orders.o_custkey=customer.c_custkey)) otherCondition=() buildRFs:RF0
    ↪ c_custkey->[o_custkey]     |
--------------PhysicalProject
    ↪
    ↪ |
----------------PhysicalOlapScan[orders] apply RFs: RF0
    ↪                                                                   |
--------------PhysicalProject
    ↪
    ↪ |
----------------filter((customer.c_nationkey=5))
    ↪                                                                       |
------------------PhysicalOlapScan[customer]
    ↪                                                                         |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

```
11 rows in set (0.02 sec)
```

如上图所示：

- Join 端：build RFs: RF0 c_custkey -> [o_custkey]表示我们以 c_custkey 列的数据作为输入，生成一个作用到 o_custkey 的 JRF，编号 0。
- scan 端：PhysicalOlapScan[orders] apply RFs：RF0 表示 orders 表被 RF0 过滤。

3. Profile

在实际执行中，BE 会将 JRF 的使用情况输出到 Profile（需要 set enable_profile=true）。我们仍然以上面的 SQL 为例，在 Profile 中查看 JRF 执行的实际情况。

- Join 端

```
HASH_JOIN_SINK_OPERATOR (id=3 , nereids_id=367):(ExecTime: 703.905us)
    - JoinType: INNER_JOIN
    。。。
    - BuildRows: 617
    。。。
    - RuntimeFilterComputeTime: 70.741us
    - RuntimeFilterInitTime: 10.882us
```

这是 Join 的 Build 侧 Profile。在这个例子中，生成 JRF 耗时 70.741us，JRF 有 617 行数据作为输入。JRF 的 Size 和类型由 Scan 端展示。

- Scan 端

```
OLAP_SCAN_OPERATOR (id=2. nereids_id=351. table name = orders(orders)):(ExecTime: 13.32
    ↪ ms)
            - RuntimeFilters: : RuntimeFilter: (id = 0, type = bloomfilter, need_
                ↪ local_merge: false, is_broadcast: true, build_bf_cardinality: false,
            。。。
            - RuntimeFilterInfo:
                - filter id = 0 filtered: 714.761K (714761)
                - filter id = 0 input: 747.862K (747862)
            。。。
            - WaitForRuntimeFilter: 6.317ms
          RuntimeFilter: (id = 0, type = bloomfilter):
                - Info: [IsPushDown = true, RuntimeFilterState = READY,
                    ↪ HasRemoteTarget = false, HasLocalTarget = true, Ignored =
                    ↪ false]
                - RealRuntimeFilterType: bloomfilter
                - BloomFilterSize: 1024
```

在这个部分，我们需要关注以下几点信息：

1. 第 5/6 行，显示这个 JRF 的输入和过滤掉的行数。如果 Filtered 行数越大，那么这个 JRF 的效果越好。
2. 第 10 行，`IsPushDown = true`，表示 JRF 计算已经下推到存储层。如果下推到存储层，那么有利于存储层实现延迟物化，可以减少 IO。
3. 第 10 行，`RuntimeFilterState = READY`，表示 Scan 节点是否应用了 JRF。因为 JRF 采用 Try-best 机制，如果 JRF 生成需要很长时间，那么 Scan 节点在等待一段时间后开始扫描数据，这样输出的数据可能没有经过 JRF 的过滤。
4. 第 12 行，`BloomFilterSize: 1024`，这是一个 Bloom Filter，它的 size 是 1024 字节。

调优

关于 Join Runtime Filter 调优，在绝大多数情况下功能为自适应，用户不需要手动调优。

1. 开关 JRF

Session 变量 `runtime_filter_mode` 可以控制是否开启 JRF。

- 打开 JRF：`set runtime_filter_mode = GLOBAL`
- 关闭 JRF：`set runtime_filter_mode = OFF`

2. 设定 JRF Type

Session 变量 `runtime_filter_type` 可以控制 JRF 的类型，包括：

- `IN(1)`
- `BLOOM(2)`
- `MIN_MAX(4)`
- `IN_OR_BLOOM(8)`

`IN_OR_BLOOM` Filter 可以让 BE 根据实际数据行数自适应选择生成 `IN` Filter 还是 BLOOM Filter。

JRF type 可以叠加，即根据一个 Join 条件生成多个类型的 JRF。括号中的整数表示 Runtime Filter Type 的枚举值。如果希望生成多个 Type 的 JRF，那么将 `runtime_filter_type` 设置为对应枚举值之和。

例如，`set runtime_filter_type = 6`，那么将同时为每个 Join 条件生成 BLOOM Filter 和 `MIN_MAX` Filter。

再比如，在 2.1 版本中，`runtime_filter_type` 的默认值是 12，即同时生成 `MIN_MAX` Filter 和 `IN_OR_BLOOM`Filter。

3. 设定等待时间

前面提到 JRF 使用的是 Try-best 机制，Scan 节点启动前会等待 JRF。Doris 系统根据运行时状态计算等待时间。但在一些特殊情况下，可能等待时间不够，导致 JRF 没有生效，那么 Scan 节点的输出数据行数会比预期多。前面我们已经在 Profile 部分介绍了如何判断是否等到了 JRF。如果 Profile 中 Scan 节点 `RuntimeFilterState = false`，那么用户可以手动设置一个更长的等待时间。

Session 变量 `runtime_filter_wait_time_ms` 可以控制 Scan 节点等待 JRF 的时间。默认值是 1000 毫秒。

4. 裁剪 JRF

在某些情况下，JRF 可能没有过滤性。比如 orders 表和 customer 表存在主外键关系，但 customer 表上没有过滤条件，那么 JRF 的输入是全体 custkey，那么 orders 表中的所有行都能通过 JRF 过滤。优化器会根据列统计信息判断 JRF 的有效性进行裁剪。

Session 变量 enable_runtime_filter_prune = true/false 可以控制是否进行裁剪。默认值为 true。

2.13.9.3.2 TopN Runtime Filter

工作原理

在 Doris 中，数据是以分块流式的方式进行处理的。因此，当 SQL 语句中包含 topN 算子时，Doris 并不会计算所有结果，而是会生成一个动态的 Filter 来提前对数据进行过滤。

以下面 SQL 语句举例：

```
select o_orderkey from orders order by o_orderdate limit 5;
```

此 SQL 语句的执行计划如下图所示：

```
mysql> explain select o_orderkey from orders order by o_orderdate limit 5;
+-----------------------------------------------------+
| Explain String(Nereids Planner)                     |
+-----------------------------------------------------+
| PLAN FRAGMENT 0                                     |
|   OUTPUT EXPRS:                                     |
|     o_orderkey[#11]                                 |
|   PARTITION: UNPARTITIONED                          |
|                                                     |
|   HAS_COLO_PLAN_NODE: false                         |
|                                                     |
|   VRESULT SINK                                      |
|      MYSQL_PROTOCAL                                 |
|                                                     |
|   2:VMERGING-EXCHANGE                               |
|      offset: 0                                      |
|      limit: 5                                       |
|      final projections: o_orderkey[#9]              |
|      final project output tuple id: 2               |
|      distribute expr lists:                         |
|                                                     |
| PLAN FRAGMENT 1                                     |
|                                                     |
|   PARTITION: HASH_PARTITIONED: O_ORDERKEY[#0]       |
|                                                     |
|   HAS_COLO_PLAN_NODE: false                         |
|                                                     |
|   STREAM DATA SINK                                  |
|     EXCHANGE ID: 02                                 |
```

```
|     UNPARTITIONED                                      |
|                                                        |
|   1:VTOP-N(119)                                        |
|   |  order by: o_orderdate[#10] ASC                    |
|   |  TOPN OPT                                          |
|   |  offset: 0                                         |
|   |  limit: 5                                          |
|   |  distribute expr lists: O_ORDERKEY[#0]             |
|   |                                                    |
|   0:VOlapScanNode(113)                                 |
|      TABLE: tpch.orders(orders), PREAGGREGATION: ON    |
|      TOPN OPT:1                                        |
|      partitions=1/1 (orders)                           |
|      tablets=3/3, tabletList=135112,135114,135116      |
|      cardinality=150000, avgRowSize=0.0, numNodes=1    |
|      pushAggOp=NONE                                    |
+--------------------------------------------------------+
41 rows in set (0.06 sec)
```

在没有 topn filter 的情况下，scan 节点会依次读入 orders 表的每个数据块，并将这些数据块传递给 TopN 节点。TopN 节点通过堆排序维护着当前已扫描数据 orders 表中排名前 5 行。

由于一个数据 Block 大约包含 1024 行数据，因此在 TopN 处理了第一个数据块后，就能找到该数据块中排名第 5 的行。

假设这个 o_orderdate 是 1995-01-01，那么 scan 节点在输出第二个数据块时，就可以使用 1995-01-01 作为过滤条件，o_orderdate 大于 1995-01-01 的行则不需要再发送给 TopN 节点进行计算。

这个阈值会进行动态更新，例如，TopN 在处理第二个经过此阈值过滤的数据块时，如果发现了更小的 o_orderdate，那么 TopN 会将阈值更新为第一个和第二个数据块中排名第 5 的 o_orderdate。

查看 TopN Runtime Filter

通过 Explain 命令，我们可以查看优化器规划的 TopN runtime filter。

```
1:VTOP-N(119)
| order by: o_orderdate[#10] ASC
| TOPN OPT
| offset: 0
| limit: 5
| distribute expr lists: O_ORDERKEY[#0]
|

0:VLapScanNode[113]
    TABLE: regression_test_nereids_tpch_p0.(orders), PREAGGREGATION: ON
    TOPN OPT: 1
    partitions=1/1 (orders)
    tablets=3/3, tabletList=135112,135114,135116
    cardinality=150000, avgRowSize=0.0, numNodes=1
```

```
    pushAggOp: NONE
```

如上述例子所示：

1. TopN 节点上会显示 TOPN OPT，表示这个 TopN 节点会产生一个 TopN Runtime Filter。
2. Scan 节点上会标注它使用的 TopN Runtime Filter 是由哪个 TopN 节点产生的。比如，例子中 11 行，表示 orders 表的 Scan 节点将使用编号为 1 的 TopN 节点生成的 Runtime Filter，因此在 Plan 中显示为 TOPN OPT: 1。

作为一个分布式数据库，Doris 还需要考虑 TopN 节点和 Scan 节点实际运行的物理机器。因为跨 BE 通信的代价比较高，所以 BE 会自适应地决定是否使用 TopN Runtime Filter，以及使用的范围。当前，我们实现了 BE 级别的 TopN Runtime Filter，即 TopN 和 Scan 在同一个 BE 里。这是因为 TopN Runtime Filter 阈值的更新只需要线程间通信，代价比较低。

调优

Session 变量 topn_filter_ratio 可以控制是否生成 TopN Runtime Filter。

如果 SQL 中 limit 的数量越少，那么 TopN Runtime Filter 的过滤性就越强。因此，系统默认情况下，只有在 limit 数量小于表中数据的一半时，才会启用生成对应的 TopN Runtime Filter。

例如，如果设置 set topn_filter_ratio=0，那么执行以下查询就不会生成 TopN Runtime Filter。

```
select o_orderkey from orders order by o_orderdate limit 20;
```

2.13.9.4 TOPN 查询优化

TOPN 查询是指下面这种 ORDER BY LIMIT 查询，在日志检索等明细查询场景中很常见，Doris 会自动对这种类型的查询进行优化。

```
SELECT * FROM tablex WHERE xxx ORDER BY c1,c2 ... LIMIT n
```

2.13.9.4.1 TOPN 查询优化的优化点

1. 执行过程中动态对排序列构建范围过滤条件（比如 c1 >= 10000），读数据时自动带上前面的条件，利用 Zonemap 索引过滤掉一些数据甚至文件。
2. 如果排序字段 c1,c2 正好是 Table Key 的前缀，则更进一步优化，读数据的时候只用读数据文件的头部或者尾部 n 行。
3. SELECT * 延迟物化，读数据和排序过程中只读排序列不读其它列，得到符合条件的行号后，再去读那 n 行需要的全部列数据，大幅减少读取和排序的列。

2.13.9.4.2 TOPN 查询优化的限制

1. 只能用于 Duplicate 表和 Unique MOW 表，因为 MOR 表用这个优化可能有结果错误。
2. 对于过大的 n，优化内存消耗会很大，所以超过 topn_opt_limit_threshold Session 变量的 n 不会使用优化。

2.13.9.4.3 配置参数和查询分析

下面两个参数都是 Session Variable，可以针对某个 SQL 或者全局设置。

1. topn_opt_limit_threshold，LIMIT n 小于这个值才会有优化，默认值 1024，将它设置为 0 可以关闭 TOPN 查询优化。
2. enable_two_phase_read_opt，是否开启优化 3，默认为 true，可以调为 false 关闭这个优化。
3. topn_filter_ratio，LIMIT n 和表总数据的比率，默认值 0.5，表示 LIMIT 数量多于表中数据的一半则不生成 filter。

检查 TOPN 查询优化是否启用

explain SQL 拿到 query plan 可以确认这个 sql 是否启用 TOPN 查询优化，以下面的为例：

- TOPN OPT 代表有优化 1
- VOlapScanNode 下面有 SORT LIMIT 代表有优化 2
- OPT TWO PHASE 代表有优化 3

```
1:VTOP-N(137)
|  order by: @timestamp18 DESC
|  TOPN OPT
|  OPT TWO PHASE
|  offset: 0
|  limit: 10
|  distribute expr lists: applicationName5
|
0:VOlapScanNode(106)
   TABLE: log_db.log_core_all_no_index(log_core_all_no_index), PREAGGREGATION: ON
   SORT INFO:
        @timestamp18
   SORT LIMIT: 10
   TOPN OPT:1
   PREDICATES: ZYCFC-TRACE-ID4 like '%flowId-1720055220933%'
   partitions=1/8 (p20240704), tablets=250/250, tabletList=1727094,1727096,1727098 ...
   cardinality=345472780, avgRowSize=0.0, numNodes=1
   pushAggOp=NONE
```

检查 TOPN 查询优化执行时是否有效果

首先，可以将 topn_opt_limit_threshold 设置为 0 关闭 TOPN 查询优化，对比开启和关闭优化的 SQL 执行时间。

开启 TOPN 查询优化后，在 Query Profile 中搜索 RuntimePredicate，关注下面几个指标：

- RowsZonemapRuntimePredicateFiltered 这个代表过滤掉的行数，越大越好
- NumSegmentFiltered 这个代表过滤掉的数据文件个数，越大越好

- BlockConditionsFilteredZonemapRuntimePredicateTime 代表过滤数据的耗时，越小越好

注意，2.0.3 之前的版本中 RuntimePredicate 的指标未独立，可以通过 Zonamap 指标大致观察。

```
SegmentIterator:
       - BitmapIndexFilterTimer:  46.54us
       - BlockConditionsFilteredBloomFilterTime:  10.352us
       - BlockConditionsFilteredDictTime:  7.299us
       - BlockConditionsFilteredTime:  202.23ms
       - BlockConditionsFilteredZonemapRuntimePredicateTime:  0ns
       - BlockConditionsFilteredZonemapTime:  402.917ms
       - BlockInitSeekCount:  399
       - BlockInitSeekTime:  11.309ms
       - BlockInitTime:  215.59ms
       - BlockLoadTime:  7s567ms
       - BlocksLoad:  392.97K  (392970)
       - CachedPagesNum:  0
       - CollectIteratorMergeTime:  0ns
       - CollectIteratorNormalTime:  0ns
       - CompressedBytesRead:  29.76  MB
       - DecompressorTimer:  427.713ms
       - ExprFilterEvalTime:  3s930ms
       - FirstReadSeekCount:  392.921K  (392921)
       - FirstReadSeekTime:  528.287ms
       - FirstReadTime:  1s134ms
       - IOTimer:  51.286ms
       - InvertedIndexFilterTime:  49.457us
       - InvertedIndexQueryBitmapCopyTime:  0ns
       - InvertedIndexQueryBitmapOpTime:  0ns
       - InvertedIndexQueryCacheHit:  0
       - InvertedIndexQueryCacheMiss:  0
       - InvertedIndexQueryTime:  0ns
       - InvertedIndexSearcherOpenTime:  0ns
       - InvertedIndexSearcherSearchTime:  0ns
       - LazyReadSeekCount:  0
       - LazyReadSeekTime:  0ns
       - LazyReadTime:  106.952us
       - NumSegmentFiltered:  0
       - NumSegmentTotal:  50
       - OutputColumnTime:  61.987ms
       - OutputIndexResultColumnTimer:  12.345ms
       - RawRowsRead:  3.929151M  (3929151)
       - RowsBitmapIndexFiltered:  0
       - RowsBloomFilterFiltered:  0
       - RowsConditionsFiltered:  6.38976M  (6389760)
       - RowsDictFiltered:  0
```

```
        - RowsInvertedIndexFiltered:  0
        - RowsKeyRangeFiltered:  0
        - RowsShortCircuitPredFiltered:  0
        - RowsShortCircuitPredInput:  0
        - RowsStatsFiltered:  6.38976M  (6389760)
        - RowsVectorPredFiltered:  0
        - RowsVectorPredInput:  0
        - RowsZonemapRuntimePredicateFiltered:  6.38976M  (6389760)
        - SecondReadTime:  0ns
        - ShortPredEvalTime:  0ns
        - TotalPagesNum:  2.301K  (2301)
        - UncompressedBytesRead:  137.99  MB
        - VectorPredEvalTime:  0ns
```

2.13.9.5 统计信息

从 2.0 版本开始，Doris 在优化器中加入了 CBO 的能力。统计信息是 CBO 的基石，其准确性直接决定了代价估算的准确性，对于选择最优 Plan 至关重要。本文主要介绍统计信息的收集和管理方法、相关配置项以及常见问题。

2.13.9.5.1 统计信息的收集

Doris 默认会开启内表的自动抽样收集，因此绝大多数情况下用户不用关注统计信息的收集。Doris 收集统计信息的对象是列，它会在表级别收集每一列的统计信息，收集的内容包括：

信息	描述
row_count	总行数
data_size	列的总数据量
avg_size_byte	列的平均每行数据量
ndv	不同值数量
min	最小值
max	最大值
null_count	空值数量

目前，系统仅支持收集基本类型列的统计信息，包括 BOOLEAN、 TINYINT、 SMALLINT、 INT、 BIGINT、 LARGEINT、 FLOAT、 DOUBLE、 DATE、 DATETIME、 STRING、 VARCHAR、 TEXT 等。

复杂类型的列会被跳过，包括 JSONB、 VARIANT、 MAP、 STRUCT、 ARRAY、 HLL、 BITMAP、 TIME、 TIMEV2 等。

统计信息的收集方式有手动和自动两种，收集的结果会保存在 `internal.__internal_schema.column_statistics` 表中。下面将详细介绍这两种收集方式。

手动收集

Doris 支持用户通过提交 ANALYZE 语句来手动触发统计信息的收集和更新。

1. 语法

具体可参阅 SQL 手册ANALYZE。

2. 示例

对 lineitem 表的所有列进行全量收集：

```
ANALYZE TABLE lineitem;
```

对 tpch100 数据库中所有表的所有列进行全量收集：

```
ANALYZE DATABASE tpch100;
```

对 lineitem 表的所有列按照 10% 的比例进行抽样收集（注意这里应更改为 PERCENT 以符合语法说明）：

```
ANALYZE TABLE lineitem WITH SAMPLE ROWS 100000;
```

对 lineitem 表的 l_orderkey 和 l_linenumber 列按照采样 100000 行进行收集：

```
ANALYZE TABLE lineitem (l_orderkey, l_linenumber) WITH SAMPLE ROWS 100000;
```

自动收集

自动收集功能自 2.0.3 版本起开始支持，且默认全天开启。用户可以通过设置 ENABLE_AUTO_ANALYZE 变量来控制该功能的启用或停用：

```
SET GLOBAL ENABLE_AUTO_ANALYZE = TRUE; // 打开自动收集
SET GLOBAL ENABLE_AUTO_ANALYZE = FALSE; // 关闭自动收集
```

在启用状态下，后台线程会定期扫描集群中 InternalCatalog 下的所有库表。对于需要收集统计信息的表，系统会自动创建并执行收集作业，无需用户手动干预。

需要注意的是，为避免自动收集大宽表造成过多资源占用，默认不收集宽度超过 300 列的表。用户可以通过修改 Session 变量 auto_analyze_table_width_threshold 的值来调整这一宽度上限，例如将其设置为 350：

```
SET GLOBAL auto_analyze_table_width_threshold = 350;
```

自动收集的默认轮询间隔为 5 分钟（此间隔可通过 fe.conf 中的 auto_check_statistics_in_minutes 配置项进行调整）。默认情况下，集群启动 5 分钟后开始第一轮遍历。当所有需要收集的表完成收集后，后台线程会休眠 5 分钟，然后开启第二轮遍历，以此类推。因此不能保证一张表在 5 分钟内一定能收集到统计信息，因为遍历一轮库表的时间是不确定的。在表较多且数据量较大的情况下，遍历一轮的时间可能会较长。

当轮询到一张表时，系统会首先判断该表是否需要收集统计信息。如果需要，则创建收集作业并开始收集；否则，跳过该表并继续轮询下一张。以下任意条件满足时，表明该表需要重新收集统计信息：

1. 表中存在无统计信息的列；
2. 表的健康度低于阈值（默认为 90，可通过 table_stats_health_threshold 变量进行调整）。健康度表示从上次收集统计信息到当前时刻，表中数据保持不变的比例：100 表示完全没有变化；0 表示全部改变；当健康度低于 90 时，表示当前的统计信息已有较大偏差，需要重新收集。通过健康度评估，可以降低不必要的重复收集，从而节省系统资源。
3. 对于内表，数据发生过变化，但在 24 小时之内没有收集过统计信息。

为了降低后台作业的开销并提高收集速度，自动收集采用采样收集方式，默认采样 4194304，即 2^22 行。如果用户希望采样更多行以获得更准确的数据分布信息，可通过调整参数 huge_table_default_sample_rows 来增加采样行数。

如果担心自动收集作业会对业务造成干扰，可根据自身需求通过设置参数 auto_analyze_start_time 和 auto_analyze_end_time 来指定自动收集作业在业务负载较低的时间段内执行。此外，也可以通过将参数 enable_auto_analyze 设置为 false 来完全停用此功能。

```
SET GLOBAL auto_analyze_start_time = "03:00:00"; // 把起始时间设置为凌晨3点
SET GLOBAL auto_analyze_end_time = "14:00:00"; // 把终止时间设置为下午2点
```

外表收集

外表通常为 Hive、Iceberg、JDBC 等类型的表。

- 在手动收集方面，Hive, Iceberg 和 JDBC 表均支持手动收集统计信息。其中，Hive 表支持手动进行全量和采样收集，而 Iceberg 和 JDBC 表则仅支持手动全量收集。其他类型的外表则不支持手动收集统计信息。
- 在自动收集方面，当前仅 Hive 表提供支持。

需要注意的是，外部 Catalog 默认情况下不参与自动收集列统计信息，只收集表的行数信息。这是因为外部 Catalog 通常包含大量历史数据，如果全部自动收集列统计信息，可能会占用过多资源。在确有需求的情况下，用户可以通过设置 Catalog 的属性来打开外部 Catalog 的自动收集列统计信息功能。

```
ALTER CATALOG <catalog_name> SET PROPERTIES ('enable.auto.analyze'='true'); //
    ↪ 打开自动收集列统计信息
ALTER CATALOG <catalog_name> SET PROPERTIES ('enable.auto.analyze'='false'); //
    ↪ 关闭自动收集列统计信息
```

如果控制整个 Catalog 的粒度太大，我们还支持在表级别打开和关闭自动列统计信息收集。

```
ALTER TABLE <table_name> SET ("auto_analyze_policy" = "enable"); //
    ↪ 打开这张表自动收集列统计信息功能（优先级高于 Catalog 的 enable.auto.analyze 属性）
ALTER TABLE <table_name> SET ("auto_analyze_policy" = "disable"); //
    ↪ 关闭这张表自动收集列统计信息功能（优先级高于 Catalog 的 enable.auto.analyze 属性）
ALTER TABLE <table_name> SET ("auto_analyze_policy" = "base_on_catalog"); // 由 Catalog 的 enable
    ↪ .auto.analyze 属性来决定这张表是否自动收集列统计信息
```

外表没有健康度的概念。在启用了 Catalog 或 Table 的自动收集列统计信息功能后，为了避免频繁收集，对于一张外表，系统默认在 24 小时之内只对其进行一次自动收集。你可以通过 external_table_auto_analyze_
↪ interval_in_millis 变量来控制外表的最小收集时间间隔。

在默认状态下，外表不会收集列统计信息，只收集行数信息。不同外表收集行数信息的方法如下：

1. 对于 Hive 表

系统首先尝试从 Hive 表的 Parameters 中获取 numRows 或 totalSize 的信息：

- 如果找到 numRows，则将其值作为表的行数。
- 如果没有找到 numRows，但找到了 totalSize 信息，则根据表的 Schema 和 totalSize 来估算表的行数。

- 如果 totalSize 也没有，默认情况下，系统会根据 Hive 表对应的文件大小和 Schema 来估算行数。如果担心获取文件大小占用过多资源，可以通过设置以下变量来关闭这一功能：

```
SET GLOBAL enable_get_row_count_from_file_list = FALSE
```

2. 对于 Iceberg 表

系统会调用 Iceberg 的 snapshot API 来获取 total-records 和 total-position-deletes 信息，以计算表的行数。

3. 对于 Paimon 表

系统会调用 Paimon 的 scan API 来获取每个 Split 包含的行数，并对 Split 行数求和来计算表的行数。

4. 对于 JDBC 表

系统会调用 JDBC 后端对应数据库的行数获取语句来获取表的行数。只有在后端数据库收集了表的行数信息的情况下，才可以获取到。当前支持获取 MySQL, Oracle, Postgresql 和 SQLServer 表的行数。

5. 对于其他外表

系统目前不支持行数的自动获取和估算。

用户可以通过以下命令来查看外表估算的行数（见查看表信息概况章节）：

```
SHOW table stats table_name;
```

- 如果 row_count 显示为 -1，则表示未能获取到行数信息或者表为空。

2.13.9.5.2 统计信息作业管理

查看统计作业

通过 SHOW ANALYZE 来查看统计信息收集作业的信息。目前，系统仅保留 20000 个历史作业的信息。请注意，仅异步作业的信息可通过该命令查看，同步作业（使用 WITH SYNC）不保留历史作业信息。

1. 语法：

具体可参阅 SQL 手册SHOW ANALYZE

2. 输出结果

包含以下列：

列名	说明
job_id	统计作业 ID
catalog_name	Catalog 名称
db_name	数据库名称
tbl_name	表名称
col_name	列名称列表（index_name:column_name）
job_type	作业类型
analysis_type	统计类型
message	作业信息
state	作业状态

列名	说明
progress	作业进度
schedule_type	调度方式
start_time	作业开始时间
end_time	作业结束时间

3. 示例：

```
mysql show analyze 245073`\G`;
*************************** 1. row ***************************
              job_id: 93021
        catalog_name: internal
             db_name: tpch
            tbl_name: region
            col_name: [region:r_regionkey,region:r_comment,region:r_name]
            job_type: MANUAL
       analysis_type: FUNDAMENTALS
             message:
               state: FINISHED
            progress: 3 Finished  |  0 Failed  |  0 In Progress  |  3 Total
       schedule_type: ONCE
          start_time: 2024-07-11 15:15:00
            end_time: 2024-07-11 15:15:33
```

查看统计任务

每个收集作业可包含一到多个任务，且每个任务对应一列的收集。用户可通过以下命令查看具体每列的统计信息收集完成情况。

1. 语法

```
SHOW ANALYZE TASK STATUS [job_id]
```

2. 示例

```
mysql> show analyze task status 93021;
+--
    ↪ -------+-------------+------------+---------+-----------------------+-----------------+----------+
    ↪
| task_id | col_name    | index_name | message | last_state_change_time | time_cost_in_ms | state
    ↪     |
+--
    ↪ -------+-------------+------------+---------+-----------------------+-----------------+----------+
    ↪
| 93022   | r_regionkey | region     |         | 2024-07-11 15:15:33    | 32883           |
    ↪ FINISHED |
```

```
| 93023   | r_comment   | region     |         | 2024-07-11 15:15:33    | 32883          |
    ↪ FINISHED |
| 93024   | r_name      | region     |         | 2024-07-11 15:15:33    | 32883          |
    ↪ FINISHED |
+--
    ↪ -------+-------------+------------+---------+------------------------+-----------------+----------+
    ↪
```

查看统计信息

用户可以通过 SHOW COLUMN STATS 命令来查看已经收集的列统计信息。

1. 语法

```
SHOW COLUMN [cached] STATS table_name [ (column_name [, ...]) ];
```

其中：

- cached: 展示当前 FE 内存缓存中的统计信息。
- table_name: 收集统计信息的目标表，可以是 db_name.table_name 形式。
- column_name: 指定的目标列，必须是 table_name 中存在的列，多个列名称用逗号分隔。如不指定，则展示所有列的信息。

2. 示例

```
mysql> show column stats region (r_regionkey)`\G`
*************************** 1. row ***************************
  column_name: r_regionkey
   index_name: region
        count: 5.0
          ndv: 5.0
     num_null: 0.0
    data_size: 20.0
avg_size_byte: 4.0
          min: 0
          max: 4
       method: FULL
         type: FUNDAMENTALS
      trigger: MANUAL
  query_times: 0
 updated_time: 2024-07-11 15:15:33
1 row in set (0.36 sec)
```

查看表信息概况

通过 SHOW TABLE STATS 查看表的统计信息收集概况。

1. 语法

```
SHOW TABLE STATS table_name;
```

其中：table_name: 目标表表名。可以是 db_name.table_name 形式。

2. 输出结果

包含以下列：

列名	说明
updated_rows	自上次 ANALYZE 以来该表的更新行数
query_times	保留列，用于在后续版本中记录该表的查询次数
row_count	表的行数（可能不反映命令执行时的准确行数）
updated_time	上次统计信息的更新时间
columns	已收集统计信息的列
trigger	统计信息触发的方式
new_partition	是否有新分区首次导入了数据
user_inject	用户是否手动注入了统计信息

3. 示例

```
mysql> show column stats region (r_regionkey)`\G`
*************************** 1. row ***************************
  column_name: r_regionkey
   index_name: region
        count: 5.0
          ndv: 5.0
     num_null: 0.0
    data_size: 20.0
avg_size_byte: 4.0
          min: 0
          max: 4
       method: FULL
         type: FUNDAMENTALS
      trigger: MANUAL
  query_times: 0
 updated_time: 2024-07-11 15:15:33
1 row in set (0.36 sec)
```

终止统计作业

通过 KILL ANALYZE 来终止当前正在运行的异步统计作业。

1. 语法

```
KILL ANALYZE job_id;
```

其中：job_id：表示统计信息作业的 ID。这是执行 ANALYZE 异步收集统计信息时返回的值，也可以通过 SHOW ANALYZE 语句获取。

2. 示例

终止 ID 为 52357 的统计作业。

```
mysql> KILL ANALYZE 52357;
```

删除统计信息

如果某个 Catalog、 Database 或 Table 被删除，用户无需手动删除其统计信息，因为后台会定期清理这些信息。

然而对于仍然存在的表，系统不会自动清除其统计信息。此时需要用户手动进行删除操作，语法如下：

```
DROP STATS table_name
```

2.13.9.5.3 会话变量及配置项

会话变量

会话变量	说明	默认值
auto_analyze_start_time	自动统计信息收集的开始时间	0:00:00
auto_analyze_end_time	自动统计信息收集的结束时间	23:59:59
enable_auto_analyze	是否开启自动收集功能	TRUE
huge_table_default_sample_rows	对大表进行采样时的行数	4194304
table_stats_health_threshold	取值范围 0-100，表示自上次统计信息收集后，数据更新达到 (100 - table_stats_health_threshold)% 时，认为统计信息已过时	90
auto_analyze_table_width_threshold	控制自动统计信息收集处理的最大表宽度，超过此列数的表不参与自动统计信息收集	300
enable_get_row_count_from_file_list	Hive 表是否通过文件大小来估算行数	TRUE（2.1.5 之前默认为 FALSE）

FE 配置项

> 备注以下 FE 配置项在通常情况下无需特别关注

FE 配置项	说明	默认值
analyze_record_limit	控制统计信息作业执行记录的持久化行数	20000
stats_cache_size	FE 侧统计信息缓存的条数	500000
statistics_simultaneously_running_task_num	可同时执行的异步统计作业数量	3
statistics_sql_mem_limit_in_bytes	控制每个统计信息 SQL 可占用的 BE 内存大小	2L * 1024 * 1024 (2GiB)

FE 配置项	说明	默认值

2.13.9.5.4 常见 FAQ

Q1：如何查看一张表是否收集了统计信息以及内容是否正确？

首先，执行 `show column stats table_name` 查看是否有统计信息输出。

其次，执行 `show column cached stats table_name` 查看缓存中是否加载了该表的统计信息。

```
mysql> show column stats test_table`\G`
Empty set (0.02 sec)

mysql> show column cached stats test_table`\G`
Empty set (0.00 sec)
```

上图显示结果为空，说明 `test_table` 表目前没有统计信息。如果有统计信息，结果将类似以下内容：

```
mysql> show column cached stats mvTestDup;
+--
    ↪ -----------+------------+-------+------+----------+-----------+---------------+------+------+--------+---
    ↪
| column_name | index_name | count | ndv  | num_null | data_size | avg_size_byte | min  | max  |
    ↪ method | type         | trigger | query_times | updated_time        |
+--
    ↪ -----------+------------+-------+------+----------+-----------+---------------+------+------+--------+---
    ↪
| key1        | mvTestDup  | 6.0   | 4.0  | 0.0      | 48.0      | 8.0           | 1    | 1001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
| key2        | mvTestDup  | 6.0   | 4.0  | 0.0      | 48.0      | 8.0           | 2    | 2001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
| value2      | mvTestDup  | 6.0   | 4.0  | 0.0      | 24.0      | 4.0           | 4    | 4001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
| value1      | mvTestDup  | 6.0   | 4.0  | 0.0      | 24.0      | 4.0           | 3    | 3001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
| mv_key1     | mv1        | 6.0   | 4.0  | 0.0      | 48.0      | 8.0           | 1    | 1001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
| value3      | mvTestDup  | 6.0   | 4.0  | 0.0      | 24.0      | 4.0           | 5    | 5001 |
    ↪ FULL   | FUNDAMENTALS | MANUAL  | 0           | 2024-07-22 10:53:25 |
+--
    ↪ -----------+------------+-------+------+----------+-----------+---------------+------+------+--------+---
    ↪
6 rows in set (0.00 sec)
```

在有统计信息的情况下，可以通过手动执行 SQL 来验证统计信息的准确性。

```
Select count(1), ndv(col1), min(col1), max(col1) from table
```

如果 count 和 ndv 的误差在一个数量级以内，那么准确度基本可以接受。

Q2：为什么一张表一直没有自动收集统计信息？

首先，查看自动收集功能是否打开：

```
Show variables like "enable_auto_analyze"  // 如果是 false，需要设置为 true：
Set global enable_auto_analyze = true
```

如果已经是 true，再确认一下表的列数。如果超过auto_analyze_table_width_threshold的值，则这个表不会参与自动收集。此时，需要修改这个值，使其大于当前表的列数：

```
Show variables like "auto_analyze_table_width_threshold"  // 如果 Value 小于表的宽度，可以修改：
Set global auto_analyze_table_width_threshold=350
```

如果列数没有超过阈值，可以执行show auto analyze，检查是否有其他收集任务正在执行（处于 running 状态）。由于自动收集是单线程串行执行，会轮询所有库表，因此执行周期可能较长。

Q3：为什么部分列没有统计信息？

目前，系统仅支持收集基本类型列的统计信息。对于复杂类型的列，如 JSONB、VARIANT、MAP、STRUCT、ARRAY、HLL、BITMAP、TIME 以及 TIMEV2 等系统会选择跳过。

Q4：报错 “Stats table not available, please make sure your cluster status is normal”

出现这种报错通常意味着内部统计信息表处于不健康状态。

首先，需要检查集群中所有的 BE（Backend）是否都处于正常状态，确保所有 BE 都在正常工作。

其次，执行以下语句，以获取到所有的 tabletId（输出结果的第一列）。

```
show tablets from internal.__internal_schema.column_statistics;
```

接着，通过 tablet_id 逐一查看每个 tablet 是否正常：

```
ADMIN DIAGNOSE TABLET tablet_id
```

如果发现有不正常的 tablet，需要先进行修复，再重新收集统计信息。

Q5：如何解决统计信息收集不及时问题？

自动收集的时间间隔具有不确定性，它与系统中表的数量及表的大小均有关联。若情况紧急，建议对表进行手动 analyze 操作。

若在导入大量数据后仍未触发自动收集，可能需要调整 table_stats_health_threshold 参数。其默认值为 90，意味着表的数据变化量需超过 10%（即 100 - 90）才会触发自动收集。可适当提高此值，例如设为 95，这样当表中数据变化量超过 5% 时，便会重新收集统计信息。

Q6：自动收集时资源占用太多，该如何解决？

自动收集采用采样方式，无需全量扫描表数据，且自动收集任务以单线程串行执行，通常系统资源占用可控，不会对正常查询任务造成影响。

对于某些特殊表，如分区众多的表或单个 Tablet 体积庞大的表，可能会出现内存占用较多的情况。

建议用户在建表时合理规划 Tablet 数量，避免产生超大 Tablet。若 Tablet 结构不易调整，建议在系统低峰期开启自动收集，或于低峰期手动收集这些大表，以免在高峰期影响业务运行。在 Doris 3.x 系列中，我们将针对此类场景进行优化。

2.14 湖仓一体

2.14.1 湖仓一体概述

湖仓一体是将数据湖和数据仓库的优势相结合的现代化大数据解决方案。其融合了数据湖的低成本、高扩展性与数据仓库的高性能、强数据治理能力，从而实现对大数据时代各类数据的高效、安全、质量可控的存储和处理分析。同时通过标准化的数据格式和元数据管理，统一了实时、历史数据，批处理和流处理，正在逐步成为企业大数据解决方案新的标准。

2.14.1.1 Doris 湖仓一体解决方案

Doris 通过可扩展的连接器框架、存算分离架构、高性能的数据处理引擎和数据生态开放性，为用户提供了优秀的湖仓一体解决方案。

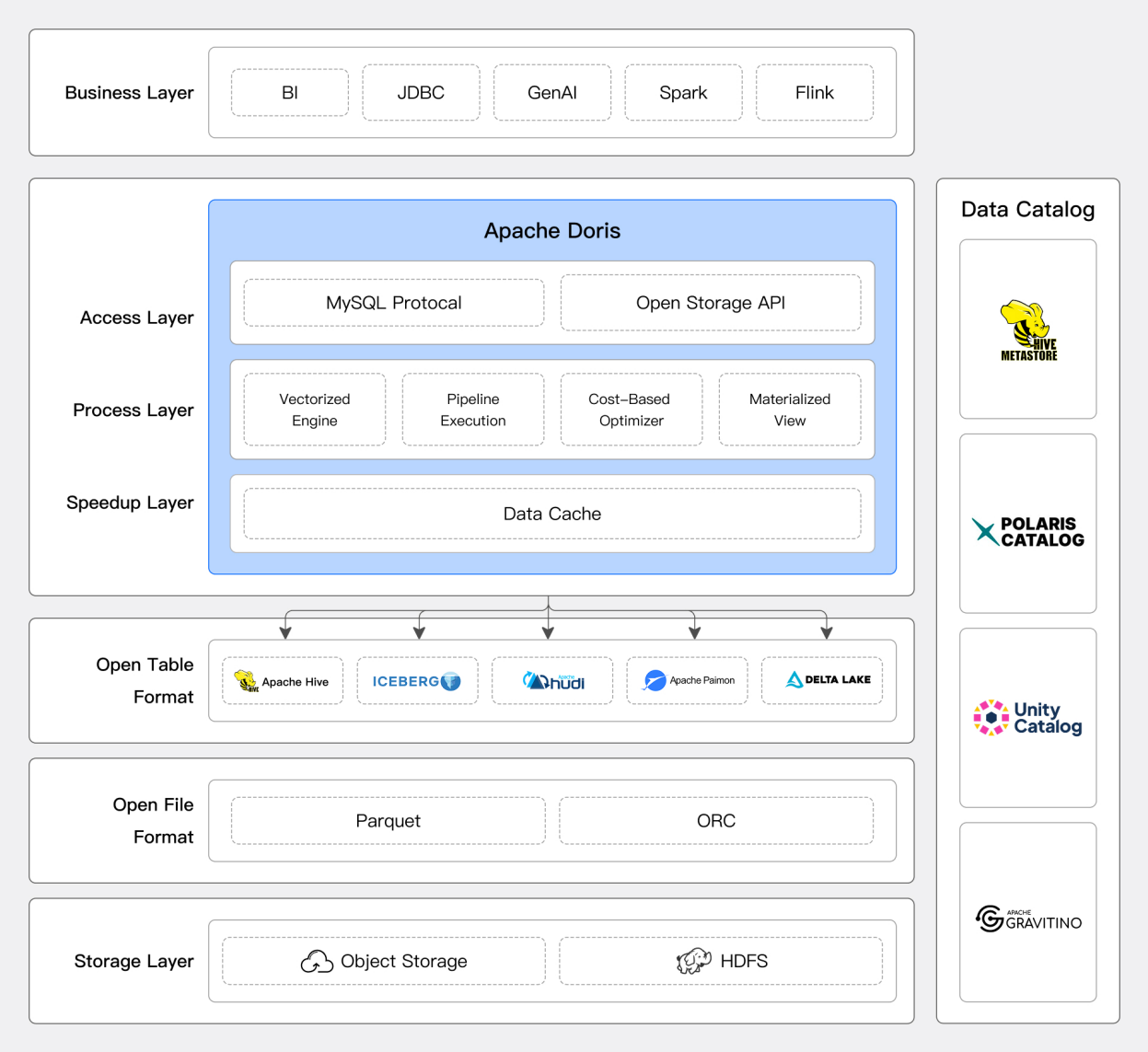


图 67: doris-lakehouse-arch

2.14.1.1.1 灵活的数据接入

Doris 通过可扩展的连接器框架，支持主流数据系统和数据格式接入，并提供基于 SQL 的统一数据分析能力，用户能够在不移动现有数据的情况下，轻松实现跨平台的数据查询与分析。具体可参阅数据目录概述

2.14.1.1.2 数据源连接器

无论是 Hive、Iceberg、Hudi、Paimon，还是支持 JDBC 协议的数据库系统，Doris 均能轻松连接并高效访问数据。

对于湖仓系统，Doris 可从元数据服务，如 Hive Metastore，AWS Glue、Unity Catalog 中获取数据表的结构和分布信息，进行合理的查询规划，并利用 MPP 架构进行分布式计算。

具体可参阅各数据目录文档，如 Iceberg Catalog

可扩展的连接器框架

Doris 提供良好的扩展性框架，帮助开发人员快速对接企业内部特有的数据源，实现数据快速互通。

Doris 定义了标准的数据目录（Catalog）、数据库（Database）、数据表（Table）三个层级，开发人员可以方便的映射到所需对接的数据源层级。Doris 同时提供标准的元数据服务和数据读取服务的接口，开发人员只需按照接口定义实现对应的访问逻辑，即可完成数据源的对接。

Doris 兼容 Trino Connector 插件，可直接将 Trino 插件包部署到 Doris 集群，经过少量配置即可访问对应的数据源。Doris 目前已经完成了 Kudu、 BigQuery、 Delta Lake 等数据源的对接。也可以 自行适配新的插件。

便捷的跨源数据处理

Doris 支持在运行时直接创建多个数据源连接器，并使用 SQL 对这些数据源进行联邦查询。比如用户可以将 Hive 中的事实表数据与 MySQL 中的维度表数据进行关联查询：

```
SELECT h.id, m.name
FROM hive.db.hive_table h JOIN mysql.db.mysql_table m
ON h.id = m.id;
```

结合 Doris 内置的作业调度 能力，还可以创建定时任务，进一步简化系统复杂度。比如用户可以将上述查询的结果，设定为每小时执行一次的例行任务，并将每次的结果，写入一张 Iceberg 表：

```
CREATE JOB schedule_load
ON SCHEDULE EVERY 1 HOUR DO
INSERT INTO iceberg.db.ice_table
SELECT h.id, m.name
FROM hive.db.hive_table h JOIN mysql.db.mysql_table m
ON h.id = m.id;
```

2.14.1.1.3 高性能的数据处理

Doris 作为分析型数据仓库，在湖仓数据处理和计算方面做了大量优化，并提供了丰富的查询加速功能：

- 执行引擎

  Doris 执行引擎基于 MPP 执行框架和 Pipeline 数据处理模型，能够很好的在多机多核的分布式环境下快速处理海量数据。同时，得益于完全的向量化执行算子，在计算性能方面，Doris 在 TPC-DS 等标准评测数据集中处于领先地位。

- 查询优化器

  Doris 能通过查询优化器自动优化和处理复杂的 SQL 请求。查询优化器针对多表关联、聚合、排序、分页等多种复杂 SQL 算子进行了深度优化，充分利用代价模型和关系代数变化，自动获取较优或最优的逻辑执行计划和物理执行计划，极大降低用户编写 SQL 的难度，提升易用性和性能。

- 缓存加速与 IO 优化

  外部数据源的访问，通常是网络访问，因此存在延迟高、稳定性差等问题。Apache Doris 提供了丰富的缓存机制，并在缓存的类型、时效性、策略方面都做了大量的优化，充分利用内存和本地高速磁盘，提升热点数据的分析性能。同时，针对网络 IO 高吞吐、低 IOPS、高延迟的特性，Doris 也进行了针对性的优化，可以提供媲美本地数据的外部数据源访问性能。

- 物化视图与透明加速

  Doris 提供丰富的物化视图更新策略，支持全量和分区级别的增量刷新，以降低构建成本并提升时效性。除手动刷新外，Doris 还支持定时刷新和数据驱动刷新，进一步降低维护成本并提高数据一致性。物化视图还具备透明加速功能，查询优化器能够自动路由到合适的物化视图，实现无缝查询加速。此外，Doris 的物化视图采用高性能存储格式，通过列存、压缩和智能索引技术，提供高效的数据访问能力，能够作为数据缓存的替代方案，提升查询效率。

如下所示，在基于 Iceberg 表格式的 1TB 的 TPCDS 标准测试集上，Doris 执行 99 个查询的总体运行仅为 Trino 的 1/3。

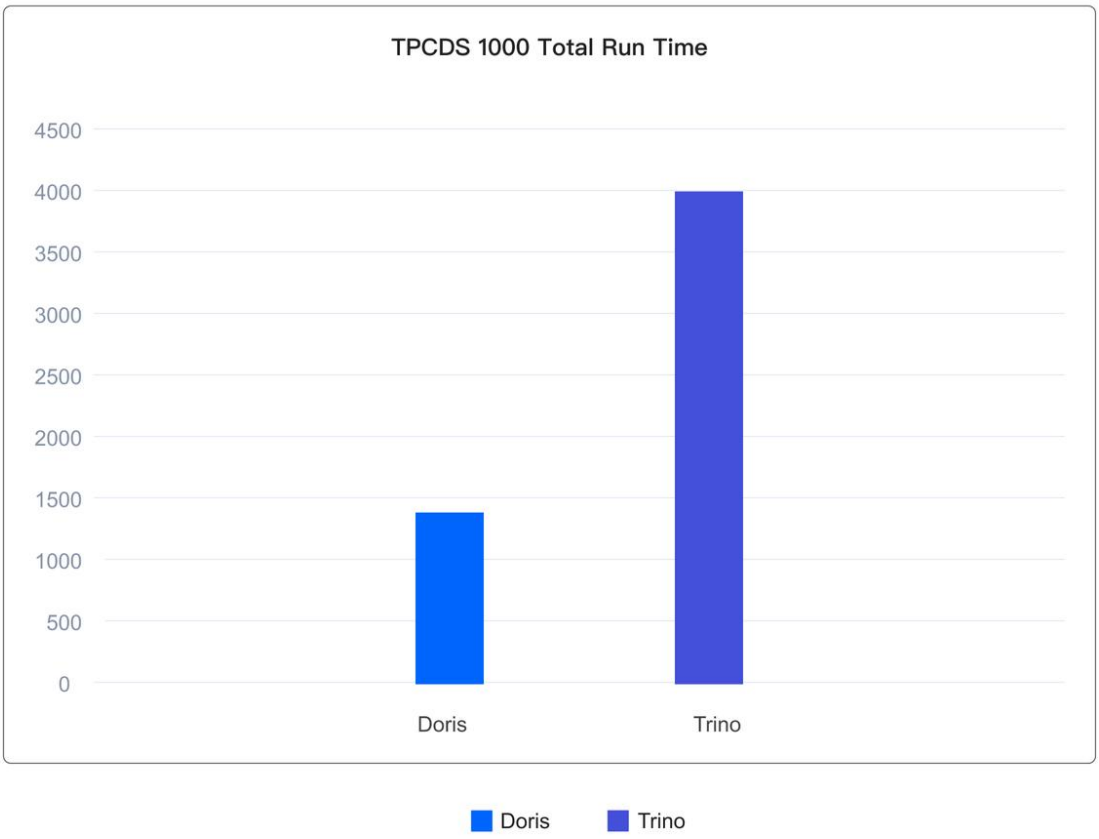


图 68: doris-tpcds

实际用户场景中，Doris 在使用一半资源的情况下，相比 Presto 平均查询延迟降低了 20%，95 分位延迟更是降低 50%。在提升用户体验的同时，极大降低了资源成本。

图 69: doris-performance

2.14.1.1.4 便捷的业务迁移

在企业整合多个数据源并实现湖仓一体转型的过程中，迁移业务的 SQL 查询到 Doris 是一项挑战，因为不同系统的 SQL 方言在语法和函数支持上存在差异。若没有合适的迁移方案，业务侧可能需要进行大量改造以适应新系统的 SQL 语法。

为了解决这个问题，Doris 提供了 SQL 方言转换服务，允许用户直接使用其他系统的 SQL 方言进行数据查询。转换服务会将这些 SQL 方言转换为 Doris SQL，极大降低了用户的迁移成本。目前，Doris 支持 Presto/Trino、Hive、PostgreSQL 和 Clickhouse 等常见查询引擎的 SQL 方言转换，在某些实际用户场景中，兼容率可达到 99% 以上。

2.14.1.1.5 现代化的部署架构

自 3.0 版本以来，Doris 支持面向云原生的存算分离架构。这一架构凭借低成本和高弹性的特点，能够有效提高资源利用率，实现计算和存储的独立扩展。

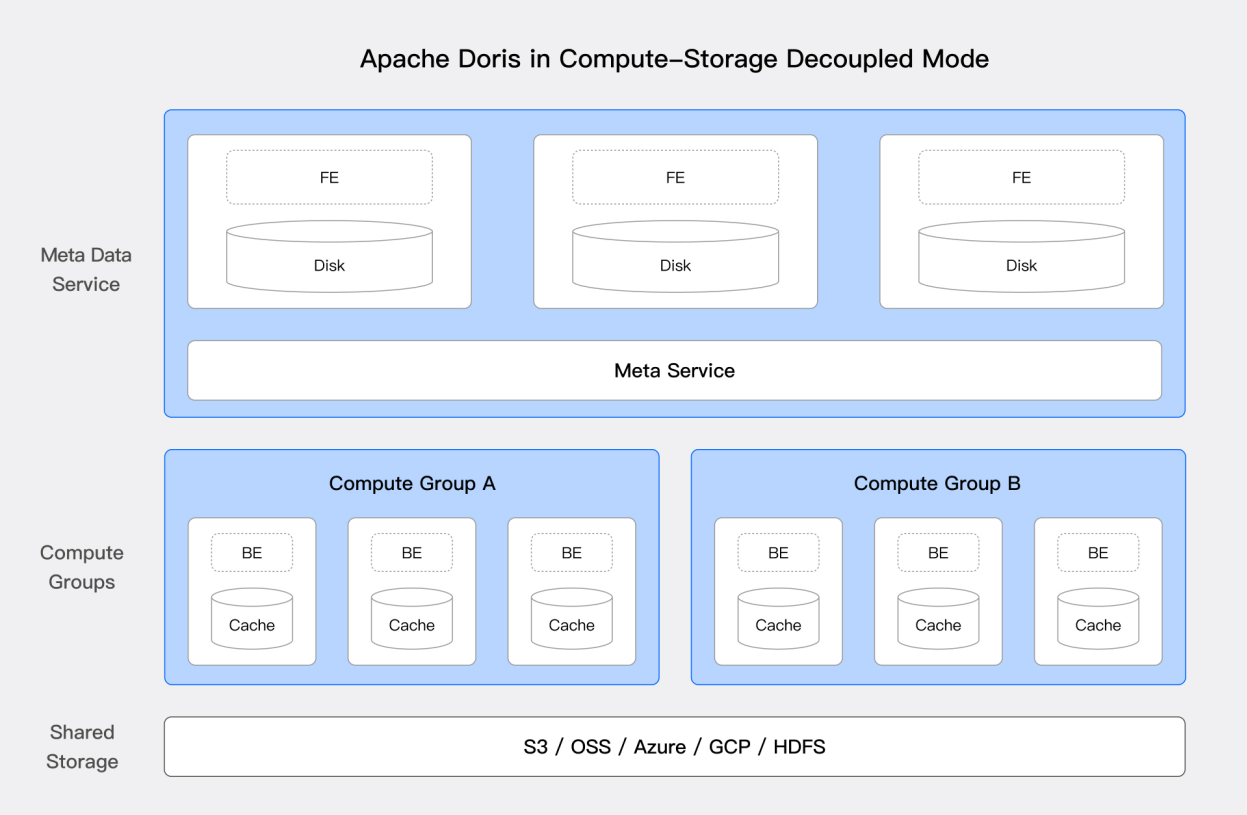


图 70: compute-storage-decouple

上图是 Doris 存算分离的系统架构，对计算与存储进行了解耦，计算节点不再存储主数据，底层共享存储层（HDFS 与对象存储）作为统一的数据主存储空间，并支持计算资源和存储资源独立扩缩容。存算分离架构为湖仓一体解决方案带来了显著的优势：

- 低成本存储：储和计算资源可独立扩展，企业可以根据需要增加存储容量而不必增加计算资源。同时，通过使用云上的对象存储，企业可以享受更低的存储成本和更高的可用性，对于比例相对较低的热点数据，依然可以使用本地高速磁盘进行缓存。
- 唯一可信来源：有数据都存储在统一的存储层中，同一份数据供不同的计算集群访问和处理，确保数据的一致性和完整性，也减少数据同步和重复存储的复杂性。
- 负载多样性：以根据不同的工作负载需求动态调配计算资源，支持批处理、实时分析和机器学习等多种应用场景。通过分离存储和计算，企业可以更灵活地优化资源使用，确保在不同负载下的高效运行。

此外，在存算一体架构下，依然可以通过弹性计算节点在湖仓数据查询场景提供弹性计算能力。

2.14.1.1.6 开放性

Doris 不仅支持开放湖表格式的访问，其自身存储的数据同样拥有良好的开放性。Doris 提供了开放存储 API，并基于 Arrow Flight SQL 协议实现了高速数据链路，具备 Arrow Flight 的速度优势以及 JDBC/ODBC 的易用性。基于该接口，用户可以使用 Python/Java/Spark/Flink 的 ABDC 客户端访问 Doris 中存储的数据。

与开放文件格式相比，开放存储 API 屏蔽了底层的文件格式的具体实现，Doris 可以通过自身存储格式中的高级特性，如丰富的索引机制来加速数据访问。同时，上层的计算引擎无需对底层存储格式的变更或新特性进行适配，所有支持的该协议的计算引擎都可以同步享受到新特性带来的收益。

2.14.1.2 湖仓一体最佳实践

Doris 在湖仓一体方案中，主要用于 湖仓查询加速、多源联邦分析和 湖仓数据处理。

2.14.1.2.1 湖仓查询加速

在该场景中，Doris 作为 计算引擎，对湖仓中数据进行查询分析加速。

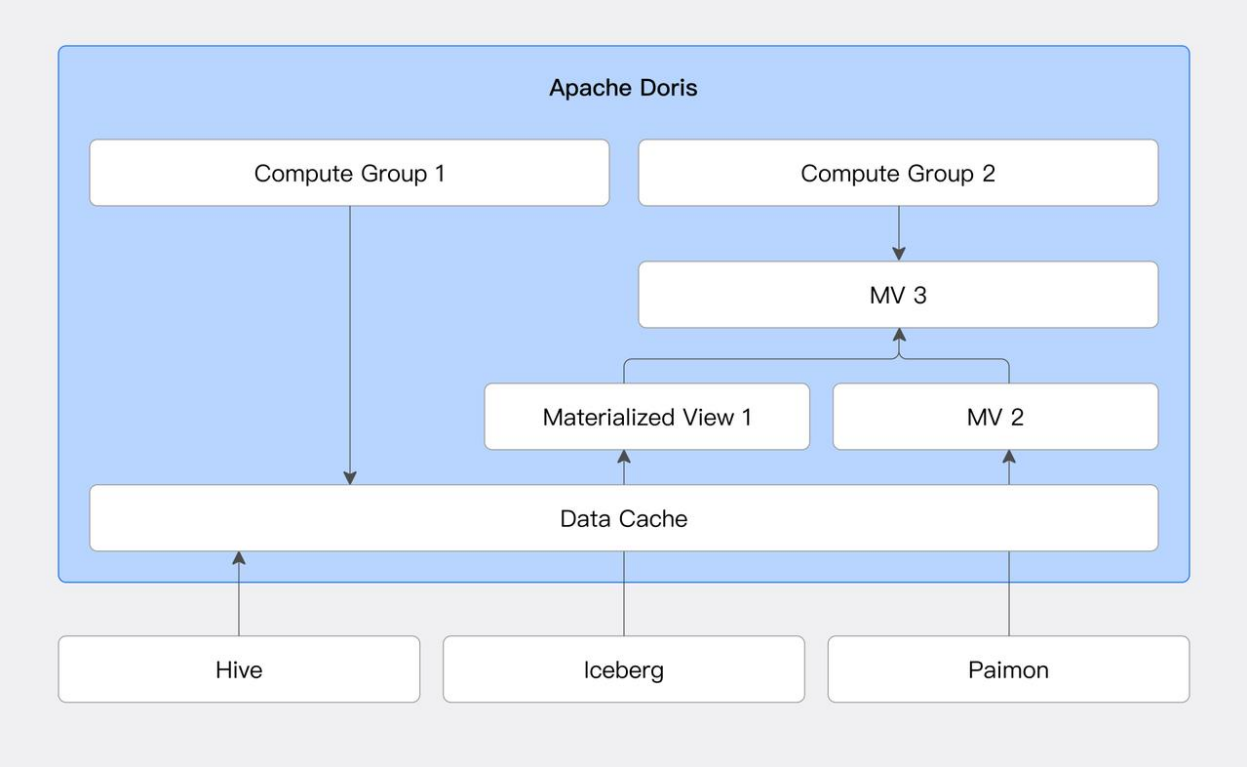


图 71: query-acceleration

缓存加速

针对 Hive、Iceberg 等湖仓系统，用户可以配置本地磁盘缓存。本地磁盘缓存会自动将查询设计的数据文件存储在本地缓存目录中，并使用 LRU 策略管理缓存的汰换。具体可参阅数据缓存文档。

物化视图与透明改写

Doris 支持对外部数据源创建物化视图。物化视图根据 SQL 定义语句，预先将计算结果存储为 Doris 内表格式。同时，Doris 的查询优化器支持基于 SPJG（SELECT-PROJECT-JOIN-GROUP-BY）模式的透明改写算法。该算法能够分析 SQL 的结构信息，自动寻找合适的物化视图进行透明改写，并选择最优的物化视图来响应查询 SQL。

该功能通过减少运行时的计算量，可显著提升查询性能。同时可以在业务无感知的情况下，通过透明改写访问到物化视图中的数据。具体可参阅物化视图 文档。

2.14.1.2.2 多源联邦分析

Doris 可以作为 统一 SQL 查询引擎，连接不同数据源进行联邦分析，解决数据孤岛。

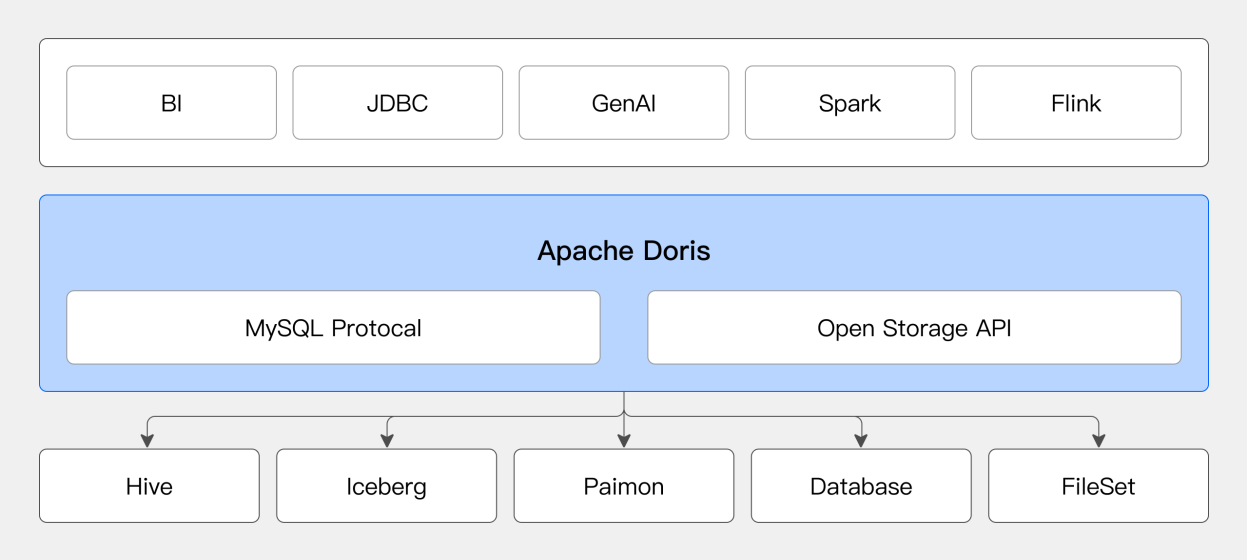


图 72: federation-query

用户可以在 Doris 中动态创建多个 Catalog 连接不同的数据源。并使用 SQL 语句对不同数据源中的数据进行任意关联查询。具体可参阅数据目录概述。

2.14.1.2.3 湖仓数据处理

在该场景中，Doris 作为数据处理引擎，对湖仓数据进行加工处理。

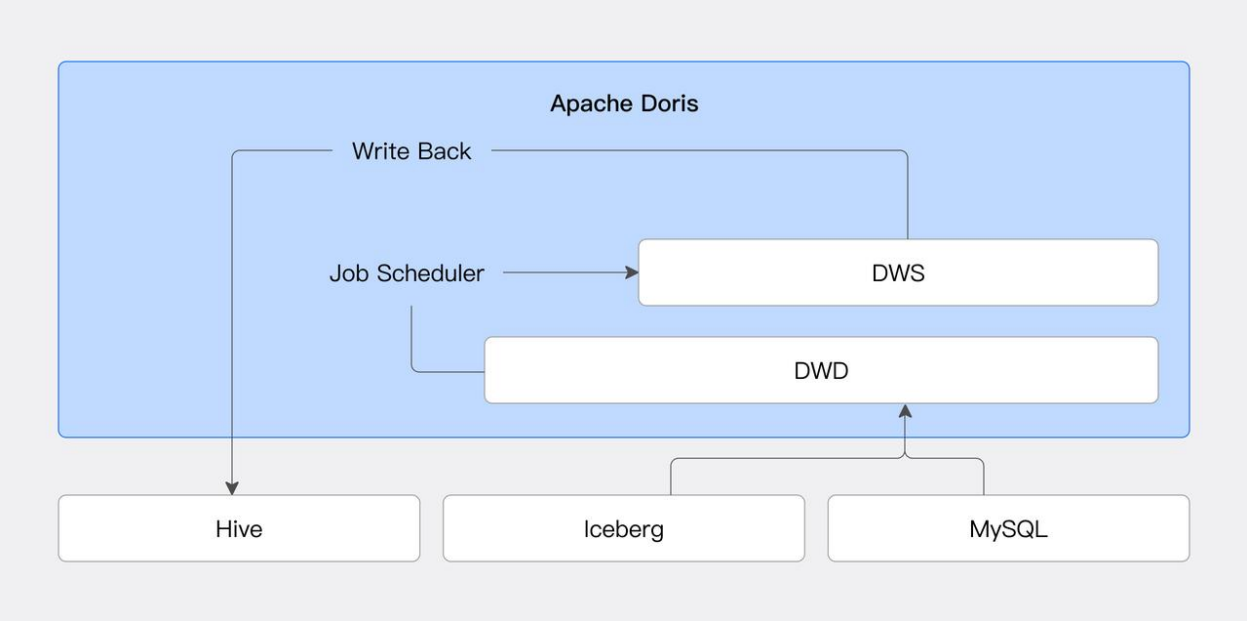


图 73: data-management

定时任务调度

Doris 通过引入 Job Scheduler 功能，可以实现高效灵活的任务调度，减少了对外部系统的依赖。结合数据源连接器，用户可以实现外部数据的定期加工入库。具体可参阅作业调度。

数据分层加工

企业通常会使用数据湖存储原始数据，在此基础上进行数据分层加工，将不同层的数据开放给不同的业务需求方。Doris 的物化视图功能支持对外部数据源创建物化视图，并支持在基于物化视图在加工，降低了分层加工的系统复杂度，提升数据处理效率。

数据写回

数据写回功能将 Doris 的湖仓数据处理能力形成闭环。户可以直接通过 Doris 在外部数据源中创建数据库、表，并写入数据。当前支持 JDBC、 Hive 和 Iceberg 三类数据源，后续会增加更多的数据源支持。具体可以参阅对应数据源的文档。

2.14.2 数据目录概述

数据目录（ Data Catalog ）用于描述一个数据源的属性。

在 Doris 中，可以创建多个数据目录指向不同的数据源（ 如 Hive、 Iceberg、 MySQL ）。 Doris 会通过数据目录，自动获取对应数据源的库、表、 Schema、 分区、 数据位置等。用户可以通过标准的 SQL 语句访问这些数据目录进行数据分析，并且可以对多个数据目录中的数据进行关联查询。

Doris 中的数据目录分为两种：

类型	说明
Internal Catalog	内置数据目录，名称固定为 `internal`，用于存储 Doris 内表数据。不可创建、更改和删除。

类型	说明
External Catalog	外部数据目录，指代所有 Internal Catalog 以外的数据目录。用户可以创建、更改、删除外部数据目录。

数据目录主要适用于以下三类场景，但不同的数据目录适用场景不同，详见对应数据目录的文档。

场景	说明
查询加速	针对湖仓数据如 Hive、 Iceberg、 Paimon 等进行直接查询加速。
数据集成	ZeroETL 方案，直接访问不同数据源生成结果数据，或让数据在不同数据源中便捷流转。
数据写回	通过 Doris 进行数据加工处理后，写回到外部数据源。

本文以 Iceberg Catalog 为例，重点介绍数据目录的基础操作。不同数据目录的详细介绍，请参阅对应的数据目录文档。

2.14.2.1 创建数据目录

通过 CREATE CATALOG 语句创建一个 Iceberg Catalog。

```
CREATE CATALOG iceberg_catalog PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'hadoop',
    'warehouse' = 's3://bucket/dir/key',
    's3.endpoint' = 's3.us-east-1.amazonaws.com',
    's3.access_key' = 'ak',
    's3.secret_key' = 'sk'
);
```

本质上，在 Doris 中创建的数据目录是作为 “代理” 访问对应数据源的元数据服务（如 Hive Metastore）和存储服务（如 HDFS/S3）。 Doris 中仅存储数据目录的连接属性等信息，不存储对应数据源实际的元数据和数据。

2.14.2.1.1 通用属性

除每个数据目录特有的属性集合外，这里介绍所有数据目录通用的属性 {CommonProperites}。

属性名	描述	示例
include_database_list	支持只同步指定的多个 Database，以 , 分隔。默认同步所有 Database。 Database 名称是大小写敏感的。当外部数据源有大量 Database，但仅需访问个别 Database 时，可以使用此参数，避免大量的元数据同步。	'include_database_list' = 'db1,db2'
exclude_database_list	支持指定不需要同步的多个 Database，以 , 分割。默认不做任何过滤，同步所有 Database。 Database 名称是大小写敏感的。适用场景同上，反向排除不需要访问的数据库。如果冲突，exclude 优先级高于 include	'exclude_database_list' = 'db1,db2'

2.14.2.1.2 列类型映射

用户创建数据目录后，Doris 会自动同步数据目录的数据库、表和 Schema。不同数据目录的列类型映射规则请参阅对应的数据目录文档。

对于当前无法映射到 Doris 列类型的外部数据类型，如 UNION, INTERVAL 等，Doris 会将列类型映射为 UNSUPPORTED 类型。对于 UNSUPPORTED 类型的查询，示例如下：

假设同步后的表 Schema 为：

```
k1 INT,
k2 INT,
k3 UNSUPPORTED,
k4 INT
```

则查询行为如下：

```
SELECT * FROM table;                -- Error: Unsupported type 'UNSUPPORTED_TYPE' in 'k3'
SELECT * EXCEPT(k3) FROM table;     -- Query OK.
SELECT k1, k3 FROM table;           -- Error: Unsupported type 'UNSUPPORTED_TYPE' in 'k3'
SELECT k1, k4 FROM table;           -- Query OK.
```

2.14.2.1.3 Nullable 属性

Doris 目前对外表列的 Nullable 属性支持有特殊限制，具体行为如下：

源类型	Doris 读取行为	Doris 写入行为
Nullable	Nullable	允许写入 Null 值
Not Null	Nullable，即依然当做可允许为 NULL 的列进行读取	允许写入 Null 值，即不对 Null 值进行严格检查。用户需要自行保证数据的完整性和一致性。

2.14.2.2 使用数据目录

2.14.2.2.1 查看数据目录

创建后，可以通过 SHOW CATALOGS 命令查看 catalog：

```
mysql> SHOW CATALOGS;
+-----------+-----------------+----------+-----------+-------------------------+---------------------+-----------
    ↪
| CatalogId | CatalogName     | Type     | IsCurrent | CreateTime              | LastUpdateTime
    ↪       | Comment                |
```

```
+-----------+-----------------+----------+-----------+-------------------------+-------------------+----------
    ↪
|     10024 | iceberg_catalog | hms      | yes       | 2023-12-25 16:11:41.687 | 2023-12-25
    ↪ 20:43:18 | NULL                    |
|         0 | internal        | internal |           | NULL                    | NULL
    ↪                | Doris internal catalog |
+-----------+-----------------+----------+-----------+-------------------------+-------------------+----------
    ↪
```

可以通过SHOW CREATE CATALOG 查看创建 Catalog 的语句。

2.14.2.2.2 切换数据目录

Doris 提供 SWITCH 语句用于将连接会话上下文切换到对应的数据目录。类似使用 USE 语句切换数据库。

切换到数据目录后，可以使用 USE 语句继续切换到指定的数据库。或通过 SHOW DATABASES 查看当前数据目录下的数据库。

```
SWITCH iceberg_catalog;

SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| test               |
| iceberg_db         |
+--------------------+

USE iceberg_db;
```

也可以通过 USE 语句，直接使用全限定名 catalog_name.database_name 切换到指定数据目录下的指定数据库：

```
USE iceberg_catalog.iceberg_db;
```

全限定名也可以用于 MySQL 命令行或 JDBC 连接串中，以兼容 MySQL 连接协议。

```

mysql -h host -P9030 -uroot -Diceberg_catalog.iceberg_db

### JDBC url
jdbc:mysql://host:9030/iceberg_catalog.iceberg_db
```

内置数据目录的固定名称为 internal。切换方式和外部数据目录一致。

2.14.2.2.3 默认数据目录

通过用户属性 default_init_catalog，为指定用户设置默认的数据目录。设置完成后，当指定用户连接 Doris 时，会自动切换到设置的数据目录。

```
SET PROPERTY default_init_catalog=hive_catalog;
```

注意 1：如果 MySQL 命令行或JDBC 连接串中已经明确指定了数据目录，则以指定的为准，default_init_catalog 用户属性不生效；注意 2：如果用户属性 default_init_catalog 设置的数据目录已经不存在，则自动切换到默认的 internal 数据目录；注意 3：该功能从 v3.1.x 版本开始生效；

2.14.2.2.4 简单查询

可以通过 Doris 支持的任意 SQL 语句查询外部数据目录中的表。

```
SELECT id, SUM(cost) FROM iceberg_db.table1
GROUP BY id ORDER BY id;
```

2.14.2.2.5 跨数据目录查询

Doris 支持跨数据目录的关联查询。

这里我们再创建一个 MySQL Catalog：

```
CREATE CATALOG mysql_catalog properties(
    'type' = 'jdbc',
    'user' = 'root',
    'password' = '123456',
    'jdbc_url' = 'jdbc:mysql://host:3306/mysql_db',
    'driver_url' = 'mysql-connector-java-8.0.25.jar',
    'driver_class' = 'com.mysql.cj.jdbc.Driver'
);
```

之后通过 SQL 对 Iceberg 表和 MySQL 表进行关联查询：

```
SELECT * FROM
iceberg_catalog.iceberg_db.table1 tbl1 JOIN mysql_catalog.mysql_db.dim_table tbl2
ON tbl1.id = tbl2.id;
```

2.14.2.2.6 数据导入

通过 INSERT 命令，可以将数据源中的数据导入到 Doris 中。

```
INSERT INTO internal.doris_db.tbl1
SELECT * FROM iceberg_catalog.iceberg_db.table1;
```

也可以通过 CTAS(Create Table As Select) 语句，使用外部数据源创建一张 Doris 内表并将数据导入：

```
CREATE TABLE internal.doris_db.tbl1
PROPERTIES('replication_num' = '1')
AS
SELECT * FROM iceberg_catalog.iceberg_db.table1;
```

2.14.2.2.7 数据写回

Doris 支持通过 INSERT 语句直接将数据写回到外部数据源。具体参阅：

- Hive Catalog
- Iceberg Catalog
- JDBC Catalog

2.14.2.3 刷新数据目录

Doris 中创建的数据目录是作为“代理”访问对应数据源的元数据服务的。Doris 会对部分元数据进行缓存。缓存可以提升元数据的访问性能，避免频繁的跨网络请求。但缓存也存在时效性问题，如果不对缓存进行刷新，则无法访问到最新的元数据。因此，Doris 提供了多种刷新数据目录的方式。

```
-- Refresh catalog
REFRESH CATALOG catalog_name;

-- Refresh specified database
REFRESH DATABASE catalog_name.db_name;

-- Refresh specified table
REFRESH TABLE catalog_name.db_name.table_name;
```

Doris 也支持关闭元数据缓存，以便能够实时访问到最新的元数据。

关于元数据缓存的详细介绍和配置，请参阅：元数据缓存

2.14.2.4 修改数据目录

可以通过 ALTER CATALOG 对数据目录的属性或名称进行修改：

```
-- Rename a catalog
ALTER CATALOG iceberg_catalog RENAME iceberg_catalog2;

-- Modify properties of a catalog
ALTER CATALOG iceberg_catalog SET PROPERTIES ('key1' = 'value1' [, 'key' = 'value2']);

-- Modify the comment of a catalog
ALTER CATALOG iceberg_catalog MODIFY COMMENT 'my iceberg catalog';
```

2.14.2.5 删除数据目录

可以通过 DROP CATALOG 删除指定的外部数据目录。

```
DROP CATALOG [IF EXISTS] iceberg_catalog;
```

从 Doris 中删除外部数据目录，并不会删除实际的数据，只是删除了 Doris 中存储的数据目录映射关系。

2.14.2.6 权限管理

外部数据目录中库表的权限管理和内表一致。具体可参阅认证和鉴权 文档。

2.14.3 数据目录

2.14.3.1 Hudi Catalog

Hudi Catalog 复用了 Hive Catalog。通过连接 Hive Metastore，或者兼容 Hive Metatore 的元数据服务，Doris 可以自动获取 Hudi 的库表信息，并进行数据查询。

使用 Docker 快速体验 Apache Doris & Hudi

2.14.3.1.1 适用场景

场景	说明
查询加速	利用 Doris 分布式计算引擎，直接访问 Hudi 数据进行查询加速。
数据集成	读取 Hudi 数据并写入到 Doris 内表。或通过 Doris 计算引擎进行 ZeroETL 操作。
数据写回	不支持。

2.14.3.1.2 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name PROPERTIES (
    'type' = 'hms', -- required
    'hive.metastore.uris' = '<metastore_thrift_url>', -- required
    {MetaStoreProperties},
    {StorageProperties},
    {HudiProperties},
    {CommonProperties}
);
```

- [MetaStoreProperties]

MetaStoreProperties 部分用于填写 Metastore 元数据服务连接和认证信息。具体可参阅【支持的元数据服务】部分。

- [StorageProperties]

StorageProperties 部分用于填写存储系统相关的连接和认证信息。具体可参阅【支持的存储系统】部分。

- [CommonProperties]

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

- {HudiProperties}

参数名称	曾用名	说明	默认值
hudi.use_ ↪ hive_sync ↪ _ ↪ partition	use_hive_ ↪ sync_ ↪ partition ↪	是否使用 Hive Metastore 已同步的分区信息。如果为 true，则会直接从 Hive Metastore 中获取分区信息。否则，会从文件系统的元数据文件中获取分区信息。通过 Hive Metastore 获取信息性能更好，但需要用户保证最新的元数据已经同步到了 Hive Metastore。	false

支持的 Hudi 版本

当前依赖的 Hudi 版本为 0.15。推荐访问 0.14 版本以上的 Hudi 数据。

支持的查询类型

表类型	支持的查询类型
Copy On Write	Snapshot Query, Time Travel, Incremental Read
Merge On Read	Snapshot Queries, Read Optimized Queries, Time Travel, Incremental Read

支持的元数据服务

- Hive Metastore

支持的存储系统

- HDFS
- AWS S3
- Google Cloud Storage
- 阿里云 OSS
- 腾讯云 COS
- 华为云 OBS
- MINIO

支持的数据格式

- Parquet
- ORC

2.14.3.1.3 列类型映射

Hudi Type	Doris Type	Comment
boolean	boolean	
int	int	
long	bigint	
float	float	
double	double	
decimal(P, S)	decimal(P, S)	
bytes	string	
string	string	
date	date	
timestamp	datetime(N)	根据精度，自动映射到 datetime(3) 或 datetime(6)
array	array	
map	map	
struct	struct	
other	UNSUPPORTED	

2.14.3.1.4 基础示例

Hudi Catalog 的创建方式和 Hive Catalog 一致。更多示例可参阅 Hive Catalog

```
CREATE CATALOG hudi_hms PROPERTIES (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://172.21.0.1:7004',
    'hadoop.username' = 'hive',
    'dfs.nameservices'='your-nameservice',
    'dfs.ha.namenodes.your-nameservice'='nn1,nn2',
    'dfs.namenode.rpc-address.your-nameservice.nn1'='172.21.0.2:4007',
    'dfs.namenode.rpc-address.your-nameservice.nn2'='172.21.0.3:4007',
    'dfs.client.failover.proxy.provider.your-nameservice'='org.apache.hadoop.hdfs.server.namenode
        ↪ .ha.ConfiguredFailoverProxyProvider'
);
```

2.14.3.1.5 查询操作

基础查询

配置好 Catalog 后，可以通过以下方式查询 Catalog 中的表数据：

```
-- 1. switch to catalog, use database and query
SWITCH hudi_ctl;
USE hudi_db;
SELECT * FROM hudi_tbl LIMIT 10;

-- 2. use hudi database directly
USE hudi_ctl.hudi_db;
SELECT * FROM hudi_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM hudi_ctl.hudi_db.hudi_tbl LIMIT 10;
```

时间旅行

每一次对 Hudi 表的写操作都会产生一个新的快照，Doris 支持读取 Hudi 表指定的 Snapshot。默认情况下，查询请求只会读取最新版本的快照。

可以通过 `hudi_meta()` 表函数查询查询指定 Hudi 表的 Timeline：

该函数自 3.1.0 版本支持。

```
SELECT * FROM hudi_meta(
    'table' = 'hudi_ctl.hudi_db.hudi_tbl',
    'query_type' = 'timeline'
);

+-------------------+---------+--------------------------+-----------+-----------------------+
| timestamp         | action | file_name                | state     | state_transition_time |
+-------------------+---------+--------------------------+-----------+-----------------------+
| 20241202171214902 | commit | 20241202171214902.commit | COMPLETED | 20241202171215756     |
| 20241202171217258 | commit | 20241202171217258.commit | COMPLETED | 20241202171218127     |
| 20241202171219557 | commit | 20241202171219557.commit | COMPLETED | 20241202171220308     |
| 20241202171221769 | commit | 20241202171221769.commit | COMPLETED | 20241202171222541     |
| 20241202171224269 | commit | 20241202171224269.commit | COMPLETED | 20241202171224995     |
| 20241202171226401 | commit | 20241202171226401.commit | COMPLETED | 20241202171227155     |
| 20241202171228827 | commit | 20241202171228827.commit | COMPLETED | 20241202171229570     |
| 20241202171230907 | commit | 20241202171230907.commit | COMPLETED | 20241202171231686     |
| 20241202171233356 | commit | 20241202171233356.commit | COMPLETED | 20241202171234288     |
| 20241202171235940 | commit | 20241202171235940.commit | COMPLETED | 20241202171236757     |
+-------------------+---------+--------------------------+-----------+-----------------------+
```

可以使用 `FOR TIME AS OF` 语句，根据快照的时间 (时间格式和 Hudi 官网保持一致) 读取历史版本的数据。示例如下：

```
SELECT * FROM hudi_tbl FOR TIME AS OF "2022-10-07 17:20:37";
SELECT * FROM hudi_tbl FOR TIME AS OF "20221007172037";
SELECT * FROM hudi_tbl FOR TIME AS OF "2022-10-07";
```

Hudi 表不支持 FOR VERSION AS OF 语句，使用该语法查询 Hudi 表将报错。

增量查询

Incremental Read 可以查询在指定时间段之间变化的数据，返回的结果集是数据在指定时间段结束时的最终状态。

Doris 提供了 @incr 语法支持 Incremental Read:

```
SELECT * from hudi_table@incr('beginTime'='xxx', ['endTime'='xxx'], ['hoodie.read.timeline.holes.
    ↪ resolution.policy'='FAIL'], ...);
```

- beginTime

必填项。时间格式和 Hudi 官网 hudi_table_changes 保持一致，支持“earliest”。

- endTime

选填，默认最新 commitTime。

可以在 @incr函数中添加更多选项，兼容 Spark Read Options。

通过 desc 查看执行计划，可以发现 Doris 将 @incr 转化为 predicates 下推给 VHUDI_SCAN_NODE:

```
|   0:VHUDI_SCAN_NODE(113)
    ↪
    ↪ |
|      table: lineitem_mor
    ↪
    ↪ |
|      predicates: (_hoodie_commit_time[#0] > '20240311151019723'), (_hoodie_commit_time[#0] <=
    ↪ '20240311151606605') |
|      inputSplitNum=1, totalFileSize=13099711, scanRanges=1
```

2.14.3.1.6 常见问题

1. 通过 JNI 调用 Java SDK 读取 Hudi 增量数据偶发卡死

   请在 be.conf 的 JAVA_OPTS_FOR_JDK_17 或 JAVA_OPTS 中添加 -Djol.skipHotspotSAAttach=true.

2.14.3.1.7 附录

版本更新记录

Doris 版本	功能支持
2.1.8/3.0.4	Hudi 依赖升级到 0.15。新增 Hadoop Hudi JNI Scanner。

2.14.3.2 MaxCompute Catalog

MaxCompute 是阿里云上的企业级 SaaS（Software as a Service）模式云数据仓库。通过 MaxCompute 提供的开放存储 SDK，Doris 可以获取 MaxCompute 的表信息并进行查询。

2.14.3.2.1 适用场景

场景	说明
数据集成	读取 MaxCompute 数据并写入到 Doris 内表。
数据写回	不支持。

2.14.3.2.2 使用须知

1. 自 2.1.7 版本开始，MaxCompute Catalog 基于 开放存储 SDK 开发，在这之前，基于 Tunnel API 进行开发。
2. 开放存储 SDK 的使用有一定的限制，请参照该 文档 中 使用限制 的章节。
3. 在 Doris 3.1.3 版本之前，MaxCompute 中的 Project 相当于 Doris 中的 Database。3.1.3 版本中，可以通过 `mc.enable.namespace.schema` 参数引入 MaxCompute 的 schema 层级。

2.14.3.2.3 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name PROPERTIES (
    'type' = 'max_compute',
    {McRequiredProperties},
    {McOptionalProperties},
    {CommonProperties}
);
```

- `{McRequiredProperties}`

属性名	说明	支持的 Doris 版本
`mc.default.project`	想要访问的 MaxCompute 项目名称。可以在 MaxCompute 项目列表 中创建和管理。	
`mc.access_key`	AccessKey。可以在 阿里云控制台 中创建和管理。	
`mc.secret_key`	SecretKey。可以在 阿里云控制台 中创建和管理。	
`mc.region`	MaxCompute 开通的地域。可以从 Endpoint 中找到对应的 Region	2.1.7（不含）之前
`mc.endpoint`	MaxCompute 开通的地域。请参照下文的如何获取 Endpoint 和 Quota 来配置。	2.1.7（含）之后

- `{McOptionalProperties}`

属性名	默认值	说明	支持的 Doris 版本
mc.tunnel_endpoint	无	参考附录中的自定义服务地址。	2.1.7（不含）之前
mc.odps_endpoint	无	参考附录中的自定义服务地址。	2.1.7（不含）之前
mc.quota	pay-as-you-go	Quota 名称。请参照下文的如何获取 Endpoint 和 Quota 来配置	2.1.7（含）之后
mc.split_strategy	byte_size	设置 split 的划分方式，可设置为按照字节大小划分 byte_size 和按照数据行数划分 row_count	2.1.7（含）之后
mc.split_byte_size	268435456	每个 split 读取的文件大小，单位为字节，默认为 256MB，当且仅当 "mc.split_strategy" = "byte_size" 时生效	2.1.7（含）之后
mc.split_row_count	1048576	每个 split 读多少行，当且仅当 "mc.split_strategy" = "row_count" 时生效	2.1.7（含）之后
mc.split_cross_partition	false	生成的 split 是否跨分区	2.1.8（含）之后
mc.connect_timeout	10s	连接 maxcompute 的超时时间	2.1.8（含）之后
mc.read_timeout	120s	读取 maxcompute 的超时时间	2.1.8（含）之后
mc.retry_count	4	超时后的重试次数	2.1.8（含）之后
mc.datetime_predicate_push_down	true	是否允许下推 timestamp/timestamp_ntz 类型的谓词条件。Doris 对这两个类型的同步会丢失精度（9 -> 6）。因此如果原数据精度高于 6 位，则条件下推可能导致结果不准确。	2.1.9/3.0.5（含）之后
mc.account_format	name	阿里云国际站和中国站的账号系统不一致，对于国际站用户，如出现如 user 'RAM$xxxxxx:xxxxx' is not a valid aliyun account 的错误，可指定该参数为 id。	3.0.9/3.1.1（含）之后
mc.enable.namespace.schema	false	是否支持 MaxCompute 的 schema 层级。详见：https://help.aliyun.com/zh/maxcompute/user-guide/schema-related-operations	3.1.3（含）之后

- [CommonProperties]

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

支持的 MaxCompute 版本

仅支持公有云版本的 MaxCompute。私有云版本支持请联系 Doris 社区支持。

支持的 MaxCompute 表

- 支持读取分区表、聚簇表、物化视图。
- 不支持读取 MaxCompute 的外部表、逻辑视图、 Delta Table。

2.14.3.2.4 层级映射

- `mc.enable.namespace.schema` 为 false

Doris	MaxCompute
Catalog	N/A
Database	Project
Table	Table

- `mc.enable.namespace.schema` 为 true

Doris	MaxCompute
Catalog	Project
Database	Schema
Table	Table

2.14.3.2.5 列类型映射

MaxCompute Type	Doris Type	Comment
bolean	boolean	
tiny	tinyint	
tinyint	tinyint	
smallint	smallint	
int	int	
bigint	bigint	
float	float	
double	double	
decimal(P, S)	decimal(P, S)	
char(N)	char(N)	
varchar(N)	varchar(N)	
string	string	
date	date	

MaxCompute Type	Doris Type	Comment
datetime	datetime(3)	固定映射到精度 3。可以通过 `SET [GLOBAL] time_zone = 'Asia/Shanghai'` 来指定时区
timestamp_ntz	datetime(6)	MaxCompute 的 `timestamp_ntz` 精度为 9, Doris 的 DATETIME 最大精度只有 6，故读取数据时会将多的部分直接截断。
timestamp	datetime(6)	自 2.1.9/3.0.5 支持。MaxCompute 的 `timestamp` 精度为 9, Doris 的 DATETIME 最大精度只有 6，故读取数据时会将多的部分直接截断。
array	array	
map	map	
struct	struct	
other	UNSUPPORTED	

2.14.3.2.6 基础示例

```
CREATE CATALOG mc_catalog PROPERTIES (
    'type' = 'max_compute',
    'mc.default.project' = 'project',
    'mc.access_key' = 'sk',
    'mc.secret_key' = 'ak',
    'mc.endpoint' = 'http://service.cn-beijing-vpc.MaxCompute.aliyun-inc.com/api'
);
```

如使用 2.1.7（不含）之前的版本，请使用如下语句。（建议升级到 2.1.8 后使用）

```
CREATE CATALOG mc_catalog PROPERTIES (
    'type' = 'max_compute',
    'mc.region' = 'cn-beijing',
    'mc.default.project' = 'project',
    'mc.access_key' = 'ak',
    'mc.secret_key' = 'sk'
    'mc.odps_endpoint' = 'http://service.cn-beijing.maxcompute.aliyun-inc.com/api',
    'mc.tunnel_endpoint' = 'http://dt.cn-beijing.maxcompute.aliyun-inc.com'
);
```

支持 Schema：

```
CREATE CATALOG mc_catalog PROPERTIES (
    'type' = 'max_compute',
    'mc.region' = 'cn-beijing',
    'mc.default.project' = 'project',
    'mc.access_key' = 'ak',
    'mc.secret_key' = 'sk'
    'mc.odps_endpoint' = 'http://service.cn-beijing.maxcompute.aliyun-inc.com/api',
    'mc.tunnel_endpoint' = 'http://dt.cn-beijing.maxcompute.aliyun-inc.com',
```

```
    'mc.enable.namespace.schema' = 'true'
);
```

2.14.3.2.7 查询操作

基础查询

```
-- 1. switch to catalog, use database and query
SWITCH mc_ctl;
USE mc_ctl;
SELECT * FROM mc_tbl LIMIT 10;

-- 2. use mc database directly
USE mc_ctl.mc_db;
SELECT * FROM mc_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM mc_ctl.mc_db.mc_tbl LIMIT 10;
```

2.14.3.2.8 附录

如何获取 Endpoint 和 Quota(适用于 Doris 2.1.7 之后)

1. 如果使用数据传输服务独享资源组

   请参照该 文档 中【使用独享数据服务资源组】章节中的【2. 授权】来开启相应的权限，并在【配额（Quota）管理】列表中，查看并复制对应的 QuotaName，指定 "mc.quota" = "QuotaName"。此时您可以选择 VPC 或公网来访问 MaxCompute，但是走 VPC 的带宽有保障，公网带宽资源小。

2. 如果使用按量付费

   请参照该 文档 中【使用开放存储（按量付费）】的章节，来开启开放存储 (Storage API) 开关，并给 Ak,SK 对应的用户赋予权限。此时 mc.quota 为默认值 pay-as-you-go，不需要额外指定该值。按量付费情况下，只能使用 VPC 来访问 MaxCompute，无法通过公网访问。只有预付费用户才能通过公网访问 MaxCompute。

3. 根据 阿里云 Endpoints 文档 中的【地域 Endpoint 对照表】来配置 mc.endpoint

使用 VPC 访问的用户，需要根据【各地域 Endpoint 对照表（阿里云 VPC 网络连接方式）】表中的【VPC 网络 Endpoint】列来配置 mc.endpoint。使用公网访问的用户，可以选择【各地域 Endpoint 对照表（阿里云经典网络连接方式）】表中的【经典网络 Endpoint】列、或者选择【各地域 Endpoint 对照表（外网连接方式)】表中的【外网 Endpoint 列来配置 mc.endpoint。

自定义服务地址 (适用于 Doris 2.1.7 之前)

在 Doris 2.1.7 之前的版本中，使用 Tunnel SDK 与 MaxCompute 交互，因此需要使用以下两个 endpoint 属性：

- mc.odps_endpoint：MaxCompute Endpoint，用于获取 MaxCompute 元数据（库表信息）。
- mc.tunnel_endpoint: Tunnel Endpoint，用于读取 MaxCompute 数据。

默认情况下，MaxCompute Catalog 根据 mc.region 和 mc.public_access 去生成 endpoint。

生成后的格式如下：

mc.public_access	mc.odps_endpoint	mc.tunnel_endpoint
false	http://service.{mc.region}.maxcompute.aliyun-inc.com/api	http://dt.{mc.region}.maxcompute.aliyun-inc.com
true	http://service.{mc.region}.maxcompute.aliyun.com/api	http://dt.{mc.region}.maxcompute.aliyun.com

用户也可以单独指定mc.odps_endpoint 和 mc.tunnel_endpoint 来自定义服务地址，适用于一些私有部署的 MaxCompute 环境。

MaxCompute Endpoint 和 Tunnel Endpoint 的配置请参见各地域及不同网络连接方式下的 Endpoint。

2.14.3.3 Delta Lake Catalog

Delta Lake Catalog 通过 Trino Connector 兼容框架，使用 Delta Lake Connector 来访问 Delta Lake 表。

> 该功能为实验功能，自 3.0.1 版本开始支持。

2.14.3.3.1 适用场景

场景	说明
数据集成	读取 Detla Lake 数据并写入到 Doris 内表。
数据写回	不支持。

2.14.3.3.2 环境准备

编译 Delta Lake Connector 插件

> 需要 JDK 17 版本。

```
$ git clone https://github.com/apache/doris-thirdparty.git
$ cd doris-thirdparty
$ git checkout trino-435
$ cd plugin/trino-delta-lake
$ mvn clean install -DskipTest
```

```
$ cd ../../lib/trino-hdfs
$ mvn clean install -DskipTest
```

完成编译后，会在 trino/plugin/trino-delta-lake/target/ 下得到 trino-delta-lake-435 目录，在 trino/
↪ lib/trino-hdfs/target/ 下得到 hdfs 目录

也可以直接下载预编译的 trino-delta-lake-435-20240724.tar.gz 及 hdfs.tar.gz 并解压。

部署 Delta Lake Connector

将 trino-delta-lake-435/ 目录放到所有 FE 和 BE 部署路径的 connectors/ 目录下（如果没有，可以手动创建），将 hdfs.tar.gz 解压到 trino-delta-lake-435/ 目录下。

```
├── bin
├── conf
├── connectors
|   ├── trino-delta-lake-435
|   |   ├── hdfs
...
```

部署完成后，建议重启 FE、 BE 节点以确保 Connector 可以被正确加载。

2.14.3.3.3 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name
PROPERTIES (
    'type' = 'trino-connector', -- required
    'trino.connector.name' = 'delta_lake', -- required
    {TrinoProperties},
    {CommonProperties}
);
```

- {TrinoProperties}

TrinoProperties 部分用于填写将传递给 Trino Connector 的属性，这些属性以trino.为前缀。理论上，Trino 支持的属性这里都支持，更多有关 Delta Lake 的信息可以参考 Trino 文档。

- [CommonProperties]

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

支持的 Delta Lake 版本

更多有关 Delta Lake 的信息可以参考 Trino 文档。

支持的元数据服务

更多有关 Delta Lake 的信息可以参考 Trino 文档。

支持的存储系统

更多有关 Delta Lake 的信息可以参考 Trino 文档。

2.14.3.3.4 列类型映射

Delta Lake Type	Trino Type	Doris Type	Comment
boolean	boolean	boolean	
int	int	int	
byte	tinyint	tinyint	
short	smallint	smallint	
long	bigint	bigint	
float	real	float	
double	double	double	
decimal(P, S)	decimal(P, S)	decimal(P, S)	
string	varchar	string	
bianry	varbinary	string	
date	date	date	
timestamp_ntz	timestamp(N)	datetime(N)	
timestamp	timestamp with time zone(N)	datetime(N)	
array	array	array	
map	map	map	
struct	row	struct	

2.14.3.3.5 基础示例

```
CREATE CATALOG delta_lake_hms properties (
    'type' = 'trino-connector',
    'trino.connector.name' = 'delta_lake',
    'trino.hive.metastore' = 'thrift',
    'trino.hive.metastore.uri'= 'thrift://ip:port',
    'trino.hive.config.resources'='/path/to/core-site.xml,/path/to/hdfs-site.xml'
);
```

2.14.3.3.6 查询操作

配置好 Catalog 后，可以通过以下方式查询 Catalog 中的表数据：

```
-- 1. switch to catalog, use database and query
SWITCH delta_lake_ctl;
USE delta_lake_db;
SELECT * FROM delta_lake_tbl LIMIT 10;

-- 2. use dalta lake database directly
USE delta_lake_ctl.delta_lake_db;
```

```
SELECT * FROM delta_lake_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM delta_lake_ctl.delta_lake_db.delta_lake_tbl LIMIT 10;
```

2.14.3.4 BigQuery Catalog

BigQuery Catalog 通过 Trino Connector 兼容框架，使用 BigQuery Connector 来访问 BigQuery 表。

> 该功能为实验功能，自 3.0.1 版本开始支持。

2.14.3.4.1 适用场景

场景	说明
数据集成	读取 BigQuery 数据并写入到 Doris 内表。
数据写回	不支持。

2.14.3.4.2 环境准备

编译 BigQuery Connector 插件

> 需要 JDK 17 版本。

```
$ git clone https://github.com/apache/doris-thirdparty.git
$ cd doris-thirdparty
$ git checkout trino-435
$ cd plugin/trino-bigquery
$ mvn clean install -DskipTest
```

完成编译后，会在 `trino/plugin/trino-bigquery/target/` 下得到 `trino-bigquery-435/` 目录。

也可以直接下载我们预编译的 trino-bigquery-435-20240724.tar.gz 并解压。

部署 BigQuery Connector

将 `trino-bigquery-435/` 目录放到所有 FE 和 BE 部署路径的 `connectors/` 目录下。(如果没有，可以手动创建)。

```
├── bin
├── conf
├── connectors
```

```
|   ├── trino-bigquery-435
...
```

部署完成后，建议重启 FE、 BE 节点以确保 Connector 可以被正确加载。

准备 Google Cloud ADC 认证

1. 安装 gcloud CLI：https://cloud.google.com/sdk/docs/install
2. 执行 `gcloud init --console-only --skip-diagnostics`
3. 执行 `gcloud auth login`
4. 执行 `gcloud auth application-default login`

这一步是生成 ADC 认证文件，生成后的 json 默认放在 `~/.config/gcloud/application_default_credentials.json`

2.14.3.4.3 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name
PROPERTIES (
    'type' = 'trino-connector', -- required
    'trino.connector.name' = 'bigquery', -- required
    {TrinoProperties},
    {CommonProperties}
);
```

- `{TrinoProperties}`

TrinoProperties 部分用于填写将传递给 Trino Connector 的属性，这些属性以`trino.`为前缀。理论上，Trino 支持的属性这里都支持，更多有关 BigQuery 的属性可以参考 Trino 文档。

- `[CommonProperties]`

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

支持的 BigQuery 版本

更多有关 BigQuery 的属性可以参考 Trino 文档。

BigQuery Type	Trino Type	Doris Type

2.14.3.4.4 列类型映射

BigQuery Type	Trino Type	Doris Type
boolean	boolean	boolean
int64	bigint	bigint
float64	double	double
numeric	decimal(P, S)	decimal(P, S)
bignumric	decimal(P, S)	decimal(P, S)
string	varchar	string
bytes	varbinary	string
date	date	date
datetime	timestamp(6)	datetime(6)
time	time(6)	string
timestamp	timestamp with time zone(6)	datetime(6)
geography	varchar	string
array	array	array
map	map	map
struct	row	struct

2.14.3.4.5 基础示例

```
CREATE CATALOG bigquery_catalog PROPERTIES (
    'type' = 'trino-connector',
    'trino.connector.name' = 'bigquery',
    'trino.bigquery.project-id' = 'your-bigquery-project-id',
    'trino.bigquery.credentials-file' = '/path/to/application_default_credentials.json',
);
```

2.14.3.4.6 查询操作

配置好 Catalog 后，可以通过以下方式查询 Catalog 中的表数据：

```
-- 1. switch to catalog, use database and query
SWITCH bigquery_ctl;
USE bigquery_db;
SELECT * FROM bigquery_tbl LIMIT 10;

-- 2. use bigquery database directly
USE bigquery_ctl.bigquery_db;
SELECT * FROM bigquery_tbl LIMIT 10;

-- 3. use full qualified name to query
```

```
SELECT * FROM bigquery_ctl.bigquery_db.bigquery_tbl LIMIT 10;
```

2.14.3.5 Kudu Catalog

Kudu Catalog 通过 Trino Connector 兼容框架，使用 Trino Kudu Connector 来访问 Kudu 表。

> 该功能为实验功能，自 3.0.1 版本开始支持。

2.14.3.5.1 适用场景

场景	说明
数据集成	读取 Kudu 数据并写入到 Doris 内表。
数据写回	不支持。

2.14.3.5.2 环境准备

编译 Kudu Connector 插件

> 需要 JDK 17 版本。

```
$ git clone https://github.com/apache/doris-thirdparty.git
$ cd doris-thirdparty
$ git checkout trino-435
$ cd plugin/trino-kudu
$ mvn clean package -Dmaven.test.skip=true
```

完成编译后，会在 `trino/plugin/trino-kudu/target/` 下得到 `trino-kudu-435/` 目录。

也可以直接下载预编译的 trino-kudu-435-20240724.tar.gz 并解压。

部署 Kudu Connector

将 `trino-kudu-435/` 目录放到所有 FE 和 BE 部署路径的 `connectors/` 目录下。(如果没有，可以手动创建)。

```
├── bin
├── conf
├── connectors
|   ├── trino-kudu-435
...
```

部署完成后，建议重启 FE、 BE 节点以确保 Connector 可以被正确加载。

2.14.3.5.3 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name PROPERTIES (
    'type' = 'trino-connector', -- required
    'trino.connector.name' = 'kudu', -- required
    {TrinoProperties},
    {CommonProperties}
);
```

- `{TrinoProperties}`

TrinoProperties 部分用于填写将传递给 Trino Connector 的属性，这些属性以`trino.`为前缀。理论上，Trino 支持的属性这里都支持，更多有关 Kudu 的属性可以参考 Trino 文档。

- `[CommonProperties]`

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

支持的 Kudu 版本

更多有关 Kudu 的信息可以参考 Trino 文档。

支持的元数据服务

更多有关 Kudu 的信息可以参考 Trino 文档。

支持的存储系统

更多有关 Kudu 的信息可以参考 Trino 文档。

2.14.3.5.4 列类型映射

Kudu Type	Trino Type	Doris Type	Comment
boolean	boolean	boolean	
int8	tinyint	tinyint	
int16	smallint	smallint	
int32	integer	int	
int64	bigint	bigint	
float	real	float	
double	double	double	
decimal(P, S)	decimal(P, S)	decimal(P, S)	
binary	varbinary	string	需要适用 HEX(col) 删除查询，才能返回和 Trino 一样的显示结果。
string	varchar	string	
date	date	date	
unixtime_micros	timestamp(3)	datetime(3)	
other	UNSUPPORTED		

2.14.3.5.5 基础示例

```
CREATE CATALOG kudu_catalog PROPERTIES (
    'type' = 'trino-connector',
    'trino.connector.name' = 'kudu',
    'trino.kudu.client.master-addresses' = 'ip1:port1,ip2:port2,ip3,port3',
    'trino.kudu.authentication.type' = 'NONE'
);
```

2.14.3.5.6 查询操作

配置好 Catalog 后，可以通过以下方式查询 Catalog 中的表数据：

```
-- 1. switch to catalog, use database and query
SWITCH kudu_ctl;
USE kudu_db;
SELECT * FROM kudu_tbl LIMIT 10;

-- 2. use kudu database directly
USE kudu_ctl.kudu_db;
SELECT * FROM kudu_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM kudu_ctl.kudu_db.kudu_tbl LIMIT 10;
```

2.14.3.6 LakeSoul Catalog

> Note! 该功能自 3.1.0 版本后被移除。如需使用，请联系开发人员。

Doris 支持通过存储在 PostgreSQL 中的元数据，访问并读取 LakeSoul 表数据。

使用 Docker 快速体验 Apache Doris & LakeSoul

2.14.3.6.1 适用场景

场景	说明
数据集成	读取 LakeSoul 数据并写入到 Doris 内表。或通过 Doris 计算引擎进行 ZeroETL 操作。
数据写回	不支持。

2.14.3.6.2 配置 Catalog

语法

```
CREATE CATALOG lakesoul_catalog PROPERTIES (
    'type' = 'lakesoul',
    {LakeSoulProperties},
    {CommonProperties}
);
```

- {LakeSoulProperties}

| 属性 | 说明 | 示例 | | — | — | — | | lakesoul.pg.username | PG 源数据库的用户名 | | | lakesoul.pg.password | PG 源数据库的密码 | | | lakesoul.pg.url | PG 元数据库的 JDBC URL | jdbc:postgresql://127.0.0.1:5432/
↪ lakesoul_test?stringtype=unspecified |

- [CommonProperties]

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

如果 LakeSoul 的数据存储在 HDFS 上，需要将 core-site.xml，hdfs-site.xml 和 hive-site.xml 放到 FE 和 BE 的 conf/ 目录下。优先读取 conf/ 目录下的 hadoop 配置文件，再读取环境变量 HADOOP_CONF_DIR 的相关配置文件。

支持的 LakeSoul 版本

当前依赖的 LakeSoul 版本为 2.6.2。

支持的 LakeSoul 格式

- 支持 LakeSoul 的主键表、无主键表
- 支持 LakeSoul MOR 表读取。

2.14.3.6.3 列类型映射

LakeSoul Type	Doris Type	Comment
boolean	boolean	
int8	tinyint	
int16	smallint	
int32	int	
int64	bigint	
float	float	
double	double	
decimal(P, S)	decimal(P, S)	
string	string	
date	date	
timestamp(S)	datetime(S)	
list	array	

LakeSoul Type	Doris Type	Comment
map	map	
row	struct	
other	UNSUPPORTED	

2.14.3.6.4 基础示例

```
CREATE CATALOG lakesoul PROPERTIES (
    'type' = 'lakesoul',
    'lakesoul.pg.username' = 'lakesoul_test',
    'lakesoul.pg.password' = 'lakesoul_test',
    'lakesoul.pg.url' = 'jdbc:postgresql://127.0.0.1:5432/lakesoul_test?stringtype=unspecified'
);
```

2.14.3.6.5 查询操作

基础查询

配置好 Catalog 后，可以通过以下方式查询 Catalog 中的表数据：

```
-- 1. switch to catalog, use database and query
SWITCH ls_ctl;
USE ls_db;
SELECT * FROM ls_tbl LIMIT 10;

-- 2. use lakesoul database directly
USE ls_ctl.ls_db;
SELECT * FROM ls_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM ls_ctl.ls_db.ls_tbl LIMIT 10;
```

2.14.3.7 JDBC Catalog

JDBC Catalog 支持通过标准 JDBC 接口连接支持 JDBC 协议的数据库。

本文档介绍 JDBC Catalog 的通用配置和使用方法。不同的 JDBC 源请参阅对应的文档。

> 备注 Doris 的 JDBC Catalog 功能依赖 Java 层读取和处理数据，整体性能在一定程度上会受到 JDK 版本的影响，比如一些内部库的实现在老版本 JDK（如 JDK 8）中效率较低，可能会导致资源消耗偏高。如果对性能有更高要求，推荐使用 Doris 3.0 版本，由于默认使用 JDK 17 编译，整体性能更优。

2.14.3.7.1 适用场景

JDBC Catalog 仅适用于数据集成，如将少量数据从数据源导入到 Doris 中，或对 JDBC 数据源中的小表进行关联查询。JDBC Catalog 无法对数据源进行查询加速，或一次性访问大量数据。

2.14.3.7.2 支持的数据库

Doris JDBC Catalog 支持连接以下数据库：

支持的数据源
MySQL
PostgreSQL
Oracle
SQL Server
IBM DB2
ClickHouse
SAP HANA
Oceanbase

可参考 开发者手册 开发新的未支持的 JDBC 数据源。

2.14.3.7.3 配置 Catalog

语法

```
CREATE CATALOG [IF NOT EXISTS] catalog_name PROPERTIES (
    'type' ='jdbc', -- required
    {JdbcProperties},
    {CommonProperties}
);
```

- {JdbcProperties}
- 必须属性

参数名称	说明	示例
user	数据源用户名	
password	数据源密码	
jdbc_url	数据源连接 URL	jdbc:mysql://host:3306
driver_url	数据源 JDBC 驱动程序文件的路径。关于驱动包安全性，详见附录。	驱动程序支持三种方式，详见下面说明。
driver_class	数据源 JDBC 驱动程序的类名	

driver_url 支持以下三种指定方式：

1. 文件名。如 mysql-connector-j-8.3.0.jar。需将 Jar 包预先存放在 FE 和 BE 部署目录下的 jdbc_

↪ drivers/ 目录下。系统会自动在这个目录下寻找。该目录的位置，也可以由 fe.conf 和 be.conf 中的 jdbc_drivers_dir 配置修改。

2. 本地绝对路径。如 file:///path/to/mysql-connector-j-8.3.0.jar。需将 Jar 包预先存放在所有 FE/BE 节点指定的路径下。
3. Http 地址。如：http://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.3.0/mysql-↪ connector-j-8.3.0.jar 系统会从这个 Http 地址下载 Driver 文件。仅支持无认证的 Http 服务。

- 可选属性

参数名称	默认值	说明
lower_case_meta_↪ names	false	是否以小写的形式同步外部数据源的库名和表名以及列名
meta_names_mapping		当外部数据源存在名称相同只有大小写不同的情况，例如 MY_TABLE 和 my_table，Doris 由于歧义而在查询 Catalog 时报错，此时需要配置 meta_names_mapping 参数来解决冲突。
only_specified_↪ database	false	是否只同步 jdbc_url 中指定的数据源的 Database（此处的 Database 为映射到 Doris 的 Database 层级）
connection_pool_min↪ _size	1	定义连接池的最小连接数，用于初始化连接池并保证在启用保活机制时至少有该数量的连接处于活跃状态。
connection_pool_max↪ _size	30	定义连接池的最大连接数，每个 Catalog 对应的每个 FE 或 BE 节点最多可持有此数量的连接。
connection_pool_max↪ _wait_time	5000	如果连接池中没有可用连接，定义客户端等待连接的最大毫秒数。
connection_pool_max↪ _life_time	1800000	设置连接在连接池中保持活跃的最大时长（毫秒）。超时的连接将被回收。同时，此值的一半将作为连接池的最小逐出空闲时间，达到该时间的连接将成为逐出候选对象。
connection_pool_↪ keep_alive	false	仅在 BE 节点上有效，用于决定是否保持达到最小逐出空闲时间但未到最大生命周期的连接活跃。默认关闭，以减少不必要的资源使用。

- [CommonProperties]

CommonProperties 部分用于填写通用属性。请参阅数据目录概述中【通用属性】部分。

2.14.3.7.4 查询操作

基础查询

```
-- 1. switch to catalog, use database and query
SWITCH mysql_ctl;
USE mysql_db;
SELECT * FROM mysql_tbl LIMIT 10;

-- 2. use mysql database directly
```

```
USE mysql_ctl.mysql_db;
SELECT * FROM mysql_tbl LIMIT 10;

-- 3. use full qualified name to query
SELECT * FROM mysql_ctl.mysql_db.mysql_tbl LIMIT 10;
```

查询优化

谓词下推

JDBC Catalog 访问数据源，本质上是选择一台 BE 节点，通过 JDBC Client 将生成好的 SQL 发送给源端并获取数据，因此性能仅取决于生成的 SQL 在源端的执行效率。Doris 会尽量将谓词条件下推并拼接到生成的 SQL 中。可以通过 EXPLAIN SQL查看到生成的 SQL 语句。

```
EXPLAIN SELECT smallint_u, sum(int_u)
FROM all_types WHERE smallint_u > 10 GROUP BY smallint_u;

...
|   0:VJdbcScanNode(206)
    ↪                                                                                |
|      TABLE: `doris_test`.`all_types`
    ↪                                                                  |
|      QUERY: SELECT `smallint_u`, `int_u` FROM `doris_test`.`all_types` WHERE ((`smallint_u` >
    ↪ 10)) |
|      PREDICATES: (smallint_u[#1] > 10)
    ↪                                                                |
|      final projections: smallint_u[#1], int_u[#3]
    ↪                                                  |
|      final project output tuple id: 1
...                                                              |
```

函数下推

对于谓词条件，Doris 和外部数据源中的语义或行为可能是不一致的。所以 Doris 通过以下参数变量对 JDBC 外表查询中的谓词条件下推进行限制和控制：

> 注：目前 Doris 只支持对 MySQL、Clickhouse、Oracle 三个数据源进行谓词下推。后续将开放更多数据源。

- enable_jdbc_oracle_null_predicate_push_down

  会话变量。默认为 false。即当谓词条件中包含 NULL 值时，该谓词条件不会下推给 Oracle 数据源。因为在 Oracle 21 版本之前，Oracle 不支持 NULL 作为操作符。

  该参数自 2.1.7 和 3.0.3 支持。

- enable_jdbc_cast_predicate_push_down

  会话变量。默认为 false。即当谓词条件中存在显式或隐式 CAST 时，该谓词条件不会下推给 JDBC 数据源。因为 CAST 的行为在不同数据库中不一致，为确保结果正确性，默认不下推 CAST。但用户可以手动验证 CAST 的行为是否一致，如果一致，可以将此参数设置为 true，让更多的谓词下推已获得更好的性能。

  该参数自 2.1.7 和 3.0.3 支持。

- 函数下推黑白名单

  签名相同的函数，在 Doris 和外部数据源中的语义可能是不一致的。Doris 中预定义了一些函数下推的黑白名单：

数据源	黑名单	白名单	说明
MySQL	- DATE_ ↪ TRUNC ↪ - MONEY_ ↪ FORMAT ↪ - NEGTIVE ↪		MySQL 还可以通过 FE 配置项 jdbc ↪ _mysql_ ↪ unsupported ↪ _ ↪ pushdown ↪ _ ↪ functions ↪ 来设置额外的黑名单，如：jdbc_mysql ↪ _ ↪ unsupported ↪ _ ↪ pushdown ↪ _ ↪ functions ↪ ==func1 ↪ ,func2
Clickhouse		- FROM ↪ _ ↪ UNIXTIME ↪ - UNIX ↪ _ ↪ TIMESTAMP ↪	

数据源	黑名单	白名单	说明
Oracle		- NVL- IFNULL ↪	

- 函数改写规则

  Doris 和外部数据源中存在一些行为一致但名称不一样函数。Doris 支持在函数下推时对这些函数进行改写。目前内置了以下改写规则：

数据源	Doris 函数	目的端函数
MySQL	nvl	ifnull
MySQL	to_date	date
Clickhouse	from_unixtime	FROM_UNIXTIME
Clickhouse	unix_timestamp	toUnixTimestamp
Oracle	ifnull	nvl

- 自定义函数下推和改写规则

3.0.7 后续版本中，Doris 支持了更加灵活的函数下推和改写规则，用户可以在 Catalog 属性中设置针对某个 Catalog 的函数下推和改写规则：

```
    create catalog jdbc properties (
    ...
    'function_rules' = '{"pushdown" : {"supported": ["to_date"], "unsupported" : ["abs"]}, "
        ↪ rewrite" : {"to_date" : "date2"}}'
    )
```

```
通过 `function_rules` 可以指定以下规则：

- `pushdown`

    指定函数下推规则，其中 `supported` 和 `unsupported`
        ↪ 数组中分别指定可以下推和不能下推的函数名称。如果同时存在于两个数组中，以 `supported`
        ↪ 优先。

    Doris 会先应用上述系统中预定义的黑白名单，再应用用户指定的黑白名单。

- `rewrite`

    定义函数改写规则，如上述示例中，会将 `to_date` 函数名称改写为 `date2` 并下推。
```

注意，只会改写允许下推的函数。

行数限制

如果在查询中带有 limit 关键字，Doris 会将 limit 下推到数据源，以减少数据传输量。可以通过 EXPLAIN 语句查看生成的 SQL 中是否包含 LIMIT 子句确认。

2.14.3.7.5 写入操作

Doris 支持将数据通过 JDBC 协议写回到对应的数据源。

```
INSERT INTO mysql_table SELECT * FROM internal.doris_db.doris_tbl;
```

2.14.3.7.6 语句透传

适用场景

Doris 支持通过透传的方式，直接在 JDBC 数据源中执行对应的 DDL、DML 语句和查询语句。该功能适用于以下场景：

- 提升复杂查询性能

默认情况下，Doris 查询优化器会对原始 SQL 进行解析，根据一定规则生成需要发送给数据源的 SQL。这个 SQL 通常是单表简单查询，无法生成聚合、关联查询等算子。比如如下查询：

```
SELECT smallint_u, sum(int_u)
FROM all_types WHERE smallint_u > 10 GROUP BY smallint_u;
```

最终生成的 SQL 为：

```
SELECT smallint_u, int_u
FROM all_types
WHERE smallint_u > 10;
```

而聚合操作是在 Doris 中完成的。因此在某些情况下，可能需要从源端通过网络读取大量数据，导致查询效率低下。通过语句透传，可以将原始 SQL 直接透传给数据源，从而使用数据源本身的计算功能完成 SQL 执行，提升查询性能。

- 统一管理

除了查询 SQL 外，语句透传功能也可以透传 DDL 和 DML 语句。因此，可以通过 Doris 直接对源端数据进行库表操作，如创建、删除表，修改表结构等。

透传 SQL

```
SELECT * FROM
QUERY(
  'catalog' = 'mysql_catalog',
```

```
  'query' = 'SELECT smallint_u, sum(int_u) FROM db.all_types WHERE smallint_u > 10 GROUP BY
      ↪ smallint_u;'
);
```

QUERY 表函数有两个参数：

- `catalog`：Catalog 名称，需要按照 Catalog 的名称填写。
- `query`：需要执行的查询语句，并且需要直接使用数据源对应的语法。

透传 DDL 和 DML

```
CALL EXECUTE_STMT("jdbc_catalog", "insert into db1.tbl1 values(1,2), (3, 4)");

CALL EXECUTE_STMT("jdbc_catalog", "delete from db1.tbl1 where k1 = 2");

CALL EXECUTE_STMT("jdbc_catalog", "create table dbl1.tbl2 (k1 int)");
```

`EXECUTE_STMT()` 函数有两个参数：

- 第一个参数：Catalog 名称，目前仅支持 JDBC 类型 Catalog。
- 第二个参数：执行语句，目前仅支持 DDL 和 DML 语句，并且需要直接使用数据源对应的语法。

使用限制

通过 `CALL EXECUTE_STMT()` 命令，Doris 会直接将用户编写的 SQL 语句发送给 Catalog 对应的 JDBC 数据源进行执行。因此，这个操作有如下限制：

- SQL 语句必须是数据源对应的语法，Doris 不会做语法和语义检查。
- SQL 语句中引用的表名建议是全限定名，即 `db.tbl` 这种格式。如果未指定 db，则会使用 JDBC Catalog 的 JDBC URL 中指定的 db 名称。
- SQL 语句中不可引用 JDBC 数据源之外的库表，也不可以引用 Doris 的库表。但可以引用在 JDBC 数据源内的，但是没有同步到 Doris JDBC Catalog 的库表。
- 执行 DML 语句，无法获取插入、更新或删除的行数，只能获取命令是否执行成功。
- 只有对 Catalog 有 LOAD 权限的用户，才能执行`CALL EXECUTE_STMT()`命令。
- 只有对 Catalog 有 SELECT 权限的用户，才能执行 `query()` 表函数。
- 创建 Catalog 时使用的 JDBC 用户，需要在源端，对所执行的语句有相应的权限。
- `query()` 表函数读取到的的数据，数据类型的支持与所查询的 catalog 类型支持的数据类型一致。

2.14.3.7.7 附录

大小写敏感设置

默认情况下，Doris 的库名和表名是大小写敏感的，而列名是大小写不敏感的，并且可以通过参数配置就行修改。同时，一些 JDBC 数据源的库名、表名、列名的大小写敏感规则和 Doris 的规则不一致，这会导致在通过 JDBC Catalog 进行名称映射时，可能会出现名称冲突等问题。这里介绍下如何解决此类问题。

显示名称与查询名称

在 Doris 中，一个对象名称（下面我们以表名称指代）可以分为【显示名称】和【使用名称】。比如对于表名，【显示名称】是指通过 `SHOW TABLES` 看到的名称。而【查询名称】是指通过可以在 `SELECT` 语句总使用的名称。

比如，一个表的真实名称是 `MyTable`。通过修改 FE 的配置项 `lower_case_table_names`，这个表名的【显示名称】和【查询名称】会有所区别：

配置项	说明	真实名称	显示名称	查询名称
`lower_case_table_names=0`	默认配置。使用原始名称存储和显示，查询时大小写敏感	`MyTable`	`MyTable`	查询时大小写敏感，必须使用：`MyTable`

配置项	说明	真实名称	显示名称	查询名称
lower_case_table_names=1	使用小写名称存储和显示，查询时大小写不敏感	MyTable	mytable	查询时大小写不敏感，比如可以使用 MyTable 或 mytable
lower_case_table_names=2	使用原始名称存储和显示，查询时大小写不敏感	MyTable	MyTable	查询时大小写不敏感，比如可以使用 MyTable 或 mytable

JDBC Catalog 名称大小写规则

Doris 本身只可以配置【表名】大小写的规则。而 JDBC Catalog 需要额外处理【库名】和【列名】。因此，我们使用额外的 Catalog 属性 `lower_case_meta_names` 来配合 `lower_case_table_names` 一起使用。

配置项	说明
`lower_case_meta_names`	创建 Catalog 时通过 `properties` 指定，作用于该 Catalog。默认为 `false`。当设置为 `true` 时，Doris 会将库、表、列名全部转换为小写存储并显示，查询时需使用 Doris 中的小写名称。

配置项	说明
lower_case_table_names	FE 配置项，在 fe.conf 中配置，作用范围为整个集群。默认为 0。

> 注：若 lower_case_meta_names = true，则此时不会参考 lower_case_table_names，一律将库名、表名、列名都转换为小写。

根据 lower_case_meta_names（true/false）和 lower_case_table_names（0/1/2）的不同组合，库名、表名、列名在 存储时和 查询时的表现方式如下表所示（“原始”表示保持外部数据源的大小写，“小写”表示自动转换为小写，“任意大小写”表示查询时可随意使用大小写）：

lower_case_table_names & lower_case_meta_names	Database 显示名称	Table 显示名称	Column 显示名称	Database 查询名称	Table 查询名称	Column 查询名称
0 & false	原始	原始	原始	原始	原始	任意大小写
0 & true	小写	小写	小写	小写	小写	任意大小写
1 & false	原始	小写	原始	原始	任意大小写	任意大小写
1 & true	小写	小写	小写	小写	任意大小写	任意大小写
2 & false	原始	原始	原始	原始	任意大小写	任意大小写
2 & true	小写	小写	小写	小写	任意大小写	任意大小写

大小写冲突检查

在通过 JDBC Catalog 进行名称映射时，可能会出现名称冲突。比如源端列名称大小写敏感，存在 ID 和 id 两列。如果设置了 lower_case_meta_names = true，则这两个列在映射成小写后会产生冲突。Doris 会根据以下规则进行冲突检查：

- 任意场景下，Doris 都会检查【列名】的大小写冲突（如 id 与 ID 是否同时存在）。
- 当 lower_case_meta_names = true 时，Doris 会检查库名、表名、列名是否存在大小写冲突（如 DORIS 与 doris 同时存在）。
- 当 lower_case_meta_names = false 且 lower_case_table_names 为 1 或 2 时，Doris 会检查【表名】是否冲突（如 orders 与 ORDERS）。
- 当 lower_case_table_names = 0 时，库名、表名均大小写敏感，无需额外转换。

大小写冲突的解决方案

当出现冲突时，Doris 会报错，须通过以下方式解决冲突。

对于存在仅大小写不同而重复的库、表、列（例如 DORIS 与 doris）导致 Doris 无法正常区分的情况，可通过为 Catalog 设置 meta_names_mapping 来指定手动映射，从而解决冲突。

配置示例：

```
{
  "databases": [
    {
      "remoteDatabase": "DORIS",
      "mapping": "doris_1"
    },
    {
      "remoteDatabase": "doris",
      "mapping": "doris_2"
    }
  ],
  "tables": [
    {
      "remoteDatabase": "DORIS",
      "remoteTable": "DORIS",
      "mapping": "doris_1"
    },
    {
      "remoteDatabase": "DORIS",
      "remoteTable": "doris",
      "mapping": "doris_2"
    }
  ],
  "columns": [
```

```
    {
      "remoteDatabase": "DORIS",
      "remoteTable": "DORIS",
      "remoteColumn": "DORIS",
      "mapping": "doris_1"
    },
    {
      "remoteDatabase": "DORIS",
      "remoteTable": "DORIS",
      "remoteColumn": "doris",
      "mapping": "doris_2"
    }
  ]
}
```

驱动包安全性

驱动包由用户上传到 Doris 集群，因此存在一定安全隐患。用户可以通过以下方式进行安全加固。

1. Doris 认为 `jdbc_drivers_dir` 目录下的所有驱动包都是安全的，不会对其进行路径检查。管理员需自行管理这个目录下的文件以确保其安全性。
2. 如使用本地路径或 HTTP 路径指定的驱动包，Doris 会做如下检查：

- 通过 FE 配置项 `jdbc_driver_secure_path` 来控制允许的驱动包路径。该配置项可配置多个路径，以分号分隔。当配置了该项时，Doris 会检查 `driver_url` 的路径前缀是否在 `jdbc_driver_secure_path` 中，如果不在其中，则会拒绝创建。

  示例：

  如配置 `jdbc_driver_secure_path = "file:///path/to/jdbc_drivers;http://path/to/jdbc_drivers"`，则只允许以 `file:///path/to/jdbc_drivers` 或 `http://path/to/jdbc_drivers` 开头的驱动包路径。

- 此参数默认为 `*`。如果为空或者 `*`，表示允许所有路径的 Jar 包。

3. 在创建数据目录时，可以通过 `checksum` 参数来指定驱动包的校验和。Doris 会在加载驱动包后，对驱动包进行校验，如果校验失败，则会拒绝创建。

连接池清理

在 Doris 中，每个 FE 和 BE 节点都会维护一个连接池，这样可以避免频繁地打开和关闭单独的数据源连接。连接池中的每个连接都可以用来与数据源建立连接并执行查询。任务完成后，这些连接会被归还到池中以便重复使用，这不仅提高了性能，还减少了建立连接时的系统开销，并帮助防止达到数据源的连接数上限。

可以根据实际情况调整连接池的大小，以便更好地适应不同工作负载。通常情况下，连接池的最小连接数应该设置为 1，以确保在启用保活机制时至少有一个连接处于活跃状态。连接池的最大连接数应该设置为一个合理的值，以避免过多的连接占用资源。

同时为了避免在 BE 上累积过多的未使用的连接池缓存，可以通过设置 BE 的 `jdbc_connection_pool_cache_clear_time_sec` 参数来指定清理缓存的时间间隔。默认值为 28800 秒（8 小时），此间隔过后，BE 将强制清理所有超过该时间未使用的连接池缓存。

凭证更新

使用 JDBC Catalog 连接外部数据源时，需谨慎更新数据库凭证。

Doris 通过连接池维持活跃连接以快速响应查询。但凭证变更后，连接池可能会继续使用旧凭证尝试建立新连接并失败。由于系统试图保持一定数量的活跃连接，这种错误尝试会重复执行，且在某些数据库系统中，频繁的失败可能导致账户被锁定。

建议在必须更改凭证时，同步更新 Doris JDBC Catalog 配置，并重启 Doris 集群，以确保所有节点使用最新凭证，防止连接失败和潜在的账户锁定。

可能遇到的账户锁定如下：

```
MySQL: account is locked
Oracle: ORA-28000: the account is locked
SQL Server: Login is locked out
```

连接池问题排查

1. HikariPool 获取连接超时错误 `Connection is not available, request timed out after 5000ms`

   - 可能的原因
     - 原因 1：网络问题（例如，服务器不可达）
     - 原因 2：身份认证问题，例如无效的用户名或密码
     - 原因 3：网络延迟过高，导致创建连接超过 5 秒超时时间
     - 原因 4：并发查询过多，超过了连接池配置的最大连接数
   - 解决方案
     - 如果只有 `Connection is not available, request timed out after 5000ms` 这一类错误，请检查原因 3 和 原因 4：
     - 检查是否存在网络延迟过高或资源耗尽的情况。
     - 调大连接池的最大连接数：

```
    ALTER CATALOG catalog_name SET PROPERTIES ('connection_pool_max_size' = '100');
```

```
  * 调大连接超时时间：
```

```
    ALTER CATALOG catalog_name SET PROPERTIES ('connection_pool_max_wait_time' = '10000');
```

```
* 如果除了 `Connection is not available, request timed out after 5000ms` 之外还有其他错误信息，
    ↪ 请检查这些附加错误：

  * 网络问题（例如，服务器不可达）可能导致连接失败。请检查网络连接是否正常。

  * 身份认证问题（例如，用户名或密码无效）也可能导致连接失败。请检查配置中使用的数据库凭据，
    ↪ 确保用户名和密码正确无误。

  * 根据具体错误信息，调查与网络、数据库或身份认证相关的问题，找出根本原因。
```

2.14.3.8 MySQL JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 MySQL 数据库。本文档介绍如何配置 MySQL 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.8.1 使用须知

要连接到 MySQL 数据库，您需要

- MySQL 5.7, 8.0 或更高版本。
- MySQL 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 MySQL JDBC 驱动程序。推荐使用 MySQL Connector/J 8.0.31 及以上版本。
- Doris 每个 FE 和 BE 节点和 MySQL 服务器之间的网络连接，默认端口为 3306。

2.14.3.8.2 连接 MySQL

```
CREATE CATALOG mysql_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwd',
    'jdbc_url' = 'jdbc:mysql://host:3306',
    'driver_url' = 'mysql-connector-j-8.3.0.jar',
    'driver_class' = 'com.mysql.cj.jdbc.Driver'
);
```

`jdbc_url` 定义要传递给 MySQL JDBC 驱动程序的连接信息和参数。支持的 URL 的参数可在 MySQL 开发指南 中找到。

连接安全

如果用户使用数据源上安装的全局信任证书配置了 TLS，则可以通过将参数附加到在 `jdbc_url` 属性中设置的 JDBC 连接字符串来启用集群和数据源之间的 TLS。

例如，对于 MySQL Connector/J 8.0 版，使用 `sslMode` 参数通过 TLS 保护连接。默认情况下，该参数设置为 PREFERRED，如果服务器启用，它可以保护连接。还可以将此参数设置为 REQUIRED，如果未建立 TLS，则会导致连接失败。

可以在通过在 `jdbc_url` 中添加 sslMode 参数来配置它：

```
'jdbc_url' = 'jdbc:mysql://host:3306/?sslMode=REQUIRED'
```

有关 TLS 配置选项的更多信息，请参阅 MySQL JDBC 安全文档。

2.14.3.8.3 层级映射

映射 MySQL 时，Doris 的一个 Database 对应于 MySQL 中的一个 Database。而 Doris 的 Database 下的 Table 则对应于 MySQL 中，该 Database 下的 Tables。即映射关系如下：

Doris	MySQL
Catalog	MySQL Server
Database	Database
Table	Table

2.14.3.8.4 列类型映射

MySQL Type	Doris Type	Comment
boolean	tinyint	
tinyint	tinyint	
smallint	smallint	
mediumint	int	
int	int	
bigint	bigint	
unsigned tinyint	smallint	Doris 不支持 unsigned 数据类型，所以 unsigned 数据类型会被映射为 Doris 对应大一个数量级的数据类型。
unsigned mediumint	int	同上。
unsigned int	bigint	同上。
unsigned bigint	largeint	同上。
float	float	
double	double	
decimal(P, S)	decimal(P, S)	
unsigned decimal(P, S)	decimal(P + 1, S) / string	如果超过 Doris 支持的最大精度，则会使用 String 承接。注意在此类型被映射为 String 时，只能支持查询，不能对 MySQL 进行写入操作。
date	date	
timestamp(S)	datetime(S)	
datetime(S)	datetime(S)	
year	smallint	Doris 不支持 year 类型，year 类型会被映射为 smallint。
time	string	Doris 不支持 time 类型，time 类型会被映射为 string。
char	char	
varchar	varchar	
json	string	为了更好的读取与计算性能均衡，Doris 会将 json 类型映射为 string 类型。
set	string	
enum	string	
bit	boolean / string	Doris 不支持 bit 类型，bit 类型会在 bit(1) 时被映射为 boolean，其他情况下映射为 string。
tinytext, text, mediumtext, longtext	string	
blob, mediumblob, longblob, tinyblob	string	
binary, varbinary	string	

MySQL Type	Doris Type	Comment
other	UNSUPPORTED	

2.14.3.8.5 附录

时区问题

通过 JDBC Catalog 访问数据时，BE 的 JNI 部分使用 JVM 时区。JVM 时区默认为 BE 部署机器的时区，这会影响 JDBC 读取数据时的时区转换。为了确保时区一致性，建议在 `be.conf` 的 `JAVA_OPTS` 中设置 JVM 时区与 Doris 会话变量的 `time_zone` 一致。

读取 MySQL 的 timestamp 类型时，请在 JDBC URL 中添加参数：`connectionTimeZone=LOCAL` 和 `forceConnectionTimeZoneToSession=true`。这些参数适用于 MySQL Connector/J 8 以上版本，可确保读取的时间为 Doris BE JVM 时区，而非 MySQL 服务端的时区。

2.14.3.8.6 常见问题

连接异常排查

- Communications link failure The last packet successfully received from the server was 7 milliseconds ago.
- 原因：
  - 网络问题：
    - 网络不稳定或连接中断。
    - 客户端和服务器之间的网络延迟过高。
  - MySQL 服务器设置
    - MySQL 服务器可能配置了连接超时参数，例如 `wait_timeout` 或 `interactive_timeout`，导致连接超时被关闭。
  - 防火墙设置
    - 防火墙规则可能阻止了客户端与服务器之间的通信。
  - 连接池设置
    - 连接池中的配置 `connection_pool_max_life_time` 可能导致连接被关闭或回收，或者未及时探活
  - 服务器资源问题
    - MySQL 服务器可能资源不足，无法处理新的连接请求。
  - 客户端配置
    - 客户端 JDBC 驱动配置错误，例如 `autoReconnect` 参数未设置或设置不当。
- 解决方案
  - 检查网络连接：
    - 确认客户端和服务器之间的网络连接稳定，避免网络延迟过高。

- 检查 MySQL 服务器配置：
  * 查看并调整 MySQL 服务器的 wait_timeout 和 interactive_timeout 参数，确保它们设置合理。
- 检查防火墙配置：
  * 确认防火墙规则允许客户端与服务器之间的通信。
- 调整连接池设置：
  * 检查并调整连接池的配置参数 connection_pool_max_life_time，确保小于 MySQL 的 wait_timeout 和 interactive_timeout 参数并大于执行时间最长的 SQL
- 监控服务器资源：
  * 监控 MySQL 服务器的资源使用情况，确保有足够的资源处理连接请求。
- 优化客户端配置：
  * 确认 JDBC 驱动的配置参数正确，例如 autoReconnect=true，确保连接能在中断后自动重连。

- java.io.EOFException MESSAGE: Can not read response from server. Expected to read 819 bytes, read 686 bytes before connection was unexpectedly lost.
- 原因：连接被 MySQL Kill 或者 MySQL 宕机
- 解决：检查 MySQL 是否有主动 kill 连接的机制，或者是否因为查询过大查崩 MySQL

其他问题

1. 读写 MySQL 的 emoji 表情出现乱码

Doris 查询 MySQL 时，由于 MySQL 之中默认的 utf8 编码为 utf8mb3，无法表示需要 4 字节编码的 emoji 表情。这里需要将 MySQL 的编码修改为 utf8mb4，以支持 4 字节编码。

可全局修改配置项

```
修改 mysql 目录下的 my.ini 文件（linux 系统为 etc 目录下的 my.cnf 文件）
[client]
default-character-set=utf8mb4

[mysql]
设置 mysql 默认字符集
default-character-set=utf8mb4

[mysqld]
设置 mysql 字符集服务器
character-set-server=utf8mb4
collation-server=utf8mb4_unicode_ci
init_connect='SET NAMES utf8mb4

修改对应表与列的类型
ALTER TABLE table_name MODIFY colum_name VARCHAR(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_
    ↪ unicode_ci;
ALTER TABLE table_name CHARSET=utf8mb4;
SET NAMES utf8mb4
```

2. 读取 MySQL DATE/DATETIME 类型出现异常

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.6)[INTERNAL_ERROR]
    ↪ UdfRuntimeException: get next block failed:
CAUSED BY: SQLException: Zero date value prohibited
CAUSED BY: DataReadException: Zero date value prohibited
```

因为 JDBC 中对于非法的 DATE/DATETIME 默认处理为抛出异常。可以通过 URL 参数 zeroDateTimeBehavior 控制该行为。可选参数为：exception,convertToNull,round, 分别为：异常报错；转为 NULL 值；转为 "0001-01-01 ↪ 00:00:00"

需要在创建 Catalog 的 jdbc_url 把 JDBC 连接串最后增加 zeroDateTimeBehavior=convertToNull , 如 "jdbc_url ↪ " = "jdbc:mysql://127.0.0.1:3306/test?zeroDateTimeBehavior=convertToNull" 这种情况下，JDBC 会把 0000-00-00 或者 0000-00-00 00:00:00 转换成 null，然后 Doris 会把当前 Catalog 的所有 Date/DateTime 类型的列按照可空类型处理，这样就可以正常读取了。

3. 读取 MySQL Catalog 或其他 JDBC Catalog 时，出现加载类失败，如 failed to load driver class com.mysql ↪ .cj.jdbc.driver in either of hikariconfig class loader

这是因为在创建 Catalog 时，填写的 driver_class 不正确，需要正确填写，如上方例子为大小写问题，应填写为 'driver_class' = 'com.mysql.cj.jdbc.Driver'

4. 读取 MySQL 出现通信链路异常

```
ERROR 1105 (HY000): errCode = 2, detailMessage = PoolInitializationException: Failed to
    ↪ initialize pool: Communications link failure

The last packet successfully received from the server was 7 milliseconds ago.  The last packet
    ↪ sent successfully to the server was 4 milliseconds ago.
CAUSED BY: CommunicationsException: Communications link failure

The last packet successfully received from the server was 7 milliseconds ago.  The last packet
    ↪ sent successfully to the server was 4 milliseconds ago.
CAUSED BY: SSLHandshakeException
```

可查看 be 的 be.out 日志，如果包含以下信息：

```
WARN: Establishing SSL connection without server's identity verification is not recommended.
According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+ requirements SSL connection must be established
    ↪ by default if explicit option isn't set.
For compliance with existing applications not using SSL the verifyServerCertificate property is
    ↪  set to 'false'.
You need either to explicitly disable SSL by setting useSSL=false, or set useSSL=true and
    ↪ provide truststore for server certificate verification.
```

可在 jdbc_url 中增加 useSSL=false，如 'jdbc_url' = 'jdbc:mysql://127.0.0.1:3306/test?useSSL=false'。

- 查询 MySQL 大数据量时，如果查询偶尔能够成功，偶尔会报如下错误，且出现该错误时 MySQL 的连接被全部断开，无法连接到 MySQL Server，过段时间后 MySQL 又恢复正常，但是之前的连接都没了：

```
ERROR 1105 (HY000): errCode = 2, detailMessage = [INTERNAL_ERROR]UdfRuntimeException: JDBC
    ↪ executor sql has error:
CAUSED BY: CommunicationsException: Communications link failure
The last packet successfully received from the server was 4,446 milliseconds ago. The last
    ↪ packet sent successfully to the server was 4,446 milliseconds ago.
```

出现上述现象时，可能是 MySQL Server 自身的内存或 CPU 资源被耗尽导致 MySQL 服务不可用，可以尝试增大 MySQL Server 的内存或 CPU 配置。

- 查询 MySQL 的过程中，如果发现和在 MySQL 库的查询结果不一致的情况

首先要先排查下查询字段中是字符串否存在有大小写情况。比如，Table 中有一个字段 c_1 中有 "aaa" 和 "AAA" 两条数据，如果在初始化 MySQL 数据库时未指定区分字符串大小写，那么 MySQL 默认是不区分字符串大小写的，但是在 Doris 中是严格区分大小写的，所以会出现以下情况：

```
MySQL 行为：
select count(c_1) from table where c_1 = "aaa"; 未区分字符串大小，所以结果为：2

Doris 行为：
select count(c_1) from table where c_1 = "aaa"; 严格区分字符串大小，所以结果为：1
```

如果出现上述现象，那么需要按照需求来调整，方式如下：

- 在 MySQL 中查询时添加 “BINARY” 关键字来强制区分大小写：`select count(c_1)from table where ↪ BINARY c_1 = "aaa";`
- 或者在 MySQL 中建表时候指定：`CREATE TABLE table (c_1 VARCHAR(255)CHARACTER SET binary);`
- 或者在初始化 MySQL 数据库时指定校对规则来区分大小写：

```
[mysqld]
character-set-server=utf8
collation-server=utf8_bin
[client]
default-character-set=utf8
[mysql]
default-character-set=utf8
```

- 查询 MySQL 的时候，出现长时间卡住没有返回结果，或着卡住很长时间并且 fe.warn.log 中出现出现大量 write lock 日志。

可以尝试在 URL 添加 socketTimeout，例如：`jdbc:mysql://host:port/database?socketTimeout=30000`，防止 JDBC 客户端在被 MySQL 关闭连接后无限等待。

- 在使用 MySQL Catalog 的过程中发现 FE 的 JVM 内存或 Threads 数持续增长不减少，并可能同时出现 Forward to master connection timed out 报错

打印 FE 线程堆栈 `jstack fe_pid > fe.js`，如果出现大量 `mysql-cj-abandoned-connection-cleanup` 线程，说明是 MySQL JDBC 驱动的问题。

按照如下方式处理：

- 升级 MySQL JDBC 驱动到 8.0.31 及以上版本
- 在 FE 和 BE conf 文件的 JAVA_OPTS 中增加 `-Dcom.mysql.cj.disableAbandonedConnectionCleanup=true` 参数，禁用 MySQL JDBC 驱动的连接清理功能，并重启集群

注意：如果 Doris 的版本在 2.0.13 及以上，或 2.1.5 及以上，则无需增加该参数，因为 Doris 已经默认禁用了 MySQL JDBC 驱动的连接清理功能。只需更换 MySQL JDBC 驱动版本即可。但是需要重启 Doris 集群来清理掉之前的泄漏线程。

2.14.3.9 PostgreSQL JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 PostgreSQL 数据库。本文档介绍如何配置 PostgreSQL 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.9.1 使用须知

要连接到 PostgreSQL 数据库，您需要

- PostgreSQL 11.x 或更高版本
- PostgreSQL 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 PostgreSQL JDBC 驱动程序。推荐使用 PostgreSQL JDBC Driver 42.5.x 及以上版本。
- Doris 每个 FE 和 BE 节点和 PostgreSQL 服务器之间的网络连接，默认端口为 5432。

2.14.3.9.2 连接 PostgreSQL

```
CREATE CATALOG postgresql_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwd',
    'jdbc_url' = 'jdbc:postgresql://host:5432/postgres',
    'driver_url' = 'postgresql-42.5.6.jar',
    'driver_class' = 'org.postgresql.Driver'
);
```

`jdbc_url` 定义要传递给 PostgreSQL JDBC 驱动程序的连接信息和参数。支持的 URL 的参数可在 PostgreSQL JDBC 驱动程序文档 中找到。

连接安全

如果您使用数据源上安装的全局信任证书配置了 TLS，则可以通过将参数附加到在 jdbc_url 属性中设置的 JDBC 连接字符串来启用集群和数据源之间的 TLS。

例如，对于版本 42 的 PostgreSQL JDBC 驱动程序，通过将 ssl=true 参数添加到 jdbc_url 配置属性中启用 TLS：

```
"jdbc_url"="jdbc:postgresql://example.net:5432/database?ssl=true"
```

有关 TLS 配置选项的更多信息，请参阅 PostgreSQL JDBC 驱动程序文档。

2.14.3.9.3 层级映射

映射 PostgreSQL 时，Doris 的一个 Database 对应于 PostgreSQL 中指定 database 下的一个 Schema（如示例中 `jdbc_url` 参数中 `postgres` 下的 schemas）。而 Doris 的 Database 下的 Table 则对应于 PostgreSQL 中，该 Schema 下的 Tables。即映射关系如下：

Doris	PostgreSQL
Catalog	Database
Database	Schema
Table	Table

2.14.3.9.4 列类型映射

PostgreSQL Type	Doris Type	
boolean	boolean	
smallint/int2	smallint	
integer/int4	int	
bigint/int8	bigint	
decimal/numeric	decimal(P, S) / string	无精度 numeric 会被映射为 string 类型，进行数值计算时需要先转换为 decimal 类型，且不支持回写。
real/float4	float	
double	double	
smallserial	smallint	
serial	int	
bigserial	bigint	
char(N)	char(N)	
varchar/text	string	
timestamp(S)/timestampz(S)	datetime(S)	
date	date	
json/jsonb	string	为了更好的读取与计算性能均衡，Doris 会将 JSON 类型映射为 STRING 类型。
time	string	Doris 不支持 time 类型，time 类型会被映射为 string。
interval	string	
point/line/lseg/box/path/polygon/circle	string	
cidr/inet/macaddr	string	

PostgreSQL Type	Doris Type	
uuid	string	
bit	boolean / string	Doris 不支持 bit 类型，bit 类型会在 bit(1) 时被映射为 boolean，其他情况下映射为 string。
other	UNSUPPORTED	

2.14.3.9.5 附录

时区问题

由于 Doris 不支持带时区的时间戳类型，所以在读取 PostgreSQL 的 timestampz 类型时，Doris 会将其映射为 DATETIME 类型，且会在读取时转换成本地时区的时间。

且由于在 JDBC 类型 Catalog 读取数据时，BE 的 Java 部分使用 JVM 时区。JVM 时区默认为 BE 部署机器的时区，这会影响 JDBC 读取数据时的时区转换。

为了确保时区一致性，建议在 `be.conf` 的 `JAVA_OPTS` 中设置 JVM 时区与 Doris session 的 `time_zone` 一致。

2.14.3.10 Oracle JDBC Catalog

Apache Doris JDBC Catalog 支持通过标准 JDBC 接口连接 Oracle 数据库。本文档介绍如何配置 Oracle 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.10.1 使用须知

要连接到 Oracle 数据库，您需要

- Oracle 19c, 18c, 12c, 11g 或 10g。
- Oracle 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载 Ojdbc8 及以上版本的 Oracle JDBC 驱动程序。
- Apache Doris 每个 FE 和 BE 节点和 Oracle 服务器之间的网络连接，默认端口为 1521；如果 Oracle RAC 启用 ONS，还需开通 6200 端口。

2.14.3.10.2 连接 Oracle

```
CREATE CATALOG oracle_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password'='pwd',
    'jdbc_url' = 'jdbc:oracle:thin:@example.net:1521:orcl',
    'driver_url' = 'ojdbc8.jar',
    'driver_class' = 'oracle.jdbc.driver.OracleDriver'
)
```

`jdbc_url` 定义要传递给 JDBC 驱动程序的连接信息和参数。使用 Oracle JDBC Thin 驱动程序时，URL 的语法可能会有所不同，具体取决于您的 Oracle 配置。例如，如果您要连接到 Oracle SID 或 Oracle 服务名称，则连接 URL 会有所不同。有关更多信息，请参阅 Oracle 数据库 JDBC 驱动程序文档。以上示例 URL 连接到名为`orcl`的 Oracle SID。

2.14.3.10.3 层级映射

映射 Oracle 时，Apache Doris 的一个 Database 对应于 Oracle 中的一个 User。而 Apache Doris 的 Database 下的 Table 则对应于 Oracle 中，该 User 下的有权限访问的 Table。即映射关系如下：

Doris	Oracle
Catalog	Database
Database	User
Table	Table

2.14.3.10.4 列类型映射

Oracle Type	Doris Type	Comment
number(P) / number(P, 0)	tinyint/smallint/int/bigint/largeint	Doris 会根据 P 的大小来选择对应的类型：P < 3：TINYINT; P < 5：SMALLINT; P < 10：INT; P < 19：BIGINT; P > 19：LARGEINT
number(P, S), 如果 (S > 0 && P > S)	decimal(P, S)	
number(P, S), 如果 (S > 0 && P < S)	decimal(S, S)	
number(P, S), 如果 (S < 0)	tinyint/smallint/int/bigint/largeint	S < 0 的情况下，Doris 会将 P 设置为 `P + |S|`，并进行和 `number(P)/ number(P, 0)` 一样的映射
number		Doris 目前不支持未指定 P 和 S 的 number 类型
decimal(P, S)	decimal(P, S)	
float/real	double	
date	date	
timestamp	datetime(S)	
char/nchar	string	
varchar2/nvarchar2	string	
long/raw/long raw/internal	string	
other	UNSUPPORTED	

2.14.3.10.5 常见问题

1. 创建或查询 Oracle Catalog 时出现 `ONS configuration failed`

在 be.conf 的 JAVA_OPTS 增加 -Doracle.jdbc.fanEnabled=false 并且升级 driver 到 https://repo1.maven.org/maven2/com/oracle/database/jdbc/ojdbc8/19.23.0.0/ojdbc8-19.23.0.0.jar

2. 创建或查询 Oracle Catalog 时出现 `Non supported character set (add orai18n.jar in your classpath) ↪ : ZHS16GBK` 异常

下载 orai18n.jar 并放到每个 FE 和 BE 的目录下的 `custom_lib/` 目录下（如不存在，手动创建即可）并重启每个 FE 和 BE。

2.14.3.11 SQL Server JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 SQL Server 数据库。本文档介绍如何配置 SQL Server 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.11.1 使用须知

要连接到 SQL Server 数据库，您需要

- SQL Server 2012 或更高版本，或 Azure SQL 数据库。
- SQL Server 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 SQL Server JDBC 驱动程序。推荐使用 SQL Server JDBC Driver 11.2.x 及以上版本。
- Doris 每个 FE 和 BE 节点和 SQL Server 服务器之间的网络连接，默认端口为 1433。

2.14.3.11.2 连接 SQL Server

```
CREATE CATALOG sqlserver_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwddd',
    'jdbc_url' = 'jdbc:sqlserver://<host>:<port>;databaseName=<databaseName>;encrypt=false',
    'driver_url' = 'mssql-jdbc-11.2.3.jre8.jar',
    'driver_class' = 'com.microsoft.sqlserver.jdbc.SQLServerDriver'
)
```

`jdbc_url` 定义要传递给 SQL Server JDBC 驱动程序的连接信息和参数。SQL Server JDBC 驱动程序文档中提供了 URL 支持的参数。

连接安全

JDBC 驱动程序以及连接器自动使用传输层安全性 (TLS) 加密和证书验证。这需要在 SQL Server 数据库主机上配置合适的 TLS 证书。

如果您没有建立必要的配置，您可以使用 encrypt 属性禁用连接字符串中的加密：

```
'jdbc_url' = 'jdbc:sqlserver://<host>:<port>;databaseName=<databaseName>;encrypt=false'
```

SQL Server JDBC 驱动程序文档的 TLS 部分详细介绍了 `trustServerCertificate`、`hostNameInCertificate`、`trustStore` 和 `trustStorePassword` 等其他参数。

2.14.3.11.3 层级映射

映射 SQLServer 时，Doris 的一个 Database 对应于 SQL Server 中指定 Database（`jdbc_url` 参数中的 `<databaseName>`）下的一个 Schema。而 Doris 的 Database 下的 Table 则对应于 SQLServer 中，Schema 下的 Tables。即映射关系如下：

Doris	SQL Server
Catalog	Database

Doris	SQL Server
Database	Schema
Table	Table

2.14.3.11.4 列类型映射

SQL Server Type	Doris Type	Comment
bit	boolean	
tinyint	smallint	SQLServer 的 tinyint 是无符号数，所以映射为 Doris 的 smallint
smallint	smallint	
int	int	
bigint	bigint	
real	float	
float	double	
money	decimal(19,4)	
smallmoney	decimal(10,4)	
decimal(P, S)/numeric(P, S)	decimal(P, S)	
date	date	
datetime/datetime2/smalldatetime	datetime(S)	
char/varchar/text/nchar/nvarchar/ntext	string	
time/datetimeoffset	string	
timestamp	string	读取二进制数据的十六进制显示，无实际意义
other	UNSUPPORTED	

2.14.3.11.5 常见问题

1. 连接 SQL Server 出现证书认证异常

```
SQLServerException: The driver could not establish a secure connection to SQL Server by using
    ↪ Secure Sockets Layer (SSL) encryption.
Error: "sun.security.validator.ValidatorException: PKIX path building failed: sun.security.
    ↪ provider.certpath.SunCertPathBuilderException:
unable to find valid certification path to requested target". ClientConnectionId:a92f3817-e8e6
    ↪ -4311-bc21-7c66
```

可在创建 Catalog 的 `jdbc_url` 把 JDBC 连接串最后增加 `encrypt=false` ，如 `"jdbc_url" = "jdbc:sqlserver
↪ ://127.0.0.1:1433;DataBaseName=doris_test;encrypt=false"`

2. 连接 SQL Server 出现 TLS 异常

```
The server selected protocol version TLS10 is not accepted by client preferences [TLS13, TLS12
    ↪ ]
```

这是因为 SQL Server 与 JDBC 客户端之间的 TLS 协议版本不匹配。连接的 SQL Server 仅支持 TLS 1.0，而 JDBC 客户端所在 JAVA 环境默认禁用了 TLS 1.0。

解决方式如下：

1. 在 SQL Server 上启用 TLS 1.2。参考：SQL Server TLS 1.2 支持
2. 启用 JDK 的 TLS 1.0。

```
vim ${JAVA_HOME}/lib/security/java.security
#找到这段
jdk.tls.disabledAlgorithms=SSLv3, TLSv1, TLSv1.1, RC4, DES, MD5withRSA, \
DH keySize < 1024, EC keySize < 224, 3DES_EDE_CBC, anon, NULL, \
include jdk.disabled.namedCurves

#删掉其中的 TLSv1, TLSv1.1 ，改成下面这样即可
jdk.tls.disabledAlgorithms=SSLv3, RC4, DES, MD5withRSA, \
DH keySize < 1024, EC keySize < 224, anon, NULL, \
include jdk.disabled.namedCurves
```

2.14.3.12 IBM Db2 JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 IBM Db2 数据库。本文档介绍如何配置 IBM Db2 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.12.1 使用须知

要连接到 IBM Db2 数据库，您需要：

- IBM Db2 11.5.x 或更高版本
- IBM Db2 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 IBM Db2 驱动程序。推荐使用 IBM db2 jcc 11.5.8.0 版本。
- Doris 每个 FE 和 BE 节点和 IBM Db2 服务器之间的网络连接，默认端口为 51000。

2.14.3.12.2 连接 IBM Db2

```
CREATE CATALOG db2_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'USERNAME',
    'password' = 'PASSWORD',
    'jdbc_url' = 'jdbc:db2://host:port/database',
    'driver_url' = 'jcc-11.5.8.0.jar',
    'driver_class' = 'com.ibm.db2.jcc.DB2Driver'
)
```

`jdbc_url` 定义要传递给 IBM Db2 驱动程序的连接信息和参数。支持的 URL 的参数可在 Db2 JDBC 驱动程序文档中找到。

2.14.3.12.3 层级映射

映射 IBM Db2 时，Doris 的 Database 对应于 DB2 中指定 DataBase（jdbc_url 参数中的 “database”）下的一个 Schema。而 Doris 的 Database 下的 Table 则对应于 DB2 中 Schema 下的 Tables。即映射关系如下：

Doris	IBM Db2
Catalog	DataBase
Database	Schema
Table	Table

2.14.3.12.4 类型映射

IBM Db2 Type	Doris Type	Comment
smallint	smallint	
int	int	
bigint	bigint	
double	double	
double precision	double	
float	double	
real	float	
decimal(P, S)	decimal(P, S)	
decfloat(P, S)	decimal(P, S)	
date	date	
timestamp(S)	datetime(S)	
char(N)	char(N)	
varchar(N)	varchar(N)	
long varchar(N)	varchar(N)	
vargraphic	string	
long vargraphic	string	
time	string	
clob	string	
xml	string	
other	UNSUPPORTED	

2.14.3.12.5 常见问题

1. 通过 JDBC Catalog 读取 IBM Db2 数据时出现 `Invalid operation: result set is closed. ERRORCODE=-4470` 异常

在创建 IBM Db2 Catalog 的 jdbc_url 连接串中添加连接参数：`allowNextOnExhaustedResultSet=1;resultSetHoldability=1;`。如：`jdbc:db2://host:port/database:allowNextOnExhaustedResultSet=1;resultSetHoldability=1;`。

2. Caught java.io.CharConversionException

这可能是因为字符集问题，可以在 be.conf 的 JAVA_OPTS 添加配置 -Ddb2.jcc.charsetDecoderEncoder=3 ↪ ，并重启 BE 尝试解决，可以尝试 1、2 等不同取值。具体可参阅：https://www.ibm.com/docs/en/content-collector/4.0.1?topic=manager-jdbc-throws-javaiocharconversionexception

2.14.3.13 Clickhouse JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 ClickHouse 数据库。本文档介绍如何配置 ClickHouse 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.13.1 使用须知

要连接到 ClickHouse 数据库，您需要

- ClickHouse 23.x 或更高版本（低于此版本未经充分测试）。
- ClickHouse 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 ClickHouse JDBC 驱动程序。推荐使用 ClickHouse JDBC Driver 0.4.6 版本。
- Doris 每个 FE 和 BE 节点和 ClickHouse 服务器之间的网络连接，默认端口为 8123。

2.14.3.13.2 连接 ClickHouse

```
CREATE CATALOG clickhouse PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwd',
    'jdbc_url' = 'jdbc:clickhouse://example.net:8123/',
    'driver_url' = 'clickhouse-jdbc-0.4.6-all.jar',
    'driver_class' = 'com.clickhouse.jdbc.ClickHouseDriver'
)
```

jdbc_url 定义要传递给 ClickHouse JDBC 驱动程序的连接信息和参数。支持的 URL 的参数可在 ClickHouse JDBC 驱动配置 中找到。

连接安全

如果您使用数据源上安装的全局信任证书配置了 TLS，则可以通过将参数附加到在 jdbc_url 属性中设置的 JDBC 连接字符串来启用集群和数据源之间的 TLS。

例如，通过将 ssl=true 参数添加到 jdbc_url 配置属性来启用 TLS：

```
'jdbc_url' = 'jdbc:clickhouse://example.net:8123/db?ssl=true'
```

有关 TLS 配置选项的更多信息，请参阅 Clickhouse JDBC 驱动程序文档 SSL 配置部分

2.14.3.13.3 层级映射

映射 ClickHouse 时，Doris 的一个 Database 对应于 ClickHouse 中的一个 Database。而 Doris 的 Database 下的 Table 则对应于 ClickHouse 中，该 Database 下的 Tables。即映射关系如下：

Doris	ClickHouse
Catalog	ClickHouse Server
Database	Database
Table	Table

2.14.3.13.4 类型映射

ClickHouse Type	Doris Type	Comment
bool	boolean	
string	string	
date/date32	date	
datetime(S)/datetime64(S)	datetime(S)	
float32	float	
float64	double	
int8	tinyint	
int16/uint8	smallint	Doris 没有 UNSIGNED 数据类型，所以扩大一个数量级
int32/uInt16	int	同上
int64/uint32	bigint	同上
int128/uint64	largeint	同上
int256/uint128/uint256	string	Doris 没有这个数量级的数据类型，采用 STRING 处理
decimal(P, S)	decimal(P, S) or string	如果超过 Doris 支持的最大精度，使用 string 承接
enum/ipv4/ipv6/uuid	string	
array	array	
other	UNSUPPORTED	

2.14.3.13.5 相关参数

- `jdbc_clickhouse_query_final`

会话变量，默认为 false。当设置为 true 时，发送给 Clickhouse 的 SQL 语句后会添加 `SETTINGS final = 1`。

2.14.3.13.6 常见问题

1. 读取 Clickhouse 数据出现 `NoClassDefFoundError: net/jpountz/lz4/LZ4Factory` 错误信息

可以先下载lz4-1.3.0.jar包并放到每个 FE 和 BE 的目录下的 `custom_lib/` 目录下（如不存在，手动创建即可）。

2.14.3.14 SAP HANA JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 SAP HANA 数据库。本文档介绍如何配置 SAP HANA 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.14.1 使用须知

要连接到 SAP HANA 数据库，您需要

- SAP HANA 2.0 或更高版本。
- SAP HANA 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 SAP HANA JDBC 驱动程序。推荐使用 ngdbc 2.4.51 以上的版本。
- Doris 每个 FE 和 BE 节点和 SAP HANA 服务器之间的网络连接，默认端口为 30015。

2.14.3.14.2 连接 SAP HANA

```
CREATE CATALOG saphana_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwd',
    'jdbc_url' = 'jdbc:sap://Hostname:Port/?optionalparameters',
    'driver_url' = 'ngdbc-2.4.51.jar',
    'driver_class' = 'com.sap.db.jdbc.Driver'
)
```

有关 SAP HANA JDBC 驱动程序支持的 JDBC URL 格式和参数的更多信息，请参阅 SAP HANA。

2.14.3.14.3 层级映射

映射 SAP HANA 时，Doris 的 Database 对应于 SAP HANA 中指定 DataBase（`jdbc_url` 参数中的 “DATABASE”）下的一个 Schema。而 Doris 的 Database 下的 Table 则对应于 SAP HANA 中 Schema 下的 Tables。即映射关系如下：

Doris	SAP HANA
Catalog	Database
Database	Schema
Table	Table

2.14.3.14.4 列类型映射

SAP HANA Type	Doris Type	Comment
boolean	boolean	
tinyint	tinyint	
smalling	smalling	
integer	int	
bigint	bigint	
smalldecimal(P, S)	decimal(P, S) or double or string	如果没有指定精度，则使用 double 类型承接。如果精度超过 Doris 支持的最大精度，则使用 string 承接。
decimal(P, S)	decimal(P, S) or double or string	同上。

SAP HANA Type	Doris Type	Comment
real	float	
double	double	
date	date	
time	string	
timestamp(S)	datetime(S)	
seconddate	datetime(S)	
varchar	string	
nvarchar	string	
alphanum	string	
shorttext	string	
char(N)	char(N)	
nchar(N)	char(N)	
other	UNSUPPORTED	

2.14.3.15 Oceanbase JDBC Catalog

Doris JDBC Catalog 支持通过标准 JDBC 接口连接 OceanBase 数据库。本文档介绍如何配置 OceanBase 数据库连接。

关于 JDBC Catalog 概述，请参阅：JDBC Catalog 概述

2.14.3.15.1 使用须知

要连接到 OceanBase 数据库，您需要

- OceanBase 3.1.0 或更高版本
- OceanBase 数据库的 JDBC 驱动程序，您可以从 Maven 仓库下载最新或指定版本的 OceanBase JDBC 驱动程序。推荐使用 OceanBase Connector/J 2.4.8 或以上版本。
- Doris 每个 FE 和 BE 节点和 OceanBase 服务器之间的网络连接。

2.14.3.15.2 连接 OceanBase

```
CREATE CATALOG oceanbase_catalog PROPERTIES (
    'type' = 'jdbc',
    'user' = 'username',
    'password' = 'pwd',
    'jdbc_url' = 'jdbc:oceanbase://host:port/db',
    'driver_url' = 'oceanbase-client-2.4.8.jar',
    'driver_class' = 'com.oceanbase.jdbc.Driver'
)
```

`jdbc_url` 定义要传递给 OceanBase JDBC 驱动程序的连接信息和参数。支持的 URL 的参数可在 OceanBase JDBC 驱动配置 中找到。

2.14.3.15.3 模式兼容

Doris 会在创建 OceanBase Catalog 时，自动识别 OceanBase 处于 MySQL 或 Oracle 模式下，以便正确解析元数据。

不同模式下的层级映射、类型映射、查询优化，与 MySQL 或 Oracle 数据库的 Catalog 处理方式相同，可参考文档

- MySQL Catalog
- Oracle Catalog

2.14.4 分析 S3/HDFS 上的文件

通过 Table Value Function 功能，Doris 可以直接将对象存储或 HDFS 上的文件作为 Table 进行查询分析。并且支持自动的列类型推断。

更多使用方式可参阅 Table Value Function 文档：

- S3：支持 S3 兼容的对象存储上的文件分析。
- HDFS：支持 HDFS 上的文件分析。
- [FILE]：请参考 4.x 文档。

2.14.4.1 基础使用

这里我们通过 S3 Table Value Function 举例说明如何对对象存储上的文件进行分析。

2.14.4.1.1 查询

```
SELECT * FROM S3 (
    'uri' = 's3://bucket/path/to/tvf_test/test.parquet',
    'format' = 'parquet',
    's3.endpoint' = 'https://s3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    's3.access_key' = 'ak',
    's3.secret_key'='sk'
)
```

其中 S3(...)是一个 TVF（Table Value Function）。Table Value Function 本质上是一张表，因此他可以出现在任意 SQL 语句中“表”可以出现的位置上。

TVF 的属性包括要分析的文件路径，文件格式、对象存储的连接信息等。其中文件路径（URI）可以使用通配符匹配多个文件，以下的文件路径都是合法的：

- 匹配指定的文件

```
s3://bucket/path/to/tvf_test/test.parquet
```

- 匹配所有 test_ 开头的文件

```
s3://bucket/path/to/tvf_test/test_*
```

- 匹配所有 .parquet 后缀的文件

```
s3://bucket/path/to/tvf_test/*.parquet
```

- 匹配 tvf_test目录下的所有文件

```
s3://bucket/path/to/tvf_test/*
```

- 匹配文件名中包含 test的文件

```
s3://bucket/path/to/tvf_test/*test*
```

2.14.4.1.2 自动推断文件列类型

可以通过 DESC FUNCTION 语法可以查看 TVF 的 Schema：

```
DESC FUNCTION s3 (
    "URI" = "s3://bucket/path/to/tvf_test/test.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "parquet",
    "use_path_style"="true"
);
+---------------+--------------+------+-------+---------+-------+
| Field         | Type         | Null | Key   | Default | Extra |
+---------------+--------------+------+-------+---------+-------+
| p_partkey     | INT          | Yes  | false | NULL    | NONE  |
| p_name        | TEXT         | Yes  | false | NULL    | NONE  |
| p_mfgr        | TEXT         | Yes  | false | NULL    | NONE  |
| p_brand       | TEXT         | Yes  | false | NULL    | NONE  |
| p_type        | TEXT         | Yes  | false | NULL    | NONE  |
| p_size        | INT          | Yes  | false | NULL    | NONE  |
| p_container   | TEXT         | Yes  | false | NULL    | NONE  |
| p_retailprice | DECIMAL(9,0) | Yes  | false | NULL    | NONE  |
| p_comment     | TEXT         | Yes  | false | NULL    | NONE  |
+---------------+--------------+------+-------+---------+-------+
```

Doris 根据以下规则推断 Schema：

- 对于 Parquet、ORC 格式，Doris 会根据文件元信息获取 Schema。
- 对于匹配多个文件的情况，会使用第一个文件的 Schema 作为 TVF 的 Schema。
- 对于 CSV、JSON 格式，Doris 会根据字段、分隔符等属性，解析第一行数据获取 Schema。

默认情况下，所有列类型均为 string。可以通过 csv_schema 属性单独指定列名和列类型。Doris 会使用指定的列类型进行文件读取。格式如下：name1:type1;name2:type2;...。如：

```
S3 (
    'uri' = 's3://bucket/path/to/tvf_test/test.parquet',
    's3.endpoint' = 'https://s3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    's3.access_key' = 'ak'
    's3.secret_key'='sk',
    'format' = 'csv',
    'column_separator' = '|',
    'csv_schema' = 'k1:int;k2:int;k3:int;k4:decimal(38,10)'
)
```

当前支持的列类型名称如下：

列类型名称
tinyint
smallint
int
bigint
largeint
float
double
decimal(p,s)
date
datetime
char
varchar
string
boolean

- 对于格式不匹配的列（比如文件中为字符串，用户定义为 int；或者其他文件和第一个文件的 Schema 不相同），或缺失列（比如文件中有 4 列，用户定义了 5 列），则这些列将返回 null。

2.14.4.2 适用场景

2.14.4.2.1 查询分析

TVF 非常适用于对存储系统上的独立文件进行直接分析，而无需事先将数据导入到 Doris 中。

可以使用任意的 SQL 语句进行文件分析，如：

```
SELECT * FROM s3(
    'uri' = 's3://bucket/path/to/tvf_test/test.parquet',
    'format' = 'parquet',
```

```
    's3.endpoint' = 'https://s3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    's3.access_key' = 'ak',
    's3.secret_key'='sk'
)
ORDER BY p_partkey LIMIT 5;
+--
    ↪ ---------+-----------------------------------------+----------------+----------+-------------------------
    ↪
| p_partkey | p_name                                    | p_mfgr         | p_brand  | p_type
    ↪                  | p_size | p_container | p_retailprice | p_comment           |
+--
    ↪ ---------+-----------------------------------------+----------------+----------+-------------------------
    ↪
|         1 | goldenrod lavender spring chocolate lace | Manufacturer#1 | Brand#13 | PROMO
    ↪ BURNISHED COPPER  |      7 | JUMBO PKG   |           901 | ly. slyly ironi     |
|         2 | blush thistle blue yellow saddle         | Manufacturer#1 | Brand#13 | LARGE
    ↪ BRUSHED BRASS     |      1 | LG CASE     |           902 | lar accounts amo    |
|         3 | spring green yellow purple cornsilk      | Manufacturer#4 | Brand#42 | STANDARD
    ↪ POLISHED BRASS |     21 | WRAP CASE   |           903 | egular deposits hag |
|         4 | cornflower chocolate smoke green pink    | Manufacturer#3 | Brand#34 | SMALL PLATED
    ↪  BRASS      |     14 | MED DRUM    |           904 | p furiously r       |
|         5 | forest brown coral puff cream            | Manufacturer#3 | Brand#32 | STANDARD
    ↪ POLISHED TIN   |     15 | SM PKG      |           905 |  wake carefully     |
+--
    ↪ ---------+-----------------------------------------+----------------+----------+-------------------------
    ↪
```

TVF 可以出现在 SQL 中，Table 能出现的任意位置。如 CTE 的 WITH 子句中，FROM 子句中等等。这样，您可以把文件当做一张普通的表进行任意分析。

您也可以用过 CREATE VIEW 语句为 TVF 创建一个逻辑视图。之后，可以像其他视图一样，对这个 TVF 进行访问、权限管理等操作，也可以让其他用户访问这个 View，而无需重复书写连接信息等属性。

```
-- Create a view based on a TVF
CREATE VIEW tvf_view AS
SELECT * FROM s3(
    'uri' = 's3://bucket/path/to/tvf_test/test.parquet',
    'format' = 'parquet',
    's3.endpoint' = 'https://s3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    's3.access_key' = 'ak',
    's3.secret_key'='sk'
);

-- Describe the view as usual
```

```
DESC tvf_view;

-- Query the view as usual
SELECT * FROM tvf_view;

-- Grant SELECT priv to other user on this view
GRANT SELECT_PRIV ON db.tvf_view TO other_user;
```

2.14.4.2.2 数据导入

TVF 可以作为 Doris 数据导入方式的一种。配合 INSERT INTO SELECT 语法，我们可以很方便的将文件导入到 Doris 中。

```
-- Create a Doris table
CREATE TABLE IF NOT EXISTS test_table
(
    id int,
    name varchar(50),
    age int
)
DISTRIBUTED BY HASH(id) BUCKETS 4
PROPERTIES("replication_num" = "1");

-- 2. Load data into table from TVF
INSERT INTO test_table (id,name,age)
SELECT cast(id as INT) as id, name, cast (age as INT) as age
FROM s3(
    'uri' = 's3://bucket/path/to/tvf_test/test.parquet',
    'format' = 'parquet',
    's3.endpoint' = 'https://s3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    's3.access_key' = 'ak',
    's3.secret_key'='sk'
);
```

2.14.4.3 注意事项

1. 如果指定的 uri 匹配不到文件，或者匹配到的所有文件都是空文件，那么 TVF 将会返回空结果集。在这种情况下使用DESC FUNCTION查看这个 TVF 的 Schema，会得到一列虚拟的列__dummy_col，该列无意义，仅作为占位符使用。
2. 如果指定的文件格式为 csv，所读文件不为空文件但文件第一行为空，则会提示错误The first line is empty, can not parse column numbers，这是因为无法通过该文件的第一行解析出 Schema。

2.14.5 元数据服务

2.14.5.1 Hive Metastore

本文档用于介绍通过 CREATE CATALOG 语句连接并访问 Hive MetaStore 服务时支持的所有参数。

2.14.5.1.1 支持的 Catalog 类型

Catalog 类型	类型标识 (type)	描述
Hive	hms	对接 Hive Metastore 的 Catalog
Iceberg	iceberg	对接 Iceberg 表格式
Paimon	paimon	对接 Apache Paimon 表格式

2.14.5.1.2 通用参数总览

以下参数为不同 Catalog 类型的通用参数。

参数名称	曾用名	是否必须	默认值	简要描述
hive.metastore.uris		是	无	Hive Metastore 的 URI 地址，支持多个逗号分隔，示例：‘hive.metastore.uris’ = ‘thrift://127.0.0.1:9083’，‘hive.metastore.uris’ = ‘thrift://127.0.0.1:9083,thrift://127.0.0.1:9084’
hive.metastore.authentication.type	hadoop.security.authentication	否	simple	Metastore 认证方式：支持 simple（默认）或 kerberos，在 3.0 及之前版本中，认证方式由 hadoop.security.authentication 属性决定。3.1 版本开始，可以单独指定 Hive Metastore 的认证方式，示例：‘hive.metastore.authentication.type’ = ‘kerberos’
hive.metastore.service.principal	hive.metastore.kerberos.principal	否	空	Hive 服务端 principal，支持 _HOST 占位符，示例：‘hive.metastore.service.principal’ = ’hive/<_HOST@EXAMPLE.COM>’
hive.metastore.client.principal	hadoop.kerberos.principal	否	空	Doris 连接到 Hive MetaStore 服务时使用的 Kerberos 主体。
hive.metastore.client.keytab	hadoop.kerberos.keytab	否	空	Kerberos keytab 文件路径
hive.metastore.username	hadoop.username	否	hadoop	Hive Metastore 用户名，非 Kerberos 模式下使用
hive.conf.resources		否	空	hive-site.xml 配置文件路径，使用相对路径

> **注：**
>
> 3.1.0 版本之前，请使用曾用名。

必填参数

- `hive.metastore.uris`：必须指定 Hive Metastore 的 URI 地址

可选参数

- `hive.metastore.authentication.type`：认证方式，默认为 `simple`，可选 `kerberos`
- `hive.metastore.service.principal`：Hive MetaStore 服务的 Kerberos 主体，当使用 Kerberos 认证时必须指定。
- `hive.metastore.client.principal`：Doris 连接到 Hive MetaStore 服务时使用的 Kerberos 主体，当使用 Kerberos 认证时必须指定。
- `hive.metastore.client.keytab`：Kerberos keytab 文件路径，当使用 Kerberos 认证时必须指定。
- `hive.metastore.username`：连接 Hive MetaStore 服务的用户名，非 Kerberos 模式下使用，默认为 hadoop。
- `hive.conf.resources`：hive-site.xml 配置文件路径，当需要通过配置文件的方式读取链接 Hive Metastore 服务的配置时使用。

认证方式

Simple 认证

- `simple`：非 Kerberos 模式，直接连接 Hive Metastore 服务。

Kerberos 认证

使用 Kerberos 认证连接 Hive Metastore 服务，需要配置以下参数：

- `hive.metastore.authentication.type`：设置为 `kerberos`
- `hive.metastore.service.principal`：Hive MetaStore 服务的 Kerberos 主体
- `hive.metastore.client.principal`：Doris 连接到 Hive MetaStore 服务时使用的 Kerberos 主体
- `hive.metastore.client.keytab`：Kerberos keytab 文件路径

```
'hive.metastore.authentication.type' = 'kerberos',
'hive.metastore.service.principal' = 'hive/_HOST@EXAMPLE.COM',
'hive.metastore.client.principal' = 'hive/doris.cluster@EXAMPLE.COM',
'hive.metastore.client.keytab' = '/etc/security/keytabs/hive.keytab'
```

使用开启 Kerberos 认证的 Hive MetaStore 服务时需要确保所有 FE 节点上都存在相同的 keytab 文件，并且运行 Doris 进程的用户具有该 keytab 文件的读权限。以及 krb5 配置文件配置正确。

Kerberos 详细配置参考 Kerberos 认证。

配置文件参数

`hive.conf.resources`

如需要通过配置文件的方式读取链接 Hive Metastore 服务的配置，可以配置 `hive.conf.resources` 参数来设置 conf 文件路径。

> 注意：hive.conf.resources 参数仅支持相对路径，请勿使用绝对路径。默认路径为 ${DORIS_HOME}/plugins/hadoop_conf/ 目录下。可通过修改 fe.conf 中的 hadoop_config_dir 来指定其他目录。

示例：'hive.conf.resources' = 'hms-1/hive-site.xml'

2.14.5.1.3 Catalog 类型数据

以下参数是除通用参数外，各个 Catalog 特有的参数说明。

Hive Catalog

参数名称	曾用名	是否必须	默认值	简要描述
type		是	无	Catalog 类型，Hive Catalog 固定为 iceberg
hive.metastore.type		否	‘hms’	Metadata Catalog 类型，Hive Metastore 固定为 hms，使用 HiveMetaStore 则必须为 hms

示例

1. 创建一个使用无认证的 Hive Metastore 作为元数据服务的 Hive Catalog，存储使用 S3 存储服务。

```
CREATE CATALOG hive_hms_s3_test_catalog PROPERTIES (
    'type' = 'hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9383',
    's3.access_key' = 'S3_ACCESS_KEY',
    's3.secret_key' = 'S3_SECRET_KEY',
    's3.region' = 's3.ap-east-1.amazonaws.com'
);
```

2. 创建一个使用开启了 Kerberos 认证的 Hive Metastore 作为元数据服务的 Hive Catalog，存储使用 S3 存储服务。

```
 CREATE CATALOG hive_hms_on_oss_kerberos_new_catalog PROPERTIES (
    'type' = 'hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9583',
    'hive.metastore.client.principal'='hive/presto-master.docker.cluster@LABS.TERADATA.COM',
    'hive.metastore.client.keytab' = '/mnt/keytabs/keytabs/hive-presto-master.keytab',
    'hive.metastore.service.principal' = 'hive/hadoop-master@LABS.TERADATA.COM',
    'hive.metastore.authentication.type'='kerberos',
```

```
    'hadoop.security.auth_to_local' = 'RULE:[2:\$1@\$0](\#)s/@.*//
                        RULE:[2:\$1@\$0](\#)s/@.*//
                        RULE:[2:\$1@\$0](\#)s/@.*//
                        DEFAULT',
    'oss.access_key' = 'OSS_ACCESS_KEY',
    'oss.secret_key' = 'OSS_SECRET_KEY',
    'oss.endpoint' = 'oss-cn-beijing.aliyuncs.com'
);
```

Iceberg Catalog

参数名称	曾用名	是否必须	默认值	简要描述
type		是	无	Catalog 类型，Iceberg 固定为 iceberg
iceberg.catalog.type		否	无	Metadata Catalog 类型，Hive Metastore 固定为 hms，使用 HiveMetaStore 则必须为 hms
warehouse		否	无	Iceberg 仓库路径

示例

1. 创建一个使用 Hive Metastore 作为元数据服务的 Iceberg Catalog，存储使用 S3 存储服务。

```
 CREATE CATALOG iceberg_hms_s3_test_catalog PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9383',
    'warehouse' = 's3://doris/iceberg_warehouse/',
    's3.access_key' = 'S3_ACCESS_KEY',
    's3.secret_key' = 'S3_SECRET_KEY',
    's3.region' = 's3.ap-east-1.amazonaws.com'
);
```

- 创建一个使用开启了 Kerberos 认证的 Hive Metastore 作为元数据服务的 Iceberg Catalog，并且处于多 kerberos 环境下。存储使用 S3 存储服务。

```
CREATE CATALOG IF NOT EXISTS iceberg_hms_on_oss_kerberos_new_catalog PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9583',
    'warehouse' = 'oss://doris/iceberg_warehouse/',
    'hive.metastore.client.principal'='hive/presto-master.docker.cluster@LABS.TERADATA.COM',
```

```
    'hive.metastore.client.keytab' = '/mnt/keytabs/keytabs/hive-presto-master.keytab',
    'hive.metastore.service.principal' = 'hive/hadoop-master@LABS.TERADATA.COM',
    'hive.metastore.authentication.type'='kerberos',
    'hadoop.security.auth_to_local' = 'RULE:[2:\$1@\$0](\#)s/@.*//
                      RULE:[2:\$1@\$0](\#)s/@.*//
                      RULE:[2:\$1@\$0](\#)s/@.*//
                      DEFAULT',
    'oss.access_key' = 'OSS_ACCESS_KEY',
    'oss.secret_key' = 'OSS_SECRET_KEY',
    'oss.endpoint' = 'oss-cn-beijing.aliyuncs.com'
);
```

Paimon Catalog

参数名称	曾用名	是否必须	默认值	简要描述
type		是	无	Catalog 类型，Iceberg 固定为 iceberg
paimon.catalog.type		否	filesystem	使用 HiveMetaStore 则必须为 hms，默认值为 filesystem，即使用文件系统存储元数据
warehouse		是	无	Paimon 仓库路径

示例

1. 创建一个使用 Hive Metastore 作为元数据服务的 Paimon Catalog，存储使用 S3 存储服务。

```
CREATE CATALOG IF NOT EXISTS paimon_hms_s3_test_catalog PROPERTIES (
    'type' = 'paimon',
    'paimon.catalog.type' = 'hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9383',
    'warehouse' = 's3://doris/paimon_warehouse/',
    's3.access_key' = 'S3_ACCESS_KEY',
    's3.secret_key' = 'S3_SECRET_KEY',
    's3.region' = 's3.ap-east-1.amazonaws.com'
);
```

- 创建一个使用开启了 Kerberos 认证的 Hive Metastore 作为元数据服务的 Paimon Catalog，并且处于多 kerberos 环境下。存储使用 S3 存储服务。

```
CREATE CATALOG IF NOT EXISTS paimon_hms_on_oss_kerberos_new_catalog PROPERTIES (
    'type' = 'paimon',
    'paimon.catalog.type' = 'hms',
```

```
    'hive.metastore.uris' = 'thrift://127.0.0.1:9583',
    'warehouse' = 's3://doris/iceberg_warehouse/',
    'hive.metastore.client.principal'='hive/presto-master.docker.cluster@LABS.TERADATA.COM',
    'hive.metastore.client.keytab' = '/mnt/keytabs/keytabs/hive-presto-master.keytab',
    'hive.metastore.service.principal' = 'hive/hadoop-master@LABS.TERADATA.COM',
    'hive.metastore.authentication.type'='kerberos',
    'hadoop.security.auth_to_local' = 'RULE:[2:\$1@\$0](\#)s/@.*//
                      RULE:[2:\$1@\$0](\#)s/@.*//
                      RULE:[2:\$1@\$0](\#)s/@.*//
                      DEFAULT',
    'oss.access_key' = 'OSS_ACCESS_KEY',
    'oss.secret_key' = 'OSS_SECRET_KEY',
    'oss.endpoint' = 'oss-cn-beijing.aliyuncs.com'
);
```

2.14.5.1.4 常见问题 FAQ

- Q1: hive-site.xml 是必须的吗？

  不是，仅当需要从中读取链接配置时使用。

- Q2: keytab 文件是否必须每个节点都存在？

  是的，所有 FE 节点必须可访问指定路径。

- Q3: 如使用回写功能，即在 Doris 中创建 Hive/Iceberg 库/表，需要注意什么？

  由于创建表涉及到存储端的元数据操作，即需要访问存储系统，因此 Hive MetaStore 服务 Server 端需要配置对应存储参数，如 S3、OSS 等存储服务的访问参数。如使用对象存储作为底层存储系统，还需要确保写入的 bucket 与配置的 Region 一致。

2.14.5.2 AWS Glue

本文档介绍通过 CREATE CATALOG 使用 AWS Glue Catalog 访问 Iceberg 表或 Hive 表时的参数配置。

2.14.5.2.1 Glue Catalog 支持的类型

AWS Glue Catalog 当前支持三种类型的 Catalog：

Catalog 类型	类型标识 (type)	描述
Hive	glue	对接 Hive Metastore 的 Catalog
Iceberg	glue	对接 Iceberg 表格式
Iceberg	rest	通过 Glue Rest Catalog 对接 Iceberg 表格式

本说明文档分别对这写类型的参数进行详细介绍，便于用户配置。

2.14.5.2.2 通用参数总览

参数名称	描述	是否必须	默认值
glue.region	AWS Glue 所在区域，例如：us-east-1	是	无
glue.endpoint	AWS Glue endpoint，例如：https://glue.us-east-1.amazonaws.com	是	无
glue.access_key	AWS Access Key ID	是	空
glue.secret_key	AWS Secret Access Key	是	空
glue.catalog_id	Glue Catalog ID（暂未支持）	否	空
glue.role_arn	IAM Role ARN，用于访问 Glue（自 3.1.2+ 支持）	否	空
glue.external_id	IAM External ID，用于访问 Glue（自 3.1.2+ 支持）	否	空

认证参数

访问 Glue 需要认证信息，支持以下两种方式：

1. Access Key 认证

通过 glue.access_key 和 glue.secret_key 提供的 Access Key 认证访问 Glue。

2. IAM Role 认证（自 3.1.2+ 起支持）

通过 glue.role_arn 提供的 IAM Role 认证访问 Glue。

该方式需要 Doris 部署在 AWS EC2 上，并且 EC2 实例需要绑定一个 IAM Role，且该 Role 需要有访问 Glue 的权限。

如果需要通过 External ID 进行访问，需要同时配置 glue.external_id。

注意事项：

- 两种方式必须至少配置一种，如果同时配置了两种方式，则优先使用 AccessKey 认证。

示例：

```
CREATE CATALOG hive_glue_catalog PRPPERTIES (
  'type' = 'hms',
  'hive.metastore.type' = 'glue',
  'glue.region' = 'us-east-1',
  'glue.endpoint' = 'https://glue.us-east-1.amazonaws.com',
  -- 使用 Access Key 认证
  'glue.access_key' = '<YOUR_ACCESS_KEY>',
  'glue.secret_key' = '<YOUR_SECRET_KEY>'
  -- 或者使用 IAM Role 认证
  -- 'glue.role_arn' = '<YOUR_ROLE_ARN>',
  -- 'glue.external_id' = '<YOUR_EXTERNAL_ID>'
);
```

Hive Glue Catalog

Hive Glue Catalog 用于访问 Hive 表，通过 AWS Glue 的 Hive Metastore 兼容接口访问 Glue。配置如下：

参数名称	描述	是否必须	默认值
type	固定为 hms	是	无
hive.metastore.type	固定为 glue	是	无
glue.region	AWS Glue 所在区域，例如：us-east-1	是	无
glue.endpoint	AWS Glue endpoint，例如：https://glue.us-east-1.amazonaws.com	是	无
glue.access_key	AWS Access Key ID	否	空
glue.secret_key	AWS Secret Access Key	否	空
glue.catalog_id	Glue Catalog ID（暂未支持）	否	空
glue.role_arn	IAM Role ARN，用于访问 Glue	否	空
glue.external_id	IAM External ID，用于访问 Glue	否	空

示例

```
CREATE CATALOG hive_glue_catalog PROPERTIES (
  'type' = 'hms',
  'hive.metastore.type' = 'glue',
  'glue.region' = 'us-east-1',
  'glue.endpoint' = 'https://glue.us-east-1.amazonaws.com',
  'glue.access_key' = 'YOUR_ACCESS_KEY',
  'glue.secret_key' = 'YOUR_SECRET_KEY'
);
```

Iceberg Glue Catalog

Iceberg Glue Catalog 通过 Glue Client 访问 Glue。配置如下：

参数名称	描述	是否必须	默认值
type	固定为 iceberg	是	无
iceberg.catalog.type	固定为 glue	是	无
warehouse	Iceberg 数据仓库路径，例如：s3://my-bucket/iceberg-warehouse/	是	s3://doris
glue.region	AWS Glue 所在区域，例如：us-east-1	是	无
glue.endpoint	AWS Glue endpoint，例如：https://glue.us-east-1.amazonaws.com	是	无
glue.access_key	AWS Access Key ID	否	空
glue.secret_key	AWS Secret Access Key	否	空
glue.catalog_id	Glue Catalog ID（暂未支持）	否	空
glue.role_arn	IAM Role ARN，用于访问 Glue（暂未支持）	否	空
glue.external_id	IAM External ID，用于访问 Glue（暂未支持）	否	空

示例

```
CREATE CATALOG iceberg_glue_catalog PROPERTIES (
  'type' = 'iceberg',
  'iceberg.catalog.type' = 'glue',
  'glue.region' = 'us-east-1',
  'glue.endpoint' = 'https://glue.us-east-1.amazonaws.com',
```

```
  'glue.access_key' = '<YOUR_ACCESS_KEY>',
  'glue.secret_key' = '<YOUR_SECRET_KEY>'
);
```

Iceberg Glue Rest Catalog

Iceberg Glue Rest Catalog 通过 Glue Rest Catalog 接口访问 Glue。目前仅支持存储在 AWS S3 Table Bucket 中的 Iceberg 表。
配置如下：

参数名称	描述	是否必须
type	固定为 iceberg	是
iceberg.catalog.type	固定为 rest	是
iceberg.rest.uri	Glue Rest 服务端点，例如：https://glue.ap-east-1.amazonaws.com/iceberg	是
warehouse	Iceberg 数据仓库路径，例如：<account_id>:s3tablescatalog/<bucket_name>	是
iceberg.rest.sigv4-enabled	启动 V4 签名格式，固定为 true	是
iceberg.rest.signing-name	签名类型，固定为 glue	是
iceberg.rest.access-key-id	访问 Glue 的 Access Key（同时也用于访问 S3 Bucket）	是
iceberg.rest.secret-access-key	访问 Glue 的 Secret Key（同时也用于访问 S3 Bucket）	是
iceberg.rest.signing-region	AWS Glue 所在区域，例如：us-east-1	是

示例

```
CREATE CATALOG glue_s3 PROPERTIES (
  'type' = 'iceberg',
  'iceberg.catalog.type' = 'rest',
  'iceberg.rest.uri' = 'https://glue.<region>.amazonaws.com/iceberg',
  'warehouse' = '<acount_id>:s3tablescatalog/<s3_table_bucket_name>',
  'iceberg.rest.sigv4-enabled' = 'true',
  'iceberg.rest.signing-name' = 'glue',
  'iceberg.rest.access-key-id' = '<ak>',
  'iceberg.rest.secret-access-key' = '<sk>',
  'iceberg.rest.signing-region' = '<region>'
);
```

2.14.5.2.3 权限策略

根据使用场景不同，可以分为 只读和 读写两类策略。

1. 只读权限

只允许读取 Glue Catalog 的数据库和表信息。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GlueCatalogReadOnly",
```

```
      "Effect": "Allow",
      "Action": [
        "glue:GetCatalog",
        "glue:GetDatabase",
        "glue:GetDatabases",
        "glue:GetTable",
        "glue:GetTables",
        "glue:GetPartitions"
      ],
      "Resource": [
        "arn:aws:glue:<region>:<account-id>:catalog",
        "arn:aws:glue:<region>:<account-id>:database/*",
        "arn:aws:glue:<region>:<account-id>:table/*/*"
      ]
    }
  ]
}
```

2. 读写权限

在只读的基础上，允许创建 / 修改 / 删除数据库和表。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GlueCatalogReadWrite",
      "Effect": "Allow",
      "Action": [
        "glue:GetCatalog",
        "glue:GetDatabase",
        "glue:GetDatabases",
        "glue:GetTable",
        "glue:GetTables",
        "glue:GetPartitions",
        "glue:CreateDatabase",
        "glue:UpdateDatabase",
        "glue:DeleteDatabase",
        "glue:CreateTable",
        "glue:UpdateTable",
        "glue:DeleteTable"
      ],
      "Resource": [
        "arn:aws:glue:<region>:<account-id>:catalog",
        "arn:aws:glue:<region>:<account-id>:database/*",
        "arn:aws:glue:<region>:<account-id>:table/*/*"
```

```
      ]
    }
  ]
}
```

注意事项

1. 占位符替换
    - <region> → 你的 AWS 区域（如 us-east-1）。
    - <account-id> → 你的 AWS 账号 ID（12 位数字）。
2. 最小权限原则
    - 如果只做查询，不要授予写权限。
    - 可以替换 * 为具体数据库、表 ARN，进一步收紧权限。
3. S3 权限
    - 上述策略只涉及 Glue Catalog
    - 如果需要读取数据文件，还需额外授予 S3 权限（如 s3:GetObject, s3:ListBucket 等）。

2.14.5.3 Google Dataproc

本文档用于介绍通过CREATE CATALOG语句连接并访问 Google Dataproc Metastore 时所支持的参数。

属性名称	描述	默认值	是否必须
hive.metastore.uri	Metastore 的 URI 地址。可以通过 Dataproc Metastore Services 页面获取	无	是

2.14.5.4 Aliyun DLF

本文档介绍如何使用 CREATE CATALOG 语句连接并访问阿里云 Dake Lake Formation(DLF) 元数据服务。

2.14.5.4.1 DLF 版本说明

- 对于 DLF 1.0 版本，Doris 通过 DLF 的 Hive Metastore 兼容接口访问 DLF。支持 Paimon Catalog 和 Hive Catalog。
- 对于 DLF 2.5 之后的版本，Doris 通过 DLF 的 Rest 接口访问 DLF。仅支持 Paimon Catalog。

DLF 1.0

参数名称	曾用名	描述
dlf.endpoint	-	DLF endpoint，详见：阿里云文档
dlf.region	-	DLF region，详见：阿里云文档
dlf.uid	-	阿里云账号 ID。可在控制台右上角个人信息查看。
dlf.access_key	-	阿里云 AccessKey，用于访问 DLF 服务。
dlf.secret_key	-	阿里云 SecretKey，用于访问 DLF 服务。

参数名称	曾用名	描述
dlf.catalog_id	dlf.catalog.id	Catalog ID。用于指定元数据目录，如果不设置则使用默认目录。
warehouse	-	Warehouse 的存储路径，仅在 Paimon Catalog 中需要填写。注意，对象存储路径，已经要以

> 注：
>
> 3.1.0 版本之前，请使用曾用名。

DLF 2.5+ (Rest Catalog)

> 自 3.1.0 版本支持

参数名称	曾用名	描述
uri	-	DLF REST URI。 示例：http://cn-beijing-vpc.dlf.aliyuncs.com
warehouse	-	Warehouse 名称。注意这里直接填写需要连接的 Catalog 的名称，而非 Paimon 表
paimon.rest.token.provider	-	token 提供方，固定填写 dlf
paimon.rest.dlf.access-key-id	-	阿里云 AccessKey，用于访问 DLF 服务。
paimon.rest.dlf.access-key-secret	-	阿里云 SecretKey，用于访问 DLF 服务。

DLF Rest Catalog 无需提供存储服务（OSS）的 Endpoint 和 Region 等信息。Doris 会利用 DLF Rest Catalog 的 Vended Credential 获取用于访问 OSS 的临时凭证信息。

2.14.5.4.2 示例

DLF 1.0

创建 Hive Catalog，以 DLF 作为元数据服务：

```
CREATE CATALOG hive_dlf_catalog WITH (
  'type' = 'hms',
  'hive.metastore.type' = 'dlf',
  'dlf.endpoint' = '<DLF_ENDPOINT>',
  'dlf.region' = '<DLF_REGION>',
  'dlf.uid' = '<YOUR_ALICLOUD_UID>',
  'dlf.access_key' = '<YOUR_ACCESS_KEY>',
  'dlf.secret_key' = '<YOUR_SECRET_KEY>'
);
```

创建 Paimon Catalog，以 DLF 作为元数据服务：

```
CREATE CATALOG paimon_dlf PROPERTIES (
    'type' = 'paimon',
    'paimon.catalog.type' = 'dlf',
    'warehouse' = 'oss://xx/yy/',
    'dlf.proxy.mode' = 'DLF_ONLY',
    'dlf.endpoint' = '<DLF_ENDPOINT>',
    'dlf.region' = '<DLF_REGION>',
    'dlf.uid' = '<YOUR_ALICLOUD_UID>',
    'dlf.access_key' = '<YOUR_ACCESS_KEY>',
    'dlf.secret_key' = '<YOUR_SECRET_KEY>'
);
```

DLF 2.5+ (Rest Catalog)

```
CREATE CATALOG paimon_dlf_test PROPERTIES (
    'type' = 'paimon',
    'paimon.catalog.type' = 'rest',
    'uri' = 'http://cn-beijing-vpc.dlf.aliyuncs.com',
    'warehouse' = 'my_catalog_name',
    'paimon.rest.token.provider' = 'dlf',
    'paimon.rest.dlf.access-key-id' = '<YOUR_ACCESS_KEY>',
    'paimon.rest.dlf.access-key-secret' = '<YOUR_SECRET_KEY>'
);
```

2.14.5.5 Iceberg Rest Catalog API

本文档用于介绍通过 CREATE CATALOG 语句连接并访问支持 Iceberg Rest Catalog 接口的元数据服务时所支持的参数。

2.14.5.5.1 参数总览

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.uri	uri	指定 Rest 服务地址	-	是

属性名称	曾用名	描述	默认值	是否必须
warehouse	warehouse	指定 iceberg warehouse	-	是
iceberg.rest.security.type		指定 Rest 服务认证方式，支持 oauth2，默认为 none，即无认证	none	否

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.oauth2.token		当使用 oauth2 认证方式时，指定 bearer token	-	否

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.oauth2.scope		当使用 oauth2 ↪ 认证方式时，指定用户授权后能够访问的资源范围和权限级别。	-	否

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.oauth2.credential		oauth2 ↪ 凭证，用于访问 server ↪ - ↪ uri ↪ 获取 token	-	否
iceberg.rest.oauth2.server-uri		用于获取 oauth2 ↪ token 的 uri 地址，配合 iceberg ↪ . ↪ rest ↪ . ↪ oauth2 ↪ . ↪ credential ↪ 使用	-	否

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.vended-credentials-enabled		是否启用 vended-credentials 功能。启用后，会同 rest 服务端获取访问存储系统的凭证信息，如 access-key 和 secret-key，不再需要	false	否

属性名称	曾用名	描述	默认值	是否必须
iceberg.rest.nested-namespace-enabled		(自3.1.2+版本支持)是否启用对Nested Namespace的支持。默认为false。如果为true，则Nested Namespace会被打平作为Database名称显示，如parent	否	

属性名称	曾用名	描述	默认值	是否必须

注：

1. oauth2 认证和 vended-credentials 功能自 3.1.0 版本开始支持。
2. 3.1.0 之前的版本，请使用曾用名。
3. AWS Glue Rest Catalog 请参阅 AWS Glue 文档

2.14.5.5.2 Nested Namespace

在 3.1.2 及后续版本中，如需完整访问 Nested Namespace，除了在 Catalog 属性中将 iceberg.rest.nested-namespace-enabled 设置为 true 外，还需开启如下全局参数：

```
SET GLOBAL enable_nested_namespace=true;
```

假设 Catalog 为 “ice”，Namespace 为 “ns1.ns2”，Table 为 “tbl1”，可参考如下方式访问 Nested Namespace：

```
mysql> USE ice.ns1.ns2;
mysql> SELECT k1 FROM ice.`ns1.ns2`.tbl1;
mysql> SELECT tbl1.k1 FROM `ns1.ns2`.tbl1;
mysql> SELECT `ns1.ns2`.tbl1.k1 FROM ice.`ns1.ns2`.tbl1;
mysql> SELECT ice.`ns1.ns2`.tbl1.k1 FROM tbl1;
mysql> REFRESH CATALOG ice;
mysql> REFRESH DATABASE ice.`ns1.ns2`;
mysql> REFRESH TABLE ice.`ns1.ns2`.tbl1;
```

2.14.5.5.3 示例配置

- 无认证的 Rest Catalog 服务

```
CREATE CATALOG minio_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'uri' = 'http://172.21.0.1:8181',
    's3.access_key' = '<ak>',
    's3.secret_key' = '<sk>',
    's3.endpoint' = 'http://10.0.0.1:9000',
    's3.region' = 'us-east-1'
);
```

- 连接 AWS Glue Rest Catalog

```
CREATE CATALOG glue_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'https://glue.<region>.amazonaws.com/iceberg',
    'warehouse' = '<acount_id>:s3tablescatalog/<s3_table_bucket_name>',
    'iceberg.rest.sigv4-enabled' = 'true',
    'iceberg.rest.signing-name' = 'glue',
    'iceberg.rest.access-key-id' = '<ak>',
    'iceberg.rest.secret-access-key' = '<sk>',
    'iceberg.rest.signing-region' = '<region>'
);
```

- 连接 Databricks Unity Iceberg Rest Catalog

```
CREATE CATALOG unity_iceberg properties(
    "uri" = "https://dbc-59918a85-6c3a.cloud.databricks.com/api/2.1/unity-catalog/iceberg-
        ↪ rest/",
    "type" = "iceberg",
    "warehouse" = "<catalog_name>",
    "iceberg.catalog.type" = "rest",
    "iceberg.rest.security.type" = "oauth2",
    "iceberg.rest.oauth2.token" = "<token>",
    "iceberg.rest.vended-credentials-enabled" = "true",
    's3.endpoint' = 'https://s3.us-east-2.amazonaws.com',
    's3.region' = 'us-east-2'
);
```

- 连接 Apache Polaris Rest Catalog

```
-- Enable vended-credentials
CREATE CATALOG polaris_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog',
    'warehouse' = '<catalog_name>',
    'iceberg.rest.security.type' = 'oauth2',
    'iceberg.rest.oauth2.credential' = 'client_id:client_secret',
    'iceberg.rest.oauth2.server-uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog/v1/oauth/
        ↪ tokens',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:doris_pr_role',
    'iceberg.rest.vended-credentials-enabled' = 'true',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
```

```
    's3.region' = 'us-west-2'
);

-- Disable vended-credentials
CREATE CATALOG polaris_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog',
    'warehouse' = '<catalog_name>',
    'iceberg.rest.security.type' = 'oauth2',
    'iceberg.rest.oauth2.credential' = '6e155b128dc06c13:ce9fbb4cc91c43ff2955f2c6545239d7',
    'iceberg.rest.oauth2.server-uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog/v1/oauth/
        ↪ tokens',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:doris_pr_role',
    's3.access_key' = '<ak>',
    's3.secret_key' = '<sk>',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2'
);
```

- 连接 Snowflake Open Catalog (自 3.1.2 版本支持)

```
-- Enable vended-credentials
CREATE CATALOG snowflake_open_catalog PROPERTIES (
    'type' = 'iceberg',
    'warehouse' = '<catalog_name>',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'https://<open_catalog_account>.snowflakecomputing.com/polaris/api/
        ↪ catalog',
    'iceberg.rest.security.type' = 'oauth2',
    'iceberg.rest.oauth2.credential' = '<client_id>:<client_secret>',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:<principal_role>',
    'iceberg.rest.vended-credentials-enabled' = 'true',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2',
    'iceberg.rest.nested-namespace-enabled' = 'true'
);
```

```
-- Disable vended-credentials
CREATE CATALOG snowflake_open_catalog PROPERTIES (
    'type' = 'iceberg',
    'warehouse' = '<catalog_name>',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'https://<open_catalog_account>.snowflakecomputing.com/polaris/api/
        ↪ catalog',
```

```
    'iceberg.rest.security.type' = 'oauth2',
    'iceberg.rest.oauth2.credential' = '<client_id>:<client_secret>',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:<principal_role>',
    's3.access_key' = '<ak>',
    's3.secret_key' = '<sk>',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2',
    'iceberg.rest.nested-namespace-enabled' = 'true'
);
```

- 连接 Apache Gravitino Rest Catalog

```
-- Enable vended-credentials
CREATE CATALOG gravitino_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://127.0.0.1:9001/iceberg/',
    'warehouse' = 's3://gravitino-iceberg-demo/warehouse',
    'iceberg.rest.vended-credentials-enabled' = 'true',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2'
);

-- Disable vended-credentials
CREATE CATALOG gravitino_iceberg PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://127.0.0.1:9001/iceberg/',
    'warehouse' = 's3://gravitino-iceberg-demo/warehouse',
    'iceberg.rest.vended-credentials-enabled' = 'false',
    's3.access_key' = '<ak>',
    's3.secret_key' = '<sk>',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2'
);
```

2.14.5.6 File System

Iceberg Catalog 和 Paimon Catalog 支持直接从文件系统访问元数据。

相关的连接信息请参阅各自 Catalog 的文档，以及对应的存储系统相关的参数。

2.14.6 存储服务

2.14.6.1 HDFS

本文档用于介绍访问 HDFS 时所需的参数。这些参数适用于：

- Catalog 属性。
- Table Valued Function 属性。
- Broker Load 属性。
- Export 属性。
- Outfile 属性。
- 备份恢复。

2.14.6.1.1 参数总览

属性名称	曾用名	描述	默认值	是否必须

属性名称	曾用名	描述	默认值	是否必须
hdfs.authentication.type	hadoop.security.authentication	用于指定认证类型。可选值为 kerberos 或 simple。如果选择 kerberos，系统将使用 Kerberos 认证同 HDFS 交互；如果使用 simple	simple	否

属性名称	曾用名	描述	默认值	是否必须
hdfs.authentication.kerberos.principal	hadoop.kerberos.principal	当认证类型为 kerberos 时，指定 Kerberos 的 principal。Kerberos principal 是一个唯一标识身份的字符串，通常包括服务名、主机名和域	-	否

属性名称	曾用名	描述	默认值	是否必须
hdfs.authentication.kerberos.keytab	hadoop.kerberos.keytab	该参数指定用于 Kerberos 认证的 keytab 文件路径。keytab 文件用于存储加密的凭证，允许系统自动进行认证，无需用户手动输	-	否

属性名称	曾用名	描述	默认值	是否必须
hdfs.impersonation.enabled	-	如果为true，将开启HDFS的im-per-son-ation功能。会使用core-site.xml中配置的代理用户，来代理Doris的登录用户，执行HDFS操作	尚未支持	-

属性名称	曾用名	描述	默认值	是否必须
hadoop.username	-	当认证类型为 simple 时，会使用此用户来访问 HDFS。默认情况下，会使用运行 Doris 进程的 Linux 系统用户进行访问	-	-

属性名称	曾用名	描述	默认值	是否必须
hadoop.config.resources	-	指定 HDFS 相关配置文件目录（需包含 hdfs-site.xml 和 core-site.xml），需使用相对路径，默认目录为（FE/BE）部署目录下的 /plugins/hadoop/-config/（可修改	-	-

属性名称	曾用名	描述	默认值	是否必须
dfs.nameservices	-	手动配置HDFS高可用集群的参数。若使用hadoop.config.resources配置，则会自动从hdfs-site.xml读取参数。需配合以下参数：dfs.ha.namenodes.your-nameservice、dfs.namenode.rpc-address.your-nameservice.nn1、dfs.client.failover.proxy.provider等	-	-

属性名称	曾用名	描述	默认值	是否必须

3.1 之前的版本，请使用曾用名。

2.14.6.1.2 认证配置

HDFS 支持两种认证方式：即

- Simple
- Kerberos

Simple 认证

Simple 认证适用于未开启 Kerberos 的 HDFS 集群。

使用 Simple 认证方式，可以设置以下参数，或直接使用默认值：

```
"hdfs.authentication.type" = "simple"
```

Simple 认证模式下，可以使用 hadoop.username 参数来指定用户名。如不指定，则默认使用当前进程运行的用户名。

示例：

使用 lakers 用户名访问 HDFS

```
"hdfs.authentication.type" = "simple",
"hadoop.username" = "lakers"
```

使用默认系统用户访问 HDFS

```
"hdfs.authentication.type" = "simple"
```

Kerberos 认证

Kerberos 认证适用于已开启 Kerberos 的 HDFS 集群。

使用 Kerberos 认证方式，需要设置以下参数：

```
"hdfs.authentication.type" = "kerberos",
"hdfs.authentication.kerberos.principal" = "<your_principal>",
"hdfs.authentication.kerberos.keytab" = "<your_keytab>"
```

Kerberos 认证模式下，需要设置 Kerberos 的 principal 和 keytab 文件路径。

Doris 将以该 `hdfs.authentication.kerberos.principal` 属性指定的主体身份访问 HDFS，使用 keytab 指定的 keytab 对该 Principal 进行认证。

> **注意：**
>
> Keytab 文件需要在每个 FE 和 BE 节点上均存在，且路径相同，同时运行 Doris 进程的用户必须具有该 keytab 文件的读权限。

示例：

```
"hdfs.authentication.type" = "kerberos",
"hdfs.authentication.kerberos.principal" = "hdfs/hadoop@HADOOP.COM",
"hdfs.authentication.kerberos.keytab" = "/etc/security/keytabs/hdfs.keytab",
```

2.14.6.1.3 高可用配置（HDFS HA）

如 HDFS 开启了 HA 模式，需要配置 `dfs.nameservices` 相关参数：

```
'dfs.nameservices' = '<your-nameservice>',
'dfs.ha.namenodes.<your-nameservice>' = '<nn1>,<nn2>',
'dfs.namenode.rpc-address.<your-nameservice>.<nn1>' = '<nn1_host:port>',
'dfs.namenode.rpc-address.<your-nameservice>.<nn2>' = '<nn2_host:port>',
'dfs.client.failover.proxy.provider.<your-nameservice>' = 'org.apache.hadoop.hdfs.server.namenode
    ↪ .ha.ConfiguredFailoverProxyProvider',
```

示例：

```
'dfs.nameservices' = 'nameservice1',
'dfs.ha.namenodes.nameservice1' = 'nn1,nn2',
'dfs.namenode.rpc-address.nameservice1.nn1' = '172.21.0.2:8088',
'dfs.namenode.rpc-address.nameservice1.nn2' = '172.21.0.3:8088',
'dfs.client.failover.proxy.provider.nameservice1' = 'org.apache.hadoop.hdfs.server.namenode.ha.
    ↪ ConfiguredFailoverProxyProvider',
```

2.14.6.1.4 配置文件

> 该功能自 3.1.0 版本支持

Doris 支持通过 `hadoop.config.resources` 参数来指定 HDFS 相关配置文件目录。

配置文件目录需包含 hdfs-site.xml 和 core-site.xml 文件，默认目录为（FE/BE）部署目录下的 /plugins/hadoop_conf/。所有 FE 和 BE 节点需配置相同的相对路径。

如果配置文件包含文档上述参数，则优先使用用户显示配置的参数。配置文件可以指定多个文件，多个文件以逗号分隔。如 hadoop/conf/core-site.xml,hadoop/conf/hdfs-site.xml。

示例：

```
-- 多个配置文件
'hadoop.config.resources'='hdfs-cluster-1/core-site.xml,hdfs-cluster-1/hdfs-site.xml'
-- 单个配置文件
'hadoop.config.resources'='hdfs-cluster-2/hdfs-site.xml'
```

2.14.6.1.5 HDFS IO 优化

在某些情况下，HDFS 的负载较高可能导致读取某个 HDFS 上的数据副本的时间较长，从而拖慢整体的查询效率。下面介绍一些相关的优化配置。

Hedged Read

HDFS Client 提供了 Hedged Read 功能。该功能可以在一个读请求超过一定阈值未返回时，启动另一个读线程读取同一份数据，哪个先返回就是用哪个结果。

注意：该功能可能会增加 HDFS 集群的负载，请酌情使用。

可以通过以下方式开启这个功能：

```
"dfs.client.hedged.read.threadpool.size" = "128",
"dfs.client.hedged.read.threshold.millis" = "500"
```

- dfs.client.hedged.read.threadpool.size

  表示用于 Hedged Read 的线程数，这些线程由一个 HDFS Client 共享。通常情况下，针对一个 HDFS 集群，BE 节点会共享一个 HDFS Client。

- dfs.client.hedged.read.threshold.millis

  读取阈值，单位毫秒。当一个读请求超过这个阈值未返回时，会触发 Hedged Read。

开启后，可以在 Query Profile 中看到相关参数：

- TotalHedgedRead

  发起 Hedged Read 的次数。

- HedgedReadWins

  Hedged Read 成功的次数（发起并且比原请求更快返回的次数）

注意，这里的值是单个 HDFS Client 的累计值，而不是单个查询的数值。同一个 HDFS Client 会被多个查询复用。

dfs.client.socket-timeout

`dfs.client.socket-timeout` 是 Hadoop HDFS 中的一个客户端配置参数，用于设置客户端与 DataNode 或 NameNode 之间建立连接或读取数据时的套接字（socket）超时时间，单位为毫秒。该参数的默认值通常为 60,000 毫秒。

将该参数的值调小，可以使客户端在遇到网络延迟、DataNode 响应慢或连接异常等问题时，更快地超时并进行重试或切换到其他节点。这有助于减少等待时间，提高系统的响应速度。例如，在某些测试中，将 `dfs.client.socket-timeout` 设置为较小的值（如 5000 毫秒），可以迅速检测到 DataNode 的延迟或故障，从而避免长时间的等待。

注意：

- 将超时时间设置得过小可能导致在网络波动或节点负载较高时频繁出现超时错误，影响任务的稳定性。
- 建议根据实际网络环境和系统负载情况，合理调整该参数的值，以在响应速度和系统稳定性之间取得平衡。
- 该参数应在客户端配置文件（如 `hdfs-site.xml`）中设置，确保客户端在与 HDFS 通信时使用正确的超时时间。

总之，合理配置 `dfs.client.socket-timeout` 参数，可以在提高 I/O 响应速度的同时，确保系统的稳定性和可靠性。

2.14.6.1.6 调试 HDFS

Hadoop 环境配置复杂，某些情况下可能出现无法连通、访问性能不佳等问题。这里提供一些第三方工具帮助用户快速排查连通性问题和基础的性能问题。

HDFS Client

- Java:https://github.com/morningman/hdfs-client-java
- CPP: https://github.com/morningman/hdfs-client-cpp

这两个工具可以用于快速验证 HDFS 连通性和读取性能。其中的大部分 Hadoop 依赖项和 Doris 本身的 Hadoop 依赖相同，因此可以最大程度模拟 Doris 访问 HDFS 的场景。

Java 版本使用通过 Java 访问 HDFS，可以模拟 Doris FE 侧访问 HDFS 的逻辑。

CPP 版本通过 C++ 调用 libhdfs 访问 HDFS，可以模拟 Doris BE 侧访问 HDFS 的逻辑。

具体使用方式可以各自代码库的 README。

2.14.6.2 S3

本文档用于介绍访问 AWS S3 时所需的参数。这些参数适用于：

- Catalog 属性。
- Table Valued Function 属性。
- Broker Load 属性。
- Export 属性。
- Outfile 属性。

2.14.6.2.1 参数总览

属性名称	曾用名	描述	默认值	是否必须
s3.endpoint		S3 服务访问地址，如 s3.us-east-1.amazonaws.com	无	否
s3.access_key		AWS Access Key。用于身份验证	无	否
s3.secret_key		AWS Secret Key。用于身份验证	无	否
s3.region		S3 所在的区域，例如：us-east-1。强烈建议配置	无	是
s3.use_path_style		是否使用 path-style（路径风格）访问。	FALSE	否
s3.connection.maximum		最大连接数，适用于高并发场景	50	否
s3.connection.request.timeout		请求超时时间（毫秒），控制连接获取超时	3000	否
s3.connection.timeout		建立连接的超时时间（毫秒）	1000	否
s3.role_arn		使用 Assume Role 模式时指定的角色 ARN	无	否
s3.external_id		配合 s3.role_arn 使用的 external ID	无	否

2.14.6.2.2 认证配置

Doris 支持以下两种方式访问 S3：

1. 直接使用 Access Key 和 Secret Key

```
"s3.access_key"="your-access-key",
"s3.secret_key"="your-secret-key",
"s3.endpoint"="s3.us-east-1.amazonaws.com",
"s3.region"="us-east-1"
```

2. Assume Role 模式

适用于跨账号、临时授权访问。通过角色授权自动获取临时凭证。

```
"s3.role_arn"="arn:aws:iam::123456789012:role/demo-role",
"s3.external_id"="external-identifier",
"s3.endpoint"="s3.us-east-1.amazonaws.com",
"s3.region"="us-east-1"
```

> 如果同时设置了 Access Key 和 Role ARN，则优先使用 Access Key 模式。

2.14.6.2.3 访问 S3 Directory Bucket

> 该功能自 3.1.0 起支持。

Amazon S3 Express One Zone（又名 Directory Bucket）提供更高性能，但 endpoint 格式不同。

- 普通 bucket：s3.us-east-1.amazonaws.com
- Directory Bucket：s3express-usw2-az1.us-west-2.amazonaws.com

更多可用区域参考：AWS 官方文档

示例：

```
"s3.access_key"="ak",
"s3.secret_key"="sk",
"s3.endpoint"="s3express-usw2-az1.us-west-2.amazonaws.com",
"s3.region"="us-west-2"
```

2.14.6.2.4 权限策略

根据使用场景不同，可以分为 只读和 读写两类策略。

1. 只读权限

只允许读取 S3 中的对象。适用于 LOAD、 TVF、 查询 EXTERNAL CATALOG 等场景。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
              "s3:GetObject",
              "s3:GetObjectVersion",
            ],
            "Resource": "arn:aws:s3:::<your-bucket>/your-prefix/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::<your-bucket>"
        }
    ]
}
```

2. 读写权限

在只读的基础上，允许删除、创建、修改对象。适用于 EXPORT、 OUTFILE 以及 EXTERNAL CATALOG 回写等场景。

```
{
    "Version": "2012-10-17",
    "Statement": [
```

```
        {
            "Effect": "Allow",
            "Action": [
              "s3:PutObject",
              "s3:GetObject",
              "s3:GetObjectVersion",
              "s3:DeleteObject",
              "s3:DeleteObjectVersion",
              "s3:AbortMultipartUpload",
              "s3:ListMultipartUploadParts"
            ],
            "Resource": "arn:aws:s3:::<your-bucket>/<your-prefix>/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetBucketVersioning",
                "s3:GetLifecycleConfiguration"
            ],
            "Resource": "arn:aws:s3:::<your-bucket>"
        }
    ]
}
```

注意事项

1. 占位符替换
    - <bucket> → 你的 S3 Bucket 名称。
    - <account-id> → 你的 AWS 账号 ID（12 位数字）。
2. 最小权限原则
    - 如果只做查询，不要授予写权限。

2.14.6.3 Azure Blob

自 3.1.3 版本起，Doris 支持访问 Azure Blob 存储。

本文档介绍访问 Microsoft Azure Blob 存储所需的参数，这些参数适用于以下场景：

- Catalog 属性
- Table Valued Function 属性
- Broker Load 属性
- Export 属性

- Outfile 属性
- Backup / Restore 属性

2.14.6.3.1 参数总览

属性名称	曾用名	描述	默认值	是否必
azure.account_name		Azure 存储账户名称（Account Name），即在 Azure 门户中创建的存储账户名称。		是
azure.account_key		Azure Blob 存储的 Account Key		是
azure.endpoint		Azure Blob 存储的访问端点，格式通常为 https://.blob.core.windows.net		是
fs.azure.support		是否启用 Azure Blob 存储	true	是

- 启用 Azure Blob 存储

必须显示的配置 `"provider" = "AZURE"` 或 `"fs.azure.support" = "true"` 以表明启用 Azure Blob 存储。

- 获取 `azure.account_name`

1. 登录 Azure 门户
2. 打开 存储帐户 (Storage Accounts)，选择目标账户
3. 在 概述 (Overview) 页面可看到 存储帐户名称 (Account Name)

```
"azure.account_name" = "myblobstorage"
```

- 获取 `azure.account_key`

1. 登录 Azure 门户
2. 打开 存储帐户 (Storage Accounts)，选择目标账户
3. 在左侧导航栏选择 访问密钥 (Access keys)
4. 在 key1 或 key2 中点击「显示密钥」，复制 Key 值

```
"azure.account_key" = "EXAMPLE_I_A...=="
```

2.14.6.4 Google Cloud Storage

文档更新中。

2.14.6.5 Aliyun OSS

本文档介绍访问阿里云 OSS 所需的参数，这些参数适用于以下场景：

- Catalog 属性
- Table Valued Function 属性
- Broker Load 属性
- Export 属性
- Outfile 属性

2.14.6.5.1 OSS

Doris 使用 S3 Client，通过 S3 兼容协议访问阿里云 OSS。

参数总览

属性名称	曾用名	描述	默认值
oss.endpoint	s3.endpoint	OSS endpoint，指定阿里云 OSS 的访问端点。注意，OSS 和 OSS HDFS 的 endpoint 不相同。	无
oss.access_key	s3.access_key	OSS Access Key，用于身份验证	无
oss.secret_key	s3.secret_key	OSS Secret Key，与 Access Key 配合使用	无
oss.region	s3.region	OSS region，指定阿里云 OSS 的区域	无
oss.use_path_style	s3.use_path_style	是否使用 path-style（路径风格）访问。兼容 MinIO 等非 AWS S3 服务建议设置为 true	FALSE
oss.connection.maximum	s3.connection.maximum	最大连接数，指定与 OSS 服务建立的最大连接数	50
oss.connection.request.timeout	s3.connection.request.timeout	请求超时时间（毫秒），指定连接 OSS 服务时的请求超时时间	3000
oss.connection.timeout	s3.connection.timeout	连接超时时间（毫秒），指定与 OSS 服务建立连接时的超时时间	1000

> 3.1 版本之前，请使用曾用名。

示例配置

```
"oss.access_key" = "your-access-key",
"oss.secret_key" = "your-secret-key",
"oss.endpoint" = "oss-cn-beijing.aliyuncs.com",
"oss.region" = "cn-beijing"
```

3.1 之前的版：

```
"s3.access_key" = "your-access-key",
"s3.secret_key" = "your-secret-key",
"s3.endpoint" = "oss-cn-beijing.aliyuncs.com",
"s3.region" = "cn-beijing"
```

使用建议

- 推荐使用 `oss.` 前缀配置参数，保证与阿里云 OSS 的一致性和清晰度。
- 3.1 之前的版本，请使用曾用名 `s3.` 作为前缀。
- 配置 `oss.region` 能提升访问的准确性和性能，建议设置。
- 连接池参数可根据并发需求调整，避免连接阻塞。

2.14.6.5.2 OSS-HDFS

OSS-HDFS 服务（JindoFS 服务）是一个阿里云云原生数据湖存储功能。基于统一的元数据管理能力，兼容 HDFS 文件系统接口，满足大数据和 AI 等领域的数据湖计算场景。

访问 OSS-HDFS 上存储的数据，和直接访问 OSS 服务稍有区别，详见本文档。

参数总览

属性名称	曾用名	描述	默认值	是否必须
oss.hdfs.endpoint	oss.endpoint	阿里云 OSS-HDFS 服务的 Endpoint，例如 cn-hangzhou.oss-dls.aliyuncs.com。	无	是
oss.hdfs.access_key	oss.access_key	OSS Access Key，用于身份验证	无	是
oss.hdfs.secret_key	oss.secret_key	OSS Secret Key，与 Access Key 配合使用	无	是
oss.hdfs.region	oss.region	OSS bucket 所在的地域 ID，例如 cn-beijing。	无	是
oss.hdfs.fs.defaultFS		3.1 版本支持。指定 OSS 的文件系统访问路径，例如 oss://my-bucket/。	无	否
oss.hdfs.hadoop.config.resources		3.1 版本支持。指定包含 OSS 文件系统配置的路径，需使用相对路径，默认目录为（FE/BE）部署目录下的 /plugins/hadoop_conf/（可修改 fe.conf/be.conf 中的 hadoop_config_dir 来更改默认路径）。所有 FE 和 BE 节点需配置相同相对路径。示例：hadoop/conf/core-site.xml, ↪ hadoop/conf/hdfs-site.xml。	无	否
fs.oss-hdfs.support	oss.hdfs.enabled	3.1 版本支持。显示声明启用 OSS-HDFS 功能。需要设置为 true	无	否

> 3.1 版本之前，请使用曾用名。

Endpoint 配置

oss.hdfs.endpoint: 用于指定 OSS-HDFS 服务的 Endpoint。

Endpoint 是访问阿里云 OSS 的入口地址，格式为 <region>.oss-dls.aliyuncs.com，例如 cn-hangzhou.oss-dls ↪ .aliyuncs.com。

我们会对格式进行强校验，确保 Endpoint 符合阿里云 OSS Endpoint 格式。

为保证向后兼容，Endpoint 配置项允许包含 https:// 或 http:// 前缀，系统在格式校验时会自动解析并忽略协议部分。

如使用曾用名，则系统会根据 endpoint 中是否包含 oss-dls 判断是否是 OSS-HDFS 服务。

配置文件

3.1 版本支持

OSS-HDFS 支持通过 oss.hdfs.hadoop.config.resources 参数来指定 HDFS 相关配置文件目录。

配置文件目录需包含 hdfs-site.xml 和 core-site.xml 文件，默认目录为（FE/BE）部署目录下的 /plugins/hadoop_conf/。所有 FE 和 BE 节点需配置相同的相对路径。

如果配置文件包含文档上述参数，则优先使用用户显示配置的参数。配置文件可以指定多个文件，多个文件以逗号分隔。如 hadoop/conf/core-site.xml,hadoop/conf/hdfs-site.xml。

示例配置

```
"fs.oss-hdfs.support" = "true",
"oss.hdfs.access_key" = "your-access-key",
"oss.hdfs.secret_key" = "your-secret-key",
"oss.hdfs.endpoint" = "cn-hangzhou.oss-dls.aliyuncs.com",
"oss.hdfs.region" = "cn-hangzhou"
```

3.1 之前的版本：

```
"oss.hdfs.enabled" = "true",
"oss.access_key" = "your-access-key",
"oss.secret_key" = "your-secret-key",
"oss.endpoint" = "cn-hangzhou.oss-dls.aliyuncs.com",
"oss.region" = "cn-hangzhou"
```

2.14.6.6 Tencent COS

本文档介绍访问腾讯云 COS 所需的参数，这些参数适用于以下场景：

- Catalog 属性
- Table Valued Function 属性
- Broker Load 属性
- Export 属性
- Outfile 属性

Doris 使用 S3 Client，通过 S3 兼容协议访问腾讯云 COS。

2.14.6.6.1 参数总览

属性名称	曾用名	描述	默认值	是否必须
cos.endpoint	s3.endpoint	COS endpoint，指定腾讯云 COS 的访问端点		是
cos.access_key	s3.access_key	COS access key，用于身份验证的 COS 访问密钥		是
cos.secret_key	s3.secret_key	COS secret key，与 access key 配合使用的访问密钥		是
cos.region	s3.region	COS region，指定腾讯云 COS 的区域		否
cos.connection.maximum	s3.connection.maximum	S3 最大连接数，指定与 COS 服务建立的最大连接数	50	否
cos.connection.request.timeouts	3.connection.timeout	S3 请求超时时间，单位为毫秒，指定连接 COS 服务时的请求超时时间	3000	否
cos.connection.timeout	s3.connection.timeout	S3 连接超时时间，单位为毫秒，指定与 COS 服务建立连接时的超时时间	1000	否

3.1 版本之前，请使用曾用名。

2.14.6.6.2 示例配置

```
"cos.access_key" = "your-access-key",
"cos.secret_key" = "your-secret-key",
"cos.endpoint" = "cos.ap-beijing.myqcloud.com",
"cos.region" = "ap-beijing"
```

3.1 之前的版：

```
"s3.access_key" = "ak",
"s3.secret_key" = "sk",
"s3.endpoint" = "cos.ap-beijing.myqcloud.com",
"s3.region" = "ap-beijing"
```

2.14.6.6.3 使用建议

- 推荐使用 `cos.` 前缀配置参数，保证与腾讯云 COS 的一致性和清晰度。
- 3.1 之前的版本，请使用曾用名 `s3.` 作为前缀。
- 配置 `cos.region` 能提升访问的准确性和性能，建议设置。
- 连接池参数可根据并发需求调整，避免连接阻塞。

2.14.6.7 Huawei OBS

本文档介绍访问华为云 OBS 所需的参数，这些参数适用于以下场景：

- Catalog 属性
- Table Valued Function 属性
- Broker Load 属性
- Export 属性
- Outfile 属性

Doris 使用 S3 Client，通过 S3 兼容协议访问华为云 OBS。

2.14.6.7.1 参数总览

属性名称	曾用名	描述	默认值	是否必须
obs.endpoint	s3.endpoint	OBS endpoint，指定华为云 OBS 的访问端点		是
obs.access_key	s3.access_key	OBS access key，用于身份验证的 OBS 访问密钥		是
obs.secret_key	s3.secret_key	OBS secret key，与 access key 配合使用的访问密钥		是
obs.region	s3.region	OBS region，指定华为云 OBS 的区域		否
obs.use_path_style	s3.use_path_style	是否使用 path-style（路径风格）访问。兼容 MinIO/Ceph 等非 AWS S3 服务建议设置为 true	FALSE	否
obs.connection.maximum	s3.connection.maximum	最大连接数，指定与 OBS 服务建立的最大连接数	50	否
obs.connection.request.timeout	s3.connection.request.timeout	请求超时时间，单位为毫秒，指定连接 OBS 服务时的请求超时时间	3000	否
obs.connection.timeout	s3.connection.timeout	连接超时时间，单位为毫秒，指定与 OBS 服务建立连接时的超时时间	1000	否

> 3.1 版本之前，请使用曾用名。

2.14.6.7.2 示例配置

```
"obs.access_key" = "your-access-key",
"obs.secret_key" = "your-secret-key",
"obs.endpoint" = "obs.cn-north-4.myhuaweicloud.com",
"obs.region" = "cn-north-4"
```

3.1 之前的版：

```
"s3.access_key" = "your-access-key",
"s3.secret_key" = "your-secret-key",
"s3.endpoint" = "obs.cn-north-4.myhuaweicloud.com",
"s3.region" = "cn-north-4",
```

2.14.6.7.3 使用建议

- 推荐使用 `obs.` 前缀配置参数，保证与华为云 OBS 的一致性和清晰度。
- 3.1 之前的版本，请使用曾用名 `s3.` 作为前缀。
- 配置 `obs.region` 能提升访问的准确性和性能，建议设置。
- 连接池参数可根据并发需求调整，避免连接阻塞。

2.14.6.8 Baidu BOS

文档更新中。

2.14.6.9 MinIO

本文档介绍访问 MinIO 所需的参数，这些参数适用于以下场景：

- Catalog 属性
- Table Valued Function 属性
- Broker Load 属性
- Export 属性
- Outfile 属性

Doris 使用 S3 Client，通过 S3 兼容协议访问 MinIO。

2.14.6.9.1 参数总览

属性名称	曾用名	描述	默认值	是否必须
minio.endpoint	s3.endpoint	Minio endpoint，Minio 的访问端点		是
minio.access_key	s3.access_key	Minio access key，用于身份验证的 Minio 访问密钥		是
minio.secret_key	s3.secret_key	Minio secret key，与 access key 配合使用的访问密钥		是
minio.connection.maximum	s3.connection.maximum	S3 最大连接数，指定与 Minio 服务建立的最大连接数	50	否
minio.connection.request.timeout	s3.connection.timeout	S3 请求超时时间，单位为毫秒，指定连接 Minio 服务时的请求超时时间	3000	否
minio.connection.timeout	s3.connection.timeout	S3 连接超时时间，单位为毫秒，指定与 Minio 服务建立连接时的超时时间	1000	否
minio.use_path_style	s3.use_path_style	是否使用 path-style（路径风格）访问。兼容 MinIO 等非 AWS S3 服务建议设置为 true	FALSE	否

使用 Path-style 访问

Minio 默认使用 Host-style 访问方式，但也支持 Path-style 访问。可以通过设置 `minio.use_path_style` 参数来切换。

- Host-style 访问（默认）: https://bucket.minio.example.com
- Path-style 访问（开启后）: https://minio.example.com/bucket

2.14.6.9.2 示例配置

```
"minio.access_key" = "your-access-key",
"minio.secret_key" = "your-secret-key",
"minio.endpoint" = "http://minio.example.com:9000"
```

3.1 之前的版本：

```
"s3.access_key" = "your-access-key",
"s3.secret_key" = "your-secret-key",
"s3.endpoint" = "http://minio.example.com:9000"
```

2.14.6.9.3 使用建议

- 推荐使用 `minio.` 前缀配置参数，保证与 MinIO 的一致性和清晰度。
- 3.1 之前的版本，请使用曾用名 `s3.` 作为前缀。
- 连接池参数可根据并发需求调整，避免连接阻塞。

2.14.7 开放文件格式

2.14.7.1 Parquet

本文档用于介绍 Doris 的 Parquet 文件格式的读写支持情况。该文档适用于以下功能。

- Catalog 中对数据的读取、写入操作。
- Table Valued Function 中对数据的读取操作。
- Broker Load 中对数据的读取操作。
- Export 中对数据的写入操作。
- Outfile 中对数据的写入操作。

2.14.7.1.1 支持的压缩格式

- umcomressed
- snappy
- lz4
- zstd
- gzip
- lzo
- brotli

2.14.7.1.2 相关参数

会话变量

- `enable_parquet_lazy_mat` (2.1+, 3.0+)

  控制 Parquet Reader 是否启用延迟物化技术。默认为 true。

- `hive_parquet_use_column_names` (2.1.6+, 3.0.3+)

  Doris 在读取 Hive 表 Parquet 数据类型时，默认会根据 Hive 表的列名从 Parquet 文件中找同名的列来读取数据。当该变量为 `false` 时，Doris 会根据 Hive 表中的列顺序从 Parquet 文件中读取数据，与列名无关。类似于 Hive 中的 `parquet.column.index.access` 变量。该参数只适用于顶层列名，对 Struct 内部无效。

BE 配置

- `enable_parquet_page_index` (2.1.5+, 3.0+)

  Parquet Reader 是否采用 Page Index 去过滤数据。这仅用于调试目的，以防页面索引有时过滤错误的数据。默认值为 false.

- `parquet_header_max_size_mb` (2.1+, 3.0+)

  读取 Parquet Page header 时所分配的最大 Buffer 大小，默认为 1M。

- `parquet_rowgroup_max_buffer_mb` (2.1+, 3.0+)

  读取 Parquet Row Group 时所分配的最大 Buffer 大小，默认为 128M。

- `parquet_column_max_buffer_mb` (2.1+, 3.0+)

  读取 Parquet Row Group 中的 Column 时所分配的最大 Buffer 大小，默认为 8M。

2.14.7.2 ORC

本文档用于介绍 Doris 的 ORC 文件格式的读写支持情况。该文档适用于以下功能。

- Catalog 中对数据的读取、写入操作。
- Table Valued Function 中对数据的读取操作。
- Broker Load 中对数据的读取操作。
- Export 中对数据的写入操作。
- Outfile 中对数据的写入操作。

2.14.7.2.1 支持的压缩格式

- umcomressed
- snappy
- lz4
- zstd
- lzo
- zlib

2.14.7.2.2 相关参数

会话变量

- `enable_orc_lazy_mat` (2.1+, 3.0+)

  控制 ORC Reader 是否启用延迟物化技术。默认为 true。

- `hive_orc_use_column_names` (2.1.6+, 3.0.3+)

  Doris 在读取 Hive 表 ORC 数据类型时，默认会根据 Hive 表的列名从 ORC 文件中找同名的列来读取数据。当该变量为 `false` 时，Doris 会根据 Hive 表中的列顺序从 Parquet 文件中读取数据，与列名无关。类似于 Hive 中的 `orc.force.positional.evolution` 变量。该参数只适用于顶层列名，对 Struct 内部无效。

- `orc_tiny_stripe_threshold_bytes` (2.1.8+, 3.0.3+)

  在 ORC 文件中如果一个 Stripe 的字节大小小于 `orc_tiny_stripe_threshold`, 我们认为该 Stripe 为 Tiny Stripe。对于多个连续的 Tiny Stripe 我们会进行读取优化，即一次性读多个 Tiny Stripe 以减少 IO 次数。如果你不想使用该优化，可以将该值设置为 0。默认为 8M。

- `orc_once_max_read_bytes` (2.1.8+, 3.0.3+)

  在使用 Tiny Stripe 读取优化的时候，会对多个 Tiny Stripe 合并成一次 IO，该参数用来控制每次 IO 请求的最大字节大小。你不应该将值设置的小于 `orc_tiny_stripe_threshold`。默认为 8M。

- `orc_max_merge_distance_bytes` (2.1.8+, 3.0.3+)

  在使用 Tiny Stripe 读取优化的时候，由于需要读取的两个 Tiny Stripe 并不一定连续，当两个 Tiny Stripe 之间距离大于该参数时，我们不会将其合并成一次 IO。默认为 1M。

- `orc_tiny_stripe_amplification_factor` (3.1.0+)

  在 Tiny Stripe 优化中，如果 ORC 文件中的列较多，而查询中只使用其中的少数列，Tiny Stripe 优化会导致严重的读取放大。当实际读取的字节数占整个 sSripe 的比例大于该参数时，将使用 Tiny Stripe 读取优化。该参数的默认值为 0.4，最小值为 0。

- `check_orc_init_sargs_success` (3.1.0+)

  检查 ORC 谓词下推是否成功，用于调试。默认为 false。

BE 配置项

- `orc_natural_read_size_mb` (2.1+, 3.0+)

  ORC Reader 一次性读取的最大字节大小。默认 8 MB。

2.14.7.3 Text/CSV/JSON

本文档用于介绍 Doris 的文本文件格式的读写支持情况。

2.14.7.3.1 Text/CSV

- Catalog

支持读取 `org.apache.hadoop.mapred.TextInputFormat` 格式的 Hive 表。

支持以下 Serde：

- `org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe`
- `org.apache.hadoop.hive.serde2.OpenCSVSerde` (2.1.7 版本支持)
- `org.apache.hadoop.hive.serde2.MultiDelimitSerDe` (3.1.0 版本支持)
- Table Valued Function
- 导入

导入功能支持的 Text/CSV 格式，详见导入相关文档。

- 导出

导出功能支持的 Text/CSV 格式，详见导出相关文档。

支持的压缩格式

- umcomressed
- gzip
- deflate
- bzip2
- zstd
- lz4
- snappy
- lzo

2.14.7.3.2 JSON

Catalog

- `org.apache.hadoop.hive.serde2.JsonSerDe`（3.0.4 版本支持）
- `org.apache.hive.hcatalog.data.JsonSerDe`（3.0.4 版本支持）

1. 支持普通类型和复杂类型。
2. 不支持 `timestamp.formats` SERDEPROPERTIES

- `org.openx.data.jsonserde.JsonSerDe`（3.0.6 版本支持）

1. 支持普通类型和复杂类型。
2. SERDEPROPERTIES: 只支持 `ignore.malformed.json` 且行为与该 JsonSerDe 一致，其他 SERDEPROPERTIES 不生效。
3. 不支持`Using Arrays` (类似于 Text/CSV, 将所有列的数据放一个数组中)。
4. 不支持`Promoting a Scalar to an Array` (提升标量返回一个的单元素数组)。
5. 默认情况下，Doris 会正常识别表的 Schema。但因为某些特殊参数不支持，可能导致自动识别 Schema 失败。此时可以通过`set read_hive_json_in_one_column = true`, 将一整行 Json 数据都放到第一列中，这样可以确保原始数据被完整读取，用户可以自行处理。该功能要求第一列的数据类型为 String.

导入

导入功能支持的 JSON 格式，详见导入相关文档。

2.14.7.3.3 字符集

Doris 目前仅支持 UTF-8 编码的字符集。而某些数据，如 Hive Text 格式表中的数据会包含非 UFT-8 编码的内容，会导致读取失败，并报错：

```
Only support csv data in utf8 codec
```

此时，可以通过设置会话变量：

```
SET enable_text_validate_utf8 = false
```

来忽略 UFT-8 编码检查，以便能够读取这些内容。注意，这个参数仅用于忽略检查，非 UTF-8 编码的内容仍会显示为乱码。

此参数自 3.0.4 版本支持。

2.14.8 数据缓存

数据缓存（Data Cache）通过缓存最近访问的远端存储系统（HDFS 或对象存储）的数据文件到本地磁盘上，加速后续访问相同数据的查询。在频繁访问相同数据的查询场景中，Data Cache 可以避免重复的远端数据访问开销，提升热点数据的查询分析性能和稳定性。

2.14.8.1 适用场景

数据缓存功能仅作用于 Hive、Iceberg、Hudi、Paimon 表的查询。对内表查询，或非文件的外表查询（如 JDBC、Elasticsearch）等无效果。

数据缓存是否能提升查询效率，取决于多方面因素，下面给出数据缓存的适用场景：

- 高速本地磁盘

建议使用高速本地磁盘，如 SSD 或 NVME 介质的本地磁盘作为数据缓存目录。不建议使用机械硬盘作为数据缓存目录。本质上，需确保本地磁盘的 IO 带宽和 IOPS 显著高于网络带宽、源端存储系统的 IO 带宽和 IOPS，才可能带来明显的性能提升。

- 足够的缓存空间大小

数据缓存使用 LRU 策略作为缓存淘汰策略。如果查询的数据并没有明显的冷热区分，则缓存数据有可能处于频繁的更新和汰换过程中，反而可能降低查询性能。推荐查询模式有明显冷热区分（如大部分查询只访问当天的数据，几乎不访问历史数据），并且缓存空间足够存储热数据的场景下开启数据缓存。

- 远端存储的 IO 延迟不稳定

这种情况通常出现在 HDFS 存储上。多数企业中不同的业务部门会共用同一套 HDFS，因此可能导致高峰期 HDFS 的 IO 延迟非常不稳定。这种情况下，如需确保 IO 延迟稳定，建议开启数据缓存。但仍需考虑前两种情况。

2.14.8.2 开启数据缓存

数据缓存功能是默认关闭的，需要在 FE 和 BE 中设置相关参数进行开启。

2.14.8.2.1 BE 配置

首先，需要在 be.conf 中配置缓存路径信息，并重启 BE 节点让配置生效。

参数	必选项	说明
enable_file_cache	是	是否启用 Data Cache，默认 false
file_cache_path	是	缓存目录的相关配置，json 格式。
clear_file_cache	否	默认 false。如果为 true，则当 BE 节点重启时，会清空缓存目录。

file_cache_path 的配置示例：

```
file_cache_path=[{"path": "/path/to/file_cache1", "total_size":53687091200},{"path": "/path/to/
    ↪ file_cache2", "total_size":53687091200},{"path": "/path/to/file_cache3", "total_size"
    ↪ :53687091200}]
```

path 是缓存的保存路径，可以配置一个或多个。建议同一块磁盘只配置一个路径。

total_size 是缓存的空间大小上限。单位是字节。超过缓存空间后，会通过 LRU 策略进行缓存数据的淘汰。

2.14.8.2.2 FE 配置

单个会话中开启 Data Cache:

```
SET enable_file_cache = true;
```

全局开启 Data Cache:

```
SET GLOBAL enable_file_cache = true;
```

注意，如果没有开启 enable_file_cache，即使 BE 配置了缓存目录，也不会使用缓存。同样，如果 BE 没有配置缓存目录，即使开启 enable_file_cache，也不会使用缓存。

2.14.8.3 缓存可观测性

2.14.8.3.1 查看缓存命中情况

执行 `set enable_profile=true` 打开会话变量，可以在 FE 的 web 页面的 Queris 标签中查看到作业的 Profile。数据缓存相关的指标如下：

```
-  FileCache:  0ns
    -  BytesScannedFromCache:  2.02  GB
    -  BytesScannedFromRemote:  0.00
    -  BytesWriteIntoCache:  0.00
    -  LocalIOUseTimer:  2s723ms
    -  NumLocalIOTotal:  444
    -  NumRemoteIOTotal:  0
    -  NumSkipCacheIOTotal:  0
    -  RemoteIOUseTimer:  0ns
    -  WriteCacheIOUseTimer:  0ns
```

- `BytesScannedFromCache`：从本地缓存中读取的数据量。
- `BytesScannedFromRemote`：从远端读取的数据量。
- `BytesWriteIntoCache`：写入缓存的数据量。
- `LocalIOUseTimer`：本地缓存的 IO 时间。
- `RemoteIOUseTimer`：远端读取的 IO 时间。
- `NumLocalIOTotal`：本地缓存的 IO 次数。
- `NumRemoteIOTotal`：远端 IO 次数。
- `WriteCacheIOUseTimer`：写入缓存的 IO 时间。

如果 `BytesScannedFromRemote` 为 `0`，表示全部命中缓存。

2.14.8.3.2 监控指标

用户可以通过系统表`file_cache_statistics` 查看各个 Backend 节点的缓存统计指标。

2.14.8.4 附录

2.14.8.4.1 原理

数据缓存将访问的远程数据缓存到本地的 BE 节点。原始的数据文件会根据访问的 IO 大小切分为 Block，Block 被存储到本地文件 `cache_path/hash(filepath).substr(0, 3)/hash(filepath)/offset` 中，并在 BE 节点中保存 Block 的元信息。当访问相同的远程文件时，doris 会检查本地缓存中是否存在该文件的缓存数据，并根据 Block 的 offset 和 size，确认哪些数据从本地 Block 读取，哪些数据从远程拉起，并缓存远程拉取的新数据。BE 节点重启的时候，扫描 `cache_path` 目录，恢复 Block 的元信息。当缓存大小达到阈值上限的时候，按照 LRU 原则清理长久未访问的 Block。

2.14.9 元数据缓存

为了提升访问外部数据源的性能，Apache Doris 会对外部数据源的元数据进行缓存。

元数据包括库、表、列信息、分区信息、快照信息、文件列表等。

本文详细介绍缓存的元数据的种类、策略和相关参数配置。

关于数据缓存，可参阅数据缓存文档。

> 该文档适用于 2.1.6 之后的版本。

2.14.9.1 缓存策略

大多数缓存都有如下三个策略指标：

- 最大缓存数量

  缓存所能容纳的最大对象个数。如最多缓存 1000 张表。当缓存数量超过阈值后，会使用 LRU（Least-Recent-Used）策略移除部分缓存。

- 淘汰时间

  - 3.0.6（含）版本前：

    缓存对象写入缓存一段时间后，该对象会被自动从缓存中移除，下次访问时，会重新从数据源拉取最新的信息并更新缓存。

    比如用户在 08:00 第一访问表 A，并写入缓存。若淘汰时间为 4 小时。则在没有因容量问题被汰换的情况下，用户在之后的 08:00-14:00 之间，都会直接访问缓存中的表 A。14:00 后，缓存被淘汰。若用户再次访问表 A，会从数据源拉取最新的信息并更新缓存。

  - 3.0.7（含）版本后：

    3.0.7 版本开始，此策略修改为 缓存对象被访问一段时候，该对象会被自动从缓存中移除。而不是写入一段时间后。每次缓存对象被访问，都会重新计时，以确保频繁被访问的对象始终在缓存中。

    比如用户在 08:00 第一访问表 A，并写入缓存。若淘汰时间为 4 小时。则在没有因容量问题被汰换的情况下，用户在之后的 08:00-14:00 之间，都会直接访问缓存中的表 A。假设用户在 09:00 时又访问了这个对象，则缓存淘汰时间将重新从 09:00 开始计算，即变为 15:00。

- 最短刷新时间

  缓存对象写入缓存一段时间后，会自动触发刷新。

  比如用户在 08:00 第一访问表 A，并写入缓存。若最短刷新时间为 10 分钟。则在没有因容量问题被汰换的情况下，用户在之后的 08:00-8:10 之间，都会直接访问缓存中的表 A。08:10，该缓存对象会被标记为【准备刷新】，当用户再次访问这个缓存对象时，仍会返回当前对象，但会同时触发缓存刷新操作。假设缓存更新需要 1 分钟，则 1 分钟后再次访问缓存，会得到更新后的缓存对象。

  注意，触发缓存刷新的时间是在【超过最短刷新时间后，第一次访问该缓存对象时】，并且是异步刷新。所以比如最短刷新时间是 10 分钟，并不意味着 10 分钟后一定会获取到最新的对象。

  该策略有别于【淘汰时间】，主要用于调整缓存的时效性，并且通过异步刷新的方式避免缓存更新阻塞当前操作。

2.14.9.2 缓存类型

2.14.9.2.1 库、表名称列表

库名称列表（Database name list）指的是一个 Catalog 下所有库的名称的列表。

表名称列表（Table name list）指的是一个库下所有表的名称列表。

名称列表仅用于需要列举名称得操作，如 SHOW TABLES 或 SHOW DATABASES 语句。

每个 Catalog 下都一个库名称列表缓存。每个库下都有一个表名称列表缓存。

- 最大缓存数量

  每个缓存有且仅有一个条目。所以最大缓存数量为 1。

- 淘汰时间

  固定 86400 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中查看到最新的名称列表，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.2 库、表对象

缓存单独的库和表对象。任何对库、表的访问操作，如查询、写入等，都会从这个缓存中获取对应的对象。

注意，该缓存中的对象所组成的列表，可能与库、表名称列表缓存中的不一致。

比如通过 SHOW TABLES 命令，从名称列表缓存中获取到 A、B、C 三个表。假设此时外部数据源增加了表 D，那么 SELECT * FROM D 可以访问到表 D，同时【表对象】缓存里会增加表 D 对象，但【表名称列表】缓存中可能依然是 A、B、C。只有当【表名称列表】缓存刷新后，才会变成 A、B、C、D。

每个 Catalog 下都一个库名称列表缓存。每个库下都有一个表名称列表缓存。

- 最大缓存数量

  由 FE 配置项 max_meta_object_cache_num 控制，默认为 1000。可以根据单个 Catalog 下数据库的数量，或单个数据库下表的数量，适当调整这个参数。

- 淘汰时间

  固定 86400 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中到最新的库或表，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.3 表 Schema

缓存表的 Schema 信息，如列名等。该缓存主要用于按需加载被访问到的表的 Schema，以防止同步大量不需要被访问的表的 Schema 而占用 FE 的内存。

该缓存由所有 Catalog 共享，全局唯一。

- 最大缓存数量

  由 FE 配置项 `max_external_schema_cache_num` 控制，默认为 10000。

  可以根据一个 Catalog 下所有表的数量，适当调整这个参数。

- 淘汰时间

  固定 86400 秒。3.0.7 版本之后，由 FE 参数 `external_cache_expire_time_seconds_after_access` 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 `external_cache_expire_time_minutes_after_access` 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的 Schema，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 `external_cache_refresh_time_minutes`。默认值不变。

2.14.9.2.4 Hive Metastore 表分区列表

用于缓存从 Hive Metastore 同步过来的表的分区列表。分区列表用于查询是进行分区裁剪。

该缓存，每个 Hive Catalog 有一个。

- 最大缓存数量

  由 FE 配置项 `max_hive_partition_table_cache_num` 控制，默认为 1000。

  可以根据一个 Catalog 下所有表的数量，适当调整这个参数。

- 淘汰时间

  固定 28800 秒。3.0.7 版本之后，由 FE 参数 `external_cache_expire_time_seconds_after_access` 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 `external_cache_expire_time_minutes_after_access` 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的分区列表，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 `external_cache_refresh_time_minutes`。默认值不变。

2.14.9.2.5 Hive Metastore 表分区属性

用于缓存 Hive 表，每个分区的属性，如文件格式，分区根路径等。每个查询，经过分区裁剪得到要访问的分区列表后，会通过该缓存获取每个分区的详细属性。

该缓存，每个 Hive Catalog 有一个。

- 最大缓存数量

  由 FE 配置项 max_hive_partition_cache_num 控制，默认为 10000。

  可以根据一个 Catalog 下，所需要访问的分区的总数量，适当调整这个参数。

- 淘汰时间

  固定 28800 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的分区属性，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.6 Hive Metastore 表分区文件列表

用于缓存 Hive 表，单个分区下的文件列表信息。该缓存用于降低文件系统的 List 操作带来的开销。

- 最大缓存数量

  由 FE 配置项 max_external_file_cache_num 控制，默认为 100000。

  可以根据所需要访问的文件数量，适当调整这个参数。

- 淘汰时间

  默认 28800 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

  如果 Catalog 属性中设置了 file.meta.cache.ttl-second 属性。则使用设置的时间。

  某些情况下，Hive 表的数据文件会频繁变动，导致缓存无法满足时效性。可以通过将该参数设置为 0，关闭该缓存。这种情况下，每次都会实时获取文件列表进行查询，性能可能降低，文件时效性提升。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的分区属性，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.7 Hudi 表分区

用于缓存 Hudi 表的分区信息。

该缓存，每个 Hudi Catalog 有一个。

- 最大缓存数量

  由 FE 配置项 max_external_table_cache_num 控制，默认为 1000。

  可以根据 Hudi 表的数量，适当调整这个参数。

- 淘汰时间

  固定 28800 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的 Hudi 分区属性，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.8 Iceberg 表信息

用于缓存 Iceberg 表对象。该对象通过 Iceberg API 加载并构建。

该缓存，每个 Iceberg Catalog 有一个。

- 最大缓存数量

  由 FE 配置项 max_external_table_cache_num 控制，默认为 1000。

  可以根据 Iceberg 表的数量，适当调整这个参数。

- 淘汰时间

  固定 28800 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的 Iceberg 表属性，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.2.9 Iceberg 表 Snapshot

用于缓存 Iceberg 表的 Snapshot 列表。该对象通过 Iceberg API 加载并构建。该缓存，每个 Iceberg Catalog 有一个。

- 最大缓存数量

  由 FE 配置项 max_external_table_cache_num 控制，默认为 1000。

  可以根据 Iceberg 表的数量，适当调整这个参数。

- 淘汰时间

  固定 28800 秒。3.0.7 版本之后，由 FE 参数 external_cache_expire_time_seconds_after_access 配置，默认 86400 秒。

- 最短刷新时间

  由 FE 配置项 external_cache_expire_time_minutes_after_access 控制。单位为分钟。默认 10 分钟。减少该时间，可以更实时的在 Doris 中访问到最新的 Iceberg 表属性，但会增加访问外部数据源的频率。

  3.0.7 版本后，配置项名称修改为 external_cache_refresh_time_minutes。默认值不变。

2.14.9.3 缓存刷新

除了上述每个缓存各自的刷新和淘汰策略外，用户也可以通过手动或定时的方式直接刷新元数据缓存。

2.14.9.3.1 手动刷新

用户可以通过 REFRESH 命令手动刷新元数据。

1. REFRESH CATALOG

   刷新指定 Catalog。

   ```
   REFRESH CATALOG ctl1 PROPERTIES("invalid_cache" = "true");
   ```

   - 该命令会刷新指定 Catalog 的库列表，表列名以及所有缓存信息等。
   - `invalid_cache` 表示是否要刷新分区和文件列表等缓存。默认为 true。如果为 false，则只会刷新 Catalog 的库、表列表，而不会刷新分区和文件列表等缓存信息。该参数适用于，用户只想同步新增删的库表信息时，可以设置为 false。

2. REFRESH DATABASE

   刷新指定 Database。

   ```
   REFRESH DATABASE [ctl.]db1 PROPERTIES("invalid_cache" = "true");
   ```

   - 该命令会刷新指定 Database 的表列名以及 Database 下的所有缓存信息等。
   - `invalid_cache` 属性含义同上。默认为 true。如果为 false，则只会刷新 Database 的表列表，而不会刷新缓存信息。该参数适用于，用户只想同步新增删的表信息时。

3. REFRESH TABLE

   刷新指定 Table。

   ```
   REFRESH TABLE [ctl.][db.]tbl1;
   ```

   - 该命令会刷新指定 Table 下的所有缓存信息等。

2.14.9.3.2 定时刷新

用户可以在创建 Catalog 时，设置该 Catalog 的定时刷新。

```
CREATE CATALOG hive PROPERTIES (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://172.0.0.1:9083',
    'metadata_refresh_interval_sec' = '3600'
);
```

在上例中，`metadata_refresh_interval_sec` 表示每 3600 秒刷新一次 Catalog。相当于每隔 3600 秒，自动执行一次：

```
REFRESH CATALOG ctl1 PROPERTIES("invalid_cache" = "true");
```

2.14.9.4 最佳实践

缓存可以显著提升元数据的访问性能，避免频繁的远程访问元数据导致性能抖动或者对元数据服务造成压力。但同时，缓存会降低数据的时效性。比如缓存刷新时间是 10 分钟，则在十分钟内，只能读到缓存的元数据。因此，需要根据情况，合理的设置缓存。

2.14.9.4.1 默认行为

这里主要介绍，默认参数配置情况下，用户可能关注的缓存行为。

- 外部数据源新增库、表后，在 Doris 中可以通过 SELECT 实时查询到。但 SHOW DATABASES 和 SHOW TABLES 可能看不到，需要手动刷新缓存，或最多等待 10 分钟。
- 外部数据源新增分区，需要手动刷新缓存，或最多等待 10 分钟后，可以查询到新分区的数据。
- 分区数据文件变动，需要手动刷新缓存，或最多等待 10 分钟后，可以查询到新分区的数据。

2.14.9.4.2 关闭 Schema 缓存

对于所有类型的 External Catalog，如果希望实时可见最新的 Table Schema，可以关闭 Schema Cache：

- 全局关闭

```
-- fe.conf
max_external_schema_cache_num=0 // 关闭 Schema 缓存。
```

- Catalog 级别关闭

```
-- Catalog property
"schema.cache.ttl-second" = "0" // 针对某个 Catalog，关闭 Schema 缓存（2.1.11, 3.0.6 支持）
```

设置完成后，Doris 会实时可见最新的 Table Schema。但此设置可能会增加元数据服务的压力。

2.14.9.4.3 关闭 Hive Catalog 元数据缓存

针对 Hive Catalog，如果想关闭缓存来查询到实时更新的数据，可以配置以下参数：

- 全局关闭

```
-- fe.conf
max_external_file_cache_num=0     // 关闭文件列表缓存
max_hive_partition_table_cache_num=0  // 关闭分区列表缓存
```

- Catalog 级别关闭

```
-- Catalog property
"file.meta.cache.ttl-second" = "0" // 针对某个 Catalog，关闭文件列表缓存
"partition.cache.ttl-second" = "0" // 针对某个 Catalog，关闭分区列表缓存（2.1.11, 3.0.6 支持
    ↪ ）
```

设置以上参数后：

- 外部数据源新增分区可以实时查询到。
- 分区数据文件变动可以实时查询到。

但会增加外部源数据（如 Hive Metastore 和 HDFS）的访问压力，可能导致元数据访问延迟不稳定等现象。

2.14.10 弹性计算节点

弹性计算节点作为一种特殊类型的 BE 节点，没有数据存储能力，只负责数据计算。因此，可以将计算节点看做是无状态的 BE 节点，可以方便的进行节点的增加和删除。

在湖仓数据分析场景中，弹性计算节点可用于查询外部数据源，如 Hive、Iceberg、Hudi、Paimon、JDBC 等。Doris 不负责外部数据源数据的存储，因此，可以使用弹性计算节点方便的扩展对外部数据源的计算能力。同时，计算节点也可以配置缓存目录，用于缓存外部数据源的热点数据，进一步加速数据读取。

弹性计算节点适用于在 Doris 存算一体模式，进行弹性资源控制。在 Doris 3.0 版本的存算分离架构下，BE 节点都是无状态的，因此不再需要单独的弹性计算节点。

2.14.10.1 计算节点的使用

2.14.10.1.1 BE 节点类型

在存算一体模式下，BE 节点分为两类：

- Mix

混合节点。即 BE 节点的默认类型。该类型的节点既参与计算，也负责 Doris 内表数据的存储。

- Computation

弹性计算节点。不负责数据的存储，只负责数据计算。

2.14.10.1.2 添加计算节点

在 BE 的 be.conf 配置文件中增加配置：

```
be_node_role=computation
```

之后启动 BE 节点，该节点就会以 Computation 类型运行。

之后可以通过连接 Doris 并执行：

```
ALTER SYSTEM ADD BACKEND
```

添加这个 BE 节点。添加成功后，在 SHOW BACKENDS 的 NodeRole 列可以看到节点类型为 computation。

2.14.10.1.3 使用计算节点

需要再 FE 配置文件 fe.conf 中配置如下参数，以启用计算节点并控制计算节点的行为：

参数名称	说明
prefer_compute_node_for_external_table	默认为 false。如果设置为 true，外部表的查询将优先分配给计算节点。如果为 false，外部表的查询将分配给任意 BE 节点。如果集群内没有计算节点，则该参数无效果。
min_backend_num_for_external_table	只有当 prefer_compute_node_for_external_table 为 true 时生效。如果集群内计算节点的个数小于这个值，外部表的查询会尝试获取一些混合节点来分配，以使节点总数达到这个值。如果集群内计算节点的个数大于这个值，外部表的查询将只分配给计算节点。在 2.0（含）版本之前，该参数的默认值为 3。2.1 版本之后，默认值为-1，表示只使用当前数量的计算节点

对 min_backend_num_for_external_table 的进一步举例说明：

假设集群内有 3 个计算节点，5 个混合节点。

如果 min_backend_num_for_external_table 设置小于等于 3。则外表查询只会使用 3 个计算节点。如果设置大于 3，假设为 6，则外表查询除了使用 3 个计算节点外，还会额外选择 3 个混合节点参与计算。

综上，该参数主要用于可参与外表计算的最少 BE 节点数量，并且会优先选择计算节点。调大这个参数，会让更多的 BE 节点（不限于计算节点）参与外表的查询处理；调小这个参数，则可以限制参与外表查询处理的 BE 节点数量。

> 注：
>
> 1. 2.1 版本之后，才支持 min_backend_num_for_external_table 设置为 -1。之前的版本，该参数必须为正数。且该参数只有在 prefer_compute_node_for_external_table = true 的情况下才生效。
> 2. 如果 min_backend_num_for_external_table 值大于总的 BE 节点数量，则最多只会选择全部的 BE。
> 3. 以上参数可以通过 ADMIN SET FRONTEND CONFIG 命令动态修改，不需要重启 FE 节点。且所有 FE 节点都需配置。或者在 fe.conf 中添加配置并重启 FE 节点。

2.14.10.2 最佳实践

2.14.10.2.1 联邦查询的负载隔离和弹性伸缩

在联邦查询场景下，用户可以专门部署一组计算节点，用于外表数据的查询。这样可以将外表的查询负载（如在 hive 上进行大数量分析）和内表的查询负载（如低延迟的快速数据分析）进行隔离。

同时，计算节点作为无状态的 BE 节点，可以方便的进行扩容和缩容。比如可以使用 k8s 部署一组弹性计算节点集群，在业务高峰期利用更多的计算节点进行数据湖分析，低谷期可以进行快速缩容以降低成本。

2.14.10.3 常见问题

1. 混合节点和计算节点能否相互转换

计算节点可以转换为混合节点。但混合节点不可以转换为计算节点。

2. 计算节点是否需要配置数据存储目录

需要。计算节点的数据存储目录不会存放用户数据，只会存放一些 BE 节点自身的信息文件，如 `cluster_id` 等。以及一些运行过程中的临时文件等。

计算节点的存储目录只需要很少的磁盘空间即可（MB 级别），并且可以随时和节点一起销毁，不会对用户数据造成影响。

3. 计算节点和混合节点是否可以配置文件缓存目录

文件缓存通过缓存最近访问的远端存储系统 (HDFS 或对象存储) 的数据文件，加速后续访问相同数据的查询。

计算节点和混合节点均可设置文件缓存目录。文件缓存目录需事先创建。

4. 计算节点是否需要通过 DECOMMISION 操作下线

不需要。计算节点可以直接通过 DROP BACKEND 操作删除。

2.14.11 统计信息

Doris 支持对外部数据源的表，如 Hive、Iceberg、Paimon 等进行自动或手动的统计信息收集。统计信息准确性直接决定了代价估算的准确性，对于选择最优查询计划至关重要，尤其在复杂查询场景下能显著提升查询执行效率。

具体可参阅统计信息 文档中的【外表收集】部分。

2.14.12 SQL 方言转换

2.14.12.1 SQL 方言转换

从 2.1 版本开始，Doris 可以支持多种 SQL 方言，如 Presto、Trino、Hive、PostgreSQL、Spark、Clickhouse 等等。通过这个功能，用户可以直接使用对应的 SQL 方言查询 Doris 中的数据，方便用户将原先的业务平滑的迁移到 Doris 中。

> 该功能目前是实验性功能，您在使用过程中如遇到任何问题，欢迎通过邮件组、GitHub Issue 等方式进行反馈。

2.14.12.1.1 部署服务

1. 下载最新版本的 SQL Convertor

> SQL 方言转换工具基于开源的 [SQLGlot](https://github.com/tobymao/sqlglot) ，由 SelectDB 进行二次开发，关于 SQLGlot 可参阅 [SQLGlot 官网](https://sqlglot.com/sqlglot.html)。
>
> SQL Convertor 并非由 Apache Doris 维护或认可，这些工作由 Committers 和 Doris PMC 监督。使用这些资源和服务完全由您自行决定，
> 社区不负责验证这些工具的许可或有效性。

2. 在任意 FE 节点，通过以下命令启动服务：

```
# 配置服务端口
vim apiserver/conf/config.conf

# 启动 SQL Converter for Apache Doris 转换服务
sh apiserver/bin/start.sh

# 如需前端界面，可在 webserver 中配置相应的端口并启动，不需要前端则可以忽略以下操作
vim webserver/conf/config.conf

# 启动前端界面
sh webserver/bin/start.sh
```

> - 该服务是一个无状态的服务，可随时启停
>
> - 在 `apiserver/conf/config.conf` 中配置 port 来指定任意一个可用端口，配置 workers 来指定启动的线程数量。在并发场景中，可以根据需要调整，默认为 1
>
> - 建议在每个 FE 节点都单独启动一个服务
>
> - 如需启动前端界面，可以在 `webserver/conf/config.conf` 中配置 SQL Converter for Apache Doris 转换服务地址，默认是 `API_HOST=http://127.0.0.1:5001`

3. 启动 Doris 集群（2.1 或更高版本）

4. 通过以下命令，在 Doris 中设置 SQL 方言转换服务的 URL：

```
MySQL> set global sql_converter_service_url = "http://127.0.0.1:5001/api/v1/convert"
```

- 127.0.0.1:5001 是 SQL 方言转换服务的部署节点 ip 和端口。
- 自 3.0.7 版本开始，允许设置多个 url 地址，已提供高可用的 SQL 方言转换服务。详见 相关参数部分介绍。

2.14.12.1.2 使用 SQL 方言

目前支持的方言类型包括：

- presto
- trino
- clickhouse
- hive
- spark
- postgres

示例：

Presto

```
CREATE TABLE  test_sqlconvert (
    id INT,
    start_time DATETIME,
    value STRING,
    arr_int ARRAY<INT>,
    arr_str ARRAY<STRING>
) ENGINE=OLAP
DUPLICATE KEY(`id`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
  "replication_allocation" = "tag.location.default: 1"
);

INSERT INTO test_sqlconvert VALUES(1, '2024-05-20 13:14:52', '2024-01-14',[1, 2, 3, 3], ['Hello',
    ↪  'World']);

SET sql_dialect = presto;

SELECT CAST(start_time AS varchar(20)) AS col1,
      array_distinct(arr_int) AS col2,
```

```
      FILTER(arr_str, x -> x LIKE '%World%') AS col3,
      to_date(value,'%Y-%m-%d') AS col4,
      YEAR(start_time) AS col5,
      date_add('month', 1, start_time) AS col6,
      REGEXP_EXTRACT_ALL(value, '-.') AS col7,
      JSON_EXTRACT('{"id": "33"}', '$.id')AS col8,
      element_at(arr_int, 1) AS col9,
      date_trunc('day',start_time) AS col10
    FROM test_sqlconvert
    WHERE date_trunc('day',start_time) = DATE '2024-05-20'
ORDER BY id;
+--
    ↪ ------------------+-----------+-----------+------------+------+---------------------+-------------+-----
    ↪
| col1                | col2      | col3      | col4       | col5 | col6                | col7
    ↪        | col8 | col9 | col10               |
+--
    ↪ ------------------+-----------+-----------+------------+------+---------------------+-------------+-----
    ↪
| 2024-05-20 13:14:52 | [1, 2, 3] | ["World"] | 2024-01-14 | 2024 | 2024-06-20 13:14:52 | ['-0','
    ↪ -1'] | "33" |    1 | 2024-05-20 00:00:00 |
+--
    ↪ ------------------+-----------+-----------+------------+------+---------------------+-------------+-----
    ↪
```

Clickhouse

```
SET sql_dialect = clickhouse;

SELECT toString(start_time) AS col1,
       arrayCompact(arr_int) AS col2,
       arrayFilter(x -> x LIKE '%World%',arr_str) AS col3,
       toDate(value) AS col4,
       toYear(start_time) AS col5,
       addMonths(start_time, 1) AS col6,
       extractAll(value, '-.') AS col7,
       JSONExtractString('{"id": "33"}' , 'id') AS col8,
       arrayElement(arr_int, 1) AS col9,
       date_trunc('day',start_time) AS col10
    FROM test_sqlconvert
    WHERE date_trunc('day',start_time)= '2024-05-20 00:00:00'
ORDER BY id;
+--
    ↪ ------------------+-----------+-----------+------------+------+---------------------+-------------+-----
    ↪
| col1                | col2      | col3      | col4       | col5 | col6                | col7
```

```
    ↪        | col8 | col9 | col10               |
+--
    ↪ ------------------+-----------+-----------+-----------+------+--------------------+-------------+-----
    ↪
| 2024-05-20 13:14:52 | [1, 2, 3] | ["World"] | 2024-01-14 | 2024 | 2024-06-20 13:14:52 | ['-0','
    ↪ -1'] | "33" |    1 | 2024-05-20 00:00:00 |
+--
    ↪ ------------------+-----------+-----------+-----------+------+--------------------+-------------+-----
    ↪
```

2.14.12.1.3 方言序列化

不同系统针对不同的列类型可能有不同的显示方式。

比如对于 NULL 值，Doris 和 Hive 显示为 `null`，而 Trino/Presto 显示为 NULL。

对于 Map 类型，Hive 显示为 `{1:null,2:null}`，而 Trino/Presto 显示为 {1=NULL, 2=NULL}。

为了最大程度保证用户迁移的行为一致性，Doris 提供了方言序列化模式选项，可以根据不同模式，返回不同的显示格式。

```
SET serde_diactor=<dialect>;
```

目前支持的序列化模式类型包括：

- doris（默认）
- hive
- presto/trino

> 注：该功能自 3.0.6 版本支持。

序列化格式对照表

以下表格显示了不同序列化模式下，各种数据类型的显示方式。未列举的类型表示显示方式一样。

Type	Doris	Hive	Presto/Trino
Bool	1, 0	1, 0	1, 0
↪			
Integer	1,	1,	1,
↪	1000	1000	1000
	↪	↪	↪
Float	1.2,	1.2,	1.2,
↪ /	3.00	3.00	3.00
↪ Decimal	↪	↪	↪
↪			

Type	Doris	Hive	Presto/Trino
Date ↪ / ↪ Datetime ↪	2025-01-01 ↪ , 2025-01-01 ↪ ↪ 10:11:11 ↪	2025-01-01 ↪ , 2025-01-01 ↪ ↪ 10:11:11 ↪	2025-01-01 ↪ , 2025-01-01 ↪ ↪ 10:11:11 ↪
String ↪	abc, 中国 ↪	abc, 中国 ↪	abc, 中国 ↪
Null ↪	null ↪	null ↪	NULL ↪
Array ↪ < ↪ bool ↪ > ↪	[1, ↪ ↪ 0] ↪	[↪ true ↪ , ↪ false ↪] ↪	[1, ↪ ↪ 0] ↪
Array ↪ < ↪ int ↪ > ↪	[1, ↪ ↪ 1000] ↪	[1,1000] ↪	[1, ↪ ↪ 1000] ↪
Array ↪ < ↪ string ↪ > ↪	[" ↪ abc ↪ ", ↪ ↪ " ↪ 中国 ↪ "] ↪	[" ↪ abc ↪ "," ↪ 中国 ↪ "] ↪	[" ↪ abc ↪ ", ↪ ↪ " ↪ 中国 ↪ "] ↪
Array ↪ < ↪ date ↪ / ↪ datetime ↪ > ↪	["2025-01-01", ↪ ↪ "2025-01-01 ↪ ↪ 10:11:11"] ↪	["2025-01-01","2025-01-01 ↪ ↪ 10:11:11"] ↪	["2025-01-01", ↪ ↪ "2025-01-01 ↪ ↪ 10:11:11"] ↪
Array ↪ < ↪ null ↪ > ↪	[↪ null ↪] ↪	[↪ null ↪] ↪	[↪ NULL ↪] ↪

Type	Doris	Hive	Presto/Trino
Map ↪ < ↪ int ↪ , ↪ ↪ string ↪ > ↪	{1:" ↪ abc ↪ ", ↪ ↪ 2:" ↪ 中国 ↪ "}	{1:" ↪ abc ↪ ",2:" ↪ 中国 ↪ "} ↪	{1= ↪ abc ↪ , ↪ ↪ 2= ↪ 中国 ↪ } ↪
Map ↪ < ↪ string ↪ , ↪ ↪ date ↪ / ↪ datetime ↪ > ↪	{" ↪ k1 ↪ ":"2022-10-01", ↪ ↪ " ↪ k2 ↪ ":"2022-10-01 ↪ ↪ 10:10:10"} ↪	{" ↪ k1 ↪ ":"2022-10-01"," ↪ k2 ↪ ":"2022-10-01 ↪ ↪ 10:10:10"} ↪	{k1 ↪ =2022-10-01, ↪ ↪ k2 ↪ =2022-10-01 ↪ ↪ 10:10:10} ↪
Map ↪ < ↪ int ↪ , ↪ ↪ null ↪ > ↪	{1: ↪ null ↪ , ↪ ↪ 2: ↪ null ↪ } ↪	{1: ↪ null ↪ ,2: ↪ null ↪ } ↪	{1= ↪ NULL ↪ , ↪ ↪ 2= ↪ NULL ↪ } ↪
Struct ↪ <> ↪	Same as map	Same as map	Same as map

2.14.12.1.4 相关参数

- 变量

变量名	示例	说明
sql_converter_service_url	set global sql_converter_service_url = "http://127.0.0.1:5001/api/v1/convert"	全局变量，用于指定 sql convetor 服务地址
sql_dialect	set sql_dialect = presto	会话变量，用于指定当前会话的方言
serde_dialect	set serde_dialect = hive	会话变量，用于指定当前会话的序列化方言格式

变量名	示例	说明
enable ↪ _ ↪ sql ↪ _ ↪ convertor ↪ _ ↪ features ↪	set ↪ enable ↪ _ ↪ sql ↪ _ ↪ convertor ↪ _ ↪ features ↪ =" ↪ ctas ↪ "	会话变量，用户指定开启 sql convertor 的某些特殊功能。ctas: 允许对 CTAS 语句中的 SELECT ↪ 部分进行转换。（该参数自 Doris 3.0.6 和 SQL Convertor 1.0.8.10 支持）

变量名	示例	说明
sql_convertor_config	set sql_convertor_config = '{"ignore_udf":["func1","func2","fucn3"]}'	会话变量，用于指定 SQL Convertor 忽略一些 UDF。在列表中的函数，SQL Convertor 不会进行转换，否则可能报错“Unknown Function”（该参数自 Doris 3.0.6 和 SQL Convertor 1.0.8.10 支持）

自 3.0.7 版本开始，允许设置多个 url 地址，以逗号分隔：

```
    set global sql_converter_service_url = "http://127.0.0.1:5001/api/v1/convert,"http
        ↪ ://127.0.0.2:5001/api/v1/convert""
```

```
Doris 会优先选择 `127.0.0.1` 的本地服务地址，当优先选择的地址不可用时，会自动切换到其他可用地址，
    ↪ 以保证服务的可用性。
```

2.14.12.1.5 最佳实践

- 指定不需要转换的函数

  在某些情况下，可能无法在 Doris 中找到和原系统完全对应的函数，或者部分经过转换后的函数，在一些特殊参数下行为和原函数不完全一致。此时，用户可以先通过 UDF 来实现和原系统完全一致的函数，注册到 Doris 中。之后，在 `sql_convertor_config` 的 `ignore_udf` 中添加这个 UDF。这样，SQL Convetor 将不会对这个函数进行转换，以便用户可以使用 UDF 来控制函数行为。

2.14.12.1.6 版本变更记录

SQL Convertor 版本变更记录

2.14.12.2 Presto/Trino SQL 转换指南

TODO

2.14.12.3 Clickhouse SQL 转换指南

TODO

2.14.12.4 Hive SQL 转换指南

TODO

2.14.12.5 PostgreSQL SQL 转换指南

TODO

2.14.13 湖仓一体最佳实践

2.14.13.1 数据湖查询调优

本文档主要介绍在针对湖上数据（ Hive、 Iceberg、 Paimon 等 ）查询的优化手段和优化策略。

2.14.13.1.1 分区裁剪

通过在查询中指定分区列条件，能够裁减掉不必要的分区，减少需要读取的数据量。

可以通过 EXPLAIN <SQL> 来查看 XXX_SCAN_NODE 的 partition 部分，可以查看分区裁剪是否生效，以及本次查询需要扫描多少分区。

如：

```
0:VPAIMON_SCAN_NODE(88)
    table: paimon_ctl.db.table
    predicates: (user_id[#4] = 431304818)
    inputSplitNum=15775, totalFileSize=951754154566, scanRanges=15775
    partition=203/0
```

2.14.13.1.2 本地数据缓存

数据缓存（Data Cache）通过缓存最近访问的远端存储系统（HDFS 或对象存储）的数据文件到本地磁盘上，加速后续访问相同数据的查询。

缓存功能默认是关闭的，请参阅数据缓存文档配置并开启。

2.14.13.1.3 HDFS 读取优化

可参考 HDFS 文档中 HDFS IO 优化部分。

2.14.13.1.4 Merge IO 优化

针对 HDFS、对象存储等远端存储系统，Doris 会通过 Merge IO 技术来优化 IO 访问。Merge IO 技术，本质上是将多个相邻的小 IO 请求，合并成一个大 IO 请求，这样可以减少 IOPS，增加 IO 吞吐。

比如原始请求需要读取文件 file1 的 [0, 10] 和 [20, 50] 两部分数据：

```
Request Range: [0, 10], [20, 50]
```

通过 Merge IO，会合并成一个请求：

```
Request Range: [0, 50]
```

在这个示例中，两次 IO 请求合并为了一次，但同时也多读了一部分数据（10-20 之间的数据）。因此，Merge IO 在降低 IO 次数的同时，可能带来潜在的读放大问题。

通过 Query Profile 可以查看 MergeIO 的具体情况：

```
- MergedSmallIO:
    - MergedBytes: 3.00 GB
    - MergedIO: 424
    - RequestBytes: 2.50 GB
    - RequestIO: 65.555K (65555)
```

其中 RequestBytes 和 RequestIO 标识原始请求的数据量和请求次数。MergedBytes 和 MergedIO 标识合并和的请求数据量和请求次数。

如果发现 MergedBytes 数据量远大于 RequestBytes，则说明读放大比较严重，可以通过下面的参数调整修改：

- merge_io_read_slice_size_bytes

  会话变量，自 3.1.3 版本支持。默认为 8MB。如果发现读放大严重，可以将此参数调小，如 64KB。并观察修改后的 IO 请求和查询延迟是否有提升。

2.14.13.2 使用 Doris 和 Hudi

作为一种全新的开放式的数据管理架构，湖仓一体（Data Lakehouse）融合了数据仓库的高性能、实时性以及数据湖的低成本、灵活性等优势，帮助用户更加便捷地满足各种数据处理分析的需求，在企业的大数据体系中已经得到越来越多的应用。

在过去多个版本中，Apache Doris 持续加深与数据湖的融合，当前已演进出一套成熟的湖仓一体解决方案。

- 自 0.15 版本起，Apache Doris 引入 Hive 和 Iceberg 外部表，尝试在 Apache Iceberg 之上探索与数据湖的能力结合。
- 自 1.2 版本起，Apache Doris 正式引入 Multi-Catalog 功能，实现了多种数据源的自动元数据映射和数据访问、并对外部数据读取和查询执行等方面做了诸多性能优化，完全具备了构建极速易用 Lakehouse 架构的能力。
- 在 2.1 版本中，Apache Doris 湖仓一体架构得到全面加强，不仅增强了主流数据湖格式（Hudi、Iceberg、Paimon 等）的读取和写入能力，还引入了多 SQL 方言兼容、可从原有系统无缝切换至 Apache Doris。在数据科学及大规模数据读取场景上，Doris 集成了 Arrow Flight 高速读取接口，使得数据传输效率实现 100 倍的提升。

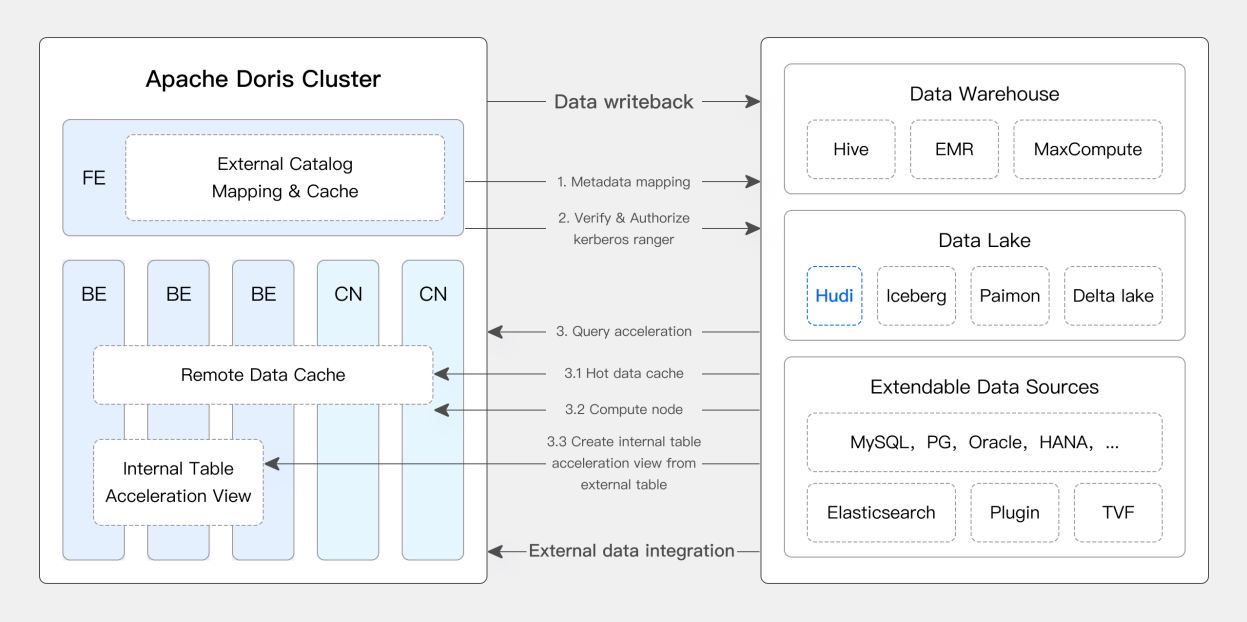


图 74: 使用 Doris 和 Hudi 构建 Lakehouse

2.14.13.2.1 Apache Doris & Hudi

Apache Hudi 是目前最主流的开放数据湖格式之一，也是事务性的数据湖管理平台，支持包括 Apache Doris 在内的多种主流查询引擎。

Apache Doris 同样对 Apache Hudi 数据表的读取能力进行了增强：

- 支持 Copy on Write Table： Snapshot Query
- 支持 Merge on Read Table： Snapshot Queries, Read Optimized Queries
- 支持 Time Travel
- 支持 Incremental Read

凭借 Apache Doris 的高性能查询执行以及 Apache Hudi 的实时数据管理能力，可以实现高效、灵活、低成本的数据查询和分析，同时也提供了强大的数据回溯、审计和增量处理功能，当前基于 Apache Doris 和 Apache Hudi 的组合已经在多个社区用户的真实业务场景中得到验证和推广：

- 实时数据分析与处理：比如金融行业交易分析、广告行业实时点击流分析、电商行业用户行为分析等常见场景下，都要求实时的数据更新及查询分析。Hudi 能够实现对数据的实时更新和管理，并保证数据的一致性和可靠性，Doris 则能够实时高效处理大规模数据查询请求，二者结合能够充分满足实时数据分析与处理的需求。
- 数据回溯与审计：对于金融、医疗等对数据安全和准确性要求极高的行业来说，数据回溯和审计是非常重要的功能。Hudi 提供了时间旅行（Time Travel）功能，允许用户查看历史数据状态，结合 Apache Doris 高效查询能力，可快速查找分析任何时间点的数据，实现精确的回溯和审计。
- 增量数据读取与分析：在进行大数据分析时往往面临着数据规模庞大、更新频繁的问题，Hudi 支持增量数据读取，这使得用户可以只需处理变化的数据，不必进行全量数据更新；同时 Apache Doris 的 Incremental Read 功能也可使这一过程更加高效，显著提升了数据处理和分析的效率。
- 跨数据源联邦查询：许多企业数据来源复杂，数据可能存储在不同的数据库中。Doris 的 Multi-Catalog 功能支持多种数据源的自动映射与同步，支持跨数据源的联邦查询。这对于需要从多个数据源中获取和整合数据进行分析的企业来说，极大地缩短了数据流转路径，提升了工作效率。

本文将在 Docker 环境下，为读者介绍如何快速搭建 Apache Doris + Apache Hudi 的测试及演示环境，并对各功能操作进行演示，帮助读者快速入门。

关于更多说明，请参阅 Hudi Catalog

2.14.13.2.2 使用指南

本文涉及所有脚本和代码可以从该地址获取：https://github.com/apache/doris/tree/master/samples/datalake/hudi

01 环境准备

本文示例采用 Docker Compose 部署，组件及版本号如下：

组件名称	版本
Apache Doris	默认 2.1.4，可修改
Apache Hudi	0.14
Apache Spark	3.4.2
Apache Hive	2.1.3
MinIO	2022-05-26T05-48-41Z

02 环境部署

1. 创建 Docker 网络

   ```
   sudo docker network create -d bridge hudi-net
   ```

2. 启动所有组件

   ```
   sudo ./start-hudi-compose.sh
   ```

> 注：启动前，可将 start-hudi-compose.sh 中的 DORIS_PACKAGE 和 DORIS_DOWNLOAD_URL 修改成需要的 Doris 版本。建议使用 2.1.4 或更高版本。

3. 启动后，可以使用如下脚本，登陆 Spark 命令行或 Doris 命令行：

```
-- Doris
sudo ./login-spark.sh

-- Spark
sudo ./login-doris.sh
```

03 数据准备

接下来先通过 Spark 生成 Hudi 的数据。如下方代码所示，集群中已经包含一张名为 customer 的 Hive 表，可以通过这张 Hive 表，创建一个 Hudi 表：

```
-- ./login-spark.sh
spark-sql> use default;

-- create a COW table
spark-sql> CREATE TABLE customer_cow
USING hudi
TBLPROPERTIES (
  type = 'cow',
  primaryKey = 'c_custkey',
  preCombineField = 'c_name'
)
PARTITIONED BY (c_nationkey)
AS SELECT * FROM customer;

-- create a MOR table
spark-sql> CREATE TABLE customer_mor
USING hudi
TBLPROPERTIES (
  type = 'mor',
  primaryKey = 'c_custkey',
  preCombineField = 'c_name'
```

```
)
PARTITIONED BY (c_nationkey)
AS SELECT * FROM customer;
```

04 数据查询

如下所示，Doris 集群中已经创建了名为 hudi 的 Catalog（可通过 SHOW CATALOGS 查看）。以下为该 Catalog 的创建语句：

```
-- 已经创建，无需再次执行
CREATE CATALOG `hudi` PROPERTIES (
    "type"="hms",
    'hive.metastore.uris' = 'thrift://hive-metastore:9083',
    "s3.access_key" = "minio",
    "s3.secret_key" = "minio123",
    "s3.endpoint" = "http://minio:9000",
    "s3.region" = "us-east-1",
    "use_path_style" = "true"
);
```

1. 手动刷新该 Catalog，对创建的 Hudi 表进行同步：

```
-- ./login-doris.sh
doris> REFRESH CATALOG hudi;
```

2. 使用 Spark 操作 Hudi 中的数据，都可以在 Doris 中实时可见，不需要再次刷新 Catalog。我们通过 Spark 分别给 COW 和 MOR 表插入一行数据：

```
spark-sql> insert into customer_cow values (100, "Customer#000000100", "jD2xZzi", "
    ↪ 25-430-914-2194", 3471.59, "BUILDING", "cial ideas. final, furious requests", 25);
spark-sql> insert into customer_mor values (100, "Customer#000000100", "jD2xZzi", "
    ↪ 25-430-914-2194", 3471.59, "BUILDING", "cial ideas. final, furious requests", 25);
```

3. 通过 Doris 可以直接查询到最新插入的数据：

```
doris> use hudi.default;
doris> select * from customer_cow where c_custkey = 100;
doris> select * from customer_mor where c_custkey = 100;
```

4. 再通过 Spark 插入 c_custkey=32 已经存在的数据，即覆盖已有数据：

```
spark-sql> insert into customer_cow values (32, "Customer#000000032_update", "jD2xZzi", "
    ↪ 25-430-914-2194", 3471.59, "BUILDING", "cial ideas. final, furious requests", 15);
spark-sql> insert into customer_mor values (32, "Customer#000000032_update", "jD2xZzi", "
    ↪ 25-430-914-2194", 3471.59, "BUILDING", "cial ideas. final, furious requests", 15);
```

5. 通过 Doris 可以查询更新后的数据：

```
doris> select * from customer_cow where c_custkey = 32;
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
| c_custkey | c_name                    | c_address | c_phone         | c_acctbal | c_
    ↪ mktsegment | c_comment                              | c_nationkey |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
|        32 | Customer#000000032_update | jD2xZzi   | 25-430-914-2194 |   3471.59 | BUILDING
    ↪     | cial ideas. final, furious requests |          15 |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
doris> select * from customer_mor where c_custkey = 32;
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
| c_custkey | c_name                    | c_address | c_phone         | c_acctbal | c_
    ↪ mktsegment | c_comment                              | c_nationkey |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
|        32 | Customer#000000032_update | jD2xZzi   | 25-430-914-2194 |   3471.59 | BUILDING
    ↪     | cial ideas. final, furious requests |          15 |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+------
    ↪
```

05 Incremental Read

Incremental Read 是 Hudi 提供的功能特性之一，通过 Incremental Read，用户可以获取指定时间范围的增量数据，从而实现对数据的增量处理。对此，Doris 可对插入 c_custkey=100 后的变更数据进行查询。如下所示，我们插入了一条 c_custkey=32 的数据：

```
doris> select * from customer_cow@incr('beginTime'='20240603015018572');
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+-------------+----------
    ↪
```

```
| c_custkey | c_name                    | c_address | c_phone         | c_acctbal | c_mktsegment
    ↪ | c_comment                         | c_nationkey |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+----------
    ↪
|        32 | Customer#000000032_update | jD2xZzi   | 25-430-914-2194 |   3471.59 | BUILDING
    ↪ | cial ideas. final, furious requests |          15 |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+----------
    ↪
spark-sql> select * from hudi_table_changes('customer_cow', 'latest_state', '20240603015018572');

doris> select * from customer_mor@incr('beginTime'='20240603015058442');
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+----------
    ↪
| c_custkey | c_name                    | c_address | c_phone         | c_acctbal | c_mktsegment
    ↪ | c_comment                         | c_nationkey |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+----------
    ↪
|        32 | Customer#000000032_update | jD2xZzi   | 25-430-914-2194 |   3471.59 | BUILDING
    ↪ | cial ideas. final, furious requests |          15 |
+--
    ↪ ---------+---------------------------+-----------+-----------------+-----------+--------------+----------
    ↪
spark-sql> select * from hudi_table_changes('customer_mor', 'latest_state', '20240603015058442');
```

06 TimeTravel

Doris 支持查询指定快照版本的 Hudi 数据，从而实现对数据的 Time Travel 功能。首先，可以通过 Spark 查询两张 Hudi 表的提交历史：

```
spark-sql> call show_commits(table => 'customer_cow', limit => 10);
20240603033556094        20240603033558249        commit        448833        0        1        1
    ↪         183        0        0
20240603015444737        20240603015446588        commit        450238        0        1        1
    ↪         202        1        0
20240603015018572        20240603015020503        commit        436692        1        0        1
    ↪         1        0        0
20240603013858098        20240603013907467        commit        44902033        100        0
    ↪         25        18751        0        0

spark-sql> call show_commits(table => 'customer_mor', limit => 10);
20240603033745977        20240603033748021        deltacommit        1240        0        1
    ↪         1        0        0        0
```

```
20240603015451860         20240603015453539         deltacommit         1434         0         1
    ↪         1         1         1         0
20240603015058442         20240603015100120         deltacommit         436691         1         0
    ↪         1         1         0         0
20240603013918515         20240603013922961         deltacommit         44904040         100         0
    ↪          25         18751         0         0
```

接着，可通过 Doris 执行 c_custkey=32 ，查询数据插入之前的数据快照。如下可看到 c_custkey=32 的数据还未更新：

> 注：Time Travel 语法暂时不支持新优化器，需要先执行 set enable_nereids_planner=false; 关闭新优化器，该问题将会在后续版本中修复。

```
doris> select * from customer_cow for time as of '20240603015018572' where c_custkey = 32 or c_
    ↪ custkey = 100;
+--
    ↪ ---------+--------------------+---------------------------------------+-----------------+-----------+----
    ↪
| c_custkey | c_name             | c_address                             | c_phone         | c_
    ↪ acctbal | c_mktsegment | c_comment                                              | c_nationkey |
+--
    ↪ ---------+--------------------+---------------------------------------+-----------------+-----------+----
    ↪
|        32 | Customer#000000032 | jD2xZzi UmId,DCtNBLXKj9q0Tlp2iQ6ZcO3J | 25-430-914-2194 |
    ↪ 3471.53 | BUILDING     | cial ideas. final, furious requests across the e |          15 |
|       100 | Customer#000000100 | jD2xZzi                               | 25-430-914-2194 |
    ↪ 3471.59 | BUILDING     | cial ideas. final, furious requests              |          25 |
+--
    ↪ ---------+--------------------+---------------------------------------+-----------------+-----------+----
    ↪
-- compare with spark-sql
spark-sql> select * from customer_mor timestamp as of '20240603015018572' where c_custkey = 32 or
    ↪  c_custkey = 100;

doris> select * from customer_mor for time as of '20240603015058442' where c_custkey = 32 or c_
    ↪ custkey = 100;
+--
    ↪ ---------+--------------------+---------------------------------------+-----------------+-----------+----
    ↪
| c_custkey | c_name             | c_address                             | c_phone         | c_
    ↪ acctbal | c_mktsegment | c_comment                                              | c_nationkey |
+--
    ↪ ---------+--------------------+---------------------------------------+-----------------+-----------+----
```

```
    ↪
|       100 | Customer#000000100 | jD2xZzi                                | 25-430-914-2194 |
    ↪ 3471.59 | BUILDING     | cial ideas. final, furious requests              |          25 |
|        32 | Customer#000000032 | jD2xZzi UmId,DCtNBLXKj9q0Tlp2iQ6ZcO3J | 25-430-914-2194 |
    ↪ 3471.53 | BUILDING     | cial ideas. final, furious requests across the e |          15 |
+--
    ↪ ---------+--------------------+----------------------------------------+-----------------+-----------+----
    ↪
spark-sql> select * from customer_mor timestamp as of '20240603015058442' where c_custkey = 32 or
    ↪  c_custkey = 100;
```

2.14.13.2.3 查询优化

Apache Hudi 中的数据大致可以分为两类 —— 基线数据和增量数据。基线数据通常是已经经过合并的 Parquet 文件，而增量数据是指由 INSERT、UPDATE 或 DELETE 产生的数据增量。基线数据可以直接读取，增量数据需要通过 Merge on Read 的方式进行读取。

对于 Hudi COW 表的查询或者 MOR 表的 Read Optimized 查询而言，其数据都属于基线数据，可直接通过 Doris 原生的 Parquet Reader 读取数据文件，且可获得极速的查询响应。而对于增量数据，Doris 需要通过 JNI 调用 Hudi 的 Java SDK 进行访问。为了达到最优的查询性能，Apache Doris 在查询时，会将一个查询中的数据分为基线和增量数据两部分，并分别使用上述方式进行读取。

为验证该优化思路，我们通过 EXPLAIN 语句来查看一个下方示例的查询中，分别有多少基线数据和增量数据。对于 COW 表来说，所有 101 个数据分片均为是基线数据（hudiNativeReadSplits=101/101），因此 COW 表全部可直接通过 Doris Parquet Reader 进行读取，因此可获得最佳的查询性能。对于 ROW 表，大部分数据分片是基线数据（hudiNativeReadSplits=100/101），一个分片数为增量数据，基本也能够获得较好的查询性能。

```
-- COW table is read natively
doris> explain select * from customer_cow where c_custkey = 32;
|   0:VHUDI_SCAN_NODE(68)                                          |
|      table: customer_cow                                         |
|      predicates: (c_custkey[#5] = 32)                            |
|      inputSplitNum=101, totalFileSize=45338886, scanRanges=101   |
|      partition=26/26                                             |
|      cardinality=1, numNodes=1                                   |
|      pushdown agg=NONE                                           |
|      hudiNativeReadSplits=101/101                                |

-- MOR table: because only the base file contains `c_custkey = 32` that is updated, 100 splits
    ↪ are read natively, while the split with log file is read by JNI.
doris> explain select * from customer_mor where c_custkey = 32;
|   0:VHUDI_SCAN_NODE(68)                                          |
|      table: customer_mor                                         |
|      predicates: (c_custkey[#5] = 32)                            |
|      inputSplitNum=101, totalFileSize=45340731, scanRanges=101   |
|      partition=26/26                                             |
|      cardinality=1, numNodes=1                                   |
```

```
|      pushdown agg=NONE                                          |
|      hudiNativeReadSplits=100/101                               |
```

可以通过 Spark 进行一些删除操作，进一步观察 Hudi 基线数据和增量数据的变化：

```
-- Use delete statement to see more differences
spark-sql> delete from customer_cow where c_custkey = 64;
doris> explain select * from customer_cow where c_custkey = 64;

spark-sql> delete from customer_mor where c_custkey = 64;
doris> explain select * from customer_mor where c_custkey = 64;
```

此外，还可以通过分区条件进行分区裁剪，从而进一步减少数据量，以提升查询速度。如下示例中，通过分区条件 c_nationkey=15 进行分区裁减，使得查询请求只需要访问一个分区（partition=1/26）的数据即可。

```
-- customer_xxx is partitioned by c_nationkey, we can use the partition column to prune data
doris> explain select * from customer_mor where c_custkey = 64 and c_nationkey = 15;
|   0:VHUDI_SCAN_NODE(68)                                                 |
|      table: customer_mor                                                |
|      predicates: (c_custkey[#5] = 64), (c_nationkey[#12] = 15)          |
|      inputSplitNum=4, totalFileSize=1798186, scanRanges=4               |
|      partition=1/26                                                     |
|      cardinality=1, numNodes=1                                          |
|      pushdown agg=NONE                                                  |
|      hudiNativeReadSplits=3/4                                           |
```

2.14.13.3 使用 Doris 和 Paimon

作为一种全新的开放式的数据管理架构，湖仓一体（Data Lakehouse）融合了数据仓库的高性能、实时性以及数据湖的低成本、灵活性等优势，帮助用户更加便捷地满足各种数据处理分析的需求，在企业的大数据体系中已经得到越来越多的应用。

在过去多个版本中，Apache Doris 持续加深与数据湖的融合，当前已演进出一套成熟的湖仓一体解决方案。

- 自 0.15 版本起，Apache Doris 引入 Hive 和 Iceberg 外部表，尝试在 Apache Iceberg 之上探索与数据湖的能力结合。
- 自 1.2 版本起，Apache Doris 正式引入 Multi-Catalog 功能，实现了多种数据源的自动元数据映射和数据访问、并对外部数据读取和查询执行等方面做了诸多性能优化，完全具备了构建极速易用 Lakehouse 架构的能力。
- 在 2.1 版本中，Apache Doris 湖仓一体架构得到全面加强，不仅增强了主流数据湖格式（Hudi、Iceberg、Paimon 等）的读取和写入能力，还引入了多 SQL 方言兼容、可从原有系统无缝切换至 Apache Doris。在数据科学及大规模数据读取场景上，Doris 集成了 Arrow Flight 高速读取接口，使得数据传输效率实现 100 倍的提升。

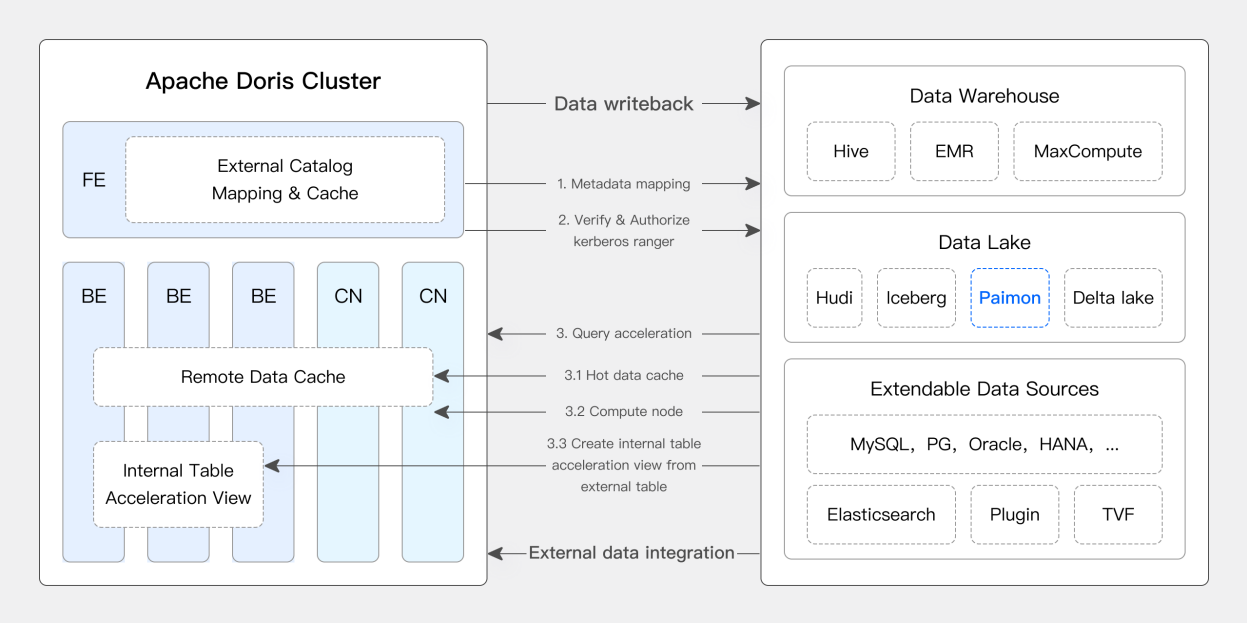


图 75: 使用 Doris 和 Paimon 构建 Lakehouse

2.14.13.3.1 Apache Doris & Paimon

Apache Paimon 是一种数据湖格式，并创新性地将数据湖格式和 LSM 结构的优势相结合，成功将高效的实时流更新能力引入数据湖架构中，这使得 Paimon 能够实现数据的高效管理和实时分析，为构建实时湖仓架构提供了强大的支撑。

为了充分发挥 Paimon 的能力，提高对 Paimon 数据的查询效率，Apache Doris 对 Paimon 的多项最新特性提供了原生支持：

- 支持 Hive Metastore、 FileSystem 等多种类型的 Paimon Catalog。
- 原生支持 Paimon 0.6 版本发布的 Primary Key Table Read Optimized 功能。
- 原生支持 Paimon 0.8 版本发布的 Primary Key Table Deletion Vector 功能。

基于 Apache Doris 的高性能查询引擎和 Apache Paimon 高效的实时流更新能力，用户可以实现：

- 数据实时入湖：借助 Paimon 的 LSM-Tree 模型，数据入湖的时效性可以降低到分钟级；同时，Paimon 支持包括聚合、去重、部分列更新在内的多种数据更新能力，使得数据流动更加灵活高效。
- 高性能数据处理分析：Paimon 所提供的 Append Only Table、Read Optimized、Deletion Vector 等技术，可与 Doris 强大的查询引擎对接，实现湖上数据的快速查询及分析响应。

未来 Apache Doris 将会逐步支持包括 Time Travel、 增量数据读取在内的 Apache Paimon 更多高级特性，共同构建统一、高性能、实时的湖仓平台。

本文将会再 Docker 环境中，为读者讲解如何快速搭建 Apache Doris + Apache Paimon 测试 & 演示环境，并展示各功能的使用操作。

关于更多说明，请参阅 Paimon Catalog

2.14.13.3.2 使用指南

本文涉及所有脚本和代码可以从该地址获取：https://github.com/apache/doris/tree/master/samples/datalake/iceberg_and_paimon

01 环境准备

本文示例采用 Docker Compose 部署，组件及版本号如下：

组件名称	版本
Apache Doris	默认 2.1.5，可修改
Apache Paimon	0.8
Apache Flink	1.18
MinIO	RELEASE.2024-04-29T09-56-05Z

02 环境部署

1. 启动所有组件

```
bash ./start_all.sh
```

2. 启动后，可以使用如下脚本，登陆 Flink 命令行或 Doris 命令行：

```
-- login flink
bash ./start_flink_client.sh

-- login doris
bash ./start_doris_client.sh
```

03 数据准备

首先登陆 Flink 命令行后，可以看到一张预构建的表。表中已经包含一些数据，我们可以通过 Flink SQL 进行查看。

```
Flink SQL> use paimon.db_paimon;
[INFO] Execute statement succeed.

Flink SQL> show tables;
+------------+
| table name |
+------------+
|   customer |
+------------+
1 row in set

Flink SQL> show create table customer;
+------------------------------------------------------------------------+
|                                                                 result |
```

```
+------------------------------------------------------------------------+
| CREATE TABLE `paimon`.`db_paimon`.`customer` (
  `c_custkey` INT NOT NULL,
  `c_name` VARCHAR(25),
  `c_address` VARCHAR(40),
  `c_nationkey` INT NOT NULL,
  `c_phone` CHAR(15),
  `c_acctbal` DECIMAL(12, 2),
  `c_mktsegment` CHAR(10),
  `c_comment` VARCHAR(117),
  CONSTRAINT `PK_c_custkey_c_nationkey` PRIMARY KEY (`c_custkey`, `c_nationkey`) NOT ENFORCED
) PARTITIONED BY (`c_nationkey`)
WITH (
  'bucket' = '1',
  'path' = 's3://warehouse/wh/db_paimon.db/customer',
  'deletion-vectors.enabled' = 'true'
)
 |
+------------------------------------------------------------------------+
1 row in set

Flink SQL> desc customer;
+--------------+----------------+-------+-----------------------------+--------+-----------+
|         name |           type |  null |                         key | extras | watermark |
+--------------+----------------+-------+-----------------------------+--------+-----------+
|    c_custkey |            INT | FALSE | PRI(c_custkey, c_nationkey) |        |           |
|       c_name |    VARCHAR(25) |  TRUE |                             |        |           |
|    c_address |    VARCHAR(40) |  TRUE |                             |        |           |
|  c_nationkey |            INT | FALSE | PRI(c_custkey, c_nationkey) |        |           |
|      c_phone |       CHAR(15) |  TRUE |                             |        |           |
|    c_acctbal | DECIMAL(12, 2) |  TRUE |                             |        |           |
| c_mktsegment |       CHAR(10) |  TRUE |                             |        |           |
|    c_comment |   VARCHAR(117) |  TRUE |                             |        |           |
+--------------+----------------+-------+-----------------------------+--------+-----------+
8 rows in set

Flink SQL> select * from customer order by c_custkey limit 4;
+--
    ↪ ---------+--------------------+--------------------------------+-------------+-----------------+---------
    ↪
| c_custkey |             c_name |                      c_address | c_nationkey |         c_phone
    ↪  | c_acctbal | c_mktsegment |                        c_comment |
+--
    ↪ ---------+--------------------+--------------------------------+-------------+-----------------+---------
    ↪
```

```
|         1 | Customer#000000001 |              IVhzIApeRb ot,c,E |          15 | 25-989-741-2988
    ↪  |    711.56 |     BUILDING | to the even, regular platel... |
|         2 | Customer#000000002 | XSTf4,NCwDVaWNe6tEgvwfmRchLXak |          13 | 23-768-687-3665
    ↪  |    121.65 |   AUTOMOBILE | l accounts. blithely ironic... |
|         3 | Customer#000000003 |                   MG9kdTD2WBHm |           1 | 11-719-748-3364
    ↪  |   7498.12 |   AUTOMOBILE |  deposits eat slyly ironic,... |
|        32 | Customer#000000032 | jD2xZzi UmId,DCtNBLXKj9q0Tl... |          15 | 25-430-914-2194
    ↪  |   3471.53 |     BUILDING | cial ideas. final, furious ... |
+--
    ↪ ---------+--------------------+--------------------------------+-------------+-----------------+---------
    ↪
4 rows in set
```

04 数据查询

如下所示，Doris 集群中已经创建了名为 paimon 的 Catalog（可通过 SHOW CATALOGS 查看）。以下为该 Catalog 的创建语句：

```
-- 已创建，无需执行
CREATE CATALOG `paimon` PROPERTIES (
    "type" = "paimon",
    "warehouse" = "s3://warehouse/wh/",
    "s3.endpoint"="http://minio:9000",
    "s3.access_key"="admin",
    "s3.secret_key"="password",
    "s3.region"="us-east-1"
);
```

你可登录到 Doris 中查询 Paimon 的数据：

```
mysql> use paimon.db_paimon;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+----------------------+
| Tables_in_db_paimon |
+----------------------+
| customer             |
+----------------------+
1 row in set (0.00 sec)

mysql> select * from customer order by c_custkey limit 4;
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
```

```
| c_custkey | c_name             | c_address                              | c_nationkey | c_phone
    ↪         | c_acctbal | c_mktsegment | c_comment
    ↪
    ↪ |
+--
    ↪ ---------+--------------------+----------------------------------------+-------------+-----------------+--
    ↪
|         1 | Customer#000000001 | IVhzIApeRb ot,c,E                      |          15 |
    ↪ 25-989-741-2988 |    711.56 | BUILDING     | to the even, regular platelets. regular,
    ↪ ironic epitaphs nag e                                          |
|         2 | Customer#000000002 | XSTf4,NCwDVaWNe6tEgvwfmRchLXak         |          13 |
    ↪ 23-768-687-3665 |    121.65 | AUTOMOBILE   | l accounts. blithely ironic theodolites
    ↪ integrate boldly: caref                                         |
|         3 | Customer#000000003 | MG9kdTD2WBHm                           |           1 |
    ↪ 11-719-748-3364 |   7498.12 | AUTOMOBILE   |  deposits eat slyly ironic, even
    ↪ instructions. express foxes detect slyly. blithely even accounts abov |
|        32 | Customer#000000032 | jD2xZzi UmId,DCtNBLXKj9q0Tlp2iQ6ZcO3J  |          15 |
    ↪ 25-430-914-2194 |   3471.53 | BUILDING     | cial ideas. final, furious requests across
    ↪ the e                                                           |
+--
    ↪ ---------+--------------------+----------------------------------------+-------------+-----------------+--
    ↪
4 rows in set (1.89 sec)
```

05 读取增量数据

我们可以通过 Flink SQL 更新 Paimon 表中的数据：

```
Flink SQL> update customer set c_address='c_address_update' where c_nationkey = 1;
[INFO] Submitting SQL update statement to the cluster...
[INFO] SQL update statement has been successfully submitted to the cluster:
Job ID: ff838b7b778a94396b332b0d93c8f7ac
```

等 Flink SQL 执行完毕后，在 Doris 中可直接查看到最新的数据：

```
mysql> select * from customer where c_nationkey=1 limit 2;
+--
    ↪ ---------+--------------------+-----------------+------------+-----------------+-----------+-----------
    ↪
| c_custkey | c_name             | c_address       | c_nationkey | c_phone         | c_acctbal |
    ↪ c_mktsegment | c_comment
    ↪
    ↪ |
+--
    ↪ ---------+--------------------+-----------------+------------+-----------------+-----------+-----------
    ↪
```

```
|         3 | Customer#000000003 | c_address_update |           1 | 11-719-748-3364 |   7498.12 |
    ↪  AUTOMOBILE    |  deposits eat slyly ironic, even instructions. express foxes detect slyly
    ↪ . blithely even accounts abov |
|       513 | Customer#000000513 | c_address_update |           1 | 11-861-303-6887 |    955.37 |
    ↪  HOUSEHOLD     | press along the quickly regular instructions. regular requests against
    ↪ the carefully ironic s          |
+--
    ↪ ---------+--------------------+-----------------+-------------+-----------------+-----------+-----------
    ↪
2 rows in set (0.19 sec)
```

性能测试

我们在 Paimon（0.8）版本的 TPCDS 1000 数据集上进行了简单的测试，分别使用了 Apache Doris 2.1.5 版本和 Trino 422 版本，均开启 Primary Key Table Read Optimized 功能。

从测试结果可以看到，Doris 在标准静态测试集上的平均查询性能是 Trino 的 3~5 倍。后续我们将针对 Deletion Vector 进行优化，进一步提升真实业务场景下的查询效率。

2.14.13.3.3 查询优化

对于基线数据来说，Apache Paimon 在 0.6 版本中引入 Primary Key Table Read Optimized 功能后，使得查询引擎可以直接访问底层的 Parquet/ORC 文件，大幅提升了基线数据的读取效率。对于尚未合并的增量数据（INSERT、UPDATE 或 DELETE 所产生的数据增量）来说，可以通过 Merge-on-Read 的方式进行读取。此外，Paimon 在 0.8 版本中还引入的 Deletion Vector 功能，能够进一步提升查询引擎对增量数据的读取效率。Apache Doris 支持通过原生的 Reader 读取 Deletion Vector 并进行 Merge on Read，我们通过 Doris 的 EXPLAIN 语句，来演示在一个查询中，基线数据和增量数据的查询方式。

```
mysql> explain verbose select * from customer where c_nationkey < 3;
+--
    ↪ ------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ------------------------------------------------------------------------------------------------
    ↪
| ..............
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VPAIMON_SCAN_NODE(68)
    ↪
```

```
    ↪ |
|      table: customer
    ↪
    ↪ |
|      predicates: (c_nationkey[#3] < 3)
    ↪
    ↪ |
|      inputSplitNum=4, totalFileSize=238324, scanRanges=4
    ↪                                                                                |
|      partition=3/0
    ↪
    ↪ |
|      backends:
    ↪
    ↪ |
|        10002
    ↪
    ↪ |
|          s3://warehouse/wh/db_paimon.db/customer/c_nationkey=1/bucket-0/data-15cee5b7-1bd7-42ca
    ↪ -9314-56d92c62c03b-0.orc start: 0 length: 66600 |
|          s3://warehouse/wh/db_paimon.db/customer/c_nationkey=1/bucket-0/data-5d50255a
    ↪ -2215-4010-b976-d5dc656f3444-0.orc start: 0 length: 44501 |
|          s3://warehouse/wh/db_paimon.db/customer/c_nationkey=2/bucket-0/data-e98fb7ef-ec2b-4ad5
    ↪ -a496-713cb9481d56-0.orc start: 0 length: 64059 |
|          s3://warehouse/wh/db_paimon.db/customer/c_nationkey=0/bucket-0/data-431be05d-50fa-401f
    ↪ -9680-d646757d0f95-0.orc start: 0 length: 63164 |
|      cardinality=18751, numNodes=1
    ↪
    ↪ |
|      pushdown agg=NONE
    ↪
    ↪ |
|      paimonNativeReadSplits=4/4
    ↪
    ↪ |
|      PaimonSplitStats:
    ↪
    ↪ |
|        SplitStat [type=NATIVE, rowCount=1542, rawFileConvertable=true, hasDeletionVector=true]
    ↪                                                  |
|        SplitStat [type=NATIVE, rowCount=750, rawFileConvertable=true, hasDeletionVector=false]
    ↪                                                  |
|        SplitStat [type=NATIVE, rowCount=750, rawFileConvertable=true, hasDeletionVector=false]
    ↪                                                  |
|      tuple ids: 0
```

```
| ..............
    ↪
    ↪ |
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
67 rows in set (0.23 sec)
```

可以看到，对于刚才通过 Flink SQL 更新的表，包含 4 个分片，并且全部分片都可以通过 Native Reader 进行访问（`paimonNativeReadSplits=4/4`）。并且第一个分片的`hasDeletionVector`的属性为`true`，表示该分片有对应的 Deletion Vector，读取时会根据 Deletion Vector 进行数据过滤。

2.14.13.4 使用 Doris 和 Iceberg

作为一种全新的开放式的数据管理架构，湖仓一体（Data Lakehouse）融合了数据仓库的高性能、实时性以及数据湖的低成本、灵活性等优势，帮助用户更加便捷地满足各种数据处理分析的需求，在企业的大数据体系中已经得到越来越多的应用。

在过去多个版本中，Apache Doris 持续加深与数据湖的融合，当前已演进出一套成熟的湖仓一体解决方案。

- 自 0.15 版本起，Apache Doris 引入 Hive 和 Iceberg 外部表，尝试在 Apache Iceberg 之上探索与数据湖的能力结合。
- 自 1.2 版本起，Apache Doris 正式引入 Multi-Catalog 功能，实现了多种数据源的自动元数据映射和数据访问、并对外部数据读取和查询执行等方面做了诸多性能优化，完全具备了构建极速易用 Lakehouse 架构的能力。
- 在 2.1 版本中，Apache Doris 湖仓一体架构得到全面加强，不仅增强了主流数据湖格式（Hudi、Iceberg、Paimon 等）的读取和写入能力，还引入了多 SQL 方言兼容、可从原有系统无缝切换至 Apache Doris。在数据科学及大规模数据读取场景上，Doris 集成了 Arrow Flight 高速读取接口，使得数据传输效率实现 100 倍的提升。

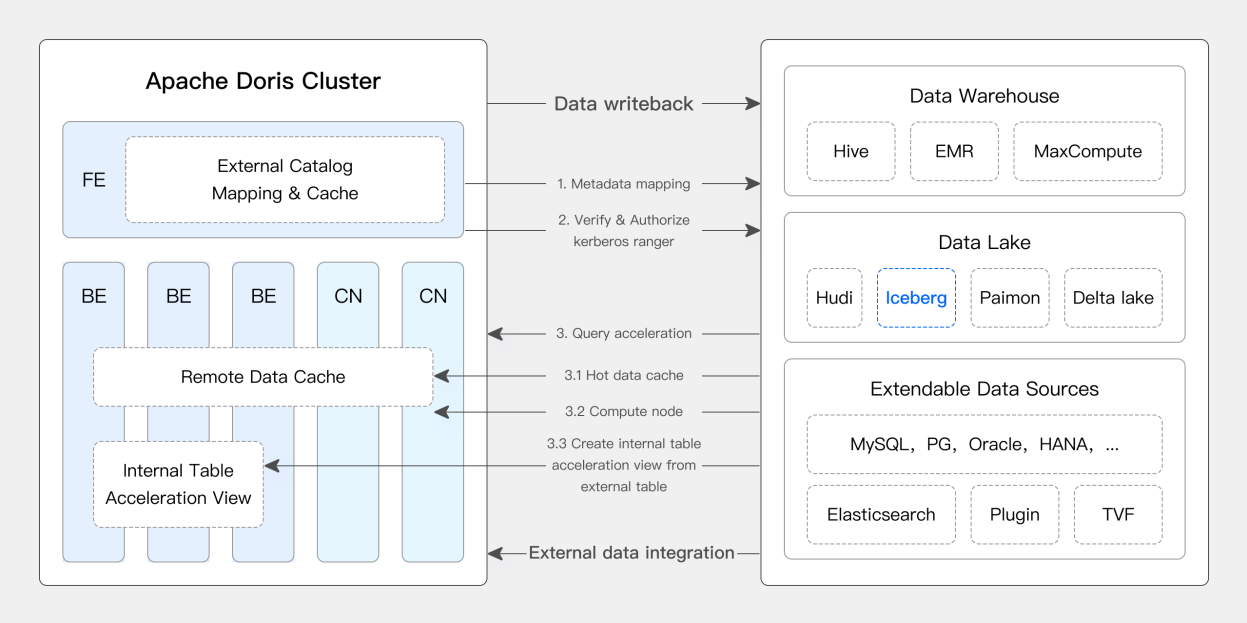


图 76: 使用 Doris 和 Iceberg 构建 Lakehouse

2.14.13.4.1 Apache Doris & Iceberg

Apache Iceberg 是一种开源、高性能、高可靠的数据湖表格式，可实现超大规模数据的分析与管理。它支持 Apache Doris 在内的多种主流查询引擎，兼容 HDFS 以及各种对象云存储，具备 ACID、Schema 演进、高级过滤、隐藏分区和分区布局演进等特性，可确保高性能查询以及数据的可靠性及一致性，其时间旅行和版本回滚功能也为数据管理带来较高的灵活性。

Apache Doris 对 Iceberg 多项核心特性提供了原生支持：

- 支持 Hive Metastore、 Hadoop、 REST、 Glue、 Google Dataproc Metastore、 DLF 等多种 Iceberg Catalog 类型。
- 原生支持 Iceberg V1/V2 表格式，以及 Position Delete、 Equality Delete 文件的读取。
- 支持通过表函数查询 Iceberg 表快照历史。
- 支持时间旅行（Time Travel）功能。
- 原生支持 Iceberg 表引擎。可以通过 Apache Doris 直接创建、管理以及将数据写入到 Iceberg 表。支持完善的分区 Transform 函数，从而提供隐藏分区和分区布局演进等能力。

用户可以基于 Apache Doris + Apache Iceberg 快速构建高效的湖仓一体解决方案，以灵活应对实时数据分析与处理的各种需求：

- 通过 Doris 高性能查询引擎对 Iceberg 表数据和其他数据源进行关联数据分析，构建统一的联邦数据分析平台。
- 通过 Doris 直接管理和构建 Iceberg 表，在 Doris 中完成对数据的清洗、加工并写入到 Iceberg 表，构建统一的湖仓数据处理平台。
- 通过 Iceberg 表引擎，将 Doris 数据共享给其他上下游系统做进一步处理，构建统一的开放数据存储平台。

未来，Apache Iceberg 将作为 Apache Doris 的原生表引擎之一，提供更加完善的湖格式数据的分析、管理功能。Apache Doris 也将逐步支持包括 Update/Delete/Merge、写回时排序、增量数据读取、元数据管理等 Apache Iceberg 更多高级特性，共同构建统一、高性能、实时的湖仓平台。

关于更多说明，请参阅 Iceberg Catalog

2.14.13.4.2 使用指南

本文档主要讲解如何在 Docker 环境下快速搭建 Apache Doris + Apache Iceberg 测试 & 演示环境，并展示各功能的使用操作。

本文涉及所有脚本和代码可以从该地址获取：https://github.com/apache/doris/tree/master/samples/datalake/iceberg_and_paimon

01 环境准备

本文示例采用 Docker Compose 部署，组件及版本号如下：

组件名称	版本
Apache Doris	默认 2.1.5，可修改
Apache Iceberg	1.4.3
MinIO	RELEASE.2024-04-29T09-56-05Z

02 环境部署

1. 启动所有组件

   ```
   bash ./start_all.sh
   ```

2. 启动后，可以使用如下脚本，登陆 Doris 命令行：

```
-- login doris
bash ./start_doris_client.sh
```

03 创建 Iceberg 表

首先登陆 Doris 命令行后，Doris 集群中已经创建了名为 Iceberg 的 Catalog（可通过 SHOW CATALOGS/SHOW CREATE CATALOG iceberg 查看）。以下为该 Catalog 的创建语句：

```
-- 已创建，无需执行
CREATE CATALOG `iceberg` PROPERTIES (
    "type" = "iceberg",
    "iceberg.catalog.type" = "rest",
    "warehouse" = "s3://warehouse/",
    "uri" = "http://rest:8181",
    "s3.access_key" = "admin",
    "s3.secret_key" = "password",
    "s3.endpoint" = "http://minio:9000"
);
```

在 Iceberg Catalog 创建数据库和 Iceberg 表：

```
mysql> SWITCH iceberg;
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE DATABASE nyc;
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE iceberg.nyc.taxis
       (
           vendor_id BIGINT,
           trip_id BIGINT,
           trip_distance FLOAT,
           fare_amount DOUBLE,
           store_and_fwd_flag STRING,
           ts DATETIME
       )
       PARTITION BY LIST (vendor_id, DAY(ts)) ()
       PROPERTIES (
           "compression-codec" = "zstd",
           "write-format" = "parquet"
       );
Query OK, 0 rows affected (0.15 sec)
```

04 数据写入

向 Iceberg 表中插入数据：

```
mysql> INSERT INTO iceberg.nyc.taxis
       VALUES
        (1, 1000371, 1.8, 15.32, 'N', '2024-01-01 9:15:23'),
        (2, 1000372, 2.5, 22.15, 'N', '2024-01-02 12:10:11'),
        (2, 1000373, 0.9, 9.01, 'N', '2024-01-01 3:25:15'),
        (1, 1000374, 8.4, 42.13, 'Y', '2024-01-03 7:12:33');
Query OK, 4 rows affected (1.61 sec)
{'status':'COMMITTED', 'txnId':'10085'}
```

通过 CREATE TABLE AS SELECT 来创建一张 Iceberg 表：

```
mysql> CREATE TABLE iceberg.nyc.taxis2 AS SELECT * FROM iceberg.nyc.taxis;
Query OK, 6 rows affected (0.25 sec)
{'status':'COMMITTED', 'txnId':'10088'}
```

05 数据查询

- 简单查询

```
mysql> SELECT * FROM iceberg.nyc.taxis;
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
4 rows in set (0.37 sec)

mysql> SELECT * FROM iceberg.nyc.taxis2;
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
4 rows in set (0.35 sec)
```

- 分区剪裁

```
mysql> SELECT * FROM iceberg.nyc.taxis where vendor_id = 2 and ts >= '2024-01-01' and ts < '
    ↪ 2024-01-02';
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
+--
    ↪ ---------+---------+--------------+-------------+-------------------+--------------------------+
    ↪
1 row in set (0.06 sec)

mysql> EXPLAIN VERBOSE SELECT * FROM iceberg.nyc.taxis where vendor_id = 2 and ts >= '
    ↪ 2024-01-01' and ts < '2024-01-02';

....
|   0:VICEBERG_SCAN_NODE(71)
|      table: taxis
|      predicates: (ts[#5] < '2024-01-02 00:00:00'), (vendor_id[#0] = 2), (ts[#5] >= '
    ↪ 2024-01-01 00:00:00')
|      inputSplitNum=1, totalFileSize=3539, scanRanges=1
|      partition=1/0
|      backends:
|        10002
|          s3://warehouse/wh/nyc/taxis/data/vendor_id=2/ts_day=2024-01-01/40e6ca404efa4a44-
    ↪ b888f23546d3a69c_5708e229-2f3d-4b68-a66b-44298a9d9815-0.zstd.parquet start: 0 length:
    ↪  3539
|      cardinality=6, numNodes=1
|      pushdown agg=NONE
|      icebergPredicatePushdown=
|           ref(name="ts") < 1704153600000000
|           ref(name="vendor_id") == 2
|           ref(name="ts") >= 1704067200000000
....
```

通过 `EXPLAIN VERBOSE` 语句的结果可知，`vendor_id = 2 and ts >= '2024-01-01' and ts <
    ↪ '2024-01-02'` 谓词条件，最终只命中一个分区（`partition=1/0`）。

同时也可知，因为在建表时指定了分区 Transform 函数 `DAY(ts)`，原始数据中的的值 `2024-01-01 03:25:15.000000` 会被转换成文件目录中的分区信息 `ts_day=2024-01-01`。

06 Time Travel

我们先再次插入几行数据：

```
INSERT INTO iceberg.nyc.taxis VALUES (1, 1000375, 8.8, 55.55, 'Y', '2024-01-01 8:10:22'), (3, 1000376, 7.4, 32.35, 'N', '2024-01-02  1:14:45');
Query OK, 2 rows affected (0.17 sec)
{'status':'COMMITTED', 'txnId':'10086'}

mysql> SELECT * FROM iceberg.nyc.taxis;
+---------+---------+---------------+-------------+-------------------+-------------------------+
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts                      |
+---------+---------+---------------+-------------+-------------------+-------------------------+
|         3 | 1000376 |           7.4 |       32.35 | N                  | 2024-01-02 01:14:45.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02 12:10:11.000000 |
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03 07:12:33.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01 09:15:23.000000 |
|         1 | 1000375 |           8.8 |       55.55 | Y                  | 2024-01-01 08:10:22.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01 03:25:15.000000 |
+---------+---------+---------------+-------------+-------------------+-------------------------+
6 rows in set (0.11 sec)
```

使用 iceberg_meta 表函数查询表的快照信息：

```
mysql> select * from iceberg_meta("table" = "iceberg.nyc.taxis", "query_type" = "snapshots");
+-------------------+---------------------+---------------------+-----------+-----------------------------
```

```
    ↪
    ↪ |
+--
    ↪ -------------------+---------------------+---------------------+-----------+-----------------------------
    ↪
| 2024-07-29 03:38:22 | 8483933166442433486 |                  -1 | append    | s3://warehouse/wh
    ↪ /nyc/taxis/metadata/snap-8483933166442433486-1-5f7b7736-8022-4ba1-9db2-51ae7553be4d.avro
    ↪ | {"added-data-files":"4","added-records":"4","added-files-size":"14156","changed-
    ↪ partition-count":"4","total-records":"4","total-files-size":"14156","total-data-files":"4
    ↪ ","total-delete-files":"0","total-position-deletes":"0","total-equality-deletes":"0"} |
| 2024-07-29 03:40:23 | 4726331391239920914 | 8483933166442433486 | append    | s3://warehouse/wh
    ↪ /nyc/taxis/metadata/snap-4726331391239920914-1-6aa3d142-6c9c-4553-9c04-08ad4d49a4ea.avro
    ↪ | {"added-data-files":"2","added-records":"2","added-files-size":"7078","changed-
    ↪ partition-count":"2","total-records":"6","total-files-size":"21234","total-data-files":"6
    ↪ ","total-delete-files":"0","total-position-deletes":"0","total-equality-deletes":"0"}  |
+--
    ↪ -------------------+---------------------+---------------------+-----------+-----------------------------
    ↪
2 rows in set (0.07 sec)
```

使用 FOR VERSION AS OF 语句查询指定快照：

```
mysql> SELECT * FROM iceberg.nyc.taxis FOR VERSION AS OF 8483933166442433486;
+--
    ↪ ---------+---------+---------------+-------------+--------------------+--------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+--------------------------+
    ↪
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+--------------------------+
    ↪
4 rows in set (0.05 sec)

mysql> SELECT * FROM iceberg.nyc.taxis FOR VERSION AS OF 4726331391239920914;
+--
```

```
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         1 | 1000375 |           8.8 |       55.55 | Y                  | 2024-01-01
    ↪ 08:10:22.000000 |
|         3 | 1000376 |           7.4 |       32.35 | N                  | 2024-01-02
    ↪ 01:14:45.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
6 rows in set (0.04 sec)
```

使用 FOR TIME AS OF 语句查询指定快照：

```
mysql> SELECT * FROM iceberg.nyc.taxis FOR TIME AS OF "2024-07-29 03:38:23";
+--
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                          |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
+--
    ↪ ---------+---------+---------------+-------------+--------------------+-------------------------+
    ↪
```

```
4 rows in set (0.04 sec)

mysql> SELECT * FROM iceberg.nyc.taxis FOR TIME AS OF "2024-07-29 03:40:22";
+--
    ↪ ---------+---------+---------------+-------------+-------------------+-------------------------+
    ↪
| vendor_id | trip_id | trip_distance | fare_amount | store_and_fwd_flag | ts
    ↪                         |
+--
    ↪ ---------+---------+---------------+-------------+-------------------+-------------------------+
    ↪
|         2 | 1000373 |           0.9 |        9.01 | N                  | 2024-01-01
    ↪ 03:25:15.000000 |
|         1 | 1000374 |           8.4 |       42.13 | Y                  | 2024-01-03
    ↪ 07:12:33.000000 |
|         2 | 1000372 |           2.5 |       22.15 | N                  | 2024-01-02
    ↪ 12:10:11.000000 |
|         1 | 1000371 |           1.8 |       15.32 | N                  | 2024-01-01
    ↪ 09:15:23.000000 |
+--
    ↪ ---------+---------+---------------+-------------+-------------------+-------------------------+
    ↪
4 rows in set (0.05 sec)
```

07 与 PyIceberg 交互

> 请使用 Doris 2.1.8/3.0.4 以上版本。

加载 Iceberg 表：

```
from pyiceberg.catalog import load_catalog

catalog = load_catalog(
    "iceberg",
    **{
        "warehouse" = "warehouse",
        "uri" = "http://rest:8181",
        "s3.access-key-id" = "admin",
        "s3.secret-access-key" = "password",
        "s3.endpoint" = "http://minio:9000"
    },
)
table = catalog.load_table("nyc.taxis")
```

读取为 Arrow Table：

```
print(table.scan().to_arrow())

pyarrow.Table
vendor_id: int64
trip_id: int64
trip_distance: float
fare_amount: double
store_and_fwd_flag: large_string
ts: timestamp[us]
----
vendor_id: [[1],[1],[2],[2]]
trip_id: [[1000371],[1000374],[1000373],[1000372]]
trip_distance: [[1.8],[8.4],[0.9],[2.5]]
fare_amount: [[15.32],[42.13],[9.01],[22.15]]
store_and_fwd_flag: [["N"],["Y"],["N"],["N"]]
ts: [[2024-01-01 09:15:23.000000],[2024-01-03 07:12:33.000000],[2024-01-01
    ↪ 03:25:15.000000],[2024-01-02 12:10:11.000000]]
```

读取为 Pandas DataFrame：

```
print(table.scan().to_pandas())

vendor_id  trip_id  trip_distance  fare_amount store_and_fwd_flag                  ts
0          1  1000371            1.8        15.32                  N   2024-01-01 09:15:23
1          1  1000374            8.4        42.13                  Y   2024-01-03 07:12:33
2          2  1000373            0.9         9.01                  N   2024-01-01 03:25:15
3          2  1000372            2.5        22.15                  N   2024-01-02 12:10:11
```

读取为 Polars DataFrame：

```
import polars as pl

print(pl.scan_iceberg(table).collect())

shape: (4, 6)
┌─────────────┬──────────┬───────────────┬─────────────┬────────────────────┬─────────────────────
    ↪
│ vendor_id ┆ trip_id ┆ trip_distance ┆ fare_amount ┆ store_and_fwd_flag ┆ ts                  │
│ ---       ┆ ---     ┆ ---           ┆ ---         ┆ ---                ┆ ---                 │
│ i64       ┆ i64     ┆ f32           ┆ f64         ┆ str                ┆ datetime[μs]        │
╞═══════════╪══════════╪═══════════════╪═════════════╪════════════════════╪═════════════════════╡
│ 1         ┆ 1000371 ┆ 1.8           ┆ 15.32       ┆ N                  ┆ 2024-01-01 09:15:23 │
│ 1         ┆ 1000374 ┆ 8.4           ┆ 42.13       ┆ Y                  ┆ 2024-01-03 07:12:33 │
│ 2         ┆ 1000373 ┆ 0.9           ┆ 9.01        ┆ N                  ┆ 2024-01-01 03:25:15 │
│ 2         ┆ 1000372 ┆ 2.5           ┆ 22.15       ┆ N                  ┆ 2024-01-02 12:10:11 │
```

```
└──────────────┴──────────┴──────────────────────┴──────────────┴──
    ↪
```

通过 pyiceberg 写入 iceberg 数据，请参阅步骤

08 附录

通过 PyIceberg 写入数据

加载 Iceberg 表：

```
from pyiceberg.catalog import load_catalog

catalog = load_catalog(
    "iceberg",
    **{
        "warehouse" = "warehouse",
        "uri" = "http://rest:8181",
        "s3.access-key-id" = "admin",
        "s3.secret-access-key" = "password",
        "s3.endpoint" = "http://minio:9000"
    },
)
table = catalog.load_table("nyc.taxis")
```

Arrow Table 写入 Iceberg：

```
import pyarrow as pa

df = pa.Table.from_pydict(
    {
        "vendor_id": pa.array([1, 2, 2, 1], pa.int64()),
        "trip_id": pa.array([1000371, 1000372, 1000373, 1000374], pa.int64()),
        "trip_distance": pa.array([1.8, 2.5, 0.9, 8.4], pa.float32()),
        "fare_amount": pa.array([15.32, 22.15, 9.01, 42.13], pa.float64()),
        "store_and_fwd_flag": pa.array(["N", "N", "N", "Y"], pa.string()),
        "ts": pa.compute.strptime(
            ["2024-01-01 9:15:23", "2024-01-02 12:10:11", "2024-01-01 3:25:15", "2024-01-03
                ↪ 7:12:33"],
            "%Y-%m-%d %H:%M:%S",
            "us",
        ),
    }
)
```

```
table.append(df)
```

Pandas DataFrame 写入 Iceberg：

```
import pyarrow as pa
import pandas as pd

df = pd.DataFrame(
    {
        "vendor_id": pd.Series([1, 2, 2, 1]).astype("int64[pyarrow]"),
        "trip_id": pd.Series([1000371, 1000372, 1000373, 1000374]).astype("int64[pyarrow]"),
        "trip_distance": pd.Series([1.8, 2.5, 0.9, 8.4]).astype("float32[pyarrow]"),
        "fare_amount": pd.Series([15.32, 22.15, 9.01, 42.13]).astype("float64[pyarrow]"),
        "store_and_fwd_flag": pd.Series(["N", "N", "N", "Y"]).astype("string[pyarrow]"),
        "ts": pd.Series(["2024-01-01 9:15:23", "2024-01-02 12:10:11", "2024-01-01 3:25:15", "
            ↪ 2024-01-03 7:12:33"]).astype("timestamp[us][pyarrow]"),
    }
)
table.append(pa.Table.from_pandas(df))
```

Polars DataFrame 写入 Iceberg：

```
import polars as pl

df = pl.DataFrame(
    {
        "vendor_id": [1, 2, 2, 1],
        "trip_id": [1000371, 1000372, 1000373, 1000374],
        "trip_distance": [1.8, 2.5, 0.9, 8.4],
        "fare_amount": [15.32, 22.15, 9.01, 42.13],
        "store_and_fwd_flag": ["N", "N", "N", "Y"],
        "ts": ["2024-01-01 9:15:23", "2024-01-02 12:10:11", "2024-01-01 3:25:15", "2024-01-03
            ↪ 7:12:33"],
    },
    {
        "vendor_id": pl.Int64,
        "trip_id": pl.Int64,
        "trip_distance": pl.Float32,
        "fare_amount": pl.Float64,
        "store_and_fwd_flag": pl.String,
        "ts": pl.String,
    },
).with_columns(pl.col("ts").str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S"))
table.append(df.to_arrow())
```

2.14.13.5 使用 Doris 和 LakeSoul

作为一种全新的开放式的数据管理架构，湖仓一体（Data Lakehouse）融合了数据仓库的高性能、实时性以及数据湖的低成本、灵活性等优势，帮助用户更加便捷地满足各种数据处理分析的需求，在企业的大数据体系中已经得到越来越多的应用。

在过去多个版本中，Apache Doris 持续加深与数据湖的融合，当前已演进出一套成熟的湖仓一体解决方案。

- 自 0.15 版本起，Apache Doris 引入 Hive 和 Iceberg 外部表，尝试在 Apache Iceberg 之上探索与数据湖的能力结合。
- 自 1.2 版本起，Apache Doris 正式引入 Multi-Catalog 功能，实现了多种数据源的自动元数据映射和数据访问、并对外部数据读取和查询执行等方面做了诸多性能优化，完全具备了构建极速易用 Lakehouse 架构的能力。
- 在 2.1 版本中，Apache Doris 湖仓一体架构得到全面加强，不仅增强了主流数据湖格式（Hudi、Iceberg、Paimon 等）的读取和写入能力，还引入了多 SQL 方言兼容、可从原有系统无缝切换至 Apache Doris。在数据科学及大规模数据读取场景上，Doris 集成了 Arrow Flight 高速读取接口，使得数据传输效率实现 100 倍的提升。

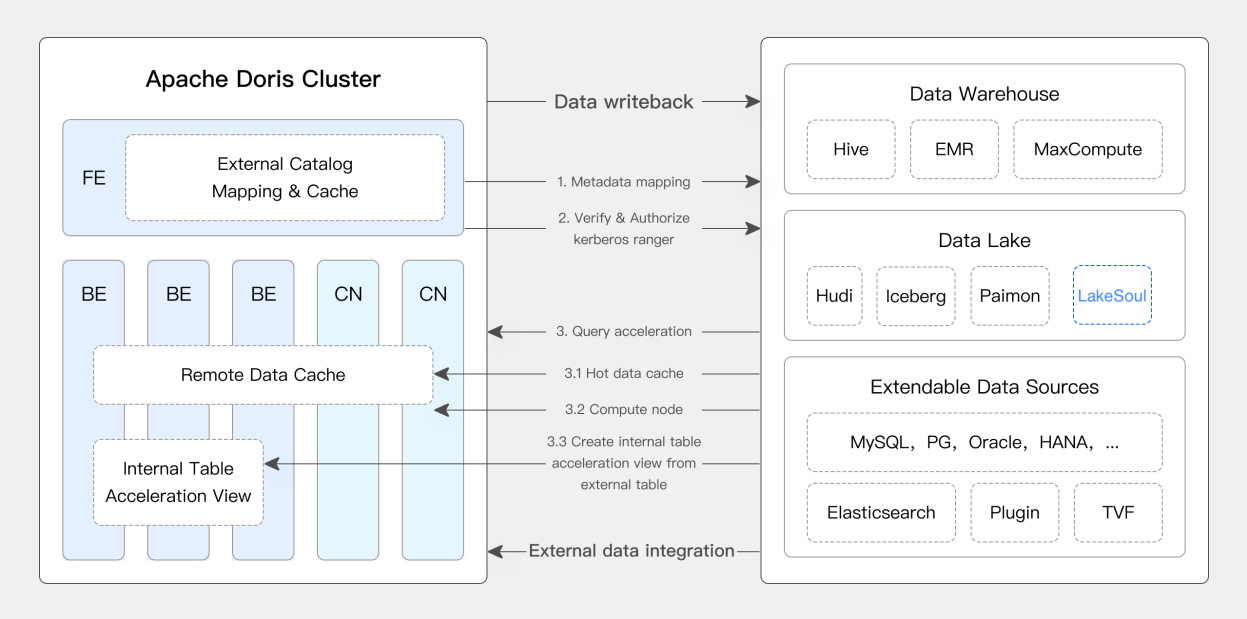


图 77: 使用 Doris 和 LakeSoul 构建 Lakehouse

2.14.13.5.1 Apache Doris & LakeSoul

LakeSoul 是由数元灵开发的云原生湖仓框架，并在 2023 年 5 月捐赠给了 Linux 基金会 AI & Data 基金会。它以元数据管理的高可扩展性、ACID 事务、高效灵活的 upsert 操作、模式演变和批流集成处理为特点。

借助 Apache Doris 的高性能查询引擎和 LakeSoul 的高效数据管理，用户可以实现：

- 实时数据入湖：利用 LakeSoul 的架构，数据可以以高效率和低延迟入湖，支持包括聚合、去重和部分列更新在内的各种数据更新能力。

- 高性能数据处理和分析：LakeSoul 的批流集成处理和模式演变等能力可以与 Doris 的强大查询引擎无缝集成，实现湖数据的快速查询和分析响应。未来，Apache Doris 将逐步支持 LakeSoul 的更多高级功能，如 CDC 流同步和自动模式演变，共同构建统一的、高性能的、实时的湖仓平台。

本文将解释如何快速搭建 Apache Doris + LakeSoul 测试和演示环境，并演示各种功能的使用方法，展示在湖仓架构中使用两个系统集成和优势。

关于更多说明，请参阅 LakeSoul Catalog

2.14.13.5.2 使用指南

本文涉及所有脚本和代码可以从该地址获取：https://github.com/apache/doris/tree/master/samples/datalake/lakesoul

01 环境准备

本文示例采用 Docker Compose 部署，组件及版本号如下：

组件名称	版本
Apache Doris	默认 3.0.2
LakeSoul	2.6.1
Postgres	14.5
Apache Spark	3.3.1
Apache Flink	1.17
MinIO	RELEASE.2024-04-29T09-56-05Z

02 环境部署

1. 启动所有组件

```
bash ./start_all.sh
```

2. 启动后，可以使用以下脚本登录到 Doris 命令行：

```
-- login doris
bash ./start_doris_client.sh
```

03 数据查询

如下所示，在 Doris 集群中已经创建了一个名为 lakesoul 的 Catalog（可使用 SHOW CATALOGS 查看）。以下是该 Catalog 的创建语句：

```
-- Already created
CREATE CATALOG `lakesoul` PROPERTIES (
  'type'='lakesoul',
  'lakesoul.pg.username'='lakesoul_test',
  'lakesoul.pg.password'='lakesoul_test',
```

```
    'lakesoul.pg.url'='jdbc:postgresql://lakesoul-meta-pg:5432/lakesoul_test?stringtype=
        ↪ unspecified',
    'minio.endpoint'='http://minio:9000',
    'minio.access_key'='admin',
    'minio.secret_key'='password'
  );
```

LakeSoul 表 `lakesoul.tpch.customer` 已加载到 Doris 中。在 Doris 中查询数据。

- 查询数据 “`sql Doris> uselakesoul.tpch ‘; Database changed`

Doris> show tables; +——————————+ | Tables_in_tpch | +——————————+ | customer_from_spark | +——————————+ 1 row in set (0.00 sec)

Doris> select * from customer_from_spark where c_nationkey = 1 order by c_custkey limit 4; +———–+——————————+———————————————————+————–+——————+————+———+ | c_custkey | c_name | c_address | c_nationkey | c_phone | c_acctbal | c_mktsegment | c_comment | +———–+——————————+———————————————————+————–+——————+————+———+ | 3 | Customer#000000003 | MG9kdTD2WBHm | 1 | 11-719-748-3364 | 7498.12 | AUTOMOBILE | deposits eat slyly ironic, even instructions. express foxes detect slyly. blithely even accounts abov | | 14 | Customer#000000014 | KXkletMIL2JQEA | 1 | 11-845-129-3851 | 5266.30 | FURNITURE | , ironic packages across the unus | | 30 | Customer#000000030 | nJDsELGAavU63Jl0c5NKsKfL8rIJQQkQnYL2QJY | 1 | 11-764-165-5076 | 9321.01 | BUILDING | lithely final requests. furiously unusual account | | 59 | Customer#000000059 | zLOCP0wh92OtBihgspOGI4 | 1 | 11-355-584-3112 | 3458.60 | MACHINERY | ously final packages haggle blithely after the express deposits. furiou | +———–+——————————+———————————————————+————–+——————+————+———+ 4 rows in set (3.14 sec)

Doris> select * from customer_from_spark where c_nationkey = 1 order by c_custkey desc limit 4; +———–+——————————+———————————————————+————–+——————+————+———+ | c_custkey | c_name | c_address | c_nationkey | c_phone | c_acctbal | c_mktsegment | c_comment | +———–+——————————+———————————————————+————–+——————+————+———+ | 14983 | Customer#000014983 | ERN3vq5Fvt4DL | 1 | 11-424-279-1846 | 841.22 | AUTOMOBILE | furiously slyly special foxes. express theodolites cajole carefully. special dinos haggle pinto | | 14968 | Customer#000014968 | ,sykKTZBzVFl7ito1750v2TRYwmkRl2nvqGHwmx | 1 | 11-669-222-9657 | 6106.77 | HOUSEHOLD | ts above the furiously even deposits haggle across | | 14961 | Customer#000014961 | JEIORcsBp6RpLYH 9gNdDyWJ | 1 | 11-490-251-5554 | 4006.35 | HOUSEHOLD | quests detect carefully final platelets! quickly final frays haggle slyly blithely final acc | | 14940 | Customer#000014940 | bNoyCxPuqSwPLjbqjEUNGN d0mSP | 1 | 11-242-677-1085 | 8829.48 | HOUSEHOLD | ver the quickly express braids. regular dependencies haggle fluffily quickly i | +———–+——————————+———————————————————+————–+——————+————+———+ 4 rows in set (0.10 sec) “ ‘

- 分区裁剪 Doris 可以对 LakeSoul 执行分区裁剪，并通过原生读取加速查询过程。我们可以通过 `explain ↪ verbose` 来检查这一点。

```
Doris> explain verbose select * from customer_from_spark where c_nationkey < 3;
```

```
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
| Explain String(Old Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_custkey`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_name`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_address`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_nationkey`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_phone`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_acctbal`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_mktsegment`
    ↪
    ↪ |
|     `lakesoul`.`tpch`.`customer_from_spark`.`c_comment`
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VplanNodeName
    ↪
    ↪ |
|      table: customer_from_spark
    ↪
    ↪ |
|      predicates: (`c_nationkey` < 3)
    ↪
    ↪ |
|      inputSplitNum=12, totalFileSize=0, scanRanges=12
    ↪
    ↪ |
|      partition=0/0
    ↪
    ↪ |
|      backends:
    ↪
    ↪ |
|        10002
    ↪
    ↪ |
|          s3://lakesoul-test-bucket/data/tpch/customer_from_spark/c_nationkey=1/part
    ↪ -00000-0568c817-d6bc-4fa1-bb9e-b311069b131c_00000.c000.parquet start: 0 length: 0 |
|          s3://lakesoul-test-bucket/data/tpch/customer_from_spark/c_nationkey=1/part-00001-
    ↪ d99a8fe6-61ab-4285-94da-2f84f8746a8a_00001.c000.parquet start: 0 length: 0 |
|          s3://lakesoul-test-bucket/data/tpch/customer_from_spark/c_nationkey=1/part-00002-8
    ↪ a8e396f-685f-4b0f-87fa-e2a3fe5be87e_00002.c000.parquet start: 0 length: 0 |
|          ... other 8 files ...
    ↪
    ↪ |
|          s3://lakesoul-test-bucket/data/tpch/customer_from_spark/c_nationkey=0/part-00003-
    ↪ d5b598cd-5bed-412c-a26f-bb4bc9c937bc_00003.c000.parquet start: 0 length: 0 |
|      numNodes=1
```

```
    ↪
    ↪ |
|      pushdown agg=NONE
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=customer_from_spark}
    ↪
    ↪ |
|   SlotDescriptor{id=0, col=c_custkey, colUniqueId=0, type=int, nullable=false,
    ↪ isAutoIncrement=false, subColPath=null}
    ↪                                                 |
|   SlotDescriptor{id=1, col=c_name, colUniqueId=1, type=text, nullable=true, isAutoIncrement
    ↪ =false, subColPath=null}                                                      |
|   SlotDescriptor{id=2, col=c_address, colUniqueId=2, type=text, nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}
    ↪                                                 |
|   SlotDescriptor{id=3, col=c_nationkey, colUniqueId=3, type=int, nullable=false,
    ↪ isAutoIncrement=false, subColPath=null}                                         |
|   SlotDescriptor{id=4, col=c_phone, colUniqueId=4, type=text, nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}
    ↪                                                   |
|   SlotDescriptor{id=5, col=c_acctbal, colUniqueId=5, type=decimalv3(15,2), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}                                       |
|   SlotDescriptor{id=6, col=c_mktsegment, colUniqueId=6, type=text, nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}                                         |
|   SlotDescriptor{id=7, col=c_comment, colUniqueId=7, type=text, nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}
    ↪                                                 |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
57 rows in set (0.03 sec)
```

通过检查 `EXPLAIN VERBOSE` 语句的结果，可以看到谓词条件 `c_nationkey < 3` 最终只命中一个分区（
↪ partition=0/0）。

04 CDC 表支持

启动 Flink CDC 作业以同步 MySQL 表。MySQL 表在启动 `start_all.sh` 时已经被加载了。

```
bash start_flink_cdc_job.sh
```

```
Start flink-cdc job...
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/flink/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/
    ↪ StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/
    ↪ org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Loading class `com.mysql.jdbc.Driver'. This is deprecated. The new driver class is `com.mysql.cj.
    ↪ jdbc.Driver'. The driver is automatically registered via the SPI and manual loading of
    ↪ the driver class is generally unnecessary.
Job has been submitted with JobID d1b3641dcd1ad85c6b373d49b1867e68
```

Flink CDC 作业将启动。我们可以通过重新创建 LakeSoul Catalog 在 `doris client` 中检查启动过程。Flink CDC 作业启动后，我们可以在 `doris client` 中看到正在同步的 LakeSoul CDC 表。

```
Doris> show tables;
+----------------------+
| Tables_in_tpch       |
+----------------------+
| customer_from_spark |
+----------------------+
2 rows in set (0.00 sec)


Doris> drop catalog if exists lakesoul;
Query OK, 0 rows affected (0.00 sec)

Doris> create catalog `lakesoul`  properties ('type'='lakesoul', 'lakesoul.pg.username'='lakesoul
    ↪ _test', 'lakesoul.pg.password'='lakesoul_test', 'lakesoul.pg.url'='jdbc:postgresql://
    ↪ lakesoul-meta-pg:5432/lakesoul_test?stringtype=unspecified', 'minio.endpoint'='http://
    ↪ minio:9000', 'minio.access_key'='admin', 'minio.secret_key'='password');
Query OK, 0 rows affected (0.01 sec)

Doris> show tables;
+----------------------+
| Tables_in_tpch       |
+----------------------+
| customer             |
| customer_from_spark |
+----------------------+
```

```
2 rows in set (0.00 sec)

Doris> select c_custkey, c_name, c_address, c_nationkey , c_phone, c_acctbal , c_mktsegment , c_
    ↪ comment from lakesoul.tpch.customer where c_custkey < 10;
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
| c_custkey | c_name             | c_address                             | c_nationkey | c_phone
    ↪         | c_acctbal | c_mktsegment | c_comment
    ↪
    ↪ |
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
|         1 | Customer#000000001 | IVhzIApeRb ot,c,E                     |          15 |
    ↪ 25-989-741-2988 |    711.56 | BUILDING     | to the even, regular platelets. regular,
    ↪ ironic epitaphs nag e                                                 |
|         3 | Customer#000000003 | MG9kdTD2WBHm                          |           1 |
    ↪ 11-719-748-3364 |   7498.12 | AUTOMOBILE   |  deposits eat slyly ironic, even
    ↪ instructions. express foxes detect slyly. blithely even accounts abov            |
|         7 | Customer#000000007 | TcGe5gaZNgVePxU5kRrvXBfkasDTea        |          18 |
    ↪ 28-190-982-9759 |   9561.95 | AUTOMOBILE   | ainst the ironic, express theodolites.
    ↪ express, even pinto beans among the exp                              |
|         8 | Customer#000000008 | I0B10bB0AymmC, 0PrRYBCP1yGJ8xcBPmWhl5 |          17 |
    ↪ 27-147-574-9335 |   6819.74 | BUILDING     | among the slyly regular theodolites kindle
    ↪ blithely courts. carefully even theodolites haggle slyly along the ide |
|         2 | Customer#000000002 | XSTf4,NCwDVaWNe6tEgvwfmRchLXak        |          13 |
    ↪ 23-768-687-3665 |    121.65 | AUTOMOBILE   | l accounts. blithely ironic theodolites
    ↪ integrate boldly: caref                                              |
|         4 | Customer#000000004 | XxVSJsLAGtn                           |           4 |
    ↪ 14-128-190-5944 |   2866.83 | MACHINERY    |  requests. final, regular ideas sleep final
    ↪ accou                                                                |
|         5 | Customer#000000005 | KvpyuHCplrB84WgAiGV6sYpZq7Tj          |           3 |
    ↪ 13-750-942-6364 |    794.47 | HOUSEHOLD    | n accounts will have to unwind. foxes cajole
    ↪  accor                                                               |
|         6 | Customer#000000006 | sKZz0CsnMD7mp4Xd0YrBvx,LREYKUWAh yVn   |          20 |
    ↪ 30-114-968-4951 |   7638.57 | AUTOMOBILE   | tions. even deposits boost according to the
    ↪ slyly bold packages. final accounts cajole requests. furious         |
|         9 | Customer#000000009 | xKiAFTjUsCuxfeleNqefumTrjS             |           8 |
    ↪ 18-338-906-3675 |   8324.07 | FURNITURE    | r theodolites according to the requests wake
    ↪  thinly excuses: pending requests haggle furiousl                    |
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
9 rows in set (1.09 sec)
```

进入 mysql client 并尝试修改数据。

```
bash start_mysql_client.sh
```

尝试从 mysql client 更新行。

```
mysql> update customer set c_acctbal=2211.26 where c_custkey=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

回到 doris client 并检查数据变化。

```
Doris> select c_custkey, c_name, c_address, c_nationkey , c_phone, c_acctbal , c_mktsegment , c_
    ↪ comment from lakesoul.tpch.customer where c_custkey < 10;
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
| c_custkey | c_name             | c_address                             | c_nationkey | c_phone
    ↪         | c_acctbal | c_mktsegment | c_comment
    ↪
    ↪ |
+--
    ↪ ---------+--------------------+---------------------------------------+-------------+-----------------+--
    ↪
|         2 | Customer#000000002 | XSTf4,NCwDVaWNe6tEgvwfmRchLXak        |          13 |
    ↪ 23-768-687-3665 |    121.65 | AUTOMOBILE   | l accounts. blithely ironic theodolites
    ↪ integrate boldly: caref                                                  |
|         4 | Customer#000000004 | XxVSJsLAGtn                           |           4 |
    ↪ 14-128-190-5944 |   2866.83 | MACHINERY    |  requests. final, regular ideas sleep final
    ↪ accou                                                                    |
|         5 | Customer#000000005 | KvpyuHCplrB84WgAiGV6sYpZq7Tj          |           3 |
    ↪ 13-750-942-6364 |    794.47 | HOUSEHOLD    | n accounts will have to unwind. foxes cajole
    ↪  accor                                                                   |
|         6 | Customer#000000006 | sKZz0CsnMD7mp4Xd0YrBvx,LREYKUWAh yVn  |          20 |
    ↪ 30-114-968-4951 |   7638.57 | AUTOMOBILE   | tions. even deposits boost according to the
    ↪ slyly bold packages. final accounts cajole requests. furious          |
|         9 | Customer#000000009 | xKiAFTjUsCuxfeleNqefumTrjS            |           8 |
    ↪ 18-338-906-3675 |   8324.07 | FURNITURE    | r theodolites according to the requests wake
    ↪  thinly excuses: pending requests haggle furiousl                        |
|         1 | Customer#000000001 | IVhzIApeRb ot,c,E                     |          15 |
    ↪ 25-989-741-2988 |   2211.26 | BUILDING     | to the even, regular platelets. regular,
    ↪ ironic epitaphs nag e                                                      |
|         3 | Customer#000000003 | MG9kdTD2WBHm                          |           1 |
    ↪ 11-719-748-3364 |   7498.12 | AUTOMOBILE   |  deposits eat slyly ironic, even
    ↪ instructions. express foxes detect slyly. blithely even accounts abov            |
|         7 | Customer#000000007 | TcGe5gaZNgVePxU5kRrvXBfkasDTea        |          18 |
    ↪ 28-190-982-9759 |   9561.95 | AUTOMOBILE   | ainst the ironic, express theodolites.
```

```
    ↪ express, even pinto beans among the exp                                    |
|         8 | Customer#000000008 | IOB1ObBOAymmC, OPrRYBCP1yGJ8xcBPmWhl5 |          17 |
    ↪ 27-147-574-9335 |   6819.74 | BUILDING     | among the slyly regular theodolites kindle
    ↪ blithely courts. carefully even theodolites haggle slyly along the ide |
+--
    ↪ ---------+--------------------+-----------------------------------------+-------------+-----------------+--
    ↪
9 rows in set (0.11 sec)
```

尝试从 `mysql client` 删除行。

```
mysql> delete from customer where c_custkey = 2;
Query OK, 1 row affected (0.01 sec)
```

回到 `doris client` 并检查数据变化。

```
Doris> select c_custkey, c_name, c_address, c_nationkey , c_phone, c_acctbal , c_mktsegment , c_
    ↪ comment from lakesoul.tpch.customer where c_custkey < 10;
+--
    ↪ ---------+--------------------+-----------------------------------------+-------------+-----------------+--
    ↪
| c_custkey | c_name             | c_address                               | c_nationkey | c_phone
    ↪         | c_acctbal | c_mktsegment | c_comment
    ↪
    ↪ |
+--
    ↪ ---------+--------------------+-----------------------------------------+-------------+-----------------+--
    ↪
|         6 | Customer#000000006 | sKZz0CsnMD7mp4Xd0YrBvx,LREYKUWAh yVn  |          20 |
    ↪ 30-114-968-4951 |   7638.57 | AUTOMOBILE   | tions. even deposits boost according to the
    ↪ slyly bold packages. final accounts cajole requests. furious          |
|         9 | Customer#000000009 | xKiAFTjUsCuxfeleNqefumTrjS            |           8 |
    ↪ 18-338-906-3675 |   8324.07 | FURNITURE    | r theodolites according to the requests wake
    ↪  thinly excuses: pending requests haggle furiousl                     |
|         1 | Customer#000000001 | IVhzIApeRb ot,c,E                     |          15 |
    ↪ 25-989-741-2988 |   2211.26 | BUILDING     | to the even, regular platelets. regular,
    ↪ ironic epitaphs nag e                                                 |
|         3 | Customer#000000003 | MG9kdTD2WBHm                          |           1 |
    ↪ 11-719-748-3364 |   7498.12 | AUTOMOBILE   |  deposits eat slyly ironic, even
    ↪ instructions. express foxes detect slyly. blithely even accounts abov            |
|         7 | Customer#000000007 | TcGe5gaZNgVePxU5kRrvXBfkasDTea        |          18 |
    ↪ 28-190-982-9759 |   9561.95 | AUTOMOBILE   | ainst the ironic, express theodolites.
    ↪ express, even pinto beans among the exp                                    |
|         8 | Customer#000000008 | IOB1ObBOAymmC, OPrRYBCP1yGJ8xcBPmWhl5 |          17 |
    ↪ 27-147-574-9335 |   6819.74 | BUILDING     | among the slyly regular theodolites kindle
    ↪ blithely courts. carefully even theodolites haggle slyly along the ide |
```

```
|         4 | Customer#000000004 | XxVSJsLAGtn                               |           4 |
    ↪ 14-128-190-5944 |   2866.83 | MACHINERY    |  requests. final, regular ideas sleep final
    ↪ accou                                                                  |
|         5 | Customer#000000005 | KvpyuHCplrB84WgAiGV6sYpZq7Tj              |           3 |
    ↪ 13-750-942-6364 |    794.47 | HOUSEHOLD    | n accounts will have to unwind. foxes cajole
    ↪  accor                                                                 |
+--
    ↪ ---------+--------------------+-----------------------------------------+-------------+-----------------+--
    ↪
8 rows in set (0.11 sec)
```

2.14.13.6 集成 Glue + AWS S3 Tables

AWS S3 Tables 是一种特殊的 S3 Bucket 类型，其对外提供 Apache Iceberg 表格式标准的读写接口，底层依托 Amazon S3，提供和 S3 本身相同的持久性、可用性、可扩展性和性能特征。此外，S3 Tables 还提供以下特性：

- 与存储在普通 S3 Buckets 中的 Iceberg 表格式相比，S3 Tables 的查询性能最多可高 3 倍，每秒事务数最多可高 10 倍。
- 自动化的表格管理。S3 Tables 会自动回 Iceberg 表数据进行优化，包括小文件合并、快照管理、垃圾文件清理等。

S3 Tables 的发布，进一步简化 Lakehouse 的架构，并为云原生的湖仓系统带来更多想象空间。包括冷热分离、数据归档、数据备份、存算分离架构，都可能基于 S3 Tables 演化出全新的架构。

得益于 Amazon S3 Tables 对 Iceberg API 的高度兼容，Apache Doris 可以和 S3 Tables 进行快速对接。本文将演示如何使用 Apache Doris 对接 S3 Tables 并进行数据分析加工。

> 该功能从 Doris 3.1 开始支持

2.14.13.6.1 使用指南

01 创建 S3 Table Bucket

S3 Table Bucket 是 S3 推出的第三种 Bucket 类型，和之前的 General purpose bucket 以及 Directory bucket 平级。

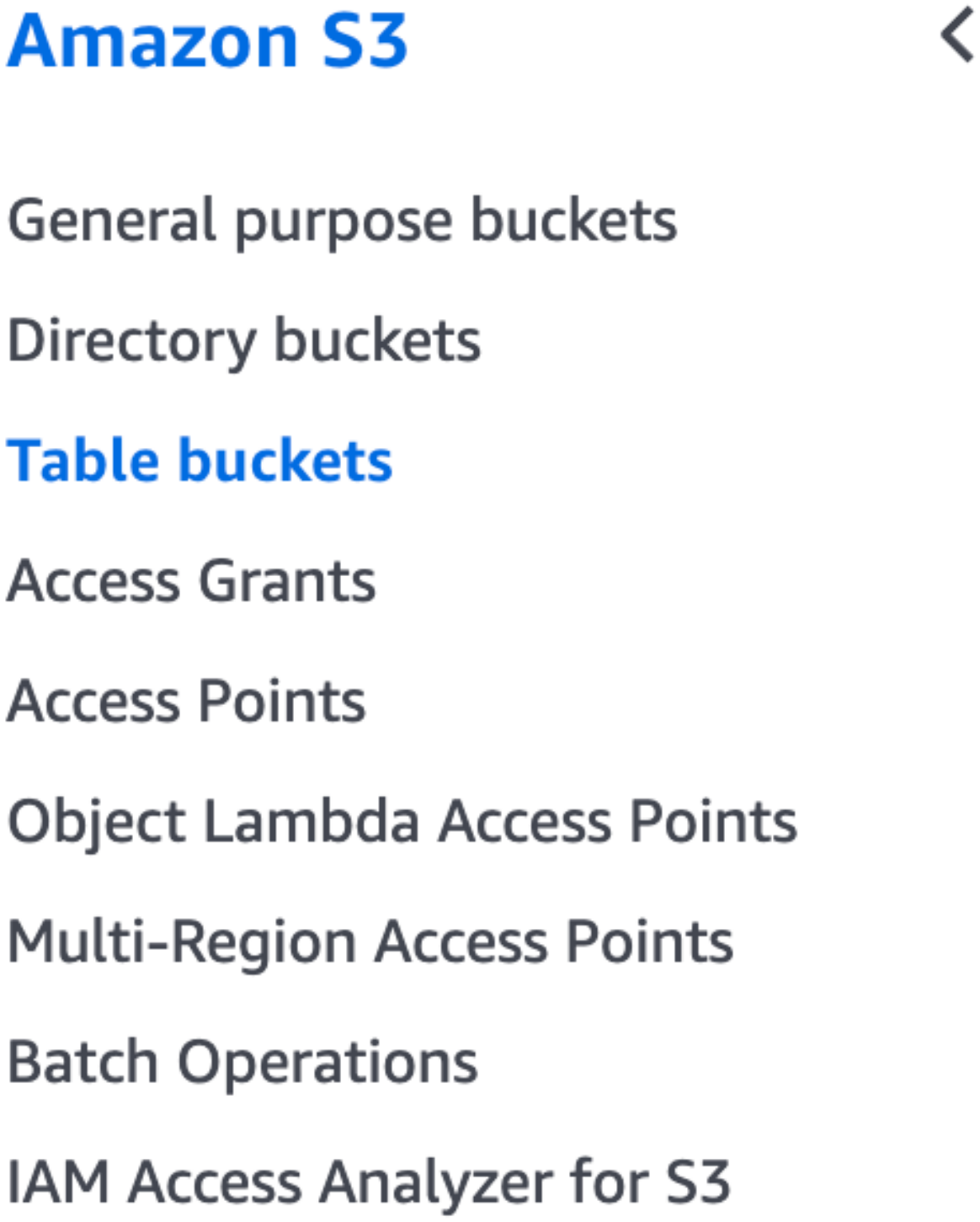


图 78: AWS S3 Table Bucket

这里我们创建一个名为 doris-s3-table-bucket 的 Table Bucket。创建后我们将得到一个 ARN 表示的 Table Bucket

图 79: AWS S3 Table Bucket Create

02 创建 Iceberg Catalog

- 创建一个 s3tables 类型的 Iceberg Catalog

```
CREATE CATALOG iceberg_s3 PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 's3tables',
    'warehouse' = 'arn:aws:s3tables:<region>:<acount_id>:bucket/<s3_table_bucket_name>',
    's3.region' = '<region>',
    's3.endpoint' = 's3.<region>.amazonaws.com',
    's3.access_key' = '<ak>',
    's3.secret_key' = '<sk>'
);
```

- 通过 Glue Rest Catalog 连接 s3 tables

```
CREATE CATALOG glue_s3 PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'https://glue.<region>.amazonaws.com/iceberg',
    'warehouse' = '<acount_id>:s3tablescatalog/<s3_table_bucket_name>',
    'iceberg.rest.sigv4-enabled' = 'true',
    'iceberg.rest.signing-name' = 'glue',
    'iceberg.rest.access-key-id' = '<ak>',
    'iceberg.rest.secret-access-key' = '<sk>',
    'iceberg.rest.signing-region' = '<region>'
);
```

03 访问 S3Tables

```
Doris > SWITCH iceberg_s3;

Doris > SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| my_namespace       |
```

```
| mysql              |
+--------------------+

Doris > USE my_namespace;

Doris > SHOW TABLES;
+------------------------+
| Tables_in_my_namespace |
+------------------------+
| my_table               |
+------------------------+

Doris > SELECT * FROM my_table;
+------+------+-------+
| id   | name | value |
+------+------+-------+
|    1 | ABC  |   100 |
|    2 | XYZ  |   200 |
+------+------+-------+
```

04 创建 S3Tables 表并写入数据

```
Doris > CREATE TABLE partition_table (
    ->   `ts` DATETIME COMMENT 'ts',
    ->   `id` INT COMMENT 'col1',
    ->   `pt1` STRING COMMENT 'pt1',
    ->   `pt2` STRING COMMENT 'pt2'
    -> )
    -> PARTITION BY LIST (day(ts), pt1, pt2) ();

Doris > INSERT INTO partition_table VALUES
    -> ("2024-01-01 08:00:00", 1000, "us-east", "PART1"),
    -> ("2024-01-02 10:00:00", 1002, "us-sout", "PART2");
Query OK, 2 rows affected
{'status':'COMMITTED', 'txnId':'1736935786473'}

Doris > SELECT * FROM partition_table;
+---------------------------+------+---------+-------+
| ts                        | id   | pt1     | pt2   |
+---------------------------+------+---------+-------+
| 2024-01-02 10:00:00.000000 | 1002 | us-sout | PART2 |
| 2024-01-01 08:00:00.000000 | 1000 | us-east | PART1 |
+---------------------------+------+---------+-------+
```

05 Time Travel

我们可以再插入一批数据，然后使用 $snapshots 系统表查看 Iceberg 的 Snapshots：

```
Doris > INSERT INTO partition_table VALUES
    -> ("2024-01-03 08:00:00", 1000, "us-east", "PART1"),
    -> ("2024-01-04 10:00:00", 1002, "us-sout", "PART2");
Query OK, 2 rows affected (9.76 sec)
{'status':'COMMITTED', 'txnId':'1736935786474'}
```

```
Doris > SELECT * FROM partition_table$snapshots`\G`
*************************** 1. row ***************************
 committed_at: 2025-01-15 23:27:01
  snapshot_id: 6834769222601914216
    parent_id: -1
    operation: append
manifest_list: s3://80afcb3f-6edf-46f2-7fhehwj6cengfwc7n6iz7ipzakd7quse1b--table-s3/metadata/snap
    ↪ -6834769222601914216-1-a6b2230d-fc0d-4c1d-8f20-94bb798f27b1.avro
      summary: {"added-data-files":"2","added-records":"2","added-files-size":"5152","changed-
          ↪ partition-count":"2","total-records":"2","total-files-size":"5152","total-data-
          ↪ files":"2","total-delete-files":"0","total-position-deletes":"0","total-equality-
          ↪ deletes":"0","iceberg-version":"Apache Iceberg 1.6.1 (commit 8
          ↪ e9d59d299be42b0bca9461457cd1e95dbaad086)"}
*************************** 2. row ***************************
 committed_at: 2025-01-15 23:30:00
  snapshot_id: 5670090782912867298
    parent_id: 6834769222601914216
    operation: append
manifest_list: s3://80afcb3f-6edf-46f2-7fhehwj6cengfwc7n6iz7ipzakd7quse1b--table-s3/metadata/snap
    ↪ -5670090782912867298-1-beeed339-be96-4710-858b-f39bb01cc3ff.avro
      summary: {"added-data-files":"2","added-records":"2","added-files-size":"5152","changed-
          ↪ partition-count":"2","total-records":"4","total-files-size":"10304","total-data-
          ↪ files":"4","total-delete-files":"0","total-position-deletes":"0","total-equality-
          ↪ deletes":"0","iceberg-version":"Apache Iceberg 1.6.1 (commit 8
          ↪ e9d59d299be42b0bca9461457cd1e95dbaad086)"}
```

使用 VERSION AS OF 语法查询不同的快照：

```
Doris > SELECT * FROM partition_table FOR VERSION AS OF 5670090782912867298;
+----------------------------+------+---------+-------+
| ts                         | id   | pt1     | pt2   |
+----------------------------+------+---------+-------+
| 2024-01-04 10:00:00.000000 | 1002 | us-sout | PART2 |
| 2024-01-03 08:00:00.000000 | 1000 | us-east | PART1 |
| 2024-01-01 08:00:00.000000 | 1000 | us-east | PART1 |
| 2024-01-02 10:00:00.000000 | 1002 | us-sout | PART2 |
+----------------------------+------+---------+-------+
```

```
Doris > SELECT * FROM partition_table FOR VERSION AS OF 6834769222601914216;
+---------------------------+------+---------+-------+
| ts                        | id   | pt1     | pt2   |
+---------------------------+------+---------+-------+
| 2024-01-02 10:00:00.000000 | 1002 | us-sout | PART2 |
| 2024-01-01 08:00:00.000000 | 1000 | us-east | PART1 |
+---------------------------+------+---------+-------+
```

06 使用 EMR Spark 访问 S3 Tables

使用 Doris 写入的数据，也可以使用 Spark 进行访问：

```
spark-shell --jars /usr/share/aws/iceberg/lib//iceberg-spark-runtime-3.5_2.12-1.6.1-amzn-1.jar \
--packages software.amazon.s3tables:s3-tables-catalog-for-iceberg-runtime:0.1.3 \
--conf spark.sql.catalog.s3tablesbucket=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.s3tablesbucket.catalog-impl=software.amazon.s3tables.iceberg.
    ↪ S3TablesCatalog \
--conf spark.sql.catalog.s3tablesbucket.warehouse=arn:aws:s3tables:us-east-1:169698000000:bucket/
    ↪ doris-s3-table-bucket \
--conf spark.sql.defaultCatalog=s3tablesbucket \
--conf spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions
```

```
scala> spark.sql("SELECT * FROM s3tablesbucket.my_namespace.`partition_table` ").show()
+-------------------+----+-------+-----+
|                 ts|  id|    pt1|  pt2|
+-------------------+----+-------+-----+
|2024-01-02 10:00:00|1002|us-sout|PART2|
|2024-01-01 08:00:00|1000|us-east|PART1|
|2024-01-04 10:00:00|1002|us-sout|PART2|
|2024-01-03 08:00:00|1000|us-east|PART1|
+-------------------+----+-------+-----+
```

2.14.13.7 集成阿里云 DLF Rest Catalog

阿里云数据湖构建 Data Lake Formation，DLF 作为云原生数据湖架构核心组成部分，帮助用户快速地构建云原生数据湖架构。数据湖构建提供湖上元数据统一管理、企业级权限控制，并无缝对接多种计算引擎，打破数据孤岛，洞察业务价值。

- 统一元数据与存储

  大数据计算引擎共享一套湖上元数据和存储，且数据可在环湖产品间流动。

- 统一权限管理

  大数据计算引擎共享一套湖表权限配置，实现一次配置，多处生效。

- 存储优化

  提供小文件合并、过期快照清理、分区整理及废弃文件清理等优化策略，提升存储效率。

- 完善的云生态支持体系

  深度整合阿里云产品，包括流批计算引擎，实现开箱即用，提升用户体验与操作便捷性。

DLF 2.5 版本开始支持 Paimon Rest Catalog。Doris 自 3.1.0 版本开始，支持集成 DLF 2.5+ 版本的 Paimon Rest Catalog，可以无缝对接 DLF，访问并分析 Paimon 表数据。本文将演示如何使用 Apache Doris 对接 DLF 2.5+ 版本并进行 Paimon 表数据访问。

> 该功能从 Doris 3.1 开始支持

2.14.13.7.1 使用指南

01 开通 DLF 服务

请参考 DLF 官方文档开通 DLF 服务，并创建相应的 Catalog、 Database 和 Table。

02 使用 EMR Spark SQL 访问 DLF

- 连接

```
spark-sql --master yarn \
    --conf spark.driver.memory=5g \
    --conf spark.sql.defaultCatalog=paimon \
    --conf spark.sql.catalog.paimon=org.apache.paimon.spark.SparkCatalog \
    --conf spark.sql.catalog.paimon.metastore=rest \
    --conf spark.sql.extensions=org.apache.paimon.spark.extensions.
        ↪ PaimonSparkSessionExtensions \
    --conf spark.sql.catalog.paimon.uri=http://<region>-vpc.dlf.aliyuncs.com \
    --conf spark.sql.catalog.paimon.warehouse=<your-catalog-name> \
    --conf spark.sql.catalog.paimon.token.provider=dlf \
    --conf spark.sql.catalog.paimon.dlf.token-loader=ecs
```

> 替换对应的 `warehouse` 和 `uri` 地址。

- 写入数据

```
USE <your-catalog-name>;

CREATE TABLE users_samples
(
    user_id INT,
    age_level STRING,
    final_gender_code STRING,
```

```
    clk BOOLEAN
);

INSERT INTO users_samples VALUES
(1, '25-34', 'M', true),
(2, '18-24', 'F', false);

INSERT INTO users_samples VALUES
(3, '25-34', 'M', true),
(4, '18-24', 'F', false);

INSERT INTO users_samples VALUES
(5, '25-34', 'M', true),
(6, '18-24', 'F', false);
```

如遇到以下错误，请尝试移除 `/opt/apps/PAIMON/paimon-dlf-2.5/lib/spark3` 下的 `paimon-jindo-x.y.z.jar` 后重启 Spark 服务并重试。

```
Ambiguous FileIO classes are:
org.apache.paimon.jindo.JindoLoader
org.apache.paimon.oss.OSSLoader
```

03 使用 Doris 链接 DLF

- 创建 Paimon Catalog

```
CREATE CATALOG paimon_dlf_test PROPERTIES (
    'type' = 'paimon',
    'paimon.catalog.type' = 'rest',
    'uri' = 'http://<region>-vpc.dlf.aliyuncs.com',
    'warehouse' = '<your-catalog-name>',
    'paimon.rest.token.provider' = 'dlf',
    'paimon.rest.dlf.access-key-id' = '<ak>',
    'paimon.rest.dlf.access-key-secret' = '<sk>'
);
```

- Doris 会使用 DLF 返回的临时凭证访问 OSS 对象存储，不需要额外提供 OSS 的凭证信息。
- 仅支持在同 VPC 内访问 DLF，注意提供正确的 uri 地址。

- 查询数据

```
SELECT * FROM users_samples ORDER BY user_id;
+---------+-----------+-------------------+------+
| user_id | age_level | final_gender_code | clk  |
```

```
+---------+-----------+-------------------+------+
|       1 | 25-34     | M                 |    1 |
|       2 | 18-24     | F                 |    0 |
|       3 | 25-34     | M                 |    1 |
|       4 | 18-24     | F                 |    0 |
|       5 | 25-34     | M                 |    1 |
|       6 | 18-24     | F                 |    0 |
+---------+-----------+-------------------+------+
```

- 查询系统表

```
SELECT snapshot_id, commit_time, total_record_count FROM users_samples$snapshots;
+-------------+-------------------------+--------------------+
| snapshot_id | commit_time             | total_record_count |
+-------------+-------------------------+--------------------+
|           1 | 2025-08-09 05:56:02.906 |                  2 |
|           2 | 2025-08-13 03:41:32.732 |                  4 |
|           3 | 2025-08-13 03:41:35.218 |                  6 |
+-------------+-------------------------+--------------------+
```

- 增量读取

```
SELECT * FROM users_samples@incr('startSnapshotId'=1, 'endSnapshotId'=2) ORDER BY user_id;
+---------+-----------+-------------------+------+
| user_id | age_level | final_gender_code | clk  |
+---------+-----------+-------------------+------+
|       3 | 25-34     | M                 |    1 |
|       4 | 18-24     | F                 |    0 |
+---------+-----------+-------------------+------+
```

2.14.13.8 从 MaxCompute 到 Doris

本文档介绍如何通过 MaxCompute Catalog 将阿里云 MaxCompute 中的数据快速导入到 Apache Doris 中。

本文档基于 Apache Doris 2.1.9 版本。

2.14.13.8.1 环境准备

01 开通 MaxCompute 开放存储 API

在 MaxCompute 控制台 左侧导航栏 -> 租户管理 -> 租户属性 -> 打开 开放存储(Storage API)开关

02 开通 MaxCompute 权限

Doris 使用 AK/SK 访问 MaxCompute 服务。请确保 AK/SK 对应的 IAM 用户，拥有对应 MaxCompute 服务的以下角色或权限：

```
{
    "Statement": [{
            "Action": ["odps:List",
                "odps:Usage"],
            "Effect": "Allow",
            "Resource": ["acs:odps:*:regions/*/quotas/pay-as-you-go"]}],
    "Version": "1"
}
```

03 确认 Doris 和 MaxCompute 网络环境

强烈建议 Doris 集群和 MaxCompute 服务在同一个 VPC 中，并确保设置了正确的安全组。

本文实例是在同 VPC 网络情况下的测试结果。

2.14.13.8.2 导入 MaxCompute 数据

01 创建 Catalog

```
CREATE CATALOG mc PROPERTIES (
  "type" = "max_compute",
  "mc.default.project" = "xxx",
  "mc.access_key" = "AKxxxxx",
  "mc.secret_key" = "SKxxxxx",
  "mc.endpoint" = "xxxxx"
);
```

如需支持 Schema 层级（3.1.3+）：

```
CREATE CATALOG mc PROPERTIES (
  "type" = "max_compute",
  "mc.default.project" = "xxx",
  "mc.access_key" = "AKxxxxx",
  "mc.secret_key" = "SKxxxxx",
  "mc.endpoint" = "xxxxx",
  'mc.enable.namespace.schema' = 'true'
);
```

具体请参阅 MaxCompute Catalog 文档。

02 导入 TPCH 数据集

我们使用 MaxCompute 公开数据集中的 TPCH 100 数据集作为示例（数据已经导入到 MaxCompute 中），并使用 CREATE TABLE AS SELECT 语句将 MaxCompute 的数据导入到 Doris 中。

该数据集有 7 张表。其中最大的 lineitem 表有 16 列，600037902 行。磁盘空间占用约为 30GB。

```
-- switch catalog
SWITCH internal;
```

```
-- create database
CREATE DATABASE tpch_100g;
-- ingest data
CREATE TABLE tpch_100g.lineitem AS SELECT * FROM mc.selectdb_test.lineitem;
CREATE TABLE tpch_100g.nation AS SELECT * FROM mc.selectdb_test.nation;
CREATE TABLE tpch_100g.orders AS SELECT * FROM mc.selectdb_test.orders;
CREATE TABLE tpch_100g.part AS SELECT * FROM mc.selectdb_test.part;
CREATE TABLE tpch_100g.partsupp AS SELECT * FROM mc.selectdb_test.partsupp;
CREATE TABLE tpch_100g.region AS SELECT * FROM mc.selectdb_test.region;
CREATE TABLE tpch_100g.supplier AS SELECT * FROM mc.selectdb_test.supplier;
```

在 Doris 集群单 BE 16C 64G 规格下，上述操作串行执行，耗时约为 6-7 分钟。

03 导入 Github Event 数据集

我们使用 MaxCompute 公开数据集中的 Github Event 数据集作为示例（数据已经导入到 MaxCompute 中），并使用 CREATE TABLE AS SELECT 语句将 MaxCompute 的数据导入到 Doris 中。

这里我们选择 dwd_github_events_odps 表的 '2015-01-01' 到 '2016-01-01' 共 365 个分区的数据。数据共 32 列，212786803 行。磁盘空间占用约为 10GB。

```
-- switch catalog
SWITCH internal;
-- create database
CREATE DATABASE github_events;
-- ingest data
CREATE TABLE github_events.dwd_github_events_odps
AS SELECT * FROM mc.github_events.dwd_github_events_odps
WHERE ds BETWEEN '2015-01-01' AND '2016-01-01';
```

在 Doris 集群单 BE 16C 64G 规格下，上述操作耗时约为 2 分钟。

2.14.13.9 集成 Apache Polaris

随着数据湖技术的不断演进，如何高效、安全地管理位于对象存储（如 AWS S3）之上的海量数据，并为上层分析引擎（如 Apache Doris）提供统一的访问入口，已成为现代数据架构的核心挑战。Apache Polaris 作为 Iceberg 的开放、标准化的 REST Catalog 服务，为此提供了完美的解决方案。它不仅负责元数据的集中管理，还通过精细化的权限控制和灵活的凭证管理机制，极大地增强了数据湖的安全性与可管理性。

本文将详细介绍如何将 Apache Doris 与 Polaris 进行集成，实现对 S3 上 Iceberg 数据的高效查询与管理。我们将一步步带你完成从环境准备到最终查询的全过程。

通过本文档，你将可以快速了解：

- AWS 环境准备：如何在 AWS 中创建并配置 S3 存储桶，以及为 Polaris 和 Doris 分别准备必须的 IAM 角色和策略，使得 Polaris 能够自身访问 S3，并能向 Doris 下发访问凭证。
- Polaris 部署与配置：如何在服务器上下载并启动 Polaris 服务，并在 Polaris 中创建 Iceberg Catalog、Namespace 及相应的 Principal/Role/权限，为 Doris 提供安全的元数据访问端点。

- Doris 连接 Polaris：说明 Doris 如何通过 OAuth2 向 Polaris 获取元数据访问令牌，并演示两种核心的底层存储访问方式：

1. 由 Polaris 发放临时 AK/SK（凭证发放机制，Credential Vending）
2. Doris 直接使用静态 AK/SK 访问 S3

- 连接方案对比：通过文字与流程图对比不同方案在元数据与存储层的工作链路、适用场景及安全性，为你提供选型参考。
- 附录：对文中出现的关键术语（如 Role, Policy, Principal 等）进行简要说明。

2.14.13.9.1 1. AWS 环境准备

在开始之前，我们需要在 AWS 上准备好 S3 存储桶和相应的 IAM 角色，这是 Polaris 管理数据和 Doris 访问数据的基础。

1.1 创建 S3 存储桶

首先，我们创建一个名为 `polaris-doris-demo` 的 S3 Bucket，用于存放后续创建的 Iceberg 表数据。

```
aws s3 mb s3://polaris-doris-demo --region us-west-2
#### 验证存储桶创建成功
aws s3 ls | grep polaris-doris-demo
```

1.2 创建访问对象存储的 IAM Role

为了实现安全的凭证管理，我们需要创建一个 IAM 角色供 Polaris 通过 STS AssumeRole 机制使用。这个设计遵循了最小权限原则和职责分离的安全最佳实践。

1. 创建信任策略文件

   创建 `polaris-trust-policy.json`文件：

```
cat > polaris-trust-policy.json << 'EOF'
{
    "Version": "2012-10-17",
    "Statement": [
    {
        "Effect": "Allow",
        "Principal": {
        "AWS": "arn:aws:iam::YOUR_ACCOUNT_ID:root"
        },
        "Action": "sts:AssumeRole",
        "Condition": {
        "StringEquals": {
            "sts:ExternalId": "polaris-doris-demo"
        }
        }
```

```
        }
        ]
    }
    EOF
```

> 注意：请将 YOUR_ACCOUNT_ID 替换为您的实际 AWS 账户 ID，可通过`aws sts get-caller-identity --query Account --output text`获取。

2. 创建 IAM Role

```
    aws iam create-role \
        --role-name polaris-doris-demo \
        --assume-role-policy-document file:///path/to/polaris-trust-policy.json \
        --description "IAM Role for Polaris to access S3 storage"
```

3. 附加 S3 访问权限策略

4. 验证创建结果

```
    aws iam get-role --role-name polaris-doris-demo
    aws iam list-attached-role-policies --role-name polaris-doris-demo
```

1.3 为 EC2 实例绑定 IAM Role（可选）

> 如不执行此步骤，则需要在 polaris 启动前设置 AWS_ACCESS_KEY_ID 与 AWS_SECRET_ACCESS_KEY

如果您的 Polaris 服务将运行在 EC2 实例上，最佳实践是为该 EC2 实例绑定 IAM 角色，而不是使用访问密钥。这样可以避免在代码中硬编码凭证，提高安全性。

1. 创建 EC2 实例角色的信任策略

   首先创建允许 EC2 服务承担此角色的信任策略文件：

```
    cat > ec2-trust-policy.json << 'EOF'
    {
    "Version": "2012-10-17",
    "Statement": [
        {
        "Effect": "Allow",
        "Principal": {
            "Service": "ec2.amazonaws.com"
        },
```

```
    "Action": "sts:AssumeRole"
    }
]
}
EOF
```

2. 创建 EC2 实例角色

```
aws iam create-role \
    --role-name polaris-ec2-role \
    --assume-role-policy-document file:///path/to/ec2-trust-policy.json \
    --description "IAM Role for EC2 instance running Polaris service"
```

3. 附加 S3 访问权限策略

```
# 附加 AmazonS3FullAccess 托管策略
aws iam attach-role-policy \
    --role-name polaris-ec2-role \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
```

4. 创建实例配置文件

```
# 创建实例配置文件
aws iam create-instance-profile \
    --instance-profile-name polaris-ec2-instance-profile

# 将角色添加到实例配置文件
aws iam add-role-to-instance-profile \
    --instance-profile-name polaris-ec2-instance-profile \
    --role-name polaris-ec2-role
```

5. 将实例配置文件附加到 EC2 实例

```
# 如果是新创建的 EC2 实例，在启动时指定
aws ec2 run-instances \
    --image-id ami-xxxxxxxxx \
    --instance-type t3.medium \
    --iam-instance-profile Name=polaris-ec2-instance-profile \
    --other-parameters...

# 如果是已存在的 EC2 实例，需要关联实例配置文件
aws ec2 associate-iam-instance-profile \
    --instance-id i-xxxxxxxxx \
    --iam-instance-profile Name=polaris-ec2-instance-profile
```

2.14.13.9.2　2. Polaris 部署与 Catalog 创建

环境准备就绪后，我们开始部署 Polaris 服务并配置 Catalog。

> 本文档采用源码快速启动的方式，更多部署方式参考：
> https://polaris.apache.org/releases/1.0.1/getting-started/deploying-polaris/

2.1 克隆源码并启动 Polaris

1. 克隆 Polaris 仓库并切换到特定版本

```
git clone https://github.com/apache/polaris.git
cd polaris
# Recommend using a released stable version
git checkout apache-polaris-1.0.1-incubating
```

2. 设置 AWS 凭证（可选）

   如果你不在 EC2 上运行 Polaris，或者 EC2 没有绑定相应的 IAM Role，你需要通过环境变量为 Polaris 提供一个有权代入 `polaris-doris-demo` 角色的 AK/SK。

```
export AWS_ACCESS_KEY_ID=YOUR_AWS_ACCESS_KEY_ID
export AWS_SECRET_ACCESS_KEY=YOUR_AWS_SECRET_ACCESS_KEY
```

3. 编译并运行 Polaris

   确保你已安装 Java 21+ 和 Docker 27+。

```
./gradlew run -Dpolaris.bootstrap.credentials=POLARIS,root,secret
```

- POLARIS is the realm
- `root` is the CLIENT_ID
- `secret` is the CLIENT_SECRET
- If credentials are not set, it will use preset credentials `POLARIS,root,s3cr3t`

此命令会启动 Polaris 服务，默认监听 8181 端口。

2.2 在 Polaris 中创建 Catalog 与 Namespace

1. 导出 ROOT 凭证

```
export CLIENT_ID=root
export CLIENT_SECRET=secret
```

2. 创建 Catalog (指向 S3 存储)

```
./polaris catalogs create \
--storage-type s3 \
--default-base-location s3://polaris-doris-test/polaris1 \
--role-arn arn:aws:iam::<account_id>:role/polaris-doris-test \
--external-id polaris-doris-test \
doris_catalog
```

```
* `--storage-type s3`: 指定底层存储为 S3。
* `--default-base-location`: Iceberg 表数据的默认根路径。
* `--role-arn`: Polaris 服务用于代入以访问 S3 的 IAM Role。
* `--external-id`: 代入角色时使用的外部 ID，与 IAM Role 信任策略中的配置保持一致。
```

3. 创建 Namespace

```
./polaris namespaces create --catalog doris_catalog doris_demo
```

这会在 doris_catalog 下创建一个名为 doris_demo 的数据库（Namespace）。

2.3 Polaris 安全角色与权限配置

为了让 Doris 能够以非 root 用户的身份访问，我们需要创建一个新的用户和角色，并授予其适当的权限。

1. 创建 Principal Role 和 Catalog Role

```
# 创建一个 Principal Role，用于聚合权限
./polaris principal-roles create doris_pr_role

# 在 doris_catalog 下创建一个 Catalog Role
./polaris catalog-roles create --catalog doris_catalog doris_catalog_role
```

2. 为 Catalog Role 授权

```
# 授予 doris_catalog_role 管理该 Catalog 内容的权限
./polaris privileges catalog grant \
    --catalog doris_catalog \
    --catalog-role doris_catalog_role \
    CATALOG_MANAGE_CONTENT
```

3. 关联 Principal Role 和 Catalog Role

```
# 将 doris_catalog_role 赋予 doris_pr_role
./polaris catalog-roles grant \
--catalog doris_catalog \
--principal-role doris_pr_role \
doris_catalog_role
```

4. 创建新的 Principal (用户) 并绑定 Role

```
# 创建一个名为 doris_user 的新用户（Principal）
./polaris principals create doris_user
# 输出示例： {"clientId": "6e155b128dc06c13", "clientSecret": "
    ↪ ce9fbb4cc91c43ff2955f2c6545239d7"}
# 请记下这对新的 client_id 和 client_secret，Doris 将使用它进行连接。

# 将 doris_user 绑定到 doris_pr_role
./polaris principal-roles grant \
doris_pr_role \
--principal doris_user
```

至此，Polaris 端的配置全部完成。我们创建了一个名为 doris_user 的用户，它通过 doris_pr_role 获得了管理 doris_catalog 的权限。

2.14.13.9.3 3. Doris 连接 Polaris

现在，我们将在 Doris 中创建一个 Iceberg Catalog，连接到刚刚配置好的 Polaris 服务。Doris 支持多种灵活的认证组合。

> 注意：此示例中我们使用 OAuth2 认证的 credential 来连接 Polaris 的 rest 服务，除此之外 Doris 还支持使用 iceberg.rest.oauth2.token 直接提供预先获取的 Bearer Token

方式一：OAuth2 + 临时存储凭证 (Credential Vending)

这是最推荐的方式。Doris 使用 OAuth2 凭证向 Polaris 认证并获取元数据，当需要读写 S3 上的数据文件时，Doris 会向 Polaris 请求一个临时的、具有最小权限的 S3 访问凭证。

使用你为 doris_user 生成的 clientId 和 clientSecret。

```
CREATE CATALOG polaris_vended PROPERTIES (
    'type' = 'iceberg',
    -- Polaris 中的 Catalog 名称
    'warehouse' = 'doris_catalog',
    'iceberg.catalog.type' = 'rest',
    -- Polaris 服务地址
```

```
    'iceberg.rest.uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog',
    -- 元数据认证方式
    'iceberg.rest.security.type' = 'oauth2',
    -- 替换为 doris_user 的 client_id:client_secret
    'iceberg.rest.oauth2.credential' = 'client_id:client_secret',
    'iceberg.rest.oauth2.server-uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog/v1/oauth/tokens
        ↪ ',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:doris_pr_role',
    -- 开启凭证发放
    'iceberg.rest.vended-credentials-enabled' = 'true'
);
```

方式二：OAuth2 + 静态存储凭证 (AK/SK)

这种方式下，Doris 同样使用 OAuth2 访问 Polaris 元数据，但访问 S3 数据时，使用的是在 Doris Catalog 配置中写死的静态 AK/SK。这种方式配置简单，适合快速测试，但安全性较低。

```
CREATE CATALOG polaris_aksk PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog',
    'iceberg.rest.warehouse' = 'doris_catalog',
    'iceberg.rest.security.type' = 'oauth2',
    'iceberg.rest.oauth2.credential' = '6e155b128dc06c13:ce9fbb4cc91c43ff2955f2c6545239d7',
    'iceberg.rest.oauth2.server-uri' = 'http://YOUR_POLARIS_HOST:8181/api/catalog/v1/oauth/tokens
        ↪ ',
    'iceberg.rest.oauth2.scope' = 'PRINCIPAL_ROLE:doris_pr_role',
    -- 直接提供 S3 访问密钥
    's3.access_key' = 'YOUR_S3_ACCESS_KEY',
    's3.secret_key' = 'YOUR_S3_SECRET_KEY',
    's3.endpoint' = 'https://s3.us-west-2.amazonaws.com',
    's3.region' = 'us-west-2'
);
```

2.14.13.9.4 4. 在 Doris 中验证连接

无论使用哪种方式创建 Catalog，你都可以通过以下 SQL 来验证端到端的连通性。

```
-- 切换到你创建的 Catalog 和在 Polaris 中配置的 Namespace
USE polaris_vended.doris_demo;

-- 创建一张 Iceberg 表
CREATE TABLE my_iceberg_table (
  id INT,
  name STRING
)
```

```
PROPERTIES (
  'write-format'='parquet'
);

-- 插入数据
INSERT INTO my_iceberg_table VALUES (1, 'Doris'), (2, 'Polaris');

-- 查询数据
SELECT * FROM my_iceberg_table;
-- 预期结果:
-- +------+---------+
-- | id   | name    |
-- +------+---------+
-- | 1    | Doris   |
-- | 2    | Polaris |
-- +------+---------+
```

如果上述操作均能成功，恭喜你！你已经成功打通了 Doris -> Polaris -> S3 的完整数据湖链路。

有关使用 Doris 管理 Iceberg 表的更多信息，请访问：

https://doris.apache.org/docs/lakehouse/catalogs/iceberg-catalog

2.14.13.10 集成 Apache Gravitino

随着数据湖技术的快速发展，如何构建统一、安全、高效的湖仓一体化数据架构成为企业数字化转型的核心挑战。Apache Gravitino 作为新一代统一元数据管理平台，为多云、多引擎环境下的数据治理提供了完整的解决方案。它不仅支持多种数据源和计算引擎的统一管理，还通过凭证管理机制（Credential Vending）确保了数据访问的安全性和可控性。

本文将深入介绍如何将 Apache Doris 与 Apache Gravitino 进行深度集成，构建基于 Iceberg REST Catalog 的现代化湖仓架构。通过 Gravitino 的统一元数据管理和动态凭证分发能力，实现对 S3 上 Iceberg 数据的高效、安全访问。

通过本文档，你将可以快速了解：

- AWS 环境准备：如何在 AWS 中创建 S3 存储桶和 IAM 角色，为 Gravitino 配置安全的凭证管理体系，实现临时凭证的动态分发机制。
- Gravitino 部署与配置：如何快速部署 Gravitino 服务，配置 Iceberg REST Catalog，并启用 vended-credentials 功能。
- Doris 连接 Gravitino：详细说明 Doris 如何通过 Gravitino 的 REST API 访问 Iceberg 数据。

2.14.13.10.1 Hands-on Guide

1. AWS 环境准备

在开始之前，我们需要在 AWS 上准备好完整的基础设施，包括 S3 存储桶和精心设计的 IAM 角色体系，这是构建安全可靠的湖仓架构的基石。

1.1 创建 S3 存储桶

首先创建一个专用的 S3 存储桶来存放 Iceberg 数据：

```
aws s3 mb s3://gravitino-iceberg-demo --region us-west-2
#### 验证存储桶创建成功
aws s3 ls | grep gravitino-iceberg-demo
```

1.2 设计 IAM 角色架构

为了实现安全的凭证管理，我们需要创建一个 IAM 角色供 Gravitino 通过 STS AssumeRole 机制使用。这个设计遵循了最小权限原则和职责分离的安全最佳实践。

创建数据访问角色

1. 创建信任策略文件

创建 gravitino-trust-policy.json 文件：

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": [
                    "arn:aws:iam::YOUR_ACCOUNT_ID:root"
                ]
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

2. 创建 IAM 角色

为简化演示，我们直接使用 AWS 管理策略。生产环境建议创建更精细的权限控制。

```
# 创建 IAM 角色
aws iam create-role \
    --role-name gravitino-iceberg-access \
    --assume-role-policy-document file://gravitino-trust-policy.json \
    --description "Gravitino Iceberg data access role"

# 附加 S3 完整访问权限（测试用，生产环境请使用精细化权限）
aws iam attach-role-policy \
    --role-name gravitino-iceberg-access \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
```

3. 验证 IAM 配置

   验证角色配置是否正确：

```
# 测试角色承担功能
aws sts assume-role \
    --role-arn arn:aws:iam::YOUR_ACCOUNT_ID:role/gravitino-iceberg-access \
    --role-session-name gravitino-test
```

成功响应示例：

```
{
    "Credentials": {
        "AccessKeyId": "ASIA***************",
        "SecretAccessKey": "****************************",
        "SessionToken": "IQoJb3JpZ2luX2VjEOj...",
        "Expiration": "2025-07-23T08:33:30+00:00"
    }
}
```

2.14.13.10.2 2. Gravitino 部署与配置

2.1 下载和安装 Gravitino

我们使用 Gravitino 的预编译版本来快速搭建环境：

```
#### 创建工作目录
mkdir gravitino-deployment && cd gravitino-deployment

#### 下载 Gravitino 主程序
wget https://github.com/apache/gravitino/releases/download/v0.9.1/gravitino-0.9.1-bin.tar.gz

#### 下载 Iceberg REST 服务器组件
wget https://github.com/apache/gravitino/releases/download/v0.9.1/gravitino-iceberg-rest-server
    ↪ -0.9.1-bin.tar.gz

#### 解压安装
tar -xzf gravitino-0.9.1-bin.tar.gz
cd gravitino-0.9.1-bin
tar -xzf ../gravitino-iceberg-rest-server-0.9.1-bin.tar.gz --strip-components=1
```

2.2 安装必要的依赖组件

为了支持 AWS S3 和凭证管理功能，需要安装额外的 JAR 包：

```
#### 创建必要的目录结构
mkdir -p catalogs/lakehouse-iceberg/libs
```

```
mkdir -p iceberg-rest-server/libs
mkdir -p logs
mkdir -p /tmp/gravitino

#### 下载 Iceberg AWS bundle
wget https://repo1.maven.org/maven2/org/apache/iceberg/iceberg-aws-bundle/1.6.1/iceberg-aws-
    ↪ bundle-1.6.1.jar \
  -P catalogs/lakehouse-iceberg/libs/

#### 下载 Gravitino AWS 支持包（vended-credentials 功能核心）
wget https://repo1.maven.org/maven2/org/apache/gravitino/gravitino-aws/0.9.1/gravitino-aws-0.9.1.
    ↪ jar \
  -P iceberg-rest-server/libs/

#### 分发 JAR 包到各个目录
cp catalogs/lakehouse-iceberg/libs/iceberg-aws-bundle-1.6.1.jar iceberg-rest-server/libs/
cp catalogs/lakehouse-iceberg/libs/iceberg-aws-bundle-1.6.1.jar libs/
cp iceberg-rest-server/libs/gravitino-aws-0.9.1.jar libs/
```

2.3 配置 Gravitino 服务

1. 主服务配置

   创建或编辑 conf/gravitino.conf 文件：

```
# Gravitino 服务器基础配置
gravitino.server.webserver.host = 0.0.0.0
gravitino.server.webserver.httpPort = 8090

# 元数据存储配置（生产环境建议使用 PostgreSQL/MySQL）
gravitino.entity.store = relational
gravitino.entity.store.relational = JDBCBackend
gravitino.entity.store.relational.jdbcUrl = jdbc:h2:file:/tmp/gravitino/gravitino.db;DB_CLOSE
    ↪ _DELAY=-1;MODE=MYSQL
gravitino.entity.store.relational.jdbcDriver = org.h2.Driver
gravitino.entity.store.relational.jdbcUser = gravitino
gravitino.entity.store.relational.jdbcPassword = gravitino

# 启用 Iceberg REST 服务
gravitino.auxService.names = iceberg-rest

# Iceberg REST 服务详细配置
gravitino.iceberg-rest.classpath = iceberg-rest-server/libs, iceberg-rest-server/conf
gravitino.iceberg-rest.host = 0.0.0.0
gravitino.iceberg-rest.httpPort = 9001
```

```
# Iceberg catalog 后端配置
gravitino.iceberg-rest.catalog-backend = jdbc
gravitino.iceberg-rest.uri = jdbc:h2:file:/tmp/gravitino/catalog_iceberg.db;DB_CLOSE_DELAY
    ↪ =-1;MODE=MYSQL
gravitino.iceberg-rest.jdbc-driver = org.h2.Driver
gravitino.iceberg-rest.jdbc-user = iceberg
gravitino.iceberg-rest.jdbc-password = iceberg123
gravitino.iceberg-rest.jdbc-initialize = true
gravitino.iceberg-rest.warehouse = s3://gravitino-iceberg-demo/warehouse
gravitino.iceberg-rest.io-impl = org.apache.iceberg.aws.s3.S3FileIO
gravitino.iceberg-rest.s3-region = us-west-2

# 启用 Vended-Credentials 功能
# 说明：Gravitino 使用这些 AK/SK 调用 STS AssumeRole，获取临时凭证分发给客户端
gravitino.iceberg-rest.credential-providers = s3-token
gravitino.iceberg-rest.s3-access-key-id = YOUR_AWS_ACCESS_KEY_ID
gravitino.iceberg-rest.s3-secret-access-key = YOUR_AWS_SECRET_ACCESS_KEY
gravitino.iceberg-rest.s3-role-arn = arn:aws:iam::YOUR_ACCOUNT_ID:role/gravitino-iceberg-
    ↪ access
gravitino.iceberg-rest.s3-region = us-west-2
gravitino.iceberg-rest.s3-token-expire-in-secs = 3600
```

2. 启动服务

```
# 启动 Gravitino 服务
./bin/gravitino.sh start

# 检查服务状态
./bin/gravitino.sh status

# 查看日志
tail -f logs/gravitino-server.log
```

3. 验证服务状态

```
# 验证主服务
curl -v http://localhost:8090/api/version

# 验证 Iceberg REST 服务
curl -v http://localhost:9001/iceberg/v1/config
```

2.4 创建 Gravitino 元数据结构

通过 REST API 创建必要的元数据结构：

```
#### 创建 MetaLake
curl -X POST -H "Accept: application/vnd.gravitino.v1+json" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "lakehouse",
    "comment": "Gravitino lakehouse for Doris integration",
    "properties": {}
  }' http://localhost:8090/api/metalakes

#### 创建 Iceberg Catalog
curl -X POST -H "Accept: application/vnd.gravitino.v1+json" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "iceberg_catalog",
    "type": "RELATIONAL",
    "provider": "lakehouse-iceberg",
    "comment": "Iceberg catalog with S3 storage and vended credentials",
    "properties": {
      "catalog-backend": "jdbc",
      "uri": "jdbc:h2:file:/tmp/gravitino/catalog_iceberg.db;DB_CLOSE_DELAY=-1;MODE=MYSQL",
      "jdbc-user": "iceberg",
      "jdbc-password": "iceberg123",
      "jdbc-driver": "org.h2.Driver",
      "jdbc-initialize": "true",
      "warehouse": "s3://gravitino-iceberg-demo/warehouse",
      "io-impl": "org.apache.iceberg.aws.s3.S3FileIO",
      "s3-region": "us-west-2"
    }
  }' http://localhost:8090/api/metalakes/lakehouse/catalogs
```

2.14.13.10.3 3. Doris 连接 Gravitino

3.1 使用 Vended Credentials

Gravitino 会动态生成临时凭证并分发给 Doris：

```
-- 创建动态凭证模式的 Catalog
CREATE CATALOG gravitino_vending PROPERTIES (
    'type' = 'iceberg',
    'warehouse' = 'warehouse',
    'iceberg.catalog.type' = 'rest',
    'iceberg.rest.uri' = 'http://127.0.0.1:9001/iceberg/',
    'iceberg.rest.vended-credentials-enabled' = 'true'
);
```

3.2 验证连接和数据操作

```
-- 验证连接
SHOW DATABASES FROM gravitino_vending;

-- 切换到 vended credentials catalog
SWITCH gravitino_vending;

-- 创建数据库和表
CREATE DATABASE demo;
USE gravitino_vending.demo;

CREATE TABLE gravitino_table (
    id INT,
    name STRING
)
PROPERTIES (
    'write-format' = 'parquet'
);

-- 插入测试数据
INSERT INTO gravitino_table VALUES (1, 'Doris'), (2, 'Gravitino');

-- 查询验证
SELECT * FROM gravitino_table;
```

2.14.13.10.4 总结

通过本指南，你应该能够成功构建一个基于 Gravitino 和 Doris 的现代化湖仓架构。这个架构不仅具备高性能和高可用性，还通过先进的安全机制确保了数据访问的安全性和合规性。随着数据规模的增长和业务需求的变化，这个架构可以灵活扩展以满足企业级的各种需求。

2.14.13.11 集成 Microsoft OneLake

OneLake 是 Microsoft Fabric 提供的统一且开放的 SaaS 数据湖，为企业提供一个逻辑上的集中数据存储位置。数据以 Parquet 格式存储，并可同时承载 Delta Lake 与 Iceberg 的元数据。这种设计使 OneLake 能在不进行数据复制或迁移的前提下，被多种分析引擎直接访问，从而简化数据管理与治理。

通过 Apache Doris 的 Iceberg Rest Catalog，可以直接访问 OneLake 中的数据，无需复制或迁移数据即可在 Doris 中执行查询与分析。借助这一能力，可以在单一数据湖中构建端到端的分析流程，实现高效的数据分析、共享和复用，发挥 OneLake 的统一存储优势与 Doris 的计算能力。

在技术层面，Doris 通过 OneLake 提供的开放表标准与底层 Parquet 数据文件，使用标准化接口读取元数据与数据文件，从而兼容多种分析场景。集成后的架构保持了数据湖的统一管理能力，使得治理、访问控制与安全策略能够集中应用，提升整个平台的可靠性与可维护性。

本文将介绍如何使用 Doris 访问 OneLake，并给出完整的环境准备与示例查询流程。

需要 Doris 3.1.4 版本。

2.14.13.11.1 Onelake 环境准备

下面先完成 OneLake（Fabric）侧的数据准备与认证配置，然后演示在 Doris 中创建 Iceberg Rest Catalog 并进行查询。

导入数据

1. 打开 Fabric 控制台，新建一个 WorkSpace（建议不要使用默认 WorkSpace，因为部分设置项可能受限）。

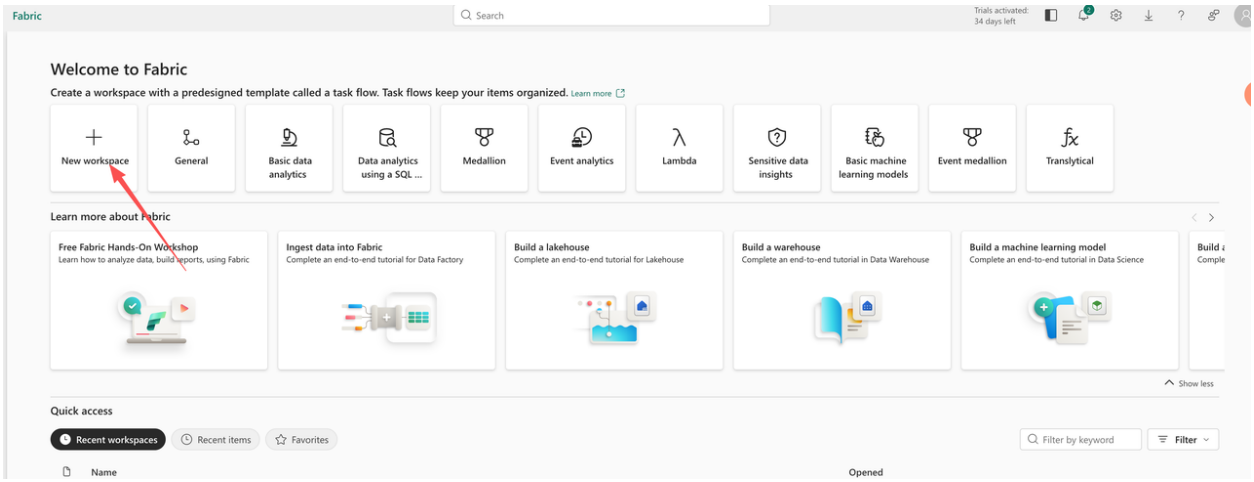


图 80: onelake1

2. 进入创建好的 WorkSpace，选择 New Item -> Lakehouse，创建一个 Lakehouse。

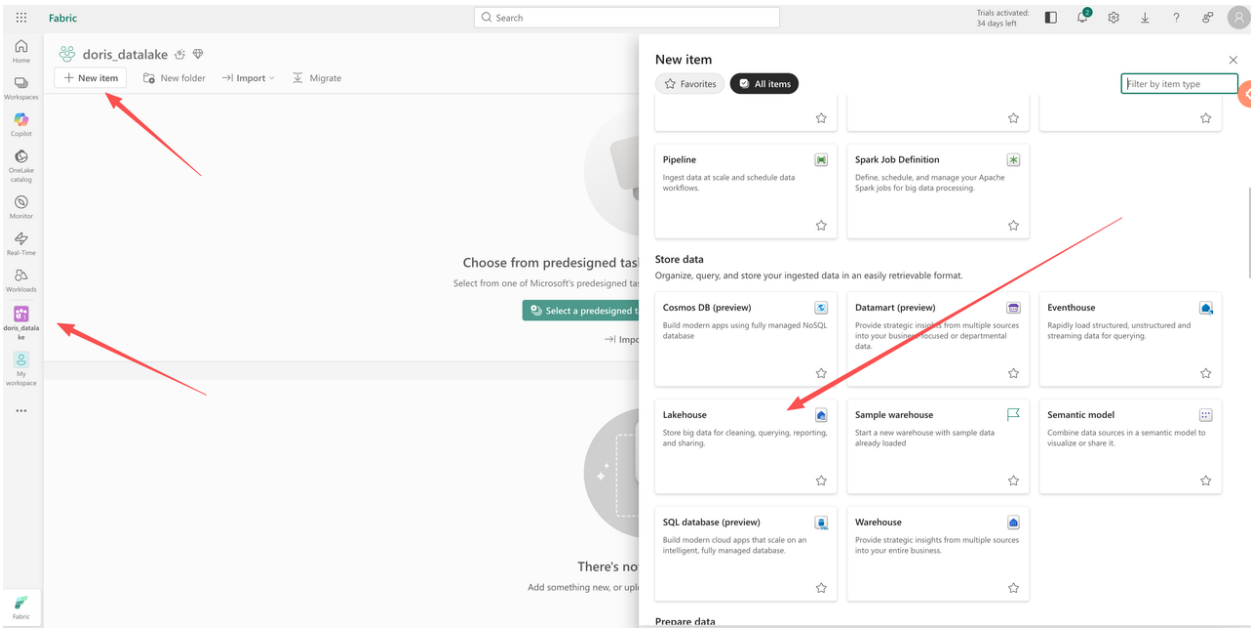


图 81: onelake2

3. 进入 WorkSpace Setting，打开页面中的相关开关（以便启用 Lakehouse 的一些功能）。

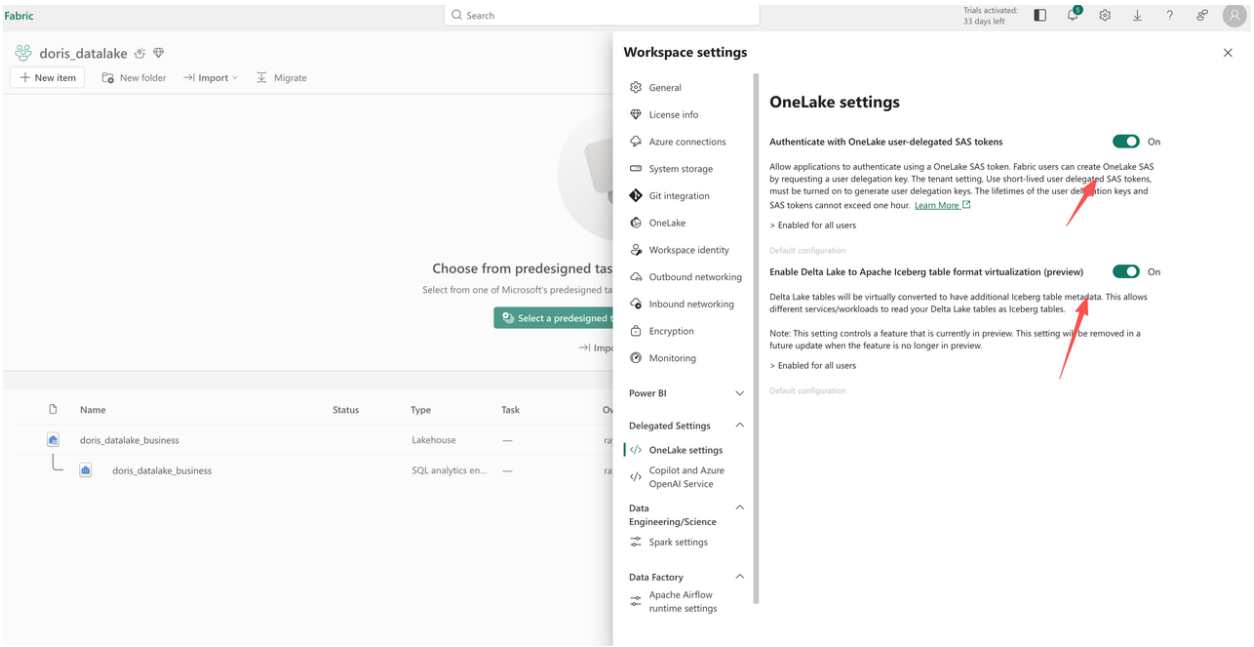


图 82: onelake3

将本地文件上传到 OneLake

为便于演示，此处采用本地 csv 文件直接上传，以下是示例文件：

1. 进入 WorkSpace 的 Files 页面，点击 Upload -> Upload Files，选择要上传的 CSV 文件。

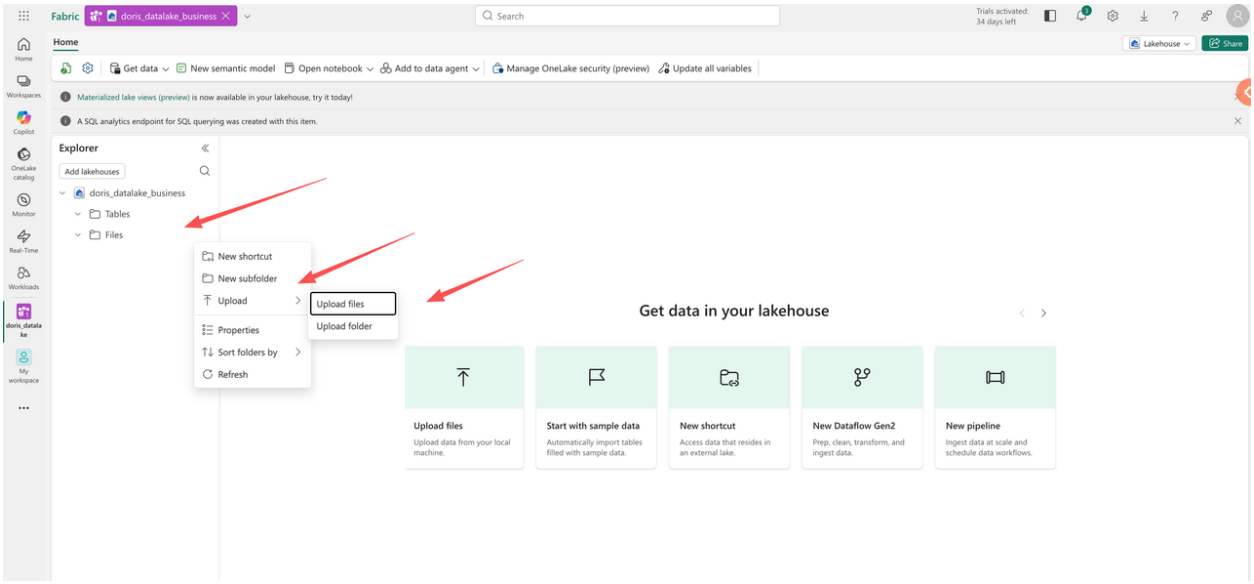


图 83: onelake4

2. 选择上传后的文件，点击 Load Tables -> New table（如果目标表已存在，也可以选择导入到现有表）。
3. 等待数据导入完成，进入 Tables 查看表及数据信息。

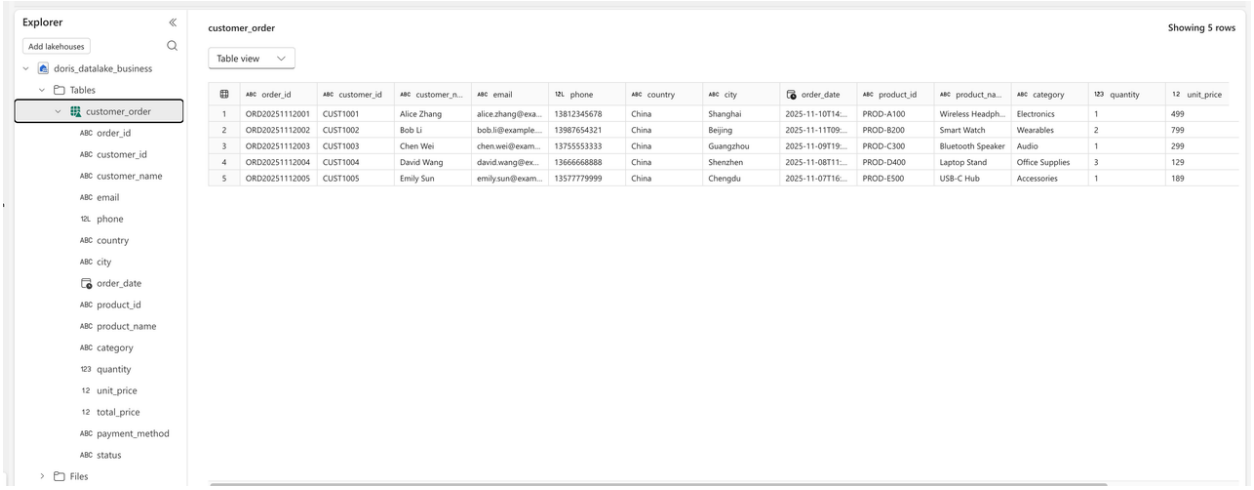


图 84: onelake5

认证信息配置

要让 Doris 通过 Iceberg Rest Catalog 访问 OneLake，需在 Azure 中为 Fabric 的访问配置应用注册与权限：

1. 打开 Azure 门户，进入 App registrations，点击 New registration，新建应用并记录以下信息以便后续配置：应用 ID、租户 ID 等。

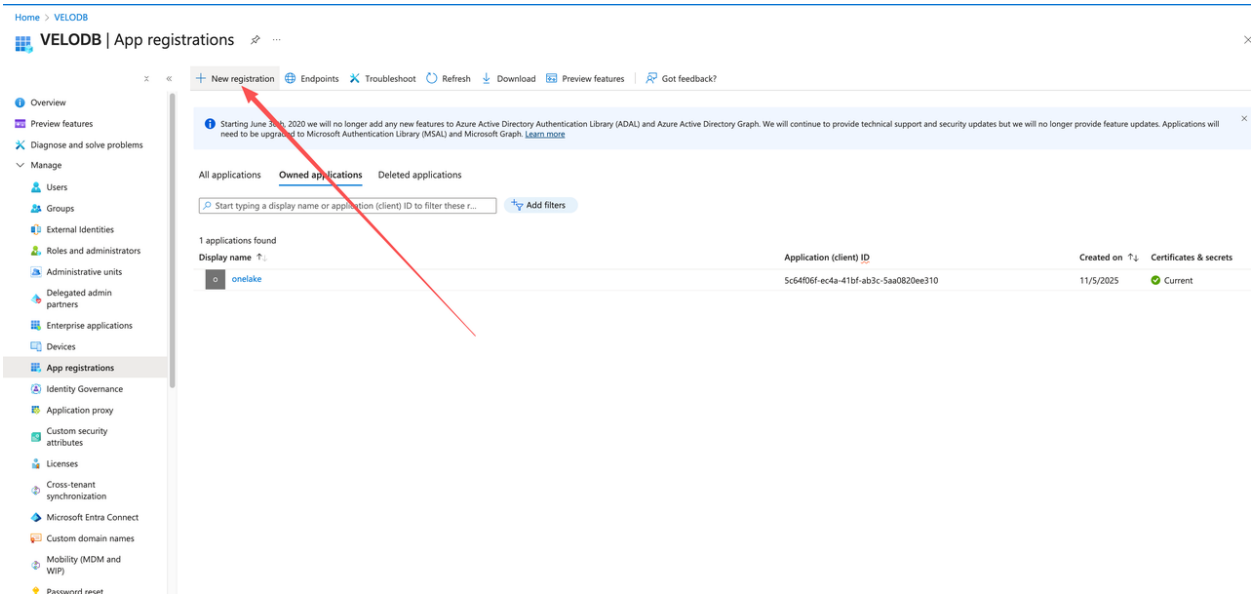


图 85: onelake6

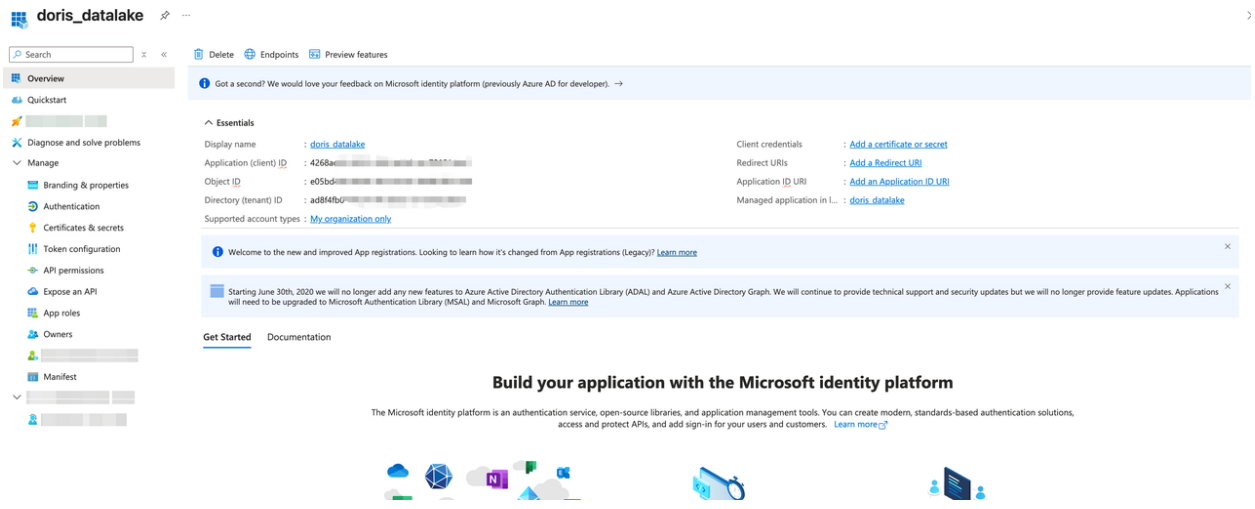


图 86: onelake7

2. 在新建应用的 API Permissions 中，添加对 Azure Storage 的相应权限（根据最小权限原则选择所需权限）。

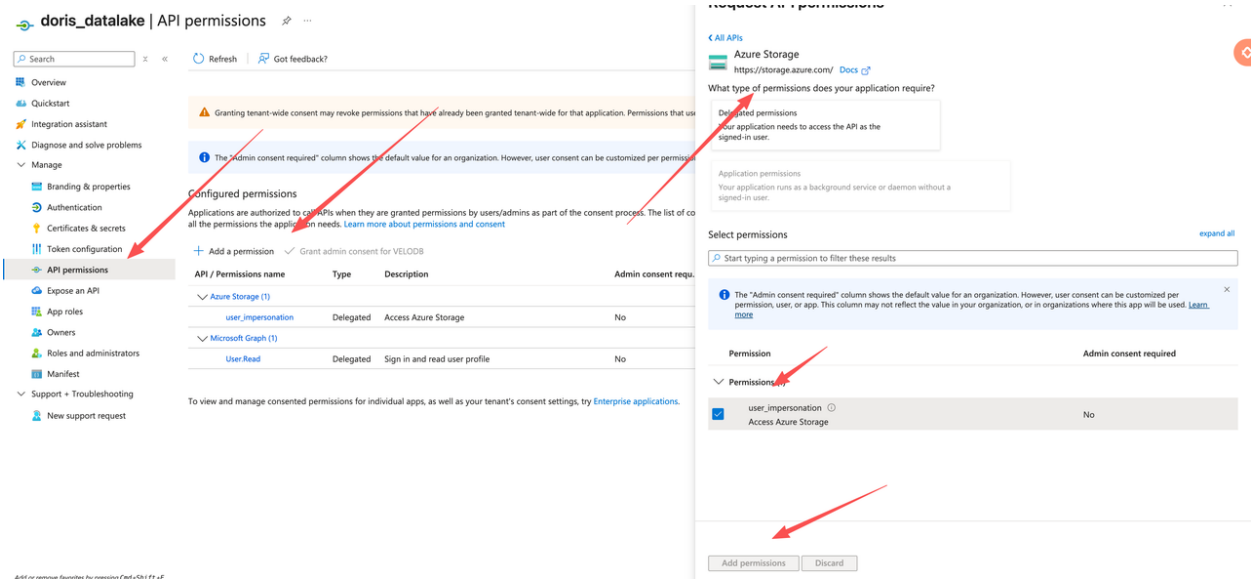


图 87: onelake8

3. 在 Certificates & secrets 中创建客户端密钥（Secret），并将生成的值妥善保存 ——此值离开页面后将无法再次查看。

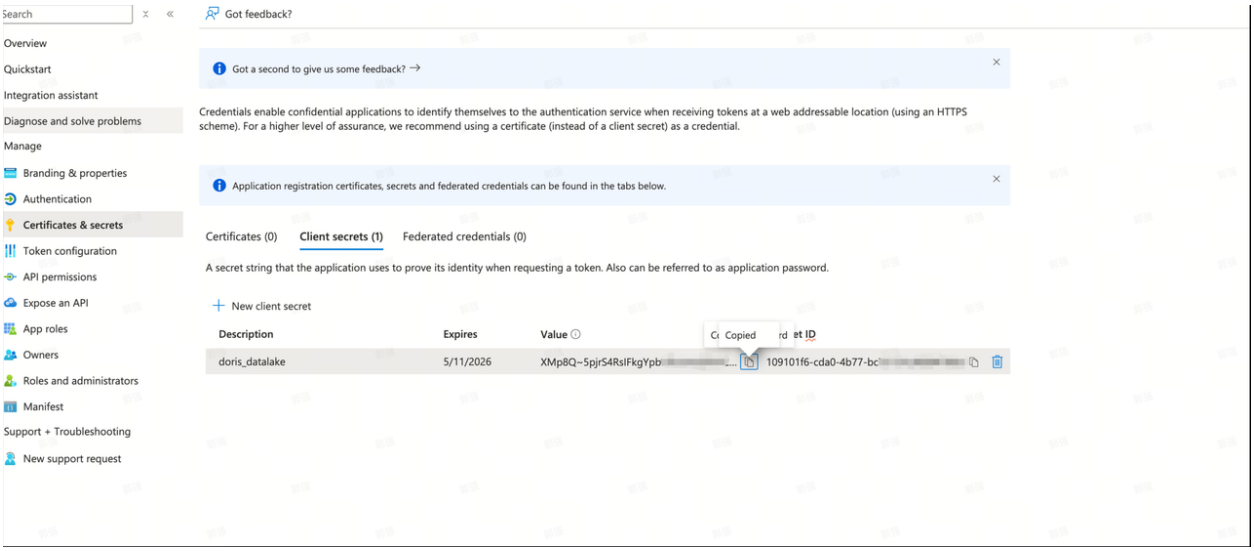


图 88: onelake9

4. 回到 Fabric 的 WorkSpace 界面，进入 Manage Access，使用应用的 DisplayName 添加该应用为 WorkSpace 的访问主体。

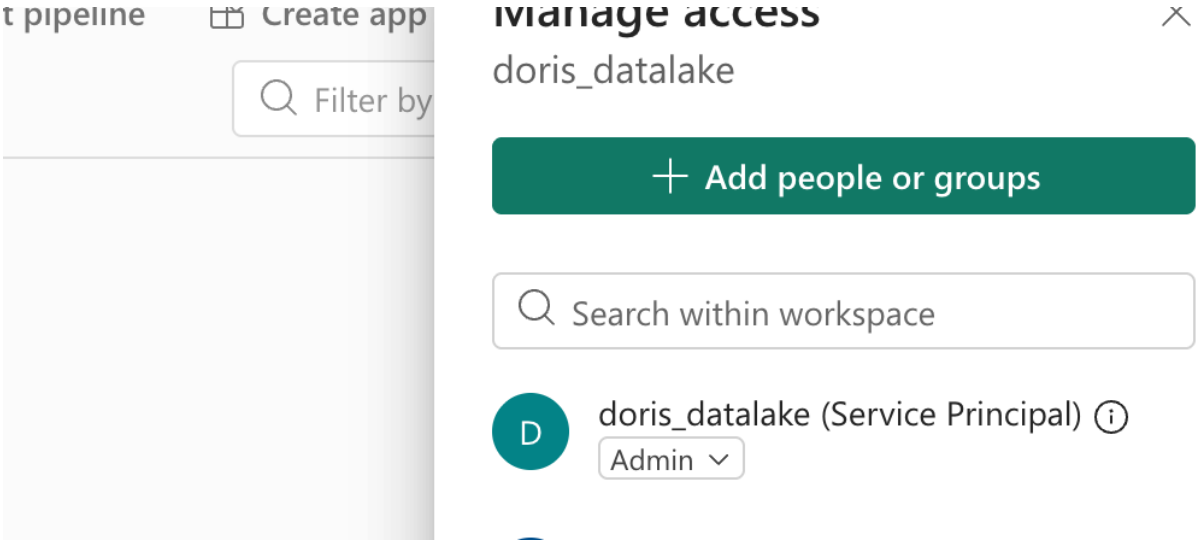


图 89: onelake10

至此，OneLake 侧的数据准备和认证配置完成。

2.14.13.11.2 在 Doris 中接入 OneLake

接下来示例如何在 Doris 中创建 Iceberg Rest Catalog 并访问 OneLake 中的表。

创建 Catalog

```
Doris> CREATE CATALOG onelake_doris PROPERTIES (
            'type' = 'iceberg',
            'iceberg.catalog.type' = 'rest',
            'uri'='https://onelake.table.fabric.microsoft.com/iceberg',
            'warehouse'='<workerspace_id>/<data_item_id>',
            'iceberg.rest.security.type'='oauth2',
            'iceberg.rest.oauth2.server-uri'='https://login.microsoftonline.com/<talent_id>/
                ↪ oauth2/v2.0/token',
            'iceberg.rest.oauth2.credential'='<oauth2.client_id>:'<oauth2.client_secret>,
            'iceberg.rest.oauth2.scope'='https://storage.azure.com/.default',
            'fs.azure.support'='true',
            'azure.endpoint'='https://onelake.dfs.fabric.microsoft.com',
            'azure.auth_type'='OAuth2',
            'azure.oauth2_account_host'='onelake.dfs.fabric.microsoft.com',
            'azure.oauth2_server_uri'='https://login.microsoftonline.com/<talent_id>/oauth2/v2.0/
                ↪ token',
            'azure.oauth2_client_id'='<oauth2.client_id>',
            'azure.oauth2_client_secret'='<oauth2.client_secret>'
        );
```

- 创建 Iceberg Rest Catalog 时需要填写的参数中，`WORKSPACE_ID` 与 `DATA_ITEM_ID` 可在 Lakehouse 页面对应链接中获取，格式类似：

```
https://app.fabric.microsoft.com/groups/<WORKSPACE_ID>/lakehouses/<DATA_ITEM_ID>
```

- 其余参数可对应之前在 Azure App registration 中获得的信息（如 client id、client secret、tenant 等）。
- 对于 OneLake，`iceberg.rest.oauth2.scope`、`uri`、`azure.oauth2_account_host`、`azure.endpoint` 等通常为固定值，请参考官方文档或示例配置填写。

配置完成后即可在 Doris 中使用 SQL 对 OneLake 中的 Iceberg 表进行查询。

基础查询分析

下面给出若干常见的业务分析示例，帮助理解如何结合 OneLake 与 Doris 进行数据分析：

```
Doris> switch onelake_doris;
Query OK, 0 rows affected

Doris> use dbo;
Database changed

Doris> show tables;
+----------------+
| Tables_in_dbo  |
+----------------+
| customer_order |
+----------------+
1 row in set
```

- 查询最近三天的新订单

便于运营或销售快速掌握近期销售情况，例如发现新下单、待处理或已完成支付的订单，便于安排发货或优先处理待处理订单。

```
Doris> SELECT order_id, customer_name, product_name, total_price, status
    -> FROM customer_order
    -> WHERE order_date >= DATE_SUB(NOW(), INTERVAL 3 DAY)
    -> ORDER BY order_date DESC;

+----------------+---------------+---------------------+-------------+-----------+
| order_id       | customer_name | product_name        | total_price | status    |
+----------------+---------------+---------------------+-------------+-----------+
| ORD20251112002 | Bob Li        | Smart Watch         |        1128 | Completed |
| ORD20251112001 | Alice Zhang   | Wireless Headphones |         499 | Completed |
| ORD20251112003 | Chen Wei      | Bluetooth Speaker   |         299 | Pending   |
+----------------+---------------+---------------------+-------------+-----------+
3 rows in set
```

- 按城市统计销售表现

统计各城市的总销售额和订单数，帮助销售/市场判断区域表现并调整营销或库存策略。

```
Doris> SELECT city,
    ->        SUM(total_price) AS total_sales,
    ->        COUNT(*) AS order_count
    -> FROM customer_order
    -> WHERE status = 'Completed'
    -> GROUP BY city
    -> ORDER BY total_sales DESC;

+----------+-------------+-------------+
| city     | total_sales | order_count |
+----------+-------------+-------------+
| Beijing  |        1128 |           1 |
| Shanghai |         499 |           1 |
| Shenzhen |         387 |           1 |
+----------+-------------+-------------+
3 rows in set
```

- 分析不同支付方式的退款率

帮助财务与风控发现潜在问题（例如某支付渠道退款率异常），进而优化支付流程或对特定渠道加强监控。

```
Doris> SELECT payment_method,
    ->        SUM(CASE WHEN status = 'Refunded' THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS refund
        ↪ _rate_percent
    -> FROM customer_order
    -> GROUP BY payment_method
    -> ORDER BY refund_rate_percent DESC;

+----------------+----------------------+
| payment_method | refund_rate_percent |
+----------------+----------------------+
| Credit Card    |            50.00000 |
| WeChat Pay     |             0.00000 |
| Alipay         |             0.00000 |
| UnionPay       |             0.00000 |
+----------------+----------------------+
4 rows in set
```

- 跨系统用户行为对比

若系统同时保留了旧系统（例如 Hive）与 OneLake 上的新系统数据，可以查询两个系统中都有行为记录的用户，并分析他们在新系统的消费行为，从而评估迁移与业务影响。

```
-- "hive_catalog" is a Hive Catalog created in Doris
Doris> SELECT
    ->     a.customer_id,
    ->     COUNT(DISTINCT b.order_id) AS new_order_count,
    ->     SUM(b.total_price) AS new_total_amount
    -> FROM hive_catalog.order_db.hive_customer_order a
    -> JOIN onelake_doris.dbo.customer_order b
    ->   ON a.customer_id = b.customer_id
    -> GROUP BY a.customer_id
    -> ORDER BY new_total_amount DESC
    -> LIMIT 100;

+-------------+-----------------+------------------+
| customer_id | new_order_count | new_total_amount |
+-------------+-----------------+------------------+
| CUST1002    |               1 |             1128 |
| CUST1001    |               1 |              499 |
| CUST1004    |               1 |              387 |
| CUST1003    |               1 |              299 |
| CUST1005    |               1 |              189 |
+-------------+-----------------+------------------+
5 rows in set
```

快照和时间旅行

```
Doris> select * from customer_order$snapshots;
+--
    ↪ -------------------------+--------------------+-----------+-----------+-------------------------------
    ↪
| committed_at               | snapshot_id         | parent_id | operation | manifest_list
    ↪
    ↪ | summary
    ↪
    ↪ |
+--
    ↪ -------------------------+--------------------+-----------+-----------+-------------------------------
    ↪
| 2025-11-12 17:21:06.692000 | 7623467350518045470 |      NULL | overwrite | abfss://181a804a-
    ↪ ea52-4579-81a4-4de243e14c8e@onelake.dfs.fabric.microsoft.com/ad29a0e3-772f-458c-9dfa-4
    ↪ ff609443c13/Tables/customer_order/metadata/snap-7623467350518045470-1-0e16b9d6-bc0d
    ↪ -4689-9952-085abd1b5f4e.avro | {"XTABLE_METADATA":"{"lastInstantSynced":"2025-11-12T09
    ↪ :02:06Z","instantsToConsiderForNextSync":[],"version":0,"sourceTableFormat":"DELTA","
    ↪ sourceIdentifier":"1"}", "added-data-files":"1", "added-records":"5", "added-files-size":
    ↪ "9434", "changed-partition-count":"1", "total-records":"5", "total-files-size":"9434", "
    ↪ total-data-files":"1", "total-delete-files":"0", "total-position-deletes":"0", "total-
```

```
    ↪ equality-deletes":"0"} |
+--
    ↪ -------------------------+--------------------+----------+----------+-----------------------------
    ↪
1 row in set

Doris> SELECT * FROM customer_order FOR VERSION AS OF 7623467350518045470;
+--
    ↪ -------------+------------+--------------+-------------------------+------------+---------+---------
    ↪
| order_id       | customer_id | customer_name | email                     | phone       | country
    ↪ | city      | order_date                 | product_id | product_name        | category
    ↪        | quantity | unit_price | total_price | payment_method | status    |
+--
    ↪ -------------+------------+--------------+-------------------------+------------+---------+---------
    ↪
| ORD20251112001 | CUST1001    | Alice Zhang   | alice.zhang@example.com | 13812345678 | China
    ↪ | Shanghai  | 2025-11-10 22:23:00.000000 | PROD-A100  | Wireless Headphones | Electronics
    ↪      |        1 |        499 |         499 | Credit Card    | Completed |
| ORD20251112002 | CUST1002    | Bob Li        | bob.li@example.com      | 13987654321 | China
    ↪ | Beijing   | 2025-11-11 17:12:00.000000 | PROD-B200  | Smart Watch         | Wearables
    ↪       |        2 |        799 |        1128 | WeChat Pay     | Completed |
| ORD20251112003 | CUST1003    | Chen Wei      | chen.wei@example.com    | 13755553333 | China
    ↪ | Guangzhou | 2025-11-10 03:40:00.000000 | PROD-C300  | Bluetooth Speaker   | Audio
    ↪           |        1 |         19 |         299 | Alipay         | Pending   |
| ORD20251112004 | CUST1004    | David Wang    | david.wang@example.com  | 13666668888 | China
    ↪ | Shenzhen  | 2025-11-08 19:15:00.000000 | PROD-D400  | Laptop Stand        | Office
    ↪ Supplies |        3 |        129 |         387 | UnionPay       | Completed |
| ORD20251112005 | CUST1005    | Emily Sun     | emily.sun@example.com   | 13577779999 | China
    ↪ | Chengdu   | 2025-11-08 00:45:00.000000 | PROD-E500  | USB-C Hub           | Accessories
    ↪      |        1 |        189 |         189 | Credit Card    | Refunded  |
+--
    ↪ -------------+------------+--------------+-------------------------+------------+---------+---------
    ↪
5 rows in set
```

2.14.13.11.3 总结

通过本文的介绍，我们展示了如何将 Apache Doris 与 Microsoft OneLake 无缝集成，实现统一数据湖架构下的高效分析。这种集成方案具有以下核心优势：

- 零数据迁移：直接访问 OneLake 中的数据，无需复制或移动数据
- 统一管理：保持数据湖的集中治理和安全策略
- 标准兼容：基于 Iceberg 开放表格式，确保跨平台互操作性

- 灵活分析：结合 Doris 强大的 OLAP 能力和 OneLake 的存储优势

无论是实时业务监控、跨系统数据对比，还是复杂的多维分析，这种架构都能为企业提供强大且灵活的数据分析能力。随着数据湖技术的不断发展，Doris 与 OneLake 的集成将为企业数字化转型提供更加坚实的数据基础设施支撑。

下一步，您可以根据实际业务需求，参考本文的配置步骤搭建自己的分析环境，并探索更多高级分析场景。

2.14.13.12 集成 Databricks Unity Catalog

随着企业在 Lakehouse 架构下统一管理不断增长的数据资产，对跨平台、高性能、受治理的数据访问能力的需求愈加迫切。Apache Doris 作为新一代实时分析型数据库，现已实现与 Databricks Unity Catalog 的深度集成，使企业能够在统一的治理体系下，通过 Doris 直接访问并高效查询 Databricks 管理的数据湖，实现数据的无缝衔接。

通过本文档，您将深入了解：

- Databricks 环境准备：如何在 Databricks 中创建 External Location、Catalog 和 Iceberg 表，以及相关的权限配置
- Doris 连接 Unity Catalog：如何通过 Doris 连接 Databricks Unity Catalog 并访问 Iceberg 表

> 注意：本功能需要 Doris 3.1.3 及以上版本。

2.14.13.12.1 Databricks 环境准备

创建 External Location

在 Unity Catalog 中，External Location 是一个用于将云对象存储中的路径与存储凭据（Storage Credential）关联的安全对象。External Location 支持外部访问，Unity Catalog 可以通过 Credential Vending 功能为外部系统发放短期凭证，允许外部系统访问这些路径。

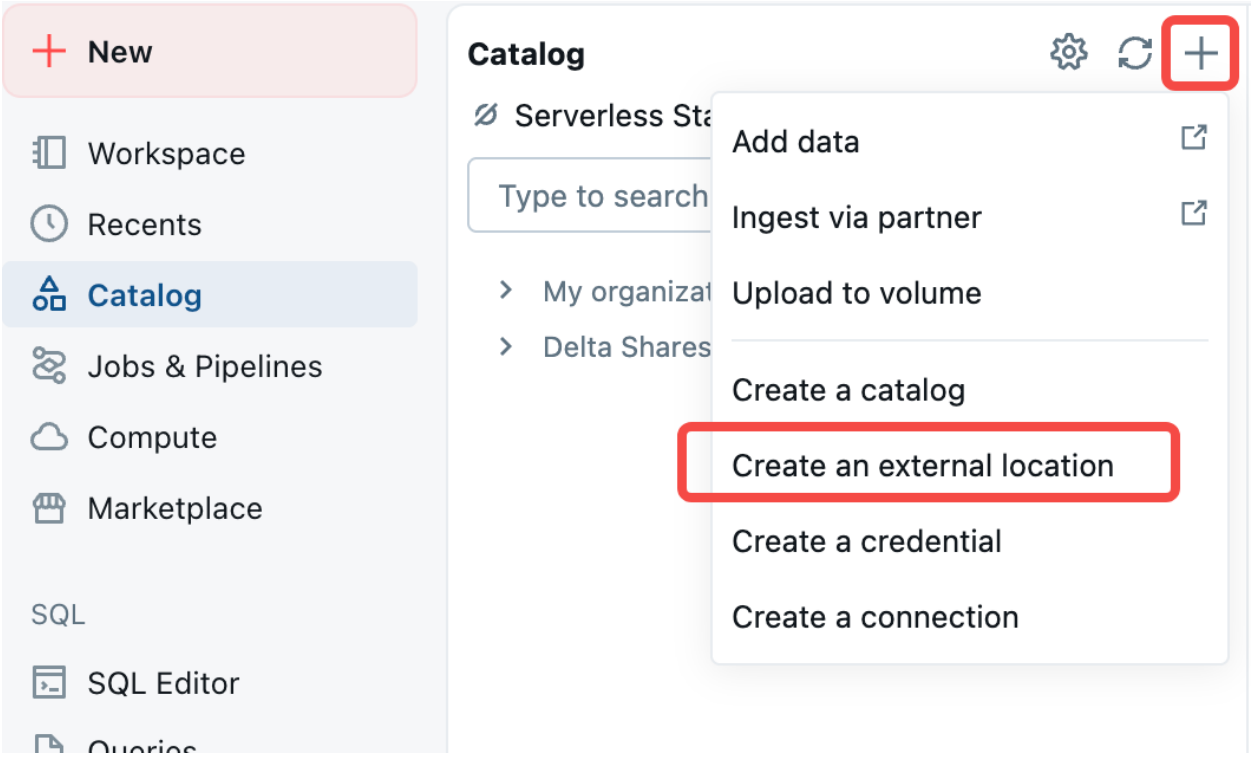


图 90: unity1

本文使用 AWS Quickstart 在 AWS S3 中创建 External Location。

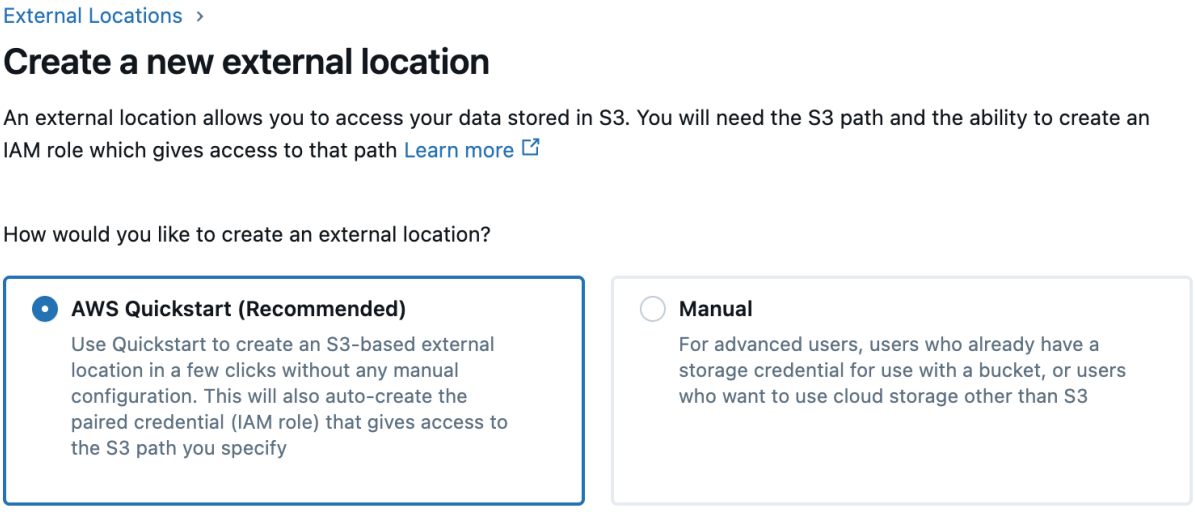


图 91: unity2

创建完成后，可以看到 External Catalog，以及对应的 Credential：

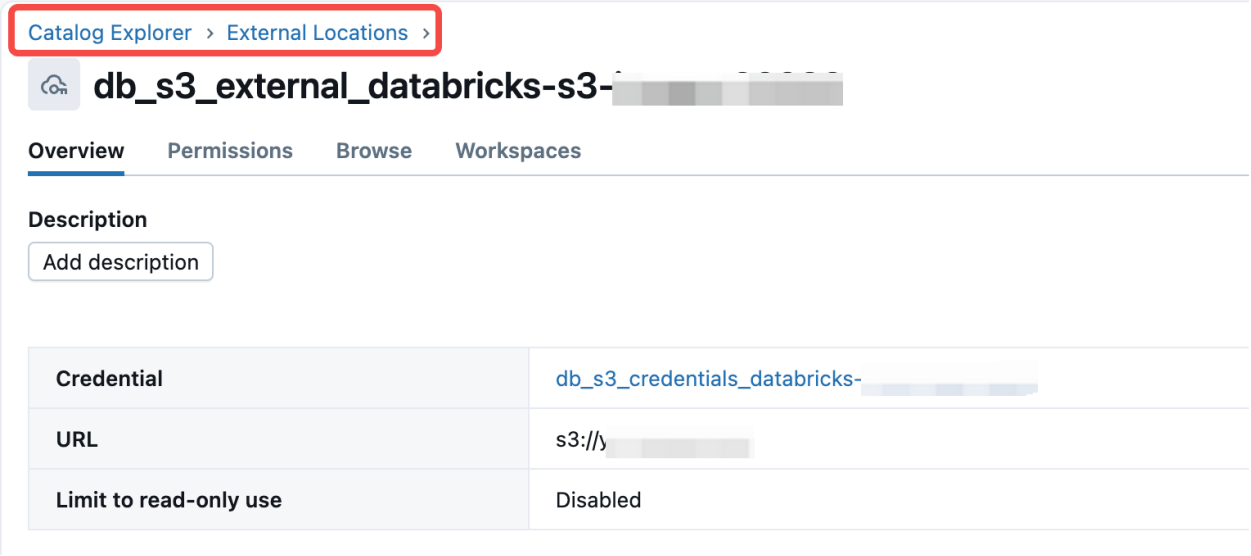


图 92: unity3

创建 Catalog

点击界面中的创建 Catalog 选项。

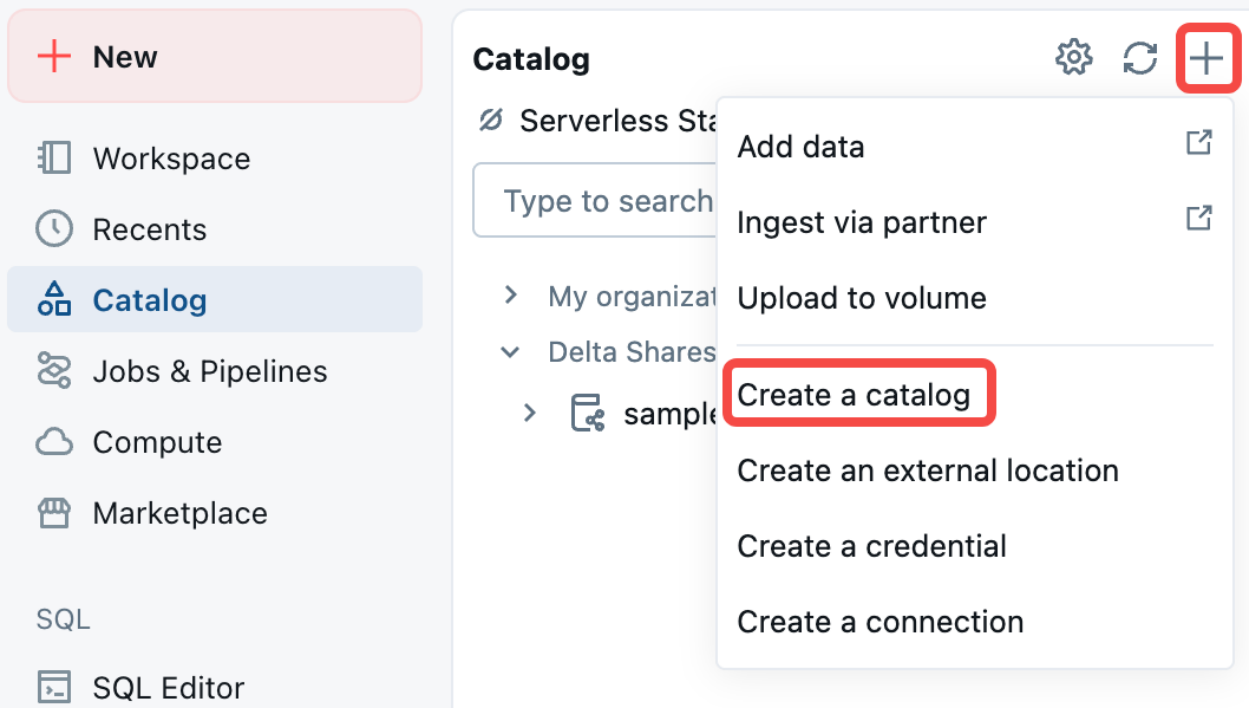


图 93: unity4

填写 Catalog 的名称。取消 `Use default storage` 的勾选，并选择刚才创建的 External Location。

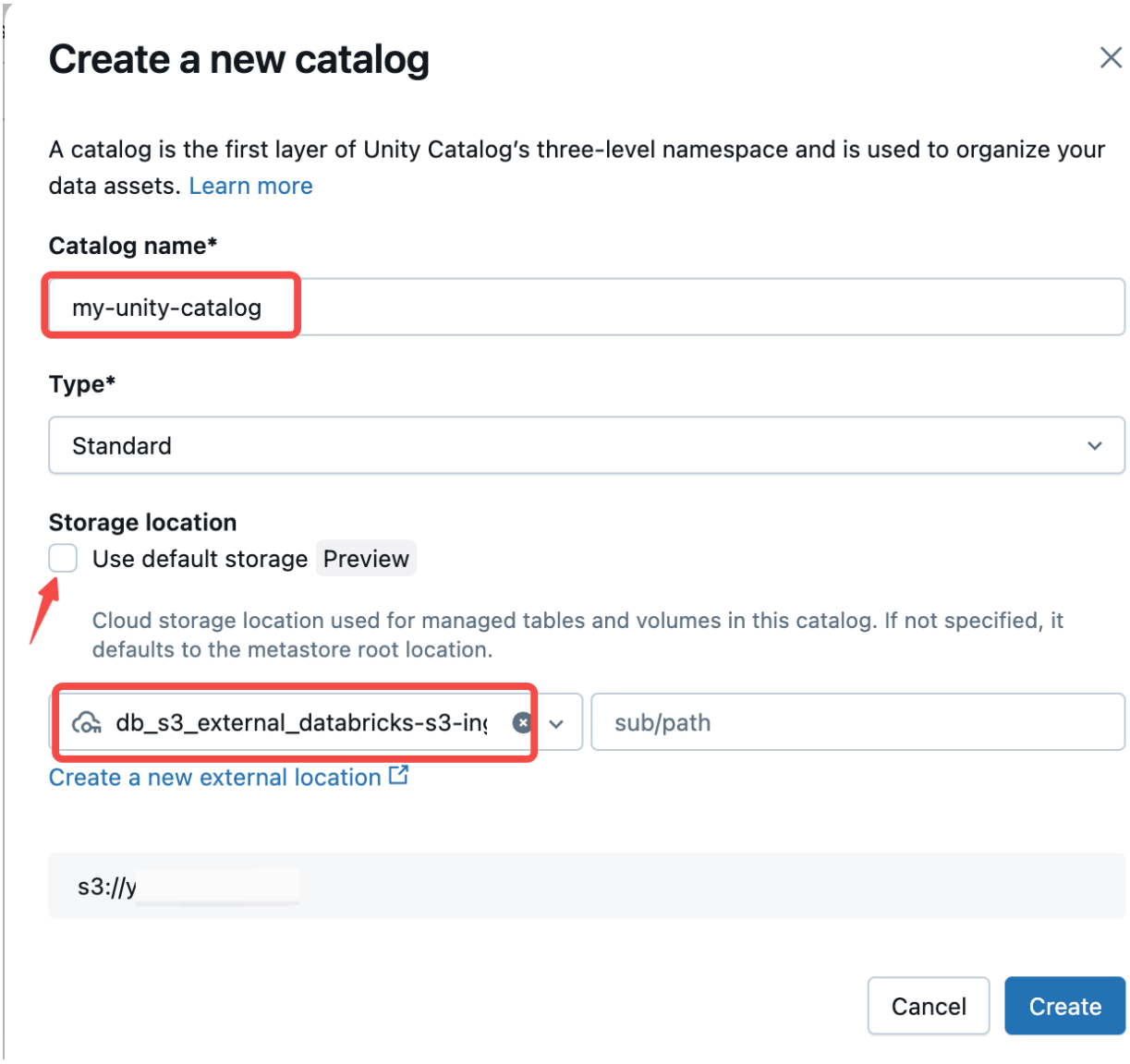


图 94: unity5

开启 External Use Schema 权限

点击刚才创建的 `Catalog` → `Permissions` → `Grant`：

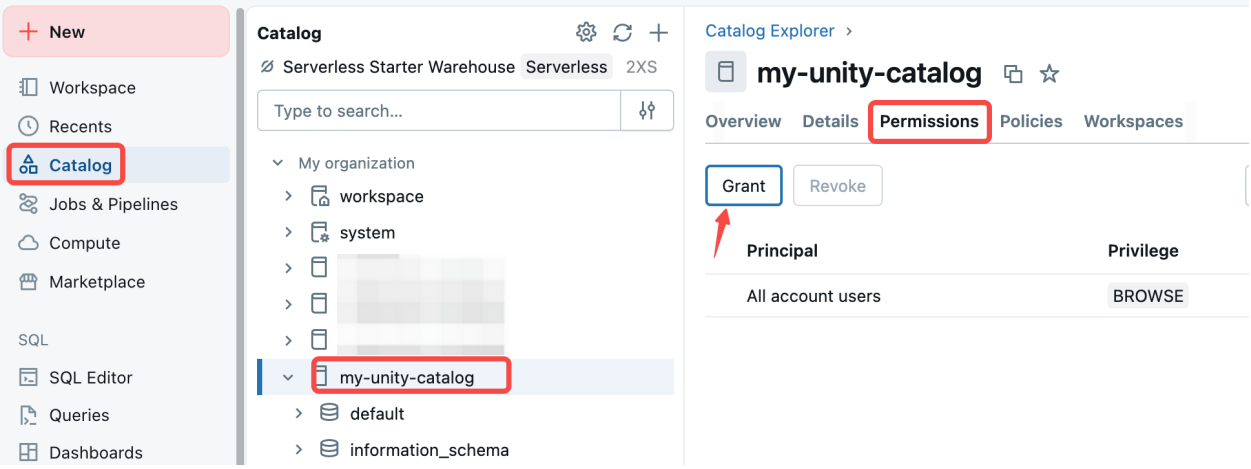


图 95: unity6

选择 `All account users`，并勾选 `EXTERNAL USE SCHEMA` 选项。

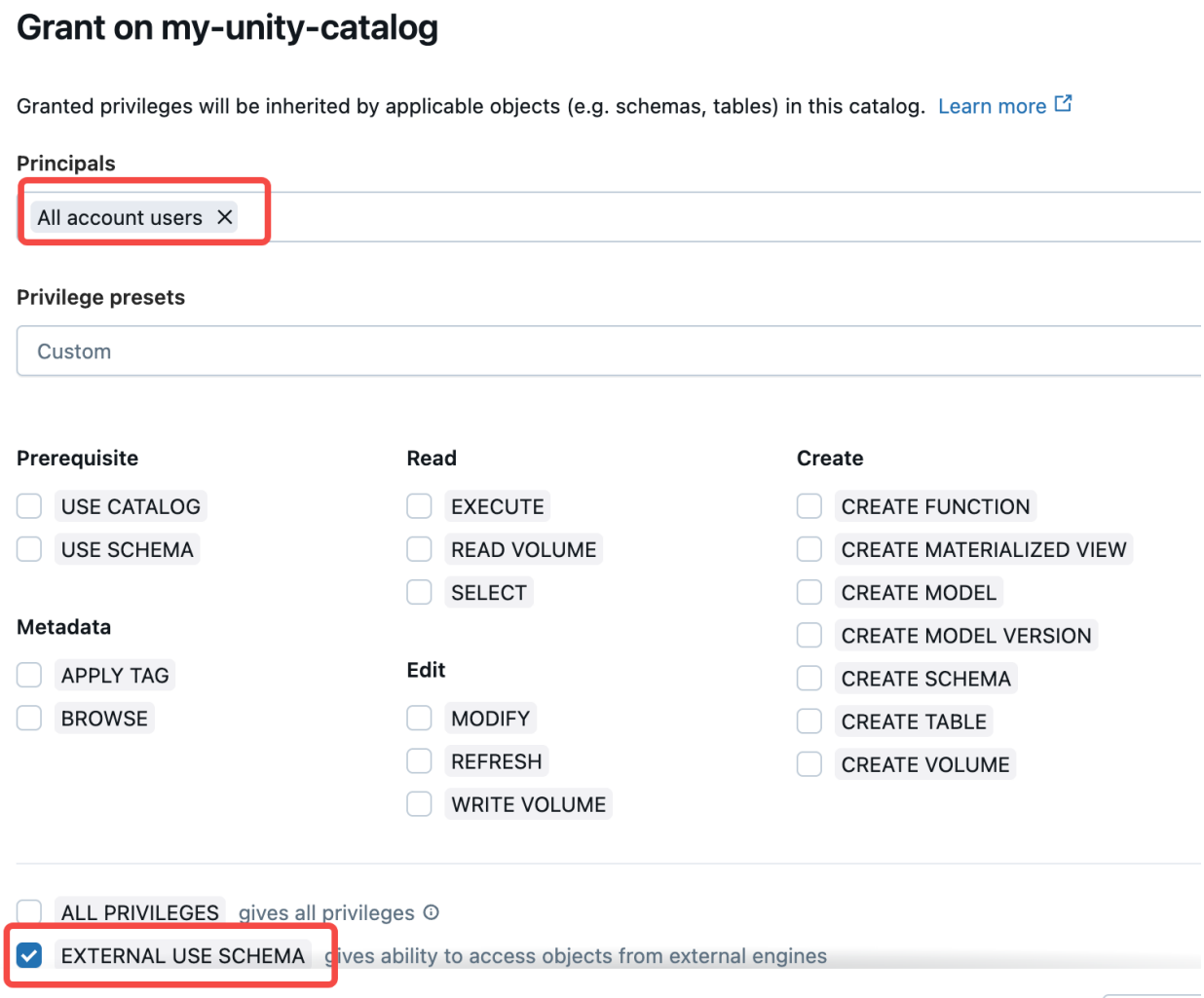


图 96: unity7

创建 Iceberg 表并写入数据

在 Databricks 的 SQL Editor 中执行以下 SQL 创建 Iceberg 表并写入数据：

```
CREATE TABLE `my-unity-catalog`.default.iceberg_table (
  id int,
  name string
) USING iceberg;

INSERT INTO `my-unity-catalog`.default.iceberg_table VALUES(1, "jack");
```

获取 Access Token

点击右上角用户头像，进入 Settings 页面，在 User → Developer 中选择 Access tokens。创建一个新的 Token，供后续 Doris 连接 Unity Catalog 时使用。Token 是一个字符串，形如：`dapi4f...`

2.14.13.12.2 Doris 连接 Unity Catalog

创建 Catalog

```
-- Use oath2 credential and vended credentials
CREATE CATALOG dbx_unity_catalog PROPERTIES (
  "uri" = "https://dbc-xx.cloud.databricks.com:443/api/2.1/unity-catalog/iceberg-rest/",
  "type" = "iceberg",
  "warehouse" = "my-unity-catalog",
  "iceberg.catalog.type" = "rest",
  "iceberg.rest.security.type" = "oauth2",
  "iceberg.rest.oauth2.credential" = "clientid:clientsecret",
  "iceberg.rest.oauth2.server-uri" = "https://dbc-xx.cloud.databricks.com:443/oidc/v1/token",
  "iceberg.rest.oauth2.scope" = "all-apis",
  "iceberg.rest.vended-credentials-enabled" = "true"
);

-- Use PAT and vended credentials
CREATE CATALOG dbx_unity_catalog PROPERTIES (
  "uri" = "https://<dbc-account>.cloud.databricks.com/api/2.1/unity-catalog/iceberg-rest/",
  "type" = "iceberg",
  "warehouse" = "my-unity-catalog",
  "iceberg.catalog.type" = "rest",
  "iceberg.rest.security.type" = "oauth2",
  "iceberg.rest.oauth2.token" = "<token>",
  "iceberg.rest.vended-credentials-enabled" = "true"
);

-- Use oath2 credential and static ak/sk for accessing aws s3
CREATE CATALOG dbx_unity_catalog PROPERTIES (
  "uri" = "https://dbc-xx.cloud.databricks.com:443/api/2.1/unity-catalog/iceberg-rest/",
  "type" = "iceberg",
  "warehouse" = "my-unity-catalog",
  "iceberg.catalog.type" = "rest",
  "iceberg.rest.security.type" = "oauth2",
  "iceberg.rest.oauth2.credential" = "clientid:clientsecret",
  "iceberg.rest.oauth2.server-uri" = "https://dbc-xx.cloud.databricks.com:443/oidc/v1/token",
  "iceberg.rest.oauth2.scope" = "all-apis",
  "s3.endpoint" = "https://s3.<region>.amazonaws.com",
  "s3.access_key" = "<ak>",
  "s3.secret_key" = "<sk>",
  "s3.region" = "<region>"
);
```

访问 Catalog

创建完成后，即可开始访问 Unity Catalog 中存储的 Iceberg 表：

```
mysql> USE dbx_unity_catalog.`default`;
Database changed

mysql> SELECT * FROM iceberg_table;
+------+------+
| id   | name |
+------+------+
|    1 | jack |
+------+------+
1 row in set (3.32 sec)
```

管理 Iceberg 表

同时，也可以直接通过 Doris 在 Unity Catalog 中创建、管理和写入 Iceberg 表：

```
-- 写入 Unity Catalog 已存在的表
INSERT INTO iceberg_table VALUES(2, "mary");

-- 创建一张分区表
CREATE TABLE partition_table (
  `ts` DATETIME COMMENT 'ts',
  `col1` BOOLEAN COMMENT 'col1',
  `pt1` STRING COMMENT 'pt1',
  `pt2` STRING COMMENT 'pt2'
)
PARTITION BY LIST (day(ts), pt1, pt2) ();

-- 插入数据
INSERT INTO partition_table VALUES("2025-11-12", true, "foo", "bar");

-- 查看表分区信息
SELECT * FROM partition_table$partitions`\G`
*************************** 1. row ***************************
                   partition: {"ts_day":"2025-11-12", "pt1":"foo", "pt2":"bar"}
                     spec_id: 0
                record_count: 1
                  file_count: 1
total_data_file_size_in_bytes: 2552
 position_delete_record_count: 0
   position_delete_file_count: 0
 equality_delete_record_count: 0
   equality_delete_file_count: 0
              last_updated_at: 2025-11-18 15:20:45.964000
     last_updated_snapshot_id: 9024874735105617773
```

2.14.13.12.3 总结

通过与 Databricks Unity Catalog 的深度集成，Apache Doris 使企业能够在统一治理框架下，以更高性能、更低成本的方式访问和分析数据湖中的核心资产。这一能力不仅增强了 Lakehouse 架构的整体一致性，也为实时分析、交互式查询以及 AI 场景带来了新的可能性。无论是数据团队、分析工程师还是平台架构师，都可以借助 Doris 在现有数据湖基础上构建更敏捷、更智能的数据应用。

2.14.13.13 在 Hive/Iceberg 上构建 TPC-H 数据集

Doris 支持通过 Trino Connector 兼容框架，使用 TPCH Connector 来快速构建 TPCH 测试集。

结合 Hive/Iceberg 表的数据写回功能，您可以快速通过 Doris 构建 Doris、 Hive、 Iceberg 表的 TPCH 测试数据集。

本文档主要介绍如何部署和使用 TPCH Connector 构建测试数据集。

> 该功能自 Doris 3.0.0 版本开始支持。

2.14.13.13.1 编译 TPCH Connector

> 需要 JDK 17 版本。

```
git clone https://github.com/trinodb/trino.git
git checkout 435
cd trino/plugin/trino-tpch
mvn clean install -DskipTest
```

完成编译后，会在 `trino/plugin/trino-tpch/target/` 下得到 `trino-tpch-435/` 目录。

也可以直接下载预编译的 trino-tpch-435.tar.gz 并解压。

2.14.13.13.2 部署 TPCH Connector

将 `trino-tpch-435/` 目录放到所有 FE 和 BE 部署路径的 `connectors/` 目录下。(如果没有，可以手动创建)。

```
├── bin
├── conf
├── connectors
|   ├── trino-tpch-435
...
```

部署完成后，建议重启 FE、 BE 节点以确保 Connector 可以被正确加载。

2.14.13.13.3 创建 TPCH Catalog

```
CREATE CATALOG `tpch` PROPERTIES (
    "type" = "trino-connector",
    "trino.connector.name" = "tpch",
    "trino.tpch.column-naming" = "STANDARD",
    "trino.tpch.splits-per-node" = "32"
);
```

其中 tpch.splits-per-node 为并发数，建议设置为 BE 单机核数的 2 倍，可以获得最优的并发度。提升数据生成效率。

"tpch.column-naming" = "STANDARD" 时，TPCH 表中的列名，都会以表名缩写开头，比如 l_orderkey，否则，是 orderkey。

2.14.13.13.4 使用 TPCH Catalog

TPCH Catalog 中预制了不同 Scale Factor 的 TPCH 数据集，可以通过 SHOW DATABASES 和 SHOW TABLES 命令查看。

```
mysql> SWITCH tpch;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| sf1                |
| sf100              |
| sf1000             |
| sf10000            |
| sf100000           |
| sf300              |
| sf3000             |
| sf30000            |
| tiny               |
+--------------------+

mysql> USE sf1;
mysql> SHOW TABLES;
+---------------+
| Tables_in_sf1 |
+---------------+
| customer      |
| lineitem      |
| nation        |
```

```
| orders        |
| part          |
| partsupp      |
| region        |
| supplier      |
+---------------+
```

通过 SELECT 语句可以直接查询这些表。

> 这些预制数据集的数据，并没有实际存储，而是在查询时实时生成的。所以这些预制数据集不适合用来直接进行 Benchmark 测试。适用于通过 INSERT INTO SELECT 将数据集写入到其他目的表（如 Doris 内表、Hive、Iceberg 等所有 Doris 支持写入的数据源）后，对目的表进行性能测试。

2.14.13.13.5 构建 TPCH 测试数据集

以下示例通过 CTAS 语句快速构建一个 Hive 上的 TPCH 测试数据集：

```
CREATE TABLE hive.tpch100.customer PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.customer  ;
CREATE TABLE hive.tpch100.lineitem PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.lineitem  ;
CREATE TABLE hive.tpch100.nation   PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.nation    ;
CREATE TABLE hive.tpch100.orders   PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.orders    ;
CREATE TABLE hive.tpch100.part     PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.part      ;
CREATE TABLE hive.tpch100.partsupp PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.partsupp  ;
CREATE TABLE hive.tpch100.region   PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.region    ;
CREATE TABLE hive.tpch100.supplier PROPERTIES("file_format" = "parquet") AS SELECT * FROM tpch.
    ↪ sf100.supplier  ;
```

> 在包含 3 个 16C BE 节点的 Doris 集群上，创建一个 TPCH 1000 的 Hive 数据集，大约需要 25 分钟，TPCH 10000 大约需要 4 到 5 个小时。

2.14.13.14 在 Hive/Iceberg 上构建 TPC-DS 数据集

Doris 支持通过 Trino Connector 兼容框架，使用 TPCDS Connector 来快速构建 TPCDS 测试集。

结合 Hive/Iceberg 表的数据写回功能，您可以快速通过 Doris 构建 Doris、 Hive、 Iceberg 表的 TPCDS 测试数据集。

本文档主要介绍如何部署和使用 TPCDS Connector 构建测试数据集。

> 该功能自 Doris 3.0.0 版本开始支持。

2.14.13.14.1 编译 TPCDS Connector

> 需要 JDK 17 版本。

```
git clone https://github.com/trinodb/trino.git
git checkout 435
cd trino/plugin/trino-tpcds
mvn clean install -DskipTest
```

完成编译后，会在 `trino/plugin/trino-tpcds/target/` 下得到 `trino-tpcds-435/` 目录。

也可以直接下载预编译的 trino-tpcds-435.tar.gz 并解压。

2.14.13.14.2 部署 TPCDS Connector

将 `trino-tpcds-435/` 目录放到所有 FE 和 BE 部署路径的 `connectors/` 目录下。(如果没有，可以手动创建)。

```
├── bin
├── conf
├── connectors
|   ├── trino-tpcds-435
...
```

部署完成后，建议重启 FE、 BE 节点以确保 Connector 可以被正确加载。

2.14.13.14.3 创建 TPCDS Catalog

```
CREATE CATALOG `tpcds` PROPERTIES (
    "type" = "trino-connector",
    "trino.connector.name" = "tpcds",
    "trino.tpcds.split-count" = "32"
);
```

其中 `tpcds.split-count` 为并发数，建议设置为 BE 单机核数的 2 倍，可以获得最优的并发度。提升数据生成效率。

2.14.13.14.4 使用 TPCDS Catalog

TPCDS Catalog 中预制了不同 Scale Factor 的 TPCDS 数据集，可以通过 SHOW DATABASES 和 SHOW TABLES 命令查看。

```
mysql> SWITCH tpcds;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| sf1                |
| sf100              |
| sf1000             |
| sf10000            |
| sf100000           |
| sf300              |
| sf3000             |
| sf30000            |
| tiny               |
+--------------------+

mysql> USE sf1;
mysql> SHOW TABLES;
+------------------------+
| Tables_in_sf1          |
+------------------------+
| call_center            |
| catalog_page           |
| catalog_returns        |
| catalog_sales          |
| customer               |
| customer_address       |
| customer_demographics  |
| date_dim               |
| dbgen_version          |
| household_demographics |
| income_band            |
| inventory              |
| item                   |
| promotion              |
| reason                 |
| ship_mode              |
| store                  |
```

```
| store_returns          |
| store_sales            |
| time_dim               |
| warehouse              |
| web_page               |
| web_returns            |
| web_sales              |
| web_site               |
+------------------------+
```

通过 SELECT 语句可以直接查询这些表。

> 这些预制数据集的数据，并没有实际存储，而是在查询时实时生成的。所以这些预制数据集不适合用来直接进行 Benchmark 测试。适用于通过 INSERT INTO SELECT 将数据集写入到其他目的表（如 Doris 内表、Hive、Iceberg 等所有 Doris 支持写入的数据源）后，对目的表进行性能测试。

2.14.13.14.5 构建 TPCDS 测试数据集

以下示例通过 CTAS 语句快速构建一个 Hive 上的 TPCDS 测试数据集：

```
CREATE TABLE hive.tpcds100.call_center            PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.call_center           ;
CREATE TABLE hive.tpcds100.catalog_page           PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.catalog_page          ;
CREATE TABLE hive.tpcds100.catalog_returns        PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.catalog_returns       ;
CREATE TABLE hive.tpcds100.catalog_sales          PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.catalog_sales         ;
CREATE TABLE hive.tpcds100.customer               PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.customer              ;
CREATE TABLE hive.tpcds100.customer_address       PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.customer_address      ;
CREATE TABLE hive.tpcds100.customer_demographics  PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.customer_demographics ;
CREATE TABLE hive.tpcds100.date_dim               PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.date_dim              ;
CREATE TABLE hive.tpcds100.dbgen_version          PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.dbgen_version         ;
CREATE TABLE hive.tpcds100.household_demographics PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.household_demographics;
CREATE TABLE hive.tpcds100.income_band            PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.income_band           ;
```

```
CREATE TABLE hive.tpcds100.inventory              PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.inventory              ;
CREATE TABLE hive.tpcds100.item                   PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.item                   ;
CREATE TABLE hive.tpcds100.promotion              PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.promotion              ;
CREATE TABLE hive.tpcds100.reason                 PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.reason                 ;
CREATE TABLE hive.tpcds100.ship_mode              PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.ship_mode              ;
CREATE TABLE hive.tpcds100.store                  PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.store                  ;
CREATE TABLE hive.tpcds100.store_returns          PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.store_returns          ;
CREATE TABLE hive.tpcds100.store_sales            PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.store_sales            ;
CREATE TABLE hive.tpcds100.time_dim               PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.time_dim               ;
CREATE TABLE hive.tpcds100.warehouse              PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.warehouse              ;
CREATE TABLE hive.tpcds100.web_page               PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.web_page               ;
CREATE TABLE hive.tpcds100.web_returns            PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.web_returns            ;
CREATE TABLE hive.tpcds100.web_sales              PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.web_sales              ;
CREATE TABLE hive.tpcds100.web_site               PROPERTIES("file_format" = "parquet") AS SELECT
    ↪  * FROM tpcds.sf100.web_site               ;
```

> 在包含 3 个 16C BE 节点的 Doris 集群上，创建一个 TPCDS 1000 的 Hive 数据集，大约需要 3 到 4 个小时。

2.15 可观测性

2.15.1 Log

本文介绍可观测性核心数之一 Log 的存储和分析实践，可观测性整体方案介绍请参考概述。

2.15.1.1 第 1 步：评估资源

在部署集群之前，首先应评估所需服务器硬件资源，包括以下几个关键步骤：

1. 评估写入资源：计算公式如下：

   - 平均写入吞吐 = 日增数据量 / 86400 s
   - 峰值写入吞吐 = 平均写入吞吐 * 写入吞吐峰值 / 均值比
   - 峰值写入所需 CPU 核数 = 峰值写入吞吐 / 单核写入吞吐

2. 评估存储资源：计算公式为 所需存储空间 = 日增数据量 / 压缩率 * 副本数 * 数据存储周期
3. 评估查询资源：查询的资源消耗随查询量和复杂度而异，建议初始预留 50% 的 CPU 资源用于查询，再根据实际测试情况进行调整。
4. 汇总整合资源：由第 1 步和第 3 步估算出所需 CPU 核数后，除以单机 CPU 核数，估算出 BE 服务器数量，再根据 BE 服务器数量和第 2 步的结果，估算出每台 BE 服务器所需存储空间，然后分摊到 4 ~ 12 块数据盘，计算出单盘存储容量。

以每天新增 100 TB 数据量（压缩前）、5 倍压缩率、2 副本、热数据存储 3 天、冷数据存储 30 天、写入吞吐峰值 / 均值比 200%、单核写入吞吐 10 MB/s、查询预留 50% CPU 资源为例，可估算出：

存算一体模式 - FE：3 台服务器，每台配置 16 核 CPU、64 GB 内存、1 块 100 GB SSD 盘 - BE：30 台服务器，每台配置 32 核 CPU、256 GB 内存、8 块 625 GB SSD 盘 - S3 对象存储空间：即为预估冷数据存储空间，540 TB

存算分离模式 - FE：3 台服务器，每台配置 16 核 CPU、64 GB 内存、1 块 100 GB SSD 盘 - BE：15 台服务器，每台配置 32 核 CPU、256 GB 内存、8 块 680 GB SSD 盘 - S3 对象存储空间：即为预估冷数据存储空间，600 TB

使用存算分离模式，写入和热数据存储只需要 1 副本，能够显著降低成本。

该例子中，各关键指标的值及具体计算方法可见下表：

关键指标（单位）	存算分离模式	存算一体模式	说明
日增数据量（TB）	100	100	根据实际需求填写
压缩率	5	5	一般为 5 ~ 10 倍（含索引），默认为 5，根据实际需求填写
副本数	1	2	根据实际需求填写，默认 1 副本，可选值：1，2，3
热数据存储周期（天）	3	3	根据实际需求填写
冷数据存储周期（天）	30	27	根据实际需求填写
总存储周期（天）	30	30	算法：热数据存储周期 + 冷数据存储周期
预估热数据存储空间（TB）	60	120	算法： 日增数据量 / 压缩率 * 副本数 * 热数据存储周期
预估冷数据存储空间（TB）	600	540	算法： 日增数据量 / 压缩率 * 副本数 * 冷数据存储周期
写入吞吐峰值 / 均值比	200%	200%	根据实际需求填写，默认 200%
单机 CPU 核数	32	32	根据实际需求填写，默认 32 核
平均写入吞吐（MB/s）	1214	2427	算法：日增数据量 / 86400 s
峰值写入吞吐（MB/s）	2427	4855	算法：平均写入吞吐 * 写入吞吐峰值 / 均值比
峰值写入所需 CPU 核数	242.7	485.5	算法：峰值写入吞吐 / 单核写入吞吐
查询预留 CPU 百分比	50%	50%	根据实际需求填写，默认 50%

关键指标（单位）	存算分离模式	存算一体模式	说明
预估 BE 服务器数	15.2	30.3	算法：峰值写入所需 CPU 核数 / 单机 CPU 核数 /（1 - 查询预留 CPU 百分比）
预估 BE 服务器数取整	15	30	算法：MAX（副本数，预估 BE 服务器数取整）
预估每台 BE 服务器存储空间（TB）	5.33	5.33	算法：预估热数据存储空间 / 预估 BE 服务器数 /（1 - 30%），其中，30% 是存储空间预留值。建议每台 BE 服务器挂载 4 ~ 12 块数据盘，以提高 I/O 能力。

2.15.1.2 第 2 步：部署集群

完成资源评估后，可以开始部署 Apache Doris 集群，推荐在物理机及虚拟机环境中进行部署。手动部署集群，可参考手动部署。

2.15.1.3 第 3 步：优化 FE 和 BE 配置

完成集群部署后，需分别优化 FE 和 BE 配置参数，以更加契合日志存储与分析的场景。

优化 FE 配置

在 fe/conf/fe.conf 目录下找到 FE 的相关配置项，并按照以下表格，调整 FE 配置。

需调整参数	说明
max_running_txn_num_per_db = 10000	高并发导入运行事务数较多，需调高参数。
streaming_label_keep_max_second = 3600 label_keep_max_second = 7200	高频导入事务标签内存占用多，保留时间调短。
enable_round_robin_create_tablet = true	创建 Tablet 时，采用 Round Robin 策略，尽量均匀。
tablet_rebalancer_type = partition	均衡 Tablet 时，采用每个分区内尽量均匀的策略。
autobucket_min_buckets = 10	将自动分桶的最小分桶数从 1 调大到 10，避免日志量增加时分桶不够。
max_backend_heartbeat_failure_tolerance_count = 10	日志场景下 BE 服务器压力较大，可能短时间心跳超时，因此将容忍次数从 1 调大到 10。

更多关于 FE 配置项的信息，可参考FE 配置项。

优化 BE 配置

在 be/conf/be.conf 目录下找到 BE 的相关配置项，并按照以下表格，调整 BE 配置。

模块	需调整参数	说明
存储	storage_root_path = /path/to/dir1;/path/to/dir2;...;/path/to/dir12	配置热数据在磁盘目录上的存储路径。
-	enable_file_cache = true	开启文件缓存。

模块	需调整参数	说明
-	file_cache_path = [{"path": "/mnt/datadisk0/file_cache", "total_size":53687091200, "query_limit": "10737418240"},{"path": "/mnt/datadisk1/file_cache", "total_size":53687091200,"query_limit": "10737418240"}]	配置冷数据的缓存路径和相关设置，具体配置说明如下：path：缓存路径total_size：该缓存路径的总大小，单位为字节，53687091200字节等于 50 GBquery_limit：单次查询可以从缓存路径中查询的最大数据量，单位为字节，10737418240 字节等于 10 GB
写入	write_buffer_size = 1073741824	增加写入缓冲区（buffer）的文件大小，减少小文件和随机 I/O 操作，提升性能。
-	max_tablet_version_num = 20000	配合建表的 time_series compaction 策略，允许更多版本暂时未合并。
Compaction	max_cumu_compaction_threads = 8	设置为 CPU 核数 / 4，意味着 CPU 资源的 1/4 用于写入，1/4 用于后台 Compaction，2/1 留给查询和其他操作。
-	inverted_index_compaction_enable = true	开启索引合并（index compaction），减少 Compaction 时的 CPU 消耗。
-	enable_segcompaction = false enable_ordered_data_compaction = false	关闭日志场景不需要的两个 Compaction 功能。
-	enable_compaction_priority_scheduling = false	低优先级 compaction 在一块盘上限制 2 个任务，会影响 compaction 速度。
-	total_permits_for_compaction_score = 200000	该参数用来控制内存，time series 策略下本身可以控制内存。
缓存	disable_storage_page_cache = true inverted_index_searcher_cache_limit = 30%	因为日志数据量较大，缓存（cache）作用有限，因此关闭数据缓存，调换为索引缓存（index cache）的方式。
-	inverted_index_cache_stale_sweep_time_sec = 3600 index_cache_entry_stay_time_after_lookup_s = 3600	让索引缓存在内存中尽量保留 1 小时。
-	enable_inverted_index_cache_on_cooldown = true enable_write_index_searcher_cache = false	开启索引上传冷数据存储时自动缓存的功能。
-	tablet_schema_cache_recycle_interval = 3600 segment_cache_capacity = 20000	减少其他缓存对内存的占用。
-	inverted_index_ram_dir_enable = true	减少写入时索引临时文件带来的 IO 开销。
线程	pipeline_executor_size = 24 doris_scanner_thread_pool_thread_num = 48	32 核 CPU 的计算线程和 I/O 线程配置，根据核数等比扩缩。
-	scan_thread_nice_value = 5	降低查询 I/O 线程的优先级，保证写入性能和时效性。
其他	string_type_length_soft_limit_bytes = 10485760	将 String 类型数据的长度限制调高至 10 MB。

模块	需调整参数	说明

-	`trash_file_expire_time_sec = 300`
	`path_gc_check_interval_second = 900`
	`path_scan_interval_second = 900`
		调快垃圾文件的回收时间。

更多关于 BE 配置项的信息，可参考BE 配置项。

2.15.1.4 第 4 步：建表

由于日志数据的写入和查询都具备明显的特征，因此，在建表时按照本节说明进行针对性配置，以提升性能表现。

配置分区分桶参数

分区按照以下说明配置：- 使用时间字段上的Range 分区 (`PARTITION BY RANGE(ts)`)，并开启动态分区 (`"dynamic_partition.enable" = "true"`)，按天自动管理分区。- 使用 Datetime 类型的时间字段作为排序 Key (`DUPLICATE KEY(ts)`)，在查询最新 N 条日志时有数倍加速。

分桶按照以下说明配置：- 分桶数量大致为集群磁盘总数的 3 倍，每个桶的数据量压缩后 5GB 左右。- 使用 Random 策略 (`DISTRIBUTED BY RANDOM BUCKETS 60`)，配合写入时的 Single Tablet 导入，可以提升批量（Batch）写入的效率。

更多关于分区分桶的信息，可参考数据划分。

配置压缩参数 - 使用 zstd 压缩算法 (`"compression" = "zstd"`), 提高数据压缩率。

配置 Compaction 参数

按照以下说明配置 Compaction 参数：

- 使用 time_series 策略 (`"compaction_policy" = "time_series"`)，以减轻写放大效应，对于高吞吐日志写入的资源写入很重要。

配置索引参数

按照以下说明操作：- 对经常查询的字段建索引 (`USING INVERTED`)。- 对需要全文检索的字段，将分词器（parser）参数赋值为 unicode，一般能满足大部分需求。如有支持短语查询的需求，将 support_phrase 参数赋值为 true；如不需要，则设置为 false，以降低存储空间。

配置存储策略

按照以下说明操作：

- 对于热存储数据，如果使用云盘，可配置 1 副本；如果使用物理盘，则至少配置 2 副本 (`"replication_num" = "2"`)。
- 配置 `log_s3` 的存储位置 (`CREATE RESOURCE "log_s3"`)，并设置 `log_policy_3day` 冷热数据分层策略 (`CREATE STORAGE POLICY log_policy_3day`)，即在超过 3 天后将数据冷却至 `log_s3` 指定的存储位置。可参考以下 SQL：

```
CREATE DATABASE log_db;
USE log_db;

-- 存算分离模式不需要
CREATE RESOURCE "log_s3"
PROPERTIES
(
    "type" = "s3",
    "s3.endpoint" = "your_endpoint_url",
    "s3.region" = "your_region",
    "s3.bucket" = "your_bucket",
    "s3.root.path" = "your_path",
    "s3.access_key" = "your_ak",
    "s3.secret_key" = "your_sk"
);

-- 存算分离模式不需要
CREATE STORAGE POLICY log_policy_3day
PROPERTIES(
    "storage_resource" = "log_s3",
    "cooldown_ttl" = "259200"
);

CREATE TABLE log_table
(
  `ts` DATETIME,
  `host` TEXT,
  `path` TEXT,
  `message` TEXT,
  INDEX idx_host (`host`) USING INVERTED,
  INDEX idx_path (`path`) USING INVERTED,
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`ts`)
PARTITION BY RANGE(`ts`) ()
DISTRIBUTED BY RANDOM BUCKETS 60
PROPERTIES (
  "compression" = "zstd",
  "compaction_policy" = "time_series",
  "dynamic_partition.enable" = "true",
  "dynamic_partition.create_history_partition" = "true",
  "dynamic_partition.time_unit" = "DAY",
```

```
  "dynamic_partition.start" = "-30",
  "dynamic_partition.end" = "1",
  "dynamic_partition.prefix" = "p",
  "dynamic_partition.buckets" = "60",
  "dynamic_partition.replication_num" = "2", -- 存算分离不需要
  "replication_num" = "2", -- 存算分离不需要
  "storage_policy" = "log_policy_3day" -- 存算分离不需要
);
```

2.15.1.5 第 5 步：采集日志

完成建表后，可进行日志采集。

Apache Doris 提供开放、通用的 Stream HTTP APIs，通过这些 APIs，你可与常用的日志采集器打通，包括 Logstash、Filebeat、 Kafka 等，从而开展日志采集工作。本节介绍了如何使用 Stream HTTP APIs 对接日志采集器。

对接 Logstash

按照以下步骤操作：

1. 下载并安装 Logstash Doris Output 插件。你可选择以下两种方式之一：

   - 直接下载：点此下载。
   - 从源码编译，并运行下方命令安装：

```
./bin/logstash-plugin install logstash-output-doris-1.2.0.gem
```

2. 配置 Logstash。需配置以下参数：

   - `logstash.yml`：配置 Logstash 批处理日志的条数和时间，用于提升数据写入性能。

```
pipeline.batch.size: 1000000
pipeline.batch.delay: 10000
```

   - `logstash_demo.conf`：配置所采集日志的具体输入路径和输出到 Apache Doris 的设置。

```
input {
    file {
    path => "/path/to/your/log"
  }
}

output {
  doris {
```

```
    http_hosts => [ "<http://fehost1:http_port>", "<http://fehost2:http_port>", "<http://fehost3:
        ↪ http_port">]
    user => "your_username"
    password => "your_password"
    db => "your_db"
    table => "your_table"

    # doris stream load http headers
    headers => {
    "format" => "json"
    "read_json_by_line" => "true"
    "load_to_single_tablet" => "true"
    }

    # field mapping: doris fileld name => logstash field name
    # %{} to get a logstash field, [] for nested field such as [host][name] for host.name
    mapping => {
    "ts" => "%{@timestamp}"
    "host" => "%{[host][name]}"
    "path" => "%{[log][file][path]}"
    "message" => "%{message}"
    }
    log_request => true
    log_speed_interval => 10
  }
}
```

3. 按照下方命令运行 Logstash，采集日志并输出至 Apache Doris。

```
./bin/logstash -f logstash_demo.conf
```

更多关于 Logstash 配置和使用的说明，可参考Logstash Doris Output Plugin。

对接 Filebeat

按照以下步骤操作：

1. 获取支持输出至 Apache Doris 的 Filebeat 二进制文件。可 点此下载 或者从 Apache Doris 源码编译。
2. 配置 Filebeat。需配置以下参数：

- `filebeat_demo.yml`：配置所采集日志的具体输入路径和输出到 Apache Doris 的设置。

```
  # input
  filebeat.inputs:
  - type: log
```

```
  enabled: true
  paths:
    - /path/to/your/log
  # multiline 可以将跨行的日志（比如 Java stacktrace）拼接起来
  multiline:
    type: pattern
    # 效果：以 yyyy-mm-dd HH:MM:SS 开头的行认为是一条新的日志，其他都拼接到上一条日志
    pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}'
    negate: true
    match: after
    skip_newline: true

processors:
# 用 js script 插件将日志中的 `\t` 替换成空格，避免 JSON 解析报错
- script:
    lang: javascript
    source: >
        function process(event) {
            var msg = event.Get("message");
            msg = msg.replace(/`\t`/g, "  ");
            event.Put("message", msg);
        }
# 用 dissect 插件做简单的日志解析
- dissect:
    # 2024-06-08 18:26:25,481 INFO (report-thread|199) [ReportHandler.cpuReport():617] begin to
        ↪  handle
    tokenizer: "%{day} %{time} %{log_level} (%{thread}) [%{position}] %{content}"
    target_prefix: ""
    ignore_failure: true
    overwrite_keys: true

# queue and batch
queue.mem:
  events: 1000000
  flush.min_events: 100000
  flush.timeout: 10s

# output
output.doris:
  fenodes: [ "http://fehost1:http_port", "http://fehost2:http_port", "http://fehost3:http_port"
      ↪  ]
  user: "your_username"
  password: "your_password"
  database: "your_db"
  table: "your_table"
```

```
    # output string format
    ## %{[agent][hostname]} %{[log][file][path]} 是filebeat自带的metadata
    ## 常用的 filebeat metadata 还是有采集时间戳 %{[@timestamp]}
    ## %{[day]} %{[time]} 是上面 dissect 解析得到字段
    codec_format_string: '{"ts": "%{[day]} %{[time]}", "host": "%{[agent][hostname]}", "path":
        ↪ "%{[log][file][path]}", "message": "%{[message]}"}'
    headers:
      format: "json"
      read_json_by_line: "true"
      load_to_single_tablet: "true"
```

3. 按照下方命令运行 Filebeat，采集日志并输出至 Apache Doris。

```
chmod +x filebeat-doris-2.1.1
./filebeat-doris-2.1.1 -c filebeat_demo.yml
```

更多关于 Filebeat 配置和使用的说明，可参考Beats Doris Output Plugin。

对接 Kafka

将 JSON 格式的日志写入 Kafka 的消息队列，创建 Kafka Routine Load，即可让 Apache Doris 从 Kafka 主动拉取数据。

可参考如下示例。其中，`property.*` 是 Librdkafka 客户端相关配置，根据实际 Kafka 集群情况配置。

```
-- 准备好 kafka 集群和 topic log__topic_
-- 创建 routine load，从 kafka log__topic_将数据导入 log_table 表
CREATE ROUTINE LOAD load_log_kafka ON log_db.log_table
COLUMNS(ts, clientip, request, status, size)
PROPERTIES (
"max_batch_interval" = "60",
"max_batch_rows" = "20000000",
"max_batch_size" = "1073741824",
"load_to_single_tablet" = "true",
"format" = "json"
)
FROM KAFKA (
"kafka_broker_list" = "host:port",
"kafka_topic" = "log__topic_",
"property.group.id" = "your_group_id",
"property.security.protocol"="SASL_PLAINTEXT",
"property.sasl.mechanism"="GSSAPI",
"property.sasl.kerberos.service.name"="kafka",
"property.sasl.kerberos.keytab"="/path/to/xxx.keytab",
"property.sasl.kerberos.principal"="<xxx@yyy.com>"
);
-- 查看 routine 的状态
SHOW ROUTINE LOAD;
```

更多关于 Kafka 配置和使用的说明，可参考Routine Load。

使用自定义程序采集日志

除了对接常用的日志采集器以外，你也可以自定义程序，通过 HTTP API Stream Load 将日志数据导入 Apache Doris。参考以下代码：

```
curl
--location-trusted
-u username:password
-H "format:json"
-H "read_json_by_line:true"
-H "load_to_single_tablet:true"
-H "timeout:600"
-T logfile.json
http://fe_host:fe_http_port/api/log_db/log_table/_stream_load
```

在使用自定义程序时，需注意以下关键点：

- 使用 Basic Auth 进行 HTTP 鉴权，用命令 `echo -n 'username:password' | base64` 进行计算。
- 设置 HTTP header “format:json”，指定数据格式为 JSON。
- 设置 HTTP header “read_json_by_line:true”，指定每行一个 JSON。
- 设置 HTTP header “load_to_single_tablet:true”，指定一次导入写入一个分桶减少导入的小文件。
- 建议写入客户端一个 Batch 的大小为 100MB ~ 1GB。如果你使用的是 Apache Doris 2.1 及更高版本，需通过服务端 Group Commit 功能，降低客户端 Batch 大小。

2.15.1.6 第 6 步：查询和分析日志

日志查询

Apache Doris 支持标准 SQL，因此，你可以通过 MySQL 客户端或者 JDBC 等方式连接到集群，执行 SQL 进行日志查询。参考以下命令：

```
mysql -h fe_host -P fe_mysql_port -u your_username -Dyour_db_name
```

下方列出常见的 5 条 SQL 查询命令，以供参考：

- 查看最新的 10 条数据

```
SELECT * FROM your_table_name ORDER BY ts DESC LIMIT 10;
```

- 查询 host 为 8.8.8.8 的最新 10 条数据

```
SELECT * FROM your_table_name WHERE host = '8.8.8.8' ORDER BY ts DESC LIMIT 10;
```

- 检索请求字段中有 error 或者 404 的最新 10 条数据。其中，MATCH_ANY 是 Apache Doris 全文检索的 SQL 语法，用于匹配参数中任一关键字。

```
SELECT * FROM your_table_name WHERE message MATCH_ANY 'error 404'
ORDER BY ts DESC LIMIT 10;
```

- 检索请求字段中有 image 和 faq 的最新 10 条数据。其中，MATCH_ALL 是 Apache Doris 全文检索的 SQL 语法，用于匹配参数中所有关键字。

```
SELECT * FROM your_table_name WHERE message MATCH_ALL 'image faq'
ORDER BY ts DESC LIMIT 10;
```

- 检索请求字段中有 image 和 faq 的最新 10 条数据。其中，MATCH_PHRASE 是 Apache Doris 全文检索的 SQL 语法，用于匹配参数中所有关键字，并且要求顺序一致。在下方例子中，a image faq b 能匹配，但是 a faq image b 不能匹配，因为 image 和 faq 的顺序与查询不一致。

```
SELECT * FROM your_table_name WHERE message MATCH_PHRASE 'image faq'
ORDER BY ts DESC LIMIT 10;
```

可视化日志分析

一些第三方厂商提供了基于 Apache Doris 的可视化日志分析开发平台，包含类 Kibana Discover 的日志检索分析界面，提供直观、易用的探索式日志分析交互。

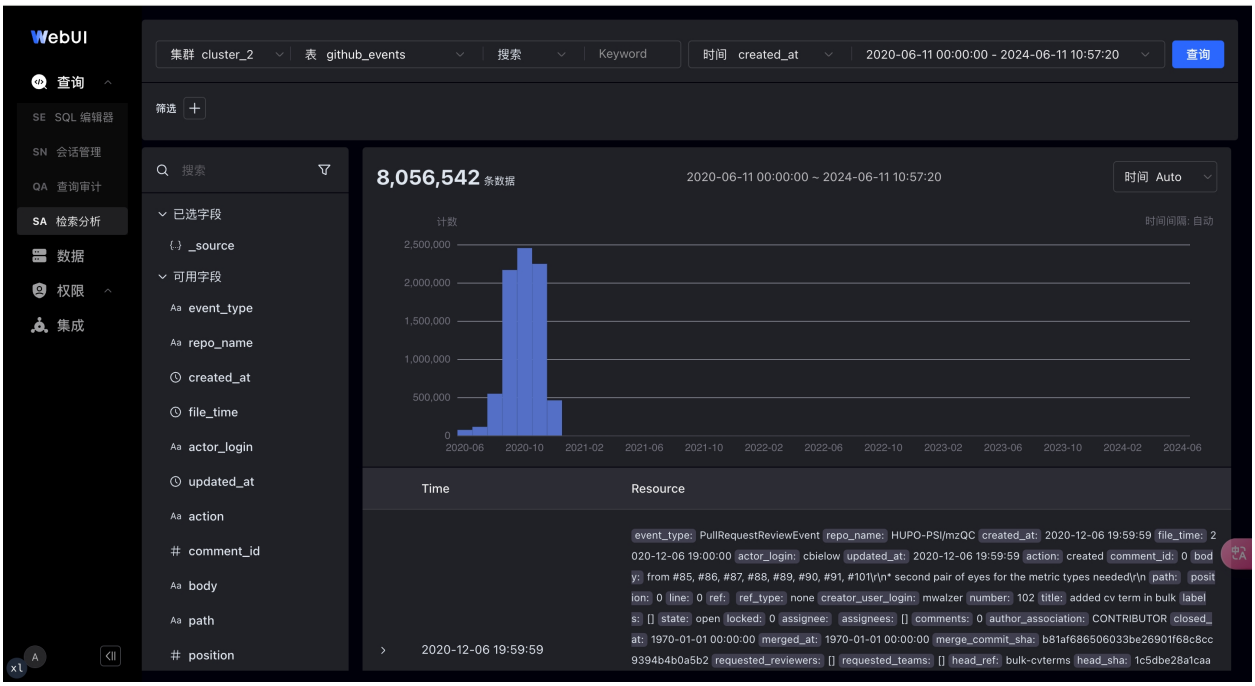


图 97: WebUI

- 支持全文检索和 SQL 两种模式

- 支持时间框和直方图上选择查询日志的时间段
- 支持信息丰富的日志明细展示，还可以展开成 JSON 或表格
- 在日志数据上下文交互式点击增加和删除筛选条件
- 搜索结果的字段 Top 值展示，便于发现异常值和进一步下钻分析

您可以联系 dev@doris.apache.org 获得更多帮助。

2.15.2 Trace

本文介绍可观测性核心数据之一 Trace 的存储分析实践，可观测性整体方案介绍请参考概述，资源评估、集群部署和优化可以参考 Log。

2.15.2.1 1. 建表

Trace 数据的写入和查询模式有明显的特征，在建表时进行针对性的配置会有更好的性能表现。参考下面的关键说明创建表：

分区和排序 - 分区使用时间字段上的 RANGE 分区，开启动态 Partition 按天自动管理分区 - 使用 service_name 和 DATETIME 类型的时间字段作为 Key，在查询指定 service 一段时间的 Trace 时有数倍加速

分桶 - 分桶个数大致是集群磁盘总数的 3 倍 - 分桶策略使用 RANDOM，配合写入时的 single tablet 导入可以提升写入 batch 效果

compaction - 使用 time_series compaction 策略减少写放大，对于高吞吐 Trace 写入的资源优化很重要

VARIANT 数据类型 - 对于 Trace 扩展字段比如 span_attributes 和 resource_attributes 使用半结构化数据类型 VARIANT，自动将 JSON 数据拆分成子列存储，提升压缩率降低存储空间，提升过滤和分析子列的性能

索引 - 对经常查询的字段建索引 - 需要全文检索的字段指定分词器 parser 参数，unicode 分词一般能满足绝大多数需求，开启 support_phrase 选项以支持短语查询，如果不需要可以设置为 false 降低存储空间

存储 - 热存数据，如果使用云盘可以配置 1 副本，如果使用物理盘至少配置 2 副本 - 使用冷热分离配置 log_s3 对象存储和 log_policy_3day 超过 3 天转存 s3 策略

```
CREATE DATABASE log_db;
USE log_db;

-- 存算分离模式不需要
CREATE RESOURCE "log_s3"
PROPERTIES
(
    "type" = "s3",
    "s3.endpoint" = "your_endpoint_url",
    "s3.region" = "your_region",
    "s3.bucket" = "your_bucket",
    "s3.root.path" = "your_path",
    "s3.access_key" = "your_ak",
    "s3.secret_key" = "your_sk"
);
```

```
-- 存算分离模式不需要
CREATE STORAGE POLICY log_policy_3day
PROPERTIES(
    "storage_resource" = "log_s3",
    "cooldown_ttl" = "259200"
);

CREATE TABLE trace_table
(
    service_name          VARCHAR(200),
    timestamp             DATETIME(6),
    service_instance_id   VARCHAR(200),
    trace_id              VARCHAR(200),
    span_id               STRING,
    trace_state           STRING,
    parent_span_id        STRING,
    span_name             STRING,
    span_kind             STRING,
    end_time              DATETIME(6),
    duration              BIGINT,
    span_attributes       VARIANT,
    events                ARRAY<STRUCT<timestamp:DATETIME(6), name:STRING, attributes:MAP<STRING,
        ↪  STRING>>>,
    links                 ARRAY<STRUCT<trace_id:STRING, span_id:STRING, trace_state:STRING,
        ↪ attributes:MAP<STRING, STRING>>>,
    status_message        STRING,
    status_code           STRING,
    resource_attributes   VARIANT,
    scope_name            STRING,
    scope_version         STRING,
    INDEX idx_timestamp(timestamp) USING INVERTED,
    INDEX idx_service_instance_id(service_instance_id) USING INVERTED,
    INDEX idx_trace_id(trace_id) USING INVERTED,
    INDEX idx_span_id(span_id) USING INVERTED,
    INDEX idx_trace_state(trace_state) USING INVERTED,
    INDEX idx_parent_span_id(parent_span_id) USING INVERTED,
    INDEX idx_span_name(span_name) USING INVERTED,
    INDEX idx_span_kind(span_kind) USING INVERTED,
    INDEX idx_end_time(end_time) USING INVERTED,
    INDEX idx_duration(duration) USING INVERTED,
    INDEX idx_span_attributes(span_attributes) USING INVERTED,
    INDEX idx_status_message(status_message) USING INVERTED,
    INDEX idx_status_code(status_code) USING INVERTED,
    INDEX idx_resource_attributes(resource_attributes) USING INVERTED,
```

```
    INDEX idx_scope_name(scope_name) USING INVERTED,
    INDEX idx_scope_version(scope_version) USING INVERTED
)
ENGINE = OLAP
DUPLICATE KEY(service_name, timestamp)
PARTITION BY RANGE(timestamp) ()
DISTRIBUTED BY RANDOM BUCKETS 250
PROPERTIES (
"compression" = "zstd",
"compaction_policy" = "time_series",
"inverted_index_storage_format" = "V2",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "250",
"dynamic_partition.replication_num" = "2", -- 存算分离不需要
"replication_num" = "2" -- 存算分离不需要
"storage_policy" = "log_policy_3day" -- 存算分离不需要
);
```

2.15.2.2 2. Trace 采集

Doris 提供开放通用的 Stream HTTP API，可以与 OpenTelemetry 等 Trace 采集系统打通。

2.15.2.2.1 OpenTelemetry 对接

1. 应用侧接入 OpenTelemetry SDK

这里我们使用一个 Spring Boot 示例应用接入 OpenTelemetry Java SDK，示例应用来自官方 demo，对路径 “/” 返回简单的 “Hello World!” 字符串。下载 OpenTelemetry Java Agent，使用 Java Agent 的优势在于无需对现有的应用做任何的修改。其他语言及其他接入方式详见 OpenTelemetry 官网：Language APIs & SDKs 或 Zero-code Instrumentation。

2. 部署配置 OpenTelemetry Collector

下载 OpenTelemetry Collector 并解压。需要下载以 “otelcol-contrib” 为前缀的包，其中的 Doris Exporter 组件能够把 trace 数据导入到 Doris 中。

创建 `otel_demo.yaml` 配置文件如下，更多配置详见 Doris Exporter 文档。

```
receivers:
  otlp: # otlp 协议，接收 OpenTelemetry Java Agent 发送的数据
    protocols:
```

```
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      traces: doris_table_name
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"
```

3. 运行 OpenTelemetry Collector

```
./otelcol-contrib --config otel_demo.yaml
```

4. 启动 Spring Boot 示例应用

在启动应用之前只需要添加几个环境变量，无需修改任何代码。

```
export JAVA_TOOL_OPTIONS="${JAVA_TOOL_OPTIONS} -javaagent:/your/path/to/opentelemetry-javaagent.
    ↪ jar" # OpenTelemetry Java Agent 的路径
export OTEL_JAVAAGENT_LOGGING="none" # 禁用 otel log，防止干扰服务本身的日志
export OTEL_SERVICE_NAME="myproject"
export OTEL_TRACES_EXPORTER="otlp" # 使用 otlp 协议发送 trace 数据
export OTEL_EXPORTER_OTLP_ENDPOINT="http://localhost:4317" # OpenTelemetry Collector 的地址

java -jar myproject-0.0.1-SNAPSHOT.jar
```

5. 访问 Spring Boot 示例应用产生 Trace 数据

curl loalhost:8080 会触发 hello 服务调用，OpenTelemetry Java Agent 会自动生成 Trace 数据，然后发送给 OpenTelemetry Collector，Collector 再通过配置的 Doris Exporter 将 Trace 数据写入 Doris 的表中（默认是 otel.otel_traces）。

2.15.2.3 3. Trace 查询

Trace 查询通常使用可视化的查询界面，比如 Grafana。

- 通过时间段和服务名筛选，展示 Trace 概览，包括延迟分布图和最细的一些 Trace

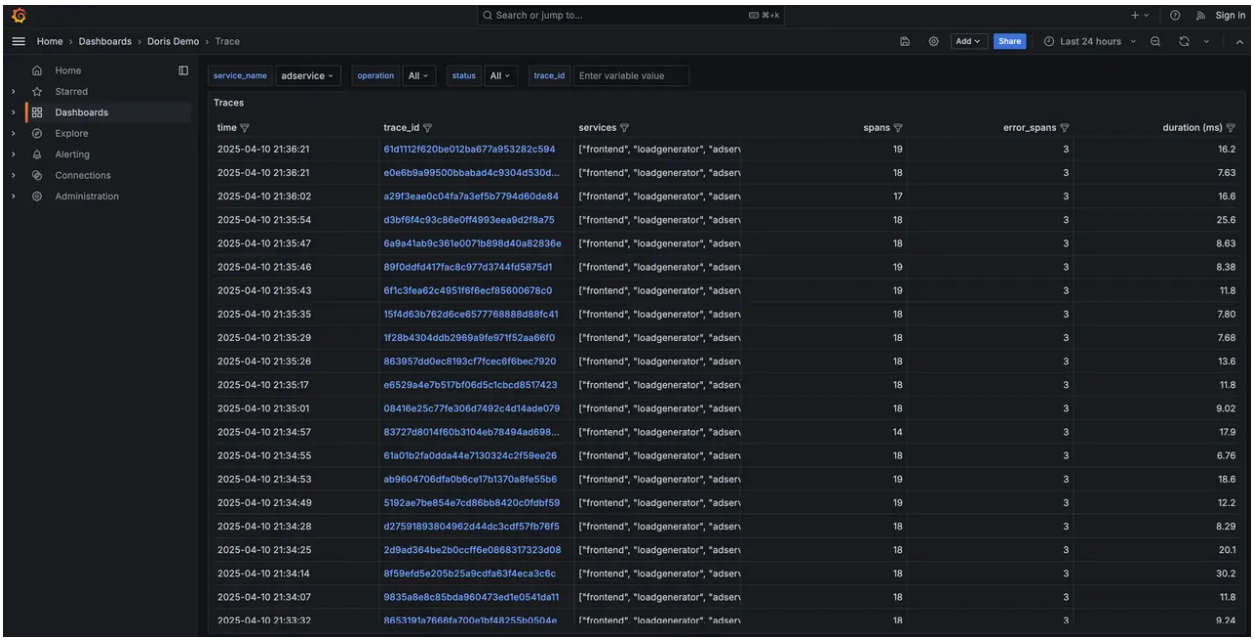


图 98: Trace 列表

- 点击链接可以查看 Trace detail

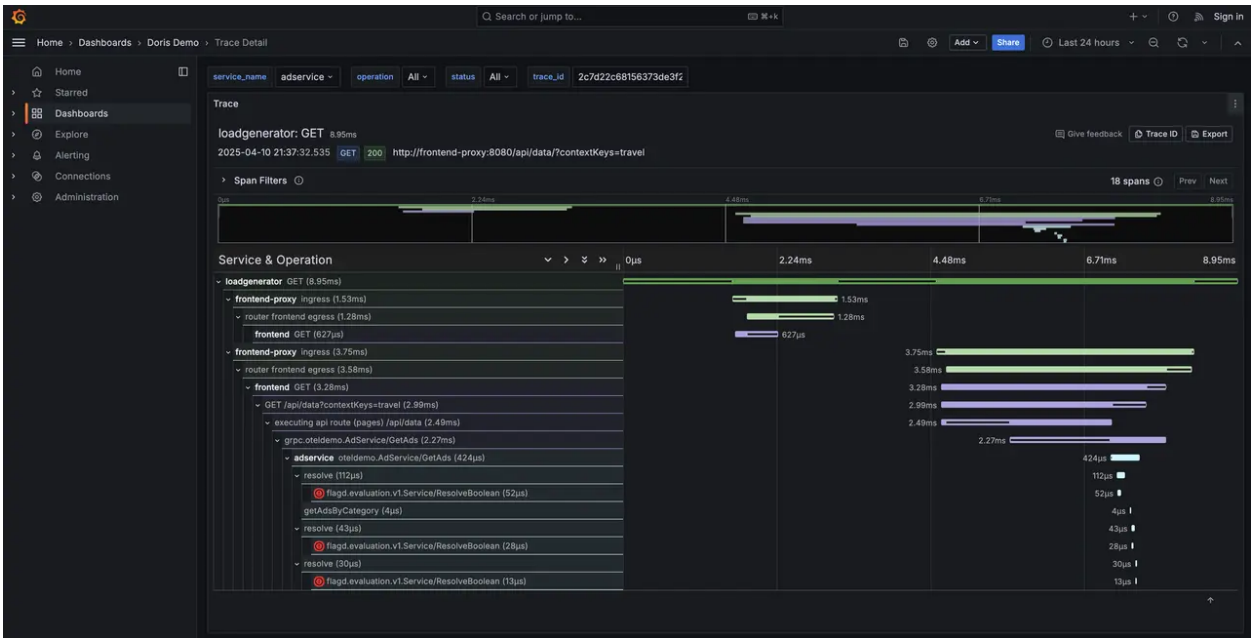


图 99: Trace 查询

2.15.3 集成

2.15.3.1 Logstash

2.15.3.1.1 介绍

Logstash 是一个日志 ETL 框架（采集，预处理，发送到存储系统），它支持自定义输出插件将数据写入存储系统，Logstash Doris output plugin 是输出到 Doris 的插件。

Logstash Doris output plugin 调用Doris Stream Load HTTP 接口将数据实时写入 Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Logstash Doris output plugin 主要有三个步骤：1. 将插件安装到 Logstash 中 2. 配置 Doris 输出地址和其他参数 3. 启动 Logstash 将数据实时写入 Doris

2.15.3.1.2 安装

获取插件

可以从官网下载或者自行从源码编译 Logstash Doris output plugin。

- 从官网下载
- 不包含依赖的安装包 https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/logstash-output-doris-1.2.0.gem
- 包含依赖的安装包 https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/logstash-output-doris-1.2.0.zip

- 从源码编译

```
cd extension/logstash/

gem build logstash-output-doris.gemspec
```

安装插件

- 普通安装

${LOGSTASH_HOME} 是 Logstash 的安装目录，运行它下面的 bin/logstash-plugin 命令安装插件

```
${LOGSTASH_HOME}/bin/logstash-plugin install logstash-output-doris-1.2.0.gem

Validating logstash-output-doris-1.2.0.gem
Installing logstash-output-doris
Installation successful
```

普通安装模式会自动安装插件依赖的 ruby 模块，对于网络不通的情况会卡住无法完成，这种情况下可以下载包含依赖的 zip 安装包进行完全离线安装，注意需要用 file:// 指定本地文件系统。

- 离线安装

```
${LOGSTASH_HOME}/bin/logstash-plugin install file:///tmp/logstash-output-doris-1.2.0.zip

Installing file: logstash-output-doris-1.2.0.zip
Resolving dependencies.........................
Install successful
```

2.15.3.1.3 参数配置

Logstash Doris output plugin 的配置如下：

配置	说明
http ↪ _ ↪ hosts ↪	Stream Load HTTP 地址，格式是字符串数组，可以有一个或者多个元素，每个元素是 host:port。例如：[“http://fe1:8030”, “http://fe2:8030”]
user	Doris 用户名，该用户需要有 doris 对应库表的导入权限
password ↪	Doris 用户的密码
db	要写入的 Doris 库名

配置	说明
table ↪	要写入的 Doris 表名
label ↪ _ ↪ prefix ↪	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}{db}{table}{yyyy，默认值是 logstash

headers
↪
	Doris Stream Load 的 headers 参数，语法格式为 ruby map，例如：
	headers => {
	“format” => “json”
	“read_json_by_line” => “true”
	}

配置	说明
mapping	Logstash 字段到 Doris 表字段的映射，参考后续章节的使用示例
message_only	一种特殊的 mapping 形式，只将 Logstash 的 @message 字段输出到 Doris，默认为 false

配置	说明
max_ ↪ retries ↪	Doris Stream Load 请求失败重试次数，默认为 -1 无限重试保证数据可靠性
log_ ↪ request ↪	日志中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 false

配置	说明
log_ ↪ speed ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

2.15.3.1.4 使用示例

TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. Logstash 配置

Logstash 主要有两类配置文件，一类是整个 Logstash 的配置文件，另一类是某个日志采集的配置文件。

整个 Logstash 的配置文件通常在 config/logstash.yml，为了提升写入 Doris 的性能需要修改 batch 大小和攒批时间，对于平均每条 i 几百字节的日志，推荐 100 万行和 10s。

```
pipeline.batch.size: 1000000
pipeline.batch.delay: 10000
```

某个日志采集的配置文件如 logstash_doris_log.conf 主要由 3 部分组成，分别对应 ETL 的各个部分：1. input 负责

读取原始数据 2. filter 负责做数据转换 3. output 负责将数据输出

```
#### 1. input 负责读取原始数据
#### file input 是一个 input plugin，可以配置读取的日志文件路径，通过 multiline codec
    ↪ 将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。file input
    ↪ 会将日志内容保存在 @message 字段中，另外还有一些元数据字段比如 host，log.file.path，
    ↪ 这里我们还通过 add_field 手动添加了一个字段 type，它的值为 fe.log 。
input {
    file {
        path => "/mnt/disk2/xiaokang/opt/doris_master/fe/log/fe.log"
        add_field => {"type" => "fe.log"}
        codec => multiline {
            # valid line starts with timestamp
            pattern => "^%{TIMESTAMP_ISO8601} "
            # any line not starting with a timestamp should be merged with the previous line
            negate => true
            what => "previous"
        }
    }
}

#### 2. filter 部分负责数据转换
#### grok 是一个常用的数据转换插件，它内置了一些常见的pattern 比如 TIMESTAMP_ISO8601 解析时间戳，
    ↪ 还支持写正则表达式提取字段。
filter {
    grok {
        match => {
            # parse log_time, level, thread, position fields from message
            "message" => "%{TIMESTAMP_ISO8601:log_time} (?<level>[A-Z]+) \((?<thread>[^\[]*)\)
                ↪ \[(?<position>[^\]]*)\]"
        }
    }
}

#### 3. output 部分负责数据输出
#### doris output 将数据输出到 Doris，使用的是 Stream Load HTTP 接口。通过 headers 参数指定了
    ↪ Stream Load 的数据格式为 JSON，通过 mapping 参数指定 Logstash 字段到 JSON 字段的映射。
    ↪ 由于 headers 指定了 "format" => "json"，Stream Load 会自动解析 JSON 字段写入对应的 Doris
    ↪ 表的字段。
output {
    doris {
        http_hosts => ["http://localhost:8630"]
        user => "root"
        password => ""
        db => "log_db"
```

```
        table => "doris_log"
        headers => {
          "format" => "json"
          "read_json_by_line" => "true"
          "load_to_single_tablet" => "true"
        }
        mapping => {
          "log_time" => "%{log_time}"
          "collect_time" => "%{@timestamp}"
          "host" => "%{[host][name]}"
          "path" => "%{[log][file][path]}"
          "type" => "%{type}"
          "level" => "%{level}"
          "thread" => "%{thread}"
          "position" => "%{position}"
          "message" => "%{message}"
        }
        log_request => true
    }
}
```

4. 运行 Logstash

```
${LOGSTASH_HOME}/bin/logstash -f config/logstash_doris_log.conf

#### log_request 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

[2024-07-08T22:35:34,772][INFO ][logstash.outputs.doris   ][main][
    ↪ e44d2a24f17d764647ce56f5fed24b9bbf08d3020c7fddcc3298800daface80a] doris stream load
    ↪ response:
{
    "TxnId": 45464,
    "Label": "logstash_log_db_doris_log_20240708_223532_539_6c20a0d1-dcab-4b8e-9bc0-76b46a929bd1
        ↪ ",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 452,
    "NumberLoadedRows": 452,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 277230,
    "LoadTimeMs": 1797,
    "BeginTxnTimeMs": 0,
```

```
    "StreamLoadPutTimeMs": 18,
    "ReadDataTimeMs": 9,
    "WriteDataTimeMs": 1758,
    "CommitAndPublishTimeMs": 18
}

#### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
[2024-07-08T22:35:38,285][INFO ][logstash.outputs.doris   ][main] total 11 MB 18978 ROWS, total
    ↪ speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
```

```
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. Doris 建表

```
CREATE DATABASE log_db;
USE log_db;


CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor.id` BIGINT,
  `actor.login` TEXT,
  `actor.display_login` TEXT,
  `actor.gravatar_id` TEXT,
  `actor.url` TEXT,
  `actor.avatar_url` TEXT,
  `repo.id` BIGINT,
  `repo.name` TEXT,
  `repo.url` TEXT,
  `payload` TEXT,
  `host` TEXT,
  `path` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor.id` (`actor.id`) USING INVERTED,
  INDEX `idx_actor.login` (`actor.login`) USING INVERTED,
  INDEX `idx_repo.id` (`repo.id`) USING INVERTED,
  INDEX `idx_repo.name` (`repo.name`) USING INVERTED,
  INDEX `idx_host` (`host`) USING INVERTED,
  INDEX `idx_path` (`path`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
```

```
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. Logstash 配置

这个配置文件和之前 TEXT 日志采集不同的有下面几点：

1. file input 的 codec 参数是 json，Logstash 会将每一行文本当作 JSON 格式解析，解析出来的字段用于后续处理
2. 没有用 filter plugin，因为不需要额外的处理转换

```
input {
    file {
        path => "/tmp/github_events/2024-04-01-23.json"
        codec => json
    }
}

output {
    doris {
        http_hosts => ["http://fe1:8630", "http://fe2:8630", "http://fe3:8630"]
        user => "root"
        password => ""
        db => "log_db"
        table => "github_events"
        headers => {
          "format" => "json"
          "read_json_by_line" => "true"
          "load_to_single_tablet" => "true"
        }
        mapping => {
          "created_at" => "%{created_at}"
          "id" => "%{id}"
          "type" => "%{type}"
          "public" => "%{public}"
          "actor.id" => "%{[actor][id]}"
          "actor.login" => "%{[actor][login]}"
```

```
          "actor.display_login" => "%{[actor][display_login]}"
          "actor.gravatar_id" => "%{[actor][gravatar_id]}"
          "actor.url" => "%{[actor][url]}"
          "actor.avatar_url" => "%{[actor][avatar_url]}"
          "repo.id" => "%{[repo][id]}"
          "repo.name" => "%{[repo][name]}"
          "repo.url" => "%{[repo][url]}"
          "payload" => "%{[payload]}"
          "host" => "%{[host][name]}"
          "path" => "%{[log][file][path]}"
        }
        log_request => true
    }
}
```

4. 运行 Logstash

```
${LOGSTASH_HOME}/bin/logstash -f logstash_github_events.conf
```

2.15.3.2 Filebeat

Beats 是一个数据采集 Agent，它支持自定义输出插件将数据写入存储系统，Beats Doris output plugin 是输出到 Doris 的插件。

Beats Doris output plugin 支持 Filebeat, Metricbeat, Packetbeat, Winlogbeat, Auditbeat, Heartbeat 。

Beats Doris output plugin 调用Doris Stream Load HTTP 接口将数据实时写入 Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Beats Doris output plugin 主要有三个步骤：1. 下载或编译包含 Doris output plugin 的 Beats 二进制程序 2. 配置 Beats 输出地址和其他参数 3. 启动 Beats 将数据实时写入 Doris

2.15.3.2.1 安装

从官网下载

https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/filebeat-doris-2.1.1

从源码编译

在 extension/beats/ 目录下执行

```
cd doris/extension/beats

go build -o filebeat-doris filebeat/filebeat.go
go build -o metricbeat-doris metricbeat/metricbeat.go
go build -o winlogbeat-doris winlogbeat/winlogbeat.go
go build -o packetbeat-doris packetbeat/packetbeat.go
go build -o auditbeat-doris auditbeat/auditbeat.go
go build -o heartbeat-doris heartbeat/heartbeat.go
```

2.15.3.2.2 参数配置

Beats Doris output plugin 的配置如下：

配置	说明
`http_hosts`	Stream Load HTTP 地址，格式是字符串数组，可以有一个或者多个元素，每个元素是 host:port。例如：[“http://fe1:8030”, “http://fe2:8030”]
`user`	Doris 用户名，该用户需要有 doris 对应库表的导入权限
`password`	Doris 用户的密码
`database`	要写入的 Doris 库名

配置	说明
table ↪	要写入的 Doris 表名
label ↪ _ ↪ prefix ↪	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}{db}{table}{yyyy，默认值是 beats
headers ↪	Doris Stream Load 的 headers 参数，语法格式为 YAML map

配置	说明
codec ↪ _ ↪ format ↪ _ ↪ string ↪	输出到 Doris Stream Load 的 format string，%{[a][b]} 代表输入中的 a.b 字段，参考后续章节的使用示例
bulk ↪ _ ↪ max ↪ _ ↪ size ↪	Doris Stream Load 的 batch size，默认为 100000

配置	说明
max_ ↪ retries ↪	Doris Stream Load 请求失败重试次数，默认为 -1 无限重试保证数据可靠性
log_ ↪ request ↪	日志中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 true

配置	说明
log_ ↪ progress ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

2.15.3.2.3 使用示例

TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. 配置

filebeat 日志采集的配置文件如 filebeat_doris_log.yml 是 YAML 格式，主要由 4 部分组成，分别对应 ETL 的各个部分：1. input 负责读取原始数据 2. processor 负责做数据转换 3. queue.mem 配置 filebeat 内部的缓冲队列 4. output 负责将数据输出

```
#### 1. input 负责读取原始数据
#### type: log 是一个 log input plugin，可以配置读取的日志文件路径，通过 multiline
    ↪ 功能将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。log input
    ↪ 会将日志内容保存在 message 字段中，另外还有一些元数据字段比如 agent.host，log.file.path。
filebeat.inputs:
```

```
- type: log
  enabled: true
  paths:
    - /path/to/your/log
  # multiline 可以将跨行的日志（比如Java stacktrace）拼接起来
  multiline:
    type: pattern
    # 效果：以 yyyy-mm-dd HH:MM:SS 开头的行认为是一条新的日志，其他都拼接到上一条日志
    pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}'
    negate: true
    match: after
    skip_newline: true

#### 2. processors 部分负责数据转换
processors:
#### 用 js script 插件将日志中的 `\t` 替换成空格，避免JSON解析报错
- script:
    lang: javascript
    source: >
        function process(event) {
            var msg = event.Get("message");
            msg = msg.replace(/`\t`/g, "  ");
            event.Put("message", msg);
        }
#### 用 dissect 插件做简单的日志解析
- dissect:
    # 2024-06-08 18:26:25,481 INFO (report-thread|199) [ReportHandler.cpuReport():617] begin to
        ↪ handle
    tokenizer: "%{day} %{time} %{log_level} (%{thread}) [%{position}] %{content}"
    target_prefix: ""
    ignore_failure: true
    overwrite_keys: true

#### 3. 内部的缓冲队列总条数，flush batch 条数，flush 时间间隔
queue.mem:
  events: 1000000
  flush.min_events: 100000
  flush.timeout: 10s

#### 4. output 部分负责数据输出
#### doris output 将数据输出到 Doris，使用的是 Stream Load HTTP 接口。通过 headers 参数指定了
    ↪ Stream Load 的数据格式为 JSON，通过 codec_format_string 参数用类似 printf
    ↪ 的方式格式化输出到 Doris 的数据。比如下面的例子基于 filebeat 内部的字段 format 出一个
    ↪ JSON，这些字段可以是 filebeat 内置字段如 agent.hostname，也可以是 processor 比如 dissect
    ↪ 生产的字段如 day，通过 %{[a][b]} 的方式引用，，Stream Load 会自动将 JSON 字段写入对应的
```

```
    ↪ Doris 表的字段。
output.doris:
  fenodes: [ "http://fehost1:http_port", "http://fehost2:http_port", "http://fehost3:http_port" ]
  user: "your_username"
  password: "your_password"
  database: "your_db"
  table: "your_table"
  # output string format
  ## %{[agent][hostname]} %{[log][file][path]} 是filebeat自带的metadata
  ## 常用的 filebeat metadata 还是有采集时间戳 %{[@timestamp]}
  ## %{[day]} %{[time]} 是上面 dissect 解析得到字段
  codec_format_string: '{"ts": "%{[day]} %{[time]}", "host": "%{[agent][hostname]}", "path": "%{[
      ↪ log][file][path]}", "message": "%{[message]}"}'
  headers:
    format: "json"
    read_json_by_line: "true"
    load_to_single_tablet: "true"
```

4. 运行 Filebeat

```
./filebeat-doris -f config/filebeat_doris_log.yml

#### log_request 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

doris stream load response:
{
    "TxnId": 45464,
    "Label": "logstash_log_db_doris_log_20240708_223532_539_6c20a0d1-dcab-4b8e-9bc0-76b46a929bd1
        ↪ ",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 452,
    "NumberLoadedRows": 452,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 277230,
    "LoadTimeMs": 1797,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 18,
    "ReadDataTimeMs": 9,
    "WriteDataTimeMs": 1758,
    "CommitAndPublishTimeMs": 18
}
```

```
#### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
total 11 MB 18978 ROWS, total speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. Doris 建表

```
CREATE DATABASE log_db;
USE log_db;


CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor` VARIANT,
  `repo` VARIANT,
  `payload` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor` (`actor`) USING INVERTED,
  INDEX `idx_host` (`repo`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. Filebeat 配置

这个配置文件和之前 TEXT 日志采集不同的有下面几点：

1. 没有用 processors，因为不需要额外的处理转换
2. output 中的 codec_format_string 很简单，直接输出整个 message，也就是原始内容

```
#### input
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /path/to/your/log

#### queue and batch
queue.mem:
  events: 1000000
  flush.min_events: 100000
  flush.timeout: 10s

#### output
output.doris:
  fenodes: [ "http://fehost1:http_port", "http://fehost2:http_port", "http://fehost3:http_port" ]
  user: "your_username"
  password: "your_password"
  database: "your_db"
  table: "your_table"
  # output string format
  ## 直接把原始文件每一行的 message 原样输出，由于 headers 指定了 format: "json"，Stream Load
      ↪ 会自动解析 JSON 字段写入对应的 Doris 表的字段。
  codec_format_string: '%{[message]}'
  headers:
    format: "json"
    read_json_by_line: "true"
    load_to_single_tablet: "true"
```

4. 运行 Filebeat

```
./filebeat-doris -f config/filebeat_github_events.yml
```

2.15.3.3 OpenTelemetry

2.15.3.3.1 介绍

OpenTelemetry（简称 OTel），是一个中立厂商的开源可观测性框架，用于监测、生成、收集和导出日志、调用链追踪和指标等可观测性数据。OpenTelemetry 定义了一套可观测性的标准和协议，被可观测性社区和厂商广泛采纳，逐渐成为可观测性领域的事实标准。

OpenTelemetry 本身实现了框架和可观测性数据采集 SDK，使应用程序和系统易于进行监测，无论使用何种编程语言、基础设施和运行时环境，而可观测性存储后端和可视化前端则留给其他工具来处理。Doris 作为一个存储后端与 OpenTelemetry 集成，提供高性能、低成本、统一的可观测性数据存储和分析能力，整体架构如下。

2.15.3.3.2 安装

从 OpenTelemetry 官方 Release 页面下载 OpenTelemetry Collector Contrib 安装包，例如 https://github.com/open-telemetry/opentelemetry-collector-releases/releases/download/v0.132.2/otelcol-contrib_0.132.2_linux_amd64.tar.gz，安装包解压缩得到 otelcol-contrib 可执行文件。

2.15.3.3.3 参数配置

OpenTelemetry Collector Doris Exporter 的核心配置如下：

配置	说明
`endpoint`	Doris FE HTTP 地址，格式是 host:port，例如：“127.0.0.1:8030”
`mysql_endpoint`	Doris FE MySQL 地址，格式是 host:port，例如：“127.0.0.1:9030”
`username`	Doris 用户名，该用户需要有对应库表的写入权限
`password`	Doris 用户的密码

配置	说明
database ↪	要写入的 Doris 库名
table ↪ . ↪ logs ↪	logs 数据写入的 Doris 表名，默认值是 otel_logs
table ↪ . ↪ traces ↪	traces 数据写入的 Doris 表名，默认值是 otel_traces
table ↪ . ↪ metrics ↪	metrics 数据写入的 Doris 表名，默认值是 otel_metrics
create ↪ _ ↪ schema ↪	是否自动创建 Doris 库表，默认值是 true

配置	说明
history ↪ _ ↪ days ↪	自动创建的 Doris 表的历史数据保留天数，默认值是 0 表示永久保留
create ↪ _ ↪ history ↪ _ ↪ days ↪	自动创建的 Doris 表的初始分区天数，默认值是 0 表示不创建分区

配置	说明
label_prefix	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}_{db}_{table}_{yyyy，默认值是 open_telemetry
headers	Doris Stream Load 的 headers 参数，语法格式为 YAML map

配置	说明
log_ ↪ progress ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

更多配置请参考 https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/exporter/dorisexporter 。

2.15.3.3.4 使用示例

TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. OpenTelemetry 配置

日志采集的配置文件如 opentelemetry_java_log.yml 主要由 3 部分组成，分别对应 ETL 的各个部分：1. receivers 负责读取原始数据 2. processors 负责做数据转换 3. exporters 负责将数据输出

```
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET
/filter/27|13%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,27|%DA%A9%D9%85%D8%AA%D8%B1%20%
    ↪ D8%A7%D8%B2%205%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,p53 HTTP/1.1" 200
    ↪ 30577 "-" "Mozilla/5.0 (compatible; AhrefsBot/6.1; +http://ahrefs.com/robot/)" "-"
```

```
#### 1. receivers 负责读取原始数据
#### filelog 是一个本地 receiver，可以配置读取本地文件系统的日志文件路径，通过 multiline
    ↪ 将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。
receivers:
  filelog:
    include:
      - /path/to/fe.log
    start_at: beginning
    multiline:
      line_start_pattern: '^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3}'  #
          ↪ 匹配时间戳作为新日志开始
    operators:
      # 解析日志
      - type: regex_parser
        regex: '^(?P<time>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3}) (?P<severity>INFO|WARN|ERROR
            ↪ ) (?P<message>.*)'
        timestamp:
          parse_from: attributes.time
          layout: '%Y-%m-%d %H:%M:%S,%f'
        severity:
          parse_from: attributes.severity
          trace: TRACE
          debug: DEBUG
          info: INFO
          warn: WARN
          error: ERROR
          fatal: FATAL

#### 2. processors 负责做数据转换
#### 这里使用了简单的 batch processor，将数据攒成批次发送。
processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

#### 3. exporters 负责将数据输出
#### doris exporter 将数据输出到 Doris，使用的是 Stream Load HTTP 接口，默认使用 Stream Load
    ↪ 的数据格式为 JSON，Stream Load 会自动解析 JSON 字段写入对应的 Doris 表的字段。
```

```
exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      logs: otel_logs
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    logs:
      receivers: [filelog]
      processors: [batch]
      exporters: [doris]
```

3. 运行 OpenTelemetry

```
./otelcol-contrib --config config/opentelemetry_java_log.yml

#### log_response 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

2025-08-18T00:33:22.543+0800    info    dorisexporter@v0.132.0/exporter_logs.go:181 log response:
{
    "TxnId": 52,
    "Label": "open_telemetry_otel_otel_logs_20250818003321_498bb8ec-040c-4982-9eb4-452b15129782",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
```

```
    "Message": "OK",
    "NumberTotalRows": 50355,
    "NumberLoadedRows": 50355,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 31130235,
    "LoadTimeMs": 680,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 3,
    "ReadDataTimeMs": 106,
    "WriteDataTimeMs": 653,
    "ReceiveDataTimeMs": 11,
    "CommitAndPublishTimeMs": 23
}

#### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
2025-08-18T00:05:00.017+0800    info    dorisexporter@v0.132.0/progress_reporter.go:63  [LOG]
    ↪ total 11 MB 18978 ROWS, total speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
```

```
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. OpenTelemetry 配置

这个配置文件和之前 TEXT 日志采集不同的是 filelog 的 type 参数是 json_parser，它会将每一行文本当作 JSON 格式解析，解析出来的字段用于后续处理。

```
receivers:
  filelog:
    include:
      - /path/to/2024-01-01-15.json
    start_at: beginning
    operators:
      - type: json_parser
        timestamp:
          parse_from: attributes.created_at
          layout: '%Y-%m-%dT%H:%M:%SZ'

processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      logs: otel_logs
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    history_days: 10
```

```
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    logs:
      receivers: [filelog]
      processors: [batch]
      exporters: [doris]
```

3. 运行 OpenTelemetry

```
./otelcol-contrib --config config/opentelemetry_json_log.yml
```

Trace 采集示例

1. OpenTelemetry 配置

创建 otel_trace.yml 配置文件如下

```
receivers:
  otlp: # otlp 协议，接收 OpenTelemetry Java Agent 发送的数据
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

exporters:
  doris:
```

```
    endpoint: http://localhost:8030 # FE HTTP 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      traces: doris_table_name
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [doris]
```

2. 运行 OpenTelemetry

```
  ./otelcol-contrib --config otel_trace.yaml
```

3. 应用侧接入 OpenTelemetry SDK

这里我们使用一个 Spring Boot 示例应用接入 OpenTelemetry Java SDK，示例应用来自官方 demo，对路径 “/” 返回简单的 “Hello World!” 字符串。下载 OpenTelemetry Java Agent，使用 Java Agent 的优势在于无需对现有的应用做任何的修改。其他语言及其他接入方式详见 OpenTelemetry 官网：Language APIs & SDKs 或 Zero-code Instrumentation。

在启动应用之前只需要添加几个环境变量，无需修改任何代码。

```
export JAVA_TOOL_OPTIONS="${JAVA_TOOL_OPTIONS} -javaagent:/your/path/to/opentelemetry-javaagent.
    ↪ jar" # OpenTelemetry Java Agent 的路径
export OTEL_JAVAAGENT_LOGGING="none" # 禁用 otel log，防止干扰服务本身的日志
export OTEL_SERVICE_NAME="myproject"
```

```
export OTEL_TRACES_EXPORTER="otlp" # 使用 otlp 协议发送 trace 数据
export OTEL_EXPORTER_OTLP_ENDPOINT="http://localhost:4317" # OpenTelemetry Collector 的地址

java -jar myproject-0.0.1-SNAPSHOT.jar
```

Trace Collection Example

1. OpenTelemetry Configuration

Create the configuration file `otel_trace.yml` as follows:

```
receivers:
  otlp:  # OTLP protocol to receive data from OpenTelemetry Java Agent
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    send_batch_size: 100000  # Number of traces per batch (recommended batch size: 100MB-1GB)
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030  # FE HTTP address
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      traces: doris_table_name
    create_schema: true  # Auto-create schema (if false, tables must be created manually)
    mysql_endpoint: localhost:9030  # FE MySQL address
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s  # HTTP Stream Load client timeout
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
```

```
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [doris]
```

2. Run OpenTelemetry

```
./otelcol-contrib --config otel_trace.yaml
```

3. Application Integration with OpenTelemetry SDK

Here we demonstrate integration using a Spring Boot example application (from the official demo), which returns “Hello World!” for the “/” endpoint.

1. Download the OpenTelemetry Java Agent.

- Advantage: No code changes required for existing applications.
- For other languages/integration methods, see OpenTelemetry docs:
  - Language APIs & SDKs
  - Zero-code Instrumentation

2. Before starting the application, set these environment variables (no code modification needed):
   “ ‘bash export JAVA_TOOL_OPTIONS=“${JAVA_TOOL_OPTIONS} -javaagent:/your/path/to/opentelemetry-javaagent.jar” # Path to OpenTelemetry Java Agent export OTEL_JAVAAGENT_LOGGING=“none” # Disable OTEL logs to avoid interference with application logs export OTEL_SERVICE_NAME=“myproject” export OTEL_TRACES_EXPORTER=“otlp” # Use OTLP protocol to send trace data export OTEL_EXPORTER_OTLP_ENDPOINT=“http://localhost:4317” # OpenTelemetry Collector address

java -jar myproject-0.0.1-SNAPSHOT.jar

```


---
{
"title": "FluentBit",
"language": "zh-CN"
}
---
```

FluentBit
[Fluent Bit](https://fluentbit.io/) 是一个快速的日志处理器和转发器，
    ↪ 它支持自定义输出插件将数据写入存储系统，Fluent Bit Doris Output Plugin 是输出到 Doris
    ↪ 的插件。

Fluent Bit Doris Output Plugin 调用 [Doris Stream Load](#stream-load) HTTP 接口将数据实时写入
    ↪ Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Fluent Bit Doris Output Plugin 主要有三个步骤：
1. 下载或编译包含 Doris Output Plugin 的 Fluent Bit 二进制程序
2. 配置 Fluent Bit 输出地址和其他参数
3. 启动 Fluent Bit 将数据实时写入 Doris

安装（alpha 版本）

从官网下载

https://apache-doris-releases.oss-accelerate.aliyuncs.com/integrations/fluent-bit-doris-3.1.9

从源码编译

克隆 https://github.com/joker-star-l/fluent-bit 的 dev 分支，在 build/ 目录下执行

cmake -DFLB_RELEASE=ON .. make “‘

编译产物为 build/bin/fluent-bit。

2.15.3.3.5 参数配置

Fluent Bit Doris output plugin 的配置如下：

配置	说明
host	Stream Load HTTP host
port	Stream Load HTTP port

配置	说明
user	Doris 用户名，该用户需要有 doris 对应库表的导入权限
password	Doris 用户的密码
database	要写入的 Doris 库名
table	要写入的 Doris 表名

配置	说明
label ↪ _ ↪ prefix ↪	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}_{timestamp}_{u，默认值是 fluentbit，如果设置为 false 则不会添加 Label
time ↪ _ ↪ key ↪	数据中要添加的时间戳列的名称，默认值是 date，如果设置为 false 则不会添加该列

配置	说明
header	Doris Stream Load 的 header 参数，可以设置多个日志
log_request	中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 true
log_progress_interval	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

配置	说明
retry_limit	Doris Stream Load 请求失败重试次数，默认为 1，如果设置为 false 则不限制重试次数
workers	执行 Doris Stream Load 的 worker 数量，默认为 2

2.15.3.3.6 使用示例

TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 `fe/log/fe.log` 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 `stacktrace` 的异常日志，`stacktrace` 是跨行的，日志采集存储需要把主日志和 `stacktrace` 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
```

```
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. 配置

Fluent Bit 日志采集的配置文件如下，doris_log.conf 用于定义 ETL 的各个部分组件，parsers.conf 用于定义不同的日志解析器。

doris_log.conf:

```
[SERVICE]
    log_level info
    # parsers file
    parsers_file parsers.conf

#### use input tail
[INPUT]
    name tail
    path /path/to/your/log
    # add log file name to the record, key is 'path'
    path_key path
    # set multiline parser
    multiline.parser multiline_java

#### parse log
[FILTER]
    match *
    name parser
    key_name log
    parser fe_log
    reserve_data true

#### add host info
[FILTER]
    name sysinfo
    match *
    # add hostname to the record, key is 'host'
    hostname_key host

#### output to doris
[OUTPUT]
    name doris
    match *
    host fehost
    port feport
    user your_username
    password your_password
    database your_db
    table your_table
    # add 'collect_time' to the record
```

```
    time_key collect_time
    # 'collect_time' is timestamp, change it to datatime
    header columns collect_time=from_unixtime(collect_time)
    log_request true
    log_progress_interval 10
```

parsers.conf:

```
[MULTILINE_PARSER]
    name          multiline_java
    type          regex
    flush_timeout 1000
    # Regex rules for multiline parsing
    # ---------------------------------
    #
    # configuration hints:
    #
    #  - first state always has the name: start_state
    #  - every field in the rule must be inside double quotes
    #
    # rules   |   state name   | regex pattern | next state name
    # --------|----------------|---------------|-----------------
    rule         "start_state"   "/(^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2})(.*)/"
        ↪   "cont"
    rule         "cont"          "/(^(?![0-9]{4}-[0-9]{2}-[0-9]{2}))(.*)/"     "cont"


[PARSER]
    name        fe_log
    format      regex
    # parse and add 'log_time', 'level', 'thread', 'position', 'message' to the record
    regex       ^(?<log_time>[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2},[0-9]{3}) (?<
        ↪ level>[^ ]+) \((?<thread>[^\)]+)\) \[(?<position>[^\]]+)\] (?<message>(`\n`|.)*)`\n`$
```

4. 运行 Fluent Bit

```
fluent-bit -c doris_log.conf

#### log stream load response

[2024/10/31 18:39:55] [ info] [output:doris:doris.1] 127.0.0.1:8040, HTTP status=200
{
    "TxnId": 32155,
    "Label": "fluentbit_1730371195_91cca1aa-c15f-45d2-b503-fe7d2e839c2a",
    "Comment": "",
    "TwoPhaseCommit": "false",
```

```
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 1,
    "NumberLoadedRows": 1,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 836,
    "LoadTimeMs": 298,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 3,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 268,
    "CommitAndPublishTimeMs": 25
}

#### log speed info

[2024/10/31 18:40:13] [ info] [output:doris:doris.1] total 0 MB 2 ROWS, total speed 0 MB/s 0 R/s,
    ↪  last 10 seconds speed 0 MB/s 0 R/s
```

JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
```

```
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. 建表

```
CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor` VARIANT,
  `repo` VARIANT,
  `payload` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor` (`actor`) USING INVERTED,
  INDEX `idx_host` (`repo`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
```

```
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. 配置

和之前 TEXT 日志采集相比，该配置文件没有使用 FILTER，因为不需要额外的处理转换。

github_events.conf:

```
[SERVICE]
    log_level info
    parsers_file github_parsers.conf

[INPUT]
    name tail
    parser github
    path /path/to/your/log

[OUTPUT]
    name doris
    match *
    host fehost
    port feport
    user your_username
    password your_password
    database your_db
    table your_table
    time_key false
    log_request true
    log_progress_interval 10
```

github_parsers.conf:

```
[PARSER]
    name github
    format json
```

4. 运行 Fluent Bit

```
fluent-bit -c github_events.conf
```

2.16 存算分离

2.16.1 概览

本文介绍存算分离与存算一体两种架构的区别、优势和适用场景，为用户的选择与使用提供参考。后文将详细说明如何部署并使用 Apache Doris 存算分离模式。如需部署存算一体模式，请参考集群部署。

2.16.1.1 存算一体 VS 存算分离

Doris 的整体架构由两类进程组成：Frontend (FE) 和 Backend (BE)。其中 FE 主要负责用户请求的接入、查询解析规划、元数据的管理、节点管理相关工作；BE 主要负责数据存储、查询计划的执行。(更多信息)

2.16.1.1.1 存算一体

在存算一体架构下，BE 节点上存储与计算紧密耦合，数据主要存储在 BE 节点上，多 BE 节点采用 MPP 分布式计算架构。

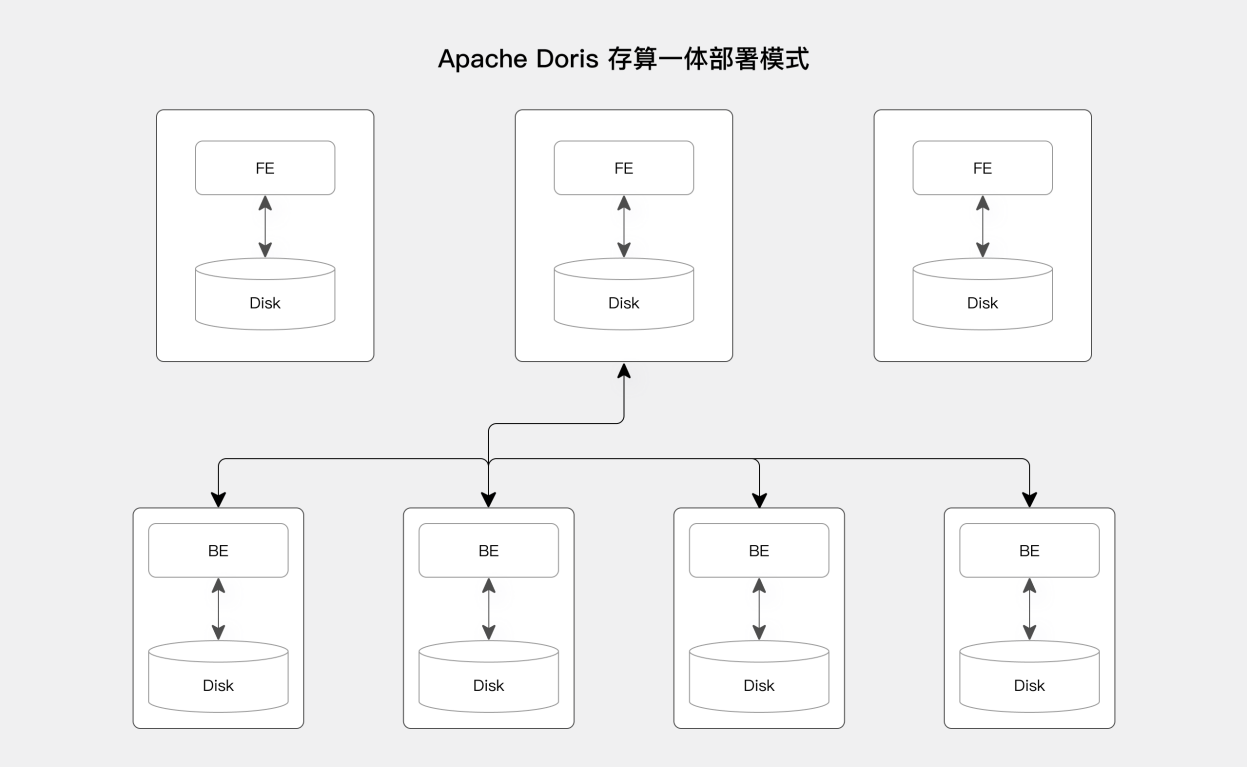


图 100: compute-storage-coupled

2.16.1.1.2 存算分离

BE 节点不再存储主数据，而是将共享存储层作为统一的数据主存储空间。同时，为了应对底层对象存储系统性能不佳和网络传输带来的性能下降，Doris 引入计算节点本地高速缓存。

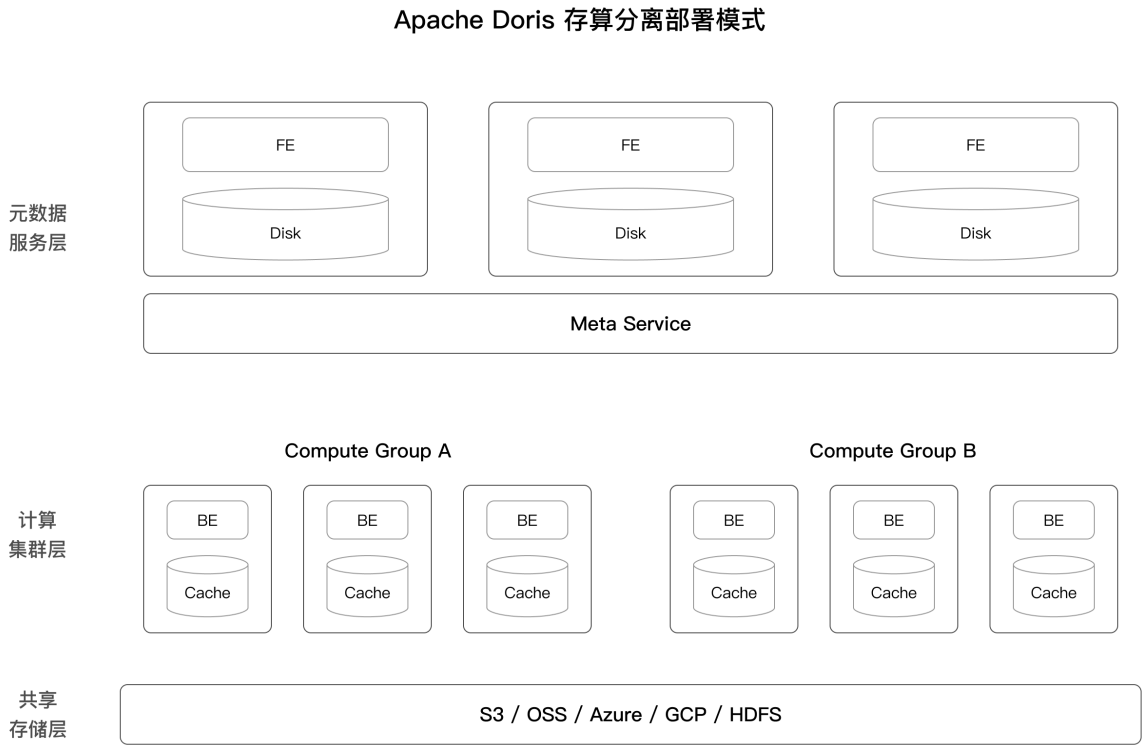


图 101: compute-storage-decoupled

元数据层：

FE 主要存放库表元数据，Job 以及权限等 MySQL 协议依赖的信息。

Meta Service 是 Doris 存算分离元数据服务，主要负责处理导入事务，Tablet Meta，Rowset Meta 以及集群资源管理。这是一个可以横向扩展的无状态服务。

计算层：

存算分离模式下的 BE 是无状态的 Doris BE 节点，BE 上会缓存一部分 Tablet 元数据和数据以提高查询性能。

计算集群（Compute Cluster）是无状态的 BE 节点组成的计算资源集合，多个计算集群共享一份数据，计算集群可以随时弹性加减节点。

> 备注
>
> 存算分离文档中的“计算集群”概念有别于 Doris【集群部署】以及后文【创建集群】中的“集群”概念。存算分离文档中提及的“计算集群”特指在 Doris 存算分离模式下，由无状态 BE 节点组成的计算资源集合，而非【集群部署】和【创建集群】中所指的由多个 Apache Doris 节点组成的完整分布式系统。

共享存储层：

共享存储主要存放数据文件，包括 Segment 文件、反向索引的索引文件等。

2.16.1.2 如何选择

2.16.1.2.1 存算一体的优点

- 部署简易：Apache Doris 不需要依赖类似外部共享文件系统或者对象存储，仅依赖物理服务器部署 FE 和 BE 两个进程即可完成集群的搭建，可以从一个节点扩展到数百个节点，同时也增强了系统的稳定性。
- 性能优异：Apache Doris 执行计算时，计算节点可直接访问本地存储数据，充分利用机器的 IO、减少不必要的网络开销、获得更极致的查询性能。

2.16.1.2.2 存算一体的适用场景

- 简单使用/快速试用 Doris，或在开发和测试环境中使用；
- 不具备可靠的共享存储，如 HDFS、Ceph、对象存储等；
- 业务线独立维护 Apache Doris，无专职 DBA 来维护 Doris 集群；
- 不需极致弹性扩缩容，不需 K8s 容器化，不需运行在公有云或者私有云上。

2.16.1.2.3 存算分离的优点

- 弹性的计算资源：不同时间点使用不同规模的计算资源服务业务请求，按需使用计算资源，节约成本。
- 负载（完全）隔离：不同业务之间可在共享数据的基础上隔离计算资源，兼具稳定性和高效率。
- 低存储成本：可以使用更低成本的对象存储，HDFS 等低成本存储。

2.16.1.2.4 存算分离的适用场景

- 已使用公有云服务
- 具备可靠的共享存储系统，比如 HDFS、Ceph、对象存储等
- 需要极致的弹性扩缩容，需要 K8S 容器化，需要运行在私有云上
- 有专职团队维护整个公司的数据仓库平台

2.16.2 Doris 存算分离模式部署准备

2.16.2.1 1. 概述

本文档介绍了 Apache Doris 存算分离模式的部署准备工作。存算分离架构旨在提高系统的可扩展性和性能，适用于大规模数据处理场景。

2.16.2.2 2. 架构组件

Doris 存算分离架构包含三个主要模块：

1. Frontend (FE)：处理用户请求和管理元数据。
2. Backend (BE)：无状态计算节点，执行查询任务。
3. Meta Service (MS)：管理元数据操作和数据回收。

2.16.2.3 3. 系统要求

2.16.2.3.1 3.1 硬件要求

- 最小配置：3 台服务器
- 推荐配置：5 台或更多服务器

2.16.2.3.2 3.2 软件依赖

- FoundationDB (FDB) 7.1.38 或更高版本
- OpenJDK 17

2.16.2.4 4. 部署规划

2.16.2.4.1 4.1 测试环境部署

单机部署所有模块，不适用于生产环境。

2.16.2.4.2 4.2 生产部署

- 3 台或更多机器部署 FDB
- 3 台或更多机器部署 FE 和 Meta Service
- 3 台或更多机器部署 BE

机器配置高时，可以考虑 FDB、 FE 和 Meta Service 混布，但是磁盘不要混用。

2.16.2.5 5. 安装步骤

2.16.2.5.1 5.1 安装 FoundationDB

本节提供了脚本 `fdb_vars.sh` 和 `fdb_ctl.sh` 配置、 部署和启动 FDB（ FoundationDB ）服务的分步指南。您可以下载 doris tools 并从 `fdb` 目录获取 `fdb_vars.sh` 和 `fdb_ctl.sh`。

> Doris 当前默认依赖的 FDB 版本为 7.1.x, 如果已经提前自行安装了 FDB, 请确认版本为 7.1.x 系列版本, 否则 Meta Service 启动会失败.

5.1.1 机器要求

通常，至少需要 3 台配备 SSD 的机器来形成具有双数据副本并允许单机故障的 FoundationDB 集群。如果没有 SSD, 也至少需要使用标准云盘或者本地盘以及标准的 Posix 文件系统作为数据的存储, 否则可能 FoundationDB 不能正常工作, 比如不能 JuiceFS 等作为 FoundationDB 的存储.

如果仅用于开发/测试目的，单台机器就足够了。

5.1.2 fdb_vars.sh 配置

必需的自定义设置

参数	描述	类型	示例	注意事项
DATA_ ↪ DIRS	指定 FoundationDB 存储的数据目录	以逗号分隔的绝对路径列表	/mnt/ ↪ foundationdb ↪ / ↪ data1 ↪ ,/ ↪ mnt/ ↪ foundationdb ↪ / ↪ data2 ↪ ,/ ↪ mnt/ ↪ foundationdb ↪ / ↪ data3 ↪	- 运行脚本前确保目录已创建 - 生产环境建议使用 SSD 和独立目录
FDB_ ↪ CLUSTER ↪ _IPS	定义集群 IP	字符串（以逗号分隔的 IP 地址）	172.200.0.2,172.200.0.3,172.200.0.4 ↪	- 生产集群至少应有 3 个 IP 地址 - 第一个 IP 地址将用作协调器 - 为高可用性，将机器放置在不同机架上
FDB_ ↪ HOME	定义 FoundationDB 主目录	绝对路径	/ ↪ fdbhome ↪	- 默认路径为 /fdbhome- 确保此路径是绝对路径
FDB_ ↪ CLUSTER ↪ _ID	定义集群 ID	字符串	SAQESzbh ↪	- 每个集群的 ID 必须唯一 - 可使用 mktemp -u ↪ XXXXXXXX 生成

参数	描述	类型	示例	注意事项
FDB_ ↪ CLUSTER ↪ _ ↪ DESC	定义 FDB 集群的描述	字符串	dorisfdb ↪	- 建议更改为对部署有意义的内容

可选的自定义设置

参数	描述	类型	示例	注意事项
MEMORY_ ↪ LIMIT ↪ _GB	定义 FDB 进程的内存限制，单位为 GB	整数	MEMORY_ ↪ LIMIT ↪ _GB ↪ =16	根据可用内存资源和 FDB 进程的要求调整此值
CPU_ ↪ CORES ↪ _ ↪ LIMIT ↪	定义 FDB 进程的 CPU 核心限制	整数	CPU_ ↪ CORES ↪ _ ↪ LIMIT ↪ =8	根据可用的 CPU 核心数量和 FDB 进程的要求设置此值

5.1.3 部署 FDB 集群

使用 `fdb_vars.sh` 配置环境后，您可以在每个节点上使用 `fdb_ctl.sh` 脚本部署 FDB 集群。

```
./fdb_ctl.sh deploy
```

此命令启动 FDB 集群的部署过程。

5.1.4 启动 FDB 服务

FDB 集群部署完成后，您可以在每个节点上使用 `fdb_ctl.sh` 脚本启动 FDB 服务。

```
./fdb_ctl.sh start
```

此命令启动 FDB 服务，使集群工作并获取 FDB 集群连接字符串，后续可以用于配置 MetaService。

2.16.2.5.2 5.2 安装 OpenJDK 17

1. 下载 OpenJDK 17
2. 解压并设置环境变量 JAVA_HOME.

2.16.2.6 6. 后续步骤

完成上述准备工作后，请参考以下文档继续部署：

1. 部署

2. 管理 Compute Group
3. 管理 Storage Vault

2.16.2.7 7. 注意事项

- 确保所有节点的时间同步
- 定期备份 FoundationDB 数据
- 根据实际负载调整 FoundationDB 和 Doris 的配置参数
- 使用标准云盘或者本地盘以及标准的 Posix 文件系统作为数据的存储, 否则 FoundationDB 可能不能正常工作
  - 比如不能 JuiceFS 等作为 FoundationDB 的存储

2.16.2.8 8. 参考资料

- FoundationDB 官方文档
- Apache Doris 官方网站

2.16.3 编译部署

2.16.3.1 1. 概述

本文档详细介绍了 Doris 存算分离模式下的编译和部署流程，重点说明了与存算一体模式的区别，特别是新增 Meta Service (MS) 模块的编译、配置和管理。

2.16.3.2 2. 获取二进制

2.16.3.2.1 2.1 直接下载

已编译好的二进制文件（包含所有 Doris 模块）可从 Doris 下载页面 获取（选择 3.0.2 或更高版本）。

2.16.3.2.2 2.2 编译产出 (可选)

使用代码库自带的 `build.sh` 脚本进行编译。新增的 MS 模块通过 `--cloud` 参数编译。

```
sh build.sh --fe --be --cloud
```

编译完成后，在 `output` 目录下会新增 `ms` 目录：

```
output
├── be
├── fe
└── ms
    ├── bin
    ├── conf
    └── lib
```

2.16.3.3 3. Meta Service 部署

2.16.3.3.1 3.1 配置

在 `./conf/doris_cloud.conf` 文件中，主要需要修改以下两个参数：

1. `brpc_listen_port`：Meta Service 的监听端口，默认为 5000。
2. `fdb_cluster`：FoundationDB 集群的连接信息，部署 FoundationDB 时可以获取。(如果使用 Doris 提供的 fdb_ctl.sh 部署的话，可在 `$FDB_HOME/conf/fdb.cluster` 文件里获取该值)。

示例配置：

```
brpc_listen_port = 5000
fdb_cluster = xxx:yyy@127.0.0.1:4500
```

注意：`fdb_cluster` 的值应与 FoundationDB 部署机器上的 `/etc/foundationdb/fdb.cluster` 文件内容一致 (如果使用 Doris 提供的 fdb_ctl.sh 部署的话，可在 `$FDB_HOME/conf/fdb.cluster` 文件里获取该值)。

示例，文件的最后一行就是要填到 doris_cloud.conf 里 fdb_cluster 字段的值

```
cat /etc/foundationdb/fdb.cluster



### This file is auto-generated, it is not to be edited by hand.
cloud_ssb:A83c8Y1S3ZbqHLL4P4HHNTTw0A83CuHj@127.0.0.1:4500
```

2.16.3.3.2 3.2 启动与停止

环境要求

确保已正确设置 `JAVA_HOME` 环境变量，指向 OpenJDK 17，进入 `ms` 目录。

启动命令

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --daemon
```

```
LIBHDFS3_CONF=
starts doris_cloud with args: --meta-service
wait and check doris_cloud start successfully
successfully started brpc listening on port=5000 time_elapsed_ms=11
doris_cloud start successfully
```

启动脚本返回值为 0 表示启动成功，否则启动失败。

> 在 3.0.4 中，启动脚本会输出更多信息：

```
2024-12-26 15:31:53 start with args: --meta-service
wait and check MetaService and Recycler start successfully
process working directory: "/mnt/disk1/doris/ms"
pid=1666015 written to file=./bin/doris_cloud.pid
version:{doris-3.0.4-release} code_version:{commit=
    ↪ fd44740fadabebfedb5da201d7ce427a5dd47c44 time=2025-01-16 18:53:00 +0800}
    ↪ build_info: ...

MetaService has been started successfully
successfully started service listening on port=5000 time_elapsed_ms=19
```

停止命令

```
bin/stop.sh
```

生产环境中请确保至少有 3 个 Meta Service 节点。

2.16.3.4 4. 数据回收功能独立部署（可选）

> Meta Service 本身具备了元数据管理和回收功能，这两个功能可以独立部署，如果你想独立部署，可以参考这一节。

准备工作

1. 创建新的工作目录（如 `recycler`）。
2. 复制 `ms` 目录内容到新目录：

```
cp -r ms recycler
```

配置

在新目录的配置文件中修改 BRPC 监听端口 `brpc_listen_port` 和 `fdb_cluster` 的值。

启动数据回收功能

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --recycler --daemon
```

启动仅元数据操作功能

```
export JAVA_HOME=${path_to_jdk_17}
bin/start.sh --meta-service --daemon
```

2.16.3.5 5. FE 和 BE 的启动流程

本节详细说明了在存算分离架构下启动 FE（Frontend）和 BE（Backend）的步骤。

2.16.3.5.1 5.1 启动顺序

1. 以 MASTER 角色启动实例的第一个 FE
2. 向实例中添加其他 FE 和 BE
3. 添加第一个 Storage Vault

2.16.3.5.2 5.2 启动 MASTER 角色的 FE

5.2.1 配置 fe.conf

在 `fe.conf` 文件中，需要配置以下关键参数：

1. `deploy_mode`

- 描述：指定 doris 启动模式
- 格式：cloud 表示存算分离模式，其它存算一体模式
- 示例：`cloud`

2. `cluster_id`

- 描述：存算分离架构下集群的唯一标识符，不同的集群必须设置不同的 cluster_id
- 格式：int 类型
- 示例：可以使用如下 shell 脚本生成一个随机 id 使用。

```
echo $(($((RANDOM << 15)) | $RANDOM))
```

> **不同的集群必须设置不同的 `cluster_id`**

3. `meta_service_endpoint`

- 描述：Meta Service 的地址和端口
- 格式：`IP地址:端口号`
- 示例：`127.0.0.1:5000`, 可以用逗号分割配置多个 meta service。

5.2.2 启动 FE

启动命令示例：

```
bin/start_fe.sh --daemon
```

第一个 FE 进程初始化集群并以 FOLLOWER 角色工作。使用 mysql 客户端连接 FE 使用 `show frontends` 确认刚才启动的 FE 是 master。

2.16.3.5.3 5.3 添加其他 FE 节点

其他节点同样根据上述步骤修改配置文件并启动，使用 mysql 客户端连接 Master 角色的 FE，并用以下 SQL 命令添加额外的 FE 节点：

```
ALTER SYSTEM ADD FOLLOWER "host:port";
```

将 host:port 替换为 FE 节点的实际地址和编辑日志端口。更多信息请参见ADD FOLLOWER 和ADD OBSERVER。

生产环境中请确保在 FOLLOWER 角色中的前端 (FE) 节点总数，包括第一个 FE，保持为奇数。一般来说，三个 FOLLOWER 就足够了。OBSERVER 角色的前端节点可以是任意数量。

2.16.3.5.4 5.4 添加 BE 节点

要向集群添加 Backend 节点，请对每个 Backend 执行以下步骤：

5.4.1 配置 be.conf

在 be.conf 文件中，需要配置以下关键参数：

1. deploy_mode

- 描述：指定 doris 启动模式
- 格式：cloud 表示存算分离模式，其它存算一体模式
- 示例：cloud

2. file_cache_path

- 描述：用于文件缓存的磁盘路径和其他参数，以数组形式表示，每个磁盘一项。path 指定磁盘路径，total_size 限制缓存的大小；-1 或 0 将使用整个磁盘空间。
- 格式:[{“path”:“/path/to/file_cache”,“total_size”:21474836480},{“path”:“/path/to/file_cache2”,“total_size”:21474836480}]
- 示例:[{“path”:“/path/to/file_cache”,“total_size”:21474836480},{“path”:“/path/to/file_cache2”,“total_size”:21474836480}]
- 默认：[{“path”:“${DORIS_HOME}/file_cache”}]

5.4.1 启动和添加 BE

1. 启动 Backend:

使用以下命令启动 Backend:

```
bin/start_be.sh --daemon
```

2. 将 Backend 添加到集群:

使用 MySQL 客户端连接到任意 Frontend，并执行:

```
ALTER SYSTEM ADD BACKEND "<ip>:<heartbeat_service_port>" [PROTERTIES propertires];
```

将 <ip> 替换为新 Backend 的 IP 地址，将 <heartbeat_service_port> 替换为其配置的心跳服务端口（默认为 9050）。

可以通过 PROPERTIES 设置 BE 所在的计算组。

更详细的用法请参考ADD BACKEND 和REMOVE BACKEND。

3. 验证 Backend 状态：

检查 Backend 日志文件（be.log）以确保它已成功启动并加入集群。

您还可以使用以下 SQL 命令检查 Backend 状态：

```
SHOW BACKENDS;
```

这将显示集群中所有 Backend 及其当前状态。

2.16.3.6 6. 创建 Storage Vault

Storage Vault 是 Doris 存算分离架构中的重要组件。它们代表了存储数据的共享存储层。您可以使用 HDFS 或兼容 S3 的对象存储创建一个或多个 Storage Vault。可以将一个 Storage Vault 设置为默认 Storage Vault，系统表和未指定 Storage Vault 的表都将存储在这个默认 Storage Vault 中。默认 Storage Vault 不能被删除。以下是为您的 Doris 集群创建 Storage Vault 的方法：

2.16.3.6.1 6.1 创建 HDFS Storage Vault

要使用 SQL 创建 Storage Vault，请使用 MySQL 客户端连接到您的 Doris 集群

```
CREATE STORAGE VAULT IF NOT EXISTS hdfs_vault
    PROPERTIES (
    "type"="hdfs",
    "fs.defaultFS"="hdfs://127.0.0.1:8020"
    );
```

2.16.3.6.2 6.2 创建 S3 Storage Vault

要使用兼容 S3 的对象存储创建 Storage Vault，请按照以下步骤操作：

1. 使用 MySQL 客户端连接到您的 Doris 集群。
2. 执行以下 SQL 命令来创建 S3 Storage Vault：

```
CREATE STORAGE VAULT IF NOT EXISTS s3_vault
    PROPERTIES (
    "type"="S3",
    "s3.endpoint"="s3.us-east-1.amazonaws.com",
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk",
```

```
    "s3.region" = "us-east-1",
    "s3.root.path" = "ssb_sf1_p2_s3",
    "s3.bucket" = "doris-build-1308700295",
    "provider" = "S3"
    );
```

要在其他对象存储上创建 Storage Vault，请参考创建 Storage Vault。

2.16.3.6.3 6.3 设置默认 Storage Vault

使用如下 SQL 语句设置一个默认 Storage Vault。

```
SET <storage_vault_name> AS DEFAULT STORAGE VAULT
```

2.16.3.7 7. 注意事项

- 仅元数据操作功能的 Meta Service 进程应作为 FE 和 BE 的 `meta_service_endpoint` 配置目标。
- 数据回收功能进程不应作为 `meta_service_endpoint` 配置目标。

2.16.4 管理 Storage Vault

Storage Vault 是 Doris 在存算分离模式中所使用的远程共享存储，可配置一个或多个 Storage Vault，可将不同表存储在不同 Storage Vault 上。

2.16.4.1 创建 Storage Vault

语法

```
CREATE STORAGE VAULT [IF NOT EXISTS] <vault_name>
PROPERTIES
("key" = "value",...)
```

是用户定义的 Storage Vault 名称，是用户接口用于访问 Storage Vault 的标识。

2.16.4.1.1 创建 HDFS Storage Vault

创建基于 HDFS 的存算分离模式 Doris 集群，需要确保所有的节点 (包括 FE / BE 节点、 Meta Service) 均有权限访问所指定的 HDFS，包括提前完成机器的 Kerberos 授权配置和连通性检查（可在对应的每个节点上使用 Hadoop Client 进行测试）等。

```
CREATE STORAGE VAULT IF NOT EXISTS hdfs_vault_demo
PROPERTIES (
    "type" = "hdfs",                                    -- required
    "fs.defaultFS" = "hdfs://127.0.0.1:8020",           -- required
    "path_prefix" = "big/data",                         -- optional,  一般按照业务名称填写
    "hadoop.username" = "user"                          -- optional
```

```
    "hadoop.security.authentication" = "kerberos"        -- optional
    "hadoop.kerberos.principal" = "hadoop/127.0.0.1@XXX" -- optional
    "hadoop.kerberos.keytab" = "/etc/emr.keytab"         -- optional
);
```

2.16.4.1.2 创建 S3 Storage Vault

```
CREATE STORAGE VAULT IF NOT EXISTS s3_vault_demo
PROPERTIES (
    "type" = "S3",                                  -- required
    "s3.endpoint" = "oss-cn-beijing.aliyuncs.com", -- required
    "s3.region" = "cn-beijing",                     -- required
    "s3.bucket" = "bucket",                         -- required
    "s3.root.path" = "big/data/prefix",             -- required
    "s3.access_key" = "ak",                         -- required
    "s3.secret_key" = "sk",                         -- required
    "provider" = "OSS",                             -- required
    "use_path_style" = "false"                      -- optional
);
```

更多参数说明及示例可见CREATE-STORAGE-VAULT。

注意提供的对象存储路径必须具有 head/get/list/put/multipartUpload/delete 访问权限。

2.16.4.2 查看 Storage Vault

语法

```
SHOW STORAGE VAULTS
```

返回结果包含 4 列，分别为 Storage Vault 名称、 Storage Vault ID、 属性以及是否为默认 Storage Vault。

2.16.4.2.1 设置默认 Storage Vault

语法

```
SET <vault_name> AS DEFAULT STORAGE VAULT
```

2.16.4.3 建表时指定 Storage Vault

建表时在 PROPERTIES 中指定 storage_vault_name，则数据会存储在指定 vault name 所对应的 Storage Vault 上。建表成功后，该表不允许再修改 storage_vault，即不支持更换 Storage Vault。

示例

```
CREATE TABLE IF NOT EXISTS supplier (
  s_suppkey int(11) NOT NULL COMMENT "",
  s_name varchar(26) NOT NULL COMMENT "",
  s_address varchar(26) NOT NULL COMMENT "",
  s_city varchar(11) NOT NULL COMMENT "",
  s_nation varchar(16) NOT NULL COMMENT "",
  s_region varchar(13) NOT NULL COMMENT "",
  s_phone varchar(16) NOT NULL COMMENT ""
)
UNIQUE KEY (s_suppkey)
DISTRIBUTED BY HASH(s_suppkey) BUCKETS 1
PROPERTIES (
  "replication_num" = "1",
  "storage_vault_name" = "hdfs_demo_vault"
);
```

2.16.4.4 创建数据库时指定 Storage Vault

创建数据库时在 PROPERTIES 中指定 storage_vault_name。如果在数据库下建表时没有指定 storage_vault_name，则表会使用数据库的 vault name 对应的 Storage Vault 进行数据的存储。用户可以通过ALTER-DATABASE更改数据库的 storage_vault_name，该行为不会改变数据库下已经创建表的storage_vault，只有新创建的表会使用更改后的storage_vault。

示例

```
CREATE DATABASE IF NOT EXIST `db_test`
PROPERTIES (
    "storage_vault_name" = "hdfs_demo_vault"
);
```

> 备注
>
> 从 3.0.5 版本支持创建库时指定 Storage Vault。
>
> 创建表时使用 Storage Vault 的优先顺序为表 -> 数据库 -> 默认 Storage Vault。即如果表的 PROPERTY 中没有指定 Storage Vault，则会搜索数据库是否指定了 Storage Vault；如果数据库也没有指定，则会继续搜索是否有默认 Storage Vault。
>
> 如果 Storage Vault 的 VAULT_NAME 属性被修改，可能会导致数据库下设置的 Storage Vault 失效而报错，用户需要根据实际情况为数据库再配置一个可用的 storage_vault_name。

2.16.4.5 更改 Storage Vault

用于更新 Storage Vault 配置的可修改属性。

S3 Storage Vault 允许修改的属性：- VAULT_NAME - s3.access_key - s3.secret_key - use_path_style

HDFS Storage Vault 禁止修改的属性：- path_prefix - fs.defaultFS

更多属性说明见CREATE-STORAGE-VAULT。

示例

```
ALTER STORAGE VAULT old_s3_vault
PROPERTIES (
    "type" = "S3",
    "VAULT_NAME" = "new_s3_vault",
    "s3.access_key" = "new_ak"
    "s3.secret_key" = "new_sk"
);
```

```
ALTER STORAGE VAULT old_hdfs_vault
PROPERTIES (
    "type" = "hdfs",
    "VAULT_NAME" = "new_hdfs_vault",
    "hadoop.username" = "hdfs"
);
```

2.16.4.6 删除 Storage Vault

暂不支持

2.16.4.7 Storage Vault 权限

向指定的 MySQL 用户授予某个 Storage Vault 的使用权限，使该用户可以进行建表时指定该 Storage Vault 或查看 Storage Vault 等操作。

2.16.4.7.1 授予

```
GRANT
    USAGE_PRIV
    ON STORAGE VAULT <vault_name>
    TO { ROLE | USER } {<role> | <user>}
```

仅 Admin 用户有权限执行 GRANT 语句，该语句用于向 User / Role 授予指定 Storage Vault 的权限。拥有某个 Storage Vault 的 USAGE_PRIV 权限的 User / Role 可进行以下操作：

- 通过 SHOW STORAGE VAULTS 查看该 Storage Vault 的信息；
- 建表时在 PROPERTIES 中指定使用该 Storage Vault。

2.16.4.7.2 撤销

```
grant usage_priv on storage vault my_storage_vault to user1
```

撤销指定的 MySQL 用户的 Storage Vault 权限。

语法

```
REVOKE
    USAGE_PRIV
    ON STORAGE VAULT <vault_name>
    FROM { ROLE | USER } {<role> | <user>}
```

仅 Admin 用户有权限执行 REVOKE 语句，用于撤销 User / Role 拥有的对指定 Storage Vault 的权限。

示例

```
revoke usage_priv on storage vault my_storage_vault from user1
```

2.16.4.8 常见问题

Q1. 如何查询特定 storage vault 被那些表引用？

1. 通过show storage vault查看 storage vault name 对应的 storage vault id
2. 执行如下 sql 语句:

```
mysql> select * from information_schema.table_properties where PROPERTY_NAME = "storage_vault_id"
    ↪  and PROPERTY_VALUE=3;
+--
    ↪ -------------+----------------------------------+--------------------------------------+------------------+
    ↪
| TABLE_CATALOG | TABLE_SCHEMA                     | TABLE_NAME                           |
    ↪ PROPERTY_NAME    | PROPERTY_VALUE |
+--
    ↪ -------------+----------------------------------+--------------------------------------+------------------+
    ↪
| internal      | regression_test_vault_p0_create | s3_92ba28c209154d968e680e58dd54d0cc | storage
    ↪ _vault_id | 3              |
+--
    ↪ -------------+----------------------------------+--------------------------------------+------------------+
    ↪
1 row in set (0.04 sec)
```

其中PROPERTY_VALUE=3替换为对应storage vault id的数值

2.16.5 计算组操作

在存算分离架构下，可以将一个或多个计算节点 (BE) 组成一个计算组 (Compute Group)。本文档介绍如何使用计算组，其中涉及的操作包括：

- 查看所有计算组
- 计算组授权
- 在用户级别绑定计算组 (default_compute_group) 以达到用户级别的隔离效果

> 注意 3.0.2 之前的版本中叫做计算集群（Compute Cluster）。

2.16.5.1 计算组使用场景

在多计算组的架构下，可以通过将一个或多个无状态的 BE 节点组成计算集群，利用计算集群指定语句 (use @) 将特定负载分配到特定的计算集群中，从而实现多导入和查询负载的物理隔离。

假设当前有两个计算集群：C1 和 C2。

- 读读隔离：在发起两个大型查询之前，分别使用 use @c1 和 use @c2，确保两个查询在不同的计算节点上运行，从而避免在访问相同数据集时因 CPU 和内存等资源竞争而相互干扰。
- 读写隔离：Doris 的数据导入会消耗大量资源，尤其是在大数据量和高频导入的场景中。为了避免查询和导入之间的资源竞争，可以通过 use @c1 和 use @c2 指定查询在 C1 上执行，导入在 C2 上执行。同时，C1 计算集群可以访问 C2 计算集群中新导入的数据。
- 写写隔离：与读写隔离类似，导入之间也可以进行隔离。例如，当系统中存在高频小量导入和大批量导入时，批量导入通常耗时较长且重试成本高，而高频小量导入耗时短且重试成本低。为了避免小量导入对批量导入的干扰，可以通过 use @c1 和 use @c2，将小量导入指定到 C1 上执行，批量导入指定到 C2 上执行。

2.16.5.2 默认计算组的选择机制

当用户未明确设置默认计算组时，系统将自动为用户选择一个具有存活计算节点且用户具有使用权限的计算组。在特定会话中确定默认计算组后，默认计算组将在该会话期间保持不变，除非用户显式更改了默认设置。

在不同次的会话中，若发生以下情况，系统可能会自动更改用户的默认计算组：

- 用户失去了在上次会话中所选择默认计算组的使用权限
- 有计算组被添加或移除
- 上次所选择的默认计算组不再具有存活计算节点

其中，情况一和情况二必定会导致系统自动选择的默认计算组更改，情况三可能会导致更改。

2.16.5.3 查看所有计算组

使用 SHOW COMPUTE GROUPS 命令可以查看当前仓库中的所有计算组。返回结果会根据用户权限级别显示不同内容：

- 具有 ADMIN 权限的用户可以查看所有计算组
- 普通用户只能查看其拥有使用权限（USAGE_PRIV）的计算组
- 如果用户没有任何计算组的使用权限，则返回结果为空

```
SHOW COMPUTE GROUPS;
```

2.16.5.4 添加计算组

操作计算组需要具备 OPERATOR 权限，即节点管理权限。有关详细信息，请参阅权限管理。默认情况下，只有 root 账号拥有 OPERATOR 权限，但可以通过 GRANT 命令将此权限授予其他账号。要添加 BE 并为其指定计算组，请使用Add BE 命令。例如：

```
ALTER SYSTEM ADD BACKEND 'host:9050' PROPERTIES ("tag.compute_group_name" = "new_group");
```

上面命令会将host:9050这台节点添加到new_group这个计算组中，您也可以不指定计算组，默认会添加到default_compute_group组里，示例：

```
ALTER SYSTEM ADD BACKEND 'host:9050';
```

2.16.5.5 授予计算组访问权限

前置条件：当前操作用户具备 ADMIN 权限，或者当前用户属于 admin role。

```
GRANT USAGE_PRIV ON COMPUTE GROUP {compute_group_name} TO {user};
```

2.16.5.6 撤销计算组访问权限

前置条件：当前操作用户具备 ADMIN 权限，或者当前用户属于 admin role。

```
REVOKE USAGE_PRIV ON COMPUTE GROUP {compute_group_name} FROM {user};
```

2.16.5.7 设置默认计算组

为当前用户设置默认计算组（此操作需要当前用户已经拥有计算组的使用权限）：

```
SET PROPERTY 'default_compute_group' = '{clusterName}';
```

为其他用户设置默认计算组（此操作需要 Admin 权限）：

```
SET PROPERTY FOR {user} 'default_compute_group' = '{clusterName}';
```

查看当前用户默认计算组，返回结果中default_compute_group 的值即为默认计算组：

```
SHOW PROPERTY;
```

查看其他用户默认计算组，此操作需要当前用户具备 admin 权限，返回结果中default_compute_group 的值即为默认计算组：

```
SHOW PROPERTY FOR {user};
```

查看当前仓库下所有可用的计算组：

```
SHOW COMPUTE GROUPS;
```

备注

- 若当前用户拥有 Admin 角色，例如：CREATE USER jack IDENTIFIED BY '123456' DEFAULT ROLE "admin"，则：
  - 可以为自身以及其他用户设置默认计算组；
  - 可以查看自身以及其他用户的 PROPERTY。
- 若当前用户无 Admin 角色，例如：CREATE USER jack1 IDENTIFIED BY '123456'，则：
  - 可以为自身设置默认计算组；
  - 可以查看自身的 PROPERTY；
  - 无法查看所有计算组，因该操作需要 GRANT ADMIN 权限。
- 若当前用户未配置默认计算组，现有系统在执行数据读写操作时将会触发错误。为解决这一问题，用户可通过执行 use @cluster 命令来指定当前 Context 所使用的计算组，或者使用 SET PROPERTY 语句来设置默认计算组。
- 若当前用户已配置默认计算组，但随后该集群被删除，则在执行数据读写操作时同样会触发错误。用户可通过执行 use @cluster 命令来重新指定当前 Context 所使用的计算组，或者利用 SET PROPERTY 语句来更新默认集群设置。

2.16.5.8 切换计算组

用户可在存算分离架构中指定使用的数据库和计算组。

语法

```
USE { [catalog_name.]database_name[@compute_group_name] | @compute_group_name }
```

若数据库或计算组名称包含是保留关键字，需用反引号将相应的名称 “ ‘ 包围。

2.16.5.9 计算组扩缩容

通过 ALTER SYSTEM ADD BACKEND 以及 ALTER SYSTEM DECOMMISION BACKEND 添加或者删除 BE 实现计算组的扩缩容。

详细操作参考存算分离相关操作

2.16.6 文件缓存

2.16.6.1 文件缓存功能介绍

在存算分离的架构中，数据被存储在远程存储。Doris 数据库通过利用本地硬盘上的缓存来加速数据访问，并采用了一种先进的多队列 LRU（Least Recently Used）策略来高效管理缓存空间。这种策略特别优化了索引和元数据的访问路径，旨在最大化地缓存用户频繁访问的数据。针对多计算组（Compute Group）的应用场景，Doris 还提供了缓存预热功能，以便在新计算组建立时，能够迅速加载特定数据（如表或分区）到缓存中，从而提升查询性能。

2.16.6.1.1 缓存的作用

在存算分离架构中，数据通常存储在远程存储系统中，如对象存储 S3、HDFS 等。在这种场景下，Doris 数据库可以利用本地磁盘空间作为缓存，将部分数据缓存到本地，从而减少对远程存储的频繁访问，提升数据访问效率，降低运行成本。

远程存储（如对象存储）的访问延迟通常较高，且可能受到 QPS（每秒查询率）和带宽限制的约束。例如，对象存储的 QPS 限制可能导致在高并发查询时出现瓶颈，而网络带宽的限制则会影响数据传输速度。通过使用本地文件缓存，Doris 可以将热点数据存储在本地磁盘上，从而显著降低查询延迟，提升查询性能。

另一方面，对象存储服务通常会根据请求次数和数据传输量收费。频繁的访问和大量的数据下载会增加查询经济成本。通过缓存机制，可以减少对对象存储的访问次数和数据传输量，从而降低费用。

Doris 的文件缓存在存算分离架构中通常缓存以下两种文件

- segment 数据文件：Doris 中内表存储数据的基本单元。缓存这些文件可以加速对数据的读取操作，提升查询性能。
- inverted index 反向索引文件：用于加速查询中的过滤操作。通过缓存这些文件，可以更快地定位到满足查询条件的数据，进一步提升查询效率，并支持复杂的查询场景。

2.16.6.1.2 缓存的配置

Doris 提供了一系列的配置项来帮助用户灵活地管理文件缓存。这些配置项包括缓存的启用、缓存路径和大小的设置、缓存块的大小、自动清理的开关以及预先淘汰机制等。以下是详细的配置说明：

1. 启用文件缓存

`enable_file_cache` 默认 `"false"`

参数说明：此配置项用于控制是否启用文件缓存功能。如果设置为`true`，则启用文件缓存；如果设置为`false`，则禁用文件缓存。

2. 配置文件缓存路径和大小

`file_cache_path` 默认 `be` 部署路径下的 `storage` 目录

参数说明：此配置项用于指定文件缓存的路径和大小。格式为 JSON 数组，每个元素是一个 JSON 对象，包含以下字段：

- `path`：缓存文件存储的路径。

- total_size：该路径下缓存的总大小（单位：字节）。
- ttl_percent：TTL 队列占用的比例（百分比）。
- normal_percent：Normal 队列占用的比例（百分比）。
- disposable_percent：Disposable 队列占用的比例（百分比）。
- index_percent：Index 队列占用的比例（百分比）。
- storage：缓存存储类型，可以是disk或memory。默认值为disk。

示例：

- 单路径配置：

```
[{"path":"/path/to/file_cache","total_size":21474836480}]
```

- 多路径配置：

```
[{"path":"/path/to/file_cache","total_size":21474836480},{"path":"/path/to/file_cache2","total_
    ↪ size":21474836480}]
```

- 内存存储配置：

```
[{"path": "xxx", "total_size":53687091200, "storage": "memory"}]
```

3. 自动清理缓存

```
clear_file_cache 默认 "false"
```

参数说明：此配置项用于控制是否在 BE 重启时自动清理已经缓存的数据。如果设置为true，则每次 BE 重启时会自动清理缓存；如果设置为false，则不会自动清理缓存。

4. 预先淘汰机制

```
enable_evict_file_cache_in_advance 默认 "true"
```

- 参数说明：此配置项用于控制是否启用预先淘汰机制。如果设置为true，则当缓存使用空间达到一定阈值后，系统会主动进行预先淘汰，留出空间为未来的查询使用；如果设置为false，则不会进行预先淘汰。

```
file_cache_enter_need_evict_cache_in_advance_percent 默认 "88"
```

- 参数说明：此配置项用于设置触发预先淘汰的阈值百分比。当缓存使用空间/inode 数量达到此百分比时，系统开始进行预先淘汰。

```
file_cache_exit_need_evict_cache_in_advance_percent 默认 "85"
```

- 参数说明：此配置项用于设置停止预先淘汰的阈值百分比。当缓存使用空间降至此百分比时，系统停止进行预先淘汰。

2.16.6.1.3 缓存的预热

Doris 提供了缓存预热功能，允许用户从远端存储主动拉取数据至本地缓存。该功能支持以下三种模式：

- 计算组间预热：将计算组 A 的缓存数据预热至计算组 B。Doris 定期收集各计算组在一段时间内被访问的表/分区的热点信息，并根据这些信息选择性地预热某些表/分区。
- 表数据预热：指定将表 A 的数据预热至新计算组。
- 分区数据预热：指定将表 A 的分区p1的数据预热至新计算组。具体用法详见 WARM-UP SQL 文档。

2.16.6.1.4 缓存的清理

Doris 提供了同步清理和异步清理两种方式：

- 同步清理：命令为`curl 'http://BE_IP:WEB_PORT/api/file_cache?op=clear&sync=true'`，命令返回则代表清理完成。当需要立即清理缓存时，Doris 会同步删除本地文件系统目录中的缓存文件，并清理内存中的管理元数据。这种方式可以快速释放空间，但可能会对正在执行的查询效率乃至系统稳定性造成一定影响，通常用于快速测试。
- 异步清理：命令为`curl 'http://BE_IP:WEB_PORT/api/file_cache?op=clear&sync=false'`，命令直接返回，清理步骤异步执行，可以观察到缓存空间逐步减小。在异步清理过程中，Doris 会遍历内存中的管理元数据，逐一删除对应的缓存文件。如果发现某些缓存文件正在被查询使用中，Doris 会延迟删除这些文件，直到它们不再被使用。这种方式可以减少对正在执行查询的影响，但完全清理干净缓存通常需要相对同步清理更长的时间。

2.16.6.1.5 缓存的观测

热点信息

Doris 每 10 分钟收集各个计算组的缓存热点信息到内部系统表，您可以通过查询语句查看热点信息。用户可以根据这些信息更好地规划缓存的使用。

> 备注在 3.0.4 版本之前，可以使用 `SHOW CACHE HOTSPOT` 语句进行缓存热度信息统计查询。从 3.0.4 版本开始，不再支持使用 `SHOW CACHE HOTSPOT` 语句进行缓存热度信息统计查询。请直接访问系统表 `__internal_schema.cloud_cache_hotspot` 进行查询。

用户通常关注计算组和库表两个维度的缓存使用情况。以下提供了一些常用的查询语句以及示例。

查看当前所有计算组中最频繁访问的表

```
-- 等价于 3.0.4 版本前的 SHOW CACHE HOTSPOT "/"
WITH t1 AS (
  SELECT
    cluster_id,
    cluster_name,
```

```
    table_id,
    table_name,
    insert_day,
    SUM(query_per_day) AS query_per_day_total,
    SUM(query_per_week) AS query_per_week_total
  FROM __internal_schema.cloud_cache_hotspot
  GROUP BY cluster_id, cluster_name, table_id, table_name, insert_day
)
SELECT
  cluster_id AS ComputeGroupId,
  cluster_name AS ComputeGroupName,
  table_id AS TableId,
  table_name AS TableName
FROM (
  SELECT
    ROW_NUMBER() OVER (
      PARTITION BY cluster_id
      ORDER BY insert_day DESC, query_per_day_total DESC, query_per_week_total DESC
    ) AS dr2,
    *
  FROM t1
) t2
WHERE dr2 = 1;
```

查看某个计算组下的所有表中最频繁访问的表

查看计算组 compute_group_name0 下的所有表中最频繁访问的表

注意：将其中的 cluster_name = "compute_group_name0" 条件替换为实际的计算组名称。

```
-- 等价于 3.0.4 版本前的 SHOW CACHE HOTSPOT '/compute_group_name0';
WITH t1 AS (
  SELECT
    cluster_id,
    cluster_name,
    table_id,
    table_name,
    insert_day,
    SUM(query_per_day) AS query_per_day_total,
    SUM(query_per_week) AS query_per_week_total
  FROM __internal_schema.cloud_cache_hotspot
  WHERE cluster_name = "compute_group_name0" -- 替换为实际的计算组名称，例如 "default_compute_
      ↪ group"
  GROUP BY cluster_id, cluster_name, table_id, table_name, insert_day
)
SELECT
  cluster_id AS ComputeGroupId,
```

```
  cluster_name AS ComputeGroupName,
  table_id AS TableId,
  table_name AS TableName
FROM (
  SELECT
    ROW_NUMBER() OVER (
      PARTITION BY cluster_id
      ORDER BY insert_day DESC, query_per_day_total DESC, query_per_week_total DESC
    ) AS dr2,
    *
  FROM t1
) t2
WHERE dr2 = 1;
```

Cache 空间以及命中率

Doris BE 节点通过 `curl {be_ip}:{brpc_port}/vars` (brpc_port 默认为 8060) 获取 cache 统计信息，指标项的名称开始为磁盘路径。

上述例子中指标前缀为 File Cache 的路径，例如前缀“mnt_disk1_gavinchou_debug_doris_cloud_be0_storage_file_cache”表示“/mnt/disk1/gavinchou/debug/doris-cloud/be0_storage_file_cache/”去掉前缀的部分为统计指标，比如“file_cache_cache_size”表示当前路径的 File Cache 大小为 26111 字节

下表为全部的指标意义 (以下表示 size 大小单位均为字节)

指标名称 (不包含路径前缀)	语义
file_cache_cache_size	当前 File Cache 的总大小
file_cache_disposable_queue_cache_size	当前 disposable 队列的大小
file_cache_disposable_queue_element_count	当前 disposable 队列里的元素个数
file_cache_disposable_queue_evict_size	从启动到当前 disposable 队列总共淘汰的数据量大小
file_cache_index_queue_cache_size	当前 index 队列的大小
file_cache_index_queue_element_count	当前 index 队列里的元素个数
file_cache_index_queue_evict_size	从启动到当前 index 队列总共淘汰的数据量大小
file_cache_normal_queue_cache_size	当前 normal 队列的大小
file_cache_normal_queue_element_count	当前 normal 队列里的元素个数
file_cache_normal_queue_evict_size	从启动到当前 normal 队列总共淘汰的数据量大小
file_cache_total_evict_size	从启动到当前，整个 File Cache 总共淘汰的数据量大小
file_cache_ttl_cache_evict_size	从启动到当前 TTL 队列总共淘汰的数据量大小
file_cache_ttl_cache_lru_queue_element_count	当前 TTL 队列里的元素个数
file_cache_ttl_cache_size	当前 TTL 队列的大小
file_cache_evict_by_heat_[A]_to_[B]	为了写入 B 缓存类型的数据而淘汰的 A 缓存类型的数据量（基于过期时间的淘汰方式
file_cache_evict_by_size_[A]_to_[B]	为了写入 B 缓存类型的数据而淘汰的 A 缓存类型的数据量（基于空间的淘汰方式）
file_cache_evict_by_self_lru_[A]	A 缓存类型的数据为了写入新数据而淘汰自身的数据量（基于 LRU 的淘汰方式）

SQL profile SQL profile 中 cache 相关的指标在 SegmentIterator 下，包括

指标名称	语义
BytesScannedFromCache	从 File Cache 读取的数据量
BytesScannedFromRemote	从远程存储读取的数据量
BytesWriteIntoCache	写入 File Cache 的数据量
LocalIOUseTimer	读取 File Cache 的耗时
NumLocalIOTotal	读取 File Cache 的次数
NumRemoteIOTotal	读取远程存储的次数
NumSkipCacheIOTotal	从远程存储读取并没有进入 File Cache 的次数
RemoteIOUseTimer	读取远程存储的耗时
WriteCacheIOUseTimer	写 File Cache 的耗时

您可以通过查询性能分析 查看查询性能分析。

2.16.6.1.6 TTL 用法

在建表时，设置相应的 PROPERTY，即可将该表的数据使用 TTL 策略进行缓存。

- file_cache_ttl_seconds: 新导入的数据期望在缓存中保留的时间，单位为秒。

```
CREATE TABLE IF NOT EXISTS customer (
  C_CUSTKEY     INTEGER NOT NULL,
  C_NAME        VARCHAR(25) NOT NULL,
  C_ADDRESS     VARCHAR(40) NOT NULL,
  C_NATIONKEY   INTEGER NOT NULL,
  C_PHONE       CHAR(15) NOT NULL,
  C_ACCTBAL     DECIMAL(15,2)   NOT NULL,
  C_MKTSEGMENT  CHAR(10) NOT NULL,
  C_COMMENT     VARCHAR(117) NOT NULL
)
DUPLICATE KEY(C_CUSTKEY, C_NAME)
DISTRIBUTED BY HASH(C_CUSTKEY) BUCKETS 32
PROPERTIES(
    "file_cache_ttl_seconds"="300"
)
```

上表中，所有新导入的数据将在缓存中被保留 300 秒。系统当前支持修改表的 TTL 时间，用户可以根据实际需求将 TTL 的时间延长或减短。

```
ALTER TABLE customer set ("file_cache_ttl_seconds"="3000");
```

> 备注
>
> 修改后的 TTL 值并不会立即生效，而会存在一定的延迟。
>
> 如果在建表时没有设置 TTL，用户同样可以通过执行 ALTER 语句来修改表的 TTL 属性。

2.16.6.1.7 实践案例

某用户拥有一系列数据表，总数据量超过 3TB，而可用缓存容量仅为 1.2TB。其中，访问频率较高的表有两张：一张是大小为 200MB 的维度表 (dimension_table)，另一张是大小为 100GB 的事实表 (fact_table)，后者每日都有新数据导入，并需要执行 T+1 查询操作。此外，其他大表访问频率不高。

在 LRU 缓存策略下，大表数据如果被查询访问，可能会替换掉需要常驻缓存的小表数据，造成性能波动。为了解决这个问题，用户采取 TTL 缓存策略，将两张表的 TTL 时间分别设置为 1 年和 1 天。

```
ALTER TABLE dimension_table set ("file_cache_ttl_seconds"="31536000");

ALTER TABLE fact_table set ("file_cache_ttl_seconds"="86400");
```

对于维度表，由于其数据量较小且变动不大，用户设置 1 年的 TTL 时间，以确保其数据在一年内都能被快速访问；对于事实表，用户每天需要进行一次表备份，然后进行全量导入，因此将其 TTL 时间设置为 1 天。

2.16.6.2 文件缓存内部原理

2.16.6.2.1 基本原理

（一）缓存切片和预读机制

Doris 采用缓存切片和预读机制来优化数据的缓存管理和读取效率。具体来说，目标文件会被按照 1MB 对齐进行切片，每一片数据在完整下载后，会被存储在本地文件系统中作为一个单独的 Block 文件。这种切片方式可以有效减少缓存的粒度，提高缓存灵活性和空间利用率。Doris 可以根据实际需求仅缓存部分数据，避免缓存整个大文件带来的空间浪费。同时，小块缓存也便于管理和淘汰，能够更精准地命中热点数据。

（二）本地文件目录组织

为了更好地管理缓存数据，Doris 采用了特定的本地文件目录组织方式。缓存可能分布在多块磁盘的多个目录中，为了实现数据在多个目录的均匀分布，Doris 会根据缓存目标文件的路径计算哈希值，并将该哈希值作为 Block 文件存放路径的最后一级目录。目录中每个 Block 文件的命名则基于缓存数据在目标文件中的偏移量。

例如，假设目标文件路径为`/remote/data/datafile1`，计算其哈希值为`12345`，则缓存的 Block 文件可能被存放在`/cache/123/12345/offset1`的路径下，其中`offset1`表示该 Block 数据在原文件中的偏移位置。

（三）多队列机制

Doris 的文件缓存采用了多队列机制，将不同类型的数据分开管理，以避免缓存污染并提高缓存的命中率。具体来说，缓存数据被分为以下几类，并分别存储在不同的队列中。这些队列按照重要程度的优先级排序如下：

- TTL 队列：存储设置了 TTL（Time-To-Live，生存时间）属性的数据。这类数据在缓存中保留的时间由 TTL 值决定，在 TTL 时间内，这些数据具有最高优先级，不会被轻易淘汰。当缓存空间不足时，系统会优先淘汰其他队列中的数据，以确保 TTL 数据能尽可能长久地存在于缓存中。TTL 是表的属性，比如设定为 3600 则表示凡是导入此表中的数据，在导入完成后 1 小时内都尽量存在于文件缓存中。应用场景：适用于希望在本地持久化的小规模数据表。例如，对于常驻表，可以设置较长的 TTL 值来保护其数据。
- Index 队列：存储索引数据，这类数据主要用于加速查询中的过滤操作，通常具有较高的访问频率。特别的，反向索引文件虽然是“索引”但因其数据量通常很大，为了避免占用 Index 队列，我们把它作为 Normal 队列处理。
- Normal 队列：存储普通数据，这些数据没有设置 TTL 属性。大部分数据都属于普通数据。

- Disposable 队列：存储临时使用的数据，比如 compaction 读取的数据。这类数据通常在使用完毕后会被淘汰，优先级最低。

通过这种多队列机制，Doris 能够根据不同类型数据的特点和使用场景，合理分配缓存空间，最大化地利用缓存资源。

（四）淘汰机制

缓存淘汰机制是文件缓存管理中的关键环节，它决定了在缓存空间不足时如何选择需要被淘汰的数据，以为新的数据腾出空间。Doris 的淘汰机制包括以下几种触发时机和目标选择策略。

淘汰发生的时机

- 空间紧张被动淘汰：
- 当本地磁盘空间或 inode 数量不足时，Doris 会触发被动淘汰机制，以释放空间。
- 达到缓存容量设定值：虽然磁盘还有空间但如果缓存空间已经达到了预先设定的容量上限，系统也会启动淘汰机制，淘汰部分数据以腾出空间。
- 主动提前淘汰：上一种淘汰属于同步淘汰，新数据需要等待旧数据换出才能进入缓存，这会影响当前查询的效率。为了避免这种极端情况的发生，Doris 会在缓存空间达到高水位线时，提前异步地清理旧缓存。
- 主动垃圾回收淘汰：虽然 LRU 策略能够淘汰无用数据，但为了进一步优化缓存空间，Doris 会主动清理一些垃圾数据，如 compaction、schema change 的原始数据、导入 commit 失败回滚掉的数据以及 drop table/partition 后的数据。
- TTL 到期：这是 TTL 类型数据独有的淘汰机制，即使数据量没有达到上限，但当其中的数据 TTL 时间到期后，这些数据会先进入到 Normal 队列中，降级为普通数据。之后，这些数据将作为 Normal 数据参与正常的淘汰过程。

淘汰目标的选择：

- 淘汰比例：多个队列共享磁盘空间，并用各自的比例作为自身空间的限制，保证其它队列有足够的空间。在空间足够（其它队列数据量没有占满其分配比例）时，队列可以使用所有剩余磁盘空间。例如系统 Normal 队列的空间被限制在总空间的 40%，但若系统只有 Normal 数据而没有其它类型的数据，那么它可以占满所有可用空间。后面随着其它队列数据的进入，各个队列的占比逐渐趋近预设比例。
- 淘汰顺序：在写入缓存空间不足时，Doris 会按照 Disposable、Normal、Index、TTL 的顺序淘汰数据。例如，如果写入 Normal 时空间不足，那么 Doris 会依次淘汰 Disposable、Index、TTL 队列超出比例部分的数据（各队列按照 LRU 的顺序选择淘汰目标）。如果按照顺序淘汰其他类型的数据后仍不能成功腾出足够的空间，那么将会触发自身类型的 LRU 淘汰。

避免目标数据淘汰的建议：

- 充足的磁盘空间：确保有足够的磁盘空间来容纳缓存数据，避免因空间不足而频繁触发淘汰机制。因为缓存清理有一定的滞后性，需要留有一定余量。根据经验，一般文件缓存空间约为查询热数据的 1.5 倍可以保证查询数据命中文件缓存。
- 大查询隔离：将大查询隔离到其他集群，避免因大查询占用大量缓存空间而影响其他查询的缓存命中率。

（五）预热机制

缓存预热是指将数据提前加载到缓存中，以便在后续查询中能够快速命中缓存，提升查询性能。Doris 提供了多种缓存预热机制：

- 手动预热：用户可以对当前集群的缓存进行预热，预热目标可以是表和分区，也可以使用一个已有集群作为参考，预热已有集群上缓存的表和分区。预热下载的数据源始终是远程存储，而不是其他集群或者其他 BE。用户执行预热指令后，无论目标是表、分区还是参考集群，最终都会转换成一个 tablet 集合，发往 tablet 所在 BE 去执行下载。BE 下载的逻辑本质上是对这个 tablet 的所有数据文件进行一次顺序读，这样数据便能缓存在本地文件缓存上。由于预热的数据量可能很大，Doris 会将整个预热任务切分成最大 20GB 粒度的批次依次执行。每完成一批数据的下载，系统会做一个存档点，方便任务中断后恢复执行。如果 BE 节点在下载过程中发生严重问题（比如宕机），或者用户手动执行了取消预热的命令，那么所有的 BE 都会停止下载并结束这次预热。用户可以通过 SHOW WARM UP JOB 浏览当前任务的执行状态（如 FINISHED、CANCELLED、RUNNING）。如果是 RUNNING 状态，则可以查看整体完成进度。用户可以对同样的表和分区进行重复的预热，Doris 会智能识别并不会重复下载已有数据，只会增量更新。
- 数据均衡触发的预热：当 BE 上的 tablet 数量分布不均时，Doris 会自动进行负载均衡，特别是在节点宕机或者运维进行扩缩容操作时。当一个 tablet 迁移到新的 BE 时，新 BE 会发起 RPC 到旧 BE（如果旧 BE 依然有响应）拉取该 tablet 之前的缓存数据的元信息，并在新 BE 节点利用元信息重新下载数据到文件缓存。这样可以保证新 BE 上的查询也能命中文件缓存。旧节点上的对应缓存数据会在旧 tablet 信息清理时一并主动淘汰以释放空间。需要注意的是，迁移后数据下载到缓存需要一定的时间，在这个时间窗口内可能会发生文件缓存未命中的情况。
- 计算集群间自动预热（3.1 以后版本支持）：在存算分离场景下，用户可能希望多个计算集群的文件缓存能够自动同步，例如数据的导入在 A 计算集群完成，而查询发生在 B 计算集群，就需要 A 集群把导入数据的缓存同步到 B 集群。Doris 提供了两种自动同步文件缓存的方式：
- 周期预热：对于查询另一个集群导入的数据实时性不高的需求，可以在WARM UPSQL 语句中加入同步周期。这样预热任务不再是执行一次就结束，而是会周期性地将一个集群上指定表和分区的数据以增量的方式同步到另一个集群上。
- 导入和 compaction 触发的预热：对于同步实时性要求高的用户，可以使用导入完成事件触发的预热功能。因为 tablet 在不同集群的分布不同，FE 会将同步目标集群上的 tablet 分布告知源集群。在源集群导入进入 commit 阶段时，会利用上一步的信息找到当前 tablet 所在目标集群的具体 BE。源集群 BE 通知目标集群对应的 BE 下载刚刚导入时上传到远程存储上的数据完成预热。对于 compaction，同样会通过类似的通知路径完成预热。

2.16.6.2.2 情景分析

（一）查询场景下文件缓存的工作原理

在查询过程中，文件缓存的作用是减少对远程存储的访问，加速数据读取。以下是查询场景下文件缓存的工作原理：

- Scanner 读取数据文件内容：当查询请求到达 Doris 时，Scanner 组件会尝试读取所需的数据文件内容。
- 查询本地文件缓存：在访问远程存储之前，Scanner 会首先查询本地文件缓存，检查所需数据是否已经缓存在本地。
- 缓存命中：如果文件缓存根据文件路径和偏移信息，在内存缓存管理元数据中找到对应的缓存数据，则返回缓存数据的 BlockFile 文件句柄集合，供 Scanner 读取。这样可以避免从远程存储下载数据，显著减少查询延迟。

- 缓存未命中：如果读取数据的范围中部分或全部未命中缓存，则 Scanner 会访问远程存储下载未命中范围对应的数据。下载完成后，这些数据会被存放在文件缓存中，供后续查询使用。同时，Doris 会根据缓存策略决定是否淘汰其他缓存数据以腾出空间。

（二）导入场景下文件缓存的工作原理

在数据导入过程中，文件缓存的作用是为后续的查询操作提前准备数据。以下是导入场景下文件缓存的工作原理：

- 数据上传到远程存储：在导入数据时，数据首先会被上传到远程存储。
- 异步写入本地缓存：同时，Doris 会异步地将这些数据写入本地磁盘的文件缓存中。这样做的目的是为了在导入完成后，紧接着的查询操作可以直接命中缓存，提升查询性能。
- 缓存类型：导入数据时，根据数据的类型和设置的属性（如是否设置了 TTL），数据会被写入到对应的缓存队列中（如 TTL 队列、Index 队列或 Normal 队列）。

（三）Compaction 场景下文件缓存的工作原理

Compaction 是 Doris 中用于优化数据存储和查询性能的操作，它会将多个小的数据文件合并成一个大的数据文件。Doris 中 Compaction 主要分为两种：负责增量数据间合并的 Cumulative Compaction，以及负责基线数据版本（以 0 为起始版本的数据）和增量数据版本合并的 Base Compaction。

在 Compaction 过程中，文件缓存的处理方式如下：

- Cumulative Compaction：输出的新数据在上传到远程存储的同时，会进入文件缓存。这一过程与数据导入时的缓存写入类似，主要是为了加速后续的查询操作。
- Base Compaction：由于 Base Compaction 通常涉及大量冷数据，且数据量较大，为了兼顾缓存命中率和避免缓存污染，Doris 只有在缓存空间足够的情况下，才会将 Base Compaction 产生的新数据写入文件缓存。用户可以通过设置 BE 配置参数`enable_file_cache_keep_base_compaction_output = true`，强制让新数据进入文件缓存，从而提高命中率。但需要注意的是，开启后可能导致其他热数据被淘汰，影响其他查询的命中率。Doris 计划在未来版本中提供更完善的自适应策略，结合历史查询统计信息来辅助判断新数据是否进入文件缓存。

（四）重启后文件缓存加载原理

在 Doris 节点重启后，文件缓存的加载过程对于恢复缓存状态和快速响应查询至关重要。在 3.1 之前的版本中，由于文件缓存 LRU 信息未持久化，重启后 LRU 队列顺序与重启前不一致，导致热数据被淘汰，影响缓存的命中率。

在 3.1 版本中，我们引入了 LRU 信息持久化功能，其原理如下：

- 定期 dump：Doris 将各个 LRU 队列的顺序信息定期 dump 到磁盘上。
- 重启后加载：在节点重启时，Doris 会从磁盘加载这些 dump 的 LRU 信息，恢复缓存队列的状态。
- 全盘扫描加载：由于周期性 dump 时间窗口可能导致元数据和磁盘上的文件不一致，因此 Doris 会在加载 LRU 信息后，对磁盘进行一次全盘扫描，以查漏补缺，确保缓存数据的完整性和准确性。
- 查询触发并发异步加载：全盘扫描需要一定时间，为了使 BE 节点能够及时提供服务，扫描期间 BE 可以用于查询。如果查询到的数据尚未被扫描到，则会先提前加载，从而减少查询延迟。

（五）扩缩容场景缓存的处理

扩缩容操作是集群管理中的常见场景，Doris 在扩缩容过程中对文件缓存的处理方式如下：

- 横向扩容：在横向扩容时，Doris 会通过均衡操作将 tablet 迁移到新增的 BE 节点上。当 tablet 迁移到目标 BE 后，目标 BE 会根据源 BE 上的缓存数据元信息，重新下载一份缓存到本地。这样可以保证新节点上的查询也能命中文件缓存。
- 横向缩容：除了与横向扩容类似的 tablet 均衡操作外，当缩容后集群整体文件缓存容量减小到实际缓存数据量以下时，会发生淘汰。淘汰过程遵循上文提到的淘汰机制，Doris 会根据缓存策略选择性地淘汰部分数据以适应新的缓存容量。
- 纵向扩容
- 增加磁盘数量：不建议通过增加磁盘数量的方式来扩容缓存，因为 Doris 目前没有实现 rehash 操作，不会进行磁盘间的均衡。而且缓存目录数量的变动可能导致之前缓存查询故障。如果确实需要增加磁盘数量，则需要清理缓存并按需预热。
- 增加单盘容量：如果是保持磁盘数量不变，仅通过增加单个磁盘的空间来扩容缓存，则需要通过 `curl http://BE_IP:WEB_PORT/api/file_cache?op=reset&capacity=123456` 命令告知 Doris BE 空间的变更。
- 纵向缩容
- 减少磁盘空间：同样需要执行上述`reset`操作。需要注意的是，当文件缓存容量减小到实际缓存数据量以下时，会发生淘汰。淘汰过程遵循上文提到的淘汰机制。
- 扩容后的预热注意事项：因为横向扩缩容会有 tablet 均衡的操作，如果此时需要预热，需要等待迁移稳定后再执行预热命令，以确保预热的效果和效率。通过监控 fe 的`doris_fe_tablet_num` 这个 metrics，如果曲线没有波动则说明预热结束。

2.16.7 数据回收

2.16.7.1 引言

在大数据时代，数据生命周期管理已成为分布式数据库系统的核心挑战之一。随着业务数据量的爆炸式增长，如何在保证数据安全的前提下实现高效的存储空间回收，成为每个数据库产品必须解决的关键问题。

Apache Doris 作为新一代实时分析型数据库，在存算分离架构下采用了标记删除（Mark-for-Deletion）的数据回收策略，并在此基础上进行了深度优化和增强。通过引入精细化的分层回收机制、灵活可配的过期保护、多重数据一致性检查以及完善的可观测性体系，同时充分考虑分布式环境的复杂性，设计了独立的 Recycler 组件、智能的并发控制、完备的监控指标等，为用户提供了一个既高效又可控的企业级数据生命周期管理方案，实现了性能、安全性和可控性的最佳平衡。

本文将深入剖析 Doris 存算分离架构下的数据回收机制，从设计理念到技术实现，从核心原理到实践调优，全面展示这一成熟解决方案的技术细节与应用价值。

2.16.7.2 1. 常规数据回收策略对比

2.16.7.2.1 1.1 同步删除

最直接的删除方式。当数据被删除（例如 drop table）时，立即将相关的 meta 以及对应文件删除，数据一旦删除就无法恢复，操作简单直接，但删除速度较慢，风险较高。

2.16.7.2.2 1.2 对账删除 (反向）

这种方式通过定期对账机制来确定哪些数据可以删除。当数据被删除（例如 drop table）时，仅仅删除 meta 数据，系统会定期进行数据对账，扫描文件数据，识别出不再被 meta 引用或已失效的数据，然后批量删除。

2.16.7.2.3 1.3 标记删除 (正向)

这种方式通过定期扫描已删除的 meta 数据来确定哪些数据可以删除。当数据被删除（例如 drop table）时，不直接删除数据，而是将要删除的 meta 标记为已删除，系统会定期扫描被标记的 meta 数据，找到对应的文件进行批量删除。

2.16.7.3 2. Doris 存算分离标记删除的好处

Doris 存算分离架构选择了标记删除方法，这一选择能够有效保证数据一致性，同时在性能、安全性和资源利用率之间达到最佳平衡。

以 drop table 为例，标记删除相比其他两种方式有以下显著优势：

2.16.7.3.1 2.1 性能优势

- 响应速度快：drop table 操作只需要标记 meta kv 数据为删除状态，无需等待文件 I/O 操作完成，用户可以立即得到响应。这在大表删除场景下尤为重要，避免了长时间阻塞。
- 批量处理效率高：定期扫描删除标记的 meta kv，可以批量处理文件删除操作，减少系统调用次数，提高整体 I/O 效率。

2.16.7.3.2 2.2 安全性优势

- 误操作保护：标记删除提供了一个缓冲期，在实际文件删除前可以恢复误删的表，显著降低了人为操作风险。
- 事务安全性：标记操作是轻量级的 meta 修改，更容易保证原子性，减少了删除过程中系统故障导致的数据不一致问题。

2.16.7.3.3 2.3 资源管理优势

- 系统负载均衡：文件删除操作可以在系统空闲时间进行，避免在业务高峰期占用大量 I/O 资源影响正常业务。
- 可控的删除节奏：可以根据系统负载动态调整删除速度，避免大量删除操作对系统造成冲击。

2.16.7.3.4 2.4 对比其他方案

- 相比同步删除：避免了删除大表时的长时间等待，提升用户体验，此外，还提供了一定的删除缓冲期，能够保证安全性，一定程度上防止人为操作事故。
- 相比对账删除：只扫描标记删除的 meta，扫描数据更加明确，减少没有必要的 I/O 操作，效率更高，不需要遍历所有文件来判断是否被引用，删除更快速，效率更高。

2.16.7.4 3. Doris 数据回收的原理

recycler 是一个单独部署的组件，负责周期性对过期的垃圾文件进行回收，一个 recycler 可以同时回收多个 instance，并且一个 instance 同一时间只能被一个 recycler 回收。

2.16.7.4.1 3.1 标记删除

每当一个执行一个 drop 命令或者系统有垃圾数据（例如 compacted rowset）产生时，对应的 meta kv 会被标记为 recycled，recycler 会定期对 instance 中的 recycle kv 进行扫描，删除对应的对象文件，后面再将 recycle kv 删除，确保删除顺序的安全性。

2.16.7.4.2 3.2 分层结构

在 recycler 对 instance 数据进行回收时，多个任务会并发进行，例如 recycle_indexes，recycle_partition，recycle_compacted_rowsets，recycle_txn 等等任务。

数据在回收过程中按照分层结构进行删除：删除 table 是会删除对应的 partitions，删除 partition 时会删除对应 tablets，删除 tablet 的时候又会删除 tablet 对应的 rowsets，删除 rowset 会删除对应的 segment 文件，最终的执行对象是 doris 的最小文件单位即 segment 文件。

以 drop table 为例子，回收过程中，系统会首先删除 segment 对象文件，成功后删除 recycle rowset kv，tablet 的 rowset 全部删除成功后会删除 recycle tablet kv，以此类推最终删除 table 中所有的对象文件以及 recycle kv。

2.16.7.4.3 3.3 过期机制

每个需要回收的对象都在其 kv 中记录有对应的过期时间，系统通过扫描各种 recycle kv 并且计算过期时间来识别要删除的对象，如果出现了用户误操作将某个 table drop，这时由于过期机制的存在，recycler 不会立刻对其数据进行删除，而是会等待一个 retition time，这为用户恢复数据提供了可能。

2.16.7.4.4 3.4 可靠性保证

1. 分阶段删除：先删除数据文件，再删除元数据，最后删除索引或分区的 key，确保删除顺序的安全性。
2. Lease 保护机制：每个 recycler 在开始回收前都要获取 lease，启动后台线程定期续 lease，只有 lease 过期或状态为 IDLE 时，新的 recycler 才能接管，保证了同一时间一个 instance 只能由一个 recycler 回收，避免并发回收导致的数据不一致问题。

2.16.7.4.5 3.5 多重检查机制

Recycler 实现了 FE 元数据、MS kv 与对象文件的多重相互检查机制（checker）。checker 在后台对所有的 Recycler kv、对象文件、FE 内存元数据三方进行正反向检查。

以 segment 文件 KV 与对象文件检查为例：- 正向检查：扫描所有 kv，检查是否都有对应的 segment 文件存在，以及 FE 内存中是否存在相应的 segment 信息。- 反向检查：扫描所有 segment 文件，验证是否都有对应的 kv，以及 FE 内存中是否存在相应的 segment 信息。

多重检查机制能够保证 recycler 删除数据的正确性。如果在某种情况下出现未回收或多回收的情况，checker 会捕获相关信息，运维人员可以根据 checker 的信息手动删除多余垃圾文件，也可以依靠对象的多版本来恢复误删的文件，提供了有效的兜底机制。

当前已实现了 segment 文件、idx 文件、delete bitmap 元数据等的正反向检查，后续将实现所有元数据的检查，进一步保证 recycler 的正确性与可靠性。

2.16.7.5 4. 观测机制

recycler 回收效率进度是用户非常关心的问题，因此我们大大提高了 recycler 的可观测性，添加了大量可视化监控指标以及必要的日志，可视化指标能够让用户直观的看到回收的进度，效率，异常等基础信息，我们也提供了更多指标可以让用户看到更加详细的信息，例如估算下一次某个 instance 做 recycle 的时间；添加的日志也可以让运维及研发更快的定位问题。

2.16.7.5.1 4.1 解答用户关心的问题

基础问题： - 仓库粒度的回收速度：每秒回收多少字节，各类对象每秒回收数量 - 仓库粒度每次回收的数据量和耗时 - 仓库粒度的回收进度：已回收数据量，待回收数据量

高级问题： - 每个存储后端的回收情况 - Recycler 回收成功时间、失败时间 - 下一次 Recycler 的预计回收时间

这些信息都可以通过 MS 面板进行实时观测。

2.16.7.5.2 4.2 观测指标

变量名	Metrics name	维度/标签	含义	例子
g_bvar_recycler_vault_recycle_status	recycler_vault_recycle_status	instance_id, resource_id, status	按实例 ID、资源 ID 和状态记录回收存储库操作的状态计数	recycler_vault_recycle_status{in 8
g_bvar_recycler_vault_recycle_task_concurrency	recycler_vault_recycle_task_concurrency	instance_id, resource_id	按实例 ID 和资源 ID 统计 vault 回收文件任务的并发数	recycler_vault_recycle_task_cor 2
g_bvar_recycler_instance_last_round_recycled_num	recycler_instance_last_round_recycled_num	instance_id, resource_type	按实例 ID 和对象类型统计最近一轮已回收的对象数量	recycler_instance_last_round_r 13

变量名	Metrics name	维度/标签	含义	例子
g_bvar_recycler_instance_last_round_to_recycle_num	recycler_instance_last_round_to_recycle_num	instance_id, resource_type	按实例 ID 和对象类型统计最近一轮需要回收的对象数量	recycler_instance_last_round_t 13
g_bvar_recycler_instance_last_round_recycled_bytes	recycler_instance_last_round_recycled_bytes	instance_id, resource_type	按实例 ID 和对象类型统计最近一轮已回收的数据大小（bytes）	recycler_instance_last_round_r 13509
g_bvar_recycler_instance_last_round_to_recycle_bytes	recycler_instance_last_round_to_recycle_bytes	instance_id, resource_type	按实例 ID 和对象类型统计最近一轮需要回收的数据大小（bytes）	recycler_instance_last_round_t 13509
g_bvar_recycler_instance_last_round_recycle_elapsed_ms	recycler_instance_last_round_recycle_elapsed_ms	instance_id, resource_type	按实例 ID 和对象类型统计最近一轮上最近一轮回收操作的耗时(ms)	recycler_instance_last_round_r 62
g_bvar_recycler_instance_recycle_round	recycler_instance_recycle_round	instance_id, resource_type	按实例 ID 和对象类型统计回收操作的轮次	recycler_instance_recycle_roun 2

变量名	Metrics name	维度/标签	含义	例子
g_bvar_recycler_instance_recycle_time_per_resource	recycler_instance_recycle_time_per_resource	instance_id, resource_type	按实例 ID 和对象类型记录回收操作的速度(代表每个资源回收需要的时间 (ms)，如果为 -1 代表没有回收)	recycler_instance_recycle_time 4.76923
g_bvar_recycler_instance_recycle_bytes_per_ms	recycler_instance_recycle_bytes_per_ms	instance_id, resource_type	按实例 ID 和对象类型记录回收操作的速度(代表每毫秒能回收的 bytes，如果为 -1 代表没有回收)	recycler_instance_recycle_byte 217.887
g_bvar_recycler_instance_recycle_total_num_instance_started	recycler_instance_recycle_total_num_instance_started	instance_id, resource_type	按实例 ID 和对象类型，从 recycler 启动以来统计回收操作的总回收数量	recycler_instance_recycle_total 49

变量名	Metrics name	维度/标签	含义	例子
g_bvar_recycler_instance_recycle_total_bytes_since_started	recycler_instance_recycle_total_bytes_since_started	instance_id, resource_type	按实例 ID 和对象类型，从 recycler 启动以来统计回收操作的总回收大小 (bytes)	recycler_instance_recycle_total 40785
g_bvar_recycler_instance_running_counter	recycler_instance_running_counter	-	统计现在有多少个 instance 在做 recycle	recycler_instance_running_cou 0
g_bvar_recycler_instance_last_recycle_duration_ms	recycler_instance_last_round_recycle_duration_ms	instance_id	按实例 ID，统计最近一轮回收的总用时	recycler_instance_last_recycle_ 64
g_bvar_recycler_instance_next_ts	recycler_instance_next_ts	instance_id	按实例 ID，根据 config 的 recy-cle_interval_seconds 估算下一次做 recycle 的时间	recycler_instance_next_ts{insta 1750400266781
g_bvar_recycler_instance_recycle_st_ts	recycler_instance_recycle_start_ts	instance_id	按实例 ID，统计总回收流程的开始时间	recycler_instance_recycle_st_ts 1750400236717

变量名	Metrics name	维度/标签	含义	例子
g_bvar_recycler_instance_recycle_ed_ts	recycler_instance_recycle_end_ts	instance_id	按实例 ID，统计总回收流程的结束时间	recycler_instance_recycle_ed_ts 1750400236781
g_bvar_recycler_instance_recycle_last_success_ts	recycler_instance_recycle_last_success_ts	instance_id	按实例 ID，统计上一次回收成功的时间	recycler_instance_recycle_last_ 1750400236781

2.16.7.6 5. 参数调优

recycler 的常见参数以及说明如下：

```
// recycler回收间隔，单位秒
CONF_mInt64(recycle_interval_seconds, "3600");

// 公共的留存时间，适用于所有没有自己retition time的对象的回收
CONF_mInt64(retention_seconds, "259200");

// 一个recycler同时可以回收的instance的最大数量
CONF_Int32(recycle_concurrency, "16");

// 被compacted的rowset的留存时间，单位秒
CONF_mInt64(compacted_rowset_retention_seconds, "1800");

// 被drop的index的留存时间，单位秒
CONF_mInt64(dropped_index_retention_seconds, "10800");

// 被drop的partition的留存时间，单位秒
CONF_mInt64(dropped_partition_retention_seconds, "10800");

// recycle的白名单，填写instance id，用逗号隔开，不填写默认回收所有instance
CONF_Strings(recycle_whitelist, "");

// recycle的黑名单，填写instance id，用逗号隔开，不填写默认回收所有instance
CONF_Strings(recycle_blacklist, "");

// 对象IO worker的并发度：例如object list, delete
CONF_mInt32(instance_recycler_worker_pool_size, "32");
```

```
// recycle对象的并发度：例如recycle_tablet，recycle_rowset
CONF_Int32(recycle_pool_parallelism, "40");

// 是否开启checker
CONF_Bool(enable_checker, "false");

// 是否开启反向checker
CONF_Bool(enable_inverted_check, "false");

// checker的间隔
CONF_mInt32(check_object_interval_seconds, "43200");

// 是否开启recycler的观测指标
CONF_Bool(enable_recycler_stats_metrics, "false");

// recycle存储后端的白名单，填写vault name，用逗号隔开，不填写默认回收所有vault
CONF_Strings(recycler_storage_vault_white_list, "");
```

2.16.7.6.1 常见调优场景 Q&A

1. 回收性能调优

Q1: 回收速度太慢怎么办？

A1: 可以从以下几个方面进行调优：- 增加并发度：- 调大 recycle_concurrency（默认 16）：增加同时回收的 instance 数量 - 调大 instance_recycler_worker_pool_size（默认 32）：增加对象 IO 操作的并发度 - 调大 recycle_pool_parallelism（默认 40）：增加回收对象的并发度 - 缩短回收间隔：将 recycle_interval_seconds 从默认 3600 秒调小，如 1800 秒 - 使用白名单机制：通过 recycle_whitelist 优先回收重要的 instance

Q2: 回收压力过大，影响业务怎么调整？

A2: 可以采用以下策略降低回收压力：- 降低并发度：- 适当减小 recycle_concurrency，避免同时回收过多 instance - 减小 instance_recycler_worker_pool_size 和 recycle_pool_parallelism - 延长回收间隔：增大 recycle_interval_seconds，如调整为 7200 秒 - 使用黑名单：通过 recycle_blacklist 暂时排除高负载的 instance - 错峰回收：在业务低峰期进行回收操作

2. 存储空间调优

Q3: 存储空间不足，需要加快垃圾清理怎么办？

A3: 可以调整各类对象的留存时间：- 缩短通用留存时间：将 retention_seconds 从默认 259200 秒（3 天）调小 - 针对性调整特定对象：- compacted_rowset_retention_seconds（默认 1800 秒）可适当缩短 - dropped_index_retention_seconds 和 dropped_partition_retention_seconds（默认 10800 秒）可根据需求调整 - 选择性回收存储后端：通过 recycler_storage_vault_white_list 优先清理特定存储

Q4: 需要保留更长时间的数据以防误删怎么办？

A4: 延长相应的留存时间：- 增大 retention_seconds 为更长时间，如 604800 秒 - 根据不同对象的重要性调整对应的 retention 参数 - 重要的 partition 可以通过 dropped_partition_retention_seconds 设置更长留存时间

3. 监控与排查调优

Q5: 如何开启更好的监控和排查能力？

A5: 建议开启以下监控功能：- 开启观测指标：设置 enable_recycler_stats_metrics = true - 开启检查机制：- 设置 enable_checker = true 开启正向检查 - 设置 enable_inverted_check = true 开启反向检查 - 调整 check_object_interval_seconds（默认 43200 秒/12 小时）为合适的检查频率

Q6: 怀疑数据一致性问题怎么排查？

A6: 利用 checker 机制进行检查：- 确保 enable_checker 和 enable_inverted_check 都为 true - 适当缩短 check_object_interval_seconds 增加检查频率 - 通过 MS 面板观察 checker 发现的异常情况 - 根据 checker 报告手动处理多余的垃圾文件或补充误删文件

4. 特殊场景调优

Q7: 某些 instance 回收异常，如何临时处理？

A7: 使用白名单和黑名单机制：- 临时跳过问题 instance：将异常 instance ID 加入 recycle_blacklist - 优先处理特定 instance：将需要优先处理的 instance ID 加入 recycle_whitelist - 存储后端选择：通过 recycler_storage_vault_white_list 选择性回收特定存储后端

Q8: 大表删除导致回收任务堆积怎么办？

A8: 综合调优策略：- 临时增大并发度参数应对积压 - 适当缩短大对象的 retention 时间 - 使用白名单优先处理积压严重的 instance - 必要时可以部署多个 recycler 分担压力

Q9: 长时间查询遇到对象存储“404 file not found”错误怎么办？

A9: 当查询执行时间很长，而查询期间 tablet 进行了 compaction 操作，对象存储上被合并的 rowset 可能已经被回收，导致查询失败并出现“404 file not found”错误。解决方案：- 增加 compacted rowset 留存时间：将 compacted_rowset_retention_seconds 从默认 1800 秒调大，如：- 对于有长查询的场景，建议调整为 7200 秒（或更长）- 根据最长查询时间来设定合适的留存时间

这样可以确保长查询在执行过程中所需的 rowset 不会被提前回收，避免查询失败。

注意：以上调优建议需要根据实际的集群规模、存储容量、业务特点等因素进行具体调整。建议在调优过程中密切关注系统负载和业务影响，逐步调整参数以找到最佳配置。

2.16.7.7 结语

Apache Doris 存算分离架构下的标记删除机制，通过巧妙平衡性能、安全性和资源利用率，Doris 不仅解决了传统数据回收方式的固有缺陷，更为用户提供了一套完整、可靠、可观测的数据管理解决方案。

从精细化的分层回收设计，到智能的过期保护机制，从完善的多重检查体系，到丰富的可观测性指标，Doris 的数据回收机制在每一个细节上都体现了对用户需求的深入理解和对技术品质的不懈追求。特别是其提供的灵活参数调优能力，使得不同规模、不同场景的用户都能找到最适合自己的配置方案。

未来，我们将继续优化和完善这一机制，在保持现有优势的基础上，进一步提升回收效率、增强智能化水平、丰富监控维度，为用户构建更加高效、可靠的实时数据分析平台。欢迎广大用户在实践中探索更多可能，与我们一起推动 Apache Doris 不断向前发展。

2.16.8 升级

2.16.8.1 概述

本指南提供了使用存储计算解耦（即，存算分离）架构升级 Doris 的分步说明。升级请使用本章节中推荐的步骤进行集群升级，Doris 集群升级可使用滚动升级的方式进行升级，无需集群节点全部停机升级，极大程度上降低对上层应用的影响。

2.16.8.2 Doris 版本说明

Doris 使用三位数的版本号格式，可以使用如下 SQL 进行查看版本：

```
MySQL [(none)]> select @@version_comment;
+-----------------------------------------------------------+
| @@version_comment                                         |
+-----------------------------------------------------------+
| Doris version doris-3.0.3-rc03-43f06a5e26 (Cloud Mode) |
+-----------------------------------------------------------+
```

> 其中3.0.3的第一个数字表示大版本号，第二个数字表示中版本号，第三个数字表示小版本号，在某些情况下，版本号会变成 4 位，如2.0.2.1，此时的最后一位数字表示这是一个紧急修复 bug 的版本，这通常意味着这个小版本有一些重大的 bug。
>
> Doris 从3.0.0版本开始支持存算分离模式部署，当以这种模式部署后，版本号后面会有 Cloud Mode 后缀，以存算一体模式启动的话，则没有这个后缀。

Doris 以存算分离模式部署之后，不支持切换成存算一体模式。同样的，存算一体模式的 Doris 也不支持切换成存算分离模式。

Doris 原则上支持从低版本升级到高版本，以及小版本降级，对于中版本或大版本，则不支持降级。

2.16.8.3 升级步骤

2.16.8.3.1 升级说明

1. 确保你的 Doris 是以存算分离模式启动的，如果你不清楚当前的 Doris 是什么部署方式，可以参考上一小节的说明。对于存算一体模式的 Doris，升级步骤可参考集群升级。
2. 确保你的 Doris 导数任务具备重试机制，以避免升级过程中，因节点重启而导致的导数任务失败。
3. 在升级之前，我们建议你检查一下各个 Doris 组件（MetaService、Recycler、Frontend、Backend）的状态正常并且无异常日志，以免升级过程中受到影响。

2.16.8.3.2 升级流程概览

1. 元数据备份
2. 升级 MetaService
3. 升级 Recycler（如有）
4. 升级 BE
5. 升级 FE
6. 先升级 Observer 角色的 FE
7. 再升级其他非 Master 角色的 FE
8. 最后升级 Master 角色的 FE

2.16.8.3.3 升级前置工作

1. 备份 Master FE 的元数据目录，元数据目录通常是 FE 目录下 doris-meta 目录，如果此目录为空，那么可能是修改了目录的位置，你可以到 FE 的配置文件（conf/fe.conf）中搜索`meta_dir`配置项。
2. 从 Doris 官方网站下载安装包，建议校验 SHA-512 码，保证下载到到安装包与 Doris 官方提供的是一致的。

2.16.8.3.4 升级流程

1. 升级 MetaService

假设以下环境变量：- `${MS_HOME}`：MetaService 的工作目录。- `${MS_PACKAGE_DIR}`：包含新 MetaService 包的目录。

按照以下步骤升级每个 MetaService 实例。

1.1. 停止当前 MetaService：

```
cd ${MS_HOME}
sh bin/stop.sh
```

1.2. 备份现有 MetaService 二进制文件：

```
mv ${MS_HOME}/bin bin_backup_$(date +%Y%m%d_%H%M%S)
mv ${MS_HOME}/lib lib_backup_$(date +%Y%m%d_%H%M%S)
```

1.3. 部署新包：

```
cp ${MS_PACKAGE_DIR}/bin ${MS_HOME}/bin
cp ${MS_PACKAGE_DIR}/lib ${MS_HOME}/lib
```

1.4. 启动新的 MetaService：

```
sh ${MS_HOME}/bin/start.sh --daemon
```

1.5. 检查新 MetaService 的状态：

确保新 MetaService 正在运行，并且在 `${MS_HOME}/log/doris_cloud.out` 中有新的版本号。

2. 升级 Recycler（如有）

如果你没有单独部署 Recycler 组件，那么可以跳过这一步。

假设以下环境变量：- `${RECYCLER_HOME}`：Recycler 的工作目录 - `${MS_PACKAGE_DIR}`：包含新 MetaService 包的目录，MetaService 和 Recycler 使用相同的包。

按照以下步骤升级每个 Recycler 实例。

2.1. 停止当前 Recycler：

```
cd ${RECYCLER_HOME}
sh bin/stop.sh
```

2.2. 备份现有 Recycler 二进制文件：

```
mv ${RECYCLER_HOME}/bin bin_backup_$(date +%Y%m%d_%H%M%S)
mv ${RECYCLER_HOME}/lib lib_backup_$(date +%Y%m%d_%H%M%S)
```

2.3. 部署新包：

```
cp ${RECYCLER_PACKAGE_DIR}/bin ${RECYCLER_HOME}/bin
cp ${RECYCLER_PACKAGE_DIR}/lib ${RECYCLER_HOME}/lib
```

2.4. 启动新的 Recycler：

```
sh ${RECYCLER_HOME}/bin/start.sh --recycler --daemon
```

2.5. 检查新 Recycler 的状态：

确保新 Recycler 正在运行，并且在 `${RECYCLER_HOME}/log/doris_cloud.out` 中有新的版本号。

3. 升级 BE

验证所有 MetaService 和 Recycler（如果单独安装）实例已升级。

假设以下环境变量：- `${BE_HOME}`：BE 的工作目录。- `${BE_PACKAGE_DIR}`：包含新 BE 包的目录。

按照以下步骤升级每个 BE 实例。

3.1. 停止当前 BE：

```
cd ${BE_HOME}
sh bin/stop_be.sh
```

3.2. 备份现有 BE 二进制文件：

```
mv ${BE_HOME}/bin bin_backup_$(date +%Y%m%d_%H%M%S)
mv ${BE_HOME}/lib lib_backup_$(date +%Y%m%d_%H%M%S)
```

3.3. 部署新包：

```
cp ${BE_PACKAGE_DIR}/bin ${BE_HOME}/bin
cp ${BE_PACKAGE_DIR}/lib ${BE_HOME}/lib
```

3.4. 启动新的 BE：

```
sh ${BE_HOME}/bin/start_be.sh --daemon
```

3.5. 检查新 BE 的状态：

确认新的 BE 是否正在运行，并且使用新版本正常运行。可以使用以下 SQL 获取状态和版本。

```
show backends;
```

4. 升级 FE

验证所有 BE 实例已升级。

假设以下环境变量：- ${FE_HOME}：FE 的工作目录。- ${FE_PACKAGE_DIR}：包含新 FE 包的目录。

按以下顺序升级 Frontend（FE）实例：1. 观察者 FE 节点 2. 非主 FE 节点 3. 主 FE 节点

按照以下步骤升级每个 Frontend（FE）节点。

4.1. 停止当前 FE：

```
cd ${FE_HOME}
sh bin/stop_fe.sh
```

4.2. 备份现有 FE 二进制文件：

```
mv ${FE_HOME}/bin bin_backup_$(date +%Y%m%d_%H%M%S)
mv ${FE_HOME}/lib lib_backup_$(date +%Y%m%d_%H%M%S)
```

4.3. 部署新包：

```
cp ${FE_PACKAGE_DIR}/bin ${FE_HOME}/bin
cp ${FE_PACKAGE_DIR}/lib ${FE_HOME}/lib
```

4.4. 启动新的 FE：

```
sh ${FE_HOME}/bin/start_fe.sh --daemon
```

4.5. 检查新 FE 的状态：

确认新的 FE 是否正在运行，并且使用新版本正常运行。可以使用以下 SQL 获取状态和版本。

```
show frontends;
```

2.16.8.4 常见问题

1. 存算一体模式的 Doris 的升级前需要关闭副本均衡功能，存算分离模式下的集群需要吗？

不需要。因为存算分离模式下，Doris 的数据存放在 HDFS 或 S3 服务上，因此不存在副本均衡的需求。

2. 有了独立的 MetaService 提供元数据服务，为什么 FE 还需要备份元数据？

因为目前 MetaService 保存了一部分元数据，FE 也保存了一部分元数据，为了稳妥起见，我们建议备份 FE 的元数据。

2.17 安全合规

2.17.1 安全概览

Doris 提供以下机制管理数据安全：

身份认证：Doris 支持用户名/密码与 LDAP 认证方式。

- 内置认证：Doris 内置了用户名/密码的认证方式，可以自定义密码策略；
- LDAP 认证：Doris 可以通过 LDAP 服务集中管理用户凭证，简化访问控制并增强系统的安全性。

权限管控：Doris 支持基于角色的访问控制或继承 Ranger 实现集中化的权限管理。

- 基于角色的访问控制（RBAC），Doris 可以根据用户角色与权限，限制其对数据库资源的访问与操作；
- Ranger 权限管理：Doris 可以通过集成 Ranger 实现集中化的权限管理，允许管理员为不同的用户和组设置细粒度的访问控制策略。

审计与日志记录：Doris 可以开启审计日志，记录用户的所有操作行为，包括登录，查询，数据修改等行为，便于事后审计与问题追踪；

数据加密与脱敏：Doris 支持对表中的数据进行加密与脱敏，防止未授权的访问造成敏感数据泄漏；

数据传输加密：Doris 支持 SSL 加密协议，确保客户端与 Doris 服务器之间的数据传输安全，防止数据在传输过程中被窃取或篡改；

细粒度访问控制：Doris 中可以基于规则配置数据行/列管控用户访问权限。

JAVA-UDF 安全：Doris 支持用户自定义函数功能，所以需要 Root 管理员审查用户 UDF 的实现，确保实现逻辑的操作安全，防止在 UDF 中执行高危操作，例如删除数据和破坏系统等。

第三方包：在使用 Doris 的 JDBC Catalog、UDF 等功能时，如需引入第三方包，管理员需自行确保这些包来源安全可信。建议仅使用来自官方渠道或受信任社区的依赖包，以降低安全风险。

2.17.2 认证与鉴权

2.17.2.1 认证与鉴权概述

Doris 的权限管理系统参照了 MySQL 的权限管理机制，做到了行级别细粒度的权限控制，基于角色的权限访问控制，并且支持白名单机制。

2.17.2.1.1 名词解释

1. 用户标识 User Identity

在权限系统中，一个用户被识别为一个 User Identity（用户标识）。用户标识由两部分组成：username 和 host。其中 username 为用户名，由英文大小写组成。host 表示该用户链接来自的 IP。User Identity 以 username@'host' 的方式呈现，表示来自 host 的 username。

User Identity 的另一种表现方式为 username@['domain']，其中 domain 为域名，可以通过 DNS 解析为一组 IP。最终表现为一组 username@'host'，所以后面我们统一使用 username@'host' 来表示。

2. 权限 Privilege

权限作用的对象是节点、数据目录、数据库或表。不同的权限代表不同的操作许可。

3. 角色 Role

Doris 可以创建自定义命名的角色。角色可以被看做是一组权限的集合。新创建的用户可以被赋予某一角色，则自动被赋予该角色所拥有的权限。后续对角色的权限变更，也会体现在所有属于该角色的用户权限上。

4. 用户属性 User Property

用户属性直接附属于某一用户，而不是用户标识。即 user@'192.%' 和 user@['domain'] 都拥有同一组用户属性，该属性属于用户 user，而不是 user@'192.%' 或 user@['domain']。

用户属性包括但不限于：用户最大连接数、导入集群配置等等。

2.17.2.1.2 认证和鉴权框架

用户登录 Apache Doris 的过程，分为认证和鉴权两部分。

- 认证：根据用户提供的凭据（如用户名、客户 IP、密码）等，进行身份验证。验证通过后，会将用户个体映射到系统内的用户标识（User Identity）上。
- 鉴权：基于获取到的用户标识，根据用户标识所对应的权限，检查用户是否有相应操作的权限。

2.17.2.1.3 认证

Doris 支持内置认证方案和以及 LDAP 的认证方案。

Doris 内置认证方案

基于 Doris 自身存储的用户名，密码等信息来认证。

管理员通过 CREATE USER 命令来创建用户，通过 SHOW ALL GRANTS 来查看创建的所有用户。

用户登录时，会判断用户名，密码及客户端的 IP 地址是否正确。

密码策略

Doris 支持以下密码策略，可以帮助用户更好的进行密码管理。

1. PASSWORD_HISTORY

   是否允许当前用户重置密码时使用历史密码。如 PASSWORD_HISTORY 10 表示禁止使用过去 10 次设置过的密码为新密码。如果设置为 PASSWORD_HISTORY DEFAULT，则会使用全局变量 password_history 中的值。0 表示不启用这个功能。默认为 0。

   示例：

   - 设置全局变量：SET GLOBAL password_history = 10
   - 为用户设置：ALTER USER user1@'ip' PASSWORD_HISTORY 10

2. PASSWORD_EXPIRE

   设置当前用户密码的过期时间。如 PASSWORD_EXPIRE INTERVAL 10 DAY 表示密码会在 10 天后过期。PASSWORD_EXPIRE NEVER 表示密码不过期。如果设置为 PASSWORD_EXPIRE DEFAULT，则会使用全局变量 default_password_lifetime 中的值（单位为天）。默认为 NEVER（或 0），表示不会过期。

   示例：

   - 设置全局变量：SET GLOBAL default_password_lifetime = 1
   - 为用户设置：ALTER USER user1@'ip' PASSWORD_EXPIRE INTERVAL 10 DAY

3. FAILED_LOGIN_ATTEMPTS 和 PASSWORD_LOCK_TIME

   设置当前用户登录时，如果使用错误的密码登录 n 次后，账户将被锁定，并设置锁定时间。如 FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY 表示如果 3 次错误登录，则账户会被锁定一天。管理员可以通过 ALTER USER 语句主动解锁被锁定的账户。

   示例：

   - 为用户设置：ALTER USER user1@'ip' FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY

4. 密码强度

   该功能由全局变量 validate_password_policy 控制。默认为 NONE/0，即不检查密码强度。如果设置为 STRONG/2，则密码必须包含“大写字母”，“小写字母”，“数字”和“特殊字符”中的 3 项，并且长度必须大于等于 8。

   示例：

   - SET validate_password_policy=STRONG

更多帮助，请参阅ALTER USER。

基于 LDAP 的认证方案

请参阅基于 LDAP 的认证方案。

2.17.2.1.4 鉴权

权限操作

- 创建用户：CREATE USER
- 修改用户：ALTER USER
- 删除用户：DROP USER

- 授权/分配角色：GRANT
- 撤权/撤销角色：REVOKE
- 创建角色：CREATE ROLE
- 删除角色：DROP ROLE
- 修改角色：ALTER ROLE
- 查看当前用户权限和角色：SHOW GRANTS
- 查看所有用户权限和角色：SHOW ALL GRANTS
- 查看已创建的角色：SHOW ROLES
- 设置用户属性：SET PROPERTY
- 查看用户属性：SHOW PROPERTY
- 修改密码：SET PASSWORD
- 查看支持的所有权限项：SHOW PRIVILEGES
- 查看行权限策略SHOW ROW POLICY
- 创建行权限策略CREATE ROW POLICY

权限类型

Doris 目前支持以下几种权限

1. Node_priv

   节点变更权限。包括 FE、 BE、 BROKER 节点的添加、 删除、 下线等操作。

   Root 用户默认拥有该权限。同时拥有 Grant_priv 和 Node_priv 的用户，可以将该权限赋予其他用户。

   该权限只能赋予 Global 级别。

2. Grant_priv

   权限变更权限。允许执行包括授权、 撤权、 添加/删除/变更用户/角色等操作。

   给其他用户/角色授权时，2.1.2 版本之前，当前用户只需要相应层级的 Grant_priv 权限，2.1.2 版本之后当前用户也要有想要授权的资源的权限。

   给其他用户分配角色时，要有 Global 级别的 Grant_priv 权限。

3. Select_priv

   对数据目录、数据库、表的只读权限。

4. Load_priv

   对数据目录、数据库、表的写权限。包括 Load、 Insert、 Delete 等。

5. Alter_priv

   对数据目录、数据库、表的更改权限。包括重命名库/表、添加/删除/变更列、添加/删除分区等操作。

6. Create_priv

   创建数据目录、数据库、表、视图的权限。

7. Drop_priv

   删除数据目录、数据库、表、视图的权限。

8. Usage_priv

   Resource 和 Workload Group 的使用权限。

9. Show_view_priv

   执行 SHOW CREATE VIEW 的权限。

权限层级

全局权限

即通过 GRANT 语句授予的 *.*.* 上的权限。被授予的权限适用于任意 Catalog 中的任意库表。

数据目录（Catalog）权限

即通过 GRANT 语句授予的 ctl.*.* 上的权限。被授予的权限适用于指定 Catalog 中的任意库表。

库级权限

即通过 GRANT 语句授予的 ctl.db.* 上的权限。被授予的权限适用于指定数据库中的任意表。

表级权限

即通过 GRANT 语句授予的 ctl.db.tbl 上的权限。被授予的权限适用于指定表的任意列。

列级权限

列权限主要用于限制用户对数据表中某些列的访问权限。具体来说，列权限允许管理员设定某些列的查看、编辑等权限，以控制用户对特定列数据的访问和操作。

可以通过 GRANT Select_priv(col1,col2)ON ctl.db.tbl TO user1 授予的指定表的部分列的权限。

目前列权限仅支持 Select_priv。

行级权限

行权限（Row Policy）使得管理员能够基于数据的某些字段来定义访问策略，从而控制哪些用户可以访问哪些数据行。

具体来说，Row Policy 允许管理员创建规则，这些规则可以基于存储在数据中的实际值来过滤或限制用户对行的访问。

从 1.2 版本开始，可以通过 CREATE ROW POLICY 命令创建行级权限。

从 2.1.2 版本开始，支持通过 Apache Ranger 的 Row Level Filter 来设置行权限。

使用权限

- Resource 权限

  Resource 权限是为 Resource 单独设置的权限，和库表等权限没有关系，只能分配 Usage_priv 和 Grant_priv 权限。

  给所有 Resource 分配权限可以通过 GRANT USAGE_PRIV ON RESOURCE '%' TO user1 语句。

- Workload Group 权限

  Workload Group 权限是为 Workload Group 单独设置的权限，和库表等权限没有关系，只能分配 Usage_priv 和 Grant_priv 权限。

  给所有 Workload Group 分配权限可以通过 GRANT USAGE_PRIV ON WORKLOAD GROUP '%' TO user1 语句。

数据脱敏

数据脱敏是一种保护敏感数据的方法，它通过对原始数据进行修改、替换或隐藏，使得脱敏后的数据在保持一定格式和特性的同时，不再包含敏感信息。

例如，管理员可以选择将信用卡号、身份证号等敏感字段的部分或全部数字替换为星号 * 或其他字符，或者将真实姓名替换为假名。

从 2.1.2 版本开始，支持通过 Apache Ranger 的 Data Masking 来为某些列设置脱敏策略，目前仅支持通过 Apache Ranger 来设置。

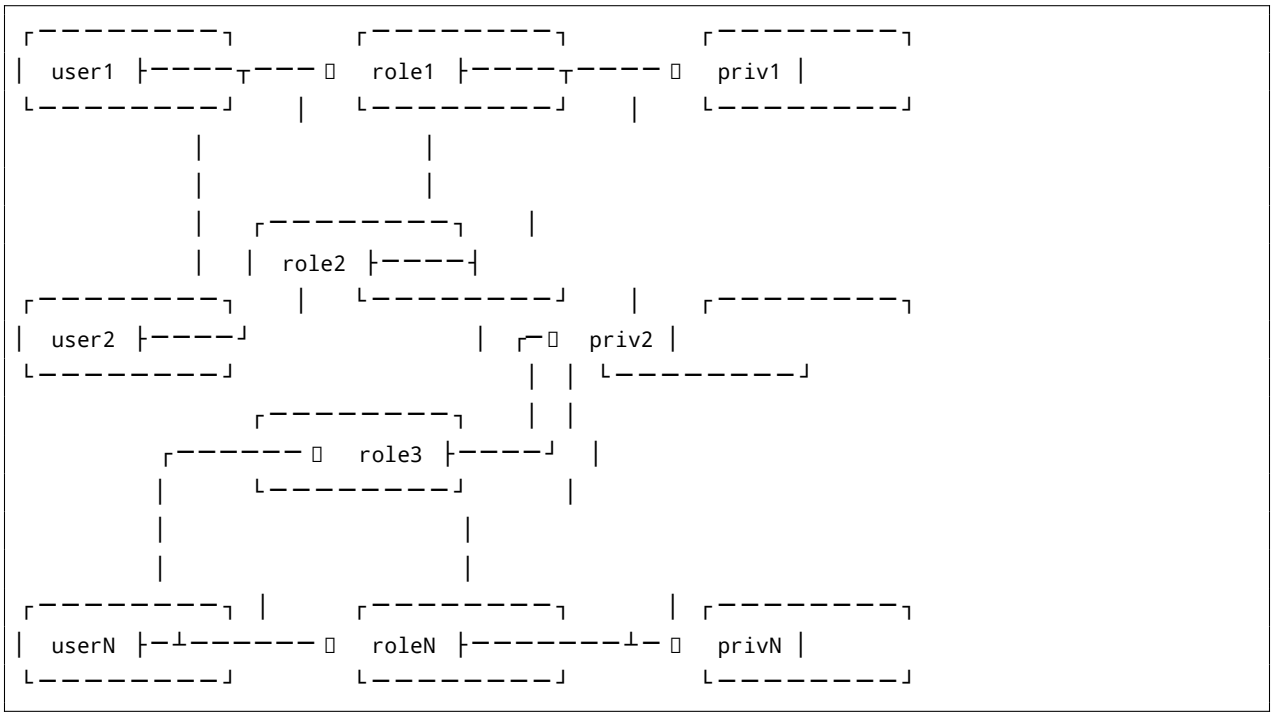
Doris 内置的鉴权方案

Doris 权限设计基于 RBAC（ Role-Based Access Control ）的权限管理模型，用户和角色关联，角色和权限关联，用户通过角色间接和权限关联。

当角色被删除时，用户自动失去该角色的所有权限。

当用户和角色取消关联，用户自动失去角色的所有权限。

当角色的权限被增加或删除，用户的权限也会随之变更。



如上图所示：

user1 和 user2 都是通过 role1 拥有了 priv1 的权限。

userN 通过 role3 拥有了 priv1 的权限，通过 roleN 拥有了 priv2 和 privN 的权限，因此 userN 同时拥有 priv1，priv2 和 privN 的权限。

为了方便用户操作，是可以直接给用户授权的，底层实现上，是为每个用户创建了一个专属于该用户的默认角色，当给用户授权时，实际上是在给该用户的默认角色授权。

默认角色不能被删除，不能被分配给其他人，删除用户时，默认角色也自动删除。

基于 Apache Ranger 的鉴权方案

请参阅基于 Apache Ranger 的鉴权方案

2.17.2.1.5 常见问题

权限项说明

1. 拥有 ADMIN 权限，或 GLOBAL 层级的 GRANT 权限的用户可以进行以下操作：
    - CREATE USER
    - DROP USER
    - ALTER USER
    - SHOW GRANTS
    - CREATE ROLE
    - DROP ROLE
    - ALTER ROLE
    - SHOW ROLES
    - SHOW PROPERTY FOR USER
2. GRANT/REVOKE
    - 拥有 ADMIN 权限，可以授予或撤销任意用户的权限。
    - 拥有 ADMIN 或 GLOBAL 层级 GRANT 权限可以把角色分配给用户。
    - 同时拥有相应层级的 GRANT 权限和要分配的权限，可以把权限分配给用户/角色。
3. SET PASSWORD
    - 拥有 ADMIN 权限，或者 GLOBAL 层级 GRANT 权限的用户，可以设置非 ROOT 用户的密码。
    - 普通用户可以设置自己对应的 User Identity 的密码。自己对应的 User Identity 可以通过 SELECT CURRENT_USER() 命令查看。
    - ROOT 用户可以修改自己的密码。

其他说明

1. Doris 初始化时，会自动创建如下用户和角色：
    - operator 角色：该角色拥有 Node_priv 和 Admin_priv，即对 Doris 的所有权限。
    - admin 角色：该角色拥有 Admin_priv，即除节点变更以外的所有权限。
    - root@‘%’：root 用户，允许从任意节点登陆，角色为 operator。
    - admin@‘%’：admin 用户，允许从任意节点登陆，角色为 admin。
2. 不支持删除或更改默认创建的用户，角色或用户的权限。
    - 不支持删除 root@‘%’ 和 admin@‘%’ 用户，但是允许创建和删除 root@‘xxx’ 和 admin@‘xxx’ 用户（xxx 指的是除了% 之外的 host）（Doris 会把这些用户视为普通用户）
    - 不支持撤销 root@‘%’ 和 admin@‘%’ 的默认角色
    - 不支持删除角色 operator 和 admin
    - 不支持操作角色 operator 和 admin 的权限

3. operator 角色的用户有且只有一个，即 Root。admin 角色的用户可以创建多个。

4. 一些可能产生冲突的操作说明

    1. 域名与 ip 冲突：
       假设创建了如下用户：
       ```
       CREATE USER user1@['domain'];
       ```
       并且授权：
       ```
       GRANT SELECT_PRIV ON *.* TO user1@['domain']
       ```
       该 domain 被解析为两个 IP：ip1 和 ip2。
       假设之后，我们对 user1@'ip1' 进行一次单独授权：
       ```
       GRANT ALTER_PRIV ON . TO user1@'ip1';
       ```
       则 user1@'ip1' 的权限会被修改为 Select_priv 和 Alter_priv。并且当我们再次变更 user1@['domain'] 的权限时，user1@'ip1' 也不会跟随改变。
    2. 重复 ip 冲突：
       假设创建了如下用户：

```
CREATE USER user1@'%' IDENTIFIED BY "12345";
CREATE USER user1@'192.%' IDENTIFIED BY "abcde";
```

```
在优先级上，``'192.%'`` 优先于 ``'%'``，因此，当用户 `user1` 从 `192.168.1.1`
    ↪ 这台机器尝试使用密码 ``'12345'`` 登陆 Doris 会被拒绝。
```

5. 忘记密码

   如果忘记了密码无法登陆 Doris，可以在 FE 的 config 文件中添加 skip_localhost_auth_check=true 参数，并且重启 FE，从而无密码在本机通过 root 登陆 Doris。

   登陆后，可以通过 SET PASSWORD 命令重置密码。

6. 任何用户都不能重置 root 用户的密码，除了 root 用户自己。

7. Admin_priv 权限只能在 GLOBAL 层级授予或撤销。

8. current_user() 和 user()

   用户可以通过 SELECT current_user() 和 SELECT user() 分别查看 current_user 和 user。其中 current_
   ↪ user 表示当前用户是以哪种身份通过认证系统的，而 user 则是用户当前实际的 User Identity。

   举例说明：

   假设创建了 user1@'192.%' 这个用户，然后以为来自 192.168.10.1 的用户 user1 登陆了系统，则此时的 current_user 为 user1@'192.%'，而 user 为 user1@'192.168.10.1'。

   所有的权限都是赋予某一个 current_user 的，真实用户拥有对应的 current_user 的所有权限。

2.17.2.1.6 最佳实践

这里举例一些 Doris 权限系统的使用场景。

1. 场景一

Doris 集群的使用者分为管理员（Admin）、开发工程师（RD）和用户（Client）。其中管理员拥有整个集群的所有权限，主要负责集群的搭建、节点管理等。开发工程师负责业务建模，包括建库建表、数据的导入和修改等。用户访问不同的数据库和表来获取数据。

在这种场景下，可以为管理员赋予 ADMIN 权限或 GRANT 权限。对 RD 赋予对任意或指定数据库表的 CREATE、DROP、ALTER、LOAD、SELECT 权限。对 Client 赋予对任意或指定数据库表 SELECT 权限。同时，也可以通过创建不同的角色，来简化对多个用户的授权操作。

2. 场景二

一个集群内有多个业务，每个业务可能使用一个或多个数据。每个业务需要管理自己的用户。在这种场景下。管理员用户可以为每个数据库创建一个拥有 DATABASE 层级 GRANT 权限的用户。该用户仅可以对用户进行指定的数据库的授权。

3. 黑名单

Doris 本身不支持黑名单，只有白名单功能，但我们可以通过某些方式来模拟黑名单。假设先创建了名为 `user@'192.%'` 的用户，表示允许来自 `192.*` 的用户登录。此时如果想禁止来自 `192.168.10.1` 的用户登录。则可以再创建一个用户 `cmy@'192.168.10.1'` 的用户，并设置一个新的密码。因为 `192.168.10.1` 的优先级高于 `192.%`，所以来自 `192.168.10.1` 将不能再使用旧密码进行登录。

2.17.2.2 身份认证

2.17.2.2.1 内置认证

关键概念

用户

在 Doris 中，一个 user_identity 唯一标识一个用户。user_identity 由两部分组成，user_name 和 host，其中 username 为用户名。host 标识用户端连接所在的主机地址。host 部分可以使用% 进行模糊匹配。如果不指定 host，默认为 ‘%’，即表示该用户可以从任意 host 连接到 Doris。

用户属性

用户属性直接附属于 user_name，而不是 user_identity，即 user@‘192.%’ 和 user@[‘domain’] 都拥有同一组用户属性。该属性属于 user，而不是 user@‘192.%’ 或 user@[‘domain’]。

用户属性包括但不限于：用户最大连接数、导入集群配置等等。

内置用户

内置用户是 Doris 默认创建的用户，并默认拥有一定的权限，包括 root 和 admin。初始密码都为空，fe 启动后，可以通过修改密码命令进行修改。不支持删除默认用户。

- root@‘%’：root 用户，允许从任意节点登录，角色为 operator。
- admin@‘%’：admin 用户，允许从任意节点登录，角色为 admin。

密码

用户登录的凭据，管理员创建用户时设置，也可以创建后由用户自己更改密码。

密码策略

Doris 支持以下密码策略，可以帮助用户更好的进行密码管理。

- PASSWORD_HISTORY

是否允许当前用户重置密码时使用历史密码。如 PASSWORD_HISTORY 10 表示禁止使用过去 10 次设置过的密码为新密码。如果设置为 PASSWORD_HISTORY DEFAULT，则会使用全局变量 password_history 中的值。0 表示不启用这个功能。默认为 0。

示例：

- 设置全局变量：SET GLOBAL password_history = 10
- 为用户设置：ALTER USER user1@'ip' PASSWORD_HISTORY 10
- PASSWORD_EXPIRE

设置当前用户密码的过期时间。如 PASSWORD_EXPIRE INTERVAL 10 DAY 表示密码会在 10 天后过期。PASSWORD ↪ _EXPIRE NEVER 表示密码不过期。如果设置为 PASSWORD_EXPIRE DEFAULT，则会使用全局变量 default_ ↪ password_lifetime 中的值（单位为天）。默认为 NEVER（或 0），表示不会过期。

示例：

- 设置全局变量：SET GLOBAL default_password_lifetime = 1
- 为用户设置：ALTER USER user1@'ip' PASSWORD_EXPIRE INTERVAL 10 DAY
- FAILED_LOGIN_ATTEMPTS 和 PASSWORD_LOCK_TIME

设置当前用户登录时，如果使用错误的密码登录 n 次后，账户将被锁定，并设置锁定时间。如 FAILED_LOGIN ↪ _ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY 表示如果 3 次错误登录，则账户会被锁定一天。管理员可以通过 ALTER USER 语句主动解锁被锁定的账户。

示例：

- 为用户设置：ALTER USER user1@'ip' FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY
- 密码强度

该功能由全局变量 validate_password_policy 控制。默认为 NONE/0，即不检查密码强度。如果设置为 STRONG/2，则密码必须包含“大写字母”，“小写字母”，“数字”和“特殊字符”中的 3 项，并且长度必须大于等于 8。

示例：

```
- `SET validate_password_policy=STRONG`
```

以上策略设置后，可以通过如下命令查看：

```
SHOW PROC "/auth/'<user>'@'<host>'";
```

注意，需要使用单引号分别包裹 user 和 host 部分。比如：

```
SHOW PROC "/auth/'root'@'%'";
SHOW PROC "/auth/'user1'@'127.0.0.1'";
```

认证机制

1. 客户端认证信息发送：客户端将用户的信息（如用户名、密码、数据库等）打包发送给 Doris 服务器。这些信息用于证明客户端的身份和请求访问的数据库。
2. 服务器认证：Doris 收到客户端的认证信息后，会进行验证。如果用户名、密码以及客户端的 IP 正确，并且该用户具有访问所选数据库的权限，则认证成功，Doris 会将用户个体映射到系统内的用户标识（User Identity）上。否则，认证失败，并返回相应的错误消息给客户端。

黑白名单

Doris 本身不支持黑名单，只有白名单功能，但我们可以通过某些方式来模拟黑名单。假设先创建了名为 `user@'192.%'` 的用户，表示允许来自 `192.*` 的用户登录。此时如果想禁止来自 `192.168.10.1` 的用户登录，则可以再创建一个用户 `cmy@'192.168.10.1'`，并设置一个新的密码。因为 `192.168.10.1` 的优先级高于 `192.%`，所以来自 `192.168.10.1` 的用户将不能再使用旧密码进行登录。

相关命令

- 创建用户：CREATE USER
- 查看用户：SHOW ALL GRANTS
- 修改用户：ALTER USER
- 修改密码：SET PASSWORD
- 删除用户：DROP USER
- 设置用户属性：SET PROPERTY
- 查看用户属性：SHOW PROPERTY

其它说明

1. 登录时 user_identity 优先级选择问题

   如上文介绍，`user_identity` 由 `user_name` 和 `host` 组成，但是用户登录的时候，只需要输入 `user_name`，所以由 Doris 根据客户端的 IP 来匹配相应的 `host`，从而决定使用哪个 `user_identity` 登录。

   如果根据客户端 ip 只能匹配到一个 `user_identity`，那么毫无疑问会匹配到这个 `user_identity`，但是当能够匹配到多个 `user_identity` 时，就会有如下的优先级问题。

   1. 域名与 ip 的优先级：

      假设创建了如下用户：

```
CREATE USER user1@['domain1'] IDENTIFIED BY "12345";
CREATE USER user1@'ip1'IDENTIFIED BY "abcde";
```

domain1 被解析为两个 IP：ip1 和 ip2。

在优先级上，ip 优先于 域名，因此，当用户 user1 从 ip1 这台机器尝试使用密码 '12345' 登录 Doris 会被拒绝。

2. 具体 ip 和 范围 ip 的优先级：

假设创建了如下用户：

```
CREATE USER user1@'%' IDENTIFIED BY "12345";
CREATE USER user1@'192.%' IDENTIFIED BY "abcde";
```

在优先级上，'192.%' 优先于 '%'，因此，当用户 user1 从 192.168.1.1 这台机器尝试使用密码 '12345' 登录 Doris 会被拒绝。

2. 忘记密码

   如果忘记了密码无法登录 Doris，可以在 FE 的 config 文件中添加 skip_localhost_auth_check=true 参数，并且重启 FE，从而无密码在 Fe 本机通过 root 登录 Doris。

   登录后，可以通过 SET PASSWORD 命令重置密码。

3. 任何用户都不能重置 root 用户的密码，除了 root 用户自己。

4. current_user() 和 user()

   用户可以通过 SELECT current_user() 和 SELECT user() 分别查看 current_user 和 user。其中 current_user 表示当前用户是以哪种身份通过认证系统的，而 user 则是用户当前实际的 User Identity。

   举例说明：

   假设创建了 user1@'192.%' 这个用户，然后以为来自 192.168.10.1 的用户 user1 登录了系统，则此时的 current_user 为 user1@'192.%'，而 user 为 user1@'192.168.10.1'。

   所有的权限都是赋予某一个 current_user 的，真实用户拥有对应的 current_user 的所有权限。

2.17.2.2.2 联邦认证

LDAP

接入第三方 LDAP 服务为 Doris 提供验证登录和组授权服务。####### LDAP 验证登录 LDAP 验证登录指的是接入 LDAP 服务的密码验证来补充 Doris 的验证登录。Doris 优先使用 LDAP 验证用户密码，如果 LDAP 服务中不存在该用户则继续使用 Doris 验证密码，如果 LDAP 密码正确但是 Doris 中没有对应账户则创建临时用户登录 Doris。

开启 LDAP 后，用户在 Doris 和 LDAP 中存在以下几种情况：

LDAP 用户	Doris 用户	密码	登录情况	登录 Doris 的用户
存在	存在	LDAP 密码	登录成功	Doris 用户
存在	存在	Doris 密码	登录失败	无
不存在	存在	Doris 密码	登录成功	Doris 用户
存在	不存在	LDAP 密码	登录成功	Ldap 临时用户

开启 LDAP 后，用户使用 mysql client 登录时，Doris 会先通过 LDAP 服务验证用户密码，如果 LDAP 存在用户且密码正确，Doris 则使用该用户登录；此时 Doris 若存在对应账户则直接登录该账户，如果不存在对应账户则为用户创建临时账户并登录该账户。临时账户具有具有相应对权限（参见 LDAP 组授权），仅对当前连接有效，doris 不会创建该用户，也不会产生创建用户对元数据。如果 LDAP 服务中不存在登录用户，则使用 Doris 进行密码认证。

以下假设已开启 LDAP 认证，配置 ldap_user_filter = (&(uid={login}))，且其他配置项都正确，客户端设置环境变量 LIBMYSQL_ENABLE_CLEARTEXT_PLUGIN=1

例如：

1. Doris 和 LDAP 中都存在账户：

存在 Doris 账户：`jack@'172.10.1.10'`，密码：`123456`

LDAP 用户节点存在属性：`uid: jack` 用户密码：`abcdef`

使用以下命令登录 Doris 可以登录 `jack@'172.10.1.10'` 账户：

```
mysql -hDoris_HOST -PDoris_PORT -ujack -p abcdef
```

使用以下命令将登录失败：

```
mysql -hDoris_HOST -PDoris_PORT -ujack -p 123456
```

2. LDAP 中存在用户，Doris 中不存在对应账户：

LDAP 用户节点存在属性：`uid: jack` 用户密码：`abcdef`

使用以下命令创建临时用户并登录 jack@‘%’，临时用户具有基本权限 DatabasePrivs：Select_priv，用户退出登录后 Doris 将删除该临时用户：

```
mysql -hDoris_HOST -PDoris_PORT -ujack -p abcdef
```

3. LDAP 不存在用户：

存在 Doris 账户：`jack@'172.10.1.10'`，密码：`123456`

使用 Doris 密码登录账户，成功：

```
mysql -hDoris_HOST -PDoris_PORT -ujack -p 123456
```

LDAP 组授权

DLAP 用户 dn 是 LDAP 组节点的 “member” 属性则 Doris 认为用户属于该组。LDAP 组授权是将 LDAP 中的 group 映射到 Doris 中的 role，并将所有对应的 role 权限授予登录用户，用户退出登录后 Doris 会撤销对应的 role 权限。在使用 LDAP 组授权前应该在 Doris 中创建相应的 role，并为 role 授权。

登录用户权限跟 Doris 用户和组权限有关，见下表：

LDAP 用户	Doris 用户	登录用户的权限
存在	存在	LDAP 组权限 + Doris 用户权限
不存在	存在	Doris 用户权限
存在	不存在	LDAP 组权限

如果登录的用户为临时用户，且不存在组权限，则该用户默认具有 information_schema 的 select_priv 权限

举例：

LDAP 用户 dn 是 LDAP 组节点的 member 属性则认为用户属于该组，Doris 会截取组 dn 的第一个 Rdn 作为组名。

例如用户 dn 为 uid=jack,ou=aidp,dc=domain,dc=com，组信息如下：

```
dn: cn=doris_rd,ou=group,dc=domain,dc=com
objectClass: groupOfNames
member: uid=jack,ou=aidp,dc=domain,dc=com
```

则组名为 doris_rd。

假如 jack 还属于 LDAP 组 doris_qa、doris_pm；Doris 存在 role：doris_rd、doris_qa、doris_pm，在使用 LDAP 验证登录后，用户不但具有该账户原有的权限，还将获得 role doris_rd、doris_qa 和 doris_pm 的权限。

> 注意：
>
> user 属于哪个 group 和 LDAP 树的组织结构无关，示例部分的 user2 并不一定属于 group2 若想让 user2 属于 group2，需要在 group2 的 member 属性中添加 user2 ####### LDAP 示例 ######## 更改 Doris 配置 1. 在 fe/conf/fe.conf 文件中配置认证方式为 ldap authentication_type=ldap。 2. 在 fe/conf/ldap.conf 文件中配置 LDAP 基本信息，3. 设置 LDAP 管理员密码：配置好 ldap.conf 文件后启动 fe，使用 root 或 admin 账号登录 Doris，执行 sql

```
set ldap_admin_password = password('ldap_admin_password');
```

使用 mysql 客户端登录

```
mysql -hDORIS_HOST -PDORIS_PORT -u user -p --enable-cleartext-plugin
输入 ldap 密码
```

注：使用其它客户端登录可以参考下文中客户端如何使用明文登录 ####### LDAP 信息缓存

为了避免频繁访问 LDAP 服务，Doris 会将 LDAP 信息缓存到内存中，可以通过 ldap.conf 中的 ldap_user_cache_timeout_s 配置项指定 LDAP 用户的缓存时间，默认为 12 小时；在修改了 LDAP 服务中的信息或者修改了 Doris 中 LDAP 用户组对应的 Role 权限后，可能因为缓存而没有及时生效，可以通过 refresh ldap 语句刷新缓存，详细查看REFRESH-LDAP。

LDAP 验证的局限

- 目前 Doris 的 LDAP 功能只支持明文密码验证，即用户登录时，密码在 client 与 fe 之间、fe 与 LDAP 服务之间以明文的形式传输。

常见问题

- 怎么判断 LDAP 用户在 doris 中有哪些角色？

使用 LDAP 用户在 doris 中登录，`show grants;`能查看当前用户有哪些角色。其中 ldapDefaultRole 是每个 ldap 用户在 doris 中都有的默认角色。

- LDAP 用户在 doris 中的角色比预期少怎么排查？

1. 通过`show roles;`查看预期的角色在 doris 中是否存在，如果不存在，需要通过`CREATE ROLE rol_name;`创建角色。
2. 检查预期的 group 是否在`ldap_group_basedn`对应的组织结构下。
3. 检查预期 group 是否包含 member 属性。
4. 检查预期 group 的 member 属性是否包含当前用户。####### LDAP 相关概念在 LDAP 中，数据是按照树型结构组织的。

示例（下文的介绍都将根据这个例子进行展开）

```
- dc=example,dc=com
 - ou = ou1
   - cn = group1
   - cn = user1
 - ou = ou2
   - cn = group2
     - cn = user2
 - cn = user3
```

LDAP 名词解释

- dc(Domain Component): 可以理解为一个组织的域名，作为树的根结点
- dn(Distinguished Name): 相当于唯一名称，例如 user1 的 dn 为 cn=user1,ou=ou1,dc=example,dc=com user2 的 dn 为 cn=user2,cn=group2,ou=ou2,dc=example,dc=com
- rdn(Relative Distinguished Name): dn 的一部分，user1 的四个 rdn 为 cn=user1 ou=ou1 dc=example 和 dc=com
- ou(Organization Unit): 可以理解为子组织，user 可以放在 ou 中，也可以直接放在 example.com 域中

- cn(common name): 名字
- group: 组，可以理解为 doris 的角色
- user: 用户，和 doris 的用户等价
- objectClass：可以理解为每行数据的类型，比如怎么区分 group1 是 group 还是 user，每种类型的数据下面要求有不同的属性，比如 group 要求有 cn 和 member（user 列表），user 要求有 cn,password,uid 等 ####### 客户端如何使用明文登录 ######## MySql Client 客户端使用 LDAP 验证需要启用 mysql 客户端明文验证插件，使用命令行登录 Doris 可以使用下面两种方式之一启用 mysql 明文验证插件：
- 设置环境变量 LIBMYSQL_ENABLE_CLEARTEXT_PLUGIN 值 1

例如在 linux 或者 mac 环境中可以使用：

```
echo "export LIBMYSQL_ENABLE_CLEARTEXT_PLUGIN=1" >> ~/.bash_profile && source ~/.bash_profile
```

- 每次登录 Doris 时添加参数 --enable-cleartext-plugin

```
mysql -hDORIS_HOST -PDORIS_PORT -u user -p --enable-cleartext-plugin

输入 ldap 密码
```

Jdbc Client

使用 Jdbc Client 登录 Doris 时，需要自定义 plugin。

首先，创建一个名为 MysqlClearPasswordPluginWithoutSSL 的类，继承自 MysqlClearPasswordPlugin。在该类中，重写 requiresConfidentiality() 方法，并返回 false。

```
public class MysqlClearPasswordPluginWithoutSSL extends MysqlClearPasswordPlugin {
@Override
public boolean requiresConfidentiality() {
    return false;
  }
}
```

在获取数据库连接时，需要将自定义的 plugin 配置到属性中

即（xxx 为自定义类的包名）- authenticationPlugins=xxx.xxx.xxx.MysqlClearPasswordPluginWithoutSSL - defaultAuthenticationPlugin=xxx.xxx.xxx.MysqlClearPasswordPluginWithoutSSL - disabledAuthenticationPlugins=com.mysql.jdbc.authentication.MysqlClearPasswordPlu

eg:

```
jdbcUrl = "jdbc:mysql://localhost:9030/mydatabase?authenticationPlugins=xxx.xxx.xxx.
    ↪ MysqlClearPasswordPluginWithoutSSL&defaultAuthenticationPlugin=xxx.xxx.xxx.
    ↪ MysqlClearPasswordPluginWithoutSSL&disabledAuthenticationPlugins=com.mysql.jdbc.
    ↪ authentication.MysqlClearPasswordPlugin";
```

2.17.2.3 鉴权管控

2.17.2.3.1 内置鉴权

关键概念

鉴权是指根据用户身份限制其访问和操作 Doris 资源的机制。

Doris 基于 RBAC（Role-Based Access Control）的权限管理模型进行权限控制。

权限

权限作用的对象是节点、数据目录、数据库或表。不同的权限代表不同的操作许可。

所有权限

权限	对象类型	描述
Admin_priv	Global	超管权限。
Node_priv	Global	节点变更权限。包括 FE、 BE、 BROKER 节点的添加、删除、下线等操作。
Grant_priv	Global,Catalog,Database,Table,Resource,WorkloadGroup	权限变更权限。允许执行包括授权、撤权、添加/删除/变更用户/角色等操作。给其他用户/角色授权时，2.1.2 版本之前，当前用户只需要相应层级的 Grant_priv 权限，2.1.2 版本之后当前用户也要有想要授权的资源的权限。给其他用户分配角色时，要有 Global 级别的 Grant_priv 权限。
Select_priv	Global,Catalog,Database,Table,Column	对数据目录、数据库、表、列的只读权限。
Load_priv	Global,Catalog,Database,Table	对数据目录、数据库、表的写权限。包括 Load、 Insert、 Delete 等。
Alter_priv	Global,Catalog,Database,Table	对数据目录、数据库、表的更改权限。包括重命名库/表、添加/删除/变更列、添加/删除分区等操作。
Create_priv	Global,Catalog,Database,Table	创建数据目录、数据库、表、视图的权限。
Drop_priv	Global,Catalog,Database,Table	删除数据目录、数据库、表、视图的权限。
Usage_priv	Resource,WorkloadGroup	Resource 和 Workload Group 的使用权限。
Show_view_priv	Global,Catalog,Database,Table	执行 SHOW CREATE VIEW 的权限。

角色

Doris 可以创建自定义命名的角色。角色可以被看做是一组权限的集合。新创建的用户可以被赋予某一角色，则自动被赋予该角色所拥有的权限。后续对角色的权限变更，也会体现在所有属于该角色的用户权限上。

内置角色

内置角色是 Doris 默认创建的角色，并默认拥有一定的权限，包括 operator 和 admin。

- operator : 拥有 Admin_priv 和 Node_priv
- admin: 拥有 Admin_priv

用户

在 Doris 中，一个 user_identity 唯一标识一个用户。user_identity 由两部分组成，user_name 和 host，其中 username 为用户名。host 标识用户端连接所在的主机地址。

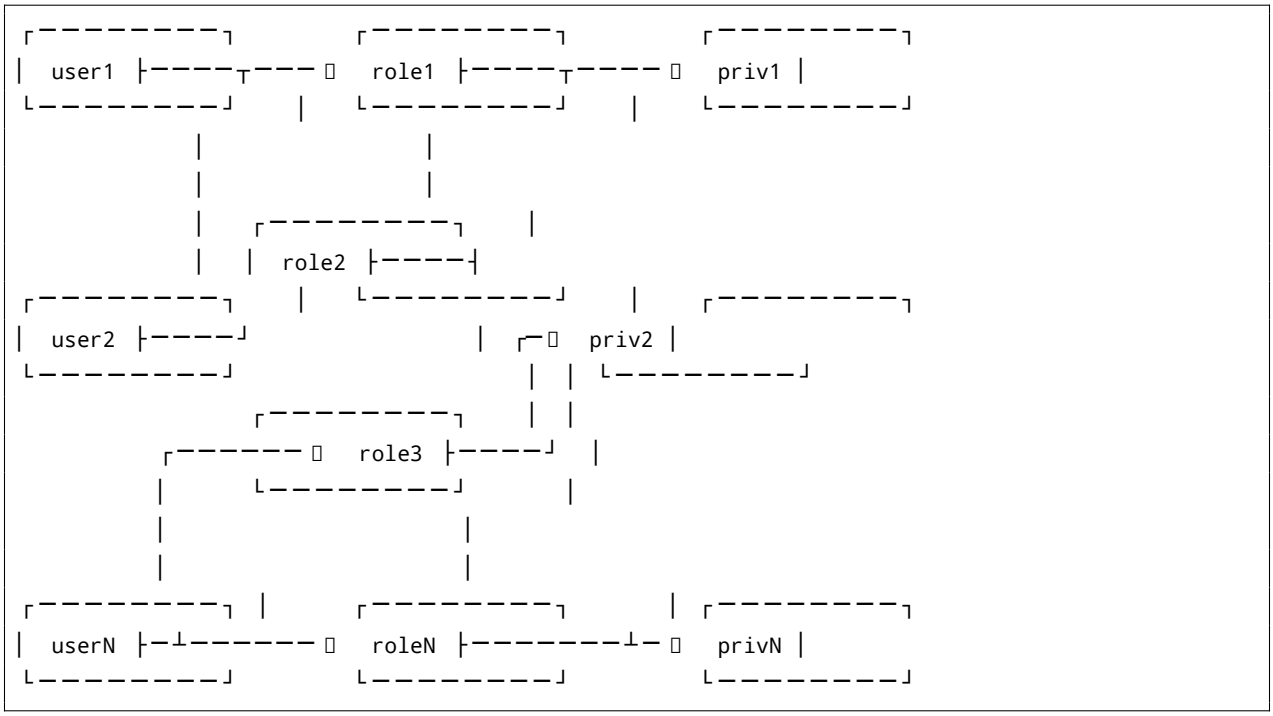
鉴权机制

Doris 权限设计基于 RBAC（Role-Based Access Control）的权限管理模型，用户和角色关联，角色和权限关联，用户通过角色间接和权限关联。

当角色被删除时，用户自动失去该角色的所有权限。

当用户和角色取消关联，用户自动失去角色的所有权限。

当角色的权限被增加或删除，用户的权限也会随之变更。



如上图所示：

user1 和 user2 都是通过 role1 拥有了 priv1 的权限。

userN 通过 role3 拥有了 priv1 的权限，通过 roleN 拥有了 priv2 和 privN 的权限，因此 userN 同时拥有 priv1，priv2 和 privN 的权限。

注意事项

- 为了方便用户操作，是可以直接给用户授权的，底层实现上，是为每个用户创建了一个专属于该用户的默认角色，当给用户授权时，实际上是在给该用户的默认角色授权。
- 默认角色不能被删除，不能被分配给其他人，删除用户时，默认角色也自动删除。

相关命令

- 授权/分配角色：GRANT
- 撤权/撤销角色：REVOKE

- 创建角色：CREATE ROLE
- 删除角色：DROP ROLE
- 修改角色：ALTER ROLE
- 查看当前用户权限和角色：SHOW GRANTS
- 查看所有用户权限和角色：SHOW ALL GRANTS
- 查看已创建的角色：SHOW ROLES
- 查看支持的所有权限项：SHOW PRIVILEGES

最佳实践

这里举例一些 Doris 权限系统的使用场景。

1. 场景一

Doris 集群的使用者分为管理员（Admin）、开发工程师（RD）和用户（Client）。其中管理员拥有整个集群的所有权限，主要负责集群的搭建、节点管理等。开发工程师负责业务建模，包括建库建表、数据的导入和修改等。用户访问不同的数据库和表来获取数据。

在这种场景下，可以为管理员赋予 ADMIN 权限或 GRANT 权限。对 RD 赋予对任意或指定数据库表的 CREATE、DROP、ALTER、LOAD、SELECT 权限。对 Client 赋予对任意或指定数据库表 SELECT 权限。同时，也可以通过创建不同的角色，来简化对多个用户的授权操作。

2. 场景二

一个集群内有多个业务，每个业务可能使用一个或多个数据。每个业务需要管理自己的用户。在这种场景下。管理员用户可以为每个数据库创建一个拥有 DATABASE 层级 GRANT 权限的用户。该用户仅可以对用户进行指定的数据库的授权。

2.17.2.3.2 Ranger 鉴权

Apache Ranger 是一个用来在 Hadoop 平台上进行监控，启用服务，以及全方位数据安全访问管理的安全框架。使用 ranger 后，会通过在 Ranger 侧配置权限代替在 Doris 中执行 Grant 语句授权。Ranger 的安装和配置见下文：安装和配置 Doris Ranger 插件

Ranger 示例

更改 Doris 配置

1. 在 fe/conf/fe.conf 文件中配置鉴权方式为 ranger access_controller_type=ranger-doris
2. 在所有 FE 的 conf 目录创建 `ranger-doris-security.xml` 文件，内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <property>
        <name>ranger.plugin.doris.policy.cache.dir</name>
        <value>/path/to/ranger/cache/</value>
```

```
    </property>
    <property>
        <name>ranger.plugin.doris.policy.pollIntervalMs</name>
        <value>30000</value>
    </property>
    <property>
        <name>ranger.plugin.doris.policy.rest.client.connection.timeoutMs</name>
        <value>60000</value>
    </property>
    <property>
        <name>ranger.plugin.doris.policy.rest.client.read.timeoutMs</name>
        <value>60000</value>
    </property>
    <property>
        <name>ranger.plugin.doris.policy.rest.url</name>
        <value>http://172.21.0.32:6080</value>
    </property>
    <property>
        <name>ranger.plugin.doris.policy.source.impl</name>
        <value>org.apache.ranger.admin.client.RangerAdminRESTClient</value>
    </property>
    <property>
        <name>ranger.plugin.doris.service.name</name>
        <value>doris</value>
    </property>
</configuration>
```

其中需要将 ranger.plugin.doris.policy.cache.dir 和 ranger.plugin.doris.policy.rest.url 改为实际值。3. 启动集群 ####### 权限示例 1. 在 Doris 中创建 user1。2. 在 Doris 中，先使用 admin 用户创建一个 Catalog：hive。3. 在 Ranger 中创建 user1。

全局权限

相当于 Doris 内部授权语句的 grant select_priv on *.*.* to user1; - catalog 同级下拉框可以找到 global 选项 - 输入框里只能输入 *

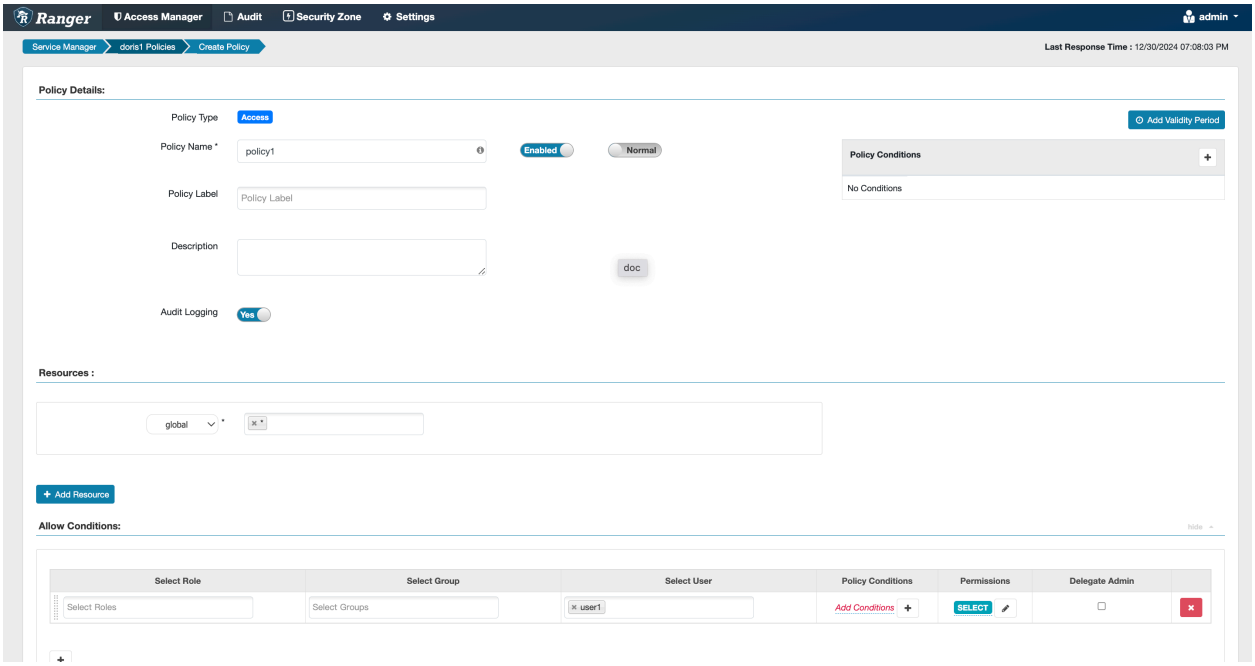


图 102: global

Catalog 权限

相当于 Doris 内部授权语句的 `grant select_priv on hive.*.* to user1;`

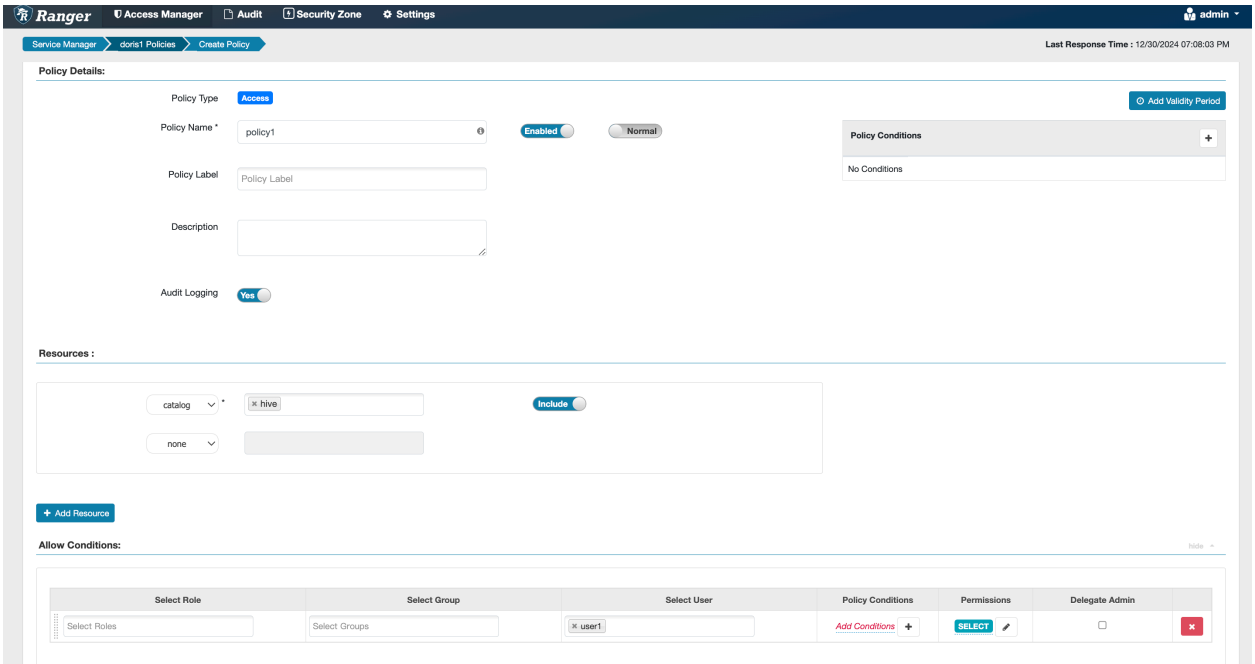


图 103: catalog

Database 权限

相当于 Doris 内部授权语句的 `grant select_priv on hive.tpch.* to user1;`

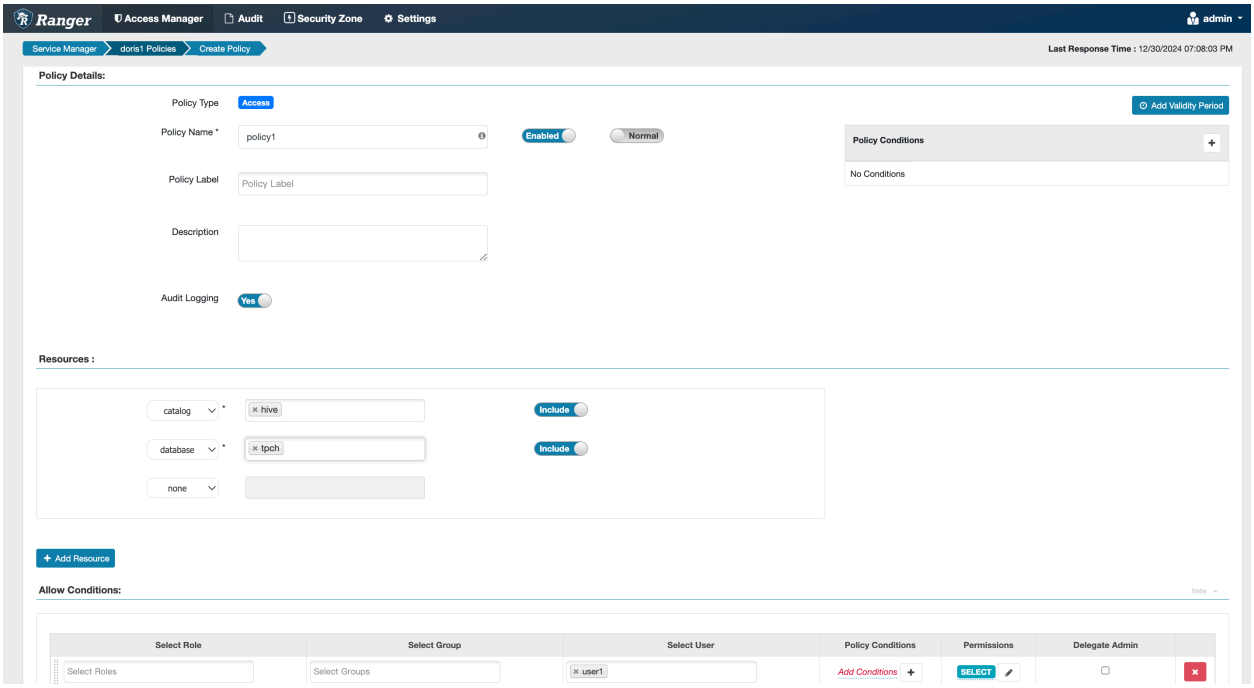


图 104: database

Table 权限

> 这里的 table 泛指表/视图/异步物化视图

相当于 Doris 内部授权语句的 `grant select_priv on hive.tpch.user to user1;`

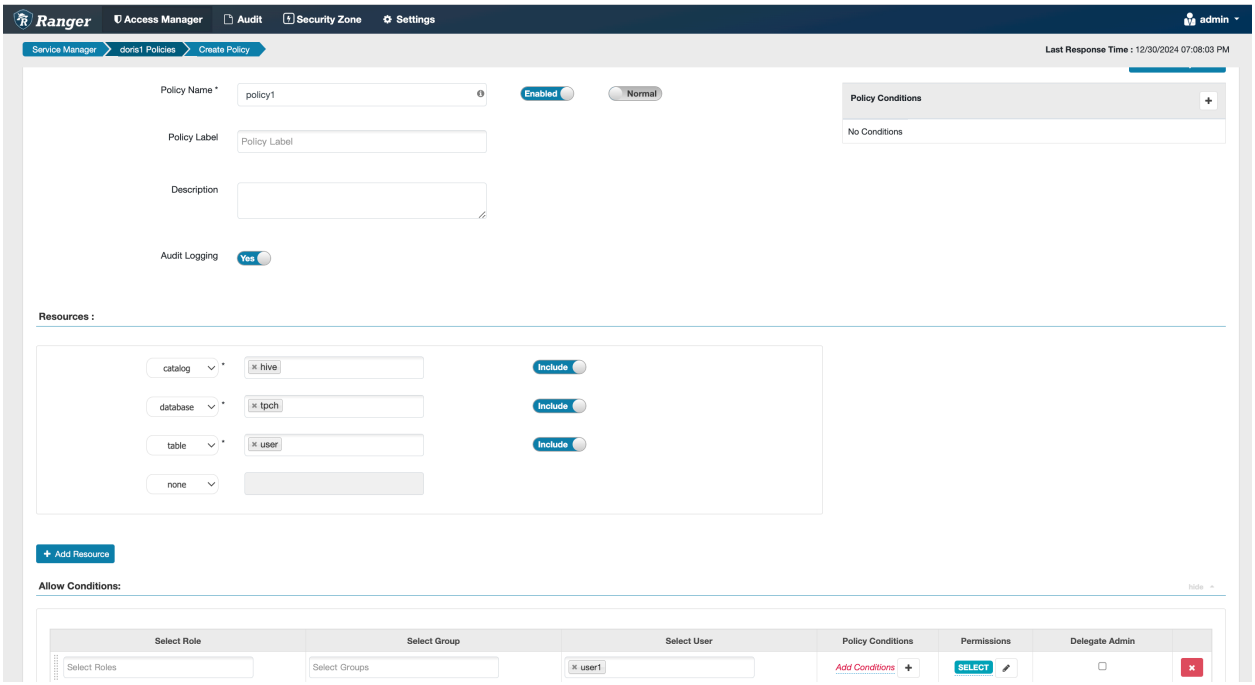


图 105: table

列权限

相当于 Doris 内部授权语句的 `grant select_priv(name,age)on hive.tpch.user to user1;`

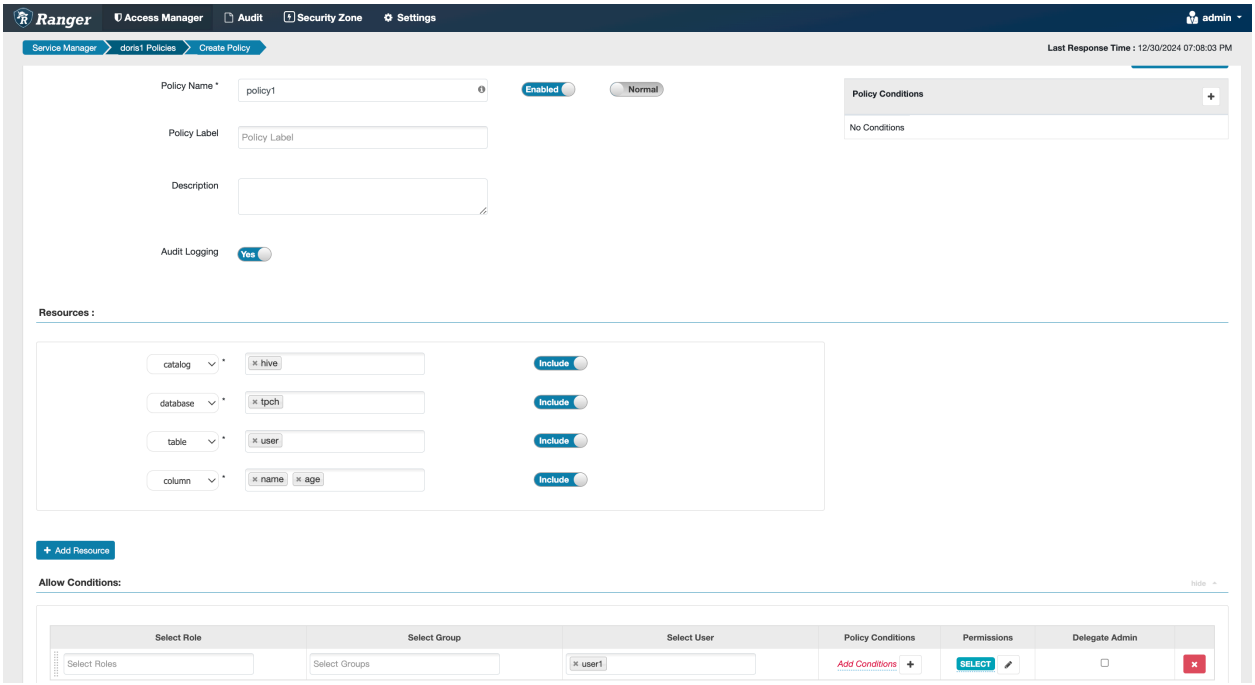


图 106: column

Resource 权限

相当于 Doris 内部授权语句的 `grant usage_priv on resource 'resource1' to user1;` - catalog 同级下拉框可以找到 resource 选项

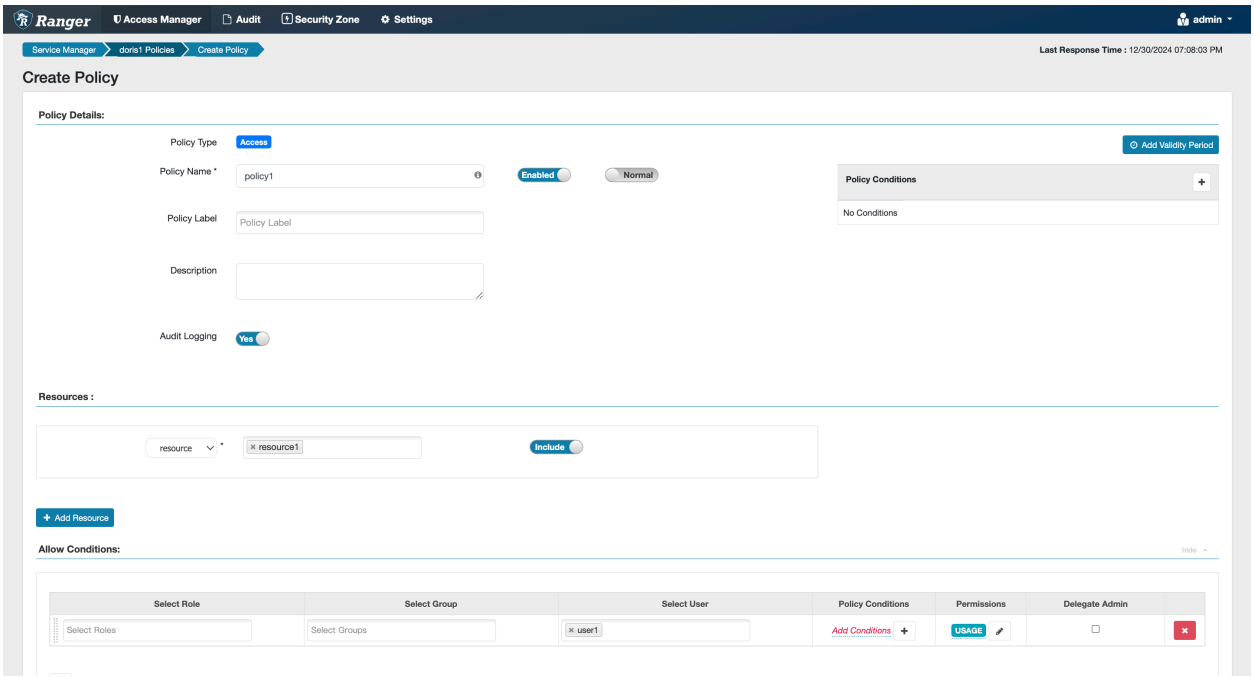


图 107: resource

Workload Group 权限

相当于 Doris 内部授权语句的 `grant usage_priv on workload group 'group1' to user1;` - catalog 同级下拉框可以找到 workload group 选项

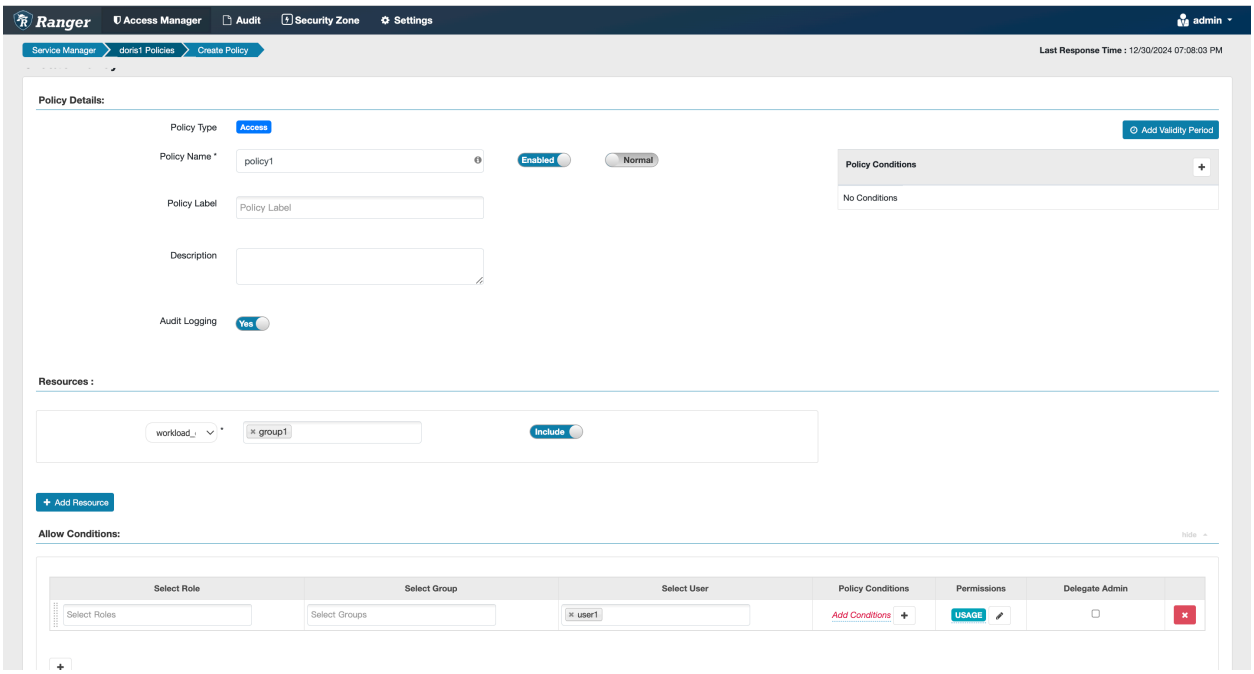


图 108: group1

Compute Group 权限

3.0.6 版本支持

相当于 Doris 内部授权语句的 `grant usage_priv on compute group 'group1' to user1;` - catalog 同级下拉框可以找到 compute group 选项

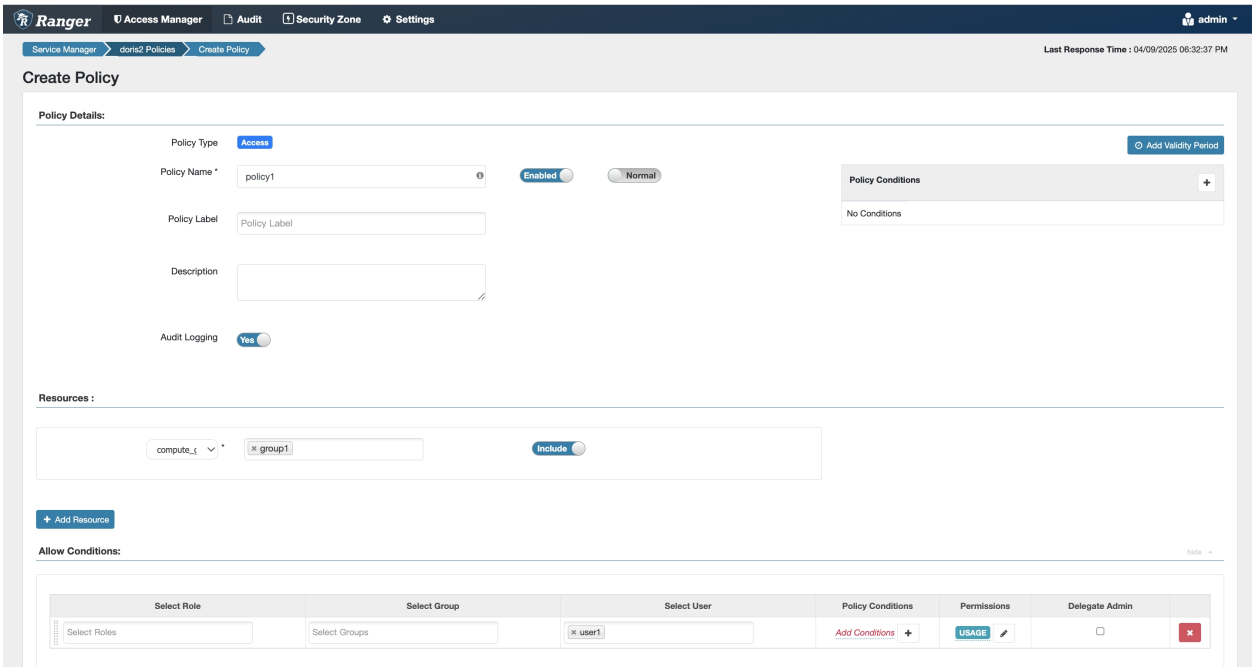


图 109: compute group

Storage Vault 权限

3.0.6 版本支持

相当于 Doris 内部授权语句的 `grant usage_priv on storage vault 'vault1' to user1;` - catalog 同级下拉框可以找到 storage vault 选项

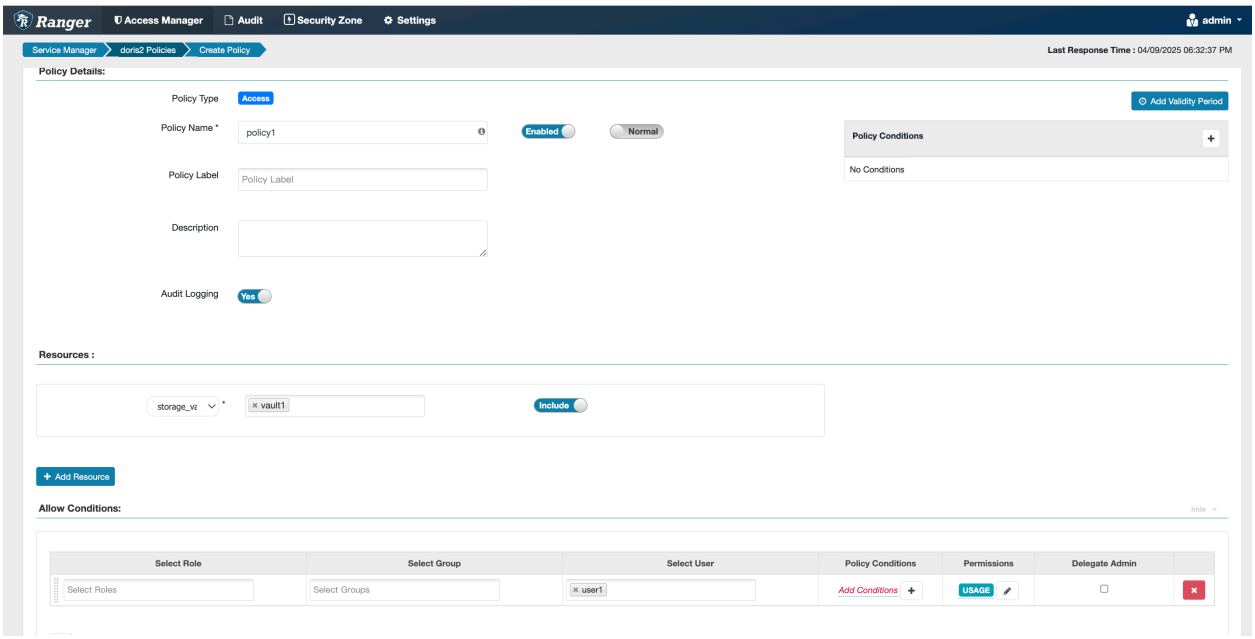


图 110: storage vault

行权限示例

2.1.3 版本支持

1. 参考权限示例给 user1 分配 internal.db1.user 表的 select 权限。
2. 在 Ranger 中添加一个 Row Level Filter policy

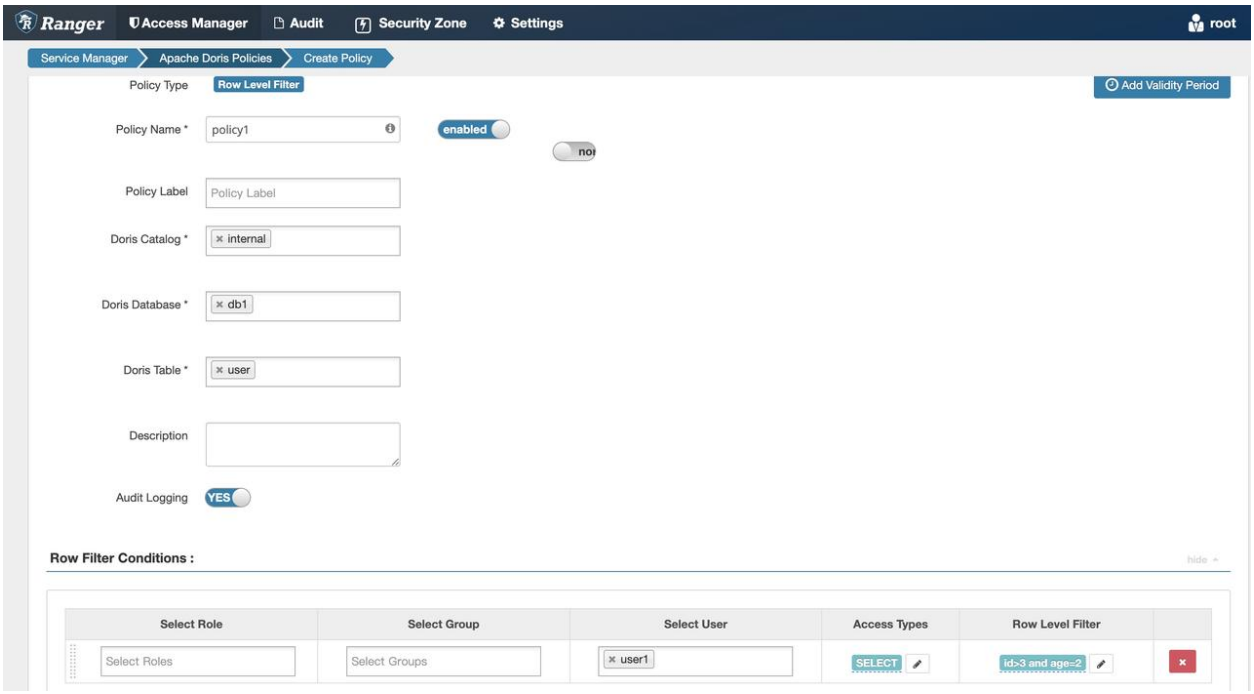


图 111: Row Policy 示例

3. 使用 user1 登录 Doris。执行 `select * from internal.db1.user`，只能看到满足 `id > 3` 且 `age = 2` 的数据。

数据脱敏示例

2.1.3 版本支持

1. 参考权限示例给 user1 分配 internal.db1.user 表的 select 权限。
2. 在 Ranger 中添加一个 Masking policy

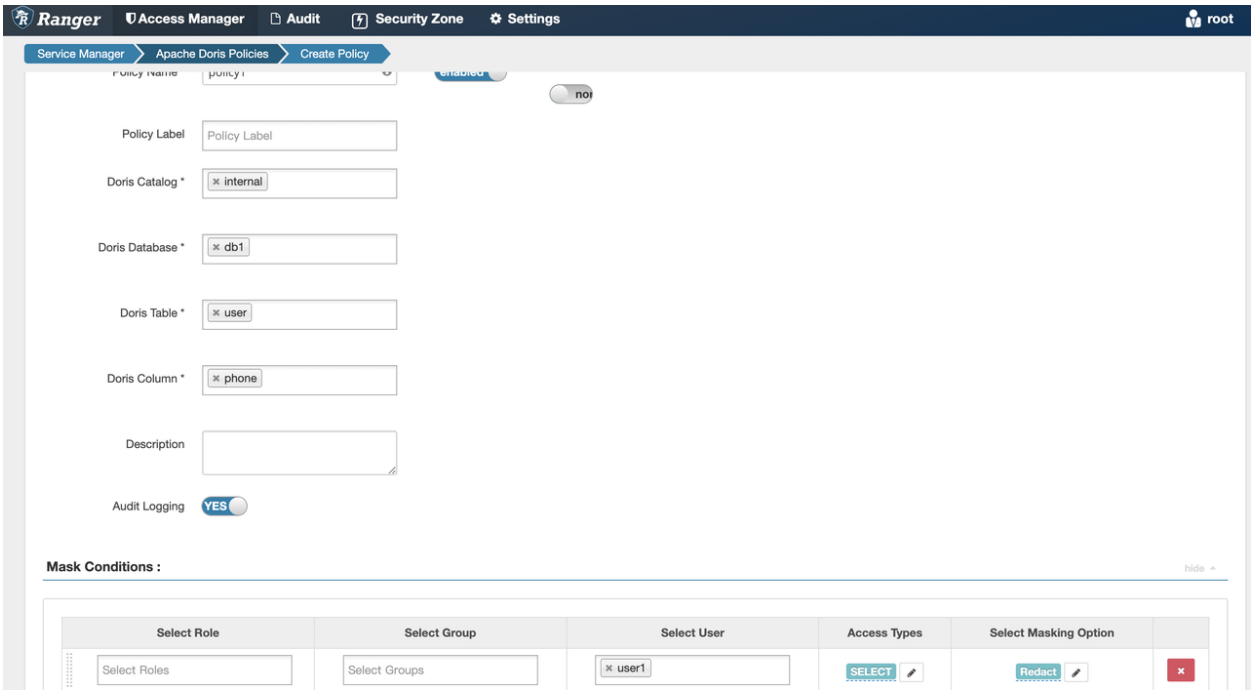


图 112: Data Mask 示例

3. 使用 user1 登录 Doris。执行 `select * from internal.db1.user`，看到的 phone 是按照指定规则脱敏后的数据。###### 常见问题
4. ranger 访问失败，怎么查看日志

在所有 FE 的 conf 目录创建 log4j.properties 文件，内容如下：

```
log4j.rootLogger = warn,stdout,D

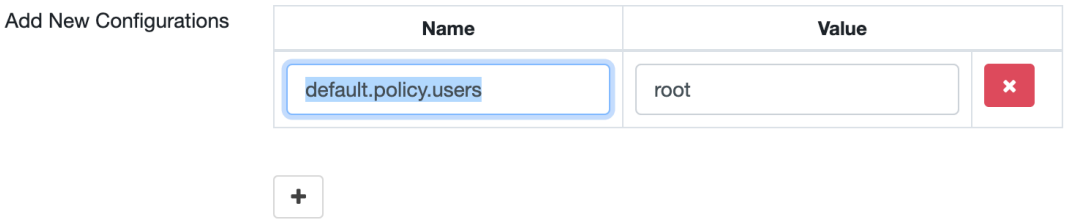
log4j.appender.stdout = org.apache.log4j.ConsoleAppender
log4j.appender.stdout.Target = System.out
log4j.appender.stdout.layout = org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern = [%-5p] %d{yyyy-MM-dd HH:mm:ss,SSS} method:%l
    ↪ %n%m%n

log4j.appender.D = org.apache.log4j.DailyRollingFileAppender
log4j.appender.D.File = /path/to/fe/log/ranger.log
log4j.appender.D.Append = true
log4j.appender.D.Threshold = INFO
log4j.appender.D.layout = org.apache.log4j.PatternLayout
log4j.appender.D.layout.ConversionPattern = %-d{yyyy-MM-dd HH:mm:ss}  [ %t:%r ] - [ %p ]  %m%
    ↪ n
```

其中 `log4j.appender.D.File` 改为实际值，用于存放 Ranger 插件的日志。2. 配置了 Row Level Filter policy，但是用户查询时报没有权限

Row Level Filter policy 仅用来限制用户访问表中数据的特定记录，仍需通过 ACCESS POLICY 为用户授权 3. 创建服务后，默认仅 admin 用户有权限，root 用户没有权限

如图所示，创建服务的时候，添加配置 default.policy.users ，如需配置多个用户拥有全部权限，用 , 分隔



4. 使用 ranger 鉴权后，内部授权还有用么？

不能用，也不能创建/删除角色

安装和配置 Doris Ranger 插件

安装插件

1. 下载以下文件

   - ranger-doris-plugin-3.0.0-SNAPSHOT.jar
   - mysql-connector-java-8.0.25.jar

2. 将下载好的文件放到 Ranger 服务的 ranger-plugins/doris 目录下，如：

```
/usr/local/service/ranger/ews/webapp/WEB-INF/classes/ranger-plugins/doris/ranger-doris-plugin
    ↪ -3.0.0-SNAPSHOT.jar
/usr/local/service/ranger/ews/webapp/WEB-INF/classes/ranger-plugins/doris/mysql-connector-java
    ↪ -8.0.25.jar
```

3. 重启 Ranger 服务。

4. 下载 ranger-servicedef-doris.json

5. 执行以下命令上传定义文件到 Ranger 服务：

```
curl -u user:password -X POST \
    -H "Accept: application/json" \
    -H "Content-Type: application/json" \
    http://172.21.0.32:6080/service/plugins/definitions \
    -d@ranger-servicedef-doris.json
```

其中用户名密码是登录 Ranger WebUI 所使用的用户名密码。

服务地址端口可以再 ranger-admin-site.xml 配置文件的 ranger.service.http.port 配置项查看。

如执行成功，会返回 Json 格式的服务定义，如：

```
{
  "id": 207,
  "guid": "d3ff9e41-f9dd-4217-bb5f-3fa9996454b6",
  "isEnabled": true,
  "createdBy": "Admin",
  "updatedBy": "Admin",
  "createTime": 1705817398112,
  "updateTime": 1705817398112,
  "version": 1,
  "name": "doris",
  "displayName": "Apache Doris",
  "implClass": "org.apache.ranger.services.doris.RangerServiceDoris",
  "label": "Doris",
  "description": "Apache Doris",
  "options": {
    "enableDenyAndExceptionsInPolicies": "true"
  },
  ...
}
```

如想重新创建，则可以使用以下命令删除服务定义后，再重新上传：

```
curl -v -u user:password -X DELETE \
http://172.21.0.32:6080/service/plugins/definitions/207
```

其中 207 是创建时返回的 id。删除前，需在 Ranger WebUI 界面删除已创建的 Doris 服务。

也可以通过以下命令列举当前已添加的服务定义，以便获取 id：

```
curl -v -u user:password -X GET \
http://172.21.0.32:6080/service/plugins/definitions/
```

配置插件

安装完毕后，打开 Ranger WebUI，可以再 Service Manger 界面中看到 Apache Doris 插件：

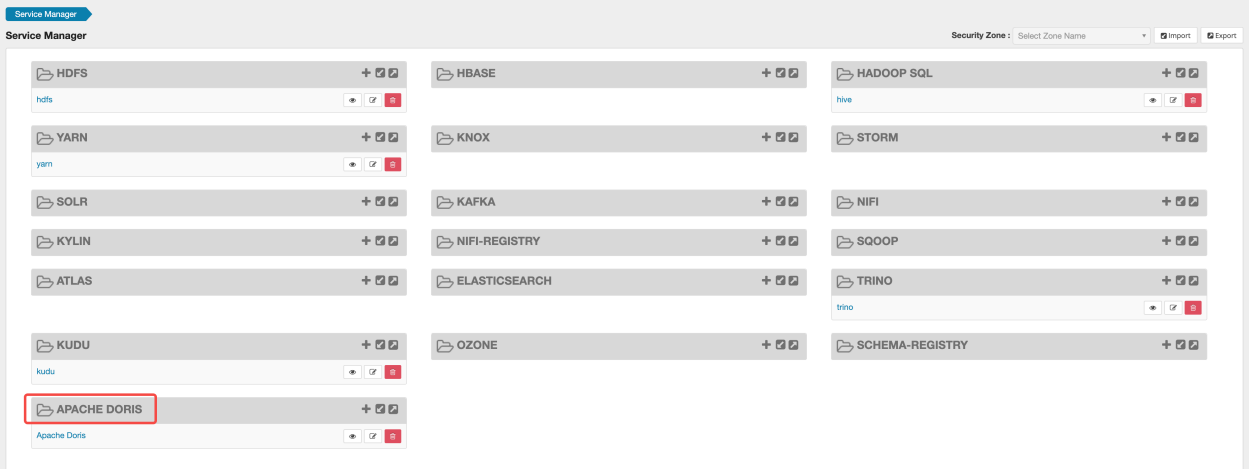


图 113: ranger

点击插件旁边的 + 号添加一个 Doris 服务：

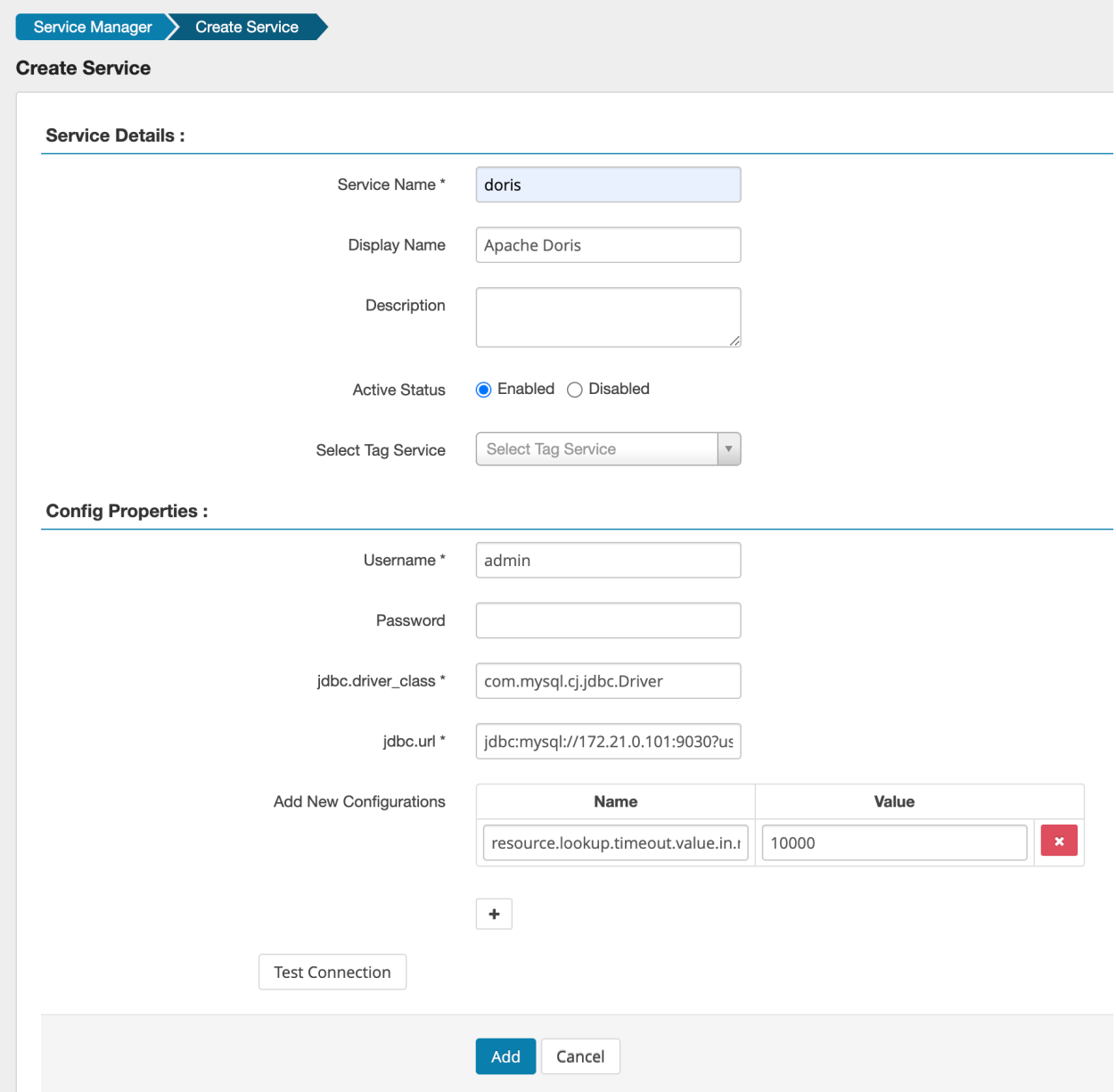


图 114: ranger2

Config Properties 部分参数含义如下：

- `Username/Pasword`：Doris 集群的用户名密码，这里建议使用 Admin 用户。
- `jdbc.driver_class`：连接 Doris 使用的 JDBC 驱动。`com.mysql.cj.jdbc.Driver`
- `jdbc.url`：Doris 集群的 JDBC url 连接串。`jdbc:mysql://172.21.0.101:9030?useSSL=false`
- 额外参数：
  - `resource.lookup.timeout.value.in.ms`：获取元信息的超时时间，建议填写 10000，即 10 秒。

可以点击 `Test Connection` 检查是否可以联通。

之后点击 Add 添加服务。

之后，可以在 Service Manger 界面的 Apache Doris 插件中看到创建的服务，点击服务，即可开始配置 Ranger。

2.17.2.3.3 数据访问控制

行权限

使用 Doris 中的行级策略，您可以对敏感数据进行精细访问控制。您可以根据在表级别定义的安全策略，来决定哪些用户或角色可以访问表中数据的特定记录。

机制

相当于为配置了 Row Policy 的用户在查询时自动加上 Row Policy 中设置的谓词

限制

不能为默认用户 root 和 admin 设置 Row Policy

相关命令

- 查看行权限策略SHOW ROW POLICY
- 创建行权限策略CREATE ROW POLICY ####### 行权限示例

1. 限制 test 用户仅能查询 table1 表中 c1=‘a’ 的数据

```
CREATE ROW POLICY test_row_policy_1 ON test.table1
AS RESTRICTIVE TO test USING (c1 = 'a');
```

列权限

使用 Doris 中的列权限，您可以对表进行精细访问控制。您可以只授予一个表中特定列的权限，来决定哪些用户或角色可以访问表的特定列

目前列权限仅支持 Select_priv

相关命令

- 授权：GRANT
- 回收权限：REVOKE

列权限示例

1. 授权 user1 查询 tbl 表的列：col1，col2.

```
GRANT Select_priv(col1,col2) ON ctl.db.tbl TO user1
```

数据脱敏

数据脱敏是一种保护敏感数据的方法，它通过对原始数据进行修改、替换或隐藏，使得脱敏后的数据在保持一定格式和特性的同时，不再包含敏感信息。

例如，管理员可以选择将信用卡号、身份证号等敏感字段的部分或全部数字替换为星号 * 或其他字符，或者将真实姓名替换为假名。

从 2.1.2 版本开始，支持通过 Apache Ranger 的 Data Masking 来为某些列设置脱敏策略，目前仅支持通过 Apache Ranger 来设置

> 为 admin/root 用户设置数据脱敏不会生效

2.17.3 审计日志

Doris 提供了对于数据库操作的审计能力，可以记录用户对数据库的登陆、查询、修改操作。在 Doris 中，可以直接通过内置系统表查询审计日志，也可以直接查看 Doris 的审计日志文件。

2.17.3.1 开启审计日志

通过全局变量 enable_audit_plugin 可以随时开启或关闭审计日志插件（默认为关闭状态），如：

```
set global enable_audit_plugin = true;
```

开启后，Doris 会将开启后的审计日志写入 audit_log 表。

可以随时关闭审计日志插件：

```
set global enable_audit_plugin = false;
```

关闭后，Doris 将会停止 audit_log 表的写入。已写入的审计日志不会变化。

2.17.3.2 查看审计日志表

> 注意
>
> 在 2.1.8 版本之前，随着系统版本的升级，审计日志字段会有增加，在升级后需要根据审计日志表中的字段，通过 ALTER TALBE 命令为 audit_log 表增加字段。

从 Doris 2.1 版本开始，Doirs 可以通过开启审计日志功能，将用户行为操作写入到 __internal_schema 库的audit_log 表中。

审计日志表是一张动态分区表，按天进行分区，默认保留最近 30 天的数据。可以通过 ALTER TABLE 语句修改动态分区的 dynamic_partition.start 属性调整动态分区的保留天数。

2.17.3.3 审计日志文件

在 fe.conf 中，LOG_DIR 定义了 FE 日志的存储路径。在 ${LOG_DIR}/fe.audit.log 中记录了这台 FE 节点执行的所有数据库操作。如果需要查看集群所有的操作，需要遍历每一台 FE 的审计日志。

2.17.3.4 审计日志相关配置

全局变量：

可以通过 `set [global] <var_name> = <var_value>` 修改审计日志变量。

变量	默认值	说明
audit_plugin_max_batch_interval_sec	60 秒	审计日志表的最大写入间隔。
audit_plugin_max_batch_bytes	50MB	审计日志表每批次最大写入数据量
audit_plugin_max_sql_length	4096	审计日志表里记录的语句的最大长度
audit_plugin_load_timeout	600 秒	审计日志导入作业的默认超时时间
audit_plugin_max_insert_stmt_length	Int.MAX	针对 INSERT 语句最大长度限制。如果大于 audit_plugin_max_sql_length，则使用 audit_plugin_max_sql_length 的值。该参数自 3.0.6 支持。

因为某些 `INSERT INTO VALUES` 语句可能过长切提交频繁，导致设计日志膨胀。因此，Doris 在 3.0.6 版本新增了 `audit_plugin_max_insert_stmt_length` 用于单独限制 `INSERT` 语句的审计长度。避免审计日志膨胀，同时可以保证 SQL 语句被完整审计。

FE 配置项：

通过修改 fe.conf 目录可以修改 FE 配置项。

配置项	说明
skip_audit_user_list	如果不希望某些用户的操作被审计日志记录，可以通过这个配置修改（自 3.0.01 支持）。如通过以下配置屏蔽 user1 与 user2 的审计日志记录： skip_audit_user_list=user1,user2

2.17.4 数据加密

2.17.4.1 传输加密

2.17.4.1.1 MySQL 安全传输

加密连接 FE

Doris 支持基于 SSL 的加密连接，当前支持 TLS1.2，TLS1.3 协议，可以通过以下配置开启 Doris 的 SSL 模式：修改 FE 配置文件`conf/fe.conf`，添加`enable_ssl = true`即可。

接下来通过mysql客户端连接 Doris，mysql 支持三种 SSL 模式：

1.mysql -uroot -P9030 -h127.0.0.1与mysql --ssl-mode=PREFERRED -uroot -P9030 -h127.0.0.1一样，都是一开始试图建立 SSL 加密连接，如果失败，则尝试使用普通连接。

2.mysql --ssl-mode=DISABLE -uroot -P9030 -h127.0.0.1，不使用 SSL 加密连接，直接使用普通连接。

3.mysql --ssl-mode=REQUIRED -uroot -P9030 -h127.0.0.1，强制使用 SSL 加密连接。

> 注意：--ssl-mode参数是 mysql5.7.11 版本引入的，低于此版本的 mysql 客户端请参考这里。Doris 开启 SSL 加密连接需要密钥证书文件验证，默认的密钥证书文件位于Doris/fe/mysql_ssl_default_certificate/certificate.p12，默认密码为doris，您可以通过修改 FE 配置文件conf/fe.conf，添加mysql_ssl_default_certificate = /path/to/your/certificate修改密钥证书文件，同时也可以通过mysql_ssl_default_certificate_password = your_password添加对应您自定义密钥书文件的密码。

Doris 还支持 mTLS：修改 FE 配置文件conf/fe.conf，添加ssl_force_client_auth=true即可。

接下来可以通过mysql客户端连接 Doris：

```
mysql -ssl-mode=VERIFY_CA -uroot -P9030 -h127.0.0.1 --tls-version=TLSv1.2 --ssl-ca=/path/to/your/ca --ssl-cert=/path/to/your/cert --ssl-key=/path/to/your/key
```

默认的 ca，cert，key 文件位于Doris/conf/mysql_ssl_default_certificate/client_certificate/，分别叫做ca.pem，client-cert.pem，client-key.pem。

你也可以通过 openssl 或者 keytool 生成自己的证书文件。

SSL 密钥证书配置

Doris 开启 SSL 功能需要配置 CA 密钥证书和 Server 端密钥证书，如需开启双向认证，还需生成 Client 端密钥证书：

- 默认的 CA 密钥证书文件位于Doris/fe/mysql_ssl_default_certificate/ca_certificate.p12，默认密码为doris，您可以通过修改 FE 配置文件conf/fe.conf，添加mysql_ssl_default_ca_certificate = /path/to/your/certificate修改 CA 密钥证书文件，同时也可以通过mysql_ssl_default_ca_certificate_password = your_password添加对应您自定义密钥证书文件的密码。
- 默认的 Server 端密钥证书文件位于Doris/fe/mysql_ssl_default_certificate/server_certificate.p12，默认密码为doris，您可以通过修改 FE 配置文件conf/fe.conf，添加mysql_ssl_default_server_certificate = /path/to/your/certificate修改 Server 端密钥证书文件，同时也可以通过mysql_ssl_default_server_certificate_password = your_password添加对应您自定义密钥证书文件的密码。
- 默认生成了一份 Client 端的密钥证书，分别存放在Doris/fe/mysql_ssl_default_certificate/client-key.pem和Doris/fe/mysql_ssl_default_certificate/client_certificate/。

自定义密钥证书文件

除了 Doris 默认的证书文件，您也可以通过openssl生成自定义的证书文件。步骤参考MySQL 生成 SSL 证书 具体如下：

1. 生成 CA、 Server 端和 Client 端的密钥和证书

```
openssl genrsa 2048 > ca-key.pem
openssl req -new -x509 -nodes -days 3600 \
        -key ca-key.pem -out ca.pem

##### 生成 server certificate, 并用上述 CA 签名
##### server-cert.pem = public key, server-key.pem = private key
openssl req -newkey rsa:2048 -days 3600 \
        -nodes -keyout server-key.pem -out server-req.pem
openssl rsa -in server-key.pem -out server-key.pem
openssl x509 -req -in server-req.pem -days 3600 \
        -CA ca.pem -CAkey ca-key.pem -set_serial 01 -out server-cert.pem

##### 生成 client certificate, 并用上述 CA 签名
##### client-cert.pem = public key, client-key.pem = private key
openssl req -newkey rsa:2048 -days 3600 \
        -nodes -keyout client-key.pem -out client-req.pem
openssl rsa -in client-key.pem -out client-key.pem
openssl x509 -req -in client-req.pem -days 3600 \
        -CA ca.pem -CAkey ca-key.pem -set_serial 01 -out client-cert.pem
```

2. 验证创建的证书。

```
openssl verify -CAfile ca.pem server-cert.pem client-cert.pem
```

3. 将您的 CA 密钥和证书和 Server 端密钥和证书分别合并到 PKCS#12 (P12) 包中。您也可以指定某个证书格式，默认 PKCS12，可以通过修改 conf/fe.conf 配置文件，添加参数 ssl_trust_store_type 指定证书格式

```
##### 打包 CA 密钥和证书
openssl pkcs12 -inkey ca-key.pem -in ca.pem -export -out ca_certificate.p12

##### 打包 Server 端密钥和证书
openssl pkcs12 -inkey server-key.pem -in server-cert.pem -export -out server_certificate.p12
```

Note 参考文档

2.17.4.1.2 HTTP 安全传输

> 从 2.0 版本开始，Doris 支持 SSL 密钥和证书配置

Doris FE 接口开启 SSL 功能需要配置密钥证书，步骤如下：

1. 购买或生成自签名 SSL 证书，生产环境建议使用 CA 颁发的证书
2. 将 SSL 证书复制到指定路径下，默认路径为 `${DORIS_HOME}/conf/ssl/`，用户也可以自己指定路径
3. 修改 FE 配置文件 `conf/fe.conf`，注意以下参数与购买或生成的 SSL 证书保持一致

- 设置 `enable_https = true` 开启 https 功能，默认为 `false`
- 设置证书路径 `key_store_path`，默认为 `${DORIS_HOME}/conf/ssl/doris_ssl_certificate.keystore`
- 设置证书密码 `key_store_password`，默认为空
- 设置证书类型 `key_store_type`，默认为 JKS
- 设置证书别名 `key_store_alias`，默认为 `doris_ssl_certificate`

2.17.4.2 加密和脱敏函数

Doris 内置了如下加密和脱敏函数。详细用法请参考 SQL 手册。

- AES_ENCRYPT
- AES_DECRYPT
- SM4_ENCRYPT
- SM4_DECRYPT
- MD5
- MD5SUM
- SM3
- SM3SUM
- SHA
- SHA2
- DIGITAL_MASKING

2.17.5 集成

2.17.5.1 AWS 认证和鉴权

Doris 支持两种 AWS 认证、鉴权方式访问 AWS 服务，`IAM User`和`Assumed Role`，本文介绍如何配置这两种认证、鉴权方式的 AWS 安全凭证并通过安全凭证使用 Doris 相关的功能来访问 AWS 的服务资源。

2.17.5.1.1 认证方式介绍

IAM User 认证鉴权

Doris 支持通过配置AWS IAM User的方式来实现对外部数据源的访问（即access_key和secret_key密钥的方式），详细的配置步骤如下(更详细的介绍请参见 AWS 官网文档 IAM users)：

Step1 登录 AWS 控制台创建 IAM User 并配置 IAM 策略

1. 登录 AWS控制台 选择 Create user 按钮

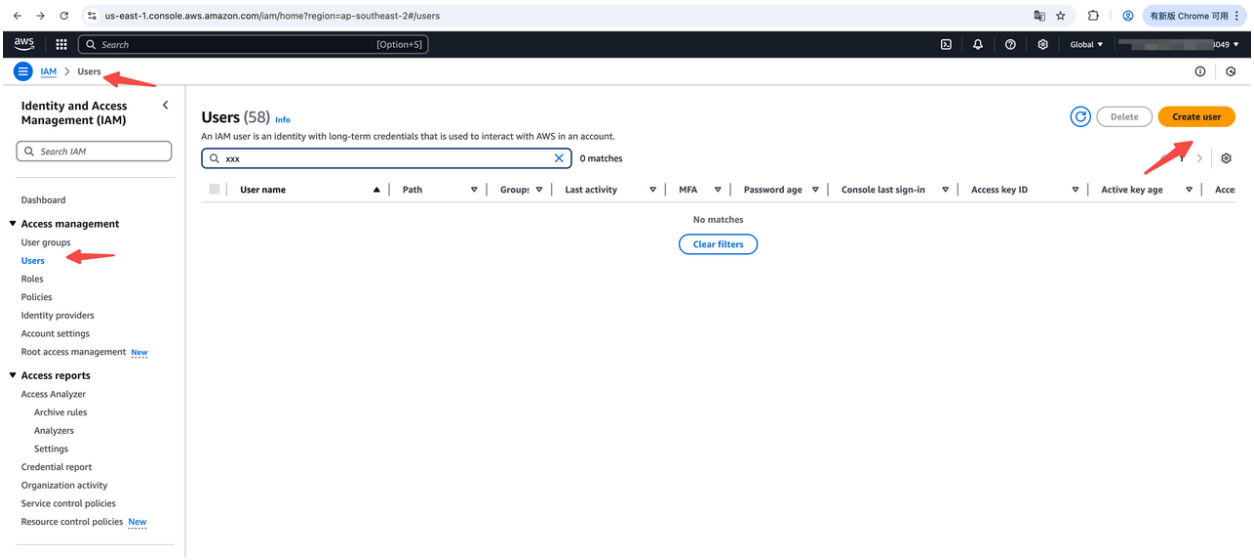


图 115:

2. 填写好 IAM user 名字后，在Set pemissions部分选择直接附加策略

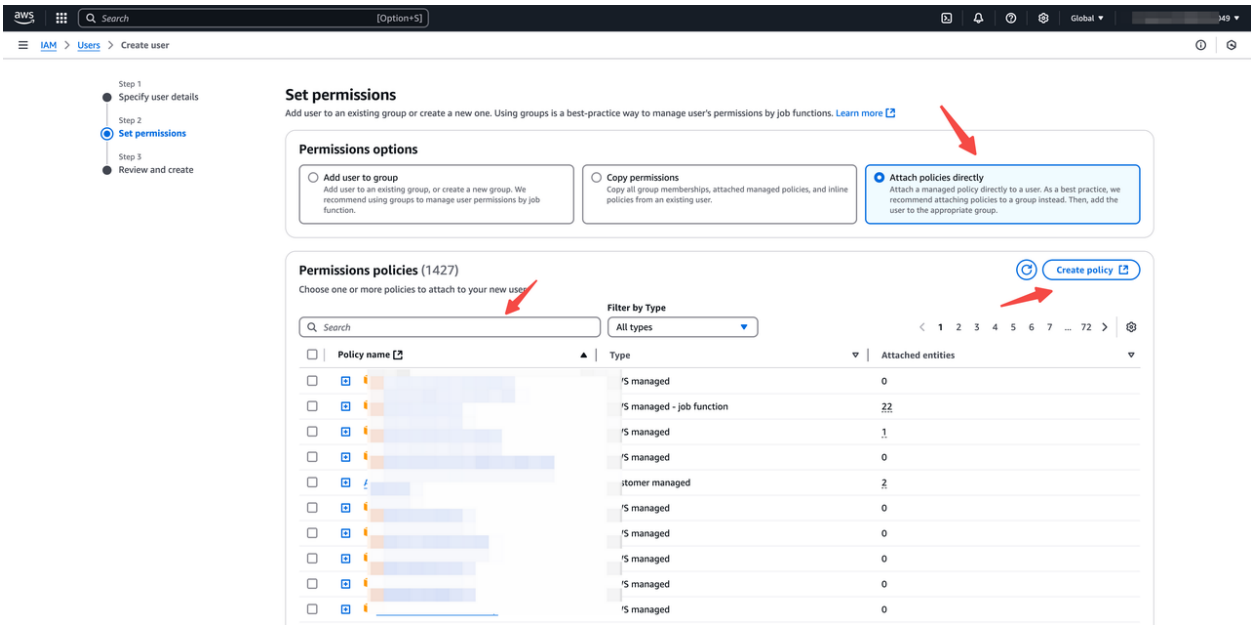


图 116:

3. 在策略编辑器中填入对应的 AWS 资源策略，下文以访问 S3 Bucket 资源为例列出了读/写策略的常见模板

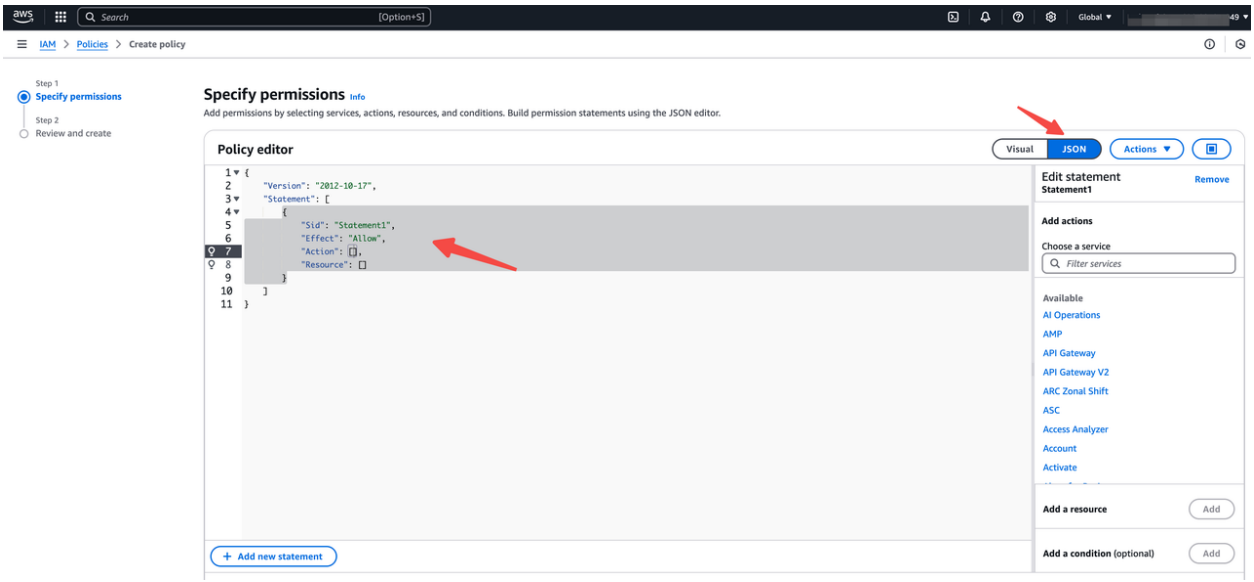


图 117:

S3 Bucket 读策略模版，适用于只需读取和列出 bucket 中对象的 Doris 功能，比如 S3 Load，TVF，External Catalog 等

注意:

1. 替换对应的 bucket name 和 prefix 路径

2. 不要添加多余的“/”分割符

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
              "s3:GetObject",
              "s3:GetObjectVersion",
            ],
            "Resource": "arn:aws:s3:::<your-bucket>/your-prefix/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::<your-bucket>"
        }
    ]
}
```

S3 Bucket 写策略模板，适用于需要读取、列出和写入 bucket 对象的 Doris 功能，比如 Export，Storage Vault，Resource，Repository 等

注意:

1. 替换对应的 bucket name 和 prefix 路径
2. 不要添加多余的“/”分割符

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
              "s3:PutObject",
              "s3:GetObject",
              "s3:GetObjectVersion",
              "s3:DeleteObject",
              "s3:DeleteObjectVersion",
              "s3:AbortMultipartUpload",
              "s3:ListMultipartUploadParts"
```

```
            ],
            "Resource": "arn:aws:s3:::<your-bucket>/<your-prefix>/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetBucketVersioning",
                "s3:GetLifecycleConfiguration"
            ],
            "Resource": "arn:aws:s3:::<your-bucket>"
        }
    ]
}
```

4. 创建 IAM User 成功后，创建 access/secret key 密钥

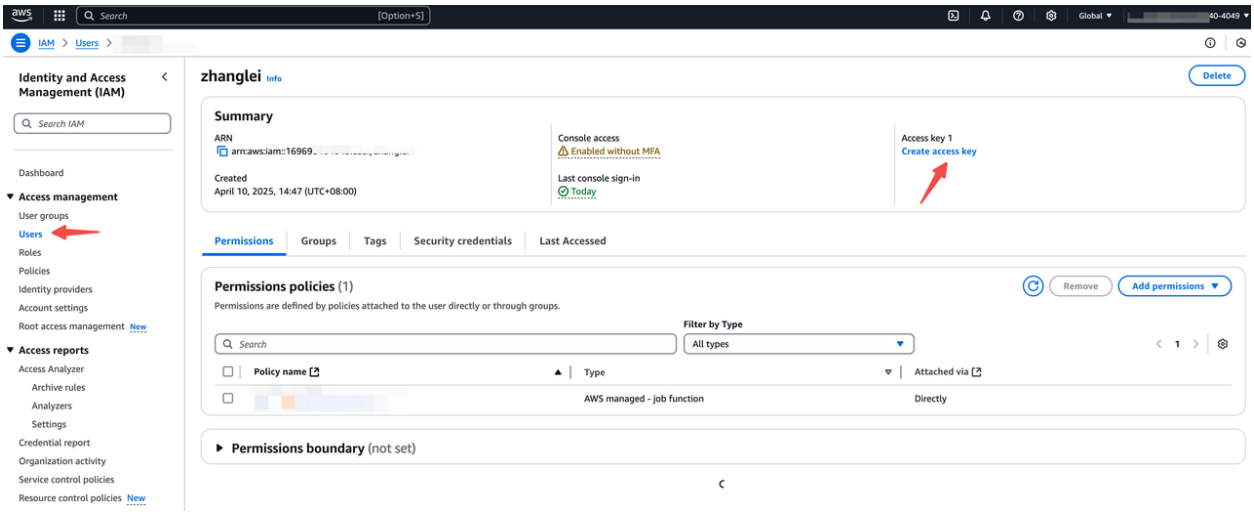


图 118:

Step2 通过访问密钥和 SQL 语句使用 Doris 对应功能

完成上述 Step1 中的所有配置后，可获得access_key和secret_key访问密钥，通过访问密钥可以使用 Doris 对应的功能，具体例子如下：

S3 Load

```
 LOAD LABEL s3_load_2022_04_01
 (
     DATA INFILE("s3://your_bucket_name/s3load_example.csv")
     INTO TABLE test_s3load
```

```
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key" = "<your-secrety-key>"
)
PROPERTIES
(
    "timeout" = "3600"
);
```

TVF

```
SELECT * FROM S3 (
    'uri' = 's3://your_bucket/path/to/tvf_test/test.parquet',
    'format' = 'parquet',
    's3.endpoint' = 's3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>"
)
```

External Catalog

```
CREATE CATALOG iceberg_catalog PROPERTIES (
    'type' = 'iceberg',
    'iceberg.catalog.type' = 'hadoop',
    'warehouse' = 's3://your_bucket/dir/key',
    's3.endpoint' = 's3.us-east-1.amazonaws.com',
    's3.region' = 'us-east-1',
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>"
);
```

Storage Vault

```
CREATE STORAGE VAULT IF NOT EXISTS s3_demo_vault
PROPERTIES (
   "type" = "S3",
   "s3.endpoint" = "s3.us-east-1.amazonaws.com",
   "s3.region" = "us-east-1",
```

```
    "s3.bucket" = "<your-bucket>",
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>",
    "s3.root.path" = "s3_demo_vault_prefix",
    "provider" = "S3",
    "use_path_style" = "false"
);
```

Export

```
EXPORT TABLE s3_test TO "s3://your_bucket/a/b/c"
PROPERTIES (
    "column_separator"="\\x07",
    "line_delimiter" = "\\x07"
) WITH S3 (
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>",
)
```

Repository

```
CREATE REPOSITORY `s3_repo`
WITH S3
ON LOCATION "s3://your_bucket/s3_repo"
PROPERTIES
(
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>"
);
```

Resource

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.bucket" = "<your-bucket>",
    "s3.access_key" = "<your-access-key>",
    "s3.secret_key"="<your-secret-key>"
);
```

您可以在不同业务逻辑里指定不同的 IAM User 的 `access_key` 和 `secret_key`，从而实现外部数据的访问控制。

Assumed Role 认证鉴权

Assumed Role 支持通过担任 AWS IAM Role 来实现对外部数据源的访问认证和鉴权 (详细的介绍请参见 AWS 官网文档 代入角色)，下图列出了 Assumed Role 所需要配置的简要流程：

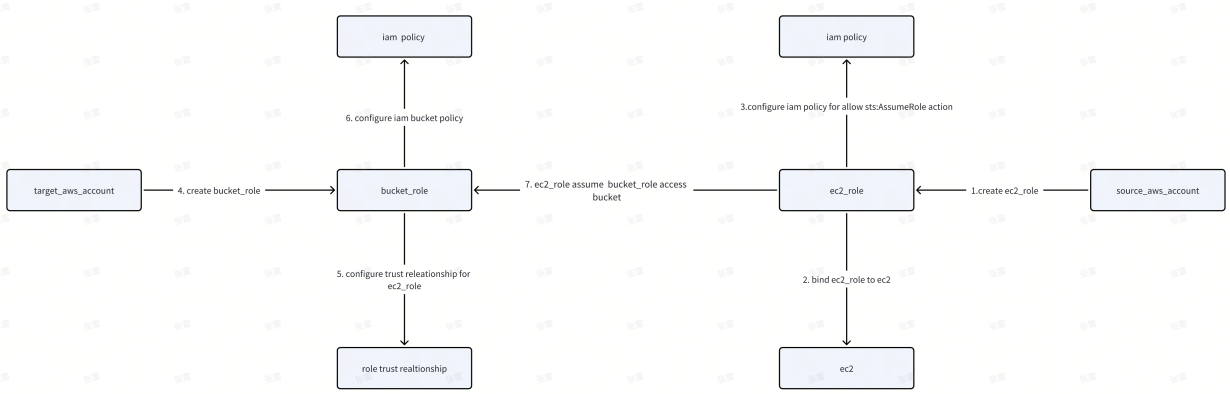


图 119:

名词介绍：

源账户(Source Account): 发起 Assume Role 的 AWS 账户 (本例中是 Doris FE/BE EC2 机器所属账户);

目标账户(Target Account): 拥有目标 S3 Bucket 的 AWS 账户;

ec2_role: 源账户创建的 Role，并且需要绑定到每一个部署 Doris FE/BE 部署 EC2 机器上;

bucket_role: 目标账户创建的 Role，并且需要关联目标 bucket 权限;

注意:

1. 源账户和目标账户可以是同一个 AWS 账户;
2. 请确保所有 Doris FE/BE 部署所在的 EC2 机器都绑定到了ec_role上，尤其是扩容的时候。

操作演示 Demo 如下：

更详细的配置步骤如下：

Step1 准备工作

1. 请确保源账户创建了一个ec2_role，Doris FE/BE 部署的 EC2 机器都绑定到了新创建的ec2_role;
2. 请确保目标账户创建了一个bucket_role和对应的 bucket;

EC2 机器绑定ec2_role成功后，role_arn查询如下图所示：

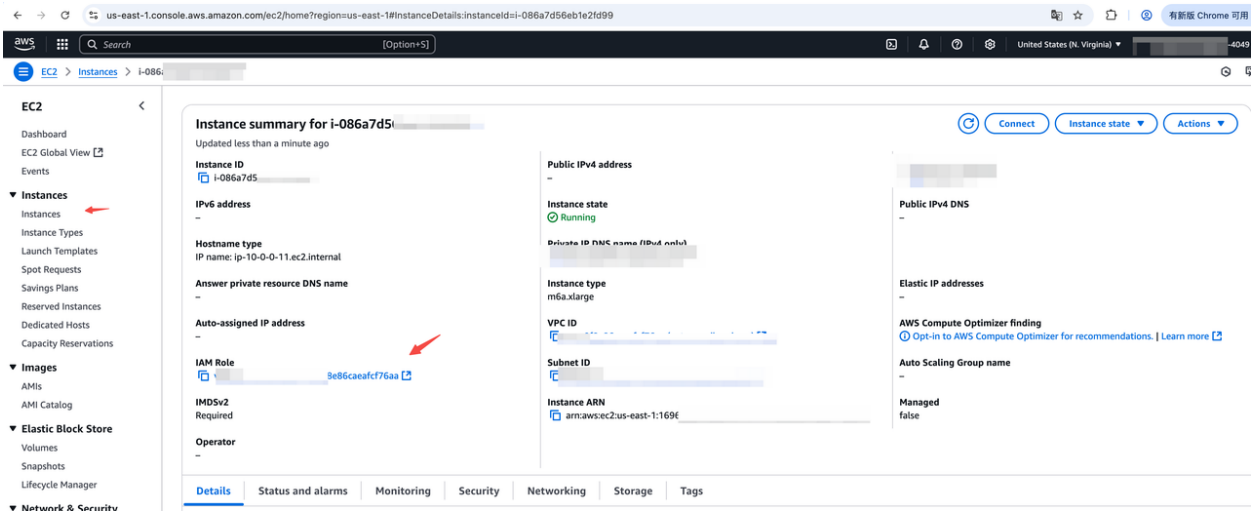


图 120:

Step2 配置源账户 IAM 角色 (EC2 实例关联角色) 权限策略

1. 登录 AWS IAM 控制台，在左侧导航栏选择 Access management > Roles;
2. 找到 EC2 实例关联角色，单击角色名称；
3. 在角色详情页的 Permissions 区域，单击 Add permissions 并选择 Create inline policy；
4. 在 Specify permissions 步骤，单击 JSON 页签，然后填入如下策略，最后，单击 Review policy；

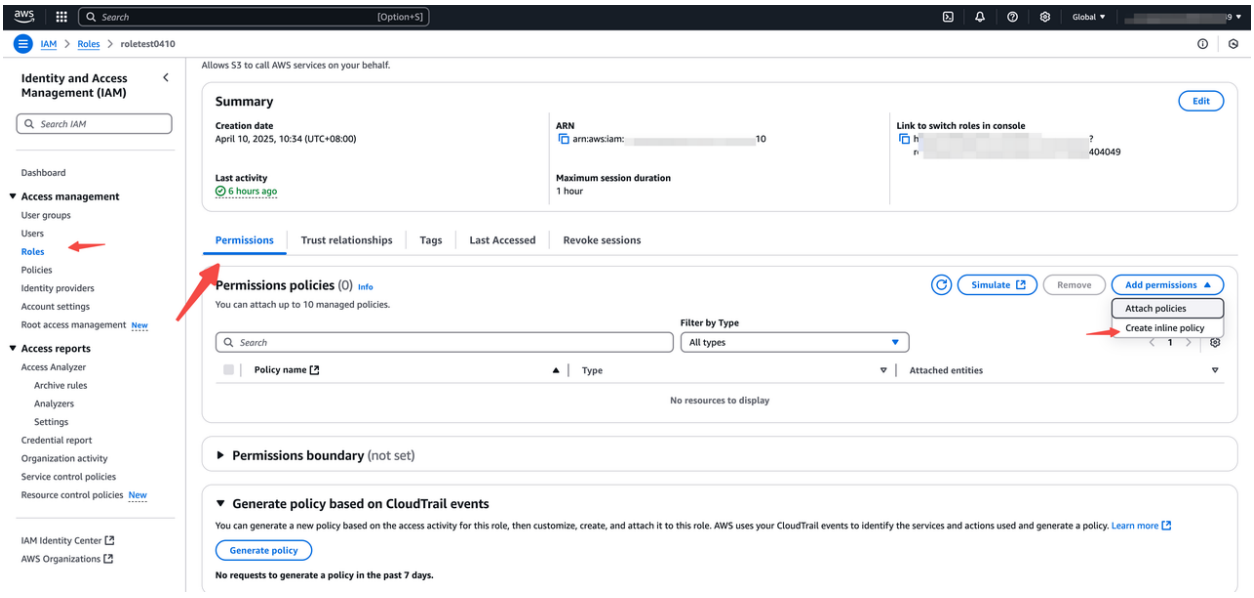


图 121:

```
{
```

```
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["sts:AssumeRole"],
            "Resource": "*"
        }
    ]
}
```

Step3 配置目标账户 IAM 角色信任策略和权限策略

1. 登录 AWS IAM 控制台，在左侧导航栏选择 Access management > Roles 找到 Assumed Target Role，单击角色名称在角色详情页上;
2. 单击 Trust relationships 页签，然后在 Trust relationships 页签上单击 Edit trust policy。在 Edit trust policy 页面中填入如下 JSON。最后，单击 Update policy(需要把下面策略中的 <ec2_iam_role_arn> 替换为 EC2 实例关联角色的 ARN);

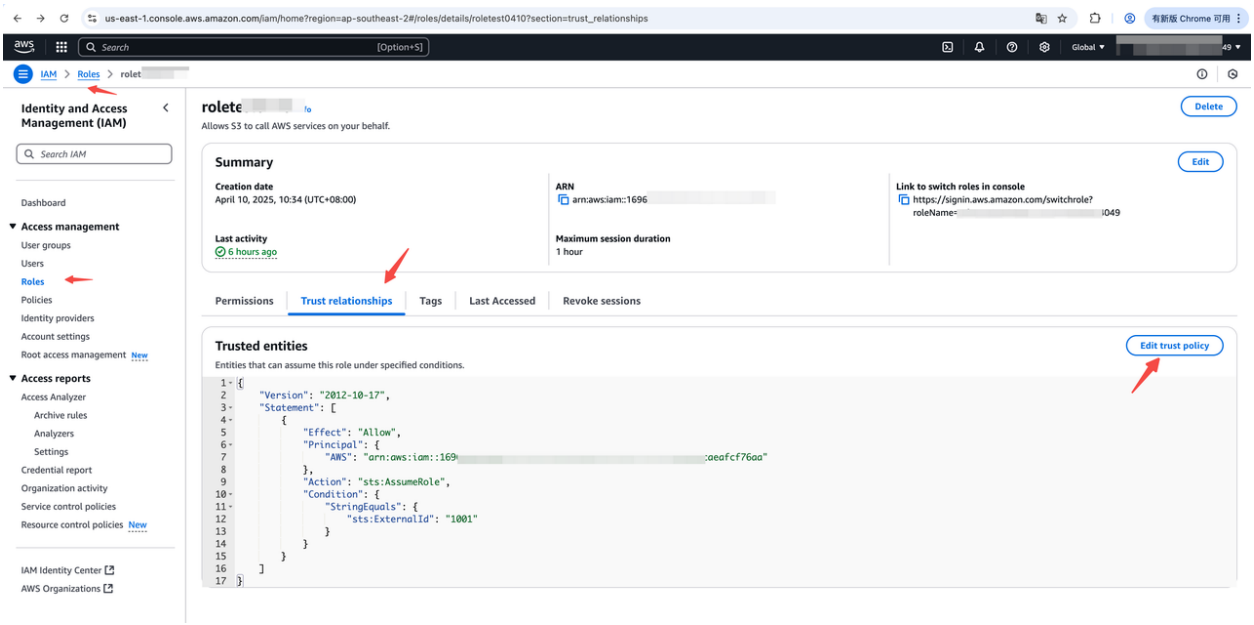


图 122:

注意：Condition 部分中的 ExternalId 是可选的字符串配置，用于区分需要使用多个源用户 assume 同一个 role 的情况，如果配置了请在对应 doris sql 语句中填入该配置，关于 ExternalId 的详细介绍，请参照aws 官方文档

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
```

```
            "Effect": "Allow",
            "Principal": {
                "AWS": "<ec2_iam_role_arn>"
            },
            "Action": "sts:AssumeRole",
            "Condition": {
                "StringEquals": {
                    "sts:ExternalId": "1001"
                }
            }
        }
    ]
}
```

3. 在角色详情页的 Permissions 区域，单击 Add permissions 并选择 Create inline policy, 在 Specify permissions 步骤，单击 JSON 页签，输入如下 JSON 策略配置; 最后，单击 Review policy;

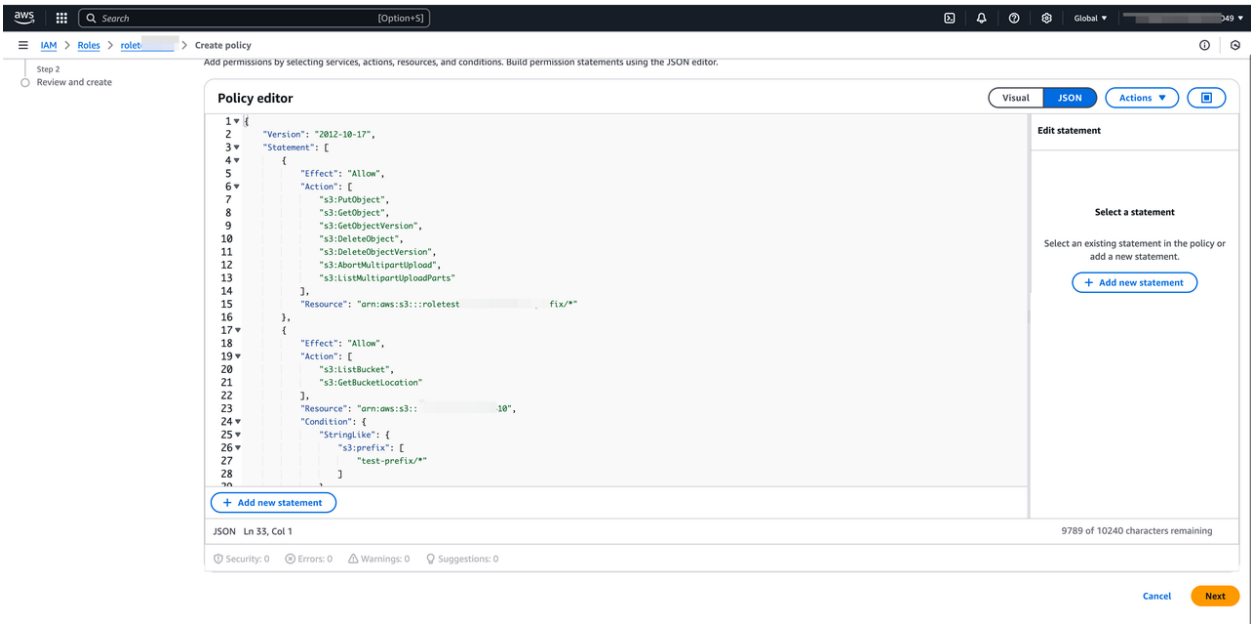


图 123:

S3 Bucket 读策略模版，适用于只需读取和 List bucket 中对象的 Doris 功能，比如 S3 Load，TVF，External Catalog 等

注意:

1. 替换对应的 bucket name 和 prefix 路径
2. 不要添加多余的 “/” 分割符

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
              "s3:GetObject",
              "s3:GetObjectVersion"
            ],
            "Resource": "arn:aws:s3:::<bucket>/<prefix>/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::<bucket>",
        }
    ]
}
```

S3 Bucket 写策略模板，适用于需要往 bucket 中读取和写入对象的 Doris 功能，比如 Export，Storage Vault，Resource，Repository 等

注意:

1. 替换对应的 bucket name 和 prefix 路径
2. 不要添加多余的 “/” 分割符

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
              "s3:PutObject",
              "s3:GetObject",
              "s3:GetObjectVersion",
              "s3:DeleteObject",
              "s3:DeleteObjectVersion",
              "s3:AbortMultipartUpload",
              "s3:ListMultipartUploadParts"
            ],
```

```
            "Resource": "arn:aws:s3:::<bucket>/<prefix>/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::<bucket>"
        }
    ]
}
```

Step4 通过 `role_arn` 和 `external_id` 字段使用 Doris 对应 SQL 功能

通过上述配置步骤完成 assume role 需要的权限配置后，可得到一个目标账户的`role_arn`信息和`external_id`(如有)，

接下来分别介绍如何通过`arn_role`和`external_id`字段使用 Doris 对应功能的 sql 语法，主要关注如下两个字段：

```
"s3.role_arn" = "<your-bucket-role-arn>",
"s3.external_id" = "<your-external-id>"      -- 可选参数
```

S3 Load

```
LOAD LABEL s3_load_2022_04_01
(
    DATA INFILE("s3://your_bucket_name/s3load_example.csv")
    INTO TABLE test_s3load
    COLUMNS TERMINATED BY ","
    FORMAT AS "CSV"
    (user_id, name, age)
)
WITH S3
(
    "provider" = "S3",
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>"      -- 可选参数
)
PROPERTIES
(
    "timeout" = "3600"
);
```

TVF

```
SELECT * FROM S3 (
    "uri" = "s3://your_bucket/path/to/tvf_test/test.parquet",
    "format" = "parquet",
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>"      -- 可选参数
)
```

External Catalog

```
CREATE CATALOG iceberg_catalog PROPERTIES (
    "type" = "iceberg",
    "iceberg.catalog.type" = "hadoop",
    "warehouse" = "s3://your_bucket/dir/key",
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>"      -- 可选参数
);
```

Storage Vault

```
CREATE STORAGE VAULT IF NOT EXISTS s3_demo_vault
PROPERTIES (
    "type" = "S3",
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.bucket" = "<your-bucket>",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>",            -- 可选参数
    "s3.root.path" = "s3_demo_vault_prefix",
    "provider" = "S3",
    "use_path_style" = "false"
);
```

Export

```
EXPORT TABLE s3_test TO "s3://your_bucket/a/b/c"
PROPERTIES (
    "column_separator"="\\x07",
    "line_delimiter" = "\\x07"
) WITH S3 (
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.role_arn" = "<your-bucket-role-arn>",
```

```
    "s3.external_id" = "<your-external-id>"
)
```

Repository

```
CREATE REPOSITORY `s3_repo`
WITH S3
ON LOCATION "s3://your_bucket/s3_repo"
PROPERTIES
(
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>"
);
```

Resource

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.bucket" = "<your-bucket>",
    "s3.role_arn" = "<your-bucket-role-arn>",
    "s3.external_id" = "<your-external-id>"
);
```

AWS EKS 集群中 IAM Role 认证鉴权

对于在 Amazon EKS 集群中运行的应用（例如 Apache Doris），要授予其 AWS Identity and Access Management（IAM）权限，Amazon EKS 提供了以下两种主要方式：

1. 服务账户的 IAM 角色（IRSA）

2. EKS 容器组身份 (Pod Identity)

这两种方式均需在 EKS 集群中正确配置 IAM Role 和对应的信任策略、IAM 策略, 具体配置方法请参阅 AWS 官方文档：

Granting AWS Identity and Access Management permissions to workloads on Amazon Elastic Kubernetes Service clusters

Doris FE/BE 自动通过AWSCredentialsProviderChain获取凭证

Bucket Policy 认证鉴权

对于 IAM Role 部署的 Doris 机器，导入、导出、TVF 的场景也支持使用 Amazon S3 存储桶策略来保护对 AWS S3 存储桶中的对象进行访问，这样可以限制只有 EC2 机器所属用户才能访问对象存储桶，具体步骤如下：

1、设置目标存储桶的 Bucket Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": [
                    "arn:aws:iam::111122223333:root"
                ]
            },
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:GetObjectVersion",
                "s3:DeleteObject",
                "s3:DeleteObjectVersion",
                "s3:AbortMultipartUpload",
                "s3:ListMultipartUploadParts"
            ],
            "Resource": "arn:aws:s3:::<bucket>/<prefix>/*"
        },
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": [
                    "arn:aws:iam::111122223333:root"
                ]
            },
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::<bucket>",
        }
    ]
}
```

请将`arn:aws:iam::111122223333:root` 替换为 ec2 机器所绑定的账户或者 Role 的 ARN

2、使用对应功能的 SQL 语法进行数据访问，不需要 ak/sk，arn 等信息

```
  SELECT * FROM S3 (
      "uri" = "s3://your_bucket/path/to/tvf_test/test.parquet",
      "format" = "parquet",
      "s3.endpoint" = "s3.us-east-1.amazonaws.com",
      "s3.region" = "us-east-1"
```

```
)
```

Doris FE/BE 自动通过`AWSCredentialsProviderChain`获取凭证

参考文档：Bucket Policy

鉴权方式最佳实践

鉴权方式	适用场景	优点	缺点
AK/SK 鉴权方式	私有化部署安全性可控或非 AWS S3 的对象存储的导入/导出/StorageVault 场景	配置简单，支持兼容 AWS S3 的对象存储	存在密钥泄漏风险，需要手动进行密钥轮换
IAM Role 鉴权方式	AWS S3 公有云安全性要求较高的导入/导出/StorageVault 场景	安全性高, 自动轮换 AWS 凭证, 权限配置集中	配置 Bucket Policy/Trust 流程复杂
Bucket Policy 鉴权方式	AWS S3 公有云，bucket 数量较少的导入/导出/StorageVault 场景	配置流程复杂度适中，遵循最小权限原则，自动探测 AWS 凭证	权限配置分散在各个 bucket policy 中

常见问题

1. 如何设置 BE 和 `Recycler` 的 Aws Sdk DEBUG 级别日志?

be.conf 和 doris_cloud.conf 配置 aws_log_level=5, 并重启进程生效 * 类型：int32 * 描述：AWS SDK 的日志级别

```
    Off = 0,
    Fatal = 1,
    Error = 2,
    Warn = 3,
    Info = 4,
    Debug = 5,
    Trace = 6
```

- 默认值：2

2. 设置 Aws Sdk DEBUG 级别日志后，be.log/recycler.log 报如下错误：

`OpenSSL SSL_connect: Connection reset by peer in connection to sts.me-south-1.amazonaws.com:443`

请检查 aws vpc 网络配置或者防火墙端口配置是否存在问题，导致无法访问 aws 对应 region 的 sts 服务 (可通过 telnet host:port 确认)

2.17.5.2 Apache Doris IAM Assume Role 工作原理

2.17.5.2.1 一、传统 AK/SK 方式访问 AWS 资源存在的问题

密钥管理困境：

- 长期暴露风险：静态 AK/SK 需硬编码于配置文件中，一旦因代码泄露、误提交或恶意窃取导致密钥扩散，攻击者可永久获得等同于密钥所有者的完整权限，引发持续性的数据泄露、资源篡改及资金损失风险；
- 审计盲区：多用户/多服务共享同一组密钥时，云操作日志仅记录密钥身份而无法关联具体使用者，无法追溯真实责任人或业务模块；
- 运维成本高：密钥轮换灾难，需手动轮换业务模块密钥，容易出错触发服务中断；
- 权限管理失控：账户级粗放授权无法满足服务/实例级的最小权限管控需求。

2.17.5.2.2 二、AWS IAM ASSUME ROLE 机制介绍

AWS IAM Assume Role 是什么？

AWS Assume Role 是一种安全身份切换机制，允许一个可信实体（如 IAM 用户、EC2 实例或外部账号）通过 STS（安全令牌服务）临时获取目标角色的权限，其运作流程如下:

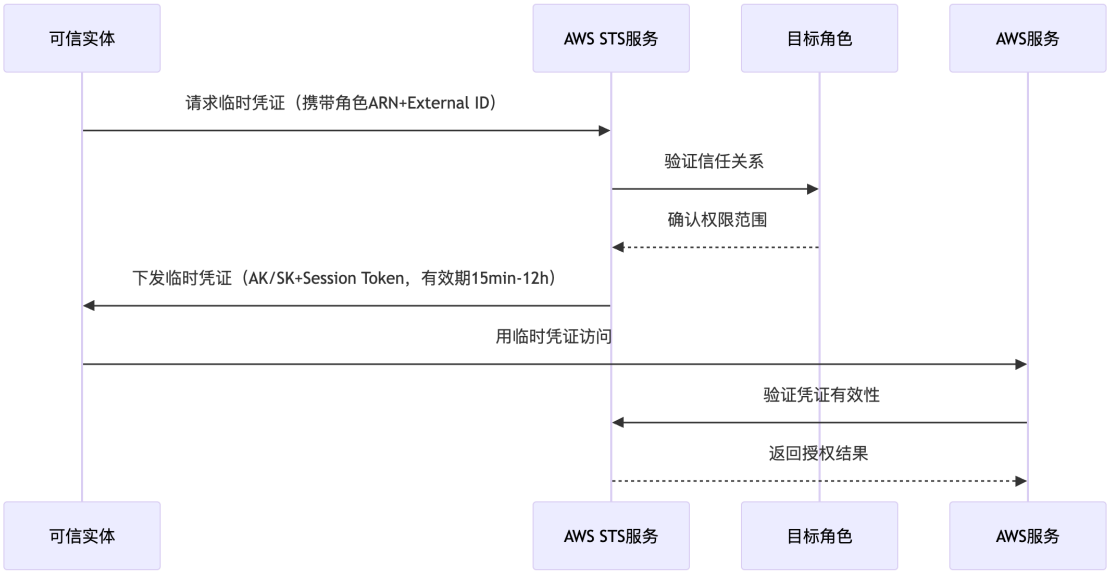


图 124:

使用 AWS IAM Assume Role 方式访问的优点:

- 动态令牌机制 (15 分钟 ~12 小时有效期) 替代永久密钥
- 通过 External ID 实现跨账号安全隔离，并且可通过 AWS 后台服务进行审计
- 基于角色的最小权限原则 (Principle of Least Privilege)

AWS IAM Assume Role 访问 S3 Bucket 的鉴权过程：

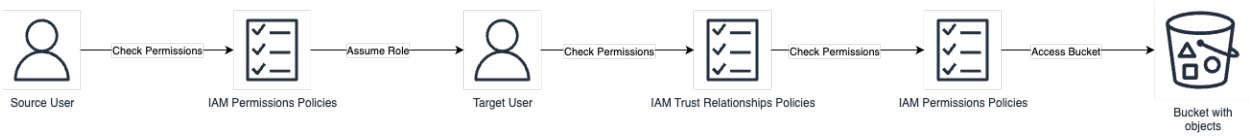


图 125:

阶段 1: 源用户身份验证

1. 权限策略检查

- 源用户发起 AssumeRole 请求时，源账户的 IAM 策略引擎首先验证：该用户是否被授权调用 sts:AssumeRole 操作？
- 检查依据：附着在源用户身份上的 IAM Permissions Policies

2. 信任关系校验

- 通过 STS 服务向目标账户发起请求：
  - 源用户是否在目标角色的信任策略白名单中？
- 检查依据：目标角色绑定的 IAM Trust Relationships Policies（明确允许哪些账号/用户担任该角色）

阶段 2: 目标角色权限激活

3. 临时凭证生成
   若信任关系验证通过，STS 生成三要素临时凭证

```
{
"AccessKeyId": "***",
"SecretAccessKey": "***",
"SessionToken": "***" // 有效期15min-12h
}
```

4. 目标角色权限验证

- 目标角色使用临时凭证访问 S3 前，目标账户的 IAM 策略引擎校验：该角色是否被授权执行请求的 S3 操作 ?(如 s3:GetObject、 s3:PutObject 等)

- 检查依据：附着在目标角色上的 IAM Permissions Policies (定义角色能做什么)

阶段 3: 资源操作执行

5. 访问存储桶
   全部验证通过后，目标角色才可执行 S3 API 操作

2.17.5.2.3 三、 Doris 如何应用 AWS IAM Assume Role 鉴权机制

1. Doris 通过将 FE、 BE 进程所部署的 AWS EC2 Instances 绑定到 Source Account 来使用 AWS IAM Assume Role 功能，主要流程如下图所示：

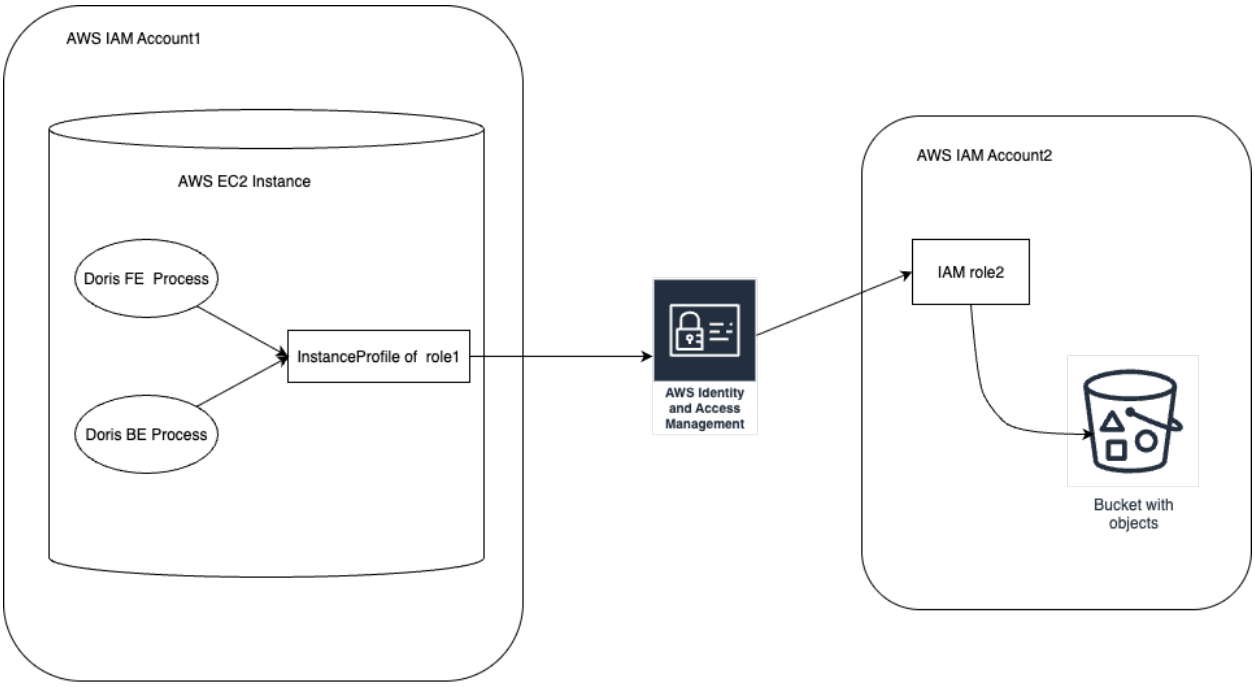


图 126:

2. 完成配置后 Doris FE/BE 进程会自动获取 EC2 Instance 的 Profile 执行 Assume Role 操作访问 Bucket。扩容时 BE 节点会自动检测新 EC2 Instance 是否成功绑定 IAM Role，防止漏配;

3. Doris 的 S3 Load、TVF、Export、Resource、Repository、Storage Vault 等功能在 3.0.6+ 版本均支持 AWS Assume Role 方式，执行 SQL 相关功能时会进行连通性检测：

```
CREATE REPOSITORY `s3_repo`
WITH S3 ON LOCATION "s3://bucket/path/"
PROPERTIES (
```

```
"s3.role_arn" = "arn:aws:iam::1234567890:role/doris-s3-role",
"s3.external_id" = "doris-external-id",
"timeout" = "3600"
  );
```

其中 “s3.role_arn” 对应填入 AWS IAM Account2 下的 Iam role2 的 arn 值, “s3.external_id” 对应填入 Trust Relationships Policies 中配置的 externalId 的值 (可选配置), 更多功能 SQL 语句功能详细参考：AWS 认证和鉴权.

3 性能测试

3.1 Star Schema Benchmark

Star Schema Benchmark(SSB) 是一个轻量级的数仓场景下的性能测试集。SSB 基于 TPC-H 提供了一个简化版的星型模型数据集，主要用于测试在星型模型下，多表关联查询的性能表现。另外，业界内通常也会将 SSB 打平为宽表模型（以下简称：SSB flat），来测试查询引擎的性能。

本文档主要介绍 Apache Doris 在 SSB 1000G 测试集上的性能表现。

在 SSB 标准测试数据集上的 13 个查询上，我们基于 Apache Doris 2.0.15.1 版本进行了测试。

3.1.1 1. 硬件环境

硬件	配置说明
机器数量	4 台阿里云主机（1 个 FE，3 个 BE）
CPU	Intel Xeon (Ice Lake) Platinum 8369B 32 核
内存	128G
磁盘	阿里云 ESSD (PL0)

3.1.2 2. 软件环境

- Doris 部署 3BE 1FE
- 内核版本：Linux version 5.15.0-101-generic
- 操作系统版本：Ubuntu 20.04 LTS (Focal Fossa)
- Doris 软件版本：Apache Doris 2.0.15.1
- JDK：openjdk version “1.8.0_352-352”

3.1.3 3. 测试数据量

SSB 表名	行数	备注
lineorder	5,999,989,709	商品订单明细表表
customer	30,000,000	客户信息表

SSB 表名	行数	备注
part	2,000,000	零件信息表
supplier	2,000,000	供应商信息表
dates	2,556	日期表
lineorder_flat	5,999,989,709	数据展平后的宽表

3.1.4 4. SSB 宽表测试结果

使用 Apache Doris 2.0.15.1 版本进行测试结果如下：

Query	Doris 2.0.15.1 (ms)
q1.1	80
q1.2	10
q1.3	110
q2.1	1680
q2.2	1210
q2.3	1060
q3.1	2010
q3.2	1560
q3.3	600
q3.4	10
q4.1	2380
q4.2	190
q4.3	120
Total	11020

3.1.5 5. 标准 SSB 测试结果

使用 Apache Doris 2.0.15.1 版本进行测试结果如下：

Query	Doris 2.0.15.1 (ms)
q1.1	330
q1.2	80
q1.3	80
q2.1	1780
q2.2	1970
q2.3	1510
q3.1	4000
q3.2	1720
q3.3	1510
q3.4	160
q4.1	4010
q4.2	840

Query	Doris 2.0.15.1 (ms)
q4.3	400
Total	19390

3.1.6 6. 环境准备

请先参照官方文档进行 Apache Doris 的安装部署，以获得一个正常运行中的 Doris 集群（至少包含 1 FE 1 BE，推荐 1 FE 3 BE）。

3.1.7 7. 数据准备

3.1.7.1 7.1 下载安装 SSB 数据生成工具。

执行以下脚本下载并编译 ssb-tools 工具。

```
sh bin/build-ssb-dbgen.sh
```

安装成功后，将在 `ssb-dbgen/` 目录下生成 `dbgen` 二进制文件。

3.1.7.2 7.2 生成 SSB 测试集

执行以下脚本生成 SSB 数据集：

```
sh bin/gen-ssb-data.sh -s 1000
```

> 注 1：通过 `sh gen-ssb-data.sh -h` 查看脚本帮助。
>
> 注 2：数据会以 `.tbl` 为后缀生成在 `ssb-data/` 目录下。文件总大小约 600GB。生成时间可能在数分钟到 1 小时不等。
>
> 注 3：默认生成 100G 的标准测试数据集

3.1.7.3 7.3 建表

3.1.7.3.1 7.3.1 准备 `doris-cluster.conf` 文件

在调用导入脚本前，需要将 FE 的 ip 端口等信息写在 `doris-cluster.conf` 文件中。

文件位置在 `${DORIS_HOME}/tools/ssb-tools/conf/` 目录下。

文件内容包括 FE 的 ip，HTTP 端口，用户名，密码以及待导入数据的 DB 名称：

```
## Any of FE host
export FE_HOST='127.0.0.1'
## http_port in fe.conf
```

```
export FE_HTTP_PORT=8030
## query_port in fe.conf
export FE_QUERY_PORT=9030
## Doris username
export USER='root'
## Doris password
export PASSWORD=''
## The database where SSB tables located
export DB='ssb'
```

3.1.7.3.2 7.3.2 执行以下脚本生成创建 SSB 表

```
sh bin/create-ssb-tables.sh -s 1000
```

或者复制 create-ssb-tables.sql 和 create-ssb-flat-table.sql 中的建表语句，在 MySQL 客户端中执行。

3.1.7.4 7.4 导入数据

我们使用以下命令完成 SSB 测试集所有数据导入及 SSB FLAT 宽表数据合成并导入到表里。

```
sh bin/load-ssb-data.sh
```

3.1.7.5 7.5 检查导入数据

```
select count(*) from part;
select count(*) from customer;
select count(*) from supplier;
select count(*) from dates;
select count(*) from lineorder;
select count(*) from lineorder_flat;
```

3.1.7.6 7.6 查询测试

SSB-FIAT 查询语句：ssb-flat-queries

标准 SSB 查询语句：ssb-queries

3.1.7.6.1 7.6.1 SSB FLAT 测试 SQL

```
--Q1.1
SELECT SUM(LO_EXTENDEDPRICE * LO_DISCOUNT) AS revenue
FROM lineorder_flat
WHERE
    LO_ORDERDATE >= 19930101
    AND LO_ORDERDATE <= 19931231
```

```
    AND LO_DISCOUNT BETWEEN 1 AND 3
    AND LO_QUANTITY < 25;

--Q1.2
SELECT SUM(LO_EXTENDEDPRICE * LO_DISCOUNT) AS revenue
FROM lineorder_flat
WHERE
    LO_ORDERDATE >= 19940101
  AND LO_ORDERDATE <= 19940131
  AND LO_DISCOUNT BETWEEN 4 AND 6
  AND LO_QUANTITY BETWEEN 26 AND 35;

--Q1.3
SELECT SUM(LO_EXTENDEDPRICE * LO_DISCOUNT) AS revenue
FROM lineorder_flat
WHERE
    weekofyear(LO_ORDERDATE) = 6
  AND LO_ORDERDATE >= 19940101
  AND LO_ORDERDATE <= 19941231
  AND LO_DISCOUNT BETWEEN 5 AND 7
  AND LO_QUANTITY BETWEEN 26 AND 35;

--Q2.1
SELECT
    SUM(LO_REVENUE), (LO_ORDERDATE DIV 10000) AS YEAR,
    P_BRAND
FROM lineorder_flat
WHERE P_CATEGORY = 'MFGR#12' AND S_REGION = 'AMERICA'
GROUP BY YEAR, P_BRAND
ORDER BY YEAR, P_BRAND;

--Q2.2
SELECT
    SUM(LO_REVENUE), (LO_ORDERDATE DIV 10000) AS YEAR,
    P_BRAND
FROM lineorder_flat
WHERE
    P_BRAND >= 'MFGR#2221'
  AND P_BRAND <= 'MFGR#2228'
  AND S_REGION = 'ASIA'
GROUP BY YEAR, P_BRAND
ORDER BY YEAR, P_BRAND;

--Q2.3
SELECT
```

```
    SUM(LO_REVENUE), (LO_ORDERDATE DIV 10000) AS YEAR,
    P_BRAND
FROM lineorder_flat
WHERE
    P_BRAND = 'MFGR#2239'
  AND S_REGION = 'EUROPE'
GROUP BY YEAR, P_BRAND
ORDER BY YEAR, P_BRAND;

--Q3.1
SELECT
    C_NATION,
    S_NATION, (LO_ORDERDATE DIV 10000) AS YEAR,
    SUM(LO_REVENUE) AS revenue
FROM lineorder_flat
WHERE
    C_REGION = 'ASIA'
  AND S_REGION = 'ASIA'
  AND LO_ORDERDATE >= 19920101
  AND LO_ORDERDATE <= 19971231
GROUP BY C_NATION, S_NATION, YEAR
ORDER BY YEAR ASC, revenue DESC;

--Q3.2
SELECT
    C_CITY,
    S_CITY, (LO_ORDERDATE DIV 10000) AS YEAR,
    SUM(LO_REVENUE) AS revenue
FROM lineorder_flat
WHERE
    C_NATION = 'UNITED STATES'
  AND S_NATION = 'UNITED STATES'
  AND LO_ORDERDATE >= 19920101
  AND LO_ORDERDATE <= 19971231
GROUP BY C_CITY, S_CITY, YEAR
ORDER BY YEAR ASC, revenue DESC;

--Q3.3
SELECT
    C_CITY,
    S_CITY, (LO_ORDERDATE DIV 10000) AS YEAR,
    SUM(LO_REVENUE) AS revenue
FROM lineorder_flat
WHERE
    C_CITY IN ('UNITED KI1', 'UNITED KI5')
```

```
  AND S_CITY IN ('UNITED KI1', 'UNITED KI5')
  AND LO_ORDERDATE >= 19920101
  AND LO_ORDERDATE <= 19971231
GROUP BY C_CITY, S_CITY, YEAR
ORDER BY YEAR ASC, revenue DESC;

--Q3.4
SELECT
    C_CITY,
    S_CITY, (LO_ORDERDATE DIV 10000) AS YEAR,
    SUM(LO_REVENUE) AS revenue
FROM lineorder_flat
WHERE
    C_CITY IN ('UNITED KI1', 'UNITED KI5')
  AND S_CITY IN ('UNITED KI1', 'UNITED KI5')
  AND LO_ORDERDATE >= 19971201
  AND LO_ORDERDATE <= 19971231
GROUP BY C_CITY, S_CITY, YEAR
ORDER BY YEAR ASC, revenue DESC;

--Q4.1
SELECT (LO_ORDERDATE DIV 10000) AS YEAR,
    C_NATION,
    SUM(LO_REVENUE - LO_SUPPLYCOST) AS profit
FROM lineorder_flat
WHERE
    C_REGION = 'AMERICA'
  AND S_REGION = 'AMERICA'
  AND P_MFGR IN ('MFGR#1', 'MFGR#2')
GROUP BY YEAR, C_NATION
ORDER BY YEAR ASC, C_NATION ASC;

--Q4.2
SELECT (LO_ORDERDATE DIV 10000) AS YEAR,
    S_NATION,
    P_CATEGORY,
    SUM(LO_REVENUE - LO_SUPPLYCOST) AS profit
FROM lineorder_flat
WHERE
    C_REGION = 'AMERICA'
  AND S_REGION = 'AMERICA'
  AND LO_ORDERDATE >= 19970101
  AND LO_ORDERDATE <= 19981231
  AND P_MFGR IN ('MFGR#1', 'MFGR#2')
GROUP BY YEAR, S_NATION, P_CATEGORY
```

```
ORDER BY
    YEAR ASC,
    S_NATION ASC,
    P_CATEGORY ASC;

--Q4.3
SELECT (LO_ORDERDATE DIV 10000) AS YEAR,
    S_CITY,
    P_BRAND,
    SUM(LO_REVENUE - LO_SUPPLYCOST) AS profit
FROM lineorder_flat
WHERE
    S_NATION = 'UNITED STATES'
  AND LO_ORDERDATE >= 19970101
  AND LO_ORDERDATE <= 19981231
  AND P_CATEGORY = 'MFGR#14'
GROUP BY YEAR, S_CITY, P_BRAND
ORDER BY YEAR ASC, S_CITY ASC, P_BRAND ASC;
```

3.1.7.6.2 7.6.2 SSB 标准测试 SQL

```
--Q1.1
SELECT SUM(lo_extendedprice * lo_discount) AS REVENUE
FROM lineorder, dates
WHERE
    lo_orderdate = d_datekey
  AND d_year = 1993
  AND lo_discount BETWEEN 1 AND 3
  AND lo_quantity < 25;

--Q1.2
SELECT SUM(lo_extendedprice * lo_discount) AS REVENUE
FROM lineorder, dates
WHERE
    lo_orderdate = d_datekey
  AND d_yearmonth = 'Jan1994'
  AND lo_discount BETWEEN 4 AND 6
  AND lo_quantity BETWEEN 26 AND 35;

--Q1.3
SELECT
    SUM(lo_extendedprice * lo_discount) AS REVENUE
FROM lineorder, dates
WHERE
    lo_orderdate = d_datekey
```

```
  AND d_weeknuminyear = 6
  AND d_year = 1994
  AND lo_discount BETWEEN 5 AND 7
  AND lo_quantity BETWEEN 26 AND 35;

--Q2.1
SELECT SUM(lo_revenue), d_year, p_brand
FROM lineorder, dates, part, supplier
WHERE
    lo_orderdate = d_datekey
  AND lo_partkey = p_partkey
  AND lo_suppkey = s_suppkey
  AND p_category = 'MFGR#12'
  AND s_region = 'AMERICA'
GROUP BY d_year, p_brand
ORDER BY p_brand;

--Q2.2
SELECT SUM(lo_revenue), d_year, p_brand
FROM lineorder, dates, part, supplier
WHERE
    lo_orderdate = d_datekey
  AND lo_partkey = p_partkey
  AND lo_suppkey = s_suppkey
  AND p_brand BETWEEN 'MFGR#2221' AND 'MFGR#2228'
  AND s_region = 'ASIA'
GROUP BY d_year, p_brand
ORDER BY d_year, p_brand;

--Q2.3
SELECT SUM(lo_revenue), d_year, p_brand
FROM lineorder, dates, part, supplier
WHERE
    lo_orderdate = d_datekey
  AND lo_partkey = p_partkey
  AND lo_suppkey = s_suppkey
  AND p_brand = 'MFGR#2239'
  AND s_region = 'EUROPE'
GROUP BY d_year, p_brand
ORDER BY d_year, p_brand;

--Q3.1
SELECT
    c_nation,
    s_nation,
```

```
    d_year,
    SUM(lo_revenue) AS REVENUE
FROM customer, lineorder, supplier, dates
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_orderdate = d_datekey
  AND c_region = 'ASIA'
  AND s_region = 'ASIA'
  AND d_year >= 1992
  AND d_year <= 1997
GROUP BY c_nation, s_nation, d_year
ORDER BY d_year ASC, REVENUE DESC;

--Q3.2
SELECT
    c_city,
    s_city,
    d_year,
    SUM(lo_revenue) AS REVENUE
FROM customer, lineorder, supplier, dates
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_orderdate = d_datekey
  AND c_nation = 'UNITED STATES'
  AND s_nation = 'UNITED STATES'
  AND d_year >= 1992
  AND d_year <= 1997
GROUP BY c_city, s_city, d_year
ORDER BY d_year ASC, REVENUE DESC;

--Q3.3
SELECT
    c_city,
    s_city,
    d_year,
    SUM(lo_revenue) AS REVENUE
FROM customer, lineorder, supplier, dates
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_orderdate = d_datekey
  AND (
            c_city = 'UNITED KI1'
```

```
        OR c_city = 'UNITED KI5'
    )
  AND (
            s_city = 'UNITED KI1'
        OR s_city = 'UNITED KI5'
    )
  AND d_year >= 1992
  AND d_year <= 1997
GROUP BY c_city, s_city, d_year
ORDER BY d_year ASC, REVENUE DESC;

--Q3.4
SELECT
    c_city,
    s_city,
    d_year,
    SUM(lo_revenue) AS REVENUE
FROM customer, lineorder, supplier, dates
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_orderdate = d_datekey
  AND (
            c_city = 'UNITED KI1'
        OR c_city = 'UNITED KI5'
    )
  AND (
            s_city = 'UNITED KI1'
        OR s_city = 'UNITED KI5'
    )
  AND d_yearmonth = 'Dec1997'
GROUP BY c_city, s_city, d_year
ORDER BY d_year ASC, REVENUE DESC;

--Q4.1
SELECT
    d_year,
    c_nation,
    SUM(lo_revenue - lo_supplycost) AS PROFIT
FROM dates, customer, supplier, part, lineorder
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_partkey = p_partkey
  AND lo_orderdate = d_datekey
```

```
  AND c_region = 'AMERICA'
  AND s_region = 'AMERICA'
  AND (
            p_mfgr = 'MFGR#1'
        OR p_mfgr = 'MFGR#2'
    )
GROUP BY d_year, c_nation
ORDER BY d_year, c_nation;

--Q4.2
SELECT
    d_year,
    s_nation,
    p_category,
    SUM(lo_revenue - lo_supplycost) AS PROFIT
FROM dates, customer, supplier, part, lineorder
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_partkey = p_partkey
  AND lo_orderdate = d_datekey
  AND c_region = 'AMERICA'
  AND s_region = 'AMERICA'
  AND (
            d_year = 1997
        OR d_year = 1998
    )
  AND (
            p_mfgr = 'MFGR#1'
        OR p_mfgr = 'MFGR#2'
    )
GROUP BY d_year, s_nation, p_category
ORDER BY d_year, s_nation, p_category;

--Q4.3
SELECT
    d_year,
    s_city,
    p_brand,
    SUM(lo_revenue - lo_supplycost) AS PROFIT
FROM dates, customer, supplier, part, lineorder
WHERE
    lo_custkey = c_custkey
  AND lo_suppkey = s_suppkey
  AND lo_partkey = p_partkey
```

```
  AND lo_orderdate = d_datekey
  AND s_nation = 'UNITED STATES'
  AND (
           d_year = 1997
       OR d_year = 1998
    )
  AND p_category = 'MFGR#14'
GROUP BY d_year, s_city, p_brand
ORDER BY d_year, s_city, p_brand;
```

3.2 TPC-H Benchmark

TPC-H 是一个决策支持基准（Decision Support Benchmark），它由一套面向业务的特别查询和并发数据修改组成。查询和填充数据库的数据具有广泛的行业相关性。这个基准测试演示了检查大量数据、执行高度复杂的查询并回答关键业务问题的决策支持系统。TPC-H 报告的性能指标称为 TPC-H 每小时复合查询性能指标 (QphH@Size)，反映了系统处理查询能力的多个方面。这些方面包括执行查询时所选择的数据库大小，由单个流提交查询时的查询处理能力，以及由多个并发用户提交查询时的查询吞吐量。

本文档主要介绍 Doris 在 TPC-H 1000G 测试集上的性能表现。

在 TPC-H 标准测试数据集上的 22 个查询上，我们基于 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行了对比测试。3.x 版本存算一体模式的性能参照 2.1.x 版本。

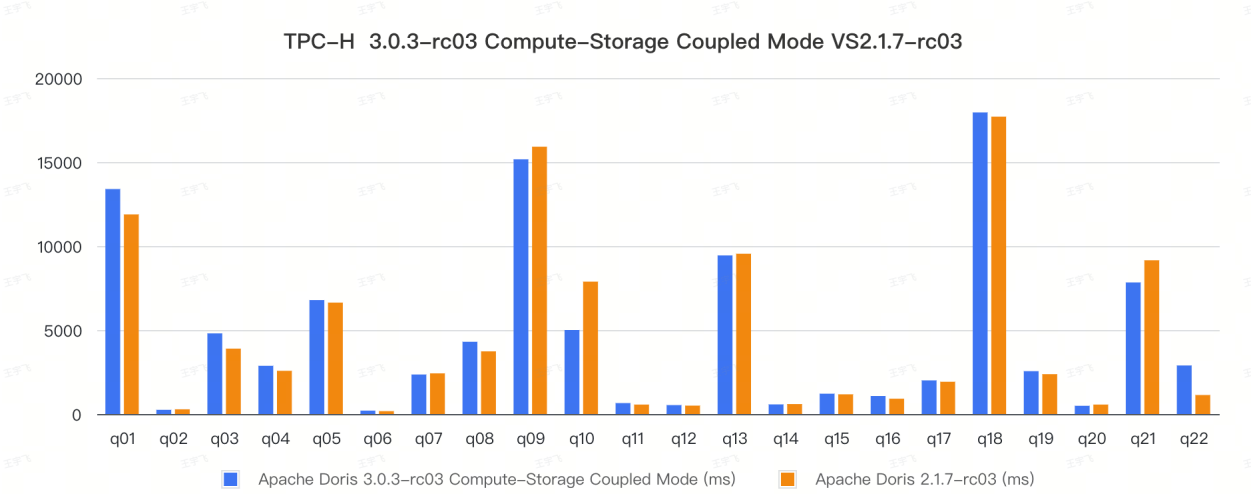


图 127: TPCH_1000G

3.2.1 1. 硬件环境

硬件	配置说明
机器数量	4 台阿里云主机（1 个 FE，3 个 BE）

硬件	配置说明
CPU	Intel Xeon (Ice Lake) Platinum 8369B 32 核
内存	128G
磁盘	阿里云 ESSD (PL0)

3.2.2 2. 软件环境

- Doris 部署 3BE 1FE
- 内核版本：Linux version 5.15.0-101-generic
- 操作系统版本：Ubuntu 20.04 LTS (Focal Fossa)
- Doris 软件版本：Apache Doris 3.0.3-rc03 (存算分离模式), Apache Doris 2.1.7-rc03
- JDK：openjdk version “17.0.2”

3.2.3 3. 测试数据量

整个测试模拟生成 TPC-H 1000G 的数据分别导入到 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行测试，下面是表的相关说明及数据量。

TPC-H 表名	行数	备注
REGION	5	区域表
NATION	25	国家表
SUPPLIER	1000 万	供应商表
PART	2 亿	零部件表
PARTSUPP	8 亿	零部件供应表
CUSTOMER	1.5 亿	客户表
ORDERS	15 亿	订单表
LINEITEM	60 亿	订单明细表

3.2.4 4. 测试 SQL

TPC-H 22 个测试查询语句：TPCH-Query-SQL

3.2.5 5. 测试结果

这里我们使用 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行对比测试，测试结果如下：

Query	Apache Doris 3.0.3-rc03 Compute-Storage Coupled Mode (ms)	Apache Doris 2.1.7-rc03 (ms)
q01	13390	11880
q02	250	280
q03	4800	3890
q04	2870	2570
q05	6780	6630

Query	Apache Doris 3.0.3-rc03 Compute-Storage Coupled Mode (ms)	Apache Doris 2.1.7-rc03 (ms)
q06	200	170
q07	2350	2420
q08	4300	3730
q09	15160	15910
q10	5000	7880
q11	650	560
q12	530	500
q13	9440	9540
q14	570	590
q15	1210	1170
q16	1070	910
q17	2000	1920
q18	17950	17700
q19	2550	2370
q20	490	560
q21	7830	9150
q22	2890	1130
Total	102280	101460

3.2.6 6. 环境准备

请先参照官方文档进行 Doris 的安装部署，以获得一个正常运行中的 Doris 集群（至少包含 1 FE 1 BE，推荐 1 FE 3 BE）。

3.2.7 7. 数据准备

3.2.7.1 7.1 下载安装 TPC-H 数据生成工具

执行以下脚本下载并编译 tpch-tools 工具。

```
sh bin/build-tpch-dbgen.sh
```

安装成功后，将在 TPC-H_Tools_v3.0.0/ 目录下生成 dbgen 二进制文件。

3.2.7.2 7.2 生成 TPC-H 测试集

执行以下脚本生成 TPC-H 数据集：

```
sh bin/gen-tpch-data.sh -s 1000
```

> 注 1：通过 `sh gen-tpch-data.sh -h` 查看脚本帮助。

注 2：数据会以 .tbl 为后缀生成在 tpch-data/ 目录下。文件总大小约 1000GB。生成时间可能在数分钟到 1 小时不等。

注 3：默认生成 100G 的标准测试数据集

3.2.7.3 7.3 建表

3.2.7.3.1 7.3.1 准备 doris-cluster.conf 文件

在调用导入脚本前，需要将 FE 的 ip 端口等信息写在 doris-cluster.conf 文件中。

文件位置在 ${DORIS_HOME}/tools/tpch-tools/conf/ 目录下。

文件内容包括 FE 的 ip，HTTP 端口，用户名，密码以及待导入数据的 DB 名称：

```
## Any of FE host
export FE_HOST='127.0.0.1'
## http_port in fe.conf
export FE_HTTP_PORT=8030
## query_port in fe.conf
export FE_QUERY_PORT=9030
## Doris username
export USER='root'
## Doris password
export PASSWORD=''
## The database where TPC-H tables located
export DB='tpch'
```

3.2.7.3.2 7.3.2 执行以下脚本生成创建 TPC-H 表

```
sh bin/create-tpch-tables.sh -s 1000
```

或者复制 create-tpch-tables.sql 中的建表语句，在 Doris 中执行。

3.2.7.4 7.4 导入数据

通过下面的命令执行数据导入：

```
sh bin/load-tpch-data.sh
```

3.2.7.5 7.5 检查导入数据

执行下面的 SQL 语句检查导入的数据与上面的数据量一致。

```
select count(*)  from  lineitem;
select count(*)  from  orders;
select count(*)  from  partsupp;
select count(*)  from  part;
select count(*)  from  customer;
select count(*)  from  supplier;
select count(*)  from  nation;
select count(*)  from  region;
select count(*)  from  revenue0;
```

3.2.7.6 7.6 查询测试

3.2.7.7 7.6.1 执行查询脚本

执行上面的测试 SQL 或者执行下面的命令

```
sh bin/run-tpch-queries.sh -s 1000
```

3.2.7.8 7.6.2 单个 SQL 执行

下面是测试时使用的 SQL 语句，你也可以从代码库里获取最新的 SQL。最新测试查询语句地址：TPC-H 测试查询语句

```
--Q1
select
    l_returnflag,
    l_linestatus,
    sum(l_quantity) as sum_qty,
    sum(l_extendedprice) as sum_base_price,
    sum(l_extendedprice * (1 - l_discount)) as sum_disc_price,
    sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)) as sum_charge,
    avg(l_quantity) as avg_qty,
    avg(l_extendedprice) as avg_price,
    avg(l_discount) as avg_disc,
    count(*) as count_order
from
    lineitem
where
    l_shipdate <= date '1998-12-01' - interval '90' day
group by
    l_returnflag,
    l_linestatus
order by
    l_returnflag,
```

```
    l_linestatus;

--Q2
select
    s_acctbal,
    s_name,
    n_name,
    p_partkey,
    p_mfgr,
    s_address,
    s_phone,
    s_comment
from
    part,
    supplier,
    partsupp,
    nation,
    region
where
    p_partkey = ps_partkey
    and s_suppkey = ps_suppkey
    and p_size = 15
    and p_type like '%BRASS'
    and s_nationkey = n_nationkey
    and n_regionkey = r_regionkey
    and r_name = 'EUROPE'
    and ps_supplycost = (
        select
            min(ps_supplycost)
        from
            partsupp,
            supplier,
            nation,
            region
        where
        p_partkey = ps_partkey
        and s_suppkey = ps_suppkey
        and s_nationkey = n_nationkey
        and n_regionkey = r_regionkey
        and r_name = 'EUROPE'
)
order by
    s_acctbal desc,
    n_name,
    s_name,
```

```
    p_partkey
limit 100;

--Q3
select
    l_orderkey,
    sum(l_extendedprice * (1 - l_discount)) as revenue,
    o_orderdate,
    o_shippriority
from
    customer,
    orders,
    lineitem
where
    c_mktsegment = 'BUILDING'
    and c_custkey = o_custkey
    and l_orderkey = o_orderkey
    and o_orderdate < date '1995-03-15'
    and l_shipdate > date '1995-03-15'
group by
    l_orderkey,
    o_orderdate,
    o_shippriority
order by
    revenue desc,
    o_orderdate
limit 10;

--Q4
select
    o_orderpriority,
    count(*) as order_count
from
    orders
where
    o_orderdate >= date '1993-07-01'
    and o_orderdate < date '1993-07-01' + interval '3' month
    and exists (
        select
            *
        from
            lineitem
        where
                l_orderkey = o_orderkey
          and l_commitdate < l_receiptdate
```

```
    )
group by
    o_orderpriority
order by
    o_orderpriority;

--Q5
select
    n_name,
    sum(l_extendedprice * (1 - l_discount)) as revenue
from
    customer,
    orders,
    lineitem,
    supplier,
    nation,
    region
where
    c_custkey = o_custkey
    and l_orderkey = o_orderkey
    and l_suppkey = s_suppkey
    and c_nationkey = s_nationkey
    and s_nationkey = n_nationkey
    and n_regionkey = r_regionkey
    and r_name = 'ASIA'
    and o_orderdate >= date '1994-01-01'
    and o_orderdate < date '1994-01-01' + interval '1' year
group by
    n_name
order by
    revenue desc;

--Q6
select
    sum(l_extendedprice * l_discount) as revenue
from
    lineitem
where
    l_shipdate >= date '1994-01-01'
    and l_shipdate < date '1994-01-01' + interval '1' year
    and l_discount between .06 - 0.01 and .06 + 0.01
    and l_quantity < 24;

--Q7
select
```

```
    supp_nation,
    cust_nation,
    l_year,
    sum(volume) as revenue
from
    (
        select
            n1.n_name as supp_nation,
            n2.n_name as cust_nation,
            extract(year from l_shipdate) as l_year,
            l_extendedprice * (1 - l_discount) as volume
        from
            supplier,
            lineitem,
            orders,
            customer,
            nation n1,
            nation n2
        where
            s_suppkey = l_suppkey
            and o_orderkey = l_orderkey
            and c_custkey = o_custkey
            and s_nationkey = n1.n_nationkey
            and c_nationkey = n2.n_nationkey
            and (
                (n1.n_name = 'FRANCE' and n2.n_name = 'GERMANY')
                or (n1.n_name = 'GERMANY' and n2.n_name = 'FRANCE')
            )
            and l_shipdate between date '1995-01-01' and date '1996-12-31'
    ) as shipping
group by
    supp_nation,
    cust_nation,
    l_year
order by
    supp_nation,
    cust_nation,
    l_year;

--Q8

select
    o_year,
    sum(case
        when nation = 'BRAZIL' then volume
```

```
        else 0
    end) / sum(volume) as mkt_share
from
    (
        select
            extract(year from o_orderdate) as o_year,
            l_extendedprice * (1 - l_discount) as volume,
            n2.n_name as nation
        from
            part,
            supplier,
            lineitem,
            orders,
            customer,
            nation n1,
            nation n2,
            region
        where
            p_partkey = l_partkey
            and s_suppkey = l_suppkey
            and l_orderkey = o_orderkey
            and o_custkey = c_custkey
            and c_nationkey = n1.n_nationkey
            and n1.n_regionkey = r_regionkey
            and r_name = 'AMERICA'
            and s_nationkey = n2.n_nationkey
            and o_orderdate between date '1995-01-01' and date '1996-12-31'
            and p_type = 'ECONOMY ANODIZED STEEL'
    ) as all_nations
group by
    o_year
order by
    o_year;

--Q9
select
    nation,
    o_year,
    sum(amount) as sum_profit
from
    (
        select
            n_name as nation,
            extract(year from o_orderdate) as o_year,
            l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity as amount
```

```
        from
            part,
            supplier,
            lineitem,
            partsupp,
            orders,
            nation
        where
            s_suppkey = l_suppkey
            and ps_suppkey = l_suppkey
            and ps_partkey = l_partkey
            and p_partkey = l_partkey
            and o_orderkey = l_orderkey
            and s_nationkey = n_nationkey
            and p_name like '%green%'
    ) as profit
group by
    nation,
    o_year
order by
    nation,
    o_year desc;

--Q10
select
    c_custkey,
    c_name,
    sum(l_extendedprice * (1 - l_discount)) as revenue,
    c_acctbal,
    n_name,
    c_address,
    c_phone,
    c_comment
from
    customer,
    orders,
    lineitem,
    nation
where
    c_custkey = o_custkey
    and l_orderkey = o_orderkey
    and o_orderdate >= date '1993-10-01'
    and o_orderdate < date '1993-10-01' + interval '3' month
    and l_returnflag = 'R'
    and c_nationkey = n_nationkey
```

```
group by
    c_custkey,
    c_name,
    c_acctbal,
    c_phone,
    n_name,
    c_address,
    c_comment
order by
    revenue desc
limit 20;


--Q11
select
    ps_partkey,
    sum(ps_supplycost * ps_availqty) as value
from
    partsupp,
    supplier,
    nation
where
    ps_suppkey = s_suppkey
    and s_nationkey = n_nationkey
    and n_name = 'GERMANY'
group by
    ps_partkey having
    sum(ps_supplycost * ps_availqty) > (
        select
        sum(ps_supplycost * ps_availqty) * 0.000002
        from
            partsupp,
            supplier,
            nation
        where
            ps_suppkey = s_suppkey
            and s_nationkey = n_nationkey
            and n_name = 'GERMANY'
    )
order by
    value desc;

--Q12
select
    l_shipmode,
```

```
    sum(case
        when o_orderpriority = '1-URGENT'
            or o_orderpriority = '2-HIGH'
            then 1
        else 0
    end) as high_line_count,
    sum(case
        when o_orderpriority <> '1-URGENT'
            and o_orderpriority <> '2-HIGH'
            then 1
        else 0
    end) as low_line_count
from
    orders,
    lineitem
where
    o_orderkey = l_orderkey
    and l_shipmode in ('MAIL', 'SHIP')
    and l_commitdate < l_receiptdate
    and l_shipdate < l_commitdate
    and l_receiptdate >= date '1994-01-01'
    and l_receiptdate < date '1994-01-01' + interval '1' year
group by
    l_shipmode
order by
    l_shipmode;

--Q13
select
    c_count,
    count(*) as custdist
from
    (
        select
            c_custkey,
            count(o_orderkey) as c_count
        from
            customer left outer join orders on
                c_custkey = o_custkey
                and o_comment not like '%special%requests%'
        group by
            c_custkey
    ) as c_orders
group by
    c_count
```

```
order by
    custdist desc,
    c_count desc;

--Q14
select
    100.00 * sum(case
        when p_type like 'PROMO%'
            then l_extendedprice * (1 - l_discount)
        else 0
    end) / sum(l_extendedprice * (1 - l_discount)) as promo_revenue
from
    lineitem,
    part
where
    l_partkey = p_partkey
    and l_shipdate >= date '1995-09-01'
    and l_shipdate < date '1995-09-01' + interval '1' month;

--Q15
select
    s_suppkey,
    s_name,
    s_address,
    s_phone,
    total_revenue
from
    supplier,
    revenue0
where
    s_suppkey = supplier_no
    and total_revenue = (
        select
            max(total_revenue)
        from
            revenue0
    )
order by
    s_suppkey;

--Q16
select
    p_brand,
    p_type,
    p_size,
```

```
    count(distinct ps_suppkey) as supplier_cnt
from
    partsupp,
    part
where
    p_partkey = ps_partkey
    and p_brand <> 'Brand#45'
    and p_type not like 'MEDIUM POLISHED%'
    and p_size in (49, 14, 23, 45, 19, 3, 36, 9)
    and ps_suppkey not in (
        select
            s_suppkey
        from
            supplier
        where
            s_comment like '%Customer%Complaints%'
    )
group by
    p_brand,
    p_type,
    p_size
order by
    supplier_cnt desc,
    p_brand,
    p_type,
    p_size;

--Q17
select
    sum(l_extendedprice) / 7.0 as avg_yearly
from
    lineitem,
    part
where
    p_partkey = l_partkey
    and p_brand = 'Brand#23'
    and p_container = 'MED BOX'
    and l_quantity < (
        select
            0.2 * avg(l_quantity)
        from
            lineitem
        where
            l_partkey = p_partkey
    );
```

```
--Q18
select
    c_name,
    c_custkey,
    o_orderkey,
    o_orderdate,
    o_totalprice,
    sum(l_quantity)
from
    customer,
    orders,
    lineitem
where
    o_orderkey  in  (
        select
            l_orderkey
        from
            lineitem
        group  by
            l_orderkey  having
                sum(l_quantity)  >  300
    )
    and  c_custkey  =  o_custkey
    and  o_orderkey  =  l_orderkey
group  by
    c_name,
    c_custkey,
    o_orderkey,
    o_orderdate,
    o_totalprice
order  by
    o_totalprice  desc,
    o_orderdate
limit  100;


--Q19
select
    sum(l_extendedprice* (1 - l_discount)) as revenue
from
    lineitem,
    part
where
    (
```

```
        p_partkey = l_partkey
        and p_brand = 'Brand#12'
        and p_container in ('SM CASE', 'SM BOX', 'SM PACK', 'SM PKG')
        and l_quantity >= 1 and l_quantity <= 1 + 10
        and p_size between 1 and 5
        and l_shipmode in ('AIR', 'AIR REG')
        and l_shipinstruct = 'DELIVER IN PERSON'
    )
    or
    (
        p_partkey = l_partkey
        and p_brand = 'Brand#23'
        and p_container in ('MED BAG', 'MED BOX', 'MED PKG', 'MED PACK')
        and l_quantity >= 10 and l_quantity <= 10 + 10
        and p_size between 1 and 10
        and l_shipmode in ('AIR', 'AIR REG')
        and l_shipinstruct = 'DELIVER IN PERSON'
    )
    or
    (
        p_partkey = l_partkey
        and p_brand = 'Brand#34'
        and p_container in ('LG CASE', 'LG BOX', 'LG PACK', 'LG PKG')
        and l_quantity >= 20 and l_quantity <= 20 + 10
        and p_size between 1 and 15
        and l_shipmode in ('AIR', 'AIR REG')
        and l_shipinstruct = 'DELIVER IN PERSON'
    );

--Q20
select
    s_name,
    s_address
from
    supplier,
    nation
where
    s_suppkey in (
        select
            ps_suppkey
        from
            partsupp
        where
            ps_partkey in (
                select
```

```
                    p_partkey
                from
                    part
                where
                        p_name like 'forest%'
            )
            and ps_availqty > (
                select
                    0.5 * sum(l_quantity)
                from
                    lineitem
                where
                    l_partkey = ps_partkey
                    and l_suppkey = ps_suppkey
                    and l_shipdate >= date '1994-01-01'
                    and l_shipdate < date '1994-01-01' + interval '1' year
            )
    )
    and s_nationkey = n_nationkey
    and n_name = 'CANADA'
order by
    s_name;

--Q21
select
    s_name,
    count(*) as numwait
from
    supplier,
    lineitem l1,
    orders,
    nation
where
    s_suppkey = l1.l_suppkey
    and o_orderkey = l1.l_orderkey
    and o_orderstatus = 'F'
    and l1.l_receiptdate > l1.l_commitdate
    and exists (
        select
            *
        from
            lineitem l2
        where
                l2.l_orderkey = l1.l_orderkey
          and l2.l_suppkey <> l1.l_suppkey
```

```
    )
    and not exists (
        select
            *
        from
            lineitem l3
        where
                l3.l_orderkey = l1.l_orderkey
          and l3.l_suppkey <> l1.l_suppkey
          and l3.l_receiptdate > l3.l_commitdate
    )
    and s_nationkey = n_nationkey
    and n_name = 'SAUDI ARABIA'
group by
    s_name
order by
    numwait desc,
    s_name
limit 100;

--Q22
select
    cntrycode,
    count(*) as numcust,
    sum(c_acctbal) as totacctbal
from
    (
        select
            substring(c_phone, 1, 2) as cntrycode,
            c_acctbal
        from
            customer
        where
            substring(c_phone, 1, 2) in
            ('13', '31', '23', '29', '30', '18', '17')
            and c_acctbal > (
                select
                    avg(c_acctbal)
                from
                    customer
                where
                    c_acctbal > 0.00
                    and substring(c_phone, 1, 2) in
                      ('13', '31', '23', '29', '30', '18', '17')
            )
```

```
            and not exists (
                select
                    *
                from
                    orders
                where
                    o_custkey = c_custkey
            )
    ) as custsale
group by
    cntrycode
order by
    cntrycode;
```

3.3 TPC-DS Benchmark

TPC-DS（Transaction Processing Performance Council Decision Support Benchmark）是一个以决策支持为重点的基准测试，旨在评估数据仓库和分析系统的性能。它是由 TPC（Transaction Processing Performance Council）组织开发的，用于比较不同系统在处理复杂查询和大规模数据分析方面的能力。

TPC-DS 的设计目标是模拟现实世界中的复杂决策支持工作负载。它通过一系列复杂的查询和数据操作来测试系统的性能，包括联接、聚合、排序、过滤、子查询等。这些查询模式涵盖了从简单到复杂的各种场景，如报表生成、数据挖掘、OLAP（联机分析处理）等。

本文档主要介绍 Doris 在 TPC-DS 1000G 测试集上的性能表现。

在 TPC-DS 标准测试数据集上的 99 个查询上，我们基于 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行了对比测试。3.x 版本存算一体模式的性能参照 2.1.x 版本。

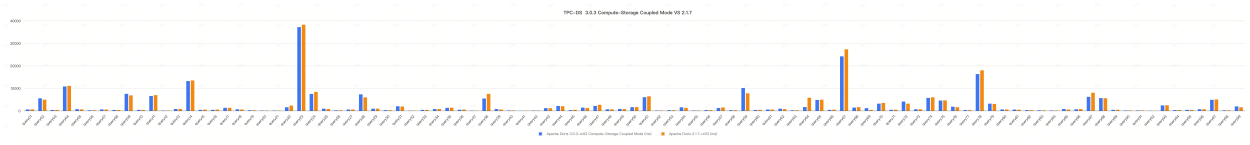


图 128: TPCDS_1000G

3.3.1 1. 硬件环境

硬件	配置说明
机器数量	4 台阿里云主机（1 个 FE，3 个 BE）
CPU	Intel Xeon (Ice Lake) Platinum 8369B 32 核
内存	128G
磁盘	阿里云 ESSD (PL0)

3.3.2 2. 软件环境

- Doris 部署 3BE 1FE
- 内核版本：Linux version 5.15.0-101-generic
- 操作系统版本：Ubuntu 20.04 LTS (Focal Fossa)
- Doris 软件版本：Apache Doris 3.0.3-rc03 (存算分离模式), Apache Doris 2.1.7-rc03
- JDK：openjdk version “17.0.2”

3.3.3 3. 测试数据量

整个测试模拟生成 TPC-DS 1000G 的数据分别导入到 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行测试，下面是表的相关说明及数据量。

TPC-DS 表名	行数
customer_demographics	1,920,800
reason	65
warehouse	20
date_dim	73,049
catalog_sales	1,439,980,416
call_center	42
inventory	783,000,000
catalog_returns	143,996,756
household_demographics	7,200
customer_address	6,000,000
income_band	20
catalog_page	30,000
item	300,000
web_returns	71,997,522
web_site	54
promotion	1,500
web_sales	720,000,376
store	1,002
web_page	3,000
time_dim	86,400
store_returns	287,999,764
store_sales	2,879,987,999
ship_mode	20
customer	12,000,000

3.3.4 4. 测试 SQL

TPC-DS 99 个测试查询语句：TPC-DS-Query-SQL

3.3.5 5. 测试结果

这里我们使用 Apache Doris 3.0.3-rc03 (存算分离模式) 和 Apache Doris 2.1.7-rc03 版本进行对比测试，测试结果如下：

Query	Apache Doris 3.0.3-rc03 Compute-Storage Coupled Mode (ms)	Apache Doris 2.1.7-rc03 (ms)
query01	580	630
query02	5540	4930
query03	350	360
query04	10790	11070
query05	710	620
query06	230	220
query07	590	550
query08	350	330
query09	7520	6830
query10	390	370
query11	6560	6960
query12	120	100
query13	780	790
query14	13200	13470
query15	400	510
query16	410	520
query17	1300	1310
query18	650	560
query19	250	200
query20	110	100
query21	110	80
query22	1570	2300
query23	37180	38240
query24	7470	8340
query25	920	780
query26	200	200
query27	550	530
query28	7300	5940
query29	920	940
query30	300	270
query31	2000	1890
query32	70	60
query33	400	350
query34	760	750
query35	1290	1370
query36	460	530
query37	80	60
query38	5450	7520
query39	760	560
query40	140	150

Query	Apache Doris 3.0.3-rc03 Compute-Storage Coupled Mode (ms)	Apache Doris 2.1.7-rc03 (ms)
query41	50	50
query42	110	100
query43	1170	1150
query44	2120	2020
query45	280	430
query46	1390	1250
query47	2160	2660
query48	660	630
query49	810	730
query50	1570	1640
query51	6030	6430
query52	120	110
query53	280	250
query54	1540	1280
query55	130	110
query56	300	290
query57	1240	1480
query58	260	240
query59	10120	7760
query60	370	380
query61	560	540
query62	920	740
query63	230	210
query64	1660	5790
query65	4800	4900
query66	400	480
query67	24190	27320
query68	1400	1600
query69	1170	380
query70	3160	3480
query71	440	460
query72	4090	3160
query73	660	660
query74	5720	5990
query75	4560	4610
query76	1800	1590
query77	330	300
query78	16300	17970
query79	3160	3040
query80	590	570
query81	540	460
query82	320	270
query83	230	220

Query	Apache Doris 3.0.3-rc03 Compute-Storage Coupled Mode (ms)	Apache Doris 2.1.7-rc03 (ms)
query84	130	130
query85	780	520
query86	660	760
query87	6200	8000
query88	5620	5560
query89	400	430
query90	150	150
query91	160	150
query92	50	40
query93	2380	2440
query94	290	340
query95	410	350
query96	680	660
query97	4870	5020
query98	200	190
query99	1940	1560
Total	251620	261320

3.3.6 6. 环境准备

请先参照官方文档进行 Doris 的安装部署，以获得一个正常运行中的 Doris 集群（至少包含 1 FE 1 BE，推荐 1 FE 3 BE）。

3.3.7 7. 数据准备

3.3.7.1 7.1 下载安装 TPC-DS 数据生成工具

执行以下脚本下载并编译 tpcds-tools 工具。

```
sh bin/build-tpcds-tools.sh
```

3.3.7.2 7.2 生成 TPC-DS 测试集

执行以下脚本生成 TPC-DS 数据集：

```
sh bin/gen-tpcds-data.sh -s 1000
```

> 注 1：通过 `sh gen-tpcds-data.sh -h` 查看脚本帮助。
>
> 注 2：数据会以 `.dat` 为后缀生成在 `tpcds-data/` 目录下。文件总大小约 1000GB。生成时间可能在数分钟到 1 小时不等。
>
> 注 3：默认生成 100G 的标准测试数据集

3.3.7.3 7.3 建表

3.3.7.3.1 7.3.1 准备 doris-cluster.conf 文件

在调用导入脚本前，需要将 FE 的 ip 端口等信息写在 doris-cluster.conf 文件中。

文件位置在 ${DORIS_HOME}/tools/tpcds-tools/conf/ 目录下。

文件内容包括 FE 的 ip，HTTP 端口，用户名，密码以及待导入数据的 DB 名称：

```
## Any of FE host
export FE_HOST='127.0.0.1'
## http_port in fe.conf
export FE_HTTP_PORT=8030
## query_port in fe.conf
export FE_QUERY_PORT=9030
## Doris username
export USER='root'
## Doris password
export PASSWORD=''
## The database where TPC-DS tables located
export DB='tpcds'
```

3.3.7.3.2 7.3.2 执行以下脚本生成创建 TPC-DS 表

```
sh bin/create-tpcds-tables.sh -s 1000
```

或者复制 create-tpcds-tables.sql 中的建表语句，在 Doris 中执行。

3.3.7.4 7.4 导入数据

通过下面的命令执行数据导入：

```
sh bin/load-tpcds-data.sh
```

3.3.7.5 7.5 查询测试

3.3.7.6 7.5.1 执行查询脚本

单个 SQL 执行或者执行下面的命令

```
sh bin/run-tpcds-queries.sh -s 1000
```

3.3.7.7 7.5.2 单个 SQL 执行

你也可以从代码库里获取最新的 SQL。最新测试查询语句地址：TPC-DS 测试查询语句

4 管理指南

4.1 集群管理

4.1.1 集群升级

Doris 提供了滚动升级的能力，在升级过程中逐步对 FE 与 BE 节点进行升级，减少停机时间，确保在升级过程中系统能够保持正常运行。

4.1.1.1 版本兼容性说明

Doris 版本号由三维组成，第一位表示重大里程碑版本，第二位表示功能版本，第三位表示 bug 修复，不在三位版本中发布新的功能。如 Doris 2.1.3 版本，其中 2 表示第 2 个里程碑版本，1 表示该里程碑下的功能版本，3 表示该功能版本下的第三个 bug fix 版本。

在版本升级时，遵循以下规则：

- 三位版本：二位版本相同时，可以跨三位版本升级，如 2.1.3 版本可以直接升级到 2.1.7 版本；
- 二位版本及一位版本：不建议跨二位版本升级，考虑到兼容性问题，建议按照二位版本号依次升级，如 3.0 版本升级到 3.3 版本，需要按照 3.0 -> 3.1 -> 3.2 -> 3.3 的执行路径升级。

详细版本说明可以参考版本规则。

4.1.1.2 升级注意事项

在升级时，需要注意以下事项：

- 版本间行为变更：在升级前需要查看 Release Note 中的行为变更以确定版本间的兼容性。
- 对集群内的任务添加重试机制：升级时节点需要依次重启，对于查询任务，Stream Load 导入作业需要添加重试机制，否则会导致任务失败；在 Routine Load 作业，通过 flink-doris-connector 或 spark-doris-connector 导入的作业，已经在代码中实现了重试机制，无需添加重试逻辑；
- 关闭副本修复与均衡功能：在升级时需要关闭副本修复与均衡功能，无论升级是否成功，升级后都需要再次打开副本修复与均衡功能。

4.1.1.3 元数据兼容性测试

> 注意
>
> 在生产环境中，建议保持 3 个以上的 FE 做高可用配置。如果只有 1 个 FE 节点，需要先做元数据兼容性测试后，再进行升级操作。元数据兼容非常重要，如果因为元数据不兼容导致的升级失败，能会导致数据丢失。建议每次升级前都进行元数据兼容性测试，在做元数据兼容性测试时，注意以下几点：

- 建议在开发机或 BE 节点上做元数据兼容性测试，尽量避免在 FE 节点上做兼容性测试
- 如果只能在 FE 节点上做兼容性测试，建议选择非 Master 节点，并停止原有 FE 进程

在升级前，建议进行元数据兼容性测试，防止升级过程中元数据不兼容导致的升级失败。

1. 备份元数据信息：

在开始升级工作前，需要备份 Master FE 节点的元数据信息。

通过 `show frontends` 中 `IsMaster` 列可以判断 Master FE 节点。在备份 FE 元信息时，无需停止 FE 节点，可以直接热备份元信息。默认情况下，FE 元数据在 `fe/doris-meta` 目录下，可以通过 `fe.conf` 文件中 `meta_dir` 参数确定元数据目录。

2. 修改测试用的 FE 的配置文件 fe.conf

```
vi ${DORIS_NEW_HOME}/conf/fe.conf
```

修改以下端口信息，将所有端口设置为与线上不同，同时修改 clusterID 参数：

```
...
## modify port
http_port = 18030
rpc_port = 19020
query_port = 19030
arrow_flight_sql_port = 19040
edit_log_port = 19010

## modify clusterIP
clusterId=<a_new_clusterID, such as 123456>
...
```

3. 将备份的 Master FE 元数据拷贝到新的兼容性测试环境中

```
cp ${DORIS_OLD_HOME}/fe/doris-meta/* ${DORIS_NEW_HOME}/fe/doris-meta
```

4. 将拷贝的元数据目文件中的 VERSION 文件中的 cluster_id 修改为新的 cluster ID，如在上例中修改为 123456：

```
vi ${DORIS_NEW_HOME}/fe/doris-meta/image/VERSION
clusterId=123456
```

5. 在测试环境中启动 FE 进程

```
sh ${DORIS_NEW_HOME}/bin/start_fe.sh --daemon --metadata_failure_recovery
```

在 2.0.2 之前的版本，需要在 fe.conf 文件中加入 metadata_failure_recovery 后在启动 FE 进程：

```
echo "metadata_failure_recovery=true" >> ${DORIS_NEW_HOME}/conf/fe.conf
sh ${DORIS_NEW_HOME}/bin/start_fe.sh --daemon
```

6. 验证 FE 启动成功，通过 mysql 命令链接当前 FE，如上文中使用 query port 为 19030：

```
mysql -uroot -P19030 -h127.0.0.1
```

4.1.1.4 升级步骤

升级过程具体流程如下：

1. 关闭副本修复与均衡功能
2. 升级 BE 节点
3. 升级 FE 节点
4. 打开副本修复与均衡功能

升级过程中，要遵循先升级 BE、在升级 FE 的原则。在升级 FE 时，先升级 Observer FE 与 Follower FE 节点，再升级 Master FE 节点。

> 注意
>
> Doris 只需要升级 FE 目录下的 /bin 和 /lib 以及 BE 目录下的 /bin 和 /lib
>
> 在 2.0.2 及之后的版本，FE 和 BE 部署路径下新增了 custom_lib/ 目录（如没有可以手动创建）。custom_lib/ 目录用于存放一些用户自定义的第三方 jar 包，如 hadoop-lzo-*.jar，orai18n.jar 等。这个目录在升级时不需要替换。

4.1.1.4.1 第 1 步：关闭副本修复与均衡功能

在升级过程中会有节点重启，可能会触发不必要的集群均衡和副本修复逻辑，先通过以下命令关闭：

```
admin set frontend config("disable_balance" = "true");
admin set frontend config("disable_colocate_balance" = "true");
admin set frontend config("disable_tablet_scheduler" = "true");
```

4.1.1.4.2 第 2 步：升级 BE 节点

备注：

为了保证您的数据安全，请使用 3 副本来存储您的数据，以避免升级误操作或失败导致的数据丢失问题。

1. 在多副本的集群中，可以选择一台 BE 节点停止进程，进行灰度升级：

```
sh ${DORIS_OLD_HOME}/be/bin/stop_be.sh
```

2. 重命名 BE 目录下的 /bin，/lib 目录：

```
mv ${DORIS_OLD_HOME}/be/bin ${DORIS_OLD_HOME}/be/bin_back
mv ${DORIS_OLD_HOME}/be/lib ${DORIS_OLD_HOME}/be/lib_back
```

3. 复制新版本的 /bin，/lib 目录到原 BE 目录下：

```
cp -r ${DORIS_NEW_HOME}/be/bin ${DORIS_OLD_HOME}/be/bin
cp -r ${DORIS_NEW_HOME}/be/lib ${DORIS_OLD_HOME}/be/lib
```

4. 启动该 BE 节点：

```
sh ${DORIS_OLD_HOME}/be/bin/start_be.sh --daemon
```

5. 连接集群，查看该节点信息：

```
show backends`\G`
```

若该 BE 节点 alive 状态为 true，且 Version 值为新版本，则该节点升级成功。

4.1.1.4.3 第 3 步：升级 FE 节点

1. 多个 FE 节点情况下，选择一个非 Master 节点进行升级，先停止运行：

```
sh ${DORIS_OLD_HOME}/fe/bin/stop_fe.sh
```

2. 重命名 FE 目录下的 /bin，/lib，/mysql_ssl_default_certificate 目录：

```
mv ${DORIS_OLD_HOME}/fe/bin ${DORIS_OLD_HOME}/fe/bin_back
mv ${DORIS_OLD_HOME}/fe/lib ${DORIS_OLD_HOME}/fe/lib_back
mv ${DORIS_OLD_HOME}/fe/mysql_ssl_default_certificate ${DORIS_OLD_HOME}/fe/mysql_ssl_default_
    ↪ certificate_back
```

3. 复制新版本的 /bin，/lib，/mysql_ssl_default_certificate 目录到原 FE 目录下：

```
cp -r ${DORIS_NEW_HOME}/fe/bin ${DORIS_OLD_HOME}/fe/bin
cp -r ${DORIS_NEW_HOME}/fe/lib ${DORIS_OLD_HOME}/fe/lib
cp -r ${DORIS_NEW_HOME}/fe/mysql_ssl_default_certificate ${DORIS_OLD_HOME}/fe/mysql_ssl_
    ↪ default_certificate
```

4. 启动该 FE 节点：

```
sh ${DORIS_OLD_HOME}/fe/bin/start_fe.sh --daemon
```

5. 连接集群，查看该节点信息：

```
show frontends`\G`
```

若该 FE 节点 alive 状态为 true，且 Version 值为新版本，则该节点升级成功。

6. 依次完成其他 FE 节点升级，最后完成 Master 节点的升级

4.1.1.4.4 第 4 步：打开副本修复与均衡功能

升级完成，并且所有 BE 节点状态变为 Alive 后，打开集群副本修复和均衡功能：

```
admin set frontend config("disable_balance" = "false");
admin set frontend config("disable_colocate_balance" = "false");
admin set frontend config("disable_tablet_scheduler" = "false");
```

4.1.2 弹性扩缩容

Doris 支持在线弹性扩容，通过动态添加或移除节点，用户无需中断服务即可满足业务增长需求或降低空闲资源的浪费。扩缩容 BE 节点时，不影响集群可用性，但会涉及到数据搬迁，建议在业务闲时进行扩缩容操作。

4.1.2.1 扩缩容 FE 集群

Doris 的 FE 节点分为以下三种角色，每一个 FE 节点都有全量的元数据信息：

- Master 节点：负责元数据的读写操作，当 Master 的元数据发生变化，会通过 BDB JE 协议同步到非 Master 节点中，同一集群中只能有一个 Master FE 节点；
- Follower 节点：负责元数据的读取操作，当 Master 节点发生故障时，Follower 节点会发起选主操作，选举出一个新的 Master 节点。在集群内，Master 与 Follower 节点总和建议为奇数个；
- Observer 节点：负责元数据的读取操作，不参与选主操作。用于扩展 FE 的读服务能力。

一般情况下，每台 FE 节点可以负责 10-20 台 BE 节点的负载操作，3 个 FE 节点可以满足大部分的业务需求。

4.1.2.1.1 扩容 FE 集群

> 提示：
>
> 在新添加 FE 节点时，需要注意以下事项：
>
> - 新添加的 FE http_port 要与原集群中所有 FE 节点相同；
> - 如果添加 Follower 节点，同一级群内 Master 与 Follower 节点数量总和建议为奇数个
> - 通过 `show frontends` 命令可以看到当前集群内节点的端口及角色信息

1. 启动 FE 节点：

```
fe/bin/start_fe.sh --helper <leader_fe_host>:<edit_log_port> --daemon
```

- 注册 FE 节点：
  - 将节点注册为 Follower FE：

```
    ALTER SYSTEM ADD FOLLOWER "<follower_host>:<edit_log_port>";
```

```
* 将节点注册为 Observer FE：
```

```
    ALTER SYSTEM ADD OBSERVER "<observer_host>:<edit_log_port>";
```

- 查看新添加的 FE 节点状态：

```
  show frontends;
```

4.1.2.1.2 缩容 FE 集群

在缩容 FE 节点时，也要保证最终集群内 Master 与 Follower 节点总和为奇数个，通过以下命令可以缩容节点：

```
ALTER SYSTEM DROP FOLLOWER[OBSERVER] "<fe_host>:<edit_log_port>";
```

在缩容后，需要手动删除 FE 目录下的文件。

4.1.2.2 扩缩容 BE 集群

4.1.2.2.1 扩容 BE 集群

1. 启动 BE 进程：

```
be/bin/start_be.sh
```

2. 注册 BE 节点：

```
ALTER SYSTEM ADD backend '<be_host>:<be_heartbeat_service_port>';
```

4.1.2.2.2 缩容 BE 集群

在缩容 BE 节点时，可以选择 DROP 或 DECOMMISSION 两种方案：

	DROP	DECOMMISSION
下线原理	直接下线节点，删除掉 BE 节点。	发起命令后，会尝试将该 BE 数据迁移到其他节点上，当迁移完成后，
生效周期	执行后立即生效。	待数据搬迁完成后，删除命令生效。根据集群现有数据量，可能在小
一副本表处理方案	可能会造成数据丢失。	不会造成数据丢失。
同时下线多个节点	可能会造成数据丢失。	不会造成数据丢失。
生产推荐	不建议生产环境使用。	推荐在生产环境使用。

- 通过以下命令，可以使用 DROP 方式删除 BE 节点：

```
ALTER SYSTEM DROP backend "<be_host>:<be_heartbeat_service_port>";
```

- 通过以下命令，可以使用 DECOMMISSION 方式删除 BE 节点：

```
ALTER SYSTEM DECOMMISSION backend "<be_host>:<be_heartbeat_service_port>";
```

DECOMMISSION 命令说明：

- DECOMMISSION 是一个异步操作。执行后，可以通过 SHOW backends; 看到该 BE 节点的 SystemDecommissioned 状态为 true。表示该节点正在进行下线；

- DECOMMISSION 命令可能会执行失败，如剩余 BE 存储空间不足以容纳下线 BE 上的数据，或者剩余机器数量不满足最小副本数时，该命令都无法完成，并且 BE 会一直处于 SystemDecommissioned 为 true 的状态；
- DECOMMISSION 的进度，可以通过 SHOW PROC ‘/backends’; 中的 TabletNum 查看，如果正在进行，TabletNum 将不断减少；
- 可以通过 CANCEL DECOMMISSION BACKEND “be_host:be_heartbeat_service_port”; 命令取消。取消后，该 BE 上的数据将维持当前剩余的数据量。后续 Doris 重新进行负载均衡；
- 可以调整 balance_slot_num_per_path 参数调整数据搬迁速率。

4.1.3 负载均衡

用户通过 FE 的查询端口（query_port，默认 9030）使用 MySQL 协议连接 Doris。当部署多个 FE 节点时，用户可以在多个 FE 之上部署负载均衡层来实现 Doris 查询的高可用。

本文档介绍多种适用于 Doris 的负载均衡方案，并介绍如何通过 Proxy Protocol 实现客户端 IP 透传。

4.1.3.1 负载均衡

本文使用以下三个 FE 节点作为示例进行步骤演示：

```
192.168.1.101:9030
192.168.1.102:9030
192.168.1.103:9030
```

代理服务器所在节点：

```
192.168.1.100
```

4.1.3.1.1 01 JDBC URL

使用 JDBC URL 中自带的负载均衡配置。

```
jdbc:mysql:loadbalance://192.168.1.101:9030,192.168.1.102:9030,192.168.1.103:9030/test_db
```

详细可以参考 MySQL 官网文档

4.1.3.1.2 02 Nginx

使用 Nginx TCP 反向代理实现 Doris 的负载均衡。

安装 Nginx

请参看 Nginx 官网正确安装 Nginx，这里以 Ubuntu 系统中安装 Nginx 1.18.0 版本为例展示 Nginx 的编译安装步骤。

1. 安装编译依赖

```
    sudo apt-get install build-essential
    sudo apt-get install libpcre3 libpcre3-dev
    sudo apt-get install zlib1g-dev
    sudo apt-get install openssl libssl-dev
```

2. 安装 Nginx

```
    sudo wget http://nginx.org/download/nginx-1.18.0.tar.gz
    sudo tar zxvf nginx-1.18.0.tar.gz
    cd nginx-1.18.0
    sudo ./configure --prefix=/usr/local/nginx --with-stream --with-http_ssl_module --with-http_
        ↪ gzip_static_module --with-http_stub_status_module
    sudo make && make install
```

配置反向代理

新建配置文件：

```
vim /usr/local/nginx/conf/default.conf
```

内容如下：

```
events {
worker_connections 1024;
}
stream {
  upstream mysqld {
      hash $remote_addr consistent;
      server 192.168.1.101:9030 weight=1 max_fails=2 fail_timeout=60s;
      server 192.168.1.102:9030 weight=1 max_fails=2 fail_timeout=60s;
      server 192.168.1.103:9030 weight=1 max_fails=2 fail_timeout=60s;
  }
  server {
   # Proxy port
      listen 6030;
      proxy_connect_timeout 300s;
      proxy_timeout 300s;
      proxy_pass mysqld;
  }
}
```

启动 Nginx

指定配置文件启动：

```
cd /usr/local/nginx
/usr/local/nginx/sbin/nginx -c conf.d/default.conf
```

验证

使用代理端口进行连接：

```
mysql -uroot -P6030 -h192.168.1.100

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| test               |
+--------------------+
2 rows in set (0.00 sec)
```

4.1.3.1.3 03 HAProxy

HAProxy 是一个使用 C 语言编写高性能 TCP/HTTP 负载均衡器。

安装 HAProxy

1. 下载 HAProxy

下载地址：https://src.fedoraproject.org/repo/pkgs/haproxy/

2. 解压

```
tar -zxvf haproxy-2.6.15.tar.gz -C /opt/
mv haproxy-2.6.15 haproxy
cd haproxy
```

3. 编译

```
yum install gcc gcc-c++ -y
make TARGET=linux-glibc PREFIX=/usr/local/haproxy
make install PREFIX=/usr/local/haproxy
```

配置 HAProxy

1. 配置 haproxy.conf 文件

   打开配置文件：

```
vim /etc/rsyslog.d/haproxy.conf
```

内容如下：

```
$ModLoad imudp
$UDPServerRun 514
local0.* /usr/local/haproxy/logs/haproxy.log
&~
```

2. 开启远程日志

```
vim /etc/sysconfig/rsyslog
```

添加内容：

```
SYSLOGD_OPTIONS="-c 2 -r -m 0"
```

参数说明：

- `-c 2`：使用兼容模式，默认是 `-c 5`。
- `-r`：开启远程日志。
- ·-m 0`：标记时间戳。单位是分钟，为 0 时，表示禁用该功能。

使修改生效：

`systemctl restart rsyslog`

3. 编辑负载均衡文件

```
vim /usr/local/haproxy/haproxy.cfg
```

```
global
    maxconn         2000
    ulimit-n        40075
    log             127.0.0.1 local0 info
    uid             200
    gid             200
    chroot          /var/empty
    daemon
    group           haproxy
    user            haproxy

defaults
    log global
```

```
        mode http
        retries 3
        option redispatch

        timeout connect 5000
        timeout client 5000
        timeout server 5000
        timeout check 2000

    frontend agent-front
        bind *:6030
        mode tcp
        default_backend forward-fe

    backend forward-fe
        mode tcp
        balance roundrobin
        server fe-1 192.168.1.101:9030 weight 1 check inter 3000 rise 2 fall 3
        server fe-2 192.168.1.102:9030 weight 1 check inter 3000 rise 2 fall 3
        server fe-3 192.168.1.103:9030 weight 1 check inter 3000 rise 2 fall 3
```

启动 HAProxy

1. 启动服务

   ```
   /opt/haproxy/haproxy -f /usr/local/haproxy/haproxy.cfg
   ```

2. 查看服务状态

   ```
   netstat -lnatp | grep -i haproxy
   ```

验证

```
mysql -h 192.168.1.100 -uroot -P6030 -p
```

4.1.3.1.4 04 ProxySQL

ProxySQL 是基于 MySQL 的开源数据库代理软件，用 C 语言编写。能实现连接管理、读写分离、负载均衡、故障切换等功能，具有高性能、可配置、动态管理等优势，常用于 Web 服务、大数据平台、云数据库等场景。

安装 ProxySQL

请参考 官方文档 正确安装 ProxySQL。

配置 ProxySQL

ProxySQL 包含配置文件 `/etc/proxysql.cnf` 与配置数据库文件 `/var/lib/proxysql/proxysql.db`。

需特别注意，若 `/var/lib/proxysql` 目录下存在 "`proxysql.db`" 文件，ProxySQL 服务仅在首次启动时读取并解析 `proxysql.cnf`，后续启动不再读取。

若要使 proxysql.cnf 配置在重启后生效，需先删除 /var/lib/proxysql/proxysql.db 再重启服务，这相当于初始化启动，会生成新的 proxysql.db 文件，原配置规则将被清除。

以下是配置文件 proxysql.cnf 的主要内容：

```
datadir="/var/lib/proxysql"         #数据目录
admin_variables=
{
    admin_credentials="admin:admin"  # Admin databse username and password.
    mysql_ifaces="0.0.0.0:6032"    # Admin database port, used for connecting admin database of
        ↪ ProxySQL
}
mysql_variables=
{
    threads=4
    max_connections=2048
    default_query_delay=0
    default_query_timeout=36000000
    have_compress=true
    poll_timeout=2000
    interfaces="0.0.0.0:6030"
    default_schema="information_schema"
    stacksize=1048576
    server_version="5.7.99"
    connect_timeout_server=3000
    monitor_username="monitor"
    monitor_password="monitor"
    monitor_history=600000
    monitor_connect_interval=60000
    monitor_ping_interval=10000
    monitor_read_only_interval=1500
    monitor_read_only_timeout=500
    ping_interval_server_msec=120000
    ping_timeout_server=500
    commands_stats=true
    sessions_sort=true
    connect_retries_on_failure=10
}
mysql_servers =
(
)
mysql_users:
(
)
mysql_query_rules:
(
```

```
)
scheduler=
(
)
mysql_replication_hostgroups=
(
)
```

连接 ProxySQL 管理数据库

```
mysql -uadmin -padmin -P6032 -hdoris01
```

```
ProxySQL > show databases;
+-----+---------------+-------------------------------------+
| seq | name          | file                                |
+-----+---------------+-------------------------------------+
| 0   | main          |                                     |
| 2   | disk          | /var/lib/proxysql/proxysql.db       |
| 3   | stats         |                                     |
| 4   | monitor       |                                     |
| 5   | stats_history | /var/lib/proxysql/proxysql_stats.db |
+-----+---------------+-------------------------------------+
5 rows in set (0.000 sec)
ProxySQL > use main;

ProxySQL > show tables;
+--------------------------------------------+
| tables                                     |
+--------------------------------------------+
| global_variables                           |
| mysql_collations                           |
| mysql_group_replication_hostgroups         |
| mysql_query_rules                          |
| mysql_query_rules_fast_routing             |
| mysql_replication_hostgroups               |
| mysql_servers                              |
| mysql_users                                |
| proxysql_servers                           |
| runtime_checksums_values                   |
| runtime_global_variables                   |
| runtime_mysql_group_replication_hostgroups |
| runtime_mysql_query_rules                  |
| runtime_mysql_query_rules_fast_routing     |
| runtime_mysql_replication_hostgroups       |
| runtime_mysql_servers                      |
| runtime_mysql_users                        |
```

```
| runtime_proxysql_servers                  |
| runtime_scheduler                         |
| scheduler                                 |
+-------------------------------------------+
20 rows in set (0.000 sec)
```

ProxySQL 配置后端 Doris FE

使用 INSERT 语句将需要被代理的 FE 节点和端口添加到 mysql_servers 表中。

其中：hostgroup_id 为 10 表示写组，为 20 表示读组。我们这里不需要读写分离，所以可以任意设置。

```
mysql -uadmin -padmin -P6032 -h127.0.0.1
```

```
ProxySQL > insert into mysql_servers(hostgroup_id,hostname,port) values(10,'192.168.0.101',9030);
Query OK, 1 row affected (0.000 sec)

ProxySQL > insert into mysql_servers(hostgroup_id,hostname,port) values(10,'192.168.0.102',9030);
Query OK, 1 row affected (0.000 sec)

ProxySQL > insert into mysql_servers(hostgroup_id,hostname,port) values(10,'192.168.0.103',9030);
Query OK, 1 row affected (0.000 sec)
```

查看结果：

```
ProxySQL > select hostgroup_id,hostname,port,status,weight from mysql_servers;
+--------------+---------------+------+--------+--------+
| hostgroup_id | hostname      | port | status | weight |
+--------------+---------------+------+--------+--------+
| 10           | 192.168.0.101 | 9030 | ONLINE | 1      |
| 20           | 192.168.0.102 | 9030 | ONLINE | 1      |
| 20           | 192.168.0.103 | 9030 | ONLINE | 1      |
+--------------+---------------+------+--------+--------+
3 rows in set (0.000 sec)
```

如果在插入过程中，出现报错：

```
ERROR 1045 (#2800): UNIQUE constraint failed: mysql_servers.hostgroup_id, mysql_servers.hostname,
    ↪  mysql_servers.port
```

说明可能之前就已经定义了其他配置，可以清空这张表，或者删除对应 host 的配置：

```
ProxySQL > select * from mysql_servers;
ProxySQL > delete from mysql_servers;
Query OK, 6 rows affected (0.000 sec)
```

保存信息：

```
ProxySQL > load mysql servers to runtime;
Query OK, 0 rows affected (0.006 sec)

ProxySQL > save mysql servers to disk;
Query OK, 0 rows affected (0.348 sec)
```

监控 Doris FE 节点配置

添加 Doris FE 节点之后，还需要监控这些后端节点。

首先在 Doris 中创建一个用于监控的用户名：

```
mysql -uroot -P9030 -h192.168.0.101
```

```
Doris > create user monitor@'192.168.0.100' identified by 'P@ssword1!';
Query OK, 0 rows affected (0.03 sec)

Doris > grant ADMIN_PRIV on *.* to monitor@'192.168.0.100';
Query OK, 0 rows affected (0.02 sec)
```

然后回到 ProxySQL 代理层节点上配置监控

```
mysql -uadmin -padmin -P6032 -h127.0.0.1
```

```
ProxySQL > set mysql-monitor_username='monitor';
Query OK, 1 row affected (0.000 sec)

ProxySQL > set mysql-monitor_password='P@ssword1!';
Query OK, 1 row affected (0.000 sec)
```

保存配置并退出：

```
ProxySQL > load mysql servers to runtime;
Query OK, 0 rows affected (0.006 sec)

ProxySQL > save mysql servers to disk;
Query OK, 0 rows affected (0.348 sec)
```

验证监控结果。

ProxySQL 监控模块的指标都保存在 monitor.log 表中。

连接监控：

```
ProxySQL > select * from mysql_server_connect_log;
+---------------+------+------------------+-------------------------+---------------+
| hostname      | port | time_start_us    | connect_success_time_us | connect_error |
+---------------+------+------------------+-------------------------+---------------+
| 192.168.0.101 | 9030 | 1548665195883957 | 762                     | NULL          |
```

```
| 192.168.0.102 | 9030 | 1548665195894099 | 399                     | NULL          |
| 192.168.0.103 | 9030 | 1548665195904266 | 483                     | NULL          |
| 192.168.0.101 | 9030 | 1548665255883715 | 824                     | NULL          |
| 192.168.0.102 | 9030 | 1548665255893942 | 656                     | NULL          |
| 192.168.0.101 | 9030 | 1548665495884125 | 615                     | NULL          |
| 192.168.0.102 | 9030 | 1548665495894254 | 441                     | NULL          |
| 192.168.0.103 | 9030 | 1548665495904479 | 638                     | NULL          |
| 192.168.0.101 | 9030 | 1548665512917846 | 487                     | NULL          |
| 192.168.0.102 | 9030 | 1548665512928071 | 994                     | NULL          |
| 192.168.0.103 | 9030 | 1548665512938268 | 613                     | NULL          |
+---------------+------+------------------+-------------------------+---------------+
20 rows in set (0.000 sec)
```

心跳监控：

```
ProxySQL > select * from mysql_server_ping_log;
+---------------+------+------------------+----------------------+------------+
| hostname      | port | time_start_us    | ping_success_time_us | ping_error |
+---------------+------+------------------+----------------------+------------+
| 192.168.0.101 | 9030 | 1548665195883407 | 98                   | NULL       |
| 192.168.0.102 | 9030 | 1548665195885128 | 119                  | NULL       |
...........
| 192.168.0.102 | 9030 | 1548665415889362 | 106                  | NULL       |
| 192.168.0.103 | 9030 | 1548665562898295 | 97                   | NULL       |
+---------------+------+------------------+----------------------+------------+
110 rows in set (0.001 sec)
```

4.1.3.2 客户端 IP 透传

多数情况下，通过代理服务连接到后端 Doris 服务后，客户端 IP 信息会丢失，Doris 服务端只能获取到代理服务器的 IP 地址信息。

自 2.1.1 版本开始，Doris 支持 Proxy Protocol 协议。利用这个协议，可以是实现客户端 IP 透传，从而在经过负载均衡后，Doris 依然可以获取客户端的真实 IP，实现白名单等权限控制。

下面分别介绍如何在 Nginx 和 Haproxy 中开启 Proxy Protocol。

4.1.3.2.1 Doris 开启 Proxy Protocol 支持

在 FE 的 `fe.conf` 中添加：

```
enable_proxy_protocol = true
```

1. 仅支持 Proxy Protocol V1。

2. 仅支持并作用于 MySQL 协议端口，不支持和影响 HTTP、 ADBC 等其他协议端口。
3. 在 Doris 3.1 版本之前，开启后，必须使用 Proxy Protocol 协议进行连接，否则连接失败。3.1 版本开始，开启 Proxy Protocol 后，依然可以使用标准的 MySQL 连接协议进行连接。

4.1.3.2.2 01 Nginx

在配置文件的 `server` 部分新增：`proxy_protocol on;`：

```
events {
worker_connections 1024;
}
stream {
  upstream mysqld {
      hash $remote_addr consistent;
      server 192.168.1.101:9030 weight=1 max_fails=2 fail_timeout=60s;
      server 192.168.1.102:9030 weight=1 max_fails=2 fail_timeout=60s;
      server 192.168.1.103:9030 weight=1 max_fails=2 fail_timeout=60s;
  }
  server {
   # Proxy port
      listen 6030;
      proxy_connect_timeout 300s;
      proxy_timeout 300s;
      proxy_pass mysqld;
      # Enable Proxy Protocol to the upstream server
      proxy_protocol on;
  }
}
```

4.1.3.2.3 02 HAProxy

在 `haproxy.cfg` 的 `backend` 部分新增 `send-proxy` 参数：

```
backend forward-fe
    mode tcp
    balance roundrobin
    server fe-1 192.168.1.101:9030 weight 1 check inter 3000 rise 2 fall 3 send-proxy
    server fe-2 192.168.1.102:9030 weight 1 check inter 3000 rise 2 fall 3 send-proxy
    server fe-3 192.168.1.103:9030 weight 1 check inter 3000 rise 2 fall 3 send-proxy
```

4.1.3.2.4 验证 IP 透传是否成功

通过代理连接 Doris：

```
mysql -uroot -P6030 -h192.168.1.100
```

验证

```
mysql> show processlist;
+--
    ↪ ----------------+------+------+-------------------+--------------------+----------+------+---------+----
    ↪
| CurrentConnected | Id   | User | Host              | LoginTime          | Catalog  | Db   |
    ↪ Command | Time | State | QueryId                              | Info             |
+--
    ↪ ----------------+------+------+-------------------+--------------------+----------+------+---------+----
    ↪
| Yes              |    1 | root | 192.168.1.101:34390 | 2024-03-17 16:32:22 | internal |      |
    ↪ Query   |    0 | OK    | 82edc460d93f4e28-8bbed058a068e259 | show processlist |
+--
    ↪ ----------------+------+------+-------------------+--------------------+----------+------+---------+----
    ↪
1 row in set (0.00 sec)
```

如果在 Host 列看到的真实的客户端 IP，则说明验证成功。否则，只能看到代理服务的 IP 地址。

同时，在 fe.audit.log 中也会记录真实的客户端 IP。

4.1.4 时区管理

Doris 支持自定义时区设置

4.1.4.1 基本概念

Doris 内部存在以下两个时区相关参数：

- system_time_zone : 当服务器启动时，系统会根据机器设置时区自动设置，设置后不可修改。
- time_zone : 集群当前时区，可以修改。集群启动时，该变量会设置为与 system_time_zone 相同，之后不再变动，除非用户手动修改。

4.1.4.2 具体操作

1. show variables like '%time_zone%'

查看当前时区相关配置

2. SET [global] time_zone = 'Asia/Shanghai'

该命令可以设置 Session 级别的时区，如使用 global 关键字，则 Doris FE 会将参数持久化，之后对所有新 Session 生效。

4.1.4.3 数据来源

时区数据包含时区名、对应时间偏移量、夏令时变化情况等。在 BE 所在机器上，其数据来源为 TZDIR 命令返回的目录，如不支持该命令，则为 /usr/share/zoneinfo 目录。

4.1.4.4 时区的影响

4.1.4.4.1 1. 函数

包括 NOW() 或 CURTIME() 等时间函数显示的值，也包括 show load, show backends 中的时间值。

但不会影响 create table 中时间类型分区列的 less than 值，也不会影响存储为 date/datetime 类型的值的显示。

受时区影响的函数：

- FROM_UNIXTIME：给定一个 UTC 时间戳，返回其在 Doris session time_zone 指定时区的日期时间，如time_zone为CST时FROM_UNIXTIME(0)返回1970-01-01 08:00:00。
- UNIX_TIMESTAMP：给定一个日期时间，返回其在 Doris session time_zone 指定时区下的 UTC 时间戳，如time_zone为CST时UNIX_TIMESTAMP('1970-01-01 08:00:00')返回0。
- CURTIME：返回当前 Doris session time_zone 指定时区的时间。
- NOW：返回当前 Doris session time_zone 指定时区的日期时间。
- CONVERT_TZ：将一个日期时间从一个指定时区转换到另一个指定时区。

4.1.4.4.2 2. 时间类型的值

对于DATE、DATETIME类型，我们支持导入数据时对时区进行转换。

- 如果数据带有时区，如“2020-12-12 12:12:12+08:00”，而 Stream Load 指定的 Header timezone 为 +00:00，则数据导入 Doris 得到实际值为 “2020-12-12 04:12:12”。
- 如果数据不带有时区，如 “2020-12-12 12:12:12”，则认为该时间为绝对时间，不发生任何转换。

4.1.4.4.3 3. 夏令时

夏令时的本质是具名时区的实际时间偏移量，在一定日期内发生改变。

例如，America/Los_Angeles时区包含一次夏令时调整，起止时间为约为每年 3 月至 11 月。即，三月份夏令时开始时，America/Los_Angeles实际时区偏移由-08:00变为-07:00，11 月夏令时结束时，又从-07:00变为-08:00。如果不希望开启夏令时，则应设定 time_zone 为 -08:00 而非 America/Los_Angeles。

4.1.4.5 使用方式

时区值可以使用多种格式给出，以下是 Doris 中完善支持的标准格式：

1. 标准具名时区格式，如“Asia/Shanghai”，“America/Los_Angeles”。此类格式来源于本机所带时区数据，如“Etc/GMT+3”等亦属此列。
2. 标准偏移格式，如“+02:30”，“-10:00”（不支持诸如“+12:03”等特殊偏移）
3. 缩写时区格式，当前仅支持：
4. “GMT”，“UTC”，等同于“+00:00”时区
5. “CST”，等同于“Asia/Shanghai”时区
6. 单字母 Z，代表 Zulu 时区，等同于“+00:00”时区

此外，对任何字母的解析不区分大小写。

注意：由于实现方式的不同，当前 Doris 存在部分其他格式在部分导入方式中得到了支持。生产环境不应当依赖这些未列于此的格式，它们的行为随时可能发生变化，请关注版本更新时的相关 changelog。

4.1.4.6 最佳实践

4.1.4.6.1 时区敏感数据

时区问题主要涉及三个影响因素：

1. session variable `time_zone` —— 集群时区
2. Stream Load、 Broker Load 等导入时指定的 header `timezone` —— 导入时区
3. 时区类型字面量“2023-12-12 08:00:00+08:00”中的“+08:00” —— 数据时区

我们可以做如下理解：

Doris 目前兼容各时区下的数据向 Doris 中进行导入。而由于 Doris 自身 `DATETIME` 等各个时间类型本身不内含时区信息，且数据在导入后不会随时区变化而变更，因此时间数据导入 Doris 时，可分为如下两类：

1. 绝对时间

   绝对时间是指，它所关联的数据场景与时区无关。对于这类数据，在导入时应该不带有任何时区后缀，它们将被原样存储。

2. 特定时区下的时间

   某个特定时区下的时间是指，它所关联的数据场景与时区有关。对于这类数据，在导入时应该带有具体时区后缀，导入时它们将被转化至 Doris 集群 `time_zone` 时区或 Stream Load/Broker Load 中指定的 header `timezone`。

   这类数据在导入后即被转化至导入时指定时区下的绝对时间存储，故后续导入和查询应当保持此时区，以免数据意义发生紊乱。

- 对于 Insert 语句，我们可以通过以下例子来说明：

```
Doris > select @@time_zone;
+---------------+
| @@time_zone   |
+---------------+
| Asia/Shanghai |
+---------------+

Doris > insert into dt values('2020-12-12 12:12:12+02:00'); --- 导入的数据中指定了时区为
    ↪ +02:00

Doris > select * from dt;
+---------------------+
| dt                  |
+---------------------+
| 2020-12-12 18:12:12 | --- 被转换为 Doris 集群时区 Asia/Shanghai，
    ↪ 后续导入和查询应当保持此时区。
+---------------------+

Doris > set time_zone = 'America/Los_Angeles';

Doris > select * from dt;
+---------------------+
| dt                  |
+---------------------+
| 2020-12-12 18:12:12 | --- 如果修改 time_zone，时间值不会随之改变，其查询时的意义发生紊乱。
+---------------------+
```

- 对于 Stream Load、Broker Load 等导入方式，我们可以通过指定 header `timezone` 来实现。例如，对于 Stream Load，我们可以通过以下例子来说明：

```
cat dt.csv
2020-12-12 12:12:12+02:00

curl --location-trusted -u root: \
 -H "Expect:100-continue" \
 -H "strict_mode: true" \
 -H "timezone: Asia/Shanghai" \
 -T dt.csv -XPUT \
 http://127.0.0.1:8030/api/test/dt/_stream_load
```

```
Doris > select @@time_zone;
+---------------+
```

```
| @@time_zone   |
+---------------+
| Asia/Shanghai |
+---------------+

Doris > select * from dt;
+---------------------+
| dt                  |
+---------------------+
| 2020-12-12 18:12:12 | --- 被转换为 Doris 集群时区 Asia/Shanghai,
    ↪ 后续导入和查询应当保持此时区。
+---------------------+
```

```
* Stream Load、Broker Load 等导入方式中，header `timezone` 会覆盖 Doris 集群 `time_
    ↪ zone`，因此在导入时应当保持一致。
* Stream Load、Broker Load 等导入方式中，header `timezone`
    ↪ 会影响导入转换中使用的函数。
* 如果导入时未指定 header `timezone`，则默认使用东八区。
```

综上所述，处理时区问题最佳的实践是：> 最佳实践 > 1. 在使用前确认该集群所表征的时区并设置 time_zone，在此之后不再更改。> > 2. 在导入时设定 header timezone 同集群 time_zone 一致。> > 3. 对于绝对时间，导入时不带时区后缀；对于有时区的时间，导入时带具体时区后缀，导入后将被转化至 Doris time_zone 时区。

4.1.4.6.2 夏令时

夏令时的起讫时间来自当前时区数据源，不一定与当年度时区所在地官方实际确认时间完全一致。该数据由 ICANN 进行维护。如果需要确保夏令时表现与当年度实际规定一致，请保证 Doris 所选择的数据源为最新的 ICANN 所公布时区数据，下载途径见下文。

4.1.4.6.3 信息更新

真实世界中的时区与夏令时相关数据，将会因各种原因而不定期发生变化。IANA 会定期记录这些变化并更新相应时区文件。如果希望 Doris 中的时区信息与最新的 IANA 数据保持一致，请采取下列方式进行更新：

1. 使用包管理器更新

根据当前操作系统使用的包管理器，用户可以使用对应的命令直接更新时区数据：

```
> sudo yum update tzdata
### apt
> sudo apt update tzdata
```

该方式更新的数据位于系统 $TZDIR 下（一般为 usr/share/zoneinfo）。

2. 直接拉取 IANA 时区数据库（推荐）

大多数 Linux 发行版的包管理器，tzdata 的同步并不及时。如果对时区数据准确性要求较高，可以直接拉取 IANA 定期公布的数据：

```
wget https://www.iana.org/time-zones/repository/tzdb-latest.tar.lz
```

然后根据解压后文件夹中的 README 文件，生成具体的 zoneinfo 数据。生成的数据应当拷贝并覆盖 $TZDIR 目录。

请注意，以上所有操作在 BE 所在机器上完成后，都必须重启对应 BE 才能生效。

4.1.4.7 拓展阅读

- 时区格式列表：List of tz database time zones
- IANA 时区数据库：IANA Time Zone Database
- ICANN 时区数据库：The tz-announce Archives

4.1.5 FQDN

本文介绍如何启用基于 FQDN（Fully Qualified Domain Name，完全限定域名）使用 Apache Doris。FQDN 是 Internet 上特定计算机或主机的完整域名。

Doris 支持 FQDN 之后，各节点之间通信完全基于 FQDN。添加各类节点时应直接指定 FQDN，例如添加 BE 节点的命令为ALTER SYSTEM ADD BACKEND "be_host:heartbeat_service_port"，

be_host 此前是 BE 节点的 IP，启动 FQDN 后，be_host 应指定 BE 节点的 FQDN。

4.1.5.1 前置条件

1. fe.conf 文件设置 enable_fqdn_mode = true。
2. 集群中的所有机器都必须配置有主机名。
3. 必须在集群中每台机器的 /etc/hosts 文件中指定集群中其他机器对应的 IP 地址和 FQDN。
4. /etc/hosts 文件中不能有重复的 IP 地址。

4.1.5.2 最佳实践

4.1.5.2.1 新集群启用 FQDN

1. 准备机器，例如想部署 3FE 3BE 的集群，可以准备 6 台机器。
2. 每台机器执行host返回结果都唯一，假设六台机器的执行结果分别为 fe1,fe2,fe3,be1,be2,be3。
3. 在 6 台机器的/etc/hosts 中配置 6 个 FQDN 对应的真实 IP，例如：

```
172.22.0.1 fe1
172.22.0.2 fe2
172.22.0.3 fe3
172.22.0.4 be1
172.22.0.5 be2
172.22.0.6 be3
```

4. 验证：可以在 FE1 上 ping fe2 等，能解析出正确的 IP 并且能 Ping 通，代表网络环境可用。
5. 每个 FE 节点的 fe.conf 设置 enable_fqdn_mode = true。
6. 参考手动部署
7. 按需在六台机器上选择几台机器部署 broker，执行ALTER SYSTEM ADD BROKER broker_name "fe1:8000","be1:8000",...;。

4.1.5.2.2 K8s 部署 Doris

Pod 意外重启后，K8s 不能保证 Pod 的 IP 不发生变化，但是能保证域名不变，基于这一特性，Doris 开启 FQDN 时，能保证 Pod 意外重启后，还能正常提供服务。

K8s 部署 Doris 的方法请参考K8s 部署 Doris

4.1.5.2.3 服务器变更 IP

按照’ 新集群启用 FQDN’ 部署好集群后，如果想变更机器的 IP，无论是切换网卡，或者是更换机器，只需要更改各机器的/etc/hosts即可。

4.1.5.2.4 旧集群启用 FQDN

前提条件：当前程序支持ALTER SYSTEM MODIFY FRONTEND "<fe_ip>:<edit_log_port>" HOSTNAME "<fe_hostname>"语法，如果不支持，需要升级到支持该语法的版本

> 注意：
>
> 至少有三台 follower 才能进行如下操作，否则会造成集群无法正常启动

接下来按照如下步骤操作：

1. 逐一对 Follower、 Observer 节点进行以下操作 (最后操作 Master 节点)：

1. 停止节点。
2. 检查节点是否停止。通过 MySQL 客户端执行show frontends，查看该 FE 节点的 Alive 状态直至变为 false
3. 为节点设置 FQDN: ALTER SYSTEM MODIFY FRONTEND "<fe_ip>:<edit_log_port>" HOSTNAME "<fe_hostname>"（停掉 master 后，会选举出新的 master 节点，用新的 master 节点来执行 sql 语句）
4. 修改节点配置。修改 FE 根目录中的conf/fe.conf文件，添加配置：enable_fqdn_mode = true。如果在刚停止的节点对应 fe.conf 添加了配置后无法正常启动，请在所有 fe.conf 中添加配置enable_fqdn_mode = true后再启动刚刚停止的 fe 节点
5. 启动节点。

2. BE 节点启用 FQDN 只需要通过 MySQL 执行以下命令，不需要对 BE 执行重启操作。

ALTER SYSTEM MODIFY BACKEND "<backend_ip>:<HeartbeatPort>" HOSTNAME "<be_hostname>"，如果你不知道端口 HeartbeatPort 是多少，请使用 show backends 命令来帮助寻找此端口;

4.1.5.3 常见问题

- 配置项 enable_fqdn_mode 可以随意更改么？

不能随意更改，更改该配置要按照’旧集群启用 FQDN’进行操作。

4.2 负载管理

4.2.1 负载管理概述

负载管理是 Doris 一项非常重要的功能，在整个系统运行中起着非常重要的作用。通过合理的负载管理策略，可以优化资源使用，提高系统的稳定性，降低响应时间。它具备以下功能：

- 资源隔离：通过划分多个 Group，并且为每个 Group 都设置一定的资源（CPU, Memory, IO）限制，确保多个用户之间、同一用户不同的任务（例如读写操作）之间互不干扰；
- 并发控制与排队：可以限制整个集群同时执行的任务数量，当超过设置的阈值时自动排队；
- 熔断：在查询的规划阶段或者执行过程中，可以根据预估的或者执行中需要读取的分区数量，扫描的数据量，分配的内存大小，执行时间等条件，自动取消任务，避免不合理的任务占用太多的系统资源。

4.2.1.1 资源划分方式

Doris 可以通过以下 3 种方式将资源分组：

- Resource Group: 以 BE 节点为最小粒度，通过设置标签（tag）的方式，划分出多个资源组；
- Workload Group: 将一个 BE 内的资源（CPU、Memory、IO）通过 Cgroup 划分出多个资源组，实现更细致的资源分配；

- Compute Group: 是存算分离模式下的一种资源组划分的方式，与 Resource Group 类似，它也是以 BE 节点为最小粒度，划分出多个资源组。

下表中记录了不同资源划分方式的特点及优势场景：

资源隔离方式	隔离粒度	软/硬限制	跨资源组查询
Resource Group	服务器节点级别，资源完全隔离；可以隔离 BE 故障	硬限制	不支持跨资源组查询，必须保证
Workload Group	BE 进程内隔离；不能隔离 BE 故障	支持硬限制与软限制	支持跨资源组查询
Compute Group	服务器节点级别，资源完全隔离；可以隔离 BE 故障	硬限制	不支持跨资源组查询

4.2.1.2 软限与硬限

- 硬限：硬限是指资源能够使用的绝对上限，租户无法超越该限制。一旦达到硬限，超出部分的资源请求将会被拒绝。硬限一般用于防止集群内资源被耗尽或不同业务之间的资源抢占，确保集群的稳定与性能；
- 软限：软限是一个可以被超越的资源限制，通常表示资源推荐使用的上限。在系统不繁忙时，租户申请的资源超过了软限，可以借用其他资源组的资源。在系统繁忙存在资源争用时，租户申请资源超过了软限，将无法继续获得资源。

使用 Resource Group / Compute Group 的方式划分资源，只支持硬限的模式。使用 Workload Group 的方式划分资源，既支持 Workload Group 软限，也支持硬限；Workload Group 软限通常被用于突发性的资源管控，如临时的查询高峰或短暂的数据写入增加。

4.2.2 资源隔离

4.2.2.1 Resource Group

Resource Group 是存算一体架构下，实现不同的负载之间物理隔离的一种机制，它的基本原理如下图所示：

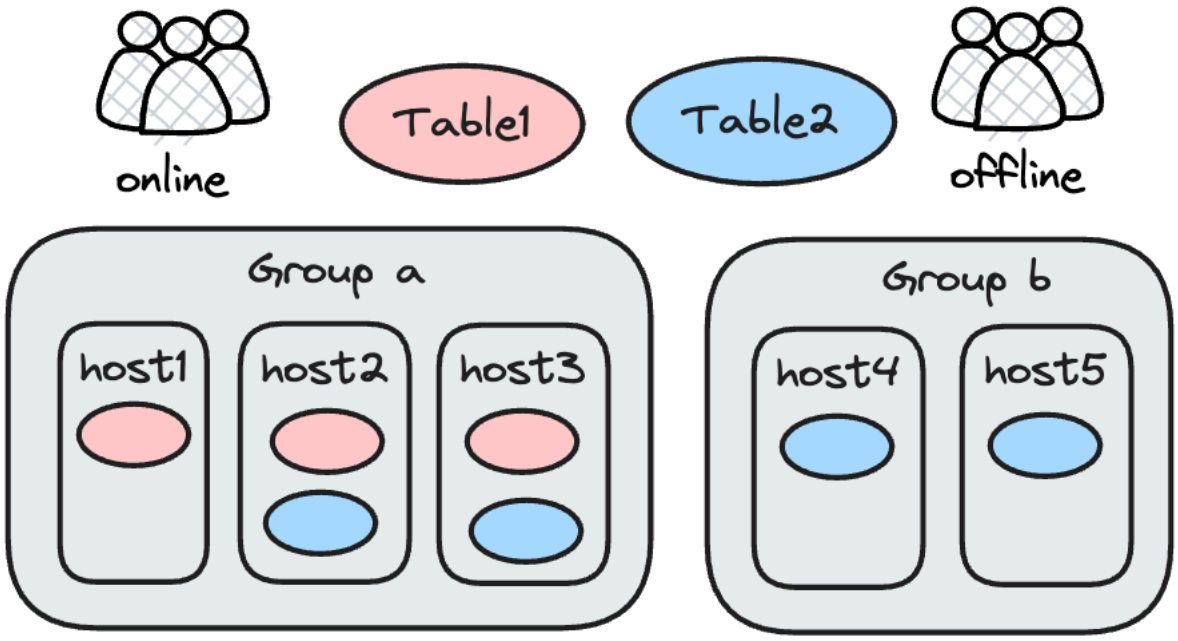


图 129: Resource Group

- 通过 Tag 的方式，把 BE 划分为不同的组，每个组通过 tag 的名字来标识，比如上图中把 host1,host2,host3 都设置为 group a, 把 host4,host5 都设置为 group b；
- 将表的不同的副本放到不同的分组中，比如上图中 table1 有 3 个副本，都位于 group a 中，table2 有 4 个副本，其中 2 个位于 group a 中，2 个副本位于 group b 中；
- 在查询时，根据不同的用户，使用不同的 Resource Group，比如 online 用户，只能访问 host1,host2,host3 上的数据，所以他可以访问 table1 和 table2；但是 offline 用户只能访问 host4，host5，所以只能访问 table2 的数据，由于 table1 在 group b 上没有对应的副本，所以访问会出错。

Resource Group 本质上是一种 Table 副本的放置策略，所以它有以下优势和限制：- 不同的 Resource Group 使用的是不同的 BE，所以它们之间完全无干扰，即使一个 group 内的某个 BE 宕机了，也不会影响其他 Group 的查询；由于导入需要多副本成功，所以如果剩下的副本数量不满足 Quorum，那么导入还是会失败；- 每个 Resource Group 至少要有一个 Table 的一个副本，比如如果要建立 5 个 Resource Group，并且每个 Resource Group 都可能访问所有的 Table，那么就需要 Table 有 5 个副本，会带来比较大的存储开销。

4.2.2.1.1 典型使用场景

- 读写隔离，可以将一个集群划分为两个 Resource Group，Offline Resource Group 用来执行 ETL 作业，Online Resource Group 负责在线查询；数据以 3 副本的方式存储，其中 2 个副本存放在 Online 资源组，1 个副本存放在 Offline 资源组。Online 资源组主要用于高并发低延迟的在线数据服务，而一些大查询或离线 ETL 操作，则可以使用 Offline 资源组中的节点执行。从而实现在统一集群内同时提供在线和离线服务的能力。
- 不同业务之间隔离，此时多个业务之间数据没有共享，可以为每个业务划分一个 Resource Group，多个业务之间没有任何干扰，这实际上是把多个物理集群合并到统一的一个大集群管理；
- 不同用户之间隔离，比如集群内有一张业务表需要共享给所有 3 个用户使用，但是希望能够尽量避免不同用户之间的资源抢占。则我们可以为这张表创建 3 个副本，分别存储在 3 个资源组中，为每个用户绑定一个资源组。

4.2.2.1.2 配置 Resource Group

为 BE 设置标签

假设当前 Doris 集群有 6 个 BE 节点。分别为 host[1-6]。在初始情况下，所有 BE 节点都属于一个默认资源组（Default）。

我们可以使用以下命令将这 6 个节点划分成 3 个资源组：group_a、group_b、group_c：

```
alter system modify backend "host1:9050" set ("tag.location" = "group_a");
alter system modify backend "host2:9050" set ("tag.location" = "group_a");
alter system modify backend "host3:9050" set ("tag.location" = "group_b");
alter system modify backend "host4:9050" set ("tag.location" = "group_b");
alter system modify backend "host5:9050" set ("tag.location" = "group_c");
alter system modify backend "host6:9050" set ("tag.location" = "group_c");
```

这里我们将 host[1-2] 组成资源组 group_a，host[3-4] 组成资源组 group_b，host[5-6] 组成资源组 group_c。

> 注：一个 BE 只能属于一个资源组。

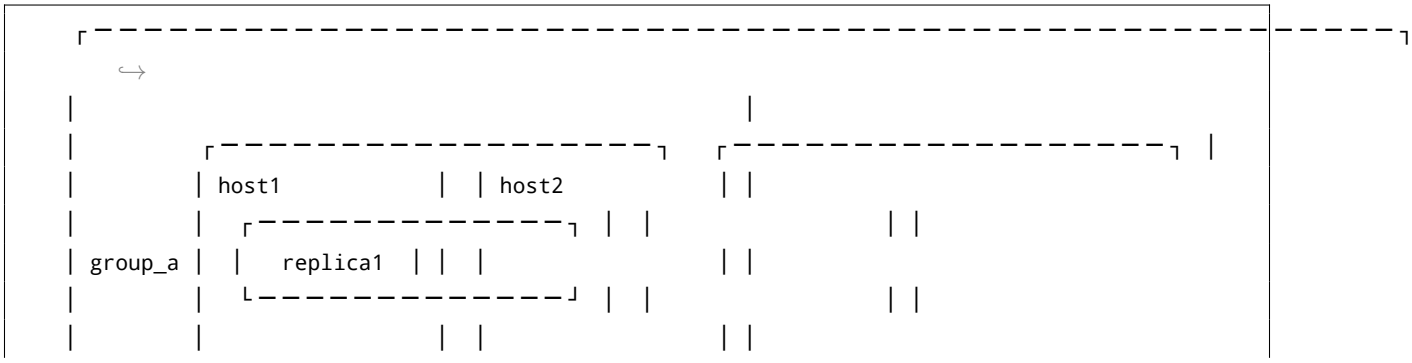
按照资源组分配数据分布

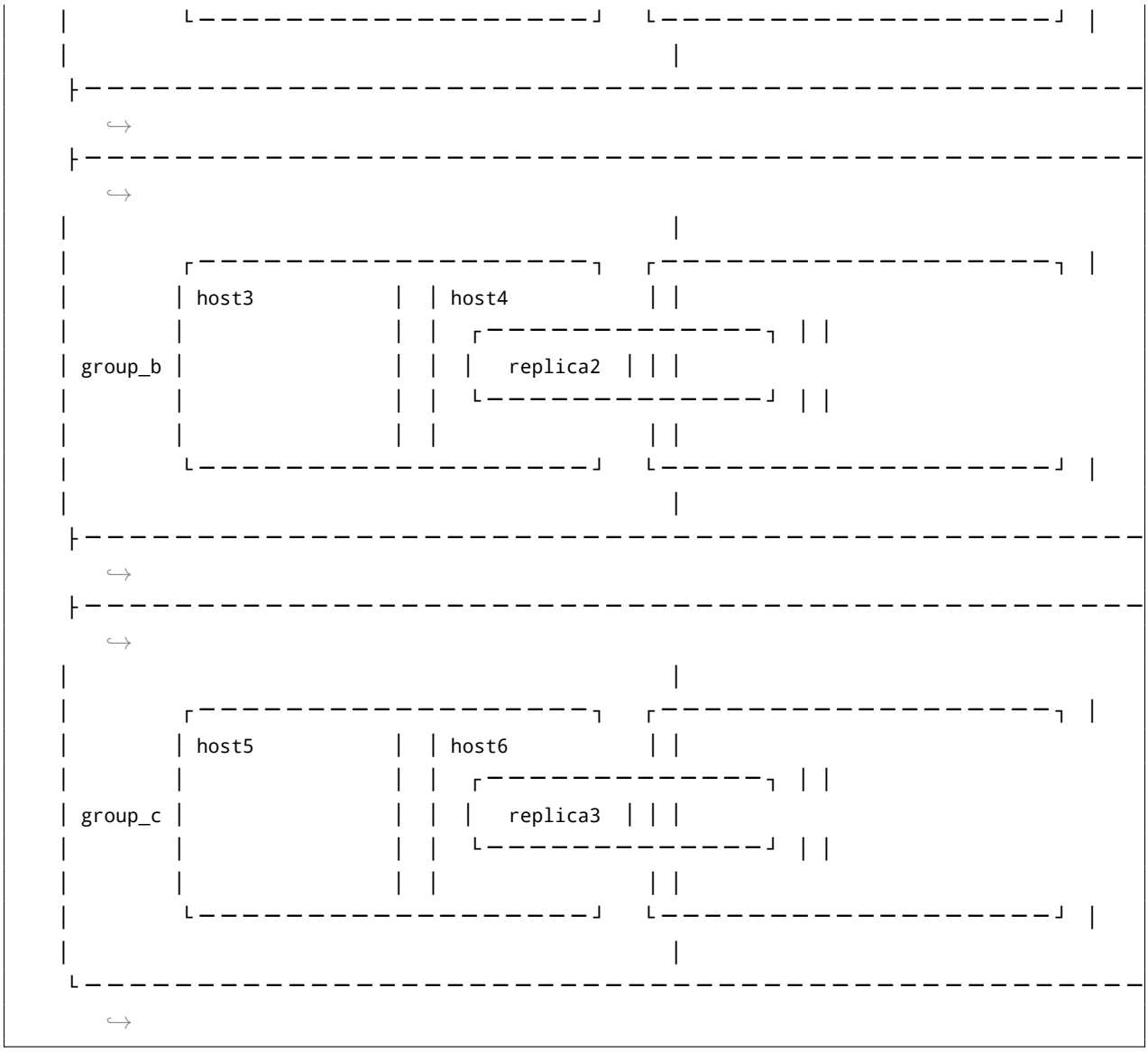
资源组划分好后可以将用户数据的不同副本分布在不同资源组。假设一张用户表 UserTable。我们希望在 3 个资源组内各存放一个副本，则可以通过如下建表语句实现：

```
create table UserTable
(k1 int, k2 int)
distributed by hash(k1) buckets 1
properties(
    "replication_allocation"="tag.location.group_a:1, tag.location.group_b:1, tag.location.
        ↪ group_c:1"
)
```

这样一来，表 UserTable 中的数据，将会以 3 副本的形式，分别存储在资源组 group_a、group_b、group_c 所在的节点中。

下图展示了当前的节点划分和数据分布：





当一个 DB 下有非常多的 Table 时，修改每个 Table 的分布策略是非常繁琐的，所以 Doris 还支持了在 database 层面设置统一的数据分布策略，但是 table 设置的优先级高于 database。比如有一个 db1, db1 下有四个 table，table1 需要的副本分布策略为 `group_a:1,group_b:2`，table2，table3, table4 需要的副本分布策略为 `group_c:1,group_b:2`

那么可以使用如下语句创建 db1：

```
CREATE DATABASE db1 PROPERTIES (
"replication_allocation" = "tag.location.group_c:1, tag.location.group_b:2"
)
```

使用如下语句创建 table1：

```
CREATE TABLE table1
(k1 int, k2 int)
distributed by hash(k1) buckets 1
properties(
```

```
"replication_allocation"="tag.location.group_a:1, tag.location.group_b:2"
)
```

table2，table3,table4 的建表语句无需再指定replication_allocation。

> 注意更改 database 的副本分布策略不会对已有的 table 产生影响。

4.2.2.1.3 为用户设置 ResourceGroup

可以通过以下语句，限制 user1 只能使用 group_a 资源组中的节点进行数据查询，user2 只能使用 group_b 资源组，而 user3 可以同时使用 3 个资源组：

```
set property for 'user1' 'resource_tags.location' = 'group_a';
set property for 'user2' 'resource_tags.location' = 'group_b';
set property for 'user3' 'resource_tags.location' = 'group_a, group_b, group_c';
```

设置完成后，user1 在发起对 UserTable 表的查询时，只会访问 group_a 资源组内节点上的数据副本，并且查询仅会使用 group_a 资源组内的节点计算资源。而 user3 的查询可以使用任意资源组内的副本和计算资源。

> 注：默认情况下，用户的 resource_tags.location 属性为空，在 2.0.2（含）之前的版本中，默认情况下，用户不受 tag 的限制，可以使用任意资源组。在 2.0.3 版本之后，默认情况下，普通用户只能使用 default 资源组。root 和 admin 用户可以使用任意资源组。

> 注意属性 resource_tags.location 每次修改完成之后，用户需要重新建立连接才能使变更生效。

4.2.2.1.4 导入作业的资源组分配

导入作业（包括 insert、 broker load、 routine load、 stream load 等）的资源使用可以分为两部分：

1. 计算资源：负责读取数据源、数据转换和分发；
2. 写入资源：负责数据编码、压缩并写入磁盘。

由于写入资源必须是数据副本所在的节点，而计算资源可以选择任意节点完成，所以在导入的场景下，Resource Group 只能限制计算部分使用的资源。

4.2.2.2 Compute Group

Compute Group 是存算分离架构下，实现不同的负载之间物理隔离的一种机制，它的基本原理如下图所示：

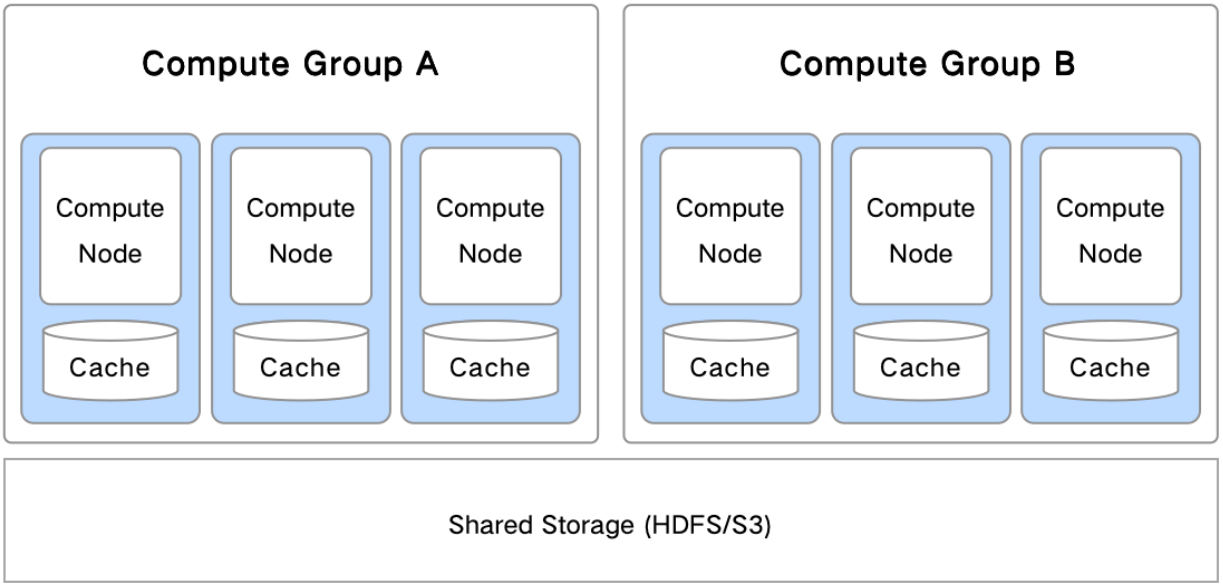


图 130: compute_group

- 一个或多个 BE 可以组成一个 Compute Group；
- BE 节点本地无状态，数据都是存储在共享存储上；
- 多个 Compute Group 之间通过共享的存储的方式来访问数据。

在保持了 Resource Group 强隔离的优点的同时，Compute Group 与 Resource Group 相比，还有以下优势：

- 成本更低，由于采用了存算分离的架构，数据位于共享存储中，所以 Compute Group 的数量不再受限于副本的数量，用户可以根据需求创建任意多的 Compute Group，存储成本不会变多；
- 更灵活，在存算分离架构下，BE 本地的数据都是缓存，所以增加 Compute Group 时不需要做笨重的数据迁移过程，新的 Compute Group 只需在查询时缓存预热即可；
- 隔离更彻底，数据的多副本存储由共享的存储层解决，所以任何 Compute Group 内的 BE 宕机不会像 Resource Group 那样导致导入失败。

> 注意 3.0.2 之前的版本中叫做计算集群（Compute Cluster）。

4.2.2.2.1 查看所有 Compute Group

可通过 SHOW COMPUTE GROUPS 查看当前仓库拥有的所有 Compute Group。

```
SHOW COMPUTE GROUPS;
```

4.2.2.2.2 添加 Compute Group

使用ADD BE命令添加 BE 并为 BE 指定 Compute Group，示例：

```
ALTER SYSTEM ADD BACKEND 'host:9050' PROPERTIES ("tag.compute_group_name" = "new_group");
```

上面命令会将host:9050这台节点添加到new_group这个 Compute Group 中，您也可以不指定 Compute Group，默认会添加到default_compute_group组里，例如：

```
ALTER SYSTEM ADD BACKEND 'host:9050';
```

4.2.2.2.3 授予 Compute Group 访问权限

```
GRANT USAGE_PRIV ON COMPUTE GROUP {compute_group_name} TO {user};
```

4.2.2.2.4 撤销 Compute Group 访问权限

```
REVOKE USAGE_PRIV ON COMPUTE GROUP {compute_group_name} FROM {user};
```

4.2.2.2.5 设置默认 Compute Group

为当前用户设置默认 Compute Group：

```
SET PROPERTY 'default_compute_group' = '{clusterName}';
```

为其他用户设置默认 Compute Group（此操作需要 Admin 权限）：

```
SET PROPERTY FOR {user} 'default_compute_group' = '{clusterName}';
```

查看当前用户默认 Compute Group，返回结果中default_compute_group 的值即为默认 Compute Group：

```
SHOW PROPERTY;
```

查看其他用户默认 Compute Group，此操作需要当前用户具备相关权限，返回结果中default_compute_group 的值即为默认 Compute Group：

```
SHOW PROPERTY FOR {user};
```

查看当前仓库下所有可用的 Compute Group：

```
SHOW COMPUTE GROUPS;
```

备注

- 若当前用户拥有 Admin 角色，例如：CREATE USER jack IDENTIFIED BY '123456' DEFAULT ↪ ROLE "admin"，则：

  - 可以为自身以及其他用户设置默认 Compute Group；
  - 可以查看自身以及其他用户的 PROPERTY。
- 若当前用户无 Admin 角色，例如：`CREATE USER jack1 IDENTIFIED BY '123456'`，则：
  - 可以为自身设置默认 Compute Group；
  - 可以查看自身的 PROPERTY；
  - 无法查看所有 Compute Group，因该操作需要 `GRANT ADMIN` 权限。
- 若当前用户未配置默认 Compute Group，现有系统在执行数据读写操作时将会触发错误。为解决这一问题，用户可通过执行 `use @cluster` 命令来指定当前 Context 所使用的 Compute Group，或者使用 `SET PROPERTY` 语句来设置默认 Compute Group。
- 若当前用户已配置默认 Compute Group，但随后该集群被删除，则在执行数据读写操作时同样会触发错误。用户可通过执行 `use @cluster` 命令来重新指定当前 Context 所使用的 Compute Group，或者利用 `SET PROPERTY` 语句来更新默认集群设置。

4.2.2.2.6 默认 Compute Group 的选择机制

当用户未明确设置默认 Compute Group 时，系统将自动为用户选择一个具有 Active BE 且用户具有使用权限的 Compute Group。在特定会话中确定默认 Compute Group 后，默认 Compute Group 将在该会话期间保持不变，除非用户显式更改了默认设置。

在不同次的会话中，若发生以下情况，系统可能会自动更改用户的默认 Compute Group：

- 用户失去了在上次会话中所选择默认 Compute Group 的使用权限
- 有 Compute Group 被添加或移除
- 上次所选择的默认 Compute Group 不再具有 Active BE

其中，情况一和情况二必定会导致系统自动选择的默认 Compute Group 更改，情况三可能会导致更改。

4.2.2.2.7 切换 Compute Group

用户可在存算分离架构中指定使用的数据库和 Compute Group。

语法

```
USE { [catalog_name.]database_name[@compute_group_name] | @compute_group_name }
```

若数据库或 Compute Group 名称包含是保留关键字，需用反引号将相应的名称 “ ‘ 包围。

4.2.2.2.8 Compute Group 扩缩容

通过 `ALTER SYSTEM ADD BACKEND` 以及 `ALTER SYSTEM DECOMMISSION BACKEND` 添加或者删除 BE 实现 Compute Group 的扩缩容。

详细操作参考存算分离相关操作

4.2.2.3 Workload Group

Workload Group 是一种进程内实现的对负载进行逻辑隔离的机制，它通过对 BE 进程内的资源（CPU，IO，Memory）进行细粒度的划分或者限制，达到资源隔离的目的，它的原理如下图所示：

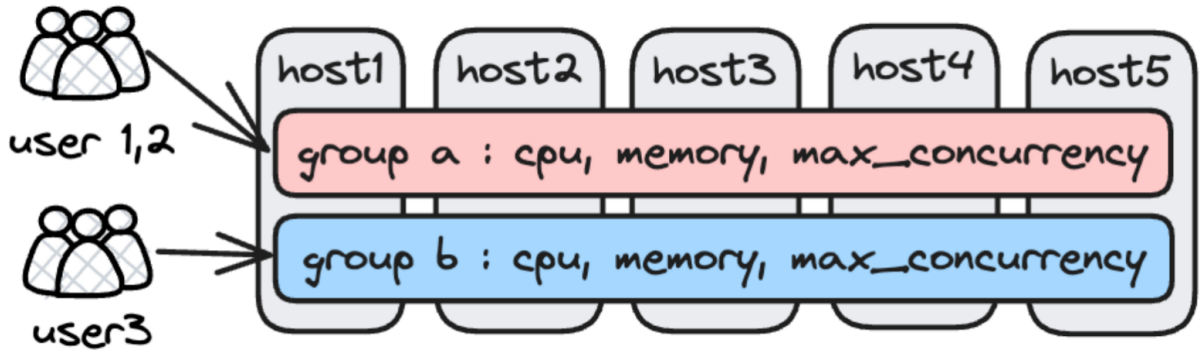


图 131: workload_group

目前支持的隔离能力包括：

- 管理 CPU 资源，支持 CPU 硬限和 CPU 软限;
- 管理内存资源，支持内存硬限和内存软限;
- 管理 IO 资源，包括读本地文件和远程文件产生的 IO。

> Workload Group 提供进程内的资源隔离能力，与进程间的资源隔离方式（Resource Group, Compute Group ）存在以下区别：
>
> 1. 进程内的资源隔离无法做到彻底的隔离性，比如高负载查询和低负载查询在同一个进程内运行，即使通过 Workload Group 对高负载分组的 CPU 使用进行限制使得整体的 CPU 使用在合理范围内，那么低负载分组的延迟也难免会受到影响，但相比于不做 CPU 管控的情况会有更好的表现。这是由于进程内部难免存在一些无法隔离的公共组件，比如公共的缓存和公共的 RPC 线程池。
> 2. 在做资源隔离方案的选择时，具体使用 Workload Group 还是基于进程的资源隔离方案（也就是把需要隔离的负载放到不同的进程），主要取决于隔离性和成本的权衡，可以容忍一定的延迟但是偏好低成本的场景，可以选择 Workload Group 的隔离方案；期望完全的隔离性同时可以接受更高的成本，那么可以选择基于进程的资源隔离方案，例如 Resource Group 或者 Compute Group，把高优负载划分到独立的 BE 节点上就可以做到比较彻底的隔离。

4.2.2.3.1 版本说明

- 自 Doris 2.0 版本开始提供 Workload Group 功能。在 Doris 2.0 版本中，Workload Group 功能不依赖于 CGroup，而 Doris 2.1 版本中需要依赖 CGroup。
- 从 Doris 1.2 升级到 2.0：建议集群升级完成后，再开启 Workload Group 功能。只升级部分 follower FE 节点，可能会因为未升级的 FE 节点没有 Workload Group 的元数据信息，导致已升级的 follower FE 节点查询失败。
- 从 Doris 2.0 升级到 2.1：由于 2.1 版本的 Workload Group 功能依赖于 CGroup，需要先配置 CGroup 环境，再升级到 Doris 2.1 版本。

4.2.2.3.2 配置 workload group

配置 CGroup 环境

Workload Group 支持对于 CPU，内存，IO 资源的管理，其中对于 CPU 的管理依赖 CGroup 组件；如果期望使用 Workload Group 管理 CPU 资源，那么首先需要进行 CGroup 环境的配置。

以下为 CGroup 环境配置流程：

1. 首先确认 BE 所在节点是否已经安装好 CGroup，输出结果中cgroup 代表目前的环境已经安装 CGroup V1，cgroup2 代表目前的环境已安装 CGroup V2，至于具体是哪个版本生效，可以通过下一步确认。

```
cat /proc/filesystems | grep cgroup
nodev   cgroup
nodev   cgroup2
nodev   cgroupfs
```

2. 通过路径名称可以确认目前生效的 CGroup 版本。“‘shell 如果存在这个路径说明目前生效的是 cgroup v1 /sys/fs/cgroup/cpu/

如果存在这个路径说明目前生效的是 cgroup v2 /sys/fs/cgroup/cgroup.controllers “‘

3. 在 CGroup 路径下新建一个名为 doris 的目录，这个目录名用户可以自行指定

```
如果是cgroup v1就在cpu目录下新建
mkdir /sys/fs/cgroup/cpu/doris


如果是cgroup v2就在直接在cgroup目录下新建
mkdir /sys/fs/cgroup/doris
```

4. 需要保证 Doris 的 BE 进程对于这个目录有读/写/执行权限 “‘shell // 如果是 CGroup v1，那么命令如下: // 1. 修改这个目录的权限为可读可写可执行 chmod 770 /sys/fs/cgroup/cpu/doris // 2. 把这个目录的归属划分给 doris 的账户 chown -R doris:doris /sys/fs/cgroup/cpu/doris

// 如果是 CGroup v2，那么命令如下: // 1. 修改这个目录的权限为可读可写可执行 chmod 770 /sys/fs/cgroup/doris // 2. 把这个目录的归属划分给 doris 的账户 chown -R doris:doris /sys/fs/cgroup/doris “‘

5. 如果目前环境里生效的是 CGroup v2 版本，那么还需要进行以下两步操作。如果是 CGroup v1 那么可以跳过当前步骤。

   - 修改根目录下的 cgroup.procs 文件权限，这是因为 CGroup v2 对于权限管控比较严格，需要具备根目录的 cgroup.procs 文件的写权限才能实现进程在 CGroup 目录之间的移动。

   ```
   chmod a+w /sys/fs/cgroup/cgroup.procs
   ```

   - 在 CGroup V2 中，cgroup.controllers 保存了当前目录可用的控制器，cgroup.subtree_control 保存了当前目录的子目录的可用控制器。因此需要确认 doris 目录是否已经启用 cpu 控制器，如果 doris 目录下的 cgroup.controllers 中不包含 cpu，那么说明 cpu 控制器未启用，可以在 doris 目录中执行以下命令，这个命令是通过修改父级目录的 cgroup.subtree_control 文件使得 doris 目录可以使用 cpu 控制器。

   ```
   // 预期该命令执行完成之后，可以在 doris 目录下看到 cpu.max 文件，且 cgroup.controllers
       ↪ 的输出包含 cpu。
   // 如果该命令执行失败，则说明 doris 目录的父级目录也未启用 cpu 控制器，需要为父级目录启用
       ↪ cpu 控制器。
   echo +cpu > ../cgroup.subtree_control
   ```

6. 修改 BE 的配置，指定 cgroup 的路径 “ ‘shell 如果是 Cgroup v1，那么配置路径如下 doris_cgroup_cpu_path = /sys/fs/cgroup/cpu/doris

如果是 Cgroup v2，那么配置路径如下 doris_cgroup_cpu_path = /sys/fs/cgroup/doris

```
7. 重启 BE，在日志（be.INFO）可以看到"add thread xxx to group"的字样代表配置成功


> 1. 建议单台机器上只部署一个 BE 实例，目前的 Workload Group 功能不支持一个机器上部署多个 BE；
> 2. 当机器重启之后，CGroup 路径下的所有配置就会清空。如果期望 CGroup 配置持久化，可以使用
    ↪ systemd 把操作设置成系统的自定义服务，这样在每次机器重启的时可以自动完成创建和授权操作
> 3. 如果是在容器内使用 CGroup，需要容器具备操作宿主机的权限。


####### 在容器中使用 Workload Group 的注意事项
Workload 的 CPU 管理是基于 CGroup 实现的，如果期望在容器中使用 Workload Group，
    ↪ 那么需要以特权模式启动容器，容器内的 Doris 进程才能具备读写宿主机 CGroup 文件的权限。
当 Doris 在容器内运行时，Workload Group 的 CPU 资源用量是在容器可用资源的情况下再划分的，
    ↪ 例如宿主机整机是 64 核，容器被分配了 8 个核的资源，Workload Group 配置的 CPU 硬限为 50%，
那么 Workload Group 实际可用核数为 4 个（8 核 * 50%）。

WorkloadGroup 的内存管理和 IO 管理功能是 Doris 内部实现，不依赖外部组件，
    ↪ 因此在容器和物理机上部署使用并没有区别。

如果要在 K8S 上使用 Doris，建议使用 Doris Operator 进行部署，可以屏蔽底层的权限细节问题。

###### 创建 Workload Group
```

mysql information_schema>create workload group if not exists g1 -> properties (-> “cpu_share”=“1024” ->); Query OK, 0 rows affected (0.03 sec)

“‘可以参考CREATE-WORKLOAD-GROUP。

此时配置的 CPU 限制为软限。自 2.1 版本起，系统会自动创建一个名为normal的 group，不可删除。

Workload Group 属性

属性名称	数据类型	默认值	取值范围	说明
cpu_share	整型	-1	[1, 10000]	可选，CPU 软限模式下生效，取值范围和使用的 CGroup 版本有关，下文有详细描述。cpu_share 代表了 Workload Group 可获得 CPU 时间的权重，值越大，可获得的 CPU 时间越多。例如，用户创建了 3 个 Workload Group g-a、g-b 和 g-c，cpu_share 分别为 10、30、40，某一时刻 g-a 和 g-b 正在跑任务，而 g-c 没有任务，此时 g-a 可获得 25% (10 / (10 + 30)) 的 CPU 资源，而 g-b 可获得 75% 的 CPU 资源。如果系统只有一个 Workload Group 正在运行，则不管其 cpu_share 的值为多少，它都可获取全部的 CPU 资源。
memory_limit	浮点	-1	(0%, 100%]	可选，开启内存硬限时代表当前 Workload Group 最大可用内存百分比，默认值代表不限制内存。所有 Workload Group 的 memory_limit 累加值不可以超过 100%，通常与 enable_memory_overcommit 属性配合使用。如果一个机器的内存为 64G，Workload Group 的 memory_limit 配置为 50%，那么该 group 的实际物理内存 =64G * 90% * 50%= 28.8G，这里的 90% 是 BE 进程可用内存配置的默认值。
enable_memory_overcommit	布尔	true	true, false	可选，用于控制当前 Workload Group 的内存限制是硬限还是软限，默认为 true。如果设置为 false，则该 workload group 为内存硬隔离，系统检测到 workload group 内存使用超出限制后将立即 cancel 组内内存占用最大的若干个任务，以释放超出的内存；如果设置为 true，则该 Workload Group 为内存软隔离，如果系统有空闲内存资源则该 Workload Group 在超出 memory_limit 的限制后可继续使用系统内存，在系统总内存紧张时会 cancel 组内内存占用最大的若干个任务，释放部分超出的内存以缓解系统内存压力。建议所有 workload group 的 memory_limit 总和低于 100%，为 BE 进程中的其他组件保留一些内存。
cpu_hard_limit	整型	-1	[1%, 100%]	可选，CPU 硬限制模式下生效，Workload Group 最大可用 CPU 百分比，不管当前机器的 CPU 资源是否被用满，Workload Group 的最大 CPU 用量都不能超过 cpu_hard_limit，所有 Workload Group 的 cpu_hard_limit 累加值不能超过 100%。2.1 版本新增属性，2.0 版本不支持该功能。
max_concurrency	整型	2147483647	[0, 2147483647]	可选，最大查询并发数，默认值为整型最大值，也就是不做并发的限制。运行中的查询数量达到最大并发时，新来的查询会进入排队的逻辑。
max_queue_size	整型	0	[0, 2147483647]	可选，查询排队队列的长度，当排队队列已满时，新来的查询会被拒绝。默认值为 0，含义是不排队。当排队队列已满时，新来的查询会直接失败。
queue_timeout	整型	0	[0, 2147483647]	可选，查询在排队队列中的最大等待时间，单位为毫秒。如果查询在队列中的排队时间超过这个值，那么就会直接抛出异常给客户端。默认值为 0，含义是不排队，查询进入队列后立即返回失败。
scan_thread_num	整型	-1	[1, 2147483647]	可选，当前 workload group 用于 scan 的线程个数。当该属性为 -1，含义是不生效，此时在 BE 上的实际取值为 BE 配置中的doris_scanner_thread_pool_thread_num。

属性名称	数据类型	默认值	取值范围	说明
max_remote_scan_thread_num	整型	-1	[1, 2147483647]	可选，读外部数据源的 scan 线程池的最大线程数。当该属性为 -1 时，实际的线程数由 BE 自行决定，通常和核数相关。
min_remote_scan_thread_num	整型	-1	[1, 2147483647]	可选，读外部数据源的 scan 线程池的最小线程数。当该属性为 -1 时，实际的线程数由 BE 自行决定，通常和核数相关。
tag	字符串	空	-	该功能已废弃，不推荐生产环境使用; 为 Workload Group 指定分组标签，相同标签的 Workload Group 资源累加值不能超过 100%；如果期望指定多个值，可以使用英文逗号分隔。
read_bytes_per_second	整型	-1	[1, 9223372036854775807]	可选，含义为读 Doris 内表时的最大 IO 吞吐，默认值为 -1，也就是不限制 IO 带宽。需要注意的是这个值并不绑定磁盘，而是绑定文件夹。比如为 Doris 配置了 2 个文件夹用于存放内表数据，那么每个文件夹的最大读 IO 不会超过该值，如果这 2 个文件夹都配置到同一块盘上，最大吞吐控制就会变成 2 倍的 read_bytes_per_second。落盘的文件目录也受该值的约束。
remote_read_bytes_per_second	整型	-1	[1, 9223372036854775807]	可选，含义为读 Doris 外表时的最大 IO 吞吐，默认值为 -1，也就是不限制 IO 带宽。

1. 目前暂不支持 CPU 的软限和硬限的同时使用，一个集群某一时刻只能是软限或者硬限，下文中会描述切换方法。
2. 所有属性均为可选，但是在创建 Workload Group 时需要指定至少一个属性。
3. 需要注意 CGroup v1 CGroup v2 版本 CPU 软限默认值是有区别的，CGroup v1 的 CPU 软限默认值为 1024，取值范围为 2 到 262144。而 CGroup v2 的 CPU 软限默认值为 100，取值范围是 1 到 10000。如果软限填了一个超出范围的值，这会导致 CPU 软限在 BE 修改失败。如果在 CGroup v1 的环境上如果按照 CGroup v2 的默认值 100 设置，这可能导致这个 workload group 的优先级在该机器上是最低的。

4.2.2.3.3 为用户设置 Workload Group

在把用户绑定到某个 Workload Group 之前，需要先确定该用户是否具有某个 Workload Group 的权限。可以使用这个用户查看 information_schema.workload_groups 系统表，返回的结果就是当前用户有权限使用的 Workload Group。下面的查询结果代表当前用户可以使用 g1 与 normal Workload Group：

```
SELECT name FROM information_schema.workload_groups;
+--------+
| name   |
+--------+
| normal |
| g1     |
+--------+
```

如果无法看到 g1 Workload Group，可以使用 ADMIN 账户执行 GRANT 语句为用户授权。例如：

```
GRANT USAGE_PRIV ON WORKLOAD GROUP 'g1' TO 'user_1'@'%';
```

这个语句的含义是把名为 g1 的 Workload Group 的使用权限授予给名为 user_1 的账户。更多授权操作可以参考grant 语句。

两种绑定方式 1. 通过设置 user property 将 user 默认绑定到 workload group，默认为normal，需要注意的这里的 value 不能填空，否则语句会执行失败。

```
set property 'default_workload_group' = 'g1';
```

执行完该语句后，当前用户的查询将默认使用’ g1’ 。

2. 通过 session 变量指定 workload group, 默认为空：

```
set workload_group = 'g1';
```

当同时使用了两种方式时为用户指定了 Workload Group，session 变量的优先级要高于 user property。

4.2.2.3.4 查看 Workload Group

1. 通过 show 语句查看

```
show workload groups;
```

可以参考SHOW-WORKLOAD-GROUPS。

2. 通过系统表查看

```
mysql [information_schema]>select * from information_schema.workload_groups where name='g1';
+-------+------+-----------+--------------+-------------------------+-----------------+----------------+--
    ↪
| ID    | NAME | CPU_SHARE | MEMORY_LIMIT | ENABLE_MEMORY_OVERCOMMIT | MAX_CONCURRENCY | MAX
    ↪ _QUEUE_SIZE | QUEUE_TIMEOUT | CPU_HARD_LIMIT | SCAN_THREAD_NUM | MAX_REMOTE_SCAN_
    ↪ THREAD_NUM | MIN_REMOTE_SCAN_THREAD_NUM | MEMORY_LOW_WATERMARK | MEMORY_HIGH_
    ↪ WATERMARK | TAG  | READ_BYTES_PER_SECOND | REMOTE_READ_BYTES_PER_SECOND |
+-------+------+-----------+--------------+-------------------------+-----------------+----------------+--
    ↪
| 14009 | g1   |      1024 | -1           | true                    |      2147483647 |
    ↪              0 |             0 | -1             |               -1 |
    ↪                           -1 |                         -1 | 50%                  | 80%
    ↪                     |      |                    -1 |                             -1 |
+-------+------+-----------+--------------+-------------------------+-----------------+----------------+--
    ↪
1 row in set (0.05 sec)
```

4.2.2.3.5 修改 Workload Group

```
mysql [information_schema]>alter workload group g1 properties('cpu_share'='2048');
Query OK, 0 rows affected (0.00 sec

mysql [information_schema]>select cpu_share from information_schema.workload_groups where name='
    ↪ g1';
+-----------+
| cpu_share |
+-----------+
|      2048 |
+-----------+
1 row in set (0.02 sec)
```

可以参考：ALTER-WORKLOAD-GROUP。

4.2.2.3.6 删除 Workload Group

```
mysql [information_schema]>drop workload group g1;
Query OK, 0 rows affected (0.01 sec)
```

可以参考：DROP-WORKLOAD-GROUP。

4.2.2.3.7 CPU 软硬限模式切换的说明

目前 Doris 暂不支持同时运行 CPU 的软限和硬限，一个 Doris 集群在任意时刻只能是 CPU 软限或者 CPU 硬限。用户可以在两种模式之间进行切换，切换方法如下：

1 假如当前的集群配置是默认的 CPU 软限，期望改成 CPU 的硬限，需要把 Workload Group 的 cpu_hard_limit 参数修改成一个有效的值

```
alter workload group test_group properties ( 'cpu_hard_limit'='20%' );
```

集群中所有的 Workload Group 都需要修改，所有 Workload Group 的 cpu_hard_limit 的累加值不能超过 100% 。

由于 CPU 的硬限无法给出一个有效的默认值，因此如果只打开开关但是不修改属性，那么 CPU 的硬限也无法生效。

2 在所有 FE 中打开 CPU 硬限的开关

```
1 修改磁盘上fe.conf的配置
experimental_enable_cpu_hard_limit = true
2 修改内存中的配置
ADMIN SET FRONTEND CONFIG ("enable_cpu_hard_limit" = "true");
```

如果用户期望从 CPU 的硬限切换回 CPU 的软限，需要在所有 FE 修改 enable_cpu_hard_limit 的值为 false 即可。CPU 软限的属性 cpu_share 默认会填充一个有效值 1024(如果之前未指定 cpu_share 的值)，用户可以根据 group 的优先级对 cpu_share 的值进行重新调整。

4.2.2.3.8 效果测试

内存硬限

Adhoc 类查询通常输入的 SQL 不确定，使用的内存资源也不确定，因此存在少数查询占用很大内存的风险。可以对这类负载可以划分到独立的分组，通过 Workload Group 对内存的硬限的功能，避免突发性的大查询占满所有内存，导致其他查询没有可用内存或者 OOM。当这个 Workload Group 的内存使用超过配置的硬限值时，会通过杀死查询的方式释放内存，避免进程内存被打满。

测试环境

1FE，1BE，BE 配置为 96 核，内存大小为 375G。

测试数据集为 clickbench，测试方法为使用 jmeter 起三并发执行 q29。

测试不开启 Workload Group 的内存硬限

1. 查看进程使用内存。ps 命令输出第四列代表进程使用的物理内存的用量，单位为 kb，可以看到当前测试负载下，进程的内存使用为 7.7G 左右。

```
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         2.0 7896792
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         2.0 7929692
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         2.0 8101232
```

2. 使用 Doris 系统表查看当前 Workload Group 的内存用量，Workload Group 的内存用量为 5.8G 左右。

```
mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+-------------------+
| wg_mem_used_mb    |
+-------------------+
| 5797.524360656738 |
+-------------------+
1 row in set (0.01 sec)

mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+-------------------+
| wg_mem_used_mb    |
+-------------------+
| 5840.246627807617 |
+-------------------+
1 row in set (0.02 sec)
```

```
mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+-------------------+
| wg_mem_used_mb    |
+-------------------+
| 5878.394917488098 |
+-------------------+
1 row in set (0.02 sec)
```

这里可以看到进程的内存使用通常要远大于一个 Workload Group 的内存用量，即使进程内只有一个 Workload Group 在跑，这是因为 Workload Group 只统计了查询和部分导入的内存，进程内的其他组件比如元数据，各种 Cache 的内存是不计算 Workload Group 内的，也不由 Workload Group 管理。

测试开启 Workload Group 的内存硬限 1. 执行 SQL 命令修改内存配置。

```
alter workload group g2 properties('memory_limit'='0.5%');
alter workload group g2 properties('enable_memory_overcommit'='false');
```

2. 执行同样的测试，查看系统表的内存用量，内存用量为 1.5G 左右。

```
mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+--------------------+
| wg_mem_used_mb     |
+--------------------+
| 1575.3877239227295 |
+--------------------+
1 row in set (0.02 sec)

mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+-------------------+
| wg_mem_used_mb    |
+-------------------+
| 1668.77405834198 |
+-------------------+
1 row in set (0.01 sec)

mysql [information_schema]>select MEMORY_USAGE_BYTES / 1024/ 1024 as wg_mem_used_mb from
    ↪ workload_group_resource_usage where workload_group_id=11201;
+--------------------+
| wg_mem_used_mb     |
+--------------------+
| 499.96760272979736 |
+--------------------+
1 row in set (0.01 sec)
```

3. 使用 ps 命令查看进程的内存用量，内存用量为 3.8G 左右。

```
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         1.0 4071364
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         1.0 4059012
[ ~]$ ps -eo pid,comm,%mem,rss | grep 1407481
1407481 doris_be         1.0 4057068
```

4. 同时客户端会观察到大量由于内存不足导致的查询失败。

```
1724074250162,14126,1c_sql,HY000 1105,"java.sql.SQLException: errCode = 2, detailMessage =
    ↪ (127.0.0.1)[MEM_LIMIT_EXCEEDED]GC wg for hard limit, wg id:11201, name:g2, used:1.71
    ↪ GB, limit:1.69 GB, backend:10.16.10.8. cancel top memory used tracker <Query#Id=4
    ↪ a0689936c444ac8-a0d01a50b944f6e7> consumption 1.71 GB. details:process memory used
    ↪ 3.01 GB exceed soft limit 304.41 GB or sys available memory 101.16 GB less than
    ↪ warning water mark 12.80 GB., Execute again after enough memory, details see be.INFO.
    ↪ ",并发 1-3,text,false,,444,0,3,3,null,0,0,0
```

这个报错信息中可以看到，Workload Group 使用了 1.7G 的内存，但是 Workload Group 的限制是 1.69G，这里的计算方式是这样的 1.69G = 物理机内存 (375) * mem_limit(be.conf 中的值，默认为 0.9) * 0.5%（Workload Group 的配置）也就是说，Workload Group 中配置的内存百分比是基于 Doris 进程可用内存再次进行计算的。

使用建议

如上文测试，硬限可以控制 Workload Group 的内存使用，但却是通过杀死查询的方式释放内存，这对用户来说体验会非常不友好，极端情况下可能会导致所有查询都失败。因此在生产环境中推荐内存硬限配合查询排队的功能一起使用，可以在限制内存使用的同时保证查询的成功率。

CPU 硬限

Doris 的负载大体可以分为三类：1. 核心报表查询，通常给公司高层查看报表使用，负载不一定很高，但是对可用性要求较高，这类查询可以划分到一个分组，配置较高优先级的软限，保证 CPU 资源不够时可以获得更多的 CPU 资源。2. Adhoc 类查询，这类查询通常偏探索分析，SQL 比较随机，具体的资源用量也比较未知，优先级通常不高。因此可以使用 CPU 硬限进行管理，并配置较低的值，避免占用过多 CPU 资源降低集群可用性。3. ETL 类查询，这类查询的 SQL 比较固定，资源用量通常也比较稳定，偶尔会出现上游数据量增长导致资源用量暴涨的情况，因此可以使用 CPU 硬限进行配置。

不同的负载对 CPU 的消耗不一样，用户对响应延时的需求也不一样。当 BE 的 CPU 被用的很满时，可用性会变差，响应延时会变高。比如可能一个 Adhoc 的分析类查询把整个集群的 CPU 打满，导致核心报表的延时变大，影响到了 SLA。所以需要 CPU 隔离机制来对不同的业务进行隔离，保障集群的可用性和 SLA。Workload Group 支持 CPU 的软限和硬限，目前比较推荐在线上环境把 Workload Group 配置成硬限。原因是 CPU 的软限通常在 CPU 被打满时才能体现出优先级的作用，但是在 CPU 被用满时，Doris 的内部组件（例如 rpc 组件）以及操作系统可用的 CPU 会减少，此时集群整体的可用性是下降比较严重的，因此生产环境通常需要避免 CPU 资源被打满的情况，当然其他资源也一样，内存资源同理。

测试环境

1FE，1BE，96 核机器。数据集为 clickbench，测试 sql 为 q29。

发起测试 1. 使用 jmeter 发起 3 并发查询，把 BE 进程的 CPU 使用压到比较高的使用率，这里测试的机器是 96 核，使用 top 命令看到 BE 进程 CPU 使用率为 7600% 的含义是该进程目前使用中的核数是 76 个。

```
  PR   NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  20    0   71.5g  30.4g  66560 S  7660   8.1  40:39.89 doris_be
```

图 132: use workload group cpu

2. 修改使用中的 Workload Group 的 CPU 硬限为 10%。

```
alter workload group g2 properties('cpu_hard_limit'='10%');
```

3. 集群开启硬限模式，此时集群中所有 Group 都会切换为硬限。

```
ADMIN SET FRONTEND CONFIG ("enable_cpu_hard_limit" = "true");
```

4. 重新压测查询负载，可以看到当前进程只能使用 9 到 10 个核，占总核数的 10% 左右。

```
  PR   NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  20    0   72.6g  21.2g  62056 S 979.7   5.6  57:23.63 doris_be
```

图 133: use workload group cpu

需要注意的是，这里的测试最好使用查询负载会比较能体现出效果，因为如果是高吞吐导入的话，可能会触发 Compaction，使得实际观测的值要比 Workload Group 配置的值大。而 Compaction 的负载目前是没有归入 Workload Group 的管理的。

5. 除了使用 Linux 的系统命令外，还可以通过使用 Doris 的系统表观察 Group 目前的 CPU 使用为 10% 左右。

```
mysql [information_schema]>select CPU_USAGE_PERCENT from workload_group_resource_usage where
    ↪ WORKLOAD_GROUP_ID=11201;
+-------------------+
| CPU_USAGE_PERCENT |
+-------------------+
|              9.57 |
+-------------------+
1 row in set (0.02 sec)
```

注意事项

1. 在实际配置的时候，所有 Group 的 CPU 累加值最好不要正好等于 100%，这主要是为了保证低延迟场景的可用性。因为需要让出一部分资源给其他组件使用。当然如果对延迟不是很敏感的场景，期望最高的资源利用率，那么可以考虑所有 Group 的 CPU 累加值配置等于 100%。
2. 目前 FE 向 BE 同步 Workload Group 元数据的时间间隔为 30 秒，因此对于 Workload Group 的变更最大需要等待 30 秒才能生效。

本地 IO 硬限

OLAP 系统在做 ETL 或者大的 Adhoc 查询时，需要读取大量的数据，Doris 为了加速数据分析过程，内部会使用多线程并行的方式对多个磁盘文件扫描，会产生巨大的磁盘 IO，就会对其他的查询（比如报表分析）产生影响。可以通过 Workload Group 对离线的 ETL 数据处理和在线的报表查询做分组，限制离线数据处理 IO 带宽的方式，降低它对在线报表分析的影响。

测试环境

1FE,1BE, 配置为 96 核。

测试数据集为 clickbench。

不开启 IO 硬限测试 1. 关闭缓存。

```
// 清空操作系统缓存
sync; echo 3 > /proc/sys/vm/drop_caches

// 禁用 BE 的 page cache
disable_storage_page_cache = true
```

2. 对 clickbench 的表执行全表扫描，执行单并发查询即可。

```
set dry_run_query = true;
select * from hits.hits;
```

3. 通过 Doris 的内表查看当前 Group 的最大吞吐为 3GB 每秒。

```
mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
+--------------------+
| mb_per_sec         |
+--------------------+
| 1146.6208400726318 |
+--------------------+
1 row in set (0.03 sec)

mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
```

```
+--------------------+
| mb_per_sec         |
+--------------------+
| 3496.2762966156006 |
+--------------------+
1 row in set (0.04 sec)

mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
+--------------------+
| mb_per_sec         |
+--------------------+
| 2192.7690029144287 |
+--------------------+
1 row in set (0.02 sec)
```

4. 使用 pidstat 命令查看进程 IO，图中第一列是进程 id，第二列是读 IO 的吞吐（单位是 kb/s）。可以看到不限制 IO 时，最大吞吐为 2G 每秒。

```
s be]$ pidstat -d 1 3600 | grep 884431
 884431  14124.00     16.00      0.00       0  doris_be
 884431    224.00      0.00      0.00       0  doris_be
 884431      0.00      8.00      0.00       0  doris_be
 884431 675788.00      8.00      0.00       0  doris_be
 884431 1891976.00      4.00      0.00       0  doris_be
 884431 2101908.00      4.00      0.00       0  doris_be
 884431 2106872.00      4.00      0.00       0  doris_be
 884431 1829000.00      8.00      0.00       0  doris_be
 884431 159376.00      4.00      0.00       0  doris_be
 884431 124836.00      4.00      0.00       0  doris_be
 884431 117048.00      8.00      0.00       0  doris_be
 884431 738164.00     12.00      0.00       0  doris_be
 884431 776224.00      4.00      0.00       0  doris_be
 884431 614728.00      4.00      0.00       0  doris_be
 884431 333784.00      8.00      0.00       0  doris_be
 884431 514560.00      8.00      0.00       0  doris_be
 884431 462176.00      8.00      0.00       0  doris_be
 884431 396720.00      8.00      0.00       0  doris_be
 884431 472272.00      4.00      0.00       0  doris_be
 884431 373544.00      4.00      0.00       0  doris_be
 884431 409288.00      0.00      0.00       0  doris_be
 884431 374816.00      4.00      0.00       0  doris_be
 884431 303184.00     12.00      0.00       0  doris_be
 884431 347520.00      0.00      0.00       0  doris_be
 884431 275096.00      0.00      0.00       0  doris_be
 884431 217592.00      0.00      0.00       0  doris_be
 884431 260428.00      4.00      0.00       0  doris_be
 884431 163272.00      8.00      0.00       0  doris_be
 884431 185872.00      8.00      0.00       0  doris_be
 884431 153172.00      4.00      0.00       0  doris_be
 884431 116780.00      0.00      0.00       0  doris_be
 884431 108364.00      4.00      0.00       0  doris_be
 884431  64804.00     12.00      0.00       0  doris_be
 884431  49116.00      0.00      0.00       0  doris_be
 884431  10700.00      8.00      0.00       0  doris_be
 884431   6188.00      4.00      0.00       0  doris_be
 884431     88.00      4.00      0.00       0  doris_be
 884431      0.00     20.00      0.00       0  doris_be
```

图 134: use workload group io

开启 IO 硬限后测试 1. 关闭缓存。

```
// 清空操作系统缓存
sync; echo 3 > /proc/sys/vm/drop_caches
```

```
// 禁用 BE 的 page cache
disable_storage_page_cache = true
```

2. 修改 Workload Group 的配置，限制每秒最大吞吐为 100M。

```
// 限制当前 Group 的读吞吐为每秒 100M
alter workload group g2 properties('read_bytes_per_second'='104857600');
```

3. 使用 Doris 系统表查看 Workload Group 的最大 IO 吞吐为每秒 98M。

```
mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
+--------------------+
| mb_per_sec         |
+--------------------+
| 97.94296646118164  |
+--------------------+
1 row in set (0.03 sec)

mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
+--------------------+
| mb_per_sec         |
+--------------------+
| 98.37584781646729  |
+--------------------+
1 row in set (0.04 sec)

mysql [information_schema]>select LOCAL_SCAN_BYTES_PER_SECOND / 1024 / 1024 as mb_per_sec
    ↪ from workload_group_resource_usage where WORKLOAD_GROUP_ID=11201;
+--------------------+
| mb_per_sec         |
+--------------------+
| 98.06641292572021  |
+--------------------+
1 row in set (0.02 sec)
```

4. 使用 pid 工具查看进程最大 IO 吞吐为每秒 131M。

```
s be]$ pidstat -d 1 3600 | grep 968609
 968609      0.00      3.67      0.00       0  doris_be
 968609 143028.00     24.00      0.00       0  doris_be
 968609 162668.00      0.00      0.00       0  doris_be
 968609  80256.00      4.00      0.00       0  doris_be
 968609  91800.00      0.00      0.00       0  doris_be
 968609 137492.00      0.00      0.00       0  doris_be
 968609 125756.00      8.00      0.00       0  doris_be
 968609 134624.00      8.00      0.00       0  doris_be
 968609 109180.00      0.00      0.00       0  doris_be
 968609 114524.00      4.00      0.00       0  doris_be
 968609 124952.00      4.00      0.00       0  doris_be
 968609 120524.00      8.00      0.00       0  doris_be
 968609 103064.00      8.00      0.00       0  doris_be
 968609 126904.00      4.00      0.00       0  doris_be
 968609 121760.00      8.00      0.00       0  doris_be
 968609 119848.00      0.00      0.00       0  doris_be
 968609 118012.00      8.00      0.00       0  doris_be
 968609 112476.00      0.00      0.00       0  doris_be
 968609 118296.00      0.00      0.00       0  doris_be
 968609 120792.00      4.00      0.00       0  doris_be
 968609 117196.00      0.00      0.00       0  doris_be
 968609 122848.00     12.00      0.00       0  doris_be
 968609 127760.00      4.00      0.00       0  doris_be
```

图 135: use workload group io

注意事项 1. 系统表中的 LOCAL_SCAN_BYTES_PER_SECOND 字段代表的是当前 Workload Group 在进程粒度的统计汇总值，比如配置了 12 个文件路径，那么 LOCAL_SCAN_BYTES_PER_SECOND 就是这 12 个文件路径 IO 的最大值，如果期望查看每个文件路径分别的 IO 吞吐，可以在 grafana 监控查看明细的值。

2. 由于操作系统和 Doris 的 Page Cache 的存在，通过 linux 的 IO 监控脚本看到的 IO 通常要比系统表看到的要小。

远程 IO 硬限

BrokerLoad 和 S3Load 是常用的大批量数据导入方式，用户可以把数据先上传到 HDFS 或者 S3，然后通过 Brokerload 和 S3Load 对数据进行并行导入。Doris 为了加快导入速度，会使用多线程并行的方式从 HDFS/S3 拉取数据，此时会对 HDFS/S3 产生巨大的压力，会导致 HDFS/S3 上运行的别的作业不稳定。可以通过 Workload Group 远程 IO 的限制功能来限制导入过程中对 HDFS/S3 的带宽，降低对其他业务的影响。

测试环境

1FE，1BE 部署在同一台机器，配置为 16 核 64G 内存。测试数据为 clickbench 数据集，测试前需要把数据集上传到 S3 上。考虑到上传时间的问题，我们只取其中的 1 千万行数据上传，然后使用 tvf 的功能查询 s3 的数据。

上传成功后可以使用命令查看 Schema 信息。

```
// 查看schema
DESC FUNCTION s3 (
    "URI" = "https://bucketname/1kw.tsv",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "csv",
    "use_path_style"="true"
);
```

测试不限制远程读的 IO 1. 发起单并发测试，全表扫描 clickbench 表。

```
// 设置只 scan 数据，不返回结果
set dry_run_query = true;

// 使用 tvf 查询 s3 的数据
SELECT * FROM s3(
    "URI" = "https://bucketname/1kw.tsv",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "csv",
    "use_path_style"="true"
);
```

2. 使用系统表查看此时的远程 IO 吞吐。可以看到这个查询的远程 IO 吞吐为 837M 每秒，需要注意的是，这里的实际 IO 吞吐受环境影响较大，如果 BE 所在的机器连接外部存储的带宽比较低，那么可能实际的吞吐会小。

```
MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|     837 |
+---------+
1 row in set (0.104 sec)

MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|     867 |
+---------+
1 row in set (0.070 sec)
```

```
MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|     867 |
+---------+
1 row in set (0.186 sec)
```

3. 使用 sar(sar -n DEV 1 3600) 命令查看机器的网络带宽，可以看到机器级别最大网络带宽为 1033M 每秒。输出的第一列为当前机器某个网卡每秒接收的字节数，单位为 KB 每秒。

```
    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
      7.56      7.56      0.00      0.00      0.00      0.00
961294.37   2492.58      0.00      0.00      0.00      0.00

    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
     19.92     19.92      0.00      0.00      0.00      0.00
1058684.12   2975.14      0.00      0.00      0.00      0.00

    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
    228.71    228.71      0.00      0.00      0.00      0.00
933606.63   2469.83      0.00      0.00      0.00      0.00

    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
      0.10      0.10      0.00      0.00      0.00      0.00
745845.31   2023.35      0.00      0.00      0.00      0.00
```

图 136: use workload group rio

测试限制远程读的 IO 1. 修改 Workload Group 的配置，限制远程读的 IO 吞吐为 100M 每秒。

```
alter workload group normal properties('remote_read_bytes_per_second'='104857600');
```

2. 发起单并发扫全表的查询。

```
// 设置只 scan 数据，不返回结果
set dry_run_query = true;

// 使用 tvf 查询 s3 的数据
SELECT * FROM s3(
    "URI" = "https://bucketname/1kw.tsv",
```

```
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "csv",
    "use_path_style"="true"
);
```

3. 使用系统表查看此时的远程读 IO 吞吐，此时的 IO 吞吐在 100M 左右，会有一定的波动，这个波动是受目前算法设计的影响，通常会有一个高峰，但不会持续很长时间，属于正常情况。

```
MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|      56 |
+---------+
1 row in set (0.010 sec)

MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|     131 |
+---------+
1 row in set (0.009 sec)

MySQL [(none)]> select cast(REMOTE_SCAN_BYTES_PER_SECOND/1024/1024 as int) as read_mb from
    ↪ information_schema.workload_group_resource_usage;
+---------+
| read_mb |
+---------+
|     111 |
+---------+
1 row in set (0.009 sec)
```

4. 使用 sar 命令（sar -n DEV 1 3600）查看目前的网卡接收流量，第一列为每秒接收的数据量，可以看到最大值变成了 207M 每秒，说明读 IO 的限制是生效的，但是由于 sar 命令看到的是机器级别的流量，因此要比 Doris 统计到的会大一些。

rxkB/s	txkB/s	rxcmp/s	txcmp/s	rxmcst/s	%ifutil
3.33	3.33	0.00	0.00	0.00	0.00
140384.12	727.60	0.00	0.00	0.00	0.00

rxkB/s	txkB/s	rxcmp/s	txcmp/s	rxmcst/s	%ifutil
2.01	2.01	0.00	0.00	0.00	0.00
212281.74	1277.05	0.00	0.00	0.00	0.00

rxkB/s	txkB/s	rxcmp/s	txcmp/s	rxmcst/s	%ifutil
1.53	1.53	0.00	0.00	0.00	0.00
89753.66	495.19	0.00	0.00	0.00	0.00

rxkB/s	txkB/s	rxcmp/s	txcmp/s	rxmcst/s	%ifutil
1.83	1.83	0.00	0.00	0.00	0.00
106358.55	545.90	0.00	0.00	0.00	0.00

图 137: use workload group rio

4.2.2.3.9 常见问题

1. 为什么配置了 CPU 的硬限但是没有生效？

- 通常有以下几种原因：
  - 环境初始化失败，需要检查 Doris CGroup 路径下的两个配置文件，这里以 CGroup V1 版本为例，如果用户指定的 Doris 的 CGroup 路径为`/sys/fs/cgroup/cpu/doris/`，那么首先需要去查看`/sys/fs/cgroup/cpu/doris/query/1/tasks`文件的内容是否包含对应 Workload Group 的线程号，路径中的 1 代表的是 Workload Group 的 id，可以通过`top -H -b -n 1 -p pid`的命令获得该 Workload Group 的线程号，通过对比确认该 Workload Group 的线程号都写入到 tasks 文件中；然后是看下`/sys/fs/cgroup/cpu/doris/query/1/cpu.cfs_quota_us`文件的值是否为 -1，如果为 -1 就说明 CPU 硬限的配置没有生效。
  - Doris BE 进程的 CPU 使用率高于 Workload Group 配置的 CPU 硬限，这种情况是符合预期的，因为 Workload Group 可以管理的 CPU 主要是查询线程和导入的 memtable 下刷线程，但是 BE 进程内通常还会有其他组件也会消耗 CPU，比如 Compaction，因此进程的 CPU 使用通常要高于 Workload Group 的配置。可以创建一个测试的 Workload Group，只压测查询负载，然后通过系统表`information_schema.workload_group_resource_usage`查看 Workload Group 的 CPU 使用，这个表只记录了 Workload Group 的 CPU 使用率，从 2.1.6 版本开始支持。
- 有用户配置了参数`cpu_resource_limit`，首先通过执行`show property for jack like 'cpu_resource_limit'`;确认用户 jack 属性中是否设置了该参数；然后通过执行`show variables like 'cpu_resource_limit'` 确认 session 变量中是否设置了该参数；该参数默认值为 -1，代表未设置。配置了这个参数之后，查询走的是独立的线程池，该线程池不受 Workload Group 的管理。直接修改这个参数可能会影响生产环境的稳定性，可以考虑逐步的把配置了该参数的查询负载迁移到 Workload Group 中管理，这个参数目前的平替是 session 变量 `num_scanner_threads`。主要流程是，先把配置了 `cpu_resource_limit` 的用户分成若干批次，迁移第一批用户的时候，首先修改这部分用户的 session 变量 `num_scanner_threads` 为

1，然后为这些用户指定 Workload Group，接着把 cpu_resource_limit 修改为 -1，观察一段时间集群是否稳定，如果稳定就继续迁移下一批用户。

2. 为什么默认的 Workload Group 的个数被限制为 15 个？

- Workload Group 主要是对单机资源的划分，一个机器上如果划分了过多的 Workload Group，那么每个 Workload Group 都只能分到很少的资源。如果业务确实需要建这么多的 Workload Group，那么可以考虑把一个集群划分为多组不同的 BE，然后为每组 BE 创建不同的 Workload Group。也可以通过修改 FE 的配置 workload_group_max_num 来临时绕开这个限制。

3. 为什么配置了较多 Workload Group 之后会报错“Resource temporarily unavailable”？

- 每个 Workload Group 都是一组独立的线程池，创建过多的 Workload Group 可能会导致 BE 进程尝试启动过多的线程，超过操作系统允许的进程的可用线程数上限。遇到这个问题通常修改操作系统的环境配置，允许 BE 进程可以创建更多的线程。

4.2.3 工作负载分析与诊断

集群的工作负载分析主要分成两个阶段：- 第一个是运行时的工作负载分析，当集群可用性下降时，可以通过监控发现资源开销比较大的查询，并进行降级处理。- 第二个是分析历史数据，比如审计日志表等，找出不合理的工作负载，并进行优化。

4.2.3.1 运行时的工作负载分析

当通过监控发现集群的可用性下降时，可以按照以下流程进行处理：1. 先通过监控大致判断目前集群的瓶颈点，比如是内存用量过高，CPU 用量过高，还是 IO 过高，如果都很高，那么可以考虑优先解决内存的问题。

2. 确定了集群瓶颈点后，可以通过 workload_group_resource_usage 表来查看目前资源用量最高的 Group，比如是内存瓶颈，那么可以先找出内存用量最高的 N 个 Group。
3. 确定了资源用量最高的 Group 之后，首先可以尝试调低这个 Group 的查询并发，因为此时集群资源已经很紧张了，要避免持续有新的查询进来耗尽集群的资源。
4. 对当前 Group 的查询进行降级，根据瓶颈点的不同，可以有不同的处理方法：
    - 如果是 CPU 瓶颈，那么可以尝试把这个 Group 的 CPU 修改为硬限，并将 cpu_hard_limit 设置为一个较低的值，主动让出 CPU 资源。
    - 如果是 IO 瓶颈，可以通过 read_bytes_per_second 参数限制该 Group 的最大 IO。
    - 如果是内存瓶颈，可以通过设置该 Group 的内存为硬限，并调低 memory_limit 的值，来释放部分内存，需要注意的是这可能会导致当前 Group 大量查询失败。
5. 完成以上步骤后，通常集群的可用性会有所恢复。此时可以做更进一步的分析，也就是引起该 Group 资源用量升高的主要原因，如果是这个 Group 的整体查询并发升高导致的还是某些大查询导致的，如果是某些大查询导致的，那么可以通过快速杀死这些大查询的方式进行故障恢复。
6. 可以使用 backend_active_tasks 结合 active_queries 找出目前集群中资源用量比较异常的 SQL，然后通过 kill 语句杀死这些 SQL 释放资源。

4.2.3.2 通过历史数据分析工作负载

目前 Doris 的审计日志表中留存了 sql 执行时的简要信息，可以用于找出历史上执行过的不合理的查询，然后做出一些调整，具体流程如下：1. 查看监控确认集群历史的资源用量情况，找出集群的瓶颈点，比如是 CPU，内存还是 IO。2. 确定了集群的瓶颈点后，可以通过审计日志表找出对应时段资源用量比较异常的 SQL，异常 SQL 有两种定义方式

1. 用户对于集群中 SQL 资源的使用量有一定的预期，比如大部分延迟都是秒级，扫描行数在千万，那么当有扫描行数在亿级别十亿级别的 SQL，就属于异常 SQL，需要人工进行处理
2. 如果用户对于集群中 SQL 资源用量也没有预期，这个时候可以通过百分位函数计算资源用量的方式，找出资源用量比较异常的 SQL。以 CPU 瓶颈为例，可以先计算历史时间段内查询 CPU 时间的 tp50/tp75/tp99/tp999，以该值为正常值，对照当前集群相同时间段内查询 CPU 时间的百分位函数，比如历史时段的 tp999 为 1min，但是当前集群相同时段 CPU 时间的 tp50 就已经是 1min，说明当前时段内相比于历史出现了大量的 CPU 时间在 1min 以上的 sql，那么 CPU 时间大于 1min 的 SQL 就可以定义为异常 SQL。其他指标的异常值的查看也是同理。
3. 对资源用量异常的 SQL 进行优化，比如 SQL 改写，表结构优化，并行度调节等方式降低单 SQL 的资源用量。
4. 如果通过审计表发现 SQL 的资源用量都比较正常，那么可以通过监控和审计查看当时执行的 SQL 的数量相比于历史时期是否有增加，如果有的话，可以跟上游业务确认对应时间段上游的访问流量是否有增加，从而选择是进行集群扩容还是排队限流操作。

4.2.3.3 常用 SQL

> 提示需要注意的是，active_queries 表记录了在 FE 运行的 query，backend_active_tasks 记录了在 BE 运行的 query，并非所有 query 运行时在 FE 注册，比如 stream load 就并未在 FE 注册。因此使用 backend_active_tasks 表 left join active_queries 如果没有匹配的结果属于正常情况。当一个 Query 是 select 查询时，那么 active_queries 和 backend_active_tasks 中记录的 queryId 是一致的。当一个 Query 是 stream load 时，那么在 active_queries 表中的 queryId 为空，backend_active_tasks 的 queryId 是该 stream load 的 Id。

1. 查看目前所有的 Workload Group 并依次按照内存/CPU/IO 降序显示。

```
select be_id,workload_group_id,memory_usage_bytes,cpu_usage_percent,local_scan_bytes_per_
    ↪ second
from workload_group_resource_usage
order by  memory_usage_bytes,cpu_usage_percent,local_scan_bytes_per_second desc
```

2. CPU 使用 topN 的 sql

```
select
        t1.query_id as be_query_id,
        t1.query_type,
        t2.query_id,
        t2.workload_group_id,
        t2.`database`,
        t1.cpu_time,
        t2.`sql`
from
(select query_id, query_type,sum(task_cpu_time_ms) as cpu_time from backend_active_tasks
    ↪ group by query_id, query_type)
        t1 left join active_queries t2
on t1.query_id = t2.query_id
order by cpu_time desc limit 10;
```

3. 内存使用 topN 的 sql

```
select
        t1.query_id as be_query_id,
        t1.query_type,
        t2.query_id,
        t2.workload_group_id,
        t1.mem_used
from
(select query_id, query_type, sum(current_used_memory_bytes) as mem_used from backend_active_
    ↪ tasks group by query_id, query_type)
        t1 left join active_queries t2
on t1.query_id = t2.query_id
order by mem_used desc limit 10;
```

4. 扫描数据量 topN 的 sql

```
select
        t1.query_id as be_query_id,
        t1.query_type,
        t2.query_id,
        t2.workload_group_id,
        t1.scan_rows,
        t1.scan_bytes
from
(select query_id, query_type, sum(scan_rows) as scan_rows,sum(scan_bytes) as scan_bytes from
    ↪ backend_active_tasks group by query_id,query_type)
        t1 left join active_queries t2
```

```
on t1.query_id = t2.query_id
order by scan_rows desc,scan_bytes desc limit 10;
```

5. 各个 workload group 的 scan 数据量

```
select
        t2.workload_group_id,
        sum(t1.scan_rows) as wg_scan_rows,
        sum(t1.scan_bytes) as wg_scan_bytes
from
(select query_id, sum(scan_rows) as scan_rows,sum(scan_bytes) as scan_bytes from backend_
    ↪ active_tasks group by query_id)
        t1 left join active_queries t2
on t1.query_id = t2.query_id
group by t2.workload_group_id
order by wg_scan_rows desc,wg_scan_bytes desc
```

6. 查看各个 workload group 排队的都是哪些查询，以及排队的时间

```
select
         workload_group_id,
         query_id,
         query_status,
         now() - queue_start_time as queued_time
from
     active_queries
where query_status='queued'
order by workload_group_id
```

4.2.4 并发控制与排队

并发控制与排队是一种资源管理机制，当多个查询同时请求资源，达到系统并发能力的上限时，Doris 会根据预设的策略和限制条件对查询进行排队管理，确保系统在高负载情况下仍能平稳运行，避免 OOM、系统卡死等问题。

Doris 的并发控制与排队机制主要通过 workload group 来实现。workload group 定义了查询的资源使用上限，包括最大并发数、排队队列长度和超时时间等参数。通过合理配置这些参数，可以达到资源管控的目的。

4.2.4.1 基本使用

```
create workload group if not exists queue_group
properties (
    "max_concurrency" = "10",
```

```
    "max_queue_size" = "20",
    "queue_timeout" = "3000"
);
```

参数说明

属性名称	数据类型	默认值	取值范围	说明
max_concurrency	整型	2147483647	[0, 2147483647]	可选，最大查询并发数，默认值为整型最大值，也就是不做并发的限制。运行中的查询数量达到最大并发时，新来的查询会进入排队的逻辑。
max_queue_size	整型	0	[0, 2147483647]	可选，查询排队队列的长度，当排队队列已满时，新来的查询会被拒绝。默认值为 0，含义是不排队。
queue_timeout	整型	0	[0, 2147483647]	可选，查询在排队队列中的最大等待时间，单位为毫秒。如果查询在队列中的排队时间超过这个值，那么就会直接抛出异常给客户端。默认值为 0，含义是不排队，查询进入队列后立即返回失败。

如果集群中目前有 1 台 FE，那么这个配置的含义为，集群中同时运行的查询数最大不超过 10 个，当最大并发已满时，新来的查询会排队，队列的长度不超过 20。查询在队列中排队的时间最长为 3s，排队超过 3s 的查询会直接返回失败给客户端。

> 目前的排队设计是不感知 FE 的个数的，排队的参数只在单 FE 粒度生效，例如：
>
> 一个 Doris 集群配置了一个 workload group，设置 max_concurrency = 1；如果集群中有 1FE，那么这个 workload group 在 Doris 集群视角看同时只会运行一个 SQL；如果有 3 台 FE，那么在 Doris 集群视角看最大可运行的 SQL 个数为 3。

4.2.4.2 查看排队状态

语法

```
show workload groups
```

示例

```
mysql [(none)]>show workload groups`\G`;
*************************** 1. row ***************************
                           Id: 1
                         Name: normal
                    cpu_share: 20
                 memory_limit: 50%
    enable_memory_overcommit: true
```

```
             max_concurrency: 2147483647
              max_queue_size: 0
               queue_timeout: 0
              cpu_hard_limit: 1%
             scan_thread_num: 16
  max_remote_scan_thread_num: -1
  min_remote_scan_thread_num: -1
       memory_low_watermark: 50%
      memory_high_watermark: 80%
                        tag:
      read_bytes_per_second: -1
remote_read_bytes_per_second: -1
           running_query_num: 0
           waiting_query_num: 0
```

```
#### 绕开排队的逻辑

在有些运维情况下，管理员账户需要绕开排队的逻辑执行 SQL 对系统进行一些管理操作，那么可以通过设置
    ↪ session 变量，来绕开排队的逻辑：
```

set bypass_workload_group = true; “ ‘

4.2.5 查询熔断

查询熔断是一种保护机制，用于防止长时间运行或消耗过多资源的查询对系统产生负面影响。当查询超过预设的资源或时间限制时，熔断机制会自动终止该查询，以避免对系统性能、资源使用以及其他查询造成不利影响。这种机制确保了集群在多用户环境下的稳定性，防止单个查询导致系统资源耗尽、响应变慢，从而提高整体的可用性和效率。

在 Doris 内，有两种熔断策略：

- 规划时熔断，即 SQL Block Rule，用于阻止符合特定模式的语句执行。阻止规则对任意的语句生效，包括 DDL 和 DML。通常，阻止规则由数据库管理员（DBA）进行配置，用以提升集群的稳定性。比如，
  - 阻止一个查询扫描过多行的数据
  - 阻止一个查询扫描过多的分区
  - 阻止一个修改全局变量的语句，以防止集群配置被意外的修改。
  - 阻止一个通常会占用非常多资源的查询模式
- 运行时熔断，即 Workload Policy，它是在运行时，实时监测查询的执行时间，扫描的数据量，消耗的内存，实现基于规则的查询熔断。

4.2.5.1 SQL Block Rule

按照阻止模式，可以分为：

- 扫描行数阻止规则
- 扫描分区数阻止规则
- 扫描分桶数阻止规则
- 查询语句正则匹配阻止规则
- 查询语句哈希值匹配阻止规则

阻止规则按照阻止范围，可以分为：

- 全局级别阻止规则
- 用户级别阻止规则

4.2.5.1.1 使用方法

全局级别阻止规则

```
CREATE SQL_BLOCK_RULE rule_001
PROPERTIES (
  "sql"="select \\* from t",
  "global" = "true",
  "enable" = "true"
)
```

这样，我们就创建了一个全局级别的阻止规则。规则名为 rule_001。配置了查询语句正则匹配规则，用于阻止所有可以被正则 `select \\* from t` 所匹配的查询语句。

由于是全局级别的阻止规则，所以任意用户执行可以被上述正则匹配的语句都会被阻止。例如：

```
MySQL root@127.0.0.1:test> select * from t;
(1105, 'errCode = 2, detailMessage = errCode = 2, detailMessage = SQL match regex SQL block rule:
    ↪  rule_001')
```

用户级别阻止规则

```
CREATE SQL_BLOCK_RULE rule_001
PROPERTIES (
  "sql"="select * from t",
  "global" = "false",
  "enable" = "true"
)
```

不同于全局级别的阻止规则。用户级别的阻止规则只对指定用户生效。当我们创建阻止规则时，设置属性“global” 为 “false”。那么这个阻止规则，将被视为用户级别的阻止规则。

为了使得用户级别的阻止规则生效。还需要为需要使用此规则的用户设置相应的属性。例如：

```
set property for 'root' 'SQL_block_rules' = 'rule_001';
```

这样，经过上面的配置，root 用户在执行查询时，将被应用名为 rule_001 的阻止规则。

```
MySQL root@127.0.0.1:test> set property for 'root' 'SQL_block_rules' = '';
Query OK, 0 rows affected
Time: 0.018s
MySQL root@127.0.0.1:test> select * from t;
+----+----+
| id | c1 |
+----+----+
| 1  | 1  |
+----+----+

1 row in set
Time: 0.027s
MySQL root@127.0.0.1:test> set property for 'root' 'SQL_block_rules' = 'rule_001';
Query OK, 0 rows affected
Time: 0.008s
MySQL root@127.0.0.1:test> select * from t;
(1105, 'errCode = 2, detailMessage = errCode = 2, detailMessage = SQL match regex SQL block rule:
    ↪  rule_001')
```

- 如果想对一个用户添加多个用户级别的阻止规则。在规则列表中列举所有的规则名字，以英文逗号隔开。
- 如果想移除一个用户的所有用户级别阻止规则。将规则列表置为空字符串即可。

其他操作

如果需要修改或者删除阻止规则，可以参考阻止规则的 SQL 手册。

4.2.5.1.2 使用场景

可以在以下几种场景使用：

- 阻止扫描超过指定行数的数据
- 阻止扫描超过指定分区数量的数据
- 阻止扫描超过指定分桶数量的数据
- 阻止特定模式的查询

阻止扫描超过指定行数的数据

由于扫描数据会显著消耗 BE 的 IO 资源和 CPU 资源。所以，不必要的数据扫描会对集群的稳定性带来比较大的挑战。日常使用中，经常会出现盲目的全表扫描操作。例如 `SELECT * FROM t`。为了防止这种查询对集群产生破坏。可以设置单个查询扫描单表行数的上限。

```
CREATE SQL_BLOCK_RULE rule_card
PROPERTIES
(
   "cardinality" = "1000",
   "global" = "true",
   "enable" = "true"
);
```

当设置了如上的规则，当单表扫描超过 1000 行时，将禁止此查询的执行。需要注意，由于扫描行数的计算是在规划阶段，而非执行阶段完成的。所以计算行数时，只会考虑分区和分桶裁剪，而不会考虑其他过滤条件对于扫描行数的影响。也就是考虑最坏情况。所以，实际扫描的行数小于设置值的查询也有可能被阻止。

阻止扫描超过指定分区数量的数据

对过多分区的扫描会显著的增加 BE 的 CPU 消耗。同时，如果查询的外表，那更有可能带来显著的网络开销和元数据拉取的开销。在日常使用中，这多是由于忘记写分区列上的过滤条件或者写错导致的。为了防止这种查询对集群产生破坏。可以设置单个查询扫描单表的分区数的上限。

```
CREATE SQL_BLOCK_RULE rule_part_num
PROPERTIES
(
   "partition_num" = "30",
   "global" = "true",
   "enable" = "true"
);
```

当设置了如上的规则，当单表扫描分区数超过 30 个时，则禁止此查询的执行。需要注意的是，由于扫描分区数的计算是在规划阶段，而非执行阶段完成的。所以有可能出现因为分区裁剪不完全，而保留了更多分区的情况。所以，实际扫描的分区数小于设置值的查询也有可能被阻止。

阻止扫描超过指定分桶数量的数据

对过多分桶的扫描会显著的增加 BE 的 CPU 消耗。为了防止这种查询对集群产生破坏。可以设置单个查询扫描单表的分区数的上限。

```
CREATE SQL_BLOCK_RULE rule_teblet_num
PROPERTIES
(
   "tablet_num" = "200",
   "global" = "true",
   "enable" = "true"
);
```

当设置了如上的规则，当单表扫描分桶数量超过 200 个时，则禁止此查询的执行。需要注意的是，由于扫描分桶数的计算是在规划阶段，而非执行阶段完成的。所以有可能出现因为分桶裁剪不完全，而保留了更多分区的情况。所以，实际扫描的分桶数小于设置值的查询也有可能被阻止。

阻止特定模式的查询

由于各种原因，比如计算复杂度高，规划时间长等，可能会希望阻止使用模式的查询。以阻止函数 abs 举例。可以使用如下的正则表达式阻止规则，完成此目的。

```
CREATE SQL_BLOCK_RULE rule_abs
PROPERTIES(
  "sql"="(?i)abs\\s*\\(.+\\)",
  "global"="true",
  "enable"="true"
);
```

上述正则表达式中

- (?i) 表示大小写不敏感
- abs 为想要阻止的目标函数
- \s* 表示在 abs 和左括号之间可以有任意个空白
- \(.+\) 匹配函数参数同理，也可以使用类似的方法阻止 set global，以防止非预期的变量改变。或者阻止 truncate table，以防止非预期的删除数据。

4.2.5.1.3 常见问题

Q：正则匹配阻止规则会对集群产生副作用吗？

A：是的。由于正则匹配是计算密集型的。当使用复杂的正则表达式，或者正则匹配规则过多时。会给 FE 的 CPU 带来显著的压力提升。所以，要谨慎添加正则匹配的阻止规则。除非必要，尽量不要使用复杂的正则表达式。

Q：可以临时关闭一个阻止规则吗？

A：可以。修改阻止规则，将其属性中的 “enable” 改为 “false” 即可。

Q：阻止规则中的正则表达式使用哪种规范？

A：阻止规则的正则表达式使用 java 的正则表达式规范。常用表达式可以参考 SQL 语法手册。完整的手册可以参考 https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html

4.2.5.2 Workload Policy

SQL Block Rule 是一种在规划时进行熔断的配置，但是由于规划中代价的计算可能不准确（尤其是针对一些非常复杂的查询时，更加难以准确估算），所以会导致规则不生效或者误判。Workload Policy 弥补了这个缺陷，它可以在查询运行时对一些指标进行实时的监测，对运行时状态不符合预期的查询进行熔断，避免不符合预期的大查询占用过多资源从而影响集群的稳定性，常用的运行时监控指标如下：

- 查询执行时间
- 查询在单 BE 上的扫描行数
- 查询在单 BE 上的扫描行数扫描字节数
- 查询的在单 BE 上的内存使用

4.2.5.2.1 版本说明

自 Doris 2.1 版本起，可以通过 Workload Policy 可以实现大查询的熔断。

版本	自 2.1 起
select	支持
insert into select	支持
insert into values	不支持
stream load	支持
routine load	支持
backup	不支持
compaction	不支持

4.2.5.2.2 创建熔断策略

使用 CREATE WORKLOAD Policy 命令可以创建资源管理策略。

在下面的例子中创建一个名为 test_cancel_Policy 的 Policy，它会取消掉集群中运行时间超过 1000ms 的查询，当前状态为启用。创建 Workload Policy 需要 admin_priv 权限。

```
create workload Policy test_cancel_Policy
Conditions(query_time > 1000)
Actions(cancel_query)
properties('enabled'='true');
```

在创建 Workload Policy 时需要指定以下内容：- Condition 表示策略触发条件，可以多个 Condition 串联，使用逗号“,”隔开，表示“与”的关系。在上例中 query_time > 1000 表示在查询时间大于 1s 时触发 Policy；目前支持的 Conditions 有：

Conditions	说明
username	查询携带的用户名，只会在 FE 触发 set_session_variable Action
be_scan_rows	一个 SQL 在单个 BE 进程内 scan 的行数，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值。
be_scan_bytes	一个 SQL 在单个 BE 进程内 scan 的字节数，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值，单位是字节。
query_time	一个 SQL 在单个 BE 进程上的运行时间，时间单位是毫秒。
query_be_memory_bytes	一个 SQL 在单个 BE 进程内使用的内存用量，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值，单位是字节。

- Action 表示条件触发时采取的动作，目前一个 Policy 只能定义一个 Action（除 set_session_variable）。在上例中，cancel_query 表示取消查询；目前支持的 Actions 有：

Actions	说明
cancel_query	取消查询。
set_session_variable	触发 set session variable 语句。同一个 policy 可以有多个 set_session_variable 选项，目前只会在 FE 由 username Condition 触发

- Properties，定义了当前 Policy 的属性，包括是否启用和优先级。

Properties	说明
enabled	取值为 true 或 false，默认值为 true，表示当前 Policy 处于启用状态，false 表示当前 Policy 处于禁用状态
priority	取值范围为 0 到 100 的正整数，默认值为 0，代表 Policy 的优先级，该值越大，优先级越高。这个属性的主要作用是，当查询匹配到多个 Policy 时，只选择优先级最高的 Policy。
workload_group	目前一个 Policy 可以绑定一个 workload group，代表这个 Policy 只对某个 Workload Group 的查询生效。默认为空，代表对所有查询生效。

4.2.5.2.3 将熔断策略绑定 Workload Group

默认情况下，Workload Policy 会对所有支持的查询生效。如果想指定 Policy 只针对与某一个 Workload Group，需要通过 workload_group 选项绑定 Workload Group。语句如下：

```
create workload Policy test_cancel_big_query
Conditions(query_time > 1000)
Actions(cancel_query)
properties('workload_group'='normal')
```

4.2.5.2.4 注意事项

- 同一个 Policy 的 Condition 和 Action 要么都是 FE 的，要么都是 BE 的，比如 set_session_variable 和 cancel_query 无法配置到同一个 Policy 中。Condition be_scan_rows 和 Condition username 无法配置到同一个 Policy 中。
- 由于目前的 Policy 是异步线程以固定时间间隔执行的，因此策略的生效存在一定的滞后性。比如用户配置了 scan 行数大于 100 万就取消查询的策略，如果此时集群资源比较空闲，那么有可能在取消策略生效之前查询就已经结束了。目前这个时间间隔为 500ms，这意味着运行时间过短的查询可能会绕过策略的检查。
- 当前支持的负载类型包括 select/insert select/stream load/broker load/routine load。
- 一个查询可能匹配到多个 Policy，但是只有优先级最高的 Policy 会生效。
- 目前不支持 Action 和 Condition 的修改，只能通过删除新建的方式修改。

4.2.5.2.5 Workload Policy 效果演示

1 session 变量修改测试

尝试修改 Admin 账户的 session 变量中的并发相关的参数

```
// 登录 admin账户查看并发参数
mySQL [(none)]>show variables like '%parallel_fragment_exec_instance_num%';
+-----------------------------------+-------+---------------+---------+
| Variable_name                     | Value | Default_Value | Changed |
+-----------------------------------+-------+---------------+---------+
| parallel_fragment_exec_instance_num | 8     | 8             | 0       |
```

```
+-------------------------------------+-------+---------------+---------+
1 row in set (0.00 sec)

// 创建修改admin账户并发参数的Policy
create workload Policy test_set_var_Policy
Conditions(username='admin')
Actions(set_session_variable 'parallel_fragment_exec_instance_num=1')

// 过段时间后再次查看admin账户的参数
mySQL [(none)]>show variables like '%parallel_fragment_exec_instance_num%';
+-------------------------------------+-------+---------------+---------+
| Variable_name                       | Value | Default_Value | Changed |
+-------------------------------------+-------+---------------+---------+
| parallel_fragment_exec_instance_num | 1     | 8             | 1       |
+-------------------------------------+-------+---------------+---------+
1 row in set (0.01 sec)
```

2 大查询熔断测试

测试对运行时间超过 3s 的查询进行熔断，以下是一个 ckbench 的 q29 运行成功时的审计日志，可以看到这个 SQL 跑完需要 4.5s 的时间

```
mySQL [hits]>SELECT REGEXP_REPLACE(Referer, '^https?://(?:www\.)?([^/]+)/.*$', '\1') AS k, AVG(
    ↪ length(Referer)) AS l, COUNT(*) AS c, MIN(Referer) FROM hits WHERE Referer <> '' GROUP BY
    ↪  k HAVING COUNT(*) > 100000 ORDER BY l DESC LIMIT 25;
+--
    ↪ ------------------------------------------------------------------------+------------------+----------+-----
    ↪
| k                                                                      | l                | c
    ↪        | min(Referer)
    ↪
    ↪ |
+--
    ↪ ------------------------------------------------------------------------+------------------+----------+-----
    ↪
| 1                                                                      | 85.4611926713085 |
    ↪ 67259319 | http://%26ad%3D1%25EA%25D0%26utm_source=web&cd=19590&input_onlist/би-2 место
    ↪ будущей кондицин                        |
| http:%2F%2Fwwww.regnancies/search&evL8gE&where=all&filmId=bEmYZc_WTDE |               69 |
    ↪ 207347 | http:%2F%2Fwwww.regnancies/search&evL8gE&where=all&filmId=bEmYZc_WTDE
    ↪                                                 |
| http://новострашная                                                    |               31 |
    ↪ 740277 | http://новострашная
    ↪
    ↪ |
| http://loveche.html?ctid                                               |               24 |
```

```
    ↪ 144901 | http://loveche.html?ctid
    ↪
    ↪ |
| http://rukodeliveresult                                                 |               23 |
    ↪ 226135 | http://rukodeliveresult
    ↪
    ↪ |
| http://holodilnik.ru                                                    |               20 |
    ↪ 133893 | http://holodilnik.ru
    ↪
    ↪ |
| http://smeshariki.ru                                                    |               20 |
    ↪ 210736 | http://smeshariki.ru
    ↪
    ↪ |
| http:%2F%2Fviewtopic                                                    |               20 |
    ↪ 391115 | http:%2F%2Fviewtopic
    ↪
    ↪ |
| http:%2F%2Fwwww.ukr                                                     |               19 |
    ↪ 655178 | http:%2F%2Fwwww.ukr
    ↪
    ↪ |
| http:%2F%2FviewType                                                     |               19 |
    ↪ 148907 | http:%2F%2FviewType
    ↪
    ↪ |
| http://state=2008                                                       |               17 |
    ↪ 139630 | http://state=2008
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------+------------------+----------+-----
    ↪
11 rows in set (4.50 sec)
```

创建一个运行时间超过 3s 就取消查询的 Policy

```
create workload Policy test_cancel_3s_query
Conditions(query_time > 3000)
Actions(cancel_query)
```

再次执行 SQL 可以看到 SQL 执行会直接报错

```
mySQL [hits]>SELECT REGEXP_REPLACE(Referer, '^https?://(?:www\.)?([^/]+)/.*$', '\1') AS k, AVG(
    ↪ length(Referer)) AS l, COUNT(*) AS c, MIN(Referer) FROM hits WHERE Referer <> '' GROUP BY
    ↪  k HAVING COUNT(*) > 100000 ORDER BY l DESC LIMIT 25;
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (127.0.0.1)[CANCELLED]query cancelled by
    ↪ workload Policy,id:12345
```

4.2.6 终止查询

可以通过 KILL 命令取消当前正在执行的操作或断开当前连接会话。本文档介绍相关操作和注意事项

4.2.6.1 获取查询标识

KILL 需通过查询标识取消对应的查询请求。查询标识包括：查询 ID（Query ID）、连接 ID（Connection ID）和 Trace ID。

可以通过以下方式获取查询标识。

4.2.6.1.1 PROCESSLIST

通过 processlist 系统表可以获取当前所有的会话连接，以及连接中正在执行的查询操作。其中包含查询 ID 和连接 ID。

```
mysql> SHOW PROCESSLIST;
+--
    ↪ ----------------+------+------+--------------------+--------------------+----------+------+---------+--
    ↪
| CurrentConnected | Id   | User | Host                | LoginTime           | Catalog  | Db   |
    ↪ Command | Time | State | QueryId                            | Info             | FE
    ↪            | CloudCluster |
+--
    ↪ ----------------+------+------+--------------------+--------------------+----------+------+---------+--
    ↪
| No               |    2 | root | 172.20.32.136:54850 | 2025-05-11 10:41:52 | internal |      |
    ↪ Query   |    6 | OK    | 12ccf7f95c1c4d2c-b03fa9c652757c15 | select sleep(20) |
    ↪ 172.20.32.152 | NULL         |
| Yes              |    3 | root | 172.20.32.136:54862 | 2025-05-11 10:41:55 | internal |      |
    ↪ Query   |    0 | OK    | b710ed990d4144ee-8b15bb53002b7710 | show processlist |
    ↪ 172.20.32.152 | NULL         |
| No               |    1 | root | 172.20.32.136:47964 | 2025-05-11 10:41:54 | internal |      |
    ↪ Sleep   |   11 | EOF   | b60daa992bac4fe4-b29466aacce67d27 |                  |
    ↪ 172.20.32.153 | NULL         |
+--
    ↪ ----------------+------+------+--------------------+--------------------+----------+------+---------+--
    ↪
```

- CurrentConnected：Yes 表示当前会话所对应的连接。
- Id：连接的唯一标识，即 Connection ID。
- QueryId：Query 的唯一标识。显示最仅执行的、或正在执行的 SQL 命令的 Query Id。

注意，在默认情况下，SHOW PROCESSLIST 仅显示当前会话所连接到的 FE 节点上的所有会话连接，并不会显示其他 FE 节点的会话连接。

如果要显示所有 FE 节点的会话连接，需设置一下会话变量：

```
SET show_all_fe_connection=true;
```

然后再执行 SHOW PROCESSLIST 命令，即可显示所有 FE 节点的会话连接。

也可以通过 information_schema 中的系统表查看：

```
SELECT * FROM information_schema.processlist;
```

processlist 默认会显示所有 FE 节点的会话连接，不需要额外进行设置。

4.2.6.1.2 TRACE ID

> 该功能自 2.1.11 和 3.0.7 版本支持。

默认情况下，系统会为每个查询自动生成 Query ID。用户需先通过 processlist 系统表获取到 Query ID 后，再进行 KILL 操作。

除此之外，用户还可以自定义 Trace ID，并通过 Trace ID 进行 KILL 操作。

```
SET session_context = "trace_id:your_trace_id"
```

其中 your_trace_id 为用户自定义的 Trace ID。可以是任意字符串，但不可以包含 ; 符号。

Trace ID 是会话级别参数，仅作用于当前会话。Doris 会将之后发生在当前会话中的查询请求映射到这个 Trace ID 上。

4.2.6.2 Kill 请求

KILL 语句支持通过取消指定的查询操作，也支持断开指定的会话连接。

普通用户可以通过 KILL 操作取消掉自身用户发送的查询。ADMIN 用户可以取消自身和任意其他用户发送的查询。

4.2.6.2.1 Kill 查询

语法：

```
KILL QUERY "query_id" | "trace_id" | connection_id;
```

KILL QUERY 用于取消一个指定的正在运行的查询操作。

- "query_id"

  通过 processlist 系统表获取到的 Query ID。需用引号包裹。如：

  ```
  KILL QUERY "d36417cc05ff41ab-9d3afe49be251055";
  ```

  该操作会尝试在所有 FE 节点查找 Query ID 并取消对应的查询。

- "trace_id"

  通过 session_context 自定义的 Trace ID。需用引号包裹。如：

  ```
  KILL QUERY "your_trace_id";
  ```

  该操作会尝试在所有 FE 节点查找 Trace ID 并取消对应的查询。

> 该功能自 2.1.11 和 3.0.7 版本支持。

- connection_id

  通过 processlist 系统表获取到的 Connection ID。必须是一个大于 0 的整数，不可用引号包裹。如：

  ```
  KILL QUERY 55;
  ```

  该操作仅作用于当前连接到的 FE 上的会话连接，会取消对应会话连接上正在执行的查询。

4.2.6.2.2 Kill 连接

Kill 连接会断开指定的会话连接，同时也会取消连接上正在执行的查询操作。

语法：

```
KILL [CONNECTION] connection_id;
```

其中 CONNECTION 关键词可以省略。

- connection_id

  通过 processlist 系统表获取到的 Connection ID。必须是一个大于 0 的整数，不可用引号包裹。如：

```
KILL CONNECTION 55;
KILL 55;
```

不同 FE 上的连接 ID 可能相同，但该操作仅作用于当前连接到的 FE 上的会话连接。

4.2.6.3 最佳实践

1. 通过自定义 Trace ID 实现查询管理

   通过自定义 Trace ID 可以预先为查询指定唯一标识，方便管控工具实现【取消查询】的功能。可以通过如下方式自定义 Trace ID：

- 每次查询前设置 session_context
  用户自行生成 Trace ID。建议使用 UUID 以确保 Trace ID 的唯一性。

```
SET session_context="trace_id:your_trace_id";
SELECT * FROM table ...;
```

- 在查询语句中添加 Trace ID

```
SELECT /*+SET_VAR(session_context=trace_id:your_trace_id)*/ * FROM table ...;
```

之后，管控工具可以通过 Trace ID 随时取消正在执行的操作。

4.2.7 调度管理

4.2.7.1 背景

在数据管理愈加精细化的需求背景下，定时调度在其中扮演着重要的角色。它通常被应用于以下场景：

- 定期数据更新，如周期性数据导入和 ETL 操作，减少人工干预，提高数据处理的效率和准确性。
- 结合 Catalog 实现外部数据源数据定期同步，确保多源数据高效、准确的整合到目标系统中，满足复杂的业务分析需求。
- 定期清理过期/无效数据，释放存储空间，避免过多过期/无效数据对系统性能产生影响。

在 Apache Doris 之前版本中，通常需要依赖于外部调度系统，如通过业务代码定时调度或者引入第三方调度工具、分布式调度平台来满足上述需求。然而，因受限于外部系统自身能力，可能无法满足 Doris 对调度策略及资源管理灵活性的要求。此外，如果外部调度系统出现故障，这不仅会增加业务风险，还需投入额外的运维时间和人力来应对。

4.2.7.2 作业调度

为解决上述问题，Apache Doris 在 2.1 版本中引入了 Job Scheduler 功能，实现了自主任务调度能力，调度的精准度可达到秒级。该功能的推出不仅保障了数据导入的完整性和一致性，更让用户能够灵活、便捷调整调度策略。同时，因减少了对外部系统的依赖，也降低了系统故障的风险和运维成本，为社区用户带来更加统一、可靠的使用体验。

Doris Job Scheduler 是一种基于预设计划运行的任务管理系统，能够在特定时间点或按照指定时间间隔触发预定义操作，实现任务的自动化执行。Job Scheduler 具备以下特点：- 高效调度：Job Scheduler 可以在指定的时间间隔内安排任务和事件，确保数据处理的高效性。采用时间轮算法保证事件能够精准做到秒级触发。- 灵活调度：Job Scheduler 提供了多种调度选项，如按分、小时、天或周的间隔进行调度，同时支持一次性调度以及循环（周期）事件调度，并且周期调度也可以指定开始时间、结束时间。- 事件池和高性能处理队列：Job Scheduler 采用 Disruptor 实现高性能的生产消费者模型，最大可能的避免任务执行过载。- 调度记录可追溯：Job Scheduler 会存储最新的 Task 执行记录（可配置），通过简单的命令即可查看任务执行记录，确保过程可追溯。- 高可用：依托于 Doris 自身的高可用机制，Job Schedule 可以很轻松的做到自恢复、高可用。

相关文档： CREATE-JOB

4.2.7.3 语法说明

一条有效的 Job 语句需包含以下内容：- 关键字 CREATE JOB 需加作业名称，它在数据库中标识唯一事件。

- ON SCHEDULE 子句用于指定 Job 作业的类型、触发时间和频率。
  - AT timestamp 用于一次性事件。它指定 JOB 仅在给定的日期和时间执行一次，AT current_timestamp 指定当前日期和时间。因 JOB 一旦创建则会立即运行，也可用于异步任务创建。
  - EVERY：用于周期性作业，可指定作业的执行频率，关键字后需指定时间间隔（周、天、小时、分钟）。
    * Interval：用于指定作业执行频率。1 DAY 表示每天执行一次，1 HOUR 表示每小时执行一次，1 MINUTE 表示每分钟执行一次，1 WEEK 表示每周执行一次。
    * 子句 EVERY 包含可选 STARTS 子句。STARTS 后面为 timestamp 值，该值用于定义开始重复的时间，CURRENT_TIMESTAMP 用于指定当前日期和时间。JOB 一旦创建则会立即运行。
    * 子句 EVERY 包含可选 ENDS 子句。ENDS 关键字后面为 timestamp 值，该值定义 JOB 事件停止运行的时间。
- DO 子句用于指定 Job 作业触发时所需执行的操作，目前仅支持 Insert 语句。

```
CREATE
JOB
  job_name
  ON SCHEDULE schedule
  [COMMENT 'string']
  DO execute_sql;

schedule: {
    AT timestamp
    | EVERY interval
    [STARTS timestamp ]
    [ENDS timestamp ]
}
interval:
    quantity { WEEK |DAY | HOUR | MINUTE}
```

下方为简单的示例：

```
CREATE JOB my_job ON SCHEDULE EVERY 1 MINUTE DO INSERT INTO db1.tbl1 SELECT * FROM db2.tbl2;
```

该语句表示创建一个名为 my_job 的作业，每分钟执行一次，执行的操作是将 db2.tbl2 中的数据导入到 db1.tbl1 中。

4.2.7.4 使用示例

创建一次性的 Job：在 2025-01-01 00:00:00 时执行一次，将 db2.tbl2 中数据导入到 db1.tbl1 中。

```
CREATE JOB my_job ON SCHEDULE AT '2025-01-01 00:00:00' DO INSERT INTO db1.tbl1 SELECT * FROM db2.
    ↪ tbl2;
```

创建周期性的 Job，未指定结束时间：在 22025-01-01 00:00:00 时开始每天执行 1 次，将 db2.tbl2 中数据导入到 db1.tbl1 中。

```
CREATE JOB my_job ON SCHEDULE EVERY 1 DAY STARTS '2025-01-01 00:00:00' DO INSERT INTO db1.tbl1
    ↪ SELECT * FROM db2.tbl2 WHERE  create_time >=  days_add(now(),-1);
```

创建周期性的 Job，指定结束时间：在 2025-01-01 00:00:00 时开始每天执行 1 次，将 db2.tbl2 中的数据导入到 db1.tbl1 中，在 2026-01-01 00:10:00 时结束。

```
CREATE JOB my_job ON SCHEDULE EVERY 1 DAY STARTS '2025-01-01 00:00:00' ENDS '2026-01-01 00:10:00'
    ↪  DO INSERT INTO db1.tbl1 SELECT * FROM db2.tbl2 WHERE create_time >=  days_add(now(),-1);
```

借助 Job 实现异步执行：由于 Job 在 Doris 中是以同步任务的形式创建的，但其执行过程却是异步进行的，这一特性使得 Job 非常适合用于实现异步任务，例如常见的 insert into select 任务。

假设需要将 db2.tbl2 中的数据导入到 db1.tbl1 中，这里只需要指定 JOB 为一次性任务，且开始时间设置为当前时间即可。

```
CREATE JOB my_job ON SCHEDULE AT current_timestamp DO INSERT INTO db1.tbl1 SELECT * FROM db2.tbl2
    ↪ ;
```

4.2.7.5 基于 Catalog 与 Job Scheduler 的数据自动同步

以某电商场景为例，用户常常需要从 MySQL 中提取业务数据，并将这些数据同步到 Doris 中进行数据分析，从而支持精准的营销活动。而 Job Scheduler 可与数据湖能力 Multi Catalog 配合，高效完成跨数据源的定期数据同步。

```
CREATE TABLE IF NOT EXISTS user.activity (
    `user_id` INT NOT NULL,
    `date` DATE NOT NULL,
    `city` VARCHAR(20),
    `age` SMALLINT,
    `sex` TINYINT,
    `last_visit_date` DATETIME DEFAULT '1970-01-01 00:00:00',
    `cost` BIGINT DEFAULT '0',
    `max_dwell_time` INT DEFAULT '0',
    `min_dwell_time` INT DEFAULT '99999'
);
INSERT INTO user.activity VALUES
    (10000, '2017-10-01', '北京', 20, 0, '2017-10-01 06:00:00', 20, 10, 10),
    (10000, '2017-10-01', '北京', 20, 0, '2017-10-01 07:00:00', 15, 2, 2),
    (10001, '2017-10-01', '北京', 30, 1, '2017-10-01 17:05:00', 2, 22, 22),
    (10002, '2017-10-02', '上海', 20, 1, '2017-10-02 12:59:00', 200, 5, 5),
    (10003, '2017-10-02', '广州', 32, 0, '2017-10-02 11:20:00', 30, 11, 11),
```

```
    (10004, '2017-10-01', '深圳', 35, 0, '2017-10-01 10:00:00', 100, 3, 3),
    (10004, '2017-10-03', '深圳', 35, 0, '2017-10-03 10:20:00', 11, 6, 6);
```

user_id	date	city	age	sex	last_visit_date	cost	max_dwell_time	min_dwell_time
10000	2017/10/1	北京	20	0	2017/10/1 6:00	20	10	10
10000	2017/10/1	北京	20	0	2017/10/1 7:00	15	2	2
10001	2017/10/1	北京	30	1	2017/10/1 17:05	2	22	22
10002	2017/10/2	上海	20	1	2017/10/2 12:59	200	5	5
10003	2017/10/2	广州	32	0	2017/10/2 11:20	30	11	11
10004	2017/10/1	深圳	35	0	2017/10/1 10:00	100	3	3
10004	2017/10/3	深圳	35	0	2017/10/3 10:20	11	6	6

以上表为例，用户希望查询符合总消费金额、最后一次访问时间、性别、所在城市这几个数值条件的用户，并将满足条件的用户信息导入到 Doris 中，以便后续的定向推送。

1. 首先，创建一张 Doris 表

```
    CREATE TABLE IF NOT EXISTS user_activity
      (
      `user_id` LARGEINT NOT NULL COMMENT "用户 id",
      `date` DATE NOT NULL COMMENT "数据灌入日期时间",
      `city` VARCHAR(20) COMMENT "用户所在城市",
      `age` SMALLINT COMMENT "用户年龄",
      `sex` TINYINT COMMENT "用户性别",
      `last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "
          ↪ 用户最后一次访问时间",
      `cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
      `max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
      `min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
      )
      AGGREGATE KEY(`user_id`, `date`, `city`, `age`, `sex`)
      DISTRIBUTED BY HASH(`user_id`) BUCKETS 1
      PROPERTIES (
      "replication_allocation" = "tag.location.default: 1"
      );
```

2. 其次，创建对应 MySQL 库的 Catalog

```
    CREATE CATALOG activity PROPERTIES (
      "type"="jdbc",
      "user"="root",
      "password"="123456",
      "jdbc_url" = "jdbc:mysql://127.0.0.1:3306/user?useSSL=false",
```

```
    "driver_url" = "mysql-connector-java-5.1.49.jar",
    "driver_class" = "com.mysql.jdbc.Driver"
    );
```

3. 最后，将 MySQL 数据导入到 Doris 中。采用 Catalog + Insert Into 的方式来导入全量数据，由于全量导入操作可能会引发系统服务波动，通常选择在业务闲暇时进行操作。

- 一次性调度：如下方代码所示，使用一次性任务来定时触发全量导入任务，触发时间为凌晨 3:00。

```
CREATE JOB one_time_load_job
  ON SCHEDULE
  AT '2024-8-10 03:00:00'
  DO
  INSERT INTO user_activity SELECT * FROM activity.user.activity
```

- 周期调度：用户也可以创建一个周期性的调度任务，定期更新最新的数据。

```
CREATE JOB schedule_load
  ON SCHEDULE EVERY 1 DAY
  DO
  INSERT INTO user_activity SELECT * FROM activity.user.activity where last_visit_date >=
      ↪ days_add(now(),-1)
```

4.2.7.6 设计与实现

高效的调度通常伴随着大量的资源消耗，高精度的调度更是如此。传统的实现方式是直接使用 Java 内置的定时调度能力 ——定时调度线程周期访问，或采用一些定时调度的工具类库，但其在精度以及内存占用上存在较大的问题。为更好保障性能的前提下降低资源的占用，我们选择 TimingWheel 算法与 Disruptor 结合，实现秒级别的任务调度。

具体来说，利用 Netty 的 HashedWheelTimer 实现时间轮算法，Job Manager 会周期性（默认十分钟）地将未来事件放入时间轮中调度。为了保证任务高效触发并避免资源过度占用，采用 Disruptor 构建单生产者多消费者模型。时间轮仅负责触发，并不直接执行任务。对于到期需触发的任务时，会将其放入 Diapatch 线程，由其负责将任务分发至相应的执行线程池，对于需立即执行的任务，则直接将其投递至相应的任务执行线程池中。

对于单次执行事件，将在调度完成后删除事件定义；对于周期性事件，时间轮中的系统事件将定期拉取下一个周期的执行任务。这样可以避免大量任务集中在一个 Bucket 中，减少无意义的遍历、提高处理效率。

而对于事务型任务，Job Scheduler 能够通过与事务的强关联以及事务回调机制，确保事务型任务的执行结果与预期一致，从而保证数据的完整性和一致性。

4.2.7.7 未来规划

Doris Job Scheduler 是一款强大且灵活的任务调度工具，是数据处理中必不可少的功能之一。除了在数据湖分析、内部 ETL 等常见场景的应用外，Job Scheduler 对于异步物化视图的实现也起到关键的作用。异步物化视图

是一个预先计算并存储的结果集，其数据更新的频率与源表的变动紧密相关。当源表数据更新频繁时，为确保物化视图中数据保持最新状态，就需要对物化视图定期刷新。因此在 2.1 版本中，我们巧妙地利用 JOB 定时调度功能，保障了物化视图与源表数据的一致性，大幅降低了人工干预的成本。未来，Doris Job Scheduler 还会支持以下特性：- 支持通过 UI 界面查看不同时段执行的任务分布情况。- 支持 JOB 流程编排，即 DAG JOB。这意味着我们可以在内部实现数仓任务编排，与 Catalog 功能叠加将会更高效地完成数据处理和分析工作。- 支持对导入任务、UPDATE、 DELETE 操作进行定时调度。

4.3 容灾管理

4.3.1 容灾管理概览

Doris 提供了容灾管理能力，通过跨集群数据同步、备份与恢复和回收站恢复三大功能，帮助用户有效应对硬件故障、软件错误或人为失误导致的数据丢失问题，确保数据的高可用性和可靠性。

4.3.1.1 1. 跨集群数据同步

Doris 的跨集群数据同步功能支持在不同的 Doris 集群间进行数据的实时复制，确保重要数据分布在多个物理隔离的集群中，实现地域级容灾。

4.3.1.1.1 主要特性：

- 实时同步：支持全量和增量同步。全量同步在初始阶段复制所有数据；增量同步持续捕获和同步数据变更，包括数据（新增、修改、删除）和表结构变更（DDL）。
- 数据一致性：通过日志机制（如 Binlog）记录数据变更，确保目标集群与源集群数据完全一致。
- 地域级容灾：支持不同地理位置集群间的同步，当一个集群发生故障时，其他集群可以快速接管业务。
- 多场景应用：适用于容灾备份、业务分离（如读写分离）、多活集群等场景。

4.3.1.1.2 应用场景示例：

某公司在不同城市部署了两个 Doris 集群，A 集群为主集群，B 集群为备份集群。通过跨集群数据同步，当 A 集群因自然灾害中断服务时，B 集群可接管业务，最大限度减少停机时间。

4.3.1.2 2. 备份与恢复

Doris 提供了备份与恢复功能，用于定期保存数据快照，防止因意外事件导致的数据丢失。

4.3.1.2.1 主要特性：

- 备份：支持对指定数据库、表或者分区进行全量备份，保存完整数据快照。
- 恢复：支持从快照中恢复库、表或者分区。

4.3.1.2.2 应用场景示例：

某公司定期对数据进行备份，并将备份文件存储在对象存储服务（如 Amazon S3）中。当误操作导致某张重要表被删除时，利用备份功能快速恢复丢失数据，确保业务正常运行。

4.3.1.3 3. 回收站恢复

Doris 提供了回收站功能，为用户提供了一种快速恢复最近删除数据的方法，减少因操作失误带来的影响。

4.3.1.3.1 主要特性：

- 临时删除：表或数据库被删除后会先移动到回收站，而不是立即永久删除。
- 保留期：删除的数据在回收站中保留一段可配置的时间，用户可在此期间选择恢复。
- 快速恢复：无需完整备份恢复，即可轻松从回收站找回误删的数据。
- 数据安全：如果不需要恢复，回收站中的数据将在保留期后自动清理。

4.3.1.3.2 应用场景示例：

某团队在例行操作中误删除了一张重要表，通过回收站功能，他们快速恢复了被删除的数据，避免了复杂的备份恢复流程，同时确保了业务的连续性。

4.3.2 备份与恢复

4.3.2.1 备份和恢复概述

4.3.2.1.1 介绍

Doris 提供了备份和恢复操作支持。这些功能使用户能够将数据从库、表或者分区备份到远程存储系统，并在需要时进行恢复。

4.3.2.1.2 要求

- 管理员权限：只有具有 ADMIN 权限的用户才能执行备份和恢复操作。

4.3.2.1.3 关键概念

快照：快照是数据库、表或分区中数据的时间点捕获，创建快照时需要指定一个快照 Label，快照完成时会生成一个时间戳，可以通过 Repository、快照 Label 和时间戳标识一个快照。

Repository：备份文件存储的远程存储位置，支持的远程存储包括 S3、Azure、GCP、OSS、COS、MinIO、HDFS 和其它兼容 S3 的对象存储。

备份操作：备份操作涉及创建数据库、表或分区的快照，将快照文件上传到远程 Repository，并存储与备份相关的元数据。

恢复操作：恢复操作涉及从远程 Repository 中备份并将其恢复到 Doris 集群。

4.3.2.1.4 关键特性

1. 备份数据：Doris 允许您通过创建快照来备份表、分区或整个数据库的数据。数据以文件格式备份并存储在 HDFS、S3 或其他兼容 S3 的远程存储系统上。

2. 恢复数据：您可以从远程 Repository 恢复备份数据到任何 Doris 集群。这包括完整数据库恢复、完整表恢复和分区级恢复，允许灵活的数据恢复。
3. 快照管理：数据以快照的形式备份。这些快照被上传到远程存储系统，并可以在需要时恢复。恢复过程涉及下载快照文件并将其映射到本地元数据以使其有效。
4. 数据迁移：除了备份和恢复，此功能还支持在不同 Doris 集群之间的数据迁移。您可以将数据备份到远程存储系统并恢复到另一个 Doris 集群，帮助进行集群迁移场景。
5. 复制控制：在恢复数据时，您可以指定恢复数据的副本数量，以确保冗余和容错。

4.3.2.1.5 限制

1. 存储与计算解耦：存算分离模式不支持备份和恢复。
2. 不支持异步物化视图 (MTMV)：不支持备份或恢复 异步物化视图 (MTMV)。在备份和恢复操作中，这些视图不被考虑。
3. 不支持具有存储策略的表：使用了存储策略 的表 不支持备份和恢复操作。
4. 增量备份：目前，Doris 仅支持全量备份。增量备份（仅存储自上次备份以来更改的数据）尚不支持，您可以可以备份特定分区来实现增量备份。
5. colocate_with 属性：在备份或恢复操作期间，Doris 不会保留表的 `colocate_with` 属性。这可能需要在恢复后重新配置共置表。
6. 动态分区支持：恢复表之后，需要使用 `ALTER TABLE` 命令手动启用此属性。
7. 单并发：一个数据库下同时只能运行一个备份或者恢复任务。

4.3.2.2 备份

有关备份的概念，请参阅备份与恢复。本指南提供了创建 Repository 和备份数据的操作步骤。

4.3.2.2.1 第 1 步。创建 Repository

根据您的存储选择适当的语句来创建 Repository。有关详细用法，请参阅创建 Repository。在不同集群使用相同路径的 Repository 进行备份时，请确保使用不同的 Label，以避免冲突造成数据错乱。

方法 1: 在 S3 上创建 Repository

要在 S3 存储上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `s3_repo`
WITH S3
ON LOCATION "s3://bucket_name/s3_repo"
PROPERTIES
(
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",
    "s3.region" = "us-east-1",
    "s3.access_key" = "ak",
```

```
    "s3.secret_key" = "sk"
);
```

- 将 bucket_name 替换为您的 S3 存储桶名称。
- 提供适当的 endpoint、 access key、 secret key 和 region 以进行 S3 设置。

方法 2: 在 Azure 上创建 Repository

自 Doris 3.1.3 开始支持

要在 Azure 存储上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `azure_repo`
WITH S3
ON LOCATION "s3://<container_name>/azure_repo"
PROPERTIES
(
    "azure.endpoint" = "https://<account_name>.blob.core.windows.net",
    "azure.account_name" = "<account_name>",
    "azure.account_key" = "<account_key>",
    "provider" = "AZURE"
);
```

- 将替换为您的 Azure 容器名称。
- 提供您的 Azure 存储帐户和密钥以进行身份验证。
- provider 必须为 AZURE。

方法 3: 在 GCP 上创建 Repository

要在 Google Cloud Platform (GCP) 存储上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `gcp_repo`
WITH S3
ON LOCATION "s3://bucket_name/backup/gcp_repo"
PROPERTIES
(
    "s3.endpoint" = "storage.googleapis.com",
    "s3.region" = "US-WEST2",
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk"
);
```

- 将 bucket_name 替换为您的 GCP 存储桶名称。
- 提供您的 GCP endpoint、 access key 和 secret key。
- s3.region 只是一个虚假的 region，任意指定一个即可，但是必须要指定。

方法 4: 在 OSS（阿里云对象存储服务）上创建 Repository

要在 OSS 上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `oss_repo`
WITH S3
ON LOCATION "s3://bucket_name/oss_repo"
PROPERTIES
(
    "s3.endpoint" = "oss.aliyuncs.com",
    "s3.region" = "cn-hangzhou",
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk"
);
```

- 将 bucket_name 替换为您的 OSS 存储桶名称。
- 提供您的 OSS endpoint、 region、 access key 和 secret key。

方法 5: 在 MinIO 上创建 Repository

要在 MinIO 存储上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `minio_repo`
WITH S3
ON LOCATION "s3://bucket_name/minio_repo"
PROPERTIES
(
    "s3.endpoint" = "yourminio.com",
    "s3.region" = "dummy-region",
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk",
    "use_path_style" = "true"
);
```

- 将 bucket_name 替换为您的 MinIO 存储桶名称。
- 提供您的 MinIO endpoint、 access key 和 secret key。
- s3.region 只是一个虚假的 region，任意指定一个即可，但是必须要指定。
- 如果您不启用 Virtual Host-style，则 ‘use_path_style’ 必须为 true。

方法 6: 在 HDFS 上创建 Repository

要在 HDFS 存储上创建 Repository，请使用以下 SQL 命令：

```
CREATE REPOSITORY `hdfs_repo`
WITH hdfs
ON LOCATION "/prefix_path/hdfs_repo"
PROPERTIES
```

```
(
    "fs.defaultFS" = "hdfs://127.0.0.1:9000",
    "hadoop.username" = "doris-test"
)
```

- 将 prefix_path 替换为真实路径。
- 提供您的 hdfs endpoint 和用户名。

4.3.2.2.2 第 2 步。备份

请参考以下语句以备份数据库、表或分区。有关详细用法，请参阅备份。

建议使用有意义的 Label 名称，例如包含备份中包含的数据库和表。

方法 1: 备份当前数据库

以下 SQL 语句将当前数据库备份到名为 example_repo 的 Repository，并使用快照 Label exampledb_20241225。

```
BACKUP SNAPSHOT exampledb_20241225
TO example_repo;
```

方法 2: 备份指定数据库

以下 SQL 语句将名为 destdb 的数据库备份到名为 example_repo 的 Repository，并使用快照 Label destdb_20241225。

```
BACKUP SNAPSHOT destdb.`destdb_20241225`
TO example_repo;
```

方法 3: 备份指定表

以下 SQL 语句将两个表备份到名为 example_repo 的 Repository，并使用快照 Label exampledb_tbl_tbl1_20241225。

```
BACKUP SNAPSHOT exampledb_tbl_tbl1_20241225
TO example_repo
ON (example_tbl, example_tbl1);
```

方法 4: 备份指定分区

以下 SQL 语句将名为 example_tbl2 的表和名为 p1 和 p2 的两个分区备份到名为 example_repo 的 Repository，并使用快照 Label example_tbl_p1_p2_tbl1_20241225。

```
BACKUP SNAPSHOT example_tbl_p1_p2_tbl1_20241225
TO example_repo
ON
(
      example_tbl PARTITION (p1,p2),
      example_tbl2
);
```

方法 5: 备份当前数据库，排除某些表

以下 SQL 语句将当前数据库备份到名为 example_repo 的 Repository，并使用快照 Label exampledb_20241225，排除两个名为 example_tbl 和 example_tbl1 的表。

```
BACKUP SNAPSHOT exampledb_20241225
TO example_repo
EXCLUDE
(
      example_tbl,
      example_tbl1
);
```

4.3.2.2.3 第 3 步。查看最近备份作业的执行情况

以下 SQL 语句可用于查看最近备份作业的执行情况。

```
mysql> show BACKUP`\G`;
*************************** 1. row ***************************
                  JobId: 17891847
           SnapshotName: exampledb_20241225
                 DbName: example_db
                  State: FINISHED
             BackupObjs: [example_db.example_tbl]
             CreateTime: 2022-04-08 15:52:29
   SnapshotFinishedTime: 2022-04-08 15:52:32
     UploadFinishedTime: 2022-04-08 15:52:38
           FinishedTime: 2022-04-08 15:52:44
        UnfinishedTasks:
               Progress:
             TaskErrMsg:
                 Status: [OK]
                Timeout: 86400
   1 row in set (0.01 sec)
```

4.3.2.2.4 第 4 步。查看 Repository 中的现有备份

以下 SQL 语句可用于查看名为 example_repo 的 Repository 中的现有备份，其中 Snapshot 列是快照 Label，Timestamp 是时间戳。

```
mysql> SHOW SNAPSHOT ON example_repo;
+-----------------+---------------------+--------+
| Snapshot        | Timestamp           | Status |
+-----------------+---------------------+--------+
| exampledb_20241225 | 2022-04-08-15-52-29 | OK     |
+-----------------+---------------------+--------+
1 row in set (0.15 sec)
```

4.3.2.2.5 第 5 步。取消备份（如有需要）

可以使用 `CANCEL BACKUP FROM db_name;` 取消一个数据库中的备份任务。更具体的用法可以参考取消备份。

4.3.2.3 恢复

4.3.2.3.1 前提条件

1. 确保您拥有管理员权限以执行恢复操作。
2. 确保您有一个有效的备份快照可供恢复，请参考备份。

4.3.2.3.2 1. 获取快照的备份时间戳

以下 SQL 语句可用于查看名为`example_repo`的 Repository 中的现有备份。

```
mysql> SHOW SNAPSHOT ON example_repo;
+-----------------+---------------------+--------+
| Snapshot            | Timestamp              | Status    |
+-----------------+---------------------+--------+
| exampledb_20241225 | 2022-04-08-15-52-29 | OK     |
+-----------------+---------------------+--------+
1 row in set (0.15 sec)
```

4.3.2.3.3 2. 从快照恢复

Option 1：恢复快照到当前数据库

以下 SQL 语句从名为`example_repo`的 Repository 中恢复标签为 `restore_label1` 和时间戳为 `2022-04-08-15-52-29` 的快照到当前数据库。

```
RESTORE SNAPSHOT `restore_label1`
FROM `example_repo`
PROPERTIES
(
    "backup_timestamp"="2022-04-08-15-52-29"
);
```

Option 2：恢复快照到指定数据库

以下 SQL 语句从名为`example_repo`的 Repository 中恢复标签为 `restore_label1` 和时间戳为 `2022-04-08-15-52-29` 的快照到名为 `destdb` 的数据库。

```
RESTORE SNAPSHOT destdb.`restore_label1`
FROM `example_repo`
PROPERTIES
(
    "backup_timestamp"="2022-04-08-15-52-29"
);
```

Option 3：从快照恢复单个表

从example_repo中的快照恢复表backup_tbl到当前数据库，快照的标签为 restore_label1，时间戳为 2022-04-08-15-52-29。

```
RESTORE SNAPSHOT `restore_label1`
FROM `example_repo`
ON ( `backup_tbl` )
PROPERTIES
(
    "backup_timestamp"="2022-04-08-15-52-29"
);
```

Option 4：从快照恢复分区和表

从example_repo中的备份快照snapshot_2恢复表backup_tbl的分区 p1 和 p2，以及表backup_tbl2到当前数据库example_db1，并将其重命名为new_tbl，快照标签为时间版本为"2018-05-04-17-11-01"。

```
RESTORE SNAPSHOT `restore_label1`
FROM `example_repo`
ON
(
    `backup_tbl` PARTITION (`p1`, `p2`),
    `backup_tbl2` AS `new_tbl`
)
PROPERTIES
(
    "backup_timestamp"="2022-04-08-15-55-43"
);
```

4.3.2.3.4 3. 查看恢复作业的执行情况

```
mysql> SHOW RESTORE`\G`;
*************************** 1. row ***************************
              JobId: 17891851
              Label: snapshot_label1
          Timestamp: 2022-04-08-15-52-29
             DbName: default_cluster:example_db1
              State: FINISHED
          AllowLoad: false
    ReplicationNum: 3
        RestoreObjs: {
  "name": "snapshot_label1",
  "database": "example_db",
  "backup_time": 1649404349050,
  "content": "ALL",
  "olap_table_list": [
```

```
      {
        "name": "backup_tbl",
        "partition_names": [
          "p1",
          "p2"
        ]
      }
    ],
    "view_list": [],
    "odbc_table_list": [],
    "odbc_resource_list": []
}
           CreateTime: 2022-04-08 15:59:01
     MetaPreparedTime: 2022-04-08 15:59:02
SnapshotFinishedTime: 2022-04-08 15:59:05
DownloadFinishedTime: 2022-04-08 15:59:12
         FinishedTime: 2022-04-08 15:59:18
     UnfinishedTasks:
             Progress:
            TaskErrMsg:
                Status: [OK]
              Timeout: 86400
1 row in set (0.01 sec)
```

4.3.3 跨集群复制

4.3.3.1 跨集群数据同步概述

4.3.3.1.1 概览

CCR (Cross Cluster Replication) 是一种跨集群数据同步机制，能够在库或表级别将源集群的数据变更同步到目标集群。

适用场景

CCR 适用于以下几种常见场景：

- 容灾备份：将企业数据备份到另一集群和机房，确保在业务中断或数据丢失时能够恢复数据。
- 读写分离：通过将数据的查询操作与写入操作分离，减小读写之间的相互影响，提升服务稳定性。对于高并发或写入压力大的场景，采用读写分离可以有效分散负载，提升数据库性能和稳定性。
- 数据集中：集团总部需统一管理和分析分布在不同地域的分公司数据，避免因数据不一致导致的管理混乱和决策错误，从而提升集团管理效率和决策质量。
- 隔离升级：在进行系统集群升级时，使用 CCR 可以在新集群中进行验证和测试，避免因版本兼容问题导致的回滚困难。用户可以逐步升级各个集群，同时保证数据一致性。

- 集群迁移：在进行 Doris 集群的机房搬迁或设备更换时，使用 CCR 可以将老集群的数据同步到新集群，确保迁移过程中的数据一致性。

任务类别

CCR 支持两种任务类型：

- 库级任务：同步整个数据库的数据。
- 表级任务：仅同步指定表的数据。注意，表级同步不支持重命名表或替换表操作。Doris 每个数据库只能同时运行一个快照任务，因此表级同步的全量同步任务需要排队执行。

4.3.3.1.2 原理与架构

名词解释

- 源集群：数据源所在的集群，通常为业务数据写入的集群。
- 目标集群：跨集群同步的目标集群。
- binlog：源集群的变更日志，包含了 schema 和数据变更。
- Syncer：一个轻量级的进程，负责同步数据。
- 上游：在库级任务中指上游库，在表级任务中指上游表。
- 下游：在库级任务中指下游库，在表级任务中指下游表。

架构说明

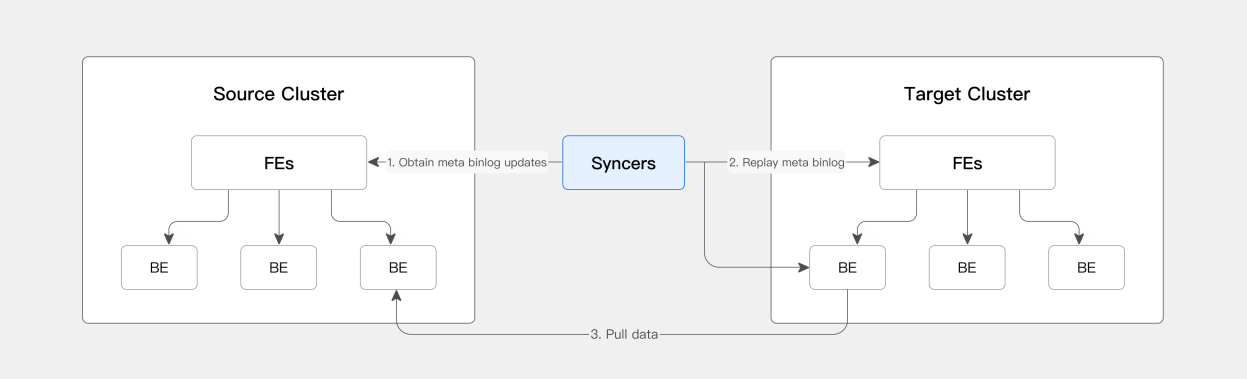


图 138: CCR 架构说明

CCR 主要依赖一个轻量级进程：Syncer。Syncer 负责从源集群获取 binlog，并将元数据应用到目标集群，通知目标集群从源集群拉取数据，从而实现全量同步和增量同步。

原理

1. 全量同步：

- CCR 任务会首先进行全量同步，将上游数据一次性完整地复制到下游。

2. 增量同步：

- 在全量同步完成后，CCR 任务会继续进行增量同步，保持上游和下游数据的一致性。

3. 重新开始全量同步的情况：

- 遇到当前不支持增量同步的 DDL 操作时，CCR 任务会重新启动全量同步。具体哪些 DDL 操作不支持增量同步，请参见功能详情。
- 如果上游的 binlog 因为过期或其他原因中断，增量同步会停止，并重新开始全量同步。

4. 重新全量同步：

- 在全量同步进行期间，增量同步会暂停。
- 全量同步完成后，下游的数据表会进行原子替换，以确保数据一致性。
- 全量同步完成后，会恢复增量同步。

同步方式

CCR 支持四种同步方式：

同步方式	原理	触发时机
Full Sync	上游进行全量备份，下游进行恢复。DB 级任务触发 DB 备份，表级任务触发表备份。	首次同步或特定操作触发。
Partial Sync	上游表或分区级别备份，下游表或分区级别恢复。	特定操作触发，触发条件请
TXN	增量数据同步，上游提交后，下游开始同步。	特定操作触发，触发条件请
SQL	在下游回放上游操作的 SQL。	特定操作触发，触发条件请

4.3.3.1.3 下载

要求：glibc >= 2.28

版本	架构	包地址	SHA256
2.1	ARM64	ccr-syncer-2.1.10-rc02-arm64.tar.xz	8ee7bd72baae87f2f226a5effbe4b10a8fac4b1fa25c9cc
2.1	X64	ccr-syncer-2.1.10-rc02-x64.tar.xz	3c36dd4a807931b00b191c3d9f3541f2e5732acb6d1d9
3.0	ARM64	ccr-syncer-3.0.6-rc02-arm64.tar.xz	7225cf8bc2acc37c09712a3655b0ff1939d574f42ed7bb
3.0	X64	ccr-syncer-3.0.6-rc02-x64.tar.xz	9fdafb90fdb337e3842dc690be40c77d64c1c1ea3b7b68

4.3.3.2 快速开始

4.3.3.2.1 1. 打开源和目标集群的 binlog 配置

在源集群和目标集群的 fe.conf 和 be.conf 中配置如下信息：

```
enable_feature_binlog=true
```

4.3.3.2.2 2. 部署 Syncer

2.1. 从如下链接下载最新的包

```
https://apache-doris-releases.oss-accelerate.aliyuncs.com/ccr-release/ccr-syncer-3.0.4-rc02-x64.
↪ tar.xz
```

2.2. 启动和停止 Syncer

```

cd bin && sh start_syncer.sh --daemon

#### 停止
sh stop_syncer.sh
```

4.3.3.2.3 3. 打开源集群中同步库/表的 Binlog

```
-- 如果是整库同步，可以执行如下脚本，使得该库下面所有的表都要打开 binlog.enable
./enable_db_binlog.sh --host $host --port $port --user $user --password $password --db $db

-- 如果是单表同步，则只需要打开 table 的 binlog.enable，在源集群上执行：
ALTER TABLE enable_binlog SET ("binlog.enable" = "true");
```

4.3.3.2.4 4. 向 Syncer 发起同步任务

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "ccr_test",
    "src": {
      "host": "localhost",
      "port": "9030",
      "thrift_port": "9020",
      "user": "root",
      "password": "",
      "database": "your_db_name",
      "table": "your_table_name"
    },
    "dest": {
      "host": "localhost",
      "port": "9030",
      "thrift_port": "9020",
      "user": "root",
      "password": "",
      "database": "your_db_name",
      "table": "your_table_name"
    }
}' http://127.0.0.1:9190/create_ccr
```

同步任务的参数说明：

name：CCR同步任务的名称，唯一即可 host、port：对应集群 Master FE的host和mysql(jdbc) 的端口 user、password：syncer以何种身份去开启事务、拉取数据等 database、table： 如果是库级别的同步，则填入your_db_name，your_table_name为空 如果是表级别同步，则需要填入your_db_name，your_table_name 向syncer发起同步任务中的name只能使用一次

4.3.3.3 操作手册

4.3.3.3.1 使用要求

网络要求

- 需要 Syncer 与上下游的 FE 和 BE 是互通的
- 下游 BE 与上游 BE 通过 Doris BE 进程使用的 IP（ show frontends/backends 看到的）是直通的。

权限要求

Syncer 同步时需要用户提供上下游的账户，该账户需要拥有下述权限：

- Select_priv 对数据库、表的只读权限。
- Load_priv 对数据库、表的写权限。包括 Load、 Insert、 Delete 等。
- Alter_priv 对数据库、表的更改权限。包括重命名库/表、添加/删除/变更列、添加/删除分区等操作。
- Create_priv 创建数据库、表、视图的权限。
- Drop_priv 删除数据库、表、视图的权限。
- Admin 权限 (之后考虑移除), 这个是用来检测 enable binlog config 的。

版本要求

- Syncer 版本 >= 下游 Doris 版本 >= 上游 Doris 版本。因此，首先升级 Syncer，然后升级下游 Doris，最后升级上游 Doris。
- Doris 2.0 的最低版本为 2.0.15，Doris 2.1 的最低版本为 2.1.6。
- 从 Syncer 版本 2.1.8 和 3.0.4 开始，Syncer 不再支持 Doris 2.0。

配置和属性要求

属性要求 - light_schema_change：Syncer 要求上下游表都设置 light_schema_table 属性，否则会导致数据同步出错。注意：最新版本的 doris 在建表时会默认设置上 light_schema_change 属性。如果使用 1.1 及之前的版本的 doris 或者升级上来的，需要在开启 Syncer 同步前，给存量 OLAP 表都设置上 light_schema_change 属性。

配置要求 - restore_reset_index_id：如果要同步的表中带有 inverted index，那么必须在目标集群上配置为 false。- ignore_backup_tmp_partitions：如果上游有创建 tmp partition，那么 doris 会禁止做 backup，因此 Syncer 同步会中断；通过在 FE 设置 ignore_backup_tmp_partitions=true 可以避免这个问题。

4.3.3.3.2 开启库中所有表的 binlog

```
bash bin/enable_db_binlog.sh -h host -p port -u user -P password -d db
```

4.3.3.3.3 启动 Syncer

假设环境变量 ${SYNCER_HOME} 被设置为 Syncer 的工作目录。可以使用 bin/start_syncer.sh 启动 Syncer。

选项	描述	命令示例	默认值
--daemon	后台运行 Syncer	bin/start_syncer ↪ .sh --daemon	false
--db_type	Syncer 可使用两种数据库保存元数据：sqlite3（本地存储）和 mysql（本地或远端存储）。当使用 mysql 存储元数据时，Syncer 会使用 CREATE IF ↪ NOT ↪ EXISTS 创建名为 ccr 的库，元数据表保存在其中。	bin/start_syncer ↪ .sh --db_type ↪ mysql	sqlite3
--db_dir	仅在数据库使用 sqlite3 时生效，可指定 SQLite3 生成的数据库文件名及路径。	bin/start_syncer ↪ .sh --db_dir ↪ /path/to/ccr. ↪ db	SYNCER_HOME/ ↪ db/ccr.db
--db_host-- ↪ db_port ↪ --db_ ↪ user--db ↪ _ ↪ password	仅在数据库使用 mysql 时生效，用于设置 MySQL 的主机、端口、用户和密码。	bin/start_syncer ↪ .sh --db_host ↪ 127.0.0.1 -- ↪ db_port 3306 ↪ --db_user ↪ root --db_ ↪ password " ↪ qwe123456"	db_host 和 db_port 默认为示例值；db_user 和 db_password 默认为空。

选项	描述	命令示例	默认值
--log_dir	指定日志输出路径	bin/start_syncer .sh --log_dir /path/to/ccr _syncer.log	SYNCER_HOME/ log/ccr_ syncer.log
--log_level	指定日志输出等级，日志格式如下：time level msg hooks。在 --daemon 下默认值为 info；前台运行时默认值为 trace，并通过 tee 保存日志到 log_dir。	bin/start_syncer .sh --log_ level info	info（后台运行）trace（前台运行）
--host-- port	指定 Syncer 的 host 和 port。host 用于区分集群中 Syncer 的实例，可理解为 Syncer 的名称，集群中 Syncer 的名称格式为 host:port。	bin/start_syncer .sh --host 127.0.0.1 -- port 9190	host 默认为 127.0.0.1port 默认为 9190
--pid_dir	指定 PID 文件保存路径。PID 文件为 stop_syncer .sh 脚本停止 Syncer 的凭据，保存对应 Syncer 的进程号。为方便集群化管理，可自定义路径。	bin/start_syncer .sh --pid_dir /path/to/ pids	SYNCER_HOME/ bin

4.3.3.3.4 停止 Syncer

可以使用 `bin/stop_syncer.sh` 停止 Syncer，有三种方法：

方法/选项	描述	命令示例	默认值
方法 1 停止单个 Syncer	指定要停止的 Syncer 的 `host` 和 `port`，注意要与启动时的 `host` 一致。	`bash bin/stop_syncer.sh --host 127.0.0.1 --port 9190`	无
方法 2 批量停止 Syncer	指定要停止的 PID 文件名，以空格分隔并用 " 包裹。	`bash bin/stop_syncer.sh --files "127.0.0.1_9190.pid 127.0.0.1_9191.pid"`	无
方法 3 停止所有 Syncer	默认停止 `pid_dir` 路径下所有 PID 文件对应的 Syncer。	`bash bin/stop_syncer.sh --pid_dir /path/to/pids`	无

方法 3 的选项如下：

选项	描述	命令示例	默认值
`--pid_dir`	指定 PID 文件所在目录，上述三种停止方法都依赖于此选项执行。	`bash bin/stop_syncer.sh --pid_dir /path/to/pids`	`SYNCER_HOME/bin`
`--host--port`	停止 `pid_dir` 路径下 `host:port` 对应的 Syncer。仅指定 `host` 时退化为方法 3；`host` 和 `port` 都不为空时生效为方法 1。	`bash bin/stop_syncer.sh --host 127.0.0.1 --port 9190`	host: 127.0.0.1port: 空

选项	描述	命令示例	默认值
--files	停止 pid_dir 路径下指定 PID 文件名对应的 Syncer，文件之间用空格分隔，并整体用 " 包裹。	bash bin/stop_ ↪ syncer.sh -- ↪ files ↪ "127.0.0.1_ ↪ 9190.pid ↪ 127.0.0.1_ ↪ 9191.pid"	无

4.3.3.3.5 Syncer 操作列表

请求的通用模板

```
curl -X POST -H "Content-Type: application/json" -d {json_body} http://ccr_syncer_host:ccr_syncer
    ↪ _port/operator
```

json_body: 以 json 的格式发送操作所需信息

operator：对应 Syncer 的不同操作

所以接口返回都是 json, 如果成功则是其中 success 字段为 true, 类似，错误的时候，是 false，然后存在ErrMsgs字段

```
{"success":true}

or

{"success":false,"error_msg":"job ccr_test not exist"}
```

创建任务

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "ccr_test",
    "src": {
    "host": "localhost",
    "port": "9030",
    "thrift_port": "9020",
    "user": "root",
    "password": "",
    "database": "demo",
    "table": "example_tbl"
    },
    "dest": {
    "host": "localhost",
    "port": "9030",
    "thrift_port": "9020",
```

```
    "user": "root",
    "password": "",
    "database": "ccrt",
    "table": "copy"
    }
}' http://127.0.0.1:9190/create_ccr
```

- name: CCR 同步任务的名称，唯一即可
- host、 port：对应集群 master 的 host 和 mysql(jdbc) 的端口
- thrift_port：对应 FE 的 rpc_port
- user、 password：Syncer 以何种身份去开启事务、拉取数据等
- database、 table：
- 如果是库级别的同步，则填入 dbName，tableName 为空
- 如果是表级别同步，则需要填入 dbName、 tableName

查看同步进度

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/get_lag
```

job_name 是 create_ccr 时创建的 name。

暂停任务

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/pause
```

恢复任务

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/resume
```

删除任务

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/delete
```

获取版本

```
curl http://ccr_syncer_host:ccr_syncer_port/version



{"version": "2.0.1"}
```

查看任务状态

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/job_status

{
"success": true,
"status": {
    "name": "ccr_db_table_alias",
    "state": "running",
    "progress_state": "TableIncrementalSync"
}
}
```

结束同步

```
curl -X POST -H "Content-Type: application/json" -d '{
    "name": "job_name"
}' http://ccr_syncer_host:ccr_syncer_port/desync
```

获取任务列表

```
curl http://ccr_syncer_host:ccr_syncer_port/list_jobs

{"success":true,"jobs":["ccr_db_table_alias"]}
```

4.3.3.3.6 Syncer 高可用

Syncer 高可用依赖 mysql，如果使用 mysql 作为后端存储，Syncer 可以发现其它 Syncer，如果一个 crash 了，其他会分担它的任务。

4.3.3.3.7 Upgrade

1. 升级 Syncer

假设以下环境变量已设置：- ${SYNCER_HOME}：Syncer 的工作目录。- ${SYNCER_PACKAGE_DIR}：包含新 Syncer 的目录。

通过以下步骤升级每个 Syncer。

1.1. 保存启动命令

将以下命令的输出保存到文件中。

```
ps -elf | grep ccr_syncer
```

1.2. 停止当前 Syncer

```
sh bin/stop_syncer.sh --pid_dir ${SYNCER_HOME}/bin
```

1.3. 备份现有的 MetaService 二进制文件

```
mv ${SYNCER_HOME}/bin bin_backup_$(date +%Y%m%d_%H%M%S)
```

1.4. 部署新包

```
cp ${SYNCER_PACKAGE_DIR}/bin ${SYNCER_HOME}/bin
```

1.5. 启动新的 Syncer

使用在 1.1 中保存的命令启动新的 Syncer。

2. 升级下游 Doris（如有必要）

按照升级 Doris 指南中的说明升级上游系统。

3. 升级上游 Doris（如有必要）

按照升级 Doris 指南中的说明升级上游系统。

4.3.3.3.8 使用须知

> 在未出现异常时，`is_being_synced`属性应该完全由 Syncer 控制开启或关闭，用户不要自行修改该属性。

注意事项

- CCR 同步期间 backup/restore job 和 binlogs 都在 FE 内存中，因此建议在 FE 给每个 ccr job 都留出 4GB 及以上的堆内存（源和目标集群都需要），同时注意修改下列配置减少无关 job 对内存的消耗：
  - 修改 FE 配置 `max_backup_restore_job_num_per_db`: 记录在内存中的每个 DB 的 backup/restore job 数量。默认值是 10，设置为 2 就可以了。
  - 修改源集群 db/table property，设置 binlog 保留限制
    * `binlog.max_bytes`: binlog 最大占用内存，建议至少保留 4GB（默认无限制）
    * `binlog.ttl_seconds`: binlog 保留时间，从 2.0.5 之前的老版本默认无限制；之后的版本默认值为一天（86400）比如要修改 binlog ttl seconds 为保留一个小时：`ALTER TABLE table SET ("binlog ↪ .ttl_seconds"="3600")`
- CCR 正确性依也赖于目标集群的事务状态，因此要保证在同步过程中事务不会过快被回收，需要调大下列配置

  - label_num_threshold：用于控制 TXN Label 数量
  - stream_load_default_timeout_second：用于控制 TXN 超时时间
  - label_keep_max_second: 用于控制 TXN 结束后保留时间
  - streaming_label_keep_max_second：同上

- 如果是 db 同步且源集群的 tablet 数量较多，那么产生的 ccr job 可能非常大，需要修改几个 FE 的配置：

  - max_backup_tablets_per_job: 一次 backup 任务涉及的 tablet 上限，需要根据 tablet 数量调整（默认值为 30w，过多的 tablet 数量会有 FE OOM 风险，优先考虑能否降低 tablet 数量）
  - thrift_max_message_size: FE thrift server 允许的单次 RPC packet 上限，默认值为 100MB，如果 tablet 数量太多导致 snapshot info 大小超过 100MB，则需要调整该限制，最大 2GB
    * Snapshot info 大小可以从 ccr syncer 日志中找到，关键字：snapshot response meta size: %d, job info size: %d，snapshot info 大小大约是 meta size + job info size。
  - fe_thrift_max_pkg_bytes: 同上，一个额外的参数，2.0 中需要调整，默认值为 20MB
  - restore_download_task_num_per_be: 发送给每个 BE download task 数量上限，默认值是 3，对 restore job 来说太小了，需要调整为 0（也就是关闭这个限制）；2.1.8 和 3.0.4 起不再需要这个配置。
  - backup_upload_task_num_per_be: 发送给每个 BE upload task 数量上限，默认值是 3，对 backup job 来说太小了，需要调整为 0（也就是关闭这个限制）；2.1.8 和 3.0.4 起不再需要这个配置。
  - 除了上述 FE 的配置外，如果 ccr job 的 db type 是 mysql，还需要调整 mysql 的一些配置：
    * mysql 服务端会限制单次 select/insert 返回/插入数据包的大小。增加下列配置以放松该限制，比如调整到上限 1GB

      ```
      [mysqld]
      max_allowed_packet = 1024MB
      ```

    * mysql client 也会有该限制，在 ccr syncer 2.1.6/2.0.15 及之前的版本，上限为 128MB；之后的版本可以通过参数 --mysql_max_allowed_packet 调整（单位 bytes），默认值为 1024MB > 注：在 2.1.8 和 3.0.4 以后，ccr syncer 不再将 snapshot info 保存在 db 中，因此默认的数据包大小已经足够了。

- 同上，BE 端也需要修改几个配置

  - thrift_max_message_size: BE thrift server 允许的单次 RPC packet 上限，默认值为 100MB，如果 tablet 数量太多导致 agent task 大小超过 100MB，则需要调整该限制，最大 2GB
  - be_thrift_max_pkg_bytes：同上，只有 2.0 中需要调整的参数，默认值为 20MB

- 即使修改了上述配置，当 tablet 继续上升时，产生的 snapshot 大小可能会超过 2GB，也就是 doris FE edit log 和 RPC message size 的阈值，导致同步失败。从 2.1.8 和 3.0.4 开始，doris 可以通过压缩 snapshot 来进一步提高备份恢复支持的 tablet 数量。可以通过下面几个参数开启压缩：

  - restore_job_compressed_serialization: 开启对 restore job 的压缩（影响元数据兼容性，默认关闭）
  - backup_job_compressed_serialization: 开启对 backup job 的压缩（影响元数据兼容性，默认关闭）
  - enable_restore_snapshot_rpc_compression: 开启对 snapshot info 的压缩，主要影响 RPC（默认开启）> 注：由于识别 backup/restore job 是否压缩需要额外的代码，而 2.1.8 和 3.0.4 之前的代码中不包含相关代码，因此一旦有 backup/restore job 生成，那么就无法回退到更早的 doris 版本。有两种情况例外：已经 cancel 或者 finished 的 backup/restore job 不会被压缩，因此在回退前等待 backup/restore job 完成或者主动取消 job 后，就能安全回退。

- Ccr 内部会使用 db/table 名作为一些内部 job 的 label，因此如果 ccr job 中碰到了 label 超过限制了，可以调整 FE 参数 `label_regex_length` 来放松该限制（默认值为 128）
- 由于 backup 暂时不支持备份带有 cooldown tablet 的表，如果碰到了会导致同步终端，因此需要在创建 ccr job 前检查是否有 table 设置了 `storage_policy` 属性。###### 性能相关参数
- 如果用户的数据量非常大，备份、恢复执行完需要的时间可能会超过一天（默认值），那么需要按需调整下列参数
  - `backup_job_default_timeout_ms` 备份/恢复任务超时时间，源、目标集群的 FE 都需要配置
  - 上游修改 binlog 保留时间：`ALTER DATABASE $db SET PROPERTIES ("binlog.ttl_seconds" = "xxxx ↪ ")`
- 下游 BE 下载速度慢
  - `max_download_speed_kbps` 下游单个 BE 中单个下载线程的下载限速，默认值为 50MB/s
  - `download_worker_count` 下游执行下载任务的线程数，默认值为 1；需要结合客户机型调整，在不影响客户正常读写时跳到最大；如果调整了这个参数，就可以不用调整 `max_download_speed_kbps`。
    * 比如客户机器网卡最大提供 1GB 的带宽，现在最大允许下载线程利用 200MB 的带宽，那么在不改变 `max_download_speed_kbps` 的情况下，`download_worker_count` 应该配置成 4。
- 限制下游 BE 下载 binlog 速度 BE 端配置参数：

```
download_binlog_rate_limit_kbs=1024 # 限制单个 BE 节点从源集群拉取 Binlog（包括 Local ↪ Snapshot）的速度为 1 MB/s
```

详细参数加说明：

1. `download_binlog_rate_limit_kbs` 参数在源集群 BE 节点配置，通过设置该参数能够有效限制数据拉取速度。
2. `download_binlog_rate_limit_kbs` 参数主要用于设置单个 BE 节点的速度，若计算集群整体速率一般需要参数值乘以集群个数。

4.3.3.4 功能详情

Doris 中的跨集群复制 (CCR) 功能，主要用于在多个集群之间高效同步数据，从而增强业务连续性和容灾能力。CCR 支持 Doris 中的多种操作，确保数据在不同集群间保持一致性。以下是 CCR 支持的主要 Doris 操作的详细情况。

1. Doris Version 中的 - 表示 Doris 2.0 及以上版本，CCR 所有版本。建议使用 Doris 使用 2.0.15 或者 2.1.6 或者更新的版本。
2. CCR Syncer 和 Doris 的版本要求：Syncer Version >= 下游 Doris Version >= 上游 Doris Version。因此升级前先升 Syncer，再升下游 Doris，最后升上游 Doris。
3. CCR 目前不支持存算分离模式。

4.3.3.4.1 库

库属性

库级别任务在 Full Sync 时会同步库的属性。

属性	是否支持	Doris version	同步方式	说明
replication_allocation	支持	-	Full Sync	
data quota	不支持			
replica quota	不支持			

修改库属性

CCR 任务不同步修改库属性操作。

属性	是否支持	上游是否可以操作	下游是否可以操作	说明
replication_allocation	不支持	不可以	不可以	上下游各自操作会导致 CCR 任务中断
data quota	不支持	可以	可以	
replica quota	不支持	可以	可以	

重命名库

不支持对上下游做重命名，如果做了，可能导致视图不能工作。

4.3.3.4.2 表

表属性

属性	是否支持	Doris version	同步方式	说明
表模型（duplicate，unique，aggregate）	支持	-	SQL	
分区分桶	支持	-	SQL	
replication_num	支持	-	SQL	
replication_allocation（resource group）	支持	-	SQL	上游必须与下游一致，BE tag 必须一致，否则 CC
colocate_with	不支持			
storage_policy	不支持			
dynamic_partition	支持	-	SQL	
storage_medium	支持	-	SQL	
auto_bucket	支持	-	SQL	
group_commit 系列	支持	-	SQL	
enable_unique_key_merge_on_write	支持	-	SQL	
enable_single_replica_compaction	支持	-	SQL	
disable_auto_compaction	支持	-	SQL	
compaction_policy	支持	-	SQL	
time_series_compaction 系列	支持	-	SQL	
binlog 系列	支持	-	SQL	
variant_enable_flatten_nested	支持	-	SQL	

属性	是否支持	Doris version	同步方式	说明
skip_write_index_on_load	支持	-	SQL	
row_strore 系列	支持	-	SQL	
seq 列	支持	-	SQL	
enable_light_schema_change	支持	-	SQL	
compression_type	支持	-	SQL	
index	支持	-	SQL	
bloom_filter_columns	支持	-	SQL	
bloom_filter_fpp	支持			
storage_cooldown_time	不支持			
generated column	支持	-	SQL	
自增 id	不支持			有问题

基础表操作

操作	是否支持	Doris version	同步方式	下游是否可以单独操作	说明
create table	支持	-	SQL/Partial Sync	不支持操作 CCR 任务同步的表。	属性参考创建表部分；大部分情况下使用 SQL 同步；部分操作，比如用户建表时设置了打开了某些 session variables，或建表语句中有倒排索引，则使用部分同步
drop table	支持	-	SQL/Full Sync	同上	2.0.15/2.1.6 前：Full Sync，之后：SQL
rename table	表级别任务不支持库级别任务支持	2.1.8/3.0.4	SQL	同上	表级别任务 rename 会导致 CCR 任务停止
replace table	支持	2.1.8/3.0.4	SQL/Full Sync	同上	DB 级别使用 SQL 同步；表级别触发全量同步
truncate table	支持	-	SQL	同上	
restore table	不支持			同上	

修改表属性

同步方式为 SQL。

属性	是否支持	Doris version	上游是否可以操作	下游是否可以操作	说明
colocate	不支持		可以	不可以，触发 full sync 下游操作会丢失	
distribution type	不支持		不可以	同上	
dynamic partition	不支持		可以	同上	
replication_num	不支持		不可以	不可以	
replication_allocation	不支持		不可以		
storage policy	不支持		不可以	不可以	
enable_light_schema_change	不支持				CCR
row_store	支持	2.1.8/3.0.4			通过
bloom_filter_columns	支持	2.1.8/3.0.4			通过
bloom_filter_fpp	支持	2.1.8/3.0.4			通过
bucket num	不支持		可以	不可以，触发 full sync 下游操作会丢失	
isBeingSynced	不支持		不可以	不可以	
compaction 系列属性	不支持		可以	不可以，触发 full sync 下游操作会丢失不可以	
skip_write_index_on_load	不支持		可以	同上	
seq 列	支持	-	可以	不可以，触发 full sync 下游操作会丢失不可以	
delete sign 列	支持	-	可以	同上	
comment	支持	2.1.8/3.0.4	可以	不可以，触发 full sync 下游操作会丢失不可以	

列操作

表中 Base Index 上的列操作。

操作	是否支持	Doris version	同步方式	下游是否可以操作	备注
add key column	支持	-	Partial Sync	不可以，会导致 CCR 任务中断	
add value column	支持	-	SQL	不可以，会导致 CCR 任务中断	
drop key column	支持	-	Partial Sync	同上	
drop value column	支持	-	SQL	同上	
modify column	支持	-	Partial Sync	同上	
order by	支持	-	Partial Sync	同上	
rename	支持	2.1.8/3.0.4	SQL	同上	
comment	支持	2.1.8/3.0.4	SQL	同上	

> add/drop value column 要求建表时设置 property `"light_schema_change" = "true"`。

表中 Rollup Index 上的列操作。

操作	是否支持	Doris Version	同步方式	备注
add key column	支持	2.1.8/3.0.4	Partial Sync	
add value column	支持	2.1.8/3.0.4	SQL	需要开启 lightning schema change
drop column	支持	2.1.8/3.0.4	Partial Sync	

操作	是否支持	Doris Version	同步方式	备注
modify column	未知	2.1.8/3.0.4	Partial Sync	Doris 不支持直接修改 rollup column 类型
order by	支持	2.1.8/3.0.4	Partial sync	

Rollup

操作	是否支持	Doris Version	同步方式	备注
add rollup	支持	2.1.8/3.0.4	Partial Sync	
drop rollup	支持	2.1.8/3.0.4	SQL	
rename rollup	支持	2.1.8/3.0.4	SQL	

索引

Inverted Index

操作	是否支持	Doris Version	同步方式	备注
create index	支持	2.1.8/3.0.4	Partial Sync	
drop index	支持	2.1.8/3.0.4	SQL	
build index	支持	2.1.8/3.0.4	SQL	

Bloom Filter

操作	是否支持	Doris Version	同步方式	备注
add bloom filter	支持	2.1.8/3.0.4	Partial Sync	
alter bloom filter	支持	2.1.8/3.0.4	Partial Sync	这里指修改 bloom_filter_columns
drop bloom filter	支持	2.1.8/3.0.4	Partial Sync	

4.3.3.4.3 数据

导入

导入方式	是否支持	Doris version	同步方式	下游是否可以操作	说明
stream load	支持（临时分区除外）	-	TXN	不可以，如果下游导入了，后续触发 full 或者 Partial Sync，下游导入的数据会丢失	上游事务可见，即数据可见时生成 binlog，下游开始同步。
broker load	支持（临时分区除外）	-	TXN	同上	同上

导入方式	是否支持	Doris version	同步方式	下游是否可以操作	说明
routine load	支持（临时分区除外）	-	TXN	同上	同上
mysql load	支持（临时分区除外）	-	TXN	同上	同上
group commit	支持（临时分区除外）	2.1	TXN	同上	同上

数据操作

操作	是否支持	Doris version	同步方式	下游是否可以操作	说明
delete	支持	-	TXN	不可以，如果下游操作，后续触发 full 或者 Partial Sync，下游操作会丢失	上游事务可见，即数据可见时生成 binlog，下游开始同步。
update	支持	-	TXN	同上	同上
insert	支持	-	TXN	同上	同上
insert into overwrite	支持（临时分区除外）	2.1.6	Partial Sync	同上	同上
insert into overwrite	支持（临时分区除外）	2.0	full sync	同上	同上
显式事务 (3.0)begin commit	不支持				

4.3.3.4.4 分区操作

操作	是否支持	Doris ver-sion	同步方式	下游是否可以单独操作	说明
add partition	支持	-	SQL	不能，后续触发 Full Sync 或者 Partial Sync 会导致下游操作丢失	cooldown time 属性及其行为未知

操作	是否支持	Doris version	同步方式	下游是否可以单独操作	说明
add temp partition	不支持			同上	backup 不支持 tmp partition，从 doris 2.1.8/3.0.4 开始，可以修改上游 FE 配置：ignore_backup_tmp_partitions 绕过该问题
drop partition	支持	-	SQL/Full Sync	同上	2.0.15/2.1.6 前：Full Sync，之后：SQL
replace partition	支持	2.1.7/3.0.3	Partial Sync	同上	Partial Sync 只支持 strict range 和 non-tmp partition 的 replace 方式，否则会触发 Full Sync。
modify partition	不支持			同上	指修改 partition 的 property
rename partition	支持	2.1.8/3.0.4	SQL	同上	

4.3.3.4.5 视图

操作	是否支持	Doris version	同步方式	备注
create view	支持	-	SQL	上下游同名时可以工作；如果下游已经存在，则会先删除在创建
alter view	支持	2.1.8/3.0.4	SQL	
drop view	支持	2.1.8/3.0.4	SQL	

> note
>
> 由于 doris 实现的限制，view 中的 column name/view name 不能和 db name 相同。

4.3.3.4.6 物化视图

同步物化视图

操作	是否支持	Doris Version	同步方式	备注
create materialized view	支持	2.1.8/3.0.4	Partial Sync	上下游同名时可以工作，不同名时需要下游手动重建 view。
drop materialized view	支持	2.1.8/3.0.4	SQL	

异步物化视图

操作	是否支持
create async materialized view	不支持
alter async materialized view	不支持
drop async materialized view	不支持
refresh	不支持
pause	不支持
resume	不支持

4.3.3.4.7 统计信息

上下游之间不同步，独立工作。

4.3.3.4.8 其它

操作	是否支持
external table	不支持
recycle bin	不支持
catalog	不支持
workload group	不支持
job	不支持
function	不支持
policy	不支持
user	不支持
cancel alter job	支持

4.3.3.5 配置说明

本文给出使用 CCR 需要调整或者关注的配置。

4.3.3.5.1 FE 配置

在 fe.conf 中配置，例如 `restore_reset_index_id = true`。

名称	说明	默认值	版本
restore_reset_index_id	如果同步的表中使用 inverted index 或者 bitmap 索引，需设置为 false。	false	从 2.1.8 及 3.0.4 开始。

名称	说明	默认值	版本
ignore_backup_tmp_partitions	避免因上游创建tmp partition导致同步中断，需设置为true。	false	从2.1.8及3.0.4开始。

名称	说明	默认值	版本
max_backup_restore_job_num_per_db	内存中每个 DB 的 backup/restore job 数量限制，建议设置为 2。	10	所有版本。

名称	说明	默认值	版本
label ↪ _ ↪ num ↪ _ ↪ threshold ↪	控制TXN Label数量，防止事务回收过快，过大会占用较多内存，过小可能导致异常情况下数据重复，默认值在大多数情	2000	2.1开始。

名称	说明	默认值	版本
restore_job_compressed_serialization	tablet 数目超过 10w 时建议配置为 true。降级前关闭配置并确保 FE 完成一次 checkpoint。2.1 升级 3.0 时，至少升级到 3.0.3。	false	从 2.1.8 和 3.0.3 开始。

名称	说明	默认值	版本
backup ↪ _ ↪ job ↪ _ ↪ compressed ↪ _ ↪ serialization ↪	tablet 数目超过 10w 时建议配置为 true。降级前关闭配置并确保 FE 完成一次 check-point。2.1 升级 3.0 时，至少升级到 3.0.3。	false	从 2.1.8 和 3.0.3 开始。

名称	说明	默认值	版本
backup ↪ _ ↪ job ↪ _ ↪ default ↪ _ ↪ timeout ↪ _ ↪ ms ↪	备份/恢复任务超时时间，源、目标集群的 FE 都需要配置。	无	根据需求设置

名称	说明	默认值	版本
enable ↪ _ ↪ restore ↪ _ ↪ snapshot ↪ _ ↪ rpc ↪ _ ↪ compression ↪	开启 snapshot info 压缩，降低 RPC 消息大小，建议设置为 true。	true	从 2.1.8 和 3.0.3 开始。

4.3.3.5.2 BE

在 be.conf 中配置，例如 `thrift_max_message_size = 2000000000`。

名称	说明	默认值	版本
thrift_max_message_size	BE thrift server 单次 RPC packet 上限，CCR 任务涉及的 tablet 数目大时，建议设置为 2000000000	100MB	所有版本
be_thrift_max_pkg_bytes	BE Thrift RPC 消息包大小限制。	20MB	2.0 特有。

名称	说明	默认值	版本
max ↪ _ ↪ download ↪ _ ↪ speed ↪ _ ↪ kbps ↪	下游BE每个down-loadworker的下载限速，默认每线程50MB/s。	50MB/s	所有版本
download ↪ _ ↪ worker ↪ _ ↪ count ↪	下载任务的线程数，结合网卡、磁盘和负载设置。	1	所有版本

4.3.3.5.3 库表属性

Create Table 或者 Alter Table 设置。

名称	说明	默认值	版本
binlog.max_bytes	binlog 最大内存占用，建议至少保留 4GB。	无限制	所有版本
binlog.ttl_seconds	binlog 保留时间。	2.0.5 之前无限制，2.0.5 开始 1 天（86400）	所有版本

4.3.3.6 性能测试

本文档中的性能数据基于默认配置，如果您面临高网络延迟或高吞吐量写入场景，可以参考操作手册进行优化。

4.3.3.6.1 测试数据集

- 数据集：TPC-H 1T

4.3.3.6.2 测试集群配置

配置项	上游配置	下游配置
FE	2 核 16 GB	2 核 16 GB
BE	3 个节点，16 核 64 GB，每节点 3*500 GB	3 个节点，16 核 64 GB，每节点 3*500 GB

4.3.3.6.3 增量同步性能测试

测试步骤

1. 在上游集群创建 TPC-H 1T 的库表信息。
2. 创建 TPC-H 1T 数据库的同步任务。
3. 等待 TPC-H 1T 数据导入完成，记录完成时间。
4. 等待下游数据同步完成，记录完成时间。

测试结论

增量同步时间差：33 秒。

4.3.3.6.4 全量同步性能测试

测试步骤

1. 在上游集群创建 TPC-H 1T 的库表信息并完成数据导入，记录完成时间。
2. 创建 TPC-H 1T 数据库的同步任务。

3. 等待下游数据同步完成，记录完成时间。

测试结论

全量同步时间差：6 分 1 秒。

4.3.3.6.5 Flink 同步性能测试

测试步骤

1. 上游使用 Flink 导入方式导入 100,000,000 条数据。
2. 创建库表的同步任务。
3. 在每个阶段观察下游同步完成时间与上游导入完成时间的差异（例如：1,000,000 条、2,000,000 条等）。
4. 记录上游最后一次导入完成时间。
5. 记录下游同步完成时间为。

测试结论

每个阶段的 lag 时间均保持在 5 秒内。

4.3.4 从回收站恢复

4.3.4.1 从回收站恢复

为了避免因误操作造成的灾难，Doris 支持意外删除的数据库、表和分区的数据恢复。在删除表或数据库后，Doris 不会立即物理删除数据。当用户执行DROP DATABASE/TABLE/PARTITION命令而不使用FORCE时，Doris 会将删除的数据库、表或分区移动到回收站。可以使用RECOVER命令从回收站恢复已删除的数据库、表或分区的所有数据，使其再次可见。

注意：如果使用DROP FORCE执行删除，则数据将立即被删除，无法恢复。

4.3.4.1.1 查询回收站

您可以使用以下命令查询回收站：

```
SHOW CATALOG RECYCLE BIN [WHERE NAME [= "name" | LIKE "name_matcher"]];
```

有关更详细的语法和最佳实践，请参阅SHOW-CATALOG-RECYCLE-BIN命令手册，您还可以在 MySql 客户端命令行中输入help SHOW CATALOG RECYCLE BIN以获取更多帮助。

4.3.4.1.2 开始数据恢复

要恢复已删除的数据，您可以使用以下命令：

1. 恢复名为example_db的数据库：

```
RECOVER DATABASE example_db;
```

2. 恢复名为example_tbl的表：

```
RECOVER TABLE example_db.example_tbl;
```

3. 恢复表example_tbl中的分区 p1：

```
RECOVER PARTITION p1 FROM example_tbl;
```

有关 RECOVER 使用的更详细的语法和最佳实践，请参阅RECOVER命令手册，您还可以在 MySql 客户端命令行中输入HELP RECOVER以获取更多帮助。

4.4 日志管理

4.4.1 FE 日志管理

本文主要介绍 Frontend(FE) 进程的日志管理。

该文档适用于 2.1.4 及之后的 Doris 版本。

4.4.1.1 日志分类

当使用 sh bin/start_fe.sh --daemon 方式启动 FE 进程后，FE 日志目录下会产生以下类型的日志文件：

- fe.log

FE 进程运行日志。FE 的主日志文件。FE 进程所有等级（DEBUG、INFO、WARN、ERROR 等）的运行日志都会打印到这个日志文件中。

- fe.warn.log

FE 进程运行日志。但只会打印 WARN 级别以上的运行日志。fe.warn.log 中的内容是 fe.log 日志内容的子集。主要用于快速查看告警或错误级别日志。

- fe.audit.log

审计日志。用于记录通过这个 FE 节点执行的所有数据库操作记录。包括 SQL、 DDL、 DML 语句等。

- fe.out

用于接收标准输出流和错误数据流的日志。比如 start 脚本中的 echo 命令输出等，或其他未被 log4j 框架捕获到的日志信息。通常作为运行日志的补充。少数情况下，需要查看 fe.out 的内容以获取更多信息。

- fe.gc.log

FE JVM 的 GC 日志。该日志的行为由 fe.conf 中的 JVM 启动项 JAVA_OPTS 控制。

4.4.1.2 日志配置

包括配置日志的存放路径、保留时间、保留数目、大小等。

以下配置项均在 `fe.conf` 文件中配置。

配置项	默认值	可选项	说明
LOG ↪ _ ↪ DIR ↪	ENV ↪ (↪ DORIS ↪ _ ↪ HOME ↪) ↪ / ↪ log ↪		所有日志的存放路径。默认为 FE 部署路径的 log ↪ / ↪ 目录下。注意这是一个环境变量，配置名需大写。

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ level ↪	INFO ↪	INFO ↪ , WARN ↪ , ERROR ↪ , FATAL ↪	fe. ↪ log ↪ 的日志等级。默认为 INFO。不建议修改，INFO 等级包含许多关键日志信息。

配置项	默认值	可选项	说明
sys_log_roll_num	10		控制 fe.log 和 fe.warn.log 一天内的最大文件数量。默认 10。当因为日志滚动或切分后，日志文件数量大于这个阈值

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ verbose ↪ _ ↪ modules ↪			可以设置指定的 Java package 下的文件开启 DEBUG 级别日志。请参阅“开启 DEBUG 日志”章节

配置项	默认值	可选项	说明
sys_log_enable	false	true, false	是否开启历史 fe.log 和 fe.warn.log 日志压缩。默认关闭。开启后，历史审计日志会使用 gzip 压缩归档

配置项	默认值	可选项	说明
log_rollover_strategy	age	age, size	日志保留策略，默认为 age，即根据时间策略保留历史日志。size 为按日志大小保留历史日志

配置项	默认值	可选项	说明
sys_log_delete_age	7d	支持格式如 7d, 10h, 60m, 120s	仅当 log_rollover_strategy 为 age 时生效。控制 fe.log 和 fe.warn.log 文件的保留天数。默认 7 天。会自动删除 7 天前的日志

配置项	默认值	可选项	说明
audit_log_delete_age	30d	支持格式如 7d, 10h, 60m, 120s	仅当 log_rollover_strategy 为 age 时生效。控制 fe.audit.log 文件的保留天数。默认 30 天。会自动删除 30 天前的日志

配置项	默认值	可选项	说明
info_sys_accumulated_file_size	4		仅当 log_rollover_strategy 为 size 时生效。控制 fe.log 文件的累计大小。默认为 4GB。当累计日志大小超过这个阈值后，会删除

配置项	默认值	可选项	说明
warn_sys_accumulated_file_size	2		仅当 log_rollover_strategy 为 size 时生效。控制 fe.warn.log 文件的累计大小。默认为 2GB。当累计日志大小超过这个阈值后，会

配置项	默认值	可选项	说明
audit_sys_accumulated_file_size	4		仅当 log_rollover_strategy 为 size 时生效。控制 fe.audit.log 文件的累计大小。默认为 4GB。当累计日志大小超过这个阈值后，会

配置项	默认值	可选项	说明
log_roll_size_mb	1024		控制 fe.log, fe.warn.log, fe.audit.log 单个文件最大大小。默认 1024MB。单个日志文件超过这个阈值后，会自动切分新的文

配置项	默认值	可选项	说明
sys_log_roll_interval	DAY	DAY, HOUR	控制 fe.log 和 fe.warn.log 的滚动间隔。默认为 1 天。即每天生成一个新日志文件

配置项	默认值	可选项	说明
audit_log_roll_num	90		控制 fe.audit.log 最大文件数量。默认 90。当因为日志滚动或切分后，日志文件数量大于这个阈值后，老的日志文件将

配置项	默认值	可选项	说明
audit_log_roll_interval	DAY	DAY, HOUR	控制 fe.audit.log 的滚动间隔。默认为 1 天。即每天生成一个新日志文件

配置项	默认值	可选项	说明
audit_log_dir	ENV_DORIS_HOME/log	可以单独指定 fe.audit.log 的存放路径。默认为 FE 部署路径的 log/ 目录下。	

配置项	默认值	可选项	说明
audit_log_modules	{"slow_query", "query", "load", "stream", "load"}		fe.audit.log 中的模块类型。默认包括慢查询、查询、导入、stream load。其中“查询”只所有 DDL、DML、SQL 操作。“慢查询”指这些操作执行时

配置项	默认值	可选项	说明
qe_slow_log_ms	5000		当 DDL、DML、SQL 语句的执行时间超过这个阈值后，会在 fe.audit.log 的 slow_query 模块中单独记录。默认 5000 ms

配置项	默认值	可选项	说明
audit_log_enable	false	true, false	是否开启历史 fe.audit.log 日志压缩。默认关闭。开启后，历史审计日志会使用 gzip 压缩归档

配置项	默认值	可选项	说明
sys_log_mode	NORMAL	NORMAL, BRIEF, ASYNC	FE 日志的输出模式，其中 NORMAL 为默认的输出模式，日志同步输出且包含位置信息。ASYNC 默认是日志异步输出且包含

配置项	默认值	可选项	说明

note 从 3.0.2 版本开始，sys_log_mode 配置默认改为 AYSNC。

sys_log_roll_num 控制的是一天的保留日志数量，而不是总数量，需要配合 sys_log_delete_
↪ age 共同确定总保留日志数量。

4.4.1.3 开启 DEBUG 日志

FE 的 Debug 级别日志可以通过修改配置文件开启，也可以通过界面或 API 在运行时打开。

- 通过配置文件开启

在 fe.conf 中添加配置项 sys_log_verbose_modules。举例如下：

```
# 仅开启类 org.apache.doris.catalog.Catalog 的 Debug 日志
sys_log_verbose_modules=org.apache.doris.catalog.Catalog

# 开启包 org.apache.doris.catalog 下所有类的 Debug 日志
sys_log_verbose_modules=org.apache.doris.catalog

# 开启包 org 下所有类的 Debug 日志
sys_log_verbose_modules=org
```

添加配置项并重启 FE 节点，即可生效。

- 通过 FE UI 界面

通过 UI 界面可以在运行时修改日志级别。无需重启 FE 节点。在浏览器打开 FE 节点的 http 端口（默认为 8030），并登陆 UI 界面。之后点击上方导航栏的 Log 标签。

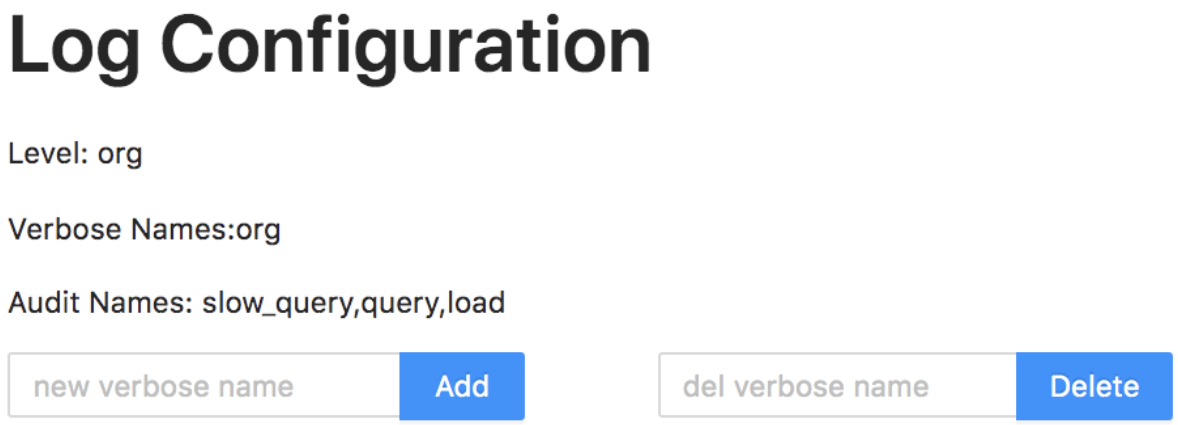


图 139: 通过 FE UI 界面

我们在 Add 输入框中可以输入包名或者具体的类名，可以打开对应的 Debug 日志。如输入 `org.apache.doris.catalog.Catalog` 则可以打开 Catalog 类的 Debug 日志：

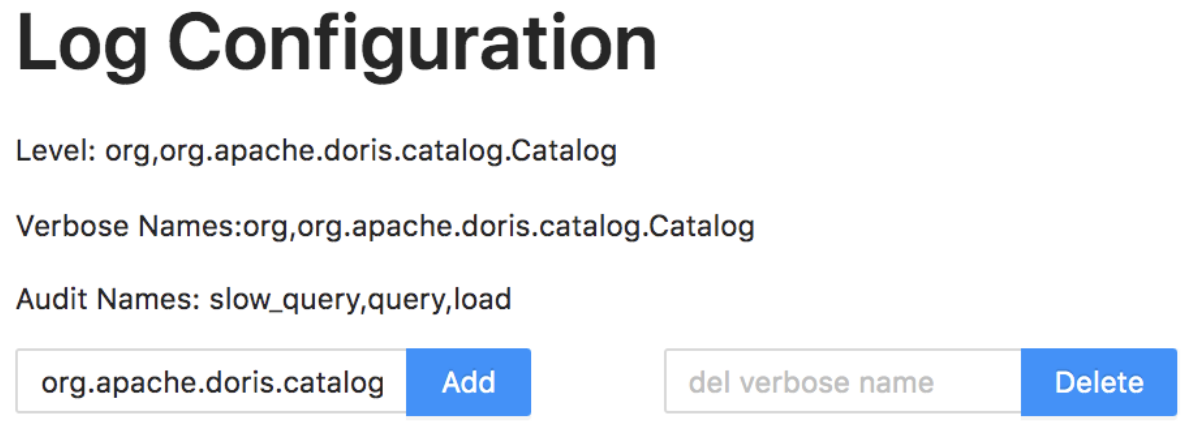


图 140: 通过 FE UI 界面

你也可以在 Delete 输入框中输入包名或者具体的类名，来关闭对应的 Debug 日志。

> 这里的修改只会影响对应的 FE 节点的日志级别。不会影响其他 FE 节点的日志级别。

- 通过 API 修改

通过以下 API 也可以在运行时修改日志级别。无需重启 FE 节点。

```
curl -X POST -uuser:passwd fe_host:http_port/rest/v1/log?add_verbose=org.apache.doris.catalog.
    ↪ Catalog
```

其中用户名密码为登陆 Doris 的 root 或 admin 用户。add_verbose 参数指定要开启 Debug 日志的包名或类名。若成功则返回：

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "LogConfiguration": {
            "VerboseNames": "org,org.apache.doris.catalog.Catalog",
            "AuditNames": "slow_query,query,load",
            "Level": "INFO"
        }
    },
    "count": 0
}
```

也可以通过以下 API 关闭 Debug 日志：

```
curl -X POST -uuser:passwd fe_host:http_port/rest/v1/log?del_verbose=org.apache.doris.catalog.
    ↪ Catalog
```

del_verbose 参数指定要关闭 Debug 日志的包名或类名。

4.4.1.4 容器环境日志配置

在某些情况下，通过容器环境（如 k8s）部署 FE 进程。所有日志需要通过标准输出流而不是文件进行输出。

此时，可以通过 sh bin/start_fe.sh --console 命令前台启动 FE 进程，并将所有日志输出到标准输出流。

为了在同一标准输出流中区分不同日志类型，会在每条日志前添加不同的前缀以示区分。如：

```
RuntimeLogger 2024-06-24 00:05:21,522 INFO (main|1) [DorisFE.start():158] Doris FE starting...
RuntimeLogger 2024-06-24 00:05:21,530 INFO (main|1) [FrontendOptions.analyzePriorityCidrs():194]
    ↪ configured prior_cidrs value: 172.20.32.136/24
RuntimeLogger 2024-06-24 00:05:21,535 INFO (main|1) [FrontendOptions.initAddrUseIp():101] local
    ↪ address: /172.20.32.136.
RuntimeLogger 2024-06-24 00:05:21,740 INFO (main|1) [ConsistencyChecker.initWorkTime():106]
    ↪ consistency checker will work from 23:00 to 23:00
RuntimeLogger 2024-06-24 00:05:21,889 ERROR (main|1) [Util.report():128] SLF4J: Class path
    ↪ contains multiple SLF4J bindings.
```

不同的前缀说明如下：

- StdoutLogger：标准输出流中的日志，对应 fe.out 中的内容。

- StderrLogger：标准错误流中的日志，对应 fe.out 中的内容。
- RuntimeLogger：对应 fe.log 中的日志。
- AuditLogger：对应 fe.audit.log 中的日志。
- 无前缀：对应 fe.gc.log 中的日志。

此外，针对容器环境还有一个额外配置参数：

配置项	默认值	可选项	说明

配置项	默认值	可选项	说明

enable_file_logger	true	true, false	是否启用文件日志。默认为 true。当使用 --console 命令启动 FE 进程时，日志会同时输出到标准输出流，以及正

配置项	默认值	可选项	说明

4.4.2 BE 日志管理

本文主要介绍 Backend(BE) 进程的日志管理。

该文档适用于 2.1.4 及之后的 Doris 版本。

4.4.2.1 日志分类

当使用 `sh bin/start_be.sh --daemon` 方式启动 BE 进程后，BE 日志目录下会产生以下类型的日志文件：

- be.INFO

BE 进程运行日志。BE 的主日志文件。BE 进程所有等级（DEBUG、INFO、WARN、ERROR 等）的运行日志都会打印到这个日志文件中。

注意这个文件是一个软链，他指向的是当前最新的 BE 运行日志文件。

- be.WARNING

BE 进程运行日志。但只会打印 WARN 级别以上的运行日志。be.WARNING 中的内容是 be.INFO 日志内容的子集。主要用于快速查看告警或错误级别日志。

注意这个文件是一个软链，他指向的是当前最新的 BE 告警日志文件。

- be.out

用于接收标准输出流和错误数据流的日志。比如 start 脚本中的 echo 命令输出等，或其他未被 glog 框架捕获到的日志信息。通常作为运行日志的补充。

通常在 BE 异常宕机情况下，需要查看这个日志来获取异常堆栈。

- jni.log

BE 进程通过 JNI 调用 Java 程序时，Java 程序打印的日志。

TODO：未来版本中，这部分日志会统一到 be.INFO 日志中。

- be.gc.log

BE JVM 的 GC 日志。该日志的行为由 be.conf 中的 JVM 启动项 JAVA_OPTS 控制。

4.4.2.2 日志配置

包括配置日志的存放路径、保留时间、保留数目、大小等。

以下配置项均在 `be.conf` 文件中配置。

配置项	默认值	可选项	说明
LOG ↪ _ ↪ DIR ↪	ENV ↪ (↪ DORIS ↪ _ ↪ HOME ↪) ↪ / ↪ log ↪		所有日志的存放路径。默认为 BE 部署路径的 log ↪ / ↪ 目录下。注意这是一个环境变量，配置名需大写。

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ level ↪	INFO ↪	INFO ↪ , WARNING ↪ , ERROR ↪ , FATAL ↪	be. ↪ INFO ↪ 的日志等级。默认为 INFO。不建议修改，INFO 等级包含许多关键日志信息。

配置项	默认值	可选项	说明
sys_log_roll_num	10		控制 be.INFO 和 be.WARNING 最大文件数量。默认 10。当因为日志滚动或切分后，日志文件数量大于这个阈值后，老的日志文

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ verbose ↪ _ ↪ modules ↪			可以设置指定代码目录下的文件开启 DEBUG 级别日志。请参阅“开启 DEBUG 日志”章节

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ verbose ↪ _ ↪ level ↪			请参阅“开启DEBUG日志”章节
sys ↪ _ ↪ log ↪ _ ↪ verbose ↪ _ ↪ flags ↪ _ ↪ v ↪			请参阅“开启DEBUG日志”章节

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ roll ↪ _ ↪ mode ↪	SIZE ↪ - ↪ MB ↪ -1024 ↪	TIME ↪ - ↪ DAY ↪ , TIME ↪ - ↪ HOUR ↪ , SIZE ↪ - ↪ MB ↪ - ↪ nnn ↪	be. ↪ INFO ↪ 和 be. ↪ WARNING ↪ 日志的滚动策略。默认为 SIZE ↪ - ↪ MB ↪ -1024 ↪ ，即每个日志达到 1024MB 大小后，生成一个新的日志文件。也可以设

配置项	默认值	可选项	说明
log_buffer_level	空	空或-1	BE日志输出模式。默认情况下，BE日志会异步下刷到磁盘日志文件中。如果设置为-1，则日志内容会实时下刷。实时下

配置项	默认值	可选项	说明
disable_compaction_trace_log	true	true, false	默认为 true，即关闭 compaction 操作的 tracing 日志。如果为 false，则会打印和 Compaction 操作相关的 tracing 日志，用于排查问题。

配置项	默认值	可选项	说明
aws ↪ _ ↪ log ↪ _ ↪ level ↪	0		用于控制aws sdk的日志等级。默认为0，表示关闭aws sdk日志。默认情况下，aws sdk日志已经被glog主动捕获，并会正常打印主

配置项	默认值	可选项	说明
s3_file_writer_log_interval_second	60		当执行 S3 Upload 操作时，会每隔 60 秒（默认）打印操作进度。

配置项	默认值	可选项	说明
enable ↪ _ ↪ debug ↪ _ ↪ log ↪ _ ↪ timeout ↪ _ ↪ secs ↪	0		当值大于0时，会打印pipeline执行引擎的一些详细执行日志。主要用于排查问题。默认情况下关闭

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ enable ↪ _ ↪ custom ↪ _ ↪ date ↪ _ ↪ time ↪ _ ↪ format ↪	false		是否允许自定义日志中的日期格式(自2.1.7版本支持)

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ custom ↪ _ ↪ date ↪ _ ↪ time ↪ _ ↪ format ↪	%Y ↪ -% ↪ m ↪ -% ↪ d ↪ ↪ % ↪ H ↪ :% ↪ M ↪ :% ↪ S ↪		默认的日志日期自定义格式，仅当 sys ↪ _ ↪ log ↪ _ ↪ enable ↪ _ ↪ custom ↪ _ ↪ date ↪ _ ↪ time ↪ _ ↪ format ↪ 为 true ↪ 时生效（自 2.1.7 版本支持）

配置项	默认值	可选项	说明
sys ↪ _ ↪ log ↪ _ ↪ custom ↪ _ ↪ date ↪ _ ↪ time ↪ _ ↪ ms ↪ _ ↪ format ↪	,{:03 ↪ d ↪ } ↪		默认的日志日期中的时间精度，仅当 sys ↪ _ ↪ log ↪ _ ↪ enable ↪ _ ↪ custom ↪ _ ↪ date ↪ _ ↪ time ↪ _ ↪ format ↪ 为 true ↪ 时生效（自 2.1.7 版本支持）

配置项	默认值	可选项	说明

4.4.2.3 开启 DEBUG 日志

4.4.2.3.1 静态配置

在 be.conf 中设置 sys_log_verbose_modules 与 sys_log_verbose_level。

```
sys_log_verbose_modules=plan_fragment_executor,olap_scan_node
sys_log_verbose_level=3
```

sys_log_verbose_modules 指定要开启的文件名，可以通过通配符 * 指定。比如：

```
sys_log_verbose_modules=*
```

表示开启所有 DEBUG 日志。

sys_log_verbose_level 表示 DEBUG 的级别。数字越大，则 DEBUG 日志越详细。取值范围在 1-10。

通常情况下，只需在 be.conf 中配置 sys_log_verbose_modules 和 sys_log_verbose_level 即可满足需求。只有在特殊情况下，比如发现某些调试日志没有按预期输出时，才需要额外设置 sys_log_verbose_flags_v，它的作用范围不受 modules 限制。

sys_log_verbose_flags_v 表示 glog 中的 FLAGS_v，用于控制全局的日志详细程度。数字越大，则 DEBUG 日志越详细。只有当 n <= FLAGS_v 时，VLOG(n) 的日志信息才会被输出，从而实现对日志输出粒度的精细控制。

4.4.2.3.2 动态调整

通过以下 RESTful API 即可：

```
curl -X POST "http://<be_host>:<webport>/api/glog/adjust?module=<module_name>&level=<level_number
    ↪ >"
```

动态调整方式同样支持通配符，例如使用 module=*&level=10 将打开所有 BE vlog。但通配符与单独的模块名互不隶属，例如将 moduleA 的 vlog 级别调整为 10，再使用 module=*&level=-1，并不会关闭 moduleA 的 vlog。

注意：动态调整的配置不会被持久化，BE 重启后将会失效。

另外无论通过何种方式，只要模块不存在，GLOG 将会创建对应日志模块（没有实际影响），并不会返回错误。

4.4.2.4 容器环境日志配置

在某些情况下，通过容器环境（如 k8s）部署 FE 进程。所有日志需要通过标准输出流而不是文件进行输出。

此时，可以通过 sh bin/start_be.sh --console 命令前台启动 BE 进程，并将所有日志输出到标准输出流。

为了在同一标准输出流中区分不同日志类型，会在每条日志前添加不同的前缀以示区分。如：

```
RuntimeLogger W20240624 00:36:46.325274 1460943 olap_server.cpp:710] Have not get FE Master
    ↪ heartbeat yet
RuntimeLogger I20240624 00:36:46.325999 1459644 olap_server.cpp:208] tablet checkpoint tasks
    ↪ producer thread started
RuntimeLogger I20240624 00:36:46.326066 1460954 olap_server.cpp:448] begin to produce tablet meta
    ↪  checkpoint tasks.
RuntimeLogger I20240624 00:36:46.326093 1459644 olap_server.cpp:213] tablet path check thread
    ↪ started
RuntimeLogger I20240624 00:36:46.326190 1459644 olap_server.cpp:219] cache clean thread started
RuntimeLogger I20240624 00:36:46.326336 1459644 olap_server.cpp:231] path gc threads started.
    ↪ number:1
RuntimeLogger I20240624 00:36:46.326643 1460958 olap_server.cpp:424] try to start path gc thread!
```

不同的前缀说明如下：

- `RuntimeLogger`：对应 `fe.log` 中的日志。

> 后续版本会增加对 jni.log 的支持。

此外，针对容器环境还有一个额外配置参数：

配置项	默认值	可选项	说明

配置项	默认值	可选项	说明
enable_file_logger	true	true, false	是否启用文件日志。默认为 true。当使用 --console 命令启动 BE 进程时，日志会同时输出到标准输出流，以及正

配置项	默认值	可选项	说明

4.5 运维监控

4.5.1 监控指标

Doris 的 FE 进程和 BE 进程都提供了完备的监控指标。监控指标可以分为两类：

1. 进程监控：主要展示 Doris 进程本身的一些监控值。
2. 节点监控：主要展示 Doris 进程所在节点机器本身的监控，如 CPU、内存、IO、网络等等。

可以通过访问 FE 或 BE 节点的 http 端口获取当前监控。如：

```
curl http://fe_host:http_port/metrics
curl http://be_host:webserver_port/metrics
```

默认返回 Prometheus 兼容格式的监控指标，如：

```
doris_fe_cache_added{type="partition"} 0
doris_fe_cache_added{type="sql"} 0
doris_fe_cache_hit{type="partition"} 0
doris_fe_cache_hit{type="sql"} 0
doris_fe_connection_total 2
```

如需获取 Json 格式的监控指标，请访问：

```
curl http://fe_host:http_port/metrics?type=json
curl http://be_host:webserver_port/metrics?type=json
```

4.5.1.1 监控等级和最佳实践

表格中的最后一列标注了监控项的重要等级。P0 表示最重要，数值越大，重要性越低。

绝大多数监控指标类型为 Counter。即累计值。你可通过间隔采集（如每 15 秒）监控值，并计算单位时间的斜率，来获得有效信息。

如可以通过计算 `doris_fe_query_err` 的斜率来获取查询错误率（error per second）。

> 欢迎完善此表格以提供更全面有效的监控指标。

4.5.1.2 FE 监控指标

4.5.1.2.1 进程监控

名称	标签	单位	含义	说明	等级
doris_fe_ ↪ cache_ ↪ added	{type=“partition”}	Num	新增的 Partition Cache 数量累计值		
	{type=“sql”}	Num	新增的 SQL Cache 数量累计值		
doris_fe_ ↪ cache_hit	{type=“partition”}	Num	命中 Partition Cache 的计数		
	{type=“sql”}	Num	命中 SQL Cache 的计数		
doris_fe_ ↪ connection ↪ _total		Num	当前 FE 的 MySQL 端口连接数	用于监控查询连接数。如果连接数超限，则新的连接将无法接入	P0
doris_fe_ ↪ counter_ ↪ hit_sql_ ↪ block_ ↪ rule		Num	被 SQL BLOCK RULE 拦截的查询数量		
doris_fe_ ↪ edit_log_ ↪ clean	{type=“failed”}	Num	清理历史元数据日志失败的次数	不应失败，如失败，需人工介入	P0
	{type=“success”}	Num	清理历史元数据日志成功的次数		
doris_fe_ ↪ edit_log	{type=“accumulated_bytes”}	字节	元数据日志写入量的累计值	通过计算斜率可以获得写入速率，来观察是否元数据写入有延迟	P0
	{type=“current_bytes”}	字节	元数据日志当前值	用于监控 editlog 大小。如果大小超限，需人工介入	P0
	{type=“read”}	Num	元数据日志读取次数的计数	通过斜率观察元数据读取频率是否正常	P0
	{type=“write”}	Num	元数据日志写入次数的计数	通过斜率观察元数据写入频率是否正常	P0
	{type=“current”}	Num	元数据日志当前数量	用于监控 editlog 数量。如果数量超限，需人工介入	P0
doris_fe_ ↪ editlog_ ↪ write_ ↪ latency_ ↪ ms		毫秒	元数据日志写入延迟的百分位统计。如 {quantile=“0.75”} 表示 75 分位的写入延迟		

名称	标签	单位	含义	说明	等级
doris_fe_ ↪ image_ ↪ clean	{type=“failed”}	Num	清理历史元数据镜像文件失败的次数	不应失败，如失败，需人工介入	P0
	{type=“success”}	Num	清理历史元数据镜像文件成功的次数		
doris_fe_ ↪ image_ ↪ push	{type=“failed”}	Num	将元数据镜像文件推送给其他 FE 节点的失败的次数		
	{type=“success”}	Num	将元数据镜像文件推送给其他 FE 节点的成功的次数		
doris_fe_ ↪ image_ ↪ write	{type=“failed”}	Num	生成元数据镜像文件失败的次数	不应失败，如失败，需人工介入	P0
	{type=“success”}	Num	生成元数据镜像文件成功的次数		
doris_fe_job		Num	当前不同作业类型以及不同作业状态的计数。如 {job=“load”, type=“INSERT”, state=“LOADING”} 表示类型为 INSERT 的导入作业，处于 LOADING 状态的作业个数	可以根据需要，观察不同类型的作业在集群中的数量	P0
doris_fe_max ↪ _journal_ ↪ id		Num	当前 FE 节点最大元数据日志 ID。如果是 Master FE，则是当前写入的最大 ID，如果是非 Master FE，则代表当前回放的元数据日志最大 ID	用于观察多个 FE 之间的 id 是否差距过大。过大则表示元数据同步出现问题	P0
doris_fe_max ↪ _tablet_ ↪ compaction ↪ _score		Num	所有 BE 节点中最大的 compaction score 值。	该值可以观测当前集群最大的 compaction score，以判断是否过高。如过高则可能出现查询或写入延迟	P0
doris_fe_qps		Num/Se	当前 FE 每秒查询数量（仅统计查询请求）	QPS	P0
doris_fe_ ↪ query_err		Num	错误查询的累积值		
doris_fe_ ↪ query_err ↪ _rate		Num/Se	每秒错误查询数	观察集群是否出现查询错误	P0
doris_fe_ ↪ query_ ↪ latency_ ↪ ms		毫秒	查询请求延迟的百分位统计。如 {quantile=“0.75”} 表示 75 分位的查询延迟	详细观察各分位查询延迟	P0

名称	标签	单位	含义	说明	等级
doris_fe_ ↪ query_ ↪ latency_ ↪ ms_db		毫秒	各个 DB 的查询请求延迟的百分位统计。如 {quantile=“0.75”,db=“test”} 表示 DB test 75 分位的查询延迟	详细观察各 DB 各分位查询延迟	P0
doris_fe_ ↪ query_ ↪ olap_ ↪ table		Num	查询内部表（OlapTable）的请求个数统计		
doris_fe_ ↪ query_ ↪ total		Num	所有查询请求的累积计数		
doris_fe_ ↪ report_ ↪ queue_ ↪ size		Num	BE 的各种定期汇报任务在 FE 端的队列长度	该值反映了汇报任务在 Master FE 节点上的阻塞程度，数值越大，表示 FE 处理能力不足	P0
doris_fe_ ↪ request_ ↪ total		Num	所有通过 MySQL 端口接收的操作请求（包括查询和其他语句）		
doris_fe_ ↪ routine_ ↪ load_ ↪ error_ ↪ rows		Num	统计集群内所有 Routine Load 作业的错误行数总和		
doris_fe_ ↪ routine_ ↪ load_ ↪ receive_ ↪ bytes		字节	统计集群内所有 Routine Load 作业接收的数据量大小		
doris_fe_ ↪ routine_ ↪ load_rows		Num	统计集群内所有 Routine Load 作业接收的数据行数		
doris_fe_ ↪ routine_ ↪ load_get_ ↪ meta_ ↪ latency		毫秒	统计集群内所有 Routine Load Job 获取元数据的延迟		
doris_fe_ ↪ routine_ ↪ load_get_ ↪ meta_ ↪ count		Num	统计集群内所有 Routine Load Job 获取元数据的操次数		

名称	标签	单位	含义	说明	等级
doris_fe_routine_load_get_meta_fail_count		Num	统计集群内所有 Routine Load Job 获取元数据失败的次数		
doris_fe_routine_load_task_execute_time		毫秒	统计集群内所有 Routine Load Task 的执行时间		
doris_fe_routine_load_task_execute_count		Num	统计集群内所有 Routine Load Task 的执行时间		
doris_fe_routine_load_lag		毫秒	统计集群内所有 Routine Load Job 的消费延迟		
doris_fe_routine_load_progress		毫秒	统计集群内所有 Routine Load Job 的消费进度		
doris_fe_routine_load_abort_task_num		毫秒	统计集群内所有 Routine Load Job 失败的 Task 数量		
doris_fe_rps		Num	当前 FE 每秒请求数量（包含查询以及其他各类语句）	和 QPS 配合来查看集群处理请求的量	P0
doris_fe_scheduled_tablet_num		Num	Master FE 节点正在调度的 tablet 数量。包括正在修复的副本和正在均衡的副本	该数值可以反映当前集群，正在迁移的 tablet 数量。如果长时间有值，说明集群不稳定	P0
doris_fe_tablet_max_compaction_score		Num	各个 BE 节点汇报的 compaction core。如 {backend=“172.21.0.1:9556”} 表示“172.21.0.1:9556” 这个 BE 的汇报值		

名称	标签	单位	含义	说明	等级
doris_fe_ ↪ tablet_ ↪ num		Num	各个 BE 节点当前 tablet 总数。如 {backend=“172.21.0.1:9556”} 表示“172.21.0.1:9556”这个 BE 的当前 tablet 数量	可以查看 tablet 分布是否均匀以及绝对值是否合理	P0
doris_fe_ ↪ tablet_ ↪ status_ ↪ count		Num	统计 Master FE 节点 Tablet 调度器所调度的 tablet 数量的累计值。		
	{type=“added”}	Num	统计 Master FE 节点 Tablet 调度器所调度的 tablet 数量的累计值。“added”表示被调度过的 tablet 数量		
	{type=“in_sched”}	Num	同上。表示被重复调度的 tablet 数量	该值如果增长较快，则说明有 tablet 长时间处于不健康状态，导致被调度器反复调度	
	{type=“not_ready”}	Num	同上。表示尚未满足调度触发条件的 tablet 数量。	该值如果增长较快，说明有大量 tablet 处于不健康状态但又无法被调度	
	{type=“total”}	Num	同上。表示累积的被检查过（但不一定被调度）的 tablet 数量。		
	{type=“unhealthy”}	Num	同上。表示累积的被检查过的不健康的 tablet 数量。		
doris_fe_ ↪ thread_ ↪ pool		Num	统计各类线程池的工作线程数和排队情况。"active_thread_num" 表示正在执行的任务数。"pool_size" 表示线程池总线程数量。"task_in_queue" 表示正在排队的任务数		
	{name=“agent-task-pool”}	Num	Master FE 用于发送 Agent Task 到 BE 的线程池		
	{name=“connect-scheduler-check-timer”}	Num	用于检查 MySQL 空闲连接是否超时的线程池		
	{name=“connect-scheduler-pool”}	Num	用于接收 MySQL 连接请求的线程池		
	{name=“mysql-nio-pool”}	Num	NIO MySQL Server 用于处理任务的线程池		

名称	标签	单位	含义	说明	等级
	{name="export-exporting-job-pool"}	Num	exporting 状态的 export 作业的调度线程池		
	{name="export-pending-job-pool"}	Num	pending 状态的 export 作业的调度线程池		
	{name="heartbeat-mgr-pool"}	Num	Master FE 用于处理各个节点心跳的线程池		
	{name="loading-load-task-scheduler"}	Num	Master FE 用于调度 Broker Load 作业中，loading task 的调度线程池		
	{name="pending-load-task-scheduler"}	Num	Master FE 用于调度 Broker Load 作业中，pending task 的调度线程池		
	{name="schema-change-pool"}	Num	Master FE 用于调度 schema change 作业的线程池		
	{name="thrift-server-pool"}	Num	FE 端 ThriftServer 的工作线程池。对应 fe.conf 中 rpc_port。用于和 BE 进行交互。		
doris_fe_txn ↪ _counter		Num	统计各个状态的导入事务的数量的累计值	可以观测导入事务的执行情况。	P0
	{type="begin"}	Num	提交的事务数量		
	{type="failed"}	Num	失败的事务数量		
	{type="reject"}	Num	被拒绝的事务数量。(如当前运行事务数大于阈值，则新的事务会被拒绝)		
	{type="succes"}	Num	成功的事务数量		
doris_fe_txn ↪ _status		Num	统计当前处于各个状态的导入事务的数量。如 {type="committed"} 表示处于 committed 状态的事务的数量	可以观测各个状态下导入事务的数量，来判断是否有堆积	P0
doris_fe_ ↪ query_ ↪ instance_ ↪ num		Num	指定用户当前正在请求的 fragment instance 数目。如 {user="test_u"} 表示用户 test_u 当前正在请求的 instance 数目	该数值可以用于观测指定用户是否占用过多查询资源	P0
doris_fe_ ↪ query_ ↪ instance_ ↪ begin		Num	指定用户请求开始的 fragment instance 数目。如 {user="test_u"} 表示用户 test_u 开始请求的 instance 数目	该数值可以用于观测指定用户是否提交了过多查询	P0
doris_fe_ ↪ query_rpc ↪ _total		Num	发往指定 BE 的 RPC 次数。如 {be="192.168.10.1"} 表示发往 ip 为 192.168.10.1 的 BE 的 RPC 次数	该数值可以观测是否向某个 BE 提交了过多 RPC	

名称	标签	单位	含义	说明	等级
doris_fe_query_rpc_failed		Num	发往指定 BE 的 RPC 失败次数。如 {be=“192.168.10.1”} 表示发往 ip 为 192.168.10.1 的 BE 的 RPC 失败次数	该数值可以观测某个 BE 是否存在 RPC 问题	
doris_fe_query_rpc_size		Num	指定 BE 的 RPC 数据大小。如 {be=“192.168.10.1”} 表示发往 ip 为 192.168.10.1 的 BE 的 RPC 数据字节数	该数值可以观测是否向某个 BE 提交了过大的 RPC	
doris_fe_txn_exec_latency_ms		毫秒	事务执行耗时的百分位统计。如 {quantile=“0.75”} 表示 75 分位的事务执行耗时	详细观察各分位事务执行耗时	P0
doris_fe_txn_publish_latency_ms		毫秒	事务 publish 耗时的百分位统计。如 {quantile=“0.75”} 表示 75 分位的事务 publish 耗时	详细观察各分位事务 publish 耗时	P0
doris_fe_txn_num		Num	指定 DB 正在执行的事务数。如 {db=“test”} 表示 DB test 当前正在执行的事务数	该数值可以观测某个 DB 是否提交了大量事务	P0
doris_fe_publish_txn_num		Num	指定 DB 正在 publish 的事务数。如 {db=“test”} 表示 DB test 当前正在 publish 的事务数	该数值可以观测某个 DB 的 publish 事务数量	P0
doris_fe_txn_replica_num		Num	指定 DB 正在执行的事务打开的副本数。如 {db=“test”} 表示 DB test 当前正在执行的事务打开的副本数	该数值可以观测某个 DB 是否打开了过多的副本，可能会影响其他事务执行	P0
doris_fe_thrift_rpc_total		Num	FE thrift 接口各个方法接收的 RPC 请求次数。如 {method=“report”} 表示 report 方法接收的 RPC 请求次数	该数值可以观测某个 thrift rpc 方法的负载	
doris_fe_thrift_rpc_latency_ms		毫秒	FE thrift 接口各个方法接收的 RPC 请求耗时。如 {method=“report”} 表示 report 方法接收的 RPC 请求耗时	该数值可以观测某个 thrift rpc 方法的负载	
doris_fe_external_schema_cache	{catalog=“hive”}	Num	指定 External Catalog 对应的 schema cache 的数量		
doris_fe_hive_meta_cache	{catalog=“hive”}	Num			
	{type="partition_value"}	Num	指定 External Hive Metastore Catalog 对应的 partition value cache 的数量		

名称	标签	单位	含义	说明	等级
	{type="partition"}	Num	指定 External Hive Metastore Catalog 对应的 partition cache 的数量		
	{type="file"}	Num	指定 External Hive Metastore Catalog 对应的 file cache 的数量		

4.5.1.2.2 JVM 监控

名称	标签	单位	含义	说明	等级
jvm_heap_size_bytes		字节	JVM 内存监控。标签包含 max, used, committed，分别对应最大值，已使用和已申请的内存	观测 JVM 内存使用情况	P0
jvm_non_heap_size_bytes		字节	JVM 堆外内存统计		
<GarbageCollector>			GC 监控。	GarbageCollector 指代具体的垃圾收集器	P0
	{type=“count”}	Num	GC 次数累计值		
	{type=“time”}	毫秒	GC 耗时累计值		
jvm_old_size_bytes		字节	JVM 老年代内存统计		P0
jvm_thread		Num	JVM 线程数统计	观测 JVM 线程数是否合理	P0
jvm_young_size_bytes		字节	JVM 新生代内存统计		P0

4.5.1.2.3 机器监控

名称	标签	单位	含义	说明	等级
system_meminfo		字节	FE 节点机器的内存监控。采集自 /proc/meminfo。包括 buffers，cached, memory_available, memory_free, memory_total		
system_snmp		FE 节点机器的网络监控。采集自 /proc/net/snmp。			

名称	标签	单位	含义	说明	等级
	{name="tcp_ ↪ in_errs ↪ "}	Num	tcp 包接收错误的次数		
	{name="tcp_ ↪ in_segs ↪ "}	Num	tcp 包发送的个数		
	{name="tcp_ ↪ out_segs ↪ "}	Num	tcp 包发送的个数		
	{name="tcp_ ↪ retrans_ ↪ segs"}	Num	tcp 包重传的个数		

4.5.1.3 BE 监控指标

4.5.1.3.1 进程监控

名称	标签	单位	含义	说明	等级
doris_be_active_ ↪ scan_context_ ↪ count		Num	展示当前由外部直接打开的 scanner 的个数		
doris_be_add_ ↪ batch_task_ ↪ queue_size		Num	记录导入时，接收 batch 的线程池的队列大小	如果大于 0，则表示导入任务的接收端出现积压	P0
agent_task_queue ↪ _size		Num	展示各个 Agent Task 处理队列的长度，如 {type="CREATE_TABLE"} 表示 CREATE_TABLE 任务队列的长度		
doris_be_brpc_ ↪ endpoint_stub ↪ _count		Num	已创建的 brpc stub 的数量，这些 stub 用于 BE 之间的交互		
doris_be_brpc_ ↪ function_ ↪ endpoint_stub ↪ _count		Num	已创建的 brpc stub 的数量，这些 stub 用于和 Remote RPC 之间交互		
doris_be_cache_ ↪ capacity			记录指定 LRU Cache 的容量		
doris_be_cache_ ↪ usage			记录指定 LRU Cache 的使用量	用于观测内存占用情况	P0

名称	标签	单位	含义	说明	等级
doris_be_cache_ ↪ usage_ratio			记录指定 LRU Cache 的使用率		
doris_be_cache_ ↪ lookup_count			记录指定 LRU Cache 被查找的次数		
doris_be_cache_ ↪ hit_count			记录指定 LRU Cache 的命中次数		
doris_be_cache_ ↪ hit_ratio			记录指定 LRU Cache 的命中率	用于观测 cache 是否有效	P0
	{name="DataPageCache"}	Num	DataPageCache 用于缓存数据的 Data Page	数据 Cache，直接影响查询效率	P0
	{name="IndexPageCache"}	Num	IndexPageCache 用于缓存数据的 Index Page	索引 Cache，直接影响查询效率	P0
	{name="LastSuccessChannelCache"}	Num	LastSuccessChannelCache 用于缓存导入接收端的 LoadChannel		
	{name="SegmentCache"}	Num	SegmentCache 用于缓存已打开的 Segment，如索引信息		
doris_be_chunk_ ↪ pool_local_ ↪ core_alloc_ ↪ count		Num	ChunkAllocator 中，从绑定的 core 的内存队列中分配内存的次数		
doris_be_chunk_ ↪ pool_other_ ↪ core_alloc_ ↪ count		Num	ChunkAllocator 中，从其他的 core 的内存队列中分配内存的次数		
doris_be_chunk_ ↪ pool_reserved ↪ _bytes		字节	ChunkAllocator 中预留的内存大小		
doris_be_chunk_ ↪ pool_system_ ↪ alloc_cost_ns		纳秒	SystemAllocator 申请内存的耗时累计值	通过斜率可以观测内存分配的耗时	P0
doris_be_chunk_ ↪ pool_system_ ↪ alloc_count		Num	SystemAllocator 申请内存的次数		
doris_be_chunk_ ↪ pool_system_ ↪ free_cost_ns		纳秒	SystemAllocator 释放内存的耗时累计值	通过斜率可以观测内存释放的耗时	P0
doris_be_chunk_ ↪ pool_system_ ↪ free_count		Num	SystemAllocator 释放内存的次数		

名称	标签	单位	含义	说明	等级
doris_be_ ↪ compaction_ ↪ bytes_total		字节	compaction 处理的数据量的累计值	记录的是 compaction 任务中，input rowset 的 disk size。通过斜率可以观测 compaction 的速率	P0
	{type=“base”}	字节	Base Compaction 的数据量累计		
	{type=“cumulative”}	字节	Cumulative Compaction 的数据量累计		
doris_be_ ↪ compaction_ ↪ deltas_total		Num	compaction 处理的 rowset 个数的累计值	记录的是 compaction 任务中，input rowset 的个数	
	{type=“base”}	Num	Base Compaction 处理的 rowset 个数累计		
	{type=“cumulative”}	Num	Cumulative Compaction 处理的 rowset 个数累计		
doris_be_disks_ ↪ compaction_ ↪ num		Num	指定数据目录上正在执行的 compaction 任务数。如 {path="/path1/"} 表示/path1 目录上正在执行的任务数	用于观测各个磁盘上的 compaction 任务数是否合理	P0
doris_be_disks_ ↪ compaction_ ↪ score		Num	指定数据目录上正在执行的 compaction 令牌数。如 {path="/path1/"} 表示/path1 目录上正在执行的令牌数		
doris_be_ ↪ compaction_ ↪ used_permits		Num	Compaction 任务已使用的令牌数量	用于反映 Compaction 的资源消耗量	
doris_be_ ↪ compaction_ ↪ waitting_ ↪ permits		Num	正在等待 Compaction 令牌的数量		
doris_be_data_ ↪ stream_ ↪ receiver_ ↪ count		Num	数据接收端 Receiver 的数量	FIXME：向量化引擎此指标缺失	
doris_be_disks_ ↪ avail_ ↪ capacity		字节	指定数据目录所在磁盘的剩余空间。如 {path="/path1/"} 表示 /path1 目录所在磁盘的剩余空间		P0

名称	标签	单位	含义	说明	等级
doris_be_disks_ ↪ local_used_ ↪ capacity		字节	指定数据目录所在磁盘的本地已使用空间		
doris_be_disks_ ↪ remote_used_ ↪ capacity		字节	指定数据目录所在磁盘的对应的远端目录的已使用空间		
doris_be_disks_ ↪ state		布尔	指定数据目录的磁盘状态。1 表示正常。0 表示异常		
doris_be_disks_ ↪ total_ ↪ capacity		字节	定数据目录所在磁盘的总容量	配合 doris_be_disks_ ↪ avail_capacity 计算磁盘使用率	P0
doris_be_engine_ ↪ requests_ ↪ total		Num	BE 上各类任务执行状态的累计值		
	{status="failed",type="xxx"}	Num	xxx 类型的任务的失败次数的累计值		
	{status="total",type="xxx"}	Num	xxx 类型的任务的总次数的累计值。	可以按需监控各类任务的失败次数	P0
	{status="skip ↪ ",type=" ↪ report_all_ ↪ tablets"}	Num	xxx 类型任务被跳过执行的次数的累计值		
doris_be_ ↪ fragment_ ↪ endpoint_ ↪ count		Num	同	FIXME: 同 doris_be_data_stream_ ↪ receiver_count 数目。并且向量化引擎缺失	
doris_be_ ↪ fragment_ ↪ request_ ↪ duration_us		微秒	所有 fragment intance 的执行时间累计	通过斜率观测 instance 的执行耗时	P0
doris_be_ ↪ fragment_ ↪ requests_ ↪ total		Num	执行过的 fragment instance 的数量累计		
doris_be_load_ ↪ channel_count		Num	当前打开的 load channel 个数	数值越大，说明当前正在执行的导入任务越多	P0
doris_be_local_ ↪ bytes_read_ ↪ total		字节	由 LocalFileReader 读取的字节数		P0
doris_be_local_ ↪ bytes_written ↪ _total		字节	由 LocalFileWriter 写入的字节数		P0

名称	标签	单位	含义	说明	等级
doris_be_local_ ↪ file_reader_ ↪ total		Num	打开的 LocalFileReader 的累计计数		
doris_be_local_ ↪ file_open_ ↪ reading		Num	当前打开的 LocalFileReader 个数		
doris_be_local_ ↪ file_writer_ ↪ total		Num	打开的 LocalFileWriter 的累计计数。		
doris_be_mem_ ↪ consumption		字节	指定模块的当前内存开销。如 {type=“compaction”} 表示 compaction 模块的当前总内存开销。	值取自相同 type 的 MemTracker。 FIXME	
doris_be_memory_ ↪ allocated_ ↪ bytes		字节	BE 进程物理内存大小，取自 /proc/self/status/VmRSS		P0
doris_be_memory_ ↪ jemalloc		字节	Jemalloc stats, 取自 je_mallctl。	含义参考： https://jemalloc.net/jemalloc.3.html	P0
doris_be_memory_ ↪ pool_bytes_ ↪ total		字节	所有 MemPool 当前占用的内存大小。统计值，不代表真实内存使用。		
doris_be_ ↪ memtable_ ↪ flush_ ↪ duration_us		微秒	memtable 写入磁盘的耗时累计值	通过斜率可以观测写入延迟	P0
doris_be_ ↪ memtable_ ↪ flush_total		Num	memtable 写入磁盘的个数累计值	通过斜率可以计算写入文件的频率	P0
doris_be_meta_ ↪ request_ ↪ duration		微秒	访问 RocksDB 中的 meta 的耗时累计	通过斜率观测 BE 元数据读写延迟	P0
	{type=“read”}	微秒	读取耗时		
	{type=“write”}	微秒	写入耗时		
doris_be_meta_ ↪ request_total		Num	访问 RocksDB 中的 meta 的次数累计	通过斜率观测 BE 元数据访问频率	P0
	{type=“read”}	Num	读取次数		
	{type=“write”}	Num	写入次数		

名称	标签	单位	含义	说明	等级
doris_be_ ↪ fragment_ ↪ instance_ ↪ count		Num	当前已接收的 fragment instance 的数量	观测是否出现 instance 堆积	P0
doris_be_process ↪ _fd_num_limit ↪ _hard		Num	BE 进程的文件句柄数硬限。通过 /proc/pid/limits 采集		
doris_be_process ↪ _fd_num_limit ↪ _soft		Num	BE 进程的文件句柄数软限。通过 /proc/pid/limits 采集		
doris_be_process ↪ _fd_num_used		Num	BE 进程已使用的文件句柄数。通过 /proc/pid/limits 采集		
doris_be_process ↪ _thread_num		Num	BE 进程线程数。通过 /proc/pid/task 采集		P0
doris_be_query_ ↪ cache_memory_ ↪ total_byte		字节	Query Cache 占用字节数		
doris_be_query_ ↪ cache_ ↪ partition_ ↪ total_count		Num	当前 Partition Cache 缓存个数		
doris_be_query_ ↪ cache_sql_ ↪ total_count		Num	当前 SQL Cache 缓存个数		
doris_be_query_ ↪ scan_bytes		字节	读取数据量的累计值。这里只统计读取 Olap 表的数据量		
doris_be_query_ ↪ scan_bytes_ ↪ per_second		字节/秒	根据 doris_be_query_scan ↪ _bytes 计算得出的读取速率	观测查询速率	P0
doris_be_query_ ↪ scan_rows		Num	读取行数的累计值。这里只统计读取 Olap 表的数据量。并且是 RawRowsRead (部分数据行可能被索引跳过，并没有真正读取，但仍会记录到这个值中)	通过斜率观测查询速率	P0
doris_be_result_ ↪ block_queue_ ↪ count		Num	当前查询结果缓存中的 fragment instance 个数	该队列仅用于被外部系统直接读取时使用。如 Spark on Doris 通过 external scan 查询数据	

名称	标签	单位	含义	说明	等级
doris_be_result_ ↪ buffer_block_ ↪ count		Num	当前查询结果缓存中的 query 个数	该数值反映当前 BE 中有多少查询的结果正在等待 FE 消费	P0
doris_be_routine ↪ _load_task_ ↪ count		Num	当前正在执行的 routine load task 个数		
doris_be_rowset_ ↪ count_ ↪ generated_and ↪ _in_use		Num	自上次启动后，新增的并且正在使用的 rowset id 个数。		
doris_be_s3_ ↪ bytes_read_ ↪ total		Num	S3FileReader 的打开累计次数		
doris_be_s3_file ↪ _open_reading		Num	当前打开的 S3FileReader 个数		
doris_be_s3_ ↪ bytes_read_ ↪ total		字节	S3FileReader 读取字节数累计值		
doris_be_scanner ↪ _thread_pool_ ↪ queue_size		Num	用于 OlapScanner 的线程池的当前排队数量	大于零则表示 Scanner 开始堆积	P0
doris_be_segment ↪ _read	{type="segment ↪ _read_total ↪ "}	Num	读取的 segment 的个数累计值		
doris_be_segment ↪ _read	{type="segment ↪ _row_total ↪ "}	Num	读取的 segment 的行数累计值	该数值也包含了被索引过滤的行数。相当于读取的 segment 个数 * 每个 segment 的总行数	
doris_be_send_ ↪ batch_thread_ ↪ pool_queue_ ↪ size		Num	导入时用于发送数据包的线程池的排队个数	大于 0 则表示有堆积	P0
doris_be_send_ ↪ batch_thread_ ↪ pool_thread_ ↪ num		Num	导入时用于发送数据包的线程池的线程数		
doris_be_small_ ↪ file_cache_ ↪ count		Num	当前 BE 缓存的小文件数量		

名称	标签	单位	含义	说明	等级
doris_be_streaming_load_current_processing		Num	当前正在运行的 stream load 任务数	仅包含 curl 命令发送的任务	
doris_be_streaming_load_duration_ms		毫秒	所有 stream load 任务执行时间的耗时累计值		
doris_be_streaming_load_requests_total		Num	stream load 任务数的累计值	通过斜率可观测任务提交频率	P0
doris_be_stream_load_pipe_count		Num	当前 stream load 数据管道的个数	包括 stream load 和 routine load 任务	
doris_be_stream_load	{type=“load_rows”}	Num	stream load 最终导入的行数累计值	包括 stream load 和 routine load 任务	P0
doris_be_stream_load	{type=“receive_bytes”	字节	stream load 接收的字节数累计值	包括 stream load 从 http 接收的数据，以及 routine load 从 kafka 读取的数据	P0
doris_be_tablet_base_max_compaction_score		Num	当前最大的 Base Compaction Score	该数值实时变化，有可能丢失峰值数据。数值越高，表示 compaction 堆积越严重	P0
doris_be_tablet_cumulative_max_compaction_score		Num	同上。当前最大的 Cumulative Compaction Score		
doris_be_tablet_version_num_distribution		Num	tablet version 数量的直方。	用于反映 tablet version 数量的分布	P0
doris_be_thrift_connections_total		Num	创建过的 thrift 连接数的累计值。如 {name="heartbeat"} 表示心跳服务的连接数累计	此数值为 BE 作为服务端的 thrift server 的连接	
doris_be_thrift_current_connections		Num	当前 thrift 连接数。如 {name="heartbeat"} 表示心跳服务的当前连接数。	同上	

名称	标签	单位	含义	说明	等级
doris_be_thrift_ ↪ opened_ ↪ clients		Num	当前已打开的 thrift 客户端的数量。如 {name="frontend"} 表示访问 FE 服务的客户端数量		
doris_be_thrift_ ↪ used_clients		Num	当前正在使用的 thrift 客户端的数量。如 {name="frontend"} 表示正在用于访问 FE 服务的客户端数量		
doris_be_timeout ↪ _canceled_ ↪ fragment_ ↪ count		Num	因超时而被取消的 fragment instance 数量累计值	这个值可能会被重复记录。比如部分 fragment instance 被多次取消	P0
doris_be_stream_ ↪ load_txn_ ↪ request	{type=“begin”}	Num	stream load 开始事务数的累计值	包括 stream load 和 routine load 任务	
doris_be_stream_ ↪ load_txn_ ↪ request	{type=“commit”}	Num	stream load 执行成功的事务数的累计值	同上	
doris_be_stream_ ↪ load_txn_ ↪ request	{type=“rollback”}		stream load 执行失败的事务数的累计值	同上	
doris_be_unused_ ↪ rowsets_count		Num	当前已废弃的 rowset 的个数	这些 rowset 正常情况下会被定期删除	
doris_be_upload_ ↪ fail_count		Num	冷热分层功能，上传到远端存储失败的 rowset 的次数累计值		
doris_be_upload_ ↪ rowset_count		Num	冷热分层功能，上传到远端存储成功的 rowset 的次数累计值		
doris_be_upload_ ↪ total_byte			字节	冷热分层功能，上传到远端存储成功的 rowset 数据量累计值	
doris_be_load_ ↪ bytes		字节	通过 tablet sink 发送的数量累计	可观测导入数据量	P0
doris_be_load_ ↪ rows		Num	通过 tablet sink 发送的行数累计	可观测导入数据量	P0
fragment_thread_ ↪ pool_queue_ ↪ size		Num	当前查询执行线程池等待队列的长度	如果大于零，则说明查询线程已耗尽，查询会出现堆积	P0
doris_be_all_ ↪ rowsets_num		Num	当前所有 rowset 的个数		P0

名称	标签	单位	含义	说明	等级
doris_be_all_segments_num		Num	当前所有 segment 的个数		P0
doris_be_heavy_work_max_threads		Num	brpc heavy 线程池线程个数		p0
doris_be_light_work_max_threads		Num	brpc light 线程池线程个数		p0
doris_be_heavy_work_pool_queue_size		Num	brpc heavy 线程池队列最大长度，超过则阻塞提交 work		p0
doris_be_light_work_pool_queue_size		Num	brpc light 线程池队列最大长度，超过则阻塞提交 work		p0
doris_be_heavy_work_active_threads		Num	brpc heavy 线程池活跃线程数		p0
doris_be_light_work_active_threads		Num	brpc light 线程池活跃线程数		p0
routine_load_get_msg_latency		毫秒	Routine Load 获取 Kafka 消息的延迟 Load		
routine_load_get_msg_count		Num	Routine Load 获取 Kafka 消息的次数 Load		
routine_load_consume_bytes		字节	Routine Load 消费 Kafka 的数据量 Load		
routine_load_consume_rows		Num	Routine Load 消费 Kafka 的行数 Load		

4.5.1.3.2 机器监控

名称	标签	单位	含义	说明	等级
doris_be_cpu		Num	CPU 相关监控指标，从 /proc/stat 采集。会分别采集每个逻辑核的各项数值。如 {device="cpu0",mode="nice"} 表示 cpu0 的 nice 值	可计算得出 CPU 使用率	P0
doris_be_disk_bytes_read		字节	磁盘读取量累计值。从 /proc/diskstats 采集。会分别采集每块磁盘的数值。如 {device="vdd"} 表示 vvd 盘的数值		

名称	标签	单位	含义	说明	等级
doris_be_disk_ ↪ bytes_written		字节	磁盘写入量累计值。采集方式同上		
doris_be_disk_io ↪ _time_ms		字节	采集方式同上	可计算得出 IO Util	P0
doris_be_disk_io ↪ _time_ ↪ weighted		字节	采集方式同上		
doris_be_disk_ ↪ reads_ ↪ completed		字节	采集方式同上		
doris_be_disk_ ↪ read_time_ms		字节	采集方式同上		
doris_be_disk_ ↪ writes_ ↪ completed		字节	采集方式同上		
doris_be_disk_ ↪ write_time_ms		字节	采集方式同上		
doris_be_fd_num_ ↪ limit		Num	系统文件句柄限制上限。从 /proc/sys/fs/file-nr 采集		
doris_be_fd_num_ ↪ used		Num	系统已使用文件句柄数。从 /proc/sys/fs/file-nr 采集		
doris_be_file_ ↪ created_total		Num	本地文件创建次数累计	所有调用 local_ ↪ file_writer 并最终 close 的文件计数	
doris_be_load_ ↪ average		Num	机器 Load Avg 指标监控。如 {mode="15_minutes"} 为 15 分钟 Load Avg	观测整机负载	P0
doris_be_max_ ↪ disk_io_util_ ↪ percent		百分比	计算得出的所有磁盘中，最大的 IO UTIL 的磁盘的数值		P0
doris_be_max_ ↪ network_ ↪ receive_bytes ↪ _rate		字节/秒	计算得出的所有网卡中，最大的接收速率		P0
doris_be_max_ ↪ network_send_ ↪ bytes_rate		字节/秒	计算得出的所有网卡中，最大的发送速率		P0
doris_be_memory_ ↪ pgpgin		字节	系统从磁盘写到内存页的数据量		
doris_be_memory_ ↪ pgpgout		字节	系统内存页写入磁盘的数据量		

名称	标签	单位	含义	说明	等级
doris_be_memory_pswpin		字节	系统从磁盘换入到内存的数量	通常情况下，swap 应该关闭，因此这个数值应该是 0	
doris_be_memory_pswpout		字节	系统从内存换入到磁盘的数量	通常情况下，swap 应该关闭，因此这个数值应该是 0	
doris_be_network_receive_bytes		字节	各个网卡的接收字节累计。采集自 /proc/net/dev		
doris_be_network_receive_packets		Num	各个网卡的接收包个数累计。采集自 /proc/net/dev		
doris_be_network_send_bytes		字节	各个网卡的发送字节累计。采集自 /proc/net/dev		
doris_be_network_send_packets		Num	各个网卡的发送包个数累计。采集自 /proc/net/dev		
doris_be_proc	{mode="ctxt_switch"}	Num	CPU 上下文切换的累计值。采集自 /proc/stat	观测是否有异常的上下文切换	P0
doris_be_proc	{mode="interrupt"}	Num	CPU 中断次数的累计值。采集自 /proc/stat		
doris_be_proc	{mode="procs_blocked"}	Num	系统当前被阻塞的进程数（如等待 IO）。采集自 /proc/stat		
doris_be_proc	{mode="procs_running"}	Num	系统当前正在执行的进程数。采集自 /proc/stat		
doris_be_snmp_tcp_in_errs		Num	tcp 包接收错误的次数。采集自 /proc/net/snmp	可观测网络错误如重传、丢包等。需和其他 snmp 指标配合使用	P0
doris_be_snmp_tcp_in_segs		Num	tcp 包发送的个数。采集自 /proc/net/snmp		
doris_be_snmp_tcp_out_segs		Num	tcp 包发送的个数。采集自 /proc/net/snmp		

名称	标签	单位	含义	说明	等级
doris_be_snmp_ ↪ tcp_retrans_ ↪ segs		Num	tcp 包重传的个数。采集自 /proc/net/snmp		

4.5.2 监控和报警

本文档主要介绍 Doris 的监控项及如何采集、展示监控项。以及如何配置报警（TODO）

Dashboard 模板点击下载

Doris 版本	Dashboard 版本
1.2.x	revision 5

Dashboard 模板会不定期更新。更新模板的方式见最后一小节。

欢迎提供更优的 dashboard。

4.5.2.1 组件

Doris 使用 Prometheus 和 Grafana 进行监控项的采集和展示。

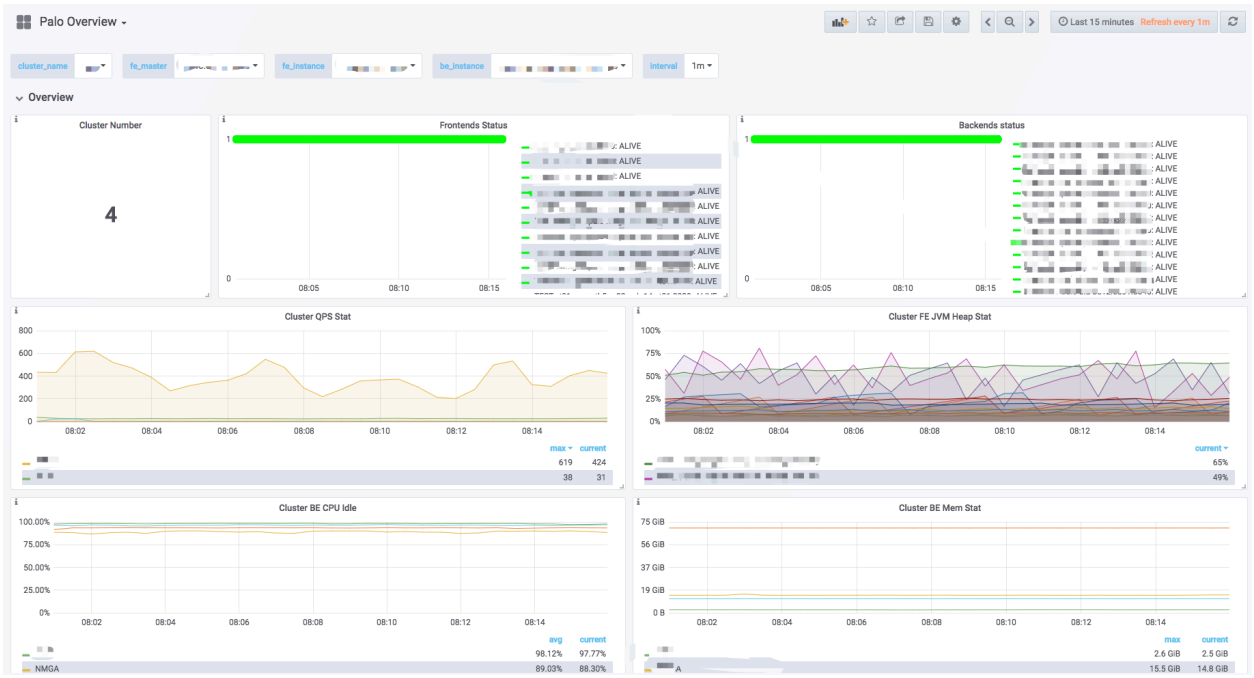


图 141: 组件

1. Prometheus

Prometheus 是一款开源的系统监控和报警套件。它可以通过 Pull 或 Push 采集被监控系统的监控项，存入自身的时序数据库中。并且通过丰富的多维数据查询语言，满足用户的不同数据展示需求。

2. Grafana

   Grafana 是一款开源的数据分析和展示平台。支持包括 Prometheus 在内的多个主流时序数据库源。通过对应的数据库查询语句，从数据源中获取展现数据。通过灵活可配置的 Dashboard，快速的将这些数据以图表的形式展示给用户。

> 注：本文档仅提供一种使用 Prometheus 和 Grafana 进行 Doris 监控数据采集和展示的方式。原则上不开发、维护这些组件。更多关于这些组件的详细介绍，请移步对应官方文档进行查阅。

4.5.2.2 监控数据

Doris 的监控数据通过 Frontend 和 Backend 的 http 接口向外暴露。监控数据以 Key-Value 的文本形式对外展现。每个 Key 还可能有不同的 Label 加以区分。当用户搭建好 Doris 后，可以在浏览器，通过以下接口访问到节点的监控数据：

- Frontend: `fe_host:fe_http_port/metrics`
- Backend: `be_host:be_web_server_port/metrics`
- Broker: 暂不提供

用户将看到如下监控项结果（示例为 FE 部分监控项）：

```
### HELP  jvm_heap_size_bytes jvm heap stat
### TYPE  jvm_heap_size_bytes gauge
jvm_heap_size_bytes{type="max"} 8476557312
jvm_heap_size_bytes{type="committed"} 1007550464
jvm_heap_size_bytes{type="used"} 156375280
### HELP  jvm_non_heap_size_bytes jvm non heap stat
### TYPE  jvm_non_heap_size_bytes gauge
jvm_non_heap_size_bytes{type="committed"} 194379776
jvm_non_heap_size_bytes{type="used"} 188201864
### HELP  jvm_young_size_bytes jvm young mem pool stat
### TYPE  jvm_young_size_bytes gauge
jvm_young_size_bytes{type="used"} 40652376
jvm_young_size_bytes{type="peak_used"} 277938176
jvm_young_size_bytes{type="max"} 907345920
### HELP  jvm_old_size_bytes jvm old mem pool stat
### TYPE  jvm_old_size_bytes gauge
jvm_old_size_bytes{type="used"} 114633448
jvm_old_size_bytes{type="peak_used"} 114633448
jvm_old_size_bytes{type="max"} 7455834112
### HELP  jvm_gc jvm gc stat
```

```
### TYPE  jvm_gc gauge
<GarbageCollector>{type="count"} 247
<GarbageCollector>{type="time"} 860
### HELP  jvm_thread jvm thread stat
### TYPE  jvm_thread gauge
jvm_thread{type="count"} 162
jvm_thread{type="peak_count"} 205
jvm_thread{type="new_count"} 0
jvm_thread{type="runnable_count"} 48
jvm_thread{type="blocked_count"} 1
jvm_thread{type="waiting_count"} 41
jvm_thread{type="timed_waiting_count"} 72
jvm_thread{type="terminated_count"} 0
...
```

这是一个以 Prometheus 格式 呈现的监控数据。我们以其中一个监控项为例进行说明：

```
### HELP  jvm_heap_size_bytes jvm heap stat
### TYPE  jvm_heap_size_bytes gauge
jvm_heap_size_bytes{type="max"} 8476557312
jvm_heap_size_bytes{type="committed"} 1007550464
jvm_heap_size_bytes{type="used"} 156375280
```

1. “#”开头的行为注释行。其中 HELP 为该监控项的描述说明；TYPE 表示该监控项的数据类型，示例中为 Gauge，即标量数据。还有 Counter、 Histogram 等数据类型。具体可见 Prometheus 官方文档 。
2. `jvm_heap_size_bytes` 即监控项的名称（ Key ）；`type="max"` 即为一个名为 `type` 的 Label，值为 `max`。一个监控项可以有多个 Label。
3. 最后的数字，如 `8476557312`，即为监控数值。

4.5.2.3 监控架构

整个监控架构如下图所示：

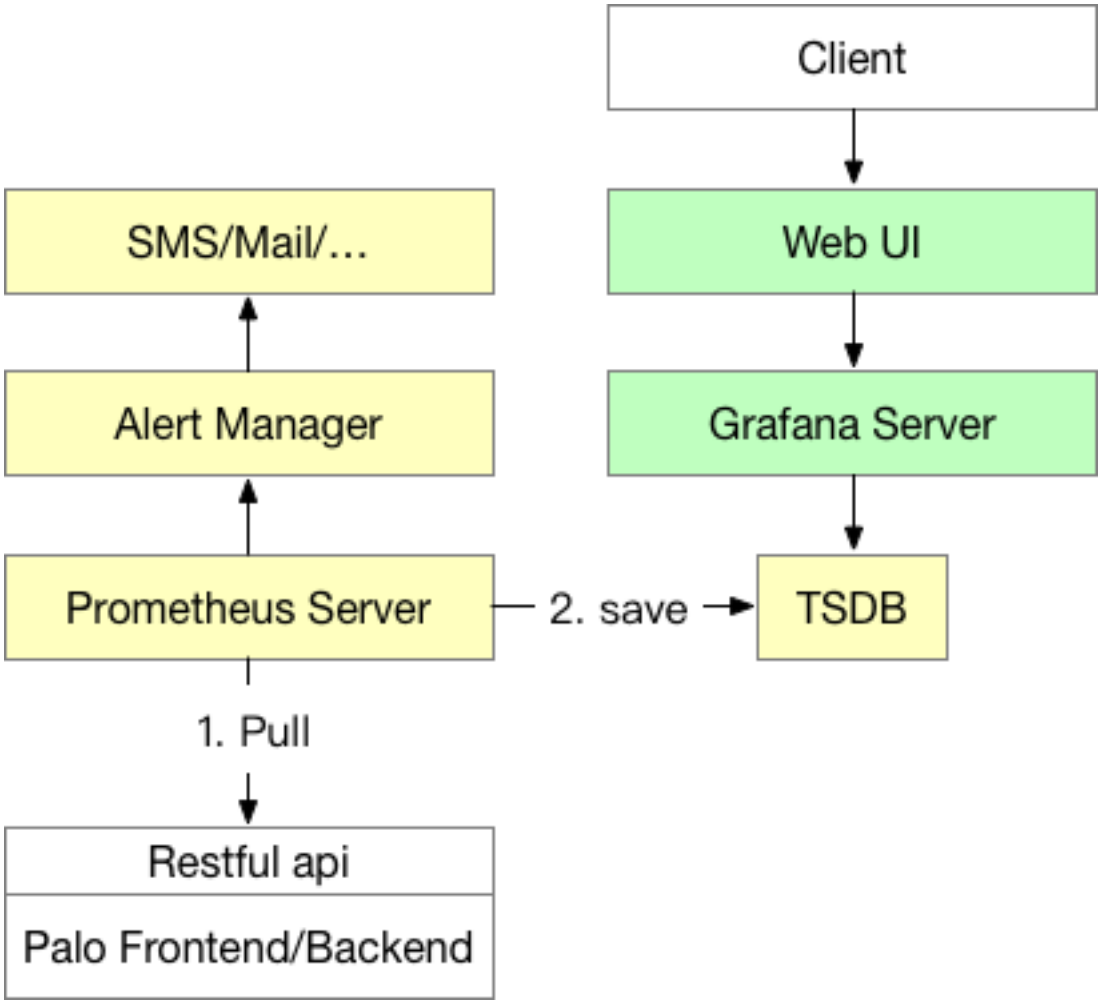


图 142: 监控架构

1. 黄色部分为 Prometheus 相关组件。Prometheus Server 为 Prometheus 的主进程，目前 Prometheus 通过 Pull 的方式访问 Doris 节点的监控接口，然后将时序数据存入时序数据库 TSDB 中（TSDB 包含在 Prometheus 进程中，无需单独部署）。Prometheus 也支持通过搭建 Push Gateway 的方式，允许被监控系统将监控数据通过 Push 的方式推到 Push Gateway，再由 Prometheus Server 通过 Pull 的方式从 Push Gateway 中获取数据。
2. Alert Manager 为 Prometheus 报警组件，需单独部署（暂不提供方案，可参照官方文档自行搭建）。通过 Alert Manager，用户可以配置报警策略，接收邮件、短信等报警。
3. 绿色部分为 Grafana 相关组件。Grafana Server 为 Grafana 的主进程。启动后，用户可以通过 Web 页面对 Grafana 进行配置，包括数据源的设置、用户设置、Dashboard 绘制等。这里也是最终用户查看监控数据的地方。

4.5.2.4 开始搭建

请在完成 Doris 的部署后，开始搭建监控系统。

4.5.2.4.1 Prometheus

1. 在 Prometheus 官网 下载最新版本的 Prometheus。这里我们以 2.43.0-linux-amd64 版本为例。

2. 在准备运行监控服务的机器上，解压下载后的 tar 文件。

3. 打开配置文件 prometheus.yml。这里我们提供一个示例配置并加以说明（配置文件为 yml 格式，一定注意统一的缩进和空格）：

   这里我们使用最简单的静态文件的方式进行监控配置。Prometheus 支持多种 服务发现 方式，可以动态的感知节点的加入和删除。

```
# my global config
global:
  scrape_interval:     15s # 全局的采集间隔，默认是 1m，这里设置为 15s
  evaluation_interval: 15s # 全局的规则触发间隔，默认是 1m，这里设置 15s

# Alertmanager configuration
alerting:
  alertmanagers:
  - static_configs:
    - targets:
      # - alertmanager:9093

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this
      ↪ config.
  - job_name: 'DORIS_CLUSTER' # 每一个 Doris 集群，我们称为一个 job。这里可以给 job
      ↪ 取一个名字，作为 Doris 集群在监控系统中的名字。
    metrics_path: '/metrics' # 这里指定获取监控项的 restful api。配合下面的 targets 中的 host
        ↪ :port，Prometheus 最终会通过 host:port/metrics_path 来采集监控项。
    static_configs: # 这里开始分别配置 FE 和 BE 的目标地址。所有的 FE 和 BE 都分别写入各自的
        ↪ group 中。
      - targets: ['fe_host1:8030', 'fe_host2:8030', 'fe_host3:8030']
        labels:
          group: fe # 这里配置了 fe 的 group，该 group 中包含了 3 个 Frontends

      - targets: ['be_host1:8040', 'be_host2:8040', 'be_host3:8040']
        labels:
          group: be # 这里配置了 be 的 group，该 group 中包含了 3 个 Backends

  - job_name: 'DORIS_CLUSTER_2' # 我们可以在一个 Prometheus 中监控多个 Doris 集群，
      ↪ 这里开始另一个 Doris 集群的配置。配置同上，以下略。
    metrics_path: '/metrics'
    static_configs:
      - targets: ['fe_host1:8030', 'fe_host2:8030', 'fe_host3:8030']
        labels:
          group: fe
```

```
      - targets: ['be_host1:8040', 'be_host2:8040', 'be_host3:8040']
        labels:
          group: be
```

4. 启动 Prometheus

   通过以下命令启动 Prometheus：

   ```
   nohup ./prometheus --web.listen-address="0.0.0.0:8181" &
   ```

   该命令将后台运行 Prometheus，并指定其 web 端口为 8181。启动后，即开始采集数据，并将数据存放在 data 目录中。

5. 停止 Prometheus

   目前没有发现正式的进程停止方式，直接 kill -9 即可。当然也可以将 Prometheus 设为一种 service，以 service 的方式启停。

6. 访问 Prometheus

   Prometheus 可以通过 web 页面进行简单的访问。通过浏览器打开 8181 端口，即可访问 Prometheus 的页面。点击导航栏中，`Status -> Targets`，可以看到所有分组 Job 的监控主机节点。正常情况下，所有节点都应为 UP，表示数据采集正常。点击某一个 Endpoint，即可看到当前的监控数值。如果节点状态不为 UP，可以先访问 Doris 的 metrics 接口（见前文）检查是否可以访问，或查询 Prometheus 相关文档尝试解决。

7. 至此，一个简单的 Prometheus 已经搭建、配置完毕。更多高级使用方式，请参阅 官方文档

4.5.2.4.2 Grafana

1. 在 Grafana 官网 下载最新版本的 Grafana。这里我们以 8.5.22.linux-amd64 版本为例。
2. 在准备运行监控服务的机器上，解压下载后的 tar 文件。
3. 打开配置文件 conf/defaults.ini。这里我们仅列举需要改动的配置项，其余配置可使用默认。

```
# Path to where grafana can store temp files, sessions, and the sqlite3 db (if that is used)
data = data

# Directory where grafana can store logs
logs = data/log

# Protocol (http, https, socket)
protocol = http

# The ip address to bind to, empty will bind to all interfaces
http_addr =

# The http port to use
http_port = 8182
```

4. 启动 Grafana

   通过以下命令启动 Grafana

   `nohup ./bin/grafana-server &`

   该命令将后台运行 Grafana，访问端口为上面配置的 8182

5. 停止 Grafana

   目前没有发现正式的进程停止方式，直接 kill -9 即可。当然也可以将 Grafana 设为一种 service，以 service 的方式启停。

6. 访问 Grafana

   通过浏览器，打开 8182 端口，可以开始访问 Grafana 页面。默认用户名密码为 admin。

7. 配置 Grafana

   初次登陆，需要根据提示设置数据源（data source）。我们这里的数据源，即上一步配置的 Prometheus。

   数据源配置的 Setting 页面说明如下：

   1. Name: 数据源的名称，自定义，比如 doris_monitor_data_source
   2. Type: 选择 Prometheus
   3. URL: 填写 Prometheus 的 web 地址，如 http://host:8181
   4. Access: 这里我们选择 Server 方式，即通过 Grafana 进程所在服务器，访问 Prometheus。
   5. 其余选项默认即可。
   6. 点击最下方 `Save & Test`，如果显示 `Data source is working`，即表示数据源可用。
   7. 确认数据源可用后，点击左边导航栏的 + 号，开始添加 Dashboard。这里我们已经准备好了 Doris 的 Dashboard 模板（本文档开头）。下载完成后，点击上方的 `New dashboard->Import dashboard->Upload .json File`，将下载的 json 文件导入。
   8. 导入后，可以命名 Dashboard，默认是 `Doris Overview`。同时，需要选择数据源，这里选择之前创建的 `doris_monitor_data_source`
   9. 点击 `Import`，即完成导入。之后，可以看到 Doris 的 Dashboard 展示。

8. 至此，一个简单的 Grafana 已经搭建、配置完毕。更多高级使用方式，请参阅 官方文档

4.5.2.5 Dashboard 说明

这里我们简要介绍 Doris Dashboard。Dashboard 的内容可能会随版本升级，不断变化，本文档不保证是最新的 Dashboard 说明。

1. 顶栏

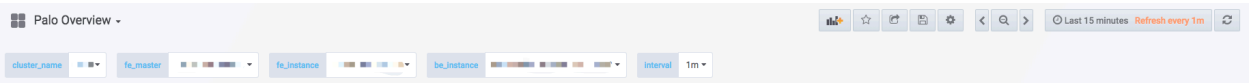


图 143: 顶栏

- 左上角为 Dashboard 名称。
- 右上角显示当前监控时间范围，可以下拉选择不同的时间范围，还可以指定定时刷新页面间隔。

- cluster_name: 即 Prometheus 配置文件中的各个 job_name，代表一个 Doris 集群。选择不同的 cluster，下方的图表将展示对应集群的监控信息。
- fe_master: 对应集群的 Master Frontend 节点。
- fe_instance: 对应集群的所有 Frontend 节点。选择不同的 Frontend，下方的图表将展示对应 Frontend 的监控信息。
- be_instance: 对应集群的所有 Backend 节点。选择不同的 Backend，下方的图表将展示对应 Backend 的监控信息。
- interval: 有些图表展示了速率相关的监控项，这里可选择以多大间隔进行采样计算速率（注：15s 间隔可能导致一些图表无法显示）。

2. Row

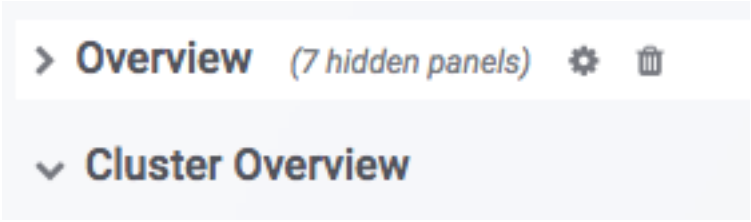


图 144: Row

Grafana 中，Row 的概念，即一组图表的集合。如上图中的 Overview、Cluster Overview 即两个不同的 Row。可以通过点击 Row，对 Row 进行折叠。当前 Dashboard 有如下 Rows（持续更新中）：

1. Overview: 所有 Doris 集群的汇总展示。
2. Cluster Overview: 选定集群的汇总展示。
3. Query Statistic: 选定集群的查询相关监控。
4. FE JVM: 选定 Frontend 的 JVM 监控。
5. BE: 选定集群的 Backends 的汇总展示。
6. BE Task: 选定集群的 Backends 任务信息的展示。

3. 图表

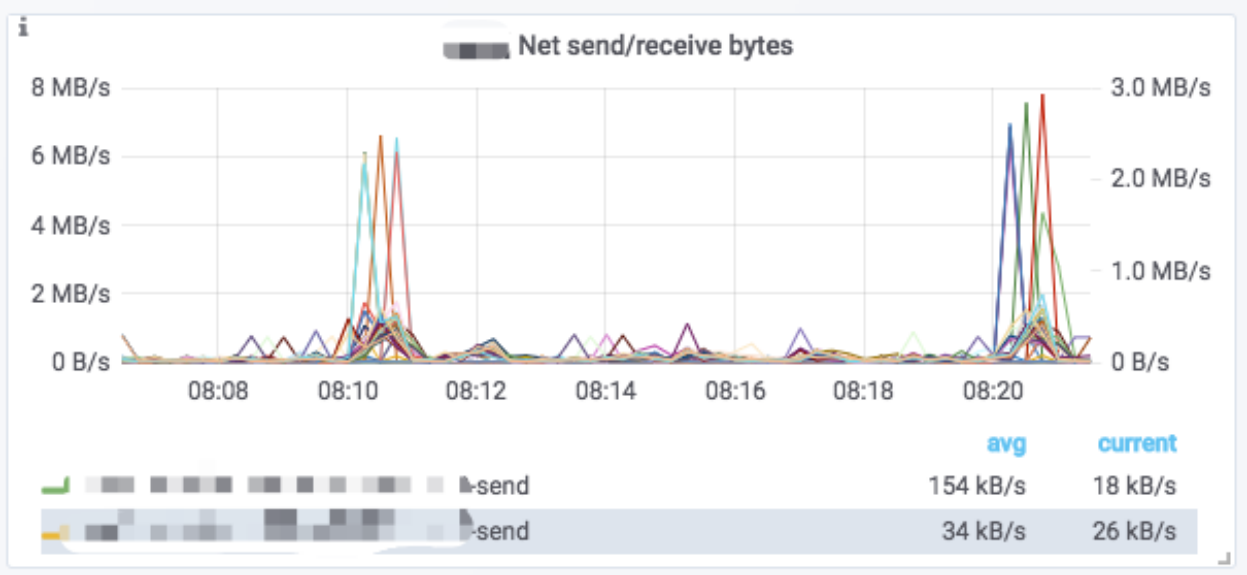


图 145: 图表

一个典型的图标分为以下几部分：

1. 鼠标悬停左上角的 i 图标，可以查看该图表的说明。
2. 点击下方的图例，可以单独查看某一监控项。再次点击，则显示所有。
3. 在图表中拖拽可以选定时间范围。
4. 标题的 [] 中显示选定的集群名称。
5. 一些数值对应左边的 Y 轴，一些对应右边的，可以通过图例末尾的 `-right` 区分。
6. 点击图表名称 `->Edit`，可以对图表进行编辑。

4.5.2.6 Dashboard 更新

1. 点击 Grafana 左边栏的 `+`，点击 `Dashboard`。
2. 点击左上角的 `New dashboard`，在点击右侧出现的 `Import dashboard`。
3. 点击 `Upload .json File`，选择最新的模板文件。
4. 选择数据源
5. 点击 `Import(Overwrite)`，完成模板更新。

4.5.3 磁盘空间管理

本文档主要介绍和磁盘存储空间有关的系统参数和处理策略。

Doris 的数据磁盘空间如果不加以控制，会因磁盘写满而导致进程挂掉。因此我们监测磁盘的使用率和剩余空间，通过设置不同的警戒水位，来控制 Doris 系统中的各项操作，尽量避免发生磁盘被写满的情况。

4.5.3.1 名词解释

- Data Dir：数据目录，在 BE 配置文件 be.conf 的 storage_root_path 中指定的各个数据目录。通常一个数据目录对应一个磁盘、因此下文中 磁盘也指代一个数据目录。

4.5.3.2 基本原理

BE 定期（每隔一分钟）会向 FE 汇报一次磁盘使用情况。FE 记录这些统计值，并根据这些统计值，限制不同的操作请求。

在 FE 中分别设置了 高水位（High Watermark）和 危险水位（Flood Stage）两级阈值。危险水位高于高水位。当磁盘使用率高于高水位时，Doris 会限制某些操作的执行（如副本均衡等）。而如果高于危险水位，则会禁止某些操作的执行（如导入）。

同时，在 BE 上也设置了 危险水位（Flood Stage）。考虑到 FE 并不能完全及时的检测到 BE 上的磁盘使用情况，以及无法控制某些 BE 自身运行的操作（如 Compaction）。因此 BE 上的危险水位用于 BE 主动拒绝和停止某些操作，达到自我保护的目的。

4.5.3.3 FE 参数

高水位：

```
storage_high_watermark_usage_percent 默认 85 (85%)。
storage_min_left_capacity_bytes 默认 2GB。
```

当磁盘空间使用率大于 storage_high_watermark_usage_percent，或者磁盘空间剩余大小小于 storage_min_
↪ left_capacity_bytes 时，该磁盘不会再被作为以下操作的目的路径：

- Tablet 均衡操作（Balance）
- Colocation 表数据分片的重分布（Relocation）
- Decommission

危险水位：

```
storage_flood_stage_usage_percent 默认 95 (95%)。
storage_flood_stage_left_capacity_bytes 默认 1GB。
```

当磁盘空间使用率大于 storage_flood_stage_usage_percent，并且磁盘空间剩余大小小于 storage_flood_
↪ stage_left_capacity_bytes 时，该磁盘不会再被作为以下操作的目的路径，并禁止某些操作：

- Tablet 均衡操作（Balance）
- Colocation 表数据分片的重分布（Relocation）
- 副本补齐
- 恢复操作（Restore）
- 数据导入（Load/Insert）

4.5.3.4 BE 参数

危险水位：

```
storage_flood_stage_usage_percent 默认 90 (90%)。
storage_flood_stage_left_capacity_bytes 默认 1GB。
```

当磁盘空间使用率大于 storage_flood_stage_usage_percent，并且磁盘空间剩余大小小于 storage_flood_
↪ stage_left_capacity_bytes 时，该磁盘上的以下操作会被禁止：

- Base/Cumulative Compaction。
- 数据写入。包括各种导入操作。
- Clone Task。通常发生于副本修复或均衡时。
- Push Task。发生在 Hadoop 导入的 Loading 阶段，下载文件。
- Alter Task。Schema Change 或 Rollup 任务。
- Download Task。恢复操作的 Downloading 阶段。

4.5.3.5 磁盘空间释放

当磁盘空间高于高水位甚至危险水位后，很多操作都会被禁止。此时可以尝试通过以下方式减少磁盘使用率，恢复系统。

- 删除表或分区

通过删除表或分区的方式，能够快速降低磁盘空间使用率，恢复集群。注意：只有 DROP 操作可以达到快速降低磁盘空间使用率的目的，DELETE 操作不可以。

```
DROP TABLE tbl;
ALTER TABLE tbl DROP PARTITION p1;
```

- 扩容 BE

扩容后，数据分片会自动均衡到磁盘使用率较低的 BE 节点上。扩容操作会根据数据量及节点数量不同，在数小时或数天后使集群到达均衡状态。

- 修改表或分区的副本

可以将表或分区的副本数降低。比如默认 3 副本可以降低为 2 副本。该方法虽然降低了数据的可靠性，但是能够快速的降低磁盘使用率，使集群恢复正常。该方法通常用于紧急恢复系统。请在恢复后，通过扩容或删除数据等方式，降低磁盘使用率后，将副本数恢复为 3。

修改副本操作为瞬间生效，后台会自动异步的删除多余的副本。

```
ALTER TABLE tbl MODIFY PARTITION p1 SET("replication_num" = "2");
```

- 删除多余文件

当 BE 进程已经因为磁盘写满而挂掉并无法启动时（此现象可能因 FE 或 BE 检测不及时而发生）。需要通过删除数据目录下的一些临时文件，保证 BE 进程能够启动。以下目录中的文件可以直接删除：

- log/：日志目录下的日志文件。
- snapshot/: 快照目录下的快照文件。
- trash/：回收站中的文件。

这种操作会对从 BE 回收站中恢复数据 产生影响。

如果 BE 还能够启动，则可以使用ADMIN CLEAN TRASH ON(BackendHost:BackendHeartBeatPort);来主动清理临时文件，会清理 所有 trash 文件和过期 snapshot 文件，这将影响从回收站恢复数据的操作。

如果不手动执行ADMIN CLEAN TRASH，系统仍将会在几分钟至几十分钟内自动执行清理，这里分为两种情况：

- 如果磁盘占用未达到 危险水位 (Flood Stage) 的 90%，则会清理过期 trash 文件和过期 snapshot 文件，此时会保留一些近期文件而不影响恢复数据。
- 如果磁盘占用已达到 危险水位 (Flood Stage) 的 90%，则会清理 所有 trash 文件和过期 snapshot 文件，此时会影响从回收站恢复数据的操作。自动执行的时间间隔可以通过配置项中的max_garbage_sweep_interval
  ↪ 和min_garbage_sweep_interval更改。

出现由于缺少 trash 文件而导致恢复失败的情况时，可能返回如下结果：

```
{"status": "Fail","msg": "can find tablet path in trash"}
```

- 删除数据文件（危险！！！）

当以上操作都无法释放空间时，需要通过删除数据文件来释放空间。数据文件在指定数据目录的 data/ 目录下。删除数据分片（Tablet）必须先确保该 Tablet 至少有一个副本是正常的，否则删除唯一副本会导致数据丢失。假设我们要删除 id 为 12345 的 Tablet：

- 找到 Tablet 对应的目录，通常位于 data/shard_id/tablet_id/ 下。如：

  data/0/12345/

- 记录 tablet id 和 schema hash。其中 schema hash 为上一步目录的下一级目录名。如下为 352781111：

  data/0/12345/352781111

- 删除数据目录：

  rm -rf data/0/12345/

- 删除 Tablet 元数据（具体参考 Tablet 元数据管理工具）

  ./lib/meta_tool --operation=delete_header --root_path=/path/to/root_path --tablet_id=12345
  ↪ --schema_hash= 352781111

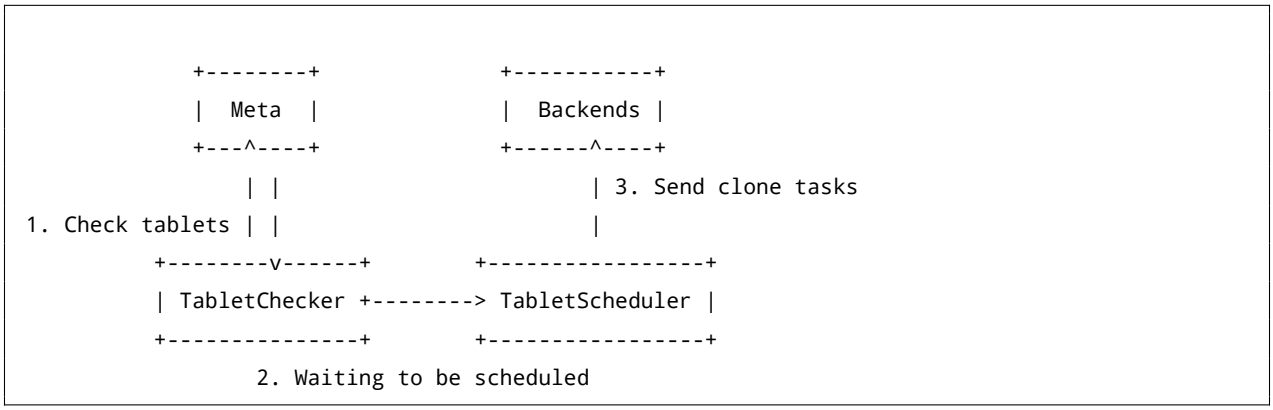
4.5.4 数据副本管理

从 0.9.0 版本开始，Doris 引入了优化后的副本管理策略，同时支持了更为丰富的副本状态查看工具。本文档主要介绍 Doris 数据副本均衡、修复方面的调度策略，以及副本管理的运维方法。帮助用户更方便的掌握和管理集群中的副本状态。

> Colocation 属性的表的副本修复和均衡可以参阅这里

4.5.4.1 名词解释

1. Tablet：Doris 表的逻辑分片，一个表有多个分片。
2. Replica：分片的副本，默认一个分片有 3 个副本。
3. Healthy Replica：健康副本，副本所在 Backend 存活，且副本的版本完整。
4. TabletChecker（TC）：是一个常驻的后台线程，用于定期扫描所有的 Tablet，检查这些 Tablet 的状态，并根据检查结果，决定是否将 tablet 发送给 TabletScheduler。
5. TabletScheduler（TS）：是一个常驻的后台线程，用于处理由 TabletChecker 发来的需要修复的 Tablet。同时也会进行集群副本均衡的工作。
6. TabletSchedCtx（TSC）：是一个 tablet 的封装。当 TC 选择一个 tablet 后，会将其封装为一个 TSC，发送给 TS。
7. Storage Medium：存储介质。Doris 支持对分区粒度指定不同的存储介质，包括 SSD 和 HDD。副本调度策略也是针对不同的存储介质分别调度的。



上图是一个简化的工作流程。

4.5.4.2 副本状态

一个 Tablet 的多个副本，可能因为某些情况导致状态不一致。Doris 会尝试自动修复这些状态不一致的副本，让集群尽快从错误状态中恢复。

一个 Replica 的健康状态有以下几种：

1. BAD

   即副本损坏。包括但不限于磁盘故障、BUG 等引起的副本不可恢复的损毁状态。

2. VERSION_MISSING

   版本缺失。Doris 中每一批次导入都对应一个数据版本。而一个副本的数据由多个连续的版本组成。而由于导入错误、延迟等原因，可能导致某些副本的数据版本不完整。

3. HEALTHY

   健康副本。即数据正常的副本，并且副本所在的 BE 节点状态正常（心跳正常且不处于下线过程中）

一个 Tablet 的健康状态由其所有副本的状态决定，有以下几种：

1. REPLICA_MISSING

   副本缺失。即存活副本数小于期望副本数。

2. VERSION_INCOMPLETE

   存活副本数大于等于期望副本数，但其中健康副本数小于期望副本数。

3. REPLICA_RELOCATING

   拥有等于 replication num 的版本完整的存活副本数，但是部分副本所在的 BE 节点处于 unavailable 状态（比如 decommission 中）

4. REPLICA_MISSING_IN_CLUSTER

   当使用多 cluster 方式时，健康副本数大于等于期望副本数，但在对应 cluster 内的副本数小于期望副本数。

5. REDUNDANT

   副本冗余。健康副本都在对应 cluster 内，但数量大于期望副本数。或者有多余的 unavailable 副本。

6. FORCE_REDUNDANT

   这是一个特殊状态。只会出现在当已存在副本数大于等于可用节点数，可用节点数大于等于期望副本数，并且存活的副本数小于期望副本数。这种情况下，需要先删除一个副本，以保证有可用节点用于创建新副本。

7. COLOCATE_MISMATCH

   针对 Colocation 属性的表的分片状态。表示分片副本与 Colocation Group 的指定的分布不一致。

8. COLOCATE_REDUNDANT

   针对 Colocation 属性的表的分片状态。表示 Colocation 表的分片副本冗余。

9. HEALTHY

   健康分片，即条件 [1-8] 都不满足。

4.5.4.3 副本修复

TabletChecker 作为常驻的后台进程，会定期检查所有分片的状态。对于非健康状态的分片，将会交给 TabletScheduler 进行调度和修复。修复的实际操作，都由 BE 上的 clone 任务完成。FE 只负责生成这些 clone 任务。

注 1：副本修复的主要思想是先通过创建或补齐使得分片的副本数达到期望值，然后再删除多余的副本。

注 2：一个 clone 任务就是完成从一个指定远端 BE 拷贝指定数据到指定目的端 BE 的过程。

针对不同的状态，我们采用不同的修复方式：

1. REPLICA_MISSING/REPLICA_RELOCATING

   选择一个低负载的，可用的 BE 节点作为目的端。选择一个健康副本作为源端。clone 任务会从源端拷贝一个完整的副本到目的端。对于副本补齐，我们会直接选择一个可用的 BE 节点，而不考虑存储介质。

2. VERSION_INCOMPLETE

   选择一个相对完整的副本作为目的端。选择一个健康副本作为源端。clone 任务会从源端尝试拷贝缺失的版本到目的端的副本。

3. REPLICA_MISSING_IN_CLUSTER

   这种状态处理方式和 REPLICA_MISSING 相同。

4. REDUNDANT

   通常经过副本修复后，分片会有冗余的副本。我们选择一个冗余副本将其删除。冗余副本的选择遵从以下优先级：

   1. 副本所在 BE 已经下线
   2. 副本已损坏
   3. 副本所在 BE 失联或在下线中
   4. 副本处于 CLONE 状态（该状态是 clone 任务执行过程中的一个中间状态）
   5. 副本有版本缺失
   6. 副本所在 cluster 不正确
   7. 副本所在 BE 节点负载高

5. FORCE_REDUNDANT

   不同于 REDUNDANT，因为此时虽然存活的副本数小于期望副本数，但是因为已经没有额外的可用节点用于创建新的副本了。所以此时必须先删除一个副本，以腾出一个可用节点用于创建新的副本。删除副本的顺序同 REDUNDANT。

6. COLOCATE_MISMATCH

   从 Colocation Group 中指定的副本分布 BE 节点中选择一个作为目的节点进行副本补齐。

7. COLOCATE_REDUNDANT

   删除一个非 Colocation Group 中指定的副本分布 BE 节点上的副本。

Doris 在选择副本节点时，不会将同一个 Tablet 的副本部署在同一个 host 的不同 BE 上。保证了即使同一个 host 上的所有 BE 都挂掉，也不会造成全部副本丢失。

4.5.4.3.1 调度优先级

TabletScheduler 里等待被调度的分片会根据状态不同，赋予不同的优先级。优先级高的分片将会被优先调度。目前有以下几种优先级。

1. VERY_HIGH
    - REDUNDANT。对于有副本冗余的分片，我们优先处理。虽然逻辑上来讲，副本冗余的紧急程度最低，但是因为这种情况处理起来最快且可以快速释放资源（比如磁盘空间等），所以我们优先处理。
    - FORCE_REDUNDANT。同上。
2. HIGH
    - REPLICA_MISSING 且多数副本缺失（比如 3 副本丢失了 2 个）
    - VERSION_INCOMPLETE 且多数副本的版本缺失
    - COLOCATE_MISMATCH 我们希望 Colocation 表相关的分片能够尽快修复完成。
    - COLOCATE_REDUNDANT
3. NORMAL
    - REPLICA_MISSING 但多数存活（比如 3 副本丢失了 1 个）
    - VERSION_INCOMPLETE 但多数副本的版本完整
    - REPLICA_RELOCATING 且多数副本需要 relocate（比如 3 副本有 2 个）
4. LOW
    - REPLICA_MISSING_IN_CLUSTER
    - REPLICA_RELOCATING 但多数副本 stable

4.5.4.3.2 手动优先级

系统会自动判断调度优先级。但是有些时候，用户希望某些表或分区的分片能够更快的被修复。因此我们提供一个命令，用户可以指定某个表或分区的分片被优先修复：

```
ADMIN REPAIR TABLE tbl [PARTITION (p1, p2, ...)];
```

这个命令，告诉 TC，在扫描 Tablet 时，对需要优先修复的表或分区中的有问题的 Tablet，给予 VERY_HIGH 的优先级。

> 注：这个命令只是一个 hint，并不能保证一定能修复成功，并且优先级也会随 TS 的调度而发生变化。并且当 Master FE 切换或重启后，这些信息都会丢失。

可以通过以下命令取消优先级：

```
ADMIN CANCEL REPAIR TABLE tbl [PARTITION (p1, p2, ...)];
```

4.5.4.3.3 优先级调度

优先级保证了损坏严重的分片能够优先被修复，提高系统可用性。但是如果高优先级的修复任务一直失败，则会导致低优先级的任务一直得不到调度。因此，我们会根据任务的运行状态，动态的调整任务的优先级，保证所有任务都有机会被调度到。

- 连续 5 次调度失败（如无法获取资源，无法找到合适的源端或目的端等），则优先级会被下调。
- 持续 30 分钟未被调度，则上调优先级。
- 同一 tablet 任务的优先级至少间隔 5 分钟才会被调整一次。

同时为了保证初始优先级的权重，我们规定，初始优先级为 VERY_HIGH 的，最低被下调到 NORMAL。而初始优先级为 LOW 的，最多被上调为 HIGH。这里的优先级调整，也会调整用户手动设置的优先级。

4.5.4.4 副本均衡

Doris 会自动进行集群内的副本均衡。目前支持两种均衡策略，负载/分区。负载均衡适合需要兼顾节点磁盘使用率和节点副本数量的场景；而分区均衡会使每个分区的副本都均匀分布在各个节点，避免热点，适合对分区读写要求比较高的场景。但是，分区均衡不考虑磁盘使用率，使用分区均衡时需要注意磁盘的使用情况。策略只能在 fe 启动前配置 tablet_rebalancer_type ，不支持运行时切换。

4.5.4.4.1 负载均衡

负载均衡的主要思想是，对某些分片，先在低负载的节点上创建一个副本，然后再删除这些分片在高负载节点上的副本。同时，因为不同存储介质的存在，在同一个集群内的不同 BE 节点上，可能存在一种或两种存储介质。我们要求存储介质为 A 的分片在均衡后，尽量依然存储在存储介质 A 中。所以我们根据存储介质，对集群的 BE 节点进行划分。然后针对不同的存储介质的 BE 节点集合，进行负载均衡调度。

同样，副本均衡会保证不会将同一个 Tablet 的副本部署在同一个 host 的 BE 上。

BE 节点负载

我们用 ClusterLoadStatistics（CLS）表示一个 cluster 中各个 Backend 的负载均衡情况。TabletScheduler 根据这个统计值，来触发集群均衡。我们当前通过 磁盘使用率和 副本数量两个指标，为每个 BE 计算一个 loadScore，作为 BE 的负载分数。分数越高，表示该 BE 的负载越重。

磁盘使用率和副本数量各有一个权重系数，分别为 capacityCoefficient 和 replicaNumCoefficient，其 和恒为 1。其中 capacityCoefficient 会根据实际磁盘使用率动态调整。当一个 BE 的总体磁盘使用率在 50% 以下，则 capacityCoefficient 值为 0.5，如果磁盘使用率在 75%（可通过 FE 配置项 `capacity_used_percent_high_water` 配置）以上，则值为 1。如果使用率介于 50% ~ 75% 之间，则该权重系数平滑增加，公式为：

`capacityCoefficient= 2 * 磁盘使用率 - 0.5`

该权重系数保证当磁盘使用率过高时，该 Backend 的负载分数会更高，以保证尽快降低这个 BE 的负载。

TabletScheduler 会每隔 20s 更新一次 CLS。

4.5.4.4.2 分区均衡

分区均衡的主要思想是，将每个分区的在各个 Backend 上的 replica 数量差（即 partition skew），减少到最小。因此只考虑副本个数，不考虑磁盘使用率。为了尽量少的迁移次数，分区均衡使用二维贪心的策略，优先均

衡 partition skew 最大的分区，均衡分区时会尽量选择，可以使整个 cluster 的在各个 Backend 上的 replica 数量差（即 cluster skew/total skew）减少的方向。

skew 统计

skew 统计信息由ClusterBalanceInfo表示，其中，partitionInfoBySkew以 partition skew 为 key 排序，便于找到 max partition skew；beByTotalReplicaCount则是以 Backend 上的所有 replica 个数为 key 排序。ClusterBalanceInfo同样保持在 CLS 中，同样 20s 更新一次。

max partition skew 的分区可能有多个，采用随机的方式选择一个分区计算。

4.5.4.4.3 均衡策略

TabletScheduler 在每轮调度时，都会通过 LoadBalancer 来选择一定数目的健康分片作为 balance 的候选分片。在下一次调度时，会尝试根据这些候选分片，进行均衡调度。

4.5.4.5 资源控制

无论是副本修复还是均衡，都是通过副本在各个 BE 之间拷贝完成的。如果同一台 BE 同一时间执行过多的任务，则会带来不小的 IO 压力。因此，Doris 在调度时控制了每个节点上能够执行的任务数目。最小的资源控制单位是磁盘（即在 be.conf 中指定的一个数据路径）。我们默认为每块磁盘配置两个 slot 用于副本修复。一个 clone 任务会占用源端和目的端各一个 slot。如果 slot 数目为零，则不会再对这块磁盘分配任务。该 slot 个数可以通过 FE 的 `schedule_slot_num_per_hdd_path` 或者 `schedule_slot_num_per_ssd_path` 参数配置。

另外，我们默认为每块磁盘提供 2 个单独的 slot 用于均衡任务。目的是防止高负载的节点因为 slot 被修复任务占用，而无法通过均衡释放空间。

4.5.4.6 副本状态查看

副本状态查看主要是查看副本的状态，以及副本修复和均衡任务的运行状态。这些状态大部分都仅存在于 Master FE 节点中。因此，以下命令需直连到 Master FE 执行。

4.5.4.6.1 副本状态

1. 全局状态检查

   通过 `SHOW PROC '/cluster_health/tablet_health';` 命令可以查看整个集群的副本状态。

```
+-------+--------------------------------+-----------+------------+-------------------+----------------------
    ↪
| DbId  | DbName                         | TabletNum | HealthyNum | ReplicaMissingNum |
    ↪ VersionIncompleteNum | ReplicaRelocatingNum | RedundantNum |
    ↪ ReplicaMissingInClusterNum | ReplicaMissingForTagNum | ForceRedundantNum |
    ↪ ColocateMismatchNum | ColocateRedundantNum | NeedFurtherRepairNum | UnrecoverableNum
    ↪ | ReplicaCompactionTooSlowNum | InconsistentNum | OversizeNum | CloningNum |
+-------+--------------------------------+-----------+------------+-------------------+----------------------
    ↪
```

```
| 10005 | default_cluster:doris_audit_db | 84        | 84         | 0                | 0
    ↪                    | 0                    | 0           | 0
    ↪                          | 0                      | 0               | 0
    ↪                   | 0                    | 0                    | 0              |
    ↪ 0                           | 0              | 0          | 0         |
| 13402 | default_cluster:ssb1           | 709       | 708        | 1                | 0
    ↪                    | 0                    | 0           | 0
    ↪                          | 0                      | 0               | 0
    ↪                   | 0                    | 0                    | 0              |
    ↪ 0                           | 0              | 0          | 0         |
| 10108 | default_cluster:tpch1          | 278       | 278        | 0                | 0
    ↪                    | 0                    | 0           | 0
    ↪                          | 0                      | 0               | 0
    ↪                   | 0                    | 0                    | 0              |
    ↪ 0                           | 0              | 0          | 0         |
| Total | 3                              | 1071      | 1070       | 1                | 0
    ↪                    | 0                    | 0           | 0
    ↪                          | 0                      | 0               | 0
    ↪                   | 0                    | 0                    | 0              |
    ↪ 0                           | 0              | 0          | 0         |
+-------+--------------------------------+-----------+------------+------------------+----------------------
    ↪
```

其中 `HealthyNum` 列显示了对应的 Database 中，有多少 Tablet 处于健康状态。`
↪ ReplicaCompactionTooSlowNum` 列显示了对应的 Database 中，有多少 Tablet 的
↪ 处于副本版本数过多的状态， `InconsistentNum` 列显示了对应的 Database 中，有多少 Tablet
↪ 处于副本不一致的状态。最后一行 `Total` 行对整个集群进行了统计。正常情况下 `TabletNum` 和
↪ `HealthNum` 应该相等。如果不相等，可以进一步查看具体有哪些 Tablet。如上图中，ssb1
↪ 数据库有 1 个 Tablet 状态不健康，则可以使用以下命令查看具体是哪一个 Tablet。

`SHOW PROC '/cluster_health/tablet_health/13402';`

其中 `13402` 为对应的 DbId。

```
+----------------------+-------------------------+-------------------------+-----------------+-----------
    ↪
| ReplicaMissingTablets | VersionIncompleteTablets | ReplicaRelocatingTablets |
    ↪ RedundantTablets | ReplicaMissingInClusterTablets | ReplicaMissingForTagTablets |
    ↪ ForceRedundantTablets | ColocateMismatchTablets | ColocateRedundantTablets |
    ↪ NeedFurtherRepairTablets | UnrecoverableTablets | ReplicaCompactionTooSlowTablets |
    ↪ InconsistentTablets | OversizeTablets |
+----------------------+-------------------------+-------------------------+-----------------+-----------
    ↪
| 14679                |                         |                         |
    ↪                  |                               |                          |
```

```
    ↪                       |                         |                          |
    ↪                          |                      |                              |
    ↪                       |                 |
+-----------------------+-------------------------+-------------------------+-----------------+-----------
    ↪
```

上图会显示具体的不健康的 Tablet ID（14679），该 Tablet 处于 ReplicaMissing 的状态。
↪ 后面我们会介绍如何查看一个具体的 Tablet 的各个副本的状态。

2. 表（分区）级别状态检查

用户可以通过以下命令查看指定表或分区的副本状态，并可以通过 WHERE 语句对状态进行过滤。如查看表 tbl1 中，分区 p1 和 p2 上状态为 OK 的副本：

```
SHOW REPLICA STATUS FROM tbl1 PARTITION (p1, p2)WHERE STATUS = "OK";
```

```
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+---
    ↪
| TabletId | ReplicaId | BackendId | Version | LastFailedVersion | LastSuccessVersion |
    ↪ CommittedVersion | SchemaHash | VersionNum | IsBad | State  | Status |
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+---
    ↪
| 29502429 | 29502432  | 10006     | 2       | -1                | 2                  | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
| 29502429 | 36885996  | 10002     | 2       | -1                | -1                 | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
| 29502429 | 48100551  | 10007     | 2       | -1                | -1                 | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
| 29502433 | 29502434  | 10001     | 2       | -1                | 2                  | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
| 29502433 | 44900737  | 10004     | 2       | -1                | -1                 | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
| 29502433 | 48369135  | 10006     | 2       | -1                | -1                 | 1
    ↪                | -1         | 2          | false | NORMAL | OK     |
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+---
    ↪
```

这里会展示所有副本的状态。其中 `IsBad` 列为 `true` 则表示副本已经损坏。而 `Status`
↪ 列则会显示另外的其他状态。具体的状态说明，可以通过 `HELP SHOW REPLICA STATUS;` 查看帮助。

`SHOW REPLICA STATUS` 命令主要用于查看副本的健康状态。
↪ 用户还可以通过以下命令查看指定表中副本的一些额外信息：

`SHOW TABLETS FROM tbl1;`

```
+----------+-----------+-----------+------------+---------+-------------+------------------+----------------
    ↪
| TabletId | ReplicaId | BackendId | SchemaHash | Version | VersionHash | LstSuccessVersion |
    ↪  LstSuccessVersionHash | LstFailedVersion | LstFailedVersionHash | LstFailedTime |
    ↪ DataSize | RowCount | State  | LstConsistencyCheckTime | CheckVersion |
    ↪ CheckVersionHash | VersionCount | PathHash             | MetaUrl              |
    ↪ CompactionStatus     |
+----------+-----------+-----------+------------+---------+-------------+------------------+----------------
    ↪
| 29502429 | 29502432  | 10006     | 1421156361 | 2       | 0           | 2                |
    ↪  0                    | -1               | 0                    | N/A           |
    ↪ 784      | 0        | NORMAL | N/A                      | -1           |     -1
    ↪               | 2            | -5822326203532286804 | url                  | url
    ↪                  |
| 29502429 | 36885996  | 10002     | 1421156361 | 2       | 0           | -1               |
    ↪  0                    | -1               | 0                    | N/A           |
    ↪ 784      | 0        | NORMAL | N/A                      | -1           |     -1
    ↪               | 2            | -1441285706148429853 | url                  | url
    ↪                  |
| 29502429 | 48100551  | 10007     | 1421156361 | 2       | 0           | -1               |
    ↪  0                    | -1               | 0                    | N/A           |
    ↪ 784      | 0        | NORMAL | N/A                      | -1           |     -1
    ↪               | 2            | -4784691547051455525 | url                  | url
    ↪                  |
+----------+-----------+-----------+------------+---------+-------------+------------------+----------------
    ↪
```

上图展示了包括副本大小、行数、版本数量、所在数据路径等一些额外的信息。

> 注：这里显示的 `State` 列的内容不代表副本的健康状态，而是副本处于某种任务下的状态，比如 CLONE、
    ↪ SCHEMA_CHANGE、ROLLUP 等。

此外，用户也可以通过以下命令，查看指定表或分区的副本分布情况，来检查副本分布是否均匀。

`SHOW REPLICA DISTRIBUTION FROM tbl1;`

```
+-----------+------------+-------+---------+
| BackendId | ReplicaNum | Graph | Percent |
+-----------+------------+-------+---------+
| 10000     | 7          |       | 7.29 %  |
| 10001     | 9          |       | 9.38 %  |
| 10002     | 7          |       | 7.29 %  |
| 10003     | 7          |       | 7.29 %  |
| 10004     | 9          |       | 9.38 %  |
```

```
| 10005     | 11          | >     | 11.46 % |
| 10006     | 18          | >     | 18.75 % |
| 10007     | 15          | >     | 15.62 % |
| 10008     | 13          | >     | 13.54 % |
+-----------+-------------+-------+---------+
```

这里分别展示了表 tbl1 的副本在各个 BE 节点上的个数、百分比，以及一个简单的图形化显示。

4. Tablet 级别状态检查

   当我们要定位到某个具体的 Tablet 时，可以使用如下命令来查看一个具体的 Tablet 的状态。如查看 ID 为 29502553 的 tablet：

   ```
   SHOW TABLET 29502553;
   ```

```
+------------------------+-----------+---------------+-----------+----------+----------+-------------+-------
    ↪
| DbName                 | TableName | PartitionName | IndexName | DbId     | TableId  |
    ↪ PartitionId | IndexId  | IsSync | DetailCmd
    ↪                                                                    |
+------------------------+-----------+---------------+-----------+----------+----------+-------------+-------
    ↪
| default_cluster:test   | test      | test          | test      | 29502391 | 29502428 |
    ↪ 29502427    | 29502428 | true   | SHOW PROC '/dbs/29502391/29502428/partitions
    ↪ /29502427/29502428/29502553'; |
+------------------------+-----------+---------------+-----------+----------+----------+-------------+-------
    ↪
```

上图显示了这个 tablet 所对应的数据库、表、分区、上卷表等信息。用户可以复制 `DetailCmd` ↪ 命令中的命令继续执行：

```
`SHOW PROC '/dbs/29502391/29502428/partitions/29502427/29502428/29502553';`
```

```
+-----------+-----------+---------+-------------+------------------+----------------------+---------------
    ↪
| ReplicaId | BackendId | Version | VersionHash | LstSuccessVersion | LstSuccessVersionHash |
    ↪  LstFailedVersion | LstFailedVersionHash | LstFailedTime | SchemaHash | DataSize |
    ↪ RowCount | State  | IsBad | VersionCount | PathHash             | MetaUrl  |
    ↪ CompactionStatus |
+-----------+-----------+---------+-------------+------------------+----------------------+---------------
    ↪
| 43734060  | 10004     | 2       | 0           | -1               | 0                    |
    ↪  -1               | 0                    | N/A           | -1         | 784      | 0
    ↪        | NORMAL | false | 2            | -8566523878520798656 | url      | url
    ↪              |
```

```
| 29502555  | 10002     | 2       | 0           | 2                 | 0                     |
    ↪  -1               | 0                    | N/A           | -1         | 784      | 0
    ↪        | NORMAL | false | 2           | 1885826196444191611  | url      | url
    ↪              |
| 39279319  | 10007     | 2       | 0           | -1                | 0                     |
    ↪  -1               | 0                    | N/A           | -1         | 784      | 0
    ↪        | NORMAL | false | 2           | 1656508631294397870  | url      | url
    ↪              |
+-----------+-----------+---------+-------------+-------------------+-----------------------+----------------
    ↪
```

上图显示了对应 Tablet 的所有副本情况。这里显示的内容和 `SHOW TABLETS FROM tbl1;` 的内容相同。
↪ 但这里可以清楚的知道，一个具体的 Tablet 的所有副本的状态。

4.5.4.6.2 副本调度任务

1. 查看等待被调度的任务

   ```
   SHOW PROC '/cluster_balance/pending_tablets';
   ```

```
+----------+--------+-----------------+---------+----------+----------+-------+---------+--------+----------
    ↪
| TabletId | Type   | Status          | State   | OrigPrio | DynmPrio | SrcBe | SrcPath |
    ↪ DestBe | DestPath | Timeout | Create              | LstSched            | LstVisit
    ↪            | Finished | Rate | FailedSched | FailedRunning | LstAdjPrio          |
    ↪ VisibleVer | VisibleVerHash      | CmtVer | CmtVerHash          | ErrMsg
    ↪                         |
+----------+--------+-----------------+---------+----------+----------+-------+---------+--------+----------
    ↪
| 4203036  | REPAIR | REPLICA_MISSING | PENDING | HIGH     | LOW      | -1    | -1      | -1
    ↪     | -1       | 0       | 2019-02-21 15:00:20 | 2019-02-24 11:18:41 | 2019-02-24
    ↪ 11:18:41 | N/A      | N/A  | 2           | 0             | 2019-02-21 15:00:43 | 1
    ↪          | 0                   | 2      | 0                   | unable to find source
    ↪  replica |
+----------+--------+-----------------+---------+----------+----------+-------+---------+--------+----------
    ↪
```

各列的具体含义如下：

* TabletId：等待调度的 Tablet 的 ID。一个调度任务只针对一个 Tablet
* Type：任务类型，可以是 REPAIR（修复）或 BALANCE（均衡）
* Status：该 Tablet 当前的状态，如 REPLICA_MISSING（副本缺失）
* State：该调度任务的状态，可能为 PENDING/RUNNING/FINISHED/CANCELLED/TIMEOUT/UNEXPECTED
* OrigPrio：初始的优先级
* DynmPrio：当前动态调整后的优先级

* SrcBe：源端 BE 节点的 ID
* SrcPath：源端 BE 节点的路径的 hash 值
* DestBe：目的端 BE 节点的 ID
* DestPath：目的端 BE 节点的路径的 hash 值
* Timeout：当任务被调度成功后，这里会显示任务的超时时间，单位秒
* Create：任务被创建的时间
* LstSched：上一次任务被调度的时间
* LstVisit：上一次任务被访问的时间。这里“被访问”指包括被调度，
  ↪ 任务执行汇报等和这个任务相关的被处理的时间点
* Finished：任务结束时间
* Rate：clone 任务的数据拷贝速率
* FailedSched：任务调度失败的次数
* FailedRunning：任务执行失败的次数
* LstAdjPrio：上一次优先级调整的时间
* CmtVer/CmtVerHash/VisibleVer/VisibleVerHash：用于执行 clone 任务的 version 信息
* ErrMsg：任务被调度和运行过程中，出现的错误信息

2. 查看正在运行的任务

   SHOW PROC '/cluster_balance/running_tablets';

   其结果中各列的含义和 pending_tablets 相同。

3. 查看已结束任务

   SHOW PROC '/cluster_balance/history_tablets';

   我们默认只保留最近 1000 个完成的任务。其结果中各列的含义和 pending_tablets 相同。如果 State 列为 FINISHED，则说明任务正常完成。如果为其他，则可以根据 ErrMsg 列的错误信息查看具体原因。

4.5.4.7 集群负载及调度资源查看

1. 集群负载

   通过以下命令可以查看集群当前的负载情况：

   SHOW PROC '/cluster_balance/cluster_load_stat/location_default';

   首先看到的是对不同存储介质的划分：

```
+---------------+
| StorageMedium |
+---------------+
| HDD           |
| SSD           |
+---------------+
```

点击某一种存储介质，可以看到包含该存储介质的 BE 节点的均衡状态：

`SHOW PROC '/cluster_balance/cluster_load_stat/location_default/HDD';`

```
+----------+-----------------+-----------+--------------+---------------+-------------+------------+-------
    ↪
| BeId     | Cluster         | Available | UsedCapacity  | Capacity       | UsedPercent |
    ↪ ReplicaNum | CapCoeff | ReplCoeff | Score              | Class |
+----------+-----------------+-----------+--------------+---------------+-------------+------------+-------
    ↪
| 10003    | default_cluster | true      | 3477875259079 | 19377459077121 | 17.948      |
    ↪ 493477     | 0.5      | 0.5       | 0.9284678149967587 | MID   |
| 10002    | default_cluster | true      | 3607326225443 | 19377459077121 | 18.616      |
    ↪ 496928     | 0.5      | 0.5       | 0.948660871419998  | MID   |
| 10005    | default_cluster | true      | 3523518578241 | 19377459077121 | 18.184      |
    ↪ 545331     | 0.5      | 0.5       | 0.9843539990641831 | MID   |
| 10001    | default_cluster | true      | 3535547090016 | 19377459077121 | 18.246      |
    ↪ 558067     | 0.5      | 0.5       | 0.9981869446537612 | MID   |
| 10006    | default_cluster | true      | 3636050364835 | 19377459077121 | 18.764      |
    ↪ 547543     | 0.5      | 0.5       | 1.0011489897614072 | MID   |
| 10004    | default_cluster | true      | 3506558163744 | 15501967261697 | 22.620      |
    ↪ 468957     | 0.5      | 0.5       | 1.0228319835582569 | MID   |
| 10007    | default_cluster | true      | 4036460478905 | 19377459077121 | 20.831      |
    ↪ 551645     | 0.5      | 0.5       | 1.057279369420761  | MID   |
| 10000    | default_cluster | true      | 4369719923760 | 19377459077121 | 22.551      |
    ↪ 547175     | 0.5      | 0.5       | 1.0964036415787461 | MID   |
+----------+-----------------+-----------+--------------+---------------+-------------+------------+-------
    ↪
```

其中一些列的含义如下：

* Available：为 true 表示 BE 心跳正常，且没有处于下线中
* UsedCapacity：字节，BE 上已使用的磁盘空间大小
* Capacity：字节，BE 上总的磁盘空间大小
* UsedPercent：百分比，BE 上的磁盘空间使用率
* ReplicaNum：BE 上副本数量
* CapCoeff/ReplCoeff：磁盘空间和副本数的权重系数
* Score：负载分数。分数越高，负载越重
* Class：根据负载情况分类，LOW/MID/HIGH。均衡调度会将高负载节点上的副本迁往低负载节点

用户可以进一步查看某个 BE 上各个路径的使用率，比如 ID 为 10001 这个 BE：

`SHOW PROC '/cluster_balance/cluster_load_stat/location_default/HDD/10001';`

```
+------------------+-----------------+--------------+--------------+---------+--------+-------------------
    ↪
| RootPath         | DataUsedCapacity | AvailCapacity | TotalCapacity | UsedPct | State  |
    ↪ PathHash             |
```

```
+------------------+-----------------+--------------+--------------+---------+--------+-----------------
    ↪
| /home/disk4/palo | 498.757 GB      | 3.033 TB     | 3.525 TB     | 13.94 % | ONLINE |
    ↪ 4883406271918338267  |
| /home/disk3/palo | 704.200 GB      | 2.832 TB     | 3.525 TB     | 19.65 % | ONLINE |
    ↪ -5467083960906519443 |
| /home/disk1/palo | 512.833 GB      | 3.007 TB     | 3.525 TB     | 14.69 % | ONLINE |
    ↪ -7733211489989964053 |
| /home/disk2/palo | 881.955 GB      | 2.656 TB     | 3.525 TB     | 24.65 % | ONLINE |
    ↪ 4870995507205544622  |
| /home/disk5/palo | 694.992 GB      | 2.842 TB     | 3.525 TB     | 19.36 % | ONLINE |
    ↪ 1916696897889786739  |
+------------------+-----------------+--------------+--------------+---------+--------+-----------------
    ↪
```

这里显示了指定 BE 上，各个数据路径的磁盘使用率情况。

2. 调度资源

用户可以通过以下命令，查看当前各个节点的 slot 使用情况：

```
SHOW PROC '/cluster_balance/working_slots';
```

```
+----------+----------------------+------------+------------+-------------+----------------------+
    ↪
| BeId     | PathHash             | AvailSlots | TotalSlots | BalanceSlot | AvgRate
    ↪              |
+----------+----------------------+------------+------------+-------------+----------------------+
    ↪
| 10000    | 8110346074333016794  | 2          | 2          | 2           | 2.459007474009069
    ↪ E7  |
| 10000    | -5617618290584731137 | 2          | 2          | 2           |
    ↪ 2.4730105014001578E7 |
| 10001    | 4883406271918338267  | 2          | 2          | 2           |
    ↪ 1.6711402709780257E7 |
| 10001    | -5467083960906519443 | 2          | 2          | 2           |
    ↪ 2.7540126380326536E7 |
| 10002    | 9137404661108133814  | 2          | 2          | 2           | 2.417217089806745
    ↪ E7  |
| 10002    | 1885826196444191611  | 2          | 2          | 2           |
    ↪ 1.6327378456676323E7 |
+----------+----------------------+------------+------------+-------------+----------------------+
    ↪
```

这里以数据路径为粒度，展示了当前 slot 的使用情况。其中 `AvgRate` 为历史统计的该路径上 clone
    ↪ 任务的拷贝速率，单位是字节/秒。

3. 优先修复查看

   以下命令，可以查看通过 ADMIN REPAIR TABLE 命令设置的优先修复的表或分区。

   ```
   SHOW PROC '/cluster_balance/priority_repair';
   ```

   其中 RemainingTimeMs 表示，这些优先修复的内容，将在这个时间后，被自动移出优先修复队列。以防止优先修复一直失败导致资源被占用。

4.5.4.7.1 调度器统计状态查看

我们收集了 TabletChecker 和 TabletScheduler 在运行过程中的一些统计信息，可以通过以下命令查看：

```
SHOW PROC '/cluster_balance/sched_stat';
```

```
+---------------------------------------------------+-------------+
| Item                                              | Value       |
+---------------------------------------------------+-------------+
| num of tablet check round                         | 12041       |
| cost of tablet check(ms)                          | 7162342     |
| num of tablet checked in tablet checker           | 18793506362 |
| num of unhealthy tablet checked in tablet checker | 7043900     |
| num of tablet being added to tablet scheduler     | 1153        |
| num of tablet schedule round                      | 49538       |
| cost of tablet schedule(ms)                       | 49822       |
| num of tablet being scheduled                     | 4356200     |
| num of tablet being scheduled succeeded           | 320         |
| num of tablet being scheduled failed              | 4355594     |
| num of tablet being scheduled discard             | 286         |
| num of tablet priority upgraded                   | 0           |
| num of tablet priority downgraded                 | 1096        |
| num of clone task                                 | 230         |
| num of clone task succeeded                       | 228         |
| num of clone task failed                          | 2           |
| num of clone task timeout                         | 2           |
| num of replica missing error                      | 4354857     |
| num of replica version missing error              | 967         |
| num of replica relocating                         | 0           |
| num of replica redundant error                    | 90          |
| num of replica missing in cluster error           | 0           |
| num of balance scheduled                          | 0           |
+---------------------------------------------------+-------------+
```

各行含义如下：

- num of tablet check round：Tablet Checker 检查次数
- cost of tablet check(ms)：Tablet Checker 检查总耗时
- num of tablet checked in tablet checker：Tablet Checker 检查过的 tablet 数量
- num of unhealthy tablet checked in tablet checker：Tablet Checker 检查过的不健康的 tablet 数量

- num of tablet being added to tablet scheduler：被提交到 Tablet Scheduler 中的 tablet 数量
- num of tablet schedule round：Tablet Scheduler 运行次数
- cost of tablet schedule(ms)：Tablet Scheduler 运行总耗时
- num of tablet being scheduled：被调度的 Tablet 总数量
- num of tablet being scheduled succeeded：被成功调度的 Tablet 总数量
- num of tablet being scheduled failed：调度失败的 Tablet 总数量
- num of tablet being scheduled discard：调度失败且被抛弃的 Tablet 总数量
- num of tablet priority upgraded：优先级上调次数
- num of tablet priority downgraded：优先级下调次数
- num of clone task：生成的 clone 任务数量
- num of clone task succeeded：clone 任务成功的数量
- num of clone task failed：clone 任务失败的数量
- num of clone task timeout：clone 任务超时的数量
- num of replica missing error：检查的状态为副本缺失的 tablet 的数量
- num of replica version missing error：检查的状态为版本缺失的 tablet 的数量（该统计值包括了 num of replica relocating 和 num of replica missing in cluster error）
- num of replica relocating：检查的状态为 replica relocating 的 tablet 的数量
- num of replica redundant error：检查的状态为副本冗余的 tablet 的数量
- num of replica missing in cluster error：检查的状态为不在对应 cluster 的 tablet 的数量
- num of balance scheduled：均衡调度的次数

> 注：以上状态都只是历史累加值。我们也在 FE 的日志中，定期打印了这些统计信息，其中括号内的数值表示自上次统计信息打印依赖，各个统计值的变化数量。

4.5.4.8 相关配置说明

4.5.4.8.1 可调整参数

以下可调整参数均为 fe.conf 中可配置参数。

- use_new_tablet_scheduler
  - 说明：是否启用新的副本调度方式。新的副本调度方式即本文档介绍的副本调度方式。
  - 默认值：true
  - 重要性：高
- tablet_repair_delay_factor_second
  - 说明：对于不同的调度优先级，我们会延迟不同的时间后开始修复。以防止因为例行重启、升级等过程中，产生大量不必要的副本修复任务。此参数为一个基准系数。对于 HIGH 优先级，延迟为基准系数 * 1；对于 NORMAL 优先级，延迟为基准系数 * 2；对于 LOW 优先级，延迟为基准系数 * 3。即优先级越低，延迟等待时间越长。如果用户想尽快修复副本，可以适当降低该参数。
  - 默认值：60 秒

  - 重要性：高

- schedule_slot_num_per_path

  - 说明：默认分配给每块磁盘用于副本修复的 slot 数目。该数目表示一块磁盘能同时运行的副本修复任务数。如果想以更快的速度修复副本，可以适当调高这个参数。单数值越高，可能对 IO 影响越大。
  - 默认值：2
  - 重要性：高

- balance_load_score_threshold

  - 说明：集群均衡的阈值。默认为 0.1，即 10%。当一个 BE 节点的 load score，不高于或不低于平均 load score 的 10% 时，我们认为这个节点是均衡的。如果想让集群负载更加平均，可以适当调低这个参数。
  - 默认值：0.1
  - 重要性：中

- storage_high_watermark_usage_percent 和 storage_min_left_capacity_bytes

  - 说明：这两个参数，分别表示一个磁盘的最大空间使用率上限，以及最小的空间剩余下限。当一块磁盘的空间使用率大于上限，或者剩余空间小于下限时，该磁盘将不再作为均衡调度的目的地址。
  - 默认值：0.85 和 2097152000（2GB）
  - 重要性：中

- disable_balance

  - 说明：控制是否关闭均衡功能。当副本处于均衡过程中时，有些功能，如 ALTER TABLE 等将会被禁止。而均衡可能持续很长时间。因此，如果用户希望尽快进行被禁止的操作。可以将该参数设为 true，以关闭均衡调度。
  - 默认值：false
  - 重要性：中

以下可调整参数均为 be.conf 中可配置参数。

- clone_worker_count

  - 说明：影响副本均衡的速度。在磁盘压力不大的情况下，可以通过调整该参数来加快副本均衡。
  - 默认值：3
  - 重要性：中

4.5.4.8.2 不可调整参数

以下参数暂不支持修改，仅作说明。

- TabletChecker 调度间隔

  TabletChecker 每 20 秒进行一次检查调度。

- TabletScheduler 调度间隔

  TabletScheduler 每 5 秒进行一次调度

- TabletScheduler 每批次调度个数

  TabletScheduler 每次调度最多 50 个 tablet。

- TabletScheduler 最大等待调度和运行中任务数

  最大等待调度任务数和运行中任务数为 2000。当超过 2000 后，TabletChecker 将不再产生新的调度任务给 TabletScheduler。

- TabletScheduler 最大均衡任务数

  最大均衡任务数为 500。当超过 500 后，将不再产生新的均衡任务。

- 每块磁盘用于均衡任务的 slot 数目

  每块磁盘用于均衡任务的 slot 数目为 2。这个 slot 独立于用于副本修复的 slot。

- 集群均衡情况更新间隔

  TabletScheduler 每隔 20 秒会重新计算一次集群的 load score。

- Clone 任务的最小和最大超时时间

  一个 clone 任务超时时间范围是 3min ~ 2hour。具体超时时间通过 tablet 的大小计算。计算公式为 (tablet size) / (5MB/s)。当一个 clone 任务运行失败 3 次后，该任务将终止。

- 动态优先级调整策略

  优先级最小调整间隔为 5min。当一个 tablet 调度失败 5 次后，会调低优先级。当一个 tablet 30min 未被调度时，会调高优先级。

4.5.4.9 相关问题

- 在某些情况下，默认的副本修复和均衡策略可能会导致网络被打满（多发生在千兆网卡，且每台 BE 的磁盘数量较多的情况下）。此时需要调整一些参数来减少同时进行的均衡和修复任务数。
- 目前针对 Colocate Table 的副本的均衡策略无法保证同一个 Tablet 的副本不会分布在同一个 host 的 BE 上。但 Colocate Table 的副本的修复策略会检测到这种分布错误并校正。但可能会出现，校正后，均衡策略再次认为副本不均衡而重新均衡。从而导致在两种状态间不停交替，无法使 Colocate Group 达成稳定。针对这种情况，我们建议在使用 Colocate 属性时，尽量保证集群是同构的，以减小副本分布在同一个 host 上的概率。

4.5.4.10 最佳实践

4.5.4.10.1 控制并管理集群的副本修复和均衡进度

在大多数情况下，通过默认的参数配置，Doris 都可以自动的进行副本修复和集群均衡。但是某些情况下，我们需要通过人工介入调整参数，来达到一些特殊的目的。如优先修复某个表或分区、禁止集群均衡以降低集群负载、优先修复非 colocation 的表数据等等。

本小节主要介绍如何通过修改参数，来控制并管理集群的副本修复和均衡进度。

1. 删除损坏副本

   某些情况下，Doris 可能无法自动检测某些损坏的副本，从而导致查询或导入在损坏的副本上频繁报错。此时我们需要手动删除已损坏的副本。该方法可以适用于：删除版本数过高导致 -235 错误的副本、删除文件已损坏的副本等等。

   首先，找到副本对应的 tablet id，假设为 10001。通过 `show tablet 10001;` 并执行其中的 `show proc` 语句可以查看对应的 tablet 的各个副本详情。

   假设需要删除的副本的 backend id 是 20001。则执行以下语句将副本标记为 bad：

```
ADMIN SET REPLICA STATUS PROPERTIES("tablet_id" = "10001", "backend_id" = "20001", "status" =
    ↪  "bad");
```

此时，再次通过 `show proc` 语句可以看到对应的副本的 `IsBad` 列值为 `true`。

被标记为 `bad` 的副本不会再参与导入和查询。同时副本修复逻辑会自动补充一个新的副本。

2. 优先修复某个表或分区

   `help admin repair table;` 查看帮助。该命令会尝试优先修复指定表或分区的 tablet。

3. 停止均衡任务

   均衡任务会占用一定的网络带宽和 IO 资源。如果希望停止新的均衡任务的产生，可以通过以下命令：

```
ADMIN SET FRONTEND CONFIG ("disable_balance" = "true");
```

4. 停止所有副本调度任务

   副本调度任务包括均衡和修复任务。这些任务都会占用一定的网络带宽和 IO 资源。可以通过以下命令停止所有副本调度任务（不包括已经在运行的，包括 colocation 表和普通表）：

```
ADMIN SET FRONTEND CONFIG ("disable_tablet_scheduler" = "true");
```

5. 停止所有 colocation 表的副本调度任务。

   colocation 表的副本调度和普通表是分开独立运行的。某些情况下，用户可能希望先停止对 colocation 表的均衡和修复工作，而将集群资源用于普通表的修复，则可以通过以下命令：

```
ADMIN SET FRONTEND CONFIG ("disable_colocate_balance" = "true");
```

6. 使用更保守的策略修复副本

   Doris 在检测到副本缺失、BE 宕机等情况下，会自动修复副本。但为了减少一些抖动导致的错误（如 BE 短暂宕机），Doris 会延迟触发这些任务。

- tablet_repair_delay_factor_second 参数。默认 60 秒。根据修复任务优先级的不同，会推迟 60 秒、120 秒、180 秒后开始触发修复任务。可以通过以下命令延长这个时间，这样可以容忍更长的异常时间，以避免触发不必要的修复任务：

```
ADMIN SET FRONTEND CONFIG ("tablet_repair_delay_factor_second" = "120");
```

7. 使用更保守的策略触发 colocation group 的重分布

   colocation group 的重分布可能伴随着大量的 tablet 迁移。colocate_group_relocate_delay_second 用于控制重分布的触发延迟。默认 1800 秒。如果某台 BE 节点可能长时间下线，可以尝试调大这个参数，以避免不必要的重分布：

```
ADMIN SET FRONTEND CONFIG ("colocate_group_relocate_delay_second" = "3600");
```

8. 更快速的副本均衡

   Doris 的副本均衡逻辑会先增加一个正常副本，然后在删除老的副本，已达到副本迁移的目的。而在删除老副本时，Doris 会等待这个副本上已经开始执行的导入任务完成，以避免均衡任务影响导入任务。但这样会降低均衡逻辑的执行速度。此时可以通过修改以下参数，让 Doris 忽略这个等待，直接删除老副本：

```
ADMIN SET FRONTEND CONFIG ("enable_force_drop_redundant_replica" = "true");
```

这种操作可能会导致均衡期间部分导入任务失败（需要重试），但会显著加速均衡速度。

总体来讲，当我们需要将集群快速恢复到正常状态时，可以考虑按照以下思路处理：

1. 找到导致高优任务报错的 tablet，将有问题的副本置为 bad。
2. 通过 admin repair 语句高优修复某些表。
3. 停止副本均衡逻辑以避免占用集群资源，等集群恢复后，再开启即可。
4. 使用更保守的策略触发修复任务，以应对 BE 频繁宕机导致的雪崩效应。
5. 按需关闭 colocation 表的调度任务，集中集群资源修复其他高优数据。

4.5.5 服务自动拉起

本文档主要介绍如何配置 Doris 集群的自动拉起，保证生产环境中出现特殊情况导致服务宕机后未及时拉起服务从而影响到业务的正常运行。

Doris 集群必须完全搭建完成后再配置 FE 和 BE 的自动拉起服务。

4.5.5.1 Systemd 配置 Doris 服务

systemd 具体使用以及参数解析可以参考这里

4.5.5.1.1 sudo 权限控制

在使用 systemd 控制 doris 服务时，需要有 sudo 权限。为了保证最小粒度的 sudo 权限分配，可以将 doris-fe 与 doris-be 服务的 systemd 控制权限分配给指定的非 root 用户。在 visudo 来配置 doris-fe 与 doris-be 的 systemctl 管理权限。

```
Cmnd_Alias DORISCTL=/usr/bin/systemctl start doris-fe,/usr/bin/systemctl stop doris-fe,/usr/bin/
    ↪ systemctl start doris-be,/usr/bin/systemctl stop doris-be

#### Allow root to run any commands anywhere
root    ALL=(ALL)       ALL
doris   ALL=(ALL)       NOPASSWD:DORISCTL
```

4.5.5.1.2 配置步骤

1. 分别在 fe.conf 和 be.conf 中添加 JAVA_HOME 变量配置，否则使用 systemctl start 将无法启动服务

   ```
   echo "JAVA_HOME=your_java_home" >> /home/doris/fe/conf/fe.conf
   echo "JAVA_HOME=your_java_home" >> /home/doris/be/conf/be.conf
   ```

2. 下载 doris-fe.service 文件：doris-fe.service

3. doris-fe.service 具体内容如下：

```
  # Licensed to the Apache Software Foundation (ASF) under one
  # or more contributor license agreements.  See the NOTICE file
  # distributed with this work for additional information
  # regarding copyright ownership.  The ASF licenses this file
  # to you under the Apache License, Version 2.0 (the
  # "License"); you may not use this file except in compliance
  # with the License.  You may obtain a copy of the License at
  #
  #   http://www.apache.org/licenses/LICENSE-2.0
  #
  # Unless required by applicable law or agreed to in writing,
  # software distributed under the License is distributed on an
  # "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
  # KIND, either express or implied.  See the License for the
  # specific language governing permissions and limitations
  # under the License.

  [Unit]
  Description=Doris FE
  After=network-online.target
  Wants=network-online.target
```

```
[Service]
Type=forking
User=root
Group=root
LimitCORE=infinity
LimitNOFILE=200000
Restart=on-failure
RestartSec=30
StartLimitInterval=120
StartLimitBurst=3
KillMode=none
ExecStart=/home/doris/fe/bin/start_fe.sh --daemon
ExecStop=/home/doris/fe/bin/stop_fe.sh

[Install]
WantedBy=multi-user.target
```

注意事项

- ExecStart、ExecStop 根据实际部署的 fe 的路径进行配置

4. 下载 doris-be.service 文件：doris-be.service
5. doris-be.service 具体内容如下：

```
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements.  See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership.  The ASF licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License.  You may obtain a copy of the License at
#
#   http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied.  See the License for the
# specific language governing permissions and limitations
# under the License.
```

```
[Unit]
Description=Doris BE
After=network-online.target
Wants=network-online.target

[Service]
Type=forking
User=root
Group=root
LimitCORE=infinity
LimitNOFILE=200000
Restart=on-failure
RestartSec=30
StartLimitInterval=120
StartLimitBurst=3
KillMode=none
ExecStart=/home/doris/be/bin/start_be.sh --daemon
ExecStop=/home/doris/be/bin/stop_be.sh

[Install]
WantedBy=multi-user.target
```

注意事项

- ExecStart、ExecStop 根据实际部署的 be 的路径进行配置

6. 服务配置

将 doris-fe.service、 doris-be.service 两个文件放到 /usr/lib/systemd/system 目录下

7. 设置自启动

添加或修改配置文件后，需要重新加载

```
systemctl daemon-reload
```

设置自启动，实质就是在 /etc/systemd/system/multi-user.target.wants/ 添加服务文件的链接

```
systemctl enable doris-fe
systemctl enable doris-be
```

8. 服务启动

```
systemctl start doris-fe
systemctl start doris-be
```

4.5.5.2 Supervisor 配置 Doris 服务

Supervisor 具体使用以及参数解析可以参考这里

Supervisor 配置自动拉起可以使用 yum 命令直接安装，也可以通过 pip 手工安装，pip 手工安装流程比较复杂，只展示 yum 方式部署，手工部署请参考这里进行安装部署。

4.5.5.2.1 配置步骤

1. yum 安装 supervisor

```
yum install epel-release
yum install -y supervisor
```

2. 启动服务并查看状态

```
systemctl enable supervisord # 开机自启动
systemctl start supervisord # 启动 supervisord 服务
systemctl status supervisord # 查看 supervisord 服务状态
ps -ef|grep supervisord # 查看是否存在 supervisord 进程
```

3. 配置 BE 进程管理

```
修改 start_be.sh 脚本，去掉最后的 & 符号

vim /path/doris/be/bin/start_be.sh
将 nohup $LIMIT ${DORIS_HOME}/lib/palo_be "$@" >> $LOG_DIR/be.out 2>&1 </dev/null &
修改为 nohup $LIMIT ${DORIS_HOME}/lib/palo_be "$@" >> $LOG_DIR/be.out 2>&1 </dev/null
```

```
创建 BE 的 supervisor 进程管理配置文件
```

```
vim /etc/supervisord.d/doris-be.ini

[program:doris_be]
process_name=%(program_name)s
directory=/path/doris/be/be
command=sh /path/doris/be/bin/start_be.sh
autostart=true
```

```
autorestart=true
user=root
numprocs=1
startretries=3
stopasgroup=true
killasgroup=true
startsecs=5
#redirect_stderr = true
#stdout_logfile_maxbytes = 20MB
#stdout_logfile_backups = 10
#stdout_logfile=/var/log/supervisor-palo_be.log
```

4. 配置 FE 进程管理

修改 start_fe.sh 脚本，去掉最后的 & 符号

```
vim /path/doris/fe/bin/start_fe.sh
```

将 `nohup $LIMIT $JAVA $final_java_opt org.apache.doris.PaloFe ${HELPER} "$@" >> $LOG_DIR/fe.out 2>&1 </dev/null &`

修改为 `nohup $LIMIT $JAVA $final_java_opt org.apache.doris.PaloFe ${HELPER} "$@" >> $LOG_DIR/fe.out 2>&1 </dev/null`

创建 FE 的 supervisor 进程管理配置文件

```
vim /etc/supervisord.d/doris-fe.ini

[program:PaloFe]
environment = JAVA_HOME="/path/jdk8"
process_name=PaloFe
directory=/path/doris/fe
command=sh /path/doris/fe/bin/start_fe.sh
autostart=true
autorestart=true
user=root
numprocs=1
startretries=3
stopasgroup=true
killasgroup=true
startsecs=10
#redirect_stderr=true
#stdout_logfile_maxbytes=20MB
#stdout_logfile_backups=10
#stdout_logfile=/var/log/supervisor-PaloFe.log
```

5. 配置 Broker 进程管理

修改 start_broker.sh 脚本，去掉最后的 & 符号

```
vim /path/apache_hdfs_broker/bin/start_broker.sh
将 nohup $LIMIT $JAVA $JAVA_OPTS org.apache.doris.broker.hdfs.BrokerBootstrap "$@" >> $BROKER
    ↪ _LOG_DIR/apache_hdfs_broker.out 2>&1 </dev/null &
修改为 nohup $LIMIT $JAVA $JAVA_OPTS org.apache.doris.broker.hdfs.BrokerBootstrap "$@" >>
    ↪ $BROKER_LOG_DIR/apache_hdfs_broker.out 2>&1 </dev/null
```

创建 Broker 的 supervisor 进程管理配置文件

```
vim /etc/supervisord.d/doris-broker.ini

[program:BrokerBootstrap]
environment = JAVA_HOME="/usr/local/java"
process_name=%(program_name)s
directory=/path/apache_hdfs_broker
command=sh /path/apache_hdfs_broker/bin/start_broker.sh
autostart=true
autorestart=true
user=root
numprocs=1
startretries=3
stopasgroup=true
killasgroup=true
startsecs=5
#redirect_stderr=true
#stdout_logfile_maxbytes=20MB
#stdout_logfile_backups=10
#stdout_logfile=/var/log/supervisor-BrokerBootstrap.log
```

6. 首先确定 Doris 服务是停止状态，然后使用 supervisor 将 Doris 自动拉起，然后确定进程是否正常启动

```
supervisorctl reload # 重新加载 Supervisor 中的所有配置文件
supervisorctl status # 查看 supervisor 状态，验证 Doris 服务进程是否正常启动

其他命令 ：
supervisorctl start all # supervisorctl start 可以开启进程
supervisorctl stop doris-be # 通过 supervisorctl stop，停止进程
```

注意事项：

- 如果使用 yum 安装的 supervisor 启动报错: pkg_resources.DistributionNotFound: The 'supervisor==3.4.0' distribution was not found

> 这个是 python 版本不兼容问题，通过 yum 命令直接安装的 supervisor 只支持 python2 版本，所以需要将 /usr/bin/supervisord 和 /usr/bin/supervisorctl 中文件内容开头 #!/usr/bin/python 改为 #!/usr/bin/python2，前提是要装 python2 版本

- 如果配置了 supervisor 对 Doris 进程进行自动拉起，此时如果 Doris 出现非正常因素导致 BE 节点宕机，那么此时本来应该输出到 be.out 中的错误堆栈信息会被 supervisor 拦截，需要在 supervisor 的 log 中查找来进一步分析。

4.6 配置管理

4.6.1 配置文件目录

FE 和 BE 的配置文件目录为 conf/。这个目录除了存放默认的 fe.conf, be.conf 等文件外，也被用于公用的配置文件存放目录。

用户可以在其中存放一些配置文件，系统会自动读取。

> 自 Doris 1.2 版本后支持该功能

4.6.1.1 hdfs-site.xml 和 hive-site.xml

在 Doris 的一些功能中，需要访问 HDFS 上的数据，或者访问 Hive metastore。

我们可以通过在功能相应的语句中，手动的填写各种 HDFS/Hive 的参数。

但这些参数非常多，如果全部手动填写，非常麻烦。

因此，用户可以将 HDFS 或 Hive 的配置文件 hdfs-site.xml/hive-site.xml 直接放置在 conf/ 目录下。Doris 会自动读取这些配置文件。

而用户在命令中填写的配置，会覆盖配置文件中的配置项。

这样，用户仅需填写少量的配置，即可完成对 HDFS/Hive 的访问。

4.6.2 FE 配置项

该文档主要介绍 FE 的相关配置项。

FE 的配置文件 fe.conf 通常存放在 FE 部署路径的 conf/ 目录下。而在 0.14 版本中会引入另一个配置文件 fe_custom.conf。该配置文件用于记录用户在运行时动态配置并持久化的配置项。

FE 进程启动后，会先读取 fe.conf 中的配置项，之后再读取 fe_custom.conf 中的配置项。fe_custom.conf 中的配置项会覆盖 fe.conf 中相同的配置项。

fe_custom.conf 文件的位置可以在 fe.conf 通过 custom_config_dir 配置项配置。

4.6.2.1 查看配置项

FE 的配置项有两种方式进行查看：

1. FE 前端页面查看

在浏览器中打开 FE 前端页面 http://fe_host:fe_http_port/variable。在 Configure Info 中可以看到当前生效的 FE 配置项。

2. 通过命令查看

FE 启动后，可以在 MySQL 客户端中，通过以下命令查看 FE 的配置项，具体语法参照SHOW-CONFIG：

```
SHOW FRONTEND CONFIG;
```

结果中各列含义如下：

- Key：配置项名称。
- Value：当前配置项的值。
- Type：配置项值类型，如整型、字符串。
- IsMutable：是否可以动态配置。如果为 true，表示该配置项可以在运行时进行动态配置。如果 false，则表示该配置项只能在 fe.conf 中配置并且重启 FE 后生效。
- MasterOnly：是否为 Master FE 节点独有的配置项。如果为 true，则表示该配置项仅在 Master FE 节点有意义，对其他类型的 FE 节点无意义。如果为 false，则表示该配置项在所有 FE 节点中均有意义。
- Comment：配置项的描述。

4.6.2.2 设置配置项

FE 的配置项有两种方式进行配置：

1. 静态配置

在 conf/fe.conf 文件中添加和设置配置项。fe.conf 中的配置项会在 FE 进程启动时被读取。没有在 fe.conf 中的配置项将使用默认值。

2. 通过 MySQL 协议动态配置

FE 启动后，可以通过以下命令动态设置配置项。该命令需要管理员权限。

```
ADMIN SET FRONTEND CONFIG ("fe_config_name" = "fe_config_value");
```

不是所有配置项都支持动态配置。可以通过 SHOW FRONTEND CONFIG; 命令结果中的 IsMutable 列查看是否支持动态配置。

如果是修改 MasterOnly 的配置项，则该命令会直接转发给 Master FE 并且仅修改 Master FE 中对应的配置项。

通过该方式修改的配置项将在 FE 进程重启后失效。

更多该命令的帮助，可以通过 HELP ADMIN SET CONFIG; 命令查看。

3. 通过 HTTP 协议动态配置

具体请参阅Set Config Action

该方式也可以持久化修改后的配置项。配置项将持久化在 fe_custom.conf 文件中，在 FE 重启后仍会生效。

4.6.2.3 应用举例

1. 修改 async_pending_load_task_pool_size

通过 SHOW FRONTEND CONFIG; 可以查看到该配置项不能动态配置（IsMutable 为 false）。则需要在 fe.conf 中添加：

```
async_pending_load_task_pool_size=20
```

之后重启 FE 进程以生效该配置。

2. 修改 dynamic_partition_enable

通过 SHOW FRONTEND CONFIG; 可以查看到该配置项可以动态配置（IsMutable 为 true）。并且是 Master FE 独有配置。则首先我们可以连接到任意 FE，执行如下命令修改配置：

```
ADMIN SET FRONTEND CONFIG ("dynamic_partition_enable" = "true");`
```

之后可以通过如下命令查看修改后的值：

```
set forward_to_master=true;
SHOW FRONTEND CONFIG;
```

通过以上方式修改后，如果 Master FE 重启或进行了 Master 切换，则配置将失效。可以通过在 fe.conf 中直接添加配置项，并重启 FE 后，永久生效该配置项。

3. 修改 max_distribution_pruner_recursion_depth

通过 SHOW FRONTEND CONFIG; 可以查看到该配置项可以动态配置（IsMutable 为 true）。并且不是 Master FE 独有配置。

同样，我们可以通过动态修改配置的命令修改该配置。因为该配置不是 Master FE 独有配置，所以需要单独连接到不同的 FE，进行动态修改配置的操作，这样才能保证所有 FE 都使用了修改后的配置值

4.6.2.4 配置项列表

4.6.2.4.1 元数据与集群管理

`meta_dir`

默认值： DorisFE.DORIS_HOME_DIR + “/doris-meta”

Doris 元数据将保存在这里。强烈建议将此目录的存储为：

1. 高写入性能（SSD）
2. 安全（RAID）

`catalog_try_lock_timeout_ms`

默认值： 5000（ms）

是否可以动态配置： true

元数据锁的 tryLock 超时配置。通常它不需要改变，除非你需要测试一些东西。

`enable_bdbje_debug_mode`

默认值： false

如果设置为 true，FE 将在 BDBJE 调试模式下启动，在 Web 页面 `System->bdbje` 可以查看相关信息，否则不可以查看

`max_bdbje_clock_delta_ms`

默认值： 5000（5 秒）

设置非主 FE 到主 FE 主机之间的最大可接受时钟偏差。每当非主 FE 通过 BDBJE 建立到主 FE 的连接时，都会检查该值。如果时钟偏差大于此值，则放弃连接。

`metadata_failure_recovery`

默认值： false

如果为 true，FE 将重置 bdbje 复制组（即删除所有可选节点信息）并应该作为 Master 启动。如果所有可选节点都无法启动，我们可以将元数据复制到另一个节点并将此配置设置为 true 以尝试重新启动 FE。

`txn_rollback_limit`

默认值： 100

尝试重新加入组时 bdbje 可以回滚的最大 txn 数

`bdbje_replica_ack_timeout_second`

默认值： 10

元数据会同步写入到多个 Follower FE，这个参数用于控制 Master FE 等待 Follower FE 发送 ack 的超时时间。当写入的数据较大时，可能 ack 时间较长，如果超时，会导致写元数据失败，FE 进程退出。此时可以适当调大这个参数。

`grpc_threadmgr_threads_nums`

默认值： 4096

在 grpc_threadmgr 中处理 grpc events 的线程数量。

bdbje_lock_timeout_second

默认值：5

bdbje 操作的 lock timeout 如果 FE WARN 日志中有很多 LockTimeoutException，可以尝试增加这个值

bdbje_heartbeat_timeout_second

默认值：30

master 和 follower 之间 bdbje 的心跳超时。默认为 30 秒，与 bdbje 中的默认值相同。如果网络遇到暂时性问题，一些意外的长 Java GC 使您烦恼，您可以尝试增加此值以减少错误超时的机会

replica_ack_policy

默认值：SIMPLE_MAJORITY

选项：ALL, NONE, SIMPLE_MAJORITY

bdbje 的副本 ack 策略。更多信息，请参见：http://docs.oracle.com/cd/E17277_02/html/java/com/sleepycat/je/Durability.ReplicaAckPolicy.html

replica_sync_policy

默认值：SYNC

选项：SYNC, NO_SYNC, WRITE_NO_SYNC

bdbje 的 Follower FE 同步策略。

master_sync_policy

默认值：SYNC

选项：SYNC, NO_SYNC, WRITE_NO_SYNC

Master FE 的 bdbje 同步策略。如果您只部署一个 Follower FE，请将其设置为 “SYNC”。如果你部署了超过 3 个 Follower FE，你可以将这个和下面的 replica_sync_policy 设置为 WRITE_NO_SYNC。更多信息，参见：http://docs.oracle.com/cd/E17277_02/html/java/com/sleepycat/je/Durability.SyncPolicy.html

bdbje_reserved_disk_bytes

用于限制 bdbje 能够保留的文件的最大磁盘空间。

默认值：1073741824

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

ignore_meta_check

默认值：false

是否可以动态配置：true

如果为 true，非主 FE 将忽略主 FE 与其自身之间的元数据延迟间隙，即使元数据延迟间隙超过 meta_delay_toleration_second。非主 FE 仍将提供读取服务。当您出于某种原因尝试停止 Master FE 较长时间，但仍希望非 Master FE 可以提供读取服务时，这会很有帮助。

meta_delay_toleration_second

默认值：300（5 分钟）

如果元数据延迟间隔超过 meta_delay_toleration_second，非主 FE 将停止提供服务

edit_log_port

默认值：9010

bdbje 端口

edit_log_type

默认值：BDB

编辑日志类型。BDB：将日志写入 bdbje LOCAL：已弃用。

edit_log_roll_num

默认值：50000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Master FE will save image every edit_log_roll_num meta journals.

force_do_metadata_checkpoint

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，则无论 jvm 内存使用百分比如何，检查点线程都会创建检查点

metadata_checkpoint_memory_threshold

默认值：60（60%）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果 jvm 内存使用百分比（堆或旧内存池）超过此阈值，则检查点线程将无法工作以避免 OOM。

max_same_name_catalog_trash_num

用于设置回收站中同名元数据的最大个数，超过最大值时，最早删除的元数据将被彻底删除，不能再恢复。0 表示不保留同名对象。< 0 表示不做限制。

注意：同名元数据的判断会局限在一定的范围内。比如同名 database 的判断会限定在相同 cluster 下，同名 table 的判断会限定在相同 database（指相同 database id）下，同名 partition 的判断会限定在相同 database（指相同 database id）并且相同 table（指相同 table id）下。

默认值：3

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

cluster_id

默认值：-1

如果节点（FE 或 BE）具有相同的集群 id，则将认为它们属于同一个 Doris 集群。Cluster id 通常是主 FE 首次启动时生成的随机整数。您也可以指定一个。

heartbeat_mgr_blocking_queue_size

默认值：1024

是否为 Master FE 节点独有的配置项：true

在 heartbeat_mgr 中存储心跳任务的阻塞队列大小。

heartbeat_mgr_threads_num

默认值：8

是否为 Master FE 节点独有的配置项：true

heartbeat_mgr 中处理心跳事件的线程数。

disable_cluster_feature

默认值：true

是否可以动态配置：true

多集群功能将在 0.12 版本中弃用，将此配置设置为 true 将禁用与集群功能相关的所有操作，包括：

1. 创建/删除集群
2. 添加、释放 BE/将 BE 添加到集群/停用集群 balance
3. 更改集群的后端数量
4. 链接/迁移数据库

enable_fqdn_mode

此配置用于 k8s 部署环境。当 enable_fqdn_mode 为 true 时，将允许更改 be 的重建 pod 的 ip。

默认值：false

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

enable_token_check

默认值：true

为了向前兼容，稍后将被删除。下载 image 文件时检查令牌。

enable_multi_tags

默认值：false

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

是否开启单 BE 的多标签功能

initial_root_password

设置 root 用户初始化 2 阶段 SHA-1 加密密码，默认为”，即不设置 root 密码。后续 root 用户的 `set password` 操作会将 root 初始化密码覆盖。

示例：如要配置密码的明文是 `root@123`，可在 Doris 执行 SQL `select password('root@123')` 获取加密密码 *A00C34073A26B40AB4307650BFB9309D6BFA6999。

默认值：空字符串

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

4.6.2.4.2 服务

`query_port`

默认值：9030

Doris FE 通过 mysql 协议查询连接端口

`arrow_flight_sql_port`

默认值：-1

Doris FE 通过 Arrow Flight SQL 协议查询连接端口

`frontend_address`

状态：已弃用，不建议使用。

类型:string

描述：显式配置 FE 的 IP 地址，不使用 InetAddress。getByName 获取 IP 地址。通常在 InetAddress 中。getByName 当无法获得预期结果时。只支持 IP 地址，不支持主机名。

默认值:0.0.0.0

`priority_networks`

默认值：空

为那些有很多 ip 的服务器声明一个选择策略。请注意，最多应该有一个 ip 与此列表匹配。这是一个以分号分隔格式的列表，用 CIDR 表示法，例如 10.10.10.0/24。如果没有匹配这条规则的 ip，会随机选择一个。

`http_port`

默认值：8030

FE http 端口，当前所有 FE http 端口都必须相同

`https_port`

默认值：8050

FE https 端口，当前所有 FE https 端口都必须相同

`enable_https`

默认值：false

是否开启 FE https 的支持，false 表示支持 http，true 表示同时支持 http 与 https，并且会自动将 http 请求重定向到 https 如果 enable_https 为 true，需要在 fe.conf 中配置 ssl 证书信息

enable_ssl

默认值：true

如果设置为 true，doris 将与 mysql 服务建立基于 SSL 协议的加密通道。

qe_max_connection

默认值：1024

每个 FE 的最大连接数

check_java_version

默认值：true

Doris 将检查已编译和运行的 Java 版本是否兼容，如果不兼容将抛出 Java 版本不匹配的异常信息，并终止启动

rpc_port

默认值：9020

FE Thrift Server 的端口

thrift_server_type

该配置表示 FE 的 Thrift 服务使用的服务模型，类型为 string, 大小写不敏感。

若该参数为 SIMPLE, 则使用 TSimpleServer 模型，该模型一般不适用于生产环境，仅限于测试使用。

若该参数为 THREADED, 则使用 TThreadedSelectorServer 模型，该模型为非阻塞式 I/O 模型，即主从 Reactor 模型，该模型能及时响应大量的并发连接请求，在多数场景下有较好的表现。

若该参数为 THREAD_POOL, 则使用 TThreadPoolServer 模型，该模型为阻塞式 I/O 模型，使用线程池处理用户连接，并发连接数受限于线程池的数量，如果能提前预估并发请求的数量，并且能容忍足够多的线程资源开销，该模型会有较好的性能表现，默认使用该服务模型

thrift_server_max_worker_threads

默认值：4096

Thrift Server 最大工作线程数

thrift_backlog_num

默认值：1024

thrift 服务器的 backlog_num 当你扩大这个 backlog_num 时，你应该确保它的值大于 linux /proc/sys/net/core/
↪ somaxconn 配置

thrift_client_timeout_ms

默认值：0

thrift 服务器的连接超时和套接字超时配置

thrift_client_timeout_ms 的默认值设置为零以防止读取超时

thrift_max_message_size

默认值：100MB

thrift 服务器接收请求消息的大小（字节数）上限。如果客户端发送的消息大小超过该值，那么 thrift 服务器会拒绝该请求并关闭连接，这种情况下，client 会遇到错误：“connection has been closed by peer”，使用者可以尝试增大该参数以绕过上述限制。

use_compact_thrift_rpc

默认值：true

是否使用压缩格式发送查询计划结构体。开启后，可以降低约 50% 的查询计划结构体大小，从而避免一些“send fragment timeout”错误。但是在某些高并发小查询场景下，可能会降低约 10% 的并发度。

grpc_max_message_size_bytes

默认值：1G

用于设置 GRPC 客户端通道的初始流窗口大小，也用于设置最大消息大小。当结果集较大时，可能需要增大该值。

max_mysql_service_task_threads_num

默认值：4096

mysql 中处理任务的最大线程数。

mysql_service_io_threads_num

默认值：4

mysql 中处理 io 事件的线程数。

mysql_nio_backlog_num

默认值：1024

mysql nio server 的 backlog_num 当你放大这个 backlog_num 时，你应该同时放大 linux /proc/sys/net/core/
↪ somaxconn文件中的值

broker_timeout_ms

默认值：10000（10 秒）

Broker rpc 的默认超时时间

backend_rpc_timeout_ms

FE 向 BE 的 BackendService 发送 rpc 请求时的超时时间，单位：毫秒。

默认值：60000

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

drop_backend_after_decommission

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

该配置用于控制系统在成功下线（Decommission）BE 后，是否 Drop 该 BE。如果为 true，则在 BE 成功下线后，会删除掉该 BE 节点。如果为 false，则在 BE 成功下线后，该 BE 会一直处于 DECOMMISSION 状态，但不会被删除。

该配置在某些场景下可以发挥作用。假设一个 Doris 集群的初始状态为每个 BE 节点有一块磁盘。运行一段时间后，系统进行了纵向扩容，即每个 BE 节点新增 2 块磁盘。因为 Doris 当前还不支持 BE 内部各磁盘间的数据均衡，所以会导致初始磁盘的数据量可能一直远高于新增磁盘的数据量。此时我们可以通过以下操作进行人工的磁盘间均衡：

1. 将该配置项置为 false。
2. 对某一个 BE 节点，执行 decommission 操作，该操作会将该 BE 上的数据全部迁移到其他节点中。
3. decommission 操作完成后，该 BE 不会被删除。此时，取消掉该 BE 的 decommission 状态。则数据会开始从其他 BE 节点均衡回这个节点。此时，数据将会均匀的分布到该 BE 的所有磁盘上。
4. 对所有 BE 节点依次执行 2，3 两个步骤，最终达到所有节点磁盘均衡的目的。

`max_backend_down_time_second`

默认值：3600（1 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果 BE 关闭了 `max_backend_down_time_second`，将触发 BACKEND_DOWN 事件。

`disable_backend_black_list`

用于禁止 BE 黑名单功能。禁止该功能后，如果向 BE 发送查询请求失败，也不会将这个 BE 添加到黑名单。该参数适用于回归测试环境，以减少偶发的错误导致大量回归测试失败。

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

`max_backend_heartbeat_failure_tolerance_count`

最大可容忍的 BE 节点心跳失败次数。如果连续心跳失败次数超过这个值，则会将 BE 状态置为 dead。该参数适用于回归测试环境，以减少偶发的心跳失败导致大量回归测试失败。

默认值：1

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

`abort_txn_after_lost_heartbeat_time_second`

丢失 be 心跳后丢弃 be 事务的时间。默认时间为三百秒，当三百秒 fe 没有接收到 be 心跳时，会丢弃该 be 的所有事务。

默认值：300(秒)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

`enable_access_file_without_broker`

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

此配置用于在通过代理访问 bos 或其他云存储时尝试跳过代理

agent_task_resend_wait_time_ms

默认值：5000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

当代理任务的创建时间被设置的时候，此配置将决定是否重新发送代理任务，当且仅当当前时间减去创建时间大于 agent_task_task_resend_wait_time_ms 时，ReportHandler 可以重新发送代理任务。

该配置目前主要用来解决 PUBLISH_VERSION 代理任务的重复发送问题，目前该配置的默认值是 5000，是个实验值。

由于把代理任务提交到代理任务队列和提交到 BE 存在一定的时间延迟，所以调大该配置的值可以有效解决代理任务的重复发送问题，

但同时会导致提交失败或者执行失败的代理任务再次被执行的时间延长。

max_agent_task_threads_num

默认值：4096

是否为 Master FE 节点独有的配置项：true

代理任务线程池中处理代理任务的最大线程数。

remote_fragment_exec_timeout_ms

默认值：30000（ms）

是否可以动态配置：true

异步执行远程 fragment 的超时时间。在正常情况下，异步远程 fragment 将在短时间内执行。如果系统处于高负载状态，请尝试将此超时设置更长的时间。

auth_token

默认值：空

用于内部身份验证的集群令牌。

enable_http_server_v2

默认值：从官方 0.14.0 release 版之后默认是 true，之前默认 false

HTTP Server V2 由 SpringBoot 实现，并采用前后端分离的架构。只有启用 HTTPv2，用户才能使用新的前端 UI 界面

http_api_extra_base_path

基本路径是所有 API 路径的 URL 前缀。一些部署环境需要配置额外的基本路径来匹配资源。此 Api 将返回在 Config.http_api_extra_base_path 中配置的路径。默认为空，表示未设置。

jetty_server_acceptors

默认值：2

jetty_server_selectors

默认值：4

jetty_server_workers

默认值：0

Jetty 的线程数量由以上三个参数控制。Jetty 的线程架构模型非常简单，分为 acceptors、selectors 和 workers 三个线程池。acceptors 负责接受新连接，然后交给 selectors 处理 HTTP 消息协议的解包，最后由 workers 处理请求。前两个线程池采用非阻塞模型，一个线程可以处理很多 socket 的读写，所以线程池数量较小。

大多数项目，acceptors 线程只需要 1 ~ 2 个，selectors 线程配置 2 ~ 4 个足矣。workers 是阻塞性的业务逻辑，往往有较多的数据库操作，需要的线程数量较多，具体数量随应用程序的 QPS 和 IO 事件占比而定。QPS 越高，需要的线程数量越多，IO 占比越高，等待的线程数越多，需要的总线程数也越多。

workers 线程池默认不做设置，根据自己需要进行设置

jetty_server_max_http_post_size

默认值：100 * 1024 * 1024（100MB）

这个是 put 或 post 方法上传文件的最大字节数，默认值：100MB

jetty_server_max_http_header_size

默认值：1048576（1M）

http header size 配置参数

http_sql_submitter_max_worker_threads

默认值：2

http 请求处理/api/query 中 sql 任务的最大线程池

http_load_submitter_max_worker_threads

默认值：2

http 请求处理/api/upload 任务的最大线程池

4.6.2.4.3 查询引擎

default_max_query_instances

默认值：-1

用户属性 max_query_instances 小于等于 0 时，使用该配置，用来限制单个用户同一时刻可使用的查询 instance 个数。该参数小于等于 0 表示无限制。

max_query_retry_time

默认值：3

是否可以动态配置：true

查询重试次数。如果我们遇到 RPC 异常并且没有将结果发送给用户，则可能会重试查询。您可以减少此数字以避免雪崩灾难。

max_dynamic_partition_num

默认值：500

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于限制创建动态分区表时可以创建的最大分区数，避免一次创建过多分区。数量由动态分区参数中的“开始”和“结束”决定。

dynamic_partition_enable

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

是否启用动态分区调度，默认启用

dynamic_partition_check_interval_seconds

默认值：600 秒，10 分钟

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

检查动态分区的频率

max_multi_partition_num

默认值：4096

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于限制批量创建分区表时可以创建的最大分区数，避免一次创建过多分区。

multi_partition_name_prefix

默认值：p_

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

使用此参数设置 multi partition 的分区名前缀，仅仅 multi partition 生效，不作用于动态分区，默认前缀是“p_”。

partition_in_memory_update_interval_secs

默认值：300 (s)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

更新内存中全局分区信息的时间

enable_concurrent_update

默认值：false

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

是否启用并发更新

lower_case_table_names

默认值：0

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

用于控制用户表表名大小写是否敏感。该配置只能在集群初始化时配置，初始化完成后集群重启和升级时不能修改。

0：表名按指定存储，比较区分大小写。1：表名以小写形式存储，比较不区分大小写。2：表名按指定存储，但以小写形式进行比较。

table_name_length_limit

默认值：64

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于控制最大的表名长度

cache_enable_sql_mode

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

如果设置为 true，SQL 查询结果集将被缓存。如果查询中所有表的所有分区最后一次访问版本时间的间隔大于 cache_last_version_interval_second，且结果集行数小于 cache_result_max_row_count，且数据大小小于 cache_result_max_data_size，则结果集会被缓存，下一条相同的 SQL 会命中缓存

如果设置为 true，FE 会启用 sql 结果缓存，该选项适用于离线数据更新场景

	case1	case2	case3	case4
enable_sql_cache	false	true	true	false
enable_partition_cache	false	false	true	true

cache_enable_partition_mode

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

如果设置为 true，FE 将从 BE cache 中获取数据，该选项适用于部分分区的实时更新。

cache_result_max_row_count

默认值：3000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

设置可以缓存的最大行数，详细的原理可以参考官方文档：操作手册 -> 分区缓存

`cache_result_max_data_size`

默认值：31457280

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

设置可以缓存的最大数据大小，单位 Bytes

`cache_last_version_interval_second`

默认值：30

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

缓存结果时上一版本的最小间隔，该参数区分离线更新和实时更新

`max_allowed_in_element_num_of_delete`

默认值：1024

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于限制 delete 语句中 Predicate 的元素个数

`max_running_rollup_job_num_per_table`

默认值：1

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

控制 Rollup 作业并发限制

`max_distribution_pruner_recursion_depth`

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

这将限制哈希分布修剪器的最大递归深度。例如：其中 a in（5 个元素）和 b in（4 个元素）和 c in（3 个元素）和 d in（2 个元素）。a/b/c/d 是分布式列，所以递归深度为 5 4 3 * 2 = 120，大于 100，因此该分发修剪器将不起作用，只会返回所有 buckets。增加深度可以支持更多元素的分布修剪，但可能会消耗更多的 CPU

通过 `SHOW FRONTEND CONFIG;` 可以查看到该配置项可以动态配置（`IsMutable` 为 true）。并且不是 Master FE 独有配置。

同样，我们可以通过动态修改配置的命令修改该配置。因为该配置不是 Master FE 独有配置，所以需要单独连接到不同的 FE，进行动态修改配置的操作，这样才能保证所有 FE 都使用了修改后的配置值

enable_local_replica_selection

默认值：false

是否可以动态配置：true

如果设置为 true，Planner 将尝试在与此前端相同的主机上选择 tablet 的副本。在以下情况下，这可能会减少网络传输：

1. N 个主机，部署了 N 个 BE 和 N 个 FE。
2. 数据有 N 个副本。
3. 高并发查询均匀发送到所有 Frontends

在这种情况下，所有 Frontends 只能使用本地副本进行查询。如果想当本地副本不可用时，使用非本地副本服务查询，请将 enable_local_replica_selection_fallback 设置为 true

enable_local_replica_selection_fallback

默认值：false

是否可以动态配置：true

与 enable_local_replica_selection 配合使用，当本地副本不可用时，使用非本地副本服务查询。

expr_depth_limit

默认值：3000

是否可以动态配置：true

限制 expr 树的深度。超过此限制可能会导致在持有 db read lock 时分析时间过长。

expr_children_limit

默认值：10000

是否可以动态配置：true

限制 expr 树的 expr 子节点的数量。超过此限制可能会导致在持有数据库读锁时分析时间过长。

be_exec_version

用于定义 fragment 之间传递 block 的序列化格式。

有时我们的一些代码改动会改变 block 的数据格式，为了使得 BE 在滚动升级的过程中能够相互兼容数据格式，我们需要从 FE 下发一个数据版本来决定以什么格式发送数据。

具体的来说，例如集群中有 2 个 BE，其中一台经过升级能够支持最新的 v_1，而另一台只支持 v_0，此时由于 FE 还未升级，所以统一下发 v_0，BE 之间以旧的数据格式进行交互。待 BE 都升级完成，我们再升级 FE，此时新的 FE 会下发 v_1，集群统一切换到新的数据格式。

默认值为max_be_exec_version，如果有特殊需要，我们可以手动设置将格式版本降低，但不应低于min_be_
↪ exec_version。

需要注意的是，我们应该始终保持该变量的值处于所有 BE 的BeExecVersionManager::min_be_exec_version ↪ 和BeExecVersionManager::max_be_exec_version之间。(也就是说如果一个已经完成更新的集群如果需要降级，应该保证先降级 FE 再降级 BE 的顺序，或者手动在设置中将该变量调低再降级 BE)

max_be_exec_version

目前支持的最新数据版本，不可修改，应与配套版本的 BE 中的BeExecVersionManager::max_be_exec_version ↪ 一致。

min_be_exec_version

目前支持的最旧数据版本，不可修改，应与配套版本的 BE 中的BeExecVersionManager::min_be_exec_version ↪ 一致。

max_query_profile_num

用于设置保存查询的 profile 的最大个数。

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

publish_version_interval_ms

默认值：10 (ms)

两个发布版本操作之间的最小间隔

publish_version_timeout_second

默认值：30 (s)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

一个事务的所有发布版本任务完成的最大等待时间

query_colocate_join_memory_limit_penalty_factor

默认值：1

是否可以动态配置：true

colocate join PlanFragment instance 的 memory_limit = exec_mem_limit / min (query_colocate_join_memory_limit_penalty_factor, instance_num)

rewrite_count_distinct_to_bitmap_hll

默认值：true

该变量为 session variable，session 级别生效。

- 类型：boolean
- 描述：仅对于 AGG 模型的表来说，当变量为 true 时，用户查询时包含 count(distinct c1) 这类聚合函数时，如果 c1 列本身类型为 bitmap，则 count distinct 会改写为 bitmap_union_count(c1)。当 c1 列本身类型为 hll，则 count distinct 会改写为 hll_union_agg(c1) 如果变量为 false，则不发生任何改写。

4.6.2.4.4 导入与导出

`enable_pipeline_load`

默认值： true

是否可以动态配置： true

是否开启 Pipeline 引擎执行 Streamload 等导入任务。

`enable_vectorized_load`

默认值： true

是否开启向量化导入

`enable_new_load_scan_node`

默认值： true

是否开启新的 file scan node

`default_max_filter_ratio`

默认值： 0

是否可以动态配置： true

是否为 Master FE 节点独有的配置项： true

可过滤数据（由于数据不规则等原因）的最大百分比。默认值为 0，表示严格模式，只要数据有一条被过滤掉整个导入失败

`max_running_txn_num_per_db`

默认值： 1000

是否可以动态配置： true

是否为 Master FE 节点独有的配置项： true

这个配置主要是用来控制同一个 DB 的并发导入个数的。

当集群中有过多的导入任务正在运行时，新提交的导入任务可能会报错：

```
current running txns on db xxx is xx, larger than limit xx
```

该遇到该错误时，说明当前集群内正在运行的导入任务超过了该配置值。此时建议在业务侧进行等待并重试导入任务。

如果使用 Connector 方式写入，该参数的值可以适当调大，上千也没有问题

`using_old_load_usage_pattern`

默认值： false

是否可以动态配置： true

是否为 Master FE 节点独有的配置项： true

如果设置为 true，处理错误的 insert stmt 仍将返回一个标签给用户。用户可以使用此标签来检查导入作业的状态。默认值为 false，表示插入操作遇到错误，不带导入标签，直接抛出异常给用户客户端。

disable_load_job

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

不禁用，如果这设置为 true

- 调用开始 txn api 时，所有挂起的导入作业都将失败
- 调用 commit txn api 时，所有准备导入作业都将失败
- 所有提交的导入作业将等待发布

commit_timeout_second

默认值：30

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

在提交一个事务之前插入所有数据的最大等待时间这是命令“commit”的超时秒数

max_unfinished_load_job

默认值：1000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

最大加载任务数，包括 PENDING、ETL、LOADING、QUORUM_FINISHED。如果超过此数量，则不允许提交导入作业。

db_used_data_quota_update_interval_secs

默认值：300 (s)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

一个主守护线程将每 db_used_data_quota_update_interval_secs 更新数据库 txn 管理器的数据库使用数据配额

为了更好的数据导入性能，在数据导入之前的数据库已使用的数据量是否超出配额的检查中，我们并不实时计算数据库已经使用的数据量，而是获取后台线程周期性更新的值。

该配置用于设置更新数据库使用的数据量的值的时间间隔

disable_show_stream_load

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

是否禁用显示 stream load 并清除内存中的 stream load 记录。

max_stream_load_record_size

默认值：5000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

可以存储在内存中的最近 stream load 记录的默认最大数量

fetch_stream_load_record_interval_second

默认值：120

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

获取 stream load 记录间隔

max_bytes_per_broker_scanner

默认值：500 * 1024 * 1024 * 1024L （500G）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

broker scanner 程序可以在一个 broker 加载作业中处理的最大字节数。通常，每个 BE 都有一个 broker scanner 程序。

default_load_parallelism

默认值：8

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

单个节点 broker load 导入的默认并发度。如果用户在提交 broker load 任务时，在 properties 中自行指定了并发度，则采用用户自定义的并发度。此参数将与max_broker_concurrency、min_bytes_per_broker_scanner等多个配置共同决定导入任务的并发度。

max_broker_concurrency

默认值：10

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

broker scanner 的最大并发数。

min_bytes_per_broker_scanner

默认值：67108864L (64M)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

单个 broker scanner 将读取的最小字节数。

period_of_auto_resume_min

默认值：5（s）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

自动恢复 Routine load 的周期

max_tolerable_backend_down_num

默认值：0

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

只要有一个 BE 宕机，Routine Load 就无法自动恢复

max_routine_load_task_num_per_be

默认值：5

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

每个 BE 的最大并发例 Routine Load 任务数。这是为了限制发送到 BE 的 Routine Load 任务的数量，并且它也应该小于 BE config routine_load_thread_pool_size（默认 10），这是 BE 上的 Routine Load 任务线程池大小。

max_routine_load_task_concurrent_num

默认值：5

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

单个 Routine Load 作业的最大并发任务数

max_routine_load_job_num

默认值：100

最大 Routine Load 作业数，包括 NEED_SCHEDULED, RUNNING, PAUSE

desired_max_waiting_jobs

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

routine load V2 版本加载的默认等待作业数，这是一个理想的数字。在某些情况下，例如切换 master，当前数量可能超过desired_max_waiting_jobs

disable_hadoop_load

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认不禁用，将来不推荐使用 hadoop 集群 load。设置为 true 以禁用这种 load 方式。

enable_spark_load

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

是否临时启用 spark load，默认不启用

注意： 这个参数在 1.2 版本中已经删除，默认开启 spark_load

spark_load_checker_interval_second

默认值：60

Spark 负载调度程序运行间隔，默认 60 秒

async_loading_load_task_pool_size

默认值：10

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

loading_load任务执行程序池大小。该池大小限制了正在运行的最大 loading_load任务数。

当前，它仅限制 broker load的 loading_load任务的数量。

async_pending_load_task_pool_size

默认值：10

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

pending_load任务执行程序池大小。该池大小限制了正在运行的最大 pending_load任务数。

当前，它仅限制 broker load和 spark load的 pending_load任务的数量。

它应该小于 max_running_txn_num_per_db的值

async_load_task_pool_size

默认值：10

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：true

此配置只是为了兼容旧版本，此配置已被 async_loading_load_task_pool_size取代，以后会被移除。

enable_single_replica_load

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

是否启动单副本数据导入功能。

min_load_timeout_second

默认值：1（1 秒）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

最小超时时间，适用于所有类型的 load

max_stream_load_timeout_second

默认值：259200（3 天）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

stream load 和 mini load 最大超时时间

max_load_timeout_second

默认值：259200（3 天）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

load 最大超时时间，适用于除 stream load 之外的所有类型的加载

stream_load_default_timeout_second

默认值：86400 * 3（3 天）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认 stream load 和 mini load 超时时间

stream_load_default_precommit_timeout_second

默认值：3600（s）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认 stream load 预提交超时时间

stream_load_default_memtable_on_sink_node

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

当 HTTP header 没有设置 memtable_on_sink_node 的时候，stream load 是否默认打开前移

insert_load_default_timeout_second

默认值：3600（1 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认 insert load 超时时间

mini_load_default_timeout_second

默认值：3600（1 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认非 stream load 类型的 mini load 的超时时间

broker_load_default_timeout_second

默认值：14400（4 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Broker load 的默认超时时间

spark_load_default_timeout_second

默认值：86400 (1 天)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

默认 Spark 导入超时时间

hadoop_load_default_timeout_second

默认值：86400 * 3 (3 天)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Hadoop 导入超时时间

load_running_job_num_limit

默认值：0

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Load 任务数量限制，默认 0，无限制

load_input_size_limit_gb

默认值：0

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Load 作业输入的数据大小，默认是 0，无限制

`load_etl_thread_num_normal_priority`

默认值：10

NORMAL 优先级 etl 加载作业的并发数。

`load_etl_thread_num_high_priority`

默认值：3

高优先级 etl 加载作业的并发数。

`load_pending_thread_num_normal_priority`

默认值：10

NORMAL 优先级挂起加载作业的并发数。

`load_pending_thread_num_high_priority`

默认值：3

高优先级挂起加载作业的并发数。加载作业优先级定义为 HIGH 或 NORMAL。所有小批量加载作业都是 HIGH 优先级，其他类型的加载作业是 NORMAL 优先级。设置优先级是为了避免慢加载作业长时间占用线程。这只是内部优化的调度策略。目前，您无法手动指定作业优先级。

`load_checker_interval_second`

默认值：5（s）

负载调度器运行间隔。加载作业将其状态从 PENDING 转移到 LOADING 到 FINISHED。加载调度程序将加载作业从 PENDING 转移到 LOADING 而 txn 回调会将加载作业从 LOADING 转移到 FINISHED。因此，当并发未达到上限时，加载作业最多需要一个时间间隔才能完成。

`label_keep_max_second`

默认值：`3 * 24 * 3600` (3 天)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

`label_keep_max_second`后将删除已完成或取消的加载作业的标签，

1. 去除的标签可以重复使用。
2. 设置较短的时间会降低 FE 内存使用量（因为所有加载作业的信息在被删除之前都保存在内存中）

在高并发写的情况下，如果出现大量作业积压，出现 `call frontend service failed`的情况，查看日志如果是元数据写占用锁的时间太长，可以将这个值调成 12 小时，或者更小 6 小时

`streaming_label_keep_max_second`

默认值：43200（12 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

对于一些高频负载工作，例如：INSERT、 STREAMING LOAD、 ROUTINE_LOAD_TASK。如果过期，则删除已完成的作业或任务。

label_clean_interval_second

默认值：1 * 3600（1 小时）

load 标签清理器将每隔 label_clean_interval_second 运行一次以清理过时的作业。

label_regex_length

默认值：128 (字符)

导入 label 的最大字符长度，默认 128 个字符。

transaction_clean_interval_second

默认值：30

如果事务 visible 或者 aborted 状态，事务将在 transaction_clean_interval_second 秒后被清除，我们应该让这个间隔尽可能短，每个清洁周期都尽快

sync_commit_interval_second

提交事务的最大时间间隔。若超过了这个时间 channel 中还有数据没有提交，consumer 会通知 channel 提交事务。

默认值：10（秒）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

sync_checker_interval_second

数据同步作业运行状态检查

默认值：10（秒）

max_sync_task_threads_num

数据同步作业线程池中的最大线程数量。

默认值：10

min_sync_commit_size

提交事务需满足的最小 event 数量。若 Fe 接收到的 event 数量小于它，会继续等待下一批数据直到时间超过了 sync_commit_interval_second 为止。默认值是 10000 个 events，如果你想修改此配置，请确保此值小于 canal 端的 canal.instance.memory.buffer.size 配置（默认 16384），否则在 ack 前 Fe 会尝试获取比 store 队列长度更多的 event，导致 store 队列阻塞至超时为止。

默认值：10000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

min_bytes_sync_commit

提交事务需满足的最小数据大小。若 Fe 接收到的数据大小小于它，会继续等待下一批数据直到时间超过了 sync_commit_interval_second 为止。默认值是 15 MB，如果你想修改此配置，请确保此值小于 canal 端的 canal.instance.memory.buffer.size 和 canal.instance.memory.buffer.memunit 的乘积（默认 16 MB），否则在 ack 前 Fe 会尝试获取比 store 空间更大的数据，导致 store 队列阻塞至超时为止。

默认值：15 * 1024 * 1024（15M）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

max_bytes_sync_commit

数据同步作业线程池中的最大线程数量。此线程池整个 FE 中只有一个，用于处理 FE 中所有数据同步作业向 BE 发送数据的任务 task，线程池的实现在 SyncTaskPool 类。

默认值：10

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

enable_outfile_to_local

默认值：false

是否允许 outfile 函数将结果导出到本地磁盘

export_tablet_num_per_task

默认值：5

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

每个导出查询计划的 tablet 数量

export_task_default_timeout_second

默认值：2 * 3600（2 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

导出作业的默认超时时间

export_running_job_num_limit

默认值：5

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

运行导出作业的并发限制，默认值为 5，0 表示无限制

export_checker_interval_second

默认值：5

导出检查器的运行间隔

enable_mow_load_force_take_ms_lock

> 自 3.1.0 版本支持

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

存算分离模式下 Merge-On-Write Unique 表上的导入是否开启强制抢锁功能。开启后，当导入事务在提交阶段尝试获取位于 meta-service 中该表的分布式锁时的等待时间超过一个阈值 (由配置mow_load_force_take_ms_lock_threshold_ms决定) 后，将强制获取该分布式锁。该功能可用于减少高频高并发导入下由于等锁导致的导入延迟长尾。

mow_load_force_take_ms_lock_threshold_ms

> 自 3.1.0 版本支持

默认值：500

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

存算分离模式下 Merge-On-Write Unique 表上的导入事务强制抢锁的超时阈值。

4.6.2.4.5 日志

log_roll_size_mb

默认值：1024（1G）

一个系统日志和审计日志的最大大小

sys_log_dir

默认值：DorisFE.DORIS_HOME_DIR + “/log”

这指定了 FE 日志目录。FE 将产生 2 个日志文件：

1. fe.log：FE 进程的所有日志。
2. fe.warn.log FE 进程的所有警告和错误日志。

sys_log_level

默认值：INFO

日志级别，可选项：INFO, WARN, ERROR, FATAL

sys_log_roll_num

默认值：10

要保存在 sys_log_roll_interval 内的最大 FE 日志文件。默认为 10，表示一天最多有 10 个日志文件

sys_log_verbose_modules

默认值：{}

详细模块。VERBOSE 级别由 log4j DEBUG 级别实现。

例如：sys_log_verbose_modules = org.apache.doris.catalog 这只会打印包 org.apache.doris.catalog 及其所有子包中文件的调试日志。

sys_log_roll_interval

默认值：DAY

可选项：

- DAY: log 前缀是 yyyyMMdd
- HOUR: log 前缀是 yyyyMMddHH

sys_log_delete_age

默认值：7d

默认为 7 天，如果日志的最后修改时间为 7 天前，则将其删除。

支持格式：

- 7d: 7 天
- 10h: 10 小时
- 60m: 60 分钟
- 120s: 120 秒

sys_log_roll_mode

默认值：SIZE-MB-1024

日志拆分的大小，每 1G 拆分一个日志文件

sys_log_enable_compress

默认值：false

控制是否压缩 fe log, 包括 fe.log 及 fe.warn.log。如果开启，则使用 gzip 算法进行压缩。

audit_log_dir

默认值：DorisFE.DORIS_HOME_DIR + “/log”

审计日志目录：这指定了 FE 审计日志目录。审计日志 fe.audit.log 包含所有请求以及相关信息，如 user，host，
↪ cost，status 等。

audit_log_roll_num

默认值：90

保留在 audit_log_roll_interval 内的最大 FE 审计日志文件。

audit_log_modules

默认值：{“slow_query”，“query”，“load”，“stream_load”}

慢查询包含所有开销超过 qe_slow_log_ms 的查询

qe_slow_log_ms

默认值：5000（5 秒）

如果查询的响应时间超过此阈值，则会在审计日志中记录为 slow_query。

audit_log_roll_interval

默认值：DAY

DAY: log 前缀是：yyyyMMdd HOUR: log 前缀是：yyyyMMddHH

audit_log_delete_age

默认值：30d

默认为 30 天，如果日志的最后修改时间为 30 天前，则将其删除。

支持格式：

- 7d 7 天
- 10 小时 10 小时
- 60m 60 分钟
- 120s 120 秒

audit_log_enable_compress

默认值：false

控制是否压缩 fe.audit.log。如果开启，则使用 gzip 算法进行压缩。

nereids_trace_log_dir

默认值：DorisFE.DORIS_HOME_DIR + “/log/nereids_trace”

用于存储 nereids trace 日志的目录

4.6.2.4.6 存储

min_replication_num_per_tablet

默认值：1

用于设置单个 tablet 的最小 replication 数量。

max_replication_num_per_tablet

默认值：32767

用于设置单个 tablet 的最大 replication 数量。

default_db_data_quota_bytes

默认值：8192PB

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于设置默认数据库数据配额大小，设置单个数据库的配额大小可以使用：

```
设置数据库数据量配额，单位为B/K/KB/M/MB/G/GB/T/TB/P/PB
ALTER DATABASE db_name SET DATA QUOTA quota;
查看配置
show data （其他用法：HELP SHOW DATA）
```

default_db_replica_quota_size

默认值：1073741824

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于设置默认数据库 Replica 数量配额大小，设置单个数据库配额大小可以使用：

```
设置数据库Replica数量配额
ALTER DATABASE db_name SET REPLICA QUOTA quota;
查看配置
show data （其他用法：HELP SHOW DATA）
```

recover_with_empty_tablet

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

在某些情况下，某些 tablet 可能会损坏或丢失所有副本。此时数据已经丢失，损坏的 tablet 会导致整个查询失败，无法查询剩余的健康 tablet。

在这种情况下，您可以将此配置设置为 true。系统会将损坏的 tablet 替换为空 tablet，以确保查询可以执行。（但此时数据已经丢失，所以查询结果可能不准确）

min_clone_task_timeout_sec 和 max_clone_task_timeout_sec

默认值：最小 3 分钟，最大两小时

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

min_clone_task_timeout_sec 和 max_clone_task_timeout_sec 用于限制克隆任务的最小和最大超时间。一般情况下，克隆任务的超时时间是通过数据量和最小传输速度（5MB/s）来估计的。但在特殊情况下，您可能需要手动设置这两个配置，以确保克隆任务不会因超时而失败。

disable_storage_medium_check

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果 disable_storage_medium_check 为 true，ReportHandler 将不会检查 tablet 的存储介质，并使得存储冷却功能失效，默认值为 false。当您不关心 tablet 的存储介质是什么时，可以将值设置为 true。

decommission_tablet_check_threshold

默认值：5000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

该配置用于控制 FE 是否执行检测（Decommission）BE 上 Tablets 状态的阈值。如果（Decommission）BE 上的 Tablets 个数大于 0 但小于该阈值，FE 会定时对该 BE 开启一项检测，

如果该 BE 上的 Tablets 数量大于 0 但是所有 Tablets 均处于被回收的状态，那么 FE 会立即下线该（Decommission）BE。注意，不要把该值配置的太大，不然在 Decommission 阶段可能会对 FE 造成性能压力。

partition_rebalance_max_moves_num_per_selection

默认值：10

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

仅在使用 PartitionRebalancer 时有效，

partition_rebalance_move_expire_after_access

默认值：600 (s)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

仅在使用 PartitionRebalancer 时有效。如果更改，缓存的移动将被清除

tablet_rebalancer_type

默认值：BeLoad

是否为 Master FE 节点独有的配置项：true

rebalancer 类型（忽略大小写）：BeLoad、Partition。如果类型解析失败，默认使用 BeLoad

max_balancing_tablets

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果 TabletScheduler 中的 balance tablet 数量超过 max_balancing_tablets，则不再进行 balance 检查

max_scheduling_tablets

默认值：2000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果 TabletScheduler 中调度的 tablet 数量超过 max_scheduling_tablets，则跳过检查。

disable_balance

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，TabletScheduler 将不会做 balance

disable_disk_balance

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，TabletScheduler 将不会做单个 BE 上磁盘之间的 balance

balance_load_score_threshold

默认值：0.1 (10%)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

集群 balance 百分比的阈值，如果一个 BE 的负载分数比平均分数低 10%，这个后端将被标记为低负载，如果负载分数比平均分数高 10%，将被标记为高负载。

capacity_used_percent_high_water

默认值：0.75（75%）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

磁盘容量的高水位使用百分比。这用于计算后端的负载分数

clone_distribution_balance_threshold

默认值：0.2

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

BE 副本数的平衡阈值。

clone_capacity_balance_threshold

默认值：0.2

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

- BE 中数据大小的平衡阈值。

平衡算法为：

1. 计算整个集群的平均使用容量（AUC）(总数据大小/BE 数）
2. 高水位为 (AUC * (1 + clone_capacity_balance_threshold))
3. 低水位为 (AUC * (1 - clone_capacity_balance_threshold))
4. 克隆检查器将尝试将副本从高水位 BE 移动到低水位 BE。

disable_colocate_balance

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

此配置可以设置为 true 以禁用自动 colocate 表的重新定位和平衡。如果 disable_colocate_balance'设置为 true，则 ColocateTableBalancer 将不会重新定位和平衡并置表。

注意：

1. 一般情况下，根本不需要关闭平衡。
2. 因为一旦关闭平衡，不稳定的 colocate 表可能无法恢复
3. 最终查询时无法使用 colocate 计划。

balance_slot_num_per_path

默认值：1

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

balance 时每个路径的默认 slot 数量

disable_tablet_scheduler

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，将关闭副本修复和均衡逻辑。

enable_force_drop_redundant_replica

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，系统会在副本调度逻辑中，立即删除冗余副本。这可能导致部分正在对对应副本写入的导入作业失败，但是会加速副本的均衡和修复速度。当集群中有大量等待被均衡或修复的副本时，可以尝试设置此参数，以牺牲部分导入成功率为代价，加速副本的均衡和修复。

colocate_group_relocate_delay_second

默认值：1800

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

重分布一个 Colocation Group 可能涉及大量的 tablet 迁移。因此，我们需要一个更保守的策略来避免不必要的 Colocation 重分布。重分布通常发生在 Doris 检测到有 BE 节点宕机后。这个参数用于推迟对 BE 宕机的判断。如默认参数下，如果 BE 节点能够在 1800 秒内恢复，则不会触发 Colocation 重分布。

allow_replica_on_same_host

默认值：false

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

是否允许同一个 tablet 的多个副本分布在同一个 host 上。这个参数主要用于本地测试是，方便搭建多个 BE 已测试某些多副本情况。不要用于非测试环境。

repair_slow_replica

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果设置为 true，会自动检测 compaction 比较慢的副本，并将迁移到其他机器，检测条件是最慢副本的版本计数超过 min_version_count_indicate_replica_compaction_too_slow 的值，且与最快副本的版本计数差异所占比例超过 valid_version_count_delta_ratio_between_replicas 的值

min_version_count_indicate_replica_compaction_too_slow

默认值：200

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

版本计数阈值，用来判断副本做 compaction 的速度是否太慢

skip_compaction_slower_replica

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

如果设置为 true，则在选择可查询副本时，将跳过 compaction 较慢的副本

valid_version_count_delta_ratio_between_replicas

默认值：0.5

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

最慢副本的版本计数与最快副本的差异有效比率阈值，如果设置 repair_slow_replica 为 true，则用于判断是否修复最慢的副本

min_bytes_indicate_replica_too_large

默认值：2 * 1024 * 1024 * 1024 (2G)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

数据大小阈值，用来判断副本的数据量是否太大

schedule_slot_num_per_hdd_path

默认值：4

对于 hdd 盘，tablet 调度程序中每个路径的默认 slot 数量

schedule_slot_num_per_ssd_path

默认值：8

对于 ssd 盘，tablet 调度程序中每个路径的默认 slot 数量

tablet_repair_delay_factor_second

默认值：60（s）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

决定修复 tablet 前的延迟时间因素。

1. 如果优先级为 VERY_HIGH，请立即修复。
2. HIGH，延迟 tablet_repair_delay_factor_second * 1；
3. 正常：延迟 tablet_repair_delay_factor_second * 2；
4. 低：延迟 tablet_repair_delay_factor_second * 3；

tablet_stat_update_interval_second

默认值：300，（5 分钟）

tablet 状态更新间隔所有 FE 将在每个时间间隔从所有 BE 获取 tablet 统计信息

storage_flood_stage_usage_percent

默认值：95（95%）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

storage_flood_stage_left_capacity_bytes

默认值：1 * 1024 * 1024 * 1024 (1GB)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

如果磁盘容量达到 storage_flood_stage_usage_percent 和 storage_flood_stage_left_capacity_bytes 以下操作将被拒绝：

1. load 作业
2. restore 工作

storage_high_watermark_usage_percent

默认值：85 (85%)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

storage_min_left_capacity_bytes

默认值：2 * 1024 * 1024 * 1024 (2GB)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

storage_high_watermark_usage_percent 限制 BE 端存储路径使用最大容量百的分比。storage_min_left_
↪ capacity_bytes限制 BE 端存储路径的最小剩余容量。如果达到这两个限制，则不能选择此存储路径作为 tablet 存储目的地。但是对于 tablet 恢复，我们可能会超过这些限制以尽可能保持数据完整性。

catalog_trash_expire_second

默认值：86400L (1 天)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

删除数据库（表/分区）后，您可以使用 RECOVER stmt 恢复它。这指定了最大数据保留时间。一段时间后，数据将被永久删除。

storage_cooldown_second

默认值：30 * 24 * 3600L （30 天）

创建表（或分区）时，可以指定其存储介质（HDD 或 SSD）。如果设置为 SSD，这将指定 tablet 在 SSD 上停留的默认时间。之后，tablet 将自动移动到 HDD。您可以在 CREATE TABLE stmt 中设置存储冷却时间。

default_storage_medium

默认值：HDD

创建表（或分区）时，可以指定其存储介质（HDD 或 SSD）。如果未设置，则指定创建时的默认介质。

enable_storage_policy

是否开启 Storage Policy 功能。该功能用户冷热数据分离功能。

默认值：false。即不开启

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

check_consistency_default_timeout_second

默认值：600（10 分钟）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

单个一致性检查任务的默认超时。设置足够长以适合您的 tablet 大小。

consistency_check_start_time

默认值：23

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

一致性检查开始时间

一致性检查器将从 consistency_check_start_time 运行到 consistency_check_end_time。

如果两个时间相同，则不会触发一致性检查。

consistency_check_end_time

默认值：23

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

一致性检查结束时间

一致性检查器将从 consistency_check_start_time 运行到 consistency_check_end_time。

如果两个时间相同，则不会触发一致性检查。

replica_delay_recovery_second

默认值：0

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

副本之间的最小延迟秒数失败，并且尝试使用克隆来恢复它。

tablet_create_timeout_second

默认值：1（s）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

创建单个副本的最长等待时间。

例如。如果您为每个表创建一个包含 m 个 tablet 和 n 个副本的表，创建表请求将在超时前最多运行 (m n tablet_create_timeout_second)。

tablet_delete_timeout_second

默认值：2

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

与 tablet_create_timeout_second 含义相同，但在删除 tablet 时使用

delete_job_max_timeout_second

默认值：300(s)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

Delete 操作的最大超时时间，单位是秒

alter_table_timeout_second

默认值：86400 * 30（1 月）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

ALTER TABLE 请求的最大超时时间。设置足够长以适合您的表格数据大小

max_replica_count_when_schema_change

OlapTable 在做 schema change 时，允许的最大副本数，副本数过大会导致 FE OOM。

默认值：100000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

history_job_keep_max_second

默认值：7 * 24 * 3600（7 天）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

某些作业的最大保留时间。像 schema 更改和 Rollup 作业。

max_create_table_timeout_second

默认值：1 * 3600（1 小时）

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

为了在创建表（索引）不等待太久，设置一个最大超时时间

4.6.2.4.7 外部表

`file_scan_node_split_num`

默认值：128

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

multi catalog 并发文件扫描线程数

`file_scan_node_split_size`

默认值：`256 * 1024 * 1024`

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

multi catalog 并发文件扫描大小

`enable_odbc_mysql_broker_table`

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

从 2.1 版本开始，我们不再支持创建 odbc, mysql 和 broker 外表。对于 odbc 外表，可以使用 jdbc 外表或者 jdbc catalog 替代。对于 broker 外表，可以使用 table valued function 替代。

`max_hive_partition_cache_num`

hive partition 的最大缓存数量。

默认值：100000

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

`hive_metastore_client_timeout_second`

hive metastore 的默认超时时间

默认值：10

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

`max_external_cache_loader_thread_pool_size`

用于 external 外部表的 meta 缓存加载线程池的最大线程数。

默认值：10

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

max_external_file_cache_num

用于 external 外部表的最大文件缓存数量。

默认值：100000

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

max_external_schema_cache_num

用于 external 外部表的最大 schema 缓存数量。

默认值：10000

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

external_cache_expire_time_minutes_after_access

设置缓存中的数据，在最后一次访问后多久失效。单位为分钟。适用于 External Schema Cache 以及 Hive Partition Cache.

默认值：1440

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

es_state_sync_interval_second

默认值：10

FE 会在每隔 es_state_sync_interval_secs 调用 es api 获取 es 索引分片信息

4.6.2.4.8 外部资源

dpp_hadoop_client_path

默认值：/lib/hadoop-client/hadoop/bin/hadoop

dpp_bytes_per_reduce

默认值：100 * 1024 * 1024L (100M)

dpp_default_cluster

默认值：palo-dpp

dpp_default_config_str

默认值：{hadoop_configs：‘mapred.job.priority=NORMAL;mapred.job.map.capacity=50;mapred.job.reduce.capacity=50;mapred.hce.replace.stream
}

dpp_config_str

默认值:{palo-dpp:{hadoop_palo_path:‘/dir’,hadoop_configs:‘fs.default.name=hdfs://host:port;mapred.job.tracker=host:port;hadoop.job.ugi
}}

yarn_config_dir

默认值：DorisFE.DORIS_HOME_DIR + “/lib/yarn-config”

默认的 Yarn 配置文件目录每次运行 Yarn 命令之前，我们需要检查一下这个路径下是否存在 config 文件，如果不存在，则创建它们。

yarn_client_path

默认值：DorisFE.DORIS_HOME_DIR + “/lib/yarn-client/hadoop/bin/yarn”

默认 Yarn 客户端路径

spark_launcher_log_dir

默认值：sys_log_dir + “/spark_launcher_log”

指定的 Spark 启动器日志目录

spark_resource_path

默认值：空

默认值的 Spark 依赖路径

spark_home_default_dir

默认值：DorisFE.DORIS_HOME_DIR + “/lib/spark2x”

默认的 Spark home 路径

spark_dpp_version

默认值：1.0.0

Spark 默认版本号

4.6.2.4.9 其他参数

tmp_dir

默认值：DorisFE.DORIS_HOME_DIR + “/temp_dir”

temp dir 用于保存某些过程的中间结果，例如备份和恢复过程。这些过程完成后，将清除此目录中的文件。

custom_config_dir

默认值：DorisFE.DORIS_HOME_DIR + “/conf”

自定义配置文件目录

配置 fe_custom.conf 文件的位置。默认为 conf/ 目录下。

在某些部署环境下，conf/ 目录可能因为系统的版本升级被覆盖掉。这会导致用户在运行是持久化修改的配置项也被覆盖。这时，我们可以将 fe_custom.conf 存储在另一个指定的目录中，以防止配置文件被覆盖。

plugin_dir

默认值：DORIS_HOME + “/plugins

插件安装目录

plugin_enable

默认值:true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

插件是否启用，默认启用

small_file_dir

默认值：DORIS_HOME_DIR + “/small_files”

保存小文件的目录

max_small_file_size_bytes

默认值：1M

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

SmallFileMgr 中单个文件存储的最大大小

max_small_file_number

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

SmallFileMgr 中存储的最大文件数

enable_metric_calculator

默认值：true

如果设置为 true，指标收集器将作为守护程序计时器运行，以固定间隔收集指标

report_queue_size

默认值：100

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

这个阈值是为了避免在 FE 中堆积过多的报告任务，可能会导致 OOM 异常等问题。

并且每个 BE 每 1 分钟会报告一次 tablet 信息，因此无限制接收报告是不可接受的。以后我们会优化 tablet 报告的处理速度

不建议修改这个值

backup_job_default_timeout_ms

默认值：86400 * 1000 (1 天)

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

备份作业的默认超时时间

backup_upload_task_num_per_be

默认值：3

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

备份过程中，分配给每个 be 的 upload 任务最大个数，默认值为 3 个。

restore_download_task_num_per_be

默认值：3

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

恢复过程中，分配给每个 be 的 download 任务最大个数，默认值为 3 个。

max_backup_restore_job_num_per_db

默认值：10

此配置用于控制每个 DB 能够记录的 backup/restore 任务的数量

max_backup_tablets_per_job

默认值：300000

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

此配置用于控制每个 backup job 最大涉及的 tablets 数量，以避免因保存过多元数据导致 FE OOM。

enable_quantile_state_type

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

是否开启 quantile_state 数据类型

enable_date_conversion

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

FE 会自动将 Date/Datetime 转换为 DateV2/DatetimeV2(0)。

enable_decimal_conversion

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

FE 将自动将 DecimalV2 转换为 DecimalV3。

proxy_auth_magic_prefix

默认值：x@8

proxy_auth_enable

默认值：false

enable_func_pushdown

默认值：true

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

在 ODBC、JDBC 的 MYSQL 外部表查询时，是否将带函数的过滤条件下推到 MYSQL 中执行

jdbc_drivers_dir

默认值：${DORIS_HOME}/jdbc_drivers;

是否可以动态配置：false

是否为 Master FE 节点独有的配置项：false

用于存放默认的 jdbc drivers

max_error_tablet_of_broker_load

默认值：3;

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

broker load job 保存的失败 tablet 信息的最大数量

default_db_max_running_txn_num

默认值：-1

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：true

用于设置默认数据库事务配额大小。

默认值设置为 -1 意味着使用 max_running_txn_num_per_db 而不是 default_db_max_running_txn_num。

设置单个数据库的配额大小可以使用：

设置数据库事务量配额

```
ALTER DATABASE db_name SET TRANSACTION QUOTA quota;
```

查看配置

```
show data （其他用法：HELP SHOW DATA）
```

prefer_compute_node_for_external_table

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

如果设置为 true，对外部表的查询将优先分配给计算节点。计算节点的最大数量由 min_backend_num_for_external_table 控制。如果设置为 false，对外部表的查询将分配给任何节点。

min_backend_num_for_external_table

默认值：3

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

仅在 prefer_compute_node_for_external_table 为 true 时生效。如果计算节点数小于此值，则对外部表的查询将尝试使用一些混合节点，让节点总数达到这个值。如果计算节点数大于这个值，外部表的查询将只分配给计算节点。

infodb_support_ext_catalog

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

当设置为 false 时，查询 information_schema 中的表时，将不再返回 external catalog 中的表的信息。

这个参数主要用于避免因 external catalog 无法访问、信息过多等原因导致的查询 information_schema 超时的问题。

enable_query_hit_stats

默认值：false

是否可以动态配置：true

是否为 Master FE 节点独有的配置项：false

控制是否启用查询命中率统计。默认为 false。

div_precision_increment

默认值：4

此变量表示增加与/运算符执行的除法操作结果规模的位数。默认为 4。

enable_convert_light_weight_schema_change

默认值：true

暂时性配置项，开启后会启动后台线程自动将所有的 olap 表修改为可 light schema change，修改结果可通过命令show convert_light_schema_change [from db] 来查看，将会展示所有非 light schema change 表的转换结果

disable_local_deploy_manager_drop_node

默认值：true

禁止 LocalDeployManager 删除节点，防止 cluster.info 文件有误导致节点被删除。

mysqldb_replace_name

默认值：mysql

Doris 为了兼用 mysql 周边工具生态，会内置一个名为 mysql 的数据库，如果该数据库与用户自建数据库冲突，请修改这个字段，为 doris 内置的 mysql database 更换一个名字

max_auto_partition_num

默认值：2000

对于自动分区表，防止用户意外创建大量分区，每个 OLAP 表允许的分区数量为max_auto_partition_num。默认 2000。

profile_manager_gc_interval_seconds

默认值：1

用于控制 ProfileManager 进行 Profile 垃圾回收的间隔时间，垃圾回收期间 ProfileManager 会把多余的以及过期的 profile 从内存和磁盘中清理掉，节省内存。

4.6.2.4.10 存算分离模式

cluster_id

默认值：-1

如果节点（FE 或 BE）具有相同的集群 ID，则将认为它们属于同一个 Doris 集群。您应该在计算和存储分离模式中指定一个随机整数。

deploy_mode

默认值：“”

描述：FE 运行的模式。cloud 表示存算分离模式。

meta_service_endpoint

默认值：“”

Meta Service 的端点应以 ‘host1:port,host2:port’ 的格式指定。此配置对于存算分离模式是必要的。

4.6.3 BE 配置项

该文档主要介绍 BE 的相关配置项。

BE 的配置文件 be.conf 通常存放在 BE 部署路径的 conf/ 目录下。而在 0.14 版本中会引入另一个配置文件 be_custom.conf。该配置文件用于记录用户在运行时动态配置并持久化的配置项。

BE 进程启动后，会先读取 be.conf 中的配置项，之后再读取 be_custom.conf 中的配置项。be_custom.conf 中的配置项会覆盖 be.conf 中相同的配置项。

4.6.3.1 查看配置项

有两种方式能够查看 BE 配置项：

1. 通过 BE 前端页面查看

   在浏览器中打开 BE 前端页面：http://be_host:be_webserver_port/varz

2. 通过命令行查看

可以在 MySQL 客户端中，通过以下命令查看 BE 的配置项，具体语法参照SHOW-CONFIG：

```
`SHOW BACKEND CONFIG;`
```

结果中各列含义如下：

```
1. BackendId: backend 节点 ID。
2. Host: backend 节点 IP。
3. Key: 配置项名称。
4. Value: 配置项的值。
5. Type: 配置项值类型，如整型、字符串。
6. 是否可以动态配置。如果为 true，表示该配置项可以在运行时进行动态配置。如果 false，
    ↪ 则表示该配置项只能在 `be.conf` 中配置并且重启 BE 后生效。
```

4.6.3.2 设置配置项

BE 的配置项有两种方式进行配置：

1. 静态配置

在 conf/be.conf 文件中添加和设置配置项。be.conf 中的配置项会在 BE 进行启动时被读取。没有在 be.conf 中的配置项将使用默认值。

2. 动态配置

BE 启动后，可以通过以下命令动态设置配置项。

```
curl -X POST http://{be_ip}:{be_http_port}/api/update_config?{key}={value}
```

在 0.13 版本及之前，通过该方式修改的配置项将在 BE 进程重启后失效。在 0.14 及之后版本中，可以通过以下命令持久化修改后的配置。修改后的配置项存储在 be_custom.conf 文件中。

```
curl -X POST http://{be_ip}:{be_http_port}/api/update_config?{key}={value}\&persist=true
```

4.6.3.3 应用举例

1. 静态方式修改 max_base_compaction_threads

通过在 be.conf 文件中添加：

```
max_base_compaction_threads=5
```

之后重启 BE 进程以生效该配置。

2. 动态方式修改 streaming_load_max_mb

BE 启动后，通过下面命令动态设置配置项 streaming_load_max_mb:

```
curl -X POST http://{be_ip}:{be_http_port}/api/update_config?streaming_load_max_mb=1024
```

返回值如下，则说明设置成功。

```
{
    "status": "OK",
    "msg": ""
}
```

BE 重启后该配置将失效。如果想持久化修改结果，使用如下命令：

```
curl -X POST http://{be_ip}:{be_http_port}/api/update_config?streaming_load_max_mb=1024\&
    ↪ persist=true
```

4.6.3.4 配置项列表

4.6.3.4.1 服务

be_port

- 类型：int32
- 描述：BE 上 Thrift Server 的端口号，用于接收来自 FE 的请求
- 默认值：9060

heartbeat_service_port

- 类型：int32
- 描述：BE 上心跳服务端口（Thrift），用于接收来自 FE 的心跳
- 默认值：9050

webserver_port

- 类型：int32

- 描述：BE 上的 HTTP Server 的服务端口
- 默认值：8040

`brpc_port`

- 类型：int32
- 描述：BE 上的 brpc 的端口，用于 BE 之间通讯
- 默认值：8060

`arrow_flight_sql_port`

- 类型：int32
- 描述：FE 上的 Arrow Flight SQL Server 的端口，用于从 Arrow Flight Client 和 BE 之间通讯
- 默认值：-1

`enable_https`

- 类型：bool
- 描述：是否支持 https. 如果是，需要在 be.conf 中配置`ssl_certificate_path`和`ssl_private_key_path`
- 默认值：false

`priority_networks`

- 描述：为那些有很多 ip 的服务器声明一个选择策略。请注意，最多应该有一个 ip 与此列表匹配。这是一个以分号分隔格式的列表，用 CIDR 表示法，例如 10.10.10.0/24，如果没有匹配这条规则的 ip，会随机选择一个。
- 默认值：空

`storage_root_path`

- 类型：string
- 描述：BE 数据存储的目录，多目录之间用英文状态的分号;分隔。可以通过路径区别存储目录的介质，HDD 或 SSD。可以添加容量限制在每个路径的末尾，通过英文状态逗号,隔开。如果用户不是 SSD 和 HDD 磁盘混合使用的情况，不需要按照如下示例一和示例二的配置方法配置，只需指定存储目录即可；也不需要修改 FE 的默认存储介质配置

示例 1 如下：

注意：如果是 SSD 磁盘要在目录后面加上.SSD,HDD 磁盘在目录后面加.HDD

```
storage_root_path=/home/disk1/doris.HDD;/home/disk2/doris.SSD;/home/disk2/doris
```

说明

```
- /home/disk1/doris.HDD，表示存储介质是HDD;
- /home/disk2/doris.SSD，表示存储介质是SSD；
- /home/disk2/doris，存储介质默认为HDD
```

示例 2 如下：

注意：不论 HHD 磁盘目录还是 SSD 磁盘目录，都无需添加后缀，storage_root_path 参数里指定 medium 即可

```
storage_root_path=/home/disk1/doris,medium:hdd;/home/disk2/doris,medium:ssd
```

说明

```
- /home/disk1/doris,medium:hdd，表示存储介质是HHD;
- /home/disk2/doris,medium:ssd，表示存储介质是SSD;
```

- 默认值：${DORIS_HOME}/storage

`heartbeat_service_thread_count`

- 类型：int32
- 描述：执行 BE 上心跳服务的线程数，默认为 1，不建议修改
- 默认值：1

`ignore_broken_disk`

- 类型：bool
- 描述：当 BE 启动时，会检查`storage_root_path` 配置下的所有路径。
- `ignore_broken_disk=true`

如果路径不存在或路径下无法进行读写文件 (坏盘)，将忽略此路径，如果有其他可用路径则不中断启动。

- `ignore_broken_disk=false`

如果路径不存在或路径下无法进行读写文件 (坏盘)，将中断启动失败退出。

- 默认值：false

`mem_limit`

- 类型：string
- 描述：限制 BE 进程使用服务器最大内存百分比。用于防止 BE 内存挤占太多的机器内存，该参数必须大于 0，当百分大于 100% 之后，该值会默认为 100%。
- 默认值：90%

`cluster_id`

- 类型：int32
- 描述：配置 BE 的所属于的集群 id。

- 该值通常由 FE 通过心跳向 BE 下发，不需要额外进行配置。当确认某 BE 属于某一个确定的 Doris 集群时，可以进行配置，同时需要修改数据目录下的 cluster_id 文件，使二者相同。
- 默认值：-1

custom_config_dir

- 描述：配置 be_custom.conf 文件的位置。默认为 conf/ 目录下。
- 在某些部署环境下，conf/目录可能因为系统的版本升级被覆盖掉。这会导致用户在运行是持久化修改的配置项也被覆盖。这时，我们可以将 be_custom.conf 存储在另一个指定的目录中，以防止配置文件被覆盖。
- 默认值：空

trash_file_expire_time_sec

- 描述：回收站清理的间隔，24 个小时，当磁盘空间不足时，trash 下的文件保存期可不遵守这个参数
- 默认值：86400

es_http_timeout_ms

- 描述：通过 http 连接 ES 的超时时间，默认是 5 秒
- 默认值：5000 (ms)

es_scroll_keepalive

- 描述：es scroll keep-alive 保持时间，默认 5 分钟
- 默认值：5 (m)

external_table_connect_timeout_sec

- 类型：int32
- 描述：和外部表建立连接的超时时间。
- 默认值：5 秒

status_report_interval

- 描述：配置文件报告之间的间隔；单位：秒
- 默认值：5

brpc_max_body_size

- 描述：这个配置主要用来修改 brpc 的参数 max_body_size。
- 有时查询失败，在 BE 日志中会出现 body_size is too large 的错误信息。这可能发生在 SQL 模式为 multi distinct + 无 group by + 超过 1T 数据量的情况下。这个错误表示 brpc 的包大小超过了配置值。此时可以通过调大该配置避免这个错误。

brpc_socket_max_unwritten_bytes

- 描述：这个配置主要用来修改 brpc 的参数 socket_max_unwritten_bytes。
- 有时查询失败，BE 日志中会出现 The server is overcrowded 的错误信息，表示连接上有过多的未发送数据。当查询需要发送较大的 bitmap 字段时，可能会遇到该问题，此时可能通过调大该配置避免该错误。

transfer_large_data_by_brpc

- 类型：bool
- 描述：该配置用来控制是否在 Tuple/Block data 长度大于 1.8G 时，将 protoBuf request 序列化后和 Tuple/Block data 一起嵌入到 controller attachment 后通过 http brpc 发送。为了避免 protoBuf request 的长度超过 2G 时的错误：Bad request, error_text=[E1003]Fail to compress request。在过去的版本中，曾将 Tuple/Block data 放入 attachment 后通过默认的 baidu_std brpc 发送，但 attachment 超过 2G 时将被截断，通过 http brpc 发送不存在 2G 的限制。
- 默认值：true

brpc_num_threads

- 描述：该配置主要用来修改 brpc 中 bthreads 的数量。该配置的默认值被设置为 -1, 这意味着 bthreads 的数量将被设置为机器的 cpu 核数。
- 用户可以将该配置的值调大来获取更好的 QPS 性能。更多的信息可以参考 https://github.com/apache/brpc/blob/master/docs/cn/benchmark.md。
- 默认值：-1

thrift_rpc_timeout_ms

- 描述：Thrift 默认超时时间
- 默认值：60000

thrift_client_retry_interval_ms

- 类型：int64
- 描述：用来为 be 的 thrift 客户端设置重试间隔，避免 FE 的 Thrift Server 发生雪崩问题，单位为 ms。
- 默认值：1000

thrift_connect_timeout_seconds

- 描述：默认 Thrift 客户端连接超时时间
- 默认值：3 (s)

thrift_server_type_of_fe

- 类型：string
- 描述：该配置表示 FE 的 Thrift 服务使用的服务模型，类型为 string, 大小写不敏感，该参数需要和 FE 的 thrift_server_type 参数的设置保持一致。目前该参数的取值有两个，THREADED和THREAD_POOL。
- 若该参数为THREADED, 该模型为非阻塞式 I/O 模型，
- 若该参数为THREAD_POOL, 该模型为阻塞式 I/O 模型。

`thrift_max_message_size`

默认值：100MB

Thrift 服务器接收请求消息的大小（字节数）上限。如果客户端发送的消息大小超过该值，那么 Thrift 服务器会拒绝该请求并关闭连接，这种情况下，client 会遇到错误：“connection has been closed by peer”，使用者可以尝试增大该参数以绕过上述限制。

`txn_commit_rpc_timeout_ms`

- 描述：txn 提交 rpc 超时
- 默认值：60000 (ms)

`txn_map_shard_size`

- 描述：txn_map_lock 分片大小，取值为 2^n，n=0,1,2,3,4。这是一项增强功能，可提高管理 txn 的性能
- 默认值：128

`txn_shard_size`

- 描述：txn_lock 分片大小，取值为 2^n，n=0,1,2,3,4，这是一项增强功能，可提高提交和发布 txn 的性能
- 默认值：1024

`unused_rowset_monitor_interval`

- 描述：清理过期 Rowset 的时间间隔
- 默认值：30 (s)

`max_client_cache_size_per_host`

- 描述：每个主机的最大客户端缓存数，BE 中有多种客户端缓存，但目前我们使用相同的缓存大小配置。如有必要，使用不同的配置来设置不同的客户端缓存。
- 默认值：10

`string_type_length_soft_limit_bytes`

- 类型：int32
- 描述：String 类型最大长度的软限，单位是字节
- 默认值：1048576

`big_column_size_buffer`

- 类型：int64
- 描述：当使用 odbc 外表时，如果 odbc 源表的某一列类型为 HLL, CHAR 或者 VARCHAR，并且列值长度超过该值，则查询报错’column value length longer than buffer length’. 可增大该值
- 默认值：65535

`small_column_size_buffer`

- 类型：int64
- 描述：当使用 odbc 外表时，如果 odbc 源表的某一列类型不是 HLL, CHAR 或者 VARCHAR，并且列值长度超过该值，则查询报错’column value length longer than buffer length’. 可增大该值
- 默认值：100

`jsonb_type_length_soft_limit_bytes`

- 类型：int32
- 描述：JSONB 类型最大长度的软限，单位是字节
- 默认值：1048576

4.6.3.4.2 查询

`fragment_pool_queue_size`

- 描述：单节点上能够处理的查询请求上限
- 默认值：4096

`fragment_pool_thread_num_min`

- 描述：查询线程数，默认最小启动 64 个线程。
- 默认值：64

`fragment_pool_thread_num_max`

- 描述：后续查询请求动态创建线程，最大创建 512 个线程。
- 默认值：2048

`doris_scan_range_row_count`

- 类型：int32
- 描述：BE 在进行数据扫描时，会将同一个扫描范围拆分为多个 ScanRange。该参数代表了每个 ScanRange 代表扫描数据范围。通过该参数可以限制单个 OlapScanner 占用 io 线程的时间。
- 默认值：524288

`doris_scanner_row_num`

- 描述：每个扫描线程单次执行最多返回的数据行数
- 默认值：16384

`doris_scanner_row_bytes`

- 描述：每个扫描线程单次执行最多返回的数据字节
- 说明：如果表的列数太多，遇到 `select *` 卡主，可以调整这个配置
- 默认值：10485760

`doris_scanner_thread_pool_queue_size`

- 类型：int32
- 描述：Scanner 线程池的队列长度。在 Doris 的扫描任务之中，每一个 Scanner 会作为一个线程 Task 提交到线程池之中等待被调度，而提交的任务数目超过线程池队列的长度之后，后续提交的任务将阻塞直到队列之中有新的空缺。
- 默认值：102400

`doris_scanner_thread_pool_thread_num`

- 类型：int32
- 描述：Scanner 线程池线程数目。在 Doris 的扫描任务之中，每一个 Scanner 会作为一个线程 Task 提交到线程池之中等待被调度，该参数决定了 Scanner 线程池的大小。
- 默认值：取决于 CPU 核心数量。等于 `max(48, num_of_cpu_cores)`

`doris_max_remote_scanner_thread_pool_thread_num`

- 类型：int32
- 描述：Remote scanner thread pool 的最大线程数。 Remote scanner thread pool 用于除内表外的所有 scan 任务的执行。
- 默认值：512

`exchg_node_buffer_size_bytes`

- 类型：int32
- 描述：ExchangeNode 节点 Buffer 队列的大小，单位为 byte。来自 Sender 端发送的数据量大于 ExchangeNode 的 Buffer 大小之后，后续发送的数据将阻塞直到 Buffer 腾出可写入的空间。
- 默认值：10485760

`doris_scan_range_max_mb`

- 类型：int32
- 描述：每个 OlapScanner 读取的最大数据量
- 默认值：1024

4.6.3.4.3 compaction

`disable_auto_compaction`

- 类型：bool
- 描述：关闭自动执行 compaction 任务
- 一般需要为关闭状态，当调试或测试环境中想要手动操作 compaction 任务时，可以对该配置进行开启
- 默认值：false

`enable_vertical_compaction`

- 类型：bool
- 描述：是否开启列式 compaction
- 默认值：true

`vertical_compaction_num_columns_per_group`

- 类型：int32
- 描述：在列式 compaction 中，组成一个合并组的列个数
- 默认值：5

`vertical_compaction_max_row_source_memory_mb`

- 类型：int32
- 描述：在列式 compaction 中，row_source_buffer 能使用的最大内存，单位是 MB。
- 默认值：200

`vertical_compaction_max_segment_size`

- 类型：int32
- 描述：在列式 compaction 中，输出的 segment 文件最大值，单位是 m 字节。
- 默认值：268435456

`enable_ordered_data_compaction`

- 类型：bool
- 描述：是否开启有序数据的 compaction
- 默认值：true

`ordered_data_compaction_min_segment_size`

- 类型：int32
- 描述：在有序数据 compaction 中，满足要求的最小 segment 大小，单位是 m 字节。
- 默认值：10485760

`max_base_compaction_threads`

- 类型：int32
- 描述：Base Compaction 线程池中线程数量的最大值，-1 表示每个磁盘一个线程。
- 默认值：4

`generate_compaction_tasks_interval_ms`

- 描述：生成 compaction 作业的最小间隔时间
- 默认值：10（ms）

`base_compaction_min_rowset_num`

- 描述：BaseCompaction 触发条件之一：Cumulative 文件数目要达到的限制，达到这个限制之后会触发 BaseCompaction
- 默认值：5

`base_compaction_min_data_ratio`

- 描述：BaseCompaction 触发条件之一：Cumulative 文件大小达到 Base 文件的比例。
- 默认值：0.3（30%）

`total_permits_for_compaction_score`

- 类型：int64
- 描述：被所有的 compaction 任务所能持有的“permits”上限，用来限制 compaction 占用的内存。
- 默认值：10000
- 可动态修改：是

`compaction_promotion_size_mbytes`

- 类型：int64
- 描述：cumulative compaction 的输出 rowset 总磁盘大小超过了此配置大小，该 rowset 将用于 base compaction。单位是 m 字节。
- 一般情况下，配置在 2G 以内，为了防止 cumulative compaction 时间过长，导致版本积压。
- 默认值：1024

`compaction_promotion_ratio`

- 类型：double
- 描述：cumulative compaction 的输出 rowset 总磁盘大小超过 base 版本 rowset 的配置比例时，该 rowset 将用于 base compaction。
- 一般情况下，建议配置不要高于 0.1，低于 0.02。
- 默认值：0.05

`compaction_promotion_min_size_mbytes`

- 类型：int64
- 描述：Cumulative compaction 的输出 rowset 总磁盘大小低于此配置大小，该 rowset 将不进行 base compaction，仍然处于 cumulative compaction 流程中。单位是 m 字节。
- 一般情况下，配置在 512m 以内，配置过大会导致 base 版本早期的大小过小，一直不进行 base compaction。
- 默认值：128

`compaction_min_size_mbytes`

- 类型：int64
- 描述：cumulative compaction 进行合并时，选出的要进行合并的 rowset 的总磁盘大小大于此配置时，才按级别策略划分合并。小于这个配置时，直接执行合并。单位是 m 字节。
- 一般情况下，配置在 128m 以内，配置过大会导致 cumulative compaction 写放大较多。
- 默认值：64

`default_rowset_type`

- 类型：string
- 描述：标识 BE 默认选择的存储格式，可配置的参数为：“ALPHA”，“BETA”。主要起以下两个作用
- 当建表的 storage_format 设置为 Default 时，通过该配置来选取 BE 的存储格式。
- 进行 Compaction 时选择 BE 的存储格式
- 默认值：BETA

`cumulative_compaction_min_deltas`

- 描述：cumulative compaction 策略：最小增量文件的数量
- 默认值：5

`cumulative_compaction_max_deltas`

- 描述：cumulative compaction 策略：最大增量文件的数量
- 默认值：1000

`base_compaction_trace_threshold`

- 类型：int32
- 描述：打印 base compaction 的 trace 信息的阈值，单位秒
- 默认值：10

base compaction 是一个耗时较长的后台操作，为了跟踪其运行信息，可以调整这个阈值参数来控制 trace 日志的打印。打印信息如下：

```
W0610 11:26:33.804431 56452 storage_engine.cpp:552] execute base compaction cost 0.00319222
BaseCompaction:546859:
  - filtered_rows: 0
   - input_row_num: 10
   - input_rowsets_count: 10
   - input_rowsets_data_size: 2.17 KB
   - input_segments_num: 10
   - merge_rowsets_latency: 100000.510ms
   - merged_rows: 0
   - output_row_num: 10
   - output_rowset_data_size: 224.00 B
   - output_segments_num: 1
0610 11:23:03.727535 (+     0us) storage_engine.cpp:554] start to perform base compaction
0610 11:23:03.728961 (+  1426us) storage_engine.cpp:560] found best tablet 546859
0610 11:23:03.728963 (+     2us) base_compaction.cpp:40] got base compaction lock
0610 11:23:03.729029 (+    66us) base_compaction.cpp:44] rowsets picked
0610 11:24:51.784439 (+108055410us) compaction.cpp:46] got concurrency lock and start to do
    ↪ compaction
0610 11:24:51.784818 (+   379us) compaction.cpp:74] prepare finished
0610 11:26:33.359265 (+101574447us) compaction.cpp:87] merge rowsets finished
0610 11:26:33.484481 (+125216us) compaction.cpp:102] output rowset built
0610 11:26:33.484482 (+     1us) compaction.cpp:106] check correctness finished
0610 11:26:33.513197 (+ 28715us) compaction.cpp:110] modify rowsets finished
0610 11:26:33.513300 (+   103us) base_compaction.cpp:49] compaction finished
0610 11:26:33.513441 (+   141us) base_compaction.cpp:56] unused rowsets have been moved to GC
    ↪ queue
```

`cumulative_compaction_trace_threshold`

- 类型：int32
- 描述：打印 cumulative compaction 的 trace 信息的阈值，单位秒
- 与 base_compaction_trace_threshold 类似。
- 默认值：2

`compaction_task_num_per_disk`

- 类型：int32
- 描述：每个磁盘（HDD）可以并发执行的 compaction 任务数量。
- 默认值：4

`compaction_task_num_per_fast_disk`

- 类型：int32
- 描述：每个高速磁盘（SSD）可以并发执行的 compaction 任务数量。

- 默认值：8

cumulative_compaction_rounds_for_each_base_compaction_round

- 类型：int32
- 描述：Compaction 任务的生产者每次连续生产多少轮 cumulative compaction 任务后生产一轮 base compaction。
- 默认值：9

cumulative_compaction_policy

- 类型：string
- 描述：配置 cumulative compaction 阶段的合并策略，目前实现了两种合并策略，num_based 和 size_based
- 详细说明，ordinary，是最初版本的 cumulative compaction 合并策略，做一次 cumulative compaction 之后直接 base compaction 流程。size_based，通用策略是 ordinary 策略的优化版本，仅当 rowset 的磁盘体积在相同数量级时才进行版本合并。合并之后满足条件的 rowset 进行晋升到 base compaction 阶段。能够做到在大量小批量导入的情况下：降低 base compact 的写入放大率，并在读取放大率和空间放大率之间进行权衡，同时减少了文件版本的数据。
- 默认值：size_based

max_cumu_compaction_threads

- 类型：int32
- 描述：Cumulative Compaction 线程池中线程数量的最大值，-1 表示每个磁盘一个线程。
- 默认值：-1

enable_segcompaction

- 类型：bool
- 描述：在导入时进行 segment compaction 来减少 segment 数量，以避免出现写入时的 -238 错误
- 默认值：true

segcompaction_batch_size

- 类型：int32
- 描述：当 segment 数量超过此阈值时触发 segment compaction，该配置也限制了单个 segment compaction 任务中的最大原始 segment 数量。
- 默认值：10

segcompaction_candidate_max_rows

- 类型：int32
- 描述：当 segment 的行数超过此大小时则会在 segment compaction 时被 compact，否则跳过
- 默认值：1048576

segcompaction_candidate_max_rows

- 类型：int32
- 描述：segment compaction 任务中允许的单个原始 segment 行数，过大的 segment 将被跳过。
- 默认值：1048576

segcompaction_candidate_max_bytes

- 类型：int64
- 描述：segment compaction 任务中允许的单个原始 segment 大小（字节），过大的 segment 将被跳过。
- 默认值：104857600

segcompaction_task_max_rows

- 类型：int32
- 描述：单个 segment compaction 任务中允许的原始 segment 总行数。
- 默认值：1572864

segcompaction_task_max_bytes

- 类型：int64
- 描述：单个 segment compaction 任务中允许的原始 segment 总大小（字节）。
- 默认值：157286400

segcompaction_num_threads

- 类型：int32
- 描述：segment compaction 线程池大小。
- 默认值：5

disable_compaction_trace_log

- 类型：bool
- 描述：关闭 compaction 的 trace 日志
- 如果设置为 true，cumulative_compaction_trace_threshold 和 base_compaction_trace_threshold 将不起作用。并且 trace 日志将关闭。
- 默认值：true

pick_rowset_to_compact_interval_sec

- 类型：int64
- 描述：选取 rowset 去合并的时间间隔，单位为秒
- 默认值：86400

max_single_replica_compaction_threads

- 类型：int32

- 描述：Single Replica Compaction 线程池中线程数量的最大值，-1 表示每个磁盘一个线程。
- 默认值：-1

update_replica_infos_interval_seconds

- 描述：更新 peer replica infos 的最小间隔时间
- 默认值：60（s）

4.6.3.4.4 导入

enable_stream_load_record

- 类型：bool
- 描述：是否开启 stream load 操作记录，默认是不启用
- 默认值：false

load_data_reserve_hours

- 描述：用于 mini load。 mini load 数据文件将在此时间后被删除
- 默认值：4（h）

push_worker_count_high_priority

- 描述：导入线程数，用于处理 HIGH 优先级任务
- 默认值：3

push_worker_count_normal_priority

- 描述：导入线程数，用于处理 NORMAL 优先级任务
- 默认值：3

enable_single_replica_load

- 描述：是否启动单副本数据导入功能
- 默认值：true

load_error_log_reserve_hours

- 描述：load 错误日志将在此时间后删除
- 默认值：48（h）

load_error_log_limit_bytes

- 描述：load 错误日志大小超过此值将被截断
- 默认值：209715200 (byte)

load_process_max_memory_limit_percent

- 描述：单节点上所有的导入线程占据的内存上限比例
- 将这些默认值设置得很大，因为我们不想在用户升级 Doris 时影响负载性能。如有必要，用户应正确设置这些配置。
- 默认值：50（%）

load_process_soft_mem_limit_percent

- 描述：soft limit 是指站单节点导入内存上限的比例。例如所有导入任务导入的内存上限是 20GB，则 soft limit 默认为该值的 50%，即 10GB。导入内存占用超过 soft limit 时，会挑选占用内存最大的作业进行下刷以提前释放内存空间。
- 默认值：50（%）

routine_load_thread_pool_size

- 描述：routine load 任务的线程池大小。这应该大于 FE 配置 ‘max_concurrent_task_num_per_be’
- 默认值：10

slave_replica_writer_rpc_timeout_sec

- 类型：int32
- 描述：单副本数据导入功能中，Master 副本和 Slave 副本之间通信的 RPC 超时时间。
- 默认值：60

max_segment_num_per_rowset

- 类型：int32
- 描述：用于限制导入时，新产生的 rowset 中的 segment 数量。如果超过阈值，导入会失败并报错 -238。过多的 segment 会导致 compaction 占用大量内存引发 OOM 错误。
- 默认值：200

high_priority_flush_thread_num_per_store

- 类型：int32
- 描述：每个存储路径所分配的用于高优导入任务的 flush 线程数量。
- 默认值：1

routine_load_consumer_pool_size

- 类型：int32
- 描述：routine load 所使用的 data consumer 的缓存数量。
- 默认值：10

multi_table_batch_plan_threshold

- 类型：int32
- 描述：一流多表使用该配置，表示攒多少条数据再进行规划。过小的值会导致规划频繁，多大的值会增加内存压力和导入延迟。
- 默认值：200

multi_table_max_wait_tables

- 类型：int32
- 描述：一流多表使用该配置，如果等待执行的表的数量大于此阈值，将请求并执行所有相关表的计划。该参数旨在避免一次同时请求和执行过多的计划。将导入过程的多表进行小批处理，可以减少单次 rpc 的压力，同时可以提高导入数据处理的实时性。
- 默认值：5

single_replica_load_download_num_workers

- 类型：int32
- 描述：单副本数据导入功能中，Slave 副本通过 HTTP 从 Master 副本下载数据文件的工作线程数。导入并发增大时，可以适当调大该参数来保证 Slave 副本及时同步 Master 副本数据。必要时也应相应地调大webserver_num_workers来提高 IO 效率。
- 默认值：64

load_task_high_priority_threshold_second

- 类型：int32
- 描述：当一个导入任务的超时时间小于这个阈值是，Doris 将认为他是一个高优任务。高优任务会使用独立的 flush 线程池。
- 默认：120

min_load_rpc_timeout_ms

- 类型：int32
- 描述：load 作业中各个 rpc 的最小超时时间。
- 默认：20

kafka_api_version_request

- 类型：bool
- 描述：如果依赖的 kafka 版本低于 0.10.0.0, 该值应该被设置为 false。
- 默认：true

kafka_broker_version_fallback

- 描述：如果依赖的 kafka 版本低于 0.10.0.0, 当 kafka_api_version_request 值为 false 的时候，将使用回退版本 kafka_broker_version_fallback 设置的值，有效值为：0.9.0.x、 0.8.x.y。

- 默认：0.10.0

`max_consumer_num_per_group`

- 描述：一个数据消费者组中的最大消费者数量，用于 routine load。
- 默认：3

`streaming_load_max_mb`

- 类型：int64
- 描述：用于限制数据格式为 csv 的一次 Stream load 导入中，允许的最大数据量。
- Stream Load 一般适用于导入几个 GB 以内的数据，不适合导入过大的数据。
- 默认值：10240（MB）
- 可动态修改：是

`streaming_load_json_max_mb`

- 类型：int64
- 描述：用于限制数据格式为 json 的一次 Stream load 导入中，允许的最大数据量。单位 MB。
- 一些数据格式，如 JSON，无法进行拆分处理，必须读取全部数据到内存后才能开始解析，因此，这个值用于限制此类格式数据单次导入最大数据量。
- 默认值：100
- 可动态修改：是

`olap_table_sink_send_interval_microseconds`.

- 描述：数据导入时，Coordinator 的 sink 节点有一个轮询线程持续向对应 BE 发送数据。该线程将每隔 `olap_table_sink_send_interval_microseconds` 微秒检查是否有数据要发送。
- 默认值：1000

`olap_table_sink_send_interval_auto_partition_factor`.

- 描述：如果我们向一个启用了自动分区的表导入数据，那么 `olap_table_sink_send_interval_microseconds` 的时间间隔就会太慢。在这种情况下，实际间隔将乘以该系数。
- 默认值：0.001

4.6.3.4.5 线程

`delete_worker_count`

- 描述：执行数据删除任务的线程数
- 默认值：3

`clear_transaction_task_worker_count`

- 描述：用于清理事务的线程数
- 默认值：1

clone_worker_count

- 描述：用于执行克隆任务的线程数
- 默认值：3

be_service_threads

- 类型：int32
- 描述：BE 上 Thrift Server Service 的执行线程数，代表可以用于执行 FE 请求的线程数。
- 默认值：64

download_worker_count

- 描述：下载线程数
- 默认值：1

drop_tablet_worker_count

- 描述：删除 tablet 的线程数
- 默认值：3

flush_thread_num_per_store

- 描述：每个 store 用于刷新内存表的线程数
- 默认值：2

num_threads_per_core

- 描述：控制每个内核运行工作的线程数。通常选择 2 倍或 3 倍的内核数量。这使核心保持忙碌而不会导致过度抖动
- 默认值：3

num_threads_per_disk

- 描述：每个磁盘的最大线程数也是每个磁盘的最大队列深度
- 默认值：0

number_slave_replica_download_threads

- 描述：每个 BE 节点上 slave 副本同步 Master 副本数据的线程数，用于单副本数据导入功能。
- 默认值：64

publish_version_worker_count

- 描述：生效版本的线程数
- 默认值：8

upload_worker_count

- 描述：上传文件最大线程数
- 默认值：1

webserver_num_workers

- 描述：webserver 默认工作线程数
- 默认值：48

send_batch_thread_pool_thread_num

- 类型：int32
- 描述：SendBatch 线程池线程数目。在 NodeChannel 的发送数据任务之中，每一个 NodeChannel 的 SendBatch 操作会作为一个线程 Task 提交到线程池之中等待被调度，该参数决定了 SendBatch 线程池的大小。
- 默认值：64

send_batch_thread_pool_queue_size

- 类型：int32
- 描述：SendBatch 线程池的队列长度。在 NodeChannel 的发送数据任务之中，每一个 NodeChannel 的 SendBatch 操作会作为一个线程 Task 提交到线程池之中等待被调度，而提交的任务数目超过线程池队列的长度之后，后续提交的任务将阻塞直到队列之中有新的空缺。
- 默认值：102400

make_snapshot_worker_count

- 描述：制作快照的线程数
- 默认值：5

release_snapshot_worker_count

- 描述：释放快照的线程数
- 默认值：5

4.6.3.4.6 内存

`disable_mem_pools`

- 类型：bool
- 描述：是否禁用内存缓存池
- 默认值：false

`buffer_pool_clean_pages_limit`

- 描述：清理可能被缓冲池保存的 Page
- 默认值：50%

`buffer_pool_limit`

- 类型：string
- 描述：buffer pool 之中最大的可分配内存
- BE 缓存池最大的内存可用量，buffer pool 是 BE 新的内存管理结构，通过 buffer page 来进行内存管理，并能够实现数据的落盘。并发的所有查询的内存申请都会通过 buffer pool 来申请。当前 buffer pool 仅作用在 AggregationNode 与 ExchangeNode。
- 默认值：20%

`chunk_reserved_bytes_limit`

- 描述：Chunk Allocator 的 reserved bytes 限制，通常被设置为 mem_limit 的百分比。默认单位字节，值必须是 2 的倍数，且必须大于 0，如果大于物理内存，将被设置为物理内存大小。增加这个变量可以提高性能，但是会获得更多其他模块无法使用的空闲内存。
- 默认值：20%

`max_memory_sink_batch_count`

- 描述：最大外部扫描缓存批次计数，表示缓存 max_memory_cache_batch_count * batch_size row，默认为 20，batch_size 的默认值为 1024，表示将缓存 20 * 1024 行。
- 默认值：20

`memtable_mem_tracker_refresh_interval_ms`

- 描述：memtable 主动下刷时刷新内存统计的周期（毫秒）
- 默认值：100

`zone_map_row_num_threshold`

- 类型：int32
- 描述：如果一个 page 中的行数小于这个值就不会创建 zonemap，用来减少数据膨胀。
- 默认值：20

enable_tcmalloc_hook

- 类型：bool
- 描述：是否 Hook TCmalloc new/delete，目前在 Hook 中统计 thread local MemTracker。
- 默认值：true

memory_mode

- 类型：string
- 描述：控制 tcmalloc 的回收。如果配置为 performance，内存使用超过 mem_limit 的 90% 时，doris 会释放 tcmalloc cache 中的内存，如果配置为 compact，内存使用超过 mem_limit 的 50% 时，doris 会释放 tcmalloc cache 中的内存。
- 默认值：performance

max_sys_mem_available_low_water_mark_bytes

- 类型：int64
- 描述：系统/proc/meminfo/MemAvailable 的最大低水位线，单位字节，默认 1.6G，实际低水位线 =min(1.6G，MemTotal * 10%)，避免在大于 16G 的机器上浪费过多内存。调大 max，在大于 16G 内存的机器上，将为 Full GC 预留更多的内存 buffer；反之调小 max，将尽可能充分使用内存。
- 默认值：1717986918

memory_limitation_per_thread_for_schema_change_bytes

- 描述：单个 schema change 任务允许占用的最大内存。
- 默认值：2147483648 (2GB)

mem_tracker_consume_min_size_bytes

- 类型：int32
- 描述：TCMalloc Hook consume/release MemTracker 时的最小长度，小于该值的 consume size 会持续累加，避免频繁调用 MemTracker 的 consume/release，减小该值会增加 consume/release 的频率，增大该值会导致 MemTracker 统计不准，理论上一个 MemTracker 的统计值与真实值相差 = (mem_tracker_consume_min_size_bytes * 这个 MemTracker 所在的 BE 线程数)。
- 默认值：1048576

cache_clean_interval

- 描述：文件句柄缓存清理的间隔，用于清理长期不用的文件句柄。同时也是 Segment Cache 的清理间隔时间。
- 默认值：1800 (s)

min_buffer_size

- 描述：最小读取缓冲区大小

- 默认值：1024 (byte)

`write_buffer_size`

- 描述：刷写前缓冲区的大小
- 导入数据在 BE 上会先写入到一个内存块，当这个内存块达到阈值后才会写回磁盘。默认大小是 100MB。过小的阈值可能导致 BE 上存在大量的小文件。可以适当提高这个阈值减少文件数量。但过大的阈值可能导致 RPC 超时
- 默认值：104857600

`remote_storage_read_buffer_mb`

- 类型：int32
- 描述：读取 hdfs 或者对象存储上的文件时，使用的缓存大小。
- 增大这个值，可以减少远端数据读取的调用次数，但会增加内存开销。
- 默认值：16MB

`file_cache_alive_time_sec`

- 类型：int64
- 描述：缓存文件的保存时间，单位：秒
- 默认值：604800（1 个星期）

`file_cache_max_size_per_disk`

- 类型：int64
- 描述：缓存占用磁盘大小，一旦超过这个设置，会删除最久未访问的缓存，为 0 则不限制大小。单位：字节。
- 默认值：0

`max_sub_cache_file_size`

- 类型：int64
- 描述：缓存文件使用 sub_file_cache 时，切分文件的最大大小，单位 B
- 默认值：104857600（100MB）

`download_cache_thread_pool_thread_num`

- 类型：int32
- 描述：DownloadCache 线程池线程数目。在 FileCache 的缓存下载任务之中，缓存下载操作会作为一个线程 Task 提交到线程池之中等待被调度，该参数决定了 DownloadCache 线程池的大小。
- 默认值：48

`download_cache_thread_pool_queue_size`

- Type: int32
- 描述：DownloadCache 线程池线程数目。在 FileCache 的缓存下载任务之中，缓存下载操作会作为一个线程 Task 提交到线程池之中等待被调度，而提交的任务数目超过线程池队列的长度之后，后续提交的任务将阻塞直到队列之中有新的空缺。
- 默认值：102400

`generate_cache_cleaner_task_interval_sec`

- 类型：int64
- 描述：缓存文件的清理间隔，单位：秒
- 默认值：43200（12 小时）

`path_gc_check`

- 类型：bool
- 描述：是否启用回收扫描数据线程检查
- 默认值：true

`path_gc_check_interval_second`

- 描述：回收扫描数据线程检查时间间隔
- 默认值：86400 (s)

`path_gc_check_step`

- 默认值：1000

`path_gc_check_step_interval_ms`

- 默认值：10 (ms)

`path_scan_interval_second`

- 默认值：86400

`scan_context_gc_interval_min`

- 描述：此配置用于上下文 gc 线程调度周期
- 默认值：5 (分钟)

4.6.3.4.7 存储

`default_num_rows_per_column_file_block`

- 类型：int32
- 描述：配置单个 RowBlock 之中包含多少行的数据。
- 默认值：1024

`disable_storage_page_cache`

- 类型：bool
- 描述：是否进行使用 page cache 进行 index 的缓存，该配置仅在 BETA 存储格式时生效
- 默认值：false

`disk_stat_monitor_interval`

- 描述：磁盘状态检查时间间隔。
- 默认值：5（s）

`max_garbage_sweep_interval`

- 描述：磁盘进行垃圾清理的最大间隔。
- 默认值：3600 (s)

`max_percentage_of_error_disk`

- 类型：int32
- 描述：存储引擎允许存在损坏硬盘的百分比，损坏硬盘超过改比例后，BE 将会自动退出。
- 默认值：0

`read_size`

- 描述：读取大小是发送到 os 的读取大小。在延迟和整个过程之间进行权衡，试图让磁盘保持忙碌但不引入寻道。对于 8 MB 读取，随机 io 和顺序 io 的性能相似
- 默认值：8388608

`min_garbage_sweep_interval`

- 描述：磁盘进行垃圾清理的最小间隔
- 默认值：180 (s)

`pprof_profile_dir`

- 描述：pprof profile 保存目录。
- 默认值：${DORIS_HOME}/log

small_file_dir

- 描述：用于保存 SmallFileMgr 下载的文件的目录。
- 默认值：${DORIS_HOME}/lib/small_file/

user_function_dir

- 描述：udf 函数目录。
- 默认值：${DORIS_HOME}/lib/udf

storage_flood_stage_left_capacity_bytes

- 描述：数据目录应该剩下的最小存储空间，默认 1G。
- 默认值：1073741824

storage_flood_stage_usage_percent

- 描述：storage_flood_stage_usage_percent 和 storage_flood_stage_left_capacity_bytes 两个配置限制了数据目录的磁盘容量的最大使用。如果这两个阈值都达到，则无法将更多数据写入该数据目录。数据目录的最大已用容量百分比
- 默认值：90（90%）

storage_medium_migrate_count

- 描述：要克隆的线程数。
- 默认值：1

storage_page_cache_limit

- 描述：缓存存储页大小。
- 默认值：20%

storage_page_cache_shard_size

- 描述：StoragePageCache 的分片大小，值为 2^n (n=0,1,2,…)。建议设置为接近 BE CPU 核数的值，可减少 StoragePageCache 的锁竞争。
- 默认值：16

index_page_cache_percentage

- 类型：int32
- 描述：索引页缓存占总页面缓存的百分比，取值为 [0, 100]。
- 默认值：10

`segment_cache_capacity`

- Type: int32
- Description: segment 元数据缓存（以 rowset id 为 key）的最大 rowset 个数。-1 代表向后兼容取值为 fd_number * 2/5
- Default value: -1

`storage_strict_check_incompatible_old_format`

- 类型：bool
- 描述：用来检查不兼容的旧版本格式时是否使用严格的验证方式。
- 配置用来检查不兼容的旧版本格式时是否使用严格的验证方式，当含有旧版本的 hdr 格式时，使用严谨的方式时，程序会打出 `fatal log` 并且退出运行；否则，程序仅打印 `warn log`.
- 默认值：true
- 可动态修改：否

`sync_tablet_meta`

- 描述：存储引擎是否开 sync 保留到磁盘上
- 默认值：true

`pending_data_expire_time_sec`

- 描述：存储引擎保留的未生效数据的最大时长
- 默认值：1800 (s)

`ignore_rowset_stale_inconsistent_delete`

- 类型：bool
- 描述：用来决定当删除过期的合并过的 rowset 后无法构成一致的版本路径时，是否仍要删除。
- 合并的过期 rowset 版本路径会在半个小时后进行删除。在异常下，删除这些版本会出现构造不出查询一致路径的问题，当配置为 false 时，程序检查比较严格，程序会直接报错退出。当配置为 true 时，程序会正常运行，忽略这个错误。一般情况下，忽略这个错误不会对查询造成影响，仅会在 fe 下发了合并过的版本时出现 -230 错误。
- 默认值：false

`create_tablet_worker_count`

- 描述：BE 创建 tablet 的工作线程数
- 默认值：3

`check_consistency_worker_count`

- 描述：计算 tablet 的校验和 (checksum) 的工作线程数

- 默认值：1

max_tablet_version_num

- 类型：int
- 描述：限制单个 tablet 最大 version 的数量。用于防止导入过于频繁，或 compaction 不及时导致的大量 version 堆积问题。当超过限制后，导入任务将被拒绝。
- 默认值：2000

number_tablet_writer_threads

- 描述：tablet 写线程数
- 默认值：16

tablet_map_shard_size

- 描述：tablet_map_lock 分片大小，值为 2^n, n=0,1,2,3,4，这是为了更好地管理 tablet
- 默认值：4

tablet_meta_checkpoint_min_interval_secs

- 描述：TabletMeta Checkpoint 线程轮询的时间间隔
- 默认值：600 (s)

tablet_meta_checkpoint_min_new_rowsets_num

- 描述：TabletMeta Checkpoint 的最小 Rowset 数目
- 默认值：10

tablet_stat_cache_update_interval_second

- 描述：tablet 状态缓存的更新间隔
- 默认值：300 (s)

tablet_rowset_stale_sweep_time_sec

- 类型：int64
- 描述：用来表示清理合并版本的过期时间，当当前时间 now() 减去一个合并的版本路径中 rowset 最近创建创建时间大于 tablet_rowset_stale_sweep_time_sec 时，对当前路径进行清理，删除这些合并过的 rowset, 单位为 s。
- 当写入过于频繁，可能会引发 fe 查询不到已经合并过的版本，引发查询 -230 错误。可以通过调大该参数避免该问题。
- 默认值：300

tablet_writer_open_rpc_timeout_sec

- 描述：在远程 BE 中打开 tablet writer 的 rpc 超时。操作时间短，可设置短超时时间
- 导入过程中，发送一个 Batch（1024 行）的 RPC 超时时间。默认 60 秒。因为该 RPC 可能涉及多个分片内存块的写盘操作，所以可能会因为写盘导致 RPC 超时，可以适当调整这个超时时间来减少超时错误（如 send batch fail 错误）。同时，如果调大 write_buffer_size 配置，也需要适当调大这个参数
- 默认值：60

`tablet_writer_ignore_eovercrowded`

- 类型：bool
- 描述：写入时可忽略 brpc 的’[E1011]The server is overcrowded’ 错误。
- 当遇到’[E1011]The server is overcrowded’ 的错误时，可以调整配置项`brpc_socket_max_unwritten_bytes`，但这个配置项不能动态调整。所以可通过设置此项为`true`来临时避免写失败。注意，此配置项只影响写流程，其他的 rpc 请求依旧会检查是否 overcrowded。
- 默认值：false

`streaming_load_rpc_max_alive_time_sec`

- 描述：TabletsChannel 的存活时间。如果此时通道没有收到任何数据，通道将被删除。
- 默认值：1200

`alter_tablet_worker_count`

- 描述：进行 schema change 的线程数
- 默认值：3

4.6.3.4.8 `alter_index_worker_count`

- 描述：进行 index change 的线程数
- 默认值：3

`ignore_load_tablet_failure`

- 类型：bool
- 描述：用来决定在有 tablet 加载失败的情况下是否忽略错误，继续启动 be
- 默认值：false

BE 启动时，会对每个数据目录单独启动一个线程进行 tablet header 元信息的加载。默认配置下，如果某个数据目录有 tablet 加载失败，则启动进程会终止。同时会在 `be.INFO` 日志中看到如下错误信息：

```
load tablets from header failed, failed tablets size: xxx, path=xxx
```

表示该数据目录共有多少 tablet 加载失败。同时，日志中也会有加载失败的 tablet 的具体信息。此时需要人工介入来对错误原因进行排查。排查后，通常有两种方式进行恢复：

1. tablet 信息不可修复，在确保其他副本正常的情况下，可以通过 `meta_tool` 工具将错误的 tablet 删除。

2. 将 ignore_load_tablet_failure 设置为 true，则 BE 会忽略这些错误的 tablet，正常启动。

report_disk_state_interval_seconds

- 描述：代理向 FE 报告磁盘状态的间隔时间
- 默认值：60 (s)

result_buffer_cancelled_interval_time

- 描述：结果缓冲区取消时间
- 默认值：300 (s)

snapshot_expire_time_sec

- 描述：快照文件清理的间隔
- 默认值：172800 (48 小时)

compress_rowbatches

- 类型：bool
- 描述：序列化 RowBatch 时是否使用 Snappy 压缩算法进行数据压缩
- 默认值：true

4.6.3.4.9 日志

sys_log_dir

- 类型：string
- 描述：BE 日志数据的存储目录
- 默认值：${DORIS_HOME}/log

sys_log_level

- 描述：日志级别，INFO < WARNING < ERROR < FATAL
- 默认值：INFO

sys_log_roll_mode

- 描述：日志拆分的大小，每 1G 拆分一个日志文件
- 默认值：SIZE-MB-1024

sys_log_roll_num

- 描述：日志文件保留的数目

- 默认值：10

sys_log_verbose_level

- 描述：日志显示的级别，用于控制代码中 VLOG 开头的日志输出
- 默认值：10

sys_log_verbose_modules

- 描述：日志打印的模块，写 olap 就只打印 olap 模块下的日志
- 默认值：空

aws_log_level

- 类型：int32
- 描述：AWS SDK 的日志级别

```
Off = 0,
Fatal = 1,
Error = 2,
Warn = 3,
Info = 4,
Debug = 5,
Trace = 6
```

- 默认值：3

log_buffer_level

- 描述：日志刷盘的策略，默认保持在内存中
- 默认值：空

4.6.3.4.10 其他

report_tablet_interval_seconds

- 描述：代理向 FE 报告 olap 表的间隔时间
- 默认值：60 (s)

report_task_interval_seconds

- 描述：代理向 FE 报告任务签名的间隔时间
- 默认值：10 (s)

enable_metric_calculator

- 描述：如果设置为 true，metric calculator 将运行，收集 BE 相关指标信息，如果设置成 false 将不运行
- 默认值：true

`enable_system_metrics`

- 描述：用户控制打开和关闭系统指标
- 默认值：true

`enable_token_check`

- 描述：用于向前兼容，稍后将被删除
- 默认值：true

`max_runnings_transactions_per_txn_map`

- 描述：txn 管理器中每个 txn_partition_map 的最大 txns 数，这是一种自我保护，以避免在管理器中保存过多的 txns
- 默认值：2000

`max_download_speed_kbps`

- 描述：最大下载速度限制
- 默认值：50000（kb/s）

`download_low_speed_time`

- 描述：下载时间限制
- 默认值：300 (s)

`download_low_speed_limit_kbps`

- 描述：下载最低限速
- 默认值：50 (KB/s)

`enable_batch_download`

> 提示该功能自 Apache Doris 3.0.4 版本起支持

- 描述：是否允许批量下载文件，建议只在开启 binlog 的情况下打开
- 默认值：false

`doris_cgroups`

- 描述：分配给 doris 的 cgroups
- 默认值：空

`priority_queue_remaining_tasks_increased_frequency`

- 描述：BlockingPriorityQueue 中剩余任务的优先级频率增加
- 默认值:512

`jdbc_drivers_dir`

- 描述：存放 jdbc driver 的默认目录。
- 默认值：`${DORIS_HOME}/jdbc_drivers`

`enable_simdjson_reader`

- 描述：是否在导入 json 数据时用 simdjson 来解析。
- 默认值：true

`enable_query_memory_overcommit`

- 描述：如果为 true，则当内存未超过 exec_mem_limit 时，查询内存将不受限制；当进程内存超过 exec_mem_limit 且大于 2GB 时，查询会被取消。如果为 false，则在使用的内存超过 exec_mem_limit 时取消查询。
- 默认值：true

`user_files_secure_path`

- 描述：`local` 表函数查询的文件的存储目录。
- 默认值：`${DORIS_HOME}`

`brpc_streaming_client_batch_bytes`

- 描述：brpc streaming 客户端发送数据时的攒批大小（字节）
- 默认值：262144

`grace_shutdown_wait_seconds`

- 描述：在云原生的部署模式下，为了节省资源一个 BE 可能会被频繁的加入集群或者从集群中移除。如果在这个 BE 上有正在运行的 Query，那么这个 Query 会失败。用户可以使用 stop_be.sh –grace 的方式来关闭一个 BE 节点，此时 BE 会等待当前正在这个 BE 上运行的所有查询都结束才会退出。同时，在这个时间范围内 FE 也不会分发新的 query 到这个机器上。如果超过 grace_shutdown_wait_seconds 这个阈值，那么 BE 也会直接退出，防止一些查询长期不退出导致节点没法快速下掉的情况。
- 默认值：120

`enable_java_support`

- 描述：BE 是否开启使用 java-jni，开启后允许 c++ 与 java 之间的相互调用。目前已经支持 hudi、 java-udf、 jdbc、 max-compute、 paimon、 preload、 avro
- 默认值：true

`group_commit_wal_path`

- 描述：Group Commit 存放 WAL 文件的目录，请参考Group Commit
- 默认值：默认在用户配置的`storage_root_path`的各个目录下创建一个名为`wal`的目录。配置示例：

```
group_commit_wal_path=/data1/storage/wal;/data2/storage/wal;/data3/storage/wal
```

`default_tzfiles_path`

- 描述：Doris 自带的时区数据库。如果系统目录下未找到时区文件，则启用该目录下的数据。
- 默认值：“${DORIS_HOME}/zoneinfo”

4.6.3.4.11 存算分离模式

`deploy_mode`

- 默认值：“”
- 描述：BE 运行的模式。`cloud` 表示算分离模式。

`meta_service_endpoint`

- 默认值：“”
- 描述：Meta Service 的端点应以 ‘host1:port,host2:port’ 的格式指定。该值通常由 FE 通过心跳传递给 BE，无需配置。

`enable_file_cache`

- 默认值：在存算分离模式下为 true，在非存算分离模式下为 false。
- 描述：是否使用文件缓存。

`file_cache_path`

- 默认值：[{“path”：“${DORIS_HOME}/file_cache”}]
- 描述：用于文件缓存的磁盘路径和其他参数，以数组形式表示，每个磁盘一个条目。`path` 指定磁盘路径，`total_size` 限制缓存的大小；-1 或 0 将使用整个磁盘空间。
- 格式:[{“path”：“/path/to/file_cache”，“total_size”:21474836480},{“path”：“/path/to/file_cache2”，“total_size”:21474836480}]

`time_series_max_tablet_version_num`

- 类型：int
- 描述：限制 time series compaction 策略的 tablet 最大 version 的数量。用于防止导入过于频繁，或 compaction 不及时导致的大量 version 堆积问题。当超过限制后，导入任务将被拒绝。从 3.0.7 版本开始支持这个参数
- 默认值：20000

4.6.4 用户配置项

该文档主要介绍了 User 级别的相关配置项。User 级别的配置生效范围为单个用户。每个用户都可以设置自己的 User property。相互不影响。

4.6.4.1 查看配置项

FE 启动后，在 MySQL 客户端，通过下面命令查看 User 的配置项：

```
SHOW PROPERTY [FOR user] [LIKE key pattern]
```

具体语法可通过命令：`help show property;` 查询。

4.6.4.2 设置配置项

FE 启动后，在 MySQL 客户端，通过下面命令修改 User 的配置项：

```
SET PROPERTY [FOR 'user'] 'key' = 'value' [, 'key' = 'value']
```

具体语法可通过命令：`help set property;` 查询。

User 级别的配置项只会对指定用户生效，并不会影响其他用户的配置。

4.6.4.3 应用举例

1. 修改用户 Billie 的 `max_user_connections`

   通过 `SHOW PROPERTY FOR 'Billie' LIKE '%max_user_connections%';` 查看 Billie 用户当前的最大链接数为 100。

   通过 `SET PROPERTY FOR 'Billie' 'max_user_connections' = '200';` 修改 Billie 用户的当前最大连接数到 200。

4.6.4.4 配置项列表

4.6.4.4.1 max_user_connections

用户最大的连接数，默认值为100。一般情况不需要更改该参数，除非查询的并发数超过了默认值。

4.6.4.4.2 max_query_instances

用户同一时间点可使用的instance个数，默认是-1，小于等于0将会使用配置default_max_query_instances.

4.6.4.4.3 resource

4.6.4.4.4 quota

4.6.4.4.5 default_load_cluster

4.6.4.4.6 load_cluster

4.7 系统表

4.7.1 概述

Doris 集群内置多个系统数据库，用于存储 Doris 系统本身的一些元数据信息。

4.7.1.1 information_schema

`information_schema` 库下的所有表都是虚拟表，本身并不存在物理实体。这些系统表包含了关于 Doris 集群及其所有数据库对象的元数据。这些对象包括数据库、表、列、权限等。也包含如 Workload Group、 Task 等功能状态信息。

每个 Catalog 下都存在一个 `information_schema` 库。其中只包含对应 Catalog 下的库表的元数据。

`information_schema` 库中得所有表都是只读状态，用户无法在这个库中修改、删除或创建表。

所有用户默认对这个库下的所有表拥有读权限，但可查询的内容会根据用户实际的权限范围而不同。比如用户 A 只拥有 `db1.table1` 的权限。则用户查询 `information_schema.tables` 表时，只会返回 `db1.table1` 相关的信息。

4.7.1.2 mysql

`mysql` 库下的所有表都是虚拟表，本身并不存在物理实体。这些系统表包含如权限等信息，主要用于兼容 MySQL 生态。

每个 Catalog 下都存在一个 `mysql` 库。但其中的表的内容完全一样。

`mysql` 库中得所有表都是只读状态，用户无法在这个库中修改、删除或创建表。

4.7.1.3 __internal_schema

`__internal_schema` 库下的所有表都是 Doris 的实体表，其存储方式和用户创建的数据表无异。Doris 会在集群创建时，自动创建这个库下的所有系统表。

默认情况下，普通用户对这个库下的表拥有只读权限。但被授权后，可以对这个库下的表进行修改、删除或创建。

4.7.2 information_schema

4.7.3 mysql

4.7.3.1 user

4.7.3.1.1 概述

查看所有用户信息

4.7.3.1.2 所属数据库

`mysql`

4.7.3.1.3 表信息

列名	类型	说明
host	character(255)	用户允许连接的 Host
user	char(32)	用户名
node_priv	char(1)	是否有 Node 权限
admin_priv	char(1)	是否有 Admin 权限
grant_priv	char(1)	是否有 Grant 权限
select_priv	char(1)	是否有 Select 权限
load_priv	char(1)	是否有用 Load 权限
alter_priv	char(1)	是否有 Alter 权限
create_priv	char(1)	是否有 Create 权限
drop_priv	char(1)	是否有 Drop 权限
usage_priv	char(1)	是否有 Usage 权限
show_view_priv	char(1)	是否有 Show View 权限
cluster_usage_priv	char(1)	是否有 Cluster 使用权限
stage_usage_priv	char(1)	是否有 Stage 使用权限
ssl_type	char(9)	永远为空，仅用于兼容 MySQL
ssl_cipher	varchar(65533)	永远为空，仅用于兼容 MySQL
x509_issuer	varchar(65533)	永远为空，仅用于兼容 MySQL
x509_subject	varchar(65533)	永远为空，仅用于兼容 MySQL
max_questions	bigint	永远为 0，仅用于兼容 MySQL
max_updates	bigint	永远为 0，仅用于兼容 MySQL
max_connections	bigint	永远为 0，仅用于兼容 MySQL
max_user_connections	bigint	允许的最大连接数量
plugin	char(64)	永远为空，仅用于兼容 MySQL
authentication_string	varchar(65533)	永远为空，仅用于兼容 MySQL
password_policy.expiration_seconds	varchar(32)	密码过期时间
password_policy.password_creation_time	varchar(32)	密码创建时间
password_policy.history_num	varchar(32)	历史密码数量
password_policy.history_passwords	varchar(65533)	历史密码
password_policy.num_failed_login	varchar(32)	允许连续登录失败次数
password_policy.password_lock_seconds	varchar(32)	触发锁定后的密码锁定时间
password_policy.failed_login_counter	varchar(32)	登录失败计数
password_policy.lock_time	varchar(32)	当前已锁定时间

4.7.4 __internal_schema

4.8 故障诊断处理

4.8.1 内存管理

4.8.1.1 概述

内存管理是 Doris 中最重要的组成部分之一，在 Doris 运行过程中，不论导入还是查询都依赖大量的内存操作。内存管理的好坏直接影响到 Doris 的稳定性和性能。

Apache Doris 作为基于 MPP 架构的 OLAP 数据库，数据从磁盘加载到内存后，会在算子间流式传递并计算，在内存中存储计算的中间结果，这种方式减少了频繁的磁盘 I/O 操作，充分利用多机多核的并行计算能力，可在性能上呈现巨大优势。

在面临内存资源消耗巨大的复杂计算和大规模作业时，有效的内存分配、统计、管控对于系统的稳定性起着十分关键的作用 ——更快的内存分配速度将有效提升查询性能，通过对内存的分配、跟踪与限制可以保证不存在内存热点，及时准确地响应内存不足并尽可能规避 OOM 和查询失败，这一系列机制都将显著提高系统稳定性；同时更精确的内存统计，也是大查询落盘的基础。

4.8.1.1.1 Doris BE 内存结构

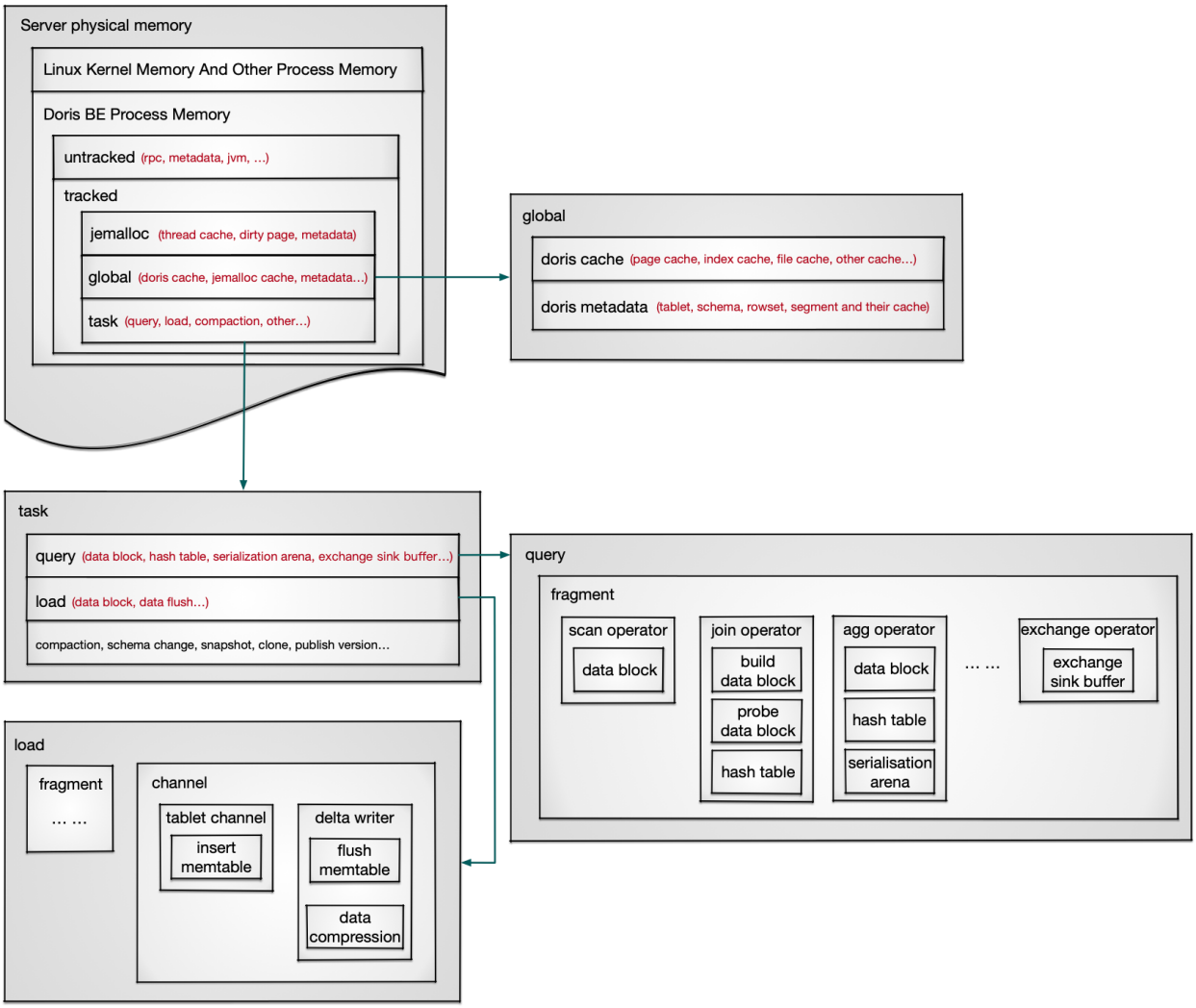


图 146: Memory Structure

```
Server physical memory: 供服务器上所有进程使用的的物理内存，`cat /proc/meminfo` 或 `free -h`
    ↪ 看到的 MemTotal。
    |
    |---> Linux Kernel Memory And Other Process Memory: Linux 内核和其他进程使用的内存。
    |
    |---> Doris BE Process Memory: Doris BE 进程使用的内存，上限是服务器物理内存减去 Linux
        ↪ 内核和其他进程使用的内存，或者 Cgroup 配置的内存大小。
            |
            |---> untracked: 没有被跟踪和管理的内存，包括 rpc，jvm，部分 metadata 等。
                ↪ 在访问外表或使用 java udf 时会用到 jvm。
            |
            |---> tracked: 被跟踪和管理的内存，允许实时查看，自动内存回收，通过参数控制大小。
                    |
                    |---> jemalloc: jemalloc 管理的 cache 和 metadata，支持参数控制，
```

```
    ↪ 内存不足时自动回收。
|
|---> global: Doris 全局共享的内存，主要包括 cache 和 metadata。
|       |
|       |---> doris cache: doris 自己管理的 cache，
    ↪ 支持单独通过参数控制容量和淘汰时长，内存不足时自动回收。
|       |
|       |---> doris metadata: BE 上存储数据的 metadata，包括数据 schema
    ↪ 等一系列内存数据结构和它们的缓存。
|
|---> task: Doris 上执行的任务使用的内存，预期在任务结束后释放，包括 query，
    ↪ load，compaction 等。
|       |
|       |---> query: 查询期间使用的内存。一个查询被拆分成多个 fragment
    ↪ 单独执行，通过数据 shuffle 相连。
|       |       |
|       |       |---> fragment: 一个 fragment 被拆分成多个 operator 以
    ↪ pipeline 的形式执行。
|       |       |       |
|       |       |       |---> operator: 包括 data block, hash table, arena,
    ↪ exchange sink buffer 等内存数据结构。
|       |
|       |---> load: 数据导入期间使用的内存。数据导入包括 fragment 读取和
    ↪ channel 写入数据两个阶段。
|       |       |
|       |       |---> fragment: 和查询的 fragment 执行相同，stream load
    ↪ 通常只有 scan operator。
|       |       |
|       |       |---> channel: tablet channel 将数据写入临时的数据结构
    ↪ memtable，然后 delta writer 将数据压缩后写入文件。
```

4.8.1.1.2 内存查看

Doris BE 使用内存跟踪器（Memory Tracker）记录进程内存使用，支持 Web 页面查看，并在内存相关报错时打印到 BE 日志中，用于内存分析和排查内存问题。

实时内存统计

实时的内存统计结果通过 Doris BE 的 Web 页面查看 `http://{be_host}:{be_web_server_port}/mem_tracker`，展示 `type=overview` 的 Memory Tracker 当前跟踪的内存大小和峰值内存大小，包括 Query/Load/Compaction/Global 等，`be_web_server_port` 默认 8040。

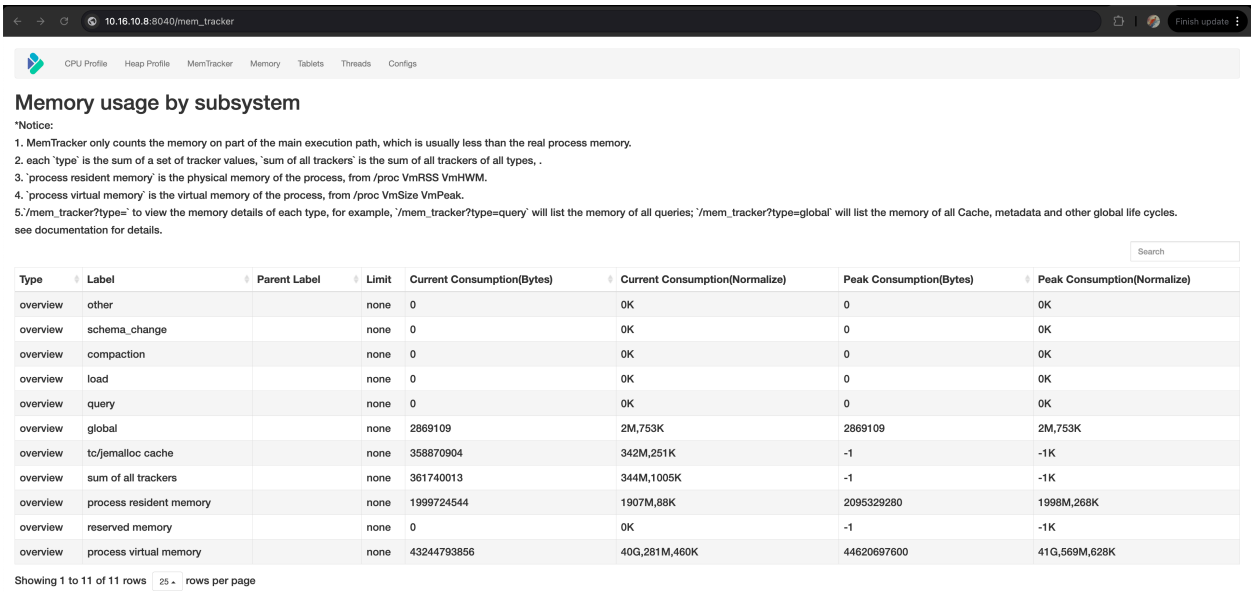


图 147: image

Memory Tracker 分为不同的类型，其中 `type=overview` 的 Memory Tracker 中除 `process resident memory`、`process virtual memory`、`sum of all trackers` 外，其他 `type=overview` 的 Memory Tracker 都可以通过 `http://{be_host}:{be_web_server_port}/mem_tracker?type=Label` 查看详情。

Memory Tracker 拥有如下的属性：

1. Label: Memory Tracker 的名称
2. Current Consumption(Bytes): 当前内存值，单位 B。
3. Current Consumption(Normalize): 当前内存值的.G.M.K 格式化输出。
4. Peak Consumption(Bytes): BE 进程启动后的内存峰值，单位 B，BE 重启后重置。
5. Peak Consumption(Normalize): BE 进程启动后内存峰值的.G.M.K 格式化输出，BE 重启后重置。
6. Parent Label: 用于表明两个 Memory Tracker 的父子关系，Child Tracker 记录的内存是 Parent Tracker 的子集，Parent 相同的不同 Tracker 记录的内存可能存在交集。

有关 Memory Tracker 的更多介绍参考内存跟踪器。

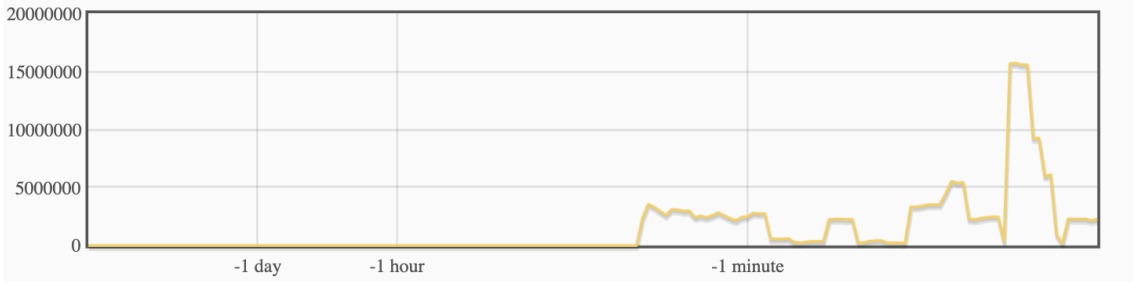
历史内存统计

历史的内存统计结果通过 Doris BE 的 Bvar 页面查看 `http://{be_host}:{brpc_port}/vars/*memory_*`，用实时内存统计页面 `http://{be_host}:{be_web_server_port}/mem_tracker` 中 Memory Tracker 的 Label 搜索 Bvar 页面，即可得到对应 Memory Tracker 跟踪的内存大小变化趋势，`brpc_port` 默认 8060。

status | vars | connections | flags | rpcz | cpu | heap | growth | co

Search :

doris_column_reader_memory_bytes : 0
doris_index_reader_memory_bytes : 0
doris_ordinal_index_memory_bytes : 0
doris_primary_key_index_memory_bytes : 0
doris_short_key_index_memory_bytes : 0
doris_zone_map_memory_bytes : 0
meminfo_process_memory_usage : 2102392648
memory_all_trackers_sum_bytes : 217250632
memory_arbitrator_process_usage_bytes : 2102392648
memory_arbitrator_refresh_interval_growth_bytes : 0
memory_arbitrator_reserve_memory_bytes : 0
memory_arbitrator_sys_available_bytes : 135568199680
memory_cgroup_usage_bytes : 8883695616
memory_compaction_trackers_sum_bytes : 0
memory_global_trackers_sum_bytes : 0
memory_jemalloc_cache_bytes : 113617080
memory_jemalloc_dirty_pages_bytes : 2277376



memory_jemalloc_metadata_bytes : 118172416
memory_jemalloc_virtual_bytes : 2086457344
memory_load_trackers_sum_bytes : 0
memory_memtrackerlimiter_cnt : 14
memory_other_trackers_sum_bytes : 0
memory_query_trackers_sum_bytes : 0
memory_schema_change_trackers_sum_bytes : 0
memory_sys_available_bytes : 135568199680
normal_memory_used : 0
process_memory_data_and_stack : 23652200448
process_memory_resident : 2216009728
process_memory_shared : 74240000
process_memory_text : 487174144
process_memory_virtual : 25661272064

图 148: Bvar Memory

当报错进程内存超限或可用内存不足时，在 `be/log/be.INFO` 日志中可以找到 `Memory Tracker Summary`，包含所有 `Type=overview` 和 `Type=global` 的 Memory Tracker，帮助使用者分析当时的内存状态，具体参考内存日志分析

4.8.1.1.3 内存分析

将 type=overview 的 Memory Tracker 对应到上述内存结构中 tracked 下的每一部分内存：

```
Doris BE Process Memory
    |
    |---> tracked: 对应 `MemTrackerLimiter Label=sum of all trackers, Type=overview`，是 Memory
        ↪ Tracker 统计到的所有内存，即除 `Label=process resident memory` 和 `Label=process
        ↪ virtual memory` 外，其他 `type=overview` 的 Memory Tracker 的 Current Consumption
        ↪ 总和。
            |
            |---> jemalloc
            |       |
            |       |---> jemalloc cache: 对应 `MemTrackerLimiter Label=tc/jemalloc_cache, Type=
                ↪ overview`，Jemalloc 缓存包括 Dirty Page、Thread Cache 两部分。
            |       |
            |       |---> jemalloc metadata: 对应 `MemTrackerLimiter Label=tc/jemalloc_metadata,
                ↪ Type=overview`，Jemalloc 的 Metadata。
            |
            |---> global: 对应 `MemTrackerLimiter Label=global, Type=overview`，包括 Cache、
                ↪ 元数据、解压缩 等生命周期和进程相同的全局 Memory Tracker，Web 页面 `http://{
                ↪ be_host}:{be_web_server_port}/mem_tracker?type=global` 展示 `type=global`
                ↪ 的所有 Memory Tracker。
            |
            |---> task
            |       |
            |       |---> query: 对应 `MemTrackerLimiter Label=query, Type=overview`，即所有
                ↪ Query Memory Tracker 的 Current Consumption 总和，Web 页面 `http://{be_host
                ↪ }:{be_web_server_port}/mem_tracker?type=query` 展示 `type=query` 的所有
                ↪ Memory Tracker。
            |       |
            |       |---> load: 对应 `MemTrackerLimiter Label=load, Type=overview`，所有 Load
                ↪ Memory Tracker 的 Current Consumption 总和，Web 页面 `http://{be_host}:{be_
                ↪ web_server_port}/mem_tracker?type=load` 展示 `type=load` 的所有 Memory
                ↪ Tracker。
            |       |
            |       |---> reserved: 对应 `MemTrackerLimiter Label=reserved_memory, Type=overview
                ↪ `，被预留的内存，查询在构建 Hash Table 等需要大内存的行为之前，会先从 Memory
                ↪ Tracker 中预留出所构建 Hash Table 大小的内存，确保后续内存申请能够满足。
            |       |
            |       |---> compaction: 对应 `MemTrackerLimiter Label=compaction, Type=overview`，
                ↪ 所有 Compaction Memory Tracker 的 Current Consumption 总和，Web 页面 `http
                ↪ ://{be_host}:{be_web_server_port}/mem_tracker?type=compaction` 展示 `type=
                ↪ compaction` 的所有 Memory Tracker。
            |       |
            |       |---> schema_change: 对应 `MemTrackerLimiter Label=schema_change, Type=
```

```
            ↪ overview`，所有 Schema Change Memory Tracker 的 Current Consumption 总和，Web
            ↪  页面 `http://{be_host}:{be_web_server_port}/mem_tracker?type=schema_change`
            ↪ 展示 `type=schema_change` 的所有 Memory Tracker。

        |       |
        |       |---> other: 对应 `MemTrackerLimiter Label=other, Type=overview`，
            ↪ 除上述之外其他任务的内存总和，例如 EngineAlterTabletTask、EngineCloneTask、
            ↪ CloudEngineCalcDeleteBitmapTask、SnapshotManager 等，Web 页面 `http://{be_
            ↪ host}:{be_web_server_port}/mem_tracker?type=other` 展示 `type=other` 的所有
            ↪ Memory Tracker。
|
|---> Doris BE 进程物理内存，对应 `MemTrackerLimiter Label=process resident memory, Type=
    ↪ overview`，Current Consumption 取自 VmRSS in `/proc/self/status`，Peak Consumption
    ↪ 取自 VmHWM in `/proc/self/status`。
|
|---> Doris BE 进程虚拟内存，对应 `MemTrackerLimiter Label=process virtual memory, Type=
    ↪ overview`，Current Consumption 取自 VmSize in `/proc/self/status`，Peak Consumption
    ↪ 取自 VmPeak in `/proc/self/status`。
```

上述内存结构中每一部分内存的分析方法：

1. Jemalloc 内存分析
2. 全局内存分析
3. Query 内存分析
4. Load 内存分析

4.8.1.1.4 内存问题 FAQ

参考内存问题 FAQ 分析常见的内存问题。

4.8.1.1.5 内存控制策略

参考内存控制策略中对内存分配、监控、回收的介绍，它们保证了 Doris BE 进程内存的高效可控。

4.8.1.2 内存问题 FAQ

Doris BE 进程内存分析主要使用 be/log/be.INFO 日志、BE 进程内存监控 (Metrics)、Doris Bvar 统计，如果触发了 OOM Killer 需要收集 dmesg -T 执行结果，如果分析查询或导入任务的内存需要收集 Query Profile，依据这些信息分析常见的内存问题。如果你自行分析无法解决问题，需要向 Doris 开发者们求助，无论使用何种途径 (Github 提交 issue，Doris 论坛创建问题，邮件或 WeChat)，都请将上述信息添加到你的问题描述中。

首先定位当前观察到的现象属于哪种内存问题，并进一步排查，通常需要先分析进程内存日志，参考内存日志分析，下面列举常见的内存问题。

4.8.1.2.1 1 查询和导入内存超限错误

当查询和导入的报错信息中出现 MEM_LIMIT_EXCEEDED 时，说明任务因为进程可用内存不足，或任务超过单次执行的内存上限而被 Cancel。

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.8)[MEM_LIMIT_EXCEEDED] xxxx .
```

若报错信息包含 Process memory not enough，说明进程可用内存不足，参考查询或导入报错进程可用内存不足进一步分析。

若报错信息中出现 memory tracker limit exceeded 时，说明任务超过单次执行内存限制，参考查询或导入报错超过单次执行内存限制进一步分析。

4.8.1.2.2 2 Doris BE OOM Crash

如果 BE 进程 Crash 后 log/be.out 中没有报错信息，执行 dmesg -T 如果看到 Out of memory: Killed process {pid} (doris_be)，说明触发了 OOM Killer，参考 OOM Killer Crash 分析进一步分析。

4.8.1.2.3 3 内存泄漏

> 如果遇到疑似内存泄漏的现象，最好的解决办法是升级到最新的三位数版本，如果你在用 Doris 2.0，就升级到 Doris 2.0.x 的最新版本，因为大概率其他人也遇到过同样的现象，大部分内存泄漏问题都在版本迭代中被修复。

如果观察到下面的现象，说明可能存在内存泄漏：

- Doris Grafana 或服务器监控发现 Doris BE 进程内存一直线性增长，且集群上任务停止后，内存也不下降。
- Memory Tracker 存在统计缺失，参考内存跟踪器中Memory Tracker 统计缺失 章节分析。

内存泄漏通常都伴随着 Memory Tracker 统计缺失，所以分析方法同样参考Memory Tracker 统计缺失 章节。

4.8.1.2.4 4 Doris BE 进程内存不下降 OR 持续上涨

如果 Doris Grafana 或服务器监控发现 Doris BE 进程内存一直线性增长，且集群上任务停止后，内存也不下降，首先参考内存跟踪器中Memory Tracker 统计缺失 章节分析是否存在 Memory Tracker 统计缺失，若 Memory Tracker 存在统计缺失则进一步分析原因。

若 Memory Tracker 不存在统计缺失，统计到了大部分内存，参考 Overview 分析 Doris BE 进程不同部分内存占用过大的原因以及减少其内存使用的方法。

4.8.1.2.5 5 虚拟内存占用大

`Label=process virtual memory` Memory Tracker 显示实时的虚拟内存大小，和 `top -p {pid}` 看到的 Doris BE 进程虚拟内存相同。

```
MemTrackerLimiter Label=process virtual memory, Type=overview, Limit=-1.00 B(-1 B), Used=44.25 GB
    ↪ (47512956928 B), Peak=44.25 GB(47512956928 B)
```

Doris 目前仍存在 Doris BE 进程虚拟内存过大的问题，通常是因为 Jemalloc 保留了大量虚拟内存映射，这也将导致 Jemalloc Metadata 占用过多的内存，参考 Jemalloc 内存分析中对 Jemalloc Metadata 内存的分析。

除此之外已知 Doris 的 Join Operator 和 Column 中缺少内存复用，这会导致在某些场景申请更多的虚拟内存，并最终缓存到 Jemalloc Retained 中，目前没有很好的解决办法，建议定期重启 Doris BE 进程。

4.8.1.2.6 6 BE 进程刚启动后进程内存就很大

这通常是因为 BE 进程启动时加载的元数据内存过大，参考 Metadata 内存分析查看 Doris BE Bvar。

- 如果 `doris_total_tablet_num` 过多，通常是因为表的分区和分桶数量过多，查看 `{fe_host}:{fe_http_port}/System?path=//dbs` 找到 Tablet 数量多的表，一个表的 Tablet 数量等于其分区数量乘以分桶数量，尝试降低其分区和分桶数量。或者删除过时不会被使用的表或分区。
- 如果 `doris_total_rowset_num` 过多，但 Tablet 数量不多，参考 `SHOW-PROC` 文档找到 Rowset 多但 Tablet 不多的表，然后手动触发 Compaction，或者等自动 Compaction 完成，具体参考元数据管理相关文档，通常存在几十万个 Rowset 时，元数据占用几个 GB 是正常现象。
- 如果 `tablet_meta_schema_columns_count` 过大，是 `doris_total_tablet_schema_num` 的成百上千倍，说明集群中存在几百上千列的大宽表，此时相同数量的 Tablet 会占用更多的内存。

4.8.1.2.7 7 Query 没有复杂算子只是简单的 Scan 数据，却要使用很大的内存

可能是读取 Segment 时打开的 Column Reader、 Index Read 占用的内存，参考 Metadata 内存分析查看 Doris BE Bvar 中的 `doris_total_segment_num`、`doris_column_reader_num`、`doris_ordinal_index_memory_bytes`、`doris_zone_map_memory_bytes`、`doris_short_key_index_memory_bytes`的变化，这个现象同样常见于读取大宽表，当打开几十万个 Column Reader 时，内存可能会占用几十 GB。

如果你在 Heap Profile 内存占比大的调用栈中看到 `Segment`，`ColumnReader` 字段，则基本可以确认是读取 Segment 时占用了大量内存。

此时只能通过修改 SQL 降低扫描的数据量，或者降低建表时指定的分桶大小，从而打开更少的 Segment。

4.8.1.2.8 8. Query Cancel 过程中卡住

> 常见于 Doris 2.1.3 之前

Query 执行期间申请的大部分内存需要等到查询结束时释放，在进程内存充足时通常无需关注查询结束阶段的快或慢，但在进程内存不足时经常会按照一定策略 Cancel 部分 Query，以释放它们的内存，避免进程触发 OOM Killer。此时如果 Query Cancel 过程中卡住，无法及时释放内存，除了会增大触发 OOM Killer 的风险，也可能会导致更多的 Query 因进程内存不足而被 Cancel。

若已知一个 Query 被 Cancel，下面依据这个 QueryID 分析其再 Cancel 过程中是否卡住。首先执行 `grep {queryID} be/log/be.INFO`，找到第一次包含 `Cancel` 关键词的日志，对应时间点就是 Query 被 Cancel 的时间。找到包含 `deregister query/load memory tracker` 关键词的日志，对应的时间点就是 Query Cancel 完成的时间，若最终触发了 OOM Killer，无法找到包含 `deregister query/load memory tracker` 关键词的日志，说明直到 OOM Killer 发生时这个 Query 仍没有 Cancel 完成，通常若 Query Cancel 过程的好时大于 3s，就任务这个 Query 在 Cancel 过程中卡住，需要进一步分析 Query 执行日志。

此外，在执行 `grep {queryID} be/log/be.INFO` 后，若看到包含 `tasks is being canceled and has not been completed yet` 关键词的日志，其后面的 QueryID 列表是表示 Memory GC 时发现这些 Query 正在被 Cancel 但没有 Cancel 完成，此时将跳过这些 Query，继续释放别处的内存，可据此判断 Memory GC 的行为是否符合预期。

4.8.1.3 内存分析

4.8.1.3.1 Jemalloc 内存分析

Doris 默认使用 Jemalloc 作为通用内存分配器，Jemalloc 自身占用的内存包括 Cache 和 Metadata 两部分，其中 Cache 包括 Thread Cache 和 Dirty Page 两部分，在 http://{be_host}:{be_web_server_port}/memz 可以实时查看到内存分配器原始的 profile。

Jemalloc Cache 内存分析

如果看到 `Label=tc/jemalloc_cache, Type=overview` Memory Trakcer 的值较大，说明 Jemalloc 或 TCMalloc Cache 内存使用多，Doris 使用 Jemalloc 作为默认的 Allocator，所以这里只分析 Jemalloc Cache 内存使用多的情况。

```
MemTrackerLimiter Label=tc/jemalloc_cache, Type=overview, Limit=-1.00 B(-1 B), Used=410.44 MB
    ↪ (430376896 B), Peak=-1.00 B(-1 B)
```

> Doris 2.1.6 之前 `Label=tc/jemalloc_cache` 还包括 Jemalloc Metadata，而且大概率是因为 Jemalloc Metadata 内存占用大导致 `Label=tc/jemalloc_cache` 过大，参考对 `Label=tc/jemalloc_metadata` Memory Tracker 的分析。

BE 进程运行过程中，Jemalloc Cache 包括两部分。

- Thread Cache，在 Thread Cache 中缓存指定数量的 Page，参考 Jemalloc opt.tcache。
- Dirty Page，所有 Arena 中可以被复用的内存 Page。

Jemalloc Cache 查看方法

查看 Doris BE 的 Web 页面 `http://{be_host}:{be_web_server_port}/memz`（webserver_port 默认 8040）可以获得 Jemalloc Profile，根据几组关键信息解读 Jemalloc Cache 的使用。

- Jemalloc Profile 中的 `tcache_bytes`是 Jemalloc Thread Cache 的总字节数。如果 `tcache_bytes` 值较大，说明 Jemalloc Thread Cache 使用的内存过大。
- Jemalloc Profile 中 `extents` 表中 `dirty` 列的值总和较大，说明 Jemalloc Dirty Page 使用的内存过大。

Thread Cache 内存过大

可能是 Thread Cache 缓存了大量大 Page，因为 Thread Cache 的上限是 Page 个数，而不是 Page 的总字节数。

考虑减小 `be.conf` 中 `JEMALLOC_CONF` 的 `lg_tcache_max`，`lg_tcache_max` 是允许缓存的 Page 字节大小上限，默认是 15，即 32 KB (2^15)，超过这个大小的 Page 将不会缓存到 Thread Cache 中。`lg_tcache_max` 对应 Jemalloc Profile 中的 `Maximum thread-cached size class`。

> Doris 2.1 之前 `be.conf` 中 `JEMALLOC_CONF` 的 `lg_tcache_max` 默认是 20，在某些场景会导致 Jemalloc Cache 过大，Doris 2.1 之后已经改回了 Jemalloc 的默认值 15。

这通常是 BE 进程中的查询或导入正在申请大量大 Size Class 的内存 Page，或者执行完一个大内存查询或导入后，Thread Cache 中缓存了大量大 Size Class 的内存 Page。Thread Cache 有两个清理时机，一是内存申请和释放到达一定次数时，回收长时间未使用的内存块；二是线程退出时回收全部 Page。此时存在一个 Bad Case，若线程后续一直没有执行新的查询或导入，从此不再分配内存，陷入一种所谓的 `idle` 状态。用户预期是查询结束后，内存是可以释放掉的，但实际上此场景下若线程没有退出，Thread Cache 并不会清理。

不过通常无需关注 Thread Cache，在进程可用内存不足时，若 Thread Cache 的大小超过 1G，Doris 将手动 Flush Thread Cache。

Dirty Page 内存过大

```
extents:        size ind       ndirty        dirty       nmuzzy        muzzy    nretained
    ↪ retained       ntotal        total
                4096   0            7        28672            1         4096           21
                    ↪ 86016           29       118784
                8192   1           11        90112            2        16384           11
                    ↪ 90112           24       196608
               12288   2            2        24576            4        49152           45
                    ↪ 552960          51       626688
               16384   3            0            0            1        16384            6
                    ↪ 98304            7       114688
               20480   4            0            0            1        20480            5
                    ↪ 102400           6       122880
               24576   5            0            0           43      1056768            2
                    ↪ 49152           45      1105920
               28672   6            0            0            0            0           13
                    ↪ 372736          13       372736
               32768   7            0            0            1        32768           13
                    ↪ 425984          14       458752
```

```
40960    8              0              0              31         1150976            35
    ↪ 1302528              66         2453504
49152    9              4          196608             2          98304              3
    ↪ 139264               9          434176
57344    10             0              0              1          57344              9
    ↪ 512000               10         569344
65536    11             3          184320             0              0              6
    ↪ 385024               9          569344
81920    12             2          147456             3          241664             38
    ↪ 2809856              43         3198976
98304    13             0              0              1          86016              6
    ↪ 557056               7          643072
114688   14             1          102400             1          106496             15
    ↪ 1642496              17         185139
```

减小 be.conf 中 JEMALLOC_CONF 的 dirty_decay_ms 到 2000 ms 或更小，be.conf 中默认 dirty_decay_ms 为 5000 ms。Jemalloc 会在 dirty_decay_ms 指定的时间内依照平滑梯度曲线释放 Dirty Page，参考 Jemalloc opt.dirty_decay_ms，当 BE 进程可用内存不足触发 Minor GC 或 Full GC 时会按照一定策略主动释放所有 Dirty Page。

> Doris 2.1 之前 be.conf 中 JEMALLOC_CONF 的 dirty_decay_ms 默认是 15000，在某些场景会导致 Jemalloc Cache 过大，Doris 2.1 之后默认值是 5000。

Jemalloc Profile 中的 extents 包含 Jemalloc 所有 arena 中不同 Page Size 的 Bucket 的统计值，其中 ndirty 是 Dirty Page 的个数，dirty 是 Dirty Page 的内存总和。参考 Jemalloc 中的 stats.arenas.<i>.extents.<j>.{extent_type}_bytes 将所有 Page Size 的 dirty 相加得到 Jemalloc 中 Dirty Page 的内存字节大小。

Jemalloc Metadata 内存分析

若 Label=tc/jemalloc_metadata，Type=overview Memory Trakcer 的值较大，说明 Jemalloc 或 TCMalloc Metadata 内存使用多，Doris 使用 Jemalloc 作为默认的 Allocator，所以这里只分析 Jemalloc Metadata 内存使用多的情况。

```
MemTrackerLimiter Label=tc/jemalloc_metadata, Type=overview, Limit=-1.00 B(-1 B), Used=144 MB
    ↪ (151759440 B), Peak=-1.00 B(-1 B)
```

> Label=tc/jemalloc_metadata Memory Tracker 在 Doris 2.1.6 之后才被添加，过去 Jemalloc Metadata 被包含在 Label=tc/jemalloc_cache Memory Tracker 中。

Jemalloc Metadata 查看方法

查看 Doris BE 的 Web 页面 http://{be_host}:{be_web_server_port}/memz（webserver_port 默认 8040）可以获得 Jemalloc Profile，查找 Jemalloc Profile 中关于 Jemalloc 整体的内存统计如下，其中 metadata 就是 Jemalloc Metadata 的内存大小。

```
Allocated: 2401232080, active: 2526302208, metadata: 535979296 (n_thp 221), resident: 2995621888,
↪  mapped: 3221979136, retained: 131542581248
```

- `Allocated` Jemalloc 为 BE 进程分配的内存总字节数。
- `active` Jemalloc 为 BE 进程分配的所有 Page 总字节数，是 Page Size 的倍数，通常大于等于 `Allocated`。
- `metadata` Jemalloc 的元数据总字节数，和分配和缓存的 Page 个数、内存碎片等因素都有关，参考文档 Jemalloc stats.metadata
- `retained` Jemalloc 保留的虚拟内存映射大小，也没有通过 munmap 或类似方法返回给操作系统，也没有强关联物理内存。参考文档 Jemalloc stats.retained

Jemalloc Metadata 内存过大

Jemalloc Metadata 大小和进程虚拟内存大小正相关，通常 Doris BE 进程虚拟内存大是因为 Jemalloc 保留了大量虚拟内存映射，即上面的 `retained`。返回给 Jemalloc 的虚拟内存默认都会缓存在 Retained 中，等待被复用，不会自动释放，也无法手动释放。

造成 Jemalloc Retained 大的根本原因是 Doris 代码层面内存复用不足，导致需要申请大量虚拟内存，这些虚拟内存释放后进入 Jemalloc Retained。通常虚拟内存和 Jemalloc Metadata 大小的比值在 300-500 之间，即若有 10T 的虚拟内存，Jemalloc Metadata 可能占用 20G。

如果遇到 Jemalloc Metadata 和 Retained 持续增大，以及进程虚拟内存过大的问题，建议考虑定时重启 Doris BE 进程，通常这只会在 Doris BE 长时间运行后出现，而且只有少数 Doris 集群会遇到。目前没有不损失性能的方法降低 Jemalloc Retained 保留的虚拟内存映射，Doris 正在持续优化内存使用。

如果频繁出现上述问题，参考下面的方法。

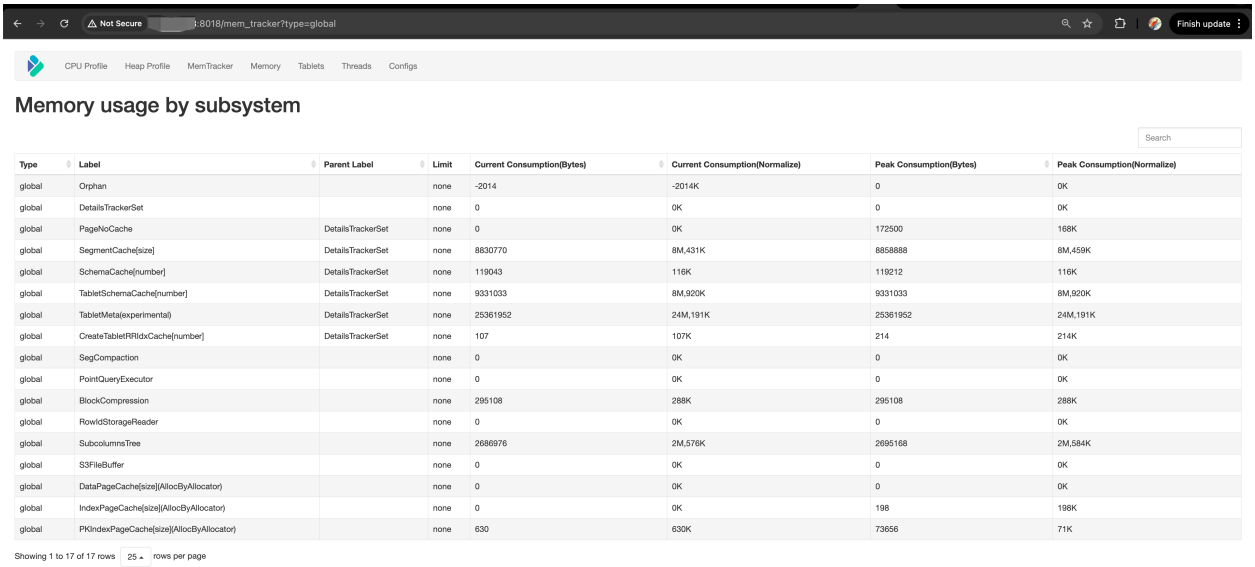
1. 一个根本解决方法是关闭 Jemalloc Retained 缓存虚拟内存映射，在 `be.conf` 中 `JEMALLOC_CONF` 后面增加 `retain:false` 后重启 BE。但查询性能可能会明显降低，测试 TPC-H Benchmark 性能会降低 3 倍左右。
2. Doris 2.1 上可以关闭 Pipelinex 和 Pipeline，执行 `set global experimental_enable_pipeline_engine=false ↪ ; set global experimental_enable_pipeline_x_engine=false;`，因为 pipelinex 和 pipeline 会申请更多的虚拟内存。这同样会导致查询性能降低。

4.8.1.3.2 全局内存分析

GLobal Memory 是 Doris 全局共享的内存，主要包括 Cache 和 Metadata。

Global Memory 查看方法

Web 页面 `http://{be_host}:{be_web_server_port}/mem_tracker?type=global` 展示 `type=global` 的所有 Memory Tracker。



Type	Label	Parent Label	Limit	Current Consumption(Bytes)	Current Consumption(Normalize)	Peak Consumption(Bytes)	Peak Consumption(Normalize)
global	Orphan		none	-2014	-2014K	0	0K
global	DetailsTrackerSet		none	0	0K	0	0K
global	PageNoCache	DetailsTrackerSet	none	0	0K	172500	168K
global	SegmentCache[size]	DetailsTrackerSet	none	8830770	8M,431K	8858888	8M,459K
global	SchemaCache[number]	DetailsTrackerSet	none	119043	116K	119212	116K
global	TabletSchemaCache[number]	DetailsTrackerSet	none	9331033	8M,920K	9331033	8M,920K
global	TabletMeta(experimental)	DetailsTrackerSet	none	25361952	24M,191K	25361952	24M,191K
global	CreateTabletRRIdxCache[number]	DetailsTrackerSet	none	107	107K	214	214K
global	SegCompaction		none	0	0K	0	0K
global	PointQueryExecutor		none	0	0K	0	0K
global	BlockCompression		none	295108	288K	295108	288K
global	RowIdStorageReader		none	0	0K	0	0K
global	SubcolumnsTree		none	2686976	2M,576K	2695168	2M,584K
global	S3FileBuffer		none	0	0K	0	0K
global	DataPageCache[size](AllocByAllocator)		none	0	0K	0	0K
global	IndexPageCache[size](AllocByAllocator)		none	0	0K	198	198K
global	PKIndexPageCache[size](AllocByAllocator)		none	630	630K	73656	71K

图 149: image

- Orphan：收集不知所属的内存，理想情况下预期等于0。
- DataPageCache\[size\](\#)：数据 Page 缓存的大小。
- IndexPageCache\[size\](\#)：数据 Page 的索引缓存的大小。
- PKIndexPageCache\[size\](\#)：数据 Page 的主键索引。
- DetailsTrackerSet：包含一些当前没有被准确跟踪的内存，这些内存不会被算在 Global 内存中，包括部分 Cache 和 元数据内存等，默认只展示 Peak Consumption 不等于 0 的 Memory Tracker，主要包括下面这些：
  - SegmentCache[size]：缓存已打开的 Segment 的内存大小，如索引信息。
  - SchemaCache[number]：缓存 Rowset Schema 的条目数。
  - TabletSchemaCache[number]：缓存 Tablet Schema 的条目数。
  - TabletMeta(experimental)：所有 Tablet Schema 的内存大小。
  - CreateTabletRRIdxCache[number]：缓存 create tabelt 索引的条目数。
  - PageNoCache：如果关闭了 page cache，这个 Memory Trakcer 将跟踪所有 Query 使用的所有 page 内存总和。
  - IOBufBlockMemory：BRPC 使用的 IOBuf 内存总和。
  - PointQueryLookupConnectionCache[number]：缓存的 Point Query Lookup Connection 条目数。
  - AllMemTableMemory：所有导入在内存中缓存等待下刷的 Memtable 内存总和。
  - MowTabletVersionCache[number]：缓存的 Mow Tablet Version 条目数。
  - MowDeleteBitmapAggCache[size]：缓存的 Mow DeleteBitmap 内存大小。
- SegCompaction：所有 SegCompaction 任务从 `Doris Allocator` 分配的内存总和。
- PointQueryExecutor：所有 Point Query 共享的一些内存。
- BlockCompression：所有 Query 共享的一些解压缩过程中使用的内存。
- RowIdStorageReader：所有 Multiget Data 请求在 RowIdStorageReader 中使用的内存。
- SubcolumnsTree：Point Query 在 SubcolumnsTree 中使用的一些内存。
- S3FileBuffer：读取 S3 时 File Buffer 分配的内存。

其中部分 Memory Tracker 标记有一些后缀，含义为：

- `[size]` 意味着 Cache Tracker 记录的是内存大小。
- `[number]` 意味着 Cache Tracker 记录的是缓存的条目数，这通常是因为目前无法准确统计内存。
- `(AllocByAllocator)` 意味着 Tracker 的值由 Doris Allocator 跟踪。
- `(experimental)` 意味着这个 Memory Tracker 还处于实验中，值可能不准确。

Global Memory 占用多

```
MemTrackerLimiter Label=global, Type=overview, Limit=-1.00 B(-1 B), Used=199.37 MB(209053204 B),
    ↪ Peak=199.37 MB(209053204 B)
```

Global Memory Tracker `Label=global, Type=overview` 的值等于所有 `Type=global` 且 `Parent Label != DetailsTrackerSet` 的 Memory Tracker 之和，主要包括 Cache 和元数据等在不同任务间共享的内存。

Cache 分析方法

参考 Cache 内存分析

Metadata 分析方法

参考 Metadata 内存分析

Orphan 分析方法

如果 Orphan Memory Tracker 值过大意味着 Memory Tracker 统计缺失，参考内存跟踪器中Memory Tracker 统计缺失中的分析。

4.8.1.3.3 Cache 内存分析

Doris 自己管理的 Cache 目前均为 LRU 淘汰策略，均支持单独通过参数控制容量和淘汰时长。

Doris Cache 类型

1. Page Cache

用于加速数据扫描。

```
- DataPageCache：缓存数据 Page。
- IndexPageCache：缓存数据 Page 的索引。
- PKIndexPageCache：缓存 Page 的主键索引。
```

2. Metadata Cache

用于加速元数据读取。

```
- SegmentCache：缓存已打开的 Segment，如索引信息。
- SchemaCache：缓存 Rowset Schema。
- TabletSchemaCache：缓存 Tablet Schema。
- CreateTabletRRIdxCache：  缓存 Create Tabelt 索引。
- MowTabletVersionCache：缓存 Mow Tablet Version。
- MowDeleteBitmapAggCache：缓存 Mow DeleteBitmap。
```

3. Cloud Cache

云上专用的缓存。

```
- CloudTabletCache：Cloud 上缓存 Tablet。
- CloudTxnDeleteBitmapCache：Cloud 上缓存 DeleteBitmap。
```

4. Inverted Index Cache

加速倒排索引。

```
- InvertedIndexSearcherCache
- InvertedIndexQueryCache
```

5. Point Query Cache

加速点查询执行，主要用于日志分析。

```
- PointQueryRowCache
- PointQueryLookupConnectionCache
```

6. Other Cache

```
- FileCache：外表查询和 Cloud 使用的文件缓存。
- CommonObjLRUCache
- LastSuccessChannelCache
```

Doris Cache 查看方法

有三种方式查看 Doris Cache 相关指标。

1. Doris BE Metrics

Web 页面 `http://{be_host}:{be_web_server_port}/metrics` 可以看到 BE 进程内存监控 (Metrics)，包括每个 Cache 的容量、使用率、元素个数、查找和命中次数等指标。

```
- `doris_be_cache_capacity{name="TabletSchemaCache"} 102400`: Cache 容量，
    ↪ 内存大小或者元素个数两种限制方法。
- `doris_be_cache_usage{name="TabletSchemaCache"} 40838`: Cache 使用量，内存大小或者元素个数，
    ↪ 对应 Cache 容量的限制。
- `doris_be_cache_usage_ratio{name="TabletSchemaCache"} 0.398809`: Cache 使用率，等于`(cache_
    ↪ usage / cache_capacity)`。
- `doris_be_cache_element_count{name="TabletSchemaCache"} 1628`: Cache 元素个数，当 Cache
    ↪ 容量限制元素个数时等于 Cache Usage。
- `doris_be_cache_lookup_count{name="TabletSchemaCache"} 63393`: 查找 Cache 的次数。
- `doris_be_cache_hit_count{name="TabletSchemaCache"} 61765`: 查找 Cache 时命中的次数。
- `doris_be_cache_hit_ratio{name="TabletSchemaCache"} 0.974319`: 命中率，等于`(hit_count / lookup
    ↪ _count)`
```

2. Doris BE Bvar

Web 页面 `http://{be_host}:{brpc_port}/vars/*cache*` 可以看到部分 Cache 独有的一些指标。

> 未来会将 Doris BE Metrics 中的指标移动到 Doris BE Bvar 中。

3. Memory Trakcer

实时查看每个 Cache 占用内存大小，参考全局内存分析，当存在内存报错时在 `be/log/be.INFO` 日志中可以找到 `Memory Tracker Summary` 中，其中包含当时的 Cache 内存大小。

Cache 内存分析

Doris BE 运行时存在各种 Cache，通常无需关注 Cache 内存，因为在 BE 进程可用内存不足时会触发内存 GC 首先清理 Cache。

但 Cache 过大会增加内存 GC 的压力，增加查询或导入报错进程可用内存不足的风险，以及 BE 进程 OOM Crash 的风险。所以如果内存持续紧张，可以考虑优先降低 Cache 的上限、关闭 Cache 或降低 Cache entry 的存活时间，更小的 Cache 在某些场景中可能会降低查询性能，但在生产环境中通常可以被容忍，调整后可以观察一段时间的查询和导入的性能。

> Doris 2.1 之前 Memory GC 还不完善，内存不足时可能无法及时释放 Cache，如果内存持续紧张，常常需要考虑手动降低 Cache 上限。

Doris 2.1.6 之后，如果希望在 BE 运行中手动清理所有 Cache，执行 `curl http://{be_host}:{be_web_server_port}/api/clear_cache/all`，将返回释放的内存大小。

下面分析不同 Cache 内存使用多的情况。

DataPageCache 内存使用多

- Doris 2.1.6 之后，执行 `curl http://{be_host}:{be_web_server_port}/api/clear_cache/DataPageCache` 可以在 BE 运行中手动清理。
- 执行 `curl -X POST http://{be_host}:{be_web_server_port}/api/update_config?disable_storage_page_cache=true` 对正在运行的 BE 禁用 DataPageCache，并默认在最长 10 分钟后清空，但这是临时方法，BE 重启后 DataPageCache 将重新生效。
- 若确认要长期减少 DataPageCache 的内存使用，参考 BE 配置项，在 `conf/be.conf` 中调小 `storage_page_cache_limit` 减小 DataPageCache 的容量，或调小 `data_page_cache_stale_sweep_time_sec` 减小 DataPageCache 缓存有效时长，或增加 `disable_storage_page_cache=true` 禁用 DataPageCache，然后重启 BE 进程。

SegmentCache 内存使用多

- Doris 2.1.6 之后，执行 `curl http://{be_host}:{be_web_server_port}/api/clear_cache/SegmentCache` 可以在 BE 运行中手动清理。
- 执行 `curl -X POST http:/{be_host}:{be_web_server_port}/api/update_config?disable_segment_cache=true` 对正在运行的 BE 禁用 SegmentCache，并默认在最长 10 分钟后清空，但这是临时方法，BE 重启后 SegmentCache 将重新生效。
- 若确认要长期减少 SegmentCache 的内存使用，参考 BE 配置项，在 `conf/be.conf` 中调整 `segment_cache_capacity` 或 `segment_cache_memory_percentage` 减小 SegmentCache 的容量，或调小 `tablet_rowset_stale_sweep_time_sec` 减小 SegmentCache 缓存有效时长，或者在 `conf/be.conf` 中增加 `disable_segment_cache=true` 禁用 SegmentCache 并重启 BE 进程。

PKIndexPageCache 内存使用多

- Doris 2.1.6 之后，执行 `curl http://{be_host}:{be_web_server_port}/api/clear_cache/PKIndexPageCache` 可以在 BE 运行中手动清理。
- 参考 BE 配置项，在 `conf/be.conf` 中调小 `pk_storage_page_cache_limit` 减小 PKIndexPageCache 的容量，或调小 `pk_index_page_cache_stale_sweep_time_sec` 减小 PKIndexPageCache 缓存有效时长，或者在 `conf/be.conf` 中增加 `disable_pk_storage_page_cache=true` 禁用 PKIndexPageCache，然后重启 BE 进程。

4.8.1.3.4 元数据内存分析

Doris BE 在内存中的元数据（Metadata）包括 Tablet、Rowset、Segment、TabletSchema、ColumnReader、PrimaryKeyIndex、BloomFilterIndex 等数据结构，有关 Doris BE 元数据的更多介绍参考文档 Doris 存储结构设计解析。

Metadata 查看方法

目前 Memory Tracker 没有准确统计 Doris BE 的元数据内存大小，通过查看 Doris BE Bvar 和 Doris BE Metrics 中的一些 Counter 去估算元数据内存大小，或者使用 Heap Profile 进一步分析。

Doris BE Bvar

Web 页面 `http://{be_host}:{brpc_port}/vars` 可以看到 Bvar 统计的一些元数据相关指标。

- `doris_total_tablet_num`：所有 Tablet 的数量。
- `doris_total_rowset_num`：所有 Rowset 的数量。
- `doris_total_segment_num`：所有打开的 Segment 数量。
- `doris_total_tablet_schema_num`：所有 Tablet Schema 的数量。
- `tablet_schema_cache_count`：Tablet Schema 被 Cache 的数量。
- `tablet_meta_schema_columns_count`：所有 Tablet Schema 中 Column 的数量。
- `tablet_schema_cache_columns_count`：Tablet Schema 中 Column 被 Cache 的数量。
- `doris_column_reader_num`：打开的 Column Reader 数量。
- `doris_column_reader_memory_bytes`：打开的 Column Reader 占用内存的字节数。
- `doris_ordinal_index_memory_bytes`：打开的 Ordinal Index 占用内存的字节数。
- `doris_zone_map_memory_bytes`：打开的 ZoneMap Index 占用内存的字节数。
- `doris_short_key_index_memory_bytes`：打开的 Short Key Index 占用内存的字节数。
- `doris_pk/index_reader_bytes`：累计的 Primary Key Index Reader 占用内存的字节数，
  ↪ 这不是实时的统计值，期待被修复。
- `doris_pk/index_reader_pages`：同上，统计的累计值。
- `doris_pk/index_reader_cached_pages`：同上，统计的累计值。
- `doris_pk/index_reader_pagindex_reader_pk_pageses`：同上，统计的累计值。
- `doris_primary_key_index_memory_bytes`：同上，统计的累计值。

Doris BE Metrics

Web 页面 `http://{be_host}:{be_web_server_port}/metrics` 可以看到 BE 进程内存监（Metrics）中的一些元数据指标。其中 Metadata Cache 相关指标参考 Cache 内存分析。

- `doris_be_all_rowsets_num`：所有 Rowset 的数量。
- `doris_be_all_segments_num`：所有 Segment 数量。

使用 Heap Profile 分析元数据内存

如果上面使用 Doris BE Bvar 和 Metrics 无法准确定位内存位置，若集群方便重启，并且现象可以被复现，参考 Heap Profile 内存分析分析 Metadata 内存。

现象复现后，如果你在 Heap Profile 内存占比大的调用栈中看到 `Tablet`，`Segment`，`TabletSchema`、`ColumnReader` 字段，则基本可以确认是元数据占用了大量内存。

4.8.1.3.5 查询内存分析

通常先使用 Query Profile 分析查询内存使用，如果 Query Profile 中统计的各个算子（Operator）内存之和远小于 Query Memory Trcker 统计到的内存，说明 Query Profile 统计到的算子内存与实际使用的内存相差较大，那么往往还需要使用 Heap Profile 进一步分析。如果 Query 因为内存超限被 Cancel，无法执行完成，此时 Query Profile 不完整，可能无法准确分析，通常直接使用 Heap Profile 分析 Query 内存使用。

查询内存查看

如果任何地方看到 `Label=query, Type=overview` Memory Tracker 的值较大，说明查询内存使用多。

```
MemTrackerLimiter Label=query, Type=overview, Limit=-1.00 B(-1 B), Used=83.32 MB(87369024 B),
    ↪ Peak=88.33 MB(92616000 B)
```

如果你已知要分析的查询，那么跳过本节继续后面的分析，否则可以参考下面的方法定位大内存查询。

首先定位大内存查询的 QueryID，在 BE web 页面 `http://{be_host}:{be_web_server_port}/mem_tracker?type=query` 中按照 `Current Consumption` 排序可以看到实时的大内存查询，在 `label` 中可以找到 QueryID。

当报错进程内存超限或可用内存不足时，在 `be.INFO` 日志中 `Memory Tracker Summary` 下半部分包含内存使用 TOP 10 的任务（查询/导入/Compaction 等）的 Memory Tracker，格式为 `MemTrackerLimiter Label=Query#Id=xxx, Type=query`，通常在 TOP 10 的任务中就能定位到大内存查询的 QueryID。

历史查询的内存统计结果可以查看`fe/log/fe.audit.log`中每个查询的`peakMemoryBytes`，或者在`be/log/be.INFO`中搜索`Deregister query/load memory tracker, queryId`查看单个 BE 上每个查询的内存峰值。

使用 Query Profile 分析查询内存使用

依据 QueryID 在 `fe/log/fe.audit.log` 中找到包括 SQL 在内的查询信息，`explain SQL` 得到查询计划，`set enable_profile=true`后执行 SQL 得到查询的 Query Profile，有关 Query Profile 的详细介绍参考文档Query Profile，这里只介绍 Query Profile 中内存相关的内容，并据此定位使用大量内存的 Operator 和数据结构。

1. 定位使用大量内存的 Operator 或内存数据结构

Query Profile 分为两部分：

- `MergedProfile`

MergedProfile 是 Query 所有 Instance Profile 的聚合结果，其中能看到每个 Fragment 的每个 Pipeline 的每个 Operator(算子) 在所有 Instance 上内存使用的 sum、avg、max、min，包括 Operator 的峰值内存 `PeakMemoryUsage` 以及 `HashTable`、`Arena` 等主要内存数据结构的峰值内存，据此定位到使用了大量内存的 Operator 和数据结构。

```
MergedProfile
          Fragments:
              Fragment  0:
                  Pipeline  :  0(instance_num=1):
                      RESULT_SINK_OPERATOR  (id=0):
                            -  MemoryUsage:  sum  ,  avg  ,  max  ,  min
                          EXCHANGE_OPERATOR  (id=20):
                                -  MemoryUsage:  sum  ,  avg  ,  max  ,  min
                                    -  PeakMemoryUsage:  sum  1.16  KB,  avg  1.16  KB,  max
                                        ↪ 1.16  KB,  min  1.16  KB
              Fragment  1:
                  Pipeline  :  1(instance_num=12):
                      AGGREGATION_SINK_OPERATOR  (id=18  ,  nereids_id=1532):
                            -  MemoryUsage:  sum  ,  avg  ,  max  ,  min
                                -  HashTable:  sum  96.00  B,  avg  8.00  B,  max  24.00  B,  min
                                    ↪  0.00
                                -  PeakMemoryUsage:  sum  1.58  MB,  avg  134.67  KB,  max
                                    ↪ 404.02  KB,  min  0.00
                                -  SerializeKeyArena:  sum  1.58  MB,  avg  134.67  KB,  max
                                    ↪ 404.00  KB,  min  0.00
                          EXCHANGE_OPERATOR  (id=17):
```

```
                                - MemoryUsage: sum , avg , max , min
                                    - PeakMemoryUsage: sum 2.25 KB, avg 192.00 B, max
                                        ↪ 768.00 B, min 0.00
```

- Execution Profile

Execution Profile 是 Query 具体每个 Instance Profile 的结果，通常依据 MergedProfile 定位到使用了大量内存的 Operator 和数据结构后，即可依据 explain SQL 后的查询计划分析其内存使用的原因，如果一些场景下需要分析 Query 在某一个 BE 结点或某一个 Instance 的内存值，可以依据 Execution Profile 进一步定位。

```
Execution Profile 36ca4f8b97834449-acae910fbee8c670:(ExecTime: 48sec201ms)
    Fragments:
        Fragment 0:
            Fragment Level Profile:    (host=TNetworkAddress(hostname:10.16.10.8, port:9013))
                ↪ :(ExecTime: 48sec111ms)
            Pipeline :1    (host=TNetworkAddress(hostname:10.16.10.8, port:9013)):
                PipelineTask (index=80):(ExecTime: 6sec267ms)
                DATA_STREAM_SINK_OPERATOR (id=17,dst_id=17):(ExecTime: 1.634ms)
                - MemoryUsage:
                    - PeakMemoryUsage: 1.50 KB
                STREAMING_AGGREGATION_OPERATOR (id=16 , nereids_id=1526):(ExecTime: 41.269ms)
                    - MemoryUsage:
                        - HashTable: 168.00 B
                        - PeakMemoryUsage: 404.16 KB
                        - SerializeKeyArena: 404.00 KB
                HASH_JOIN_OPERATOR (id=15 , nereids_id=1520):(ExecTime: 6sec150ms)
                        - MemoryUsage:
                            - PeakMemoryUsage: 3.22 KB
                            - ProbeKeyArena: 3.22 KB
                    LOCAL_EXCHANGE_OPERATOR (PASSTHROUGH) (id=-12):(ExecTime: 67.950ms)
                            - MemoryUsage:
                                - PeakMemoryUsage: 1.41 MB
```

2. HASH_JOIN_SINK_OPERATOR 内存占用多

```
HASH_JOIN_SINK_OPERATOR (id=12 , nereids_id=1304):(ExecTime: 1min14sec)
    - JoinType: INNER_JOIN
    - BroadcastJoin: true
    - BuildRows: 600.030257M (600030257)
    - InputRows: 600.030256M (600030256)
    - MemoryUsage:
        - BuildBlocks: 15.65 GB
        - BuildKeyArena: 0.00
        - HashTable: 6.24 GB
        - PeakMemoryUsage: 21.89 GB
```

可见主要使用内存的 Hash Join Build 阶段的 BuildBlocks 和 HashTable，通常 Hash Join 的 Build 阶段使用内存太多，首先确认 Join Reorder 顺序是否合理，通常正确的顺序是小表用于 Hash Join Build，大表用于 Hash Join Probe，这样可以最小化 Hash Join 整体的内存使用，并通常具有更好的性能。

为了确认 Join Reorder 顺序是否合理，我们找到 id=12 的 HASH_JOIN_OPERATOR 的 profile，可以看到 ProbeRows 只有 196240 行，所以这个 Hash Join Reorder 正确的顺序应该交换左表和右表的位置，可以 `set disable_join_reorder=true` 关闭 Join Reorder 并手动指定左表和右表的顺序后执行 Query 验证，进一步可参考查询优化器中 Join Reorder 相关的文档。

```
HASH_JOIN_OPERATOR  (id=12  ,  nereids_id=1304):(ExecTime:  8sec223ms)
    -  BlocksProduced:  227
    -  MemoryUsage:
        -  PeakMemoryUsage:  0.00
        -  ProbeKeyArena:  0.00
    -  ProbeRows:  196.24K  (196240)
    -  RowsProduced:  786.22K  (786220)
```

使用 Heap Profile 分析查询内存使用

如果上面使用 Query Profile 无法准确定位内存的使用位置，若集群方便重启，并且现象可以被复现，参考 Heap Profile 内存分析分析 Query 内存。

在 Query 执行前 Dump 一次 Heap Profile，在 Query 执行过程中再 Dump 一次 Heap Profile，通过使用 `jeprof --dot lib/doris_be --base=heap_dump_file_1 heap_dump_file_2` 对比两个 Heap Profile 之间的内存变化，可以得出代码中的每个函数在 Query 执行过程中使用的内存占比，对照代码即可定位内存使用位置，因为 Query 执行过程中内存实时变化，所以可能需要在 Query 执行过程中多次 Dump Heap Profile 并对比分析。

4.8.1.3.6 导入内存分析

Doris 数据导入分为 Fragment 读取和 Channel 写入两个阶段，其中 Fragment 和查询的 Fragment 执行逻辑相同，不过 Stream Load 通常只有 Scan Operator。Channel 主要将数据写入临时的数据结构 Memtable，然后 Delta Writer 将数据压缩后写入文件。

导入内存查看

如果任何地方看到 `Label=load, Type=overview` Memory Tracker 的值较大，说明导入内存使用多。

```
MemTrackerLimiter Label=load, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0 B)
```

Doris 导入的内存分为两部分，第一部分是 Fragment 执行使用的内存，第二部分是 MemTable 的构造和下刷过程中使用的内存。

在 BE web 页面 `http://{be_host}:{be_web_server_port}/mem_tracker?type=global` 中找到 `Label=AllMemTableMemory, Parent Label=DetailsTrackerSet` 的 Memory Tracker 是这台 BE 结点上所有导入任务构造和下刷 MemTable 使用的内存。报错进程内存超限或可用内存不足时，在 be.INFO 日志中 `Memory Tracker Summary` 也可以找到这个 Memory Tracker。

```
MemTracker Label=AllMemTableMemory, Parent Label=DetailsTrackerSet, Used=25.08 MB(26303456 B),
    ↪ Peak=25.08 MB(26303456 B)
```

导入内存分析

如果 ‘Label=AllMemTableMemory 的值很小，则导入任务主要使用内存的位置是执行 Fragment，分析方式和查询内存分析相同，此处不再赘述。

如果 Label=AllMemTableMemory 的值很大，则可能 MemTable 下刷不及时，可以考虑减小 be.conf 中 load_process_max_memory_limit_percent 和 load_process_soft_mem_limit_percent 的值，这可以让 MemTable 更频繁的下刷，从而在内存中缓存的 MemTable 更少，但写入的文件数量会变多，如果写入了太多的小文件会增加 Compaction 的压力，如果 Compaction 不及时将导致元数据内存变大，查询变慢，甚至文件数量超出限制后导入将报错。

在导入执行过程中查看 BE Web 页面 /mem_tracker?type=load，依据 Label=MemTableManualInsert 和 Label=MemTableHookFlush 两组 Memory Tracker 的值，可以定位 MemTable 内存使用大的 LoadID 和 TabltID。

4.8.1.3.7 查询报错 Process Memory Not Enough

当 Query 的报错信息中出现 MEM_LIMIT_EXCEEDED 且包含 Process memory not enough 时，说明因为进程可用内存不足被 Cancel。

首先解析错误信息，确认 Cancel 的原因、Cancel 时 Query 自身使用的内存大小、以及进程的内存状态。Query 被 Cancel 的原因通常有如下三种：

1. 被 Cancel 的 Query 自身内存过大。
2. 被 Cancel 的 Query 自身内存较小，有其他内存更大的 Query 存在。
3. 全局共享的 Cahce、元数据等内存过大，或者查询和导入任务之外的其他任务内存过大

错误信息解析

进程可用内存不足分为两种情况，一是进程当前内存超出配置的内存上限，二是系统剩余可用内存低于水位线。存在三个路径会 Cancel 查询等任务：

- 如果报错信息包含cancel top memory used，说明任务在内存 Full GC 中被 Cancel。
- 如果报错信息包含cancel top memory overcommit，说明任务在内存 Minor GC 中被 Cancel。
- 如果报错信息包含Allocator sys memory check failed，说明任务从 Doris Allocator 申请内存失败后被 Cancel。

在对下面报错信息的解析后，

- 若查询和导入自身使用的内存占到进程内存的很大比例，参考 [Query 自身内存过大] 分析查询和导入的内存使用，尝试调整参数或优化 SQL 来减少执行需要的内存。
- 若任务自身使用的内存很少，参考 [查询和导入之外的进程内存过大] 尝试减少进程其他位置的内存使用，从而保留更多的内存用于查询等任务执行。

有关内存限制和水位线计算方法、内存 GC 的更多介绍见内存控制策略

1 在内存 Full GC 中被 Cancel

若 BE 进程内存超过进程内存上限（ MemLimit ）或系统剩余可用内存低于内存低水位线 (LowWaterMark) 时触发 Full GC，此时会优先 Cancel 内存最大的任务。

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.8)[MEM_LIMIT_EXCEEDED]Process memory
    ↪ not enough, cancel top memory used query: <Query#Id=e862471398b14e71-9361a1ab8153cb29>
    ↪ consumption 866.97 MB, backend 10.16.10.8, process memory used 3.12 GB exceed limit 3.01
    ↪ GB or sys available memory 191.25 GB less than low water mark 3.20 GB. Execute again
    ↪ after enough memory, details see be.INFO.
```

错误信息解析：

1. (10.16.10.8): 查询过程中内存不足的 BE 节点。
2. query: <Query#Id=e862471398b14e71-9361a1ab8153cb29> consumption 866.97 MB：当前被 cancel 的 queryID，Query 本身使用了 866.97 MB 内存。
3. process memory used 3.12 GB exceed limit 3.01 GB or sys available memory 191.25 GB less than low water mark 3.20 GB 进程内存超限的原因，此处是 BE 进程使用的物理内存 3.12 GB 超过了 3.01 GB 的 MemLimit，当前操作系统剩余可供 BE 使用的内存为 191.25 GB 仍高于 LowWaterMark 3.20 GB。

2 在内存 Minor GC 中被 Cancel

若 Doris BE 进程内存超过进程内存软限（SoftMemLimit）或系统剩余可用内存低于内存警告水位线（WarningWaterMark）时触发 Minor GC，此时会优先 Cancel 内存超限比例最大的 Query。

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.8)[MEM_LIMIT_EXCEEDED]Process memory
    ↪ not enough, cancel top memory overcommit query: <Query#Id=e862471398b14e71-9361
    ↪ a1ab8153cb29> consumption 866.97 MB, backend 10.16.10.8, process memory used 2.12 GB
    ↪ exceed soft limit 2.71 GB or sys available memory 3.25 GB less than warning water mark
    ↪ 6.40 GB. Execute again after enough memory, details see be.INFO.
```

错误信息解析：

process memory used 3.12 GB exceed soft limit 6.02 GB or sys available memory 3.25 GB less than warning water mark 6.40 GB 进程内存超限的原因，此处是当前操作系统剩余可供 BE 使用的内存为 3.25 GB 低于 WarningWaterMark 6.40 GB, BE 进程使用的物理内存 2.12 GB 没有超过 2.71 GB 的 SoftMemLimit。

3 从 Allocator 申请内存失败

Doris BE 的大内存申请都会通过 Doris Allocator 分配，并在分配时检查内存大小，如果进程可用内存不足则会抛出异常和尝试 Cancel 当前 Query。

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.8)[MEM_LIMIT_EXCEEDED]PreCatch error
    ↪ code:11, [E11] Allocator sys memory check failed: Cannot alloc:4294967296, consuming
    ↪ tracker:<Query#Id=457efb1fdae74d3b-b4fffdcfd4baaf32>, peak used 405956032, current used
    ↪ 386704704, exec node:<>, process memory used 2.23 GB exceed limit 3.01 GB or sys
    ↪ available memory 181.67 GB less than low water mark 3.20 GB.
```

错误信息解析：

1. consuming tracker:<Query#Id=457efb1fdae74d3b-b4fffdcfd4baaf32>, peak used 405956032, current used 386704704, exec node:VAGGREGATION_NODE (id=7)>：当前被 Cancel 的 queryID，Query 当前使

用了 386704704 Bytes 内存，Query 内存峰值为 405956032 Bytes，正在执行的算子为 VAGGREGATION_NODE (id=7)>。

2. Cannot alloc:4294967296: 当前申请 4 GB 内存时失败，因为当前进程内存 2.23 GB 加上 4 GB 将大于 3.01 GB 的 MemLimit。

被 Cancel 的 Query 自身内存过大

参考查询内存分析或导入内存分析分析查询和导入的内存使用，尝试调整参数或优化 SQL 来减少执行需要的内存。

需要注意的是，若任务从 Allocator 申请内存失败后被 Cancel，Cannot alloc 或 try alloc 显示 Query 当前正在申请的内存过大，此时需要关注此处的内存申请是否合理，在 be/log/be.INFO 搜索 Allocator sys memory check failed 可以找到申请内存的栈。

被 Cancel 的 Query 自身内存较小，有其他内存更大的 Query 存在

通常是因为内存更大的 Query 在 Cancel 阶段卡住，无法及时释放内存。Full GC 会先按照内存从大到小的顺序 Cancel Query，再按照内存从大到小的顺序 Cancel Load。若 Query 在内存 Full GC 中被 Cancel，但此时 BE 进程中存在其他 Query 的内存大于当前被 Cancel 的 Query，需要关注这些更大内存的 Query 是否在 Cancel 过程中卡住。

首先执行 grep {queryID} be/log/be.INFO 找到 Query 被 Cancel 的时间点，然后在上下文搜索 Memory Tracker Summary 找到进程内存统计日志，若 Memory Tracker Summary 中存在使用内存更大的 Query 存在。执行 grep {更大内存的queryID} be/log/be.INFO 确认是否有 Cancel 关键词的日志，对应时间点就是 Query 被 Cancel 的时间，若该 Query 同样被 Cancel，且这个更大内存的 Query 被 Cancel 的时间点和当前 Query 被 Cancel 的时间点不同，参考内存问题 FAQ 中Query Cancel 过程中卡住 分析这个更大内存的 Query 是否在 Cancel 过程中卡住。有关 Memory Tracker Summary 的分析参考内存日志分析。

查询和导入任务之外的进程内存过大

尝试定位内存位置并考虑减少内存使用，保留更多的内存用于查询和导入执行。

任务因进程可用内存不足被 Cancel 的时间点可以在 be/log/be.INFO 中找到进程内存统计日志，执行 grep queryID be/log/be.INFO 找到 Query 被 Cancel 的时间点，然后在上下文搜索 Memory Tracker Summary 找到进程内存统计日志，然后参考内存日志分析中进程内存统计日志分析 章节进一步分析。在分析前先参考内存跟踪器中Memory Tracker 统计缺失 章节分析 Memory Tracker 是否存在统计缺失。

若 Memory Tracker 存在统计缺失，则参考Memory Tracker 统计缺失 章节进一步分析。否则 Memory Tracker 统计了大部分内存，不存在统计缺失，参考 Overview 分析 Doris BE 进程不同部分内存占用过大的原因以及减少其内存使用的方法。

4.8.1.3.8 查询报错 Memory Tracker Limit Exceeded

当查询或导入的报错信息中出现 MEM_LIMIT_EXCEEDED 且包含 memory tracker limit exceeded 时，说明任务超过单次执行内存限制。

错误信息解析

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.8)[MEM_LIMIT_EXCEEDED]PreCatch error
    ↪ code:11, [E11] Allocator mem tracker check failed, [MEM_LIMIT_EXCEEDED]failed alloc size
    ↪ 1.03 MB, memory tracker limit exceeded, tracker label:Query#Id=f78208b15e064527-
    ↪ a84c5c0b04c04fcf, type:query, limit 100.00 MB, peak used 99.29 MB, current used 99.25 MB.
    ↪  backend 10.16.10.8, process memory used 2.65 GB. exec node:<execute:<ExecNode:VHASH_JOIN
    ↪ _NODE (id=4)>>, can `set exec_mem_limit=8G` to change limit, details see be.INFO.
```

错误信息分为两部分：

1. failed alloc size 1.03 MB, memory tracker limit exceeded, tracker label:Query#Id=f78208b15e064527-a84c5c0b04c04fcf, type:query, limit 100.00 MB, peak used 99.29 MB, current used 99.25 MB：当前正在执行 Query f78208b15e064527-a84c5c0b04c04fcf 在尝试申请 1.03 MB 内存的过程中发现查询超过单次执行的内存上限，查询内存上限是 100 MB（Session Variables 中的 exec_mem_limit），当前已经使用 99.25 MB，内存峰值是 99.29 MB。
2. backend 10.16.10.8, process memory used 2.65 GB. exec node:<execute:<ExecNode:VHASH_JOIN_NODE (id=4)>>, can set exec_mem_limit=8G to change limit, details see be.INFO.：本次内存申请的位置是VHASH_JOIN_NODE (id=4)，并提示可通过 set exec_mem_limit 来调高单次查询的内存上限。

单次执行内存限制和内存超发

show variables; 可以查看 Doris Session Veriable，其中的 exec_mem_limit 是单次查询和导入的执行内存限制，但从 Doris 1.2 开始支持查询内存超发 (overcommit)，旨在允许查询设置更灵活的内存限制，内存充足时即使查询内存超过上限也不会被 Cancel，所以通常用户无需关注查询内存使用。直到内存不足时，查询会在尝试分配新内存时等待一段时间，此时会基于一定规则优先 Cancel mem_used 与 exec_mem_limit 比值大的 Query。如果等待过程中内存释放的大小满足需求，查询将继续执行，否则将抛出异常并终止查询。

如果希望关闭查询内存超发，参考 BE 配置项，在 conf/be.conf 中增加 enable_query_memory_overcommit=false，此时单次查询和导入的内存超过 exec_mem_limit 即会被 Cancel。如果你希望避免大查询对集群稳定性造成的负面影响，或者希望准确控制集群上的任务执行来保证足够的稳定性，那么可以考虑关闭查询内存超发。

查询内存分析

如果需要分析查询的内存使用，参考查询内存分析。

set enable_profile=true 开启 Query Profile 后，在任务超过单次执行的内存上限时，在 be/log/be.INFO 将打印查询正在申请内存的调用栈，并可以看到查询每个算子当前使用的内存和峰值，参考内存日志分析分析 Process Memory Summary 和 Memory Tracker Summary，帮助确认当前查询内存使用是否符合预期。

```
Allocator mem tracker check failed, [MEM_LIMIT_EXCEEDED]failed alloc size 32.00 MB, memory
    ↪ tracker limit exceeded, tracker label:Query#I
d=41363cb6ba734ad5-bc8720bdf9b3090d, type:query, limit 100.00 MB, peak used 75.32 MB, current
    ↪ used 72.62 MB. backend 10.16.10.8, process memory used 2.33 GB. exec node:<>, can `set
    ↪ exec_mem_limit=8G`
 to change limit, details see be.INFO.
Process Memory Summary:
    os physical memory 375.81 GB. process memory used 2.33 GB(= 2.60 GB[vm/rss] - 280.53 MB[tc/
        ↪ jemalloc_cache] + 0[reserved] + 0B[waiting_refresh]), limit 338.23 GB, soft limit
        ↪ 304.41 GB. sys availab
le memory 337.33 GB(= 337.33 GB[proc/available] - 0[reserved] - 0B[waiting_refresh]), low water
    ↪ mark 6.40 GB, warning water mark 12.80 GB.
```

```
Memory Tracker Summary:    MemTrackerLimiter Label=Query#Id=41363cb6ba734ad5-bc8720bdf9b3090d,
    ↪ Type=query, Limit=100.00 MB(104857600 B), Used=72.62 MB(76146688 B), Peak=75.32 MB
    ↪ (78981248 B)
    MemTracker Label=HASH_JOIN_SINK_OPERATOR, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=122.00 B(122 B), Peak=122.00 B(122 B)
    MemTracker Label=VDataStreamRecvr:41363cb6ba734ad5-bc8720bdf9b309fe, Parent Label=Query#Id
        ↪ =41363cb6ba734ad5-bc8720bdf9b3090d, Used=0(0 B), Peak=384.00 B(384 B)
    MemTracker Label=local data queue mem tracker, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=0(0 B), Peak=384.00 B(384 B)
    MemTracker Label=HASH_JOIN_SINK_OPERATOR, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=21.73 MB(22790276 B), Peak=21.73 MB(22790276 B)
    MemTracker Label=VDataStreamRecvr:41363cb6ba734ad5-bc8720bdf9b309fe, Parent Label=Query#Id
        ↪ =41363cb6ba734ad5-bc8720bdf9b3090d, Used=0(0 B), Peak=2.23 MB(2342912 B)
    MemTracker Label=local data queue mem tracker, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=0(0 B), Peak=2.23 MB(2342912 B)
    MemTracker Label=HASH_JOIN_SINK_OPERATOR, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=24.03 MB(25201284 B), Peak=24.03 MB(25201284 B)
    MemTracker Label=VDataStreamRecvr:41363cb6ba734ad5-bc8720bdf9b309fe, Parent Label=Query#Id
        ↪ =41363cb6ba734ad5-bc8720bdf9b3090d, Used=1.08 MB(1130496 B), Peak=7.17 MB(7520256 B)
    MemTracker Label=local data queue mem tracker, Parent Label=Query#Id=41363cb6ba734ad5-
        ↪ bc8720bdf9b3090d, Used=1.08 MB(1130496 B), Peak=7.17 MB(7520256 B)
```

4.8.1.3.9 OOM Killer Crash 分析

如果 BE 进程 Crash 后 `log/be.out` 中没有报错信息，执行 `dmesg -T` 如果看到下面的日志，说明触发了 OOM Killer，可见 `20240718 15:03:59` 时 pid 为 360303 的 doris_be 进程物理内存（anon-rss）约 60 GB。

```
[Thu Jul 18 15:03:59 2024] Out of memory: Killed process 360303 (doris_be) total-vm:213416916kB,
    ↪ anon-rss:62273128kB, file-rss:0kB, shmem-rss:0kB, UID:0 pgtables:337048kB oom_score_adj:0
```

理想情况下，Doris 会定时检测操作系统剩余可用内存，并在内存不足时采取包括阻止后续内存申请、触发内存 GC 在内的一系列操作来避免触发 OOM Killer，但刷新内存状态和内存 GC 都具有一定的滞后性，并且很难完全 Catch 所有大内存申请，在集群压力过大时仍有一定几率触发 OOM Killer，导致 BE 进程 Crash。此外如果进程内存状态异常，导致内存 GC 无法释放内存，导致进程实际可用内存减少，这将加剧集群的内存压力。

如果不幸触发了 OOM Killer，首先依据日志分析 BE 进程触发 OOM Killer 前的内存状态和任务执行情况，然后针对性调参让集群恢复稳定。

找到触发 OOM Killer 时间点前的内存日志

触发 OOM Killer 时意味着进程可用内存不足，参考内存日志分析在 `be/log/be.INFO` 触发 OOM Killer 时间点自下而上找到最后一次打印的 `Memory Tracker Summary` 关键词并分析 BE 进程的主要内存位置。

> `less be/log/be.INFO` 打开文件后，首先跳转到触发 OOM Killer 对应时间的日志，以上面 `dmesg -T` 的结果为例，输入 `/20240718 15:03:59` 后回车搜索对应时间，如果搜不到，可能是触

> 发 OOM Killer 的时间有些偏差，可以搜索 `/20240718 15:03:`。日志跳转到对应时间后，输入 `/Memory Tracker Summary` 后回车搜素关键词，默认会在日志向下搜索，如果搜索不到或时间对应不上，需要 `shift + n` 先上搜索，找到最后一次打印的 `Memory Tracker Summary` 以及同时打印的 `Process Memory Summary` 内存日志。

集群内存压力过大导致触发 OOM Killer

若满足如下现象，那么可以认为是集群内存压力过大，导致在某一时刻进程内存状态没有及时刷新，内存 GC 没能及时释放内存，导致没能有效控制 BE 进程内存。

> Doris 2.1 之前 Memory GC 还不完善，内存持续紧张时往往更容易触发 OOM Killer。

- 对 `Memory Tracker Summary` 的分析发现查询和其他任务、各个 Cache、元数据等内存使用都合理。
- 对应时间段的 BE 进程内存监控显示长时间维持在较高的内存使用率，不存在内存泄漏的迹象
- 定位 `be/log/be.INFO` 中 OOM Killer 时间点前的内存日志，自下而上搜索 GC 关键字，发现 BE 进程频繁执行内存 GC。

此时参考 BE 配置项在`be/conf/be.conf`中调小`mem_limit`，调大 `max_sys_mem_available_low_water_mark_bytes` ，有关内存限制和水位线计算方法、内存 GC 的更多介绍见内存控制策略。

此外还可以调节其他参数控制内存状态刷新和 GC，包括 `memory_gc_sleep_time_ms`，`soft_mem_limit_frac`，`memory_maintenance_sleep_time_ms`，`process_minor_gc_size`，`process_full_gc_size`，`enable_query_memory_overcommit`，`thread_wait_gc_max_milliseconds` 等。

一些异常问题导致触发 OOM Killer

若不满足集群内存压力过大的现象，那么可能此时内存状态异常，内存 GC 可能无法及时释放内存，下面列举一些常见的导致触发 OOM Killer 的异常问题。

Memory Tracker 统计缺失

若日志 `Memory Tracker Summary` 中 `Label=process resident memory` Memory Tracker 减去 `Label=sum of all trackers` Memory Tracker 差值较大，或者 Orphan Memory Tracker 值过大，说明 Memory Tracker 存在统计缺失，参考内存跟踪器中Memory Tracker 统计缺失 章节进一步分析。

Query Cancel 过程中卡住

再 `be/log/be.INFO` 日志中定位到 OOM Killer 的时间点，然后在上下文搜索 `Memory Tracker Summary` 找到进程内存统计日志，若 `Memory Tracker Summary` 中存在使用内存较大的 Query。执行 `grep {queryID} be/log/be.INFO` 确认是否有 `Cancel` 关键词的日志，对应时间点就是 Query 被 Cancel 的时间，若该 Query 已经被 Cancel，且 Query 被 Cancel 的时间点和触发 OOM Killer 的时间点相隔较久，参考内存问题 FAQ 中对Query Cancel 过程中卡住 的分析。有关 `Memory Tracker Summary` 的分析参考内存日志分析。

Jemalloc Metadata 内存占用大

内存 GC 目前无法释放 Jemalloc Metadata，参考内存跟踪器中对 Label=tc/jemalloc_metadata Memory Tracker 的分析，减少内存使用。

Jemalloc Cache 内存占用大

> 常见于 Doris 2.0

Doris 2.0 be.conf 中 JEMALLOC_CONF 的 lg_tcache_max 默认值是 20，这在某些场景会导致 Jemalloc Cache 太大且无法自动释放，参考 Jemalloc 内存分析减少 Jemalloc Cache 内存占用。

4.8.1.3.10 内存日志分析

be/log/be.INFO 中的进程内存日志主要分为两类，一是进程内存状态日志，包括进程内存大小和系统剩余可用内存大小。二是更加详细的进程内存统计日志，包含 Memory Tracker 统计的内存大小。

进程内存状态日志分析

Doris BE 进程内存每次增长或减少 256 MB 都会在 log/be.INFO 日志打印一次进程内存状态，另外进程内存不足时，也会随其他日志一起打印进程内存状态。

```
os physical memory 375.81 GB. process memory used 4.09 GB(= 3.49 GB[vm/rss] - 410.44 MB[tc/
    ↪ jemalloc_cache] + 1 GB[reserved] + 0B[waiting_refresh]), limit 3.01 GB, soft limit 2.71
    ↪ GB. sys available memory 134.41 GB(= 135.41 GB[proc/available] - 1 GB[reserved] - 0B[
    ↪ waiting_refresh]), low water mark 3.20 GB, warning water mark 6.40 GB.
```

1. os physical memory 375.81 GB 指系统物理内存 375.81 GB。
2. process memory used 4.09 GB(= 3.49 GB[vm/rss] - 410.44 MB[tc/jemalloc_cache] + 1 GB[reserved ↪] + 0B[waiting_refresh])

   - 当前我们认为 BE 进程使用了 4.09 GB 内存，实际 BE 进程使用的物理内存 vm/rss 是 3.49 GB，
   - 其中有 410.44 MB 是 tc/jemalloc_cache，这部分 Cache 会在之后执行过程中被优先复用，所以这里不将其算作 BE 进程内存。
   - reserved 是在执行过程中被预留的内存，通常在构建 HashTable 等会耗费大量内存的操作前会提前预留 HashTable 的内存，确保构建 HashTable 的过程不会因为内存不足而终止，这部分预留的内存被计算在 BE 进程内存中，即使实际上还没有被分配。
   - waiting_refresh 是两次内存状态刷新的间隔中申请的大内存，Doris 内存状态刷新的间隔默认是 100ms，为避免两次内存状态刷新的间隔中发生大量内存申请，在内存超限后没有及时感知和触发内存 GC，所以间隔中申请的大内存被计算在 BE 进程内存中，每次内存状态刷新后waiting_refresh都将清 0，

3. sys available memory 134.41 GB(= 135.41 GB[proc/available] - 1 GB[reserved] - 0B[waiting_ ↪ refresh])

   - 当前 BE 进程剩余可使用的内存是 134.41 GB，系统中实际可提供给 BE 进程使用的内存 proc/available 是 135.41 GB.

- 其中有 1GB 的内存已经被预留，所以在计算 BE 进程剩余可用内存时减去 reserved，关于 reserved 和 waiting_refresh 的介绍参考上面对 BE 进程内存的注解。

4. limit 3.01 GB, soft limit 2.71 GB 和 low water mark 3.20 GB, warning water mark 6.40 GB，有关 MemLimit 和 WaterMark 的更多介绍见内存限制和水位线计算方法。

进程内存统计日志分析

当进程可用内存不足后，BE 大多数位置的内存申请都会感知，尝试做出预定的回调方法，包括触发 Memory GC 或 Cancel 查询等，并打印进程内存统计日志，打印默认间隔是 1s，日志分为两部分 Process Memory Summary 和 Memory Tracker Summary 两部分，在be/log/be.INFO 中可以找到，据此确认当前进程内存使用是否符合预期。

```
Process Memory Summary:
    os physical memory 375.81 GB. process memory used 4.09 GB(= 3.49 GB[vm/rss] - 410.44 MB[tc/
        ↪ jemalloc_cache] + 1 GB[reserved] + 0B[waiting_refresh]), limit 3.01 GB, soft limit
        ↪ 2.71 GB. sys available memory 134.41 GB(= 135.41 GB[proc/available] - 1 GB[reserved]
        ↪ - 0B[waiting_refresh]), low water mark 3.20 GB, warning water mark 6.40 GB.
Memory Tracker Summary:
    MemTrackerLimiter Label=other, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0 B)
    MemTrackerLimiter Label=schema_change, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
        ↪ =0(0 B)
    MemTrackerLimiter Label=compaction, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0
        ↪  B)
    MemTrackerLimiter Label=load, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0 B)
    MemTrackerLimiter Label=query, Type=overview, Limit=-1.00 B(-1 B), Used=83.32 MB(87369024 B),
        ↪  Peak=88.33 MB(92616000 B)
    MemTrackerLimiter Label=global, Type=overview, Limit=-1.00 B(-1 B), Used=199.37 MB(209053204
        ↪ B), Peak=199.37 MB(209053204 B)
    MemTrackerLimiter Label=tc/jemalloc_cache, Type=overview, Limit=-1.00 B(-1 B), Used=410.44 MB
        ↪ (430376896 B), Peak=-1.00 B(-1 B)
    MemTrackerLimiter Label=tc/jemalloc_metadata, Type=overview, Limit=-1.00 B(-1 B), Used=144 MB
        ↪ (151759440 B), Peak=-1.00 B(-1 B)
    MemTrackerLimiter Label=sum of all trackers, Type=overview, Limit=-1.00 B(-1 B), Used=114.80
        ↪ MB(726799124 B), Peak=-1.00 B(-1 B)
    MemTrackerLimiter Label=process resident memory, Type=overview, Limit=-1.00 B(-1 B), Used
        ↪ =3.49 GB(3743289344 B), Peak=3.49 GB(3743289344 B)
    MemTrackerLimiter Label=reserved_memory, Type=overview, Limit=-1.00 B(-1 B), Used=0(0 B),
        ↪ Peak=-1.00 B(-1 B)
    MemTrackerLimiter Label=process virtual memory, Type=overview, Limit=-1.00 B(-1 B), Used
        ↪ =44.25 GB(47512956928 B), Peak=44.25 GB(47512956928 B)
    MemTrackerLimiter Label=Orphan, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0 B)
    MemTrackerLimiter Label=DetailsTrackerSet, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B),
        ↪ Peak=0(0 B)
    MemTracker Label=IOBufBlockMemory, Parent Label=DetailsTrackerSet, Used=1.41 MB(1474560 B),
        ↪ Peak=1.41 MB(1474560 B)
```

```
MemTracker Label=SegmentCache[size], Parent Label=DetailsTrackerSet, Used=1.64 MB(1720543 B),
    ↪  Peak=18.78 MB(19691997 B)
MemTracker Label=SchemaCache[number], Parent Label=DetailsTrackerSet, Used=9.21 KB(9428 B),
    ↪ Peak=9.21 KB(9428 B)
MemTracker Label=TabletSchemaCache[number], Parent Label=DetailsTrackerSet, Used=9.29 MB
    ↪ (9738798 B), Peak=9.29 MB(9738798 B)
MemTracker Label=TabletMeta(experimental), Parent Label=DetailsTrackerSet, Used=25.08 MB
    ↪ (26303456 B), Peak=25.08 MB(26303456 B)
MemTracker Label=RuntimeFilterMergeControllerEntity(experimental), Parent Label=
    ↪ DetailsTrackerSet, Used=32.00 B(32 B), Peak=32.00 B(32 B)
MemTrackerLimiter Label=SegCompaction, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
    ↪ =0(0 B)
MemTrackerLimiter Label=PointQueryExecutor, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B),
    ↪ Peak=0(0 B)
MemTrackerLimiter Label=BlockCompression, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
    ↪ =0(0 B)
MemTrackerLimiter Label=RowIdStorageReader, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B),
    ↪ Peak=0(0 B)
MemTrackerLimiter Label=SubcolumnsTree, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
    ↪ =0(0 B)
MemTrackerLimiter Label=S3FileBuffer, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0
    ↪  B)
MemTrackerLimiter Label=DataPageCache[size](\#), Type=global, Limit=-1.00 B(-1 B), Used
    ↪ =198.70 MB(208357157 B), Peak=198.73 MB(208381892 B)
MemTrackerLimiter Label=IndexPageCache[size](\#), Type=global, Limit=-1.00 B(-1 B), Used
    ↪ =679.73 KB(696047 B), Peak=679.73 KB(696047 B)
MemTrackerLimiter Label=PKIndexPageCache[size](\#), Type=global, Limit=-1.00 B(-1 B), Used
    ↪ =0(0 B), Peak=0(0 B)
MemTrackerLimiter Label=Query#Id=529e3cb37dff464c-93bd9eafa8944ea6, Type=query, Limit=2.00 GB
    ↪ (2147483648 B), Used=83.32 MB(87369024 B), Peak=88.33 MB(92616000 B)
MemTrackerLimiter Label=MemTableTrackerSet, Type=load, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
    ↪ =0(0 B)
MemTrackerLimiter Label=SnapshotManager, Type=other, Limit=-1.00 B(-1 B), Used=0(0 B), Peak
    ↪ =0(0 B)
MemTracker Label=AllMemTableMemory, Parent Label=DetailsTrackerSet, Used=0(0 B), Peak=0(0 B)
```

Process Memory Summary 是进程内存状态，参考上文进程内存状态日志分析。

Memory Tracker Summary 是进程 Memory Tracker 汇总，包含所有 Type=overview 和 Type=global 的 Memory Tracker，帮助使用者分析当时的内存状态，参考 Overview 分析每一部分内存的含义。

4.8.1.3.11 Heap Profile 分析内存

Heap Profile 支持实时查看进程内存使用，并可以看到调用栈，所以这通常需要对代码有一些了解，需要注意的是 Heap Profile 记录的是虚拟内存，需要修改配置后重启 Doris BE 进程，并且现象可以被复现。

Doris 使用 Jemalloc 作为默认的 Allocator，参照下面的方法使用 Heap Profile。

1. 将 be.conf 中 JEMALLOC_CONF 的 prof_active:false 修改为 prof_active:true 并重启 Doris BE。
2. 执行 curl http://be_host:8040/jeheap/dump 后会在 ${DORIS_HOME}/log 目录看到生成的 profile 文件。
3. 执行 jeprof --dot ${DORIS_HOME}/lib/doris_be ${DORIS_HOME}/log/profile_file 后将终端输出的文本贴到在线 dot 绘图网站，生成内存分配图。

以上流程基于 Doris 2.1.8 和 3.0.4 及之后的版本，用于实时的分析内存，如需长时间观测内存，或观测内存申请的累积值，更多有关 Jemalloc Heap Profile 的使用方法参考 Jemalloc Heap Profile

如果在 Heap Profile 内存占比大的调用栈中看到 Segment，TabletSchema、ColumnReader 字段，说明元数据占用内存大。

如果集群运行一段时间后静置时 BE 内存不下降，此时在 Heap Profile 内存占比大的调用栈中看到 Agg，Join，Filter，Sort，Scan 等字段，查看对应时间段的 BE 进程内存监控若呈现持续上升的趋势，那么有理由怀疑存在内存泄漏，依据调用栈对照代码继续分析。

如果集群上任务执行期间在 Heap Profile 内存占比大的调用栈中看到 Agg，Join，Filter，Sort，Scan 等字段，任务结束后内存正常释放，说明大部分内存被正在运行的任务使用，不存在泄漏，如果此时 Label=query，Type=overview Memory Tracker 的值占总内存的比例，小于 Heap Profile 中包含上述字段的内存调用栈占总内存的比例，说明 Label=query，Type=overview Memory Tracker 统计的不准确，可以在社区及时反馈。

4.8.1.4 内存特性

4.8.1.4.1 内存跟踪器

Doris BE 使用内存跟踪器（Memory Tracker）记录进程内存使用，包括查询、导入、Compaction、Schema Change 等任务生命周期中使用的内存，以及各项缓存。支持 Web 页面实时查看，并在内存相关报错时打印到 BE 日志中，用于内存分析和排查内存问题。

有关 Memory Tracker 的查看方法，以及不同 Memory Tracker 所代表内存占用过大的原因以及减少其内存使用的分析方法在 Overview 中已结合 Doris BE 内存结构一起介绍。本文只介绍 Memory Tracker 原理、结构，以及一些常见问题。

内存跟踪原理

Memory Tracker 依赖 Doris Allocator 跟踪内存的每次申请和释放，有关 Doris Allocator 的介绍参考内存控制策略。

进程内存：Doris BE 会定时从系统获取 Doris BE 进程内存，兼容 Cgroup。

任务内存：每个查询、导入、Compaction 等任务初始化时都会创建自己唯一的 Memory Tracker，在执行过程中将 Memory Tracker 放入 TLS（Thread Local Storage）中，Doris 主要的内存数据结构都继承自 Allocator，Allocator 每次申请和释放内存都会记录到 TLS 的 Memory Tracker 中。

算子内存：任务的不同执行算子也会创建自己的 Memory Trakcer，比如 Join/Agg/Sink 等，支持手动跟踪内存或放入 TLS 中由 Doris Allocator 记录，用于执行逻辑控制，以及 Query Profile 中分析不同算子的内存使用情况。

全局内存：全局内存主要包括 Cache 和元数据等在不同任务间共享的内存。每个 Cache 有自己唯一的 Memory Tracker，由 Doris Allocator 或手动跟踪；元数据内存目前没有统计完全，更多要依赖 Metrics 和 Bvar 统计的各种元数据 Counter 进行分析。

其中 Doris BE 进程内存因为取自操作系统，可以认为是完全准确的，其他 Memory Tracker 因为实现上的局限性，跟踪的内存通常只是真实内存的一个子集，导致大多数情况下所有 Memory Tracker 之和要小于 Doris BE 进程物

理内存，存在一定的缺失，不过 Memory Tracker 记录到的内存在大多数情况下可信度较高，可以放心的用于内存分析。此外 Memory Tracker 实际跟踪的是虚拟内存，而不是通常更关注的物理内存，它们之间也存在一定的误差。

Memory Tracker 结构

根据使用方式 Memory Tracker 分为两类，第一类 Memory Tracker Limiter，在每个查询、导入、Compaction 等任务和全局 Cache、TabletMeta 唯一，用于观测和控制内存使用；第二类 Memory Tracker，主要用于跟踪查询执行过程中的内存热点，如 Join/Aggregation/Sort/窗口函数中的 HashTable、序列化的中间数据等，来分析查询中不同算子的内存使用情况，以及用于导入数据下刷的内存控制。

二者之间的父子关系只用于快照的打印，使用 Lable 名称关联，相当于一层软链接，不依赖父子关系同时消费，生命周期互不影响，减少理解和使用的成本。所有 Memory Tracker 存放在一组 Map 中，并提供打印所有 Memory Tracker Type 的快照、打印 Query/Load/Compaction 等 Task 的快照、获取当前使用内存最多的一组 Query/Load、获取当前过量使用内存最多的一组 Query/Load 等方法。

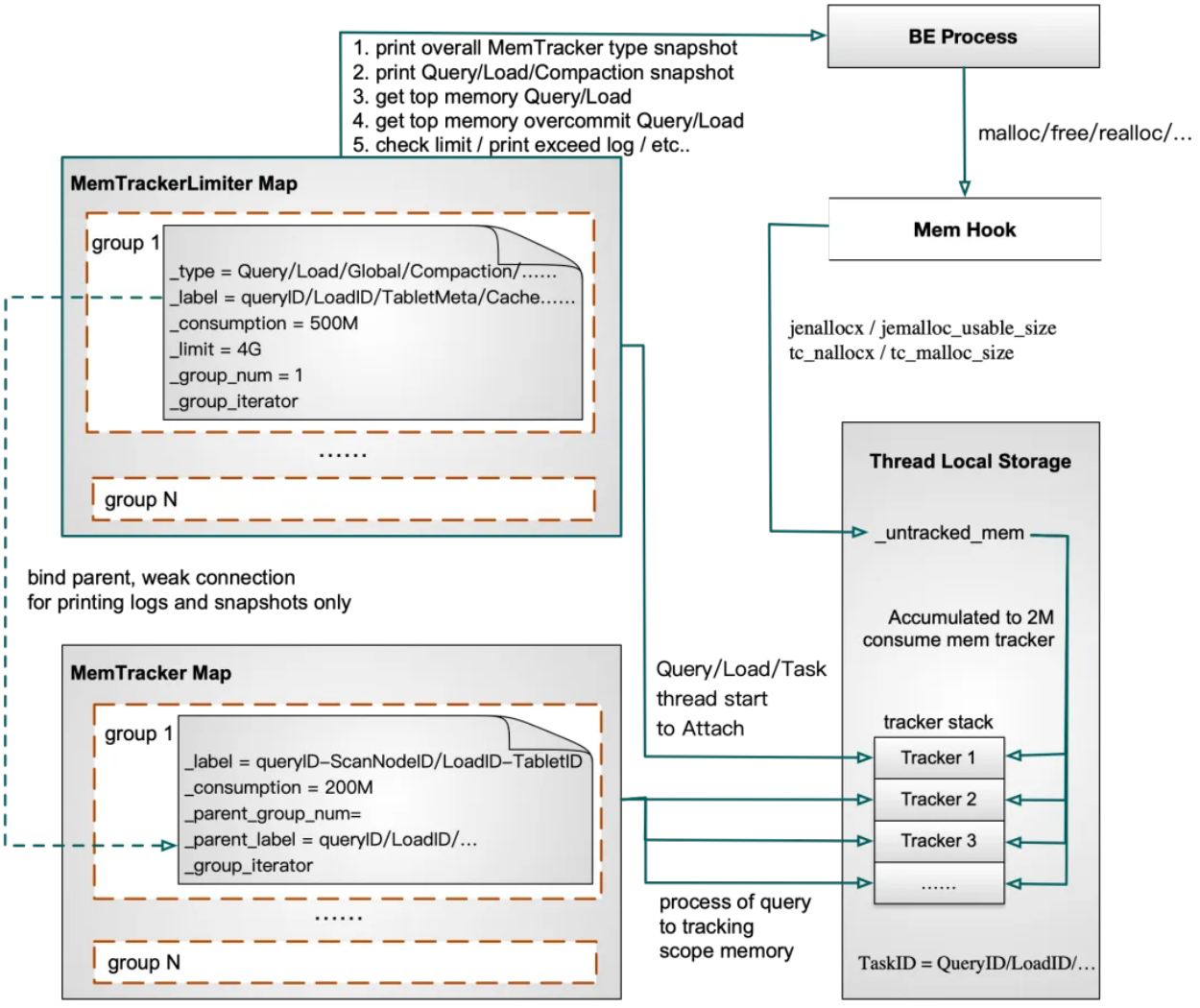


图 150: Memory Tracker Implement

Memory Tracker 统计缺失

Doris 2.1 之前和之后的版本中 Memory Tracker 统计缺失的现象不同。

Memory Tracker 统计缺失现象

1. Doris 2.1 之后 Memory Tracker 统计缺失有两个现象。

- `Label=process resident memory` Memory Tracker 减去 `Label=sum of all trackers` Memory Tracker 的差值过大。
- Orphan Memory Tracker 值过大。

2. Doris 2.1 之前 Orphan Memory Tracker 值过大意味着 Memory Tracker 统计缺失。

Memory Tracker 统计缺失分析

> 在 Doris 2.1.5 之前的版本中发现 Memory Tracker 统计缺失或 BE 进程内存不下降，优先参考 Cache 内存分析分析 SegmentCache 内存使用，尝试关闭 Segment Cache 后继续测试。

> 在 Doris 2.1.5 之前的版本中 Segment Cache Memory Tacker 不准确，这是因为包括 Primary Key Index 在内的一些 Index 内存统计的是不准确的，导致 Segment Cache 内存没有得到有效限制，经常占用过大的内存，尤其是在成百上千列的大宽表上，参考 Metadata 内存分析如果你发现 Doris BE Metrics 中 `doris_be_cache_usage{name="SegmentCache"}` 不大，但 Doris BE Bvar 中 `doris_column_reader_num` 很大，则需要怀疑 Segment Cache 的内存占用，如果你在 Heap Profile 内存占比大的调用栈中看到 `Segment`，`ColumnReader` 字段，则基本可以确认是 Segment Cache 占用了大量内存。

如果观察到上述现象，若集群方便重启，并且现象可以被复现，参考 Heap Profile 内存分析使用 Jemalloc Heap Profile 分析进程内存。

否则可以先参考 Metadata 内存分析分析 Doris BE 的元数据内存。

Memory Tracker 统计缺失原因

下面介绍 Memory Tracker 统计缺失的原因，涉及到 Memory Tracker 的实现，通常无需关注。

Doris 2.1 之后

1. `Label=process resident memory` Memory Tracker 减去 `Label=sum of all trackers` Memory Tracker 的差值过大。

若 Label=sum of all trackers Memory Tracker 的值占到 Label=process resident memory Memory Tracker 的 70% 以上，通常说明 Memory Tracker 统计到了 Doris BE 进程的大部分内存，通常只需要分析 Memory Tracker 定位内存位置。

若 Label=sum of all trackers Memory Tracker 的值占到 Label=process resident memory Memory Tracker 的 70% 以下，说明 Memory Tracker 统计缺失，此时 Memory Tracker 可能无法准确定位内存位置。

Label=process resident memory Memory Tracker 减去 Label=sum of all trackers Memory Tracker 的差值是没有使用 Doris Allocator 分配的内存，Doris 主要内存数据结构都继承自 Doris Allocator，但仍有一部分内存没有使用 Doris Allocator 分配，包括元数据内存、RPC 内存等，也可能是存在内存泄漏，此时除了分析内存值大的 Memory Tracker 外，通常还需要关注元数据内存是否合理，是否存在内存泄漏等。

2. Orphan Memory Tracker 值过大

```
MemTrackerLimiter Label=Orphan, Type=global, Limit=-1.00 B(-1 B), Used=0(0 B), Peak=0(0 B)
```

Orphan Memory Tracker 是默认 Memory Tracker，值为正数或负数都意味着 Doris Allocator 分配的内存跟踪不准，值越大，意味着 Memory Tracker 整体统计结果的可信度越低。其统计值有两个来源：

- 如果线程开始时 TLS 中没有绑定 Memory Tracker，那么 Doris Allocator 会默认将内存记录到 Orphan Memory Tracker 中，意味着这部分内存不知所属，有关 Doris Allocator 记录内存的原理参考上文内存跟踪原理。
- Query 或 Load 等任务 Memory Tracker 析构时如果值不等于 0，，通常意味着这部分内存没有释放，将把这部分剩余的内存记录到 Orphan Memory Tracker 中，相当于将剩余内存交由 Orphan Memory Tracker 继续跟踪。从而保证 Orphan Memory Tracker 和其他 Memory Tracker 之和等于 Doris Allocator 分配出去的所有内存。

理想情况下，期望 Orphan Memory Tracker 的值接近 0。所以我们希望所有线程开始时都 Attach 一个 Orphan 之外的 Memory Tracker，比如 Query 或 Load Memory Tracker。并且所有 Query 或 Load Memory Tracker 析构时都等于 0，这意味着 Query 或 Load 执行过程中使用的内存在析构时都已经被释放。

如果 Orphan Memory Tracker 不等于 0 且值较大，这意味着有大量不知所属的内存没有被释放，或者 Query 和 Load 执行结束后有大量的内存没有被释放，

Doris 2.1 之前

Doris 2.1 之前将不知所属的内存都统计到 Label=Orphan Memory Tracker 中，所以 Orphan Memory Tracker 值过大意味着 Memory Tracker 统计缺失。

4.8.1.4.2 内存控制策略

Doris Allocator 作为系统中大块内存申请的统一入口，在合适的时机干预限制内存分配的过程，确保内存申请的高效可控。

Doris MemoryArbitrator 作为内存仲裁器，实时监控 Doris BE 进程的内存使用，并定时更新内存状态和收集内存相关统计信息的快照。

Doris MemoryReclamation 作为内存回收器，在可用内存不足时触发内存 GC 回收部分内存，保证集群上大部分任务执行的稳定性。

Doris Allocator

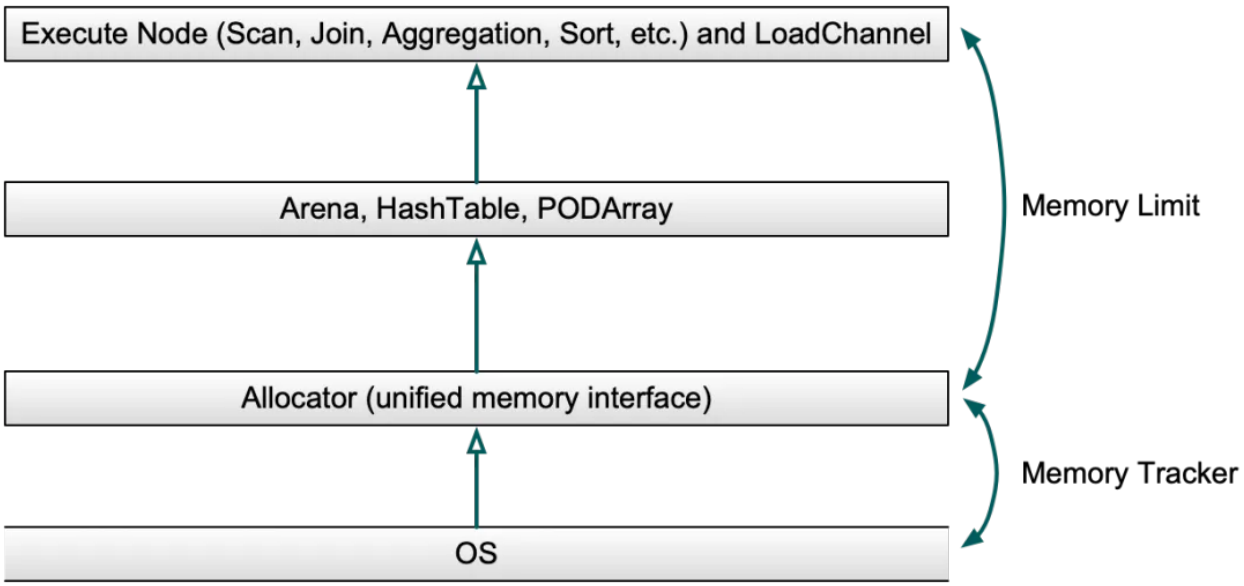


图 151: Memory Management Overview

Allocator 从系统申请内存，并在申请过程中使用 MemTracker 跟踪内存申请和释放的大小，执行算子所需批量申请的大内存将交由不同的数据结构管理。

查询执行过程中大块内存的分配主要使用 Arena、HashTable、PODArray 这三个数据结构管理，Allocator 作为 Arena、PODArray、 HashTable 的统一内存接口，实现内存统一管理和局部的内存复用。

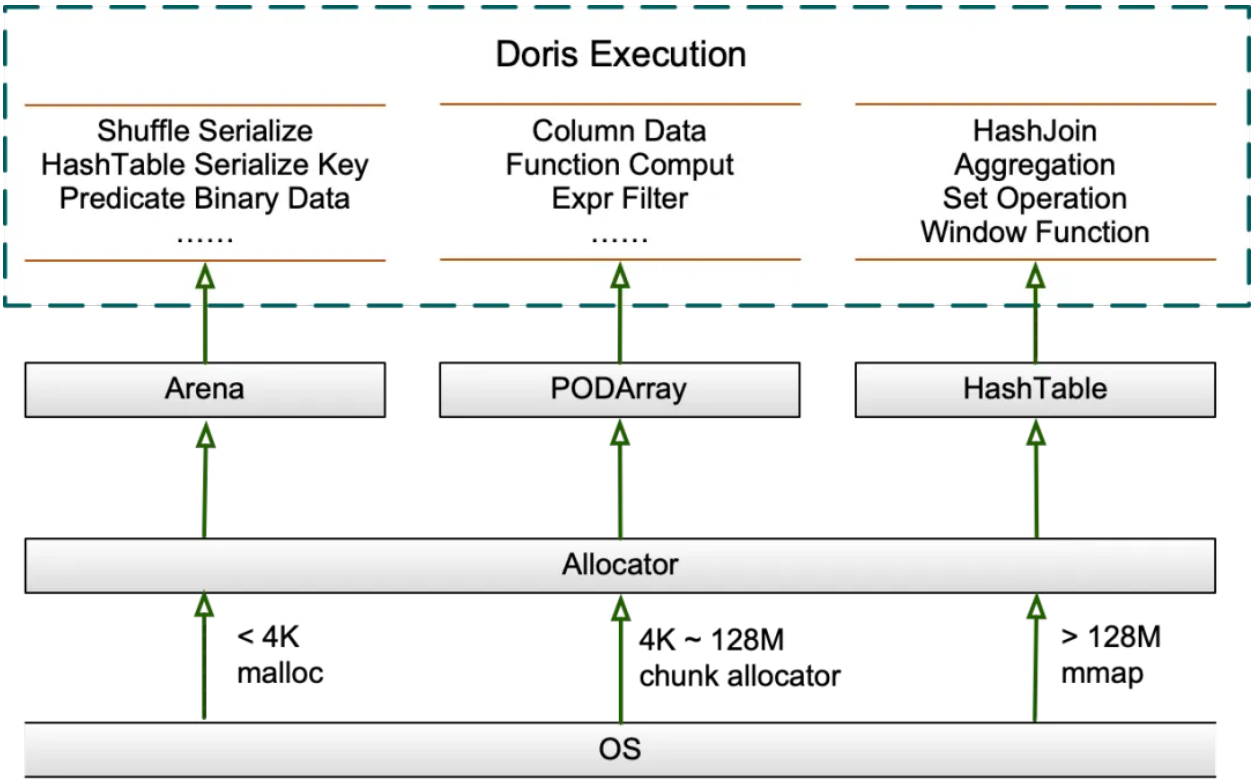


图 152: Memory Allocator

Allocator 使用通用内存分配器申请内存，在 Jemalloc 和 TCMalloc 的选择上，Doris 之前在高并发测试时 TCMalloc 中 CentralFreeList 的 Spin Lock 能占到查询总耗时的 40%，虽然关闭 aggressive memory decommit 能有效提升性能，但这会浪费非常多的内存，为此不得不单独用一个线程定期回收 TCMalloc 的缓存。Jemalloc 在高并发下性能优于 TCMalloc 且成熟稳定，在 Doris 1.2.2 版本中切换为 Jemalloc，调优后在大多数场景下性能和 TCMalloc 持平，并使用更少的内存，高并发场景的性能也有明显提升。

Arena

Arena 是一个内存池，维护一个内存块列表，并从中分配内存以响应 alloc 请求，从而减少从系统申请内存的次数以提升性能，内存块被称为 Chunk，在内存池的整个生命周期内存在，在析构时统一释放，这通常和查询生命周期相同，并支持内存对齐，主要用于保存 Shuffle 过程中序列化/反序列化数据、HashTable 中序列化 Key 等。

Chunk 初始 4096 字节，内部使用游标记录分配过的内存位置，如果当前 Chunk 剩余大小无法满足当前内存申请，则申请一个新的 Chunk 添加到列表中，为减少从系统申请内存的次数，在当前 Chunk 小于 128M 时，每次新申请的 Chunk 大小加倍，在当前 Chunk 大于 128M 时，新申请的 Chunk 大小在满足本次内存申请的前提下至多额外分配 128M，避免浪费过多内存，默认之前的 Chunk 不会再参与后续 alloc。

HashTable

Doris 中的 HashTable 主要在 Hash Join、聚合、集合运算、窗口函数中应用，主要使用的 PartitionedHashTable 最多包含 16 个子 HashTable，支持两个 HashTable 的并行化合并，每个子 Hash Join 独立扩容，预期可减少总内存的使用，扩容期间的延迟也将被分摊。

在 HashTable 小于 8M 时将以 4 的倍数扩容，在 HashTable 大于 8M 时将以 2 的倍数扩容，在 HashTable 小于 2G 时扩容因子为 50%，即在 HashTable 被填充到 50% 时触发扩容，在 HashTable 大于 2G 后扩容因子被调整为 75%，为

了避免浪费过多内存，在构建 HashTable 前通常会依据数据量预扩容。此外 Doris 为不同场景设计了不同的 HashTable，比如聚合场景使用 PHmap 优化并发性能。

PODArray

PODArray 是一个 POD 类型的动态数组，与 std::vector 的区别在于不会初始化元素，支持部分 std::vector 的接口，同样支持内存对齐并以 2 的倍数扩容，PODArray 析构时不会调用每个元素的析构函数，而是直接释放掉整块内存，主要用于保存 String 等 Column 中的数据，此外在函数计算和表达式过滤中也被大量使用。

内存复用

Doris 在执行层做了大量内存复用，可见的内存热点基本都被屏蔽。比如对数据块 Block 的复用贯穿 Query 执行的始终；比如 Shuffle 的 Sender 端始终保持一个 Block 接收数据，一个 Block 在 RPC 传输中，两个 Block 交替使用；还有存储层在读一个 Tablet 时复用谓词列循环读数、过滤、拷贝到上层 Block、Clear；导入 Aggregate Key 表时缓存数据的 MemTable 到达一定大小预聚合收缩后继续写入等等。

此外 Doris 会在数据 Scan 开始前依据 Scanner 个数和线程数预分配一批 Free Block，每次调度 Scanner 时会从中获取一个 Block 并传递到存储层读取数据，读取完成后会将 Block 放到生产者队列中，供上层算子消费并进行后续计算，上层算子将数据拷走后会将 Block 重新放回 Free Block 中，用于下次 Scanner 调度，从而实现内存复用，数据 Scan 完成后 Free Block 会在之前预分配的线程统一释放，避免内存申请和释放不在同一个线程而导致的额外开销，Free Block 的个数一定程度上还控制着数据 Scan 的并发。

内存 GC

Doris BE 会定时从系统获取进程的物理内存和系统当前剩余可用内存，并收集所有查询、导入、Compaction 任务 MemTracker 的快照，当 BE 进程内存超限或系统剩余可用内存不足时，Doris 将释放 Cache 和终止部分查询或导入来释放内存，这个过程由一个单独的 GC 线程定时执行。

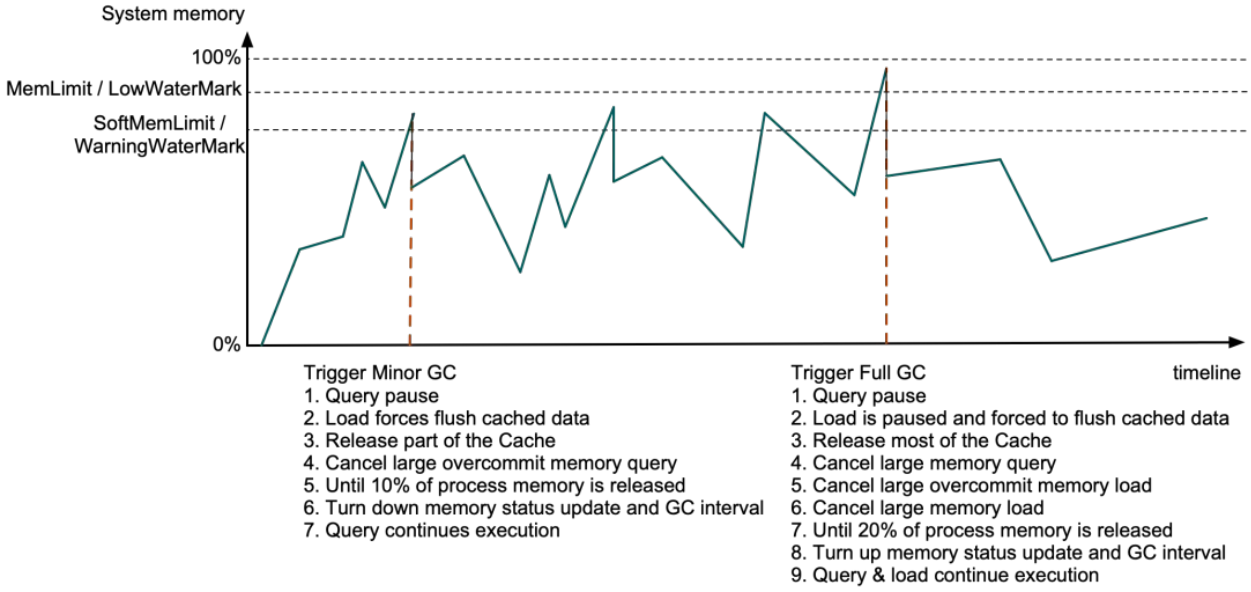


图 153: Memory GC

若 Doris BE 进程内存超过 SoftMemLimit（默认系统总内存的 81%）或系统剩余可用内存低于 Warning 水位线（通常不大于 3.2GB）时触发 Minor GC，此时查询会在 Allocator 分配内存时暂停，同时导入强制下刷缓存中的数据，

并释放部分 Data Page Cache 以及过期的 Segment Cache 等，若释放的内存不足进程内存的 10%，若启用了查询内存超发，则从内存超发比例大的查询开始 Cancel，直到释放 10% 的进程内存或没有查询可被 Cancel，然后调低系统内存状态获取间隔和 GC 间隔，其他查询在发现剩余内存后将继续执行。

若 BE 进程内存超过 MemLimit（默认系统总内存的 90%）或系统剩余可用内存低于 Low 水位线（通常不大于 1.6GB）时触发 Full GC，此时除上述操作外，导入在强制下刷缓存数据时也将暂停，并释放全部 Data Page Cache 和大部分其他 Cache，如果释放的内存不足 20%，将开始按一定策略在所有查询和导入的 MemTracker 列表中查找，依次 Cancel 内存占用大的查询、内存超发比例大的导入、内存占用大的导入，直到释放 20% 的进程内存后，调高系统内存状态获取间隔和 GC 间隔，其他查询和导入也将继续执行，GC 的耗时通常在几百 us 到几十 ms 之间。

内存限制和水位线计算方法

- 进程内存上限 MemLimit = `be.conf/mem_limit * PhysicalMemory`, 默认系统总内存的 90%，具体参考。
- 进程内存软限 SoftMemLimit = `be.conf/mem_limit * PhysicalMemory * be.conf/soft_mem_limit_frac`, 默认系统总内存的 81%。
- 系统剩余可用内存低水位线 LowWaterMark = `be.conf/max_sys_mem_available_low_water_mark_bytes`，默认等于 -1，此时 LowWaterMark = `min(PhysicalMemory - MemLimit, PhysicalMemory * 0.05)`, 在 64G 内存的机器上 LowWaterMark 的值略小于 3.2 GB（因为 `PhysicalMemory` 的真实值往往小于 64G）。
- 系统剩余可用内存警告水位线 WarningWaterMark = `2 * LowWaterMark`，在 64G 内存的机器上 `WarningWaterMark` 默认略小于 6.4 GB。

系统剩余可用内存计算

当错误信息中系统可用内存小于低水位线时，同样当作进程内存超限处理，其中系统可用内存的值来自于`/proc/meminfo`中的`MemAvailable`，当`MemAvailable`不足时继续内存申请可能返回 std::bad_alloc 或者导致 BE 进程 OOM，因为刷新进程内存统计和 BE 内存 GC 都具有一定的滞后性，所以预留小部分内存 buffer 作为低水位线，尽可能避免 OOM。

其中`MemAvailable`是操作系统综合考虑当前空闲的内存、buffer、cache、内存碎片等因素给出的一个在尽可能不触发 swap 的情况下可以提供给用户进程使用的内存总量，一个简单的计算公式：`MemAvailable = MemFree - LowWaterMark + (PageCache - min(PageCache / 2, LowWaterMark))`，和 cmd `free`看到的`available`值相同，具体可参考：

why-is-memavailable-a-lot-less-than-memfreebufferscached

Linux MemAvailable

低水位线默认最大 3.2G（2.1.5 之前默认 1.6G），根据 `MemTotal`、`vm/min_free_kbytes`、`confg::mem_limit`、`config::max_sys_mem_available_low_water_mark_bytes`共同算出，并避免浪费过多内存。其中`MemTotal`是系统总内存，取值同样来自`/proc/meminfo`；`vm/min_free_kbytes`是操作系统给内存 GC 过程预留的 buffer，取值通常在 0.4% 到 5% 之间，某些云服务器上`vm/min_free_kbytes`可能为 5%，这会导致直观上系统可用内存比真实值少；调大`config::max_sys_mem_available_low_water_mark_bytes`将在大于 64G 内存的机器上，为 Full GC 预留更多的内存 buffer，反之调小将尽可能充分使用内存。

4.8.2 Compaction 优化

Doris 通过类似 LSM-Tree 的结构写入数据，在后台通过 Compaction 机制不断将小文件合并成有序的大文件，同时也会处理数据的删除、更新等操作。适当的调整 Compaction 的策略，可以极大地提升导入效率和查询效率。Doris 提供如下几种 compaction 方式进行调优：

4.8.2.1 Vertical compaction

> 自 Doris 1.2.2 版本起支持 Vertical compaction

Vertical compaction 是 Doris 1.2.2 版本中实现的新的 Compaction 算法，用于解决大宽表场景下的 Compaction 执行效率和资源开销问题。可以有效降低 Compaction 的内存开销，并提升 Compaction 的执行速度。

实际测试中，Vertical compaction 使用内存仅为原有 compaction 算法的 1/10，同时 compaction 速率提升 15%。

Vertical compaction 中将按行合并的方式改变为按列组合并，每次参与合并的粒度变成列组，降低单次 compaction 内部参与的数据量，减少 compaction 期间的内存使用。

开启和配置方法 (BE 配置)：- `enable_vertical_compaction = true` 可以开启该功能

- `vertical_compaction_num_columns_per_group = 5` 每个列组包含的列个数，经测试，默认 5 列一组 compaction 的效率及内存使用较友好
- `vertical_compaction_max_segment_size` 用于配置 vertical compaction 之后落盘文件的大小，默认值 268435456(字节)

4.8.2.2 Segment compaction

Segment compaction 主要应对单批次大数据量的导入场景。和 Vertical compaction 的触发机制不同，Segment compaction 是在导入过程中，针对一批次数据内，多个 Segment 进行的合并操作。这种机制可以有效减少最终生成的 Segment 数量，避免 -238（OLAP_ERR_TOO_MANY_SEGMENTS）错误的出现。Segment compaction 有以下特点：

- 可以减少导入产生的 segment 数量
- 合并过程与导入过程并行，不会额外增加导入时间
- 导入过程中的内存和计算资源的使用量会有增加，但因为平摊在整个导入过程中所以涨幅较低
- 经过 Segment compaction 后的数据在进行后续查询以及标准 compaction 时会有资源和性能上的优势

开启和配置方法 (BE 配置)：

- `enable_segcompaction = true` 可以使能该功能
- `segcompaction_batch_size` 用于配置合并的间隔。默认 10 表示每生成 10 个 segment 文件将会进行一次 segment compaction。一般设置为 10 - 30，过大的值会增加 segment compaction 的内存用量。

如有以下场景或问题，建议开启此功能：

- 导入大量数据触发 OLAP_ERR_TOO_MANY_SEGMENTS (errcode -238) 错误导致导入失败。此时建议打开 segment compaction 的功能，在导入过程中对 segment 进行合并控制最终的数量。
- 导入过程中产生大量的小文件：虽然导入数据量不大，但因为低基数数据，或因为内存紧张触发 memtable 提前下刷，产生大量小 segment 文件也可能会触发 OLAP_ERR_TOO_MANY_SEGMENTS 导致导入失败。此时建议打开该功能。
- 导入大量数据后立即进行查询：刚导入完成、标准 compaction 还没有完成工作时，此时 segment 文件过多会影响后续查询效率。如果用户有导入后立即查询的需求，建议打开该功能。
- 导入后标准 compaction 压力大：segment compaction 本质上是把标准 compaction 的一部分压力放在了导入过程中进行处理，此时建议打开该功能。

不建议使用的情况：- 导入操作本身已经耗尽了内存资源时，不建议使用 segment compaction 以免进一步增加内存压力使导入失败。

关于 segment compaction 的实现和测试结果可以查阅此链接。

4.8.2.3 单副本 compaction

默认情况下，多个副本的 compaction 是独立进行的，每个副本在都需要消耗 CPU 和 IO 资源。开启单副本 compaction 后，在一个副本进行 compaction 后，其他几个副本拉取 compaction 后的文件，因此 CPU 资源只需要消耗 1 次，节省了 N - 1 倍 CPU 消耗（N 是副本数）。

单副本 compaction 在表的 PROPERTIES 中通过参数 `enable_single_replica_compaction` 指定，默认为 false 不开启，设置为 true 开启。

该参数可以在建表时指定，或者通过 `ALTER TABLE table_name SET("enable_single_replica_compaction" = "true")` 来修改。

4.8.2.4 Compaction 策略

Compaction 策略决定什么时候将哪些小文件合并成大文件。Doris 当前提供了 2 种 compaction 策略，通过表属性的 `compaction_policy` 参数指定。

4.8.2.4.1 size_based compaction 策略

size_based compaction 策略是默认策略，对大多数场景适用。

```
"compaction_policy" = "size_based"
```

4.8.2.4.2 time_series compaction 策略

time_series compaction 策略是为日志、时序等场景优化的策略。它利用时序数据具有时间局部性的特点，将相邻时间写入的小文件合并成大文件，每个文件只会参与一次 compaction 就合并成比较大的文件，减少反复 compaction 带来的写放大。

```
"compaction_policy" = "time_series"
```

time_series compaction 策略在下面 3 个条件任意一个满足的时候触发小文件合并：- 未合并的文件大小超过 `time_series_compaction_goal_size_mbytes` (默认 1GB) - 未合并的文件个数超过 `time_series_compaction_file_count_threshold` (默认 2000) - 距离上次合并的时间超过 `time_series_compaction_time_threshold_seconds` (默认 1 小时)

上述参数在表的 PROPERTIES 中设置，可以在建表时指定，或者通过 `ALTER TABLE table_name SET("name" = "value")` 修改。

4.8.2.5 Compaction 并发控制

Compaction 在后台执行需要消耗 CPU 和 IO 资源，可以通过控制 compaction 并发线程数来控制资源消耗。

compaction 并发线程数在 BE 的配置文件中配置，包括下面几个：- `max_base_compaction_threads`：base compaction 的线程数，默认是 4 - `max_cumu_compaction_threads`： cumulative compaction 的线程数，默认是 -1，表示每块盘 1 个线程 - `max_single_replica_compaction_threads`： 单副本 compaction 拉取数据文件的线程数，默认是 10

4.8.3 元数据运维

> 注意除非绝对必要，否则请避免使用 `metadata_failure_recovery`，使用可能会导致元数据截断、元数据丢失以及元数据 Split-brains 的发生。强烈建议谨慎使用此功能，以防止由于不规范的操作程序导致数据不可恢复的损坏。

本文档主要介绍在实际生产环境中，如何对 Doris 的元数据进行管理。包括 FE 节点建议的部署方式、一些常用的操作方法、以及常见错误的解决方法。

在阅读本文当前，请先阅读 Doris 元数据设计文档了解 Doris 元数据的工作原理。

4.8.3.1 重要提示

- 当前元数据的设计是无法向后兼容的。即如果新版本有新增的元数据结构变动（可以查看 FE 代码中的 `FeMetaVersion.java` 文件中是否有新增的 VERSION），那么在升级到新版本后，通常是无法再回滚到旧版本的。所以，在升级 FE 之前，请务必按照升级文档 中的操作，测试元数据兼容性。

4.8.3.2 元数据目录结构

我们假设在 fe.conf 中指定的 `meta_dir` 的路径为 `/path/to/doris-meta`。那么一个正常运行中的 Doris 集群，元数据的目录结构应该如下：

```
/path/to/doris-meta/
            |-- bdb/
            |   |-- 00000000.jdb
```

```
|   |-- je.config.csv
|   |-- je.info.0
|   |-- je.info.0.lck
|   |-- je.lck
|   `-- je.stat.csv
`-- image/
    |-- ROLE
    |-- VERSION
    `-- image.xxxx
```

1. bdb 目录

   我们将 bdbje 作为一个分布式的 kv 系统，存放元数据的 journal。这个 bdb 目录相当于 bdbje 的“数据目录”。

   其中 .jdb 后缀的是 bdbje 的数据文件。这些数据文件会随着元数据 journal 的不断增多而越来越多。当 Doris 定期做完 image 后，旧的日志就会被删除。所以正常情况下，这些数据文件的总大小从几 MB 到几 GB 不等（取决于使用 Doris 的方式，如导入频率等）。当数据文件的总大小大于 10GB，则可能需要怀疑是否是因为 image 没有成功，或者分发 image 失败导致的历史 journal 一直无法删除。

   je.info.0 是 bdbje 的运行日志。这个日志中的时间是 UTC+0 时区的。通过这个日志，也可以查看一些 bdbje 的运行情况。

2. image 目录

   image 目录用于存放 Doris 定期生成的元数据镜像文件。通常情况下，你会看到有一个 image.xxxxx 的镜像文件。其中 xxxxx 是一个数字。这个数字表示该镜像包含 xxxxx 号之前的所有元数据 journal。而这个文件的生成时间（通过 ls -al 查看即可）通常就是镜像的生成时间。

   你也可能会看到一个 image.ckpt 文件。这是一个正在生成的元数据镜像。通过 du -sh 命令应该可以看到这个文件大小在不断变大，说明镜像内容正在写入这个文件。当镜像写完后，会自动重名为一个新的 image.xxxxx 并替换旧的 image 文件。

   只有角色为 Master 的 FE 才会主动定期生成 image 文件。每次生成完后，都会推送给其他非 Master 角色的 FE。当确认其他所有 FE 都收到这个 image 后，Master FE 会删除 bdbje 中旧的元数据 journal。所以，如果 image 生成失败，或者 image 推送给其他 FE 失败时，都会导致 bdbje 中的数据不断累积。

   ROLE 文件记录了 FE 的类型（FOLLOWER 或 OBSERVER），是一个文本文件。

   VERSION 文件记录了这个 Doris 集群的 cluster id，以及用于各个节点之间访问认证的 token，也是一个文本文件。

   ROLE 文件和 VERSION 文件只可能同时存在，或同时不存在（如第一次启动时）。

4.8.3.3 基本操作

4.8.3.3.1 启动单节点 FE

单节点 FE 是最基本的一种部署方式。一个完整的 Doris 集群，至少需要一个 FE 节点。当只有一个 FE 节点时，这个节点的类型为 Follower，角色为 Master。

1. 第一次启动

1. 假设在 fe.conf 中指定的 `meta_dir` 的路径为 `/path/to/doris-meta`。
2. 确保 `/path/to/doris-meta` 已存在，权限正确，且目录为空。
3. 直接通过 `bash bin/start_fe.sh --daemon` 即可启动。
4. 启动后，你应该可以在 fe.log 中看到如下日志：
    - Palo FE starting…
    - image does not exist: /path/to/doris-meta/image/image.0
    - transfer from INIT to UNKNOWN
    - transfer from UNKNOWN to MASTER
    - the very first time to open bdb, dbname is 1
    - start fencing, epoch number is 1
    - finish replay in xxx msec
    - QE service start
    - thrift server started

    以上日志不一定严格按照这个顺序，但基本类似。
5. 单节点 FE 的第一次启动通常不会遇到问题。如果你没有看到以上日志，一般来说是没有仔细按照文档步骤操作，请仔细阅读相关 wiki。

2. 重启

    1. 直接使用 `bash bin/start_fe.sh` 可以重新启动已经停止的 FE 节点。
    2. 重启后，你应该可以在 fe.log 中看到如下日志：
        - Palo FE starting…
        - finished to get cluster id: xxxx, role: FOLLOWER and node name: xxxx
        - 如果重启前还没有 image 产生，则会看到：
            - image does not exist: /path/to/doris-meta/image/image.0
        - 如果重启前有 image 产生，则会看到：
            - start load image from /path/to/doris-meta/image/image.xxx. is ckpt: false
            - finished load image in xxx ms
        - transfer from INIT to UNKNOWN
        - replayed journal id is xxxx, replay to journal id is yyyy
        - transfer from UNKNOWN to MASTER
        - finish replay in xxx msec
        - master finish replay journal, can write now.
        - begin to generate new image: image.xxxx
        - start save image to /path/to/doris-meta/image/image.ckpt. is ckpt: true
        - finished save image /path/to/doris-meta/image/image.ckpt in xxx ms. checksum is xxxx
        - push image.xxx to other nodes. totally xx nodes, push succeeded xx nodes
        - QE service start
        - thrift server started

        以上日志不一定严格按照这个顺序，但基本类似。

3. 常见问题

    对于单节点 FE 的部署，启停通常不会遇到什么问题。如果有问题，请先参照相关 wiki，仔细核对你的操作步骤。

4.8.3.3.2 添加 FE

添加 FE 流程在弹性扩缩容 有详细介绍，不再赘述。这里主要说明一些注意事项，以及常见问题。

1. 注意事项

    - 在添加新的 FE 之前，一定先确保当前的 Master FE 运行正常（连接是否正常，JVM 是否正常，image 生成是否正常，bdbje 数据目录是否过大等等）
    - 第一次启动新的 FE，一定确保添加了 `--helper` 参数指向 Master FE。再次启动时可不用添加 `--helper` ↪ 。（如果指定了 `--helper`，FE 会直接询问 helper 节点自己的角色，如果没有指定，FE 会尝试从 `doris-meta/image/` 目录下的 ROLE 和 VERSION 文件中获取信息）。
    - 第一次启动新的 FE，一定确保这个 FE 的 `meta_dir` 已经创建、权限正确且为空。
    - 启动新的 FE，和执行 `ALTER SYSTEM ADD FOLLOWER/OBSERVER` 语句在元数据添加 FE，这两个操作的顺序没有先后要求。如果先启动了新的 FE，而没有执行语句，则新的 FE 日志中会一直滚动 `current node is not added to the group. please add it first.` 字样。当执行语句后，则会进入正常流程。
    - 请确保前一个 FE 添加成功后，再添加下一个 FE。
    - 建议直接连接到 MASTER FE 执行 `ALTER SYSTEM ADD FOLLOWER/OBSERVER` 语句。

2. 常见问题

    1. this node is DETACHED

        当第一次启动一个待添加的 FE 时，如果 Master FE 上的 doris-meta/bdb 中的数据很大，则可能在待添加的 FE 日志中看到 `this node is DETACHED.` 字样。这时，bdbje 正在复制数据，你可以看到待添加的 FE 的 bdb/ 目录正在变大。这个过程通常会在数分钟不等（取决于 bdbje 中的数据量）。之后，fe.log 中可能会有一些 bdbje 相关的错误堆栈信息。如果最终日志中显示 `QE service start` 和 `thrift server started`，则通常表示启动成功。可以通过 mysql-client 连接这个 FE 尝试操作。如果没有出现这些字样，则可能是 bdbje 复制日志超时等问题。这时，直接再次重启这个 FE，通常即可解决问题。

    2. 各种原因导致添加失败

        - 如果添加的是 OBSERVER，因为 OBSERVER 类型的 FE 不参与元数据的多数写，理论上可以随意启停。因此，对于添加 OBSERVER 失败的情况。可以直接杀死 OBSERVER FE 的进程，清空 OBSERVER 的元数据目录后，重新进行一遍添加流程。
        - 如果添加的是 FOLLOWER，因为 FOLLOWER 是参与元数据多数写的。所以有可能 FOLLOWER 已经加入 bdbje 选举组内。如果这时只有两个 FOLLOWER 节点（包括 MASTER），那么停掉一个 FE，可能导致另一个 FE 也因无法进行多数写而退出。此时，我们应该先通过 `ALTER SYSTEM DROP FOLLOWER` 命令，从元数据中删除新添加的 FOLLOWER 节点，然后再杀死 FOLLOWER 进程，清空元数据，重新进行一遍添加流程。

4.8.3.3.3 删除 FE

通过 `ALTER SYSTEM DROP FOLLOWER/OBSERVER` 命令即可删除对应类型的 FE。以下有几点注意事项：

- 对于 OBSERVER 类型的 FE，直接 DROP 即可，无风险。
- 对于 FOLLOWER 类型的 FE。首先，应保证在有奇数个 FOLLOWER 的情况下（3 个或以上），开始删除操作。

1. 如果删除非 MASTER 角色的 FE，建议连接到 MASTER FE，执行 DROP 命令，再杀死进程即可。
2. 如果要删除 MASTER FE，先确认有奇数个 FOLLOWER FE 并且运行正常。然后先杀死 MASTER FE 的进程。这时会有某一个 FE 被选举为 MASTER。在确认剩下的 FE 运行正常后，连接到新的 MASTER FE，执行 DROP 命令删除之前老的 MASTER FE 即可。

4.8.3.4 高级操作

4.8.3.4.1 FE 元数据恢复模式

元数据恢复模式使用不当或操作错误容易造成生产环境不可恢复的数据损坏，因此不再提供元数据恢复模式的操作文档；如果确有需求，请联系 Doris 社区的开发者

4.8.3.4.2 FE 类型变更

如果你需要将当前已有的 FOLLOWER/OBSERVER 类型的 FE，变更为 OBSERVER/FOLLOWER 类型，请先按照前面所述的方式删除 FE，再添加对应类型的 FE 即可

4.8.3.4.3 FE 迁移

如果你需要将一个 FE 从当前节点迁移到另一个节点，分以下几种情况。

1. 非 MASTER 节点的 FOLLOWER，或者 OBSERVER 迁移

   直接添加新的 FOLLOWER/OBSERVER 成功后，删除旧的 FOLLOWER/OBSERVER 即可。

2. 单节点 MASTER 迁移

   如果你是开发者，这可通过元数据恢复模式进行操作，如果你是使用者，不建议使用元数据恢复模式，建议通过重新搭建环境通过外表的方式转移数据

3. 一组 FOLLOWER 从一组节点迁移到另一组新的节点

   在新的节点上部署 FE，通过添加 FOLLOWER 的方式先加入新节点。再逐台 DROP 掉旧节点即可。在逐台 DROP 的过程中，MASTER 会自动选择在新的 FOLLOWER 节点上。

4.8.3.4.4 更换 FE 端口

FE 目前有以下几个端口

- edit_log_port：bdbje 的通信端口
- http_port：http 端口，也用于推送 image
- rpc_port：FE 的 thrift server port
- query_port：Mysql 连接端口
- arrow_flight_sql_port: Arrow Flight SQL 连接端口

1. edit_log_port

   如果需要更换这个端口，如果是多节点可按节点扩缩容的步骤下线旧节点，重新加入修改配置后的新节点；如果是单节点，参见 FE 迁移中“单节点 MASTER 迁移”

2. http_port

   所有 FE 的 http_port 必须保持一致。所以如果要修改这个端口，则所有 FE 都需要同时停机修改后并重启。

3. rpc_port

   修改配置后，直接重启 FE 即可。Master FE 会通过心跳将新的端口告知 BE。只有 Master FE 的这个端口会被使用。但仍然建议所有 FE 的端口保持一致。

4. query_port

   修改配置后，直接重启 FE 即可。这个只影响到 mysql 的连接目标。

5. arrow_flight_sql_port

   修改配置后，直接重启 FE 即可。这个只影响到 Arrow Flight SQL 的连接目标。

4.8.3.4.5 查看 BDBJE 中的数据 (仅用于调试)

FE 的元数据日志以 Key-Value 的方式存储在 BDBJE 中。某些异常情况下，可能因为元数据错误而无法启动 FE。在这种情况下，Doris 提供一种方式可以帮助用户查询 BDBJE 中存储的数据，以方便进行问题排查。

首先需在 fe.conf 中增加配置：`enable_bdbje_debug_mode=true`，之后通过 `bash start_fe.sh --daemon` 启动 FE。

此时，FE 将进入 debug 模式，仅会启动 http server 和 MySQL server，并打开 BDBJE 实例，但不会进行任何元数据的加载及后续其他启动流程。

这时，我们可以通过访问 FE 的 web 页面，或通过 MySQL 客户端连接到 Doris 后，通过 `show proc "/bdbje";` 来查看 BDBJE 中存储的数据。

```
mysql> show proc "/bdbje";
+----------+---------------+---------+
| DbNames  | JournalNumber | Comment |
+----------+---------------+---------+
| 110589   | 4273          |         |
| epochDB  | 4             |         |
| metricDB | 430694        |         |
+----------+---------------+---------+
```

第一级目录会展示 BDBJE 中所有的 database 名称，以及每个 database 中的 entry 数量。

```
mysql> show proc "/bdbje/110589";
+-----------+
| JournalId |
+-----------+
| 1         |
| 2         |

...
| 114858    |
| 114859    |
| 114860    |
| 114861    |
```

```
+-----------+
4273 rows in set (0.06 sec)
```

进入第二级，则会罗列指定 database 下的所有 entry 的 key。

```
mysql> show proc "/bdbje/110589/114861";
+-----------+--------------+--------------------------------------------+
| JournalId | OpType       | Data                                       |
+-----------+--------------+--------------------------------------------+
| 114861    | OP_HEARTBEAT | org.apache.doris.persist.HbPackage@6583d5fb |
+-----------+--------------+--------------------------------------------+
1 row in set (0.05 sec)
```

第三级则可以展示指定 key 的 value 信息。

4.8.3.5 最佳实践

FE 的部署推荐，在安装与部署文档中有介绍，这里再做一些补充。

- 如果你并不十分了解 FE 元数据的运行逻辑，或者没有足够 FE 元数据的运维经验，我们强烈建议在实际使用中，只部署一个 FOLLOWER 类型的 FE 作为 MASTER，其余 FE 都是 OBSERVER，这样可以减少很多复杂的运维问题！ 不用过于担心 MASTER 单点故障导致无法进行元数据写操作。首先，如果你配置合理，FE 作为 java 进程很难挂掉。其次，如果 MASTER 磁盘损坏（概率非常低），我们也可以用 OBSERVER 上的元数据，通过 元数据恢复模式 的方式手动恢复。
- FE 进程的 JVM 一定要保证足够的内存。我们强烈建议 FE 的 JVM 内存至少在 10GB 以上，推荐 32GB 至 64GB。并且部署监控来监控 JVM 的内存使用情况。因为如果 FE 出现 OOM，可能导致元数据写入失败，造成一些无法恢复的故障！
- FE 所在节点要有足够的磁盘空间，以防止元数据过大导致磁盘空间不足。同时 FE 日志也会占用十几 G 的磁盘空间。

4.8.3.6 其他常见问题

1. fe.log 中一直滚动 `meta out of date. current time: xxx, synchronized time: xxx, has log: xxx, fe type: xxx`

   这个通常是因为 FE 无法选举出 Master。比如配置了 3 个 FOLLOWER，但是只启动了一个 FOLLOWER，则这个 FOLLOWER 会出现这个问题。通常，只要同时重新启动所有 FOLLOWER 就可以了。如果启动起来后，仍然没有解决问题，那么可能需要进一步排查是否有其他未知问题。

2. `Clock delta: xxxx ms. between Feeder: xxxx and this Replica exceeds max permissible delta: xxxx ms.`

   bdbje 要求各个节点之间的时钟误差不能超过一定阈值。如果超过，节点会异常退出。我们默认设置的阈值为 5000 ms，由 FE 的参数 `max_bdbje_clock_delta_ms` 控制，可以酌情修改。但我们建议使用 ntp 等时钟同步方式保证 Doris 集群各主机的时钟同步。

3. image/ 目录下的镜像文件很久没有更新

   Master FE 会默认每 50000 条元数据 journal，生成一个镜像文件。在一个频繁使用的集群中，通常每隔半天到几天的时间，就会生成一个新的 image 文件。如果你发现 image 文件已经很久没有更新了（比如超过一个星期），则可以顺序的按照如下方法，查看具体原因：

   1. 在 Master FE 的 fe.log 中搜索 `memory is not enough to do checkpoint. Committed memory xxxx Bytes, used memory xxxx Bytes.` 字样。如果找到，则说明当前 FE 的 JVM 内存不足以用于生成镜像（通常我们需要预留一半的 FE 内存用于 image 的生成）。那么需要增加 JVM 的内存并重启 FE 后，再观察。每次 Master FE 重启后，都会直接生成一个新的 image。也可用这种重启方式，主动地生成新的 image。注意，如果是多 FOLLOWER 部署，那么当你重启当前 Master FE 后，另一个 FOLLOWER FE 会变成 MASTER，则后续的 image 生成会由新的 Master 负责。因此，你可能需要修改所有 FOLLOWER FE 的 JVM 内存配置。
   2. 在 Master FE 的 fe.log 中搜索 `begin to generate new image: image.xxxx`。如果找到，则说明开始生成 image 了。检查这个线程的后续日志，如果出现 `checkpoint finished save image.xxxx`，则说明 image 写入成功。如果出现 `Exception when generate new image file`，则生成失败，需要查看具体的错误信息。

4. bdb/ 目录的大小非常大，达到几个 G 或更多

   如果在排除无法生成新的 image 的错误后，bdb 目录在一段时间内依然很大。则可能是因为 Master FE 推送 image 不成功。可以在 Master FE 的 fe.log 中搜索 `push image.xxxx to other nodes. totally xx nodes, push succeeded yy nodes`。如果 yy 比 xx 小，则说明有的 FE 没有被推送成功。可以在 fe.log 中查看到具体的错误 `Exception when pushing image file. url = xxx`。

   同时，你也可以在 FE 的配置文件中添加配置：`edit_log_roll_num=xxxx`。该参数设定了每多少条元数据 journal，做一次 image。默认是 50000。可以适当改小这个数字，使得 image 更加频繁，从而加速删除旧的 journal。

5. FOLLOWER FE 接连挂掉

   因为 Doris 的元数据采用多数写策略，即一条元数据 journal 必须至少写入多数个 FOLLOWER FE 后（比如 3 个 FOLLOWER，必须写成功 2 个），才算成功。而如果写入失败，FE 进程会主动退出。那么假设有 A、B、C 三个 FOLLOWER，C 先挂掉，然后 B 再挂掉，那么 A 也会跟着挂掉。所以如 最佳实践 一节中所述，如果你没有丰富的元数据运维经验，不建议部署多 FOLLOWER。

6. fe.log 中出现 `get exception when try to close previously opened bdb database. ignore it`

   如果后面有 `ignore it` 字样，通常无需处理。如果你有兴趣，可以在 `BDBEnvironment.java` 搜索这个错误，查看相关注释说明。

7. 从 `show frontends;` 看，某个 FE 的 Join 列为 `true`，但是实际该 FE 不正常

   通过 `show frontends;` 查看到的 Join 信息。该列如果为 `true`，仅表示这个 FE 曾经加入过集群。并不能表示当前仍然正常的存在于集群中。如果为 `false`，则表示这个 FE 从未加入过集群。

8. 关于 FE 的配置 `master_sync_policy`, `replica_sync_policy` 和 `txn_rollback_limit`

   `master_sync_policy` 用于指定当 Leader FE 写元数据日志时，是否调用 fsync(), `replica_sync_policy` 用于指定当 FE HA 部署时，其他 Follower FE 在同步元数据时，是否调用 fsync()。在早期的 Doris 版本中，这两个参数默认是 `WRITE_NO_SYNC`，即都不调用 fsync()。在最新版本的 Doris 中，默认已修改为 `SYNC`，即都调用 fsync()。调用 fsync() 会显著降低元数据写盘的效率。在某些环境下，IOPS 可能降至几百，延迟增加到 2-3ms（但对于 Doris 元数据操作依然够用）。因此我们建议以下配置：

1. 对于单 Follower FE 部署，master_sync_policy 设置为 SYNC，防止 FE 系统宕机导致元数据丢失。
2. 对于多 Follower FE 部署，可以将 master_sync_policy 和 replica_sync_policy 设为 WRITE_NO_SYNC，因为我们认为多个系统同时宕机的概率非常低。

如果在单 Follower FE 部署中，master_sync_policy 设置为 WRITE_NO_SYNC，则可能出现 FE 系统宕机导致元数据丢失。这时如果有其他 Observer FE 尝试重启时，可能会报错：

```
Node xxx must rollback xx total commits(numPassedDurableCommits of which were durable) to the
    ↪  earliest point indicated by transaction xxxx in order to rejoin the replication
    ↪ group, but the transaction rollback limit of xxx prohibits this.
```

意思有部分已经持久化的事务需要回滚，但条数超过上限。这里我们的默认上限是 100，可以通过设置 `txn_
    ↪ rollback_limit` 改变。该操作仅用于尝试正常启动 FE，但已丢失的元数据无法恢复。

4.8.4 FE 锁管理

FE 锁管理用于检测 FE 进程中可能出现的死锁和慢锁问题。方便用户定位线上问题以及监控锁的占用情况。

> 该功能为实验功能，自 2.1.6 版本开始支持。

4.8.4.1 死锁检测

FE 锁管理模块提供了死锁检测功能，用于自动检测死锁，该功能默认关闭，可以通过配置参数开启。

如果开启死锁检测功能，则会周期性地检测死锁，默认周期为 5 分钟，我们也可以设置 deadlock_detection_
↪ interval_minute 参数，来调整检测周期。

我们会再日志中输出死锁检测的结果，如果检测到死锁，则会输出对应的告警日志。可以搜索关键字 Deadlocks detected 来查看是否存在死锁。

日志内容示例 (原始日志为一行文本)：

```
2024-08-15 12:55:46  [ pool-1-thread-1:1034 ] - [ WARN ]  Find dead lock, info ["Thread-0" prio=5
    ↪  Id=15 WAITING on java.util.concurrent.locks.ReentrantLock$NonfairSync@5b7e0fca owned by
    ↪ "Thread-1" Id=16
    at java.base@17.0.6/jdk.internal.misc.Unsafe.park(Native Method)
    -  waiting on java.util.concurrent.locks.ReentrantLock$NonfairSync@5b7e0fca
    at java.base@17.0.6/java.util.concurrent.locks.LockSupport.park(LockSupport.java:211)
    at java.base@17.0.6/java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(
        ↪ AbstractQueuedSynchronizer.java:715)
    at java.base@17.0.6/java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(
        ↪ AbstractQueuedSynchronizer.java:938)
    at java.base@17.0.6/java.util.concurrent.locks.ReentrantLock$Sync.lock(ReentrantLock.java
        ↪ :153)
```

```
    at java.base@17.0.6/java.util.concurrent.locks.ReentrantLock.lock(ReentrantLock.java:322)
    at app//org.example.lock.MonitoredReentrantLock.lock(MonitoredReentrantLock.java:22)
    at app//org.example.Main.lambda$testDeadLock$3(Main.java:79)
    ...

    Number of locked synchronizers = 1
    - java.util.concurrent.locks.ReentrantLock$NonfairSync@9abbac5

, "Thread-1" prio=5 Id=16 WAITING on java.util.concurrent.locks.ReentrantLock$NonfairSync@9abbac5
    ↪  owned by "Thread-0" Id=15
    at java.base@17.0.6/jdk.internal.misc.Unsafe.park(Native Method)
    -  waiting on java.util.concurrent.locks.ReentrantLock$NonfairSync@9abbac5
    at java.base@17.0.6/java.util.concurrent.locks.LockSupport.park(LockSupport.java:211)
    at java.base@17.0.6/java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(
        ↪ AbstractQueuedSynchronizer.java:715)
    at java.base@17.0.6/java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(
        ↪ AbstractQueuedSynchronizer.java:938)
    at java.base@17.0.6/java.util.concurrent.locks.ReentrantLock$Sync.lock(ReentrantLock.java
        ↪ :153)
    at java.base@17.0.6/java.util.concurrent.locks.ReentrantLock.lock(ReentrantLock.java:322)
    at app//org.example.lock.MonitoredReentrantLock.lock(MonitoredReentrantLock.java:22)
    at app//org.example.Main.lambda$testDeadLock$4(Main.java:93)
    ...

    Number of locked synchronizers = 1
    - java.util.concurrent.locks.ReentrantLock$NonfairSync@5b7e0fca

]
```

4.8.4.1.1 配置参数

fe.conf 配置文件中的死锁检测相关参数如下：

参数名	参数说明	默认值
enable_deadlock_detection	是否开启死锁检测功能	false
deadlock_detection_interval_minute	死锁检测周期，单位为分钟	5

4.8.4.2 慢锁检测

FE 锁管理模块提供了慢锁检测功能，我们会监控所有 DB、Table、Transaction 相关的锁，如果锁的持有时间超过一定阈值（默认为 10 秒），则会输出对应告警日志。

我们可以搜索日志关键字 `Lock held for` 来查看是否存在慢锁。

日志内容示例（原始日志为一行文本）：

```
2024-08-12 16:38:51,004 INFO (mysql-nio-pool-0|242) [StreamEncoder.writeBytes():234] 2024-08-12
    ↪ 16:38:51  [ mysql-nio-pool-0:47482 ] - [ WARN ]  Thread ID: 242, Thread Name: mysql-nio-
    ↪ pool-0 - Lock held for 1923 ms, exceeding hold timeout of 1923 ms Thread stack trace:
    ↪ at java.base/java.lang.Thread.getStackTrace(Thread.java:1610)
    at org.apache.doris.common.lock.AbstractMonitoredLock.afterUnlock(AbstractMonitoredLock.java
        ↪ :59)
    at org.apache.doris.common.lock.MonitoredReentrantLock.unlock(MonitoredReentrantLock.java:59)
    at org.apache.doris.datasource.InternalCatalog.unlock(InternalCatalog.java:370)
    at org.apache.doris.datasource.InternalCatalog.createDb(InternalCatalog.java:443)
    at org.apache.doris.catalog.Env.createDb(Env.java:3150)
    at org.apache.doris.qe.DdlExecutor.execute(DdlExecutor.java:168)
    at org.apache.doris.qe.StmtExecutor.handleDdlStmt(StmtExecutor.java:3066)
    at org.apache.doris.qe.StmtExecutor.executeByLegacy(StmtExecutor.java:1059)
    at org.apache.doris.qe.StmtExecutor.execute(StmtExecutor.java:644)
    at org.apache.doris.qe.StmtExecutor.queryRetry(StmtExecutor.java:562)
    at org.apache.doris.qe.StmtExecutor.execute(StmtExecutor.java:552)
    at org.apache.doris.qe.ConnectProcessor.executeQuery(ConnectProcessor.java:385)
    at org.apache.doris.qe.ConnectProcessor.handleQuery(ConnectProcessor.java:237)
    at org.apache.doris.qe.MysqlConnectProcessor.handleQuery(MysqlConnectProcessor.java:272)
    at org.apache.doris.qe.MysqlConnectProcessor.dispatch(MysqlConnectProcessor.java:300)
    at org.apache.doris.qe.MysqlConnectProcessor.processOnce(MysqlConnectProcessor.java:359)
    at org.apache.doris.mysql.ReadListener.lambda$handleEvent$0(ReadListener.java:52)
    at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1136)
    at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:635)
    at java.base/java.lang.Thread.run(Thread.java:833)
```

4.8.4.2.1 配置参数

fe.conf 配置文件中的慢锁检测相关参数如下：

参数名	参数说明	默认值
max_lock_hold_threshold_seconds	慢锁告警阈值，单位为秒	10

4.8.5 Tablet 本地调试

Doris 线上运行过程中，因为各种原因，可能出现各种各样的 bug。例如：副本不一致，数据存在版本 diff 等。这时候需要将线上的 tablet 的副本数据拷贝到本地环境进行复现，然后进行问题定位。

4.8.5.1 1. 获取有问题的 Tablet 的信息

可以通过 BE 日志确认 tablet id，然后通过以下命令获取信息（假设 tablet id 为 10020）。

获取 tablet 所在的 DbId/TableId/PartitionId 等信息。

```
mysql> show tablet 10020`\G`
*************************** 1. row ***************************
       DbName: default_cluster:db1
    TableName: tbl1
PartitionName: tbl1
    IndexName: tbl1
         DbId: 10004
      TableId: 10016
  PartitionId: 10015
      IndexId: 10017
       IsSync: true
        Order: 1
    DetailCmd: SHOW PROC '/dbs/10004/10016/partitions/10015/10017/10020';
```

执行上一步中的 DetailCmd 获取 BackendId/SchemaHash 等信息。

```
mysql>  SHOW PROC '/dbs/10004/10016/partitions/10015/10017/10020'`\G`
*************************** 1. row ***************************
        ReplicaId: 10021
        BackendId: 10003
          Version: 3
LstSuccessVersion: 3
 LstFailedVersion: -1
    LstFailedTime: NULL
       SchemaHash: 785778507
    LocalDataSize: 780
   RemoteDataSize: 0
         RowCount: 2
            State: NORMAL
            IsBad: false
     VersionCount: 3
         PathHash: 7390150550643804973
          MetaUrl: http://192.168.10.1:8040/api/meta/header/10020
 CompactionStatus: http://192.168.10.1:8040/api/compaction/show?tablet_id=10020
```

创建 tablet 快照并获取建表语句

```
mysql> admin copy tablet 10020 properties("backend_id" = "10003", "version" = "2")`\G`
*************************** 1. row ***************************
         TabletId: 10020
        BackendId: 10003
               Ip: 192.168.10.1
             Path: /path/to/be/storage/snapshot/20220830101353.2.3600
ExpirationMinutes: 60
  CreateTableStmt: CREATE TABLE `tbl1` (
  `k1` int(11) NULL,
```

```
  `k2` int(11) NULL
) ENGINE=OLAP
DUPLICATE KEY(`k1`, `k2`)
DISTRIBUTED BY HASH(k1) BUCKETS 1
PROPERTIES (
"replication_num" = "1",
"version_info" = "2"
);
```

admin copy tablet 命令可以为指定的 tablet 生成对应副本和版本的快照文件。快照文件存储在 Ip 字段所示节点的 Path 目录下。

该目下会有一个 tablet id 命名的目录，将这个目录整体打包后备用。(注意，该目录最多保留 60 分钟，之后会自动删除)。

```
cd /path/to/be/storage/snapshot/20220830101353.2.3600
tar czf 10020.tar.gz 10020/
```

该命令还会同时生成这个 tablet 对应的建表语句。注意，这个建表语句并不是原始的建表语句，他的分桶数和副本数都是 1，并且指定了 versionInfo 字段。该建表语句是用于之后在本地加载 tablet 时使用的。

至此，我们已经获取到所有必要的信息，清单如下：

1. 打包好的 tablet 数据，如 10020.tar.gz。
2. 建表语句。

4.8.5.2 2. 本地加载 Tablet

1. 搭建本地调试环境

   在本地部署一个单节点的 Doris 集群 (1FE、1BE)，部署版本和线上集群保持一致。如线上部署的版本是 DORIS-1.1.1, 本地环境也同样部署 DORIS-1.1.1 的版本。

2. 建表

   使用上一步中得到的建表语句，在本地环境中创建一张表。

3. 获取新建的表的 tablet 的信息

   因为新建表的分桶数和副本数都为 1，所以只会有一个一副本的 tablet：

```
mysql> show tablets from tbl1`\G`
*************************** 1. row ***************************
               TabletId: 10017
              ReplicaId: 10018
              BackendId: 10003
             SchemaHash: 44622287
                Version: 1
        LstSuccessVersion: 1
```

```
          LstFailedVersion: -1
             LstFailedTime: NULL
             LocalDataSize: 0
            RemoteDataSize: 0
                 RowCount: 0
                    State: NORMAL
    LstConsistencyCheckTime: NULL
              CheckVersion: -1
              VersionCount: -1
                  PathHash: 7390150550643804973
                   MetaUrl: http://192.168.10.1:8040/api/meta/header/10017
          CompactionStatus: http://192.168.10.1:8040/api/compaction/show?tablet_id=10017
```

```
    mysql> show tablet 10017`\G`
    *************************** 1. row ***************************
           DbName: default_cluster:db1
        TableName: tbl1
    PartitionName: tbl1
        IndexName: tbl1
             DbId: 10004
          TableId: 10015
      PartitionId: 10014
          IndexId: 10016
           IsSync: true
            Order: 0
        DetailCmd: SHOW PROC '/dbs/10004/10015/partitions/10014/10016/10017';
```

这里我们要记录如下信息：

* TableId

* PartitionId

* TabletId

* SchemaHash

同时，我们还需要到调试环境 BE 节点的数据目录下，确认新的 tablet 所在的 shard id：

```
    cd /path/to/storage/data/*/10017 && pwd
```

这个命令会进入 10017 这个 tablet 所在目录并展示路径。这里我们会看到类似如下的路径：

```
    /path/to/storage/data/0/10017
```

其中 `0` 既是 shard id。

4. 修改 Tablet 数据

   解压第一步中获取到的 tablet 数据包。编辑器打开其中的 10017.hdr.json 文件，并修改以下字段为上一步中获取到的信息：

```
  "table_id":10015
  "partition_id":10014
  "tablet_id":10017
  "schema_hash":44622287
  "shard_id":0
```

5. 加载新 tablet

   首先，停止调试环境的 BE 进程（./bin/stop_be.sh）。然后将 10017.hdr.json 文件同级目录所在的所有.dat 文件，拷贝到 /path/to/storage/data/0/10017/44622287 目录下。这个目录既是在第 3 步中，我们获取到的调试环境 tablet 所在目录。10017/44622287 分别是 tablet id 和 schema hash。

   通过 meta_tool 工具删除原来的 tablet meta。该工具位于 be/lib 目录下。

```
./lib/meta_tool --root_path=/path/to/storage --operation=delete_meta --tablet_id=10017 --
    ↪ schema_hash=44622287
```

其中 `/path/to/storage` 为 BE 的数据根目录。如删除成功，会出现 delete successfully 日志。

通过 `meta_tool` 工具加载新的 tablet meta。

```
./lib/meta_tool --root_path=/path/to/storage --operation=load_meta --json_meta_path=/path/to
    ↪ /10017.hdr.json
```

如加载成功，会出现 load successfully 日志。

6. 验证

   重新启动调试环境的 BE 进程 (./bin/start_be.sh)。对表进行查询，如果正确，则可以查询出加载的 tablet 的数据，或复现线上问题。

4.8.6 Tablet 元数据管理工具

4.8.6.1 背景

在最新版本的代码中，我们在 BE 端引入了 RocksDB，用于存储 tablet 的元信息，以解决之前通过 header 文件的方式存储元信息，带来的各种功能和性能方面的问题。当前每一个数据目录（root_path），都会有一个对应的 RocksDB 实例，其中以 key-value 的方式，存放对应 root_path 上的所有 tablet 的元数据。

为了方便进行这些元数据的维护，我们提供了在线的 http 接口方式和离线的 meta_tool 工具以完成相关的管理操作。

其中 http 接口仅用于在线的查看 tablet 的元数据，可以在 BE 进程运行的状态下使用。

而 meta_tool 工具则仅用于离线的各类元数据管理操作，必须先停止 BE 进程后，才可使用。

meta_tool 工具存放在 BE 的 lib/ 目录下。

4.8.6.2 操作

4.8.6.2.1 查看 Tablet Meta

查看 Tablet Meta 信息可以分为在线方法和离线方法

在线

访问 BE 的 http 接口，获取对应的 Tablet Meta 信息：

api：

```
http://{host}:{port}/api/meta/header/{tablet_id}
```

- host: BE 的 hostname
- port: BE 的 http 端口
- tablet_id: tablet id

举例：

```
http://be_host:8040/api/meta/header/14156
```

最终查询成功的话，会将 Tablet Meta 以 json 形式返回。

离线

基于 meta_tool 工具获取某个盘上的 Tablet Meta。

命令：

```
./lib/meta_tool --root_path=/path/to/root_path --operation=get_meta --tablet_id=xxx --schema_hash
    ↪ =xxx
```

root_path: 在 be.conf 中配置的对应的 root_path 路径。

结果也是按照 json 的格式展现 Tablet Meta。

4.8.6.2.2 加载 header

加载 header 的功能是为了完成实现 tablet 人工迁移而提供的。该功能是基于 json 格式的 Tablet Meta 实现的，所以如果涉及 shard 字段、 version 信息的更改，可以直接在 Tablet Meta 的 json 内容中更改。然后使用以下的命令进行加载。

命令：

```
./lib/meta_tool --operation=load_meta --root_path=/path/to/root_path --json_meta_path=path
```

4.8.6.2.3 删除 header

为了实现从某个 be 的某个盘中删除某个 tablet 元数据的功能。可以单独删除一个 tablet 的元数据，或者批量删除一组 tablet 的元数据。

删除单个 tablet 元数据：

```
./lib/meta_tool --operation=delete_meta --root_path=/path/to/root_path --tablet_id=xxx --schema_
    ↪ hash=xxx
```

删除一组 tablet 元数据：

```
./lib/meta_tool --operation=batch_delete_meta --tablet_file=/path/to/tablet_file.txt
```

其中 `tablet_file.txt` 中的每一行表示一个 tablet 的信息。格式为：

```
root_path,tablet_id,schema_hash
```

每一行各个列用逗号分隔。

`tablet_file` 文件示例：

```
/output/be/data/,14217,352781111
/output/be/data/,14219,352781111
/output/be/data/,14223,352781111
/output/be/data/,14227,352781111
/output/be/data/,14233,352781111
/output/be/data/,14239,352781111
```

批量删除会跳过 `tablet_file` 中 tablet 信息格式不正确的行。并在执行完成后，显示成功删除的数量和错误数量。

4.8.6.2.4 展示 pb 格式的 TabletMeta

这个命令是为了查看旧的基于文件的管理的 PB 格式的 Tablet Meta，以 json 的格式展示 Tablet Meta。

命令：

```
./lib/meta_tool --operation=show_meta --root_path=/path/to/root_path --pb_header_path=path
```

4.8.6.2.5 展示 pb 格式的 Segment meta

这个命令是为了查看 SegmentV2 的 segment meta 信息，以 json 形式展示出来

命令：

“‘shell ./meta_tool –operation=show_segment_footer –file=/path/to/segment/file

4.8.7 数据修复

对于 Unique Key Merge on Write 表，在某些 Doris 的版本中存在 Bug，可能会导致系统在计算 Delete Bitmap 时出现错误，导致出现重复主键，此时可以利用 Full Compaction 功能进行数据的修复。本功能对于非 Unique Key Merge on Write 表无效。

该功能需要 Doris 版本 2.0+。

使用该功能，需要尽可能停止导入，否则可能会出现导入超时等问题。

4.8.7.1 简要原理说明

执行 Full Compaction 后，会对 Delete Bitmap 进行重新计算，将错误的 Delete Bitmap 数据删除，以完成数据的修复。

4.8.7.2 使用说明

```
POST /api/compaction/run?tablet_id={int}&compact_type=full
```

或

```
POST /api/compaction/run?table_id={int}&compact_type=full
```

注意，`tablet_id` 和 `table_id` 只能指定一个，不能够同时指定，指定 `table_id` 后会自动对此 table 下所有 tablet 执行 `full_compaction`。

4.8.7.3 使用例子

```
curl -X POST "http://127.0.0.1:8040/api/compaction/run?tablet_id=10015&compact_type=full"
curl -X POST "http://127.0.0.1:8040/api/compaction/run?table_id=10104&compact_type=full"
```

4.9 OPEN API

4.9.1 总览

OPEN API 作为 Apache Doris 运维管理操作的补充，主要用于数据库管理人员进行一些管理操作。

> OPEN API 目前都是 unstable 的，仅建议开发人员测试和使用。我们可能会在后续版本对接口行为进行变更。在生产环境中，建议使用 SQL 命令完成操作。

4.9.1.1 安全认证

通过以下配置，可以开启 FE BE API 的安全认证：

配置名称	配置文件	默认值	说明
enable_all_http_auth	be.conf	false	开启 BE HTTP 端口（默认 8040）的认证。开启后，访问 BE 的 HTTP API 需要 ADMIN 用户登录。

配置名称	配置文件	默认值	说明
enable_brpc_builtin_services	be.conf	true	是否对外开启brpc内置服务(默认8060)。关闭后，将不能访问HTTP协议的8060端口。(自2.1.7版本支持)

配置名称	配置文件	默认值	说明
enable_all_http_auth	fe.conf	false	开启 FE HTTP 端口（默认 8030）的认证。开启后，访问 FE 的 HTTP API 需要对应的用户权限。

NOTE FE 和 BE 的 HTTP API 的权限要求，各个版本不尽相同，具体请参阅对应的 API 文档。

4.9.2 FE HTTP API

4.9.2.1 Config Action

4.9.2.1.1 Request

```
GET /rest/v1/config/fe/
```

4.9.2.1.2 Description

Config Action 用于获取当前 FE 的配置信息

4.9.2.1.3 Path parameters

无

4.9.2.1.4 Query parameters

- conf_item

  可选参数。返回 FE 的配置信息中的指定项。

4.9.2.1.5 Request body

无

4.9.2.1.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": ["Name", "Value"],
        "rows": [{
            "Value": "DAY",
            "Name": "sys_log_roll_interval"
        }, {
            "Value": "23",
            "Name": "consistency_check_start_time"
        }, {
            "Value": "4096",
            "Name": "max_mysql_service_task_threads_num"
        }, {
            "Value": "1000",
            "Name": "max_unfinished_load_job"
        }, {
            "Value": "100",
            "Name": "max_routine_load_job_num"
        }, {
            "Value": "SYNC",
```

```
            "Name": "master_sync_policy"
        }]
    },
    "count": 0
}
```

返回结果同 `System Action`。是一个表格的描述。

4.9.2.2 HA Action

4.9.2.2.1 Request

```
GET /rest/v1/ha
```

4.9.2.2.2 Description

HA Action 用于获取 FE 集群的高可用组信息。

4.9.2.2.3 Path parameters

无

4.9.2.2.4 Query parameters

无

4.9.2.2.5 Request body

无

4.9.2.2.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "Observernodes": [],
        "CurrentJournalId": [{
            "Value": 433648,
            "Name": "FrontendRole"
        }],
        "Electablenodes": [{
            "Value": "host1",
            "Name": "host1"
        }],
```

```
        "allowedFrontends": [{
            "Value": "name: 192.168.1.1_9213_1597652404352, role: FOLLOWER, 192.168.1.1:9213",
            "Name": "192.168.1.1_9213_1597652404352"
        }],
        "removedFrontends": [],
        "CanRead": [{
            "Value": true,
            "Name": "Status"
        }],
        "databaseNames": [{
            "Value": "433436 ",
            "Name": "DatabaseNames"
        }],
        "FrontendRole": [{
            "Value": "MASTER",
            "Name": "FrontendRole"
        }],
        "CheckpointInfo": [{
            "Value": 433435,
            "Name": "Version"
        }, {
            "Value": "2020-09-03T02:07:37.000+0000",
            "Name": "lastCheckPointTime"
        }]
    },
    "count": 0
}
```


4.9.2.3 Hardware Info Action

4.9.2.3.1 Request

```
GET /rest/v1/hardware_info/fe/
```

4.9.2.3.2 Description

Hardware Info Action 用于获取当前 FE 的硬件信息。

4.9.2.3.3 Path parameters

无

4.9.2.3.4 Query parameters

无

4.9.2.3.5 Request body

无

4.9.2.3.6 Response


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "VersionInfo": {
            "Git": "git://host/core@5bc28f4c36c20c7b424792df662fc988436e679e",
            "Version": "trunk",
            "BuildInfo": "cmy@192.168.1",
            "BuildTime": "二, 05 9月 2019 11:07:42 CST"
        },
        "HardwareInfo": {
            "NetworkParameter": "...",
            "Processor": "...",
            "OS": "...",
            "Memory": "...",
            "FileSystem": "...",
            "NetworkInterface": "...",
            "Processes": "...",
            "Disk": "..."
        }
    },
    "count": 0
}
```


- 其中 HardwareInfo 字段中的各个值的内容，都是以 html 格式展现的硬件信息文本。

4.9.2.4 Help Action

4.9.2.4.1 Request

```
GET /rest/v1/help
```

4.9.2.4.2 Description

用于通过模糊查询获取帮助。

4.9.2.4.3 Path parameters

无

4.9.2.4.4 Query parameters

- query

  需要进行匹配的关键词，如 array、 select 等。

4.9.2.4.5 Request body

无

4.9.2.4.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":{"fuzzy":"No Fuzzy Matching Topic","matching":"No Matching Category"},
    "count":0
}
```

4.9.2.5 Log Action

4.9.2.5.1 Request

```
GET /rest/v1/log
```

4.9.2.5.2 Description

GET 用于获取 Doris 最新的一部分 WARNING 日志，POST 方法用于动态设置 FE 的日志级别。

4.9.2.5.3 Path parameters

无

4.9.2.5.4 Query parameters

- add_verbose

  POST 方法可选参数。开启指定 Package 的 DEBUG 级别日志。

- del_verbose

  POST 方法可选参数。关闭指定 Package 的 DEBUG 级别日志。

4.9.2.5.5 Request body

无

4.9.2.5.6 Response

```
GET /rest/v1/log

{
    "msg": "success",
    "code": 0,
    "data": {
        "LogContents": {
            "logPath": "/home/disk1/cmy/git/doris/core-for-ui/output/fe/log/fe.warn.log",
            "log": "<pre>2020-08-26 15:54:30,081 WARN (UNKNOWN 10.81.85.89_9213_1597652404352(-1)
                ↪ |1) [Catalog.notifyNewFETypeTransfer():2356] notify new FE type transfer:
                ↪ UNKNOWN</br>2020-08-26 15:54:32,089 WARN (RepNode 10.81.85.89_9213_
                ↪ 1597652404352(-1)|61) [Catalog.notifyNewFETypeTransfer():2356] notify new FE
                ↪ type transfer: MASTER</br>2020-08-26 15:54:35,121 WARN (stateListener|73) [
                ↪ Catalog.replayJournal():2510] replay journal cost too much time: 2975
                ↪ replayedJournalId: 232383</br>2020-08-26 15:54:48,117 WARN (
                ↪ leaderCheckpointer|75) [Catalog.replayJournal():2510] replay journal cost too
                ↪  much time: 2812 replayedJournalId: 232383</br></pre>",
            "showingLast": "603 bytes of log"
        },
        "LogConfiguration": {
            "VerboseNames": "org",
            "AuditNames": "slow_query,query",
            "Level": "INFO"
        }
    },
    "count": 0
}
```

其中 `data.LogContents.log` 表示最新一部分 `fe.warn.log` 中的日志内容。

```
POST /rest/v1/log?add_verbose=org

{
    "msg": "success",
    "code": 0,
    "data": {
        "LogConfiguration": {
            "VerboseNames": "org",
            "AuditNames": "slow_query,query",
            "Level": "INFO"
        }
    },
    "count": 0
}
```

4.9.2.6 Login Action

4.9.2.6.1 Request

`POST /rest/v1/login`

4.9.2.6.2 Description

用于登录服务。

4.9.2.6.3 Path parameters

无

4.9.2.6.4 Query parameters

无

4.9.2.6.5 Request body

无

4.9.2.6.6 Response

- 登录成功

```
{
    "msg": "Login success!",
    "code": 200
}
```

- 登录失败

```
{
    "msg": "Error msg...",
    "code": xxx,
    "data": "Error data...",
    "count": 0
}
```

4.9.2.7 Logout Action

4.9.2.7.1 Request

```
POST /rest/v1/logout
```

4.9.2.7.2 Description

Logout Action 用于退出当前登录。

4.9.2.7.3 Path parameters

无

4.9.2.7.4 Query parameters

无

4.9.2.7.5 Request body

无

Response

```
{
    "msg": "OK",
    "code": 0
}
```

4.9.2.8 Query Profile Action

4.9.2.8.1 Request

```
GET /rest/v1/query_profile/<query_id>
```

4.9.2.8.2 Description

Query Profile Action 用于获取 Query 的 profile

4.9.2.8.3 Path parameters

- <query_id>

  可选参数。当不指定时，返回最新的 query 列表。当指定时，返回指定 query 的 profile。

4.9.2.8.4 Query parameters

无

4.9.2.8.5 Request body

无

4.9.2.8.6 Response

- Not specify <query_id>

```
GET /rest/v1/query_profile/
{
    "msg": "success",
    "code": 0,
    "data": {
        "href_column": ["Query ID"],
        "column_names": ["Query ID", "User", "Default Db", "Sql Statement", "Query Type", "
            ↪ Start Time", "End Time", "Total", "Query State"],
        "rows": [{
            "User": "root",
            "__hrefPath": ["/query_profile/d73a8a0b004f4b2f-b4829306441913da"],
            "Query Type": "Query",
            "Total": "5ms",
            "Default Db": "default_cluster:db1",
            "Sql Statement": "select * from tbl1",
            "Query ID": "d73a8a0b004f4b2f-b4829306441913da",
            "Start Time": "2020-09-03 10:07:54",
            "Query State": "EOF",
            "End Time": "2020-09-03 10:07:54"
        }, {
            "User": "root",
            "__hrefPath": ["/query_profile/fd706dd066824c21-9d1a63af9f5cb50c"],
            "Query Type": "Query",
            "Total": "6ms",
            "Default Db": "default_cluster:db1",
            "Sql Statement": "select * from tbl1",
            "Query ID": "fd706dd066824c21-9d1a63af9f5cb50c",
            "Start Time": "2020-09-03 10:07:54",
            "Query State": "EOF",
            "End Time": "2020-09-03 10:07:54"
        }]
    },
    "count": 3
}
```

```
The returned result is the same as `System Action`, which is a table description.
```

- Specify <query_id>

```
GET /rest/v1/query_profile/<query_id>

{
    "msg": "success",
    "code": 0,
    "data": "Query:</br>    Summary:</br>...",
    "count": 0
}
```

```
`data` is the text content of the profile.
```

4.9.2.9 Session Action

4.9.2.9.1 Request

```
GET /rest/v1/session

GET /rest/v1/session/all
```

4.9.2.9.2 Description

Session Action 用于获取当前的会话信息。

4.9.2.9.3 Path parameters

无

4.9.2.9.4 Query parameters

无

4.9.2.9.5 Request body

无

4.9.2.9.6 获取当前 FE 的会话信息

```
GET /rest/v1/session
```

4.9.2.9.7 Response


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": ["Id", "User", "Host", "Cluster", "Db", "Command", "Time", "State", "Info
            ↪ "],
        "rows": [{
            "User": "root",
            "Command": "Sleep",
            "State": "",
            "Cluster": "default_cluster",
            "Host": "10.81.85.89:31465",
            "Time": "230",
            "Id": "0",
            "Info": "",
            "Db": "db1"
        }]
    },
    "count": 2
}
```


4.9.2.9.8 获取所有 FE 的会话信息

```
GET /rest/v1/session/all
```

4.9.2.9.9 Response


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": ["FE", "Id", "User", "Host", "Cluster", "Db", "Command", "Time", "State",
            ↪  "Info"],
        "rows": [{
            "FE": "10.14.170.23",
            "User": "root",
            "Command": "Sleep",
            "State": "",
            "Cluster": "default_cluster",
            "Host": "10.81.85.89:31465",
            "Time": "230",
            "Id": "0",
            "Info": "",
            "Db": "db1"
```

```
        },
        {
            "FE": "10.14.170.24",
            "User": "root",
            "Command": "Sleep",
            "State": "",
            "Cluster": "default_cluster",
            "Host": "10.81.85.88:61465",
            "Time": "460",
            "Id": "1",
            "Info": "",
            "Db": "db1"
        }]
    },
    "count": 2
}
```

返回结果同 `System Action`。是一个表格的描述。

4.9.2.10 System Action

4.9.2.10.1 Request

```
GET /rest/v1/system
```

4.9.2.10.2 Description

System Action 用于 Doris 内置的 Proc 系统的相关信息。

4.9.2.10.3 Path parameters

无

4.9.2.10.4 Query parameters

- `path`

  可选参数，指定 proc 的 path

4.9.2.10.5 Request body

无

4.9.2.10.6 Response

以 /dbs/10003/10054/partitions/10053/10055 为例：


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "href_columns": ["TabletId", "MetaUrl", "CompactionStatus"],
        "column_names": ["TabletId", "ReplicaId", "BackendId", "SchemaHash", "Version", "
            ↪ VersionHash", "LstSuccessVersion", "LstSuccessVersionHash", "LstFailedVersion", "
            ↪ LstFailedVersionHash", "LstFailedTime", "DataSize", "RowCount", "State", "
            ↪ LstConsistencyCheckTime", "CheckVersion", "CheckVersionHash", "VersionCount", "
            ↪ PathHash", "MetaUrl", "CompactionStatus"],
        "rows": [{
            "SchemaHash": "1294206575",
            "LstFailedTime": "\\N",
            "LstFailedVersion": "-1",
            "MetaUrl": "URL",
            "__hrefPaths": ["http://192.168.100.100:8030/rest/v1/system?path=/dbs/10003/10054/
                ↪ partitions/10053/10055/10056", "http://192.168.100.100:8043/api/meta/header
                ↪ /10056", "http://192.168.100.100:8043/api/compaction/show?tablet_id=10056"],
            "CheckVersionHash": "-1",
            "ReplicaId": "10057",
            "VersionHash": "4611804212003004639",
            "LstConsistencyCheckTime": "\\N",
            "LstSuccessVersionHash": "4611804212003004639",
            "CheckVersion": "-1",
            "Version": "6",
            "VersionCount": "2",
            "State": "NORMAL",
            "BackendId": "10032",
            "DataSize": "776",
            "LstFailedVersionHash": "0",
            "LstSuccessVersion": "6",
            "CompactionStatus": "URL",
            "TabletId": "10056",
            "PathHash": "-3259732870068082628",
            "RowCount": "21"
        }]
    },
    "count": 1
}
```


其中 data 部分的 column_names 是表头信息，href_columns 表示表中的哪些列是超链接列。rows 数组中的每个元素表示一行。其中 __hrefPaths 不是表数据，而是超链接列的链接 URL，和 href_columns 中的列一一对应。

4.9.2.11 Colocate Meta Action

4.9.2.11.1 Request

`GET /api/colocate` `POST/DELETE /api/colocate/group_stable` `POST /api/colocate/bucketseq`

4.9.2.11.2 Description

获取/修改 colocate group 信息。

4.9.2.11.3 Path parameters

无

4.9.2.11.4 Query parameters

- `db_id`

  指定数据库 id

- `group_id`

  指定组 id

4.9.2.11.5 Request body

无

4.9.2.11.6 Response

TO DO

4.9.2.12 Meta Action

```
GET /image
GET /info
GET /version
GET /put
GET /journal_id
GET /role
GET /check
GET /dump
```

4.9.2.12.1 Description

这是一组 FE 元数据相关的 API，除了 /dump 以外，都为 FE 节点之间内部通讯用。

4.9.2.12.2 Path parameters

TODO

4.9.2.12.3 Query parameters

TODO

4.9.2.12.4 Request body

TODO

4.9.2.12.5 Response

TODO

4.9.2.13 Cluster Action

4.9.2.13.1 Request

```
GET /rest/v2/manager/cluster/cluster_info/conn_info
```

4.9.2.13.2 集群连接信息

```
GET /rest/v2/manager/cluster/cluster_info/conn_info
```

Description

用于获取集群 http、 mysql 连接信息。

4.9.2.13.3 Path parameters

无

4.9.2.13.4 Query parameters

无

4.9.2.13.5 Request body

无

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "http": [
```

```
            "fe_host:http_ip"
        ],
        "mysql": [
            "fe_host:query_ip"
        ]
    },
    "count": 0
}
```

Examples

```
GET /rest/v2/manager/cluster/cluster_info/conn_info

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "http": [
            "127.0.0.1:8030"
        ],
        "mysql": [
            "127.0.0.1:9030"
        ]
    },
    "count": 0
}
```

4.9.2.14 Node Action

4.9.2.14.1 Request

```
GET /rest/v2/manager/node/frontends

GET /rest/v2/manager/node/backends

GET /rest/v2/manager/node/brokers

GET /rest/v2/manager/node/configuration_name

GET /rest/v2/manager/node/node_list

POST /rest/v2/manager/node/configuration_info

POST /rest/v2/manager/node/set_config/fe

POST /rest/v2/manager/node/set_config/be

POST /rest/v2/manager/node/{action}/be
```

POST /rest/v2/manager/node/{action}/fe

POST /rest/v2/manager/node/{action}/broker (3.0.7+)

4.9.2.14.2 获取 fe, be, broker 节点信息

GET /rest/v2/manager/node/frontends

GET /rest/v2/manager/node/backends

GET /rest/v2/manager/node/brokers

Description

用于获取集群获取 fe, be, broker 节点信息。

Response

```
frontends:
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
            "Name",
            "IP",
            "HostName",
            "EditLogPort",
            "HttpPort",
            "QueryPort",
            "RpcPort",
            "ArrowFlightSqlPort",
            "Role",
            "IsMaster",
            "ClusterId",
            "Join",
            "Alive",
            "ReplayedJournalId",
            "LastHeartbeat",
            "IsHelper",
            "ErrMsg",
            "Version"
        ],
        "rows": [
            [
                ...
            ]
        ]
    },
    "count": 0
```

```
}
```

```
backends:
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
            "BackendId",
            "Cluster",
            "IP",
            "HostName",
            "HeartbeatPort",
            "BePort",
            "HttpPort",
            "BrpcPort",
            "LastStartTime",
            "LastHeartbeat",
            "Alive",
            "SystemDecommissioned",
            "ClusterDecommissioned",
            "TabletNum",
            "DataUsedCapacity",
            "AvailCapacity",
            "TotalCapacity",
            "UsedPct",
            "MaxDiskUsedPct",
            "ErrMsg",
            "Version",
            "Status"
        ],
        "rows": [
            [
                ...
            ]
        ]
    },
    "count": 0
}
```

```
brokers:
{
    "msg": "success",
    "code": 0,
    "data": {
```

```
        "column_names": [
            "Name",
            "IP",
            "HostName",
            "Port",
            "Alive",
            "LastStartTime",
            "LastUpdateTime",
            "ErrMsg"
        ],
        "rows": [
            [
                ...
            ]
        ]
    },
    "count": 0
}
```

4.9.2.14.3 获取节点配置信息

`GET /rest/v2/manager/node/configuration_name`

`GET /rest/v2/manager/node/node_list`

`POST /rest/v2/manager/node/configuration_info`

Description

- `configuration_name` 用于获取节点配置项名称。
- `node_list` 用于获取节点列表。
- `configuration_info` 用于获取节点配置详细信息。

Query parameters

`GET /rest/v2/manager/node/configuration_name`

无

`GET /rest/v2/manager/node/node_list`

无

`POST /rest/v2/manager/node/configuration_info`

- type

值为 fe 或 be，用于指定获取 fe 的配置信息或 be 的配置信息。

Request body

GET /rest/v2/manager/node/configuration_name

无

GET /rest/v2/manager/node/node_list

无

POST /rest/v2/manager/node/configuration_info

```
{
    "conf_name": [
        ""
    ],
    "node": [
        ""
    ]
}
```

若不带body，body中的参数都使用默认值。

conf_name 用于指定返回哪些配置项的信息， 默认返回所有配置项信息；

node 用于指定返回哪些节点的配置项信息，默认为全部fe节点或be节点配置项信息。

Response

GET /rest/v2/manager/node/configuration_name

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "backend":[
            ""
        ],
        "frontend":[
            ""
        ]
    },
    "count": 0
}
```

GET /rest/v2/manager/node/node_list

```
{
    "msg": "success",
```

```
    "code": 0,
    "data": {
        "backend": [
            ""
        ],
        "frontend": [
            ""
        ]
    },
    "count": 0
}
```


POST /rest/v2/manager/node/configuration_info?type=fe


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
            "配置项",
            "节点",
            "节点类型",
            "配置值类型",
            "MasterOnly",
            "配置值",
            "可修改"
        ],
        "rows": [
            [
                ""
            ]
        ]
    },
    "count": 0
}
```


POST /rest/v2/manager/node/configuration_info?type=be


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
            "配置项",
            "节点",
            "节点类型",
```

```
            "配置值类型",
            "配置值",
            "可修改"
        ],
        "rows": [
            [
                ""
            ]
        ]
    },
    "count": 0
}
```

Examples

1. 获取 fe agent_task_resend_wait_time_ms 配置项信息：

   POST /rest/v2/manager/node/configuration_info?type=fe

   Body:

```
    {
        "conf_name":[
            "agent_task_resend_wait_time_ms"
        ]
    }
```

```
Response:
```

```
    {
        "msg": "success",
        "code": 0,
        "data": {
            "column_names": [
                "配置项",
                "节点",
                "节点类型",
                "配置值类型",
                "MasterOnly",
                "配置值",
                "可修改"
            ],
            "rows": [
                [
                    "agent_task_resend_wait_time_ms",
                    "127.0.0.1:8030",
```

```
                    "FE",
                    "long",
                    "true",
                    "50000",
                    "true"
                ]
            ]
        },
        "count": 0
    }
```

4.9.2.14.4 修改配置值

`POST /rest/v2/manager/node/set_config/fe`

`POST /rest/v2/manager/node/set_config/be`

Description

用于修改 fe 或 be 节点配置值

Request body

```
{
    "config_name":{
        "node":[
            ""
        ],
        "value":"",
        "persist":
    }
}
```

- `config_name` 为对应的配置项；
- node 为关键字，表示要修改的节点列表;
- `value` 为配置的值；
- `persist` 为 true 表示永久修改，false 表示临时修改。永久修改重启后能生效，临时修改重启后失效。

Response

`GET /rest/v2/manager/node/configuration_name`

```
{
    "msg": "",
    "code": 0,
```

```
    "data": {
        "failed":[
            {
                "config_name":"name",
                "value"="",
                "node":"",
                "err_info":""
            }
        ]
    },
    "count": 0
}
```

- `failed` 表示修改失败的配置信息。

Examples

1. 修改 fe 127.0.0.1:8030 节点中 `agent_task_resend_wait_time_ms` 和 `alter_table_timeout_second` 配置值：

   `POST /rest/v2/manager/node/set_config/fe`

   Body:

```
    {
        "agent_task_resend_wait_time_ms":{
            "node":[
                "127.0.0.1:8030"
            ],
            "value":"10000",
            "persist":"true"
        },
        "alter_table_timeout_second":{
            "node":[
                "127.0.0.1:8030"
            ],
            "value":"true",
            "persist":"true"
        }
    }
```

```
Response:
```

```
    {
        "msg": "success",
        "code": 0,
        "data": {
```

```
            "failed": [
                {
                    "config_name": "alter_table_timeout_second",
                    "node": "10.81.85.89:8837",
                    "err_info": "Unsupported configuration value type.",
                    "value": "true"
                }
            ]
        },
        "count": 0
    }
```

`agent_task_resend_wait_time_ms` 配置值修改成功，`alter_table_timeout_second` 修改失败。

4.9.2.14.5 操作 be 节点

`POST /rest/v2/manager/node/{action}/be`

Description

用于添加/删除/下线 be 节点

action：ADD/DROP/DECOMMISSION

Request body

```
{
    "hostPorts": ["127.0.0.1:9050"],
    "properties": {
        "tag.location": "test"
    }
}
```

- `hostPorts` 需要操作的一组 be 节点地址 `ip:heartbeat_port`
- `properties` 添加节点时传入的配置，目前只用于配置 tag, 不传使用默认 tag

Response

```
{
    "msg": "Error",
    "code": 1,
    "data": "errCode = 2, detailMessage = Same backend already exists[127.0.0.1:9050]",
    "count": 0
}
```

Examples

1. 添加 be 节点

POST /rest/v2/manager/node/ADD/be

Request

```
{
    "hostPorts": ["127.0.0.1:9050"]
}
```

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

2. 删除 be 节点

POST /rest/v2/manager/node/DROP/be

Request

```
{
    "hostPorts": ["127.0.0.1:9050"]
}
```

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

3. 下线 be 节点

POST /rest/v2/manager/node/DECOMMISSION/be

Request

```
{
    "hostPorts": ["127.0.0.1:9050"]
}
```

Response

```
  {
      "msg": "success",
      "code": 0,
      "data": null,
      "count": 0
  }
```

4.9.2.14.6 操作 fe 节点

`POST /rest/v2/manager/node/{action}/fe`

Description

用于添加/删除 fe 节点

action：ADD/DROP

Request body

```
{
    "role": "FOLLOWER",
    "hostPort": "127.0.0.1:9030"
}

role FOLLOWER/OBSERVER
hostPort 需要操作的 fe 节点地址 ip:edit_log_port
```

Response

```
{
    "msg": "Error",
    "code": 1,
    "data": "errCode = 2, detailMessage = frontend already exists name: 127.0.0.1:9030_
        ↪ 1670495889415, role: FOLLOWER, 127.0.0.1:9030",
    "count": 0
}
```

Examples

1. 添加 FOLLOWER 节点

   `POST /rest/v2/manager/node/ADD/fe`

   Request

   ```
   {
       "role": "FOLLOWER",
       "hostPort": "127.0.0.1:9030"
   }
   ```

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

2. 删除 FOLLOWER 节点

```
POST /rest/v2/manager/node/DROP/fe
```

Request

```
{
    "role": "FOLLOWER",
    "hostPort": "127.0.0.1:9030"
}
```

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

4.9.2.14.7 操作 broker 节点

```
POST /rest/v2/manager/node/{action}/broker
```

自 3.0.7 支持。

Description

用于添加/删除 broker 节点

action：ADD/DROP/DROP_ALL

Request body

```
{
    "brokerName": "your_broker_name",
    "hostPortList": "broker_ip:broker_port"
}
```

Response

```
{
    "msg": "Error",
    "code": 1,
    "data": "errCode = 2, detailMessage = xxxx",
    "count": 0
}
```

Examples

1. 添加 BROKER 节点

   POST /rest/v2/manager/node/ADD/broker

   Request

```
    {
        "brokerName": "hdfs_broker",
        "hostPortList": "127.0.0.1:8001"
    }
```

```
Response
```

```
    {
        "msg": "success",
        "code": 0,
        "data": null,
        "count": 0
    }
```

2. 删除 BROKER 节点

```
POST /rest/v2/manager/node/DROP/broker
```

Request

```
   {
       "brokerName": "hdfs_broker",
       "hostPortList": "127.0.0.1:8001"
   }
```

Response

```
   {
       "msg": "success",
       "code": 0,
       "data": null,
       "count": 0
   }
```

3. 删除一组 BROKER 节点

```
POST /rest/v2/manager/node/DROP_ALL/broker
```

Request

```
{
    "brokerName": "hdfs_broker",
    "hostPortList": ""
}
```

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

4.9.2.15 Query Profile Action

4.9.2.15.1 Request

```
GET /rest/v2/manager/query/query_info

GET /rest/v2/manager/query/trace/{trace_id}

GET /rest/v2/manager/query/sql/{query_id}

GET /rest/v2/manager/query/profile/text/{query_id}

GET /rest/v2/manager/query/profile/graph/{query_id}

GET /rest/v2/manager/query/profile/json/{query_id}

GET /rest/v2/manager/query/profile/fragments/{query_id}

GET /rest/v2/manager/query/current_queries

GET /rest/v2/manager/query/kill/{query_id}
```

4.9.2.15.2 获取查询信息

```
GET /rest/v2/manager/query/query_info
```

Description

可获取集群所有 fe 节点 select 查询信息。

Query parameters

- query_id

  可选，指定返回查询的 queryID，默认返回所有查询的信息。

- search

  可选，指定返回包含字符串的查询信息，目前仅进行字符串匹配。

- is_all_node

  可选，若为 true 则返回所有 fe 节点的查询信息，若为 false 则返回当前 fe 节点的查询信息。默认为 true。

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
            "Query ID",
            "FE 节点",
            "查询用户",
            "执行数据库",
            "Sql",
            "查询类型",
            "开始时间",
            "结束时间",
            "执行时长",
            "状态"
        ],
        "rows": [
            [
                ...
            ]
        ]
    },
    "count": 0
}
```

Admin 和 Root 用户可以查看所有 Query。普通用户仅能查看自己发送的 Query。

Examples

```
GET /rest/v2/manager/query/query_info

{
    "msg": "success",
    "code": 0,
    "data": {
        "column_names": [
```

```
            "Query ID",
            "FE 节点",
            "查询用户",
            "执行数据库",
            "Sql",
            "查询类型",
            "开始时间",
            "结束时间",
            "执行时长",
            "状态"
        ],
        "rows": [
            [
                "d7c93d9275334c35-9e6ac5f295a7134b",
                "127.0.0.1:8030",
                "root",
                "default_cluster:testdb",
                "select c.id, c.name, p.age, p.phone, c.date, c.cost from cost c join people p on
                    ↪  c.id = p.id where p.age > 20 order by c.id",
                "Query",
                "2021-07-29 16:59:12",
                "2021-07-29 16:59:12",
                "109ms",
                "EOF"
            ]
        ]
    },
    "count": 0
}
```

4.9.2.15.3 通过 Trace Id 获取 Query Id

`GET /rest/v2/manager/query/trace_id/{trace_id}`

Description

通过 Trace Id 获取 Query Id.

在执行一个 Query 前，先设置一个唯一的 trace id:

`set session_context="trace_id:your_trace_id";`

在同一个 Session 链接内执行 Query 后，可以通过 trace id 获取 query id。

Path parameters

- `{trace_id}`

  用户设置的 trace id.

Query parameters

Response

```
{
    "msg": "success",
    "code": 0,
    "data": "fb1d9737de914af1-a498d5c5dec638d3",
    "count": 0
}
```

Admin 和 Root 用户可以查看所有 Query。普通用户仅能查看自己发送的 Query。若指定 trace id 不存在或无权限，则返回 Bad Request：

```
{
    "msg": "Bad Request",
    "code": 403,
    "data": "error messages",
    "count": 0
}
```

4.9.2.15.4 获取指定查询的 sql 和文本 profile

`GET /rest/v2/manager/query/sql/{query_id}`

`GET /rest/v2/manager/query/profile/text/{query_id}`

Description

用于获取指定 Query ID 的 SQL 和 profile 文本。

Path parameters

- `query_id`

  query id.

Query parameters

- `is_all_node`

  可选，若为 true 则在所有 FE 节点中查询指定 query id 的信息，若为 false 则在当前连接的 FE 节点中查询指定 query id 的信息。默认为 true。

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "sql": ""
```

```
    },
    "count": 0
}
```

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "profile": ""
    },
    "count": 0
}
```

Admin 和 Root 用户可以查看所有 Query。普通用户仅能查看自己发送的 Query。若指定 query id 不存在或无权限，则返回 Bad Request：

```
{
    "msg": "Bad Request",
    "code": 403,
    "data": "error messages",
    "count": 0
}
```

Examples

1. 获取 sql：

```
    GET /rest/v2/manager/query/sql/d7c93d9275334c35-9e6ac5f295a7134b

    Response:
    {
        "msg": "success",
        "code": 0,
        "data": {
            "sql": "select c.id, c.name, p.age, p.phone, c.date, c.cost from cost c join people p
                ↪  on c.id   = p.id where p.age > 20 order by c.id"
        },
        "count": 0
    }
```

4.9.2.15.5 获取指定查询 fragment 和 instance 信息

```
GET /rest/v2/manager/query/profile/fragments/{query_id}
```

自 2.1.1 起，此接口被弃用。你仍然可以从 http://:/QueryProfile 上下载 profile 文件。

Description

用于获取指定 query id 的 fragment 名称，instance id、主机 IP 及端口和执行时长。

Path parameters

- `query_id`

  query id.

Query parameters

- `is_all_node`

  可选，若为 true 则在所有 fe 节点中查询指定 query id 的信息，若为 false 则在当前连接的 fe 节点中查询指定 query id 的信息。默认为 true。

Response

```
{
    "msg": "success",
    "code": 0,
    "data": [
        {
            "fragment_id": "",
            "time": "",
            "instance_id": {
                "": {
                  "host": "",
                  "active_time": ""
                }
            }
        }
    ],
    "count": 0
}
```

Admin 和 Root 用户可以查看所有 Query。普通用户仅能查看自己发送的 Query。若指定 query id 不存在或无权限，则返回 Bad Request：

```
{
    "msg": "Bad Request",
    "code": 403,
```

```
    "data": "error messages",
    "count": 0
}
```

Examples

```
GET /rest/v2/manager/query/profile/fragments/d7c93d9275334c35-9e6ac5f295a7134b

Response:
{
    "msg": "success",
    "code": 0,
    "data": [
        {
            "fragment_id": "0",
            "time": "36.169ms",
            "instance_id": {
                "d7c93d9275334c35-9e6ac5f295a7134e": {
                    "host": "172.19.0.4:9060",
                    "active_time": "36.169ms"
                }
            }
        },
        {
            "fragment_id": "1",
            "time": "20.710ms",
            "instance_id": {
                "d7c93d9275334c35-9e6ac5f295a7134c": {
                    "host": "172.19.0.5:9060",
                    "active_time": "20.710ms"
                }
            }
        },
        {
            "fragment_id": "2",
            "time": "7.83ms",
            "instance_id": {
                "d7c93d9275334c35-9e6ac5f295a7134d": {
                    "host": "172.19.0.6:9060",
                    "active_time": "7.83ms"
                },
                "d7c93d9275334c35-9e6ac5f295a7134f": {
                    "host": "172.19.0.7:9060",
                    "active_time": "10.873ms"
                }
            }
```

```
        }
    ],
    "count": 0
}
```

4.9.2.15.6 获取指定 query id 树状 profile 信息

`GET /rest/v2/manager/query/profile/graph/{query_id}`

Description

获取指定 query id 树状 profile 信息，同 `show query profile` 指令。

Path parameters

- `query_id`

  query id.

Query parameters

- `fragment_id` 和 `instance_id`

  可选，这两个参数需同时指定或同时不指定。
  同时不指定则返回 profile 简易树形图，相当于`show query profile '/query_id';`
  同时指定则返回指定 instance 详细 profile 树形图，相当于`show query profile '/query_id/fragment_id/
  ↪ instance_id'`.

- `is_all_node`

  可选，若为 true 则在所有 fe 节点中查询指定 query id 的信息，若为 false 则在当前连接的 fe 节点中查询指定 query id 的信息。默认为 true。

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "graph":""
    },
    "count": 0
}
```

Admin 和 Root 用户可以查看所有 Query。普通用户仅能查看自己发送的 Query。若指定 query id 不存在或无权限，则返回 Bad Request：

```
{
    "msg": "Bad Request",
    "code": 403,
```

```
    "data": "error messages",
    "count": 0
}
```

4.9.2.15.7 正在执行的 query

`GET /rest/v2/manager/query/current_queries`

Description

同 `show proc "/current_query_stmts"`，返回当前正在执行的 query

Path parameters

Query parameters

- `is_all_node`

  可选，若为 true 则返回所有 FE 节点当前正在执行的 query 信息。默认为 true。

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "columnNames": ["Frontend", "QueryId", "ConnectionId", "Database", "User", "ExecTime", "
            ↪ SqlHash", "Statement"],
        "rows": [
            ["172.19.0.3", "108e47ab438a4560-ab1651d16c036491", "2", "", "root", "6074", "1
                ↪ a35f62f4b14b9d7961b057b77c3102f", "select sleep(60)"],
            ["172.19.0.11", "3606cad4e34b49c6-867bf6862cacc645", "3", "", "root", "9306", "1
                ↪ a35f62f4b14b9d7961b057b77c3102f", "select sleep(60)"]
        ]
    },
    "count": 0
}
```

4.9.2.15.8 取消 query

`POST /rest/v2/manager/query/kill/{query_id}`

Description

取消执行连接中正在执行的 query

Path parameters

- `{query_id}`

  query id. 你可以通过 trace_id 接口，获取 query id。

Query parameters

Response

```
{
    "msg": "success",
    "code": 0,
    "data": null,
    "count": 0
}
```

4.9.2.16 Backends Action

4.9.2.16.1 Request

```
GET /api/backends
```

4.9.2.16.2 Description

Backends Action 返回 Backends 列表，包括 Backend 的 IP、PORT 等信息。

4.9.2.16.3 Path parameters

无

4.9.2.16.4 Query parameters

- is_alive

  可选参数。是否返回存活的 BE 节点。默认为 false，即返回所有 BE 节点。

4.9.2.16.5 Request body

无

4.9.2.16.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "backends": [
            {
                "ip": "192.1.1.1",
                "http_port": 8040,
                "is_alive": true
```

```
            }
        ]
    },
    "count": 0
}
```

4.9.2.17 Bootstrap Action

4.9.2.17.1 Request

```
GET /api/bootstrap
```

4.9.2.17.2 Description

用于判断 FE 是否启动完成。当不提供任何参数时，仅返回是否启动成功。如果提供了 token 和 cluster_id，则返回更多详细信息。

4.9.2.17.3 Path parameters

无

4.9.2.17.4 Query parameters

- cluster_id

  集群 id。可以在 doris-meta/image/VERSION 文件中查看。

- token

  集群 token。可以在 doris-meta/image/VERSION 文件中查看。

4.9.2.17.5 Request body

无

4.9.2.17.6 Response

- 不提供参数

```
{
    "msg": "OK",
    "code": 0,
    "data": null,
    "count": 0
}
```

```
code 为 0 表示 FE 节点启动成功。非 0 的错误码表示其他错误。
```

- 提供 token 和 cluster_id

```
{
    "msg": "OK",
    "code": 0,
    "data": {
        "queryPort": 9030,
        "rpcPort": 9020,
        "arrowFlightSqlPort": 9040,
        "maxReplayedJournal": 17287
    },
    "count": 0
}
```

```
* `queryPort` 是 FE 节点的 MySQL 协议端口。
* `rpcPort` 是 FE 节点的 thrift RPC 端口。
* `maxReplayedJournal` 表示 FE 节点当前回放的最大元数据日志 id。
* `arrowFlightSqlPort` 是 FE 节点的 Arrow Flight SQL 协议端口。
```

4.9.2.17.7 Examples

1. 不提供参数

```
GET /api/bootstrap

Response:
{
    "msg": "OK",
    "code": 0,
    "data": null,
    "count": 0
}
```

2. 提供 token 和 cluster_id

```
GET /api/bootstrap?cluster_id=935437471&token=ad87f6dd-c93f-4880-bcdb-8ca8c9ab3031

Response:
{
    "msg": "OK",
```

```
        "code": 0,
        "data": {
            "queryPort": 9030,
            "rpcPort": 9020,
            "arrowFlightSqlPort": 9040,
            "maxReplayedJournal": 17287
        },
        "count": 0
    }
```

4.9.2.18 Cancel Load Action

4.9.2.18.1 Request

`POST /api/<db>/_cancel`

4.9.2.18.2 Description

用于取消掉指定 label 的导入任务。执行完成后，会以 Json 格式返回这次导入的相关内容。当前包括以下字段 Status: 是否成功 cancel Success: 成功 cancel 事务其他：cancel 失败 Message: 具体的失败信息

4.9.2.18.3 Path parameters

- <db>

  指定数据库名称

4.9.2.18.4 Query parameters

- <label>

  指定导入 label

4.9.2.18.5 Request body

无

4.9.2.18.6 Response

- 取消成功

```
    {
        "msg": "OK",
        "code": 0,
        "data": null,
```

```
    "count": 0
}
```

- 取消失败

```
{
    "msg": "Error msg...",
    "code": 1,
    "data": null,
    "count": 0
}
```

4.9.2.18.7 Examples

1. 取消指定 label 的导入事务

```
POST /api/example_db/_cancel?label=my_label1

Response:
{
    "msg": "OK",
    "code": 0,
    "data": null,
    "count": 0
}
```

4.9.2.19 Check Decommission Action

4.9.2.19.1 Request

`GET /api/check_decommission`

4.9.2.19.2 Description

用于判断指定的 BE 是否能够被下线。比如判断节点下线后，剩余的节点是否能够满足空间要求和副本数要求等。

4.9.2.19.3 Path parameters

无

4.9.2.19.4 Query parameters

- host_ports

  指定一个多个 BE，由逗号分隔。如：ip1:port1,ip2:port2,...。

  其中 port 为 BE 的 heartbeat port。

4.9.2.19.5 Request body

无

4.9.2.19.6 Response

返回可以被下线的节点列表

```
{
    "msg": "OK",
    "code": 0,
    "data": ["192.168.10.11:9050", "192.168.10.11:9050"],
    "count": 0
}
```

4.9.2.19.7 Examples

1. 查看指定 BE 节点是否可以下线

```
    GET /api/check_decommission?host_ports=192.168.10.11:9050,192.168.10.11:9050

    Response:
    {
        "msg": "OK",
        "code": 0,
        "data": ["192.168.10.11:9050"],
        "count": 0
    }
```

4.9.2.20 Check Storage Type Action

4.9.2.20.1 Request

```
GET /api/_check_storagetype
```

4.9.2.20.2 Description

用于检查指定数据库下的表的存储格式否是行存格式。（行存格式已废弃）

4.9.2.20.3 Path parameters

无

4.9.2.20.4 Query parameters

- db

  指定数据库

4.9.2.20.5 Request body

无

4.9.2.20.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "tbl2": {},
        "tbl1": {}
    },
    "count": 0
}
```

如果表名后有内容，则会显示存储格式为行存的 base 或者 rollup 表。

4.9.2.20.7 Examples

1. 检查指定数据库下表的存储格式是否为行存

```
    GET /api/_check_storagetype

    Response:
    {
        "msg": "success",
        "code": 0,
        "data": {
            "tbl2": {},
            "tbl1": {}
        },
        "count": 0
    }
```

4.9.2.21 Connection Action

4.9.2.21.1 Request

`GET /api/connection`

4.9.2.21.2 Description

给定一个 connection id，返回这个连接当前正在执行的，或最后一次执行完成的 query id。

connection id 可以通过 MySQL 命令 `show processlist;` 中的 id 列查看。

4.9.2.21.3 Path parameters

无

4.9.2.21.4 Query parameters

- `connection_id`

  指定的 connection id

4.9.2.21.5 Request body

无

4.9.2.21.6 Response

```
{
    "msg": "OK",
    "code": 0,
    "data": {
        "query_id": "b52513ce3f0841ca-9cb4a96a268f2dba"
    },
    "count": 0
}
```

4.9.2.21.7 Examples

1. 获取指定 connection id 的 query id

```
    GET /api/connection?connection_id=101

    Response:
    {
        "msg": "OK",
```

```
        "code": 0,
        "data": {
            "query_id": "b52513ce3f0841ca-9cb4a96a268f2dba"
        },
        "count": 0
    }
```

4.9.2.22 Extra Basepath Action

4.9.2.22.1 Request

```
GET /api/basepath
```

4.9.2.22.2 Description

获取 http 的 basepath。

4.9.2.22.3 Path parameters

无

4.9.2.22.4 Query parameters

无

4.9.2.22.5 Request body

无

4.9.2.22.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":{"enable":false,"path":""},
    "count":0
}
```

4.9.2.23 Fe Version Info Action

4.9.2.23.1 Request

```
GET /api/fe_version_info
```

4.9.2.23.2 Description

用于获取 fe 节点的版本信息。

4.9.2.23.3 Path parameters

无

4.9.2.23.4 Query parameters

无

4.9.2.23.5 Request body

无

4.9.2.23.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":{
        "feVersionInfo":{
            "dorisBuildVersionPrefix":"doris",
            "dorisBuildVersionMajor":0,
            "dorisBuildVersionMinor":0,
            "dorisBuildVersionPatch":0,
            "dorisBuildVersionRcVersion":"trunk",
            "dorisBuildVersion":"doris-0.0.0-trunk",
            "dorisBuildHash":"git://4b7b503d1cb3/data/doris/doris/be/../
                ↪ @a04f9814fe5a09c0d9e9399fe71cc4d765f8bff1",
            "dorisBuildShortHash":"a04f981",
            "dorisBuildTime":"Fri, 09 Sep 2022 07:57:02 UTC",
            "dorisBuildInfo":"root@4b7b503d1cb3",
            "dorisJavaCompileVersion":"openjdk full version \"1.8.0_332-b09\""
        }
    },
    "count":0
}
```

4.9.2.23.7 Examples

```
GET /api/fe_version_info

Response:
{
```

```
    "msg":"success",
    "code":0,
    "data":{
        "feVersionInfo":{
            "dorisBuildVersionPrefix":"doris",
            "dorisBuildVersionMajor":0,
            "dorisBuildVersionMinor":0,
            "dorisBuildVersionPatch":0,
            "dorisBuildVersionRcVersion":"trunk",
            "dorisBuildVersion":"doris-0.0.0-trunk",
            "dorisBuildHash":"git://4b7b503d1cb3/data/doris/doris/be/../
                ↪ @a04f9814fe5a09c0d9e9399fe71cc4d765f8bff1",
            "dorisBuildShortHash":"a04f981",
            "dorisBuildTime":"Fri, 09 Sep 2022 07:57:02 UTC",
            "dorisBuildInfo":"root@4b7b503d1cb3",
            "dorisJavaCompileVersion":"openjdk full version \"1.8.0_332-b09\""
        }
    },
    "count":0
}
```

4.9.2.24 Get DDL Statement Action

4.9.2.24.1 Request

`GET /api/_get_ddl`

4.9.2.24.2 Description

用于获取指定表的建表语句、建分区语句和建 rollup 语句。

4.9.2.24.3 Path parameters

无

4.9.2.24.4 Query parameters

- db

  指定数据库

- table

  指定表

4.9.2.24.5 Request body

无

4.9.2.24.6 Response

```
{
    "msg": "OK",
    "code": 0,
    "data": {
        "create_partition": ["ALTER TABLE `tbl1` ADD PARTITION ..."],
        "create_table": ["CREATE TABLE `tbl1` ...],
        "create_rollup": ["ALTER TABLE `tbl1` ADD ROLLUP ..."]
    },
    "count": 0
}
```

4.9.2.24.7 Examples

1. 获取指定表的 DDL 语句

```
    GET GET /api/_get_ddl?db=db1&table=tbl1

    Response
    {
        "msg": "OK",
        "code": 0,
        "data": {
            "create_partition": [],
            "create_table": ["CREATE TABLE `tbl1` (`\n`  `k1` int(11) NULL COMMENT \"\",`\n`  `k2
                ↪ ` int(11) NULL COMMENT \"\"`\n`) ENGINE=OLAP\nDUPLICATE KEY(`k1`, `k2`)\
                ↪ nCOMMENT \"OLAP\"\nDISTRIBUTED BY HASH(`k1`) BUCKETS 1\nPROPERTIES (`\n`\"
                ↪ replication_num\" = \"1\",`\n`\"version_info\" = \"1,0\",`\n`\"in_memory\" =
                ↪ \"false\",`\n`\"storage_format\" = \"DEFAULT\"`\n`);"],
            "create_rollup": []
        },
        "count": 0
    }
```

4.9.2.25 Get Load Info Action

4.9.2.25.1 Request

```
GET /api/<db>/_load_info
```

4.9.2.25.2 Description

用于获取指定 label 的导入作业的信息。

4.9.2.25.3 Path parameters

- <db>

  指定数据库

4.9.2.25.4 Query parameters

- label

  指定导入 Label

4.9.2.25.5 Request body

无

4.9.2.25.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "dbName": "default_cluster:db1",
        "tblNames": ["tbl1"],
        "label": "my_label",
        "clusterName": "default_cluster",
        "state": "FINISHED",
        "failMsg": "",
        "trackingUrl": ""
    },
    "count": 0
}
```

4.9.2.25.7 Examples

1. 获取指定 label 的导入作业信息

```
    GET /api/example_db/_load_info?label=my_label

    Response
    {
        "msg": "success",
```

```
    "code": 0,
    "data": {
        "dbName": "default_cluster:db1",
        "tblNames": ["tbl1"],
        "label": "my_label",
        "clusterName": "default_cluster",
        "state": "FINISHED",
        "failMsg": "",
        "trackingUrl": ""
    },
    "count": 0
}
```

4.9.2.26 Get Load State

4.9.2.26.1 Request

```
GET /api/<db>/get_load_state
```

4.9.2.26.2 Description

返回指定 label 的导入事务的状态执行完毕后，会以 Json 格式返回这次导入的相关内容。当前包括以下字段：Label：本次导入的 label，如果没有指定，则为一个 uuid Status：此命令是否成功执行，Success 表示成功执行 Message：具体的执行信息 State: 只有在 Status 为 Success 时才有意义 UNKNOWN: 没有找到对应的 Label PREPARE: 对应的事务已经 prepare，但尚未提交 COMMITTED: 事务已经提交，不能被 cancel VISIBLE: 事务提交，并且数据可见，不能被 cancel ABORTED: 事务已经被 ROLLBACK，导入已经失败

4.9.2.26.3 Path parameters

- <db>

  指定数据库

4.9.2.26.4 Query parameters

- label

  指定导入 label

4.9.2.26.5 Request body

无

4.9.2.26.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": "VISIBLE",
    "count": 0
}
```

如 label 不存在，则返回：

```
{
    "msg": "success",
    "code": 0,
    "data": "UNKNOWN",
    "count": 0
}
```

4.9.2.26.7 Examples

1. 获取指定 label 的导入事务的状态。

```
    GET /api/example_db/get_load_state?label=my_label

    {
        "msg": "success",
        "code": 0,
        "data": "VISIBLE",
        "count": 0
    }
```

4.9.2.27 Get FE log file

4.9.2.27.1 Request

```
HEAD /api/get_log_file
```

```
GET /api/get_log_file
```

4.9.2.27.2 Description

用户可以通过该 HTTP 接口获取 FE 的日志文件。

其中 HEAD 请求用于获取指定日志类型的日志文件列表。GET 请求用于下载指定的日志文件。

4.9.2.27.3 Path parameters

无

4.9.2.27.4 Query parameters

- type

  指定日志类型，支持如下类型：

  - fe.audit.log：FE 审计日志

- file

  指定的文件名。

4.9.2.27.5 Request body

无

4.9.2.27.6 Response

- HEAD

```
HTTP/1.1 200 OK
file_infos: {"fe.audit.log":24759,"fe.audit.log.20190528.1":132934}
content-type: text/html
connection: keep-alive
```

返回的 header 中罗列出了当前所有指定类型的日志文件，以及每个文件的大小。

- GET

  以文本形式下载指定日志文件

4.9.2.27.7 Examples

1. 获取对应类型的日志文件列表

```
HEAD /api/get_log_file?type=fe.audit.log

Response:

HTTP/1.1 200 OK
file_infos: {"fe.audit.log":24759,"fe.audit.log.20190528.1":132934}
content-type: text/html
connection: keep-alive
```

在返回的 header 中，`file_infos` 字段以 json 格式展示文件列表以及对应文件大小（单位字节）

2. 下载日志文件

```
GET /api/get_log_file?type=fe.audit.log&file=fe.audit.log.20190528.1

Response:

< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Content-Disposition: attachment;fileName=fe.audit.log
< Content-Type: application/octet-stream;charset=UTF-8
< Transfer-Encoding: chunked

... File Content ...
```

4.9.2.28 Get Small File Action

4.9.2.28.1 Request

`GET /api/get_small_file`

4.9.2.28.2 Description

通过文件 id，下载在文件管理器中的文件。

Path parameters

无

4.9.2.28.3 Query parameters

- `token`

  集群的 token。可以在 `doris-meta/image/VERSION` 文件中查看。

- `file_id`

  文件管理器中显示的文件 id。文件 id 可以通过 `SHOW FILE` 命令查看。

4.9.2.28.4 Request body

无

4.9.2.28.5 Response

```
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Content-Disposition: attachment;fileName=ca.pem
< Content-Type: application/json;charset=UTF-8
< Transfer-Encoding: chunked

... File Content ...
```

如有错误，则返回：

```
{
    "msg": "File not found or is not content",
    "code": 1,
    "data": null,
    "count": 0
}
```

4.9.2.28.6 Examples

1. 下载指定 id 的文件

```
    GET /api/get_small_file?token=98e8c0a6-3a41-48b8-a72b-0432e42a7fe5&file_id=11002

    Response:

    < HTTP/1.1 200
    < Vary: Origin
    < Vary: Access-Control-Request-Method
    < Vary: Access-Control-Request-Headers
    < Content-Disposition: attachment;fileName=ca.pem
    < Content-Type: application/json;charset=UTF-8
    < Transfer-Encoding: chunked

    ... File Content ...
```

4.9.2.29 Get WAL size

4.9.2.29.1 Request

```
GET fe_host:fe_http_port/api/get_wal_size?host_ports=host1:port1,host2:port2...
```

4.9.2.29.2 Description

用户可以通过该 HTTP 接口获取指定 BE 的 WAL 文件的数目，若不指定 BE，则默认返回所有 BE 的 WAL 文件的数目。

4.9.2.29.3 Path parameters

无

4.9.2.29.4 Query parameters

- `host_ports`

  BE 的 ip 和 http 端口。

4.9.2.29.5 Request body

无

4.9.2.29.6 Response

```
{
"msg": "OK",
"code": 0,
"data": ["192.168.10.11:9050:1", "192.168.10.11:9050:0"],
"count": 0
}
```

4.9.2.29.7 Examples

1. 获取所有 BE 的 WAL 文件的数目。

```
curl -u root: "127.0.0.1:8038/api/get_wal_size"

Response:
{
"msg": "OK",
"code": 0,
"data": ["192.168.10.11:9050:1", "192.168.10.11:9050:0"],
"count": 0
}
```

在返回的结果中，BE 后跟的数字即为对应 BE 的 WAL 文件数目。

2. 获取指定 BE 的 WAL 文件的数目。

```
curl -u root: "127.0.0.1:8038/api/get_wal_size?192.168.10.11:9050"

Response:
{
"msg": "OK",
"code": 0,
"data": ["192.168.10.11:9050:1"],
"count": 0
}
```

在返回的结果中，BE 后跟的数字即为对应 BE 的 WAL 文件数目。

4.9.2.30 Health Action

4.9.2.30.1 Request

```
GET /api/health
```

4.9.2.30.2 Description

返回集群当前存活的 BE 节点数和宕机的 BE 节点数。

4.9.2.30.3 Path parameters

无

4.9.2.30.4 Query parameters

无

4.9.2.30.5 Request body

无

4.9.2.30.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "online_backend_num": 10,
        "total_backend_num": 10
    },
    "count": 0
}
```

4.9.2.31 Meta Info Action

Meta Info Action 用于获取集群内的元数据信息。如数据库列表，表结构等。

4.9.2.31.1 数据库列表

Request

```
GET /api/meta/namespaces/<ns_name>/databases
```

Description

获取所有数据库名称列表，按字母序排列。

Path parameters

无

Query parameters

- `limit`

  限制返回的结果行数

- `offset`

  分页信息，需要和 `limit` 一起使用

Request body

无

Response

```
{
    "msg": "OK",
    "code": 0,
    "data": [
       "db1", "db2", "db3", ...
    ],
    "count": 3
}
```

- data 字段返回数据库名列表。

4.9.2.31.2 表列表

Request

```
GET /api/meta/namespaces/<ns_name>/databases/<db_name>/tables
```

Description

获取指定数据库中的表列表，按字母序排列。

Path parameters

- <db_name>

  指定数据库名称

Query parameters

- limit

  限制返回的结果行数

- offset

  分页信息，需要和 limit 一起使用

Request body

无

Response

```
{
    "msg": "OK",
    "code": 0,
    "data": [
       "tbl1", "tbl2", "tbl3", ...
    ],
    "count": 0
}
```

- data 字段返回表名称列表。

4.9.2.31.3 表结构信息

Request

```
GET /api/meta/namespaces/<ns_name>/databases/<db_name>/tables/<tbl_name>/schema
```

Description

获取指定数据库中，指定表的表结构信息。

Path parameters

- <db_name>

  指定数据库名称

- <tbl_name>

  指定表名称

Query parameters

- with_mv

  可选项，如果未指定，默认返回 base 表的表结构。如果指定，则还会返回所有 rollup 的信息。

Request body

无

Response

```
GET /api/meta/namespaces/default/databases/db1/tables/tbl1/schema

{
    "msg": "success",
    "code": 0,
    "data": {
        "tbl1": {
            "schema": [{
                    "Field": "k1",
                    "Type": "INT",
                    "Null": "Yes",
                    "Extra": "",
                    "Default": null,
                    "Key": "true"
                },
                {
                    "Field": "k2",
                    "Type": "INT",
                    "Null": "Yes",
                    "Extra": "",
                    "Default": null,
                    "Key": "true"
                }
            ],
            "is_base": true
        }
    },
    "count": 0
}
```

```
GET /api/meta/namespaces/default/databases/db1/tables/tbl1/schema?with_mv?=1
```

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "tbl1": {
            "schema": [{
                    "Field": "k1",
                    "Type": "INT",
                    "Null": "Yes",
                    "Extra": "",
                    "Default": null,
                    "Key": "true"
                },
                {
                    "Field": "k2",
                    "Type": "INT",
                    "Null": "Yes",
                    "Extra": "",
                    "Default": null,
                    "Key": "true"
                }
            ],
            "is_base": true
        },
        "rollup1": {
            "schema": [{
                "Field": "k1",
                "Type": "INT",
                "Null": "Yes",
                "Extra": "",
                "Default": null,
                "Key": "true"
            }],
            "is_base": false
        }
    },
    "count": 0
}
```

- data 字段返回 base 表或 rollup 表的表结构信息。

4.9.2.32 Meta Replay State Action

（未实现）

4.9.2.32.1 Request

```
GET /api/_meta_replay_state
```

4.9.2.32.2 Description

获取 FE 节点元数据回放的状态。

4.9.2.32.3 Path parameters

无

4.9.2.32.4 Query parameters

无

4.9.2.32.5 Request body

无

4.9.2.32.6 Response

TODO

4.9.2.32.7 Examples

TODO

4.9.2.33 Metrics Action

4.9.2.33.1 Request

```
GET /api/metrics
```

4.9.2.33.2 Description

获取 doris metrics 信息。

4.9.2.33.3 Path parameters

无

4.9.2.33.4 Query parameters

- `type`

  可选参数。默认输出全部 metrics 信息，有以下取值：

  - `core` 输出核心 metrics 信息
  - `json` 以 json 格式输出 metrics 信息

4.9.2.33.5 Request body

无

4.9.2.33.6 Response

TO DO

4.9.2.34 Profile Action

4.9.2.34.1 Request

> **注意：**
> 使用该接口需要 admin_priv，建议使用 query profile action 来查看 profile

`GET /api/profile` `GET /api/profile/text`

4.9.2.34.2 Description

用于获取指定 query id 的 query profile 如果 query_id 不存在，直接返回 404 NOT FOUND 错误如果 query_id 存在，返回下列文本的 profile:

```
Query:
  Summary:
     - Query ID: a0a9259df9844029-845331577440a3bd
     - Start Time: 2020-06-15 14:10:05
     - End Time: 2020-06-15 14:10:05
     - Total: 8ms
     - Query Type: Query
     - Query State: EOF
     - Doris Version: trunk
     - User: root
     - Default Db: default_cluster:test
     - Sql Statement: select * from table1
  Execution Profile a0a9259df9844029-845331577440a3bd:(Active: 7.315ms, % non-child: 100.00%)
    Fragment 0:
      Instance a0a9259df9844029-845331577440a3be (host=TNetworkAddress(hostname:172.26.108.176,
          ↪ port:9560)):(Active: 1.523ms, % non-child: 0.24%)
         - MemoryLimit: 2.00 GB
         - PeakUsedReservation: 0.00
         - PeakMemoryUsage: 72.00 KB
         - RowsProduced: 5
         - AverageThreadTokens: 0.00
```

```
 - PeakReservation: 0.00
BlockMgr:
   - BlocksCreated: 0
   - BlockWritesOutstanding: 0
   - BytesWritten: 0.00
   - TotalEncryptionTime: 0ns
   - BufferedPins: 0
   - TotalReadBlockTime: 0ns
   - TotalBufferWaitTime: 0ns
   - BlocksRecycled: 0
   - TotalIntegrityCheckTime: 0ns
   - MaxBlockSize: 8.00 MB
DataBufferSender (dst_fragment_instance_id=a0a9259df9844029-845331577440a3be):
   - AppendBatchTime: 9.23us
     - ResultSendTime: 956ns
     - TupleConvertTime: 5.735us
   - NumSentRows: 5
OLAP_SCAN_NODE (id=0):(Active: 1.506ms, % non-child: 20.59%)
   - TotalRawReadTime: 0ns
   - CompressedBytesRead: 6.47 KB
   - PeakMemoryUsage: 0.00
   - RowsPushedCondFiltered: 0
   - ScanRangesComplete: 0
   - ScanTime: 25.195us
   - BitmapIndexFilterTimer: 0ns
   - BitmapIndexFilterCount: 0
   - NumScanners: 65
   - RowsStatsFiltered: 0
   - VectorPredEvalTime: 0ns
   - BlockSeekTime: 1.299ms
   - RawRowsRead: 1.91K (1910)
   - ScannerThreadsVoluntaryContextSwitches: 0
   - RowsDelFiltered: 0
   - IndexLoadTime: 911.104us
   - NumDiskAccess: 1
   - ScannerThreadsTotalWallClockTime: 0ns
     - MaterializeTupleTime: 0ns
     - ScannerThreadsUserTime: 0ns
     - ScannerThreadsSysTime: 0ns
   - TotalPagesNum: 0
   - RowsReturnedRate: 3.319K /sec
   - BlockLoadTime: 539.289us
   - CachedPagesNum: 0
   - BlocksLoad: 384
   - UncompressedBytesRead: 0.00
```

```
           - RowsBloomFilterFiltered: 0
           - TabletCount : 1
           - RowsReturned: 5
           - ScannerThreadsInvoluntaryContextSwitches: 0
           - DecompressorTimer: 0ns
           - RowsVectorPredFiltered: 0
           - ReaderInitTime: 6.498ms
           - RowsRead: 5
           - PerReadThreadRawHdfsThroughput: 0.0 /sec
           - BlockFetchTime: 4.318ms
           - ShowHintsTime: 0ns
           - TotalReadThroughput: 0.0 /sec
           - IOTimer: 1.154ms
           - BytesRead: 48.49 KB
           - BlockConvertTime: 97.539us
           - BlockSeekCount: 0
```

如果为 text 接口，直接返回 profile 的纯文本内容

4.9.2.34.3 Path parameters

无

4.9.2.34.4 Query parameters

- query_id

  指定的 query id

4.9.2.34.5 Request body

无

4.9.2.34.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "profile": "query profile ..."
    },
    "count": 0
}
```

4.9.2.34.7 Examples

1. 获取指定 query_id 的 query profile

```
GET /api/profile?query_id=f732084bc8e74f39-8313581c9c3c0b58

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "profile": "query profile ..."
    },
    "count": 0
}
```

2. 获取指定 query_id 的 query profile 的纯文本

```
GET /api/profile/text?query_id=f732084bc8e74f39-8313581c9c3c0b58

Response:
    Summary:
    - Profile ID: 48bdf6d75dbb46c9-998b9c0368f4561f
    - Task Type: QUERY
    - Start Time: 2023-12-20 11:09:41
    - End Time: 2023-12-20 11:09:45
    - Total: 3s680ms
    - Task State: EOF
    - User: root
    - Default Db: tpcds
    - Sql Statement: with customer_total_return as
  select sr_customer_sk as ctr_customer_sk
  ,sr_store_sk as ctr_store_sk
  ,sum(SR_FEE) as ctr_total_return
  ...
```

4.9.2.35 Query Detail Action

4.9.2.35.1 Request

```
GET /api/query_detail
```

4.9.2.35.2 Description

用于获取指定时间点之后的所有查询的信息

4.9.2.35.3 Path parameters

无

4.9.2.35.4 Query parameters

- event_time

  指定的时间点（Unix 时间戳，单位毫秒），获取该时间点之后的查询信息。

4.9.2.35.5 Request body

无

4.9.2.35.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "query_details": [{
            "eventTime": 1596462699216,
            "queryId": "f732084bc8e74f39-8313581c9c3c0b58",
            "startTime": 1596462698969,
            "endTime": 1596462699216,
            "latency": 247,
            "state": "FINISHED",
            "database": "db1",
            "sql": "select * from tbl1"
        }, {
            "eventTime": 1596463013929,
            "queryId": "ed2d0d80855d47a5-8b518a0f1472f60c",
            "startTime": 1596463013913,
            "endTime": 1596463013929,
            "latency": 16,
            "state": "FINISHED",
            "database": "db1",
            "sql": "select k1 from tbl1"
        }]
    },
    "count": 0
}
```

4.9.2.35.7 Examples

1. 获取指定时间点之后的查询详情。

```
GET /api/query_detail?event_time=1596462079958

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "query_details": [{
            "eventTime": 1596462699216,
            "queryId": "f732084bc8e74f39-8313581c9c3c0b58",
            "startTime": 1596462698969,
            "endTime": 1596462699216,
            "latency": 247,
            "state": "FINISHED",
            "database": "db1",
            "sql": "select * from tbl1"
        }, {
            "eventTime": 1596463013929,
            "queryId": "ed2d0d80855d47a5-8b518a0f1472f60c",
            "startTime": 1596463013913,
            "endTime": 1596463013929,
            "latency": 16,
            "state": "FINISHED",
            "database": "db1",
            "sql": "select k1 from tbl1"
        }]
    },
    "count": 0
}
```

4.9.2.36 Query Schema Action

4.9.2.36.1 Request

```
POST /api/query_schema/<ns_name>/<db_name>
```

4.9.2.36.2 Description

Query Schema Action 可以返回给定的 SQL 有关的表的建表语句。可以用于本地测试一些查询场景。该 API 在 1.2 版本中发布。

4.9.2.36.3 Path parameters

- <db_name>

  指定数据库名称。该数据库会被视为当前 session 的默认数据库，如果在 SQL 中的表名没有限定数据库名称的话，则使用该数据库。

4.9.2.36.4 Query parameters

无

4.9.2.36.5 Request body

```
text/plain

sql
```

- sql 字段为具体的 SQL

4.9.2.36.6 Response

- 返回结果集

```
    CREATE TABLE `tbl1` (
      `k1` int(11) NULL,
      `k2` int(11) NULL
    ) ENGINE=OLAP
    DUPLICATE KEY(`k1`, `k2`)
    COMMENT 'OLAP'
    DISTRIBUTED BY HASH(`k1`) BUCKETS 3
    PROPERTIES (
    "replication_allocation" = "tag.location.default: 1",
    "in_memory" = "false",
    "storage_format" = "V2",
    "disable_auto_compaction" = "false"
    );

    CREATE TABLE `tbl2` (
      `k1` int(11) NULL,
      `k2` int(11) NULL
    ) ENGINE=OLAP
    DUPLICATE KEY(`k1`, `k2`)
    COMMENT 'OLAP'
    DISTRIBUTED BY HASH(`k1`) BUCKETS 3
    PROPERTIES (
    "replication_allocation" = "tag.location.default: 1",
    "in_memory" = "false",
```

```
"storage_format" = "V2",
"disable_auto_compaction" = "false"
);
```

4.9.2.36.7 Example

1. 在本地文件 1.sql 中写入 SQL

```
select tbl1.k2 from tbl1 join tbl2 on tbl1.k1 = tbl2.k1;
```

2. 使用 curl 命令获取建表语句

```
curl -X POST -H 'Content-Type: text/plain'  -uroot: http://127.0.0.1:8030/api/query_schema/
    ↪ internal/db1 -d@1.sql
```

4.9.2.37 Query Stats Action

4.9.2.37.1 Request

```
查看
get api/query_stats/<catalog_name>
get api/query_stats/<catalog_name>/<db_name>
get api/query_stats/<catalog_name>/<db_name>/<tbl_name>

清空
delete api/query_stats/<catalog_name>/<db_name>
delete api/query_stats/<catalog_name>/<db_name>/<tbl_name>
```

4.9.2.37.2 Description

获取或者删除指定的 catalog 数据库或者表中的统计信息，如果是 doris catalog 可以使用 default_cluster

4.9.2.37.3 Path parameters

- <catalog_name>

  指定的 catalog 名称

- <db_name>

  指定的数据库名称

- <tbl_name>

  指定的表名称

4.9.2.37.4 Query parameters

- summary 如果为 true 则只返回 summary 信息，否则返回所有的表的详细统计信息，只在 get 时使用

4.9.2.37.5 Request body

```
GET /api/query_stats/default_cluster/test_query_db/baseall?summary=false
{
    "msg": "success",
    "code": 0,
    "data": {
        "summary": {
            "query": 2
        },
        "detail": {
            "baseall": {
                "summary": {
                    "query": 2
                }
            }
        }
    },
    "count": 0
}
```

4.9.2.37.6 Response

- 返回结果集

4.9.2.37.7 Example

2. 使用 curl 命令获取统计信息

```
    curl --location -u root: 'http://127.0.0.1:8030/api/query_stats/default_cluster/test_query_db
        ↪ /baseall?summary=false'
```

4.9.2.38 Row Count Action

4.9.2.38.1 Request

```
GET /api/rowcount
```

4.9.2.38.2 Description

用于手动更新指定表的行数统计信息。在更新行数统计信息的同时，也会以 JSON 格式返回表以及对应 rollup 的行数

4.9.2.38.3 Path parameters

无

4.9.2.38.4 Query parameters

- db

  指定的数据库

- table

  指定的表名

4.9.2.38.5 Request body

无

4.9.2.38.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "tbl1": 10000
    },
    "count": 0
}
```

4.9.2.38.7 Examples

1. 更新并获取指定 Table 的行数

```
    GET /api/rowcount?db=example_db&table=tbl1

    Response:
    {
        "msg": "success",
        "code": 0,
        "data": {
            "tbl1": 10000
        },
```

```
        "count": 0
    }
```

4.9.2.39 Set Config Action

4.9.2.39.1 Request

```
GET /api/_set_config
```

4.9.2.39.2 Description

用于动态设置 FE 的参数。该命令等同于通过 `ADMIN SET FRONTEND CONFIG` 命令。但该命令仅会设置对应 FE 节点的配置。并且不会自动转发 `MasterOnly` 配置项给 Master FE 节点。

4.9.2.39.3 Path parameters

无

4.9.2.39.4 Query parameters

- `confkey1=confvalue1`

  指定要设置的配置名称，其值为要修改的配置值。

- `persist`

  是否要将修改的配置持久化。默认为 false，即不持久化。如果为 true，这修改后的配置项会写入 `fe_custom.conf` 文件中，并在 FE 重启后仍会生效。

- `reset_persist`

  是否要清空原来的持久化配置，只在 persist 参数为 true 时生效。为了兼容原来的版本，reset_persist 默认为 true。
  如果 persist 设为 true，不设置 reset_persist 或 reset_persist 为 true，将先清空`fe_custom.conf`文件中的配置再将本次修改的配置写入`fe_custom.conf`；
  如果 persist 设为 true，reset_persist 为 false，本次修改的配置项将会增量添加到`fe_custom.conf`。

4.9.2.39.5 Request body

无

4.9.2.39.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "set": {
```

```
            "key": "value"
        },
        "err": [
            {
               "config_name": "",
               "config_value": "",
               "err_info": ""
            }
        ],
        "persist":""
    },
    "count": 0
}
```

set 字段表示设置成功的配置。err 字段表示设置失败的配置。persist 字段表示持久化信息。

4.9.2.39.7 Examples

1. 设置 storage_min_left_capacity_bytes 、 replica_ack_policy 和 agent_task_resend_wait_time_ms 三个配置的值。

```
    GET /api/_set_config?storage_min_left_capacity_bytes=1024&replica_ack_policy=SIMPLE_MAJORITY&
        ↪ agent_task_resend_wait_time_ms=true

    Response:
    {
    "msg": "success",
    "code": 0,
    "data": {
        "set": {
            "storage_min_left_capacity_bytes": "1024"
        },
        "err": [
            {
                "config_name": "replica_ack_policy",
                "config_value": "SIMPLE_MAJORITY",
                "err_info": "Not support dynamic modification."
            },
            {
                "config_name": "agent_task_resend_wait_time_ms",
                "config_value": "true",
                "err_info": "Unsupported configuration value type."
            }
        ],
        "persist": ""
```

```
},
"count": 0
}

storage_min_left_capacity_bytes 设置成功;
replica_ack_policy 设置失败，原因是该配置项不支持动态修改;
agent_task_resend_wait_time_ms 设置失败，因为该配置项类型为 long，设置 boolean 类型失败。
```

2. 设置 max_bytes_per_broker_scanner 并持久化

```
GET /api/_set_config?max_bytes_per_broker_scanner=21474836480&persist=true&reset_persist=
    ↪ false

Response:
{
"msg": "success",
"code": 0,
"data": {
    "set": {
        "max_bytes_per_broker_scanner": "21474836480"
    },
    "err": [],
    "persist": "ok"
},
"count": 0
}
```

fe/conf 目录生成 fe_custom.conf：

```
#THIS IS AN AUTO GENERATED CONFIG FILE.
#You can modify this file manually, and the configurations in this file
#will overwrite the configurations in fe.conf
#Wed Jul 28 12:43:14 CST 2021
max_bytes_per_broker_scanner=21474836480
```

4.9.2.40 Show Data Action

4.9.2.40.1 Request

```
GET /api/show_data
```

4.9.2.40.2 Description

用于获取集群的总数据量，或者指定数据库的数据量。单位字节。

4.9.2.40.3 Path parameters

无

4.9.2.40.4 Query parameters

- db

  可选。如果指定，则获取指定数据库的数据量。

4.9.2.40.5 Request body

无

4.9.2.40.6 Response

1. 指定数据库的数据量。

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "default_cluster:db1": 381
    },
    "count": 0
}
```

2. 总数据量

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "__total_size": 381
    },
    "count": 0
}
```

4.9.2.40.7 Examples

1. 获取指定数据库的数据量

```
GET /api/show_data?db=db1

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "default_cluster:db1": 381
    },
    "count": 0
}
```

2. 获取集群总数据量

```
GET /api/show_data

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "__total_size": 381
    },
    "count": 0
}
```

4.9.2.41 Show Meta Info Action

4.9.2.41.1 Request

`GET /api/show_meta_info`

4.9.2.41.2 Description

用于显示一些元数据信息

4.9.2.41.3 Path parameters

无

4.9.2.41.4 Query parameters

- action

  指定要获取的元数据信息类型。目前支持如下：

  - SHOW_DB_SIZE

    获取指定数据库的数据量大小，单位为字节。

  - SHOW_HA

    获取 FE 元数据日志的回放情况，以及可选举组的情况。

4.9.2.41.5 Request body

无

4.9.2.41.6 Response

- SHOW_DB_SIZE

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "default_cluster:information_schema": 0,
        "default_cluster:db1": 381
    },
    "count": 0
}
```

- SHOW_HA

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "can_read": "true",
        "role": "MASTER",
        "is_ready": "true",
        "last_checkpoint_version": "1492",
        "last_checkpoint_time": "1596465109000",
        "current_journal_id": "1595",
        "electable_nodes": "",
        "observer_nodes": "",
        "master": "10.81.85.89"
    },
```

```
    "count": 0
}
```

4.9.2.41.7 Examples

1. 查看集群各个数据库的数据量大小

```
GET /api/show_meta_info?action=show_db_size

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "default_cluster:information_schema": 0,
        "default_cluster:db1": 381
    },
    "count": 0
}
```

2. 查看 FE 选举组情况

```
GET /api/show_meta_info?action=show_ha

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "can_read": "true",
        "role": "MASTER",
        "is_ready": "true",
        "last_checkpoint_version": "1492",
        "last_checkpoint_time": "1596465109000",
        "current_journal_id": "1595",
        "electable_nodes": "",
        "observer_nodes": "",
        "master": "10.81.85.89"
    },
    "count": 0
}
```

4.9.2.42 Show Proc Action

4.9.2.42.1 Request

```
GET /api/show_proc
```

4.9.2.42.2 Description

用于获取 PROC 信息。

4.9.2.42.3 Path parameters

无

4.9.2.42.4 Query parameters

- path

  指定的 Proc Path

- forward

  是否转发给 Master FE 执行

4.9.2.42.5 Request body

无

4.9.2.42.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": [
        proc infos ...
    ],
    "count": 0
}
```

4.9.2.42.7 Examples

1. 查看 `/statistic` 信息

```
    GET /api/show_proc?path=/statistic

    Response:
    {
        "msg": "success",
        "code": 0,
```

```
    "data": [
        ["10003", "default_cluster:db1", "2", "3", "3", "3", "3", "0", "0", "0"],
        ["10013", "default_cluster:doris_audit_db__", "1", "4", "4", "4", "4", "0", "0",
            ↪ "0"],
        ["Total", "2", "3", "7", "7", "7", "7", "0", "0", "0"]
    ],
    "count": 0
}
```

2. 转发到 Master 执行

```
GET /api/show_proc?path=/statistic&forward=true

Response:
{
    "msg": "success",
    "code": 0,
    "data": [
        ["10003", "default_cluster:db1", "2", "3", "3", "3", "3", "0", "0", "0"],
        ["10013", "default_cluster:doris_audit_db__", "1", "4", "4", "4", "4", "0", "0",
            ↪ "0"],
        ["Total", "2", "3", "7", "7", "7", "7", "0", "0", "0"]
    ],
    "count": 0
}
```

4.9.2.43 Show Runtime Info Action

4.9.2.43.1 Request

```
GET /api/show_runtime_info
```

4.9.2.43.2 Description

用于获取 FE JVM 的 Runtime 信息

4.9.2.43.3 Path parameters

无

4.9.2.43.4 Query parameters

无

4.9.2.43.5 Request body

无

4.9.2.43.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "free_mem": "855642056",
        "total_mem": "1037959168",
        "thread_cnt": "98",
        "max_mem": "1037959168"
    },
    "count": 0
}
```

4.9.2.43.7 Examples

1. 获取当前 FE 节点的 JVM 信息

```
    GET /api/show_runtime_info

    Response:
    {
        "msg": "success",
        "code": 0,
        "data": {
            "free_mem": "855642056",
            "total_mem": "1037959168",
            "thread_cnt": "98",
            "max_mem": "1037959168"
        },
        "count": 0
    }
```

4.9.2.44 Show Table Data Action

4.9.2.44.1 Request

```
GET /api/show_table_data
```

4.9.2.44.2 Description

用于获取所有 internal 源下所有数据库所有表的数据量，或者指定数据库或指定表的数据量。单位字节。

4.9.2.44.3 Path parameters

无

4.9.2.44.4 Query parameters

- db

  可选。如果指定，则获取指定数据库下表的数据量。

- `table`

  可选。如果指定，则获取指定表的数据量。

- `single_replica`

  可选。如果指定，则获取表单副本所占用的数据量。

4.9.2.44.5 Request body

无

4.9.2.44.6 Response

1. 指定数据库所有表的数据量。

```
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":9024548244,
            "revenue0":0,
            "customer":1906421482
        }
    },
    "count":0
}
```

2. 指定数据库指定表的数据量。

```
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":9024548244
        }
    },
    "count":0
}
```

3. 指定数据库指定表单副本的数据量。

```
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":3008182748
        }
    },
    "count":0
}
```

4.9.2.44.7 Examples

1. 获取指定数据库的数据量

```
GET /api/show_table_data?db=tpch

Response:
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":9024548244,
            "revenue0":0,
            "customer":1906421482
        }
    },
    "count":0
}
```

2. 指定数据库指定表的数据量。

```
GET /api/show_table_data?db=tpch&table=partsupp

Response:
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":9024548244
        }
    },
    "count":0
}
```

3. 指定数据库指定表单副本的数据量。

```
GET /api/show_table_data?db=tpch&table=partsupp&single_replica=true

Response:
{
    "msg":"success",
    "code":0,
    "data":{
        "tpch":{
            "partsupp":3008182748
        }
    },
    "count":0
}
```

4.9.2.45 Statement Execution Action

4.9.2.45.1 Request

```
POST /api/query/<ns_name>/<db_name>
```

4.9.2.45.2 Description

Statement Execution Action 用于执行语句并返回结果。

4.9.2.45.3 Path parameters

- <db_name>

  指定数据库名称。该数据库会被视为当前 session 的默认数据库，如果在 SQL 中的表名没有限定数据库名称的话，则使用该数据库。

4.9.2.45.4 Query parameters

无

4.9.2.45.5 Request body

```
{
    "stmt" : "select * from tbl1"
}
```

- sql 字段为具体的 SQL

Response

- 返回结果集

```
    {
        "msg": "success",
        "code": 0,
        "data": {
            "type": "result_set",
            "data": [
                [1],
                [2]
            ],
            "meta": [{
                "name": "k1",
                "type": "INT"
            }],
            "status": {},
            "time": 10
        },
        "count": 0
    }
```

```
* type 字段为 `result_set` 表示返回结果集。需要根据 meta 和 data 字段获取并展示结果。meta
    ↪ 字段描述返回的列信息。data 字段返回结果行。其中每一行的中的列类型，需要通过 meta
    ↪ 字段内容判断。status 字段返回 MySQL 的一些信息，如告警行数，状态码等。time
    ↪ 字段返回语句执行时间，单位毫秒。
```

- 返回执行结果

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "type": "exec_status",
        "status": {},
        "time": 10
    },
    "count": 0
}
```

```
* type 字段为 `exec_status` 表示返回执行结果。目前收到该返回结果，则都表示语句执行成功。
```

4.9.2.46 Table Query Plan Action

4.9.2.46.1 Request

```
POST /api/<db>/<table>/_query_plan
```

4.9.2.46.2 Description

给定一个 SQL，用于获取该 SQL 对应的查询计划。

该接口目前用于 Spark-Doris-Connector 中，Spark 获取 Doris 的查询计划。

4.9.2.46.3 Path parameters

- <db>

  指定数据库

- <table>

  指定表

4.9.2.46.4 Query parameters

无

4.9.2.46.5 Request body

```
{
    "sql": "select * from db1.tbl1;"
}
```

4.9.2.46.6 Response


```
{
    "msg": "success",
    "code": 0,
    "data": {
        "partitions": {
            "10039": {
                "routings": ["10.81.85.89:9062"],
                "version": 2,
                "versionHash": 982459448378619656,
                "schemaHash": 1294206575
            }
        },
        "opaqued_query_plan": "DAABDAACDwABDAAAAAEIAAEAAAAACAACAAAAAAgAAwAAAAAKAAT/////////
            ↪ w8ABQgAAAABAAAAAA8ABgIAAAABAAIACAAMABIIAAEAAAAADwACCwAAAAIAAAACazEAAAACazIPAAMIAAAAAgAAAAUAAAAFAg
            ↪ /////CAAX/////
            ↪ wAADwABDAAAAAEIAAEAAAAQDAACDwABDAAAAAEIAAEAAAAADAACCAABAAAABQAAAAgABAAAAAAMAA8IAAEAAAABCAACAAAAAA
            ↪ /////CAAX/////
            ↪ wAADAAFCAABAAAABgwACAAADAAGCAABAAAAAA8AAgwAAAAAAAoABwAAAAAAAAAACgAIAAAAAAAAAAAADQACCgwAAAABAAAAAA
            ↪ /////
            ↪ wgABQAAAAQIAAYAAAAACAAHAAAAAAsACAAAAAJrMQgACQAAAAACAAoBAAgAAQAAAAEIAAIAAAAADAADDwABDAAAAAEIAAEAAA
            ↪ ////8
            ↪ IAAUAAAAICAAGAAAAAAgABwAAAAELAAgAAAACazIIAAkAAAABAgAKAQAPAAIMAAAAAQgAAQAAAAAIAAIAAAAMCAADAAAAAQoA
            ↪ +h6eMxAAA=",
        "status": 200
    },
    "count": 0
}
```


其中 opaqued_query_plan 为查询计划的二进制格式。

4.9.2.46.7 Examples

1. 获取指定 sql 的查询计划

```
    POST /api/db1/tbl1/_query_plan
    {
        "sql": "select * from db1.tbl1;"
    }

    Response:
    {
        "msg": "success",
        "code": 0,
```

```
        "data": {
            "partitions": {
                "10039": {
                    "routings": ["192.168.1.1:9060"],
                    "version": 2,
                    "versionHash": 982459448378619656,
                    "schemaHash": 1294206575
                }
            },
            "opaqued_query_plan": "DAABDAACDwABD...",
            "status": 200
        },
        "count": 0
    }
```

4.9.2.47 Table Row Count Action

4.9.2.47.1 Request

```
GET /api/<db>/<table>/_count
```

4.9.2.47.2 Description

用于获取指定表的行数统计信息。该接口目前用于 Spark-Doris-Connector 中，Spark 获取 Doris 的表统计信息。

4.9.2.47.3 Path parameters

- <db>

  指定数据库

- <table>

  指定表

4.9.2.47.4 Query parameters

无

4.9.2.47.5 Request body

无

4.9.2.47.6 Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "size": 1,
        "status": 200
    },
    "count": 0
}
```

其中 `data.size` 字段表示指定表的行数。

4.9.2.47.7 Examples

1. 获取指定表的行数。

```
    GET /api/db1/tbl1/_count

    Response:
    {
        "msg": "success",
        "code": 0,
        "data": {
            "size": 1,
            "status": 200
        },
        "count": 0
    }
```

4.9.2.48 Table Schema Action

4.9.2.48.1 Request

`GET /api/<db>/<table>/_schema`

4.9.2.48.2 Description

用于获取指定表的表结构信息。该接口目前用于 Spark/Flink Doris Connector 中，获取 Doris 的表结构信息。

4.9.2.48.3 Path parameters

- <db>

  指定数据库

- <table>

  指定表

4.9.2.48.4 Query parameters

无

4.9.2.48.5 Request body

无

4.9.2.48.6 Response

- http 接口返回如下：

```
{
"msg": "success",
"code": 0,
"data": {
    "properties": [{
        "type": "INT",
        "name": "k1",
        "comment": "",
        "aggregation_type":""
    }, {
        "type": "INT",
        "name": "k2",
        "comment": "",
        "aggregation_type":"MAX"
    }],
    "keysType":UNIQUE_KEYS,
    "status": 200
},
"count": 0
}
```

- http v2 接口返回如下：

```
{
"msg": "success",
"code": 0,
"data": {
    "properties": [{
        "type": "INT",
        "name": "k1",
        "comment": ""
```

```
    }, {
        "type": "INT",
        "name": "k2",
        "comment": ""
    }],
    "keysType":UNIQUE_KEYS,
    "status": 200
},
"count": 0
}
```

注意：区别为http方式比http v2方式多返回aggregation_type字段，http v2开启是通过enable_http_server_v2进行设置，具体参数说明详见 fe 参数设置

4.9.2.48.7 Examples

1. **通过 http 获取指定表的表结构信息。**

```
GET /api/db1/tbl1/_schema

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "properties": [{
            "type": "INT",
            "name": "k1",
            "comment": "",
            "aggregation_type":""
        }, {
            "type": "INT",
            "name": "k2",
            "comment": "",
            "aggregation_type":"MAX"
        }],
        "keysType":UNIQUE_KEYS,
        "status": 200
    },
    "count": 0
}
```

2. **通过 http v2 获取指定表的表结构信息。**

```
GET /api/db1/tbl1/_schema

Response:
{
    "msg": "success",
    "code": 0,
    "data": {
        "properties": [{
            "type": "INT",
            "name": "k1",
            "comment": ""
        }, {
            "type": "INT",
            "name": "k2",
            "comment": ""
        }],
        "keysType":UNIQUE_KEYS,
        "status": 200
    },
    "count": 0
}
```

4.9.2.49 Upload Action

Upload Action 目前主要服务于 FE 的前端页面，用于用户导入一些测试性质的小文件。

4.9.2.49.1 上传导入文件

用于将文件上传到 FE 节点，可在稍后用于导入该文件。目前仅支持上传最大 100MB 的文件。

Request

```
POST /api/<namespace>/<db>/<tbl>/upload
```

Path parameters

- <namespace>

  命名空间，目前仅支持 default_cluster

- <db>

  指定的数据库

- <tbl>

  指定的表

Query parameters

- column_separator

  可选项，指定文件的分隔符。默认为 \t

- preview

  可选项，如果设置为 true，则返回结果中会显示最多 10 行根据 column_separator 切分好的数据行。

Request body

要上传的文件内容，Content-type 为 multipart/form-data

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "id": 1,
        "uuid": "b87824a4-f6fd-42c9-b9f1-c6d68c5964c2",
        "originFileName": "data.txt",
        "fileSize": 102400,
        "absPath": "/path/to/file/data.txt"
        "maxColNum" : 5
    },
    "count": 1
}
```

4.9.2.49.2 导入已上传的文件

Request

```
PUT /api/<namespace>/<db>/<tbl>/upload
```

Path parameters

- <namespace>

  命名空间，目前仅支持 default_cluster

- <db>

  指定的数据库

- <tbl>

  指定的表

Query parameters

- file_id

  指定导入的文件 id，文件 id 由上传导入文件的 API 返回。

- `file_uuid`

  指定导入的文件 uuid，文件 uuid 由上传导入文件的 API 返回。

Header

Header 中的可选项同 Stream Load 请求中 header 的可选项。

Request body

要上传的文件内容，Content-type 为 `multipart/form-data`

Response

```
{
    "msg": "success",
    "code": 0,
    "data": {
        "TxnId": 7009,
        "Label": "9dbdfb0a-120b-47a2-b078-4531498727cb",
        "Status": "Success",
        "Message": "OK",
        "NumberTotalRows": 3,
        "NumberLoadedRows": 3,
        "NumberFilteredRows": 0,
        "NumberUnselectedRows": 0,
        "LoadBytes": 12,
        "LoadTimeMs": 71,
        "BeginTxnTimeMs": 0,
        "StreamLoadPutTimeMs": 1,
        "ReadDataTimeMs": 0,
        "WriteDataTimeMs": 13,
        "CommitAndPublishTimeMs": 53
    },
    "count": 1
}
```

Example

```
PUT /api/default_cluster/db1/tbl1/upload?file_id=1&file_uuid=b87824a4-f6fd-42c9-b9f1-c6d68c5964c2
```

4.9.2.50 Import Action

4.9.2.50.1 Request

`POST /api/import/file_review`

4.9.2.50.2 Description

查看格式为 CSV 或 PARQUET 的文件内容。

4.9.2.50.3 Path parameters

无

4.9.2.50.4 Query parameters

无

4.9.2.50.5 Request body

TO DO

4.9.2.50.6 Response

TO DO

4.9.2.51 Meta Info Action

4.9.2.51.1 Request

```
GET /api/meta/namespaces/<ns>/databases GET /api/meta/namespaces/<ns>/databases/<db>/tables GET /
↪ api/meta/namespaces/<ns>/databases/<db>/tables/<tbl>/schema
```

4.9.2.51.2 Description

获取集群内的元数据信息，包括数据库列表、表列表以及表结构等。

4.9.2.51.3 Path parameters

- `ns`

  指定集群名。

- `db`

  指定数据库。

- `tbl`

  指定数据表。

4.9.2.51.4 Query parameters

无

4.9.2.51.5 Request body

无

4.9.2.51.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":["数据库列表" / "数据表列表" /"表结构"],
    "count":0
}
```

4.9.2.52 代码打桩

代码打桩，是指在 FE 或 BE 源码中插入一段代码，当程序执行到这里时，可以改变程序的变量或行为，这样的一段代码称为一个木桩。

主要用于单元测试或回归测试，用来构造正常方法无法实现的异常。

每一个木桩都有一个名称，可以随便取名，可以通过一些机制控制木桩的开关，还可以向木桩传递参数。

FE 和 BE 都支持代码打桩，打桩完后要重新编译 BE 或 FE。

4.9.2.52.1 木桩代码示例

FE 桩子示例代码

```
private Status foo() {
    // dbug_fe_foo_do_nothing 是一个木桩名字，
    // 打开这个木桩之后，DebugPointUtil.isEnable("dbug_fe_foo_do_nothing") 将会返回 true
    if (DebugPointUtil.isEnable("dbug_fe_foo_do_nothing")) {
        return Status.Nothing;
    }

     do_foo_action();

     return Status.Ok;
}
```

BE 桩子示例代码

```
void Status foo() {

     // dbug_be_foo_do_nothing 是一个木桩名字，
     // 打开这个木桩之后，DBUG_EXECUTE_IF 将会执行宏参数中的代码块
     DBUG_EXECUTE_IF("dbug_be_foo_do_nothing",  { return Status.Nothing; });

     do_foo_action();

     return Status.Ok;
}
```

4.9.2.52.2 总开关

需要把木桩总开关 enable_debug_points 打开之后，才能激活木桩。默认情况下，木桩总开关是关闭的。

总开关enable_debug_points 分别在 FE 的 fe.conf 和 BE 的 be.conf 中配置。

4.9.2.52.3 打开木桩

打开总开关后，还需要通过向 FE 或 BE 发送 http 请求的方式，打开或关闭指定名称的木桩，只有这样当代码执行到这个木桩时，相关代码才会被执行。

API

```
POST /api/debug_point/add/{debug_point_name}[?timeout=<int>&execute=<int>]
```

参数

- debug_point_name 木桩名字。必填。
- timeout 超时时间，单位为秒。超时之后，木桩失活。默认值 -1 表示永远不超时。可选。
- execute 木桩最大执行次数。默认值 -1 表示不限执行次数。可选。

Request body

无

Response

```
{
    msg: "OK",
    code: 0
}
```

Examples

打开木桩 foo，最多执行 5 次。

```
curl -X POST "http://127.0.0.1:8030/api/debug_point/add/foo?execute=5"
```

4.9.2.52.4 向木桩传递参数

激活木桩时，除了前文所述的 timeout 和 execute，还可以传递其它自定义参数。一个参数是一个形如 key=value 的 key-value 对，在 url 的路径部分，紧跟在木桩名称后，以字符 '?' 开头。

API

```
POST /api/debug_point/add/{debug_point_name}[?k1=v1&k2=v2&k3=v3...]
```

- k1=v1 k1 为参数名称，v1 为参数值，多个参数用 & 分隔。

Request body

无

Response

```
{
    msg: "OK",
    code: 0
}
```

Examples

假设 FE 在 fe.conf 中有配置 http_port=8030，则下面的请求激活 FE 中的木桩foo，并传递了两个参数 percent 和 duration：

```
curl -u root: -X POST "http://127.0.0.1:8030/api/debug_point/add/foo?percent=0.5&duration=3"
```

注意：
1、在 FE 或 BE 的代码中，参数名和参数值都是字符串。
2、在 FE 或 BE 的代码中和 http 请求中，参数名称和值都是大小写敏感的。
3、发给 FE 或 BE 的 http 请求，路径部分格式是相同的，只是 IP 地址和端口号不同。

在 FE 和 BE 代码中使用参数

激活 FE 中的木桩OlapTableSink.write_random_choose_sink并传递参数 needCatchUp 和 sinkNum: > 注意：可能需要用户名和密码

```
curl -u root: -X POST "http://127.0.0.1:8030/api/debug_point/add/OlapTableSink.write_random_
    ↪ choose_sink?needCatchUp=true&sinkNum=3"
```

在 FE 代码中使用木桩 OlapTableSink.write_random_choose_sink 的参数 needCatchUp 和 sinkNum：

```
private void debugWriteRandomChooseSink(Tablet tablet, long version, Multimap<Long, Long>
    ↪ bePathsMap) {
    DebugPoint debugPoint = DebugPointUtil.getDebugPoint("OlapTableSink.write_random_choose_sink"
        ↪ );
    if (debugPoint == null) {
        return;
    }
    boolean needCatchup = debugPoint.param("needCatchUp", false);
    int sinkNum = debugPoint.param("sinkNum", 0);
    ...
}
```

激活 BE 中的木桩TxnManager.prepare_txn.random_failed并传递参数 percent:

```
curl -X POST "http://127.0.0.1:8040/api/debug_point/add/TxnManager.prepare_txn.random_failed?
    ↪ percent=0.7
```

在 BE 代码中使用木桩 TxnManager.prepare_txn.random_failed 的参数 percent：

```
DBUG_EXECUTE_IF("TxnManager.prepare_txn.random_failed",
        {if (rand() % 100 < (100 * dp->param("percent", 0.5))) {
                LOG_WARNING("TxnManager.prepare_txn.random_failed random failed");
                return Status::InternalError("debug prepare txn random failed");
        }}
);
```

4.9.2.52.5 关闭木桩

API

```
POST /api/debug_point/remove/{debug_point_name}
```

参数

- debug_point_name 木桩名字。必填。

Request body

无

Response

```
{
    msg: "OK",
    code: 0
}
```

Examples

关闭木桩foo。

```
curl -X POST "http://127.0.0.1:8030/api/debug_point/remove/foo"
```

4.9.2.52.6 清除所有木桩

API

```
POST /api/debug_point/clear
```

Request body

无

Response

```
{
    msg: "OK",
    code: 0
}
```

Examples

清除所有木桩。

```
curl -X POST "http://127.0.0.1:8030/api/debug_point/clear"
```

4.9.2.52.7 在回归测试中使用木桩

> 提交 PR 时，社区 CI 系统默认开启 FE 和 BE 的enable_debug_points配置。

回归测试框架提供方法函数来开关指定的木桩，它们声明如下：

```
// 打开木桩，name 是木桩名称，params 是一个key-value列表，是传给木桩的参数
def enableDebugPointForAllFEs(String name, Map<String, String> params = null);
def enableDebugPointForAllBEs(String name, Map<String, String> params = null);
// 关闭木桩，name 是木桩的名称
def disableDebugPointForAllFEs(String name);
def disableDebugPointForAllFEs(String name);
```

需要在调用测试 action 之前调用 enableDebugPointForAllFEs() 或 enableDebugPointForAllBEs() 来开启木桩，这样执行到木桩代码时，相关代码才会被执行，然后在调用测试 action 之后调用 disableDebugPointForAllFEs
↪ () 或 disableDebugPointForAllBEs() 来关闭木桩。

并发问题

FE 或 BE 中开启的木桩是全局生效的，同一个 Pull Request 中，并发跑的其它测试，可能会受影响而意外失败。为了避免这种情况，我们规定，使用木桩的回归测试，必须放在 regression-test/suites/fault_injection_p0 目录下，且组名必须设置为 nonConcurrent，社区 CI 系统对于这些用例，会串行运行。

Examples

```
// 测试用例的.groovy 文件必须放在 regression-test/suites/fault_injection_p0 目录下，
// 且组名设置为 'nonConcurrent'
suite('debugpoint_action', 'nonConcurrent') {
    try {
        // 打开所有FE中，名为 "PublishVersionDaemon.stop_publish" 的木桩
        // 传参数 timeout
        // 与上面curl调用时一样，execute 是执行次数，timeout 是超时秒数
        GetDebugPoint().enableDebugPointForAllFEs('PublishVersionDaemon.stop_publish', [timeout
            ↪ :1])
```

```
        // 打开所有BE中，名为 "Tablet.build_tablet_report_info.version_miss" 的木桩
        // 传参数 tablet_id, version_miss 和 timeout
        GetDebugPoint().enableDebugPointForAllBEs('Tablet.build_tablet_report_info.version_miss',
                                                  [tablet_id:'12345', version_miss:true, timeout
                                                      ↪ :1])

        // 测试用例，会触发木桩代码的执行
        sql """CREATE TABLE tbl_1 (k1 INT, k2 INT)
               DUPLICATE KEY (k1)
               DISTRIBUTED BY HASH(k1)
               BUCKETS 3
               PROPERTIES ("replication_allocation" = "tag.location.default: 1");
            """
        sql "INSERT INTO tbl_1 VALUES (1, 10)"
        sql "INSERT INTO tbl_1 VALUES (2, 20)"
        order_qt_select_1_1 'SELECT * FROM tbl_1'

    } finally {
        GetDebugPoint().disableDebugPointForAllFEs('PublishVersionDaemon.stop_publish')
        GetDebugPoint().disableDebugPointForAllBEs('Tablet.build_tablet_report_info.version_miss
            ↪ ')
    }
}
```

4.9.2.53 Statistic Action

4.9.2.53.1 Request

```
GET /rest/v2/api/cluster_overview
```

4.9.2.53.2 Description

获取集群统计信息、库表数量等。

4.9.2.53.3 Path parameters

无

4.9.2.53.4 Query parameters

无

4.9.2.53.5 Request body

无

4.9.2.53.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":{"diskOccupancy":0,"remainDisk":5701197971457,"feCount":1,"tblCount":27,"beCount":1,"
        ↪ dbCount":2},
    "count":0
}
```

4.9.3 BE HTTP API

4.9.3.1 检查连接缓存

4.9.3.1.1 Request

```
GET /api/check_rpc_channel/{host_to_check}/{remot_brpc_port}/{payload_size}
```

4.9.3.1.2 Description

该功能用于检查 brpc 的连接缓存。

4.9.3.1.3 Path parameters

- `host_to_check`

  需要查检的 IP。

- `remot_brpc_port`

  需要查检的端口。

- `payload_size`

  负载大小，单位 B，取值范围 1~1024000。

4.9.3.1.4 Request body

无

4.9.3.1.5 Response

```
    {
        "msg":"success",
        "code":0,
        "data": "open brpc connection to {host_to_check}:{remot_brpc_port} success.",
        "count":0
    }
```

4.9.3.1.6 Examples

```
curl http://127.0.0.1:8040/api/check_rpc_channel/127.0.0.1/8060/1024000
```

4.9.3.2 重置连接缓存

4.9.3.2.1 Request

```
GET /api/reset_rpc_channel/{endpoints}
```

4.9.3.2.2 Description

该功能用于重置 brpc 的连接缓存。

4.9.3.2.3 Path parameters

- endpoints 支持如下形式：
  - all
  - host1:port1,host2:port2

4.9.3.2.4 Request body

无

4.9.3.2.5 Response

```
{
    "msg":"success",
    "code":0,
    "data": "no cached channel.",
    "count":0
}
```

4.9.3.2.6 Examples

```
curl http://127.0.0.1:8040/api/reset_rpc_channel/all
```

```
curl http://127.0.0.1:8040/api/reset_rpc_channel/1.1.1.1:8080,2.2.2.2:8080
```

4.9.3.3 查看 Compaction 状态

4.9.3.3.1 Request

```
GET /api/compaction/run_status GET /api/compaction/show?tablet_id={int}
```

4.9.3.3.2 Description

用于查看某个 BE 节点总体的 compaction 状态，或者指定 tablet 的 compaction 状态。

4.9.3.3.3 Query parameters

- tablet_id
  - tablet 的 id

4.9.3.3.4 Request body

无

4.9.3.3.5 Response

整体 Compaction 状态

```
{
  "CumulativeCompaction": {
         "/home/disk1" : [10001, 10002],
         "/home/disk2" : [10003]
  },
  "BaseCompaction": {
         "/home/disk1" : [10001, 10002],
         "/home/disk2" : [10003]
  }
}
```

该结构表示某个数据目录下，正在执行 compaction 任务的 tablet 的 id，以及 compaction 的类型。

指定 tablet 的 Compaction 状态

```
{
    "cumulative policy type": "SIZE_BASED",
    "cumulative point": 50,
    "last cumulative failure time": "2019-12-16 18:13:43.224",
    "last base failure time": "2019-12-16 18:13:23.320",
    "last cumu success time": ,
    "last base success time": "2019-12-16 18:11:50.780",
    "rowsets": [
        "[0-48] 10 DATA OVERLAPPING 574.00 MB",
        "[49-49] 2 DATA OVERLAPPING 574.00 B",
        "[50-50] 0 DELETE NONOVERLAPPING 574.00 B",
        "[51-51] 5 DATA OVERLAPPING 574.00 B"
    ],
    "missing_rowsets": [],
    "stale version path": [
```

```
        {
            "path id": "2",
            "last create time": "2019-12-16 18:11:15.110 +0800",
            "path list": "2-> [0-24] -> [25-48]"
        },
        {
            "path id": "1",
            "last create time": "2019-12-16 18:13:15.110 +0800",
            "path list": "1-> [25-40] -> [40-48]"
        }
    ]
}
```

结果说明：

- cumulative policy type：当前 tablet 所使用的 cumulative compaction 策略。
- cumulative point：base 和 cumulative compaction 的版本分界线。在 point（不含）之前的版本由 base compaction 处理。point（含）之后的版本由 cumulative compaction 处理。
- last cumulative failure time：上一次尝试 cumulative compaction 失败的时间。默认 10min 后才会再次尝试对该 tablet 做 cumulative compaction。
- last base failure time：上一次尝试 base compaction 失败的时间。默认 10min 后才会再次尝试对该 tablet 做 base compaction。
- rowsets：该 tablet 当前的 rowset 集合。如 [0-48] 表示 0-48 版本。第二位数字表示该版本中 segment 的数量。DELETE 表示 delete 版本。DATA 表示数据版本。OVERLAPPING 和 NONOVERLAPPING 表示 segment 数据是否重叠。
- missing_rowsets: 缺失的版本。
- stale version path：该 table 当前被合并 rowset 集合的合并版本路径，该结构是一个数组结构，每个元素表示一个合并路径。每个元素中包含了三个属性：path id 表示版本路径 id，last create time 表示当前路径上最近的 rowset 创建时间，默认在这个时间半个小时之后这条路径上的所有 rowset 会被过期删除。

4.9.3.3.6 Examples

```
curl http://192.168.10.24:8040/api/compaction/show?tablet_id=10015
```

4.9.3.4 触发 Compaction

4.9.3.4.1 Request

`POST /api/compaction/run?tablet_id={int}&compact_type={enum}` `POST /api/compaction/run?table_id={int}&compact_type=full` 注意，table_id=xxx 只有在 compact_type=full 时指定才会生效。`GET /api/compaction/run_status?tablet_id={int}`

4.9.3.4.2 Description

用于手动触发 Compaction 以及状态查询。

4.9.3.4.3 Query parameters

- `tablet_id`
  - tablet 的 id
- `table_id`
  - table 的 id。注意，table_id=xxx 只有在 compact_type=full 时指定才会生效，并且 tablet_id 和 table_id 只能指定一个，不能够同时指定，指定 table_id 后会自动对此 table 下所有 tablet 执行 full_compaction。
- `compact_type`
  - 取值为`base`或`cumulative`或`full`。full_compaction 的使用场景请参考数据恢复。

4.9.3.4.4 Request body

无

4.9.3.4.5 Response

触发 Compaction

若 tablet 不存在，返回 JSON 格式的错误：

```
{
    "status": "Fail",
    "msg": "Tablet not found"
}
```

若 compaction 执行任务触发失败时，返回 JSON 格式的错误：

```
{
    "status": "Fail",
    "msg": "fail to execute compaction, error = -2000"
}
```

若 compaction 执行触发成功时，则返回 JSON 格式的结果：

```
{
    "status": "Success",
    "msg": "compaction task is successfully triggered."
}
```

结果说明：

- status：触发任务状态，当成功触发时为 Success；当因某些原因（比如，没有获取到合适的版本）时，返回 Fail。
- msg：给出具体的成功或失败的信息。

查询状态

若 tablet 不存在，返回 JSON 格式：

```
{
    "status": "Fail",
    "msg": "Tablet not found"
}
```

若 tablet 存在并且 tablet 不在正在执行 compaction，返回 JSON 格式：

```
{
    "status" : "Success",
    "run_status" : false,
    "msg" : "this tablet_id is not running",
    "tablet_id" : 11308,
    "schema_hash" : 700967178,
    "compact_type" : ""
}
```

若 tablet 存在并且 tablet 正在执行 compaction，返回 JSON 格式：

```
{
    "status" : "Success",
    "run_status" : true,
    "msg" : "this tablet_id is running",
    "tablet_id" : 11308,
    "schema_hash" : 700967178,
    "compact_type" : "cumulative"
}
```

结果说明：

- run_status：获取当前手动 compaction 任务执行状态

Examples

```
curl -X POST "http://127.0.0.1:8040/api/compaction/run?tablet_id=10015&compact_type=cumulative"
```

4.9.3.5 查询元信息

4.9.3.5.1 Request

```
GET /api/meta/header/{tablet_id}?byte_to_base64={bool}
```

4.9.3.5.2 Description

查询 tablet 元信息

4.9.3.5.3 Path parameters

- `tablet_id` table 的 id

4.9.3.5.4 Query parameters

- `byte_to_base64` 是否按 base64 编码，选填，默认`false`。

4.9.3.5.5 Request body

无

4.9.3.5.6 Response

```
{
    "table_id": 148107,
    "partition_id": 148104,
    "tablet_id": 148193,
    "schema_hash": 2090621954,
    "shard_id": 38,
    "creation_time": 1673253868,
    "cumulative_layer_point": -1,
    "tablet_state": "PB_RUNNING",
    ...
}
```

4.9.3.5.7 Examples

```
curl "http://127.0.0.1:8040/api/meta/header/148193&byte_to_base64=true"
```

4.9.3.6 做快照

4.9.3.6.1 Request

```
GET /api/snapshot?tablet_id={int}&schema_hash={int}"
```

4.9.3.6.2 Description

该功能用于 tablet 做快照。

4.9.3.6.3 Query parameters

- `tablet_id` 需要做快照的 table 的 id
- `schema_hash` schema hash

4.9.3.6.4 Request body

无

4.9.3.6.5 Response

```
    /path/to/snapshot
```

4.9.3.6.6 Examples

```
    curl "http://127.0.0.1:8040/api/snapshot?tablet_id=123456&schema_hash=1111111"
```

4.9.3.7 检查 tablet 文件丢失

4.9.3.7.1 Request

```
GET /api/check_tablet_segment_lost?repair={bool}
```

4.9.3.7.2 Description

在 BE 节点上，可能会因为一些异常情况导致数据文件丢失，但是元数据显示正常，这种副本异常不会被 FE 检测到，也不能被修复。当用户查询时，会报错`failed to initialize storage reader`。该接口的功能是检测出当前 BE 节点上所有存在文件丢失的 tablet。

4.9.3.7.3 Query parameters

- `repair`
  - 设置为`true`时，存在文件丢失的 tablet 都会被设为`SHUTDOWN`状态，该副本会被作为坏副本处理，进而能够被 FE 检测和修复。
  - 设置为`false`时，只会返回所有存在文件丢失的 tablet，并不做任何处理。

4.9.3.7.4 Request body

无

4.9.3.7.5 Response

返回值是当前BE节点上所有存在文件丢失的tablet

```
{
    status: "Success",
    msg: "Succeed to check all tablet segment",
    num: 3,
    bad_tablets: [
        11190,
        11210,
        11216
    ],
    set_bad: true,
    host: "172.3.0.101"
}
```

4.9.3.7.6 Examples

```
curl http://127.0.0.1:8040/api/check_tablet_segment_lost?repair=false
```

4.9.3.8 BE 的配置信息

4.9.3.8.1 Request

GET /api/show_config POST /api/update_config?{key}={val}

4.9.3.8.2 Description

查询/更新 BE 的配置信息

4.9.3.8.3 Query parameters

- persist 是否持久化，选填，默认false。
- key 配置项名。
- val 配置项值。

4.9.3.8.4 Request body

无

4.9.3.8.5 Response

查询

```
[["agent_task_trace_threshold_sec","int32_t","2","true"], ...]
```

更新

```
[
    {
        "config_name": "agent_task_trace_threshold_sec",
        "status": "OK",
        "msg": ""
    }
]
```

```
[
    {
        "config_name": "agent_task_trace_threshold_sec",
        "status": "OK",
        "msg": ""
    },
    {
        "config_name": "enable_segcompaction",
        "status": "BAD",
        "msg": "set enable_segcompaction=false failed, reason: [NOT_IMPLEMENTED_ERROR]'enable_
            ↪ segcompaction' is not support to modify."
    },
    {
        "config_name": "enable_time_lut",
        "status": "BAD",
        "msg": "set enable_time_lut=false failed, reason: [NOT_IMPLEMENTED_ERROR]'enable_time_lut
            ↪ ' is not support to modify."
    }
]
```

4.9.3.8.6 Examples

```
curl "http://127.0.0.1:8040/api/show_config"
```

```
curl -X POST "http://127.0.0.1:8040/api/update_config?agent_task_trace_threshold_sec=2&persist=
    ↪ true"
```

```
curl -X POST "http://127.0.0.1:8040/api/update_config?agent_task_trace_threshold_sec=2&enable_
    ↪ merge_on_write_correctness_check=true&persist=true"
```

4.9.3.9 metrics 信息

4.9.3.9.1 Request

`GET /metrics?type={enum}&with_tablet={bool}`

4.9.3.9.2 Description

prometheus 监控采集接口

4.9.3.9.3 Query parameters

- `type` 输出方式，选填，默认全部输出，另有以下取值：
  - `core`: 只输出核心采集项
  - `json`: 以 json 格式输出
- `with_tablet` 是否输出 tablet 相关的采集项，选填，默认`false`。

4.9.3.9.4 Request body

无

4.9.3.9.5 Response

```
doris_be__max_network_receive_bytes_rate LONG 60757
doris_be__max_network_send_bytes_rate LONG 16232
doris_be_process_thread_num LONG 1120
doris_be_process_fd_num_used LONG 336
, , ,
```

4.9.3.9.6 Examples

```
curl "http://127.0.0.1:8040/metrics?type=json&with_tablet=true"
```

4.9.3.10 查询 tablet 分布

4.9.3.10.1 Request

`GET /api/tablets_distribution?group_by={enum}&partition_id={int}`

4.9.3.10.2 Description

获取 BE 节点上每一个 partition 下的 tablet 在不同磁盘上的分布情况

4.9.3.10.3 Query parameters

- group_by 分组，当前只支持partition
- partition_id 指定 partition 的 id，选填，默认返回所有 partition。

4.9.3.10.4 Request body

无

4.9.3.10.5 Response

```
{
    msg: "OK",
    code: 0,
    data: {
        host: "***",
        tablets_distribution: [
            {
                partition_id:***,
                disks:[
                    {
                        disk_path:"***",
                        tablets_num:***,
                        tablets:[
                            {
                                tablet_id:***,
                                schema_hash:***,
                                tablet_size:***
                            },

                            ...

                        ]
                    },

                    ...

                ]
            }
        ]
    },
    count: ***
}
```

4.9.3.10.6 Examples

```
curl "http://127.0.0.1:8040/api/tablets_distribution?group_by=partition&partition_id=123"
```

4.9.3.11 迁移 tablet

4.9.3.11.1 Request

```
GET /api/tablet_migration?goal={enum}&tablet_id={int}&schema_hash={int}&disk={string}
```

4.9.3.11.2 Description

在 BE 节点上迁移单个 tablet 到指定磁盘

4.9.3.11.3 Query parameters

- goal
  - run：提交迁移任务
  - status：查询任务的执行状态
- tablet_id 需要迁移的 tablet 的 id
- schema_hash schema hash
- disk 目标磁盘。

4.9.3.11.4 Request body

无

4.9.3.11.5 Response

提交结果

```
{
    status: "Success",
    msg: "migration task is successfully submitted."
}
```

或

```
{
    status: "Fail",
    msg: "Migration task submission failed"
}
```

执行状态

```
{
    status: "Success",
    msg: "migration task is running",
    dest_disk: "xxxxxx"
}
```

或

```
{
    status: "Success",
    msg: "migration task has finished successfully",
    dest_disk: "xxxxxx"
}
```

或

```
{
    status: "Success",
    msg: "migration task failed.",
    dest_disk: "xxxxxx"
}
```

4.9.3.11.6 Examples

```
curl "http://127.0.0.1:8040/api/tablet_migration?goal=run&tablet_id=123&schema_hash=333&disk
    ↪ =/disk1"
```

4.9.3.12 查询 tablet 信息

4.9.3.12.1 Request

`GET /tablets_json?limit={int}`

4.9.3.12.2 Description

获取特定 BE 节点上指定数量的 tablet 的 tablet id 和 schema hash 信息

4.9.3.12.3 Query parameters

- `limit` 返回的 tablet 数量，选填，默认 1000 个，可填`all`返回全部 tablet。

4.9.3.12.4 Request body

无

4.9.3.12.5 Response

```
{
    msg: "OK",
    code: 0,
    data: {
        host: "10.38.157.107",
        tablets: [
            {
                tablet_id: 11119,
                schema_hash: 714349777
            },

                ...

            {
                tablet_id: 11063,
                schema_hash: 714349777
            }
        ]
    },
    count: 30
}
```

4.9.3.12.6 Examples

```
curl http://127.0.0.1:8040/api/tablets_json?limit=123
```

4.9.3.13 Checksum

4.9.3.13.1 Request

```
GET /api/checksum?tablet_id={int}&version={int}&schema_hash={int}
```

4.9.3.13.2 Description

checksum

4.9.3.13.3 Query parameters

- `tablet_id` 需要校验的 tablet 的 id
- `version` 需要校验的 tablet 的 version
- `schema_hash` schema hash

4.9.3.13.4 Request body

无

4.9.3.13.5 Response

```
    1843743562
```

4.9.3.13.6 Examples

```
    curl "http://127.0.0.1:8040/api/checksum?tablet_id=1&version=1&schema_hash=-1"
```

4.9.3.14 下载 load 日志

4.9.3.14.1 Request

```
GET /api/_load_error_log?token={string}&file={string}
```

4.9.3.14.2 Description

下载 load 错误日志文件。

4.9.3.14.3 Query parameters

- `file` 文件路径
- token token

4.9.3.14.4 Request body

无

4.9.3.14.5 Response

```
文件
```

4.9.3.14.6 Examples

```
    curl "http://127.0.0.1:8040/api/_load_error_log?file=a&token=1"
```

4.9.3.15 填充坏副本

4.9.3.15.1 Request

```
POST /api/pad_rowset?tablet_id={int}&start_version={int}&end_version={int}
```

4.9.3.15.2 Description

该功能用于使用一个空的 rowset 填充损坏的副本。

4.9.3.15.3 Query parameters

- `tablet_id` table 的 id
- `start_version` 起始版本
- `end_version` 终止版本

4.9.3.15.4 Request body

无

4.9.3.15.5 Response

```
{
    msg: "OK",
    code: 0
}
```

4.9.3.15.6 Examples

```
curl -X POST "http://127.0.0.1:8040/api/pad_rowset?tablet_id=123456&start_version=1111111&end
    ↪ _version=1111112"
```

4.9.3.16 BE 版本信息

4.9.3.16.1 Request

```
GET /api/be_version_info
```

4.9.3.16.2 Description

用于获取 be 节点的版本信息。

4.9.3.16.3 Path parameters

无

4.9.3.16.4 Query parameters

无

4.9.3.16.5 Request body

无

4.9.3.16.6 Response

```
{
    "msg":"success",
    "code":0,
    "data":{
        "beVersionInfo":{
            "dorisBuildVersionPrefix":"doris",
            "dorisBuildVersionMajor":0,
            "dorisBuildVersionMinor":0,
            "dorisBuildVersionPatch":0,
            "dorisBuildVersionRcVersion":"trunk",
            "dorisBuildVersion":"doris-0.0.0-trunk",
            "dorisBuildHash":"git://4b7b503d1cb3/data/doris/doris/be/../
                ↪ @a04f9814fe5a09c0d9e9399fe71cc4d765f8bff1",
            "dorisBuildShortHash":"a04f981",
            "dorisBuildTime":"Fri, 09 Sep 2022 07:57:02 UTC",
            "dorisBuildInfo":"root@4b7b503d1cb3"
        }
    },
    "count":0
}
```

4.9.3.16.7 Examples

```
curl http://127.0.0.1:8040/api/be_version_info
```

4.9.3.17 BE 探活

4.9.3.17.1 Request

```
GET /api/health
```

4.9.3.17.2 Description

给监控服务提供的探活接口，请求能响应代表 BE 状态正常。

4.9.3.17.3 Query parameters

无

4.9.3.17.4 Request body

无

4.9.3.17.5 Response

```
{"status": "OK","msg": ""}
```

4.9.3.17.6 Examples

```
curl http://127.0.0.1:8040/api/health
```

4.9.3.18 重加载 tablet

4.9.3.18.1 Request

```
GET /api/reload_tablet?tablet_id={int}&schema_hash={int}&path={string}"
```

4.9.3.18.2 Description

该功能用于重加载 tablet 数据。

4.9.3.18.3 Query parameters

- `tablet_id` 需要重加载的 table 的 id
- `schema_hash` schema hash
- `path` 文件路径

4.9.3.18.4 Request body

无

4.9.3.18.5 Response

```
load header succeed
```

4.9.3.18.6 Examples

```
curl "http://127.0.0.1:8040/api/reload_tablet?tablet_id=123456&schema_hash=1111111&path=/abc"
```

4.9.3.19 恢复 tablet

4.9.3.19.1 Request

```
POST /api/restore_tablet?tablet_id={int}&schema_hash={int}"
```

4.9.3.19.2 Description

该功能用于恢复 trash 目录中被误删的 tablet 数据。

4.9.3.19.3 Query parameters

- `tablet_id` 需要恢复的 table 的 id
- `schema_hash` schema hash

4.9.3.19.4 Request body

无

4.9.3.19.5 Response

```
{
    msg: "OK",
    code: 0
}
```

4.9.3.19.6 Examples

```
curl -X POST "http://127.0.0.1:8040/api/restore_tablet?tablet_id=123456&schema_hash=1111111"
```

4.9.3.20 调整 BE VLOG

4.9.3.20.1 Request

```
POST /api/glog/adjust?module=<module_name>&level=<level_number>
```

4.9.3.20.2 Description

该功能用于动态调整 BE 端 VLOG 日志。

4.9.3.20.3 Query parameters

- `module_name` 要设置 VLOG 的模块，对应 BE 无后缀名的文件名
- `level_number` VLOG 级别，从 1 到 10，另外 -1 为关闭

4.9.3.20.4 Request body

无

4.9.3.20.5 Response

```
{
    msg: "adjust vlog of xxx from -1 to 10 succeed",
    code: 0
}
```

4.9.3.20.6 Examples

```
curl -X POST "http://127.0.0.1:8040/api/glog/adjust?module=vrow_distribution&level=-1"
```

5 生态扩展

5.1 Spark Doris Connector

Spark Doris Connector 可以支持通过 Spark 读取 Doris 中存储的数据，也支持通过 Spark 写入数据到 Doris。

代码库地址：https://github.com/apache/doris-spark-connector

- 支持从 Doris 中通过 RDD、 DataFrame 以及 Spark SQL 方式批量读取数据, 推荐使用 DataFrame 或 Spark SQL。
- 支持使用 DataFrame 和 Spark SQL 批量或流式地将数据写入 Doris。
- 支持在 Doris 端完成数据过滤，减少数据传输量。

5.1.1 版本兼容

Connector	Spark	Doris	Java	Scala
25.1.0	3.5 - 3.1, 2.4	1.0 +	8	2.12, 2.11
25.0.1	3.5 - 3.1, 2.4	1.0 +	8	2.12, 2.11
25.0.0	3.5 - 3.1, 2.4	1.0 +	8	2.12, 2.11
1.3.2	3.4 - 3.1, 2.4, 2.3	1.0 - 2.1.6	8	2.12, 2.11
1.3.1	3.4 - 3.1, 2.4, 2.3	1.0 - 2.1.0	8	2.12, 2.11
1.3.0	3.4 - 3.1, 2.4, 2.3	1.0 - 2.1.0	8	2.12, 2.11
1.2.0	3.2, 3.1, 2.3	1.0 - 2.0.2	8	2.12, 2.11
1.1.0	3.2, 3.1, 2.3	1.0 - 1.2.8	8	2.12, 2.11
1.0.1	3.1, 2.3	0.12 - 0.15	8	2.12, 2.11

5.1.2 使用

5.1.2.1 Maven

```
<dependency>
    <groupId>org.apache.doris</groupId>
    <artifactId>spark-doris-connector-spark-3.5</artifactId>
    <version>25.1.0</version>
</dependency>
```

> tip
>
> 从 24.0.0 版本开始，Doris connector 包命名规则发生调整：1. 不再包含 Scala 版本信息 2. 对于 Spark 2.x 版本，统一使用名称为 spark-doris-connector-spark-2 的包，并且默认只基于 Scala 2.11 版本编译，需要 Scala 2.12 版本的请自行编译。3. 对于 Spark 3.x 版本，根据具体 Spark 版本使用使用名称为 spark-doris-connector-spark-3.x 的包，其中 Spark 3.0 版本可以使用 spark-doris-connector-spark-3.1 的包。

备注

1. 请根据不同的 Spark 和 Scala 版本替换相应的 Connector 版本。
2. 也可从这里下载相关版本 jar 包。

5.1.2.2 编译

编译时，可直接运行 sh build.sh，具体可参考这里。

编译成功后，会在 dist 目录生成目标 jar 包，如：spark-doris-connector-spark-3.5-25.1.0.jar。将此文件复制到 Spark 的 ClassPath 中即可使用 Spark-Doris-Connector。例如，Local 模式运行的 Spark，将此文件放入 jars/ 文件夹下。Yarn集群模式运行的Spark，则将此文件放入预部署包中。也可以

2. 在源码目录下执行：sh build.sh 根据提示输入你需要的 Scala 与 Spark 版本进行编译。

编译成功后，会在 dist 目录生成目标 jar 包，如：spark-doris-connector-spark-3.5-25.1.0.jar。将此文件复制到 Spark 的 ClassPath 中即可使用 Spark-Doris-Connector。

例如，Local 模式运行的 Spark，将此文件放入 jars/ 文件夹下。Yarn集群模式运行的Spark，则将此文件放入预部署包中。

例如将 spark-doris-connector-spark-3.5-25.1.0.jar 上传到 hdfs 并在 spark.yarn.jars 参数上添加 hdfs 上的 Jar 包路径

```
1. 上传 `spark-doris-connector-spark-3.5-25.1.0.jar` 到 hdfs。

hdfs dfs -mkdir /spark-jars/
```

```
hdfs dfs -put /your_local_path/spark-doris-connector-spark-3.5-25.1.0.jar /spark-jars/

2. 在集群中添加 `spark-doris-connector-spark-3.5-25.1.0.jar` 依赖。
spark.yarn.jars=hdfs:///spark-jars/spark-doris-connector-spark-3.5-25.1.0.jar
```

5.1.3 使用示例

5.1.3.1 批量读取

5.1.3.1.1 RDD

```
import org.apache.doris.spark._

val dorisSparkRDD = sc.dorisRDD(
   tableIdentifier = Some("$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME"),
   cfg = Some(Map(
      "doris.fenodes" -> "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT",
      "doris.request.auth.user" -> "$YOUR_DORIS_USERNAME",
      "doris.request.auth.password" -> "$YOUR_DORIS_PASSWORD"
   ))
)

dorisSparkRDD.collect()
```

5.1.3.1.2 DataFrame

```
val dorisSparkDF = spark.read.format("doris")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
        .load()

dorisSparkDF.show(5)
```

5.1.3.1.3 Spark SQL

```
CREATE TEMPORARY VIEW spark_doris
   USING doris
   OPTIONS(
   "table.identifier"="$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME",
   "fenodes"="$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT",
   "user"="$YOUR_DORIS_USERNAME",
```

```
   "password"="$YOUR_DORIS_PASSWORD"
);

SELECT * FROM spark_doris;
```

5.1.3.1.4 pySpark

```
dorisSparkDF = spark.read.format("doris")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
        .load()
// show 5 lines data
dorisSparkDF.show(5)
```

5.1.3.1.5 通过 Arrow Flight SQL 方式读取

从 24.0.0 版本开始，支持通过 Arrow Flight SQL 方式读取数据（需要 Doris 版本 >= 2.1.0 ）。

设置 doris.read.mode 为 arrow，设置 doris.read.arrow-flight-sql.port 为 FE 配置的 Arrow Flight SQL 端口，服务端配置方式参考 基于 Arrow Flight SQL 的高速数据传输链路。

```
val df = spark.read.format("doris")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("doris.user", "$YOUR_DORIS_USERNAME")
        .option("doris.password", "$YOUR_DORIS_PASSWORD")
        .option("doris.read.mode", "arrow")
        .option("doris.read.arrow-flight-sql.port", "12345")
        .load()

df.show()
```

5.1.3.2 批量写入

5.1.3.2.1 DataFrame

```
val mockDataDF = List(
   (3, "440403001005", "21.cn"),
   (1, "4404030013005", "22.cn"),
   (33, null, "23.cn")
).toDF("id", "mi_code", "mi_name")
mockDataDF.show(5)
```

```
mockDataDF.write.format("doris")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
        //其他选项
        //指定要写入的列
        .option("doris.write.fields", "$YOUR_FIELDS_TO_WRITE")
        // 从 1.3.0 版本开始，支持覆盖写入
        // .mode(SaveMode.Overwrite)
        .save()
```

5.1.3.2.2 Spark SQL

```
CREATE TEMPORARY VIEW spark_doris
   USING doris
   OPTIONS(
   "table.identifier"="$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME",
   "fenodes"="$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT",
   "user"="$YOUR_DORIS_USERNAME",
   "password"="$YOUR_DORIS_PASSWORD"
);

INSERT INTO spark_doris VALUES ("VALUE1", "VALUE2", ...);
-- insert into select
INSERT INTO spark_doris SELECT * FROM YOUR_TABLE;
-- insert overwrite
INSERT OVERWRITE SELECT * FROM YOUR_TABLE;
```

5.1.3.3 流式写入

5.1.3.3.1 DataFrame

结构化数据写入

```
val df = spark.readStream.format("your_own_stream_source").load()

df.writeStream
        .format("doris")
        .option("checkpointLocation", "$YOUR_CHECKPOINT_LOCATION")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
```

```
        .start()
        .awaitTermination()
```

直接写入

如果数据流的第一列数据符合 Doris 表结构，比如列顺序相同的 CSV 数据，或者字段名一致的 JSON 数据，可以通过设置 doris.sink.streaming.passthrough 选项为 true 来直接将这列数据写入，而不用再转换为 DataFrame

以 Kafka 源为例。

假设要写入的表结构如下：

```
CREATE TABLE `t2` (
                    `c0` int NULL,
                    `c1` varchar(10) NULL,
                    `c2` date NULL
) ENGINE=OLAP
DUPLICATE KEY(`c0`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`c0`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

消息的 value 为 {"c0":1,"c1":"a","dt":"2024-01-01"} 格式的 JSON 数据。

```
val kafkaSource = spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "$YOUR_KAFKA_SERVERS")
        .option("startingOffsets", "latest")
        .option("subscribe", "$YOUR_KAFKA_TOPICS")
        .load()

// 选择 value 为 DataFrame 的第一列
kafkaSource.selectExpr("CAST(value as STRING)")
        .writeStream
        .format("doris")
        .option("checkpointLocation", "$YOUR_CHECKPOINT_LOCATION")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
        // 设置此选项为 true，会将 DataFrame 的第一列直接写入
        .option("doris.sink.streaming.passthrough", "true")
        .option("doris.sink.properties.format", "json")
        .start()
        .awaitTermination()
```

5.1.3.3.2 以 JSON 格式写入

设置 doris.sink.properties.format 为 json

```
val df = spark.readStream.format("your_own_stream_source").load()

df.write.format("doris")
        .option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
        .option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
        .option("user", "$YOUR_DORIS_USERNAME")
        .option("password", "$YOUR_DORIS_PASSWORD")
        .option("doris.sink.properties.format", "json")
        .save()
```

5.1.3.4 Spark Doris Catalog

从 24.0.0 版本开始, 支持通过 Spark Catalog 方式访问 Doris。

5.1.3.4.1 Catalog Config

选项名称	是否必须	注释
spark.sql.catalog.your_catalog_name	是	设置 Catalog 提供者的类名, 对于 Doris 来说唯一的有效值为 org.apache.doris.spark.catalog.DorisTableCatalog。
spark.sql.catalog.your_catalog_name.doris.fenodes	是	设置 Doris FE 节点，格式为 fe_ip:fe_http_port。
spark.sql.catalog.your_catalog_name.doris.query.port	否	设置 Doris FE 查询端口, 当 spark.sql.catalog.your_catalog_name.doris.fe.auto.fetch 为 true 时，此选项可以不设置。
spark.sql.catalog.your_catalog_name.doris.user	是	设置 Doris 用户。
spark.sql.catalog.your_catalog_name.doris.password	是	设置 Doris 密码。
spark.sql.defaultCatalog	否	设置 Spark SQL 默认 catalog。

> 所有适用于 DataFrame 和 Spark SQL 的连接器参数都可以为 catalog 设置。
> 例如，如果要以 json 格式写入数据，可以将选项 spark.sql.catalog.your_catalog_name.doris.sink.properties.format 设置为 json。

5.1.3.4.2 DataFrame

```
val conf = new SparkConf()
```

```
conf.set("spark.sql.catalog.your_catalog_name", "org.apache.doris.spark.catalog.DorisTableCatalog
    ↪ ")
conf.set("spark.sql.catalog.your_catalog_name.doris.fenodes", "192.168.0.1:8030")
conf.set("spark.sql.catalog.your_catalog_name.doris.query.port", "9030")
conf.set("spark.sql.catalog.your_catalog_name.doris.user", "root")
conf.set("spark.sql.catalog.your_catalog_name.doris.password", "")
val spark = builder.config(conf).getOrCreate()
spark.sessionState.catalogManager.setCurrentCatalog("your_catalog_name")

// show all databases
spark.sql("show databases")

// use databases
spark.sql("use your_doris_db")

// show tables in test
spark.sql("show tables")

// query table
spark.sql("select * from your_doris_table")

// write data
spark.sql("insert into your_doris_table values(xxx)")
```

5.1.3.4.3 Spark SQL

设置必要参数并启动 Spark SQL CLI.

```
spark-sql \
--conf "spark.sql.catalog.your_catalog_name=org.apache.doris.spark.catalog.DorisTableCatalog" \
--conf "spark.sql.catalog.your_catalog_name.doris.fenodes=192.168.0.1:8030" \
--conf "spark.sql.catalog.your_catalog_name.doris.query.port=9030" \
--conf "spark.sql.catalog.your_catalog_name.doris.user=root" \
--conf "spark.sql.catalog.your_catalog_name.doris.password=" \
--conf "spark.sql.defaultCatalog=your_catalog_name"
```

在 Spark SQL CLI 中执行查询.

```
-- show all databases
show databases;

-- use databases
use your_doris_db;

// show tables in test
show tables;
```

```
-- query table
select * from your_doris_table;

-- write data
insert into your_doris_table values(xxx);
insert into your_doris_table select * from your_source_table;

-- access table with full name
select * from your_catalog_name.your_doris_db.your_doris_table;
insert into your_catalog_name.your_doris_db.your_doris_table values(xxx);
insert into your_catalog_name.your_doris_db.your_doris_table select * from your_source_table;
```

5.1.3.5 Java 示例

`samples/doris-demo/spark-demo/` 下提供了 Java 版本的示例，可供参考，这里

5.1.4 配置

5.1.4.1 通用配置项

Key	Default Value	Comment
doris.fenodes	–	Doris FE http 地址，支持多个地址，使用逗号分隔
doris.table.identifier	–	Doris 表名，如：db1.tbl1
doris.user	–	访问 Doris 的用户名
doris.password	空字符串	访问 Doris 的密码
doris.request.retries	3	向 Doris 发送请求的重试次数
doris.request.connect.timeout.ms	30000	向 Doris 发送请求的连接超时时间
doris.request.read.timeout.ms	30000	向 Doris 发送请求的读取超时时间
doris.request.query.timeout.s	21600	查询 doris 的超时时间，默认值为 6 小时，-1 表示无超时限制
doris.request.tablet.size	1	一个 RDD Partition 对应的 Doris Tablet 个数。此数值设置越小，则会生成越多的 Partition。从而提升 Spark 侧的并行度，但同时会对 Doris 造成更大的压力。
doris.read.field	–	读取 Doris 表的列名列表，多列之间使用逗号分隔
doris.batch.size	4064	一次从 BE 读取数据的最大行数。增大此数值可减少 Spark 与 Doris 之间建立连接的次数。从而减轻网络延迟所带来的额外时间开销。
doris.exec.mem.limit	8589934592	单个查询的内存限制。默认为 8GB，单位为字节
doris.write.fields	–	指定写入 Doris 表的字段或者字段顺序，多列之间使用逗号分隔。默认写入时要按照 Doris 表字段顺序写入全部字段。
doris.sink.batch.size	500000	单次写 BE 的最大行数
doris.sink.max-retries	0	写 BE 失败之后的重试次数，从 1.3.0 版本开始，默认值为 0，即默认不进行重试。当设置该参数大于 0 时，会进行批次级别的失败重试，会在 Spark Executor 内存中缓存 `doris.sink.batch.size` 所配置大小的数据，可能需要适当增大内存分配。

Key	Default Value	Comment
doris.sink.retry.interval.ms	10000	配置重试次数之后，每次重试的间隔，单位 ms
doris.sink.properties.format	csv	Stream Load 的数据格式。共支持 3 种格式：csv，json，arrow 更多参数详情
doris.sink.properties.*	-	Stream Load 的导入参数。例如: 指定列分隔符：`'doris.sink.properties.column_separator' = ','`等 更多参数详情
doris.sink.task.partition.size	-	Doris 写入任务对应的 Partition 个数。Spark RDD 经过过滤等操作，最后写入的 Partition 数可能会比较大，但每个 Partition 对应的记录数比较少，导致写入频率增加和计算资源浪费。此数值设置越小，可以降低 Doris 写入频率，减少 Doris 合并压力。该参数配合 doris.sink.task.use.repartition 使用。
doris.sink.task.use.repartition	false	是否采用 repartition 方式控制 Doris 写入 Partition 数。默认值为 false，采用 coalesce 方式控制（注意：如果在写入之前没有 Spark action 算子，可能会导致整个计算并行度降低）。如果设置为 true，则采用 repartition 方式（注意：可设置最后 Partition 数，但会额外增加 shuffle 开销）。
doris.sink.batch.interval.ms	0	每个批次 sink 的间隔时间，单位 ms。
doris.sink.enable-2pc	false	是否开启两阶段提交。开启后将会在作业结束时提交事务，而部分任务失败时会将所有预提交状态的事务会滚。
doris.sink.auto-redirect	true	是否重定向 StreamLoad 请求。开启后 StreamLoad 将通过 FE 写入，不再显式获取 BE 信息。
doris.enable.https	false	是否开启 FE Https 请求。
doris.https.key-store-path	-	Https key store 路径。
doris.https.key-store-type	JKS	Https key store 类型。
doris.https.key-store-password	-	Https key store 密码。
doris.read.mode	thrift	Doris 读取模式，可选项 `thrift` 和 `arrow`。
doris.read.arrow-flight-sql.port	-	Doris FE 的 Arrow Flight SQL 端口，当 `doris.read.mode` 为 `arrow` 时，用于通过 Arrow Flight SQL 方式读取数据。服务端配置方式参考 基于 Arrow Flight SQL 的高速数据传输链路
doris.sink.label.prefix	spark-doris	Stream Load 方式写入时的导入标签前缀。
doris.thrift.max.message.size	2147483647	通过 Thrift 方式读取数据时，消息的最大尺寸。
doris.fe.auto.fetch	false	是否自动获取 FE 信息，当设置为 true 时，会根据 `doris.fenodes` 配置的节点请求所有 FE 节点信息，无需额外配置多个节点以及单独配置 `doris.read.arrow-flight-sql.port` 和 `doris.query.port`。
doris.read.bitmap-to-string	false	是否将 Bitmap 类型转换为数组索引组成的字符串读取。具体结果形式参考函数定义 BITMAP_TO_STRING。
doris.read.bitmap-to-base64	false	是否将 Bitmap 类型转换为 Base64 编码后的字符串读取。具体结果形式参考函数定义 BITMAP_TO_BASE64。
doris.query.port	-	Doris FE 查询端口，用于覆盖写入以及 Catalog 的元数据获取。

5.1.4.2 SQL 和 Dataframe 专有配置

Key	Default Value	Comment
doris.filter.query.in.max.count	10000	谓词下推中，in 表达式 value 列表元素最大数量。超过此数量，则 in 表达式条件过滤在 Spark 侧处理。

5.1.4.3 Structured Streaming 专有配置

Key	Default Value	Comment
doris.sink.streaming.passthrough	false	将第一列的值不经过处理直接写入。

5.1.4.4 RDD 专有配置

Key	Default Value	Comment
doris.request.auth.user	–	访问 Doris 的用户名
doris.request.auth.password	–	访问 Doris 的密码
doris.filter.query	–	过滤读取数据的表达式，此表达式透传给 Doris。Doris 使用此表达式完成源端数据过滤

5.1.5 Doris 和 Spark 列类型映射关系

Doris Type	Spark Type
NULL_TYPE	DataTypes.NullType
BOOLEAN	DataTypes.BooleanType
TINYINT	DataTypes.ByteType
SMALLINT	DataTypes.ShortType
INT	DataTypes.IntegerType
BIGINT	DataTypes.LongType
FLOAT	DataTypes.FloatType
DOUBLE	DataTypes.DoubleType
DATE	DataTypes.DateType
DATETIME	DataTypes.TimestampType
DECIMAL	DecimalType
CHAR	DataTypes.StringType
LARGEINT	DecimalType
VARCHAR	DataTypes.StringType
STRING	DataTypes.StringType
JSON	DataTypes.StringType
VARIANT	DataTypes.StringType
TIME	DataTypes.DoubleType
HLL	DataTypes.StringType
Bitmap	DataTypes.StringType

从 24.0.0 版本开始，Bitmap 类型读取返回类型为字符串，默认返回字符串值 Read unsupported。

5.1.6 常见问题

1. 如何写入 Bitmap 类型？

在 Spark SQL 中，通过 insert into 方式写入数据时，如果 doris 的目标表中包含 BITMAP 或 HLL 类型的数据时，需要设置参数 doris.ignore-type 为对应类型，并通过 doris.write.fields 对列进行映射转换，使用方式如下：

BITMAP

```
CREATE TEMPORARY VIEW spark_doris
USING doris
OPTIONS(
"table.identifier"="$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME",
"fenodes"="$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT",
"user"="$YOUR_DORIS_USERNAME",
"password"="$YOUR_DORIS_PASSWORD"
"doris.ignore-type"="bitmap",
"doris.write.fields"="col1,col2,col3,bitmap_col2=to_bitmap(col2),bitmap_col3=bitmap_hash(col3)
    ↪ "
);
```

HLL

```
CREATE TEMPORARY VIEW spark_doris
USING doris
OPTIONS(
"table.identifier"="$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME",
"fenodes"="$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT",
"user"="$YOUR_DORIS_USERNAME",
"password"="$YOUR_DORIS_PASSWORD"
"doris.ignore-type"="hll",
"doris.write.fields"="col1,hll_col1=hll_hash(col1)"
);
```

从 24.0.0 版本开始，doris.ignore-type 被废除，写入时无需添加该参数。

2. 如何使用 overwrite 写入？

从 1.3.0 版本开始，支持 overwrite 模式写入（只支持全表级别的数据覆盖），具体使用方式如下 DataFrame

```
resultDf.format("doris")
  .option("doris.fenodes","$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
  // your own options
  .mode(SaveMode.Overwrite)
  .save()
```

SQL

```
INSERT OVERWRITE your_target_table SELECT * FROM your_source_table;
```

3. 如何读取 Bitmap 类型

从 24.0.0 版本开始，支持通过 Arrow Flight SQL 方式读取转换后的 Bitmap 数据（需要 Doris 版本 >= 2.1.0)。

Bitmap to string

以 DataFrame 方式为例，设置 doris.read.bitmap-to-string 为 true, 具体结果格式见选项定义。

```
spark.read.format("doris")
.option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
.option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
.option("user", "$YOUR_DORIS_USERNAME")
.option("password", "$YOUR_DORIS_PASSWORD")
.option("doris.read.bitmap-to-string","true")
.load()
```

Bitmap to base64

以 DataFrame 方式为例，设置 doris.read.bitmap-to-base64 为 true, 具体结果格式见选项定义。

```
spark.read.format("doris")
.option("doris.table.identifier", "$YOUR_DORIS_DATABASE_NAME.$YOUR_DORIS_TABLE_NAME")
.option("doris.fenodes", "$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
.option("user", "$YOUR_DORIS_USERNAME")
.option("password", "$YOUR_DORIS_PASSWORD")
.option("doris.read.bitmap-to-base64","true")
.load()
```

4. DataFrame方式写入时报错：org.apache.spark.sql.AnalysisException: TableProvider implementation
   ↪ doris cannot be written with ErrorIfExists mode, please use Append or Overwrite modes
   ↪ instead.

需要添加 save mode 为 append。

```
resultDf.format("doris")
 .option("doris.fenodes","$YOUR_DORIS_FE_HOSTNAME:$YOUR_DORIS_FE_RESFUL_PORT")
 // your own options
 .mode(SaveMode.Append)
 .save()
```

5.2 Flink Doris Connector

Flink Doris Connector是通过 Flink 来读取和写入数据到 Doris 集群，同时集成了FlinkCDC，可以更便捷的对上游 MySQL 等数据库进行整库同步。

使用 FlinkConnector 可以完成以下操作：

- 读取 Doris 中的数据：Flink Connector 支持从 BE 中并行读取数据，提高了数据读取的效率；
- 向 Doris 中写入数据：在 Flink 中进行攒批后，通过 Stream Load 批量导入到 Doris 中；
- 使用 Lookup Join 方式进行维表关联：通过攒批与异步查询加速维表关联的性能；
- 整库同步：通过 FlinkCDC 完成 MySQL、Oracle、PostgreSQL 等数据库的整库同步，包含自动建表与 DDL 操作。

5.2.1 版本说明

Connector Version	Flink Version	Doris Version	Java Version	Scala Version
1.0.3	1.11,1.12,1.13,1.14	0.15+	8	2.11,2.12
1.1.1	1.14	1.0+	8	2.11,2.12
1.2.1	1.15	1.0+	8	-
1.3.0	1.16	1.0+	8	-
1.4.0	1.15,1.16,1.17	1.0+	8	-
1.5.2	1.15,1.16,1.17,1.18	1.0+	8	-
1.6.1	1.15,1.16,1.17,1.18,1.19	1.0+	8	-
24.0.1	1.15,1.16,1.17,1.18,1.19,1.20	1.0+	8	-
25.0.0	1.15,1.16,1.17,1.18,1.19,1.20	1.0+	8	-
25.1.0	1.15,1.16,1.17,1.18,1.19,1.20	1.0+	8	-

5.2.2 使用方式

可以分别使用 Jar 方式和 Maven 方式使用 Flink Doris Connector。

5.2.2.0.1 Jar

可在这里下载 Flink Doris Connector 对应版本的 Jar 包，将此文件复制到 `Flink` 的 `classpath` 中即可使用 `Flink-Doris-Connector` 。`Standalone` 模式运行的 `Flink` ，将此文件放入 `lib/` 文件夹下。`Yarn` 集群模式运行的 `Flink` ，则将此文件放入预部署包中。

5.2.2.0.2 Maven

Maven 中使用的时候，可以直接在 Pom 文件中加入如下依赖

```
<dependency>
  <groupId>org.apache.doris</groupId>
  <artifactId>flink-doris-connector-${flink.version}</artifactId>
```

```
  <version>${connector.version}</version>
</dependency>
```

例如：

```
<dependency>
  <groupId>org.apache.doris</groupId>
  <artifactId>flink-doris-connector-1.16</artifactId>
  <version>25.1.0</version>
</dependency>
```

5.2.3 使用原理

5.2.3.1 从 Doris 中读取数据

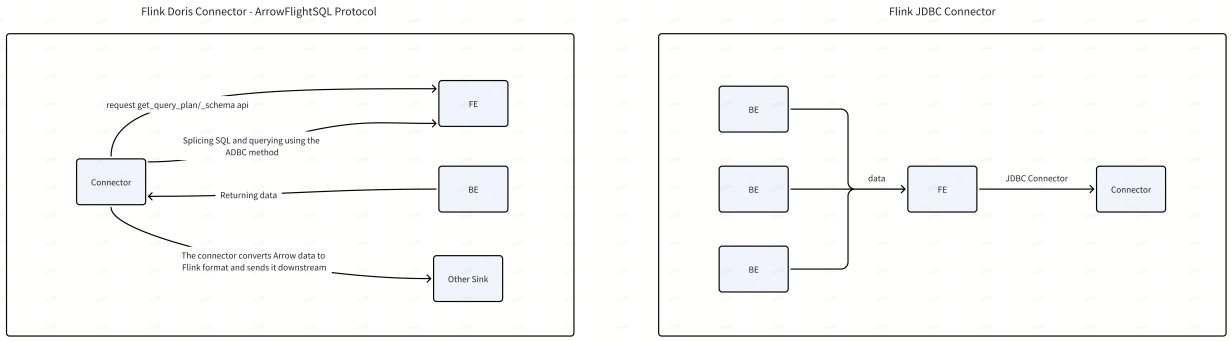


图 154: FlinkConnectorPrinciples-JDBC-Doris

在读取数据时，相较于 Flink JDBC Connector，Flink Doris Connector 具备更高的性能，推荐优先使用：

- Flink JDBC Connector：虽然 Doris 兼容 MySQL 协议，但不建议通过 Flink JDBC Connector 读写 Doris 集群。此方式会导致数据在单个 FE 节点上串行读写，形成瓶颈，影响性能。
- Flink Doris Connector：自 Doris 2.1 版本后，默认使用 ADBC 协议作为 Flink Doris Connector 读取协议，读取时经过以下步骤：

a. Flink Doris Connector 首先从 FE 获取查询计划中的 Tablet ID 信息

b. 生成查询语句 SELECT * FROM tbs TABLET(id1, id2, id3)

c. 然后通过 FE 的 ADBC 端口执行查询

d. 由 BE 直接返回数据，避免数据流经 FE，从而消除 FE 单点瓶颈

5.2.3.2 向 Doris 中写入数据

在使用 Flink Doris Connector 写入数据时，会在 Flink 内存中进行攒批操作，在通过 Stream Load 批量导入。Doris Flink Connector 提供了两种攒批模式，默认使用基于 Flink Checkpoint 的流式写入方式：

	流式写入	批量写入
触发条件	依赖 Flink 的 Checkpoint，跟随 Flink 的 Checkpoint 周期写入到 Doris 中	基于 Connector 内的时间阈值、数据量阈值进行周期性提交，写入到 Doris 中
一致性	Exactly-Once	At-Least-Once，基于主键模型可以保证 Exactly-Once
延迟	受 Checkpoint 时间间隔限制，通常较高	独立的批处理机制，灵活调整
容错与恢复	与 Flink 状态恢复完全一致	依赖外部去重逻辑（如 Doris 主键去重）

5.2.4 快速上手

5.2.4.0.1 准备工作

5.2.4.0.2 Flink 集群部署

以 Standalone 集群为例：

1. 下载 Flink 的安装包，Flink 1.18.1;
2. 解压后，将 Flink Doris Connector 包放到 /lib 下;
3. 进入目录，运行 bin/start-cluster.sh 启动 Flink 集群;
4. 可通过 jps 命令验证 Flink 集群是否成功启动。

5.2.4.0.3 初始化 Doris 表

运行以下语句创建 Doris 表

```
CREATE DATABASE test;

CREATE TABLE test.student (
  `id` INT,
  `name` VARCHAR(256),
  `age` INT
)
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
```

```
"replication_allocation" = "tag.location.default: 3"
);

INSERT INTO test.student values(1,"James",18);
INSERT INTO test.student values(2,"Emily",28);

CREATE TABLE test.student_trans (
  `id` INT,
  `name` VARCHAR(256),
  `age` INT
)
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 3"
);
```

5.2.4.0.4 运行 FlinkSQL 任务

启动 FlinkSQL Client

```
bin/sql-client.sh
```

运行 FlinkSQL

```
CREATE TABLE Student (
    id STRING,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '127.0.0.1:8030',
      'table.identifier' = 'test.student',
      'username' = 'root',
      'password' = ''
);

CREATE TABLE StudentTrans (
    id STRING,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '127.0.0.1:8030',
```

```
      'table.identifier' = 'test.student_trans',
      'username' = 'root',
      'password' = '',
      'sink.label-prefix' = 'doris_label'
);

INSERT INTO StudentTrans SELECT id, concat('prefix_',name), age+1 FROM Student;
```

5.2.4.0.5 查询数据

```
mysql> select * from test.student_trans;
+------+--------------+------+
| id   | name         | age  |
+------+--------------+------+
|    1 | prefix_James |   19 |
|    2 | prefix_Emily |   29 |
+------+--------------+------+
2 rows in set (0.02 sec)
```

5.2.5 场景与操作

5.2.5.1 读取 Doris 中的数据

Flink 读取 Doris 中数据时，目前 Doris Source 是有界流，不支持以 CDC 的方式持续读取。可以通过 Thrift 和 ArrowFlightSQL 方式 (24.0.0 版本之后支持) 读取 Doris 中数据，2.1 版本后推荐使用 ArrowFlightSQL 方式：

- Thrift：通过调用 BE 的 thrift 接口读取数据，具体流程可参考 通过 Thrift 接口读取数据
- ArrowFlightSQL：基于 Doris2.1，通过 Arrow Flight SQL 协议高速读取大批量数据，具体可参考 基于 Arrow Flight SQL 的高速数据传输链路。

5.2.5.1.1 使用 FlinkSQL 读取数据

Thrift 方式

```
CREATE TABLE student (
    id INT,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '127.0.0.1:8030',  -- Fe的host:HttpPort
      'table.identifier' = 'test.student',
      'username' = 'root',
      'password' = ''
```

```
);

SELECT * FROM student;
```

ArrowFlightSQL 方式

```
CREATE TABLE student (
    id INT,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '{fe.conf:http_port}',
      'table.identifier' = 'test.student',
      'source.use-flight-sql' = 'true',
      'source.flight-sql-port' = '{fe.conf:arrow_flight_sql_port}',
      'username' = 'root',
      'password' = ''
);

SELECT * FROM student;
```

5.2.5.1.2 使用 DataStream API 读取数据

使用 DataStream API 读取时，需要提前在程序 POM 文件中引入依赖，参考使用方式章节

```
final StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
DorisOptions option = DorisOptions.builder()
        .setFenodes("127.0.0.1:8030")
        .setTableIdentifier("test.student")
        .setUsername("root")
        .setPassword("")
        .build();

DorisReadOptions readOptions = DorisReadOptions.builder().build();
DorisSource<List<?>> dorisSource = DorisSource.<List<?>>builder()
        .setDorisOptions(option)
        .setDorisReadOptions(readOptions)
        .setDeserializer(new SimpleListDeserializationSchema())
        .build();

env.fromSource(dorisSource, WatermarkStrategy.noWatermarks(), "doris source").print();
env.execute("Doris Source Test");
```

完整代码参考：DorisSourceDataStream.java

5.2.5.2 向 Doris 中写入数据

Flink 写入使用 Stream Load 的方式进行写入，支持流式写入和攒批写入模式。

> 流式写入和攒批写入区别
>
> Connector1.5.0 之后支持攒批写入，攒批写入不依赖 Checkpoint，将数据缓存在内存中，根据攒批参数来控制写入时机。流式写入必须开启 Checkpoint，在整个 Checkpoint 期间持续的将上游数据写入到 Doris 中，不会一直将数据缓存在内存中。

5.2.5.2.1 使用 FlinkSQL 写入数据

写入测试使用 Flink 的 Datagen 来模拟上游持续产生的数据

```
-- enable checkpoint
SET 'execution.checkpointing.interval' = '30s';

CREATE TABLE student_source (
    id INT,
    name STRING,
    age INT
) WITH (
  'connector' = 'datagen',
  'rows-per-second' = '1',
  'fields.name.length' = '20',
  'fields.id.min' = '1',
  'fields.id.max' = '100000',
  'fields.age.min' = '3',
  'fields.age.max' = '30'
);

-- doris sink
CREATE TABLE student_sink (
    id INT,
    name STRING,
    age INT
    )
    WITH (
      'connector' = 'doris',
      'fenodes' = '10.16.10.6:28737',
      'table.identifier' = 'test.student',
      'username' = 'root',
      'password' = 'password',
      'sink.label-prefix' = 'doris_label'
      --'sink.enable.batch-mode' = 'true'  增加该配置可以走攒批写入
```

```
);

INSERT INTO student_sink SELECT * FROM student_source;
```

5.2.5.2.2 使用 DataStream API 写入数据

通过 DataStream api 写入的时候，可以使用不同的序列化方式对上游数据序列化后写入 Doris 表。

> Connector 内部已经包含 HttpClient4.5.13 版本，如果项目中有单独引用 HttpClient，需要确保版本一致。

普通 String 格式

当上游是 csv 或 json 数据格式时，可以直接使用 SimpleStringSerializer 序列化数据。

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
env.enableCheckpointing(30000);
DorisSink.Builder<String> builder = DorisSink.builder();

DorisOptions dorisOptions = DorisOptions.builder()
        .setFenodes("10.16.10.6:28737")
        .setTableIdentifier("test.student")
        .setUsername("root")
        .setPassword("")
        .build();

Properties properties = new Properties();
// 上游是json数据的时候，需要开启以下配置
properties.setProperty("read_json_by_line", "true");
properties.setProperty("format", "json");

// 上游是 csv 写入时，需要开启配置
//properties.setProperty("format", "csv");
//properties.setProperty("column_separator", ",");

DorisExecutionOptions executionOptions = DorisExecutionOptions.builder()
       .setLabelPrefix("label-doris")
       .setDeletable(false)
       //.setBatchMode(true)  开启攒批写入
       .setStreamLoadProp(properties)
       .build();

builder.setDorisReadOptions(DorisReadOptions.builder().build())
```

```
        .setDorisExecutionOptions(executionOptions)
        .setSerializer(new SimpleStringSerializer())
        .setDorisOptions(dorisOptions);

List<String> data = new ArrayList<>();
data.add("{\"id\":3,\"name\":\"Michael\",\"age\":28}");
data.add("{\"id\":4,\"name\":\"David\",\"age\":38}");

env.fromCollection(data).sinkTo(builder.build());
env.execute("doris test");
```

完整代码参考： DorisSinkExample.java

RowData 格式

RowData 是 Flink 内部的格式，如果上游传入的是 RowData 格式，则需要使用 RowDataSerializer 序列化数据。

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
env.enableCheckpointing(10000);
env.setParallelism(1);

DorisSink.Builder<RowData> builder = DorisSink.builder();

Properties properties = new Properties();
properties.setProperty("column_separator", ",");
properties.setProperty("line_delimiter", "`\n`");
properties.setProperty("format", "csv");
// 上游是 json 写入时，需要开启配置
// properties.setProperty("read_json_by_line", "true");
// properties.setProperty("format", "json");
DorisOptions.Builder dorisBuilder = DorisOptions.builder();
dorisBuilder
        .setFenodes("10.16.10.6:28737")
        .setTableIdentifier("test.student")
        .setUsername("root")
        .setPassword("");
DorisExecutionOptions.Builder executionBuilder = DorisExecutionOptions.builder();
executionBuilder.setLabelPrefix(UUID.randomUUID().toString()).setDeletable(false).
    ↪ setStreamLoadProp(properties);

// flink rowdata ‘s schema
String[] fields = {"id","name", "age"};
DataType[] types = {DataTypes.INT(), DataTypes.VARCHAR(256), DataTypes.INT()};

builder.setDorisExecutionOptions(executionBuilder.build())
        .setSerializer(
                RowDataSerializer.builder() // serialize according to rowdata
```

```
                        .setType(LoadConstants.CSV)
                        .setFieldDelimiter(",")
                        .setFieldNames(fields)
                        .setFieldType(types)
                        .build())
        .setDorisOptions(dorisBuilder.build());

// mock rowdata source
DataStream<RowData> source =
        env.fromElements("")
                .flatMap(
                        new FlatMapFunction<String, RowData>() {
                            @Override
                            public void flatMap(String s, Collector<RowData> out)
                                    throws Exception {
                                GenericRowData genericRowData = new GenericRowData(3);
                                genericRowData.setField(0, 1);
                                genericRowData.setField(1, StringData.fromString("Michael"));
                                genericRowData.setField(2, 18);
                                out.collect(genericRowData);

                                GenericRowData genericRowData2 = new GenericRowData(3);
                                genericRowData2.setField(0, 2);
                                genericRowData2.setField(1, StringData.fromString("David"));
                                genericRowData2.setField(2, 38);
                                out.collect(genericRowData2);
                            }
                        });

source.sinkTo(builder.build());
env.execute("doris test");
```

完整代码参考：DorisSinkExampleRowData.java

Debezium 格式

对于上游是 Debezium 数据格式的数据，如 FlinkCDC 或 Kafka 中 Debezium 格式数据，可以使用 JsonDebeziumSchemaSerializer 序列化。

```
// enable checkpoint
env.enableCheckpointing(10000);

Properties props = new Properties();
props.setProperty("format", "json");
props.setProperty("read_json_by_line", "true");
DorisOptions dorisOptions = DorisOptions.builder()
        .setFenodes("127.0.0.1:8030")
```

```
        .setTableIdentifier("test.student")
        .setUsername("root")
        .setPassword("").build();

DorisExecutionOptions.Builder  executionBuilder = DorisExecutionOptions.builder();
executionBuilder.setLabelPrefix("label-prefix")
        .setStreamLoadProp(props)
        .setDeletable(true);

DorisSink.Builder<String> builder = DorisSink.builder();
builder.setDorisReadOptions(DorisReadOptions.builder().build())
        .setDorisExecutionOptions(executionBuilder.build())
        .setDorisOptions(dorisOptions)
        .setSerializer(JsonDebeziumSchemaSerializer.builder().setDorisOptions(dorisOptions).build
            ↪ ());

env.fromSource(mySqlSource, WatermarkStrategy.noWatermarks(), "MySQL Source")
        .sinkTo(builder.build());
```

完整代码参考：CDCSchemaChangeExample.java

多表写入格式

目前 DorisSink 支持单个 Sink 同步多张表，需要将数据以及库表一起传递给 Sink，使用 RecordWithMetaSerializer 序列化即可。

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
env.setParallelism(1);
DorisSink.Builder<RecordWithMeta> builder = DorisSink.builder();
Properties properties = new Properties();
properties.setProperty("column_separator", ",");
properties.setProperty("line_delimiter", "`\n`");
properties.setProperty("format", "csv");
DorisOptions.Builder dorisBuilder = DorisOptions.builder();
dorisBuilder
        .setFenodes("10.16.10.6:28737")
        .setTableIdentifier("")
        .setUsername("root")
        .setPassword("");

DorisExecutionOptions.Builder executionBuilder = DorisExecutionOptions.builder();

executionBuilder
        .setLabelPrefix("label-doris")
        .setStreamLoadProp(properties)
        .setDeletable(false)
        .setBatchMode(true);
```

```
builder.setDorisReadOptions(DorisReadOptions.builder().build())
        .setDorisExecutionOptions(executionBuilder.build())
        .setDorisOptions(dorisBuilder.build())
        .setSerializer(new RecordWithMetaSerializer());

RecordWithMeta record = new RecordWithMeta("test", "student_1", "1,David,18");
RecordWithMeta record1 = new RecordWithMeta("test", "student_2", "1,Jack,28");
env.fromCollection(Arrays.asList(record, record1)).sinkTo(builder.build());
```

完整代码参考：DorisSinkMultiTableExample.java

5.2.5.3 Lookup Join

使用 Lookup Join 的能力可以优化 Flink 中维表关联的性能。当使用 Flink JDBC Connector 进行维表关联时，会遇到以下问题：

- Flink JDBC Connector 采用同步查询模式，即上游数据（如 Kafka）发送一条数据后，会立即查询 Doris 维表，导致高并发场景下查询延迟较高。
- JDBC 方式执行的查询通常是逐条点查，Doris 更推荐批量查询以提升查询效率。

使用 Lookup Join 的方式进行维表关联，在 Flink Doris Connector 中具有以下优势：

- 批量缓存上游数据，避免逐条查询带来的高延迟和数据库压力。
- 异步执行关联查询，提升数据吞吐量并减少 Doris 查询负载。

```
CREATE TABLE fact_table (
  `id` BIGINT,
  `name` STRING,
  `city` STRING,
  `process_time` as proctime()
) WITH (
  'connector' = 'kafka',
  ...
);

create table dim_city(
  `city` STRING,
  `level` INT ,
  `province` STRING,
  `country` STRING
) WITH (
  'connector' = 'doris',
  'fenodes' = '127.0.0.1:8030',
```

```
  'jdbc-url' = 'jdbc:mysql://127.0.0.1:9030',
  'table.identifier' = 'dim.dim_city',
  'username' = 'root',
  'password' = ''
);

SELECT a.id, a.name, a.city, c.province, c.country,c.level
FROM fact_table a
LEFT JOIN dim_city FOR SYSTEM_TIME AS OF a.process_time AS c
ON a.city = c.city
```

5.2.5.4 整库同步

Flink Doris Connector 中集成了Flink CDC，可以更便捷的将 MySQL 等关系型数据库同步到 Doris 中，同时包含自动创建表，Schema Change 等。目前支持同步的数据库包括：MySQL、Oracle、PostgreSQL、SQLServer、MongoDB、DB2。

> 注意
>
> 1. 使用整库同步时需要在 `$FLINK_HOME/lib` 目录下添加对应的 Flink CDC 包依赖 (Fat Jar)，比如 `flink-sql-connector-mysql-cdc-${version}.jar`，`flink-sql-connector-oracle-cdc-${version}.jar`，FlinkCDC 从 3.1 版本与之前版本不兼容，下载地址分别为为FlinkCDC3.x，FlinkCDC 2.x。
> 2. Connector 24.0.0 之后依赖的 Flink CDC 版本需要在 3.1 以上，下载地址见这里，FlinkCDC 如果需使用 Flink CDC 同步 MySQL 和 Oracle，还需要在 `$FLINK_HOME/lib` 下增加相关的 JDBC 驱动。

5.2.5.4.1 MySQL 整库同步

启动 Flink 集群后，可直接运行一下命令。

```
<FLINK_HOME>bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.16-24.0.1.jar \
    mysql-sync-database \
    --database test_db \
    --mysql-conf hostname=127.0.0.1 \
    --mysql-conf port=3306 \
    --mysql-conf username=root \
    --mysql-conf password=123456 \
    --mysql-conf database-name=mysql_db \
    --including-tables "tbl1|test.*" \
    --sink-conf fenodes=127.0.0.1:8030 \
```

```
    --sink-conf username=root \
    --sink-conf password=123456 \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.5.4.2 Oracle 整库同步

```
<FLINK_HOME>bin/flink run \
     -Dexecution.checkpointing.interval=10s \
     -Dparallelism.default=1 \
     -c org.apache.doris.flink.tools.cdc.CdcTools \
     ./lib/flink-doris-connector-1.16-24.0.1.jar \
     oracle-sync-database \
     --database test_db \
     --oracle-conf hostname=127.0.0.1 \
     --oracle-conf port=1521 \
     --oracle-conf username=admin \
     --oracle-conf password="password" \
     --oracle-conf database-name=XE \
     --oracle-conf schema-name=ADMIN \
     --including-tables "tbl1|tbl2" \
     --sink-conf fenodes=127.0.0.1:8030 \
     --sink-conf username=root \
     --sink-conf password=\
     --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
     --sink-conf sink.label-prefix=label \
     --table-conf replication_num=1
```

5.2.5.4.3 PostgreSQL 整库同步

```
<FLINK_HOME>/bin/flink run \
     -Dexecution.checkpointing.interval=10s \
     -Dparallelism.default=1\
     -c org.apache.doris.flink.tools.cdc.CdcTools \
     ./lib/flink-doris-connector-1.16-24.0.1.jar \
     postgres-sync-database \
     --database db1\
     --postgres-conf hostname=127.0.0.1 \
     --postgres-conf port=5432 \
     --postgres-conf username=postgres \
     --postgres-conf password="123456" \
     --postgres-conf database-name=postgres \
     --postgres-conf schema-name=public \
```

```
    --postgres-conf slot.name=test \
    --postgres-conf decoding.plugin.name=pgoutput \
    --including-tables "tbl1|tbl2" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password=\
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.5.4.4 SQLServer 整库同步

```
<FLINK_HOME>/bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    ./lib/flink-doris-connector-1.16-24.0.1.jar \
    sqlserver-sync-database \
    --database db1\
    --sqlserver-conf hostname=127.0.0.1 \
    --sqlserver-conf port=1433 \
    --sqlserver-conf username=sa \
    --sqlserver-conf password="123456" \
    --sqlserver-conf database-name=CDC_DB \
    --sqlserver-conf schema-name=dbo \
    --including-tables "tbl1|tbl2" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password=\
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.5.4.5 DB2 整库同步

```
<FLINK_HOME>bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.16-24.0.1.jar \
    db2-sync-database \
    --database db2_test \
    --db2-conf hostname=127.0.0.1 \
    --db2-conf port=50000 \
```

```
    --db2-conf username=db2inst1 \
    --db2-conf password=doris123456 \
    --db2-conf database-name=testdb \
    --db2-conf schema-name=DB2INST1 \
    --including-tables "FULL_TYPES|CUSTOMERS" \
    --single-sink true \
    --use-new-schema-change true \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password=123456 \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.5.4.6 MongoDB 整库同步

```
<FLINK_HOME>/bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    ./lib/flink-doris-connector-1.18-24.0.1.jar \
    mongodb-sync-database \
    --database doris_db \
    --schema-change-mode debezium_structure \
    --mongodb-conf hosts=127.0.0.1:27017 \
    --mongodb-conf username=flinkuser \
    --mongodb-conf password=flinkpwd \
    --mongodb-conf database=test \
    --mongodb-conf scan.startup.mode=initial \
    --mongodb-conf schema.sample-percent=0.2 \
    --including-tables "tbl1|tbl2" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password= \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --sink-conf sink.enable-2pc=false \
    --table-conf replication_num=1
```

5.2.5.4.7 AWS Aurora MySQL 整库同步

```
<FLINK_HOME>bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
```

```
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.18-25.0.0.jar \
    mysql-sync-database \
    --database testwd \
    --mysql-conf hostname=xxx.us-east-1.rds.amazonaws.com \
    --mysql-conf port=3306 \
    --mysql-conf username=admin \
    --mysql-conf password=123456 \
    --mysql-conf database-name=test \
    --mysql-conf server-time-zone=UTC \
    --including-tables "student" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password= \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.5.4.8 AWS RDS MySQL 整库同步

```
<FLINK_HOME>bin/flink run \
    -Dexecution.checkpointing.interval=10s \
    -Dparallelism.default=1 \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.18-25.0.0.jar \
    mysql-sync-database \
    --database testwd \
    --mysql-conf hostname=xxx.ap-southeast-1.rds.amazonaws.com \
    --mysql-conf port=3306 \
    --mysql-conf username=admin \
    --mysql-conf password=123456 \
    --mysql-conf database-name=test \
    --mysql-conf server-time-zone=UTC \
    --including-tables "student" \
    --sink-conf fenodes=127.0.0.1:8030 \
    --sink-conf username=root \
    --sink-conf password= \
    --sink-conf jdbc-url=jdbc:mysql://127.0.0.1:9030 \
    --sink-conf sink.label-prefix=label \
    --table-conf replication_num=1
```

5.2.6 使用说明

5.2.6.1 参数配置

5.2.6.1.1 通用配置项

Key	Default Value	Required	Comment
fenodes	–	Y	Doris FE http 地址，支持多个地址，使用逗号分隔
benodes	–	N	Doris BE http 地址，支持多个地址，使用逗号分隔
jdbc-url	–	N	jdbc 连接信息，如：jdbc:mysql://127.0.0.1:9030
table.identifier	–	Y	Doris 表名，如：db.tbl
username	–	Y	访问 Doris 的用户名
password	–	Y	访问 Doris 的密码
auto-redirect	TRUE	N	是否重定向 StreamLoad 请求。开启后 StreamLoad 将通过 FE 写入，不再显示
doris.request.retries	3	N	向 Doris 发送请求的重试次数
doris.request.connect.timeout	30s	N	向 Doris 发送请求的连接超时时间
doris.request.read.timeout	30s	N	向 Doris 发送请求的读取超时时间

5.2.6.1.2 Source 配置项

Key	Default Value	Required	Comment
doris.request.query.timeout	21600s	N	查询 Doris 的超时时间，默认值为 6 小时
doris.request.tablet.size	1	N	一个 Partition 对应的 Doris Tablet 个数。此数值设置越小，则会生成越多的 Partition。从而提升 Flink 侧的并行度，但同时会对 Doris 造成更大的压力。
doris.batch.size	4064	N	一次从 BE 读取数据的最大行数。增大此数值可减少 Flink 与 Doris 之间建立连接的次数。从而减轻网络延迟所带来的额外时间开销。
doris.exec.mem.limit	8192mb	N	单个查询的内存限制。默认为 8GB，单位为字节
source.use-flight-sql	FALSE	N	是否使用 Arrow Flight SQL 读取
source.flight-sql-port	-	N	使用 Arrow Flight SQL 读取时，FE 的 arrow_flight_sql_port

DataStream 专有配置项

Key	Default Value	Required	Comment
doris.read.field	–	N	读取 Doris 表的列名列表，多列之间使用逗号分隔
doris.filter.query	–	N	过滤读取数据的表达式，此表达式透传给 Doris。Doris 使用此表达式完成源端数据过

5.2.6.1.3 Sink 配置项

Key	Default Value	Required	Comment
sink.label-prefix	–	Y	Stream load 导入使用的 label 前缀。2pc 场景下要求全局唯一，用来保证 Flink 的 EOS 语义。

Key	Default Value	Required	Comment
sink.properties.*	–	N	Stream Load 的导入参数。例如：‘sink.properties.column_separator’ = ‘,’ 定义列分隔符，‘sink.properties.escape_delimiters’ = ‘true’ 特殊字符作为分隔符，\x01 会被转换为二进制的 0x01。JSON 格式导入 ‘sink.properties.format’ = ‘json’ ，‘sink.properties.read_json_by_line’ = ‘true’ 详细参数参考这里。Group Commit 模式例如：‘sink.properties.group_commit’ = ‘sync_mode’ 设置 group commit 为同步模式。flink connector 从 1.6.2 开始支持导入配置 group commit，详细使用和限制参考group commit 。
sink.enable-delete	TRUE	N	是否启用删除。此选项需要 Doris 表开启批量删除功能 (Doris0.15+ 版本默认开启)，只支持 Unique 模型。
sink.enable-2pc	TRUE	N	是否开启两阶段提交 (2pc)，默认为 true，保证 Exactly-Once 语义。关于两阶段提交可参考这里。
sink.buffer-size	1MB	N	写数据缓存 buffer 大小，单位字节。不建议修改，默认配置即可
sink.buffer-count	3	N	写数据缓存 buffer 个数。不建议修改，默认配置即可
sink.max-retries	3	N	Commit 失败后的最大重试次数，默认 3 次
sink.enable.batch-mode	FALSE	N	是否使用攒批模式写入 Doris，开启后写入时机不依赖 Checkpoint，通过 sink.buffer-flush.max-rows/sink.buffer-flush.max-bytes/sink.buffer-flush.interval 参数来控制写入时机。同时开启后将不保证 Exactly-once 语义，可借助 Uniq 模型做到幂等
sink.flush.queue-size	2	N	攒批模式下，缓存的队列大小。
sink.buffer-flush.max-rows	500000	N	攒批模式下，单个批次最多写入的数据行数。
sink.buffer-flush.max-bytes	100MB	N	攒批模式下，单个批次最多写入的字节数。
sink.buffer-flush.interval	10s	N	攒批模式下，异步刷新缓存的间隔
sink.ignore.update-before	TRUE	N	是否忽略 update-before 事件，默认忽略。

5.2.6.1.4 Lookup Join 配置项

Key	Default Value	Required	Comment
lookup.cache.max-rows	-1	N	lookup 缓存的最大行数，默认值 -1，不开启缓存
lookup.cache.ttl	10s	N	lookup 缓存的最大时间，默认 10s
lookup.max-retries	1	N	lookup 查询失败后的重试次数
lookup.jdbc.async	FALSE	N	是否开启异步的 lookup，默认 false
lookup.jdbc.read.batch.size	128	N	异步 lookup 下，每次查询的最大批次大小
lookup.jdbc.read.batch.queue-size	256	N	异步 lookup 时，中间缓冲队列的大小
lookup.jdbc.read.thread-size	3	N	每个 task 中 lookup 的 jdbc 线程数

5.2.6.1.5 整库同步配置项

语法

```
<FLINK_HOME>bin/flink run \
    -c org.apache.doris.flink.tools.cdc.CdcTools \
    lib/flink-doris-connector-1.16-1.6.1.jar \
    <mysql-sync-database|oracle-sync-database|postgres-sync-database|sqlserver-sync-database|
        ↪ mongodb-sync-database> \
    --database <doris-database-name> \
    [--job-name <flink-job-name>] \
    [--table-prefix <doris-table-prefix>] \
    [--table-suffix <doris-table-suffix>] \
    [--including-tables <mysql-table-name|name-regular-expr>] \
    [--excluding-tables <mysql-table-name|name-regular-expr>] \
    --mysql-conf <mysql-cdc-source-conf> [--mysql-conf <mysql-cdc-source-conf> ...] \
    --oracle-conf <oracle-cdc-source-conf> [--oracle-conf <oracle-cdc-source-conf> ...] \
    --postgres-conf <postgres-cdc-source-conf> [--postgres-conf <postgres-cdc-source-conf> ...] \
    --sqlserver-conf <sqlserver-cdc-source-conf> [--sqlserver-conf <sqlserver-cdc-source-conf>
        ↪ ...] \
    --sink-conf <doris-sink-conf> [--table-conf <doris-sink-conf> ...] \
    [--table-conf <doris-table-conf> [--table-conf <doris-table-conf> ...]]
```

配置

Key	Comment
–job-name	Flink 任务名称，非必需
–database	同步到 Doris 的数据库名
–table-prefix	Doris 表前缀名，例如 –table-prefix ods_。
–table-suffix	同上，Doris 表的后缀名。
–including-tables	需要同步的 MySQL 表，可以使用 | 分隔多个表，并支持正则表达式。比如 –including-tables table1
–excluding-tables	不需要同步的表，用法同上。
–mysql-conf	MySQL CDCSource 配置，例如 –mysql-conf hostname=127.0.0.1，您可以在这里查看所有配置 MySQL-CDC，其中 hostname/username/password/database-name 是必需的。同步的库表中含有非主键表时，必须设置 scan.incremental.snapshot.chunk.key-column，且只能选择非空类型的一个字段。例如：scan.incremental.snapshot.chunk.key-column=database.table:column,database.table1:column…，不同的库表列之间用，隔开。
–oracle-conf	Oracle CDCSource 配置，例如 –oracle-conf hostname=127.0.0.1，您可以在这里查看所有配置 Oracle-CDC，其中 hostname/username/password/database-name/schema-name 是必需的。
–postgres-conf	Postgres CDCSource 配置，例如 –postgres-conf hostname=127.0.0.1，您可以在这里查看所有配置 Postgres-CDC，其中 hostname/username/password/database-name/schema-name/slot.name 是必需的。

Key	Comment
–sqlserver-conf	SQLServer CDCSource 配置，例如 –sqlserver-conf hostname=127.0.0.1，您可以在这里查看所有配置 SQLServer-CDC，其中 hostname/username/password/database-name/schema-name 是必需的。
–db2-conf	SQLServer CDCSource 配置，例如 –db2-conf hostname=127.0.0.1，您可以在这里查看所有配置 DB2-CDC，其中 hostname/username/password/database-name/schema-name 是必需的。 |
–mongodb-conf	MongoDB CDCSource 配置，例如 –mongodb-conf hosts=127.0.0.1:27017，您可以在这里查看所有配置 Mongo-CDC，其中 hosts/username/password/database 是必须的。其中 –mongodb-conf schema.sample-percent 为自动采样 mongodb 数据为 Doris 建表的配置，默认为 0.2
–sink-conf	Doris Sink 的所有配置，可以在这里查看完整的配置项。
–table-conf	Doris 表的配置项，即 properties 中包含的内容（其中 table-buckets 例外，非 properties 属性）。例如 –table-conf replication_num=1，而 –table-conf table-buckets=“tbl1:10,tbl2:20,a.:30,b.:40,.*:50“表示按照正则表达式顺序指定不同表的 buckets 数量，如果没有匹配到则采用 BUCKETS AUTO 建表。
–schema-change-mode	解析 schema change 的模式，支持 debezium_structure、 sql_parser 两种解析模式，默认采用 debezium_structure 模式。 debezium_structure 解析上游 CDC 同步数据时所使用的数据结构，通过解析该结构判断 DDL 变更操作。 sql_parser 通过解析上游 CDC 同步数据时的 DDL 语句，从而判断 DDL 变更操作，因此该解析模式更加准确。使用例子：–schema-change-mode debezium_structure。 24.0.0 后支持
–single-sink	是否使用单个 Sink 同步所有表，开启后也可自动识别上游新创建的表，自动创建表。
–multi-to-one-origin	将上游多张表写入同一张表时，源表的配置，比如：–multi-to-one-origin “a_.*|b_.*”，具体参考#208
–multi-to-one-target	与 multi-to-one-origin 搭配使用，目标表的配置，比如：–multi-to-one-target “a|b”
–create-table-only	是否只仅仅同步表的结构

5.2.6.2 类型映射

Doris Type	Flink Type
NULL_TYPE	NULL
BOOLEAN	BOOLEAN
TINYINT	TINYINT
SMALLINT	SMALLINT
INT	INT
BIGINT	BIGINT
FLOAT	FLOAT
DOUBLE	DOUBLE
DATE	DATE
DATETIME	TIMESTAMP

Doris Type	Flink Type
DECIMAL	DECIMAL
CHAR	STRING
LARGEINT	STRING
VARCHAR	STRING
STRING	STRING
DECIMALV2	DECIMAL
ARRAY	ARRAY
MAP	STRING
JSON	STRING
VARIANT	STRING
IPV4	STRING
IPV6	STRING

5.2.6.3 监控指标

Flink 提供了多种Metrics用于监测 Flink 集群的指标，以下为 Flink Doris Connector 新增的监控指标。

Name	Metric Type	Description
totalFlushLoadBytes	Counter	已经刷新导入的总字节数
flushTotalNumberRows	Counter	已经导入处理的总行数
totalFlushLoadedRows	Counter	已经成功导入的总行数
totalFlushTimeMs	Counter	已经成功导入完成的总时间
totalFlushSucceededNumber	Counter	已经成功导入的次数
totalFlushFailedNumber	Counter	失败导入的次数
totalFlushFilteredRows	Counter	数据质量不合格的总行数
totalFlushUnselectedRows	Counter	被 where 条件过滤的总行数
beginTxnTimeMs	Histogram	向 Fe 请求开始一个事务所花费的时间，单位毫秒
putDataTimeMs	Histogram	向 Fe 请求获取导入数据执行计划所花费的时间
readDataTimeMs	Histogram	读取数据所花费的时间
writeDataTimeMs	Histogram	执行写入数据操作所花费的时间
commitAndPublishTimeMs	Histogram	向 Fe 请求提交并且发布事务所花费的时间
loadTimeMs	Histogram	导入完成的时间

5.2.7 最佳实践

5.2.7.1 FlinkSQL 通过 CDC 快速接入 MySQL 数据

```
-- enable checkpoint
SET 'execution.checkpointing.interval' = '10s';

CREATE TABLE cdc_mysql_source (
  id int
  ,name VARCHAR
```

```
  ,PRIMARY KEY (id) NOT ENFORCED
) WITH (
 'connector' = 'mysql-cdc',
 'hostname' = '127.0.0.1',
 'port' = '3306',
 'username' = 'root',
 'password' = 'password',
 'database-name' = 'database',
 'table-name' = 'table'
);

-- 支持同步 insert/update/delete 事件
CREATE TABLE doris_sink (
id INT,
name STRING
)
WITH (
  'connector' = 'doris',
  'fenodes' = '127.0.0.1:8030',
  'table.identifier' = 'database.table',
  'username' = 'root',
  'password' = '',
  'sink.properties.format' = 'json',
  'sink.properties.read_json_by_line' = 'true',
  'sink.enable-delete' = 'true',  -- 同步删除事件
  'sink.label-prefix' = 'doris_label'
);

insert into doris_sink select id,name from cdc_mysql_source;
```

5.2.7.2 Flink 进行部分列更新

```
CREATE TABLE doris_sink (
    id INT,
    name STRING,
    bank STRING,
    age int
)
WITH (
  'connector' = 'doris',
  'fenodes' = '127.0.0.1:8030',
  'table.identifier' = 'database.table',
  'username' = 'root',
  'password' = '',
  'sink.properties.format' = 'json',
```

```
  'sink.properties.read_json_by_line' = 'true',
  'sink.properties.columns' = 'id,name,bank,age', -- 需要更新的列
  'sink.properties.partial_columns' = 'true' -- 开启部分列更新
);
```

5.2.7.3 Flink 导入 Bitmap 数据

```
CREATE TABLE bitmap_sink (
dt int,
page string,
user_id int
)
WITH (
  'connector' = 'doris',
  'fenodes' = '127.0.0.1:8030',
  'table.identifier' = 'test.bitmap_test',
  'username' = 'root',
  'password' = '',
  'sink.label-prefix' = 'doris_label',
  'sink.properties.columns' = 'dt,page,user_id,user_id=to_bitmap(user_id)'
)
```

5.2.7.4 FlinkCDC 更新 key 列

一般在业务数据库中，会使用编号来作为表的主键，比如 Student 表，会使用编号 (id) 来作为主键，但是随着业务的发展，数据对应的编号有可能是会发生变化的。在这种场景下，使用 Flink CDC + Doris Connector 同步数据，便可以自动更新 Doris 主键列的数据。

原理

Flink CDC 底层的采集工具是 Debezium，Debezium 内部使用 op 字段来标识对应的操作：op 字段的取值分别为 c、u、d、r，分别对应 create、update、delete 和 read。而对于主键列的更新，Flink CDC 会向下游发送 DELETE 和 INSERT 事件，同时数据同步到 Doris 中后，就会自动更新主键列的数据。

使用

Flink 程序可参考上面 CDC 同步的示例，成功提交任务后，在 MySQL 侧执行 Update 主键列的语句 (update student set id = ‘1002’ where id = ‘1001’)，即可修改 Doris 中的数据。

5.2.7.5 Flink 根据根据指定列删除数据

一般 Kafka 中的消息会使用特定字段来标记操作类型，比如 { “op_type”：“delete” ,data:{…}}。针对这类数据，希望将 op_type=delete 的数据删除掉。

DorisSink 默认会根据 RowKind 来区分事件的类型，通常这种在 cdc 情况下可以直接获取到事件类型，对隐藏列DORIS_DELETE_SIGN进行赋值达到删除的目的，而 Kafka 则需要根据业务逻辑判断，显示的传入隐藏列的值。

```
-- 比如上游数据：{"op_type":"delete",data:{"id":1,"name":"zhangsan"}}
CREATE TABLE KAFKA_SOURCE(
  data STRING,
  op_type STRING
) WITH (
  'connector' = 'kafka',
  ...
);

CREATE TABLE DORIS_SINK(
  id INT,
  name STRING,
  __DORIS_DELETE_SIGN__ INT
) WITH (
  'connector' = 'doris',
  'fenodes' = '127.0.0.1:8030',
  'table.identifier' = 'db.table',
  'username' = 'root',
  'password' = '',
  'sink.enable-delete' = 'false',         -- false 表示不从 RowKind 获取事件类型
  'sink.properties.columns' = 'id, name, __DORIS_DELETE_SIGN__'  -- 显示指定 streamload 的导入列
);

INSERT INTO DORIS_SINK
SELECT json_value(data,'$.id') as id,
json_value(data,'$.name') as name,
if(op_type='delete',1,0) as __DORIS_DELETE_SIGN__
from KAFKA_SOURCE;
```

5.2.7.6 Flink CDC 同步 DDL 语句

一般同步 MySQL 等上游数据源的时候，上游增加或删除字段的时候需要同步在 Doris 中进行 Schema Change 操作。

对于这种场景，通常需要编写 DataStream API 的程序，同时使用 DorisSink 提供的 JsonDebeziumSchemaSerializer 序列化，便可以自动做到 SchemaChange，具体可参考CDCSchemaChangeExample.java

在 Connector 提供的整库同步工具中，无需额外配置，自动会同步上游 DDL，并在 Doris 进行 SchemaChange 操作。

5.2.8 常见问题

1. errCode = 2, detailMessage = Label [label_0_1] has already been used, relate to txn [19650]

Exactly-Once 场景下，Flink Job 重启时必须从最新的 Checkpoint/Savepoint 启动，否则会报如上错误。不要求 Exactly-Once 时，也可通过关闭 2PC 提交（sink.enable-2pc=false）或更换不同的 sink.label-prefix 解决。

2. errCode = 2, detailMessage = transaction [19650] not found

   发生在 Commit 阶段，checkpoint 里面记录的事务 ID，在 FE 侧已经过期，此时再次 commit 就会出现上述错误。此时无法从 checkpoint 启动，后续可通过修改 fe.conf 的 `streaming_label_keep_max_second` 配置来延长过期时间，默认 12 小时。Doris2.0 版本后还会受到 fe.conf 中 `label_num_threshold` 配置的限制 (默认 2000)，可以调大或者改为 -1（-1 表示只受时间限制）。

3. errCode = 2, detailMessage = current running txns on db 10006 is 100, larger than limit 100

这是因为同一个库并发导入超过了 100，可通过调整 fe.conf 的参数 `max_running_txn_num_per_db` 来解决，具体可参考max_running_txn_num_per_db。同时，一个任务频繁修改 label 重启，也可能会导致这个错误。2pc 场景下 (Duplicate/Aggregate 模型)，每个任务的 label 需要唯一，并且从 checkpoint 重启时，flink 任务才会主动 abort 掉之前已经 precommit 成功，没有 commit 的 txn，频繁修改 label 重启，会导致大量 precommit 成功的 txn 无法被 abort，占用事务。在 Unique 模型下也可关闭 2pc，可以实现幂等写入。

4. tablet writer write failed, tablet_id=190958, txn_id=3505530, err=-235

通常发生在 Connector1.1.0 之前，是由于写入频率过快，导致版本过多。可以通过设置 sink.batch.size 和 sink.batch.interval 参数来降低 Streamload 的频率。在 Connector1.1.0 之后，默认写入时机是由 Checkpoint 控制，可以通过增加 Checkpoint 间隔来降低写入频率。频率。

5. Flink 导入有脏数据，如何跳过？

Flink 在数据导入时，如果有脏数据，比如字段格式、长度等问题，会导致 StreamLoad 报错，此时 Flink 会不断的重试。如果需要跳过，可以通过禁用 StreamLoad 的严格模式 (strict_mode=false,max_filter_ratio=1) 或者在 Sink 算子之前对数据做过滤。

6. Flink 机器与 BE 机器的网络不通，如何配置？

Flink 向 Doris 发起写入时，Doris 会重定向到 BE 进行写入，此时返回的地址是 BE 的内网 IP，即通过即通过show backends看到的 IP，此时 Flink 与 Doris 网络不通的，会报错。这时可以在 benodes 中配置 BE 的外网 IP 即可。

7. stream load error: HTTP/1.1 307 Temporary Redirect

Flink 会先向 FE 请求，收到 307 后会向重定向后的 BE 请求。当 FE 在 FullGC/压力大/网络延迟的时候，HttpClient 默认会在一定时间 (3 秒) 没有等到响应会发送数据，由于默认情况下请求体是 InputStream，当收到 307 响应时，数据无法重放，会直接报错。有三种方式可以解决：1. 升级到 Connector25.1.0 以上，调长了默认时间；2. 修改 auto-redirect=false，直接向 BE 发起请求（不适用部分云上场景）；3. 主键模型可以开启攒批模式。

5.3 Doris Kafka Connector

Kafka Connect 是一款可扩展、可靠的在 Apache Kafka 和其他系统之间进行数据传输的工具，可以定义 Connectors 将大量数据迁入迁出 Kafka。

Doris 社区提供了 doris-kafka-connector 插件，可以将 Kafka topic 中的数据写入到 Doris 中。

5.3.1 版本说明

Connector Version	Kafka Version	Doris Version	Java Version
1.0.0	2.4+	2.0+	8
1.1.0	2.4+	2.0+	8
24.0.0	2.4+	2.0+	8
25.0.0	2.4+	2.0+	8

5.3.2 使用方式

5.3.2.1 下载

doris-kafka-connector

maven 依赖

```
<dependency>
  <groupId>org.apache.doris</groupId>
  <artifactId>doris-kafka-connector</artifactId>
  <version>25.0.0</version>
</dependency>
```

5.3.2.2 Standalone 模式启动

在 $KAFKA_HOME 下创建 plugins 目录，将下载好的 doris-kafka-connector jar 包放入其中

配置 config/connect-standalone.properties

```

bootstrap.servers=127.0.0.1:9092

## 修改为创建的 plugins 目录
## 注意：此处请填写 Kafka 的直接路径。例如：plugin.path=/opt/kafka/plugins
plugin.path=$KAFKA_HOME/plugins

## 建议将 Kafka 的 max.poll.interval.ms 时间调大到 30 分钟以上，默认 5 分钟
## 避免 Stream Load 导入数据消费超时，消费者被踢出消费群组
max.poll.interval.ms=1800000
consumer.max.poll.interval.ms=1800000
```

配置 doris-connector-sink.properties

在 config 目录下创建 doris-connector-sink.properties，并配置如下内容：

```
name=test-doris-sink
connector.class=org.apache.doris.kafka.connector.DorisSinkConnector
topics=topic_test
doris.topic2table.map=topic_test:test_kafka_tbl
doris.urls=10.10.10.1
doris.http.port=8030
```

```
doris.query.port=9030
doris.user=root
doris.password=
doris.database=test_db
buffer.count.records=10000
buffer.flush.time=120
buffer.size.bytes=5000000
enable.combine.flush=true
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.json.JsonConverter
value.converter.schemas.enable=false
```

启动 Standalone

```
$KAFKA_HOME/bin/connect-standalone.sh -daemon $KAFKA_HOME/config/connect-standalone.properties
    ↪ $KAFKA_HOME/config/doris-connector-sink.properties
```

> 注意：一般不建议在生产环境中使用 standalone 模式

5.3.2.3 Distributed 模式启动

在 $KAFKA_HOME 下创建 plugins 目录，将下载好的 doris-kafka-connector jar 包放入其中

配置 config/connect-distributed.properties

```
## 修改 broker 地址
bootstrap.servers=127.0.0.1:9092

## 修改 group.id，同一集群的需要一致
group.id=connect-cluster

## 修改为创建的 plugins 目录
## 注意：此处请填写 Kafka 的直接路径。例如：plugin.path=/opt/kafka/plugins
plugin.path=$KAFKA_HOME/plugins

## 建议将 Kafka 的 max.poll.interval.ms 时间调大到 30 分钟以上，默认 5 分钟
## 避免 Stream Load 导入数据消费超时，消费者被踢出消费群组
max.poll.interval.ms=1800000
consumer.max.poll.interval.ms=1800000
```

启动 Distributed

```
$KAFKA_HOME/bin/connect-distributed.sh -daemon $KAFKA_HOME/config/connect-distributed.properties
```

增加 Connector

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"test-doris-sink-cluster",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"topic_test",
    "doris.topic2table.map": "topic_test:test_kafka_tbl",
    "doris.urls":"10.10.10.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test_db",
    "enable.combine.flush": "true",
    "buffer.count.records":"10000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"5000000",
    "key.converter":"org.apache.kafka.connect.storage.StringConverter",
    "value.converter":"org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}'
```

操作 Connector

```
## 查看 connector 状态
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/status -X GET
## 删除当前 connector
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster -X DELETE
## 暂停当前 connector
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/pause -X PUT
## 重启当前 connector
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/resume -X PUT
## 重启 connector 内的 tasks
curl -i http://127.0.0.1:8083/connectors/test-doris-sink-cluster/tasks/0/restart -X POST
```

参考： Connect REST Interface

> 注意 kafka-connect 首次启动时，会往 kafka 集群中创建 `config.storage.topic` `offset.storage.topic` `status.storage.topic` 三个 topic 用于记录 kafka-connect 的共享连接器配置、偏移数据和状态更新。How to Use Kafka Connect - Get Started

5.3.2.4 访问 SSL 认证的 Kafka 集群

通过 kafka-connect 访问 SSL 认证的 Kafka 集群需要用户提供用于认证 Kafka Broker 公钥的证书文件（client.truststore.jks）。您可以在 connect-distributed.properties 文件中增加以下配置：

```
## Connect worker
security.protocol=SSL
ssl.truststore.location=/var/ssl/private/client.truststore.jks
ssl.truststore.password=test1234

## Embedded consumer for sink connectors
consumer.security.protocol=SSL
consumer.ssl.truststore.location=/var/ssl/private/client.truststore.jks
consumer.ssl.truststore.password=test1234
```

关于通过 Kafka-Connect 连接 SSL 认证的 Kafka 集群配置说明可以参考：Configure Kafka Connect

5.3.2.5 死信队列

默认情况下，转换过程中或转换过程中遇到的任何错误都会导致连接器失败。每个连接器配置还可以通过跳过它们来容忍此类错误，可选择将每个错误和失败操作的详细信息以及有问题的记录（具有不同级别的详细信息）写入死信队列以便记录。

```
errors.tolerance=all
errors.deadletterqueue.topic.name=test_error_topic
errors.deadletterqueue.context.headers.enable=true
errors.deadletterqueue.topic.replication.factor=1
```

5.3.3 配置项

Key	Enum	Default Value	Required	Description
name	-	-	Y	Connect 应用名称，必须是在 Kafka Connect 环境中唯一
connector.class	-	-	Y	org.apache.doris.kafka.connector.DorisSinkConnector
topics	-	-	Y	订阅的 topic 列表，逗号分隔：topic1,topic2
doris.urls	-	-	Y	Doris FE 连接地址。如果有多个，中间用逗号分割：10.20.30.1,10.20.30.2,10.20.30.3
doris.http.port	-	-	Y	Doris HTTP 协议端口
doris.query.port	-	-	Y	Doris MySQL 协议端口
doris.user	-	-	Y	Doris 用户名
doris.password	-	-	Y	Doris 密码
doris.database	-	-	Y	要写入的数据库。多个库时可以为空，同时在 topic2table.map 需要配置具体的库名称
doris.topic2table.map	-	-	Y	topic 和 table 表的对应关系，例：topic1:tb1,topic2:tb2 如果留空，默认将 topic 名称作为写入的 table。多个库的格式为 topic1:db1.tbl1,topic2:db2.tbl2
buffer.count.records	-	50000	N	单次 Stream Load 写入的条数。
buffer.flush.time	-	120	N	buffer 刷新间隔，单位秒，默认 120 秒
buffer.size.bytes	-	104857600(100MB)	N	单次 Stream Load 写入的数据大小。

Key	Enum	Default Value	Required	Description
enable.combine.flush	true, false	false	N	是否将所有分区的数据合并在一起写入。默认值为 false。开启后只能保证 at_least_once 语义。
jmx	-	true	N	通过 JMX 获取 Connector 内部监控指标，请参考：Doris-Connector-JMX
label.prefix	-	${name}	N	Stream load 导入数据时的 label 前缀。默认为 Connector 应用名称。
auto.redirect	-	true	N	是否重定向 StreamLoad 请求。开启后 StreamLoad 将通过 FE 重定向到需要写入数据的 BE，并且不再显示获取 BE 信息
sink.properties.*	-	'sink.properties.format':'json', 'sink.properties.read_json_by_line':'true'	N	Stream Load 的导入参数。例如：定义列分隔符'sink.properties.column_separator':',' 详细参数参考这里。开启 Group Commit，例如开启 sync_mode 模式的 group commit："sink.properties.group_commit":"sync_mode"。 Group Commit 可以配置 off_mode、 sync_mode、 async_mode 三种模式，具体使用参考：Group-Commit 开启部分列更新，例如开启更新指定 col2 的部分列："sink.properties.partial_columns":"true", "sink.properties.columns": "col2",
delivery.guarantee	at_least_once, exactly_once	at_least_once	N	消费 Kafka 数据导入至 doris 时，数据一致性的保障方式。支持 at_least_once exactly_once，默认为 at_least_once 。 Doris 需要升级至 2.1.0 以上，才能保障数据的 exactly_once
converter.mode	normal, debezium_ingestion	normal	N	使用 Connector 消费 Kafka 数据时，上游数据的类型转换模式。 normal表示正常消费 Kafka 中的数据，不经过任何类型转换。 debezium_ingestion表示当 Kafka 上游的数据通过 Debezium 等 CDC（Changelog Data Capture，变更数据捕获）工具采集时，上游数据需要经过特殊的类型转换才能支持。
debezium.schema.evolution	none, basic	none	N	通过 Debezium 采集上游数据库系统（如 MySQL），发生结构变更时，可以将增加的字段同步到 Doris 中。none表示上游数据库系统发生结构变更时，不同步变更后的结构到 Doris 中。basic表示同步上游数据库的数据变更操作。由于列结构变更是一个危险操作（可能会导致误删 Doris 表结构的列），目前仅支持同步上游增加列的操作。当列被重命名后，则旧列保持原样，Connector 会在目标表中新增一列，将重命名后的新增数据 Sink 到新列中。
enable.delete	-	false	N	Debezium 同步下，是否同步删除记录，默认 false，非 Debezium 同步下，需要在消息中拼接删除标记

Key	Enum	Default Value	Required	Description
database.time_zone		UTC	N	当 converter.mode 为非 normal 模式时，对于日期数据类型（如 datetime, date, timestamp 等等）提供指定时区转换的方式，默认为 UTC 时区。
avro.topic2schema.filepath			N	通过读取本地提供的 Avro Schema 文件，来解析 Topic 中的 Avro 文件内容，实现与 Confluent 提供 Schema 注册中心解耦。此配置需要与 key.converter 或 value.converter 前缀一起使用，例如配置 avro-user、avro-product Topic 的本地 Avro Schema 文件如下：`"value.converter.avro.topic2schema.filepath":"avro-user:file:///opt/avro_user.avsc, avro-product:file:///opt/avro_product.avsc"` 具体使用可以参考：#32
record.tablename.field			N	开启该参数后，可实现一个 Topic 的数据流向多个 Doris 表。配置详情参考：#58
max.retries		10	N	任务失败前重试错误的最大次数。
retry.interval.ms		6000	N	发生错误后，尝试重试之前的等待时间，单位为毫秒，默认 6000 毫秒。
behavior.on.null.values	ignore, fail	ignore	N	如何处理 null 值的记录，默认跳过不处理。

其他 Kafka Connect Sink 通用配置项可参考：connect_configuring

5.3.4 类型映射

Doris-kafka-connector 使用逻辑或原始类型映射来解析列的数据类型。原始类型是指使用 Kafka connect 的 Schema 表示的简单数据类型。逻辑数据类型通常是采用 Struct 结构表示复杂类型，或者日期时间类型。

Kafka 原始类型	Doris 类型
INT8	TINYINT
INT16	SMALLINT
INT32	INT
INT64	BIGINT
FLOAT32	FLOAT
FLOAT64	DOUBLE
BOOLEAN	BOOLEAN
STRING	STRING
BYTES	STRING

Kafka 逻辑类型	Doris 类型
org.apache.kafka.connect.data.Decimal	DECIMAL
org.apache.kafka.connect.data.Date	DATE
org.apache.kafka.connect.data.Time	STRING
org.apache.kafka.connect.data.Timestamp	DATETIME

Debezium 逻辑类型	Doris 类型
io.debezium.time.Date	DATE
io.debezium.time.Time	String
io.debezium.time.MicroTime	DATETIME
io.debezium.time.NanoTime	DATETIME
io.debezium.time.ZonedTime	DATETIME
io.debezium.time.Timestamp	DATETIME
io.debezium.time.MicroTimestamp	DATETIME
io.debezium.time.NanoTimestamp	DATETIME
io.debezium.time.ZonedTimestamp	DATETIME
io.debezium.data.VariableScaleDecimal	DOUBLE

5.3.5 最佳实践

5.3.5.1 同步普通 JSON 数据

1. 导入数据样本在 Kafka 中，有以下样本数据

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-data-topic --
    ↪ from-beginning
{"user_id":1,"name":"Emily","age":25}
{"user_id":2,"name":"Benjamin","age":35}
{"user_id":3,"name":"Olivia","age":28}
{"user_id":4,"name":"Alexander","age":60}
{"user_id":5,"name":"Ava","age":17}
{"user_id":6,"name":"William","age":69}
{"user_id":7,"name":"Sophia","age":32}
{"user_id":8,"name":"James","age":64}
{"user_id":9,"name":"Emma","age":37}
{"user_id":10,"name":"Liam","age":64}
```

2. 创建需要导入的表在 Doris 中，创建被导入的表，具体语法如下

```
CREATE TABLE test_db.test_kafka_connector_tbl(
user_id            BIGINT       NOT NULL COMMENT "user id",
name               VARCHAR(20)           COMMENT "name",
age                INT                   COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 12;
```

3. 创建导入任务在部署 Kafka-connect 的机器上，通过 curl 命令提交如下导入任务

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d
    ↪ '{
```

```
"name":"test-doris-sink-cluster",
"config":{
"connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
"tasks.max":"10",
"topics":"test-data-topic",
"doris.topic2table.map": "test-data-topic:test_kafka_connector_tbl",
"doris.urls":"10.10.10.1",
"doris.user":"root",
"doris.password":"",
"doris.http.port":"8030",
"doris.query.port":"9030",
"doris.database":"test_db",
"buffer.count.records":"10000",
"buffer.flush.time":"120",
"buffer.size.bytes":"5000000",
"enable.combine.flush": "true",
"key.converter":"org.apache.kafka.connect.storage.StringConverter",
"value.converter": "org.apache.kafka.connect.json.JsonConverter",
"value.converter.schemas.enable": "false"
}
}'
```

5.3.5.2 同步 Debezium 组件采集的数据

1. MySQL 数据库中有如下表 “‘sql CREATE TABLE test.test_user (user_id int NOT NULL , name varchar(20), age int, PRIMARY KEY (user_id)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

insert into test.test_user values(1, ‘zhangsan’,20); insert into test.test_user values(2, ‘lisi’,21); insert into test.test_user values(3, ‘wangwu’,22); “ ‘

2. 在 Doris 创建被导入的表

```
CREATE TABLE test_db.test_user(
user_id            BIGINT       NOT NULL COMMENT "user id",
name               VARCHAR(20)           COMMENT "name",
age                INT                   COMMENT "age"
)
UNIQUE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 12;
```

3. 部署 Debezium connector for MySQL 组件，参考：Debezium connector for MySQL

4. 创建 doris-kafka-connector 导入任务假设通过 Debezium 采集到的 MySQL 表数据在 `mysql_debezium.test.test_user` Topic 中

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d
    ↪ '{
"name":"test-debezium-doris-sink",
"config":{
"connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
"tasks.max":"10",
"topics":"mysql_debezium.test.test_user",
"doris.topic2table.map": "mysql_debezium.test.test_user:test_user",
"doris.urls":"10.10.10.1",
"doris.user":"root",
"doris.password":"",
"doris.http.port":"8030",
"doris.query.port":"9030",
"doris.database":"test_db",
"buffer.count.records":"10000",
"buffer.flush.time":"30",
"buffer.size.bytes":"5000000",
"enable.combine.flush": "true",
"converter.mode":"debezium_ingestion",
"enable.delete":"true",
"key.converter":"org.apache.kafka.connect.json.JsonConverter",
"value.converter":"org.apache.kafka.connect.json.JsonConverter"
}
}'
```

5.3.5.3 同步 Avro 序列化数据

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"doris-avro-test",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"avro_topic",
    "tasks.max":"10",
    "doris.topic2table.map": "avro_topic:avro_tab",
    "doris.urls":"127.0.0.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test",
    "buffer.count.records":"100000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"10000000",
    "enable.combine.flush": "true",
```

```
    "key.converter":"io.confluent.connect.avro.AvroConverter",
    "key.converter.schema.registry.url":"http://127.0.0.1:8081",
    "value.converter":"io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url":"http://127.0.0.1:8081"
  }
}'
```

5.3.5.4 同步 Protobuf 序列化数据

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name":"doris-protobuf-test",
  "config":{
    "connector.class":"org.apache.doris.kafka.connector.DorisSinkConnector",
    "topics":"proto_topic",
    "tasks.max":"10",
    "doris.topic2table.map": "proto_topic:proto_tab",
    "doris.urls":"127.0.0.1",
    "doris.user":"root",
    "doris.password":"",
    "doris.http.port":"8030",
    "doris.query.port":"9030",
    "doris.database":"test",
    "buffer.count.records":"100000",
    "buffer.flush.time":"120",
    "buffer.size.bytes":"10000000",
    "enable.combine.flush": "true",
    "key.converter":"io.confluent.connect.protobuf.ProtobufConverter",
    "key.converter.schema.registry.url":"http://127.0.0.1:8081",
    "value.converter":"io.confluent.connect.protobuf.ProtobufConverter",
    "value.converter.schema.registry.url":"http://127.0.0.1:8081"
  }
}'
```

5.3.5.5 使用 Kafka Connect SMT 转换数据

数据样例如下:

```
{
  "registertime": 1513885135404,
  "userid": "User_9",
  "regionid": "Region_3",
  "gender": "MALE"
}
```

假设需要在 Kafka 消息中硬编码新增一个列，可以使用 InsertField。另外，也可以使用 TimestampConverter 将 Bigint 类型 timestamp 转换成时间字符串。

```
curl -i http://127.0.0.1:8083/connectors -H "Content-Type: application/json" -X POST -d '{
  "name": "insert_field_tranform",
  "config": {
    "connector.class": "org.apache.doris.kafka.connector.DorisSinkConnector",
    "tasks.max": "1",
    "topics": "users",
    "doris.topic2table.map": "users:kf_users",
    "buffer.count.records": "10000",
    "buffer.flush.time": "10",
    "buffer.size.bytes": "5000000",
    "doris.urls": "127.0.0.1:8030",
    "doris.user": "root",
    "doris.password": "123456",
    "doris.http.port": "8030",
    "doris.query.port": "9030",
    "doris.database": "testdb",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false",
    "transforms": "InsertField,TimestampConverter",
    // Insert Static Field
    "transforms.InsertField.type": "org.apache.kafka.connect.transforms.InsertField$Value",
    "transforms.InsertField.static.field": "repo",
    "transforms.InsertField.static.value": "Apache Doris",
    // Convert Timestamp Format
    "transforms.TimestampConverter.type": "org.apache.kafka.connect.transforms.
        ↪ TimestampConverter$Value",
    "transforms.TimestampConverter.field": "registertime",
    "transforms.TimestampConverter.format": "yyyy-MM-dd HH:mm:ss.SSS",
    "transforms.TimestampConverter.target.type": "string"
  }
}'
```

样例数据经过 SMT 的处理之后，变成如下所示：

```
{
  "userid": "User_9",
  "regionid": "Region_3",
  "gender": "MALE",
  "repo": "Apache Doris",// Static field added
  "registertime": "2017-12-21 03:38:55.404"  // Unix timestamp converted to string
}
```

更多关于 Kafka Connect Single Message Transforms (SMT) 使用案例, 可以参考文档 SMT documentation.

5.3.6 常见问题

1. 读取 JSON 类型的数据报如下错误：

```
Caused by: org.apache.kafka.connect.errors.DataException: JsonConverter with schemas.enable
    ↪ requires "schema" and "payload" fields and may not contain additional fields. If you are
    ↪ trying to deserialize plain JSON data, set schemas.enable=false in your converter
    ↪ configuration.
    at org.apache.kafka.connect.json.JsonConverter.toConnectData(JsonConverter.java:337)
    at org.apache.kafka.connect.storage.Converter.toConnectData(Converter.java:91)
    at org.apache.kafka.connect.runtime.WorkerSinkTask.lambda$convertAndTransformRecord$4(
        ↪ WorkerSinkTask.java:536)
    at org.apache.kafka.connect.runtime.errors.RetryWithToleranceOperator.execAndRetry(
        ↪ RetryWithToleranceOperator.java:180)
    at org.apache.kafka.connect.runtime.errors.RetryWithToleranceOperator.execAndHandleError(
        ↪ RetryWithToleranceOperator.java:214)
```

原因：是因为使用 org.apache.kafka.connect.json.JsonConverter 转换器需要匹配“schema”和“payload”字段。

两种解决方案，任选其一：1. 将 org.apache.kafka.connect.json.JsonConverter 更换为 org.apache.kafka.
↪ connect.storage.StringConverter 2. 启动模式为 Standalone 模式，则将 config/connect-standalone.properties 中 value.converter.schemas.enable 或 key.converter.schemas.enable 改成 false；启动模式为 Distributed 模式，则将 config/connect-distributed.properties 中 value.converter.schemas.enable 或 key.converter.schemas.enable 改成 false

2. 消费超时，消费者被踢出消费群组：

```
org.apache.kafka.clients.consumer.CommitFailedException: Offset commit cannot be completed since
    ↪ the consumer is not part of an active group for auto partition assignment; it is likely
    ↪ that the consumer was kicked out of the group.
        at org.apache.kafka.clients.consumer.internals.ConsumerCoordinator.
            ↪ sendOffsetCommitRequest(ConsumerCoordinator.java:1318)
        at org.apache.kafka.clients.consumer.internals.ConsumerCoordinator.doCommitOffsetsAsync(
            ↪ ConsumerCoordinator.java:1127)
        at org.apache.kafka.clients.consumer.internals.ConsumerCoordinator.commitOffsetsAsync(
            ↪ ConsumerCoordinator.java:1093)
        at org.apache.kafka.clients.consumer.KafkaConsumer.commitAsync(KafkaConsumer.java:1590)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.doCommitAsync(WorkerSinkTask.java:361)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.doCommit(WorkerSinkTask.java:376)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.commitOffsets(WorkerSinkTask.java:467)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.commitOffsets(WorkerSinkTask.java:381)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.iteration(WorkerSinkTask.java:221)
        at org.apache.kafka.connect.runtime.WorkerSinkTask.execute(WorkerSinkTask.java:206)
        at org.apache.kafka.connect.runtime.WorkerTask.doRun(WorkerTask.java:204)
        at org.apache.kafka.connect.runtime.WorkerTask.run(WorkerTask.java:259)
        at org.apache.kafka.connect.runtime.isolation.Plugins.lambda$withClassLoader$1(Plugins.
            ↪ java:181)
```

```
        at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:539)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java
            ↪ :1136)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java
            ↪ :635)
        at java.base/java.lang.Thread.run(Thread.java:833)
```

解决方案：

将 Kafka 中 `max.poll.interval.ms` 根据场景进行调大，默认值是 300000 - 如果是 Standalone 模式启动，则在 config/connect-standalone.properties 的配置文件中增加 `max.poll.interval.ms` 和 `consumer.max.poll.interval.ms` 参数，并配置参数值。- 如果是 Distributed 模式启动，则在 config/connect-distributed.properties 的配置文件增加 `max.poll.interval.ms` 和 `consumer.max.poll.interval.ms` 参数，并配置参数值。

调整参数后，重启 kafka-connect

3. Doris-kafka-connector 从 1.0.0 或 1.1.0 升级到 24.0.0 版本报错

```
org.apache.kafka.common.config.ConfigException: Topic 'connect-status' supplied via the 'status.
    ↪ storage.topic' property is required to have 'cleanup.policy=compact' to guarantee
    ↪ consistency and durability of connector and task statuses, but found the topic currently
    ↪ has 'cleanup.policy=delete'. Continuing would likely result in eventually losing
    ↪ connector and task statuses and problems restarting this Connect cluster in the future.
    ↪ Change the 'status.storage.topic' property in the Connect worker configurations to use a
    ↪ topic with 'cleanup.policy=compact'.
    at org.apache.kafka.connect.util.TopicAdmin.verifyTopicCleanupPolicyOnlyCompact(TopicAdmin.
        ↪ java:581)
    at org.apache.kafka.connect.storage.KafkaTopicBasedBackingStore.lambda$topicInitializer$0(
        ↪ KafkaTopicBasedBackingStore.java:47)
    at org.apache.kafka.connect.util.KafkaBasedLog.start(KafkaBasedLog.java:247)
    at org.apache.kafka.connect.util.KafkaBasedLog.start(KafkaBasedLog.java:231)
    at org.apache.kafka.connect.storage.KafkaStatusBackingStore.start(KafkaStatusBackingStore.
        ↪ java:228)
    at org.apache.kafka.connect.runtime.AbstractHerder.startServices(AbstractHerder.java:164)
    at org.apache.kafka.connect.runtime.distributed.DistributedHerder.run
```

解决方案： 调整 `connect-configs` `connect-status` Topic 的清除策略为 compact

```
$KAFKA_HOME/bin/kafka-configs.sh --alter --entity-type topics --entity-name connect-configs --add
    ↪ -config cleanup.policy=compact --bootstrap-server 127.0.0.1:9092
$KAFKA_HOME/bin/kafka-configs.sh --alter --entity-type topics --entity-name connect-status --add-
    ↪ config cleanup.policy=compact --bootstrap-server 127.0.0.1:9092
```

4. `debezium_ingestion` 转换模式下，表结构变更失败

```
[2025-01-07 14:26:20,474] WARN [doris-normal_test_sink-connector|task-0] Table 'test_sink' cannot
    ↪  be altered because schema evolution is disabled. (org.apache.doris.kafka.connector.
    ↪ converter.RecordService:183)
```

```
[2025-01-07 14:26:20,475] ERROR [doris-normal_test_sink-connector|task-0] WorkerSinkTask{id=doris
    ↪ -normal_test_sink-connector-0} Task threw an uncaught and unrecoverable exception. Task
    ↪ is being killed and will not recover until manually restarted. Error: Cannot alter table
    ↪ org.apache.doris.kafka.connector.model.TableDescriptor@67cd8027 because schema evolution
    ↪ is disabled (org.apache.kafka.connect.runtime.WorkerSinkTask:612)
org.apache.doris.kafka.connector.exception.SchemaChangeException: Cannot alter table org.apache.
    ↪ doris.kafka.connector.model.TableDescriptor@67cd8027 because schema evolution is disabled
    at org.apache.doris.kafka.connector.converter.RecordService.alterTableIfNeeded(RecordService.
        ↪ java:186)
    at org.apache.doris.kafka.connector.converter.RecordService.checkAndApplyTableChangesIfNeeded
        ↪ (RecordService.java:150)
    at org.apache.doris.kafka.connector.converter.RecordService.processStructRecord(RecordService
        ↪ .java:100)
    at org.apache.doris.kafka.connector.converter.RecordService.getProcessedRecord(RecordService.
        ↪ java:305)
    at org.apache.doris.kafka.connector.writer.DorisWriter.putBuffer(DorisWriter.java:155)
    at org.apache.doris.kafka.connector.writer.DorisWriter.insertRecord(DorisWriter.java:124)
    at org.apache.doris.kafka.connector.writer.StreamLoadWriter.insert(StreamLoadWriter.java:151)
    at org.apache.doris.kafka.connector.service.DorisDefaultSinkService.insert(
        ↪ DorisDefaultSinkService.java:154)
    at org.apache.doris.kafka.connector.service.DorisDefaultSinkService.insert(
        ↪ DorisDefaultSinkService.java:135)
    at org.apache.doris.kafka.connector.DorisSinkTask.put(DorisSinkTask.java:97)
    at org.apache.kafka.connect.runtime.WorkerSinkTask.deliverMessages(WorkerSinkTask.java:583)
    at org.apache.kafka.connect.runtime.WorkerSinkTask.poll(WorkerSinkTask.java:336)
    at org.apache.kafka.connect.runtime.WorkerSinkTask.iteration(WorkerSinkTask.java:237)
    at org.apache.kafka.connect.runtime.WorkerSinkTask.execute(WorkerSinkTask.java:206)
    at org.apache.kafka.connect.runtime.WorkerTask.doRun(WorkerTask.java:202)
    at org.apache.kafka.connect.runtime.WorkerTask.run(WorkerTask.java:257)
    at org.apache.kafka.connect.runtime.isolation.Plugins.lambda$withClassLoader$1(Plugins.java
        ↪ :177)
    at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:515)
    at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
    at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1128)
    at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:628)
    at java.base/java.lang.Thread.run(Thread.java:829)
```

解决方案：

在 debezium_ingestion 转换模式下，默认表结构变更是关闭的，需要配置 debezium.schema.evolution 为 basic 以便开启表结构变更。需要注意的是：开启表结构变更并不能准确的保持此变更列为 Doris 表中的唯一列（详见 debezium.schema.evolution 参数说明）。如需要保持上下游只存在唯一列，最好是手动添加变更列到 Doris 表中，再重新启动 Connector 任务，Connector 将会接着未消费的 offset 继续消费，保持数据的一致性。

5.4 Doris Operator

5.4.1 Doris Kubernetes Operator

为满足用户在 Kubernetes 平台上对 Doris 的高效部署和运维需求诞生的 Kubernetes Operator（简称：Doris Operator），集成了原生 Kubernetes 资源的复杂管理能力，并融合了 Doris 组件间的分布式协同、用户集群形态的按需定制等经验，为用户提供了一个更简洁、高效、易用的容器化部署方案。旨在实现 Doris 在 Kubernetes 上的高效管控，帮助用户减少运维管理和学习成本的同时，提供强大的功能和灵活的配置能力。

Doris Operator 基于 Kubernetes CustomResourceDefinitions（CRD）实现了 Doris 在 Kubernetes 平台的配置、管理和调度。Doris Operator 能够根据用户自定义的期望状态，自动创建 Pods 及其他资源以启动服务。通过自动注册机制，可将所有启动的服务整合成一个完整的 Doris 集群。这一实现显著降低了在 Doris 集群中处理配置信息、节点发现与注册、访问通信及健康检查等生产环境必备操作的复杂性和学习成本。

5.4.1.1 Doris Operator 架构形态

Doris Operator 的设计基于二层调度器的原理。每个组件的第一层调度使用原生的 StatefulSet 和 Service 资源直接管理相应的 Pod 服务，这使其能够完全兼容开源 Kubernetes 集群，包括公有云、私有云以及自建的 Kubernetes 平台。

基于 Doris Operator 提供的部署定义，用户可自定义 Doris 部署状态，并通过 Kubernetes 的 kubectl 管理命令将其下发到 Kubernetes 集群中。Doris Operator 会根据自定义状态将每个服务的部署转换为 StatefulSet 及其附属资源（如 Service），再通过 StatefulSet 调度出期望的 Pods。它通过抽象 Doris 集群的终态，简化了 StatefulSet 规格中不必要的配置，从而降低了用户的学习成本。

5.4.1.2 关键能力

- 终态部署：

Kubernetes 采用终态运维模式来管理服务，而 Doris Operator 则定义了一种能够描述 Doris 集群的资源类型——DorisCluster。用户可以参考相关文档和使用示例，轻松配置所需的集群。用户通过 Kubernetes 的命令行工具 kubectl，可以将配置下发到 Kubernetes 集群中。Doris Operator 会自动构建所需集群，并实时更新集群状态至相应的资源中。这一过程确保了集群的高效管理与监控，极大地简化了运维操作。

- 易扩展：

Doris Operator 在基于云盘的环境中支持并发实时横向扩容。Doris 的所有组件服务均通过 Kubernetes 的 StatefulSet 进行部署和管理。在部署或扩容时，采用 StatefulSet 的 Parallel 模式创建 Pods，这样理论上可以在启动一个节点的时间内启动所有副本。每个副本的启动互不干扰，当某个服务启动失败时，其他服务的启动不会受到影响。Doris Operator 采用并发模式启动服务，并内置分布式架构，极大简化了服务扩展的过程。用户只需设置副本数量，即可轻松完成扩容，彻底解放了运维操作的复杂性。

- 无感变更：

在分布式环境中，服务重启可能会引发服务的暂时不稳定。尤其对于数据库这类对稳定性要求极高的服务而言，如何在重启过程中保证服务的稳定性是一个非常重要的课题。Doris 在 Kubernetes 上通过以下三种机制确保服务重启过程中的稳定性，从而实现业务在重启和升级过程中无感知的体验。

1. 优雅退出
2. 滚动重启
3. 主动停止查询分配

- 宿主机系统配置：

在某些场景中，需要配置宿主机系统参数来达到 Apache Doris 的理想性能。而在容器化场景下，宿主机的部署不确定和参数修改难度高给用户带来挑战。为解决该问题，Doris Operator 通过利用 Kubernetes 的初始化容器，实现了宿主机参数的可配置化。Doris Operator 允许用户配置在宿主机上执行的命令，并通过初始化容器使其生效。为了提升可用性，Doris Operator 抽象了 Kubernetes 初始化容器的配置方式，使宿主机命令的设置更加简单直观。

- 持久化配置：

Doris Operator 采用 Kubernetes StorageClass 模式为各个服务提供存储配置。它允许用户自定义挂载目录，在自定义启动配置时，如果修改了存储目录，可以在自定义资源中将该目录设置为持久化位置，从而使服务使用容器内指定的目录来存储数据。

- 运行时调试：

容器化服务对于 Trouble Shooting 来说最大挑战之一是如何在运行时进行调试。Doris Operator 在追求可用性和易用性的同时，也为问题定位提供了更便利的条件。在 Doris 的基础镜像中，预置了多种用于问题定位的工具。当需要实时查看状态时，可以通过 kubectl 提供的 exec 命令进入容器，使用内置工具进行故障排查。当服务因未知原因无法启动时，Doris Operator 提供了 Debug 运行模式。当一个 Pod 被设置为 Debug 启动模式时，容器将自动进入运行状态。这时可通过 exec 命令进入容器，手动启动服务并进行问题定位。详细请参考此文档

5.4.1.3 兼容性

Doris Operator 开发按照标准的 K8s 规范进行，兼容所有标准 K8s 平台，包含主流云厂商提供的和基于标准自建的 K8s 平台和用户自建平台。

5.4.1.3.1 云厂商兼容性

在主流云厂商的容器化服务平台上，完全兼容。使用 Doris Operator 环境准备及使用建议，可参考以下文档：

- 阿里云
- AWS

5.4.1.4 安装及使用

5.4.1.4.1 前提条件

部署前需要对宿主机系统进行检查参考操作系统检查

5.4.1.4.2 部署 Doris Operator

详细安装文档可参考 Doris Operator 安装的存算一体版本 或存算分离版本

5.4.2 在阿里云上的部署建议

5.4.2.1 阿里云容器服务 ACK

阿里云容器服务 ACK 属于购买 ECS 实例后，托管容器化服务的，因此可以获得完全访问控制权限来进行相关系统参数调整，使用实例镜像：Alibaba Cloud Linux 3 当前系统参数完全满足运行 Doris 需求。不符合要求的也能够通过 K8s 特权模式在容器内进行修正，以保证稳定运行。
阿里云 ACK 集群，使用 Doris Operator 部署，大部分环境要求，ECS 默认配置即可满足，未满足的，Doris Operator 可自行修正。用户亦可手动修正，如下：

5.4.2.1.1 已存在集群

若容器服务集群已经创建，则可以参考此文档进行修改：操作系统检查
重点关注 BE 启动参数要求：
1. 禁用和关闭 swap：swapon --show 未开启则无输出 2. 查看系统最大打开文件句柄数 ulimit -n 3. 查看修改虚拟内存区域数量 sysctl vm.max_map_count 4. 透明大页是否关闭 cat /sys/kernel/mm/transparent_hugepage/enabled 是否包含 never
对应参数的默认值如下：

```
[root@iZj6c12a1czxk5oer9rbp8Z ~]# swapon --show
[root@iZj6c12a1czxk5oer9rbp8Z ~]# ulimit -n
65535
[root@iZj6c12a1czxk5oer9rbp8Z ~]# sysctl vm.max_map_count
vm.max_map_count = 262144
[root@iZj6c12a1czxk5oer9rbp8Z ~]# cat /sys/kernel/mm/transparent_hugepage/enabled
[always] madvise never
```

5.4.2.1.2 新建集群

若集群未购买和创建，则可以在阿里云容器服务 ACK 控制台点击“创建集群”购买，可以按需调整配置，上述参数可以在创建集群的“节点池配置”步骤中在“实例预自定义数据”添加系统调整脚本。在集群启动后，重启节点即可实现配置完成。参考脚本如下：

```
###!/bin/bash
chmod +x /etc/rc.d/rc.local
echo "sudo systemctl stop firewalld.service" >> /etc/rc.d/rc.local
echo "sudo systemctl disable firewalld.service" >> /etc/rc.d/rc.local
echo "sysctl -w vm.max_map_count=2000000" >> /etc/rc.d/rc.local
echo "swapoff -a" >> /etc/rc.d/rc.local
current_limit=$(ulimit -n)
desired_limit=1000000
config_file="/etc/security/limits.conf"
 if [ "$current_limit" -ne "$desired_limit" ]; then
```

```
    echo "* soft nofile 1000000" >> "$config_file"
    echo "* hard nofile 1000000" >> "$config_file"
fi
```

5.4.2.2 阿里云容器服务 ACS

ACS 服务是以 K8s 为使用界面供给容器算力资源的云计算服务，提供按需计费的弹性算力资源。和上述 ACK 不同的是不需要关注具体使用 ECS。需要注意使用 ACS 的事项如下：

5.4.2.2.1 镜像仓库访问

使用 ACS 推荐使用配套的阿里云镜像 Container Registry(ACR) 个人版和企业版按需开启。
在配置好镜像仓库和镜像中转的环境后，需要把 Doris 提供的官方镜像迁移到对应的阿里云镜像仓库中。

若使用私有镜像仓库开启了鉴权，可以参考以下步骤：

1. 需要提前设置类型为 `docker-registry` 的 `secret` 用以配置访问镜像仓库的身份认证信息。

```
kubectl create secret docker-registry image-hub-secret --docker-server={your-server} --docker-
    ↪ username={your-username} --docker-password={your-pwd}
```

2. 在 DCR 上配置使用上述步骤的 secret：

```
spec:
  feSpec:
    replicas: 1
    image: crpi-4q6quaxa0ta96k7h-vpc.cn-hongkong.personal.cr.aliyuncs.com/selectdb-test/doris.
        ↪ fe-ubuntu:3.0.3
    imagePullSecrets:
    - name: image-hub-secret
  beSpec:
    replicas: 3
    image: crpi-4q6quaxa0ta96k7h-vpc.cn-hongkong.personal.cr.aliyuncs.com/selectdb-test/doris.
        ↪ be-ubuntu:3.0.3
    imagePullSecrets:
    - name: image-hub-secret
    systemInitialization:
      initImage: crpi-4q6quaxa0ta96k7h-vpc.cn-hongkong.personal.cr.aliyuncs.com/selectdb-test/
          ↪ alpine:latest
```

5.4.2.2.2 Be systemInitialization

目前阿里云逐步推送在完全托管的 ACS 服务上提供开启特权模式的能力（部分地域可能暂未开启，可提交工单申请开启能力加白）。

Doris BE 节点启动需要依赖一些特殊环境参数比如，修改虚拟内存区域数量 `sysctl -w vm.max_map_count =2000000`
在容器内部设置此参数需要修改宿主机配置，因此常规的 K8s 集群需要在 pod 内开启特权模式。Operator 通过 `systemInitialization` 为 BE pod 添加 `InitContainer` 来执行此类操作。

> 提示 如果当前集群无法使用特权模式，则无法启动 BE 节点。可以选择 ACK 容器服务 + 宿主机的形式来部署集群。

5.4.2.2.3 Service 限制

由于 ACS 服务是以 K8s 为使用界面供给容器算力资源的云计算服务，提供算力资源。其 node 是虚拟计算资源，用户无需关注，按使用资源量收费，可以无限拓展，即，不存在常规的 node 这个物理概念：

```
$ kubectl get nodes
NAME                            STATUS   ROLES   AGE   VERSION
virtual-kubelet-cn-hongkong-d   Ready    agent   27h   v1.31.1-aliyun.1
```

因此，部署 Doris 集群时 serviceType 禁用 NodePort 模式，允许使用 ClusterIP 和 LB 模式。
- ClusterIP 模式：

Operator 默认的网络模式，具体使用和访问方式可参考此文档

- 负载均衡模式：

使用时可以通过如下方式来配置（注意事项：https://help.aliyun.com/zh/ack/ack-managed-and-ack-dedicated/user-guide/considerations-for-configuring-a-loadbalancer-type-service-1 ）：

- 通过 Operator 提供的 DCR 的 service annotations 来配置 LB 接入，步骤如下：
  1. 已通过负载均衡控制台创建 CLB 或 NLB 实例，且该实例与 ACK 集群处于同一地域。如果尚未创建，请参见创建和管理 CLB 实例和创建和管理 NLB 实例。
  2. 通过 DCR 配置，上述 LB 的访问 annotations，参考格式如下：

```
feSpec:
  replicas: 3
  image: crpi-4q6quaxa0ta96k7h-vpc.cn-hongkong.personal.cr.aliyuncs.com/selectdb-test/
      ↪ doris.fe-ubuntu:3.0.3
  service:
    type: LoadBalancer
    annotations:
      service.beta.kubernetes.io/alibaba-cloud-loadbalancer-address-type: "intranet"
```

- 通过 ACS 控制台托管 LB 服务，生成绑定 FE 或 BE 对应资源管控的 statefulset 的 service 步骤如下：
  1. serviceType 为 ClusterIP（默认策略）

2. 可以通过阿里云控制台界面：容器计算服务 ACS -> 集群列表 -> 集群 -> 服务，通过 创建 按钮创建负载均衡服务。
3. 在创建 服务 的界面选择新建的 LB，会和 `service` 绑定，也会随着该 `service` 的注销而注销。但是此 `service` 不受 Doris Operator 管控。

5.4.3 在 AWS 上的部署建议

5.4.3.1 AWS 容器服务 EKS

5.4.3.1.1 新建集群

EKS 集群中运行的容器是托管在 EC2 实例上的，需要根据 Doris 的要求对 EC2 实例进行系统级配置。在集群创建时，需要用户确认 EKS 模式，自治模式或普通模式。这里推荐不使用自治模式，因为自治模式的计算资源是通过内置节点池来分配和回收资源，在每一次的资源申请或则释放，都会进行现有资源的重新整合，对于 statefulset 这类有状态服务尤其是启动耗时长和 Doris 这种有严格的分布式协同要求的服务，会造成共享节点池的所有服务动荡，直接现象就是，有可能引起整个 Doris 集群的全部节点漂移（这比重启更恐怖，这个过程不会滚动重启，而是之前稳定运行的服务在节点上时，该节点被强制释放，K8s 调度这些 pod 去新的节点）对生产环境有很大的安全隐患。
- 如上内容，自治模式适用于无状态的服务运维部署，安装 Doris 集群推荐非自治模式 - 推荐使用操作系统镜像：Amazon Linux 2

5.4.3.1.2 已有集群

在已有集群上（非自治模式），可以通过 Doris Operator 运行 Doris 集群，除非该集群被限制使用 K8s 的特权模式。建议已有集群配置新的节点组来单独进行 Doris 集群资源的部署和维护，涉及到 Doris BE 运行的系统设置，可能会对宿主机的系统参数进行调整。

5.4.3.1.3 镜像仓库访问

在 EKS 上如果需要访问 DockerHub 公共镜像仓库，需要为集群添加 `Amazon VPC CNI`, `CoreDNS`, `kube-proxy` 等网络插件，并为集群配置 VPC 时，选择可访问公共环境的子网。

5.4.3.1.4 特权模式说明

EKS 下，EC2 实例是完全属于当前 EKS 用户的，不存在不同用户集群在资源池中相互影响而禁掉 K8s 特权模式的情况。

- 若您的 EKS 允许特权模式（默认允许），则无需关心系统参数，Doris Operator 默认会为 Doris 运行调整系统参数。
- 若不允许特权模式，则需要在宿主机上进行如下系统参数调整：

- 修改虚拟内存区域数量：`sysctl -w vm.max_map_count=2000000` 调整虚拟内存的最大映射数量。通过 `sysctl vm.max_map_count` 查看。
- 关闭透明大页：透明大页对性能可能有不利影响，因此需要关闭它。通过 cat /sys/kernel/mm/transparent_hugepage/enabled 是否包含 never 来判断。

- 设置最大打开文件句柄数：通过修改 /etc/security/limits.conf 来调整最大文件句柄数。通过 ulimit -n 来查看。
- 禁用 swap：swapoff -a 用于禁用所有 swap 分区和文件。通过 swapon --show 验证，未开启则无输出。

5.4.3.1.5 存储配置

Doris Operator 在生产环境一定需要用到持久化配置，用来保存节点状态，推荐 EBS 存储。

需要有以下注意事项：

- 在集群配置安装或者管理界面，为其添加 EBS 存储插件，若使用 EKS 自治模式（不推荐），则推荐安装 EFS，并且存储插件需要拥有相应的 角色权限
- 保证 EKS 节点的 IAM 角色有以下权限：
- AmazonEC2FullAccess
- AmazonEKSWorkerNodePolicy
- AmazonEKS_CNI_Policy
- AmazonSSMManagedInstanceCore

5.4.3.1.6 计算资源池配置

- 节点组配置（推荐）
  可以在集群创建界面创建节点组，也可以在集群初始化完成后进行节点组的添加。通过 EC2 > 启动模版 > 创建启动模板来设置节点池的节点组启动模板。通过模板注入脚本来自动化调整 EC2 实例的系统环境配置，确保节点在启动时自动配置所需的系统参数。通过配置节点模板的方式，也可以实现在使用 EKS 自动弹性扩缩容的时候，自动配置新增节点系统参数的能力。
  示例启动脚本：

```
#!/bin/bash
chmod +x /etc/rc.d/rc.local
echo "sudo systemctl stop firewalld.service" >> /etc/rc.d/rc.local
echo "sudo systemctl disable firewalld.service" >> /etc/rc.d/rc.local
echo "sysctl -w vm.max_map_count=2000000" >> /etc/rc.d/rc.local
echo "swapoff -a" >> /etc/rc.d/rc.local
current_limit=$(ulimit -n)
desired_limit=1000000
config_file="/etc/security/limits.conf"
if [ "$current_limit" -ne "$desired_limit" ]; then
  echo "* soft nofile 1000000" >> "$config_file"
  echo "* hard nofile 1000000" >> "$config_file"
fi
```

另外，创建节点组的时候，若想通过命令行访问，需要配置远程节点访问权限。

- 内置节点池配置（不推荐）

开启 EKS 自治模式使用的资源池，在创建节点池时，您可以选择自定义 EC2 实例类型，调整实例的 CPU、内存等资源。在节点池配置时，可以为 EC2 实例添加启动脚本来进行系统参数的调整。但是此类型资源池需要使用自治模式，并且对集群的管理自由度降低。具体修改操作详情参考：操作系统检查

5.5 Doris Streamloader

5.5.1 概述

Doris Streamloader 是一款用于将数据导入 Doris 数据库的专用客户端工具。相比于直接使用 curl 的单并发导入，该工具可以提供多并发导入的功能，降低大数据量导入的耗时。拥有以下功能：

- 并发导入，实现 Stream Load 的多并发导入。可以通过 workers 值设置并发数。
- 多文件导入，一次导入可以同时导入多个文件及目录，支持设置通配符以及会自动递归获取文件夹下的所有文件。
- 断点续传，在导入过程中可能出现部分失败的情况，支持在失败点处进行继续传输。
- 自动重传，在导入出现失败的情况后，无需手动重传，工具会自动重传默认的次数，如果仍然不成功，打印出手动重传的命令。

5.5.2 获取与安装

源代码：https://github.com/apache/doris-streamloader 二进制文件：https://doris.apache.org/zh-CN/download

> 获取结果即为可执行二进制。

5.5.3 使用方法

```
doris-streamloader --source_file={FILE_LIST} --url={FE_OR_BE_SERVER_URL}:{PORT} --header={
    ↪ STREAMLOAD_HEADER} --db={TARGET_DATABASE} --table={TARGET_TABLE}
```

1. FILE_LIST 支持：

- 单个文件

  例如：导入单个文件 file.csv

```
doris-streamloader --source_file="dir" --url="http://localhost:8330" --header="column_
    ↪ separator:|?columns:col1,col2" --db="testdb" --table="testtbl"
```

- 单个目录

  例如：导入单个目录 dir

```
doris-streamloader --source_file="dir" --url="http://localhost:8330" --header="column_
    ↪ separator:|?columns:col1,col2" --db="testdb" --table="testtbl"
```

- 带通配符的文件名（需要用引号包围）

  例如：导入 file0.csv, file1.csv, file2.csv

```
doris-streamloader --source_file="file*" --url="http://localhost:8330" --header="column_
    ↪ separator:|?columns:col1,col2" --db="testdb" --table="testtbl"
```

- 逗号分隔的文件名列表

  例如：导入 file0.csv, file1.csv file2.csv

```
doris-streamloader --source_file="file0.csv,file1.csv,file2.csv" --url="http://localhost
    ↪ :8330" --header="column_separator:|?columns:col1,col2" --db="testdb" --table="testtbl
    ↪ "
```

- 逗号分隔的目录列表

例如：导入 dir1, dir2,dir3

```
 doris-streamloader --source_file="dir1,dir2,dir3" --url="http://localhost:8330" --header="
    ↪ column_separator:|?columns:col1,col2" --db="testdb" --table="testtbl"
```

2. STREAMLOAD_HEADER 支持 Stream Load 的所有参数，多个参数之间用 '?' 分隔。

用法举例：

```
doris-streamloader --source_file="data.csv" --url="http://localhost:8330" --header="column_
    ↪ separator:|?columns:col1,col2" --db="testdb" --table="testtbl"
```

上述参数均为必要参数，下面介绍可选参数：

参数名	含义	默认值	建议
-u	数据库用户名	root	

参数名	含义	默认值	建议
-p	数据库用户对应的密码	空字符串	

参数名	含义	默认值	建议
–compress	导入数据是否在 HTTP 传输时压缩	false	保持默认，打开后压缩解压会分别增加工具和 Doris BE 的 CPU 压力，所以仅在数据源所在机器网络带宽瓶颈时打开

参数名	含义	默认值	建议
– timeout	向 Doris 发送 HTTP 请求的超时时间，单位：秒	60*60*10	保持默认
– batch	文件批量读取和发送的粒度，单位：行	4096	保持默认

参数名	含义	默认值	建议
– batch_byte	文件批量读取和发送的粒度，单位：byte	943718400 (900MB)	保持默认

参数名	含义	默认值	建议
–workers	导入的并发数	0	设置成 0 为自动模式，会根据导入数据的大小，磁盘的吞吐量，Stream Load 导入速度计算一个值。也可以手动设置，性能好的集

参数名	含义	默认值	建议
–disk_throughput	磁盘的吞吐量，单位MB/s	800	通常保持默认即可。该值参与–workers的自动推算过程。如果希望通过工具能计算出一个适当的workers数，可以根据实际磁

参数名	含义	默认值	建议
–streamload_throughput	Stream Load 导入实际的吞吐大小，单位 MB/s	100	通常保持默认即可。该值参与 –workers 的自动推算过程。默认值是通过每日性能测试环境给出的 Stream Load 吞吐量以及性能可

参数名	含义	默认值	建议
– max_byte_per_task	每个导入任务数据量的最大大小，超过这个值剩下的数据会被拆分到一个新的导入任务中。	107374182400 (100G)	建议设置一个很大的值来减少导入的版本数。但如果遇到 body exceed max size 错误且不想调整 streaming_load_max_mb 参数（需重启 be），又或是

参数名	含义	默认值	建议

| –check_utf8 |

是否对导入数据的编码进行检查：

1）false，那么不做检查直接将原始数据导入; 2）true，那么对数据中非 utf-8 编码的字符用 ﷽ 进行替代

| true | 保持默认 | |–debug | 打印 Debug 日志 | false | 保持默认 | |–auto_retry| 自动重传失败的 worker 序号和 task 序号的列表 | 空字符串 | 仅导入失败时重传使用，正常导入无需关心。失败时会提示具体参数内容，复制执行即可。例：如果 –auto_retry=“1,1,2,1” 则表示：需要重传的 task 为：第一个 worker 的第一个 task，第二个 worker 的第一个 task。 | |–auto_retry_times | 自动重传的次数 | 3 | 保持默认，如果不想重传需要把这个值设置为 0 | |–auto_retry_interval | 自动重传的间隔 | 60 | 保持默认，如果 Doris 因宕机导致失败，建议根据实际 Doris 重启的时间间隔来设置该值 | |–log_filename | 日志存储的位置 | “” | 默认将日志打印到控制台上，如果要打印到日志文件中，可以设置存储日志文件的路径，如 –log_filename = “/var/log” |

5.5.4 结果说明

无论成功与失败，都会显示最终的结果，结果参数说明：

参数名	描述
Status	导入成功（Success）与否（Failed）
TotalRows	想要导入文件中所有的行数
FailLoadRows	想要导入文件中没有导入的行数
LoadedRows	实际导入 Doris 的行数
FilteredRows	实际导入过程中被 Doris 过滤的行数
UnselectedRows	实际导入过程中被 Doris 忽略的行数
LoadBytes	实际导入的 byte 大小
LoadTimeMs	实际导入的时间
LoadFiles	实际导入的文件列表

具体例子如下：

- 导入成功，成功信息如下：

```
Load Result: {
        "Status": "Success",
        "TotalRows": 120,
        "FailLoadRows": 0,
        "LoadedRows": 120,
        "FilteredRows": 0,
        "UnselectedRows": 0,
        "LoadBytes": 40632,
```

```
        "LoadTimeMs": 971,
        "LoadFiles": [
                "basic.csv",
                "basic_data1.csv",
                "basic_data2.csv",
                "dir1/basic_data.csv",
                "dir1/basic_data.csv.1",
                "dir1/basic_data1.csv"
        ]
}
```

- 导入失败：如果导入过程中部分数据没有导入失败了，会给出重传信息，如：

```
load has some error, and auto retry failed, you can retry by :
./doris-streamloader --source_file /mnt/disk1/laihui/doris/tools/tpch-tools/bin/tpch-data/
    ↪ lineitem.tbl.1  --url="http://127.0.0.1:8239" --header="column_separator:|?columns: l_
    ↪ orderkey, l_partkey, l_suppkey, l_linenumber, l_quantity, l_extendedprice, l_discount,
    ↪ l_tax, l_returnflag,l_linestatus, l_shipdate,l_commitdate,l_receiptdate,l_shipinstruct,
    ↪ l_shipmode,l_comment,temp" --db="db" --table="lineitem1" -u root -p "" --compress=false
    ↪  --timeout=36000 --workers=3 --batch=4096 --batch_byte=943718400 --max_byte_per_task
    ↪ =1073741824 --check_utf8=true --report_duration=1 --auto_retry="2,1;1,1;0,1" --auto_
    ↪ retry_times=0 --auto_retry_interval=60
```

只需复制运行该命令即可，auto_retry 说明可参考，并给出失败的结果信息：

```
Load Result: {
      "Status": "Failed",
      "TotalRows": 1,
      "FailLoadRows": 1,
      "LoadedRows": 0,
      "FilteredRows": 0,
      "UnselectedRows": 0,
      "LoadBytes": 0,
      "LoadTimeMs": 104,
      "LoadFiles": [
              "/mnt/disk1/laihui/doris/tools/tpch-tools/bin/tpch-data/lineitem.tbl.1"
      ]
}
```

5.5.5 最佳实践

5.5.5.1 1. 参数推荐

1. 必要参数，一定要配置：--source_file=FILE_LIST --url=FE_OR_BE_SERVER_URL_WITH_PORT --header=
   ↪ STREAMLOAD_HEADER --db=TARGET_DATABASE --table=TARGET_TABLE，如果需要导入多个文件时，推荐使用 source_file 方式。

2. workers，默认值为 CPU 核数，在 CPU 核数过多的场景（比如 96 核）会产生太多的并发，需要减少这个值，推荐一般设置为 8 即可。

3. max_byte_per_task，可以设置一个很大的值来减少导入的 version 数。但如果遇到 body exceed max size 错误且不想调整 streaming_load_max_mb 参数（需重启 BE），又或是遇到 -238 TOO MANY SEGMENT 错误，可以临时调小这个配置，一般使用默认即可。

4. 影响 version 数的两个参数：

- workers：worker 数越多，版本号越多，并发越高，一般使用 8 即可。
- max_byte_per_task：max_byte_per_task 越大，单个 version 数据量越大，version 数越少，但是这个值过大可能会遇到 -238 TOO MANY SEGMENT的问题。一般使用默认值即可。

5.5.5.2 2. 推荐命令

设置必要参数以及设置 workers=8 即可。

```
./doris-streamloader --source_file="demo.csv,demoFile*.csv,demoDir" --url="http://127.0.0.1:8030"
    ↪  --header="column_separator:," --db="demo" --table="test_load" --u="root" --workers=8
```

5.5.5.3 3. FAQ

- 在导入过程中，遇到了部分子任务失败的问题，当时没有断点续传的功能，导入失败后重新删表导入，如果遇到这个问题，工具会进行自动重传，如果重传失败会打印出重传命令，复制后可以手动重传。
- 该工具的默认单个导入是 100G，超过了 BE 默认的 streaming_load_max_mb 阈值如果不希望重启 BE，可以减少 max_byte_per_task 这个参数的大小。

查看 streaming_load_max_mb 大小的方法：

```
 -curl "http://127.0.0.1:8040/api/show_config"
```

- 导入过程如果遇到 -238 TOO MANY SEGMENT 的问题，可以减少 max_byte_per_task 的大小。

5.6 BI

5.6.1 Apache Superset

Apache Superset 是一个开源的数据挖掘平台，支持丰富的数据源连接，多种可视化方式，并能够对用户实现细粒度的权限控制。该工具主要特点是可自助分析、自定义仪表盘、分析结果可视化（导出）、用户/角色权限控制，还集成了一个 SQL 编辑器，可以进行 SQL 编辑查询等。

在 Apache Superset 3.1 版本中，提供了官方连接方式，正式支持了 Apache Doris 的内部数据和外部数据进行查询和可视化处理。推荐使用 Apache Doris 2.0.4 及以上版本。

通过这个连接方式，Superset 可以将 Apache Doris 数据库和表作为数据源进行集成。要启用此功能，请遵循下面的设置指南：

- 使用前所需的设置
- 在 Apache Superset 中配置 Apache Doris 数据源
- 在 Apache Superset 中构建可视化
- 连接和使用技巧

5.6.1.1 安装 Superset 和 Doris Python 客户端

1. 安装 Python3，建议版本为 3.1.11。
2. 安装 Apache Superset 3.1 及其以上的版本。具体参见 安装 Superset 从 PyPI 库。
3. 在 Apache Superset 服务器上安装 Apache Doris 的 Python 客户端，可以参考如下命令：

```
pip install pydoris
```

校验安装结果：

```
-> pip list | grep pydoris
pydoris                       1.1.0
```

环境确认无误后，接下来，就可以在 Superset 中配置一个 Doris 数据源并开始构建数据可视化！

5.6.1.2 在 Superset 中配置 Doris 数据源

现在您已安装了 Pydoris 和 Apache Superset 驱动程序，让我们来看一下如何在 Superset 中定义一个连接到 Doris 中 tpch 数据库的数据源。

1. 要通过 pydoris 连接到 Apache Doris，您需要配置 SQLAlchemy URI 连接字符串：

按此格式完成配置：

```
doris://<User>:<Password>@<Host>:<Port>/<Catalog>.<Database>
```

URI 参数说明如下：

参数	含义	示例
User	用户名	testuser
Password	密码	xxxxxx
Host	数据库 host	127.0.1.28
Port	数据库 query port	9030
Catalog	Doris Catalog，查询外表和数据湖时使用，内表为 internal	internal
Database	数据库名	tpch

2. 对 Superset 进行访问。

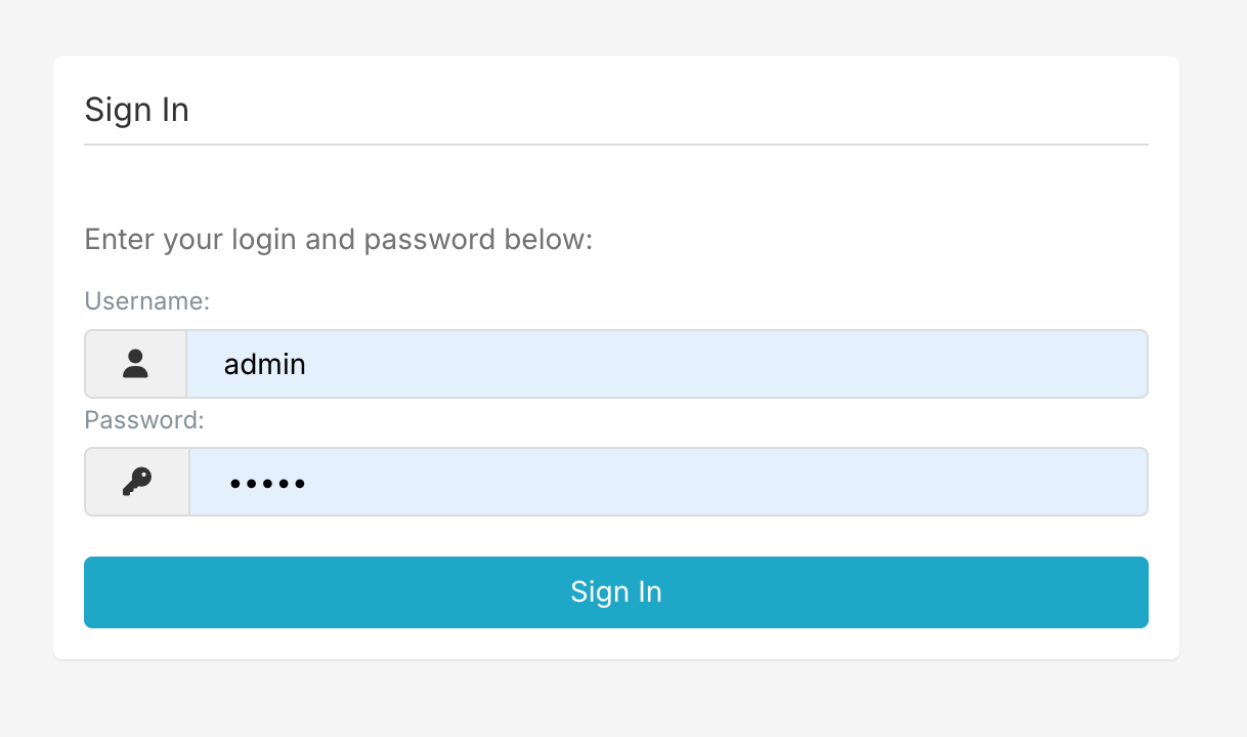


图 155:

3. 完成登录后，点击右上角 Settings -> Database Connectors

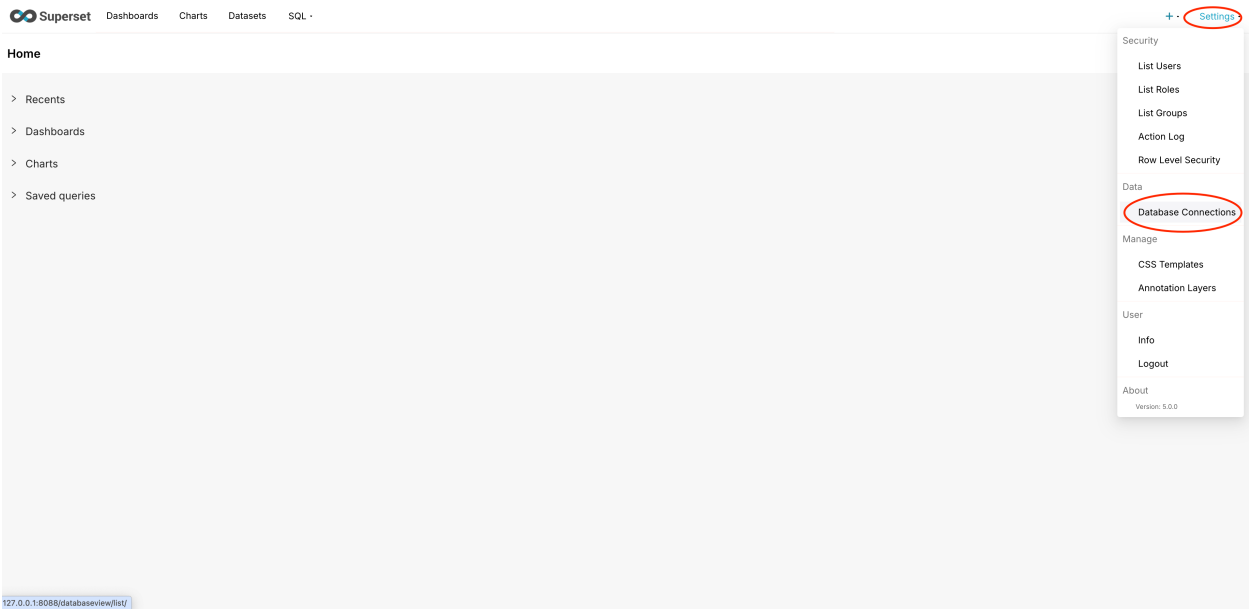


图 156:

4. 点击添加 Database，在 Connect a database 弹窗上，选择 Apache Doris：

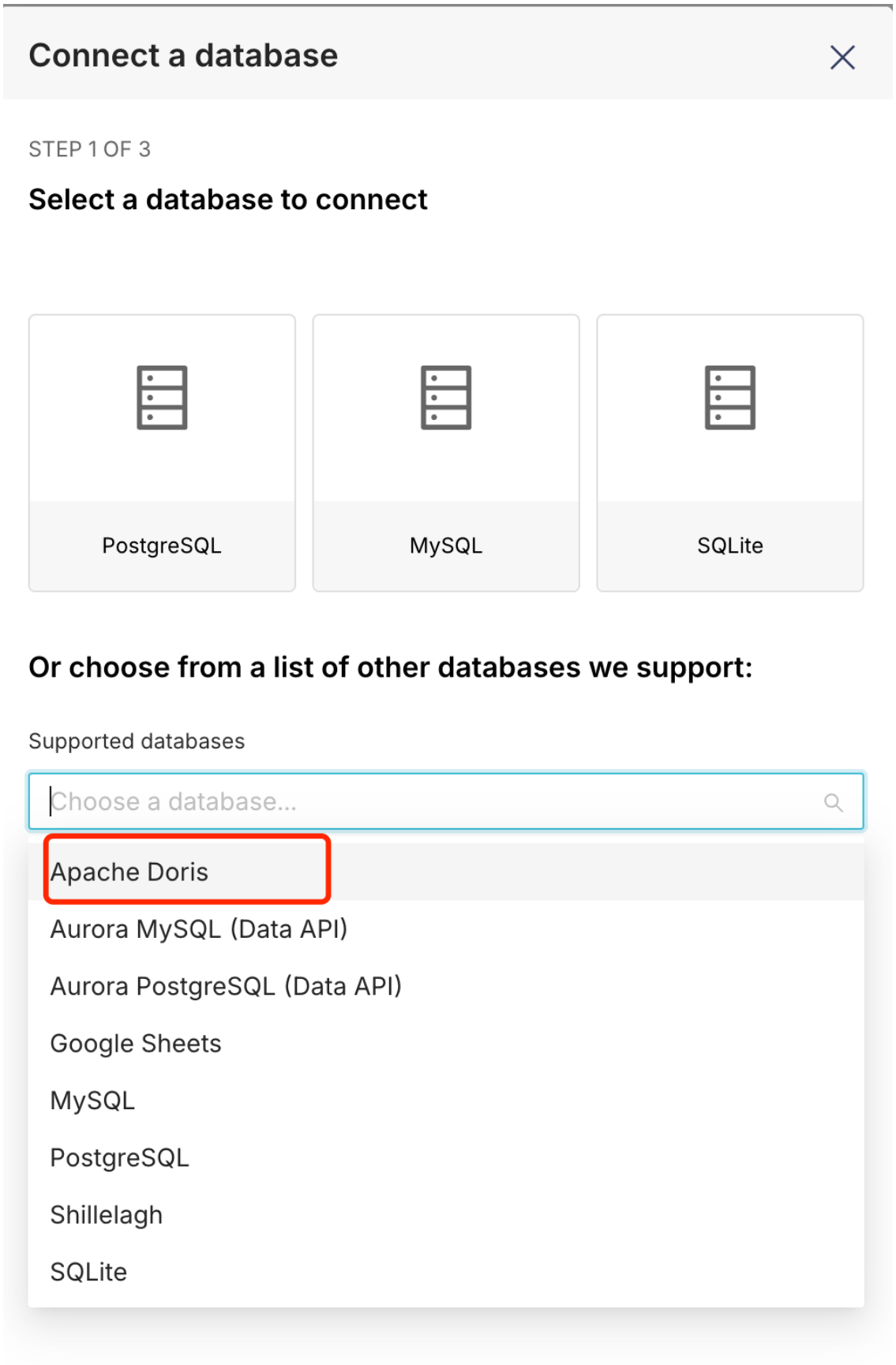

Connect a database
STEP 1 OF 3
Select a database to connect
PostgreSQL
MySQL
SQLite
Or choose from a list of other databases we support:
Supported databases
Choose a database...
Apache Doris
Aurora MySQL (Data API)
Aurora PostgreSQL (Data API)
Google Sheets
MySQL
PostgreSQL
Shillelagh
SQLite

图 157: 1840

5. 在连接信息中填写 SQLAlchemy URI，行连接验证无误后，点击 Connect。

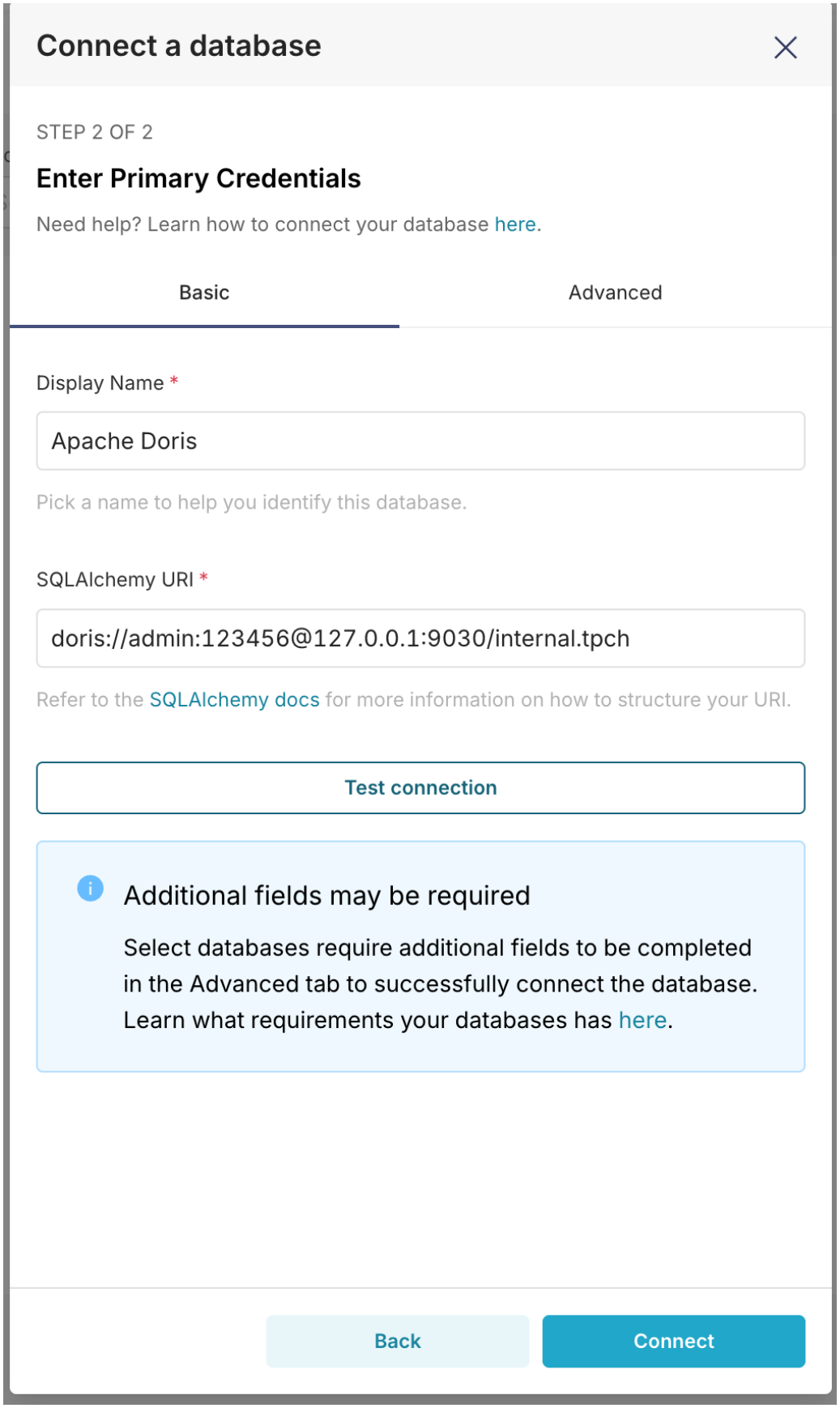

Connect a database
STEP 2 OF 2
Enter Primary Credentials
Need help? Learn how to connect your database here.
Basic
Advanced
Display Name *
Apache Doris
Pick a name to help you identify this database.
SQLAlchemy URI *
doris://admin:123456@127.0.0.1:9030/internal.tpch
Refer to the SQLAlchemy docs for more information on how to structure your URI.
Test connection
Additional fields may be required
Select databases require additional fields to be completed in the Advanced tab to successfully connect the database. Learn what requirements your databases has here.
Back
Connect

图 158:
1842

6. 自此添加数据源完成

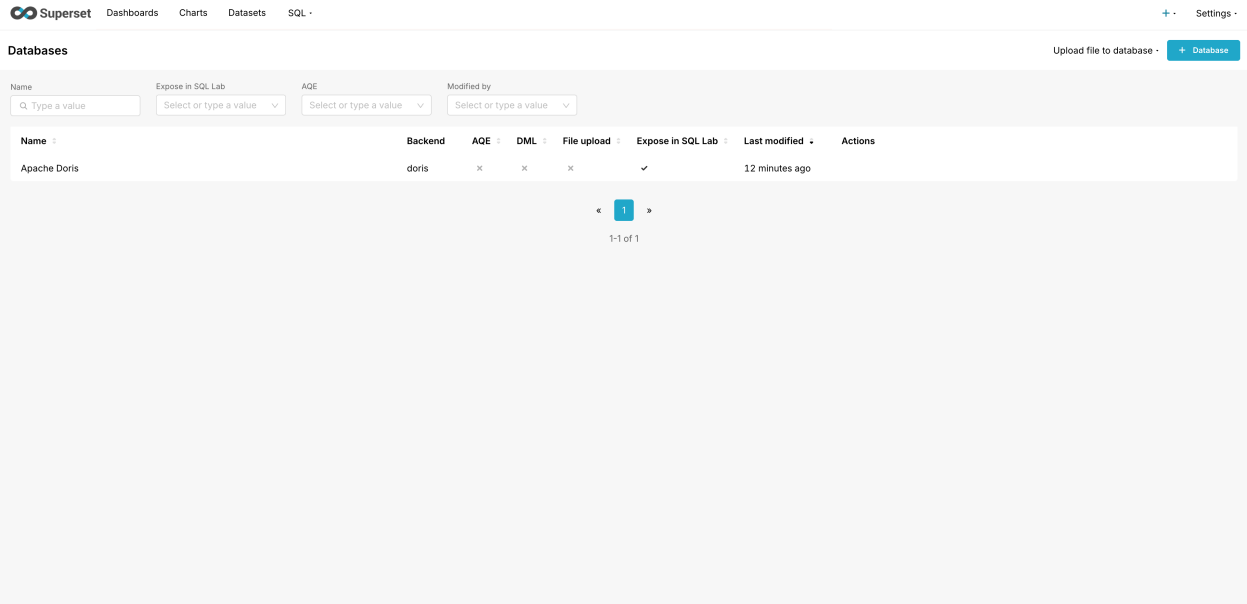


图 159:

接下来，就可以在 Superset 中构建一些可视化了！

5.6.1.3 在 Superset 中构建可视化

我们选择 TPC-H 数据作为数据源，Doris TPC-H 数据源构建方式参考此文档

现在我们在 Superset 中配置了 Doris 数据源，让我们可视化数据…

假设我们需要分析不同货运方式的订单的金额随时间增长曲线用以成本分析

1. 点击 Datasets 添加 Dataset

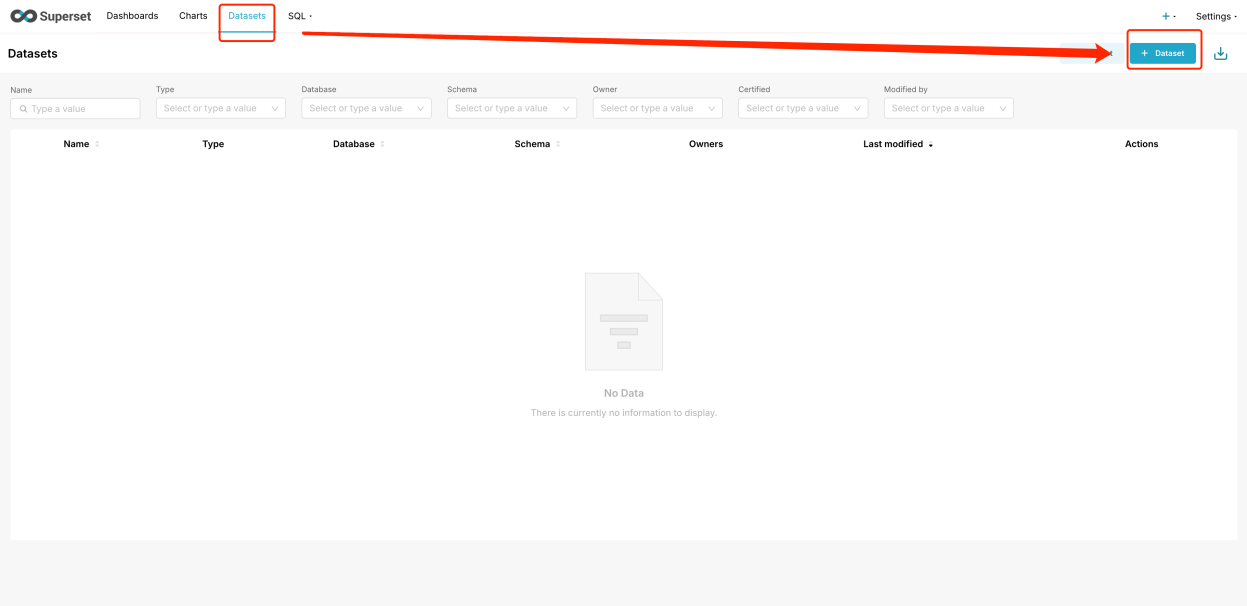


图 160:

2. 依次选择，然后点击右下角 Create dataset and create chart

   - Database： Doris
   - Schema： tpch
   - Table： lineitem

图 161:

3. 编辑 lineitem 这个 Dateset

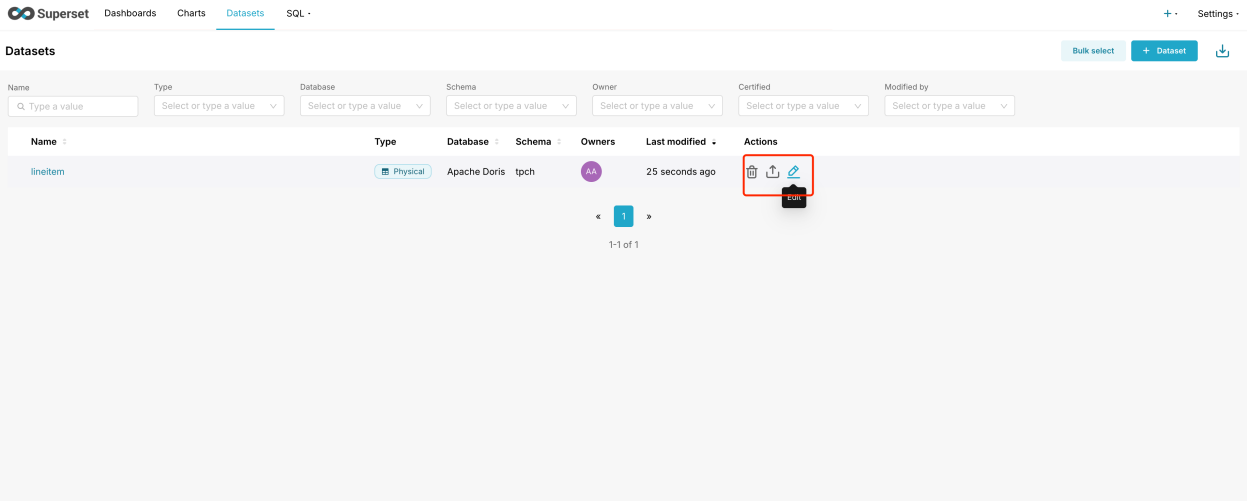


图 162:

4. 点击 Metrics -> Add item，为其添加计算指标

    - Metric Key：Revenue
    - SQL expression：SUM(l_extendedprice*（1 -l_discount))

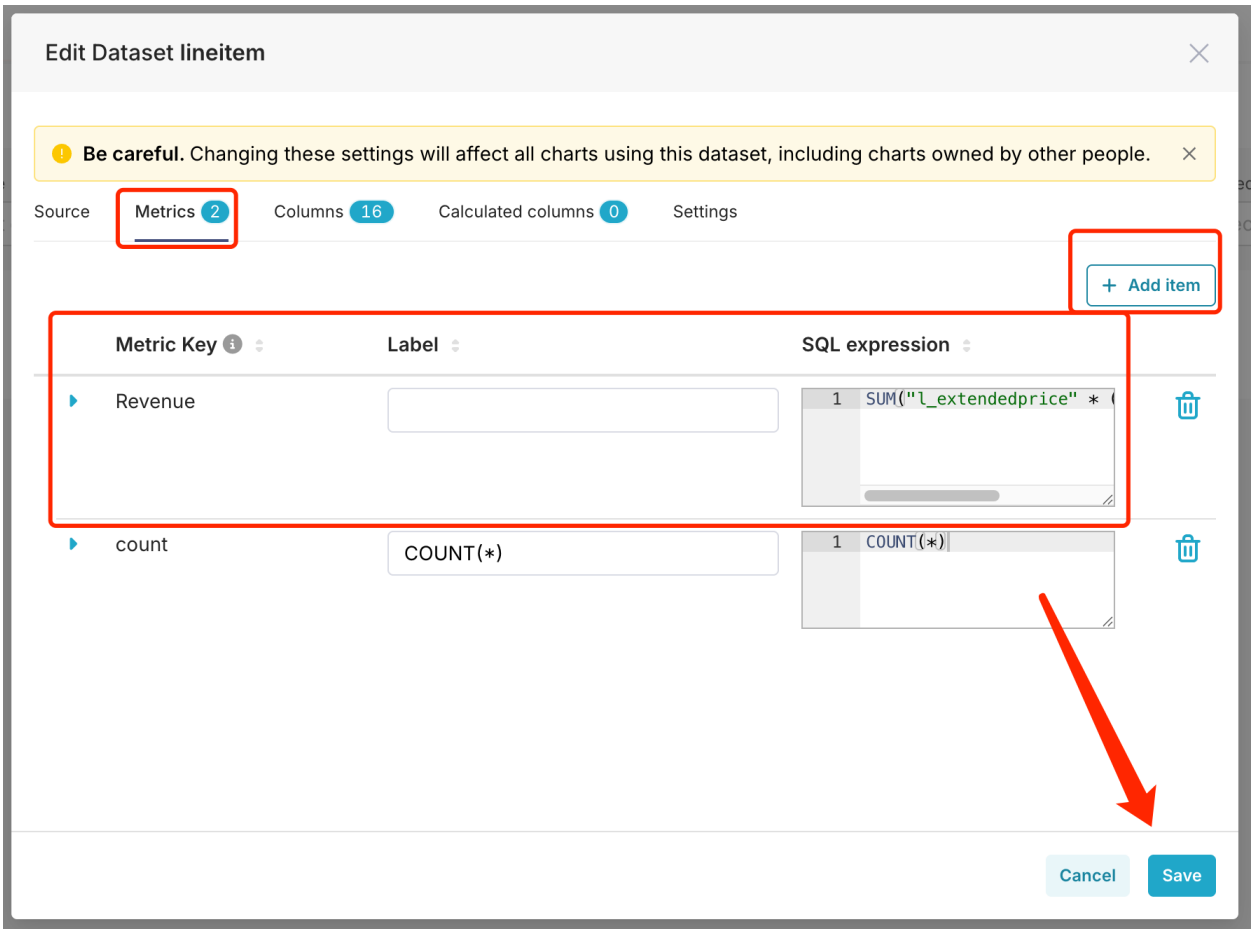


图 163:

5. 进入 Chart -> 添加 Chart，dateset 选中 lineitem ，chart 类型选中 Line Chart

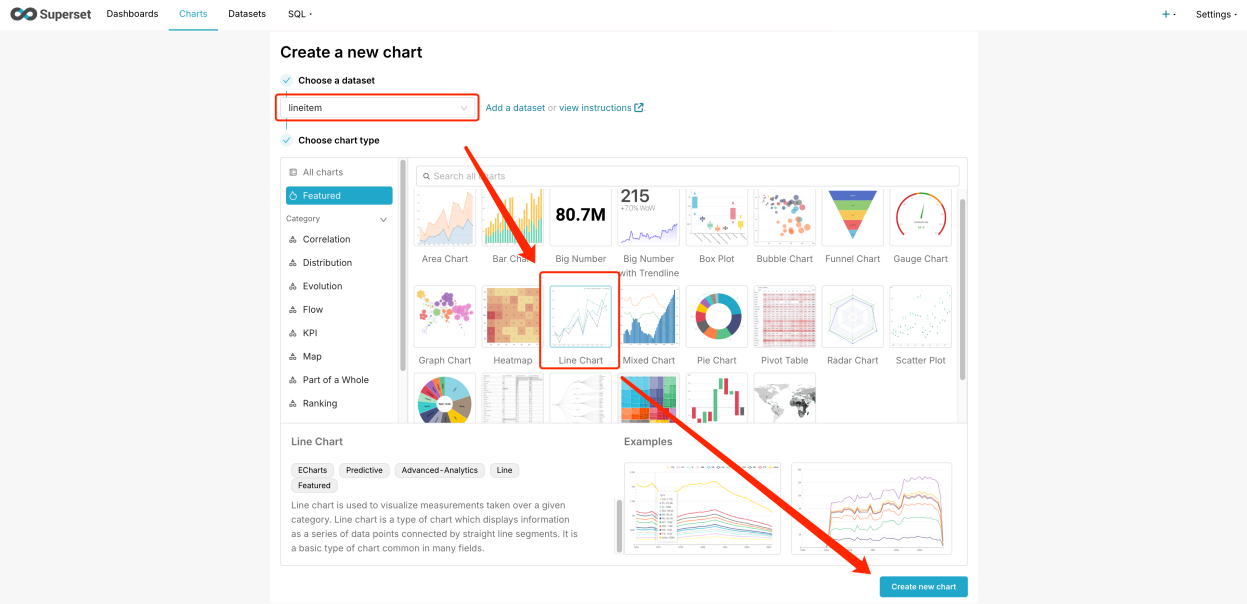


图 164:

6. 将 l_shipdate 拖拽到 X 轴，并且设置时间粒度，同时依次将 Revenum 自定义指标和数据列 l_shipmode 分别拖拽到 Metrcs 和 Dimensions 处

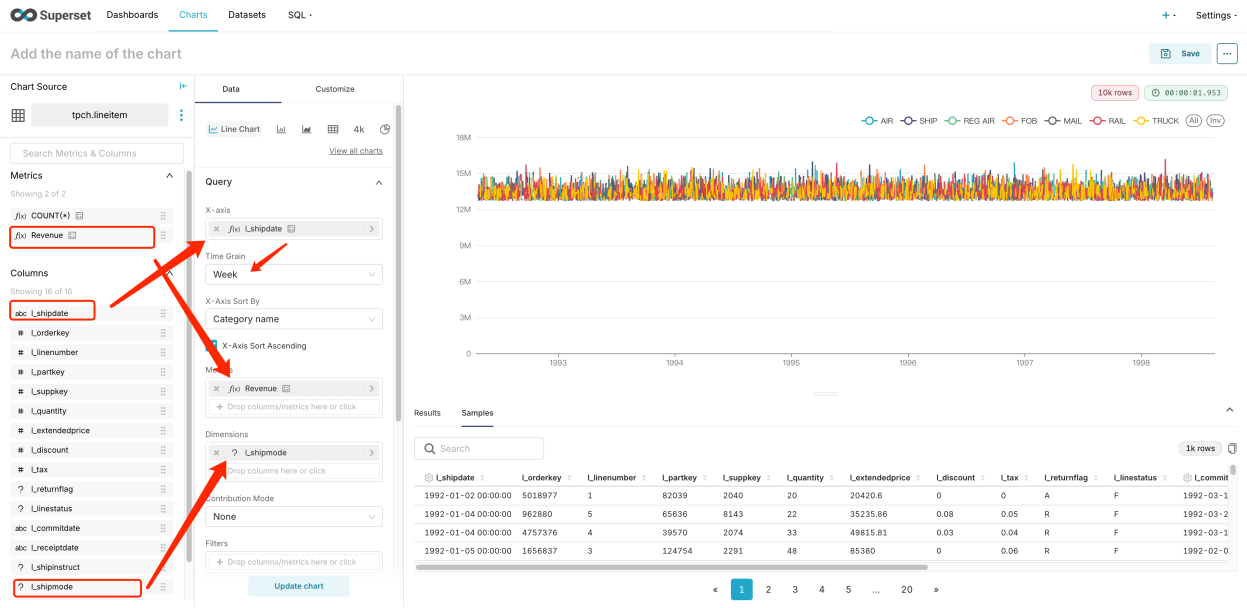


图 165:

7. 点击 Update chart 即可查看看板内容。点击 Save 保存看板

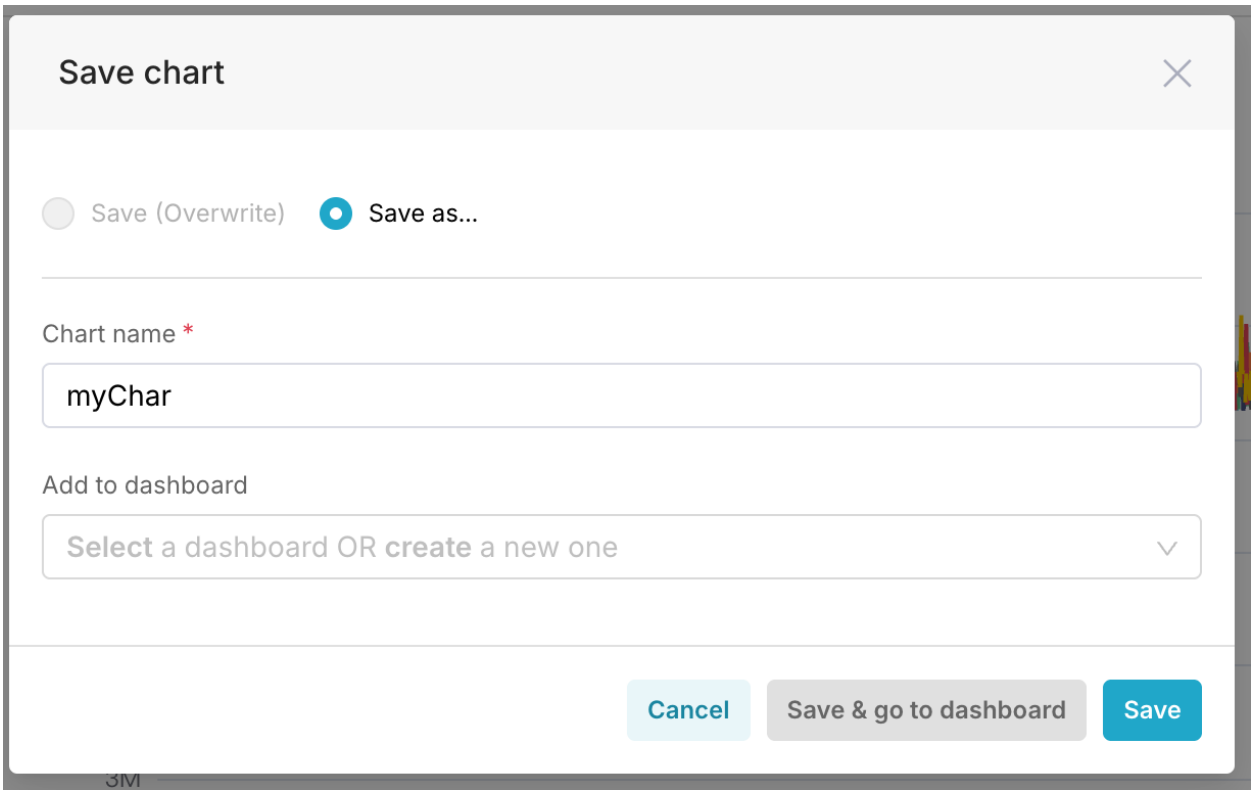


图 166:

至此，已经成功将 Superset 连接到 Apache Doris，并实现了数据分析和可视化看板制作。

5.6.1.4 连接和使用技巧

- 在 Superset 环境下预先安装 pydoris，才可以在创建数据库的时候选择 Apache Doris
- 根据实际需求，合理创建 doris 库表，按时间分区分桶，可有效减少谓词过滤和大部分数据传输
- 建议使用 VPC 私有连接，避免公网访问引入安全风险。
- 细化 Doris 用户账号角色和访问权限，避免过度下放权限。

5.6.2 FineBI

5.6.2.1 FineBI 介绍

FineBI 是一款商业智能产品，其具有数据处理、即时分析、多维度分析 Dashboard 等多种功能系统架构，FineBI 支持丰富的数据源连接、多种视图对表的分析管理。FineBI 可以顺利支持 Apache Doris 的内部数据和外部数据的建模与可视化处理。

5.6.2.2 前置条件

安装 FineBI 5.0 及以上版本，下载链接：https://www.finebi.com/

5.6.2.3 登录与连接

1. 创建登录的账户，并使用该账户登录 FineBI

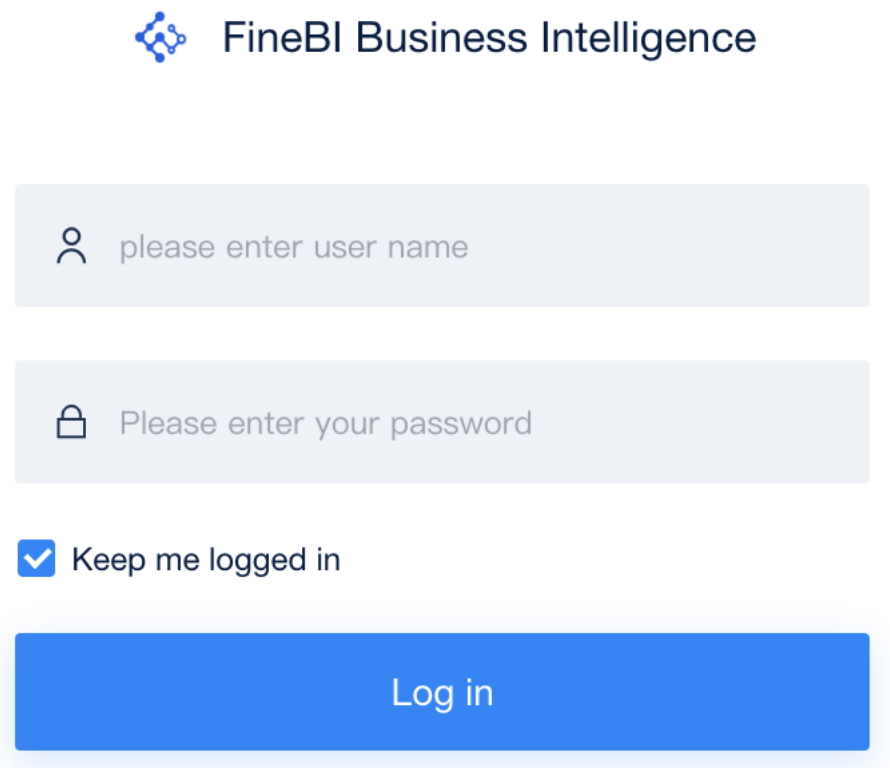


图 167: login page

2. 这里选择内置数据库，若需要选择外部数据库配置文档可参考：https://help.fanruan.com/finebi/doc-view-437.html

> Note 这里建议选择内置数据库作为帆软 BI 的信息存储库，这里选择的数据库类型不是查询分析数据的目标库，而是存储维护帆软 BI 模型、仪表盘等信息的数据库，帆软需要对其进行增删改查的操作。

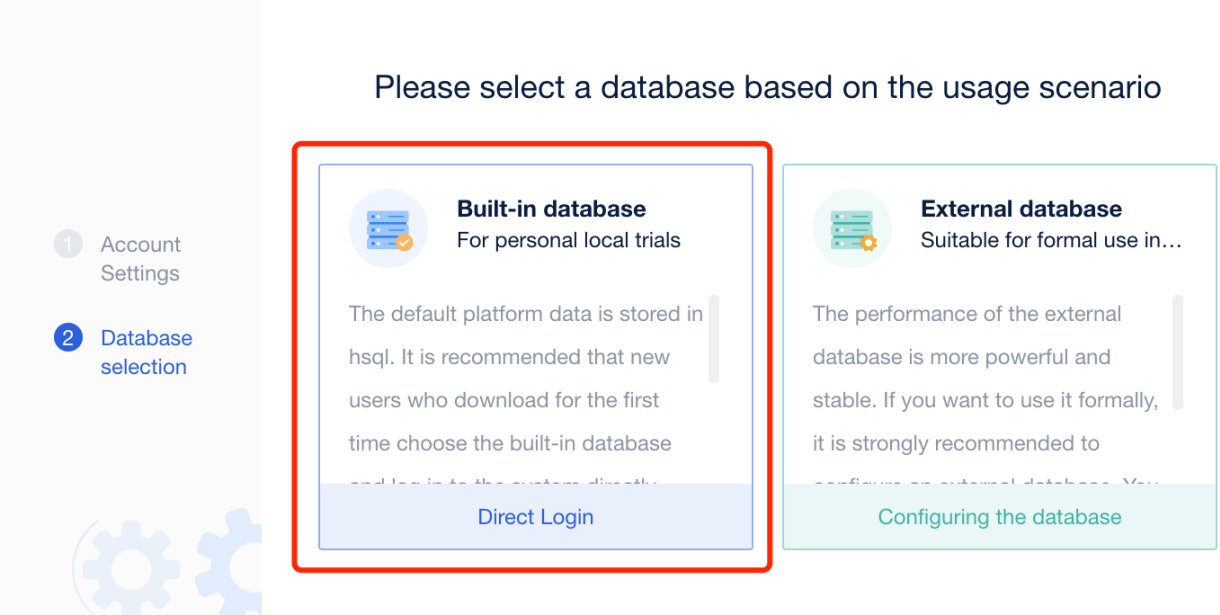


图 168: select database

3. 点击管理系统按钮之后选择数据连接中的数据库连接，进而创建一个新的数据库连接

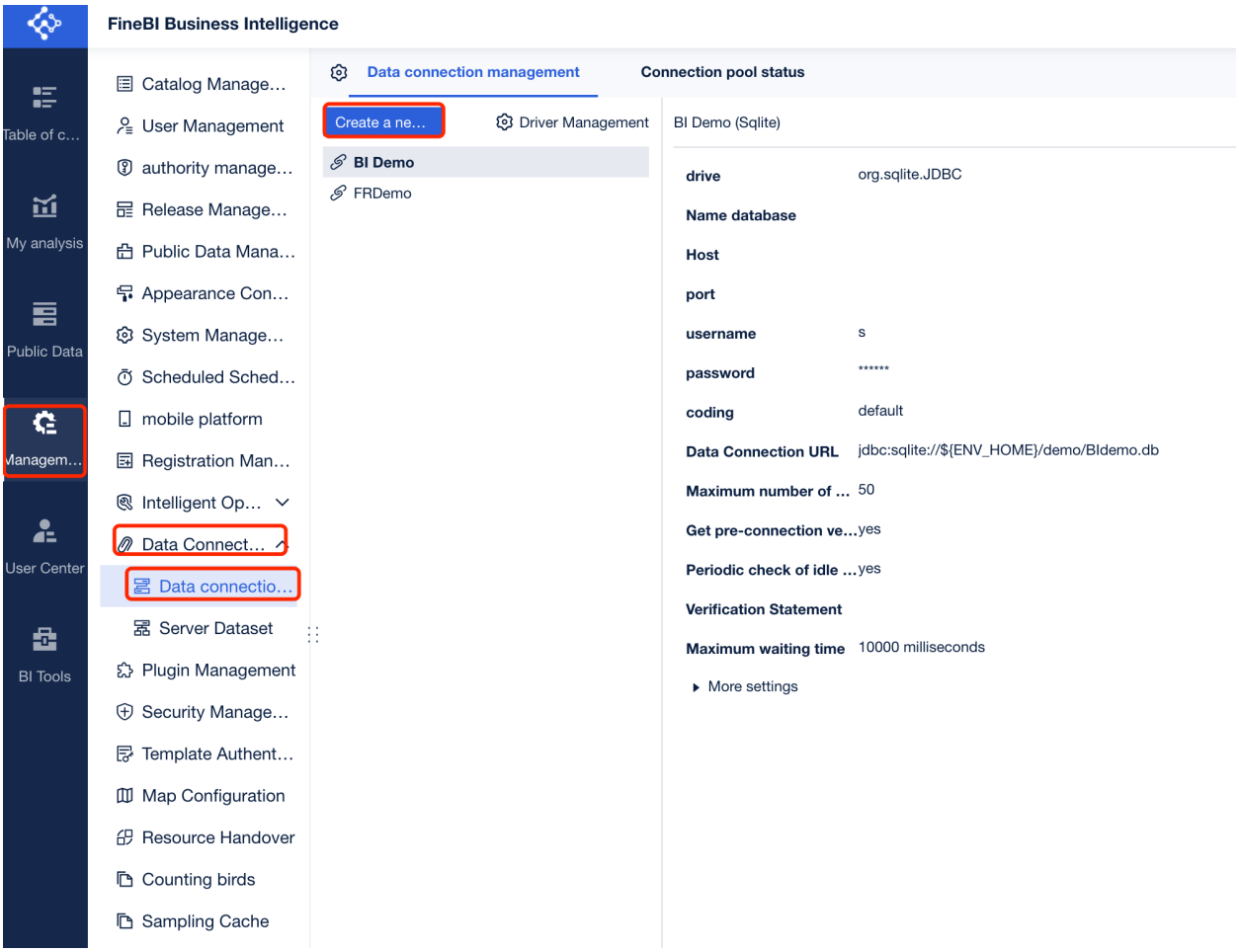


图 169: data connection

4. 在新数据库连接选择界面选择 MySQL

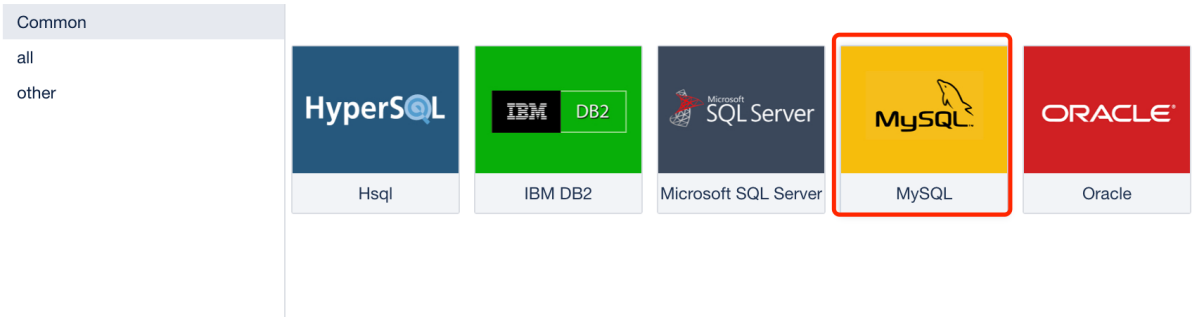


图 170: select connection

5. 填写 Doris 数据库的相关连接信息

- 参数说明如下：
  - Username：用于登录 Doris 集群的用户名，如 admin。
  - Password：用于登录 Doris 集群的用户密码。
  - Host：Doris 集群的 FE 主机 IP 地址。
  - Port：Doris 集群的 FE 查询端口，如 9030。
  - Coding：Doris 集群中的编码格式。
  - Name Database：Doris 集群中的目标数据库。

MySQL

Data connection name	Doris_connection
drive	default / com.mysql.jdbc.Driver
Name database	tpch
Host	[redacted]
port	9030
username	root
password	••••••
coding	default
Data Connection URL	jdbc:mysql:, [redacted] pch
Maximum number of ...	50
Get pre-connection ve...	yes
Periodic check of idle ...	yes
Verification Statement	Use default statement if empty
Maximum waiting time	10000 millisecond

▸ SSH Setup

▸ SSL Settings

▸ More settings

图 171: connection information

6. 点击测试链接。如果输入的连接信息正确则会弹出连接成功

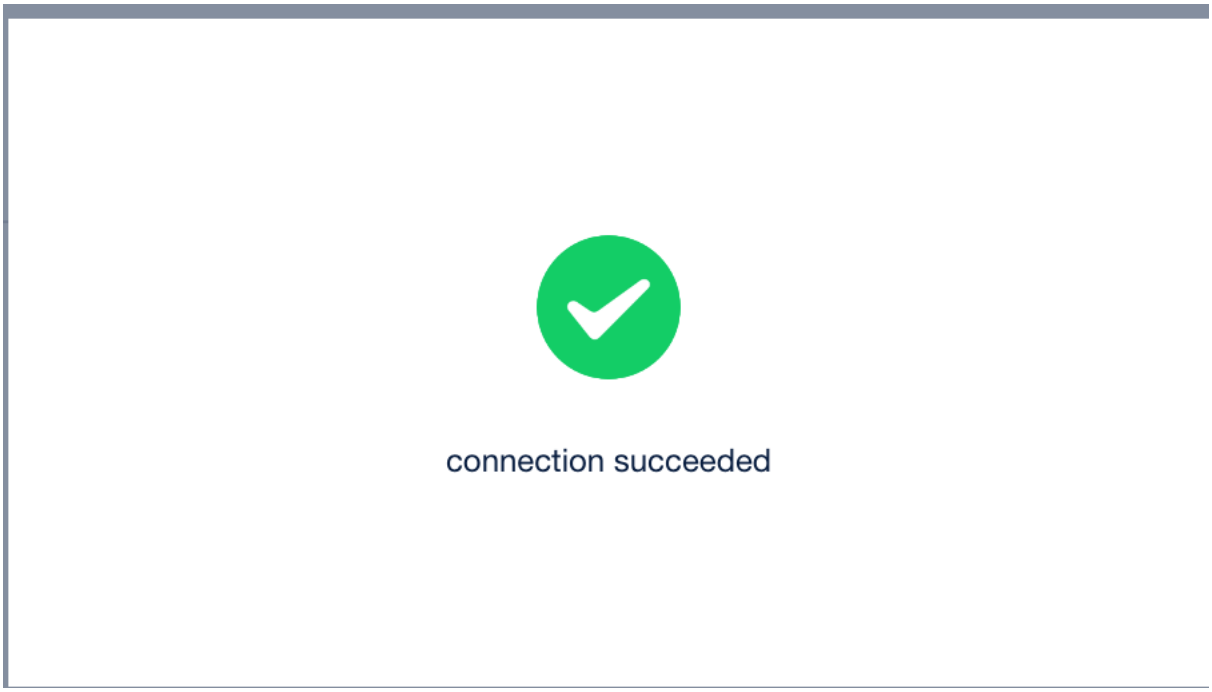


图 172: connection test

5.6.2.4 创建数据模型

1. 在公共数据中点击创建一个新的数据集，添加 Doris 数据集时可点击数据库表

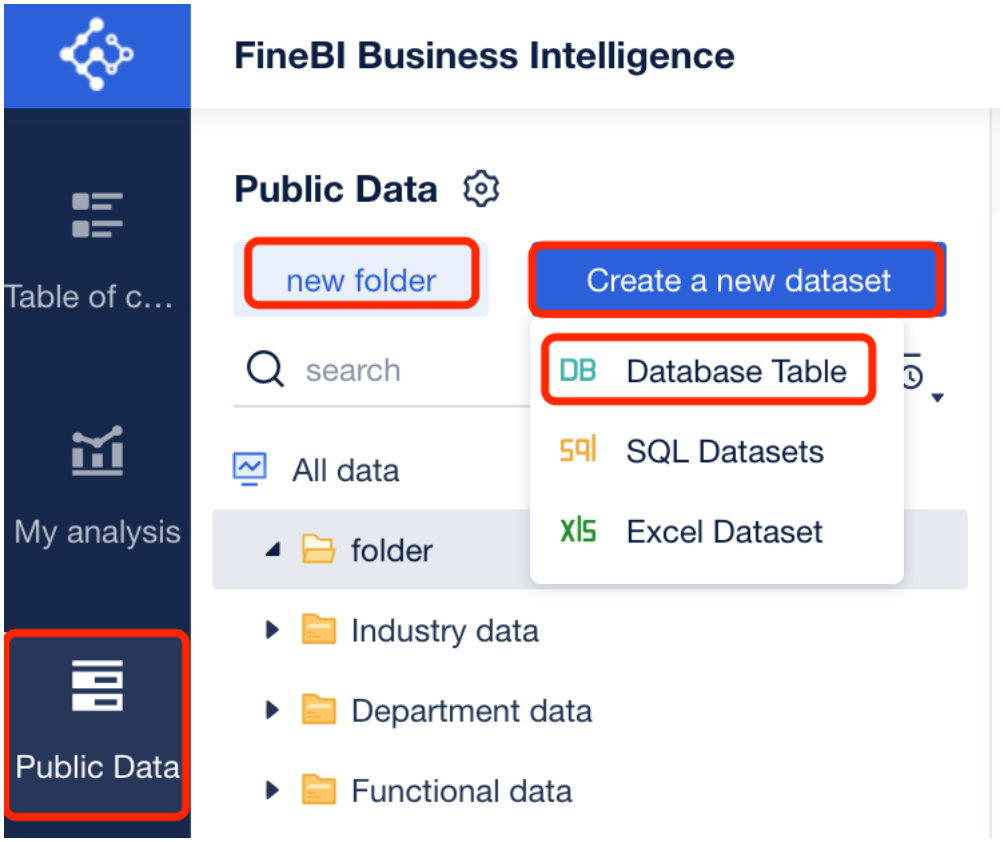


图 173: new dataset

2. 选择已有数据库连接下需要导入的表

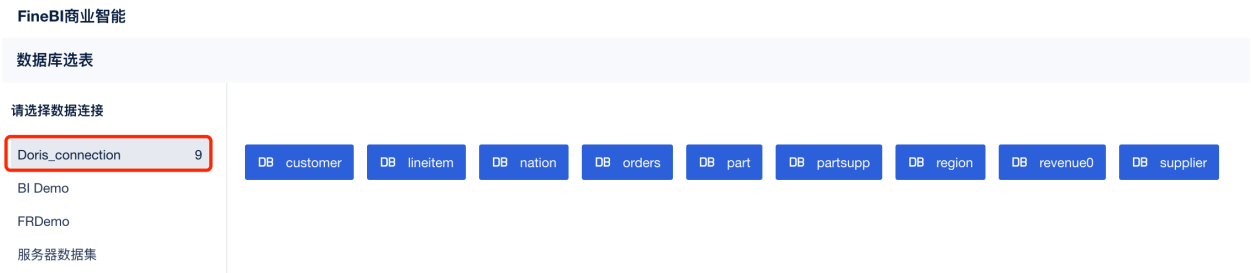


图 174: select table

3. 导入表后需要对每个导入的表进行刷新，只有刷新后才可以在主题中对该表进行数据分析

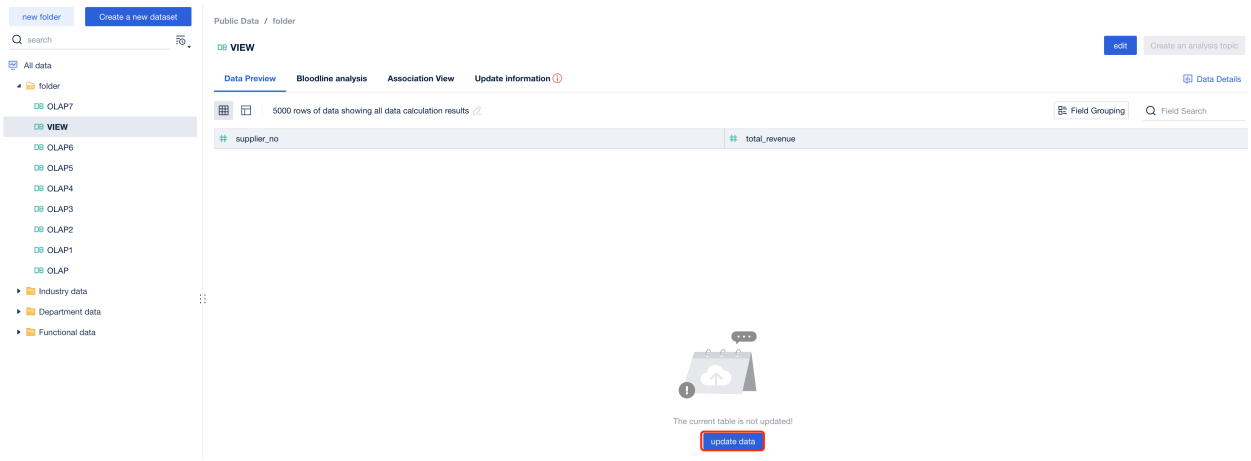


图 175: refresh table

4. 在编辑的主题中添加导入的公共数据，然后即可按照业务逻辑进行罗盘分析与配置。

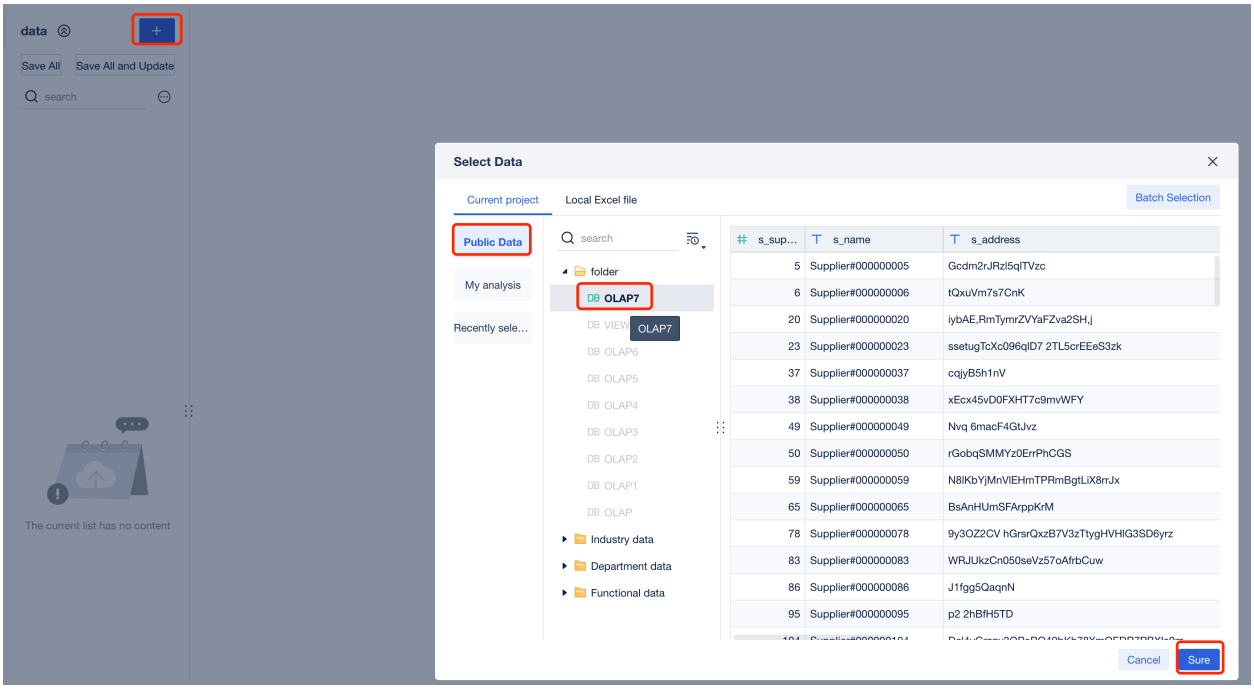


图 176: data analysis

5.6.3 Power BI

Microsoft Power BI 可以从 Apache Doris 查询或加载数据到内存。

您可以使用 Power BI Desktop，用于创建仪表板和可视化的 Windows 桌面应用程序。

本教程将指导您完成以下过程：

- 安装 mysql ODBC 驱动程序
- 将 Doris Power BI 连接器安装到 Power BI Desktop
- 从 Doris 查询数据以在 Power BI Desktop 中可视化

5.6.3.1 前提条件

5.6.3.1.1 Power BI 安装

本教程假定您已经在 Windows 计算机上安装了 Microsoft Power BI Desktop。您可以在 这里 下载并安装 Power BI Desktop。

我们建议您更新到最新版本的 Power BI。

5.6.3.1.2 连接信息

收集您的 Apache Doris 连接详细信息

您需要以下详细信息以连接到您的 Apache Doris 实例：

参数	含义	示例
Doris Data Source	数据库连接串，host + port	127.0.1.28:9030
Database	数据库名	test_db
Data Connectivity Mode	数据连接模式，包含 Import 和 DirectQuery	DirectQuery
SQL Statement	SQL 语句，必须包含 Database，仅适用于 Import 模式	select * from database.table
User Name	用户名	admin
Password	密码	xxxxxx

5.6.3.2 Power BI Desktop

要开始在 Power BI Desktop 中查询数据，您需要完成以下步骤：

1. 安装 Mysql ODBC 驱动程序
2. 查找 Doris 连接器
3. 连接到 Doris
4. 查询和可视化数据

5.6.3.2.1 安装 ODBC 驱动程序

下载安装 Mysql ODBC，并配置（版本为 5.3）。

执行提供的 `.msi` 安装程序并按照向导进行操作。

MySQL Connector/ODBC 5.3 - Setup Wizard
Welcome to the Setup Wizard for MySQL Connector/ODBC 5.3
The Setup Wizard will install MySQL Connector/ODBC 5.3 release 5.3.13 on your computer. To continue, click Next.
MySQL
< Back
Next >
Cancel

图 177:

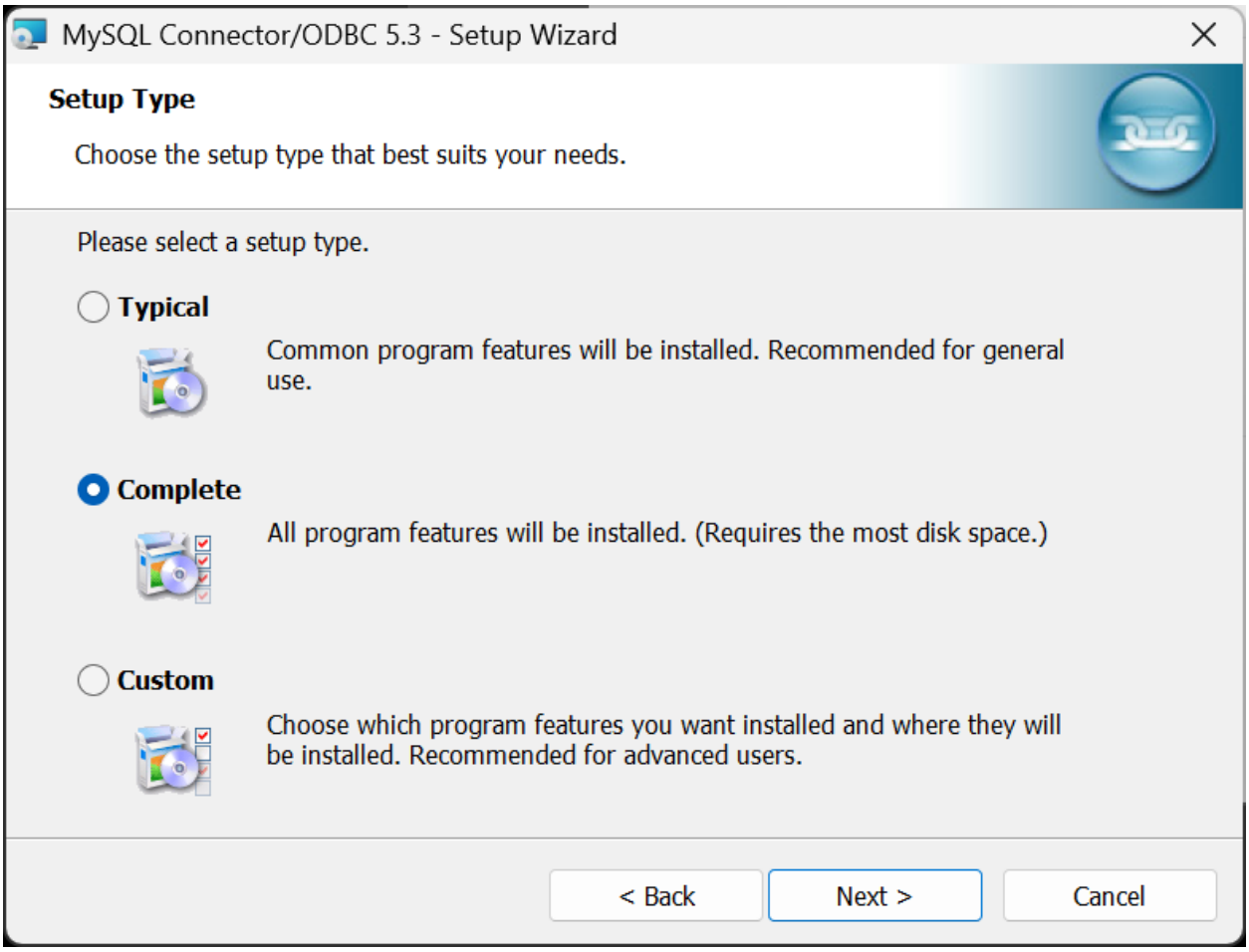

MySQL Connector/ODBC 5.3 - Setup Wizard
Setup Type
Choose the setup type that best suits your needs.
Please select a setup type.
Typical
Common program features will be installed. Recommended for general use.
Complete
All program features will be installed. (Requires the most disk space.)
Custom
Choose which program features you want installed and where they will be installed. Recommended for advanced users.
< Back
Next >
Cancel

图 178:

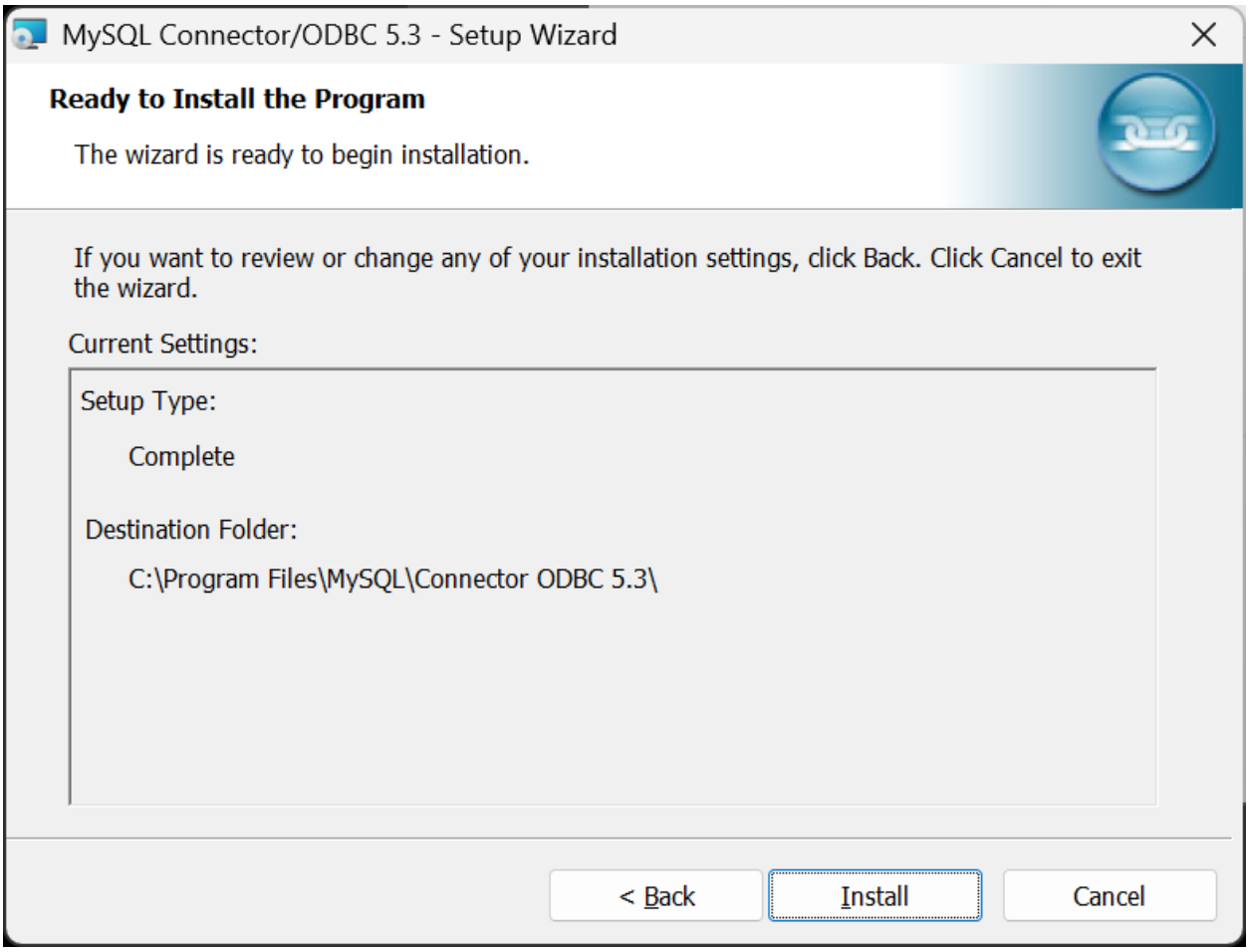

MySQL Connector/ODBC 5.3 - Setup Wizard
Ready to Install the Program
The wizard is ready to begin installation.
If you want to review or change any of your installation settings, click Back. Click Cancel to exit the wizard.
Current Settings:
Setup Type:
Complete
Destination Folder:
C:\Program Files\MySQL\Connector ODBC 5.3\
< Back
Install
Cancel

图 179:

安装完成

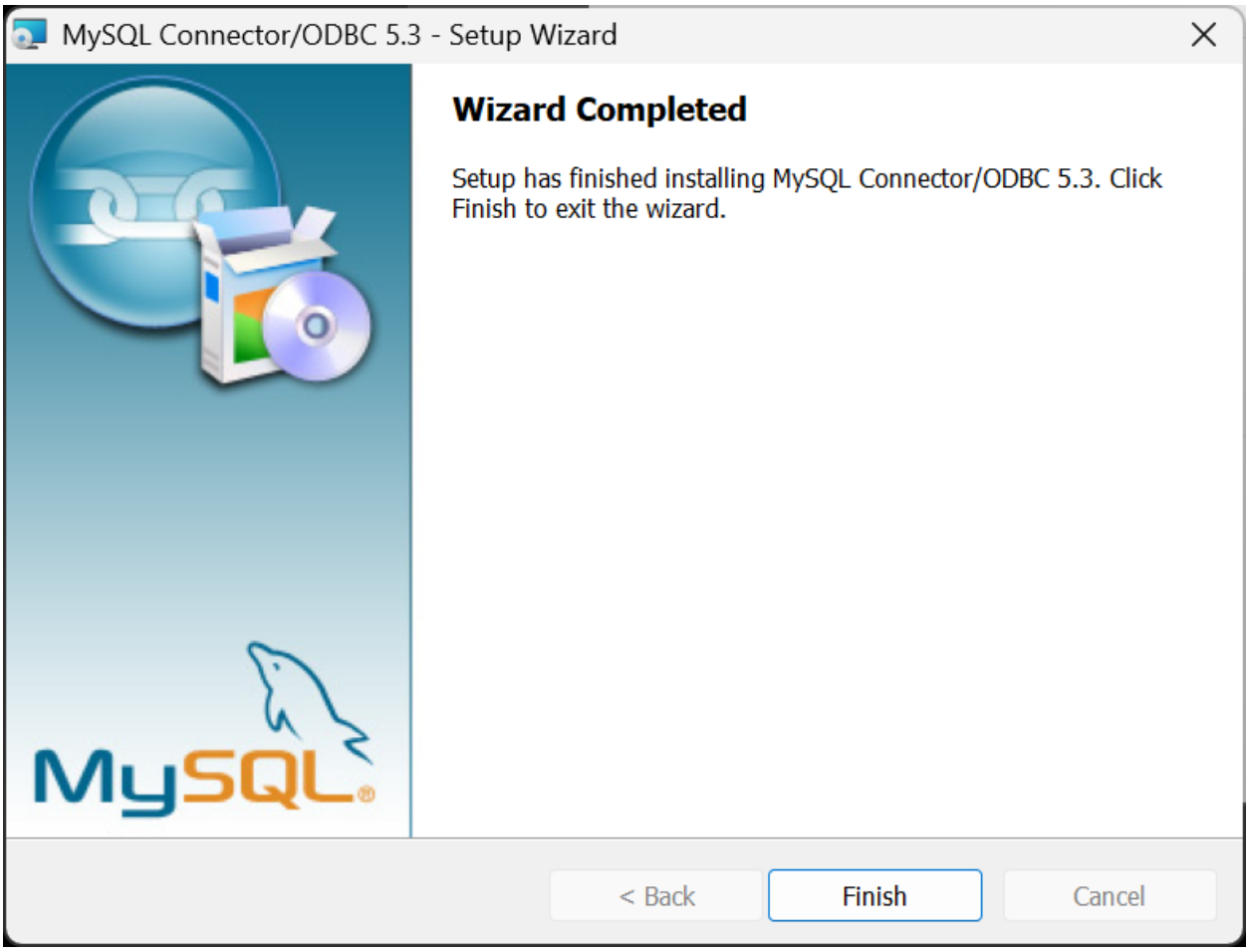


图 180:

验证 ODBC 驱动程序

当驱动程序安装完成后，您可以通过以下方式验证安装是否成功：

在开始菜单中输入 ‘ODBC’ 并选择 “ODBC 数据源 (64 位)”。

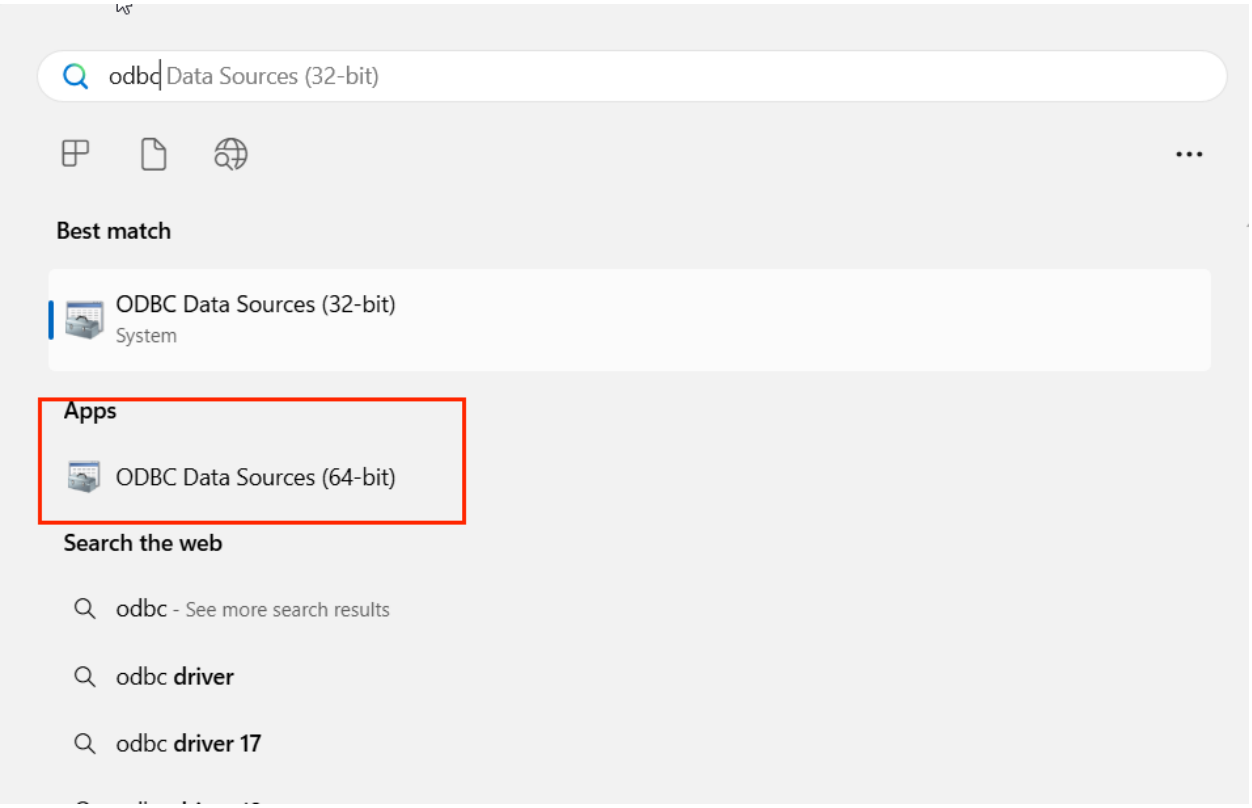


图 181:

验证 mysql 驱动程序是否列出。

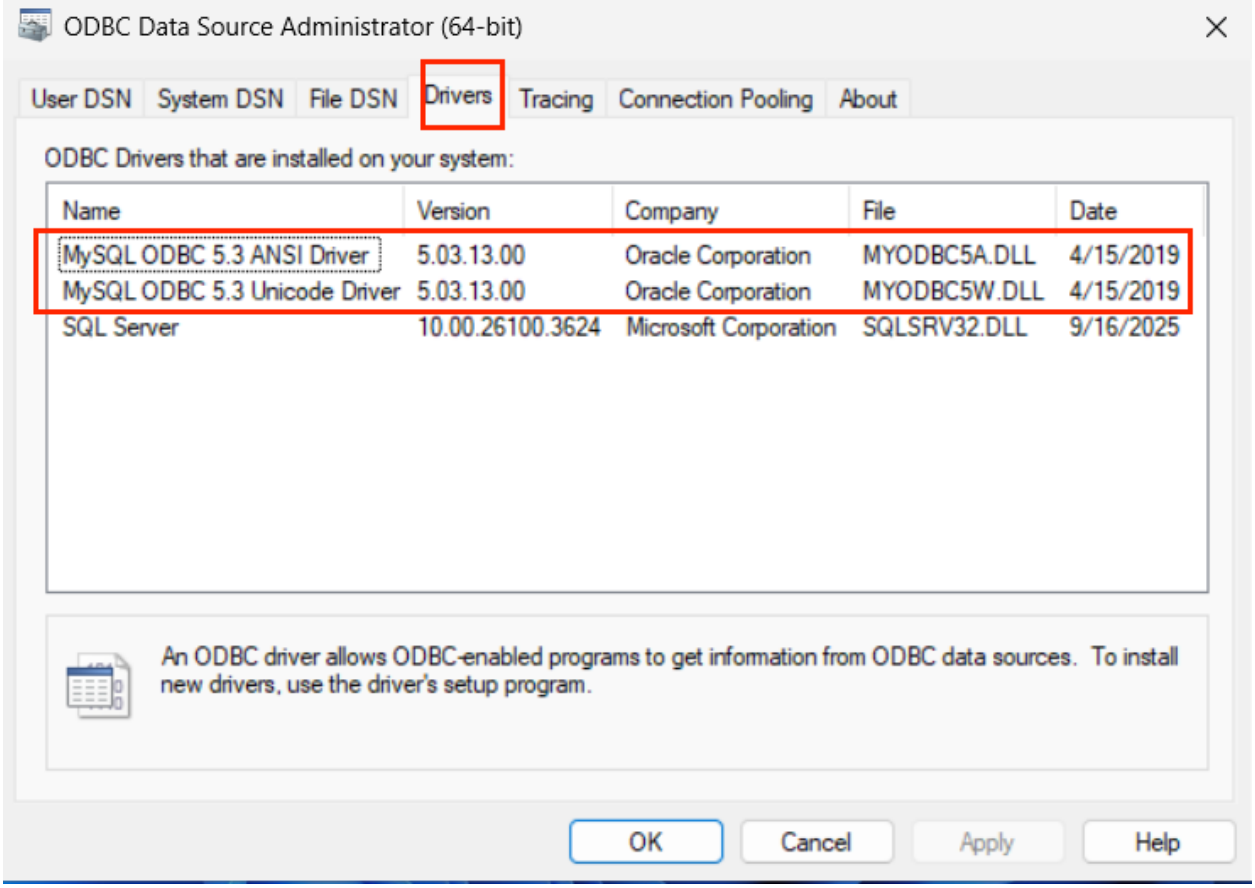


图 182:

5.6.3.2.2 安装 Doris 连接器

当前 Power BI 自定义 Connector 暂时关闭认证通道，因此 Doris 提供的自定义 Connector 是属于未经认证的，对于未认证连接器，配置方式 (https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-connector-extensibility#custom-connectors) 如下：

1. 假定 power_bi_path 为 windows 操作系统中 Power BI Desktop 的目录，一般默认为：`power_bi_path = C:\Program Files\Power BI Desktop` 参考此处路径 `%power_bi_path%\Custom Connectors folder`，放置 `Doris.mez` 自定义连接器文件（如果路径不存在，按需手动创建）。
2. 在 Power BI Desktop 中，选择 `File > Options and settings > Options > Security`，在 `Data Extensions` 下，勾选 `(Not Recommended)Allow any extension to load without validation or warning`。可以屏蔽掉未认证连接器的限制。

首先，选择 `File`

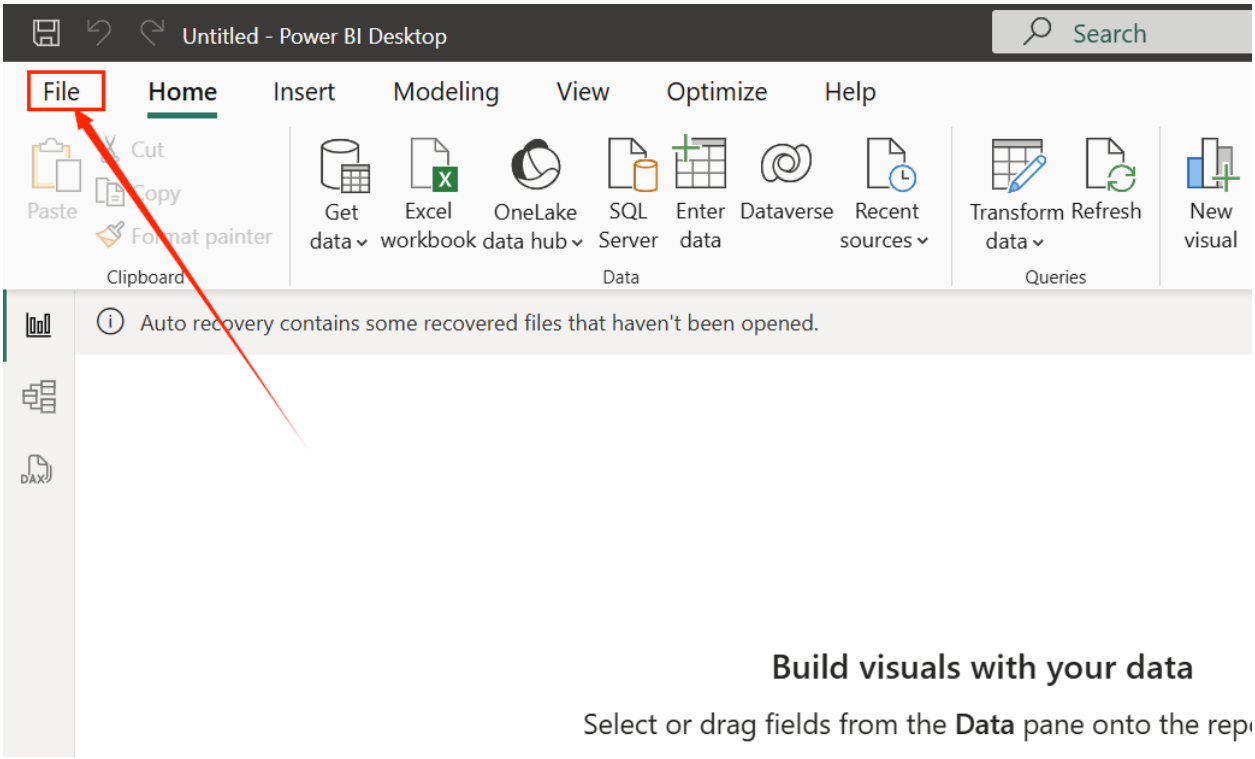


图 183:

然后，选择 `Options and settings` > `Options`

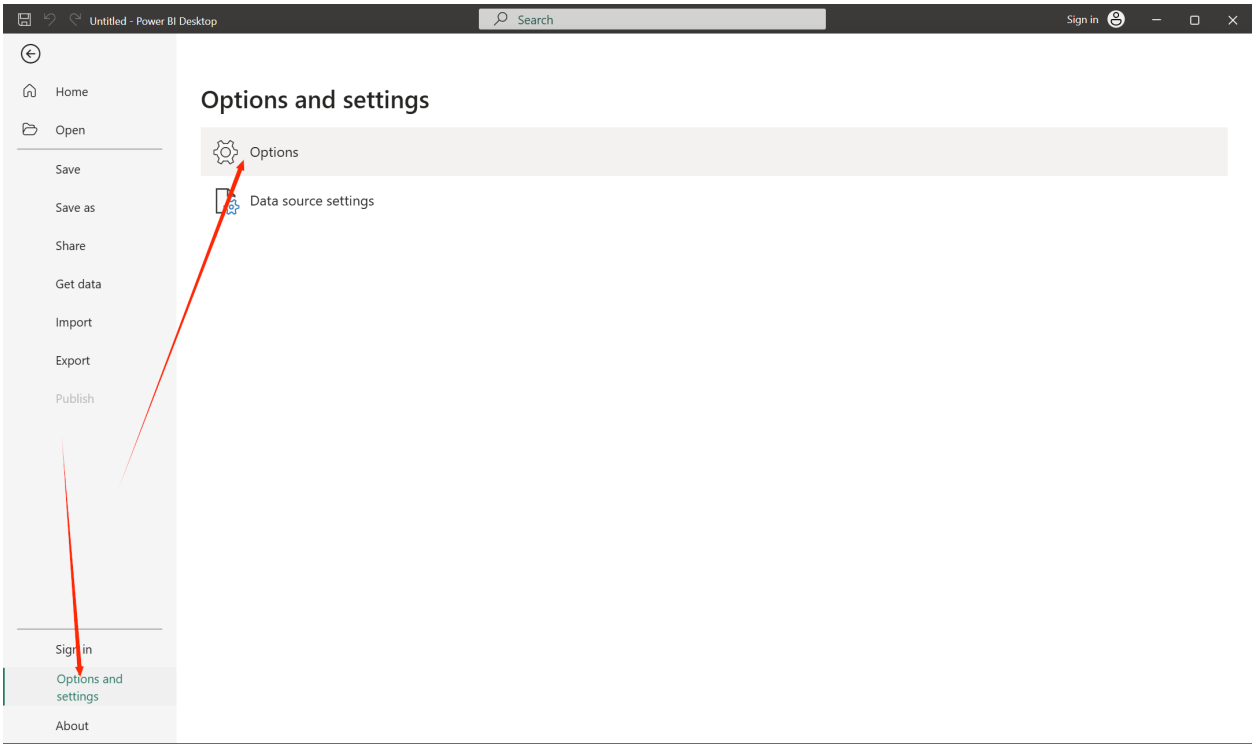


图 184:

进入 Options 界面，GLOBAL >Security，在 Data Extensions 下，

勾选 (Not Recommended)Allow any extension to load without validation or warning 选项。

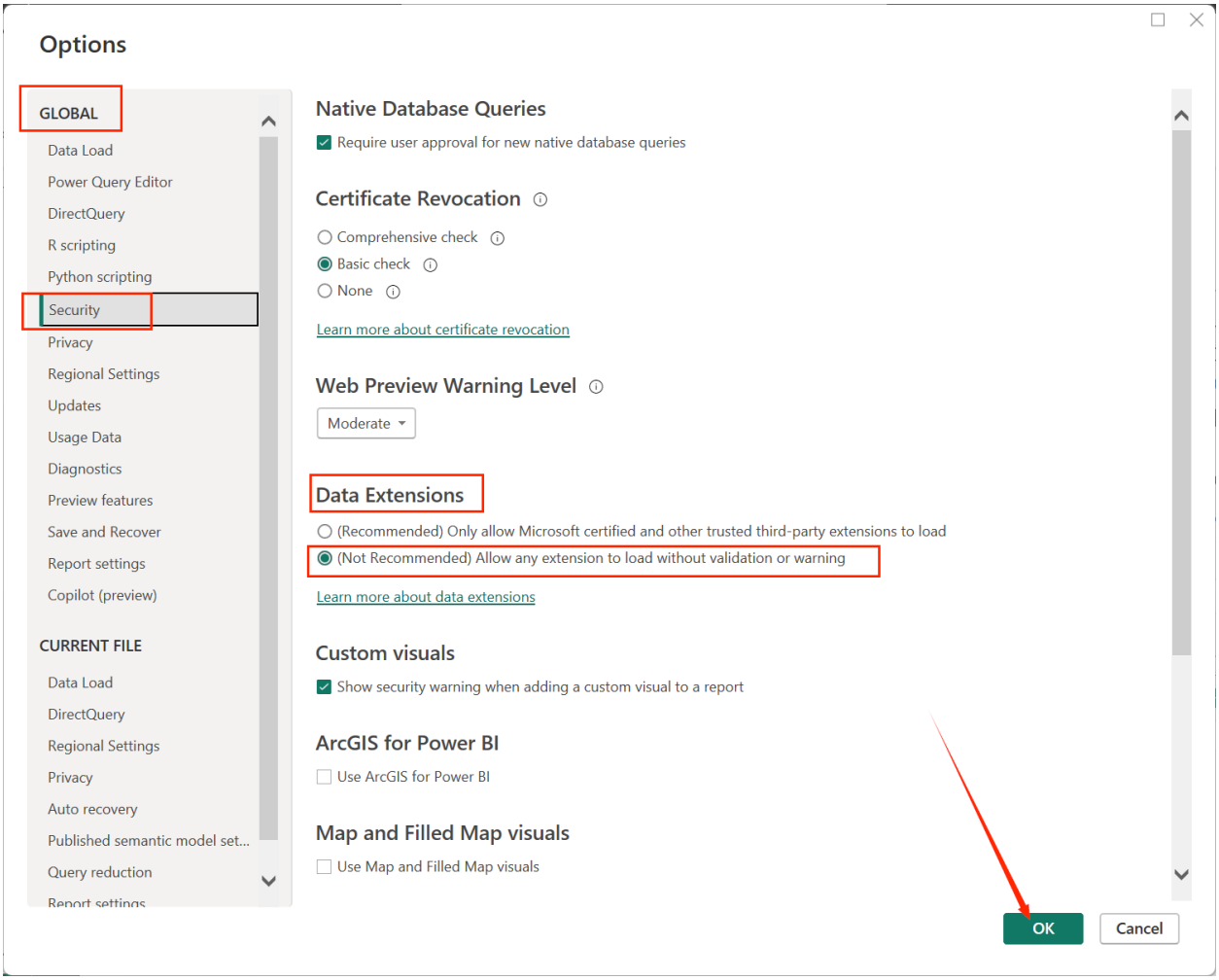


图 185:

选择 ok，然后重启 Power BI Desktop。

5.6.3.2.3 查找 Doris 连接器

1. 启动 Power BI Desktop
2. 在 Power BI Desktop 打开界面点击新建报表。若已有本地报表可以选择打开已有报表

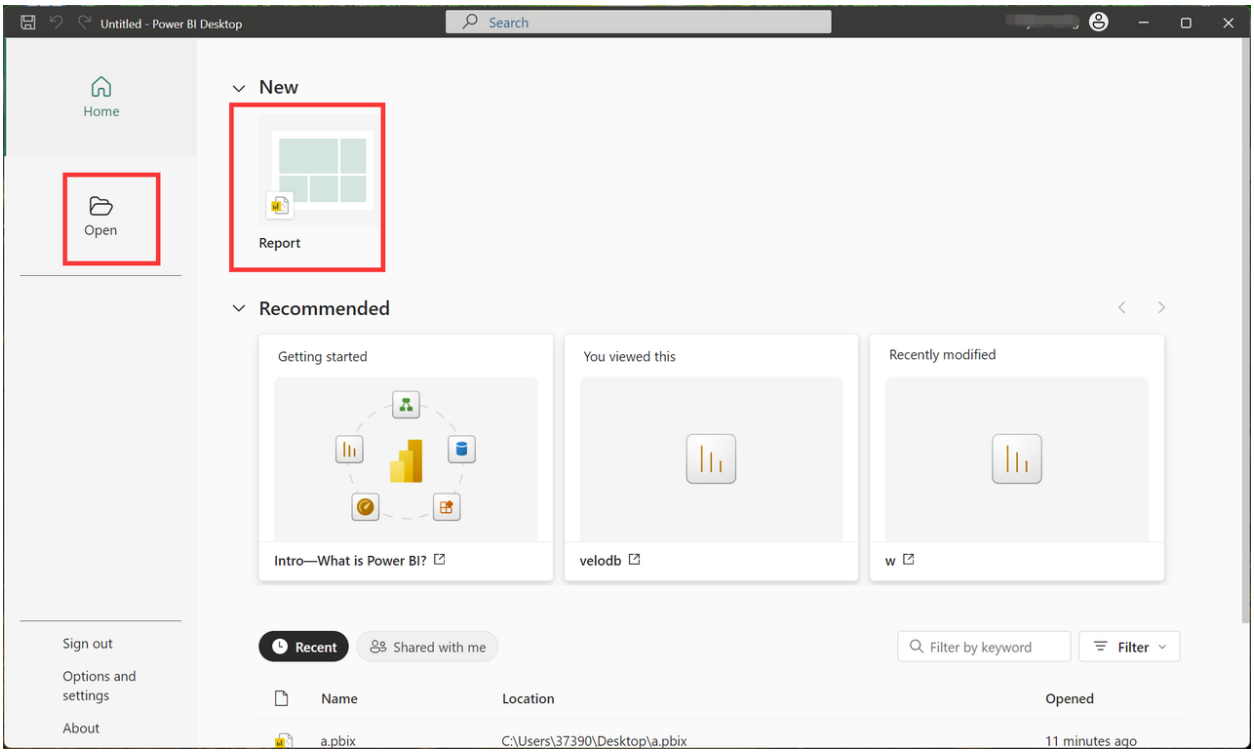


图 186:

3. 点击获取数据，在弹出窗口中选择 Doris 数据库

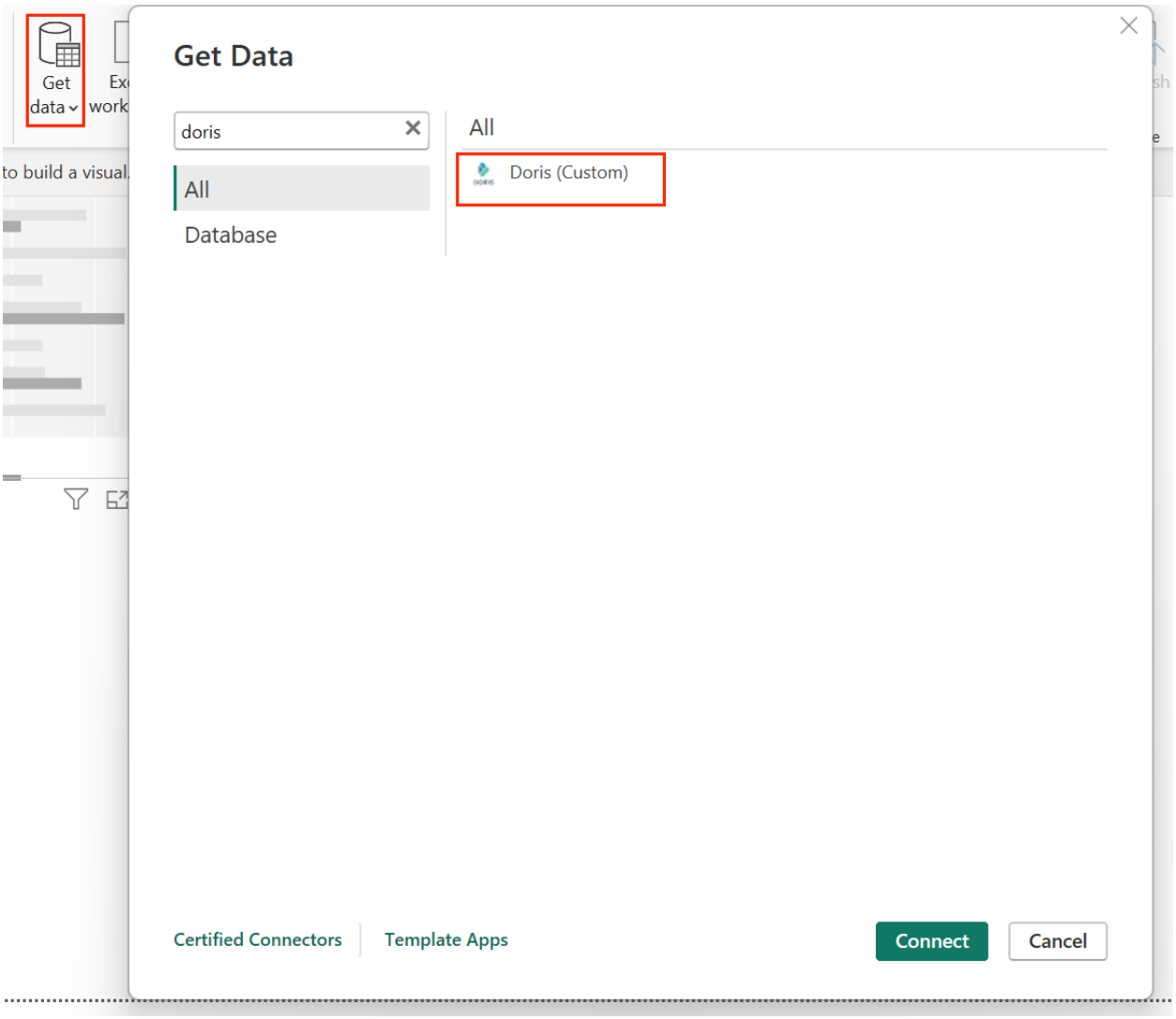


图 187:

5.6.3.2.4 连接到 Doris

选择连接器，并输入 Doris 实例凭据：

- Doris Data Source（必填）- 您的实例域名/地址或者 host:port。
- Database（必填）- 您的数据库名。
- SQL statement - 预先执行的 sql 语句（仅在 'Import' 模式下可用）
- 数据连接模式 - DirectQuery/Import

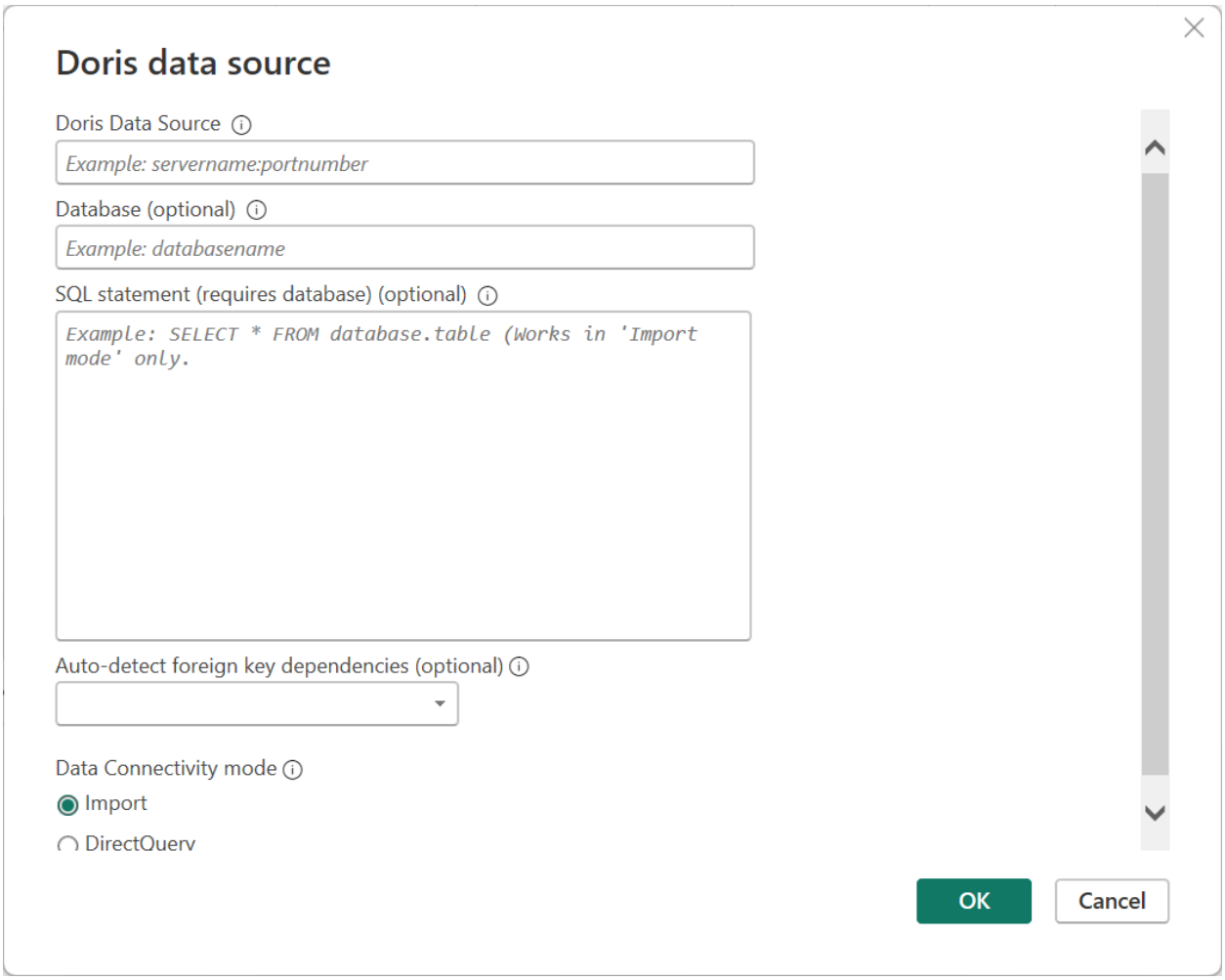


图 188:

备注

我们建议选择 DirectQuery 以直接查询 Doris。

如果您有少量数据的用例，可以选择导入模式，整个数据将加载到 Power BI。

- 指定用户名和密码

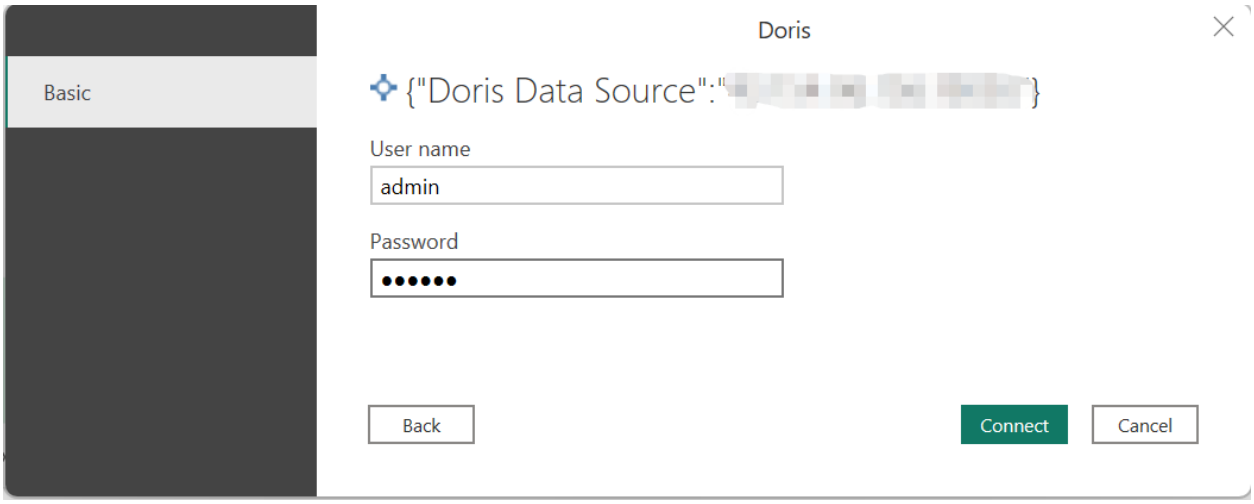


图 189:

5.6.3.2.5 查询和可视化数据

最后，您应该在导航器视图中看到数据库和表。选择所需的表并单击“加载”以从 Apache Doris 加载表结构和预览数据。

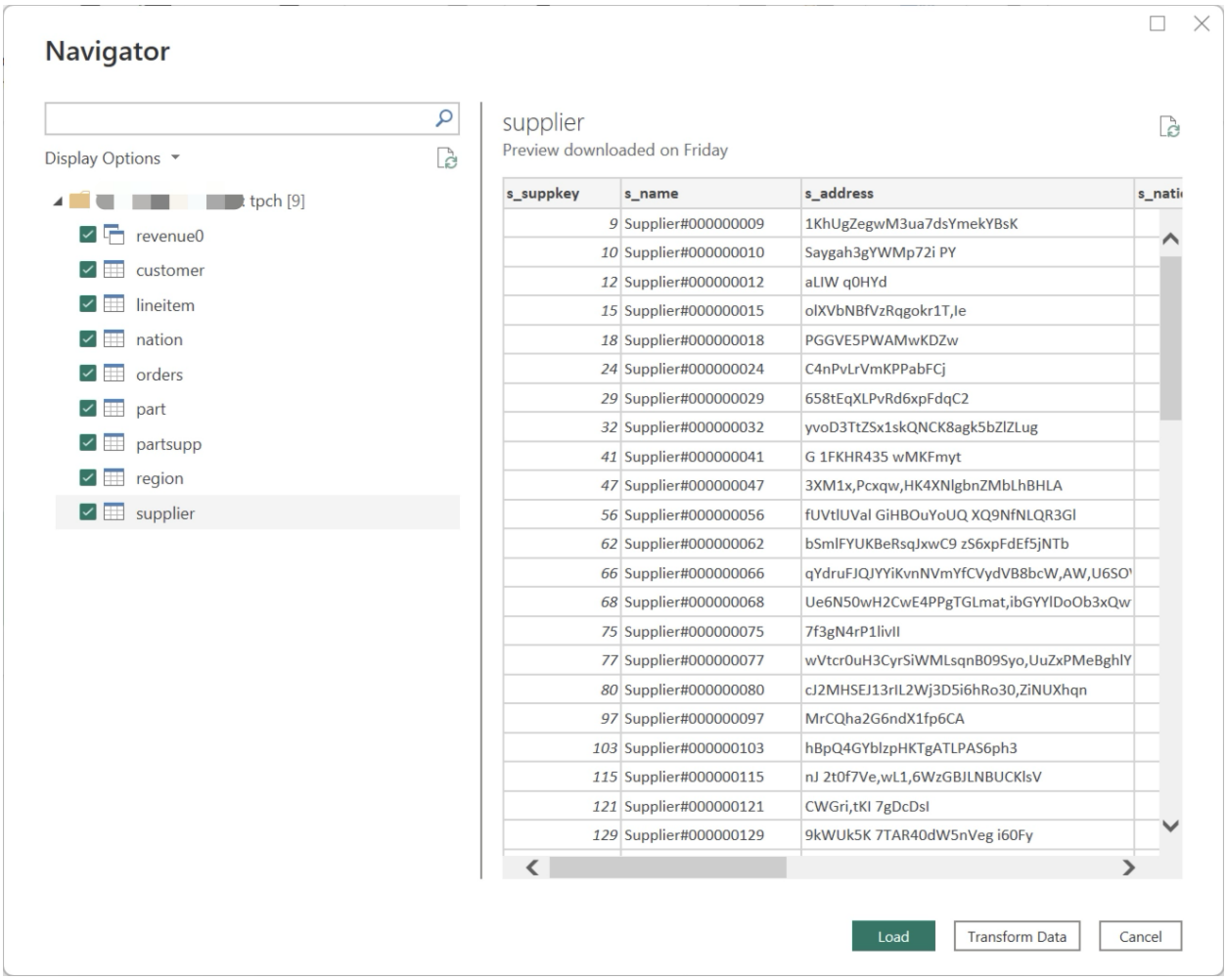


图 190:

导入完成后，您的 Doris 数据应在 Power BI 中如常访问，配置需要的统计罗盘。

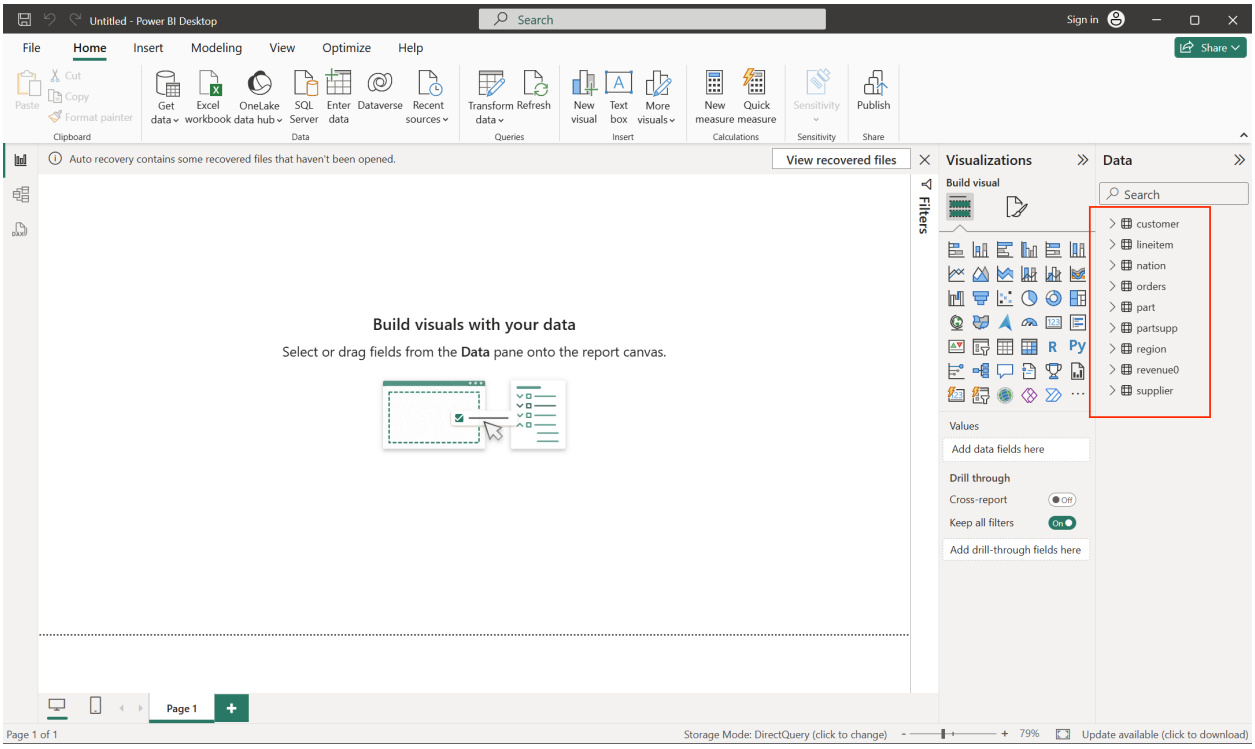


图 191:

5.6.3.3 在 Power BI 中构建可视化

我们选择 TPC-H 数据作为数据源，Doris TPC-H 数据源构建方式参考此文档 现在我们在 Power BI 中配置了 Doris 数据源，让我们可视化数据…

假设我们需要知道在各个地区的订单营收统计，接下来按照此需求进行看板构建

1. 首先进行表模型关系的创建，点击 Model view

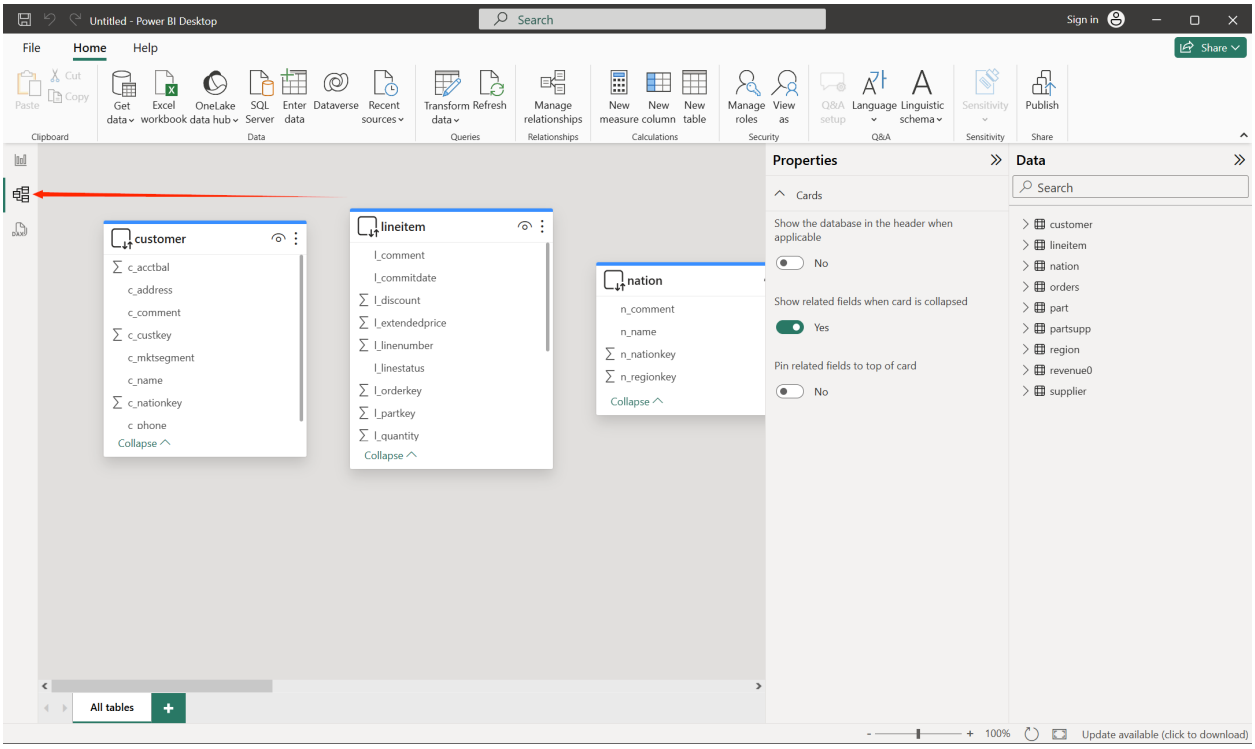


图 192:

2. 通过按需拖拽，将这四张表放置在同一屏幕下，然后进行关联字段的拖拽

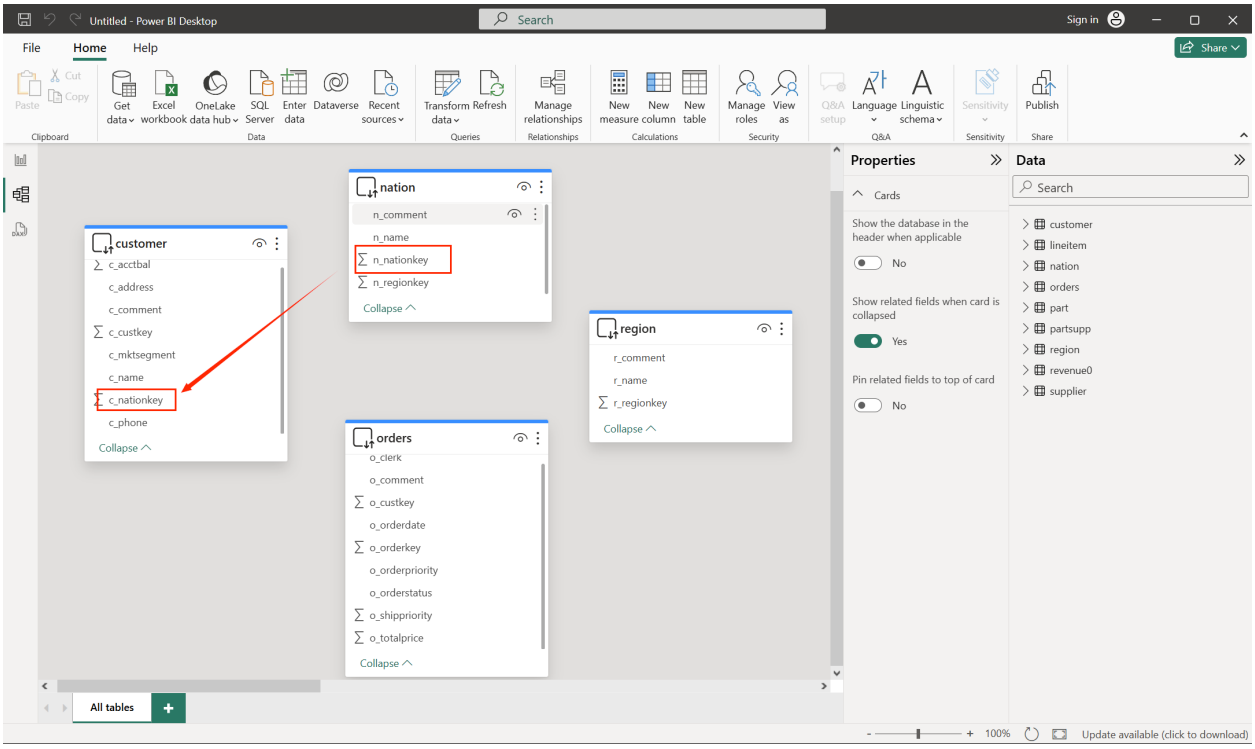

Untitled - Power BI Desktop
Search
Sign in
File
Home
Help
Share
Cut
Copy
Paste
Get data
Excel workbook
OneLake data hub
SQL Server
Enter data
Dataverse
Recent sources
Transform data
Refresh
Manage relationships
New measure
New column
New table
Manage roles
View as
Q&A setup
Language
Linguistic schema
Sensitivity
Publish
Clipboard
Data
Queries
Relationships
Calculations
Security
Q&A
Sensitivity
Share
customer
c_acctbal
c_address
c_comment
c_custkey
c_mktsegment
c_name
c_nationkey
c_phone
Collapse
nation
n_comment
n_name
n_nationkey
n_regionkey
Collapse
region
r_comment
r_name
r_regionkey
Collapse
orders
o_clerk
o_comment
o_custkey
o_orderdate
o_orderkey
o_orderpriority
o_orderstatus
o_shippriority
o_totalprice
Collapse
All tables
Properties
Cards
Show the database in the header when applicable
No
Show related fields when card is collapsed
Yes
Pin related fields to top of card
No
Data
Search
customer
lineitem
nation
orders
part
partsupp
region
revenue0
supplier
100%
Update available (click to download)

图 193:

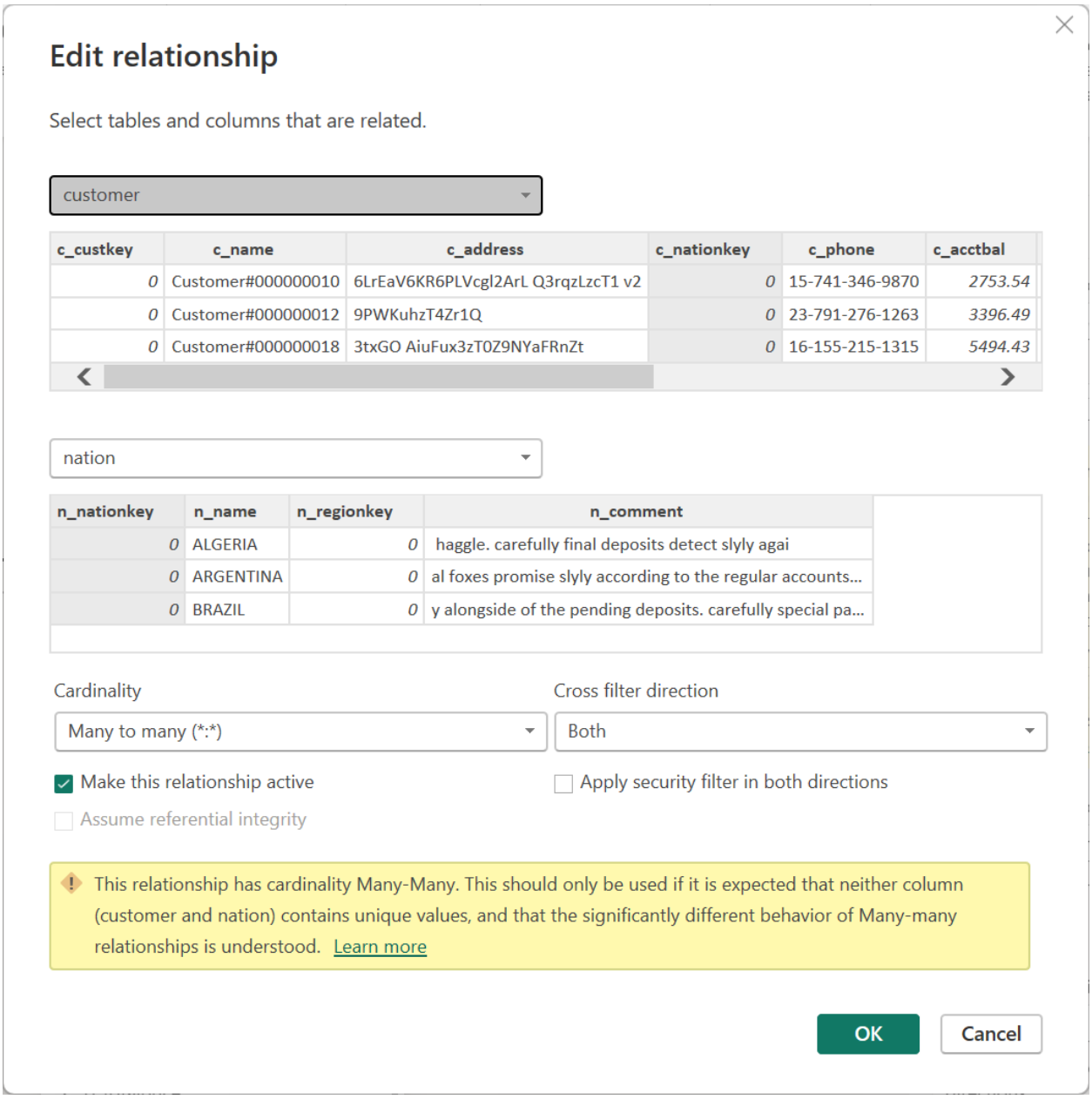


图 194:

四张表关联关系如下：

- customer： c_nationkey – nation : n_nationkey
- customer： c_custkey – orders : o_custkey
- nation : n_regionkey – region : r_regionkey

3. 关联后结果如下：

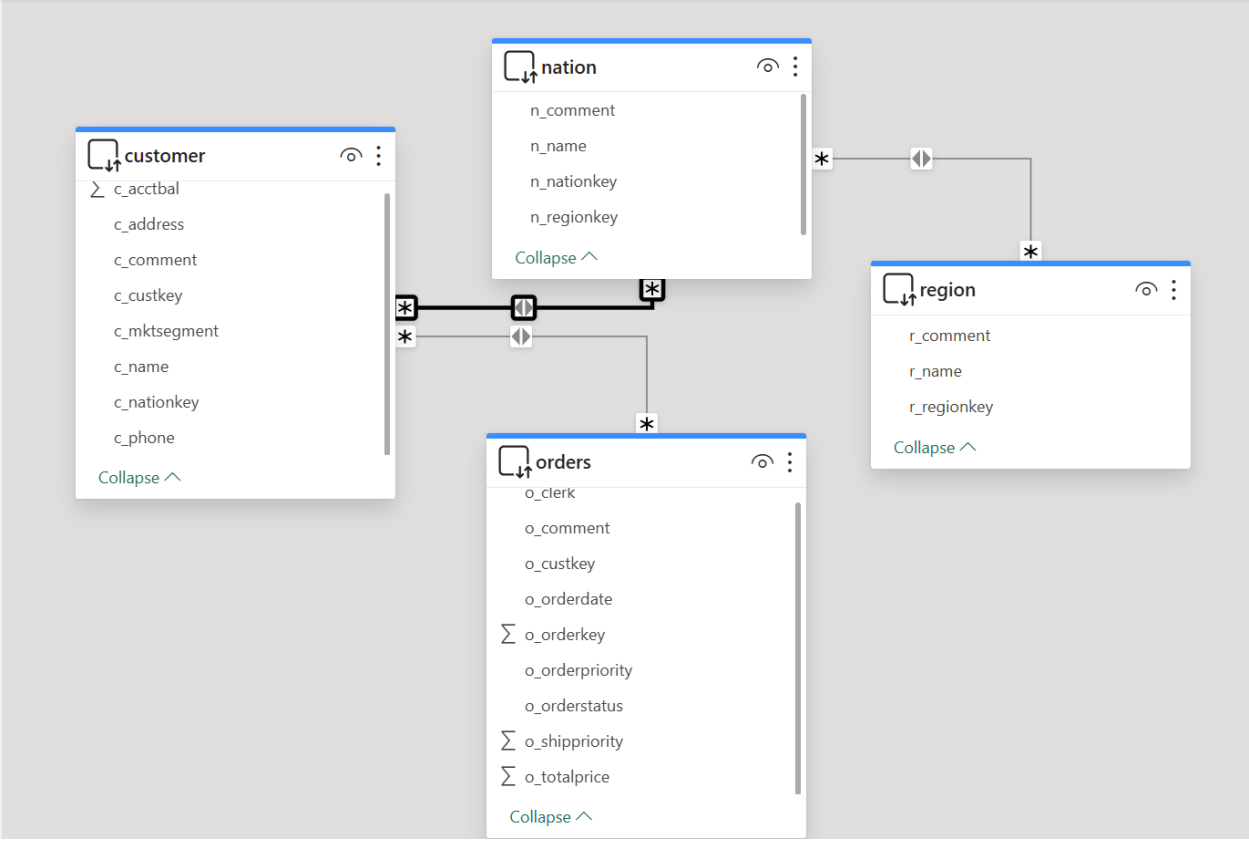


图 195:

4. 返回 Report view 工作台，进行仪表盘构建。
5. 将 orders 表中的 o_totalprice 拖拽到仪表盘

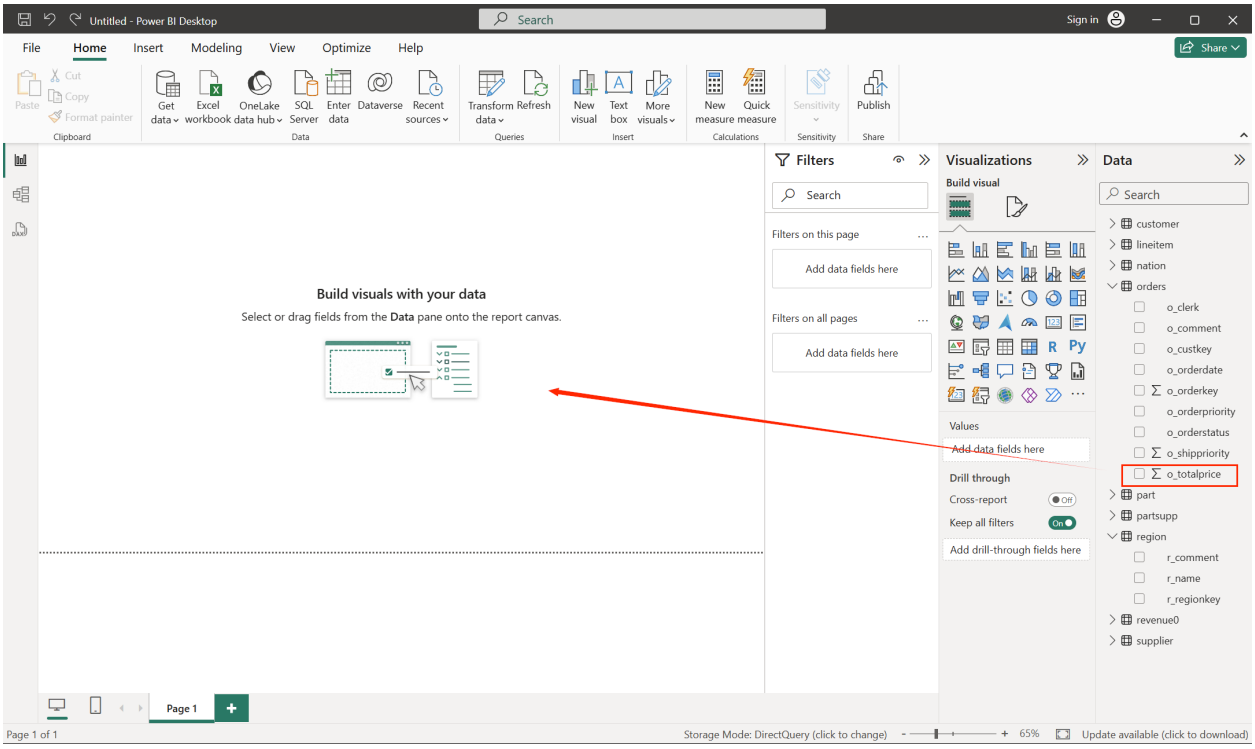


图 196:

6. 将 region 表中的 r_name 拖拽到 X 列

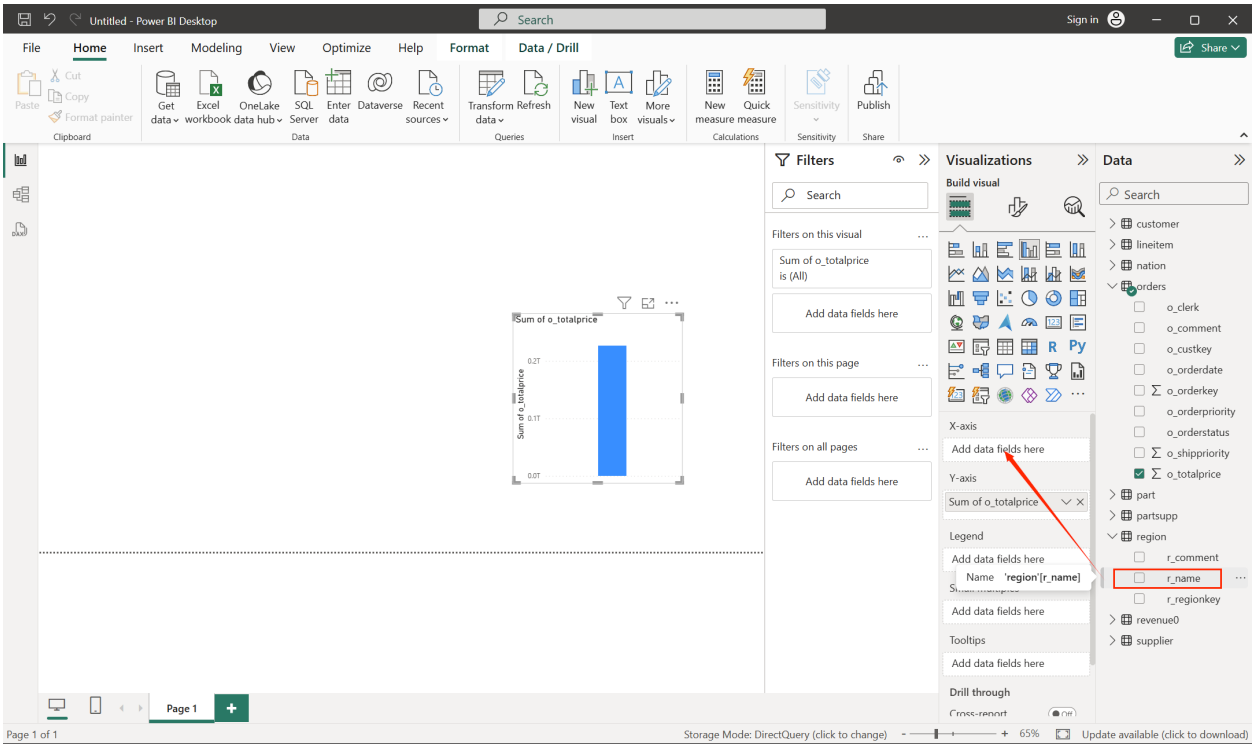


图 197:

7. 现在得到预期看板内容

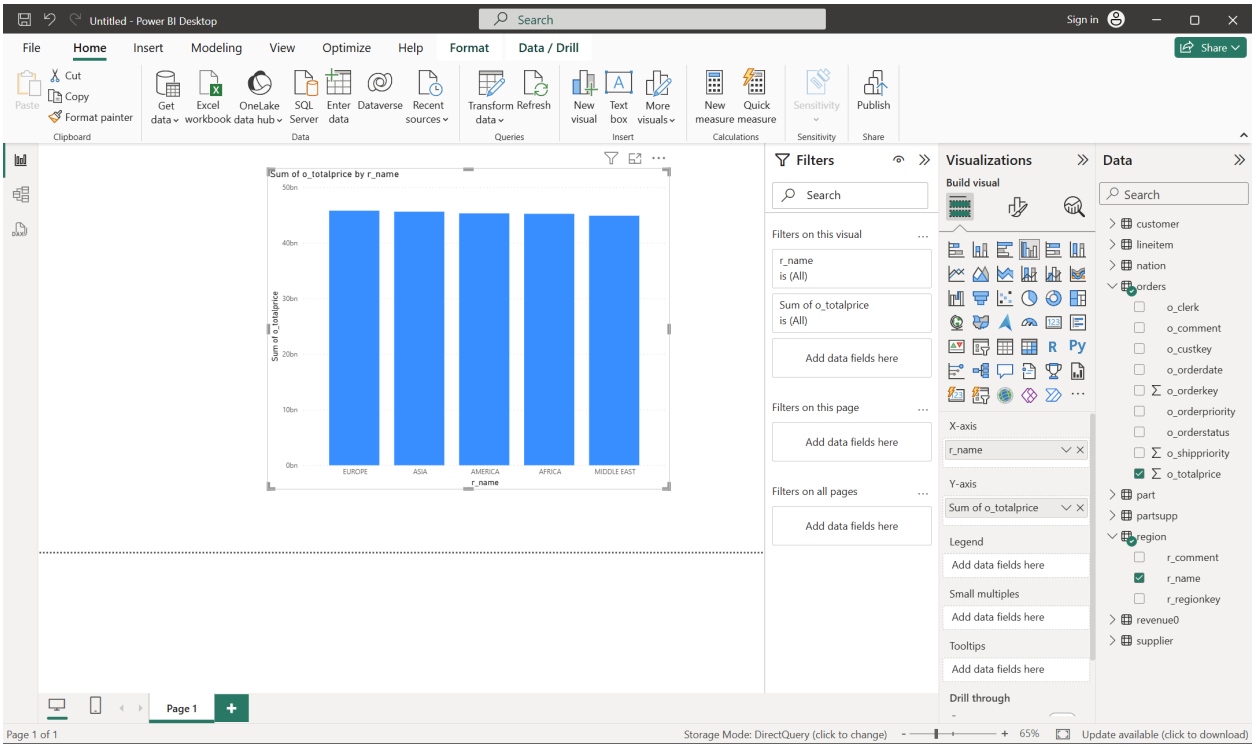


图 198:

8. 点击工作台左上角保存按钮，把创建好的统计罗盘保存至本地

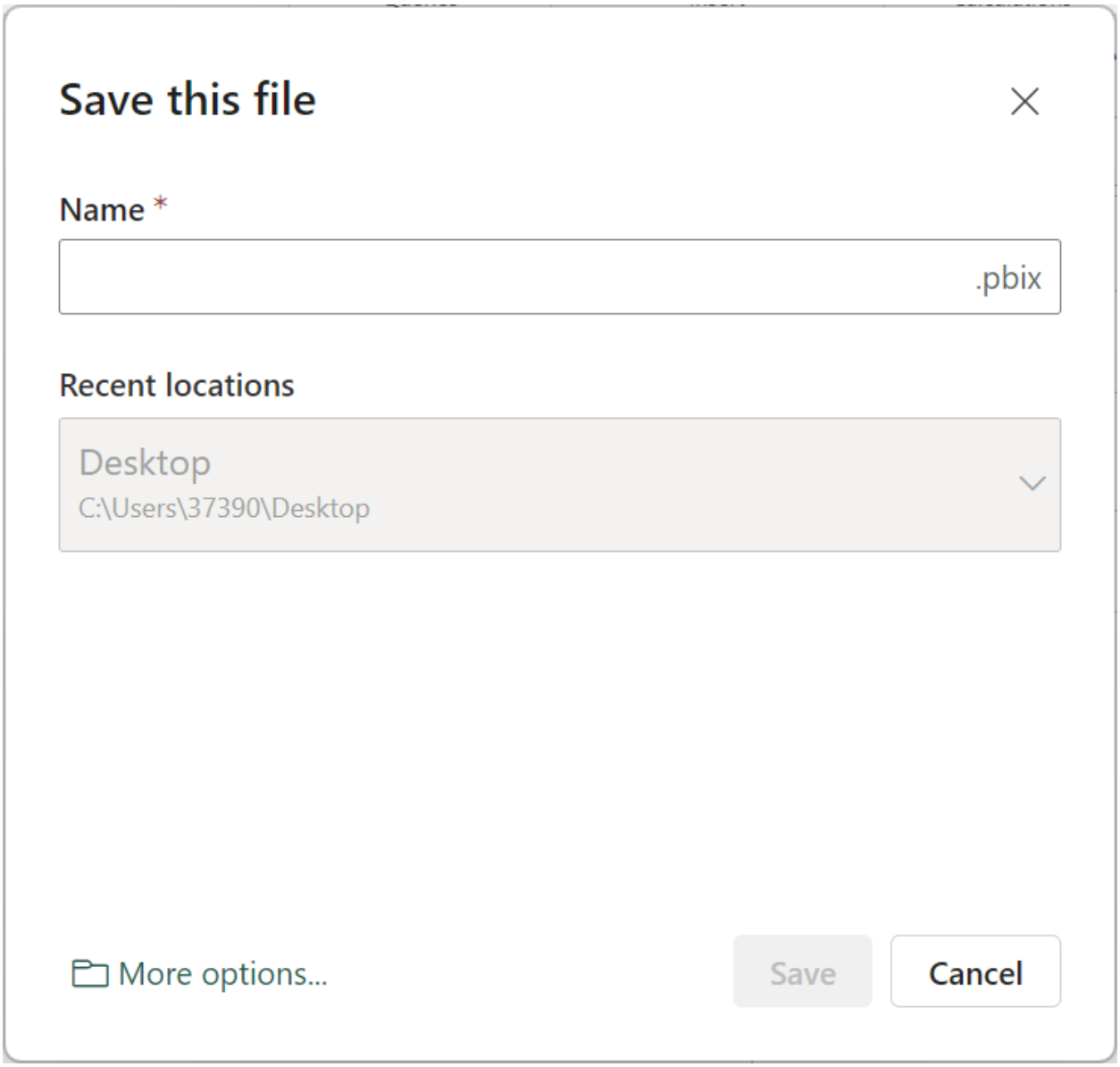


图 199:

至此，已经成功将 Power BI 连接到 Apache Doris，并实现了数据分析和可视化看板制作。

5.6.4 Tableau

对于在 Tableau 上实现 Apache Doris 访问，Tableau 官方的 MySQL 连接器可以满足需求。该连接器基于 MySQL JDBC Driver 实现访问数据。

通过 MySQL 连接器，Tableau 可以将 Apache Doris 数据库和表作为数据源进行集成。要启用此功能，请遵循下面的设置指南：

- 使用前所需的设置

- 在 Tableau 中配置 Apache Doris 数据源
- 在 Tableau 中构建可视化
- 连接和使用技巧

5.6.4.1 安装 Tableau 和 JDBC 驱动

1. 下载并安装 Tableau desktop。
2. 获取 MySQL JDBC （版本为 8.3.0)。
3. JDBC 驱动放置路径
    - MacOS：JDBC 驱动 jar 包放置路径：`~/Library/Tableau/Drivers`
    - Windows：假定 `tableau_path` 为 windows 操作系统中 tableau 的安装目录，一般默认为：`tableau_path = C:\Program Files\Tableau` 则，JDBC 驱动 jar 包放置路径：`%tableau_path%\Drivers\`

接下来，就可以在 Tableau 中配置一个 Doris 数据源并开始构建数据可视化！

5.6.4.2 在 Tableau 中配置 Doris 数据源

现在您已安装并设置了 JDBC 和 Connector 驱动程序，让我们来看一下如何在 Tableau 中定义一个连接到 Doris 中 tpch 数据库的数据源。

1. 收集您的连接详细信息

要通过 JDBC 连接到 Apache Doris，您需要以下信息：

参数	含义	示例
Server	数据库 host	127.0.1.28
Port	数据库 MySQL 端口	9030
Database	数据库名	tpch
Username	用户名	testuser
Password	密码	
Init SQL Statement	初始化 SQL 语句	`select * from database.table`

2. 启动 Tableau。(如果您已经在运行它，请重新启动。)
3. 从左侧菜单中，点击 To a Server 部分下的 More。在可用连接器列表中搜索 mysql

图 200:

4. 点击 MySql，将会弹出以下对话框：

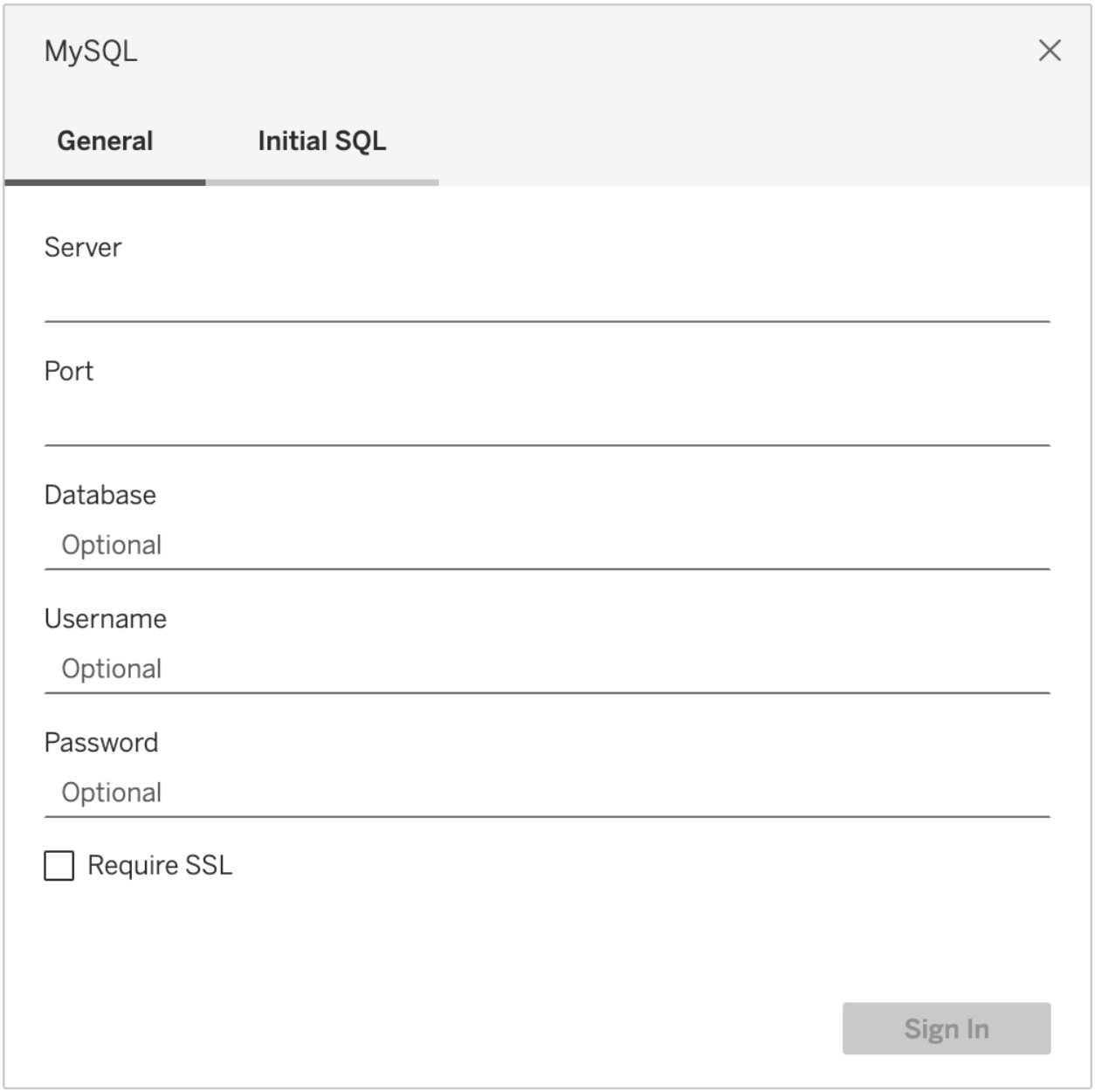
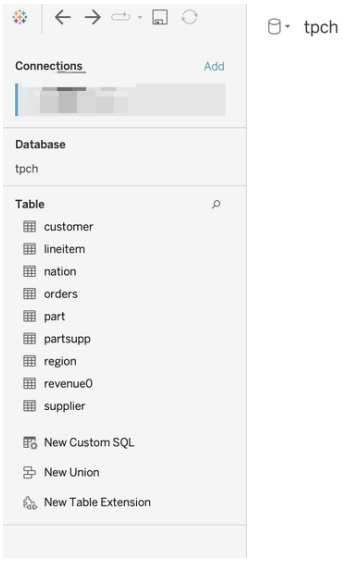


图 201:

5. 按照对话框提示输入相应的连接信息。



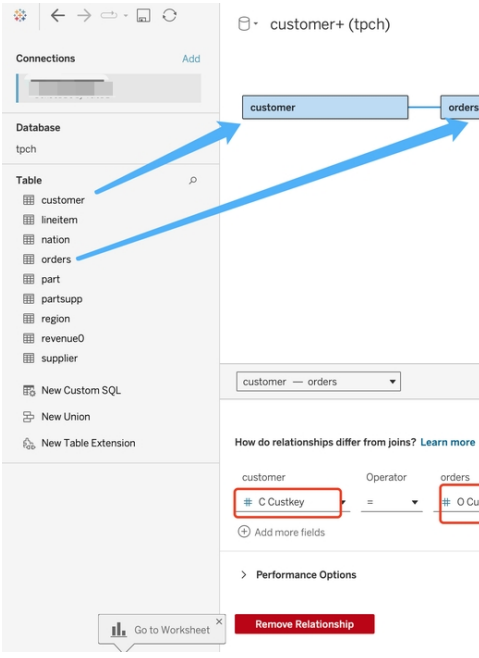
6. 在上述输入框完成后，即可点击 Sign In 按钮，您应该会看到一个新的 Tableau 工作簿：

接下来，就可以在 Tableau 中构建一些可视化了！

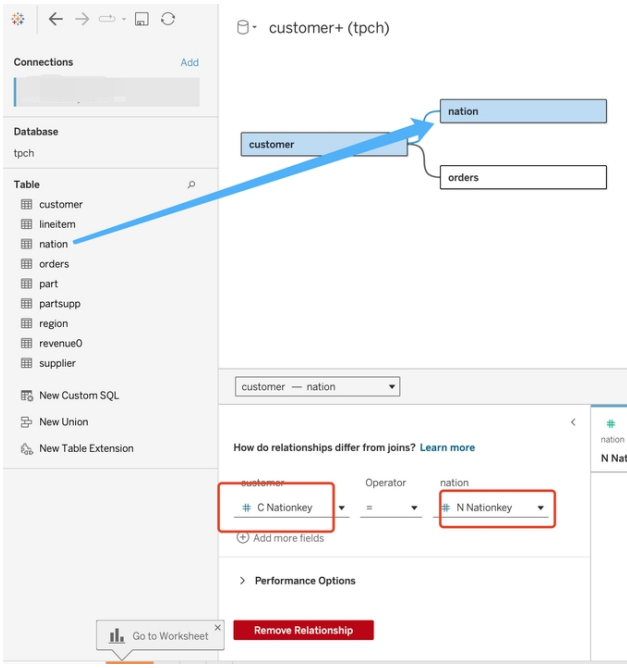
5.6.4.3 在 Tableau 中构建可视化

我们选择 TPC-H 数据作为数据源，Doris TPC-H 数据源构建方式参考此文档

现在我们在 Tableau 中配置了 Doris 数据源，让我们可视化数据

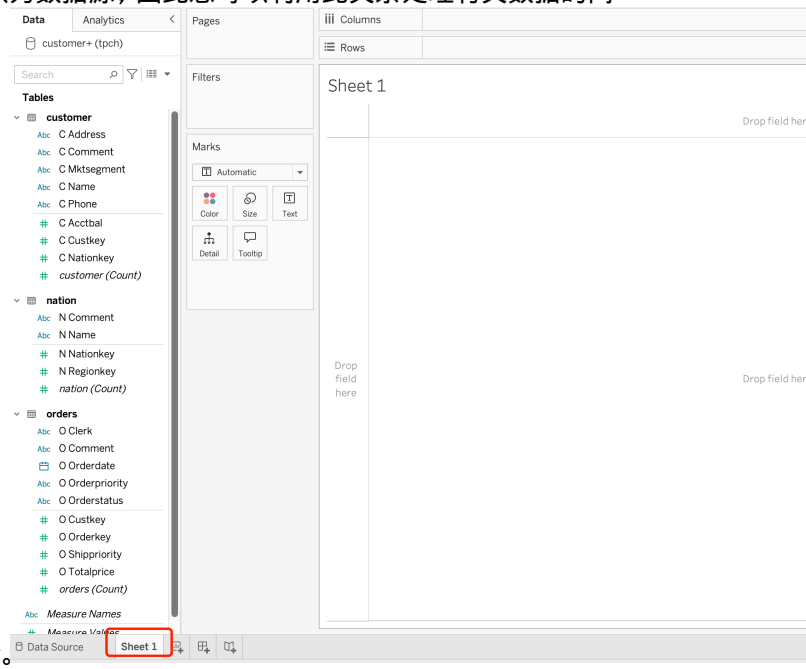


1. 将 customer 表和 orders 表拖到工作簿中。并在下方为他们选定表关联字段 Custkey



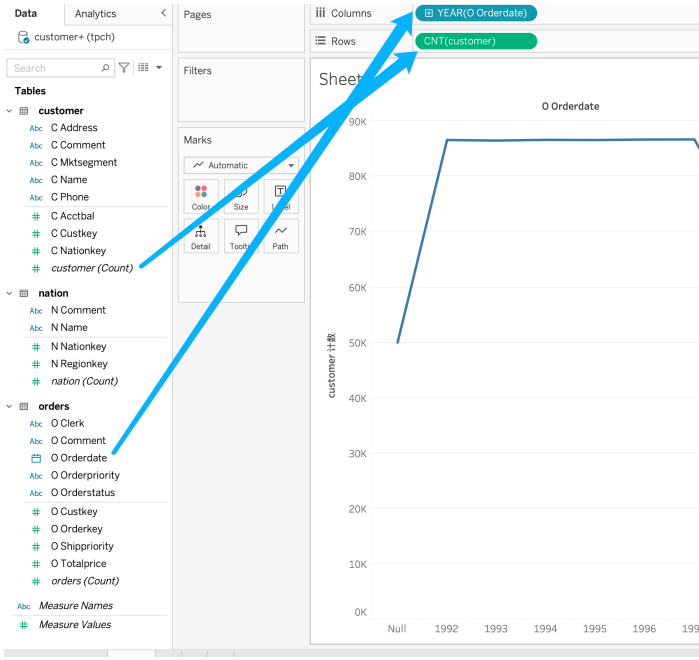
2. 将 nation 表拖到工作簿中并与 customer 表选定表关联字段 Nationkey

3. 现在您已经将 customer 表、orders 表和 nation 表关联为数据源，因此您可以利用此关系处理有关数据的问



题。选择工作簿底部的 Sheet 1 选项卡，进入工作台。

4. 假设您想知道每年的用户量汇总。将 OrderDate 从 orders 拖动到 Columns 区域（水平字段），然后将 cus-



tomer(count) 从 customer 拖到 Rows。Tableau 将生成以下折线图：

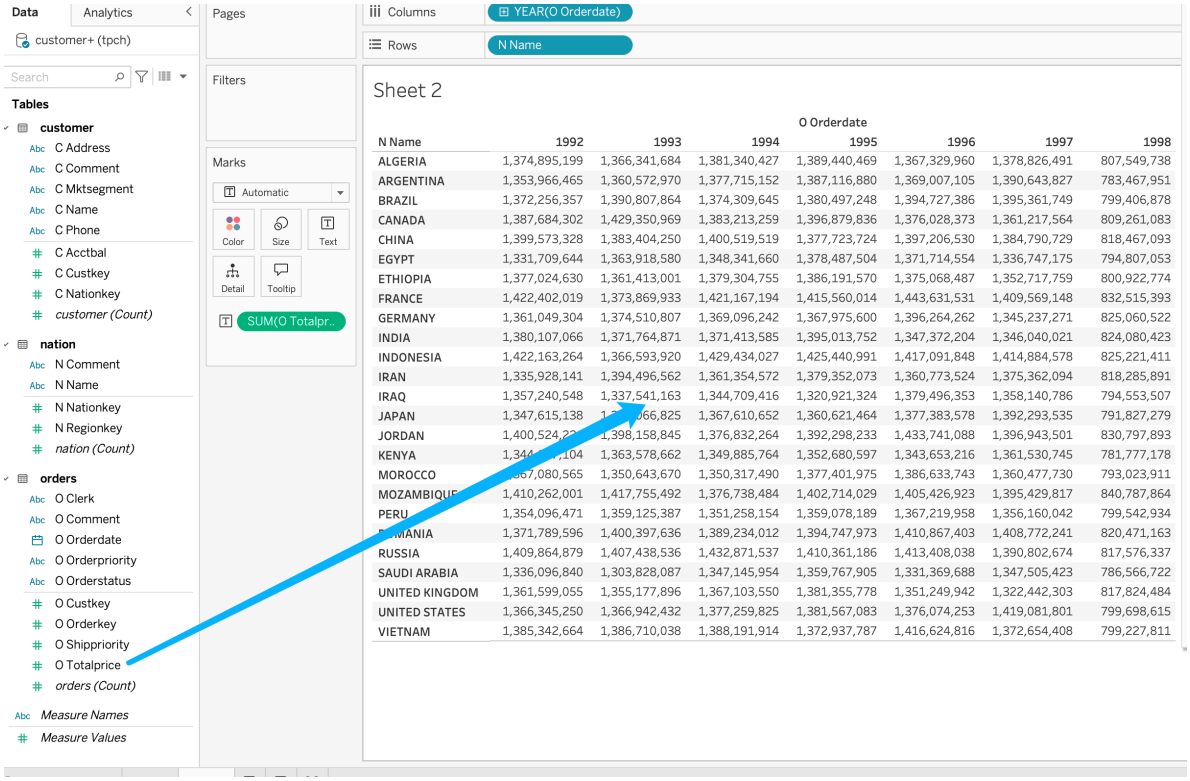
一张简单的折线图就制作完成了，但该数据集是通过 tpch 脚本和默认规则自动生成的非实际数据，不具备参考性，旨在测试可用与否。

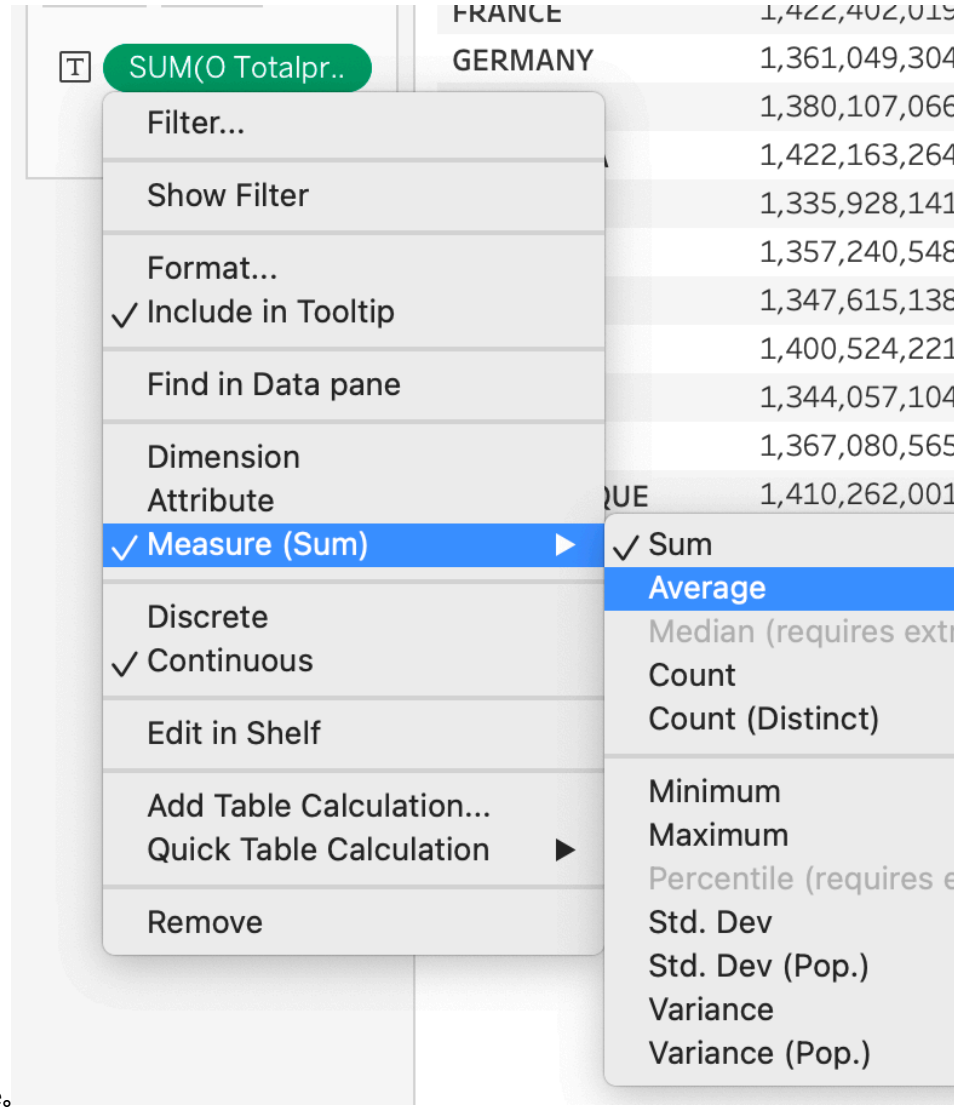
5. 假设您想知道按地域（国别）和年份计算的平均订单金额（美元）：

   - 点击 New Worksheet 选项卡创建新表
   - 将 Name 从 nation 表拖入 Rows
   - 将 OrderDate 从 orders 表拖入 Columns

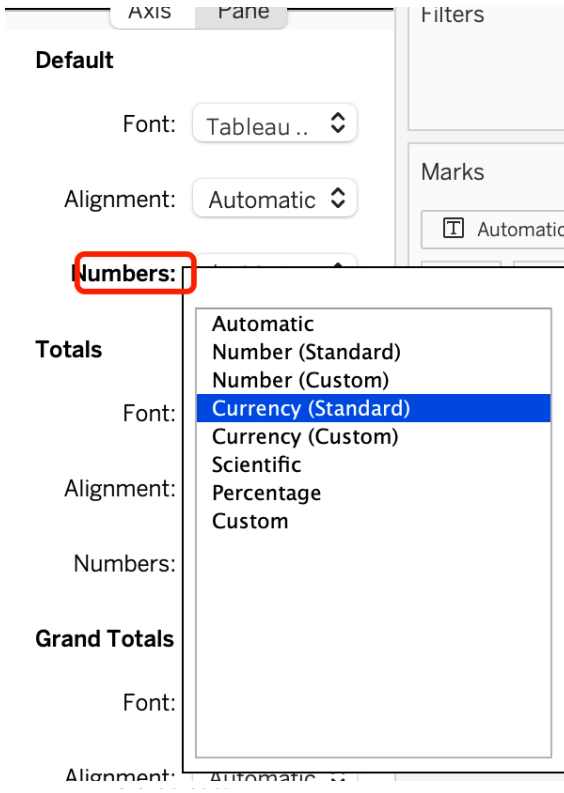
您应该会看到以下内容：

6. 注意：Abc 值只是填充值，因为您未将聚合逻辑定义到该图标，因此需要您拖动度量到表格上。将 Totalprice 从 orders 表拖到表格中间。请注意默认的计算是对 Totalprices 进行 SUM：


Data
Analytics
customer+ (tpch)
Search
Tables
customer
C Address
C Comment
C Mktsegment
C Name
C Phone
C Acctbal
C Custkey
C Nationkey
customer (Count)
nation
N Comment
N Name
N Nationkey
N Regionkey
nation (Count)
orders
O Clerk
O Comment
O Orderdate
O Orderpriority
O Orderstatus
O Custkey
O Orderkey
O Shippriority
O Totalprice
orders (Count)
Measure Names
Measure Values
Pages
Filters
Marks
Automatic
Color
Size
Text
Detail
Tooltip
SUM(O Totalpr..
Columns
YEAR(O Orderdate)
Rows
N Name
Sheet 2
O Orderdate
N Name 1992 1993 1994 1995 1996 1997 1998
ALGERIA 1,374,895,199 1,366,341,684 1,381,340,427 1,389,440,469 1,367,329,960 1,378,826,491 807,549,738
ARGENTINA 1,353,966,465 1,360,572,970 1,377,715,152 1,387,116,880 1,369,007,105 1,390,643,827 783,467,951
BRAZIL 1,372,256,357 1,390,807,864 1,374,309,645 1,380,497,248 1,394,727,386 1,395,361,749 799,406,878
CANADA 1,387,684,302 1,429,350,969 1,383,213,259 1,396,879,836 1,376,028,373 1,361,217,564 809,261,083
CHINA 1,399,573,328 1,383,404,250 1,400,519,519 1,377,723,724 1,397,206,530 1,384,790,729 818,467,093
EGYPT 1,331,709,644 1,363,918,580 1,348,341,660 1,378,487,504 1,371,714,554 1,336,747,175 794,807,053
ETHIOPIA 1,377,024,630 1,361,413,001 1,379,304,755 1,386,191,570 1,375,068,487 1,352,717,759 800,922,774
FRANCE 1,422,402,019 1,373,869,933 1,421,167,194 1,415,560,014 1,443,631,531 1,409,569,148 832,515,393
GERMANY 1,361,049,304 1,374,510,807 1,369,096,242 1,367,975,600 1,396,264,262 1,345,237,271 825,060,522
INDIA 1,380,107,066 1,371,764,871 1,371,413,585 1,395,013,752 1,347,372,204 1,346,040,021 824,080,423
INDONESIA 1,422,163,264 1,366,593,920 1,429,434,027 1,425,440,991 1,417,091,848 1,414,884,578 825,221,411
IRAN 1,335,928,141 1,394,496,562 1,361,354,572 1,379,352,073 1,360,773,524 1,375,362,094 818,285,891
IRAQ 1,357,240,548 1,337,541,163 1,344,709,416 1,320,921,324 1,379,496,353 1,358,140,786 794,553,507
JAPAN 1,347,615,138 1,366,825 1,367,610,652 1,360,621,464 1,377,383,578 1,392,293,535 791,827,279
JORDAN 1,400,524,2 1,398,158,845 1,376,832,264 1,392,298,233 1,433,741,088 1,396,943,501 830,797,893
KENYA 1,34 104 1,363,578,662 1,349,885,764 1,352,680,597 1,343,653,216 1,361,530,745 781,777,178
MOROCCO 080,565 1,350,643,670 1,350,317,490 1,377,401,975 1,386,633,743 1,360,477,730 793,023,911
MOZAMBIQUE 1,410,262,001 1,417,755,492 1,376,738,484 1,402,714,029 1,405,426,923 1,395,429,817 840,787,864
PERU 1,354,096,471 1,359,125,387 1,351,258,154 1,359,078,189 1,367,219,958 1,356,160,042 799,542,934
MANIA 1,371,789,596 1,400,397,636 1,389,234,012 1,394,747,973 1,410,867,403 1,408,772,241 820,471,163
RUSSIA 1,409,864,879 1,407,438,536 1,432,871,537 1,410,361,186 1,413,408,038 1,390,802,674 817,576,337
SAUDI ARABIA 1,336,096,840 1,303,828,087 1,347,145,954 1,359,767,905 1,331,369,688 1,347,505,423 786,566,722
UNITED KINGDOM 1,361,599,055 1,355,177,896 1,367,103,550 1,381,355,778 1,351,249,942 1,322,442,303 817,824,484
UNITED STATES 1,366,345,250 1,366,942,432 1,377,259,825 1,381,567,083 1,376,074,253 1,419,081,801 799,698,615
VIETNAM 1,385,342,664 1,386,710,038 1,388,191,914 1,372,937,787 1,416,624,816 1,372,654,408 799,227,811



7. 点击 SUM 并将 Measure 更改为 Average。



8. 从同一下拉菜单中选择 `Format` 将 `Numbers` 更改为 `Currency (Standard)`：

9. 得到一张符合预期的表格：

N Name	1992	1993	1994	1995	1996	1997	
ALGERIA	$151,587.12	$152,018.43	$152,146.76	$151,619.43	$152,043.81	$152,137.98	$153,
ARGENTINA	$151,112.33	$152,393.93	$151,247.68	$151,996.15	$151,288.22	$152,432.73	$149,
BRAZIL	$151,680.82	$153,189.54	$150,956.68	$150,973.01	$150,374.92	$151,983.63	$150,
CANADA	$152,811.84	$152,904.47	$150,365.61	$151,063.03	$149,861.51	$150,427.40	$150,
CHINA	$150,653.75	$149,621.92	$150,723.15	$149,997.14	$151,837.27	$150,618.96	$152,
EGYPT	$150,697.03	$151,681.34	$151,057.77	$151,782.37	$151,336.56	$149,708.50	$150,
ETHIOPIA	$151,487.86	$151,066.69	$151,989.50	$150,836.95	$152,328.40	$150,519.39	$150,
FRANCE	$152,144.83	$151,007.91	$150,213.21	$152,473.07	$150,425.29	$151,664.42	$150,
GERMANY	$150,558.55	$151,762.26	$151,936.11	$150,990.68	$152,347.44	$150,490.80	$151,
INDIA	$150,568.08	$152,384.46	$150,324.85	$152,945.26	$150,493.94	$148,898.23	$151,
INDONESIA	$150,461.62	$150,307.29	$152,586.89	$153,289.71	$150,947.15	$151,583.95	$151,
IRAN	$151,123.09	$151,856.32	$150,028.06	$150,633.62	$151,956.84	$151,371.57	$152,
IRAQ	$150,203.69	$150,640.97	$150,347.65	$149,917.30	$152,649.81	$152,121.50	$150,
JAPAN	$151,758.46	$151,144.32	$151,990.51	$150,361.53	$150,813.92	$152,563.39	$150,
JORDAN	$152,545.93	$152,056.43	$150,539.28	$152,030.82	$153,538.35	$151,331.76	$151,
KENYA	$151,750.83	$151,206.33	$150,371.59	$150,465.03	$150,616.88	$151,685.69	$150,
MOROCCO	$152,320.95	$150,691.03	$151,279.13	$150,618.04	$152,427.59	$149,832.35	$150,
MOZAMBIQUE	$151,218.31	$152,463.22	$151,373.12	$150,296.16	$150,894.02	$149,339.66	$151,
PERU	$150,992.02	$152,385.40	$152,117.32	$151,581.33	$151,744.72	$150,533.92	$151,
ROMANIA	$149,481.27	$150,937.45	$151,052.95	$150,247.55	$150,508.58	$151,366.95	$152,
RUSSIA	$151,500.63	$151,810.87	$152,644.25	$150,182.22	$150,234.70	$149,919.44	$149,
SAUDI ARABIA	$151,039.66	$150,418.56	$151,042.26	$150,267.20	$150,658.56	$151,031.77	$150,
UNITED KINGDOM	$150,154.28	$150,659.02	$151,278.47	$150,490.88	$150,573.87	$149,259.85	$153,
UNITED STATES	$150,511.70	$151,495.34	$152,351.75	$151,040.46	$152,338.56	$152,360.08	$151,
VIETNAM	$151,089.83	$151,718.82	$150,448.89	$151,254.58	$152,325.25	$149,363.92	$152,

至此，已经成功将 Tableau 连接到 Apache Doris，并实现了数据分析和可视化看板制作。

5.6.4.4 连接和使用技巧

性能优化

- 根据实际需求，合理创建 doris 库表，按时间分区分桶，可有效减少谓词过滤和大部分数据传输
- 适当的数据预聚合，可以通过 Doris 侧创建物化视图的方式。
- 设置合理的刷新计划，均衡刷新的计算资源消耗和看板数据时效性

安全配置

- 建议使用 VPC 私有连接，避免公网访问引入安全风险。
- 配置安全组限制访问。
- 启用 SSL/TLS 连接等访问方式。
- 细化 Doris 用户账号角色和访问权限，避免过度下放权限。

5.6.5 QuickSight

QuickSight 可以通过官方 MySQL 数据源以 Directly query 或 Import 模式连接到 Apache Doris

5.6.5.1 前提条件

- Apache Doris 版本要求不低于 3.1.2
- 网络连通性（VPC、安全组配置），需要结合 Doris 部署环境进行配置，以保证 AWS 服务器能访问到你的 Doris 集群。
- 在连接到 Doris 的 MySQL client 上运行如下 SQL 来调整声明 MySQL 的兼容版本：

```
SET GLOBAL version = '8.3.99';
```

校验结果：

```
mysql> show variables like "version";
+---------------+--------+---------------+---------+
| Variable_name | Value  | Default_Value | Changed |
+---------------+--------+---------------+---------+
| version       | 8.3.99 | 5.7.99        | 1       |
+---------------+--------+---------------+---------+
1 row in set (0.01 sec)
```

5.6.5.2 将 QuickSight 连接到 Apache Doris

首先，访问 https://quicksight.aws.amazon.com，导航到数据集并点击“新建数据集”：

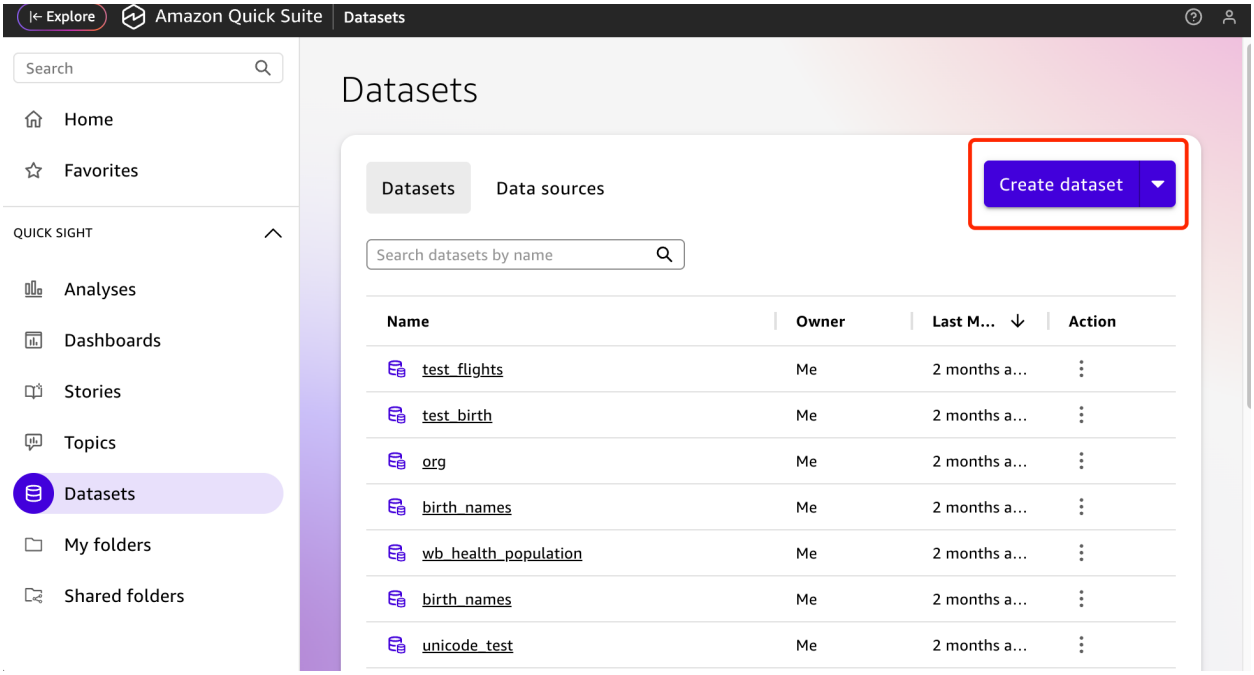


图 202:

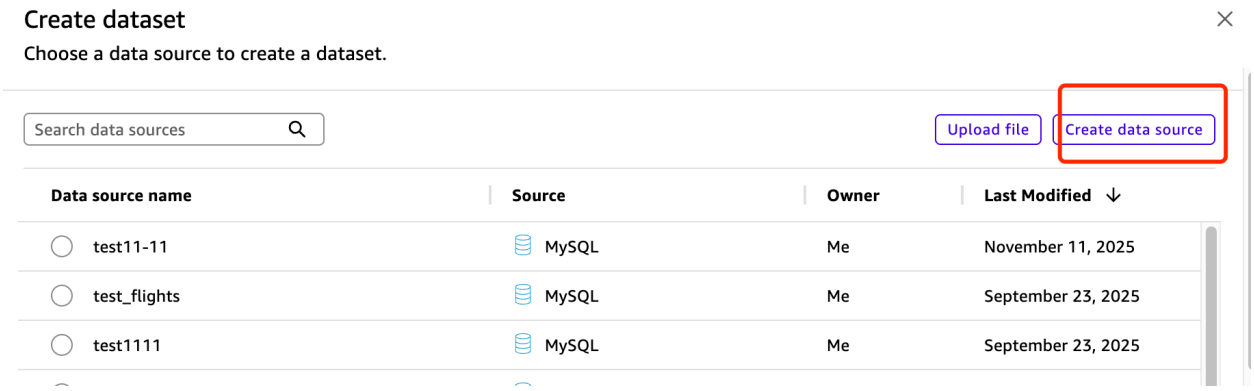


图 203:

搜索 QuickSight 捆绑的官方 MySQL 连接器：

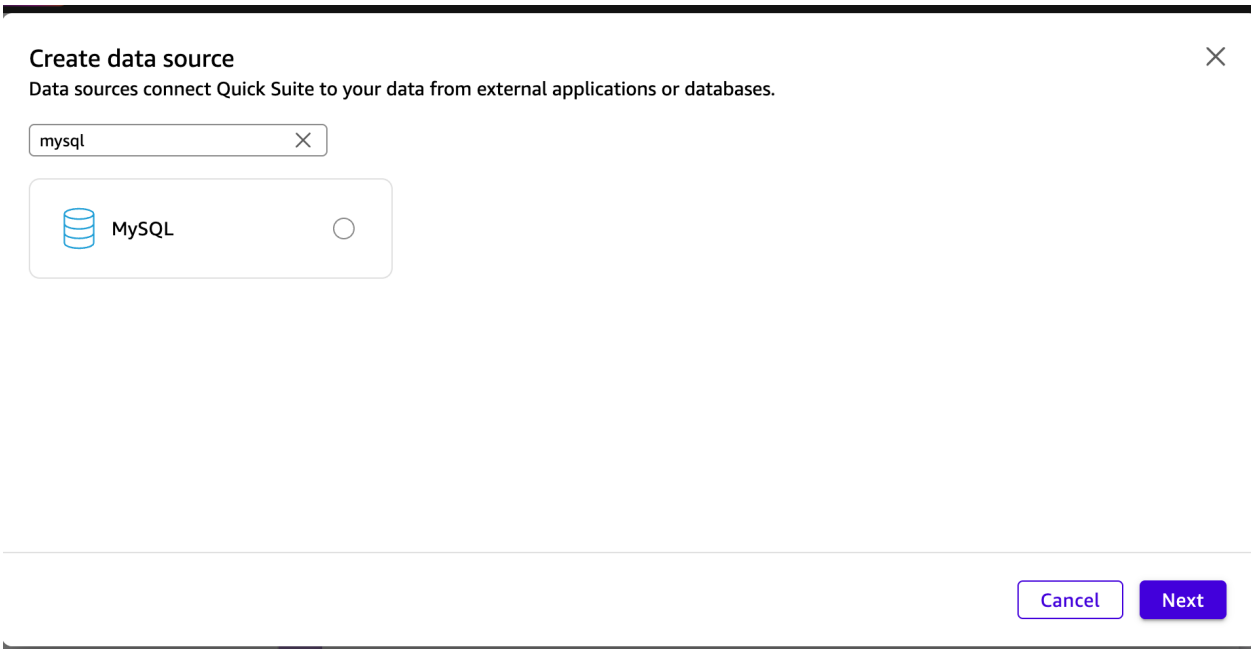


图 204:

指定您的连接详细信息。请注意，MySQL 接口端口默认为 9030，具体取决于您的 Fe `query_port` 配置可能会有所不同。

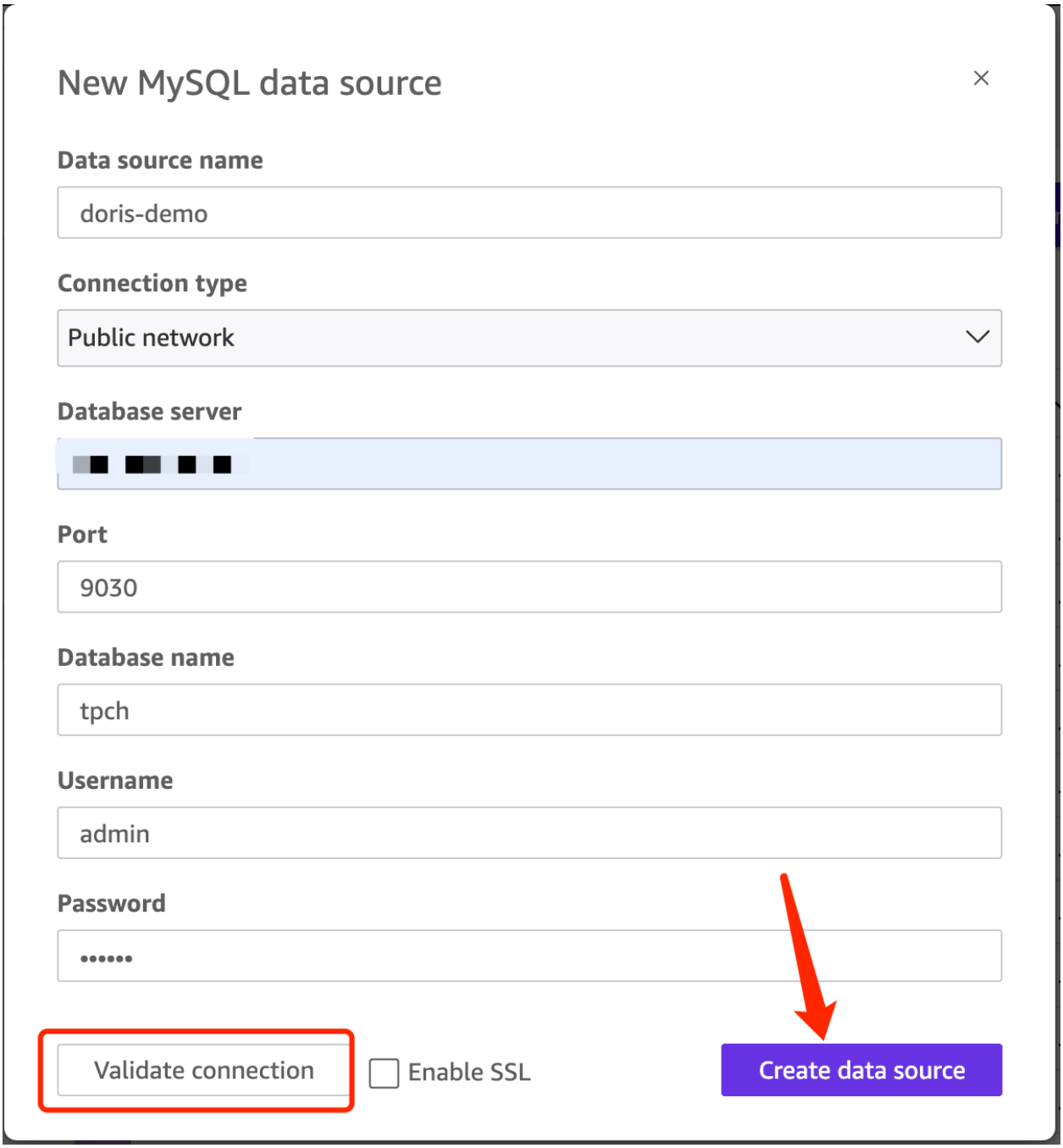


图 205:

现在，您可以从列表中选择一个表：

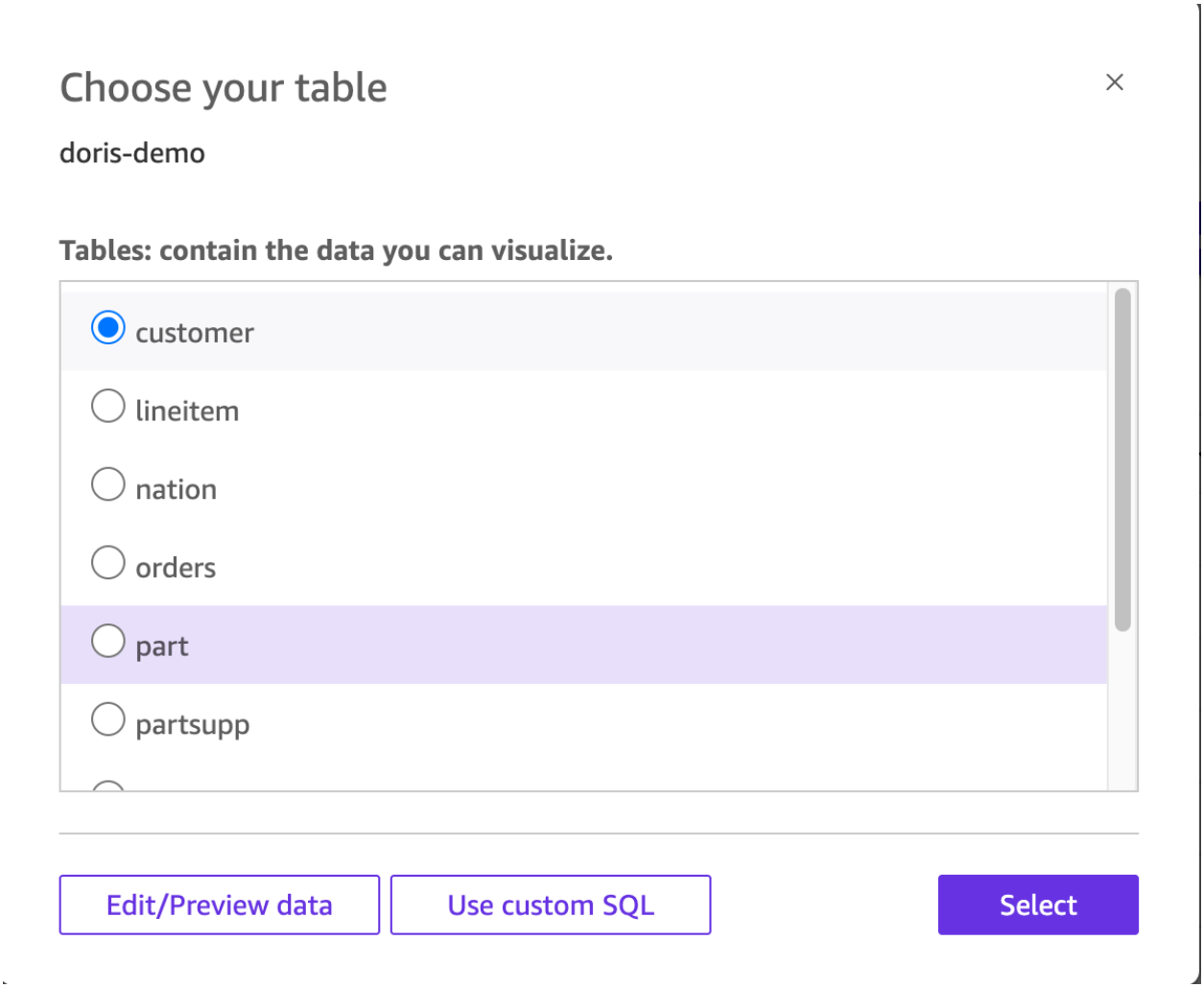


图 206:

这里推荐您选择 “Directly query” 模式：

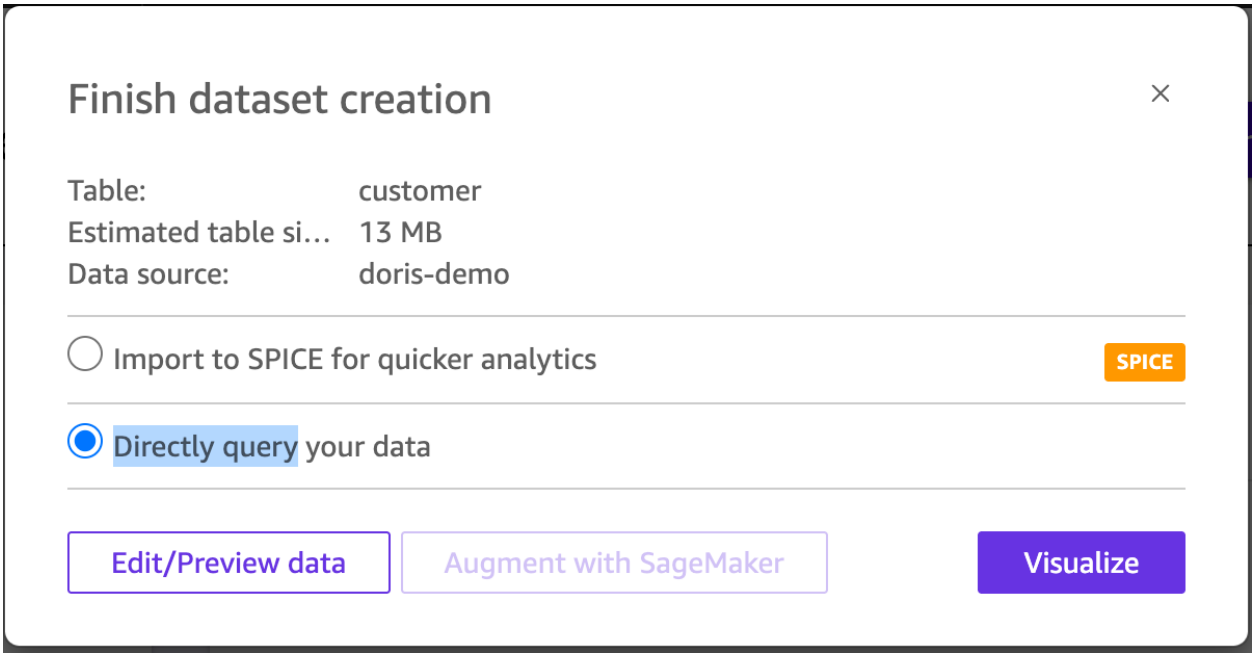


图 207:

此外，通过点击“Edit/Preview data”，您应该能够看到内部结构的表结构或调整自定义 SQL，并且可以在此处进行数据集的调整：

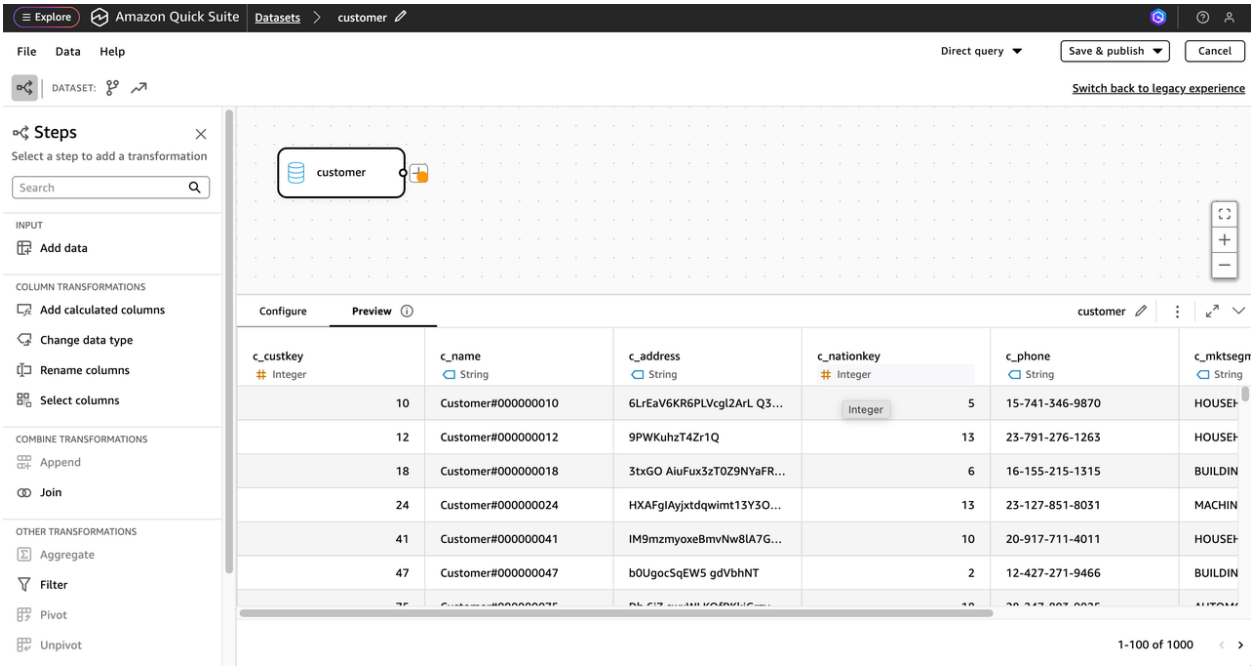


图 208:

现在，您可以继续发布数据集并创建新的可视化！

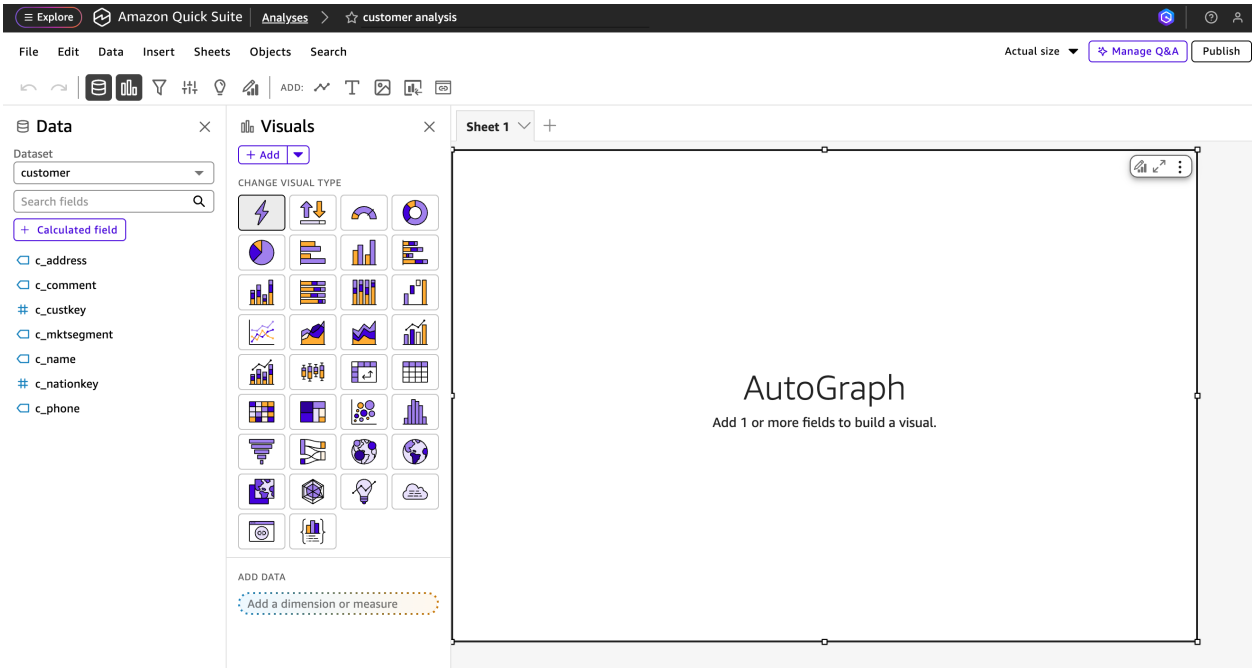


图 209:

5.6.5.3 在 QuickSight 中构建可视化

我们选择 TPC-H 数据作为数据源，Doris TPC-H 数据源构建方式参考此文档

现在我们在 QuickSight 中配置了 Doris 数据源，让我们可视化数据…

由于 Doris 在多表关联场景下的出色性能，我们选择一个设计多表关联的场景，假设我们需要知道各个国家不同状态的订单统计，接下来按照此需求进行看板构建

1. 使用上述步骤创建的 Data source 添加以下表作为 Dataset

- customer
- nation
- orders

2. 点击创建数据集

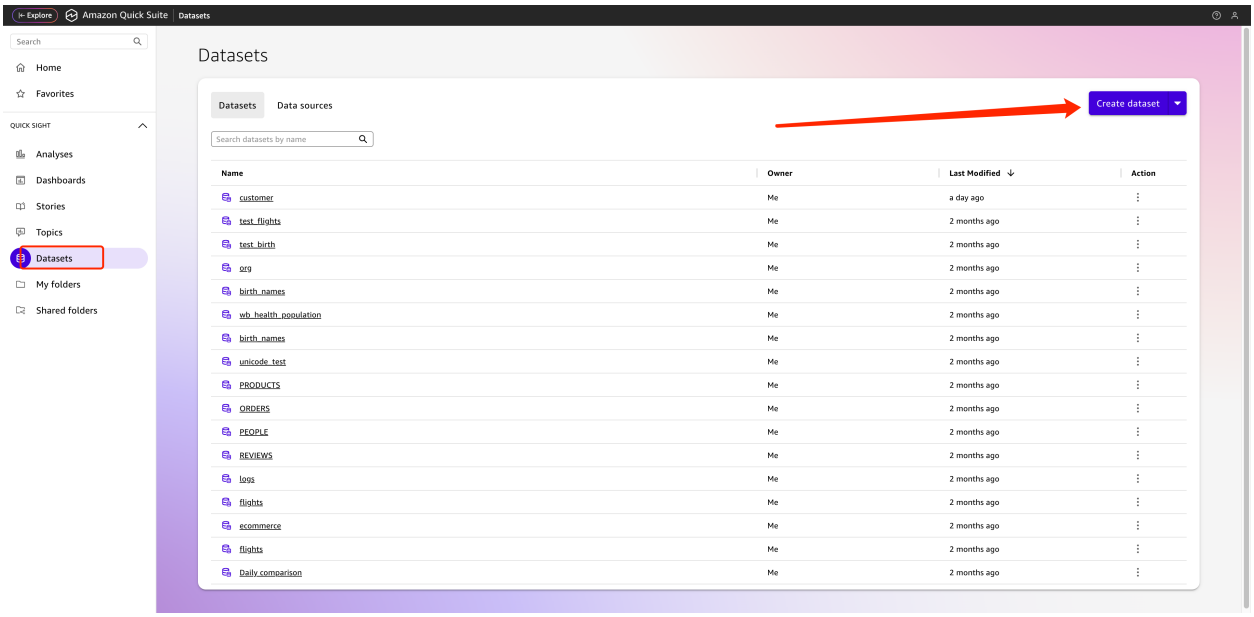


图 210:

3. 选用上述步骤创建的数据源

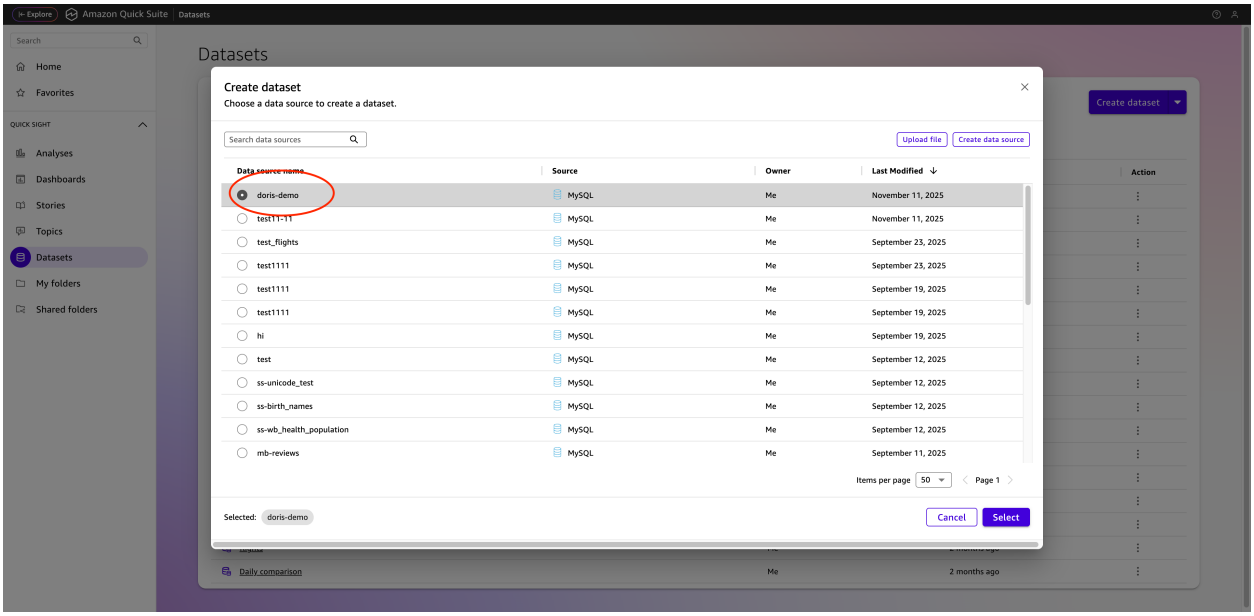


图 211:

4. 选择需要的表

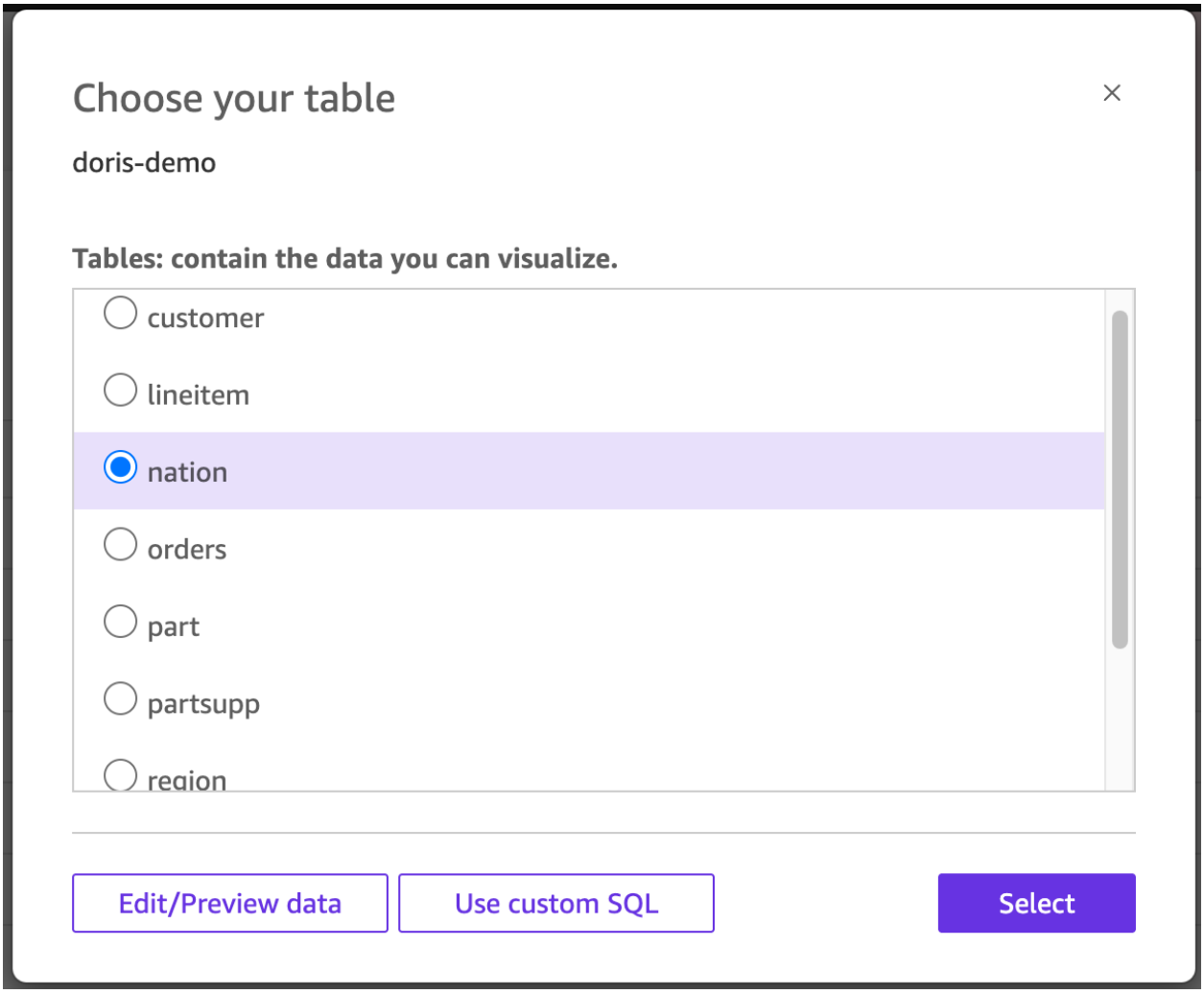


图 212:

选择 Directly query 模式

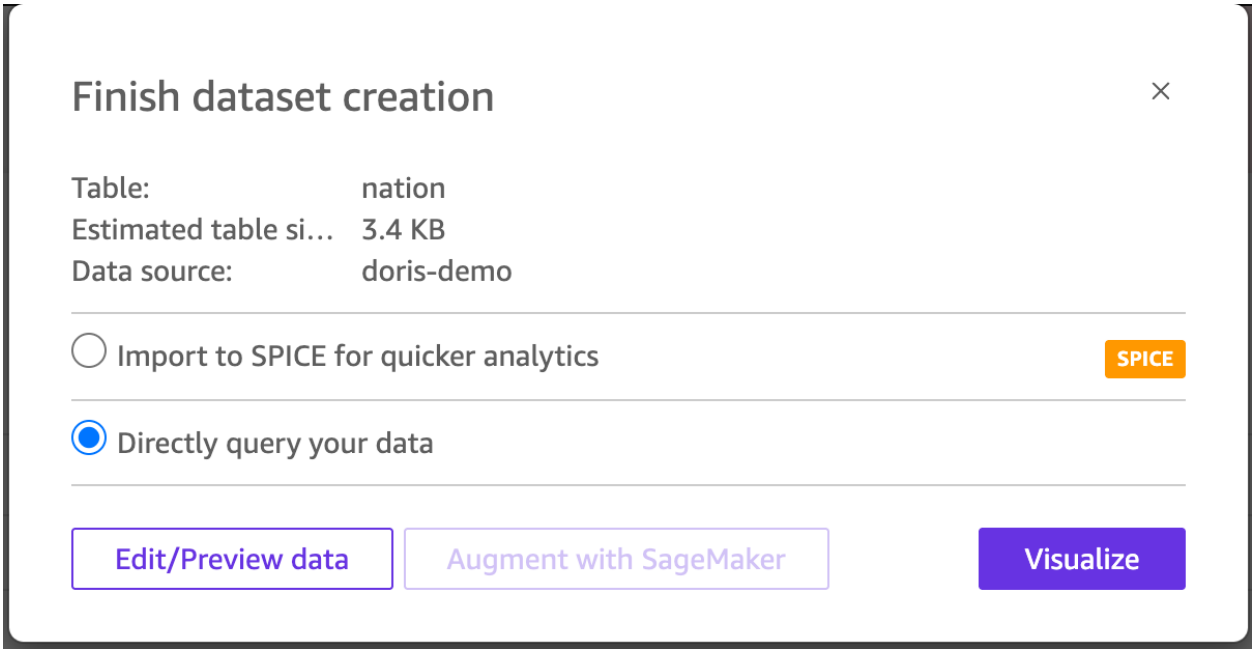


图 213:

点击 Visualize 创建数据源，按照此步骤为其他表也创建数据源

5. 进入仪表盘制作工作台，点击当前 Dataset 下拉框，选择添加新的数据集

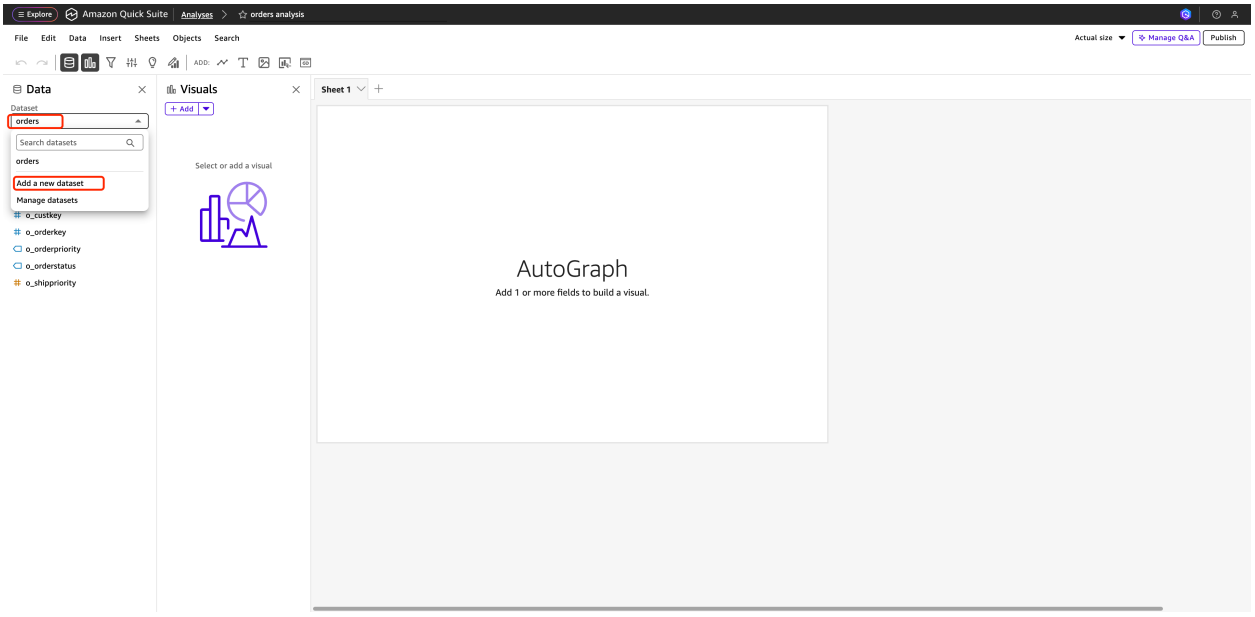


图 214:

6. 将所有的数据集依次勾选，点击 Select，添加入该仪表盘

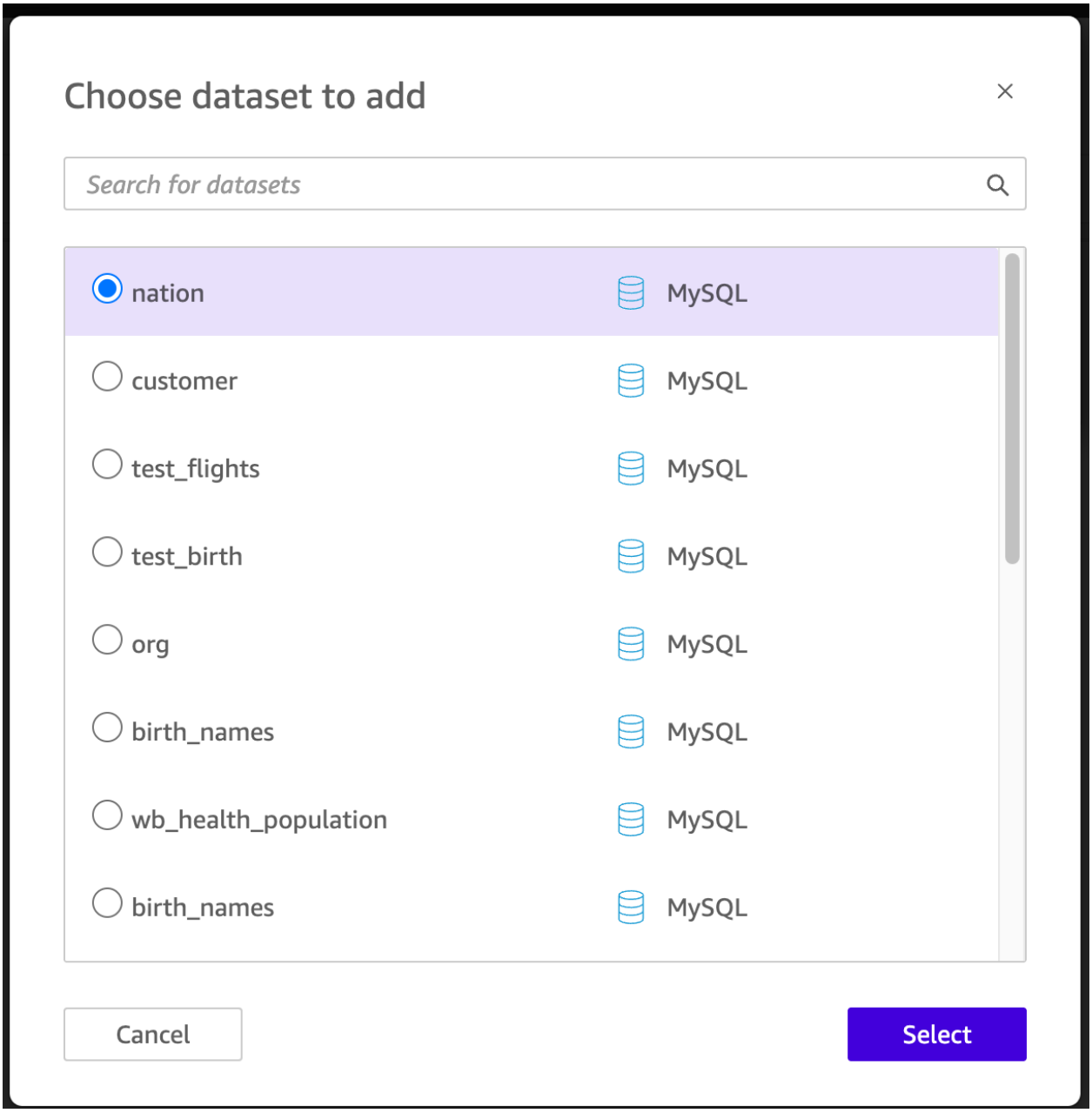


图 215:

7. 完成后点击 nation 的操作界面进入编辑数据集界面，我们接下来为数据集进行列关联

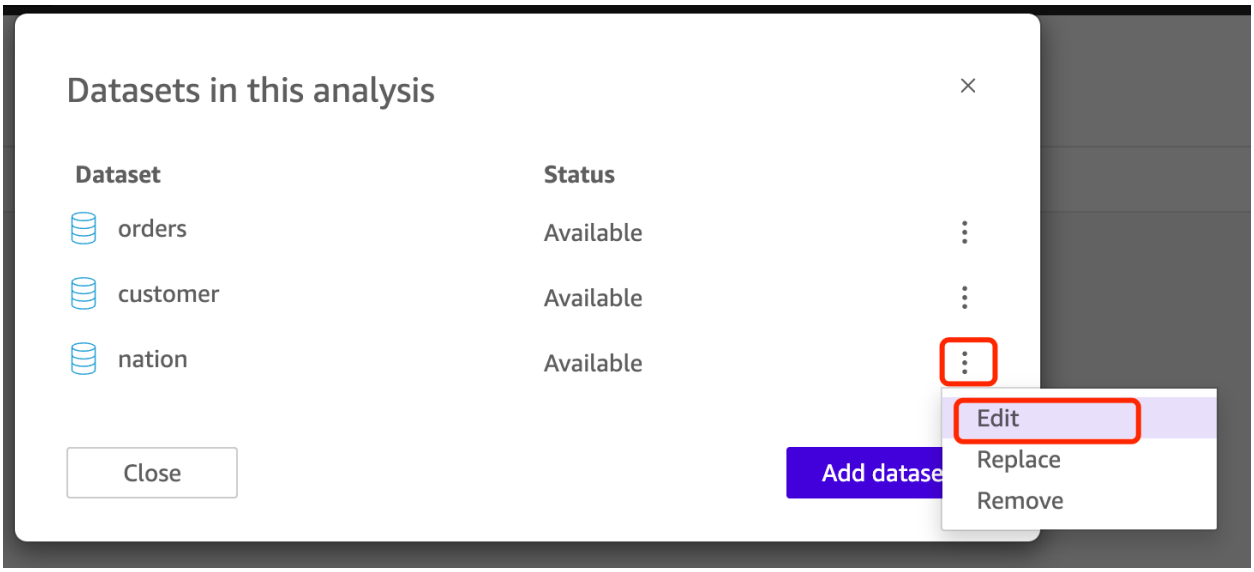


图 216:

8. 如图点击 Add data 添加数据源

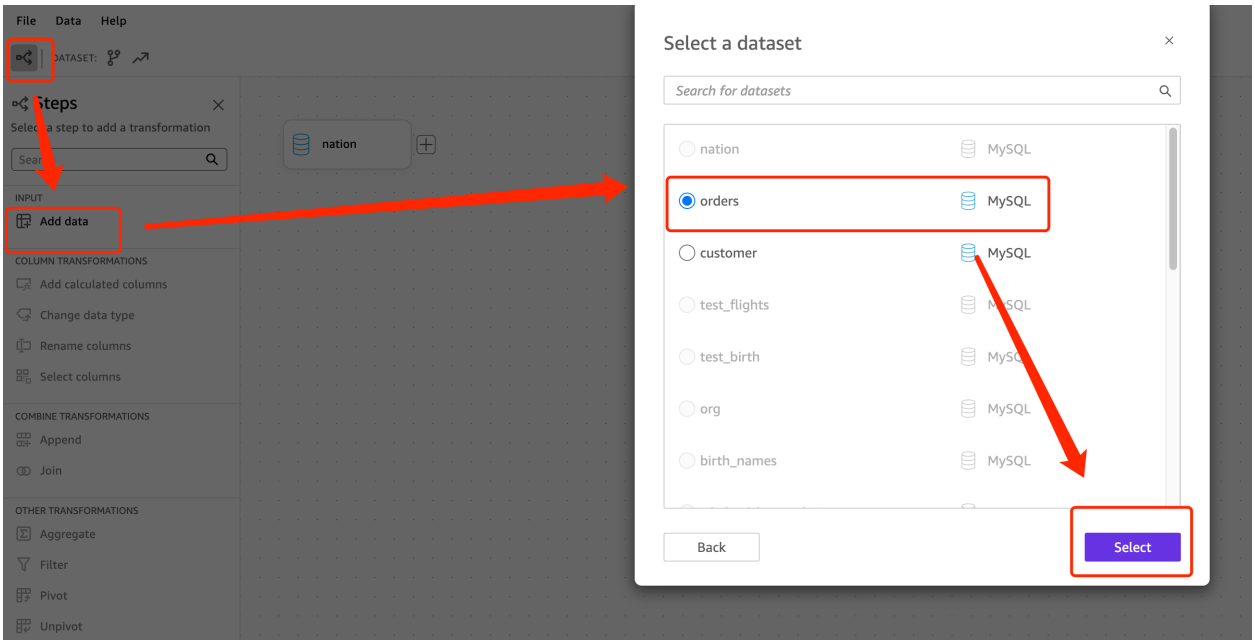


图 217:

9. 将三张表添加进去后，进行关联键，关联关系如下：

- customer： c_nationkey – nation : n_nationkey
- customer： c_custkey – orders : o_custkey

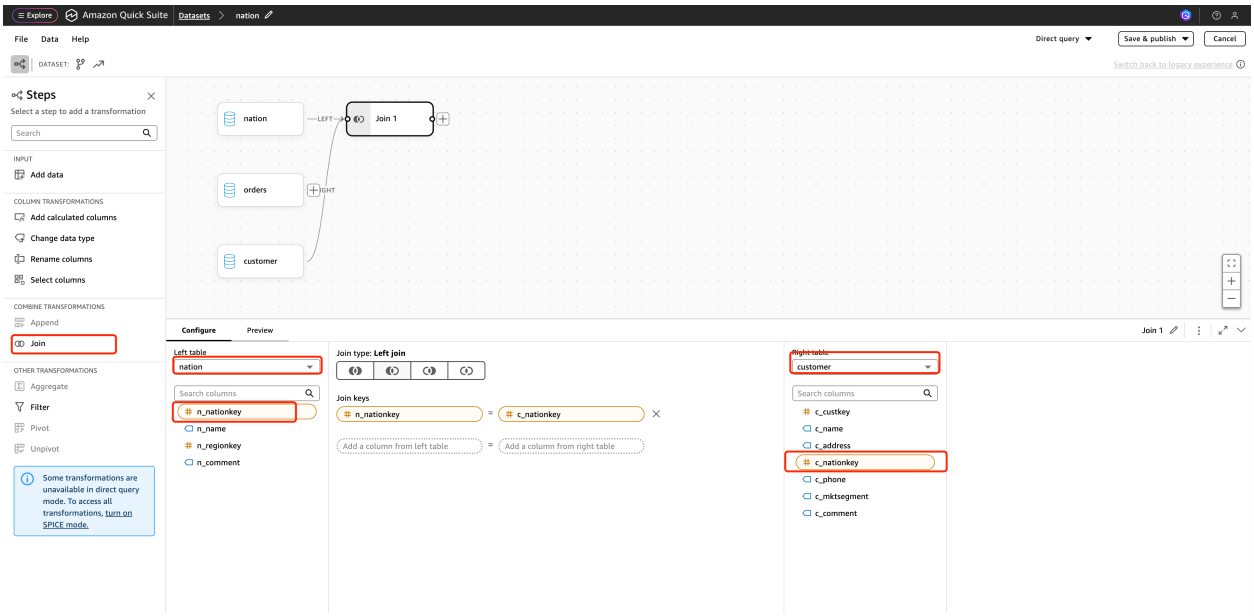


图 218:

10. 最终关联完成，点击右上角 Save & publish 发布

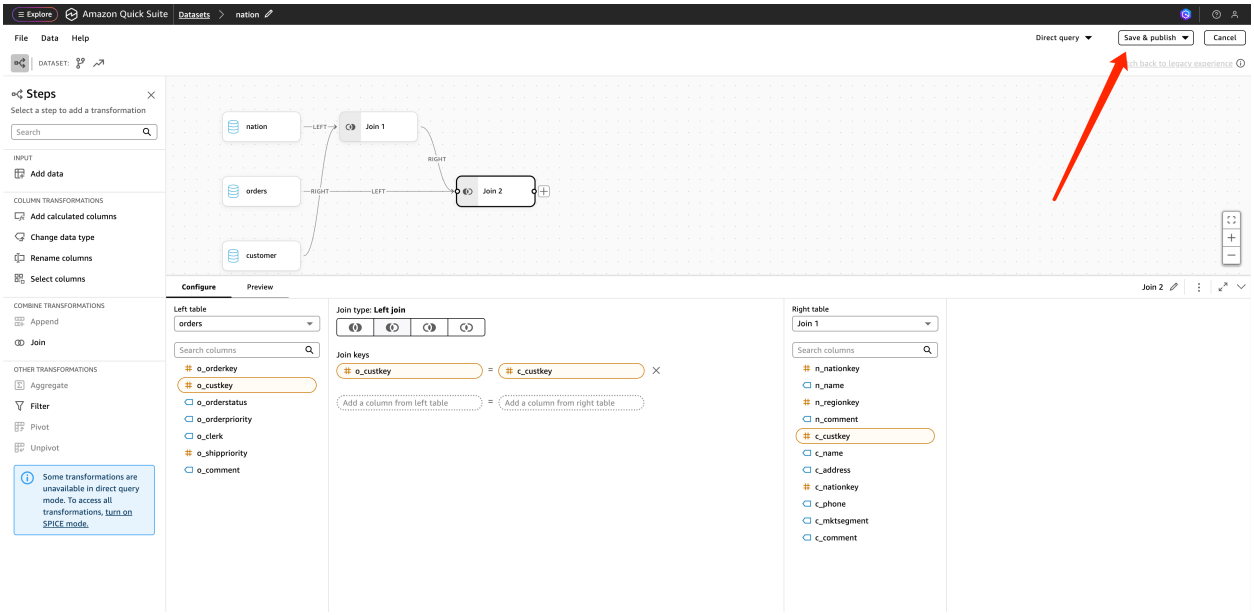


图 219:

11. 回到刚刚添加三个数据源的 Analyses 界面，点击 n_name 出现按国家名称的订单总数统计图

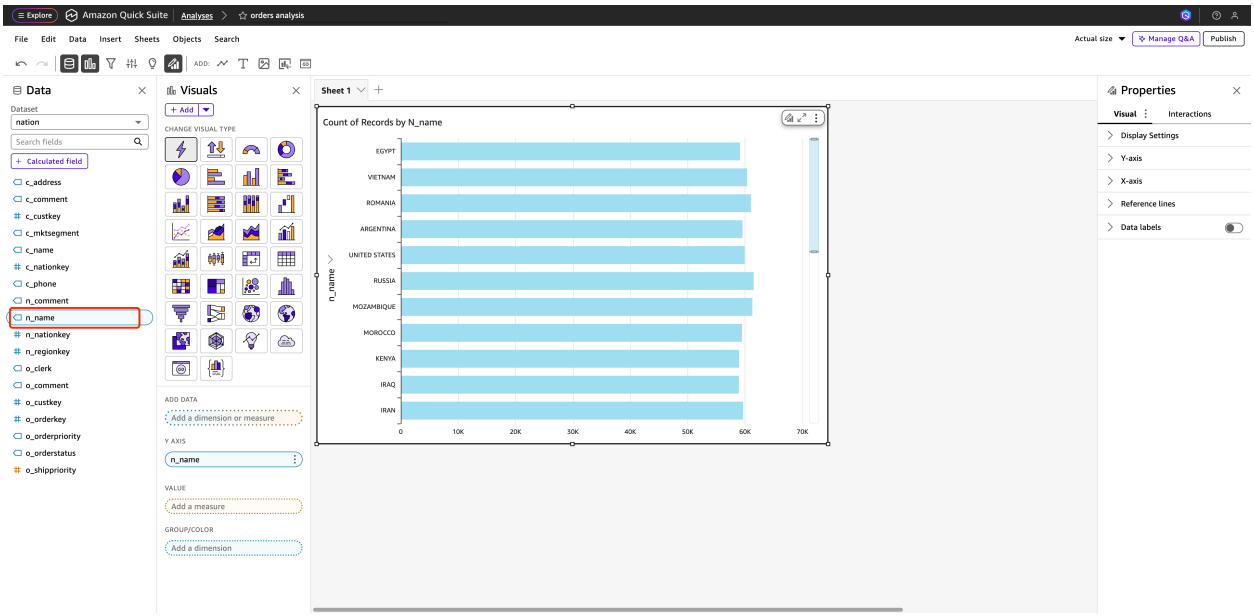


图 220:

12. 点击 VALUE 选中 o_orderkey ，点击 GROUP/COLOR 选中 o_orderstatus ，即可得到需求看板

图 221:

13. 点击右上角 Publish 即可完成看板发布

至此，已经成功将 QuickSight 连接到 Apache Doris，并实现了数据分析和可视化看板制作。

5.6.6 Quick BI

5.6.6.1 Quick BI 介绍

Quick BI 是一款基于数据仓库的商业智能工具，它可以帮助企业快速搭建起酷炫的可视化分析罗盘。Quick BI 支持多种数据源，包括 MySQL、Oracle、SQL Server、Apache Doris 等数据库，以及 Excel、CSV、JSON 等文件格式。它提供了丰富的可视化组件，如表格、图表、地图等，用户可以通过简单的拖拽操作，轻松实现数据的可视化分析。#### 数据连接与应用 1. 登录 Quick BI 并建立一个工作区。2. 在当前的工作区下点击新建数据源

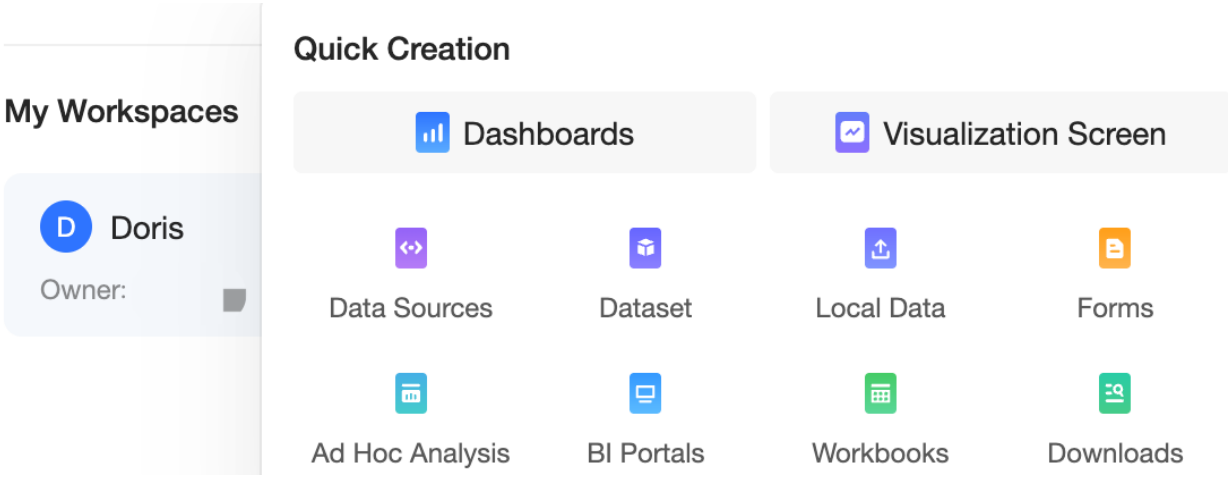


图 222: create workspace

3. 在新建的数据源中选择 Apache Doris，并填写对应 Doris 的连接信息。

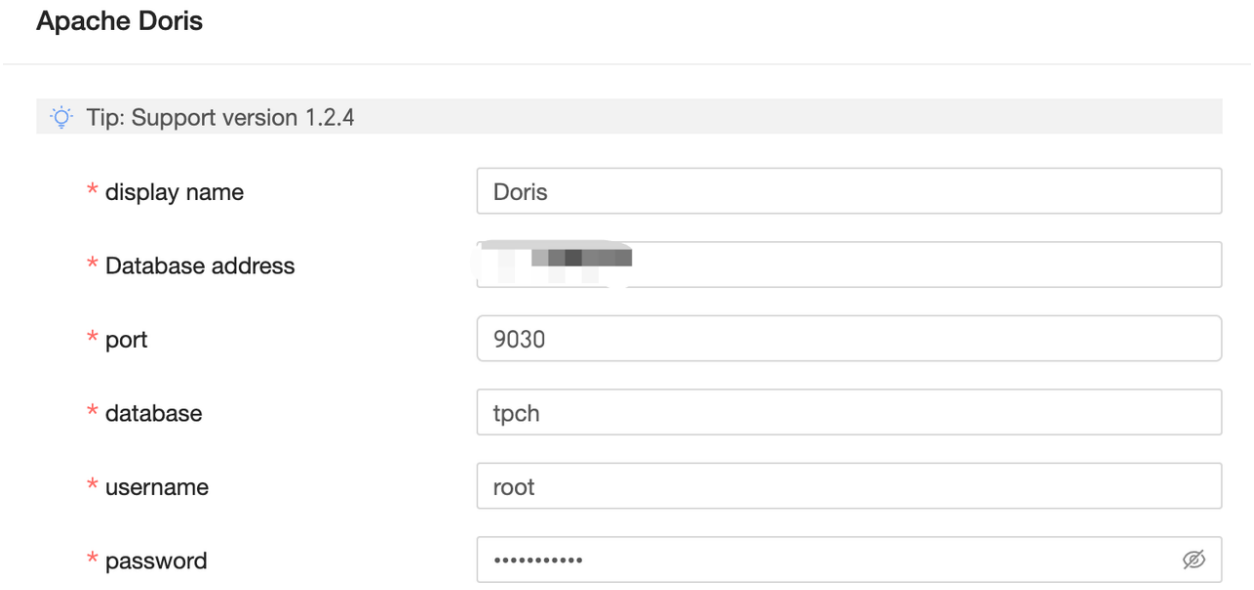


图 223: Doris information

4. 连接成功后，可在数据源列表中看到我们创建的数据源。

图 224: data source

5. 在创建的数据源中创建一个数据集，此处以 TPC-H 数据集为例。创建数据集后即可设置对应的报表。

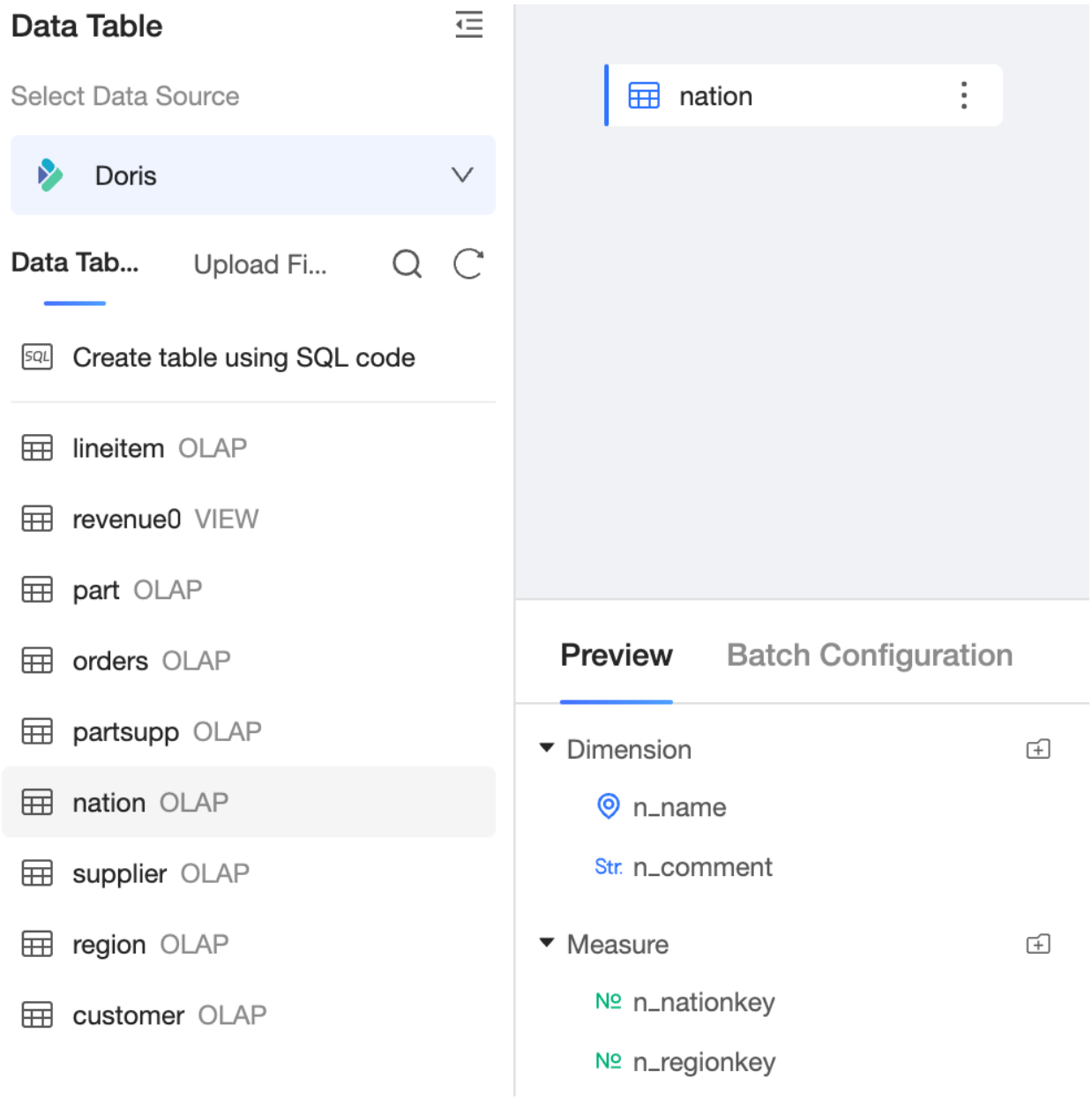


图 225: Doris table

5.6.7 Smartbi

5.6.7.1 Smartbi 介绍

Smartbi 是软件服务、应用连接器的集合，其可以连接到多种数据源，包括 Oracle、SQL Server、MySQL 和 Doris 等，以便用户可以轻松地整合和清洗数据。借助 Smartbi 的数据建模功能，用户不仅可以创建复杂的关系模型，还能编写数据分析表达式并建立数据关系，从而为高级的数据分析和可视化奠定基础。Smartbi 提供了丰富多样的可视化选项，包括各种类型的图表、地理地图、交互式仪表板，以及支持用户自定义的可视化工具。这些功能强大的工具帮助用户更直观、更全面地理解和展示他们的数据，提升数据分析的效果和效率。

5.6.7.2 数据连接与应用

1. 登录 Smartbi 之后点击数据连接。

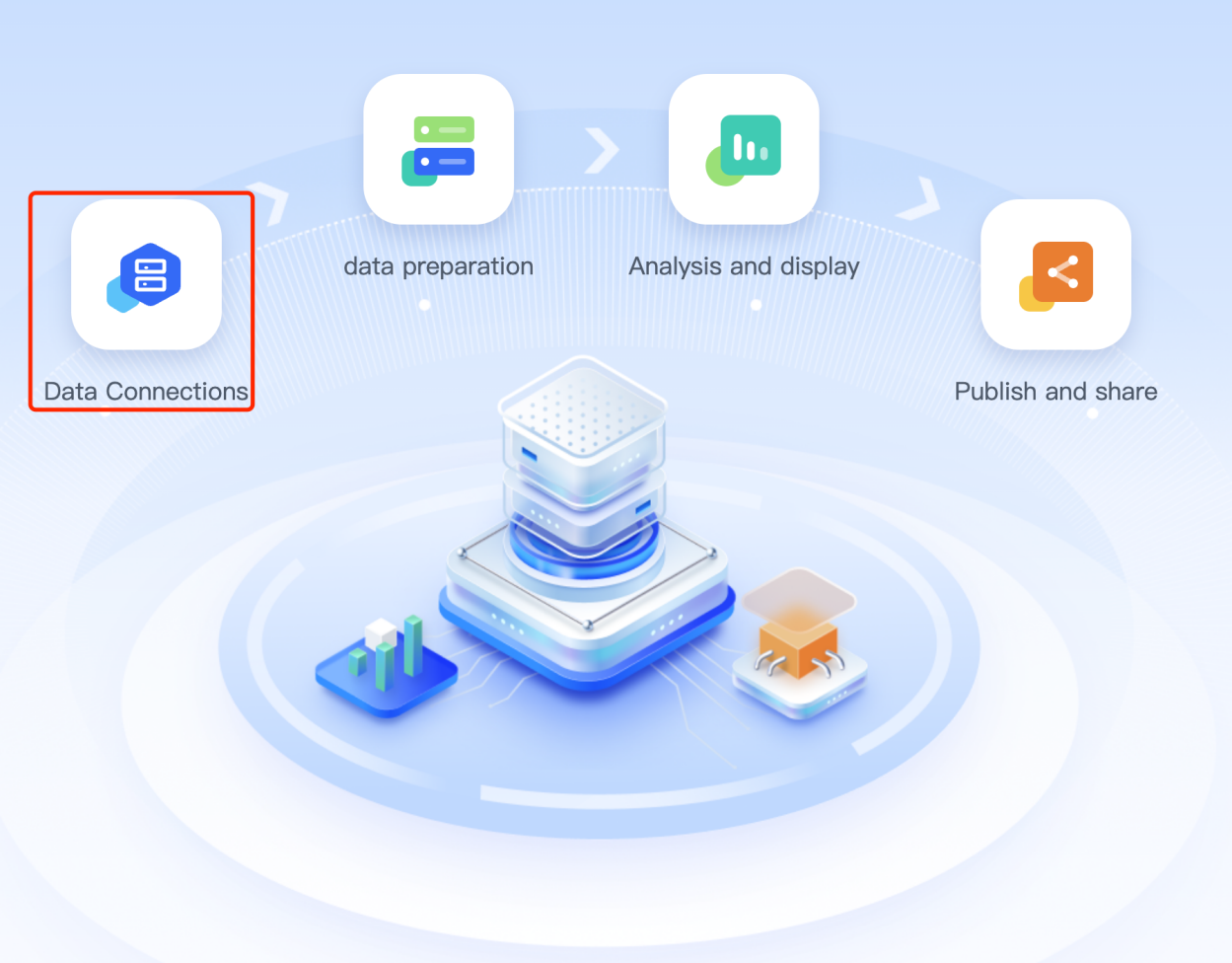


图 226: main page

2. 在数据连接页面中可以看到各个待连接的数据库列表，选择 Doris 数据库。

图 227: selectdb

3. 在选择数据库之后，填写 Doris 数据库的连接信息。

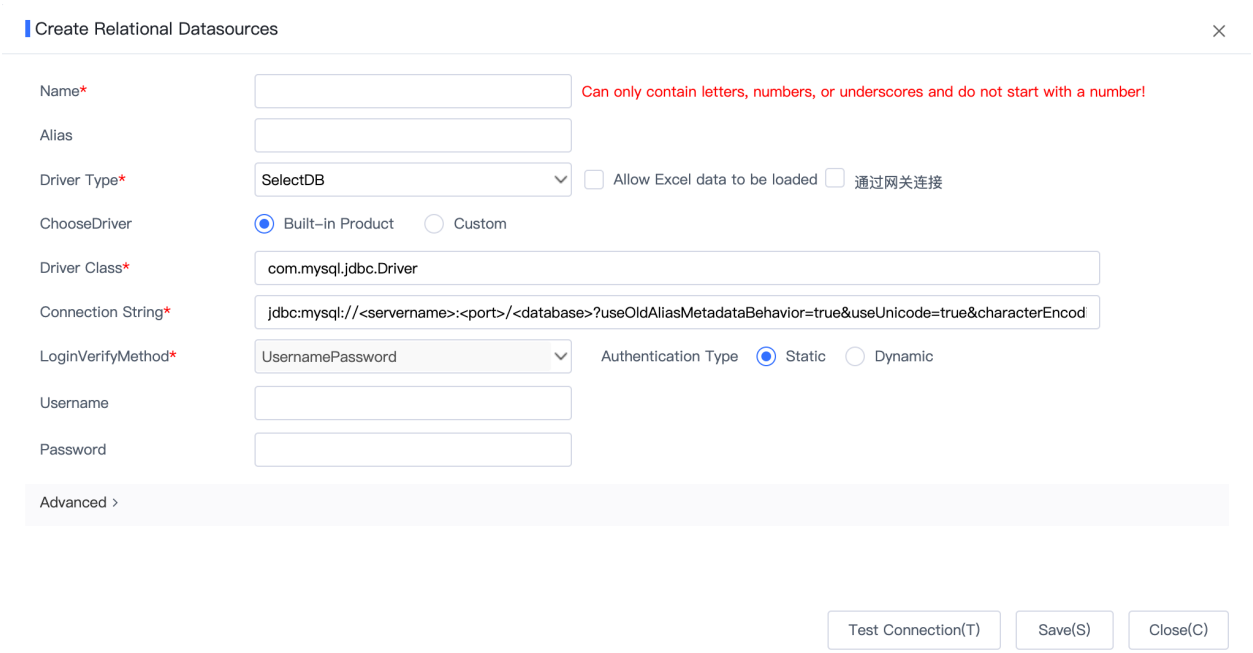


图 228: data source connection

4. 若填写信息无误，且连接成功后会显示测试通过。

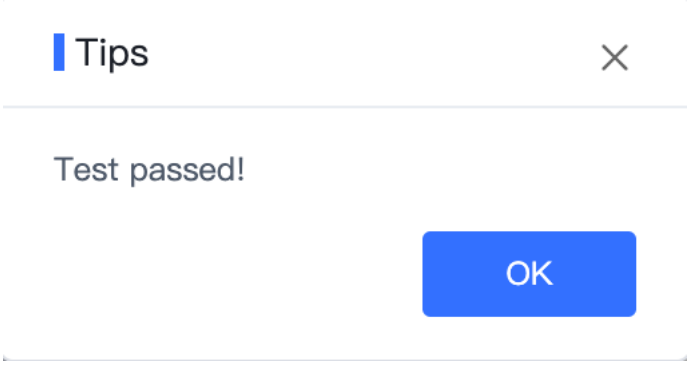


图 229: test passed

5. 连接成功后，在数据分析模块电子服务看板，可以自定义设置所需要的报表信息。

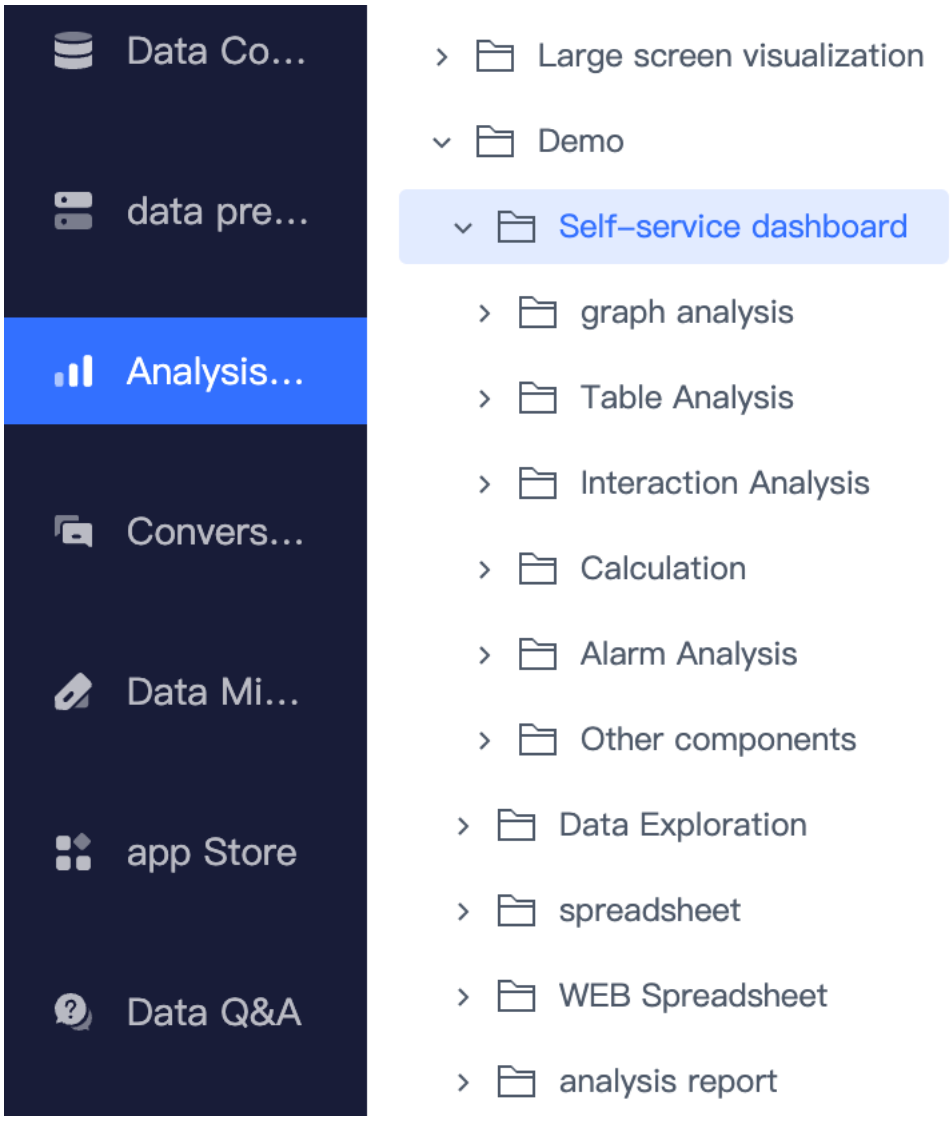


图 230: data analysis

5.7 SQL Clients

5.7.1 CloudDM

5.7.1.1 介绍

CloudDM 由 Clougence 研发，是一款为团队和个人用户而设计的跨平台数据库工具，帮助实现安全、高效、合规的数据库变更与管理。

CloudDM 针对 Apache Doris 的一些特性提供专项适配，并为 Apache Doris 提供数据访问、数据脱敏、可视化编辑、数据库 CI/CD 能力。

5.7.1.2 前置条件

已安装 CloudDM。可以访问 https://www.cdmgr.com/ 下载并安装 CloudDM。

5.7.1.3 添加数据源

> 备注当前验证使用 CloudDM 2.8.0.0 版本。

1. 登录 CloudDM。
2. 在导航栏点击 数据源管理 > 新增数据源。
3. 选择 Doris 数据源。

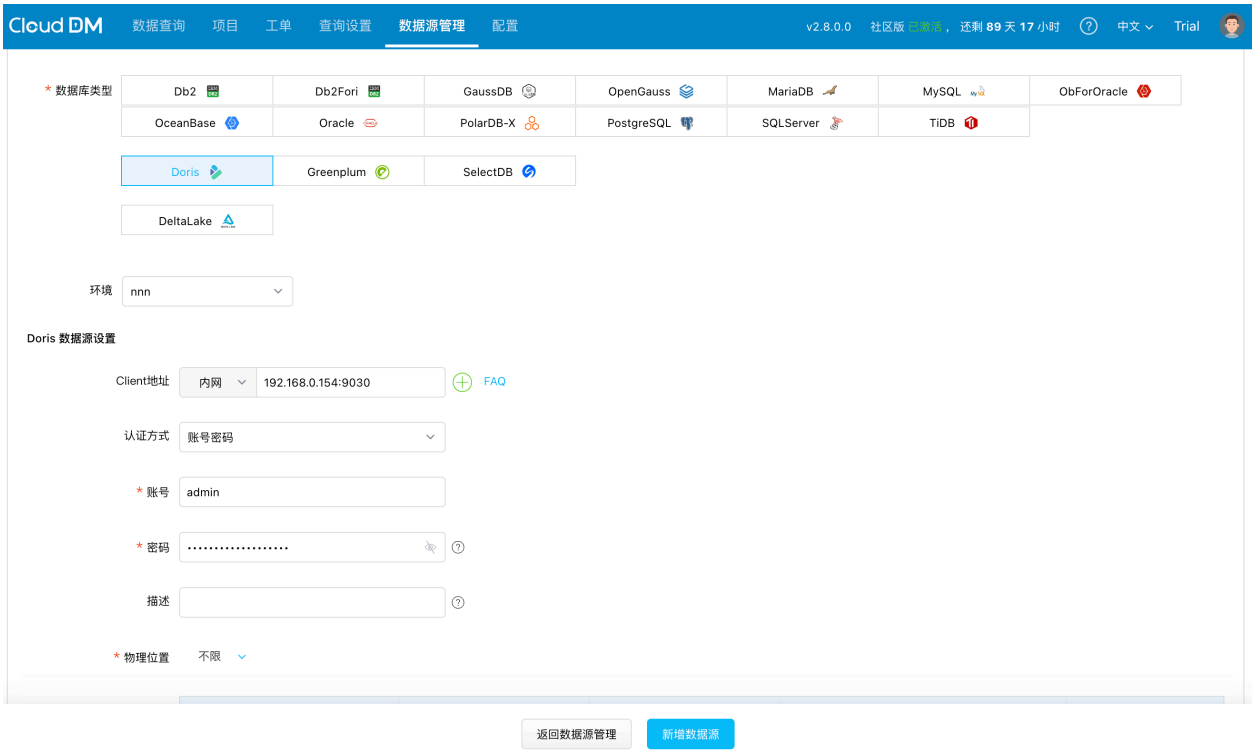


图 231: 添加数据源

4. 在添加数据源页面中，配置以下连接信息：

- Client 地址：Doris 集群机器的 FE 查询端口，如 hostID:9030。
- 账号：用于登录 Doris 集群的用户名，如 admin。
- 密码：用于登录 Doris 集群的用户密码。

> `Doris 分为 internal catalog 和 external catalog，CloudDM 可以同时管理它们。`

> 备注通过 catalog.db 的 Database 形式来管理连接 Doris 的 external catalog 需要 Doris 版本在 2.1.0 及以上。

5. 在上方点击 查询设置 > 查询配置，为 Doris 实例启用数据管理，并测试连接。

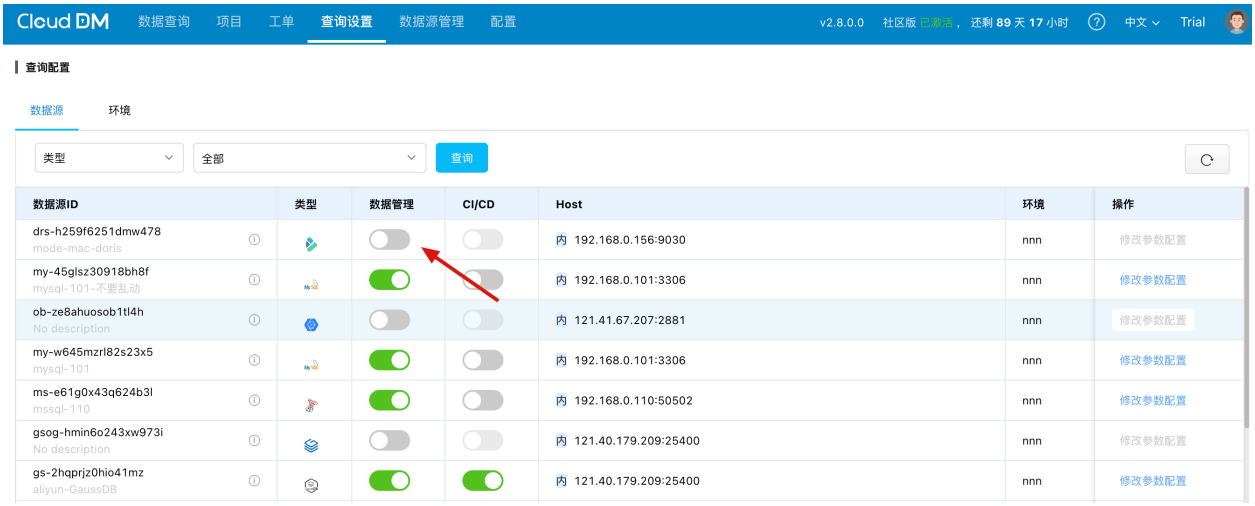


图 232: 启用数据源

6. 访问数据

   数据库连接建立以后，可以在左侧的数据库连接导航看到已连接的数据源，并且可以通过 CloudDM 连接并管理数据库。

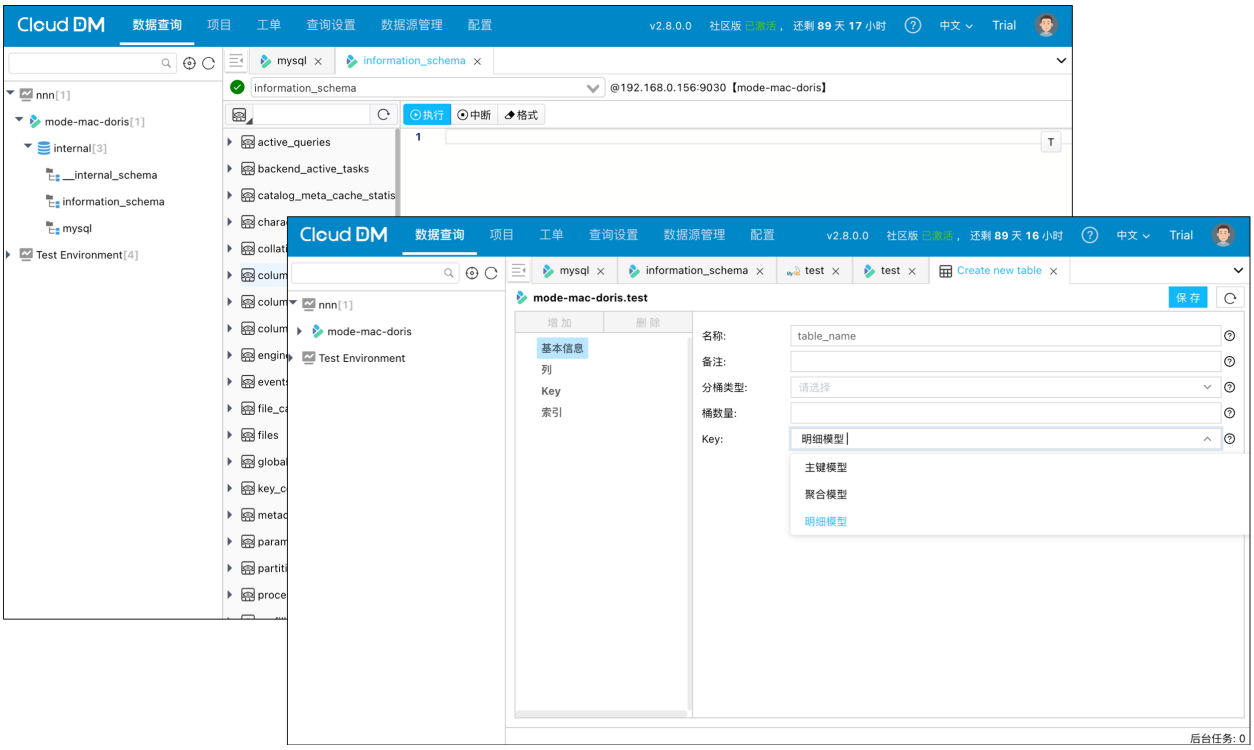


图 233: 建立连接

5.7.1.4 功能支持

- 查询客户端
- 可视化管理 Doris 中的数据库对象
- 控制台编写 SQL 操作 Doris
- 查询结果导出
- 团队化使用
- 语句级授权，粒度表级别
- 工单审批
- 数据库 CI/CD
- 敏感数据脱敏
- SQL 审核规则

5.7.2 DBeaver

5.7.2.1 介绍

DBeaver 是一款跨平台数据库工具，适用于开发人员、数据库管理员、分析师和所有处理数据的人。

Apache Doris 高度兼容 MySQL 协议，可以使用 DBeaver 的 MySQL 驱动器连接 Apache Doris，并查询 internal catalog 和 external catalog 中的数据。

5.7.2.2 前置条件

已安装 Dbeaver 可以访问 https://dbeaver.io 下载安装 DBeaver

5.7.2.3 添加数据源

> 备注当前验证使用 DBeaver 24.0.0 版本

1. 启动 DBeaver
2. 在 DBeaver 窗口左上角单击加号 (+) 图标，或者在菜单栏选择 Database > New Database Connection，打开 Connect to a database 界面。

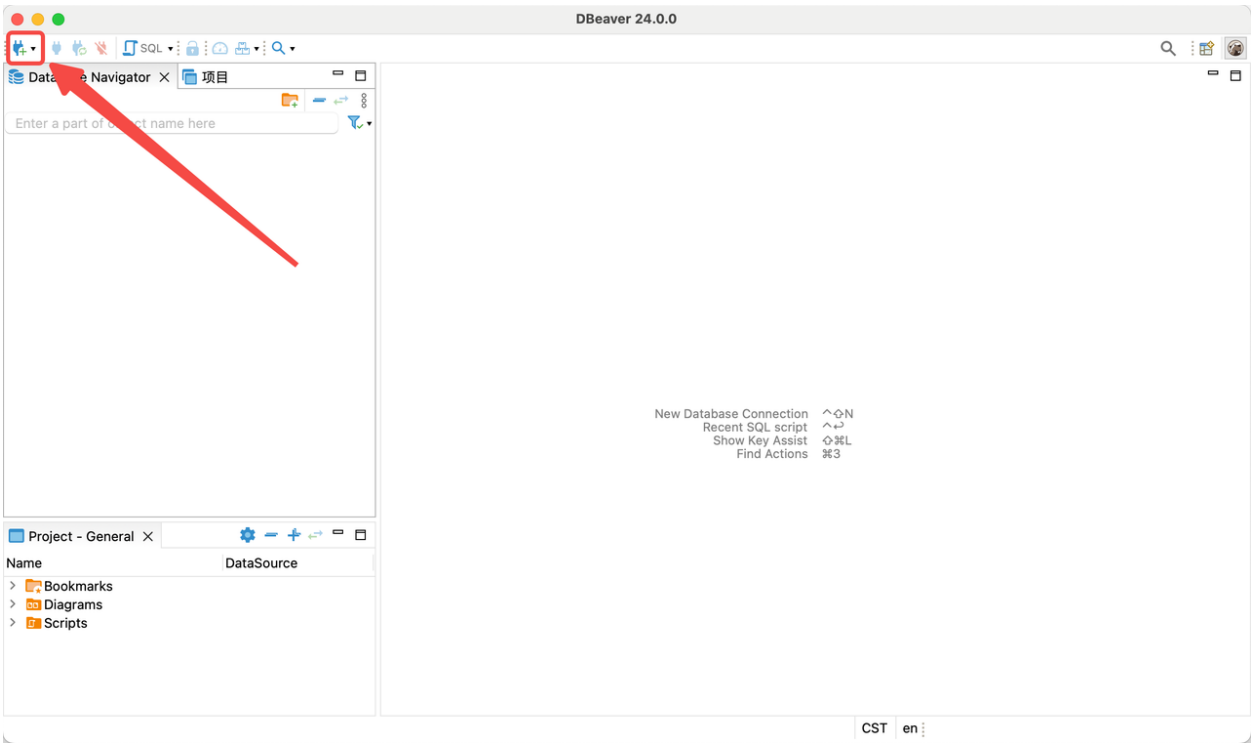


图 234: 添加连接 1

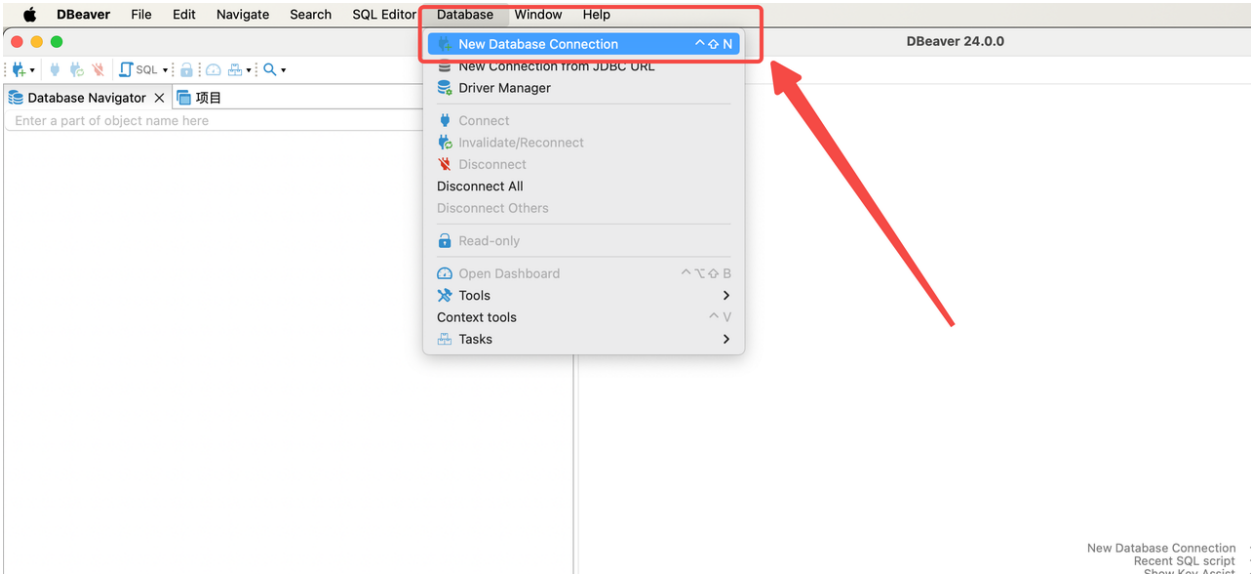


图 235: 添加连接 2

3. 选择 MySQL 驱动器

   在 Select your database 窗口，选择 MySQL 。

图 236: 选择驱动

4. 配置 Doris 连接

   在 Connection Settings 窗口的 main 标签页，配置以下连接信息：

- Server Host：Doris 集群的 FE 主机 IP 地址。
- Port：Doris 集群的 FE 查询端口，如 9030。
- Database：Doris 集群中的目标数据库。
- Username：用于登录 Doris 集群的用户名，如 admin。
- Password：用于登录 Doris 集群的用户密码。

```
Database 可以用于区别 internal catalog 和 external catalog，如仅填写 Database 名称
    ↪ ，则当前数据源默认连接 internal catalog，如填写格式为 catalog.db，
    ↪ 则当前数据源默认连接 Database 中所填写的 catalog，DBeaver
    ↪ 中展示的库表也为所连接 catalog 中的库表，因此可以使用 DBeaver 的 MySQL
    ↪ 驱动器来创建多个 Doris 数据源来管理 Doris 中不同的 Catalog。
```

备注通过 catalog.db 的 Database 形式来管理连接 Doris 的 external catalog 需要 Doris 版本在 2.1.0 及以上

- internal catalog

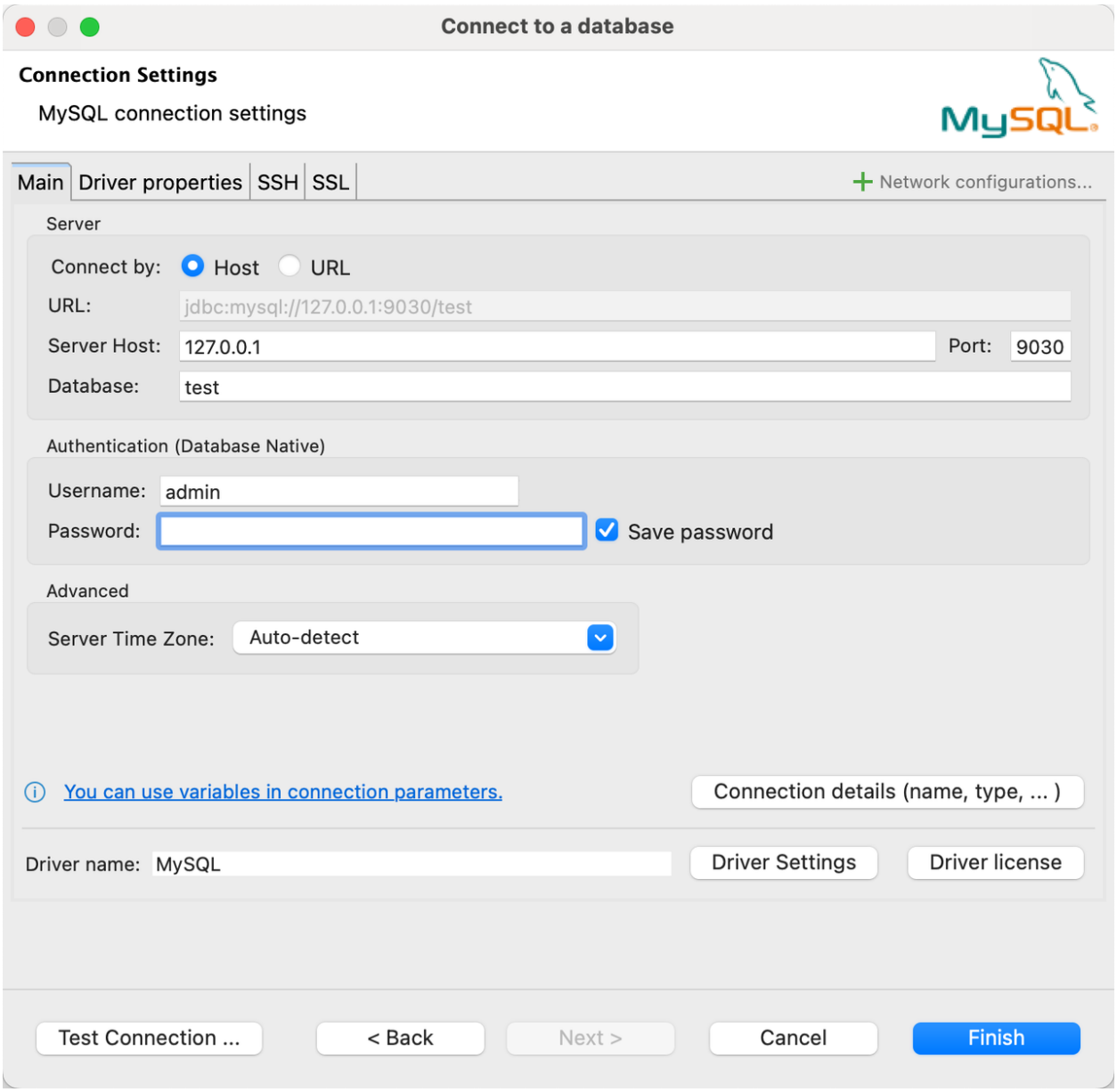


图 237: 连接 internal catalog

- external catalog

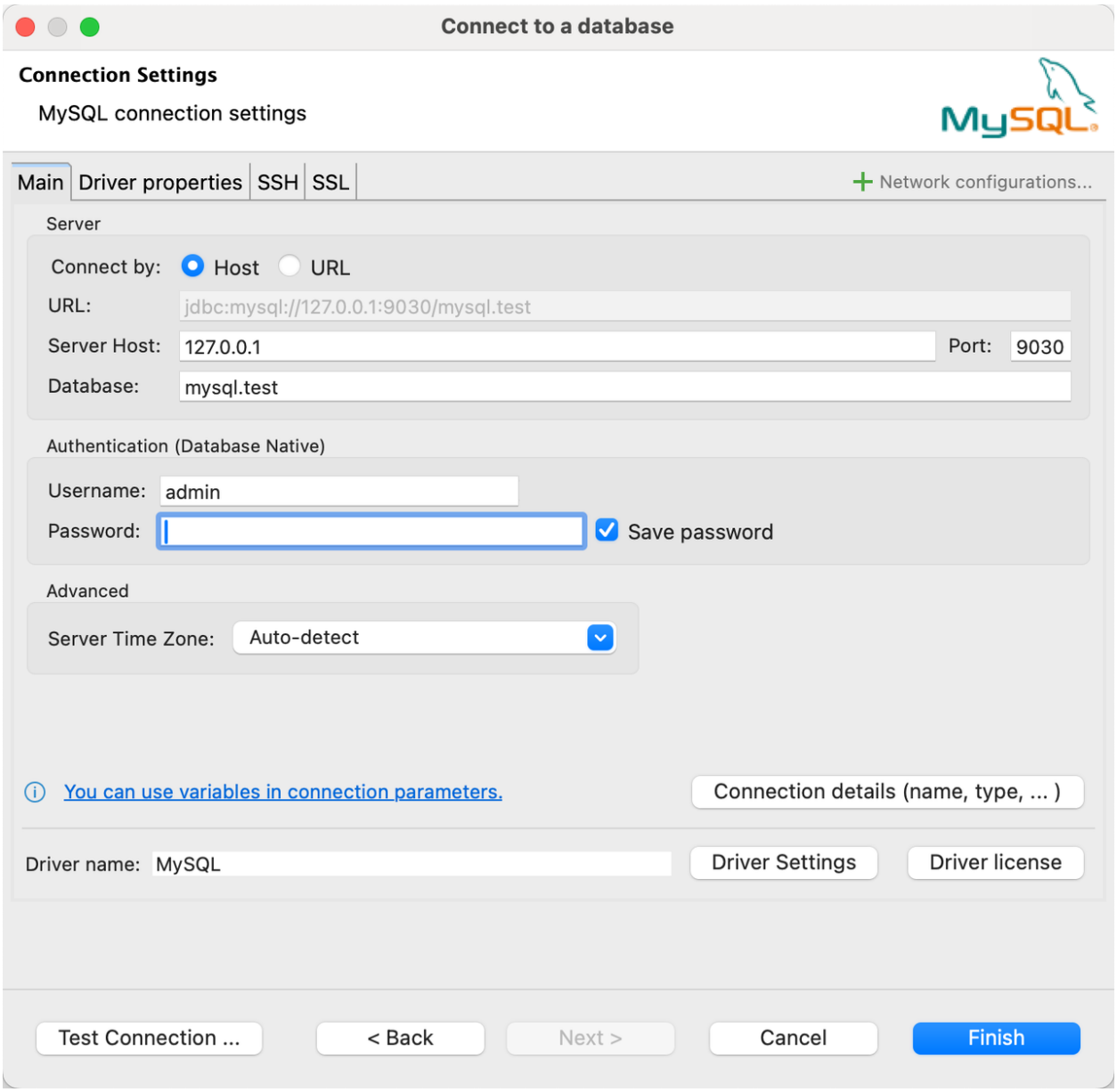


图 238: 连接 external catalog

5. 测试数据源连接

   在填写完连接信息后，单击左下角 Test Connection 验证数据库连接信息的准确性。DBeaver 返回如下对话框，确认配置连接信息。单击 OK 即确认配置连接信息无误。然后单击右下角 Finish 完成连接配置。

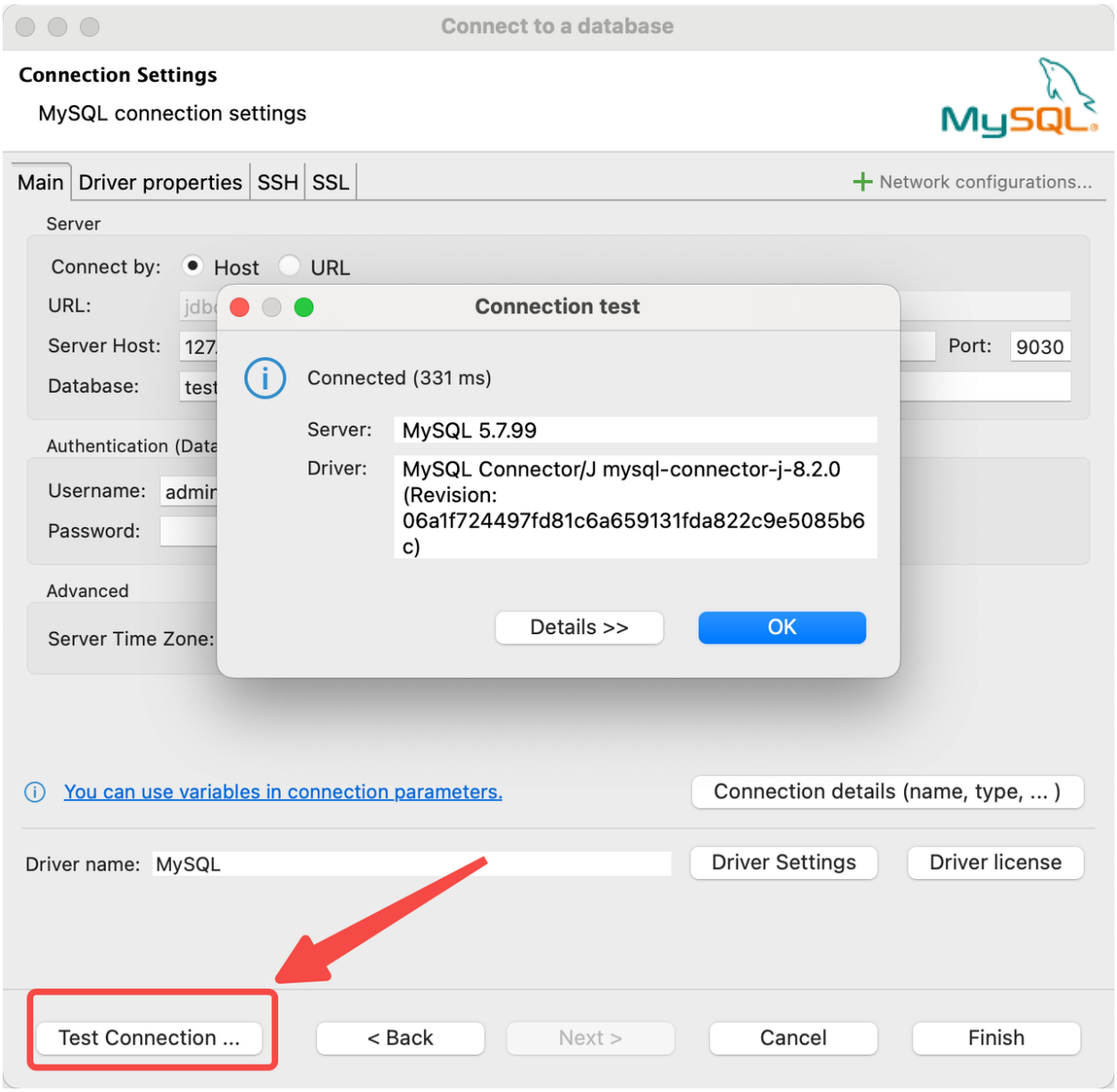


图 239: 测试连接

6. 连接数据库

   数据库连接建立以后，可以在左侧的数据库连接导航看到已创建的数据源连接，并且可以通过 DBeaver 连接并管理数据库。

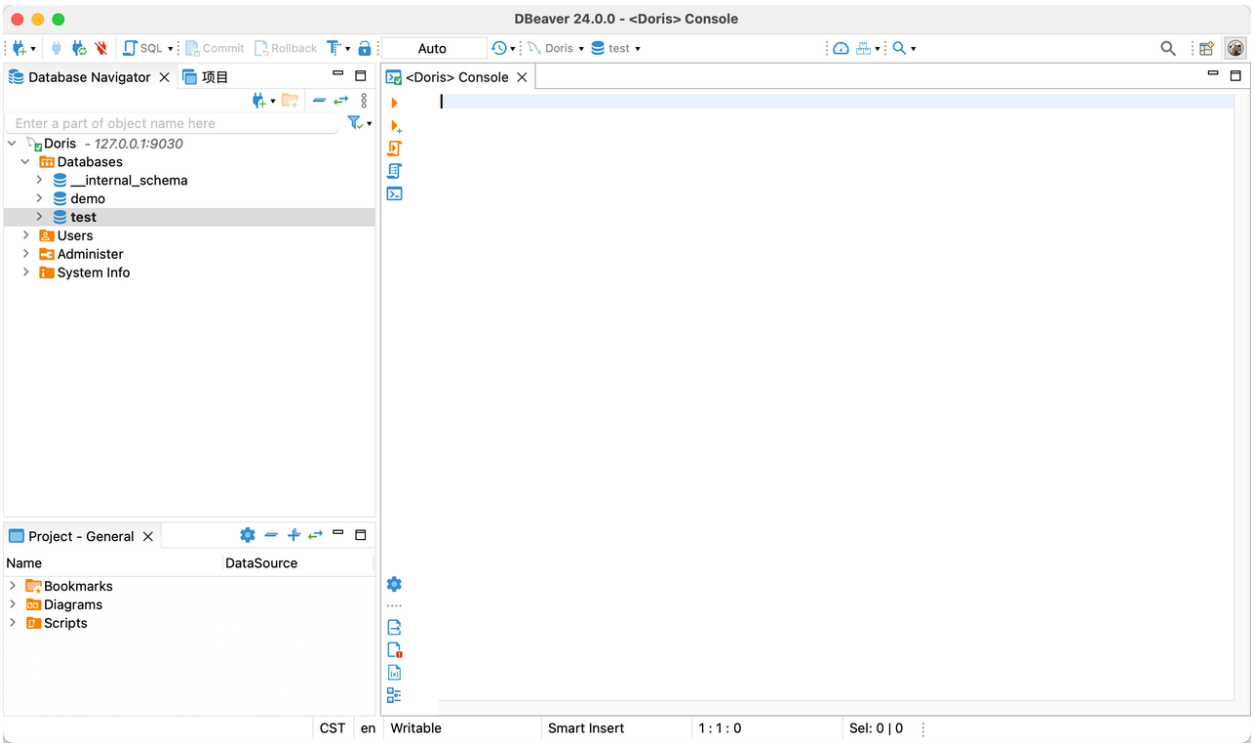


图 240: 建立连接

5.7.2.4 功能支持

- 完全支持
- 可视化查看类
  - Databases
  - Tables
  - Views
- Users
- Administer
  - Session Manager
- System Info
  - Session Variables
  - Global Variables
  - Engines
  - Charsets
  - User Priviages
  - Plugin

- 操作类
  - SQL 编辑器
  - SQL 控制台
- 基本支持

  基本支持的部分意为可以点击查看不会报错，但由于存在协议兼容问题，可能存在显示不全
- 可视化查看类
  - 仪表盘
  - Users/user/properties
  - Session Status
  - Global Status
- 不支持

  不支持部分意为使用 DBeaver 管理 Doris 进行某些可视化操作时可能会报错，或者某些可视化操作未经验证如可视化创建库表、schema change、增删改数据等

5.7.3 DataGrip

5.7.3.1 介绍

DataGrip 是 JetBrains 出品的适用于关系数据库和 NoSQL 数据库的强大跨平台数据库工具。

Apache Doris 高度兼容 MySQL 协议，可以使用 DataGrip 的 MySQL 数据源连接 Apache Doris，并查询 internal catalog 和 external catalog 中的数据。

5.7.3.2 前置条件

已安装 DataGrip 可以访问 www.jetbrains.com/datagrip/ 下载安装 DataGrip

5.7.3.3 添加数据源

> 备注当前验证使用 DataGrip 2023.3.4 版本

1. 启动 DataGrip
2. 在 DataGrip 窗口左上角单击加号 (+) 图标，选择 MySQL 数据源

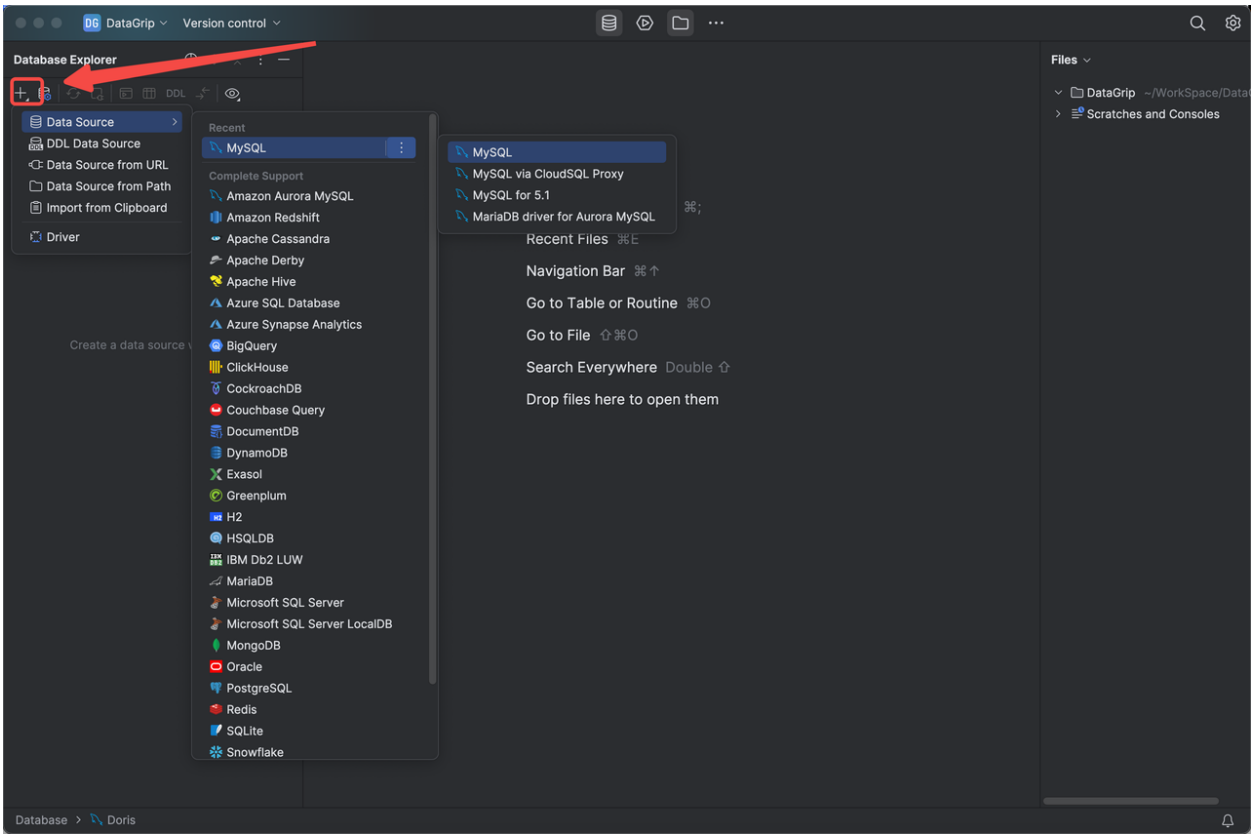


图 241: 添加数据源

3. 配置 Doris 连接

   在 Data Sources and Drivers 窗口的 General 标签页，配置以下连接信息：

- Host：Doris 集群的 FE 主机 IP 地址。
- Port：Doris 集群的 FE 查询端口，如 9030。
- Database：Doris 集群中的目标数据库。
- User：用于登录 Doris 集群的用户名，如 admin。
- Password：用于登录 Doris 集群的用户密码。

```
Database 可以用于区别 internal catalog 和 external catalog，如仅填写 Database 名称
    ↪ ，则当前数据源默认连接 internal catalog，如填写格式为 catalog.db，
    ↪ 则当前数据源默认连接 Database 中所填写的 catalog，DataGrip
    ↪ 中展示的库表也为所连接 catalog 中的库表，以此可以使用 DataGrip 的 MySQL
    ↪ 数据源来创建多个 Doris 数据源来管理 Doris 中不同的 Catalog。
```

备注通过 catalog.db 的 Database 形式来管理连接 Doris 的 external catalog 需要 Doris 版本在 2.1.0 及以上

- internal catalog

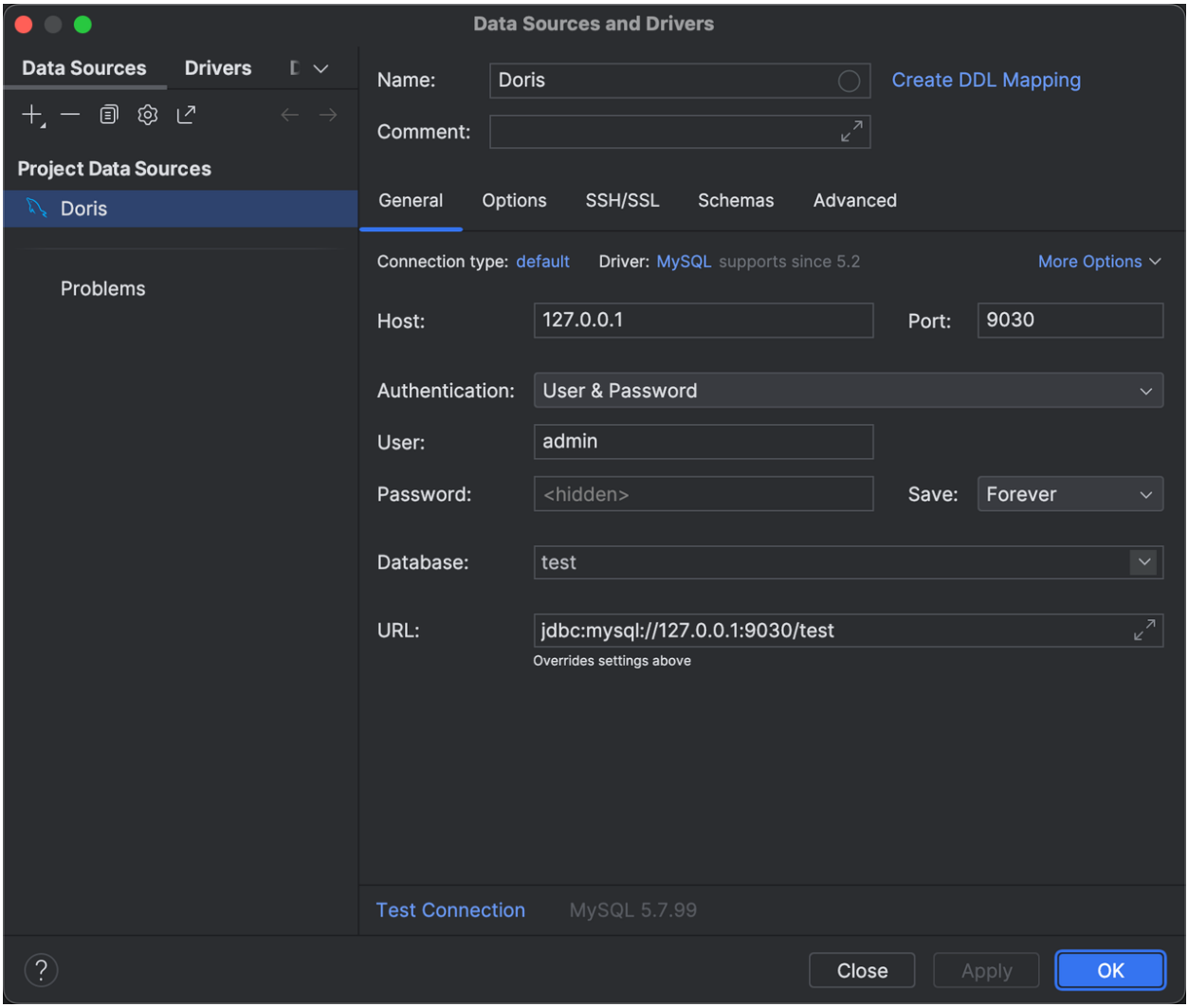


图 242: 连接 internal catalog

- external catalog

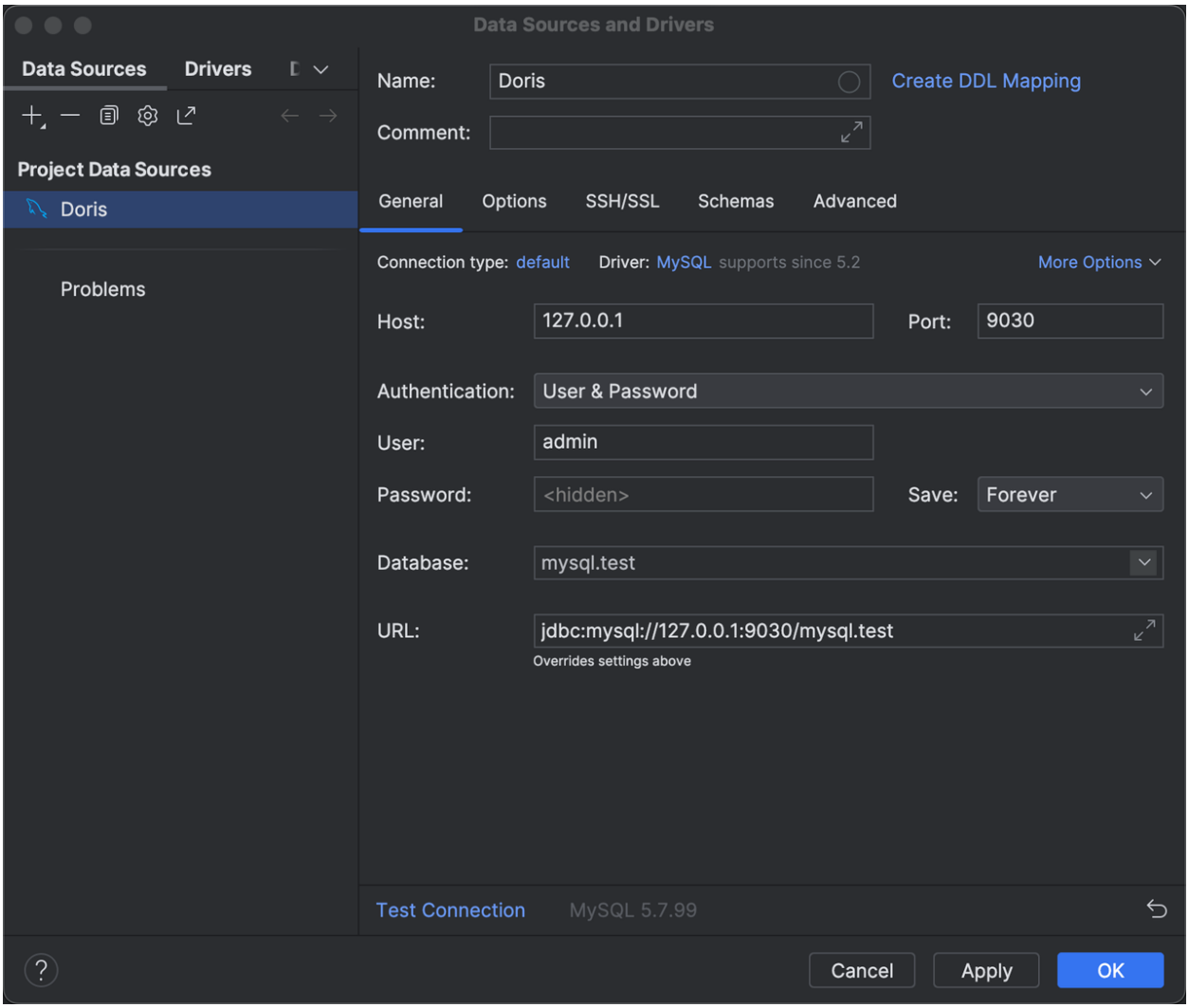


图 243: 连接 external catalog

5. 测试数据源连接

   在填写完连接信息后，单击左下角 Test Connection 验证数据库连接信息的准确性。DataGrip 返回如下对弹窗则测试连接成功。然后单击右下角 OK 完成连接配置。

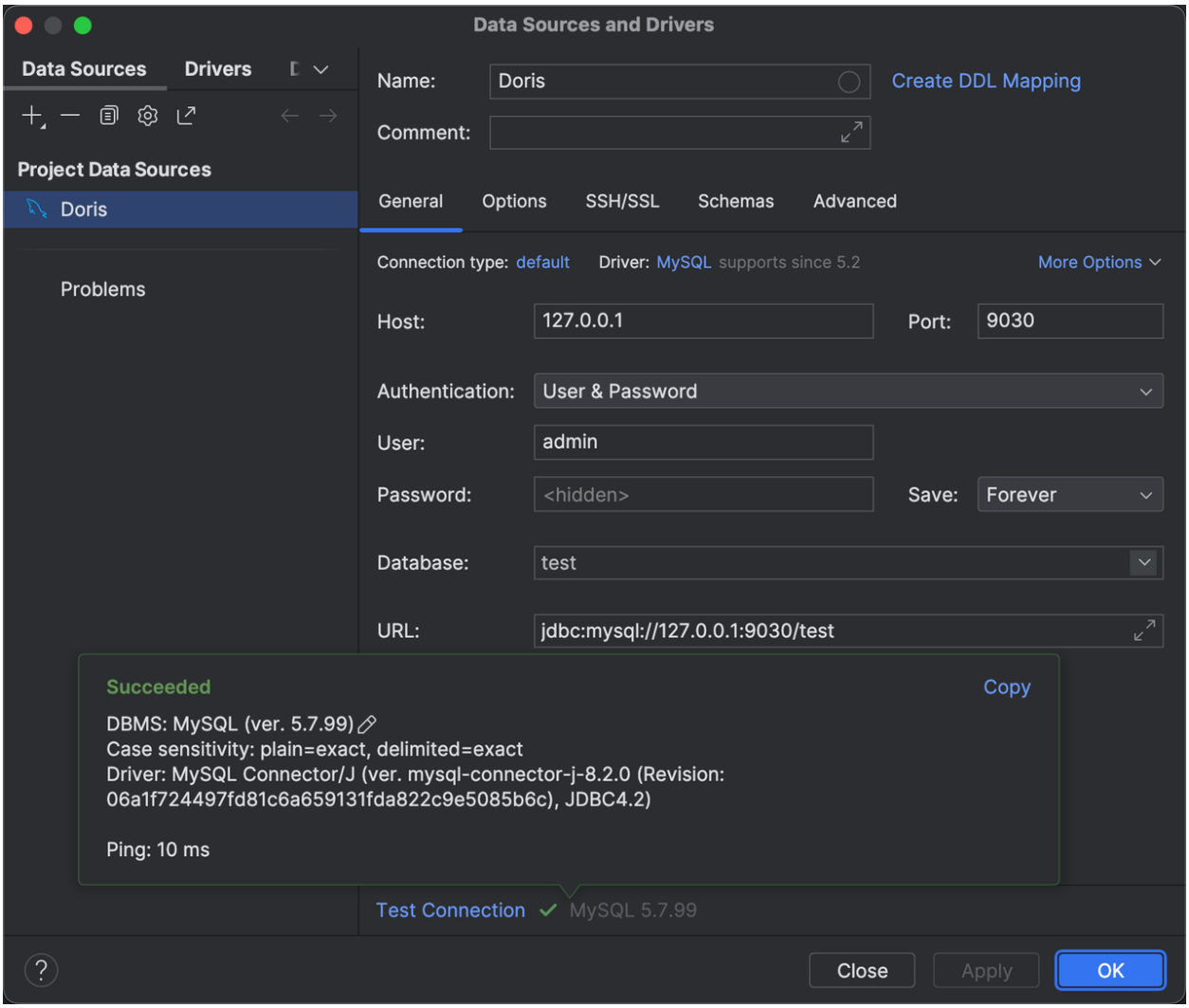


图 244: 测试连接

6. 连接数据库

   数据库连接建立以后，可以在左侧的数据库连接导航看到已创建的数据源连接，并且可以通过 DataGrip 连接并管理数据库。

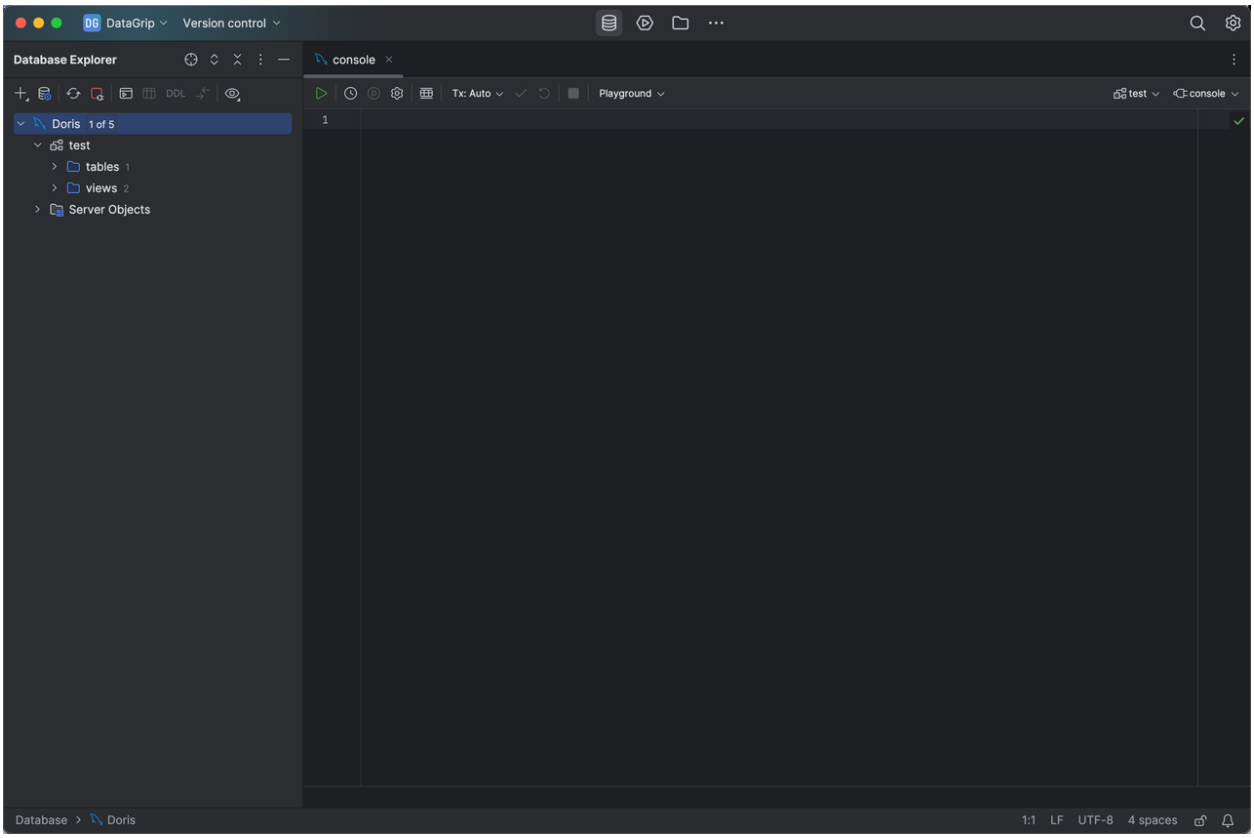


图 245: 建立连接

5.7.3.4 功能支持

基本支持大部分可视化查看操作，以及 SQL 控制台编写 SQL 操作 Doris，不支持或未经验证各种创建库表、schema change、增删改数据操作。

5.8 可观测性

5.8.1 Logstash

5.8.1.1 介绍

Logstash 是一个日志 ETL 框架（采集，预处理，发送到存储系统），它支持自定义输出插件将数据写入存储系统，Logstash Doris output plugin 是输出到 Doris 的插件。

Logstash Doris output plugin 调用Doris Stream Load HTTP 接口将数据实时写入 Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Logstash Doris output plugin 主要有三个步骤：1. 将插件安装到 Logstash 中 2. 配置 Doris 输出地址和其他参数 3. 启动 Logstash 将数据实时写入 Doris

5.8.1.2 安装

5.8.1.2.1 获取插件

可以从官网下载或者自行从源码编译 Logstash Doris output plugin。

- 从官网下载
- 不包含依赖的安装包 https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/logstash-output-doris-1.2.0.gem
- 包含依赖的安装包 https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/logstash-output-doris-1.2.0.zip
- 从源码编译

```
cd extension/logstash/

gem build logstash-output-doris.gemspec
```

5.8.1.2.2 安装插件

- 普通安装

${LOGSTASH_HOME} 是 Logstash 的安装目录，运行它下面的 bin/logstash-plugin 命令安装插件

```
${LOGSTASH_HOME}/bin/logstash-plugin install logstash-output-doris-1.2.0.gem

Validating logstash-output-doris-1.2.0.gem
Installing logstash-output-doris
Installation successful
```

普通安装模式会自动安装插件依赖的 ruby 模块，对于网络不通的情况会卡住无法完成，这种情况下可以下载包含依赖的 zip 安装包进行完全离线安装，注意需要用 file:// 指定本地文件系统。

- 离线安装

```
${LOGSTASH_HOME}/bin/logstash-plugin install file:///tmp/logstash-output-doris-1.2.0.zip

Installing file: logstash-output-doris-1.2.0.zip
Resolving dependencies.........................
Install successful
```

5.8.1.3 参数配置

Logstash Doris output plugin 的配置如下：

配置	说明
http ↪ _ ↪ hosts ↪	Stream Load HTTP 地址，格式是字符串数组，可以有一个或者多个元素，每个元素是 host:port。例如：[“http://fe1:8030”, “http://fe2:8030”]
user	Doris 用户名，该用户需要有 doris 对应库表的导入权限
password ↪	Doris 用户的密码
db	要写入的 Doris 库名

配置	说明
table ↪	要写入的 Doris 表名
label ↪ _ ↪ prefix ↪	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}{db}{table}{yyyy，默认值是 logstash

headers
↪
	Doris Stream Load 的 headers 参数，语法格式为 ruby map，例如：
	headers => {
	“format” => “json”
	“read_json_by_line” => “true”
	}

配置	说明
mapping ↪	Logstash 字段到 Doris 表字段的映射，参考后续章节的使用示例
message ↪ _ ↪ only ↪	一种特殊的 mapping 形式，只将 Logstash 的 @message 字段输出到 Doris，默认为 false

配置	说明
max_ ↪ retries ↪	Doris Stream Load 请求失败重试次数，默认为 -1 无限重试保证数据可靠性
log_ ↪ request ↪	日志中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 false

配置	说明
log_ ↪ speed ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

5.8.1.4 使用示例

5.8.1.4.1 TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. Logstash 配置

Logstash 主要有两类配置文件，一类是整个 Logstash 的配置文件，另一类是某个日志采集的配置文件。

整个 Logstash 的配置文件通常在 config/logstash.yml，为了提升写入 Doris 的性能需要修改 batch 大小和攒批时间，对于平均每条 i 几百字节的日志，推荐 100 万行和 10s。

```
pipeline.batch.size: 1000000
pipeline.batch.delay: 10000
```

某个日志采集的配置文件如 logstash_doris_log.conf 主要由 3 部分组成，分别对应 ETL 的各个部分：1. input 负责

读取原始数据 2. filter 负责做数据转换 3. output 负责将数据输出

```
### 1. input 负责读取原始数据
### file input 是一个 input plugin，可以配置读取的日志文件路径，通过 multiline codec
    ↪ 将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。file input
    ↪ 会将日志内容保存在 @message 字段中，另外还有一些元数据字段比如 host，log.file.path，
    ↪ 这里我们还通过 add_field 手动添加了一个字段 type，它的值为 fe.log 。
input {
    file {
        path => "/mnt/disk2/xiaokang/opt/doris_master/fe/log/fe.log"
        add_field => {"type" => "fe.log"}
        codec => multiline {
            # valid line starts with timestamp
            pattern => "^%{TIMESTAMP_ISO8601} "
            # any line not starting with a timestamp should be merged with the previous line
            negate => true
            what => "previous"
        }
    }
}

### 2. filter 部分负责数据转换
### grok 是一个常用的数据转换插件，它内置了一些常见的pattern 比如 TIMESTAMP_ISO8601 解析时间戳，
    ↪ 还支持写正则表达式提取字段。
filter {
    grok {
        match => {
            # parse log_time, level, thread, position fields from message
            "message" => "%{TIMESTAMP_ISO8601:log_time} (?<level>[A-Z]+) \((?<thread>[^\[]*)\)
                ↪ \[(?<position>[^\]]*)\]"
        }
    }
}

### 3. output 部分负责数据输出
### doris output 将数据输出到 Doris，使用的是 Stream Load HTTP 接口。通过 headers 参数指定了
    ↪ Stream Load 的数据格式为 JSON，通过 mapping 参数指定 Logstash 字段到 JSON 字段的映射。
    ↪ 由于 headers 指定了 "format" => "json"，Stream Load 会自动解析 JSON 字段写入对应的 Doris
    ↪ 表的字段。
output {
    doris {
        http_hosts => ["http://localhost:8630"]
        user => "root"
        password => ""
        db => "log_db"
```

```
        table => "doris_log"
        headers => {
          "format" => "json"
          "read_json_by_line" => "true"
          "load_to_single_tablet" => "true"
        }
        mapping => {
          "log_time" => "%{log_time}"
          "collect_time" => "%{@timestamp}"
          "host" => "%{[host][name]}"
          "path" => "%{[log][file][path]}"
          "type" => "%{type}"
          "level" => "%{level}"
          "thread" => "%{thread}"
          "position" => "%{position}"
          "message" => "%{message}"
        }
        log_request => true
    }
}
```

4. 运行 Logstash

```
${LOGSTASH_HOME}/bin/logstash -f config/logstash_doris_log.conf

### log_request 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

[2024-07-08T22:35:34,772][INFO ][logstash.outputs.doris   ][main][
    ↪ e44d2a24f17d764647ce56f5fed24b9bbf08d3020c7fddcc3298800daface80a] doris stream load
    ↪ response:
{
    "TxnId": 45464,
    "Label": "logstash_log_db_doris_log_20240708_223532_539_6c20a0d1-dcab-4b8e-9bc0-76b46a929bd1
        ↪ ",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 452,
    "NumberLoadedRows": 452,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 277230,
    "LoadTimeMs": 1797,
    "BeginTxnTimeMs": 0,
```

```
    "StreamLoadPutTimeMs": 18,
    "ReadDataTimeMs": 9,
    "WriteDataTimeMs": 1758,
    "CommitAndPublishTimeMs": 18
}

### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
[2024-07-08T22:35:38,285][INFO ][logstash.outputs.doris   ][main] total 11 MB 18978 ROWS, total
    ↪ speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

5.8.1.4.2 JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
```

```
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. Doris 建表

```
CREATE DATABASE log_db;
USE log_db;


CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor.id` BIGINT,
  `actor.login` TEXT,
  `actor.display_login` TEXT,
  `actor.gravatar_id` TEXT,
  `actor.url` TEXT,
  `actor.avatar_url` TEXT,
  `repo.id` BIGINT,
  `repo.name` TEXT,
  `repo.url` TEXT,
  `payload` TEXT,
  `host` TEXT,
  `path` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor.id` (`actor.id`) USING INVERTED,
  INDEX `idx_actor.login` (`actor.login`) USING INVERTED,
  INDEX `idx_repo.id` (`repo.id`) USING INVERTED,
  INDEX `idx_repo.name` (`repo.name`) USING INVERTED,
  INDEX `idx_host` (`host`) USING INVERTED,
  INDEX `idx_path` (`path`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
```

```
"replication_num" = "1",
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. Logstash 配置

这个配置文件和之前 TEXT 日志采集不同的有下面几点：

1. file input 的 codec 参数是 json，Logstash 会将每一行文本当作 JSON 格式解析，解析出来的字段用于后续处理
2. 没有用 filter plugin，因为不需要额外的处理转换

```
input {
    file {
        path => "/tmp/github_events/2024-04-01-23.json"
        codec => json
    }
}

output {
    doris {
        http_hosts => ["http://fe1:8630", "http://fe2:8630", "http://fe3:8630"]
        user => "root"
        password => ""
        db => "log_db"
        table => "github_events"
        headers => {
          "format" => "json"
          "read_json_by_line" => "true"
          "load_to_single_tablet" => "true"
        }
        mapping => {
          "created_at" => "%{created_at}"
          "id" => "%{id}"
          "type" => "%{type}"
          "public" => "%{public}"
          "actor.id" => "%{[actor][id]}"
```

```
          "actor.login" => "%{[actor][login]}"
          "actor.display_login" => "%{[actor][display_login]}"
          "actor.gravatar_id" => "%{[actor][gravatar_id]}"
          "actor.url" => "%{[actor][url]}"
          "actor.avatar_url" => "%{[actor][avatar_url]}"
          "repo.id" => "%{[repo][id]}"
          "repo.name" => "%{[repo][name]}"
          "repo.url" => "%{[repo][url]}"
          "payload" => "%{[payload]}"
          "host" => "%{[host][name]}"
          "path" => "%{[log][file][path]}"
        }
        log_request => true
    }
}
```

4. 运行 Logstash

```
${LOGSTASH_HOME}/bin/logstash -f logstash_github_events.conf
```

5.8.2 Filebeat

Beats 是一个数据采集 Agent，它支持自定义输出插件将数据写入存储系统，Beats Doris output plugin 是输出到 Doris 的插件。

Beats Doris output plugin 支持 Filebeat, Metricbeat, Packetbeat, Winlogbeat, Auditbeat, Heartbeat 。

Beats Doris output plugin 调用Doris Stream Load HTTP 接口将数据实时写入 Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Beats Doris output plugin 主要有三个步骤：1. 下载或编译包含 Doris output plugin 的 Beats 二进制程序 2. 配置 Beats 输出地址和其他参数 3. 启动 Beats 将数据实时写入 Doris

5.8.2.1 安装

5.8.2.1.1 从官网下载

https://apache-doris-releases.oss-accelerate.aliyuncs.com/extension/filebeat-doris-2.1.1

5.8.2.1.2 从源码编译

在 extension/beats/ 目录下执行

```
cd doris/extension/beats

go build -o filebeat-doris filebeat/filebeat.go
go build -o metricbeat-doris metricbeat/metricbeat.go
```

```
go build -o winlogbeat-doris winlogbeat/winlogbeat.go
go build -o packetbeat-doris packetbeat/packetbeat.go
go build -o auditbeat-doris auditbeat/auditbeat.go
go build -o heartbeat-doris heartbeat/heartbeat.go
```

5.8.2.2 参数配置

Beats Doris output plugin 的配置如下：

配置	说明
http ↪ _ ↪ hosts ↪	Stream Load HTTP 地址，格式是字符串数组，可以有一个或者多个元素，每个元素是 host:port。例如：[“http://fe1:8030”, “http://fe2:8030”]
user	Doris 用户名，该用户需要有 doris 对应库表的导入权限

配置	说明
password	Doris 用户的密码
database	要写入的 Doris 库名
table	要写入的 Doris 表名
label_prefix	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}{db}{table}{yyyy，默认值是 beats
headers	Doris Stream Load 的 headers 参数，语法格式为 YAML map

配置	说明
codec ↪ _ ↪ format ↪ _ ↪ string ↪	输出到 Doris Stream Load 的 format string，%{[a][b]} 代表输入中的 a.b 字段，参考后续章节的使用示例
bulk ↪ _ ↪ max ↪ _ ↪ size ↪	Doris Stream Load 的 batch size，默认为 100000

配置	说明
max_ ↪ retries ↪	Doris Stream Load 请求失败重试次数，默认为 -1 无限重试保证数据可靠性
log_ ↪ request ↪	日志中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 true

配置	说明
log_ ↪ progress ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

5.8.2.3 使用示例

5.8.2.3.1 TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. 配置

filebeat 日志采集的配置文件如 filebeat_doris_log.yml 是 YAML 格式，主要由 4 部分组成，分别对应 ETL 的各个部分：1. input 负责读取原始数据 2. processor 负责做数据转换 3. queue.mem 配置 filebeat 内部的缓冲队列 4. output 负责将数据输出

```
### 1. input 负责读取原始数据
### type: log 是一个 log input plugin，可以配置读取的日志文件路径，通过 multiline
    ↪ 功能将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。log input
    ↪ 会将日志内容保存在 message 字段中，另外还有一些元数据字段比如 agent.host，log.file.path。
filebeat.inputs:
```

```
- type: log
  enabled: true
  paths:
    - /path/to/your/log
  # multiline 可以将跨行的日志（比如Java stacktrace）拼接起来
  multiline:
    type: pattern
    # 效果：以 yyyy-mm-dd HH:MM:SS 开头的行认为是一条新的日志，其他都拼接到上一条日志
    pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}'
    negate: true
    match: after
    skip_newline: true

### 2. processors 部分负责数据转换
processors:
### 用 js script 插件将日志中的 `\t` 替换成空格，避免JSON解析报错
- script:
    lang: javascript
    source: >
        function process(event) {
            var msg = event.Get("message");
            msg = msg.replace(/`\t`/g, "  ");
            event.Put("message", msg);
        }
### 用 dissect 插件做简单的日志解析
- dissect:
    # 2024-06-08 18:26:25,481 INFO (report-thread|199) [ReportHandler.cpuReport():617] begin to
        ↪ handle
    tokenizer: "%{day} %{time} %{log_level} (%{thread}) [%{position}] %{content}"
    target_prefix: ""
    ignore_failure: true
    overwrite_keys: true

### 3. 内部的缓冲队列总条数，flush batch 条数，flush 时间间隔
queue.mem:
  events: 1000000
  flush.min_events: 100000
  flush.timeout: 10s

### 4. output 部分负责数据输出
### doris output 将数据输出到 Doris，使用的是 Stream Load HTTP 接口。通过 headers 参数指定了
    ↪ Stream Load 的数据格式为 JSON，通过 codec_format_string 参数用类似 printf
    ↪ 的方式格式化输出到 Doris 的数据。比如下面的例子基于 filebeat 内部的字段 format 出一个
    ↪ JSON，这些字段可以是 filebeat 内置字段如 agent.hostname，也可以是 processor 比如 dissect
    ↪ 生产的字段如 day，通过 %{[a][b]} 的方式引用，，Stream Load 会自动将 JSON 字段写入对应的
```

```
    ↪ Doris 表的字段。
output.doris:
  fenodes: [ "http://fehost1:http_port", "http://fehost2:http_port", "http://fehost3:http_port" ]
  user: "your_username"
  password: "your_password"
  database: "your_db"
  table: "your_table"
  # output string format
  ## %{[agent][hostname]} %{[log][file][path]} 是filebeat自带的metadata
  ## 常用的 filebeat metadata 还是有采集时间戳 %{[@timestamp]}
  ## %{[day]} %{[time]} 是上面 dissect 解析得到字段
  codec_format_string: '{"ts": "%{[day]} %{[time]}", "host": "%{[agent][hostname]}", "path": "%{[
      ↪ log][file][path]}", "message": "%{[message]}"}'
  headers:
    format: "json"
    read_json_by_line: "true"
    load_to_single_tablet: "true"
```

4. 运行 Filebeat

```
./filebeat-doris -f config/filebeat_doris_log.yml

### log_request 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

doris stream load response:
{
    "TxnId": 45464,
    "Label": "logstash_log_db_doris_log_20240708_223532_539_6c20a0d1-dcab-4b8e-9bc0-76b46a929bd1
        ↪ ",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 452,
    "NumberLoadedRows": 452,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 277230,
    "LoadTimeMs": 1797,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 18,
    "ReadDataTimeMs": 9,
    "WriteDataTimeMs": 1758,
    "CommitAndPublishTimeMs": 18
}
```

```
### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
total 11 MB 18978 ROWS, total speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

5.8.2.3.2 JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. Doris 建表

```
CREATE DATABASE log_db;
USE log_db;


CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor` VARIANT,
  `repo` VARIANT,
  `payload` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor` (`actor`) USING INVERTED,
  INDEX `idx_host` (`repo`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. Filebeat 配置

这个配置文件和之前 TEXT 日志采集不同的有下面几点：

1. 没有用 processors，因为不需要额外的处理转换
2. output 中的 codec_format_string 很简单，直接输出整个 message，也就是原始内容

```
### input
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /path/to/your/log

### queue and batch
queue.mem:
  events: 1000000
  flush.min_events: 100000
  flush.timeout: 10s

### output
output.doris:
  fenodes: [ "http://fehost1:http_port", "http://fehost2:http_port", "http://fehost3:http_port" ]
  user: "your_username"
  password: "your_password"
  database: "your_db"
  table: "your_table"
  # output string format
  ## 直接把原始文件每一行的 message 原样输出，由于 headers 指定了 format: "json"，Stream Load
      ↪ 会自动解析 JSON 字段写入对应的 Doris 表的字段。
  codec_format_string: '%{[message]}'
  headers:
    format: "json"
    read_json_by_line: "true"
    load_to_single_tablet: "true"
```

4. 运行 Filebeat

```
./filebeat-doris -f config/filebeat_github_events.yml
```

5.8.3 OpenTelemetry

5.8.3.1 介绍

OpenTelemetry（简称 OTel），是一个中立厂商的开源可观测性框架，用于监测、生成、收集和导出日志、调用链追踪和指标等可观测性数据。OpenTelemetry 定义了一套可观测性的标准和协议，被可观测性社区和厂商广泛采纳，逐渐成为可观测性领域的事实标准。

OpenTelemetry 本身实现了框架和可观测性数据采集 SDK，使应用程序和系统易于进行监测，无论使用何种编程语言、基础设施和运行时环境，而可观测性存储后端和可视化前端则留给其他工具来处理。Doris 作为一个存储后端与 OpenTelemetry 集成，提供高性能、低成本、统一的可观测性数据存储和分析能力，整体架构如下。

5.8.3.2 安装

从 OpenTelemetry 官方 Release 页面下载 OpenTelemetry Collector Contrib 安装包，例如 https://github.com/open-telemetry/opentelemetry-collector-releases/releases/download/v0.132.2/otelcol-contrib_0.132.2_linux_amd64.tar.gz，安装包解压缩得到 otelcol-contrib 可执行文件。

5.8.3.3 参数配置

OpenTelemetry Collector Doris Exporter 的核心配置如下：

配置	说明
`endpoint`	Doris FE HTTP 地址，格式是 host:port，例如：“127.0.0.1:8030”
`mysql_endpoint`	Doris FE MySQL 地址，格式是 host:port，例如：“127.0.0.1:9030”
`username`	Doris 用户名，该用户需要有对应库表的写入权限
`password`	Doris 用户的密码

配置	说明
database ↪	要写入的 Doris 库名
table ↪ . ↪ logs ↪	logs 数据写入的 Doris 表名，默认值是 otel_logs
table ↪ . ↪ traces ↪	traces 数据写入的 Doris 表名，默认值是 otel_traces
table ↪ . ↪ metrics ↪	metrics 数据写入的 Doris 表名，默认值是 otel_metrics
create ↪ _ ↪ schema ↪	是否自动创建 Doris 库表，默认值是 true

配置	说明
history ↪ _ ↪ days ↪	自动创建的 Doris 表的历史数据保留天数，默认值是 0 表示永久保留
create ↪ _ ↪ history ↪ _ ↪ days ↪	自动创建的 Doris 表的初始分区天数，默认值是 0 表示不创建分区

配置	说明
label_prefix	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}{db}{table}{yyyy，默认值是 open_telemetry
headers	Doris Stream Load 的 headers 参数，语法格式为 YAML map

配置	说明
log_ ↪ progress ↪ _ ↪ interval ↪	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

更多配置请参考 https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/exporter/dorisexporter 。

5.8.3.4 使用示例

5.8.3.4.1 TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 fe/log/fe.log 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 stacktrace 的异常日志，stacktrace 是跨行的，日志采集存储需要把主日志和 stacktrace 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. OpenTelemetry 配置

日志采集的配置文件如 opentelemetry_java_log.yml 主要由 3 部分组成，分别对应 ETL 的各个部分：1. receivers 负责读取原始数据 2. processors 负责做数据转换 3. exporters 负责将数据输出

```
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET
/filter/27|13%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,27|%DA%A9%D9%85%D8%AA%D8%B1%20%
    ↪ D8%A7%D8%B2%205%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,p53 HTTP/1.1" 200
    ↪ 30577 "-" "Mozilla/5.0 (compatible; AhrefsBot/6.1; +http://ahrefs.com/robot/)" "-"
```

```
### 1. receivers 负责读取原始数据
### filelog 是一个本地 receiver，可以配置读取本地文件系统的日志文件路径，通过 multiline
    ↪ 将非时间开头的行拼接到上一行后面，实现 stacktrace 和主日志合并的效果。
receivers:
  filelog:
    include:
      - /path/to/fe.log
    start_at: beginning
    multiline:
      line_start_pattern: '^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3}'  #
          ↪ 匹配时间戳作为新日志开始
    operators:
      # 解析日志
      - type: regex_parser
        regex: '^(?P<time>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3}) (?P<severity>INFO|WARN|ERROR
            ↪ ) (?P<message>.*)'
        timestamp:
          parse_from: attributes.time
          layout: '%Y-%m-%d %H:%M:%S,%f'
        severity:
          parse_from: attributes.severity
          trace: TRACE
          debug: DEBUG
          info: INFO
          warn: WARN
          error: ERROR
          fatal: FATAL

### 2. processors 负责做数据转换
### 这里使用了简单的 batch processor，将数据攒成批次发送。
processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

### 3. exporters 负责将数据输出
```

```
### doris exporter 将数据输出到 Doris，使用的是 Stream Load HTTP 接口，默认使用 Stream Load
    ↪ 的数据格式为 JSON，Stream Load 会自动解析 JSON 字段写入对应的 Doris 表的字段。
exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      logs: otel_logs
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    logs:
      receivers: [filelog]
      processors: [batch]
      exporters: [doris]
```

3. 运行 OpenTelemetry

```
./otelcol-contrib --config config/opentelemetry_java_log.yml

### log_response 为 true 时日志会输出每次 Stream Load 的请求参数和响应结果

2025-08-18T00:33:22.543+0800    info    dorisexporter@v0.132.0/exporter_logs.go:181 log response:
{
    "TxnId": 52,
    "Label": "open_telemetry_otel_otel_logs_20250818003321_498bb8ec-040c-4982-9eb4-452b15129782",
    "Comment": "",
```

```
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 50355,
    "NumberLoadedRows": 50355,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 31130235,
    "LoadTimeMs": 680,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 3,
    "ReadDataTimeMs": 106,
    "WriteDataTimeMs": 653,
    "ReceiveDataTimeMs": 11,
    "CommitAndPublishTimeMs": 23
}

### 默认每隔 10s 会日志输出速度信息，包括自启动以来的数据量（MB 和 ROWS），总速度（MB/s 和 R/S
    ↪ ），最近 10s 速度
2025-08-18T00:05:00.017+0800    info    dorisexporter@v0.132.0/progress_reporter.go:63  [LOG]
    ↪ total 11 MB 18978 ROWS, total speed 0 MB/s 632 R/s, last 10 seconds speed 1 MB/s 1897 R/s
```

5.8.3.4.2 JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
```

```
    "id": 780125623,
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. OpenTelemetry 配置

这个配置文件和之前 TEXT 日志采集不同的是 filelog 的 type 参数是 json_parser，它会将每一行文本当作 JSON 格式解析，解析出来的字段用于后续处理。

```
receivers:
  filelog:
    include:
      - /path/to/2024-01-01-15.json
    start_at: beginning
    operators:
      - type: json_parser
        timestamp:
          parse_from: attributes.created_at
          layout: '%Y-%m-%dT%H:%M:%SZ'

processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
```

```
      logs: otel_logs
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    logs:
      receivers: [filelog]
      processors: [batch]
      exporters: [doris]
```

3. 运行 OpenTelemetry

```
./otelcol-contrib --config config/opentelemetry_json_log.yml
```

5.8.3.4.3 Trace 采集示例

1. OpenTelemetry 配置

创建 otel_trace.yml 配置文件如下

```
receivers:
  otlp: # otlp 协议，接收 OpenTelemetry Java Agent 发送的数据
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    send_batch_size: 100000 # 每个批次的数据条数，建议 batch 的数据量在 100M-1G 之间
```

```
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030 # FE HTTP 地址
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      traces: doris_table_name
    create_schema: true # 是否自动创建 schema，如果设置为 false，则需要手动建表
    mysql_endpoint: localhost:9030  # FE MySQL 地址
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s # http stream load 客户端超时时间
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
      queue_size: 1000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
    headers:
      load_to_single_tablet: "true"

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [doris]
```

2. 运行 OpenTelemetry

```
./otelcol-contrib --config otel_trace.yaml
```

3. 应用侧接入 OpenTelemetry SDK

这里我们使用一个 Spring Boot 示例应用接入 OpenTelemetry Java SDK，示例应用来自官方 demo，对路径 “/” 返回简单的 “Hello World!” 字符串。下载 OpenTelemetry Java Agent，使用 Java Agent 的优势在于无需对现有的应用做任何的修改。其他语言及其他接入方式详见 OpenTelemetry 官网：Language APIs & SDKs 或 Zero-code Instrumentation。

在启动应用之前只需要添加几个环境变量，无需修改任何代码。

```
export JAVA_TOOL_OPTIONS="${JAVA_TOOL_OPTIONS} -javaagent:/your/path/to/opentelemetry-javaagent.
    ↪ jar" # OpenTelemetry Java Agent 的路径
export OTEL_JAVAAGENT_LOGGING="none" # 禁用 otel log，防止干扰服务本身的日志
export OTEL_SERVICE_NAME="myproject"
export OTEL_TRACES_EXPORTER="otlp" # 使用 otlp 协议发送 trace 数据
export OTEL_EXPORTER_OTLP_ENDPOINT="http://localhost:4317" # OpenTelemetry Collector 的地址

java -jar myproject-0.0.1-SNAPSHOT.jar
```

5.8.3.4.4 Trace Collection Example

1. OpenTelemetry Configuration

Create the configuration file `otel_trace.yml` as follows:

```
receivers:
  otlp:  # OTLP protocol to receive data from OpenTelemetry Java Agent
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    send_batch_size: 100000  # Number of traces per batch (recommended batch size: 100MB-1GB)
    timeout: 10s

exporters:
  doris:
    endpoint: http://localhost:8030  # FE HTTP address
    database: doris_db_name
    username: doris_username
    password: doris_password
    table:
      traces: doris_table_name
    create_schema: true  # Auto-create schema (if false, tables must be created manually)
    mysql_endpoint: localhost:9030  # FE MySQL address
    history_days: 10
    create_history_days: 10
    timezone: Asia/Shanghai
    timeout: 60s  # HTTP Stream Load client timeout
    log_response: true
    sending_queue:
      enabled: true
      num_consumers: 20
```

```
    queue_size: 1000
  retry_on_failure:
    enabled: true
    initial_interval: 5s
    max_interval: 30s
  headers:
    load_to_single_tablet: "true"

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [doris]
```

2. Run OpenTelemetry

```
./otelcol-contrib --config otel_trace.yaml
```

3. Application Integration with OpenTelemetry SDK

Here we demonstrate integration using a Spring Boot example application (from the official demo), which returns “Hello World!” for the “/” endpoint.

1. Download the OpenTelemetry Java Agent.

   - Advantage: No code changes required for existing applications.
   - For other languages/integration methods, see OpenTelemetry docs:
     - Language APIs & SDKs
     - Zero-code Instrumentation

2. Before starting the application, set these environment variables (no code modification needed):
   “ ‘bash export JAVA_TOOL_OPTIONS=“${JAVA_TOOL_OPTIONS} -javaagent:/your/path/to/opentelemetry-javaagent.jar” # Path to OpenTelemetry Java Agent export OTEL_JAVAAGENT_LOGGING=“none” # Disable OTEL logs to avoid interference with application logs export OTEL_SERVICE_NAME=“myproject” export OTEL_TRACES_EXPORTER=“otlp” # Use OTLP protocol to send trace data export OTEL_EXPORTER_OTLP_ENDPOINT=“http://localhost:4317” # OpenTelemetry Collector address

java -jar myproject-0.0.1-SNAPSHOT.jar

```
---
```

```
{
"title": "FluentBit",
"language": "zh-CN"
}
---

### FluentBit
[Fluent Bit](https://fluentbit.io/) 是一个快速的日志处理器和转发器，
    ↪ 它支持自定义输出插件将数据写入存储系统，Fluent Bit Doris Output Plugin 是输出到 Doris
    ↪ 的插件。

Fluent Bit Doris Output Plugin 调用 [Doris Stream Load](#stream-load) HTTP 接口将数据实时写入
    ↪ Doris，提供多线程并发，失败重试，自定义 Stream Load 格式和参数，输出写入速度等能力。

使用 Fluent Bit Doris Output Plugin 主要有三个步骤：
1. 下载或编译包含 Doris Output Plugin 的 Fluent Bit 二进制程序
2. 配置 Fluent Bit 输出地址和其他参数
3. 启动 Fluent Bit 将数据实时写入 Doris

#### 安装（alpha 版本）

##### 从官网下载

https://apache-doris-releases.oss-accelerate.aliyuncs.com/integrations/fluent-bit-doris-3.1.9

##### 从源码编译

克隆 https://github.com/joker-star-l/fluent-bit 的 dev 分支，在 build/ 目录下执行
```

cmake -DFLB_RELEASE=ON .. make “ ‘

编译产物为 build/bin/fluent-bit。

5.8.3.5 参数配置

Fluent Bit Doris output plugin 的配置如下：

配置	说明
host	Stream Load HTTP host
port	Stream Load HTTP port

配置	说明
user	Doris 用户名，该用户需要有 doris 对应库表的导入权限
password	Doris 用户的密码
database	要写入的 Doris 库名
table	要写入的 Doris 表名

配置	说明
label_prefix	Doris Stream Load Label 前缀，最终生成的 Label 为 {label_prefix}_{timestamp}_{u，默认值是 fluentbit，如果设置为 false 则不会添加 Label
time_key	数据中要添加的时间戳列的名称，默认值是 date，如果设置为 false 则不会添加该列

配置	说明
header	Doris Stream Load 的 header 参数，可以设置多个
log_request	日志中是否输出 Doris Stream Load 请求和响应元数据，用于排查问题，默认为 true
log_progress_interval	日志中输出速度的时间间隔，单位是秒，默认为 10，设置为 0 可以关闭这种日志

配置	说明
retry ↪ _ ↪ limit ↪	Doris Stream Load 请求失败重试次数，默认为 1，如果设置为 false 则不限制重试次数
workers ↪	执行 Doris Stream Load 的 worker 数量，默认为 2

5.8.3.6 使用示例

5.8.3.6.1 TEXT 日志采集示例

该示例以 Doris FE 的日志为例展示 TEXT 日志采集。

1. 数据

FE 日志文件一般位于 Doris 安装目录下的 `fe/log/fe.log` 文件，是典型的 Java 程序日志，包括时间戳，日志级别，线程名，代码位置，日志内容等字段。不仅有正常的日志，还有带 `stacktrace` 的异常日志，`stacktrace` 是跨行的，日志采集存储需要把主日志和 `stacktrace` 组合成一条日志。

```
2024-07-08 21:18:01,432 INFO (Statistics Job Appender|61) [StatisticsJobAppender.
    ↪ runAfterCatalogReady():70] Stats table not available, skip
2024-07-08 21:18:53,710 WARN (STATS_FETCH-0|208) [StmtExecutor.executeInternalQuery():3332]
    ↪ Failed to run internal SQL: OriginStatement{originStmt='SELECT * FROM __internal_schema.
    ↪ column_statistics WHERE part_id is NULL  ORDER BY update_time DESC LIMIT 500000', idx=0}
```

```
org.apache.doris.common.UserException: errCode = 2, detailMessage = tablet 10031 has no queryable
    ↪  replicas. err: replica 10032's backend 10008 does not exist or not alive
        at org.apache.doris.planner.OlapScanNode.addScanRangeLocations(OlapScanNode.java:931) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
        at org.apache.doris.planner.OlapScanNode.computeTabletInfo(OlapScanNode.java:1197) ~[
            ↪ doris-fe.jar:1.2-SNAPSHOT]
```

2. 建表

表结构包括日志的产生时间，采集时间，主机名，日志文件路径，日志类型，日志级别，线程名，代码位置，日志内容等字段。

```
CREATE TABLE `doris_log` (
  `log_time` datetime NULL COMMENT 'log content time',
  `collect_time` datetime NULL COMMENT 'log agent collect time',
  `host` text NULL COMMENT 'hostname or ip',
  `path` text NULL COMMENT 'log file path',
  `type` text NULL COMMENT 'log type',
  `level` text NULL COMMENT 'log level',
  `thread` text NULL COMMENT 'log thread',
  `position` text NULL COMMENT 'log code position',
  `message` text NULL COMMENT 'log message',
  INDEX idx_host (`host`) USING INVERTED COMMENT '',
  INDEX idx_path (`path`) USING INVERTED COMMENT '',
  INDEX idx_type (`type`) USING INVERTED COMMENT '',
  INDEX idx_level (`level`) USING INVERTED COMMENT '',
  INDEX idx_thread (`thread`) USING INVERTED COMMENT '',
  INDEX idx_position (`position`) USING INVERTED COMMENT '',
  INDEX idx_message (`message`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase"
      ↪ = "true") COMMENT ''
) ENGINE=OLAP
DUPLICATE KEY(`log_time`)
COMMENT 'OLAP'
PARTITION BY RANGE(`log_time`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-7",
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.create_history_partition" = "true",
"compaction_policy" = "time_series"
);
```

3. 配置

Fluent Bit 日志采集的配置文件如下，`doris_log.conf` 用于定义 ETL 的各个部分组件，`parsers.conf` 用于定义不同的日志解析器。

doris_log.conf:

```
[SERVICE]
    log_level info
    # parsers file
    parsers_file parsers.conf

### use input tail
[INPUT]
    name tail
    path /path/to/your/log
    # add log file name to the record, key is 'path'
    path_key path
    # set multiline parser
    multiline.parser multiline_java

### parse log
[FILTER]
    match *
    name parser
    key_name log
    parser fe_log
    reserve_data true

### add host info
[FILTER]
    name sysinfo
    match *
    # add hostname to the record, key is 'host'
    hostname_key host

### output to doris
[OUTPUT]
    name doris
    match *
    host fehost
    port feport
    user your_username
    password your_password
    database your_db
    table your_table
```

```
    # add 'collect_time' to the record
    time_key collect_time
    # 'collect_time' is timestamp, change it to datatime
    header columns collect_time=from_unixtime(collect_time)
    log_request true
    log_progress_interval 10
```

parsers.conf:

```
[MULTILINE_PARSER]
    name          multiline_java
    type          regex
    flush_timeout 1000
    # Regex rules for multiline parsing
    # ---------------------------------
    #
    # configuration hints:
    #
    #  - first state always has the name: start_state
    #  - every field in the rule must be inside double quotes
    #
    # rules   |   state name   | regex pattern | next state name
    # --------|----------------|---------------|-----------------
    rule         "start_state"   "/(^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2})(.*)/"
        ↪   "cont"
    rule         "cont"          "/(^(?![0-9]{4}-[0-9]{2}-[0-9]{2}))(.*)/"     "cont"


[PARSER]
    name        fe_log
    format      regex
    # parse and add 'log_time', 'level', 'thread', 'position', 'message' to the record
    regex       ^(?<log_time>[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2},[0-9]{3}) (?<
        ↪ level>[^ ]+) \((?<thread>[^\)]+)\) \[(?<position>[^\]]+)\] (?<message>(`\n`|.)*)`\n`$
```

4. 运行 Fluent Bit

```
fluent-bit -c doris_log.conf

### log stream load response

[2024/10/31 18:39:55] [ info] [output:doris:doris.1] 127.0.0.1:8040, HTTP status=200
{
    "TxnId": 32155,
    "Label": "fluentbit_1730371195_91cca1aa-c15f-45d2-b503-fe7d2e839c2a",
    "Comment": "",
```

```
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 1,
    "NumberLoadedRows": 1,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 836,
    "LoadTimeMs": 298,
    "BeginTxnTimeMs": 0,
    "StreamLoadPutTimeMs": 3,
    "ReadDataTimeMs": 0,
    "WriteDataTimeMs": 268,
    "CommitAndPublishTimeMs": 25
}

### log speed info

[2024/10/31 18:40:13] [ info] [output:doris:doris.1] total 0 MB 2 ROWS, total speed 0 MB/s 0 R/s,
    ↪  last 10 seconds speed 0 MB/s 0 R/s
```

5.8.3.6.2 JSON 日志采集示例

该样例以 github events archive 的数据为例展示 JSON 日志采集。

1. 数据

github events archive 是 github 用户操作事件的归档数据，格式是 JSON，可以从 https://www.gharchive.org/ 下载，比如下载 2024 年 1 月 1 日 15 点的数据。

```
wget https://data.gharchive.org/2024-01-01-15.json.gz
```

下面是一条数据样例，实际一条数据一行，这里为了方便展示进行了格式化。

```
{
  "id": "37066529221",
  "type": "PushEvent",
  "actor": {
    "id": 46139131,
    "login": "Bard89",
    "display_login": "Bard89",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Bard89",
    "avatar_url": "https://avatars.githubusercontent.com/u/46139131?"
  },
  "repo": {
    "id": 780125623,
```

```
    "name": "Bard89/talk-to-me",
    "url": "https://api.github.com/repos/Bard89/talk-to-me"
  },
  "payload": {
    "repository_id": 780125623,
    "push_id": 17799451992,
    "size": 1,
    "distinct_size": 1,
    "ref": "refs/heads/add_mvcs",
    "head": "f03baa2de66f88f5f1754ce3fa30972667f87e81",
    "before": "85e6544ede4ae3f132fe2f5f1ce0ce35a3169d21"
  },
  "public": true,
  "created_at": "2024-04-01T23:00:00Z"
}
```

2. 建表

```
CREATE TABLE github_events
(
  `created_at` DATETIME,
  `id` BIGINT,
  `type` TEXT,
  `public` BOOLEAN,
  `actor` VARIANT,
  `repo` VARIANT,
  `payload` TEXT,
  INDEX `idx_id` (`id`) USING INVERTED,
  INDEX `idx_type` (`type`) USING INVERTED,
  INDEX `idx_actor` (`actor`) USING INVERTED,
  INDEX `idx_host` (`repo`) USING INVERTED,
  INDEX `idx_payload` (`payload`) USING INVERTED PROPERTIES("parser" = "unicode", "support_phrase
      ↪ " = "true")
)
ENGINE = OLAP
DUPLICATE KEY(`created_at`)
PARTITION BY RANGE(`created_at`) ()
DISTRIBUTED BY RANDOM BUCKETS 10
PROPERTIES (
"replication_num" = "1",
"compaction_policy" = "time_series",
"enable_single_replica_compaction" = "true",
"dynamic_partition.enable" = "true",
"dynamic_partition.create_history_partition" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-30",
```

```
"dynamic_partition.end" = "1",
"dynamic_partition.prefix" = "p",
"dynamic_partition.buckets" = "10",
"dynamic_partition.replication_num" = "1"
);
```

3. 配置

和之前 TEXT 日志采集相比，该配置文件没有使用 FILTER，因为不需要额外的处理转换。

github_events.conf:

```
[SERVICE]
    log_level info
    parsers_file github_parsers.conf

[INPUT]
    name tail
    parser github
    path /path/to/your/log

[OUTPUT]
    name doris
    match *
    host fehost
    port feport
    user your_username
    password your_password
    database your_db
    table your_table
    time_key false
    log_request true
    log_progress_interval 10
```

github_parsers.conf:

```
[PARSER]
    name github
    format json
```

4. 运行 Fluent Bit

```
fluent-bit -c github_events.conf
```

5.9 More

5.9.1 CloudCanal

CloudCanal 是一款全自研、可视化、自动化的数据迁移、同步工具，支持 30+ 款流行关系型数据库、实时数仓、消息中间件、缓存数据库和搜索引擎之间的数据互通，具备实时高效、精确互联、稳定可拓展、一站式、混合部署、复杂数据转换等优点。

5.9.1.1 功能说明

CloudCanal 提供可视化的界面，可轻松实现数据的结构迁移、全量迁移、增量同步、校验与订正等，此外还可通过设置参数，完成更多精细化、自定义的数据同步配置。目前支持从以下数据源集成数据到 Doris。

源端数据源	结构迁移	全量迁移	增量同步	校验订正
MySQL/MariaDB/AuroraMySQL	支持	支持	支持	支持
Oracle	支持	支持	支持	支持
PostgreSQL/AuroraPostgreSQL	支持	支持	支持	支持
SQL Server	支持	支持	支持	支持
Kafka	不支持	不支持	支持	不支持
AutoMQ	不支持	不支持	支持	不支持
TiDB	支持	支持	支持	支持
Hana	支持	支持	支持	支持
PolarDB-X	支持	支持	支持	支持

> 更多功能及参数设置，请参考 CloudCanal 数据链路说明。

5.9.1.2 下载安装

请参考 全新安装 (Docker Linux/MacOS)，前往 CloudCanal 官网 下载安装私有部署版本。

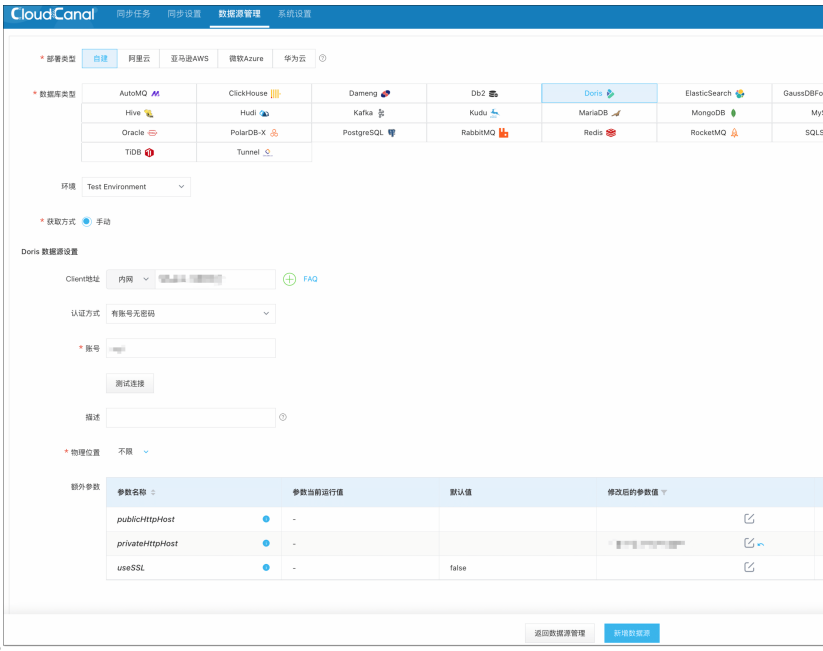
5.9.1.3 使用示例

以下以 MySQL 为例，演示如何实现 MySQL 到 Doris 的数据迁移同步。

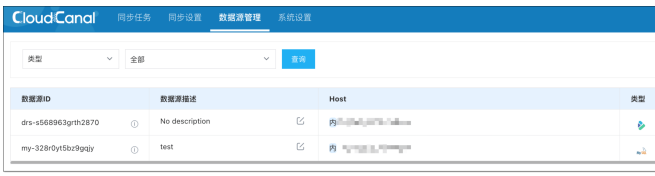
5.9.1.3.1 添加数据源

1. 登录 CloudCanal 控制台，点击 数据源管理 > 新增数据源。

2. 分别选择 MySQL 和 Doris 数据源，并填写相应信息。

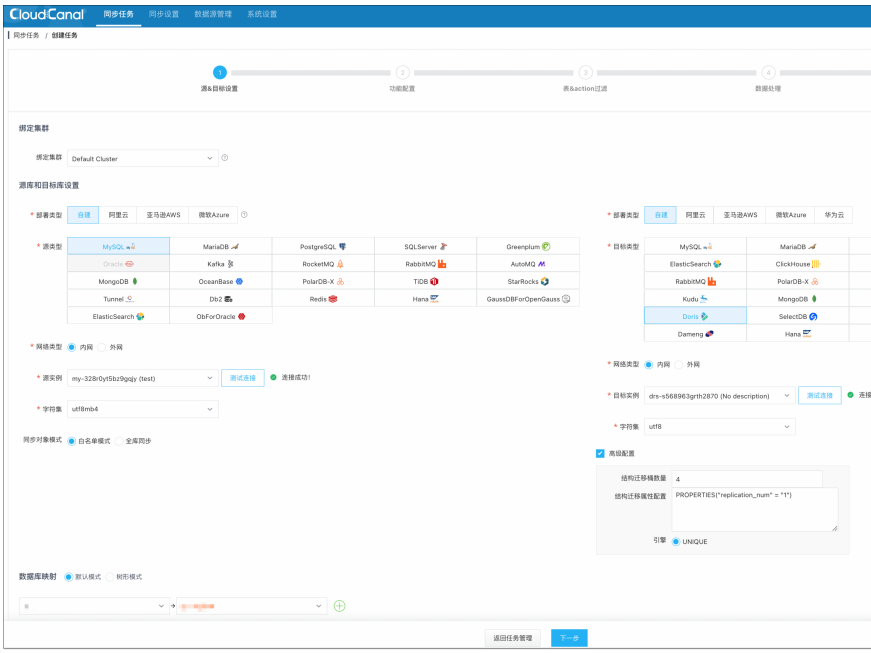


3. 点击 测试连接，连接成功后，点击 新增数据源，完成数据源添加。



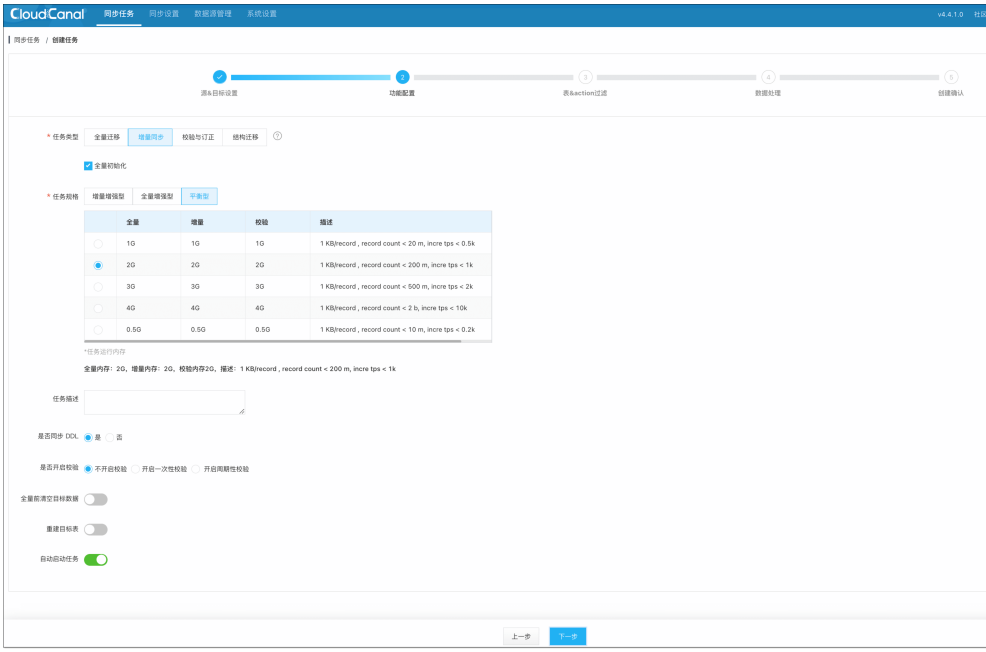
5.9.1.3.2 创建任务

1. 点击 同步任务 > 创建任务。

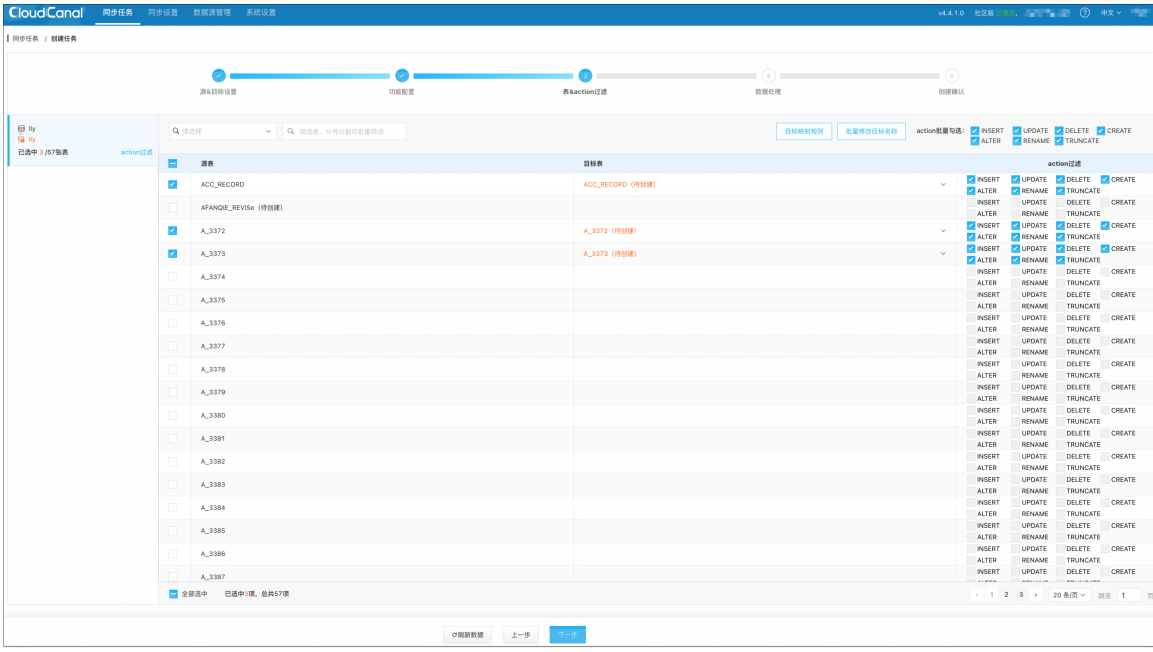


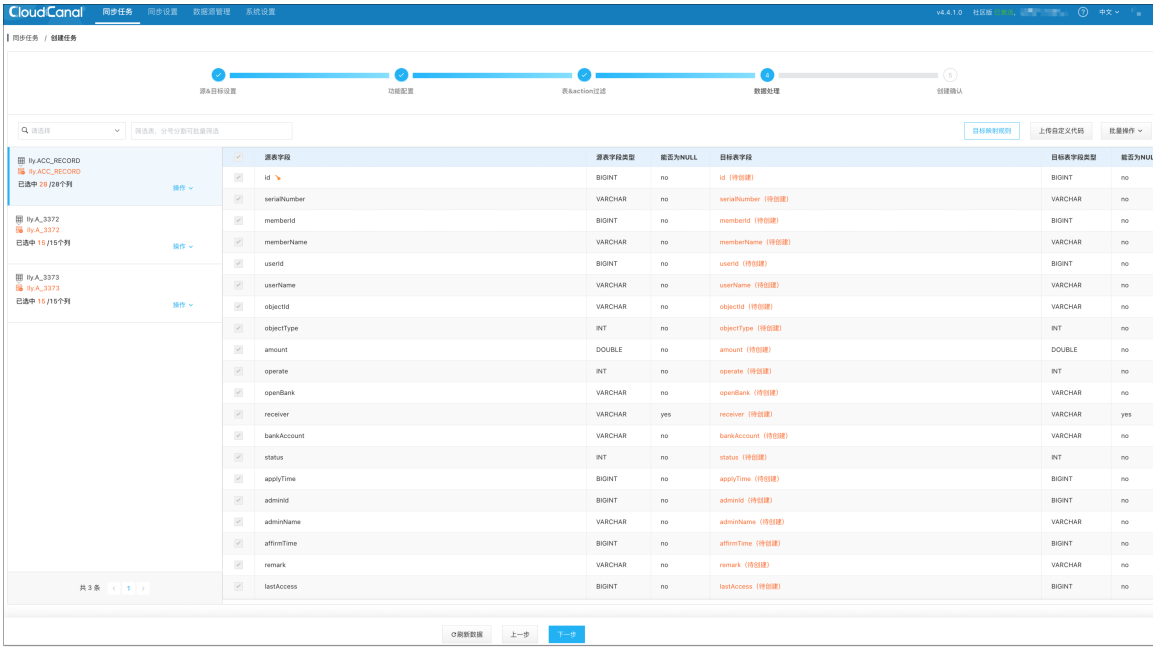
2. 选择源和目标数据源，并分别点击 测试连接。

3. 选择 数据同步并勾选 全量初始化。



4. 选择需要同步的表。





5. 选择需要同步的列。

6. 确认创建任务。

7. 任务自动运行。CloudCanal 会自动进行任务流转，其中的步骤包括：

- 结构迁移：将源端的表结构迁移到对端，如果同名表在对端已存在，则忽略。
- 全量数据迁移：已存在的存量数据将会完整迁移到对端，支持断点续传。
- 增量数据同步：增量数据将会持续地同步到对端数据库，并且保持实时（秒级别延迟）。

5.9.2 DataX Doriswriter

DataX Doriswriter 插件，支持将 MySQL、Oracle、SqlServer 等多种数据源中的数据通过 Stream Load 的方式同步到 Doris 中。

> 注意 1. 需要配合 DataX 服务一起使用。2. DataX 支持多种数据源，可参考这里。

5.9.2.1 使用

5.9.2.1.1 直接下载 DataX 安装包

DataX 官方提供了安装包，已经包含了 DataX 可直接下载使用，可参考这里

5.9.2.1.2 自行编译 DorisWriter 插件

下载 DorisWriter 的插件源码

1. 运行 `init-env.sh`
2. 编译 doriswriter：

> 单独编译 doriswriter 插件：

```
mvn clean install -pl plugin-rdbms-util,doriswriter -DskipTests
```

> 如需编译整个 DataX 项目可参考这里

> 编译错误

如遇到如下编译错误：

```
Could not find artifact com.alibaba.datax:datax-all:pom:0.0.1-SNAPSHOT ...
```

可尝试以下方式解决：

1. 下载 [alibaba-datax-maven-m2-20210928.tar.gz](https://doris-thirdparty-repo.bj.bcebos.com/thirdparty/alibaba-datax-maven-m2-20210928.tar.gz)
2. 解压后，将得到的 `alibaba/datax/` 目录，拷贝到所使用的 maven 对应的 `.m2/repository/com/alibaba/` 下，再次尝试编译。

5.9.2.1.3 Datax DorisWriter 参数介绍：

- jdbcUrl
  - 描述：Doris 的 JDBC 连接串，用户执行 preSql 或 postSQL。
  - 必选：是
  - 默认值：无
- loadUrl
- 描述：作为 Stream Load 的连接目标。格式为 “ip:port”。其中 IP 是 FE 节点 IP，port 是 FE 节点的 http_port。可以填写多个，多个之间使用英文状态的逗号隔开：，，doriswriter 将以轮询的方式访问。
- 必选：是
- 默认值：无

- username
  - 描述：访问 Doris 数据库的用户名
  - 必选：是
  - 默认值：无
- password
  - 描述：访问 Doris 数据库的密码
  - 必选：否
  - 默认值：空
- connection.selectedDatabase
  - 描述：需要写入的 Doris 数据库名称。
  - 必选：是
  - 默认值：无
- connection.table
- 描述：需要写入的 Doris 表名称。
  - 必选：是
  - 默认值：无
- flushInterval
  - 描述：数据写入批次的时间间隔。如果这个时间间隔设置的太小会造成 Doris 写阻塞问题，错误代码 -235，同时如果你这个时间设置太小，maxBatchRows 和 batchSize 参数设置的有很大，那么很可能达不到你这设置的数据量大小，也会执行导入。
  - 必选：否
  - 默认值：30000（ms）
- column
  - 描述：目的表需要写入数据的字段，这些字段将作为生成的 Json 数据的字段名。字段之间用英文逗号分隔。例如：“column”: [“id”，“name”，“age”]。
  - 必选：是
  - 默认值：否
- preSql
- 描述：写入数据到目的表前，会先执行这里的标准语句。
- 必选：否
- 默认值：无
- postSql
- 描述：写入数据到目的表后，会执行这里的标准语句。

- 必选：否
- 默认值：无
- maxBatchRows
- 描述：每批次导入数据的最大行数。和 batchSize 共同控制每批次的导入记录行数。每批次数据达到两个阈值之一，即开始导入这一批次的数据。
- 必选：否
- 默认值：500000
- batchSize
- 描述：每批次导入数据的最大数据量。和 maxBatchRows 共同控制每批次的导入数量。每批次数据达到两个阈值之一，即开始导入这一批次的数据。
- 必选：否
- 默认值：94371840
- maxRetries
- 描述：每批次导入数据失败后的重试次数。
- 必选：否
- 默认值：3
- labelPrefix
- 描述：每批次导入任务的 label 前缀。最终的 label 将有 `labelPrefix + UUID` 组成全局唯一的 label，确保数据不会重复导入
- 必选：否
- 默认值：`datax_doris_writer_`
- loadProps
- 描述：StreamLoad 的请求参数，详情参照 StreamLoad 介绍页面。Stream load

  这里包括导入的数据格式：format 等，导入数据格式默认我们使用 csv，支持 JSON，具体可以参照下面类型转换部分，也可以参照上面 Stream load 官方信息
- 必选：否
- 默认值：无

5.9.2.1.4 示例

1.Stream 读取数据后导入至 Doris

该示例插件的使用说明请参阅 这里

2.Mysql 读取数据后导入至 Doris

1.Mysql 表结构

```
CREATE TABLE `t_test`(
 `id`bigint(30) NOT NULL,
 `order_code` varchar(30) DEFAULT NULL COMMENT '',
 `line_code` varchar(30) DEFAULT NULL COMMENT '',
 `remark` varchar(30) DEFAULT NULL COMMENT '',
 `unit_no` varchar(30) DEFAULT NULL COMMENT '',
 `unit_name` varchar(30) DEFAULT NULL COMMENT '',
 `price` decimal(12,2) DEFAULT NULL COMMENT '',
 PRIMARY KEY(`id`) USING BTREE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 ROW_FORMAT=DYNAMIC COMMENT='';
```

2.Doris 表结构

```
CREATE TABLE `ods_t_test` (
 `id` bigint(30) NOT NULL,
 `order_code` varchar(30) DEFAULT NULL COMMENT '',
 `line_code` varchar(30) DEFAULT NULL COMMENT '',
 `remark` varchar(30) DEFAULT NULL COMMENT '',
 `unit_no` varchar(30) DEFAULT NULL COMMENT '',
 `unit_name` varchar(30) DEFAULT NULL COMMENT '',
 `price` decimal(12,2) DEFAULT NULL COMMENT ''
) ENGINE=OLAP
UNIQUE KEY(`id`, `order_code`)
DISTRIBUTED BY HASH(`order_code`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 3",
"in_memory" = "false",
"storage_format" = "V2"
);
```

3. 创建 datax 脚本

my_import.json

```
{
    "job": {
        "content": [
            {
                "reader": {
                    "name": "mysqlreader",
```

```
                "parameter": {
                    "column": ["id","order_code","line_code","remark","unit_no","unit_name","
                        ↪ price"],
                    "connection": [
                        {
                            "jdbcUrl": ["jdbc:mysql://localhost:3306/demo"],
                            "table": ["employees_1"]
                        }
                    ],
                    "username": "root",
                    "password": "xxxxx",
                    "where": ""
                }
            },
            "writer": {
                "name": "doriswriter",
                "parameter": {
                    "loadUrl": ["127.0.0.1:8030"],
                    "column": ["id","order_code","line_code","remark","unit_no","unit_name","
                        ↪ price"],
                    "username": "root",
                    "password": "xxxxxx",
                    "postSql": ["select count(1) from all_employees_info"],
                    "preSql": [],
                    "flushInterval":30000,
                    "connection": [
                      {
                        "jdbcUrl": "jdbc:mysql://127.0.0.1:9030/demo",
                        "selectedDatabase": "demo",
                        "table": ["all_employees_info"]
                      }
                    ],
                    "loadProps": {
                        "format": "json",
                        "strip_outer_array":"true",
                        "line_delimiter": "\\x02"
                    }
                }
            }
        }
    ],
    "setting": {
        "speed": {
            "channel": "1"
        }
```

```
        }
    }
}
```

备注：

```
"loadProps": {
  "format": "json",
  "strip_outer_array":"true",
  "line_delimiter": "\\x02"
}
```

1. 这里我们使用了 JSON 格式导入数据
2. line_delimiter 默认是换行符，可能会和数据中的值冲突，我们可以使用一些特殊字符或者不可见字符，避免导入错误
3. strip_outer_array：在一批导入数据中表示多行数据，Doris 在解析时会将数组展开，然后依次解析其中的每一个 Object 作为一行数据
4. 更多 Stream load 参数请参照Stream load 文档
5. 如果是 CSV 格式我们可以这样使用

```
"loadProps": {
   "format": "csv",
   "column_separator": "\\x01",
   "line_delimiter": "\\x02"
}
```

CSV 格式要特别注意行列分隔符，避免和数据中的特殊字符冲突，这里建议使用隐藏字符，默认列分隔符是：t，行分隔符：n

4. 执行 DataX 任务，具体参考 DataX 官网

```
python bin/datax.py my_import.json
```

执行之后我们可以看到下面的信息

```
2022-11-16 14:28:54.012 [job-0] INFO  JobContainer - jobContainer starts to do prepare ...
2022-11-16 14:28:54.012 [job-0] INFO  JobContainer - DataX Reader.Job [mysqlreader] do prepare
    ↪ work .
2022-11-16 14:28:54.013 [job-0] INFO  JobContainer - DataX Writer.Job [doriswriter] do prepare
    ↪ work .
2022-11-16 14:28:54.020 [job-0] INFO  JobContainer - jobContainer starts to do split ...
2022-11-16 14:28:54.020 [job-0] INFO  JobContainer - Job set Channel-Number to 1 channels.
2022-11-16 14:28:54.023 [job-0] INFO  JobContainer - DataX Reader.Job [mysqlreader] splits to [1]
    ↪  tasks.
```

```
2022-11-16 14:28:54.023 [job-0] INFO  JobContainer - DataX Writer.Job [doriswriter] splits to [1]
    ↪  tasks.
2022-11-16 14:28:54.033 [job-0] INFO  JobContainer - jobContainer starts to do schedule ...
2022-11-16 14:28:54.036 [job-0] INFO  JobContainer - Scheduler starts [1] taskGroups.
2022-11-16 14:28:54.037 [job-0] INFO  JobContainer - Running by standalone Mode.
2022-11-16 14:28:54.041 [taskGroup-0] INFO  TaskGroupContainer - taskGroupId=[0] start [1]
    ↪ channels for [1] tasks.
2022-11-16 14:28:54.043 [taskGroup-0] INFO  Channel - Channel set byte_speed_limit to -1, No bps
    ↪ activated.
2022-11-16 14:28:54.043 [taskGroup-0] INFO  Channel - Channel set record_speed_limit to -1, No
    ↪ tps activated.
2022-11-16 14:28:54.049 [taskGroup-0] INFO  TaskGroupContainer - taskGroup[0] taskId[0]
    ↪ attemptCount[1] is started
2022-11-16 14:28:54.052 [0-0-0-reader] INFO  CommonRdbmsReader$Task - Begin to read record by Sql
    ↪ : [select taskid,projectid,taskflowid,templateid,template_name,status_task from dwd_
    ↪ universal_tb_task
] jdbcUrl:[jdbc:mysql://localhost:3306/demo?yearIsDateType=false&zeroDateTimeBehavior=
    ↪ convertToNull&tinyInt1isBit=false&rewriteBatchedStatements=true].
Wed Nov 16 14:28:54 GMT+08:00 2022 WARN: Establishing SSL connection without server's identity
    ↪ verification is not recommended. According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+
    ↪ requirements SSL connection must be established by default if explicit option isn't set.
    ↪ For compliance with existing applications not using SSL the verifyServerCertificate
    ↪ property is set to 'false'. You need either to explicitly disable SSL by setting useSSL=
    ↪ false, or set useSSL=true and provide truststore for server certificate verification.
2022-11-16 14:28:54.071 [0-0-0-reader] INFO  CommonRdbmsReader$Task - Finished read record by Sql
    ↪ : [select taskid,projectid,taskflowid,templateid,template_name,status_task from dwd_
    ↪ universal_tb_task
] jdbcUrl:[jdbc:mysql://localhost:3306/demo?yearIsDateType=false&zeroDateTimeBehavior=
    ↪ convertToNull&tinyInt1isBit=false&rewriteBatchedStatements=true].
2022-11-16 14:28:54.104 [Thread-1] INFO  DorisStreamLoadObserver - Start to join batch data: rows
    ↪ [2] bytes[438] label[datax_doris_writer_c4e08cb9-c157-4689-932f-db34acc45b6f].
2022-11-16 14:28:54.104 [Thread-1] INFO  DorisStreamLoadObserver - Executing stream load to: '
    ↪ http://127.0.0.1:8030/api/demo/dwd_universal_tb_task/_stream_load', size: '441'
2022-11-16 14:28:54.224 [Thread-1] INFO  DorisStreamLoadObserver - StreamLoad response :{"Status"
    ↪ :"Success","BeginTxnTimeMs":0,"Message":"OK","NumberUnselectedRows":0,"
    ↪ CommitAndPublishTimeMs":17,"Label":"datax_doris_writer_c4e08cb9-c157-4689-932f-
    ↪ db34acc45b6f","LoadBytes":441,"StreamLoadPutTimeMs":1,"NumberTotalRows":2,"
    ↪ WriteDataTimeMs":11,"TxnId":217056,"LoadTimeMs":31,"TwoPhaseCommit":"false","
    ↪ ReadDataTimeMs":0,"NumberLoadedRows":2,"NumberFilteredRows":0}
2022-11-16 14:28:54.225 [Thread-1] INFO  DorisWriterManager - Async stream load finished: label[
    ↪ datax_doris_writer_c4e08cb9-c157-4689-932f-db34acc45b6f].
2022-11-16 14:28:54.249 [taskGroup-0] INFO  TaskGroupContainer - taskGroup[0] taskId[0] is
    ↪ successed, used[201]ms
2022-11-16 14:28:54.250 [taskGroup-0] INFO  TaskGroupContainer - taskGroup[0] completed it's
    ↪ tasks.
```

```
2022-11-16 14:29:04.048 [job-0] INFO  StandAloneJobContainerCommunicator - Total 2 records, 214
    ↪ bytes | Speed 21B/s, 0 records/s | Error 0 records, 0 bytes |  All Task WaitWriterTime
    ↪ 0.000s |  All Task WaitReaderTime 0.000s | Percentage 100.00%
2022-11-16 14:29:04.049 [job-0] INFO  AbstractScheduler - Scheduler accomplished all tasks.
2022-11-16 14:29:04.049 [job-0] INFO  JobContainer - DataX Writer.Job [doriswriter] do post work.
Wed Nov 16 14:29:04 GMT+08:00 2022 WARN: Establishing SSL connection without server's identity
    ↪ verification is not recommended. According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+
    ↪ requirements SSL connection must be established by default if explicit option isn't set.
    ↪ For compliance with existing applications not using SSL the verifyServerCertificate
    ↪ property is set to 'false'. You need either to explicitly disable SSL by setting useSSL=
    ↪ false, or set useSSL=true and provide truststore for server certificate verification.
2022-11-16 14:29:04.187 [job-0] INFO  DorisWriter$Job - Start to execute preSqls:[select count(1)
    ↪  from dwd_universal_tb_task]. context info:jdbc:mysql://172.16.0.13:9030/demo.
2022-11-16 14:29:04.204 [job-0] INFO  JobContainer - DataX Reader.Job [mysqlreader] do post work.
2022-11-16 14:29:04.204 [job-0] INFO  JobContainer - DataX jobId [0] completed successfully.
2022-11-16 14:29:04.204 [job-0] INFO  HookInvoker - No hook invoked, because base dir not exists
    ↪ or is a file: /data/datax/hook
2022-11-16 14:29:04.205 [job-0] INFO  JobContainer -
         [total cpu info] =>
                averageCpu                     | maxDeltaCpu                    | minDeltaCpu
                -1.00%                         | -1.00%                         | -1.00%


         [total gc info] =>
                 NAME                 | totalGCCount       | maxDeltaGCCount    | minDeltaGCCount
                     ↪      | totalGCTime        | maxDeltaGCTime     | minDeltaGCTime
                 PS MarkSweep         | 1                  | 1                  | 1
                     ↪                  | 0.017s             | 0.017s             | 0.017s
                 PS Scavenge          | 1                  | 1                  | 1
                     ↪                  | 0.007s             | 0.007s             | 0.007s

2022-11-16 14:29:04.205 [job-0] INFO  JobContainer - PerfTrace not enable!
2022-11-16 14:29:04.206 [job-0] INFO  StandAloneJobContainerCommunicator - Total 2 records, 214
    ↪ bytes | Speed 21B/s, 0 records/s | Error 0 records, 0 bytes |  All Task WaitWriterTime
    ↪ 0.000s |  All Task WaitReaderTime 0.000s | Percentage 100.00%
2022-11-16 14:29:04.206 [job-0] INFO  JobContainer -
任务启动时刻                    : 2022-11-16 14:28:53
任务结束时刻                    : 2022-11-16 14:29:04
任务总计耗时                    :                 10s
任务平均流量                    :               21B/s
记录写入速度                    :              0rec/s
读出记录总数                    :                   2
读写失败总数                    :                   0
```

5.9.3 DBT Doris Adapter

DBT(Data Build Tool) 是专注于做 ELT（提取、加载、转换）中的 T（Transform）——“转换数据”环节的组件 dbt-doris adapter 是基于dbt-core 开发，依赖于mysql-connector-python驱动对 doris 进行数据转换。

代码仓库：https://github.com/apache/doris/tree/master/extension/dbt-doris

5.9.3.1 版本支持

doris	python	dbt-core	dbt-doris
>=1.2.5	>=3.8,<=3.10	>=1.5.0	<=0.3
>=1.2.5	>=3.9	>=1.8.0	>=0.4

5.9.3.2 dbt-doris adapter 使用

5.9.3.2.1 dbt-doris adapter 安装

使用 pip 安装：

```
pip install dbt-doris
```

安装行为会默认安装所有 dbt 运行的依赖，可以使用如下命令查看验证：

```
dbt --version
```

如果系统未识别 dbt 这个命令，可以创建一条软连接：

```
ln -s /usr/local/python3/bin/dbt /usr/bin/dbt
```

5.9.3.2.2 dbt-doris adapter 初始化

```
dbt init
```

会出现询问式命令行，输入相应配置如下即可初始化一个 dbt 项目：

名称	默认值	含义
project		项目名
database		输入对应编号选择适配器
host		doris 的 host
port	9030	doris 的 MySQL Protocol Port
schema		在 dbt-doris 中，等同于 database，库名
username		doris 的 username
password		doris 的 password
threads	1	dbt-doris 中并行度（设置与集群能力不匹配的并行度会增加 dbt 运行失败风险）

5.9.3.2.3 dbt-doris adapter 运行

相关 dbt 运行文档，可参考此处。进入到刚刚创建的项目目录下面，执行默认的 dbt 模型：

```
dbt run
```

可以看到运行了两个 model：my_first_dbt_model 和 my_second_dbt_model

他们分别是物化表 table 和视图 view。

可以登陆 doris，查看 my_first_dbt_model 和 my_second_dbt_model 的数据结果及建表语句。

5.9.3.2.4 dbt-doris adapter 物化方式

dbt-doris 的物化方式（Materialization）支持以下三种：

1. view
2. table
3. incremental

View

使用view作为物化模式，在 Models 每次运行时都会通过 create view as 语句重新构建为视图。(默认情况下，dbt 的物化方式为 view)

```
优点：没有存储额外的数据，源数据之上的视图将始终包含最新的记录。
缺点：执行较大转换或嵌套在其他view之上的view查询速度很慢。
建议：通常从模型的视图开始，只有当存在性能问题时才更改为另一个物化方式。view
    ↪ 最适合不进行重大转换的模型，例如重命名，列变更。
```

配置项：

```
models:
  <resource-path>:
    +materialized: view
```

或者在 model 文件里面写

```
{{ config(materialized = "view") }}
```

Table

使用 table 物化模式时，您的模型在每次运行时都会通过 create table as select 语句重建为表。对于 dbt 的 tablet 物化，dbt-doris 采用以下步骤保证数据更迭时候的原子性：

1. create table this_table_temp as {{ model sql}}，首先创建临时表。
2. 判断 this_table 是否不存在，即是首次创建，执行rename，将临时表变更为最终表。

3. 若已经存在，则 `alter table this_table REPLACE WITH TABLE this_table_temp PROPERTIES('swap' = 'False')`，此操作可以交换表名并且删除this_table_temp临时表，此过程通过 Doris 内核的事务机制保证本次操作原子性。

> 优点：table查询速度会比view快。
> 缺点：table需要较长时间才能构建或重建，会额外存储数据，而且不能够做增量数据同步。
> 建议：建议对 BI 工具查询的model或下游查询、转换等操作较慢的model使用table物化方式。

配置项：

```
models:
  <resource-path>:
    +materialized: table
    +duplicate_key: [ <column-name>, ... ],
    +replication_num: int,
    +partition_by: [ <column-name>, ... ],
    +partition_type: <engine-type>,
    +partition_by_init: [<pertition-init>, ... ]
    +distributed_by: [ <column-name>, ... ],
    +buckets: int | 'auto',
    +properties: {<key>:<value>,...}
```

或者在 model 文件里面写

```
{{ config(
    materialized = "table",
    duplicate_key = [ "<column-name>", ... ],
    replication_num = "<int>"
    partition_by = [ "<column-name>", ... ],
    partition_type = "<engine-type>",
    partition_by_init = ["<pertition-init>", ... ]
    distributed_by = [ "<column-name>", ... ],
    buckets = "<int>" | "auto",
    properties = {"<key>":"<value>",...}
      ...
    ]
) }}
```

上述配置项详情如下：

配置项	描述	Required?
materialized	该表的物化形式（对应创建表模型为明细模型（Duplicate））	Required
duplicate_key	明细模型的排序列	Optional
replication_num	表副本数	Optional
partition_by	表分区列	Optional
partition_type	表分区类型，range 或 list .(default: RANGE)	Optional

配置项	描述	Required?
partition_by_init	初始化的表分区	Optional
distributed_by	表桶区列	Optional
buckets	分桶数量	Optional
properties	建表的其他配置	Optional

Incremental

以上次运行 dbt 的 incremental model 结果为基准，增量的将记录插入或更新到表中。doris 的增量实现有两种方式，此项设计两种增量（incremental_strategy 设置）的策略：

- insert_overwrite：依赖于 unique 模型，如果有增量需求，在初始化该模型的数据时就指定物化为 incremental，通过指定聚合列进行聚合，实现增量数据的覆盖。
- append：依赖于duplicate模型，仅仅对增量数据做追加，不涉及修改任何历史数据。因此不需要指定 unique_key。

优点：只需转换新记录，可显著减少构建时间。
缺点：incremental模式需要额外的配置，是 dbt 的高级用法，需要复杂场景的支持和对应组件的适配。
建议：增量模型最适合基于事件相关的场景或 dbt 运行变得太慢时使用增量模型

配置项：

```
models:
  <resource-path>:
    +materialized: incremental
    +incremental_strategy: <strategy>
    +unique_key: [ <column-name>, ... ],
    +replication_num: int,
    +partition_by: [ <column-name>, ... ],
    +partition_type: <engine-type>,
    +partition_by_init: [<pertition-init>, ... ]
    +distributed_by: [ <column-name>, ... ],
    +buckets: int | 'auto',
    +properties: {<key>:<value>,...}
```

或者在 model 文件里面写

```
{{ config(
    materialized = "incremental",
    incremental_strategy = "<strategy>"
    unique_key = [ "<column-name>", ... ],
    replication_num = "<int>"
    partition_by = [ "<column-name>", ... ],
    partition_type = "<engine-type>",
    partition_by_init = ["<pertition-init>", ... ]
```

```
    distributed_by = [ "<column-name>", ... ],
    buckets = "<int>" | "auto",
    properties = {"<key>":"<value>",...}
      ...
    )
}}
```

上述配置项详情如下：

配置项	描述	Required?
materialized	该表的物化形式	Required
incremental_strategy	增量策略	Optional
unique_key	unique 表的 key 列	Optional
replication_num	表副本数	Optional
partition_by	表分区列	Optional
partition_type	表分区类型，range 或 list .(default: RANGE)	Optional
partition_by_init	初始化的表分区	Optional
distributed_by	表桶区列	Optional
buckets	分桶数量	Optional
properties	建表的其他配置	Optional

5.9.3.2.5 dbt-doris adapter seed

seed 是用于加载 csv 等数据文件时的功能模块，它是一种加载文件入库参与模型构建的一种方式，但有以下注意事项：

1. seed 不应用于加载原始数据（例如，从生产数据库导出大型 CSV 文件）。
2. 由于 seed 是受版本控制的，因此它们最适合包含特定于业务的逻辑的文件，例如国家/地区代码列表或员工的用户 ID。
3. 对于大文件，使用 dbt 的 seed 功能加载 CSV 的性能不佳。应该考虑使用 streamload 等方式将这些 CSV 加载到 doris 中。

用户可以在 dbt project 的目录下面看到 seeds 的目录，在里面上传 csv 文件和 seed 配置文件并运行

```
 dbt seed --select seed_name
```

常见 seed 配置文件写法，支持对列类型的定义：

```
seeds:
  seed_name: # 种子名称，在 seed 构建后，会作为表名
    config:
      schema: demo_seed # 在 seed 构建后，会作为 database 的一部分
      full_refresh: true
      replication_num: 1
      column_types:
```

```
        id: bigint
        phone: varchar(32)
        ip: varchar(15)
        name: varchar(20)
        cost: DecimalV3(19,10)
```

5.9.3.3 使用示例

5.9.3.3.1 视图模型样例参考

```
{{ config(materialized='view') }}

select
    u.user_id,
    max(o.create_time) as create_time,
    sum (o.cost) as balance
from {{ ref('sell_order') }} as o
left join {{ ref('sell_user') }} as u
on u.account_id=o.account_id
group by u.user_id
order by u.user_id
```

5.9.3.3.2 表模型样例参考

```
{{ config(materialized='table') }}

select
    u.user_id,
    max(o.create_time) as create_time,
    sum (o.cost) as balance
from {{ ref('sell_order') }} as o
left join {{ ref('sell_user') }} as u
on u.account_id=o.account_id
group by u.user_id
order by u.user_id
```

5.9.3.3.3 增量模型样例参考 (duplicate 模式)

建表为 duplicate 模式，无数据聚合，不需要指定 unique_key

```
{{ config(
    materialized='incremental',
    replication_num=1
) }}
```

```
with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select * from source_data
```

5.9.3.3.4 增量模型样例参考 (unique 模式)

建表为 unique 模式，数据聚合，必须指定 unique_key

```
{{ config(
materialized='incremental',
unique_key=['account_id','create_time']
) }}

with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select * from source_data
```

5.9.3.3.5 增量模型全量刷新样例参考

```
{{ config(
    materialized='incremental',
    full_refresh = true
)}}

select * from
 {{ source('dbt_source', 'sell_user') }}
```

5.9.3.3.6 设置分桶规则样例参考

此处 buckets 可以填 auto 或者正整数，分别代表自动分桶和设置固定分桶数

```
{{ config(
    materialized='incremental',
    unique_key=['account_id',"create_time"],
    distributed_by=['account_id'],
    buckets='auto'
```

```
) }}

with source_data as (
    select
        *
    from {{ ref('sell_order') }}
)

select
    *
    from source_data

{% if is_incremental() %}
    where
    create_time > (select max(create_time) from {{this}})
{% endif %}
```

5.9.3.3.7 设置副本数样例参考

```
{{ config(
    materialized='table',
    replication_num=1
)}}

with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select * from source_data
```

5.9.3.3.8 动态分区样例参考

```
{{ config(
    materialized='incremental',
    partition_by = 'create_time',
    partition_type = 'range',
    -- 这里的 properties 是 create table 语句中的 properties，这里面写了动态分区的相关配置
    properties = {
        "dynamic_partition.time_unit":"DAY",
        "dynamic_partition.end":"8",
        "dynamic_partition.prefix":"p",
        "dynamic_partition.buckets":"4",
```

```
        "dynamic_partition.create_history_partition":"true",
        "dynamic_partition.history_partition_num":"3"
    }
) }}

with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select
    *
    from source_data

{% if is_incremental() %}
    where
    create_time = DATE_SUB(CURDATE(), INTERVAL 1 DAY)
{% endif %}
```

5.9.3.3.9 常规分区样例参考

```
{{ config(
    materialized='incremental',
    partition_by = 'create_time',
    partition_type = 'range',
    -- 这里的 partition_by_init 是指的 创建分区表的历史分区，当前 doris
        ↪ 版本的历史分区需要手动指定
    partition_by_init = [
        "PARTITION `p20240601` VALUES [(\"2024-06-01\"),  (\"2024-06-02\"))",
        "PARTITION `p20240602` VALUES [(\"2024-06-02\"),  (\"2024-06-03\"))"
    ]
 )}}

with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select
    *
    from source_data

{% if is_incremental() %}
```

```
    where
    -- 如果提供了 my_date 变量，则使用该通路（通过 dbt run --vars '{"my_date": "\"2024-06-03\""}'
        ↪  命令）如果没有提供 my_date 变量（直接 dbt run ），则使用当前日期的前一天 ，
        ↪ 这里的增量选择建议直接使用 doris 的 CURDATE() 函数，这个通路也是生产环境经常走的。
    create_time = {{ var('my_date' , 'DATE_SUB(CURDATE(), INTERVAL 1 DAY)') }}

{% endif %}
```

5.9.3.3.10 批处理日期设置参数样例参考

```
{{ config(
    materialized='incremental',
    partition_by = 'create_time',
    partition_type = 'range',
    ...
)}}

with source_data as (
    select
        *
    from {{ ref('sell_order2') }}
)

select
    *
    from source_data

{% if is_incremental() %}
    where
    -- 如果提供了 my_date 变量，则使用该通路（通过 dbt run --vars '{"my_date": "\"2024-06-03\""}'
        ↪  命令）如果没有提供 my_date 变量（直接 dbt run ），则使用当前日期的前一天 ，
        ↪ 这里的增量选择建议直接使用 doris 的 CURDATE() 函数，这个通路也是生产环境经常走的。
    create_time = {{ var('my_date' , 'DATE_SUB(CURDATE(), INTERVAL 1 DAY)') }}

{% endif %}
```

5.9.3.3.11 自定义表数据列类型及精度样例参考

schema.yaml 文件对 models 中 columns 的 data_type 配置如下：

```
models:
  - name: sell_user
    description: "A dbt model named sell_user"
    columns:
      - name: user_id
```

```
        data_type: BIGINT
      - name: account_id
        data_type: VARCHAR(12)
      - name: status
      - name: cost_sum
        data_type: DECIMAL(38,9)
      - name: update_time
        data_type: DATETIME
      - name: create_time
        data_type: DATETIME
```

5.9.3.3.12 访问 catalog 样例参考

Data Catalog 是 Doris 数据湖功能中指向不同的数据源，其层级在 Database 之上。对其访问推荐通过 dbt-doris 内置 Macros : `catalog_source`

```
{{ config(materialized='table', replication_num=1) }}

select *
--  use macros 'catalog_source' not macros 'source'
--  catalog name is 'mysql_catalog'
--  database name is 'dbt_source'
--  table name is 'sell_user'
from {{ catalog_source('mysql_catalog', 'dbt_source', 'sell_user') }}
```

5.9.4 Seatunnel Doris Sink

SeaTunnel 是一个非常简单易用的超高性能分布式数据集成平台，支持海量数据的实时同步。每天稳定高效地同步数百亿数据

5.9.4.1 Connector-V2

2.3.1 版本的 Apache SeaTunnel Connector-V2 支持了 Doris Sink，并且支持 exactly-once 的精准一次写入和 CDC 数据同步

5.9.4.1.1 插件代码

SeaTunnel Doris Sink 插件代码

5.9.4.1.2 参数列表

name	type	required	default value
fenodes	string	yes	-
username	string	yes	-

name	type	required	default value
password	string	yes	-
table.identifier	string	yes	-
sink.label-prefix	string	yes	-
sink.enable-2pc	bool	no	true
sink.enable-delete	bool	no	false
doris.config	map	yes	-

`fenodes [string]`

Doris 集群 FE 节点地址，格式为 `"fe_ip:fe_http_port,..."`

`username [string]`

Doris 用户名

`password [string]`

Doris 用户密码

`table.identifier [string]`

Doris 表名称，格式为 DBName.TableName

`sink.label-prefix [string]`

Stream Load 导入使用的标签前缀。在 2pc 场景下，需要全局唯一性来保证 SeaTunnel 的 EOS 语义

`sink.enable-2pc [bool]`

是否启用两阶段提交 (2pc)，默认为 true，以确保 exact - once 语义。关于两阶段提交，请参考这里

`sink.enable-delete [bool]`

是否启用删除。该选项需要 Doris 表开启批量删除功能 (默认开启 0.15+ 版本)，且只支持 Unique 表模型。你可以在这个链接获得更多细节：

批量删除

`doris.config [map]`

Stream Load `data_desc` 的参数，你可以在这个链接获得更多细节：

更多 Stream Load 参数

5.9.4.1.3 使用示例

使用 JSON 格式导入数据

```
sink {
    Doris {
        fenodes = "doris_fe:8030"
        username = root
        password = ""
        table.identifier = "test.table_sink"
```

```
        sink.enable-2pc = "true"
        sink.label-prefix = "test_json"
        doris.config = {
            format="json"
            read_json_by_line="true"
        }
    }
}
```

使用 CSV 格式导入数据

```
sink {
    Doris {
        fenodes = "doris_fe:8030"
        username = root
        password = ""
        table.identifier = "test.table_sink"
        sink.enable-2pc = "true"
        sink.label-prefix = "test_csv"
        doris.config = {
          format = "csv"
          column_separator = ","
          line_delimiter = "`\n`"
        }
    }
}
```

5.9.4.2 Connector-V1

2.1.0 的 Apache SeaTunnel 支持 Doris 的连接器，SeaTunnel 可以通过 Spark 引擎和 Flink 引擎同步数据至 Doris 中。

5.9.4.2.1 Flink Doris Sink

插件代码

Seatunnel Flink Sink Doris 插件代码

参数列表

配置项	类型	必填	默认值	支持引擎
fenodes	string	yes	-	Flink
database	string	yes	-	Flink
table	string	yes	-	Flink
user	string	yes	-	Flink
password	string	yes	-	Flink
batch_size	int	no	100	Flink

配置项	类型	必填	默认值	支持引擎
interval	int	no	1000	Flink
max_retries	int	no	1	Flink
doris.*	-	no	-	Flink

`fenodes [string]`

Doris Fe Http 访问地址，eg: 127.0.01:8030

`database [string]`

写入 Doris 的库名

`table [string]`

写入 Doris 的表名

`user [string]`

Doris 访问用户

`password [string]`

Doris 访问用户密码

`batch_size [int]`

单次写 Doris 的最大行数，默认值 100

`interval [int]`

flush 间隔时间 (毫秒)，超过该时间后异步线程将缓存中数据写入 Doris。设置为 0 表示关闭定期写入。

`max_retries [int]`

写 Doris 失败之后的重试次数

`doris.* [string]`

Stream load 的导入参数。例如:‘doris.column_separator’ =’,’ 等

更多 Stream Load 参数配置

Examples

Socket 数据写入 Doris

```
env {
  execution.parallelism = 1
}
source {
    SocketStream {
      host = 127.0.0.1
      port = 9999
      result_table_name = "socket"
      field_name = "info"
```

```
    }
}
transform {
}
sink {
  DorisSink {
      fenodes = "127.0.0.1:8030"
      user = root
      password = 123456
      database = test
      table = test_tbl
      batch_size = 5
      max_retries = 1
      interval = 5000
    }
}
```

启动命令

```
sh bin/start-seatunnel-flink.sh --config config/flink.streaming.conf
```

5.9.4.2.2 Spark Sink Doris

插件代码

Spark Sink Doris 的插件代码在这里

参数列表

参数名	参数类型	是否必要	默认值	引擎类型
fenodes	string	yes	-	Spark
database	string	yes	-	Spark
table	string	yes	-	Spark
user	string	yes	-	Spark
password	string	yes	-	Spark
batch_size	int	yes	100	Spark
doris.*	string	no	-	Spark

`fenodes [string]`

Doris Fe 节点地址:8030

`database [string]`

写入 Doris 的库名

`table [string]`

写入 Doris 的表名

user [string]

Doris 访问用户

password [string]

Doris 访问用户密码

batch_size [string]

Spark 通过 Stream Load 方式写入，每个批次提交条数

doris. [string]

Stream Load 方式写入的 Http 参数优化，在官网参数前加上’Doris.’ 前缀

更多 Stream Load 参数配置

Examples

Hive 迁移数据至 Doris

```
env{
  spark.app.name = "hive2doris-template"
}

spark {
  spark.sql.catalogImplementation = "hive"
}

source {
  hive {
    preSql = "select * from tmp.test"
    result_table_name = "test"
  }
}

transform {
}


sink {

Console {

  }

Doris {
   fenodes="xxxx:8030"
   database="tmp"
   table="test"
   user="root"
```

```
    password="root"
    batch_size=1000
    doris.column_separator="`\t`"
    doris.columns="date_key,date_value,day_in_year,day_in_month"
    }
}
```

启动命令

```
sh bin/start-waterdrop-spark.sh --master local[4] --deploy-mode client --config ./config/spark.
    ↪ conf
```

5.9.5 Kettle Doris Plugin

5.9.5.1 Kettle Doris Plugin

Kettle Doris 的插件，用于在 Kettle 中通过 Stream Load 将其他数据源的数据写入到 Doris 中。

这个插件是利用 Doris 的 Stream Load 功能进行数据导入的。需要配合 Kettle 服务一起使用。

5.9.5.2 关于 Kettle

Kettle 是一款开源的 ETL（Extract, Transform, Load）工具，最早由 Pentaho 公司开发，Kettle 是 Pentaho 产品套件中的核心组件之一，主要用于数据集成和数据处理，能够轻松完成从各种来源提取数据、对数据进行清洗和转换，并将其加载到目标系统中的任务。

更多信息请参阅：https://pentaho.com/

5.9.5.3 使用手册

5.9.5.3.1 下载 Kettle 安装

Kettle 下载地址：https://pentaho.com/download/#download-pentaho 下载后解压，运行 spoon.sh 即可启动 kettle 也可以自行编译，参考编译章节

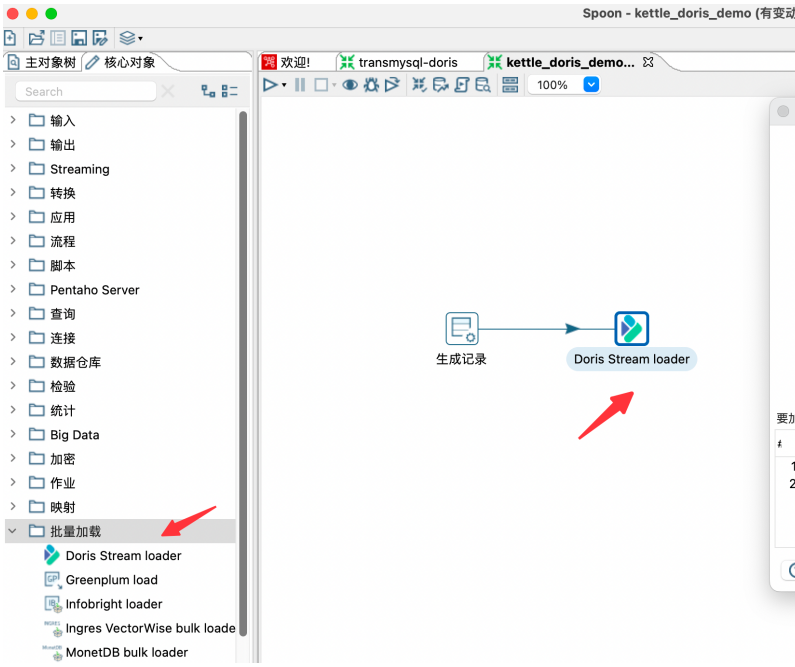
5.9.5.3.2 编译 Kettle Doris Plugin

```
cd doris/extension/kettle
mvn clean package -DskipTests
```

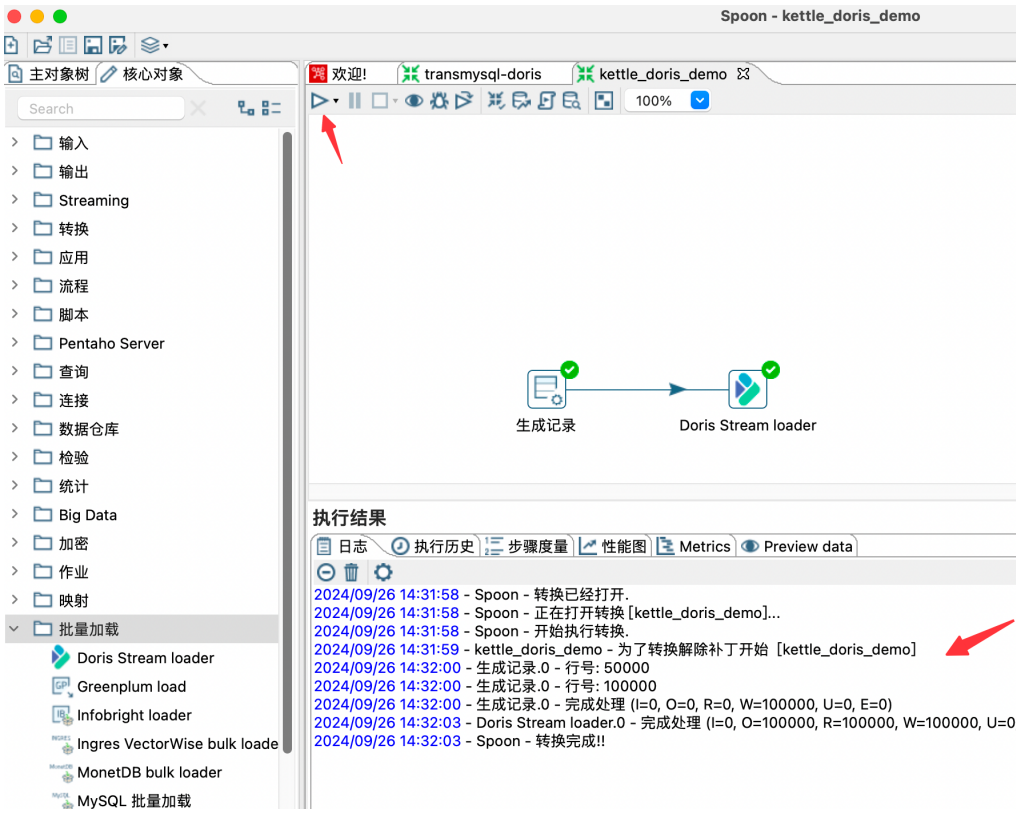
编译完成后，将插件包解压后拷贝到 kettle 的 plugins 目录下

```
cd assemblies/plugin/target
unzip doris-stream-loader-plugins-9.4.0.0-343.zip
cp -r doris-stream-loader ${KETTLE_HOME}/plugins/
mvn clean package -DskipTests
```

5.9.5.3.3 构建作业



在 Kettle 中的批量加载中找到 Doris Stream Loader，构建作业



点击开始运行作业即可完成数据同步

5.9.5.3.4 参数说明

Key	Default Value	Required	Comment
Step name	–	Y	步骤名称
fenodes	–	Y	Doris FE http 地址，支持多个地址，使用逗号分隔
数据库	–	Y	Doris 的写入数据库
目标表	–	Y	Doris 的写入表
用户名	–	Y	访问 Doris 的用户名
密码	–	N	访问 Doris 的密码
单次导入最大行数	10000	N	单次导入的最大行数
单次导入最大字节	10485760(10MB)	N	单次导入的最大字节大小
导入重试次数	3	N	导入失败之后的重试次数
Stream Load 属性	–	N	Stream Load 的请求头
删除模式	N	N	是否开启删除模式。默认情况下，Stream Load 执行插入操作，开启删除模式后

{“title”：“Kyuubi”，“language”：“zh-CN”}

5.9.6 Kyuubi

5.9.6.1 介绍

Apache Kyuubi 是一个分布式和多租户网关，用于在 Lakehouse 上提供 Serverless SQL，可连接包括 Spark、Flink、Hive、JDBC 等引擎，并对外提供 Thrift、Trino 等接口协议供灵活对接。其中 Apache Kyuubi 实现了 JDBC Engine 并支持 Doris 方言，并可用于对接 Doris 作为数据源。 Apache Kyuubi 可提供高可用、服务发现、租户隔离、统一认证、生命周期管理等一系列特性。

5.9.6.2 下载 Apache Kyuubi

5.9.6.3 配置方法

5.9.6.3.1 下载 Apache Kyuubi

从官网下载 Apache Kyuubi 1.6.0 或以上版本的安装包后解压。

Apache Kyuubi 下载地址：https://kyuubi.apache.org/zh/releases.html

5.9.6.3.2 配置 Doris 作为 Kyuubi 数据源

- 修改配置文件 `$KYUUBI_HOME/conf/kyuubi-defaults.conf`

```
kyuubi.engine.type=jdbc
kyuubi.engine.jdbc.type=doris
kyuubi.engine.jdbc.driver.class=com.mysql.cj.jdbc.Driver
kyuubi.engine.jdbc.connection.url=jdbc:mysql://xxx:xxx
kyuubi.engine.jdbc.connection.user=***
kyuubi.engine.jdbc.connection.password=***
```

配置项	说明
kyuubi.engine.type	引擎类型。请使用 jdbc
kyuubi.engine.jdbc.type	JDBC 服务类型。这里请指定为 doris
kyuubi.engine.jdbc.driver.class	连接 JDBC 服务使用的驱动类名。请使用 com.mysql.cj.jdbc.Driver
kyuubi.engine.jdbc.connection.url	JDBC 服务连接。这里请指定 Doris FE 上的 mysql server 连接地址
kyuubi.engine.jdbc.connection.user	JDBC 服务用户名
kyuubi.engine.jdbc.connection.password	JDBC 服务密码

- 其他相关配置参考 Apache Kyuubi 配置说明 。

5.9.6.3.3 添加 MySQL 驱动

添加 Mysql JDB C 驱动 `mysql-connector-j-8.X.X.jar` 到 `$KYUUBI_HOME/externals/engines/jdbc` 目录下。

5.9.6.3.4 启动 Kyuubi 服务

`$KYUUBI_HOME/bin/kyuubi start` 启动后，Kyuubi 默认监听 10009 端口提供 Thrift 协议。

5.9.6.4 使用方法

以下例子展示通过 Apache Kyuubi 的 beeline 工具经 Thrift 协议查询 Doris。

5.9.6.4.1 建立连接

```
$ $KYUUBI_HOME/bin/beeline -u "jdbc:hive2://xxxx:10009/"
```

5.9.6.4.2 执行查询

执行查询语句 `select * from demo.expamle_tbl;` 并得到结果。

```
0: jdbc:hive2://xxxx:10009/> select * from demo.example_tbl;

2023-03-07 09:29:14.771 INFO org.apache.kyuubi.operation.ExecuteStatement: Processing anonymous's
    ↪  query[bdc59dd0-ceea-4c02-8c3a-23424323f5db]: PENDING_STATE -> RUNNING_STATE, statement:
select * from demo.example_tbl
2023-03-07 09:29:14.786 INFO org.apache.kyuubi.operation.ExecuteStatement: Query[bdc59dd0-ceea-4
    ↪ c02-8c3a-23424323f5db] in FINISHED_STATE
```

```
2023-03-07 09:29:14.787 INFO org.apache.kyuubi.operation.ExecuteStatement: Processing anonymous's
    ↪  query[bdc59dd0-ceea-4c02-8c3a-23424323f5db]: RUNNING_STATE -> FINISHED_STATE, time taken
    ↪ : 0.015 seconds
+----------+-------------+-------+------+------+----------------------+-------+----------------+-------------
    ↪
| user_id  |    date     | city  | age  | sex  |    last_visit_date     | cost  | max_dwell_time
    ↪  | min_dwell_time  |
+----------+-------------+-------+------+------+----------------------+-------+----------------+-------------
    ↪
| 10000    | 2017-10-01  | 北京   | 20   | 0    | 2017-10-01 07:00:00.0  | 70    | 10
    ↪              | 2               |
| 10001    | 2017-10-01  | 北京   | 30   | 1    | 2017-10-01 17:05:45.0  | 4     | 22
    ↪              | 22              |
| 10002    | 2017-10-02  | 上海   | 20   | 1    | 2017-10-02 12:59:12.0  | 400   | 5
    ↪               | 5               |
| 10003    | 2017-10-02  | 广州   | 32   | 0    | 2017-10-02 11:20:00.0  | 60    | 11
    ↪              | 11              |
| 10004    | 2017-10-01  | 深圳   | 35   | 0    | 2017-10-01 10:00:15.0  | 200   | 3
    ↪               | 3               |
| 10004    | 2017-10-03  | 深圳   | 35   | 0    | 2017-10-03 10:20:22.0  | 22    | 6
    ↪               | 6               |
+----------+-------------+-------+------+------+----------------------+-------+----------------+-------------
    ↪
6 rows selected (0.068 seconds)
```

5.9.7 AutoMQ Load

AutoMQ 是基于云重新设计的云原生 Kafka。通过将存储分离至对象存储，在保持和 Apache Kafka 100% 兼容的前提下，为用户提供高达 10 倍的成本优势以及百倍的弹性优势。通过其创新的共享存储架构，在保证高吞吐、低延迟的性能指标下实现了秒级分区迁移、流量自平衡、秒级自动弹性等能力。

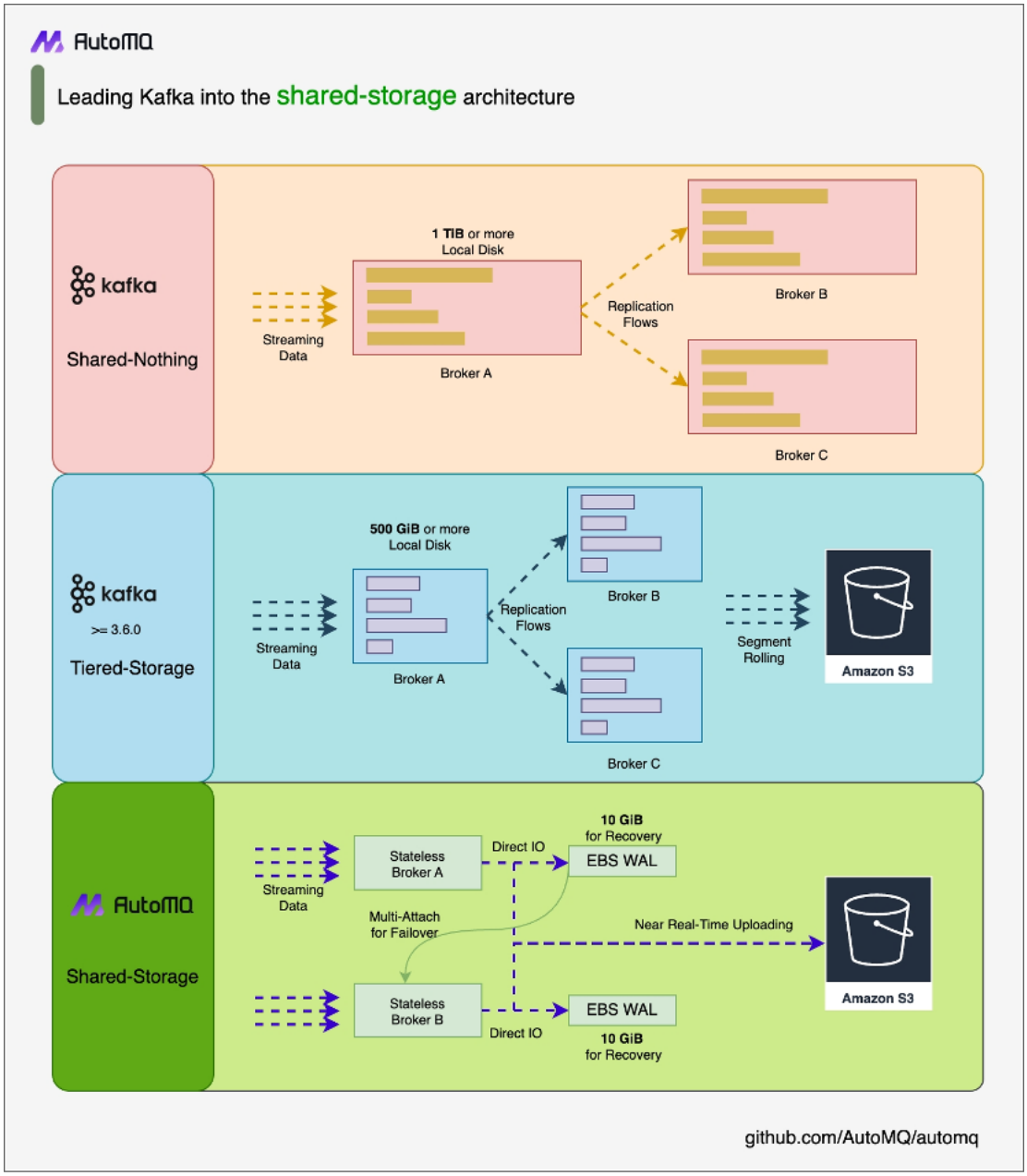


图 246: AutoMQ Storage Architecture

5.9.7.1 环境准备

5.9.7.1.1 准备 Apache Doris 和测试数据

确保当前已准备好可用的 Apache Doris 集群。为了便于演示，我们参考快速开始文档在 Linux 上部署了一套测试用的 Apache Doris 环境。创建库和测试表：

```
create database automq_db;
CREATE TABLE automq_db.users (
                                id bigint NOT NULL,
```

```
                                name string NOT NULL,
                                timestamp string NULL,
                                status string NULL

) DISTRIBUTED BY hash (id) PROPERTIES ('replication_num' = '1');
```

5.9.7.1.2 准备 Kafka 命令行工具

从 AutoMQ Releases 下载最新的 TGZ 包并解压。假设解压目录为 $AUTOMQ_HOME，在本文中将会使用 $AUTOMQ_HOME/bin 下的工具命令来创建主题和生成测试数据。

5.9.7.1.3 准备 AutoMQ 和测试数据

参考 AutoMQ 官方部署文档部署一套可用的集群，确保 AutoMQ 与 Apache Doris 之间保持网络连通。在 AutoMQ 中快速创建一个名为 example_topic 的主题，并向其中写入一条测试 JSON 数据，按照以下步骤操作。

创建 Topic

使用 Apache Kafka 命令行工具创建主题，需要确保当前拥有 Kafka 环境的访问权限并且 Kafka 服务正在运行。以下是创建主题的命令示例：

```
$AUTOMQ_HOME/bin/kafka-topics.sh --create --topic exampleto_topic --bootstrap-server
    ↪ 127.0.0.1:9092  --partitions 1 --replication-factor 1
```

在执行命令时，需要将 topic 和 bootstarp-server 替换为实际使用的 AutoMQ Bootstarp Server 地址。创建完主题后，可以使用以下命令来验证主题是否已成功创建。

```
$AUTOMQ_HOME/bin/kafka-topics.sh --describe example_topic --bootstrap-server 127.0.0.1:9092
```

生成测试数据

生成一条 JSON 格式的测试数据，和前文的表需要对应。

```
{
  "id": 1,
  "name": "测试用户",
  "timestamp": "2023-11-10T12:00:00",
  "status": "active"
}
```

写入测试数据

通过 Kafka 的命令行工具或编程方式将测试数据写入到名为 example_topic 的主题中。下面是一个使用命令行工具的示例：

```
echo '{"id": 1, "name": "测试用户", "timestamp": "2023-11-10T12:00:00", "status": "active"}' | sh
    ↪  kafka-console-producer.sh --broker-list 127.0.0.1:9092 --topic example_topic
```

使用如下命令可以查看刚写入的 topic 数据：

```
sh $AUTOMQ_HOME/bin/kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic example_
    ↪ topic --from-beginning
```

> 注意：在执行命令时，需要将 topic 和 bootstarp-server 替换为实际使用的 AutoMQ Bootstarp Server 地址。

5.9.7.2 创建 Routine Load 导入作业

在 Apache Doris 的命令行中创建一个接收 JSON 数据的 Routine Load 作业，用来持续导入 AutoMQ Kafka topic 中的数据。具体 Routine Load 的参数说明请参考 Doris Routine Load。

```
CREATE ROUTINE LOAD automq_example_load ON users
COLUMNS(id, name, timestamp, status)
PROPERTIES
(
    "format" = "json",
    "jsonpaths" = "[\"$.id\",\"$.name\",\"$.timestamp\",\"$.status\"]"
 )
FROM KAFKA
(
    "kafka_broker_list" = "127.0.0.1:9092",
    "kafka_topic" = "example_topic",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

> 注意：在执行命令时，需要将 kafka_broker_list 替换为实际使用的 AutoMQ Bootstarp Server 地址。

5.9.7.3 验证数据导入

首先，检查 Routine Load 导入作业的状态，确保任务正在运行中。

```
show routine load`\G`;
```

然后查询 Apache Doris 数据库中的相关表，可以看到数据已经被成功导入。

```
select * from users;
+------+--------------+---------------------+--------+
| id   | name         | timestamp           | status |
+------+--------------+---------------------+--------+
|    1 | 测试用户     | 2023-11-10T12:00:00 | active |
```

```
|    2 | 测试用户      | 2023-11-10T12:00:00 | active |
+------+--------------+---------------------+--------+
2 rows in set (0.01 sec)
```

5.9.8 Hive Bitmap UDF

Hive Bitmap UDF 提供了在 Hive 表中生成 bitmap、bitmap 运算等 UDF，Hive 中的 bitmap 与 Doris bitmap 完全一致，Hive 中的 bitmap 可以直接导入 doris。

主要目的：1. 减少数据导入 doris 时间, 除去了构建字典、 bitmap 预聚合等流程；

2. 节省 Hive 存储，使用 bitmap 对数据压缩，减少了存储成本；
3. 提供在 Hive 中 bitmap 的灵活运算，比如：交集、并集、差集运算，计算后的 bitmap 也可以直接导入 doris；

5.9.8.1 使用方法

5.9.8.1.1 在 Hive 中创建 Bitmap 类型表

```
-- 例子：创建 Hive Bitmap 表
CREATE TABLE IF NOT EXISTS `hive_bitmap_table`(
  `k1`   int       COMMENT '',
  `k2`   String    COMMENT '',
  `k3`   String    COMMENT '',
  `uuid` binary    COMMENT 'bitmap'
) comment  'comment'

-- 例子：创建普通 Hive 表
CREATE TABLE IF NOT EXISTS `hive_table`(
    `k1`   int       COMMENT '',
    `k2`   String    COMMENT '',
    `k3`   String    COMMENT '',
    `uuid` int       COMMENT ''
) comment  'comment'
```

5.9.8.1.2 Hive Bitmap UDF 使用：

Hive Bitmap UDF 需要在 Hive/Spark 中使用，首先需要编译 fe 得到 hive-udf-jar-with-dependencies.jar。编译准备工作：如果进行过 ldb 源码编译可直接编译 fe，如果没有进行过 ldb 源码编译，则需要手动安装 thrift，可参考：FE 开发环境搭建 中的编译与安装

```
--clone doris 源码
git clone https://github.com/apache/doris.git
cd doris
```

```
git submodule update --init --recursive
--安装 thrift
--进入 fe 目录
cd fe
--执行 maven 打包命令（fe 的子 module 会全部打包）
mvn package -Dmaven.test.skip=true
--也可以只打 hive-udf module
mvn package -pl hive-udf -am -Dmaven.test.skip=true
```

打包编译完成进入 hive-udf 目录会有 target 目录，里面就会有打包完成的 hive-udf.jar 包

```
-- 加载 hive bitmap udf jar 包 （需要将编译好的 hive-udf jar 包上传至 HDFS）
add jar hdfs://node:9001/hive-udf-jar-with-dependencies.jar;

-- 创建 UDAF 函数
create temporary function to_bitmap as 'org.apache.doris.udf.ToBitmapUDAF' USING JAR 'hdfs://node
    ↪ :9001/hive-udf-jar-with-dependencies.jar';
create temporary function bitmap_union as 'org.apache.doris.udf.BitmapUnionUDAF' USING JAR 'hdfs
    ↪ ://node:9001/hive-udf-jar-with-dependencies.jar';

-- 创建 UDF 函数
create temporary function bitmap_count as 'org.apache.doris.udf.BitmapCountUDF' USING JAR 'hdfs
    ↪ ://node:9001/hive-udf-jar-with-dependencies.jar';
create temporary function bitmap_and as 'org.apache.doris.udf.BitmapAndUDF' USING JAR 'hdfs://
    ↪ node:9001/hive-udf-jar-with-dependencies.jar';
create temporary function bitmap_or as 'org.apache.doris.udf.BitmapOrUDF' USING JAR 'hdfs://node
    ↪ :9001/hive-udf-jar-with-dependencies.jar';
create temporary function bitmap_xor as 'org.apache.doris.udf.BitmapXorUDF' USING JAR 'hdfs://
    ↪ node:9001/hive-udf-jar-with-dependencies.jar';

-- 例子：通过 to_bitmap 生成 bitmap 写入 Hive Bitmap 表
insert into hive_bitmap_table
select
    k1,
    k2,
    k3,
    to_bitmap(uuid) as uuid
from
    hive_table
group by
    k1,
    k2,
    k3

-- 例子：bitmap_count 计算 bitmap 中元素个数
```

```
select k1,k2,k3,bitmap_count(uuid) from hive_bitmap_table

-- 例子：bitmap_union 用于计算分组后的 bitmap 并集
select k1,bitmap_union(uuid) from hive_bitmap_table group by k1
```

5.9.8.2 Hive bitmap 导入 doris

创建 Hive 表指定为 TEXT 格式，此时，对于 Binary 类型，Hive 会以 bash64 编码的字符串形式保存，此时可以通过 Hive Catalog 的形式，直接将位图数据通过 bitmap_from_bash64 函数插入到 Doris 内部。

以下是一个完整的例子：

1. 在 Hive 中创建 Hive 表

```
CREATE TABLE IF NOT EXISTS `test`.`hive_bitmap_table`(
`k1`   int       COMMENT '',
`k2`   String    COMMENT '',
`k3`   String    COMMENT '',
`uuid` binary    COMMENT 'bitmap'
) stored as textfile
```

2. 在 Doris 中创建 Catalog

```
CREATE CATALOG hive PROPERTIES (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9083'
);
```

3. 创建 Doris 内表

```
CREATE TABLE IF NOT EXISTS `test`.`doris_bitmap_table`(
    `k1`   int                   COMMENT '',
    `k2`   String                COMMENT '',
    `k3`   String                COMMENT '',
    `uuid` BITMAP  BITMAP_UNION  COMMENT 'bitmap'
)
AGGREGATE KEY(k1, k2, k3)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

4. 从 Hive 插入数据到 Doris 中

```
insert into doris_bitmap_table select k1, k2, k3, bitmap_from_base64(uuid) from hive.test.hive_
    ↪ bitmap_table;
```

5.9.9 Hive HLL UDF

Hive HLL UDF 提供了在 hive 表中生成 HLL 运算等 UDF，Hive 中的 HLL 与 Doris HLL 完全一致，Hive 中的 HLL 可以通过 Spark HLL Load 导入 Doris。关于 HLL 更多介绍可以参考：使用 HLL 近似去重

函数简介：1. UDAF

```
· to_hll：聚合函数，返回一个 Doris HLL 列，类似于 to_bitmap 函数

· hll_union：聚合函数，功能同 Doris 的 BE 同名函数，计算分组的并集，返回一个 Doris HLL 列，类似于
    ↪ bitmap_union 函数
```

2. UDF

·hll_cardinality：返回添加到 HLL 的不同元素的数量，类似于 bitmap_count 函数

主要目的：1. 减少数据导入 Doris 时间, 除去了构建字典、HLL 预聚合等流程；2. 节省 Hive 存储，使用 HLL 对数据压缩，极大减少了存储成本，相对于 Bitmap 的统计更加节省存储；3. 提供在 Hive 中 HLL 的灵活运算：并集、基数统计，计算后的 HLL 也可以直接导入 Doris；

注意事项：HLL 统计为近似计算有一定误差，大概 1%~2% 左右。

5.9.9.1 使用方法

5.9.9.1.1 在 Hive 中创建 HLL 类型和普通表，往普通表插入测试数据

```
-- 创建一个测试数据库，以 hive_test 为例：
use hive_test;

-- 例子：创建 Hive HLL 表
CREATE TABLE IF NOT EXISTS `hive_hll_table`(
  `k1`   int       COMMENT '',
  `k2`   String    COMMENT '',
  `k3`   String    COMMENT '',
  `uuid` binary    COMMENT 'hll'
) comment  'comment'

-- 例子：创建普通 Hive 表，插入测试数据
CREATE TABLE IF NOT EXISTS `hive_table`(
    `k1`   int       COMMENT '',
    `k2`   String    COMMENT '',
    `k3`   String    COMMENT '',
    `uuid` int       COMMENT ''
) comment  'comment'

insert into hive_table select 1, 'a', 'b', 12345;
insert into hive_table select 1, 'a', 'c', 12345;
```

```
insert into hive_table select 2, 'b', 'c', 23456;
insert into hive_table select 3, 'c', 'd', 34567;
```

5.9.9.1.2 Hive HLL UDF 使用：

Hive HLL UDF 需要在 Hive/Spark 中使用，首先需要编译 fe 得到 hive-udf.jar。编译准备工作：如果进行过 ldb 源码编译可直接编译 fe，如果没有进行过 ldb 源码编译，则需要手动安装 thrift，可参考：FE 开发环境搭建 中的编译与安装

```
--clone doris 源码
git clone https://github.com/apache/doris.git
cd doris
git submodule update --init --recursive

--安装 thrift，已安装可略过
--进入 fe 目录
cd fe

--执行 maven 打包命令（fe 的子 module 会全部打包）
mvn package -Dmaven.test.skip=true
--也可以只打 hive-udf module
mvn package -pl hive-udf -am -Dmaven.test.skip=true

-- 打包编译完成进入 hive-udf 目录会有 target 目录，里面就会有打包完成的 hive-udf.jar 包
-- 需要将编译好的 hive-udf.jar 包上传至 HDFS，这里以传至 hdfs 的根目录为示例：
hdfs dfs -put hive-udf/target/hive-udf.jar /
```

下面进入 Hive 中进行 SQL 语句操作：

```
-- 加载 hive hll udf jar 包，根据实际情况更改 hostname 和 port
add jar hdfs://hostname:port/hive-udf.jar;

-- 创建 UDAF 函数
create temporary function to_hll as 'org.apache.doris.udf.ToHllUDAF' USING JAR 'hdfs://hostname:
    ↪ port/hive-udf.jar';
create temporary function hll_union as 'org.apache.doris.udf.HllUnionUDAF' USING JAR 'hdfs://
    ↪ hostname:port/hive-udf.jar';


-- 创建 UDF 函数
create temporary function hll_cardinality as 'org.apache.doris.udf.HllCardinalityUDF' USING JAR '
    ↪ hdfs://node:9000/hive-udf.jar';


-- 例子：通过 to_hll 这个 UDAF 进行聚合生成 hll 写入 Hive HLL 表
insert into hive_hll_table
```

```
select
    k1,
    k2,
    k3,
    to_hll(uuid) as uuid
from
    hive_table
group by
    k1,
    k2,
    k3

-- 例子：hll_cardinality 计算 hll 中元素个数
select k1, k2, k3, hll_cardinality(uuid) from hive_hll_table;
+-----+-----+-----+------+
| k1  | k2  | k3  | _c3  |
+-----+-----+-----+------+
| 1   | a   | b   | 1    |
| 1   | a   | c   | 1    |
| 2   | b   | c   | 1    |
| 3   | c   | d   | 1    |
+-----+-----+-----+------+

-- 例子：hll_union 用于计算分组后的 hll 并集，将返回 3 行
select k1, hll_union(uuid) from hive_hll_table group by k1;

-- 例子：也可以合并后继续统计
select k3, hll_cardinality(hll_union(uuid)) from hive_hll_table group by k3;
+-----+------+
| k3  | _c1  |
+-----+------+
| b   | 1    |
| c   | 2    |
| d   | 1    |
+-----+------+
```

5.9.9.1.3 Hive HLL UDF 说明

5.9.9.2 Hive HLL 导入 doris

5.9.9.2.1 方法一：Catalog（推荐）

创建 Hive 表指定为 TEXT 格式，对于 Binary 类型，Hive 会以 base64 编码的字符串形式保存，此时可以通过 Hive Catalog 的形式，直接将 HLL 数据通过hll_from_base64 函数插入到 Doris 内部。

以下是一个完整的例子：

1. 在 Hive 中创建 Hive 表

```
CREATE TABLE IF NOT EXISTS `hive_hll_table`(
`k1`   int       COMMENT '',
`k2`   String    COMMENT '',
`k3`   String    COMMENT '',
`uuid` binary    COMMENT 'hll'
) stored as textfile

-- 然后可以沿用前面的步骤基于普通表使用 to_hll 函数往 hive_hll_table 插入数据，这里不再赘述
```

2. 在 Doris 中创建 Catalog

```
CREATE CATALOG hive PROPERTIES (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://127.0.0.1:9083'
);
```

3. 创建 Doris 内表

```
CREATE TABLE IF NOT EXISTS `doris_test`.`doris_hll_table`(
    `k1`   int                  COMMENT '',
    `k2`   varchar(10)          COMMENT '',
    `k3`   varchar(10)          COMMENT '',
    `uuid` HLL  HLL_UNION  COMMENT 'hll'
)
AGGREGATE KEY(k1, k2, k3)
DISTRIBUTED BY HASH(`k1`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);
```

4. 从 Hive 插入数据到 Doris 中

```
insert into doris_hll_table select k1, k2, k3, hll_from_base64(uuid) from hive.hive_test.hive_hll
    ↪ _table;

-- 可以查看导入后的数据，结合 hll_to_base64 进行解码
select *, hll_to_base64(uuid) from doris_hll_table;
+------+------+------+------+---------------------+
| k1   | k2   | k3   | uuid | hll_to_base64(uuid) |
```

```
+------+------+------+------+---------------------+
|    1 | a    | b    | NULL | AQFw+a9MhpKhoQ==    |
|    1 | a    | c    | NULL | AQFw+a9MhpKhoQ==    |
|    2 | b    | c    | NULL | AQGyB7kbWBxh+A==    |
|    3 | c    | d    | NULL | AQFYbJB5VpNBhg==    |
+------+------+------+------+---------------------+

-- 也可以进一步使用 Doris 原生的 HLL 函数进行统计，可以看到和前面在 Hive 中统计的结果一致
select k3, hll_cardinality(hll_union(uuid)) from doris_hll_table group by k3;
+------+----------------------------------+
| k3   | hll_cardinality(hll_union(uuid)) |
+------+----------------------------------+
| b    |                                1 |
| d    |                                1 |
| c    |                                2 |
+------+----------------------------------+

-- 此时，查外表的数据，也就是查导入前的数据进行统计、对比也能校验数据正确性
select k3, hll_cardinality(hll_union(hll_from_base64(uuid))) from hive.hive_test.hive_hll_table
    ↪ group by k3;
+------+-------------------------------------------------------+
| k3   | hll_cardinality(hll_union(hll_from_base64(uuid))) |
+------+-------------------------------------------------------+
| d    |                                                     1 |
| b    |                                                     1 |
| c    |                                                     2 |
+------+-------------------------------------------------------+
```

5.9.9.2.2 方法二：Spark Load

详见：Spark Load -> 基本操作 -> 创建导入 (示例 3：上游数据源是 hive binary 类型情况)

6 常见问题

6.1 常见运维问题

本文档主要用于记录 Doris 使用过程中的运维常见问题。会不定期更新。

文中的出现的 BE 二进制文件名称 `doris_be`，在之前的版本中为 `palo_be`。

6.1.0.1 Q1. 通过 DECOMMISSION 下线 BE 节点时，为什么总会有部分 tablet 残留？

在下线过程中，通过 show backends 查看下线节点的 tabletNum，会观察到 tabletNum 数量在减少，说明数据分片正在从这个节点迁移走。当数量减到 0 时，系统会自动删除这个节点。但某些情况下，tabletNum 下降到一定数值后就不变化。这通常可能有以下两种原因：

1. 这些 tablet 属于刚被删除的表、分区或物化视图。而刚被删除的对象会保留在回收站中。而下线逻辑不会处理这些分片。可以通过修改 FE 的配置参数 catalog_trash_expire_second 来修改对象在回收站中驻留的时间。当对象从回收站中被删除后，这些 tablet 就会被处理了。
2. 这些 tablet 的迁移任务出现了问题。此时需要通过 `show proc "/cluster_balance"` 来查看具体任务的错误了。

对于以上情况，可以先通过 `show proc "/cluster_health/tablet_health";` 查看集群是否还有 unhealthy 的分片，如果为 0，则可以直接通过 drop backend 语句删除这个 BE。否则，还需要具体查看不健康分片的副本情况。

6.1.0.2 Q2. priority_networks 应该如何设置？

priority_networks 是 FE、 BE 都有的配置参数。这个参数主要用于帮助系统选择正确的网卡 IP 作为自己的 IP。建议任何情况下，都显式的设置这个参数，以防止后续机器增加新网卡导致 IP 选择不正确的问题。

priority_networks 的值是 CIDR 格式表示的。分为两部分，第一部分是点分十进制的 IP 地址，第二部分是一个前缀长度。比如 10.168.1.0/8 会匹配所有 10.xx.xx.xx 的 IP 地址，而 10.168.1.0/16 会匹配所有 10.168.xx.xx 的 IP 地址。

之所以使用 CIDR 格式而不是直接指定一个具体 IP，是为了保证所有节点都可以使用统一的配置值。比如有两个节点：10.168.10.1 和 10.168.10.2，则我们可以使用 10.168.10.0/24 来作为 priority_networks 的值。

6.1.0.3 Q3. FE 的 Master、 Follower、 Observer 都是什么？

首先明确一点，FE 只有两种角色：Follower 和 Observer。而 Master 只是一组 Follower 节点中选择出来的一个 FE。Master 可以看成是一种特殊的 Follower。所以当我们被问及一个集群有多少 FE，都是什么角色时，正确的回答当时应该是所有 FE 节点的个数，以及 Follower 角色的个数和 Observer 角色的个数。

所有 Follower 角色的 FE 节点会组成一个可选择组，类似 Paxos 一致性协议里的组概念。组内会选举出一个 Follower 作为 Master。当 Master 挂了，会自动选择新的 Follower 作为 Master。而 Observer 不会参与选举，因此 Observer 也不会成为 Master。

一条元数据日志需要在多数 Follower 节点写入成功，才算成功。比如 3 个 FE，2 个写入成功才可以。这也是为什么 Follower 角色的个数需要是奇数的原因。

Observer 角色和这个单词的含义一样，仅仅作为观察者来同步已经成功写入的元数据日志，并且提供元数据读服务。他不会参与多数写的逻辑。

通常情况下，可以部署 1 Follower + 2 Observer 或者 3 Follower + N Observer。前者运维简单，几乎不会出现 Follower 之间的一致性协议导致这种复杂错误情况（企业大多使用这种方式）。后者可以保证元数据写的高可用，如果是高并发查询场景，可以适当增加 Observer。

6.1.0.4 Q4. 节点新增加了新的磁盘，为什么数据没有均衡到新的磁盘上？

当前 Doris 的均衡策略是以节点为单位的。也就是说，是按照节点整体的负载指标（分片数量和总磁盘利用率）来判断集群负载。并且将数据分片从高负载节点迁移到低负载节点。如果每个节点都增加了一块磁盘，则从节点整体角度看，负载并没有改变，所以无法触发均衡逻辑。

此外，Doris 目前并不支持单个节点内部，各个磁盘间的均衡操作。所以新增磁盘后，不会将数据均衡到新的磁盘。

但是，数据在节点之间迁移时，Doris 会考虑磁盘的因素。比如一个分片从 A 节点迁移到 B 节点，会优先选择 B 节点中，磁盘空间利用率较低的磁盘。

这里我们提供 3 种方式解决这个问题：

1. 重建新表

通过 create table like 语句建立新表，然后使用 insert into select 的方式将数据从老表同步到新表。因为创建新表时，新表的数据分片会分布在新的磁盘中，从而数据也会写入新的磁盘。这种方式适用于数据量较小的情况（几十 GB 以内）。

2. 通过 Decommission 命令

decommission 命令用于安全下线一个 BE 节点。该命令会先将该节点上的数据分片迁移到其他节点，然后在删除该节点。前面说过，在数据迁移时，会优先考虑磁盘利用率低的磁盘，因此该方式可以“强制”让数据迁移到其他节点的磁盘上。当数据迁移完成后，我们在 cancel 掉这个 decommission 操作，这样，数据又会重新均衡回这个节点。当我们对所有 BE 节点都执行一遍上述步骤后，数据将会均匀的分布在所有节点的所有磁盘上。

注意，在执行 decommission 命令前，先执行以下命令，以避免节点下线完成后被删除。

```
admin set frontend config("drop_backend_after_decommission" = "false");
```

3. 使用 API 手动迁移数据

Doris 提供了HTTP API，可以手动指定一个磁盘上的数据分片迁移到另一个磁盘上。

6.1.0.5 Q5. 如何正确阅读 FE/BE 日志？

很多情况下我们需要通过日志来排查问题。这里说明一下 FE/BE 日志的格式和查看方式。

1. FE

FE 日志主要有：

- fe.log：主日志。包括除 fe.out 外的所有内容。
- fe.warn.log：主日志的子集，仅记录 WARN 和 ERROR 级别的日志。
- fe.out：标准/错误输出的日志（stdout 和 stderr）。
- fe.audit.log：审计日志，记录这个 FE 接收的所有 SQL 请求。

一条典型的 FE 日志如下：

```
2021-09-16 23:13:22,502 INFO (tablet scheduler|43) [BeLoadRebalancer.
    ↪ selectAlternativeTabletsForCluster():85] cluster is balance: default_cluster with
    ↪ medium: HDD. skip
```

- `2021-09-16 23:13:22,502`：日志时间。
- `INFO`：日志级别，默认是`INFO`。
- `(tablet scheduler|43)`：线程名称和线程 id。通过线程 id，就可以查看这个线程上下文信息，方面排查这个线程发生的事情。
- `BeLoadRebalancer.selectAlternativeTabletsForCluster():85`：类名、方法名和代码行号。
- `cluster is balance xxx`：日志内容。

通常情况下我们主要查看 fe.log 日志。特殊情况下，有些日志可能输出到了 fe.out 中。

2. BE

BE 日志主要有：

- be.INFO：主日志。这其实是个软连，连接到最新的一个 be.INFO.xxxx 上。
- be.WARNING：主日志的子集，仅记录 WARN 和 FATAL 级别的日志。这其实是个软连，连接到最新的一个 be.WARN.xxxx 上。
- be.out：标准/错误输出的日志（ stdout 和 stderr ）。

一条典型的 BE 日志如下：

```
I0916 23:21:22.038795 28087 task_worker_pool.cpp:1594] finish report TASK. master host:
    ↪ 10.10.10.10, port: 9222
```

- `I0916 23:21:22.038795`：日志等级和日期时间。大写字母 I 表示 INFO，W 表示 WARN，F 表示 FATAL。
- `28087`：线程 id。通过线程 id，就可以查看这个线程上下文信息，方面排查这个线程发生的事情。
- `task_worker_pool.cpp:1594`：代码文件和行号。
- `finish report TASK xxx`：日志内容。

通常情况下我们主要查看 be.INFO 日志。特殊情况下，如 BE 宕机，则需要查看 be.out。

6.1.0.6 Q6. FE/BE 节点挂了应该如何排查原因？

1. BE

BE 进程是 C/C++ 进程，可能会因为一些程序 Bug（内存越界，非法地址访问等）或 Out Of Memory（OOM）导致进程挂掉。此时我们可以通过以下几个步骤查看错误原因：

1. 查看 be.out

   BE 进程实现了在程序因异常情况退出时，会打印当前的错误堆栈到 be.out 里（注意是 be.out，不是 be.INFO 或 be.WARNING）。通过错误堆栈，通常能够大致获悉程序出错的位置。

   注意，如果 be.out 中出现错误堆栈，通常情况下是因为程序 bug，普通用户可能无法自行解决，欢迎前往微信群、github discussion 或 dev 邮件组寻求帮助，并贴出对应的错误堆栈，以便快速排查问题。

2. dmesg

   如果 be.out 没有堆栈信息，则大概率是因为 OOM 被系统强制 kill 掉了。此时可以通过 dmesg -T 这个命令查看 linux 系统日志，如果最后出现 Memory cgroup out of memory: Kill process 7187 (doris_be) score 1007 or sacrifice child 类似的日志，则说明是 OOM 导致的。

   内存问题可能有多方面原因，如大查询、导入、compaction 等。Doris 也在不断优化内存使用。欢迎前往微信群、github discussion 或 dev 邮件组寻求帮助。

3. 查看 be.INFO 中是否有 F 开头的日志。

   F 开头的日志是 Fatal 日志。如 F0916，表示 9 月 16 号的 Fatal 日志。Fatal 日志通常表示程序断言错误，断言错误会直接导致进程退出（说明程序出现了 Bug）。欢迎前往微信群、github discussion 或 dev 邮件组寻求帮助。

4. FE

FE 是 java 进程，健壮程度要优于 C/C++ 程序。通常 FE 挂掉的原因可能是 OOM（Out-of-Memory）或者是元数据写入失败。这些错误通常在 fe.log 或者 fe.out 中有错误堆栈。需要根据错误堆栈信息进一步排查。

6.1.0.7 Q7. 关于数据目录 SSD 和 HDD 的配置，建表有时候会遇到报错 `Failed to find enough host with storage medium and tag`

Doris 支持一个 BE 节点配置多个存储路径。通常情况下，每块盘配置一个存储路径即可。同时，Doris 支持指定路径的存储介质属性，如 SSD 或 HDD。SSD 代表高速存储设备，HDD 代表低速存储设备。

如果集群只有一种介质比如都是 HDD 或者都是 SSD，最佳实践是不用在 be.conf 中显式指定介质属性。如果遇到上述报错`Failed to find enough host with storage medium and tag`，一般是因为 be.conf 中只配置了 SSD 的介质，而建表阶段中显式指定了`properties {"storage_medium" = "hdd"}`；同理如果 be.conf 只配置了 HDD 的介质，而而建表阶段中显式指定了`properties {"storage_medium" = "ssd"}`也会出现上述错误。解决方案可以修改建表的 properties 参数与配置匹配；或者将 be.conf 中 SSD/HDD 的显式配置去掉即可。

通过指定路径的存储介质属性，我们可以利用 Doris 的冷热数据分区存储功能，在分区级别将热数据存储在 SSD 中，而冷数据会自动转移到 HDD 中。

需要注意的是，Doris 并不会自动感知存储路径所在磁盘的实际存储介质类型。这个类型需要用户在路径配置中显式的表示。比如路径“/path/to/data1.SSD”即表示这个路径是 SSD 存储介质。而“data1.SSD”就是实际的目录名称。Doris 是根据目录名称后面的“.SSD”后缀来确定存储介质类型的，而不是实际的存储介质类型。也就是说，用户可以指定任意路径为 SSD 存储介质，而 Doris 仅识别目录后缀，不会去判断存储介质是否匹配。如果不写后缀，则默认为 HDD。

换句话说，“.HDD”和“.SSD”只是用于标识存储目录“相对”的“低速”和“高速”之分，而并不是标识实际的存储介质类型。所以如果 BE 节点上的存储路径没有介质区别，则无需填写后缀。

6.1.0.8 Q8. 多个 FE，在使用 Nginx 实现 web UI 负载均衡时，无法登录

Doris 可以部署多个 FE，在访问 Web UI 的时候，如果使用 Nginx 进行负载均衡，因为 Session 问题会出现不停的提示要重新登录，这个问题其实是 Session 共享的问题，Nginx 提供了集中 Session 共享的解决方案，这里我们使用的是 nginx 中的 ip_hash 技术，ip_hash 能够将某个 ip 的请求定向到同一台后端，这样一来这个 ip 下的某个客户端和某个后端就能建立起稳固的 session，ip_hash 是在 upstream 配置中定义的：

```
upstream  doris.com {
   server    172.22.197.238:8030 weight=3;
   server    172.22.197.239:8030 weight=4;
   server    172.22.197.240:8030 weight=4;
   ip_hash;
}
```

完整的 Nginx 示例配置如下：

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;


include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';

    access_log  /var/log/nginx/access.log  main;

    sendfile            on;
    tcp_nopush          on;
    tcp_nodelay         on;
    keepalive_timeout   65;
    types_hash_max_size 2048;

    include             /etc/nginx/mime.types;
    default_type        application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/ngx_core_module.html#include
    # for more information.
```

```
    include /etc/nginx/conf.d/*.conf;
    #include /etc/nginx/custom/*.conf;
    upstream  doris.com {
      server    172.22.197.238:8030 weight=3;
      server    172.22.197.239:8030 weight=4;
      server    172.22.197.240:8030 weight=4;
      ip_hash;
    }

    server {
        listen       80;
        server_name  gaia-pro-bigdata-fe02;
        if ($request_uri ~ _load) {
           return 307 http://$host$request_uri ;
        }

        location / {
            proxy_pass http://doris.com;
            proxy_redirect default;
        }
        error_page   500 502 503 504  /50x.html;
        location = /50x.html {
            root   html;
        }
    }
}
```

6.1.0.9 Q9. FE 启动失败，fe.log 中一直滚动“wait catalog to be ready. FE type UNKNOWN”

这种问题通常有两个原因：

1. 本次 FE 启动时获取到的本机 IP 和上次启动不一致，通常是因为没有正确设置 `priority_network` 而导致 FE 启动时匹配到了错误的 IP 地址。需修改 `priority_network` 后重启 FE。
2. 集群内多数 Follower FE 节点未启动。比如有 3 个 Follower，只启动了一个。此时需要将另外至少一个 FE 也启动，FE 可选举组方能选举出 Master 已提供服务。

如果以上情况都不能解决，可以按照 Doris 官网文档中的元数据运维文档进行恢复。

6.1.0.10 Q10. Lost connection to MySQL server at ‘reading initial communication packet’, system error: 0

如果使用 MySQL 客户端连接 Doris 时出现如下问题，这通常是因为编译 FE 时使用的 jdk 版本和运行 FE 时使用的 jdk 版本不同导致的。注意使用 docker 编译镜像编译时，默认的 JDK 版本是 openjdk 11，可以通过命令切换到 openjdk 8（详见编译文档）。

6.1.0.11 Q11. recoveryTracker should overlap or follow on disk last VLSN of 4,422,880 recoveryFirst= 4,422,882 UNEXPECTED_STATE_FATAL

有时重启 FE，会出现如上错误（通常只会出现在多 Follower 的情况下）。并且错误中的两个数值相差 2。导致 FE 启动失败。

这是 bdbje 的一个 bug，尚未解决。遇到这种情况，只能通过元数据运维文档 中的故障恢复进行操作来恢复元数据了。

6.1.0.12 Q12. Doris 编译安装 JDK 版本不兼容问题

在自己使用 Docker 编译 Doris 的时候，编译完成安装以后启动 FE，出现 `java.lang.Suchmethoderror: java.nio ↪ . ByteBuffer. limit (I)Ljava/nio/ByteBuffer;` 异常信息，这是因为 Docker 里默认是 JDK 11，如果你的安装环境是使用 JDK8，需要在 Docker 里 JDK 环境切换成 JDK8，具体切换方法参照编译文档

6.1.0.13 Q13. 本地启动 FE 或者启动单元测试报错 Cannot find external parser table action_table.dat

执行如下命令

```
cd fe && mvn clean install -DskipTests
```

如果还报同样的错误，手动执行如下命令

```
cp fe-core/target/generated-sources/cup/org/apache/doris/analysis/action_table.dat fe-core/target
    ↪ /classes/org/apache/doris/analysis
```

6.1.0.14 Q14. Doris 升级到 1.0 以后版本通过 ODBC 访问 MySQL 外表报错 `Failed to set ciphers to use (2026)`

这个问题出现在 doris 升级到 1.0 版本以后，且使用 Connector/ODBC 8.0.x 以上版本，Connector/ODBC 8.0.x 有多种获取方式，比如通过 yum 安装的方式获取的 `/usr/lib64/libmyodbc8w.so` 依赖的是 `libssl.so.10` 和 `libcrypto ↪ .so.10` 而 doris 1.0 以后版本中 openssl 已经升级到 1.1 且内置在 doris 二进制包中，因此会导致 openssl 的冲突进而出现类似如下的错误

```
ERROR 1105 (HY000): errCode = 2, detailMessage = driver connect Error: HY000 [MySQL][ODBC 8.0(w)
    ↪ Driver]SSL connection error: Failed to set ciphers to use (2026)
```

解决方式是使用`Connector/ODBC 8.0.28` 版本的 ODBC Connector，并且在操作系统处选择 `Linux - Generic`, 这个版本的 ODBC Driver 使用 openssl 1.1 版本。或者使用低版本的 ODBC Connector，比如Connector/ODBC 5.3.14。具体使用方式见 ODBC 外表使用文档。

可以通过如下方式验证 MySQL ODBC Driver 使用的 openssl 版本

```
ldd /path/to/libmyodbc8w.so |grep libssl.so
```

如果输出包含 `libssl.so.10` 则使用过程中可能出现问题，如果包含`libssl.so.1.1` 则与 doris 1.0 兼容

6.1.0.15 Q15. 升级到 1.2 版本，BE NoClassDefFoundError 问题启动失败

> Java UDF 依赖错误从 Doris 1.2 版本后开始支持

如果升级后启动 be 出现下面这种 Java NoClassDefFoundError 错误

```
Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/doris/udf/JniUtil
Caused by: java.lang.ClassNotFoundException: org.apache.doris.udf.JniUtil
```

需要从官网下载 apache-doris-java-udf-jar-with-dependencies-1.2.0 的 Java UDF 函数依赖包，放到 BE 安装目录下的 lib 目录，然后重新启动 BE

6.1.0.16 Q16. 升级到 1.2 版本，BE 启动显示 Failed to initialize JNI 问题

> Java 环境问题从 Doris 1.2 版本后开始支持

如果升级后启动 BE 出现下面这种 Failed to initialize JNI 错误

```
Failed to initialize JNI: Failed to find the library libjvm.so.
```

需要在系统设置 JAVA_HOME 环境变量，或者在 be.conf 中设置 JAVA_HOME 变量，然后重新启动 BE 节点。

6.2 数据操作问题

本文档主要用于记录 Doris 使用过程中的数据操作常见问题。会不定期更新。

6.2.0.1 Q1. 使用 Stream Load 访问 FE 的公网地址导入数据，被重定向到内网 IP？

当 stream load 的连接目标为 FE 的 http 端口时，FE 仅会随机选择一台 BE 节点做 http 307 redirect 操作，因此用户的请求实际是发送给 FE 指派的某一个 BE 的。而 redirect 返回的是 BE 的 ip，也即内网 IP。所以如果你是通过 FE 的公网 IP 发送的请求，很有可能因为 redirect 到内网地址而无法连接。

通常的做法，一种是确保自己能够访问内网 IP 地址，或者是给所有 BE 上层架设一个负载均衡，然后直接将 stream load 请求发送到负载均衡器上，由负载均衡将请求透传到 BE 节点。

6.2.0.2 Q2. Doris 是否支持修改列名？

在 1.2.0 版本之后，开启 "light_schema_change"="true" 选项时，可以支持修改列名。

在 1.2.0 版本之前或未开启 "light_schema_change"="true" 选项时，不支持修改列名，原因如下：

Doris 支持修改数据库名、表名、分区名、物化视图（Rollup）名称，以及列的类型、注释、默认值等等。但遗憾的是，目前不支持修改列名。

因为一些历史原因，目前列名称是直接写入到数据文件中的。Doris 在查询时，也是通过列名查找到对应的列的。所以修改列名不仅是简单的元数据修改，还会涉及到数据的重写，是一个非常重的操作。

我们不排除后续通过一些兼容手段来支持轻量化的列名修改操作。

6.2.0.3 Q3. Unique Key 模型的表是否支持创建物化视图？

不支持。

Unique Key 模型的表是一个对业务比较友好的表，因为其特有的按照主键去重的功能，能够很方便的同步数据频繁变更的业务数据库。因此，很多用户在将数据接入到 Doris 时，会首先考虑使用 Unique Key 模型。

但遗憾的是，Unique Key 模型的表是无法建立物化视图的。原因在于，物化视图的本质，是通过预计算来将数据“预先算好”，这样在查询时直接返回已经计算好的数据，来加速查询。在物化视图中，“预计算”的数据通常是一些聚合指标，比如求和、求 count。这时，如果数据发生变更，如 update 或 delete，因为预计算的数据已经丢失了明细信息，因此无法同步的进行更新。比如一个求和值 5，可能是 1+4，也可能是 2+3。因为明细信息的丢失，我们无法区分这个求和值是如何计算出来的，因此也就无法满足更新的需求。

6.2.0.4 Q4. tablet writer write failed, tablet_id=27306172, txn_id=28573520, err=-235 or -238

这个错误通常发生在数据导入操作中。错误码为 -235。这个错误的含义是，对应 tablet 的数据版本超过了最大限制（默认 500，由 BE 参数 `max_tablet_version_num` 控制），后续写入将被拒绝。比如问题中这个错误，即表示 27306172 这个 tablet 的数据版本超过了限制。

这个错误通常是因为导入的频率过高，大于后台数据的 compaction 速度，导致版本堆积并最终超过了限制。此时，我们可以先通过 show tablet 27306172 语句，然后执行结果中的 show proc 语句，查看 tablet 各个副本的情况。结果中的 versionCount 即表示版本数量。如果发现某个副本的版本数量过多，则需要降低导入频率或停止导入，并观察版本数是否有下降。如果停止导入后，版本数依然没有下降，则需要去对应的 BE 节点查看 be.INFO 日志，搜索 tablet id 以及 compaction 关键词，检查 compaction 是否正常运行。关于 compaction 调优相关，可以参阅 ApacheDoris 公众号文章：Doris 最佳实践 -Compaction 调优 (3)

-238 错误通常出现在同一批导入数据量过大的情况，从而导致某一个 tablet 的 Segment 文件过多（默认是 200，由 BE 参数 `max_segment_num_per_rowset` 控制）。此时建议减少一批次导入的数据量，或者适当提高 BE 配置参数值来解决。在 2.0 版本及以后，可以通过打开 segment compaction 功能来减少 Segment 文件数量 (BE config 中 `enable_segcompaction=true`)。

6.2.0.5 Q5. tablet 110309738 has few replicas: 1, alive backends: [10003]

这个错误可能发生在查询或者导入操作中。通常意味着对应 tablet 的副本出现了异常。

此时，可以先通过 show backends 命令检查 BE 节点是否有宕机，如 isAlive 字段为 false，或者 LastStartTime 是最近的某个时间（表示最近重启过）。如果 BE 有宕机，则需要去 BE 对应的节点，查看 be.out 日志。如果 BE 是因为异常原因宕机，通常 be.out 中会打印异常堆栈，帮助排查问题。如果 be.out 中没有错误堆栈。则可以通过 linux 命令 dmesg -T 检查是否是因为 OOM 导致进程被系统 kill 掉。

如果没有 BE 节点宕机，则需要通过 show tablet 110309738 语句，然后执行结果中的 show proc 语句，查看 tablet 各个副本的情况，进一步排查。

6.2.0.6 Q6. disk xxxxx on backend xxx exceed limit usage

通常出现在导入、Alter 等操作中。这个错误意味着对应 BE 的对应磁盘的使用量超过了阈值（默认 95%）此时可以先通过 show backends 命令，其中 MaxDiskUsedPct 展示的是对应 BE 上，使用率最高的那块磁盘的使用率，如果超过 95%，则会报这个错误。

此时需要前往对应 BE 节点，查看数据目录下的使用量情况。其中 trash 目录和 snapshot 目录可以手动清理以释放空间。如果是 data 目录占用较大，则需要考虑删除部分数据以释放空间了。具体可以参阅磁盘空间管理。

6.2.0.7 Q7. 通过 Java 程序调用 stream load 导入数据，在一批次数据量较大时，可能会报错 Broken Pipe

除了 Broken Pipe 外，还可能出现一些其他的奇怪的错误。

这个情况通常出现在开启 httpv2 后。因为 httpv2 是使用 spring boot 实现的 http 服务，并且使用 tomcat 作为默认内置容器。但是 tomcat 对 307 转发的处理似乎有些问题，所以后面将内置容器修改为了 jetty。此外，在 java 程序中的 apache http client 的版本需要使用 4.5.13 以后的版本。之前的版本，对转发的处理也存在一些问题。

所以这个问题可以有两种解决方式：

1. 关闭 httpv2

在 fe.conf 中添加 enable_http_server_v2=false 后重启 FE。但是这样无法再使用新版 UI 界面，并且之后的一些基于 httpv2 的新接口也无法使用。（正常的导入查询不受影响）。

2. 升级

可以升级到 Doris 0.15 及之后的版本，已修复这个问题。

6.2.0.8 Q8. 执行导入、查询时报错 -214

在执行导入、查询等操作时，可能会遇到如下错误：

```
failed to initialize storage reader. tablet=63416.1050661139.aa4d304e7a7aff9c-f0fa7579928c85a0,
    ↪ res=-214, backend=192.168.100.10
```

-214 错误意味着对应 tablet 的数据版本缺失。比如如上错误，表示 tablet 63416 在 192.168.100.10 这个 BE 上的副本的数据版本有缺失。（可能还有其他类似错误码，都可以用如下方式进行排查和修复）。

通常情况下，如果你的数据是多副本的，那么系统会自动修复这些有问题的副本。可以通过以下步骤进行排查：

首先通过 `show tablet 63416` 语句并执行结果中的 `show proc xxx` 语句来查看对应 tablet 的各个副本情况。通常我们需要关心 `Version` 这一列的数据。

正常情况下，一个 tablet 的多个副本的 Version 应该是相同的。并且和对应分区的 VisibleVersion 版本相同。

你可以通过 `show partitions from tblx` 来查看对应的分区版本（tablet 对应的分区可以在 `show tablet` 语句中获取。）

同时，你也可以访问 `show proc` 语句中的 CompactionStatus 列中的 URL（在浏览器打开即可）来查看更具体的版本信息，来检查具体丢失的是哪些版本。

如果长时间没有自动修复，则需要通过 show proc "/cluster_balance" 语句，查看当前系统正在执行的 tablet 修复和调度任务。可能是因为有大量的 tablet 在等待被调度，导致修复时间较长。可以关注 pending_tablets 和 running_tablets 中的记录。

更进一步的，可以通过 admin repair 语句来指定优先修复某个表或分区，具体可以参阅 help admin repair;

如果依然无法修复，那么在多副本的情况下，我们使用 admin set replica status 命令强制将有问题的副本下线。具体可参阅 help admin set replica status 中将副本状态置为 bad 的示例。(置为 bad 后，副本将不会再被访问。并且会后续自动修复。但在操作前，应先确保其他副本是正常的)

6.2.0.9 Q9. Not connected to 192.168.100.1:8060 yet, server_id=384

在导入或者查询时，我们可能遇到这个错误。如果你去对应的 BE 日志中查看，也可能会找到类似错误。

这是一个 RPC 错误，通常有两种可能：1. 对应的 BE 节点宕机。2. rpc 拥塞或其他错误。

如果是 BE 节点宕机，则需要查看具体的宕机原因。这里只讨论 rpc 拥塞的问题。

一种情况是 OVERCROWDED，即表示 rpc 源端有大量未发送的数据超过了阈值。BE 有两个参数与之相关：

1. brpc_socket_max_unwritten_bytes：默认 1GB，如果未发送数据超过这个值，则会报错。可以适当修改这个值以避免 OVERCROWDED 错误。(但这个治标不治本，本质上还是有拥塞发生)。
2. tablet_writer_ignore_eovercrowded：默认为 false。如果设为 true，则 Doris 会忽略导入过程中出现的 OVERCROWDED 错误。这个参数主要为了避免导入失败，以提高导入的稳定性。

第二种是 rpc 的包大小超过 max_body_size。如果查询中带有超大 String 类型，或者 bitmap 类型时，可能出现这个问题。可以通过修改以下 BE 参数规避：

```
brpc_max_body_size：默认 3GB.
```

6.2.0.10 Q10. Broker load org.apache.thrift.transport.TTransportException: java.net.SocketException: Broken pipe

出现这个问题的原因可能是到从外部存储 (例如 HDFS) 导入数据的时候，因为目录下文件太多，列出文件目录的时间太长，这里 Broker RPC Timeout 默认是 10 秒，这里需要适当调整超时时间。

修改 fe.conf 配置文件，添加下面的参数：

```
broker_timeout_ms = 10000
###这里默认是10秒，需要适当加大这个参数
```

这里添加参数，需要重启 FE 服务。

6.2.0.11 Q11.Routine load ReasonOfStateChanged: ErrorReason{code=errCode = 104, msg='be 10004 abort task with reason: fetch failed due to requested offset not available on the broker: Broker: Offset out of range'}

出现这个问题的原因是因为 kafka 的清理策略默认为 7 天，当某个 routine load 任务因为某种原因导致任务暂停，长时间没有恢复，当重新恢复任务的时候 routine load 记录了消费的 offset，而 kafka 的清理策略已经清理了对应的 offset，就会出现这个问题

所以这个问题可以用 alter routine load 解决方式：

查看 kafka 最小的 offset , 使用 ALTER ROUTINE LOAD 命令修改 offset，重新恢复任务即可

```
ALTER ROUTINE LOAD FOR db.tb
FROM kafka
(
 "kafka_partitions" = "0",
 "kafka_offsets" = "xxx",
 "property.group.id" = "xxx"
);
```

6.2.0.12 Q12. ERROR 1105 (HY000): errCode = 2, detailMessage = (192.168.90.91)[CANCELLED][INTERNAL_ERROR]error setting certificate verify locations: CAfile: /etc/ssl/certs/ca-certificates.crt CApath: none

```
yum install -y ca-certificates
ln -s /etc/pki/ca-trust/extracted/openssl/ca-bundle.trust.crt /etc/ssl/certs/ca-certificates.crt
```

6.2.0.13 Q13. create partition failed. partition numbers will exceed limit variable max_auto_partition_num

对自动分区表导入数据时，为防止意外创建过多分区，我们使用了 FE 配置项`max_auto_partition_num`管控此类表自动创建时的最大分区数。如果确需创建更多分区，请修改 FE master 节点的该配置项。

6.2.0.14 Q14. Doris 在使用 Select Into Outfile 导出文件到本地时，是否可以导出到指定 BE 所在服务器？

不可以。使用 Select Into Outfile 导出文件到本地时，会随机导出到某个本地路径，不支持导出到指定路径。更多关于 Select Into Outfile 的信息，可参考Select Into Outfile。

6.3 常见查询问题

6.3.0.1 Q1. 查询报错：Failed to get scan range, no queryable replica found in tablet: xxxx

这种情况是因为对应的 tablet 没有找到可以查询的副本，通常原因可能是 BE 宕机、副本缺失等。可以先通过 `show tablet tablet_id` 语句，然后执行后面的 `show proc` 语句，查看这个 tablet 对应的副本信息，检查副本是否完整。同时还可以通过 `show proc "/cluster_balance"` 信息来查询集群内副本调度和修复的进度。

关于数据副本管理相关的命令，可以参阅数据副本管理。

6.3.0.2 Q2. show backends/frontends 查看到的信息不完整

在执行如`show backends/frontends` 等某些语句后，结果中可能会发现有部分列内容不全。比如 show backends 结果中看不到磁盘容量信息等。

通常这个问题会出现在集群有多个 FE 的情况下，如果用户连接到非 Master FE 节点执行这些语句，就会看到不完整的信息。这是因为，部分信息仅存在于 Master FE 节点。比如 BE 的磁盘使用量信息等。所以只有在直连 Master FE 后，才能获得完整信息。

当然，用户也可以在执行这些语句前，先执行 `set forward_to_master=true;` 这个会话变量设置为 true 后，后续执行的一些信息查看类语句会自动转发到 Master FE 获取结果。这样，不论用户连接的是哪个 FE，都可以获取到完整结果了。

6.3.0.3 Q3. invalid cluster id: xxxx

这个错误可能会在 show backends 或 show frontends 命令的结果中出现。通常出现在某个 FE 或 BE 节点的错误信息列中。这个错误的含义是，Master FE 向这个节点发送心跳信息后，该节点发现心跳信息中携带的 cluster id 和本地存储的 cluster id 不同，所以拒绝回应心跳。

Doris 的 Master FE 节点会主动发送心跳给各个 FE 或 BE 节点，并且在心跳信息中会携带一个 cluster_id。cluster_id 是在一个集群初始化时，由 Master FE 生成的唯一集群标识。当 FE 或 BE 第一次收到心跳信息后，则会将 cluster_id 以文件的形式保存在本地。FE 的该文件在元数据目录的 image/目录下，BE 则在所有数据目录下都有一个 cluster_id 文件。之后，每次节点收到心跳后，都会用本地 cluster_id 的内容和心跳中的内容作比对，如果不一致，则拒绝响应心跳。

该机制是一个节点认证机制，以防止接收到集群外的节点发送来的错误的心跳信息。

如果需要恢复这个错误。首先要先确认所有节点是否都是正确的集群中的节点。之后，对于 FE 节点，可以尝试修改元数据目录下的 image/VERSION 文件中的 cluster_id 值后重启 FE。对于 BE 节点，则可以删除所有数据目录下的 cluster_id 文件后重启 BE。

6.3.0.4 Q4. Unique Key 模型查询结果不一致

某些情况下，当用户使用相同的 SQL 查询一个 Unique Key 模型的表时，可能会出现多次查询结果不一致的现象。并且查询结果总在 2-3 种之间变化。

这可能是因为，在同一批导入数据中，出现了 key 相同但 value 不同的数据，这会导致，不同副本间，因数据覆盖的先后顺序不确定而产生的结果不一致的问题。

比如表定义为 k1, v1。一批次导入数据如下：

```
1, "abc"
1, "def"
```

那么可能副本 1 的结果是 1, "abc"，而副本 2 的结果是 1, "def"。从而导致查询结果不一致。

为了确保不同副本之间的数据先后顺序唯一，可以参考Sequence Column 功能。

6.3.0.5 Q5. 查询 bitmap/hll 类型的数据返回 NULL 的问题

在 1.1.x 版本中，在开启向量化的情况下，执行查询数据表中 bitmap 类型字段返回结果为 NULL 的情况下，

1. 首先你要 `set return_object_data_as_binary=true;`
2. 关闭向量化 `set enable_vectorized_engine=false;`
3. 关闭 SQL 缓存 `set [global] enable_sql_cache = false;`

这里是因为 bitmap / hll 类型在向量化执行引擎中：输入均为 NULL，则输出的结果也是 NULL 而不是 0

6.3.0.6 Q6. 访问对象存储时报错：curl 77: Problem with the SSL CA cert

如果 be.INFO 日志中出现 `curl 77: Problem with the SSL CA cert` 错误。可以尝试通过以下方式解决：

1. 在 https://curl.se/docs/caextract.html 下载证书：cacert.pem
2. 拷贝证书到指定位置：`sudo cp /tmp/cacert.pem /etc/ssl/certs/ca-certificates.crt`
3. 重启 BE 节点。

6.3.0.7 Q7. 导入报错：“Message”：“[INTERNAL_ERROR]single replica load is disabled on BE.”

1. be.conf 中增加 enable_single_replica_load = true
2. 重启 BE 节点。

6.4 常见数据湖问题

6.4.1 证书问题

1. 查询时报错 `curl 77: Problem with the SSL CA cert.`。说明当前系统证书过旧，需要更新本地证书。

- 可以从 `https://curl.haxx.se/docs/caextract.html` 下载最新的 CA 证书。
- 将下载后的 cacert-xxx.pem 放到`/etc/ssl/certs/`目录，例如：`sudo cp cacert-xxx.pem /etc/ssl/certs/ca-certificates.crt`。

2. 查询时报错：`ERROR 1105 (HY000): errCode = 2, detailMessage = (x.x.x.x)[CANCELLED][INTERNAL_ERROR]error setting certificate verify locations: CAfile: /etc/ssl/certs/ca-certificates.crt CApath: none.`

```
yum install -y ca-certificates
ln -s /etc/pki/ca-trust/extracted/openssl/ca-bundle.trust.crt /etc/ssl/certs/ca-certificates.crt
```

6.4.2 Kerberos

1. 连接 Kerberos 认证的 Hive Metastore 报错：`GSS initiate failed`

通常是因为 Kerberos 认证信息填写不正确导致的，可以通过以下步骤排查：

```
1. 1.2.1 之前的版本中，Doris 依赖的 libhdfs3 库没有开启 gsasl。请更新至 1.2.2 之后的版本。
2. 确认对各个组件，设置了正确的 keytab 和 principal，并确认 keytab 文件存在于所有 FE、BE 节点上。

    1. `hadoop.kerberos.keytab`/`hadoop.kerberos.principal`：用于 Hadoop hdfs 访问，填写 hdfs 对应的值。
    2. `hive.metastore.kerberos.principal`：用于 hive metastore。

3. 尝试将 principal 中的 ip 换成域名（不要使用默认的 `_HOST` 占位符）
4. 确认 `/etc/krb5.conf` 文件存在于所有 FE、BE 节点上。
```

2. 通过 Hive Catalog 连接 Hive 数据库报错：`RemoteException: SIMPLE authentication is not enabled. Available:[TOKEN, KERBEROS].`

   如果在 `show databases` 和 `show tables` 都是没问题的情况下，查询的时候出现上面的错误，我们需要进行下面两个操作：

   - fe/conf、 be/conf 目录下需放置 core-site.xml 和 hdfs-site.xml
   - BE 节点执行 Kerberos 的 kinit 然后重启 BE，然后再去执行查询即可。

3. 查询配置了 Kerberos 的外表，遇到该报错：`GSSException: No valid credentials provided (Mechanism level: Failed to find any Kerberos Ticket)`，一般重启 FE 和 BE 能够解决该问题。

   - 重启所有节点前可在`"${DORIS_HOME}/be/conf/be.conf"`中的 JAVA_OPTS 参数里配置`-Djavax.security.auth.useSubjectCredsOnly=false`，通过底层机制去获取 JAAS credentials 信息，而不是应用程序。
   - 在JAAS Troubleshooting中可获取更多常见 JAAS 报错的解决方法。

4. 在 Catalog 中配置 Kerberos 时，报错`Unable to obtain password from user`的解决方法：

   - 用到的 principal 必须在 klist 中存在，使用`klist -kt your.keytab`检查。
   - 检查 catalog 配置是否正确，比如漏配`yarn.resourcemanager.principal`。
   - 若上述检查没问题，则当前系统 yum 或者其他包管理软件安装的 JDK 版本存在不支持的加密算法，建议自行安装 JDK 并设置`JAVA_HOME`环境变量。
   - Kerberos 默认使用 AES-256 来进行加密。如果使用 Oracle JDK，则必须安装 JCE。如果是 OpenJDK，OpenJDK 的某些发行版会自动提供无限强度的 JCE，因此不需要安装 JCE。
   - JCE 与 JDK 版本是对应的，需要根据 JDK 的版本来选择 JCE 版本，下载 JCE 的 zip 包并解压到`$JAVA_HOME/jre/lib/security`目录下：
   - JDK6：JCE6
   - JDK7：JCE7
   - JDK8：JCE8

5. 使用 KMS 访问 HDFS 时报错：`java.security.InvalidKeyException: Illegal key size`

升级 JDK 版本到 >= Java 8 u162 的版本。或者下载安装 JDK 相应的 JCE Unlimited Strength Jurisdiction Policy Files。

6. 在 Catalog 中配置 Kerberos 时，如果报错`SIMPLE authentication is not enabled. Available:[TOKEN, KERBEROS]`，那么需要将`core-site.xml`文件放到`"${DORIS_HOME}/be/conf"`目录下。

   如果访问 HDFS 报错`No common protection layer between client and server`，检查客户端和服务端的`hadoop.rpc.protection`属性，使他们保持一致。

```
    <?xml version="1.0" encoding="UTF-8"?>
    <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

    <configuration>

        <property>
            <name>hadoop.security.authentication</name>
```

```
            <value>kerberos</value>
        </property>

    </configuration>
```

7. 在使用 Broker Load 时，配置了 Kerberos，如果报错Cannot locate default realm.。

将 -Djava.security.krb5.conf=/your-path 配置项添加到 Broker Load 启动脚本的 start_broker.sh 的 JAVA_OPTS里。

8. 当在 Catalog 里使用 Kerberos 配置时，不能同时使用hadoop.username属性。
9. 使用 JDK 17 访问 Kerberos

   如果使用 JDK 17 运行 Doris 并访问 Kerberos 服务，可能会出现因使用已废弃的加密算法而导致无法访问的现象。需要在 krb5.conf 中添加 allow_weak_crypto=true 属性。或升级 Kerberos 的加密算法。

   详情参阅：https://seanjmullan.org/blog/2021/09/14/jdk17#kerberos

6.4.3 JDBC Catalog

1. 通过 JDBC Catalog 连接 SQLServer 报错：unable to find valid certification path to requested target

请在 jdbc_url 中添加 trustServerCertificate=true 选项。

2. 通过 JDBC Catalog 连接 MySQL 数据库，中文字符乱码，或中文字符条件查询不正确

请在 jdbc_url 中添加 useUnicode=true&characterEncoding=utf-8

> 注：1.2.3 版本后，使用 JDBC Catalog 连接 MySQL 数据库，会自动添加这些参数。

3. 通过 JDBC Catalog 连接 MySQL 数据库报错：Establishing SSL connection without server's identity verification is not recommended

请在 jdbc_url 中添加 useSSL=true

4. 使用 JDBC Catalog 将 MySQL 数据同步到 Doris 中，日期数据同步错误。需要校验下 MySQL 的版本是否与 MySQL 的驱动包是否对应，比如 MySQL8 以上需要使用驱动 com.mysql.cj.jdbc.Driver。
5. 单个字段过大，查询时 BE 侧 Java 内存 OOM

Jdbc Scanner 在通过 jdbc 读取时，由 session variable batch_size 决定每批次数据在 JVM 中处理的数量，如果单个字段过大，导致 字段大小 * batch_size(近似值，由于 JVM 中 static 以及数据 copy 占用) 超过 JVM 内存限制，就会出现 OOM。

解决方法：

- 减小 batch_size 的值，可以通过 set batch_size = 512; 来调整，默认值为 4064。
- 增大 BE 的 JVM 内存，通过修改 JAVA_OPTS 参数中的 -Xmx 来调整 JVM 最大堆内存大小。例如："-Xmx8g。

6.4.4 Hive Catalog

1. 通过 Hive Catalog 访问 Iceberg 或 Hive 表报错：failed to get schema 或 Storage schema reading not supported

可以尝试以下方法：

- 在 Hive 的 lib/ 目录放上 iceberg 运行时有关的 jar 包。
- 在 hive-site.xml 配置：

```
metastore.storage.schema.reader.impl=org.apache.hadoop.hive.metastore.
    ↪ SerDeStorageSchemaReader
```

配置完成后需要重启 Hive Metastore。

- 在 Catalog 属性中添加 "get_schema_from_table" = "true"

  该参数自 2.1.10 和 3.0.6 版本支持。

2. 连接 Hive Catalog 报错：Caused by: java.lang.NullPointerException

如 fe.log 中有如下堆栈：

```
Caused by: java.lang.NullPointerException
    at org.apache.hadoop.hive.ql.security.authorization.plugin.
        ↪ AuthorizationMetaStoreFilterHook.getFilteredObjects(
        ↪ AuthorizationMetaStoreFilterHook.java:78) ~[hive-exec-3.1.3-core.jar:3.1.3]
    at org.apache.hadoop.hive.ql.security.authorization.plugin.
        ↪ AuthorizationMetaStoreFilterHook.filterDatabases(AuthorizationMetaStoreFilterHook
        ↪ .java:55) ~[hive-exec-3.1.3-core.jar:3.1.3]
    at org.apache.hadoop.hive.metastore.HiveMetaStoreClient.getAllDatabases(
        ↪ HiveMetaStoreClient.java:1548) ~[doris-fe.jar:3.1.3]
    at org.apache.hadoop.hive.metastore.HiveMetaStoreClient.getAllDatabases(
        ↪ HiveMetaStoreClient.java:1542) ~[doris-fe.jar:3.1.3]
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method) ~[?:1.8.0_181]
```

可以尝试在 create catalog 语句中添加 "metastore.filter.hook" = "org.apache.hadoop.hive.metastore.
↪ DefaultMetaStoreFilterHookImpl" 解决。

3. 如果创建 Hive Catalog 后能正常show tables，但查询时报java.net.UnknownHostException: xxxxx

   可以在 CATALOG 的 PROPERTIES 中添加

```
'fs.defaultFS' = 'hdfs://<your_nameservice_or_actually_HDFS_IP_and_port>'
```

4. Hive 1.x 的 orc 格式的表可能会遇到底层 orc 文件 schema 中列名为 _col0，_col1，_col2… 这类系统列名，此时需要在 catalog 配置中添加 hive.version 为 1.x.x，这样就会使用 hive 表中的列名进行映射。

```
CREATE CATALOG hive PROPERTIES (
    'hive.version' = '1.x.x'
);
```

5. 使用 Catalog 查询表数据时发现与 Hive Metastore 相关的报错：Invalid method name，需要设置hive.version参数。

```
CREATE CATALOG hive PROPERTIES (
    'hive.version' = '2.x.x'
);
```

6. 查询 ORC 格式的表，FE 报错 Could not obtain block 或 Caused by: java.lang.NoSuchFieldError: types

对于 ORC 文件，在默认情况下，FE 会访问 HDFS 获取文件信息，进行文件切分。部分情况下，FE 可能无法访问到 HDFS。可以通过添加以下参数解决：

"hive.exec.orc.split.strategy" = "BI"

其他选项：HYBRID（默认），ETL。

7. 在 hive 上可以查到 hudi 表分区字段的值，但是在 doris 查不到。

   doris 和 hive 目前查询 hudi 的方式不一样，doris 需要在 hudi 表结构的 avsc 文件里添加上分区字段，如果没加，就会导致 doris 查询 partition_val 为空（即使设置了 hoodie.datasource.hive_sync.partition_fields=partition_val 也不可以）

```
{
    "type": "record",
    "name": "record",
    "fields": [{
        "name": "partition_val",
        "type": [
            "null",
            "string"
            ],
        "doc": "Preset partition field, empty string when not partitioned",
        "default": null
        },
```

```
        {
        "name": "name",
        "type": "string",
        "doc": "Name"
        },
        {
        "name": "create_time",
        "type": "string",
        "doc": "Creation time"
        }
    ]
}
```

8. 查询 hive 外表，遇到该报错：`java.lang.ClassNotFoundException: Class com.hadoop.compression.lzo.LzoCodec not found`

去 hadoop 环境搜索`hadoop-lzo-*.jar`放在`"${DORIS_HOME}/fe/lib/"`目录下并重启 fe。

从 2.0.2 版本起，可以将这个文件放置在 FE 的 `custom_lib/` 目录下（如不存在，手动创建即可），以防止升级集群时因为 lib 目录被替换而导致文件丢失。

9. 创建 hive 表指定 serde 为 `org.apache.hadoop.hive.contrib.serde2.MultiDelimitserDe`，访问表时报错：`storage schema reading not supported`

在 hive-site.xml 文件中增加以下配置，并重启 hms 服务：

```
<property>
   <name>metastore.storage.schema.reader.impl</name>
   <value>org.apache.hadoop.hive.metastore.SerDeStorageSchemaReader</value>
</property>
```

10. 报错：java.security.InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty

FE 日志中完整报错信息如下：

```
org.apache.doris.common.UserException: errCode = 2, detailMessage = S3 list path failed. path
    ↪ =s3://bucket/part-*,msg=errors while get file status listStatus on s3://bucket: com.
    ↪ amazonaws.SdkClientException: Unable to execute HTTP request: Unexpected error: java.
    ↪ security.InvalidAlgorithmParameterException: the trustAnchors parameter must be non-
    ↪ empty: Unable to execute HTTP request: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty
org.apache.doris.common.UserException: errCode = 2, detailMessage = S3 list path exception.
    ↪ path=s3://bucket/part-*, err: errCode = 2, detailMessage = S3 list path failed. path=
    ↪ s3://bucket/part-*,msg=errors while get file status listStatus on s3://bucket: com.
    ↪ amazonaws.SdkClientException: Unable to execute HTTP request: Unexpected error: java.
    ↪ security.InvalidAlgorithmParameterException: the trustAnchors parameter must be non-
    ↪ empty: Unable to execute HTTP request: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty
```

```
org.apache.hadoop.fs.s3a.AWSClientIOException: listStatus on s3://bucket: com.amazonaws.
    ↪ SdkClientException: Unable to execute HTTP request: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty:
    ↪ Unable to execute HTTP request: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty
Caused by: com.amazonaws.SdkClientException: Unable to execute HTTP request: Unexpected error
    ↪ : java.security.InvalidAlgorithmParameterException: the trustAnchors parameter must
    ↪ be non-empty
Caused by: javax.net.ssl.SSLException: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty
Caused by: java.lang.RuntimeException: Unexpected error: java.security.
    ↪ InvalidAlgorithmParameterException: the trustAnchors parameter must be non-empty
Caused by: java.security.InvalidAlgorithmParameterException: the trustAnchors parameter must
    ↪ be non-empty
```

尝试更新FE节点CA证书，使用 `update-ca-trust（CentOS/RockyLinux）`，然后重启FE进程即可。

11. BE 报错：java.lang.InternalError

    如果在 be.INFO 中看到类似如下错误：

```
W20240506 15:19:57.553396 266457 jni-util.cpp:259] java.lang.InternalError
        at org.apache.hadoop.io.compress.zlib.ZlibDecompressor.init(Native Method)
        at org.apache.hadoop.io.compress.zlib.ZlibDecompressor.<init>(ZlibDecompressor.java
            ↪ :114)
        at org.apache.hadoop.io.compress.GzipCodec$GzipZlibDecompressor.<init>(GzipCodec.java
            ↪ :229)
        at org.apache.hadoop.io.compress.GzipCodec.createDecompressor(GzipCodec.java:188)
        at org.apache.hadoop.io.compress.CodecPool.getDecompressor(CodecPool.java:183)
        at org.apache.parquet.hadoop.CodecFactory$HeapBytesDecompressor.<init>(CodecFactory.
            ↪ java:99)
        at org.apache.parquet.hadoop.CodecFactory.createDecompressor(CodecFactory.java:223)
        at org.apache.parquet.hadoop.CodecFactory.getDecompressor(CodecFactory.java:212)
        at org.apache.parquet.hadoop.CodecFactory.getDecompressor(CodecFactory.java:43)
```

是因为 Doris 自带的 libz.a 和系统环境中的 libz.so 冲突了。

为了解决这个问题，需要先执行 `export LD_LIBRARY_PATH=/path/to/be/lib:$LD_LIBRARY_PATH` 然后重启
    ↪ BE 进程。

12. 在插入 hive 数据的时候报错：HiveAccessControlException Permission denied: user [user_a] does
    ↪ not have [UPDATE] privilege on [database/table]。

    因为插入数据之后，需要更新对应的统计信息，这个更新的操作需要 alter 权限，所以要在 ranger 上给该用户新增 alter 权限。

6.4.5 HDFS

1. 访问 HDFS 3.x 时报错：`java.lang.VerifyError: xxx`

1.2.1 之前的版本中，Doris 依赖的 Hadoop 版本为 2.8。需更新至 2.10.2。或更新 Doris 至 1.2.2 之后的版本。

2. 使用 Hedged Read 优化 HDFS 读取慢的问题。

   在某些情况下，HDFS 的负载较高可能导致读取某个 HDFS 上的数据副本的时间较长，从而拖慢整体的查询效率。HDFS Client 提供了 Hedged Read 功能。该功能可以在一个读请求超过一定阈值未返回时，启动另一个读线程读取同一份数据，哪个先返回就是用哪个结果。

   注意：该功能可能会增加 HDFS 集群的负载，请酌情使用。

   可以通过以下方式开启这个功能：

```
create catalog regression properties (
    'type'='hms',
    'hive.metastore.uris' = 'thrift://172.21.16.47:7004',
    'dfs.client.hedged.read.threadpool.size' = '128',
    'dfs.client.hedged.read.threshold.millis' = "500"
);
```

```
`dfs.client.hedged.read.threadpool.size` 表示用于 Hedged Read 的线程数，这些线程由一个 HDFS
    ↪ Client 共享。通常情况下，针对一个 HDFS 集群，BE 节点会共享一个 HDFS Client。

`dfs.client.hedged.read.threshold.millis` 是读取阈值，单位毫秒。当一个读请求超过这个阈值未返回时
    ↪ ，会触发 Hedged Read。

开启后，可以在 Query Profile 中看到相关参数：

`TotalHedgedRead`: 发起 Hedged Read 的次数。

`HedgedReadWins`：Hedged Read 成功的次数（发起并且比原请求更快返回的次数）

注意，这里的值是单个 HDFS Client 的累计值，而不是单个查询的数值。同一个 HDFS Client
    ↪ 会被多个查询复用。
```

3. `Couldn't create proxy provider class org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvi`
   ↪

   在 FE 和 BE 的 start 脚本中，会将环境变量 HADOOP_CONF_DIR 加入 CLASSPATH。如果 HADOOP_CONF_DIR 设置错误，比如指向了不存在的路径或错误路径，则可能加载到错误的 xxx-site.xml 文件，从而读取到错误的信息。

   需检查 HADOOP_CONF_DIR 是否配置正确，或将这个环境变量删除。

4. `BlockMissingExcetpion: Could not obtain block: BP-XXXXXXXXX No live nodes contain current`
   ↪ `block`

   可能的处理方式有：

- 通过 `hdfs fsck file -files -blocks -locations` 来查看具体该文件是否健康。
- 通过 `telnet` 来检查与 datanode 的连通性。
- 查看 datanode 日志。

如果出现以下错误：

`org.apache.hadoop.hdfs.server.datanode.DataNode: Failed to read expected SASL data transfer protection handshake from client at /XXX.XXX.XXX.XXX:XXXXX. Perhaps the client is running an older version of Hadoop which does not support SASL data transfer protection` 则为当前 hdfs 开启了加密传输方式，而客户端未开启导致的错误。

使用下面的任意一种解决方案即可：

- 拷贝 hdfs-site.xml 以及 core-site.xml 到 be/conf 和 fe/conf 目录。(推荐)
- 在 hdfs-site.xml 找到相应的配置 `dfs.data.transfer.protection`，并且在 catalog 里面设置该参数。

6.4.6 DLF Catalog

1. 使用 DLF Catalog 时，BE 读在取 JindoFS 数据出现`Invalid address`，需要在`/ets/hosts`中添加日志中出现的域名到 IP 的映射。
2. 读取数据无权限时，使用`hadoop.username`属性指定有权限的用户。
3. DLF Catalog 中的元数据和 DLF 保持一致。当使用 DLF 管理元数据时，Hive 新导入的分区，可能未被 DLF 同步，导致出现 DLF 和 Hive 元数据不一致的情况，对此，需要先保证 Hive 元数据被 DLF 完全同步。

6.4.7 其他问题

1. Binary 类型映射到 Doris 后，查询乱码

   Doris 原生不支持 Binary 类型，所以各类数据湖或数据库中的 Binary 类型映射到 Doris 中，通常使用 String 类型进行映射。String 类型只能展示可打印字符。如果需要查询 Binary 的内容，可以使用 `TO_BASE64()` 函数转换为 Base64 编码后，在进行下一步处理。

2. 分析 Parquet 文件

   在查询 Parquet 文件时，由于不同系统生成的 Parquet 文件格式可能有所差异，比如 RowGroup 的数量，索引的值等，有时需要检查 Parquet 文件的元数据进行问题定位或性能分析。这里提供一个工具帮助用户更方便的分析 Parquet 文件：

   1. 下载并解压 Apache Parquet Cli 1.14.0
   2. 将需要分析的 Parquet 文件下载到本地，假设路径为 `/path/to/file.parquet`
   3. 使用如下命令分析 Parquet 文件元信息：

      ```
      ./parquet-tools meta /path/to/file.parquet
      ```

   4. 更多功能，可参阅 Apache Parquet Cli 文档

6.5 常见 BI 问题

6.5.1 Power BI

6.5.1.1 Q1. JDBC 拉取表到 Desktop Power BI 时报错 Timeout expired. The timeout period elapsed prior to completion of the operation or the server is not responding.

通常这是 Power BI 在拉取数据源的时间超时，在填写数据源服务器和数据库时点击高级选项，其中有个超时时间，把该时间配置的较高。

6.5.1.2 Q2. 2.1.x 版本 JDBC 连接 Power BI 时报错读取数据时报错，给定的关键字目前不在字典中。

先在数据库中执行 show collation，一般情况下会只有 utf8mb4_900_bin，charset 为 utf8mb4 这一行结果。该报错的主要原因是在连接 Power BI 时需要找 33 号 ID，即需要该表中有 33 ID 的行，需要升级至 2.1.5 版本以上。

6.5.1.3 Q3. 连接时报错从提供程序读取数据时出错：索引和计数必须引用该字符串内的位置。

该问题原因是连接过程会加载全局参数，该 SQL 出现了列名和 values 相同的情况

```
SELECT
@@max_allowed_packet  as max_allowed_packet, @@character_set_client ,@@character_set_connection ,
@@license,@@sql_mode ,@@lower_case_table_names , @@autocommit ;
```

可以在当前版本关闭新优化器也可以升级到 2.0.7 或者 2.1.6 及以上版本。

6.5.1.4 Q4. JDBC 连接 2.1.x 版本报错从提供读取数据时出错：“Character set ‘utf8mb3’ is not supported by .Net.Framework”。

该问题易在 2.1.x 版本遇到，如果遇到该问题则需要把 JDBC Driver 升级到 8.0.32。

6.5.2 Tableau

6.5.2.1 Q1. 2.0.x 报错 Tableau 无法连接到数据源，错误代码：37CE01A3。

在当前版本关闭新优化器或者升级至 2.0.7 及以上版本。

6.5.2.2 Q2. 报错 SSL connection error:protocol version mismatch 无法连接到 MySQL 服务器。

该报错原因是 Doris 开启了 SSL 验证，但是连接过程中未使用 SSL 连接，需要在 fe.conf 里面关闭 enable_ssl 变量。

6.5.2.3 Q3. 连接时报错 Unsupported command (COM_STMT_PREPARED)。

MySQL 驱动版本安装不恰当，需要改安装为 MySQL5.1.x 版本的连接驱动。

6.6 数据正确性问题

本文档主要用于记录 Doris 使用过程中关于数据正确性的常见问题。会不定期更新。

表格中的“表出现重复 key 数据”均指在 merge-on-write Unique 表中出现重复 key 数据。 merge-on-write Unique 表上的重复 key 问题都可以通过触发 full compaction来进行修复，其他类型的正确性问题可能需要根据情况来确定修复方案，如有需要，请联系社区支持。

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
merge-on-write Unique 表上部分列更新导入将之前已经被删除的数据补齐上来	部分列更新时指定了_ ↪ _ ↪ DORIS ↪ _ ↪ DELETE ↪ _ ↪ SIGN ↪ _ ↪ _ ↪ 列，且存量历史数据中有被_ ↪ _ ↪ DORIS ↪ _ ↪ DELETE ↪ _ ↪ SIGN ↪ _ ↪ _ ↪ 列标记删除	<2.1.8, <3.0.4	>=2.1.8, >=3.0.4	存算一体，存算分离，部分列更新	#46194

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算分离模式下在 merge-on-write Unique 表上有并发导入	<3.0.4	>=3.0.4	存算分离	#46039

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算分离模式下在 merge-on-write Unique 表上同时存在导入与导入之间的并发以及导入和 com-paction 之间的并发	<3.0.4	>=3.0.4	存算分离	#44975

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
系统生成的自增列的值出现0/出现重复值	BE 和 FE 之间存在网络异常	<2.1.8, <3.0.3	>=2.1.8, >=3.0.3	存算一体，存算分离，自增列	#43774

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
使用 Stream Load 向 merge-on-write Unique 导入数据时，对于满足delete ↪ 参数所指定的删除条件的数据，导入后没有被删除掉	使用 Stream Load 导入数据时，设置了merge ↪ _ ↪ type ↪ : ↪ ↪ MERGE ↪ , partial ↪ _ ↪ columns ↪ : ↪ ↪ true ↪ 和delete ↪ 参数	<2.0.15, <2.17, <3.0.3	>=2.0.15, >=2.17, >=3.0.3	存算一体，存算分离，部分列更新	#40730

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
使用部分列更新导入后，部分自增列数据被非预期更新为新的系统生成的值	表中存在在 Value 列上的自增列，且部分列更新导入没有指定这个自增列上的值	<2.1.6, <3.0.2	>=2.1.6, >=3.0.2	存算一体，存算分离，自增列	#39996

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复key数据	用户使用ALTER ↪ ↪ TABLE ↪ ↪ tbl ↪ ↪ ENABLE ↪ ↪ FEATURE ↪ ↪ " ↪ SEQUENCE ↪ _ ↪ LOAD ↪ " ↪ ↪ WITH ↪ ↪ ... ↪ 语句给一个不支持sequence列的merge-on-write Unique表添加了sequence列功	<2.0.15, <2.1.6, <3.0.2	>=2.0.15, >=2.1.6, >=3.0.2	存算一体，存算分离	#39958

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算分离模式下在 merge-on-write Unique 表上存在导入与导入之间的并发或导入和 com-paction 之间的并发	<3.0.1	>=3.0.1	存算分离	#39018

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
使用部分列更新导入后，merge-on-write Unique 表中部分数据错乱	merge-on-write Unique 表上有并发的部分列更新导入，并且导入过程中有 BE 重启	<2.0.15, <2.1.6, <3.0.2	>=2.0.15, >=2.1.6, >=3.0.2	存算一体，存算分离，部分列更新	#38331

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算分离模式下在 merge-on-write Unique 表上存在导入和 compaction 之间的并发	<3.0.2	>=3.0.2	存算分离	#37670, #41309, #39791

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	merge-on-write Unique 表上有 sequence 列，表上存在单次数据量很大的导入，且触发了 segment compaction	<2.0.15, <2.1.6, <3.0.2	>=2.0.15, >=2.1.6, >=3.0.2	存算一体，存算分离	#38369

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算一体模式下 merge-on-write Unique 表上有失败的 full clone	<2.0.13, <2.1.5, <3.0.0	>=2.0.13, >=2.1.5, >=3.0.0	存算一体	#37001

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	存算分离模式下在 merge-on-write Unique 表上有 Stream Load 的导入且导入内部存在失败重试过程	<3.0.0	>=3.0.0	存算分离	#36670

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
merge-on-write Unique 表上多副本数据不一致	merge-on-write Unique 表上有过指定了_ ↪ _ ↪ DORIS ↪ _ ↪ DELETE ↪ _ ↪ SIGN ↪ _ ↪ _ ↪ 列的部分列更新导入，且在导入的时候不同副本上 Base Com-paction 进度不	<2.0.15, <2.1.5, <3.0.0	>=2.0.15, >=2.1.5, >=3.0.0	存算一体，存算分离，部分列更新	#36210

问题现象	发生条件	影响版本	Fix 版本	影响范围	Fix PR
表出现重复 key 数据	merge-on-write Unique 表上有并发的部分列更新导入，并且导入过程中有 BE 重启	<2.0.11, <2.1.4, <3.0.0	>=2.0.11, >=2.1.4, >=3.0.0	存算一体，存算分离，部分列更新	#35739

6.7 常见导入问题

6.7.1 导入通用问题

6.7.1.1 报错“[DATA_QUALITY_ERROR] Encountered unqualified data”

问题描述：导入报数据质量错误。

解决方案：- Stream Load 和 Insert Into 结果中会返回错误 URL，Broker Load 可通过 Show Load 命令查看对应错误 URL。- 通过浏览器或 curl 命令访问错误 URL 查看具体的数量质量错误原因。- 通过 strict_mode 和 max_filter_ratio 参数项来控制能容忍的错误率。

6.7.1.2 报错“[E-235] Failed to init rowset builder”

问题描述：-235 错误是因为导入频率过高，数据未能及时 compaction，超过版本限制。

解决方案：- 增加每批次导入数据量，降低导入频率。- 在 be.conf 中调大 max_tablet_version_num 参数, 建议不超过 5000。

6.7.1.3 报错“[E-238] Too many segments in rowset”

问题描述：-238 错误是因为单个 rowset 下的 segment 数量超限。

常见原因：- 建表时 bucket 数配置过小。- 数据出现倾斜，建议使用更均衡的分桶键。

6.7.1.4 报错“Transaction commit successfully, BUT data will be visible later”

问题描述：数据导入成功但暂时不可见。

原因：通常是由于系统资源压力导致事务 publish 延迟。

6.7.1.5 报错“Failed to commit kv txn [...] Transaction exceeds byte limit”

问题描述：存算分离模式下，单次导入涉及的 partition 和 tablet 过多, 超过事务大小的限制。

解决方案：- 分批按 partition 导入数据, 减小单次导入涉及到的 partition 数量。- 优化表结构减少 partition 和 tablet 数量。

6.7.1.6 CSV 文件最后一列出现额外的 “’˚’

问题描述：通常是 windows 换行符导致。

解决方案：指定正确的换行符：-H "line_delimiter:\\r\\n"

6.7.1.7 CSV 带引号数据导入为 null

问题描述：带引号的 CSV 数据导入后值变为 null。

解决方案：使用 trim_double_quotes 参数去除字段外层双引号。

6.7.2 Stream Load

6.7.2.1 导入慢的原因

- CPU、IO、内存、网卡资源有瓶颈。
- 客户端机器到 BE 机器网络慢, 通过客户端机器到 BE 机器的 Ping 时延可以做初步的判断。
- Webserver 线程数瓶颈，单 BE 上 Stream Load 并发数太高 (超过 be.conf webserver_num_workers 配置) 可能导致线程数据瓶颈。
- Memtable Flush 线程数瓶颈，通过 BE metrics 查看 doris_be_flush_thread_pool_queue_size 看排队是否比较严重。可以适当调大 be.conf flush_thread_num_per_store 参数来解决。

6.7.2.2 特殊字符列名处理

列名中含有特殊字符时需要使用单引号配合反引号方式指定 columns 参数：

```
curl --location-trusted -u root:"" \
    -H 'columns:`@coltime`,colint,colvar' \
    -T a.csv \
    -H "column_separator:," \
    http://127.0.0.1:8030/api/db/loadtest/_stream_load
```

6.7.3 Routine Load

6.7.3.1 较严重的 Bug 修复

问题描述	发生条件	影响范围	临时解决方案	受影响版本	修复版本	修复 PR
当至少一个 Job 连接 Kafka 时发生超时，会影响其他 Job 的导入速度，导致全局 Routine Load 导入变慢	存在至少一个 Job 连接 Kafka 时发生超时	存算分离 存算一体	通过停止或手动暂停该 Job 来解决。	<2.1.9 <3.0.5	2.1.9 3.0.5	#47530
重启 FE Master 后，用户数据可能丢失	Job 设置的 Offset 为 OFFSET_END，重启 FE	存算分离	将消费模式更改为 OFFSET_BEGINNING。	3.0.2-3.0.4	3.0.5	#46149
导入过程中产生大量小事务，导致 Compaction 无法及时完成，并持续报 -235 错误。	Doris 消费速度过快，或 Kafka 数据流量呈小批量趋势	存算分离 存算一体	暂停 Routine Load Job，并执行以下命令：`ALTER ROUTINE LOAD FOR jobname FROM kafka ("property.enable.partition.eof" = "false");`	<2.1.8 <3.0.4	2.1.8 3.0.4	#45528, #44949, #39975
Kafka 第三方库析构卡住，导致无法正常消费数据。	Kafka 删除 Topic（可能不止此条件）	存算分离 存算一体	重启所有 BE 节点。	<2.1.8 <3.0.4	2.1.8 3.0.4	#44913
Routine Load 调度卡住	当 FE 向 Meta Service 中止事务时发生超时	存算分离	重启 FE 节点。	<3.0.2	3.0.2	#41267

问题描述	发生条件	影响范围	临时解决方案	受影响版本	修复版本	修复 PR
Routine Load 重启问题	重启 BE 节点	存算分离存算一体	手动恢复 Job。	<2.1.7 <3.0.2	2.1.7 3.0.2	#3727

6.7.3.2 默认配置优化

优化内容	合入版本	对应 PR
增加了 Routine Load 的超时时间	2.1.7 3.0.3	#42042, #40818
调整了 max_batch_interval 的默认值	2.1.8 3.0.3	#42491
移除了 max_batch_interval 的限制	2.1.5 3.0.0	#29071
调整了 max_batch_rows 和 max_batch_size 的默认值	2.1.5 3.0.0	#36632

6.7.3.3 可观测优化

优化内容	合入版本	对应 PR
增加了可观测性相关的 Metrics 指标	3.0.5	#48209, #48171, #48963

6.7.3.4 报错“failed to get latest offset”

问题描述：Routine Load 无法获取 Kafka 最新的 Offset。

常见原因：- 一般都是到 kafka 的网络不通, ping 或者 telnet kafka 的域名确认下 - 三方库的 bug 导致的获取超时，错误为:java.util.concurrent.TimeoutException: Waited X seconds

6.7.3.5 报错“failed to get partition meta: Local:’Broker transport failure”

问题描述：Routine Load 无法获取 Kafka Topic 的 Partition Meta。

常见原因：- 一般都是到 kafka 的网络不通, ping 或者 telnet kafka 的域名确认下 - 如果使用的是域名的方式，可以在/etc/hosts 配置域名映射

6.7.3.6 报错“Broker: Offset out of range”

问题描述：消费的 offset 在 kafka 中不存在，可能是因为该 offset 已经被 kafka 清理掉了。

解决方案: - 需要重新指定 offset 进行消费，例如可以指定 offset 为 OFFSET_BEGINNING。 - 需要根据导入速度设置合理的 kafka log 清理参数：log.retention.hours、 log.retention.bytes 等。

7 SQL 手册

7.1 基础元素

7.1.1 SQL 类型

7.1.1.1 数据类型概览

7.1.1.1.1 数值类型

包括以下 4 种：

1. BOOLEAN 类型：

两种取值，0 代表 false，1 代表 true。更多信息参考 BOOLEAN 文档。

2. 整数类型：

都是有符号整数，xxINT 的差异是占用字节数和表示范围

- TINYINT 占 1 字节，范围 [-128, 127], 更多信息参考 TINYINT 文档。
- SMALLINT 占 2 字节，范围 [-32768, 32767], 更多信息参考 SMALLINT 文档。
- INT 占 4 字节，范围 [-2147483648, 2147483647], 更多信息参考 INT 文档。
- BIGINT 占 8 字节，范围 [-9223372036854775808, 9223372036854775807], 更多信息参考 BIGINT 文档。
- LARGEINT 占 16 字节，范围 [-2^127, 2^127 - 1], 更多信息参考 LARGEINT 文档。

3. 浮点数类型：

不精确的浮点数类型 FLOAT 和 DOUBLE，和常见编程语言中的 float 和 double 对应。更多信息参考 FLOAT、DOUBLE 文档。

4. 定点数类型：

精确的定点数类型 DECIMAL，用于金融等精度要求严格准确的场景。更多信息参考 DECIMAL 文档。

7.1.1.1.2 日期类型

日期类型包括 DATE、TIME 和 DATETIME，DATE 类型只存储日期精确到天，DATETIME 类型存储日期和时间，可以精确到微秒。TIME 类型只存储时间，且暂时不支持建表存储，只能在查询过程中使用。

对日期类型进行计算，或将其转换为数字，请使用类似 TIME_TO_SEC, DATE_DIFF, UNIX_TIMESTAMP 等函数，直接将其 CAST 为数字类型的结果不受保证。在未来的版本中，此类 CAST 行为将会被禁止。

更多信息参考 DATE、TIME 和 DATETIME 文档。

7.1.1.1.3 字符串类型

字符串类型支持定长和不定长，总共有以下 3 种：

1. CHAR(M)：定长字符串，固定长度 M 字节，M 的范围是 [1, 255]。
2. VARCHAR(M)：不定长字符串，M 是最大长度，M 的范围是 [1, 65533]。
3. STRING：不定长字符串，默认最长 1048576 字节（1MB），可调大到 2147483643 字节（2GB），BE 配置 string_type_length_soft_limit_bytes。

7.1.1.1.4 半结构化类型

针对 JSON 半结构化数据，支持 3 类不同场景的半结构化数据类型：

1. 支持嵌套的固定 schema，适合分析的数据类型 ARRAY、 MAP STRUCT：常用于用户行为和画像分析，湖仓一体查询数据湖中 Parquet 等格式的数据等场景。由于 schema 相对固定，没有动态 schema 推断的开销，写入和分析性能很高。
2. 支持嵌套的不固定 schema，适合分析的数据类型 VARIANT：常用于 Log, Trace, IoT 等分析场景，schema 灵活可以写入任何合法的 JSON 数据，并自动展开成子列采用列式存储，存储压缩率高，聚合过滤排序等分析性能很好。
3. 支持嵌套的不固定 schema，适合点查的数据类型 JSON：常用于高并发点查场景，schema 灵活可以写入任何合法的 JSON 数据，采用二进制格式存储，提取字段的性能比普通 JSON String 快 2 倍以上。

7.1.1.1.5 聚合类型

聚合类型存储聚合的结果或者中间状态，用于加速聚合查询，包括下面几种：

1. BITMAP：用于精确去重，如 UV 统计，人群圈选等场景。配合 bitmap_union、bitmap_union_count、bitmap_hash、bitmap_hash64 等 BITMAP 函数使用。
2. HLL：用于近似去重，性能优于 COUNT DISTINCT。配合 hll_union_agg、 hll_raw_agg、 hll_cardinality、 hll_hash 等 HLL 函数使用。
3. QUANTILE_STATE：用于分位数近似计算，性能优于 PERCENTILE。配合 QUANTILE_PERCENT、 QUANTILE_UNION、TO_QUANTILE_STATE 等函数使用。
4. AGG_STATE：用于聚合计算加速，配合 state/merge/union 聚合函数组合器使用。

7.1.1.1.6 IP 类型

IP 类型以二进制形式存储 IP 地址，比用字符串存储更省空间查询速度更快，支持 2 种类型：

1. IPv4：以 4 字节二进制存储 IPv4 地址，配合 ipv4_* 系列函数使用。
2. IPv6：以 16 字节二进制存储 IPv6 地址，配合 ipv6_* 系列函数使用。

7.1.1.2 数值类型

7.1.1.2.1 BOOLEAN

BOOLEAN

描述

```
BOOL, BOOLEAN
与TINYINT一样，0代表false，1代表true
```

keywords

```
BOOLEAN
```

7.1.1.2.2 TINYINT

TINYINT

描述

```
TINYINT
1字节有符号整数，范围[-128, 127]
```

keywords

```
TINYINT
```

7.1.1.2.3 SMALLINT

SMALLINT

描述

```
SMALLINT
2字节有符号整数，范围[-32768, 32767]
```

keywords

```
SMALLINT
```

7.1.1.2.4 INT

INT

描述

```
INT
4字节有符号整数，范围[-2147483648, 2147483647]
```

keywords

```
INT
```

7.1.1.2.5 BIGINT

BIGINT

描述

```
BIGINT
8字节有符号整数，范围[-9223372036854775808, 9223372036854775807]
```

keywords

```
BIGINT
```

7.1.1.2.6 LARGEINT

LARGEINT

描述

```
LARGEINT
16字节有符号整数，范围[-2^127 + 1 ~ 2^127 - 1]
```

keywords

```
LARGEINT
```

7.1.1.2.7 FLOAT

FLOAT

描述

```
FLOAT
4字节浮点数
```

keywords

```
FLOAT
```

7.1.1.2.8 DOUBLE

DOUBLE

描述

```
DOUBLE
8字节浮点数
```

keywords

```
DOUBLE
```

7.1.1.2.9 DECIMAL

DECIMAL

DECIMAL

描述

```
DECIMAL(P[,S])
高精度定点数，P 代表一共有多少个有效数字(precision)，S 代表小数位有多少数字(scale)。
有效数字 P 的范围是 [1, MAX_P]，enable_decimal256=false时，MAX_P=38，enable_decimal256=true时，
    ↪ MAX_P=76。
小数位数字数量 S 的范围是 [0, P]。

P默认值是38，S默认是9（DECIMAL(38, 9)）。

enable_decimal256 的默认值是false，设置为true 可以获得更加精确的结果，但是会带来一些性能损失。

decimal类型在输出时，小数点后总是显示S位数字，即使小数的后缀是0。比如类型decimal(18, 6)的数字123
    ↪ .456，会输出成123.456000。
```

精度推演

DECIMAL 有一套很复杂的类型推演规则，针对不同的表达式，会应用不同规则进行精度推断。

四则运算

假定 e1(p1, s1) 和 e2(p2, s2) 是两个 DECIMAL 类型的数字，运算结果精度推演规则如下：

运算	结果 precision	结果 scale	溢出时结果 precision	溢出时结果 scale	中间结果 e1 类型	中间 e2 类型
e1 + e2	max(p1 - s1,p2 - s2) + max(s1, s2) + 1	max(s1, s2)	MAX_P	min(MAX_P, p) - max(p1 - s1,p2 - s2)	按照结果 cast	按照结果 cast
e1 - e2	max(p1 - s1,p2 - s2) + max(s1, s2) + 1	max(s1, s2)	MAX_P	min(MAX_P, p) - max(p1 - s1,p2 - s2)	按照结果 cast	按照结果 cast

|e1 * e2|p1 + p2|s1 + s2|MAX_P|

precision - scale < MAX_P - decimal_overflow_scale：min(scale, MAX_P - (precision - scale))

precision - scale > MAX_P - decimal_overflow_scale，且 scale < decimal_overflow_scale：s1 + s2

precision - scale > MAX_P - decimal_overflow_scale，scale >= decimal_overflow_scale：decimal_overflow_scale

| 不变 | 不变 | |e1 / e2|p1 + s2 + div_precision_increment|s1 + div_precision_increment|MAX_P|

precision - s1 小于 MAX_P - decimal_overflow_scale：(MAX_P - (precision - s1))+ div_precision_increment

precision - s1 大于 MAX_P - decimal_overflow_scale，且 s1 小于decimal_overflow_scale：s1 + div_precision_
↪ increment

precision - s1 大于 MAX_P - decimal_overflow_scale，且 s1 大于等于decimal_overflow_scale：decimal_overflow_scale + div_precision_increment

|p 按照结果 cast，s 按照结果 +e2.scale cast| | |e1 % e2|max(p1 - s1,p2 - s2) + max(s1, s2) |max(s1, s2)|MAX_P|min(MAX_P, p) - max(p1 - s1,p2 - s2) | 按照结果 cast| 按照结果 cast|

表格中计算溢出时结果scale的规则中，precision表示结果precision列中precision，scale表示结果scale列中的scale。

div_precision_increment是 FE 的配置参数，参见div_precision_increment。

decimal_overflow_scale是 FE 的 session variable，表示当 decimal 数值计算结果精度溢出时，计算结果最多可保留的小数位数，默认值是 6。

值得注意的是，除法计算的过程是 DECIMAL(p1, s1) / DECIMAL(p2, s2) 先转换成 DECIMAL(p1 + s2 + div_precision_increment, s1 + s2) / DECIMAL(p2, s2) 然后再进行计算，所以可能会出现 DECIMAL(p1 + s2 + div_precision_increment, s1 + div_precision_increment) 是满足 DECIMAL 的范围，但是由于先转换成了 DECIMAL(p1 + s2 + div_precision_increment, s1 + s2) 导致超出范围，Doris 默认情况下会报Arithmetic overflow错误。

示例

乘法不溢出

```
create table test_decimal_mul_no_overflow(f1 decimal(19, 9), f2 decimal(19, 9)) properties('
    ↪ replication_num'='1');
insert into test_decimal_mul_no_overflow values('9999999999.999999999', '9999999999.999999999');
```

根据乘法结果精度的计算规则，结果类型是 decimal(38, 18)，不会溢出：

```
explain verbose select f1, f2, f1 * f2 from test_decimal_mul_no_overflow;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
```

```
|     f1 * f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test.test_decimal_mul_no_overflow(test_decimal_mul_no_overflow), PREAGGREGATION: ON
    ↪                                     |
|      partitions=1/1 (test_decimal_mul_no_overflow)
    ↪                                                                                  |
|      tablets=10/10, tabletList=1750210355691,1750210355693,1750210355695 ...
    ↪                                                        |
|      cardinality=1, avgRowSize=3065.0, numNodes=1
    ↪                                                                                   |
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (f1[#0] * f2[#1])
    ↪                                                                              |
|      final project output tuple id: 1
    ↪
```

```
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_mul_no_overflow}
    ↪                                                                    |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_mul_no_overflow}
    ↪                                                                    |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,18), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 * f2 from test_decimal_mul_no_overflow;
+----------------------+----------------------+----------------------------------------+
| f1                   | f2                   | f1 * f2                                |
+----------------------+----------------------+----------------------------------------+
| 9999999999.999999999 | 9999999999.999999999 | 99999999999999999980.000000000000000001 |
+----------------------+----------------------+----------------------------------------+
```

乘法溢出规则

```
create table test_decimal_mul_overflow1(f1 decimal(20, 5), f2 decimal(21, 6)) properties('
    ↪ replication_num'='1');
insert into test_decimal_mul_overflow1 values('12345678901234.12345', '12345678901234.123456');
```

根据乘法结果精度的计算规则，默认配置下 (enable_decimal256=false, decimal_overflow_scale=6, div_
↪ precision_increment=4)，正常计算出来的结果类型是decimal(41, 11)，precision 溢出了，需要按照溢出时的规则重新计算：MAX_P - decimal_overflow_scale = 38 - 6 = 32，precision - scale = 41 - 11 = 30 < 32，适用规则 1，最终结果 scale = min(11, 38 - 30) = 8，最终结果类型是 decimal(38, 8)：

```
 explain verbose select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 * f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|       MYSQL_PROTOCAL
    ↪
    ↪ |
|
```

```
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
```

```
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test.test_decimal_mul_overflow1(test_decimal_mul_overflow1), PREAGGREGATION: ON
    ↪                                      |
|      partitions=1/1 (test_decimal_mul_overflow1)
    ↪                                                                                  |
|      tablets=10/10, tabletList=1750210355791,1750210355793,1750210355795 ...
    ↪                                                        |
|      cardinality=1, avgRowSize=3115.0, numNodes=1
    ↪                                                                                |
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (f1[#0] * f2[#1])
    ↪                                                                          |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_mul_overflow1}
    ↪                                                                                  |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_mul_overflow1}
    ↪                                                                                  |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
```

```
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,8), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+----------------------+-----------------------+--------------------------------------+
| f1                   | f2                    | f1 * f2                              |
+----------------------+-----------------------+--------------------------------------+
| 12345678901234.12345 | 12345678901234.123456 | 152415787532377393748917544.09724464 |
+----------------------+-----------------------+--------------------------------------+
```

如果调大decimal_overflow_scale的值，比如set decimal_overflow_scale=9;，按照溢出时的规则进行计算：MAX_P - decimal_overflow_scale = 38 - 9 = 29，precision - scale = 41 - 11 = 30 > 29，且 scale > decimal_overflow_
↪ scale，适用溢出规则 3，最终计算出的结果类型为：decimal(38,9)：

```
explain verbose select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 * f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
```

```
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test.test_decimal_mul_overflow1(test_decimal_mul_overflow1), PREAGGREGATION: ON
    ↪                                        |
|      partitions=1/1 (test_decimal_mul_overflow1)
    ↪                                                                                  |
|      tablets=10/10, tabletList=1750210355963,1750210355965,1750210355967 ...
    ↪                                                        |
|      cardinality=1, avgRowSize=3145.0, numNodes=1
    ↪                                                                                |
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (f1[#0] * f2[#1])
    ↪                                                                          |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
```

```
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_mul_overflow1}
    ↪                                                                                |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_mul_overflow1}
    ↪                                                                                |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+----------------------+----------------------+--------------------------------------+
| f1                   | f2                   | f1 * f2                              |
+----------------------+----------------------+--------------------------------------+
| 12345678901234.12345 | 12345678901234.123456 | 152415787532377393748917544.097244643 |
+----------------------+----------------------+--------------------------------------+
```

如果继续调大decimal_overflow_scale的值，比如set decimal_overflow_scale=12;，按照溢出时的规则进行计算：MAX_P - decimal_overflow_scale = 38 - 12 = 26，precision - scale = 41 - 11 = 30 > 26，且 scale < decimal_
↪ overflow_scale，此时适用溢出规则 2，最终计算出的结果类型为：decimal(38,11)：

```
explain verbose select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
```

```
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 * f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
```

```
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|     TABLE: test.test_decimal_mul_overflow1(test_decimal_mul_overflow1), PREAGGREGATION: ON
    ↪                                       |
|     partitions=1/1 (test_decimal_mul_overflow1)
    ↪                                                                                  |
|     tablets=10/10, tabletList=1750210355963,1750210355965,1750210355967 ...
    ↪                                                      |
|     cardinality=1, avgRowSize=3145.0, numNodes=1
    ↪                                                                                |
```

```
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (f1[#0] * f2[#1])
    ↪                                                                                |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_mul_overflow1}
    ↪                                                                                |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_mul_overflow1}
    ↪                                                                                |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(20,5), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(21,6), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,11), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 * f2 from test_decimal_mul_overflow1;
+----------------------+----------------------+----------------------------------------+
| f1                   | f2                   | f1 * f2                                |
+----------------------+----------------------+----------------------------------------+
| 12345678901234.12345 | 12345678901234.123456 | 152415787532377393748917544.09724464320 |
+----------------------+----------------------+----------------------------------------+
```

乘法溢出时开启 decimal256

```
create table test_decimal_mul_overflow_dec256(f1 decimal(38, 19), f2 decimal(38, 19)) properties(
    ↪ 'replication_num'='1');
insert into test_decimal_mul_overflow_dec256 values('9999999999999999999.9999999999999999999', '
    ↪ 9999999999999999999.9999999999999999999');
```

默认情况下 (enable_decimal256=false)，相乘的实际结果会溢出，此时可以开启 decimal256：set enable_
↪ decimal256=true，可以计算出精确的结果，结果类型为 decimal(76, 38)：

```
set enable_decimal256=true;

elect f1, f2, f1 * f2 from test_decimal_mul_overflow_dec256;
+--
    ↪ ---------------------------------------+----------------------------------------+----------------------
    ↪
| f1                                     | f2                                              | f1 * f2
    ↪                                                                                  |
+--
    ↪ ---------------------------------------+----------------------------------------+----------------------
    ↪
| 9999999999999999999.9999999999999999999 | 9999999999999999999.9999999999999999999 |
    ↪ 99999999999999999999999999999999999998.00000000000000000000000000000000000001 |
+--
    ↪ ---------------------------------------+----------------------------------------+----------------------
    ↪
```

除法不溢出

```
create table test_decimal_div_no_overflow(f1 decimal(19, 9), f2 decimal(19, 9)) properties('
    ↪ replication_num'='1');

insert into test_decimal_div_no_overflow values('1234567890.123456789', '234567890.123456789');
```

根据除法结果精度的计算规则，默认配置下 (enable_decimal256=false, decimal_overflow_scale=6, div_
↪ precision_increment=4)，正常计算出来的结果类型是decimal(19 + 9 + 4, 9 + 4)，即decimal(32, 13)，
precision 没有溢出，结果的最终类型就是decimal(32, 13)：

```
 explain verbose select f1, f2, f1 / f2 from test_decimal_div_no_overflow;
+--
    ↪ ---------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
```

```
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
```

```
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test_decimal.test_decimal_div_no_overflow(test_decimal_div_no_overflow),
    ↪ PREAGGREGATION: ON                                  |
|      partitions=1/1 (test_decimal_div_no_overflow)
```

```
    ↪                                                                                                                  |
|      tablets=10/10, tabletList=1750210335692,1750210335694,1750210335696 ...
    ↪                                                                |
|      cardinality=1, avgRowSize=0.0, numNodes=1
    ↪                                                                                                                  |
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(32,22)) / f2[#1])
    ↪                                                        |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_div_no_overflow}
    ↪                                                                                                                  |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_div_no_overflow}
    ↪                                                                                                                  |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(32,13), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |

select f1, f2, f1 / f2 from test_decimal_div_no_overflow;
+----------------------+--------------------+-----------------+
```

```
| f1                    | f2                    | f1 / f2         |
+-----------------------+-----------------------+-----------------+
| 1234567890.123456789 | 234567890.123456789 | 5.2631580966759 |
+-----------------------+-----------------------+-----------------+
```

如果期望结果保留更多小数位，可以调大div_precision_increment，比如admin set frontend config('div_precision_increment'='8');，则根据上述计算规则，计算出的结果类型为decimal(36, 17)：

```
admin set frontend config('div_precision_increment'='8');
explain verbose select f1, f2, f1 / f2 from test_decimal_div_no_overflow;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
```

```
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|       TABLE: test.test_decimal_div_no_overflow(test_decimal_div_no_overflow), PREAGGREGATION: ON
    ↪                                                 |
|       partitions=1/1 (test_decimal_div_no_overflow)
    ↪                                                                                          |
|       tablets=10/10, tabletList=1750210354910,1750210354912,1750210354914 ...
    ↪                                                        |
|       cardinality=1, avgRowSize=3120.0, numNodes=1
    ↪                                                                                  |
|       pushAggOp=NONE
    ↪
    ↪ |
|       desc: 0
    ↪
    ↪ |
|       final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(36,26)) / f2[#1])
    ↪                                                    |
|       final project output tuple id: 1
    ↪
    ↪ |
|       tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_div_no_overflow}
    ↪                                                                                      |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
```

```
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_div_no_overflow}
    ↪                                                                                  |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(19,9), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(36,17), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 / f2 from test_decimal_div_no_overflow;
+----------------------+---------------------+---------------------+
| f1                   | f2                  | f1 / f2             |
+----------------------+---------------------+---------------------+
| 1234567890.123456789 | 234567890.123456789 | 5.26315809667590986 |
+----------------------+---------------------+---------------------+
```

除法溢出规则 1

```
create table test_decimal_div_overflow1(f1 decimal(27, 8), f2 decimal(27, 8)) properties('
    ↪ replication_num'='1');

insert into test_decimal_div_overflow1 values('123456789012345678.12345678', '
    ↪ 23456789012345678.12345678');
```

根据除法结果精度的计算规则，默认配置下 (enable_decimal256=false, decimal_overflow_scale=6, div_
↪ precision_increment=4)，正常计算出来的结果类型是decimal(27 + 8 + 4, 8 + 4)，即decimal(39, 12)。precision 溢出了，需要按照溢出时的规则重新计算：MAX_P - decimal_overflow_scale = 38 - 6 = 32，precision - s1 = 39 - 8 = 31 < 32，所以适用溢出时scale规则 1，结果 scale 为 (MAX_P - (precision - s1))+ div_precision_increment = (38 - (39 - 8)) + 4 = 11，结果类型为decimal(38, 11)：

```
explain verbose select f1, f2, f1 / f2 from test_decimal_div_overflow1;
+--
    ↪ ------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ------------------------------------------------------------------------------------------------------------
    ↪
```

```
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
```

```
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test_decimal.test_decimal_div_overflow1(test_decimal_div_overflow1), PREAGGREGATION
    ↪ : ON                                        |
|      partitions=1/1 (test_decimal_div_overflow1)
    ↪                                                                                          |
|      tablets=10/10, tabletList=1750210336251,1750210336253,1750210336255 ...
    ↪                                                      |
```

```
|      cardinality=1, avgRowSize=3455.0, numNodes=1
    ↪                                                
```

```
| 123456789012345678.12345678 | 23456789012345678.12345678 | 5.26315809667 |
+-----------------------------+----------------------------+---------------+
```

如果调大decimal_overflow_scale的值，比如set decimal_overflow_scale=8;，按照溢出时的规则进行计算：MAX_P - decimal_overflow_scale = 38 - 8 = 30，precision - s1 = 39 - 8 = 31 > 30，且 s1 == decimal_overflow_scale，适用溢出规则 3，最终计算出的结果类型为：decimalv3(38,12)：

```
set decimal_overflow_scale=8;
explain verbose select f1, f2, f1 / f2 from test_decimal_div_overflow1;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
```

```
|     ↪
|     ↪ |
|       MYSQL_PROTOCAL
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   1:VEXCHANGE
|     ↪
|     ↪ |
|       offset: 0
|     ↪
|     ↪ |
|       distribute expr lists:
|     ↪
|     ↪ |
|       tuple ids: 1N
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
| PLAN FRAGMENT 1
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   PARTITION: RANDOM
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   HAS_COLO_PLAN_NODE: false
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   STREAM DATA SINK
|     ↪
|     ↪ |
|     EXCHANGE ID: 01
```

```
|     ↪
|     ↪ |
|     UNPARTITIONED
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   0:VOlapScanNode(59)
|     ↪
|     ↪ |
|       TABLE: test.test_decimal_div_overflow1(test_decimal_div_overflow1), PREAGGREGATION: ON
|     ↪                                                 |
|       partitions=1/1 (test_decimal_div_overflow1)
|     ↪                                                                                   |
|       tablets=10/10, tabletList=1750210355035,1750210355037,1750210355039 ...
|     ↪                                                 |
|       cardinality=1, avgRowSize=3355.0, numNodes=1
|     ↪                                                                                   |
|       pushAggOp=NONE
|     ↪
|     ↪ |
|       desc: 0
|     ↪
|     ↪ |
|       final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(38,20)) / f2[#1])
|     ↪                                                 |
|       final project output tuple id: 1
|     ↪
|     ↪ |
|       tuple ids: 0
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
| Tuples:
|     ↪
|     ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_div_overflow1}
|     ↪                                                                                   |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(27,8), nullable=true,
|     ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(27,8), nullable=true,
|     ↪ isAutoIncrement=false, subColPath=null}       |
```

```
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_div_overflow1}
    ↪                                                
```

```
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
```

```
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|       TABLE: test_decimal.test_decimal(test_decimal), PREAGGREGATION: ON
    ↪                                                                   |
|       partitions=1/1 (test_decimal)
    ↪
    ↪ |
|       tablets=10/10, tabletList=1750210334096,1750210334098,1750210334100 ...
    ↪                                                       |
|       cardinality=1, avgRowSize=3250.0, numNodes=1
    ↪                                                                                |
```

```
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(38,12)) / f2[#1])
    ↪                                                  |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal}
    ↪                                                                                                |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal}
    ↪                                                                                                |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,8), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 / f2 from test_decimal;
+-------------------------+-------------------------+-------------+
| f1                      | f2                      | f1 / f2     |
+-------------------------+-------------------------+-------------+
| 123456789012345678.1234 | 23456789012345678.1234 |  5.26315809 |
+-------------------------+-------------------------+-------------+
```

如果期望结果保留更多小数位，可以调大div_precision_increment，比如admin set frontend config('div_precision_increment'='8');，则根据上述计算规则，计算出的结果类型为decimal(38, 12)：

```
admin set frontend config('div_precision_increment'='8');

explain verbose select f1, f2, f1 / f2 from test_decimal;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
```

```
|     ↪ |
|       MYSQL_PROTOCAL
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   1:VEXCHANGE
|     ↪
|     ↪ |
|       offset: 0
|     ↪
|     ↪ |
|       distribute expr lists:
|     ↪
|     ↪ |
|       tuple ids: 1N
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
| PLAN FRAGMENT 1
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   PARTITION: RANDOM
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   HAS_COLO_PLAN_NODE: false
|     ↪
|     ↪ |
|
|     ↪
|     ↪ |
|   STREAM DATA SINK
|     ↪
|     ↪ |
|     EXCHANGE ID: 01
|     ↪
```

```
|     ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|      TABLE: test_decimal.test_decimal(test_decimal), PREAGGREGATION: ON
    ↪                                                                  |
|      partitions=1/1 (test_decimal)
    ↪
    ↪ |
|      tablets=10/10, tabletList=1750210334096,1750210334098,1750210334100 ...
    ↪                                                          |
|      cardinality=2, avgRowSize=3240.0, numNodes=1
    ↪                                                                                  |
|      pushAggOp=NONE
    ↪
    ↪ |
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(38,16)) / f2[#1])
    ↪                                                  |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal}
    ↪
    ↪ |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
```

```
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal}
    ↪
    ↪ |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,12), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 / f2 from test_decimal;
+-------------------------+------------------------+-----------------+
| f1                      | f2                     | f1 / f2         |
+-------------------------+------------------------+-----------------+
| 123456789012345678.1234 | 23456789012345678.1234 |  5.263158096675 |
+-------------------------+------------------------+-----------------+
```

如果开启 decimal256(`set enable_decimal256 = true;`)，则正常计算出的结果 precision 没有溢出，结果类型是`decimal(46, 8)`：

```
set enable_decimal256=true;

admin set frontend config('div_precision_increment'='4');

explain verbose select f1, f2, f1 / f2 from test_decimal;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
```

```
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
```

```
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
    ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|       TABLE: test_decimal.test_decimal(test_decimal), PREAGGREGATION: ON
    ↪                                                                    |
|       partitions=1/1 (test_decimal)
    ↪
    ↪ |
|       tablets=10/10, tabletList=1750210334096,1750210334098,1750210334100 ...
    ↪                                                        |
|       cardinality=2, avgRowSize=3240.0, numNodes=1
    ↪                                                                                |
|       pushAggOp=NONE
    ↪
    ↪ |
```

```
|      desc: 0
    ↪
    ↪ |
|      final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(46,12)) / f2[#1])
    ↪                                                 |
|      final project output tuple id: 1
    ↪
    ↪ |
|      tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal}
    ↪                                                                                                  |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal}
    ↪                                                                                                  |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(38,4), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}      |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(46,8), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |

select f1, f2, f1 / f2 from test_decimal;
+-------------------------+-------------------------+-------------+
| f1                      | f2                      | f1 / f2     |
+-------------------------+-------------------------+-------------+
| 123456789012345678.1234 | 23456789012345678.1234 |  5.26315809 |
+-------------------------+-------------------------+-------------+
```

除法溢出规则 3

```
create table test_decimal_div_overflow3(f1 decimal(38, 7), f2 decimal(38, 7)) properties('
    ↪ replication_num'='1');
```

```
insert into test_decimal_div_overflow3 values('123456789012345678.1234567', '
    ↪ 23456789012345678.1234567');
```

根据除法结果精度的计算规则，默认配置下(enable_decimal256=false, decimal_overflow_scale=6, div_
↪ precision_increment=4)，正常计算出来的结果类型是decimal(38 + 7 + 4, 7 + 4)，即decimal(49, 11)。
precision 溢出了，需要按照溢出时的规则重新计算：MAX_P - decimal_overflow_scale = 38 - 6 = 32，precision
- s1 = 49 - 7 = 42 > 32，s1 = 7 > decimal_overflow_scale，所以适用溢出时scale规则 3，结果 scale 为decimal_
↪ overflow_scale + div_precision_increment = 6 + 4 = 10，结果类型为decimal(38, 10)：

```
explain verbose select f1, f2, f1 / f2 from test_decimal_div_overflow3;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| Explain String(Nereids Planner)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| PLAN FRAGMENT 0
    ↪
    ↪ |
|   OUTPUT EXPRS:
    ↪
    ↪ |
|     f1[#2]
    ↪
    ↪ |
|     f2[#3]
    ↪
    ↪ |
|     f1 / f2[#4]
    ↪
    ↪ |
|   PARTITION: UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
```

```
    ↪ |
|   VRESULT SINK
    ↪
    ↪ |
|      MYSQL_PROTOCAL
    ↪
    ↪ |
|
    ↪
    ↪ |
|   1:VEXCHANGE
    ↪
    ↪ |
|      offset: 0
    ↪
    ↪ |
|      distribute expr lists:
    ↪
    ↪ |
|      tuple ids: 1N
    ↪
    ↪ |
|
    ↪
    ↪ |
| PLAN FRAGMENT 1
    ↪
    ↪ |
|
    ↪
    ↪ |
|   PARTITION: RANDOM
    ↪
    ↪ |
|
    ↪
    ↪ |
|   HAS_COLO_PLAN_NODE: false
    ↪
    ↪ |
|
    ↪
    ↪ |
|   STREAM DATA SINK
    ↪
```

```
|     ↪ |
|     EXCHANGE ID: 01
    ↪
    ↪ |
|     UNPARTITIONED
    ↪
    ↪ |
|
    ↪
    ↪ |
|   0:VOlapScanNode(59)
    ↪
    ↪ |
|     TABLE: test_decimal.test_decimal_div_overflow3(test_decimal_div_overflow3), PREAGGREGATION
    ↪ : ON                                        |
|     partitions=1/1 (test_decimal_div_overflow3)
    ↪                                                                                          |
|     tablets=10/10, tabletList=1750210336825,1750210336827,1750210336829 ...
    ↪                                                          |
|     cardinality=1, avgRowSize=0.0, numNodes=1
    ↪                                                                                        |
|     pushAggOp=NONE
    ↪
    ↪ |
|     desc: 0
    ↪
    ↪ |
|     final projections: f1[#0], f2[#1], (CAST(f1[#0] AS decimalv3(38,17)) / f2[#1])
    ↪                                                  |
|     final project output tuple id: 1
    ↪
    ↪ |
|     tuple ids: 0
    ↪
    ↪ |
|
    ↪
    ↪ |
| Tuples:
    ↪
    ↪ |
| TupleDescriptor{id=0, tbl=test_decimal_div_overflow3}
    ↪                                                                                  |
|   SlotDescriptor{id=0, col=f1, colUniqueId=0, type=decimalv3(38,7), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
```

```
|   SlotDescriptor{id=1, col=f2, colUniqueId=1, type=decimalv3(38,7), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|
    ↪
    ↪ |
| TupleDescriptor{id=1, tbl=test_decimal_div_overflow3}
    ↪                                                                                |
|   SlotDescriptor{id=2, col=f1, colUniqueId=0, type=decimalv3(38,7), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=3, col=f2, colUniqueId=1, type=decimalv3(38,7), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null}       |
|   SlotDescriptor{id=4, col=null, colUniqueId=null, type=decimalv3(38,10), nullable=true,
    ↪ isAutoIncrement=false, subColPath=null} |
```

计算结果：

```
select f1, f2, f1 / f2 from test_decimal_div_overflow3;
+---------------------------+---------------------------+--------------+
| f1                        | f2                        | f1 / f2      |
+---------------------------+---------------------------+--------------+
| 123456789012345678.1234567 | 23456789012345678.1234567 | 5.2631580966 |
+---------------------------+---------------------------+--------------+
```

聚合运算

- SUM / MULTI_DISTINCT_SUM: SUM(DECIMAL(a, b)) -> DECIMAL(MAX_P, b).
- AVG: AVG(DECIMAL(a, b)) -> DECIMAL(MAX_P, max(b, 4)).

默认规则

除上述提到的函数外，其余表达式都使用默认规则进行精度推演。即对于表达式 `expr(DECIMAL(a, b))`，结果类型同样也是 DECIMAL(a, b)。

调整结果精度

不同用户对 DECIMAL 的精度要求各不相同，上述规则为当前 Doris 的默认行为，如果用户有不同的精度需求，可以通过以下方式进行精度调整：1. 如果期望的结果精度大于默认精度，可以通过调整入参精度来调整结果精度。例如用户期望计算`AVG(col)`得到 DECIMAL(x, y) 作为结果，其中`col`的类型为 DECIMAL(a, b)，则可以改写表达式为`AVG(CAST(col as DECIMAL(x, y)))`。2. 如果期望的结果精度小于默认精度，可以通过对输出结果求近似得到想要的精度。例如用户期望计算`AVG(col)`得到 DECIMAL(x, y) 作为结果，其中`col`的类型为 DECIMAL(a, b)，则可以改写表达式为`ROUND(AVG(col), y)`。

为什么需要 DECIMAL

Doris 中的 DECIMAL 是真正意义上的高精度定点数，Decimal 有以下核心优势：1. 可表示范围更大。DECIMAL 中 precision 和 scale 的取值范围都进行了明显扩充。2. 性能更高。老版本的 DECIMAL 在内存中需要占用 16 bytes，在存储中占用 12 bytes，而 DECIMAL 进行了自适应调整（如下表格）。

precision	占用空间（内存/磁盘）
0 < precision <= 9	4 bytes
9 < precision <= 18	8 bytes
18 < precision <= 38	16 bytes
38 < precision <= 76	32 bytes

3. 更完备的精度推演。对于不同的表达式，应用不同的精度推演规则对结果的精度进行推演。

keywords

```
DECIMAL
```

7.1.1.3 日期类型

7.1.1.3.1 DATE

DATE

name

DATE

描述

```
DATE类型
    日期类型，目前的取值范围是['0000-01-01', '9999-12-31']，默认的打印形式是'yyyy-MM-dd'
```

举例

```
SELECT DATE('2003-12-31 01:02:03');
+----------------------------+
| DATE('2003-12-31 01:02:03') |
+----------------------------+
| 2003-12-31                 |
+----------------------------+
```

keywords

```
DATE
```

7.1.1.3.2 TIME

TIME

name

TIME

描述

TIME 类型时间类型，可以作为查询结果出现，暂时不支持建表存储。表示范围为 [-838:59:59, 838:59:59]。当前 Doris 中，TIME 作为计算结果的正确性是有保证的（如 timediff 等函数），但不推荐手动 CAST 产生 TIME 类型。TIME 类型不会在常量折叠中进行计算。

举例

```
mysql> select timediff('2020-01-01 12:05:03', '2020-01-01 08:02:15');
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
| timediff(cast('2020-01-01 12:05:03' as DATETIMEV2(0)), cast('2020-01-01 08:02:15' as DATETIMEV2
    ↪ (0))) |
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
| 04:02:48
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
1 row in set (0.12 sec)

mysql> select timediff('2020-01-01', '2000-01-01');
+-----------------------------------------------------------------------------------+
| timediff(cast('2020-01-01' as DATETIMEV2(0)), cast('2000-01-01' as DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------+
| 838:59:59                                                                         |
+-----------------------------------------------------------------------------------+
1 row in set (0.11 sec)
```

keywords

```
TIME
```

7.1.1.3.3 DATETIME

DATETIME

DATETIMEV2

描述

DATETIME([P]) 日期时间类型，可选参数 P 表示时间精度，取值范围是 [0, 6]，即最多支持 6 位小数（微秒）。不设置时为 0。取值范围是 [‘0000-01-01 00:00:00[.000000]’, ‘9999-12-31 23:59:59[.999999]’]. 打印的形式是’yyyy-MM-dd HH:mm:ss.SSSSSS’

note

DATETIME 支持了最多到微秒的时间精度。在使用 BE 端解析导入的 DATETIME 类型数据时（如使用 Stream load、Spark load 等），或开启新优化器后在 FE 端解析 DATETIME 类型数据时，将会对超出当前精度的小数进行四舍五入。将带有小数秒部分的 DATETIME 值插入到具有较少小数位的相同类型的列中会导致四舍五入。

DATETIME 读入时支持解析时区，格式为原本 DATETIME 字面量后紧贴时区：

```
<date> <time>[<timezone>]
```

关于<timezone>的具体支持格式，请见时区。需要注意的是，DATE, DATEV2, DATETIME, DATETIMEV2 类型均不包含时区信息。例如，一个输入的时间字符串 “2012-12-12 08:00:00+08:00” 经解析并转换至当前时区 “+02:00”，得到实际值 “2012-12-12 02:00:00” 后存储于 DATETIME 列中，则之后无论本集群环境变量如何改变，该值本身都不会发生变化。

举例

```
mysql> select @@time_zone;
+----------------+
| @@time_zone    |
+----------------+
| Asia/Hong_Kong |
+----------------+
1 row in set (0.11 sec)

mysql> insert into dtv23 values ("2020-12-12 12:12:12Z"), ("2020-12-12 12:12:12GMT"), ("
    ↪ 2020-12-12 12:12:12+02:00"), ("2020-12-12 12:12:12America/Los_Angeles");
Query OK, 4 rows affected (0.17 sec)

mysql> select * from dtv23;
+--------------------------+
| k0                       |
+--------------------------+
| 2020-12-12 20:12:12.000 |
| 2020-12-12 20:12:12.000 |
| 2020-12-13 04:12:12.000 |
| 2020-12-12 18:12:12.000 |
+--------------------------+
4 rows in set (0.15 sec)
```

keywords

```
DATETIME
```

7.1.1.4 字符串类型

7.1.1.4.1 CHAR

description

CHAR(M)

定长字符串，M 代表的是定长字符串的字节长度。M 的范围是 1-255

keywords

CHAR

7.1.1.4.2 VARCHAR

description

VARCHAR(M)

变长字符串，M 代表的是变长字符串的字节长度。M 的范围是 1-65533。

注意：变长字符串是以 UTF-8 编码存储的，因此通常英文字符占 1 个字节，中文字符占 3 个字节。

keywords

VARCHAR

7.1.1.4.3 STRING

description

STRING

变长字符串，默认支持 1048576 字节（1MB），可调大到 2147483643 字节（2G），可通过 be 配置 `string_type_length_soft_limit_bytes`调整。String 类型只能用在 value 列，不能用在 key 列和分区分桶列 String 类型只能用在 value 列，不能用在 key 列和分区分桶列。

注意：变长字符串是以 UTF-8 编码存储的，因此通常英文字符占 1 个字节，中文字符占 3 个字节。

keywords

STRING

7.1.1.5 半结构化类型

7.1.1.5.1 ARRAY

描述

ARRAY<T>

由 T 类型元素组成的数组，不能作为 key 列使用。目前支持在 Duplicate 和 Unique 模型的表中使用。

2.0 版本之后支持在 Unique 模型的表中非 key 列使用。

T 支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL, DATE,
DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

举例

建表示例如下：

```
mysql> CREATE TABLE `array_test` (
  `id` int(11) NULL COMMENT "",
  `c_array` ARRAY<int(11)> NULL COMMENT ""
) ENGINE=OLAP
DUPLICATE KEY(`id`)
COMMENT "OLAP"
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"in_memory" = "false",
"storage_format" = "V2"
);
```

插入数据示例：

```
mysql> INSERT INTO `array_test` VALUES (1, [1,2,3,4,5]);
mysql> INSERT INTO `array_test` VALUES (2, [6,7,8]), (3, []), (4, null);
```

查询数据示例：

```
mysql> SELECT * FROM `array_test`;
+------+-----------------+
| id   | c_array         |
+------+-----------------+
|    1 | [1, 2, 3, 4, 5] |
|    2 | [6, 7, 8]       |
|    3 | []              |
|    4 | NULL            |
+------+-----------------+
```

7.1.1.5.2 MAP

MAP

name

MAP

描述

MAP<K, V>

由 K, V 类型元素组成的 map，不能作为 key 列使用。目前支持在 Duplicate，Unique 模型的表中使用。

K,V 支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL, DECIMALV3, DATE,
DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

举例

建表示例如下：

```
CREATE TABLE IF NOT EXISTS test.simple_map (
  `id` INT(11) NULL COMMENT "",
  `m` Map<STRING, INT> NULL COMMENT ""
) ENGINE=OLAP
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"storage_format" = "V2"
);
```

插入数据示例：

```
mysql> INSERT INTO simple_map VALUES(1, {'a': 100, 'b': 200});
```

```
curl --location-trusted -uroot: -T events.json -H "format: json" -H "read_json_by_line: true"
    ↪ http://fe_host:8030/api/test/simple_map/_stream_load
##### 返回结果
{
    "TxnId": 106134,
    "Label": "5666e573-9a97-4dfc-ae61-2d6b61fdffd2",
    "Comment": "",
    "TwoPhaseCommit": "false",
    "Status": "Success",
    "Message": "OK",
    "NumberTotalRows": 10293125,
    "NumberLoadedRows": 10293125,
    "NumberFilteredRows": 0,
    "NumberUnselectedRows": 0,
    "LoadBytes": 2297411459,
    "LoadTimeMs": 66870,
    "BeginTxnTimeMs": 1,
    "StreamLoadPutTimeMs": 80,
    "ReadDataTimeMs": 6415,
    "WriteDataTimeMs": 10550,
    "CommitAndPublishTimeMs": 38
}
```

查询数据示例：

```
mysql> SELECT * FROM simple_map;
+------+-----------------------------+
| id   | m                           |
+------+-----------------------------+
|    1 | {'a':100, 'b':200}          |
|    2 | {'b':100, 'c':200, 'd':300} |
|    3 | {'a':10, 'd':200}           |
+------+-----------------------------+
```

查询 map 列示例：

```
mysql> SELECT m FROM simple_map;
+-----------------------------+
| m                           |
+-----------------------------+
| {'a':100, 'b':200}          |
| {'b':100, 'c':200, 'd':300} |
| {'a':10, 'd':200}           |
+-----------------------------+
```

map 取值示例：

```
mysql> SELECT m['a'] FROM simple_map;
+-----------------------------+
| %element_extract%(`m`, 'a') |
+-----------------------------+
|                         100 |
|                        NULL |
|                          10 |
+-----------------------------+
```

map 支持的 functions 示例：

```
##### map construct

mysql> SELECT map('k11', 1000, 'k22', 2000)['k11'];
+-----------------------------------------------------------+
| %element_extract%(map('k11', 1000, 'k22', 2000), 'k11') |
+-----------------------------------------------------------+
|                                                      1000 |
+-----------------------------------------------------------+

mysql> SELECT map('k11', 1000, 'k22', 2000)['nokey'];
+-------------------------------------------------------------+
| %element_extract%(map('k11', 1000, 'k22', 2000), 'nokey') |
```

```
+-----------------------------------------------------------+
|                                                      NULL |
+-----------------------------------------------------------+
1 row in set (0.06 sec)

##### map size

mysql> SELECT map_size(map('k11', 1000, 'k22', 2000));
+----------------------------------------+
| map_size(map('k11', 1000, 'k22', 2000)) |
+----------------------------------------+
|                                      2 |
+----------------------------------------+

mysql> SELECT id, m, map_size(m) FROM simple_map ORDER BY id;
+------+-----------------------------+---------------+
| id   | m                           | map_size(`m`) |
+------+-----------------------------+---------------+
|    1 | {"a":100, "b":200}          |             2 |
|    2 | {"b":100, "c":200, "d":300} |             3 |
|    2 | {"a":10, "d":200}           |             2 |
+------+-----------------------------+---------------+
3 rows in set (0.04 sec)

##### map_contains_key

mysql> SELECT map_contains_key(map('k11', 1000, 'k22', 2000), 'k11');
+-----------------------------------------------------------+
| map_contains_key(map('k11', 1000, 'k22', 2000), 'k11') |
+-----------------------------------------------------------+
|                                                         1 |
+-----------------------------------------------------------+
1 row in set (0.08 sec)

mysql> SELECT id, m, map_contains_key(m, 'k1') FROM simple_map ORDER BY id;
+------+-----------------------------+-----------------------------+
| id   | m                           | map_contains_key(`m`, 'k1') |
+------+-----------------------------+-----------------------------+
|    1 | {"a":100, "b":200}          |                           0 |
|    2 | {"b":100, "c":200, "d":300} |                           0 |
|    2 | {"a":10, "d":200}           |                           0 |
+------+-----------------------------+-----------------------------+
3 rows in set (0.10 sec)

mysql> SELECT id, m, map_contains_key(m, 'a') FROM simple_map ORDER BY id;
```

```
+------+-----------------------------+---------------------------+
| id   | m                           | map_contains_key(`m`, 'a') |
+------+-----------------------------+---------------------------+
|    1 | {"a":100, "b":200}          |                         1 |
|    2 | {"b":100, "c":200, "d":300} |                         0 |
|    2 | {"a":10, "d":200}           |                         1 |
+------+-----------------------------+---------------------------+
3 rows in set (0.17 sec)

##### map_contains_value

mysql> SELECT map_contains_value(map('k11', 1000, 'k22', 2000), NULL);
+-----------------------------------------------------------+
| map_contains_value(map('k11', 1000, 'k22', 2000), NULL) |
+-----------------------------------------------------------+
|                                                         0 |
+-----------------------------------------------------------+
1 row in set (0.04 sec)

mysql> SELECT id, m, map_contains_value(m, '100') FROM simple_map ORDER BY id;
+------+-----------------------------+-----------------------------+
| id   | m                           | map_contains_value(`m`, 100) |
+------+-----------------------------+-----------------------------+
|    1 | {"a":100, "b":200}          |                           1 |
|    2 | {"b":100, "c":200, "d":300} |                           1 |
|    2 | {"a":10, "d":200}           |                           0 |
+------+-----------------------------+-----------------------------+
3 rows in set (0.11 sec)

##### map_keys

mysql> SELECT map_keys(map('k11', 1000, 'k22', 2000));
+-----------------------------------------+
| map_keys(map('k11', 1000, 'k22', 2000)) |
+-----------------------------------------+
| ["k11", "k22"]                          |
+-----------------------------------------+
1 row in set (0.04 sec)

mysql> SELECT id, map_keys(m) FROM simple_map ORDER BY id;
+------+-----------------+
| id   | map_keys(`m`)   |
+------+-----------------+
|    1 | ["a", "b"]      |
|    2 | ["b", "c", "d"] |
```

```
|    2 | ["a", "d"]      |
+------+-----------------+
3 rows in set (0.19 sec)

##### map_values

mysql> SELECT map_values(map('k11', 1000, 'k22', 2000));
+-----------------------------------------+
| map_values(map('k11', 1000, 'k22', 2000)) |
+-----------------------------------------+
| [1000, 2000]                            |
+-----------------------------------------+
1 row in set (0.03 sec)

mysql> SELECT id, map_values(m) FROM simple_map ORDER BY id;
+------+-----------------+
| id   | map_values(`m`) |
+------+-----------------+
|    1 | [100, 200]      |
|    2 | [100, 200, 300] |
|    2 | [10, 200]       |
+------+-----------------+
3 rows in set (0.18 sec)
```

keywords

```
MAP
```

7.1.1.5.3 STRUCT

STRUCT

name

STRUCT

描述

`STRUCT<field_name:field_type [COMMENT 'comment_string'], ... >`

由多个 Field 组成的结构体，也可被理解为多个列的集合。不能作为 Key 使用，目前 STRUCT 仅支持在 Duplicate 模型的表中使用。

一个 Struct 中的 Field 的名字和数量固定，总是为 Nullable，一个 Field 通常由下面部分组成。

- field_name: Field 的标识符，不可重复
- field_type: Field 的类型
- COMMENT: Field 的注释，可选 (暂不支持)

当前可支持的类型有：

```
BOOLEAN, TINYINT, SMALLINT, INT, BIGINT, LARGEINT, FLOAT, DOUBLE, DECIMAL, DECIMALV3, DATE,
DATEV2, DATETIME, DATETIMEV2, CHAR, VARCHAR, STRING
```

在将来的版本我们还将完善：

```
TODO:支持嵌套 STRUCT 或其他的复杂类型
```

举例

建表示例如下：

```
mysql> CREATE TABLE `struct_test` (
  `id` int(11) NULL,
  `s_info` STRUCT<s_id:int(11), s_name:string, s_address:string> NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"storage_format" = "V2",
"light_schema_change" = "true",
"disable_auto_compaction" = "false"
);
```

插入数据示例：

Insert:

```
INSERT INTO `struct_test` VALUES (1, {1, 'sn1', 'sa1'});
INSERT INTO `struct_test` VALUES (2, struct(2, 'sn2', 'sa2'));
INSERT INTO `struct_test` VALUES (3, named_struct('s_id', 3, 's_name', 'sn3', 's_address', 'sa3')
    ↪ );
```

Stream load:

test.csv:

```
1|{"s_id":1, "s_name":"sn1", "s_address":"sa1"}
2|{s_id:2, s_name:sn2, s_address:sa2}
3|{"s_address":"sa3", "s_name":"sn3", "s_id":3}
```

示例：

```
curl --location-trusted -u root -T test.csv  -H "label:test_label" http://host:port/api/test/
    ↪ struct_test/_stream_load
```

查询数据示例：

```
mysql> select * from struct_test;
+------+-------------------+
| id   | s_info            |
+------+-------------------+
|    1 | {1, 'sn1', 'sa1'} |
|    2 | {2, 'sn2', 'sa2'} |
|    3 | {3, 'sn3', 'sa3'} |
+------+-------------------+
3 rows in set (0.02 sec)
```

keywords

```
STRUCT
```

7.1.1.5.4 JSON

JSON 数据类型，用二进制格式高效存储 JSON 数据，通过 JSON 函数访问其内部字段。

默认支持 1048576 字节（1 MB），可调大到 2147483643 字节（2 GB），可通过 BE 配置`string_type_length_soft_limit_bytes` 调整。

```
与普通 String 类型存储的 JSON 字符串相比，JSON 类型有两点优势
1. 数据写入时进行 JSON 格式校验
2. 二进制存储格式更加高效，通过json_extract等函数可以高效访问JSON内部字段，比get_json_xx 函数快几倍
```

> 注意在 1.2.x 版本中，JSON 类型的名字是 JSONB，为了尽量跟 MySQL 兼容，从 2.0.0 版本开始改名为 JSON，老的表仍然可以使用。

语法

定义

```
json_column_name JSON
```

写入 - INSERT INTO VALUE 格式是引号包围的字符串。例如：

```
INSERT INTO table_name(id, json_column_name) VALUES (1, '{"k1": "100"}')
```

- STREAM LOAD 对应列的格式是字符串，不需要额外引号包围。例如：

```
12  {"k1":"v31", "k2": 300}
13  []
14  [123, 456]
```

查询 - 直接将整个 JSON 列 SELECT 出来

```
SELECT json_column_name FROM table_name;
```

- 从 JSON 中提取需要的字段，或者其他信息，参考 JSON 函数，例如：

```
SELECT json_extract(json_column_name, '$.k1') FROM table_name;
```

- JSON 类型可以与整数、字符串、BOOLEAN、ARRAY、MAP 进行类型转换 CAST，例如：

```
SELECT CAST('{"k1": "100"}' AS JSON)
SELECT CAST(json_column_name AS String) FROM table_name;
SELECT CAST(json_extract(json_column_name, '$.k1') AS INT) FROM table_name;
```

> JSON 类型暂时不能用于 GROUP BY，ORDER BY，比较大小

使用示例

用一个从建表、导数据、查询全周期的例子说明JSON数据类型的功能和用法。

创建库表

```
CREATE DATABASE testdb;

USE testdb;

CREATE TABLE test_json (
  id INT,
  j JSON
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 10
PROPERTIES("replication_num" = "1");
```

导入数据

stream load 导入 test_json.csv 测试数据

- 测试数据有 2 列，第一列 ID，第二列是 JSON
- 测试数据有 25 行，其中前 18 行的 JSON 是合法的，后 7 行的 JSON 是非法的

```
1   `\N`
2   null
3   true
```

```
4   false
5   100
6   10000
7   1000000000
8   1152921504606846976
9   6.18
10  "abcd"
11  {}
12  {"k1":"v31", "k2": 300}
13  []
14  [123, 456]
15  ["abc", "def"]
16  [null, true, false, 100, 6.18, "abc"]
17  [{"k1":"v41", "k2": 400}, 1, "a", 3.14]
18  {"k1":"v31", "k2": 300, "a1": [{"k1":"v41", "k2": 400}, 1, "a", 3.14]}
19  ''
20  'abc'
21  abc
22  100x
23  6.a8
24  {x
25  [123, abc]
```

- 由于有 28% 的非法数据，默认会失败报错“too many filtered rows”

```
curl --location-trusted -u root: -T test_json.csv http://127.0.0.1:8840/api/testdb/test_json
    ↪ /_stream_load
{
"TxnId": 12019,
"Label": "744d9821-9c9f-43dc-bf3b-7ab048f14e32",
"TwoPhaseCommit": "false",
"Status": "Fail",
"Message": "too many filtered rows",
"NumberTotalRows": 25,
"NumberLoadedRows": 18,
"NumberFilteredRows": 7,
"NumberUnselectedRows": 0,
"LoadBytes": 380,
"LoadTimeMs": 48,
"BeginTxnTimeMs": 0,
"StreamLoadPutTimeMs": 1,
"ReadDataTimeMs": 0,
"WriteDataTimeMs": 45,
"CommitAndPublishTimeMs": 0,
"ErrorURL": "http://172.21.0.5:8840/api/_load_error_log?file=__shard_2/error_log_insert_stmt
```

```
    ↪ _95435c4bf5f156df-426735082a9296af_95435c4bf5f156df_426735082a9296af"
}
```

- 设置容错率参数 ‘max_filter_ratio: 0.3’

```
curl --location-trusted -u root: -H 'max_filter_ratio: 0.3' -T test_json.csv http
    ↪ ://127.0.0.1:8840/api/testdb/test_json/_stream_load
{
"TxnId": 12017,
"Label": "f37a50c1-43e9-4f4e-a159-a3db6abe2579",
"TwoPhaseCommit": "false",
"Status": "Success",
"Message": "OK",
"NumberTotalRows": 25,
"NumberLoadedRows": 18,
"NumberFilteredRows": 7,
"NumberUnselectedRows": 0,
"LoadBytes": 380,
"LoadTimeMs": 68,
"BeginTxnTimeMs": 0,
"StreamLoadPutTimeMs": 2,
"ReadDataTimeMs": 0,
"WriteDataTimeMs": 45,
"CommitAndPublishTimeMs": 19,
"ErrorURL": "http://172.21.0.5:8840/api/_load_error_log?file=__shard_0/error_log_insert_stmt
    ↪ _a1463f98a7b15caf-c79399b920f5bfa3_a1463f98a7b15caf_c79399b920f5bfa3"
}
```

- 查看 stream load 导入的数据，JSON 类型的列 j 会自动转成 JSON String 展示

```
mysql> SELECT * FROM test_json ORDER BY id;
+------+-----------------------------------------------------------------+
| id   | j                                                               |
+------+-----------------------------------------------------------------+
|    1 |                                                            NULL |
|    2 |                                                            null |
|    3 |                                                            true |
|    4 |                                                           false |
|    5 |                                                             100 |
|    6 |                                                           10000 |
|    7 |                                                      1000000000 |
|    8 |                                              1152921504606846976 |
|    9 |                                                            6.18 |
|   10 |                                                          "abcd" |
|   11 |                                                             {} |
```

```
|   12 |                                          {"k1":"v31","k2":300} |
|   13 |                                                             [] |
|   14 |                                                      [123,456] |
|   15 |                                                  ["abc","def"] |
|   16 |                               [null,true,false,100,6.18,"abc"] |
|   17 |                             [{"k1":"v41","k2":400},1,"a",3.14] |
|   18 | {"k1":"v31","k2":300,"a1":[{"k1":"v41","k2":400},1,"a",3.14]} |
+------+------------------------------------------------------------------+
18 rows in set (0.03 sec)
```

insert into 插入数据

- insert 1 条数据，总数据从 18 条增加到 19 条 “ ‘ mysql> INSERT INTO test_json VALUES(26, ‘{“k1”:“v1”, “k2”: 200}’); Query OK, 1 row affected (0.09 sec) {‘label’:‘insert_4ece6769d1b42fd_ac9f25b3b8f3dc02’, ‘status’:‘VISIBLE’, ‘txnId’:‘12016’}

mysql> SELECT * FROM test_json ORDER BY id; +——+————————————————————————+ | id | j | +——+———————————————————————+ | 1 | NULL | | 2 | null | | 3 | true | | 4 | false | | 5 | 100 | | 6 | 10000 | | 7 | 1000000000 | | 8 | 1152921504606846976 | | 9 | 6.18 | | 10 | “abcd” | | 11 | {} | | 12 | {“k1”:“v31”,“k2”:300} | | 13 | [] | | 14 | [123,456] | | 15 | [“abc”,“def”] | | 16 | [null,true,false,100,6.18,“abc”] | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | | 26 | {“k1”:“v1”,“k2”:200} | +——+————————————————————————+ 19 rows in set (0.03 sec)

######## 查询

######### 用 json_extract 取 json 内的某个字段

1. 获取整个 json，$ 在 json path 中代表 root，即整个 json

+——+————————————————————————+————————————————————————+ | id | j | json_extract(j ↪ , ‘$’) | +——+————————————————————————+————————————————————————+ | 1 | NULL | NULL | | 2 | null | null | | 3 | true | true | | 4 | false | false | | 5 | 100 | 100 | | 6 | 10000 | 10000 | | 7 | 1000000000 | 1000000000 | | 8 | 1152921504606846976 | 1152921504606846976 | | 9 | 6.18 | 6.18 | | 10 | “abcd” | “abcd” | | 11 | {} | {} | | 12 | {“k1”:“v31”,“k2”:300} | {“k1”:“v31”,“k2”:300} | | 13 | [] | [] | | 14 | [123,456] | [123,456] | | 15 | [“abc”,“def”] | [“abc”,“def”] | | 16 | [null,true,false,100,6.18,“abc”] | [null,true,false,100,6.18,“abc”] | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:40 | | 26 | {“k1”:“v1”,“k2”:200} | {“k1”:“v1”,“k2”:200} | +——+————————————————————————+————————————————————————+ 19 rows in set (0.03 sec)

1. 获取 k1 字段，没有 k1 字段的行返回 NULL

mysql> SELECT id, j, json_extract(j, $.k1’) FROM test_json ORDER BY id; +——+——————————————————————————————————+——————————

$$-----------------+|id|j|json_extract('j','.k1') |$$ +——+——————————————————————————+——————————-+ | 1 | NULL | NULL | | 2 | null | NULL | | 3 | true | NULL | | 4 | false | NULL | | 5 | 100 | NULL | | 6 | 10000 | NULL | | 7 | 1000000000 | NULL | | 8 | 1152921504606846976 | NULL | | 9 | 6.18 | NULL | | 10 | “abcd” | NULL | | 11 | {} | NULL | | 12 | {“k1”:“v31”,“k2”:300} | “v31” | | 13 | [] | NULL | | 14 | [123,456] | NULL | | 15 | [“abc”,“def”] | NULL | | 16 | [null,true,false,100,6.18,“abc”] | NULL | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | NULL | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | “v31” | | 26 | {“k1”:“v1”,“k2”:200} | “v1” | +——+——————————————————————————+——————————-+ 19 rows in set (0.03 sec)

1．获取顶层数组的第 0 个元素

mysql> SELECT id, j, json_extract(j, $'[0]')FROMtest_jsonORDERBYid;+-------+---+$$-----------------+|id|j|json_extract('j','[0]') |$$ +——+——————————————————————————+——————————-+ | 1 | NULL | NULL | | 2 | null | NULL | | 3 | true | NULL | | 4 | false | NULL | | 5 | 100 | NULL | | 6 | 10000 | NULL | | 7 | 1000000000 | NULL | | 8 | 1152921504606846976 | NULL | | 9 | 6.18 | NULL | | 10 | “abcd” | NULL | | 11 | {} | NULL | | 12 | {“k1”:“v31”,“k2”:300} | NULL | | 13 | [] | NULL | | 14 | [123,456] | 123 | | 15 | [“abc”,“def”] | “abc” | | 16 | [null,true,false,100,6.18,“abc”] | null | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | {“k1”:“v41”,“k2”:400} | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | NULL | | 26 | {“k1”:“v1”,“k2”:200} | NULL | +——+——————————————————————————+——————————-+ 19 rows in set (0.03 sec)

1．获取整个 json array

mysql> SELECT id, j, json_extract(j, $'.a1')FROMtest_jsonORDERBYid;+-------+---+$$----------+|id|j|json_extract('j','.a1') |$$ +——+——————————————————————————+——————————-+ | 1 | NULL | NULL | | 2 | null | NULL | | 3 | true | NULL | | 4 | false | NULL | | 5 | 100 | NULL | | 6 | 10000 | NULL | | 7 | 1000000000 | NULL | | 8 | 1152921504606846976 | NULL | | 9 | 6.18 | NULL | | 10 | “abcd” | NULL | | 11 | {} | NULL | | 12 | {“k1”:“v31”,“k2”:300} | NULL | | 13 | [] | NULL | | 14 | [123,456] | NULL | | 15 | [“abc”,“def”] | NULL | | 16 | [null,true,false,100,6.18,“abc”] | NULL | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | NULL | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | | 26 | {“k1”:“v1”,“k2”:200} | NULL | +——+——————————————————————————+——————————-+ 19 rows in set (0.02 sec)

1．获取 json array 中嵌套 object 的字段

mysql> SELECT id, j, json_extract(j, $'.a1[0]'), $json_extract(j,'.a1[0].k1')$ FROM test_json ORDER BY id; +——+——————————————————————————+——————————-+——————————-+ | id | j | json_extract(j, $'.a1[0]')|$$json_extract('j','.a1[0].k1') |$$ +——+——————————————————————————+——————————-+——————————-+ | 1 | NULL | NULL | NULL | | 2 | null | NULL | NULL | | 3 | true | NULL | NULL | | 4 | false | NULL | NULL | | 5 | 100 | NULL | NULL | | 6 | 10000 | NULL | NULL | | 7 | 1000000000 | NULL | NULL | | 8 | 1152921504606846976 | NULL | NULL | | 9 | 6.18 | NULL | NULL | | 10 | “abcd” | NULL | NULL | | 11 | {} | NULL | NULL | | 12 | {“k1”:“v31”,“k2”:300} | NULL | NULL | | 13 | [] | NULL | NULL | | 14 | [123,456] | NULL | NULL | | 15 | [“abc”,“def”] | NULL | NULL | | 16 |

[null,true,false,100,6.18,“abc”] | NULL | NULL | | 17 | [{“k1”:“v41”,“k2”:400},1,“a”,3.14] | NULL | NULL | | 18 | {“k1”:“v31”,“k2”:300,“a1”:[{“k1”:“v41”,“k2”:400},1,“a”,3.14]} | {“k1”:“v41”,“k2”:400} | “v41” | | 26 | {“k1”:“v1”,“k2”:200} | NULL | NULL | +——+————————————————————————+————————————+————————————+ 19 rows in set (0.02 sec)

1．获取具体类型的
- json_extract_string 获取 String 类型字段，非 String 类型转成 String

mysql> SELECT id, j, json_extract_string(j, ‘’)$FROM test_j son ORDER BY id;+------+------------------

$-\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ +\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -

mysql> SELECT id, j, json_extract_double(j, '.k2')FROMtestjsonORDERBYid;+−−−−−−+−−−−−−

+——+————————————————————————+————————————+ 19 rows in set (0.03 sec)

######### 用 json_exists_path 检查 json 内的某个字段是否存在

mysql> SELECT id, j, json_exists_path(j, ‘’)$FROM test_j son ORDER BY id;+−−−−−−+−−−−−−

{ “k1” : “v31” , “k2” :300, “a1” :[{ “k1” : “v41” , “k2” :400},1, “a” ,3.14]} | object | | 26 | { “k1” : “v1” , “k2” :200} | object | +——+————————————————————————+————————-+ 19 rows in set (0.02 sec)

mysql> select id, j, json_type(j, ‘.k1′)$from test_j son order by id;$ +————+————————————————————————

7.1.1.5.5 VARIANT

VARIANT

描述

VARIANT 类型用于存储半结构化 JSON 数据，可包含不同基础类型（整数、字符串、布尔等）以及一层数组与嵌套对象。写入时会自动基于 JSON Path 推断子列结构与类型，并将高频路径物化为独立子列，充分利用列式存储和向量化执行，兼顾灵活性与性能。

使用 VARIANT 类型

建表语法

建表时将列类型声明为 VARIANT：

```
CREATE TABLE IF NOT EXISTS ${table_name} (
    k BIGINT,
    v VARIANT
)
PROPERTIES("replication_num" = "1");
```

通过 Schema Template 约束部分 Path 的类型（更多见“扩展类型”）：

> 该功能自 3.1.0 版本支持

```
CREATE TABLE IF NOT EXISTS ${table_name} (
    k BIGINT,
    v VARIANT <
        'id' : INT,            -- path 为 id 的子列被限制为 INT 类型
        'message*' : STRING,   -- 前缀匹配 message* 的子列被限制为 STRING 类型
        'tags*' : ARRAY<TEXT>  -- 前缀匹配 tags* 的子列被限制为 ARRAY<TEXT> 类型
    >
)
PROPERTIES("replication_num" = "1");
```

查询语法

```
-- 访问嵌套字段（返回类型为 VARIANT，需要显式或隐式 CAST 才能聚合/比较）
SELECT v['properties']['title'] FROM ${table_name};

-- 聚合前显式 CAST 为确定类型
SELECT CAST(v['properties']['title'] AS STRING) AS title
FROM ${table_name}
GROUP BY title;

-- 数组查询示例
```

```
SELECT *
FROM ${table_name}
WHERE ARRAY_CONTAINS(CAST(v['tags'] AS ARRAY<TEXT>), 'Doris');
```

基本类型

VARIANT 自动推断的子列基础类型包括：

支持的类型

TinyInt

NULL（等价于 JSON 的 null）

BigInt(64 bit)Double

String(Text)

Jsonb

Variant（嵌套对象）

Array<T>（仅支持一维）

简单的 INSERT 示例：

```
INSERT INTO vartab VALUES
  (1, 'null'),
  (2, NULL),
  (3, 'true'),
  (4, '-17'),
  (5, '123.12'),
  (6, '1.912'),
  (7, '"A quote"'),
  (8, '[-1, 12, false]'),
  (9, '{ "x": "abc", "y": false, "z": 10 }'),
  (10, '"2021-01-01"');
```

提示：日期/时间戳等非标准 JSON 类型在未指定 Schema 时会以字符串形式存储；如需较高计算效率，建议将其提取为静态列或在 Schema Template 中明确声明类型。

扩展类型（Schema Template）

> 该功能自 3.1.0 版本支持

除基本类型外，VARIANT 还可通过 Schema Template 声明以下扩展类型：

- Number（扩展）
- Decimal：Decimal32 / Decimal64 / Decimal128 / Decimal256

- LargeInt
- Datetime
- Date
- IPV4 / IPV6
- Boolean
- ARRAY<T>（T 为以上任意类型，仅支持一维）

注意：预定义的 Schema 只能在建表时指定，当前不支持通过 ALTER 修改（后续可能支持“新增”子列定义，但不支持修改既有子列类型）。

示例：

```
CREATE TABLE test_var_schema (
    id BIGINT NOT NULL,
    v1 VARIANT<
        'large_int_val': LARGEINT,
        'string_val': STRING,
        'decimal_val': DECIMAL(38, 9),
        'datetime_val': DATETIME,
        'ip_val': IPV4
    > NULL
)
PROPERTIES ("replication_num" = "1");

INSERT INTO test_var_schema VALUES (1, '{
    "large_int_val" : "12322222222222222222222",
    "string_val" : "Hello World",
    "decimal_val" : 1.11111111,
    "datetime_val" : "2025-05-16 11:11:11",
    "ip_val" : "127.0.0.1"
}');

SELECT variant_type(v1) FROM test_var_schema;

+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| variant_type(v1)
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| {"datetime_val":"datetimev2","decimal_val":"decimal128i","ip_val":"ipv4","large_int_val":"
    ↪ largeint","string_val":"string"} |
+--
    ↪ ----------------------------------------------------------------------------------------------------
```

```
    ↪
```

{"date": 2020-01-01} 与 {"ip": 127.0.0.1} 均为非法 JSON 文本，正确格式应为 {"date": "2020-01-01"} 与 {"ip": "127.0.0.1"}。

一旦指定 Schema，若 JSON 实际类型与 Schema 冲突且无法转换，将保存为 NULL。例如：

```
INSERT INTO test_var_schema VALUES (1, '{
  "decimal_val" : "1.11111111",
  "ip_val" : "127.xxxxxx.xxxx",
  "large_int_val" : "aaabbccc"
}');

-- 仅 decimal_val 保留
SELECT * FROM test_var_schema;

+------+------------------------------+
| id   | v1                           |
+------+------------------------------+
|    1 | {"decimal_val":1.111111110} |
+------+------------------------------+
```

Schema 仅指导“存储层”的持久化类型，计算逻辑仍以实际数据的动态类型为准：

```
-- 实际 v['a'] 的运行时类型仍可能是 STRING
SELECT variant_type(CAST('{"a" : "12345"}' AS VARIANT<'a' : INT>)['a']);
```

通配符与匹配顺序：

```
CREATE TABLE test_var_schema (
    id BIGINT NOT NULL,
    v1 VARIANT<
        'enumString*' : STRING,
        'enum*' : ARRAY<TEXT>,
        'ip*' : IPV6
    > NULL
)
PROPERTIES ("replication_num" = "1");

-- 若 enumString1 同时匹配上述两个 pattern，则采用定义顺序中第一个匹配到的类型（STRING）
```

如列名中包含 * 且希望按名称精确匹配，可使用：

```
v1 VARIANT<
    MATCH_NAME 'enumString*' : STRING
> NULL
```

匹配成功的子路径默认会展开为独立列。若匹配子列过多导致列数暴增，建议开启 variant_enable_typed_paths_to_sparse（见“配置”）。

类型冲突与提升规则

当同一路径出现不兼容类型（如同一字段既出现整数又出现字符串）时，将提升为 JSONB 类型以避免信息丢失：

```
{"a" : 12345678}
{"a" : "HelloWorld"}
-- a 将被提升为 JSONB
```

转换规则如下表格：

源类型	当前类型	最终类型
TinyInt	BigInt	BigInt
TinyInt	Double	Double
TinyInt	String	JSONB
TinyInt	Array	JSONB
BigInt	Double	JSONB
BigInt	String	JSONB
BigInt	Array	JSONB
Double	String	JSONB
Double	Array	JSONB
Array<Double>	Array<String>	Array<Jsonb>

若需严格限制子列类型（以稳定索引和存储），请结合 Schema Template 明确声明类型。

Variant 索引

索引选择

VARIANT 支持对子列建立 BloomFilter 与 Inverted Index 两类索引。- 高基数等值/IN 过滤：优先使用 BloomFilter（更省存储、写入更高效）。- 需要分词、短语、范围检索：使用 Inverted Index，并根据需求设置 parser/analyzer 等属性。

```
...
PROPERTIES("replication_num" = "1", "bloom_filter_columns" = "v");

-- 利用 BloomFilter 做等值/IN 过滤
SELECT * FROM tbl WHERE v['id'] = 12345678;
SELECT * FROM tbl WHERE v['id'] IN (1, 2, 3);
```

给 VARIANT 列创建 Inverted Index 后，所有子列将继承相同的索引属性（如分词方式）。

```
CREATE TABLE IF NOT EXISTS tbl (
    k BIGINT,
    v VARIANT,
    INDEX idx_v(v) USING INVERTED PROPERTIES("parser" = "english")
```

```
);

-- 全部子列继承 english 分词属性
SELECT * FROM tbl WHERE v['id_1'] MATCH 'Doris';
SELECT * FROM tbl WHERE v['id_2'] MATCH 'Apache';
```

根据子路径指定索引

> 该功能自 3.1.0 版本支持

在 3.1.x/4.0 及之后的版本中，可为 VARIANT 的部分子列单独指定索引属性，甚至在同一路径上同时配置“分词与不分词”的两种倒排索引。指定 Path 索引需配合 Path 类型（Schema Template）使用。

```
-- 常用属性：field_pattern（目标子路径）、analyzer、parser、support_phrase 等
CREATE TABLE IF NOT EXISTS tbl (
    k BIGINT,
    v VARIANT<'content' : STRING>,
    INDEX idx_tokenized(v) USING INVERTED PROPERTIES("parser" = "english", "field_pattern" = "
        ↪ content"),
    INDEX idx_v(v) USING INVERTED PROPERTIES("field_pattern" = "content")
);

-- v.content 同时具备分词与不分词的倒排索引
SELECT * FROM tbl WHERE v['content'] MATCH 'Doris';
SELECT * FROM tbl WHERE v['content'] = 'Doris';
```

支持通配符的 Path 索引：

```
CREATE TABLE IF NOT EXISTS tbl (
    k BIGINT,
    v VARIANT<'pattern_*' : STRING>,
    INDEX idx_tokenized(v) USING INVERTED PROPERTIES("parser" = "english", "field_pattern" = "
        ↪ pattern_*"),
    INDEX idx_v(v) USING INVERTED -- 全局指定非分词索引
);

SELECT * FROM tbl WHERE v['pattern_1'] MATCH 'Doris';
SELECT * FROM tbl WHERE v['pattern_1'] = 'Doris';
```

注意：2.1.7+ 仅支持 InvertedIndex V2 属性（文件更少、写入 IOPS 更低，适配存算分离）。2.1.8+ 不再支持离线 Build Index 构建。

索引失效问题

1. 类型变更导致索引丢失：子列类型发生不兼容变更（如 INT→JSONB）会丢失索引。可通过 Schema Template 固定类型与索引。
2. 查询类型不匹配：

```
-- v['id'] 实际为 STRING，按 INT 进行等值会导致索引失效
SELECT * FROM tbl WHERE v['id'] = 123456;
```

3. 索引配置错误：索引作用于“子列”，对 VARIANT 整体无效。“‘sql – v 本身不具备索引能力 SELECT * FROM tbl WHERE v MATCH ‘Doris’；

– 若需对整体 JSON 文本建索引，可额外存字符串列并建索引 CREATE TABLE IF NOT EXISTS tbl (k BIGINT, v VARIANT, v_str STRING, INDEX idx_v_str(v_str) USING INVERTED PROPERTIES(“parser” = “english”)); SELECT * FROM tbl WHERE v_str MATCH ‘Doris’；“ ‘

嵌套类型

> 该功能为实验特性，如需使用建表时需指定 session 变量 `set enable_variant_flatten_nested = true`

```
{
  "nested" : [{"field1" : 123, "field11" : "123"}, {"field2" : 456, "field22" : "456"}]
}
```

在上面的 JSON 中，数组 nested 包含的对象（object）被称为嵌套数组类型。需要注意的是，目前仅支持一层数组的展开。且顶层数组无法展开：

顶层数组

```
 [{"field1" : 123, "field11" : "123"}, {"field2" : 456, "field22" : "456"}]
```

多层数组

```
{
  "nested" : [{"field1" : 123, "field11" : "123"}, {"field2" : 456, "nested2" : []}]
}
```

多层嵌套

```
{
  "nested" : [{"field1" : 123, "field11" : "123"}, {"field2" : 456, "nested2" : [{"a" : 123}]}]
}
```

以上的数据目前无法展开

建表

```
CREATE TABLE `simple_nested_test` (
  `k` bigint NULL,
  `v` variant NULL
) ENGINE=OLAP
DUPLICATE KEY(`k`)
DISTRIBUTED BY HASH(`k`) BUCKETS 8
PROPERTIES (
  "variant_enable_flatten_nested" = "true"
)
```

insert 与导入

INSERT INTO VALUES

```
CREATE TABLE IF NOT EXISTS variant_tbl (
    k BIGINT,
    v VARIANT
) PROPERTIES("replication_num" = "1");

INSERT INTO variant_tbl VALUES (1, '{"a" : 123}');

select * from variant_tbl;
+------+-----------+
| k    | v         |
+------+-----------+
|    1 | {"a":123} |
+------+-----------+

-- 其中 v['a'] 是 Variant 类型
select v['a'] from variant_tbl;
+--------+
| v['a'] |
+--------+
| 123    |
+--------+

-- v['a']['no_such_key'] 对于不存在的 JSON 键，将返回 NULL
select v['a']['no_such_key'] from variant_tbl;;
+-----------------------+
| v['a']['no_such_key'] |
+-----------------------+
| NULL                  |
+-----------------------+
```

导入（Stream Load）

```
curl --location-trusted -u root: -T gh_2022-11-07-3.json \
  -H "read_json_by_line:true" -H "format:json" \
  http://127.0.0.1:8030/api/test_variant/github_events/_stream_load
```

参考variant

导入完成后可用 SELECT count(*) 或 SELECT * ... LIMIT 1 验证。为提升高并发导入性能，推荐建表选择 RANDOM 分桶并开启 Group Commit（参见官方“Group Commit”文档）。

支持的运算与 CAST 规则

- VARIANT 本身不支持与其他类型直接比较/运算，两个 VARIANT 之间也不支持直接比较。
- 如需比较、过滤、聚合、排序，请对子列显式或隐式 CAST 到确定类型。

```
-- 显式 CAST
SELECT CAST(v['arr'] AS ARRAY<TEXT>) FROM tbl;
SELECT * FROM tbl WHERE CAST(v['decimal'] AS DECIMAL(27, 9)) = 1.111111111;
SELECT * FROM tbl WHERE CAST(v['date'] AS DATE) = '2021-01-02';

-- 隐式 CAST
SELECT * FROM tbl WHERE v['bool'];
SELECT * FROM tbl WHERE v['str'] MATCH 'Doris';
```

- VARIANT 本身不可直接用于 ORDER BY、GROUP BY、JOIN KEY 或聚合参数；对子列 CAST 后可正常使用。
- 字符串类型可隐式转换为 VARIANT。

VARIANT	Castable	Coercible
ARRAY	✓	□
BOOLEAN	✓	✓
DATE/DATETIME	✓	✓
FLOAT	✓	✓
IPV4/IPV6	✓	✓
DECIMAL	✓	✓
MAP	□	□
TIMESTAMP	✓	✓
VARCHAR	✓	✓
JSON	✓	✓

限制

- variant_max_subcolumns_count(仅 3.1 以上的版本支持)：默认 0（不限制 Path 物化列数）。建议在生产设置为 2048（Tablet 级别）以控制列数。超过阈值后，低频/稀疏路径会被收敛到共享数据结构，从该结构查询可能带来性能下降（详见“配置”）。

- 若 Schema Template(仅 3.1 以上的版本支持) 指定了 Path 类型，则该 Path 会被强制提取；当 variant_enable_typed_paths_to_sparse = true 时，它也会计入阈值，可能被收敛到共享结构。
- JSON key 长度 ≤ 255。
- 不支持作为主键或排序键。
- 不支持与其他类型嵌套（如 Array<Variant>、Struct<Variant>）。
- 读取整个 VARIANT 列会扫描所有子字段。若列包含大量子字段，建议额外存储原始 JSON 的 STRING/JSONB 列，以优化如 LIKE 等整体匹配：

```
CREATE TABLE example_table (
  id INT,
  data_variant VARIANT
);
SELECT * FROM example_table WHERE data_variant LIKE '%doris%';

-- 更优做法：额外保留原始 JSON 字符串列用于整体检索
CREATE TABLE example_table (
  id INT,
  data_string STRING,
  data_variant VARIANT
);
SELECT * FROM example_table WHERE data_string LIKE '%doris%';
```

配置

> 自 3.1.0 版本支持 variant_max_subcolumns_count 和 variant_enable_typed_paths_to_sparse 属性，且不支持 ALTER 修改改

```
CREATE TABLE example_table (
  id INT,
  data_variant VARIANT<
      'path_1' : INT,
      'path_2' : STRING,
      properties(
          'variant_max_subcolumns_count' = '2048',
          'variant_enable_typed_paths_to_sparse' = 'true'
      )
  >
);
```

属性

描述

variant_max_subcolumns_count (仅 3.1 以上的版本支持)

控制 Path 物化列数的上限；超过后新增路径可能存放于共享数据结构。默认 0 表示不限，建议设置为 2048；不推荐超过 10000。

variant_enable_typed_paths_to_sparse (仅 3.1 以上的版本支持)

默认指定了 Path 类型后，该 Path 一定会被提取（不计入 variant_max_subcolumns_count）。设置为 true 后也会计入阈值，可能被收敛到共享结构。

达到上限后的行为与调优建议：

1. 超过上限后，新路径写入共享结构；Rowset 合并后也可能触发部分路径回收为共享结构 (仅 3.1 以上的版本支持)。
2. 系统会优先保留非空比例高、访问频率高的路径为物化列 (仅 3.1 以上的版本支持)。
3. 若接近 10000 物化列，对硬件要求较高（建议单机 ≥128G 内存、≥32C）（仅 3.1 以上的版本支持）。
4. 写入侧调优：适度增大客户端 batch_size，或使用 Group Commit（按需增大 group_commit_interval_ms /group_commit_data_bytes）。
5. 若无分桶裁剪需求，建议采用 RANDOM 分桶，并开启 single tablet 导入以降低 compaction 写放大。
6. BE 配置可按导入压力调整 max_cumu_compaction_threads（建议 ≥8）、vertical_compaction_num_columns_per_group=500（提升纵向合并效率，增加内存占用）、segment_cache_memory_percentage=20（提升元数据缓存命中）。
7. 关注 Compaction Score；若持续上升说明 Compaction 跟不动，需要降低导入压力。
8. 避免大范围 SELECT * 或直接扫描 VARIANT；尽量使用具体路径投影 SELECT v['path']。

另：当出现 Stream Load 报错 [DATA_QUALITY_ERROR]Reached max column size limit 2048 时（只有 2.1.x 和 3.0.x 版本会出现该报错），说明合并后的 Tablet Schema 达到列数上限。可按需调整 BE 配置 variant_max_merged_tablet_schema_size（不建议超过 4096，需较高配置机器）。

查看列数、列类型

> 方案一自 3.1.0 版本开始支持，之前的版本建议用 DESC

方案一：使用 variant_type 查看行级 Schema（开销更大但更精确）：

```
SELECT variant_type(v) FROM variant_tbl;
```

方案二：扩展 DESC 展示已物化的子列（仅展示被提取的路径）：

```
SET describe_extend_variant_column = true;
DESC variant_tbl;
```

```
DESCRIBE ${table_name} PARTITION ($partition_name);
```

两种方式可结合使用：方案一精确、方案二高效。

对比 JSON 类型

- 存储：JSON 类型以 JSONB（行存）写入；VARIANT 写入时类型推断并列存化，压缩率更高、存储更小。
- 查询：JSON 需解析；VARIANT 直接列式扫描，通常显著更快。

改造的 ClickBench 测试结果（43 条查询）：- 存储：VARIANT 相比 JSON 约节省 65% 存储空间。- 查询：VARIANT 较 JSON 提速 8 倍以上，性能接近静态列。

存储空间

类型	存储空间
预定义静态列	12.618 GB
VARIANT 类型	12.718 GB
JSON 类型	35.711 GB

节省约 65% 存储容量

查询次数	预定义静态列	VARIANT 类型	JSON 类型
第一次查询 (cold)	233.79s	248.66s	大部分查询超时
第二次查询 (hot)	86.02s	94.82s	789.24s
第三次查询 (hot)	83.03s	92.29s	743.69s

常见问题

1. VARIANT 中的 `null` 与 SQL NULL 有区别吗？

- 没有区别，两者等价。

2. 为什么我的查询/索引没有生效？

- 请检查是否对路径做了正确的 CAST、是否因为类型冲突被提升为 JSONB、或是否误以为给 VARIANT “整体”建的索引可用于子列。

7.1.1.6 聚合类型

7.1.1.6.1 HLL(HyperLogLog)

HLL(HyperLogLog)

描述

HLL HLL 不能作为 key 列使用，支持在 Aggregate 模型、Duplicate 模型和 Unique 模型的表中使用。在 Aggregate 模型表中使用时，建表时配合的聚合类型为 HLL_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。并且 HLL 列只能通过配套的 hll_union_agg、 hll_raw_agg、 hll_cardinality、 hll_hash 进行查询或使用。

HLL 是模糊去重，在数据量大的情况性能优于 Count Distinct。 HLL 的误差通常在 1% 左右，有时会达到 2%。

举例

```
select hour, HLL_UNION_AGG(pv) over(order by hour) uv from(
   select hour, HLL_RAW_AGG(device_id) as pv
   from metric_table -- 查询每小时的累计UV
   where datekey=20200622
group by hour order by 1
) final;
```

keywords

```
HLL,HYPERLOGLOG
```

7.1.1.6.2 BITMAP

BITMAP

描述

BITMAP

BITMAP 类型的列可以在 Aggregate 表、 Unique 表或 Duplicate 表中使用。在 Unique 表或 duplicate 表中使用时，其必须作为非 key 列使用。在 Aggregate 表中使用时，其必须作为非 key 列使用，且建表时配合的聚合类型为 BITMAP_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。并且 BITMAP 列只能通过配套的 bitmap_union_count、 bitmap_union、 bitmap_hash、 bitmap_hash64 等函数进行查询或使用。

离线场景下使用 BITMAP 会影响导入速度，在数据量大的情况下查询速度会慢于 HLL，并优于 Count Distinct。注意：实时场景下 BITMAP 如果不使用全局字典，使用了 bitmap_hash() 可能会导致有千分之一左右的误差。如果这个误差不可接受，可以使用 bitmap_hash64。

举例

建表示例如下：

```
create table metric_table (
  datekey int,
  hour int,
  device_id bitmap BITMAP_UNION
)
aggregate key (datekey, hour)
distributed by hash(datekey, hour) buckets 1
properties(
  "replication_num" = "1"
);
```

插入数据示例：

```
insert into metric_table values
(20200622, 1, to_bitmap(243)),
(20200622, 2, bitmap_from_array([1,2,3,4,5,434543])),
(20200622, 3, to_bitmap(287667876573));
```

查询数据示例：

```
select hour, BITMAP_UNION_COUNT(pv) over(order by hour) uv from(
   select hour, BITMAP_UNION(device_id) as pv
   from metric_table -- 查询每小时的累计UV
   where datekey=20200622
group by hour order by 1
) final;
```

在查询时，可以设置会话变量return_object_data_as_binary为 true，这样 bitmap 会以二进制的形式返回。

keywords

```
BITMAP
```

7.1.1.6.3 QUANTILE_STATE

QUANTILE_STATE

描述

```
QUANTILE_STATE
```

在 2.0 中我们支持了 agg_state 功能，推荐使用 agg_state quantile_union(quantile_state not null) 来代替本类型。

QUANTILE_STATE 不能作为 key 列使用，支持在 Aggregate 模型、Duplicate 模型和 Unique 模型的表中使用。在 Aggregate 模型表中使用时，建表时配合的聚合类型为 QUANTILE_UNION。用户不需要指定长度和默认值。长度根据数据的聚合程度系统内控制。并且 QUANTILE_STATE 列只能通过配套的 QUANTILE_PERCENT、QUANTILE_UNION、TO_QUANTILE_STATE 等函数进行查询或使用。

QUANTILE_STATE 是一种计算分位数近似值的类型，在导入时会对相同的 key，不同 value 进行预聚合，当 value 数量不超过 2048 时采用明细记录所有数据，当 value 数量大于 2048 时采用 TDigest 算法，对数据进行聚合（聚类）保存聚类后的质心点。

相关函数：

QUANTILE_UNION(QUANTILE_STATE): 此函数为聚合函数，用于将不同的分位数计算中间结果进行聚合操作。此函数返回的结果仍是 QUANTILE_STATE

TO_QUANTILE_STATE(DOUBLE raw_data [,FLOAT compression]): 此函数将数值类型转化成 QUANTILE_STATE 类型 compression 参数是可选项，可设置范围是 [2048, 10000]，值越大，后续分位数近似计算的精度越高，内存消耗越大，计算耗时越长。compression 参数未指定或设置的值在 [2048, 10000] 范围外，以 2048 的默认值运行

QUANTILE_PERCENT(QUANTILE_STATE, percent): 此函数将分位数计算的中间结果变量（QUANTILE_STATE）转化为具体的分位数数值

举例

```
select QUANTILE_PERCENT(QUANTILE_UNION(v1), 0.5) from test_table group by k1, k2, k3;
```

注意事项

使用前可以通过如下命令打开 QUANTILE_STATE 开关：

```
$ mysql-client > admin set frontend config("enable_quantile_state_type"="true");
```

这种方式下 QUANTILE_STATE 开关会在 Fe 进程重启后重置，或者在 fe.conf 中添加enable_quantile_state_type=
↪ true配置项可永久生效。

keywords

```
QUANTILE_STATE, QUANTILE_UNION, TO_QUANTILE_STATE, QUANTILE_PERCENT
```

7.1.1.6.4 AGG_STATE

AGG_STATE

描述

```
AGG_STATE不能作为key列使用，建表时需要同时声明聚合函数的签名。
用户不需要指定长度和默认值。实际存储的数据大小与函数实现有关。
```

AGG_STATE 只能配合state /merge/union 函数组合器使用。

需要注意的是，聚合函数的签名也是类型的一部分，不同签名的 agg_state 无法混合使用。比如如果建表声明的签名为max_by(int,int), 那就无法插入max_by(bigint,int)或者group_concat(varchar)。此处 nullable 属性也是签名的一部分，如果能确定不会输入 null 值，可以将参数声明为 not null，这样可以获得更小的存储大小和减少序列化/反序列化开销。

注意：因为agg_state存储的是聚合函数的中间结果，所以读写过程都强依赖于聚合函数的具体实现，如果在 Doris 版本升级时对聚合函数实现做了修改，则可能会造成不兼容的情况。如果出现不兼容的情况，使用到对应agg_state的物化视图需要drop并重新创建，另外涉及到的基础聚合表则会直接不可用，所以需要慎重使用agg_state。

举例

建表示例如下：

```
create table a_table(
    k1 int null,
    k2 agg_state<max_by(int not null,int)> generic,
    k3 agg_state<group_concat(string)> generic
)
aggregate key (k1)
distributed BY hash(k1) buckets 3
properties("replication_num" = "1");
```

这里的 k2 和 k3 分别以 max_by 和 group_concat 为聚合类型。

插入数据示例：

```
insert into a_table values(1,max_by_state(3,1),group_concat_state('a'));
insert into a_table values(1,max_by_state(2,2),group_concat_state('bb'));
insert into a_table values(2,max_by_state(1,3),group_concat_state('ccc'));
```

对于 agg_state 列，插入语句必须用 state 函数来生成对应的 agg_state 数据，这里的函数和入参类型都必须跟 agg_state 完全对应。

查询数据示例：

```
mysql [test]>select k1,max_by_merge(k2),group_concat_merge(k3) from a_table group by k1 order
    ↪ by k1;
+------+--------------------+--------------------------+
| k1   | max_by_merge(`k2`) | group_concat_merge(`k3`) |
+------+--------------------+--------------------------+
|    1 |                  2 | bb,a                     |
|    2 |                  1 | ccc                      |
+------+--------------------+--------------------------+
```

如果需要获取实际结果，则要用对应的 merge 函数。

```
mysql [test]>select max_by_merge(u2),group_concat_merge(u3) from (
  select k1,max_by_union(k2) as u2,group_concat_union(k3) u3 from a_table group by k1 order by
      ↪ k1
  ) t;
+--------------------+--------------------------+
| max_by_merge(`u2`) | group_concat_merge(`u3`) |
+--------------------+--------------------------+
|                  1 | ccc,bb,a                 |
+--------------------+--------------------------+
```

如果想要在过程中只聚合 agg_state 而不获取实际结果，可以使用 union 函数。

更多的例子参见datatype_p0/agg_state

keywords

```
AGG_STATE
```

7.1.1.7 IP 类型

7.1.1.7.1 IPV4

IPV4

描述

IPv4 类型，以 UInt32 的形式存储在 4 个字节中，用于表示 IPv4 地址。取值范围是 [‘0.0.0.0’, ‘255.255.255.255’]。

超出取值范围或者格式非法的输入将返回NULL

举例

建表示例如下：

```
CREATE TABLE ipv4_test (
  `id` int,
  `ip_v4` ipv4
) ENGINE=OLAP
DISTRIBUTED BY HASH(`id`) BUCKETS 4
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

插入数据示例：

```
insert into ipv4_test values(1, '0.0.0.0');
insert into ipv4_test values(2, '127.0.0.1');
insert into ipv4_test values(3, '59.50.185.152');
insert into ipv4_test values(4, '255.255.255.255');
insert into ipv4_test values(5, '255.255.255.256'); // invalid data
```

查询数据示例：

```
mysql> select * from ipv4_test order by id;
+------+-----------------+
| id   | ip_v4           |
+------+-----------------+
|    1 | 0.0.0.0         |
|    2 | 127.0.0.1       |
|    3 | 59.50.185.152   |
|    4 | 255.255.255.255 |
|    5 | NULL            |
+------+-----------------+
```

keywords

IPV4

7.1.1.7.2 IPV6

IPV6

描述

IPv6 类型，以 UInt128 的形式存储在 16 个字节中，用于表示 IPv6 地址。取值范围是 ['::' , 'ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff']。

超出取值范围或者格式非法的输入将返回NULL

举例

建表示例如下：

```
CREATE TABLE ipv6_test (
  `id` int,
```

```
  `ip_v6` ipv6
) ENGINE=OLAP
DISTRIBUTED BY HASH(`id`) BUCKETS 4
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

插入数据示例：

```
insert into ipv6_test values(1, '::');
insert into ipv6_test values(2, '2001:16a0:2:200a::2');
insert into ipv6_test values(3, 'ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff');
insert into ipv6_test values(4, 'ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffffg'); // invalid data
```

查询数据示例：

```
mysql> select * from ipv6_test order by id;
+------+-----------------------------------------+
| id   | ip_v6                                   |
+------+-----------------------------------------+
|    1 | ::                                      |
|    2 | 2001:16a0:2:200a::2                     |
|    3 | ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff |
|    4 | NULL                                    |
+------+-----------------------------------------+
```

keywords

IPV6

7.1.2 字面量

7.1.2.1 数值类型字面量

使用数字字面量表示法来指定定点数和浮点数。

7.1.2.1.1 整数字面量

整数表示为一系列数字。可能有符号。例如：+1，-2，345。

Doris 依据输入数值，来决定使用何种类型来存储整数字面量。范围映射关系，参见下表：

数值范围	类型
-2^8 ~ 2^8 - 1	TINYINT
-2^16 ~ 2^16 - 1	SMALLINT
-2^32 ~ 2^32 - 1	INT
-2^64 ~ 2^64 - 1	BIGINT
-2^128 ~ 2^128 - 1	LARGEINT

7.1.2.1.2 定点和浮点数字面量

定点和浮点数字面量有整数部分或小数部分，或两者兼有。它们可能有符号。例如：1、.2、3.4、-5、-6.78、+9.10。

也可以用科学记数法表示，包括尾数和指数。其中一部分或两部分都可能有符号。例如：1.2E3、1.2E-3、-1.2E3、-1.2E-3。

此种表示的数字，优先被解析为定点数。定点数的支持范围受变量enable_decimal256 控制。当enable_decimal256为TRUE时，其最大精度为 76。当enable_decimal256为FALSE时，其最大精度为 38。

当数字需要的精度超过定点数可以表示的最大值时，其被解析为浮点数，类型为 DOUBLE。

7.1.2.2 字符串类型字面量

7.1.2.2.1 描述

字符串是一系列字节或字符，用单引号（'）或双引号（"）字符括起来。例如：

```
'a string'
"another string"
```

7.1.2.2.2 转义字符

在字符串中，除非启用了 NO_BACKSLASH_ESCAPES SQL 模式，否则某些序列具有特殊含义。这些序列均以反斜杠 () 开头，反斜杠称为转义字符。Doris 可以识别的转义字符列到下表中

转义字符	意义
\0	ASCII 字符 NUL（X'00'）
\'	单引号（'）
\"	双引号（"）
\b	退格符
\n	换行符
\r	回车符
\t	制表符
\Z	ASCII 26 (Control+Z)
\\	反斜杠（\）
\%	百分号 %。详细信息参考表格后的注意事项
_	下划线 _。详细信息参考表格后的注意事项

注意事项

1. 在模式匹配的上下文中，通常会将 % 和 “” 解释为通配符字符，但使用\%和 _ 序列可以搜索 “%” 和 “” 的字面量实例。有关详细信息，请参阅 “模式匹配操作符” 章节中对 LIKE 操作符的描述。如果在模式匹配的上下文之外使用 \% 或 _，它们会被计算为字符串 \% 和 _，而不是 % 和 _。

2. 表格以外的转义字符中的反斜杠会被直接忽略。例如'\y' 和 'y'是等价的。

7.1.2.2.3 在字符串字面量中使用引号

在字符串中包含引号字符有几种方法：

- 在以单引号（ ‘ ）括起来的字符串中，单引号可以写作两个单引号（ ’’ ）。
- 在以双引号（ “ ）括起来的字符串中，双引号可以写作两个双引号（ ”“ ）。
- 在引号字符前加上转义字符（ \ ）。
- 在以双引号括起来的字符串中包含单引号时，无需进行特殊处理，也不必将单引号加倍或转义。同样地，在以单引号括起来的字符串中包含双引号时，也无需进行特殊处理。

7.1.2.3 日期类型字面量

7.1.2.3.1 描述

和标准 SQL 一致，Doris 要求使用类型关键字和字符串来指定时间字面量。关键字和字符串之间的空格是可选的。例如：

```
DATE '2008-08-08'
TIMESTAMP'2008-08-08 20:08:08'
```

7.1.2.3.2 日期格式

DATE 字面

- 使用 - 分隔的字符串，采用 'YYYY-MM-DD' 或 'YY-MM-DD'格式。Doris 也兼容 MySQL 的非标准分隔符格式，但是不推荐使用。
- 作为没有分隔符的字符串，采用 'YYYYMMDD' 或 'YYMMDD' 格式（前提是该字符串在日期上有意义）。

DATEIME 字面量

- 使用 - 分隔的字符串，采用 'YYYY-MM-DD hh:mm:ss' 或 'YY-MM-DD hh:mm:ss'格式。Doris 也兼容 MySQL 的非标准分隔符格式，但是不推荐使用。日期和时间之间的分隔符可以是空格（ ` ）也可以是T ‘。不同于 MySQL 8.4 及更早的版本，Doris 不支持其他任何其他的时间和日期之间的分隔符。
- 作为没有分隔符的字符串，采用 'YYYYMMDDhhmmss' 或 'YYMMDDhhmmss' 格式（前提是该字符串在日期上有意义）。

DATETIME 字面量可以包含一个最多达到微秒（6 位数字）精度的小数秒部分。小数部分应该始终用点号（ . ）与时间的其余部分分隔开；不识别其他的小数秒分隔符。

两位数年份

包含两位数年份值的日期是有歧义的，因为世纪是未知的。Doris 使用以下规则来解释两位数的年份值：

- 范围在 70-99 之间的年份值会被解释为 1970-1999 年。
- 范围在 00-69 之间的年份值会被解释为 2000-2069 年。

时区

DATE 和 DATETIME 字面量可以使用时区后缀。使用时区时，时区需要紧邻之前的日期或时间部分，之间不得有空格。例如：

```
TIMESTAMP '2008-08-08 20:08:08+08:00'
```

Doris 支持的时区格式有：

- 时区时间偏移量：格式为 {+ | -}hh:mm。如东八区为：+08:00
- 时区名。如上海时区为：Asia/Shanghai

错误值的处理

当遇到不能解析为合法日期字面量的值时，Doris 会直接报错。例如

```
SELECT date '071332'
```

会产生如下错误：

```
date/datetime literal [071332] is invalid
```

7.1.3 NULL

7.1.3.1 NULL 基础介绍

如果一行中的某一列没有值，那么就说该列为 NULL。NULL 可以出现在任何没有被“非空”（NOT NULL）限制的数据类型的列中。当实际值未知或者某个值没有意义时，使用 NULL。

不要使用 NULL 表示数值 0 或者空字符串。它们之间并不相等。

任何包含 NULL 的算术表达式的结果总是 NULL。例如，NULL 加上 10 还是 NULL。事实上，当给定一个 NULL 作为操作数时，所有运算符都返回 NULL。

7.1.3.2 NULL 作为函数参数

当给定一个 NULL 作为参数时，大多数标量函数返回 NULL。您可以使用 IFNULL 函数在出现空值时返回一个值。例如，表达式 IFNULL(arg,0) 在 arg 为 NULL 时返回 0，在 arg 不为 NULL 时返回其值。每个函数的具体行为，请参阅“函数”章节

7.1.3.3 NULL 和比较运算符

要测试结果是否为 NULL，只能使用比较条件 IS NULL 和 IS NOT NULL。如果使用一个结果取决于 NULL 的条件，则结果为 UNKNOWN。由于 NULL 表示缺少数据，因此 NULL 不能等于或不等于任何值或其他 NULL。

7.1.3.4 NULL 在条件中

计算结果为 UNKNOWN 的条件其作用几乎与 FALSE 相同。例如，在 WHERE 子句中包含计算结果为 UNKNOWN 的条件的 SELECT 语句将不返回任何行。但是，计算结果为 UNKNOWN 的条件与 FALSE 的不同之处在于，对 UNKNOWN 条件评估结果进行进一步操作的结果也将评估为 UNKNOWN。因此，NOT FALSE 的计算结果为 TRUE，但 NOT UNKNOWN 的计算结果为 UNKNOWN。

下表显示了条件中涉及 NULL 的各种评估的示例。如果在 SELECT 语句的 WHERE 子句中使用计算结果为 UNKNOWN 的条件，那么该查询将不返回任何行。

Condition	Value of A	Evaluation
`a IS NULL`	10	FALSE
`a IS NOT NULL`	10	TRUE
`a IS NULL`	NULL	TRUE
`a IS NOT NULL`	NULL	FALSE
`a = NULL`	10	UNKNOWN
`a != NULL`	10	UNKNOWN
`a = NULL`	NULL	UNKNOWN
`a != NULL`	NULL	UNKNOWN
`a = 10`	NULL	UNKNOWN
`a != 10`	NULL	UNKNOWN

7.1.4 对象标识符

7.1.4.1 描述

每个数据库对象，例如表、列、索引，都有一个名称。在 SQL 语句中，这些名称被称作对象标识符。标识符可以加引号，也可以不加引号。如果标识符包含特殊字符或是保留关键字，则在每次引用时都必须使用反引号（ ‘)。关于保留关键字的详细信息，请参阅保留关键字章节。

7.1.4.2 对象对标识符的限制

在 Doris 中，对象标识符可以通过变量 `enable_unicode_name_support` 控制，决定是否支持 unicode 字符。当开启 unicode 字符支持时，标识符中，可以使用 unicode 中任意语言的文字字符。但是不能使用标点，符号等其他字符。

在 Doris 中，不同的对象对于标识符有不同的限制，下面列举了不同对象的具体限制。

7.1.4.2.1 表名

模式	标识符限制
关闭 unicode 模式	`^[a-zA-Z][a-zA-Z0-9-_]*$`
开启 unicode 模式	`^[a-zA-Z\\p{L}][a-zA-Z0-9-_\\p{L}]*$`

7.1.4.2.2 列名

模式	标识符限制
关闭 unicode 模式	`^[.a-zA-Z0-9_+-/?@#$%^&*\"\\s,:]{1,256}$`
开启 unicode 模式	`^[.a-zA-Z0-9_+-/?@#$%^&*\"\\s,:\\p{L}]{1,256}$`

7.1.4.2.3 OUTFILE 文件名

模式	标识符限制
关闭 unicode 模式	`^[_a-zA-Z][a-zA-Z0-9-_]{0,63}$`
开启 unicode 模式	`^[_a-zA-Z\\p{L}][a-zA-Z0-9-_\\p{L}]{0,63}$`

7.1.4.2.4 用户名

模式	标识符限制
关闭 unicode 模式	`^[a-zA-Z][a-zA-Z0-9.-_]*$`
开启 unicode 模式	`^[a-zA-Z\\p{L}][a-zA-Z0-9.-_\\p{L}]*$`

7.1.4.2.5 LABEL 名

模式	标识符限制
关闭 unicode 模式	`^[-_A-Za-z0-9:]{1,N}$`，其中 N 的值取决于 FE 配置中的 `label_regex_length` 参数，默认值为 128。
开启 unicode 模式	`^[-_A-Za-z0-9:\\p{L}]{1,N}$`，其中 N 的值取决于 FE 配置中的 label_regex_length 参数，默认值为 128。

7.1.4.2.6 其他

模式	标识符限制
关闭 unicode 模式	`^[a-zA-Z][a-zA-Z0-9-_]{0,63}$`
开启 unicode 模式	`^[a-zA-Z\\p{L}][a-zA-Z0-9-_\\p{L}]{0,63}$`

7.1.5 保留关键字

7.1.5.1 描述

关键字是指在 SQL 中有特殊意义的词语。保留关键字是一类特殊的关键字，如果想使用保留关键字作为对象标识符，则必须使用反引号（ ‘）。列如FROM。有关对象标识符的详细信息，请参阅对象标识符章节。

7.1.5.2 保留关键字列表

- ACCOUNT_LOCK
- ACCOUNT_UNLOCK
- ADD
- ADMIN
- ALL
- ALTER
- ANALYZE
- AND
- ANTI
- APPEND
- AS
- ASC
- AUTO
- BACKEND
- BETWEEN
- BIGINT
- BINARY
- BINLOG
- BY
- CANCEL
- CASE
- CAST
- CHAR
- CHARACTER
- CLEAN
- COLLATE
- COLUMN
- CONSTRAINT
- CONVERT_LIGHT_SCHEMA_CHANGE_PROCESS
- CREATE
- CROSS
- CUBE
- CURRENT
- DATABASE
- DATABASES
- DECOMMISSION
- DEFAULT
- DELETE
- DESC
- DESCRIBE
- DISK
- DISTINCT
- DISTRIBUTED

- DISTRIBUTION
- DIV
- DOUBLE
- DROP
- DROPP
- DUMP
- DUPLICATE
- ELSE
- ENTER
- EXCEPT
- EXECUTE
- EXISTS
- EXPLAIN
- EXPORT
- EXTENDED
- EXTRACT
- FALSE
- FLOAT
- FOLLOWER
- FOLLOWING
- FOR
- FORCE
- FOREIGN
- FROM
- FRONTEND
- FULL
- FUNCTIONS
- GRANT
- GRANTS
- GROUP
- HAVING
- HLL
- IF
- IN
- INDEX
- INFILE
- INNER
- INSERT
- INSTALL
- INT
- INTEGER
- INTERMEDIATE
- INTERSECT
- INTERVAL
- INTO

- IS
- JOIN
- KEY
- KEYS
- KILL
- LARGEINT
- LATERAL
- LEFT
- LIKE
- LIMIT
- LIST
- LOAD
- LOW_PRIORITY
- MATCH
- MAXVALUE
- MINUS
- NATURAL
- NOT
- NULL
- OBSERVER
- ON
- OR
- ORDER
- OUTER
- OUTFILE
- OVER
- OVERWRITE
- PARTITION
- PLAY
- PRECEDING
- PREPARE
- PRIMARY
- PROCEDURE
- RANGE
- READ
- REAL
- REBALANCE
- REFERENCES
- REGEXP
- RELEASE
- RENAME
- REPAIR
- REPLICA
- REVOKE
- RIGHT

- ROLE
- ROLES
- ROW
- ROWS
- SCHEMAS
- SELECT
- SEMI
- SET
- SETS
- SHOW
- SIGNED
- SMALLINT
- SQL_BLOCK_RULE
- SUPERUSER
- SWITCH
- SYNC
- SYSTEM
- TABLE
- TABLESAMPLE
- TABLET
- TABLETS
- TERMINATED
- THEN
- TINYINT
- TO
- TRASH
- TRIM
- TRUE
- TYPE_CAST
- UNBOUNDED
- UNINSTALL
- UNION
- UNIQUE
- UNSIGNED
- UPDATE
- USE
- USING
- VALUES
- WHEN
- WHERE
- WHITELIST
- WITH
- WORKLOAD
- WRITE
- XOR

7.1.6 变量

7.1.6.1 描述

在 Doris 中，变量分为系统变量和用户变量。这两者都是大小写不敏感的。

系统变量会影响 Doris 的行为。系统变量和用户变量均可用于用户查询中。

7.1.6.2 系统变量

系统变量是由 Doris 预定义的一组变量，用于控制数据库的行为和性能。主要特点如下：

- 变量类型：
  只读变量：这些变量的值由系统设定，用户无法修改，如 `version`、`current_timestamp` 等
  可修改变量：用户可以在运行时修改这些变量的值，如 `exec_mem_limit`、`time_zone` 等
- 作用域：
  全局变量（Global）：影响所有会话，通过 `SET GLOBAL` 设置
  会话变量（Session）：仅影响当前会话，通过 `SET` 设置
  部分变量同时具有全局和会话两种作用域
- 访问方式：
  使用 `SHOW VARIABLES` 查看所有系统变量
  使用 `SHOW VARIABLES LIKE 'pattern'` 按模式匹配查看特定变量
- 持久性：
  全局变量的修改在系统重启后会恢复默认值，会话重启不会恢复默认值
  会话变量的修改在会话结束后失效

7.1.6.3 用户自定义变量

用户自定义变量是一种在会话中临时存储数据的机制。主要特点如下：

- 命名规则：
  必须以 `@` 符号作为前缀
  变量名可以包含字母、数字和下划线
  大小写不敏感
- 作用域：
  仅在当前会话中有效
  会话结束后变量自动销毁
  不同会话的同名变量相互独立
- 赋值方式：
  使用 `SET @var_name = value` 语法赋值
  支持使用表达式计算结果赋值

- 数据类型：

  可以存储数字（整数、浮点数）

  可以存储字符串

  可以存储日期时间值

  可以存储 NULL 值

  类型在赋值时自动确定

7.1.6.4 系统变量定义及查询语句

- SHOW VARIABLES

  可以通过 `SHOW VARIABLES LIKE 'variable_name'` 来查看变量

```
SHOW VARIABLES LIKE '%time_zone%';
+------------------+----------------+----------------+---------+
| Variable_name    | Value          | Default_Value  | Changed |
+------------------+----------------+----------------+---------+
| system_time_zone | Asia/Hong_Kong | Asia/Hong_Kong | 0       |
| time_zone        | Asia/Hong_Kong | Asia/Hong_Kong | 0       |
+------------------+----------------+----------------+---------+
```

或者可以通过使用`SHOW VARIABLES`来查看所有的变量

```
SHOW VARIABLES
+--
    ↪ ----------------------------------------------------------------+-----------------------------------
    ↪
| Variable_name                                                  | Value
    ↪                                | Default_Value                          | Changed |
+--
    ↪ ----------------------------------------------------------------+-----------------------------------
    ↪
| DML_PLAN_RETRY_TIMES                                           | 3
    ↪                                     | 3                                      | 0
    ↪       |
| adaptive_pipeline_task_serial_read_on_limit                    | 10000
    ↪                                | 10000                                  | 0       |
| allow_modify_materialized_view_data                            | false
    ↪                                | false                                  | 0       |
| allow_partition_column_nullable                                | true
    ↪                                 | true                                   | 0       |
| analyze_timeout                                                | 43200
    ↪                                | 43200                                  | 0       |
. . .
```

```
| version                                                
```

```
+------------------+----------------+----------------+---------+
2 rows in set (0.00 sec)
```

全局生效

```
SET GLOBAL exec_mem_limit = 137438953472;
show variables like '%exec_mem_limit%';
+----------------+--------------+---------------+---------+
| Variable_name  | Value        | Default_Value | Changed |
+----------------+--------------+---------------+---------+
| exec_mem_limit | 137438953472 | 2147483648    | 1       |
+----------------+--------------+---------------+---------+
1 row in set (0.01 sec)
```

- UNSET

  语法：

```
UNSET (GLOBAL | SESSION | LOCAL)? VARIABLE (ALL | identifier)
unset global variable exec_mem_limit;
show variables like '%exec_mem_limit%';
+----------------+------------+---------------+---------+
| Variable_name  | Value      | Default_Value | Changed |
+----------------+------------+---------------+---------+
| exec_mem_limit | 2147483648 | 2147483648    | 0       |
+----------------+------------+---------------+---------+
1 row in set (0.00 sec)
```

7.1.6.5 用户变量定义及查询语句

用户定义变量可以用如下语句来定义

```
SET @var_name = constant_value|constant_expr;
```

设置例子

```
set @v1 = "A";
set @v2 = 32+33;
set @v3 = str_to_date("2024-12-29 10:11:12", '%Y-%m-%d %H:%i:%s');
```

可以用于查询中

```
select @v1, @v2, @v3;
+------+------+---------------------+
| @v1  | @v2  | @v3                 |
+------+------+---------------------+
```

```
| A    |   65 | 2024-12-29 10:11:12 |
+------+------+---------------------+
1 row in set (0.01 sec)
```

7.1.7 注释

7.1.7.1 描述

注释可以使您的应用程序更易于阅读和维护。例如，可以在语句中包含一个注释，描述该语句在应用程序中的用途。SQL 语句中的注释（HINT 除外）不会影响语句的执行。有关使用这种特定形式的注释的 HINT，请参阅“HINT”章节。

在 SQL 语句中，注释可以出现在任何关键字、参数或标点符号之间。您可以通过两种方式在语句中包含注释：

- 多行注释：以斜杠和星号（/）开始注释，接着是注释的文本。这个文本可以跨越多行。以星号和斜杠（/）结束注释。开始和结束字符与文本之间不需要用空格或换行符分隔。
- 单行注释：以两个连字符（-）开始注释，接着是注释的文本。这个文本不能延伸到新行。注释以换行符结束。

7.1.7.2 示例

7.1.7.2.1 多行注释

```
SELECT /* This is a multi-line comment
          that can span multiple lines */
       column_name
FROM   table_name;
```

7.1.7.2.2 单行注释

```
SELECT column_name -- This is a single-line comment
FROM   table_name;
```

7.1.8 操作符

7.1.8.1 条件操作符

7.1.8.1.1 逻辑操作符

描述

逻辑条件将两个组成部分的条件的结果进行组合，基于它们生成一个单一的结果，或者对一个条件的结果进行取反。

操作符介绍

操作符	作用	示例
NOT	如果以下条件为 FALSE，则返回 TRUE。如果为 TRUE，则返回 FALSE。如果为 UNKNOWN，则保持 UNKNOWN。	SELECT NOT (TRUE)
AND	如果两个组成部分的条件都为 TRUE，则返回 TRUE。如果其中任何一个为 FALSE，则返回 FALSE。否则返回 UNKNOWN。	SELECT TRUE AND FALSE
OR	如果任一组成部分的条件为 TRUE，则返回 TRUE。如果两者都为 FALSE，则返回 FALSE。否则返回 UNKNOWN。	SELECT TRUE OR NULL

真值表

NOT 真值表

	TRUE	FALSE	UNKNOWN
NOT	FALSE	TRUE	UNKNOWN

AND 真值表

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR 真值表

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

7.1.8.1.2 比较操作符

描述

比较条件将一个表达式与另一个表达式进行比较。比较的结果可以是 TRUE、 FALSE 或 UNKNOWN。

操作符介绍

操作符	作用	示例
=	等值比较。当比较的两侧任意一个值为 UNKNOWN 时，结果为 UNKNWON。	SELECT 1 = 1
<=>	NULL 安全的等值比较。不同于等值比较。NULL 安全的等值比较将 NULL 视作一个可比较的值。当比较两侧均为 NULL 时，返回 TRUE。当只有一侧为 NULL 时，返回 FALSE。此操作符永远不会返回 UNKNOWN。	SELECT NULL <=> NULL

操作符	作用	示例
!= `` <>	不等比较	SELECT 1 != 1
< `` >	大于比较和小与比较	SELECT 1 > 1
<= `` >=	大于等于比较和小于等于比较	SELECT 1 >= 1
<x> BETWEEN <y> AND ↪ <z>	等价于 <x> >= <y> and <x> <= <z>。大于等于 <y> 且小于等于 <z>	SELECT 1 BETWEEN 0 ↪ AND 2

7.1.8.1.3 真值检测操作符

描述

此操作符只限于使用检测是否为 TRUE，FALSE 或 NULL。关于 NULL 的介绍，请参阅 Nulls 章节。

操作符介绍

操作符	作用	示例
IS [NOT] TRUE	检测 x 是否为 TRUE。当 x 为 TRUE 时，返回 TRUE [FALSE]。否则返回 FALSE[TRUE]	SELECT 1 IS NOT TRUE
IS [NOT] FALSE	检测 x 是否为 FALSE。当 x 为 FALSE 时，返回 TRUE [FALSE]。否则返回 FALSE[TRUE]	SELECT 1 IS NOT ↪ FALSE
IS [NOT] NULL	检测 x 是否为 NULL。当 x 为 NULL 时，返回 TRUE [FALSE]。否则返回 FALSE[TRUE]	SELECT 1 IS NOT NULL

7.1.8.1.4 模式匹配操作符

描述

模式匹配操作符用于比较字符类型的数据。

操作符介绍

操作符	作用	示例
[NOT] LIKE	如果与模式 [不] 匹配，则为 TRUE。在中，字符% 与任何零个或多个字符的字符串匹配 (空值除外)。字符 _ 与任何单个字符匹配。如果通配符字符前面有转义字符，则将其视为文字字符。	SELECT 'ABCD' LIKE ↪ '%C_'
[NOT] {REGEXP | RLIKE}	如果与模式 [不] 匹配，则为 TRUE。正则表达式的具体规则，请参考本文后续的 REGEXP 章节。	SELECT 'ABCD' REGEXP ↪ 'A.*D'

LIKE

LIKE 条件指定一个涉及模式匹配的测试。等值比较运算符 (=) 将一个字符值精确匹配到另一个字符值，而 LIKE 条件则通过在第一个值中搜索第二个值指定的模式，将一个字符值的一部分与另一个字符值进行匹配。

语法如下：

```
<char1> [ NOT ] LIKE <char2>
```

其中：

- char1 是一个字符表达式（如字符列），称为搜索值。
- char2 是一个字符表达式，通常是一个字面量，称为模式。

所有字符表达式（char1、char2）都可以是 CHAR、VARCHAR 或 STRING 数据类型中的任何一种。如果它们不同，则 Doris 会将它们全部转换为 VARCHAR 或者 STRING。

模式中可以包含特殊的模式匹配字符：

- 模式中的下划线 (_) 与值中的一个字符完全匹配。
- 模式中的百分号 (%) 可以与值中的零个或多个字符匹配。模式 '%' 不能与 NULL 匹配。

REGEXP（RLIKE）

REGEXP 与 LIKE 条件类似，不同之处在于 REGEXP 执行正则表达式匹配，而不是 LIKE 执行的简单模式匹配。此条件使用输入字符集定义的字符来评估字符串。

语法如下：

```
<char1> [ NOT ] { REGEXP | RLIKE } <char2>
```

其中：

- char1 是一个字符表达式（如字符列），称为搜索值。
- char2 是一个字符表达式，通常是一个字面量，称为模式。

所有字符表达式（char1、char2）都可以是 CHAR、VARCHAR 或 STRING 数据类型中的任何一种。如果它们不同，则 Doris 会将它们全部转换为 VARCHAR 或者 STRING。

7.1.8.1.5 全文检索操作符

描述

全文检索操作符判断一个列是否满足指定的全文检索条件（关键词、短语等），结果可以是 TRUE、FALSE 或 UNKNOWN。

限制：

1. 左操作数必须是一个列名，右操作数必须是字符串字面常量。
2. 只能用于 WHERE 子句，不能用于 SELECT、GROUP BY、ORDER BY 等其他子句。可以与 WHERE 子句中的其他条件进行 AND OR NOT 逻辑组合。

操作符介绍

操作符	作用	示例
MATCH	等同 MATCH_ANY	`SELECT * FROM t WHERE column1 ↪ MATCH 'word1 word2'`
MATCH_ANY	对 column_name 和 string_literal 按照 column_name 的索引分词器进行分词，如果 column_name 分出来的词包含 string_literal 分词后的任意一个词，结果为 TRUE，否则为 FALSE。	`SELECT * FROM t WHERE column1 ↪ MATCH_ANY 'word1 word2'`
MATCH_ALL	对 column_name 和 string_literal 按照 column_name 的索引分词器进行分词，如果 column_name 分出来的词包含 string_literal 分词后的每一个词，结果为 TRUE，否则为 FALSE。	`SELECT * FROM t WHERE column1 ↪ MATCH_ALL 'word1 word2'`
MATCH_PHRASE	对 column_name 和 string_literal 按照 column_name 的索引分词器进行分词，如果 column_name 分出来的词包含 string_literal 分词后的每一个词、而且词的顺序一致（即为短语），结果为 TRUE，否则为 FALSE。	`SELECT * FROM t WHERE column1 ↪ MATCH_PHRASE 'word1 word2'`
MATCH_PHRASE_PREFIX	MATCH_PHRASE 的扩展功能，允许 string_literal 分词后的最后一个词只匹配前缀而不是整个词。类似 Web 搜索引擎 suggest 的效果。	`SELECT * FROM t WHERE column1 ↪ MATCH_PHRASE_PREFIX 'word1 ↪ wor'`
MATCH_PHRASE_EDGE	MATCH_PHRASE_PREFIX 的扩展功能，允许 string_literal 分词后第一个词匹配后缀，最后一个词匹配前缀	`SELECT * FROM t WHERE column1 ↪ MATCH_PHRASE_EDGE 'rd wor'`
MATCH_REGEXP	MATCH_ANY 的扩展功能，string_literal 指定一个正则表达式，column_name 分出来的词与正则表达式匹配。	`SELECT * FROM t WHERE column1 ↪ MATCH_REGEXP 'word.'`

7.1.8.1.6 IN 操作符

描述

IN 操作符测试一个值是否属于某个值列表或子查询中的成员。

操作符介绍

操作符	作用	示例
IN	测试是否等于任意的成员。如果是，则返回 TRUE。	`SELECT 1 IN (SELECT 2)`
NOT IN	测试是否不等于所有的成员。如果是，则返回 TRUE。	`SELECT 1 NOT IN (3, 4, 5)`

7.1.8.1.7 EXISTS 操作符

描述

EXISTS 条件用于测试子查询中行的存在性。

操作符介绍

操作符	作用	示例
EXISTS	如果子查询返回至少一条数据，则返回 TRUE	`SELECT department_id FROM departments d WHERE ↪ EXISTS (SELECT * FROM employees e WHERE d. ↪ department_id = e.department_id)ORDER BY ↪ department_id;`

7.1.8.2 位操作符

7.1.8.2.1 描述

位操作符对一个表达式或者两个表达式按照位进行制定的操作。位操作符只能接收 BIGINT 类型作为参数。所以，位操作处理的表达式都会被转换为 BIGINT 类型。

7.1.8.2.2 操作符介绍

操作符	作用	示例
&	按位执行与操作。当两个表达式对应的位均为 1 时，结果对应的位置为 1，否则为 0。	`SELECT 1 & 2`
|	按位执行或操作。当两个表达式对应的位任意一个为 1 时，结果对应的位置为 1，否则为 0。	`SELECT 1 | 2`
^	按位执行异或操作。当两个表达式对应不同时，结果对应的位置为 1，否则为 0。	`SELECT 1 ^ 2`
~	按位执行取反操作。当表达式的位为 1 时，结果对应的位置为 0，否则为 1。	`SELECT ~1`

7.1.8.3 赋值操作符

7.1.8.3.1 描述

赋值操作符的作用是，将操作符右侧的表达式，赋给左侧的表达式。在 Doris 中，赋值操作符只能在 UPDATE 语句的 SET 部分，以及 SET 语句中使用。详细请参考 UPDATE 语句和 SET 语句。

7.1.8.3.2 操作符介绍

操作符	作用	示例
=	将的结果赋值给。	`SET enable_profile = true`

7.1.8.4 操作符优先级

7.1.8.4.1 描述

操作符优先级决定了在表达式中各个操作符的计算顺序。当表达式中包含多个操作符时，Doris 会根据操作符的优先级从高到低依次进行计算。

7.1.8.4.2 操作符优先级

优先级从上到下依次递减，最上面具有最高的优先级。

优先级	操作符
1	!
2	+(一元正号), - (一元负号), ~ (一元位反转) ^
3	*, /, %, DIV
4	-, +
5	&
6	|
7	=(比较), <=>, >=, >, <=, <, <>, !=, IS, LIKE, REGEXP, MATCH, IN
8	NOT
9	AND, &&
10	XOR
11	OR
12	||

7.2 SQL 函数

7.2.1 标量函数

7.2.1.1 数值函数

7.2.1.1.1 ABS

描述

返回一个值的绝对值。

语法

```
ABS(<x>)
```

参数

参数	说明
<x>	需要被计算绝对值的值

返回值

参数 x 的绝对值

举例

```
select abs(-2);
```

```
+---------+
| abs(-2) |
+---------+
|       2 |
+---------+
```

```
select abs(3.254655654);
```

```
+------------------+
| abs(3.254655654) |
+------------------+
|      3.254655654 |
+------------------+
```

```
select abs(-3254654236547654354654767);
```

```
+---------------------------------+
| abs(-3254654236547654354654767) |
+---------------------------------+
| 3254654236547654354654767       |
+---------------------------------+
```

7.2.1.1.2 ACOS

描述

返回x的反余弦值，若 x不在-1到 1的范围之内，则返回 nan.

语法

```
ACOS(<x>)
```

参数

参数	说明
<x>	需要被计算反余弦的值

返回值

参数 x 的反余弦值

举例

```
select acos(1);
```

```
+-----------+
| acos(1.0) |
+-----------+
|         0 |
+-----------+
```

```
select acos(0);
```

```
+--------------------+
| acos(0.0)          |
+--------------------+
| 1.5707963267948966 |
+--------------------+
```

```
select acos(-2);
```

```
+------------+
| acos(-2.0) |
+------------+
|        nan |
+------------+
```

7.2.1.1.3 ASIN

描述

返回x的反正弦值，若 x不在-1到 1的范围之内，则返回 nan.

语法

```
ASIN(<x>)
```

参数

参数	说明
<x>	需要被计算反正弦的值

返回值

参数 x 的反正弦值

举例

```
select asin(0.5);
```

```
+--------------------+
```

```
| asin(0.5)           |
+---------------------+
| 0.52359877559829893 |
+---------------------+
```

```
select asin(2);
```

```
+-----------+
| asin(2.0) |
+-----------+
|       nan |
+-----------+
```

7.2.1.1.4 ATAN

描述

返回x的反正切值，x为弧度值。

语法

```
ATAN(<x>)
```

参数

参数	说明
<x>	需要被计算反正切的值

返回值

参数 x 的反正切值

举例

```
select atan(0);
```

```
+-----------+
| atan(0.0) |
+-----------+
|         0 |
+-----------+
```

```
select atan(2);
```

```
+--------------------+
| atan(2.0)          |
+--------------------+
```

```
| 1.1071487177940904 |
+--------------------+
```

7.2.1.1.5 ATAN2

描述

返回 ‘y’ / ‘x’ 的反正切。

语法

```
ATAN2(<y>, <x>)
```

参数

参数	说明
<x>	参与计算的反正切的值，表示水平坐标（或 x 值），从原点 (0,0) 沿 x 轴的距离。
<y>	参与计算的反正切的值，表示垂直坐标（或 y 值），从原点 (0,0) 沿 y 轴的距离。

返回值

参数 y / x 的反正切值

举例

```
select atan2(0.1, 0.2);
```

```
+---------------------+
| atan2(0.1, 0.2)     |
+---------------------+
| 0.46364760900080609 |
+---------------------+
```

```
select atan2(1.0, 1.0);
```

```
+---------------------+
| atan2(1.0, 1.0)     |
+---------------------+
| 0.78539816339744828 |
+---------------------+
```

7.2.1.1.6 BIN

描述

将十进制数字转换为二进制文本。

语法

```
bin(<a>)
```

参数

参数	说明
<a>	需要转换的十进制值

返回值

参数 <a> 的二进制表示。当 <a> 为负数时，结果为其 64 位补码表示。

举例

```
select bin(0);
```

```
+--------+
| bin(0) |
+--------+
| 0      |
+--------+
```

```
select bin(-1);
```

```
+------------------------------------------------------------------+
| bin(-1)                                                          |
+------------------------------------------------------------------+
| 1111111111111111111111111111111111111111111111111111111111111111 |
+------------------------------------------------------------------+
```

```
select bin(123);
```

```
+----------+
| bin(123) |
+----------+
| 1111011  |
+----------+
```

7.2.1.1.7 CBRT

描述

计算参数的立方根

语法

```
cbrt(<a>)
```

参数

参数	说明
<a>	浮点参数

返回值

参数 <a> 的立方根，浮点数。

举例

```
select cbrt(0);
```

```
+-------------------------+
| cbrt(cast(0 as DOUBLE)) |
+-------------------------+
|                     0.0 |
+-------------------------+
```

```
select cbrt(-111);
```

```
+----------------------------+
| cbrt(cast(-111 as DOUBLE)) |
+----------------------------+
|         -4.805895533705333 |
+----------------------------+
```

```
select cbrt(1234);
```

```
+----------------------------+
| cbrt(cast(1234 as DOUBLE)) |
+----------------------------+
|         10.726014668827325 |
+----------------------------+
```

7.2.1.1.8 CEIL

描述

对浮点及定点小数按特定位数向上取整，返回取整过后的浮点或定点数。

语法

```
CEIL(<a>[, <d>])
```

参数

参数	说明
<a>	浮点 (Double) 或定点 (Decimal) 参数，表示要进行取整的参数
<d>	可选的，整数，表示舍入目标位数，正数为向小数点后舍入，负数为向小数点前舍入，0 表示舍入到整数。不填写时等

返回值

按照下面规则返回最小的大于或者等于 <a> 的舍入数字：

舍入到 1/(10^d) 位，即，使结果可整除1/(10^d)。如果 1/(10^d) 不精确，则舍入位数为相应数据类型的最接近的数字。

对于 Decimal 类型的入参 <a>，假设其类型为 Decimal(p, s)，则返回值类型为：

- Decimal(p, 0)，若 <d> <= 0
- Decimal(p, <d>)，若 0 < <d> <= s
- Decimal(p, s)，若 <d> > s

别名

- DCEIL
- CEILING

举例

```
select ceil(123.456);
```

```
+---------------+
| ceil(123.456) |
+---------------+
|           124 |
+---------------+
```

```
select ceil(123.456, 2);
```

```
+------------------+
| ceil(123.456, 2) |
+------------------+
|           123.46 |
+------------------+
```

```
select ceil(123.456, -2);
```

```
+-------------------+
| ceil(123.456, -2) |
+-------------------+
|               200 |
+-------------------+
```

```
select ceil(123.45, 1), ceil(123.45), ceil(123.45, 0), ceil(123.45, -1);
```

```
+-----------------+--------------+-----------------+------------------+
| ceil(123.45, 1) | ceil(123.45) | ceil(123.45, 0) | ceil(123.45, -1) |
+-----------------+--------------+-----------------+------------------+
|           123.5 |          124 |             124 |              130 |
+-----------------+--------------+-----------------+------------------+
```

```
select ceil(x, 2) from ( select cast(123.456 as decimal(6,3)) as x from numbers("number"="5") )t;
```

```
+------------+
| ceil(x, 2) |
+------------+
|     123.46 |
|     123.46 |
|     123.46 |
|     123.46 |
|     123.46 |
+------------+
```

7.2.1.1.9 CONV

描述

对输入的数字进行进制转换

语法

```
CONV(<input>, <from_base>, <to_base>)
```

参数

参数	说明
<input>	需要进行进制转换的参数，可为字符串或整数
<from_base>	数字，原始进制，范围应在 [2,36] 以内
<to_base>	数字，目标进制，范围应在 [2,36] 以内

返回值

转换后目标进制 <to_base> 下的数字，以字符串形式返回。

举例

```
SELECT CONV(15,10,2);
```

```
+-----------------+
```

```
| conv(15, 10, 2) |
+-----------------+
| 1111            |
+-----------------+
```

```
SELECT CONV('ff',16,10);
```

```
+--------------------+
| conv('ff', 16, 10) |
+--------------------+
| 255                |
+--------------------+
```

```
SELECT CONV(230,10,16);
```

```
+-------------------+
| conv(230, 10, 16) |
+-------------------+
| E6                |
+-------------------+
```

7.2.1.1.10 COS

描述

计算参数的余弦值

语法

```
COS(<a>)
```

参数

参数	说明
<a>	浮点数，要计算参数的弧度值

返回值

参数 <a> 的余弦值，弧度制表示。

举例

```
select cos(1);
```

```
+---------------------+
| cos(1.0)            |
+---------------------+
```

```
| 0.54030230586813977 |
+---------------------+
```

```
select cos(0);
```

```
+------------------------+
| cos(cast(0 as DOUBLE)) |
+------------------------+
|                    1.0 |
+------------------------+
```

```
select cos(Pi());
```

```
+-----------+
| cos(pi()) |
+-----------+
|        -1 |
+-----------+
```

7.2.1.1.11 COSH

描述

返回 x 的双曲余弦。

语法

COSH(<x>)

参数

参数	说明
<x>	需要被计算双曲余弦的值

返回值

参数 x 的双曲余弦值

示例

```
select cosh(0);
```

```
+-------------------------+
| cosh(cast(0 as DOUBLE)) |
+-------------------------+
|                     1.0 |
+-------------------------+
```

```
select cosh(1);
```

```
+-------------------------+
| cosh(cast(1 as DOUBLE)) |
+-------------------------+
|       1.543080634815244 |
+-------------------------+
```

```
select cosh(-1);
```

```
+--------------------------+
| cosh(cast(-1 as DOUBLE)) |
+--------------------------+
|        1.543080634815244 |
+--------------------------+
```

7.2.1.1.12 DEGREES

描述

输入一个 double 类型的浮点数，由弧度转换为角度。

- 当参数为 NUL 时，返回 NULL

语法

```
DEGREES(<a>)
```

参数

参数	说明
<a>	需要由弧度转换为角度的值

返回值

参数 a 的角度

- 当参数为 NUL 时，返回 NULL

举例

```
select degrees(3.14),degrees(1),degrees(-1),degrees(NULL)
```

```
+------------------------------+--------------------------+---------------------------+----------------+
    ↪
```

```
| degrees(cast(3.14 as DOUBLE)) | degrees(cast(1 as DOUBLE)) | degrees(cast(-1 as DOUBLE)) |
    ↪ degrees(NULL) |
+-------------------------------+----------------------------+-----------------------------+-----------------+
    ↪
|             179.9087476710785 |          57.29577951308232 |          -57.29577951308232 |
    ↪           NULL |
+-------------------------------+----------------------------+-----------------------------+-----------------+
    ↪
```

7.2.1.1.13 E

描述

返回常量 e 值。

语法

```
E()
```

举例

```
select e();
```

```
+-------------------+
| e()               |
+-------------------+
| 2.718281828459045 |
+-------------------+
```

7.2.1.1.14 EXP

描述

返回以e为底的x的幂。

别名

- DEXP

语法

```
EXP(<x>)
```

参数

参数	说明
<x>	自变量

返回值

返回一个 double。特殊情况：

- 当参数为NULL时，返回 NULL

举例

```
select exp(2);
```

```
+------------------+
| exp(2.0)         |
+------------------+
| 7.38905609893065 |
+------------------+
```

```
select exp(3.4);
```

```
+--------------------+
| exp(3.4)           |
+--------------------+
| 29.964100047397011 |
+--------------------+
```

7.2.1.1.15 FLOOR

描述

对浮点及定点小数按特定位数向下取整，返回取整过后的浮点或定点数。

语法

```
FLOOR(<a>[, <d>])
```

参数

参数	说明
<a>	浮点 (Double) 或定点 (Decimal) 参数，表示要进行取整的参数
<d>	可选的，整数，表示舍入目标位数，正数为向小数点后舍入，负数为向小数点前舍入，0 表示舍入到整数。不填写时等

返回值

按照下面规则返回最大的小于或者等于 <a> 的舍入数字：

舍入到 1/(10^d) 位，即，使结果可整除1/(10^d)。如果 1/(10^d) 不精确，则舍入位数为相应数据类型的最接近的数字。

对于 Decimal 类型的入参 <a>，假设其类型为 Decimal(p, s)，则返回值类型为：

- Decimal(p, 0)，若 <d> <= 0
- Decimal(p, <d>)，若 0 < <d> <= s
- Decimal(p, s)，若 <d> > s

别名

- DFLOOR

举例

```
select floor(123.456);
```

```
+----------------+
| floor(123.456) |
+----------------+
|            123 |
+----------------+
```

```
select floor(123.456, 2);
```

```
+-------------------+
| floor(123.456, 2) |
+-------------------+
|            123.45 |
+-------------------+
```

```
select floor(123.456, -2);
```

```
+--------------------+
| floor(123.456, -2) |
+--------------------+
|                100 |
+--------------------+
```

```
select floor(123.45, 1), floor(123.45), floor(123.45, 0), floor(123.45, -1);
```

```
+------------------+---------------+------------------+-------------------+
| floor(123.45, 1) | floor(123.45) | floor(123.45, 0) | floor(123.45, -1) |
+------------------+---------------+------------------+-------------------+
|            123.4 |           123 |              123 |               120 |
+------------------+---------------+------------------+-------------------+
```

```
select floor(x, 2) from ( select cast(123.456 as decimal(6,3)) as x from numbers("number"="5") )t
    ↪ ;
```

```
+-------------+
| floor(x, 2) |
+-------------+
|      123.45 |
|      123.45 |
|      123.45 |
|      123.45 |
|      123.45 |
+-------------+
```

7.2.1.1.16 FMOD

描述

求浮点数类型，a / b 的余数，整数类型请使用 mod 函数。

语法

```
MOD(<col_a> , <col_b>)
```

参数

参数	说明
<col_a>	被除数
<col_b>	除数不能为 0

返回值

返回一个浮点数。特殊情况：

- 当 col_a 为 NULL 或 col_b 为 NULL时，返回 NULL

举例

```
select fmod(10.1, 3.2);
```

```
+-----------------+
| fmod(10.1, 3.2) |
+-----------------+
|      0.50000024 |
+-----------------+
```

```
select fmod(10.1, 0);
```

```
+---------------+
| fmod(10.1, 0) |
```

```
+---------------+
|          NULL |
+---------------+
```

7.2.1.1.17 FORMAT_ROUND

描述

将数字格式化为类似于“#,###,###.##”的格式，四舍五入到指定位小数，并将结果作为字符串返回。

> 该函数自 3.0.6 版本开始支持.

语法

```
FORMAT_ROUND(<number>, <D>)
```

参数

参数	说明
<number>	需要被格式化的数字
<D>	小数位数，范围 [0, 1024]

返回值

返回格式化后的字符串。特殊情况：

- 当参数为 NULL 时，返回 NULL
- 如果 D 为 0，结果将没有小数点或小数部分。
- 如果 D 不在 [0, 1024] 范围内，将返回错误提示参数应在该范围内。

举例

```
mysql> select format_round(17014116, 2);
+---------------------------+
| format_round(17014116, 2) |
+---------------------------+
| 17,014,116.00             |
+---------------------------+
```

```
mysql> select format_round(1123.456, 2);
+---------------------------+
| format_round(1123.456, 2) |
+---------------------------+
```

```
| 1,123.46                  |
+---------------------------+
```

```
mysql> select format_round(1123.4, 2);
+-------------------------+
| format_round(1123.4, 2) |
+-------------------------+
| 1,123.40                |
+-------------------------+
```

```
mysql> select format_round(123456, 0);
+-------------------------+
| format_round(123456, 0) |
+-------------------------+
| 123,456                 |
+-------------------------+
```

```
mysql> select format_round(123456, 3);
+-------------------------+
| format_round(123456, 3) |
+-------------------------+
| 123,456.000             |
+-------------------------+
```

```
mysql> select format_round(123456.123456, 0);
+--------------------------------+
| format_round(123456.123456, 0) |
+--------------------------------+
| 123,456                        |
+--------------------------------+
```

```
mysql> select format_round(123456.123456, 3);
+--------------------------------+
| format_round(123456.123456, 3) |
+--------------------------------+
| 123,456.123                    |
+--------------------------------+
```

```
mysql> select format_round(123456.123456, 6);
+--------------------------------+
| format_round(123456.123456, 6) |
+--------------------------------+
| 123,456.123456                 |
+--------------------------------+
```

```
mysql> SELECT format_round(-0.01, -1);
ERROR 1105 (HY000): errCode = 2, detailMessage = (127.0.0.1)[INVALID_ARGUMENT]The second argument
    ↪  is -1, it should be in range [0, 1024].
mysql> SELECT format_round(-0.01, -1500);
ERROR 1105 (HY000): errCode = 2, detailMessage = (127.0.0.1)[INVALID_ARGUMENT]The second argument
    ↪  is -1500, it should be in range [0, 1024].
```

7.2.1.1.18 LN

描述

返回以e为底的x的自然对数。

别名

- DLOG1

语法

```
LN(<x>)
```

参数

参数	说明
<x>	真数必须大于 0

返回值

返回一个浮点数。特殊情况：

- 当 x IS NULL 时，返回NULL

举例

```
select ln(1);
```

```
+-----------------------+
| ln(cast(1 as DOUBLE)) |
+-----------------------+
|                   0.0 |
+-----------------------+
```

```
select ln(e());
```

```
+---------+
| ln(e()) |
+---------+
|     1.0 |
+---------+
```

```
select ln(10);
```

```
+------------------------+
| ln(cast(10 as DOUBLE)) |
+------------------------+
|      2.302585092994046 |
+------------------------+
```

7.2.1.1.19 LOG

描述

返回基于底数b的x的对数。

语法

```
LOG(<b>[, <x>])
```

参数

参数	说明
<b>	底数必须大于 0 且不等于 1
<x>	可选，真数必须大于 0，默认为自然数 e

返回值

返回一个浮点数。特殊情况：

- 当 b 为 NULL 或 x 为 NULL 时，返回NULL

举例

```
select log(5,1);
```

```
+---------------+
| log(5.0, 1.0) |
+---------------+
|             0 |
+---------------+
```

```
select log(3),ln(3);
```

```
+--------------------+--------------------+
| log(3)             | ln(3)              |
+--------------------+--------------------+
| 1.0986122886681098 | 1.0986122886681098 |
+--------------------+--------------------+
```

```
select log(3,20);
```

```
+--------------------+
| log(3.0, 20.0)     |
+--------------------+
| 2.7268330278608417 |
+--------------------+
```

```
select log(2,65536);
```

```
+-------------------+
| log(2.0, 65536.0) |
+-------------------+
|                16 |
+-------------------+
```

```
select log(5,NULL);
```

```
+------------------------------+
| log(cast(5 as DOUBLE), NULL) |
+------------------------------+
|                         NULL |
+------------------------------+
```

```
select log(NULL,3);
```

```
+------------------------------+
| log(NULL, cast(3 as DOUBLE)) |
+------------------------------+
|                         NULL |
+------------------------------+
```

7.2.1.1.20 LOG10

描述

返回以10为底的x的自然对数。

别名

- DLOG10

语法

```
LOG10(<x>)
```

参数

参数	说明
<x>	真数必须大于 0

返回值

返回一个浮点数。

- 当 x IS NULL：返回 NULL

举例

```
select log10(1);
```

```
+--------------------------+
| log10(cast(1 as DOUBLE)) |
+--------------------------+
|                      0.0 |
+--------------------------+
```

```
select log10(10);
```

```
+---------------------------+
| log10(cast(10 as DOUBLE)) |
+---------------------------+
|                       1.0 |
+---------------------------+
```

```
select log10(16);
```

```
+--------------------------+
| log10(cast(16 as DOUBLE)) |
+--------------------------+
|        1.2041199826559248 |
+--------------------------+
```

7.2.1.1.21 LOG2

描述

返回以2为底的x的自然对数。

语法

```
LOG2(<x>)
```

参数

参数	说明
<x>	真数 should be greater than 0

返回值

返回一个浮点数。特殊情况：

- 当 x 为 NULL 时，返回 NULL

举例

```
select log2(1);
```

```
+-------------------------+
| log2(cast(1 as DOUBLE)) |
+-------------------------+
|                     0.0 |
+-------------------------+
```

```
select log2(2);
```

```
+-------------------------+
| log2(cast(2 as DOUBLE)) |
+-------------------------+
|                     1.0 |
+-------------------------+
```

```
select log2(10);
```

```
+--------------------------+
| log2(cast(10 as DOUBLE)) |
+--------------------------+
|       3.3219280948873626 |
+--------------------------+
```

7.2.1.1.22 MOD

描述

求整数类型，a / b 的余数，浮点类型请使用 fmod 函数。

语法

```
MOD(<col_a> , <col_b>)
```

参数

参数	说明
<col_a>	被除数
<col_b>	除数不能为 0

返回值

返回一个整型。特殊情况：

- 当 col_a 为 NULL 或 col_b 为 NULL时，返回 NULL

举例

```
select mod(10, 3);
```

```
+----------+
| (10 % 3) |
+----------+
|        1 |
+----------+
```

```
select mod(10, 0);
```

```
+----------+
| (10 % 0) |
+----------+
```

```
|      NULL |
+-----------+
```

7.2.1.1.23 MONEY_FORMAT

描述

将数字按照货币格式输出，整数部分每隔 3 位用逗号分隔，小数部分保留 2 位

语法

```
MONEY_FORMAT(<number>)
```

参数

参数	说明
<number>	需要被格式化的数字

返回值

返回货币格式的字符串。特殊情况：

- 当参数为 NULL 时，返回 NULL

举例

```
select money_format(17014116);
```

```
+------------------------+
| money_format(17014116) |
+------------------------+
| 17,014,116.00          |
+------------------------+
```

```
select money_format(1123.456);
```

```
+------------------------+
| money_format(1123.456) |
+------------------------+
| 1,123.46               |
+------------------------+
```

```
select money_format(1123.4);
```

```
+----------------------+
| money_format(1123.4) |
```

```
+----------------------+
| 1,123.40             |
+----------------------+
```

7.2.1.1.24 NEGATIVE

描述

返回传参 x 的负值

语法

```
NEGATIVE(<x>)
```

参数

参数	说明
<x>	自变量支持类型BIGINT DOUBLE DECIMAL

返回值

返回整型或者浮点数。特殊情况：

- 当参数为 NULL 时，返回 NULL
- 当参数为 0 时，返回 0

举例

```
SELECT negative(-10);
```

```
+---------------+
| negative(-10) |
+---------------+
|            10 |
+---------------+
```

```
SELECT negative(12);
```

```
+--------------+
| negative(12) |
+--------------+
|          -12 |
+--------------+
```

```
SELECT negative(0);
```

```
+-------------+
| negative(0) |
+-------------+
|           0 |
+-------------+
```

```
SELECT negative(null);
```

```
+----------------+
| negative(NULL) |
+----------------+
|           NULL |
+----------------+
```

7.2.1.1.25 NORMAL_CDF

描述

计算正态分布在值 x 处的累积分布函数 (CDF)。

- 当正态分布的标准差小于等于 0，则返回 NULL。

语法

```
NORMAL_CDF(<mean>, <sd>, <x>)
```

参数

参数	说明
<mean>	正态分布的均值
<sd>	正态分布的标准差
<x>	要评价的值

返回值

返回某一正态随机变量在值 x 处的累积分布函数 (CDF) 值。

- 当正态分布的标准差小于等于 0，则返回 NULL。

举例

```
select normal_cdf(10, 9, 10);
```

```
+------------------------------------------------------------------------+
| normal_cdf(cast(10 as DOUBLE), cast(9 as DOUBLE), cast(10 as DOUBLE)) |
```

```
+------------------------------------------------------------------------+
|                                                                    0.5 |
+------------------------------------------------------------------------+
```

```
select NORMAL_CDF(10, 0, 10);
```

```
+------------------------------------------------------------------------+
| normal_cdf(cast(10 as DOUBLE), cast(0 as DOUBLE), cast(10 as DOUBLE)) |
+------------------------------------------------------------------------+
|                                                                   NULL |
+------------------------------------------------------------------------+
```

7.2.1.1.26 PI

描述

返回常量Pi值。

语法

```
PI()
```

返回值

返回一个常量 Pi，类型为浮点数

举例

```
select Pi();
```

```
+-------------------+
| pi()              |
+-------------------+
| 3.141592653589793 |
+-------------------+
```

7.2.1.1.27 PMOD

描述

返回模运算 x mod y 在模系中的最小正数解，即通过计算 (x % y + y) % y 得出结果。

语法

```
PMOD(<x> , <y>)
```

参数

参数	说明
<x>	被除数
<y>	除数不能为 0

返回值

返回一个整型或浮点数。特殊情况：

- 当 x=0 时，返回 0。
- 当 x is NULL 或 y is NULL 时，返回 NULL。

举例

```
SELECT PMOD(13,5);
```

```
+-------------+
| pmod(13, 5) |
+-------------+
|           3 |
+-------------+
```

```
SELECT PMOD(-13,5);
```

```
+--------------+
| pmod(-13, 5) |
+--------------+
|            2 |
+--------------+
```

```
SELECT PMOD(0,-12);
```

```
+--------------+
| pmod(0, -12) |
+--------------+
|            0 |
+--------------+
```

```
SELECT PMOD(0,null);
```

```
+-------------------------------+
| pmod(cast(0 as DOUBLE), NULL) |
+-------------------------------+
|                          NULL |
+-------------------------------+
```

7.2.1.1.28 POSITIVE

描述

返回数值本身。

语法

```
POSITIVE(<x>)
```

参数

参数	说明
<x>	需要返回的数值

返回值

返回一个整型或者浮点数。特殊情况：

- 当 x is NULL 时，返回 NULL

举例

```
SELECT positive(-10);
```

```
+---------------+
| positive(-10) |
+---------------+
|           -10 |
+---------------+
```

```
SELECT positive(10.111);
```

```
+------------------+
| positive(10.111) |
+------------------+
|           10.111 |
+------------------+
```

```
SELECT positive(12);
```

```
+--------------+
| positive(12) |
+--------------+
|           12 |
+--------------+
```

```
SELECT positive(null);
```

```
+----------------+
| positive(NULL) |
+----------------+
|           NULL |
+----------------+
```

7.2.1.1.29 POW

描述

用于计算 a 的 b 次方。

别名

- POWER
- FPOW
- DPOW

语法

```
POW(<a>, <b>)
```

参数

参数	说明
<a>	基数
<b>	指数

返回值

返回参数 a 的 b 次方。

特殊情况：

- 当 a 或 b 为 NULL 时，返回 NULL。
- 当 b = 0 且 a 不为 NULL 时，永远返回 1。

示例

```
select pow(2, 0);
```

```
+-----------------------------------------+
| pow(cast(2 as DOUBLE), cast(0 as DOUBLE)) |
+-----------------------------------------+
```

```
|                                          1 |
+--------------------------------------------+
```

```
select pow(2, 10);
```

```
+-------------------------------------------+
| pow(cast(2 as DOUBLE), cast(10 as DOUBLE)) |
+-------------------------------------------+
|                                     1024 |
+-------------------------------------------+
```

```
select pow(1.2, 2);
```

```
+-------------------------------------------+
| pow(cast(1.2 as DOUBLE), cast(2 as DOUBLE)) |
+-------------------------------------------+
|                                      1.44 |
+-------------------------------------------+
```

```
select pow(1.2, 2.1);
```

```
+-----------------------------------------------+
| pow(cast(1.2 as DOUBLE), cast(2.1 as DOUBLE)) |
+-----------------------------------------------+
|                           1.4664951016517147 |
+-----------------------------------------------+
```

```
select pow(2, null);
```

```
+------------------------------+
| pow(cast(2 as DOUBLE), NULL) |
+------------------------------+
|                         NULL |
+------------------------------+
```

```
select pow(null, 2);
```

```
+------------------------------+
| pow(NULL, cast(2 as DOUBLE)) |
+------------------------------+
|                         NULL |
+------------------------------+
```

7.2.1.1.30 RADIANS

描述

返回x的弧度值，从度转换为弧度。

语法

```
RADIANS(<x>)
```

参数

参数	说明
<x>	需要被计算的从度

返回值

返回一个整型或者浮点数。特殊情况：

- 当参数 x is NULL，返回 NULL

举例

```
select radians(0);
```

```
+----------------------------+
| radians(cast(0 as DOUBLE)) |
+----------------------------+
|                        0.0 |
+----------------------------+
```

```
select radians(30);
```

```
+-----------------------------+
| radians(cast(30 as DOUBLE)) |
+-----------------------------+
|          0.5235987755982988 |
+-----------------------------+
```

```
select radians(90);
```

```
+-----------------------------+
| radians(cast(90 as DOUBLE)) |
+-----------------------------+
|          1.5707963267948966 |
+-----------------------------+
```

7.2.1.1.31 RANDOM

描述

返回 0-1 之间的随机数，或者根据参数返回需要的随机数。

- 注意：所有参数必须为常量。

别名

- RAND

语法

```
RANDOM() --生成 0-1 之间的随机数

RANDOM(<seed>) --根据 seed 种子值，生成一个 0-1 之间的固定随机数序列

RANDOM(<a> , <b>) --生成 a-b 之间的随机数
```

参数

参数	说明
<seed>	随机数生成器的种子值根据种子值返回一个 0-1 之间的固定随机数序列
<a>	随机数的下限
<b>	随机数的上限必须小于下限

返回值

- 不传参时：返回 0-1 之间的随机数。
- 传入单个参数seed时：根据传入的种子值seed，返回一个 0-1 之间的固定随机数序列。
- 传入两个参数a和b时：返回 a-b 之间的随机整数。

举例

```
select random();
```

```
+--------------------+
| random()           |
+--------------------+
| 0.8047437125910604 |
+--------------------+
```

```
select rand(1.2);
```

```
+---------------------+
| rand(1)             |
+---------------------+
| 0.13387664401253274 |
+---------------------+
```

```
select rand(-20, -10);
```

```
+------------------+
| random(-20, -10) |
+------------------+
|              -10 |
+------------------+
```

7.2.1.1.32 ROUND

描述

将数字 x 四舍五入后保留 d 位小数。- 如果没有指定 d，则将 x 四舍五入到最近的整数。- 如果 d 为负数，则结果小数点左边 d 位为 0。- 如果 x 或 d 为 null ，返回 null。- 如果 d 为一个列，并且第一个参数为 Decimal 类型，那么结果 Decimal 会跟入参 Decimal 具有相同的小数部分长度。

别名

- DROUND

语法

```
ROUND(<x> [ , <d> ])
```

参数

参数	说明
<x>	需要四舍五入的数值
<d>	可选，四舍五入需要保留的小数位数

举例

```
select round(2.4);
```

```
+------------+
| round(2.4) |
+------------+
|          2 |
+------------+
```

```
select round(2.5);
```

```
+------------+
| round(2.5) |
+------------+
|          3 |
+------------+
```

```
select round(-3.4);
```

```
+-------------+
| round(-3.4) |
+-------------+
|          -3 |
+-------------+
```

```
select round(-3.5);
```

```
+-------------+
| round(-3.5) |
+-------------+
|          -4 |
+-------------+
```

```
select round(1667.2725, 2);
```

```
+----------------------+
| round(1667.2725, 2) |
+----------------------+
|              1667.27 |
+----------------------+
```

```
select round(1667.2725, -2);
```

```
+-----------------------+
| round(1667.2725, -2) |
+-----------------------+
|                 1700 |
+-----------------------+
```

```
CREATE TABLE test_enhanced_round (
    rid int, flo float, dou double,
    dec90 decimal(9, 0), dec91 decimal(9, 1), dec99 decimal(9, 9),
```

```
    dec100 decimal(10,0), dec109 decimal(10,9), dec1010 decimal(10,10),
    number int DEFAULT 1)
DISTRIBUTED BY HASH(rid)
PROPERTIES("replication_num" = "1" );

INSERT INTO test_enhanced_round
VALUES
(1, 12345.123, 123456789.123456789,
    123456789, 12345678.1, 0.123456789,
    123456789.1, 1.123456789, 0.123456789, 1);

SELECT number, dec90, round(dec90, number), dec91, round(dec91, number), dec99, round(dec99,
    ↪ number) FROM test_enhanced_round order by rid;
```

```
+--------+-----------+----------------------+------------+----------------------+-------------+-----------------
    ↪
| number | dec90     | round(dec90, number) | dec91      | round(dec91, number) | dec99       |
    ↪ round(dec99, number) |
+--------+-----------+----------------------+------------+----------------------+-------------+-----------------
    ↪
|      1 | 123456789 |            123456789 | 12345678.1 |           12345678.1 | 0.123456789 |
    ↪           0.100000000 |
+--------+-----------+----------------------+------------+----------------------+-------------+-----------------
    ↪
```

注意事项

2.5 会舍入到 3，如果想要舍入到 2 的算法，请使用 round_bankers 函数。

7.2.1.1.33 ROUND_BANKERS

描述

将x使用银行家舍入法后，保留 d 位小数，d默认为 0。

如果d为负数，则小数点左边d位为 0。

如果x或d为 null，返回 null。

如果 d 为一个列，并且第一个参数为 Decimal 类型，那么结果 Decimal 会跟入参 Decimal 具有相同的小数部分长度。

根据银行家舍入算法的规则，当需要舍入到指定的小数位时：

- 如果要舍入的数字是 5，且后面没有其他非零数字，则会检查前一位数字：
- 如果前一位数字是偶数，则直接舍去；
- 如果前一位数字是奇数，则向上进一。

- 如果要舍入的数字大于 5 或者其后有非 0 的数字，则按照传统的四舍五入规则进行：即大于等于 5 则进位，否则舍去。

例如：

- 对于数值 2.5，由于 5 前面的数字 2 是偶数，因此结果将舍入为 2。
- 对于数值 3.5，由于 5 前面的数字 3 是奇数，因此结果将舍入为 4。
- 对于数值 2.51，因为 5 后面的数字不是 0，所以直接进位，结果为 3。

语法

```
ROUND_BANKERS(<x> [ , <d>])
```

参数

参数	说明
<x>	需要四舍五入的数值
<d>	可选，四舍五入需要保留的小数位数，默认为 0

返回值

返回一个整型或者浮点数：

- 默认情况，参数 d = 0 , 返回 x 根据银行家舍入算法计算过的整数。
- d 为负数, 返回小数点左边第一位为 0 的整数。
- x 和 d is NULL , 返回 NULL。
- d 为一个列时, 且 x 为 Decimal 类型, 返回相同精度的浮点数。

举例

```
select round_bankers(0.4);
```

```
+--------------------+
| round_bankers(0.4) |
+--------------------+
|                  0 |
+--------------------+
```

```
select round_bankers(-3.5);
```

```
+---------------------+
| round_bankers(-3.5) |
```

```
+---------------------+
|                  -4 |
+---------------------+
```

```
select round_bankers(-3.4);
```

```
+---------------------+
| round_bankers(-3.4) |
+---------------------+
|                  -3 |
+---------------------+
```

```
select round_bankers(10.755, 2);
```

```
+--------------------------+
| round_bankers(10.755, 2) |
+--------------------------+
|                    10.76 |
+--------------------------+
```

```
select round_bankers(10.745, 2);
```

```
+--------------------------+
| round_bankers(10.745, 2) |
+--------------------------+
|                    10.74 |
+--------------------------+
```

```
select round_bankers(1667.2725, -2);
```

```
+------------------------------+
| round_bankers(1667.2725, -2) |
+------------------------------+
|                         1700 |
+------------------------------+
```

```
SELECT number
, round_bankers(number * 2.5, number - 1) AS rb_decimal_column
, round_bankers(number * 2.5, 0) AS rb_decimal_literal
, round_bankers(cast(number * 2.5 AS DOUBLE), number - 1) AS rb_double_column
, round_bankers(cast(number * 2.5 AS DOUBLE), 0) AS rb_double_literal
FROM test_enhanced_round
WHERE rid = 1;
```

```
+--------+------------------+--------------------+------------------+-------------------+
| number | rb_decimal_column | rb_decimal_literal | rb_double_column | rb_double_literal |
+--------+------------------+--------------------+------------------+-------------------+
|      1 |              2.0 |                  2 |                2 |                 2 |
+--------+------------------+--------------------+------------------+-------------------+
```

7.2.1.1.34 SIGN

描述

返回x的符号。负数，零或正数分别对应 -1，0 或 1。

语法

```
SIGN(x)
```

参数

参数	说明
<x>	自变量

返回值

返回一个整型：

- 当 x > 0 时，返回 1，代表整数。
- 当 x = 0 时，返回 0，代表零。
- 当 x < 0 时，返回 -1，代表负数。
- 当 x is NULL 时，返回 NULL。

举例

```
select sign(3);
```

```
+-------------------------+
| sign(cast(3 as DOUBLE)) |
+-------------------------+
|                       1 |
+-------------------------+
```

```
select sign(0);
```

```
+-------------------------+
| sign(cast(0 as DOUBLE)) |
+-------------------------+
|                       0 |
+-------------------------+
```

```
select sign(-10.0);
```

```
+-----------------------------+
| sign(cast(-10.0 as DOUBLE)) |
+-----------------------------+
|                          -1 |
+-----------------------------+
```

```
select sign(null);
```

```
+------------+
| sign(NULL) |
+------------+
|       NULL |
+------------+
```

7.2.1.1.35 SIN

描述

计算参数的正弦值

语法

```
SIN(<a>)
```

参数

参数	说明
<a>	浮点数，要计算参数的弧度值

返回值

参数 <a> 的正弦值，弧度制表示。

语法

```
SIN(<x>)
```

参数

参数	说明
<x>	弧度值

返回值

返回浮点数。特殊情况：

- 当 x is NULL 时，返回 NULL.

举例

```
select sin(1);
```

```
+------------------------+
| sin(cast(1 as DOUBLE)) |
+------------------------+
|     0.8414709848078965 |
+------------------------+
```

```
select sin(0);
```

```
+------------------------+
| sin(cast(0 as DOUBLE)) |
+------------------------+
|                    0.0 |
+------------------------+
```

```
select sin(Pi());
```

```
+-----------------------------------+
| sin(pi())                         |
+-----------------------------------+
| 0.00000000000000012246467991473532 |
+-----------------------------------+
```

7.2.1.1.36 SQRT

描述

返回一个值的平方根，要求输入值大于或等于 0。

别名

- NVL

语法

```
SQRT(<a>)
```

参数

参数	说明
<a>	需要被计算平方根的值

返回值

参数 a 的平方根

示例

```
select sqrt(9),sqrt(2)
```

```
+-------------------------+-------------------------+
| sqrt(cast(9 as DOUBLE)) | sqrt(cast(2 as DOUBLE)) |
+-------------------------+-------------------------+
|                     3.0 |      1.4142135623730951 |
+-------------------------+-------------------------+
```

7.2.1.1.37 TAN

描述

返回 x 的正切值，x 为弧度值

语法

```
TAN(<x>)
```

参数

参数	说明
<x>	需要被计算正切值的值

返回值

返回 x 的正切值

举例

```
select tan(0),tan(1),tan(-1);
```

```
+------------------------+------------------------+-------------------------+
| tan(cast(0 as DOUBLE)) | tan(cast(1 as DOUBLE)) | tan(cast(-1 as DOUBLE)) |
```

```
+-----------------------+-----------------------+-------------------------+
|                     0 |     1.5574077246549023 |     -1.5574077246549023 |
+-----------------------+-----------------------+-------------------------+
```

7.2.1.1.38 TANH

描述

返回 x 的双曲正切值。

语法

```
TANH(<x>)
```

参数

参数	说明
<x>	需要被计算双曲正切值

返回值

参数 x 的双曲正切值。

举例

```
select tanh(0),tanh(1);
```

```
+-------------------------+-------------------------+
| tanh(cast(0 as DOUBLE)) | tanh(cast(1 as DOUBLE)) |
+-------------------------+-------------------------+
|                       0 |      0.7615941559557649 |
+-------------------------+-------------------------+
```

7.2.1.1.39 TRUNCATE

描述

按照保留小数的位数 d 对 x 进行数值截取。

语法

```
TRUNCATE(<x>, <d>)
```

参数

参数	说明
<x>	需要被数值截取的值
<d>	保留小数的位数

返回值

按照保留小数的位数 d 对 x 进行数值截取。截取规则：

如果 d 为字面量：

- 当d > 0时：保留x的d位小数
- 当d = 0时：将x的小数部分去除，只保留整数部分
- 当d < 0时：将x的小数部分去除，整数部分按照 d所指定的位数，采用数字0进行替换

如果 d 为一个列，并且第一个参数为 Decimal 类型，那么结果 Decimal 会跟入参 Decimal 具有相同的小数部分长度。

举例

d 为字面量

```
select truncate(124.3867, 2),truncate(124.3867, 0),truncate(124.3867, -2);
```

```
+----------------------+----------------------+-----------------------+
| truncate(124.3867, 2) | truncate(124.3867, 0) | truncate(124.3867, -2) |
+----------------------+----------------------+-----------------------+
|                124.38 |                  124 |                   100 |
+----------------------+----------------------+-----------------------+
```

d 为一个列

```
select cast("123.123456" as Decimal(9,6)), number, truncate(cast ("123.123456" as Decimal(9,6)),
    ↪ number) from numbers("number"="5");
```

```
+--------------------------------------+--------+-----------------------------------------------------------------
    ↪
| cast('123.123456' as DECIMALV3(9, 6)) | number | truncate(cast('123.123456' as DECIMALV3(9, 6))
    ↪ , cast(number as INT)) |
+--------------------------------------+--------+-----------------------------------------------------------------
    ↪
|                             123.123456 |      0 |
    ↪                                                          123.000000 |
|                             123.123456 |      1 |
    ↪                                                          123.100000 |
|                             123.123456 |      2 |
    ↪                                                          123.120000 |
|                             123.123456 |      3 |
    ↪                                                          123.123000 |
|                             123.123456 |      4 |
    ↪                                                          123.123400 |
```

```
+--------------------------------------+--------+-------------------------------------------------------------------------
    ↪
```

7.2.1.1.40 WIDTH_BUCKET

描述

构造等宽直方图，其中直方图范围被划分为相同大小的区间，并在计算后返回表达式的值所在的桶号。特殊情况：

- 该函数返回一个整数值或空值（如果任何输入为空值则返回空值）。

语法

```
WIDTH_BUCKET(<expr>, <min_value>, <max_value>, <num_buckets>)
```

参数

参数	说明

参数	说明
<↪ expr ↪ > ↪	创建直方图的表达式。此表达式必须计算为数值或可隐式转换为数值的值，此值的范围必须为 -(2^53 ↪ ↪ - ↪ ↪ 1) ↪

参数	说明
< ↪ min ↪ _ ↪ value ↪ > ↪	表达式可接受范围的最低值点。参数必须为数值且不等于< ↪ max ↪ _ ↪ value ↪ > ↪ ，范围必须为-(2^53 ↪ ↪ - ↪ ↪ 1) ↪ to 2^53 ↪ ↪ - ↪ ↪ 1 ↪ (含)

参数	说明
< ↪ max ↪ _ ↪ value ↪ > ↪	表达式可接受范围的最高值点。参数必须为数值且不等于< ↪ min ↪ _ ↪ value ↪ > ↪ ，范围必须为 -(2^53 ↪ ↪ - ↪ ↪ 1) ↪ to 2^53 ↪ ↪ - ↪ ↪ 1 ↪ (含)

参数	说明
 ↪ num ↪ _ ↪ buckets ↪ > ↪	分桶的数量，必须是正整数值。将表达式中的一个值分配给每个存储桶，然后该函数返回相应的存储桶编号

参数	说明

返回值

返回表达式值所在的桶号。当表达式超出范围时，函数返回规则如下：

- 如果表达式的值小于min_value返回0.
- 如果表达式的值大于或等于max_value返回num_buckets + 1.
- 如果任意参数为null返回null.

举例

```
DROP TABLE IF EXISTS width_bucket_test;

CREATE TABLE IF NOT EXISTS width_bucket_test (
`k1` int NULL COMMENT "",
`v1` date NULL COMMENT "",
`v2` double NULL COMMENT "",
`v3` bigint NULL COMMENT ""
) ENGINE=OLAP
DUPLICATE KEY(`k1`)
DISTRIBUTED BY HASH(`k1`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1",
"storage_format" = "V2"
);

INSERT INTO width_bucket_test VALUES
(1, "2022-11-18", 290000.00, 290000),
(2, "2023-11-18", 320000.00, 320000),
(3, "2024-11-18", 399999.99, 399999),
(4, "2025-11-18", 400000.00, 400000),
(5, "2026-11-18", 470000.00, 470000),
(6, "2027-11-18", 510000.00, 510000),
(7, "2028-11-18", 610000.00, 610000),
(8, null, null, null);
```

```
SELECT * FROM width_bucket_test ORDER BY k1;
```

```
+------+------------+-----------+--------+
| k1   | v1         | v2        | v3     |
+------+------------+-----------+--------+
|    1 | 2022-11-18 |    290000 | 290000 |
|    2 | 2023-11-18 |    320000 | 320000 |
```

```
|    3 | 2024-11-18 | 399999.99 | 399999 |
|    4 | 2025-11-18 |    400000 | 400000 |
|    5 | 2026-11-18 |    470000 | 470000 |
|    6 | 2027-11-18 |    510000 | 510000 |
|    7 | 2028-11-18 |    610000 | 610000 |
|    8 | NULL       |      NULL |   NULL |
+------+------------+-----------+--------+
```

```
SELECT k1, v1, v2, v3, width_bucket(v1, date('2023-11-18'), date('2027-11-18'), 4) AS w FROM
    ↪ width_bucket_test ORDER BY k1;
```

```
+------+------------+-----------+--------+------+
| k1   | v1         | v2        | v3     | w    |
+------+------------+-----------+--------+------+
|    1 | 2022-11-18 |    290000 | 290000 |    0 |
|    2 | 2023-11-18 |    320000 | 320000 |    1 |
|    3 | 2024-11-18 | 399999.99 | 399999 |    2 |
|    4 | 2025-11-18 |    400000 | 400000 |    3 |
|    5 | 2026-11-18 |    470000 | 470000 |    4 |
|    6 | 2027-11-18 |    510000 | 510000 |    5 |
|    7 | 2028-11-18 |    610000 | 610000 |    5 |
|    8 | NULL       |      NULL |   NULL | NULL |
+------+------------+-----------+--------+------+
```

```
SELECT k1, v1, v2, v3, width_bucket(v2, 200000, 600000, 4) AS w FROM width_bucket_test ORDER BY
    ↪ k1;
```

```
+------+------------+-----------+--------+------+
| k1   | v1         | v2        | v3     | w    |
+------+------------+-----------+--------+------+
|    1 | 2022-11-18 |    290000 | 290000 |    1 |
|    2 | 2023-11-18 |    320000 | 320000 |    2 |
|    3 | 2024-11-18 | 399999.99 | 399999 |    2 |
|    4 | 2025-11-18 |    400000 | 400000 |    3 |
|    5 | 2026-11-18 |    470000 | 470000 |    3 |
|    6 | 2027-11-18 |    510000 | 510000 |    4 |
|    7 | 2028-11-18 |    610000 | 610000 |    5 |
|    8 | NULL       |      NULL |   NULL | NULL |
+------+------------+-----------+--------+------+
```

```
SELECT k1, v1, v2, v3, width_bucket(v3, 200000, 600000, 4) AS w FROM width_bucket_test ORDER BY
    ↪ k1;
```

```
+------+------------+-----------+--------+------+
| k1   | v1         | v2        | v3     | w    |
```

```
+------+------------+-----------+--------+------+
|    1 | 2022-11-18 |    290000 | 290000 |    1 |
|    2 | 2023-11-18 |    320000 | 320000 |    2 |
|    3 | 2024-11-18 | 399999.99 | 399999 |    2 |
|    4 | 2025-11-18 |    400000 | 400000 |    3 |
|    5 | 2026-11-18 |    470000 | 470000 |    3 |
|    6 | 2027-11-18 |    510000 | 510000 |    4 |
|    7 | 2028-11-18 |    610000 | 610000 |    5 |
|    8 | NULL       |      NULL |   NULL | NULL |
+------+------------+-----------+--------+------+
```

7.2.1.2 字符串函数

7.2.1.2.1 APPEND_TRAILING_CHAR_IF_ABSENT

描述

用于在字符串末尾添加特定字符（如空格、特定符号等），如果该字符不存在于字符串的结尾时进行添加。函数的作用是确保字符串以特定字符结尾。

语法

```
APPEND_TRAILING_CHAR_IF_ABSENT ( <str> , <trailing_char> )
```

参数

参数	说明
<str>	需要判断的目标字符串
<trailing_char>	需要添加到字符串末尾的字符（如果该字符不存在的话）

返回值

参数 <str> 与 <trailing_char> 拼接后的字符串（如果 <trailing_char> 不存在于 <str> ）

举例

```
SELECT APPEND_TRAILING_CHAR_IF_ABSENT('a','c'),APPEND_TRAILING_CHAR_IF_ABSENT('ac', 'c'),APPEND_
    ↪ TRAILING_CHAR_IF_ABSENT('ac', 'cd')
```

```
+-----------------------------------------+-----------------------------------------+----------------------------
    ↪
| append_trailing_char_if_absent('a', 'c') | append_trailing_char_if_absent('ac', 'c') | append_
    ↪ trailing_char_if_absent('ac', 'cd') |
+-----------------------------------------+-----------------------------------------+----------------------------
    ↪
| ac                                      | ac                                      | accd
    ↪                                       |
```

```
+----------------------------------------+-----------------------------------------+----------
    ↪
```

7.2.1.2.2 ASCII

描述

返回字符串第一个字符对应的 ascii 码

语法

```
ASCII ( <str> )
```

参数

参数	说明
<str>	需要被计算第一个字符的 ascii 码的字符串

返回值

参数第一个字符的 ascii 码

举例

```
SELECT ASCII('1'),ASCII('234')
```

```
+------------+--------------+
| ascii('1') | ascii('234') |
+------------+--------------+
|         49 |           50 |
+------------+--------------+
```

7.2.1.2.3 AUTO_PARTITION_NAME

> tip Supported since Apache Doris 2.1.6

Description

Syntax

```
VARCHAR AUTO_PARTITION_NAME('RANGE', 'VARCHAR unit', DATETIME datetime)
```

```
VARCHAR AUTO_PARTITION_NAME('LIST', VARCHAR,...)
```

Generate datetime partition names by unit following RANGE’s partition name rules

Convert strings to partition names following LIST's partition name rules

The datetime parameter is a legal date expression.

The unit parameter is the time interval you want, the available values are: [second, minute, hour, day, month, year]. If unit does not match one of these options, a syntax error will be returned.

Supported since Doris 2.1.6

Example

```
mysql> select auto_partition_name('range', 'years', '123');
ERROR 1105 (HY000): errCode = 2, detailMessage = range auto_partition_name must accept year|month
    ↪ |day|hour|minute|second for 2nd argument

mysql> select auto_partition_name('range', 'year', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'year', '2022-12-12 19:20:30')   |
+-----------------------------------------------------------------+
| p20220101000000                                                 |
+-----------------------------------------------------------------+

mysql> select auto_partition_name('range', 'month', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'month', '2022-12-12 19:20:30')  |
+-----------------------------------------------------------------+
| p20221201000000                                                 |
+-----------------------------------------------------------------+

mysql> select auto_partition_name('range', 'day', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'day', '2022-12-12 19:20:30')    |
+-----------------------------------------------------------------+
| p20221212000000                                                 |
+-----------------------------------------------------------------+

mysql> select auto_partition_name('range', 'hour', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'hour', '2022-12-12 19:20:30')   |
+-----------------------------------------------------------------+
| p20221212190000                                                 |
+-----------------------------------------------------------------+

mysql> select auto_partition_name('range', 'minute', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'minute', '2022-12-12 19:20:30') |
+-----------------------------------------------------------------+
| p20221212192000                                                 |
```

```
+-----------------------------------------------------------------+

mysql> select auto_partition_name('range', 'second', '2022-12-12 19:20:30');
+-----------------------------------------------------------------+
| auto_partition_name('range', 'second', '2022-12-12 19:20:30') |
+-----------------------------------------------------------------+
| p20221212192030                                                 |
+-----------------------------------------------------------------+

mysql> select auto_partition_name('list', 'helloworld');
+-----------------------------------------+
| auto_partition_name('list', 'helloworld') |
+-----------------------------------------+
| phelloworld10                           |
+-----------------------------------------+

mysql> select auto_partition_name('list', 'hello', 'world');
+-----------------------------------------------+
| auto_partition_name('list', 'hello', 'world') |
+-----------------------------------------------+
| phello5world5                                 |
+-----------------------------------------------+

mysql> select auto_partition_name('list', "你好");
+-------------------------------------+
| auto_partition_name('list', "你好") |
+-------------------------------------+
| p4f60597d2                          |
+-------------------------------------+
```

Keywords

```
AUTO_PARTITION_NAME,AUTO,PARTITION,NAME
```

7.2.1.2.4 CHAR

描述

将每个参数解释为整数，并返回一个字符串，该字符串由这些整数的代码值给出的字符组成。特殊情况：

- 如果结果字符串对于给定字符集是非法的，相应的转换结果为 NULL 值。
- 大于 255 的参数将转换为多个结果字节。例如，`char(15049882)`等价于`char(229, 164, 154)`。

语法

```
CHAR ( <expr> [ , <expr> ... ] [ USING <charset_name> ] )
```

参数

参数	说明
<expr>	需要被计算为字符的整数
<charset_name>	返回值的编码，目前只支持 utf8

返回值

参数列表 <expr> 对应字符组成的字符串。特殊情况：

- 如果结果字符串对于给定字符集是非法的，相应的转换结果为 NULL 值。
- 大于 255 的参数将转换为多个结果字节。例如，CHAR(15049882)等价于CHAR(229, 164, 154)。

举例

```
SELECT CHAR(68, 111, 114, 105, 115),CHAR(15049882, 15179199, 14989469),CHAR(255)
```

```
+-------------------------------------+-------------------------------------------+------------------+
    ↪
| char('utf8', 68, 111, 114, 105, 115) | char('utf8', 15049882, 15179199, 14989469) | char('utf8
    ↪ ', 255) |
+-------------------------------------+-------------------------------------------+------------------+
    ↪
| Doris                               | 多睿丝                                     | NULL
    ↪              |
+-------------------------------------+-------------------------------------------+------------------+
    ↪
```

7.2.1.2.5 CHAR_LENGTH

描述

计算字符串的长度，对于多字节字符，返回字符数。

目前仅支持 utf8 编码

别名

- CHARACTER_LENGTH

语法

```
CHAR_LENGTH ( <str> )
```

参数

参数	说明
<str>	需要计算长度的字符串

返回值

字符串的长度。

举例

```
select CHAR_LENGTH("abc"),CHAR_LENGTH("中国")
```

```
+-------------------------+---------------------------+
| character_length('abc') | character_length('中国')   |
+-------------------------+---------------------------+
|                       3 |                         2 |
+-------------------------+---------------------------+
```

7.2.1.2.6 CONCAT

描述

将多个字符串连接起来。特殊情况：

- 如果参数中任意一个值是 NULL，那么返回的结果就是 NULL

语法

```
CONCAT ( <expr> [ , <expr> ... ] )
```

参数

参数	说明
<expr>	需要被连接到一起的字符串

返回值

参数列表 <expr> 连接到一起的字符串。特殊情况：

- 如果参数中任意一个值是 NULL，那么返回的结果就是 NULL

举例

```
SELECT  CONCAT("a", "b"),CONCAT("a", "b", "c"),CONCAT("a", null, "c")
```

```
+-----------------+----------------------+-----------------------+
| concat('a', 'b') | concat('a', 'b', 'c') | concat('a', NULL, 'c') |
```

```
+------------------+----------------------+------------------------+
| ab               | abc                  | NULL                   |
+------------------+----------------------+------------------------+
```

7.2.1.2.7 CONCAT_WS

描述

使用第一个参数 sep 作为连接符，将第二个参数及后续所有参数（或一个数组、多个数组中的所有字符串）拼接成一个字符串。特殊情况：

- 若分隔符为 NULL，则返回 NULL。
- CONCAT_WS 函数不跳过空字符串（“”），但会跳过 NULL 值。
- CONCAT_WS 函数不跳过任何数组参数中的空字符串，但会跳过数组中的 NULL 值。
- CONCAT_WS 函数不会跳过 NULL 如果输入多个数组参数，会返回空字符串。
- 第一个参数必须为字符串类型（string 或 varchar），其他参数必须为相同类型，即均为字符串类型（string 或 varchar）或均为数组类型（ARRAY）。###### 语法

```
CONCAT_WS ( <sep> , <str> [ , <str> ] )
CONCAT_WS ( <sep> , <array> [ , <array> ])
```

参数

参数	说明
<sep>	用于拼接字符串的连接符，类型为 string 或 varchar
<str>	待拼接的字符串，类型为 string 或 varchar
<array>	待拼接的数组，类型为 ARRAY，且数组元素为 string 或 varchar

返回值

参数 <sep> 或者 <array> 数组使用 <str> 拼接后字符串。特殊情况：

- 如果分隔符是 NULL，返回 NULL。
- 如果多个数组参数中含有 NULL 参数，则返回空字符串 ###### 举例

将字符串通过 or 拼接到一起

```
SELECT CONCAT_WS("or", "d", "is"),CONCAT_WS(NULL, "d", "is"),CONCAT_WS('or', 'd', NULL, 'is')
```

```
+----------------------------+---------------------------+----------------------------------------+
    ↪
| concat_ws('or', 'd', 'is') | concat_ws(NULL, 'd', 'is') | concat_ws('or', 'd', NULL, 'is') |
+----------------------------+---------------------------+----------------------------------------+
    ↪
| doris                      | NULL                      | doris                                  |
```

```
+----------------------------+----------------------------+----------------------------------------+
    ↪
```

将 array 数组通过 or 拼接到一起

```
SELECT CONCAT_WS("or", ["d", "is"]),CONCAT_WS(NULL, ["d", "is"]),CONCAT_WS("or", ["d", NULL,"is"
    ↪ ])
```

```
+------------------------------+------------------------------+----------------------------------+
    ↪
| concat_ws('or', ['d', 'is']) | concat_ws(NULL, ['d', 'is']) | concat_ws('or', ['d', NULL, 'is
    ↪ ']) |
+------------------------------+------------------------------+----------------------------------+
    ↪
| doris                        | NULL                         | doris
    ↪                              |
+------------------------------+------------------------------+----------------------------------+
    ↪
```

拼接多个数组

```
mysql> SELECT CONCAT_WS("-", ["a", "b"], ["c", NULL], ["d"]);

+-----------------------------------------------+
| CONCAT_WS("-", ["a", "b"], ["c", NULL], ["d"]) |
+-----------------------------------------------+
| a-b-c-d                                       |
+-----------------------------------------------+
```

如果在多个数组参数中包含 NULL 参数，则返回空字符串;

```
mysql>  SELECT CONCAT_WS("-", ["a", "b"], null,["c", NULL], ["d"]);
+-----------------------------------------------------+
| CONCAT_WS("-", ["a", "b"], null,["c", NULL], ["d"]) |
+-----------------------------------------------------+
|                                                     |
+-----------------------------------------------------+
```

处理空字符串

```
mysql> SELECT CONCAT_WS("|", "hello", "", "world", NULL);

+-----------------------------------------+
| CONCAT_WS("|", "hello", "", "world", NULL) |
+-----------------------------------------+
| hello||world                            |
+-----------------------------------------+
```

混合字符串和数组（无效）

```
mysql> SELECT CONCAT_WS(",", "a", ["b", "c"]);

ERROR 1105 (HY000): errCode = 2, detailMessage = can not cast from origin type ARRAY<VARCHAR(1)>
    ↪ to target type=VARCHAR(65533)
```

全为 NULL 的输入

```
 mysql> SELECT CONCAT_WS("x", NULL, NULL);

+----------------------------+
| CONCAT_WS("x", NULL, NULL) |
+----------------------------+
|                            |
+----------------------------+
```

Chiese Charactors concat

```
mysql> SELECT CONCAT_WS("x", '中文', '中文');

+-----------------------------------+
| CONCAT_WS("x", '中文', '中文')     |
+-----------------------------------+
| 中文x中文                          |
+-----------------------------------+
```

中文字符拼接

```
mysql> SELECT CONCAT_WS("x", ['中文'], ['中文']);
+---------------------------------------+
| CONCAT_WS("x", ['中文'], ['中文'])     |
+---------------------------------------+
| 中文x中文                              |
+---------------------------------------+
```

插入数据并拼接

```
DROP TABLE IF EXISTS test_concat_ws_1;

CREATE TABLE test_concat_ws_1 (id INT, a ARRAY<VARCHAR>, b ARRAY<VARCHAR>) ENGINE=OLAP
    ↪ DISTRIBUTED BY HASH(id) BUCKETS 1 PROPERTIES ('replication_num' = '1')

INSERT INTO test_concat_ws_1 VALUES (1, ['a','b'], ['css',null,'d']), (2, ['x',null], ['y','z'])
    ↪ ,(3,['你好','世界'],['Doris',null,'Nereids'])

SELECT concat_ws('-', a, b) FROM test_concat_ws_1 ORDER BY id
```

```
+-----------------------------+
| concat_ws('-', a, b)        |
+-----------------------------+
| a-b-css-d                   |
| x-y-z                       |
| 你好-世界-Doris-Nereids     |
+-----------------------------+
```

7.2.1.2.8 CUT_TO_FIRST_SIGNIFICANT_SUBDOMAIN

描述

CUT_TO_FIRST_SIGNIFICANT_SUBDOMAIN 函数用于从 URL 中提取域名的有效部分，包含顶级域名直至 “第一个有效子域” 。如果输入的 URL 不合法，则返回空字符串。

语法

```
CUT_TO_FIRST_SIGNIFICANT_SUBDOMAIN(<url>)
```

参数

参数	说明
<url>	需要处理的 URL 字符串。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示提取出的域名部分。

特殊情况：- 如果 url 为 NULL，返回 NULL - 如果 url 不是有效的域名格式，返回空字符串

示例

1. 基本域名处理

```
SELECT cut_to_first_significant_subdomain('www.baidu.com');
```

```
+-----------------------------------------------------+
| cut_to_first_significant_subdomain('www.baidu.com')  |
+-----------------------------------------------------+
| baidu.com                                           |
+-----------------------------------------------------+
```

2. 多级域名处理

```
SELECT cut_to_first_significant_subdomain('www.google.com.cn');
```

```
+-------------------------------------------------------+
| cut_to_first_significant_subdomain('www.google.com.cn')   |
+-------------------------------------------------------+
| google.com.cn                                           |
+-------------------------------------------------------+
```

3. 无效域名处理

```
SELECT cut_to_first_significant_subdomain('wwwwwwww');
```

```
+-----------------------------------------------+
| cut_to_first_significant_subdomain('wwwwwwww')  |
+-----------------------------------------------+
|                                                 |
+-----------------------------------------------+
```

7.2.1.2.9 DIGITAL_MASKING

描述

digital_masking 函数用于对数字进行脱敏处理。根据指定的脱敏规则，将数字的部分字符替换为 * 。别名函数，原始函数为 concat(left(id,3),'****',right(id,4))。

语法

```
DIGITAL_MASKING( <digital_number> )
```

参数

参数	说明
<digital_number>	需要脱敏的数字字符串

返回值

返回脱敏后的数字字符串。

举例

```
select digital_masking(13812345678);
```

```
+------------------------------+
| digital_masking(13812345678) |
+------------------------------+
| 138****5678                  |
+------------------------------+
```

7.2.1.2.10 DOMAIN

描述

提取字符串 URL 中的域名

语法

```
DOMAIN ( <url> )
```

参数

参数	说明
<url>	需要提取域名的 URL

返回值

参数 <url> 的域名

举例

```
SELECT DOMAIN("https://doris.apache.org/docs/gettingStarted/what-is-apache-doris")
```

```
+-----------------------------------------------------------------------------+
| domain('https://doris.apache.org/docs/gettingStarted/what-is-apache-doris') |
+-----------------------------------------------------------------------------+
| doris.apache.org                                                            |
+-----------------------------------------------------------------------------+
```

7.2.1.2.11 DOMAIN_WITHOUT_WWW

描述

提取字符串 URL 中不带前缀 www 的域名

语法

```
DOMAIN_WITHOUT_WWW ( <url> )
```

参数

参数	说明
<url>	需要提取不带 www 域名的 URL

返回值

参数 <url> 不带前缀 www 的域名

举例

```
SELECT DOMAIN_WITHOUT_WWW("https://www.apache.org/docs/gettingStarted/what-is-apache-doris")
```

```
+-----------------------------------------------------------------------------------------+
| domain_without_www('https://www.apache.org/docs/gettingStarted/what-is-apache-doris') |
+-----------------------------------------------------------------------------------------+
| apache.org                                                                              |
+-----------------------------------------------------------------------------------------+
```

7.2.1.2.12 ELT

描述

在指定的索引处返回一个字符串。特殊情况：

- 如果指定的索引处没有字符串，则返回 NULL。

语法

```
ELT ( <pos> , <str> [ , <str> ] )
```

参数

参数	说明
<pos>	指定的索引值
<str>	需要指定索引的字符串

返回值

参数 <str> 指定索引值的字符串。特殊情况：

- 如果指定的索引处没有字符串，则返回 NULL。

举例

```
SELECT ELT(1, 'aaa', 'bbb'),ELT(2, 'aaa', 'bbb'), ELT(0, 'aaa', 'bbb'),ELT(2, 'aaa', 'bbb')
```

```
+----------------------+----------------------+----------------------+----------------------+
| elt(1, 'aaa', 'bbb') | elt(2, 'aaa', 'bbb') | elt(0, 'aaa', 'bbb') | elt(2, 'aaa', 'bbb') |
+----------------------+----------------------+----------------------+----------------------+
| aaa                  | bbb                  | NULL                 | bbb                  |
+----------------------+----------------------+----------------------+----------------------+
```

7.2.1.2.13 ENDS_WITH

描述

如果字符串以指定后缀结尾则返回 true，否则返回 false。特殊情况：

- 两个参数任意一个参数为 NULL，返回 NULL。

语法

```
ENDS_WITH ( <str> ,  <suffix> )
```

参数

参数	说明
str	指定需要判断的原始字符串
suffix	指定需要判断的结尾字符串

返回值

true 或者 false，类型为 BOOLEAN。特殊情况：

- 两个参数任意一个参数为 NULL，返回 NULL。

举例

```
SELECT ENDS_WITH("Hello doris", "doris"),ENDS_WITH("Hello doris", "Hello")
```

```
+-----------------------------------+-----------------------------------+
| ends_with('Hello doris', 'doris') | ends_with('Hello doris', 'Hello') |
+-----------------------------------+-----------------------------------+
|                                 1 |                                 0 |
+-----------------------------------+-----------------------------------+
```

7.2.1.2.14 EXTRACT_URL_PARAMETER

描述

返回 URL 中 name 参数的值（如果存在），否则为空字符串。

如果有许多具有此名称的参数，则返回第一个出现的参数。

此函数的工作假设参数名称在 URL 中的编码方式与在传递参数中的编码方式完全相同。

如果想获取 URL 中的其他部分，可以使用 parse_url

语法

```
EXTRACT_URL_PARAMETER ( <url> , <name> )
```

参数

参数	说明
<url>	需要返回参数的 url 字符串
<name>	需要返回的参数名称

返回值

参数 <name> 在 <url> 中的参数值

举例

```
SELECT EXTRACT_URL_PARAMETER ("http://doris.apache.org?k1=aa&k2=bb&test=cc#999", "k2")
```

```
+-----------------------------------------------------------------------------+
| extract_url_parameter('http://doris.apache.org?k1=aa&k2=bb&test=cc#999', 'k2') |
+-----------------------------------------------------------------------------+
| bb                                                                          |
+-----------------------------------------------------------------------------+
```

7.2.1.2.15 FIND_IN_SET

描述

返回 strlist 中第一次出现 str 的位置（从 1 开始计数）。

strlist 是用逗号分隔的字符串。特殊情况：

- 如果没有找到，返回 0。
- 任一参数为 NULL，返回 NULL。

语法

```
FIND_IN_SET ( <str> , <strlist> )
```

参数

参数	说明
<str>	需要查找的字符串
<strlist>	需要被查找的字符串

返回值

参数 <str> 在参数 <strlist> 第一次出现的位置。特殊情况：- 如果没有找到，返回 0。- 任一参数为 NULL，返回 NULL。

举例

```
SELECT FIND_IN_SET("b", "a,b,c")
```

```
| find_in_set('b', 'a,b,c') |
+---------------------------+
|                         2 |
+---------------------------+
```

7.2.1.2.16 FIRST_SIGNIFICANT_SUBDOMAIN

描述

在 URL 中提取出“第一个有效子域”返回，若不合法则会返回空字符串。

语法

```
FIRST_SIGNIFICANT_SUBDOMAIN ( <url> )
```

参数

参数	说明
<url>	需要提取“第一个有效子域”的 URL

返回值

<url> 中第一个有效子域。

举例

```
SELECT FIRST_SIGNIFICANT_SUBDOMAIN("www.baidu.com"),first_significant_subdomain("www.google.com.
    ↪ cn"),first_significant_subdomain("wwwwwwww")
```

```
+-----------------------------------------------+-------------------------------------------------------+-------------
    ↪
| first_significant_subdomain('www.baidu.com') | first_significant_subdomain('www.google.com.cn')
    ↪  | first_significant_subdomain('wwwwwwww') |
+-----------------------------------------------+-------------------------------------------------------+-------------
    ↪
| baidu                                         | google
    ↪                                             |                                                       |
+-----------------------------------------------+-------------------------------------------------------+-------------
    ↪
```

7.2.1.2.17 FROM_BASE64

描述

返回对输入的字符串进行 Base64 解码后的结果。特殊情况：

- 当输入字符串不正确时（出现非 Base64 编码后可能出现的字符串）将会返回 NULL

语法

```
FROM_BASE64 ( <str> )
```

参数

参数	说明
<str>	需要被 Base64 解码的字符串

返回值

参数通过 Base64 解码后的结果。特殊情况：

- 当输入字符串不正确时（出现非 Base64 编码后可能出现的字符串）将会返回 NULL

举例

```
SELECT FROM_BASE64('MQ=='),FROM_BASE64('MjMO'),FROM_BASE64(NULL)
```

```
+---------------------+---------------------+-------------------+
| from_base64('MQ==') | from_base64('MjMO') | from_base64(NULL) |
+---------------------+---------------------+-------------------+
| 1                   | 234                 | NULL              |
+---------------------+---------------------+-------------------+
```

7.2.1.2.18 HEX

描述

如果输入参数是数字，返回十六进制值的字符串表示形式。

如果输入参数是字符串，则将每个字符转化为两个十六进制的字符，将转化后的所有字符拼接为字符串输出。

语法

```
HEX ( <str> )
```

参数

参数	说明
<str>	输入参数是数字或者字符串

返回值

参数 <str> 的十六进制结果。

举例

输入参数是数字

```
SELECT HEX(12),HEX(-1)
```

```
+---------+------------------+
| hex(12) | hex(-1)          |
```

```
+---------+------------------+
| C       | FFFFFFFFFFFFFFFF |
+---------+------------------+
```

输入参数是字符串

```
SELECT HEX('1'),HEX('@'),HEX('12')
```

```
+----------+----------+-----------+
| hex('1') | hex('@') | hex('12') |
+----------+----------+-----------+
| 31       | 40       | 3132      |
+----------+----------+-----------+
```

7.2.1.2.19 INITCAP

描述

将参数中包含的单词首字母大写，其余字母转为小写。单词是由非字母数字字符分隔的字母数字字符序列。

语法

```
INITCAP ( <str> )
```

参数

参数	说明
<str>	需要被转化的字符串

返回值

参数 <str> 中单词首字母大写，其余字母小写的结果。

举例

```
SELECT INITCAP('hello hello.,HELLO123HELlo')
```

```
+---------------------------------------+
| initcap('hello hello.,HELLO123HELlo') |
+---------------------------------------+
| Hello Hello.,hello123hello            |
+---------------------------------------+
```

7.2.1.2.20 INSTR

描述

返回 substr 在 str 中第一次出现的位置（从 1 开始计数）。特殊情况：

- 如果 substr 不在 str 中出现，则返回 0。

语法

```
INSTR ( <str> , <substr> )
```

参数

参数	说明
<str>	需要查找的字符串
<substr>	需要被查找的字符串

返回值

参数 <substr> 在 <str> 中第一次出现的位置（从 1 开始计数）。特殊情况：

- 如果 <substr> 不在 <str> 中出现，则返回 0。

举例

```
SELECT INSTR("abc", "b"),INSTR("abc", "d")
```

```
+------------------+------------------+
| instr('abc', 'b') | instr('abc', 'd') |
+------------------+------------------+
|                2 |                0 |
+------------------+------------------+
```

7.2.1.2.21 INT_TO_UUID

描述

对于输入的已编码 LARGEINT，转译为原始的 uuid 字符串。

语法

```
INT_TO_UUID ( <int128> )
```

参数

参数	说明
<int128>	已编码的 LARGEINT 值

返回值

参数 <int128> 原始的 uuid 字符串。

举例

```
SELECT INT_TO_UUID(957219555148694080917592900713939528 76)
```

```
+-----------------------------------------------------+
| int_to_uuid(957219555148694080917592900713939528 76) |
+-----------------------------------------------------+
| 6ce4766f-6783-4b30-b357-bba1c7600348                |
+-----------------------------------------------------+
```

7.2.1.2.22 LCASE/LOWER

描述

将参数中所有的字符串都转换成小写。

别名

- LOWER

语法

```
LCASE ( <str> )
LOWER ( <str> )
```

参数

参数	说明
<str>	需要被转换成小写的字符串

返回值

参数 <str> 转换成小写的字符串

举例

```
SELECT LOWER("AbC123"),LCASE("AbC123")
```

```
+-----------------+-----------------+
| lower('AbC123') | lower('AbC123') |
+-----------------+-----------------+
| abc123          | abc123          |
+-----------------+-----------------+
```

```
SELECT LOWER("ҚAZAҚ123"),LCASE("ҒAРЫШ");
```

```
+---------------------+---------------------+
```

```
| LOWER("ҚAZAҚ123")   | LCASE("FAРЫШ")      |
+---------------------+---------------------+
| қazaқ123            | ғарыш               |
+---------------------+---------------------+
```

7.2.1.2.23 LENGTH

描述

返回字符串的字节数。

语法

```
LENGTH ( <str> )
```

参数

参数	说明
<str>	需要计算字节的字符串

返回值

字符串 <str> 的字节数。

举例

```
SELECT LENGTH("abc"),length("中国")
```

```
+---------------+------------------+
| length('abc') | length('中国')   |
+---------------+------------------+
|             3 |                6 |
+---------------+------------------+
```

7.2.1.2.24 LOCATE

描述

返回 substr 在 str 中出现的位置（从 1 开始计数）。如果指定第 3 个参数 pos，则从 str 以 pos 下标开始的字符串处开始查找 substr 出现的位置。如果没有找到，返回 0

语法

```
LOCATE ( <substr> , <str> [, <pos> ] )
```

参数

参数	说明
substr	需要查找的子字符串
str	需要被查找的字符串
pos	如果指定了此参数，则 str 以 pos 下标开始的字符串处开始查找 substr 出现的位置

返回值

substr 在 str 中出现的位置（从 1 开始计数）

举例

```
SELECT locate('bar', 'foobarbar'),locate('xbar', 'foobar'),locate('bar', 'foobarbar', 5)
```

```
+----------------------------+--------------------------+------------------------------+
| locate('bar', 'foobarbar') | locate('xbar', 'foobar') | locate('bar', 'foobarbar', 5) |
+----------------------------+--------------------------+------------------------------+
|                          4 |                        0 |                            7 |
+----------------------------+--------------------------+------------------------------+
```

7.2.1.2.25 LPAD

描述

返回 str 中长度为 len（从首字母开始算起）的字符串。

如果 len 大于 str 的长度，则在 str 的前面不断补充 pad 字符，直到该字符串的长度达到 len 为止。

如果 len 小于 str 的长度，该函数相当于截断 str 字符串，只返回长度为 len 的字符串。len 指的是字符长度而不是字节长度。

特殊情况：

- 除包含 NULL 值外，如果 pad 为空，则返回值为空串。

语法

```
LPAD ( <str> , <len> , <pad>)
```

参数

参数	说明
<str>	需要被补充的字符串
<len>	需要填充的字符串的长度，指的是字符长度而不是字节长度
<pad>	需要在原始字符串左边补充的字符串

返回值

填充后的字符串。特殊情况：

- 除包含 NULL 值外，如果 pad 为空，则返回值为空串。

举例

```
SELECT LPAD("hi", 5, "xy"),LPAD("hi", 1, "xy"),LPAD("", 0, "")
```

```
+---------------------+---------------------+-----------------+
| lpad('hi', 5, 'xy') | lpad('hi', 1, 'xy') | lpad('', 0, '') |
+---------------------+---------------------+-----------------+
| xyxhi               | h                   |                 |
+---------------------+---------------------+-----------------+
```

7.2.1.2.26 LTRIM

描述

LTRIM 函数用于去除字符串左侧（开头部分）连续出现空格或指定字符。

语法

```
LTRIM( <str> [, <trim_chars> ] )
```

参数

参数	说明
<str>	需要被修剪的字符串
<trim_chars>	可选参数。如果提供了该参数，LTRIM 函数将去除<str>左侧出现的<trim_chars>中的所有字符。如果未提供此参数，LTRIM 函数将仅去除<str>左侧的空格字符。

返回值

返回<str>左侧去除<trim_chars>后的字符串。特殊情况：

- 任意两个参数中有一个为 NULL，则返回 NULL

举例

```
SELECT ltrim('   ab d') str;
```

```
+------+
| str  |
+------+
| ab d |
+------+
```

```
SELECT ltrim('ababccaab','ab') str;
```

```
+-------+
| str   |
+-------+
| ccaab |
+-------+
```

7.2.1.2.27 LTRIM_IN

描述

LTRIM_IN 函数用于移除字符串左侧的指定字符。当不指定移除字符集合时，默认移除左侧的空格；当指定字符集合时，将移除左侧出现的所有指定字符（不考虑字符集合顺序）。LTRIM_IN 的特点是会移除指定字符集合中的任意字符组合，而 LTRIM 函数则是按照完整的字符串匹配进行移除。

语法

```
LTRIM_IN(<str>[, <rhs>])
```

参数

参数	说明
<str>	需要处理的字符串。类型：VARCHAR
<rhs>	可选参数，要移除的字符集合。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示处理后的字符串。

特殊情况：- 如果 str 为 NULL，返回 NULL - 如果不指定 rhs，移除左侧所有空格 - 如果指定 rhs，移除左侧出现在 rhs 中的所有字符，直到遇到第一个不在 rhs 中的字符

示例

1. 移除左侧空格

```
SELECT ltrim_in('   ab d') str;
```

```
+------+
| str  |
+------+
| ab d |
+------+
```

2. 移除指定字符集合

```
SELECT ltrim_in('ababccaab', 'ab') str;
```

```
+-------+
| str   |
+-------+
| ccaab |
+-------+
```

3. 与 LTRIM 函数的对比

```
SELECT ltrim_in('abcd', 'ae'),ltrim('abcd', 'abe');
```

```
+------------------------+----------------------+
| ltrim_in('abcd', 'ae') | ltrim('abcd', 'abe') |
+------------------------+----------------------+
| bcd                    | abcd                 |
+------------------------+----------------------+
```

7.2.1.2.28 MASK

描述

MASK 函数主要作用是对数据进行屏蔽，以保护敏感信息，常用于数据脱敏场景。其行为默认是将输入的字符串中的大写字母转换为X，小写字母转换为x，数字转换为n。

语法

```
MASK(<str> [, <upper> [, <lower> [, <number> ]]])
```

参数

参数	说明
<str>	需要被脱敏的数据
<upper>	可选参数，替换大写字母字符，默认是X。如果输入多个字符串，则会取第一个字符，如果输入的是非 ASCII 字符，
<lower>	可选参数，替换小写字母字符，默认是x。如果输入多个字符串，则会取第一个字符，如果输入的是非 ASCII 字符，
<number>	可选参数，替换数字字符，默认是n。如果输入多个字符串，则会取第一个字符，如果输入的是非 ASCII 字符，则会

返回值

返回字母和数字被替换后的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 非字母和数字会原样返回

举例

```
select mask('abc123EFG');
```

```
+-------------------+
| mask('abc123EFG') |
+-------------------+
| xxxnnnXXX         |
+-------------------+
```

```
select mask(null);
```

```
+------------+
| mask(NULL) |
+------------+
| NULL       |
+------------+
```

```
select mask('abc123EFG', '*', '#', '$');
```

```
+----------------------------------+
| mask('abc123EFG', '*', '#', '$') |
+----------------------------------+
| ###$$$***                        |
+----------------------------------+
```

7.2.1.2.29 MASK_FIRST_N

描述

MASK_FIRST_N 函数主要作用是对数据的前 N 个字节进行屏蔽，以保护敏感信息，常用于数据脱敏场景。其行为是将前 N 个字节中的大写字母替换为X，小写字母替换为x，数字替换为n。

语法

```
MASK_FIRST_N( <str> [, <n> ])
```

参数

参数	说明
<str>	需要被脱敏的数据
<n>	可选参数，限制只让前 N 个字节进行数据屏蔽，默认是对整个字符串进行数据屏蔽

返回值

返回前 N 个字节中，字母和数字被替换后的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 非字母和数字会原样返回

举例

```
select mask_first_n("1234-5678-8765-4321", 4);
```

```
+----------------------------------------+
| mask_first_n('1234-5678-8765-4321', 4) |
+----------------------------------------+
| nnnn-5678-8765-4321                    |
+----------------------------------------+
```

```
select mask_first_n("1234-5678-8765-4321", null);
```

```
+-----------------------------------------+
| mask_first_n('1234-5678-8765-4321', NULL) |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

7.2.1.2.30 MASK_LAST_N

描述

MASK_LAST_N 函数主要作用是对数据的后 N 个字节进行屏蔽，以保护敏感信息，常用于数据脱敏场景。其行为是将后 N 个字节中的大写字母替换为X，小写字母替换为x，数字替换为n。

语法

```
MASK_LAST_N( <str> [, <n> ])
```

参数

参数	说明
<str>	需要被脱敏的数据
<n>	可选参数，限制只让后 N 个字节进行数据屏蔽，默认是对整个字符串进行数据屏蔽

返回值

返回后 N 个字节中，字母和数字被替换后的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 非字母和数字会原样返回

举例

```
select mask_last_n("1234-5678-8765-4321", 4);
```

```
+--------------------------------------+
| mask_last_n('1234-5678-8765-4321', 4) |
+--------------------------------------+
| 1234-5678-8765-nnnn                  |
+--------------------------------------+
```

```
select mask_last_n("1234-5678-8765-4321", null);
```

```
+-----------------------------------------+
| mask_last_n('1234-5678-8765-4321', NULL) |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

7.2.1.2.31 MULTI_MATCH_ANY

multi_match_any

描述

语法

```
TINYINT multi_match_any(VARCHAR haystack, ARRAY<VARCHAR> patterns)
```

检查字符串 haystack 是否与 re2 语法中的正则表达式 patterns 相匹配。如果都没有匹配的正则表达式返回 0，否则返回 1。

举例

```
mysql> select multi_match_any('Hello, World!', ['hello', '!', 'world']);
+-----------------------------------------------------------+
| multi_match_any('Hello, World!', ['hello', '!', 'world']) |
+-----------------------------------------------------------+
| 1                                                         |
+-----------------------------------------------------------+

mysql> select multi_match_any('abc', ['A', 'bcd']);
+--------------------------------------+
| multi_match_any('abc', ['A', 'bcd']) |
+--------------------------------------+
| 0                                    |
+--------------------------------------+
```

keywords

```
MULTI_MATCH,MATCH,ANY
```

7.2.1.2.32 MULTI_SEARCH_ALL_POSITIONS

multi_search_all_positions

描述

语法

```
ARRAY<INT> multi_search_all_positions(VARCHAR haystack, ARRAY<VARCHAR> needles)
```

返回一个 ARRAY，其中第 i 个元素为 needles 中第 i 个元素 needle，在字符串 haystack 中首次出现的位置。位置从 1 开始计数，0 代表未找到该元素。大小写敏感。

举例

```
mysql> select multi_search_all_positions('Hello, World!', ['hello', '!', 'world']);
+---------------------------------------------------------------------+
| multi_search_all_positions('Hello, World!', ['hello', '!', 'world']) |
+---------------------------------------------------------------------+
| [0,13,0]                                                            |
+---------------------------------------------------------------------+

select multi_search_all_positions("Hello, World!", ['hello', '!', 'world', 'Hello', 'World']);
+-----------------------------------------------------------------------------------------------+
| multi_search_all_positions('Hello, World!', ARRAY('hello', '!', 'world', 'Hello', 'World')) |
+-----------------------------------------------------------------------------------------------+
| [0, 13, 0, 1, 8]                                                                              |
+-----------------------------------------------------------------------------------------------+
```

keywords

```
MULTI_SEARCH,SEARCH,POSITIONS
```

7.2.1.2.33 NGRAM_SEARCH

Description

```
DOUBLE ngram_search(VARCHAR text,VARCHAR pattern,INT gram_num)
```

计算 text 和 pattern 的 N-gram 相似度。相似度从 0 到 1，相似度越高证明两个字符串越相似。其中pattern，gram_num必须为常量。如果text或者pattern的长度小于gram_num，返回 0。

N-gram 相似度（N-gram similarity）是一种基于 N-gram（N 元语法）的文本相似度计算方法。N-gram 是指将一个文本串分成连续的 N 个字符或词语的集合。例如，对于字符串 “text”，当 N=2 时，其二元组（bi-gram）为：{“te”，“ex”，“xt”}。

N-gram 相似度的计算为 2 * |Intersection| / (|text set| + |pattern set|)

其中 |text set|， |pattern set| 为 text 和 pattern 的 N-gram，Intersection为两个集合的交集。

注意，根据定义，相似度为 1 不代表两个字符串相同。

仅支持 ASCII 编码。

Syntax

```
DOUBLE ngram_search(VARCHAR text,VARCHAR pattern,INT gram_num)
```

Example

```
mysql> select ngram_search('123456789' , '12345' , 3);
+----------------------------------------+
| ngram_search('123456789', '12345', 3) |
+----------------------------------------+
|                                    0.6 |
+----------------------------------------+

mysql> select ngram_search("abababab","bababababa",2);
+-----------------------------------------+
| ngram_search('abababab', 'bababababa', 2) |
+-----------------------------------------+
|                                       1 |
+-----------------------------------------+
```

keywords

```
NGRAM_SEARCH,NGRAM,SEARCH
```

7.2.1.2.34 OVERLAY

描述

OVERLAY 函数用于将一个字符串替换另一个字符串中的一部分。

语法

```
OVERLAY(<str>, <pos>, <len>, <newstr>)
```

参数

参数	说明
<str>	需要被替换的字符串
<pos>	需要被替换的字符串的起始位置，从 1 开始计数，如果输入的位置不在<str>的长度范围内，则不会进行替换
<len>	需要被替换的长度，当<len>小于 0 或超过字符串其余部分的长度范围时，则会从<pos>开始替换掉其余字符串
<newstr>	用于替换的新字符串

返回值

返回从指定位置、用新字符串替换指定长度后的字符。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- <pos>如果不在<str>的长度范围内，则不会进行替换

- 当<len>小于 0 或超过字符串其余部分的长度范围时，则会从<pos>开始替换掉其余字符串

举例

```
select overlay('Quadratic', 3, 4, 'What');
```

```
+-----------------------------------+
| overlay('Quadratic', 3, 4, 'What') |
+-----------------------------------+
| QuWhattic                         |
+-----------------------------------+
```

```
select overlay('Quadratic', null, 4, 'What');
```

```
+--------------------------------------+
| overlay('Quadratic', NULL, 4, 'What') |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

```
select overlay('Quadratic', -1, 4, 'What');
```

```
+-----------------------------------+
| overlay('Quadratic', -1, 4, 'What') |
+-----------------------------------+
| Quadratic                         |
+-----------------------------------+
```

```
select overlay('Quadratic', 2, -4, 'What');
```

```
+-----------------------------------+
| overlay('Quadratic', 2, -4, 'What') |
+-----------------------------------+
| QWhat                             |
+-----------------------------------+
```

7.2.1.2.35 PARSE_URL

描述

PARSE_URL 函数主要用于解析 URL 字符串，并从中提取各种组成部分，如协议、主机、路径、查询参数等。

语法

```
PARSE_URL( <url>, <name> )
```

参数

参数	说明
<url>	需要被解析的 URL
<name>	需要提取的部分，可选的值有PROTOCOL，HOST，PATH，REF，AUTHORITY，FILE，USERINFO，PORT，QUERY（不区分大小写）

返回值

返回<url>指定的部分。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- <name>传入其他非法值，则会报错

举例

```
SELECT parse_url ('https://doris.apache.org/', 'HOST');
```

```
+-----------------------------------------------+
| parse_url('https://doris.apache.org/', 'HOST') |
+-----------------------------------------------+
| doris.apache.org                              |
+-----------------------------------------------+
```

```
SELECT parse_url ('https://doris.apache.org/', null);
```

```
+--------------------------------------------+
| parse_url('https://doris.apache.org/', NULL) |
+--------------------------------------------+
| NULL                                       |
+--------------------------------------------+
```

相关命令

如果想获取 QUERY 中的特定参数，可使用 extract_url_parameter。

7.2.1.2.36 PRINTF

描述

使用指定的 printf 格式字符串和参数返回格式化后的字符串。

> 该函数自 3.0.6 版本开始支持.

语法

```
PRINTF(<format>, [<args>, ...])
```

参数

参数	说明
<format>	printf 格式字符串。
<args>	要格式化的参数。

返回值

使用 printf 模式格式化后的字符串。

示例

```
select printf("hello world");
```

```
+-----------------------+
| printf("hello world") |
+-----------------------+
| hello world           |
+-----------------------+
```

```
select printf('%d-%s-%.2f', 100, 'test', 3.14);
```

```
+---------------------------------------+
| printf('%d-%s-%.2f', 100, 'test', 3.14) |
+---------------------------------------+
| 100-test-3.14                         |
+---------------------------------------+
```

```
select printf('Int: %d, Str: %s, Float: %.2f, Hex: %x', 255, 'test', 3.14159, 255);
```

```
+-------------------------------------------------------------------------------+
| printf('Int: %d, Str: %s, Float: %.2f, Hex: %x', 255, 'test', 3.14159, 255) |
+-------------------------------------------------------------------------------+
| Int: 255, Str: test, Float: 3.14, Hex: ff                                     |
+-------------------------------------------------------------------------------+
```

7.2.1.2.37 PROTOCOL

描述

PROTOCOL 函数主要用于提取 URL 字符串中的协议部分。

语法

```
PROTOCOL( <url> )
```

参数

参数	说明
<url>	需要被解析的 URL

返回值

返回<url>中的协议部分。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL

举例

```
SELECT protocol('https://doris.apache.org/');
```

```
+---------------------------------------+
| protocol('https://doris.apache.org/') |
+---------------------------------------+
| https                                 |
+---------------------------------------+
```

```
SELECT protocol(null);
```

```
+----------------+
| protocol(NULL) |
+----------------+
| NULL           |
+----------------+
```

相关命令

如果想提取 URL 中的其他部分，可使用 parse_url。

7.2.1.2.38 QUOTE

quote

描述

语法

```
VARCHAR quote(VARCHAR str)
```

将参数中所有的字符串按原样输出，并用” 套起来

举例

```
mysql> select quote('hello world!\`\t`');
+--------------------------+
| quote('hello world!`\t`') |
+--------------------------+
| 'hello world!`\t`'        |
+--------------------------+
```

keywords

```
QUOTE
```

7.2.1.2.39 REGEXP

描述

对字符串 str 执行正则表达式匹配，匹配成功时返回 true，否则返回 false。pattern 为正则表达式模式。需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP(<str>, <pattern>)
```

参数

参数	描述
<str>	字符串类型。表示要执行正则表达式匹配的字符串，可以是表中的列或字面值字符串。
<pattern>	字符串类型。用于与字符串匹配的正则表达式模式。正则表达式提供了定义复杂搜索模式的强大方式，包括字符

返回值

REGEXP 函数返回布尔值（BOOLEAN）。如果字符串匹配正则表达式模式，函数返回 true（在 SQL 中表示为 1）；如果不匹配，返回 false（在 SQL 中表示为 0）。

例子

```
CREATE TABLE test ( k1 VARCHAR(255) ) properties("replication_num"="1")

INSERT INTO test (k1) VALUES ('billie eillish'), ('It\'s ok'), ('billie jean'), ('hello world');
```

```
--- 查找k1字段中以'billie'开头的所有数据
SELECT k1 FROM test WHERE k1 REGEXP '^billie'
--------------

+----------------+
| k1             |
+----------------+
| billie eillish |
| billie jean    |
+----------------+
2 rows in set (0.02 sec)
```

```
--- 查找k1字段中以'ok'结尾的数据:
SELECT k1 FROM test WHERE k1 REGEXP 'ok$'
--------------

+---------+
| k1      |
+---------+
| It's ok |
+---------+
1 row in set (0.03 sec)
```

中文测试

```
mysql> select regexp('这是一段中文 This is a passage in English 1234567', '\\p{Han}');
+-------------------------------------------------------------------------------+
| ('这是一段中文 This is a passage in English 1234567' regexp '\p{Han}')        |
+-------------------------------------------------------------------------------+
|                                                                             1 |
+-------------------------------------------------------------------------------+
```

插入然后进行简单的变量字符串匹配

```
CREATE TABLE test_regexp (
    id INT,
    name VARCHAR(255)
) PROPERTIES("replication_num"="1");

INSERT INTO test_regexp (id, name) VALUES
    (1, 'Alice'),
    (2, 'Bob'),
    (3, 'Charlie'),
    (4, 'David');

--查找以'A'开头的名字
SELECT id, name FROM test_regexp WHERE name REGEXP '^A';
```

```
+------+-------+
| id   | name  |
+------+-------+
|    1 | Alice |
+------+-------+
```

特殊字符匹配

```
-- 插入具有特殊字符的名字
INSERT INTO test_regexp (id, name) VALUES
    (5, 'Anna-Maria'),
```

```
    (6, 'John_Doe');

-- 查找包含'-'字符的名字
SELECT id, name FROM test_regexp WHERE name REGEXP '-';
```

```
+------+------------+
| id   | name       |
+------+------------+
|    5 | Anna-Maria |
+------+------------+
```

结尾字符匹配

```
-- Find names ending with 'e'
SELECT id, name FROM test_regexp WHERE name REGEXP 'e$';
```

```
+------+---------+
| id   | name    |
+------+---------+
|    1 | Alice   |
|    3 | Charlie |
+------+---------+
```

emoji 字符匹配

```
SELECT 'Hello' REGEXP '□';
```

```
+-----------------------+
| 'Hello' REGEXP '□'    |
+-----------------------+
|                     0 |
+-----------------------+
```

‘str’ 是 NULL 值，则返回 NULL 值

```
mysql> SELECT REGEXP(NULL, '^billie');
+-------------------------+
| REGEXP(NULL, '^billie') |
+-------------------------+
|                    NULL |
+-------------------------+
```

‘pattern’ 是 NULL 值，则返回 NULL 值

```
mysql> SELECT REGEXP('billie eillish', NULL);
+--------------------------------+
| REGEXP('billie eillish', NULL) |
```

```
+--------------------------------+
|                           NULL |
+--------------------------------+
```

所有参数都是 NULL 值，则返回 NULL 值

```
mysql> SELECT REGEXP(NULL, NULL);
+--------------------+
| REGEXP(NULL, NULL) |
+--------------------+
|               NULL |
+--------------------+
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
SELECT REGEXP('Hello, World!', '([a-z');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INTERNAL_ERROR]Invalid regex
    ↪ expression: ([a-z
```

7.2.1.2.40 REGEXP_EXTRACT

描述

此函数用于对给定字符串 STR 执行正则匹配，并提取符合指定模式的第 POS 个匹配部分。若函数要返回匹配结果，该模式必须与 STR 的某些部分完全匹配。

若未找到匹配项，将返回空字符串。需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

str 参数为’string’ 类型，表示要进行正则匹配的字符串。pattern 参数为’string’ 类型，表示目标正则表达式模式。pos 参数为 ‘integer’ 类型，用于指定字符串中开始搜索正则表达式匹配的位置。位置从 1 开始，此参数必须指定。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP_EXTRACT(<str>, <pattern>, <pos>)
```

参数

参数	描述
<str>	需要进行正则匹配的列，类型为’string’。
<pattern>	目标正则表达式模式，类型为’string’。
<pos>	用于指定字符串中开始搜索正则表达式匹配位置的参数，为整数值，表示字符串中的字符位置（从 1 开始）。pos

返回值

模式的匹配部分，类型为 Varchar。若未找到匹配项，将返回空字符串

示例

提取第一个匹配部分, 在此示例中，正则表达式 ([[:lower:]]+)C([[:lower:]]+) 匹配字符串中一个或多个小写字母后跟 ‘C’ 再跟一个或多个小写字母的部分。 ‘C’ 之前的第一个捕获组 ([[:lower:]]+) 匹配 ‘b’，因此结果为 ‘b’

```
mysql> SELECT regexp_extract('AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 1);
+-----------------------------------------------------------------+
| regexp_extract('AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 1) |
+-----------------------------------------------------------------+
| b                                                               |
+-----------------------------------------------------------------+
```

提取第二个匹配部分, 这里， ‘C’ 之后的第二个捕获组 ([[:lower:]]+) 匹配 ‘d’，因此结果为 ‘d’。

```
mysql> SELECT regexp_extract('AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 2);
+-----------------------------------------------------------------+
| regexp_extract('AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 2) |
+-----------------------------------------------------------------+
| d                                                               |
+-----------------------------------------------------------------+
```

匹配中文字符, 模式 (\p{Han}+)(.+) 首先匹配一个或多个中文字符 (\p{Han}+)，然后匹配字符串的剩余部分 ((.+))。第二个捕获组匹配字符串的非中文部分，因此结果为 ‘This is a passage in English 1234567’。

```
mysql> select regexp_extract('这是一段中文 This is a passage in English 1234567', '(\\p{Han}+)
    ↪ (.+)', 2);
+-----------------------------------------------------------------------------------------------------+
| regexp_extract('这是一段中文 This is a passage in English 1234567', '(\p{Han}+)(.+)', 2)
    ↪ |
+-----------------------------------------------------------------------------------------------------+
| This is a passage in English 1234567                                                                |
+-----------------------------------------------------------------------------------------------------+
```

插入变量值并执行匹配, 此示例向表中插入数据，然后使用 REGEXP_EXTRACT 函数根据存储的模式和位置从存储的字符串中提取匹配部分。

```
CREATE TABLE test_table_for_regexp_extract (
        id INT,
        text_data VARCHAR(500),
        pattern VARCHAR(100),
        pos INT
    ) PROPERTIES ("replication_num"="1");

INSERT INTO test_table_for_regexp_extract VALUES
```

```
    (1, 'AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 1),
    (2, 'AbCdE', '([[:lower:]]+)C([[:lower:]]+)', 2),
    (3, '这是一段中文 This is a passage in English 1234567', '(\\p{Han}+)(.+)', 2);

SELECT id, regexp_extract(text_data, pattern, pos) as extract_result FROM test_table_for_regexp_
    ↪ extract ORDER BY id;
```

```
+------+----------------+
| id   | extract_result |
+------+----------------+
|    1 | b              |
|    2 | d              |
|    3 | This is a passage in English 1234567 |
+------+----------------+
```

无匹配的模式, 由于模式 ([[:digit:]]+) (一个或多个数字) 与字符串 'AbCdE' 的任何部分都不匹配，因此返回空字符串

```
SELECT regexp_extract('AbCdE', '([[:digit:]]+)', 1);
```

```
+-----------------------------------------------+
| regexp_extract('AbCdE', '([[:digit:]]+)', 1)  |
+-----------------------------------------------+
|                                               |
+-----------------------------------------------+
```

emoji 字符匹配

```
SELECT regexp_extract('Text 😊 More 😂', '😊|😂',0);
```

```
+-----------------------------------------------------+
| regexp_extract('Text 😊 More 😂', '😊|😂',0)                |
+-----------------------------------------------------+
| 😊                                                  |
+-----------------------------------------------------+
1 row in set (0.02 sec)
```

'str' 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_EXTRACT(NULL, '([a-z]+)', 1);
+--------------------------------------+
| REGEXP_EXTRACT(NULL, '([a-z]+)', 1) |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

'pattern' 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_EXTRACT('Hello World', NULL, 1);
+---------------------------------------+
| REGEXP_EXTRACT('Hello World', NULL, 1) |
+---------------------------------------+
| NULL                                  |
+---------------------------------------+
```

‘pos’ 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_EXTRACT('Hello World', '([a-z]+)', NULL);
+-------------------------------------------------+
| REGEXP_EXTRACT('Hello World', '([a-z]+)', NULL) |
+-------------------------------------------------+
| NULL                                            |
+-------------------------------------------------+
```

全部参数是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_EXTRACT(NULL, NULL, NULL);
+----------------------------------+
| REGEXP_EXTRACT(NULL, NULL, NULL) |
+----------------------------------+
| NULL                             |
+----------------------------------+
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
SELECT regexp_extract('AbCdE', '([[:digit:]]+', 1);
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INVALID_ARGUMENT]Could not compile
    ↪ regexp pattern: ([[:digit:]]+
Error: missing ): ([[:digit:]]+
```

7.2.1.2.41 REGEXP_EXTRACT_ALL

描述

REGEXP_EXTRACT_ALL 函数用于对给定字符串 str 执行正则表达式匹配，所有与指定 pattern 匹配的文本串当中的与第一个子模式匹配的部分。为了使函数返回表示模式匹配部分的字符串数组，该模式必须与输入字符串 str 的一部分完全匹配。如果没有匹配项，或模式不包含任何子模式，则返回空字符串。

需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP_EXTRACT_ALL(<str>, <pattern>)
```

参数

参数	描述
<str>	该参数为 String 类型。表示要执行正则表达式匹配的输入字符串。可以是字面值字符串或包含字符串数据的表列
<pattern>	该参数也为 String 类型。指定用于与输入字符串匹配的正则表达式模式。该模式可以包含各种正则表达式构造，

返回值

函数返回表示输入字符串中与指定正则表达式的第一个子模式匹配部分的字符串数组。返回类型为 String 值数组。如果未找到匹配项，或模式没有子模式，则返回空数组。

例子

围绕 ‘C’ 的小写字母基本匹配, 在这个示例中，模式 ([[:lower:]]+)C([[:lower:]]+) 匹配字符串中一个或多个小写字母后跟 ‘C’ 再跟一个或多个小写字母的部分。 ‘C’ 之前的第一个子模式 ([[:lower:]]+) 匹配 ‘b’，因此结果为 [‘b’]。

```
mysql> SELECT regexp_extract_all('AbCdE', '([[:lower:]]+)C([[:lower:]]+)');
+-----------------------------------------------------------------+
| regexp_extract_all('AbCdE', '([[:lower:]]+)C([[:lower:]]+)') |
+-----------------------------------------------------------------+
| ['b']                                                           |
+-----------------------------------------------------------------+
```

字符串中的多个匹配项, 在这里，模式在字符串中匹配两个部分。第一个匹配的第一个子模式匹配 ‘b’，第二个匹配的第一个子模式匹配 ‘f’。因此结果为 [‘b’, ‘f’]。

```
mysql> SELECT regexp_extract_all('AbCdEfCg', '([[:lower:]]+)C([[:lower:]]+)');
+-------------------------------------------------------------------+
| regexp_extract_all('AbCdEfCg', '([[:lower:]]+)C([[:lower:]]+)') |
+-------------------------------------------------------------------+
| ['b','f']                                                         |
+-------------------------------------------------------------------+
```

从键值对中提取键, 该模式匹配字符串中的键值对。第一个子模式捕获键，因此结果为键的数组 [‘abc’, ‘def’, ‘ghi’]。

```
mysql> SELECT regexp_extract_all('abc=111, def=222, ghi=333','("[^"]+"|\\w+)=("[^"]+"|\\w+)');
+-----------------------------------------------------------------------------------+
| regexp_extract_all('abc=111, def=222, ghi=333', '("[^"]+"|\w+)=("[^"]+"|\w+)') |
+-----------------------------------------------------------------------------------+
| ['abc','def','ghi']                                                               |
+-----------------------------------------------------------------------------------+
```

匹配汉字, 模式 (\p{Han}+)(.+) 首先通过第一个子模式 (\p{Han}+) 匹配一个或多个汉字，因此结果为 [‘这是一段中文’]。

```
mysql> select regexp_extract_all('这是一段中文 This is a passage in English 1234567', '(\\p{Han
    ↪ }+)(.+)');
+--
    ↪ ----------------------------------------------------------------------------------------+
    ↪
| regexp_extract_all('这是一段中文 This is a passage in English 1234567', '(\\p{Han}+)(.+)')
    ↪       |
+--
    ↪ ----------------------------------------------------------------------------------------+
    ↪
| ['这是一段中文']
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------+
    ↪
```

插入数据并使用 REGEXP_EXTRACT_ALL

```
CREATE TABLE test_regexp_extract_all (
    id INT,
    text_content VARCHAR(255),
    pattern VARCHAR(255)
) PROPERTIES ("replication_num"="1");


INSERT INTO test_regexp_extract_all VALUES
(1, 'apple1, banana2, cherry3', '([a-zA-Z]+)\\d'),
(2, 'red#123, blue#456, green#789', '([a-zA-Z]+)#\\d+'),
(3, 'hello@example.com, world@test.net', '([a-zA-Z]+)@');


SELECT id, regexp_extract_all(text_content, pattern) AS extracted_data
FROM test_regexp_extract_all;
```

```
+------+----------------------+
| id   | extracted_data       |
+------+----------------------+
|    1 | ['apple', 'banana', 'cherry'] |
|    2 | ['red', 'blue', 'green']      |
|    3 | ['hello', 'world']            |
+------+----------------------+
```

没有匹配到，返回空字符串

```
SELECT REGEXP_EXTRACT_ALL('ABC', '(\\d+)');
```

```
+-----------------------------------+
| REGEXP_EXTRACT_ALL('ABC', '(\\d+)') |
+-----------------------------------+
|                                   |
+-----------------------------------+
```

emoji 字符匹配

```
mysql> SELECT REGEXP_EXTRACT_ALL('🍎 🍌,🍇 🍉', '(🍎|🍌)');
+-----------------------------------------------------------+
| REGEXP_EXTRACT_ALL('🍎 🍌,🍇 🍉', '(🍎|🍌)')                    |
+-----------------------------------------------------------+
| ['🍎','🍌']                                                |
+-----------------------------------------------------------+
```

‘Str’ 是 NULL, 返回 NULL

```
SELECT regexp_extract_all(NULL, '([a-z]+)');
```

```
+-------------------------------------+
| regexp_extract_all(NULL, '([a-z]+)') |
+-------------------------------------+
| NULL                                |
+-------------------------------------+
```

‘pattern’ 是 NULL, 返回 NULL

```
SELECT regexp_extract_all('Hello World', NULL);
```

```
+----------------------------------------+
| regexp_extract_all('Hello World', NULL) |
+----------------------------------------+
| NULL                                   |
+----------------------------------------+
```

全部参数都是 NULL，返回 NULL

```
SELECT regexp_extract_all(NULL,NULL);
```

```
+-------------------------------+
| regexp_extract_all(NULL,NULL) |
+-------------------------------+
| NULL                          |
+--------
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
SELECT regexp_extract_all('hello (world) 123', '([[:alpha:]+');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INVALID_ARGUMENT]Could not compile
    ↪ regexp pattern: ([[:alpha:]+
Error: missing ]: [[:alpha:]+
```

7.2.1.2.42 REGEXP_EXTRACT_OR_NULL

描述

从文本字符串中提取与目标正则表达式模式匹配的第一个子字符串，并根据表达式组索引从中提取特定组。

需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

> 从 Apache Doris 3.0.2 版本开始支持

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP_EXTRACT_OR_NULL(<str>, <pattern>, <pos>)
```

参数

Parameter	Description

Parameter	Description

<
↪ str
↪ >
↪

字符串参数。表示要执行正则表达式匹配的文本字符串。此字符串可以包含任意字符组合，函数将在其中搜索与匹

Parameter	Description
< ↪ pattern ↪ > ↪	字符串参数。它是目标正则表达式模式。此模式可以包含各种正则表达式元字符和字符类，精确定义要匹配的子字符串

Parameter	Description
< ↪ pos ↪ > ↪	整数参数。表示要提取的表达式组的索引。索引从 1 开始。如果设置为 0，则返回整个第一个匹配的子字符串。如果为负数或

Parameter	Description

返回值

返回字符串类型，结果为与匹配的部分。

如果输入的为 0，则返回整个第一个匹配的子字符串。如果输入的无效（负数或超过表达式组数量），则返回 NULL。如果正则表达式匹配失败，则返回 NULL。如果 <pos> < 0, 则返回 NULL; 如果 pos > 参数字符串<str>的长度, 返回 NULL;

例子

从匹配中提取特定组, 正则表达式 ([[:lower:]]+)C([[:lower:]]+) 查找由 'C' 分隔的一个或多个小写字母序列。索引为 1 的组对应第一个小写字母序列，因此返回 'b'。

```
SELECT REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 1);
```

```
+-------------------------------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 1) |
+-------------------------------------------------------------------------+
| b                                                                       |
+-------------------------------------------------------------------------+
```

返回整个匹配的子字符串当为 0 时，返回匹配模式的整个第一个子字符串。

```
SELECT REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 0);
```

```
+-------------------------------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 0) |
+-------------------------------------------------------------------------+
| bCd                                                                     |
+-------------------------------------------------------------------------+
```

无效的组索引, 由于模式只有 2 个组，索引 5 超出范围，因此返回 NULL。

```
SELECT REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 5);
```

```
+-------------------------------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[:lower:]]+)', 5) |
+-------------------------------------------------------------------------+
| NULL                                                                    |
+-------------------------------------------------------------------------+
```

不匹配的正则表达式, 字符串 'AbCdE' 中没有部分完全匹配模式

```
SELECT REGEXP_EXTRACT_OR_NULL('AbCdE', '([[:lower:]]+)C([[:upper:]]+)', 1);
```

```
+-----------------------------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('AbCdE', '([[:lower:]]+)C([[:upper:]]+)', 1) |
+-----------------------------------------------------------------------+
| NULL                                                                  |
+-----------------------------------------------------------------------+
```

中文字符匹配, 模式(\p{Han}+)(.+)首先匹配一个或多个中文字符，然后匹配任何剩余字符。索引为 2 的组表示中文字符后的非中文部分。

```
select REGEXP_EXTRACT_OR_NULL('这是一段中文 This is a passage in English 1234567', `(\\p{Han}+)
    ↪ (.+)`, 2);
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| REGEXP_EXTRACT_OR_NULL('这是一段中文 This is a passage in English 1234567', `(\\p{Han}+)(.+)`,
    ↪ 2)       |
+-----------------------------------------------------------------------------------------------------------+
    ↪
|  This is a passage in English 1234567
    ↪                                                                    |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

向表中插入数据并执行提取

```
CREATE TABLE test_regexp_extract_or_null (
    id INT,
    text_column VARCHAR(255),
    pattern_column VARCHAR(255),
    position_column INT
) PROPERTIES ("replication_num"="1");

INSERT INTO test_regexp_extract_or_null VALUES
(1, 'abc123def', '([a-z]+)([0-9]+)([a-z]+)', 2),
(2, 'Hello World', '([A-Z][a-z]+) ([A-Z][a-z]+)', 0),
(3, '123-456-789', '([0-9]{3})-([0-9]{3})-([0-9]{3})', 3),
(4, 'example@example.com', '([a-z]+)@([a-z]+)\\.([a-z]+)', 1),
(5, '测试文本 test text', `(\\p{Han}+) (.+)`, 1);

SELECT id, REGEXP_EXTRACT_OR_NULL(text_column, pattern_column, position_column) AS extracted_
    ↪ result
FROM test_regexp_extract_or_null
ORDER BY id;
```

```
+------+-----------------+
| id   | extracted_result|
+------+-----------------+
|    1 | 123             |
|    2 | Hello World     |
|    3 | 789             |
|    4 | example         |
|    5 | 测试文本          |
+------+-----------------+
```

emoji 字符匹配

```
SELECT regexp_extract_or_null('□ □ □', '□|□|□',0);
```

```
+-----------------------------------------------------------+
| regexp_extract_or_null('□ □ □', '□|□|□',0)           |
+-----------------------------------------------------------+
| □                                                         |
+-----------------------------------------------------------+
```

‘str’ 是 NULL, 返回 NULL

```
SELECT REGEXP_EXTRACT_OR_NULL(NULL, '([a-z]+)', 1);
```

```
+--------------------------------------------+
| REGEXP_EXTRACT_OR_NULL(NULL, '([a-z]+)', 1) |
+--------------------------------------------+
| NULL                                       |
+--------------------------------------------+
```

‘pattern’ 是 NULL, 返回 NULL

```
SELECT REGEXP_EXTRACT_OR_NULL('Hello World', NULL, 1);
```

```
+-------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('Hello World', NULL, 1) |
+-------------------------------------------------+
| NULL                                            |
+-------------------------------------------------+
```

‘pos’ 是 NULL, 返回 NULL

```
SELECT REGEXP_EXTRACT_OR_NULL('Hello World', '([a-z]+)', NULL);
```

```
+-------------------------------------------------------+
| REGEXP_EXTRACT_OR_NULL('Hello World', '([a-z]+)', NULL) |
```

```
+------------------------------------------------------+
| NULL                                                 |
+------------------------------------------------------+
```

全部参数都是 NULL, 返回 NULL

```
SELECT REGEXP_EXTRACT_OR_NULL(NULL,NULL,NULL);
```

```
+---------------------------------------+
| REGEXP_EXTRACT_OR_NULL(NULL,NULL,NULL) |
+---------------------------------------+
| NULL                                  |
+---------------------------------------+
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
mysql> SELECT REGEXP_EXTRACT_OR_NULL('123AbCdExCx', '([[:lower:]]+)C([[]ower:]]+)', 1);
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INVALID_ARGUMENT]Could not compile
    ↪ regexp pattern: ([[:lower:]]+)C([[:lower:]+)
Error: missing ]: [[:lower:]+)
```

7.2.1.2.43 REGEXP_REPLACE

描述

对 STR 字符串进行正则匹配，将匹配 pattern 的部分替换为新字符串。

需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP_REPLACE(<str>, <pattern>, <repl>)
```

参数

参数	描述

参数	描述
< ↪ str ↪ > ↪	此参数为 Var-char 类型。表示要执行正则表达式匹配的字符串。可以是字面量字符串或包含字符串值的表列。

参数	描述
< ↪ pattern ↪ > ↪	此参数为 Varchar 类型。是用于匹配字符串的正则表达式模式。该模式可以包含各种正则表达式元字符和构造，以定义复

参数	描述
 ↪ repl ↪ > ↪	此参数为 Varchar 类型。是将替换中匹配部分的字符串。如果想引用模式中捕获的组，可以使用 \1、\2 等反向引用，其中 \1

参数	描述

返回值

函数返回替换操作后的结果字符串。返回值为 Varchar 类型。如果中没有部分匹配，则返回原始的。

示例

测试基本替换示例, 在此示例中，字符串 'a b c' 中的所有空格都被替换为连字符。

```
mysql> SELECT regexp_replace('a b c', ' ', '-');
+-----------------------------------+
| regexp_replace('a b c', ' ', '-') |
+-----------------------------------+
| a-b-c                             |
+-----------------------------------+
```

测试使用捕获组, 这里，字符 'b' 被模式中的组 (b) 捕获，然后使用替换字符串中的反向引用 \1 将其替换为

```
mysql> SELECT regexp_replace('a b c', '(b)', `<\\1>`);
+----------------------------------------+
| regexp_replace('a b c', '(b)', `<\1>`) |
+----------------------------------------+
| a <b> c                                |
+----------------------------------------+
```

测试匹配中文字符, 此示例将字符串中所有连续的中文字符替换为 '123'

```
mysql> select regexp_replace('这是一段中文 This is a passage in English 1234567', `\\p{Han}+`, '
    ↪ 123');
+-----------------------------------------------------------------------------------------------+
| regexp_replace('这是一段中文 This is a passage in English 1234567', `\p{Han}+`, '123')       |
+-----------------------------------------------------------------------------------------------+
| 123This is a passage in English 1234567                                                       |
+-----------------------------------------------------------------------------------------------+
```

插入和测试用例, 在这组测试用例中，我们创建一个表来存储原始字符串、模式和替换字符串。然后插入各种测试数据，并使用相应的模式和替换字符串对原始字符串执行 REGEXP_REPLACE 操作。最后，我们检索并显示替换后的字符串。

```
-- 创建一个表来存储测试数据
CREATE TABLE test_table_for_regexp_replace (
        id INT,
        original_text VARCHAR(500),
        pattern VARCHAR(100),
        replacement VARCHAR(100)
    ) PROPERTIES ("replication_num"="1");
```

```
-- 插入测试数据
INSERT INTO test_table_for_regexp_replace VALUES
    (1, 'Hello, World!', ',', '-'),
    (2, 'apple123', '[0-9]', '*'),
    (3, 'aabbcc', '(aa|bb|cc)', 'XX'),
    (4, '123-456-7890', '-', ' '),
    (5, 'test,data', ',', ';'),
    (6, 'a1b2c3', '[0-9]', '#'),
    (7, 'book keeper', 'oo|ee', '**'),
    (8, 'abababab', '(ab)', 'XY'),
    (9, 'aabbcc', '(aa|bb|cc)', 'ZZ'),
    (10, 'apple,banana', ',', ' - ');

-- 对插入的数据执行替换操作
SELECT id, regexp_replace(original_text, pattern, replacement) as replaced_text FROM test_table_
    ↪ for_regexp_replace ORDER BY id;
```

```
+------+------------------+
| id   | replaced_text    |
+------+------------------+
|    1 | Hello- World!    |
|    2 | apple***         |
|    3 | XXXXYY           |
|    4 | 123 456 7890     |
|    5 | test;data        |
|    6 | a#b#c#           |
|    7 | b**k k**per      |
|    8 | XYXYXY           |
|    9 | ZZZZYY           |
|   10 | apple - banana   |
+------+------------------+
```

emoji 字符替换

```
SELECT regexp_replace('🌍 Earth 🍔 Food', '🌍|🍔', '*');
```

```
+-----------------------------------------------------------+
| regexp_replace('🌍 Earth 🍔 Food', '🌍|🍔', '*')                |
+-----------------------------------------------------------+
| * Earth * Food                                            |
+-----------------------------------------------------------+
```

‘str’ 是 NULL 值，返回 NULL 值

```
mysql> SELECT REGEXP_REPLACE(NULL, ' ', '-');
```

```
+--------------------------------+
| REGEXP_REPLACE(NULL, ' ', '-') |
+--------------------------------+
| NULL                           |
+--------------------------------+
```

‘pattern’ 是 NULL 值，则返回 NULL 值

```
mysql> SELECT REGEXP_REPLACE('Hello World', NULL, '-');
+-----------------------------------------+
| REGEXP_REPLACE('Hello World', NULL, '-') |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

‘repl’ 是 NULL 值，则返回 NULL 值

```
mysql> SELECT REGEXP_REPLACE('Hello World', ' ', NULL);
+-----------------------------------------+
| REGEXP_REPLACE('Hello World', ' ', NULL) |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

全部参数都是 NULL 值，返回 NULL

```
mysql> SELECT REGEXP_REPLACE(NULL, NULL, NULL);
+----------------------------------+
| REGEXP_REPLACE(NULL, NULL, NULL) |
+----------------------------------+
| NULL                             |
+----------------------------------+
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
SELECT regexp_replace('a b c', '(b', '<\\1>');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INVALID_ARGUMENT]Could not compile
    ↪ regexp pattern: (b
Error: missing ): (b
```

7.2.1.2.44 REGEXP_REPLACE_ONE

描述

REGEXP_REPLACE_ONE 函数是一个强大的工具，用于对给定字符串执行正则表达式匹配。它允许您查找并替换字符串中首次出现的特定模式。

在处理文本数据时，您经常需要根据特定规则操作字符串。正则表达式提供了一种灵活高效的方式来定义这些规则。此函数接受一个字符串 str、一个正则表达式模式 pattern 和一个替换字符串 repl。然后，它会在 str 中搜索首个匹配 pattern 的部分，并将其替换为 repl。

需要注意的是，在处理字符集匹配时，应使用 Utf-8 标准字符类。这确保函数能够正确识别和处理来自不同语言的各种字符。

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

支持的字符匹配种类: https://github.com/google/re2/wiki/Syntax

语法

```
REGEXP_REPLACE_ONE(`<str>`, `<pattern>`, `<repl>`)
```

参数

参数	描述
<str>	此参数为字符串类型。表示要执行正则表达式匹配的字符串。这是您想要修改的目标字符串。
<pattern>	此参数也为字符串类型。它是一个正则表达式模式。函数将在 <str> 字符串中搜索此模式的首次出现。
<repl>	这也是一个字符串参数。它包含将替换 <str> 中首个匹配 <pattern> 部分的字符串。

返回值

函数返回替换操作后的结果字符串。返回类型为 Varchar。如果 <str> 中没有部分匹配 <pattern>，则返回原始的 <str>。

示例

将首个空格替换为连字符, 在此示例中，输入字符串 <str> 为 ‘a b c’，正则表达式模式 <pattern> 为单个空格 ‘’，替换字符串 <repl> 为连字符 ‘-’。函数在字符串 ‘a b c’ 中搜索首个空格，并将其替换为连字符。因此输出为 ‘a-b c’。

```
mysql> SELECT regexp_replace_one('a b c', ' ', '-');

+-----------------------------------+
| regexp_replace_one('a b c', ' ', '-') |
+-----------------------------------+
| a-b c                             |
+-----------------------------------+
```

替换首个匹配组, 这里，输入字符串 <str> 为 ‘a b b’，正则表达式模式 <pattern> 为 ‘(b)’，这是一个匹配字符 ‘b’ 的捕获组。替换字符串 <repl> 为 ‘<\1>’，其中 \1 指第一个捕获组 (在本例中，即匹配的 ‘b’)。函数在字符串 ‘a b b’ 中找到首个 ‘b’，并将其替换为’<b> ‘。因此，输出为’a <b> b’

```
mysql> SELECT regexp_replace_one('a b b', '(b)', '<\\1>');
+---------------------------------------+
| regexp_replace_one('a b b', '(b)', ``<\1>``) |
+---------------------------------------+
| a `<b>` b                                 |
```

```
+----------------------------------------+
```

替换首个中文字符，输入字符串 <str> 是一个包含中文字符和英文文本的长字符串。正则表达式模式 <pattern > 为 \p {Han}，这是一个匹配任何中文字符的 Unicode 字符类。替换字符串 <repl> 为 '123'。函数在字符串中搜索首个中文字符，并将其替换为 '123'。因此输出为 '123 是一段中文 This is a passage in English 1234567'

```
mysql> select regexp_replace_one('这是一段中文 This is a passage in English 1234567', '\\p{Han}',
    ↪  '123');
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
| regexp_replace_one('这是一段中文 This is a passage in English 1234567', `\p{Han}`, '123')
    ↪       |
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
| 123是一段中文This is a passage in English 1234567
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
```

Insert data into a table and perform replacement。创建一个名为 test_table_for_regexp_replace_one 的表，包含四列：id（整数）、text_data（要执行替换的字符串）、pattern（用于匹配的正则表达式模式）和 repl（替换字符串）。然后，向表中插入十行数据，每行包含四列的不同值。最后，使用 SELECT 语句查询表。对于每一行，使用相应的 pattern 和 repl 值对 text_data 列应用 REGEXP_REPLACE_ONE 函数。替换结果别名为 replaced_result。行按 id 列排序

```
CREATE TABLE test_table_for_regexp_replace_one (
        id INT,
        text_data VARCHAR(500),
        pattern VARCHAR(100),
        repl VARCHAR(100)
    ) PROPERTIES ("replication_num"="1");

INSERT INTO test_table_for_regexp_replace_one VALUES
    (1, 'Hello World', ' ', '-'),
    (2, 'apple123', '[0-9]', 'X'),
    (3, 'aabbcc', '(aa)', 'AA'),
    (4, '123-456-7890', '[0-9][0-9][0-9]', 'XXX'),
    (5, 'test,data', ',', ';'),
    (6, 'a1b2c3', '[a-z][0-9]', 'X'),
    (7, 'book keeper', 'oo', 'OO'),
    (8, 'ababab', '(ab)', 'AB'),
    (9, 'aabbcc', '(bb)', 'BB'),
    (10, 'apple,banana', '[aeiou]', 'X');
```

```
SELECT id, regexp_replace_one(text_data, pattern, repl) as replaced_result FROM test_table_for_
    ↪ regexp_replace_one ORDER BY id;
```

```
+------+-----------------+
| id   | replaced_result |
+------+-----------------+
|    1 | Hello-World     |
|    2 | appleX23        |
|    3 | AAbbcc          |
|    4 | XXX-456-7890    |
|    5 | test;data       |
|    6 | Xb2c3           |
|    7 | BOOk keeper     |
|    8 | ABabab          |
|    9 | aaBBcc          |
|   10 | Xpple,banana    |
+------+-----------------+
```

emoji 字符替换

```
SELECT regexp_replace_one('□ □ □', '□|□', '[SMILE]');
```

```
+-----------------------------------------------------------+
| regexp_replace_one('□ □ □', '□|□', '[SMILE]')             |
+-----------------------------------------------------------+
| [SMILE][SMILE] □                                          |
+-----------------------------------------------------------+
```

‘str’ 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_REPLACE_ONE(NULL, ' ', '-');
+------------------------------------+
| REGEXP_REPLACE_ONE(NULL, ' ', '-') |
+------------------------------------+
| NULL                               |
+------------------------------------+
```

‘pattern’ 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_REPLACE_ONE('Hello World', NULL, '-');
+---------------------------------------------+
| REGEXP_REPLACE_ONE('Hello World', NULL, '-') |
+---------------------------------------------+
| NULL                                        |
+---------------------------------------------+
```

‘repl’ 是 NULL, 则返回 NULL

```
mysql> SELECT REGEXP_REPLACE_ONE('Hello World', ' ', NULL);
+-----------------------------------------------+
| REGEXP_REPLACE_ONE('Hello World', ' ', NULL) |
+-----------------------------------------------+
| NULL                                          |
+-----------------------------------------------+
```

全部参数是 NULL，则返回 NULL

```
mysql> SELECT REGEXP_REPLACE_ONE(NULL, NULL, NULL);
+--------------------------------------+
| REGEXP_REPLACE_ONE(NULL, NULL, NULL) |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

如果 ‘pattern’ 参数不符合正则表达式，则抛出错误

```
SELECT regexp_replace_one('a b b', '(b', `<\\1>`);
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (10.16.10.2)[INVALID_ARGUMENT]Could not compile
    ↪ regexp pattern: (b
Error: missing ): (b
```

7.2.1.2.45 REPEAT

描述

REPEAT 函数用于将一个字符串重复指定的次数。

语法

```
REPEAT( <str>, <count> )
```

参数

参数	说明
<str>	需要被重复的字符串
<count>	重复的次数，必须为非负整数，小于 1 时将返回空串

返回值

返回重复指定的次数的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL

举例

```
SELECT repeat("a", 3);
```

```
+----------------+
| repeat('a', 3) |
+----------------+
| aaa            |
+----------------+
```

```
SELECT repeat("a", -1);
```

```
+-----------------+
| repeat('a', -1) |
+-----------------+
|                 |
+-----------------+
```

7.2.1.2.46 REPLACE

描述

REPLACE 函数用于将字符串中的一部分字符替换为其他字符。

语法

```
REPLACE ( <str>, <old>, <new> )
```

参数

参数	说明
<str>	需要被替换的字符串
<old>	需要被替换掉的子字符串，如果<old>不在<str>中，则不会进行替换
<new>	用于替换 old 的新子字符串

返回值

返回替换掉子字符串后的新字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL

举例

```
SELECT replace('hello world', 'world', 'universe');
```

```
+-------------------------------------------+
| replace('hello world', 'world', 'universe') |
```

```
+-------------------------------------------+
| hello universe                            |
+-------------------------------------------+
```

7.2.1.2.47 REPLACE_EMPTY

描述

REPLACE_EMPTY 函数用于将字符串中的一部分字符替换为其他字符。和 REPLACE 函数不同的是，当 old 为空字符串时，会将 new 字符串插入到 str 字符串的每个字符前，以及 str 字符串的最后。

除此之外，其他行为和 REPLACE() 函数完全一致。

该函数主要用于兼容 Presto、 Trino，其行为了 Presto、 Trino 中的 REPLACE() 函数完全一致。

自 2.1.5 版本支持。

语法

```
REPLACE_EMPTY ( <str>, <old>, <new> )
```

参数

参数	说明
<str>	需要被替换的字符串
<old>	需要被替换掉的子字符串，如果<old>不在<str>中，则不会进行替换，如果 <old> 为空字符串时，会将 <new> 字符串插入到 <str> 字符串的每个字符前
<new>	用于替换 <old> 的新子字符串

返回值

返回替换掉子字符串后的新字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 如果 old 为空字符串时，则返回将 new 字符串插入到 str 字符串的每个字符前的字符串

举例

```
SELECT replace('hello world', 'world', 'universe');
```

```
+-------------------------------------------+
| replace('hello world', 'world', 'universe') |
+-------------------------------------------+
| hello universe                            |
+-------------------------------------------+
```

```
SELECT replace_empty("abc", '', 'xyz');
```

```
+--------------------------------+
| replace_empty('abc', '', 'xyz') |
+--------------------------------+
| xyzaxyzbxyzcxyz                |
+--------------------------------+
```

```
SELECT replace_empty("", "", "xyz");
```

```
+------------------------------+
| replace_empty('', '', 'xyz') |
+------------------------------+
| xyz                          |
+------------------------------+
```

7.2.1.2.48 REVERSE

描述

REVERSE 函数用于将字符串中的字符顺序颠倒或将数组中的元素顺序颠倒。

语法

```
REVERSE( <seq> )
```

参数

参数	说明
<seq>	需要被反转顺序的字符串或数组

返回值

返回反转顺序后的字符串或数组。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL

举例

```
SELECT reverse('hello');
```

```
+------------------+
| REVERSE('hello') |
+------------------+
| olleh            |
+------------------+
```

```
SELECT reverse(['hello', 'world']);
```

```
+-----------------------------+
| reverse(['hello', 'world']) |
+-----------------------------+
| ["world", "hello"]          |
+-----------------------------+
```

7.2.1.2.49 RPAD

描述

用于在原始字符串的右侧填充指定的字符，直到达到指定的长度。

语法

```
RPAD ( <str> , <len> , <pad>)
```

参数

参数	说明
<str>	需要被填充的字符串
<len>	最终结果字符串的总长度，指的是字符长度而不是字节长度
<pad>	用于填充的字符串

返回值

返回填充后的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 如果<pad>为空且<len>大于<str>的长度，则返回值为空串。
- 如果<len>小于<str>的长度，则返回<str>截取至<len>的字符串。
- 如果<len>小于 0，则返回值为 NULL

举例

```
SELECT rpad('hello', 1, '');
```

```
+----------------------+
| rpad('hello', 1, '') |
+----------------------+
| h                    |
+----------------------+
```

```
SELECT rpad('hello', 10, 'world');
```

```
+----------------------------+
| rpad('hello', 10, 'world') |
+----------------------------+
| helloworld                 |
+----------------------------+
```

```
SELECT rpad('hello', 10, '');
```

```
+-----------------------+
| rpad('hello', 10, '') |
+-----------------------+
|                       |
+-----------------------+
```

7.2.1.2.50 RTRIM

描述

RTRIM 函数用于去除字符串右侧（结尾部分）连续出现空格或指定字符。

语法

```
RTRIM( <str> [, <trim_chars> ] )
```

参数

参数	说明
<str>	需要被修建的字符串
<trim_chars>	可选参数。如果提供了该参数，RTRIM 函数将去除<str>右侧出现的<trim_chars>中的所有字符。如果未提供此参数，RTRIM 函数将仅去除<str>右侧的空格字符。

返回值

返回<str>右侧去除<trim_chars>后的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL

举例

```
SELECT rtrim('ab d   ') str;
```

```
+------+
| str  |
+------+
| ab d |
+------+
```

```
SELECT rtrim('ababccaab','ab') str;
```

```
+---------+
| str     |
+---------+
| ababcca |
+---------+
```

7.2.1.2.51 RTRIM_IN

描述

RTRIM_IN 函数用于移除字符串右侧的指定字符。当不指定移除字符集合时，默认移除右侧的空格；当指定字符集合时，将移除右侧出现的所有指定字符（不考虑字符集合顺序）。RTRIM_IN 的特点是会移除指定字符集合中的任意字符组合，而 RTRIM 函数则是按照完整的字符串匹配进行移除。

语法

```
RTRIM_IN(<str>[, <rhs>])
```

参数

参数	说明
<str>	需要处理的字符串。类型：VARCHAR
<rhs>	可选参数，要移除的字符集合。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示处理后的字符串。

特殊情况：- 如果 str 为 NULL，返回 NULL - 如果不指定 rhs，移除右侧所有空格 - 如果指定 rhs，移除右侧出现在 rhs 中的所有字符，直到遇到第一个不在 rhs 中的字符

示例

1. 移除右侧空格

```
SELECT rtrim_in('ab d   ') str;
```

```
+------+
| str  |
+------+
| ab d |
+------+
```

2. 移除指定字符集合

```
-- RTRIM_IN 会移除右端的'a'和'b'中的任意字符
SELECT rtrim_in('ababccaab', 'ab') str;
```

```
+---------+
| str     |
+---------+
| ababcc  |
+---------+
```

3. 与 RTRIM 函数的对比

```
SELECT rtrim_in('ababccaab', 'ab'),rtrim('ababccaab', 'ab');
```

```
+-----------------------------+--------------------------+
| rtrim_in('ababccaab', 'ab') | rtrim('ababccaab', 'ab') |
+-----------------------------+--------------------------+
| ababcc                      | ababcca                  |
+-----------------------------+--------------------------+
```

7.2.1.2.52 SPACE

描述

生成由指定数量的空格组成的字符串。

语法

```
SPACE ( <len> )
```

参数

参数	说明
<len>	要生成的空格的数量

返回值

返回指定数量的空格组成的字符串。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- <len>小于 0 时，返回空字符串

举例

```
SELECT space(10);
```

```
+------------+
| space(10)  |
+------------+
|            |
+------------+
```

```
SELECT space(null);
```

```
+-------------+
| space(NULL) |
+-------------+
| NULL        |
+-------------+
```

7.2.1.2.53 SPLIT_BY_STRING

描述

将输入字符串按照指定的字符串拆分成字符串数组。

语法

```
SPLIT_BY_STRING ( <str>, <separator> )
```

参数

参数	说明
<str>	需要分割的字符串
<separator>	用于分割的字符串

返回值

返回按照指定的字符串拆分成字符串数组。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- <separator>为空字符串时，<str>会按字节进行拆分

举例

```
SELECT split_by_string('hello','l');
```

```
+-------------------------------+
| split_by_string('hello', 'l') |
+-------------------------------+
| ["he", "", "o"]               |
+-------------------------------+
```

```
SELECT split_by_string('hello','');
```

```
+------------------------------+
| split_by_string('hello', '') |
+------------------------------+
| ["h", "e", "l", "l", "o"]    |
+------------------------------+
```

7.2.1.2.54 SPLIT_PART

描述

用于将一个字符串按照指定的分隔符拆分成多个部分，并返回其中的一个部分。

语法

```
SPLIT_PART ( <str>, <separator>, <part_index> )
```

参数

参数	说明
<str>	需要分割的字符串
<separator>	用于分割的字符串
<part_index>	要返回的部分的索引，从 1 开始计数

返回值

返回根据分割符拆分后的字符串的指定部分。特殊情况：

- 任意参数中有一个为 NULL，则返回 NULL
- 当<part_index>为 0 时，返回 NULL

举例

```
select split_part("hello world", " ", 1);
```

```
+----------------------------------+
| split_part('hello world', ' ', 1) |
+----------------------------------+
| hello                            |
+----------------------------------+
```

```
SELECT split_part('apple,banana,cherry', ',', 0);
```

```
+-----------------------------------------+
| split_part('apple,banana,cherry', ',', 0) |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

7.2.1.2.55 STARTS_WITH

描述

STARTS_WITH 函数检查字符串是否以指定的前缀开头。如果字符串以指定的前缀开头，则返回 true；否则返回 false。

语法

```
STARTS_WITH(<str>, <prefix>)
```

参数

参数	说明
<str>	要检查的字符串。类型：VARCHAR
<prefix>	要匹配的前缀字符串。类型：VARCHAR

返回值

返回 BOOLEAN 类型。

特殊情况： - 如果任何参数为 NULL，则返回 NULL

举例

1. 匹配成功

```
SELECT starts_with('hello world', 'hello');
```

```
+-------------------------------------+
| starts_with('hello world', 'hello') |
+-------------------------------------+
|                                   1 |
+-------------------------------------+
```

2. 匹配失败

```
SELECT starts_with('hello world', 'world');
```

```
+-------------------------------------+
| starts_with('hello world', 'world') |
```

```
+------------------------------------+
|                                  0 |
+------------------------------------+
```

7.2.1.2.56 STRCMP

描述

STRCMP 函数用于按照字典顺序比较两个字符串。该函数将返回一个整数值来表示两个字符串的比较结果。

语法

```
STRCMP(<str0>, <str1>)
```

参数

参数	说明
<str0>	第一个要比较的字符串。类型：VARCHAR
<str1>	第二个要比较的字符串。类型：VARCHAR

返回值

返回 TINYINT 类型，表示比较结果：- 返回 0：如果 str0 和 str1 相同 - 返回 1：如果 str0 在字典顺序上大于 str1 - 返回 -1：如果 str0 在字典顺序上小于 str1

特殊情况：- 如果任意参数为 NULL，返回 NULL

示例

1. 相同字符串比较

```
SELECT strcmp('test', 'test');
```

```
+------------------------+
| strcmp('test', 'test') |
+------------------------+
|                      0 |
+------------------------+
```

2. 第一个字符串较大

```
SELECT strcmp('test1', 'test');
```

```
+-------------------------+
| strcmp('test1', 'test') |
+-------------------------+
|                       1 |
+-------------------------+
```

3. 第一个字符串较小

```
SELECT strcmp('test', 'test1');
```

```
+-------------------------+
| strcmp('test', 'test1') |
+-------------------------+
|                      -1 |
+-------------------------+
```

7.2.1.2.57 STRLEFT

描述

STRLEFT 函数用于返回字符串左边指定长度的部分。长度的单位为 UTF8 字符。

别名

LEFT

语法

```
STRLEFT(<str>, <len>)
```

参数

参数	说明
<str>	需要截取的字符串。类型：VARCHAR
<len>	要返回的字符数量。类型：INT

返回值

返回 VARCHAR 类型，表示截取的字符串。

特殊情况：- 如果任意参数为 NULL，返回 NULL - 如果 len 小于等于 0，返回空字符串 “” - 如果 len 大于字符串长度，返回整个字符串

示例

1. 基本用法

```
SELECT strleft('Hello doris', 5);
```

```
+------------------------+
| strleft('Hello doris', 5) |
+------------------------+
| Hello                  |
+------------------------+
```

2. 负数长度处理

```
SELECT strleft('Hello doris', -5);
```

```
+----------------------------+
| strleft('Hello doris', -5) |
+----------------------------+
|                            |
+----------------------------+
```

3. NULL 参数处理

```
SELECT strleft('Hello doris', NULL);
```

```
+-----------------------------+
| strleft('Hello doris', NULL) |
+-----------------------------+
| NULL                        |
+-----------------------------+
```

4. NULL 字符串处理

```
SELECT strleft(NULL, 3);
```

```
+------------------+
| strleft(NULL, 3) |
+------------------+
| NULL             |
+------------------+
```

7.2.1.2.58 STRRIGHT

描述

STRRIGHT 函数用于返回字符串右边指定长度的部分。长度的单位为 UTF8 字符。

别名

RIGHT

语法

```
STRRIGHT(<str>, <len>)
```

参数

参数	说明
<str>	需要截取的字符串。类型：VARCHAR
<len>	要返回的字符数量。类型：INT

返回值

返回 VARCHAR 类型，表示截取的字符串。

特殊情况：- 如果任意参数为 NULL，返回 NULL - 如果 len 为负数，返回从第 abs(len) 个字符开始向右的部分 - 如果 len 大于字符串长度，返回整个字符串

示例

1. 基本用法

```
SELECT strright('Hello doris', 5);
```

```
+----------------------------+
| strright('Hello doris', 5) |
+----------------------------+
| doris                      |
+----------------------------+
```

2. 负数长度处理

```
SELECT strright('Hello doris', -7);
```

```
+-----------------------------+
| strright('Hello doris', -7) |
+-----------------------------+
| doris                       |
+-----------------------------+
```

3. NULL 参数处理

```
SELECT strright('Hello doris', NULL);
```

```
+-------------------------------+
| strright('Hello doris', NULL) |
+-------------------------------+
| NULL                          |
+-------------------------------+
```

4. NULL 字符串处理

```
SELECT strright(NULL, 5);
```

```
+-------------------+
| strright(NULL, 5) |
+-------------------+
| NULL              |
+-------------------+
```

7.2.1.2.59 SUB_REPLACE

描述

sub_replace 函数用于替换字符串中的子字符串。可以指定要替换的子字符串和替换的目标字符串，返回用new_str 字符串替换 str 中从 start 开始长度为 len 的新字符串。其中 start, len 为负整数，返回 NULL, 且 len 的默认值为 new_str 的长度。

语法

```
sub_replace(<str>, <new_str>, [ ,<start> [ , <len> ] ])
```

参数

参数	说明
<str>	要进行替换操作的目标字符串
<new_str>	用于替换的目标字符串
<start>	是替换操作开始的位置，表示从字符串中的哪个位置开始进行替换
<len>	是一个可选参数，表示要替换的子字符串的长度

返回值

返回替换后的字符串。

举例

```
select sub_replace("this is origin str","NEW-STR",1);
```

```
+-----------------------------------------------+
| sub_replace('this is origin str', 'NEW-STR', 1) |
+-----------------------------------------------+
| tNEW-STRorigin str                            |
+-----------------------------------------------+
```

```
select sub_replace("doris","***",1,2);
```

```
+----------------------------------+
| sub_replace('doris', '***', 1, 2) |
+----------------------------------+
| d***is                           |
+----------------------------------+
```

7.2.1.2.60 SUBSTRING

描述

SUBSTRING 函数用于从字符串中提取子字符串。可以指定起始位置和长度，支持正向和反向提取。字符串中第一个字符的位置为 1。

别名

SUBSTR

语法

```
SUBSTRING(<str>, <pos> [, <len>])
```

参数

参数	说明
<str>	源字符串。类型：VARCHAR
<pos>	起始位置，可以为负数。类型：INT
<len>	可选参数，要提取的长度。类型：INT

返回值

返回 VARCHAR 类型，表示提取的子字符串。

特殊情况：- 如果任意参数为 NULL，返回 NULL - 如果 pos 为 0，返回空字符串 - 如果 pos 为负数，从字符串末尾开始向前计数 - 如果 pos 超出字符串长度，返回空字符串 - 如果不指定 len，则返回从 pos 到字符串末尾的所有字符

示例

1. 基本用法（指定起始位置）

```
SELECT substring('abc1', 2);
```

```
+-----------------------------+
| substring('abc1', 2)        |
+-----------------------------+
| bc1                         |
+-----------------------------+
```

2. 使用负数位置

```
SELECT substring('abc1', -2);
```

```
+-----------------------------+
| substring('abc1', -2)       |
+-----------------------------+
| c1                          |
+-----------------------------+
```

3. 位置为 0 的情况

```
SELECT substring('abc1', 0);
```

```
+----------------------+
| substring('abc1', 0) |
+----------------------+
|                      |
+----------------------+
```

4. 位置超出字符串长度

```
SELECT substring('abc1', 5);
```

```
+-----------------------------+
| substring('abc1', 5)        |
+-----------------------------+
|                             |
+-----------------------------+
```

5. 指定长度参数

```
SELECT substring('abc1def', 2, 2);
```

```
+-----------------------------+
| substring('abc1def', 2, 2)  |
+-----------------------------+
| bc                          |
+-----------------------------+
```

7.2.1.2.61 SUBSTRING_INDEX

描述

SUBSTRING_INDEX 函数用于截取字符串，根据指定的分隔符和出现次数来确定截取位置。该函数支持从左边或右边开始计数。

语法

```
SUBSTRING_INDEX(<content>, <delimiter>, <field>)
```

参数

参数	说明
<content>	需要截取的字符串。类型：VARCHAR
<delimiter>	分隔符，大小写敏感且多字节安全。类型：VARCHAR
<field>	分隔符出现的次数。正数从左计数，负数从右计数。类型：INT

返回值

返回 VARCHAR 类型，表示截取后的子字符串。

特殊情况：- 如果 field > 0，返回从左边起第 field 个分隔符之前的子串 - 如果 field < 0，返回从右边起第 |field| 个分隔符之后的子串 - 如果 field = 0，当 content 不为 NULL 时返回空串，content 为 NULL 时返回 NULL - 如果任意参数为 NULL，返回 NULL

示例

1. 从左边截取第一个空格之前的内容

```
SELECT substring_index('hello world', ' ', 1);
```

```
+-----------------------------------------+
| substring_index('hello world', ' ', 1) |
+-----------------------------------------+
| hello                                   |
+-----------------------------------------+
```

2. 从左边截取所有内容（分隔符次数大于实际出现次数）

```
SELECT substring_index('hello world', ' ', 2);
```

```
+-----------------------------------------+
| substring_index('hello world', ' ', 2) |
+-----------------------------------------+
| hello world                             |
+-----------------------------------------+
```

3. 从右边截取最后一个空格之后的内容

```
SELECT substring_index('hello world', ' ', -1);
```

```
+-----------------------------------------+
| substring_index('hello world', ' ', -1) |
+-----------------------------------------+
| world                                    |
+-----------------------------------------+
```

4. field 为 0 的情况

```
SELECT substring_index('hello world', ' ', 0);
```

```
+-----------------------------------------+
| substring_index('hello world', ' ', 0) |
+-----------------------------------------+
|                                         |
+-----------------------------------------+
```

7.2.1.2.62 TO_BASE64

描述

TO_BASE64 函数用于将输入的字符串转换为 Base64 编码格式。Base64 编码可以将任意二进制数据转换成由 64 个字符组成的字符串。

语法

```
TO_BASE64(<str>)
```

参数

参数	说明
<str>	需要进行 Base64 编码的字符串。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示 Base64 编码后的字符串。

特殊情况：- 如果输入为 NULL，返回 NULL - 如果输入为空字符串，返回空字符串

示例

1. 单字符编码

```
SELECT to_base64('1');
```

```
+----------------+
| to_base64('1') |
+----------------+
| MQ==           |
+----------------+
```

2. 多字符编码

```
SELECT to_base64('234');
```

```
+------------------+
| to_base64('234') |
+------------------+
| MjM0             |
+------------------+
```

7.2.1.2.63 TOKENIZE

描述

TOKENIZE 函数使用指定的分词器对字符串进行分词, 并返回分词结果。该函数特别适用于测试和理解在使用倒排索引进行全文搜索时, 文本将如何被分析处理。

版本差异 TOKENIZE 函数在 3.0 和 3.1+ 版本之间存在行为差异: - 3.0 版本: 使用 parser 参数, 返回简单字符串数组 - 3.1+ 版本: 支持 built_in_analyzer 和自定义 analyzer, 返回 JSON 对象数组, 功能更强大

关于 3.0 版本的具体用法, 请参见3.0 版本特性 章节。

3.1+ 版本特性 (推荐)

语法

```
VARCHAR TOKENIZE(VARCHAR str, VARCHAR properties)
```

参数

- str: 要进行分词的输入字符串, 类型: VARCHAR
- properties: 指定分词器配置的属性字符串, 类型: VARCHAR

properties 参数支持以下键值对 (格式: "key1"="value1", "key2"="value2"):

属性	描述	示例值
built_in_analyzer	内置分词器类型	"standard", "english", "chinese", "unicode", "icu", "basic", "ik", "none"
analyzer	自定义分词器名称 (通过 CREATE INVERTED INDEX ANALYZER 创建)	"my_custom_analyzer"

属性	描述	示例值
parser	内置分词器类型 (向后兼容)	"chinese", "english", "unicode"
parser_mode	中文分词器的分词模式	"fine_grained", "coarse_grained"
support_phrase	启用短语支持 (存储位置信息)	"true", "false"
lower_case	将词条转换为小写	"true", "false"
char_filter_type	字符过滤器类型	"char_replace"
char_filter_pattern	要替换的字符 (与 char_filter_type 配合使用)	"._=:,"
char_filter_replacement	替换字符 (与 char_filter_type 配合使用)	" " (空格)
stopwords	停用词配置	"none"

返回值

返回包含分词结果 JSON 数组的 VARCHAR 类型字符串。数组中的每个元素是一个对象, 具有以下结构: - token:

分词后的词条 - `position`: (可选) 当启用 `support_phrase` 时, 词条的位置索引

示例

示例 1: 使用内置分词器

```
-- 标准分词器
SELECT TOKENIZE("Hello World", '"built_in_analyzer"="standard"');
```

```
[{ "token": "hello" }, { "token": "world" }]
```

```
-- 英语分词器(带词干提取)
SELECT TOKENIZE("running quickly", '"built_in_analyzer"="english"');
```

```
[{ "token": "run" }, { "token": "quick" }]
```

```
-- 中文分词器
SELECT TOKENIZE('我来到北京清华大学', '"built_in_analyzer"="chinese"');
```

```
[{ "token": "我" }, { "token": "来到" }, { "token": "北京" }, { "token": "清华大学" }]
```

```
-- Unicode 分词器
SELECT TOKENIZE('Apache Doris数据库', '"built_in_analyzer"="unicode"');
```

```
[{ "token": "apache" }, { "token": "doris" }, { "token": "数" }, { "token": "据" }, { "token": "
    ↪ 库" }]
```

```
-- ICU 分词器处理多语言文本
SELECT TOKENIZE("Hello World 世界", '"built_in_analyzer"="icu"');
```

```
[{ "token": "hello" }, { "token": "world" }, { "token": "世界" }]
```

```
-- 基础分词器
SELECT TOKENIZE("GET /images/hm_bg.jpg HTTP/1.0", '"built_in_analyzer"="basic"');
```

```
[{ "token": "get" }, { "token": "images" }, { "token": "hm" }, { "token": "bg" }, { "token": "jpg
    ↪ " }, { "token": "http" }, { "token": "1" }, { "token": "0" }]
```

```
-- IK 分词器处理中文文本
SELECT TOKENIZE("中华人民共和国国歌", '"built_in_analyzer"="ik"');
```

```
[{ "token": "中华人民共和国" }, { "token": "国歌" }]
```

示例 2: 中文分词器的细粒度模式

```
SELECT TOKENIZE('我来到北京清华大学', '"built_in_analyzer"="chinese", "parser_mode"="fine_grained
    ↪ "');
```

```
[{ "token": "我" }, { "token": "来到" }, { "token": "北京" }, { "token": "清华" }, { "token": "
    ↪ 清华大学" }, { "token": "华大" }, { "token": "大学" }]
```

示例 3: 使用字符过滤器

```
SELECT TOKENIZE('GET /images/hm_bg.jpg HTTP/1.0 test:abc=bcd',
    '"built_in_analyzer"="unicode","char_filter_type" = "char_replace","char_filter_pattern" = ".
        ↪ _=:,","char_filter_replacement" = " "');
```

```
[{ "token": "get" }, { "token": "images" }, { "token": "hm" }, { "token": "bg" }, { "token": "jpg
    ↪ " }, { "token": "http" }, { "token": "1" }, { "token": "0" }, { "token": "test" }, { "
    ↪ token": "abc" }, { "token": "bcd" }]
```

示例 4: 启用短语支持 (位置信息)

```
SELECT TOKENIZE("Hello World", '"built_in_analyzer"="standard", "support_phrase"="true"');
```

```
[{ "token": "hello", "position": 0 }, { "token": "world", "position": 1 }]
```

示例 5: 使用自定义分词器

首先创建一个自定义分词器:

```
CREATE INVERTED INDEX ANALYZER lowercase_delimited
PROPERTIES (
    "tokenizer" = "standard",
    "token_filter" = "asciifolding, lowercase"
);
```

然后在 TOKENIZE 中使用:

```
SELECT TOKENIZE("FOO-BAR", '"analyzer"="lowercase_delimited"');
```

```
[{ "token": "foo" }, { "token": "bar" }]
```

3.0 版本特性

> 3.0 版本的功能相比 3.1+ 版本有所限制, 建议升级到 3.1+ 以获得增强功能。

语法

```
ARRAY<VARCHAR> TOKENIZE(VARCHAR str, VARCHAR properties)
```

参数

3.0 版本的 properties 参数支持:

属性	描述	示例值
parser	内置分词器类型	"chinese", "english", "unicode"
parser_mode	中文分词器的分词模式	"fine_grained", "coarse_grained"
char_filter_type	字符过滤器类型	"char_replace"
char_filter_pattern	要替换的字符	"._=:,"
char_filter_replacement	替换字符	" " (空格)
stopwords	停用词配置	"none"

3.0 版本不支持: - built_in_analyzer 参数 - analyzer 参数 (自定义分词器) - support_phrase 参数 - lower_case 参数 - 额外的分词器: icu, basic, ik, standard

返回值

返回 ARRAY<VARCHAR> 类型, 包含分词后的字符串数组 (简单字符串数组, 不是 JSON 对象)。

示例

示例 1: 使用中文分词器

```
SELECT TOKENIZE('我来到北京清华大学', "'parser'='chinese'");
```

```
["我", "来到", "北京", "清华大学"]
```

示例 2: 中文分词器的细粒度模式

```
SELECT TOKENIZE('我来到北京清华大学', "'parser'='chinese', 'parser_mode'='fine_grained'");
```

```
["我", "来到", "北京", "清华", "清华大学", "华大", "大学"]
```

示例 3: 使用 Unicode 分词器

```
SELECT TOKENIZE('Apache Doris数据库', "'parser'='unicode'");
```

```
["apache", "doris", "数", "据", "库"]
```

示例 4: 使用字符过滤器

```
SELECT TOKENIZE('GET /images/hm_bg.jpg HTTP/1.0 test:abc=bcd',
    '"parser"="unicode","char_filter_type" = "char_replace","char_filter_pattern" = "._=:,","char
        ↪ _filter_replacement" = " "');
```

```
["get", "images", "hm", "bg", "jpg", "http", "1", "0", "test", "abc", "bcd"]
```

示例 5: 停用词配置

```
SELECT TOKENIZE('华夏智胜新税股票A', '"parser"="unicode"');
```

```
["华", "夏", "智", "胜", "新", "税", "股", "票"]
```

```
SELECT TOKENIZE('华夏智胜新税股票A', '"parser"="unicode","stopwords" = "none"');
```

```
["华", "夏", "智", "胜", "新", "税", "股", "票", "a"]
```

注意事项

1. 版本兼容性:

- 3.0 版本使用 parser 参数, 返回简单字符串数组
- 3.1+ 版本同时支持 parser(向后兼容) 和 built_in_analyzer, 返回 JSON 对象数组
- 3.1+ 版本新增自定义分词器、更多内置分词器和短语支持功能

2. 支持的分词器:

- 3.0 版本: chinese, english, unicode
- 3.1+ 版本: standard, english, chinese, unicode, icu, basic, ik, none

3. 分词模式: parser_mode 属性主要用于 chinese 分词器:

- fine_grained: 细粒度模式, 生成更详细的词条, 包含重叠片段
- coarse_grained: 粗粒度模式 (默认), 标准分词

4. 字符过滤器: 需要同时使用 char_filter_type、char_filter_pattern 和 char_filter_replacement 来在分词前替换特定字符。
5. 性能考虑: TOKENIZE 函数主要用于测试和调试分词器配置。在生产环境的全文搜索中, 应使用带有 MATCH 谓词的倒排索引。
6. 与倒排索引的兼容性: 在 TOKENIZE 中使用的相同分词器配置可以应用于创建表时的倒排索引:

```
CREATE TABLE example (
content TEXT,
INDEX idx_content(content) USING INVERTED PROPERTIES("parser"="chinese")
)
```

7. 测试分词器行为: 使用 TOKENIZE 可以在创建倒排索引之前预览文本的分词效果, 有助于为您的数据选择最合适的分词器。

关键字

TOKENIZE, STRING, 全文搜索, 倒排索引, 分词器

7.2.1.2.64 TOP_LEVEL_DOMAIN

描述

TOP_LEVEL_DOMAIN 函数用于从 URL 中提取顶级域名。如果输入的 URL 不合法，则返回空字符串。

语法

```
TOP_LEVEL_DOMAIN(<url>)
```

参数

参数	说明
<url>	需要提取顶级域名的 URL 字符串。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示提取出的顶级域名。

特殊情况：- 如果 url 为 NULL，返回 NULL - 如果 url 不是合法的 URL 格式，返回空字符串 - 对于多级域名（如.com.cn），返回最后一级域名

示例

1. 基本域名处理

```
SELECT top_level_domain('www.baidu.com');
```

```
+-----------------------------------+
| top_level_domain('www.baidu.com') |
+-----------------------------------+
| com                               |
+-----------------------------------+
```

2. 多级域名处理

```
SELECT top_level_domain('www.google.com.cn');
```

```
+--------------------------------------+
| top_level_domain('www.google.com.cn') |
+--------------------------------------+
| cn                                   |
+--------------------------------------+
```

3. 无效 URL 处理

```
SELECT top_level_domain('wwwwwwww');
```

```
+------------------------------+
| top_level_domain('wwwwwwww') |
+------------------------------+
|                              |
+------------------------------+
```

7.2.1.2.65 TRANSLATE

描述

TRANSLATE 函数用于字符串替换，将源字符串中的字符按照映射规则进行转换。它会将源字符串中出现在 from 字符串中的字符替换为 to 字符串中对应位置的字符。

语法

```
TRANSLATE(<source>, <from>, <to>)
```

参数

参数	说明
<source>	需要进行转换的源字符串。类型：VARCHAR
<from>	要被替换的字符集合。类型：VARCHAR
<to>	替换后的字符集合。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示替换后的字符串。

特殊情况：- 如果任意参数为 NULL，返回 NULL - 如果 from 字符串中有重复字符，只使用第一次出现的位置 - 如果源字符不在 from 字符串中，该字符将保持不变 - 如果 from 字符串中字符的位置超出了 to 字符串的长度，对应的源字符将被删除 - 如果 from 和 to 都为空字符串，返回原字符串

示例

1. 基本字符替换

```
SELECT translate('abcd', 'a', 'z');
```

```
+------------------------------+
| translate('abcd', 'a', 'z') |
+------------------------------+
| zbcd                         |
+------------------------------+
```

2. 多次替换相同字符

```
SELECT translate('abcda', 'a', 'z');
```

```
+------------------------------+
| translate('abcda', 'a', 'z') |
+------------------------------+
| zbcdz                        |
+------------------------------+
```

3. 特殊字符替换

```
SELECT translate('Palhoça', 'ç', 'c');
```

```
+--------------------------------+
| translate('Palhoça', 'ç', 'c') |
+--------------------------------+
| Palhoca                        |
+--------------------------------+
```

4. 字符删除（to 字符串为空）

```
SELECT translate('abcd', 'a', '');
```

```
+----------------------------+
| translate('abcd', 'a', '') |
+----------------------------+
| bcd                        |
+----------------------------+
```

5. from 字符串中的重复字符

```
SELECT translate('abcd', 'aac', 'zq');
```

```
+--------------------------------+
| translate('abcd', 'aac', 'zq') |
+--------------------------------+
| zbd                            |
+--------------------------------+
```

7.2.1.2.66 TRIM

描述

用于删除字符串两端的空格或指定字符，当没有 rhs 参数时，将参数 str 中右侧和左侧开始部分连续出现的空格去掉，否则去掉 rhs

语法

```
RTIM( <str> [ , <rhs>])
```

必选参数

参数	描述
<str>	删除该字符串两端的空格

可选参数

参数	描述
<rhs>	去掉该指定字符

返回值

删除两端的空格或指定字符后的字符串

示例

```
SELECT trim('   ab d   ') str;
```

```
+------+
| str  |
+------+
| ab d |
+------+
```

```
SELECT trim('ababccaab','ab') str;
```

```
+------+
| str  |
+------+
| cca  |
+------+
```

7.2.1.2.67 TRIM_IN

描述

当没有 rhs 参数时，将参数 str 中右侧和左侧开始部分连续出现的空格去掉；当有 rhs 参数时，在字符串的两端查找并移除 rhs 字符集合中的任何字符（不考虑顺序）

语法

```
TRIM_IN( <str> [ , <rhs>])
```

必选参数

参数	描述
<str>	删除该字符串两端的空格

可选参数

参数	描述
<rhs>	去掉该指定字符

返回值

删除两端的空格或指定字符后的字符串

示例

1. 移除字符串两端的空格：

```
SELECT trim_in('   ab d   ') str;
```

```
+------+
| str  |
+------+
|  ab d|
+------+
```

```
SELECT trim_in('ababccaab','ab') str;
```

```
+------+
| str  |
+------+
| cc   |
+------+
```

7.2.1.2.68 UCASE/UPPER

描述

用于将字符串转换为大写字母

语法

```
UCASE( <str> )
UPPER( <str> )
```

必选参数

参数	描述
<str>	需转换为大写字母的字符串

返回值

转换大写字母后的值

示例

```
SELECT UCASE("aBc123"),UPPER("aBc123");
```

```
+-----------------+-----------------+
| UCASE("aBc123") | UPPER("aBc123") |
+-----------------+-----------------+
| ABC123          | ABC123          |
+-----------------+-----------------+
```

```
SELECT UCASE("Кириллица"),UPPER("Бәйтерек");
```

```
+----------------------------+---------------------------+
| UCASE("Кириллица")         | UPPER("Бәйтерек")         |
+----------------------------+---------------------------+
| КИРИЛЛИЦА                  | БӘЙТЕРЕК                  |
+----------------------------+---------------------------+
```

7.2.1.2.69 UNHEX

描述

unhex 函数用于将十六进制字符串转换为原始字符串。将每两个十六进制字符转换为一个字节。当传入的参数是无效值时，将返回空字符串。unhex_null 函数与unhex函数作用一致。但是当传入的参数是无效值时，将返回NULL。

> unhex_null 函数自 3.0.6 版本开始支持。

语法

```
UNHEX(<str>)
```

参数

参数	说明
<str>	16 进制字符字符串

返回值

输入字符串，如果字符串长度为 0 或者为奇数，unhex函数返回空串，unhex_null函数返回NULL；如果字符串

中包含[0-9]、[a-f]、[A-F]之外的字符，unhex函数返回空串，unhex_null函数返回NULL；其他情况每两个字符为一组转化为 16 进制后的字符，然后拼接成字符串输出。

举例

```
select unhex('@');
```

```
+------------+
| unhex('@') |
+------------+
|            |
+------------+
```

```
select unhex_null('@');
```

```
+-----------------+
| unhex_null('@') |
+-----------------+
| NULL            |
+-----------------+
```

```
select unhex('41');
```

```
+-------------+
| unhex('41') |
+-------------+
| A           |
+-------------+
```

```
select unhex('4142');
```

```
+---------------+
| unhex('4142') |
+---------------+
| AB            |
+---------------+
```

7.2.1.2.70 URL_DECODE

描述

将 URL 转换为解码字符串。

语法

```
URL_DECODE( <str> )
```

必选参数

参数	描述
<str>	待解码的字符串

返回值

解码后的值

示例

```
select  URL_DECODE('Doris+Q%26A');
```

```
+---------------------------+
| url_decode('Doris+Q%26A') |
+---------------------------+
| Doris Q&A                 |
+---------------------------+
```

描述

7.2.1.2.71 URL_ENCODE

使用 UTF-8 编码完成所提供文本的 URL 编码。通常用于对作为 URL 的一部分传递的参数信息进行编码

语法

```
URL_ENCODE(  <str>  )
```

必选参数

参数	描述
<str>	待编码的字符串

返回值

UTF-8 编码完成所提供文本的 URL 编码

示例

```
select  URL_ENCODE('Doris Q&A');
```

```
+-------------------------+
| url_encode('Doris Q&A') |
+-------------------------+
| Doris+Q%26A             |
+-------------------------+
```

7.2.1.2.72 UUID

描述

用于返回一个随机的 uuid 字符串

语法

```
UUID()
```

返回值

返回一个随机的 uuid 字符串

示例

```
select UUID();
```

```
+--------------------------------------+
| UUID()                               |
+--------------------------------------+
| 29077778-fc5e-4603-8368-6b5f8fd55c24 |
+--------------------------------------+
```

7.2.1.2.73 XPATH_STRING

描述

XPATH_STRING 函数用于解析 XML 字符串并返回第一个匹配 XPath 表达式的 XML 节点。

> 该函数自 3.0.6 版本开始支持.

语法

```
XPATH_STRING(<xml_string>, <xpath_expression>)
```

参数

参数	描述
<xml_string>	源字符串。类型：VARCHAR
<xpath_expression>	XPath 表达式。类型：VARCHAR

返回值

返回 VARCHAR 类型，表示匹配 XPath 表达式的第一个 XML 节点的内容。

特殊情况：- 如果 XML 或 XPath 格式不正确，函数会报错。

示例

1. 基本节点值提取

```
SELECT xpath_string('<a>123</a>', '/a');
```

```
+-----------------------------------+
| xpath_string('<a>123</a>', '/a')  |
+-----------------------------------+
| 123                               |
+-----------------------------------+
```

2. 嵌套元素提取

```
SELECT xpath_string('<a><b>123</b></a>', '/a/b');
```

```
+-------------------------------------------+
| xpath_string('<a><b>123</b></a>', '/a/b')  |
+-------------------------------------------+
| 123                                       |
+-------------------------------------------+
```

3. 使用属性

```
SELECT xpath_string('<a><b id="1">123</b></a>', '//b[@id="1"]');
```

```
+-----------------------------------------------------------+
| xpath_string('<a><b id="1">123</b></a>', '//b[@id="1"]') |
+-----------------------------------------------------------+
| 123                                                       |
+-----------------------------------------------------------+
```

4. 使用位置谓词

```
SELECT xpath_string('<a><b>1</b><b>2</b></a>', '/a/b[2]');
```

```
+---------------------------------------------------+
| xpath_string('<a><b>1</b><b>2</b></a>', '/a/b[2]') |
+---------------------------------------------------+
| 2                                                 |
+---------------------------------------------------+
```

5. 处理 CDATA 和注释

```
SELECT xpath_string('<a><![CDATA[123]]></a>', '/a'), xpath_string('<a><!-- comment -->123</a
    ↪ >', '/a');
```

```
+-----------------------------------------------+-----------------------------------------------+
    ↪
| xpath_string('<a><![CDATA[123]]></a>', '/a')  | xpath_string('<a><!-- comment -->123</a>',
    ↪  '/a')  |
+-----------------------------------------------+-----------------------------------------------+
    ↪
| 123                                           | 123
    ↪                                                 |
+-----------------------------------------------+-----------------------------------------------+
    ↪
```

7.2.1.3 时间日期函数

7.2.1.3.1 CONVERT_TZ

描述

转换 datetime 值，从 from_tz 给定时区转到 to_tz 给定时区，并返回结果值。特殊情况：- 如果参数无效该函数返回 NULL。

语法

```
CONVERT_TZ(<dt>, <from_tz>, <to_tz>)
```

参数

参数	说明
<dt>	需要被转换的 datetime 值
<from_tz>	dt 的原始时区
<to_tz>	需要转换的时区

返回值

转换后的 datetime 值

示例

```
select CONVERT_TZ('2019-08-01 13:21:03', 'Asia/Shanghai', 'America/Los_Angeles');
```

```
+------------------------------------------------------------------------+
| convert_tz('2019-08-01 13:21:03', 'Asia/Shanghai', 'America/Los_Angeles') |
+------------------------------------------------------------------------+
| 2019-07-31 22:21:03                                                    |
+------------------------------------------------------------------------+
```

```
select CONVERT_TZ('2019-08-01 13:21:03', '+08:00', 'America/Los_Angeles');
```

```
+-----------------------------------------------------------------------+
| convert_tz('2019-08-01 13:21:03', '+08:00', 'America/Los_Angeles') |
+-----------------------------------------------------------------------+
| 2019-07-31 22:21:03                                                   |
+-----------------------------------------------------------------------+
```

7.2.1.3.2 CURDATE,CURRENT_DATE

描述

获取当前的日期，以 DATE 类型返回。

别名

- curdate
- current_date

语法

```
CURDATE()
```

返回值

当前的日期。

示例

```
SELECT CURDATE();
```

```
+------------+
| CURDATE()  |
+------------+
| 2019-12-20 |
+------------+
```

```
SELECT CURDATE() + 0;
```

```
+---------------+
| CURDATE() + 0 |
+---------------+
|      20191220 |
+---------------+
```

7.2.1.3.3 CURTIME,CURRENT_TIME

Description

Retrieves the current time and returns it as a TIME type.

Alias

- CURTIME
- CURRENT_TIME

Syntax

```
CURTIME()
```

Return Value

Returns the current time.

Examples

```
mysql> select current_time();
```

```
+----------------+
| current_time() |
+----------------+
| 15:25:47       |
+----------------+
```

描述

7.2.1.3.4 DATE

提取日期时间中的日期。

语法

```
DATE(<datetime>)
```

参数

参数	说明
<datetime>	合法的日期表达式

返回值

返回日期时间的日期部分。

举例

```
select date('2010-12-02 19:28:30');
```

```
+-------------------------------------------------+
| date(cast('2010-12-02 19:28:30' as DATETIMEV2(0))) |
+-------------------------------------------------+
| 2010-12-02                                      |
+-------------------------------------------------+
```

7.2.1.3.5 DATE_ADD

描述

向日期添加指定的时间间隔。

别名

- date_add
- days_add
- adddate

语法

```
DATE_ADD(<date>, <expr> <time_unit>)
```

参数

参数	说明
<date>	合法的日期值
<expr>	希望添加的时间间隔
<time_unit>	枚举值：YEAR, QUARTER, MONTH, DAY, HOUR, MINUTE, SECOND

返回值

返回计算后的日期。

举例

```
select date_add('2010-11-30 23:59:59', INTERVAL 2 DAY);
```

```
+-------------------------------------------------+
| date_add('2010-11-30 23:59:59', INTERVAL 2 DAY) |
+-------------------------------------------------+
| 2010-12-02 23:59:59                             |
+-------------------------------------------------+
```

7.2.1.3.6 DATE_CEIL

描述

date_ceil 将日期转化为指定的时间间隔周期的最近上取整时刻。

语法

```
DATE_CEIL(<datetime>, INTERVAL <period> <type>)
```

参数

参数	说明
datetime	参数是合法的日期表达式
period	参数是指定每个周期有多少个单位组成，开始的时间起点为 0001-01-01T00:00:00
type	参数可以是：YEAR, QUARTER, MONTH, DAY, HOUR, MINUTE, SECOND

> QUARTER 支持从 3.0.8 和 3.1.0 版本开始。

返回值

返回的是一个日期或时间值，表示将输入值向上舍入到指定单位的结果。

举例

```
select date_ceil("2023-07-13 22:28:18",interval 5 second);
```

```
+-----------------------------------------------------------+
| second_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-13 22:28:20                                       |
+-----------------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 minute);
+-----------------------------------------------------------+
| minute_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-13 22:30:00                                       |
+-----------------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 hour);
```

```
+-----------------------------------------------------------+
| hour_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-13 23:00:00                                       |
+-----------------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 day);
```

```
+-----------------------------------------------------------+
| day_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-15 00:00:00                                       |
+-----------------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 month);
```

```
+-------------------------------------------------------------+
| month_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-------------------------------------------------------------+
| 2023-12-01 00:00:00                                         |
+-------------------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 quarter);
```

```
+-----------------------------------------------------+
| date_ceil("2023-07-13 22:28:18",interval 5 quarter) |
+-----------------------------------------------------+
| 2024-10-01 00:00:00                                 |
+-----------------------------------------------------+
```

```
select date_ceil("2023-07-13 22:28:18",interval 5 year);
```

```
+-----------------------------------------------------------+
| year_ceil('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2026-01-01 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.7 DATE_FLOOR

描述

`date_floor` 将日期转化为指定的时间间隔周期的最近下取整时刻。

语法

```
DATE_FLOOR(<datetime>, INTERVAL <period> <type>)
```

参数

参数	说明
`datetime`	参数是合法的日期表达式

参数	说明
period	参数是指定每个周期有多少个单位组成，开始的时间起点为 0001-01-01T00:00:00
type	参数可以是：YEAR, QUARTER, MONTH, DAY, HOUR, MINUTE, SECOND

QUARTER 支持从 3.0.8 和 3.1.0 版本开始。

返回值

返回的是一个日期或时间值，表示将输入值向下舍入到指定单位的结果。

举例

```
select date_floor("0001-01-01 00:00:16",interval 5 second);
```

```
+-----------------------------------------------------------+
| second_floor('0001-01-01 00:00:16', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 0001-01-01 00:00:15                                       |
+-----------------------------------------------------------+
```

```
select date_floor("0001-01-01 00:00:18",interval 5 second);
```

```
+-----------------------------------------------------------+
| second_floor('0001-01-01 00:00:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 0001-01-01 00:00:15                                       |
+-----------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 minute);
```

```
+-----------------------------------------------------------+
| minute_floor('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-13 22:25:00                                       |
+-----------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 hour);
```

```
+-----------------------------------------------------------+
| hour_floor('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-13 18:00:00                                       |
+-----------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 day);
```

```
+-----------------------------------------------------------+
| day_floor('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2023-07-10 00:00:00                                       |
+-----------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 month);
```

```
+-------------------------------------------------------------+
| month_floor('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-------------------------------------------------------------+
| 2023-07-01 00:00:00                                         |
+-------------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 quarter);
```

```
+-------------------------------------------------------+
| date_floor("2023-07-13 22:28:18",interval 5 quarter) |
+-------------------------------------------------------+
| 2023-07-01 00:00:00                                   |
+-------------------------------------------------------+
```

```
select date_floor("2023-07-13 22:28:18",interval 5 year);
```

```
+-----------------------------------------------------------+
| year_floor('2023-07-13 22:28:18', 5, '0001-01-01 00:00:00') |
+-----------------------------------------------------------+
| 2021-01-01 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.8 DATE_FORMAT

描述

将日期类型按照 format 的类型转化为字符串。

语法

```
DATE_FORMAT(<date>, <format>)
```

参数

参数	说明
<date>	合法的日期值
<format>	规定日期/时间的输出格式

支持的 format 格式：

格式符	描述
%a	三字母缩写星期名
%b	三字母缩写月名
%c	月，数值 (0-12)
%D	带有英文后缀的月中的天 (0th, 1st, 2nd, 3rd, …)
%d	月的天，数值 (00-31)
%e	月的天，数值 (0-31)
%f	微秒 (000000-999999)
%H	小时 (00-23)
%h	小时 (01-12)
%I	小时 (01-12)
%i	分钟，数值 (00-59)
%j	年的天 (001-366)
%k	小时 (0-23)
%l	小时 (1-12)
%M	月名
%m	月，数值 (00-12)
%p	AM 或 PM
%r	时间，12-小时（hh:mm:ss, 后跟 AM 或 PM）
%S	秒 (00-59)
%s	秒 (00-59)
%T	时间，24-小时 (hh:mm:ss)
%U	周 (00-53) 星期日是一周的第一天
%u	周 (00-53) 星期一是一周的第一天
%V	周 (01-53) 星期日是一周的第一天，与%X 使用
%v	周 (01-53) 星期一是一周的第一天，与%x 使用
%W	周中日的名称 (Sunday-Saturday)
%w	周的天（0= 星期日，6= 星期六）
%X	年，其中的星期日是周的第一天，4 位，与%V 使用
%x	年，其中的星期一是周的第一天，4 位，与%v 使用
%Y	年，4 位
%y	年，2 位
%%	用于表示%
%x	对于任何未出现在上列的 x，表示 x 本身

还可以使用三种特殊格式：

```
yyyyMMdd
```

```
yyyy-MM-dd
yyyy-MM-dd HH:mm:ss
```

返回值

格式化后的日期字符串，特殊情况：

- 当前支持最大 128 字节的字符串，如果返回值长度超过 128，则返回 NULL。

举例

```
select date_format('2009-10-04 22:23:00', '%W %M %Y');
```

```
+-----------------------------------------------+
| date_format('2009-10-04 22:23:00', '%W %M %Y') |
+-----------------------------------------------+
| Sunday October 2009                           |
+-----------------------------------------------+
```

```
select date_format('2007-10-04 22:23:00', '%H:%i:%s');
```

```
+-----------------------------------------------+
| date_format('2007-10-04 22:23:00', '%H:%i:%s') |
+-----------------------------------------------+
| 22:23:00                                      |
+-----------------------------------------------+
```

```
select date_format('1900-10-04 22:23:00', '%D %y %a %d %m %b %j');
```

```
+------------------------------------------------------------+
| date_format('1900-10-04 22:23:00', '%D %y %a %d %m %b %j') |
+------------------------------------------------------------+
| 4th 00 Thu 04 10 Oct 277                                   |
+------------------------------------------------------------+
```

```
select date_format('1997-10-04 22:23:00', '%H %k %I %r %T %S %w');
```

```
+------------------------------------------------------------+
| date_format('1997-10-04 22:23:00', '%H %k %I %r %T %S %w') |
+------------------------------------------------------------+
| 22 22 10 10:23:00 PM 22:23:00 00 6                         |
+------------------------------------------------------------+
```

```
select date_format('1999-01-01 00:00:00', '%X %V');
```

```
+-------------------------------------------+
| date_format('1999-01-01 00:00:00', '%X %V') |
+-------------------------------------------+
| 1998 52                                   |
+-------------------------------------------+
```

```
select date_format('2006-06-01', '%d');
```

```
+-----------------------------------------+
| date_format('2006-06-01 00:00:00', '%d') |
+-----------------------------------------+
| 01                                      |
+-----------------------------------------+
```

```
select date_format('2006-06-01', '%%%d');
```

```
+-------------------------------------------+
| date_format('2006-06-01 00:00:00', '%%%d') |
+-------------------------------------------+
| %01                                       |
+-------------------------------------------+
```

7.2.1.3.9 DATE_SUB

描述

从日期减去指定的时间间隔。

别名

- days_sub
- date_sub
- subdate

语法

```
DATE_ADD(<date>, <expr> <time_unit>)
```

参数

参数	说明
<date>	合法的日期值
<expr>	希望减去的时间间隔
<time_unit>	枚举值：YEAR, QUARTER, MONTH, DAY, HOUR, MINUTE, SECOND

返回值

返回计算后的日期。

举例

```
select date_sub('2010-11-30 23:59:59', INTERVAL 2 DAY);
```

```
+-----------------------------------------------+
| date_sub('2010-11-30 23:59:59', INTERVAL 2 DAY) |
+-----------------------------------------------+
| 2010-11-28 23:59:59                           |
+-----------------------------------------------+
```

7.2.1.3.10 DATE_TRUNC

描述

将 datetime 按照指定的时间单位截断。

语法

```
DATE_TRUNC(<datetime>, <time_unit>)
DATE_TRUNC(<time_unit>, <datetime>)
```

参数

参数	说明
<datetime>	合法的日期表达式
<time_unit>	希望截断的时间间隔，可选的值如下：[second,minute,hour,day,week,month,quarter,year]

返回值

按照指定的时间单位截断的时间

举例

```
select date_trunc('2010-12-02 19:28:30', 'second');
```

```
+-----------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'second')     |
+-----------------------------------------------+
| 2010-12-02 19:28:30                           |
+-----------------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'minute');
```

```
+-----------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'minute')     |
```

```
+-------------------------------------------------+
| 2010-12-02 19:28:00                             |
+-------------------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'hour');
```

```
+-------------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'hour')       |
+-------------------------------------------------+
| 2010-12-02 19:00:00                             |
+-------------------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'day');
```

```
+-------------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'day')        |
+-------------------------------------------------+
| 2010-12-02 00:00:00                             |
+-------------------------------------------------+
```

```
select date_trunc('2023-4-05 19:28:30', 'week');
```

```
+-----------------------------------------+
| date_trunc('2023-04-05 19:28:30', 'week') |
+-----------------------------------------+
| 2023-04-03 00:00:00                       |
+-----------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'month');
```

```
+-------------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'month')      |
+-------------------------------------------------+
| 2010-12-01 00:00:00                             |
+-------------------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'quarter');
```

```
+-------------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'quarter')    |
+-------------------------------------------------+
| 2010-10-01 00:00:00                             |
+-------------------------------------------------+
```

```
select date_trunc('2010-12-02 19:28:30', 'year');
```

```
+-----------------------------------------------+
| date_trunc('2010-12-02 19:28:30', 'year')     |
+-----------------------------------------------+
| 2010-01-01 00:00:00                           |
+-----------------------------------------------+
```

7.2.1.3.11 DATEDIFF

描述

计算给定日期之间的差值。

语法

```
DATEDIFF(<expr1>, <expr2>)
```

参数

参数	说明
<expr1>	日期被减数
<expr2>	日期减数

返回值

返回 expr1 - expr2 的值，结果精确到天。

举例

```
select datediff(CAST('2007-12-31 23:59:59' AS DATETIME), CAST('2007-12-30' AS DATETIME));
```

```
+-----------------------------------------------------------------------------+
| datediff(CAST('2007-12-31 23:59:59' AS DATETIME), CAST('2007-12-30' AS DATETIME)) |
+-----------------------------------------------------------------------------+
|                                                                           1 |
+-----------------------------------------------------------------------------+
```

```
select datediff(CAST('2010-11-30 23:59:59' AS DATETIME), CAST('2010-12-31' AS DATETIME));
```

```
+-----------------------------------------------------------------------------+
| datediff(CAST('2010-11-30 23:59:59' AS DATETIME), CAST('2010-12-31' AS DATETIME)) |
+-----------------------------------------------------------------------------+
|                                                                         -31 |
+-----------------------------------------------------------------------------+
```

7.2.1.3.12 DAY

描述

获得日期中的天信息，返回值范围从 1-31。

别名

- dayofmonth

语法

```
DAY(<dt>)
```

参数

参数	说明
<dt>	合法的日期表达式

返回值

返回日期中的天信息

举例

```
select day('1987-01-31');
```

```
+----------------------------+
| day('1987-01-31 00:00:00') |
+----------------------------+
|                         31 |
+----------------------------+
```

7.2.1.3.13 DAY_CEIL

描述

将日期转化为指定的时间间隔周期的最近向上取整时刻。

语法

```
DAY_CEIL(<datetime>)
DAY_CEIL(<datetime>, <origin>)
DAY_CEIL(<datetime>, <period>)
DAY_CEIL(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	合法的日期表达式
<period>	参数是指定每个周期有多少天组成
<origin>	开始的时间起点，如果不填，默认是 0001-01-01T00:00:00

返回值

返回最近向上取整时刻的日期。

举例

```
select day_ceil("2023-07-13 22:28:18", 5);
```

```
+-----------------------------------------------------------+
| day_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2023-07-17 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.14 DAY_FLOOR

描述

将日期转化为指定的时间间隔周期的最近向下取整时刻。

语法

```
DAY_FLOOR(<datetime>)
DAY_FLOOR(<datetime>, <origin>)
DAY_FLOOR(<datetime>, <period>)
DAY_FLOOR(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	合法的日期表达式
<period>	参数是指定每个周期有多少天组成
<origin>	开始的时间起点，如果不填，默认是 0001-01-01T00:00:00

返回值

返回最近向下取整时刻的日期。

举例

```
select day_floor("2023-07-13 22:28:18", 5);
```

```
+-----------------------------------------------------------+
| day_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2023-07-12 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.15 DAYNAME

描述

计算日期表达式对应的日期名字。

语法

```
DAYNAME(<dt>)
```

参数

参数	说明
<dt>	需要被计算的日期表达式

返回值

返回日期表达式对应的日期名字。

举例

```
select dayname('2007-02-03 00:00:00');
```

```
+--------------------------------+
| dayname('2007-02-03 00:00:00') |
+--------------------------------+
| Saturday                       |
+--------------------------------+
```

7.2.1.3.16 DAYOFWEEK

描述

返回日期的工作日索引值，即星期日为 1，星期一为 2，星期六为 7。

语法

```
DAYOFWEEK(<dt>)
```

参数

参数	说明
<dt>	需要被计算的日期表达式

返回值

返回日期的工作日索引值。

举例

```
select dayofweek('2019-06-25');
```

```
text +----------------------------------+ | dayofweek('2019-06-25 00:00:00')| +----------------------------------
↪  | 3 | +----------------------------------+text
```

```
select dayofweek(cast(20190625 as date));
```

```
+-----------------------------------+
| dayofweek(CAST(20190625 AS DATE)) |
+-----------------------------------+
|                                 3 |
+-----------------------------------+
```

7.2.1.3.17 DAYOFYEAR

描述

获得日期中对应当年中的哪一天。

语法

```
DAYOFYEAR(<dt>)
```

参数

参数	说明
<dt>	需要被计算的日期表达式

返回值

返回日期中对应当年中的哪一天。

举例

```
select dayofyear('2007-02-03 00:00:00');
```

```
+----------------------------------+
| dayofyear('2007-02-03 00:00:00') |
+----------------------------------+
```

```
|                                  34 |
+-------------------------------------+
```

7.2.1.3.18 EXTRACT

描述

extract 函数用于从日期或时间值中提取指定的部分，如年份、月份、日、小时、分钟、秒等。该函数常用于从日期时间字段中提取具体的时间组件进行计算、比较或展示。

语法

```
EXTRACT(<unit> FROM <datetime>)
```

参数

参数	说明
unit ↪	提取 DATETIME 某个指定单位的值。单位可以为 year, month, day, hour, minute, second 或者 microsecond

参数	说明
datetime ↪	参数是合法的日期表达式

返回值

返回的是提取出的日期或时间的某个部分（如整数），具体的部分取决于提取的单位。

举例

```
select extract(year from '2022-09-22 17:01:30') as year,
extract(month from '2022-09-22 17:01:30') as month,
extract(day from '2022-09-22 17:01:30') as day,
extract(hour from '2022-09-22 17:01:30') as hour,
extract(minute from '2022-09-22 17:01:30') as minute,
extract(second from '2022-09-22 17:01:30') as second,
extract(microsecond from cast('2022-09-22 17:01:30.000123' as datetimev2(6))) as microsecond;
```

```
+------+-------+------+------+--------+--------+-------------+
| year | month | day  | hour | minute | second | microsecond |
+------+-------+------+------+--------+--------+-------------+
| 2022 |     9 |   22 |   17 |      1 |     30 |         123 |
+------+-------+------+------+--------+--------+-------------+
```

7.2.1.3.19 FROM_DAYS

描述

给定一个天数，返回一个 DATE。

- 注意，为了和 mysql 保持一致的行为，不存在 0000-02-29 这个日期。

语法

```
FROM_DAYS(<dt>)
```

参数

参数	说明
<dt>	天数

返回值

返回对应天数的日期。

举例

```
select from_days(730669),from_days(5),from_days(59), from_days(60);
```

```
+------------------+--------------+--------------+--------------+
| from_days(730669) | from_days(5) | from_days(59) | from_days(60) |
+------------------+--------------+--------------+--------------+
| 2000-07-03        | 0000-01-05   | 0000-02-28    | 0000-03-01    |
+------------------+--------------+--------------+--------------+
```

描述

7.2.1.3.20 FROM_ISO8601_DATE

将 ISO8601 格式的日期表达式转化为 date 类型的日期表达式。

语法

```
from_iso8601_date(<dt>)
```

参数

参数	说明
<dt>	ISO8601 格式的日期

返回值

date 类型的日期表达式。

举例

```
SELECT from_iso8601_date('0000-01'),from_iso8601_date('0000-W01'),from_iso8601_date('0000-059');
```

```
+-----------------------------+-----------------------------+-----------------------------+
| from_iso8601_date('0000-01') | from_iso8601_date('0000-W01') | from_iso8601_date('0000-059') |
+-----------------------------+-----------------------------+-----------------------------+
| 0000-01-01                   | 0000-01-03                   | 0000-02-28                   |
+-----------------------------+-----------------------------+-----------------------------+
```

描述

7.2.1.3.21 FROM_MICROSECOND

函数将输入的 Unix 时间戳（以微秒为单位）转换为 DATETIME 类型的日期时间值。

语法

```
FROM_MICROSECOND(<unix_timestamp>)
```

参数

参数	说明
<unix_timestamp>	必填，输入的 Unix 时间戳，表示从 1970-01-01 00:00:00 UTC 开始的微秒数。

返回值

- 返回一个 DATETIME 类型的值，表示输入的 Unix 时间戳对应的日期时间。
- 如果 <unix_timestamp> 为 NULL，函数返回 NULL。
- 如果 <unix_timestamp> 超出有效范围，函数返回错误。

举例

```
SELECT FROM_MICROSECOND(1700000000000000);
```

```
+-----------------------------------+
| from_microsecond(1700000000000000) |
+-----------------------------------+
| 2023-11-15 06:13:20               |
+-----------------------------------+
```

描述

7.2.1.3.22 FROM_MILLISECOND

函数将输入的 Unix 时间戳（以毫秒为单位）转换为 DATETIME 类型的日期时间值。

语法

```
FROM_MILLISECOND(<millisecond>)
```

参数

参数	说明
<millisecond>	必填，输入的 Unix 时间戳，表示从 1970-01-01 00:00:00 UTC 开始的毫秒数。

返回值

- 返回一个 DATETIME 类型的值，表示输入的 Unix 时间戳对应的日期时间。

- 如果为 NULL，函数返回 NULL。
- 如果超出有效范围，函数返回错误。

举例

```
SELECT FROM_MILLISECOND(1700000000000);
```

```
+---------------------------------+
| from_millisecond(1700000000000) |
+---------------------------------+
| 2023-11-15 06:13:20             |
+---------------------------------+
```

7.2.1.3.23 FROM_SECOND

描述

函数将输入的 Unix 时间戳（以秒为单位）转换为 DATETIME 类型的日期时间值。

语法

```
FROM_SECOND(<unix_timestamp>)
```

参数

参数	说明
<unix_timestamp>	必填，输入的 Unix 时间戳，表示从 1970-01-01 00:00:00 UTC 开始的秒数。

返回值

- 返回一个 DATETIME 类型的值，表示输入的 Unix 时间戳对应的日期时间。
- 如果 <unix_timestamp> 为 NULL，函数返回 NULL。
- 如果 <unix_timestamp> 超出有效范围，函数返回错误。

举例

```
SELECT FROM_SECOND(1700000000);
```

```
+-------------------------+
| from_second(1700000000) |
+-------------------------+
| 2023-11-15 06:13:20     |
+-------------------------+
```

7.2.1.3.24 FROM_SECOND

描述

函数将输入的 Unix 时间戳（以秒为单位）转换为 DATETIME 类型的日期时间值。

语法

```
FROM_SECOND(<unix_timestamp>)
```

参数

参数	说明
<unix_timestamp>	必填，输入的 Unix 时间戳，表示从 1970-01-01 00:00:00 UTC 开始的秒数。

返回值

- 返回一个 DATETIME 类型的值，表示输入的 Unix 时间戳对应的日期时间。
- 如果 <unix_timestamp> 为 NULL，函数返回 NULL。
- 如果 <unix_timestamp> 超出有效范围，函数返回错误。

举例

```
SELECT FROM_SECOND(1700000000);
```

```
+-------------------------+
| from_second(1700000000) |
+-------------------------+
| 2023-11-15 06:13:20     |
+-------------------------+
```

7.2.1.3.25 FROM_UNIXTIME

描述

将 unix 时间戳转化为对应的 time 格式。特殊情况：

- 目前支持的 unix_timestamp 范围为 [0, 32536771199]，超出范围的 unix_timestamp 将会得到 NULL

语法

```
FROM_UNIXTIME(<unix_timestamp> [, <string_format>])
```

参数

参数	说明
<unix_timestamp>	unix 时间戳
<string_format>	format 格式，默认为%Y-%m-%d %H:%i:%s

返回值

返回指定格式的日期。

举例

```
mysql> select from_unixtime(1196440219);
+---------------------------+
| from_unixtime(1196440219) |
+---------------------------+
| 2007-12-01 00:30:19       |
+---------------------------+

mysql> select from_unixtime(1196440219, 'yyyy-MM-dd HH:mm:ss');
+--------------------------------------------------+
| from_unixtime(1196440219, 'yyyy-MM-dd HH:mm:ss') |
+--------------------------------------------------+
| 2007-12-01 00:30:19                              |
+--------------------------------------------------+

mysql> select from_unixtime(1196440219, '%Y-%m-%d');
+---------------------------------------+
| from_unixtime(1196440219, '%Y-%m-%d') |
+---------------------------------------+
| 2007-12-01                            |
+---------------------------------------+

mysql> select from_unixtime(1196440219, '%Y-%m-%d %H:%i:%s');
+------------------------------------------------+
| from_unixtime(1196440219, '%Y-%m-%d %H:%i:%s') |
+------------------------------------------------+
| 2007-12-01 00:30:19                            |
+------------------------------------------------+
```

7.2.1.3.26 HOUR

描述

获得日期时间中的小时的信息。

语法

```
HOUR(<dt>)
```

参数

参数	说明
<dt>	需要计算的日期

返回值

返回日期中的小时的信息。返回值范围从 0-23。当参数为 Time，返回值可以大于 24

举例

```
select hour('2018-12-31 23:59:59');
```

```
+-----------------------------+
| hour('2018-12-31 23:59:59') |
+-----------------------------+
|                          23 |
+-----------------------------+
```

```
select cast(4562632 as time),hour(cast(4562632 as time)), minute(cast(4562632 as time)),second(
    ↪ cast(4562632 as time));
```

```
+----------------------+----------------------------+------------------------------+----------------------------
    ↪
| cast(4562632 as TIME) | hour(cast(4562632 as TIME)) | minute(cast(4562632 as TIME)) | second(
    ↪ cast(4562632 as TIME)) |
+----------------------+----------------------------+------------------------------+----------------------------
    ↪
| 456:26:32            |                        456 |                           26 |
    ↪                            32 |
+----------------------+----------------------------+------------------------------+----------------------------
    ↪
```

7.2.1.3.27 HOUR_CEIL

描述

将日期转化为指定的时间间隔周期的最近向上取整时刻。

语法

```
HOUR_CEIL(<datetime>)
HOUR_CEIL(<datetime>, <origin>)
HOUR_CEIL(<datetime>, <period>)
HOUR_CEIL(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	合法的日期表达式
<period>	指定每个周期有多少小时组成
<origin>	开始的时间起点，如果不填，默认是 0001-01-01T00:00:00

返回值

返回指定的时间间隔周期的最近向上取整时刻。

举例

```
mysql> select hour_ceil("2023-07-13 22:28:18", 5);
+-----------------------------------------------------------+
| hour_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2023-07-14 02:00:00                                       |
+-----------------------------------------------------------+
1 row in set (0.03 sec)
```

7.2.1.3.28 HOUR_FLOOR

描述

将日期转化为指定的时间间隔周期的最近向下取整时刻。

语法

```
HOUR_FLOOR(<datetime>)
HOUR_FLOOR(<datetime>, <origin>)
HOUR_FLOOR(<datetime>, <period>)
HOUR_FLOOR(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	合法的日期表达式
<period>	指定每个周期有多少小时组成
<origin>	开始的时间起点，如果不填，默认是 0001-01-01T00:00:00

返回值

返回指定的时间间隔周期的最近向下取整时刻。

举例

```
select hour_floor("2023-07-13 22:28:18", 5);
```

```
+-----------------------------------------------------------+
| hour_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2023-07-13 21:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.29 HOURS_ADD

描述

返回一个新的日期时间值，该值是在输入的日期时间上增加指定的小时数。

语法

```
HOURS_ADD(<date>, <hours>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIME 或 DATE
<hours>	要增加的小时数，类型为 INT

返回值

返回类型为 DATETIME，返回以输入日期时间为基准，增加指定小时数后的时间值。

举例

```
SELECT HOURS_ADD('2020-02-02 02:02:02', 1);
```

```
+-----------------------------------------------------------+
| hours_add(cast('2020-02-02 02:02:02' as DATETIMEV2(0)), 1) |
+-----------------------------------------------------------+
| 2020-02-02 03:02:02                                       |
+-----------------------------------------------------------+
```

7.2.1.3.30 HOURS_DIFF

描述

开始时间到结束时间相差几小时。

语法

```
HOURS_DIFF(<end_date>, <start_date>)
```

参数

参数	说明
<end_date>	结束时间，类型为 DATETIME 或 DATE
<start_date>	开始时间，类型为 DATETIME 或 DATE

返回值

返回类型为 INT，返回开始时间到结束时间相差的小时数。

举例

```
SELECT HOURS_DIFF('2020-12-25 22:00:00', '2020-12-25 21:00:00');
```

```
+-----------------------------------------------------------------------------------------------------------------+
    ↪
| hours_diff(cast('2020-12-25 22:00:00' as DATETIMEV2(0)), cast('2020-12-25 21:00:00' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 1 |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.3.31 HOURS_SUB

描述

返回一个新的日期时间值，该值是从输入的日期时间中减去指定的小时数。

语法

```
HOURS_SUB(<date>, <hours>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIME 或 DATE
<hours>	要减去的小时数，类型为 INT

返回值

返回类型为 DATETIME，返回以输入日期时间为基准，减去指定小时数后的时间值。

举例

```
SELECT HOURS_SUB('2020-02-02 02:02:02', 1);
```

```
+-----------------------------------------------------------+
| hours_sub(cast('2020-02-02 02:02:02' as DATETIMEV2(0)), 1) |
+-----------------------------------------------------------+
| 2020-02-02 01:02:02                                       |
+-----------------------------------------------------------+
```

7.2.1.3.32 LAST_DAY

描述

返回输入日期所在月份的最后一天的日期。根据不同月份，返回日期的具体日期值为：

- 28 日：非闰年的二月
- 29 日：闰年的二月
- 30 日：四月、六月、九月、十一月
- 31 日：一月、三月、五月、七月、八月、十月、十二月

语法

```
LAST_DAY(<date>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIME 或 DATE

返回值

返回类型为 DATE，返回输入日期所在月份的最后一天的日期。

举例

```
SELECT LAST_DAY('2000-02-03');
```

```
+----------------------------------------------+
| last_day(cast('2000-02-03' as DATETIMEV2(0))) |
+----------------------------------------------+
| 2000-02-29                                   |
+----------------------------------------------+
```

7.2.1.3.33 MAKEDATE

描述

根据指定的年份和一年中的天数（dayofyear）构建并返回一个日期。

特殊情况：- 当 <day_of_year> 小于等于 0 时，返回 NULL - 当 <day_of_year> 超过当年天数时，会自动往后顺延到下一年

语法

```
MAKEDATE(<year>, <day_of_year>)
```

参数

参数	说明
year	指定的年份，类型为 INT
dayofyear	一年中的第几天（1-366），类型为 INT

返回值

返回类型为 DATE，返回以指定年份和一年中的第几天构建的日期。

举例

```
SELECT MAKEDATE(2021, 1), MAKEDATE(2021, 100), MAKEDATE(2021, 400);
```

```
+------------------+--------------------+--------------------+
| makedate(2021, 1) | makedate(2021, 100) | makedate(2021, 400) |
+------------------+--------------------+--------------------+
| 2021-01-01        | 2021-04-10          | 2022-02-04          |
+------------------+--------------------+--------------------+
```

7.2.1.3.34 MICROSECOND

描述

从日期时间值中提取微秒部分的值。返回的范围是 0 到 999999。

语法

```
MICROSECOND(<date>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIMEV2，精度需要大于 0

返回值

返回类型为 INT，返回日期时间值中的微秒部分。取值范围为 0 到 999999。对于精度小于 6 的输入，不足的位数补 0。

举例

```
SELECT MICROSECOND(CAST('1999-01-02 10:11:12.000123' AS DATETIMEV2(6))) AS microsecond;
```

```
+-------------+
| microsecond |
+-------------+
|         123 |
+-------------+
```

描述

7.2.1.3.35 MICROSECOND_TIMESTAMP

函数将输入的 DATETIME 值转换为从 1970-01-01 00:00:00 UTC 开始的 Unix 时间戳（以微秒为单位）。

语法

```
MICROSECOND_TIMESTAMP(<datetime>)
```

参数

参数	说明
<datetime>	必填，输入的 DATETIME 值，表示要转换为 Unix 时间戳的日期时间。

返回值

返回一个整数，表示输入日期时间对应的 Unix 时间戳（以微秒为单位）。如果 <datetime> 为 NULL，函数返回 NULL。如果 <datetime> 超出有效范围，函数可能返回错误或异常值。

举例

```
SELECT MICROSECOND_TIMESTAMP('2025-01-23 12:34:56.123456');
```

```
+-----------------------------------------------------------------------------+
| microsecond_timestamp(cast('2025-01-23 12:34:56.123456' as DATETIMEV2(6))) |
+-----------------------------------------------------------------------------+
|                                                           1737606896123456 |
+-----------------------------------------------------------------------------+
```

7.2.1.3.36 MICROSECONDS_ADD

描述

向日期时间值中添加指定的微秒数，返回一个新的日期时间值。

语法

```
MICROSECONDS_ADD(<basetime>, <delta>)
```

参数

参数	说明
<basetime>	输入的日期时间值，类型为 DATETIMEV2
<delta>	要添加的微秒数，类型为 INT，1 秒 = 1,000,000 微秒

返回值

返回类型为 DATETIMEV2，返回以输入日期时间为基准，增加指定微秒数后的时间值。返回值的精度与输入参数 basetime 的精度相同。

举例

```
SELECT NOW(3) AS current_time, MICROSECONDS_ADD(NOW(3), 100000) AS after_add;
```

```
+-------------------------+----------------------------+
| current_time            | after_add                  |
+-------------------------+----------------------------+
| 2025-01-16 11:48:10.505 | 2025-01-16 11:48:10.605000 |
+-------------------------+----------------------------+
```

注意：- NOW(3) 返回精度为 3 位小数的当前时间 - 添加 100000 微秒（0.1 秒）后，时间增加了 0.1 秒

7.2.1.3.37 MICROSECONDS_DIFF

描述

计算两个日期时间值之间的微秒差值。结果为 <end_date> 减去 <start_date> 的微秒数。

语法

```
MICROSECONDS_DIFF(<end_date>, <start_date>)
```

参数

参数	说明
<end_date>	结束时间，类型为 DATETIMEV2
<start_date>	开始时间，类型为 DATETIMEV2

返回值

返回类型为 INT，表示两个时间之间的微秒差值。- 如果 <end_date> 大于 <start_date>，返回正数 - 如果 <end_date> 小于 <start_date>，返回负数 - 1 秒 = 1,000,000 微秒

举例

```
SELECT MICROSECONDS_DIFF('2020-12-25 21:00:00.623000','2020-12-25 21:00:00.123000');
```

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| microseconds_diff(cast('2020-12-25 21:00:00.623000' as DATETIMEV2(3)), cast('2020-12-25
    ↪ 21:00:00.123000' as DATETIMEV2(3))) |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 500000 |
```

```
+-----------------------------------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.3.38 MICROSECONDS_SUB

描述

从日期时间值中减去指定的微秒数，返回一个新的日期时间值。

语法

```
MICROSECONDS_SUB(<basetime>, <delta>)
```

参数

参数	说明
<basetime>	输入的日期时间值，类型为 DATETIMEV2
<delta>	要减去的微秒数，类型为 INT，1 秒 = 1,000,000 微秒

返回值

返回类型为 DATETIMEV2，返回以输入日期时间为基准，减去指定微秒数后的时间值。返回值的精度与输入参数 basetime 的精度相同。

举例

```
SELECT NOW(3) AS current_time, MICROSECONDS_SUB(NOW(3), 100000) AS after_sub;
```

```
+-------------------------+----------------------------+
| current_time            | after_sub                  |
+-------------------------+----------------------------+
| 2025-01-16 11:52:22.296 | 2025-01-16 11:52:22.196000 |
+-------------------------+----------------------------+
```

注意： - NOW(3) 返回精度为 3 位小数的当前时间 - 减去 100000 微秒（0.1 秒）后，时间减少了 0.1 秒 - 函数的计算结果与输入时间的精度有关

描述

7.2.1.3.39 MILLISECOND_TIMESTAMP

函数将输入的 DATETIME 值转换为从 1970-01-01 00:00:00 UTC 开始的 Unix 时间戳（以毫秒为单位）。

语法

```
MILLISECOND_TIMESTAMP(<datetime>)
```

参数

参数	说明
<datetime>	必填，输入的 DATETIME 值，表示要转换为 Unix 时间戳（毫秒）的日期时间。

返回值

- 返回一个整数，表示输入日期时间对应的 Unix 时间戳（以毫秒为单位）。
- 如果 <datetime> 为 NULL，函数返回 NULL。
- 如果 <datetime> 超出有效范围，函数可能返回错误或异常值。

举例

```
SELECT MILLISECOND_TIMESTAMP('2025-01-23 12:34:56');
```

```
+-----------------------------------------------------------------------+
| millisecond_timestamp(cast('2025-01-23 12:34:56' as DATETIMEV2(0))) |
+-----------------------------------------------------------------------+
|                                                         1737606896000 |
+-----------------------------------------------------------------------+
```

7.2.1.3.40 MILLISECONDS_ADD

描述

向日期时间值中添加指定的毫秒数，返回一个新的日期时间值。

语法

```
MILLISECONDS_ADD(<basetime>, <delta>)
```

参数

参数	说明
<basetime>	输入的日期时间值，类型为 DATETIMEV2
<delta>	要添加的毫秒数，类型为 INT，1 秒 = 1,000 毫秒 = 1,000,000 微秒

返回值

返回类型为 DATETIMEV2，返回以输入日期时间为基准，增加指定毫秒数后的时间值。返回值的精度与输入参数 basetime 的精度相同。

举例

```
SELECT MILLISECONDS_ADD('2023-09-08 16:02:08.435123', 1);
```

```
+-----------------------------------------------------------------------------+
| milliseconds_add(cast('2023-09-08 16:02:08.435123' as DATETIMEV2(6)), 1) |
```

```
+----------------------------------------------------------------------------+
| 2023-09-08 16:02:08.436123                                                 |
+----------------------------------------------------------------------------+
```

注意：- 示例中添加 1 毫秒后，时间从.435123 增加到.436123 - 1 毫秒等于 1000 微秒 - 函数的计算结果与输入时间的精度有关，示例使用了 6 位小数精度

7.2.1.3.41 MILLISECONDS_DIFF

描述

计算两个日期时间值之间的毫秒差值。结果为 <end_date> 减去 <start_date> 的毫秒数。

语法

```
MILLISECONDS_DIFF(<enddate>, <startdate>)
```

参数

参数	说明
<end_date>	结束时间，类型为 DATETIMEV2
<start_date>	开始时间，类型为 DATETIMEV2

返回值

返回类型为 INT，表示两个时间之间的毫秒差值。- 如果 <end_date> 大于 <start_date>，返回正数 - 如果 <end_date> 小于 <start_date>，返回负数 - 1 秒 = 1,000 毫秒 - 1 毫秒 = 1,000 微秒

举例

```
SELECT MILLISECONDS_DIFF('2020-12-25 21:00:00.623000', '2020-12-25 21:00:00.123000');
```

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| milliseconds_diff(cast('2020-12-25 21:00:00.623000' as DATETIMEV2(3)), cast('2020-12-25
    ↪ 21:00:00.123000' as DATETIMEV2(3))) |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 500 |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
```

注意：- 示例中的时间差为 0.5 秒，即 500 毫秒 - 函数的计算结果与输入时间的精度有关，示例使用了 3 位小数精度 - 结果只返回毫秒差值，不包含微秒部分

7.2.1.3.42 MILLISECONDS_SUB

描述

从日期时间值中减去指定的毫秒数，返回一个新的日期时间值。

语法

```
MILLISECONDS_SUB(<basetime>, <delta>)
```

参数

参数	说明
<basetime>	输入的日期时间值，类型为 DATETIMEV2
<delta>	要减去的毫秒数，类型为 INT，1 秒 = 1,000 毫秒 = 1,000,000 微秒

返回值

返回类型为 DATETIMEV2，返回以输入日期时间为基准，减去指定毫秒数后的时间值。返回值的精度与输入参数 basetime 的精度相同。

举例

```
SELECT MILLISECONDS_SUB('2023-09-08 16:02:08.435123', 1);
```

```
+-----------------------------------------------------------------------------+
| milliseconds_sub(cast('2023-09-08 16:02:08.435123' as DATETIMEV2(6)), 1) |
+-----------------------------------------------------------------------------+
| 2023-09-08 16:02:08.434123                                                  |
+-----------------------------------------------------------------------------+
```

注意：- 示例中减去 1 毫秒后，时间从.435123 减少到.434123 - 1 毫秒等于 1000 微秒 - 函数的计算结果与输入时间的精度有关，示例使用了 6 位小数精度 - 结果保留了微秒级别的精度

7.2.1.3.43 MINUTE

描述

从日期时间值中提取分钟部分的值。返回值范围为 0 到 59。

语法

```
MINUTE(<date>)
```

参数

参数	说明
<date>	输入的日期时间值，类型可以是 DATE、DATETIME、DATETIMEV2 或 TIME

返回值

返回类型为 INT，表示分钟值，取值范围为 0-59。

举例

```
SELECT MINUTE('2018-12-31 23:59:59');
```

```
+-----------------------------------------------------+
| minute(cast('2018-12-31 23:59:59' as DATETIMEV2(0))) |
+-----------------------------------------------------+
|                                                  59 |
+-----------------------------------------------------+
```

注意：- 输入参数可以是多种时间相关类型 - 返回值始终是 0-59 之间的整数 - 如果输入参数为 NULL，则返回 NULL

7.2.1.3.44 MINUTE_CEIL

描述

将日期时间向上取整到最近的指定分钟周期。如果指定了起始时间（ origin ），则以该时间为基准计算周期。

语法

```
MINUTE_CEIL(<datetime>)
MINUTE_CEIL(<datetime>, <origin>)
MINUTE_CEIL(<datetime>, <period>)
MINUTE_CEIL(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	需要向上取整的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	分钟周期值，类型为 INT，表示每个周期包含的分钟数
<origin>	周期的起始时间点，类型为 DATETIME 或 DATETIMEV2，默认值为 0001-01-01 00:00:00

返回值

返回类型为 DATETIMEV2，返回以输入日期时间为基准，向上取整到最近的指定分钟周期后的时间值。返回值的精度与输入参数 datetime 的精度相同。

举例

```
SELECT MINUTE_CEIL("2023-07-13 22:28:18", 5);
```

```
+-----------------------------------------------------------+
| minute_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
```

```
| 2023-07-13 22:30:00                                          |
+--------------------------------------------------------------+
```

注意：- 不指定 period 时，默认以 1 分钟为周期 - period 必须为正整数 - 结果总是向未来时间取整

最佳实践

还可参阅 date_ceil

7.2.1.3.45 MINUTE_FLOOR

描述

将日期时间向下取整到最近的指定分钟周期。如果指定了起始时间（origin），则以该时间为基准计算周期。

语法

```
MINUTE_FLOOR(<datetime>)
MINUTE_FLOOR(<datetime>, <origin>)
MINUTE_FLOOR(<datetime>, <period>)
MINUTE_FLOOR(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	需要向下取整的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	分钟周期值，类型为 INT，表示每个周期包含的分钟数
<origin>	周期的起始时间点，类型为 DATETIME 或 DATETIMEV2，默认值为 0001-01-01 00:00:00

返回值

返回类型为 DATETIMEV2，表示向下取整后的日期时间值。

举例

```
SELECT MINUTE_FLOOR("2023-07-13 22:28:18", 5);
```

```
+--------------------------------------------------------------+
| minute_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+--------------------------------------------------------------+
| 2023-07-13 22:25:00                                          |
+--------------------------------------------------------------+
```

注意：- 不指定 period 时，默认以 1 分钟为周期 - period 必须为正整数 - 结果总是向过去时间取整 - 与 MINUTE_CEIL 相反，MINUTE_FLOOR 总是舍去超出周期的部分

最佳实践

还可参阅 date_floor

7.2.1.3.46 MINUTES_ADD

描述

向日期时间值中添加指定的分钟数，返回一个新的日期时间值。

语法

```
MINUTES_ADD(<date>, <minutes>)
```

参数

参数	说明
<date>	输入的日期时间值，类型可以是 DATE、DATETIME 或 DATETIMEV2
<minutes>	要添加的分钟数，类型为 INT，可以为正数或负数

返回值

返回类型为 DATETIME，表示添加指定分钟数后的日期时间值。

举例

```
SELECT MINUTES_ADD("2020-02-02", 1);
```

```
+-----------------------------------------------------+
| minutes_add(cast('2020-02-02' as DATETIMEV2(0)), 1) |
+-----------------------------------------------------+
| 2020-02-02 00:01:00                                 |
+-----------------------------------------------------+
```

注意：- 当添加的分钟数为负数时，相当于减去对应的分钟数 - 函数会自动处理跨小时、跨天的情况 - 如果输入参数为 NULL，则返回 NULL

7.2.1.3.47 MINUTES_DIFF

描述

计算两个日期时间值之间的分钟差值。结果为 <end_date> 减去 <start_date> 的分钟数。

语法

```
MINUTES_DIFF(<end_date>, <start_date>)
```

参数

参数	说明
<end_date>	结束时间，类型可以是 DATE、DATETIME 或 DATETIMEV2
<start_date>	开始时间，类型可以是 DATE、DATETIME 或 DATETIMEV2

返回值

返回类型为 INT，表示两个时间之间的分钟差值。- 如果 <end_date> 大于 <start_date>，返回正数 - 如果 <end_date> 小于 <start_date>，返回负数

举例

```
SELECT MINUTES_DIFF('2020-12-25 22:00:00', '2020-12-25 21:00:00');
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| minutes_diff(cast('2020-12-25 22:00:00' as DATETIMEV2(0)), cast('2020-12-25 21:00:00' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 60 |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

注意：- 计算只考虑完整的分钟数，秒和毫秒部分会被忽略 - 如果任一输入参数为 NULL，则返回 NULL - 可以处理跨天、跨月、跨年的时间差计算

7.2.1.3.48 MINUTES_SUB

描述

从日期时间值中减去指定的分钟数，返回一个新的日期时间值。

语法

```
MINUTES_SUB(<date>, <minutes>)
```

参数

参数	说明
<date>	输入的日期时间值，类型可以是 DATE、DATETIME 或 DATETIMEV2
<minutes>	要减去的分钟数，类型为 INT，可以为正数或负数

返回值

返回类型为 DATETIME，表示减去指定分钟数后的日期时间值。

举例

```
SELECT MINUTES_SUB("2020-02-02 02:02:02", 1);
```

```
+---------------------------------------------------------+
```

```
| minutes_sub(cast('2020-02-02 02:02:02' as DATETIMEV2(0)), 1) |
+--------------------------------------------------------------+
| 2020-02-02 02:01:02                                          |
+--------------------------------------------------------------+
```

注意：- 当减去的分钟数为负数时，相当于加上对应的分钟数 - 函数会自动处理跨小时、跨天的情况 - 如果输入参数为 NULL，则返回 NULL - 结果会保留原始时间的秒数部分

7.2.1.3.49 MONTH

描述

从日期时间值中提取月份值。返回值范围为 1 到 12，分别代表一年中的 12 个月。

语法

```
MONTH(<date>)
```

参数

参数	说明
<date>	输入的日期时间值，类型可以是 DATE、DATETIME 或 DATETIMEV2

返回值

返回类型为 INT，表示月份值：- 范围：1 到 12 - 1 表示一月，12 表示十二月 - 如果输入为 NULL，返回 NULL

举例

```
SELECT MONTH('1987-01-01');
```

```
+-------------------------------------------+
| month(cast('1987-01-01' as DATETIMEV2(0))) |
+-------------------------------------------+
|                                         1 |
+-------------------------------------------+
```

7.2.1.3.50 MONTH_CEIL

描述

将日期时间值向上取整到最近的指定月份周期。如果指定了起始时间（origin），则以该时间为基准计算周期。

语法

```
MONTH_CEIL(<datetime>)
MONTH_CEIL(<datetime>, <origin>)
MONTH_CEIL(<datetime>, <period>)
MONTH_CEIL(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	需要向上取整的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	月份周期值，类型为 INT，表示每个周期包含的月数
<origin>	周期的起始时间点，类型为 DATETIME 或 DATETIMEV2，默认值为 0001-01-01 00:00:00

返回值

返回类型为 DATETIME，表示向上取整后的日期时间值。结果的时间部分将被设置为 00:00:00。

举例

```
SELECT MONTH_CEIL("2023-07-13 22:28:18", 5);
```

```
+------------------------------------------------------------+
| month_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+------------------------------------------------------------+
| 2023-10-01 00:00:00                                        |
+------------------------------------------------------------+
```

注意：- 不指定 period 时，默认以 1 个月为周期 - period 必须为正整数 - 结果总是向未来时间取整 - 返回值的时间部分总是 00:00:00

最佳实践

还可参阅 date_ceil

7.2.1.3.51 MONTH_FLOOR

描述

将日期时间值向下取整到最近的指定月份周期。如果指定了起始时间（ origin ），则以该时间为基准计算周期。

语法

```
MONTH_FLOOR(<datetime>)
MONTH_FLOOR(<datetime>, <origin>)
MONTH_FLOOR(<datetime>, <period>)
MONTH_FLOOR(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	需要向下取整的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	月份周期值，类型为 INT，表示每个周期包含的月数
<origin>	周期的起始时间点，类型为 DATETIME 或 DATETIMEV2，默认值为 0001-01-01 00:00:00

返回值

返回类型为 DATETIME，表示向下取整后的日期时间值。结果的时间部分将被设置为 00:00:00。

举例

```
SELECT MONTH_FLOOR("2023-07-13 22:28:18", 5);
```

```
+-------------------------------------------------------------+
| month_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-------------------------------------------------------------+
| 2023-05-01 00:00:00                                         |
+-------------------------------------------------------------+
```

注意：- 不指定 period 时，默认以 1 个月为周期 - period 必须为正整数 - 结果总是向过去时间取整 - 返回值的时间部分总是 00:00:00 - 与 MONTH_CEIL 相反，MONTH_FLOOR 总是舍去超出周期的部分

最佳实践

还可参阅 date_floor

7.2.1.3.52 MONTHNAME

描述

返回日期对应的英文月份名称。返回值为完整的英文月份名称（January 到 December）。

语法

```
MONTHNAME(<date>)
```

参数

参数	说明
<date>	输入的日期时间值，类型可以是 DATE、DATETIME 或 DATETIMEV2

返回值

返回类型为 VARCHAR，表示月份的英文名称：- 返回值范围：January, February, March, April, May, June, July, August, September, October, November, December - 如果输入为 NULL，返回 NULL - 返回值首字母大写，其余字母小写

举例

```
SELECT MONTHNAME('2008-02-03 00:00:00');
```

```
+-----------------------------------------------------+
| monthname(cast('2008-02-03 00:00:00' as DATETIMEV2(0))) |
+-----------------------------------------------------+
| February                                            |
+-----------------------------------------------------+
```

7.2.1.3.53 MONTHS_ADD

描述

MONTHS_ADD 函数用于对指定日期添加或减去指定的月份数，并返回计算后的新日期

语法

```
MONTHS_ADD(<datetime/date>, <nums>)
```

参数

参数	说明
<datetime/date>	需要被计算加减月份的日期值
<nums>	需要加减的月份数

返回值

返回值与输入的类型一致特殊情况：- 输入为 0000-00-00 或 0000-00-00 00:00:00 时，返回 NULL - 输入为 NULL 时，返回 NULL - 如果输入为 months_add(“9999-12-31”,1)，将返回 NULL

举例

```
select months_add("2020-01-31 02:02:02", 1),months_add("2020-01-31", 1),months_add("2020-01-31",
    ↪ -1);
```

```
+-------------------------------------------------------------+-----------------------------------------+----
    ↪
| months_add(cast('2020-01-31 02:02:02' as DATETIMEV2(0)), 1) | months_add(cast('2020-01-31' as
    ↪ DATEV2), 1) | months_add(cast('2020-01-31' as DATEV2), -1) |
+-------------------------------------------------------------+-----------------------------------------+----
    ↪
| 2020-02-29 02:02:02                                         | 2020-02-29
    ↪                                  | 2019-12-31                                      |
+-------------------------------------------------------------+-----------------------------------------+----
    ↪
```

7.2.1.3.54 MONTHS_BETWEEN

描述

MONTHS_BETWEEN 函数计算两个日期之间的月份数（浮点数）。它接收两个日期参数和一个可选的布尔参数。

> 该函数自 3.0.6 版本开始支持.

注意： 当 <enddate> 和 <startdate> 都是各自月份的最后一天时，函数会进行特殊处理。它会返回完整的月份差值，而不考虑基于天数的分数部分。这确保了在比较一个月末与另一个月末时的一致性。

例如：-months_between('2024-01-31', '2024-02-29') 将返回 -1.0，因为两个日期都是各自月份的最后一天（1 月 31 日和 2 月 29 日），所以结果被视为完整的月份差值，不进行分数调整。-months_between('2024-01-29', '2024-02-29') 也将返回 -1.0，因为月份中的日期相同。-months_between('2024-01-30', '2024-02-29') 将返回 -0.96774194，因为月份中的日期不同且不是月末。

语法

```
MONTHS_BETWEEN(<enddate>, <startdate> [, <round_type>])
```

参数

参数	说明
<enddate>	结束日期，表示差值计算中的较晚日期。支持 DATE（如 YYYY-MM-DD）或 DATETIME（如 YYYY-MM-DD HH:MM:SS）类型。
<startdate>	开始日期，表示差值计算中的较早日期。支持 DATE（如 YYYY-MM-DD）或 DATETIME（如 YYYY-MM-DD HH:MM:SS）类型。
<round_type>	是否将结果四舍五入到第八位小数。支持 true 或 false。默认为 true。

返回值

返回 <enddate> 减去 <startdate> 得到的月份数（浮点数）

结果 = (<enddate>.year - <startdate>.year) * 12 + <enddate>.month - <startdate>.month + (<enddate>.day - <startdate>.day) / 31.0

- 当 <enddate> 或 <startdate> 为 NULL，或两者都为 NULL 时，返回 NULL
- 当 <round_type> 为 true 时，结果四舍五入到第八位小数。

示例

```
select months_between('2020-12-26','2020-10-25'),months_between('2020-10-25 10:00:00','2020-12-26
    ↪  11:00:00',false);
```

```
+-----------------------------------------+-----------------------------------------------------------------
    ↪
| months_between('2020-12-26','2020-10-25') | months_between('2020-10-25 10:00:00','2020-12-26
    ↪ 11:00:00',false) |
+-----------------------------------------+-----------------------------------------------------------------
    ↪
|                                2.03225806 |
    ↪ -2.032258064516129 |
+-----------------------------------------+-----------------------------------------------------------------
    ↪
```

7.2.1.3.55 MONTHS_DIFF

描述

MONTHS_DIFF 函数用于计算两个日期之间相差的月份数。该函数接受两个日期参数，返回第一个日期减去第二个日期后的相差月份数。

语法

```
MONTHS_DIFF(<enddate>, <startdate>)
```

参数

参数	说明
<enddate>	结束日期，表示计算差值时的较晚日期。支持日期类型（如 DATE）或日期时间类型（如 DATETIME）
<startdate>	开始日期，表示计算差值时的较早日期。支持日期类型（如 DATE）或日期时间类型（如 DATETIME）

返回值

返回 <enddate> 减去 <startdate> 所得月份数 - 当<enddate>与<startdate>任意为 NULL，或者两者都为 NULL 时，返回 NULL

举例

```
select months_diff('2020-12-25','2020-10-25'),months_diff('2020-10-25 10:00:00','2020-12-25
    ↪ 11:00:00');
```

```
+-------------------------------------------------------------------------------------+-------------------------
    ↪
| months_diff(cast('2020-12-25' as DATETIMEV2(0)), cast('2020-10-25' as DATETIMEV2(0))) | months_
    ↪ diff(cast('2020-10-25 10:00:00' as DATETIMEV2(0)), cast('2020-12-25 11:00:00' as
    ↪ DATETIMEV2(0))) |
+-------------------------------------------------------------------------------------+-------------------------
    ↪
|                                                                                   2 |
    ↪
    ↪ -2 |
+-------------------------------------------------------------------------------------+-------------------------
    ↪
```

7.2.1.3.56 MONTHS_SUB

描述

MONTHS_SUB 函数用于对指定日期添加或减去指定的月份数，并返回计算后的新日期

语法

```
MONTHS_SUB(<datetime/date>,  <nums>)
```

参数

参数	说明
<datetime/date>	需要被计算加减月份的日期值
<nums>	需要加减的月份数

返回值

返回值与输入的类型一致特殊情况：- 输入为 0000-00-00 或 0000-00-00 00:00:00 时，返回 NULL - 输入为 NULL 时，返回 NULL - 如果输入为 months_sub(“9999-12-31”,-1)，将返回 NULL

举例

```
select months_sub("2020-01-31 02:02:02", 1),months_sub("2020-01-31", 1),months_sub("2020-01-31",
    ↪ -1);
```

```
+-----------------------------------------------------------+-----------------------------------------+----
    ↪
| months_sub(cast('2020-01-31 02:02:02' as DATETIMEV2(0)), 1) | months_sub(cast('2020-01-31' as
    ↪ DATEV2), 1) | months_sub(cast('2020-01-31' as DATEV2), -1) |
+-----------------------------------------------------------+-----------------------------------------+----
    ↪
| 2019-12-31 02:02:02                                       | 2019-12-31
    ↪                                  | 2020-02-29                                      |
+-----------------------------------------------------------+-----------------------------------------+----
    ↪
```

7.2.1.3.57 NEXT_DAY

描述

NEXT_DAY 函数用于返回给定日期之后第一个匹配指定星期几的日期。

> 该函数自 3.0.6 版本开始支持.

语法

```
NEXT_DAY(<datetime/date>, <day_of_week>)
```

参数

参数	描述
<datetime/date>	用于查找下一个星期几的日期。
<day_of_week>	用于标识星期几的字符串表达式。

<day_of_week> 必须是以下值之一（不区分大小写）：- ‘SU’，‘SUN’，‘SUNDAY’ - ‘MO’，‘MON’，‘MONDAY’ - ‘TU’，‘TUE’，‘TUESDAY’ - ‘WE’，‘WED’，‘WEDNESDAY’ - ‘TH’，‘THU’，‘THURSDAY’ - ‘FR’，‘FRI’，‘FRIDAY’ - ‘SA’，‘SAT’，‘SATURDAY’

返回值

无论输入是 DATETIME 还是 DATE 类型，都返回 DATE 类型的值。

特殊情况：- 如果 <datetime/date> 输入为 NULL，函数返回 NULL。- 如果输入是 NEXT_DAY(“9999-12-31 12:00:00”, <day_of_week>)，函数将返回与输入相同的值。

示例

```
select next_day("2020-01-31 02:02:02", "MONDAY"),next_day("2020-01-31", "MONDAY");
```

```
+-------------------------------------------+----------------------------------+
| next_day("2020-01-31 02:02:02", "MONDAY")  | next_day("2020-01-31", "MONDAY")  |
+-------------------------------------------+----------------------------------+
| 2020-02-03                                | 2020-02-03                       |
+-------------------------------------------+----------------------------------+
```

7.2.1.3.58 NOW

描述

函数用于获取当前系统时间，返回值为日期时间类型（DATETIME）。可以选择性地指定精度以调整返回值的小数秒部分的位数。

语法

```
NOW([<precision>])
```

参数

参数	说明
< ↪ precision ↪ >	可选参数，表示返回值的小数秒部分的精度，取值范围为 0 到 6。默认为 0，即不返回小数秒部分。受限于 JDK 实现，如果用户使用 JDK8 构建 FE，则精度最多支持到毫秒（小数点后三位），更大的精度位将全部填充 0。如果用户有更高精度需求，请使用 JDK11。

返回值

- 返回当前系统时间，类型为 DATETIME
- 如果指定的 <precision> 超出范围（如为负数或大于 6），函数会返回错误。

举例

```
select NOW(),NOW(3),NOW(6);
```

```
+---------------------+-------------------------+----------------------------+
| now()               | now(3)                  | now(6)                     |
+---------------------+-------------------------+----------------------------+
| 2025-01-23 11:08:35 | 2025-01-23 11:08:35.561 | 2025-01-23 11:08:35.562000 |
+---------------------+-------------------------+----------------------------+
```

7.2.1.3.59 QUARTER

描述

函数用于返回指定日期所属的季度（1 到 4）。每个季度包含三个月：- 第 1 季度：1 月至 3 月 - 第 2 季度：4 月至 6 月 - 第 3 季度：7 月至 9 月 - 第 4 季度：10 月至 12 月

语法

```
QUARTER(<datetime>)
```

参数

参数	说明
<datetime>	输入的日期或日期时间值，必须是有效的 DATE 或 DATETIME 类型。

返回值

- 返回一个整数，表示输入日期所属的季度，范围为 1 到 4。
- 如果输入值为 NULL，函数返回 NULL。
- 如果输入值为非法日期（如 0000-00-00），函数返回 NULL。

举例

```
SELECT QUARTER('2025-01-16'),QUARTER('2025-01-16 01:11:10');
```

```
+-----------------------------------------+-------------------------------------------------------+
| quarter(cast('2025-01-16' as DATETIME)) | quarter(cast('2025-01-16 01:11:10' as DATETIME)) |
+-----------------------------------------+-------------------------------------------------------+
|                                       1 |                                                     1 |
+-----------------------------------------+-------------------------------------------------------+
```

7.2.1.3.60 QUARTERS_ADD

描述

函数用于在指定的日期或日期时间值基础上，增加或减少指定的季度数，并返回计算后的日期值。

语法

```
QUARTERS_ADD(<date/datetime>, <quarters>)
```

参数

参数	说明
<date/datetime>	输入的日期或日期时间值，支持 DATE 或 DATETIME 类型。
<quarters>	要增加或减少的季度数，正整数表示增加，负整数表示减少。

返回值

- 返回一个日期值，与输入的日期类型一致。
- 如果 <date/datetime> 为 NULL，函数返回 NULL。
- 如果 <date/datetime> 为非法日期（如 0000-00-00），函数返回 NULL。

举例

```
select quarters_add("2020-01-31 02:02:02", 1);
```

```
+-------------------------------------------------------------+
| quarters_add(cast('2020-01-31 02:02:02' as DATETIMEV2(0)), 1) |
+-------------------------------------------------------------+
| 2020-04-30 02:02:02                                         |
+-------------------------------------------------------------+
```

7.2.1.3.61 QUARTERS_SUB

描述

函数用于在指定的日期或日期时间值基础上，增加或减少指定的季度数，并返回计算后的日期值。

语法

```
QUARTERS_SUB(<date/datetime>, <quarters>)
```

参数

参数	说明
<date/datetime>	输入的日期或日期时间值，支持 DATE 或 DATETIME 类型。
<quarters>	要增加或减少的季度数，正整数表示增加，负整数表示减少。

返回值

- 返回一个日期值，与输入的日期类型一致。
- 如果 <date/datetime> 为 NULL，函数返回 NULL。

- 如果 <date/datetime> 为非法日期（如 0000-00-00），函数返回 NULL。

举例

```
select quarters_sub("2020-01-31 02:02:02", 1);
```

```
+-------------------------------------------------------------+
| quarters_sub(cast('2020-01-31 02:02:02' as DATETIMEV2(0)), 1) |
+-------------------------------------------------------------+
| 2019-10-31 02:02:02                                         |
+-------------------------------------------------------------+
```

7.2.1.3.62 SEC_TO_TIME

描述

SEC_TO_TIME 函数将一个以秒为单位的值转换为 TIME 类型，返回格式为 HH:MM:SS。输入的秒数表示从一天的起点时间（00:00:00）开始计算的时间。###### 语法

```
SEC_TO_TIME(<seconds>)
```

参数

参数	说明
<seconds>	必填，输入的秒数，表示从一天起点时间（00:00:00）开始计算的秒数，支持正整数或负整数类型。

返回值

- 返回一个 TIME 类型值，格式为 HH:MM:SS，表示从起点时间（00:00:00）开始计算的时间。
- 如果输入的 <seconds> 为 NULL，函数返回 NULL。

举例

```
SELECT SEC_TO_TIME(59738);
```

```
+--------------------+
| sec_to_time(59738) |
+--------------------+
| 16:35:38           |
+--------------------+
```

7.2.1.3.63 SECOND

描述

函数用于返回指定日期时间值中的秒数部分。秒数的范围是 0 到 59。

语法

```
SECOND(<datetime>)
```

参数

参数	说明
<datetime>	输入的日期或日期时间值，支持 DATE 或 DATETIME 类型。

返回值

- 返回一个整数，表示输入日期时间值中的秒数部分，范围为 0 到 59。
- 如果输入值为 NULL，函数返回 NULL。
- 如果输入值为非法日期（如 0000-00-00 00:00:00），函数返回 NULL。

举例

```
select second('2018-12-31 23:59:59');
```

```
+-----------------------------------------------------+
| second(cast('2018-12-31 23:59:59' as DATETIMEV2(0))) |
+-----------------------------------------------------+
|                                                  59 |
+-----------------------------------------------------+
```

7.2.1.3.64 SECOND_CEIL

描述

函数用于将输入的日期时间值向上对齐到指定的秒周期边界，并返回对齐后的日期时间值。

语法

```
SECOND_CEIL(<datetime>[, <period>][, <origin_datetime>])
```

参数

参数	说明
<datetime>	必填，输入的日期时间值，支持 DATETIME 类型。
<period>	可选，表示每个周期由多少秒组成，支持正整数类型（INT）。默认为 1 秒。
<origin_datetime>	可选，对齐的时间起点，支持 DATETIME 类型。如果未指定，默认为 0001-01-01T00:00:00。

返回值

- 返回一个日期时间值，表示输入日期时间向上对齐后的结果。

- 如果输入的 <datetime> 为 NULL，返回 NULL。
- 如果输入的 <datetime> 为非法日期（如 0000-00-00T00:00:00），返回 NULL。

举例

只指定参数<datetime>

```
SELECT SECOND_CEIL('2025-01-23 12:34:56');
```

```
+-----------------------------------------------------------+
| second_ceil(cast('2025-01-23 12:34:56' as DATETIMEV2(0))) |
+-----------------------------------------------------------+
| 2025-01-23 12:34:56                                       |
+-----------------------------------------------------------+
```

指定参数 <datetime>，<origin_datetime>

```
SELECT SECOND_CEIL('2025-01-23 12:34:56', '2025-01-01 00:00:00');
```

```
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| second_ceil(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), cast('2025-01-01 00:00:00' as
    ↪ DATETIMEV2(0))) |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| 2025-01-23 12:34:56
    ↪                                                                                                 |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
```

指定参数 <datetime>，<period>

```
SELECT SECOND_CEIL('2025-01-23 12:34:56', 5)
```

```
+-----------------------------------------------------------+
| second_ceil(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2025-01-23 12:35:00                                       |
+-----------------------------------------------------------+
```

同时指定 <datetime>，<period>，<origin_datetime>

```
SELECT SECOND_CEIL('2025-01-23 12:34:56', 10, '2025-01-23 12:00:00');
```

```
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| second_ceil(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 10, cast('2025-01-23 12:00:00' as
    ↪ DATETIMEV2(0))) |
```

```
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| 2025-01-23 12:35:00
    ↪                                                                                                             |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.3.65 SECOND_FLOOR

描述

函数用于将输入的日期时间值向下对齐到指定的秒周期边界，并返回对齐后的日期时间值。

语法

```
SECOND_FLOOR(<datetime>[, <period>][, <origin_datetime>])
```

参数

参数	说明
<datetime>	必填，输入的日期时间值，支持 DATETIME 类型。
<period>	可选，表示每个周期由多少秒组成，支持正整数类型（INT）。默认为 1 秒。
<origin_datetime>	可选，对齐的时间起点，支持 DATETIME 类型。如果未指定，默认为 0001-01-01T00:00:00。

返回值

- 返回一个日期时间值，表示输入日期时间向上对齐后的结果。
- 如果输入的 <datetime> 为 NULL，返回 NULL。
- 如果输入的 <datetime> 为非法日期（如 0000-00-00T00:00:00），返回 NULL。

举例

只指定参数<datetime>

```
SELECT SECOND_FLOOR('2025-01-23 12:34:56');
```

```
+-----------------------------------------------------------+
| second_floor(cast('2025-01-23 12:34:56' as DATETIMEV2(0))) |
+-----------------------------------------------------------+
| 2025-01-23 12:34:56                                       |
+-----------------------------------------------------------+
```

指定参数 <datetime>，<origin_datetime>

```
SELECT SECOND_FLOOR('2025-01-23 12:34:56', '2025-01-01 00:00:00');
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| second_floor(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), cast('2025-01-01 00:00:00' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| 2025-01-23 12:34:56
    ↪                                                                                          |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

指定参数 <datetime>, <period>

```
SELECT SECOND_FLOOR('2025-01-23 12:34:56', 5);
```

```
+-----------------------------------------------------------------+
| second_floor(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------------+
| 2025-01-23 12:34:55                                             |
+-----------------------------------------------------------------+
```

同时指定 <datetime>, <period>, <origin_datetime>

```
SELECT SECOND_FLOOR('2025-01-23 12:34:56', 10, '2025-01-23 12:00:00');
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| second_floor(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 10, cast('2025-01-23 12:00:00' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| 2025-01-23 12:34:50
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.3.66 SECONDS_ADD

描述

函数用于在指定的日期时间值上增加或减少指定的秒数，并返回计算后的日期时间值

语法

```
SECONDS_ADD(<datetime>, <seconds>)
```

参数

参数	说明
<datetime>	必填，输入的日期时间值，支持 DATETIME 或 DATE 类型
<seconds>	必填，要增加或减少的秒数，支持整数类型（INT）。正数表示增加秒数，负数表示减少秒数。

返回值

- 返回一个日期时间值，类型与输入的 <datetime> 类型一致。
- 如果 <datetime> 为 NULL，函数返回 NULL。
- 如果 <datetime> 为非法日期（如 0000-00-00T00:00:00），函数返回 NULL。

举例

```
SELECT SECONDS_ADD('2025-01-23 12:34:56', 30),SECONDS_ADD('2025-01-23 12:34:56', -30);
```

```
+-------------------------------------------------------------+-------------------------------------------------------------
    ↪
| seconds_add(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 30) | seconds_add(cast('2025-01-23
    ↪ 12:34:56' as DATETIMEV2(0)), -30) |
+-------------------------------------------------------------+-------------------------------------------------------------
    ↪
| 2025-01-23 12:35:26                                         | 2025-01-23 12:34:26
    ↪                                          |
+-------------------------------------------------------------+-------------------------------------------------------------
    ↪
```

7.2.1.3.67 SECONDS_DIFF

描述

函数用于计算两个日期时间值之间的时间差，以秒为单位返回差值。

语法

```
SECONDS_DIFF(<end_datetime>, <start_datetime>)
```

参数

参数	说明
<end_datetime>	必填，结束的日期时间值，支持 DATETIME 或 DATE 类型。
<start_datetime>	必填，起始的日期时间值，支持 DATETIME 或 DATE 类型。

返回值

- 返回一个整数，表示两个日期时间值之间的秒差：

- 如果 <end_datetime> 晚于 <start_datetime>，返回正数。
- 如果 <end_datetime> 早于 <start_datetime>，返回负数。
- 如果 <end_datetime> 和 <start_datetime> 相等，返回 0。
- 如果任一参数为 NULL，函数返回 NULL。
- 如果输入的日期时间值为非法日期（如 0000-00-00T00:00:00），函数返回 NULL。

举例

```
SELECT SECONDS_DIFF('2025-01-23 12:35:56', '2025-01-23 12:34:56');
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| seconds_diff(cast('2025-01-23 12:35:56' as DATETIMEV2(0)), cast('2025-01-23 12:34:56' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 60 |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.3.68 SECONDS_SUB

描述

函数用于在指定的日期时间值上减少或增加指定的秒数，并返回计算后的日期时间值

语法

```
SECONDS_SUB(<datetime>, <seconds>)
```

参数

参数	说明
<datetime>	必填，输入的日期时间值，支持 DATETIME 或 DATE 类型
<seconds>	必填，要减少或增加的秒数，支持整数类型（INT）。正数表示增加秒数，负数表示减少秒数。

返回值

- 返回一个日期时间值，类型与输入的 <datetime> 类型一致。
- 如果 <datetime> 为 NULL，函数返回 NULL。
- 如果 <datetime> 为非法日期（如 0000-00-00T00:00:00），函数返回 NULL。

举例

```
SELECT SECONDS_SUB('2025-01-23 12:34:56', 30),SECONDS_SUB('2025-01-23 12:34:56', -30);
```

```
+-----------------------------------------------------------------+-----------------------------------------------------------------
    ↪
| seconds_sub(cast('2025-01-23 12:34:56' as DATETIMEV2(0)), 30) | seconds_sub(cast('2025-01-23
    ↪ 12:34:56' as DATETIMEV2(0)), -30) |
+-----------------------------------------------------------------+-----------------------------------------------------------------
    ↪
| 2025-01-23 12:34:26                                             | 2025-01-23 12:35:26
    ↪                                                |
+-----------------------------------------------------------------+-----------------------------------------------------------------
    ↪
```

7.2.1.3.69 STR_TO_DATE

描述

函数将输入的日期时间字符串根据指定的格式转换为 DATETIME 类型的值。

语法

```
STR_TO_DATE(<datetime_str>, <format>)
```

参数

参数	说明
<datetime_str>	必填，输入的日期时间字符串，表示要转换的日期或时间。
<format>	必填，指定的日期时间格式字符串，如 %Y-%m-%d %H:%i:%s 等，具体格式参数详见 DATE_FORMAT 文档

除此之外，<format> 额外支持以下若干代用格式，并按照正规 format 格式解读：

代用输入	解读为
yyyyMMdd	%Y%m%d
yyyy-MM-dd	%Y-%m-%d
yyyy-MM-dd HH:mm:ss	%Y-%m-%d %H:%i:%s

返回值

- 返回一个 DATETIME 类型值，表示转换后的日期时间。
- 如果输入的 <datetime_str> 或 <format> 无效，函数返回 NULL。

举例

将常见的日期时间字符串转换为 DATETIME 类型

```
SELECT STR_TO_DATE('2025-01-23 12:34:56', '%Y-%m-%d %H:%i:%s'),STR_TO_DATE('2025-01-23 12:34:56',
    ↪  'yyyy-MM-dd HH:mm:ss');
```

```
+-----------------------------------------------------+-----------------------------------------------------
    ↪
| str_to_date('2025-01-23 12:34:56', '%Y-%m-%d %H:%i:%s') | str_to_date('2025-01-23 12:34:56', '
    ↪ yyyy-MM-dd HH:mm:ss') |
+-----------------------------------------------------+-----------------------------------------------------
    ↪
| 2025-01-23 12:34:56.000000                          | 2025-01-23 12:34:56.000000
    ↪                                  |
+-----------------------------------------------------+-----------------------------------------------------
    ↪
```

其他

```
select STR_TO_DATE('200442 Monday', '%X%V %W'),STR_TO_DATE('2023','%Y');
```

```
+-----------------------------------------+---------------------------+
| str_to_date('200442 Monday', '%X%V %W') | str_to_date('2023', '%Y') |
+-----------------------------------------+---------------------------+
| 2004-10-18                              | 2023-01-01                |
+-----------------------------------------+---------------------------+
```

7.2.1.3.70 TIME_TO_SEC

描述

函数将输入的 TIME 或 DATETIME 类型的值转换为以秒为单位的总时间。如果输入为 DATETIME 类型，函数会自动提取时间部分（HH:MM:SS）。

语法

```
TIME_TO_SEC(<time>)
```

参数

参数	说明
<time>	必填，支持 TIME 或 DATETIME 类型的值。如果输入为 DATETIME 类型，函数会提取时间部分进行计算。

返回值

- 返回一个整数，表示输入时间值转换为总秒数的结果。
- 如果输入的 <time> 为 NULL，函数返回 NULL。

举例

```
SELECT TIME_TO_SEC('16:32:18'),TIME_TO_SEC('2025-01-01 16:32:18');
```

```
+--------------------------------------+-------------------------------------------------+
| time_to_sec(cast('16:32:18' as TIME)) | time_to_sec(cast('2025-01-01 16:32:18' as TIME)) |
+--------------------------------------+-------------------------------------------------+
|                                59538 |                                           59538 |
+--------------------------------------+-------------------------------------------------+
```

7.2.1.3.71 TIMEDIFF

描述

TIMEDIFF 函数用于计算两个日期时间值之间的差值。该函数接受两个参数，并返回其差值，结果为 TIME 类型。

语法

```
TIMEDIFF(<end_datetime>, <start_datetime>)
```

参数

参数	说明
<end_datetime>	结束的时间或日期时间值
<start_datetime>	开始的时间或日期时间值

返回值

返回一个 TIME 类型的值，表示两个输入之间的时间差：- 当 <end_datetime> 晚于 <start_datetime> 时，返回正的时间差。- 当 <end_datetime> 早于 <start_datetime> 时，返回负的时间差。- 当 <end_datetime> 和 <start_datetime> 相等时，返回 00:00:00。- 如果 <end_datetime> 或 <start_datetime> 为 NULL，函数返回 NULL。- 如果 <end_datetime> 或 <start_datetime> 为非法时间或日期时间值（如 0000-00-00 00:00:00），函数返回 NULL。

举例

```
SELECT TIMEDIFF('2024-07-20 16:59:30','2024-07-11 16:35:21');
```

```
+-------------------------------------------------------------------------------------------------------+
    ↪
| timediff(cast('2024-07-20 16:59:30' as DATETIMEV2(0)), cast('2024-07-11 16:35:21' as DATETIMEV2
    ↪ (0))) |
+-------------------------------------------------------------------------------------------------------+
    ↪
| 216:24:09
    ↪
    ↪ |
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.3.72 TIMESTAMP

描述

TIMESTAMP 函数有两种用法：

1. 将日期时间字符串转换为 DATETIME 类型
2. 将两个参数组合成一个 DATETIME 类型

语法

```
TIMESTAMP(string)
TIMESTAMP(date, time)
```

参数

参数	说明
string	日期时间字符串
date	日期值，可以是 DATE 类型或格式正确的日期字符串
time	时间值，可以是 TIME 类型或格式正确的时间字符串

返回值

返回类型为 DATETIME。

举例

```
-- 将字符串转换为 DATETIME
SELECT TIMESTAMP('2019-01-01 12:00:00');
```

```
+-----------------------------------+
| timestamp('2019-01-01 12:00:00')  |
+-----------------------------------+
| 2019-01-01 12:00:00               |
+-----------------------------------+
```

```
-- 将日期和时间组合成 DATETIME
SELECT TIMESTAMP('2019-01-01', '12:00:00');
```

```
+---------------------------------------+
| timestamp('2019-01-01', '12:00:00')   |
+---------------------------------------+
| 2019-01-01 12:00:00                   |
+---------------------------------------+
```

7.2.1.3.73 TIMESTAMPADD

描述

timestampadd 函数用于将指定的时间单位（如年、月、日、小时、分钟、秒等）添加到一个日期上。这个函数通常用于日期和时间的计算。

语法

```
TIMESTAMPADD(<unit>, <interval>, <datetime_expr>)
```

参数

参数	说明
unit	时间单位，指定要添加的时间单位，常见的值有 SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, YEAR
interval	要添加的时间间隔，通常是一个整数，可以是正数或负数，表示添加或减去的时间长度
datetime_expr	合法的目标日期，为 DATETIME 类型

返回值

返回新的日期时间，表示在指定时间点上添加或减去指定时间间隔后的结果。

如果输入的目标日期不合法，则返回 NULL。

举例

```
SELECT TIMESTAMPADD(MINUTE,1,'2019-01-02');
```

```
+------------------------------------------------+
| timestampadd(MINUTE, 1, '2019-01-02 00:00:00') |
+------------------------------------------------+
| 2019-01-02 00:01:00                            |
+------------------------------------------------+
```

```
SELECT TIMESTAMPADD(WEEK,1,'2019-01-02');
```

```
+-----------------------------------------------+
| timestampadd(WEEK, 1, '2019-01-02 00:00:00') |
+-----------------------------------------------+
| 2019-01-09 00:00:00                           |
+-----------------------------------------------+
```

```
SELECT TIMESTAMPADD(WEEK,1,'1196440219');
```

```
+-------------------------------------------------------------+
| timestampadd(WEEK, 1, CAST('1196440219' AS datetimev2(6))) |
+-------------------------------------------------------------+
| NULL                                                        |
+-------------------------------------------------------------+
```

7.2.1.3.74 TIMESTAMPDIFF

描述

timestampdiff 函数用于计算两个日期之间的差值，返回两个日期之间的时间间隔。可以计算两者之间的差异，以指定的时间单位（如秒、分钟、小时、天、月、年等）返回结果。

语法

```
TIMESTAMPDIFF(<unit>, <datetime_expr1>, <datetime_expr2>)
```

参数

参数	说明
unit	时间单位，指定要返回差异的单位，支持 SECOND、 MINUTE、 HOUR、 DAY、 MONTH、 QUARTER、 YEAR

> QUARTER 支持从 3.0.8 和 3.1.0 版本开始。

|datetime_expr1| 第一个日期时间，合法的日期格式 | |datetime_expr2| 第二个日期时间，合法的日期格式 |

返回值

返回两个日期时间之间的差异，单位根据 unit 参数确定。

如果输入的参数不合法，则返回 NULL

举例

```
SELECT TIMESTAMPDIFF(MONTH,'2003-02-01','2003-05-01');
```

```
+-----------------------------------------------------------------------+
| timestampdiff(MONTH, '2003-02-01 00:00:00', '2003-05-01 00:00:00') |
+-----------------------------------------------------------------------+
|                                                                     3 |
+-----------------------------------------------------------------------+
```

```
SELECT TIMESTAMPDIFF(QUARTER,'2002-05-01','2001-01-01');
```

```
+-------------------------------------------------+
| TIMESTAMPDIFF(QUARTER,'2002-05-01','2001-01-01') |
+-------------------------------------------------+
|                                              -5 |
+-------------------------------------------------+
```

```
SELECT TIMESTAMPDIFF(YEAR,'2002-05-01','2001-01-01');
```

```
+-----------------------------------------------------------------+
| timestampdiff(YEAR, '2002-05-01 00:00:00', '2001-01-01 00:00:00') |
+-----------------------------------------------------------------+
|                                                              -1 |
+-----------------------------------------------------------------+
```

```
SELECT TIMESTAMPDIFF(MINUTE,'2003-02-01','2003-05-01 12:05:55');
```

```
+-------------------------------------------------------------------+
| timestampdiff(MINUTE, '2003-02-01 00:00:00', '2003-05-01 12:05:55') |
+-------------------------------------------------------------------+
|                                                            128885 |
+-------------------------------------------------------------------+
```

```
SELECT  TIMESTAMPDIFF(MINUTE,'2003-02-01','1196440219');
```

```
+-----------------------------------------------------------------------------------+
| timestampdiff(MINUTE, '2003-02-01 00:00:00', CAST('1196440219' AS datetimev2(6))) |
+-----------------------------------------------------------------------------------+
|                                                                              NULL |
+-----------------------------------------------------------------------------------+
```

7.2.1.3.75 TO_DATE

描述

日期转换函数，用于将日期时间（DATETIME）转换为日期类型（DATE），即去掉时间部分，仅保留日期（YYYY-MM-DD）

语法

```
TO_DATE(<datetime_value>)
```

必选参数

参数	描述
datetime_value	DATETIME 类型日期时间

举例

将 2020-02-02 00:00:00 转换为 2020-02-02

```
select to_date("2020-02-02 00:00:00");
```

```
+-------------------------------+
```

```
| to_date('2020-02-02 00:00:00') |
+--------------------------------+
| 2020-02-02                     |
+--------------------------------+
```

7.2.1.3.76 TO_DAYS

描述

日期计算函数，它用于将日期转换为天数数值，即计算从公元 0 年 12 月 31 日（基准日期）到指定日期的总天数。

语法

```
TO_DAYS(<datetime_or_date_value>)
```

必选参数

参数	描述
<datetime_or_date_value>	datetime 或者 date 类型日期时间

举例

查询 2007 年 10 月 7 日距今有多少天

```
select to_days('2007-10-07');
```

```
+---------------------------------------+
| to_days(cast('2007-10-07' as DATEV2)) |
+---------------------------------------+
|                                733321 |
+---------------------------------------+
```

```
select to_days('2007-10-07 10:03:09');
```

```
+-----------------------------------------------+
| to_days(cast('2007-10-07 10:03:09' as DATEV2)) |
+-----------------------------------------------+
|                                        733321 |
+-----------------------------------------------+
```

7.2.1.3.77 TO_ISO8601

描述

将日期时间值转换为 ISO8601 格式的字符串。

语法

```
TO_ISO8601(<dt>)
```

参数

参数	说明
<dt>	输入的日期时间值，类型为 DATETIME 或 DATE

返回值

返回类型为 VARCHAR，表示 ISO8601 格式的日期时间字符串。

举例

```
SELECT TO_ISO8601('2020-01-01 12:30:45');
```

```
+-------------------------------------+
| to_iso8601('2020-01-01 12:30:45.0') |
+-------------------------------------+
| 2020-01-01T12:30:45                 |
+-------------------------------------+
```

7.2.1.3.78 TO_MONDAY

to_monday

描述

语法

```
DATE to_monday(DATETIME date)
```

将日期或带时间的日期向下舍入到最近的星期一。作为一种特殊情况，日期参数 1970-01-01、1970-01-02、1970-01-03 和 1970-01-04 返回日期 1970-01-01

举例

```
MySQL [(none)]> select to_monday('2022-09-10');
+----------------------------------+
| to_monday('2022-09-10 00:00:00') |
+----------------------------------+
| 2022-09-05                       |
+----------------------------------+
```

keywords

```
MONDAY
```

7.2.1.3.79 UNIX_TIMESTAMP

描述

将 Date 或者 Datetime 类型转化为 unix 时间戳。

如果没有参数，则是将当前的时间转化为时间戳。

参数需要是 Date 或者 Datetime 类型。

Format 的格式请参阅 `date_format` 函数的格式说明。

该函数受时区影响。

语法

```
UNIX_TIMESTAMP([DATETIME date[, STRING fmt]])
```

参数

参数	描述
<↩ date↩ >↩	待转换的日期时间值，类型为 datetime↩ 或 date↩ 类型，可转换范围为‘1970-01-01 00:00:01.000000 UTC’至‘3001-01-19 03:14:07.999999 UTC’。

参数	描述
<↩ fmt ↩ > ↩	date 参数指代需要转换为时间戳的特定部分，其类型为 string ↩ 。若提供该参数，则仅将与格式匹配的部分转换为时间戳。

参数	描述

返回值

根据输入返回两种类型

1. 若是输入的 date (只有 datetime 类型才可以有 scale 不为零) scale 不为 0 或者带有 format 参数返回一个时间戳，类型为 Decimal，最高六位小数精度

2. 若是输入的 date scale 为 0 并且不带有 format 参数返回一个时间戳，类型为 INT

支持的输入范围为 ‘1970-01-01 00:00:01.000000 UTC’ 至 ‘3001-01-19 03:14:07.999999 UTC’。早于最小值的时间返回 0；超过最大值的时间返回 0。

任意参数为 null 则返回 null

举例

```
---以下都是在 UTC 时区返回的结果
set time_zone= 'UTC';

---显示当前时间的时间戳
mysql> select unix_timestamp();
+------------------+
| unix_timestamp() |
+------------------+
|       1753933330 |
+------------------+

---输入一个 datetime 显示该时间的
mysql> select unix_timestamp('2007-11-30 10:30:19');
+---------------------------------------+
| unix_timestamp('2007-11-30 10:30:19') |
+---------------------------------------+
|                            1196389819 |
+---------------------------------------+

---匹配 format 显示给出的 datetime 对应时间戳
mysql> select unix_timestamp('2007-11-30 10:30-19', '%Y-%m-%d %H:%i-%s');
+------------------------------------------------------------+
| unix_timestamp('2007-11-30 10:30-19', '%Y-%m-%d %H:%i-%s') |
+------------------------------------------------------------+
|                                          1196389819.000000 |
+------------------------------------------------------------+
```

```
---只匹配年日月显示时间戳
mysql> select unix_timestamp('2007-11-30 10:30%3A19', '%Y-%m-%d');
+---------------------------------------------------+
| unix_timestamp('2007-11-30 10:30%3A19', '%Y-%m-%d') |
+---------------------------------------------------+
|                                  1196352000.000000 |
+---------------------------------------------------+


---带有其他字符匹配
mysql> select unix_timestamp('2007-11-30 10:30%3A19', '%Y-%m-%d %H:%i%%3A%s');
+------------------------------------------------------------------+
| unix_timestamp('2007-11-30 10:30%3A19', '%Y-%m-%d %H:%i%%3A%s') |
+------------------------------------------------------------------+
|                                                1196389819.000000 |
+------------------------------------------------------------------+


---超出最小范围内的时间返回 0
mysql> SELECT UNIX_TIMESTAMP('1970-01-01 00:00:00');
+---------------------------------------+
| UNIX_TIMESTAMP('1970-01-01 00:00:00') |
+---------------------------------------+
|                                     0 |
+---------------------------------------+


---输入时间并且 scale 不为 0
mysql> SELECT UNIX_TIMESTAMP('2015-11-13 10:20:19.123');
+-----------------------------------------+
| UNIX_TIMESTAMP('2015-11-13 10:20:19.123') |
+-----------------------------------------+
|                           1447381219.123 |
+-----------------------------------------+

---超过允许时间的最大范围

mysql> SELECT UNIX_TIMESTAMP('3001-01-19 03:14:07.999999');
+-----------------------------------------+
| UNIX_TIMESTAMP('3001-01-19 03:14:07.999999') |
+-----------------------------------------+
|                                0.000000 |
+-----------------------------------------+
```

```
---任意参数为 null 则返回 Null
mysql> select unix_timestamp(NULL);
+----------------------+
| unix_timestamp(NULL) |
+----------------------+
|                 NULL |
+----------------------+

mysql> select unix_timestamp('2038-01-19 11:14:08',null);
+--------------------------------------------+
| unix_timestamp('2038-01-19 11:14:08',null) |
+--------------------------------------------+
|                                       NULL |
+--------------------------------------------+
```

keywords

```
UNIX_TIMESTAMP,UNIX,TIMESTAMP
```

7.2.1.3.80 UTC_TIMESTAMP

utc_timestamp

描述

语法

```
DATETIME UTC_TIMESTAMP()
```

返回当前 UTC 日期和时间在“YYYY-MM-DD HH:MM:SS”或

“YYYYMMDDHHMMSS”格式的一个值

根据该函数是否用在字符串或数字语境中

举例

```
mysql> select utc_timestamp(),utc_timestamp() + 1;
+---------------------+---------------------+
| utc_timestamp()     | utc_timestamp() + 1 |
+---------------------+---------------------+
| 2019-07-10 12:31:18 |      20190710123119 |
+---------------------+---------------------+
```

keywords

```
UTC_TIMESTAMP,UTC,TIMESTAMP
```

7.2.1.3.81 WEEK

week

描述

语法

```
INT WEEK(DATE date[, INT mode])
```

返回指定日期的星期数。mode 的值默认为 0。参数 mode 的作用参见下面的表格：

Mode	星期的第一天	星期数的范围	第一个星期的定义
0	星期日	0-53	这一年中的第一个星期日所在的星期
1	星期一	0-53	这一年的日期所占的天数大于等于 4 天的第一个星期
2	星期日	1-53	这一年中的第一个星期日所在的星期
3	星期一	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
4	星期日	0-53	这一年的日期所占的天数大于等于 4 天的第一个星期
5	星期一	0-53	这一年中的第一个星期一所在的星期
6	星期日	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
7	星期一	1-53	这一年中的第一个星期一所在的星期

参数为 Date 或者 Datetime 类型

举例

```
mysql> select week('2020-1-1');
+------------------+
| week('2020-1-1') |
+------------------+
|                0 |
+------------------+
```

```
mysql> select week('2020-7-1',1);
+----------------------+
| week('2020-7-1', 1) |
+----------------------+
|                   27 |
+----------------------+
```

keywords

```
WEEK
```

7.2.1.3.82 WEEK_CEIL

描述

将日期时间值向上舍入到最接近的指定周间隔。如果提供了起始时间（origin），则使用该时间作为计算间隔的参考点。

语法

```
WEEK_CEIL(<datetime>)
WEEK_CEIL(<datetime>, <origin>)
WEEK_CEIL(<datetime>, <period>)
WEEK_CEIL(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	要向上舍入的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	周间隔值，类型为 INT，表示每个间隔的周数
<origin>	间隔的起始点，类型为 DATETIME 或 DATETIMEV2；默认为 0001-01-01 00:00:00

返回值

返回类型为 DATETIME，表示向上舍入后的日期时间值。结果的时间部分将被设置为 00:00:00。

注意： - 如果未指定周期，则默认为 1 周间隔。 - 周期必须是正整数。 - 结果总是向上舍入到未来的时间。 - 返回值的时间部分始终设置为 00:00:00。

举例

```
SELECT WEEK_CEIL('2023-07-13 22:28:18', 2);
```

```
+-----------------------------------------------------------+
| week_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 2) |
+-----------------------------------------------------------+
| 2023-07-17 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.83 WEEK_FLOOR

描述

将日期时间值向下舍入到最接近的指定周间隔。如果提供了起始时间（origin），则使用该时间作为计算间隔的参考点。

语法

```
WEEK_FLOOR(<datetime>)
WEEK_FLOOR(<datetime>, <origin>)
WEEK_FLOOR(<datetime>, <period>)
WEEK_FLOOR(<datetime>, <period>, <origin>)
```

参数

参数	说明
<datetime>	要向下舍入的日期时间值，类型为 DATETIME 或 DATETIMEV2
<period>	周间隔值，类型为 INT，表示每个间隔的周数
<origin>	间隔的起始点，类型为 DATETIME 或 DATETIMEV2；默认为 0001-01-01 00:00:00

返回值

返回类型为 DATETIME，表示向下舍入后的日期时间值。结果的时间部分将被设置为 00:00:00。

注意： - 如果未指定周期，则默认为 1 周间隔。 - 周期必须是正整数。 - 结果总是向下舍入到过去的时间。 - 返回值的时间部分始终设置为 00:00:00。

举例

```
SELECT WEEK_FLOOR('2023-07-13 22:28:18', 2);
```

```
+-----------------------------------------------------------+
| week_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 2) |
+-----------------------------------------------------------+
| 2023-07-03 00:00:00                                       |
+-----------------------------------------------------------+
```

7.2.1.3.84 WEEKDAY

weekday

描述

语法

```
INT WEEKDAY (DATETIME date)
```

WEEKDAY 函数返回日期的工作日索引值，即星期一为 0，星期二为 1，星期日为 6

参数为 Date 或者 Datetime 类型或者可以 cast 为 Date 或者 Datetime 类型的数字

注意 WEEKDAY 和 DAYOFWEEK 的区别：

```
          +-----+-----+-----+-----+-----+-----+-----+
          | Sun | Mon | Tues| Wed | Thur| Fri | Sat |
          +-----+-----+-----+-----+-----+-----+-----+
  weekday |  6  |  0  |  1  |  2  |  3  |  4  |  5  |
          +-----+-----+-----+-----+-----+-----+-----+
dayofweek |  1  |  2  |  3  |  4  |  5  |  6  |  7  |
          +-----+-----+-----+-----+-----+-----+-----+
```

举例

```
mysql> select weekday('2019-06-25');
+------------------------------+
```

```
| weekday('2019-06-25 00:00:00') |
+--------------------------------+
|                              1 |
+--------------------------------+

mysql> select weekday(cast(20190625 as date));
+---------------------------------+
| weekday(CAST(20190625 AS DATE)) |
+---------------------------------+
|                               1 |
+---------------------------------+
```

keywords

```
WEEKDAY
```

7.2.1.3.85 WEEKOFYEAR

weekofyear

描述

语法

```
INT WEEKOFYEAR(DATETIME date)
```

获得一年中的第几周

参数为 Date 或者 Datetime 类型

举例

```
mysql> select weekofyear('2008-02-20 00:00:00');
+-----------------------------------+
| weekofyear('2008-02-20 00:00:00') |
+-----------------------------------+
|                                 8 |
+-----------------------------------+
```

keywords

```
WEEKOFYEAR
```

7.2.1.3.86 WEEKS_ADD

描述

函数用于在指定的日期或时间值上增加（或减少）一定数量的周

语法

```
WEEKS_ADD(<datetime_or_date_value>, <weeks_value>)
```

必选参数

参数	描述
<datetime_or_date_value>	DATETIME 或者 DATE 类型的输入值
<weeks_value>	整数，表示要增加或减少的周数（正数表示增加，负数表示减少）

举例

1. 在 2020-02-02 02:02:02 时间上增加一周

```
select weeks_add("2020-02-02 02:02:02", 1);
```

```
+-------------------------------------+
| weeks_add('2020-02-02 02:02:02', 1) |
+-------------------------------------+
| 2020-02-09 02:02:02                 |
+-------------------------------------+
```

2. 在 2020-02-02 02:02:02 时间上减少一周

```
select weeks_add("2020-02-02 02:02:02", -1);
```

```
+-----------------------------------------------------------+
| weeks_add(cast('2020-02-02 02:02:02' as DATETIMEV2(0)), -1) |
+-----------------------------------------------------------+
| 2020-01-26 02:02:02                                       |
+-----------------------------------------------------------+
```

3. 给 2020-02-02 日期增加一周

```
select weeks_add("2020-02-02", 1);
```

```
+-----------------------------------------+
| weeks_add(cast('2020-02-02' as DATEV2), 1) |
+-----------------------------------------+
| 2020-02-09                              |
+-----------------------------------------+
```

7.2.1.3.87 WEEKS_DIFF

描述

用于计算两个日期或时间值之间相差的完整周数（以 7 天为单位）。

语法

```
WEEKS_DIFF(<end_date>, <start_date>)
```

必选参数

参数名称	数据类型	描述
end_date	DATE, DATETIME	较晚的日期或者日期时间
start_date	DATE, DATETIME	较早的日期或者日期时间

举例

1. 2020-12-25 与 2020-10-25 相差多少周

```
select weeks_diff('2020-12-25','2020-10-25');
```

```
+--------------------------------------------------------+
| weeks_diff('2020-12-25 00:00:00', '2020-10-25 00:00:00') |
+--------------------------------------------------------+
|                                                      8 |
+--------------------------------------------------------+
```

2. 2020-12-25 10:10:02 与 2020-10-25 12:10:02 相差多少周

```
select weeks_diff('2020-12-25 10:10:02','2020-10-25 12:10:02');
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| weeks_diff(cast('2020-12-25 10:10:02' as DATETIMEV2(0)), cast('2020-10-25 12:10:02' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 8 |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

3. 2020-12-25 10:10:02 与 2020-10-25 相差多少周

```
select weeks_diff('2020-12-25 10:10:02','2020-10-25');
```

```
+-------------------------------------------------------------------------------------+
| weeks_diff(cast('2020-12-25 10:10:02' as DATETIMEV2(0)), cast('2020-10-25' as DATEV2)) |
+-------------------------------------------------------------------------------------+
|                                                                                   8 |
+-------------------------------------------------------------------------------------+
```

7.2.1.3.88 WEEKS_SUB

描述

用于在指定的日期或时间值上减少一定数量的周（即减去 weeks * 7 天）。

语法

```
WEEKS_SUB(<date_value>, <week_period>)
```

必选参数

参数名称	描述
date_value	DATE 或 DATETIME 类型的输入值。
week_period	整数，表示要减少的周数（正数表示减少，负数表示增加）。

举例

1. 在 2020-02-02 02:02:02 日期时间上减去一周

```
select weeks_sub("2020-02-02 02:02:02", 1);
```

```
+-------------------------------------+
| weeks_sub('2020-02-02 02:02:02', 1) |
+-------------------------------------+
| 2020-01-26 02:02:02                 |
+-------------------------------------+
```

2. 在 2020-02-02 日期上减去一周

```
select weeks_sub("2020-02-02", 1);
```

```
+-------------------------------------------+
| weeks_sub(cast('2020-02-02' as DATEV2), 1) |
+-------------------------------------------+
| 2020-01-26                                |
+-------------------------------------------+
```

7.2.1.3.89 YEAR

year

描述

语法

```
INT YEAR(DATETIME date)
```

返回 date 类型的 year 部分，范围从 1000-9999

参数为 Date 或者 Datetime 类型

举例

```
mysql> select year('1987-01-01');
+-----------------------------+
| year('1987-01-01 00:00:00') |
+-----------------------------+
|                        1987 |
+-----------------------------+
```

keywords

```
YEAR
```

7.2.1.3.90 YEAR_CEIL

year_ceil

描述

语法

```
DATETIME YEAR_CEIL(DATETIME datetime)
DATETIME YEAR_CEIL(DATETIME datetime, DATETIME origin)
DATETIME YEAR_CEIL(DATETIME datetime, INT period)
DATETIME YEAR_CEIL(DATETIME datetime, INT period, DATETIME origin)
```

将日期转化为指定的时间间隔周期的最近上取整时刻。

- datetime：参数是合法的日期表达式。
- period：参数是指定每个周期有几年组成。
- origin：开始的时间起点，如果不填，默认是 0001-01-01T00:00:00。

举例

```
mysql> select year_ceil("2023-07-13 22:28:18", 5);
+-----------------------------------------------------------+
| year_ceil(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------------+
| 2025-01-01 00:00:00                                       |
+-----------------------------------------------------------+
1 row in set (0.02 sec)
```

keywords

```
YEAR_CEIL, YEAR, CEIL
```

Best Practice

还可参阅 date_ceil

7.2.1.3.91 YEAR_FLOOR

描述

用于将给定的日期向下取整到指定的年份间隔起点。它支持多个变体，可按不同方式指定起始时间 (origin) 和周期 (period) 进行取整。

语法

```
YEAR_FLOOR(<date_value>, [<period> | <origin_date_value>])
YEAR_FLOOR(<date_value>, <period>, <origin_date_value>)
```

参数

参数	类型	说明
<date_value>	DATE, DATETIME	需要取整的 DATE 或 DATETIME 输入值。
<origin_date_value ↪ >	DATE, DATETIME	用作基准的 DATE 或 DATETIME 输入值，如果不填，默认值为 0001-01-01T00:00:00。
<period>	INT	取整的时间间隔，正整数，表示以多少年为周期进行取整。

举例

1. 按整年取整

```
SELECT YEAR_FLOOR('2023-07-13 22:28:18');
```

```
+-----------------------------------------------------------+
| year_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0))) |
+-----------------------------------------------------------+
| 2023-01-01 00:00:00                                       |
+-----------------------------------------------------------+
```

```
SELECT YEAR_FLOOR('2023-07-13');
```

```
+-------------------------------------------------+
| year_floor(cast('2023-07-13' as DATETIMEV2(0))) |
+-------------------------------------------------+
| 2023-01-01 00:00:00                             |
+-------------------------------------------------+
```

2. 以 origin 为基准取整

```
SELECT YEAR_FLOOR('2023-07-13 22:28:18', '2020-03-15');
```

```
+-------------------------------------------------------------------------------------------------------+
    ↪
```

```
| year_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), cast('2020-03-15' as DATETIMEV2
    ↪ (0))) |
+-----------------------------------------------------------------------------------------------+
    ↪
| 2023-03-15 00:00:00
    ↪                                                                                |
+-----------------------------------------------------------------------------------------------+
    ↪
```

3. 以 period 为单位取整

```
SELECT YEAR_FLOOR('2023-07-13', 5);
```

```
   +-----------------------------------------------------+
| year_floor(cast('2023-07-13' as DATETIMEV2(0)), 5) |
+-----------------------------------------------------+
| 2020-01-01 00:00:00                                 |
+-----------------------------------------------------+
```

4. 以 origin 和 period 取整

```
SELECT YEAR_FLOOR('2023-07-13 22:28:18', 5, '2018-06-01');
```

```
+-----------------------------------------------------------------------------------------------+
    ↪
| year_floor(cast('2023-07-13 22:28:18' as DATETIMEV2(0)), 5, cast('2018-06-01' as
    ↪ DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------------+
    ↪
| 2023-06-01 00:00:00
    ↪                                                                              |
+-----------------------------------------------------------------------------------------------+
    ↪
```

```
SELECT YEAR_FLOOR('2023-07-13', 5, '2016-01-01');
```

```
+-----------------------------------------------------------------------------------------+
| year_floor(cast('2023-07-13' as DATETIMEV2(0)), 5, cast('2016-01-01' as DATETIMEV2(0))) |
+-----------------------------------------------------------------------------------------+
| 2021-01-01 00:00:00                                                                     |
+-----------------------------------------------------------------------------------------+
```

7.2.1.3.92 YEARS_ADD

描述

返回一个新的日期时间值，该值是在输入的日期时间上增加指定的年数。

语法

```
YEARS_ADD(<date>, <years>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIME 或 DATE
<years>	要增加的年数，类型为 INT

返回值

返回与输入 <date> 类型相同的值（ DATETIME 或 DATE ），表示在输入日期时间的基础上增加指定年数后的时间值。

举例

```
SELECT YEARS_ADD('2020-01-31 02:02:02', 1);
```

```
+-------------------------------------+
| years_add('2020-01-31 02:02:02', 1) |
+-------------------------------------+
| 2021-01-31 02:02:02                 |
+-------------------------------------+
```

7.2.1.3.93 YEARS_DIFF

描述

计算两个日期时间值之间相差的年数。

语法

```
YEARS_DIFF(<enddate>, <startdate>)
```

参数

参数	说明
<enddate>	结束日期，类型为 DATETIME 或 DATE
<startdate>	开始日期，类型为 DATETIME 或 DATE

返回值

返回类型为 INT，表示两个日期之间相差的年数。

举例

```
SELECT YEARS_DIFF('2020-12-25', '2019-10-25');
```

```
+-----------------------------------------------------------+
| years_diff('2020-12-25 00:00:00', '2019-10-25 00:00:00') |
+-----------------------------------------------------------+
|                                                         1 |
+-----------------------------------------------------------+
```

7.2.1.3.94 YEARS_SUB

描述

返回一个新的日期时间值，该值是从输入的日期时间中减去指定的年数。

语法

```
YEARS_SUB(<date>, <years>)
```

参数

参数	说明
<date>	输入的日期时间值，类型为 DATETIME 或 DATE
<years>	要减去的年数，类型为 INT

返回值

返回与输入 <date> 类型相同的值（DATETIME 或 DATE），表示从输入日期时间中减去指定年数后的时间值。

举例

```
SELECT YEARS_SUB('2020-02-02 02:02:02', 1);
```

```
+-------------------------------------+
| years_sub('2020-02-02 02:02:02', 1) |
+-------------------------------------+
| 2019-02-02 02:02:02                 |
+-------------------------------------+
```

7.2.1.3.95 YEARWEEK

yearweek

描述

语法

```
INT YEARWEEK(DATE date[, INT mode])
```

返回指定日期的年份和星期数。mode 的值默认为 0。当日期所在的星期属于上一年时，返回的是上一年的年份和星期数；当日期所在的星期属于下一年时，返回的是下一年的年份，星期数为 1。参数 mode 的作用参见下面的表格：

Mode	星期的第一天	星期数的范围	第一个星期的定义
0	星期日	1-53	这一年中的第一个星期日所在的星期
1	星期一	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
2	星期日	1-53	这一年中的第一个星期日所在的星期
3	星期一	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
4	星期日	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
5	星期一	1-53	这一年中的第一个星期一所在的星期
6	星期日	1-53	这一年的日期所占的天数大于等于 4 天的第一个星期
7	星期一	1-53	这一年中的第一个星期一所在的星期

参数为 Date 或者 Datetime 类型

举例

```
mysql> select yearweek('2021-1-1');
+-----------------------+
| yearweek('2021-1-1') |
+-----------------------+
|                202052 |
+-----------------------+
```

```
mysql> select yearweek('2020-7-1');
+-----------------------+
| yearweek('2020-7-1') |
+-----------------------+
|                202026 |
+-----------------------+
```

```
mysql> select yearweek('2024-12-30',1);
+-----------------------------------+
| yearweek('2024-12-30 00:00:00', 1) |
+-----------------------------------+
|                            202501 |
+-----------------------------------+
```

keywords

```
YEARWEEK
```

7.2.1.4 地理位置函数

7.2.1.4.1 ST_ANGLE

描述

输入三个点，第一条线表示是 point1 点为第一个端点，point2 点为第二个端点相连的直线，第二条线表示对是 point1 为第一个端点，point2 为第二个端点相连的直线，它们表示两条相交的线。返回第一条直线顺时针到第二条直线的夹角，每个端点坐标的 x 为纬度，都要求范围在 [-180,180] ，每个坐标的 y 轴要求范围在 [-90,90]。

语法

```
ST_ANGLE( <point1>, <point2>, <point3>)
```

参数

参数	说明
<point1>	第一条直线的第一个端点, 类型为 GeoPoint
<point2>	第一条直线的第二个端点且是第二条直线的第一个端点, 类型为 GeoPoint
<point3>	第二条直线的第二个端点, 类型为 GeoPint

返回值

这些线之间的夹角以弧度表示，范围为 [0, 2pi)，为一个 double 类型浮点数。夹角按顺时针方向从第一条线开始测量，直至第二条线。

ST_ANGLE 存在以下边缘情况：

- 如果点 2 和点 3 相同，则返回 NULL。
- 如果点 2 和点 1 相同，则返回 NULL。
- 如果点 2 和点 3 是完全对映点，则返回 NULL。
- 如果点 2 和点 1 是完全对映点，则返回 NULL。
- 如果某个点超过 x 和 y的范围，则返回 NULL
- 任意坐标为 NULL, 返回 NULL

举例

```
SELECT ST_Angle(ST_Point(1, 0),ST_Point(0, 0),ST_Point(0, 1));
```

第一条线到第二条线顺时针弧度为 4.7

```
+----------------------------------------------------------------------+
| st_angle(st_point(1.0, 0.0), st_point(0.0, 0.0), st_point(0.0, 1.0)) |
+----------------------------------------------------------------------+
|                                                    4.71238898038469 |
+----------------------------------------------------------------------+
```

```
SELECT ST_Angle(ST_Point(0, 0),ST_Point(1, 0),ST_Point(0, 1));
```

第一条线到第二条线顺时针弧度为 0.78

```
+-----------------------------------------------------------------------+
| st_angle(st_point(0.0, 0.0), st_point(1.0, 0.0), st_point(0.0, 1.0)) |
+-----------------------------------------------------------------------+
|                                                    0.78547432161873854 |
+-----------------------------------------------------------------------+
```

```
SELECT ST_Angle(ST_Point(1, 0),ST_Point(0, 0),ST_Point(1, 0));
```

两线重叠，顺时针角度为 0, 返回 0

```
+-----------------------------------------------------------------------+
| st_angle(st_point(1.0, 0.0), st_point(0.0, 0.0), st_point(1.0, 0.0)) |
+-----------------------------------------------------------------------+
|                                                                     0 |
+-----------------------------------------------------------------------+
```

点 2 和点 3 是同一点，返回 NULL

```
SELECT ST_Angle(ST_Point(1, 0),ST_Point(0, 0),ST_Point(0, 0));
```

```
+-----------------------------------------------------------------------+
| st_angle(st_point(1.0, 0.0), st_point(0.0, 0.0), st_point(0.0, 0.0)) |
+-----------------------------------------------------------------------+
|                                                                  NULL |
+-----------------------------------------------------------------------+
```

两点对映，返回 NULL

```
SELECT ST_Angle(ST_Point(0, 0),ST_Point(-30, 0),ST_Point(150, 0));
```

```
+-------------------------------------------------------------------------+
| st_angle(st_point(0.0, 0.0), st_point(-30.0, 0.0), st_point(150.0, 0.0)) |
+-------------------------------------------------------------------------+
|                                                                    NULL |
+-------------------------------------------------------------------------+
```

任意一点是 NULL，则返回 NULL

```
mysql> SELECT ST_Angle(NULL,ST_Point(-30, 0),ST_Point(-150, 0)) ;
+-------------------------------------------------+
| ST_Angle(NULL,ST_Point(-30, 0),ST_Point(-150, 0)) |
+-------------------------------------------------+
|                                            NULL |
+-------------------------------------------------+
```

任意坐标超过规定点的范围，返回 NULL

```
mysql> SELECT ST_Angle(ST_Point(0, 0),ST_Point(-30, 0),ST_Point(180, 91));
+-----------------------------------------------------------+
| ST_Angle(ST_Point(0, 0),ST_Point(-30, 0),ST_Point(180, 91)) |
+-----------------------------------------------------------+
|                                                      NULL |
+-----------------------------------------------------------+
```

任一坐标为 NULL, 返回 NULL

```
mysql> SELECT ST_Angle(NULL,ST_Point(-30, 0),ST_Point(150, 90));
+---------------------------------------------------+
| ST_Angle(NULL,ST_Point(-30, 0),ST_Point(150, 90)) |
+---------------------------------------------------+
|                                              NULL |
+---------------------------------------------------+
```

7.2.1.4.2 ST_ANGLE_SPHERE

描述

计算地球表面上两点之间的圆心角（单位为度）。输入参数依次为点 X 的经度、点 X 的纬度、点 Y 的经度、点 Y 的纬度。圆心角是指地球球心处，连接两点的弧所对的角度。

语法

```
ST_ANGLE_SPHERE( <x_lng>, <x_lat>, <y_lng>, <y_lat>)
```

参数

参数	说明
<x_lng>	经度数据，类型为DOUBLE，合理的取值范围是 [-180, 180]
<y_lng>	经度数据，类型为DOUBLE, 合理的取值范围是 [-180, 180]
<x_lat>	纬度数据，类型为DOUBLE, 合理的取值范围是 [-90, 90]
<y_lat>	纬度数据，类型为DOUBLE, 合理的取值范围是 [-90, 90]

返回值

返回两点之间的圆心角，单位为度，类型为 DOUBLE，范围为 [0, 180]。

ST_ANGLE_SPHERE 存在以下边缘情况：

- 若任何输入参数为 NULL，返回 NULL。
- 若任何坐标超出范围（如经度 > 180、纬度 <-90），返回 NULL。
- 若两点为同一点（经纬度完全相同），返回 0。
- 若两点为对映点（地球上直径相对的点），返回 180。

举例

两个邻近点的圆心角计算

```
select ST_Angle_Sphere(116.35620117, 39.939093, 116.4274406433, 39.9020987219);
```

```
+-------------------------------------------------------------------------+
| st_angle_sphere(116.35620117, 39.939093, 116.4274406433, 39.9020987219) |
+-------------------------------------------------------------------------+
|                                                       0.0659823452409903 |
+-------------------------------------------------------------------------+
```

赤道上经度差 45° 的两点

```
select ST_Angle_Sphere(0, 0, 45, 0);
```

```
+---------------------------------------+
| st_angle_sphere(0.0, 0.0, 45.0, 0.0) |
+---------------------------------------+
|                                    45 |
+---------------------------------------+
```

两点坐标完全相同

```
mysql> SELECT ST_ANGLE_SPHERE(30, 60, 30, 60);
+---------------------------------+
| ST_ANGLE_SPHERE(30, 60, 30, 60) |
+---------------------------------+
|                               0 |
+---------------------------------+
```

对映点（直径相对的点）

```
mysql> SELECT ST_ANGLE_SPHERE(0, 0, 180, 0);
+-------------------------------+
| ST_ANGLE_SPHERE(0, 0, 180, 0) |
+-------------------------------+
|                           180 |
+-------------------------------+
```

跨东西经度的两点（如 170°E 和 - 170°W）

```
mysql> SELECT ST_ANGLE_SPHERE(170, 30, -170, 30);
+-----------------------------------+
| ST_ANGLE_SPHERE(170, 30, -170, 30) |
+-----------------------------------+
|                 17.298330210575152 |
+-----------------------------------+
```

跨赤道的南北两点

```
mysql> SELECT ST_ANGLE_SPHERE(0, 45, 0, -45);
+---------------------------------+
| ST_ANGLE_SPHERE(0, 45, 0, -45) |
+---------------------------------+
|              89.99999999999999 |
+---------------------------------+
```

无效经度（超出范围）

```
mysql> SELECT ST_ANGLE_SPHERE(190, 30, 10, 30);
+----------------------------------+
| ST_ANGLE_SPHERE(190, 30, 10, 30) |
+----------------------------------+
|                             NULL |
+----------------------------------+
```

任意参数为 NULL

```
mysql> SELECT ST_ANGLE_SPHERE(NULL, 30, 10, 30);
+-----------------------------------+
| ST_ANGLE_SPHERE(NULL, 30, 10, 30) |
+-----------------------------------+
|                              NULL |
+-----------------------------------+
```

纬度超出范围（如 91°N）

```
mysql> SELECT ST_ANGLE_SPHERE(0, 0, 180, 91);
+---------------------------------+
| ST_ANGLE_SPHERE(0, 0, 180, 91) |
+---------------------------------+
|                            NULL |
+---------------------------------+
```

7.2.1.4.3 ST_AREA_SQUARE_KM

描述

计算地球球面上闭合区域的面积，单位为平方千米。输入参数为表示地球表面区域的几何对象（如多边形、圆形、多面体等）。

对于非闭合几何对象（如点、线段），其面积为 0；对于无效几何对象（如自相交多边形），返回 NULL。

语法

```
ST_AREA_SQUARE_KM( <geo>)
```

参数

参数	说明
<geo>	地球球面上的几何对象，支持 GeoPolygon、 GeoCircle、 GeoMultiPolygon 等闭合区域类型。

返回值

ST_AREA_SQUARE_Km 存在以下边缘情况：

若输入参数为 NULL，返回 NULL。若输入为非闭合几何对象（如点 GeoPoint、线段 GeoLineString），返回 0。若输入几何对象无效（如自相交多边形、未闭合多边形），返回 NULL。若输入坐标超出经纬度范围（经度 [-180, 180]，纬度 [-90, 90]），返回 NULL。

举例

简单正方形区域（经纬度范围较小），边长约 1° 的正方形区域，面积约为 12,364 平方千米

```
SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))"));
```

```
+----------------------------------------------------------------------+
| st_area_square_km(st_polygon('POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))')) |
+----------------------------------------------------------------------+
|                                                 12364.036567076409 |
+----------------------------------------------------------------------+
```

更大范围的矩形区域

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"));
+-----------------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))")) |
+-----------------------------------------------------------------------------+
|                                                       1233204.7035253085 |
+-----------------------------------------------------------------------------+
```

包含曲线的复杂多边形

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 5 1, 10 0, 5 -1, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 5 1, 10 0, 5 -1, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                     123725.16642083102 |
+-------------------------------------------------------------------------+
```

跨经度 180° 的区域

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((179 0, 180 0, 180 1, 179 1, 179 0))"));
+-----------------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((179 0, 180 0, 180 1, 179 1, 179 0))")) |
+-----------------------------------------------------------------------------+
|                                                       12364.036567076282 |
+-----------------------------------------------------------------------------+
```

南半球区域

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 -10, 10 -10, 10 -20, 0 -20, 0 -10))"));
+-----------------------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((0 -10, 10 -10, 10 -20, 0 -20, 0 -10))")) |
+-----------------------------------------------------------------------------------+
|                                                                 1195196.6541230455 |
+-----------------------------------------------------------------------------------+
```

无效多边形（自相交）

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"));
+-----------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))")) |
+-----------------------------------------------------------------------+
|                                                                  NULL |
+-----------------------------------------------------------------------+
```

超出经纬度范围

```
mysql> SELECT ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 200 0, 200 1, 0 1, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Km(ST_Polygon("POLYGON ((0 0, 200 0, 200 1, 0 1, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                                    NULL |
+-------------------------------------------------------------------------+
```

空对象

```
mysql> SELECT ST_Area_Square_Km(NULL);
+-------------------------+
| ST_Area_Square_Km(NULL) |
+-------------------------+
|                    NULL |
+-------------------------+
```

计算圆的区域面积

```
mysql> SELECT ST_Area_Square_Km(ST_Circle(0, 0, 1));
+---------------------------------------+
| ST_Area_Square_Km(ST_Circle(0, 0, 1)) |
+---------------------------------------+
|                 3.141592653589787e-06 |
+---------------------------------------+
```

点对象（无面积）

```
SELECT ST_Area_Square_Km(ST_Point(0, 1));
```

```
+-----------------------------------------+
| st_area_square_Km(st_point(0.0, 1.0)) |
+-----------------------------------------+
|                                       0 |
+-----------------------------------------+
```

线段对象（无面积）

```
SELECT ST_Area_Square_Km(ST_LineFromText("LINESTRING (1 1, 2 2)"));
```

```
+-----------------------------------------------------------------+
| st_area_square_Km(st_linefromtext('LINESTRING (1 1, 2 2)')) |
+-----------------------------------------------------------------+
|                                                               0 |
+-----------------------------------------------------------------+
```

7.2.1.4.4 ST_AREA_SQUARE_METERS

描述

计算地球球面上闭合区域的面积，单位为平方米。输入参数为表示地球表面区域的几何对象（如多边形、圆形、多面体等）。

对于非闭合几何对象（如点、线段），其面积为 0；对于无效几何对象（如自相交多边形），返回 NULL。

语法

```
ST_AREA_SQUARE_METERS( <geo>)
```

参数

参数	说明
<geo>	地球球面上的几何对象，支持 GeoPolygon、GeoCircle、GeoMultiPolygon 等闭合区域类型。

返回值

返回区域的面积，单位为平方米，类型为 DOUBLE。

ST_AREA_SQUARE_METERS 存在以下边缘情况：

- 若输入参数为 NULL，返回 NULL。
- 若输入为非闭合几何对象（如点 GeoPoint、线段 GeoLineString），返回 0。
- 若输入几何对象无效（如自相交多边形、未闭合多边形），返回 NULL。
- 若输入坐标超出经纬度范围（经度 [-180, 180]，纬度 [-90, 90]），返回 NULL。

举例

圆形区域（半径 1 度的圆）

```
SELECT ST_Area_Square_Meters(ST_Circle(0, 0, 1));
```

```
+-------------------------------------------------+
| st_area_square_meters(st_circle(0.0, 0.0, 1.0)) |
+-------------------------------------------------+
|                              3.1415926535897869 |
+-------------------------------------------------+
```

点对象（无面积）

```
SELECT ST_Area_Square_Meters(ST_Point(0, 1));
```

```
+-----------------------------------------+
| st_area_square_meters(st_point(0.0, 1.0)) |
+-----------------------------------------+
|                                       0 |
+-----------------------------------------+
```

线段对象（无面积）

```
SELECT ST_Area_Square_Meters(ST_LineFromText("LINESTRING (1 1, 2 2)"));
```

```
+-----------------------------------------------------------------+
| st_area_square_meters(st_linefromtext('LINESTRING (1 1, 2 2)')) |
+-----------------------------------------------------------------+
|                                                               0 |
+-----------------------------------------------------------------+
```

简单正方形区域（经纬度范围较小）

```
mysql> SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                     12364036567.076408 |
+-------------------------------------------------------------------------+
```

复杂多边形区域

```
mysql> SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 5 1, 10 0, 5 -1, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 5 1, 10 0, 5 -1, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                       123725166420.83101 |
+-------------------------------------------------------------------------+
```

跨经度 180° 的矩形区域

```
mysql> SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((179 0, 180 0, 180 1, 179 1, 179 0))"));
+-----------------------------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((179 0, 180 0, 180 1, 179 1, 179 0))")) |
+-----------------------------------------------------------------------------------------+
|                                                                     12364036567.07628 |
+-----------------------------------------------------------------------------------------+
```

南半球圆形区域

```
mysql> SELECT ST_Area_Square_Meters(ST_Circle(0, -30, 2));
+--------------------------------------------+
| ST_Area_Square_Meters(ST_Circle(0, -30, 2)) |
+--------------------------------------------+
|                          12.566370614359073 |
+--------------------------------------------+
```

无效多边形（自相交）

```
mysql>  SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                                    NULL |
+-------------------------------------------------------------------------+
```

未闭合多边形

```
mysql> SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 0, 1 1, 0 1))"));
+-----------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 1 0, 1 1, 0 1))")) |
+-----------------------------------------------------------------------+
|                                                                  NULL |
+-----------------------------------------------------------------------+
```

坐标超出范围

```
mysql>  SELECT ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 200 0, 200 1, 0 1, 0 0))"));
+-------------------------------------------------------------------------+
| ST_Area_Square_Meters(ST_Polygon("POLYGON ((0 0, 200 0, 200 1, 0 1, 0 0))")) |
+-------------------------------------------------------------------------+
|                                                                    NULL |
+-------------------------------------------------------------------------+
```

空对象输入

```
mysql> SELECT ST_Area_Square_Meters(NULL);
+-----------------------------+
| ST_Area_Square_Meters(NULL) |
+-----------------------------+
|                        NULL |
+-----------------------------+
```

7.2.1.4.5 ST_ASBINARY

描述

将一个几何图形转换为标准的 WKB (Well-Known Binary) 二进制表示形式。WKB 是一种用于表示地理空间数据的二进制格式，广泛应用于地理信息系统 (GIS) 中。

目前支持的几何图形类型包括：Point（点）、LineString（线）、Polygon（多边形）

语法

```
ST_ASBINARY( <geo>)
```

参数

参数	说明
<geo>	需要转换为 WKB 格式的几何图形对象, 包括：Point（点）、LineString（线）、Polygon（多边形）

返回值

返回该几何图形的 WKB 二进制表示，以十六进制字符串形式展示（例如：`\x01010000...`）。

ST_ASBINARY 存在以下边缘情况：

- 若输入参数为 NULL，返回 NULL。
- 若输入的几何图形类型不支持，返回 NULL。

举例

点对象转换

```
select ST_AsBinary(st_point(24.7, 56.7));
```

```
+--------------------------------------------+
| st_asbinary(st_point(24.7, 56.7))          |
+--------------------------------------------+
| `\x0101000000333333333b338409a99999999594c40` |
+--------------------------------------------+
```

线对象转换

```
select ST_AsBinary(ST_GeometryFromText("LINESTRING (1 1, 2 2)"));
```

```
+-----------------------------------------------------------------------------------------+
| st_asbinary(st_geometryfromtext('LINESTRING (1 1, 2 2)'))                               |
+-----------------------------------------------------------------------------------------+
| `\x0102000000020000000000000000000f03f000000000000f03f0000000000000040000000000000040` |
+-----------------------------------------------------------------------------------------+
```

多边形对象转换

```
select ST_AsBinary(ST_Polygon("POLYGON ((114.104486 22.547119,114.093758 22.547753,114.096504
    ↪ 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486 22.547119))"));
```

```
+-----------------------------------------------------------------------------------------------------------------------------------
    ↪
| st_asbinary(st_polygon('POLYGON ((114.104486 22.547119,114.093758 22.547753,114.096504
    ↪ 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486 22.547119))'))
    ↪                                                                 |
+-----------------------------------------------------------------------------------------------------------------------------------
    ↪
| `\
    ↪ x010300000001000000060000000f3380ce6af865c402d05a4fd0f8c364041ef8d2100865c403049658a398c3640b9fb1c1f2d865c
    ↪ ` |
+-----------------------------------------------------------------------------------------------------------------------------------
    ↪
```

不支持的类型 MultiPolyGon 返回 NULL

```
mysql> SELECT ST_AsBinary(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2,
    ↪ 3 2, 3 3, 2 3, 2 2)))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsBinary(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2
    ↪ 3, 2 2)))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------
    ↪
| NULL
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------------------
    ↪
```

NULL 输入

```
mysql> SELECT ST_AsBinary(NULL);
+-------------------+
| ST_AsBinary(NULL) |
+-------------------+
| NULL              |
+-------------------+
```

7.2.1.4.6 ST_ASTEXT

描述

将一个几何图形转换为 WKT (Well-Known Text) 文本表示形式。WKT 是一种用于表示地理空间数据的文本格式，广泛应用于地理信息系统 (GIS) 中。

目前支持的几何图形类型包括：Point（点）、LineString（线）、Polygon（多边形）、MultiPolygon（多多边形），Circle(圆)

别名

- ST_ASWKT

语法

```
ST_ASTEXT( <geo>)
```

参数	说明
<geo>	需要转换为 WKT 格式的几何图形对象

返回值

该几何图形的 WKT 表示形式 ST_ASTEXT 存在以下边缘情况：

- 若输入参数为 NULL，返回 NULL。

举例

点对象转换

```
SELECT ST_AsText(ST_Point(24.7, 56.7));
```

```
+---------------------------------+
| st_astext(st_point(24.7, 56.7)) |
+---------------------------------+
| POINT (24.7 56.7)               |
+---------------------------------+
```

线对象转换

```
mysql> SELECT ST_AsText(ST_GeometryFromText("LINESTRING (1 1, 2 2)"));
+-----------------------------------------------------------+
| ST_AsText(ST_GeometryFromText("LINESTRING (1 1, 2 2)")) |
+-----------------------------------------------------------+
| LINESTRING (1 1, 2 2)                                     |
+-----------------------------------------------------------+
```

多边形对象转换

```
mysql> SELECT ST_AsText(ST_Polygon("POLYGON ((114.104486 22.547119,114.093758
    ↪ 22.547753,114.096504 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486
    ↪ 22.547119))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_Polygon("POLYGON ((114.104486 22.547119,114.093758 22.547753,114.096504
    ↪ 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486 22.547119))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| POLYGON ((114.104486 22.547119, 114.093758 22.547753, 114.096504 22.532057, 114.104229
    ↪ 22.539826, 114.106203 22.54268, 114.104486 22.547119))                       |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多多边形对象转换

```
mysql> SELECT ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3
    ↪ 2, 3 3, 2 3, 2 2)))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2 3,
    ↪  2 2)))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2 3, 2 2)))
    ↪                                     |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

圆的对象转换

```
mysql> SELECT ST_AsText(ST_Circle(116.39748, 39.90882, 0.5));
+-----------------------------------------------+
| ST_AsText(ST_Circle(116.39748, 39.90882, 0.5)) |
+-----------------------------------------------+
| CIRCLE ((116.39748 39.90882), 0.5)             |
+-----------------------------------------------+
```

NULL 输入

```
mysql> SELECT ST_AsText(NULL);
+-----------------+
| ST_AsText(NULL) |
+-----------------+
| NULL            |
+-----------------+
```

7.2.1.4.7 ST_AZIMUTH

描述

计算地球球面上两点之间的方位角（单位为弧度）。方位角是从起点（点 1）的真北方向线开始，顺时针旋转到两点连线的角度。

方位角广泛用于导航和地理信息系统中，表示从一点到另一点的方向。

语法

```
ST_AZIMUTH( <point1>, <point2>)
```

参数

参数	说明
<point1>	起点，类型为 GeoPoint，用于计算方位角的基准点
<point2>	终点，类型为 GeoPoint，需要计算相对于起点的方向

返回值

返回两点之间的方位角，单位为弧度，范围为 [0, 2π)。方位角按顺时针方向从真北开始测量，具体方向对应关系：

- 正北方：0 弧度
- 正东方：π/2 弧度（约 1.5708）
- 正南方：π 弧度（约 3.1416）
- 正西方：3π/2 弧度（约 4.7124）

ST_Azimuth 存在以下边缘情况：

- 若两点为同一点（经纬度完全相同），返回 NULL（方向无意义）。
- 若两点为对映点（地球直径两端的点），返回 NULL（存在两个可能方向，无法唯一确定）。
- 若输入参数不是 GeoPoint 类型（如线、多边形），或为空几何对象，返回 NULL。

举例

正西方（从 (1,0) 到 (0,0)）

```
SELECT st_azimuth(ST_Point(1, 0),ST_Point(0, 0));
```

```
+-----------------------------------------------------+
| st_azimuth(st_point(1.0, 0.0), st_point(0.0, 0.0)) |
+-----------------------------------------------------+
|                                   4.71238898038469 |
+-----------------------------------------------------+
```

正东方（从 (0,0) 到 (1,0)）

```
SELECT st_azimuth(ST_Point(0, 0),ST_Point(1, 0));
```

```
+-----------------------------------------------------+
| st_azimuth(st_point(0.0, 0.0), st_point(1.0, 0.0)) |
+-----------------------------------------------------+
|                                 1.5707963267948966 |
+-----------------------------------------------------+
```

正北方（从 (0,0) 到 (0,1)）

```
SELECT st_azimuth(ST_Point(0, 0),ST_Point(0, 1));
```

```
+-----------------------------------------------------+
| st_azimuth(st_point(0.0, 0.0), st_point(0.0, 1.0)) |
+-----------------------------------------------------+
|                                                   0 |
+-----------------------------------------------------+
```

两点对映

```
SELECT st_azimuth(ST_Point(-30, 0),ST_Point(150, 0));
```

```
+-------------------------------------------------------+
| st_azimuth(st_point(-30.0, 0.0), st_point(150.0, 0.0)) |
+-------------------------------------------------------+
|                                                  NULL |
+-------------------------------------------------------+
```

东北方向（从 (0,0) 到 (1,1)）

```
mysql> SELECT st_azimuth(ST_Point(0, 0), ST_Point(1, 1));
+-------------------------------------------+
| st_azimuth(ST_Point(0, 0), ST_Point(1, 1)) |
+-------------------------------------------+
|                         0.7854743216187384 |
+-------------------------------------------+
```

跨经度 180° 的东方（从 (170, 0) 到 (-170, 0)）

```
mysql> SELECT st_azimuth(ST_Point(170, 0), ST_Point(-170, 0));
+-------------------------------------------------+
| st_azimuth(ST_Point(170, 0), ST_Point(-170, 0)) |
+-------------------------------------------------+
|                              1.5707963267948966 |
+-------------------------------------------------+
```

输入非点类型，返回 NULL

```
mysql> SELECT st_azimuth(ST_LineFromText("LINESTRING (0 0, 1 1)"), ST_Point(1, 0));
+-----------------------------------------------------------------------+
| st_azimuth(ST_LineFromText("LINESTRING (0 0, 1 1)"), ST_Point(1, 0)) |
+-----------------------------------------------------------------------+
|                                                                  NULL |
+-----------------------------------------------------------------------+
```

7.2.1.4.8 ST_CIRCLE

描述

将一个 WKT（Well Known Text）转化为地球球面上的一个圆。

语法

```
ST_CIRCLE( <center_lng>, <center_lat>, <radius>)
```

参数

参数	说明
<center_lng>	圆心的经度，类型为 DOUBLE，取值范围为 [-180, 180]
<center_lat>	圆心的纬度，类型为 DOUBLE，取值范围为 [-180, 180]
<radius>	圆的半径，类型为 DOUBLE，单位为米

返回值

返回地球球面上的圆，类型为 GeoCircle。其 WKT 表示形式为 CIRCLE (( ),)，其中包含圆心坐标和半径信息。

ST_CIRCLE 存在以下边缘情况：

- 若任何输入参数为 NULL，返回 NULL。
- 若超出 [-180, 180] 或超出 [-90, 90]，返回 NULL。
- 若为 0，返回以圆心为唯一点的特殊圆（WKT 表示为 CIRCLE (( ), 0) ）。

举例

基本用法（正常圆）

```
SELECT ST_AsText(ST_Circle(111, 64, 10000));
```

```
+-----------------------------------------+
| st_astext(st_circle(111.0, 64.0, 10000.0)) |
+-----------------------------------------+
| CIRCLE ((111 64), 10000)                  |
+-----------------------------------------+
```

赤道上的圆（纬度 0°）

```
mysql> SELECT ST_AsText(ST_Circle(0, 0, 5000));
+---------------------------------+
| ST_AsText(ST_Circle(0, 0, 5000)) |
+---------------------------------+
| CIRCLE ((0 0), 5000)             |
+---------------------------------+
```

半径为 0 的圆（退化为点）

```
mysql> SELECT ST_AsText(ST_Circle(120, 30, 0));
+---------------------------------+
| ST_AsText(ST_Circle(120, 30, 0)) |
+---------------------------------+
| CIRCLE ((120 30), 0)             |
+---------------------------------+
```

无效参数（经度超出范围）

```
mysql> SELECT ST_AsText(ST_Circle(190, 30, 1000));
+-----------------------------------+
| ST_AsText(ST_Circle(190, 30, 1000)) |
+-----------------------------------+
| NULL                              |
+-----------------------------------+
```

参数为 NULL

```
mysql> SELECT ST_AsText(ST_Circle(NULL, 30, 1000));
+--------------------------------------+
| ST_AsText(ST_Circle(NULL, 30, 1000)) |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

7.2.1.4.9 ST_CONTAINS

描述

判断一个几何图形（shape1）是否完全包含另一个几何图形（shape2）。若 shape1 包含 shape2 的所有点，则返回 1；否则返回 0。

- 对于点：点必须位于多边形内部或边界上。
- 对于线：线的所有点必须位于多边形内部或边界上。
- 对于多边形：被包含的多边形必须完全位于外部多边形内部（边界可以重叠）。###### 语法

```
ST_CONTAINS( <shape1>, <shape2>)
```

参数

参数	说明
<shape1>	用于判断是否包含其他图形的几何图形，支持 Polygon 类型。
<shape2>	用于判断是否被包含的几何图形, 支持 Point、 Line,Polygon 等类型。

返回值

返回 1:shape1 图形可包含图形 shape2

返回 0:shape1 图形不可包含图形 shape2

ST_CONTAINS 存在以下边缘情况：

- 若任一输入参数为 NULL，返回 NULL。
- 若输入的几何图形无效（如自相交的多边形），返回 NULL。
- 若 shape2 的边界与 shape1 的边界部分重叠，但 shape2 有部分点在 shape1 外部，返回 0。

举例

多边形包含点（点在内部）

```
SELECT ST_Contains(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(5, 5));
```

```
+-------------------------------------------------------------------------------------+
| st_contains(st_polygon('POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))'), st_point(5.0, 5.0)) |
```

```
+-----------------------------------------------------------------------------------------+
|                                                                                       1 |
+-----------------------------------------------------------------------------------------+
```

多边形不包含点（点在外部）

```
SELECT ST_Contains(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(50, 50));
```

```
+-----------------------------------------------------------------------------------------------+
| st_contains(st_polygon('POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))'), st_point(50.0, 50.0)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             0 |
+-----------------------------------------------------------------------------------------------+
```

多边形包含线（线完全在内部）

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (2 2, 8 8)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (2 2, 8 8)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形不包含线（线部分在外部）

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (5 5, 15 15)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (5 5, 15 15)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

```
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形包含多边形（内部多边形完全被包含）

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((2 2, 8 2, 8 8, 2 8, 2 2))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((2
    ↪ 2, 8 2, 8 8, 2 8, 2 2))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形不包含多边形（内部多边形部分在外部）

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((5 5, 15 5, 15 15, 5 15, 5 5))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((5
    ↪ 5, 15 5, 15 15, 5 15, 5 5))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形包含边界点（点在多边形边界上）

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0,
    ↪ 5));
+-----------------------------------------------------------------------------------------+
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0, 5)) |
+-----------------------------------------------------------------------------------------+
|                                                                                       0 |
+-----------------------------------------------------------------------------------------+
```

参数为 NULL（返回 NULL）

```
mysql> SELECT ST_Contains(NULL, ST_Point(5, 5));
+-----------------------------------+
| ST_Contains(NULL, ST_Point(5, 5)) |
+-----------------------------------+
|                              NULL |
+-----------------------------------+
```

自相交多边形作为参数

```
mysql> SELECT ST_Contains(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"), ST_Point(0.5, 0.5)
    ↪ );
+-----------------------------------------------------------------------------------------+
| ST_Contains(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"), ST_Point(0.5, 0.5)) |
+-----------------------------------------------------------------------------------------+
|                                                                                    NULL |
+-----------------------------------------------------------------------------------------+
```

7.2.1.4.10 ST_DISJOINT

描述

判断两个几何图形是否完全不相交（即没有任何公共点）。若两图形的边界、内部均无交集，则返回 1，否则返回 0。

> 备注从 Apache Doris 3.0.6 开始支持该函数

语法

```
ST_DISJOINT( <shape1>, <shape2> )
```

参数

参数	说明
<shape1>	用于判断是否不相交的第一个几何图形，支持 Point、 Line、 Polygon,Circle
<shape2>	用于判断是否不相交的第二个几何图形，支持 Point、 Line、 Polygon,Circle

返回值

返回 1: shape1 图形与图形 shape2 不相交

返回 0: shape1 图形与图形 shape2 相交

ST_DISJOINT 存在以下边缘情况：

- 若任一输入参数为 NULL，返回 NULL。
- 若输入的几何图形无效，返回 NULL。
- 若输入为空几何图形，返回 1（空图形无任何点，与任何图形均不相交）。

举例

多边形与内部点（相交，返回 0）

```
SELECT ST_Disjoint(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(5, 5));
```

```
+-------------------------------------------------------------------------------------+
| ST_Disjoint(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(5, 5)) |
+-------------------------------------------------------------------------------------+
|                                                                                   0 |
+-------------------------------------------------------------------------------------+
```

多边形与外部点（不相交，返回 1）

```
SELECT ST_Disjoint(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(50, 50));
```

```
+---------------------------------------------------------------------------------------+
| ST_Disjoint(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(50, 50)) |
+---------------------------------------------------------------------------------------+
|                                                                                     1 |
+---------------------------------------------------------------------------------------+
```

点与线不相交（点在线外）

```
mysql> SELECT ST_Disjoint(ST_Linefromtext("LINESTRING (0 0, 10 10)"), ST_Point(1, 2));
+--------------------------------------------------------------------------+
| ST_Disjoint(ST_Linefromtext("LINESTRING (0 0, 10 10)"), ST_Point(1, 2)) |
+--------------------------------------------------------------------------+
|                                                                        1 |
```

点与线相交（点在线上）

```
mysql> SELECT ST_Disjoint(ST_Linefromtext("LINESTRING (0 0, 10 10)"), ST_Point(5, 5));
+-----------------------------------------------------------------------------+
| ST_Disjoint(ST_Linefromtext("LINESTRING (0 0, 10 10)"), ST_Point(5, 5)) |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

线与线相交

```
mysql> SELECT ST_Disjoint(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("
    ↪ LINESTRING (0 10, 10 0)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
| ST_Disjoint(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("LINESTRING (0 10,
    ↪ 10 0)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
```

线与线不相交（平行且无重叠）

```
mysql> SELECT ST_Disjoint(  ST_Linefromtext("LINESTRING (0 0, 10 0)"),  ST_Linefromtext("
    ↪ LINESTRING (0 1, 10 1)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
| ST_Disjoint(  ST_Linefromtext("LINESTRING (0 0, 10 0)"),  ST_Linefromtext("LINESTRING (0 1, 10
    ↪ 1)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
```

多边形与多边形不相交（完全分离）

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((20 20, 30 20, 30 30, 20 30, 20 20))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((20
    ↪ 20, 30 20, 30 30, 20 30, 20 20))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形相交（部分重叠）

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((5 5, 15 5, 15 15, 5 15, 5 5))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((5
    ↪ 5, 15 5, 15 15, 5 15, 5 5))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

空几何图形与任何图形不相交

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ GeometryFromText("POINT EMPTY"));
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_GeometryFromText("
    ↪ POINT EMPTY")) |
```

```
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ NULL |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

无效几何图形(自相交多边形)作为参数

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),  ST_Point(0.5,
    ↪ 0.5));
+-------------------------------------------------------------------------------------------+
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),  ST_Point(0.5, 0.5)) |
+-------------------------------------------------------------------------------------------+
|                                                                                      NULL |
+-------------------------------------------------------------------------------------------+
```

NULL 参数返回 NULL

```

mysql> SELECT ST_Disjoint(ST_Point(0,0), NULL);
+----------------------------------+
| ST_Disjoint(ST_Point(0,0), NULL) |
+----------------------------------+
|                             NULL |
+----------------------------------+
```

圆与多边形相交

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Circle(5, 2.5,
    ↪  2000));
+-------------------------------------------------------------------------------------------+
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Circle(5, 2.5, 2000)) |
+-------------------------------------------------------------------------------------------+
|                                                                                         0 |
+-------------------------------------------------------------------------------------------+
```

圆与多边形不相交

```
mysql> SELECT ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(20,
    ↪  5, 5));
+-------------------------------------------------------------------------------------------+
| ST_Disjoint(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(20, 5, 5)) |
+-------------------------------------------------------------------------------------------+
```

```
|                                                
```

```
| ST_DISTANCE_SPHERE(0, 0, 1, 0) |
+--------------------------------+
|             111195.10117748393 |
+--------------------------------+
```

实际跨经度 20° 的近距点

```
-- 点X(170, 0)，点Y(-170, 0)（经度差20°，而非340°，取最短路径）
mysql> SELECT ST_DISTANCE_SPHERE(170, 0, -170, 0);
+-------------------------------------+
| ST_DISTANCE_SPHERE(170, 0, -170, 0) |
+-------------------------------------+
|                   2223902.023549678 |
+-------------------------------------+
```

无效参数（纬度超出范围）

```
mysql> SELECT ST_DISTANCE_SPHERE(116, 39, 120, 91);
+--------------------------------------+
| ST_DISTANCE_SPHERE(116, 39, 120, 91) |
+--------------------------------------+
|                                 NULL |
+--------------------------------------+
```

NULL 参数返回 NULL

```
mysql> SELECT ST_DISTANCE_SPHERE(NULL, 39.9, 116.4, 30);
+-----------------------------------------+
| ST_DISTANCE_SPHERE(NULL, 39.9, 116.4, 30) |
+-----------------------------------------+
|                                    NULL |
+-----------------------------------------+
```

两个坐标完全一样，返回 0

```
mysql>  SELECT ST_DISTANCE_SPHERE(1,1 , 1, 1);
+---------------------------------+
| ST_DISTANCE_SPHERE(1,1 , 1, 1) |
+---------------------------------+
|                               0 |
+---------------------------------+
```

7.2.1.4.12 ST_GEOMETRYFROMTEXT

描述

将一个 WKT（Well Known Text）转化为对应的内存的几何形式

别名

- ST_GEOMFROMTEXT

语法

```
ST_GEOMETRYFROMTEXT( <wkt>)
```

参数

参数	说明
<wkt>	图形的内存形式

支持的 WKT 格式: - POINT - 空间中的单个点 - LINESTRING - 连接的线段序列 - POLYGON - 由一个或多个环定义的封闭区域, 要求至少有三个不同的点且首尾闭合 - MULTIPOLYGON - 多边形的集合, 要求多边形之间仅能存在有限个离散点的接触

> 备注从 Apache Doris 3.0.6 开始支持 MULTIPOLYGON 格式解析

返回值

WKB 的对应的几何存储形式

当输入的 WKT 格式不符合规范或输入为 NULL 时返回 NULL。

举例

```
-- POINT 样例
SELECT ST_AsText(ST_GeometryFromText("POINT (1 1)"));
```

```
+--------------------------------------------+
| ST_AsText(ST_GeometryFromText("POINT (1 1)")) |
+--------------------------------------------+
| POINT (1 1)                                |
+--------------------------------------------+
```

```
-- POINT 不合法样例(端点过多)
SELECT ST_AsText(ST_GeometryFromText("POINT (1 1, 2 2)"));
```

```
+-------------------------------------------------+
| ST_AsText(ST_GeometryFromText("POINT (1 1, 2 2)")) |
+-------------------------------------------------+
```

```
| NULL                                                |
+-----------------------------------------------------+
```

```
-- LINESTRING 样例
SELECT ST_AsText(ST_GeometryFromText("LINESTRING (1 1, 2 2)"));
```

```
+-----------------------------------------------------------+
| st_astext(st_geometryfromtext('LINESTRING (1 1, 2 2)')) |
+-----------------------------------------------------------+
| LINESTRING (1 1, 2 2)                                     |
+-----------------------------------------------------------+
```

```
-- LINESTRING 不合法样例(端点过少)
SELECT ST_AsText(ST_GeometryFromText("LINESTRING (1 1)"));
```

```
+-----------------------------------------------------+
| ST_AsText(ST_GeometryFromText("LINESTRING (1 1)")) |
+-----------------------------------------------------+
| NULL                                                |
+-----------------------------------------------------+
```

```
-- POLYGON 样例
SELECT ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))"));
```

```
+-------------------------------------------------------------------------+
| ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))")) |
+-------------------------------------------------------------------------+
| POLYGON ((0 0, 1 0, 1 1, 0 1, 0 0))                                     |
+-------------------------------------------------------------------------+
```

```
-- POLYGON 不合法样例(首尾不闭合)
SELECT ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 1 1, 0 1))"));
```

```
+-----------------------------------------------------------------+
| ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 1 1, 0 1))")) |
+-----------------------------------------------------------------+
| NULL                                                            |
+-----------------------------------------------------------------+
```

```
-- POLYGON 不合法样例(端点过少)
SELECT ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 0 0))"));
```

```
+-----------------------------------------------------------+
| ST_AsText(ST_GeometryFromText("POLYGON ((0 0, 1 0, 0 0))")) |
```

```
+------------------------------------------------------------+
| NULL                                                       |
+------------------------------------------------------------+
```

```
-- MULTIPOLYGON 样例
SELECT ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3,
    ↪  2 3, 2 2)))"));
```

```
+-------------------------------------------------------------------------------------------------------------+
    ↪
| ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2 3,
    ↪  2 2)))")) |
+-------------------------------------------------------------------------------------------------------------+
    ↪
| MULTIPOLYGON (((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2 3, 2 2)))
    ↪                                    |
+-------------------------------------------------------------------------------------------------------------+
    ↪
```

```
-- MULTIPOLYGON 样例(仅有有限个离散点接触)
SELECT ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 10 0, 10 10, 0 10, 0 0), (4 4, 6 4, 6
    ↪ 6, 4 6, 4 4)), ((4 5, 5 4, 6 5, 5 6, 4 5)))"));
```

```
+-------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 10 0, 10 10, 0 10, 0 0), (4 4, 6 4, 6 6, 4
    ↪ 6, 4 4)), ((4 5, 5 4, 6 5, 5 6, 4 5)))")) |
+-------------------------------------------------------------------------------------------------------------
    ↪
| MULTIPOLYGON (((0 0, 10 0, 10 10, 0 10, 0 0), (4 4, 6 4, 6 6, 4 6, 4 4)), ((4 5, 5 4, 6 5, 5 6,
    ↪  4 5)))                                    |
+-------------------------------------------------------------------------------------------------------------
    ↪
```

```
-- MULTIPOLYGON 不合法样例(存在重叠部分)
SELECT ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 10 0, 10 10, 0 10, 0 0)), ((10 0, 20
    ↪ 0, 20 10, 10 10, 10 0)))"));
```

```
+-------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_GeometryFromText("MULTIPOLYGON (((0 0, 10 0, 10 10, 0 10, 0 0)), ((10 0, 20 0, 20
    ↪ 10, 10 10, 10 0)))")) |
+-------------------------------------------------------------------------------------------------------------
    ↪
```

```
| NULL
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
-- 输入 NULL
SELECT ST_AsText(ST_GeometryFromText(NULL));
```

```
+--------------------------------------+
| ST_AsText(ST_GeometryFromText(NULL)) |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

7.2.1.4.13 ST_GEOMETRYFROMWKB

描述

将一个标准图形 WKB（Well-known binary）转化为对应的内存的几何形式

别名

- ST_GEOMFROMWKB

语法

```
ST_GEOMETRYFROMWKB( <wkb>)
```

参数

参数	说明
<wkb>	二进制形式的几何数据，通常由 ST_AsBinary 或其他 WKB 生成函数返回。支持所有标准几何类型（POINT、LINESTRING、POLYGON、CIRCLE 几种集合类型

返回值

返回对应 WKB 表示的几何对象。如果输入为 NULL 或无效 WKB 格式，返回 NULL。

举例

POINT 类型参数

```
select ST_AsText(ST_GeometryFromWKB(ST_AsBinary(ST_Point(24.7, 56.7))));
```

```
+-----------------------------------------------------------------+
| st_astext(st_geometryfromwkb(st_asbinary(st_point(24.7, 56.7)))) |
+-----------------------------------------------------------------+
| POINT (24.7 56.7)                                               |
+-----------------------------------------------------------------+
```

```
select ST_AsText(ST_GeomFromWKB(ST_AsBinary(ST_Point(24.7, 56.7))));
```

```
+-----------------------------------------------------------+
| st_astext(st_geomfromwkb(st_asbinary(st_point(24.7, 56.7)))) |
+-----------------------------------------------------------+
| POINT (24.7 56.7)                                         |
+-----------------------------------------------------------+
```

LINE 类型参数

```
select ST_AsText(ST_GeometryFromWKB(ST_AsBinary(ST_GeometryFromText("LINESTRING (1 1, 2 2)"))));
```

```
+-------------------------------------------------------------------------------------------+
| st_astext(st_geometryfromwkb(st_asbinary(st_geometryfromtext('LINESTRING (1 1, 2 2)')))) |
+-------------------------------------------------------------------------------------------+
| LINESTRING (1 1, 2 2)                                                                     |
+-------------------------------------------------------------------------------------------+
```

POLYGON 类型参数

```
select ST_AsText(ST_GeometryFromWKB(ST_AsBinary(ST_Polygon("POLYGON ((114.104486
    ↪ 22.547119,114.093758 22.547753,114.096504 22.532057,114.104229 22.539826,114.106203
    ↪ 22.542680,114.104486 22.547119))"))));
```

```
+-------------------------------------------------------------------------------------------------------------------------------
    ↪
| st_astext(st_geometryfromwkb(st_asbinary(st_polygon('POLYGON ((114.104486 22.547119,114.093758
    ↪ 22.547753,114.096504 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486
    ↪ 22.547119))')))) |
+-------------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
| POLYGON ((114.104486 22.547119, 114.093758 22.547753, 114.096504 22.532057, 114.104229
    ↪ 22.539826, 114.106203 22.54268, 114.104486 22.547119))
    ↪                                                    |
+-----------------------------------------------------------------------------------------------------------
    ↪
```

```
select ST_AsText(ST_GeomFromWKB(ST_AsBinary(ST_Polygon("POLYGON ((114.104486 22.547119,114.093758
    ↪  22.547753,114.096504 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486
    ↪ 22.547119))"))));
```

```
+-----------------------------------------------------------------------------------------------------------
    ↪
| st_astext(st_geomfromwkb(st_asbinary(st_polygon('POLYGON ((114.104486 22.547119,114.093758
    ↪ 22.547753,114.096504 22.532057,114.104229 22.539826,114.106203 22.542680,114.104486
    ↪ 22.547119))')))) |
+-----------------------------------------------------------------------------------------------------------
    ↪
| POLYGON ((114.104486 22.547119, 114.093758 22.547753, 114.096504 22.532057, 114.104229
    ↪ 22.539826, 114.106203 22.54268, 114.104486 22.547119))
    ↪                                                    |
+-----------------------------------------------------------------------------------------------------------
    ↪
```

输入为 NULL

```
mysql> SELECT ST_AsText(ST_GeometryFromWKB(NULL));
+--------------------------------------+
| ST_AsText(ST_GeometryFromWKB(NULL)) |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

有效的的 wkb 格式

```
mysql> select ST_AsText(ST_GeometryFromWKB("\
    ↪ x0103000000010000000500000000000000000024400000000000000000000000000000244000000000000024400000000000000000
    ↪ "));
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_GeometryFromWKB("\
    ↪ x0103000000010000000500000000000000000024400000000000000000000000000000244000000000000024400000000000000000
    ↪ ")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

```
| POLYGON ((10 0, 10 10, 0 10, 0 0, 10 0))
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

无效的 wkb 格式

```

mysql> select ST_AsText(ST_GeometryFromWKB("
    ↪ \0103000000010000000500000000000000000024400000000000000000000000000000244000000000000024400000000000000000
    ↪ "));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_AsText(ST_GeometryFromWKB("
    ↪ \0103000000010000000500000000000000000024400000000000000000000000000000244000000000000024400000000000000000
    ↪ ")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| NULL
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.4.14 ST_INTERSECTS

描述

判断两个几何图形是否相交，即存在至少一个公共点（包括边界接触或内部重叠）

> 备注从 Apache Doris 3.0.6 开始支持该函数

语法

```
ST_INTERSECTS( <shape1>, <shape2>)
```

参数

参数	说明
<shape1>	用于判断是否相交的第一个几何图形，支持 Point、Line、Polygon、Circle 等类型。
<shape2>	用于判断是否相交的第二个几何图形，支持 Point、Line、Polygon、Circle 等类型。

返回值

返回 1: shape1 图形与图形 shape2 相交

返回 0: shape1 图形与图形 shape2 不相交

ST_INTERSECTS 存在以下边缘情况：

- 若任一输入参数为 NULL，返回 NULL。
- 若输入的几何图形无效，返回 NULL。
- 若输入为空几何图形，返回 NULL。
- 若两个图形仅边界相切（存在唯一公共点），返回 1（边界接触视为相交）。

举例

点在多边形内部（相交，返回 1）

```
SELECT ST_Intersects(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(5, 5));
```

```
+-------------------------------------------------------------------------------------+
| ST_Intersects(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), ST_Point(5, 5)) |
+-------------------------------------------------------------------------------------+
|                                                                                   1 |
+-------------------------------------------------------------------------------------+
```

点在多边形边界上（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0,
    ↪  5));
+-------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0, 5)) |
+-------------------------------------------------------------------------------------+
|                                                                                   1 |
+-------------------------------------------------------------------------------------+
```

点在多边形外部（不相交，返回 0）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point
    ↪ (50, 50));
+-------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(50, 50)) |
+-------------------------------------------------------------------------------------+
|                                                                                   0 |
+-------------------------------------------------------------------------------------+
```

线与线交叉（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("
    ↪ LINESTRING (0 10, 10 0)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("LINESTRING (0 10,
    ↪  10 0)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
```

线与线有一个公共点

```
mysql> SELECT ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 2 2)"),  ST_Linefromtext("
    ↪ LINESTRING (2 2, 4 0)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 2 2)"),  ST_Linefromtext("LINESTRING (2 2, 4
    ↪ 0)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
```

线与线平行且分离（不相交，返回 0）

```
mysql> SELECT ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 10 0)"),  ST_Linefromtext("
    ↪ LINESTRING (0 1, 10 1)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| ST_INTERSECTS(  ST_Linefromtext("LINESTRING (0 0, 10 0)"),  ST_Linefromtext("LINESTRING (0 1,
    ↪ 10 1)")) |
```

```
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------+
    ↪
```

线穿过多边形内部（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (2 2, 8 8)"));
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (2 2, 8 8)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
```

线与多边形边界相切（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (0 5, 5 5)"));
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (0 5, 5 5)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------
    ↪
```

线完全在多边形外部（不相交，返回 0）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (11 1, 11 9)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (11 1, 11 9)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形重叠（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon(
    ↪ "POLYGON ((5 5, 15 5, 15 15, 5 15, 5 5))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((5
    ↪  5, 15 5, 15 15, 5 15, 5 5))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形完全分离（不相交，返回 0）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon(
    ↪ "POLYGON ((20 20, 30 20, 30 30, 20 30, 20 20))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

```
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON
    ↪ ((20 20, 30 20, 30 30, 20 30, 20 20))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形边界接触（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((5 0, 10 0, 10 5, 5 5, 5 0))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Polygon("POLYGON ((5 0,
    ↪ 10 0, 10 5, 5 5, 5 0))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
```

圆与点在内部（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Circle(0, 0, 1000),  ST_Point(0.005, 0));
+-------------------------------------------------------------+
| ST_INTERSECTS(  ST_Circle(0, 0, 1000),  ST_Point(0.005, 0)) |
+-------------------------------------------------------------+
|                                                           1 |
+-------------------------------------------------------------+
```

圆与线相切（相交，返回 1）

```
mysql> SELECT ST_INTERSECTS(  ST_Circle(0, 0, 1000),  ST_Linefromtext("LINESTRING (0.01 0.01,
    ↪ 0.02 0.02)"));
+-----------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Circle(0, 0, 1000),  ST_Linefromtext("LINESTRING (0.01 0.01, 0.02 0.02)")) |
```

```
+-----------------------------------------------------------------------------------------------+
|                                                                                             1 |
+-----------------------------------------------------------------------------------------------+
```

圆与多边形完全分离（不相交，返回 0）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle
    ↪ (20, 5, 5));
+-----------------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(20, 5, 5)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             0 |
+-----------------------------------------------------------------------------------------------+
```

圆与多边形相交

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Circle(5,
    ↪ 2.5, 2000));
+-----------------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Circle(5, 2.5, 2000)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             1 |
+-----------------------------------------------------------------------------------------------+
```

空几何图形

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ GeometryFromText("POINT EMPTY"));
+--
    ↪ ------------------------------------------------------------------------------------------------
    ↪
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_GeometryFromText("
    ↪ POINT EMPTY")) |
+--
    ↪ ------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ NULL |
+--
    ↪ ------------------------------------------------------------------------------------------------
    ↪
```

无效多边形（返回 NULL）

```
mysql> SELECT ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),  ST_Point(0.5,
    ↪ 0.5));
```

```
+-----------------------------------------------------------------------------------------------+
| ST_INTERSECTS(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),  ST_Point(0.5, 0.5)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                          NULL |
+-----------------------------------------------------------------------------------------------+
```

参数为 NULL（返回 NULL）

```
mysql> SELECT ST_INTERSECTS(NULL, ST_Point(5, 5));
+-------------------------------------+
| ST_INTERSECTS(NULL, ST_Point(5, 5)) |
+-------------------------------------+
|                                NULL |
+-------------------------------------+
```

7.2.1.4.15 ST_LINEFROMTEXT

描述

将一个 WKT（Well Known Text）转化为一个 Line 形式的内存表现形式

别名

- ST_LINESTRINGFROMTEXT

语法

```
ST_LINEFROMTEXT( <wkt>)
```

参数

参数	说明
<wkt>	符合 LINE 类型的 WKT 字符串，格式为：

“LINE (x1 y1, x2 y2)” 其中，(x1 y1), (x2 y2) 是线段的顶点坐标，坐标值为数值（整数或小数）。 |

返回值

返回 Line 类型的几何对象，该对象在内存中以 Doris 内部的空间数据格式存储，可直接作为参数传入其他空间函数（如 ST_LENGTH、ST_INTERSECTS 等）进行计算。

- 若输入的 WKT 字符串格式无效（如顶点数量不足 2 个、语法错误、坐标非数值等），返回 NULL。
- 若输入为 NULL 或空字符串，返回 NULL。

举例

正常 LINE 类型

```
mysql> SELECT ST_AsText(ST_LineFromText("LINESTRING (1 1, 2 2)"));
+-------------------------------------------------+
| ST_AsText(ST_LineFromText("LINESTRING (1 1, 2 2)")) |
+-------------------------------------------------+
| LINESTRING (1 1, 2 2)                           |
+-------------------------------------------------+
```

无效 WKT (顶点数量不足)

```
mysql> SELECT ST_LineFromText("LINESTRING (1 1)");
+-------------------------------------+
| ST_LineFromText("LINESTRING (1 1)") |
+-------------------------------------+
| NULL                                |
+-------------------------------------+
```

无效 WKT (语法错误)

```
mysql> SELECT ST_LineFromText("LINESTRING (1 1, 2 2");
+----------------------------------------+
| ST_LineFromText("LINESTRING (1 1, 2 2") |
+----------------------------------------+
| NULL                                   |
+----------------------------------------+
```

无效 WKT(顶点数量太多)

```
mysql> SELECT ST_LineFromText("LINESTRING (1 1,2 2,3 3)");
+-------------------------------------------------------------------------------------+
| ST_LineFromText("LINESTRING (1 1,2 2,3 3)")                                         |
+-------------------------------------------------------------------------------------+
|     ??_<???'???X?? ?
                       ??????(Qjm? ?'???X??3|??lW<?`?????H??       |
+-------------------------------------------------------------------------------------+
```

输入 NULL

```
mysql> SELECT ST_LineFromText(NULL);
+-----------------------+
| ST_LineFromText(NULL) |
+-----------------------+
| NULL                  |
+-----------------------+
```

7.2.1.4.16 ST_POINT

描述

通过给定的 X 坐标值，Y 坐标值返回对应的 Point。

当前这个值只是在球面集合上有意义，X/Y 对应的是经度/纬度 (longitude/latitude);

语法

```
ST_POINT( <x>, <y>)
```

参数

参数	说明
<x>	点的 X 坐标值（经度），取值范围：-180.0 至 180.0（单位：度）
<y>	点的 Y 坐标值（纬度），取值范围：-90.0 至 90.0（单位：度）

返回值

返回 Point 类型的几何对象，表示一个二维坐标点。

- 若或超出有效经纬度范围，函数可能返回 NULL 或生成无效点（取决于 Doris 版本）。
- 若任一参数为 NULL，则返回 NULL。

举例

正常经纬度点

```
SELECT ST_AsText(ST_Point(24.7, 56.7));
```

```
+---------------------------------+
| st_astext(st_point(24.7, 56.7)) |
+---------------------------------+
| POINT (24.7 56.7)               |
+---------------------------------+
```

无效经度（超出范围）

```
mysql> SELECT ST_Point(200, 50);
+-------------------+
| ST_Point(200, 50) |
+-------------------+
| NULL              |
+-------------------+
```

无效纬度（超出范围）

```
mysql> SELECT ST_Point(116, -100);
+---------------------+
```

```
| ST_Point(116, -100) |
+---------------------+
| NULL                |
+---------------------+
```

任一参数为 NULL

```
mysql> SELECT ST_Point(NULL, 50);
+--------------------+
| ST_Point(NULL, 50) |
+--------------------+
| NULL               |
+--------------------+
```

```
mysql> SELECT ST_Point(50, NULL);
+--------------------+
| ST_Point(50, NULL) |
+--------------------+
| NULL               |
+--------------------+
```

7.2.1.4.17 ST_POLYGON

描述

将 WKT（Well-Known Text）格式的字符串转换为内存中的多边形（Polygon）几何对象

别名

- st_polygonfromtext
- st_polyfromtext

语法

```
ST_POLYGON( <wkt>)
```

参数

参数	说明
<wkt>	由 POLYGON 函数生成的一个多边形的 wkt 文本

返回值

返回 Polygon 类型的几何对象，该对象在内存中以 Doris 内部的空间数据格式存储，可直接作为参数传入其他空间函数（如 ST_AREA、ST_CONTAINS 等）进行计算

- 若输入的 WKT 字符串格式无效（如环未闭合、语法错误），返回 NULL。

- 若输入为 NULL 或空字符串，返回 NULL。

举例

基本多边形

```
SELECT ST_AsText(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"));
```

```
+-----------------------------------------------------------------+
| st_astext(st_polygon('POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))')) |
+-----------------------------------------------------------------+
| POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))                         |
+-----------------------------------------------------------------+
```

自相交多边形 (无效)

```
mysql> select st_polygon('POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))');
+---------------------------------------------------+
| st_polygon('POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))') |
+---------------------------------------------------+
| NULL                                              |
+---------------------------------------------------+
```

无效 WKT (环未闭合)

```
mysql> SELECT ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10))");
+---------------------------------------------------+
| ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10))") |
+---------------------------------------------------+
| NULL                                              |
+---------------------------------------------------+
```

无效 WKT (语法错误)

```

mysql> SELECT ST_Polygon("POLYGON (0 0, 10 0, 10 10, 0 10, 0 0)");
+---------------------------------------------------+
| ST_Polygon("POLYGON (0 0, 10 0, 10 10, 0 10, 0 0)") |
+---------------------------------------------------+
| NULL                                              |
+---------------------------------------------------+
```

输入 NULL

```
mysql> SELECT ST_Polygon(NULL);
+------------------+
| ST_Polygon(NULL) |
+------------------+
```

```
| NULL              |
+-------------------+
```

7.2.1.4.18 ST_TOUCHES

描述

判断两个几何图形是否仅边界接触（即边界存在公共点，但内部无任何交集）。具体来说：

若两个图形的边界有至少一个公共点，且内部完全不相交（无重叠区域），则返回 1；若边界无公共点，或内部存在交集（即使边界也接触），则返回 0。

该函数用于区分“仅边界接触”与“内部相交”两种场景，例如相邻多边形的共享边（仅边界接触）、线与多边形的切点（仅边界接触）等

语法

```
ST_TOUCHES( <shape1>, <shape2>)
```

参数

参数	说明
<shape1>	用于判断是否接触的第一个几何图形，支持 Point、LineString、Polygon、Circle 等类型。
<shape2>	用于判断是否接触的第二个几何图形，支持 Point、LineString、Polygon、Circle 等类型

返回值

返回 1: shape1 图形与图形 shape2 相接触

返回 0: shape1 图形与图形 shape2 不接触

边缘情况 ST_TOUCHES 存在以下边缘情况：

- 若任一输入参数为 NULL，返回 NULL。
- 若输入的几何图形无效，返回 NULL。
- 若输入为空几何图形（如 POINT EMPTY），NULL。
- 若一个图形完全包含另一个图形（内部有交集），即使边界接触，返回 0。
- 点与图形的边界接触时（点无内部），返回 1（点的“边界”即自身，与图形边界重合且无内部交集）。

举例

点在多边形界上

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0, 5)
    ↪ );
+-----------------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(0, 5)) |
+-----------------------------------------------------------------------------------------+
|                                                                                       1 |
+-----------------------------------------------------------------------------------------+
```

点不在多边形边界上 (在几何图形内部)

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(5, 5)
    ↪ );
+-----------------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Point(5, 5)) |
+-----------------------------------------------------------------------------------------+
|                                                                                       0 |
+-----------------------------------------------------------------------------------------+
```

线与线端点接触

```
mysql> SELECT ST_TOUCHES(  ST_Linefromtext("LINESTRING (0 0, 2 2)"),  ST_Linefromtext("LINESTRING
    ↪  (2 2, 4 0)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| ST_TOUCHES(  ST_Linefromtext("LINESTRING (0 0, 2 2)"),  ST_Linefromtext("LINESTRING (2 2, 4 0)"
    ↪ )) |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
```

线与线内部相交 (不是边界接触，返回 0)

```
mysql> SELECT ST_TOUCHES(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("
    ↪ LINESTRING (0 10, 10 0)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
| ST_TOUCHES(  ST_Linefromtext("LINESTRING (0 0, 10 10)"),  ST_Linefromtext("LINESTRING (0 10, 10
    ↪  0)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------+
    ↪
```

线与多边形边界相切

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (10 5, 15 5)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (10 5, 15 5)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

线穿过多边形内部（内部相交，返回 0）

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ Linefromtext("LINESTRING (5 5, 15 5)"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Linefromtext("
    ↪ LINESTRING (5 5, 15 5)")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形共享边界（仅边界接触，返回 1）

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((5 0, 10 0, 10 5, 5 5, 5 0))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
```

```
    ↪
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 5 0, 5 5, 0 5, 0 0))"),  ST_Polygon("POLYGON ((5 0, 10
    ↪ 0, 10 5, 5 5, 5 0))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

多边形与多边形部分重叠（内部相交，返回 0）

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("
    ↪ POLYGON ((5 5, 15 5, 15 15, 5 15, 5 5))"));
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Polygon("POLYGON ((5 5,
    ↪  15 5, 15 15, 5 15, 5 5))")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

空图形与多边形（不接触，返回 0）

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_
    ↪ GeometryFromText("POINT EMPTY"))
    -> ;
+--
    ↪ ----------------------------------------------------------------------------------------------------------------+
    ↪
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_GeometryFromText("POINT
    ↪  EMPTY")) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
```

```
    ↪ NULL |
+--
    ↪ ----------------------------------------------------------------------------------------------------------+
    ↪
```

无效自相交多边形（返回 NULL）

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),ST_Point(0.5, 0.5));
+-----------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 1 1, 0 1, 1 0, 0 0))"),ST_Point(0.5, 0.5)) |
+-----------------------------------------------------------------------------------+
|                                                                              NULL |
+-----------------------------------------------------------------------------------+
```

参数为 NULL（返回 NULL）

```
mysql> SELECT ST_TOUCHES(NULL, ST_Point(5, 5));
+----------------------------------+
| ST_TOUCHES(NULL, ST_Point(5, 5)) |
+----------------------------------+
|                             NULL |
+----------------------------------+
```

```
mysql> SELECT ST_TOUCHES(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), NULL);
+-------------------------------------------------------------------------+
| ST_TOUCHES(ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"), NULL) |
+-------------------------------------------------------------------------+
|                                                                    NULL |
+-------------------------------------------------------------------------+
```

圆与线相切

```
mysql> SELECT ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Linefromtext("LINESTRING (-10 5, 10 5)"));
+-----------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Linefromtext("LINESTRING (-10 5, 10 5)")) |
+-----------------------------------------------------------------------------------+
|                                                                                 1 |
+-----------------------------------------------------------------------------------+
```

圆与点边界相交

```
mysql> SELECT ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Point(5, 0));
+-----------------------------------------------------+
| ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Point(5, 0)) |
+-----------------------------------------------------+
```

```
|                                                  1 |
+----------------------------------------------------+
```

两圆相切

```
mysql> SELECT ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Circle(10, 0, 5));
+-----------------------------------------------------------+
| ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Circle(10, 0, 5)) |
+-----------------------------------------------------------+
|                                                         1 |
+-----------------------------------------------------------+
```

点在圆内部不相切

```
mysql> SELECT ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Point(4, 0));
+-----------------------------------------------------+
| ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Point(4, 0)) |
+-----------------------------------------------------+
|                                                   0 |
+-----------------------------------------------------+
```

线与圆相交，返回 0

```
mysql> SELECT ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Linefromtext("LINESTRING (-10 0, 10 0)"));
+-------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Circle(0, 0, 5),  ST_Linefromtext("LINESTRING (-10 0, 10 0)")) |
+-------------------------------------------------------------------------------+
|                                                                             0 |
+-------------------------------------------------------------------------------+
```

圆与多边形相切

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(15,
    ↪ 5, 5));
+-----------------------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(15, 5, 5)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             1 |
+-----------------------------------------------------------------------------------------------+
```

圆与多边形相交，返回 0

```
mysql> SELECT ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(8,
    ↪ 5, 3));
+-----------------------------------------------------------------------------------------------+
| ST_TOUCHES(  ST_Polygon("POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))"),  ST_Circle(8, 5, 3)) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             0 |
```

```
+-----------------------------------------------------------------------------------------+
```

7.2.1.4.19 ST_X

描述

当 point 是一个合法的 POINT 类型时，返回对应的 X 坐标值

语法

```
ST_X( <point>)
```

参数

参数	说明
<point>	待提取 X 坐标的几何对象，必须是有效的 POINT 类型（二维点），x 纬度范围为 [-180,180],y 纬度范围为 [-90,90]

返回值

几何坐标中的 X 值, 类型为双浮点精度小数 Double

- 若输入为有效的 POINT 对象，返回该点的 X 坐标（双精度浮点数 Double）。
- 若输入为 NULL、非 POINT 类型对象、空点（POINT EMPTY）或无效点（如三维点），返回 NULL。

举例

提取有效点的 X 坐标

```
SELECT ST_X(ST_Point(24.7, 56.7));
```

```
+----------------------------+
| st_x(st_point(24.7, 56.7)) |
+----------------------------+
|                       24.7 |
+----------------------------+
```

输入为空点（POINT EMPTY）

```
mysql> SELECT ST_X(ST_GeometryFromText("POINT EMPTY"));
+-----------------------------------------+
| ST_X(ST_GeometryFromText("POINT EMPTY")) |
+-----------------------------------------+
|                                    NULL |
+-----------------------------------------+
```

输入为三维点（不支持）

```
mysql> SELECT ST_X(ST_GeometryFromText("POINT (10 20 30)"));
+-----------------------------------------------+
| ST_X(ST_GeometryFromText("POINT (10 20 30)")) |
+-----------------------------------------------+
|                                          NULL |
+-----------------------------------------------+
```

输入为 NULL

```
mysql> SELECT ST_X(NULL);
+------------+
| ST_X(NULL) |
+------------+
|       NULL |
+------------+
```

纬度超出范围

```
mysql> SELECT ST_X(ST_Point(244.7, 56.7));
+-----------------------------+
| ST_X(ST_Point(244.7, 56.7)) |
+-----------------------------+
|                        NULL |
+-----------------------------+
```

经度超出范围

```
mysql> SELECT ST_X(ST_Point(44.7, 156.7));
+-----------------------------+
| ST_X(ST_Point(44.7, 156.7)) |
+-----------------------------+
|                        NULL |
+-----------------------------+
```

7.2.1.4.20 ST_Y

描述

当 point 是一个合法的 POINT 类型时，返回对应的 Y 坐标值。在地理空间场景中，Y 坐标通常对应纬度（Latitude），取值范围为 [-90.0, 90.0]（单位：度）

语法

```
ST_Y( <point>)
```

参数

参数	说明
<point>	待提取 Y 坐标的几何对象，必须是有效的 POINT 类型（二维点）。其中，Y（纬度）范围为 [-90.0, 90.0]，X（经度）范围为 [-180.0, 180.0]。

返回值

几何坐标中的 Y 值，类型为双精度浮点数（Double）。

若输入为有效的 POINT 对象，返回该点的 Y 坐标（纬度）。若输入为 NULL、空点（POINT EMPTY）、三维点或无效点（如纬度超出范围），返回 NULL。

举例

正常坐标

```
SELECT ST_Y(ST_Point(24.7, 56.7));
```

```
+----------------------------+
| ST_Y(ST_Point(24.7, 56.7)) |
+----------------------------+
| 56.7                       |
+----------------------------+
```

输入为空点（POINT EMPTY

```
mysql> SELECT ST_Y(ST_GeometryFromText("POINT EMPTY"));
+-----------------------------------------+
| ST_Y(ST_GeometryFromText("POINT EMPTY")) |
+-----------------------------------------+
|                                    NULL |
+-----------------------------------------+
```

输入为三维点（不支持）

```
mysql> SELECT ST_Y(ST_GeometryFromText("POINT (10 20 30)"));
+-----------------------------------------------+
| ST_Y(ST_GeometryFromText("POINT (10 20 30)")) |
+-----------------------------------------------+
|                                          NULL |
+-----------------------------------------------+
```

输入为 NULL

```
mysql> SELECT ST_Y(NULL);
+------------+
| ST_Y(NULL) |
+------------+
|       NULL |
+------------+
```

纬度超出范围（无效点）

```
mysql> SELECT ST_Y(ST_Point(116.4, 91));
+---------------------------+
| ST_Y(ST_Point(116.4, 91)) |
+---------------------------+
|                      NULL |
+---------------------------+
```

经度超出范围（无效点）

```
mysql> SELECT ST_Y(ST_Point(190, 39.9));
+---------------------------+
| ST_Y(ST_Point(190, 39.9)) |
+---------------------------+
|                      NULL |
+---------------------------+
```

7.2.1.5 加密和信息摘要函数

7.2.1.5.1 AES_DECRYPT

描述

AES 解密函数。该函数与 MySQL 中的 AES_DECRYPT 函数行为一致。默认采用 AES_128_ECB 算法，padding 模式为 PKCS7。

> 截止 3.0.2，两参数版本，会无视 session variable `block_encryption_mode`，始终使用 AES_128_ECB 算法进行解密。因此不推荐调用。
>
> 3.0.3 起，该行为恢复正常。

语法

```
AES_DECRYPT( <str>, <key_str>[, <init_vector>][, <encryption_mode>])
```

参数

参数	说明
<str>	为待解密文本

参数	说明
<key_str>	为密钥。注意此密钥并非16进制编码，而是编码后的字符串表示。例如对于128位密钥加密，key_str长度应为16。如果密

参数	说明
<init_ ↪ vector>	为算法中使用到的初始向量，仅在特定算法下生效，如不指定，则 Doris 使用内置向量

参数	说明
<encryption ↪ _mode>	为加密算法，可选值见于变量

返回值

如果解密成功：返回解密后的数据，通常是明文的二进制表示。

如果解密失败：返回 NULL。

示例

解密成功

使用默认算法

```
set block_encryption_mode='';
select aes_decrypt(from_base64('wr2JEDVXzL9+2XtRhgIloA=='),'F3229A0B371ED2D9441B830D21A390C3');
```

```
+--------------------------------------------------------------------------------+
| aes_decrypt(from_base64('wr2JEDVXzL9+2XtRhgIloA=='), '***', '', 'AES_128_ECB') |
+--------------------------------------------------------------------------------+
| text                                                                           |
+--------------------------------------------------------------------------------+
```

使用 AES_256_CBC 算法

```
set block_encryption_mode="AES_256_CBC";
select aes_decrypt(from_base64('3dym0E7/+1zbrLIaBVNHSw=='),'F3229A0B371ED2D9441B830D21A390C3');
```

```
+--------------------------------------------------------------------------------+
| aes_decrypt(from_base64('3dym0E7/+1zbrLIaBVNHSw=='), '***', '', 'AES_256_CBC') |
+--------------------------------------------------------------------------------+
| text                                                                           |
+--------------------------------------------------------------------------------+
```

使用 AES_256_CBC 算法并设置初始向量

```
select AES_DECRYPT(FROM_BASE64('tsmK1HzbpnEdR2//WhO+MA=='),'F3229A0B371ED2D9441B830D21A390C3', '
    ↪ 0123456789');
```

```
+-----------------------------------------------------------------------------------------+
| aes_decrypt(from_base64('tsmK1HzbpnEdR2//WhO+MA=='), '***', '0123456789', 'AES_256_CBC') |
+-----------------------------------------------------------------------------------------+
| text                                                                                    |
+-----------------------------------------------------------------------------------------+
```

解密失败

```
select AES_DECRYPT(FROM_BASE64('tsmK1H3zbpnEdR2//WhO+MA=='),'F3229A0B371ED2D9441B830D21A390C3', '
    ↪ 0123456789');
```

```
+-----------------------------------------------------------------------------------------+
| aes_decrypt(from_base64('tsmK1H3zbpnEdR2//WhO+MA=='), '***', '0123456789', 'AES_256_CBC') |
+-----------------------------------------------------------------------------------------+
| NULL                                                                                    |
+-----------------------------------------------------------------------------------------+
```

7.2.1.5.2 AES_ENCRYPT

描述

AES 加密函数。该函数与 MySQL 中的 AES_ENCRYPT 函数行为一致。默认采用 AES_128_ECB 算法，padding 模式为 PKCS7。

AES_ENCRYPT 函数对于传入的密钥，并不是直接使用，而是会进一步做处理，具体步骤如下：

1. 根据使用的加密算法，确定密钥的字节数，比如使用 AES_128_ECB 算法，则密钥字节数为 `128 / 8 = 16`（如果使用 AES_256_ECB 算法，则密钥字节数为 `256 / 8 = 32`）；
2. 然后针对用户输入的密钥，第 `i` 位和第 `16*k+i` 位进行异或，如果用户输入的密钥不足 16 位，则后面补 0；
3. 最后，再使用新生成的密钥进行加密；

> 截止 3.0.2，两参数版本，会无视 session variable `block_encryption_mode`，始终使用 AES_128_ECB 算法进行加密。因此不推荐调用。
>
> 3.0.3 起，该行为恢复正常。

语法

```
AES_ENCRYPT( <str>, <key_str>[, <init_vector>][, <encryption_mode>])
```

参数

参数	说明
<str>	为待加密文本

参数	说明
<key_str>	为密钥。注意此密钥并非16进制编码，而是编码后的字符串表示。例如对于128位密钥加密，key_str长度应为16。如果密

参数	说明
<init_ ↪ vector>	为算法中使用到的初始向量，仅在特定算法下生效，如不指定，则 Doris 使用内置向量

参数	说明
<encryption ↪ _mode>	为加密算法，可选值见于变量

返回值

返回二进制的加密后的数据

示例

使用 AES_128_ECB 算法

```
set block_encryption_mode='';
select to_base64(aes_encrypt('text','F3229A0B371ED2D9441B830D21A390C3'));
```

```
+-------------------------------------------------------------+
| to_base64(aes_encrypt('text', '***', '', 'AES_128_ECB')) |
+-------------------------------------------------------------+
| wr2JEDVXzL9+2XtRhgIloA==                                    |
+-------------------------------------------------------------+
```

使用 AES_256_CBC 算法

```
set block_encryption_mode="AES_256_CBC";
select to_base64(aes_encrypt('text','F3229A0B371ED2D9441B830D21A390C3'));
```

```
+-------------------------------------------------------------+
| to_base64(aes_encrypt('text', '***', '', 'AES_256_CBC')) |
+-------------------------------------------------------------+
| 3dymOE7/+1zbrLIaBVNHSw==                                    |
+-------------------------------------------------------------+
```

使用 AES_256_CBC 算法并设置初始向量

```
select to_base64(aes_encrypt('text','F3229A0B371ED2D9441B830D21A390C3', '0123456789'));
```

```
+------------------------------------------------------------------+
| to_base64(aes_encrypt('text', '***', '0123456789', 'AES_256_CBC')) |
+------------------------------------------------------------------+
| tsmK1HzbpnEdR2//WhO+MA==                                         |
+------------------------------------------------------------------+
```

7.2.1.5.3 CRC32

描述

使用 CRC32 算法计算结果。

语法

```
CRC32( <str> )
```

参数

参数	说明
<str>	需要被计算 CRC 的值

返回值

返回字符串的 CRC 值。

示例

```
select crc32("abc"),crc32("中国");
```

```
+--------------+-----------------+
| crc32('abc') | crc32('中国')   |
+--------------+-----------------+
|    891568578 |       737014929 |
+--------------+-----------------+
```

7.2.1.5.4 MD5

描述

计算 MD5 128-bit

语法

```
MD5( <str> )
```

参数

参数	说明
<str>	需要被计算 MD5 的值

返回值

返回字符串的 MD5 值。

示例

```
select md5("abc");
```

```
+----------------------------------+
| md5('abc')                       |
+----------------------------------+
| 900150983cd24fb0d6963f7d28e17f72 |
+----------------------------------+
```

7.2.1.5.5 MD5SUM

描述

计算多个字符串 MD5 128-bit

语法

```
MD5SUM( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 MD5 的值

返回值

返回多个字符串的 MD5 值。

示例

```
select md5("abcd"),md5sum("ab","cd");
```

```
+----------------------------------+----------------------------------+
| md5('abcd')                      | md5sum('ab', 'cd')               |
+----------------------------------+----------------------------------+
| e2fc714c4727ee9395f324cd2e7f331f | e2fc714c4727ee9395f324cd2e7f331f |
+----------------------------------+----------------------------------+
```

7.2.1.5.6 MURMUR_HASH3_32

描述

计算 32 位 murmur3 hash 值

-注：在计算 hash 值时，更推荐使用xxhash_32，而不是murmur_hash3_32。

语法

```
MURMUR_HASH3_32( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 32 位 murmur3 hash 的值

返回值

返回输入字符串的 32 位 murmur3 hash 值。

-当参数为 NULL 时，返回 NULL

示例

```
select murmur_hash3_32(null), murmur_hash3_32("hello"), murmur_hash3_32("hello", "world");
```

```
+----------------------+--------------------------+----------------------------------+
| murmur_hash3_32(NULL) | murmur_hash3_32('hello') | murmur_hash3_32('hello', 'world') |
+----------------------+--------------------------+----------------------------------+
|                 NULL |               1321743225 |                        984713481 |
+----------------------+--------------------------+----------------------------------+
```

7.2.1.5.7 MURMUR_HASH3_64

描述

计算 64 位 murmur3 hash 值

与MURMUR_HASH3_64_V2的区别是：此版本专门为 64 位输出优化，性能略优于 v2 版本, 但与标准库实现不一致。

-注：经过测试 xxhash_64 的性能大约是 murmur_hash3_64 的 2 倍，所以在计算 hash 值时，更推荐使用xxhash_64，而不是murmur_hash3_64。

语法

```
MURMUR_HASH3_64( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 64 位 murmur3 hash 的值

返回值

返回输入字符串的 64 位 murmur3 hash 值。

任一参数输入为 NULL 时返回 NULL。

示例

```
select murmur_hash3_64(null), murmur_hash3_64("hello"), murmur_hash3_64("hello", "world");
```

```
+----------------------+-------------------------+----------------------------------+
| murmur_hash3_64(NULL) | murmur_hash3_64('hello') | murmur_hash3_64('hello', 'world') |
+----------------------+-------------------------+----------------------------------+
|                  NULL |     -3215607508166160593 |               3583109472027628045 |
+----------------------+-------------------------+----------------------------------+
```

7.2.1.5.8 MURMUR_HASH3_64_V2

描述

计算 64 位 murmur3 hash 值

与MURMUR_HASH3_64的区别是：此版本复用 MurmurHash3 的 128 位处理函数，仅输出第一个 64 位哈希值，与标准库的行为保持一致。

-注：经过测试 xxhash_64 的性能大约是 murmur_hash3_64_v2 的 2 倍，所以在计算 hash 值时，更推荐使用xxhash_64，而不是murmur_hash3_64。

语法

```
MURMUR_HASH3_64_V2( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 64 位 murmur3 hash 的值

返回值

返回输入字符串的 64 位 murmur3 hash 值。

任一参数为 NULL 时返回 NULL

示例

```
select murmur_hash3_64_v2(null), murmur_hash3_64_v2("hello"), murmur_hash3_64_v2("hello", "world"
    ↪ );
```

```
+----------------------+-------------------------+----------------------------------+
| murmur_hash3_64(NULL) | murmur_hash3_64('hello') | murmur_hash3_64('hello', 'world') |
+----------------------+-------------------------+----------------------------------+
```

```
|                  NULL |     -3215607508166160593 |               3583109472027628045 |
+-----------------------+--------------------------+-----------------------------------+
```

7.2.1.5.9 SHA1

描述

使用 SHA1 算法对信息进行摘要处理。

别名

SHA

语法

```
SHA1( <str> )
```

参数

参数	说明
<str>	需要被计算 sha1 的值

返回值

返回输入字符串的 sha1 值

示例

```
select sha("123"), sha1("123");
```

```
+------------------------------------------+------------------------------------------+
| sha1('123')                              | sha1('123')                              |
+------------------------------------------+------------------------------------------+
| 40bd001563085fc35165329ea1ff5c5ecbdbbeef | 40bd001563085fc35165329ea1ff5c5ecbdbbeef |
+------------------------------------------+------------------------------------------+
```

7.2.1.5.10 SHA2

描述

使用 SHA2 对信息进行摘要处理。

语法

```
SHA2( <str>, <digest_length>)
```

参数

参数	说明
<str>	待加密的内容
<digest_length>	摘要长度，支持 224, 256, 384, 512

返回值

返回输入字符串的 sha2 值

示例

```
select sha2('abc', 224), sha2('abc', 384), sha2(NULL, 512);
```

```
+----------------------------------------------------------+----------------------------------------------------------
    ↪
| sha2('abc', 224)                                         | sha2('abc', 384)
    ↪                                                                                                    | sha2(
    ↪ NULL, 512) |
+----------------------------------------------------------+----------------------------------------------------------
    ↪
| 23097d223405d8228642a477bda255b32aadbce4bda0b3f7e36c9da7 |
    ↪ cb00753f45a35e8bb5a03d699ac65007272c32ab0eded1631a8b605a43ff5bed8086072ba1e7cc2358baeca134c825a7
    ↪  | NULL             |
+----------------------------------------------------------+----------------------------------------------------------
    ↪
```

7.2.1.5.11 SM3

描述

计算 SM3 256-bit

语法

```
SM3( <str> )
```

参数

参数	说明
<str>	需要被计算 sm3 的值

返回值

返回输入字符串的 sm3 值

示例

```
select sm3("abcd");
```

```
+------------------------------------------------------------------+
| sm3('abcd')                                                      |
+------------------------------------------------------------------+
| 82ec580fe6d36ae4f81cae3c73f4a5b3b5a09c943172dc9053c69fd8e18dca1e |
+------------------------------------------------------------------+
```

7.2.1.5.12 SM3SUM

描述

计算多个字符串 SM3 256-bit

语法

```
SM3SUM( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 sm3 的值

返回值

返回输入多个字符串的 sm3 值

示例

```
select sm3sum("ab","cd");
```

```
+------------------------------------------------------------------+
| sm3sum('ab', 'cd')                                               |
+------------------------------------------------------------------+
| 82ec580fe6d36ae4f81cae3c73f4a5b3b5a09c943172dc9053c69fd8e18dca1e |
+------------------------------------------------------------------+
```

7.2.1.5.13 SM4_DECRYPT

描述

SM4 是一种国家标准的对称密钥加密算法，广泛应用于金融、通信、电子商务等领域。SM4_DECRYPT 函数用于对数据进行 SM4 解密。默认采用 SM4_128_ECB 算法。

> 截止 3.0.2，两参数版本，会无视 session variable `block_encryption_mode`，始终使用 SM4_128_ECB 算法进行解密。因此不推荐调用。
>
> 3.0.3 起，该行为恢复正常。

语法

```
SM4_DECRYPT( <str>, <key_str>[, <init_vector>][, <encryption_mode>])
```

参数

参数	说明
<str>	为待解密文本

参数	说明
<key_str>	为密钥。注意此密钥并非16进制编码，而是编码后的字符串表示。例如对于128位密钥加密，key_str长度应为16。如果密

参数	说明
<init_ ↪ vector>	为算法中使用到的初始向量，仅在特定算法下生效，如不指定，则 Doris 使用内置向量

参数	说明
<encryption ↪ _mode>	为加密算法，可选值见于变量

返回值

如果解密成功：返回解密后的数据，通常是明文的二进制表示。

如果解密失败：返回 NULL。

示例

解密成功

使用默认算法

```
set block_encryption_mode='';
select SM4_DECRYPT(FROM_BASE64('aDjwRflBrDjhBZIOFNw3Tg=='),'F3229A0B371ED2D9441B830D21A390C3');
```

```
+-----------------------------------------------------------------------------+
| sm4_decrypt(from_base64('aDjwRflBrDjhBZIOFNw3Tg=='), '***', '', 'SM4_128_ECB') |
+-----------------------------------------------------------------------------+
| text                                                                        |
+-----------------------------------------------------------------------------+
```

使用 SM4_128_CBC 算法

```
set block_encryption_mode="SM4_128_CBC";
select SM4_DECRYPT(FROM_BASE64('FSYstvOmH2cXy7B/072Mug=='),'F3229A0B371ED2D9441B830D21A390C3');
```

```
+-----------------------------------------------------------------------------+
| sm4_decrypt(from_base64('FSYstvOmH2cXy7B/072Mug=='), '***', '', 'SM4_128_CBC') |
+-----------------------------------------------------------------------------+
| text                                                                        |
+-----------------------------------------------------------------------------+
```

使用 SM4_128_CBC 算法并初始向量

```
select SM4_DECRYPT(FROM_BASE64('1Y4NGIukSbv9OrkZnRD1bQ=='),'F3229A0B371ED2D9441B830D21A390C3', '
    ↪ 0123456789');
```

```
+-----------------------------------------------------------------------------------------+
| sm4_decrypt(from_base64('1Y4NGIukSbv9OrkZnRD1bQ=='), '***', '0123456789', 'SM4_128_CBC') |
+-----------------------------------------------------------------------------------------+
| text                                                                                    |
+-----------------------------------------------------------------------------------------+
```

解密失败

```
set block_encryption_mode='';
select SM4_DECRYPT(FROM_BASE64('aDjwRflBrDjhBZIdOFNw3Tg=='),'F3229A0B371ED2D9441B830D21A390C3');
```

```
+-------------------------------------------------------------------------------+
| sm4_decrypt(from_base64('aDjwRflBrDjhBZIdOFNw3Tg=='), '***', '', 'SM4_128_ECB') |
+-------------------------------------------------------------------------------+
| NULL                                                                          |
+-------------------------------------------------------------------------------+
```

7.2.1.5.14 SM4_ENCRYPT

描述

SM4 是一种国家标准的对称密钥加密算法，广泛应用于金融、通信、电子商务等领域。SM4_ENCRYPT 函数用于对数据进行 SM4 加密。默认采用 SM4_128_ECB 算法。

> 截止 3.0.2，两参数版本，会无视 session variable block_encryption_mode，始终使用 SM4_128_ECB 算法进行加密。因此不推荐调用。
>
> 3.0.3 起，该行为恢复正常。

语法

```
SM4_ENCRYPT( <str>, <key_str>[, <init_vector>][, <encryption_mode>])
```

参数

参数	说明
<str>	为待加密文本

参数	说明
<key_str>	为密钥。注意此密钥并非16进制编码，而是编码后的字符串表示。例如对于128位密钥加密，key_str长度应为16。如果密

参数	说明
<init_ ↪ vector>	为算法中使用到的初始向量，仅在特定算法下生效，如不指定，则 Doris 使用内置向量

参数	说明
<encryption ↪ _mode>	为加密算法，可选值见于变量

返回值

返回二进制的加密后的数据

示例

使用默认算法

```
set block_encryption_mode='';
select TO_BASE64(SM4_ENCRYPT('text','F3229A0B371ED2D9441B830D21A390C3'));
```

```
+-----------------------------------------------------------+
| to_base64(sm4_encrypt('text', '***', '', 'SM4_128_ECB')) |
+-----------------------------------------------------------+
| aDjwRflBrDjhBZIOFNw3Tg==                                  |
+-----------------------------------------------------------+
```

使用 SM4_128_CBC 算法

```
set block_encryption_mode="SM4_128_CBC";
select TO_BASE64(SM4_ENCRYPT('text','F3229A0B371ED2D9441B830D21A390C3'));
```

```
+-----------------------------------------------------------+
| to_base64(sm4_encrypt('text', '***', '', 'SM4_128_CBC')) |
+-----------------------------------------------------------+
| FSYstvOmH2cXy7B/072Mug==                                  |
+-----------------------------------------------------------+
```

使用 SM4_128_CBC 算法并设置初始向量

```
set block_encryption_mode="SM4_128_CBC";
select to_base64(SM4_ENCRYPT('text','F3229A0B371ED2D9441B830D21A390C3', '0123456789'));
```

```
+------------------------------------------------------------------+
| to_base64(sm4_encrypt('text', '***', '0123456789', 'SM4_128_CBC')) |
+------------------------------------------------------------------+
| 1Y4NGIukSbv9OrkZnRD1bQ==                                         |
+------------------------------------------------------------------+
```

7.2.1.5.15 XXHASH_32

描述

计算输入字符串的 32 位 xxhash 值

-注：在计算 hash 值时，更推荐使用xxhash_32，而不是murmur_hash3_32。

语法

```
XXHASH_32( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 32 位 xxhash 的值

返回值

返回输入字符串的 32 位 xxhash 值。

举例

```
select xxhash_32(NULL), xxhash_32("hello"), xxhash_32("hello", "world");
```

```
+-----------------+--------------------+-----------------------------+
| xxhash_32(NULL) | xxhash_32('hello') | xxhash_32('hello', 'world') |
+-----------------+--------------------+-----------------------------+
|            NULL |          -83855367 |                   -920844969 |
+-----------------+--------------------+-----------------------------+
```

7.2.1.5.16 XXHASH_64

描述

计算输入字符串的 64 位 xxhash 值

-注：经过测试 xxhash_64 的性能大约是 murmur_hash3_64 的 2 倍，所以在计算 hash 值时，更推荐使用xxhash_64，而不是murmur_hash3_64。

别名

- XXHASH3_64

语法

```
XXHASH_64( <str> [ , <str> ... ] )
```

参数

参数	说明
<str>	需要被计算 64 位 xxhash 的值

返回值

返回输入字符串的 64 位 xxhash 值。

举例

```
select xxhash_64(NULL), xxhash_64("hello"), xxhash_64("hello", "world");
```

```
+-----------------+----------------------+-----------------------------+
| xxhash_64(NULL) | xxhash_64('hello')   | xxhash_64('hello', 'world') |
+-----------------+----------------------+-----------------------------+
|            NULL | -7685981735718036227 |          7001965798170371843 |
+-----------------+----------------------+-----------------------------+
```

7.2.1.6 向量函数

7.2.1.6.1 COSINE_DISTANCE

描述

计算两个向量（向量值为坐标）之间的余弦距离

语法

```
COSINE_DISTANCE(<array1>, <array2>)
```

参数

参数	说明

参数	说明
< ↪ array1 ↪ > ↪	第一个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array2保持一致

参数	说明
< ↪ array2 ↪ > ↪	第二个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array1保持一致

参数	说明

返回值

返回两个向量（向量值为坐标）之间的余弦距离。如果输入 array 为 NULL，或者 array 中任何元素为 NULL，则返回 NULL。

举例

```
SELECT COSINE_DISTANCE([1, 2], [2, 3]),COSINE_DISTANCE([3, 6], [4, 7]);
```

```
+---------------------------------+---------------------------------+
| cosine_distance([1, 2], [2, 3]) | cosine_distance([3, 6], [4, 7]) |
+---------------------------------+---------------------------------+
|            0.007722123286332261 |           0.0015396467945875125 |
+---------------------------------+---------------------------------+
```

7.2.1.6.2 INNER_PRODUCT

描述

计算两个大小相同的向量的标量积

语法

```
INNER_PRODUCT(<array1>, <array2>)
```

参数

参数	说明
< ↪ array1 ↪ > ↪	第一个向量，输入数组的子类型支持：TINYINT、SMALL-INT、INT、BIG-INT、LARGEINT、FLOAT、DOU-BLE，元素数量需与 ar-ray2 保持一致

参数	说明
<array2>	第二个向量，输入数组的子类型支持：TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE，元素数量需与array1保持一致

返回值

返回两个大小相同的向量的标量积。如果输入 array 为 NULL，或者 array 中任何元素为 NULL，则返回 NULL。

举例

```
SELECT INNER_PRODUCT([1, 2], [2, 3]),INNER_PRODUCT([3, 6], [4, 7]);
```

```
+-------------------------------+-------------------------------+
| inner_product([1, 2], [2, 3]) | inner_product([3, 6], [4, 7]) |
+-------------------------------+-------------------------------+
|                             8 |                            54 |
+-------------------------------+-------------------------------+
```

7.2.1.6.3 L1_DISTANCE

描述

计算 L1 空间中两点（向量值为坐标）之间的距离

语法

```
L1_DISTANCE(<array1>, <array2>)
```

参数

参数	说明

参数	说明
< ↪ array1 ↪ > ↪	第一个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array2保持一致

参数	说明
< ↪ array2 ↪ > ↪	第二个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array1保持一致

参数	说明

返回值

返回 L1 空间中两点（向量值为坐标）之间的距离。如果输入 array 为 NULL，或者 array 中任何元素为 NULL，则返回 NULL。

举例

```
SELECT L1_DISTANCE([4, 5], [6, 8]),L1_DISTANCE([3, 6], [4, 5]);
```

```
+----------------------------+----------------------------+
| l1_distance([4, 5], [6, 8]) | l1_distance([3, 6], [4, 5]) |
+----------------------------+----------------------------+
|                          5 |                          2 |
+----------------------------+----------------------------+
```

7.2.1.6.4 L2_DISTANCE

描述

计算欧几里得空间中两点（向量值为坐标）之间的距离

语法

```
L2_DISTANCE(<array1>, <array2>)
```

参数

参数	说明

参数	说明
< ↪ array1 ↪ > ↪	第一个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array2保持一致

参数	说明
< ↪ array2 ↪ > ↪	第二个向量(向量值为坐标),输入数组的子类型支持:TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE,元素数量需与array1保持一致

参数	说明

返回值

返回欧几里得空间中两点（向量值为坐标）之间的距离。如果输入 array 为 NULL，或者 array 中任何元素为 NULL，则返回 NULL。

举例

```
SELECT L2_DISTANCE([4, 5], [6, 8]),L2_DISTANCE([3, 6], [4, 5]);
```

```
+-----------------------------+-----------------------------+
| l2_distance([4, 5], [6, 8]) | l2_distance([3, 6], [4, 5]) |
+-----------------------------+-----------------------------+
|           3.605551275463989 |          1.4142135623730951 |
+-----------------------------+-----------------------------+
```

7.2.1.7 位处理函数

7.2.1.7.1 BIT_LENGTH

描述

用于返回一个字符串的二进制表示中位数（即总的二进制位数）。它计算的是字符串的二进制编码所占的位数。

语法

```
BIT_LENGTH( <str>)
```

参数

参数	说明
<str>	需计算的字符串

返回值

返回 <str> 的二进制表示中所占用的位数，包括所有的 0 和 1。

举例

```
select BIT_LENGTH("abc"), BIT_LENGTH("中国"), BIT_LENGTH(123);
```

```
+------------------+----------------------+-----------------------------------------+
| bit_length('abc') | bit_length('中国')   | bit_length(cast(123 as VARCHAR(65533))) |
+------------------+----------------------+-----------------------------------------+
```

```
|                24 |                   48 |                                       24 |
+-------------------+----------------------+------------------------------------------+
```

7.2.1.7.2 BITAND

描述

用于执行按位与（bitwise AND）运算。按位与运算会对两个整数的每一位进行比较，当两个对应的二进制位都为 1 时，结果才为 1，否则为 0。

整数范围：TINYINT、 SMALLINT、 INT、 BIGINT、 LARGEINT

语法

```
BITAND( <lhs>, <rhs>)
```

参数

参数	说明
<lhs>	参与按位与运算的第一个数
<rhs>	参与按位与运算的第二个数

返回值

返回两个整数与运算的结果。

举例

```
select BITAND(3,5), BITAND(4,7);
```

```
+---------+---------+
| (3 & 5) | (4 & 7) |
+---------+---------+
|       1 |       4 |
+---------+---------+
```

7.2.1.7.3 BIT_COUNT

描述

用于返回一个整数值的二进制表示中 1 的位数。这个函数可以用于快速统计某个整数在二进制表示中“活跃”的位数，通常用于分析数据的分布或进行某些位运算

语法

```
BIT_COUNT( <x>)
```

参数

参数	说明
<x>	统计整型 x 的二进制表示中 1 的个数。整型可以是：TINYINT、 SMALLINT、 INT、 BIGINT、 LARGEINT

返回值

返回 <x> 的二进制表示中 1 的个数

举例

```
select BIT_COUNT(8), BIT_COUNT(-1);
```

```
+--------------+---------------+
| bit_count(8) | bit_count(-1) |
+--------------+---------------+
|            1 |             8 |
+--------------+---------------+
```

7.2.1.7.4 BITNOT

描述

用于对整数进行按位取反操作。

整数范围：TINYINT、 SMALLINT、 INT、 BIGINT、 LARGEINT。

语法

```
BITNOT( <x>)
```

参数

参数	说明
<x>	参与运算整数

返回值

返回一个整数取反运算的结果

举例

```
select BITNOT(7), BITNOT(-127);
```

```
+-------+----------+
| (~ 7) | (~ -127) |
+-------+----------+
|    -8 |      126 |
+-------+----------+
```

7.2.1.7.5 BITOR

描述

用于对两个整数进行按位或操作。

整数范围： TINYINT、 SMALLINT、 INT、 BIGINT、 LARGEINT

语法

```
BITOR( <lhs>, <rhs>)
```

参数

参数	说明
<lhs>	参与运算的第一个数
<rhs>	参与运算的第二个数

返回值

返回两个整数或运算的结果。

举例

```
select BITOR(3,5), BITOR(4,7);
```

```
+---------+---------+
| (3 | 5) | (4 | 7) |
+---------+---------+
|       7 |       7 |
+---------+---------+
```

7.2.1.7.6 BIT_SHIFT_LEFT

描述

用于左移操作的函数，通常用于执行位移操作，将二进制数字的所有位向左移动指定的位数。它是位运算的一种形式，常用于处理二进制数据或进行高效的数学计算。

对于 BIGINT 类型的最大值 9223372036854775807，进行一位左移的结果将得到 -2。###### 语法

```
BIT_SHIFT_LEFT( <x>, <bits>)
```

参数

参数	说明
<x>	需要进行位移的数字
<bits>	需要左移的位数。它是一个整数，决定了 <x> 将被左移多少位

返回值

返回一个整数，表示左移操作后的结果。

举例

```
select BIT_SHIFT_LEFT(5, 2), BIT_SHIFT_LEFT(-5, 2), BIT_SHIFT_LEFT(9223372036854775807, 1);
```

```
+----------------------+----------------------+---------------------------------------+
| bit_shift_left(5, 2) | bit_shift_left(-5, 2) | bit_shift_left(9223372036854775807, 1) |
+----------------------+----------------------+---------------------------------------+
|                   20 |                  -20 |                                    -2 |
+----------------------+----------------------+---------------------------------------+
```

7.2.1.7.7 BIT_SHIFT_RIGHT

描述

用于右移位运算，通常用于将二进制数字的所有位向右移动指定的位数。这种操作通常用于处理二进制数据，或者用于一些数学计算（如除法的高效实现）。

对 -1 逻辑右移一位得到的结果是 BIGINT_MAX(9223372036854775807)。

对数字右移负数为得到对结果始终为 0。

语法

```
BIT_SHIFT_RIGHT( <x>, <bits>)
```

参数

参数	说明
<x>	需要进行位移的数字
<bits>	需要右移的位数。它是一个整数，决定了 <x> 将被右移多少位

返回值

返回一个整数，表示右移操作后的结果。

举例

```
select BIT_SHIFT_RIGHT(1024,3), BIT_SHIFT_RIGHT(-1,1), BIT_SHIFT_RIGHT(100, -1);
```

```
+-------------------------+----------------------+-------------------------+
| bit_shift_right(1024, 3) | bit_shift_right(-1, 1) | bit_shift_right(100, -1) |
+-------------------------+----------------------+-------------------------+
|                     128 |    9223372036854775807 |                       0 |
+-------------------------+----------------------+-------------------------+
```

7.2.1.7.8 XOR

描述

用于对两个 BOOLEAN 值进行按位异或操作。

语法

```
 <lhs> XOR <rhs>
```

参数

参数	说明
<lhs>	参与运算的第一个 BOOLEAN 值
<rhs>	参与运算的第二个 BOOLEAN 值

返回值

返回两个 BOOLEAN 值的异或值。

示例

```
select true XOR false,true XOR true;
```

```
+------------------+-----------------+
| xor(TRUE, FALSE) | xor(TRUE, TRUE) |
+------------------+-----------------+
|                1 |               0 |
+------------------+-----------------+
```

7.2.1.8 ARRAY 函数

7.2.1.8.1 ARRAY

描述

根据参数构造并返回 array, 参数可以是多列或者常量

语法

```
ARRAY([ <element> [, ...] ])
```

参数

参数	说明
<element>	参数可以是多列或者常量

返回值

返回一个数组

举例

```
select array("1", 2, 1.1);
```

```
+-----------------------------------------------+
| array('1', cast(2 as TEXT), cast(1.1 as TEXT)) |
+-----------------------------------------------+
| ["1", "2", "1.1"]                             |
+-----------------------------------------------+
```

```
select array(null, 1);
```

```
+----------------+
| array(NULL, 1) |
+----------------+
| [NULL, 1]      |
+----------------+
```

```
select array(1, 2, 3);
```

```
+----------------+
| array(1, 2, 3) |
+----------------+
| [1, 2, 3]      |
+----------------+
```

```
select array(qid, creationDate, null) from nested  limit 4;
```

```
+-------------------------------------------------------+
| array(cast(qid as DATETIMEV2(0)), creationDate, NULL) |
+-------------------------------------------------------+
| [null, "2009-06-16 07:40:56", null]                   |
| [null, "2009-06-16 07:50:05", null]                   |
| [null, "2009-06-16 08:09:18", null]                   |
| [null, "2009-06-16 08:15:45", null]                   |
+-------------------------------------------------------+
```

7.2.1.8.2 ARRAY_APPLY

描述

数组以特定的二元条件符过滤元素，并返回过滤后的结果

语法

```
ARRAY_APPLY(<arr>, <op>, <val>)
```

参数

参数	说明
<arr>	输入的数组
<op>	过滤条件，条件包括 =, >=, <=, >, <, !=
<val>	过滤的条件值，如果是 null，则返回 null，仅支持常量

返回值

过滤后的数组

举例

```
select array_apply([1, 2, 3, 4, 5], ">=", 2);
```

```
+-------------------------------------------+
| array_apply(ARRAY(1, 2, 3, 4, 5), '>=', 2) |
+-------------------------------------------+
| [2, 3, 4, 5]                              |
+-------------------------------------------+
```

```
select array_apply([1000000, 1000001, 1000002], "=", "1000002");
```

```
+-------------------------------------------------------------+
| array_apply(ARRAY(1000000, 1000001, 1000002), '=', 1000002) |
+-------------------------------------------------------------+
| [1000002]                                                   |
+-------------------------------------------------------------+
```

7.2.1.8.3 ARRAY_AVG

描述

返回数组中所有元素的平均值，数组中的NULL值会被跳过。空数组以及元素全为NULL值的数组，结果返回NULL值。

语法

```
ARRAY_AVG(<arr>)
```

参数

参数	说明
<arr>	用于计算平均值的数组

返回值

返回一个常量，特殊情况：- 数组中的NULL值会被跳过。- 数组的字符串会被跳过

举例

```
create table array_type_table(
    k1 INT,
    k2 Array<int>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
    'replication_num' = '1'
);
insert into array_type_table values (0, []), (1, [NULL]), (2, [1, 2, 3]), (3, [1, NULL, 3]);
select k2, array_avg(k2) from array_type_table;
```

```
+--------------+-----------------+
| k2           | array_avg(`k2`) |
+--------------+-----------------+
| []           |            NULL |
| [NULL]       |            NULL |
| [1, 2, 3]    |               2 |
| [1, NULL, 3] |               2 |
+--------------+-----------------+
```

```
select array_avg(['test',2,1,null]);
```

```
+-----------------------------------------------------------+
| array_avg(cast(['test', '2', '1', NULL] as ARRAY<DOUBLE>)) |
+-----------------------------------------------------------+
|                                                       1.5 |
+-----------------------------------------------------------+
```

7.2.1.8.4 ARRAY_COMPACT

描述

从数组中删除连续地重复元素，结果值的顺序由源数组中的顺序决定。

语法

```
ARRAY_COMPACT(<arr>)
```

参数

参数	说明
<arr>	用于删除其中连续重复元素的数组

返回值

不存在连续重复元素的数组。

举例

```
select array_compact([1, 2, 3, 3, null, null, 4, 4]);
```

```
+-----------------------------------------------------+
| array_compact(ARRAY(1, 2, 3, 3, NULL, NULL, 4, 4)) |
+-----------------------------------------------------+
| [1, 2, 3, NULL, 4]                                  |
+-----------------------------------------------------+
```

```
select array_compact(['aaa','aaa','bbb','ccc','ccccc',null, null,'dddd']);
```

```
+-------------------------------------------------------------------------------+
| array_compact(ARRAY('aaa', 'aaa', 'bbb', 'ccc', 'ccccc', NULL, NULL, 'dddd')) |
+-------------------------------------------------------------------------------+
| ['aaa', 'bbb', 'ccc', 'ccccc', NULL, 'dddd']                                  |
+-------------------------------------------------------------------------------+
```

```
select array_compact(['2015-03-13','2015-03-13']);
```

```
+-------------------------------------------------+
| array_compact(ARRAY('2015-03-13', '2015-03-13')) |
+-------------------------------------------------+
| ['2015-03-13']                                  |
+-------------------------------------------------+
```

7.2.1.8.5 ARRAY_CONCAT

描述

将输入的所有数组拼接为一个数组

语法

```
ARRAY_CONCAT(<arr1> [,<arr2> , ...])
```

参数

参数	说明
<arr1>	源数组
<arr2>	要添加到 arr1 的数组

返回值

拼接好的数组，特殊情况：- 如果数组是 NULL（非NULL），则返回 NULL

举例

```
select array_concat([1, 2], [7, 8], [5, 6]);
```

```
+-----------------------------------------------------+
| array_concat(ARRAY(1, 2), ARRAY(7, 8), ARRAY(5, 6)) |
+-----------------------------------------------------+
| [1, 2, 7, 8, 5, 6]                                  |
+-----------------------------------------------------+
```

```
select array_concat([1, 2], [7, 8], [5, 6], NULL);
```

```
+-------------------------------------------+
| array_concat([1, 2], [7, 8], [5, 6], NULL) |
+-------------------------------------------+
| NULL                                      |
+-------------------------------------------+
```

```
CREATE TABLE array_test (
    id int,
    col2 ARRAY<INT>,
    col3 ARRAY<INT>
)
duplicate key (id)
distributed by hash(id) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_test (id, col2, col3) VALUES
(1,[1, 2, 3], [3, 4, 5]),
(2,[1, NULL, 2], [NULL]),
(3,[1, 2, 3], NULL),
(4,[], []);

select col2, col3, array_concat(col2, col3) from array_test;
```

```
+--------------+-----------+-----------------------------+
| col2         | col3      | array_concat(`col2`, `col3`) |
+--------------+-----------+-----------------------------+
| [1, 2, 3]    | [3, 4, 5] | [1, 2, 3, 3, 4, 5]          |
| [1, NULL, 2] | [NULL]    | [1, NULL, 2, NULL]          |
| [1, 2, 3]    | NULL      | NULL                        |
| []           | []        | []                          |
+--------------+-----------+-----------------------------+
```

7.2.1.8.6 ARRAY_CONTAINS

array_contains

array_contains

描述

语法

```
BOOLEAN array_contains(ARRAY<T> arr, T value)
```

判断数组中是否包含 value。返回结果如下：

```
1    - value在数组arr中存在；
0    - value不存在数组arr中；
NULL - arr为NULL时。
```

举例

```
mysql> SELECT id,c_array,array_contains(c_array, 5) FROM `array_test`;
+------+-----------------+-----------------------------+
| id   | c_array         | array_contains(`c_array`, 5) |
+------+-----------------+-----------------------------+
|    1 | [1, 2, 3, 4, 5] |                           1 |
|    2 | [6, 7, 8]       |                           0 |
|    3 | []              |                           0 |
|    4 | NULL            |                        NULL |
+------+-----------------+-----------------------------+

mysql> select array_contains([null, 1], null);
+--------------------------------------+
| array_contains(ARRAY(NULL, 1), NULL) |
+--------------------------------------+
|                                    1 |
+--------------------------------------+
1 row in set (0.00 sec)
```

keywords

ARRAY,CONTAIN,CONTAINS,ARRAY_CONTAINS

7.2.1.8.7 ARRAY_COUNT

描述

使用 lambda 表达式作为输入参数，对其他输入 ARRAY 参数的内部数据进行相应的表达式计算。返回使得 `lambda(array1[i], ...)` 返回值不为 0 的元素数量。如果找不到到满足此条件的元素，则返回 0。

lambda 表达式中输入的参数为 1 个或多个，必须和后面输入的数组列数一致，且所有输入的 array 的元素个数必须相同。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。

语法

```
ARRAY_COUNT(<arr>),
ARRAY_COUNT(<lambda>, <arr> [ , <arr> ... ] )
```

参数

参数	说明
< ↪ lambda ↪ >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr ↪ >	ARRAY 数组

返回值

经过表达式 lambda 计算之后 ARRAY，不为 0 的元素数量。如果找不到到满足此条件的元素，则返回 0。

举例

```
select array_count(x -> x, [0, 1, 2, 3]);
```

```
+-----------------------------------------------------+
| array_count(array_map([x] -> x(0), ARRAY(0, 1, 2, 3))) |
+-----------------------------------------------------+
```

```
|                                                         3 |
+-----------------------------------------------------------+
```

```
select array_count(x -> x > 2, [0, 1, 2, 3]);
```

```
+-------------------------------------------------------------+
| array_count(array_map([x] -> x(0) > 2, ARRAY(0, 1, 2, 3))) |
+-------------------------------------------------------------+
|                                                           1 |
+-------------------------------------------------------------+
```

```
select array_count(x -> x is null, [null, null, null, 1, 2]);
```

```
+-------------------------------------------------------------------------------+
| array_count(array_map([x] -> x(0) IS NULL, ARRAY(NULL, NULL, NULL, 1, 2))) |
+-------------------------------------------------------------------------------+
|                                                                             3 |
+-------------------------------------------------------------------------------+
```

```
select array_count(x -> power(x,2)>10, [1, 2, 3, 4, 5]);
```

```
+-------------------------------------------------------------------------------+
| array_count(array_map([x] -> power(x(0), 2.0) > 10.0, ARRAY(1, 2, 3, 4, 5))) |
+-------------------------------------------------------------------------------+
|                                                                             2 |
+-------------------------------------------------------------------------------+
```

```
select *, array_count((x, y) -> x>y, c_array1, c_array2) from array_test;
```

```
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
| id   | c_array1        | c_array2                | array_count(array_map([x, y] -> x(0) > y(1),
    ↪  `c_array1`, `c_array2`)) |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] |
    ↪                                                                                 2 |
|    2 | [6, 7, 8]       | [10, 12, 13]            |
    ↪                                                                                 0 |
|    3 | [1]             | [-100]                  |
    ↪                                                                                 1 |
|    4 | [1, NULL, 2]    | [NULL, 3, 1]            |
    ↪                                                                                 1 |
|    5 | []              | []                      |
    ↪                                                                                 0 |
```

```
|    6 | NULL             | NULL                     |
    ↪                                                                                  0 |
+------+------------------+--------------------------+-----------------------------------------------------------------
    ↪
```

7.2.1.8.8 ARRAY_CUM_SUM

描述

返回数组的累计和。数组中的NULL值会被跳过，并在结果数组的相同位置设置NULL。

语法

```
ARRAY_CUM_SUM(<arr>)
```

参数

参数	说明
<arr>	用于计算累加和的数组

返回值

返回一个数组。特殊情况：- 数组中的NULL值会被跳过，并在结果数组的相同位置设置NULL。

举例

```
create table array_type_table(
    k1 INT,
    k2 Array<int>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
    'replication_num' = '1'
);
insert into array_type_table values (0, []),
(1, [NULL]),
(2, [1, 2, 3, 4]),
(3, [1, NULL, 3, NULL, 5]);
select k2, array_cum_sum(k2) from array_type_table;
```

```
+----------------------+-----------------------+
| k2                   | array_cum_sum(`k2`)   |
+----------------------+-----------------------+
| []                   | []                    |
| [NULL]               | [NULL]                |
| [1, 2, 3, 4]         | [1, 3, 6, 10]         |
```

```
| [1, NULL, 3, NULL, 5] | [1, NULL, 4, NULL, 9] |
+-----------------------+-----------------------+
```

7.2.1.8.9 ARRAY_DIFFERENCE

描述

计算相邻数组元素之间的差异。返回一个数组，其中第一个元素将为 0，第二个元素是 a[1]-a[0] 之间的差值。注意若 NULL 值存在，返回结果为 NULL

语法

```
ARRAY_DIFFERENCE(<arr>)
```

参数

参数	说明
<arr>	用于相邻数组元素之间的差异的数组

返回值

返回一个数组。特殊情况：- 如果 NULL 值存在，返回结果为 NULL

举例

```
CREATE TABLE array_type_table (
   k1 INT,
   k2 ARRAY<INT>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table (k1, k2) VALUES
(0, []),
(1, [NULL]),
(2, [1, 2, 3]),
(3, [1, NULL, 3]),
(4, [0, 1, 2, 3, NULL, 4, 6]),
(5, [1, 2, 3, 4, 5, 4, 3, 2, 1]),
(6, [6, 7, 8]);
select *,array_difference(k2) from array_type_table;
```

```
+------+------------------------------+---------------------------------+
| k1   | k2                           | array_difference(`k2`)          |
+------+------------------------------+---------------------------------+
```

```
|    0 | []                          | []                               |
|    1 | [NULL]                      | [NULL]                           |
|    2 | [1, 2, 3]                   | [0, 1, 1]                        |
|    3 | [1, NULL, 3]                | [0, NULL, NULL]                  |
|    4 | [0, 1, 2, 3, NULL, 4, 6]    | [0, 1, 1, 1, NULL, NULL, 2]      |
|    5 | [1, 2, 3, 4, 5, 4, 3, 2, 1] | [0, 1, 1, 1, 1, -1, -1, -1, -1] |
|    6 | [6, 7, 8]                   | [0, 1, 1]                        |
+------+-----------------------------+----------------------------------+
```

7.2.1.8.10 ARRAY_DISTINCT

描述

返回去除了重复元素的数组，如果输入数组为 NULL，则返回 NULL。

语法

```
ARRAY_DISTINCT(<arr> )
```

参数

参数	说明
<arr>	可能包含要删除的重复元素的数组

返回值

返回去除了重复元素的数组。特殊情况：- 如果输入数组为 NULL，则返回 NULL。

举例

```
CREATE TABLE array_test (
    k1 INT,
    k2 ARRAY<INT>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_test VALUES
(1, [1, 2, 3, 4, 5]),
(2, [6, 7, 8]),
(3, []),
(4, NULL),
(5, [1, 2, 3, 4, 5, 4, 3, 2, 1]),
(6, [1, 2, 3, NULL]),
(7, [1, 2, 3, NULL, NULL]);
```

```
select k1, k2, array_distinct(k2) from array_test;
```

```
+------+------------------------------+--------------------------+
| k1   | k2                           | array_distinct(k2)       |
+------+------------------------------+--------------------------+
| 1    | [1, 2, 3, 4, 5]              | [1, 2, 3, 4, 5]          |
| 2    | [6, 7, 8]                    | [6, 7, 8]                |
| 3    | []                           | []                       |
| 4    | NULL                         | NULL                     |
| 5    | [1, 2, 3, 4, 5, 4, 3, 2, 1] | [1, 2, 3, 4, 5]          |
| 6    | [1, 2, 3, NULL]              | [1, 2, 3, NULL]          |
| 7    | [1, 2, 3, NULL, NULL]        | [1, 2, 3, NULL]          |
+------+------------------------------+--------------------------+
```

```
CREATE TABLE array_test01 (
   k1 INT,
   k2 ARRAY<VARCHAR>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_test01 VALUES
(1, ['a', 'b', 'c', 'd', 'e']),
(2, ['f', 'g', 'h']),
(3, ['']),
(3, [NULL]),
(5, ['a', 'b', 'c', 'd', 'e', 'a', 'b', 'c']),
(6, NULL),
(7, ['a', 'b', NULL]),
(8, ['a', 'b', NULL, NULL]);
select k1, k2, array_distinct(k2) from array_test01;
```

```
+------+-----------------------------------------+--------------------------+
| k1   | k2                                      | array_distinct(`k2`)     |
+------+-----------------------------------------+--------------------------+
| 1    | ['a', 'b', 'c', 'd', 'e']               | ['a', 'b', 'c', 'd', 'e'] |
| 2    | ['f', 'g', 'h']                         | ['f', 'g', 'h']          |
| 3    | ['']                                    | ['']                     |
| 3    | [NULL]                                  | [NULL]                   |
| 5    | ['a', 'b', 'c', 'd', 'e', 'a', 'b', 'c'] | ['a', 'b', 'c', 'd', 'e'] |
| 6    | NULL                                    | NULL                     |
| 7    | ['a', 'b', NULL]                        | ['a', 'b', NULL]         |
| 8    | ['a', 'b', NULL, NULL]                  | ['a', 'b', NULL]         |
```

```
+------+-----------------------------------------+--------------------------+
```

7.2.1.8.11 ARRAY_ENUMERATE

描述

返回数组下标，例如 [1, 2, 3, ···, length (arr)]

语法

```
ARRAY_ENUMERATE(<arr>)
```

参数

参数	说明
<arr>	需要返回数组下标的数组

返回值

返回包含数组下标的数组，特殊情况：- 如果参数是 NULL，则返回 NULL

举例

```
create table array_type_table(
    k1 INT,
    k2 Array<STRING>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
    'replication_num' = '1'
);
insert into array_type_table values (0, []),
("1", [NULL]),
("2", ["1", "2", "3"]),
("3", ["1", NULL, "3"]),
("4", NULL);
select k2, array_enumerate(k2) from array_type_table;
```

```
+------------------+-----------------------+
| k2               | array_enumerate(`k2`) |
+------------------+-----------------------+
| []               | []                    |
| [NULL]           | [1]                   |
| ['1', '2', '3']  | [1, 2, 3]             |
| ['1', NULL, '3'] | [1, 2, 3]             |
| NULL             | NULL                  |
```

```
+------------------+-----------------------+
```

7.2.1.8.12 ARRAY_ENUMERATE_UNIQ

描述

返回与源数组大小相同的数组，指示每个元素在具有相同值的元素中的位置，例如 array_enumerate_uniq([1, 2, 1, 4]) = [1, 1, 2, 1] 该函数也可接受多个大小相同的数组作为参数，这种情况下，返回的是数组中相同位置的元素组成的元组在具有相同值的元组中的位置。例如 array_enumerate_uniq([1, 2, 1, 1, 2], [2, 1, 2, 2, 1]) = [1, 1, 2, 3, 2]

语法

```
ARRAY_ENUMERATE_UNIQ(<arr1> [,<arr2> , ... ])
```

参数

Parameter	Description
<arr1>	需要计算的数组 arr1
<arr2>	需要计算的数组 arr2

返回值

返回与源数组大小相同的数组，指示每个元素在具有相同值的元素中的位置。

举例

```
select array_enumerate_uniq([1, 2, 3, 1, 2, 3]);
```

```
+-----------------------------------------------------+
| array_enumerate_uniq(ARRAY(1, 2, 3, 1, 2, 3))       |
+-----------------------------------------------------+
| [1, 1, 1, 2, 2, 2]                                  |
+-----------------------------------------------------+
```

```
select array_enumerate_uniq([1, 1, 1, 1, 1], [2, 1, 2, 1, 2], [3, 1, 3, 1, 3]);
```

```
+-------------------------------------------------------------------------------------------+
| array_enumerate_uniq(ARRAY(1, 1, 1, 1, 1), ARRAY(2, 1, 2, 1, 2), ARRAY(3, 1, 3, 1, 3)) |
+-------------------------------------------------------------------------------------------+
| [1, 1, 2, 1, 3]                                                                           |
+-------------------------------------------------------------------------------------------+
```

7.2.1.8.13 ARRAY_EXCEPT

描述

返回一个数组，包含所有在 array1 内但不在 array2 内的元素，不包含重复项，如果输入参数为 NULL，则返回 NULL

语法

```
ARRAY_EXCEPT(<arr1> , <arr2> )
```

参数

Parameter	Description
<arr1>	源数组
<arr2>	需要与 arr1 比较的元素数组

返回值

返回一个数组，特殊情况：- 如果输入参数为 NULL，则返回 NULL

举例

```
CREATE TABLE array_type_table (
    k1 INT,
    k2 ARRAY<INT>,
    k3 ARRAY<INT>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table VALUES
(1, [1, 2, 3], [2, 4, 5]),
(2, [2, 3], [1, 5]),
(3, [1, 1, 1], [2, 2, 2]);
select k1,k2,k3,array_except(k2,k3) from array_type_table;
```

```
+------+-----------------+--------------+--------------------------+
| k1   | k2              | k3           | array_except(`k2`, `k3`) |
+------+-----------------+--------------+--------------------------+
|    1 | [1, 2, 3]       | [2, 4, 5]    | [1, 3]                   |
|    2 | [2, 3]          | [1, 5]       | [2, 3]                   |
|    3 | [1, 1, 1]       | [2, 2, 2]    | [1]                      |
+------+-----------------+--------------+--------------------------+
```

```
CREATE TABLE array_type_table_nullable (
    k1 INT,
    k2 ARRAY<INT>,
    k3 ARRAY<INT>
)
duplicate key (k1)
```

```
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_nullable VALUES
(1, [1, NULL, 3], [1, 3, 5]),
(2, [NULL, NULL, 2], [2, NULL, 4]),
(3, NULL, [1, 2, 3]);
select k1,k2,k3,array_except(k2,k3) from array_type_table_nullable;
```

```
+------+-----------------+--------------+--------------------------+
| k1   | k2              | k3           | array_except(`k2`, `k3`) |
+------+-----------------+--------------+--------------------------+
|    1 | [1, NULL, 3]    | [1, 3, 5]    | [NULL]                   |
|    2 | [NULL, NULL, 2] | [2, NULL, 4] | []                       |
|    3 | NULL            | [1, 2, 3]    | NULL                     |
+------+-----------------+--------------+--------------------------+
```

```
CREATE TABLE array_type_table_varchar (
    k1 INT,
    k2 ARRAY<VARCHAR>,
    k3 ARRAY<VARCHAR>
)
    duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_varchar VALUES
(1, ['hello', 'world', 'c++'], ['I', 'am', 'c++']),
(2, ['a1', 'equals', 'b1'], ['a2', 'equals', 'b2']),
(3, ['hasnull', NULL, 'value'], ['nohasnull', 'nonull', 'value']),
(3, ['hasnull', NULL, 'value'], ['hasnull', NULL, 'value']);
select k1,k2,k3,array_except(k2,k3) from array_type_table_varchar;
```

```
+------+---------------------------+---------------------------------+----------------------+
| k1   | k2                        | k3                              | array_except(k2, k3) |
+------+---------------------------+---------------------------------+----------------------+
|    1 | ["hello", "world", "c++"]  | ["I", "am", "c++"]              | ["hello", "world"]   |
|    2 | ["a1", "equals", "b1"]     | ["a2", "equals", "b2"]          | ["a1", "b1"]         |
|    3 | ["hasnull", null, "value"] | ["hasnull", null, "value"]      | []                   |
|    3 | ["hasnull", null, "value"] | ["nohasnull", "nonull", "value"] | ["hasnull", null]    |
+------+---------------------------+---------------------------------+----------------------+
```

```
CREATE TABLE array_type_table_decimal (
```

```
    k1 INT,
    k2 ARRAY<DECIMAL(10, 2)>,
    k3 ARRAY<DECIMAL(10, 2)>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_decimal VALUES
(1, [1.1, 2.1, 3.44], [2.1, 3.4, 5.4]),
(2, [NULL, 2, 5], [NULL, NULL, 5.4]),
(1, [1, NULL, 2, 5], [1, 3.1, 5.4]);
select k1,k2,k3,array_except(k2,k3) from array_type_table_decimal;
```

```
+------+--------------------------+-------------------+----------------------+
| k1   | k2                       | k3                | array_except(k2, k3) |
+------+--------------------------+-------------------+----------------------+
|    1 | [1.00, null, 2.00, 5.00] | [1.00, 3.10, 5.40] | [null, 2.00, 5.00]   |
|    1 | [1.10, 2.10, 3.44]       | [2.10, 3.40, 5.40] | [1.10, 3.44]         |
|    2 | [null, 2.00, 5.00]       | [null, null, 5.40] | [2.00, 5.00]         |
+------+--------------------------+-------------------+----------------------+
```

7.2.1.8.14 ARRAY_EXISTS

描述

使用一个可选 lambda 表达式作为输入参数，对其他的输入 ARRAY 参数的内部数据做对应表达式计算。当计算返回非 0 时，返回 1；否则返回 0。在 lambda 表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。

在没有使用 lambda 作为参数时，array1 作为计算结果。

语法

```
ARRAY_EXISTS([ <lambda>, ] <arr1> [, <arr2> , ...] )
```

参数

Parameter	Description
<lambda>	lambda 表达式，可以执行合法的标量函数，不支持聚合函数等
<arr1>	需要计算的数组 arr1
<arr2>	需要计算的数组 arr2

返回值

返回使用表达式计算后的数组，特殊情况：- 如果数组里面包含 NULL 或者本身为 NULL，则返回 NULL

举例

```
CREATE TABLE array_test2 (
    id INT,
    c_array1 ARRAY<INT>,
    c_array2 ARRAY<INT>
)
duplicate key (id)
distributed by hash(id) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_test2 VALUES
(1, [1, 2, 3, 4, 5], [10, 20, -40, 80, -100]),
(2, [6, 7, 8], [10, 12, 13]),
(3, [1], [-100]),
(4, NULL, NULL);
select *, array_exists(x->x>1,[1,2,3]) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+-----------------------------------------------+
    ↪
| id   | c_array1        | c_array2                | array_exists([x] -> x(0) > 1, ARRAY(1, 2, 3)
    ↪ ) |
+------+-----------------+-------------------------+-----------------------------------------------+
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 1, 1]
    ↪                                      |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [0, 1, 1]
    ↪                                      |
|    3 | [1]             | [-100]                  | [0, 1, 1]
    ↪                                      |
|    4 | NULL            | NULL                    | [0, 1, 1]
    ↪                                      |
+------+-----------------+-------------------------+-----------------------------------------------+
    ↪
```

```
select c_array1, c_array2, array_exists(x->x%2=0,[1,2,3]) from array_test2 order by id;
```

```
+-----------------+-------------------------+-------------------------------------------------+
| c_array1        | c_array2                | array_exists([x] -> x(0) % 2 = 0, ARRAY(1, 2, 3)) |
+-----------------+-------------------------+-------------------------------------------------+
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 1, 0]                                       |
| [6, 7, 8]       | [10, 12, 13]            | [0, 1, 0]                                       |
| [1]             | [-100]                  | [0, 1, 0]                                       |
| NULL            | NULL                    | [0, 1, 0]                                       |
+-----------------+-------------------------+-------------------------------------------------+
```

```
select c_array1, c_array2, array_exists(x->abs(x)-1,[1,2,3]) from array_test2 order by id;
```

```
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
| c_array1        | c_array2                | array_exists(cast(array_map(x -> (abs(x) - 1), [1,
    ↪ 2, 3]) as ARRAY<BOOLEAN>)) |
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 1, 1]
    ↪                                                                                |
| [6, 7, 8]       | [10, 12, 13]            | [0, 1, 1]
    ↪                                                                                |
| [1]             | [-100]                  | [0, 1, 1]
    ↪                                                                                |
| NULL            | NULL                    | [0, 1, 1]
    ↪                                                                                |
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
```

```
select c_array1, c_array2, array_exists((x,y)->x>y,c_array1,c_array2) from array_test2 order by
    ↪ id;
```

```
+-----------------+-------------------------+-----------------------------------------------------------------+
    ↪
| c_array1        | c_array2                | array_exists([x, y] -> x(0) > y(1), `c_array1`, `c_
    ↪ array2`) |
+-----------------+-------------------------+-----------------------------------------------------------------+
    ↪
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 0, 1, 0, 1]
    ↪                                                |
| [6, 7, 8]       | [10, 12, 13]            | [0, 0, 0]
    ↪                                                    |
| [1]             | [-100]                  | [1]
    ↪                                                        |
| NULL            | NULL                    | NULL
    ↪                                                       |
+-----------------+-------------------------+-----------------------------------------------------------------+
    ↪
```

```
select *, array_exists(c_array1) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+--------------------------+
| id   | c_array1        | c_array2                | array_exists(`c_array1`) |
+------+-----------------+-------------------------+--------------------------+
```

```
|    1 | [1, 2, 3, 0, 5] | [10, 20, -40, 80, -100] | [1, 1, 1, 0, 1]          |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [1, 1, 1]                |
|    3 | [1]             | [-100]                  | [1]                      |
|    4 | NULL            | NULL                    | NULL                     |
+------+-----------------+-------------------------+--------------------------+
```

7.2.1.8.15 ARRAY_FILTER

描述

使用 lambda 表达式作为输入参数，计算筛选另外的输入参数 ARRAY 列的数据。并过滤掉在结果中 0 和 NULL 的值。

语法

```
ARRAY_FILTER(<lambda>, <arr>)
ARRAY_FILTER(<arr>, <filter_column>)
```

参数

参数	说明
< ↪ lambda ↪ >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr ↪ >	ARRAY 数组

返回值

对其输入 ARRAY 参数的内部数据做对应表达式计算。 过滤掉在结果中 0 和 NULL 的值

举例

```
select c_array,array_filter(c_array,[0,1,0]) from array_test;
```

```
+-----------------+-----------------------------------------------------+
| c_array         | array_filter(`c_array`, ARRAY(FALSE, TRUE, FALSE)) |
+-----------------+-----------------------------------------------------+
```

```
| [1, 2, 3, 4, 5] | [2]                                                    |
| [6, 7, 8]       | [7]                                                    |
| []              | []                                                     |
| NULL            | NULL                                                   |
+-----------------+--------------------------------------------------------+
```

```
select array_filter(x->(x > 1),[1,2,3,0,null]);
```

```
+-----------------------------------------------------------------------------------------------+
| array_filter(ARRAY(1, 2, 3, 0, NULL), array_map([x] -> (x(0) > 1), ARRAY(1, 2, 3, 0, NULL))) |
+-----------------------------------------------------------------------------------------------+
| [2, 3]                                                                                        |
+-----------------------------------------------------------------------------------------------+
```

```
select *, array_filter(x->x>0,c_array2) from array_test2;
```

```
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
| id   | c_array1        | c_array2                | array_filter(`c_array2`, array_map([x] -> x
    ↪ (0) > 0, `c_array2`)) |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [10, 20, 80]
    ↪                                                        |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [10, 12, 13]
    ↪                                                        |
|    3 | [1]             | [-100]                  | []
    ↪                                                          |
|    4 | NULL            | NULL                    | NULL
    ↪                                                        |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
```

```
select *, array_filter(x->x%2=0,c_array2) from array_test2;
```

```
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
| id   | c_array1        | c_array2                | array_filter(`c_array2`, array_map([x] -> x
    ↪ (0) % 2 = 0, `c_array2`)) |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [10, 20, -40, 80, -100]
    ↪                                                  |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [10, 12]
    ↪                                                        |
```

```
|    3 | [1]             | [-100]                  | [-100]
    ↪                                                                  |
|    4 | NULL            | NULL                    | NULL
    ↪                                                                    |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
```

```
select *, array_filter(x->(x*(-10)>0),c_array2) from array_test2;
```

```
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
| id   | c_array1        | c_array2                | array_filter(`c_array2`, array_map([x] -> (x
    ↪ (0) * (-10) > 0), `c_array2`)) |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [-40, -100]
    ↪                                                                  |
|    2 | [6, 7, 8]       | [10, 12, 13]            | []
    ↪                                                                      |
|    3 | [1]             | [-100]                  | [-100]
    ↪                                                                 |
|    4 | NULL            | NULL                    | NULL
    ↪                                                                    |
+------+-----------------+-------------------------+-------------------------------------------------------------
    ↪
```

```
select *, array_filter(x->x>0, array_map((x,y)->(x>y), c_array1,c_array2)) as res from array_
    ↪ test2;
```

```
+------+-----------------+-------------------------+--------+
| id   | c_array1        | c_array2                | res    |
+------+-----------------+-------------------------+--------+
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [1, 1] |
|    2 | [6, 7, 8]       | [10, 12, 13]            | []     |
|    3 | [1]             | [-100]                  | [1]    |
|    4 | NULL            | NULL                    | NULL   |
+------+-----------------+-------------------------+--------+
```

7.2.1.8.16 ARRAY_FIRST

描述

使用一个 lambda 表达式和一个 ARRAY 作为输入参数，lambda 表达式为布尔型，用于对 ARRAY 中的每个元素进行判断返回值。返回数组中的第一个 lambda(arr1[i]) 值不为 0 的元素。当数组中所有元素进行 lambda(arr1[i]) 都为 0 时，结果返回NULL值。

语法

```
ARRAY_FIRST(<lambda>, <arr>)
```

参数

参数	说明
<lambda>	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr>	ARRAY 数组

返回值

返回值不为 0 的索引。如果没找到满足此条件的索引，则返回 NULL。

举例

```
select array_first(x->x>2, [1,2,3,0]);
```

```
+-----------------------------------------------------------------------------------------------------+
    ↪
| array_first(array_filter(ARRAY(1, 2, 3, 0), array_map([x] -> x(0) > 2, ARRAY(1, 2, 3, 0))), -1)
    ↪ |
+-----------------------------------------------------------------------------------------------------+
    ↪
|                                                                                                   3
    ↪ |
+-----------------------------------------------------------------------------------------------------+
    ↪
```

```
select array_first(x->x>4, [1,2,3,0]);
```

```
+-----------------------------------------------------------------------------------------------------+
    ↪
| array_first(array_filter(ARRAY(1, 2, 3, 0), array_map([x] -> x(0) > 4, ARRAY(1, 2, 3, 0))), -1)
    ↪ |
+-----------------------------------------------------------------------------------------------------+
    ↪
|                                                                                                NULL
    ↪ |
+-----------------------------------------------------------------------------------------------------+
    ↪
```

```
select array_first(x->x>1, [1,2,3,0]);
```

```
+-----------------------------------------------------------------------------------------------+
| array_first(array_filter(ARRAY(1, 2, 3, 0), array_map([x] -> x > 1, ARRAY(1, 2, 3, 0))), 1) |
+-----------------------------------------------------------------------------------------------+
|                                                                                             2 |
+-----------------------------------------------------------------------------------------------+
```

7.2.1.8.17 ARRAY_FIRST_INDEX

描述

使用 lambda 表达式作为输入参数，对其他输入 ARRAY 参数的内部数据进行相应的表达式计算。返回第一个使得 `lambda(array1[i], ...)` 返回值不为 0 的索引。如果没找到满足此条件的索引，则返回 0。

在 lambda 表达式中输入的参数为 1 个或多个，所有输入的 array 的元素数量必须一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。

语法

```
ARRAY_FIRST_INDEX(<lambda>, <arr> [, ...])
```

参数

参数	说明
<lambda>	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr>	ARRAY 数组

返回值

返回值不为 0 的索引。如果没找到满足此条件的索引，则返回 0。

举例

```
select array_first_index(x->x+1>3, [2, 3, 4]);
```

```
+-----------------------------------------------------------------+
| array_first_index(array_map([x] -> x(0) + 1 > 3, ARRAY(2, 3, 4))) |
+-----------------------------------------------------------------+
|                                                               2 |
+-----------------------------------------------------------------+
```

```
select array_first_index(x -> x is null, [null, 1, 2]);
```

```
+-------------------------------------------------------------------+
| array_first_index(array_map([x] -> x(0) IS NULL, ARRAY(NULL, 1, 2))) |
+-------------------------------------------------------------------+
|                                                                 1 |
+-------------------------------------------------------------------+
```

```
select array_first_index(x->power(x,2)>10, [1, 2, 3, 4]);
```

```
+-----------------------------------------------------------------------------------+
| array_first_index(array_map([x] -> power(x(0), 2.0) > 10.0, ARRAY(1, 2, 3, 4))) |
+-----------------------------------------------------------------------------------+
|                                                                                 4 |
+-----------------------------------------------------------------------------------+
```

```
select col2, col3, array_first_index((x,y)->x>y, col2, col3) from array_test;
```

```
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
| col2         | col3         | array_first_index(array_map([x, y] -> x(0) > y(1), `col2`, `col3
    ↪ `)) |
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
| [1, 2, 3]    | [3, 4, 5]    |
    ↪                                                                       0 |
| [1, NULL, 2] | [NULL, 3, 1] |
    ↪                                                                       3 |
| [1, 2, 3]    | [9, 8, 7]    |
    ↪                                                                       0 |
| NULL         | NULL         |
    ↪                                                                       0 |
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
```

7.2.1.8.18 ARRAY_INTERSECT

描述

返回一个数组，包含 array1 和 array2 的交集中的所有元素，不包含重复项，如果输入参数为 NULL，则返回 NULL

语法

```
ARRAY_INTERSECT(<arr1> , <arr2> )
```

参数

Parameter	Description
<arr1>	源数组
<arr2>	与 arr1 求交集的数组

返回值

返回包含 array1 和 array2 的交集中的所有元素的数组，特殊情况：- 如果输入参数为 NULL，则返回 NULL

注意事项

仅支持向量化引擎中使用

举例

```
CREATE TABLE array_type_table (
    k1 INT,
    k2 ARRAY<INT>,
    k3 ARRAY<INT>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table VALUES
(1, [1, 2, 3], [2, 4, 5]),
(2, [2, 3], [1, 5]),
(3, [1, 1, 1], [2, 2, 2]);
select k1,k2,k3,array_intersect(k2,k3) from array_type_table;
```

```
+------+-----------------+--------------+-----------------------------+
| k1   | k2              | k3           | array_intersect(`k2`, `k3`) |
+------+-----------------+--------------+-----------------------------+
|    1 | [1, 2, 3]       | [2, 4, 5]    | [2]                         |
|    2 | [2, 3]          | [1, 5]       | []                          |
|    3 | [1, 1, 1]       | [2, 2, 2]    | []                          |
+------+-----------------+--------------+-----------------------------+
```

```
CREATE TABLE array_type_table_nullable (
    k1 INT,
    k2 ARRAY<INT>,
    k3 ARRAY<INT>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_nullable VALUES
(1, [1, NULL, 3], [1, 3, 5]),
(2, [NULL, NULL, 2], [2, NULL, 4]),
(3, NULL, [1, 2, 3]);
select k1,k2,k3,array_intersect(k2,k3) from array_type_table_nullable;
```

```
+------+-----------------+--------------+-----------------------------+
| k1   | k2              | k3           | array_intersect(`k2`, `k3`) |
+------+-----------------+--------------+-----------------------------+
|    1 | [1, NULL, 3]    | [1, 3, 5]    | [1, 3]                      |
|    2 | [NULL, NULL, 2] | [2, NULL, 4] | [NULL, 2]                   |
|    3 | NULL            | [1, 2, 3]    | NULL                        |
+------+-----------------+--------------+-----------------------------+
```

```
CREATE TABLE array_type_table_varchar (
    k1 INT,
    k2 ARRAY<VARCHAR>,
    k3 ARRAY<VARCHAR>
)
    duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_varchar VALUES
(1, ['hello', 'world', 'c++'], ['I', 'am', 'c++']),
(2, ['a1', 'equals', 'b1'], ['a2', 'equals', 'b2']),
(3, ['hasnull', NULL, 'value'], ['nohasnull', 'nonull', 'value']),
(3, ['hasnull', NULL, 'value'], ['hasnull', NULL, 'value']);
select k1,k2,k3,array_intersect(k2,k3) from array_type_table_varchar;
```

```
+------+---------------------------+---------------------------------+---------------------------+
    ↪
| k1   | k2                        | k3                              | array_intersect(k2, k3)
    ↪    |
```

```
+------+---------------------------+---------------------------------+---------------------------+
    ↪
|    1 | ["hello", "world", "c++"]  | ["I", "am", "c++"]              | ["c++"]
    ↪                     |
|    2 | ["a1", "equals", "b1"]     | ["a2", "equals", "b2"]          | ["equals"]
    ↪                 |
|    3 | ["hasnull", null, "value"] | ["hasnull", null, "value"]      | [null, "value", "hasnull
    ↪ "] |
|    3 | ["hasnull", null, "value"] | ["nohasnull", "nonull", "value"] | ["value"]
    ↪                   |
+------+---------------------------+---------------------------------+---------------------------+```
    ↪
```

```
CREATE TABLE array_type_table_decimal (
    k1 INT,
    k2 ARRAY<DECIMAL(10, 2)>,
    k3 ARRAY<DECIMAL(10, 2)>
)
    duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_type_table_decimal VALUES
(1, [1.1, 2.1, 3.44], [2.1, 3.4, 5.4]),
(2, [NULL, 2, 5], [NULL, NULL, 5.4]),
(3, [1, NULL, 2, 5], [1, 3.1, 5.4]);
select k1,k2,k3,array_intersect(k2,k3) from array_type_table_decimal;
```

```
+------+--------------------------+-------------------+-------------------------+
| k1   | k2                       | k3                | array_intersect(k2, k3) |
+------+--------------------------+-------------------+-------------------------+
|    1 | [1.10, 2.10, 3.44]       | [2.10, 3.40, 5.40] | [2.10]                  |
|    2 | [null, 2.00, 5.00]       | [null, null, 5.40] | [null]                  |
|    3 | [1.00, null, 2.00, 5.00] | [1.00, 3.10, 5.40] | [1.00]                  |
+------+--------------------------+-------------------+-------------------------+
```

7.2.1.8.19 ARRAY_JOIN

描述

根据分隔符 (sep) 和替换 NULL 的字符串 (null_replace), 将数组中的所有元素组合成一个新的字符串。

语法

```
ARRAY_JOIN(<arr> , <sep> [, <null_replace>])
```

参数

Parameter	Description
<arr>	需要 join 的数组
<sep>	分隔符
<null_replace>	替换 NULL 的字符串

返回值

返回一个新的字符串，特殊情况：- 若 sep 为 NULL，则返回值为 NULL。- 若 null_replace 为 NULL，则返回值也为 NULL。- 若 sep 为空字符串，则不应用任何分隔符。- 若 null_replace 为空字符串或者不指定，则直接丢弃数组中的 NULL 元素。

举例

```
CREATE TABLE array_test (
                            k1 INT,
                            k2 ARRAY<INT>
)
    duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);

INSERT INTO array_test VALUES
                           (1, [1, 2, 3, 4, 5]),
                           (2, [6, 7, 8]),
                           (3, []),
                           (4, NULL),
                           (5, [1, 2, 3, 4, 5, 4, 3, 2, 1]),
                           (6, [1, 2, 3, NULL]),
                           (7, [4, 5, 6, NULL, NULL]);
select k1, k2, array_join(k2, '_', 'null') from array_test order by k1;
```

```
+------+------------------------------+-------------------------------------+
| k1   | k2                           | array_join(`k2`, '_', 'null')       |
+------+------------------------------+-------------------------------------+
|  1   | [1, 2, 3, 4, 5]              | 1_2_3_4_5                           |
|  2   | [6, 7, 8]                    | 6_7_8                               |
|  3   | []                           |                                     |
|  4   | NULL                         | NULL                                |
|  5   | [1, 2, 3, 4, 5, 4, 3, 2, 1] | 1_2_3_4_5_4_3_2_1                   |
|  6   | [1, 2, 3, NULL]              | 1_2_3_null                          |
|  7   | [4, 5, 6, NULL, NULL]        | 4_5_6_null_null                     |
+------+------------------------------+-------------------------------------+
```

```
select k1, k2, array_join(k2, '_') from array_test order by k1;
```

```
+------+-----------------------------+----------------------------+
| k1   | k2                          | array_join(`k2`, '_')      |
+------+-----------------------------+----------------------------+
|  1   | [1, 2, 3, 4, 5]             | 1_2_3_4_5                  |
|  2   | [6, 7, 8]                   | 6_7_8                      |
|  3   | []                          |                            |
|  4   | NULL                        | NULL                       |
|  5   | [1, 2, 3, 4, 5, 4, 3, 2, 1] | 1_2_3_4_5_4_3_2_1          |
|  6   | [1, 2, 3, NULL]             | 1_2_3                      |
|  7   | [4, 5, 6, NULL, NULL]       | 4_5_6                      |
+------+-----------------------------+----------------------------+
```

```
CREATE TABLE array_test01 (
    k1 INT,
    k2 ARRAY<STRING>
)
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
  'replication_num' = '1'
);

INSERT INTO array_test01 VALUES
(1, ['a', 'b', 'c', 'd']),
(2, ['e', 'f', 'g', 'h']),
(3, [NULL, 'a', NULL, 'b', NULL, 'c']),
(4, ['d', 'e', NULL, ' ']),
(5, [' ', NULL, 'f', 'g']);
select k1, k2, array_join(k2, '_', 'null') from array_test01 order by k1;
```

```
+------+-----------------------------------+-----------------------------------+
| k1   | k2                                | array_join(`k2`, '_', 'null')     |
+------+-----------------------------------+-----------------------------------+
|  1   | ['a', 'b', 'c', 'd']              | a_b_c_d                           |
|  2   | ['e', 'f', 'g', 'h']              | e_f_g_h                           |
|  3   | [NULL, 'a', NULL, 'b', NULL, 'c'] | null_a_null_b_null_c              |
|  4   | ['d', 'e', NULL, ' ']             | d_e_null_                         |
|  5   | [' ', NULL, 'f', 'g']             |  _null_f_g                        |
+------+-----------------------------------+-----------------------------------+
```

```
select k1, k2, array_join(k2, '_') from array_test01 order by k1;
```

```
+------+-----------------------------------+----------------------------+
| k1   | k2                                | array_join(`k2`, '_')      |
+------+-----------------------------------+----------------------------+
|  1   | ['a', 'b', 'c', 'd']              | a_b_c_d                    |
|  2   | ['e', 'f', 'g', 'h']              | e_f_g_h                    |
|  3   | [NULL, 'a', NULL, 'b', NULL, 'c'] | a_b_c                      |
|  4   | ['d', 'e', NULL, ' ']             | d_e_                       |
|  5   | [' ', NULL, 'f', 'g']             |  _f_g                      |
+------+-----------------------------------+----------------------------+
```

7.2.1.8.20 ARRAY_LAST

描述

使用一个 lambda 表达式和一个 ARRAY 作为输入参数，lambda 表达式为布尔型，用于对 ARRAY 中的每个元素进行判断返回值。返回数组中的最后一个 lambda(arr1[i]) 值不为 0 的元素。当数组中所有元素进行 lambda(arr1[i]) 都为 0 时，结果返回NULL值。

语法

```
ARRAY_LAST(<lambda>, <arr>)
```

参数

参数	说明
< ↪ lambda ↪ >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr ↪ >	ARRAY 数组

返回值

返回值最后不为 0 的索引。如果没找到满足此条件的索引，则返回 NULL。

举例

```
select array_last(x->x>2, [1,2,3,0]) ;
```

```
+-----------------------------------------------------------------------------------------------+
    ↪
```

```
| array_last(array_filter(ARRAY(1, 2, 3, 0), array_map([x] -> x(0) > 2, ARRAY(1, 2, 3, 0))), -1)
    ↪ |
+-----------------------------------------------------------------------------------------------+
    ↪
|                                                                                             3
    ↪ |
+-----------------------------------------------------------------------------------------------+
    ↪
```

```
select array_last(x->x>4, [1,2,3,0]) ;
```

```
+-----------------------------------------------------------------------------------------------+
    ↪
| array_last(array_filter(ARRAY(1, 2, 3, 0), array_map([x] -> x(0) > 4, ARRAY(1, 2, 3, 0))), -1)
    ↪ |
+-----------------------------------------------------------------------------------------------+
    ↪
|                                                                                          NULL
    ↪ |
+-----------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.8.21 ARRAY_LAST_INDEX

描述

使用 lambda 表达式作为输入参数，对其他输入 ARRAY 参数的内部数据进行相应的表达式计算。返回最后一个使得 `lambda(array1[i], ...)` 返回值不为 0 的索引。如果没找到满足此条件的索引，则返回 0。

在 lambda 表达式中输入的参数为 1 个或多个，所有输入的 array 的元素数量必须一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。

语法

```
ARRAY_LAST_INDEX(<lambda>, <arr> [, ...])
```

参数

参数	说明
< ↪ lambda ↪ >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr ↪ >	ARRAY 数组

返回值

返回值最后不为 0 的索引。如果没找到满足此条件的索引，则返回 0。

举例

```
select array_last_index(x -> x is null, [null, null, 1, 2]);
```

```
+-----------------------------------------------------------------------+
| array_last_index(array_map([x] -> x IS NULL, ARRAY(NULL, NULL, 1, 2))) |
+-----------------------------------------------------------------------+
```

```
|                                                                    2 |
+----------------------------------------------------------------------+
```

```
select array_last_index(x->x='s', ['a', 's', 's', 's', 'b']);
```

```
+--------------------------------------------------------------------------------+
| array_last_index(array_map([x] -> x = 's', ARRAY('a', 's', 's', 's', 'b'))) |
+--------------------------------------------------------------------------------+
|                                                                              4 |
+--------------------------------------------------------------------------------+
```

```
select array_last_index(x->power(x,2)>10, [1, 4, 3, 4]);
```

```
+--------------------------------------------------------------------------------+
| array_last_index(array_map([x] -> power(x, 2.0) > 10.0, ARRAY(1, 4, 3, 4))) |
+--------------------------------------------------------------------------------+
|                                                                              4 |
+--------------------------------------------------------------------------------+
```

```
select col2, col3, array_last_index((x,y)->x>y, col2, col3) from array_test;
```

```
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
| col2         | col3         | array_last_index(array_map([x, y] -> x(0) > y(1), `col2`, `col3`)
    ↪ ) |
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
| [1, 2, 3]    | [3, 4, 5]    |
    ↪                                                                    0 |
| [1, NULL, 2] | [NULL, 3, 1] |
    ↪                                                                    3 |
| [1, 2, 3]    | [9, 8, 7]    |
    ↪                                                                    0 |
| NULL         | NULL         |
    ↪                                                                    0 |
+--------------+--------------+-------------------------------------------------------------------------+
    ↪
```

7.2.1.8.22 ARRAY_MAP

描述

使用一个 lambda 表达式作为输入参数，对其他的输入 ARRAY 参数的内部数据做对应表达式计算。在 lambda 表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。

语法

```
ARRAY_MAP(lambda, <arr> [ , <arr> ... ] )
```

参数

参数	说明
lambda	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr>	ARRAY 数组

返回值

经过表达式 lambda 计算之后 ARRAY 数组。

举例

```
CREATE TABLE array_test2 (
                             id INT,
                             c_array1 ARRAY<INT>,
                             c_array2 ARRAY<INT>
)
    duplicate key (id)
distributed by hash(id) buckets 1
properties(
    'replication_num' = '1'
    );
INSERT INTO array_test2 (id, c_array1, c_array2) VALUES
                                                    (1, [1, 2, 3, 4, 5], [10, 20, -40, 80,
                                                        ↪ -100]),
                                                    (2, [6, 7, 8], [10, 12, 13]),
                                                    (3, [1], [-100]),
                                                    (4, NULL, NULL);
select *, array_map(x->x,[1,2,3]) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+-----------------------------------------+
| id   | c_array1        | c_array2                | array_map([x] -> x(0), ARRAY(1, 2, 3)) |
+------+-----------------+-------------------------+-----------------------------------------+
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [1, 2, 3]                               |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [1, 2, 3]                               |
|    3 | [1]             | [-100]                  | [1, 2, 3]                               |
|    4 | NULL            | NULL                    | [1, 2, 3]                               |
+------+-----------------+-------------------------+-----------------------------------------+
```

```
select *, array_map(x->x+2,[1,2,3]) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+-----------------------------------------------+
| id   | c_array1        | c_array2                | array_map([x] -> x(0) + 2, ARRAY(1, 2, 3)) |
+------+-----------------+-------------------------+-----------------------------------------------+
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [3, 4, 5]                                     |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [3, 4, 5]                                     |
|    3 | [1]             | [-100]                  | [3, 4, 5]                                     |
|    4 | NULL            | NULL                    | [3, 4, 5]                                     |
+------+-----------------+-------------------------+-----------------------------------------------+
```

```
select c_array1, c_array2, array_map(x->x,[1,2,3]) from array_test2 order by id;
```

```
+-----------------+-------------------------+-----------------------------------------+
| c_array1        | c_array2                | array_map([x] -> x(0), ARRAY(1, 2, 3)) |
+-----------------+-------------------------+-----------------------------------------+
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [1, 2, 3]                               |
```

```
| [6, 7, 8]       | [10, 12, 13]            | [1, 2, 3]                                |
| [1]             | [-100]                  | [1, 2, 3]                                |
| NULL            | NULL                    | [1, 2, 3]                                |
+-----------------+-------------------------+------------------------------------------+
```

```
select c_array1, c_array2, array_map(x->power(x,2),[1,2,3]) from array_test2 order by id;
```

```
+-----------------+-------------------------+-------------------------------------------------------+
    ↪
| c_array1        | c_array2                | array_map([x] -> power(x(0), 2.0), ARRAY(1, 2, 3))
    ↪ |
+-----------------+-------------------------+-------------------------------------------------------+
    ↪
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [1, 4, 9]
    ↪ |
| [6, 7, 8]       | [10, 12, 13]            | [1, 4, 9]
    ↪ |
| [1]             | [-100]                  | [1, 4, 9]
    ↪ |
| NULL            | NULL                    | [1, 4, 9]
    ↪ |
+-----------------+-------------------------+-------------------------------------------------------+
    ↪
```

```
select c_array1, c_array2, array_map((x,y)->x+y,c_array1,c_array2) from array_test2 order by id;
```

```
+-----------------+-------------------------+-------------------------------------------------------------+
    ↪
| c_array1        | c_array2                | array_map([x, y] -> x(0) + y(1), `c_array1`, `c_
    ↪ array2`) |
+-----------------+-------------------------+-------------------------------------------------------------+
    ↪
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [11, 22, -37, 84, -95]
    ↪                                     |
| [6, 7, 8]       | [10, 12, 13]            | [16, 19, 21]
    ↪                                            |
| [1]             | [-100]                  | [-99]
    ↪                                                    |
| NULL            | NULL                    | NULL
    ↪                                                     |
+-----------------+-------------------------+-------------------------------------------------------------+
    ↪
```

```
select c_array1, c_array2, array_map((x,y)->power(x,2)+y,c_array1, c_array2) from array_test2
    ↪ order by id;
```

```
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
| c_array1        | c_array2                | array_map([x, y] -> power(x(0), 2.0) + y(1), `c_
    ↪ array1`, `c_array2`) |
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
| [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [11, 24, -31, 96, -75]
    ↪                                                 |
| [6, 7, 8]       | [10, 12, 13]            | [46, 61, 77]
    ↪                                                              |
| [1]             | [-100]                  | [-99]
    ↪                                                                           |
| NULL            | NULL                    | NULL
    ↪                                                                            |
+-----------------+-------------------------+-------------------------------------------------------------------------
    ↪
```

```
select *,array_map(x->x=3,c_array1) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+----------------------------------------+
| id   | c_array1        | c_array2                | array_map([x] -> x(0) = 3, `c_array1`) |
+------+-----------------+-------------------------+----------------------------------------+
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 0, 1, 0, 0]                        |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [0, 0, 0]                              |
|    3 | [1]             | [-100]                  | [0]                                    |
|    4 | NULL            | NULL                    | NULL                                   |
+------+-----------------+-------------------------+----------------------------------------+
```

```
select *,array_map(x->x>3,c_array1) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+----------------------------------------+
| id   | c_array1        | c_array2                | array_map([x] -> x(0) > 3, `c_array1`) |
+------+-----------------+-------------------------+----------------------------------------+
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 0, 0, 1, 1]                        |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [1, 1, 1]                              |
|    3 | [1]             | [-100]                  | [0]                                    |
|    4 | NULL            | NULL                    | NULL                                   |
+------+-----------------+-------------------------+----------------------------------------+
```

```
select *,array_map((x,y)->x>y,c_array1,c_array2) from array_test2 order by id;
```

```
+------+-----------------+-------------------------+-------------------------------------------------------------+
    ↪
```

```
| id   | c_array1        | c_array2                | array_map([x, y] -> x(0) > y(1), `c_array1`,
    ↪  `c_array2`) |
+------+-----------------+-------------------------+-----------------------------------------------------------+
    ↪
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [0, 0, 1, 0, 1]
    ↪                                          |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [0, 0, 0]
    ↪                                             |
|    3 | [1]             | [-100]                  | [1]
    ↪                                                      |
|    4 | NULL            | NULL                    | NULL
    ↪                                                     |
+------+-----------------+-------------------------+-----------------------------------------------------------+
    ↪
```

```
select array_map(x->cast(x as string), c_array1) from test_array_map_function;
```

```
+-----------------+-----------------------------------------------------+
| c_array1        | array_map([x] -> CAST(x(0) AS CHARACTER), `c_array1`) |
+-----------------+-----------------------------------------------------+
| [1, 2, 3, 4, 5] | ['1', '2', '3', '4', '5']                           |
| [6, 7, 8]       | ['6', '7', '8']                                     |
| []              | []                                                  |
| NULL            | NULL                                                |
+-----------------+-----------------------------------------------------+
```

7.2.1.8.23 ARRAY_MATCH_ALL

描述

检查数组中的所有元素是否都满足给定条件。如果数组包含 NULL 元素且所有非 NULL 元素都满足条件，则返回 NULL。

语法

```
array_match_all(lambda, <arr> [, <arr> ...])
```

参数

- `lambda`: 定义检查条件的 lambda 表达式
- `<arr>`: 一个或多个要检查的数组。lambda 函数将应用于这些数组的每个元素

返回值

返回一个可空的布尔值：- 如果数组中所有元素都满足条件，则返回 `true` - 如果数组中任何元素不满足条件，则返回 `false` - 如果数组包含 NULL 元素且所有非 NULL 元素都满足条件，则返回 NULL

示例

```
-- 检查数组中的所有数字是否都大于 5
mysql> SELECT array_match_all(x -> x > 5, [1, 2, 3, 4, 7]);
+-----------------------------------------------+
| array_match_all(x -> x > 5, [1, 2, 3, 4, 7]) |
+-----------------------------------------------+
|                                             0 |
+-----------------------------------------------+

-- 检查数组中的所有数字x是否都大于数字i
mysql> SELECT array_match_all((x, i) -> x > i, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]);
+-----------------------------------------------------------------------+
| array_match_all((x, i) -> x > i, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]) |
+-----------------------------------------------------------------------+
|                                                                     0 |
+-----------------------------------------------------------------------+
```

注意事项

1. 函数处理 NULL 值的方式：

- 如果存在 NULL 元素且所有非 NULL 元素都满足条件，则返回 NULL
- 如果任何非 NULL 元素不满足条件，则无论是否存在 NULL 元素都返回 false

2. 该函数适用于：

- 验证数组中的所有元素是否满足特定条件
- 与其他数组函数组合进行复杂的数组操作

7.2.1.8.24 ARRAY_MATCH_ANY

描述

检查数组中是否有任何元素满足给定条件。如果数组包含 NULL 元素且所有非 NULL 元素都不满足条件，则返回 NULL。

语法

```
array_match_any(lambda, <arr> [, <arr> ...])
```

参数

- lambda: 定义检查条件的 lambda 表达式
- <arr>: 一个或多个要检查的数组。lambda 函数将应用于这些数组的每个元素

返回值

返回一个可空的布尔值：- 如果数组中任何元素满足条件，则返回 true - 如果数组中所有元素都不满足条件，则返回 false - 如果数组包含 NULL 元素且所有非 NULL 元素都不满足条件，则返回 NULL

示例

```
-- 检查数组中是否有任何数字大于 5
mysql> SELECT array_match_any(x -> x > 5, [1, 2, 3, 4, 7]);
+-----------------------------------------------+
| array_match_any(x -> x > 5, [1, 2, 3, 4, 7]) |
+-----------------------------------------------+
|                                             1 |
+-----------------------------------------------+

-- 检查数组中是否有任何数字大于另一个数组中对应位置的数字
mysql> SELECT array_match_any((x, i) -> x > i, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]);
+-------------------------------------------------------------------------+
| array_match_any((x, i) -> x > i, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]) |
+-------------------------------------------------------------------------+
|                                                                       0 |
+-------------------------------------------------------------------------+

mysql> SELECT array_match_any((x, i) -> i > x, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]);
+-------------------------------------------------------------------------+
| array_match_any((x, i) -> i > x, [1, 2, 3, 4, 5], [1, 2, 3, 4, 7]) |
+-------------------------------------------------------------------------+
|                                                                       1 |
+-------------------------------------------------------------------------+
```

注意事项

1. 函数处理 NULL 值的方式：

- 如果存在 NULL 元素且所有非 NULL 元素都不满足条件，则返回 NULL
- 如果任何非 NULL 元素满足条件，则无论是否存在 NULL 元素都返回 true

2. 该函数适用于：

- 检查数组中是否有任何元素满足特定条件
- 与其他数组函数组合进行复杂的数组操作

7.2.1.8.25 ARRAY_MAX

描述

返回数组中最大的元素，数组中的NULL值会被跳过。空数组以及元素全为NULL值的数组，结果返回NULL值。

语法

```
ARRAY_MAX(<arr>)
```

参数

参数	说明
<arr>	ARRAY 数组

返回值

返回数组中最大的元素，特殊情况：- 数组中的NULL值会被跳过。- 空数组以及元素全为NULL值的数组，结果返回NULL值。

举例

```
create table array_type_table(
    k1 INT,
    k2 Array<int>
        )
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
    'replication_num' = '1'
    );
insert into array_type_table values (0, []), (1, [NULL]), (2, [1, 2, 3]), (3, [1, NULL, 3]);
select k2, array_max(k2) from array_type_table;
```

```
+--------------+-----------------+
| k2           | array_max(`k2`) |
+--------------+-----------------+
| []           |            NULL |
| [NULL]       |            NULL |
| [1, 2, 3]    |               3 |
| [1, NULL, 3] |               3 |
+--------------+-----------------+
```

7.2.1.8.26 ARRAY_MIN

描述

返回数组中最小的元素，数组中的NULL值会被跳过。空数组以及元素全为NULL值的数组，结果返回NULL值。

语法

```
ARRAY_MIN(<arr>)
```

参数

参数	说明
<arr>	ARRAY 数组

返回值

返回数组中最小的元素，特殊情况：- 数组中的NULL值会被跳过。- 空数组以及元素全为NULL值的数组，结果返回NULL值。

举例

```
create table array_type_table(
    k1 INT,
    k2 Array<int>
    )
duplicate key (k1)
distributed by hash(k1) buckets 1
properties(
    'replication_num' = '1'
    );
insert into array_type_table values (0, []), (1, [NULL]), (2, [1, 2, 3]), (3, [1, NULL, 3]);
select k2, array_min(k2) from array_type_table;
```

```
+--------------+-----------------+
| k2           | array_min(`k2`) |
+--------------+-----------------+
| []           |            NULL |
| [NULL]       |            NULL |
| [1, 2, 3]    |               1 |
| [1, NULL, 3] |               1 |
+--------------+-----------------+
```

7.2.1.8.27 ARRAY_POPBACK

描述

返回移除最后一个元素后的数组，如果输入参数为 NULL，则返回 NULL

语法

```
ARRAY_POPBACK(<arr>)
```

参数

参数	说明
<arr>	ARRAY 数组

返回值

返回移除最后一个元素后的数组。特殊情况：- 如果输入参数为 NULL，则返回 NULL

举例

```
select array_popback(['test', NULL, 'value']);
```

```
+-----------------------------------------------------+
| array_popback(ARRAY('test', NULL, 'value'))         |
+-----------------------------------------------------+
| ["test", NULL]                                        |
+-----------------------------------------------------+
```

7.2.1.8.28 ARRAY_POPFRONT

描述

返回移除第一个元素后的数组，如果输入参数为 NULL，则返回 NULL

语法

```
ARRAY_POPFRONT(<arr>)
```

参数

参数	说明
<arr>	ARRAY 数组

返回值

返回移除第一个元素后的数组。特殊情况：- 如果输入参数为 NULL，则返回 NULL

举例

```
select array_popfront(['test', NULL, 'value']);
```

```
+-----------------------------------------------------+
| array_popfront(ARRAY('test', NULL, 'value'))        |
+-----------------------------------------------------+
| [NULL, "value"]                                       |
+-----------------------------------------------------+
```

7.2.1.8.29 ARRAY_POSITION

描述

返回value在数组中第一次出现的位置/索引。

语法

```
ARRAY_POSITION(<arr>, <vaule>)
```

参数

参数	说明
<arr>	ARRAY 数组
<vaule>	待查询的元素

返回值

value 在 arr 中的位置（从 1 开始计算）。特殊情况：- 0，如果 value 在 arr 中不存在；- NULL，如果数组为 NULL。

举例

```
CREATE TABLE array_test (
                            id INT,
                            c_array ARRAY<INT>,
                            array_position INT
)
    duplicate key (id)
distributed by hash(id) buckets 1
properties(
  'replication_num' = '1'
);
INSERT INTO array_test (id, c_array, array_position) VALUES
                                                        (1, [1, 2, 3, 4, 5], 5),
                                                        (2, [6, 7, 8], 0),
                                                        (3, [], 0),
                                                        (4, NULL, NULL);
SELECT id,c_array,array_position(c_array, 5) FROM `array_test`;
```

```
+------+-----------------+------------------------------+
| id   | c_array         | array_position(`c_array`, 5) |
+------+-----------------+------------------------------+
|    1 | [1, 2, 3, 4, 5] |                            5 |
|    2 | [6, 7, 8]       |                            0 |
|    3 | []              |                            0 |
|    4 | NULL            |                         NULL |
+------+-----------------+------------------------------+
```

```
select array_position([1, null], null);
```

```
+-------------------------------------+
| array_position(ARRAY(1, NULL), NULL) |
+-------------------------------------+
```

```
|                                    2 |
+--------------------------------------+
```

7.2.1.8.30 ARRAY_PRODUCT

描述

计算数组中所有元素的乘积

语法

```
ARRAY_PRODUCT(<arr>)
```

参数

参数	说明
<arr>	对应数组

返回值

返回数组中所有元素的乘积，数组中的 NULL 值会被跳过。空数组以及元素全为 NULL 值的数组，结果返回 NULL 值。

举例

```
SELECT ARRAY_PRODUCT([1, 2, 3]),ARRAY_PRODUCT([1, NULL, 3]),ARRAY_PRODUCT([NULL]);
```

```
+-------------------------+----------------------------+-----------------------------------------+
    ↪
| array_product([1, 2, 3]) | array_product([1, NULL, 3]) | array_product(cast([NULL] as ARRAY<
    ↪ DOUBLE>)) |
+-------------------------+----------------------------+-----------------------------------------+
    ↪
|                       6 |                          3 |
    ↪                                          NULL |
+-------------------------+----------------------------+-----------------------------------------+
    ↪
```

7.2.1.8.31 ARRAY_PUSHBACK

描述

将 value 添加到数组的尾部。

语法

```
ARRAY_PUSHBACK(<arr>, <value>)
```

参数

参数	说明
<arr>	对应数组
<value>	待添加的值

返回值

返回添加 value 后的数组

举例

```
SELECT ARRAY_PUSHBACK([1, 2], 3),ARRAY_PUSHBACK([3, 4], 6);
```

```
+--------------------------+--------------------------+
| array_pushback([1, 2], 3) | array_pushback([3, 4], 6) |
+--------------------------+--------------------------+
| [1, 2, 3]                | [3, 4, 6]                |
+--------------------------+--------------------------+
```

7.2.1.8.32 ARRAY_PUSHFRONT

描述

将 value 添加到数组的开头

语法

```
ARRAY_PUSHFRONT(<arr>, <value>)
```

参数

参数	说明
<arr>	对应数组
<value>	待添加的值

返回值

返回添加 value 后的数组

举例

```
SELECT ARRAY_PUSHFRONT([1, 2], 3),ARRAY_PUSHFRONT([3, 4], 6);
```

```
+---------------------------+---------------------------+
| array_pushfront([1, 2], 3) | array_pushfront([3, 4], 6) |
+---------------------------+---------------------------+
| [3, 1, 2]                 | [6, 3, 4]                 |
+---------------------------+---------------------------+
```

7.2.1.8.33 ARRAY_RANGE

描述

1. 生成 int 数组
2. 生成日期时间数组

别名

- SEQUENCE

语法

```
ARRAY_RANGE(<end>)
ARRAY_RANGE(<start>, <end>)
ARRAY_RANGE(<start>, <end>, <step>)
ARRAY_RANGE(<start_datetime>, <end_datetime>)
ARRAY_RANGE(<start_datetime>, <end_datetime>, INTERVAL <interval_step> <unit>)
```

参数

参数	说明
<start>	起始值，为正整数，默认为 0
<end>	结束值，为正整数

参数	说明
< step >	步长，为正整数，默认为 1
< start_datetime >	起始日期，为 datetimev2 类型
< end_datetime >	结束日期，为 datetimev2 类型
< interval_step >	间隔值，默认为 1

参数	说明
< ↪ unit ↪ > ↪	间隔单位，支持年(year)/季度(quarter)/月(month)/周(week)/日(day)/小时(hour)/分钟(minute)/秒(second)，默认为日(day)

quarter 支持从 3.0.8 和 3.1.0 版本开始。

返回值

1. 返回一个数组，从 start 到 end - 1, 步长为 step。如果第三个参数 step 为负数或者零，函数结果将为 NULL
2. 返回 start_datetime 和最接近 end_datetime 之间的 datetimev2 数组（按 Interval_step UNIT 计算）。如果第三个参数 interval_step 为负数或者零，函数结果将为 NULL

举例

```
SELECT ARRAY_RANGE(0,20,2),ARRAY_RANGE(cast('2019-05-15 12:00:00' as datetimev2(0)), cast('
    ↪ 2022-05-17 12:00:00' as datetimev2(0)), interval 2 year);
```

```
+-----------------------------------+-----------------------------------------------------------------------------
    ↪
| array_range(0, 20, 2)               | array_range_year_unit(cast('2019-05-15 12:00:00' as
    ↪ DATETIMEV2(0)), cast('2022-05-17 12:00:00' as DATETIMEV2(0)), 2) |
+-----------------------------------+-----------------------------------------------------------------------------
    ↪
| [0, 2, 4, 6, 8, 10, 12, 14, 16, 18] | ["2019-05-15 12:00:00", "2021-05-15 12:00:00"]
    ↪                                                                    |
+-----------------------------------+-----------------------------------------------------------------------------
    ↪
```

7.2.1.8.34 ARRAY_REMOVE

描述

移除数组中所有的指定元素

语法

```
ARRAY_REMOVE(<arr>, <val>)
```

参数

参数	说明
<arr>	对应数组
<val>	指定元素

返回值

返回移除所有的指定元素后的数组，如果输入参数为 NULL，则返回 NULL

举例

```
SELECT ARRAY_REMOVE(['test', NULL, 'value'], 'value');
```

```
+-----------------------------------------------+
| array_remove(['test', NULL, 'value'], 'value') |
+-----------------------------------------------+
| ["test", null]                                |
+-----------------------------------------------+
```

7.2.1.8.35 ARRAY_REPEAT

描述

生成一个包含 n 个重复元素 T 的数组

语法

```
ARRAY_REPEAT(<element>, <n>)
```

参数

参数	说明
<n>	元数个数
<element>	指定元素

返回值

返回一个数组，包含 n 个重复的 element 元素。array_with_constant 与 array_repeat 功能相同，用来兼容 hive 语法格式。

举例

```
SELECT ARRAY_REPEAT("hello", 2),ARRAY_REPEAT(12345, 3);
```

```
+--------------------------+-----------------------+
| array_repeat('hello', 2) | array_repeat(12345, 3) |
+--------------------------+-----------------------+
| ["hello", "hello"]       | [12345, 12345, 12345]  |
+--------------------------+-----------------------+
```

7.2.1.8.36 ARRAY_REVERSE_SORT

描述

将数组中的元素降序排列

语法

```
ARRAY_REVERSE_SORT(<arr>)
```

参数

参数	说明
<arr>	对应数组

返回值

返回按降序排列后的数组，如果输入数组为 NULL，则返回 NULL。如果数组元素包含 NULL, 则输出的排序数组会将 NULL 放在最后面。

举例

```
SELECT ARRAY_REVERSE_SORT([1, 2, 3, 6]),ARRAY_REVERSE_SORT([1, 4, 3, 5, NULL]),ARRAY_REVERSE_SORT
    ↪ ([NULL]);
```

```
+---------------------------------+----------------------------------------+-------------------------
    ↪
| array_reverse_sort([1, 2, 3, 6]) | array_reverse_sort([1, 4, 3, 5, NULL]) | array_reverse_sort
    ↪ ([NULL]) |
+---------------------------------+----------------------------------------+-------------------------
    ↪
| [6, 3, 2, 1]                     | [5, 4, 3, 1, null]                     | [null]
    ↪                      |
+---------------------------------+----------------------------------------+-------------------------
    ↪
```

7.2.1.8.37 ARRAY_REVERSE_SPLIT

描述

1. 传入两个长度相等的 ARRAY 且第二个为 Array<Boolean>，按照 cond 中为 true 的位置右侧作为分割点，分割 arr 数组。
2. 高阶函数，传入一个 lambda 表达式和至少一个 arr，则按照 lambda 表达式运算得到的 Array<Boolean> 结果，其中为 true 的位置右侧作为分割点，分割 arg0 。

语法

```
ARRAY_REVERSE_SPLIT(<arr>, <cond>)
ARRAY_REVERSE_SPLIT(<lambda>, <arr> [, ...])
```

参数

参数	说明
< ↪ lambda ↪ >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr ↪ >	ARRAY 数组

返回值

返回一个 ARRAY 类型的结果，其中按照对应条件分割后的数组

举例

```
select array_reverse_split([1,2,3,4,5], [1,0,1,0,0]);
```

```
+------------------------------------------------------------------------------+
| array_reverse_split([1, 2, 3, 4, 5], cast([1, 0, 1, 0, 0] as ARRAY<BOOLEAN>)) |
+------------------------------------------------------------------------------+
```

```
| [[1], [2, 3], [4, 5]]                                                          |
+--------------------------------------------------------------------------------+
```

```
select array_reverse_split((x,y)->y, [1,2,3,4,5], [1,0,0,0,0]);
```

```
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| array_reverse_split([1, 2, 3, 4, 5], cast(array_map((x, y) -> y, [1, 2, 3, 4, 5], [1, 0, 0, 0,
    ↪ 0]) as ARRAY<BOOLEAN>)) |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| [[1], [2, 3, 4, 5]]
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
select array_reverse_split((x,y)->(y+1), ['a', 'b', 'c', 'd'], [-1, -1, 0, -1]);
```

```
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| array_reverse_split(['a', 'b', 'c', 'd'], cast(array_map((x, y) -> (y + 1), ['a', 'b', 'c', 'd
    ↪ '], [-1, -1, 0, -1]) as ARRAY<BOOLEAN>)) |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| [["a", "b", "c"], ["d"]]
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
select array_reverse_split(x->(year(x)>2013),["2020-12-12", "2013-12-12", "2015-12-12", null]);
```

```
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| array_reverse_split(['2020-12-12', '2013-12-12', '2015-12-12', NULL], array_map(x -> (year(cast
    ↪ (x as DATEV2)) > 2013), ['2020-12-12', '2013-12-12', '2015-12-12', NULL])) |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| [["2020-12-12"], ["2013-12-12", "2015-12-12"], [null]]
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.8.38 ARRAY_SHUFFLE

描述

将数组中元素进行随机排列

别名

- SHUFFLE

语法

```
ARRAY_SHUFFLE(<array>, <seed>)
```

参数

参数	说明
<array>	要进行随机排列的数组
<seed>	可选参数，是设定伪随机数生成器用于生成伪随机数的初始数值

返回值

将数组中元素进行随机排列。其中，参数 array1 为要进行随机排列的数组，可选参数 seed 是设定伪随机数生成器用于生成伪随机数的初始数值。shuffle 与 array_shuffle 功能相同。

举例

```
SELECT ARRAY_SHUFFLE([1, 2, 3, 6]),ARRAY_SHUFFLE([1, 4, 3, 5, NULL],1);
```

```
+-----------------------------+-------------------------------------+
| array_shuffle([1, 2, 3, 6]) | array_shuffle([1, 4, 3, 5, NULL], 1) |
+-----------------------------+-------------------------------------+
| [2, 6, 3, 1]                | [4, 1, 3, 5, null]                   |
+-----------------------------+-------------------------------------+
```

7.2.1.8.39 ARRAY_SIZE

描述

计算数组中元素的数量

别名

- SIZE
- CARDINALITY

语法

```
ARRAY_SIZE(<arr>)
```

参数

参数	说明
<arr>	待计算的数组

返回值

返回数组中元素数量，如果输入数组为 NULL，则返回 NULL

举例

```
SELECT ARRAY_SIZE(['a', 'b', 'c']),ARRAY_SIZE([NULL]),ARRAY_SIZE([]);
```

```
+-----------------------------+---------------------+-----------------+
| cardinality(['a', 'b', 'c']) | cardinality([NULL]) | cardinality([]) |
+-----------------------------+---------------------+-----------------+
|                           3 |                   1 |               0 |
+-----------------------------+---------------------+-----------------+
```

7.2.1.8.40 ARRAY_SLICE

描述

指定起始位置和长度，从一个数组中提取一部分元素，形成一个新的子数组

语法

```
ARRAY_SLICE(<arr>, <off>, <len>)
```

参数

参数	说明
<arr>	对应数组

参数	说明
< ↪ off ↪ > ↪	起始位置，如果 off 是正数，则表示从左侧开始的偏移量，如果 off 是负数，则表示从右侧开始的偏移量，当指定的 off 不在数

参数	说明
< ↪ len ↪ > ↪	提前长度，如果 len 是负数，则表示长度为 0

返回值

返回一个子数组，包含所有从指定位置开始的指定长度的元素，如果输入参数为 NULL，则返回 NULL

举例

```
SELECT ARRAY_SLICE([1, 2, 3, 6],2,3),ARRAY_SLICE([1, 4, 3, 5, NULL],-2,1),ARRAY_SLICE([1, 3,
    ↪ 5],0);
```

```
+---------------------------------+-----------------------------------------+--------------------------+
    ↪
| array_slice([1, 2, 3, 6], 2, 3) | array_slice([1, 4, 3, 5, NULL], -2, 1) | array_slice([1, 3,
    ↪ 5], 0) |
+---------------------------------+-----------------------------------------+--------------------------+
    ↪
| [2, 3, 6]                       | [5]                                     | []
    ↪                          |
+---------------------------------+-----------------------------------------+--------------------------+
    ↪
```

7.2.1.8.41 ARRAY_SORT

描述

将数组中的元素升序排列

语法

```
ARRAY_SORT(<arr>)
```

参数

参数	说明
<arr>	对应数组

返回值

返回按升序排列后的数组，如果输入数组为 NULL，则返回 NULL。如果数组元素包含 NULL, 则输出的排序数组会将 NULL 放在最前面。

举例

```
SELECT ARRAY_SORT([1, 2, 3, 6]),ARRAY_SORT([1, 4, 3, 5, NULL]),ARRAY_SORT([NULL]);
```

```
+--------------------------+--------------------------------+--------------------+
| array_sort([1, 2, 3, 6]) | array_sort([1, 4, 3, 5, NULL]) | array_sort([NULL]) |
+--------------------------+--------------------------------+--------------------+
| [1, 2, 3, 6]             | [null, 1, 3, 4, 5]             | [null]             |
+--------------------------+--------------------------------+--------------------+
```

7.2.1.8.42 ARRAY_SORTBY

描述

首先将 key 列升序排列，然后将 src 列按此顺序排序后的对应列做为结果返回; 如果输入数组 src 为 NULL，则返回 NULL。如果输入数组 key 为 NULL，则直接返回 src 数组。如果输入数组 key 元素包含 NULL, 则输出的排序数组会将 NULL 放在最前面。

语法

```
ARRAY_SORTBY(<src>, <key>)
ARRAY_SORTBY(<lambda>, <arr> [, ...])
```

参数

参数	说明
< lambda 表达式 >	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr >	ARRAY 数组

返回值

返回一个排序后的 ARRAY 类型的结果

举例

```
select array_sortby(['a','b','c'],[3,2,1]);
```

```
+-------------------------------------------------+
| array_sortby(ARRAY('a', 'b', 'c'), ARRAY(3, 2, 1)) |
+-------------------------------------------------+
```

```
| ['c', 'b', 'a']                                    |
+----------------------------------------------------+
```

```
select array_sortby([1,2,3,4,5],[10,5,1,20,80]);
```

```
+------------------------------------------------------------+
| array_sortby(ARRAY(1, 2, 3, 4, 5), ARRAY(10, 5, 1, 20, 80)) |
+------------------------------------------------------------+
| [3, 2, 1, 4, 5]                                            |
+------------------------------------------------------------+
```

```
select *,array_sortby(c_array1,c_array2) from test_array_sortby order by id;
```

```
+------+-----------------+-------------------------+--------------------------------------+
| id   | c_array1        | c_array2                | array_sortby(`c_array1`, `c_array2`) |
+------+-----------------+-------------------------+--------------------------------------+
|    0 | NULL            | [2]                     | NULL                                 |
|    1 | [1, 2, 3, 4, 5] | [10, 20, -40, 80, -100] | [5, 3, 1, 2, 4]                      |
|    2 | [6, 7, 8]       | [10, 12, 13]            | [6, 7, 8]                            |
|    3 | [1]             | [-100]                  | [1]                                  |
|    4 | NULL            | NULL                    | NULL                                 |
|    5 | [3]             | NULL                    | [3]                                  |
|    6 | [1, 2]          | [2, 1]                  | [2, 1]                               |
|    7 | [NULL]          | [NULL]                  | [NULL]                               |
|    8 | [1, 2, 3]       | [3, 2, 1]               | [3, 2, 1]                            |
+------+-----------------+-------------------------+--------------------------------------+
```

```
select *, array_map((x,y)->(y+x),c_array1,c_array2) as arr_sum,array_sortby((x,y)->(y+x),c_array1
    ↪ ,c_array2) as arr_sort from array_test2;
```

```
+------+-----------------+--------------+----------------+-----------------+
| id   | c_array1        | c_array2     | arr_sum        | arr_sort        |
+------+-----------------+--------------+----------------+-----------------+
|    1 | [1, 2, 3]       | [10, 11, 12] | [11, 13, 15]   | [1, 2, 3]       |
|    2 | [4, 3, 5]       | [10, 20, 30] | [14, 23, 35]   | [4, 3, 5]       |
|    3 | [-40, 30, -100] | [30, 10, 20] | [-10, 40, -80] | [-100, -40, 30] |
+------+-----------------+--------------+----------------+-----------------+
```

7.2.1.8.43 ARRAY_SPLIT

描述

1. 传入两个长度相等的 ARRAY 且第二个为 Array<Boolean>，则按照 cond 中为 true 的位置左侧作为分割点，分割 arr 。

2. 高阶函数，传入一个 lambda 表达式和至少一个 ARRAY arr，则按照 lambda 表达式运算得到的 Array<Boolean> 结果，其中为 true 的位置左侧作为分割点，分割 arr。

语法

```
ARRAY_SPLIT(<arr>, <cond>)
ARRAY_SPLIT(<lambda>, arr [, ...])
```

参数

参数	说明
<lambda>	lambda 表达式，表达式中输入的参数为 1 个或多个，必须和后面的输入 array 列数量一致。在 lambda 中可以执行合法的标量函数，不支持聚合函数等。
<arr>	ARRAY 数组

返回值

返回一个 ARRAY 类型的结果，其中按照对应条件分割后的数组

举例

```
select array_split([1,2,3,4,5], [1,0,1,0,0]);
```

```
+-----------------------------------------------------------------------+
| array_split([1, 2, 3, 4, 5], cast([1, 0, 1, 0, 0] as ARRAY<BOOLEAN>)) |
+-----------------------------------------------------------------------+
| [[1, 2], [3, 4, 5]]                                                   |
+-----------------------------------------------------------------------+
```

```
select array_split((x,y)->y, [1,2,3,4,5], [1,0,0,0,0]);
```

```
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| array_split([1, 2, 3, 4, 5], cast(array_map((x, y) -> y, [1, 2, 3, 4, 5], [1, 0, 0, 0, 0]) as
    ↪ ARRAY<BOOLEAN>)) |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| [[1, 2, 3, 4, 5]]
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
```

```
select array_split((x,y)->(y+1), ['a', 'b', 'c', 'd'], [-1, -1, 0, -1]);
```

```
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| array_split(['a', 'b', 'c', 'd'], cast(array_map((x, y) -> (y + 1), ['a', 'b', 'c', 'd'], [-1,
    ↪ -1, 0, -1]) as ARRAY<BOOLEAN>)) |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| [["a", "b"], ["c", "d"]]
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
```

```
select array_split(x->(year(x)>2013),["2020-12-12", "2013-12-12", "2015-12-12", null]);
```

```
+-----------------------------------------------------------------------------------------------------------------------
    ↪
```

```
| array_split(['2020-12-12', '2013-12-12', '2015-12-12', NULL], array_map(x -> (year(cast(x as
    ↪ DATEV2)) > 2013), ['2020-12-12', '2013-12-12', '2015-12-12', NULL])) |
+-----------------------------------------------------------------------------------------------------------------
    ↪
| [["2020-12-12", "2013-12-12"], ["2015-12-12"], [null]]
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.8.44 ARRAY_SUM

描述

计算数组中所有元素之和

语法

```
ARRAY_SUM(<src>)
```

参数

参数	说明
<src>	对应数组

返回值

返回数组中所有元素之和，数组中的 NULL 值会被跳过。空数组以及元素全为 NULL 值的数组，结果返回 NULL 值。

举例

```
SELECT ARRAY_SUM([1, 2, 3, 6]),ARRAY_SUM([1, 4, 3, 5, NULL]),ARRAY_SUM([NULL]);
```

```
+-------------------------+-------------------------------+----------------------------------------+
    ↪
| array_sum([1, 2, 3, 6]) | array_sum([1, 4, 3, 5, NULL]) | array_sum(cast([NULL] as ARRAY<
    ↪ BOOLEAN>)) |
+-------------------------+-------------------------------+----------------------------------------+
    ↪
|                      12 |                            13 |
    ↪ NULL |
+-------------------------+-------------------------------+----------------------------------------+
    ↪
```

7.2.1.8.45 ARRAY_UNION

描述

将多个数组进行合并，不包含重复元素，生成一个新数组

语法

```
ARRAY_UNION(<array>, <array> [, ... ])
```

参数

参数	说明
<array>	待合并的数组

返回值

返回一个数组，包含 array1 和 array2 的并集中的所有元素，不包含重复项，如果输入参数为 NULL，则返回 NULL

举例

```
SELECT ARRAY_UNION([1, 2, 3, 6],[1, 2, 5]),ARRAY_UNION([1, 4, 3, 5, NULL],[1,6,10]);
```

```
+-------------------------------------+-----------------------------------------+
| array_union([1, 2, 3, 6], [1, 2, 5]) | array_union([1, 4, 3, 5, NULL], [1, 6, 10]) |
+-------------------------------------+-----------------------------------------+
| [3, 2, 1, 6, 5]                     | [null, 10, 3, 1, 6, 4, 5]                   |
+-------------------------------------+-----------------------------------------+
```

7.2.1.8.46 ARRAY_WITH_CONSTANT

描述

生成一个包含 n 个重复元素 element 的数组

语法

```
ARRAY_WITH_CONSTANT(<n>, <element>)
```

参数

参数	说明
<n>	元数个数
<element>	指定元素

返回值

返回一个数组，包含 n 个重复的 element 元素。array_repeat 与 array_with_constant 功能相同，用来兼容 hive 语法格式。

举例

```
SELECT ARRAY_WITH_CONSTANT(2, "hello"),ARRAY_WITH_CONSTANT(3, 12345);
```

```
+---------------------------------+-------------------------------+
| array_with_constant(2, 'hello') | array_with_constant(3, 12345) |
+---------------------------------+-------------------------------+
| ["hello", "hello"]              | [12345, 12345, 12345]         |
+---------------------------------+-------------------------------+
```

7.2.1.8.47 ARRAY_ZIP

描述

将所有数组合并成一个单一的数组。结果数组包含源数组中按参数列表顺序分组的相应元素。

语法

```
ARRAY_ZIP(<array>[, <array> ])
```

参数

参数	说明
<array>	输入数组

返回值

返回来自源数组的元素分组成结构体的数组。结构体中的数据类型与输入数组的类型相同，并按照传递数组的顺序排列。

举例

```
SELECT ARRAY_ZIP(['a', 'b', 'c'], [1, 2, 3]);
```

```
+-----------------------------------------------------+
| array_zip(['a', 'b', 'c'], [1, 2, 3])               |
+-----------------------------------------------------+
| [{"1":"a", "2":1}, {"1":"b", "2":2}, {"1":"c", "2":3}] |
+-----------------------------------------------------+
```

7.2.1.8.48 ARRAYS_OVERLAP

描述

判断 left 和 right 数组中是否包含公共元素

语法

```
ARRAYS_OVERLAP(<left>, <right>)
```

参数

参数	说明
<left>	待判断的数组
<right>	待判断的数组

返回值

返回判断结果：1：left 和 right 数组存在公共元素；0：left 和 right 数组不存在公共元素；NULL：left 或者 right 数组为 NULL；或者 left 和 right 数组中，任意元素为 NULL

举例

```
SELECT ARRAYS_OVERLAP(['a', 'b', 'c'], [1, 2, 'b']);
```

```
+-------------------------------------------------+
| arrays_overlap(['a', 'b', 'c'], ['1', '2', 'b']) |
+-------------------------------------------------+
|                                               1 |
+-------------------------------------------------+
```

7.2.1.8.49 COUNTEQUAL

描述

判断数组中包含 value 元素的个数

语法

```
COUNTEQUAL(<arr>, <value>)
```

参数

参数	说明
<arr>	输入数组
<value>	判断元素

返回值

返回判断的结果如下：num：value 在 array 中的数量；0：value 不存在数组 arr 中；NULL：如果数组为 NULL。

举例

```
SELECT COUNTEQUAL(NULL,1),COUNTEQUAL([1, 2, 3, 'c'],2),COUNTEQUAL([],'b');
```

```
+--------------------+-------------------------------------------------+-------------------------------------------
    ↪
| countequal(NULL, 1) | countequal(['1', '2', '3', 'c'], cast(2 as TEXT)) | countequal(cast([] as
    ↪  ARRAY<TEXT>), 'b') |
```

```
+--------------------+-----------------------------------------------------+-----------------------------------------
    ↪
|               NULL |                                                   1 |
    ↪                                          0 |
+--------------------+-----------------------------------------------------+-----------------------------------------
    ↪
```

7.2.1.9 MAP 函数

7.2.1.9.1 MAP

描述

使用若干组键值对构造一个特定类型的 Map<K, V>

语法

```
MAP( <key1> , <value1> [, <key2>,<value2> ... ])
```

参数

参数	说明
<key>	构造 map 的 key，大小写敏感
<value>	构造 map 的 value

返回值

返回由若干组键值对构造的特定类型 Map<K, V>

举例

```
select map(1, "100", 0.1, 2),map(1, "100", 0.1, 2)[1];
```

```
+-------------------------------------------------------------------------------------+-----------------------
    ↪
| map(cast(1 as DECIMALV3(2, 1)), '100', cast(0.1 as DECIMALV3(2, 1)), cast(2 as TEXT)) | element
    ↪ _at(map(cast(1 as DECIMALV3(2, 1)), '100', cast(0.1 as DECIMALV3(2, 1)), cast(2 as TEXT))
    ↪ , cast(1 as DECIMALV3(2, 1))) |
+-------------------------------------------------------------------------------------+-----------------------
    ↪
| {1.0:"100", 0.1:"2"}                                                                | 100
    ↪
    ↪ |
+-------------------------------------------------------------------------------------+-----------------------
    ↪
```

7.2.1.9.2 MAP_CONTAINS_KEY

描述

判断给定 map 中是否包含特定的键 key

语法

```
MAP_CONTAINS_KEY(<map>, <key>)
```

参数

参数	说明
<map>	输入的 map 内容
<key>	需要检索的 key

返回值

判断给定 map 中是否包含特定的键 key, 存在返回 1 , 不存在返回 0。

举例

```
select map_contains_key(map(null, 1, 2, null), null),map_contains_key(map(1, "100", 0.1, 2),
    ↪ 0.11);
```

```
+-------------------------------------------+-------------------------------------------------------------------
    ↪
| map_contains_key(map(NULL, 1, 2, NULL), NULL) | map_contains_key(cast(map(cast(1 as DECIMALV3
    ↪ (2, 1)), '100', cast(0.1 as DECIMALV3(2, 1)), cast(2 as TEXT)) as MAP<DECIMALV3(3, 2),
    ↪ VARCHAR(3)>), cast(0.11 as DECIMALV3(3, 2))) |
+-------------------------------------------+-------------------------------------------------------------------
    ↪
|                                         1 |
    ↪
    ↪ 0 |
+-------------------------------------------+-------------------------------------------------------------------
    ↪
```

7.2.1.9.3 MAP_CONTAINS_VALUE

描述

判断给定 map 中是否包含特定的值 value

语法

```
MAP_CONTAINS_VALUE(<map>, <value>)
```

参数

参数	说明
<map>	输入的 map 内容
<value>	需要检索的 value

返回值

判断给定 map 中是否包含特定的值 value, 存在返回 1 , 不存在返回 0。

举例

```
select map_contains_value(map(null, 1, 2, null), null),map_contains_value(map(1, "100", 0.1, 2),
    ↪ 101);
```

```
+-------------------------------------------------+-----------------------------------------------------------------
    ↪
| map_contains_value(map(NULL, 1, 2, NULL), NULL) | map_contains_value(map(cast(1 as DECIMALV3(2,
    ↪  1)), '100', cast(0.1 as DECIMALV3(2, 1)), cast(2 as TEXT)), cast(101 as VARCHAR(3))) |
+-------------------------------------------------+-----------------------------------------------------------------
    ↪
|                                               1 |
    ↪
    ↪ 0 |
+-------------------------------------------------+-----------------------------------------------------------------
    ↪
```

7.2.1.9.4 MAP_KEYS

描述

将给定 map 的键提取成一个对应类型的 ARRAY

语法

```
MAP_KEYS(<map>)
```

参数

参数	说明
<map>	输入的 map 内容

返回值

将给定 map 的键提取成一个对应类型的 ARRAY

举例

```
select map_keys(map()),map_keys(map(1, "100", 0.1, 2));
```

```
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
| map_keys(map()) | map_keys(map(cast(1 as DECIMALV3(2, 1)), '100', cast(0.1 as DECIMALV3(2, 1)),
    ↪  cast(2 as TEXT))) |
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
| []              | [1.0, 0.1]
    ↪                                                                                 |
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
```

7.2.1.9.5 MAP_SIZE

描述

计算 Map 中元素的个数

语法

```
MAP_SIZE(<map>)
```

参数

参数	说明
<map>	输入的 map 内容

返回值

返回 Map 中元素的个数

举例

```
select map_size(map()),map_size(map(1, "100", 0.1, 2));
```

```
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
| map_size(map()) | map_size(map(cast(1 as DECIMALV3(2, 1)), '100', cast(0.1 as DECIMALV3(2, 1)),
    ↪  cast(2 as TEXT))) |
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
|               0 |
    ↪
    ↪ 2 |
+-----------------+-----------------------------------------------------------------------------------------------
    ↪
```

7.2.1.9.6 MAP_VALUES

描述

将给定 map 的值提取成一个对应类型的 ARRAY

语法

```
MAP_VALUES(<map>)
```

参数

参数	说明
<map>	输入的 map 内容

返回值

将给定 map 的值提取成一个对应类型的 ARRAY

举例

```
select map_values(map()),map_values(map(1, "100", 0.1, 2));
```

```
+------------------+-----------------------------------------------------------------------------------------------
    ↪
| map_values(map()) | map_values(map(cast(1 as DECIMALV3(2, 1)), '100', cast(0.1 as DECIMALV3(2,
    ↪ 1)), cast(2 as TEXT))) |
+------------------+-----------------------------------------------------------------------------------------------
    ↪
| []                | ["100", "2"]
    ↪                                                                                 |
+------------------+-----------------------------------------------------------------------------------------------
    ↪
```

7.2.1.9.7 STR_TO_MAP

描述

将字符串转换为 Map<String, String> 类型。

> 该函数自 3.0.6 版本开始支持.

语法

```
STR_TO_MAP(<str> [, <pair_delimiter> [, <key_value_delimiter>]])
```

参数

参数	说明
<str>	要转换为 map 的字符串
<pair_delimiter>	字符串中键值对的分割符，默认为 ,
<key_value_delimiter>	字符串中键和值的分割符，默认为 :

返回值

返回从字符串构造的 Map<String, String>。

示例

```
select str_to_map('a=1&b=2&c=3', '&', '=') as map1, str_to_map('x:10|y:20|z:30', '|', ':') as
    ↪ map2;
```

```
+-----------------------------+--------------------------------+
| map1                        | map2                           |
+-----------------------------+--------------------------------+
| {"a":"1", "b":"2", "c":"3"} | {"x":"10", "y":"20", "z":"30"} |
+-----------------------------+--------------------------------+
```

7.2.1.10 STRUCT 函数

7.2.1.10.1 NAMED_STRUCT

描述

根据给定的字符串和值构造并返回 struct , 注意事项：

- 参数个数必须为非 0 偶数，奇数位是 field 的名字，必须为常量字符串，偶数位是 field 的值，可以是多列或常量

语法

```
NAMED_STRUCT( <field_name> , <filed_value> [ , <field_name> , <filed_value> ... ] )
```

参数

参数	说明
<field_name>	构造 struct 的奇数位输入内容，field 的名字
<filed_value>	构造 struct 的偶数位输入内容，field 的值，可以是多列或常量

返回值

根据给定的字符串和值构造并返回 struct

举例

```
select named_struct('f1', 1, 'f2', 'a', 'f3', "abc"),named_struct('a', null, 'b', "v");
```

```
+-----------------------------------------------+----------------------------------+
| named_struct('f1', 1, 'f2', 'a', 'f3', 'abc') | named_struct('a', NULL, 'b', 'v') |
+-----------------------------------------------+----------------------------------+
| {"f1":1, "f2":"a", "f3":"abc"}                | {"a":null, "b":"v"}               |
+-----------------------------------------------+----------------------------------+
```

7.2.1.10.2 STRUCT

描述

根据给定的值构造并返回 struct，参数可以是多列或常量

语法

```
STRUCT( <expr1> [ , <expr2> ... ] )
```

参数

参数	说明
<expr>	构造 struct 的输入内容

返回值

根据给定的值构造并返回 struct，参数可以是多列或常量

举例

```
select struct(1, 'a', "abc"),struct(null, 1, null),struct(cast('2023-03-16' as datetime));
```

```
+--------------------------------------+--------------------------------------+-------------------------------------
    ↪
| struct(1, 'a', 'abc')                | struct(NULL, 1, NULL)                | struct(cast
    ↪ ('2023-03-16' as DATETIMEV2(0))) |
+--------------------------------------+--------------------------------------+-------------------------------------
    ↪
| {"col1":1, "col2":"a", "col3":"abc"} | {"col1":null, "col2":1, "col3":null} | {"col1
    ↪ ":"2023-03-16 00:00:00"}              |
+--------------------------------------+--------------------------------------+-------------------------------------
    ↪
```

7.2.1.10.3 STRUCT_ELEMENT

描述

返回 struct 数据列内的某一 field

语法

```
STRUCT_ELEMENT( <struct>, '<filed_location>/<filed_name>')
```

参数

参数	说明
<struct>	输入的 struct 列，如果是 null，则返回 null
<filed_location>	field 的位置，起始位置从 1 开始，仅支持常量
<filed_name>	field 的名字，仅支持常量，大小写敏感

返回值

返回指定的 field 列，类型为任意类型

举例

```
select struct_element(named_struct('f1', 1, 'f2', 'a'), 'f2'),struct_element(named_struct('f1',
    ↪ 1, 'f2', 'a'), 1);
```

```
+-----------------------------------------------------+-----------------------------------------------------+
    ↪
| struct_element(named_struct('f1', 1, 'f2', 'a'), 'f2') | struct_element(named_struct('f1', 1, '
    ↪ f2', 'a'), 1) |
+-----------------------------------------------------+-----------------------------------------------------+
    ↪
| a                                                   |
    ↪                                                   1 |
+-----------------------------------------------------+-----------------------------------------------------+
    ↪
```

```
select struct_col, struct_element(struct_col, 'f1') from test_struct;
```

```
+-----------------------------------------------+-------------------------------------+
| struct_col                                    | struct_element(`struct_col `, 'f1') |
+-----------------------------------------------+-------------------------------------+
| {1, 2, 3, 4, 5}                               |                                   1 |
| {1, 1000, 10000000, 100000000000, 100000000000} |                                   1 |
| {5, 4, 3, 2, 1}                               |                                   5 |
| NULL                                          |                                NULL |
| {1, NULL, 3, NULL, 5}                         |                                   1 |
+-----------------------------------------------+-------------------------------------+
```

7.2.1.11 JSON 函数

7.2.1.11.1 GET_JSON_BIGINT

描述

解析并获取 json 字符串内指定路径的整型 (BIGINT) 内容。

语法

```
GET_JSON_BIGINT( <json_str>, <json_path>)
```

必选参数

参数	描述
<json_str>	需要从中提取数据的 JSON 字符串。
<json_path>	JSON 路径，指定字段的位置。路径可以使用点号表示法。

注意事项

其中 <json_str> 必须以 $ 符号作为开头，使用. 作为路径分割符。如果路径中包含. ，则可以使用双引号包围。使用 [ ] 表示数组下标，从 0 开始。 path 的内容不能包含 “, [和]。如果 <json_str> 格式不对，或 <json_path> 格式不对，或无法找到匹配项，则返回 NULL。

另外，推荐使用 jsonb 类型和 jsonb_extract_XXX 函数实现同样的功能。

特殊情况处理如下： - 如果 <json_path> 指定的字段在 JSON 中不存在，返回 NULL - 如果 <json_path> 指定的字段在 JSON 中的实际类型和 json_extract_t 指定的类型不一致，如果能无损转换成指定类型返回指定类型 t，如果不能则返回 NULL

返回值

- 返回路径指向字段的 BIGINT 值。
- 如果指定路径没有找到对应的字段，或者字段值无法转换为 BIGINT 类型，返回 NULL。

示例

1. 获取 key 为 “k1” 的 value

```
SELECT get_json_bigint('{"k1":1, "k2":"2"}', "$.k1");
```

```
+-----------------------------------------------+
| get_json_bigint('{"k1":1, "k2":"2"}', '$.k1') |
+-----------------------------------------------+
|                                             1 |
+-----------------------------------------------+
```

2. 获取 key 为 “my.key” 的数组中第二个元素

```
SELECT get_json_bigint('{"k1":"v1", "my.key":[1, 1678708107000, 3]}', '$."my.key"[1]');
```

```
+-----------------------------------------------------------------------------------+
| get_json_bigint('{"k1":"v1", "my.key":[1, 1678708107000, 3]}', '$."my.key"[1]') |
+-----------------------------------------------------------------------------------+
|                                                                     1678708107000 |
+-----------------------------------------------------------------------------------+
```

3. 获取二级路径为 k1.key -> k2 的数组中，第一个元素 “ ‘sql

SELECT get_json_bigint(‘{ “k1.key” :{ “k2” :[1678708107000, 2]}}’ , ‘$. “k1.key” .k2[0]’);

“ ‘

```
+-----------------------------------------------------------------------------------+
| get_json_bigint('{"k1.key":{"k2":[1678708107000, 2]}}', '$."k1.key".k2[0]') |
+-----------------------------------------------------------------------------------+
|                                                                     1678708107000 |
+-----------------------------------------------------------------------------------+
```

7.2.1.11.2 GET_JSON_DOUBLE

描述

函数用于从 JSON 文档中提取一个字段的值并将其转换为 DOUBLE 类型。该函数会返回指定路径上的字段值，如果该值无法转换为 DOUBLE 类型，或者路径指向的字段不存在，则返回 NULL。

语法

```
DOUBLE GET_JSON_DOUBLE( <json_str>, <json_path>)
```

必选参数

参数	描述
<json_str>	需要从中提取数据的 JSON 字符串。
<json_path>	JSON 路径，指定字段的位置。路径可以使用点号表示法。

返回值

- 返回路径指向字段的 DOUBLE 值。
- 如果指定路径没有找到对应的字段，或者字段值无法转换为 DOUBLE 类型，返回 NULL。

注意事项

解析并获取 json 字符串内指定路径的浮点型内容。其中 <json_path> 必须以 $ 符号作为开头，使用. 作为路径分割符。如果路径中包含. ，则可以使用双引号包围。使用 [] 表示数组下标，从 0 开始。path 的内容不能包含 “, [和]。如果 <json_str> 格式不对，或 json_path 格式不对，或无法找到匹配项，则返回 NULL。

另外，推荐使用 jsonb 类型和 jsonb_extract_XXX 函数实现同样的功能。

特殊情况处理如下：- 如果 json_path 指定的字段在 JSON 中不存在，返回 NULL - 如果 json_path 指定的字段在 JSON 中的实际类型和 json_extract_t 指定的类型不一致，如果能无损转换成指定类型返回指定类型 t，如果不能则返回 NULL

示例

1. 获取 key 为 “k1” 的 value

```
SELECT get_json_double('{"k1":1.3, "k2":"2"}', "$.k1");
```

```
+-----------------------------------------------+
| get_json_double('{"k1":1.3, "k2":"2"}', '$.k1') |
+-----------------------------------------------+
|                                           1.3 |
+-----------------------------------------------+
```

2. 获取 key 为 “my.key” 的数组中第二个元素

```
SELECT get_json_double('{"k1":"v1", "my.key":[1.1, 2.2, 3.3]}', '$."my.key"[1]');
```

```
+-------------------------------------------------------------------------+
| get_json_double('{"k1":"v1", "my.key":[1.1, 2.2, 3.3]}', '$."my.key"[1]') |
+-------------------------------------------------------------------------+
|                                                                     2.2 |
+-------------------------------------------------------------------------+
```

3. 获取二级路径为 k1.key -> k2 的数组中，第一个元素

```
SELECT get_json_double('{"k1.key":{"k2":[1.1, 2.2]}}', '$."k1.key".k2[0]');
```

```
+-----------------------------------------------------------------------+
| get_json_double('{"k1.key":{"k2":[1.1, 2.2]}}', '$."k1.key".k2[0]') |
+-----------------------------------------------------------------------+
|                                                                   1.1 |
+-----------------------------------------------------------------------+
```

7.2.1.11.3 GET_JSON_INT

描述

函数用于从 JSON 文档中提取一个字段的值并将其转换为 INT 类型。该函数会返回指定路径上的字段值，如果该值无法转换为 INT 类型，或者路径指向的字段不存在，则返回 NULL。

语法

```
GET_JSON_INT( <json_str>, <json_path>)
```

必选参数

参数	描述
<json_str>	需要从中提取数据的 JSON 字符串。
<json_path>	JSON 路径，指定字段的位置。路径可以使用点号表示法。

注意事项

解析并获取 json 字符串内指定路径的整型内容。其中 <json_path> 必须以 $ 符号作为开头，使用. 作为路径分割符。如果路径中包含. ，则可以使用双引号包围。使用 [] 表示数组下标，从 0 开始。path 的内容不能包含 “, [和]。如果 <json_str> 格式不对，或 <json_path> 格式不对，或无法找到匹配项，则返回 NULL。

另外，推荐使用 jsonb 类型和 jsonb_extract_XXX 函数实现同样的功能。

特殊情况处理如下：- 如果 <json_path> 指定的字段在 JSON 中不存在，返回 NULL - 如果 <json_path> 指定的字段在 JSON 中的实际类型和 json_extract_t 指定的类型不一致，如果能无损转换成指定类型返回指定类型 t，如果不能则返回 NULL

示例

1. 获取 key 为 “k1” 的 value

```
SELECT get_json_int('{"k1":1, "k2":"2"}', "$.k1");
```

```
+-----------------------------------------+
| get_json_int('{"k1":1, "k2":"2"}', '$.k1') |
+-----------------------------------------+
|                                       1 |
+-----------------------------------------+
```

2. 获取 key 为 “my.key” 的数组中第二个元素

```
SELECT get_json_int('{"k1":"v1", "my.key":[1, 2, 3]}', '$."my.key"[1]');
```

```
+-----------------------------------------------------------------+
| get_json_int('{"k1":"v1", "my.key":[1, 2, 3]}', '$."my.key"[1]') |
```

```
+-------------------------------------------------------------------+
|                                                                 2 |
+-------------------------------------------------------------------+
```

3. 获取二级路径为 k1.key -> k2 的数组中，第一个元素

```
SELECT get_json_int('{"k1.key":{"k2":[1, 2]}}', '$."k1.key".k2[0]');
```

```
+-------------------------------------------------------------+
| get_json_int('{"k1.key":{"k2":[1, 2]}}', '$."k1.key".k2[0]') |
+-------------------------------------------------------------+
|                                                           1 |
+-------------------------------------------------------------+
```

7.2.1.11.4 GET_JSON_STRING

描述

函数用于从 JSON 文档中提取一个字段的值，并将其转换为 STRING 类型。该函数返回指定路径上的字段值，如果该值无法转换为字符串，或者路径指向的字段不存在，则返回 NULL。

语法

```
GET_JSON_STRING( <json_str>, <json_path>)
```

必选参数

参数	描述
<json_str>	需要从中提取数据的 JSON 字符串。
<json_path>	JSON 路径，指定字段的位置。路径可以使用点号表示法。

返回值

返回路径指向字段的 STRING 值。如果指定路径没有找到对应的字段，或者字段值无法转换为 STRING 类型，返回 NULL。

注意事项

解析并获取 json 字符串内指定路径的字符串内容。其中 <json_path> 必须以 $ 符号作为开头，使用. 作为路径分割符。如果路径中包含. ，则可以使用双引号包围。使用 [] 表示数组下标，从 0 开始。path 的内容不能包含 “, [和]。如果 <json_str> 格式不对，或 <json_path> 格式不对，或无法找到匹配项，则返回 NULL。

另外，推荐使用 jsonb 类型和 jsonb_extract_XXX 函数实现同样的功能。

特殊情况处理如下：- 如果 <json_path> 指定的字段在 JSON 中不存在，返回 NULL - 如果 <json_path> 指定的字段在 JSON 中的实际类型和 json_extract_t 指定的类型不一致，如果能无损转换成指定类型返回指定类型 t，如果不能则返回 NULL

示例

1. 获取 key 为 “k1” 的 value

```
SELECT get_json_string('{"k1":"v1", "k2":"v2"}', "$.k1");
```

```
+-----------------------------------------------------+
| get_json_string('{"k1":"v1", "k2":"v2"}', '$.k1') |
+-----------------------------------------------------+
| v1                                                  |
+-----------------------------------------------------+
```

2. 获取 key 为 “my.key” 的数组中第二个元素

```
SELECT get_json_string('{"k1":"v1", "my.key":["e1", "e2", "e3"]}', '$."my.key"[1]');
```

```
+-----------------------------------------------------------------------------------+
| get_json_string('{"k1":"v1", "my.key":["e1", "e2", "e3"]}', '$."my.key"[1]') |
+-----------------------------------------------------------------------------------+
| e2                                                                                |
+-----------------------------------------------------------------------------------+
```

3. 获取二级路径为 k1.key -> k2 的数组中，第一个元素

```
SELECT get_json_string('{"k1.key":{"k2":["v1", "v2"]}}', '$."k1.key".k2[0]');
```

```
+-----------------------------------------------------------------------+
| get_json_string('{"k1.key":{"k2":["v1", "v2"]}}', '$."k1.key".k2[0]') |
+-----------------------------------------------------------------------+
| v1                                                                    |
+-----------------------------------------------------------------------+
```

4. 获取数组中，key 为 “k1” 的所有 value

```
SELECT get_json_string('[{"k1":"v1"}, {"k2":"v2"}, {"k1":"v3"}, {"k1":"v4"}]', '$.k1');
```

```
+-----------------------------------------------------------------------------------+
| get_json_string('[{"k1":"v1"}, {"k2":"v2"}, {"k1":"v3"}, {"k1":"v4"}]', '$.k1') |
+-----------------------------------------------------------------------------------+
| ["v1","v3","v4"]                                                                  |
+-----------------------------------------------------------------------------------+
```

7.2.1.11.5 JSON_ARRAY

描述

生成一个包含制定元素的 json 数组，未指定时返回空数组

语法

```
JSON_ARRAY(<a>, ...)
```

参数

参数	描述
<a>, ...	要包含在 JSON 数组中的元素。可以是单个或者多个任意类型的值，包括NULL。如果没有指定元素，则返回一个空

返回值

返回一个包含指定元素的 JSON 数组。特殊情况：* 如果指定的元素为NULL，则返回NULL。

示例

```
select json_array();
```

```
+--------------+
| json_array() |
+--------------+
| []           |
+--------------+
```

```
select json_array(null);
```

```
+--------------------+
| json_array('NULL') |
+--------------------+
| [NULL]             |
+--------------------+
```

```
SELECT json_array(1, "abc", NULL, TRUE, CURTIME());
```

```
+-----------------------------------------------+
| json_array(1, 'abc', 'NULL', TRUE, curtime()) |
+-----------------------------------------------+
| [1, "abc", NULL, TRUE, "10:41:15"]            |
+-----------------------------------------------+
```

```
select json_array("a", null, "c");
```

```
+-----------------------------+
| json_array('a', 'NULL', 'c') |
+-----------------------------+
| ["a", NULL, "c"]            |
+-----------------------------+
```

7.2.1.11.6 JSON_CONTAINS

描述

用于判断一个 JSON 文档是否包含指定的 JSON 元素。如果指定的元素存在于 JSON 文档中，则返回 1，否则返回 0。如果 JSON 文档或查询的元素无效，则返回 NULL。

语法

```
JSON_CONTAINS(<json_str>, <candidate> [, <json_path>])
```

必选参数

参数	描述
<json_str>	需要检查的 JSON 字符串。
<candidate>	用于检查是否包含的 JSON 元素。

可选参数

参数	描述
<json_path>	可选的 JSON 路径，指定检查的 JSON 子文档。如果不提供，默认为根文档。

返回值

- 如果 json_elem 存在于 json_doc 中，则返回 1。
- 如果 json_elem 不存在于 json_doc 中，则返回 0。
- 如果任何参数无效或 JSON 文档格式不正确，则返回 NULL。

示例

```
SET @j = '{"a": 1, "b": 2, "c": {"d": 4}}';
SET @j2 = '1';
SELECT JSON_CONTAINS(@j, @j2, '$.a');
```

```
+-------------------------------+
| JSON_CONTAINS(@j, @j2, '$.a') |
+-------------------------------+
```

```
|                               1 |
+---------------------------------+
```

```
SELECT JSON_CONTAINS(@j, @j2, '$.b');
+---------------------------------+
| JSON_CONTAINS(@j, @j2, '$.b') |
+---------------------------------+
|                               0 |
+---------------------------------+
```

```
SET @j2 = '{"d": 4}';
SELECT JSON_CONTAINS(@j, @j2, '$.a');
```

```
+---------------------------------+
| JSON_CONTAINS(@j, @j2, '$.a') |
+---------------------------------+
|                               0 |
+---------------------------------+
```

```
SELECT JSON_CONTAINS(@j, @j2, '$.c');
```

```
+---------------------------------+
| JSON_CONTAINS(@j, @j2, '$.c') |
+---------------------------------+
|                               1 |
+---------------------------------+
```

```
SELECT json_contains('[1, 2, {"x": 3}]', '1');
+----------------------------------------+
| json_contains('[1, 2, {"x": 3}]', '1') |
+----------------------------------------+
|                                      1 |
+----------------------------------------+
```

7.2.1.11.7 JSON_EXISTS_PATH

描述

用来判断 `json_path` 指定的字段在 JSON 数据中是否存在，如果存在返回 TRUE，不存在返回 FALSE

语法

```
JSON_EXISTS_PATH (<json_str>,  <path>)
```

别名

- JSONB_EXISTS_PATH

参数

参数	描述
<json_str>	要包含在 JSON 数组中的元素。可以是任意类型的值，包括NULL。如果没有指定元素，则返回一个空数组。
<path>	要插入的 JSON 路径。如果是 NULL ，则返回 NULL

返回值

如果存在返回 TRUE，不存在返回 FALSE

举例

```
SELECT JSON_EXISTS_PATH('{"id": 123, "name": "doris"}', '$.name');
```

```
+-----------------------------------------------------------------------------+
| jsonb_exists_path(cast('{"id": 123, "name": "doris"}' as JSON), '$.name') |
+-----------------------------------------------------------------------------+
|                                                                           1 |
+-----------------------------------------------------------------------------+
```

```
SELECT JSON_EXISTS_PATH('{"id": 123, "name": "doris"}', '$.age');
```

```
+-----------------------------------------------------------------------------+
| jsonb_exists_path(cast('{"id": 123, "name": "doris"}' as JSON), '$.age') |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

```
SELECT JSONB_EXISTS_PATH('{"id": 123, "name": "doris"}', '$.age');
```

```
+-----------------------------------------------------------------------------+
| jsonb_exists_path(cast('{"id": 123, "name": "doris"}' as JSON), '$.age') |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

7.2.1.11.8 JSON_EXTRACT

描述

JSON_EXTRACT 是一系列函数，从 JSON 类型的数据中提取 json_path 指定的字段，根据要提取的字段类型不同提供不同的系列函数。* JSON_EXTRACT 对 VARCHAR 类型的 json string 返回 VARCHAR 类型 * JSON_EXTRACT_NO_QUOTES 对 VARCHAR 类型的 json string 返回 VARCHAR 类型, 如果 json 字段值为 string，则将去掉双引号。* JSON_EXTRACT_ISNULL 返回是否为 json null 的 BOOLEAN 类型 * JSON_EXTRACT_BOOL 返回 BOOLEAN 类型 * JSON_EXTRACT_INT 返回 INT 类型 * JSON_EXTRACT_BIGINT 返回 BIGINT 类型 * JSON_EXTRACT_LARGEINT 返回 LARGEINT 类型 * JSON_EXTRACT_DOUBLE 返回 DOUBLE 类型 * JSON_EXTRACT_STRING 返回 STRING 类型

> JSON_EXTRACT_NO_QUOTES 函数自 3.0.6 版本开始支持.

别名

- JSONB_EXTRACT 同 JSON_EXTRACT
- JSON_EXTRACT_NO_QUOTES 同 JSON_EXTRACT_NO_QUOTES
- JSONB_EXTRACT_ISNULL 同 JSON_EXTRACT_ISNULL
- JSONB_EXTRACT_BOOL 同 JSON_EXTRACT_BOOL
- JSONB_EXTRACT_INT 同 JSON_EXTRACT_INT
- JSONB_EXTRACT_BIGINT 同 JSON_EXTRACT_BIGINT
- JSONB_EXTRACT_LARGEINT 同 JSON_EXTRACT_LARGEINT
- JSONB_EXTRACT_DOUBLE 同 JSON_EXTRACT_DOUBLE
- JSONB_EXTRACT_STRING 同 JSON_EXTRACT_STRING

语法

```
JSON_EXTRACT (<json_str>, <path>[, path] ...)
```

```
JSON_EXTRACT_NO_QUOTES (<json_str>, <path>[, path] ...)
```

```
JSON_EXTRACT_ISNULL (<json_str>, <path>)
```

```
JSON_EXTRACT_BOOL (<json_str>, <path>)
```

```
JSON_EXTRACT_INT (<json_str>, <path>)
```

```
JSON_EXTRACT_BIGINT (<json_str>, <path>)
```

```
JSON_EXTRACT_LARGEINT (<json_str>, <path>)
```

```
JSON_EXTRACT_DOUBLE (<json_str>, <path>)
```

```
JSON_EXTRACT_STRING (<json_str>, <path>)
```

别名函数除函数名称之外，语法和用法与上述函数保持一致。

参数

参数	描述
<json_str>	要提取的 JSON 类型的参数或者字段
<path>	要从目标 JSON 中提取目标元素的 JSON 路径。

json_path 的语法如下 * $''jsonroot*'.k1'jsonobjectkey'k1'-key., json_path`SELECT json_extract('k1.a : abc, k2 : 300','. "k1.a
* '[i]' 代表 json array 中下标为 i 的元素 - 获取 json_array 的最后一个元素可以用'$[last]''[last-1]'，以此类推。

返回值

根据要提取的字段类型不同，返回目标 JSON 中指定 JSON_PATH 的数据类型。特殊情况处理如下：* 如果 json_path 指定的字段在 JSON 中不存在，返回 NULL * 如果 json_path 指定的字段在 JSON 中的实际类型和 json_extract_t 指定的类型不一致，如果能无损转换成指定类型返回指定类型 t，如果不能则返回 NULL

示例

```
SELECT json_extract('{"id": 123, "name": "doris"}', '$.id');
```

```
+-----------------------------------------------------+
| json_extract('{"id": 123, "name": "doris"}', '$.id') |
+-----------------------------------------------------+
| 123                                                 |
+-----------------------------------------------------+
```

```
SELECT json_extract('[1, 2, 3]', '$.[1]');
```

```
+-----------------------------------+
| json_extract('[1, 2, 3]', '$.[1]') |
+-----------------------------------+
| 2                                 |
+-----------------------------------+
```

```
SELECT json_extract('{"k1": "v1", "k2": { "k21": 6.6, "k22": [1, 2] } }', '$.k1', '$.k2.k21', '$.
    ↪ k2.k22', '$.k2.k22[1]');
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| json_extract('{"k1": "v1", "k2": { "k21": 6.6, "k22": [1, 2] } }', '$.k1', '$.k2.k21', '$.k2.
    ↪ k22', '$.k2.k22[1]') |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

```
| ["v1",6.6,[1,2],2]
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
SELECT json_extract('{"id": 123, "name": "doris"}', '$.aaa', '$.name');
```

```
+-----------------------------------------------------------------+
| json_extract('{"id": 123, "name": "doris"}', '$.aaa', '$.name') |
+-----------------------------------------------------------------+
| [null,"doris"]                                                  |
+-----------------------------------------------------------------+
```

```
SELECT json_extract_no_quotes('{"id": 123, "name": "doris"}', '$.name');
```

```
+--------------------------------------------------------------------+
| json_extract_no_quotes('{"id": 123, "name": "doris"}', '$.name') |
+--------------------------------------------------------------------+
| doris                                                              |
+--------------------------------------------------------------------+
```

```
SELECT JSON_EXTRACT_ISNULL('{"id": 123, "name": "doris"}', '$.id');
```

```
+-----------------------------------------------------------------------------+
| jsonb_extract_isnull(cast('{"id": 123, "name": "doris"}' as JSON), '$.id') |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

```
SELECT JSON_EXTRACT_BOOL('{"id": 123, "name": "NULL"}', '$.id');
```

```
+--------------------------------------------------------------------------+
| jsonb_extract_bool(cast('{"id": 123, "name": "NULL"}' as JSON), '$.id') |
+--------------------------------------------------------------------------+
|                                                                     NULL |
+--------------------------------------------------------------------------+
```

```
SELECT JSON_EXTRACT_INT('{"id": 123, "name": "NULL"}', '$.id');
```

```
+-------------------------------------------------------------------------+
| jsonb_extract_int(cast('{"id": 123, "name": "NULL"}' as JSON), '$.id') |
+-------------------------------------------------------------------------+
|                                                                     123 |
+-------------------------------------------------------------------------+
```

```
SELECT JSON_EXTRACT_INT('{"id": 123, "name": "doris"}', '$.name');
```

```
+-----------------------------------------------------------------------------+
| jsonb_extract_int(cast('{"id": 123, "name": "doris"}' as JSON), '$.name') |
+-----------------------------------------------------------------------------+
|                                                                        NULL |
+-----------------------------------------------------------------------------+
```

```
SELECT JSON_EXTRACT_STRING('{"id": 123, "name": "doris"}', '$.name');
```

```
+--------------------------------------------------------------------------------+
| jsonb_extract_string(cast('{"id": 123, "name": "doris"}' as JSON), '$.name') |
+--------------------------------------------------------------------------------+
| doris                                                                          |
+--------------------------------------------------------------------------------+
```

7.2.1.11.9 JSON_EXTRACT_BIGINT

7.2.1.11.10 JSON_EXTRACT_BOOL

7.2.1.11.11 JSON_EXTRACT_DOUBLE

7.2.1.11.12 JSON_EXTRACT_INT

7.2.1.11.13 JSON_EXTRACT_ISNULL

7.2.1.11.14 JSON_EXTRACT_LARGEINT

7.2.1.11.15 JSON_EXTRACT_STRING

7.2.1.11.16 JSON_INSERT

描述

JSON_INSERT 函数用于在 JSON 中插入数据并返回结果。

语法

```
JSON_INSERT (<json_str>, <path>,  <val>[, <path>,  <val>, ...])
```

参数

参数	描述
<json_str>	要插入的 JSON 对象。可以是任意类型元素的 JSON 对象，包括NULL，如果没有指定元素，则返回一个空数组。如果 json_str 不是有效的 JSON 或任何 path 参数不是有效的路径表达式或包含了 * 通配符，则会返回错误
<path>	要插入的 JSON 路径。如果是 NULL ，则返回 NULL
<val>	要插入 JSON 的值。如果是 NULL ，则会在对应的位置插入 NULL 的 value 值。

需要注意的是，路径值对按从左到右的顺序进行评估。

如果 JSON 中不存在该路径，则路径值对会添加该值到 JSON 中，如果路径标识某个类型的值，则：

- 对于现有对象中不存在的成员，会将新成员添加到该对象中并与新值相关联。
- 对于现有数组结束后的位置，该数组将扩展为包含新值。如果现有值不是数组，则自动转换为数组，然后再扩展为包含新值的数组。

否则，对于 JSON 中不存在的某个路径的路径值对将被忽略且不会产生任何影响。

返回值

返回一个 JSON 值。

示例

```
select json_insert(null, null, null);
```

```
+---------------------------------+
| json_insert(NULL, NULL, 'NULL') |
+---------------------------------+
| NULL                            |
+---------------------------------+
```

```
select json_insert('{"k": 1}', "$.k", 2);
```

```
+--------------------------------------+
| json_insert('{\"k\": 1}', '$.k', '2') |
+--------------------------------------+
| {"k":1}                              |
+--------------------------------------+
```

```
select json_insert('{"k": 1}', "$.j", 2);
```

```
+--------------------------------------+
| json_insert('{\"k\": 1}', '$.j', '2') |
+--------------------------------------+
| {"k":1,"j":2}                        |
+--------------------------------------+
```

```
select json_insert('{"k": 1}', "$.j", null);
```

```
+-----------------------------------------------+
| json_insert('{"k": 1}', '$.j', 'NULL', '660') |
+-----------------------------------------------+
| {"k":1,"j":null}                              |
+-----------------------------------------------+
```

7.2.1.11.17 JSON_KEYS

描述

用于从 JSON 对象的顶级值中返回键。这些键作为数组返回，或者如果给定了路径参数，则返回所选路径的顶级键。您需要将 JSON 文档作为函数的参数提供。您还可以（可选地）提供第二个参数，以指定 JSON 文档中“顶级”路径从何处开始。其中，json_doc 是 JSON 文档，path 是一个可选参数，用于确定 JSON 文档中“顶级”路径从何处开始。

语法

```
JSON_KEYS(<str> [, <path>])
```

别名

- JSONB_KEYS

必选参数

参数	描述
<str>	需要提取键的 JSON 字符串。

可选参数

参数	描述
<path>	可选的 JSON 路径，指定检查的 JSON 子文档。如果不提供，默认为根文档。

返回值

- 返回 JSON 文档的键名列表（一个 JSON 数组）。
- 如果 str 不是一个有效的 JSON 对象，返回 NULL。
- 如果 JSON 对象没有键，返回一个空数组。

注意事项

- 如果所选对象为空，则结果数组为空。如果顶级值包含嵌套的子对象，返回值不包括这些子对象的键。

示例

```
SELECT JSON_KEYS('{"a": 1, "b": {"c": 30}}');
```

```
+-----------------------------------------------------+
| json_keys(cast('{"a": 1, "b": {"c": 30}}' as JSON)) |
+-----------------------------------------------------+
| ["a", "b"]                                          |
+-----------------------------------------------------+
```

```
SELECT JSON_KEYS('{"a": 1, "b": {"c": 30}}', '$.b');
```

```
+-------------------------------------------------------------+
| json_keys(cast('{"a": 1, "b": {"c": 30}}' as JSON), '$.b') |
+-------------------------------------------------------------+
| ["c"]                                                       |
+-------------------------------------------------------------+
```

```
SELECT JSON_KEYS('{}');
```

```
+-------------------------------+
| json_keys(cast('{}' as JSON)) |
+-------------------------------+
| []                            |
+-------------------------------+
```

```
SELECT JSON_KEYS('[1,2]');
```

```
+---------------------------------+
| json_keys(cast('[1,2]' as JSON)) |
+---------------------------------+
| NULL                            |
+---------------------------------+
```

```
 SELECT JSON_KEYS('[]');
```

```
+------------------------------+
| json_keys(cast('[]' as JSON)) |
+------------------------------+
| NULL                         |
+------------------------------+
```

7.2.1.11.18 JSON_LENGTH

描述

JSON_LENGTH 函数用于返回给定 JSON 文档的长度或元素个数。如果 JSON 文档是一个数组，则返回数组中元素的个数；如果 JSON 文档是一个对象，则返回对象中键值对的个数。如果 JSON 文档为空或无效，返回 NULL。

语法

```
JSON_LENGTH(<json_str> [ , <json_path>])
```

必选参数

参数	描述
<json_str>	需要检查长度的 JSON 字符串。

可选参数

参数	描述
<json_path>	如果指定 path，该 JSON_LENGTH() 函数返回与 JSON 文档中的路径匹配的数据的长度，否则返回 JSON 文档的长度

注意事项

该函数根据以下规则计算 JSON 文档的长度：- 标量的长度为 1。例如： ‘1’，‘ “x” ’，‘true’，‘false’，‘null’ 的长度均为 1。- 数组的长度是数组元素的数量。例如： ‘[1, 2]’ 的长度为 2。- 对象的长度是对象成员的数量。例如： ‘{ “x” : 1}’ 的长度为 1

返回值

- 对于 JSON 数组，返回数组中元素的个数。
- 对于 JSON 对象，返回对象中键值对的个数。
- 对于无效的 JSON 字符串，返回 NULL。
- 对于其他类型 (如字符串、数字、布尔值、 null 等)，返回 NULL。

示例

```
SELECT json_length('{"k1":"v31","k2":300}');
```

```
+-------------------------------------+
| json_length('{"k1":"v31","k2":300}') |
+-------------------------------------+
|                                   2 |
+-------------------------------------+
```

```
SELECT json_length('"abc"');
```

```
+----------------------+
| json_length('"abc"') |
+----------------------+
|                    1 |
+----------------------+
```

```
SELECT json_length('{"x": 1, "y": [1, 2]}', '$.y');
```

```
+-----------------------------------------------+
| json_length('{"x": 1, "y": [1, 2]}', '$.y') |
+-----------------------------------------------+
|                                             2 |
+-----------------------------------------------+
```

7.2.1.11.19 JSON_OBJECT

描述

生成一个或者多个包含指定 Key-Value 对的 json object, 当 Key 值为 NULL 或者传入参数为奇数个时，返回异常错误。

语法

```
JSON_OBJECT (<key>, <value>[,<key>, <value>, ...])
```

参数

参数	描述
<key>	指定生成的 json object 的 Key-Value 中的 Key 值
<value>	指定生成的 json object 的 Key-Value 中的 Value 值

注意事项

- 按照惯例，参数列表由交替的键和值组成。
- Key 按照 JSON 定义强制转换为文本。
- Value 参数按照可以转换为 json 的方式进行转换，现在我们支持 array/struct/map/json 作为值

返回值

返回一个 json object。特殊情况如下：* 如果没有传入参数，返回一个空的 json object。* 如果传入的参数个数为奇数个，返回异常错误。* 如果传入的 Key 为 NULL，返回异常错误。* 如果传入的 Value 为 NULL，返回的 json object 中该 Key-Value 对的 Value 值为 NULL。

示例

```
select json_object();
```

```
+---------------+
| json_object() |
+---------------+
| {}            |
+---------------+
```

```
select json_object('time',curtime());
```

```
+--------------------------------+
| json_object('time', curtime()) |
+--------------------------------+
| {"time": "10:49:18"}           |
+--------------------------------+
```

```
SELECT json_object('id', 87, 'name', 'carrot');
```

```
+---------------------------------------+
| json_object('id', 87, 'name', 'carrot') |
+---------------------------------------+
| {"id": 87, "name": "carrot"}            |
+---------------------------------------+
```

```
select json_object('username',null);
```

```
+--------------------------------+
| json_object('username', 'NULL') |
+--------------------------------+
| {"username": NULL}             |
+--------------------------------+
```

```
select json_object(null,null);
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = json_object key can't be NULL: json_object(NULL)
```

```
-- support array as object value
SELECT json_object('id', 1, 'level', array('"aaa"','"bbb"'));
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| json_object('id', cast(1 as VARCHAR(65533)), 'level', cast(array('"aaa"', '"bbb"') as JSON),
    ↪ '6267') |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| {"id":1,"level":["\"aaa\"","\"bbb\""]}
    ↪                                                                    |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

```
-- support map as object value
SELECT json_object('id', 1, 'level', map('a', 'b', 'c', 'd'));
```

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| json_object('id', cast(1 as VARCHAR(65533)), 'level', cast(map('a', 'b', 'c', 'd') as JSON),
    ↪ '6267') |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| {"id":1,"level":{"a":"b","c":"d"}}
    ↪                                                                |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

```
-- support struct as object value
SELECT json_object('id', 1, 'level', named_struct('name', 'a', 'age', 1));
```

```
+-----------------------------------------------------------------------------------------------------------
    ↪
| json_object('id', cast(1 as VARCHAR(65533)), 'level', cast(named_struct('name', 'a', 'age', 1)
    ↪ as JSON), '6267') |
+-----------------------------------------------------------------------------------------------------------
    ↪
| {"id":1,"level":{"name":"a","age":1}}
    ↪                                                                |
+-----------------------------------------------------------------------------------------------------------
    ↪
```

```
-- support json as object value
SELECT json_object('id', 1, 'level', cast('{\"a\":\"b\"}' as JSON));
```

```
+-----------------------------------------------------------------------------------------+
| json_object('id', cast(1 as VARCHAR(65533)), 'level', cast('{"a":"b"}' as JSON), '6267') |
```

```
+-----------------------------------------------------------------------------------------+
| {"id":1,"level":{"a":"b"}}                                                              |
+-----------------------------------------------------------------------------------------+
```

7.2.1.11.20 JSON_PARSE

描述

将原始 JSON 字符串解析成 JSON 二进制格式。为了满足不同的异常数据处理需求，提供不同的 JSON_PARSE 系列函数，具体如下：* JSON_PARSE 解析 JSON 字符串，当输入的字符串不是合法的 JSON 字符串时，报错。* JSON_PARSE_ERROR_TO_INVALID 解析 JSON 字符串，当输入的字符串不是合法的 JSON 字符串时，返回 NULL。* JSON_PARSE_ERROR_TO_NULL 解析 JSON 字符串，当输入的字符串不是合法的 JSON 字符串时，返回 NULL。* JSON_PARSE_ERROR_TO_VALUE 解析 JSON 字符串，当输入的字符串不是合法的 JSON 字符串时，返回参数 default_json_str 指定的默认值。* JSON_PARSE_NOTNULL 解析 JSON 字符串，当输入的字符串不是合法的 JSON 字符串时，返回 NULL。

别名

- JSONB_PARSE 同 JSON_PARSE
- JSONB_PARSE_ERROR_TO_INVALID 同 JSON_PARSE_ERROR_TO_INVALID
- JSONB_PARSE_ERROR_TO_NULL 同 JSON_PARSE_ERROR_TO_NULL
- JSONB_PARSE_ERROR_TO_VALUE 同 JSON_PARSE_ERROR_TO_VALUE
- JSONB_PARSE_NOTNULL 同 JSON_PARSE_NOTNULL

语法

```
JSON_PARSE (<json_str>)
```

```
JSON_PARSE_ERROR_TO_INVALID (<json_str>)
```

```
JSON_PARSE_ERROR_TO_NULL (<json_str>)
```

```
JSON_PARSE_ERROR_TO_VALUE (<json_str>, <default_json_str>)
```

```
JSONB_PARSE_NOTNULL (<json_str>)
```

参数

参数	描述
<json_str>	要提取的 JSON 类型的参数或者字段
<default_json_str>	当输入的字符串不是合法的 JSON 字符串时，返回参数 default_json_str 指定的默认值。

返回值

- json_str 为 NULL 时，都返回 NULL

- json_str 为非法 JSON 字符串时
- JSON_PARSE 报错
- JSON_PARSE_ERROR_TO_INVALID 返回 NULL
- JSON_PARSE_ERROR_TO_NULL 返回 NULL
- JSON_PARSE_ERROR_TO_VALUE 返回参数 default_json_str 指定的默认值
- JSON_PARSE_NOTNULL 返回 NULL

示例

1. 正常 JSON 字符串解析

```
SELECT json_parse('{"k1":"v31","k2":300}');
```

```
+-------------------------------------+
| json_parse('{"k1":"v31","k2":300}') |
+-------------------------------------+
| {"k1":"v31","k2":300}               |
+-------------------------------------+
```

```
SELECT json_parse_error_to_invalid('{"k1":"v31","k2":300}');
```

```
+-----------------------------------------------------+
| jsonb_parse_error_to_invalid('{"k1":"v31","k2":300}') |
+-----------------------------------------------------+
| {"k1":"v31","k2":300}                               |
+-----------------------------------------------------+
```

```
SELECT json_parse_notnull('{"a":"b"}');
```

```
+---------------------------------+
| jsonb_parse_notnull('{"a":"b"}') |
+---------------------------------+
| {"a":"b"}                       |
+---------------------------------+
```

```
SELECT json_parse_error_to_value('{"k1":"v31","k2":300}','{}');
```

```
+-----------------------------------------------------------+
| jsonb_parse_error_to_value('{"k1":"v31","k2":300}', '{}') |
+-----------------------------------------------------------+
| {"k1":"v31","k2":300}                                     |
+-----------------------------------------------------------+
```

2. 非法 JSON 字符串解析

```
SELECT json_parse('invalid json');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = json parse error: Invalid document:
    ↪ document must be an object or an array for value: invalid json
```

```
SELECT json_parse_error_to_invalid('invalid json');
```

```
+-----------------------------------------------+
| jsonb_parse_error_to_invalid('invalid json') |
+-----------------------------------------------+
| NULL                                          |
+-----------------------------------------------+
```

```
SELECT json_parse_notnull('invalid json');
```

```
+-----------------------------------------+
| jsonb_parse_error_to_null('invalid json') |
+-----------------------------------------+
| NULL                                    |
+-----------------------------------------+
```

```
SELECT json_parse_error_to_value('invalid json', '{}');
```

```
+--------------------------------------------------+
| json_parse_error_to_value('invalid json', '{}') |
+--------------------------------------------------+
| {}                                               |
+--------------------------------------------------+
```

7.2.1.11.21 JSON_PARSE_ERROR_TO_INVALID

7.2.1.11.22 JSON_PARSE_ERROR_TO_NULL

7.2.1.11.23 JSON_PARSE_ERROR_TO_VALUE

7.2.1.11.24 JSON_PARSE_NOTNULL

7.2.1.11.25 JSON_PARSE_NOTNULL_ERROR_TO_INVALID

描述

用于解析 JSON 字符串。如果 JSON 字符串格式无效或发生解析错误，函数会返回一个无效的 JSON 对象（通常是 {}）。该函数的主要作用是确保在出现 JSON 格式错误时，返回一个安全的默认值，避免由于解析错误导致的查询失败。

语法

```
JSON_PARSE_NOTNULL_ERROR_TO_INVALID( <str> )
```

别名

- JSONB_PARSE_NOTNULL_ERROR_TO_INVALID

必选参数

参数	描述
<str>	需要解析的 JSON 字符串。该参数应该是一个有效的字符串类型，包含 JSON 格式的数据。如果 JSON 格式无效，函数会返回一个无效的 JSON 对象。

返回值

返回一个无效的 JSON 对象（通常是 {}）

示例

```
SELECT JSON_PARSE_NOTNULL_ERROR_TO_INVALID('{"name": "Alice", "age": 30}') AS parsed_json;
```

```
+--------------------------+
| parsed_json              |
+--------------------------+
| {"name":"Alice","age":30} |
+--------------------------+
```

7.2.1.11.26 JSON_PARSE_NOTNULL_ERROR_TO_VALUE

描述

函数用于解析 JSON 字符串。如果 JSON 字符串格式无效或发生解析错误，函数会返回用户指定的默认值，而不是返回无效的 JSON 对象。该函数的主要作用是提供一个默认值，用于替代解析错误时的无效结果，确保查询返回合理的值。

语法

```
JSON_PARSE_NOTNULL_ERROR_TO_VALUE(< str >, <default_value>)
```

别名

- JSONB_PARSE_NOTNULL_ERROR_TO_VALUE

必选参数

参数	描述
<str>	需要解析的 JSON 字符串。该参数应为一个有效的 JSON 字符串。如果 JSON 格式无效，函数会返回 default_val
<default_value>	解析错误时返回的默认值。该参数可以是任意类型，用于替代无效的 JSON 格式数据。

返回值

返回一个 JSON 对象。如果输入的 JSON 字符串有效，返回解析后的 JSON 对象；如果无效，返回用户指定的 default_value。

示例

```
SELECT JSON_PARSE_NOTNULL_ERROR_TO_VALUE('{"name": "Alice", "age": 30}', '{"name": "Unknown", "
    ↪ age": 0}') AS parsed_json;
```

```
+-----------------------------------------+
| parsed_json                             |
+-----------------------------------------+
| {"name":"Alice","age":30}               |
+-----------------------------------------+
```

7.2.1.11.27 JSON_PARSE_NULLABLE

描述

JSON_PARSE_NULLABLE 函数用于解析一个 JSON 字符串为有效的 JSON 对象。如果输入字符串无效或为 NULL，它将返回 NULL，而不会抛出错误。

语法

```
JSON_PARSE_NULLABLE( <str> )
```

别名

- JSONB_PARSE_NULLABLE

必选参数

参数	描述
<str>	需要解析的 JSON 格式的输入字符串。

返回值

如果输入字符串是有效的 JSON，返回对应的 JSON 对象。如果输入字符串无效或为 NULL，返回 NULL。

示例

1. 有效的 JSON 字符串：

```
SELECT JSON_PARSE_NULLABLE('{"name": "John", "age": 30}');
```

```
+-----------------------------------------------------+
| JSON_PARSE_NULLABLE('{"name": "John", "age": 30}')  |
+-----------------------------------------------------+
| {"name": "John", "age": 30}                         |
+-----------------------------------------------------+
```

2. 无效的 JSON 字符串：

```
SELECT JSON_PARSE_NULLABLE('{"name": "John", "age": }');
```

```
+-----------------------------------------------------+
| JSON_PARSE_NULLABLE('{"name": "John", "age": }')    |
+-----------------------------------------------------+
| NULL                                                |
+-----------------------------------------------------+
```

7.2.1.11.28 JSON_PARSE_NULLABLE_ERROR_TO_INVALID

描述

JSON_PARSE_NULLABLE_ERROR_TO_INVALID 函数用于解析一个 JSON 字符串为有效的 JSON 对象。如果输入的 JSON 字符串无效，它将返回一个“无效的 JSON”标记（通常为 INVALID_JSON），而不会抛出错误。如果输入为 NULL，也会返回 INVALID_JSON。

语法

```
JSON_PARSE_NULLABLE_ERROR_TO_INVALID( <str> )
```

别名

- JSONB_PARSE_NULLABLE_ERROR_TO_INVALID

必选参数

参数	描述
<str>	需要解析的 JSON 格式的输入字符串。

返回值

如果输入字符串是有效的 JSON，返回对应的 JSON 对象。如果输入字符串无效或为 NULL，返回 INVALID_JSON 标记。

举例

1. 有效的 JSON 字符串：“‘sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_INVALID(‘{“name”: “John”, “age”: 30}’);

“‘

```
+-------------------------------------------------------------+
| JSON_PARSE_NULLABLE_ERROR_TO_INVALID('{"name": "John", "age": 30}') |
+-------------------------------------------------------------+
| {"name": "John", "age": 30}                                 |
+-------------------------------------------------------------+
```

2. 无效的 JSON 字符串：“‘sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_INVALID(‘{“name”: “John”, “age”: }’);

sql +————————————————————————————+ | JSON_PARSE_NULLABLE_ERROR_TO_INVALID(‘{“name”: “John”, “age”: }’) | +————————————————————————————+ | INVALID_JSON | +————————————————————————————+

3. 输入为 NULL：sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_INVALID(NULL);

sql +————————————————————————————+ | JSON_PARSE_NULLABLE_ERROR_TO_INVALID(NULL) | +————————————————————————————+ | INVALID_JSON | +————————————————————————————+

“‘

7.2.1.11.29 JSON_PARSE_NULLABLE_ERROR_TO_NULL

描述

JSON_PARSE_NULLABLE_ERROR_TO_NULL 函数用于解析一个 JSON 字符串为有效的 JSON 对象。如果输入的 JSON 字符串无效，它将返回 NULL，而不会抛出错误。如果输入为 NULL，则直接返回 NULL。

语法

```
JSON_PARSE_NULLABLE_ERROR_TO_NULL( <str> )
```

别名

- JSONB_PARSE_NULLABLE_ERROR_TO_NULL

必选参数

参数	描述
<str>	需要解析的 JSON 格式的输入字符串。

返回值

如果输入字符串是有效的 JSON，返回对应的 JSON 对象。如果输入字符串无效或为 NULL，返回 NULL。

举例

1. 有效的 JSON 字符串：

```
SELECT JSON_PARSE_NULLABLE_ERROR_TO_NULL('{"name": "John", "age": 30}');
```

```
+-------------------------------------------------------------+
| JSON_PARSE_NULLABLE_ERROR_TO_NULL('{"name": "John", "age": 30}') |
+-------------------------------------------------------------+
| {"name": "John", "age": 30}                                  |
+-------------------------------------------------------------+
```

2. 无效的 JSON 字符串：

```
SELECT JSON_PARSE_NULLABLE_ERROR_TO_NULL('{"name": "John", "age": }');
```

```
+-------------------------------------------------------------+
| JSON_PARSE_NULLABLE_ERROR_TO_NULL('{"name": "John", "age": }') |
+-------------------------------------------------------------+
| NULL                                                         |
+-------------------------------------------------------------+
```

3. 输入为 NULL：

```
SELECT JSON_PARSE_NULLABLE_ERROR_TO_NULL(NULL);
```

```
+-------------------------------------------------------------+
| JSON_PARSE_NULLABLE_ERROR_TO_NULL(NULL)                     |
+-------------------------------------------------------------+
| NULL                                                        |
+-------------------------------------------------------------+
```

7.2.1.11.30 JSON_PARSE_NULLABLE_ERROR_TO_VALUE

描述

JSON_PARSE_NULLABLE_ERROR_TO_VALUE 函数用于解析一个 JSON 字符串为有效的 JSON 对象。如果输入的 JSON 字符串无效，它将返回用户指定的默认值，而不是抛出错误。如果输入为 NULL，则返回默认值。

语法

```
JSON_PARSE_NULLABLE_ERROR_TO_VALUE( <str> , <default_value>)
```

别名

- JSONB_PARSE_NULLABLE_ERROR_TO_VALUE

必选参数

参数	描述
<str>	需要解析的 JSON 格式的输入字符串。
<default_value>	解析失败时返回的默认值。

返回值

如果输入字符串是有效的 JSON，返回对应的 JSON 对象。如果输入字符串无效或为 NULL，返回 default_value 参数指定的默认值。

举例

1. 有效的 JSON 字符串：“‘sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_VALUE(‘{“name”: “John”, “age”: 30}’, ‘default’);

sql +————————————————————+ | JSON_PARSE_NULLABLE_ERROR_TO_VALUE(‘{“name”: “John”, “age”: 30}’, ‘default’) | +————————————————————+ | {“name”: “John”, “age”: 30} | +————————————————————+

2. 无效的 JSON 字符串：sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_VALUE(‘{“name”: “John”, “age”: }’, ‘default’);

sql +————————————————————————————+ | JSON_PARSE_NULLABLE_ERROR_TO_VALUE(‘{“name”: “John”, “age”:}’, ‘default’) | +————————————————————————————+ | default | +————————————————————————————+

3．输入为 NULL： sql SELECT JSON_PARSE_NULLABLE_ERROR_TO_VALUE(NULL, ‘default’);

sql +————————————————————————————+ | JSON_PARSE_NULLABLE_ERROR_TO_VALUE(NULL, ‘default’) | +————————————————————————————+ | default | +————————————————————————————+

“ ‘

7.2.1.11.31 JSON_QUOTE

描述

将 json_value 用双引号（ “ ）括起来，跳过其中包含的特殊转义字符。

语法

```
JSON_QUOTE (<a>)
```

参数

参数	描述
<a>	要括起来的 json_value 的值

返回值

返回一个 json_value。特殊情况如下：* 如果传入的参数为 NULL，返回 NULL。

举例

```
SELECT json_quote('null'), json_quote('"null"');
```

```
+--------------------+----------------------+
| json_quote('null') | json_quote('"null"') |
+--------------------+----------------------+
| "null"             | "\"null\""           |
+--------------------+----------------------+
```

```
SELECT json_quote('[1, 2, 3]');
```

```
+-------------------------+
| json_quote('[1, 2, 3]') |
+-------------------------+
| "[1, 2, 3]"             |
+-------------------------+
```

```
SELECT json_quote(null);
```

```
+------------------+
| json_quote(null) |
+------------------+
| NULL             |
+------------------+
```

```
select json_quote("`\n`\b\r`\t`");
```

```
+------------------------+
| json_quote('`\n`\b\r`\t`') |
+------------------------+
| "`\n`\b\r`\t`"             |
+------------------------+
```

7.2.1.11.32 JSON_REPLACE

描述

JSON_REPLACE 函数用于在 JSON 中更新数据并返回结果。

语法

```
JSON_REPLACE (<json_str>, <path>, <val>[, <jsonPath>, <val>, ...])
```

参数

参数	描述
<json_str>	要替换的 JSON 数据。可以是任意类型元素的 JSON 对象，包括NULL，如果没有指定元素，则返回一个空数组。如果 json_str 不是有效的 JSON，则会返回错误
<path>	要替换的 JSON 路径。
<val>	要替换 JSON_PATH Key 对应 value 的值。如果是 NULL ，则会在对应的位置插入 NULL 的 value 值。

返回值

如果 json_str 和 path 都为 NULL，则返回 NULL。

否则，如果 json_str 不是有效的 JSON 或任何 path 参数不是有效的路径表达式或包含了 * 通配符，则会返回错误。

路径值对按从左到右的顺序进行评估。

如果 JSON 中已存在某个路径，则路径值对会将现有 JSON 值覆盖为新值。否则，对于 JSON 中不存在的某个路径的路径值对将被忽略且不会产生任何影响。

示例

```
select json_replace(null, null, null);
```

```
+---------------------------------+
| json_replace(NULL, NULL, 'NULL') |
+---------------------------------+
| NULL                            |
+---------------------------------+
```

```
select json_replace('{"k": 1}', "$.k", 2);
```

```
+---------------------------------------+
| json_replace('{\"k\": 1}', '$.k', '2') |
+---------------------------------------+
| {"k":2}                               |
+---------------------------------------+
```

```
select json_replace('{"k": 1}', "$.j", 2);
```

```
+---------------------------------------+
| json_replace('{\"k\": 1}', '$.j', '2') |
+---------------------------------------+
| {"k":1}                               |
+---------------------------------------+
```

```
select json_replace(null, null, 's');
```

```
+-------------------------------------+
| json_replace(NULL, NULL, 's', '006') |
+-------------------------------------+
| NULL                                |
+-------------------------------------+
```

7.2.1.11.33 JSON_SEARCH

描述

JSON_SEARCH 函数用于在 JSON 文档中查找指定的值。如果找到该值，则返回值的路径。如果没有找到该值，则返回 NULL。该函数可以在 JSON 数据结构中递归查找。

语法

```
JSON_SEARCH(<str>, <one_or_all>, <search_value> [, <start_path> [, <escape_char>]])
```

必选参数

参数	描述
<str>	需要搜索的 JSON 文档（可以是 JSON 字符串或 JSON 对象）。
<one_or_all>	指定是否查找所有匹配的值。可以取值 'one' 或 'all'。
<search_value>	需要查找的值，搜索目标。

可选参数

参数	描述
<start_path>	指定开始搜索的路径。如果没有提供，则从整个 JSON 文档开始搜索。
<escape_char>	指定用于转义路径中的特殊字符。

返回值

如果找到匹配的值，返回一个 JSON 路径（字符串类型），指向匹配的值。如果没有找到匹配的值，返回 NULL。

注意事项

one_or_all 参数决定了是否查找所有匹配的值。‘one’ 会返回第一个匹配的路径，‘all’ 会返回所有匹配的路径。如果没有找到匹配值，函数会返回 NULL。使用 start_path 参数可以限制搜索的范围，使得查询更加高效。

示例

1. 查找一个值（one）：“‘sql SELECT JSON_SEARCH(‘{“name”: “John”, “age”: 30}’, ‘one’, ‘John’);

sql +————————————————————–+ | JSON_SEARCH(‘{“name”: “John”, “age”: 30}’, ‘one’, ‘John’) | +————————————————————–+ | $.name | +————————————————————–+

“‘2. 查找多个匹配值（all）：

```
SELECT JSON_SEARCH('{"person": {"name": "John", "age": 30}, "name": "John"}', 'all', 'John');
```

```
+-----------------------------------------------------------------+
| JSON_SEARCH('{"person": {"name": "John", "age": 30}, "name": "John"}', 'all', 'John') |
+-----------------------------------------------------------------+
| $.name                                                          |
| $.person.name                                                   |
+-----------------------------------------------------------------+
```

3. 没有找到匹配值：

```
SELECT JSON_SEARCH('{"name": "John", "age": 30}', 'one', 'Alice');
```

```
+-----------------------------------------------+
| JSON_SEARCH('{"name": "John", "age": 30}', 'one', 'Alice') |
+-----------------------------------------------+
| NULL                                          |
+-----------------------------------------------+
```

4. 指定搜索起始路径：

```
SELECT JSON_SEARCH('{"person": {"name": "John", "age": 30}}', 'one', 'John', '$.person');
```

```
+-----------------------------------------------------------------+
| JSON_SEARCH('{"person": {"name": "John", "age": 30}}', 'one', 'John', '$.person') |
+-----------------------------------------------------------------+
| $.name                                                          |
+-----------------------------------------------------------------+
```

7.2.1.11.34 JSON_SET

描述

json_set 函数在 JSON 中插入或更新数据并返回结果。

语法

```
JSON_SET (<json_str>,  <path>,  <val> [, <path>,  <val>, ...])
```

参数

参数	描述
<jsonStr>	要插入的 JSON 对象。可以是任意类型元素的 JSON 对象，包括NULL，如果没有指定元素，则返回一个空数组。如果 json_str 不是有效的 JSON 或任何 path 参数不是有效的路径表达式或包含了 * 通配符，则会返回错误
<jsonPath>	要插入的 JSON 路径。如果是 NULL ，则返回 NULL
<val>	要插入 JSON 的值。如果是 NULL ，则会在对应的位置插入 NULL 的 value 值。

返回值

json_set 函数在 JSON 中插入或更新数据并返回结果。如果 json_str 或 path 为 NULL，则返回 NULL。否则，如果 json_str 不是有效的 JSON 或任何 path 参数不是有效的路径表达式或包含了 * 通配符，则会返回错误。

路径值对按从左到右的顺序进行评估。

如果 JSON 中已存在某个路径，则路径值对会将现有 JSON 值覆盖为新值。如果 JSON 中不存在该路径，则路径值对会添加该值到 JSON 中，如果路径标识某个类型的值，则：

- 对于现有对象中不存在的成员，会将新成员添加到该对象中并与新值相关联。
- 对于现有数组结束后的位置，该数组将扩展为包含新值。如果现有值不是数组，则自动转换为数组，然后再扩展为包含新值的数组。

否则，对于 JSON 中不存在的某个路径的路径值对将被忽略且不会产生任何影响。

示例

```
select json_set(null, null, null);
```

```
+------------------------------+
| json_set(NULL, NULL, 'NULL') |
+------------------------------+
| NULL                         |
+------------------------------+
```

```
select json_set('{"k": 1}', "$.k", 2);
```

```
+-----------------------------------+
| json_set('{\"k\": 1}', '$.k', '2') |
+-----------------------------------+
| {"k":2}                           |
+-----------------------------------+
```

```
select json_set('{"k": 1}', "$.j", 2);
```

```
+-----------------------------------+
| json_set('{\"k\": 1}', '$.j', '2') |
+-----------------------------------+
| {"k":1,"j":2}                     |
+-----------------------------------+
```

7.2.1.11.35 JSON_TYPE

描述

用来判断 JSON 对象中 <json_path> 指定的字段的类型，如果字段不存在返回 NULL，如果存在返回下面的类型之一：* object * array * null * bool * int * bigint * largeint * double * string

语法

```
JSON_TYPE(<json_object>, <json_path>)
```

别名

- JSONB_TYPE

参数

- <json_object>: JSON 类型的表达式。
- <json_path>: String 类型，比如 "$.key"。

返回值

Nullable<String>: 返回对应字段的类型。

使用说明

- 如果 <json_object> 或者 <json_path> 是 NULL，返回 NULL。
- 如果 <json_path> 不是一个合法路径，函数报错。
- 如果 <json_path> 指定的字段，返回 NULL。

示例

1. Double 类型

```
select json_type('{"key1": 1234.44}', '$.key1');
```

```
+-----------------------------------------+
| json_type('{"key1": 1234.44}', '$.key1') |
+-----------------------------------------+
| double                                  |
+-----------------------------------------+
```

2. BOOLEAN 类型

```
select json_type('{"key1": true}', '$.key1');
```

```
+--------------------------------------+
| json_type('{"key1": true}', '$.key1') |
+--------------------------------------+
| bool                                 |
+--------------------------------------+
```

3. NULL 参数

```
select json_type(NULL, '$.key1');
```

```
+--------------------------+
| json_type(NULL, '$.key1') |
+--------------------------+
| NULL                     |
+--------------------------+
```

4. NULL 参数 2

```
select json_type('{"key1": true}', NULL);
```

```
+---------------------------------+
| json_type('{"key1": true}', NULL) |
+---------------------------------+
| NULL                            |
+---------------------------------+
```

5. json_path 参数指定的字段不存在

```
select json_type('{"key1": true}', '$.key2');
```

```
+--------------------------------------+
| json_type('{"key1": true}', '$.key2') |
+--------------------------------------+
| NULL                                 |
+--------------------------------------+
```

6. 错误的 json_path 参数

```
select json_type('{"key1": true}', '$.');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = [INVALID_ARGUMENT]Json path error: Invalid
    ↪ Json Path for value: $.
```

7.2.1.11.36 JSON_UNQUOTE

描述

这个函数将去掉 JSON 值中的引号，并将结果作为 utf8mb4 字符串返回。如果参数为 NULL，则返回 NULL。

在字符串中显示的如下转义序列将被识别，对于所有其他转义序列，反斜杠将被忽略。

转义序列	序列表示的字符
”	双引号 “
_	退格字符
\f	换页符
\n	换行符
°	回车符
\t	制表符
\	反斜杠
tabularnewline \uxxxx	Unicode 值 XXXX 的 UTF-8 字节

语法

```
JSON_UNQUOTE (<a>)
```

参数

参数	描述
<a>	要去除引号的元素。

返回值

返回一个 utf8mb4 字符串。特殊情况如下：* 如果传入的参数为 NULL，返回 NULL。* 如果传入的参数不是一个带有双引号的值，则会返回值本身。* 如果传入的参数不是一个字符串，则会被自动转换为字符串后，返回值本身。

举例

```
SELECT json_unquote('"doris"');
```

```
+-------------------------+
| json_unquote('"doris"') |
+-------------------------+
| doris                   |
+-------------------------+
```

```
SELECT json_unquote('[1, 2, 3]');
```

```
+---------------------------+
| json_unquote('[1, 2, 3]') |
+---------------------------+
| [1, 2, 3]                 |
+---------------------------+
```

```
SELECT json_unquote(null);
```

```
+--------------------+
| json_unquote(NULL) |
+--------------------+
| NULL               |
+--------------------+
```

```
SELECT json_unquote('"\\ttest"');
```

```
+--------------------------+
| json_unquote('"\ttest"') |
+--------------------------+
|       test                    |
+--------------------------+
```

```
select json_unquote('"doris');
```

```
+------------------------+
| json_unquote('"doris') |
+------------------------+
| "doris                 |
+------------------------+
```

```
select json_unquote('doris');
```

```
+-----------------------+
| json_unquote('doris') |
+-----------------------+
| doris                 |
+-----------------------+
```

```
select json_unquote(1);
```

```
+----------------------------------------+
| json_unquote(cast(1 as VARCHAR(65533))) |
+----------------------------------------+
| 1                                      |
+----------------------------------------+
```

7.2.1.11.37 JSON_VALID

描述

JSON_VALID 函数返回 0 或 1 以表明是否为有效的 JSON, 如果参数是 NULL 则返回 NULL。

语法

```
JSON_VALID( <str> )
```

必选参数

参数	描述
<str>	需要解析的 JSON 格式的输入字符串。

别名

- JSONB_VALID

举例

1. 正常 JSON 字符串

```
SELECT json_valid('{"k1":"v31","k2":300}');
+------------------------------------+
| json_valid('{"k1":"v31","k2":300}') |
+------------------------------------+
|                                  1 |
```

```
+-------------------------------------+
1 row in set (0.02 sec)
```

2. 无效的 JSON 字符串

```
SELECT json_valid('invalid json');
+----------------------------+
| json_valid('invalid json') |
+----------------------------+
|                          0 |
+----------------------------+
```

3. NULL 参数

```
SELECT json_valid(NULL);
+------------------+
| json_valid(NULL) |
+------------------+
|             NULL |
+------------------+
```

7.2.1.12 IP 函数

7.2.1.12.1 CUT_IPV6

描述

接受一个 IPv6 类型的地址，并以文本格式返回一个包含指定字节数的地址的字符串。

语法

```
CUT_IPV6(<ipv6>, <cut_ipv6_bytes>, <cut_ipv4_bytes>)
```

参数

Parameter	Description
<ipv6>	ipv6 类型的地址
<cut_ipv6_bytes>	需要对 ipv6 地址剪切的字节数
<cut_ipv4_bytes>	如果第一个参数是 ipv4 类型的地址，需要对 ipv4 地址剪切的字节数

返回值

返回一个包含指定字节数的地址的字符串。

举例

```
select cut_ipv6(to_ipv6('2001:0DB8:AC10:FE01:FEED:BABE:CAFE:F00D'), 10, 0);
```

```
+-------------------------------------------------------------------+
| cut_ipv6(to_ipv6('2001:0DB8:AC10:FE01:FEED:BABE:CAFE:F00D'), 10, 0) |
+-------------------------------------------------------------------+
| 2001:db8:ac10::                                                   |
+-------------------------------------------------------------------+
```

7.2.1.12.2 IPV4_CIDR_TO_RANGE

描述

接收一个 IPv4 和一个包含 CIDR 的 Int16 值。返回一个结构体，其中包含两个 IPv4 字段分别表示子网的较低范围（min）和较高范围（max）。

语法

```
IPV4_CIDR_TO_RANGE(<ip_v4>, <cidr>)
```

参数

Parameter	Description
<ip_v4>	字符串类型的 ipv4 地址
<cidr>	cidr 值

返回值

返回一个结构体，其中包含两个 IPv4 字段分别表示子网的较低范围（min）和较高范围（max）- 如果输入为 NULL，则返回 NULL

举例

```
SELECT ipv4_cidr_to_range(ipv4_string_to_num('192.168.5.2'), 16) as re1, ipv4_cidr_to_range(to_
    ↪ ipv4('192.168.5.2'), 16) as re2, ipv4_cidr_to_range(NULL, NULL) as re3;
```

```
+-----------------------------------------------+-----------------------------------------------+------+
    ↪
| re1                                           | re2
    ↪                                             | re3  |
+-----------------------------------------------+-----------------------------------------------+------+
    ↪
| {"min":"192.168.0.0", "max":"192.168.255.255"} | {"min":"192.168.0.0", "max":"192.168.255.255"}
    ↪  | NULL |
+-----------------------------------------------+-----------------------------------------------+------+
    ↪
```

7.2.1.12.3 IPV4_NUM_TO_STRING

描述

接受一个类型为 Int16、Int32、Int64 且大端表示的 IPv4 的地址，返回相应 IPv4 的字符串表现形式，格式为 A.B.C.D (以点分割的十进制数字)。

别名

- INET_NTOA

语法

```
IPV4_NUM_TO_STRING(<ipv4_num>)
```

参数

Parameter	Description
<ipv4_num>	由 ipv4 地址转换而来的 int 值

返回值

返回相应 IPv4 的字符串表现形式，格式为 A.B.C.D (以点分割的十进制数字)。- 对于负数或超过 4294967295 (即 '255.255.255.255') 的入参都返回 NULL，表示无效输入。

举例

```
select ipv4_num_to_string(3232235521);
```

```
+--------------------------------+
| ipv4_num_to_string(3232235521) |
+--------------------------------+
| 192.168.0.1                    |
+--------------------------------+
```

```
select num,ipv4_num_to_string(num) from ipv4_bi;
```

```
+------------+---------------------------+
| num        | ipv4_num_to_string(`num`) |
+------------+---------------------------+
|         -1 | NULL                      |
|          0 | 0.0.0.0                   |
| 2130706433 | 127.0.0.1                 |
| 4294967295 | 255.255.255.255           |
| 4294967296 | NULL                      |
+------------+---------------------------+
```

7.2.1.12.4 IPV4_STRING_TO_NUM

描述

获取包含 IPv4 地址的字符串，格式为 A.B.C.D（点分隔的十进制数字）。返回一个 BIGINT 数字，表示相应的大端 IPv4 地址。

语法

```
IPV4_STRING_TO_NUM(<ipv4_string>)
```

参数

Parameter	Description
<ipv4_string>	字符串类型的 ipv4 地址，例如 'A.B.C.D'

返回值

返回一个 BIGINT 数字，表示相应的大端 IPv4 地址 - 如果输入字符串不是有效的 IPv4 地址或者 NULL，将返回错误

举例

```
select ipv4_string_to_num('192.168.0.1');
```

```
+-----------------------------------+
| ipv4_string_to_num('192.168.0.1') |
+-----------------------------------+
| 3232235521                        |
+-----------------------------------+
```

```
select ipv4_string_to_num('invalid');
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (172.17.0.2)[CANCELLED][INVALID_ARGUMENT][E33]
    ↪ Invalid IPv4 value
```

7.2.1.12.5 IPV4_STRING_TO_NUM_OR_DEFAULT

描述

获取包含 IPv4 地址的字符串，格式为 A.B.C.D（点分隔的十进制数字）。返回一个 BIGINT 数字，表示相应的大端 IPv4 地址。

语法

```
IPV4_STRING_TO_NUM_OR_DEFAULT(<ipv4_string>)
```

参数

Parameter	Description
<ipv4_string>	字符串类型的 ipv4 地址，例如 ‘A.B.C.D’

返回值

返回一个 BIGINT 数字，表示相应的大端 IPv4 地址 - 如果输入字符串不是有效的 IPv4 地址或者 NULL，将返回 0

举例

```
select ipv4_string_to_num_or_default('192.168.0.1');
```

```
+-----------------------------------------------+
| ipv4_string_to_num_or_default('192.168.0.1') |
+-----------------------------------------------+
| 3232235521                                    |
+-----------------------------------------------+
```

```
select str, ipv4_string_to_num_or_default(str) from ipv4_str;
```

```
+-----------------+-------------------------------------+
|str              | ipv4_string_to_num_or_default(str) |
+-----------------+-------------------------------------+
| 0.0.0.0         | 0                                   |
| 127.0.0.1       | 2130706433                          |
| 255.255.255.255 | 4294967295                          |
| invalid         | 0                                   |
+-----------------+-------------------------------------+
```

7.2.1.12.6 IPV4_STRING_TO_NUM_OR_NULL

描述

获取包含 IPv4 地址的字符串，格式为 A.B.C.D（点分隔的十进制数字）。返回一个 BIGINT 数字，表示相应的大端 IPv4 地址。

别名

- INET_ATON

语法

```
IPV4_STRING_TO_NUM_OR_NULL(<ipv4_string>)
```

参数

Parameter	Description
<ipv4_string>	字符串类型的 ipv4 地址，例如 ‘A.B.C.D’

返回值

返回一个 BIGINT 数字，表示相应的大端 IPv4 地址 - 如果输入字符串不是有效的 IPv4 地址，将返回 NULL

举例

```
select ipv4_string_to_num_or_null('192.168.0.1');
```

```
+-------------------------------------------+
| ipv4_string_to_num_or_null('192.168.0.1') |
+-------------------------------------------+
| 3232235521                                |
+-------------------------------------------+
```

```
select str, ipv4_string_to_num_or_null(str) from ipv4_str;
```

```
+-----------------+---------------------------------+
|str              | ipv4_string_to_num_or_null(str) |
+-----------------+---------------------------------+
| 0.0.0.0         | 0                               |
| 127.0.0.1       | 2130706433                      |
| 255.255.255.255 | 4294967295                      |
| invalid         | NULL                            |
+-----------------+---------------------------------+
```

7.2.1.12.7 IPV4_TO_IPV6

描述

接受一个类型为 IPv4 的地址，返回相应 IPv6 的形式。

语法

```
IPV4_TO_IPV6(<ipv4>)
```

参数

Parameter	Description
<ipv4>	ipv4 类型的地址

返回值

返回转化后的 ipv6 地址

举例

```
select ipv6_num_to_string(ipv4_to_ipv6(to_ipv4('192.168.0.1')));
```

```
+-----------------------------------------------------------+
| ipv6_num_to_string(ipv4_to_ipv6(to_ipv4('192.168.0.1'))) |
+-----------------------------------------------------------+
| ::ffff:192.168.0.1                                        |
+-----------------------------------------------------------+
```

7.2.1.12.8 IPV6_CIDR_TO_RANGE

描述

接收一个 IPv6 和一个包含 CIDR 的 Int16 值。返回一个结构体，其中包含两个 IPv6 字段分别表示子网的较低范围（min）和较高范围（max）。

语法

```
IPV6_CIDR_TO_RANGE(ip_v6, cidr)
```

参数

Parameter	Description
<ip_v6>	字符串类型的 ipv6 地址
<cidr>	cidr 值

返回值

返回一个结构体，其中包含两个 IPv6 字段分别表示子网的较低范围（min）和较高范围（max）。- 如果输入为 NULL, 则返回 NULL

举例

```
SELECT ipv6_cidr_to_range(to_ipv6('2001:0db8:0000:85a3:0000:0000:ac1f:8001'), 32), ipv6_cidr_to_
    ↪ range(NULL, NULL);
```

```
+-------------------------------------------------------------------------+-----------------------------
    ↪
| ipv6_cidr_to_range(to_ipv6('2001:0db8:0000:85a3:0000:0000:ac1f:8001'), 32) | ipv6_cidr_to_range
    ↪ (NULL, NULL) |
+-------------------------------------------------------------------------+-----------------------------
    ↪
| {"min":"2001:db8::", "max":"2001:db8:ffff:ffff:ffff:ffff:ffff:ffff"}       | NULL
    ↪                              |
+-------------------------------------------------------------------------+-----------------------------
    ↪
```

7.2.1.12.9 IPV6_NUM_TO_STRING

描述

接受字符串类型的二进制格式的 IPv6 地址。以文本格式返回此地址的字符串。

别名

- INET6_NTOA

语法

```
IPV6_NUM_TO_STRING(<ipv6_num>)
```

参数

Parameter	Description
<ipv6_num>	以字符串类型呈现的 ipv6 地址的二进制编码值

返回值

以文本格式返回 ipv6 地址的字符串。- 如果输入字符串不是有效的 IPv6 地址的二进制编码，将返回 NULL。

举例

```
select ipv6_num_to_string(unhex('2A0206B8000000000000000000000011')) as addr, ipv6_num_to_string(
    ↪ "-23vno12i34nlfwlsj");
```

```
+--------------+-----------------------------------------+
| addr         | ipv6_num_to_string('-23vno12i34nlfwlsj') |
+--------------+-----------------------------------------+
| 2a02:6b8::11 | NULL                                    |
+--------------+-----------------------------------------+
```

7.2.1.12.10 IPV6_STRING_TO_NUM

描述

IPv6NumToString 的反向函数，它接受一个 IP 地址字符串并返回二进制格式的 IPv6 地址。

语法

```
IPV6_STRING_TO_NUM(<ipv6_string>)
```

参数

Parameter	Description
<ipv6_string>	字符串类型的 ipv6 地址

返回值

返回二进制格式的 IPv6 地址。- 如果输入非法的 IP 地址或者 NULL，会抛出异常 - 如果输入字符串包含有效的 IPv4 地址，则返回其等效的 IPv6 地址。

举例

```
select hex(ipv6_string_to_num('1111::ffff')), hex(ipv6_string_to_num('192.168.0.1'));
```

```
+--------------------------------------+--------------------------------------+
| hex(ipv6_string_to_num('1111::ffff')) | hex(ipv6_string_to_num('192.168.0.1')) |
+--------------------------------------+--------------------------------------+
| 11110000000000000000000000FFFF       | 00000000000000000000FFFFC0A80001       |
+--------------------------------------+--------------------------------------+
```

```
select hex(ipv6_string_to_num('notaaddress'));
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = (172.17.0.2)[CANCELLED][E33] Invalid IPv6 value
```

7.2.1.12.11 IPV6_STRING_TO_NUM_OR_DEFAULT

描述

IPv6NumToString 的反向函数，它接受一个 IP 地址字符串并返回二进制格式的 IPv6 地址。

语法

```
IPV6_STRING_TO_NUM_OR_DEFAULT(<ipv6_string>)
```

参数

Parameter	Description
<ipv6_string>	字符串类型的 ipv6 地址

返回值

返回二进制格式的 IPv6 地址 - 如果输入非法的 IP 地址，会返回 0 - 如果输入字符串包含有效的 IPv4 地址，则返回其等效的 IPv6 地址。- ###### 举例

```
select hex(ipv6_string_to_num_or_default('1111::ffff')) as r1,hex(ipv6_string_to_num_or_default('
    ↪ 192.168.0.1')) as r2, hex(ipv6_string_to_num_or_default('notaaddress')) as r3;
```

```
+----------------------------------+----------------------------------+----------------------------------+
    ↪
| r1                               | r2                               | r3
    ↪                                  |
+----------------------------------+----------------------------------+----------------------------------+
    ↪
```

```
| 11110000000000000000000000FFFF | 00000000000000000000FFFFC0A80001 |
    ↪ 00000000000000000000000000000000 |
+----------------------------------+----------------------------------+-------------------------------+
    ↪
```

7.2.1.12.12 IPV6_STRING_TO_NUM_OR_NULL

描述

IPv6NumToString 的反向函数，它接受一个 IP 地址字符串并返回二进制格式的 IPv6 地址。

别名

- INET6_ATON

语法

```
IPV6_STRING_TO_NUM_OR_NULL(<ipv6_string>)
```

参数

Parameter	Description
<ipv6_string>	字符串类型的 ipv6 地址

返回值

返回二进制格式的 IPv6 地址 - 如果输入非法的 IP 地址，会返回 NULL - 如果输入字符串包含有效的 IPv4 地址，则返回其等效的 IPv6 地址

举例

```
select hex(ipv6_string_to_num_or_null('1111::ffff')) as r1, hex(ipv6_string_to_num_or_null('
    ↪ 192.168.0.1')) as r2, hex(ipv6_string_to_num_or_null('notaaddress')) as r3;
```

```
+----------------------------------+----------------------------------+------+
| r1                               | r2                               | r3   |
+----------------------------------+----------------------------------+------+
| 11110000000000000000000000FFFF | 00000000000000000000FFFFC0A80001 | NULL |
+----------------------------------+----------------------------------+------+
```

7.2.1.12.13 IS_IP_ADDRESS_IN_RANGE

描述

判断 IP（IPv4 或 IPv6）地址是否包含在以 CIDR 表示法表示的网络中。

语法

```
IS_IP_ADDRESS_IN_RANGE(ip_str, cidr_prefix)
```

参数

Parameter	Description
<ip_str> <cidr_prefix>	字符串类型的 ipv4 或者 ipv6 地址 cidr 前缀

返回值

如果 IP（IPv4 或 IPv6）地址是包含在以 CIDR 表示法表示的网络中，则返回 true，否则返回 false。- 如果输入值为 NULL, 则返回 NULL

举例

```
SELECT is_ip_address_in_range('127.0.0.1', '127.0.0.0/8') as v4, is_ip_address_in_range('::ffff
    ↪ :192.168.0.1', '::ffff:192.168.0.4/128') as v6, is_ip_address_in_range('127.0.0.1', NULL)
    ↪  as nil;
```

```
+------+------+------+
| v4   | v6   | nil  |
+------+------+------+
|    1 |    0 | NULL |
+------+------+------+
```

7.2.1.12.14 IS_IPV4_COMPAT

描述

该函数采用以数字形式表示的二进制字符串形式的 IPv6 地址，由 INET6_ATON 返回。INET6_ATON 是 IPV6_STRING_TO_NUM_OR_NULL 的别名 - IPv4 兼容地址的格式为：：ipv4_address

语法

```
IS_IPV4_COMPAT(INET6_ATON(<ipv4_addr>))
```

参数

Parameter	Description
<ipv4_addr>	兼容 ipv4 的地址，例如 ‘::ipv4_address’

返回值

如果参数是有效的 IPv4 兼容 IPv6 地址，则返回 1，否则返回 0 - 如果输入为 NULL, 则返回 NULL

举例

```
SELECT IS_IPV4_COMPAT(INET6_ATON('::ffff:10.0.5.9')) AS re1, IS_IPV4_COMPAT(INET6_ATON('
    ↪ ::10.0.5.9')) AS re2;
```

```
+------+------+
| re1  | re2  |
+------+------+
|    0 |    1 |
+------+------+
```

7.2.1.12.15 IS_IPV4_MAPPED

描述

该函数采用以数字形式表示的二进制字符串形式的 IPv6 地址，由 INET6_ATON 返回。INET6_ATON 是 IPV6_STRING_TO_NUM_OR_NULL 的别名 - IPv4 映射地址的格式为 `::ffff:ipv4_address`

语法

```
IS_IPV4_MAPPED(INET6_ATON(<ipv4_addr>))
```

参数

Parameter	Description
<ipv4_addr>	兼容 ipv4 的地址，例如 '::ipv4_address'

返回值

如果参数是有效的 IPv4 映射 IPv6 地址，则返回 1，否则返回 0，- 如果输入为 NULL, 则返回 NULL

举例

```
SELECT IS_IPV4_MAPPED(INET6_ATON('::ffff:10.0.5.9')) AS re1, IS_IPV4_MAPPED(INET6_ATON('
    ↪ ::10.0.5.9')) AS re2;
```

```
+------+------+
| re1  | re2  |
+------+------+
|    1 |    0 |
+------+------+
```

7.2.1.12.16 IS_IPV4_STRING

描述

接收一个表示形式为字符串的 IPv4 地址作为参数，如果为格式正确且合法的 IPv4 地址，返回 true；反之，返回 false。

语法

```
IS_IPV4_STRING(<ipv4_str>)
```

参数

Parameter	Description
<ipv4_str>	字符串类型的 ipv4 地址

返回值

如果为格式正确且合法的 IPv4 地址，返回 true；反之，返回 false。- 如果输入为 NULL, 则返回 NULL

举例

```
CREATE TABLE `test_is_ipv4_string` (
      `id` int,
      `ip_v4` string
    ) ENGINE=OLAP
    DISTRIBUTED BY HASH(`id`) BUCKETS 4
    PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
    );

insert into test_is_ipv4_string values(0, NULL), (1, '0.0.0.'), (2, ''), (3, '.'), (4, '
    ↪ 255.255.255.255');

select id, ip_v4, is_ipv4_string(ip_v4) from test_is_ipv4_string order by id;
```

```
+------+-----------------+-----------------------+
| id   | ip_v4           | is_ipv4_string(ip_v4) |
+------+-----------------+-----------------------+
|    0 | NULL            |                  NULL |
|    1 | 0.0.0.          |                     0 |
|    2 |                 |                     0 |
|    3 | .               |                     0 |
|    4 | 255.255.255.255 |                     1 |
+------+-----------------+-----------------------+
```

7.2.1.12.17 IS_IPV6_STRING

描述

接收一个表示形式为字符串的 IPv6 地址作为参数，如果为格式正确且合法的 IPv6 地址，返回 true；反之，返回 false。

语法

```
IS_IPV6_STRING(<ipv6_str>)
```

参数

Parameter	Description
<ipv6_str>	字符串类型的 ipv6 地址

返回值

如果为格式正确且合法的 IPv6 地址，返回 1 (true)；反之，返回 0 (false)。- 如果输入为 NULL, 则返回 NULL

举例

```
CREATE TABLE `test_is_ipv6_string` (
      `id` int,
      `ip_v6` string
    ) ENGINE=OLAP
    DISTRIBUTED BY HASH(`id`) BUCKETS 4
    PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
    );

insert into test_is_ipv6_string values(0, NULL), (1, '::'), (2, ''), (3, '2001:1b70:a1:610::b102
    ↪ :2'), (4, 'ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffffg');

select id, ip_v6, is_ipv6_string(ip_v6) from test_is_ipv6_string order by id;
```

```
+------+-----------------------------------------+----------------------+
| id   | ip_v6                                   | is_ipv6_string(ip_v6) |
+------+-----------------------------------------+----------------------+
|    0 | NULL                                    |                 NULL |
|    1 | ::                                      |                    1 |
|    2 |                                         |                    0 |
|    3 | 2001:1b70:a1:610::b102:2                |                    1 |
|    4 | ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffffg |                    0 |
+------+-----------------------------------------+----------------------+
```

7.2.1.12.18 TO_IPV4

描述

输入 IPv4 地址的字符串形式，并返回 IPv4 类型的值。

语法

```
TO_IPV4(<ipv4_str>)
```

参数

Parameter	Description
<ipv4_str>	字符串类型的 ipv4 地址

返回值

返回 IPv4 类型的值，该值的二进制形式等于 ipv4_string_to_num 函数返回值的二进制形式。- 如果 IPv4 地址为非法格式，则抛出异常

举例

```
SELECT to_ipv4('255.255.255.255');
```

```
+----------------------------+
| to_ipv4('255.255.255.255') |
+----------------------------+
| 255.255.255.255            |
+----------------------------+
```

7.2.1.12.19 TO_IPV4_OR_DEFAULT

描述

输入 IPv4 地址的字符串形式，并返回 IPv4 类型的值。

语法

```
TO_IPV4_OR_DEFAULT(<ipv4_str>)
```

参数

Parameter	Description
<ipv4_str>	字符串类型的 ipv4 地址

返回值

返回 IPv4 类型的值。- 如果 IPv4 地址的格式非法，则返回 0.0.0.0

举例

```
SELECT to_ipv4_or_default('255.255.255.255'), to_ipv4_or_default('.'), to_ipv4_or_default(NULL);
```

```
+--------------------------------------+------------------------+--------------------------+
| to_ipv4_or_default('255.255.255.255') | to_ipv4_or_default('.') | to_ipv4_or_default(NULL) |
+--------------------------------------+------------------------+--------------------------+
| 255.255.255.255                      | 0.0.0.0                | 0.0.0.0                  |
+--------------------------------------+------------------------+--------------------------+
```

7.2.1.12.20 TO_IPV4_OR_NULL

描述

输入 IPv4 地址的字符串形式，并返回 IPv4 类型的值。

语法

```
TO_IPV4_OR_NULL(<ipv4_str>)
```

参数

Parameter	Description
<ipv4_str>	字符串类型的 ipv4 地址

返回值

返回 IPv4 类型的值。- 如果 IPv4 地址的格式非法，则返回 NULL

举例

```
SELECT to_ipv4_or_null('255.255.255.255'), to_ipv4_or_null('.'), to_ipv4_or_null(NULL);
```

```
+-----------------------------------+---------------------+-----------------------+
| to_ipv4_or_null('255.255.255.255') | to_ipv4_or_null('.') | to_ipv4_or_null(NULL) |
+-----------------------------------+---------------------+-----------------------+
| 255.255.255.255                   | NULL                | NULL                  |
+-----------------------------------+---------------------+-----------------------+
```

7.2.1.12.21 TO_IPV6

描述

输入 IPv6 地址的字符串形式，并返回 IPv6 类型的值。该值的二进制形式等于 ipv6_string_to_num 函数返回值的二进制形式。

语法

```
TO_IPV6(<ipv6_str>)
```

参数

Parameter	Description
<ipv6_str>	字符串类型的 ipv6 地址

返回值

返回 IPv6 类型的值。- 如果 IPv6 地址为非法格式，则抛出异常

举例

```
SELECT to_ipv6('::'),to_ipv6('2001:1b70:a1:610::b102:2');
```

```
+---------------+-------------------------------------+
| to_ipv6('::') | to_ipv6('2001:1b70:a1:610::b102:2') |
+---------------+-------------------------------------+
| ::            | 2001:1b70:a1:610::b102:2            |
+---------------+-------------------------------------+
```

7.2.1.12.22 TO_IPV6_OR_DEFAULT

描述

输入 IPv6 地址的字符串形式，并返回 IPv6 类型的值。

语法

```
TO_IPV6_OR_DEFAULT(<ipv6_str>)
```

参数

Parameter	Description
<ipv6_str>	字符串类型的 ipv6 地址

返回值

返回 IPv6 类型的值。- 如果 IPv6 地址的格式非法，则返回::

举例

```
SELECT to_ipv6_or_default('.'), to_ipv6_or_default('2001:1b70:a1:610::b102:2');
```

```
+-------------------------+-------------------------------------------------+
| to_ipv6_or_default('.') | to_ipv6_or_default('2001:1b70:a1:610::b102:2') |
+-------------------------+-------------------------------------------------+
| ::                      | 2001:1b70:a1:610::b102:2                        |
+-------------------------+-------------------------------------------------+
```

7.2.1.12.23 TO_IPV6_OR_NULL

描述

输入 IPv6 地址的字符串形式，并返回 IPv6 类型的值。

语法

```
TO_IPV6_OR_NULL(<ipv6_str>)
```

参数

Parameter	Description
<ipv6_str>	字符串类型的 ipv6 地址

返回值

返回 IPv6 类型的值。- 如果 IPv6 地址的格式非法，则返回 NULL

举例

```
SELECT to_ipv6_or_null('.'), to_ipv6_or_null('2001:1b70:a1:610::b102:2');
```

```
+----------------------+--------------------------------------------+
| to_ipv6_or_null('.') | to_ipv6_or_null('2001:1b70:a1:610::b102:2') |
+----------------------+--------------------------------------------+
| NULL                 | 2001:1b70:a1:610::b102:2                   |
+----------------------+--------------------------------------------+
```

7.2.1.13 BITMAP 函数

7.2.1.13.1 BITMAP_AND

描述

计算两个及以上输入的 BITMAP 的交集，返回新的 BITMAP.

语法

```
BITMAP_AND(<bitmap>, <bitmap>,[, <bitmap>...])
```

参数

参数	说明
<bitmap>	被求交集的原 BITMAP 之一

返回值

返回一个 BITMAP
- 当参数存在空值时，返回 NULL

举例

```
select bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap
    ↪ _from_string('1,2,3,4,5')));
```

```
+-------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from
    ↪ _string('1,2,3,4,5'))) |
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| 1,2
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

```
select bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap
    ↪ _from_string('1,2,3,4,5'),bitmap_empty()));
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from
    ↪ _string('1,2,3,4,5'), bitmap_empty())) |
+-----------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

```
select bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap
    ↪ _from_string('1,2,3,4,5'),NULL));
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_and(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from
    ↪ _string('1,2,3,4,5'), NULL)) |
+-----------------------------------------------------------------------------------------------------------------
    ↪
| NULL
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.13.2 BITMAP_AND_COUNT

描述

计算两个及以上输入 BITMAP 的交集，返回交集的个数。

语法

```
BITMAP_AND_COUNT(<bitmap>, <bitmap>,[, <bitmap>...])
```

参数

参数	说明
<bitmap>	被求交集的原 BITMAP 之一

返回值

返回整数 - 当参数存在空值时，返回 NULL

举例

```
select bitmap_and_count(bitmap_from_string('1,2,3'),bitmap_from_string('3,4,5'));
```

```
+-----------------------------------------------------------------------------+
| bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5')) |
+-----------------------------------------------------------------------------+
|                                                                           1 |
+-----------------------------------------------------------------------------+
```

```
select bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_
    ↪ string('1,2,3,4,5'));
```

```
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| (bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_string
    ↪ ('1,2,3,4,5'))) |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 2 |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
```

```
select bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_
    ↪ string('1,2,3,4,5'),bitmap_empty());
```

```
+-------------------------------------------------------------------------------------------------------------------+
    ↪
| (bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_string
    ↪ ('1,2,3,4,5'), bitmap_empty())) |
+-------------------------------------------------------------------------------------------------------------------+
    ↪
```

```
|
    ↪
    ↪ 0 |
+-------------------------------------------------------------------------------------------------------------------
    ↪
```

```
select bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_
    ↪ string('1,2,3,4,5'), NULL);
```

```
+-------------------------------------------------------------------------------------------------------------------
    ↪
| (bitmap_and_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2'), bitmap_from_string
    ↪ ('1,2,3,4,5'), NULL)) |
+-------------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ NULL |
+-------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.13.3 BITMAP_AND_NOT,BITMAP_ANDNOT

描述

将两个 BITMAP 进行与非操作并返回计算结果，其中入参第一个叫 基准 BITMAP，第二个叫 排除 BITMAP。

别名

- BITMAP_ANDNOT

语法

```
BITMAP_AND_NOT(<bitmap1>, <bitmap2>)
```

参数

参数	说明
<bitmap1>	被求与非的基准 BITMAP
<bitmap2>	被求与非的排除 BITMAP

返回值

返回一个 BITMAP。- 当参数存在空值时，返回 NULL

举例

```
select bitmap_count(bitmap_and_not(bitmap_from_string('1,2,3'),bitmap_from_string('3,4,5'))) cnt;
```

```
+------+
| cnt  |
+------+
|    2 |
+------+
```

```
select bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'),bitmap_from_string('3,4,5')));
```

```
+-----------------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'))) |
+-----------------------------------------------------------------------------------------+
| 1,2                                                                                     |
+-----------------------------------------------------------------------------------------+
```

```
select bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'),bitmap_empty()));
```

```
+-------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'), bitmap_empty())) |
+-------------------------------------------------------------------------------+
| 1,2,3                                                                         |
+-------------------------------------------------------------------------------+
```

```
select bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'),NULL));
```

```
+--------------------------------------------------------------------+
| bitmap_to_string(bitmap_and_not(bitmap_from_string('1,2,3'), NULL)) |
+--------------------------------------------------------------------+
| NULL                                                               |
+--------------------------------------------------------------------+
```

7.2.1.13.4 BITMAP_AND_NOT_COUNT,BITMAP_ANDNOT_COUNT

描述

将两个 BITMAP 进行与非操作并返回计算结果集的元素个数，其中入参第一个叫 基准 BITMAP，第二个叫 排除 BITMAP。

别名

- BITMAP_ANDNOT_COUNT

语法

```
BITMAP_AND_NOT_COUNT(<bitmap1>, <bitmap2>)
```

参数

参数	说明
<bitmap1>	被求与非的基准 BITMAP
<bitmap2>	被求与非的排除 BITMAP

返回值

返回整数。- 当参数存在空值时，返回 NULL

举例

```
select bitmap_and_not_count(null, bitmap_from_string('1,2,3')) banc1, bitmap_and_not_count(bitmap
    ↪ _from_string('1,2,3') ,null) banc2;
```

```
+-------+-------+
| banc1 | banc2 |
+-------+-------+
|     0 |     0 |
+-------+-------+
```

```
select bitmap_and_not_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5')) banc;
```

```
+------+
| banc |
+------+
|    2 |
+------+
```

7.2.1.13.5 BITMAP_CONTAINS

描述

计算输入值是否在 BITMAP 中，返回值是 boolean 值。

语法

```
BITMAP_CONTAINS(<bitmap>, <bigint>)
```

参数

参数	说明
<bitmap>	BITMAP 集合
<bitint>	被判断是否存在的整数

返回值

返回一个 boolean - 当参数存在空时，返回 NULL

举例

```
select bitmap_contains(to_bitmap(1),2) cnt1, bitmap_contains(to_bitmap(1),1) cnt2;
```

```
+------+------+
| cnt1 | cnt2 |
+------+------+
|    0 |    1 |
+------+------+
```

7.2.1.13.6 BITMAP_COUNT

描述

计算输入 BITMAP 的元素个数

语法

```
BITMAP_COUNT(<bitmap>)
```

参数

参数	说明
<bitmap>	BITMAP 集合

返回值

返回一个整数

举例

```
select bitmap_count(to_bitmap(1)) cnt;
```

```
+------+
| cnt  |
+------+
|    1 |
+------+
```

7.2.1.13.7 BITMAP_EMPTY

描述

构建一个空 BITMAP。主要用于 insert 或 stream load 时填充默认值。例如：

```
cat data | curl --location-trusted -u user:passwd -T - -H "columns: dt,page,v1,v2=bitmap_empty()"
    ↪    http://127.0.0.1:8040/api/test_database/test_table/_stream_load
```

语法

```
BITMAP_EMPTY()
```

返回值

返回一个无元素的空 BITMAP。

举例

```
select bitmap_to_string(bitmap_empty());
```

```
+----------------------------------+
| bitmap_to_string(bitmap_empty()) |
+----------------------------------+
|                                  |
+----------------------------------+
```

```
select bitmap_count(bitmap_empty());
```

```
+------------------------------+
| bitmap_count(bitmap_empty()) |
+------------------------------+
|                            0 |
+------------------------------+
```

7.2.1.13.8 BITMAP_FROM_ARRAY

描述

将一个 TINYINT/SMALLINT/INT/BIGINT 类型的数组转化为一个 BITMAP，当输入字段不合法时，结果返回 NULL

语法

```
BITMAP_FROM_ARRAY(<arr>)
```

参数

参数	说明
<arr>	整形数组

返回值

返回一个 BITMAP - 当输入字段不合法时，结果返回 NULL

举例

```
SELECT bitmap_to_string(bitmap_from_array(array(1, 0, 1, 1, 0, 1, 0))) AS bs;
```

```
+------+
```

```
| bs    |
+-------+
| 0,1   |
+-------+
```

7.2.1.13.9 BITMAP_FROM_BASE64

描述

将一个 base64 字符串（可以由 bitmap_to_base64 函数转换来）转化为一个 BITMAP。当输入字符串不合法时，返回 NULL。

语法

```
BITMAP_FROM_BASE64(<base64_str>)
```

参数

参数	说明
<base64_str>	base64 字符串 (可以由bitmap_to_base64函数转换来)

返回值

返回一个 BITMAP - 当输入字段不合法时，结果返回 NULL

举例

```
select bitmap_to_string(bitmap_from_base64("AA==")) bts;
```

```
+-------+
| bts   |
+-------+
|       |
+-------+
```

```
select bitmap_to_string(bitmap_from_base64("AQEAAAA=")) bts;
```

```
+-------+
| bts   |
+-------+
| 1     |
+-------+
```

```
select bitmap_to_string(bitmap_from_base64("AjowAAACAAAAAAAAAJgAAAAYAAAAGgAAAAEAf5Y=")) bts;
```

```
+------------+
| bts        |
```

```
+-----------+
| 1,9999999 |
+-----------+
```

7.2.1.13.10 BITMAP_FROM_STRING

描述

将一个字符串转化为一个 BITMAP，字符串是由逗号分隔的一组 unsigned bigint 数字组成。(数字取值在:0 ~ 18446744073709551615) 比如 “0, 1, 2” 字符串会转化为一个 Bitmap，其中的第 0, 1, 2 位被设置。当输入字段不合法时，返回 NULL

语法

```
BITMAP_FROM_STRING(<str>)
```

参数

参数	说明
<str>	数组字符串，比如 “0, 1, 2” 字符串会转化为一个 Bitmap，其中的第 0, 1, 2 位被设置

返回值

返回一个 BITMAP - 当输入字段不合法时，结果返回 NULL

举例

```
select bitmap_to_string(bitmap_from_string("0, 1, 2")) bts;
```

```
+-------+
| bts   |
+-------+
| 0,1,2 |
+-------+
```

```
select bitmap_from_string("-1, 0, 1, 2") bfs;
```

```
+------+
| bfs  |
+------+
| NULL |
+------+
```

```
select bitmap_to_string(bitmap_from_string("0, 1, 18446744073709551615")) bts;
```

```
+--------------------------+
| bts                      |
```

```
+--------------------------+
| 0,1,18446744073709551615 |
+--------------------------+
```

7.2.1.13.11 BITMAP_HAS_ALL

描述

判断一个 Bitmap 是否包含另一个 Bitmap 的全部元素。

语法

```
bitmap_has_all(<bitmap1>, <bitmap2>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 bitmap

返回值

如果 <bitmap1> 包含 <bitmap2> 的全部元素，则返回 true；
如果 <bitmap2> 包含的元素为空，返回 true；
否则返回 false。

示例

检查一个 Bitmap 是否包含另一个 Bitmap 的全部元素：

```
select bitmap_has_all(bitmap_from_string('0, 1, 2'), bitmap_from_string('1, 2'));
```

结果如下：

```
+-----------------------------------------------------------------------------+
| bitmap_has_all(bitmap_from_string('0, 1, 2'), bitmap_from_string('1, 2')) |
+-----------------------------------------------------------------------------+
|                                                                           1 |
+-----------------------------------------------------------------------------+
```

检查一个空 Bitmap 是否包含另一个 Bitmap 的全部元素：

```
select bitmap_has_all(bitmap_empty(), bitmap_from_string('1, 2'));
```

结果如下：

```
+-----------------------------------------------------------+
| bitmap_has_all(bitmap_empty(), bitmap_from_string('1, 2')) |
+-----------------------------------------------------------+
```

```
|                                                          0 |
+------------------------------------------------------------+
```

7.2.1.13.12 BITMAP_HAS_ANY

描述

计算两个 Bitmap 是否存在相交元素。

语法

```
bitmap_has_any(<bitmap1>, <bitmap2>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap

返回值

如果两个 Bitmap 存在相同元素，返回 true；
如果两个 Bitmap 不存在相同元素，返回 false。

示例

检查一个 Bitmap 是否包含另一个 Bitmap 的任意元素：

```
mysql> select bitmap_has_any(to_bitmap(1), to_bitmap(2));
```

结果如下：

```
+-------------------------------------------+
| bitmap_has_any(to_bitmap(1), to_bitmap(2)) |
+-------------------------------------------+
|                                         0 |
+-------------------------------------------+
```

检查一个 Bitmap 是否包含自身的任意元素：

```
mysql> select bitmap_has_any(to_bitmap(1), to_bitmap(1));
```

结果如下：

```
+-------------------------------------------+
| bitmap_has_any(to_bitmap(1), to_bitmap(1)) |
+-------------------------------------------+
|                                         1 |
+-------------------------------------------+
```

7.2.1.13.13 BITMAP_HASH

描述

对任意类型的输入，计算其 32 位的哈希值，并返回包含该哈希值的 Bitmap。

语法

```
bitmap_hash(<expr>)
```

参数

参数	描述
<expr>	任何值或字段表达式

返回值

包含参数 <expr> 的 64 位 hash 值的 Bitmap。

> note
>
> 该函数使用的哈希算法为 MurMur3。
> MurMur3 算法是一种高性能的、低碰撞率的散列算法，其计算出来的值接近于随机分布，并且能通过卡方分布测试。需要注意的是，不同硬件平台、不同 Seed 值计算出来的散列值可能不同。
> 关于此算法的性能可以参考 Smhasher 排行榜。

举例

如果你想计算某个值的 MurMur3，你可以：

```
select bitmap_to_array(bitmap_hash('hello'))[1];
```

结果如下：

```
+-----------------------------------------------------------+
| %element_extract%(bitmap_to_array(bitmap_hash('hello')), 1) |
+-----------------------------------------------------------+
|                                                 1321743225 |
+-----------------------------------------------------------+
```

如果你想统计某一列去重后的个数，可以使用位图的方式，某些场景下性能比 count distinct 好很多：

```
select bitmap_count(bitmap_union(bitmap_hash(`word`))) from `words`;
```

结果如下：

```
+-----------------------------------------------+
| bitmap_count(bitmap_union(bitmap_hash(`word`))) |
+-----------------------------------------------+
|                                      33263478 |
+-----------------------------------------------+
```

7.2.1.13.14 BITMAP_HASH64

描述

对任意类型的输入计算 64 位的哈希值，返回包含该哈希值的 Bitmap。

语法

```
bitmap_hash64(<expr>)
```

参数

参数	描述
<expr>	任何值或字段表达式

返回值

包含参数 <expr> 的 64 位 hash 值的 Bitmap。

示例

计算一个值的 64 位哈希值，你可以使用：

```
select bitmap_to_string(bitmap_hash64('hello'));
```

结果如下：

```
+-----------------------------------------+
| bitmap_to_string(bitmap_hash64('hello')) |
+-----------------------------------------+
| 1523113656554339l023                     |
+-----------------------------------------+
```

7.2.1.13.15 BITMAP_MAX

描述

计算并返回 Bitmap 中的最大值。

语法

```
bitmap_max(<bitmap>)
```

参数

参数	描述
<bitmap>	Bitmap 类型列或表达式

返回值

Bitmap 中的最大值。
若 Bitmap 为空则返回 NULL。

示例

计算一个空 Bitmap 的最大值：

```
select bitmap_max(bitmap_from_string('')) value;
```

结果如下：

```
+-------+
| value |
+-------+
|  NULL |
+-------+
```

计算包含多个元素的 Bitmap 的最大值：

```
select bitmap_max(bitmap_from_string('1,9999999999')) value;
```

结果如下：

```
+------------+
| value      |
+------------+
| 9999999999 |
+------------+
```

7.2.1.13.16 BITMAP_MIN

描述

计算并返回 Bitmap 中的最小值。

语法

```
bitmap_min(<bitmap>)
```

参数

参数	描述
<bitmap>	Bitmap 类型列或表达式

返回值

Bitmap 中的最小值。
若 Bitmap 为空则返回 NULL。

示例

计算一个空 Bitmap 的最小值：

```
select bitmap_min(bitmap_from_string('')) value;
```

结果如下：

```
+-------+
| value |
+-------+
|  NULL |
+-------+
```

计算包含多个元素的 Bitmap 的最小值：

```
select bitmap_min(bitmap_from_string('1,9999999999')) value;
```

结果如下：

```
+-------+
| value |
+-------+
|     1 |
+-------+
```

7.2.1.13.17 BITMAP_NOT

描述

计算第一个 Bitmap 减去第二个 Bitmap 之后的集合，并返回为新的 Bitmap。

语法

```
bitmap_not(<bitmap1>, <bitmap2>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap

返回值

<bitmap1> 减去 <bitmap2> 后集合的 Bitmap。

示例

计算两个 Bitmap 之间的差集：

```
select bitmap_to_string(bitmap_not(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4')));
```

结果如下，因 <bitmap1> 的所有元素也在 <bitmap2> 中，所以结果是一个空的 Bitmap：

```
+-----------------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_not(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'))) |
+-----------------------------------------------------------------------------------------+
|                                                                                         |
+-----------------------------------------------------------------------------------------+
1 row in set (0.01 sec)
```

计算 <bitmap1> 中存在但 <bitmap2> 中不存在的元素的差集：

```
select bitmap_to_string(bitmap_not(bitmap_from_string('2,3,5'), bitmap_from_string('1,2,3,4')));
```

结果如下，将是一个包含元素 5 的 Bitmap：

```
+-----------------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_not(bitmap_from_string('2,3,5'), bitmap_from_string('1,2,3,4'))) |
+-----------------------------------------------------------------------------------------+
| 5                                                                                       |
+-----------------------------------------------------------------------------------------+
1 row in set (0.01 sec)
```

7.2.1.13.18 BITMAP_OR

描述

计算两个及以上的 Bitmap 的并集，返回新的 Bitmap。

语法

```
bitmap_or(<bitmap1>, <bitmap2>, ..., <bitmapN>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap
...	...
<bitmapN>	第 N 个 Bitmap

返回值

多个 Bitmap 并集的 Bitmap。

示例

计算两个相同 Bitmap 的并集：

```
select bitmap_count(bitmap_or(to_bitmap(1), to_bitmap(1))) cnt;
```

结果如下：

```
+------+
| cnt  |
+------+
|    1 |
+------+
```

将两个相同 Bitmap 的并集转换为字符串：

```
select bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(1)));
```

结果如下：

```
+-----------------------------------------------------------+
| bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(1))) |
+-----------------------------------------------------------+
| 1                                                         |
+-----------------------------------------------------------+
```

计算两个不同 Bitmap 的并集：

```
select bitmap_count(bitmap_or(to_bitmap(1), to_bitmap(2))) cnt;
```

结果如下：

```
+------+
| cnt  |
+------+
|    2 |
+------+
```

将两个不同 Bitmap 的并集转换为字符串：

```
select bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2)));
```

结果如下：

```
+-----------------------------------------------------------+
| bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2))) |
+-----------------------------------------------------------+
| 1,2                                                       |
+-----------------------------------------------------------+
```

计算多个 Bitmap（包括 NULL 值）的并集：

```
select bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2), to_bitmap(10), to_bitmap(0), NULL))
    ↪ ;
```

结果如下：

```
+-----------------------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2), to_bitmap(10), to_bitmap(0), NULL)) |
+-----------------------------------------------------------------------------------------------+
| 0,1,2,10                                                                                      |
+-----------------------------------------------------------------------------------------------+
```

计算多个 Bitmap（包括空 Bitmap）的并集：

```
select bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2), to_bitmap(10), to_bitmap(0), bitmap
    ↪ _empty()));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2), to_bitmap(10), to_bitmap(0), bitmap_
    ↪ empty())) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| 0,1,2,10
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------+
    ↪
```

计算由字符串和单个值创建的 Bitmap 的并集：

```
select bitmap_to_string(bitmap_or(to_bitmap(10), bitmap_from_string('1,2'), bitmap_from_string('
    ↪ 1,2,3,4,5')));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| bitmap_to_string(bitmap_or(to_bitmap(10), bitmap_from_string('1,2'), bitmap_from_string
    ↪ ('1,2,3,4,5'))) |
+-----------------------------------------------------------------------------------------------------------+
    ↪
| 1,2,3,4,5,10
    ↪
    ↪ |
```

```
+-----------------------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.13.19 BITMAP_OR_COUNT

描述

计算两个及以上输入 Bitmap 的并集，返回并集的元素个数。

语法

```
bitmap_or_count(<bitmap1>, <bitmap2>, ..., <bitmapN>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap
...	...
<bitmapN>	第 N 个 Bitmap

返回值

多个 Bitmap 并集的元素个数。
如果有 Bitmap 为 NULL 则返回 NULL。

示例

计算一个非空 Bitmap 和一个空 Bitmap 的并集中的元素数量：

```
select bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_empty());
```

结果如下：

```
+-----------------------------------------------------------+
| bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_empty()) |
+-----------------------------------------------------------+
|                                                         3 |
+-----------------------------------------------------------+
```

计算两个相同 Bitmap 的并集中的元素数量：

```
select bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2,3'));
```

结果如下：

```
+-------------------------------------------------------------------------+
| bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2,3')) |
+-------------------------------------------------------------------------+
```

```
|                                                                                 3 |
+-----------------------------------------------------------------------------------+
```

计算两个不同 Bitmap 的并集中的元素数量：

```
select bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'));
```

结果如下：

```
+-----------------------------------------------------------------------------------+
| bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5')) |
+-----------------------------------------------------------------------------------+
|                                                                                 5 |
+-----------------------------------------------------------------------------------+
```

计算多个 Bitmap（包括一个空 Bitmap）的并集中的元素数量：

```
select bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'), to_bitmap(100),
    ↪ bitmap_empty());
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------------+
    ↪
| bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'), to_bitmap(100),
    ↪ bitmap_empty()) |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 6 |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
```

计算多个 Bitmap（包括一个 NULL 值）的并集中的元素数量：

```
select bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'), to_bitmap(100),
    ↪ NULL);
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------+
    ↪
| bitmap_or_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'), to_bitmap(100), NULL)
    ↪  |
+-----------------------------------------------------------------------------------------------------------+
    ↪
|                                                                                                      NULL
    ↪  |
```

```
+-----------------------------------------------------------------------------------------------------+
    ↪
```

7.2.1.13.20 BITMAP_REMOVE

描述

从 Bitmap 列中删除指定的值。

语法

```
bitmap_remove(<bitmap>, <value>)
```

参数

参数	描述
<bitmap>	Bitmap 值
<value>	要删除的值

返回值

删除后的 Bitmap。

若要删除的值不存在，则返回原 Bitmap；
若要删除的值为 NULL，则返回 NULL。

示例

从 Bitmap 中移除一个值：

```
select bitmap_to_string(bitmap_remove(bitmap_from_string('1, 2, 3'), 3)) res;
```

结果如下：

```
+------+
| res  |
+------+
| 1,2  |
+------+
```

从 Bitmap 中移除一个 NULL 值：

```
select bitmap_to_string(bitmap_remove(bitmap_from_string('1, 2, 3'), null)) res;
```

结果如下：

```
+------+
| res  |
+------+
| NULL |
+------+
```

7.2.1.13.21 BITMAP_SUBSET_IN_RANGE

描述

返回 Bitmap 指定范围内的子集 (不包括范围结束)。

语法

```
bitmap_subset_in_range(<bitmap>, <range_start_include>, <range_end_exclude>)
```

参数

参数	描述
<bitmap>	Bitmap 值
<range_start_include>	范围开始 (包含)
<range_end_exclude>	范围结束 (不包含)

返回值

指定范围的子集 Bitmap。

示例

获取 Bitmap 中位于范围 0 到 9 内的子集：

```
select bitmap_to_string(bitmap_subset_in_range(bitmap_from_string('1,2,3,4,5'), 0, 9)) value;
```

结果如下：

```
+-----------+
| value     |
+-----------+
| 1,2,3,4,5 |
+-----------+
```

获取 Bitmap 中位于范围 2 到 3 内的子集：

```
select bitmap_to_string(bitmap_subset_in_range(bitmap_from_string('1,2,3,4,5'), 2, 3)) value;
```

结果如下：

```
+-------+
| value |
+-------+
| 2     |
+-------+
```

7.2.1.13.22 BITMAP_SUBSET_LIMIT

描述

从不小于指定位置 position 开始，按照指定基数 cardinality_limit 为上限截取 Bitmap 元素，返回一个 Bitmap 子集。

语法

```
bitmap_subset_limit(<bitmap>, <position>, <cardinality_limit>)
```

参数

参数	描述
<bitmap>	Bitmap 值
<position>	范围开始的位置（包含）
<cardinality_limit>	基数上限

返回值

指定范围的子集 Bitmap。

示例

获取从位置 0 开始，基数限制为 3 的 Bitmap 子集：

```
select bitmap_to_string(bitmap_subset_limit(bitmap_from_string('1,2,3,4,5'), 0, 3)) value;
```

结果如下：

```
+-----------+
| value     |
+-----------+
| 1,2,3     |
+-----------+
```

获取从位置 4 开始，基数限制为 3 的 Bitmap 子集：

```
select bitmap_to_string(bitmap_subset_limit(bitmap_from_string('1,2,3,4,5'), 4, 3)) value;
```

结果如下：

```
+-------+
| value |
+-------+
| 4,5   |
+-------+
```

7.2.1.13.23 BITMAP_TO_ARRAY

描述

将一个 Bitmap 转化成一个 Array 数组。

语法

```
bitmap_to_array(<bitmap>)
```

参数

参数	描述
<bitmap>	Bitmap 类型列或表达式

返回值

Bitmap 所有 Bit 位构成的的数组。
若 Bitmap 为 NULL 则返回 NULL。

示例

将 NULL Bitmap 转换为数组：

```
select bitmap_to_array(null);
```

结果如下：

```
+------------------------+
| bitmap_to_array(NULL)  |
+------------------------+
| NULL                   |
+------------------------+
```

将空 Bitmap 转换为数组：

```
select bitmap_to_array(bitmap_empty());
```

结果如下：

```
+---------------------------------+
| bitmap_to_array(bitmap_empty()) |
+---------------------------------+
| []                              |
+---------------------------------+
```

将包含单个元素的 Bitmap 转换为数组：

```
select bitmap_to_array(to_bitmap(1));
```

结果如下：

```
+-------------------------------+
| bitmap_to_array(to_bitmap(1)) |
+-------------------------------+
| [1]                           |
+-------------------------------+
```

将包含多个元素的 Bitmap 转换为数组：

```
select bitmap_to_array(bitmap_from_string('1,2,3,4,5'));
```

结果如下：

```
+-------------------------------------------------+
| bitmap_to_array(bitmap_from_string('1,2,3,4,5')) |
+-------------------------------------------------+
| [1, 2, 3, 4, 5]                                 |
+-------------------------------------------------+
```

7.2.1.13.24 BITMAP_TO_BASE64

描述

将一个 Bitmap 转化成一个 Base64 编码后的字符串。

语法

```
bitmap_to_base64(<bitmap>)
```

参数

参数	描述
<bitmap>	Bitmap 类型列或表达式

返回值

Bitmap 基于 Base64 编码后的字符串。
若 Bitmap 为 NULL 时，返回值为 NULL。

> note
>
> BE 配置项 enable_set_in_bitmap_value 会改变 bitmap 值在内存中的具体格式，因此会影响此函数的结果。
> 由于不能保证 bitmap 中元素的顺序，因此不能保证相同内容的 bitmap 生成的 base64 结果始终相同，但可以保证 bitmap_from_base64 解码后的 bitmap 相同。

示例

将 NULL Bitmap 转换为 Base64 字符串：

```
select bitmap_to_base64(null);
```

结果如下：

```
+------------------------+
| bitmap_to_base64(NULL) |
+------------------------+
| NULL                   |
+------------------------+
```

将空 Bitmap 转换为 Base64 字符串：

```
select bitmap_to_base64(bitmap_empty());
```

结果如下：

```
+----------------------------------+
| bitmap_to_base64(bitmap_empty()) |
+----------------------------------+
| AA==                             |
+----------------------------------+
```

将包含单个元素的 Bitmap 转换为 Base64 字符串：

```
select bitmap_to_base64(to_bitmap(1));
```

结果如下：

```
+--------------------------------+
| bitmap_to_base64(to_bitmap(1)) |
+--------------------------------+
| AQEAAAA=                       |
+--------------------------------+
```

将包含多个元素的 Bitmap 转换为 Base64 字符串：

```
select bitmap_to_base64(bitmap_from_string("1,9999999"));
```

结果如下：

```
+----------------------------------------------------------+
| bitmap_to_base64(bitmap_from_string("1,9999999"))        |
+----------------------------------------------------------+
| AjowAAACAAAAAAAAAJgAAAAYAAAAGgAAAAEAf5Y=                 |
+----------------------------------------------------------+
```

7.2.1.13.25 BITMAP_TO_STRING

描述

将一个 Bitmap 转化成一个逗号分隔的字符串，字符串中包含所有设置的 Bit 位。

语法

```
bitmap_to_string(<bitmap>)
```

参数

参数	描述
<bitmap>	Bitmap 类型列或表达式

返回值

包含 Bitmap 所有 Bit 位的字符串，以逗号分隔。
若 Bitmap 为 NULL 时，返回值为 NULL。

示例

将 NULL Bitmap 转换为字符串：

```
select bitmap_to_string(null);
```

结果如下：

```
+------------------------+
| bitmap_to_string(NULL) |
+------------------------+
| NULL                   |
+------------------------+
```

将空 Bitmap 转换为字符串：

```
select bitmap_to_string(bitmap_empty());
```

结果如下：

```
+----------------------------------+
| bitmap_to_string(bitmap_empty()) |
+----------------------------------+
|                                  |
+----------------------------------+
```

将包含单个元素的 Bitmap 转换为字符串：

```
select bitmap_to_string(to_bitmap(1));
```

结果如下：

```
+--------------------------------+
| bitmap_to_string(to_bitmap(1)) |
+--------------------------------+
| 1                              |
+--------------------------------+
```

将包含多个元素的 Bitmap 转换为字符串：

```
select bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2)));
```

结果如下：

```
+-----------------------------------------------------------+
| bitmap_to_string(bitmap_or(to_bitmap(1), to_bitmap(2))) |
+-----------------------------------------------------------+
| 1,2                                                       |
+-----------------------------------------------------------+
```

7.2.1.13.26 BITMAP_XOR

描述

计算两个及以上输入 Bitmap 的差集，返回新的 Bitmap。

语法

```
bitmap_xor(<bitmap1>, <bitmap2>, ..., <bitmapN>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap
...	...
<bitmapN>	第 N 个 Bitmap

返回值

多个 Bitmap 差集的 Bitmap。

示例

计算两个 Bitmap 的对称差集：

```
select bitmap_count(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'))) cnt;
```

结果如下：

```
+------+
| cnt  |
+------+
|    2 |
+------+
```

将两个 Bitmap 的对称差集转换为字符串：

```
select bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4')));
```

结果如下：

```
+-----------------------------------------------------------------------------------------+
| bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'))) |
+-----------------------------------------------------------------------------------------+
| 1,4                                                                                     |
+-----------------------------------------------------------------------------------------+
```

计算三个 Bitmap 的对称差集：

```
select bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'),
    ↪ bitmap_from_string('3,4,5')));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_
    ↪ from_string('3,4,5'))) |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| 1,3,5
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
```

计算多个 Bitmap（包括一个空 Bitmap）的对称差集：

```
select bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'),
    ↪ bitmap_from_string('3,4,5'), bitmap_empty()));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_
    ↪ from_string('3,4,5'), bitmap_empty())) |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| 1,3,5
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
```

计算多个 Bitmap（包括一个 NULL 值）的对称差集：

```
select bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'),
    ↪ bitmap_from_string('3,4,5'), NULL));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_to_string(bitmap_xor(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_
    ↪ from_string('3,4,5'), NULL)) |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
| NULL
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.13.27 BITMAP_XOR_COUNT

描述

将两个及以上 Bitmap 集合进行异或操作并返回结果集的大小。

语法

```
bitmap_xor_count(<bitmap1>, <bitmap2>, ..., <bitmapN>)
```

参数

参数	描述
<bitmap1>	第一个 Bitmap
<bitmap2>	第二个 Bitmap
...	...
<bitmapN>	第 N 个 Bitmap

返回值

将两个及以上 Bitmap 集合进行异或操作得到的差集 Bitmap 元素数量。
当输入的 Bitmap 参数中有 NULL 时，结果为 NULL。

示例

计算两个 Bitmap 的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5'));
```

结果如下：

```
+-----------------------------------------------------------------------------+
| bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('3,4,5')) |
+-----------------------------------------------------------------------------+
|                                                                           4 |
+-----------------------------------------------------------------------------+
```

计算两个相同 Bitmap 的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2,3'));
```

结果如下：

```
+-----------------------------------------------------------------------------+
| bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('1,2,3')) |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

计算两个不同 Bitmap 的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('4,5,6'));
```

结果如下：

```
+-----------------------------------------------------------------------------+
| bitmap_xor_count(bitmap_from_string('1,2,3'), bitmap_from_string('4,5,6')) |
+-----------------------------------------------------------------------------+
|                                                                           6 |
+-----------------------------------------------------------------------------+
```

计算三个 Bitmap 的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_
    ↪ string('3,4,5'));
```

结果如下：

```
+-----------------------------------------------------------------------------------------------------------------+
    ↪
| bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_string
    ↪ ('3,4,5'))    |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
|
    ↪
    ↪ 3 |
+-----------------------------------------------------------------------------------------------------------------+
    ↪
```

计算多个 Bitmap（包括一个空 Bitmap）的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_
    ↪ string('3,4,5'), bitmap_empty());
```

结果如下：

```
+-------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_string
    ↪ ('3,4,5'), bitmap_empty())    |
+-------------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 3 |
+-------------------------------------------------------------------------------------------------------------------
    ↪
```

计算多个 Bitmap（包括一个 NULL 值）的对称差集中的元素数量：

```
select bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_
    ↪ string('3,4,5'), NULL);
```

结果如下：

```
+-------------------------------------------------------------------------------------------------------------------
    ↪
| bitmap_xor_count(bitmap_from_string('2,3'), bitmap_from_string('1,2,3,4'), bitmap_from_string
    ↪ ('3,4,5'), NULL)   |
+-------------------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ NULL |
+-------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.1.13.28 SUB_BITMAP

描述

从指定位置 position 开始，截取指定个数 cardinality_limit 的 Bitmap 元素，返回一个 Bitmap 子集。

语法

```
sub_bitmap(<bitmap>, <position>, <cardinality_limit>)
```

返回值

指定范围的子集 Bitmap。

参数

参数	描述
<bitmap>	Bitmap 值
<position>	范围开始的位置（包含）
<cardinality_limit>	基数上限

示例

获取从位置 0 开始，基数限制为 3 的 Bitmap 子集：

```
select bitmap_to_string(sub_bitmap(bitmap_from_string('1,0,1,2,3,1,5'), 0, 3)) value;
```

结果如下：

```
+-------+
| value |
+-------+
| 0,1,2 |
+-------+
```

获取从位置 -3 开始，基数限制为 2 的 Bitmap 子集：

```
select bitmap_to_string(sub_bitmap(bitmap_from_string('1,0,1,2,3,1,5'), -3, 2)) value;
```

结果如下：

```
+-------+
| value |
+-------+
| 2,3   |
+-------+
```

获取从位置 2 开始，基数限制为 100 的 Bitmap 子集：

```
select bitmap_to_string(sub_bitmap(bitmap_from_string('1,0,1,2,3,1,5'), 2, 100)) value;
```

结果如下：

```
+-------+
| value |
+-------+
| 2,3,5 |
+-------+
```

7.2.1.13.29 TO_BITMAP

描述

将一个无符号的长整型数转换为 Bitmap。

输入为取值在 0 ~ 18446744073709551615 区间的 unsigned bigint，输出为包含该元素的 bitmap。

语法

```
to_bitmap(<expr>)
```

参数

参数	描述
<expr>	无符号的长整型数，范围为 0 ~ 18446744073709551615

返回值

包含对应长整型数的 Bitmap。
当输入值不在对应范围内时，则返回 NULL。

示例

将一个整数转换为 Bitmap 并计算 Bitmap 中的元素数量：

```
select bitmap_count(to_bitmap(10));
```

结果如下：

```
+-----------------------------+
| bitmap_count(to_bitmap(10)) |
+-----------------------------+
|                           1 |
+-----------------------------+
```

将一个负整数转换为 Bitmap（该整数在有效范围之外），并将其转换为字符串：

```
select bitmap_to_string(to_bitmap(-1));
```

结果如下：

```
+---------------------------------+
| bitmap_to_string(to_bitmap(-1)) |
+---------------------------------+
|                                 |
+---------------------------------+
```

7.2.1.14 HLL 函数

7.2.1.14.1 HLL_CARDINALITY

描述

HLL_CARDINALITY 用于计算 HLL（HyperLogLog）类型值的基数。HLL 是一种近似计数的算法，适用于大规模数据集的基数估算。

语法

```
HLL_CARDINALITY(<hll>)
```

参数

参数	说明
<hll>	HLL 类型的值，表示需要计算基数的数据集合。

返回值

返回 HLL 类型值的基数，即数据集合中不重复元素的估算数。

举例

```
select HLL_CARDINALITY(uv_set) from test_uv;
```

```
+---------------------------+
| hll_cardinality(`uv_set`) |
+---------------------------+
|                         3 |
+---------------------------+
```

7.2.1.14.2 HLL_EMPTY

描述

HLL_EMPTY 用于返回一个 HLL（HyperLogLog）类型的空值，表示一个没有任何元素的数据集合。

语法

```
HLL_EMPTY()
```

返回值

返回一个 HLL 类型的空值，表示一个没有任何元素的数据集合。

举例

```
select hll_cardinality(hll_empty());
```

```
+-----------------------------+
| hll_cardinality(hll_empty()) |
+-----------------------------+
```

```
|                              0 |
+--------------------------------+
```

7.2.1.14.3 HLL_FROM_BASE64

描述

将一个 base64 编码的字符串（通常由 HLL_TO_BASE64 函数生成）转换为 HLL 类型。如果输入字符串不合法或为 NULL，则返回 NULL。

语法

```
HLL_FROM_BASE64(<input>)
```

参数

参数	说明
<input>	base64 编码的字符串，通常由 HLL_TO_BASE64 函数生成。如果字符串不合法，则返回 NULL。

示例

```
select hll_union_agg(hll_from_base64(hll_to_base64(pv))), hll_union_agg(pv) from test_hll;
```

```
+-----------------------------------------------------+--------------------+
| hll_union_agg(hll_from_base64(hll_to_base64(pv))) | hll_union_agg(pv) |
+-----------------------------------------------------+--------------------+
|                                                   3 |                  3 |
+-----------------------------------------------------+--------------------+
```

```
select hll_cardinality(hll_from_base64(hll_to_base64(hll_hash('abc'))));
```

```
+-----------------------------------------------------------------------+
| hll_cardinality(hll_from_base64(hll_to_base64(hll_hash('abc')))) |
+-----------------------------------------------------------------------+
|                                                                     1 |
+-----------------------------------------------------------------------+
```

```
select hll_cardinality(hll_from_base64(hll_to_base64(hll_hash(''))));
```

```
+-------------------------------------------------------------+
| hll_cardinality(hll_from_base64(hll_to_base64(hll_hash('')))) |
+-------------------------------------------------------------+
|                                                           1 |
+-------------------------------------------------------------+
```

```
select hll_cardinality(hll_from_base64(hll_to_base64(hll_hash(NULL))));
```

```
+-----------------------------------------------------------------+
| hll_cardinality(hll_from_base64(hll_to_base64(hll_hash(NULL)))) |
+-----------------------------------------------------------------+
|                                                               0 |
+-----------------------------------------------------------------+
```

7.2.1.14.4 HLL_HASH

描述

将一个值转换为 HLL（HyperLogLog）类型。该函数通常用于数据加载时，将普通类型的值转换为 HLL 类型，常用于处理大数据集的去重和计数操作。

特殊情况：- 如果输入值为 NULL，则返回 NULL。

语法

```
HLL_HASH(<value>)
```

参数

参数	说明
<value>	需要转换为 HLL 类型的值。可以是字符串、数字或任意数据类型。

返回值

返回一个 HLL 类型的值。返回结果类型为 HLL。

举例

```
select HLL_CARDINALITY(HLL_HASH('abc'));
```

```
+----------------------------------+
| hll_cardinality(HLL_HASH('abc')) |
+----------------------------------+
|                                1 |
+----------------------------------+
```

7.2.1.14.5 HLL_TO_BASE64

描述

将一个 HLL 类型转换为一个 base64 编码的字符串。如果输入为 NULL，则返回 NULL。

语法

```
HLL_TO_BASE64(<hll_input>)
```

参数

参数	说明
<hll_input>	HLL 类型数据，如果输入为 NULL，则返回 NULL。

示例

```
select hll_to_base64(NULL);
```

```
+---------------------+
| hll_to_base64(NULL) |
+---------------------+
| NULL                |
+---------------------+
```

```
select hll_to_base64(hll_empty());
```

```
+----------------------------+
| hll_to_base64(hll_empty()) |
+----------------------------+
| AA==                       |
+----------------------------+
```

```
select hll_to_base64(hll_hash('abc'));
```

```
+--------------------------------+
| hll_to_base64(hll_hash('abc')) |
+--------------------------------+
| AQEC5XSzrpDsdw==               |
+--------------------------------+
```

```
select hll_union_agg(hll_from_base64(hll_to_base64(pv))), hll_union_agg(pv) from test_hll;
```

```
+-----------------------------------------------------+-------------------+
| hll_union_agg(hll_from_base64(hll_to_base64(pv))) | hll_union_agg(pv) |
+-----------------------------------------------------+-------------------+
|                                                   3 |                 3 |
+-----------------------------------------------------+-------------------+
```

```
select hll_cardinality(hll_from_base64(hll_to_base64(hll_hash('abc'))));
```

```
+------------------------------------------------------------------+
| hll_cardinality(hll_from_base64(hll_to_base64(hll_hash('abc')))) |
+------------------------------------------------------------------+
|                                                                1 |
+------------------------------------------------------------------+
```

7.2.1.15 系统函数

7.2.1.15.1 CONNECTION_ID

描述

获取当前 sql 客户端的连接编号。

语法

```
CONNECTION_ID()
```

返回值

当前 sql 客户端的连接编号。

举例

```
select connection_id();
```

```
+-----------------+
| connection_id() |
+-----------------+
|             549 |
+-----------------+
```

7.2.1.15.2 CURRENT_CATALOG

描述

获取当前 sql 客户端的连接的 catalog。

语法

```
CURRENT_CATALOG()
```

返回值

当前 sql 客户端的连接的 catalog 名称。

举例

```
select current_catalog();
```

```
+-------------------+
| current_catalog() |
+-------------------+
| internal          |
+-------------------+
```

7.2.1.15.3 CURRENT_USER

描述

获取当前的用户名及其 IP 白名单规则。

语法

```
CURRENT_USER()
```

返回值

返回当前的用户名及其 IP 白名单。

格式： `<user_name>@<ip_white_list>`

举例

- root 用户，无 IP 限制

```
select current_user();
```

```
+----------------+
| current_user() |
+----------------+
| 'root'@'%'     |
+----------------+
```

- doris 用户，IP 白名单为 192.168.*

```
select current_user();
```

```
+---------------------+
| current_user()      |
+---------------------+
| 'doris'@'192.168.%' |
+---------------------+
```

7.2.1.15.4 DATABASE

描述

获取当前 sql 客户端的连接的 database。

别名

- SCHEMA

语法

```
DATABASE()
```

或

```
SCHEMA()
```

返回值

当前 sql 客户端的连接的 database 的名称。

举例

```
select database(),schema();
```

```
+------------+------------+
| database() | database() |
+------------+------------+
| test       | test       |
+------------+------------+
```

7.2.1.15.5 LAST_QUERY_ID

描述

获取当前用户上一次查询的 query id。

语法

```
last_query_id()
```

返回值

返回当前用户上一次查询的 query id。

举例

```
select last_query_id();
```

```
+-----------------------------------+
| last_query_id()                   |
+-----------------------------------+
| 128f913c3ec84a1e-b834bd0262cc090a |
+-----------------------------------+
```

7.2.1.15.6 SESSION_USER

描述

获取 Doris 连接的当前用户名和 IP，兼容 MySQL 协议。

语法

```
session_user()
```

返回值

返回 Doris 连接的当前用户名和 IP。格式： `<user_name>@<ip>`

举例

```
select session_user();
```

```
+----------------------+
| session_user()       |
+----------------------+
| 'root'@'10.244.2.10' |
+----------------------+
```

7.2.1.15.7 USER

描述

获取 Doris 连接的当前用户名和 IP。

语法

```
USER()
```

返回值

返回 Doris 连接的当前用户名和 IP。

格式： `<user_name>@<ip>`

举例

```
select user();
```

```
+----------------------+
| user()               |
+----------------------+
| 'root'@'10.244.2.5' |
+----------------------+
```

7.2.1.15.8 VERSION

描述

无实际意义，兼容 MySQL 协议。

语法

```
VERSION()
```

返回值

兼容 MySQL 协议，固定返回 “5.7.99”。

举例

```
select version();
```

```
+-----------+
| version() |
+-----------+
| 5.7.99    |
+-----------+
```

7.2.1.16 其他函数

7.2.1.16.1 CONVERT_TO

描述

将指定的 VARCHAR 列的字符编码转换为目标字符集，常用于 ORDER BY 子句中对包含中文的列进行按拼音排序。当前仅支持将 <character> 转换为 'gbk' 编码。

语法

```
CONVERT_TO(<column>, <character>)
```

参数

参数	说明
<column>	需要转换字符编码的 VARCHAR 列。
<character>	目标字符集，目前仅支持 'gbk'。

返回值

返回转换编码后的 VARCHAR 值，可用于 ORDER BY 子句中按拼音顺序排序。

举例

```
SELECT * FROM class_test ORDER BY class_name;
```

```
+----------+------------+-------------+
| class_id | class_name | student_ids |
+----------+------------+-------------+
|        6 | asd        | [6]         |
|        7 | qwe        | [7]         |
|        8 | z          | [8]         |
|        2 | 哈         | [2]         |
|        3 | 哦         | [3]         |
|        1 | 啊         | [1]         |
|        4 | 张         | [4]         |
|        5 | 我         | [5]         |
+----------+------------+-------------+
```

```
SELECT * FROM class_test ORDER BY CONVERT_TO(class_name, 'gbk');
```

```
+----------+------------+-------------+
| class_id | class_name | student_ids |
+----------+------------+-------------+
|        6 | asd        | [6]         |
|        7 | qwe        | [7]         |
|        8 | z          | [8]         |
|        1 | 啊         | [1]         |
|        2 | 哈         | [2]         |
|        3 | 哦         | [3]         |
|        5 | 我         | [5]         |
|        4 | 张         | [4]         |
+----------+------------+-------------+
```

7.2.1.16.2 ESQUERY

描述

ESQUERY(<field>, <query_dsl>) 函数用于将无法用 SQL 表达的查询下推到 Elasticsearch 进行过滤。
第一个参数 <field> 用于关联索引，第二个参数 <query_dsl> 为 Elasticsearch 的基本 Query DSL 的 JSON 表达式，JSON 需使用 {} 包裹，并且必须包含且仅包含一个根键，例如 match_phrase、geo_shape、bool 等。

语法

```
ESQUERY(<field>, <query_dsl>)
```

参数

参数	说明
<field>	需要查询的字段，用于关联 Elasticsearch 索引。
<query_dsl>	Elasticsearch Query DSL 的 JSON 表达式，需使用 {} 包裹，并且根键必须唯一（如 match_phrase、geo_shape、bool）。

返回值

返回一个布尔值，表示该文档是否匹配提供的 Elasticsearch 查询 DSL。

举例

```
-- match_phrase 查询:
SELECT * FROM es_table
WHERE ESQUERY(k4, '{
    "match_phrase": {
       "k4": "doris on es"
    }
}');
```

```
-- geo_shape 查询:
SELECT * FROM es_table
WHERE ESQUERY(k4, '{
  "geo_shape": {
     "location": {
        "shape": {
           "type": "envelope",
           "coordinates": [
              [13, 53],
              [14, 52]
           ]
        },
        "relation": "within"
     }
  }
}');
```

7.2.1.16.3 FIELD

描述

返回 <expr> 在参数列表中的位置（基于 1 的索引），常用于 ORDER BY 子句中实现自定义排序。如果 <expr> 不在参数列表中，或 <expr> 为 NULL，则返回 0。在自定义排序中，不在参数列表中的数据会被排到最前面，可通过 asc 或 desc 控制整体排序顺序；对于 NULL 值，可以使用 nulls first 或 nulls last 控制排序顺序。

语法

```
FIELD(<expr>, <param> [, ...])
```

参数

参数	说明
<expr>	要搜索的值。
<param>	用于比较的一系列值。

返回值

返回 <expr> 在 <param> 参数列表中的位置（基于 1 的索引）。如果 <expr> 不存在于参数列表中，或者 <expr> 为 NULL，则返回 0。

举例

```
SELECT k1, k7 FROM baseall WHERE k1 IN (1,2,3) ORDER BY FIELD(k1,2,1,3);
```

```
+------+------------+
| k1   | k7         |
+------+------------+
|    2 | wangyu14   |
|    1 | wangjing04 |
|    3 | yuanyuan06 |
+------+------------+
```

```
SELECT class_name FROM class_test ORDER BY FIELD(class_name, 'Suzi', 'Ben', 'Henry');
```

```
+------------+
| class_name |
+------------+
| Suzi       |
| Suzi       |
| Ben        |
| Ben        |
| Henry      |
| Henry      |
+------------+
```

```
SELECT class_name FROM class_test ORDER BY FIELD(class_name, 'Suzi', 'Ben', 'Henry') DESC;
```

```
+------------+
| class_name |
+------------+
| Henry      |
| Henry      |
| Ben        |
| Ben        |
| Suzi       |
| Suzi       |
+------------+
```

```
SELECT class_name FROM class_test ORDER BY FIELD(class_name, 'Suzi', 'Ben', 'Henry') NULLS FIRST;
```

```
+------------+
| class_name |
+------------+
| NULL       |
| Suzi       |
| Suzi       |
| Ben        |
| Ben        |
| Henry      |
| Henry      |
+------------+
```

7.2.1.16.4 G

7.2.1.17 QUANTILE 函数

7.2.1.17.1 QUANTILE_PERCENT

Description

QUANTILE_PERCENT 函数用于计算给定百分比的分位数值。它接受两个参数：一个 quantile_state 列和一个表示百分比的常量浮点数。该函数返回一个浮点数，表示给定百分比位置的分位数值。

Syntax

```
QUANTILE_PERCENT(<quantile_state>, <percent>)
```

Parameters

参数	描述
<quantile_state>	目标列。
<percent>	目标百分比。

Return value

返回一个 Double 类型的分位数值。

Example

```
CREATE TABLE IF NOT EXISTS quantile_state_agg_test (
         `dt` int(11) NULL COMMENT "",
         `id` int(11) NULL COMMENT "",
         `price` quantile_state QUANTILE_UNION NOT NULL COMMENT ""
        ) ENGINE=OLAP
        AGGREGATE KEY(`dt`, `id`)
        COMMENT "OLAP"
```

```
        DISTRIBUTED BY HASH(`dt`) BUCKETS 1
        PROPERTIES ("replication_num" = "1");

INSERT INTO quantile_state_agg_test VALUES(20220201,0, to_quantile_state(1, 2048));

INSERT INTO quantile_state_agg_test VALUES(20220201,1, to_quantile_state(-1, 2048)),
            (20220201,1, to_quantile_state(0, 2048)),(20220201,1, to_quantile_state(1, 2048)),
            (20220201,1, to_quantile_state(2, 2048)),(20220201,1, to_quantile_state(3, 2048));

SELECT dt, id, quantile_percent(quantile_union(price), 0) FROM quantile_state_agg_test GROUP BY
    ↪ dt, id ORDER BY dt, id
```

结果为

```
+----------+------+---------------------------------------------+
| dt       | id   | quantile_percent(quantile_union(price), 0) |
+----------+------+---------------------------------------------+
| 20220201 |    0 |                                           1 |
| 20220201 |    1 |                                          -1 |
+----------+------+---------------------------------------------+
```

7.2.1.17.2 QUANTILE_STATE_EMPTY

Description

返回一个空的 quantile_state 类型列。

Syntax

```
QUANTILE_STATE_EMPTY()
```

Return value

一个空的 quantile_state 类型列。

Example

```
select quantile_percent(quantile_union(quantile_state_empty()), 0)
```

结果为

```
+-----------------------------------------------------------+
| quantile_percent(quantile_union(quantile_state_empty()), 0) |
+-----------------------------------------------------------+
|                                                      NULL |
+-----------------------------------------------------------+
```

7.2.1.17.3 TO_QUANTILE_STATE

Description

此函数将数值类型转化成 QUANTILE_STATE 类型。compression 参数是可选项，可设置范围是 [2048, 10000]，值越大，后续分位数近似计算的精度越高，内存消耗越大，计算耗时越长。compression 参数未指定或设置的值在 [2048, 10000] 范围外，以 2048 的默认值运行

Syntax

```
TO_QUANTILE_STATE(<raw_data> <compression>)
```

Parameters

参数	描述
<raw_data>	目标列。
<compression>	压缩阈值。

Return value

转换之后的 QUANTILE_STATE 类型的列。

Example

```
CREATE TABLE IF NOT EXISTS ${tableName_21} (
          `dt` int(11) NULL COMMENT "",
          `id` int(11) NULL COMMENT "",
          `price` quantile_state QUANTILE_UNION NOT NULL COMMENT ""
        ) ENGINE=OLAP
        AGGREGATE KEY(`dt`, `id`)
        COMMENT "OLAP"
        DISTRIBUTED BY HASH(`dt`) BUCKETS 1
        PROPERTIES ("replication_num" = "1");

INSERT INTO quantile_state_agg_test VALUES(20220201,0, to_quantile_state(1, 2048));

INSERT INTO quantile_state_agg_test VALUES(20220201,1, to_quantile_state(-1, 2048)),
            (20220201,1, to_quantile_state(0, 2048)),(20220201,1, to_quantile_state(1, 2048)),
            (20220201,1, to_quantile_state(2, 2048)),(20220201,1, to_quantile_state(3, 2048));

SELECT dt, id, quantile_percent(quantile_union(price), 0) FROM quantile_state_agg_test GROUP BY
    ↪ dt, id ORDER BY dt, id
```

结果为

```
+----------+------+-----------------------------------------+
| dt       | id   | quantile_percent(quantile_union(price), 0) |
+----------+------+-----------------------------------------+
| 20220201 |    0 |                                       1 |
```

```
| 20220201 |    1 |                                              -1 |
+----------+------+---------------------------------------------+
```

7.2.1.18 条件函数

7.2.1.18.1 COALESCE

描述

返回参数列表中从左到右第一个非空表达式。如果所有参数都为 NULL，则返回 NULL。

语法

```
COALESCE(<expr> [, ...])
```

参数

参数	说明
<expr>	需要逐个检查的表达式序列，所有表达式必须具有兼容的数据类型。

返回值

参数列表中第一个非空表达式。如果所有参数都为 NULL，则返回 NULL。

示例

```
SELECT COALESCE(NULL, '1111', '0000');
```

```
+--------------------------------+
| coalesce(NULL, '1111', '0000') |
+--------------------------------+
| 1111                           |
+--------------------------------+
```

7.2.1.18.2 GREATEST

描述

比较多个表达式的大小，并返回其中的最大值。如果任意参数为 NULL，则返回 NULL。

语法

```
GREATEST(<expr> [, ...])
```

参数

参数	描述
<expr>	需要比较的多个表达式，支持 TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE、STRING、DATETIME 和 DECIMAL 类型。

返回值

- 返回给定表达式中的最大值。
- 如果任意参数为 NULL，则返回 NULL。

示例

```
SELECT GREATEST(-1, 0, 5, 8);
```

```
+----------------------+
| GREATEST(-1, 0, 5, 8) |
+----------------------+
|                    8 |
+----------------------+
```

```
SELECT GREATEST(-1, 0, 5, NULL);
```

```
+--------------------------+
| GREATEST(-1, 0, 5, NULL) |
+--------------------------+
| NULL                     |
+--------------------------+
```

```
SELECT GREATEST(6.3, 4.29, 7.6876);
```

```
+----------------------------+
| GREATEST(6.3, 4.29, 7.6876) |
+----------------------------+
|                     7.6876 |
+----------------------------+
```

```
SELECT GREATEST('2022-02-26 20:02:11', '2020-01-23 20:02:11', '2020-06-22 20:02:11');
```

```
+-------------------------------------------------------------------------------+
| GREATEST('2022-02-26 20:02:11', '2020-01-23 20:02:11', '2020-06-22 20:02:11') |
+-------------------------------------------------------------------------------+
| 2022-02-26 20:02:11                                                           |
+-------------------------------------------------------------------------------+
```

7.2.1.18.3 IF

描述

如果表达式 <condition> 成立，则返回 <value_true>；否则返回 <value_false_or_null>。
返回类型：<value_true> 表达式的结果类型。

语法

```
IF(<condition>, <value_true>, <value_false_or_null>)
```

参数

参数	说明
<condition>	用于判断的布尔表达式。
<value_true>	当 <condition> 为真时返回的值。
<value_false_or_null>	当 <condition> 为假时返回的值。

举例

```
SELECT user_id, IF(user_id = 1, "true", "false") AS test_if FROM test;
```

```
+---------+---------+
| user_id | test_if |
+---------+---------+
| 1       | true    |
| 2       | false   |
+---------+---------+
```

7.2.1.18.4 IFNULL

描述

如果 <expr1> 的值不为 NULL，则返回 <expr1>；否则返回 <expr2>。

别名

- NVL

语法

```
IFNULL(<expr1>, <expr2>)
```

参数

参数	描述
<expr1> <expr2>	需要判断是否为 NULL 的表达式。 <expr1> 为 NULL 时返回的值。

返回值

- 如果 <expr1> 不为 NULL，则返回 <expr1>。
- 否则，返回 <expr2>。

举例

```
SELECT IFNULL(1, 0);
```

```
+--------------+
| IFNULL(1, 0) |
+--------------+
|            1 |
+--------------+
```

```
SELECT IFNULL(NULL, 10);
```

```
+------------------+
| IFNULL(NULL, 10) |
+------------------+
|               10 |
+------------------+
```

7.2.1.18.5 LEAST

描述

比较多个表达式的大小，返回其中的最小值。如果任意参数为 NULL，则返回 NULL。

语法

```
LEAST(<expr> [, ...])
```

参数

参数	描述
<expr>	需要进行比较的表达式。支持的类型包括 TINYINT、SMALLINT、INT、BIGINT、LARGEINT、FLOAT、DOUBLE、STRING、DATETIME 和 DECIMAL。

返回值

- 返回多个输入表达式中的最小值。
- 如果任意参数为 NULL，则返回 NULL。

示例

```
SELECT LEAST(-1, 0, 5, 8);
```

```
+--------------------+
| LEAST(-1, 0, 5, 8) |
+--------------------+
|                 -1 |
+--------------------+
```

```
SELECT LEAST(-1, 0, 5, NULL);
```

```
+-----------------------+
| LEAST(-1, 0, 5, NULL) |
+-----------------------+
| NULL                  |
+-----------------------+
```

```
SELECT LEAST(6.3, 4.29, 7.6876);
```

```
+--------------------------+
| LEAST(6.3, 4.29, 7.6876) |
+--------------------------+
|                     4.29 |
+--------------------------+
```

```
SELECT LEAST('2022-02-26 20:02:11', '2020-01-23 20:02:11', '2020-06-22 20:02:11');
```

```
+------------------------------------------------------------------------------+
| LEAST('2022-02-26 20:02:11', '2020-01-23 20:02:11', '2020-06-22 20:02:11') |
+------------------------------------------------------------------------------+
| 2020-01-23 20:02:11                                                          |
+------------------------------------------------------------------------------+
```

7.2.1.18.6 NOT_NULL_OR_EMPTY

描述

not_null_or_empty 函数用于判断给定的值是否为非 NULL 且非空。如果输入值不为 NULL 且不为空，则返回 true；否则返回 false。

语法

```
NOT_NULL_OR_EMPTY (<str>)
```

参数

参数	说明
<str>	字符串类型

返回值

如果字符串为空字符串或者 NULL，返回 false；否则返回 true。

举例

```
select not_null_or_empty(null);
```

```
+-------------------------+
| not_null_or_empty(NULL) |
+-------------------------+
|                       0 |
+-------------------------+
```

```
select not_null_or_empty("");
```

```
+-----------------------+
| not_null_or_empty('') |
+-----------------------+
|                     0 |
+-----------------------+
```

```
select not_null_or_empty("a");
```

```
+------------------------+
| not_null_or_empty('a') |
+------------------------+
|                      1 |
+------------------------+
```

7.2.1.18.7 NULL_OR_EMPTY

描述

null_or_empty 函数用于判断给定的值是否为 NULL 或空。如果输入值为 NULL 或空字符串，则返回 true，否则返回 false。

语法

```
NULL_OR_EMPTY (<str>)
```

参数

参数	说明
<str>	字符串类型

返回值

如果字符串为空字符串或者 NULL，返回 true；否则返回 false。

示例

```
select null_or_empty(null);
```

```
+----------------------+
| null_or_empty(NULL) |
+----------------------+
|                    1 |
+----------------------+
```

```
select null_or_empty("");
```

```
+-------------------+
| null_or_empty('') |
+-------------------+
|                 1 |
+-------------------+
```

```
select null_or_empty("a");
```

```
+--------------------+
| null_or_empty('a') |
+--------------------+
|                  0 |
+--------------------+
```

7.2.1.18.8 NULLIF

描述

如果两个输入值相等，则返回 NULL；否则返回第一个输入值。该函数等价于以下 CASE WHEN 表达式：

```
CASE
    WHEN <expr1> = <expr2> THEN NULL
    ELSE <expr1>
END
```

语法

```
NULLIF(<expr1>, <expr2>)
```

参数

参数	说明
<expr1>	需要进行比较的第一个输入值。
<expr2>	需要与第一个输入值进行比较的第二个值。

返回值

- 如果 <expr1> 等于 <expr2>，则返回 NULL。
- 否则，返回 <expr1> 的值。

举例

```
SELECT NULLIF(1, 1);
```

```
+--------------+
| NULLIF(1, 1) |
+--------------+
|         NULL |
+--------------+
```

```
SELECT NULLIF(1, 0);
```

```
+--------------+
| NULLIF(1, 0) |
+--------------+
|            1 |
+--------------+
```

7.2.2 聚合函数

7.2.2.1 ANY_VALUE

7.2.2.1.1 描述

返回分组中表达式或列的任意一个值。如果存在非 NULL 值，返回任意非 NULL 值，否则返回 NULL。

7.2.2.1.2 别名

- ANY

7.2.2.1.3 语法

```
ANY_VALUE(<expr>)
```

7.2.2.1.4 参数

参数	说明
<expr>	要聚合的列或表达式

7.2.2.1.5 返回值

如果存在非 NULL 值，返回任意非 NULL 值，否则返回 NULL。

7.2.2.1.6 举例

```
select id, any_value(name) from cost2 group by id;
```

```
+------+-------------------+
| id   | any_value(`name`) |
+------+-------------------+
|    3 | jack              |
|    2 | jack              |
+------+-------------------+
```

7.2.2.2 APPROX_COUNT_DISTINCT

7.2.2.2.1 描述

APPROX_COUNT_DISTINCT 函数基于 HyperLogLog 算法实现，使用固定大小的内存估算列基数。该算法基于尾部零分布假设进行计算，具体精确程度取决于数据分布。基于 Doris 使用的固定桶大小，该算法相对标准误差为 0.8125% 更详细具体的分析，详见相关论文

7.2.2.2.2 语法

```
APPROX_COUNT_DISTINCT(<expr>)
```

7.2.2.2.3 参数说明

参数	说明
<expr>	需要获取值的表达式

7.2.2.2.4 返回值

返回 BIGINT 类型的值。

7.2.2.2.5 举例

```
select approx_count_distinct(query_id) from log_statis group by datetime;
```

```
+-----------------+
| approx_count_distinct(`query_id`) |
+-----------------+
| 17721           |
+-----------------+
```

7.2.2.3 ARRAY_AGG

7.2.2.3.1 描述

将一列中的值（包括空值 null）串联成一个数组，可以用于多行转一行（行转列）。

7.2.2.3.2 语法

```
ARRAY_AGG(<col>)
```

7.2.2.3.3 参数

参数	说明
<col>	确定要放入数组的值的表达式（通常是列名）

7.2.2.3.4 返回值

返回 ARRAY 类型的值，特殊情况：

- 数组中元素不保证顺序。
- 返回转换生成的数组。数组中的元素类型与 col 类型一致。

7.2.2.3.5 举例

```
select * from test_doris_array_agg;
```

```
+------+------+

| c1   | c2   |

+------+------+

|    1 | a    |

|    1 | b    |

|    2 | c    |

|    2 | NULL |

|    3 | NULL |

+------+------+
```

```
select c1, array_agg(c2) from test_doris_array_agg group by c1;
```

```
+------+-----------------+

| c1   | array_agg(`c2`) |

+------+-----------------+

|    1 | ["a","b"]       |

|    2 | [NULL,"c"]      |
```

```
|    3 | [NULL]          |

+------+-----------------+
```

7.2.2.4 AVG

7.2.2.4.1 描述

计算指定列或表达式的所有非 NULL 值的平均值。

7.2.2.4.2 语法

```
AVG([DISTINCT] <expr>)
```

7.2.2.4.3 参数

参数	说明
<expr>	是一个表达式或列，通常是一个数值列或者能够转换为数值的表达式
[DISTINCT]	是一个可选的关键字，表示对 expr 中的重复值进行去重后再计算平均值

7.2.2.4.4 返回值

返回所选列或表达式的平均值，如果组内的所有记录均为 NULL，则该函数返回 NULL

7.2.2.4.5 举例

```
SELECT datetime, AVG(cost_time) FROM log_statis group by datetime;
```

```
+---------------------+--------------------+
| datetime            | avg(`cost_time`)   |
+---------------------+--------------------+
| 2019-07-03 21:01:20 | 25.827794561933533 |
+---------------------+--------------------+
```

```
SELECT datetime, AVG(distinct cost_time) FROM log_statis group by datetime;
```

```
+---------------------+---------------------------+
| datetime            | avg(DISTINCT `cost_time`) |
+---------------------+---------------------------+
| 2019-07-04 02:23:24 |        20.666666666666668 |
+---------------------+---------------------------+
```

7.2.2.5 AVG_WEIGHTED

7.2.2.5.1 描述

计算加权算术平均值，即返回结果为：所有对应数值和权重的乘积相累加，除总的权重和。如果所有的权重和等于 0, 将返回 NaN。

7.2.2.5.2 语法

```
AVG_WEIGHTED(<x>, <weight>)
```

7.2.2.5.3 参数

参数	说明
<x>	是需要计算平均值的数值表达式，可以是一个列名、常量或复杂的数值表达式
<weight>	是一个数值表达式，通常可以是一个列名、常量或其他数值计算结果

7.2.2.5.4 返回值

所有对应数值和权重的乘积相累加，除总的权重和，如果所有的权重和等于 0, 将返回 NaN。

7.2.2.5.5 举例

```
select k1,k2 from test_doris_avg_weighted;
```

```
+------+------+
| k1   | k2   |
+------+------+
|   10 |  100 |
|   20 |  200 |
|   30 |  300 |
|   40 |  400 |
+------+------+
```

```
select avg_weighted(k2,k1) from test_doris_avg_weighted;
```

```
+--------------------------------------+
| avg_weighted(k2, cast(k1 as DOUBLE)) |
+--------------------------------------+
|                                  300 |
+--------------------------------------+
```

7.2.2.6 BITMAP_AGG

7.2.2.6.1 描述

聚合某列的值（不包括任何空值）返回一行 bitmap 值，即多行转一行。

7.2.2.6.2 语法

```
BITMAP_AGG(<expr>)
```

7.2.2.6.3 参数

参数	说明
< ↪ expr ↪ > ↪	待合并数值的列或表达式，expr 的类型需要为 TINYINT,SMALLINT,INT,LA 和 BIG-INT 类型，也支持可以转化为以上类型的 VAR-CHAR。

7.2.2.6.4 返回值

返回 BITMAP 类型的值。特殊情况：

- 如果某个值小于 0 或者大于 18446744073709551615，该值会被忽略，不会合并到 Bitmap 中

7.2.2.6.5 举例

```
select * from test_bitmap_agg;
```

```
+------+------+------+------+------+-------------+----------------------+
| id   | k0   | k1   | k2   | k3   | k4          | k5                   |
+------+------+------+------+------+-------------+----------------------+
|    1 |   10 | 110  |   11 |  300 | 10000000000 | 0                    |
|    2 |   20 | 120  |   21 |  400 | 20000000000 | 200000000000000      |
|    3 |   30 | 130  |   31 |  350 | 30000000000 | 300000000000000      |
|    4 |   40 | 140  |   41 |  500 | 40000000000 | 18446744073709551616 |
|    5 |   50 | 150  |   51 |  250 | 50000000000 | 18446744073709551615 |
|    6 |   60 | 160  |   61 |  600 | 60000000000 | -1                   |
|    7 |   60 | 160  |  120 |  600 | 60000000000 | NULL                 |
+------+------+------+------+------+-------------+----------------------+
```

```
select bitmap_to_string(bitmap_agg(k0)) from test_bitmap_agg;
```

```
+----------------------------------+
| bitmap_to_string(bitmap_agg(k0)) |
+----------------------------------+
| 10,20,30,40,50,60                |
+----------------------------------+
```

```
select bitmap_to_string(bitmap_agg(k5)) from test_bitmap_agg;
```

```
+-----------------------------------------------------------+
| bitmap_to_string(bitmap_agg(cast(k5 as BIGINT)))          |
+-----------------------------------------------------------+
| 0,200000000000000,300000000000000,18446744073709551615 |
+-----------------------------------------------------------+
```

7.2.2.7 BITMAP_INTERSECT

7.2.2.7.1 描述

聚合函数，用于计算分组后的 bitmap 交集。常见使用场景如：计算用户留存率。

7.2.2.7.2 语法

```
BITMAP_INTERSECT(BITMAP <value>)
```

7.2.2.7.3 参数

参数	说明
<value>	支持 bitmap 的数据类型

7.2.2.7.4 返回值

返回值的数据类型为 BITMAP。

7.2.2.7.5 举例

表结构

```
KeysType: AGG_KEY
Columns: tag varchar, date datetime, user_id bitmap bitmap_union
```

求今天和昨天不同 tag 下的用户留存

```
select tag, bitmap_intersect(user_id) from (select tag, date, bitmap_union(user_id) user_id from
    ↪ table where date in ('2020-05-18', '2020-05-19') group by tag, date) a group by tag;
```

和 bitmap_to_string 函数组合使用可以获取交集的具体数据

求今天和昨天不同 tag 下留存的用户都是哪些

```
select tag, bitmap_to_string(bitmap_intersect(user_id)) from (select tag, date, bitmap_union(user
    ↪ _id) user_id from table where date in ('2020-05-18', '2020-05-19') group by tag, date) a
    ↪ group by tag;
```

7.2.2.8 BITMAP_UNION

7.2.2.8.1 描述

计算输入 Bitmap 的并集，返回新的 bitmap

7.2.2.8.2 语法

```
BITMAP_UNION(<expr>)
```

7.2.2.8.3 参数

参数	说明
<expr>	支持 BITMAP 的数据类型

7.2.2.8.4 返回值

返回值的数据类型为 BITMAP。

7.2.2.8.5 举例

```
select dt,page,bitmap_to_string(user_id) from pv_bitmap;
```

```
+------+------+----------------------------+
| dt   | page | bitmap_to_string(user_id) |
+------+------+----------------------------+
|    1 | 100  | 100,200,300                |
|    2 | 200  | 300                        |
+------+------+----------------------------+
```

计算 user_id 的去重值：

```
select bitmap_count(bitmap_union(user_id)) from pv_bitmap;
```

```
+-------------------------------------+
| bitmap_count(bitmap_union(user_id)) |
+-------------------------------------+
|                                   3 |
+-------------------------------------+
```

Create table

建表时需要使用聚合模型，数据类型是 bitmap , 聚合函数是 bitmap_union

```
CREATE TABLE `pv_bitmap` (
  `dt` int(11) NULL COMMENT "",
  `page` varchar(10) NULL COMMENT "",
  `user_id` bitmap BITMAP_UNION NULL COMMENT ""
) ENGINE=OLAP
AGGREGATE KEY(`dt`, `page`)
COMMENT "OLAP"
DISTRIBUTED BY HASH(`dt`) BUCKETS 2;
```

注：当数据量很大时，最好为高频率的 bitmap_union 查询建立对应的 rollup 表

```
ALTER TABLE pv_bitmap ADD ROLLUP pv (page, user_id);
```

Data Load

TO_BITMAP(expr) : 将 0 ~ 18446744073709551615 的 unsigned bigint 转为 bitmap

BITMAP_EMPTY(): 生成空 bitmap 列，用于 insert 或导入的时填充默认值

BITMAP_HASH(expr)或者BITMAP_HASH64(expr): 将任意类型的列通过 Hash 的方式转为 bitmap

Stream Load

```
cat data | curl --location-trusted -u user:passwd -T - -H "columns: dt,page,user_id, user_id=to_
    ↪ bitmap(user_id)"   http://host:8410/api/test/testDb/_stream_load
```

```
cat data | curl --location-trusted -u user:passwd -T - -H "columns: dt,page,user_id, user_id=
    ↪ bitmap_hash(user_id)"   http://host:8410/api/test/testDb/_stream_load
```

```
cat data | curl --location-trusted -u user:passwd -T - -H "columns: dt,page,user_id, user_id=
    ↪ bitmap_empty()"   http://host:8410/api/test/testDb/_stream_load
```

Insert Into

id2 的列类型是 bitmap

```
insert into bitmap_table1 select id, id2 from bitmap_table2;
```

id2 的列类型是 bitmap

```
INSERT INTO bitmap_table1 (id, id2) VALUES (1001, to_bitmap(1000)), (1001, to_bitmap(2000));
```

id2 的列类型是 bitmap

```
insert into bitmap_table1 select id, bitmap_union(id2) from bitmap_table2 group by id;
```

id2 的列类型是 int

```
insert into bitmap_table1 select id, to_bitmap(id2) from table;
```

id2 的列类型是 String

```
insert into bitmap_table1 select id, bitmap_hash(id_string) from table;
```

7.2.2.9 BITMAP_UNION_COUNT

7.2.2.9.1 描述

计算输入 Bitmap 的并集，返回其基数

7.2.2.9.2 语法

```
BITMAP_UNION_COUNT(<expr>)
```

7.2.2.9.3 参数

参数	说明
<expr>	支持 BITMAP 的数据类型

7.2.2.9.4 返回值

返回 Bitmap 并集的大小，即去重后的元素个数

7.2.2.9.5 举例

```
select dt,page,bitmap_to_string(user_id) from pv_bitmap;
```

```
+------+------+----------------------------+
| dt   | page | bitmap_to_string(user_id) |
+------+------+----------------------------+
|    1 | 100  | 100,200,300                |
|    2 | 200  | 300                        |
+------+------+----------------------------+
```

计算 user_id 的去重值：

```
select bitmap_union_count(user_id) from pv_bitmap;
```

```
+-------------------------------------+
| bitmap_count(bitmap_union(user_id)) |
+-------------------------------------+
|                                   3 |
+-------------------------------------+
```

7.2.2.10 BITMAP-UNION-INT

7.2.2.10.1 描述

计算 TINYINT,SMALLINT 和 INT 类型的列中不同值的个数，返回值和 COUNT(DISTINCT expr) 相同

7.2.2.10.2 语法

```
BITMAP_UNION_INT(<expr>)
```

7.2.2.10.3 参数

参数	说明
<expr>	支持 TINYINT，SMALLINT 和 INT 类型的列或列表达式

7.2.2.10.4 返回值

返回列中不同值的个数

7.2.2.10.5 举例

```
select dt,page,bitmap_to_string(user_id) from pv_bitmap;
```

```
+------+------+----------------------------+
| dt   | page | bitmap_to_string(user_id) |
+------+------+----------------------------+
|    1 | 100  | 100,200,300                |
|    1 | 300  | 300                        |
|    2 | 200  | 300                        |
+------+------+----------------------------+
```

```
select bitmap_union_int(dt) from pv_bitmap;
```

```
+----------------------+
| bitmap_union_int(dt) |
+----------------------+
|                    2 |
+----------------------+
```

7.2.2.11 COLLECT_LIST

7.2.2.11.1 描述

聚合函数，用于将某一列的所有值聚集成一个数组

7.2.2.11.2 别名

- GROUP_ARRAY

7.2.2.11.3 语法

```
COLLECT_LIST(<expr> [,<max_size>])
```

7.2.2.11.4 参数

参数	说明
<expr>	要聚合的列或表达式
<max_size>	可选参数，通过设置该参数能够将结果数组的大小限制为 max_size 个元素

7.2.2.11.5 返回值

返回类型是 ARRAY，该数组包含所有值。特殊情况：

- 如果值为 NULL，则会过滤

7.2.2.11.6 举例

```
select k1,k2,k3 from collect_list_test order by k1;
```

```
+------+------------+-------+
| k1   | k2         | k3    |
+------+------------+-------+
|    1 | 2023-01-01 | hello |
|    2 | 2023-01-02 | NULL  |
|    2 | 2023-01-02 | hello |
|    3 | NULL       | world |
|    3 | 2023-01-02 | hello |
|    4 | 2023-01-02 | sql   |
|    4 | 2023-01-03 | sql   |
+------+------------+-------+
```

```
select collect_list(k1),collect_list(k1,3) from collect_list_test;
```

```
+-------------------------+-------------------------+
| collect_list(`k1`)      | collect_list(`k1`,3)    |
+-------------------------+-------------------------+
| [1,2,2,3,3,4,4]         | [1,2,2]                 |
+-------------------------+-------------------------+
```

```
select k1,collect_list(k2),collect_list(k3,1) from collect_list_test group by k1 order by k1;
```

```
+------+-------------------------+-------------------------+
| k1   | collect_list(`k2`)      | collect_list(`k3`,1)    |
+------+-------------------------+-------------------------+
|    1 | [2023-01-01]            | [hello]                 |
|    2 | [2023-01-02,2023-01-02] | [hello]                 |
|    3 | [2023-01-02]            | [world]                 |
|    4 | [2023-01-02,2023-01-03] | [sql]                   |
+------+-------------------------+-------------------------+
```

7.2.2.12 COLLECT_SET

7.2.2.12.1 描述

聚合函数，聚合指定列的所有唯一值，去除重复的元素，并返回一个集合类型的结果。

7.2.2.12.2 别名

- GROUP_UNIQ_ARRAY

7.2.2.12.3 语法

```
COLLECT_SET(<expr> [,<max_size>])
```

7.2.2.12.4 参数

参数	说明
<expr>	要聚合的列或表达式
<max_size>	可选参数，通过设置该参数能够将结果数组的大小限制为 max_size 个元素

7.2.2.12.5 返回值

返回类型是 ARRAY，该数组包含去重后的所有值，特殊情况：

- 如果值为 NULL，则会过滤

7.2.2.12.6 举例

```
select k1,k2,k3 from collect_set_test order by k1;
```

```
+------+------------+-------+
| k1   | k2         | k3    |
+------+------------+-------+
|    1 | 2023-01-01 | hello |
|    2 | 2023-01-01 | NULL  |
|    2 | 2023-01-02 | hello |
|    3 | NULL       | world |
|    3 | 2023-01-02 | hello |
|    4 | 2023-01-02 | doris |
|    4 | 2023-01-03 | sql   |
+------+------------+-------+
```

```
select collect_set(k1),collect_set(k1,2) from collect_set_test;
```

```
+-------------------------+--------------------------+
| collect_set(`k1`)       | collect_set(`k1`,2)      |
+-------------------------+--------------------------+
| [4,3,2,1]               | [1,2]                    |
+-------------------------+--------------------------+
```

```
select k1,collect_set(k2),collect_set(k3,1) from collect_set_test group by k1 order by k1;
```

```
+------+-------------------------+-------------------------+
| k1   | collect_set(`k2`)       | collect_set(`k3`,1)     |
+------+-------------------------+-------------------------+
|    1 | [2023-01-01]            | [hello]                 |
|    2 | [2023-01-01,2023-01-02] | [hello]                 |
|    3 | [2023-01-02]            | [world]                 |
|    4 | [2023-01-02,2023-01-03] | [sql]                   |
+------+-------------------------+-------------------------+
```

7.2.2.13 CORR_WELFORD

7.2.2.13.1 描述

采用 Welford 算法计算两个随机变量的皮尔逊系数，能够有效降低计算误差。

7.2.2.13.2 语法

```
CORR_WELFORD(<expr1>, <expr2>)
```

7.2.2.13.3 参数

参数	说明
<expr1>	Double 表达式（列）
<expr2>	Double 表达式（列）

7.2.2.13.4 返回值

返回值为 DOUBLE 类型，expr1 和 expr2 的协方差，除 expr1 和 expr2 的标准差乘积，特殊情况：

- 如果 expr1 或 expr2 的标准差为 0, 将返回 0。
- 如果 expr1 或者 expr2 某一列为 NULL 时，该行数据不会被统计到最终结果中。

7.2.2.13.5 举例

```
select * from test_corr;
```

```
+------+------+------+
| id   | k1   | k2   |
+------+------+------+
|    1 |   20 |   22 |
```

```
|    1 |   10 |   20 |
|    2 |   36 |   21 |
|    2 |   30 |   22 |
|    2 |   25 |   20 |
|    3 |   25 | NULL |
|    4 |   25 |   21 |
|    4 |   25 |   22 |
|    4 |   25 |   20 |
+------+------+------+
```

```
select id,corr_welford(k1,k2) from test_corr group by id;
```

```
+------+---------------------+
| id   | corr_welford(k1,k2) |
+------+---------------------+
|    2 |  0.4539206495016017 |
|    4 |                   0 |
|    3 |                NULL |
|    1 |                   1 |
+------+---------------------+
```

7.2.2.14 CORR

7.2.2.14.1 描述

计算两个随机变量的皮尔逊系数

7.2.2.14.2 语法

```
CORR(<expr1>, <expr2>)
```

7.2.2.14.3 参数

参数	说明
<expr1>	数值型表达式（列）
<expr2>	数值型表达式（列）

7.2.2.14.4 返回值

返回值为 DOUBLE 类型，expr1 和 expr2 的协方差，除 expr1 和 expr2 的标准差乘积，特殊情况：

- 如果 expr1 或 expr2 的标准差为 0, 将返回 0。
- 如果 expr1 或者 expr2 某一列为 NULL 时，该行数据不会被统计到最终结果中。

7.2.2.14.5 举例

```
select * from test_corr;
```

```
+------+------+------+
| id   | k1   | k2   |
+------+------+------+
|    1 |   20 |   22 |
|    1 |   10 |   20 |
|    2 |   36 |   21 |
|    2 |   30 |   22 |
|    2 |   25 |   20 |
|    3 |   25 | NULL |
|    4 |   25 |   21 |
|    4 |   25 |   22 |
|    4 |   25 |   20 |
+------+------+------+
```

```
select id,corr(k1,k2) from test_corr group by id;
```

```
+------+--------------------+
| id   | corr(k1, k2)       |
+------+--------------------+
|    4 |                  0 |
|    1 |                  1 |
|    3 |               NULL |
|    2 | 0.4539206495016019 |
+------+--------------------+
```

7.2.2.15 COUNT

7.2.2.15.1 描述

返回指定列的非 NULL 记录数，或者记录总数

7.2.2.15.2 语法

```
COUNT(DISTINCT <expr> [,<expr>,...])
COUNT(*)
COUNT(<expr>)
```

7.2.2.15.3 参数

参数	说明
<expr>	条件表达式（列名）

7.2.2.15.4 返回值

返回值为数值类型。如果 expr 为 NULL，则不参数统计

7.2.2.15.5 举例

```
select * from test_count;
```

```
+------+------+------+
| id   | name | sex  |
+------+------+------+
|    1 | 1    |    1 |
|    2 | 2    |    1 |
|    3 | 3    |    1 |
|    4 | 0    |    1 |
|    4 | 4    |    1 |
|    5 | NULL |    1 |
+------+------+------+
```

```
select count(*) from test_count;
```

```
+----------+
| count(*) |
+----------+
|        6 |
+----------+
```

```
select count(name) from test_insert;
```

```
+-------------+
| count(name) |
+-------------+
|           5 |
+-------------+
```

```
select count(distinct sex) from test_insert;
```

```
+----------------------+
| count(DISTINCT sex) |
+----------------------+
|                    1 |
+----------------------+
```

```
select count(distinct id,sex) from test_insert;
```

```
+-------------------------+
| count(DISTINCT id, sex) |
+-------------------------+
|                       5 |
+-------------------------+
```

7.2.2.16 COUNT_BY_ENUM

7.2.2.16.1 描述

将列中数据看作枚举值，统计每个枚举值的个数。返回各个列枚举值的个数，以及非 null 值的个数与 null 值的个数。

7.2.2.16.2 语法

```
COUNT_BY_ENUM(<expr1>, <expr2>, ... , <exprN>)
```

7.2.2.16.3 参数

参数	说明
<expr1>	至少填写一个输入。值为字符串（STRING）类型的列

7.2.2.16.4 返回值

返回一个 JSONArray 字符串。

例如：

```
[{
  "cbe": {
    "F": 100,
    "M": 99
  },
  "notnull": 199,
  "null": 1,
  "all": 200
}, {
  "cbe": {
    "20": 10,
    "30": 5,
    "35": 1
```

```
  },
  "notnull": 16,
  "null": 184,
  "all": 200
}, {
  "cbe": {
    "北京": 10,
    "上海": 9,
    "广州": 20,
    "深圳": 30
  },
  "notnull": 69,
  "null": 131,
  "all": 200
}]
```

说明：返回值为一个 JSON array 字符串，内部对象的顺序是输入参数的顺序。* cbe：根据枚举值统计非 null 值的统计结果 * notnull：非 null 的个数 * null：null 值个数 * all：总数，包括 null 值与非 null 值

7.2.2.16.5 举例

```
CREATE TABLE count_by_enum_test(
                `id` varchar(1024) NULL,
                `f1` text REPLACE_IF_NOT_NULL NULL,
                `f2` text REPLACE_IF_NOT_NULL NULL,
                `f3` text REPLACE_IF_NOT_NULL NULL
                )
AGGREGATE KEY(`id`)
DISTRIBUTED BY HASH(id) BUCKETS 3
PROPERTIES (
    "replication_num" = "1"
);
```

```
INSERT into count_by_enum_test (id, f1, f2, f3) values
                                        (1, "F", "10", "北京"),
                                        (2, "F", "20", "北京"),
                                        (3, "M", NULL, "上海"),
                                        (4, "M", NULL, "上海"),
                                        (5, "M", NULL, "广州");
```

```
SELECT * from count_by_enum_test;
```

```
+------+------+------+--------+
| id   | f1   | f2   | f3     |
+------+------+------+--------+
```

```
| 2    | F    | 20   | 北京   |
| 3    | M    | NULL | 上海   |
| 4    | M    | NULL | 上海   |
| 5    | M    | NULL | 广州   |
| 1    | F    | 10   | 北京   |
+------+------+------+--------+
```

```
select count_by_enum(f1) from count_by_enum_test;
```

```
+-----------------------------------------------------+
| count_by_enum(`f1`)                                 |
+-----------------------------------------------------+
| [{"cbe":{"M":3,"F":2},"notnull":5,"null":0,"all":5}] |
+-----------------------------------------------------+
```

```
select count_by_enum(f2) from count_by_enum_test;
```

```
+-----------------------------------------------------+
| count_by_enum(`f2`)                                  |
+-----------------------------------------------------+
| [{"cbe":{"10":1,"20":1},"notnull":2,"null":3,"all":5}] |
+-----------------------------------------------------+
```

```
select count_by_enum(f1,f2,f3) from count_by_enum_test;
```

```
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| count_by_enum(`f1`, `f2`, `f3`)
    ↪
    ↪ |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
| [{"cbe":{"M":3,"F":2},"notnull":5,"null":0,"all":5},{"cbe":{"20":1,"10":1},"notnull":2,"null
    ↪ ":3,"all":5},{"cbe":{"广州":1,"上海":2,"北京":2},"notnull":5,"null":0,"all":5}]       |
+-------------------------------------------------------------------------------------------------------------------------
    ↪
```

7.2.2.17 COVAR

7.2.2.17.1 描述

计算两个数值型变量之间的协方差

7.2.2.17.2 别名

- COVAR_POP

7.2.2.17.3 语法

```
COVAR(<expr1>, <expr2>)
```

7.2.2.17.4 参数

参数	说明
<expr1>	数值型表达式或列
<expr2>	数值型表达式或列

7.2.2.17.5 返回值

返回 expr1 和 expr2 的协方差值，特殊情况：

- 如果 expr1 或者 expr2 某一列为 NULL 时，该行数据不会被统计到最终结果中。

7.2.2.17.6 举例

```
select covar(x,y) from baseall;
```

```
+---------------------+
| covar(x, y)          |
+---------------------+
| 0.89442719099991586 |
+---------------------+
```

7.2.2.18 COVAR_SAMP

7.2.2.18.1 描述

计算两个数值型变量之间的样本协方差

7.2.2.18.2 语法

```
COVAR_SAMP(<expr1>, <expr2>)
```

7.2.2.18.3 参数

参数	说明
<expr1>	数值型表达式或列
<expr2>	数值型表达式或列

7.2.2.18.4 返回值

返回 expr1 和 expr2 的样本协方差，特殊情况：

- 如果 expr1 或者 expr2 某一列为 NULL 时，该行数据不会被统计到最终结果中。

7.2.2.18.5 举例

```
select covar_samp(x,y) from baseall;
```

```
+---------------------+
| covar_samp(x, y)    |
+---------------------+
| 0.89442719099991586 |
+---------------------+
```

7.2.2.19 GROUP_ARRAY_INTERSECT

7.2.2.19.1 描述

求出所有行中输入数组中的交集元素，返回一个新的数组

7.2.2.19.2 语法

```
GROUP_ARRAY_INTERSECT(<expr>)
```

7.2.2.19.3 参数

参数	说明
<expr>	需要求交集的数组列或数组值

7.2.2.19.4 返回值

返回一个包含交集结果的数组

7.2.2.19.5 举例

```
select c_array_string from group_array_intersect_test where id in (18, 20);
```

```
+------+---------------------------+
| id   | col                       |
+------+---------------------------+
|    1 | ["a", "b", "c", "d", "e"] |
|    2 | ["a", "b"]                |
|    3 | ["a", null]               |
+------+---------------------------+
```

```
select group_array_intersect(col) from group_array_intersect_test;
```

```
+----------------------------+
| group_array_intersect(col) |
+----------------------------+
| ["a"]                      |
+----------------------------+
```

7.2.2.20 GROUP_BIT_AND

7.2.2.20.1 描述

对单个整数列或表达式中的所有值执行按位 and 运算

7.2.2.20.2 语法

```
GROUP_BIT_AND(<expr>)
```

7.2.2.20.3 参数

参数	说明
<expr>	支持所有 INT 类型

7.2.2.20.4 返回值

返回一个整数值

7.2.2.20.5 举例

```
select * from group_bit;
```

```
+-------+
| value |
+-------+
```

```
|     3 |
|     1 |
|     2 |
|     4 |
+-------+
```

```
select group_bit_and(value) from group_bit;
```

```
+------------------------+
| group_bit_and(`value`) |
+------------------------+
|                      0 |
+------------------------+
```

7.2.2.21 GROUP_BIT_OR

7.2.2.21.1 描述

对单个整数列或表达式中的所有值执行按位 or 运算

7.2.2.21.2 语法

```
GROUP_BIT_OR(<expr>)
```

7.2.2.21.3 参数

参数	说明
<expr>	支持所有 INT 类型

7.2.2.21.4 返回值

返回一个整数值

7.2.2.21.5 举例

```
select * from group_bit;
```

```
+-------+
| value |
+-------+
|     3 |
|     1 |
|     2 |
```

```
|     4 |
+-------+
```

```
mysql> select group_bit_or(value) from group_bit;
```

```
+-----------------------+
| group_bit_or(`value`) |
+-----------------------+
|                     7 |
+-----------------------+
```

7.2.2.22 GROUP_BIT_XOR

7.2.2.22.1 描述

对单个整数列或表达式中的所有值执行按位 xor 运算

7.2.2.22.2 语法

```
GROUP_BIT_XOR(<expr>)
```

7.2.2.22.3 参数

参数	说明
<expr>	支持所有 INT 类型

7.2.2.22.4 返回值

返回一个整数值

7.2.2.22.5 举例

```
select * from group_bit;
```

```
+-------+
| value |
+-------+
|     3 |
|     1 |
|     2 |
|     4 |
+-------+
```

```
select group_bit_xor(value) from group_bit;
```

```
+------------------------+
| group_bit_xor(`value`) |
+------------------------+
|                      4 |
+------------------------+
```

7.2.2.23 GROUP_BITMAP_XOR

7.2.2.23.1 描述

主要用于合并多个 bitmap 的值，并对结果进行按位 xor 计算

7.2.2.23.2 语法

```
GROUP_BITMAP_XOR(<expr>)
```

7.2.2.23.3 参数

参数	说明
<expr>	支持 bitmap 的数据类型

7.2.2.23.4 返回值

返回值的数据类型为 BITMAP。

7.2.2.23.5 举例

```
 select page, bitmap_to_string(user_id) from pv_bitmap;
```

```
+------+-----------------------------+
| page | bitmap_to_string(`user_id`) |
+------+-----------------------------+
| m    | 4,7,8                       |
| m    | 1,3,6,15                    |
| m    | 4,7                         |
+------+-----------------------------+
```

```
select page, bitmap_to_string(group_bitmap_xor(user_id)) from pv_bitmap group by page;
```

```
+------+-----------------------------------------------+
| page | bitmap_to_string(group_bitmap_xor(`user_id`)) |
+------+-----------------------------------------------+
| m    | 1,3,6,8,15                                    |
+------+-----------------------------------------------+
```

7.2.2.24 GROUP_CONCAT

7.2.2.24.1 描述

GROUP_CONCAT 函数将结果集中的多行结果连接成一个字符串

7.2.2.24.2 语法

```
GROUP_CONCAT([DISTINCT] <str>[, <sep>] [ORDER BY { <col_name> | <expr>} [ASC | DESC]])
```

7.2.2.24.3 参数

参数	说明
<str>	必选。需要连接值的表达式
<sep>	可选。字符串之间的连接符号
<col_name>	可选。用于指定排序的列
<expr>	可选。用于指定排序的表达式

7.2.2.24.4 返回值

返回 VARCHAR 类型的数值。

7.2.2.24.5 举例

```
select value from test;
```

```
+-------+
| value |
+-------+
| a     |
| b     |
| c     |
| c     |
+-------+
```

```
select GROUP_CONCAT(value) from test;
```

```
+----------------------+
| GROUP_CONCAT(`value`) |
+----------------------+
| a, b, c, c            |
+----------------------+
```

```
select GROUP_CONCAT(DISTINCT value) from test;
```

```
+----------------------+
| GROUP_CONCAT(`value`) |
+----------------------+
| a, b, c               |
+----------------------+
```

```
select GROUP_CONCAT(value, " ") from test;
```

```
+---------------------------+
| GROUP_CONCAT(`value`, ' ') |
+---------------------------+
| a b c c                   |
+---------------------------+
```

```
select GROUP_CONCAT(value, NULL) from test;
```

```
+---------------------------+
| GROUP_CONCAT(`value`, NULL)|
+---------------------------+
| NULL                      |
+---------------------------+
```

7.2.2.25 GROUPING

7.2.2.25.1 描述

用于在包含 CUBE、ROLLUP 或 GROUPING SETS 的 SQL 语句中，判断某个在 GROUP BY 子句中的列或表达式是否为汇总结果。当结果集中的数据行是由 CUBE、ROLLUP 或 GROUPING SETS 操作产生的汇总行时，该函数返回 1；否则返回 0。GROUPING 函数可在 SELECT、HAVING 和 ORDER BY 子句中使用。

ROLLUP、CUBE 或 GROUPING SETS 操作产生的汇总结果会以 NULL 作为被分组列的值，因此 GROUPING 函数通常用于区分这些 NULL 值与表中实际存在的 NULL 值。

7.2.2.25.2 语法

```
GROUPING( <column_expression> )
```

7.2.2.25.3 参数

参数	说明
<column_expression>	在 GROUP BY 子句中包含的列或表达式。

7.2.2.25.4 返回值

返回 BIGINT 值。若该列或表达式对应的数据行为汇总行，则返回 1；否则返回 0。

7.2.2.25.5 举例

下面的例子使用 camp 列进行分组操作，并统计 occupation 的数量，同时利用 GROUPING 函数区分汇总行与表中实际存在的 NULL 值。

```
CREATE TABLE `roles` (
  role_id       INT,
  occupation    VARCHAR(32),
  camp          VARCHAR(32),
  register_time DATE
)
UNIQUE KEY(role_id)
DISTRIBUTED BY HASH(role_id) BUCKETS 1
PROPERTIES (
  "replication_allocation" = "tag.location.default: 1"
);

INSERT INTO `roles` VALUES
(0, 'who am I', NULL, NULL),
(1, 'mage', 'alliance', '2018-12-03 16:11:28'),
(2, 'paladin', 'alliance', '2018-11-30 16:11:28'),
(3, 'rogue', 'horde', '2018-12-01 16:11:28'),
(4, 'priest', 'alliance', '2018-12-02 16:11:28'),
(5, 'shaman', 'horde', NULL),
(6, 'warrior', 'alliance', NULL),
(7, 'warlock', 'horde', '2018-12-04 16:11:28'),
(8, 'hunter', 'horde', NULL);

SELECT
  camp,
  COUNT(occupation) AS occ_cnt,
  GROUPING(camp) AS grouping
FROM
  `roles`
GROUP BY
  ROLLUP(camp);
```

在上述查询中，结果集中 camp 列出现了两个 NULL 值。其中，第一个 NULL（GROUPING 返回 1）表示该行为 ROLLUP 操作产生的汇总行，其 occ_cnt 为所有 camp 的 occupation 计数；第二个 NULL（GROUPING 返回 0）表示表中实际存在的 NULL 值。

```
+----------+---------+----------+
| camp     | occ_cnt | grouping |
+----------+---------+----------+
| NULL     |       9 |        1 |
| NULL     |       1 |        0 |
| alliance |       4 |        0 |
| horde    |       4 |        0 |
+----------+---------+----------+
```

7.2.2.26 GROUPING_ID

7.2.2.26.1 描述

GROUPING_ID 是一个用于计算分组层级的函数。当 SQL 语句中使用了 GROUP BY 子句时，该函数可以在 SELECT、HAVING 或 ORDER BY 子句中使用，返回一个 BIGINT 值，该值表示各分组列聚合情况对应的二进制位图转换为十进制后的结果。

7.2.2.26.2 语法

```
GROUPING_ID ( <column_expression>[ ,...n ] )
```

7.2.2.26.3 参数

参数	说明
<column_expression>	在 GROUP BY 子句中包含的列或表达式，可以指定多个参数。

7.2.2.26.4 返回值

返回一个 BIGINT 值，表示各分组列的聚合情况对应的二进制位图转换为十进制后的结果。

7.2.2.26.5 示例

示例 A: 识别分组层级

下面的例子按部门和职级统计雇员人数，并使用 GROUPING_ID 函数计算每一行的聚合程度。

```
SELECT
  department,
  CASE
    WHEN GROUPING_ID(department, level) = 0 THEN level
    WHEN GROUPING_ID(department, level) = 1 THEN CONCAT('Total: ', department)
```

```
    WHEN GROUPING_ID(department, level) = 3 THEN 'Total: Company'
    ELSE 'Unknown'
  END AS 'Job Title',
  COUNT(uid) AS 'Employee Count'
FROM employee
GROUP BY ROLLUP(department, level)
ORDER BY GROUPING_ID(department, level) ASC;
```

```
+--------------------+--------------------------+----------------+
| department         | Job Title                | Employee Count |
+--------------------+--------------------------+----------------+
| Board of Directors | Senior                   |              2 |
| Technology         | Senior                   |              3 |
| Sales              | Senior                   |              1 |
| Sales              | Assistant                |              2 |
| Sales              | Trainee                  |              1 |
| Marketing          | Senior                   |              1 |
| Marketing          | Trainee                  |              2 |
| Marketing          | Assistant                |              1 |
| Board of Directors | Total: Board of Directors |              2 |
| Technology         | Total: Technology        |              3 |
| Sales              | Total: Sales             |              4 |
| Marketing          | Total: Marketing         |              4 |
| NULL               | Total: Company           |             13 |
+--------------------+--------------------------+----------------+
```

示例 B: 过滤结果集

下面的例子返回部门中级别为 “Senior” 的雇员统计数据。

```
SELECT
  department,
  CASE
    WHEN GROUPING_ID(department, level) = 0 THEN level
    WHEN GROUPING_ID(department, level) = 1 THEN CONCAT('Total: ', department)
    WHEN GROUPING_ID(department, level) = 3 THEN 'Total: Company'
    ELSE 'Unknown'
  END AS 'Job Title',
  COUNT(uid)
FROM employee
GROUP BY ROLLUP(department, level)
HAVING `Job Title` IN ('Senior');
```

```
+--------------------+-----------+--------------+
| department         | Job Title | count(`uid`) |
+--------------------+-----------+--------------+
```

```
| Board of Directors | Senior    |            2 |
| Technology         | Senior    |            3 |
| Sales              | Senior    |            1 |
| Marketing          | Senior    |            1 |
+--------------------+-----------+--------------+
```

7.2.2.27 HISTOGRAM

7.2.2.27.1 描述

HISTOGRAM（直方图）函数用于描述数据分布情况，它使用“等高”的分桶策略，并按照数据的值大小进行分桶，并用一些简单的数据来描述每个桶，比如落在桶里的值的个数。

7.2.2.27.2 别名

HIST

7.2.2.27.3 语法

```
HISTOGRAM(<expr>[, <num_buckets>])
```

7.2.2.27.4 参数

参数	说明
expr	需要获取第一个值的表达式
num_buckets	可选。用于限制直方图桶（bucket）的数量，默认值 128

7.2.2.27.5 返回值

返回直方图估算后的 JSON 类型的值。特殊情况：- 当参数为 NULL 时，返回 NULL。

7.2.2.27.6 举例

```
SELECT histogram(c_float) FROM histogram_test;
```

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| histogram(`c_float`)
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| {"num_buckets":3,"buckets":[{"lower":"0.1","upper":"0.1","count":1,"pre_sum":0,"ndv":1},...]} |
```

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
```

```
SELECT histogram(c_string, 2) FROM histogram_test;
```

```
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| histogram(`c_string`)
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
| {"num_buckets":2,"buckets":[{"lower":"str1","upper":"str7","count":4,"pre_sum":0,"ndv":3},...]}
    ↪  |
+-----------------------------------------------------------------------------------------------------------------------------
    ↪
```

查询结果说明：

```
{
    "num_buckets": 3,
    "buckets": [
        {
            "lower": "0.1",
            "upper": "0.2",
            "count": 2,
            "pre_sum": 0,
            "ndv": 2
        },
        {
            "lower": "0.8",
            "upper": "0.9",
            "count": 2,
            "pre_sum": 2,
            "ndv": 2
        },
        {
            "lower": "1.0",
            "upper": "1.0",
            "count": 2,
            "pre_sum": 4,
            "ndv": 1
        }
    ]
}
```

字段说明：
- num_buckets：桶的数量
- buckets：直方图所包含的桶
  - lower：桶的上界
  - upper：桶的下界
  - count：桶内包含的元素数量
  - pre_sum：前面桶的元素总量
  - ndv：桶内不同值的个数

> 直方图总的元素数量 = 最后一个桶的元素数量（count）+ 前面桶的元素总量（pre_sum）。

7.2.2.28 HLL_RAW_AGG

7.2.2.28.1 描述

HLL_RAW_AGG 函数是一种聚合函数，主要用于将多个 HyperLogLog 数据结构合并成一个

7.2.2.28.2 别名

- HLL_UNION

7.2.2.28.3 语法

```
HLL_RAW_AGG(<hll>)
```

7.2.2.28.4 参数

参数	说明
<hll>	需要被计算 HyperLogLog 类型表达式

7.2.2.28.5 返回值

返回被聚合后的 HLL 类型。

7.2.2.28.6 举例

```
select HLL_CARDINALITY(HLL_RAW_AGG(uv_set)) from test_uv;
```

```
+-----------------------------------------+
|   HLL_CARDINALITY(HLL_RAW_AGG(`uv_set`)) |
+-----------------------------------------+
|                                   17721 |
```

```
+----------------------------------------+
```

7.2.2.29 HLL_UNION_AGG

7.2.2.29.1 描述

HLL_UNION_AGG 函数是一种聚合函数，主要用于将多个 HyperLogLog 数据结构合并，估算合并后基数的近似值。

7.2.2.29.2 语法

```
hll_union_agg(<hll>)
```

7.2.2.29.3 参数

参数	说明
<hll>	需要被计算 HyperLogLog 类型表达式

7.2.2.29.4 返回值

返回 BIGINT 类型的基数值。

7.2.2.29.5 举例

```
select HLL_UNION_AGG(uv_set) from test_uv;
```

```
+-------------------------+
| HLL_UNION_AGG(`uv_set`) |
+-------------------------+
| 17721                   |
+-------------------------+
```

7.2.2.30 INTERSECT_COUNT

7.2.2.30.1 描述

INTERSECT_COUNT 函数用于计算 Bitmap 数据结构的交集元素的数量。

7.2.2.30.2 语法

```
INTERSECT_COUNT(<bitmap_column>, <column_to_filter>, <filter_values>)
```

7.2.2.30.3 参数

参数	说明
<bitmap_column>	需要获取第一个值的表达式
<column_to_filter>	可选。需要进行过滤的维度列
<filter_values>	可选。过滤维度列的不同取值

7.2.2.30.4 返回值

返回 BIGINT 类型的值。

7.2.2.30.5 举例

```
select dt,bitmap_to_string(user_id) from pv_bitmap where dt in (3,4);
```

```
+------+-----------------------------+
| dt   | bitmap_to_string(`user_id`) |
+------+-----------------------------+
| 4    | 1,2,3                       |
| 3    | 1,2,3,4,5                   |
+------+-----------------------------+
```

```
select intersect_count(user_id,dt,3,4) from pv_bitmap;
```

```
+----------------------------------------+
| intersect_count(`user_id`, `dt`, 3, 4) |
+----------------------------------------+
|                                      3 |
+----------------------------------------+
```

7.2.2.31 KURT,KURT_POP,KURTOSIS

7.2.2.31.1 描述

KURTOSIS 函数用于计算数据的峰度值。此函数使用的公式为第四阶中心矩 / (方差的平方) - 3。

7.2.2.31.2 别名

KURT_POP,KURTOSIS

7.2.2.31.3 语法

```
KURTOSIS(<expr>)
```

7.2.2.31.4 参数说明

参数	说明
<expr>	需要获取值的表达式

7.2.2.31.5 返回值

返回 DOUBLE 类型的值。特殊情况：

- 当方差为零时，返回 NULL

7.2.2.31.6 举例

```
select * from statistic_test;
```

```
+-----+------+------+
| tag | val1 | val2 |
+-----+------+------+
|   1 |  -10 |   -10|
|   2 |  -20 |  NULL|
|   3 |  100 |  NULL|
|   4 |  100 |  NULL|
|   5 | 1000 |  1000|
+-----+------+------+
```

```
select kurt(val1), kurt(val2) from statistic_test;
```

```
+-------------------+--------------------+
| kurt(val1)        | kurt(val2)         |
+-------------------+--------------------+
| 0.162124583734851 | -1.3330994719286338 |
+-------------------+--------------------+
```

```
// 每组只有一行数据，结果为 NULL
select kurt(val1), kurt(val2) from statistic_test group by tag;
```

```
+------------+------------+
| kurt(val1) | kurt(val2) |
+------------+------------+
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
+------------+------------+
```

7.2.2.32 LINEAR_HISTOGRAM

7.2.2.32.1 描述

LINEAR_HISTOGRAM 函数用于描述数据分布情况，它使用“等宽”的分桶策略，并按照数据的值大小进行分桶。

7.2.2.32.2 语法

```
`LINEAR_HISTOGRAM(<expr>, DOUBLE <interval>[, DOUBLE <offset>)`
```

7.2.2.32.3 参数说明

参数	说明
interval	必须。桶的宽度
offset	可选。默认为 0，范围是 [0, interval)

7.2.2.32.4 返回值

返回计算后的 JSON 类型的值。

7.2.2.32.5 举例

```
select linear_histogram(a, 2) from histogram_test;
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| linear_histogram(a, cast(2 as DOUBLE))
    ↪
    ↪ |
+-----------------------------------------------------------------------------------------------------------------
    ↪
| {"num_buckets":6,"buckets":[{"lower":0.0,"upper":2.0,"count":2,"acc_count":2},{"lower":2.0,"
    ↪ upper":4.0,"count":4,"acc_count":6},{"lower":4.0,"upper":6.0,"count":4,"acc_count":10},{"
    ↪ lower":6.0,"upper":8.0,"count":4,"acc_count":14},{"lower":8.0,"upper":10.0,"count":4,"acc
    ↪ _count":18},{"lower":10.0,"upper":12.0,"count":2,"acc_count":20}]} |
+-----------------------------------------------------------------------------------------------------------------
    ↪
```

```
select linear_histogram(a, 2, 1) from histogram_test;
```

```
+-----------------------------------------------------------------------------------------------------------------
    ↪
| linear_histogram(a, cast(2 as DOUBLE), cast(1 as DOUBLE))
    ↪
    ↪ |
```

```
+-----------------------------------------------------------------------------------------------------------------------------------------------------
    ↪
| {"num_buckets":5,"buckets":[{"lower":1.0,"upper":3.0,"count":4,"acc_count":4},{"lower":3.0,"
    ↪ upper":5.0,"count":4,"acc_count":8},{"lower":5.0,"upper":7.0,"count":4,"acc_count":12},{"
    ↪ lower":7.0,"upper":9.0,"count":4,"acc_count":16},{"lower":9.0,"upper":11.0,"count":4,"acc
    ↪ _count":20}]} |
+-----------------------------------------------------------------------------------------------------------------------------------------------------
    ↪
```

字段说明：

- `num_buckets`：桶的数量。
- `buckets`：直方图所包含的桶。
  - `lower`：桶的下界。（包含在内）
  - `upper`：桶的上界。（不包含在内）
  - `count`：桶内包含的元素数量。
  - `acc_count`：前面桶与当前桶元素的累计总量。

7.2.2.33 MAP_AGG

7.2.2.33.1 描述

MAP_AGG 函数用于根据多行数据中的键值对形成一个映射结构。

7.2.2.33.2 语法

```
MAP_AGG(<expr1>, <expr2>)
```

7.2.2.33.3 参数说明

参数	说明
<expr1>	用于指定作为键的表达式。
<expr2>	用于指定作为对应的值的表达式。

7.2.2.33.4 返回值

返回映射后的 MAP 类型的值。

7.2.2.33.5 举例

```
select `n_nationkey`, `n_name`, `n_regionkey` from `nation`;
```

```
+-------------+----------------+-------------+
```

```
| n_nationkey | n_name         | n_regionkey |
+-------------+----------------+-------------+
|           0 | ALGERIA        |           0 |
|           1 | ARGENTINA      |           1 |
|           2 | BRAZIL         |           1 |
|           3 | CANADA         |           1 |
|           4 | EGYPT          |           4 |
|           5 | ETHIOPIA       |           0 |
|           6 | FRANCE         |           3 |
|           7 | GERMANY        |           3 |
|           8 | INDIA          |           2 |
|           9 | INDONESIA      |           2 |
|          10 | IRAN           |           4 |
|          11 | IRAQ           |           4 |
|          12 | JAPAN          |           2 |
|          13 | JORDAN         |           4 |
|          14 | KENYA          |           0 |
|          15 | MOROCCO        |           0 |
|          16 | MOZAMBIQUE     |           0 |
|          17 | PERU           |           1 |
|          18 | CHINA          |           2 |
|          19 | ROMANIA        |           3 |
|          20 | SAUDI ARABIA   |           4 |
|          21 | VIETNAM        |           2 |
|          22 | RUSSIA         |           3 |
|          23 | UNITED KINGDOM |           3 |
|          24 | UNITED STATES  |           1 |
+-------------+----------------+-------------+
```

```
select `n_regionkey`, map_agg(`n_nationkey`, `n_name`) from `nation` group by `n_regionkey`;
```

```
+-------------+-------------------------------------------------------------------------------+
| n_regionkey | map_agg(`n_nationkey`, `n_name`)                                              |
+-------------+-------------------------------------------------------------------------------+
|           1 | {1:"ARGENTINA", 2:"BRAZIL", 3:"CANADA", 17:"PERU", 24:"UNITED STATES"}        |
|           0 | {0:"ALGERIA", 5:"ETHIOPIA", 14:"KENYA", 15:"MOROCCO", 16:"MOZAMBIQUE"}        |
|           3 | {6:"FRANCE", 7:"GERMANY", 19:"ROMANIA", 22:"RUSSIA", 23:"UNITED KINGDOM"}     |
|           4 | {4:"EGYPT", 10:"IRAN", 11:"IRAQ", 13:"JORDAN", 20:"SAUDI ARABIA"}             |
|           2 | {8:"INDIA", 9:"INDONESIA", 12:"JAPAN", 18:"CHINA", 21:"VIETNAM"}              |
+-------------+-------------------------------------------------------------------------------+
```

```
select n_regionkey, map_agg(`n_name`, `n_nationkey` % 5) from `nation` group by `n_regionkey`;
```

```
+-------------+-------------------------------------------------------------------------+
| n_regionkey | map_agg(`n_name`, (`n_nationkey` % 5))                                  |
```

```
+-------------+-------------------------------------------------------------------------+
|           2 | {"INDIA":3, "INDONESIA":4, "JAPAN":2, "CHINA":3, "VIETNAM":1}           |
|           0 | {"ALGERIA":0, "ETHIOPIA":0, "KENYA":4, "MOROCCO":0, "MOZAMBIQUE":1}     |
|           3 | {"FRANCE":1, "GERMANY":2, "ROMANIA":4, "RUSSIA":2, "UNITED KINGDOM":3} |
|           1 | {"ARGENTINA":1, "BRAZIL":2, "CANADA":3, "PERU":2, "UNITED STATES":4}    |
|           4 | {"EGYPT":4, "IRAN":0, "IRAQ":1, "JORDAN":3, "SAUDI ARABIA":0}           |
+-------------+-------------------------------------------------------------------------+
```

7.2.2.34 MAX

7.2.2.34.1 描述

MAX 函数返回表达式的最大值。

7.2.2.34.2 语法

```
MAX(<expr>)
```

7.2.2.34.3 参数说明

参数	说明
expr	需要获取值的表达式

7.2.2.34.4 返回值

返回与输入表达式相同的数据类型。

7.2.2.34.5 举例

```
select max(scan_rows) from log_statis group by datetime;
```

```
+------------------+
| max(`scan_rows`) |
+------------------+
|          4671587 |
+------------------+
```

7.2.2.35 MAX_BY

7.2.2.35.1 描述

MAX_BY 函数用于根据指定列的最大值，返回对应的的关联值。

7.2.2.35.2 语法

```
MAX_BY(<expr1>, <expr2>)
```

7.2.2.35.3 参数说明

参数	说明
<expr1>	用于指定对应关联的表达式。
<expr2>	用于指定最大值统计的表达式。

7.2.2.35.4 返回值

返回与输入表达式相同的数据类型。

7.2.2.35.5 举例

```
select * from tbl;
```

```
+------+------+------+------+
| k1   | k2   | k3   | k4   |
+------+------+------+------+
|    0 | 3    | 2    |  100 |
|    1 | 2    | 3    |    4 |
|    4 | 3    | 2    |    1 |
|    3 | 4    | 2    |    1 |
+------+------+------+------+
```

```
select max_by(k1, k4) from tbl;
```

```
+--------------------+
| max_by(`k1`, `k4`) |
+--------------------+
|                  0 |
+--------------------+
```

7.2.2.36 MIN

7.2.2.36.1 描述

MIN 函数返回表达式的最小值。

7.2.2.36.2 语法

```
MIN(expr)
```

7.2.2.36.3 参数说明

参数	说明
expr	需要获取值的表达式

7.2.2.36.4 返回值

返回与输入表达式相同的数据类型。

7.2.2.36.5 举例

```
select MIN(scan_rows) from log_statis group by datetime;
```

```
+------------------+
| MIN(`scan_rows`) |
+------------------+
|                0 |
+------------------+
```

7.2.2.37 MIN_BY

7.2.2.37.1 描述

MIN_BY 函数用于根据指定列的最小值，返回对应的的关联值。

7.2.2.37.2 语法

```
MIN_BY(<expr1>, <expr2>)
```

7.2.2.37.3 参数说明

参数	说明
<expr1>	用于指定对应关联的表达式。
<expr2>	用于指定最小值统计的表达式。

7.2.2.37.4 返回值

返回与输入表达式相同的数据类型。

7.2.2.37.5 举例

```
select * from tbl;
```

```
+------+------+------+------+
| k1   | k2   | k3   | k4   |
+------+------+------+------+
|    0 | 3    | 2    |  100 |
|    1 | 2    | 3    |    4 |
|    4 | 3    | 2    |    1 |
|    3 | 4    | 2    |    1 |
+------+------+------+------+
```

```
select min_by(k1, k4) from tbl;
```

```
+--------------------+
| min_by(`k1`, `k4`) |
+--------------------+
|                  4 |
+--------------------+
```

7.2.2.38 PERCENTILE

7.2.2.38.1 描述

计算精确的百分位数，适用于小数据量。先对指定列降序排列，然后取精确的第 p 位百分数。p 的值介于 0 到 1 之间，如果 p 不指向精确的位置，则返回所指位置两侧相邻数值在 p 处的线性插值，注意这不是两数字的平均数。特殊情况：- 当输入的列为 NULL 时，返回 NULL

7.2.2.38.2 语法

```
PERCENTILE(<col>, <p>)
```

7.2.2.38.3 参数

参数	说明
<col>	需要被计算精确的百分位数的列，必须是整数类型的列。
<p>	需要精确的百分位数，常量，取值为 [0.0, 1.0]。

7.2.2.38.4 返回值

返回指定列的精确的百分位数，类型为 DOUBLE。

7.2.2.38.5 举例

```
-- 创建示例表
```

```
CREATE TABLE sales_data
(
    product_id INT,
    sale_price DECIMAL(10, 2)
) DUPLICATE KEY(`product_id`)
DISTRIBUTED BY HASH(`product_id`) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

-- 插入示例数据
INSERT INTO sales_data VALUES
(1, 10.00),
(1, 15.00),
(1, 20.00),
(1, 25.00),
(1, 30.00),
(1, 35.00),
(1, 40.00),
(1, 45.00),
(1, 50.00),
(1, 100.00);

-- 计算不同百分位的销售价格
SELECT
    percentile(sale_price, 0.5)  as median_price,     -- 中位数
    percentile(sale_price, 0.75) as p75_price,        -- 75 分位数
    percentile(sale_price, 0.90) as p90_price,        -- 90 分位数
    percentile(sale_price, 0.95) as p95_price,        -- 95 分位数
    percentile(null, 0.99)       as p99_null          -- null 的 99 分位数
FROM sales_data;
```

```
+--------------+-----------+-------------------+-------------------+----------+
| median_price | p75_price | p90_price         | p95_price         | p99_null |
+--------------+-----------+-------------------+-------------------+----------+
|         32.5 |     43.75 | 54.99999999999998 | 77.49999999999994 |     NULL |
+--------------+-----------+-------------------+-------------------+----------+
```

7.2.2.39 PERCENTILE_APPROX

7.2.2.39.1 描述

PERCENTILE_APPROX 函数用于计算近似百分位数，主要用于大数据集的场景。与 PERCENTILE 函数相比，它具有以下特点：

1. 内存效率：使用固定大小的内存，即使在处理低基数列（不同元素数很少）时也能保持较低的内存消耗
2. 性能优势：适合处理低基数大规模数据集，计算速度快
3. 精度可调：通过 compression 参数可以在精度和性能之间做平衡

7.2.2.39.2 语法

```
PERCENTILE_APPROX(<col>, <p> [, <compression>])
```

7.2.2.39.3 参数

参数	说明
<col>	需要计算百分位数的列
<p>	百分位数值，取值范围 [0.0, 1.0]，例如 0.99 表示 99 分位数
<compression>	可选参数，压缩度，取值范围 [2048, 10000]。值越大，精度越高，但内存消耗也越大。如果不指定或超出

7.2.2.39.4 返回值

返回一个 DOUBLE 类型的值，表示计算得到的近似百分位数。

7.2.2.39.5 举例

```
-- 创建示例表
CREATE TABLE response_times (
    request_id INT,
    response_time DECIMAL(10, 2)
) DUPLICATE KEY(`request_id`)
DISTRIBUTED BY HASH(`request_id`) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

-- 插入示例数据
INSERT INTO response_times VALUES
(1, 10.5),
(2, 15.2),
(3, 20.1),
(4, 25.8),
(5, 30.3),
(6, 35.7),
(7, 40.2),
(8, 45.9),
(9, 50.4),
(10, 100.6);
```

```
-- 使用不同压缩度计算 99 分位数
SELECT
    percentile_approx(response_time, 0.99) as p99_default,          -- 默认压缩度
    percentile_approx(response_time, 0.99, 2048) as p99_fast,       -- 低压缩度，更快
    percentile_approx(response_time, 0.99, 10000) as p99_accurate   -- 高压缩度，更精确
FROM response_times;
```

```
+------------------+------------------+------------------+
| p99_default      | p99_fast         | p99_accurate     |
+------------------+------------------+------------------+
| 100.5999984741211 | 100.5999984741211 | 100.5999984741211 |
+------------------+------------------+------------------+
```

7.2.2.40 PERCENTILE_ARRAY

7.2.2.40.1 描述

PERCENTILE_ARRAY 函数用于计算精确的百分位数数组，允许一次性计算多个百分位数值。这个函数主要适用于小数据量。

主要特点：1. 精确计算：提供精确的百分位数结果，而不是近似值 2. 批量处理：可以一次计算多个百分位数 3. 适用范围：最适合处理数据量较小的场景

7.2.2.40.2 语法

```
PERCENTILE_ARRAY(<col>, <array_p>)
```

7.2.2.40.3 参数

参数	说明
<col>	需要计算百分位数的列
<array_p>	百分位数数组，数组中的每个元素必须在 [0.0, 1.0] 范围内，例如 [0.5, 0.95, 0.99]

7.2.2.40.4 返回值

返回一个 DOUBLE 类型的数组，包含了对应于输入百分位数数组的计算结果。

7.2.2.40.5 举例

```
-- 创建示例表
CREATE TABLE sales_data (
    id INT,
```

```
    amount DECIMAL(10, 2)
) DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

-- 插入示例数据
INSERT INTO sales_data VALUES
(1, 10.5),
(2, 15.2),
(3, 20.1),
(4, 25.8),
(5, 30.3),
(6, 35.7),
(7, 40.2),
(8, 45.9),
(9, 50.4),
(10, 100.6);

-- 计算多个百分位数
SELECT percentile_array(amount, [0.25, 0.5, 0.75, 0.9]) as percentiles
FROM sales_data;
```

```
+-----------------------------------------+
| percentiles                             |
+-----------------------------------------+
| [21.25, 32.5, 43.75, 54.99999999999998] |
+-----------------------------------------+
```

7.2.2.41 QUANTILE_UNION

7.2.2.41.1 描述

QUANTILE_UNION 函数用于合并多个分位数计算的中间结果。这个函数通常与 QUANTILE_STATE 配合使用，特别适用于需要分阶段计算分位数的场景。

7.2.2.41.2 语法

```
QUANTILE_UNION(<query_state>)
```

7.2.2.41.3 参数

参数	说明
<query_state>	由 TO_QUANTILE_STATE 函数生成的中间状态

7.2.2.41.4 返回值

返回一个可以用于进一步分位数计算的聚合状态。此函数返回的结果仍是 QUANTILE_STATE。

7.2.2.41.5 举例

```
-- 创建示例表
CREATE TABLE response_times (
    request_id INT,
    response_time DOUBLE,
    region STRING
) DUPLICATE KEY(request_id)
DISTRIBUTED BY HASH(request_id) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

-- 插入示例数据
INSERT INTO response_times VALUES
(1, 10.5, 'east'),
(2, 15.2, 'east'),
(3, 20.1, 'west'),
(4, 25.8, 'east'),
(5, 30.3, 'west'),
(6, 35.7, 'east'),
(7, 40.2, 'west'),
(8, 45.9, 'east'),
(9, 50.4, 'west'),
(10, 100.6, 'east');

-- 按区域计算响应时间的 50% 分位数
SELECT
    region,
    QUANTILE_PERCENT(
        QUANTILE_UNION(
            TO_QUANTILE_STATE(response_time, 2048)
        ),
        0.5
    ) AS median_response_time
FROM response_times
GROUP BY region;
```

```
+--------+----------------------+
| region | median_response_time |
+--------+----------------------+
| west   |                35.25 |
| east   |                30.75 |
+--------+----------------------+
```

7.2.2.42 RETENTION

7.2.2.42.1 描述

留存函数将一组条件作为参数，类型为 1 到 32 个UInt8类型的参数，用来表示事件是否满足特定条件。任何条件都可以指定为参数。

除了第一个以外，条件成对适用：如果第一个和第二个是真的，第二个结果将是真的，如果第一个和第三个是真的，第三个结果将是真的，等等。

简单来讲，返回值数组第 1 位表示event_1的真假，第二位表示event_1真假与event_2真假相与，第三位表示event_1真假与event_3真假相与，等等。如果event_1为假，则返回全是 0 的数组。

当有多个列参与计算时，如果任意一列出现了 NULL 值，则 NULL 值的当前行不会参与聚合计算，被直接丢弃可以在计算列上加 IF NULL 函数处理 NULL 值，详情见后续示例。

7.2.2.42.2 语法

```
RETENTION(<event_1> [, <event_2>, ... , <event_n>]);
```

7.2.2.42.3 参数

参数	说明
<event_n>	第n个事件条件，类型为UInt8，取值为 1 或 0

7.2.2.42.4 返回值

- 1: 条件满足。
- 0: 条件不满足。

由 1 和 0 组成的最大长度为 32 位的数组，最终输出数组的长度与输入参数长度相同。如果在没有任何数据参与聚合的情况下，会返回 NULL 值

7.2.2.42.5 举例

1. 创建示例表，插入示例数据

```
CREATE TABLE retention_test(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

INSERT into retention_test values
(0, '2022-10-12'),
(0, '2022-10-13'),
(0, '2022-10-14'),
(1, '2022-10-12'),
(1, '2022-10-13'),
(2, '2022-10-12');
```

2. 正常计算用户留存

```
SELECT
    uid,
    RETENTION(date = '2022-10-12') AS r,
    RETENTION(date = '2022-10-12', date = '2022-10-13') AS r2,
    RETENTION(date = '2022-10-12', date = '2022-10-13', date = '2022-10-14') AS r3
FROM retention_test
GROUP BY uid
ORDER BY uid ASC;
```

```
+------+------+--------+-----------+
| uid  | r    | r2     | r3        |
+------+------+--------+-----------+
|    0 | [1]  | [1, 1] | [1, 1, 1] |
|    1 | [1]  | [1, 1] | [1, 1, 0] |
|    2 | [1]  | [1, 0] | [1, 0, 0] |
+------+------+--------+-----------+
```

3. 特殊情况 NULL 值处理，重新建表以及插入数据

```
CREATE TABLE retention_test2(
    `uid` int,
    `flag` boolean,
    `flag2` boolean
) DUPLICATE KEY(uid)
```

```
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_allocation" = "tag.location.default: 1"
);

INSERT into retention_test2 values (0, false, false), (1, true,  NULL);

SELECT * from retention_test2;
```

```
+------+------+-------+
| uid  | flag | flag2 |
+------+------+-------+
|    0 |    1 |  NULL |
|    1 |    0 |     0 |
+------+------+-------+
```

4. 空表计算时，没有任何数据参与聚合，返回 NULL 值

```
SELECT RETENTION(date = '2022-10-12') AS r FROM retention_test2 where uid is NULL;
```

```
+------+
| r    |
+------+
| NULL |
+------+
```

5. 仅 flag 一列参与计算，由于 uid = 0 时，flag 为真，返回 1

```
select retention(flag) from retention_test2;
```

```
+-----------------+
| retention(flag) |
+-----------------+
| [1]             |
+-----------------+
```

6. 当 flag,flag2 两列参与计算时，uid = 0 的行，由于 flag2 为 NULL 值，所以这行未参与聚合计算，仅 uid = 1 参与聚合计算，返回结果为 0

```
select retention(flag,flag2) from retention_test2;
```

```
+-----------------------+
| retention(flag,flag2) |
+-----------------------+
| [0, 0]                |
+-----------------------+
```

7. 如果需要解决 NULL 值问题，可以用 IFNULL 函数将 NULL 转换成 false，这样 uid = 0,1 两行都会参与聚合计算

```
select retention(flag,IFNULL(flag2,false)) from retention_test2;;
```

```
+-------------------------------------+
| retention(flag,IFNULL(flag2,false)) |
+-------------------------------------+
| [1, 0]                              |
+-------------------------------------+
```

7.2.2.43 SEQUENCE_COUNT

7.2.2.43.1 描述

计算与模式匹配的事件链的数量。该函数搜索不重叠的事件链。当前链匹配后，它开始搜索下一个链。

警告！

在同一秒钟发生的事件可能以未定义的顺序排列在序列中，会影响最终结果。

7.2.2.43.2 语法

```
SEQUENCE_COUNT(<pattern>, <timestamp>, <cond_1> [, <cond_2>, ..., <cond_n>]);
```

7.2.2.43.3 参数

参数	说明
<pattern>	模式字符串，可参考下面的模式语法

参数	说明
< ↪ timestamp ↪ > ↪	包含时间的列。典型的时间类型是: Date ↪ 和 DateTime ↪ 。也可以使用任何支持的 UInt ↪ 数据类型。

参数	说明
<cond_n>	事件链的约束条件。数据类型是：UInt8。最多可以传递32个条件参数。该函数只考虑这些条件中描述的事件。如果序列包

参数	说明

模式语法

- (?N) — 在位置 N 匹配条件参数。条件在编号 [1, 32] 范围。例如，(?1) 匹配传递给 cond_1 参数。
- .* — 匹配任何事件的数字。不需要条件参数来匹配这个模式。
- (?t operator value) — 分开两个事件的时间。单位为秒。
- t表示为两个时间的差值，单位为秒。例如：(?1)(?t>1800)(?2) 匹配彼此发生超过 1800 秒的事件，(?1)(?t>10000)(?2)匹配彼此发生超过 10000 秒的事件。这些事件之间可以存在任意数量的任何事件。您可以使用 >=, >, <, <=, == 运算符。

7.2.2.43.4 返回值

匹配的非重叠事件链数。

7.2.2.43.5 举例

匹配例子

```
-- 创建示例表
CREATE TABLE sequence_count_test1(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_num" = "1"
);

-- 插入示例数据
INSERT INTO sequence_count_test1(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 13:28:02', 2),
(3, '2022-11-02 16:15:01', 1),
(4, '2022-11-02 19:05:04', 2),
(5, '2022-11-02 20:08:44', 3);

-- 查询示例
SELECT
    SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 3) as c1,
    SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 2) as c2,
```

```
    SEQUENCE_COUNT('(?1)(?t>=3600)(?2)', date, number = 1, number = 2) as c3
FROM sequence_count_test1;
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
|    1 |    2 |    2 |
+------+------+------+
```

不匹配例子

```
-- 创建示例表
CREATE TABLE sequence_count_test2(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_num" = "1"
);

-- 插入示例数据
INSERT INTO sequence_count_test2(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 11:41:00', 7),
(3, '2022-11-02 16:15:01', 3),
(4, '2022-11-02 19:05:04', 4),
(5, '2022-11-02 21:24:12', 5);

-- 查询示例
SELECT
    SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 2) as c1,
    SEQUENCE_COUNT('(?1)(?2).*', date, number = 1, number = 2) as c2,
    SEQUENCE_COUNT('(?1)(?t>3600)(?2)', date, number = 1, number = 7) as c3
FROM sequence_count_test2;
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
|    0 |    0 |    0 |
+------+------+------+
```

特殊例子

```
-- 创建示例表
```

```
CREATE TABLE sequence_count_test3(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_num" = "1"
);

-- 插入示例数据
INSERT INTO sequence_count_test3(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 11:41:00', 7),
(3, '2022-11-02 16:15:01', 3),
(4, '2022-11-02 19:05:04', 4),
(5, '2022-11-02 21:24:12', 5);

-- 查询示例
SELECT SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 5) FROM sequence_count_test3;
```

```
+-------------------------------------------------------------------+
| sequence_count('(?1)(?2)', `date`, `number` = 1, `number` = 5) |
+-------------------------------------------------------------------+
|                                                                 1 |
+-------------------------------------------------------------------+
```

上面为一个非常简单的匹配例子，该函数找到了数字 5 跟随数字 1 的事件链。它跳过了它们之间的数字 7，3，4，因为该数字没有被描述为事件。如果我们想在搜索示例中给出的事件链时考虑这个数字，我们应该为它创建一个条件。

现在，考虑如下执行语句：

```
SELECT SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 5, number = 4) FROM sequence_count_
    ↪ test3;
```

```
+-----------------------------------------------------------------------------+
| sequence_count('(?1)(?2)', `date`, `number` = 1, `number` = 5, `number` = 4) |
+-----------------------------------------------------------------------------+
|                                                                           0 |
+-----------------------------------------------------------------------------+
```

您可能对这个结果有些许疑惑，在这种情况下，函数找不到与模式匹配的事件链，因为数字 4 的事件发生在 1 和 5 之间。如果在相同的情况下，我们检查了数字 6 的条件，则序列将与模式匹配。

```
SELECT SEQUENCE_COUNT('(?1)(?2)', date, number = 1, number = 5, number = 6) FROM sequence_count_
    ↪ test3;
```

```
+-----------------------------------------------------------------------------+
| sequence_count('(?1)(?2)', `date`, `number` = 1, `number` = 5, `number` = 6) |
+-----------------------------------------------------------------------------+
|                                                                           1 |
+-----------------------------------------------------------------------------+
```

7.2.2.44 SEQUENCE_MATCH

7.2.2.44.1 描述

检查序列是否包含与模式匹配的事件链。

警告！

在同一秒钟发生的事件可能以未定义的顺序排列在序列中，会影响最终结果。

7.2.2.44.2 语法

```
SEQUENCE_MATCH(<pattern>, <timestamp>, <cond_1> [, <cond_2>, ..., <cond_n>])
```

7.2.2.44.3 参数

参数	说明
<pattern>	模式字符串，可参考下面的模式语法

参数	说明
<↩ timestamp ↩ > ↩	包含时间的列。典型的时间类型是: Date ↩ 和 DateTime ↩ 。也可以使用任何支持的 UInt ↩ 数据类型。

参数	说明
<cond_n>	事件链的约束条件。数据类型是：UInt8。最多可以传递32个条件参数。该函数只考虑这些条件中描述的事件。如果序列包

参数	说明

模式语法

- (?N) — 在位置 N 匹配条件参数。条件在编号 [1, 32] 范围。例如，(?1) 匹配传递给 cond1 参数。
- .* — 匹配任何事件的数字。不需要条件参数来匹配这个模式。
- (?t operator value) — 分开两个事件的时间。单位为秒。
- t表示为两个时间的差值，单位为秒。例如：(?1)(?t>1800)(?2) 匹配彼此发生超过 1800 秒的事件，(?1)(?t>10000)(?2)匹配彼此发生超过 10000 秒的事件。这些事件之间可以存在任意数量的任何事件。您可以使用 >=, >, <, <=, == 运算符。

7.2.2.44.4 返回值

1：模式匹配。

0：模式不匹配。

7.2.2.44.5 举例

匹配例子

```
CREATE TABLE sequence_match_test1(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_num" = "1"
);

INSERT INTO sequence_match_test1(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 13:28:02', 2),
(3, '2022-11-02 16:15:01', 1),
(4, '2022-11-02 19:05:04', 2),
(5, '2022-11-02 20:08:44', 3);


SELECT
sequence_match('(?1)(?2)', date, number = 1, number = 3) as c1,
sequence_match('(?1)(?2)', date, number = 1, number = 2) as c2,
sequence_match('(?1)(?t>=3600)(?2)', date, number = 1, number = 2) as c3
FROM sequence_match_test1;
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
|    1 |    1 |    1 |
+------+------+------+
```

不匹配例子

```
CREATE TABLE sequence_match_test2(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
PROPERTIES (
    "replication_num" = "1"
);

INSERT INTO sequence_match_test2(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 11:41:00', 7),
(3, '2022-11-02 16:15:01', 3),
(4, '2022-11-02 19:05:04', 4),
(5, '2022-11-02 21:24:12', 5);

SELECT
sequence_match('(?1)(?2)', date, number = 1, number = 2) as c1,
sequence_match('(?1)(?2).*', date, number = 1, number = 2) as c2,
sequence_match('(?1)(?t>3600)(?2)', date, number = 1, number = 7) as c3
FROM sequence_match_test2;
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
|    0 |    0 |    0 |
+------+------+------+
```

特殊例子

```
CREATE TABLE sequence_match_test3(
    `uid` int COMMENT 'user id',
    `date` datetime COMMENT 'date time',
    `number` int NULL COMMENT 'number'
) DUPLICATE KEY(uid)
DISTRIBUTED BY HASH(uid) BUCKETS AUTO
```

```
PROPERTIES (
    "replication_num" = "1"
);

INSERT INTO sequence_match_test3(uid, date, number) values
(1, '2022-11-02 10:41:00', 1),
(2, '2022-11-02 11:41:00', 7),
(3, '2022-11-02 16:15:01', 3),
(4, '2022-11-02 19:05:04', 4),
(5, '2022-11-02 21:24:12', 5);

SELECT sequence_match('(?1)(?2)', date, number = 1, number = 5)
FROM sequence_match_test3;
```

```
+-------------------------------------------------------------------+
| sequence_match('(?1)(?2)', `date`, `number` = 1, `number` = 5) |
+-------------------------------------------------------------------+
|                                                                 1 |
+-------------------------------------------------------------------+
```

上面为一个非常简单的匹配例子，该函数找到了数字 5 跟随数字 1 的事件链。它跳过了它们之间的数字 7，3，4，因为该数字没有被描述为事件。如果我们想在搜索示例中给出的事件链时考虑这个数字，我们应该为它创建一个条件。

现在，考虑如下执行语句：

```
SELECT sequence_match('(?1)(?2)', date, number = 1, number = 5, number = 4)
FROM sequence_match_test3;
```

```
+-------------------------------------------------------------------------------+
| sequence_match('(?1)(?2)', `date`, `number` = 1, `number` = 5, `number` = 4) |
+-------------------------------------------------------------------------------+
|                                                                             0 |
+-------------------------------------------------------------------------------+
```

您可能对这个结果有些许疑惑，在这种情况下，函数找不到与模式匹配的事件链，因为数字 4 的事件发生在 1 和 5 之间。如果在相同的情况下，我们检查了数字 6 的条件，则序列将与模式匹配。

```
SELECT sequence_match('(?1)(?2)', date, number = 1, number = 5, number = 6)
FROM sequence_match_test3;
```

```
+-------------------------------------------------------------------------------+
| sequence_match('(?1)(?2)', `date`, `number` = 1, `number` = 5, `number` = 6) |
+-------------------------------------------------------------------------------+
|                                                                             1 |
+-------------------------------------------------------------------------------+
```

7.2.2.45 SKEW,SKEW_POP,SKEWNESS

7.2.2.45.1 描述

返回表达式的 斜度。用来计算斜度的公式是 3阶中心矩 / ((方差)^{1.5}), 当方差为零时，SKEWNESS 会返回 NULL。

相关命令

kurt

7.2.2.45.2 别名

- SKEW
- SKEW_POP

7.2.2.45.3 语法

```
SKEWNESS(<col>)
```

7.2.2.45.4 参数

参数	说明
<col>	需要被计算斜度的列

7.2.2.45.5 返回值

返回表达式的斜度，Double 类型。

7.2.2.45.6 举例

```
CREATE TABLE statistic_test(
    tag int,
    val1 double not null,
    val2 double null
) DISTRIBUTED BY HASH(tag)
PROPERTIES (
    "replication_num"="1"
);

INSERT INTO statistic_test VALUES
(1, -10, -10),
(2, -20, NULL),
(3, 100, NULL),
(4, 100, NULL),
```

```
(5, 1000,1000);

-- NULL 值会被忽略
SELECT
  skew(val1),
  skew(val2)
FROM statistic_test;
```

```
+--------------------+------------+
| skew(val1)         | skew(val2) |
+--------------------+------------+
| 1.4337199628825619 |          0 |
+--------------------+------------+
```

```
-- 每组仅包含一行，结果为 NULL。
SELECT
  skew(val1),
  skew(val2)
FROM statistic_test
GROUP BY tag;
```

```
+------------+------------+
| skew(val1) | skew(val2) |
+------------+------------+
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
|       NULL |       NULL |
+------------+------------+
```

7.2.2.46 STDDEV,STDDEV_POP

7.2.2.46.1 描述

返回 expr 表达式的标准差

7.2.2.46.2 别名

- STDDEV_POP

7.2.2.46.3 语法

```
STDDEV(<expr>)
```

7.2.2.46.4 参数

参数	说明
<expr>	需要被计算标准差的值

7.2.2.46.5 返回值

返回参数 expr 的标准差

7.2.2.46.6 举例

```
-- 创建示例表
CREATE TABLE score_table (
    student_id INT,
    score DOUBLE
) DISTRIBUTED BY HASH(student_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO score_table VALUES
(1, 85),
(2, 90),
(3, 82),
(4, 88),
(5, 95);

-- 计算所有学生分数的标准差
SELECT STDDEV(score) as score_stddev
FROM score_table;
```

```
+-------------------+
| score_stddev      |
+-------------------+
| 4.427188724235729 |
+-------------------+
```

7.2.2.47 STDDEV_SAMP

7.2.2.47.1 描述

返回 expr 表达式的样本标准差

7.2.2.47.2 语法

```
STDDEV_SAMP(<expr>)
```

7.2.2.47.3 参数

参数	说明
<expr>	需要被计算标准差的值

7.2.2.47.4 返回值

返回参数 expr 的样本标准差

7.2.2.47.5 举例

```
-- 创建示例表
CREATE TABLE score_table (
    student_id INT,
    score DOUBLE
) DISTRIBUTED BY HASH(student_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO score_table VALUES
(1, 85),
(2, 90),
(3, 82),
(4, 88),
(5, 95);

-- 计算所有学生分数的样本标准差
SELECT STDDEV_SAMP(score) as score_stddev
FROM score_table;
```

```
+-------------------+
| score_stddev      |
+-------------------+
| 4.949747468305831 |
+-------------------+
```

7.2.2.48 SUM

7.2.2.48.1 描述

用于返回选中字段所有值的和。

7.2.2.48.2 语法

```
SUM(<expr>)
```

7.2.2.48.3 参数

参数	说明
<expr>	要计算和的字段

7.2.2.48.4 返回值

返回选中字段所有值的和。

7.2.2.48.5 举例

```
-- 创建示例表
CREATE TABLE sales_table (
    product_id INT,
    price DECIMAL(10,2),
    quantity INT
) DISTRIBUTED BY HASH(product_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO sales_table VALUES
(1, 99.99, 2),
(2, 159.99, 1),
(3, 49.99, 5),
(4, 299.99, 1),
(5, 79.99, 3);

-- 计算销售总金额
SELECT SUM(price * quantity) as total_sales
FROM sales_table;
```

```
+-------------+
| total_sales |
+-------------+
```

```
|     1149.88 |
+-------------+
```

7.2.2.49 SUM0

7.2.2.49.1 描述

用于返回选中字段所有值的和。与 SUM 函数不同的是，当输入值全为 NULL 时，SUM0 返回 0 而不是 NULL。

7.2.2.49.2 语法

```
SUM0(<expr>)
```

7.2.2.49.3 参数

参数	说明
<expr>	要计算和的字段

7.2.2.49.4 返回值

返回选中字段所有值的和。如果所有值都为 NULL，则返回 0。

7.2.2.49.5 举例

```
-- 创建示例表
CREATE TABLE sales_table (
    product_id INT,
    price DECIMAL(10,2),
    quantity INT,
    discount DECIMAL(10,2)
) DISTRIBUTED BY HASH(product_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO sales_table VALUES
(1, 99.99, 2, NULL),
(2, 159.99, 1, NULL),
(3, 49.99, 5, NULL),
(4, 299.99, 1, NULL),
(5, 79.99, 3, NULL);
```

```
-- 对比 SUM 和 SUM0 的区别
SELECT
    SUM(discount) as sum_discount,    -- 返回 NULL
    SUM0(discount) as sum0_discount   -- 返回 0
FROM sales_table;
```

```
+--------------+---------------+
| sum_discount | sum0_discount |
+--------------+---------------+
|         NULL |          0.00 |
+--------------+---------------+
```

7.2.2.50 TOPN

7.2.2.50.1 描述

TOPN 函数用于返回指定列中出现频率最高的 N 个值。它是一个近似计算函数，返回结果的顺序是按照计数值从大到小排序。

7.2.2.50.2 语法

```
TOPN(<expr>, <top_num> [, <space_expand_rate>])
```

7.2.2.50.3 参数

参数	说明
<expr>	要统计的列或表达式

参数	说明
 ↪ top ↪ _ ↪ num ↪ > ↪	要返回的最高频率值的数量，必须是正整数

参数	说明
 ↪ space ↪ _ ↪ expand ↪ _ ↪ rate ↪ > ↪	可选项，该值用来设置 Space-Saving 算法中使用的 counter 个数counter ↪ _ ↪ numbers ↪ ↪ = ↪ ↪ top ↪ _ ↪ num ↪ ↪ * ↪ ↪ space ↪ _ ↪ expand ↪ _ ↪ rate ↪ space_expand_rate 的值越大，结果越准确，默

参数	说明

7.2.2.50.4 返回值

返回一个 JSON 字符串，包含值和对应的出现次数。

7.2.2.50.5 举例

```
-- 创建示例表
CREATE TABLE page_visits (
    page_id INT,
    user_id INT,
    visit_date DATE
) DISTRIBUTED BY HASH(page_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO page_visits VALUES
(1, 101, '2024-01-01'),
(2, 102, '2024-01-01'),
(1, 103, '2024-01-01'),
(3, 101, '2024-01-01'),
(1, 104, '2024-01-01'),
(2, 105, '2024-01-01'),
(1, 106, '2024-01-01'),
(4, 107, '2024-01-01');

-- 查找访问量最高的前 3 个页面
SELECT TOPN(page_id, 3) as top_pages
FROM page_visits;
```

```
+---------------------+
| top_pages           |
+---------------------+
| {"1":4,"2":2,"4":1} |
+---------------------+
```

7.2.2.51 TOPN_ARRAY

7.2.2.51.1 描述

TOPN_ARRAY 函数返回指定列中出现频率最高的 N 个值的数组。与 TOPN 函数不同，TOPN_ARRAY 返回一个数组类型，便于后续处理和分析。

7.2.2.51.2 语法

```
TOPN_ARRAY(<expr>, <top_num> [, <space_expand_rate>])
```

7.2.2.51.3 参数

参数	说明
<expr>	要统计的列或表达式
<top_num>	要返回的最高频率值的数量，必须是正整数

参数	说明
 ↪ space ↪ _ ↪ expand ↪ _ ↪ rate ↪ > ↪	可选项，该值用来设置 Space-Saving 算法中使用的 counter 个数counter ↪ _ ↪ numbers ↪ ↪ = ↪ ↪ top ↪ _ ↪ num ↪ ↪ * ↪ ↪ space ↪ _ ↪ expand ↪ _ ↪ rate ↪ space_expand_rate 的值越大，结果越准确，默

参数	说明

7.2.2.51.4 返回值

返回一个数组，包含出现频率最高的 N 个值。

7.2.2.51.5 举例

```
-- 创建示例表
CREATE TABLE page_visits (
    page_id INT,
    user_id INT,
    visit_date DATE
) DISTRIBUTED BY HASH(page_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO page_visits VALUES
(1, 101, '2024-01-01'),
(2, 102, '2024-01-01'),
(1, 103, '2024-01-01'),
(3, 101, '2024-01-01'),
(1, 104, '2024-01-01'),
(2, 105, '2024-01-01'),
(1, 106, '2024-01-01'),
(4, 107, '2024-01-01');

-- 查找访问量最高的前 3 个页面
SELECT TOPN_ARRAY(page_id, 3) as top_pages
FROM page_visits;
```

```
+-----------+
| top_pages |
+-----------+
| [1, 2, 4] |
+-----------+
```

7.2.2.52 TOPN_WEIGHTED

7.2.2.52.1 描述

TOPN_WEIGHTED 函数返回指定列中出现频率最高的 N 个值，并且可以为每个值指定权重。与普通的 TOPN 函数不同，TOPN_WEIGHTED 允许通过权重来调整值的重要性。

7.2.2.52.2 语法

```
TOPN_WEIGHTED(<expr>, <weight>, <top_num> [, <space_expand_rate>])
```

7.2.2.52.3 参数

参数	说明
<expr>	要统计的列或表达式
<weight>	用于调整权重的列或表达式

参数	说明
<top_num>	要返回的最高频率值的数量，必须是正整数

参数	说明
<↪ space↪ _↪ expand↪ _↪ rate↪ >↪	可选项，该值用来设置Space-Saving算法中使用的counter个数counter↪ _↪ numbers↪ ↪ =↪ ↪ top↪ _↪ num↪ ↪ *↪ ↪ space↪ _↪ expand↪ _↪ rate↪ space_expand_rate的值越大，结果越准确，默

参数	说明

7.2.2.52.4 返回值

返回一个数组，包含值和对应的加权计数。

7.2.2.52.5 举例

```
-- 创建示例表
CREATE TABLE product_sales (
    product_id INT,
    sale_amount DECIMAL(10,2),
    sale_date DATE
) DISTRIBUTED BY HASH(product_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO product_sales VALUES
(1, 100.00, '2024-01-01'),
(2, 50.00, '2024-01-01'),
(1, 150.00, '2024-01-01'),
(3, 75.00, '2024-01-01'),
(1, 200.00, '2024-01-01'),
(2, 80.00, '2024-01-01'),
(1, 120.00, '2024-01-01'),
(4, 90.00, '2024-01-01');

-- 查找销售额最高的前 3 个产品（按销售金额加权）
SELECT TOPN_WEIGHTED(product_id, sale_amount, 3) as top_products
FROM product_sales;
```

```
+--------------+
| top_products |
+--------------+
| [1, 2, 4]    |
+--------------+
```

7.2.2.53 VAR_SAMP,VARIANCE_SAMP

7.2.2.53.1 描述

VAR_SAMP 函数计算指定表达式的样本方差。与 VARIANCE（总体方差）不同，VAR_SAMP 使用 n-1 作为除数，这在统计学上被认为是对总体方差的无偏估计。

7.2.2.53.2 别名

- VARIANCE_SAMP

7.2.2.53.3 语法

```
VAR_SAMP(<expr>)
```

7.2.2.53.4 参数

参数	描述
<expr>	要计算样本方差的列或表达式

7.2.2.53.5 返回值

返回一个 DOUBLE 类型的值，表示计算得到的样本方差。

7.2.2.53.6 举例

```
-- 创建示例表
CREATE TABLE student_scores (
    student_id INT,
    score DECIMAL(4,1)
) DISTRIBUTED BY HASH(student_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO student_scores VALUES
(1, 85.5),
(2, 92.0),
(3, 78.5),
(4, 88.0),
(5, 95.5),
(6, 82.0),
(7, 90.0),
(8, 87.5);
```

```
-- 计算学生成绩的样本方差
SELECT
    VAR_SAMP(score) as sample_variance,
    VARIANCE(score) as population_variance
FROM student_scores;
```

```
+------------------+---------------------+
| sample_variance  | population_variance |
+------------------+---------------------+
| 29.4107142857143 |   25.73437500000001 |
+------------------+---------------------+
```

7.2.2.54 VARIANCE,VAR_POP,VARIANCE_POP

7.2.2.54.1 描述

VARIANCE 函数计算指定表达式的统计方差。它衡量了数据值与其算术平均值之间的差异程度。

7.2.2.54.2 别名

- VAR_POP
- VARIANCE_POP

7.2.2.54.3 语法

```
VARIANCE(<expr>)
```

7.2.2.54.4 参数

参数	说明
<expr>	要计算方差的列或表达式

7.2.2.54.5 返回值

返回一个 DOUBLE 类型的值，表示计算得到的方差。

7.2.2.54.6 举例

```
-- 创建示例表
CREATE TABLE student_scores (
    student_id INT,
    score DECIMAL(4,1)
```

```
) DISTRIBUTED BY HASH(student_id)
PROPERTIES (
    "replication_num" = "1"
);

-- 插入测试数据
INSERT INTO student_scores VALUES
(1, 85.5),
(2, 92.0),
(3, 78.5),
(4, 88.0),
(5, 95.5),
(6, 82.0),
(7, 90.0),
(8, 87.5);
-- 计算学生成绩的方差
SELECT VARIANCE(score) as score_variance
FROM student_scores;
```

```
+-------------------+
| score_variance    |
+-------------------+
| 25.73437499999998 |
+-------------------+
```

7.2.2.55 WINDOW_FUNNEL

7.2.2.55.1 描述

WINDOW_FUNNEL 函数用于分析用户行为序列，它在指定的时间窗口内搜索事件链，并计算事件链中完成的最大步骤数。这个函数特别适用于转化漏斗分析，比如分析用户从访问网站到最终购买的转化过程。

漏斗分析函数按照如下算法工作：

- 搜索到满足满足条件的第一个事件，设置事件长度为 1，此时开始滑动时间窗口计时。
- 如果事件在时间窗口内按照指定的顺序发生，事件长度累计增加。如果事件没有按照指定的顺序发生，事件长度不增加。
- 如果搜索到多个事件链，漏斗分析函数返回最大的长度。

7.2.2.55.2 语法

```
WINDOW_FUNNEL(<window>, <mode>, <timestamp>, <event_1>[, event_2, ... , event_n])
```

7.2.2.55.3 参数

参数	说明
<window>	滑动时间窗口大小，单位为秒
<mode>	模式，共有四种模式，分别为default, deduplication, fixed, increase，详细请参见下面的模式
<timestamp>	指定时间列，类型为 DATETIME, 滑动窗口沿着此列工作
<event_n>	表示事件的布尔表达式。

模式

```
- `default`: 默认模式。

- `deduplication`: 当某个事件重复发生时，这个重复发生的事件会阻止后续的处理过程。如，指定事件链为
    ↪ [event1='A', event2='B', event3='C', event4='D']，原始事件链为"A-B-C-B-D"。由于 B
    ↪ 事件重复，最终的结果事件链为 A-B-C，最大长度为 3。

- `fixed`: 不允许事件的顺序发生交错，即事件发生的顺序必须和指定的事件链顺序一致。如，指定事件链为
    ↪ [event1='A', event2='B', event3='C', event4='D']，原始事件链为"A-B-D-C"，则结果事件链为 A
    ↪ -B，最大长度为 2

- `increase`: 选中的事件的时间戳必须按照指定事件链严格递增。
```

7.2.2.55.4 返回值

返回一个整数，表示在指定时间窗口内完成的最大连续步骤数。

7.2.2.55.5 举例

举例 1: default 模式

使用默认模式，筛选出不同user_id对应的最大连续事件数，时间窗口为5分钟：

```
CREATE TABLE events(
    user_id BIGINT,
    event_name VARCHAR(64),
    event_timestamp datetime,
    phone_brand varchar(64),
    tab_num int
) distributed by hash(user_id) buckets 3 properties("replication_num" = "1");

INSERT INTO
    events
VALUES
    (100123, '登录', '2022-05-14 10:01:00', 'HONOR', 1),
    (100123, '访问', '2022-05-14 10:02:00', 'HONOR', 2),
    (100123, '下单', '2022-05-14 10:04:00', 'HONOR', 3),
    (100123, '付款', '2022-05-14 10:10:00', 'HONOR', 4),
```

```
    (100125, '登录', '2022-05-15 11:00:00', 'XIAOMI', 1),
    (100125, '访问', '2022-05-15 11:01:00', 'XIAOMI', 2),
    (100125, '下单', '2022-05-15 11:02:00', 'XIAOMI', 6),
    (100126, '登录', '2022-05-15 12:00:00', 'IPHONE', 1),
    (100126, '访问', '2022-05-15 12:01:00', 'HONOR', 2),
    (100127, '登录', '2022-05-15 11:30:00', 'VIVO', 1),
    (100127, '访问', '2022-05-15 11:31:00', 'VIVO', 5);

SELECT
    user_id,
    window_funnel(
        300,
        "default",
        event_timestamp,
        event_name = '登录',
        event_name = '访问',
        event_name = '下单',
        event_name = '付款'
    ) AS level
FROM
    events
GROUP BY
    user_id
order BY
    user_id;
```

```
+---------+-------+
| user_id | level |
+---------+-------+
|  100123 |     3 |
|  100125 |     3 |
|  100126 |     2 |
|  100127 |     2 |
+---------+-------+
```

对于uesr_id=100123，因为付款事件发生的时间超出了时间窗口，所以匹配到的事件链是登陆-访问-下单。

举例 2: deduplication 模式

使用deduplication模式，筛选出不同user_id对应的最大连续事件数，时间窗口为1小时：

```
CREATE TABLE events(
    user_id BIGINT,
    event_name VARCHAR(64),
    event_timestamp datetime,
    phone_brand varchar(64),
    tab_num int
```

```
) distributed by hash(user_id) buckets 3 properties("replication_num" = "1");

INSERT INTO
    events
VALUES
    (100123, '登录', '2022-05-14 10:01:00', 'HONOR', 1),
    (100123, '访问', '2022-05-14 10:02:00', 'HONOR', 2),
    (100123, '登录', '2022-05-14 10:03:00', 'HONOR', 3),
    (100123, '下单', '2022-05-14 10:04:00', "HONOR", 4),
    (100123, '付款', '2022-05-14 10:10:00', 'HONOR', 4),
    (100125, '登录', '2022-05-15 11:00:00', 'XIAOMI', 1),
    (100125, '访问', '2022-05-15 11:01:00', 'XIAOMI', 2),
    (100125, '下单', '2022-05-15 11:02:00', 'XIAOMI', 6),
    (100126, '登录', '2022-05-15 12:00:00', 'IPHONE', 1),
    (100126, '访问', '2022-05-15 12:01:00', 'HONOR', 2),
    (100127, '登录', '2022-05-15 11:30:00', 'VIVO', 1),
    (100127, '访问', '2022-05-15 11:31:00', 'VIVO', 5);

SELECT
    user_id,
    window_funnel(
        3600,
        "deduplication",
        event_timestamp,
        event_name = '登录',
        event_name = '访问',
        event_name = '下单',
        event_name = '付款'
    ) AS level
FROM
    events
GROUP BY
    user_id
order BY
    user_id;
```

```
+---------+-------+
| user_id | level |
+---------+-------+
|  100123 |     2 |
|  100125 |     3 |
|  100126 |     2 |
|  100127 |     2 |
+---------+-------+
```

对于uesr_id=100123，匹配到访问事件后，登录事件重复出现，所以匹配到的事件链是登陆-访问。

举例 3: fixed 模式

使用fixed模式，筛选出不同user_id对应的最大连续事件数，时间窗口为1小时：

```
CREATE TABLE events(
    user_id BIGINT,
    event_name VARCHAR(64),
    event_timestamp datetime,
    phone_brand varchar(64),
    tab_num int
) distributed by hash(user_id) buckets 3 properties("replication_num" = "1");

INSERT INTO
    events
VALUES
    (100123, '登录', '2022-05-14 10:01:00', 'HONOR', 1),
    (100123, '访问', '2022-05-14 10:02:00', 'HONOR', 2),
    (100123, '下单', '2022-05-14 10:03:00', "HONOR", 4),
    (100123, '登录 2', '2022-05-14 10:04:00', 'HONOR', 3),
    (100123, '付款', '2022-05-14 10:10:00', 'HONOR', 4),
    (100125, '登录', '2022-05-15 11:00:00', 'XIAOMI', 1),
    (100125, '访问', '2022-05-15 11:01:00', 'XIAOMI', 2),
    (100125, '下单', '2022-05-15 11:02:00', 'XIAOMI', 6),
    (100126, '登录', '2022-05-15 12:00:00', 'IPHONE', 1),
    (100126, '访问', '2022-05-15 12:01:00', 'HONOR', 2),
    (100127, '登录', '2022-05-15 11:30:00', 'VIVO', 1),
    (100127, '访问', '2022-05-15 11:31:00', 'VIVO', 5);

SELECT
    user_id,
    window_funnel(
        3600,
        "fixed",
        event_timestamp,
        event_name = '登录',
        event_name = '访问',
        event_name = '下单',
        event_name = '付款'
    ) AS level
FROM
    events
GROUP BY
    user_id
order BY
    user_id;
```

```
+---------+-------+
| user_id | level |
+---------+-------+
|  100123 |     3 |
|  100125 |     3 |
|  100126 |     2 |
|  100127 |     2 |
+---------+-------+
```

对于uesr_id=100123，匹配到下单事件后，事件链被登录2事件打断，所以匹配到的事件链是登陆-访问-下单。

举例 4: increase 模式

使用increase模式，筛选出不同user_id对应的最大连续事件数，时间窗口为1小时：

```
CREATE TABLE events(
    user_id BIGINT,
    event_name VARCHAR(64),
    event_timestamp datetime,
    phone_brand varchar(64),
    tab_num int
) distributed by hash(user_id) buckets 3 properties("replication_num" = "1");

INSERT INTO
    events
VALUES
    (100123, '登录', '2022-05-14 10:01:00', 'HONOR', 1),
    (100123, '访问', '2022-05-14 10:02:00', 'HONOR', 2),
    (100123, '下单', '2022-05-14 10:04:00', "HONOR", 4),
    (100123, '付款', '2022-05-14 10:04:00', 'HONOR', 4),
    (100125, '登录', '2022-05-15 11:00:00', 'XIAOMI', 1),
    (100125, '访问', '2022-05-15 11:01:00', 'XIAOMI', 2),
    (100125, '下单', '2022-05-15 11:02:00', 'XIAOMI', 6),
    (100126, '登录', '2022-05-15 12:00:00', 'IPHONE', 1),
    (100126, '访问', '2022-05-15 12:01:00', 'HONOR', 2),
    (100127, '登录', '2022-05-15 11:30:00', 'VIVO', 1),
    (100127, '访问', '2022-05-15 11:31:00', 'VIVO', 5);

SELECT
    user_id,
    window_funnel(
        3600,
        "increase",
        event_timestamp,
        event_name = '登录',
        event_name = '访问',
```

```
        event_name = '下单',
        event_name = '付款'
    ) AS level
FROM
    events
GROUP BY
    user_id
order BY
    user_id;
```

```
+---------+-------+
| user_id | level |
+---------+-------+
|  100123 |     3 |
|  100125 |     3 |
|  100126 |     2 |
|  100127 |     2 |
+---------+-------+
```

对于uesr_id=100123，付款事件的时间戳与下单事件的时间戳发生在同一秒，没有递增，所以匹配到的事件链是登陆-访问-下单。

7.2.3 函数 Combinators

7.2.3.1 FOREACH

7.2.3.1.1 描述

将表的聚合函数转换为聚合相应数组项并返回结果数组的数组的聚合函数。例如，sum_foreach 对于数组 [1, 2], [3, 4, 5] 和 [6, 7] 返回结果 [10, 13, 5] 之后将相应的数组项添加在一起。

7.2.3.1.2 语法

```
AGGREGATE_FUNCTION_FOREACH(arg...)
```

7.2.3.1.3 举例

```
mysql [test]>select a , s from db;
+-----------+---------------+
| a         | s             |
+-----------+---------------+
| [1, 2, 3] | ["ab", "123"] |
| [20]      | ["cd"]        |
| [100]     | ["efg"]       |
| NULL      | NULL          |
```

```
| [null, 2] | [null, "c"]   |
+-----------+---------------+

mysql [test]>select sum_foreach(a) from db;
+----------------+
| sum_foreach(a) |
+----------------+
| [121, 4, 3]    |
+----------------+

mysql [test]>select count_foreach(s) from db;
+------------------+
| count_foreach(s) |
+------------------+
| [3, 2]           |
+------------------+

mysql [test]>select array_agg_foreach(a) from db;
+----------------------------------+
| array_agg_foreach(a)             |
+----------------------------------+
| [[1, 20, 100, null], [2, 2], [3]] |
+----------------------------------+

mysql [test]>select map_agg_foreach(a,a) from db;
+---------------------------------------+
| map_agg_foreach(a, a)                 |
+---------------------------------------+
| [{1:1, 20:20, 100:100}, {2:2}, {3:3}] |
+---------------------------------------+
```

keywords

FOREACH

7.2.3.2 MERGE

7.2.3.2.1 描述

将聚合中间结果进行聚合并计算获得实际结果。结果的类型与AGGREGATE_FUNCTION一致。

7.2.3.2.2 语法

```
AGGREGATE_FUNCTION_MERGE(agg_state)
```

7.2.3.2.3 举例

```
mysql [test]>select avg_merge(avg_state(1)) from d_table;
+-------------------------+
| avg_merge(avg_state(1)) |
+-------------------------+
|                       1 |
+-------------------------+
```

Keywords

AGG_STATE, MERGE

7.2.3.3 STATE

7.2.3.3.1 描述

返回聚合函数的中间结果，可以用于后续的聚合或者通过 merge 组合器获得实际计算结果，也可以直接写入 agg_state 类型的表保存下来。结果的类型为 agg_state，agg_state 中的函数签名为AGGREGATE_FUNCTION(arg...)。

7.2.3.3.2 语法

```
AGGREGATE_FUNCTION_STATE(arg...)
```

7.2.3.3.3 举例

```
mysql [test]>select avg_merge(t) from (select avg_union(avg_state(1)) as t from d_table group by
    ↪ k1)p;
+----------------+
| avg_merge(`t`) |
+----------------+
|              1 |
+----------------+
```

keywords

AGG_STATE,STATE

7.2.3.4 UNION

7.2.3.4.1 描述

将多个聚合中间结果聚合为一个。结果的类型为 agg_state，函数签名与入参一致。

7.2.3.4.2 语法

```
AGGREGATE_FUNCTION_UNION(agg_state)
```

7.2.3.4.3 举例

```
mysql [test]>select avg_merge(t) from (select avg_union(avg_state(1)) as t from d_table group by
    ↪ k1)p;
+----------------+
| avg_merge(`t`) |
+----------------+
|              1 |
+----------------+
```

keywords

AGG_STATE, UNION

7.2.4 分析函数 (窗口函数)

7.2.4.1 CUME_DIST

7.2.4.1.1 描述

CUME_DIST (Cumulative Distribution) 是一种窗口函数，它计算当前行值在排序后结果集中的相对排名。它返回的是当前行值在结果集中的累积分布值，范围从 0 到 1。对于给定的行，其累积分布值等于：(小于或等于当前行值的行数) / (窗口分区中的总行数)。

7.2.4.1.2 语法

```
CUME_DIST()
```

7.2.4.1.3 返回值

返回 DOUBLE 类型的数值，范围从 0 到 1。

7.2.4.1.4 举例

假设有一个表格 sales 包含销售数据，其中包括销售员姓名 (sales_person)、销售额 (sales_amount) 和销售日期 (sales_date)。我们想要计算每个销售员在每个销售日期的销售额占当日总销售额的累积百分比。

```
SELECT
    sales_person,
    sales_date,
    sales_amount,
    CUME_DIST() OVER (PARTITION BY sales_date ORDER BY sales_amount ASC) AS cumulative_sales_
        ↪ percentage
FROM
    sales;
```

假设表格 sales 中的数据如下：

```
+------+--------------+------------+--------------+
| id   | sales_person | sales_date | sales_amount |
+------+--------------+------------+--------------+
|    1 | Alice        | 2024-02-01 |         2000 |
|    2 | Bob          | 2024-02-01 |         1500 |
|    3 | Alice        | 2024-02-02 |         1800 |
|    4 | Bob          | 2024-02-02 |         1200 |
|    5 | Alice        | 2024-02-03 |         2200 |
|    6 | Bob          | 2024-02-03 |         1900 |
|    7 | Tom          | 2024-02-03 |         2000 |
|    8 | Jerry        | 2024-02-03 |         2000 |
+------+--------------+------------+--------------+
```

执行上述 SQL 查询后，结果将显示每个销售员在每个销售日期的销售额以及其在该销售日期的累积百分比排名。

```
+--------------+------------+--------------+-----------------------------+
| sales_person | sales_date | sales_amount | cumulative_sales_percentage |
+--------------+------------+--------------+-----------------------------+
| Bob          | 2024-02-01 |         1500 |                         0.5 |
| Alice        | 2024-02-01 |         2000 |                           1 |
| Bob          | 2024-02-02 |         1200 |                         0.5 |
| Alice        | 2024-02-02 |         1800 |                           1 |
| Bob          | 2024-02-03 |         1900 |                        0.25 |
| Tom          | 2024-02-03 |         2000 |                        0.75 |
| Jerry        | 2024-02-03 |         2000 |                        0.75 |
| Alice        | 2024-02-03 |         2200 |                           1 |
+--------------+------------+--------------+-----------------------------+
```

在这个例子中，CUME_DIST() 函数根据每个销售日期对销售额进行排序，然后计算每个销售员在该销售日期的销售额占当日总销售额的累积百分比。由于我们使用了 PARTITION BY sales_date，所以计算是在每个销售日期内进行的，销售员在不同日期的销售额被分别计算。

7.2.4.2 DENSE_RANK

7.2.4.2.1 描述

DENSE_RANK() 是一种窗口函数，用于计算分组内值的排名，与 RANK() 不同的是，DENSE_RANK() 返回的排名是连续的，不会出现空缺数字。排名值从 1 开始按顺序递增，如果出现相同的值，它们将具有相同的排名。

7.2.4.2.2 语法

```
DENSE_RANK()
```

7.2.4.2.3 返回值

返回 BIGINT 类型的排名值，从 1 开始。

7.2.4.2.4 举例

```
select x, y, dense_rank() over(partition by x order by y) as rank from int_t;
```

```
+-----+-----+------+
| x   | y   | rank |
| --- | --- | ---- |
| 1   | 1   | 1    |
| 1   | 2   | 2    |
| 1   | 2   | 2    | -- 相同值具有相同排名 |
| 2   | 1   | 1    |
| 2   | 2   | 2    |
| 2   | 3   | 3    | -- 排名连续，没有空缺 |
| 3   | 1   | 1    |
| 3   | 1   | 1    |
| 3   | 2   | 2    |
+-----+-----+------+
```

7.2.4.3 FIRST_VALUE

7.2.4.3.1 描述

FIRST_VALUE() 是一个窗口函数，用于返回窗口分区中有序数据集的第一个值。可以通过 IGNORE NULLS 选项来控制是否忽略空值。

7.2.4.3.2 语法

```
FIRST_VALUE(<expr>[, <ignore_null>])
```

7.2.4.3.3 参数

参数	说明
expr	需要获取第一个值的表达式
ignore_null	可选。参数 ignore_null 默认值为 false, 设置后会忽略空值

7.2.4.3.4 返回值

返回与输入表达式相同的数据类型。

7.2.4.3.5 举例

```
WITH example_data AS (
    SELECT 1 as column1, NULL as column2, 'A' as group_name
    UNION ALL
    SELECT 1, 10, 'A'
    UNION ALL
    SELECT 1, NULL, 'A'
    UNION ALL
    SELECT 1, 20, 'A'
    UNION ALL
    SELECT 2, NULL, 'B'
    UNION ALL
    SELECT 2, 30, 'B'
    UNION ALL
    SELECT 2, 40, 'B'
)
SELECT
    group_name,
    column1,
    column2,
    FIRST_VALUE(column2) OVER (
        PARTITION BY column1
        ORDER BY column2 NULLS LAST
    ) AS first_value_default,
    FIRST_VALUE(column2, true) OVER (
        PARTITION BY column1
        ORDER BY column2
    ) AS first_value_ignore_null
FROM example_data
ORDER BY column1, column2;
```

```
+------------+---------+---------+--------------------+-------------------------+
| group_name | column1 | column2 | first_value_default | first_value_ignore_null |
+------------+---------+---------+--------------------+-------------------------+
| A          |       1 |    NULL |                 10 |                    NULL |
| A          |       1 |    NULL |                 10 |                    NULL |
| A          |       1 |      10 |                 10 |                      10 |
| A          |       1 |      20 |                 10 |                      10 |
| B          |       2 |    NULL |                 30 |                    NULL |
| B          |       2 |      30 |                 30 |                      30 |
| B          |       2 |      40 |                 30 |                      30 |
+------------+---------+---------+--------------------+-------------------------+
```

7.2.4.4 LAG

7.2.4.4.1 描述

LAG() 是一个窗口函数，用于访问当前行之前的行数据，而无需进行自连接。它可以获取分区内当前行之前第 N 行的值。

7.2.4.4.2 语法

```
LAG ( <expr>, <offset>, <default> )
```

7.2.4.4.3 参数

参数	说明
expr	需要获取值的表达式
offset	向前偏移的行数。
default	当偏移超出窗口范围时返回的默认值

7.2.4.4.4 返回值

返回与输入表达式相同的数据类型。

7.2.4.4.5 举例

计算每个销售员当前销售额与前一天销售额的差值：

```
select stock_symbol, closing_date, closing_price,
lag(closing_price,1, 0) over (partition by stock_symbol order by closing_date) as "yesterday
    ↪ closing"
from stock_ticker
order by closing_date;
```

```
+--------------+---------------------+---------------+-------------------+
| stock_symbol | closing_date        | closing_price | yesterday closing |
| ------------ | ------------------- | ------------- | ----------------- |
| JDR          | 2014-09-13 00:00:00 | 12.86         | 0                 |
| JDR          | 2014-09-14 00:00:00 | 12.89         | 12.86             |
| JDR          | 2014-09-15 00:00:00 | 12.94         | 12.89             |
| JDR          | 2014-09-16 00:00:00 | 12.55         | 12.94             |
| JDR          | 2014-09-17 00:00:00 | 14.03         | 12.55             |
| JDR          | 2014-09-18 00:00:00 | 14.75         | 14.03             |
| JDR          | 2014-09-19 00:00:00 | 13.98         | 14.75             |
+--------------+---------------------+---------------+-------------------+
```

7.2.4.5 LAST_VALUE

7.2.4.5.1 描述

LAST_VALUE() 是一个窗口函数，用于返回窗口范围内的最后一个值。可以通过 IGNORE NULLS 选项来控制是否忽略空值。

7.2.4.5.2 语法

```
LAST_VALUE(<expr>[, <ignore_null>])
```

7.2.4.5.3 参数

参数	说明
expr	需要获取最后一个值的表达式
ignore_null	可选。参数 ignore_null 默认值为 false, 设置后会忽略空值

7.2.4.5.4 返回值

返回与输入表达式相同的数据类型。

7.2.4.5.5 举例

```
WITH example_data AS (
    SELECT 1 as id, 21 as myday, '04-21-11' as time_col, NULL as state
    UNION ALL
    SELECT 2, 21, '04-21-12', 2
    UNION ALL
    SELECT 3, 21, '04-21-13', 3
    UNION ALL
    SELECT 4, 22, '04-22-10-21', NULL
    UNION ALL
    SELECT 5, 22, '04-22-10-22', NULL
    UNION ALL
    SELECT 6, 22, '04-22-10-23', 5
    UNION ALL
    SELECT 7, 22, '04-22-10-24', NULL
    UNION ALL
    SELECT 8, 22, '04-22-10-25', 9
    UNION ALL
    SELECT 9, 23, '04-23-11', NULL
    UNION ALL
    SELECT 10, 23, '04-23-12', 10
    UNION ALL
    SELECT 11, 23, '04-23-13', NULL
    UNION ALL
```

```
    SELECT 12, 24, '02-24-10-21', NULL
)
SELECT
    *,
    last_value(`state`, 1) OVER(
        PARTITION BY `myday`
        ORDER BY `time_col` DESC
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) as ignore_null,
    last_value(`state`, 0) OVER(
        PARTITION BY `myday`
        ORDER BY `time_col` DESC
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) as not_ignore_null,
    last_value(`state`) OVER(
        PARTITION BY `myday`
        ORDER BY `time_col` DESC
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) as ignore_null_default
FROM example_data
ORDER BY `id`, `myday`, `time_col`;
```

```
+------+-------+-------------+-------+-------------+-----------------+---------------------+
| id   | myday | time_col    | state | ignore_null | not_ignore_null | ignore_null_default |
+------+-------+-------------+-------+-------------+-----------------+---------------------+
|    1 |    21 | 04-21-11    |  NULL |           2 |            NULL |                NULL |
|    2 |    21 | 04-21-12    |     2 |           2 |            NULL |                NULL |
|    3 |    21 | 04-21-13    |     3 |           2 |               2 |                   2 |
|    4 |    22 | 04-22-10-21 |  NULL |        NULL |            NULL |                NULL |
|    5 |    22 | 04-22-10-22 |  NULL |           5 |            NULL |                NULL |
|    6 |    22 | 04-22-10-23 |     5 |           5 |            NULL |                NULL |
|    7 |    22 | 04-22-10-24 |  NULL |           5 |               5 |                   5 |
|    8 |    22 | 04-22-10-25 |     9 |           9 |            NULL |                NULL |
|    9 |    23 | 04-23-11    |  NULL |          10 |            NULL |                NULL |
|   10 |    23 | 04-23-12    |    10 |          10 |            NULL |                NULL |
|   11 |    23 | 04-23-13    |  NULL |          10 |              10 |                  10 |
|   12 |    24 | 02-24-10-21 |  NULL |        NULL |            NULL |                NULL |
+------+-------+-------------+-------+-------------+-----------------+---------------------+
```

7.2.4.6 LEAD

7.2.4.6.1 描述

LEAD() 是一个窗口函数，用于访问当前行之后的行数据，而无需进行自连接。它可以获取分区内当前行之后第 N 行的值。

7.2.4.6.2 语法

```
LEAD ( <expr> [ , <offset> [ , <default> ] ] )
```

7.2.4.6.3 参数

参数	说明
expr	需要获取值的表达式
offset	可选。向后偏移的行数。默认值为 1。
default	可选。当偏移超出窗口范围时返回的默认值。默认为 NULL

7.2.4.6.4 返回值

返回与输入表达式相同的数据类型。

7.2.4.6.5 举例

计算每个销售员当前销售额与下一天销售额的差值：

```
select stock_symbol, closing_date, closing_price,
case
(lead(closing_price,1, 0)
over (partition by stock_symbol order by closing_date)-closing_price) > 0
when true then "higher"
when false then "flat or lower"
end as "trending"
from stock_ticker
order by closing_date;
```

```
+--------------+---------------------+---------------+---------------+
| stock_symbol | closing_date        | closing_price | trending      |
| ------------ | ------------------- | ------------- | ------------- |
| JDR          | 2014-09-13 00:00:00 | 12.86         | higher        |
| JDR          | 2014-09-14 00:00:00 | 12.89         | higher        |
| JDR          | 2014-09-15 00:00:00 | 12.94         | flat or lower |
| JDR          | 2014-09-16 00:00:00 | 12.55         | higher        |
| JDR          | 2014-09-17 00:00:00 | 14.03         | higher        |
| JDR          | 2014-09-18 00:00:00 | 14.75         | flat or lower |
| JDR          | 2014-09-19 00:00:00 | 13.98         | flat or lower |
+--------------+---------------------+---------------+---------------+
```

7.2.4.7 NTILE

7.2.4.7.1 描述

NTILE() 是一个窗口函数，用于将有序数据集平均分配到指定数量的桶中。桶的编号从 1 开始顺序编号，直到指定的桶数。当数据无法平均分配时，优先将多出的记录分配给编号较小的桶，使得各个桶中的行数最多相差 1。

7.2.4.7.2 语法

```
NTILE( <constant_value> )
```

7.2.4.7.3 参数

参数	说明
constant_value	必需。指定要分配的桶数量，必须是正整数

7.2.4.7.4 返回值

返回 BIGINT 类型的桶编号，范围从 1 到指定的桶数。

7.2.4.7.5 使用说明

如果语句中同时包含 NTILE 函数的 ORDER BY 子句和输出结果的 ORDER BY 子句，这两个排序是独立的：- NTILE 函数的 ORDER BY 决定了行被分配到哪个桶中 - 输出的 ORDER BY 决定了结果的显示顺序

7.2.4.7.6 举例

```
SELECT
    name,
    score,
    NTILE(4) OVER (ORDER BY score DESC) as quarter
FROM student_scores;
```

```
+----------+-------+---------+
| name     | score | quarter |
+----------+-------+---------+
| Alice    | 98    | 1       |  -- 前 25% 的成绩
| Bob      | 95    | 1       |
| Charlie  | 90    | 2       |  -- 前 25-50% 的成绩
| David    | 85    | 2       |
| Eve      | 82    | 3       |  -- 前 50-75% 的成绩
| Frank    | 78    | 3       |
| Grace    | 75    | 4       |  -- 后 25% 的成绩
| Henry    | 70    | 4       |
+----------+-------+---------+
```

7.2.4.8 OVERVIEW

7.2.4.8.1 描述

窗口函数（也称为分析函数）是一类特殊的内置函数，它能在保留原始行的基础上进行计算。与聚合函数不同，窗口函数：

- 在特定窗口范围内处理数据，而不是按 GROUP BY 分组
- 为结果集的每一行计算一个值
- 可以在 SELECT 列表中添加额外的列
- 在查询处理中最后执行（在 JOIN、 WHERE、 GROUP BY 之后）

窗口函数常用于金融和科学计算领域，用于分析趋势、计算离群值和数据分桶等场景。

7.2.4.8.2 语法

```
function(<args>) OVER(
    [PARTITION BY <expr> [, <expr> ...]]
    [ORDER BY <expr> [ASC | DESC] [, <expr> [ASC | DESC] ...]]
    [<window_clause>]
)
```

7.2.4.8.3 参数

参数	说明
<args>	窗口函数的输入参数，具体参数根据所使用的函数而定

参数	说明
< ↪ function ↪ >	支持的函数包括：AVG(), COUNT(), DENSE_RANK(), FIRST_VALUE(), LAG(), LAST_VALUE(), LEAD(), MAX(), MIN(), RANK(), ROW_NUMBER(), SUM() 和所有聚合函数
< ↪ partition ↪ _by>	类似于 GROUP BY，按指定列对数据进行分组
<order_ ↪ by>	定义窗口内数据的排序方式

参数	说明
<window ↪ _ ↪ clause ↪ >	定义窗口范围，语法为：ROWS BETWEEN [{ m | UNBOUNDED } PRECEDING | CURRENT ROW] [AND [CURRENT ROW | { UNBOUNDED | n } FOLLOWING]]

7.2.4.8.4 返回值

返回与输入表达式相同的数据类型。

7.2.4.8.5 举例

1. 假设我们有如下的股票数据，股票代码是 JDR，closing price 是每天的收盘价。

```
create table stock_ticker (stock_symbol string, closing_price decimal(8,2), closing_date
    ↪ timestamp);
...load some data...
select * from stock_ticker order by stock_symbol, closing_date
```

```
| stock_symbol | closing_price | closing_date        |
|--------------|---------------|---------------------|
| JDR          | 12.86         | 2014-10-02 00:00:00 |
| JDR          | 12.89         | 2014-10-03 00:00:00 |
| JDR          | 12.94         | 2014-10-04 00:00:00 |
| JDR          | 12.55         | 2014-10-05 00:00:00 |
| JDR          | 14.03         | 2014-10-06 00:00:00 |
| JDR          | 14.75         | 2014-10-07 00:00:00 |
| JDR          | 13.98         | 2014-10-08 00:00:00 |
```

2. 这个查询使用分析函数产生 moving_average 这一列，它的值是 3 天的股票均价，即前一天、当前以及后一天三天的均价。第一天没有前一天的值，最后一天没有后一天的值，所以这两行只计算了两天的均值。这里 Partition By 没有起到作用，因为所有的数据都是 JDR 的数据，但如果还有其他股票信息，Partition By 会保证分析函数值作用在本 Partition 之内。

```
select stock_symbol, closing_date, closing_price,
avg(closing_price) over (partition by stock_symbol order by closing_date
rows between 1 preceding and 1 following) as moving_average
from stock_ticker;
```

```
| stock_symbol | closing_date        | closing_price | moving_average |
|--------------|---------------------|---------------|----------------|
| JDR          | 2014-10-02 00:00:00 | 12.86         | 12.87          |
| JDR          | 2014-10-03 00:00:00 | 12.89         | 12.89          |
| JDR          | 2014-10-04 00:00:00 | 12.94         | 12.79          |
| JDR          | 2014-10-05 00:00:00 | 12.55         | 13.17          |
| JDR          | 2014-10-06 00:00:00 | 14.03         | 13.77          |
| JDR          | 2014-10-07 00:00:00 | 14.75         | 14.25          |
| JDR          | 2014-10-08 00:00:00 | 13.98         | 14.36          |
```

7.2.4.9 PERCENT_RANK

7.2.4.9.1 描述

PERCENT_RANK() 是一个窗口函数，用于计算分区或结果集中行的相对排名，返回值范围从 0.0 到 1.0。对于给定的行，其计算公式为：(rank - 1) / (total_rows - 1)，其中 rank 是当前行的排名，total_rows 是分区中的总行数。

7.2.4.9.2 语法

```
PERCENT_RANK()
```

7.2.4.9.3 返回值

返回 DOUBLE 类型的数值，范围从 0.0 到 1.0：- 对于分区内的第一行，始终返回 0 - 对于分区内的最后一行，始终返回 1 - 对于相同的值，返回相同的百分比排名

7.2.4.9.4 举例

```
CREATE TABLE test_percent_rank (
    productLine VARCHAR,
    orderYear INT,
    orderValue DOUBLE,
    percentile_rank DOUBLE
) ENGINE=OLAP
```

```
DISTRIBUTED BY HASH(`orderYear`) BUCKETS 4
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);
```

```
INSERT INTO test_percent_rank (productLine, orderYear, orderValue, percentile_rank) VALUES
('Motorcycles', 2003, 2440.50, 0.00),
('Trains', 2003, 2770.95, 0.17),
('Trucks and Buses', 2003, 3284.28, 0.33),
('Vintage Cars', 2003, 4080.00, 0.50),
('Planes', 2003, 4825.44, 0.67),
('Ships', 2003, 5072.71, 0.83),
('Classic Cars', 2003, 5571.80, 1.00),
('Motorcycles', 2004, 2598.77, 0.00),
('Vintage Cars', 2004, 2819.28, 0.17),
('Planes', 2004, 2857.35, 0.33),
('Ships', 2004, 4301.15, 0.50),
('Trucks and Buses', 2004, 4615.64, 0.67),
('Trains', 2004, 4646.88, 0.83),
('Classic Cars', 2004, 8124.98, 1.00),
('Ships', 2005, 1603.20, 0.00),
('Motorcycles', 2005, 3774.00, 0.17),
('Planes', 2005, 4018.00, 0.50),
('Vintage Cars', 2005, 5346.50, 0.67),
('Classic Cars', 2005, 5971.35, 0.83),
('Trucks and Buses', 2005, 6295.03, 1.00);
```

```
SELECT
    productLine,
    orderYear,
    orderValue,
    ROUND(
    PERCENT_RANK()
    OVER (
        PARTITION BY orderYear
        ORDER BY orderValue
    ),2) percentile_rank
FROM
    test_percent_rank
ORDER BY
    orderYear;
```

```
+------------------+-----------+------------+-----------------+
| productLine      | orderYear | orderValue | percentile_rank |
+------------------+-----------+------------+-----------------+
```

```
| Motorcycles      |      2003 |     2440.5 |               0 |
| Trains           |      2003 |    2770.95 |            0.17 |
| Trucks and Buses |      2003 |    3284.28 |            0.33 |
| Vintage Cars     |      2003 |       4080 |             0.5 |
| Planes           |      2003 |    4825.44 |            0.67 |
| Ships            |      2003 |    5072.71 |            0.83 |
| Classic Cars     |      2003 |     5571.8 |               1 |
| Motorcycles      |      2004 |    2598.77 |               0 |
| Vintage Cars     |      2004 |    2819.28 |            0.17 |
| Planes           |      2004 |    2857.35 |            0.33 |
| Ships            |      2004 |    4301.15 |             0.5 |
| Trucks and Buses |      2004 |    4615.64 |            0.67 |
| Trains           |      2004 |    4646.88 |            0.83 |
| Classic Cars     |      2004 |    8124.98 |               1 |
| Ships            |      2005 |     1603.2 |               0 |
| Motorcycles      |      2005 |       3774 |             0.2 |
| Planes           |      2005 |       4018 |             0.4 |
| Vintage Cars     |      2005 |     5346.5 |             0.6 |
| Classic Cars     |      2005 |    5971.35 |             0.8 |
| Trucks and Buses |      2005 |    6295.03 |               1 |
+------------------+-----------+------------+-----------------+
```

7.2.4.10 RANK

7.2.4.10.1 描述

RANK() 是一个窗口函数，用于返回有序数据集中值的排名。排名从 1 开始按顺序递增。当出现相同值时，这些值获得相同的排名，但会导致排名序列出现间隔。例如，如果前两行并列排名第 1，则下一个不同的值将排名第 3（而不是第 2）。

7.2.4.10.2 语法

```
RANK()
```

7.2.4.10.3 返回值

返回 BIGINT 类型的排名值。对于相同的值返回相同的排名，但会在序列中产生间隔。

7.2.4.10.4 举例

```
SELECT
    department,
    employee_name,
    salary,
```

```
    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) as salary_rank
FROM employees;
```

```
+------------+---------------+--------+-------------+
| department | employee_name | salary | salary_rank |
+------------+---------------+--------+-------------+
| Sales      | Alice         | 10000  | 1           |
| Sales      | Bob           | 10000  | 1           |
| Sales      | Charlie       | 8000   | 3           |  -- 注意这里是 3 而不是 2
| IT         | David         | 12000  | 1           |
| IT         | Eve           | 11000  | 2           |
| IT         | Frank         | 11000  | 2           |
| IT         | Grace         | 9000   | 4           |  -- 注意这里是 4 而不是 3
+------------+---------------+--------+-------------+
```

在这个例子中，数据按部门分区，并在每个部门内根据工资进行排名。当出现相同工资时（如 Alice 和 Bob、Eve 和 Frank），它们获得相同的排名，但会导致后续排名出现间隔。

7.2.4.11 ROW_NUMBER

7.2.4.11.1 描述

ROW_NUMBER() 是一个窗口函数，用于为分区内的每一行分配一个唯一的序号。序号从 1 开始连续递增。与 RANK() 和 DENSE_RANK() 不同，ROW_NUMBER() 即使对于相同的值也会分配不同的序号，确保每行都有唯一的编号。

7.2.4.11.2 语法

```
ROW_NUMBER()
```

7.2.4.11.3 返回值

返回 BIGINT 类型的序号，从 1 开始连续递增。在每个分区内，序号都是唯一的。

7.2.4.11.4 举例

```
select x, y, row_number() over(partition by x order by y) as rank from int_t;
```

```
+-----+-----+------+
| x   | y   | rank |
| --- | --- | ---- |
| 1   | 1   | 1    |
```

```
| 1    | 2    | 2     |
| 1    | 2    | 3     |
| 2    | 1    | 1     |
| 2    | 2    | 2     |
| 2    | 3    | 3     |
| 3    | 1    | 1     |
| 3    | 1    | 2     |
| 3    | 2    | 3     |
+-----+-----+------+
```

7.2.5 表函数

7.2.5.1 EXPLODE

7.2.5.1.1 描述

explode 函数接受一个数组，会将数组的每个元素映射为单独的行。通常与 LATERAL VIEW 配合使用，以将嵌套数据结构展开为标准的平面表格式。explode 和 explode_outer 区别主要在于空值处理。

7.2.5.1.2 语法

```
EXPLODE(<array>)
EXPLODE_OUTER(<array>)
```

7.2.5.1.3 参数

参数	说明
<arr>	array 类型

7.2.5.1.4 返回值

当数组不为空或 NULL 时，explode 和 explode_outer 的返回值相同。

当数据为空或 NULL 时：

explode 不会产生任何行，并且会过滤掉这些记录。

explode_outer 如果数组是空的，explode_outer 会生成一行记录，但展开的列值会是 NULL。如果数组为 NULL，同样会保留一行，并返回 NULL。

7.2.5.1.5 举例

```
select e1 from (select 1 k1) as t lateral view explode([1,2,3]) tmp1 as e1;
```

```
+------+
| e1   |
+------+
|    1 |
|    2 |
|    3 |
+------+
```

```
select e1 from (select 1 k1) as t lateral view explode_outer(null) tmp1 as e1;
```

```
+------+
| e1   |
+------+
| NULL |
+------+
```

```
select e1 from (select 1 k1) as t lateral view explode([]) tmp1 as e1;
Empty set (0.010 sec)
```

```
select e1 from (select 1 k1) as t lateral view explode([null,1,null]) tmp1 as e1;
```

```
+------+
| e1   |
+------+
| NULL |
|    1 |
| NULL |
+------+
```

```
select e1 from (select 1 k1) as t lateral view explode_outer([null,1,null]) tmp1 as e1;
```

```
+------+
| e1   |
+------+
| NULL |
|    1 |
| NULL |
+------+
```

7.2.5.2 EXPLODE_BITMAP

7.2.5.2.1 描述

explode_bitmap 表函数，接受一个位图（bitmap）类型的数据，将位图中的每个 bit（位）映射为单独的行。通常用于处理位图数据，将位图中的每个元素展开成单独的记录。需配合 Lateral View 使用。

explode_bitmap_outer 与 explode_bitmap 类似，但在处理空值或 NULL 时，行为有所不同。它允许空位图或 NULL 位图的记录存在，并在返回结果中将空位图或者 NULL 位图展开为 NULL 行。

7.2.5.2.2 语法

```
EXPLODE_BITMAP(<bitmap>)
EXPLODE_BITMAP_OUTER(<bitmap>)
```

7.2.5.2.3 参数

参数	说明
<bitmap>	bitmap 类型

7.2.5.2.4 返回值

返回位图中每一位对应的行，其中每一行包含一个位值。

7.2.5.2.5 举例

```
CREATE TABLE example1 (
    k1 INT
)DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
insert into example1 values(1),(2),(3),(4),(5),(6);
```

```
select k1 from example1 order by k1;
```

```
+------+
| k1   |
+------+
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
+------+
```

```
select k1, e1 from example1 lateral view explode_bitmap(bitmap_empty()) tmp1 as e1 order by k1,
    ↪ e1;
Empty set
```

```
select k1, e1 from example1 lateral view explode_bitmap(bitmap_from_string("1")) tmp1 as e1 order
    ↪  by k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    1 |    1 |
|    2 |    1 |
|    3 |    1 |
|    4 |    1 |
|    5 |    1 |
|    6 |    1 |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_bitmap(bitmap_from_string("1,2")) tmp1 as e1
    ↪ order by k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    1 |    1 |
|    1 |    2 |
|    2 |    1 |
|    2 |    2 |
|    3 |    1 |
|    3 |    2 |
|    4 |    1 |
|    4 |    2 |
|    5 |    1 |
|    5 |    2 |
|    6 |    1 |
|    6 |    2 |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_bitmap(bitmap_from_string("1,1000")) tmp1 as e1
    ↪ order by k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
```

```
|    1 |    1 |
|    1 | 1000 |
|    2 |    1 |
|    2 | 1000 |
|    3 |    1 |
|    3 | 1000 |
|    4 |    1 |
|    4 | 1000 |
|    5 |    1 |
|    5 | 1000 |
|    6 |    1 |
|    6 | 1000 |
+------+------+
```

```
select k1, e1, e2 from example1
lateral view explode_bitmap(bitmap_from_string("1,1000")) tmp1 as e1
lateral view explode_split("a,b", ",") tmp2 as e2 order by k1, e1, e2;
```

```
+------+------+------+
| k1   | e1   | e2   |
+------+------+------+
|    1 |    1 | a    |
|    1 |    1 | b    |
|    1 | 1000 | a    |
|    1 | 1000 | b    |
|    2 |    1 | a    |
|    2 |    1 | b    |
|    2 | 1000 | a    |
|    2 | 1000 | b    |
|    3 |    1 | a    |
|    3 |    1 | b    |
|    3 | 1000 | a    |
|    3 | 1000 | b    |
|    4 |    1 | a    |
|    4 |    1 | b    |
|    4 | 1000 | a    |
|    4 | 1000 | b    |
|    5 |    1 | a    |
|    5 |    1 | b    |
|    5 | 1000 | a    |
|    5 | 1000 | b    |
|    6 |    1 | a    |
|    6 |    1 | b    |
|    6 | 1000 | a    |
|    6 | 1000 | b    |
```

```
+------+------+------+
```

```
CREATE TABLE example (
    k1 INT,
    v1 bitmap
)DUPLICATE KEY(k1)
DISTRIBUTED BY HASH(k1) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
insert into example values(1,to_bitmap('10101')),(2,to_bitmap('0')),(3,to_bitmap(NULL));
```

```
SELECT id, k, v
FROM example
LATERAL VIEW explode_json_object(value_json) exploded_table AS k , v;
```

```
+------+-------+
| k1   | bit   |
+------+-------+
|    2 |     0 |
|    1 | 10101 |
+------+-------+
```

```
SELECT id, k, v
FROM example
LATERAL VIEW explode_json_object_outer(value_json) exploded_table AS k, v;
```

```
+------+-------+
| k1   | bit   |
+------+-------+
|    2 |     0 |
|    1 | 10101 |
|    3 |  NULL |
+------+-------+
```

7.2.5.3 EXPLODE_JSON_ARRAY_DOUBLE

7.2.5.3.1 描述

explode_json_array_double 表函数，接受一个 JSON 数组，其中每个元素是双精度浮点数类型，将该 JSON 数组中的每个浮点数展开为多行，每行包含一个浮点数。配合 LATERAL VIEW 使用。

explode_json_array_double_outer 和 explode_json_array_double 类似，对于空值或 NULL 的处理不同。

如果 JSON 字符串本身为 NULL，OUTER 版本会返回一行，且该行中的值为 NULL。普通版本会完全忽略这类记录。

如果 JSON 数组为空，OUTER 版本会返回一行，且该行的值为 NULL。普通版本则不会返回任何结果。

7.2.5.3.2 语法

```
EXPLODE_JSON_ARRAY_DOUBLE(<json>)
EXPLODE_JSON_ARRAY_DOUBLE_OUTER(<json>)
```

7.2.5.3.3 参数

参数	说明
<json>	json 类型

7.2.5.3.4 返回值

展开 JSON 数组，每个元素生成一行，返回双精度浮点数列。

7.2.5.3.5 举例

```
CREATE TABLE json_array_example (
    id INT,
    json_array STRING
)DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
INSERT INTO json_array_example (id, json_array) VALUES
(1, '[1, 2, 3, 4, 5]'),
(2, '[1.1, 2.2, 3.3, 4.4]'),
(3, '["apple", "banana", "cherry"]'),
(4, '[{"a": 1}, {"b": 2}, {"c": 3}]'),
(5, '[]'),
(6, 'NULL');
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_DOUBLE(json_array) tmp1 AS e1
WHERE id = 2;
```

```
+------+------+
| id   | e1   |
+------+------+
|    2 |  1.1 |
|    2 |  2.2 |
|    2 |  3.3 |
|    2 |  4.4 |
```

```
+------+------+
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_DOUBLE(json_array) tmp1 AS e1
WHERE id = 6;
Empty set (0.01 sec)
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_DOUBLE_OUTER(json_array) tmp1 AS e1
WHERE id = 6;
```

```
+------+------+
| id   | e1   |
+------+------+
|    6 | NULL |
+------+------+
```

7.2.5.4 EXPLODE_JSON_ARRAY_INT

7.2.5.4.1 描述

explode_json_array_int 表函数，接受一个 JSON 数组，其中每个元素是整数类型，将该 JSON 数组中的每个整数展开为多行，每行包含一个整数。配合 LATERAL VIEW 使用。

explode_json_array_int_outer 和 explode_json_array_int 类似，对于空值或 NULL 的处理不同。

如果 JSON 字符串本身为 NULL，OUTER 版本会返回一行，且该行中的值为 NULL。普通版本会完全忽略这类记录。

如果 JSON 数组为空，OUTER 版本会返回一行，且该行的值为 NULL。普通版本则不会返回任何结果。

7.2.5.4.2 语法

```
EXPLODE_JSON_ARRAY_INT(<json>)
EXPLODE_JSON_ARRAY_INT_OUTER(<json>)
```

7.2.5.4.3 参数

参数	说明
<json>	json 类型

7.2.5.4.4 返回值

展开 JSON 数组，每个元素生成一行，返回整数列。

7.2.5.4.5 举例

```
CREATE TABLE json_array_example (
    id INT,
    json_array STRING
)DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
INSERT INTO json_array_example (id, json_array) VALUES
(1, '[1, 2, 3, 4, 5]'),
(2, '[1.1, 2.2, 3.3, 4.4]'),
(3, '["apple", "banana", "cherry"]'),
(4, '[{"a": 1}, {"b": 2}, {"c": 3}]'),
(5, '[]'),
(6, 'NULL');
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_INT(json_array) tmp1 AS e1
WHERE id = 1;
```

```
+------+------+
| id   | e1   |
+------+------+
|    1 |    1 |
|    1 |    2 |
|    1 |    3 |
|    1 |    4 |
|    1 |    5 |
+------+------+
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_INT(json_array) tmp1 AS e1
WHERE id = 5;
Empty set (0.01 sec)
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_INT_OUTER(json_array) tmp1 AS e1
WHERE id = 5;
```

```
+------+------+
| id   | e1   |
+------+------+
|    5 | NULL |
+------+------+
```

7.2.5.5 EXPLODE_JSON_ARRAY_JSON

7.2.5.5.1 描述

explode_json_array_json 表函数，接受一个 JSON 数组，其中每个元素是 JSON 对象类型，将该 JSON 数组中的每个 JSON 对象展开为多行，每行包含一个 JSON 对象。配合 LATERAL VIEW 使用。

7.2.5.5.2 语法

```
EXPLODE_JSON_ARRAY_JSON(<json>)
EXPLODE_JSON_ARRAY_JSON_OUTER(<json>)
```

7.2.5.5.3 参数

参数	说明
<json>	json 类型

7.2.5.5.4 返回值

展开 JSON 数组，每个元素生成一行，返回 JSON 对象列。

7.2.5.5.5 举例

```
CREATE TABLE json_array_example (
    id INT,
    json_array STRING
)DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
INSERT INTO json_array_example (id, json_array) VALUES
(1, '[1, 2, 3, 4, 5]'),
(2, '[1.1, 2.2, 3.3, 4.4]'),
(3, '["apple", "banana", "cherry"]'),
(4, '[{"a": 1}, {"b": 2}, {"c": 3}]'),
```

```
(5, '[]'),
(6, 'NULL');
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_JSON(json_array) tmp1 AS e1
WHERE id = 4;
```

```
+------+---------+
| id   | e1      |
+------+---------+
|    4 | {"a":1} |
|    4 | {"b":2} |
|    4 | {"c":3} |
+------+---------+
```

7.2.5.6 EXPLODE_JSON_ARRAY_STRING

7.2.5.6.1 描述

explode_json_array_string 表函数，接受一个 JSON 数组，其中每个元素是字符串类型，将该 JSON 数组中的每个字符串展开为多行，每行包含一个字符串。配合 LATERAL VIEW 使用。

explode_json_array_string_outer 和 explode_json_array_string 类似，对于空值或 NULL 的处理不同。

如果 JSON 字符串本身为 NULL，OUTER 版本会返回一行，且该行中的值为 NULL。普通版本会完全忽略这类记录。

如果 JSON 数组为空，OUTER 版本会返回一行，且该行的值为 NULL。普通版本则不会返回任何结果。

7.2.5.6.2 语法

```
EXPLODE_JSON_ARRAY_STRING(<json>)
EXPLODE_JSON_ARRAY_STRING_OUTER(<json>)
```

7.2.5.6.3 参数

参数	说明
<json>	json 类型

7.2.5.6.4 返回值

展开 JSON 数组，每个元素生成一行，返回整数列。

7.2.5.6.5 举例

```
CREATE TABLE json_array_example (
    id INT,
    json_array STRING
)DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
INSERT INTO json_array_example (id, json_array) VALUES
(1, '[1, 2, 3, 4, 5]'),
(2, '[1.1, 2.2, 3.3, 4.4]'),
(3, '["apple", "banana", "cherry"]'),
(4, '[{"a": 1}, {"b": 2}, {"c": 3}]'),
(5, '[]'),
(6, 'NULL');
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_STRING(json_array) tmp1 AS e1
WHERE id = 3;
```

```
+------+--------+
| id   | e1     |
+------+--------+
|    3 | apple  |
|    3 | banana |
|    3 | cherry |
+------+--------+
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_STRING(json_array) tmp1 AS e1
WHERE id = 6;
Empty set (0.02 sec)
```

```
SELECT id, e1
FROM json_array_example
LATERAL VIEW EXPLODE_JSON_ARRAY_STRING_OUTER(json_array) tmp1 AS e1
WHERE id = 6;
```

```
+------+------+
| id   | e1   |
+------+------+
|    6 | NULL |
```

```
+------+------+
```

7.2.5.7 EXPLODE_JSON_OBJECT

7.2.5.7.1 描述

explode_json_object 将 JSON 对象展开为多行，每行包含一个键值对。通常用于处理 JSON 数据，将 JSON 对象展开为更易查询的格式。该函数只支持包含元素的 JSON 对象。

explode_json_object_outer 与 explode_json_object 类似，但在处理空值和 NULL 值时有不同的行为。它可以保留空的或 NULL 的 JSON 对象，返回相应的记录。

7.2.5.7.2 语法

```
EXPLODE_JSON_OBJECT(<json>)
EXPLODE_JSON_OBJECT_OUTER(<json>)
```

7.2.5.7.3 参数

参数	说明
<json>	json 类型

7.2.5.7.4 返回值

当 JSON 对象不为空或 NULL 时，explode_json_object 和 explode_json_object_outer 的返回值相同。每个键值对生成一行，键作为列之一，值作为另一个列。

当 JSON 对象为空或 NULL 时：

explode_json_object 不会返回任何行。

explode_json_object_outer 会返回一行，展开列的值为 NULL。

7.2.5.7.5 举例

```
CREATE TABLE example (
    id INT,
    value_json json
) DUPLICATE KEY(id)
DISTRIBUTED BY HASH(`id`) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
INSERT INTO example VALUES
(1, '{"key1": "value1", "key2": "value2"}'),
```

```
(2, '{}'),
(3, NULL);
```

```
select * from example;
```

```
+------+-----------------------------------+
| id   | value_json                        |
+------+-----------------------------------+
|    2 | {}                                |
|    1 | {"key1":"value1","key2":"value2"} |
|    3 | NULL                              |
+------+-----------------------------------+
```

```
SELECT id, k, v
FROM example
LATERAL VIEW explode_json_object(value_json) exploded_table AS k , v;
```

```
+------+------+----------+
| id   | k    | v        |
+------+------+----------+
|    1 | key1 | "value1" |
|    1 | key2 | "value2" |
+------+------+----------+
```

```
SELECT id, k, v
FROM example
LATERAL VIEW explode_json_object_outer(value_json) exploded_table AS k, v;
```

```
+------+------+----------+
| id   | k    | v        |
+------+------+----------+
|    3 | NULL | NULL     |
|    1 | key1 | "value1" |
|    1 | key2 | "value2" |
|    2 | NULL | NULL     |
+------+------+----------+
```

7.2.5.8 EXPLODE_MAP

7.2.5.8.1 描述

explode_mpa 函数接受一个 map (映射类型)，将 map (映射类型) 展开成多个行，每行包含一个键值对。通常与 LATERAL VIEW 配合使用，可以支持多个 Lateral view。仅支持新优化器。

explode_map 和 explode_map_outer 区别主要在于空值处理。

7.2.5.8.2 语法

```
EXPLODE_MAP(map<k,v>)
EXPLODE_MAP_OUTER(map<k,v>)
```

7.2.5.8.3 参数

参数	说明
map<k,v>	map 类型

7.2.5.8.4 返回值

当 map 不为空或 NULL 时，explode_map 和 explode_map_outer 的返回值相同。

当数据为空或 NULL 时：

explode_map 只处理包含元素的 map。如果 map 是空的或为 NULL，explode_map 不会返回任何行。

explode_map_outer 如果 map 是空的或为 NULL，会保留空 map 或 NULL 的记录，返回的行将包含 NULL 值。

7.2.5.8.5 举例

```
SET enable_nereids_planner=true
```

```
SET enable_fallback_to_original_planner=false
```

```
CREATE TABLE IF NOT EXISTS `sdu`(
                    `id` INT NULL,
                    `name` TEXT NULL,
                    `score` MAP<TEXT,INT> NULL
                ) ENGINE=OLAP
                DUPLICATE KEY(`id`)
                COMMENT 'OLAP'
                DISTRIBUTED BY HASH(`id`) BUCKETS 1
                PROPERTIES ("replication_allocation" = "tag.location.default: 1");
Query OK, 0 rows affected (0.15 sec)
```

```
insert into sdu values (0, "zhangsan", {"Chinese":"80","Math":"60","English":"90"}), (1, "lisi",
    ↪ {"null":null}), (2, "wangwu", {"Chinese":"88","Math":"90","English":"96"}), (3, "lisi2",
    ↪ {null:null}), (4, "amory", NULL);
Query OK, 5 rows affected (0.23 sec)
{'label':'label_9b35d9d9d59147f5_bffb974881ed2133', 'status':'VISIBLE', 'txnId':'4005'}
```

```
select * from sdu order by id;
```

```
+------+----------+----------------------------------------+
| id   | name     | score                                  |
+------+----------+----------------------------------------+
|    0 | zhangsan | {"Chinese":80, "Math":60, "English":90} |
|    1 | lisi     | {"null":null}                          |
|    2 | wangwu   | {"Chinese":88, "Math":90, "English":96} |
|    3 | lisi2    | {null:null}                            |
|    4 | amory    | NULL                                   |
+------+----------+----------------------------------------+
```

```
select name, k,v from sdu lateral view explode_map(score) tmp as k,v;
```

```
+----------+---------+------+
| name     | k       | v    |
+----------+---------+------+
| zhangsan | Chinese |   80 |
| zhangsan | Math    |   60 |
| zhangsan | English |   90 |
| lisi     | null    | NULL |
| wangwu   | Chinese |   88 |
| wangwu   | Math    |   90 |
| wangwu   | English |   96 |
| lisi2    | NULL    | NULL |
+----------+---------+------+
```

```
select name, k,v from sdu lateral view explode_map_outer(score) tmp as k,v;
```

```
+----------+---------+------+
| name     | k       | v    |
+----------+---------+------+
| zhangsan | Chinese |   80 |
| zhangsan | Math    |   60 |
| zhangsan | English |   90 |
| lisi     | null    | NULL |
| wangwu   | Chinese |   88 |
| wangwu   | Math    |   90 |
| wangwu   | English |   96 |
| lisi2    | NULL    | NULL |
| amory    | NULL    | NULL |
+----------+---------+------+
```

```
select name, k,v,k1,v1 from sdu lateral view explode_map_outer(score) tmp as k,v lateral view
    ↪ explode_map(score) tmp2 as k1,v1;
```

```
+----------+---------+------+---------+------+
| name     | k       | v    | k1      | v1   |
+----------+---------+------+---------+------+
| zhangsan | Chinese |   80 | Chinese |   80 |
| zhangsan | Chinese |   80 | Math    |   60 |
| zhangsan | Chinese |   80 | English |   90 |
| zhangsan | Math    |   60 | Chinese |   80 |
| zhangsan | Math    |   60 | Math    |   60 |
| zhangsan | Math    |   60 | English |   90 |
| zhangsan | English |   90 | Chinese |   80 |
| zhangsan | English |   90 | Math    |   60 |
| zhangsan | English |   90 | English |   90 |
| lisi     | null    | NULL | null    | NULL |
| wangwu   | Chinese |   88 | Chinese |   88 |
| wangwu   | Chinese |   88 | Math    |   90 |
| wangwu   | Chinese |   88 | English |   96 |
| wangwu   | Math    |   90 | Chinese |   88 |
| wangwu   | Math    |   90 | Math    |   90 |
| wangwu   | Math    |   90 | English |   96 |
| wangwu   | English |   96 | Chinese |   88 |
| wangwu   | English |   96 | Math    |   90 |
| wangwu   | English |   96 | English |   96 |
| lisi2    | NULL    | NULL | NULL    | NULL |
+----------+---------+------+---------+------+
```

7.2.5.9 EXPLODE_NUMBERS

7.2.5.9.1 描述

explode_numbers 表函数，接受一个整数 n，将范围的所有数字展开为多行，每行一个数字。常用于生成连续数字的序列，配合 LATERAL VIEW 使用。

explode_numbers_outer 与 explode_numbers 不同的是，会在表函数生成 0 行数据时添加一行Null数据。

7.2.5.9.2 语法

```
EXPLODE_NUMBERS(<n>)
EXPLODE_NUMBERS_OUTER(<n>)
```

7.2.5.9.3 参数

参数	说明
<n>	整数类型

7.2.5.9.4 返回值

返回一个 [0,n) 的序列

- 当为 0 或者 NULL 时不返回

7.2.5.9.5 举例

```
select e1 from (select 1 k1) as t lateral view explode_numbers(5) tmp1 as e1;
```

```
+------+
| e1   |
+------+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
+------+
```

```
select e1 from (select 1 k1) as t lateral view explode_numbers(0) tmp1 as e1;
Empty set
```

```
select e1 from (select 1 k1) as t lateral view explode_numbers_outer(0) tmp1 as e1;
```

```
+------+
| e1   |
+------+
| NULL |
+------+
```

7.2.5.10 EXPLODE_SPLIT

7.2.5.10.1 描述

explode_split 表函数用于将字符串按照指定分隔符拆分为多个子字符串，并将每个子字符串展开为一行。每个子字符串作为单独的行返回，通常与 LATERAL VIEW 一起使用，便于将长字符串拆解为单独的部分，进行更细粒度的查询。

explode_split_outer 与 explode_split 类似。但与 explode_split 不同的是，它在处理空值和 NULL 值时会有不同的行为，能够处理空的或 NULL 的字符串。

7.2.5.10.2 语法

```
EXPLODE_SPLIT(<str>, <delimiter>)
EXPLODE_SPLIT_OUTER(<str>, <delimiter>)
```

7.2.5.10.3 参数

参数	说明
<str>	字符串类型
<delimiter>	分割符

7.2.5.10.4 返回值

返回拆分后的子字符串序列。如果字符串为空或 NULL，不返回任何行。

7.2.5.10.5 举例

```
select * from example1 order by k1;
```

```
+------+---------+
| k1   | k2      |
+------+---------+
|    1 |         |
|    2 | NULL    |
|    3 | ,       |
|    4 | 1       |
|    5 | 1,2,3   |
|    6 | a, b, c |
+------+---------+
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 1 order by
    ↪ k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    1 |      |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 2 order by
    ↪ k1, e1;
Empty set
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 3 order by
    ↪ k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
```

```
|    3 |      |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 4 order by
    ↪ k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    4 | 1    |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 5 order by
    ↪ k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    5 | 2    |
|    5 | 3    |
|    5 | 1    |
+------+------+
```

```
select k1, e1 from example1 lateral view explode_split(k2, ',') tmp1 as e1 where k1 = 6 order by
    ↪ k1, e1;
```

```
+------+------+
| k1   | e1   |
+------+------+
|    6 |  b   |
|    6 |  c   |
|    6 |  a   |
+------+------+
```

```
CREATE TABLE example2 (
    id INT,
    str string null
)DUPLICATE KEY(id)
DISTRIBUTED BY HASH(`id`) BUCKETS AUTO
PROPERTIES (
"replication_allocation" = "tag.location.default: 1");
```

```
insert into example2 values (1,''),(2,NUll),(3,"1"),(4,"1,2,3"),(5,"a,b,c");
```

```
select id, e1 from example2 lateral view explode_split(str, ',') tmp1 as e1 where id = 2 order by
    ↪  id, e1;
Empty set (0.02 sec)
```

```
select id, e1 from example2 lateral view explode_split_outer(str, ',') tmp1 as e1 where id = 2
    ↪ order by id, e1;
```

```
+------+------+
| id   | e1   |
+------+------+
|    2 | NULL |
+------+------+
```

7.2.6 表值函数

7.2.6.1 BACKENDS

7.2.6.1.1 描述

表函数，生成 backends 临时表，可以查看当前 doris 集群中的 BE 节点信息。

7.2.6.1.2 语法

```
BACKENDS()
```

7.2.6.1.3 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	全局	

7.2.6.1.4 返回值

Field	Description
BackendId	每个 Backend 节点的唯一标识符。
Host	Backend 节点的 IP 地址或主机名。
HeartbeatPort	用于健康检查（心跳）的端口。
BePort	Backend 节点与集群通信时使用的端口。
HttpPort	Backend 节点的 HTTP 端口。
BrpcPort	用于 BRPC 通信的端口。
ArrowFlightSqlPort	Arrow Flight SQL 端口（用于与 Apache Arrow 集成，进行高性能数据传输）。
LastStartTime	Backend 节点最后一次启动的时间戳。

Field	Description
LastHeartbeat	接收到的最后一次心跳时间戳。
Alive	Backend 节点是否处于活动状态（True/False）。
SystemDecommissioned	该 Backend 节点是否已被弃用。
TabletNum	该 Backend 节点管理的 Tablet 数量。
DataUsedCapacity	该 Backend 节点使用的磁盘空间（以 MB 为单位）。
TrashUsedCapacity	该 Backend 节点垃圾空间的使用情况（以 MB 为单位）。
AvailCapacity	该 Backend 节点的可用磁盘空间。
TotalCapacity	该 Backend 节点的总磁盘容量。
UsedPct	该 Backend 节点的磁盘使用百分比。
MaxDiskUsedPct	所有 Tablet 的最大磁盘使用百分比。
RemoteUsedCapacity	远程存储的磁盘空间使用情况（如果适用）。
Tag	与 Backend 节点关联的标签，通常用于节点分类（例如，位置等）。
ErrMsg	Backend 节点的错误信息。
Version	Backend 节点的版本。
Status	Backend 节点的当前状态，包括 Tablet 的成功/失败报告、加载时间和查询状态等。
HeartbeatFailureCounter	心跳失败的计数，如果有的话。
NodeRole	Backend 节点的角色，例如 `mix` 表示节点同时处理存储和查询。
CpuCores	Backend 节点的 CPU 核心数。
Memory	Backend 节点的内存大小。

7.2.6.1.5 示例

查看 backends 集群信息

```
select * from backends();
```

```
+-----------+-----------+---------------+--------+----------+----------+--------------------+--------------------
    ↪
| BackendId | Host      | HeartbeatPort | BePort | HttpPort | BrpcPort | ArrowFlightSqlPort |
    ↪ LastStartTime       | LastHeartbeat       | Alive | SystemDecommissioned | TabletNum |
    ↪ DataUsedCapacity | TrashUsedCapacity | AvailCapacity | TotalCapacity | UsedPct |
    ↪ MaxDiskUsedPct | RemoteUsedCapacity | Tag                      | ErrMsg | Version
    ↪                 | Status
    ↪
    ↪ | HeartbeatFailureCounter | NodeRole | CpuCores | Memory    |
+-----------+-----------+---------------+--------+----------+----------+--------------------+--------------------
    ↪
| 10020     | 10.xx.xx.90 | 9050          | 9060   | 8040     | 8060     | -1                 |
    ↪ 2025-01-13 14:11:31 | 2025-01-16 13:24:55 | true  | false                | 359       |
    ↪ 295.328 MB       | 0.000             | 231.236 GB    | 3.437 TB      | 93.43 % | 93.43 %
    ↪        | 0.000              | {"location" : "default"} |        | doris-0.0.0--83f899b32b
    ↪  | {"lastSuccessReportTabletsTime":"2025-01-16 13:24:07","lastStreamLoadTime
    ↪ ":1737004982210,"isQueryDisabled":false,"isLoadDisabled":false,"isActive":true,"
    ↪ currentFragmentNum":0,"lastFragmentUpdateTime":1737004982195} | 0                    |
```

```
    ↪  mix       | 96        | 375.81 GB |
+-----------+-----------+---------------+--------+----------+----------+--------------------+-----------------
    ↪
```

7.2.6.2 CATALOGS

7.2.6.2.1 描述

CATALOGS() 函数生成一个临时的 catalogs 表，允许查看当前 Doris 中所有已创建的 catalogs 信息，其结果综合了 `show catalogs` 和 `show catalog xxx` 的信息。

该函数用于 FROM 子句中，便于查询和分析 Doris 中的 catalog 数据。

7.2.6.2.2 语法

```
CATALOGS()
```

7.2.6.2.3 返回值

查看 catalog() 函数的返回字段

```
desc function catalogs();
```

```
+-------------+--------+------+-------+---------+-------+
| Field       | Type   | Null | Key   | Default | Extra |
+-------------+--------+------+-------+---------+-------+
| CatalogId   | BIGINT | No   | false | NULL    | NONE  |
| CatalogName | TEXT   | No   | false | NULL    | NONE  |
| CatalogType | TEXT   | No   | false | NULL    | NONE  |
| Property    | TEXT   | No   | false | NULL    | NONE  |
| Value       | TEXT   | No   | false | NULL    | NONE  |
+-------------+--------+------+-------+---------+-------+
```

字段含义如下：

字段名称	类型	说明
CatalogId	BIGINT	Catalog 的唯一标识符，用于区分不同的 catalog 实例。
CatalogName	TEXT	Catalog 的名称，用于标识 Doris 中的 catalog。
CatalogType	TEXT	Catalog 的类型，例如 hms（Hive Metastore）、es（Elasticsearch）。
Property	TEXT	与 catalog 相关的属性名称或配置项。
Value	TEXT	属性的值，描述 catalog 配置的具体内容。

7.2.6.2.4 示例

查看 doris 集群所有 catalog 信息

```
select * from catalogs()
```

```
+-----------+-------------+-------------+-------------------------------------------+-----------------------------
    ↪
| CatalogId | CatalogName | CatalogType | Property                                  | Value
    ↪                                                                    |
+-----------+-------------+-------------+-------------------------------------------+-----------------------------
    ↪
|     16725 | hive        | hms         | dfs.client.failover.proxy.provider.HANN   | org.apache
    ↪ .hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider |
|     16725 | hive        | hms         | dfs.ha.namenodes.HANN                     | nn1,nn2
    ↪                                                                     |
|     16725 | hive        | hms         | create_time                               | 2023-07-13
    ↪  16:24:38.968                                                  |
|     16725 | hive        | hms         | ipc.client.fallback-to-simple-auth-allowed | true
    ↪                                                                       |
|     16725 | hive        | hms         | dfs.namenode.rpc-address.HANN.nn1         | nn1_host:
    ↪ rpc_port                                                           |
|     16725 | hive        | hms         | hive.metastore.uris                       | thrift
    ↪ ://127.0.0.1:7004                                                    |
|     16725 | hive        | hms         | dfs.namenode.rpc-address.HANN.nn2         | nn2_host:
    ↪ rpc_port                                                           |
|     16725 | hive        | hms         | type                                      | hms
    ↪                                                                        |
|     16725 | hive        | hms         | dfs.nameservices                          | HANN
    ↪                                                                       |
|         0 | internal    | internal    | NULL                                      | NULL
    ↪                                                                       |
|     16726 | es          | es          | create_time                               | 2023-07-13
    ↪  16:24:44.922                                                  |
|     16726 | es          | es          | type                                      | es
    ↪                                                                         |
|     16726 | es          | es          | hosts                                     | http
    ↪ ://127.0.0.1:9200                                                     |
+-----------+-------------+-------------+-------------------------------------------+-----------------------------
    ↪
```

7.2.6.3 FRONTENDS

7.2.6.3.1 描述

表函数，生成 frontends 临时表，可以查看当前 doris 集群中的 FE 节点信息。

7.2.6.3.2 语法

```
FRONTENDS()
```

7.2.6.3.3 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	全局	

7.2.6.3.4 返回值

Field	Description
Name	Frontend 节点的唯一名称。
Host	Frontend 节点的 IP 地址或主机名。
EditLogPort	用于编辑日志通信的端口。
HttpPort	Frontend 节点的 HTTP 端口。
QueryPort	Frontend 节点用于执行查询的端口。
RpcPort	用于 RPC 通信的端口。
ArrowFlightSqlPort	Arrow Flight SQL 端口（用于与 Apache Arrow 集成，进行高性能数据传输）。
Role	Frontend 节点的角色（例如：FOLLOWER）。
IsMaster	表示该节点是否是主节点（True/False）。
ClusterId	该 Frontend 节点所属集群的标识符。
Join	表示该 Frontend 节点是否已经加入集群（True/False）。
Alive	表示该 Frontend 节点是否存活（True/False）。
ReplayedJournalId	该 Frontend 节点最后重放的日志 ID。
LastStartTime	该 Frontend 节点最后一次启动的时间戳。
LastHeartbeat	该 Frontend 节点接收到的最后一次心跳时间戳。
IsHelper	表示该 Frontend 节点是否是辅助节点（True/False）。
ErrMsg	该 Frontend 节点的错误信息。
Version	该 Frontend 节点的版本。
CurrentConnected	表示该 Frontend 节点当前是否连接到集群（Yes/No）。

7.2.6.3.5 示例

查看 frontends 集群信息

```
select * from frontends();
```

```
+-----------------------------------------+-----------+-------------+----------+-----------+---------+----------
    ↪
| Name                                    | Host      | EditLogPort | HttpPort | QueryPort |
    ↪ RpcPort | ArrowFlightSqlPort | Role     | IsMaster | ClusterId | Join | Alive |
    ↪ ReplayedJournalId | LastStartTime       | LastHeartbeat       | IsHelper | ErrMsg |
    ↪ Version                 | CurrentConnected |
```

```
+----------------------------------------+------------+-------------+----------+-----------+---------+----------
    ↪
| fe_f4642d47_62a2_44a2_b79d_3259050ab9de | 10.xx.xx.90 | 9010        | 8030     | 9030      |
    ↪ 9020    | -1               | FOLLOWER | true     | 917153130 | true | true  | 555248
    ↪            | 2025-01-13 14:11:31 | 2025-01-16 14:27:56 | true     |        | doris
    ↪ -0.0.0--83f899b32b | Yes              |
+----------------------------------------+------------+-------------+----------+-----------+---------+----------
    ↪
```

7.2.6.4 HDFS

7.2.6.4.1 描述

HDFS 表函数（table-valued-function,tvf），可以让用户像访问关系表格式数据一样，读取并访问 HDFS 上的文件内容。目前支持csv/csv_with_names/csv_with_names_and_types/json/parquet/orc文件格式。

7.2.6.4.2 语法

```
HDFS(
    "uri" = "<uri>",
    "fs.defaultFS" = "<fs_defaultFS>",
    "hadoop.username" = "<hadoop_username>",
    "format" = "<format>",
    [, "<optional_property_key>" = "<optional_property_value>" [, ...] ]
  );
```

7.2.6.4.3 必填参数 (Required Parameters)

参数	说明
uri	访问 HDFS 的 URI。如果 URI 路径不存在或文件为空，HDFS TVF 将返回空集合。
fs.defaultFS	HDFS 的默认文件系统 URI
hadoop.username	必填，可以是任意字符串，但不能为空。
format	文件格式，必填，目前支持 csv/csv_with_names/csv_with_names_and_types/json/parquet/orc/avro

7.2.6.4.4 可选参数 (Optional Parameters)

上述语法中的 optional_property_key 可以按需从以下列表中选取对应的参数，optional_property_value 则为该参数的值

参数	说明	备注
hadoop. ↪ security ↪ . ↪ authentication ↪	HDFS 安全认证类型	
hadoop. ↪ username ↪	备用 HDFS 用户名	
hadoop. ↪ kerberos ↪ . ↪ principal ↪	Kerberos 主体	
hadoop. ↪ kerberos ↪ . ↪ keytab	Kerberos 密钥表	
dfs. ↪ client ↪ .read. ↪ shortcircuit ↪	启用短路读取	
dfs. ↪ domain ↪ . ↪ socket ↪ .path	域套接字路径	
dfs. ↪ nameservices ↪	HA 模式下的命名服务	
dfs.ha. ↪ namenodes ↪ .your- ↪ nameservices ↪	HA 模式下的 namenode 节点配置	

参数	说明	备注
dfs. ↪ namenode ↪ .rpc- ↪ address ↪ .your- ↪ nameservices ↪ .your- ↪ namenode ↪	指定 namenode 的 RPC 地址	
dfs. ↪ client ↪ . ↪ failover ↪ .proxy ↪ . ↪ provider ↪ .your- ↪ nameservices ↪	指定 failover 的代理提供程序	
column_ ↪ separator ↪	列分割符，默认为 \t	
line_ ↪ delimiter ↪	行分割符，默认为 \n	
compress_ ↪ type	目前支持 UNKNOWN/PLAIN/GZ/LZO/BZ2/LZ4FRAME/DEFLATE/SNAPPYBLOCK。默认值为 UNKNOWN, 将会根据 uri 的后缀自动推断类型	
read_json ↪ _by_ ↪ line	对 JSON 格式导入，默认为 true	参考：JSON Load
strip_ ↪ outer_ ↪ array	对 JSON 格式导入，默认为 false	参考：JSON Load
json_root	对 JSON 格式导入，默认为空	参考：JSON Load
json_ ↪ paths	对 JSON 格式导入，默认为空	参考：JSON Load
num_as_ ↪ string	对 JSON 格式导入，默认为 false	参考：JSON Load
fuzzy_ ↪ parse	对 JSON 格式导入，默认为 false	参考：JSON Load

参数	说明	备注
trim_double_quotes	对 CSV 格式导入，布尔类型，默认为 false，为 true 时裁剪每个字段的外层双引号	
skip_lines	对 CSV 格式导入，整数类型，默认为 0，跳过 CSV 文件前几行，csv_with_names 或 csv_with_names_and_types 时失效	
path_partition_keys	指定文件路径中携带的分区列名，例如/path/to/city=beijing/date=“2023-07-09”, 则填写path_partition_keys=“city,date”，将会自动从路径中读取相应列名和列值进行导入	
resource	指定 Resource 名，HDFS TVF 可以利用已有的 HFDS Resource 来直接访问 HDFS。创建 HDFS Resource 的方法可以参照CREATE-RESOURCE	仅支持 2.1.4 及以上版本

7.2.6.4.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
USAGE_PRIV	表	
SELECT_PRIV	表	

7.2.6.4.6 示例

- 读取并访问 HDFS 存储上的 CSV 格式文件

```
  select * from hdfs(
            "uri" = "hdfs://127.0.0.1:842/user/doris/csv_format_test/student.csv",
            "fs.defaultFS" = "hdfs://127.0.0.1:8424",
            "hadoop.username" = "doris",
            "format" = "csv");
```

```
+------+---------+------+
| c1   | c2      | c3   |
+------+---------+------+
| 1    | alice   | 18   |
| 2    | bob     | 20   |
| 3    | jack    | 24   |
| 4    | jackson | 19   |
| 5    | liming  | 18   |
+------+---------+------+
```

- 读取并访问 HA 模式的 HDFS 存储上的 CSV 格式文件

```
select * from hdfs(
        "uri" = "hdfs://127.0.0.1:842/user/doris/csv_format_test/student.csv",
        "fs.defaultFS" = "hdfs://127.0.0.1:8424",
        "hadoop.username" = "doris",
        "format" = "csv",
        "dfs.nameservices" = "my_hdfs",
        "dfs.ha.namenodes.my_hdfs" = "nn1,nn2",
        "dfs.namenode.rpc-address.my_hdfs.nn1" = "nanmenode01:8020",
        "dfs.namenode.rpc-address.my_hdfs.nn2" = "nanmenode02:8020",
        "dfs.client.failover.proxy.provider.my_hdfs" = "org.apache.hadoop.hdfs.server.
            ↪ namenode.ha.ConfiguredFailoverProxyProvider");
```

```
+------+---------+------+
| c1   | c2      | c3   |
+------+---------+------+
| 1    | alice   | 18   |
| 2    | bob     | 20   |
| 3    | jack    | 24   |
| 4    | jackson | 19   |
| 5    | liming  | 18   |
+------+---------+------+
```

- 可以配合 desc function 使用。

```
desc function hdfs(
            "uri" = "hdfs://127.0.0.1:8424/user/doris/csv_format_test/student_with_names.csv",
            "fs.defaultFS" = "hdfs://127.0.0.1:8424",
            "hadoop.username" = "doris",
            "format" = "csv_with_names");
```

7.2.6.5 ICEBERG_META

7.2.6.5.1 描述

iceberg_meta 表函数（table-valued-function,tvf），可以用于读取 iceberg 表的各类元数据信息，如操作历史、生成的快照、文件元数据等。

7.2.6.5.2 语法

```
ICEBERG_META(
    "table" = "<table>",
    "query_type" = "<query_type>"
  );
```

7.2.6.5.3 必填参数

iceberg_meta 表函数 tvf 中的每一个参数都是一个 "key"="value" 对

Field	Description
<table>	完整的表名，需要按照目录名。库名. 表名的格式，填写需要查看的 iceberg 表名。
<query_type>	元数据类型，目前仅支持 snapshots。

7.2.6.5.4 示例（Examples）

- 读取并访问 iceberg 表格式的 snapshots 元数据。

```
select * from iceberg_meta("table" = "ctl.db.tbl", "query_type" = "snapshots");
```

- 可以配合desc function使用

```
desc function iceberg_meta("table" = "ctl.db.tbl", "query_type" = "snapshots");
```

- 查看 iceberg 表的 snapshots

```
select * from iceberg_meta("table" = "iceberg_ctl.test_db.test_tbl", "query_type" = "
    ↪ snapshots");
```

```
+-----------------------+----------------+--------------+-----------+-------------------+-----------------
    ↪
|      committed_at     |  snapshot_id   |   parent_id   | operation |   manifest_list   |
    ↪            summary           |
+-----------------------+----------------+--------------+-----------+-------------------+-----------------
    ↪
|  2022-09-20 11:14:29  |  64123452344   |       -1      |  append   | hdfs:/path/to/m1  |
    ↪ {"flink.job-id":"xxm1", ...} |
|  2022-09-21 10:36:35  |  98865735822   |  64123452344  | overwrite | hdfs:/path/to/m2  |
    ↪ {"flink.job-id":"xxm2", ...} |
|  2022-09-21 21:44:11  |  51232845315   |  98865735822  | overwrite | hdfs:/path/to/m3  |
    ↪ {"flink.job-id":"xxm3", ...} |
+-----------------------+----------------+--------------+-----------+-------------------+-----------------
    ↪
```

- 根据 snapshot_id 字段筛选

```
select * from iceberg_meta("table" = "iceberg_ctl.test_db.test_tbl", "query_type" = "
    ↪ snapshots") where snapshot_id = 98865735822;
```

```
+-----------------------+---------------+--------------+-----------+------------------+-----------------
    ↪
|      committed_at      |  snapshot_id   |   parent_id   | operation |   manifest_list   |
    ↪            summary           |
+-----------------------+---------------+--------------+-----------+------------------+-----------------
    ↪
|  2022-09-21 10:36:35   |  98865735822   |  64123452344  | overwrite | hdfs:/path/to/m2  |
    ↪ {"flink.job-id":"xxm2", ...} |
+-----------------------+---------------+--------------+-----------+------------------+-----------------
    ↪
```

7.2.6.6 JOBS

7.2.6.6.1 描述

表函数，生成任务临时表，可以查看某个任务类型中的 job 信息。

7.2.6.6.2 语法

```
JOBS(
    "type"="<type>"
)
```

7.2.6.6.3 必填参数 (Required Parameters)

字段名	描述
<type>	任务的类型：`insert`：insert into 类型的任务。`mv`：物化视图类型的任务。

7.2.6.6.4 返回值

- `jobs("type"="insert")` insert 类型的 job 返回值

| 字段名 | 描述 | |————————|————————————| | Id | job id | | Name | job 名称 | | Definer | job 定义者 | | ExecuteType | 执行类型 | | RecurringStrategy | 循环策略 | | Status | job 状态 | | ExecuteSql | 执行 SQL | | CreateTime | job 创建时间 | | SucceedTaskCount | 成功任务数量 | | FailedTaskCount | 失败任务数量 | | CanceledTaskCount | 取消任务数量 | | Comment | job 注释 |

- `jobs("type"="mv")` MV 类型的 job 返回值

| 字段名 | 描述 | |————————|————————————| | Id | job id | | Name | job 名称 | | MvId | 物化视图 id | | MvName | 物化视图名称 | | MvDatabaseId | 物化视图所属 db id | | MvDatabaseName | 物化视图所属 db 名称 | | ExecuteType | 执行类型 | | RecurringStrategy | 循环策略 | | Status | job 状态 | | CreateTime | task 创建时间 |

7.2.6.6.5 示例

查看所有物化视图的 job

```
select * from jobs("type"="mv");
```

```
+-------+------------------+-------+--------------------------+-------------+-------------------------------------
    ↪
| Id    | Name             | MvId  | MvName                   | MvDatabaseId | MvDatabaseName
    ↪                                          | ExecuteType | RecurringStrategy | Status  |
    ↪ CreateTime          |
+-------+------------------+-------+--------------------------+-------------+-------------------------------------
    ↪
| 23369 | inner_mtmv_23363 | 23363 | range_date_up_union_mv1  | 21805        | regression_test_
    ↪ nereids_rules_p0_mv_create_part_and_up | MANUAL      | MANUAL TRIGGER    | RUNNING |
    ↪ 2025-01-08 18:19:10 |
| 23377 | inner_mtmv_23371 | 23371 | range_date_up_union_mv2  | 21805        | regression_test_
    ↪ nereids_rules_p0_mv_create_part_and_up | MANUAL      | MANUAL TRIGGER    | RUNNING |
    ↪ 2025-01-08 18:19:10 |
| 21794 | inner_mtmv_21788 | 21788 | test_tablet_type_mtmv_mv | 16016        | zd
    ↪                                                    | MANUAL      | MANUAL TRIGGER    |
    ↪ RUNNING | 2025-01-08 12:26:06 |
| 19508 | inner_mtmv_19494 | 19494 | mv1                      | 16016        | zd
    ↪                                                    | MANUAL      | MANUAL TRIGGER    |
    ↪ RUNNING | 2025-01-07 22:13:31 |
+-------+------------------+-------+--------------------------+-------------+-------------------------------------
    ↪
```

查看所有 insert 任务的 job

```
select * from jobs("type"="insert");
```

```
+---------------+---------------+---------+-------------+-----------------------------------------+---------
    ↪
| Id            | Name          | Definer | ExecuteType | RecurringStrategy
    ↪                           | Status  | ExecuteSql
    ↪                                                   | CreateTime          |
    ↪ SucceedTaskCount | FailedTaskCount | CanceledTaskCount | Comment |
+---------------+---------------+---------+-------------+-----------------------------------------+---------
    ↪
| 78533940810334 | insert_tab_job | root    | RECURRING   | EVERY 10 MINUTE STARTS 2025-01-17
    ↪ 14:42:53 | RUNNING | INSERT INTO test.insert_tab SELECT * FROM test.example_table |
    ↪ 2025-01-17 14:32:53 | 0               | 0               | 0                 |         |
+---------------+---------------+---------+-------------+-----------------------------------------+---------
    ↪
```

7.2.6.7 LOCAL

7.2.6.7.1 描述

Local 表函数（table-valued-function,tvf），可以让用户像访问关系表格式数据一样，读取并访问 be 上的文件内容。目前支持csv/csv_with_names/csv_with_names_and_types/json/parquet/orc文件格式。

7.2.6.7.2 语法

```
LOCAL(
  "file_path" = "<file_path>",
  "backend_id" = "<backend_id>",
  "format" = "<format>"
  [, "<optional_property_key>" = "<optional_property_value>" [, ...] ]
  );
```

7.2.6.7.3 Required Parameters

参数	说明	备注
file_path	待读取文件的路径，该路径是一个相对于 user_files_secure_path 目录的相对路径，其中 user_files_secure_path 参数是be 的一个配置项。路径中不能包含 ..，可以使用 glob 语法进行模糊匹配，如：logs/*.log	
backend_id	文件所在的 BE 节点的 ID，可以通过 show backends 命令得到	在 2.1.1 之前的版本中，Doris 仅支持指定某个 BE 节点读取该节点上的本地数据文件
format	文件格式，必填，当前支持 csv/csv_with_names/csv_with_names_and_types/json/parquet/orc	

7.2.6.7.4 Optional Parameters

参数	说明	备注
shared_storage	默认为 false。如果为 true，表示指定的文件存在于共享存储上（比如 NAS）。共享存储必须兼容 POXIS 文件接口，并且同时挂载在所有 BE 节点上。当 shared_storage 为 true 时，可以不设置 backend_id，Doris 可能会利用到所有 BE 节点进行数据访问。如果设置了 backend_id，则仍然仅在指定 BE 节点上执行。	从 2.1.2 版本开始支持
column_separator	列分隔符，选填，默认为 \t	

参数	说明	备注
line_delimiter	行分隔符，选填，默认为 \n	
compress_type	压缩类型，选填，目前支持 UNKNOWN/PLAIN/GZ/LZO/BZ2/LZ4FRAME/DEFLATE/SNAPPYBLOCK，默认值为 UNKNOWN，将根据 uri 后缀自动推断类型	
read_json_by_line	对于 JSON 格式的导入，选填，默认为 true	参照：Json Load
strip_outer_array	对于 JSON 格式的导入，选填，默认为 false	参照：Json Load
json_root	对于 JSON 格式的导入，选填，默认为空	参照：Json Load
json_paths	对于 JSON 格式的导入，选填，默认为空	参照：Json Load
num_as_string	对于 JSON 格式的导入，选填，默认为 false	参照：Json Load
fuzzy_parse	对于 JSON 格式的导入，选填，默认为 false	参照：Json Load
trim_double_quotes	对于 CSV 格式的导入，选填，默认为 false，为 true 时表示裁剪掉 CSV 文件每个字段最外层的双引号	csv 格式
skip_lines	对于 CSV 格式的导入，选填，默认为 0，表示跳过 CSV 文件的前几行。当设置格式为 csv_with_names 或 csv_with_names_and_types 时，该参数会失效	csv 格式
path_partition_keys	选填，指定文件路径中携带的分区列名，例如 /path/to/city=beijing/date="2023-07-09"，则填写 path_partition_keys="city,date"，将从路径中自动读取相应的列名和列值进行导入	

7.2.6.7.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	全局	

7.2.6.7.6 注意事项

- 关于 local tvf 的更详细使用方法可以参照 S3 tvf, 唯一不同的是访问存储系统的方式不一样。
- 通过 local tvf 访问 NAS 上的数据

NAS 共享存储允许同时挂载到多个节点。每个节点都可以像访问本地文件一样访问共享存储中的文件。因此，可以将 NAS 视为本地文件系统，通过 local tvf 进行访问。

当设置 `"shared_storage" = "true"` 时，Doris 会认为所指定的文件可以在任意 BE 节点访问。当使用通配符指定了一组文件时，Doris 会将访问文件的请求分发到多个 BE 节点上，这样可以利用多个节点的进行分布式文件扫描，提升查询性能。

7.2.6.7.7 示例

分析指定 BE 上的日志文件：

```
select * from local(
        "file_path" = "log/be.out",
        "backend_id" = "10006",
        "format" = "csv")
       where c1 like "%start_time%" limit 10;
```

```
+-----------------------------------------------------+
| c1                                                  |
+-----------------------------------------------------+
| start time: 2023 年 08 月 07 日 星期一 23:20:32 CST       |
| start time: 2023 年 08 月 07 日 星期一 23:32:10 CST       |
| start time: 2023 年 08 月 08 日 星期二 00:20:50 CST       |
| start time: 2023 年 08 月 08 日 星期二 00:29:15 CST       |
+-----------------------------------------------------+
```

读取和访问位于路径`${DORIS_HOME}/student.csv`的 csv 格式文件：

```
select * from local(
      "file_path" = "student.csv",
      "backend_id" = "10003",
      "format" = "csv");
```

```
+------+---------+--------+
| c1   | c2      | c3     |
+------+---------+--------+
| 1    | alice   | 18     |
| 2    | bob     | 20     |
| 3    | jack    | 24     |
| 4    | jackson | 19     |
| 5    | liming  | d18    |
+------+---------+--------+
```

访问 NAS 上的共享数据：

```
select * from local(
        "file_path" = "/mnt/doris/prefix_*.txt",
        "format" = "csv",
        "column_separator" =",",
        "shared_storage" = "true");
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
| 1    | 2    | 3    |
| 1    | 2    | 3    |
| 1    | 2    | 3    |
| 1    | 2    | 3    |
| 1    | 2    | 3    |
+------+------+------+
```

可以配合desc function使用

```
desc function local(
      "file_path" = "student.csv",
      "backend_id" = "10003",
      "format" = "csv");
```

```
+-------+------+------+-------+---------+-------+
| Field | Type | Null | Key   | Default | Extra |
+-------+------+------+-------+---------+-------+
| c1    | TEXT | Yes  | false | NULL    | NONE  |
| c2    | TEXT | Yes  | false | NULL    | NONE  |
| c3    | TEXT | Yes  | false | NULL    | NONE  |
+-------+------+------+-------+---------+-------+
```

7.2.6.8 NUMBERS

7.2.6.8.1 描述

表函数，生成一张只含有一列的临时表，列名为number，如果指定了const_value，则所有元素值均为const_value，否则为 [0,number) 递增。

7.2.6.8.2 语法

```
NUMBERS(
    "number" = "<number>"
    [, "<const_value>" = "<const_value>" ]
  );
```

7.2.6.8.3 必填参数

字段	描述
number	行数

7.2.6.8.4 选填参数

字段	描述
const_value	常量值

7.2.6.8.5 返回值

字段名	类型	描述
number	BIGINT	指定每行返回的值

7.2.6.8.6 举例

```
select * from numbers("number" = "5");
```

```
+--------+
| number |
+--------+
|      0 |
|      1 |
|      2 |
|      3 |
|      4 |
+--------+
```

```
select * from numbers("number" = "5", "const_value" = "-123");
```

```
+--------+
| number |
+--------+
|   -123 |
|   -123 |
|   -123 |
|   -123 |
|   -123 |
+--------+
```

7.2.6.9 PARTITION_VALUES

7.2.6.9.1 描述

表函数，生成分区值临时表，可以查看某个 TABLE 的分区值列表。

该函数用于 FROM 子句中，仅支持 hive 表

7.2.6.9.2 语法

```
PARTITION_VALUES(
    "catalog"="<catalog>",
    "database"="<database>",
    "table"="<table>"
)
```

7.2.6.9.3 必填参数 (Required Parameters)

字段	描述
<catalog>	指定需要查询的集群 catalog 名。
<database>	指定需要查询的集群数据库名。
<table>	指定需要查询的集群表名。

7.2.6.9.4 返回值

要查的表有几个分区字段，该表就有几列

7.2.6.9.5 示例

hive3 CATALOG 下 multi_catalog 的 text_partitioned_columns 的建表语句如下：

```
CREATE TABLE `text_partitioned_columns`(
  `t_timestamp` timestamp)
PARTITIONED BY (
 `t_int` int,
 `t_float` float,
 `t_string` string)
```

数据如下：

```
mysql> select * from text_partitioned_columns;
+---------------------------+-------+---------+----------+
| t_timestamp               | t_int | t_float | t_string |
+---------------------------+-------+---------+----------+
| 2023-01-01 00:00:00.000000 |  NULL |     0.1 | test1    |
| 2023-01-02 00:00:00.000000 |  NULL |     0.2 | test2    |
```

```
| 2023-01-03 00:00:00.000000 |   100 |     0.3 | test3    |
+----------------------------+-------+---------+----------+
```

查看 hive3 CATALOG 下 multi_catalog 的 text_partitioned_columns 的分区值列表

```
select * from partition_values("catalog"="hive3", "database" = "multi_catalog","table" = "text_
    ↪ partitioned_columns");
```

```
+-------+---------+----------+
| t_int | t_float | t_string |
+-------+---------+----------+
|   100 |     0.3 | test3    |
|  NULL |     0.2 | test2    |
|  NULL |     0.1 | test1    |
+-------+---------+----------+
```

7.2.6.10 PARTITIONS

7.2.6.10.1 描述

表函数，生成分区临时表，可以查看某个 TABLE 的分区列表。

7.2.6.10.2 语法

```
PARTITIONS(
    "catalog"="<catalog>",
    "database"="<database>",
    "table"="<table>"
)
```

7.2.6.10.3 必填参数 (Required Parameters)

字段	描述
<catalog>	指定需要查询的集群 catalog 名。
<database>	指定需要查询的集群数据库名。
<table>	指定需要查询的集群表名。

7.2.6.10.4 返回值

字段名	描述
PartitionId	分区 ID
PartitionName	分区名称

字段名	描述
VisibleVersion	分区版本
VisibleVersionTime	分区版本提交时间
State	分区状态
PartitionKey	分区键
Range	分区范围
DistributionKey	分布键
Buckets	分桶数量
ReplicationNum	副本数
StorageMedium	存储介质
CooldownTime	冷却时间
RemoteStoragePolicy	远程存储策略
LastConsistencyCheckTime	上次一致性检查时间
DataSize	数据大小
IsInMemory	是否存在内存
ReplicaAllocation	分布策略
IsMutable	是否可变
SyncWithBaseTables	是否和基表数据同步（针对异步物化视图的分区）
UnsyncTables	和哪个基表数据不同步（针对异步物化视图的分区）

7.2.6.10.5 示例

查看 internal CATALOG 下 test 的 example_table 的分区列表

```
select * from partitions("catalog"="internal","database"="test","table"="example_table");
```

```
+-------------+--------------+----------------+--------------------+--------+--------------+-------------------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range                                                                                 |
    ↪ DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+-------------+--------------+----------------+--------------------+--------+--------------+-------------------
    ↪
|       43209 | p1            |              1 | 2025-01-17 12:35:22 | NORMAL | created_at   | [
    ↪ types: [DATEV2]; keys: [0000-01-01]; ..types: [DATEV2]; keys: [2023-01-01]; ) | id
    ↪               |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                     | `\N`                        | 0.000    |          0 | tag.location.
    ↪ default: 1 |         1 |                 1 | `\N`           |
|       43210 | p2            |              1 | 2025-01-17 12:35:22 | NORMAL | created_at   | [
    ↪ types: [DATEV2]; keys: [2023-01-01]; ..types: [DATEV2]; keys: [2024-01-01]; ) | id
    ↪               |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                     | `\N`                        | 0.000    |          0 | tag.location.
    ↪ default: 1 |         1 |                 1 | `\N`           |
```

```
|       43211 | p3            |              1 | 2025-01-17 12:35:22 | NORMAL | created_at   | [
    ↪ types: [DATEV2]; keys: [2024-01-01]; ..types: [DATEV2]; keys: [2025-01-01]; ) | id
    ↪               |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                       | `\N`                        | 0.000    |          0 | tag.location.
    ↪ default: 1 |         1 |                  1 | `\N`           |
|       43212 | p4            |              1 | 2025-01-17 12:35:22 | NORMAL | created_at   | [
    ↪ types: [DATEV2]; keys: [2025-01-01]; ..types: [DATEV2]; keys: [2026-01-01]; ) | id
    ↪               |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                       | `\N`                        | 0.000    |          0 | tag.location.
    ↪ default: 1 |         1 |                  1 | `\N`           |
+-------------+--------------+----------------+---------------------+--------+--------------+-----------------
    ↪
```

查看 example_table 下的分区名称为 partition1 的分区信息

```
select * from partitions("catalog"="internal","database"="test","table"="example_table") where
    ↪ PartitionName = "p1";
```

```
+-------------+--------------+----------------+---------------------+--------+--------------+-----------------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range                                                                                    |
    ↪ DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+-------------+--------------+----------------+---------------------+--------+--------------+-----------------
    ↪
|       43209 | p1            |              1 | 2025-01-17 12:35:22 | NORMAL | created_at   | [
    ↪ types: [DATEV2]; keys: [0000-01-01]; ..types: [DATEV2]; keys: [2023-01-01]; ) | id
    ↪               |      10 |              1 | HDD           | 9999-12-31 23:59:59 |
    ↪                       | `\N`                        | 0.000    |          0 | tag.location.
    ↪ default: 1 |         1 |                  1 | `\N`           |
+-------------+--------------+----------------+---------------------+--------+--------------+-----------------
    ↪
```

查看 user_tab 下的分区名称为 partition1 的分区 id

```
select PartitionId from partitions("catalog"="internal","database"="test","table"="example_table"
    ↪ ) where PartitionName = "p1";
```

```
+-------------+
| PartitionId |
+-------------+
|       43209 |
+-------------+
```

7.2.6.11 QUERY

7.2.6.11.1 描述

query 表函数（table-valued-function,tvf），可用于将查询语句直接透传到某个 catalog 进行数据查询

Doris 2.1.3 版本开始支持，当前仅支持透传查询 jdbc catalog。需要先在 Doris 中创建对应的 catalog。

7.2.6.11.2 语法

```
QUERY(
    "catalog" = "<catalog>",
    "query" = "<query_sql>"
  );
```

7.2.6.11.3 必填参数

query 表函数 tvf 中的每一个参数都是一个 "key"="value" 对

字段	描述
catalog	catalog 名称，需要按照 catalog 的名称填写
query	需要执行的查询语句

7.2.6.11.4 举例

可以配合desc function使用

```
desc function query("catalog" = "jdbc", "query" = "select * from test.student");
```

```
+-------+------+------+-------+---------+-------+
| Field | Type | Null | Key   | Default | Extra |
+-------+------+------+-------+---------+-------+
| id    | int  | Yes  | true  | NULL    |       |
| name  | text | Yes  | false | NULL    | NONE  |
+-------+------+------+-------+---------+-------+
```

透传查询 jdbc catalog 数据源中的表

```
select * from query("catalog" = "jdbc", "query" = "select * from test.student");
```

```
+------+---------+
| id   | name    |
+------+---------+
| 1    | alice   |
| 2    | bob     |
| 3    | jack    |
```

```
+------+---------+
```

```
select * from query("catalog" = "jdbc", "query" = "select * from test.score");
```

```
+------+---------+
| id   | score   |
+------+---------+
| 1    | 100     |
| 2    | 90      |
| 3    | 80      |
+------+---------+
```

透传关联查询 jdbc catalog 数据源中的表

```
select * from query("catalog" = "jdbc", "query" = "select a.id, a.name, b.score from test.student
    ↪  a join test.score b on a.id = b.id");
```

```
+------+---------+---------+
| id   | name    | score   |
+------+---------+---------+
| 1    | alice   | 100     |
| 2    | bob     | 90      |
| 3    | jack    | 80      |
+------+---------+---------+
```

7.2.6.12 S3

7.2.6.12.1 描述

S3 表函数（table-valued-function,tvf），可以让用户像访问关系表格式数据一样，读取并访问 S3 兼容的对象存储上的文件内容。目前支持csv/csv_with_names/csv_with_names_and_types/json/parquet/orc文件格式。

7.2.6.12.2 语法

```
S3(
    "uri" = "<uri>",
    "s3.access_key" = "<s3_access_key>",
    "s3.secret_key" = "<s3_secret_key>",
    "s3.region" = "<s3_region>",
    "s3.endpoint" = "<s3_endpoint>",
    "format" = "<format>"
    [, "<optional_property_key>" = "<optional_property_value>" [, ...] ]
  )
```

7.2.6.12.3 必选参数

参数	描述
uri	用于访问 S3 的 URI，该函数会根据 use_path_style 参数来决定使用路径样式（Path Style）还是虚拟托管样式（Virtual-hosted Style）进行访问
s3.access_key	访问 S3 的访问密钥
s3.secret_key	访问 S3 的秘密密钥
s3.region	S3 存储所在的区域
s3.endpoint	S3 存储的端点地址
format	文件格式，支持 csv/csv_with_names/csv_with_names_and_types/json/parquet/orc

7.2.6.12.4 可选参数

参数	描述	备注
s3.session_token	S3 session token	
use_path_style	默认为false，S3 SDK 默认使用 Virtual-hosted Syle 方式。但某些对象存储系统可能没开启或没支持 Virtual-hosted Style 方式的访问，此时我们可以添加 use_path_style 参数来强制使用 Path Style 方式。比如 minio 默认情况下只允许 path style 访问方式，所以在访问 MinIO 时要加上 use_path_style=true	
force_parsing_by_standard_uri	默认为 false，用于强制将非标准的 URI 解析为标准 URI	
column_separator	列分割符，默认为\t	
line_delimiter	行分割符，默认为\n	
compress_type	目前支持 UNKNOWN/PLAIN/GZ/LZO/BZ2/LZ4FRAME/DEFLATE/SNAPPYBLOCK。默认值为 UNKNOWN, 将会根据 uri 的后缀自动推断类型	
read_json_by_line	默认为 "true"，适用于 JSON 格式的导入	JSON Load
strip_outer_array	默认为 "false"，适用于 JSON 格式的导入	JSON Load
json_root	默认为空，适用于 JSON 格式的导入	JSON Load
jsonpaths	默认为空，适用于 JSON 格式的导入	JSON Load
num_as_string	默认为 false，适用于 JSON 格式的导入	JSON Load
fuzzy_parse	默认为 false，适用于 JSON 格式的导入	
trim_double_quotes	默认为 false，用于 CSV 格式的导入，裁剪掉每个字段最外层的双引号	
skip_lines	默认为 0，表示跳过 CSV 文件的前几行。适用于 csv_with_names 或 csv_with_names_and_types 格式时失效	
path_partition_keys	指定文件路径中携带的分区列名，例如 /path/to/city=beijing/date="2023-07-09", 则填写 path_partition_keys="city,date"，将会自动从路径中读取相应列名和列值进行导入	
resource	指定 Resource 名，S3 TVF 可以利用已有的 S3 Resource 来直接访问 S3。创建 S3 Resource 的方法可以参照 CREATE-RESOURCE。该功能自 2.1.4 版本开始支持。	

参数	描述	备注

7.2.6.12.5 注意事项

1. 对于 AWS S3，标准 uri styles 有以下几种：> 1. AWS Client Style(Hadoop S3 Style): s3://my-bucket ↪ /path/to/file?versionId=abc123&partNumber=77&partNumber=88。> 2. Virtual Host Style:https://my-bucket.s3.us-west-1.amazonaws.com/resources/doc.txt?versionId= ↪ abc123&partNumber=77&partNumber=88。> 3. Path Style:https://s3.us-west-1.amazonaws ↪ .com/my-bucket/resources/doc.txt?versionId=abc123&partNumber=77&partNumber ↪ =88。> > 除了支持以上三个标准常见的 URI Styles, 还支持其他一些 URI Styles（也许不常见，但也有可能有）：> 1. Virtual Host AWS Client (Hadoop S3) Mixed Style:

```
>> `s3://my-bucket.s3.us-west-1.amazonaws.com/resources/doc.txt?versionId=abc123&partNumber=77&
    ↪ partNumber=88`
```

2. Path AWS Client (Hadoop S3) Mixed Style:

```
>   > `s3://s3.us-west-1.amazonaws.com/my-bucket/resources/doc.txt?versionId=abc123&
    ↪ partNumber=77&partNumber=88`
```

详细使用案例可以参考下方的示例

2. 直接查询 TVF 或基于该 TVF 创建 View，需要拥有该 Resource 的 USAGE 权限，查询基于 TVF 创建的 View，只需要该 View 的 SELECT 权限。

7.2.6.12.6 示例

- 读取并访问 S3 兼容的对象存储上的 CSV 格式文件

```
select * from s3("uri" = "http://127.0.0.1:9312/test2/student1.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
```

```
                "s3.endpoint" = "http://127.0.0.1:9312",
                "s3.region" = "us-east-1",
                "format" = "csv",
                "use_path_style" = "true") order by c1;
```

- 可以配合 desc function 使用

```
Desc function s3("uri" = "http://127.0.0.1:9312/test2/student1.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv",
                 "use_path_style" = "true");
```

- 不同 url schema 的写法

http:// 、 https:// 使用示例：

```
-- 注意 URI Bucket 写法以及`use_path_style`参数设置，HTTP 同理。
-- 由于设置了 `"use_path_style"="true"`，所以将采用 Path Style 的方式访问 S3。
select * from s3(
    "uri" = "https://endpoint/bucket/file/student.csv",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint" = "endpoint",
    "s3.region" = "region",
    "format" = "csv",
    "use_path_style"="true");

-- 注意 URI Bucket 写法以及 use_path_style 参数设置，http 同理。
-- 由于设置了 `"use_path_style"="false"`，所以将采用 Virtual-hosted Style 方式访问 S3。
select * from s3(
    "uri" = "https://bucket.endpoint/bucket/file/student.csv",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint" = "endpoint",
    "s3.region" = "region",
    "format" = "csv",
    "use_path_style"="false");

-- 阿里云 OSS 和腾讯云 COS 采用 Virtual-hosted Style 方式访问 S3。
-- OSS
select * from s3(
```

```
    "uri" = "http://example-bucket.oss-cn-beijing.aliyuncs.com/your-folder/file.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint" = "oss-cn-beijing.aliyuncs.com",
    "s3.region" = "oss-cn-beijing",
    "format" = "parquet",
    "use_path_style" = "false");
-- COS
select * from s3(
    "uri" = "https://example-bucket.cos.ap-hongkong.myqcloud.com/your-folder/file.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint" = "cos.ap-hongkong.myqcloud.com",
    "s3.region" = "ap-hongkong",
    "format" = "parquet",
    "use_path_style" = "false");

-- MinIO
select * from s3(
    "uri" = "s3://bucket/file.csv",
    "s3.endpoint" = "http://172.21.0.101:9000",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.region" = "us-east-1",
    "format" = "csv"
);

-- 百度云 BOS 采用兼容 S3 协议的 Virtual-hosted Style 方式访问 S3。
-- BOS
select * from s3(
    "uri" = "https://example-bucket.s3.bj.bcebos.com/your-folder/file.parquet",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.region" = "bj",
    "s3.endpoint" = "http://bj.bcebos.com",
    "format" = "parquet",
    "use_path_style" = "false");
```

- s3:// 使用示例：

```
-- 注意 URI Bucket 写法，无需设置 `use_path_style` 参数。
-- 将采用 Virtual-hosted Style 方式访问 S3。
select * from s3(
    "uri" = "s3://bucket/file/student.csv",
    "s3.endpoint"= "endpont",
```

```
    "s3.region"= "region",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "format" = "csv");
```

- 其它支持的 URI 风格示例：

```
-- Virtual Host AWS Client (Hadoop S3) Mixed Style。通过设置 `use_path_style = false` 以及 `
    ↪ force_parsing_by_standard_uri = true` 来使用。
select * from s3(
    "URI" = "s3://my-bucket.s3.us-west-1.amazonaws.com/resources/doc.txt?versionId=abc123&
        ↪ partNumber=77&partNumber=88",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint"= "endpont",
    "s3.region"= "region",
    "format" = "csv",
    "use_path_style"="false",
    "force_parsing_by_standard_uri"="true");

-- Path AWS Client (Hadoop S3) Mixed Style。通过设置 `use_path_style = true` 以及 `force_
    ↪ parsing_by_standard_uri = true` 来使用。
select * from s3(
    "URI" = "s3://s3.us-west-1.amazonaws.com/my-bucket/resources/doc.txt?versionId=abc123&
        ↪ partNumber=77&partNumber=88",
    "s3.access_key"= "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint"= "endpont",
    "s3.region"= "region",
    "format" = "csv",
    "use_path_style"="true",
    "force_parsing_by_standard_uri"="true");
```

- CSV format

由于 S3 table-valued-function 事先并不知道 Table Schema，所以会先读一遍文件来解析出 Table Schema。

csv 格式：S3 table-valued-function 读取 S3 上的文件并当作 CSV 文件来处理，读取文件中的第一行用于解析 Table Schema。文件第一行的列个数 n 将作为 Table Schema 的列个数，Table Schema 的列名则自动取名为 `c1，c2，...，cn`，列类型都设置为 `String`, 举例：

student1.csv 文件内容为：

```
1,ftw,12
2,zs,18
3,ww,20
```

- 使用 S3 TVF

```
select * from s3("uri" = "http://127.0.0.1:9312/test2/student1.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv",
                 "use_path_style" = "true") order by c1;
```

```
+------+------+------+
| c1   | c2   | c3   |
+------+------+------+
| 1    | ftw  | 12   |
| 2    | zs   | 18   |
| 3    | ww   | 20   |
+------+------+------+
```

- 可以配合 `desc function S3()` 来查看 Table Schema

```
Desc function s3("uri" = "http://127.0.0.1:9312/test2/student1.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv",
                 "use_path_style" = "true");
```

```
+-------+------+------+-------+---------+-------+
| Field | Type | Null | Key   | Default | Extra |
+-------+------+------+-------+---------+-------+
| c1    | TEXT | Yes  | false | NULL    | NONE  |
| c2    | TEXT | Yes  | false | NULL    | NONE  |
| c3    | TEXT | Yes  | false | NULL    | NONE  |
+-------+------+------+-------+---------+-------+
```

- csv_with_names format

`csv_with_names` 格式：解析文件的第一行作为 Table Schema 的列个数和列名，列类型则都设置为 `String`, 举例：student_with_names.csv 文件内容为

```
id,name,age
1,ftw,12
2,zs,18
3,ww,20
```

- 使用 S3 tvf

```
select * from s3("uri" = "http://127.0.0.1:9312/test2/student_with_names.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv_with_names",
                 "use_path_style" = "true") order by id;
```

```
+------+------+------+
| id   | name | age  |
+------+------+------+
| 1    | ftw  | 12   |
| 2    | zs   | 18   |
| 3    | ww   | 20   |
+------+------+------+
```

- 同样配合 `desc function S3()` 可查看 Table Schema

```
Desc function s3("uri" = "http://127.0.0.1:9312/test2/student_with_names.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv_with_names",
                 "use_path_style" = "true");
```

```
+-------+------+------+-------+---------+-------+
| Field | Type | Null | Key   | Default | Extra |
+-------+------+------+-------+---------+-------+
| id    | TEXT | Yes  | false | NULL    | NONE  |
| name  | TEXT | Yes  | false | NULL    | NONE  |
| age   | TEXT | Yes  | false | NULL    | NONE  |
+-------+------+------+-------+---------+-------+
```

- csv_with_names_and_types format

`csv_with_names_and_types` 格式：目前暂不支持从 CSV 文件中解析出 Column Type。使用该 Format 时，S3 TVF 会解析文件的第一行作为 Table Schema 的列个数和列名，列类型则都设置为 String，同时将忽略该文件的第二行。

`student_with_names_and_types.csv` 文件内容为

```
id,name,age
INT,STRING,INT
1,ftw,12
2,zs,18
3,ww,20
```

- 使用 S3 TVF

```
select * from s3("uri" = "http://127.0.0.1:9312/test2/student_with_names_and_types.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv_with_names_and_types",
                 "use_path_style" = "true") order by id;
```

```
+------+------+------+
| id   | name | age  |
+------+------+------+
| 1    | ftw  | 12   |
| 2    | zs   | 18   |
| 3    | ww   | 20   |
+------+------+------+
```

- 同样配合 `desc function S3()` 可查看 Table Schema

```
Desc function s3("uri" = "http://127.0.0.1:9312/test2/student_with_names_and_types.csv",
                 "s3.access_key"= "minioadmin",
                 "s3.secret_key" = "minioadmin",
                 "s3.endpoint" = "http://127.0.0.1:9312",
                 "s3.region" = "us-east-1",
                 "format" = "csv_with_names_and_types",
                 "use_path_style" = "true");
```

```
+-------+------+------+-------+---------+-------+
| Field | Type | Null | Key   | Default | Extra |
+-------+------+------+-------+---------+-------+
| id    | TEXT | Yes  | false | NULL    | NONE  |
| name  | TEXT | Yes  | false | NULL    | NONE  |
| age   | TEXT | Yes  | false | NULL    | NONE  |
+-------+------+------+-------+---------+-------+
```

- JSON format

json 格式：JSON 格式涉及到较多的可选参数，各个参数的意义可以参考：Json Load。S3 TVF 查询 JSON 格式文件时根据 json_root 和 jsonpaths 参数定位到一个 JSON 对象，将该对象的中的 key 作为 Table Schema 的列名，列类型都设置为 String。举例：

data.json 文件

```
[{"id":1, "name":"ftw", "age":18}]
[{"id":2, "name":"xxx", "age":17}]
[{"id":3, "name":"yyy", "age":19}]
```

- 使用 S3 TVF 查询

```
select * from s3(
    "uri" = "http://127.0.0.1:9312/test2/data.json",
    "s3.access_key"= "minioadmin",
    "s3.secret_key" = "minioadmin",
    "s3.endpoint" = "http://127.0.0.1:9312",
    "s3.region" = "us-east-1",
    "format" = "json",
    "strip_outer_array" = "true",
    "read_json_by_line" = "true",
    "use_path_style"="true");
+------+------+------+
| id   | name | age  |
+------+------+------+
| 1    | ftw  | 18   |
| 2    | xxx  | 17   |
| 3    | yyy  | 19   |
+------+------+------+

select * from s3(
    "uri" = "http://127.0.0.1:9312/test2/data.json",
    "s3.access_key"= "minioadmin",
    "s3.secret_key" = "minioadmin",
    "s3.endpoint" = "http://127.0.0.1:9312",
    "s3.region" = "us-east-1",
    "format" = "json",
    "strip_outer_array" = "true",
    "jsonpaths" = `"[\"$.id\", \"$.age\"]"`,
    "use_path_style"="true");
+------+------+
| id   | age  |
+------+------+
| 1    | 18   |
```

```
| 2    | 17   |
| 3    | 19   |
+------+------+
```

- Parquet format

parquet 格式：S3 TVF 支持从 Parquet 文件中解析出 Table Schema 的列名、列类型。举例：

```
select * from s3(
   "uri" = "http://127.0.0.1:9312/test2/test.snappy.parquet",
   "s3.access_key"= "minioadmin",
   "s3.secret_key" = "minioadmin",
   "s3.endpoint" = "http://127.0.0.1:9312",
   "s3.region" = "us-east-1",
   "format" = "parquet",
   "use_path_style"="true") limit 5;
```

```
     +-----------+-----------------------------------------+----------------+----------+----------------------
         ↪
     | p_partkey | p_name                                  | p_mfgr         | p_brand  | p_
         ↪ type                  | p_size | p_container | p_retailprice | p_comment
         ↪           |
     +-----------+-----------------------------------------+----------------+----------+----------------------
         ↪
     |         1 | goldenrod lavender spring chocolate lace | Manufacturer#1 | Brand#13 | PROMO
         ↪  BURNISHED COPPER  |      7 | JUMBO PKG   |           901 | ly. slyly ironi     |
     |         2 | blush thistle blue yellow saddle         | Manufacturer#1 | Brand#13 | LARGE
         ↪  BRUSHED BRASS     |      1 | LG CASE     |           902 | lar accounts amo    |
     |         3 | spring green yellow purple cornsilk      | Manufacturer#4 | Brand#42 |
         ↪ STANDARD POLISHED BRASS |     21 | WRAP CASE   |           903 | egular deposits
         ↪ hag |
     |         4 | cornflower chocolate smoke green pink    | Manufacturer#3 | Brand#34 | SMALL
         ↪  PLATED BRASS      |     14 | MED DRUM    |           904 | p furiously r       |
     |         5 | forest brown coral puff cream            | Manufacturer#3 | Brand#32 |
         ↪ STANDARD POLISHED TIN   |     15 | SM PKG      |           905 |  wake carefully
         ↪     |
     +-----------+-----------------------------------------+----------------+----------+----------------------
         ↪
```

```
MySQL [(none)]> desc function s3(
    "uri" = "http://127.0.0.1:9312/test2/test.snappy.parquet",
    "s3.access_key"= "minioadmin",
    "s3.secret_key" = "minioadmin",
    "s3.endpoint" = "http://127.0.0.1:9312",
    "s3.region" = "us-east-1",
```

```
    "format" = "parquet",
    "use_path_style"="true");
```

```
+---------------+--------------+------+-------+---------+-------+
| Field         | Type         | Null | Key   | Default | Extra |
+---------------+--------------+------+-------+---------+-------+
| p_partkey     | INT          | Yes  | false | NULL    | NONE  |
| p_name        | TEXT         | Yes  | false | NULL    | NONE  |
| p_mfgr        | TEXT         | Yes  | false | NULL    | NONE  |
| p_brand       | TEXT         | Yes  | false | NULL    | NONE  |
| p_type        | TEXT         | Yes  | false | NULL    | NONE  |
| p_size        | INT          | Yes  | false | NULL    | NONE  |
| p_container   | TEXT         | Yes  | false | NULL    | NONE  |
| p_retailprice | DECIMAL(9,0) | Yes  | false | NULL    | NONE  |
| p_comment     | TEXT         | Yes  | false | NULL    | NONE  |
+---------------+--------------+------+-------+---------+-------+
```

- orc format

orc 格式：和 parquet format 使用方法一致，将 format 参数设置为 orc。

```
select * from s3(
    "uri" = "http://127.0.0.1:9312/test2/test.snappy.orc",
    "s3.access_key"= "minioadmin",
    "s3.secret_key" = "minioadmin",
    "s3.endpoint" = "http://127.0.0.1:9312",
    "s3.region" = "us-east-1",
    "format" = "orc",
    "use_path_style"="true") limit 5;
```

```
  +-----------+-----------------------------------------+----------------+----------+-------------------------
    ↪
| p_partkey | p_name                                    | p_mfgr         | p_brand  | p_type
    ↪                  | p_size | p_container | p_retailprice | p_comment           |
+-----------+-----------------------------------------+----------------+----------+-------------------------+-
    ↪
|         1 | goldenrod lavender spring chocolate lace | Manufacturer#1 | Brand#13 | PROMO
    ↪ BURNISHED COPPER  |      7 | JUMBO PKG   |           901 | ly. slyly ironi     |
|         2 | blush thistle blue yellow saddle         | Manufacturer#1 | Brand#13 | LARGE
    ↪ BRUSHED BRASS     |      1 | LG CASE     |           902 | lar accounts amo    |
|         3 | spring green yellow purple cornsilk      | Manufacturer#4 | Brand#42 | STANDARD
    ↪ POLISHED BRASS |     21 | WRAP CASE   |           903 | egular deposits hag |
|         4 | cornflower chocolate smoke green pink    | Manufacturer#3 | Brand#34 | SMALL
    ↪ PLATED BRASS      |     14 | MED DRUM    |           904 | p furiously r       |
```

```
|         5 | forest brown coral puff cream             | Manufacturer#3 | Brand#32 | STANDARD
    ↪ POLISHED TIN    |     15 | SM PKG      |           905 |  wake carefully     |
+-----------+-------------------------------------------+----------------+----------+-------------------------+-
    ↪
```

- URI 包含通配符

URI 可以使用通配符来读取多个文件。注意：如果使用通配符要保证各个文件的格式是一致的 (尤其是 csv/csv_with_names/csv_with_names_and_types 算做不同的格式)，S3 TVF 用第一个文件来解析出 Table Schema。如下两个 CSV 文件：

```
// file1.csv
1,aaa,18
2,qqq,20
3,qwe,19

// file2.csv
5,cyx,19
6,ftw,21
```

- 可以在 URI 上使用通配符来导入。

```
select * from s3(
        "uri" = "http://127.0.0.1:9312/test2/file*.csv",
        "s3.access_key"= "minioadmin",
        "s3.secret_key" = "minioadmin",
        "s3.endpoint" = "http://127.0.0.1:9312",
        "s3.region" = "us-east-1",
        "format" = "csv",
        "use_path_style"="true");
```

```
 +------+------+------+
| c1   | c2   | c3   |
+------+------+------+
| 1    | aaa  | 18   |
| 2    | qqq  | 20   |
| 3    | qwe  | 19   |
| 5    | cyx  | 19   |
| 6    | ftw  | 21   |
+------+------+------+
```

- 配合 `insert into` 和 `cast` 使用 S3 TVF

```
-- 创建 Doris 内部表
CREATE TABLE IF NOT EXISTS ${testTable}
    (
        id int,
        name varchar(50),
        age int
    )
    COMMENT "my first table"
    DISTRIBUTED BY HASH(id) BUCKETS 32
    PROPERTIES("replication_num" = "1");

-- 使用 S3 插入数据
insert into ${testTable} (id,name,age)
select cast (id as INT) as id, name, cast (age as INT) as age
from s3(
    "uri" = "${uri}",
    "s3.access_key"= "${ak}",
    "s3.secret_key" = "${sk}",
    "s3.endpoint" = "http://127.0.0.1:9312",
    "s3.region" = "us-east-1",
    "format" = "${format}",
    "strip_outer_array" = "true",
    "read_json_by_line" = "true",
    "use_path_style" = "true");
```

7.2.6.13 TASKS

7.2.6.13.1 描述

表函数，生成 tasks 临时表，可以查看当前 doris 集群中的 job 产生的 tasks 信息。

7.2.6.13.2 语法

```
TASKS(
    "type"="<type>"
)
```

7.2.6.13.3 必填参数 (Required Parameters)

字段名	描述
<type>	任务的类型：`insert`：insert into 类型的任务。`mv`：物化视图类型的任务。

7.2.6.13.4 返回值

- `tasks("type"="insert")` insert 类型的 tasks 返回值

| 字段名 | 描述 | |————-|————————| | TaskId | task id | | JobId | job id | | JobName | job 名称 | | Label | label | | Status | task 状态 | | ErrorMsg | task 失败信息 | | CreateTime| task 创建时间 | | FinishTime| task 结束时间 | | TrackingUrl| task tracking url | | LoadStatistic| task 统计信息 | | User | 执行用户 |

- `tasks("type"="mv")` MV 类型的 tasks 返回值

| 字段名 | 描述 | |——————-|———————————-| | TaskId | task id | | JobId | job id | | JobName | job 名称 | | MvId | 物化视图 id | | MvName | 物化视图名称 | | MvDatabaseId | 物化视图所属 db id | | MvDatabaseName | 物化视图所属 db 名称 | | Status | task 状态 | | ErrorMsg | task 失败信息 | | CreateTime | task 创建时间 | | StartTime | task 开始运行时间 | | FinishTime | task 结束运行时间 | | DurationMs | task 运行时间 | | TaskContext | task 运行参数 | | RefreshMode | 刷新模式 | | NeedRefreshPartitions | 本次 task 需要刷新的分区信息 | | CompletedPartitions | 本次 task 刷新完成的分区信息 | | Progress | task 运行进度 |

7.2.6.13.5 示例

查看所有物化视图的 tasks

```
select * from tasks("type"="mv");
```

```
+----------------+-------+-----------------+-------+--------------------------+-------------+-----------------
    ↪
| TaskId          | JobId | JobName          | MvId  | MvName                   | MvDatabaseId |
    ↪ MvDatabaseName                                          | Status  | ErrorMsg | CreateTime
    ↪          | StartTime           | FinishTime          | DurationMs | TaskContext
    ↪                                                  | RefreshMode | NeedRefreshPartitions
    ↪                          | CompletedPartitions                           | Progress      |
    ↪  LastQueryId                        |
+----------------+-------+-----------------+-------+--------------------------+-------------+-----------------
    ↪
| 509478985247053 | 23369 | inner_mtmv_23363 | 23363 | range_date_up_union_mv1  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:10 | 2025-01-08 18:19:10 | 2025-01-08 18:19:10 | 233        | {"triggerMode":"
    ↪ SYSTEM","isComplete":false}                | COMPLETE    | ["p_20231001_20231101"]
    ↪                          | ["p_20231001_20231101"]                       | 100.00% (1/1) |
    ↪ 71897c47d0d94fd2-9ca52a0e6eb3bff5 |
| 509486915704885 | 23369 | inner_mtmv_23363 | 23363 | range_date_up_union_mv1  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:17 | 2025-01-08 18:19:17 | 2025-01-08 18:19:17 | 227        | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | PARTIAL     | ["p_20231101_20231201"]
    ↪                          | ["p_20231101_20231201"]                       | 100.00% (1/1) | 9
    ↪ bf5ff69d4cc4c78-b50505436c8410c4 |
```

```
| 509487197275880 | 23369 | inner_mtmv_23363 | 23363 | range_date_up_union_mv1  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:18 | 2025-01-08 18:19:18 | 2025-01-08 18:19:18 | 191        | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | PARTIAL     | ["p_20231101_20231201"]
    ↪                       | ["p_20231101_20231201"]                      | 100.00% (1/1) | 5
    ↪ b3b4525b6774b5b-89b070042cdcbcd5 |
| 509478131194211 | 23377 | inner_mtmv_23371 | 23371 | range_date_up_union_mv2  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:10 | 2025-01-08 18:19:10 | 2025-01-08 18:19:10 | 156        | {"triggerMode":"
    ↪ SYSTEM","isComplete":false}                | COMPLETE    | ["p_20231001_20231101"]
    ↪                       | ["p_20231001_20231101"]                      | 100.00% (1/1) | 6
    ↪ d0a0782819b446e-b9da5d5de513ce00 |
| 509486057129101 | 23377 | inner_mtmv_23371 | 23371 | range_date_up_union_mv2  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:17 | 2025-01-08 18:19:17 | 2025-01-08 18:19:18 | 213        | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | PARTIAL     | ["p_20231101_20231201"]
    ↪                       | ["p_20231101_20231201"]                      | 100.00% (1/1) |
    ↪ f1303483e3db43e7-aa424acc32dc39ca |
| 509486143784554 | 23377 | inner_mtmv_23371 | 23371 | range_date_up_union_mv2  | 21805        |
    ↪ regression_test_nereids_rules_p0_mv_create_part_and_up | SUCCESS |          | 2025-01-08
    ↪ 18:19:18 | 2025-01-08 18:19:18 | 2025-01-08 18:19:18 | 151        | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | PARTIAL     | ["p_20231101_20231201"]
    ↪                       | ["p_20231101_20231201"]                      | 100.00% (1/1) | 8
    ↪ d29b11ac41f4fe0-9d7c86372707310b |
| 488317385772600 | 21794 | inner_mtmv_21788 | 21788 | test_tablet_type_mtmv_mv | 16016        |
    ↪ zd                                                  | SUCCESS |          | 2025-01-08
    ↪ 12:26:29 | 2025-01-08 12:26:29 | 2025-01-08 12:26:29 | 1          | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":true}  | NOT_REFRESH | []
    ↪                                             | `\N`
    ↪                                             | `\N`            |
    ↪                                    |
| 437156301250803 | 19508 | inner_mtmv_19494 | 19494 | mv1                      | 16016        |
    ↪ zd                                                  | SUCCESS |          | 2025-01-07
    ↪ 22:13:48 | 2025-01-07 22:13:48 | 2025-01-07 22:17:45 | 236985     | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | COMPLETE    | ["p_20210101_MAXVALUE","p_
    ↪ 20200101_20210101"] | ["p_20210101_MAXVALUE","p_20200101_20210101"] | 100.00% (2/2) |
    ↪ 7965b4ddce8a4480-8884e9701679c1c4 |
| 439689059641969 | 19508 | inner_mtmv_19494 | 19494 | mv1                      | 16016        |
    ↪ zd                                                  | SUCCESS |          | 2025-01-07
    ↪ 22:55:59 | 2025-01-07 22:55:59 | 2025-01-07 22:55:59 | 35         | {"triggerMode":"
    ↪ MANUAL","partitions":[],"isComplete":false} | NOT_REFRESH | []
    ↪                                             | `\N`
    ↪                                             | `\N`            |
    ↪                                    |
+-----------------+-------+-----------------+-------+--------------------------+-------------+----------------
```

```
    ↪
```

查看所有 insert 任务的 tasks

```
select * from tasks("type"="insert");
```

```
+----------------+----------------+----------------+------------------------------+---------+----------+--------
    ↪
| TaskId         | JobId          | JobName        | Label                        | Status  |
    ↪ ErrorMsg | CreateTime          | StartTime           | FinishTime          | TrackingUrl
    ↪ | LoadStatistic | User |
+----------------+----------------+----------------+------------------------------+---------+----------+--------
    ↪
| 79133848479750 | 78533940810334 | insert_tab_job | 78533940810334_79133848479750 | SUCCESS |
    ↪          | 2025-01-17 14:42:54 | 2025-01-17 14:42:54 | 2025-01-17 14:42:54 |
    ↪ |               | root |
+----------------+----------------+----------------+------------------------------+---------+----------+--------
    ↪
```

7.3 SQL 语句

7.3.1 数据查询

7.3.1.1 SELECT

7.3.1.1.1 描述

主要介绍 Select 语法使用

语法：

```
SELECT
    [hint_statement, ...]
    [ALL | DISTINCT | DISTINCTROW | ALL EXCEPT ( col_name1 [, col_name2, col_name3, ...] )]
    select_expr [, select_expr ...]
    [FROM table_references
      [PARTITION partition_list]
      [TABLET tabletid_list]
      [TABLESAMPLE sample_value [ROWS | PERCENT]
        [REPEATABLE pos_seek]]
    [WHERE where_condition]
    [GROUP BY [GROUPING SETS | ROLLUP | CUBE] {col_name | expr | position}]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position}
      [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

```
    [INTO OUTFILE 'file_name']
```

语法说明：

1. select_expr, ... 检索并在结果中显示的列，使用别名时，as 为自选。
2. select_expr, ... 检索的目标表（一个或者多个表（包括子查询产生的临时表）
3. where_definition 检索条件（表达式），如果存在 WHERE 子句，其中的条件对行数据进行筛选。where_condition 是一个表达式，对于要选择的每一行，其计算结果为 true。如果没有 WHERE 子句，该语句将选择所有行。在 WHERE 表达式中，您可以使用除聚合函数之外的任何 MySQL 支持的函数和运算符
4. `ALL | DISTINCT`：对结果集进行刷选，all 为全部，distinct/distinctrow 将刷选出重复列，默认为 all
5. `ALL EXCEPT`：对全部（all）结果集进行筛选，except 指定要从全部结果集中排除的一个或多个列的名称。输出中将忽略所有匹配的列名称。
6. `INTO OUTFILE 'file_name'`：保存结果至新文件（之前不存在）中，区别在于保存的格式。
7. `Group by having`：对结果集进行分组，having 出现则对 group by 的结果进行刷选。`Grouping Sets`、`Rollup`、Cube 为 group by 的扩展，详细可参考GROUPING SETS 设计文档。
8. `Order by`: 对最后的结果进行排序，Order by 通过比较一列或者多列的大小来对结果集进行排序。

Order by 是比较耗时耗资源的操作，因为所有数据都需要发送到 1 个节点后才能排序，排序操作相比不排序操作需要更多的内存。

如果需要返回前 N 个排序结果，需要使用 LIMIT 从句；为了限制内存的使用，如果用户没有指定 LIMIT 从句，则默认返回前 65535 个排序结果。

9. `Limit n`: 限制输出结果中的行数，`limit m,n` 表示从第 m 行开始输出 n 条记录，使用 limit m,n 的时候要加上 order by 才有意义，否则每次执行的数据可能会不一致
10. `Having` 从句不是过滤表中的行数据，而是过滤聚合函数产出的结果。

通常来说 `having` 要和聚合函数（例如:`COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`）以及 `group by` 从句一起使用。

11. SELECT 支持使用 PARTITION 显式分区选择，其中包含 `table_reference` 中表的名称后面的分区或子分区（或两者）列表。
12. `[TABLET tids] TABLESAMPLE n [ROWS | PERCENT] [REPEATABLE seek]`: 在 FROM 子句中限制表的读取行数，根据指定的行数或百分比从表中伪随机的选择数个 Tablet，REPEATABLE 指定种子数可使选择的样本再次返回，此外也可手动指定 TableID，注意这只能用于 OLAP 表。
13. `hint_statement`: 在 selectlist 前面使用 hint 表示可以通过 hint 去影响优化器的行为以期得到想要的执行计划，详情可参考joinHint 使用文档。

语法约束：

1. SELECT 也可用于检索计算的行而不引用任何表。

2. 所有的字句必须严格地按照上面格式排序，一个 HAVING 子句必须位于 GROUP BY 子句之后，并位于 ORDER BY 子句之前。
3. 别名关键词 AS 自选。别名可用于 group by，order by 和 having
4. Where 子句：执行 WHERE 语句以确定哪些行应被包含在 GROUP BY 部分中，而 HAVING 用于确定应使用结果集中的哪些行。
5. HAVING 子句可以引用总计函数，而 WHERE 子句不能引用，如 count,sum,max,min,avg，同时，where 子句可以引用除总计函数外的其他函数。Where 子句中不能使用列别名来定义条件。
6. Group by 后跟 with rollup 可以对结果进行一次或者多次统计。

联接查询：

Doris 支持以下 JOIN 语法

```
JOIN
table_references:
    table_reference [, table_reference] …
table_reference:
    table_factor
  | join_table
table_factor:
    tbl_name [[AS] alias]
        [{USE|IGNORE|FORCE} INDEX (key_list)]
  | ( table_references )
  | { OJ table_reference LEFT OUTER JOIN table_reference
        ON conditional_expr }
join_table:
    table_reference [INNER | CROSS] JOIN table_factor [join_condition]
  | table_reference LEFT [OUTER] JOIN table_reference join_condition
  | table_reference NATURAL [LEFT [OUTER]] JOIN table_factor
  | table_reference RIGHT [OUTER] JOIN table_reference join_condition
  | table_reference NATURAL [RIGHT [OUTER]] JOIN table_factor
join_condition:
    ON conditional_expr
```

UNION 语法：

```
SELECT ...
UNION [ALL| DISTINCT] SELECT ......
[UNION [ALL| DISTINCT] SELECT ...]
```

UNION 用于将多个 SELECT 语句的结果组合到单个结果集中。

第一个 SELECT 语句中的列名称用作返回结果的列名称。在每个 SELECT语句的相应位置列出的选定列应具有相同的数据类型。（例如，第一个语句选择的第一列应该与其他语句选择的第一列具有相同的类型。）

默认行为 UNION是从结果中删除重复的行。可选 DISTINCT 关键字除了默认值之外没有任何效果，因为它还指定了重复行删除。使用可选 ALL 关键字，不会发生重复行删除，结果包括所有 SELECT 语句中的所有匹配行

WITH 语句：

要指定公用表表达式，请使用 WITH 具有一个或多个逗号分隔子句的子句。每个子条款都提供一个子查询，用于生成结果集，并将名称与子查询相关联。下面的示例定义名为的 CTE cte1 和 cte2 中 WITH 子句，并且是指在它们的顶层 SELECT 下面的 WITH 子句：

```
WITH
  cte1 AS (SELECT a, b FROM table1),
  cte2 AS (SELECT c, d FROM table2)
SELECT b, d FROM cte1 JOIN cte2
WHERE cte1.a = cte2.c;
```

在包含该 WITH子句的语句中，可以引用每个 CTE 名称以访问相应的 CTE 结果集。

CTE 名称可以在其他 CTE 中引用，从而可以基于其他 CTE 定义 CTE。

目前不支持递归的 CTE。

7.3.1.1.2 示例

1. 查询年龄分别是 18,20,25 的学生姓名

```
select Name from student where age in (18,20,25);
```

2. ALL EXCEPT 示例

```
-- 查询除了学生年龄的所有信息
select * except(age) from student;
```

3. GROUP BY 示例

```
--查询 tb_book 表，按照 type 分组，求每类图书的平均价格，
select type,avg(price) from tb_book group by type;
```

4. DISTINCT 使用

```
--查询 tb_book 表，除去重复的 type 数据
select distinct type from tb_book;
```

5. ORDER BY 示例

对查询结果进行升序（默认）或降序（DESC）排列。升序 NULL 在最前面，降序 NULL 在最后面

```
--查询 tb_book 表中的所有记录，按照 id 降序排列，显示三条记录
select * from tb_book order by id desc limit 3;
```

6. LIKE 模糊查询

可实现模糊查询，它有两种通配符：%和_，%可以匹配一个或多个字符，_可以匹配一个字符

```
--查找所有第二个字符是 h 的图书
select * from tb_book where name like('_h%');
```

7. LIMIT 限定结果行数

```
--1.降序显示 3 条记录
select * from tb_book order by price desc limit 3;

--2.从 id=1 显示 4 条记录
select * from tb_book where id limit 1,4;
```

8. CONCAT 联合多列

```
--把 name 和 price 合并成一个新的字符串输出
select id,concat(name,":",price) as info,type from tb_book;
```

9. 使用函数和表达式

```
--计算 tb_book 表中各类图书的总价格
select sum(price) as total,type from tb_book group by type;
--price 打八折
select *,(price * 0.8) as "八折" from tb_book;
```

10. UNION 示例

```
 SELECT a FROM t1 WHERE a = 10 AND B = 1 ORDER by a LIMIT 10
 UNION
 SELECT a FROM t2 WHERE a = 11 AND B = 2 ORDER by a LIMIT 10;
```

11. WITH 子句示例

```
 WITH cte AS
 (
   SELECT 1 AS col1, 2 AS col2
   UNION ALL
   SELECT 3, 4
 )
 SELECT col1, col2 FROM cte;
```

12. JOIN 示例

```
SELECT * FROM t1 LEFT JOIN (t2, t3, t4)
                 ON (t2.a = t1.a AND t3.b = t1.b AND t4.c = t1.c)
```

等同于

```
SELECT * FROM t1 LEFT JOIN (t2 CROSS JOIN t3 CROSS JOIN t4)
                 ON (t2.a = t1.a AND t3.b = t1.b AND t4.c = t1.c)
```

13. INNER JOIN

```
SELECT t1.name, t2.salary
  FROM employee AS t1 INNER JOIN info AS t2 ON t1.name = t2.name;

SELECT t1.name, t2.salary
  FROM employee t1 INNER JOIN info t2 ON t1.name = t2.name;
```

14. LEFT JOIN

```
SELECT left_tbl.*
  FROM left_tbl LEFT JOIN right_tbl ON left_tbl.id = right_tbl.id
  WHERE right_tbl.id IS NULL;
```

15. RIGHT JOIN

```
mysql> SELECT * FROM t1 RIGHT JOIN t2 ON (t1.a = t2.a);
+------+------+------+------+
| a    | b    | a    | c    |
+------+------+------+------+
|    2 | y    |    2 | z    |
| NULL | NULL |    3 | w    |
+------+------+------+------+
```

16. TABLESAMPLE

```
--在t1中伪随机的抽样1000行。注意实际是根据表的统计信息选择若干Tablet，被选择的Tablet
    ↪ 总行数可能大于1000，所以若想明确返回1000行需要加上Limit。
SELECT * FROM t1 TABLET(10001) TABLESAMPLE(1000 ROWS) REPEATABLE 2 limit 1000;
```

7.3.1.1.3 关键词

SELECT

最佳实践

1. 关于 SELECT 子句的一些附加知识

- 可以使用 AS alias_name 为 select_expr 指定别名。别名用作表达式的列名，可用于 GROUP BY，ORDER BY 或 HAVING 子句。AS 关键字是在指定列的别名时养成使用 AS 是一种好习惯。
- FROM 后的 table_references 指示参与查询的一个或者多个表。如果列出了多个表，就会执行 JOIN 操作。而对于每一个指定表，都可以为其定义别名
- SELECT 后被选择的列，可以在 ORDER IN 和 GROUP BY 中，通过列名、列别名或者代表列位置的整数（从 1 开始）来引用

```
SELECT college, region, seed FROM tournament
  ORDER BY region, seed;

SELECT college, region AS r, seed AS s FROM tournament
  ORDER BY r, s;

SELECT college, region, seed FROM tournament
  ORDER BY 2, 3;
```

- 如果 ORDER BY 出现在子查询中，并且也应用于外部查询，则最外层的 ORDER BY 优先。
- 如果使用了 GROUP BY，被分组的列会自动按升序排列（就好像有一个 ORDER BY 语句后面跟了同样的列）。如果要避免 GROUP BY 因为自动排序生成的开销，添加 ORDER BY NULL 可以解决：

```
SELECT a, COUNT(b) FROM test_table GROUP BY a ORDER BY NULL;
```

- 当使用 ORDER BY 或 GROUP BY 对 SELECT 中的列进行排序时，服务器仅使用 max_sort_length 系统变量指示的初始字节数对值进行排序。
- Having 子句一般应用在最后，恰好在结果集被返回给 MySQL 客户端前，且没有进行优化。（而 LIMIT 应用在 HAVING 后）

  SQL 标准要求：HAVING 必须引用在 GROUP BY 列表中或者聚合函数使用的列。然而，MySQL 对此进行了扩展，它允许 HAVING 引用 Select 子句列表中的列，还有外部子查询的列。

  如果 HAVING 引用的列具有歧义，会有警告产生。下面的语句中，col2 具有歧义：

```
SELECT COUNT(col1) AS col2 FROM t GROUP BY col2 HAVING col2 = 2;
```

- 切记不要在该使用 WHERE 的地方使用 HAVING。HAVING 是和 GROUP BY 搭配的。

- HAVING 子句可以引用聚合函数，而 WHERE 不能。

```
SELECT user, MAX(salary) FROM users
  GROUP BY user HAVING MAX(salary) > 10;
```

- LIMIT 子句可用于约束 SELECT 语句返回的行数。LIMIT 可以有一个或者两个参数，都必须为非负整数。

```
/*取回结果集中的 6~15 行*/
SELECT * FROM tbl LIMIT 5,10;
/*那如果要取回一个设定某个偏移量之后的所有行，可以为第二参数设定一个非常大的常量。
    ↪ 以下查询取回从第 96 行起的所有数据*/
SELECT * FROM tbl LIMIT 95,18446744073709551615;
/*若 LIMIT 只有一个参数，则参数指定应该取回的行数，偏移量默认为 0，即从第一行起*/
```

- SELECT…INTO 可以让查询结果写入到文件中

2. SELECT 关键字的修饰符

- 去重

  ALL 和 DISTINCT 修饰符指定是否对结果集中的行（应该不是某个列）去重。

  ALL 是默认修饰符，即满足要求的所有行都要被取回来。

  DISTINCT 删除重复的行。

3. 子查询的主要优势

- 子查询允许结构化的查询，这样就可以把一个语句的每个部分隔离开。
- 有些操作需要复杂的联合和关联。子查询提供了其它的方法来执行这些操作

4. 加速查询

- 尽可能利用 Doris 的分区分桶作为数据过滤条件，减少数据扫描范围
- 充分利用 Doris 的前缀索引字段作为数据过滤条件加速查询速度

4. UNION

- 只使用 union 关键词和使用 union disitnct 的效果是相同的。由于去重工作是比较耗费内存的，因此使用 union all 操作查询速度会快些，耗费内存会少些。如果用户想对返回结果集进行 order by 和 limit 操作，需要将 union 操作放在子查询中，然后 select from subquery，最后把 subquery 和 order by 放在子查询外面。

```
select * from (select age from student_01 union all select age from student_02) as t1
order by age limit 4;

+-------------+
|     age     |
+-------------+
|      18     |
|      19     |
|      20     |
|      21     |
+-------------+
4 rows in set (0.01 sec)
```

4. JOIN

- 在 inner join 条件里除了支持等值 join，还支持不等值 join，为了性能考虑，推荐使用等值 join。
- 其它 join 只支持等值 join

7.3.1.2 EXPLAIN

7.3.1.2.1 描述

EXPLAIN 语句用于展示 Doris 对于给定的查询所规划的查询计划。Doris 查询优化器的核心目标在于，针对任意给定的查询，生成一个高效且优化的执行计划。该优化器充分利用统计信息、数据特性以及 Doris 本身的功能优势，例如 HASH JOIN、分区和分桶等，来精心制定执行计划。然而，由于路径搜索的固有理论限制以及优化器实现过程中的实际情况，有时生成的执行计划可能无法达到预期的执行效果。为了进一步提升执行性能，我们的首要任务是深入分析优化器当前生成的执行计划。本文将介绍如何使用 EXPLAIN 语句，以便为后续的优化工作奠定坚实基础。

7.3.1.2.2 语法

```
plain text {EXPLAIN | DESC} [VERBOSE] <query_block>
```

7.3.1.2.3 必选参数

> 需要查看执行计划的查询语句。

7.3.1.2.4 可选参数

[VERBOSE]

> 是否展示明细信息。当指定 VERBOSE 时，展示明细信息。否则，展示简略信息。详细信息包括每个算子上的详细信息，算子使用的 tuple 序号，以及对每个 tuple 的详细说明。

7.3.1.2.5 返回结果

基本概念

为了能够更好的理解 EXPLAIN 所展示的信息，这里先介绍几个 DORIS 执行计划的核心概念。

名称	解释
PLAN	执行计划，一个查询会被执行规划器翻译成一个执行计划，之后执行计划会提供给执行引擎执行。
FRAGMENT	执行片段。由于 DORIS 是一个分布式执行引擎。一个完整的执行计划会被切分为多个单机的执行片段。一个 FRAGMENT 表是一个完整的单机执行片段。多个 FRAGMENT 组合在一起，构成一个完整的 PLAN。
PLAN NODE	算子。执行计划的最小单位。一个 FRAGMENT 由多个算子构成。每一个算子负责一个实际的执行逻辑，比如聚合，连接等。

返回结果结构

Doris EXPLAIN 语句的结果是一个完整的 PLAN。PLAN 内部是按照执行顺序从后到前有序排列的 FRAGMENT。FRAGMENT 内部是按照执行顺序从后到前有序排列的算子（PLAN NODE）。

示例如下：

```
+------------------------------------------------+
| Explain String(Nereids Planner)                |
+------------------------------------------------+
| PLAN FRAGMENT 0                                |
|   OUTPUT EXPRS:                                |
|     cnt[#10]                                   |
|     cnt[#11]                                   |
|   PARTITION: UNPARTITIONED                     |
|                                                |
|   HAS_COLO_PLAN_NODE: false                    |
|                                                |
|   VRESULT SINK                                 |
|      MYSQL_PROTOCAL                            |
|                                                |
|   7:VEXCHANGE                                  |
|      offset: 0                                 |
```

```
|      distribute expr lists:                        |
|                                                    |
| PLAN FRAGMENT 1                                    |
|                                                    |
|   PARTITION: RANDOM                                |
|                                                    |
|   HAS_COLO_PLAN_NODE: false                        |
|                                                    |
|   STREAM DATA SINK                                 |
|     EXCHANGE ID: 07                                |
|     UNPARTITIONED                                  |
|                                                    |
|   6:VHASH JOIN(354)                                |
|   |  join op: INNER JOIN(BROADCAST)[]              |
|   |  equal join conjunct: cnt[#7] = cnt[#5]        |
|   |  cardinality=1                                 |
|   |  vec output tuple id: 8                        |
|   |  vIntermediate tuple ids: 7                    |
|   |  hash output slot ids: 5 7                     |
|   |  distribute expr lists:                        |
|   |  distribute expr lists:                        |
|   |                                                |
|   |----4:VEXCHANGE                                 |
|   |       offset: 0                                |
|   |       distribute expr lists:                   |
|   |                                                |
|   5:VEXCHANGE                                      |
|      offset: 0                                     |
|      distribute expr lists:                        |
|                                                    |
| PLAN FRAGMENT 2                                    |
|   ...                                              |
|                                                    |
| PLAN FRAGMENT 3                                    |
|   ...                                              |
+----------------------------------------------------+
```

算子与其孩子节点之间，以虚线连接。当一个算子存在多个孩子时，孩子算子从上之下排布，表示从右至左的关系。以上面的示例为例。6 号算子 VHASH JOIN 的左孩子是 5 号 EXCHANGE 算子，右孩子是 4 号 EXCHANGE 算子。

Fragment 字段说明

名称	解释
PARTITION	展示当前 Fragment 的数据分布情况
HAS_COLO_PLAN_NODE	当前 fragment 中是否存在 colocate 的算子

名称	解释
Sink	fragment 数据输出的方式，具体方式先下表

Sink 方式

名称	解释
STREAM DATA SINK	向下一个 Fragment 输出数据。这里主要包含两行信息。第一行：数据发送给哪个下游的 EXCHANGE NODE 节点。第二行：数据按照何种方式发送 - UNPARTITIONED 下游的每个 instance 都会获得全量的数据。这一般出现在两种情况下。一个是 broadcast join，另外一个是需要单 instance 计算的逻辑，比如全局的 limit，order by 等。- RANDOM 下游的每个 instance 获得随机的一组数据，不同 instance 之间的数据不重复。- HASH_PARTITIONED 以后续列出的 slot 为 key 做 hash，将同一个 hash 分片的数据发送到同一个下游的 instance 中。这多用于 partition hash join，两阶段聚合的第二阶段等算子的上游。
RESULT SINK	向 FE 发送结果数据。第一行，表名发送数据采用的协议。现在有 MySQL 协议和 arrow 协议
OLAP TABLE SINK	向 OLAP 表中写入数据
MultiCastDataSinks	多发算子，下面包含多个 STREAM DATA SINK。每一个 STREAM DATA SINK 都发送全量的数据给下游。

Tuple 信息说明

在使用 VERBOSE 模式时，会输出 Tuple 信息。Tuple 信息描述了一行数据内的 SLOT 信息。包括 SLOT 的类型，nullable 等。

输出的信息包含多个 TupleDescriptor，每个 TupleDescriptor 中又包含多个 SlotDescriptor。示例如下：

```
Tuples:
TupleDescriptor{id=0, tbl=t1}
  SlotDescriptor{id=0, col=c1, colUniqueId=0, type=int, nullable=true, isAutoIncrement=false,
      ↪ subColPath=null}
  SlotDescriptor{id=2, col=c3, colUniqueId=2, type=int, nullable=true, isAutoIncrement=false,
      ↪ subColPath=null}
```

TupleDescriptor

名称	解释
id	tuple descriptor 的 id
tbl	tuple 对应的表，如果没有则留空

SlotDescriptor

名称	解释
id	slot descriptor 的 id

名称	解释
col	slot 对应的列，如果没有则留空
colUniqueId	slot 对应的列的 unique id，如果没有则为 -1
type	slot 的类型
nullable	slot 对应的数据是否可能为 null
isAutoIncrement	是否是自增列
subColPath	列中的子列路径，当前只应用于 variant 类型

算子说明

算子列表

名称	解释
AGGREGATE	聚合算子
ANALYTIC	窗口函数算子
ASSERT NUMBER OF ROWS	检查下游输出行数算子
EXCHANGE	数据交换接收算子
MERGING-EXCHANGE	带排序和限制输出行数功能的数据交换接收算子
HASH JOIN	哈希连接算子
NESTED LOOP JOIN	嵌套循环连接算子
PartitionTopN	分组内数据预过滤算子
REPEAT_NODE	数据重复生成算子
DataGenScanNode	表值函数算子
EsScanNode	ES 表扫描算子
HIVE_SCAN_NODE	Hive 表扫描算子
HUDI_SCAN_NODE	Hudi 表扫描算子
ICEBERG_SCAN_NODE	Iceberg 表扫描算子
PAIMON_SCAN_NODE	Paimon 表扫描算子
JdbcScanNode	Jdbc 表扫描算子
OlapScanNode	Olap 表扫描算子
SELECT	过滤算子
UNION	集合并集算子
EXCEPT	集合差集算子
INTERSECT	集合交集算子
SORT	排序算子
TOP-N	排序并返回前 N 个结果算子
TABLE FUNCTION NODE	表函数算子（lateral view）

通用字段

名称	解释
limit	限制输出行数
offset	输出前偏移的行数
conjuncts	对于当前节点的结果做过滤。在 projections 前执行。

名称	解释
projections	当前算子结束后，再进行的投影操作。在 conjuncts 后执行。
project output tuple id	投影之后的输出 tuple，可以通过 tuple desc 看到具体的数据 tuple 内的 slot 排列
cardinality	优化器预估的行数
distribute expr lists	当前节点的孩子节点的原始数据分布方式
表达式的 slot id	slot id 对应的具体 slot 可以在 verbose 模式中的 tuple 列表中找到。通过此列表，可以查看 slot 的类型和 nullable 属性等信息。表现形式是表达式后的[#5]

AGGREGATE

名称	解释
（聚合阶段）	聚合阶段有前后两个词表示。第一个词有两个可选，update 和 merge。update 表示本地聚合。merge 表示全局聚合。第二个词表示当前数据是否被序列化。serialize 表示数据处在序列化的状态，finalize 表示已经完成最终计算。
STREAMING	只有多阶段聚合的局部聚合算子截断有此标识。表示当前聚合节点可能使用 STREAMING 模式。即透不进行实际计算，直接将输入数据透传给下一阶段的聚合算子。
output	当前聚合算子的输出。所有本地预聚合的函数，都会被冠以 partial 前缀
group by	聚合的 key

ANALYTIC

名称	解释
functions	当前的窗口函数名字
partition by	对应窗口函数中 over 后面的 partition by。开窗表达式。
order by	窗内排序的表达式和排序方式
window	窗口范围

ASSERT NUMBER OF ROWS

名称	解释
EQ	下游的输出必须满足等于此约束的行数

HASH JOIN

名称	解释
join op	连接的类型
equal join conjunct	连接条件中的等值条件
other join predicates	连接条件中，除等值条件外的其他条件

名称	解释
mark join predicates	mark join 所使用的条件
other predicates	在 join 执行后的过滤谓词
runtime filters	生成的 runtime filter
output slot ids	最终输出的 slot 列表
hash output slot ids	hash 连接执行后，执行其他连接条件前，输出的 slot 列表
isMarkJoin	是否为 mark join

NESTED LOOP JOIN

名称	解释
join op	连接的类型
join conjuncts	连接的条件
mark join predicates	mark join 所使用的条件
predicates	在 join 执行后的过滤谓词
runtime filters	生成的 runtime filter
output slot ids	最终输出的 slot 列表
isMarkJoin	是否为 mark join

PartitionTopN

名称	解释
functions	应用分组过滤优化的窗口函数
has global limit	是否有全局的 limit 行数限制
partition limit	分组内的 limit 行数
partition topn phase	当前阶段：TWO_PHASE_GLOBAL_PTOPN：global 阶段，数据按照 partition key shuffle 之后执行 TWO_PHASE_LOCAL_PTOPN：local 阶段，数据按照 partition key shuffle 之前执行

REPEAT_NODE

名称	解释
repeat	每一行数据会生成重复生成多少行，以及其对应的聚合列 slot id 列表
exprs	数据重复后输出数据的表达式列表

DataGenScanNode

名称	解释
table value function	表函数名字

EsScanNode

名称	解释
SORT COLUMN	返回结果排序列
LOCAL_PREDICATES	在 Doris 内执行的过滤条件
REMOTE_PREDICATES	在 ES 内执行的过滤条件
ES index/type	查询的 ES 的 index 和 type

HIVE_SCAN_NODE

名称	解释
inputSplitNum	扫描的分段数量
totalFileSize	扫描的总文件大小
scanRanges	扫描分段信息
partition	扫描分区数
backends	每个 BE 需要扫描的具体数据信息
cardinality	优化器预估的扫描行数
avgRowSize	优化器预估的每行数据平均大小
numNodes	当前算子使用的 BE 数量
pushdown agg	下压到扫描中的聚合计算

HUDI_SCAN_NODE

名称	解释
inputSplitNum	扫描的分段数量
totalFileSize	扫描的总文件大小
scanRanges	扫描分段信息
partition	扫描分区数
backends	每个 BE 需要扫描的具体数据信息
cardinality	优化器预估的扫描行数
avgRowSize	优化器预估的每行数据平均大小
numNodes	当前算子使用的 BE 数量
pushdown agg	下压到扫描中的聚合计算
hudiNativeReadSplits	使用 native 方式读取的分片数量

ICEBERG_SCAN_NODE

名称	解释
inputSplitNum	扫描的分段数量
totalFileSize	扫描的总文件大小
scanRanges	扫描分段信息
partition	扫描分区数
backends	每个 BE 需要扫描的具体数据信息

名称	解释
cardinality	优化器预估的扫描行数
avgRowSize	优化器预估的每行数据平均大小
numNodes	当前算子使用的 BE 数量
pushdown agg	下压到扫描中的聚合计算
icebergPredicatePushdown	下压给 iceberg api 的过滤条件

PAIMON_SCAN_NODE

名称	解释
inputSplitNum	扫描的分段数量
totalFileSize	扫描的总文件大小
scanRanges	扫描分段信息
partition	扫描分区数
backends	每个 BE 需要扫描的具体数据信息
cardinality	优化器预估的扫描行数
avgRowSize	优化器预估的每行数据平均大小
numNodes	当前算子使用的 BE 数量
pushdown agg	下压到扫描中的聚合计算
paimonNativeReadSplits	使用 native 方式读取的分片数量

JdbcScanNode

名称	解释
TABLE	扫描的 JDBC 侧的表名
QUERY	扫描使用的查询语句

OlapScanNode

名称	解释
TABLE	当前算子扫描的表。表名后面的括号，表明当前命中的同步物化视图的名字
SORT INFO	规划出 SCAN 预排序时，有此字段。表明 SCAN 输出采用了局部预排序，和预截断。
SORT LIMIT	规划出 SCAN 预排序时，有此字段。表明预截断的截断数据长度。
TOPN OPT	规划出 TOP-N Runtime Filter 时，有此字段。
PREAGGREGATION	是否开启预聚合，聚合模型和主键模型 MOR 需要关注此字段。当为 ON 时，表名存储层数据以满足上层需求，不需要执行额外的聚合操作。为 OFF 时，会执行额外的聚合操作。
partitions	当前扫描的 PARTITION 个数，总 PARTITION 个数，以及扫描的 PARTITION 名字列表
tablets	扫描的 TABLET 个数和表的总 TABLET 个数
tabletList	扫描的 TABLET 的列表
avgRowSize	优化器预估的每行数据大小

名称	解释
numNodes	当前扫描被分配到的 BE 的个数
pushAggOp	通过读取 zonemap 元数据返回结果。支持 MIN，MAX，COUNT 三个聚合信息

UNION

名称	解释
constant exprs	常量表达式列表，输出中将包含这些常量表达式
child exprs	孩子的输出通过此表达式列表投影后，作为集合算子的输入

EXCEPT

名称	解释
child exprs	孩子的输出通过此表达式列表投影后，作为集合算子的输入

INTERSECT

名称	解释
child exprs	孩子的输出通过此表达式列表投影后，作为集合算子的输入

SORT

名称	解释
order by	排序的键，以及具体的排序顺序

TABLE FUNCTION NODE

名称	解释
table function	使用的 table function 的名字
lateral view tuple id	新生成的列对应的 tuple 的 id
output slot id	当前节点经过列裁剪后输出的列的 slot 列表

TOP-N

名称	解释
order by	排序的键，以及具体的排序顺序
TOPN OPT	命中 topn runtime filter 优化时有此字段
OPT TWO PHASE	命中 topn 延迟物化时，有此字段

7.3.2 数据修改

7.3.2.1 DML

7.3.2.1.1 INSERT

描述

该语句是完成数据插入操作。

```
INSERT INTO table_name
    [ PARTITION (p1, ...) ]
    [ WITH LABEL label]
    [ (column [, ...]) ]
    [ [ hint [, ...] ] ]
    { VALUES ( { expression | DEFAULT } [, ...] ) [, ...] | query }
```

Parameters

> tablet_name: 导入数据的目的表。可以是 db_name.table_name 形式
>
> partitions: 指定待导入的分区，必须是 table_name 中存在的分区，多个分区名称用逗号分隔
>
> label: 为 Insert 任务指定一个 label
>
> column_name: 指定的目的列，必须是 table_name 中存在的列
>
> expression: 需要赋值给某个列的对应表达式
>
> DEFAULT: 让对应列使用默认值
>
> query: 一个普通查询，查询的结果会写入到目标中
>
> hint: 用于指示 INSERT 执行行为的一些指示符。目前 hint 有三个可选值/*+ STREAMING */、/*+ SHUFFLE */或/*+ NOSHUFFLE */ 1. STREAMING：目前无实际作用，只是为了兼容之前的版本，因此保留。(之前的版本加上这个 hint 会返回 label，现在默认都会返回 label) 2. SHUFFLE：当目标表是分区表，开启这个 hint 会进行 repartiiton。3. NOSHUFFLE：即使目标表是分区表，也不会进行 repartiiton，但会做一些其他操作以保证数据正确落到各个分区中。

对于开启了 merge-on-write 的 Unique 表，还可以使用 insert 语句进行部分列更新的操作。要使用 insert 语句进行部分列更新，需要将会话变量 enable_unique_key_partial_update 的值设置为 true(该变量默认值为 false，即默认无法通过 insert 语句进行部分列更新)。进行部分列更新时，插入的列必须至少包含所有的 Key 列，同时指定需要更新的列。如果插入行 Key 列的值在原表中存在，则将更新具有相同 key 列值那一行的数据。如果插入行 Key 列的值在原表中不存在，则将向表中插入一条新的数据，此时 insert 语句中没有指定的列必须有默认值或可以为 null，这些缺失列会首先尝试用默认值填充，如果该列没有默认值，则尝试使用 null 值填充，如果该列不能为 null，则本次插入失败。

需要注意的是，控制 insert 语句是否开启严格模式的会话变量enable_insert_strict的默认值为 true，即 insert 语句默认开启严格模式，而在严格模式下进行部分列更新不允许更新不存在的 key。所以，在使用 insert 语句

进行部分列更新的时候如果希望能插入不存在的 key，需要在enable_unique_key_partial_update设置为 true 的基础上同时将enable_insert_strict设置为 false。

注意：

当前执行 INSERT 语句时，对于有不符合目标表格式的数据，默认的行为是过滤，比如字符串超长等。但是对于有要求数据不能够被过滤的业务场景，可以通过设置会话变量 enable_insert_strict 为 true 来确保当有数据被过滤掉的时候，INSERT 不会被执行成功。

示例

test 表包含两个列c1, c2。

1. 向test表中导入一行数据

```
INSERT INTO test VALUES (1, 2);
INSERT INTO test (c1, c2) VALUES (1, 2);
INSERT INTO test (c1, c2) VALUES (1, DEFAULT);
INSERT INTO test (c1) VALUES (1);
```

其中第一条、第二条语句是一样的效果。在不指定目标列时，使用表中的列顺序来作为默认的目标列。第三条、第四条语句表达的意思是一样的，使用c2列的默认值，来完成数据导入。

2. 向test表中一次性导入多行数据

```
INSERT INTO test VALUES (1, 2), (3, 2 + 2);
INSERT INTO test (c1, c2) VALUES (1, 2), (3, 2 * 2);
INSERT INTO test (c1) VALUES (1), (3);
INSERT INTO test (c1, c2) VALUES (1, DEFAULT), (3, DEFAULT);
```

其中第一条、第二条语句效果一样，向test表中一次性导入两条数据第三条、第四条语句效果已知，使用c2列的默认值向test表中导入两条数据

3. 向 test 表中导入一个查询语句结果

```
INSERT INTO test SELECT * FROM test2;
INSERT INTO test (c1, c2) SELECT * from test2;
```

4. 向 test 表中导入一个查询语句结果，并指定 partition 和 label

```
INSERT INTO test PARTITION(p1, p2) WITH LABEL `label1` SELECT * FROM test2;
INSERT INTO test WITH LABEL `label1` (c1, c2) SELECT * from test2;
```

关键词

```
INSERT
```

最佳实践

1. 查看返回结果

INSERT 操作是一个同步操作，返回结果即表示操作结束。用户需要根据返回结果的不同，进行对应的处理。

1. 执行成功，结果集为空

   如果 insert 对应 select 语句的结果集为空，则返回如下：

```
mysql> insert into tbl1 select * from empty_tbl;
Query OK, 0 rows affected (0.02 sec)
```

`Query OK` 表示执行成功。`0 rows affected` 表示没有数据被导入。

2. 执行成功，结果集不为空

   在结果集不为空的情况下。返回结果分为如下几种情况：

   1. Insert 执行成功并可见：

```
mysql> insert into tbl1 select * from tbl2;
Query OK, 4 rows affected (0.38 sec)
{'label':'insert_8510c568-9eda-4173-9e36-6adc7d35291c', 'status':'visible', 'txnId':'
    ↪ 4005'}

mysql> insert into tbl1 with label my_label1 select * from tbl2;
Query OK, 4 rows affected (0.38 sec)
{'label':'my_label1', 'status':'visible', 'txnId':'4005'}

mysql> insert into tbl1 select * from tbl2;
Query OK, 2 rows affected, 2 warnings (0.31 sec)
{'label':'insert_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'visible', 'txnId':'
    ↪ 4005'}

mysql> insert into tbl1 select * from tbl2;
Query OK, 2 rows affected, 2 warnings (0.31 sec)
{'label':'insert_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'committed', 'txnId':'
    ↪ 4005'}
```

`Query OK` 表示执行成功。`4 rows affected` 表示总共有 4 行数据被导入。`2 warnings` 表示被过滤的行数。

同时会返回一个 `json` 串：

```
{'label':'my_label1', 'status':'visible', 'txnId':'4005'}
{'label':'insert_f0747f0e-7a35-46e2-affa-13a235f4020d', 'status':'committed', 'txnId
    ↪ ':'4005'}
{'label':'my_label1', 'status':'visible', 'txnId':'4005', 'err':'some other error'}
```

`label` 为用户指定的 label 或自动生成的 label。Label 是该 Insert Into 导入作业的标识。
↪ 每个导入作业，都有一个在单 database 内部唯一的 Label。

`status` 表示导入数据是否可见。如果可见，显示 `visible`，如果不可见，显示 `committed`。

`txnId` 为这个 insert 对应的导入事务的 id。

`err` 字段会显示一些其他非预期错误。

当需要查看被过滤的行时，用户可以通过如下语句

```
show load where label="xxx";
```

返回结果中的 URL 可以用于查询错误的数据，具体见后面 **查看错误行** 小结。

数据不可见是一个临时状态，这批数据最终是一定可见的

可以通过如下语句查看这批数据的可见状态：

```
show transaction where id=4005;
```

返回结果中的 `TransactionStatus` 列如果为 `visible`，则表述数据可见。

3. 执行失败

   执行失败表示没有任何数据被成功导入，并返回如下：

```
mysql> insert into tbl1 select * from tbl2 where k1 = "a";
ERROR 1064 (HY000): all partitions have no load data. url: http://10.74.167.16:8042/api/_
    ↪ load_error_log?file=__shard_2/error_log_insert_stmt_ba8bb9e158e4879-
    ↪ ae8de8507c0bf8a2_ba8bb9e158e4879_ae8de8507c0bf8a2
```

其中 `ERROR 1064 (HY000): all partitions have no load data` 显示失败原因。后面的 url
↪ 可以用于查询错误的数据：

```
show load warnings on "url";
```

可以查看到具体错误行。

2. 超时时间

INSERT 操作的超时时间由max(insert_timeout, query_timeout)控制。二者均为环境变量，insert_timeout 默认为 4 小时，query_timeout 默认为 5 分钟。超时则作业会被取消。引入insert_timeout的原因是让 insert 语句默认拥有较长的超时时间，使导入任务不受普通查询默认较短的超时时间的影响。

3. Label 和原子性

INSERT 操作同样能够保证导入的原子性，可以参阅导入事务和原子性 文档。

当需要使用 CTE(Common Table Expressions) 作为 insert 操作中的查询部分时，必须指定 WITH LABEL 和 column 部分。

4. 过滤阈值

与其他导入方式不同，INSERT 操作不能指定过滤阈值（max_filter_ratio）。默认的过滤阈值为 1，即素有错误行都可以被忽略。

对于有要求数据不能够被过滤的业务场景，可以通过设置会话变量 enable_insert_strict 为 true 来确保当有数据被过滤掉的时候，INSERT 不会被执行成功。

5. 性能问题

不建议使用 VALUES 方式进行单行的插入。如果必须这样使用，请将多行数据合并到一个 INSERT 语句中进行批量提交。

7.3.2.1.2 INSERT OVERWRITE

描述

该语句的功能是重写表或表的某些分区

```
INSERT OVERWRITE table table_name
    [ PARTITION (p1, ... | *) ]
    [ WITH LABEL label]
    [ (column [, ...]) ]
    [ [ hint [, ...] ] ]
    { VALUES ( { expression | DEFAULT } [, ...] ) [, ...] | query }
```

Parameters

table_name: 需要重写的目的表。这个表必须存在。可以是 db_name.table_name 形式

partitions: 需要重写的目标分区，支持两种形式：

1. 分区名。必须是 table_name 中存在的分区，多个分区名称用逗号分隔。
2. 星号 (*)。开启自动检测分区功能。写入操作将会自动检测数据所涉及的分区，并覆写这些分区。

label: 为 Insert 任务指定一个 label

column_name: 指定的目的列，必须是 table_name 中存在的列

expression: 需要赋值给某个列的对应表达式

DEFAULT: 让对应列使用默认值

query: 一个普通查询，查询的结果会重写到目标中

hint: 用于指示 INSERT 执行行为的一些指示符。目前 hint 有三个可选值/*+ STREAMING */、/*+ SHUFFLE */或/*+ NOSHUFFLE */

1. STREAMING：目前无实际作用，只是为了兼容之前的版本，因此保留。(之前的版本加上这个 hint 会返回 label，现在默认都会返回 label)
2. SHUFFLE：当目标表是分区表，开启这个 hint 会进行 repartiiton。
3. NOSHUFFLE：即使目标表是分区表，也不会进行 repartiiton，但会做一些其他操作以保证数据正确落到各个分区中。

注意：

1. 在当前版本中，会话变量 enable_insert_strict 默认为 true，如果执行 INSERT OVERWRITE 语句时，对于有不符合目标表格式的数据被过滤掉的话会重写目标表失败 (比如重写分区时，不满足所有分区条件的数据会被过滤)。
2. INSERT OVERWRITE 语句会首先创建一个新表，将需要重写的数据插入到新表中，最后原子性的用新表替换旧表并修改名称。因此，在重写表的过程中，旧表中的数据在重写完毕之前仍然可以正常访问。

For Auto Partition Table

如果 INSERT OVERWRITE 的目标表是自动分区表，那么行为受到 Session Variable enable_auto_create_when_overwrite 的控制，具体行为如下：

1. 若未指定 PARTITION (覆写整表)，当 enable_auto_create_when_overwrite 为 true，在覆写整表已有数据的同时，对于没有对应分区的数据，按照该表的自动分区规则创建分区，并容纳这些原本没有对应分区的数据。如果 enable_auto_create_when_overwrite 为 false，未找到分区的数据将累计错误行直到失败。
2. 如果指定了覆写的 PARTITION，那么在此过程中，AUTO PARTITION 表表现得如同普通分区表一样，不满足现有分区条件的数据将被过滤，而非创建新的分区。
3. 若指定 PARTITION 为 partition(*) (自动检测分区并覆写)，当 enable_auto_create_when_overwrite 为 true，对于那些在表中有对应分区的数据，覆写它们对应的分区，其他已有分区不变。同时，对于没有对应分区的数据，按照该表的自动分区规则创建分区，并容纳这些原本没有对应分区的数据。如果 enable_auto_create_when_overwrite 为 false，未找到分区的数据将累计错误行直到失败。

enable_auto_create_when_overwrite 自 3.0.3 引入，在没有 enable_auto_create_when_overwrite 的版本，行为如同该变量值为 false。

速查结论如下：

1. 对于开启 enable_auto_create_when_overwrite 的自动分区表：

	覆写的分区	清空其他分区	无分区的数据自动创建
无标识（全表）	所有	√	√
指定分区	写明的分区	×	×
partition(*)	数据所属的分区	×	√

2. 对于普通表、关闭 enable_auto_create_when_overwrite 的自动分区表：

	覆写的分区	清空其他分区	无分区的数据自动创建
无标识（全表）	所有	√	×
指定分区	写明的分区	×	×
partition(*)	数据所属的分区	×	×

示例如下：

```
mysql> create table auto_list(
    ->              k0 varchar null
    ->          )
    ->          auto partition by list (k0)
    ->          (
    ->              PARTITION p1 values in (("Beijing"), ("BEIJING")),
    ->              PARTITION p2 values in (("Shanghai"), ("SHANGHAI")),
    ->              PARTITION p3 values in (("xxx"), ("XXX")),
    ->              PARTITION p4 values in (("list"), ("LIST")),
    ->              PARTITION p5 values in (("1234567"), ("7654321"))
    ->          )
    ->          DISTRIBUTED BY HASH(`k0`) BUCKETS 1
    ->          properties("replication_num" = "1");
Query OK, 0 rows affected (0.14 sec)

mysql> insert into auto_list values ("Beijing"),("Shanghai"),("xxx"),("list"),("1234567");
Query OK, 5 rows affected (0.22 sec)

mysql> insert overwrite table auto_list partition(*) values ("BEIJING"), ("new1");
Query OK, 2 rows affected (0.28 sec)

mysql> select * from auto_list;
```

```
+----------+ --- p1 被覆写，new1 得到了新分区，其他分区数据未变
| k0       |
+----------+
| 1234567  |
| BEIJING  |
| list     |
| xxx      |
| new1     |
| Shanghai |
+----------+
6 rows in set (0.48 sec)

mysql> insert overwrite table auto_list values ("SHANGHAI"), ("new2");
Query OK, 2 rows affected (0.17 sec)

mysql> select * from auto_list;
+----------+ --- 整表原有数据被覆写，同时 new2 得到了新分区
| k0       |
+----------+
| new2     |
| SHANGHAI |
+----------+
2 rows in set (0.15 sec)
```

示例

假设有test 表。该表包含两个列c1, c2，两个分区p1,p2。建表语句如下所示

```
CREATE TABLE IF NOT EXISTS test (
  `c1` int NOT NULL DEFAULT "1",
  `c2` int NOT NULL DEFAULT "4"
) ENGINE=OLAP
UNIQUE KEY(`c1`)
PARTITION BY LIST (`c1`)
(
PARTITION p1 VALUES IN ("1","2","3"),
PARTITION p2 VALUES IN ("4","5","6")
)
DISTRIBUTED BY HASH(`c1`) BUCKETS 3
PROPERTIES (
  "replication_allocation" = "tag.location.default: 1",
  "in_memory" = "false",
  "storage_format" = "V2"
);
```

Overwrite Table

1. VALUES 的形式重写test表

```
# 单行重写
INSERT OVERWRITE table test VALUES (1, 2);
INSERT OVERWRITE table test (c1, c2) VALUES (1, 2);
INSERT OVERWRITE table test (c1, c2) VALUES (1, DEFAULT);
INSERT OVERWRITE table test (c1) VALUES (1);
# 多行重写
INSERT OVERWRITE table test VALUES (1, 2), (3, 2 + 2);
INSERT OVERWRITE table test (c1, c2) VALUES (1, 2), (3, 2 * 2);
INSERT OVERWRITE table test (c1, c2) VALUES (1, DEFAULT), (3, DEFAULT);
INSERT OVERWRITE table test (c1) VALUES (1), (3);
```

- 第一条语句和第二条语句的效果一致，重写时如果不指定目标列，会使用表中的列顺序来作为默认的目标列。重写成功后表test中只有一行数据。
- 第三条语句和第四条语句的效果一致，没有指定的列c2会使用默认值 4 来完成数据重写。重写成功后表test中只有一行数据。
- 第五条语句和第六条语句的效果一致，在语句中可以使用表达式（如2+2，2*2），执行语句的时候会计算出表达式的结果再重写表test。重写成功后表test中有两行数据。
- 第七条语句和第八条语句的效果一致，没有指定的列c2会使用默认值 4 来完成数据重写。重写成功后表test中有两行数据。

2. 查询语句的形式重写test表，表test2和表test的数据格式需要保持一致，如果不一致会触发数据类型的隐式转换

```
INSERT OVERWRITE table test SELECT * FROM test2;
INSERT OVERWRITE table test (c1, c2) SELECT * from test2;
```

- 第一条语句和第二条语句的效果一致，该语句的作用是将数据从表test2中取出，使用取出的数据重写表test。重写成功后表test中的数据和表test2中的数据保持一致。

3. 重写 test 表并指定 label

```
INSERT OVERWRITE table test WITH LABEL `label1` SELECT * FROM test2;
INSERT OVERWRITE table test WITH LABEL `label2` (c1, c2) SELECT * from test2;
```

- 使用 label 会将此任务封装成一个异步任务，执行语句之后，相关操作都会异步执行，用户可以通过SHOW LOAD;命令查看此label导入作业的状态。需要注意的是 label 具有唯一性。

Overwrite Table Partition

使用 INSERT OVERWRITE 重写分区时，实际我们是将如下三步操作封装为一个事务并执行，如果中途失败，已进行的操作将会回滚：

1. 假设指定重写分区 p1，首先创建一个与重写的目标分区结构相同的空临时分区 pTMP
2. 向 pTMP 中写入数据
3. 使用 pTMP 原子替换 p1 分区

举例如下：

1. VALUES 的形式重写test表分区P1和p2

```
# 单行重写
INSERT OVERWRITE table test PARTITION(p1,p2) VALUES (1, 2);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1, c2) VALUES (1, 2);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1, c2) VALUES (1, DEFAULT);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1) VALUES (1);
# 多行重写
INSERT OVERWRITE table test PARTITION(p1,p2) VALUES (1, 2), (4, 2 + 2);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1, c2) VALUES (1, 2), (4, 2 * 2);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1, c2) VALUES (1, DEFAULT), (4, DEFAULT);
INSERT OVERWRITE table test PARTITION(p1,p2) (c1) VALUES (1), (4);
```

以上语句与重写表不同的是，它们都是重写表中的分区。分区可以一次重写一个分区也可以一次重写多个分区
↪ 。需要注意的是，只有满足对应分区过滤条件的数据才能够重写成功。
↪ 如果重写的数据中有数据不满足其中任意一个分区，那么本次重写会失败。一个失败的例子如下所示

```
INSERT OVERWRITE table test PARTITION(p1,p2) VALUES (7, 2);
```

以上语句重写的数据`c1=7`分区`p1`和`p2`的条件都不满足，因此会重写失败。

2. 查询语句的形式重写test表分区P1和p2，表test2和表test的数据格式需要保持一致，如果不一致会触发数据类型的隐式转换

```
INSERT OVERWRITE table test PARTITION(p1,p2) SELECT * FROM test2;
INSERT OVERWRITE table test PARTITION(p1,p2) (c1, c2) SELECT * from test2;
```

3. 重写 test 表分区P1和p2并指定 label

```
INSERT OVERWRITE table test PARTITION(p1,p2) WITH LABEL `label3` SELECT * FROM test2;
INSERT OVERWRITE table test PARTITION(p1,p2) WITH LABEL `label4` (c1, c2) SELECT * from test2
    ↪ ;
```

Overwrite Auto Detect Partition

当 INSERT OVERWRITE 命令指定的 PARTITION 子句为 PARTITION(*) 时，此次覆写将会自动检测分区数据所在的分区。例如：

```
mysql> create table test(
    -> k0 int null
    -> )
    -> partition by range (k0)
    -> (
    -> PARTITION p10 values less than (10),
    -> PARTITION p100 values less than (100),
    -> PARTITION pMAX values less than (maxvalue)
    -> )
    -> DISTRIBUTED BY HASH(`k0`) BUCKETS 1
    -> properties("replication_num" = "1");
Query OK, 0 rows affected (0.11 sec)

mysql> insert into test values (1), (2), (15), (100), (200);
Query OK, 5 rows affected (0.29 sec)

mysql> select * from test order by k0;
+------+
| k0   |
+------+
|    1 |
|    2 |
|   15 |
|  100 |
|  200 |
+------+
5 rows in set (0.23 sec)

mysql> insert overwrite table test partition(*) values (3), (1234);
Query OK, 2 rows affected (0.24 sec)

mysql> select * from test order by k0;
+------+
| k0   |
+------+
|    3 |
|   15 |
| 1234 |
+------+
3 rows in set (0.20 sec)
```

可以看到，数据 3、1234 所在的分区 p10 和 pMAX 中的全部数据均被覆写，而 p100 分区未发生变化。该操作可以理解为 INSERT OVERWRITE 操作时通过 PARTITION 子句指定覆写特定分区的语法糖，它的实现原理与指定重写特定分区相同。通过 PARTITION(*) 的语法，在覆写大量分区数据时我们可以免于手动填写全部分区名的繁

琐。

关键词

INSERT OVERWRITE, OVERWRITE, AUTO DETECT

7.3.2.1.3 SHOW LAST INSERT

描述

该语法用于查看在当前 session 连接中，最近一次 insert 操作的结果

语法：

```
SHOW LAST INSERT
```

返回结果示例：

```
    TransactionId: 64067
            Label: insert_ba8f33aea9544866-8ed77e2844d0cc9b
         Database: default_cluster:db1
            Table: t1
TransactionStatus: VISIBLE
       LoadedRows: 2
     FilteredRows: 0
```

说明：

- TransactionId：事务 id
- Label：insert 任务对应的 label
- Database：insert 对应的数据库
- Table：insert 对应的表
- TransactionStatus：事务状态
  - PREPARE：准备阶段
  - PRECOMMITTED：预提交阶段
  - COMMITTED：事务成功，但数据不可见
  - VISIBLE：事务成功且数据可见
  - ABORTED：事务失败
- LoadedRows：导入的行数
- FilteredRows：被过滤的行数

示例

关键词

```
SHOW, LAST, INSERT
```

最佳实践

7.3.2.1.4 UPDATE

描述

该语句是为进行对数据进行更新的操作，UPDATE 语句目前仅支持 UNIQUE KEY 模型。

UPDATE 操作目前只支持更新 Value 列，Key 列的更新可参考使用 FlinkCDC 更新 Key 列。

语法

```
[cte]
UPDATE target_table [table_alias]
    SET assignment_list
    [ FROM additional_tables]
    WHERE condition
```

Required Parameters

- target_table: 待更新数据的目标表。可以是 ‘db_name.table_name’ 形式
- assignment_list: 待更新的目标列，形如 ‘col_name = value, col_name = value’ 格式
- WHERE condition: 期望更新的条件，一个返回 true 或者 false 的表达式即可

Optional Parameters

- cte: 通用表达式。可以是 ‘WITH a AS SELECT * FROM tbl’ 形式
- table_alias: 表的别名
- FROM additional_tables: 指定一个或多个表，用于选中更新的行，或者获取更新的值。注意，如需要在此列表中再次使用目标表，需要为其显式指定别名。

Note

当前 UPDATE 语句仅支持在 UNIQUE KEY 模型上的行更新。

示例

test 表是一个 unique 模型的表，包含：k1, k2, v1, v2 四个列。其中 k1, k2 是 key，v1, v2 是 value，聚合方式是 Replace。

1. 将 ‘test’ 表中满足条件 k1 =1 , k2 =2 的 v1 列更新为 1

```
UPDATE test SET v1 = 1 WHERE k1=1 and k2=2;
```

2. 将 ‘test’ 表中 k1=1 的列的 v1 列自增 1

```
UPDATE test SET v1 = v1+1 WHERE k1=1;
```

3. 使用t2和t3表连接的结果，更新t1

```
-- 创建 t1, t2, t3 三张表
CREATE TABLE t1
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
UNIQUE KEY (id)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1', "function_column.sequence_col" = "c4");

CREATE TABLE t2
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

CREATE TABLE t3
  (id INT)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

-- 插入数据
INSERT INTO t1 VALUES
  (1, 1, '1', 1.0, '2000-01-01'),
  (2, 2, '2', 2.0, '2000-01-02'),
  (3, 3, '3', 3.0, '2000-01-03');

INSERT INTO t2 VALUES
  (1, 10, '10', 10.0, '2000-01-10'),
  (2, 20, '20', 20.0, '2000-01-20'),
  (3, 30, '30', 30.0, '2000-01-30'),
  (4, 4, '4', 4.0, '2000-01-04'),
  (5, 5, '5', 5.0, '2000-01-05');

INSERT INTO t3 VALUES
  (1),
  (4),
  (5);

-- 更新 t1
UPDATE t1
  SET t1.c1 = t2.c1, t1.c3 = t2.c3 * 100
  FROM t2 INNER JOIN t3 ON t2.id = t3.id
  WHERE t1.id = t2.id;
```

预期结果为，更新了t1表id为1的列

```
+----+----+----+--------+------------+
| id | c1 | c2 | c3     | c4         |
```

```
+----+----+----+--------+------------+
| 1  | 10 | 1  | 1000.0 | 2000-01-01 |
| 2  | 2  | 2  |    2.0 | 2000-01-02 |
| 3  | 3  | 3  |    3.0 | 2000-01-03 |
+----+----+----+--------+------------+
```

4. 使用 cte 更新表

```
create table orders(
    o_orderkey bigint,
    o_totalprice decimal(15, 2)
) unique key(o_orderkey)
distributed by hash(o_orderkey) buckets 1
properties (
    "replication_num" = "1"
);

insert into orders values
(1, 34.1),
(2, 432.8);

create table lineitem(
    l_linenumber int,
    o_orderkey bigint,
    l_discount  decimal(15, 2)
) unique key(l_linenumber)
distributed by hash(l_linenumber) buckets 1
properties (
    "replication_num" = "1"
);

insert into lineitem values
(1, 1, 1.23),
(2, 1, 3.21),
(3, 2, 18.08),
(4, 2, 23.48);

with discount_orders as (
    select * from orders
    where o_totalprice > 100
)
update lineitem  set l_discount = l_discount*0.9
from discount_orders
where lineitem.o_orderkey = discount_orders.o_orderkey;
```

关键词

```
UPDATE
```

7.3.2.1.5 DELETE

描述

该语句用于按条件删除指定 table（base index）partition 中的数据。

该操作会同时删除和此 base index 相关的 rollup index 的数据。

语法

语法一：该语法只能指定过滤谓词

```
DELETE FROM table_name [table_alias] [PARTITION partition_name | PARTITIONS (partition_name [,
    ↪ partition_name])]
WHERE
column_name op { value | value_list } [ AND column_name op { value | value_list } ...];
```

语法二：该语法只能在 UNIQUE KEY 模型表上使用

```
[cte]
DELETE FROM table_name [table_alias]
    [PARTITION partition_name | PARTITIONS (partition_name [, partition_name])]
    [USING additional_tables]
    WHERE condition
```

Required Parameters

- table_name: 指定需要删除数据的表
- column_name: 属于 table_name 的列
- op: 逻辑比较操作符，可选类型包括：=, >, <, >=, <=, !=, in, not in
- value | value_list: 做逻辑比较的值或值列表
- WHERE condition: 指定一个用于选择删除行的条件

Optional Parameters

- cte: 通用表达式。可以是 'WITH a AS SELECT * FROM tbl' 形式
- PARTITION partition_name | PARTITIONS (partition_name [, partition_name]): 指定执行删除数据的分区名，如果表不存在此分区，则报错
- table_alias: 表的别名
- USING additional_tables: 如果需要在 WHERE 语句中使用其他的表来帮助识别需要删除的行，则可以在 USING 中指定这些表或者查询。

Note

1. 使用聚合类的表模型（AGGREGATE、UNIQUE）只能指定 key 列上的条件。

2. 当选定的 key 列不存在于某个 rollup 中时，无法进行 delete。
3. 语法一中，条件之间只能是“与”的关系。若希望达成“或”的关系，需要将条件分写在两个 DELETE 语句中。
4. 语法一中，如果为分区表，需要指定分区，如果不指定，doris 会从条件中推断出分区。两种情况下，doris 无法从条件中推断出分区：1) 条件中不包含分区列；2) 分区列的 op 为 not in。如果分区表不是 Unique 表，当分区表未指定分区，或者无法从条件中推断分区的时候，需要设置会话变量 delete_without_partition 为 true，此时 delete 会应用到所有分区。
5. 该语句可能会降低执行后一段时间内的查询效率。影响程度取决于语句中指定的删除条件的数量。指定的条件越多，影响越大。

示例

1. 删除 my_table partition p1 中 k1 列值为 3 的数据行

```
DELETE FROM my_table PARTITION p1
    WHERE k1 = 3;
```

2. 删除 my_table partition p1 中 k1 列值大于等于 3 且 k2 列值为 “abc” 的数据行

```
DELETE FROM my_table PARTITION p1
WHERE k1 >= 3 AND k2 = "abc";
```

3. 删除 my_table partition p1, p2 中 k1 列值大于等于 3 且 k2 列值为 “abc” 的数据行

```
DELETE FROM my_table PARTITIONS (p1, p2)
WHERE k1 >= 3 AND k2 = "abc";
```

4. 使用t2和t3表连接的结果，删除t1中的数据，删除的表只支持 unique 模型

```
-- 创建 t1, t2, t3 三张表
CREATE TABLE t1
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
UNIQUE KEY (id)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1', "function_column.sequence_col" = "c4");

CREATE TABLE t2
  (id INT, c1 BIGINT, c2 STRING, c3 DOUBLE, c4 DATE)
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

CREATE TABLE t3
  (id INT)
```

```
DISTRIBUTED BY HASH (id)
PROPERTIES('replication_num'='1');

-- 插入数据
INSERT INTO t1 VALUES
  (1, 1, '1', 1.0, '2000-01-01'),
  (2, 2, '2', 2.0, '2000-01-02'),
  (3, 3, '3', 3.0, '2000-01-03');

INSERT INTO t2 VALUES
  (1, 10, '10', 10.0, '2000-01-10'),
  (2, 20, '20', 20.0, '2000-01-20'),
  (3, 30, '30', 30.0, '2000-01-30'),
  (4, 4, '4', 4.0, '2000-01-04'),
  (5, 5, '5', 5.0, '2000-01-05');

INSERT INTO t3 VALUES
  (1),
  (4),
  (5);

-- 删除 t1 中的数据
DELETE FROM t1
  USING t2 INNER JOIN t3 ON t2.id = t3.id
  WHERE t1.id = t2.id;
```

预期结果为，删除了t1表id为1的列

```
+----+----+----+--------+------------+
| id | c1 | c2 | c3     | c4         |
+----+----+----+--------+------------+
| 2  | 2  | 2  |    2.0 | 2000-01-02 |
| 3  | 3  | 3  |    3.0 | 2000-01-03 |
+----+----+----+--------+------------+
```

5. 使用 cte 关联删除

```
create table orders(
 o_orderkey bigint,
 o_totalprice decimal(15, 2)
) unique key(o_orderkey)
distributed by hash(o_orderkey) buckets 1
properties (
"replication_num" = "1"
);
```

```
insert into orders values
(1, 34.1),
(2, 432.8);

create table lineitem(
l_linenumber int,
o_orderkey bigint,
l_discount  decimal(15, 2)
) unique key(l_linenumber)
distributed by hash(l_linenumber) buckets 1
properties (
"replication_num" = "1"
);

insert into lineitem values
(1, 1, 1.23),
(2, 1, 3.21),
(3, 2, 18.08),
(4, 2, 23.48);

with discount_orders as (
select * from orders
where o_totalprice > 100
)
delete from lineitem
using discount_orders
where lineitem.o_orderkey = discount_orders.o_orderkey;
```

关键词

```
DELETE
```

最佳实践

7.3.2.1.6 SHOW DELETE

描述

该语句用于展示已执行成功的历史 delete 任务

语法

```
SHOW DELETE [FROM <db_name>]
```

可选参数

: 需要展示的数据库名称

返回值

列名	描述
TableName	执行删除操作的表名。
PartitionName	受删除操作影响的分区名称。
CreateTime	删除操作执行时的时间戳。
DeleteCondition	删除操作使用的条件，指定被删除的行。
State	删除操作的状态。

示例

1. 到 test 库下查看所有历史 delete 任务

```
show delete;
```

```
+---------------+---------------+---------------------+-----------------+----------+
| TableName     | PartitionName | CreateTime          | DeleteCondition | State    |
+---------------+---------------+---------------------+-----------------+----------+
| iceberg_table | *             | 2025-03-14 15:53:32 | id EQ "1"       | FINISHED |
+---------------+---------------+---------------------+-----------------+----------+
```

2. 展示数据库 tpch 的所有历史 delete 任务

```
show delete from tpch;
```

```
+-----------+---------------+---------------------+-------------------+----------+
| TableName | PartitionName | CreateTime          | DeleteCondition   | State    |
+-----------+---------------+---------------------+-------------------+----------+
| customer  | *             | 2025-03-14 15:45:19 | c_custkey EQ "18" | FINISHED |
+-----------+---------------+---------------------+-------------------+----------+
```

7.3.2.2 导入导出

7.3.2.2.1 BROKER LOAD

描述

Broker Load 是 Doris 的数据导入方式，主要用于从远程存储系统（如 HDFS 或 S3）导入大规模数据。它通过 MySQL API 发起，是异步导入方式。导入进度和结果可以通过 SHOW LOAD 查询。

在早期版本中，S3 和 HDFS Load 依赖于 Broker 进程，但随着版本优化，现在直接从数据源读取，不再依赖额外的 Broker 进程。尽管如此，由于语法相似，S3 Load、 HDFS Load 和 Broker Load 都被统称为 Broker Load。

语法

```
LOAD LABEL [<db_name>.]<load_label>
(
[ { MERGE | APPEND | DELETE } ]
DATA INFILE
(
"<file_path>"[, ...]
)
[ NEGATIVE ]
INTO TABLE `<table_name>`
[ PARTITION ( <partition_name> [ , ... ] ) ]
[ COLUMNS TERMINATED BY "<column_separator>" ]
[ LINES TERMINATED BY "<line_delimiter>" ]
[ FORMAT AS "<file_type>" ]
[ COMPRESS_TYPE AS "<compress_type>" ]
[ (<column_list>) ]
[ COLUMNS FROM PATH AS (<column_name> [ , ... ] ) ]
[ SET (<column_mapping>) ]
[ PRECEDING FILTER <predicate> ]
[ WHERE <predicate> ]
[ DELETE ON <expr> ]
[ ORDER BY <source_sequence> ]
[ PROPERTIES ("<key>"="<value>" [, ...] ) ]
)
WITH BROKER "<broker_name>"
(   <broker_properties>
    [ , ... ])
[ PROPERTIES (
    <load_properties>
    [ , ... ]) ]
[COMMENT "<comment>" ];
```

必选参数

1. <db_name> > 指定导入的数据库名。

2. <load_label> > 每个导入任务需要指定一个唯一的 Label，后续可以通过该 Label 查询作业进度。

3. <table_name> > 指定导入任务对应的表。

4.<file_path> > 指定需要导入的文件路径。可以是多个路径，也可以使用通配符。路径最终必须匹配到文件，若只匹配到目录则导入会失败。

5. <broker_name> > 指定需要使用的 Broker 服务名称。比如在公有云 Doris 中。 Broker 服务名称为 bos。

6. <broker_properties> > 指定 broker 所需的信息。这些信息通常被用于 Broker 能够访问远端存储系统。如 BOS 或 HDFS。 >>text > (> "username" = "user", > "password" = "pass", > ... >)>

可选参数

1. merge | append | delete
> 数据合并类型。默认为 append，表示本次导入是普通的追加写操作。merge 和 delete 类型仅适用于 unique key 模型表。merge 类型需要配合 [delete on] 语句使用，以标注 delete flag 列。而 delete 类型则表示本次导入的所有数据皆为删除数据。

2. negative
> 表示“负”导入，这种方式仅针对具有整型 sum 聚合类型的聚合数据表。将导入数据中的 sum 聚合列对应的整型数值取反，用于冲抵错误数据。

3. <partition_name>
> 指定仅导入表的某些分区，比如：partition (p1, p2,…)，其他不在分区范围内的数据会被忽略。

4. <column_separator>
> 指定列分隔符，仅在 CSV 格式下有效，且只能指定单字节分隔符。

5. <line_delimiter>
> 指定行分隔符，仅在 CSV 格式下有效，且只能指定单字节分隔符。

6. <file_type>
> 指定文件格式，支持 csv（默认）、parquet、orc 格式。

7. <compress_type>
> 指定文件压缩类型，支持 gz、bz2、lz4frame。

8. <column_list>
> 指定原始文件中的列顺序。

9. columns from path as (<c1>, <c2>,...)
> 指定从导入文件路径中抽取的列。

10. <column_mapping>
> 指定列的转换函数。

11. preceding filter <predicate>
> 数据先根据 column list 和 columns from path as 拼接为原始数据行，再根据前置过滤条件进行过滤。

12. where <predicate>
> 根据条件对导入数据进行过滤。

13. delete on <expr>
> 配合 merge 导入模式使用，仅适用于 unique key 模型的表。用于指定导入数据中表示删除标志（delete flag）的列及计算关系。

14. <source_sequence>
> 仅适用于 unique key 模型的表。用于指定导入数据中表示 sequence col 的列，主要用于导入时保证数据顺序。

15. properties ("<key>"="<value>",...)
> 指定导入文件格式的参数。适用于 CSV、JSON 等格式。例如，可以指定 json_root、jsonpaths、fuzzy_parse 等参数。
> enclose: 包围符；当 CSV 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为 “,”，包围符为 “ ‘ “，数据为”a,’b,c’ ”，则 “b,c” 会被解析为一个字段。
> 注意：当 enclose 设置为 " 时，trim_double_quotes 一定要设置为 true。

> escape: 转义符。用于转义在字段中出现的与包围符相同的字符。例如数据为“a,‘b,’c’ ”，包围符为“”’，希望“b,’c”被作为一个字段解析，则需要指定单字节转义符，例如“”，然后将数据修改为“a,‘b,’c’ ”。

16. < load_properties > 可选参数如下，并可根据实际环境情况添加。

参数	参数说明
timeout	导入超时时间，默认为 4 小时，单位秒。
max_filter_ratio	最大容忍可过滤（数据不规范等原因）的数据比例，默认零容忍，取值范围为 0 到 1。
exec_mem_limit	导入内存限制，默认为 2GB，单位为字节。
strict_mode	是否对数据进行严格限制，默认为 false。
partial_columns	布尔类型，为 true 时表示使用部分列更新，默认值为 false，仅在表模型为 Unique 且采用 Merge on Write 时设置。
timezone	指定时区，影响一些受时区影响的函数，如 strftime、alignment_timestamp、from_unixtime 等，具体请查阅 时区 文档。如果不指定，则使用“Asia/Shanghai”。
load_parallelism	导入并发度，默认为 1，调大导入并发度会启动多个执行计划同时执行导入任务，加快导入速度。
send_batch_parallelism	设置发送批处理数据的并行度。如果并行度的值超过 BE 配置中的 max_send_batch_parallelism_per_job，则会使用 max_send_batch_parallelism_per_job 的值。
load_to_single_tablet	布尔类型，为 true 时表示支持将数据导入到对应分区的单个 tablet，默认值为 false，作业的任务数取决于整体并发度，仅在导入带有 random 分桶的 OLAP 表时设置。
priority	设置导入任务的优先级，可选 HIGH/NORMAL/LOW，默认为 NORMAL。对于处于 PENDING 状态的导入任务，更高优先级的任务将优先进入 LOADING 状态。
comment	指定导入任务的备注信息。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	对指定的库表的导入权限

举例

1. 从 HDFS 导入一批数据，导入文件 file.txt，按逗号分隔，导入到表 my_table。

```
LOAD LABEL example_db.label1
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file.txt")
    INTO TABLE `my_table`
    COLUMNS TERMINATED BY ","
)
```

```
WITH BROKER hdfs
(
    "username"="hdfs_user",
    "password"="hdfs_password"
);
```

2. 从 HDFS 导入数据，使用通配符匹配两批文件。分别导入到两个表中。使用通配符匹配导入两批文件 file-10* 和 file-20*。分别导入到 my_table1 和my_table2 两张表中。其中 my_table1 指定导入到分区 p1 中，并且将导入源文件中第列和第三列的值 +1 后导入。

```
LOAD LABEL example_db.label2
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file-10*")
    INTO TABLE `my_table1`
    PARTITION (p1)
    COLUMNS TERMINATED BY ","
    (k1, tmp_k2, tmp_k3)
    SET (
        k2 = tmp_k2 + 1,
        k3 = tmp_k3 + 1
    ),
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file-20*")
    INTO TABLE `my_table2`
    COLUMNS TERMINATED BY ","
    (k1, k2, k3)
)
WITH BROKER hdfs
(
    "username"="hdfs_user",
    "password"="hdfs_password"
);
```

3. 从 HDFS 导入一批数据。指定分隔符为 Hive 的默认分隔符 \\x01，并使用通配符 * 指定 data 目录下所有目录的所文件。使用简单认证，同时配置 namenode HA。

```
LOAD LABEL example_db.label3
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/user/doris/data/*/*")
    INTO TABLE `my_table`
    COLUMNS TERMINATED BY "\\x01"
)
WITH BROKER my_hdfs_broker
(
    "username" = "",
```

```
    "password" = "",
    "fs.defaultFS" = "hdfs://my_ha",
    "dfs.nameservices" = "my_ha",
    "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
    "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
    "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
    "dfs.client.failover.proxy.provider.my_ha" = "org.apache.hadoop.hdfs.server.    namenode.
        ↪ ha.ConfiguredFailoverProxyProvider"
);
```

4. 导入 Parquet 格式数据，指定 FORMAT 为 parquet。默认是通过文件后缀判断

```
LOAD LABEL example_db.label4
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/file")
    INTO TABLE `my_table`
    FORMAT AS "parquet"
    (k1, k2, k3)
)
WITH BROKER hdfs
(
    "username"="hdfs_user",
    "password"="hdfs_password"
);
```

5. 导入数据，并提取文件路径中的分区字段。my_table 表中的列为 k1，k2，k3，city，utc_date。其中 hdfs://hdfs_host:hdfs_port/user/doris/data/input/dir/city=beijing 目下包括如下文件：

```
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-01/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-02/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-03/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-04/0000.csv
```

文件中只包含 k1，k2，k3 三列数据，city，utc_date 这两列数据会从文件路径中提取。

```
LOAD LABEL example_db.label10
(
    DATA INFILE("hdfs://hdfs_host:hdfs_port/input/city=beijing/*/*")
    INTO TABLE `my_table`
    FORMAT AS "csv"
    (k1, k2, k3)
    COLUMNS FROM PATH AS (city, utc_date)
)
WITH BROKER hdfs
```

```
(
    "username"="hdfs_user",
    "password"="hdfs_password"
);
```

6. 对待导入数据进行过滤。只有原始数据中，k1 = 1，并且转换后，k1 > k2 的行才会被导入。

```
LOAD LABEL example_db.label6
(
    DATA INFILE("hdfs://host:port/input/file")
    INTO TABLE `my_table`
    (k1, k2, k3)
    SET (
        k2 = k2 + 1
    )
    PRECEDING FILTER k1 = 1
    WHERE k1 > k2
)
WITH BROKER hdfs
(
    "username"="user",
    "password"="pass"
);
```

7. 导入数据，提取文件路径中的时间分区字段，并且时间包含%3A (在 hdfs 路径中，不允许有 ':' ，所有 ':' 会由%3A 替换)

```
LOAD LABEL example_db.label7
(
    DATA INFILE("hdfs://host:port/user/data/*/test.txt")
    INTO TABLE `tbl12`
    COLUMNS TERMINATED BY ","
    (k2,k3)
    COLUMNS FROM PATH AS (data_time)
    SET (
        data_time=str_to_date(data_time, '%Y-%m-%d %H%%3A%i%%3A%s')
    )
)
WITH BROKER hdfs
(
    "username"="user",
    "password"="pass"
);
```

路径下有如下文件：

```
/user/data/data_time=2020-02-17 00%3A00%3A00/test.txt
/user/data/data_time=2020-02-18 00%3A00%3A00/test.txt
```

表结构为：

```
data_time DATETIME,
k2        INT,
k3        INT
```

8. 从 HDFS 导入一批数据，指定超时时间和过滤比例。使用明文 my_hdfs_broker 的 broker。简单认证。并且将原有数据中与导入数据中 v2 大于 100 的列相匹配的列删除，其他列正常导入

```
LOAD LABEL example_db.label8
(
    MERGE DATA INFILE("HDFS://test:802/input/file")
    INTO TABLE `my_table`
    (k1, k2, k3, v2, v1)
    DELETE ON v2 > 100
)
WITH HDFS
(
    "hadoop.username"="user",
    "password"="pass"
)
PROPERTIES
(
    "timeout" = "3600",
    "max_filter_ratio" = "0.1"
);
```

使用 MERGE 方式导入。my_table 必须是一张 Unique Key 的表。当导入数据中的 v2 列的值大于 100 时，该行会被认为是一个删除行。

导入任务的超时时间是 3600 秒，并且允许错误率在 10% 以内。

9. 导入时指定 source_sequence 列，保证 UNIQUE_KEYS 表中的替换顺序：

```
LOAD LABEL example_db.label9
(
    DATA INFILE("HDFS://test:802/input/file")
    INTO TABLE `my_table`
    COLUMNS TERMINATED BY ","
    (k1,k2,source_sequence,v1,v2)
    ORDER BY source_sequence
```

```
)
WITH HDFS
(
    "hadoop.username"="user",
    "password"="pass"
)
```

my_table 必须是 Unique Key 模型表，并且指定了 Sequcence Col。数据会按照源数据中 source_sequence 列的值来保证顺序性。

10. 从 HDFS 导入一批数据，指定文件格式为 json 并指定 json_root、 jsonpaths

```
LOAD LABEL example_db.label10
(
    DATA INFILE("HDFS://test:port/input/file.json")
    INTO TABLE `my_table`
    FORMAT AS "json"
    PROPERTIES(
      "json_root" = "$.item",
      "jsonpaths" = "[$.id, $.city, $.code]"
    )
)
with HDFS (
"hadoop.username" = "user"
"password" = ""
)
PROPERTIES
(
"timeout"="1200",
"max_filter_ratio"="0.1"
);
```

`jsonpaths` 可与 `column list` 及 `SET (column_mapping)`配合：

```
LOAD LABEL example_db.label10
(
    DATA INFILE("HDFS://test:port/input/file.json")
    INTO TABLE `my_table`
    FORMAT AS "json"
    (id, code, city)
    SET (id = id * 10)
    PROPERTIES(
      "json_root" = "$.item",
      "jsonpaths" = "[$.id, $.code, $.city]"
    )
```

```
)
with HDFS (
"hadoop.username" = "user"
"password" = ""
)
PROPERTIES
(
"timeout"="1200",
"max_filter_ratio"="0.1"
);
```

11. 从腾讯云 cos 中以 csv 格式导入数据。

```
LOAD LABEL example_db.label10
(
DATA INFILE("cosn://my_bucket/input/file.csv")
INTO TABLE `my_table`
(k1, k2, k3)
)
WITH BROKER "broker_name"
(
    "fs.cosn.userinfo.secretId" = "xxx",
    "fs.cosn.userinfo.secretKey" = "xxxx",
    "fs.cosn.bucket.endpoint_suffix" = "cos.xxxxxxxxx.myqcloud.com"
)
```

12. 导入 CSV 数据时去掉双引号，并跳过前 5 行。

```
LOAD LABEL example_db.label12
(
DATA INFILE("cosn://my_bucket/input/file.csv")
INTO TABLE `my_table`
(k1, k2, k3)
PROPERTIES("trim_double_quotes" = "true", "skip_lines" = "5")
)
WITH BROKER "broker_name"
(
    "fs.cosn.userinfo.secretId" = "xxx",
    "fs.cosn.userinfo.secretKey" = "xxxx",
    "fs.cosn.bucket.endpoint_suffix" = "cos.xxxxxxxxx.myqcloud.com"
)
```

7.3.2.2.2 MYSQL LOAD

描述

使用 MySql 客户端将本地数据文件导入到 Doris 中。MySQL Load 是一种同步导入方式，执行导入后即返回导入结果。可以通过 LOAD DATA 语句的返回结果判断导入是否成功。MySQL Load 可以保证一批导入任务的原子性，要么全部导入成功，要么全部导入失败。

语法

```
LOAD DATA
[ LOCAL ]
INFILE "<file_name>"
INTO TABLE "<tbl_name>"
[ PARTITION (<partition_name> [, ... ]) ]
[ COLUMNS TERMINATED BY "<column_separator>" ]
[ LINES TERMINATED BY "<line_delimiter>" ]
[ IGNORE <number> {LINES | ROWS} ]
[ (<col_name_or_user_var> [, ... ] ) ]
[ SET (col_name={<expr> | DEFAULT} [, col_name={<expr> | DEFAULT}] ...) ]
[ PROPERTIES ("<key>" = "<value>" [ , ... ]) ]
```

必选参数

1. <file_name>

> 填写本地文件路径，可以是相对路径，也可以是绝对路径。目前只支持单个文件，不支持多个文件。

2. <tbl_name>

> 表名可以指定数据库名，如案例所示。也可以省略，则会使用当前用户所在的数据库。

可选参数

1. LOCAL

> 指定LOCAL表示读取客户端文件。不指定表示读取 FE 服务端本地文件。导入 FE 本地文件的功能默认是关闭的，需要在 FE 节点上设置mysql_load_server_secure_path来指定安全路径，才能打开该功能。

2. <partition_name>

支持指定多个分区导入，多个分区逗号隔开。

3. <column_separator>

指定列分隔符。

4. <line_delimiter>

指定行分隔符。

5. IGNORE <number> { LINES | ROWS }

用户跳过 CSV 的表头，可以跳过任意行数。该语法也可以用IGNORE num ROWS代替。

6. <col_name_or_user_var>

列映射语法，具体参数详见导入的数据转换 的列映射章节。

7. properties ("<key>"="<value>",...)

参数	参数说明
max_filter_ratio	最大容忍可过滤（数据不规范等原因）的数据比例，默认零容忍。
timeout	指定导入的超时时间，单位秒。默认是 600 秒。可设置范围为 1 秒 ~ 259200 秒。
strict_mode	用户指定此次导入是否开启严格模式，默认为关闭。
timezone	指定本次导入所使用的时区，默认为东八区。该参数会影响所有导入涉及的和时区有关的函数结果。
exec_mem_limit	导入内存限制，默认为 2GB，单位为字节。
trim_double_quotes	布尔类型，默认值为 false，为 true 时表示裁剪掉导入文件每个字段最外层的双引号。

参数	参数说明
enclose	包围符。当 csv 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为 “,”，包围符为 “ ‘ “，数据为”a,’b,c’ ”，则 “b,c” 会被解析为一个字段。注意：当 enclose 设置为"时，trim_double_quotes 一定要设置为 true。
escape	转义符。用于转义在 csv 字段中出现的与包围符相同的字符。例如数据为 “a, ‘b,’ c’ ”，包围符为 “ ‘ “，希望”b,’ c “被作为一个字段解析，则需要指定单字节转义符，例如” “，然后将数据修改为”a,’b,’c’ ”。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	对指定的库表的导入权限

注意事项

- MySQL Load 以语法LOAD DATA开头，无须指定 LABEL

举例

1. 将客户端本地文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表。指定超时时间为 100 秒

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("timeout"="100")
```

2. 将服务端本地文件’/root/testData’(需设置 FE 配置mysql_load_server_secure_path为/root) 中的数据导入到数据库’testDb’ 中’testTbl’ 的表。指定超时时间为 100 秒

```
LOAD DATA
INFILE '/root/testData'
INTO TABLE testDb.testTbl
PROPERTIES ("timeout"="100")
```

3. 将客户端本地文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表，允许 20% 的错误率

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("max_filter_ratio"="0.2")
```

4. 将客户端本地文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表，允许 20% 的错误率，并且指定文件的列名

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
(k2, k1, v1)
PROPERTIES ("max_filter_ratio"="0.2")
```

5. 将本地文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表中的 p1, p2 分区，允许 20% 的错误率。

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PARTITION (p1, p2)
PROPERTIES ("max_filter_ratio"="0.2")
```

6. 将本地行分隔符为0102, 列分隔符为0304的 CSV 文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表中。

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
COLUMNS TERMINATED BY '0304'
LINES TERMINATED BY '0102'
```

7. 将本地文件’testData’ 中的数据导入到数据库’testDb’ 中’testTbl’ 的表中的 p1, p2 分区，并跳过前面 3 行。

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PARTITION (p1, p2)
IGNORE 1 LINES
```

8. 导入数据进行严格模式过滤，并设置时区为 Africa/Abidjan

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("strict_mode"="true", "timezone"="Africa/Abidjan")
```

9. 导入数据进行限制导入内存为 10GB, 并在 10 分钟超时

```
LOAD DATA LOCAL
INFILE 'testData'
INTO TABLE testDb.testTbl
PROPERTIES ("exec_mem_limit"="10737418240", "timeout"="600")
```

7.3.2.2.3 SHOW LOAD

描述

该语句用于展示指定的导入任务的执行情况。

语法

```
SHOW LOAD
[FROM <db_name>]
[
   WHERE
   [LABEL  = [ "<your_label>" | LIKE "<label_matcher>"]]
   [ STATE = { " PENDING " | " ETL " | " LOADING " | " FINISHED " | " CANCELLED " } ]
]
[ORDER BY { <col_name> | <expr> | <position> }]
[LIMIT <limit>[OFFSET <offset>]];
```

可选参数

1. <db_name>

> 不指定 db_name，使用当前默认数据库。

2. <label_matcher>

> 使用 LABEL LIKE = "<label_matcher>"，则会匹配导入任务的 label 包含 label_matcher 的导入任务。

3. <your_label>

使用 LABEL = "<your_label>"，则精确匹配指定的 label。

4. STATE = { ” PENDING ” | ” ETL ” | ” LOADING ” | ” FINISHED ” | ” CANCELLED ” }

指定了 PENDING 表示匹配 LOAD = “PENDING” 状态的 job，其余状态词同理。

5. <col_name>

指定结果集中用于排序的列名。

6. <expr>

使用表达式进行排序。

7. <position>

按列在 SELECT 列表中的位置（从 1 开始）排序。

8. <limit>

如果指定了 LIMIT，则显示 limit 条匹配记录。否则全部显示。

9. <offset>

指定从偏移量 offset 开始显示查询结果。默认情况下偏移量为 0。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	库（Database）	需要对库表的导入权限

返回值

返回指定导入任务的详细状态。

举例

1. 展示默认 db 的所有导入任务

```
SHOW LOAD;
```

2. 展示指定 db 的导入任务，label 中包含字符串 “2014_01_02”，展示最老的 10 个

```
SHOW LOAD FROM example_db WHERE LABEL LIKE "2014_01_02" LIMIT 10;
```

3. 展示指定 db 的导入任务，指定 label 为 “load_example_db_20140102” 并按 LoadStartTime 降序排序

```
SHOW LOAD FROM example_db WHERE LABEL = "load_example_db_20140102" ORDER BY LoadStartTime
    ↪ DESC;
```

4. 展示指定 db 的导入任务，指定 label 为 “load_example_db_20140102” ，state 为 “loading”, 并按 LoadStartTime 降序排序

```
SHOW LOAD FROM example_db WHERE LABEL = "load_example_db_20140102" AND STATE = "loading"
    ↪ ORDER BY LoadStartTime DESC;
```

5. 展示指定 db 的导入任务并按 LoadStartTime 降序排序，并从偏移量 5 开始显示 10 条查询结果

```
SHOW LOAD FROM example_db ORDER BY LoadStartTime DESC limit 5,10;
SHOW LOAD FROM example_db ORDER BY LoadStartTime DESC limit 10 offset 5;
```

6. 小批量导入是查看导入状态的命令

```
curl --location-trusted -u {user}:{passwd} http://{hostname}:{port}/api/{database}/_load_info
    ↪ ?label={labelname}
```

7.3.2.2.4 SHOW STREAM LOAD

描述

该语句用于展示指定的 Stream Load 任务的执行情况

语法：

```
SHOW STREAM LOAD
[FROM db_name]
[
  WHERE
  [LABEL [ = "your_label" | LIKE "label_matcher"]]
  [STATUS = ["SUCCESS"|"FAIL"]]
]
[ORDER BY ...]
[LIMIT limit][OFFSET offset];
```

说明：

1. 默认 BE 是不记录 Stream Load 的记录，如果你要查看需要在 BE 上启用记录，配置参数是：enable_stream
   ↪ _load_record=true ，具体怎么配置请参照BE 配置项
2. 如果不指定 db_name，使用当前默认 db
3. 如果使用 LABEL LIKE，则会匹配 Stream Load 任务的 label 包含 label_matcher 的任务
4. 如果使用 LABEL = ，则精确匹配指定的 label
5. 如果指定了 STATUS，则匹配 STREAM LOAD 状态
6. 可以使用 ORDER BY 对任意列组合进行排序
7. 如果指定了 LIMIT，则显示 limit 条匹配记录。否则全部显示
8. 如果指定了 OFFSET，则从偏移量 offset 开始显示查询结果。默认情况下偏移量为 0。

示例

1. 展示默认 db 的所有 Stream Load 任务

```
    SHOW STREAM LOAD;
```

2. 展示指定 db 的 Stream Load 任务，label 中包含字符串 “2014_01_02”，展示最老的 10 个

```
  SHOW STREAM LOAD FROM example_db WHERE LABEL LIKE "2014_01_02" LIMIT 10;
```

2. 展示指定 db 的 Stream Load 任务，指定 label 为 “load_example_db_20140102”

```
SHOW STREAM LOAD FROM example_db WHERE LABEL = "load_example_db_20140102";
```

2. 展示指定 db 的 Stream Load 任务，指定 status 为 “success”, 并按 StartTime 降序排序

```
SHOW STREAM LOAD FROM example_db WHERE STATUS = "success" ORDER BY StartTime DESC;
```

2. 展示指定 db 的导入任务并按 StartTime 降序排序，并从偏移量 5 开始显示 10 条查询结果

```
SHOW STREAM LOAD FROM example_db ORDER BY StartTime DESC limit 5,10;
SHOW STREAM LOAD FROM example_db ORDER BY StartTime DESC limit 10 offset 5;
```

关键词

SHOW, STREAM, LOAD

7.3.2.2.5 SHOW CREATE LOAD

描述

该语句用于展示导入作业的创建语句。

语法：

```
SHOW CREATE LOAD FOR <load_name>;
```

必选参数

<load_name>

例行导入作业名称

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN/NODE_PRIV	库（Database）	需要集群管理员权限

返回值

返回指定导入作业的创建语句。

举例

- 展示默认 db 下指定导入作业的创建语句

```
SHOW CREATE LOAD for test_load
```

7.3.2.2.6 CANCEL LOAD

描述

该语句用于撤销指定 label 的导入作业。或者通过模糊匹配批量撤销导入作业

语法

```
CANCEL LOAD
[FROM <db_name>]
WHERE [LABEL = "<load_label>" | LABEL like "<label_pattern>" | STATE = { "PENDING" | "ETL" | "
    ↪ LOADING" } ]
```

必选参数

1. <db_name>

> 撤销导入作业名称

可选参数

1. <load_label>

> 如果使用 LABEL = "<load_label>"，则精确匹配指定的 label。

2. <label_pattern>

> 如果使用 LABEL LIKE "<label_pattern>"，则会匹配导入任务的 label 包含 label_matcher 的导入任务。

3. STATE = {” PENDING ” | ” ETL ” | ” LOADING ” | ” FINISHED ” | ” CANCELLED ” }

> 指定了 PENDING 表示撤销 STATE = “PENDING” 状态的 job，其余状态同理。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	库（Database）	需要对库表的导入权限

注意事项

- 1.2.0 版本之后支持根据 State 取消作业。
- 只能取消处于 PENDING、 ETL、 LOADING 状态的未完成的导入作业。
- 当执行批量撤销时，Doris 不会保证所有对应的导入作业原子的撤销。即有可能仅有部分导入作业撤销成功。用户可以通过 SHOW LOAD 语句查看作业状态，并尝试重复执行 CANCEL LOAD 语句。

举例

1. 撤销数据库 example_db 上，label 为 `example_db_test_load_label` 的导入作业

```
CANCEL LOAD
FROM example_db
WHERE LABEL = "example_db_test_load_label";
```

2. 撤销数据库 exampledb 上，所有包含 example 的导入作业。

```
CANCEL LOAD
FROM example_db
WHERE LABEL like "example_";
```

3. 取消状态为 LOADING 的导入作业。

```
CANCEL LOAD
FROM example_db
WHERE STATE = "loading";
```

7.3.2.2.7 SHOW LOAD WARNINGS

描述

如果导入任务失败且错误信息为 `ETL_QUALITY_UNSATISFIED`，则说明存在导入质量问题，如果想看到这些有质量问题的导入任务，该语句就是完成这个操作的。

语法：

```
SHOW LOAD WARNINGS
[FROM <db_name>]
[
   WHERE
   [LABEL  = [ "<your_label>" ]]
   [LOAD_JOB_ID = ["<job_id>"]]
]
```

可选参数

1. <db_name>

> 如果不指定 db_name，使用当前默认数据库。

2. <your_label>

> 如果使用 LABEL = <your_label>，则精确匹配指定的 label。

3. <job_id>

> 如果指定了 LOAD_JOB_ID = <job_id>，则精确匹配指定的 JOB_ID。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	库（Database）	需要对库表的导入权限

返回值

返回指定 db 的导入任务中存在质量问题的数据。

举例

- 展示指定 db 的导入任务中存在质量问题的数据，指定 label 为 “load_demo_20210112”

```
SHOW LOAD WARNINGS FROM demo WHERE LABEL = "load_demo_20210112"
```

7.3.2.2.8 CREATE ROUTINE LOAD

描述

例行导入（Routine Load）功能支持用户提交一个常驻的导入任务，通过不断地从指定的数据源读取数据，将数据导入到 Doris 中。

目前仅支持通过无认证或者 SSL 认证方式，从 Kafka 导入 CSV 或 Json 格式的数据。导入 Json 格式数据使用示例

语法

```
CREATE ROUTINE LOAD [<db>.]<job_name> [ON <tbl_name>]
[<merge_type>]
[<load_properties>]
[<job_properties>]
FROM <data_source> [<data_source_properties>]
[COMMENT "<comment>"]
```

必选参数

1. [<db>.]<job_name>

> 导入作业的名称，在同一个 database 内，相同名称只能有一个 job 在运行。

2. FROM <data_source>

> 数据源的类型。当前支持：KAFKA

3. <data_source_properties>

> 1. <kafka_broker_list>
>
> Kafka 的 broker 连接信息。格式为 ip:host。多个 broker 之间以逗号分隔。
>
> text "kafka_broker_list" = "broker1:9092,broker2:9092"

2. <kafka_topic>

指定要订阅的 Kafka 的 topic。 text "kafka_topic" = "my_topic"

可选参数

1. <tbl_name>

指定需要导入的表的名称，可选参数，如果不指定，则采用动态表的方式，这个时候需要 Kafka 中的数据包含表名的信息。

目前仅支持从 Kafka 的 Value 中获取表名，且需要符合这种格式：以 json 为例：table_name|{"
↪ col1": "val1", "col2": "val2"}, 其中 tbl_name 为表名，以 | 作为表名和表数据的分隔符。

csv 格式的数据也是类似的，如：table_name|val1,val2,val3。注意，这里的 table_name 必须和 Doris 中的表名一致，否则会导致导入失败。

tips: 动态表不支持 columns_mapping 参数。如果你的表结构和 Doris 中的表结构一致，且存在大量的表信息需要导入，那么这种方式将是不二选择。

2. <merge_type>

数据合并类型。默认为 APPEND，表示导入的数据都是普通的追加写操作。MERGE 和 DELETE 类型仅适用于 Unique Key 模型表。其中 MERGE 类型需要配合 [DELETE ON] 语句使用，以标注 Delete Flag 列。而 DELETE 类型则表示导入的所有数据皆为删除数据。

tips: 当使用动态多表的时候，请注意此参数应该符合每张动态表的类型，否则会导致导入失败。

3. <load_properties>

用于描述导入数据。组成如下：

```
[column_separator],
[columns_mapping],
```

```
[preceding_filter],
[where_predicates],
[partitions],
[DELETE ON],
[ORDER BY]
```

1. <column_separator>

指定列分隔符，默认为 \t

COLUMNS TERMINATED BY ","

2. <columns_mapping>

用于指定文件列和表中列的映射关系，以及各种列转换等。关于这部分详细介绍，可以参阅 [列的映射，转换与过滤] 文档。

(k1, k2, tmpk1, k3 = tmpk1 + 1)

tips: 动态表不支持此参数。

3. <preceding_filter>

过滤原始数据。关于这部分详细介绍，可以参阅 [列的映射，转换与过滤] 文档。

WHERE k1 > 100 and k2 = 1000

tips: 动态表不支持此参数。

4. <where_predicates>

根据条件对导入的数据进行过滤。关于这部分详细介绍，可以参阅 [列的映射，转换与过滤] 文档。

WHERE k1 > 100 and k2 = 1000

tips: 当使用动态多表的时候，请注意此参数应该符合每张动态表的列，否则会导致导入失败。通常在使用动态多表的时候，我们仅建议通用公共列使用此参数。

5. <partitions>

指定导入目的表的哪些 partition 中。如果不指定，则会自动导入到对应的 partition 中。

PARTITION(p1, p2, p3)

tips: 当使用动态多表的时候，请注意此参数应该符合每张动态表，否则会导致导入失败。

6. <DELETE ON>

需配合 MEREGE 导入模式一起使用，仅针对 Unique Key 模型的表。用于指定导入数据中表示 Delete Flag 的列和计算关系。

DELETE ON v3 >100

tips: 当使用动态多表的时候，请注意此参数应该符合每张动态表，否则会导致导入失败。

7. <ORDER BY>

仅针对 Unique Key 模型的表。用于指定导入数据中表示 Sequence Col 的列。主要用于导入时保证数据顺序。

tips: 当使用动态多表的时候，请注意此参数应该符合每张动态表，否则会导致导入失败。

4. <job_properties>

用于指定例行导入作业的通用参数。

text PROPERTIES ("key1" = "val1", "key2" = "val2")

目前我们支持以下参数：

1. <desired_concurrent_number>

   期望的并发度。一个例行导入作业会被分成多个子任务执行。这个参数指定一个作业最多有多少任务可以同时执行。必须大于 0。默认为 5。

这个并发度并不是实际的并发度，实际的并发度，会通过集群的节点数、负载情况，以及数据源的情况综合考虑。

"desired_concurrent_number" = "3"

2. <max_batch_interval>/<max_batch_rows>/<max_batch_size>

这三个参数分别表示：

1．每个子任务最大执行时间，单位是秒。必须大于等于 1。默认为 10。
2．每个子任务最多读取的行数。必须大于等于 200000。默认是 20000000。
3．每个子任务最多读取的字节数。单位是字节，范围是 100MB 到 10GB。默认是 1G。

这三个参数，用于控制一个子任务的执行时间和处理量。当任意一个达到阈值，则任务结束。

```text
"max_batch_interval" = "20",
"max_batch_rows" = "300000",
"max_batch_size" = "209715200"
```

3. <max_error_number>

   采样窗口内，允许的最大错误行数。必须大于等于 0。默认是 0，即不允许有错误行。

   采样窗口为 max_batch_rows * 10。即如果在采样窗口内，错误行数大于 max_error_number，则会导致例行作业被暂停，需要人工介入检查数据质量问题。

   被 where 条件过滤掉的行不算错误行。

4. <strict_mode>

   是否开启严格模式，默认为关闭。如果开启后，非空原始数据的列类型变换如果结果为 NULL，则会被过滤。指定方式为：

   "strict_mode" = "true"

   strict mode 模式的意思是：对于导入过程中的列类型转换进行严格过滤。严格过滤的策略如下：

   1. 对于列类型转换来说，如果 strict mode 为 true，则错误的数据将被 filter。这里的错误数据是指：原始数据并不为空值，在参与列类型转换后结果为空值的这一类数据。
   2. 对于导入的某列由函数变换生成时，strict mode 对其不产生影响。
   3. 对于导入的某列类型包含范围限制的，如果原始数据能正常通过类型转换，但无法通过范围限制的，strict mode 对其也不产生影响。例如：如果类型是 decimal(1,0), 原始数据为 10，则属于可以通过类型转换但不在列声明的范围内。这种数据 strict 对其不产生影响。

   strict mode 与 source data 的导入关系

   这里以列类型为 TinyInt 来举例

   注：当表中的列允许导入空值时

source data	source data example	string to int	strict_mode	result
空值	\N	N/A	true or false	NULL
not null	aaa or 2000	NULL	true	invalid data(filtered)
not null	aaa	NULL	false	NULL
not null	1	1	true or false	correct data

这里以列类型为 Decimal(1,0) 举例

注：当表中的列允许导入空值时

source data	source data example	string to int	strict_mode	result
空值	\N	N/A	true or false	NULL
not null	aaa	NULL	true	invalid data(filtered)
not null	aaa	NULL	false	NULL
not null	1 or 10	1	true or false	correct data

注意：10 虽然是一个超过范围的值，但是因为其类型符合 decimal 的要求，所以 strict mode 对其不产生影响。10 最后会在其他 ETL 处理流程中被过滤。但不会被 strict mode 过滤。

5. <timezone>

   指定导入作业所使用的时区。默认为使用 Session 的 timezone 参数。该参数会影响所有导入涉及的和时区有关的函数结果。

   "timezone" = "Asia/Shanghai"

6. <format>

   指定导入数据格式，默认是 csv，支持 json 格式。

   "format" = "json"

7. <jsonpaths>

   当导入数据格式为 json 时，可以通过 jsonpaths 指定抽取 Json 数据中的字段。

   -H "jsonpaths: [\"$.k2\", \"$.k1\"]"

8. <strip_outer_array>

   当导入数据格式为 json 时，strip_outer_array 为 true 表示 Json 数据以数组的形式展现，数据中的每一个元素将被视为一行数据。默认值是 false。

   -H "strip_outer_array: true"

9. <json_root>

   当导入数据格式为 json 时，可以通过 json_root 指定 Json 数据的根节点。Doris 将通过 json_root 抽取根节点的元素进行解析。默认为空。

   -H "json_root: $.RECORDS"

10. <send_batch_parallelism>

    整型，用于设置发送批处理数据的并行度，如果并行度的值超过 BE 配置中的 max_send_batch_parallelism_per_job，那么作为协调点的 BE 将使用 max_send_batch_parallelism_per_job 的值。

    "send_batch_parallelism" = "10"

11. <load_to_single_tablet>

    布尔类型，为 true 表示支持一个任务只导入数据到对应分区的一个 tablet，默认值为 false，该参数只允许在对带有 random 分桶的 olap 表导数的时候设置。

    "load_to_single_tablet" = "true"

12. <partial_columns>

    布尔类型，为 true 表示使用部分列更新，默认值为 false，该参数只允许在表模型为 Unique 且采用 Merge on Write 时设置。一流多表不支持此参数。

    "partial_columns" = "true"

13. <max_filter_ratio>

    采样窗口内，允许的最大过滤率。必须在大于等于 0 到小于等于 1 之间。默认值是 0。

    采样窗口为 max_batch_rows * 10。即如果在采样窗口内，错误行数/总行数大于 max_filter_ratio，则会导致例行作业被暂停，需要人工介入检查数据质量问题。

    被 where 条件过滤掉的行不算错误行。

14. <enclose>

    包围符。当 csv 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为“,”，包围符为“‘“，数据为”a,’b,c’”，则“b,c”会被解析为一个字段。

    注意：当 enclose 设置为"时，trim_double_quotes 一定要设置为 true。

15. <escape>

    转义符。用于转义在 csv 字段中出现的与包围符相同的字符。例如数据为“a,‘b,’c’”，包围符为“”’，希望“b,’c 被作为一个字段解析，则需要指定单字节转义符，例如 \，然后将数据修改为 a,'b,\'c'。

5. <data_source_properties> 中的可选属性

1. <kafka_partitions>/<kafka_offsets>

   指定需要订阅的 kafka partition，以及对应的每个 partition 的起始 offset。如果指定时间，则会从大于等于该时间的最近一个 offset 处开始消费。

   offset 可以指定从大于等于 0 的具体 offset，或者：

   - OFFSET_BEGINNING: 从有数据的位置开始订阅。
   - OFFSET_END: 从末尾开始订阅。
   - 时间格式，如：“2021-05-22 11:00:00”

   如果没有指定，则默认从 OFFSET_END 开始订阅 topic 下的所有 partition。

```
"kafka_partitions" = "0,1,2,3",
"kafka_offsets" = "101,0,OFFSET_BEGINNING,OFFSET_END"
```

```
"kafka_partitions" = "0,1,2,3",
"kafka_offsets" = "2021-05-22 11:00:00,2021-05-22 11:00:00,2021-05-22
    ↪ 11:00:00"
```

注意，时间格式不能和 OFFSET 格式混用。

2. <property>

   指定自定义 kafka 参数。功能等同于 kafka shell 中“–property”参数。

   当参数的 value 为一个文件时，需要在 value 前加上关键词：“FILE:”。

   关于如何创建文件，请参阅CREATE FILE 命令文档。

   更多支持的自定义参数，请参阅 librdkafka 的官方 CONFIGURATION 文档中，client 端的配置项。如：

```
"property.client.id" = "12345",
"property.ssl.ca.location" = "FILE:ca.pem"
```

```
    ↪
    ↪
    ↪
                                                
```

示例

- 为 example_db 的 example_tbl 创建一个名为 test1 的 Kafka 例行导入任务。指定列分隔符和 group.id 和 client.id，并且自动默认消费所有分区，且从有数据的位置（OFFSET_BEGINNING）开始订阅

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS TERMINATED BY ",",
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100)
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "property.group.id" = "xxx",
    "property.client.id" = "xxx",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

- 为 example_db 创建一个名为 test1 的 Kafka 例行动态多表导入任务。指定列分隔符和 group.id 和 client.id，并且自动默认消费所有分区，且从有数据的位置（OFFSET_BEGINNING）开始订阅

我们假设需要将 Kafka 中的数据导入到 example_db 中的 test1 以及 test2 表中，我们创建了一个名为 test1 的例行导入任务，同时将 test1 和 test2 中的数据写到一个名为 my_topic 的 Kafka 的 topic 中，这样就可以通过一个例行导入任务将 Kafka 中的数据导入到两个表中。

```
CREATE ROUTINE LOAD example_db.test1
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
```

```
    "property.group.id" = "xxx",
    "property.client.id" = "xxx",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

- 为 example_db 的 example_tbl 创建一个名为 test1 的 Kafka 例行导入任务。导入任务为严格模式。

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100),
PRECEDING FILTER k1 = 1,
WHERE k1 > 100 and k2 like "%doris%"
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "true"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "kafka_partitions" = "0,1,2,3",
    "kafka_offsets" = "101,0,0,200"
);
```

- 通过 SSL 认证方式，从 Kafka 集群导入数据。同时设置 client.id 参数。导入任务为非严格模式，时区为 Africa/Abidjan

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100),
WHERE k1 > 100 and k2 like "%doris%"
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false",
    "timezone" = "Africa/Abidjan"
)
FROM KAFKA
(
```

```
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "property.security.protocol" = "ssl",
    "property.ssl.ca.location" = "FILE:ca.pem",
    "property.ssl.certificate.location" = "FILE:client.pem",
    "property.ssl.key.location" = "FILE:client.key",
    "property.ssl.key.password" = "abcdefg",
    "property.client.id" = "my_client_id"
);
```

- 导入 Json 格式数据。默认使用 Json 中的字段名作为列名映射。指定导入 0,1,2 三个分区，起始 offset 都为 0

```
CREATE ROUTINE LOAD example_db.test_json_label_1 ON table1
COLUMNS(category,price,author)
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false",
    "format" = "json"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "kafka_partitions" = "0,1,2",
    "kafka_offsets" = "0,0,0"
);
```

- 导入 Json 数据，并通过 Jsonpaths 抽取字段，并指定 Json 文档根节点

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(category, author, price, timestamp, dt=from_unixtime(timestamp, '%Y%m%d'))
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false",
    "format" = "json",
    "jsonpaths" = "[\"$.category\",\"$.author\",\"$.price\",\"$.timestamp\"]",
```

```
    "json_root" = "$.RECORDS"
    "strip_outer_array" = "true"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "kafka_partitions" = "0,1,2",
    "kafka_offsets" = "0,0,0"
);
```

- 为 example_db 的 example_tbl 创建一个名为 test1 的 Kafka 例行导入任务。并且使用条件过滤。

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
WITH MERGE
COLUMNS(k1, k2, k3, v1, v2, v3),
WHERE k1 > 100 and k2 like "%doris%",
DELETE ON v3 >100
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "20",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200",
    "strict_mode" = "false"
)
FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "kafka_partitions" = "0,1,2,3",
    "kafka_offsets" = "101,0,0,200"
);
```

- 导入数据到含有 sequence 列的 Unique Key 模型表中

```
CREATE ROUTINE LOAD example_db.test_job ON example_tbl
COLUMNS TERMINATED BY ",",
COLUMNS(k1,k2,source_sequence,v1,v2),
ORDER BY source_sequence
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "30",
```

```
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200"
) FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
    "kafka_topic" = "my_topic",
    "kafka_partitions" = "0,1,2,3",
    "kafka_offsets" = "101,0,0,200"
);
```

- 从指定的时间点开始消费

```
CREATE ROUTINE LOAD example_db.test_job ON example_tbl
PROPERTIES
(
    "desired_concurrent_number"="3",
    "max_batch_interval" = "30",
    "max_batch_rows" = "300000",
    "max_batch_size" = "209715200"
) FROM KAFKA
(
    "kafka_broker_list" = "broker1:9092,broker2:9092",
    "kafka_topic" = "my_topic",
    "kafka_default_offsets" = "2021-05-21 10:00:00"
);
```

7.3.2.2.9 ALTER ROUTINE LOAD

描述

该语法用于修改已经创建的例行导入作业。只能修改处于 PAUSED 状态的作业。

语法

```
ALTER ROUTINE LOAD FOR [<db>.]<job_name>
[<job_properties>]
FROM [<data_source>]
[<data_source_properties>]
```

必选参数

1. [<db>.]<job_name>

指定要修改的作业名称。标识符必须以字母字符开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来。

标识符不能使用保留关键字。有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1. <job_properties>

指定需要修改的作业参数。目前支持修改的参数包括：

- desired_concurrent_number
- max_error_number
- max_batch_interval
- max_batch_rows
- max_batch_size
- jsonpaths
- json_root
- strip_outer_array
- strict_mode
- timezone
- num_as_string
- fuzzy_parse
- partial_columns
- max_filter_ratio

2. <data_source_properties>

数据源的相关属性。目前支持：

- kafka_partitions
- kafka_offsets
- kafka_broker_list
- kafka_topic
- 自定义 property，如 property.group.id

3. <data_source>

数据源的类型。当前支持：

- KAFKA

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

注意事项

- kafka_partitions 和 kafka_offsets 用于修改待消费的 kafka partition 的 offset，仅能修改当前已经消费的 partition。不能新增 partition。

示例

- 将 desired_concurrent_number 修改为 1

```
ALTER ROUTINE LOAD FOR db1.label1
PROPERTIES
(
    "desired_concurrent_number" = "1"
);
```

- 将 desired_concurrent_number 修改为 10，修改 partition 的 offset，修改 group id

```
ALTER ROUTINE LOAD FOR db1.label1
PROPERTIES
(
    "desired_concurrent_number" = "10"
)
FROM kafka
(
    "kafka_partitions" = "0, 1, 2",
    "kafka_offsets" = "100, 200, 100",
    "property.group.id" = "new_group"
);
```

7.3.2.2.10 PAUSE ROUTINE LOAD

描述

该语法用于暂停一个或所有 Routine Load 作业。被暂停的作业可以通过 RESUME 命令重新运行。

语法

```
PAUSE [ALL] ROUTINE LOAD FOR <job_name>
```

必选参数

1. <job_name>

> 指定要暂停的作业名称。如果指定了 ALL，则无需指定 job_name。

可选参数

1. [ALL]

> 可选参数。如果指定 ALL，则表示暂停所有例行导入作业。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

注意事项

- 作业被暂停后，可以通过 RESUME 命令重新启动
- 暂停操作不会影响已经下发到 BE 的任务，这些任务会继续执行完成

示例

- 暂停名称为 test1 的例行导入作业。

```
PAUSE ROUTINE LOAD FOR test1;
```

- 暂停所有例行导入作业。

```
PAUSE ALL ROUTINE LOAD;
```

7.3.2.2.11 RESUME ROUTINE LOAD

描述

该语法用于重启一个或所有被暂停的 Routine Load 作业。重启的作业，将继续从之前已消费的 offset 继续消费。

语法

```
RESUME [ALL] ROUTINE LOAD FOR <job_name>
```

必选参数

1. <job_name>

> 指定要重启的作业名称。如果指定了 ALL，则无需指定 job_name。

可选参数

1. [ALL]

> 可选参数。如果指定 ALL，则表示重启所有被暂停的例行导入作业。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

注意事项

- 只能重启处于 PAUSED 状态的作业
- 重启后的作业会从上次消费的位置继续消费数据
- 如果作业被暂停时间过长，可能会因为 Kafka 数据过期导致重启失败

示例

- 重启名称为 test1 的例行导入作业。

```
RESUME ROUTINE LOAD FOR test1;
```

- 重启所有例行导入作业。

```
RESUME ALL ROUTINE LOAD;
```

7.3.2.2.12 STOP ROUTINE LOAD

描述

该语法用于停止一个 Routine Load 作业。被停止的作业无法再重新运行，这与 PAUSE 命令不同。如果需要重新导入数据，需要创建新的导入作业。

语法

```
STOP ROUTINE LOAD FOR <job_name>;
```

必选参数

1. <job_name>

> 指定要停止的作业名称。可以是以下形式：
>
> - <job_name>: 停止当前数据库下指定名称的作业
> - <db_name>.<job_name>: 停止指定数据库下指定名称的作业

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

注意事项

- 停止操作是不可逆的，被停止的作业无法通过 RESUME 命令重新启动
- 停止操作会立即生效，正在执行的任务会被中断
- 建议在停止作业前先通过 SHOW ROUTINE LOAD 命令检查作业状态
- 如果只是临时暂停作业，建议使用 PAUSE 命令

示例

- 停止名称为 test1 的例行导入作业。

```
STOP ROUTINE LOAD FOR test1;
```

- 停止指定数据库下的例行导入作业。

```
STOP ROUTINE LOAD FOR example_db.test1;
```

7.3.2.2.13 SHOW ROUTINE LOAD

描述

该语句用于展示 Routine Load 作业运行状态。可以查看指定作业或所有作业的状态信息。

语法

```
SHOW [ALL] ROUTINE LOAD [FOR <jobName>];
```

可选参数

1. `[ALL]`

> 可选参数。如果指定，则会显示所有作业（包括已停止或取消的作业）。否则只显示当前正在运行的作业。

2. `[FOR <jobName>]`

> 可选参数。指定要查看的作业名称。如果不指定，则显示当前数据库下的所有作业。
>
> 支持以下形式：
>
> - `job_name`: 显示当前数据库下指定名称的作业
> - `db_name.job_name`: 显示指定数据库下指定名称的作业

返回结果

字段名	说明
Id	作业 ID
Name	作业名称
CreateTime	作业创建时间
PauseTime	最近一次作业暂停时间
EndTime	作业结束时间
DbName	对应数据库名称
TableName	对应表名称（多表情况下显示 multi-table）
IsMultiTbl	是否为多表
State	作业运行状态
DataSourceType	数据源类型：KAFKA
CurrentTaskNum	当前子任务数量

字段名	说明
JobProperties	作业配置详情
DataSourceProperties	数据源配置详情
CustomProperties	自定义配置
Statistic	作业运行状态统计信息
Progress	作业运行进度
Lag	作业延迟状态
ReasonOfStateChanged	作业状态变更的原因
ErrorLogUrls	被过滤的质量不合格的数据的查看地址
OtherMsg	其他错误信息

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

注意事项

- State 状态说明：
- NEED_SCHEDULE：作业等待被调度
- RUNNING：作业运行中
- PAUSED：作业被暂停
- STOPPED：作业已结束
- CANCELLED：作业已取消
- Progress 说明：
- 对于 Kafka 数据源，显示每个分区当前已消费的 offset
- 例如 {“0”：“2”} 表示 Kafka 分区 0 的消费进度为 2
- Lag 说明：
- 对于 Kafka 数据源，显示每个分区的消费延迟
- 例如 {“0”:10} 表示 Kafka 分区 0 的消费延迟为 10

示例

- 展示名称为 test1 的所有例行导入作业（包括已停止或取消的作业）

```
SHOW ALL ROUTINE LOAD FOR test1;
```

- 展示名称为 test1 的当前正在运行的例行导入作业

```
SHOW ROUTINE LOAD FOR test1;
```

- 显示 example_db 下，所有的例行导入作业（包括已停止或取消的作业）。结果为一行或多行。

```
use example_db;
SHOW ALL ROUTINE LOAD;
```

- 显示 example_db 下，所有正在运行的例行导入作业

```
use example_db;
SHOW ROUTINE LOAD;
```

- 显示 example_db 下，名称为 test1 的当前正在运行的例行导入作业

```
SHOW ROUTINE LOAD FOR example_db.test1;
```

- 显示 example_db 下，名称为 test1 的所有例行导入作业（包括已停止或取消的作业）。结果为一行或多行。

```
SHOW ALL ROUTINE LOAD FOR example_db.test1;
```

7.3.2.2.14 SHOW ROUTINE LOAD TASK

描述

该语法用于查看一个指定的 Routine Load 作业的当前正在运行的子任务情况。

语法

```
SHOW ROUTINE LOAD TASK WHERE JobName = <job_name>;
```

必选参数

1. <job_name>

> 要查看的例行导入作业名称。

返回结果

返回结果包含以下字段：

字段名	说明
TaskId	子任务的唯一 ID
TxnId	子任务对应的导入事务 ID
TxnStatus	子任务对应的导入事务状态。为 null 时表示子任务还未开始调度
JobId	子任务对应的作业 ID
CreateTime	子任务的创建时间
ExecuteStartTime	子任务被调度执行的时间，通常晚于创建时间
Timeout	子任务超时时间，通常是作业设置的 `max_batch_interval` 的两倍
BeId	执行这个子任务的 BE 节点 ID
DataSourceProperties	子任务准备消费的 Kafka Partition 的起始 offset。是一个 Json 格式字符串。Key 为 Partition Id，Value 为消费的起始 offset

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD TASK 需要对表有 LOAD 权限

注意事项

- TxnStatus 为 null 不代表任务出错，可能是任务还未开始调度
- DataSourceProperties 中的 offset 信息可用于追踪数据消费进度
- Timeout 时间到达后，任务会自动结束，无论是否完成数据消费

示例

- 展示名为 test1 的例行导入任务的子任务信息。

```
SHOW ROUTINE LOAD TASK WHERE JobName = "test1";
```

7.3.2.2.15 SHOW CREATE ROUTINE LOAD

描述

该语句用于展示例行导入作业的创建语句。

结果中的 kafka partition 和 offset 展示的当前消费的 partition，以及对应的待消费的 offset。该结果不一定是实时的消费位点，应以 show routine load 的结果为准。

语法

```
SHOW [ALL] CREATE ROUTINE LOAD for <load_name>;
```

必选参数

1. <load_name>

> 例行导入作业名称

可选参数

1. [ALL]

> 可选参数，代表获取所有作业，包括历史作业

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	表（Table）	SHOW ROUTINE LOAD 需要对表有 LOAD 权限

示例

- 展示默认 db 下指定例行导入作业的创建语句

```
SHOW CREATE ROUTINE LOAD for test_load
```

7.3.2.2.16 CREATE SYNC JOB

描述

数据同步 (Sync Job) 功能支持用户提交一个常驻的数据同步作业，通过从指定的远端地址读取 Binlog 日志，增量同步用户在 MySQL 数据库中数据更新操作的 CDC (Change Data Capture) 信息。

用户可通过SHOW SYNC JOB 查看数据同步作业的状态。

语法

```
CREATE SYNC [<db>.]<job_name>
(<channel_desc> [, ... ])
<binlog_desc>
```

where:

```
channel_desc
  : FROM <mysql_db>.<src_tbl> INTO <des_tbl> [ <columns_mapping> ]
```

```
binlog_desc
  : FROM BINLOG ("<key>" = "<value>" [, ... ])
```

必选参数

1. <job_name>

同步作业名称，是当前数据库中作业的唯一标识。相同 <job_name> 的作业在同一时刻只能有一个在运行。

2. <channel_desc>

用于描述 MySQL 源表到 Doris 目标表之间的映射关系。

- <mysql_db.src_tbl>
  指定 MySQL 端的数据库及源表。
- <des_tbl>
  指定 Doris 端的目标表。目标表必须为 Unique 表，并且需开启表的 batch delete 功能（详见 help alter table 中的“批量删除功能”）。
- <columns_mapping>(可选)
  指定 MySQL 源表和 Doris 目标表之间的列映射关系。如果不指定，FE 会默认按照列顺序一一对应。
  > 注意： 不支持使用 col_name = expr 的形式指定列映射。>> 示例： >- 假设目标表列为 (k1, k2, v1)，可通过调整顺序实现 (k2, k1, v1)；>- 或者通过映射忽略源数据中的多余列，例如 (k2, k1, v1, dummy_column)。

3. <binlog_desc>

用来描述远端数据源，目前仅支持 Canal 数据源。

对于 Canal 数据源，相关属性均以 canal. 为前缀：

- canal.server.ip：Canal 服务器的地址
- canal.server.port：Canal 服务器的端口
- canal.destination：实例的标识

- `canal.batchSize`：获取数据的最大 batch 大小（默认值为 8192）
- `canal.username`：实例的用户名
- `canal.password`：实例的密码
- `canal.debug` (可选)：设置为 true 时，会打印出每个 batch 及每行数据的详细信息

注意事项

- 当前数据同步作业仅支持连接 Canal 服务器。
- 同一数据库中，相同 <job_name> 的作业在同一时刻只能有一个运行。
- Doris 目标表必须为 Unique 表，且需启用 batch delete 功能，否则数据同步可能失败。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
LOAD_PRIV	表	该操作只能由拥有导入表的 LOAD_PRIV 权限的用户或角色执行。

示例

1. 简单示例： 为 `test_db` 数据库的目标表 `test_tbl` 创建一个名为 `job1` 的数据同步作业，连接本地 Canal 服务器，对应 MySQL 源表 `mysql_db1.tbl1`。

```
CREATE SYNC `test_db`.`job1`
(
    FROM `mysql_db1`.`tbl1` INTO `test_tbl`
)
FROM BINLOG
(
    "type" = "canal",
    "canal.server.ip" = "127.0.0.1",
    "canal.server.port" = "11111",
    "canal.destination" = "example",
    "canal.username" = "",
    "canal.password" = ""
);
```

2. 多表同步及列映射示例： 为 `test_db` 数据库的多张表创建一个名为 `job1` 的数据同步作业，对应多个 MySQL 源表，并显式指定列映射。

```
CREATE SYNC `test_db`.`job1`
(
```

```
    FROM `mysql_db`.`t1` INTO `test1` (k1, k2, v1),
    FROM `mysql_db`.`t2` INTO `test2` (k3, k4, v2)
)
FROM BINLOG
(
    "type" = "canal",
    "canal.server.ip" = "xx.xxx.xxx.xx",
    "canal.server.port" = "12111",
    "canal.destination" = "example",
    "canal.username" = "username",
    "canal.password" = "password"
);
```

7.3.2.2.17 PAUSE SYNC JOB

描述

通过 job_name 暂停一个数据库内正在运行的常驻数据同步作业。被暂停的作业将停止同步数据，并保持消费的最新位置，直到用户恢复该作业。

语法

```
PAUSE SYNC JOB [<db>.]<job_name>
```

必选参数

1. <job_name> > 要暂停的同步作业的名称。

可选参数

1. <db> > 如果使用 [.] 前缀指定了一个数据库，那么该作业将处于指定的数据库中；否则，将使用当前数据库。

权限控制

任意用户或角色都可以执行该操作

示例

1. 暂停名称为 job_name 的数据同步作业。

```
PAUSE SYNC JOB `job_name`;
```

7.3.2.2.18 RESUME SYNC JOB

描述

通过 job_name 恢复当前数据库中已暂停的常驻数据同步作业。恢复后，作业将从上一次暂停前保存的最新位置继续同步数据。

语法

```
RESUME SYNC JOB [<db>.]<job_name>
```

必选参数

1. <job_name> > 指定要恢复的数据同步作业的名称。

可选参数

1. <db> > 如果使用 [.] 前缀指定了一个数据库，那么该作业将处于指定的数据库中；否则，将使用当前数据库。

权限控制

任意用户或角色都可以执行该操作

示例

1. 恢复名称为 job_name 的数据同步作业。

```
RESUME SYNC JOB `job_name`;
```

7.3.2.2.19 STOP SYNC JOB

描述

此语句通过 job_name 停止一个数据库内非停止状态的常驻数据同步作业。

语法

```
STOP SYNC JOB [<db>.]<job_name>
```

必选参数

1. <job_name> > 要暂停的同步作业的名称。

可选参数

1. <db> > 如果使用 [.] 前缀指定了一个数据库，那么该作业将处于指定的数据库中；否则，将使用当前数据库。

权限控制

任意用户或角色都可以执行该操作

示例

1. 停止名称为 job_name 的数据同步作业

```
STOP SYNC JOB `job_name`;
```

7.3.2.2.20 SHOW SYNC JOB

描述

此语句用于显示所有数据库中的常驻数据同步作业状态。

语法

```
SHOW SYNC JOB [FROM db_name]
```

可选参数

1. <db_name> > 显示指定数据库下的所有数据同步作业状态。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限之一：

权限	对象	说明
ADMIN_PRIV, SELECT_PRIV, LOAD_PRIV, ALTER_PRIV, CREATE_PRIV, DROP_PRIV, SHOW_VIEW_PRIV	数据库 db_name	执行此操作需至少拥有上述权限中的一项。

示例

1. 显示当前数据库的所有数据同步作业状态。

```
SHOW SYNC JOB;
```

2. 显示 test_db 数据库下的所有数据同步作业状态。

```
SHOW SYNC JOB FROM `test_db`;
```

7.3.2.2.21 SYNC

描述

该语句用于同步非 Master Frontend（FE）节点的元数据。在 Apache Doris 中，只有 Master FE 节点可以写入元数据，其他 FE 节点的元数据写入操作都会转发至 Master 节点。在 Master 节点完成元数据写入操作后，非 Master 节点会存在短暂的元数据同步延迟。可以使用该语句强制同步元数据。

语法

```
SYNC;
```

权限控制

任意用户或角色都可以执行该操作

示例

同步元数据：

```
SYNC;
```

7.3.2.2.22 EXPORT

描述

EXPORT 命令用于将指定表的数据导出为文件到指定位置。目前支持通过 Broker 进程，S3 协议或 HDFS 协议，导出到远端存储，如 HDFS，S3，BOS，COS（腾讯云）上。

EXPORT 是一个异步操作，该命令会提交一个 EXPORT JOB 到 Doris，任务提交成功立即返回。执行后可使用 SHOW EXPORT 命令查看进度。

语法：

```
EXPORT TABLE <table_name>
[ PARTITION ( <partation_name> [ , ... ] ) ]
[ <where_clause> ]
TO <export_path>
[ <properties> ]
WITH <target_storage>
[ <broker_properties> ];
```

必选参数

1. <table_name>

当前要导出的表的表名。支持 Doris 本地表、视图 View、 Catalog 外表数据的导出。

2. <export_path>

导出的文件路径。可以是目录，也可以是文件目录加文件前缀，如hdfs://path/to/my_file_

可选参数

1. <where_clause>

可以指定导出数据的过滤条件。

2. <partation_name>

可以只导出指定表的某些指定分区，只对 Doris 本地表有效。

3. <properties>

用于指定一些导出参数。

```
[ PROPERTIES ("<key>"="<value>" [, ... ]) ]
```

可以指定如下参数：
- label: 可选参数，指定此次 Export 任务的 Label，当不指定时系统会随机生成一个 Label。

- column_separator：指定导出的列分隔符，默认为 \t，支持多字节。该参数只用于 CSV 文件格式。
- line_delimiter：指定导出的行分隔符，默认为 \n，支持多字节。该参数只用于 CSV 文件格式。
- columns：指定导出表的某些列。
- format：指定导出作业的文件格式，支持：parquet, orc, csv, csv_with_names、 csv_with_names_and_types。默认为 CSV 格式。

- max_file_size：导出作业单个文件大小限制，如果结果超过这个值，将切割成多个文件。max_file_size取值范围是 [5MB, 2GB], 默认为 1GB。（当指定导出为 orc 文件格式时，实际切分文件的大小将是 64MB 的倍数，如：指定 max_file_size = 5MB, 实际将以 64MB 为切分；指定 max_file_size = 65MB, 实际将以 128MB 为切分）
- parallelism：导出作业的并发度，默认为1，导出作业会开启parallelism个数的线程去执行select into outfile语句。（如果 Parallelism 个数大于表的 Tablets 个数，系统将自动把 Parallelism 设置为 Tablets 个数大小，即每一个select into outfile语句负责一个 Tablets）
- delete_existing_files: 默认为 false，若指定为 true，则会先删除export_path所指定目录下的所有文件，然后导出数据到该目录下。例如：“export_path” = “/user/tmp”, 则会删除 “/user/” 下所有文件及目录；“file_path” = “/user/tmp/”, 则会删除 “/user/tmp/” 下所有文件及目录。
- with_bom: 默认为 false，若指定为 true，则导出的文件编码为带有 BOM 的 UTF8 编码（只对 csv 相关的文件格式生效）。
- data_consistency: 可以设置为 none / partition，默认为 partition。指示以何种粒度切分导出表，none 代表 Tablets 级别，partition代表 Partition 级别。
- timeout：导出作业的超时时间，默认为 2 小时，单位是秒。
- compress_type：(自 2.1.5 支持) 当指定导出的文件格式为 Parquet / ORC 文件时，可以指定 Parquet / ORC 文件使用的压缩方式。Parquet 文件格式可指定压缩方式为 SNAPPY，GZIP，BROTLI，ZSTD，LZ4 及 PLAIN，默认值为 SNAPPY。ORC 文件格式可指定压缩方式为 PLAIN，SNAPPY，ZLIB 以及 ZSTD，默认值为 ZLIB。该参数自 2.1.5 版本开始支持。（PLAIN 就是不采用压缩）。自 3.1.1 版本开始，支持对 CSV 格式指定压缩算法，目前支持 “plain”，“gz”，“bz2”，“snappyblock”，“lz4block”，“zstd”。

> 注意要使用 delete_existing_files 参数，还需要在 fe.conf 中添加配置enable_delete_existing_files = true并重启 fe，此时 delete_existing_files 才会生效。delete_existing_files = true 是一个危险的操作，建议只在测试环境中使用。

4. <target_storage>
存储介质，可选 BROKER、 S3、 HDFS。

5. <broker_properties>
根据 <target_storage> 不同的存储介质，需要指定不同的属性。

- BROKER
  可以通过 Broker 进程写数据到远端存储上。这里需要定义相关的连接信息供 Broker 使用。

```
WITH BROKER "broker_name"
("<key>"="<value>" [,...])
```

Broker 相关属性：
- username: 用户名 - password: 密码 - hadoop.security.authentication: 指定认证方式为 kerberos - kerberos_principal: 指定 kerberos 的 principal - kerberos_keytab: 指定 kerberos 的 keytab 文件路径。该文件必须为 Broker 进程所在服务器上的文件的绝对路径。并且可以被 Broker 进程访问

- HDFS

可以直接将数据写到远端 HDFS 上。

```
WITH HDFS ("<key>"="<value>" [,...])
```

HDFS 相关属性：
- `fs.defaultFS`: namenode 地址和端口 - `hadoop.username`: HDFS 用户名 - `dfs.nameservices`: name service 名称，与 hdfs-site.xml 保持一致 - `dfs.ha.namenodes.[nameservice ID]`: namenode 的 id 列表，与 hdfs-site.xml 保持一致 - `dfs.namenode.rpc-address.[nameservice ID].[name node ID]`: Name node 的 rpc 地址，数量与 namenode 数量相同，与 hdfs-site.xml 保持一致

对于开启 kerberos 认证的 Hadoop 集群，还需要额外设置如下 PROPERTIES 属性： - `dfs.namenode.kerberos.principal`: HDFS namenode 服务的 principal 名称 - `hadoop.security.authentication`: 认证方式设置为 kerberos - `hadoop.kerberos.principal`: 设置 Doris 连接 HDFS 时使用的 Kerberos 主体 - `hadoop.kerberos.keytab`: 设置 keytab 本地文件路径

- S3

可以直接将数据写到远端 S3 对象存储上。

```
WITH S3 ("<key>"="<value>" [,...])
```

S3 相关属性：- `s3.endpoint` - `s3.region` - `s3.secret_key` - `s3.access_key` - `use_path_style`: (选填) 默认为 `false`。S3 SDK 默认使用 Virtual-hosted Style 方式。但某些对象存储系统可能没开启或不支持 Virtual-hosted Style 方式的访问，此时可以添加 `use_path_style` 参数来强制使用 Path Style 访问方式。

返回值

列名	类型	说明
jobId	long	导出作业的唯一标识符。
label	string	导出作业的标签。
dbId	long	数据库的标识符。
tableId	long	表的标识符。
state	string	当前作业的状态。
path	string	导出文件的路径。
partitions	string	导出的分区名称列表，多个分区名称用逗号分隔。
progress	int	导出作业的当前进度（百分比）。
createTimeMs	string	作业创建时间的毫秒值，格式化为日期时间。
exportStartTimeMs	string	导出作业开始时间的毫秒值，格式化为日期时间。
exportFinishTimeMs	string	导出作业结束时间的毫秒值，格式化为日期时间。
failMsg	string	导出作业失败时的错误信息。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV	库（Database）	需要对数据库、表的读权限。

注意事项

并发执行

一个 Export 作业可以设置parallelism参数来并发导出数据。parallelism参数实际就是指定执行 EXPORT 作业的线程数量。当设置"data_consistency" = "none"时，每一个线程会负责导出表的部分 Tablets。

一个 Export 作业的底层执行逻辑实际上是SELECT INTO OUTFILE语句，parallelism参数设置的每一个线程都会去执行独立的SELECT INTO OUTFILE语句。

Export 作业拆分成多个SELECT INTO OUTFILE的具体逻辑是：将该表的所有 tablets 平均的分给所有 parallel 线程，如：- num(tablets) = 40, parallelism = 3，则这 3 个线程各自负责的 tablets 数量分别为 14，13，13 个。- num(tablets) = 2, parallelism = 3，则 Doris 会自动将 parallelism 设置为 2，每一个线程负责一个 tablets。

当一个线程负责的 tablest 超过 maximum_tablets_of_outfile_in_export 数值（默认为 10，可在 fe.conf 中添加maximum_tablets_of_outfile_in_export参数来修改该值）时，该线程就会拆分为多个SELECT INTO OUTFILE ↪ 语句，如：- 一个线程负责的 tablets 数量分别为 14，maximum_tablets_of_outfile_in_export = 10，则该线程负责两个SELECT INTO OUTFILE语句，第一个SELECT INTO OUTFILE语句导出 10 个 tablets，第二个SELECT ↪ INTO OUTFILE语句导出 4 个 tablets，两个SELECT INTO OUTFILE语句由该线程串行执行。

当所要导出的数据量很大时，可以考虑适当调大parallelism参数来增加并发导出。若机器核数紧张，无法再增加parallelism 而导出表的 Tablets 又较多时，可以考虑调大maximum_tablets_of_outfile_in_export来增加一个SELECT INTO OUTFILE语句负责的 tablets 数量，也可以加快导出速度。

若希望以 Parition 粒度导出 Table，可以设置 Export 属性 "data_consistency" = "partition"，此时 Export 任务并发的线程会以 Parition 粒度来划分为多个 Outfile 语句，不同的 Outfile 语句导出的 Parition 不同，而同一个 Outfile 语句导出的数据一定属于同一个 Partition。如：设置 "data_consistency" = "partition" 后

- num(partition) = 40, parallelism = 3，则这 3 个线程各自负责的 Partition 数量分别为 14，13，13 个。
- num(partition) = 2, parallelism = 3，则 Doris 会自动将 Parallelism 设置为 2，每一个线程负责一个 Partition。

内存限制

通常一个 Export 作业的查询计划只有 扫描-导出 两部分，不涉及需要太多内存的计算逻辑。所以通常 2GB 的默认内存限制可以满足需求。

但在某些场景下，比如一个查询计划，在同一个 BE 上需要扫描的 Tablet 过多，或者 Tablet 的数据版本过多时，可能会导致内存不足。可以调整 Session 变量 exec_mem_limit 来调大内存使用限制。

其他事项

- 不建议一次性导出大量数据。一个 Export 作业建议的导出数据量最大在几十 GB。过大的导出会导致更多的垃圾文件和更高的重试成本。如果表数据量过大，建议按照分区导出。
- 如果 Export 作业运行失败，已经生成的文件不会被删除，需要用户手动删除。
- Export 作业会扫描数据，占用 IO 资源，可能会影响系统的查询延迟。

- 目前在 Export 时只是简单检查 Tablets 版本是否一致，建议在执行 Export 过程中不要对该表进行导入数据操作。
- 一个 Export Job 允许导出的分区数量最大为 2000，可以在 `fe.conf` 中添加参数 `maximum_number_of_export_partitions` 并重启 FE 来修改该设置。

示例

Export 数据到本地

> Export 数据到本地文件系统，需要在 `fe.conf` 中添加 `enable_outfile_to_local=true` 并且重启 FE。

- 将 Test 表中的所有数据导出到本地存储，默认导出 CSV 格式文件

```
EXPORT TABLE test TO "file:///home/user/tmp/";
```

- 将 Test 表中的 k1,k2 列导出到本地存储，默认导出 CSV 文件格式，并设置 Label

```
EXPORT TABLE test TO "file:///home/user/tmp/"
PROPERTIES (
  "label" = "label1",
  "columns" = "k1,k2"
);
```

- 将 Test 表中的 k1 < 50 的行导出到本地存储，默认导出 CSV 格式文件，并以 , 作为列分割符

```
EXPORT TABLE test WHERE k1 < 50 TO "file:///home/user/tmp/"
PROPERTIES (
  "columns" = "k1,k2",
  "column_separator"=","
);
```

- 将 Test 表中的分区 p1,p2 导出到本地存储，默认导出 csv 格式文件

```
EXPORT TABLE test PARTITION (p1,p2) TO "file:///home/user/tmp/"
PROPERTIES ("columns" = "k1,k2");
```

- 将 Test 表中的所有数据导出到本地存储，导出其他格式的文件 “ ‘sql – parquet EXPORT TABLE test TO “file:///home/user/tmp/” PROPERTIES (“columns” = “k1,k2” , “format” = “parquet”);

– orc EXPORT TABLE test TO “file:///home/user/tmp/” PROPERTIES (“columns” = “k1,k2” , “format” = “orc”);

– csv(csv_with_names) , Use ‘AA’ as the column separator and ‘zz’ as the row separator EXPORT TABLE test TO “file:///home/user/tmp/” PROPERTIES (“format” = “csv_with_names” , “column_separator” = “AA” , “line_delimiter” = “zz”);

– csv(csv_with_names_and_types) EXPORT TABLE test TO “file:///home/user/tmp/” PROPERTIES (“format” = “csv_with_names_and_types”); “ ‘

- 设置 max_file_sizes 属性
  当导出文件大于 5MB 时，将切割数据为多个文件，每个文件最大为 5MB。

```
-- When the exported file is larger than 5MB, the data will be split into multiple files, with
    ↪ each file having a maximum size of 5MB.
EXPORT TABLE test TO "file:///home/user/tmp/"
PROPERTIES (
  "format" = "parquet",
  "max_file_size" = "5MB"
);
```

- 设置 parallelism 属性

```
EXPORT TABLE test TO "file:///home/user/tmp/"
PROPERTIES (
  "format" = "parquet",
  "max_file_size" = "5MB",
  "parallelism" = "5"
);
```

- 设置 delete_existing_files 属性
  Export 导出数据时会先将/home/user/目录下所有文件及目录删除，然后导出数据到该目录下。

```
-- When exporting data, all files and directories under the `/home/user/` directory will be
    ↪ deleted first, and then the data will be exported to this directory.
EXPORT TABLE test TO "file:///home/user/tmp"
PROPERTIES (
  "format" = "parquet",
  "max_file_size" = "5MB",
  "delete_existing_files" = "true"
);
```

Export 到 S3

- 将 s3_test 表中的所有数据导出到 S3 上，以不可见字符 \x07 作为列或者行分隔符。如果需要将数据导出到 minio，还需要指定 use_path_style=true。

```
EXPORT TABLE s3_test TO "s3://bucket/a/b/c"
PROPERTIES (
  "column_separator"="\\x07",
  "line_delimiter" = "\\x07"
```

```
) WITH S3 (
  "s3.endpoint" = "xxxxx",
  "s3.region" = "xxxxx",
  "s3.secret_key"="xxxx",
  "s3.access_key" = "xxxxx"
)
```

export 到 HDFS

- 将 Test 表中的所有数据导出到 HDFS 上，导出文件格式为 Parquet，导出作业单个文件大小限制为 512MB，保留所指定目录下的所有文件。

```
EXPORT TABLE test TO "hdfs://hdfs_host:port/a/b/c/"
PROPERTIES(
    "format" = "parquet",
    "max_file_size" = "512MB",
    "delete_existing_files" = "false"
)
with HDFS (
"fs.defaultFS"="hdfs://hdfs_host:port",
"hadoop.username" = "hadoop"
);
```

Export 通过 Broker 节点

需要先启动 Broker 进程，并在 FE 中添加该 Broker。- 将 Test 表中的所有数据导出到 HDFS 上

```
EXPORT TABLE test TO "hdfs://hdfs_host:port/a/b/c"
WITH BROKER "broker_name"
(
  "username"="xxx",
  "password"="yyy"
);
```

- 将 testTbl 表中的分区 p1,p2 导出到 HDFS 上，以 “,” 作为列分隔符，并指定 Label

```
EXPORT TABLE testTbl PARTITION (p1,p2) TO "hdfs://hdfs_host:port/a/b/c"
PROPERTIES (
  "label" = "mylabel",
  "column_separator"=","
)
WITH BROKER "broker_name"
(
  "username"="xxx",
  "password"="yyy"
);
```

- 将 testTbl 表中的所有数据导出到 HDFS 上，以不可见字符 \x07 作为列或者行分隔符。

```
EXPORT TABLE testTbl TO "hdfs://hdfs_host:port/a/b/c"
PROPERTIES (
  "column_separator"="\\x07",
  "line_delimiter" = "\\x07"
)
WITH BROKER "broker_name"
(
  "username"="xxx",
  "password"="yyy"
)
```

7.3.2.2.23 CANCEL EXPORT

描述

该语句用于撤销指定 label 的 EXPORT 作业，或者通过模糊匹配批量撤销 EXPORT 作业

语法

```
CANCEL EXPORT
[ FROM <db_name> ]
WHERE [ LABEL = "<export_label>" | LABEL like "<label_pattern>" | STATE = "<state>" ]
```

可选参数

1. <db_name>

导出的数据任务的归属库名。如果省略，默认为当前数据库。

2. <export_label>

每个导入需要指定一个唯一的 Label。停止这个任务需要指定该 label。

3. <label_pattern>

模糊匹配的 label 表达式。如果要撤销多个 EXPORT 作业，可以使用 LIKE 进行模糊匹配。

4. <state>

state 可选项：PENDING、 IN_QUEUE、 EXPORTING。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	库（Database）	需要对数据库的修改权限。

注意事项

1. 只能取消处于 PENDING、 IN_QUEUE、 EXPORTING 状态的未完成的导出作业。
2. 当执行批量撤销时，Doris 不会保证所有对应的 EXPORT 作业原子的撤销。即有可能仅有部分 EXPORT 作业撤销成功。用户可以通过 SHOW EXPORT 语句查看作业状态，并尝试重复执行 CANCEL EXPORT 语句。
3. 当撤销EXPORTING状态的作业时，有可能作业已经导出部分数据到存储系统上，用户需要自行处理 (删除) 该部分导出数据。

示例

- 撤销数据库 example_db 上，label 为 example_db_test_export_label 的 EXPORT 作业

```
CANCEL EXPORT
FROM example_db
WHERE LABEL = "example_db_test_export_label" and STATE = "EXPORTING";
```

- 撤销数据库 exampledb 上，所有包含 example 的 EXPORT 作业。

```
CANCEL EXPORT
FROM example_db
WHERE LABEL like "%example%";
```

- 取消状态为 PENDING 的导入作业。

```
CANCEL EXPORT
FROM example_db
WHERE STATE = "PENDING";
```

7.3.2.2.24 SHOW EXPORT

描述

该语句用于展示指定的导出任务的执行情况

语法

```
SHOW EXPORT
[ FROM <db_name> ]
  [
    WHERE
      [ ID = <job_id> ]
      [ STATE = { "PENDING" | "EXPORTING" | "FINISHED" | "CANCELLED" } ]
      [ LABEL = <label> ]
   ]
[ ORDER BY <column_name> [ ASC | DESC ] [, column_name [ ASC | DESC ] ... ] ]
[ LIMIT <limit> ];
```

可选参数

1. <db_name>：可选参数，如果不指定，使用当前默认数据库。

2. <job_id>：可选参数，用于指定要展示的导出作业 ID。

3. <label>：可选参数，用于指定要展示的导出作业的标签。

4. <column_name>：可选参数，用于指定排序的列名。

5. <limit>：可选参数，如果指定了该参数，则仅显示指定条数的匹配记录；如果未指定，则显示全部记录。

返回值

列名	类型	说明
JobId	string	作业的唯一 ID
Label	string	该导出作业的标签，如果 Export 没有指定，则系统会默认生成一个。
State	string	作业状态：- PENDING：作业待调度 - EXPORTING：数据导出中 - FINISHED：作业成功 - CANCELLED：作业失败
Progress	string	作业进度。该进度以查询计划为单位。假设一共 10 个线程，当前已完成 3 个，则进度为 30%。
TaskInfo	json	以 Json 格式展示的作业信息：- db：数据库名 - tbl：表名 - partitions：指定导出的分区，空 列表表示所有分区 - column_separator：导出文件的列分隔符 - line_delimiter：导出文件的行分隔符 - tablet num：涉及的总 Tablet 数量 - broker：使用的 broker 的名称 - coord num：查询计划的个数 - max_file_size：一个导出文件的最大大小 - delete_existing_files：是否删除导出目录下已存在的文件及目录 - columns：指定需要导出的列名，空值代表导出所有列 - format：导出的文件格式
Path	string	远端存储上的导出路径
CreateTime	string	作业的创建时间
StartTime	string	作业开始调度时间
FinishTime	string	作业结束时间
Timeout	int	作业超时时间（单位：秒）。该时间从 CreateTime 开始计算。
ErrorMsg	string	如果作业出现错误，这里会显示错误原因
OutfileInfo	string	如果作业导出成功，这里会显示具体的 SELECT INTO OUTFILE 结果信息

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV	库（Database）	需要对数据库、表的读权限。

示例

- 展示默认 db 的所有导出任务

```
SHOW EXPORT;
```

- 展示指定 db 的导出任务，按 StartTime 降序排序

```
SHOW EXPORT FROM example_db ORDER BY StartTime DESC;
```

- 展示指定 db 的导出任务，state 为 “exporting”, 并按 StartTime 降序排序

```
SHOW EXPORT FROM example_db WHERE STATE = "exporting" ORDER BY StartTime DESC;
```

- 展示指定 db，指定 job_id 的导出任务

```
SHOW EXPORT FROM example_db WHERE ID = job_id;
```

- 展示指定 db，指定 label 的导出任务

```
SHOW EXPORT FROM example_db WHERE LABEL = "mylabel";
```

7.3.2.2.25 OUTFILE

描述

SELECT INTO OUTFILE 命令用于将查询结果导出为文件。目前支持通过 Broker 进程，S3 协议或 HDFS 协议，导出到远端存储，如 HDFS，S3，BOS，COS（腾讯云）上。

语法：

```
<query_stmt>
INTO OUTFILE "<file_path>"
[ FORMAT AS <format_as> ]
[ <properties> ]
```

必选参数

1. <query_stmt>

查询语句，必须是是合法的 SQL，参考 query 语句文档。

2. <file_path>

文件存储的路径及文件前缀。指向文件存储的路径以及文件前缀。如 hdfs://path/to/my_file_。
最终的文件名将由 my_file_、文件序号以及文件格式后缀组成。其中文件序号由 0 开始，数量为文件被分割的数量。如：
- my_file_abcdefg_0.csv - my_file_abcdefg_1.csv - my_file_abcdegf_2.csv

也可以省略文件前缀，只指定文件目录，如hdfs://path/to/

可选参数

1. <format_as>

指定导出格式。目前支持如下格式：
- CSV (默认) - PARQUET - CSV_WITH_NAMES - CSV_WITH_NAMES_AND_TYPES - ORC

注：PARQUET、 CSV_WITH_NAMES、 CSV_WITH_NAMES_AND_TYPES、 ORC 在 1.2 版本开始支持。

2. <properties>

```
[ PROPERTIES ("<key>"="<value>" [, ... ]) ]
```

目前支持通过 Broker 进程，或通过 S3/HDFS 协议进行导出。

自身导出文件相关的属性 - column_separator: 列分隔符，只用于 CSV 相关格式。在 1.2 版本开始支持多字节分隔符，如：“\x01”，“abc”。- line_delimiter: 行分隔符，只用于 CSV 相关格式。在 1.2 版本开始支持多字节分隔符，如：“\x01”，“abc”。- max_file_size: 单个文件大小限制，如果结果超过这个值，将切割成多个文件，max_file_size 取值范围是 [5MB, 2GB], 默认为 1GB。（当指定导出为 OCR 文件格式时，实际切分文件的大小将是 64MB 的倍数，如：指定 max_file_size = 5MB, 实际将以 64 MB 为切分；指定 max_file_size = 65MB, 实际将以 128 MB 为切分）- delete_existing_files: 默认为 false，若指定为 true，则会先删除 file_path 指定的目录下的所有文件，然后导出数据到该目录下。例如：“file_path” = “/user/tmp”, 则会删除 “/user/” 下所有文件及目录；“file_path” = “/user/tmp/”, 则会删除 “/user/tmp/” 下所有文件及目录。- file_suffix: 指定导出文件的后缀，若不指定该参数，将使用文件格式的默认后缀。- compress_type：当指定导出的文件格式为 Parquet / ORC 文件时，可以指定 Parquet / ORC 文件使用的压缩方式。Parquet 文件格式可指定压缩方式为 SNAPPY，GZIP，BROTLI，ZSTD，LZ4 及 PLAIN，默认值为 SNAPPY。ORC 文件格式可指定压缩方式为 PLAIN，SNAPPY，ZLIB 以及 ZSTD，默认值为 ZLIB。该参数自 2.1.5 版本开始支持。（PLAIN 就是不采用压缩）。自 3.1.1 版本开始，支持对 CSV 格式指定压缩算法，目前支持 “plain”，“gz”，“bz2”，“snappyblock”，“lz4block”，“zstd”。

Broker 相关属性（需加前缀 broker.）
- broker.name: broker: 名称 - broker.hadoop.security.authentication: 指定认证方式为 kerberos - broker.
↪ kerberos_principal: 指定 kerberos 的 principal - broker.kerberos_keytab: 指定 kerberos 的 keytab 文件路径。该文件必须为 Broker 进程所在服务器上的文件的绝对路径。并且可以被 Broker 进程访问

HDFS 相关属性 - fs.defaultFS: namenode 地址和端口 - hadoop.username: hdfs 用户名 - dfs.nameservices: name service 名称，与 hdfs-site.xml 保持一致 - dfs.ha.namenodes.[nameservice ID]: namenode 的 id 列表，与 hdfs-site.xml 保持一致 - dfs.namenode.rpc-address.[nameservice ID].[name node ID]: Name node 的 rpc 地址，数量与 namenode 数量相同，与 hdfs-site.xml 保持一致 - dfs.client.failover.proxy.provider.[nameservice ID]: HDFS 客户端连接活跃 namenode 的 java 类，通常是 “org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider”

对于开启 kerberos 认证的 Hadoop 集群，还需要额外设置如下 PROPERTIES 属性： - dfs.namenode.kerberos.
↪ principal: HDFS namenode 服务的 principal 名称 - hadoop.security.authentication: 认证方式设置为 kerberos - hadoop.kerberos.principal: 设置 Doris 连接 HDFS 时使用的 Kerberos 主体 - hadoop.kerberos.keytab: 设置 keytab 本地文件路径

S3 协议则直接执行 S3 协议配置即可： - s3.endpoint - s3.access_key - s3.secret_key - s3.region - use_path_
↪ style: (选填) 默认为 false。S3 SDK 默认使用 Virtual-hosted Style 方式。但某些对象存储系统可能没开启或不支持 Virtual-hosted Style 方式的访问，此时可以添加 use_path_style 参数来强制使用 Path Style 访问方式。

注意：若要使用 delete_existing_files 参数，还需要在 fe.conf 中添加配置enable_delete_
↪ existing_files = true并重启 fe，此时 delete_existing_files 才会生效。delete_existing_files = true

是一个危险的操作，建议只在测试环境中使用。

返回值

Outfile 语句返回的结果，各个列的含义如下：

列名	类型	说明
FileNumber	int	最终生成的文件个数
TotalRows	int	结果集行数
FileSize	int	导出文件总大小。单位字节。
URL	string	导出的文件路径的前缀，多个文件会以后缀 _0,_1 依次编号。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV	库（Database）	需要对数据库、表的读权限。

注意事项

数据类型映射

- 所有文件类型都支持导出基本数据类型，而对于复杂数据类型（ARRAY/MAP/STRUCT），当前只有 csv、orc、csv_with_names 和 csv_with_names_and_types 支持导出复杂类型，且不支持嵌套复杂类型。
- Parquet、ORC 文件格式拥有自己的数据类型，Doris 的导出功能能够自动将 Doris 的数据类型导出到 Parquet/ORC 文件格式的对应数据类型。以下是 Apache Doris 数据类型和 Parquet/ORC 文件格式的数据类型映射关系表：

1. Doris 导出到 ORC 文件格式的数据类型映射表：| Doris Type | Orc Type | |———————-|———-| | boolean | boolean | | tinyint | tinyint | | smallint | smallint | | int | int | | bigint | bigint | | largeInt | string | | date | string | | datev2 | string | | datetime | string | | datetimev2 | timestamp | | float | float | | double | double | | char / varchar / string | string | | decimal | decimal | | struct | struct | | map | map | | array | array |
2. Doris 导出到 Parquet 文件格式的数据类型映射表：

Doris 导出到 Parquet 文件格式时，会先将 Doris 内存数据转换为 Arrow 内存数据格式，然后由 Arrow 写出到 Parquet 文件格式。Doris 数据类型到 Arrow 数据类的映射关系为：| Doris Type | Arrow Type | |———————-|———-| | boolean | boolean | | tinyint | int8 | | smallint | int16 | | int | int32 | | bigint | int64 | | largeInt | utf8 | | date | utf8 | | datev2 | utf8 | | datetime | utf8 | | datetimev2 | utf8 | | float | float32 | | double | float64 | | char / varchar / string | utf8 | | decimal | decimal128 | | struct | struct | | map | map | | array | list |

导出数据量和导出效率

该功能本质上是执行一个 SQL 查询命令。最终的结果是单线程输出的。所以整个导出的耗时包括查询本身的耗时，和最终结果集写出的耗时。如果查询较大，需要设置会话变量 query_timeout 适当的延长查询超时时间。

导出文件的管理

Doris 不会管理导出的文件。包括导出成功的，或者导出失败后残留的文件，都需要用户自行处理。

导出到本地文件

导出到本地文件时需要先在 fe.conf 中配置enable_outfile_to_local=true

```
select * from tbl1 limit 10
INTO OUTFILE "file:///home/work/path/result_";
```

导出到本地文件的功能不适用于公有云用户，仅适用于私有化部署的用户。并且默认用户对集群节点有完全的控制权限。Doris 对于用户填写的导出路径不会做合法性检查。如果 Doris 的进程用户对该路径无写权限，或路径不存在，则会报错。同时处于安全性考虑，如果该路径已存在同名的文件，则也会导出失败。

Doris 不会管理导出到本地的文件，也不会检查磁盘空间等。这些文件需要用户自行管理，如清理等。

结果完整性保证

该命令是一个同步命令，因此有可能在执行过程中任务连接断开了，从而无法获悉导出的数据是否正常结束，或是否完整。此时可以使用 success_file_name 参数要求任务成功后，在目录下生成一个成功文件标识。用户可以通过这个文件，来判断导出是否正常结束。

并发导出

设置 Session 变量set enable_parallel_outfile = true;可开启 Outfile 并发导出。

示例

- 使用 Broker 方式导出，将简单查询结果导出到文件 hdfs://path/to/result.txt。指定导出格式为 CSV。使用 my_broker 并设置 kerberos 认证信息。指定列分隔符为 ,，行分隔符为 \n。

```
SELECT * FROM tbl
INTO OUTFILE "hdfs://path/to/result_"
FORMAT AS CSV
PROPERTIES
(
    "broker.name" = "my_broker",
    "broker.hadoop.security.authentication" = "kerberos",
    "broker.kerberos_principal" = "doris@YOUR.COM",
    "broker.kerberos_keytab" = "/home/doris/my.keytab",
    "column_separator" = ",",
    "line_delimiter" = "`\n`",
    "max_file_size" = "100MB"
);
```

最终生成文件如如果不大于 100MB，则为：result_0.csv。如果大于 100MB，则可能为 result_0.csv, result
↪ _1.csv, ...。

- 将简单查询结果导出到文件 hdfs://path/to/result.parquet。指定导出格式为 PARQUET。使用 my_broker 并设置 kerberos 认证信息。

```
SELECT c1, c2, c3 FROM tbl
INTO OUTFILE "hdfs://path/to/result_"
FORMAT AS PARQUET
PROPERTIES
(
    "broker.name" = "my_broker",
    "broker.hadoop.security.authentication" = "kerberos",
    "broker.kerberos_principal" = "doris@YOUR.COM",
    "broker.kerberos_keytab" = "/home/doris/my.keytab"
);
```

- 将 CTE 语句的查询结果导出到文件 hdfs://path/to/result.txt。默认导出格式为 CSV。使用 my_broker 并设置 HDFS 高可用信息。使用默认的行列分隔符。

```
WITH
x1 AS
(SELECT k1, k2 FROM tbl1),
x2 AS
(SELECT k3 FROM tbl2)
SELEC k1 FROM x1 UNION SELECT k3 FROM x2
INTO OUTFILE "hdfs://path/to/result_"
PROPERTIES
(
    "broker.name" = "my_broker",
    "broker.username"="user",
    "broker.password"="passwd",
    "broker.dfs.nameservices" = "my_ha",
    "broker.dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
    "broker.dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
    "broker.dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
    "broker.dfs.client.failover.proxy.provider" = "org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider"
);
```

最终生成文件如如果不大于 1GB，则为：result_0.csv。如果大于 1GB，则可能为 result_0.csv, result_1.
↪ csv, ...。

- 将 UNION 语句的查询结果导出到文件 bos://bucket/result.txt。指定导出格式为 PARQUET。使用 my_
↪ broker 并设置 HDFS 高可用信息。PARQUET 格式无需指定列分割符。导出完成后，生成一个标识文件。

```
SELECT k1 FROM tbl1 UNION SELECT k2 FROM tbl1
INTO OUTFILE "bos://bucket/result_"
FORMAT AS PARQUET
PROPERTIES
(
    "broker.name" = "my_broker",
    "broker.bos_endpoint" = "http://bj.bcebos.com",
    "broker.bos_accesskey" = "xxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "broker.bos_secret_accesskey" = "yyyyyyyyyyyyyyyyyyyyyyyyyyy"
);
```

- 将 Select 语句的查询结果导出到文件 `s3a://${bucket_name}/path/result.txt`。指定导出格式为 CSV。导出完成后，生成一个标识文件。

```
select k1,k2,v1 from tbl1 limit 100000
into outfile "s3a://my_bucket/export/my_file_"
FORMAT AS CSV
PROPERTIES
(
    "broker.name" = "hdfs_broker",
    "broker.fs.s3a.access.key" = "xxx",
    "broker.fs.s3a.secret.key" = "xxxx",
    "broker.fs.s3a.endpoint" = "https://cos.xxxxxx.myqcloud.com/",
    "column_separator" = ",",
    "line_delimiter" = "`\n`",
    "max_file_size" = "1024MB",
    "success_file_name" = "SUCCESS"
)
```

最终生成文件如如果不大于 1GB，则为：`my_file_0.csv`。如果大于 1GB，则可能为 `my_file_0.csv, result_1.csv, ...`。在 cos 上验证

1. 不存在的 `path` 会自动创建
2. access.key/secret.key/endpoint需要和cos的同学确认。尤其是endpoint的值，不需要填写bucket_name。

- 使用 S3 协议导出到 bos，并且并发导出开启。

```
set enable_parallel_outfile = true;
select k1 from tb1 limit 1000
into outfile "s3://my_bucket/export/my_file_"
format as csv
properties
```

```
(
    "s3.endpoint" = "http://s3.bd.bcebos.com",
    "s3.access_key" = "xxxx",
    "s3.secret_key" = "xxx",
    "s3.region" = "bd"
)
```

最终生成的文件前缀为 `my_file_{fragment_instance_id}_`。

- 使用 S3 协议导出到 bos，并且并发导出 Session 变量开启。注意：但由于查询语句带了一个顶层的排序节点，所以这个查询即使开启并发导出的 Session 变量，也是无法并发导出的。

```
set enable_parallel_outfile = true;
select k1 from tb1 order by k1 limit 1000
into outfile "s3://my_bucket/export/my_file_"
format as csv
properties
(
    "s3.endpoint" = "http://s3.bd.bcebos.com",
    "s3.access_key" = "xxxx",
    "s3.secret_key" = "xxx",
    "s3.region" = "bd"
)
```

- 使用 HDFS 方式导出，将简单查询结果导出到文件 `hdfs://${host}:${fileSystem_port}/path/to/result.txt`。指定导出格式为 CSV，用户名为 work。指定列分隔符为 `,`，行分隔符为 `\n`。

```
-- fileSystem_port 默认值为 9000
SELECT * FROM tbl
INTO OUTFILE "hdfs://${host}:${fileSystem_port}/path/to/result_"
FORMAT AS CSV
PROPERTIES
(
    "fs.defaultFS" = "hdfs://ip:port",
    "hadoop.username" = "work"
);
```

如果 Hadoop 集群开启高可用并且启用 Kerberos 认证，可以参考如下 SQL 语句：

```
SELECT * FROM tbl
INTO OUTFILE "hdfs://path/to/result_"
FORMAT AS CSV
PROPERTIES
(
'fs.defaultFS'='hdfs://hacluster/',
```

```
    'dfs.nameservices'='hacluster',
    'dfs.ha.namenodes.hacluster'='n1,n2',
    'dfs.namenode.rpc-address.hacluster.n1'='192.168.0.1:8020',
    'dfs.namenode.rpc-address.hacluster.n2'='192.168.0.2:8020',
    'dfs.client.failover.proxy.provider.hacluster'='org.apache.hadoop.hdfs.server.namenode.ha.
        ↪ ConfiguredFailoverProxyProvider',
    'dfs.namenode.kerberos.principal'='hadoop/_HOST@REALM.COM'
    'hadoop.security.authentication'='kerberos',
    'hadoop.kerberos.principal'='doris_test@REALM.COM',
    'hadoop.kerberos.keytab'='/path/to/doris_test.keytab'
    );
```

最终生成文件如如果不大于 100 MB，则为：result_0.csv。如果大于 100 MB，则可能为 result_0.csv, result
↪ _1.csv, ...。

- 将 Select 语句的查询结果导出到腾讯云 cos 的文件 cosn://${bucket_name}/path/result.txt。指定导出格式为 CSV。导出完成后，生成一个标识文件。

```
    select k1,k2,v1 from tbl1 limit 100000
    into outfile "cosn://my_bucket/export/my_file_"
    FORMAT AS CSV
    PROPERTIES
    (
        "broker.name" = "broker_name",
        "broker.fs.cosn.userinfo.secretId" = "xxx",
        "broker.fs.cosn.userinfo.secretKey" = "xxxx",
        "broker.fs.cosn.bucket.endpoint_suffix" = "cos.xxxxxx.myqcloud.com",
        "column_separator" = ",",
        "line_delimiter" = "`\n`",
        "max_file_size" = "1024MB",
        "success_file_name" = "SUCCESS"
    )
```

7.3.2.2.26 CLEAN LABEL

描述

用于手动清理历史导入作业的 Label。清理后，Label 可以重复使用。常用于一些程序设置的自动导入任务，重复执行时，设置导入固定字符串的 label，在每次导入任务发起前，先执行清理该 label 的语句。

语法

```
CLEAN LABEL [ <label> ] FROM <db_name>;
```

必选参数

1. <db_name>
label 归属库名。

可选参数

1. <label>
要清理的 label。如果省略，默认为当前数据库所有的 label。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	库（Database）	需要对数据库的修改权限。

示例

- 清理 db1 中，Label 为 label1 的导入作业。

```
CLEAN LABEL label1 FROM db1;
```

- 清理 db1 中所有历史 Label。

```
CLEAN LABEL FROM db1;
```

7.3.2.3 备份恢复

7.3.2.3.1 CREATE REPOSITORY

描述

该语句用于创建仓库。仓库用于属于备份或恢复。

语法

```
CREATE [READ ONLY] REPOSITORY <repo_name>
    WITH [ S3 | HDFS ]
    ON LOCATION <repo_location>
    PROPERTIES (
              -- S3 or HDFS storage property
              <storage_property>
              [ , ... ]
    )
```

必选参数

> 仓库的唯一名称

> 仓库的存储路径

> 仓库的属性。此处需要根据选择的是 S3 存储还是 HDFS 存储介质来选择对应的参数

可选参数如下，并可根据实际环境情况添加

参数	说明
s3.endpoint	S3 服务端点
s3.access_key	S3 访问密钥
s3.secret_key	S3 秘密密钥
s3.region	S3 区域
use_path_style	是否使用路径样式访问 S3（适用于 MinIO）
fs.defaultFS	Hadoop 默认文件系统 URI
hadoop.username	Hadoop 用户名

Note:

Doris 支持使用`AWS Assume Role`的方式创建位于 AWS S3 上的 Repository，请参考AWS 集成.

权限控制

权限	对象	说明
ADMIN_PRIV	整个集群管理权限	仅 root 或 superuser 用户可以创建仓库

注意事项

- 如果是只读仓库，则只能在仓库上进行恢复。如果不是，则可以进行备份和恢复操作。
- 根据 S3、 HDFS 的不同类型，PROPERTIES 有所不同，具体见示例。
- ON LOCATION , 如果是 S3 , 这里后面跟的是 S3 的 Bucket Name。
- 在做数据迁移操作时，需要在源集群和目的集群创建完全相同的仓库，以便目的集群可以通过这个仓库，查看到源集群备份的数据快照。
- 任何用户都可以通过 SHOW REPOSITORIES 命令查看已经创建的仓库。

举例

创建名为 s3_repo 的仓库

```
CREATE REPOSITORY `s3_repo`
WITH S3
ON LOCATION "s3://s3-repo"
PROPERTIES
(
    "s3.endpoint" = "http://s3-REGION.amazonaws.com",
    "s3.access_key" = "AWS_ACCESS_KEY",
    "s3.secret_key"="AWS_SECRET_KEY",
    "s3.region" = "REGION"
);
```

Note:

Doris 支持使用AWS Assume Role的方式创建位于 AWS S3 上的 Repository，请参考AWS 集成.

创建名为 hdfs_repo 的仓库

```
CREATE REPOSITORY `hdfs_repo`
WITH hdfs
ON LOCATION "hdfs://hadoop-name-node:54310/path/to/repo/"
PROPERTIES
(
    "fs.defaultFS"="hdfs://hadoop-name-node:54310",
    "hadoop.username"="user"
);
```

创建名为 minio_repo 的仓库。

```
CREATE REPOSITORY `minio_repo`
WITH S3
ON LOCATION "s3://minio_repo"
PROPERTIES
(
    "s3.endpoint" = "http://minio.com",
    "s3.access_key" = "MINIO_USER",
    "s3.secret_key"="MINIO_PASSWORD",
    "s3.region" = "REGION",
    "use_path_style" = "true"
);
```

使用临时秘钥创建名为 minio_repo 的仓库

```
CREATE REPOSITORY `minio_repo`
WITH S3
ON LOCATION "s3://minio_repo"
PROPERTIES
(
    "s3.endpoint" = "AWS_ENDPOINT",
    "s3.access_key" = "AWS_TEMP_ACCESS_KEY",
    "s3.secret_key" = "AWS_TEMP_SECRET_KEY",
    "s3.session_token" = "AWS_TEMP_TOKEN",
    "s3.region" = "AWS_REGION"
)
```

使用腾讯云 COS 创建仓库

```
CREATE REPOSITORY `cos_repo`
WITH S3
ON LOCATION "s3://backet1/"
PROPERTIES
```

```
(
    "s3.access_key" = "ak",
    "s3.secret_key" = "sk",
    "s3.endpoint" = "http://cos.ap-beijing.myqcloud.com",
    "s3.region" = "ap-beijing"
);
```

7.3.2.3.2 DROP REPOSITORY

描述

该语句用于删除一个已创建的仓库。

语法

```
DROP REPOSITORY <repo_name>;
```

必选参数

> 仓库的唯一名称

权限控制

权限	对象	说明
ADMIN_PRIV	整个集群管理权限	仅 root 或 superuser 用户可以创建仓库

注意事项

- 删除仓库，仅仅是删除该仓库在 Doris 中的映射，不会删除实际的仓库数据。删除后，可以再次通过指定相同的 LOCATION 映射到该仓库。

举例

删除名为 example_repo 的仓库：

```
DROP REPOSITORY `example_repo`;
```

7.3.2.3.3 SHOW CREATE REPOSITORY

描述

该语句用于展示仓库的创建语句。

语法

```
SHOW CREATE REPOSITORY for <repo_name>;
```

必选参数

> 仓库的唯一名称

举例

展示指定仓库的创建语句

```
SHOW CREATE REPOSITORY for example_repo;
```

7.3.2.3.4 SHOW REPOSITORIES

描述

该语句用于查看当前已创建的仓库

语法

```
SHOW REPOSITORIES;
```

返回值

字段	说明
RepoId	仓库的唯一标识符（ID）
RepoName	仓库的名称
CreateTime	仓库创建的时间
IsReadOnly	是否为只读仓库，`false` 表示不是只读仓库，`true` 表示是只读仓库
Location	仓库中用于备份数据的根目录
Broker	-
Type	仓库类型，目前可以支持 S3 与 HDFS
ErrMsg	仓库的错误信息。如果没有错误，通常为 NULL

举例

查看已创建的仓库：

```
SHOW REPOSITORIES;
```

```
+--------+--------------+---------------------+------------+----------+--------+------+--------+
| RepoId | RepoName     | CreateTime          | IsReadOnly | Location | Broker | Type | ErrMsg |
+--------+--------------+---------------------+------------+----------+--------+------+--------+
| 43411  | example_repo | 2025-01-17 18:50:47 | false      | s3://rep1  | -      | S3   | NULL
    ↪ |
+--------+--------------+---------------------+------------+----------+--------+------+--------+
```

7.3.2.3.5 BACKUP

描述

该语句用于备份指定数据库下的数据。该命令为异步操作，提交成功后，需通过 SHOW BACKUP 命令查看进度。

语法

```
BACKUP SNAPSHOT [<db_name>.]<snapshot_name>
TO `<repository_name>`
[ { ON | EXCLUDE } ]
    ( <table_name> [ PARTITION ( <partition_name> [, ...] ) ]
    [, ...] ) ]

[ PROPERTIES ( "<key>" = "<value>" [ , ... ] )]
```

必选参数

1.<db_name>

需要备份的数据所属的数据库名

2.<snapshot_name>

指定数据快照名。快照名不可重复，全局唯一

3.<repository_name>

仓库名。您可以通过 CREATE REPOSITORY 创建仓库

可选参数

1.<table_name>

需要备份的表名。如不指定则备份整个数据库。

- ON 子句中标识需要备份的表和分区。如果不指定分区，则默认备份该表的所有分区
- EXCLUDE 子句中标识不需要备份的表和分区。备份除了指定的表或分区之外这个数据库中所有表的所有分区数据。

2.<partition_name>

需要备份的分区名。如不指定则备份对应表的所有分区。

3.[PROPERTIES ("<key>" = "<value>" [, ...])]

数据快照属性，格式为 <key> = <value>，目前支持以下属性：

- “type” = “full”：表示这是一次全量更新（默认）
- “timeout” = “3600”：任务超时时间，默认为一天。单位秒。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：| 权限 | 对象 | 说明 | |:————|:————|:————-| | LOAD_PRIV | 用户（User）或角色（Role）| 用户或者角色拥有 LOAD_PRIV 权限才能进行此操作 |

注意事项：

- 仅支持备份 OLAP 类型的表。
- 同一数据库下只能有一个正在执行的 BACKUP 或 RESTORE 任务。

- 备份操作会备份指定表或分区的基础表及物化视图，并且仅备份一副本。异步物化视图还不支持。
- 备份操作的效率取决于数据量、Compute Node 节点数量以及文件数量。备份数据分片所在的每个 Compute Node 都会参与备份操作的上传阶段。节点数量越多，上传的效率越高，文件数据量只涉及到的分片数，以及每个分片中文件的数量。如果分片非常多，或者分片内的小文件较多，都可能增加备份操作的时间。

示例

1. 全量备份 example_db 下的表 example_tbl 到仓库 example_repo 中：

```
BACKUP SNAPSHOT example_db.snapshot_label1
TO example_repo
ON (example_tbl)
PROPERTIES ("type" = "full");
```

2. 全量备份 example_db 下，表 example_tbl 的 p1, p2 分区，以及表 example_tbl2 到仓库 example_repo 中：

```
BACKUP SNAPSHOT example_db.snapshot_label2
TO example_repo
ON
(
    example_tbl PARTITION (p1,p2),
    example_tbl2
);
```

3. 全量备份 example_db 下除了表 example_tbl 的其他所有表到仓库 example_repo 中：

```
BACKUP SNAPSHOT example_db.snapshot_label3
TO example_repo
EXCLUDE (example_tbl);
```

4. 全量备份 example_db 下的表到仓库 example_repo 中：

```
BACKUP SNAPSHOT example_db.snapshot_label3
TO example_repo;
```

7.3.2.3.6 CANCEL BACKUP

描述

该语句用于取消一个正在进行的 BACKUP 任务。

语法

```
CANCEL BACKUP FROM <db_name>;
```

参数

1.<db_name>

备份任务所属数据库名。

示例

1. 取消 example_db 下的 BACKUP 任务。

```
CANCEL BACKUP FROM example_db;
```

7.3.2.3.7 RESTORE

描述

该语句用于将之前通过 BACKUP 命令备份的数据，恢复到指定数据库下。该命令为异步操作。提交成功后，需通过 SHOW RESTORE 命令查看进度。

语法

```
RESTORE SNAPSHOT [<db_name>.]<snapshot_name>
FROM `<repository_name>`
[ { ON | EXCLUDE } ] (
    `<table_name>` [PARTITION (`<partition_name>`, ...)] [AS `<table_alias>`]
    [, ...] ) ]
)
[ PROPERTIES ( "<key>" = "<value>" [ , ... ] )]
```

必选参数

1.<db_name>

需要恢复的数据所属的数据库名

2.<snapshot_name>

数据快照名

3.<repository_name>

仓库名。您可以通过 CREATE REPOSITORY 创建仓库

4.[PROPERTIES ("<key>" = "<value>" [, ...])]

恢复操作属性，格式为 <key> = <value>，目前支持以下属性：

- “backup_timestamp” = “2018-05-04-16-45-08”：指定了恢复对应备份的哪个时间版本，必填。该信息可以通过 SHOW SNAPSHOT ON repo; 语句获得。

- “replication_num” = “3”：指定恢复的表或分区的副本数。默认为 3。若恢复已存在的表或分区，则副本数必须和已存在表或分区的副本数相同。同时，必须有足够的 host 容纳多个副本。
- “reserve_replica” = “true”：默认为 false。当该属性为 true 时，会忽略 replication_num 属性，恢复的表或分区的副本数将与备份之前一样。支持多个表或表内多个分区有不同的副本数。
- “reserve_dynamic_partition_enable” = “true”：默认为 false。当该属性为 true 时，恢复的表会保留该表备份之前的’dynamic_partition_enable’ 属性值。该值不为 true 时，则恢复出来的表的’dynamic_partition_enable’ 属性值会设置为 false。
- “timeout” = “3600”：任务超时时间，默认为一天。单位秒。
- “meta_version” = 40：使用指定的 meta_version 来读取之前备份的元数据。注意，该参数作为临时方案，仅用于恢复老版本 Doris 备份的数据。最新版本的备份数据中已经包含 meta version，无需再指定。
- “clean_tables”：表示是否清理不属于恢复目标的表。例如，如果恢复之前的目标数据库有备份中不存在的表，指定 `clean_tables` 就可以在恢复期间删除这些额外的表并将其移入回收站。该功能自 Apache Doris 2.1.6 版本起支持。
- “clean_partitions”：表示是否清理不属于恢复目标的分区。例如，如果恢复之前的目标表有备份中不存在的分区，指定 `clean_partitions` 就可以在恢复期间删除这些额外的分区并将其移入回收站。该功能自 Apache Doris 2.1.6 版本起支持。
- “atomic_restore”：先将数据加载到临时表中，再以原子方式替换原表，确保恢复过程中不影响目标表的读写。
- “force_replace”：当表存在且架构与备份表不同时，强制替换。
- 注意，要启用 “force_replace”，必须启用 “atomic_restore”

可选参数

1.`<table_name>`

需要恢复的表名。如不指定则恢复整个数据库

- ON 子句中标识需要恢复的表和分区。如果不指定分区，则默认恢复该表的所有分区。所指定的表和分区必须已存在于仓库备份中
- EXCLUDE 子句中标识不需要恢复的表和分区。除了所指定的表或分区之外仓库中所有其他表的所有分区将被恢复

2.`<partition_name>`

需要恢复的分区名。如不指定则恢复对应表的所有分区

3.`<table_alias>`

表别名

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：| 权限 | 对象 | 说明 | |:————|:————|:————–| | LOAD_PRIV | 用户（User）或角色（Role）| 用户或者角色拥有 LOAD_PRIV 权限才能进行此操作 |

注意事项：

- 仅支持恢复 OLAP 类型的表。
- 同一数据库下只能有一个正在执行的 BACKUP 或 RESTORE 任务。

- 可以将仓库中备份的表恢复替换数据库中已有的同名表，但须保证两张表的表结构完全一致。表结构包括：表名、列、分区、Rollup 等等。
- 可以指定恢复表的部分分区，系统会检查分区 Range 或者 List 是否能够匹配。
- 可以通过 AS 语句将仓库中备份的表名恢复为新的表。但新表名不能已存在于数据库中。分区名称不能修改。
- 恢复操作的效率：在集群规模相同的情况下，恢复操作的耗时基本等同于备份操作的耗时。如果想加速恢复操作，可以先通过设置 replication_num 参数，仅恢复一个副本，之后在通过调整副本数ALTER TABLE PROPERTY，将副本补齐。

示例

1. 从 example_repo 中恢复备份 snapshot_1 中的表 backup_tbl 到数据库 example_db1，时间版本为 “2018-05-04-16-45-08”。恢复为 1 个副本：

```
RESTORE SNAPSHOT example_db1.`snapshot_1`
FROM `example_repo`
ON ( `backup_tbl` )
PROPERTIES
(
    "backup_timestamp"="2018-05-04-16-45-08",
    "replication_num" = "1"
);
```

2. 从 example_repo 中恢复备份 snapshot_2 中的表 backup_tbl 的分区 p1,p2，以及表 backup_tbl2 到数据库 example_db1，并重命名为 new_tbl，时间版本为 “2018-05-04-17-11-01”。默认恢复为 3 个副本：

```
RESTORE SNAPSHOT example_db1.`snapshot_2`
FROM `example_repo`
ON
(
    `backup_tbl` PARTITION (`p1`, `p2`),
    `backup_tbl2` AS `new_tbl`
)
PROPERTIES
(
    "backup_timestamp"="2018-05-04-17-11-01"
);
```

3. 从 example_repo 中恢复备份 snapshot_3 中除了表 backup_tbl 的其他所有表到数据库 example_db1，时间版本为 “2018-05-04-18-12-18”。

```
RESTORE SNAPSHOT example_db1.`snapshot_3`
FROM `example_repo`
```

```
EXCLUDE ( `backup_tbl` )
PROPERTIES
(
    "backup_timestamp"="2018-05-04-18-12-18"
);
```

7.3.2.3.8 SHOW RESTORE

描述

该语句用于查看 RESTORE 任务

语法

```
SHOW [BRIEF] RESTORE [FROM <db_name>]
```

参数

1.<db_name>

恢复任务所属数据库名。

返回

- BRIEF: 仅返回精简格式的 RESTORE 任务信息，不包含 RestoreObjs, Progress, TaskErrMsg 三列

列名	说明
JobId	唯一作业 id
Label	要恢复的备份的名称
Timestamp	要恢复的备份的时间版本
DbName	所属数据库

| State | 当前阶段：

PENDING：提交作业后的初始状态。

SNAPSHOTING：执行快照中。

DOWNLOAD：快照完成，准备下载仓库中的快照。

DOWNLOADING：快照下载中。

COMMIT：快照下载完成，准备生效。

COMMITTING：生效中。

FINISHED：作业成功。

CANCELLED：作业失败。

| | AllowLoad | 恢复时是否允许导入（当前不支持）| | ReplicationNum | 指定恢复的副本数 | | ReserveReplica | 是否保留副本 | | ReplicaAllocation | 是否保留动态分区启用 | | RestoreJobs | 要恢复的表和分区 | | CreateTime | 任务提交

时间 | | MetaPreparedTime | 元数据准备完成时间 | | SnapshotFinishedTime | 快照完成时间 | | DownloadFinishedTime | 快照下载完成时间 | | FinishedTime | 作业结束时间 | | UnfinishedTasks | 在 SNAPSHOTING、DOWNLOADING 和 COMMITTING 阶段会显示还未完成的子任务 id | | Progress | 任务进度 | | TaskErrMsg | 显示任务的错误信息 | | Status | 如果作业失败，显示失败信息 | | Timeout | 作业超时时间，单位秒 |

示例

1. 查看 example_db 下最近一次 RESTORE 任务。

```
SHOW RESTORE FROM example_db;
```

7.3.2.3.9 CANCEL RESTORE

描述

该语句用于取消一个正在进行的 RESTORE 任务。

语法

```
CANCEL RESTORE FROM <db_name>;
```

参数

1.<db_name>

恢复任务所属数据库名。

注意事项：

- 当取消处于 COMMIT 或之后阶段的恢复左右时，可能导致被恢复的表无法访问。此时只能通过再次执行恢复作业进行数据恢复。

示例

1. 取消 example_db 下的 RESTORE 任务。

```
CANCEL RESTORE FROM example_db;
```

7.3.2.3.10 SHOW SNAPSHOT

描述

该语句用于查看仓库中已存在的备份。

语法

```
SHOW SNAPSHOT ON `<repo_name>`
[WHERE SNAPSHOT = "<snapshot_name>" [AND TIMESTAMP = "<backup_timestamp>"]];
```

参数

1.<repo_name>

备份所选的仓库名称。

2.<snapshot_name>

备份名称。

3.<backup_timestamp>

备份时间戳。

返回

列名	说明
Snapshot	备份的名称
Timestamp	对应备份的时间版本
Status	如果备份正常，则显示 OK，否则显示错误信息
Database	备份数据原属的数据库名称
Details	以 Json 的形式，展示整个备份的数据目录及文件结构

示例

1. 查看仓库 example_repo 中已有的备份

```
SHOW SNAPSHOT ON example_repo;
```

2. 仅查看仓库 example_repo 中名称为 backup1 的备份：

```
SHOW SNAPSHOT ON example_repo WHERE SNAPSHOT = "backup1";
```

3. 查看仓库 example_repo 中名称为 backup1 的备份，时间版本为 “2018-05-05-15-34-26” 的详细信息：

```
SHOW SNAPSHOT ON example_repo WHERE SNAPSHOT = "backup1" AND TIMESTAMP = "2018-05-05-15-34-26";
```

7.3.3 账户、角色和权限

7.3.3.1 CREATE USER

7.3.3.1.1 描述

CREATE USER 语句用于创建一个 Doris 用户。

7.3.3.1.2 语法

```
CREATE USER [IF EXISTS] <user_identity> [IDENTIFIED BY <password>]
[DEFAULT ROLE <role_name>]
[<password_policy>]
[<comment>]

password_policy:
    1. PASSWORD_HISTORY { <n> | DEFAULT }
    2. PASSWORD_EXPIRE { DEFAULT | NEVER | INTERVAL <n> { DAY | HOUR | SECOND }}
    3. FAILED_LOGIN_ATTEMPTS <n>
    4. PASSWORD_LOCK_TIME { UNBOUNDED | <n> { DAY | HOUR | SECOND }}
```

7.3.3.1.3 必选参数

1. <user_identity>

> 一个用户的唯一标识，语法为：‘user_name’@‘host’ user_identity 由两部分组成，user_name 和 host，其中 username 为用户名。host 标识用户端连接所在的主机地址。host 部分可以使用% 进行模糊匹配。如果不指定 host，默认为 ‘%’，即表示该用户可以从任意 host 连接到 Doris。host 部分也可指定为 domain，，即使用中括号包围，则 Doris 会认为这个是一个 domain，并尝试解析其 ip 地址。

7.3.3.1.4 可选参数

1. <password>

> 指定用户密码

2. <role_name>

> 指定用户角色。如果指定了角色，则会自动将该角色所拥有的权限赋予新创建的这个用户。如果不指定，则该用户默认没有任何权限。指定的 ROLE 必须已经存在。

3. <password_policy>

用于指定密码认证登录相关策略的子句，目前支持以下策略：

PASSWORD_HISTORY { <n> | DEFAULT }

是否允许当前用户重置密码时使用历史密码。如 PASSWORD_HISTORY 10 表示禁止使用过去 10 次设置过的密码为新密码。如果设置为 PASSWORD_HISTORY DEFAULT，则会使用全局变量 password_history 中的值。0 表示不启用这个功能。默认为 0。

PASSWORD_EXPIRE { DEFAULT | NEVER | INTERVAL <n> { DAY | HOUR | SECOND }}

设置当前用户密码的过期时间。如 PASSWORD_EXPIRE INTERVAL 10 DAY 表示密码会在 10 天后过期。PASSWORD_EXPIRE NEVER 表示密码不过期。如果设置为 PASSWORD_EXPIRE DEFAULT，则会使用全局变量 default_password_lifetime 中的值。默认为 NEVER（或 0），表示不会过期。

FAILED_LOGIN_ATTEMPTS <n>

设置当前用户登录时，如果使用错误的密码登录 n 次后，账户将被锁定。如 FAILED_LOGIN_
↪ ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY 表示如果 3 次错误登录，则账户会被锁定。

PASSWORD_LOCK_TIME { UNBOUNDED | <n> { DAY | HOUR | SECOND }}

设置如果账户被锁定，将设置锁定时间。如 PASSWORD_LOCK_TIME 1 DAY 表示账户会被锁定一天。

4. <comment>

指定用户注释

7.3.3.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	全局	

7.3.3.1.6 示例

- 创建一个无密码用户（不指定 host，则等价于 jack@‘%’）

```
CREATE USER 'jack';
```

- 创建一个有密码用户，允许从 ‘172.10.1.10’ 登陆

```
CREATE USER jack@'172.10.1.10' IDENTIFIED BY '123456';
```

- 为了避免传递明文，用例 2 也可以使用下面的方式来创建

```
CREATE USER jack@'172.10.1.10' IDENTIFIED BY PASSWORD '*6BB4837EB74329105EE4568DDA7DC67ED2CA2AD9'
    ↪ ;
后面加密的内容可以通过 PASSWORD() 获得到，例如:
SELECT PASSWORD('123456');
```

- 创建一个允许从 ‘192.168’ 子网登陆的用户，同时指定其角色为 example_role

```
CREATE USER 'jack'@'192.168.%' DEFAULT ROLE 'example_role';
```

- 创建一个允许从域名 ‘example_domain’ 登陆的用户

```
CREATE USER 'jack'@['example_domain'] IDENTIFIED BY '12345';
```

- 创建一个用户，并指定一个角色

```
CREATE USER 'jack'@'%' IDENTIFIED BY '12345' DEFAULT ROLE 'my_role';
```

- 创建一个用户，设定密码 10 天后过期，并且设置如果 3 次错误登录则账户会被锁定一天。

```
CREATE USER 'jack' IDENTIFIED BY '12345' PASSWORD_EXPIRE INTERVAL 10 DAY FAILED_LOGIN_ATTEMPTS 3
    ↪ PASSWORD_LOCK_TIME 1 DAY;
```

- 创建一个用户，并限制不可重置密码为最近 8 次是用过的密码。

```
CREATE USER 'jack' IDENTIFIED BY '12345' PASSWORD_HISTORY 8;
```

- 创建一个用户并添加注释

```
CREATE USER 'jack' COMMENT "this is my first user";
```

7.3.3.2 ALTER USER

7.3.3.2.1 描述

ALTER USER 语句用于修改一个用户的账户属性，包括密码、和密码策略等

7.3.3.2.2 语法

```
ALTER USER [IF EXISTS] <user_identity> [IDENTIFIED BY <password>]
[<password_policy>]
[<comment>]

ALTER USER [IF EXISTS] <user_identity> [IDENTIFIED BY <password>]
[<password_policy>]
[<comment>]

password_policy:
    1. PASSWORD_HISTORY { <n> | DEFAULT }
    2. PASSWORD_EXPIRE { DEFAULT | NEVER | INTERVAL <n> { DAY | HOUR | SECOND }}
    3. FAILED_LOGIN_ATTEMPTS <n>
    4. PASSWORD_LOCK_TIME { UNBOUNDED | <n> { DAY | HOUR | SECOND }}
    5. ACCOUNT_UNLOCK
```

7.3.3.2.3 必选参数

1. <user_identity>

> 一个用户的唯一标识，语法为：‘user_name’@‘host’ user_identity 由两部分组成，user_name 和 host，其中 username 为用户名。host 标识用户端连接所在的主机地址。host 部分可以使用% 进行模糊匹配。如果不指定 host，默认为 ‘%’，即表示该用户可以从任意 host 连接到 Doris。host 部分也可指定为 domain，，即使用中括号包围，则 Doris 会认为这个是一个 domain，并尝试解析其 ip 地址。

7.3.3.2.4 可选参数

1. <password>

> 指定用户密码

2. <password_policy>

> 用于指定密码认证登录相关策略的子句，目前支持以下策略：
>
> PASSWORD_HISTORY { <n> | DEFAULT }

是否允许当前用户重置密码时使用历史密码。如 PASSWORD_HISTORY 10 表示禁止使用过去 10 次设置过的密码为新密码。如果设置为 PASSWORD_HISTORY DEFAULT，则会使用全局变量 password_history 中的值。0 表示不启用这个功能。默认为 0。

PASSWORD_EXPIRE { DEFAULT | NEVER | INTERVAL <n> { DAY | HOUR | SECOND }}

设置当前用户密码的过期时间。如 PASSWORD_EXPIRE INTERVAL 10 DAY 表示密码会在 10 天后过期。PASSWORD_EXPIRE NEVER 表示密码不过期。如果设置为 PASSWORD_EXPIRE DEFAULT，则会使用全局变量 default_password_lifetime 中的值。默认为 NEVER（或 0），表示不会过期。

FAILED_LOGIN_ATTEMPTS <n>

设置当前用户登录时，如果使用错误的密码登录 n 次后，账户将被锁定。如 FAILED_LOGIN_
↪ ATTEMPTS 3 表示如果 3 次错误登录，则账户会被锁定。被锁定的账户可以通过 ALTER USER 语句主动解锁。

PASSWORD_LOCK_TIME { UNBOUNDED | <n> { DAY | HOUR | SECOND }}

设置如果账户被锁定，将设置锁定时间。如 PASSWORD_LOCK_TIME 1 DAY 表示账户会被锁定一天。

ACCOUNT_UNLOCK

解锁用户

3. <comment>

指定用户注释

7.3.3.2.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	全局	

7.3.3.2.6 注意事项

1. 从 2.0 版本开始，此命令不再支持修改用户角色，相关操作请使用 GRANT 和 REVOKE。
2. 在一个 ALTER USER 命令中，只能同时对以下账户属性中的一项进行修改：

- 修改密码
- 修改 PASSWORD_HISTORY
- 修改 PASSWORD_EXPIRE
- 修改 FAILED_LOGIN_ATTEMPTS 和 PASSWORD_LOCK_TIME

- 解锁用户

7.3.3.2.7 示例

- 修改用户的密码

```
ALTER USER jack@'%' IDENTIFIED BY "12345";
```

- 修改用户的密码策略

```
ALTER USER jack@'%' FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 1 DAY;
```

- 解锁一个用户

```
ALTER USER jack@'%' ACCOUNT_UNLOCK
```

- 修改一个用户的注释

```
ALTER USER jack@'%' COMMENT "this is my first user"
```

7.3.3.3 DROP USER

7.3.3.3.1 描述

DROP USER 语句用于删除一个用户。

7.3.3.3.2 语法

```
 DROP USER '<user_identity>'
```

7.3.3.3.3 必选参数

1. <user_identity>

> 指定的用户 identity。

7.3.3.3.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能进行此操作

7.3.3.3.5 示例

1. 删除用户 jack@‘192.%’

```
DROP USER 'jack'@'192.%'
```

7.3.3.4 SET PASSWORD

7.3.3.4.1 描述

SET PASSWORD 语句用于修改一个用户的登录密码。

7.3.3.4.2 语法

```
SET PASSWORD [FOR <user_identity>] =
    [PASSWORD(<plain_password>)]|[<hashed_password>]
```

7.3.3.4.3 必选参数

1. <plain_password>

> 输入的是明文密码，以密码 123456 为例，直接使用字符串123456。

2. <hashed_password>

> 输入的是已加密的密码。以密码 123456 为例，直接使用字符串*6
> ↪ BB4837EB74329105EE4568DDA7DC67ED2CA2AD9，字符串为函数 PASSWORD('123456') 的返回值。

7.3.3.4.4 可选参数

1. <user_identity>

必须完全匹配在使用 CREATE USER 创建用户时指定的 user_identity，否则会报错用户不存在。如果不指定 user_identity，则当前用户为 ‘username’@‘ip’，这个当前用户，可能无法匹配任何 user_identity。可以通过 SHOW GRANTS 查看当前用户。

7.3.3.4.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能修改所有用户的密码，否则只能修改当前用

7.3.3.4.6 注意事项

- 如果 FOR user_identity 字段不存在，那么修改当前用户的密码。

7.3.3.4.7 示例

- 修改当前用户的密码

```
SET PASSWORD = PASSWORD('123456')
SET PASSWORD = '*6BB4837EB74329105EE4568DDA7DC67ED2CA2AD9'
```

- 修改指定用户密码

```
SET PASSWORD FOR 'jack'@'192.%' = PASSWORD('123456')
SET PASSWORD FOR 'jack'@['domain'] = '*6BB4837EB74329105EE4568DDA7DC67ED2CA2AD9'
```

7.3.3.5 CREATE ROLE

7.3.3.5.1 描述

CREATE ROLE 语句用于创建一个无权限的角色，后续可以通过 GRANT 命令赋予该角色权限。

7.3.3.5.2 语法

```
 CREATE ROLE <role_name> [<comment>];
```

7.3.3.5.3 必选参数

1. <role_name>：

> 指定角色名称。

7.3.3.5.4 可选参数

2. <comment>

> 指定角色注释。

7.3.3.5.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	全局	

7.3.3.5.6 示例

- 创建一个角色

```
CREATE ROLE role1;
```

- 创建一个角色并添加注释

```
CREATE ROLE role2 COMMENT "this is my first role";
```

7.3.3.6 ALTER ROLE

7.3.3.6.1 描述

ALTER ROLE 语句用于修改一个角色的注释

7.3.3.6.2 语法

```
 ALTER ROLE <role_name> COMMENT <comment>;
```

7.3.3.6.3 必选参数

1. <role_name>

> 指定角色名称。

2. <comment>

> 指定角色注释。

7.3.3.6.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	全局	

7.3.3.6.5 示例

- 修改一个角色的注释

```
ALTER ROLE role1 COMMENT "this is my first role";
```

7.3.3.7 DROP ROLE

7.3.3.7.1 描述

DROP ROLE 语句用于用户删除角色

7.3.3.7.2 语法

```
  DROP ROLE [IF EXISTS] <role_name>;
```

7.3.3.7.3 必选参数

1. <role_name>

指定角色名称。

7.3.3.7.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能进行此操作

7.3.3.7.5 注意事项

- 删除角色不会影响以前属于角色的用户的权限。它仅相当于解耦来自用户的角色。用户从角色获得的权限不会改变。

7.3.3.7.6 示例

- 删除一个角色

```
DROP ROLE role1;
```

7.3.3.8 SHOW ROLES

7.3.3.8.1 描述

SHOW ROLES 语句用于展示所有已创建的角色信息，包括角色名称，包含的用户以及权限。

7.3.3.8.2 语法

```
SHOW ROLES
```

7.3.3.8.3 返回值

列名	类型	说明
Name	string	角色名称
Comment	string	注释
Users	string	包含的用户
GlobalPrivs	string	全局权限
CatalogPrivs	string	Catalog 权限

列名	类型	说明
DatabasePrivs	string	数据库权限
TablePrivs	string	表权限
ResourcePrivs	string	资源权限
WorkloadGroupPrivs	string	WorkloadGroup 权限

7.3.3.8.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
GRANT_PRIV	用户（User）或角色（Role）	用户或者角色拥有 GRANT_PRIV 权限才能进行此操作

7.3.3.8.5 注意事项

Doris 会为每个用户创建一个默认角色，如果想展示出默认角色，可以 `set show_user_default_role=true;`

7.3.3.8.6 示例

- 查看已创建的角色

```
SHOW ROLES
```

7.3.3.9 GRANT TO

7.3.3.9.1 描述

GRANT 命令用于：

1. 将指定的权限授予某用户或角色。
2. 将指定角色授予某用户。

相关命令

- REVOKE FROM
- SHOW GRANTS
- CREATE ROLE
- CREATE WORKLOAD GROUP
- CREATE RESOURCE
- CREATE STORAGE VAULT

7.3.3.9.2 语法

将指定的权限授予某用户或角色

```
GRANT <privilege_list>
ON { <priv_level>
    | RESOURCE <resource_name>
    | WORKLOAD GROUP <workload_group_name>
    | COMPUTE GROUP <compute_group_name>
    | STORAGE VAULT <storage_vault_name>
   }
TO { <user_identity> | ROLE <role_name> }
```

将指定角色授予某用户

```
GRANT <role_list> TO <user_identity>
```

7.3.3.9.3 必选参数

1. <privilege_list>

需要赋予的权限列表，以逗号分隔。当前支持如下权限：

- NODE_PRIV：集群节点操作权限，包括节点上下线等操作。
- ADMIN_PRIV：除 NODE_PRIV 以外的所有权限。
- GRANT_PRIV：操作权限的权限，包括创建删除用户、角色，授权和撤权，设置密码等。
- SELECT_PRIV：对指定的库或表的读取权限。
- LOAD_PRIV：对指定的库或表的导入权限。
- ALTER_PRIV：对指定的库或表的 schema 变更权限。
- CREATE_PRIV：对指定的库或表的创建权限。
- DROP_PRIV：对指定的库或表的删除权限。
- USAGE_PRIV：对指定资源、 Workload Group、 Compute Group 的使用权限。
- SHOW_VIEW_PRIV：查看 view 创建语句的权限。

旧版权限转换：- ALL 和 READ_WRITE 会被转换成：SELECT_PRIV, LOAD_PRIV, ALTER_PRIV, CREATE_PRIV, DROP_PRIV。 - READ_ONLY 会被转换为 SELECT_PRIV。

2. <priv_level>

支持以下四种形式：

- ..*：权限可以应用于所有 catalog 及其中的所有库表。
- catalog_name..：权限可以应用于指定 catalog 中的所有库表。
- catalog_name.db.*：权限可以应用于指定库下的所有表。
- catalog_name.db.tbl：权限可以应用于指定库下的指定表。

3. <resource_name>

指定 resource 名，支持% 和 * 匹配所有资源，不支持通配符，比如 res*。

4. <workload_group_name>

指定 workload group 名，支持% 和 * 匹配所有 workload group，不支持通配符。

5. <compute_group_name>

指定 compute group 名称，支持% 和 * 匹配所有 compute group，不支持通配符。

6. <storage_vault_name>

指定 storage vault 名称，支持% 和 * 匹配所有 storage vault，不支持通配符。

7. <user_identity>

指定接收权限的用户。必须为使用 CREATE USER 创建过的 user_identity。 user_identity 中的 host 可以是域名，如果是域名的话，权限的生效时间可能会有 1 分钟左右的延迟。

8. <role_name>

指定接收权限的角色。如果指定的角色不存在，则会自动创建。

9. <role_list>

需要赋予的角色列表，以逗号分隔，指定的角色必须存在。

7.3.3.9.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	用户（User）或角色（Role）	用户或者角色拥有 GRANT_PRIV 权限才能进行 GRANT 操作

7.3.3.9.5 示例

- 授予所有 catalog 和库表的权限给用户：

```
GRANT SELECT_PRIV ON *.*.* TO 'jack'@'%';
```

- 授予指定库表的权限给用户：

```
GRANT SELECT_PRIV,ALTER_PRIV,LOAD_PRIV ON ctl1.db1.tbl1  TO 'jack'@'192.8.%';
```

- 授予指定库表的权限给角色：

```
GRANT LOAD_PRIV ON ctl1.db1.* TO ROLE 'my_role';
```

- 授予所有 resource 的使用权限给用户：

```
GRANT USAGE_PRIV ON RESOURCE * TO 'jack'@'%';
```

- 授予指定 resource 的使用权限给用户：

```
GRANT USAGE_PRIV ON RESOURCE 'spark_resource' TO 'jack'@'%';
```

- 授予指定 resource 的使用权限给角色：

```
GRANT USAGE_PRIV ON RESOURCE 'spark_resource' TO ROLE 'my_role';
```

- 将指定 role 授予某用户：

```
GRANT 'role1','role2' TO 'jack'@'%';
```

- 将指定 workload group 授予用户：

```
GRANT USAGE_PRIV ON WORKLOAD GROUP 'g1' TO 'jack'@'%';
```

- 匹配所有 workload group 授予用户：

```
GRANT USAGE_PRIV ON WORKLOAD GROUP '%' TO 'jack'@'%';
```

- 将指定 workload group 授予角色：

```
GRANT USAGE_PRIV ON WORKLOAD GROUP 'g1' TO ROLE 'my_role';
```

- 允许用户查看指定 view 的创建语句：

```
GRANT SHOW_VIEW_PRIV ON db1.view1 TO 'jack'@'%';
```

- 授予用户对指定 compute group 的使用权限：

```
GRANT USAGE_PRIV ON COMPUTE GROUP 'group1' TO 'jack'@'%';
```

- 授予角色对指定 compute group 的使用权限：

```
GRANT USAGE_PRIV ON COMPUTE GROUP 'group1' TO ROLE 'my_role';
```

- 授予用户对所有 compute group 的使用权限：

```
GRANT USAGE_PRIV ON COMPUTE GROUP '*' TO 'jack'@'%';
```

- 授予用户对指定 storage vault 的使用权限：

```
GRANT USAGE_PRIV ON STORAGE VAULT 'vault1' TO 'jack'@'%';
```

- 授予角色对指定 storage vault 的使用权限：

```
GRANT USAGE_PRIV ON STORAGE VAULT 'vault1' TO ROLE 'my_role';
```

- 授予用户对所有 storage vault 的使用权限：

```
GRANT USAGE_PRIV ON STORAGE VAULT '*' TO 'jack'@'%';
```

7.3.3.10 REVOKE FROM

7.3.3.10.1 描述

REVOKE 命令用于：

1. 撤销某用户或某角色的指定权限。
2. 撤销先前授予某用户的指定角色。

相关命令

- GRANT TO
- SHOW GRANTS
- CREATE ROLE
- CREATE WORKLOAD GROUP
- CREATE RESOURCE
- CREATE STORAGE VAULT

7.3.3.10.2 语法

撤销某用户或某角色的指定权限

```
REVOKE <privilege_list>
ON { <priv_level>
   | RESOURCE <resource_name>
   | WORKLOAD GROUP <workload_group_name>
   | COMPUTE GROUP <compute_group_name>
   | STORAGE VAULT <storage_vault_name>
   }
FROM { <user_identity> | ROLE <role_name> }
```

撤销先前授予某用户的指定角色

```
REVOKE <role_list> FROM <user_identity>
```

7.3.3.10.3 必选参数

1. <privilege_list>

需要撤销的权限列表，以逗号分隔。支持的权限包括：

- NODE_PRIV：集群节点操作权限
- ADMIN_PRIV：管理员权限
- GRANT_PRIV：授权权限
- SELECT_PRIV：查询权限
- LOAD_PRIV：数据导入权限
- ALTER_PRIV：修改权限
- CREATE_PRIV：创建权限
- DROP_PRIV：删除权限
- USAGE_PRIV：使用权限
- SHOW_VIEW_PRIV：查看视图定义权限

2. <priv_level>

指定权限的作用范围。支持以下格式：

- ..*：所有 catalog、数据库和表
- catalog_name..：指定 catalog 中的所有数据库和表
- catalog_name.db.*：指定数据库中的所有表
- catalog_name.db.tbl：指定数据库中的特定表

3. <resource_name>

指定 resource 名称。支持% （匹配任意字符串）和 _（匹配任意单个字符）通配符。

4. <workload_group_name>

指定 workload group 名称。支持% （匹配任意字符串）和 _（匹配任意单个字符）通配符。

5. <compute_group_name>

指定 compute group 名称。支持% （匹配任意字符串）和 _（匹配任意单个字符）通配符。

6. <storage_vault_name>

指定 storage vault 名称。支持% （匹配任意字符串）和 _（匹配任意单个字符）通配符。

7. <user_identity>

指定要撤销权限的用户。必须是使用 CREATE USER 创建的用户。user_identity 中的 host 可以是域名，如果是域名，权限的撤销时间可能会有 1 分钟左右的延迟。

8. <role_name>

指定要撤销权限的角色。该角色必须存在。

9. <role_list>

需要撤销的角色列表，以逗号分隔。指定的所有角色必须存在。

7.3.3.10.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	用户（User）或角色（Role）	用户或者角色拥有 GRANT_PRIV 权限才能进行 GRANT 操作

7.3.3.10.5 示例

- 撤销用户在特定数据库上的 SELECT 权限：

```
REVOKE SELECT_PRIV ON db1.* FROM 'jack'@'192.%';
```

- 撤销用户对资源的使用权限：

```
REVOKE USAGE_PRIV ON RESOURCE 'spark_resource' FROM 'jack'@'192.%';
```

- 撤销用户的角色：

```
REVOKE 'role1','role2' FROM 'jack'@'192.%';
```

- 撤销用户对 workload group 的使用权限：

```
REVOKE USAGE_PRIV ON WORKLOAD GROUP 'g1' FROM 'jack'@'%';
```

- 撤销用户对所有 workload group 的使用权限：

```
REVOKE USAGE_PRIV ON WORKLOAD GROUP '%' FROM 'jack'@'%';
```

- 撤销角色对 workload group 的使用权限：

```
REVOKE USAGE_PRIV ON WORKLOAD GROUP 'g1' FROM ROLE 'test_role';
```

- 撤销用户对 compute group 的使用权限：

```
REVOKE USAGE_PRIV ON COMPUTE GROUP 'group1' FROM 'jack'@'%';
```

- 撤销角色对 compute group 的使用权限：

```
REVOKE USAGE_PRIV ON COMPUTE GROUP 'group1' FROM ROLE 'my_role';
```

- 撤销用户对 storage vault 的使用权限：

```
REVOKE USAGE_PRIV ON STORAGE VAULT 'vault1' FROM 'jack'@'%';
```

- 撤销角色对 storage vault 的使用权限：

```
REVOKE USAGE_PRIV ON STORAGE VAULT 'vault1' FROM ROLE 'my_role';
```

- 撤销用户对所有 storage vault 的使用权限：

```
REVOKE USAGE_PRIV ON STORAGE VAULT '%' FROM 'jack'@'%';
```

7.3.3.11 SHOW PRIVILEGES

7.3.3.11.1 描述

SHOW PRIVILEGES 语句用于显示数据库系统中当前可用的权限列表。它帮助用户了解系统支持的权限类型以及每种权限的详细信息。

7.3.3.11.2 语法

```
SHOW PRIVILEGES
```

7.3.3.11.3 返回值

列名	说明
Privilege	权限名
Context	可作用范围
Comment	说明

7.3.3.11.4 权限控制

执行此 SQL 命令的用户不需要具有特定的权限。

7.3.3.11.5 示例

查看所有权限项

```
SHOW PRIVILEGES
```

```
+-------------+-------------------------------------------------------+-------------------------------------
    ↪
| Privilege   | Context                                               | Comment
    ↪                                      |
+-------------+-------------------------------------------------------+-------------------------------------
    ↪
| Node_priv   | GLOBAL                                                | Privilege for cluster
    ↪ node operations         |
| Admin_priv  | GLOBAL                                                | Privilege for admin user
    ↪                        |
| Grant_priv  | GLOBAL,CATALOG,DATABASE,TABLE,RESOURCE,WORKLOAD GROUP | Privilege for granting
    ↪ privilege              |
| Select_priv | GLOBAL,CATALOG,DATABASE,TABLE                         | Privilege for select data
    ↪  in tables           |
| Load_priv   | GLOBAL,CATALOG,DATABASE,TABLE                         | Privilege for loading
    ↪ data into tables        |
| Alter_priv  | GLOBAL,CATALOG,DATABASE,TABLE                         | Privilege for alter
    ↪ database or table         |
| Create_priv | GLOBAL,CATALOG,DATABASE,TABLE                         | Privilege for creating
    ↪ database or table      |
| Drop_priv   | GLOBAL,CATALOG,DATABASE,TABLE                         | Privilege for dropping
    ↪ database or table      |
| Usage_priv  | RESOURCE,WORKLOAD GROUP                               | Privilege for using
    ↪ resource or workloadGroup |
+-------------+-------------------------------------------------------+-------------------------------------
    ↪
```

7.3.3.12 SHOW GRANTS

7.3.3.12.1 描述

该语句用于查看用户权限。

7.3.3.12.2 语法

```
SHOW [ALL] GRANTS [FOR <user_identity>];
```

7.3.3.12.3 可选参数

1. [ALL]

是否查看所有用户的权限。

2. <user_identity>

指定要查看权限的用户。必须为通过 CREATE USER 命令创建的 user_identity。

7.3.3.12.4 返回值

列名	说明
UserIdentity	用户标识
Comment	注释
Password	是否设置密码
Roles	拥有的角色
GlobalPrivs	拥有的全局权限
CatalogPrivs	拥有的 catalog 权限
DatabasePrivs	拥有的数据库权限
TablePrivs	拥有的表权限
ColPrivs	拥有的列权限
ResourcePrivs	拥有的资源权限
WorkloadGroupPrivs	拥有的 WorkloadGroup 权限

7.3.3.12.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	用户（User）或角色（Role）	用户或者角色拥有 GRANT_PRIV 权限才能查看所有用户权限操作，否则只能查看

7.3.3.12.6 注意事项

- SHOW ALL GRANTS 可以查看所有用户的权限，但需要有 GRANT_PRIV 权限。
- 如果指定 user_identity，则查看该指定用户的权限。且该 user_identity 必须为通过 CREATE USER 命令创建的。
- 如果不指定 user_identity，则查看当前用户的权限。
- Doris 基于 RBAC（Role-Based Access Control）的权限管理模型进行权限控制，因此这里展示出来的权限其实是用户所有角色的权限合集，如果想查看具体权限来源于哪个角色，可以通过 SHOW ROLES 查看

7.3.3.12.7 示例

1. 查看所有用户权限信息

```
SHOW ALL GRANTS;
```

```
+--------------+---------+----------+----------+---------------------+--------------+-----------------------
    ↪
| UserIdentity | Comment | Password | Roles    | GlobalPrivs         | CatalogPrivs |
    ↪ DatabasePrivs                                                          | TablePrivs |
    ↪ ColPrivs | ResourcePrivs | WorkloadGroupPrivs |
+--------------+---------+----------+----------+---------------------+--------------+-----------------------
    ↪
| 'root'@'%'   | ROOT    | No       | operator | Node_priv,Admin_priv | NULL         |
    ↪ internal.information_schema: Select_priv; internal.mysql: Select_priv | NULL       |
    ↪ NULL     | NULL          | normal: Usage_priv |
| 'admin'@'%'  | ADMIN   | No       | admin    | Admin_priv           | NULL         |
    ↪ internal.information_schema: Select_priv; internal.mysql: Select_priv | NULL       |
    ↪ NULL     | NULL          | normal: Usage_priv |
| 'jack'@'%'   |         | No       |          | NULL                 | NULL         |
    ↪ internal.information_schema: Select_priv; internal.mysql: Select_priv | NULL       |
    ↪ NULL     | NULL          | normal: Usage_priv |
+--------------+---------+----------+----------+---------------------+--------------+-----------------------
    ↪
```

2. 查看指定 user 的权限

```
SHOW GRANTS FOR jack@'%';
```

```
+--------------+---------+----------+-------+-------------+--------------+-----------------------------------
    ↪
| UserIdentity | Comment | Password | Roles | GlobalPrivs | CatalogPrivs | DatabasePrivs
    ↪                                                          | TablePrivs | ColPrivs |
    ↪ ResourcePrivs | WorkloadGroupPrivs |
+--------------+---------+----------+-------+-------------+--------------+-----------------------------------
    ↪
| 'jack'@'%'   |         | No       |       | NULL        | NULL         | internal.
    ↪ information_schema: Select_priv; internal.mysql: Select_priv | NULL       | NULL
    ↪ | NULL          | normal: Usage_priv |
+--------------+---------+----------+-------+-------------+--------------+-----------------------------------
    ↪
```

3. 查看当前用户的权限

```
SHOW GRANTS;
```

```
+--------------+---------+----------+----------+---------------------+-------------+-------------------------
    ↪
| UserIdentity | Comment | Password | Roles    | GlobalPrivs          | CatalogPrivs |
    ↪ DatabasePrivs                                                          | TablePrivs |
    ↪ ColPrivs | ResourcePrivs | WorkloadGroupPrivs |
+--------------+---------+----------+----------+---------------------+-------------+-------------------------
    ↪
| 'root'@'%'   | ROOT    | No       | operator | Node_priv,Admin_priv | NULL         |
    ↪ internal.information_schema: Select_priv; internal.mysql: Select_priv | NULL       |
    ↪ NULL     | NULL          | normal: Usage_priv |
+--------------+---------+----------+----------+---------------------+-------------+-------------------------
    ↪
```

7.3.3.13 REFRESH LDAP

7.3.3.13.1 描述

该语句用于刷新 Doris 中 LDAP 的缓存信息。修改 LDAP 服务中用户信息或者修改 Doris 中 LDAP 用户组对应的 role 权限，可能因为缓存的原因不会立即生效，可通过该语句刷新缓存。

7.3.3.13.2 语法

```
REFRESH LDAP [ALL | FOR <user_name>];
```

7.3.3.13.3 可选参数

1. [ALL]

是否刷新所有用户的 LDAP 缓存信息。

2. <user_name>

指定要刷新 LDAP 缓存信息的用户。

7.3.3.13.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能刷新所有用户的 LDAP 缓存信息，否则只能刷新当前用户的 LDAP 缓存信息

7.3.3.13.5 注意事项

- Doris 中 LDAP 信息缓存默认时间为 12 小时，可以通过 `SHOW FRONTEND CONFIG LIKE 'ldap_user_cache_timeout_s'`; 查看。
- `REFRESH LDAP ALL` 刷新所有用户的 LDAP 缓存信息，但需要有 `ADMIN_PRIV` 权限。
- 如果指定 `user_name`，则刷新指定用户的 LDAP 缓存信息。
- 如果不指定 `user_name`，则刷新当前用户的 LDAP 缓存信息。

7.3.3.13.6 示例

1. 刷新所有 LDAP 用户缓存信息

```
REFRESH LDAP ALL;
```

2. 刷新当前 LDAP 用户的缓存信息

```
REFRESH LDAP;
```

3. 刷新指定 LDAP 用户 jack 的缓存信息

```
REFRESH LDAP FOR jack;
```

7.3.3.14 SET PROPERTY

7.3.3.14.1 描述

`SET PROPERTY` 语句用于设置用户属性，包括分配给用户的资源和导入集群设置。

相关命令

- CREATE USER
- SHOW PROPERTY

7.3.3.14.2 语法

```
SET PROPERTY [ FOR '<user_name>' ] '<key_1>' = '<value_1>' [, '<key_2>' = '<value_2>', ...];
```

7.3.3.14.3 必选参数

1. <key_n>

要设置的属性键。可用的键包括：

- max_user_connections：最大连接数。
- max_query_instances：用户同一时间点执行查询可以使用的 instance 个数。
- sql_block_rules：设置 SQL 阻止规则。设置后，该用户发送的查询如果匹配规则，则会被拒绝。
- cpu_resource_limit：限制查询的 CPU 资源。详见会话变量 cpu_resource_limit 的介绍。-1 表示未设置。
- exec_mem_limit：限制查询的内存使用。详见会话变量 exec_mem_limit 的介绍。-1 表示未设置。
- resource_tags：指定用户的资源标签权限。
- query_timeout：指定用户的查询超时。
- default_workload_group：指定用户的默认工作负载组。
- default_compute_group：指定用户的默认计算组。

注：如果未设置 cpu_resource_limit 和 exec_mem_limit，则默认使用会话变量中的值。

2. <value_n>

为指定键设置的值。

7.3.3.14.4 可选参数

1. <user_name>

要设置属性的用户名。如果省略，则为当前用户设置属性。

7.3.3.14.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能进行 SET PROPERTY 操作

7.3.3.14.6 注意事项

- 用户属性是针对用户的，而不是针对 user_identity。例如，如果通过 CREATE USER 语句创建了两个用户 'jack'@'%' 和 'jack'@'192.%'，则使用 SET PROPERTY 语句只能针对用户 'jack'，而不是 'jack'@'%' 或 'jack'@'192.%'。

7.3.3.14.7 示例

- 设置用户 ‘jack’ 的最大连接数为 1000：

```
SET PROPERTY FOR 'jack' 'max_user_connections' = '1000';
```

- 设置用户 'jack' 的最大查询实例数为 3000：

```
SET PROPERTY FOR 'jack' 'max_query_instances' = '3000';
```

- 为用户 'jack' 设置 SQL 阻止规则：

```
SET PROPERTY FOR 'jack' 'sql_block_rules' = 'rule1, rule2';
```

- 设置用户 'jack' 的 CPU 资源限制：

```
SET PROPERTY FOR 'jack' 'cpu_resource_limit' = '2';
```

- 设置用户 'jack' 的资源标签权限：

```
SET PROPERTY FOR 'jack' 'resource_tags.location' = 'group_a, group_b';
```

- 设置用户 'jack' 的内存使用限制（以字节为单位）：

```
SET PROPERTY FOR 'jack' 'exec_mem_limit' = '2147483648';
```

- 设置用户 'jack' 的查询超时时间（以秒为单位）：

```
SET PROPERTY FOR 'jack' 'query_timeout' = '500';
```

- 设置用户 'jack' 的默认工作负载组：

```
SET PROPERTY FOR 'jack' 'default_workload_group' = 'group1';
```

- 设置用户 'jack' 的默认计算组：

```
SET PROPERTY FOR 'jack' 'default_compute_group' = 'compute_group1';
```

7.3.3.15 SHOW PROPERTY

7.3.3.15.1 描述

该语句用于查看用户的属性

7.3.3.15.2 语法

```
SHOW {ALL PROPERTIES | PROPERTY [FOR <user_name>]} [LIKE <key>]
```

7.3.3.15.3 可选参数

1. [ALL PROPERTIES]

是否查看所有用户的属性。

2. <user_name>

查看指定用户的属性。如果未指定，检查当前用户的。

3. <key>

模糊匹配可以通过属性名来完成。

7.3.3.15.4 返回值

- 若语句中使用的是PROPERTY

列名	说明
Key	属性名
Value	属性值

- 若语句中使用的是PROPERTIES

列名	说明
User	用户名
Properties	对应用户各个 property 的 key:value

7.3.3.15.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	用户（User）或角色（Role）	用户或者角色拥有 GRANT_PRIV 权限才能查看所有用户属性， SHOW PROPERTY查看当前用户属性不需要GRANT_PRIV权限

7.3.3.15.6 注意事项

- SHOW ALL PROPERTIES 可以查看所有用户的属性。
- 如果指定 user_name，则查看该指定用户的属性。
- 如果不指定 user_name，则查看当前用户的属性。
- SHOW PROPERTY查看当前用户属性不需要GRANT_PRIV权限。

7.3.3.15.7 示例

- 查看 jack 用户的属性

```
SHOW PROPERTY FOR 'jack';
```

```
+-----------------------------------+--------+
| Key                               | Value  |
+-----------------------------------+--------+
| cpu_resource_limit                | -1     |
| default_load_cluster              |        |
| default_workload_group            | normal |
| exec_mem_limit                    | -1     |
| insert_timeout                    | -1     |
| max_query_instances               | 3000   |
| max_user_connections              | 1000   |
| parallel_fragment_exec_instance_num | -1     |
| query_timeout                     | -1     |
| resource_tags                     |        |
| sql_block_rules                   |        |
+-----------------------------------+--------+
```

- 查看 jack 用户 limit 相关属性

```
SHOW PROPERTY FOR 'jack' LIKE '%limit%';
```

```
+--------------------+-------+
| Key                | Value |
+--------------------+-------+
| cpu_resource_limit | -1    |
| exec_mem_limit     | -1    |
+--------------------+-------+
```

- 查看所有用户 limit 相关属性

```
SHOW ALL PROPERTIES LIKE '%limit%';
```

```
+-------+------------------------------------------------------------+
| User  | Properties                                                 |
+-------+------------------------------------------------------------+
| root  | {
  "cpu_resource_limit": "-1",
  "exec_mem_limit": "-1"
} |
| admin | {
  "cpu_resource_limit": "-1",
  "exec_mem_limit": "-1"
} |
| jack  | {
  "cpu_resource_limit": "-1",
  "exec_mem_limit": "-1"
} |
+-------+------------------------------------------------------------+
```

7.3.4 会话

7.3.4.1 上下文

7.3.4.1.1 USE COMPUTE GROUP

描述

在存算分离版本中，指定使用计算集群

语法

```
USE { [ <catalog_name>. ]<database_name>[ @<compute_group_name> ] | @<compute_group_name> }
```

必选参数

<compute_group_name>**：计算集群名字**

返回值

切换计算集群成功返回“Database changed**”，切换失败返回相应错误提示信息**

示例

1. **指定使用该计算集群** compute_cluster

```
use @compute_cluster;
Database changed
```

2. **同时指定使用该数据库** mysql **和计算集群** compute_cluster

```
use mysql@compute_cluster
Database changed
```

权限控制

执行此 SQL 命令成功的前置条件是，拥有 compute group 的使用权限 USAGE_PRIV，参考权限文档。

权限（Privilege）	对象（Object）	说明（Notes）
USAGE_PRIV	Compute group	计算集群使用权限

若用户无 compute group 权限，而去指定 compute group 会报错，注 test 为普通用户无集群使用权限

```
mysql -utest -h175.40.1.1 -P9030

use @compute_cluster;
ERROR 5042 (42000): errCode = 2, detailMessage = USAGE denied to user test'@'127.0.0.1' for
    ↪ compute group 'compute_cluster'
```

注意事项

1. 如果 database 名字或者 compute group 名字是保留的关键字，需要用反引号，例如：

```
use @`create`
```

2. 若 compute group 不存在，返回报错信息

```
mysql> use @compute_group_not_exist;
ERROR 5098 (42000): errCode = 2, detailMessage = Compute Group compute_group_not_exist not
    ↪ exist
```

7.3.4.1.2 SWITCH CATALOG

描述

该语句用于切换数据目录（catalog）。

语法

```
SWITCH <catalog_name>
```

必选参数

1. <catalog_name> > 要切换的数据目录名称。

权限控制

权限	对象	说明
SELECT_PRIV	数据目录	需要对要切换的数据目录（catalog）有 SELECT_PRIV 权限。

示例

1. 切换到数据目录 hive

```
SWITCH hive;
```

7.3.4.1.3 USE

描述

用于切换到指定的数据库或计算组。

语法

```
USE { [<catalog_name>.]<database_name>[@<compute_group_name>] | @<compute_group_name> }
```

必选参数

切换到指定的数据库。

1. <database_name> > 要切换的数据库名称。> 如果未指定数据目录，则默认为当前数据目录。

只切换到指定的计算组。

1. <compute_group_name> > 要切换的计算组名称。

可选参数

切换到指定的数据库。

1. <catalog_name> > 要切换的数据目录名称。

2. <compute_group_name> > 要切换的计算组名称。

权限控制

权限	对象	说明
SELECT_PRIV	数据目录、数据库	需要对要切换的数据目录、数据有 SELECT_PRIV 权限。
USAGE_PRIV	计算组	需要对要切换的计算组有 USAGE_PRIV 权限。

示例

1. 如果 demo 数据库存在，尝试使用它：

```
use demo;
```

2. 如果 demo 数据库在 hms_catalog 的 Catalog 下存在，尝试切换到 hms_catalog, 并使用它：

```
use hms_catalog.demo;
```

3. 如果 demo 数据库在当前目录中存在，并且您想使用名为 ‘cg1’ 的计算组，请尝试访问它：

```
use demo@cg1;
```

4. 如果您只想使用名为 ‘cg1’ 的计算组，请尝试访问它：

```
use @cg1;
```

7.3.4.2 变量

7.3.4.2.1 SET VARIABLE

描述

该语句主要是用来修改 Doris 系统变量，这些系统变量可以分为全局以及会话级别层面来修改，有些也可以进行动态修改。你也可以通过 SHOW VARIABLE 来查看这些系统变量。

语法

```
SET variable_assignment [, variable_assignment] [ ... ]
```

其中

```
variable_assignment
  : <user_var_name> = <expr>
  | [ <effective_scope> ] <system_var_name> = <expr>
```

必选参数

1. <user_var_name> > 指定用户层级的变量，比如：@@your_variable_name 等以@@开头的变量名称

2. <system_var_name> > 指定系统层级的变量，比如 exec_mem_limit 等

可选参数

1. <effective_scope>

> 生效范围的取值可以是GLOBAL或者SESSION或者LOCAL之一，如果不指定该值，默认为SESSION。LOCAL是SESSION的一个别名。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

Privilege	Object	Notes
ADMIN_PRIV	Session	set global variables 需要 admin 权限

注意事项

- 只有 ADMIN 用户可以设置变量的全局生效
- 全局生效的变量影响当前会话和此后的新会话，不影响当前已经存在的其他会话。

示例

- 设置时区为东八区

```
SET time_zone = "Asia/Shanghai";
```

- 设置全局的执行内存大小

```
SET GLOBAL exec_mem_limit = 137438953472
```

- 设置用户变量

```
SET @@your_variable_name = your_variable_value;
```

7.3.4.2.2 UNSET VARIABLE

描述

该语句主要是用来恢复 Doris 系统变量为默认值，可以是全局也可以是会话级别。

语法

```
UNSET [<effective_scope>] VARIABLE (<variable_name>)
```

必选参数

1. <variable_name> > 指定变量名称，如果需要 unset 全部变量，可以写一个ALL关键字

可选参数

2. <effective_scope> > 生效范围的取值可以是GLOBAL或者SESSION或者LOCAL之一，如果不指定该值，默认为SESSION。 LOCAL是SESSION的一个别名。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

Privilege	Object	Notes
ADMIN_PRIV	Session	unset global variables 需要 admin 权限

注意事项

- 只有 ADMIN 用户可以设置变量的全局生效
- 使用 GLOBAL 恢复变量值时仅在执行命令的当前会话和之后打开的会话中生效，不会恢复当前已有的其它会话中的值。

示例

- 恢复时区为默认值东八区

```
UNSET VARIABLE time_zone;
```

- 恢复全局的执行内存大小

```
UNSET GLOBAL VARIABLE exec_mem_limit;
```

- 从全局范围恢复所有变量的值

```
UNSET GLOBAL VARIABLE ALL;
```

7.3.4.2.3 SHOW VARIABLES

描述

该语句是用来显示 Doris 系统变量，可以通过条件查询

语法

```
SHOW [<effective_scope>] VARIABLES [<like_pattern> | <where>]
```

可选参数

1. <effective_scope> > 生效范围的取值可以是GLOBAL或者SESSION或者LOCAL之一，如果不指定该值，默认为SESSION。LOCAL是SESSION的一个别名。

2. <like_pattern> > 使用 like 语句去匹配和过滤最终结果

3. <where> > 使用 where 语句去匹配和过滤最终结果

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

Privilege	Object	Notes
Any_PRIV	Session	SHOW VARIABLES 命令不需要任何权限

返回值

Variable_name	Value	Default_Value	Changed
variable name1	value1	default value1	0/1
variable name2	value2	default value2	0/1

注意事项

- show variables 主要是用来查看系统变量的值。
- 执行 SHOW VARIABLES 命令不需要任何权限，只要求能够连接到服务器就可以。
- 返回值部分中的Changed列，0 表示没有改变过，1 表示改变过。
- 使用SHOW语句的一些限制：
- where 语法中不能使用or语句
- 列名在等值左侧
- 只支持等值连接
- 使用 like 语句表示用 variable_name 进行匹配。
- % 百分号通配符可以用在匹配模式中的任何位置。

示例

- 这里默认的就是对 Variable_name 进行匹配，这里是准确匹配

```
show variables like 'max_connections';
```

- 通过百分号 (%) 这个通配符进行匹配，可以匹配多项

```
show variables like '%connec%';
```

- 使用 Where 子句进行匹配查询

```
show variables where variable_name = 'version';
```

7.3.4.3 查询

7.3.4.3.1 SHOW PROCESSLIST

描述

显示用户正在运行的线程。该命令只能看到当前 FE 的连接信息，如果需要看到整个集群的连接信息，则需要添加 session variable:

```
SET SHOW_ALL_FE_CONNECTION = TRUE;
```

语法

```
SHOW [FULL] PROCESSLIST
```

可选参数

1. FULL

> 表示是否查看其他用户的连接信息

返回值

列名	说明
CurrentConnected	是否为当前连接
Id	这个线程的唯一标识
User	启动这个线程的用户
Host	记录了发送请求的客户端的 IP 和端口号
LoginTime	建立连接的时间
Catalog	当前执行的命令是在哪一个数据目录上
Db	当前执行的命令是在哪一个数据库上，如果没有指定数据库，则该值为 NULL
Command	此刻该线程正在执行的命令
Time	上一条命令提交到当前状态的时间，单位为秒
State	线程的状态
QueryId	当前查询语句的 ID
Info	一般记录的是线程执行的语句，默认只显示前 100 个字符

常见的 Command 类型如下：

列名	说明
Query	该线程正在执行一个语句
Sleep	正在等待客户端向它发送执行语句
Quit	该线程正在退出
Kill	正在执行 kill 语句

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库	若需要查看其他用户的连接信息则需要 ADMIN 权限

示例

```
SHOW PROCESSLIST
```

```
+------------------+------+------+----------------+---------------------+----------+------+---------+------+----
    ↪
| CurrentConnected | Id   | User | Host           | LoginTime           | Catalog  | Db   |
    ↪ Command | Time | State | QueryId                            | Info                 |
+------------------+------+------+----------------+---------------------+----------+------+---------+------+----
    ↪
| Yes              |    0 | root | 127.0.0.1:34650 | 2025-01-21 12:01:02 | internal | test |
    ↪ Query   |    0 | OK    | c84e397193a54fe7-bbe9bc219318b75e | select 1             |
|                  |    1 | root | 127.0.0.1:34776 | 2025-01-21 12:01:07 | internal |      |
    ↪ Sleep   |   29 | EOF   | 886ffe2894314f50-8dd73a6ca06699e4 | show full processlist |
+------------------+------+------+----------------+---------------------+----------+------+---------+------+----
    ↪
```

7.3.4.3.2 KILL QUERY

描述

每个 Doris 的连接都在一个单独的线程中运行。使用该语句终止线程。

语法

```
KILL [CONNECTION] <processlist_id>
```

变种语法

```
KILL QUERY [ { <processlist_id> | <query_id> } ]
```

必选参数

1. <processlist_id>

> 表示需要被杀掉的连接线程 ID

2. <query_id>

表示需要被杀掉的查询 ID

可选参数

1. CONNECTION

表示是否是当前连接的线程

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	数据库	若需要 KILL 语句需要获得 GRANT 权限

注意事项

- 线程进程列表标识符可以从 SHOW PROCESSLIST 输出的 Id 列查询
- Connection ID 可以从 SELECT CONNECTION_ID() 来查询

示例

```
select connection_id()
```

```
+-----------------+
| connection_id() |
+-----------------+
| 48              |
+-----------------+
```

```
SHOW PROCESSLIST
```

```
+------------------+------+------+--------------------+---------------------+----------+---------+---------+-----
    ↪
| CurrentConnected | Id   | User | Host               | LoginTime           | Catalog  | Db
    ↪ | Command | Time | State | QueryId                           | Info
    ↪                                                                                  |
+------------------+------+------+--------------------+---------------------+----------+---------+---------+-----
    ↪
```

```
| Yes              |   48 | root | 10.16.xx.xx:44834   | 2025-01-21 16:49:47 | internal | test |
    ↪ Query   |    0 | OK    | e6e4ce9567b04859-8eeab8d6b5513e38 | SHOW PROCESSLIST
    ↪                                                                          |
|                  |   50 | root | 192.168.xx.xx:52837 | 2025-01-21 16:51:34 | internal |      |
    ↪ Sleep   | 1837 | EOF   | deaf13c52b3b4a3b-b25e8254b50ff8cb | SELECT @@session.transaction
    ↪ _isolation                                                 |
|                  |   51 | root | 192.168.xx.xx:52843 | 2025-01-21 16:51:35 | internal |      |
    ↪ Sleep   |  907 | EOF   | 437f219addc0404f-9befe7f6acf9a700 | /* ApplicationName=DBeaver
    ↪ Ultimate 23.1.3 - Metadata */ SHOW STATUS                  |
|                  |   55 | root | 192.168.xx.xx:55533 | 2025-01-21 17:09:32 | internal | test |
    ↪ Sleep   |  271 | EOF   | f02603dc163a4da3-beebbb5d1ced760c | /* ApplicationName=DBeaver
    ↪ Ultimate 23.1.3 - SQLEditor <Console> */ SELECT DATABASE() |
|                  |   47 | root | 10.16.xx.xx:35678   | 2025-01-21 16:21:56 | internal | test |
    ↪ Sleep   | 3528 | EOF   | f4944c543dc34a99-b0d0f3986c8f1c98 | select * from test
    ↪                                                                      |
+------------------+------+------+--------------------+---------------------+----------+---------+---------+-----
    ↪
```

```
kill 51
```

7.3.4.3.3 CLEAN PROFILE

描述

用于手动清理所有历史 query 或 load 的 profile 信息。

语法

```
CLEAN ALL PROFILE
```

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
GRANT_PRIV	数据库	若执行 CLEAN 语句需要获得 GRANT 权限

示例

```
CLEAN ALL PROFILE
```

7.3.4.3.4 SHOW QUERY STATS

描述

该语句用于展示数据库中历史查询命中的库表列的情况

语法

```
SHOW QUERY STATS [ { [FOR <db_name>] | [FROM <table_name>] } ] [ALL] [VERBOSE]];
```

可选参数

1. <db_name>

> 若填写表示展示数据库的命中情况

2. <table_name>

> 若填写表示查询某表的查询命中情况

3. ALL

> ALL 可以指定是否展示所有 index 的查询命中情况

4. VERBOSE

> VERBOSE 可以展示更详细的命中情况

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	DATABASE	需要对查询的数据库有 SELECT 权限

注意事项

- 支持查询数据库和表的历史查询命中情况，重启 fe 后数据会重置，每个 fe 单独统计。
- 通过 FOR DATABASE 和 FROM TABLE 可以指定查询数据库或者表的命中情况，后面分别接数据库名或者表名。

- ALL 和 VERBOSE 可以展示更详细的命中情况，这两个参数可以单独使用，也可以一起使用，但是必须放在最后而且只能用在表的查询上。
- 如果没有 use 任何数据库那么直接执行SHOW QUERY STATS 将展示所有数据库的命中情况。
- 命中结果中可能有两列：QueryCount 表示该列被查询次数，FilterCount 表示该列作为 where 条件被查询的次数。

示例

```
show query stats from baseall
```

```
+-------+------------+-------------+
| Field | QueryCount | FilterCount |
+-------+------------+-------------+
| k0    | 0          | 0           |
| k1    | 0          | 0           |
| k2    | 0          | 0           |
| k3    | 0          | 0           |
| k4    | 0          | 0           |
| k5    | 0          | 0           |
| k6    | 0          | 0           |
| k10   | 0          | 0           |
| k11   | 0          | 0           |
| k7    | 0          | 0           |
| k8    | 0          | 0           |
| k9    | 0          | 0           |
| k12   | 0          | 0           |
| k13   | 0          | 0           |
+-------+------------+-------------+
```

```
select k0, k1,k2, sum(k3) from baseall  where k9 > 1 group by k0,k1,k2
```

```
+------+------+--------+-------------+
| k0   | k1   | k2     | sum(`k3`)   |
+------+------+--------+-------------+
|    0 |    6 |  32767 |        3021 |
|    1 |   12 |  32767 | -2147483647 |
|    0 |    3 |   1989 |        1002 |
|    0 |    7 | -32767 |        1002 |
|    1 |    8 |    255 |  2147483647 |
|    1 |    9 |   1991 | -2147483647 |
|    1 |   11 |   1989 |       25699 |
|    1 |   13 | -32767 |  2147483647 |
|    1 |   14 |    255 |         103 |
|    0 |    1 |   1989 |        1001 |
```

```
|    0 |    2 |   1986 |        1001 |
|    1 |   15 |   1992 |        3021 |
+------+------+--------+-------------+
```

```
show query stats from baseall;
```

```
+-------+------------+-------------+
| Field | QueryCount | FilterCount |
+-------+------------+-------------+
| k0    | 1          | 0           |
| k1    | 1          | 0           |
| k2    | 1          | 0           |
| k3    | 1          | 0           |
| k4    | 0          | 0           |
| k5    | 0          | 0           |
| k6    | 0          | 0           |
| k10   | 0          | 0           |
| k11   | 0          | 0           |
| k7    | 0          | 0           |
| k8    | 0          | 0           |
| k9    | 1          | 1           |
| k12   | 0          | 0           |
| k13   | 0          | 0           |
+-------+------------+-------------+
```

```
show query stats from baseall all
```

```
+-----------+------------+
| IndexName | QueryCount |
+-----------+------------+
| baseall   | 1          |
+-----------+------------+
```

```
show query stats from baseall all verbose
```

```
+-----------+-------+------------+-------------+
| IndexName | Field | QueryCount | FilterCount |
+-----------+-------+------------+-------------+
| baseall   | k0    | 1          | 0           |
|           | k1    | 1          | 0           |
|           | k2    | 1          | 0           |
|           | k3    | 1          | 0           |
|           | k4    | 0          | 0           |
|           | k5    | 0          | 0           |
|           | k6    | 0          | 0           |
```

```
|           | k10   | 0          | 0           |
|           | k11   | 0          | 0           |
|           | k7    | 0          | 0           |
|           | k8    | 0          | 0           |
|           | k9    | 1          | 1           |
|           | k12   | 0          | 0           |
|           | k13   | 0          | 0           |
+-----------+-------+------------+-------------+
```

```
show query stats for test_query_db
```

```
+----------------------------+------------+
| TableName                  | QueryCount |
+----------------------------+------------+
| compaction_tbl             | 0          |
| bigtable                   | 0          |
| empty                      | 0          |
| tempbaseall                | 0          |
| test                       | 0          |
| test_data_type             | 0          |
| test_string_function_field | 0          |
| baseall                    | 1          |
| nullable                   | 0          |
+----------------------------+------------+
```

```
show query stats
```

```
+-----------------+------------+
| Database        | QueryCount |
+-----------------+------------+
| test_query_db   | 1          |
+-----------------+------------+
```

7.3.4.3.5 CLEAN QUERY STATS

描述

该语句用清空查询统计信息

语法

```
CLEAN [ { ALL| DATABASE | TABLE } ] QUERY STATS [ { [ FOR <db_name>] | [ { FROM | IN } ] <table_
    ↪ name>]];
```

必选参数

1. ALL

ALL 可以清空所有统计信息

2. DATABASE

DATABASE 表示清空某个数据库的统计信息

3. TABLE

TABLE 表示清空某个表的统计信息

可选参数

1. <db_name>

若填写表示清空对应数据库的统计信息

2. <table_name>

若填写表示清空对应表的统计信息

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	ALL	如果指定 ALL 则需要 ADMIN 权限
ALTER_PRIV	数据库	如果指定数据库则需要对应数据库的 ALTER 权限
ADMIN_PRIV	表	如果指定表则需要对应表的 alter 权限

示例

```
clean all query stats
```

```
clean database query stats for test_query_db
```

```
clean table query stats from test_query_db.baseall
```

7.3.4.3.6 PLAN REPLAYER DUMP

描述

PLAN REPLAYER DUMP 是 Doris 用户用来生成执行规划诊断文件的工具。用于捕捉查询优化器的状态和输入数据，方便调试和分析查询优化问题。其输出为对应诊断文件的 http 地址。

语法

```
PLAN REPLAYER DUMP <query>
```

必选参数

<query>

- 指的是对应的 DML 里面的 query 语句
- 如果不是 query 语句则会报 parse 错误
- 有关更多详细信息，请参阅SELECT语法

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）, 视图（View）	当执行时，需要拥有被查询的表，视图或物化视图的 SELECT_PRIV 权限

示例

基础示例

```
create database test_replayer;
use database test_replayer;
create table t1 (c1 int, c11 int) distributed by hash(c1) buckets 3 properties('replication_num'
    ↪ = '1');
plan replayer dump select * from t1;
```

执行结果示例：

```
+-------------------------------------------------------------------------------+
| Plan Replayer dump url                                                        |
| Plan Replayer dump url |
+-------------------------------------------------------------------------------+
```

```
| http://127.0.0.1:8030/api/minidump?query_id=6e7441f741e94afd-ad3ba69429ad18ec |
+------------------------------------------------------------------------------+
```

可以使用 curl 或者 wget 获取对应的文件，例如：

```
wget http://127.0.0.1:8030/api/minidump?query_id=6e7441f741e94afd-ad3ba69429ad18ec
```

当需要权限的时候可以把用户名和密码包含在

```
wget --header="Authorization: Basic $(echo -n 'root:' | base64)" http://127.0.0.1:8030/api/
    ↪ minidump?query_id=6e7441f741e94afd-ad3ba69429ad18ec
```

“ ‘

7.3.4.3.7 PLAN REPLAYER PLAY

描述

PLAN REPLAYER PLAY 是 Doris 开发者用来分析优化器问题的工具。其根据PLAN REPLAYER DUMP生成的诊断文件，在对应版本的 fe 下可以加载元数据和统计信息用于开发者复现和调试问题。

语法

```
PLAN REPLAYER PLAY <absolute-directory-of-dumpfile>;
```

必选参数

<absolute-directory-of-dumpfile>

- 指定对应 dump 文件的绝对路径的字符串。
- 标识符必须以双引号围起来，而且是对应文件的绝对路径。

示例

当我们有一个 dumpfile: /home/wangwu/dumpfile.json 时，可以使用下面 sql 来复现场景

```
PLAN REPLAYER PLAY "/home/wangwu/dumpfile.json";
```

7.3.4.4 连接

7.3.4.4.1 KILL CONNECTION

描述

杀死一个指定连接 ID 的连接。进而会杀死此连接对应的查询。

语法

```
KILL [ CONNECTION ] <connection_id>
```

必选参数

<connection_id>

> 链接的 ID。可以通过 SHOW PROCESSLIST 语句查询。

权限控制

执行此 SQL 命令的用户必须是此连接所属的用户，或者至少具有ADMIN_PRIV权限

示例

查询 connection_id:

```
show processlist;
```

结果如下：

```
+--
    ↪ ----------------+----+------+-----------------+---------------------+----------+------+----------+------+-
    ↪
| CurrentConnected | Id | User | Host            | LoginTime           | Catalog  | Db   |
    ↪ Command | Time | State | QueryId                             | Info             | FE
    ↪           | CloudCluster |
+--
    ↪ ----------------+----+------+-----------------+---------------------+----------+------+----------+------+-
    ↪
| Yes              | 16 | root | 127.0.0.1:63746 | 2024-11-04 20:18:07 | internal | test | Query
    ↪   | 0    | OK    | e4d69a1cce81468d-91c9ae32b17540e9 | show processlist | 172.16.123.1 |
    ↪ NULL          |
+--
    ↪ ----------------+----+------+-----------------+---------------------+----------+------+----------+------+-
    ↪
```

发送 KILL 命令

```
KILL CONNECTION 16;
```

7.3.5 事务

7.3.5.1 BEGIN

7.3.5.1.1 描述

开启一个显式事务。用户可以指定 Label，如未指定，系统自动生成 Label。

7.3.5.1.2 语法

```
BEGIN [ WITH LABEL <label> ]
```

7.3.5.1.3 可选参数

[WITH LABEL <label>]

> 显式指定该事务关联的 Label，如未指定，系统自动生成label 。

7.3.5.1.4 注意事项

- 如果开启了一个显式事务，没有执行提交或回滚，再次执行 BEGIN 命令不生效

7.3.5.1.5 示例

使用系统自动生成的 Label 开启显式事务

```
mysql> BEGIN;
{'label':'txn_insert_624a0e16ef4c43d4-9814c7fa3e83a705', 'status':'PREPARE', 'txnId':''}
```

指定 Label 开启显式事务

```
mysql> BEGIN WITH LABEL load_1;
{'label':'load_1', 'status':'PREPARE', 'txnId':''}
```

7.3.5.2 COMMIT

7.3.5.2.1 描述

提交一个显式事务。与 BEGIN 成对使用。

7.3.5.2.2 语法

```
COMMIT
```

7.3.5.2.3 注意事项

- 如果没有开启显式事务，执行该命令不生效

7.3.5.2.4 示例

以下示例创建了一个名为 test 的表，开启事务，写入两行数据后，提交事务。然后执行查询。

```
CREATE TABLE `test` (
  `ID` int NOT NULL,
  `NAME` varchar(100) NULL,
  `SCORE` int NULL
) ENGINE=OLAP
DUPLICATE KEY(`ID`)
DISTRIBUTED BY HASH(`ID`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 3"
);

BEGIN;
INSERT INTO test VALUES(1, 'Alice', 100);
INSERT INTO test VALUES(2, 'Bob', 100);
COMMIT;
SELECT * FROM test;
```

7.3.5.3 ROLLBACK

7.3.5.3.1 描述

回滚一个显式事务。与 BEGIN 成对使用。

7.3.5.3.2 语法

```
ROLLBACK
```

7.3.5.3.3 注意事项

- 如果没有开启显式事务，执行该命令不生效

7.3.5.3.4 示例

以下示例创建了一个名为 test 的表，开启事务，写入两行数据后，回滚事务。然后执行查询。

```
CREATE TABLE `test` (
  `ID` int NOT NULL,
  `NAME` varchar(100) NULL,
  `SCORE` int NULL
) ENGINE=OLAP
DUPLICATE KEY(`ID`)
```

```
DISTRIBUTED BY HASH(`ID`) BUCKETS 1
PROPERTIES (
    "replication_allocation" = "tag.location.default: 3"
);

BEGIN;
INSERT INTO test VALUES(1, 'Bob', 100);
INSERT INTO test VALUES(2, 'Bob', 100);
ROLLBACK;
SELECT * FROM test;
```

7.3.5.4 SHOW TRANSACTION

7.3.5.4.1 描述

该语法用于查看指定 transaction id 或 label 的事务详情。

语法

```
SHOW TRANSACTION
[FROM <db_name>]
WHERE
[id = <transaction_id> | label = <label_name>];
```

7.3.5.4.2 必选参数

1. <transaction_id>

需要查看事务详情的 transaction id

2. <label_name>

需要查看事务详情的 label

7.3.5.4.3 可选参数

1. <db_name>

需要查看事务详情的 database

7.3.5.4.4 返回值

Column name	Description
TransactionId	事务 id
Label	导入任务对应的 label
Coordinator	负责事务协调的节点
TransactionStatus	事务状态

Column name	Description
PREPARE	准备阶段
COMMITTED	事务成功，但数据不可见
VISIBLE	事务成功且数据可见
ABORTED	事务失败
LoadJobSourceType	导入任务的类型
PrepareTime	事务开始时间
CommitTime	事务提交成功的时间
FinishTime	数据可见的时间
Reason	错误信息
ErrorReplicasCount	有错误的副本数
ListenerId	相关的导入作业的 id
TimeoutMs	事务超时时间，单位毫秒

7.3.5.4.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
LOAD_PRIV	Database	

7.3.5.4.6 示例

1. 查看 id 为 4005 的事务：

```
SHOW TRANSACTION WHERE ID=4005;
```

2. 指定 db 中，查看 id 为 4005 的事务：

```
SHOW TRANSACTION FROM db WHERE ID=4005;
```

3. 查看 label 为 label_name 的事务：

```
SHOW TRANSACTION WHERE LABEL = 'label_name';
```

7.3.6 数据目录

7.3.6.1 CREATE CATALOG

7.3.6.1.1 描述

该语句用于创建外部数据目录（catalog）

7.3.6.1.2 语法

```
CREATE CATALOG [IF NOT EXISTS] <catalog_name> [ COMMENT "<comment>"]
    PROPERTIES ("<key>"="<value>" [, ... ]);
```

- hms：Hive MetaStore
- es：Elasticsearch
- jdbc：数据库访问的标准接口 (JDBC), 当前支持 MySQL 和 PostgreSQL

7.3.6.1.3 必选参数

1. <catalog_name>

需要创建 catalog 的名字

2. "<key>"="<value>"

需要创建 catalog 的参数

7.3.6.1.4 可选参数

1. <comment>

需要创建 catalog 的注释

7.3.6.1.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
CREATE_PRIV	Catalog	需要有对应 catalog 的 CREATE_PRIV 权限

7.3.6.1.6 示例

1. 新建数据目录 hive

```
    CREATE CATALOG hive comment 'hive catalog' PROPERTIES (
        'type'='hms',
        'hive.metastore.uris' = 'thrift://127.0.0.1:7004',
        'dfs.nameservices'='HANN',
        'dfs.ha.namenodes.HANN'='nn1,nn2',
        'dfs.namenode.rpc-address.HANN.nn1'='nn1_host:rpc_port',
        'dfs.namenode.rpc-address.HANN.nn2'='nn2_host:rpc_port',
        'dfs.client.failover.proxy.provider.HANN'='org.apache.hadoop.hdfs.server.namenode.ha.
            ↪ ConfiguredFailoverProxyProvider'
    );
```

2. 新建数据目录 es

```
    CREATE CATALOG es PROPERTIES (
        "type"="es",
        "hosts"="http://127.0.0.1:9200"
    );
```

3. 新建数据目录 jdbc

   mysql

```
    CREATE CATALOG jdbc PROPERTIES (
        "type"="jdbc",
        "user"="root",
        "password"="123456",
        "jdbc_url" = "jdbc:mysql://127.0.0.1:3316/doris_test?useSSL=false",
        "driver_url" = "https://doris-community-test-1308700295.cos.ap-hongkong.myqcloud.com/jdbc
            ↪ _driver/mysql-connector-java-8.0.25.jar",
        "driver_class" = "com.mysql.cj.jdbc.Driver"
    );
```

```
**postgresql**
```

```
    CREATE CATALOG jdbc PROPERTIES (
        "type"="jdbc",
        "user"="postgres",
        "password"="123456",
        "jdbc_url" = "jdbc:postgresql://127.0.0.1:5432/demo",
        "driver_url" = "file:///path/to/postgresql-42.5.1.jar",
        "driver_class" = "org.postgresql.Driver"
    );
```

```
**clickhouse**
```

```
    CREATE CATALOG jdbc PROPERTIES (
        "type"="jdbc",
        "user"="default",
        "password"="123456",
        "jdbc_url" = "jdbc:clickhouse://127.0.0.1:8123/demo",
        "driver_url" = "file:///path/to/clickhouse-jdbc-0.3.2-patch11-all.jar",
        "driver_class" = "com.clickhouse.jdbc.ClickHouseDriver"
    )
```

```
**oracle**
```

```
    CREATE CATALOG jdbc PROPERTIES (
        "type"="jdbc",
        "user"="doris",
        "password"="123456",
        "jdbc_url" = "jdbc:oracle:thin:@127.0.0.1:1521:helowin",
        "driver_url" = "file:///path/to/ojdbc8.jar",
        "driver_class" = "oracle.jdbc.driver.OracleDriver"
    );
```

```
**SQLServer**
```

```
    CREATE CATALOG sqlserver_catalog PROPERTIES (
        "type"="jdbc",
        "user"="SA",
        "password"="Doris123456",
        "jdbc_url" = "jdbc:sqlserver://localhost:1433;DataBaseName=doris_test",
        "driver_url" = "file:///path/to/mssql-jdbc-11.2.3.jre8.jar",
        "driver_class" = "com.microsoft.sqlserver.jdbc.SQLServerDriver"
    );
```

```
**SAP HANA**
```

```
    CREATE CATALOG saphana_catalog PROPERTIES (
       "type"="jdbc",
       "user"="SYSTEM",
       "password"="SAPHANA",
       "jdbc_url" = "jdbc:sap://localhost:31515/TEST",
       "driver_url" = "file:///path/to/ngdbc.jar",
       "driver_class" = "com.sap.db.jdbc.Driver"
    );
```

```
**Trino**
```

```
    CREATE CATALOG trino_catalog PROPERTIES (
       "type"="jdbc",
       "user"="hadoop",
       "password"="",
       "jdbc_url" = "jdbc:trino://localhost:8080/hive",
       "driver_url" = "file:///path/to/trino-jdbc-389.jar",
       "driver_class" = "io.trino.jdbc.TrinoDriver"
    );
```

```
**OceanBase**
```

```
CREATE CATALOG oceanbase_catalog PROPERTIES (
    "type"="jdbc",
    "user"="root",
    "password"="",
    "jdbc_url" = "jdbc:oceanbase://localhost:2881/demo",
    "driver_url" = "file:///path/to/oceanbase-client-2.4.2.jar",
    "driver_class" = "com.oceanbase.jdbc.Driver"
);
```

7.3.6.2 ALTER CATALOG

7.3.6.2.1 描述

该语句用于设置指定数据目录的属性。

7.3.6.2.2 语法

1) 重命名数据目录

```
ALTER CATALOG <catalog_name> RENAME <new_catalog_name>;
```

2) 设置数据目录属性

```
ALTER CATALOG <catalog_name> SET PROPERTIES ('<key>' = '<value>' [, ... ]);
```

3) 修改数据目录注释

```
ALTER CATALOG <catalog_name> MODIFY COMMENT "<new catalog comment>";
```

7.3.6.2.3 必选参数

1. <catalog_name>

需要修改的 Catalog 名称

2. <new_catalog_name>

修改后的新 Catalog 名称

3. '<key>' = '<value>'

需要修改/添加的 Catalog 属性的 key 和 value

4. <new catalog comment>

修改后的 Catalog 注释

7.3.6.2.4 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	Catalog	需要有对应 catalog 的 ALTER_PRIV 权限

7.3.6.2.5 注意事项

1) 重命名数据目录

- `internal` 是内置数据目录，不允许重命名
- 对 `catalog_name` 拥有 Alter 权限才允许对其重命名
- 重命名数据目录后，如需要，请使用 REVOKE 和 GRANT 命令修改相应的用户权限。

2) 设置数据目录属性

- 不可更改数据目录类型，即 `type` 属性
- 不可更改内置数据目录 `internal` 的属性
- 更新指定属性的值为指定的 value。如果 SET PROPERTIES 从句中的 key 在指定 catalog 属性中不存在，则新增此 key。

3) 修改数据目录注释

- `internal` 是内置数据目录，不允许修改注释

7.3.6.2.6 示例

1. 将数据目录 ctlg_hive 重命名为 hive

```
ALTER CATALOG ctlg_hive RENAME hive;
```

2. 更新名为 hive 数据目录的属性 `hive.metastore.uris`

```
ALTER CATALOG hive SET PROPERTIES ('hive.metastore.uris'='thrift://172.21.0.1:9083');
```

3. 更改名为 hive 数据目录的注释

```
ALTER CATALOG hive MODIFY COMMENT "new catalog comment";
```

7.3.6.3 DROP CATALOG

7.3.6.3.1 描述

该语句用于删除外部数据目录（catalog）

7.3.6.3.2 语法

```
DROP CATALOG [IF EXISTS] <catalog_name>;
```

7.3.6.3.3 必选参数

1. <catalog_name>

需要删除 catalog 的名字

7.3.6.3.4 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
DROP_PRIV	Catalog	需要有对应 catalog 的 DROP_PRIV 权限

7.3.6.3.5 示例

1. 删除数据目录 hive

```
DROP CATALOG hive;
```

7.3.6.4 SHOW CREATE CATALOG

7.3.6.4.1 描述

该语句查看 Doris 数据目录的创建语句。

7.3.6.4.2 语法

```
SHOW CREATE CATALOG <catalog_name>;
```

7.3.6.4.3 必选参数

1. <catalog_name>

需要查看创建语句的 catalog 的名字

7.3.6.4.4 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV / SELECT_PRIV / LOAD_PRIV / ALTER_PRIV / CREATE_PRIV / SHOW_VIEW_PRIV / DROP_PRIV	Catalog	需要有上述权限中的一种就可以

7.3.6.4.5 示例

1. 查看 Doris 中 oracle 数据目录的创建语句

```
SHOW CREATE CATALOG oracle;
```

```
+--
    ↪ -------+----------------------------------------------------------------------------------------------------
    ↪
| Catalog | CreateCatalog
    ↪
    ↪ |
+--
    ↪ -------+----------------------------------------------------------------------------------------------------
    ↪
| oracle  |
CREATE CATALOG `oracle` PROPERTIES (
"user" = "XXX",
"type" = "jdbc",
"password" = "*XXX",
"jdbc_url" = "XXX",
"driver_url" = "XXX",
"driver_class" = "oracle.jdbc.driver.OracleDriver",
"checksum" = "XXX"
); |
+--
    ↪ -------+----------------------------------------------------------------------------------------------------
    ↪
```

7.3.6.5 SHOW-CATALOG

7.3.6.5.1 描述

该语句用于显示已存在是数据目录（catalog）属性

7.3.6.5.2 语法

```
SHOW CATALOG <catalog_name>
```

7.3.6.5.3 必填参数

1. <catalog_name>

需要显示 catalog 的名字

7.3.6.5.4 返回值

7.3.6.5.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV / SELECT_PRIV / LOAD_PRIV / ALTER_PRIV / CREATE_PRIV / SHOW_VIEW_PRIV / DROP_PRIV	Catalog	需要有上述权限中的一种就可以

7.3.6.5.6 示例

1. 查看创建的数据目录属性

```
SHOW CATALOG test_mysql;
```

```
  +--
      ↪ --------------+-------------------------------------------------------------------------------
      ↪
| Key             | Value
    ↪
    ↪ |
+--
    ↪ --------------+-------------------------------------------------------------------------------
    ↪
| checksum        | fdf55dcef04b09f2eaf42b75e61ccc9a
    ↪
    ↪ |
| create_time     | 2025-02-17 17:21:13.099
    ↪
    ↪ |
| driver_class    | com.mysql.cj.jdbc.Driver
    ↪
    ↪ |
| driver_url      | mysql-connector-j-8.3.0.jar
    ↪
    ↪ |
```

```
| jdbc_url       | jdbc:mysql://127.0.0.1:3306/db
    ↪
    ↪ |
| password       | *XXX
    ↪
    ↪ |
| type           | jdbc
    ↪
    ↪ |
| use_meta_cache | true
    ↪
    ↪ |
| user           | root
    ↪
    ↪ |
+--
    ↪ --------------+----------------------------------------------------------------------------------
    ↪
```

7.3.6.6 SHOW-CATALOGS

7.3.6.6.1 描述

该语句用于显示已存在是数据目录（catalog）

7.3.6.6.2 语法

```
SHOW CATALOGS [LIKE <catalog_name>]
```

说明：

LIKE：可按照 CATALOG 名进行模糊查询

7.3.6.6.3 可选参数

1. <catalog_name>

需要显示 catalog 的名字

7.3.6.6.4 返回值

Column name	Description
CatalogId	数据目录唯一 ID
CatalogName	数据目录名称，其中 internal 是默认内置的 catalog，不可修改
Type	数据目录类型

Column name	Description
IsCurrent	是否为当前正在使用的数据目录
CreateTime	创建时间
LastUpdateTime	最后更新时间
Comment	备注

7.3.6.6.5 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV / SELECT_PRIV / LOAD_PRIV / ALTER_PRIV / CREATE_PRIV / SHOW_VIEW_PRIV / DROP_PRIV	Catalog	需要有上述权限中的一种就可以

7.3.6.6.6 示例

1. 查看当前已创建的数据目录

```
SHOW CATALOGS;
```

```
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+---------------------+------
    ↪
| CatalogId | CatalogName | Type     | IsCurrent | CreateTime              | LastUpdateTime
    ↪      | Comment                |
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+---------------------+------
    ↪
|    130100 | hive        | hms      |           | 2023-12-25 16:11:41.687 | 2023-12-25
    ↪ 20:43:18 | NULL                   |
|         0 | internal    | internal | yes       | UNRECORDED              | NULL
    ↪                | Doris internal catalog |
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+---------------------+------
    ↪
```

2. 按照目录名进行模糊搜索

```
SHOW CATALOGS LIKE 'hi%';
```

```
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+-----------------------+------
    ↪
| CatalogId | CatalogName | Type     | IsCurrent | CreateTime              | LastUpdateTime
    ↪      | Comment                |
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+-----------------------+------
    ↪
|    130100 | hive        | hms      |           | 2023-12-25 16:11:41.687 | 2023-12-25
    ↪ 20:43:18 | NULL                     |
+--
    ↪ ---------+-------------+----------+-----------+-------------------------+-----------------------+------
    ↪
```

7.3.6.7 REFRESH

7.3.6.7.1 描述

该语句用于刷新指定 Catalog/Database/Table 的元数据。

7.3.6.7.2 语法

```
REFRESH CATALOG <catalog_name>;
REFRESH DATABASE [<catalog_name>.]<database_name>;
REFRESH TABLE [[<catalog_name>.]<database_name>.]<table_name>;
```

7.3.6.7.3 必选参数

1. <catalog_name>

需要刷新的 catalog 的名字

2. [<catalog_name>.]<database_name>

需要刷新的 catalog 里面 database 的名字

3. [[<catalog_name>.]<database_name>.]<table_name>

需要刷新的 catalog 里面 table 的名字

7.3.6.7.4 权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV / SELECT_PRIV / LOAD_PRIV / ALTER_PRIV / CREATE_PRIV / SHOW_VIEW_PRIV / DROP_PRIV	Catalog	需要有上述权限中的一种就可以

7.3.6.7.5 注意事项

刷新 Catalog 的同时，会强制使对象相关的 Cache 失效。包括 Partition Cache、Schema Cache、File Cache 等。

7.3.6.7.6 示例

1. 刷新 hive catalog

```
REFRESH CATALOG hive;
```

2. 刷新 database1

```
REFRESH DATABASE ctl.database1;
REFRESH DATABASE database1;
```

3. 刷新 table1

```
REFRESH TABLE ctl.db.table1;
REFRESH TABLE db.table1;
REFRESH TABLE table1;
```

7.3.7 数据库

7.3.7.1 CREATE DATABASE

7.3.7.1.1 描述

该语句用于新建数据库（database）

7.3.7.1.2 语法

```
CREATE DATABASE [IF NOT EXISTS] <db_name>
    [PROPERTIES ("<key>"="<value>"[, ... ])];
```

7.3.7.1.3 必选参数

** 1. <db_name>** > 数据库名称

7.3.7.1.4 可选参数

** 1. <PROPERTIES>** > 该数据库的附加信息

7.3.7.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
CREATE_PRIV	对应数据库	需要对对应数据库具有创建权限

7.3.7.1.6 注意事项

如果要为 db 下的 table 指定默认的副本分布策略，需要指定<replication_allocation>（table 的<replication_allocation>属性优先级会高于 db）：

```
PROPERTIES (
  "replication_allocation" = "tag.location.default:3"
)
```

如果要为 db 下的 table 指定默认的 Storage Vault，需要指定<storage_vault_name>（table 的<storage_vault_name>属性优先级会高于 db）：

```
PROPERTIES (
  "storage_vault_name" = "hdfs_demo_vault"
)
```

> **备注**
>
> 从 3.0.5 版本支持指定 db 的 storage_vault_name。

7.3.7.1.7 示例

- 新建数据库 db_test

```
CREATE DATABASE db_test;
```

- 新建数据库并设置默认的副本分布：

```
CREATE DATABASE `db_test`
PROPERTIES (
 "replication_allocation" = "tag.location.group_1:3"
);
```

- 新建数据库并设置默认的 Storage Vault：

```
CREATE DATABASE `db_test`
PROPERTIES (
 "storage_vault_name" = "hdfs_demo_vault"
);
```

7.3.7.2 ALTER DATABASE

7.3.7.2.1 描述

该语句用于设置指定 db 的属性和改动 db 名字以及设定 db 的多种 quota。

7.3.7.2.2 语法

```
ALTER DATABASE <db_name> RENAME <new_name>
ALTER DATABASE <db_name> SET { DATA | REPLICA | TRANSACTION } QUOTA <quota>
ALTER DATABASE <db_name> SET <PROPERTIES> ("<key>" = "<value>" [, ...])
```

7.3.7.2.3 必选参数

** 1. <db_name>** > 数据库名称

** 2. <new_db_name>** > 新的数据库名称

** 3. <quota>** > 数据库数据量配额或者数据库的副本数量配额

** 4. <PROPERTIES>** > 该数据库的附加信息

7.3.7.2.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	对应数据库	需要对对应数据库具有变更权限

7.3.7.2.5 注意事项

重命名数据库后，如需要，请使用 REVOKE 和 GRANT 命令修改相应的用户权限。数据库的默认数据量配额为

8192PB，默认副本数量配额为 1073741824。

7.3.7.2.6 示例

- 设置指定数据库数据量配额

```
ALTER DATABASE example_db SET DATA QUOTA 10995116277760;
```

- 将数据库 example_db 重命名为 example_db2

```
ALTER DATABASE example_db RENAME example_db2;
```

- 设定指定数据库副本数量配额

```
ALTER DATABASE example_db SET REPLICA QUOTA 102400;
```

- 修改 db 下 table 的默认副本分布策略（该操作仅对新建的 table 生效，不会修改 db 下已存在的 table）

```
ALTER DATABASE example_db SET PROPERTIES("replication_allocation" = "tag.location.default:2")
    ↪ ;
```

- 取消 db 下 table 的默认副本分布策略（该操作仅对新建的 table 生效，不会修改 db 下已存在的 table）

```
ALTER DATABASE example_db SET PROPERTIES("replication_allocation" = "");
```

- 修改 db 下 table 的默认 Storage Vault（该操作仅对新建的 table 生效，不会修改 db 下已存在的 table）

```
ALTER DATABASE example_db SET PROPERTIES("storage_vault_name" = "hdfs_demo_vault");
```

- 取消 db 下 table 的默认 Storage Vault（该操作仅对新建的 table 生效，不会修改 db 下已存在的 table）

```
ALTER DATABASE example_db SET PROPERTIES("storage_vault_name" = "");
```

> 备注
>
> 从 3.0.5 版本支持指定 db 的 `storage_vault_name`。

7.3.7.3 DROP DATABASE

7.3.7.3.1 描述

该语句用于删除数据库（database）

7.3.7.3.2 语法

```
DROP DATABASE [IF EXISTS] <db_name> [FORCE];
```

7.3.7.3.3 必选参数

** 1. <db_name>** > 数据库名称

7.3.7.3.4 可选参数

** 1. FORCE** > 强制删除，不走回收站

7.3.7.3.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
DROP_PRIV	对应数据库	需要对对应数据库具有删除权限

7.3.7.3.6 注意事项

如果执行 DROP DATABASE FORCE，则系统不会检查该数据库是否存在未完成的事务，数据库将直接被删除并且不能被恢复，一般不建议执行此操作

7.3.7.3.7 示例

- 删除数据库 db_test

```
    DROP DATABASE db_test;
```

7.3.7.4 SHOW CREATE DATABASE

7.3.7.4.1 描述

该语句查看 doris 内置数据库或者 catalog 数据库的创建信息。

7.3.7.4.2 语法

```
SHOW CREATE DATABASE [<catalog>.]<db_name>;
```

7.3.7.4.3 必选参数

** 1. <db_name>** > 数据库名称

7.3.7.4.4 可选参数

** 1. <catalog>** > 表示内部表还是外部表

7.3.7.4.5 返回结果

列	描述
Database	数据库名称
Create Database	对应数据库创建语句

7.3.7.4.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV	对应数据库	需要对对应数据库具有读取权限

7.3.7.4.7 示例

- 查看 doris 中 test 数据库的创建情况

```
SHOW CREATE DATABASE test;
```

```
+----------+------------------------+
| Database | Create Database        |
+----------+------------------------+
| test     | CREATE DATABASE `test` |
+----------+------------------------+
```

- 查看 hive catalog 中数据库 hdfs_text 的创建信息

```
SHOW CREATE DATABASE hdfs_text;
```

```
+-----------+-----------------------------------------------------------------------------------+
    ↪
| Database  | Create Database
    ↪                                                                                  |
+-----------+-----------------------------------------------------------------------------------+
    ↪
| hdfs_text | CREATE DATABASE `hdfs_text` LOCATION 'hdfs://HDFS1009138/hive/warehouse/hdfs_
    ↪ text' |
+-----------+-----------------------------------------------------------------------------------+
    ↪
```

7.3.7.5 SHOW DATABASES

7.3.7.5.1 描述

该语句用于展示当前可见的数据库

7.3.7.5.2 语法

```
SHOW DATABASES [FROM <catalog>] [<filter_expr>];
```

7.3.7.5.3 可选参数

** 1. <catalog>** > 对应 catalog

** 2. <filter_expr>** > 进行指定条件的过滤

7.3.7.5.4 返回结果

列	描述
Database	数据库名称

7.3.7.5.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV	对应数据库	需要对对应数据库具有读取权限

7.3.7.5.6 示例

- 展示当前所有的数据库名称。

```
SHOW DATABASES;
```

```
+--------------------+
| Database           |
+--------------------+
| test               |
| information_schema |
+--------------------+
```

- 会展示hms_catalog中所有的数据库名称。

```
SHOW DATABASES FROM hms_catalog;
```

```
+---------------+
| Database      |
+---------------+
| default       |
| tpch          |
+---------------+
```

- 展示当前所有经过表示式like 'infor%'过滤后的数据库名称。

```
SHOW DATABASES like 'infor%';
```

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
+--------------------+
```

7.3.7.6 SHOW DATABASE ID

描述

该语句用于根据 database id 查找对应的 database name（仅管理员使用）

7.3.7.6.1 语法

```
SHOW DATABASE <database_id>
```

7.3.7.6.2 必选参数

** 1. <database_id>** > 数据库对应 id 号

7.3.7.6.3 返回结果

列	描述
DbName	数据库名称

7.3.7.6.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	整个集群	需要对整个集群具有管理权限

7.3.7.6.5 示例

- 根据 database id 查找对应的 database name

```
SHOW DATABASE 10396;
```

```
+------------+
| DbName     |
+------------+
| example_db |
+------------+
```

7.3.8 表和视图

7.3.8.1 表

7.3.8.1.1 CREATE TABLE

描述

在当前或指定的数据库中创建一个新表。一个表可以有多个列，每个列的定义包括名称、数据类型，以及可选的以下属性：

- 是否是 key
- 是否有聚合语义
- 是否是生成列
- 是否要求值（NOT NULL）
- 是否是自增列
- 是否有插入时的默认值
- 是否有更新时的默认值

此外，此命令还支持以下变体：

- CREATE TABLE ··· AS SELECT（创建一个已填充数据的表；也称为 CTAS）
- CREATE TABLE ··· LIKE（创建一个现有表的空副本）

语法

```
CREATE [ EXTERNAL ] TABLE [ IF NOT EXISTS ] <table_name>
    (<columns_definition> [ <indexes_definition> ])
    [ ENGINE = <table_engine_type> ]
    [ <key_type> KEY (<key_cols>)
        [ CLUSTER BY (<cluster_cols>) ]
    ]
    [ COMMENT '<table_comment>' ]
    [ <partitions_definition> ]
    [ DISTRIBUTED BY { HASH (<distribute_cols>) | RANDOM }
        [ BUCKETS { <bucket_count> | AUTO } ]
    ]
    [ <roll_up_definition> ]
    [ PROPERTIES (
          -- 表属性
          <table_property>
          -- 其他表属性
          [ , ... ])
    ]
```

其中：

```
columns_definition
  : -- 列定义
    <col_name> <col_type>
      [ KEY ]
      [ <col_aggregate_type> ]
      [ [ GENERATED ALWAYS ] AS (<col_generate_expression>) ]
      [ [NOT] NULL ]
      [ AUTO_INCREMENT(<col_auto_increment_start_value>) ]
      [ DEFAULT <col_default_value> ]
      [ ON UPDATE CURRENT_TIMESTAMP (<col_on_update_precision>) ]
      [ COMMENT '<col_comment>' ]
    -- 其他列定义
    [ , <col_name> <col_type> [ ... ] ]
```

```
indexes_definition
  : -- 索引定义
    INDEX [ IF NOT EXISTS ]
      <index_name> (<index_cols>)
      [ USING <index_type> ]
      [ PROPERTIES (
```

```
            -- 表属性
            <index_property>
            -- 其他表属性
            [ , ... ])
      ]
      [ COMMENT '<index_comment>' ]
    -- 其他索引定义
    [ , <index_name> (<index_cols>) [ ... ] ]
```

```
partitions_definition
  : AUTO PARTITION BY RANGE(<auto_partition_function>(<auto_partition_arguments>))
    <origin_partitions_definition>
  | AUTO PARTITION BY LIST(<partition_cols>)
    <origin_partitions_definition>
  | PARTITION BY <partition_type> (<partition_cols>)
    <origin_partitions_definition>
```

- 其中：

```
    <origin_partitions_definition>
    : (
        -- 分区定义
        <one_partition_definition>
        -- 其他分区定义
        [ , ... ]
      )

    <one_partition_definition>
    : PARTITION [ IF NOT EXISTS ] <partition_name>
        VALUES LESS THAN <partition_value_list>
    | PARTITION [ IF NOT EXISTS ] <partition_name>
        VALUES [ <partition_lower_bound>, <partition_upper_bound>)
    | FROM <partition_lower_bound> TO <partition_upper_bound>
        INTERVAL <n> [ <datetime_unit> ]
    | PARTITION [ IF NOT EXISTS ] <partition_name>
        VALUES IN {
            (<partition_value> [, <partition_value> [ ... ] ])
            | <partition_value>
        }
```

```
roll_up_definition
  : ROLLUP (
        -- 聚合定义
        <rollup_name> (<rollup_cols>)
```

```
        [ DUPLICATE KEY (<duplicate_cols>) ]
        -- 其他聚合定义
        [ , <rollup_name> (<rollup_cols>) [ ... ] ]
    )
```

变种语法

CREATE TABLE ··· AS SELECT (也称为 CTAS)

创建一个填充 query 返回数据的表：

```
CREATE
    [ EXTERNAL ]
  TABLE [ IF NOT EXISTS ] <table_name>
    [(<col_name> [ , <col_name> [ ... ] ] )]
    [ <indexesDefinition> ]
    [ ENGINE = <table_engine_type> ]
    [ <key_type> KEY (<key_cols>)
        [ CLUSTER BY (<cluster_cols>) ]
    ]
    [ COMMENT '<table_comment>' ]
    [ <partitionsDefinition> ]
    [ DISTRIBUTED BY { HASH (<distribute_cols>) | RANDOM }
        [ BUCKETS { <bucket_count> | AUTO } ]
    ]
    [ <rollUpDefinition> ]
    [ PROPERTIES (
          -- 表属性
          <table_property>
          -- 其他表属性
          [ , ... ])
    ]
    AS <query>
```

CREATE TABLE ··· LIKE

创建一个新表，其列定义与现有表相同，但不从现有表中复制数据。列的所有属性将被复制到新的表中。如果指定了 rollup 列表，那么原表中对应名字的 rollup 也将复制到新的表中：

```
CREATE TABLE <table_name> LIKE <source_table>
[ WITH ROLLUP (<rollup_names>) ]
```

必选参数

指定表的标识符（即名称）；在创建表的数据库（Database）中必须唯一。

标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。

标识符不能使用保留关键字。

有关更多详细信息，请参阅标识符要求和保留关键字。

指定列标识符（即名称）。在创建的表中必须唯一。

标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符），数字或符号@开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。

有关更多详细信息，请参阅标识符要求和保留关键字。

指定列的数据类型。

有关可以为表列指定的数据类型的详细信息，请参阅数据类型章节。

在 CTAS 中为必选参数。指定填充数据的 SELECT 语句。

在 CREATE TABLE ··· LIKE 中为必选参数。指定被复制的原表。

可选参数

数据模型相关参数

表的数据模型。可选值为 DUPLICATE（明细模型）、UNIQUE（主键模型）、AGGREGATE（聚合模型）。有关数据模型的详细信息，请参阅数据模型章节。

表的 key 列。Doris 中 Key 列必须是表的前 K 个列。单个 tablet 中的数据会按照这些列保持有序。关于 Key 的限制，以及如何选择 Key 列，请参阅数据模型章节中的各个小节。

数据局部排序列，只能在数据模型为 UNIQUE（主键模型）时使用。当指定 <cluster_cols> 后，数据按照<cluster_cols>排序，而不再使用<key_cols>。

列的聚合方式。只有当表是聚合模型时可以使用。聚合方式的详细信息，请参阅聚合模型章节。

分桶相关参数

和

分桶列和分桶数。明细模型的分桶列可以是任意的列，聚合模型和主键模型的分桶列必须和 key 列保持一致。分桶数是任意的正整数。有关分桶的详细信息，请参阅手动分桶和自动分桶章节。

列的默认值相关参数

[GENERATED ALWAYS] AS ()

生成列。使用当前列之前的列，通过表达式，对当前列生成数据。生成列是一种特殊的数据库表列，其值由其他列的值计算而来，而不是直接由用户插入或更新。该功能支持预先计算表达式的结果，并存储在数据库中，适用于需要频繁查询或进行复杂计算的场景。

AUTO_INCREMENT()

在导入数据时，Doris 会为在自增列上没有指定值的数据行分配一个表内唯一的值。指定自增列的起始值。自增列的详细信息，请参阅自增列章节。

DEFAULT

列的默认值。当写入时不包含此列时，使用此默认值填充。当默认值不显式设置时，使用 NULL 填充。可用的默认值包括：

- NULL：全部类型均可用，用 NULL 作为默认值。
- 数值字面量：只能是数值类型时使用。
- 字符串字面量：只能是字符串类型时使用。
- CURRENT_DATE：只能是 date 类型时使用。用当前日期作为默认值。
- CURRENT_TIMESTAMP [ ]：只能是 datetime 类型时使用。用当前时间作为默认值。可以指定时间精度。
- PI：只能是 double 类型时使用。用圆周率作为默认值。
- E：只能是 double 类型时使用。用数学常数作为默认值。
- BITMAP_EMPTY：只能当列是 bitmap 类型时使用。填充空的 bitmap。

ON UPDATE CURRENT_TIMESTAMP ()

当数据更新时，如果没有指定此列的值，则使用当前时间戳更新此列数据。只能在 UNIQUE（主键模型）的表上使用。

索引相关参数

指定索引标识符（即名称）。在创建的表中必须唯一。有关标识符的更多详细信息，请参阅标识符要求和保留关键字。

添加索引的列列表。必须是表中存在的列。

索引的类型。当前仅支持 INVERTED

索引的属性。详细说明请参考倒排索引章节。

自动分区相关参数

有关分区的详细介绍，请参阅自动分区章节。

手动分区相关参数

有关分区的详细介绍，请参阅“手动分区”章节。

Doris 支持 RANGE 分区和 LIST 分区。详情请参阅手动分区章节。

分区标识符（即名称）。在创建的表中必须唯一。有关标识符的更多详细信息，请参阅标识符要求和保留关键字。

VALUES LESS THAN

range 分区。分区数据范围从下界到。
如果是表示上界，可以用 MAX_VALUE 简化书写。
的格式如下：((col_1_value, ...), (col_1_value, ...), ...)

VALUES [,)

range 分区。分区数据范围从到。只创建一个分区。
和格式如下：(col_1_value, ...)

FROM TO

INTERVAL [ ]

range 分区。分区数据范围从到。每建个创建一个分区。
和格式如下：(col_1_value, ...)

VALUES IN {

([, […]])

|

}

list 分区。分区列等于的属于此分区。
格式如下：(col_1_value, ...)

同步物化视图相关

注意 rollup 可以创建的同步物化视图功能有限。已不再推荐使用。推荐使用独立的语句创建同步物化视图。详情请参阅 CREATE MATERIALIZED VIEW 语句和同步物化视图章节。

同步物化视图的标示符（即名称）。在创建的表中必须唯一。有关标识符的更多详细信息，请参阅标识符要求和保留关键字。

同步物化视图包含的列。

表属性相关参数

属性名	作用
replication_num	副本数。默认副本数为 3。如果 BE 节点数量小于 3，则需指定副本数小于等于 BE 节点数量。在 0.15 版本后，该属性将自动转换成 `replication_allocation` 属性，如：`"replication_num" = "3"` 会自动转换成 `"replication_allocation" = "tag.location.default:3"`。
replication_allocation	根据 Tag 设置副本分布情况。该属性可以完全覆盖 `replication_num` 属性的功能。
min_load_replica_num	设定数据导入成功所需的最小副本数，默认值为 -1。当该属性小于等于 0 时，表示导入数据仍需多数派副本成功。
is_being_synced	用于标识此表是否是被 CCR 复制而来并且正在被 syncer 同步，默认为 `false`。如果设置为 `true`，`colocate_with`和`storage_policy`属性将被擦除。`dynamic partition`和`auto bucket`功能将会失效。即在`show create table`中显示开启状态，但不会实际生效。当`is_being_synced`被设置为 `false` 时，这些功能将会恢复生效。这个属性仅供 CCR 外围模块使用，在 CCR 同步的过程中不要手动设置。
storage_medium	声明表数据的初始存储介质
storage_cooldown_time	设定表数据的初始存储介质的到期时间。超过此时间后，会自动降级到第一级别的存储介质上。
colocate_with	当需要使用 Colocation Join 功能时，使用这个参数设置 Colocation Group。
bloom_filter_columns	用户指定需要添加 Bloom Filter 索引的列名称列表。各个列的 Bloom Filter 索引是独立的，并不是组合索引。列如：`"bloom_filter_columns" = "k1, k2, k3"`
compression	Doris 表的默认压缩方式是 LZ4。1.1 版本后，支持将压缩方式指定为 ZSTD 以获得更高的压缩比。

属性名	作用
function_column.sequence_col	当使用 Unique Key 模型时，可以指定一个 Sequence 列，当 Key 列相同时，将按照 Sequence 列进行 REPLACE(较大值替换较小值，否则无法替换)。 `function_column.sequence_col`用来指定 sequence 列到表中某一列的映射，该列可以为整型和时间类型（DATE、DATETIME），创建后不能更改该列的类型。如果设置了`function_column.sequence_col`, `function_column.sequence_type`将被忽略。
function_column.sequence_type	当使用 Unique Key 模型时，可以指定一个 Sequence 列，当 Key 列相同时，将按照 Sequence 列进行 REPLACE(较大值替换较小值，否则无法替换) 这里我们仅需指定顺序列的类型，支持时间类型或整型。Doris 会创建一个隐藏的顺序列。
enable_unique_key_merge_on_write	Unique 表是否使用 Merge-on-Write 实现。该属性在 2.1 版本之前默认关闭，从 2.1 版本开始默认开启。
light_schema_change	是否使用 Light Schema Change 优化。如果设置成 `true`, 对于值列的加减操作，可以更快地，同步地完成。该功能在 2.0.0 及之后版本默认开启。
disable_auto_compaction	是否对这个表禁用自动 Compaction。如果这个属性设置成 `true`, 后台的自动 Compaction 进程会跳过这个表的所有 Tablet。
enable_single_replica_compaction	是否对这个表开启单副本 Compaction。如果这个属性设置成 `true`, 这个表的 Tablet 的所有副本只有一个进行实际的 compaction 动作，其他副本的从该副本拉取完成 compaction 的 rowset。
enable_duplicate_without_keys_by_default	当配置为`true`时，如果创建表的时候没有指定 Unique、Aggregate 或 Duplicate 时，会默认创建一个没有排序列和前缀索引的 Duplicate 模型的表。
skip_write_index_on_load	是否对这个表开启数据导入时不写索引。如果这个属性设置成 `true`, 数据导入的时候不写索引（目前仅对倒排索引生效），而是在 Compaction 的时候延迟写索引。这样可以避免首次写入和 Compaction 重复写索引的 CPU 和 IO 资源消耗，提升高吞吐导入的性能。
compaction_policy	配置这个表的 Compaction 的合并策略，仅支持配置为 time_series 或者 size_basedtime_series: 当 rowset 的磁盘体积积攒到一定大小时进行版本合并。合并后的 rowset 直接晋升到 base compaction 阶段。在时序场景持续导入的情况下有效降低 compact 的写入放大率。此策略将使用 time_series_compaction 为前缀的参数调整 Compaction 的执行
time_series_compaction_goal_size_mbytes	Compaction 的合并策略为 time_series 时，将使用此参数来调整每次 Compaction 输入的文件的大小，输出的文件大小和输入相当

属性名	作用
time_series_compaction_file_count_threshold	Compaction 的合并策略为 time_series 时，将使用此参数来调整每次 Compaction 输入的文件数量的最小值。一个 Tablet 中，文件数超过该配置，就会触发 Compaction
time_series_compaction_time_threshold_seconds	Compaction 的合并策略为 time_series 时，将使用此参数来调整 Compaction 的最长时间间隔，即长时间未执行过 Compaction 时，就会触发一次 Compaction，单位为秒
time_series_compaction_level_threshold	Compaction 的合并策略为 time_series 时，此参数默认为 1，当设置为 2 时用来控住对于合并过一次的段再合并一层，保证段大小达到 time_series_compaction_goal_size_mbytes，能达到段数量减少的效果。
group_commit_interval_ms	配置这个表的 Group Commit 攒批间隔。单位为 ms，默认值为 10000ms，即 10s。Group Commit 的下刷时机取决于 `group_commit_interval_ms`以及`group_commit_data_bytes`哪个先到设置的值。
group_commit_data_bytes	配置这个表的 Group Commit 攒批数据大小。单位为 bytes，默认值为 134217728，即 128MB。Group Commit 的下刷时机取决于 `group_commit_interval_ms`以及`group_commit_data_bytes` 哪个先到设置的值。
enable_mow_light_delete	是否在 Unique 表 Mow 上开启 Delete 语句写 Delete predicate。若开启，会提升 Delete 语句的性能，但 Delete 后进行部分列更新可能会出现部分数据错误的情况。若关闭，会降低 Delete 语句的性能来保证正确性。此属性的默认值为 `false`。此属性只能在 Unique Merge-on-Write 表上开启。
动态分区相关属性	动态分区相关参考数据划分 - 动态分区
enable_unique_key_skip_bitmap_column	是否在 Unique Merge-on-Write 表上开启灵活列更新功能。此属性只能在 Unique Merge-on-Write 表上开启。(自 3.1.0 版本起支持)

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
CREATE_PRIV	数据库（Database）	
SELECT_PRIV	表（Table），视图（View）	当执行 CTAS 时，需要拥有被查询的表，视图或物化视图的 SELECT_PRIV 权限

注意事项

- 数据库（Database）中不能包含具有相同名称的表（Table），视图（View）。
- 表名，列名，rollup 名，不能使用保留关键字（Reserved Keywords）
- CREATE TABLE ··· LIKE：
- 只能对 Doris 内表使用此命令
- 显示指定的 rollup 才会被复制

- 所有的同步物化视图都不会被复制
- CREATE TABLE ··· AS SELECT (CTAS)：
- 如果 SELECT 列表中列名的别名是有效的列，则在 CTAS 语句中不需要列定义；如果省略，列名和数据类型将从基础查询中推断出来：

```
CREATE TABLE <table_name> AS SELECT ...
```

- 或者，可以使用以下语法明确指定名称：

```
CREATE TABLE <table_name> ( <col1_name> , <col2_name> , ... ) AS SELECT ...
```

- 分区和分桶
- 一个表必须指定分桶列，但可以不指定分区。关于分区和分桶的具体介绍，可参阅数据划分 文档。
- Doris 中的表可以分为分区表和无分区的表。这个属性在建表时确定，之后不可更改。即对于分区表，可以在之后的使用过程中对分区进行增删操作，而对于无分区的表，之后不能再进行增加分区等操作。
- 分区列和分桶列在表创建之后不可更改，既不能更改分区和分桶列的类型，也不能对这些列进行任何增删操作。
- 动态分区
- 动态分区功能主要用于帮助用户自动的管理分区。通过设定一定的规则，Doris 系统定期增加新的分区或删除历史分区。可参阅动态分区 文档查看更多帮助。
- 自动分区
- 自动分区功能文档参见自动分区。
- 同步物化视图
- 用户可以在建表的同时创建多个同步物化视图（ ROLLUP ）。同步物化视图也可以在建表之后添加。写在建表语句中可以方便用户一次性创建所有同步物化视图。
- 如果在建表时创建好同步物化视图，则后续的所有数据导入操作都会同步生成同步物化视图的数据。同步物化视图的数量可能会影响数据导入的效率。
- 关于物化视图的介绍，请参阅文档同步物化视图。
- 索引
- 用户可以在建表的同时创建多个列的索引。索引也可以在建表之后再添加。
- 如果在之后的使用过程中添加索引，如果表中已有数据，则需要重写所有数据，因此索引的创建时间取决于当前数据量。

示例

基础示例

明细模型

```
CREATE TABLE t1
(
  c1 INT,
  c2 STRING
)
DUPLICATE KEY(c1)
```

```
DISTRIBUTED BY HASH(c1)
PROPERTIES (
  'replication_num' = '1'
)
```

聚合模型

```
CREATE TABLE t2
(
  c1 INT,
  c2 INT MAX
)
AGGREGATE KEY(c1)
DISTRIBUTED BY HASH(c1)
PROPERTIES (
  'replication_num' = '1'
)
```

主键模型

```
CREATE TABLE t3
(
  c1 INT,
  c2 INT
)
UNIQUE KEY(c1)
DISTRIBUTED BY HASH(c1)
PROPERTIES (
  'replication_num' = '1'
)
```

使用生成列

```
CREATE TABLE t4
(
  c1 INT,
  c2 INT GENERATED ALWAYS AS (c1 + 1)
)
DUPLICATE KEY(c1)
DISTRIBUTED BY HASH(c1)
PROPERTIES (
  'replication_num' = '1'
)
```

指定列的默认值

```
CREATE TABLE t5
```

```
(
  c1 INT,
  c2 INT DEFAULT 10
)
DUPLICATE KEY(c1)
DISTRIBUTED BY HASH(c1)
PROPERTIES (
  'replication_num' = '1'
)
```

分桶方式

```
CREATE TABLE t6
(
  c1 INT,
  c2 INT
)
DUPLICATE KEY(c1)
DISTRIBUTED BY RANDOM
PROPERTIES (
  'replication_num' = '1'
)
```

自动分区

```
CREATE TABLE t7
(
  c1 INT,
  c2 DATETIME NOT NULL
)
DUPLICATE KEY(c1)
AUTO PARTITION BY RANGE(date_trunc(c2, 'day')) ()
DISTRIBUTED BY RANDOM
PROPERTIES (
  'replication_num' = '1'
)
```

range 分区

```
CREATE TABLE t8
(
  c1 INT,
  c2 DATETIME NOT NULL
)
DUPLICATE KEY(c1)
PARTITION BY RANGE(c2) (
  FROM ('2020-01-01') TO ('2020-01-10') INTERVAL 1 DAY
```

```
)
DISTRIBUTED BY RANDOM
PROPERTIES (
  'replication_num' = '1'
)
```

list 分区

```
CREATE TABLE t9
(
  c1 INT,
  c2 DATE NOT NULL
)
DUPLICATE KEY(c1)
PARTITION BY LIST(c2) (
  PARTITION p1 VALUES IN (('2020-01-01'),('2020-01-02'))
)
DISTRIBUTED BY RANDOM
PROPERTIES (
  'replication_num' = '1'
)
```

存储介质和冷却时间

```
CREATE TABLE example_db.table_hash
(
    k1 BIGINT,
    k2 LARGEINT,
    v1 VARCHAR(2048),
    v2 SMALLINT DEFAULT "10"
)
UNIQUE KEY(k1, k2)
DISTRIBUTED BY HASH (k1, k2) BUCKETS 32
PROPERTIES(
    "storage_medium" = "SSD",
    "storage_cooldown_time" = "2015-06-04 00:00:00"
);
```

通过 storage_policy 属性设置表的冷热分层数据迁移策略

1. 需要先创建 s3 resource 和 storage policy，表才能关联迁移策略成功

```
    -- 非分区表
    CREATE TABLE IF NOT EXISTS create_table_use_created_policy
    (
        k1 BIGINT,
```

```
    k2 LARGEINT,
    v1 VARCHAR(2048)
)
UNIQUE KEY(k1)
DISTRIBUTED BY HASH (k1) BUCKETS 3
PROPERTIES(
    "storage_policy" = "test_create_table_use_policy",
    "replication_num" = "1"
);

-- 分区表
CREATE TABLE create_table_partion_use_created_policy
(
    k1 DATE,
    k2 INT,
    V1 VARCHAR(2048) REPLACE
) PARTITION BY RANGE (k1) (
    PARTITION p1 VALUES LESS THAN ("2022-01-01") ("storage_policy" = "test_create_table_
        ↪ partition_use_policy_1" ,"replication_num"="1"),
    PARTITION p2 VALUES LESS THAN ("2022-02-01") ("storage_policy" = "test_create_table_
        ↪ partition_use_policy_2" ,"replication_num"="1")
) DISTRIBUTED BY HASH(k2) BUCKETS 1;
```

Colocation Group

```
CREATE TABLE t1 (
    id int(11) COMMENT "",
    value varchar(8) COMMENT ""
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 10
PROPERTIES (
    "colocate_with" = "group1"
);

CREATE TABLE t2 (
    id int(11) COMMENT "",
    value1 varchar(8) COMMENT "",
    value2 varchar(8) COMMENT ""
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 10
PROPERTIES (
    "colocate_with" = "group1"
);
```

索引

```
CREATE TABLE example_db.table_hash
(
    k1 TINYINT,
    k2 DECIMAL(10, 2) DEFAULT "10.5",
    v1 CHAR(10) REPLACE,
    v2 INT SUM,
    INDEX k1_idx (k1) USING INVERTED COMMENT 'my first index'
)
AGGREGATE KEY(k1, k2)
DISTRIBUTED BY HASH(k1) BUCKETS 32
PROPERTIES (
    "bloom_filter_columns" = "k2"
);
```

设置表的副本属性

```
CREATE TABLE example_db.table_hash
(
    k1 TINYINT,
    k2 DECIMAL(10, 2) DEFAULT "10.5"
)
DISTRIBUTED BY HASH(k1) BUCKETS 32
PROPERTIES (
    "replication_allocation"="tag.location.group_a:1, tag.location.group_b:2"
);
```

动态分区

该表每天提前创建 3 天的分区，并删除 3 天前的分区。例如今天为2020-01-08，则会创建分区名为p20200108, p20200109, p20200110, p20200111的分区。分区范围分别为：

```
[types: [DATE]; keys: [2020-01-08]; ··types: [DATE]; keys: [2020-01-09]; )
[types: [DATE]; keys: [2020-01-09]; ··types: [DATE]; keys: [2020-01-10]; )
[types: [DATE]; keys: [2020-01-10]; ··types: [DATE]; keys: [2020-01-11]; )
[types: [DATE]; keys: [2020-01-11]; ··types: [DATE]; keys: [2020-01-12]; )
CREATE TABLE example_db.dynamic_partition
(
    k1 DATE,
    k2 INT,
    k3 SMALLINT,
    v1 VARCHAR(2048),
    v2 DATETIME DEFAULT "2014-02-04 15:36:00"
)
DUPLICATE KEY(k1, k2, k3)
PARTITION BY RANGE (k1) ()
```

```
DISTRIBUTED BY HASH(k2) BUCKETS 32
PROPERTIES(
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-3",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "32"
);
```

设置动态分区的副本属性

```
CREATE TABLE example_db.dynamic_partition
(
    k1 DATE,
    k2 INT,
    k3 SMALLINT,
    v1 VARCHAR(2048),
    v2 DATETIME DEFAULT "2014-02-04 15:36:00"
)
PARTITION BY RANGE (k1) ()
DISTRIBUTED BY HASH(k2) BUCKETS 32
PROPERTIES(
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.start" = "-3",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "32",
    "dynamic_partition.replication_allocation" = "tag.location.group_a:3"
 );
```

CTAS 示例

```
CREATE TABLE t10
PROPERTIES (
  'replication_num' = '1'
)
AS SELECT * FROM t1
```

CREATE TABLE ··· LIKE 示例

```
CREATE TABLE t11 LIKE t10
```

7.3.8.1.2 DESCRIBE

描述

该语句用于展示指定 table 的 schema 信息

语法

```
DESC[RIBE] [<ctl_name>.][<db_name>.]<table_name> [ALL];
```

必选参数

1.<table_name>

> 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1.<ctl_name>.<db_name>

> 指定数据目录和数据库的标识符（即名称）。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Database`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2.ALL

> 仅针对内表有效。返回内表的所有 Index 信息。

返回值

不指定 ALL 的情况下，返回值如下：

列名	说明
Field	列名
Type	数据类型
Null	是否允许为 NULL 值
Key	是否为 key 列
Default	默认值
Extra	显示一些额外的信息

在 3.0.7 版本中，新增会话变量 show_column_comment_in_describe。当指定为 true 时，将额外增加 Comment 列，用于显示列的注释信息。

指定 ALL 的情况下，针对内表，返回值如下：

列名	说明
IndexName	表名
IndexKeysType	表模型
Field	列名
Type	数据类型
Null	是否允许为 NULL 值
Key	是否为 key 列
Default	默认值
Extra	显示一些额外的信息
Visible	是否可见
DefineExpr	定义表达式
WhereClause	过滤条件相关的定义

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	当执行 DESC 时，需要拥有被查询的表的 SELECT_PRIV 权限

注意事项

- 如果指定 ALL，则显示该 table 的所有 index(rollup) 的 schema

举例

1. 显示 Base 表 Schema

```
DESC test_table;
```

```
+---------+-------------+------+-------+---------+-------+
| Field   | Type        | Null | Key   | Default | Extra |
+---------+-------------+------+-------+---------+-------+
| user_id | bigint      | No   | true  | NULL    |       |
| name    | varchar(20) | Yes  | false | NULL    | NONE  |
| age     | int         | Yes  | false | NULL    | NONE  |
+---------+-------------+------+-------+---------+-------+
```

2. 显示表所有 index 的 schema

```
DESC demo.test_table ALL;
```

```
+------------+--------------+---------+-------------+--------------+------+-------+---------+-------+---------+
    ↪
| IndexName  | IndexKeysType | Field   | Type        | InternalType | Null | Key   | Default |
    ↪ Extra | Visible | DefineExpr | WhereClause |
+------------+--------------+---------+-------------+--------------+------+-------+---------+-------+---------+
    ↪
| test_table | DUP_KEYS      | user_id | bigint      | bigint       | No   | true  | NULL    |
    ↪       | true    |            |             |
|            |               | name    | varchar(20) | varchar(20)  | Yes  | false | NULL    |
    ↪ NONE  | true    |            |             |
|            |               | age     | int         | int          | Yes  | false | NULL    |
    ↪ NONE  | true    |            |             |
+------------+--------------+---------+-------------+--------------+------+-------+---------+-------+---------+
    ↪
```

7.3.8.1.3 ALTER TABLE COLUMN

描述

该语句用于对已有 table 进行 Schema change 操作。schema change 是异步的，任务提交成功则返回，之后可使用SHOW ALTER TABLE COLUMN 命令查看进度。

> Doris 在建表之后有物化索引的概念，在建表成功后为 base 表，物化索引为 base index，基于 base 表可以创建 rollup index。其中 base index 和 rollup index 都是物化索引，在进行 schema change 操作时如果不指定 rollup_index_name 默认基于 base 表进行操作。Doris 在 1.2.0 支持了 light schema change 轻量表结构变更，对于值列的加减操作，可以更快地，同步地完成。可以在建表时手动指定 “light_schema_change” = ‘true’，2.0.0 及之后版本该参数默认开启。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

schema change 的 alter_clause 支持如下几种修改方式：

1. 添加列，向指定的 index 位置进行列添加

语法

```
ALTER TABLE [database.]table table_name ADD COLUMN column_name column_type [KEY | agg_type] [
    ↪ DEFAULT "default_value"]
[AFTER column_name|FIRST]
[TO rollup_index_name]
[PROPERTIES ("key"="value", ...)]
```

Example

1. 向 example_db.my_table 的 key_1 后添加一个 key 列 new_col (非聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN new_col INT KEY DEFAULT "0" AFTER key_1;
```

2. 向 example_db.my_table 的 value_1 后添加一个 value 列 new_col (非聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN new_col INT DEFAULT "0" AFTER value_1;
```

3. 向 example_db.my_table 的 key_1 后添加一个 key 列 new_col (聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN new_col INT KEY DEFAULT "0" AFTER key_1;
```

4. 向 example_db.my_table 的 value_1 后添加一个 value 列 new_col SUM 聚合类型 (聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN new_col INT SUM DEFAULT "0" AFTER value_1;
```

5. 将 new_col 添加到 example_db.my_table 表的首列位置 (非聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN new_col INT KEY DEFAULT "0" FIRST;
```

- 聚合模型如果增加 value 列，需要指定 agg_type
- 非聚合模型（如 DUPLICATE KEY）如果增加 key 列，需要指定 KEY 关键字
- 不能在 rollup index 中增加 base index 中已经存在的列（如有需要，可以重新创建一个 rollup index）

2. 添加多列，向指定的 index 位置进行多列添加

语法

```
ALTER TABLE [database.]table table_name ADD COLUMN (column_name1 column_type [KEY | agg_type]
    ↪ DEFAULT "default_value", ...)
[TO rollup_index_name]
[PROPERTIES ("key"="value", ...)]
```

Example

1. 向 example_db.my_table 中添加多列，new_col 和 new_col2 都是 SUM 聚合类型 (聚合模型)

```
ALTER TABLE example_db.my_table
ADD COLUMN (new_col1 INT SUM DEFAULT "0" ,new_col2 INT SUM DEFAULT "0");
```

2. 向 example_db.my_table 中添加多列 (非聚合模型)，其中 new_col1 为 KEY 列，new_col2 为 value 列

```
ALTER TABLE example_db.my_table
ADD COLUMN (new_col1 INT key DEFAULT "0" , new_col2 INT DEFAULT "0");
```

- 聚合模型如果增加 value 列，需要指定 agg_type
- 聚合模型如果增加 key 列，需要指定 KEY 关键字
- 不能在 rollup index 中增加 base index 中已经存在的列（如有需要，可以重新创建一个 rollup index）

3. 删除列，从指定 index 中删除一列

语法

```
ALTER TABLE [database.]table table_name DROP COLUMN column_name
[FROM rollup_index_name]
```

Example

1. 从 example_db.my_table 中删除 col1 列

```
ALTER TABLE example_db.my_table DROP COLUMN col1;
```

- 不能删除分区列
- 聚合模型不能删除 KEY 列
- 如果是从 base index 中删除列，则如果 rollup index 中包含该列，也会被删除

4. 修改指定列类型以及列位置

语法

```
ALTER TABLE [database.]table table_name MODIFY COLUMN column_name column_type [KEY | agg_type] [
    ↪ NULL | NOT NULL] [DEFAULT "default_value"]
[AFTER column_name|FIRST]
[FROM rollup_index_name]
[PROPERTIES ("key"="value", ...)]
```

Example

1. 修改 example_db.my_table 的 key 列 col1 的类型为 BIGINT，并移动到 col2 列后面。

```
ALTER TABLE example_db.my_table
MODIFY COLUMN col1 BIGINT KEY DEFAULT "1" AFTER col2;
```

无论是修改 key 列还是 value 列都需要声明完整的 column 信息

2. 修改 example_db.my_table 的 val1 列最大长度。原 val1 为 (val1 VARCHAR(32) REPLACE DEFAULT “abc”)

```
ALTER TABLE example_db.my_table
MODIFY COLUMN val1 VARCHAR(64) REPLACE DEFAULT "abc";
```

只能修改列的类型，列的其他属性维持原样

3. 修改 Duplicate key 表 Key 列的某个字段的长度

```
ALTER TABLE example_db.my_table
MODIFY COLUMN k3 VARCHAR(50) KEY NULL COMMENT 'to 50';
```

- 聚合模型如果修改 value 列，需要指定 agg_type
- 非聚合类型如果修改 key 列，需要指定 KEY 关键字
- 只能修改列的类型，列的其他属性维持原样（即其他属性需在语句中按照原属性显式的写出，参见 example 8）
- 分区列和分桶列不能做任何修改
- 目前支持以下类型的转换（精度损失由用户保证）
  - TINYINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE 类型向范围更大的数字类型转换

- TINTINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE/DECIMAL 转换成 VARCHAR
- VARCHAR 支持修改最大长度
- VARCHAR/CHAR 转换成 TINTINT/SMALLINT/INT/BIGINT/LARGEINT/FLOAT/DOUBLE
- VARCHAR/CHAR 转换成 DATE (目前支持 “%Y-%m-%d”, “%y-%m-%d”, “%Y%m%d”, “%y%m%d”, “%Y/%m/%d,” %y/%m/%d “六种格式化格式)
- DATETIME 转换成 DATE(仅保留年-月-日信息，例如：2019-12-09 21:47:05 <-> 2019-12-09)
- DATE 转换成 DATETIME(时分秒自动补零，例如：2019-12-09 <-> 2019-12-09 00:00:00)
- FLOAT 转换成 DOUBLE
- INT 转换成 DATE (如果 INT 类型数据不合法则转换失败，原始数据不变)
- 除 DATE 与 DATETIME 以外都可以转换成 STRING，但是 STRING 不能转换任何其他类型

5. 对指定表的列进行重新排序

语法

```
ALTER TABLE [database.]table table_name ORDER BY (column_name1, column_name2, ...)
[FROM rollup_index_name]
[PROPERTIES ("key"="value", ...)]
```

Example

1. 调整 example_db.my_table 的 key 列和 value 列的顺序（非聚合模型）

```
CREATE TABLE `my_table`(
`k_1` INT NULL,
`k_2` INT NULL,
`v_1` INT NULL,
`v_2` varchar NULL,
`v_3` varchar NULL
) ENGINE=OLAP
DUPLICATE KEY(`k_1`, `k_2`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`k_1`) BUCKETS 5
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);

ALTER TABLE example_db.my_table ORDER BY (k_2,k_1,v_3,v_2,v_1);

mysql> desc my_table;
+-------+------------+------+-------+---------+-------+
| Field | Type       | Null | Key   | Default | Extra |
+-------+------------+------+-------+---------+-------+
```

```
| k_2   | INT         | Yes  | true  | NULL    |       |
| k_1   | INT         | Yes  | true  | NULL    |       |
| v_3   | VARCHAR(*) | Yes  | false | NULL    | NONE  |
| v_2   | VARCHAR(*) | Yes  | false | NULL    | NONE  |
| v_1   | INT         | Yes  | false | NULL    | NONE  |
+-------+------------+------+-------+---------+-------+
```

2. 同时执行添加列和列排序操作

```
CREATE TABLE `my_table` (
`k_1` INT NULL,
`k_2` INT NULL,
`v_1` INT NULL,
`v_2` varchar NULL,
`v_3` varchar NULL
) ENGINE=OLAP
DUPLICATE KEY(`k_1`, `k_2`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`k_1`) BUCKETS 5
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);

ALTER TABLE example_db.my_table
ADD COLUMN col INT DEFAULT "0" AFTER v_1,
ORDER BY (k_2,k_1,v_3,v_2,v_1,col);

mysql> desc my_table;
+-------+------------+------+-------+---------+-------+
| Field | Type       | Null | Key   | Default | Extra |
+-------+------------+------+-------+---------+-------+
| k_2   | INT         | Yes  | true  | NULL    |       |
| k_1   | INT         | Yes  | true  | NULL    |       |
| v_3   | VARCHAR(*) | Yes  | false | NULL    | NONE  |
| v_2   | VARCHAR(*) | Yes  | false | NULL    | NONE  |
| v_1   | INT         | Yes  | false | NULL    | NONE  |
| col   | INT         | Yes  | false | 0       | NONE  |
+-------+------------+------+-------+---------+-------+
```

- index 中的所有列都要写出来
- value 列在 key 列之后
- key 列只能调整 key 列的范围内进行调整，value 列同理

关键词

```
ALTER, TABLE, COLUMN, ALTER TABLE
```

最佳实践

7.3.8.1.4 ALTER TABLE PARTITION

描述

该语句用于对有 partition 的 table 进行修改操作。

这个操作是同步的，命令返回表示执行完毕。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

partition 的 alter_clause 支持如下几种修改方式

1. 增加分区

语法：

```
ADD PARTITION [IF NOT EXISTS] partition_name
partition_desc ["key"="value"]
[DISTRIBUTED BY HASH (k1[,k2 ...]) [BUCKETS num]]
```

注意：

- partition_desc 支持以下两种写法
- VALUES LESS THAN [MAXVALUE|(“value1”, …)]
- VALUES [(“value1”, …), (“value1”, …))
- 分区为左闭右开区间，如果用户仅指定右边界，系统会自动确定左边界
- 如果没有指定分桶方式，则自动使用建表使用的分桶方式和分桶数。
- 如指定分桶方式，只能修改分桶数，不可修改分桶方式或分桶列。如果指定了分桶方式，但是没有指定分桶数，则分桶数会使用默认值 10，不会使用建表时指定的分桶数。如果要指定分桶数，则必须指定分桶方式。
- [“key” = “value”] 部分可以设置分区的一些属性，具体说明见 CREATE TABLE
- 如果建表时用户未显式创建 Partition，则不支持通过 ALTER 的方式增加分区
- 如果用户使用的是 List Partition 则可以增加 default partition，default partition 将会存储所有不满足其他分区键要求的数据。
- ALTER TABLE table_name ADD PARTITION partition_name

2. 删除分区

语法：

```
DROP PARTITION [IF EXISTS] partition_name [FORCE]
```

注意：

- 使用分区方式的表至少要保留一个分区。
- 执行 DROP PARTITION 一段时间内，可以通过 RECOVER 语句恢复被删除的分区。详见 SQL 手册 - 数据库管理 -RECOVER 语句
- 如果执行 DROP PARTITION FORCE，则系统不会检查该分区是否存在未完成的事务，分区将直接被删除并且不能被恢复，一般不建议执行此操作

3. 修改分区属性

语法：

```
MODIFY PARTITION p1|(p1[, p2, ...]) SET ("key" = "value", ...)
```

说明：

- 当前支持修改分区的下列属性：
- storage_medium
- storage_cooldown_time
- replication_num
- in_memory
- 对于单分区表，partition_name 同表名。

示例

1. 增加分区，现有分区 [MIN, 2013-01-01)，增加分区 [2013-01-01, 2014-01-01)，使用默认分桶方式

```
ALTER TABLE example_db.my_table
ADD PARTITION p1 VALUES LESS THAN ("2014-01-01");
```

2. 增加分区，使用新的分桶数

```
ALTER TABLE example_db.my_table
ADD PARTITION p1 VALUES LESS THAN ("2015-01-01")
DISTRIBUTED BY HASH(k1) BUCKETS 20;
```

3. 增加分区，使用新的副本数

```
ALTER TABLE example_db.my_table
ADD PARTITION p1 VALUES LESS THAN ("2015-01-01")
("replication_num"="1");
```

4. 修改分区副本数

```
ALTER TABLE example_db.my_table
MODIFY PARTITION p1 SET("replication_num"="1");
```

5. 批量修改指定分区

```
ALTER TABLE example_db.my_table
MODIFY PARTITION (p1, p2, p4) SET("replication_num"="1");
```

6. 批量修改所有分区

```
ALTER TABLE example_db.my_table
MODIFY PARTITION (*) SET("storage_medium"="HDD");
```

7. 删除分区

```
ALTER TABLE example_db.my_table
DROP PARTITION p1;
```

8. 批量删除分区

```
ALTER TABLE example_db.my_table
DROP PARTITION p1,
DROP PARTITION p2,
DROP PARTITION p3;
```

9. 增加一个指定上下界的分区

```
ALTER TABLE example_db.my_table
ADD PARTITION p1 VALUES [("2014-01-01"), ("2014-02-01"));
```

10. 批量增加数字类型和时间类型的分区

```
ALTER TABLE example_db.my_table ADD PARTITIONS FROM (1) TO (100) INTERVAL 10;
ALTER TABLE example_db.my_table ADD PARTITIONS FROM ("2023-01-01") TO ("2025-01-01") INTERVAL 1
    ↪ YEAR;
ALTER TABLE example_db.my_table ADD PARTITIONS FROM ("2023-01-01") TO ("2025-01-01") INTERVAL 1
    ↪ MONTH;
ALTER TABLE example_db.my_table ADD PARTITIONS FROM ("2023-01-01") TO ("2025-01-01") INTERVAL 1
    ↪ WEEK;
ALTER TABLE example_db.my_table ADD PARTITIONS FROM ("2023-01-01") TO ("2025-01-01") INTERVAL 1
    ↪ DAY;
```

关键词

```
ALTER, TABLE, PARTITION, ALTER TABLE
```

7.3.8.1.5 ALTER TABLE ROLLUP

描述

该语句用于对已有 table 进行 rollup 进行修改操作。rollup 是异步操作，任务提交成功则返回，之后可使用SHOW ALTER 命令查看进度。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

rollup 的 alter_clause 支持如下几种创建方式

1. 创建 rollup index

语法：

```
ADD ROLLUP rollup_name (column_name1, column_name2, ...)
[FROM from_index_name]
[PROPERTIES ("key"="value", ...)]
```

properties: 支持设置超时时间，默认超时时间为 1 天。

2. 批量创建 rollup index

语法：

```
ADD ROLLUP [rollup_name (column_name1, column_name2, ...)
                    [FROM from_index_name]
                    [PROPERTIES ("key"="value", ...)],...]
```

注意：

- 如果没有指定 from_index_name，则默认从 base index 创建
- rollup 表中的列必须是 from_index 中已有的列
- 在 properties 中，可以指定存储格式。具体请参阅 CREATE TABLE

3. 删除 rollup index

语法：

```
DROP ROLLUP rollup_name [PROPERTIES ("key"="value", ...)]
```

4. 批量删除 rollup index

语法：

```
DROP ROLLUP [rollup_name [PROPERTIES ("key"="value", ...)],...]
```

注意：

- 不能删除 base index

示例

1. 创建 index: example_rollup_index，基于 base index（k1,k2,k3,v1,v2）。列式存储。

```
ALTER TABLE example_db.my_table
ADD ROLLUP example_rollup_index(k1, k3, v1, v2);
```

2. 创建 index: example_rollup_index2，基于 example_rollup_index（k1,k3,v1,v2）

```
ALTER TABLE example_db.my_table
ADD ROLLUP example_rollup_index2 (k1, v1)
FROM example_rollup_index;
```

3. 创建 index: example_rollup_index3, 基于 base index (k1,k2,k3,v1), 自定义 rollup 超时时间一小时。

```
ALTER TABLE example_db.my_table
ADD ROLLUP example_rollup_index(k1, k3, v1)
PROPERTIES("timeout" = "3600");
```

4. 删除 index: example_rollup_index2

```
ALTER TABLE example_db.my_table
DROP ROLLUP example_rollup_index2;
```

5. 批量删除 Rollup

```
ALTER TABLE example_db.my_table
DROP ROLLUP example_rollup_index2,example_rollup_index3;
```

关键词

```
ALTER, TABLE, ROLLUP, ALTER TABLE
```

7.3.8.1.6 ALTER TABLE RENAME

描述

该语句用于对已有 table 属性的某些名称进行重命名操作。这个操作是同步的，命令返回表示执行完毕。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

rename 的 alter_clause 支持对以下名称进行修改

1. 修改表名

语法：

```
RENAME new_table_name;
```

2. 修改 rollup index 名称

语法：

```
RENAME ROLLUP old_rollup_name new_rollup_name;
```

3. 修改 partition 名称

语法：

```
RENAME PARTITION old_partition_name new_partition_name;
```

4. 修改 column 名称

修改 column 名称

语法：

```
RENAME COLUMN old_column_name new_column_name;
```

注意： - 建表时需要在 property 中设置 light_schema_change=true

示例

1. 将名为 table1 的表修改为 table2

```
ALTER TABLE table1 RENAME table2;
```

2. 将表 example_table 中名为 rollup1 的 rollup index 修改为 rollup2

```
ALTER TABLE example_table RENAME ROLLUP rollup1 rollup2;
```

3. 将表 example_table 中名为 p1 的 partition 修改为 p2

```
ALTER TABLE example_table RENAME PARTITION p1 p2;
```

4. 将表 example_table 中名为 c1 的 column 修改为 c2

```
ALTER TABLE example_table RENAME COLUMN c1 c2;
```

关键词

```
ALTER, TABLE, RENAME, ALTER TABLE
```

7.3.8.1.7 ALTER TABLE REPLACE

描述

对两个表进行原子的替换操作。该操作仅适用于 OLAP 表。

```
ALTER TABLE [db.]tbl1 REPLACE WITH TABLE tbl2
[PROPERTIES('swap' = 'true')];
```

将表 tbl1 替换为表 tbl2。

如果 swap 参数为 `true`，则替换后，名称为 `tbl1` 表中的数据为原 `tbl2` 表中的数据。而名称为 `tbl2` 表中的数据为原 `tbl1` 表中的数据。即两张表数据发生了互换。

如果 swap 参数为 `false`，则替换后，名称为 `tbl1` 表中的数据为原 `tbl2` 表中的数据。而名称为 `tbl2` 表被删除。

原理

替换表功能，实际上是将以下操作集合变成一个原子操作。

假设要将表 A 替换为表 B，且 swap 为 `true`，则操作如下：

1. 将表 B 重名为表 A。
2. 将表 A 重名为表 B。

如果 swap 为 `false`，则操作如下：

1. 删除表 A。
2. 将表 B 重名为表 A。

注意事项

1. swap 参数默认为 `true`。即替换表操作相当于将两张表数据进行交换。

2. 如果设置 swap 参数为 false，则被替换的表（表 A）将被删除，且无法恢复。
3. 替换操作仅能发生在两张 OLAP 表之间，且不会检查两张表的表结构是否一致。
4. 替换操作不会改变原有的权限设置。因为权限检查以表名称为准。

示例

1. 将 tbl1 与 tbl2 进行原子交换，不删除任何表（注：如果删除的话，实际上删除的是 tbl1，只是将 tbl2 重命名为 tbl1。）

```
ALTER TABLE tbl1 REPLACE WITH TABLE tbl2;
```

或

```
ALTER TABLE tbl1 REPLACE WITH TABLE tbl2 PROPERTIES('swap' = 'true') ;
```

2. 将 tbl1 与 tbl2 进行交换，删除 tbl2 表（保留名为tbl1, 数据为tbl2的表）

```
ALTER TABLE tbl1 REPLACE WITH TABLE tbl2 PROPERTIES('swap' = 'false') ;
```

关键词

```
ALTER, TABLE, REPLACE, ALTER TABLE
```

最佳实践

1. 原子的覆盖写操作

某些情况下，用户希望能够重写某张表的数据，但如果采用先删除再导入的方式进行，在中间会有一段时间无法查看数据。这时，用户可以先使用 CREATE TABLE LIKE 语句创建一个相同结构的新表，将新的数据导入到新表后，通过替换操作，原子的替换旧表，以达到目的。分区级别的原子覆盖写操作，请参阅临时分区文档。

7.3.8.1.8 ALTER TABLE PROPERTY

> 分区属性与表属性的一些区别 - 分区属性一般主要关注分桶数 (buckets)、存储介质 (storage_medium)、副本数 (replication)、冷热分离存储策略 (storage_policy)；- 对于已经创建的分区，可以使用 alter table {tableName} modify partition({partitionName}) set ({key}={value}) 来修改，但是分桶数 (buckets) 不能修改；- 对于未创建的动态分区 (dynamic partition)，可以使用 alter table {tableName} set (dynamic_partition.{key} = {value}) 来修改其属性；- 对于未创建的自动分区 (auto partition)，可以使用 alter table {tableName} set ({key} = {value}) 来修改其属性；- 若用户想修改分区的属性，需要修改已经创建分区的属性，同时也要修改未创建分区的属性 - 除了上面几个属性，其他均为表级别属性 - 具体属性可以参考建表属性

描述

该语句用于对已有 table 的 property 进行修改操作。这个操作是同步的，命令返回表示执行完毕。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

property 的 alter_clause 支持如下几种修改方式

1. 修改表的 bloom filter 列

```
ALTER TABLE example_db.my_table SET ("bloom_filter_columns"="k1,k2,k3");
```

也可以合并到上面的 schema change 操作中（注意多子句的语法有少许区别）

```
ALTER TABLE example_db.my_table
DROP COLUMN col2
PROPERTIES ("bloom_filter_columns"="k1,k2,k3");
```

2. 修改表的 Colocate 属性

```
ALTER TABLE example_db.my_table set ("colocate_with" = "t1");
```

3. 将表的分桶方式由 Hash Distribution 改为 Random Distribution

```
ALTER TABLE example_db.my_table set ("distribution_type" = "random");
```

4. 修改表的动态分区属性 (支持未添加动态分区属性的表添加动态分区属性)

```
ALTER TABLE example_db.my_table set ("dynamic_partition.enable" = "false");
```

如果需要在未添加动态分区属性的表中添加动态分区属性，则需要指定所有的动态分区属性 (注：非分区表不支持添加动态分区属性)

```
ALTER TABLE example_db.my_table set (
  "dynamic_partition.enable" = "true",
  "dynamic_partition.time_unit" = "DAY",
  "dynamic_partition.end" = "3",
  "dynamic_partition.prefix" = "p",
  "dynamic_partition.buckets" = "32"
);
```

5. 修改表的 in_memory 属性，只支持修改为’false’

```
ALTER TABLE example_db.my_table set ("in_memory" = "false");
```

6. 启用批量删除功能

```
ALTER TABLE example_db.my_table ENABLE FEATURE "BATCH_DELETE";
```

注意：

- 只能用在 unique 表
- 用于旧表支持批量删除功能，新表创建时已经支持

7. 启用按照 sequence column 的值来保证导入顺序的功能

```
ALTER TABLE example_db.my_table ENABLE FEATURE "SEQUENCE_LOAD" WITH PROPERTIES (
  "function_column.sequence_type" = "Date"
);
```

注意：

- 只能用在 unique 表
- sequence_type 用来指定 sequence 列的类型，可以为整型和时间类型
- 只支持新导入数据的有序性，历史数据无法更改

8. 将表的默认分桶数改为 50

```
ALTER TABLE example_db.my_table MODIFY DISTRIBUTION DISTRIBUTED BY HASH(k1) BUCKETS 50;
```

注意：

- 只能用在分区类型为 RANGE，采用哈希分桶的非 colocate 表

9. 修改表注释

```
ALTER TABLE example_db.my_table MODIFY COMMENT "new comment";
```

10. 修改列注释

```
ALTER TABLE example_db.my_table MODIFY COLUMN k1 COMMENT "k1", MODIFY COLUMN k2 COMMENT "k2";
```

11. 修改引擎类型

    仅支持将 MySQL 类型修改为 ODBC 类型。 driver 的值为 odbc.init 配置中的 driver 名称。

```
ALTER TABLE example_db.mysql_table MODIFY ENGINE TO odbc PROPERTIES("driver" = "MySQL");
```

12. 修改副本数

```
ALTER TABLE example_db.mysql_table SET ("replication_num" = "2");
ALTER TABLE example_db.mysql_table SET ("default.replication_num" = "2");
ALTER TABLE example_db.mysql_table SET ("replication_allocation" = "tag.location.default: 1");
ALTER TABLE example_db.mysql_table SET ("default.replication_allocation" = "tag.location.default:
    ↪ 1");
```

注：1. default 前缀的属性表示修改表的默认副本分布。这种修改不会修改表的当前实际副本分布，而只影响分区表上新建分区的副本分布。2. 对于非分区表，修改不带 default 前缀的副本分布属性，会同时修改表的默认副本分布和实际副本分布。即修改后，通过 `show create table` 和 `show partitions from tbl` 语句可以看到副本分布数据都被修改了。3. 对于分区表，表的实际副本分布是分区级别的，即每个分区有自己的副本分布，可以通过 `show partitions from tbl` 语句查看。如果想修改实际副本分布，请参阅 `ALTER TABLE PARTITION`。

13. [Experimental] 打开`light_schema_change`

对于建表时未开启 light_schema_change 的表，可以通过如下方式打开。

```
ALTER TABLE example_db.mysql_table SET ("light_schema_change" = "true");
```

示例

1. 修改表的 bloom filter 列

```
ALTER TABLE example_db.my_table SET (
  "bloom_filter_columns"="k1,k2,k3"
);
```

也可以合并到上面的 schema change 操作中（注意多子句的语法有少许区别）

```
ALTER TABLE example_db.my_table
DROP COLUMN col2
PROPERTIES (
  "bloom_filter_columns"="k1,k2,k3"
);
```

2. 修改表的 Colocate 属性

```
ALTER TABLE example_db.my_table set ("colocate_with" = "t1");
```

3. 将表的分桶方式由 Hash Distribution 改为 Random Distribution

```
ALTER TABLE example_db.my_table set (
  "distribution_type" = "random"
);
```

4. 修改表的动态分区属性 (支持未添加动态分区属性的表添加动态分区属性)

```
ALTER TABLE example_db.my_table set (
  "dynamic_partition.enable" = "false"
);
```

如果需要在未添加动态分区属性的表中添加动态分区属性，则需要指定所有的动态分区属性 (注：非分区表不支持添加动态分区属性)

```
ALTER TABLE example_db.my_table set (
  "dynamic_partition.enable" = "true",
  "dynamic_partition.time_unit" = "DAY",
  "dynamic_partition.end" = "3",
  "dynamic_partition.prefix" = "p",
  "dynamic_partition.buckets" = "32"
);
```

5. 修改表的 in_memory 属性，只支持修改为’ false’

```
ALTER TABLE example_db.my_table set ("in_memory" = "false");
```

6. 启用批量删除功能

```
ALTER TABLE example_db.my_table ENABLE FEATURE "BATCH_DELETE";
```

7. 启用按照 sequence column 的值来保证导入顺序的功能

```
ALTER TABLE example_db.my_table ENABLE FEATURE "SEQUENCE_LOAD" WITH PROPERTIES (
  "function_column.sequence_type" = "Date"
);
```

8. 将表的默认分桶数改为 50

```
ALTER TABLE example_db.my_table MODIFY DISTRIBUTION DISTRIBUTED BY HASH(k1) BUCKETS 50;
```

9. 修改表注释

```
ALTER TABLE example_db.my_table MODIFY COMMENT "new comment";
```

10. 修改列注释

```
ALTER TABLE example_db.my_table MODIFY COLUMN k1 COMMENT "k1", MODIFY COLUMN k2 COMMENT "k2";
```

11. 修改引擎类型

```
ALTER TABLE example_db.mysql_table MODIFY ENGINE TO odbc PROPERTIES("driver" = "MySQL");
```

12. 给表添加冷热分层数据迁移策略

```
ALTER TABLE create_table_not_have_policy set ("storage_policy" = "created_create_table_
    ↪ alter_policy");
```

注：表没有关联过 storage policy，才能被添加成功，一个表只能添加一个 storage policy

13. 给表的 partition 添加冷热分层数据迁移策略

```
ALTER TABLE create_table_partition MODIFY PARTITION (*) SET("storage_policy"="created_create
    ↪ _table_partition_alter_policy");
```

注：表的 partition 没有关联过 storage policy，才能被添加成功，一个表只能添加一个 storage policy

关键词

```
ALTER, TABLE, PROPERTY, ALTER TABLE
```

7.3.8.1.9 ALTER TABLE COMMENT

描述

该语句用于对已有 table 的 comment 进行修改。这个操作是同步的，命令返回表示执行完毕。

语法：

```
ALTER TABLE [database.]table alter_clause;
```

1. 修改表注释

语法：

```
MODIFY COMMENT "new table comment";
```

2. 修改列注释

语法：

```
MODIFY COLUMN col1 COMMENT "new column comment";
```

示例

1. 将名为 table1 的 comment 修改为 table1_comment

```
ALTER TABLE table1 MODIFY COMMENT "table1_comment";
```

2. 将名为 table1 的 col1 列的 comment 修改为 table1_col1_comment

```
ALTER TABLE table1 MODIFY COLUMN col1 COMMENT "table1_col1_comment";
```

关键词

```
ALTER, TABLE, COMMENT, ALTER TABLE
```

最佳实践

7.3.8.1.10 ALTER TABLE ADD GENERATED COLUMN

ALTER TABLE 和生成列

不支持使用 ALTER TABLE ADD COLUMN 增加一个生成列，不支持使用 ALTER TABLE MODIFY COLUMN 修改生成列信息。支持使用 ALTER TABLE 对生成列顺序进行修改，修改生成列名称和删除生成列。

不支持的场景报错如下：

```
mysql> CREATE TABLE test_alter_add_column(a int, b int) properties("replication_num"="1");
Query OK, 0 rows affected (0.14 sec)
mysql> ALTER TABLE test_alter_add_column ADD COLUMN c int AS (a+b);
ERROR 1105 (HY000): errCode = 2, detailMessage = Not supporting alter table add generated columns
    ↪ .
mysql> ALTER TABLE test_alter MODIFY COLUMN c int KEY AS (a+b+1);
ERROR 1105 (HY000): errCode = 2, detailMessage = Not supporting alter table modify generated
    ↪ columns.
```

REORDER COLUMN

```
ALTER TABLE products ORDER BY (product_id, total_value, price, quantity);
```

注意：修改后的列顺序仍然需要满足生成列建表时的顺序限制。####### RENAME COLUMN

```
ALTER TABLE products RENAME COLUMN total_value new_name;
```

注意：如果表中某列（生成列或者普通列）被其它生成列引用，需要先删除其它生成列后，才能修改此生成列的名称。####### DROP COLUMN

```
ALTER TABLE products DROP COLUMN total_value;
```

注意：如果表中某列（生成列或者普通列）被其它生成列引用，需要先删除其它生成列后，才能删除此被引用的生成列或者普通列。

7.3.8.1.11 SET FRONTEND CONFIG

描述

该语句用于设置集群的配置项（当前仅支持设置 FE 的配置项）。

语法：

```
ADMIN SET \{ALL FRONTENDS | FRONTEND\} CONFIG ("<fe_config_key>" = "<fe_config_value>")
```

必选参数

{ALL FRONTENDS | FRONTEND}

> ALL FRONTENDS：代表 Doris 集群中的所有 FE 节点 FRONTEND：代表当前连接的 FE 节点，即用户当前交互的 FE 节点

可选参数

需要修改的 <fe_config_key>、<fe_config_value> 可通过 SHOW FRONTEND CONFIG 命令查看

> 提示
>
> - 使用 ALL 关键字后配置参数将应用于所有 FE(除 master_only 参数外)。
> - 该语法不会持久化修改的配置，FE 重启后，修改的配置失效。如需持久化，需要在 fe.conf 中同步添加配置项。

示例

1. 设置 disable_balance 为 true

```
ADMIN SET FRONTEND CONFIG ("disable_balance" = "true");
```

2. 设置所有 FE 节点的 disable_balance 为 true

```
ADMIN SET ALL FRONTENDS CONFIG ("disable_balance" = "true");
```

7.3.8.1.12 SHOW ALTER TABLE

描述

该语句用于展示当前正在进行的各类修改任务的执行情况

```
SHOW ALTER [TABLE [COLUMN | ROLLUP] [FROM db_name]];
```

说明：

1. TABLE COLUMN：展示修改列的 ALTER 任务
2. 支持语法 [WHERE TableName|CreateTime|FinishTime|State] [ORDER BY] [LIMIT]
3. TABLE ROLLUP：展示创建或删除 ROLLUP 的任务
4. 如果不指定 db_name，使用当前默认 db

Result

SHOW ALTER TABLE COLUMN

字段名	描述
JobId	每个 Schema Change 作业的唯一 ID。
TableName	对应 Schema Change 的基表的表名。
CreateTime	作业创建时间。
FinishedTime	作业完成时间。如果未完成，显示 “N/A”。
IndexName	此修改中涉及的一个基表/同步物化视图的名称。
IndexId	新基表/同步物化视图的 ID。
OriginIndexId	此修改中涉及的一个基表/同步物化视图的 ID。
SchemaVersion	以 M:N 的格式显示。M 代表 Schema Change 的版本，N 代表对应的 Hash 值。每次 Schema Change 都会增加版本。
TransactionId	用于转换历史数据的事务 ID。
State	作业的阶段。 - PENDING: 作业正在等待在队列中调度。 - WAITING_TXN: 等待分界事务 ID 前的导入任务完成。 - RUNNING: 正在进行历史数据转换。 - FINISHED: 作业成功完成。 - CANCELLED: 作业失败。
Msg	如果作业失败，显示失败信息。
Progress	作业进度。仅在 RUNNING 状态下显示。进度以 M/N 的形式显示。N 是 Schema Change 中涉及的副本的总数。M 是已完成历史数据转换的副本数。
Timeout	作业超时时间，以秒为单位。

示例

1. 展示默认 db 的所有修改列的任务执行情况

```
SHOW ALTER TABLE COLUMN;
```

2. 展示某个表最近一次修改列的任务执行情况

```
SHOW ALTER TABLE COLUMN WHERE TableName = "table1" ORDER BY CreateTime DESC LIMIT 1;
```

3. 展示指定 db 的创建或删除 ROLLUP 的任务执行情况

```
SHOW ALTER TABLE ROLLUP FROM example_db;
```

关键词

```
SHOW, ALTER
```

最佳实践

7.3.8.1.13 TRUNCATE TABLE

描述

该语句用于清空指定表和分区的数据 ###### 语法

```
TRUNCATE TABLE [<db_name>.]<table_name>[ PARTITION ( <partition_name1> [, <partition_name2> ... ]
    ↪  ) ];
```

必选参数

1.<db_name> > 指定数据库的标识符（即名称）。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如My Database）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

2.<table_name> > 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如My Object）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1.<partition_name> > 指定分区的标识符（即名称）。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如My Object）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
Drop_priv	表（Table）	TRUNCATE TABLE 属于表 DROP 操作

注意事项

- 该语句清空数据，但保留表或分区。
- 不同于 DELETE，该语句只能整体清空指定的表或分区，不能添加过滤条件。
- 不同于 DELETE，使用该方式清空数据不会对查询性能造成影响。
- 该操作删除的数据不可恢复。
- 使用该命令时，表状态需为 NORMAL，即不允许正在进行 SCHEMA CHANGE 等操作。
- 该命令可能会导致正在进行的导入失败。

示例

1. 清空 example_db 下的表 tbl

```
TRUNCATE TABLE example_db.tbl;
```

2. 清空表 tbl 的 p1 和 p2 分区

```
TRUNCATE TABLE tbl PARTITION(p1, p2);
```

7.3.8.1.14 DROP TABLE

描述

该语句用于删除 Table。###### 语法

```
DROP TABLE [IF EXISTS] [<db_name>.]<table_name> [FORCE];
```

必选参数

1.<table_name> > 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1.<db_name> > 指定数据库的标识符（即名称）。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Database`）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

2.FORCE > 如果指定，则系统不会检查该表是否存在未完成的事务，表将直接被删除并且不能被恢复，一般不建议执行此操作。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
Drop_priv	表（Table）	DROP TABLE 属于表 DROP 操作

注意事项

- 执行 DROP TABLE 一段时间内，可以通过 RECOVER 语句恢复被删除的表。详见 RECOVER 语句。
- 如果执行 DROP TABLE FORCE，则系统不会检查该表是否存在未完成的事务，表将直接被删除并且不能被恢复，一般不建议执行此操作。

示例

1. 删除一个 Table

```
DROP TABLE my_table;
```

2. 如果存在，删除指定 Database 的 Table

```
DROP TABLE IF EXISTS example_db.my_table;
```

3. 如果存在，删除指定 Database 的 Table，强制删除

```
DROP TABLE IF EXISTS example_db.my_table FORCE;
```

7.3.8.1.15 SHOW CREATE TABLE

描述

该语句用于展示数据表的创建语句。

语法

```
SHOW [BRIEF] CREATE TABLE [<db_name>.]<table_name>
```

必选参数

1.<table_name> > 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如My Object）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1.BRIEF > 仅显示表的基本信息，不包括列的定义。

2.<db_name> > 指定数据库的标识符（即名称）。> > 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如My Database）。> > 标识符不能使用保留关键字。> > 有关更多详细信息，请参阅标识符要求和保留关键字。

返回值

列名	说明
Table	表名
Create Table	建表语句

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
Select_priv	表（Table）	SHOW CREATE TABLE 属于表 SELECT 操作

示例

1. 查看某个表的建表语句

```
SHOW CREATE TABLE demo.test_table;
```

2. 查看某个表的简化建表语句

```
SHOW BRIEF CREATE TABLE demo.test_table;
```

7.3.8.1.16 SHOW TABLES

描述

该语句用于展示当前 db 下所有的 table 以及 view。

语法

```
SHOW [ FULL ] TABLES [ FROM [ <catalog_name>.]<db_name> ][ LIKE <like_condition> ]
```

可选参数

1. FULL > 语句中加了此参数，返回结果会多三列值，分别为 Table_type（表类型）、Storage_format（存储格式）、Inverted_index_storage_format（倒排索引存储格式）。

2. FROM [<catalog_name>.]<db_name> > FROM 子句中可以指定查询的 catalog 名称以及 database 的名称。

2. LIKE <like_condition> > LIKE 子句中可以按照表名进行模糊查询。

返回值

列名（Column）	类型（DataType）	说明（Notes）
Tables_in_	字符串	<db_name>所在数据库下面所有的表以及视图。
Table_type	字符串	表以及视图类型。
Storage_format	字符串	表以及视图存储格式。
Inverted_index_storage_format	字符串	表以及视图倒排索引存储格式。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table），视图（View）	只能展示具有查询权限的表以及视图。

注意事项

- 语句中不指定 FROM 子句需要 use 到对应的 database 下面执行。

示例

- 查看 DB 下所有表

```
SHOW TABLES;
```

```
+---------------------------------+
| Tables_in_demo                  |
+---------------------------------+
| ads_client_biz_aggr_di_20220419 |
| cmy1                            |
| cmy2                            |
| intern_theme                    |
| left_table                      |
+---------------------------------+
```

- 按照表名进行模糊查询

```
SHOW TABLES LIKE '%cm%'
```

```
+----------------+
| Tables_in_demo |
+----------------+
| cmy1           |
| cmy2           |
+----------------+
```

- 使用 FULL 按照查询 db 下的表以及视图

```
SHOW FULL TABLES
```

```
+----------------+------------+---------------+------------------------------+
| Tables_in_demo | Table_type | Storage_format | Inverted_index_storage_format |
+----------------+------------+---------------+------------------------------+
| test_table     | BASE TABLE | V2             | V1                           |
| test_view      | VIEW       | NONE           | NONE                         |
+----------------+------------+---------------+------------------------------+
```

7.3.8.1.17 SHOW TABLE ID

描述

该语句用于根据 table id 查找对应的 database name, table name。

语法

```
SHOW TABLE <table_id>
```

必选参数

1. <table_id> > 需要查找 database name, table name 表的 <table_id>。

返回值

列名（Column）	类型（DataType）	说明（Notes）
DbName	字符串	数据库名称
TableName	字符串	数据表名称
DbId	字符串	数据库 ID

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据表（table）	目前仅支持 ADMIN 权限执行此操作

示例

1. 根据 table id 查找对应的 database name, table name

```
SHOW TABLE 2261121
```

```
+--------+------------+---------+
| DbName | TableName  | DbId    |
+--------+------------+---------+
| demo   | test_table | 2261034 |
+--------+------------+---------+
```

7.3.8.1.18 SHOW TABLE STATUS

描述

该语句用于展示一个表或者视图的一些信息。

语法

```
SHOW TABLE STATUS [ FROM [ <catalog_name>.]<db_name> ] [ LIKE <like_condition> ]
```

可选参数

1. FROM [<catalog_name>.]<db_name> > FROM 子句中可以指定查询的 catalog 名称以及 database 的名称。

2. LIKE <like_condition> > LIKE 子句中可以按照表名进行模糊查询。

返回值

列名（Column）	类型（DataType）	说明（Notes）
Name	字符串	表名称
Engine	字符串	表的存储引擎
Version	字符串	版本
Row_format	字符串	行格式。对于 MyISAM 引擎，这可能是 Dynamic，Fixed 或 Compressed。动态行的行长度可变，例如 Varchar 或 Blob 类型字段。固定行是指行长度不变，例如 Char 和 Integer 类型字段。
Rows	字符串	表中的行数。对于非事务性表，这个值是精确的，对于事务性引擎，这个值通常是估算的。
Avg_row_length	整型	平均每行包括的字节数
Data_length	整型	整个表的数据量 (单位：字节)
Max_data_length	整型	表可以容纳的最大数据量
Index_length	整型	索引占用磁盘的空间大小
Data_free	整型	对于 MyISAM 引擎，标识已分配，但现在未使用的空间，并且包含了已被删除行的空间。
Auto_increment	整型	下一个 Auto_increment 的值
Create_time	Datetime	表的创建时间
Update_time	Datetime	表的最近更新时间
Check_time	Datetime	使用 check table 或 myisamchk 工具检查表的最近时间
Collation	字符串	表的默认字符集，目前只支持 utf-8
Checksum	字符串	如果启用，则对整个表的内容计算时的校验和
Create_options	字符串	指表创建时的其他所有选项
Comment	字符串	表注释

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	表（Table），视图（View）	目前仅支持 ADMIN 权限执行此操作

注意事项

- 该语句主要用于兼容 MySQL 语法，目前仅显示 Comment 等少量信息。

示例

- 查看当前数据库下所有表的信息

```
SHOW TABLE STATUS
```

```
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
| Name       | Engine | Version | Row_format | Rows | Avg_row_length | Data_length | Max_data
    ↪ _length | Index_length | Data_free | Auto_increment | Create_time         | Update_
    ↪ time         | Check_time | Collation | Checksum | Create_options | Comment |
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
| test_table | Doris  |    NULL | NULL       |    0 |              0 |           0 |
    ↪            NULL |         NULL |      NULL |           NULL | 2025-01-22 11:45:36 |
    ↪ 2025-01-22 11:45:36 | NULL       | utf-8     |     NULL | NULL           |         |
| test_view  | View   |    NULL | NULL       |    0 |              0 |           0 |
    ↪            NULL |         NULL |      NULL |           NULL | 2025-01-22 11:46:32 |
    ↪ NULL                | NULL       | utf-8     |     NULL | NULL           |         |
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
```

- 查看指定数据库下，名称包含 example 的表的信息

```
SHOW TABLE STATUS FROM db LIKE "%test%"
```

```
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
| Name       | Engine | Version | Row_format | Rows | Avg_row_length | Data_length | Max_data
    ↪ _length | Index_length | Data_free | Auto_increment | Create_time         | Update_
    ↪ time         | Check_time | Collation | Checksum | Create_options | Comment |
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
| test_table | Doris  |    NULL | NULL       |    0 |              0 |           0 |
    ↪            NULL |         NULL |      NULL |           NULL | 2025-01-22 11:45:36 |
    ↪ 2025-01-22 11:45:36 | NULL       | utf-8     |     NULL | NULL           |         |
```

```
| test_view  | View    |    NULL | NULL       |    0 |              0 |           0 |
    ↪             NULL |         NULL |      NULL |           NULL | 2025-01-22 11:46:32 |
    ↪ NULL                | NULL       | utf-8     |     NULL | NULL           |         |
+------------+--------+---------+------------+------+----------------+-------------+-----------------+-------
    ↪
```

7.3.8.1.19 SHOW CONVERT LIGHT SCHEMA CHANGE PROCESS

描述

用来查看将非 light schema change 的 olpa 表转换为 light schema change 表的情况。

语法

```
SHOW CONVERT_LIGHT_SCHEMA_CHANGE_PROCESS [ FROM <db_name> ]
```

可选参数

1. FROM <db_name> > FROM 子句中可以指定查询的 database 的名称。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库）	目前仅支持 ADMIN 权限执行此操作

注意事项

- 执行此语句需要开启配置 enable_convert_light_weight_schema_change。

示例

- 查看在 database test 上的转换情况

```
SHOW CONVERT_LIGHT_SCHEMA_CHANGE_PROCESS FROM test;
```

- 查看全局的转换情况

```
SHOW CONVERT_LIGHT_SCHEMA_CHANGE_PROCESS;
```

7.3.8.1.20 SHOW PARTITION

描述

SHOW PARTITION 用于展示指定分区的详细信息。包括所属数据库名字和 ID，所属表名字和 ID 以及分区名字。

语法

```
SHOW PARTITION <partition_id>
```

必选参数

<partition_id>

> 分区的 ID。分区 ID 可以通过 SHOW PARTITIONS 等方式获得。更多信息请参阅“SHOW PARTITIONS”章节

权限控制

执行此 SQL 命令的用户至少具有ADMIN_PRIV权限

示例

查询分区 ID 为 13004 的分区信息：

```
SHOW PARTITION 13004;
```

结果如下：

```
+--------+-----------+---------------+-------+---------+
| DbName | TableName | PartitionName | DbId  | TableId |
+--------+-----------+---------------+-------+---------+
| ods    | sales     | sales         | 13003 | 13005   |
+--------+-----------+---------------+-------+---------+
```

7.3.8.1.21 SHOW PARTITION ID

描述

该语句用于根据 partition id 查找对应的 database name, table name, partition name

语法

```
SHOW PARTITION <partition_id>
```

必选参数

1. <partition_id>

> 分区 id

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV		

示例

1. 根据 partition id 查找对应的 database name, table name, partition name

```
    SHOW PARTITION 10002;
```

7.3.8.1.22 SHOW PARTITIONS

描述

该语句用于展示分区信息。支持 Internal catalog 和 Hive Catalog。

对于 Hive Catalog：

支持返回所有分区，包括多级分区

语法

```
SHOW [ TEMPORARY ] PARTITIONS
    FROM [ <db_name>. ] <table_name>
    [ <where_clause> ]
    [ ORDER BY <order_by_key> ]
    [ LIMIT <limit_rows> ];
```

必选参数

1. <table_name>

需要指定查看分区信息的表名称。

可选参数

1. TEMPORARY

是否需要列出临时分区

2. <db_name>

需要指定查看分区信息的数据库名称。

3. <where_clause>

过滤条件，支持 PartitionId,PartitionName,State,Buckets,ReplicationNum,LastConsistencyCheckTime 等列的过滤。

需要注意的是：1. 目前 where子句等操作符。对字符型的 PartitionName, State 只支持=、!=、like 操作符。对其余的只支持 =、!=、>、<、>=、<= 操作符。2. where子句使用上面的操作符时，列名需要在左侧。3. where子句可以包含AND。

4. <order_by_key>

排序条件，支持 PartitionId,PartitionName,State,Buckets,ReplicationNum,LastConsistencyCheckTime 等列的排序。

5. <limit_rows>

返回的最大行数。

返回值

列名	类型	说明
PartitionId	bigint	分区 ID
PartitionName	varchar	分区名称
VisibleVersion	int	该分区下，最大的 tablet 的 VisibleVersion
VisibleVersionTime	datetime	该分区下，最近一次的 VisibleVersionde 时间
State	varchar	分区状态
PartitionKey	datetime	分区键
Range	datetime	该分区的分区区间
DistributionKey	varchar	该分区的分布键
Buckets	int	该分区的分桶数
ReplicationNum	int	该分区的副本书
StorageMedium	varchar	该分区的存储介质
CooldownTime	datetime	该分区的降冷时间，如果没有冷热分离，该字段的值为 [9999-12-31 23:59:59]，即一直为
RemoteStoragePolicy	varchar	该分区的远端存储策略
LastConsistencyCheckTime	datetime	该分区的上一次的分区一致性检查的时间
DataSize	int	该分区下的数据大小
IsInMemory	boolean	是否为内存分区，默认为 false
ReplicaAllocation	varchar	该分区的副本分布策略
IsMutable	boolean	该分区是否为可变分区，默认为 true
SyncWithBaseTables	boolean	该分区是否和基表的数据同步保持一致
UnsyncTables	varchar	该分区是否是非同步表的分区

权限控制

需要具备要查看的表的 SHOW 权限。

示例

1. 展示指定 db 下指定表的所有非临时分区信息

```
SHOW PARTITIONS FROM t_agg;
```

```
+-------------+---------------+----------------+---------------------+--------+-------------+-------+-----------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range | DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime
    ↪ | RemoteStoragePolicy | LastConsistencyCheckTime | DataSize  | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
```

```
+-------------+--------------+---------------+--------------------+--------+-------------+-------+----------
    ↪
| 170307      | t_agg        | 4             | 2024-11-05 16:13:40 | NORMAL |             |
    ↪       | k1             | 1       | 1              | HDD           | 9999-12-31 23:59:59
    ↪ |                     | NULL                    | 806.000 B | false      | tag.location.
    ↪ default: 1 | true      | true               | NULL         |
+-------------+--------------+---------------+--------------------+--------+-------------+-------+----------
    ↪
```

2. 展示指定 db 下指定表的所有临时分区信息

```
SHOW TEMPORARY PARTITIONS FROM t_temp;
```

```
+-------------+--------------+---------------+--------------------+--------+-------------+-----------------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey |
    ↪ Range
    ↪
    ↪ | DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+-------------+--------------+---------------+--------------------+--------+-------------+-----------------
    ↪
| 828863      | tp2020        | 1             | 2025-01-22 16:19:50 | NORMAL | create_time  | [
    ↪ types: [DATETIMEV2]; keys: [2020-01-01 00:00:00]; ..types: [DATETIMEV2]; keys:
    ↪ [2021-01-01 00:00:00]; ) | reference_no    | 1       | 1              | SSD           |
    ↪ 9999-12-31 23:59:59 |                     | NULL                    | 0.000    | false
    ↪      | tag.location.default: 1 | true      | true               | NULL         |
+-------------+--------------+---------------+--------------------+--------+-------------+-----------------
    ↪
```

3. 展示指定 db 下指定表的指定非临时分区的信息，并对结果进行过滤

```
SHOW PARTITIONS FROM t_agg WHERE PartitionName = "p2024";
```

```
+-------------+--------------+---------------+--------------------+--------+----------------+---------------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey    |
    ↪  Range
    ↪
    ↪ | DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize  | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
```

```
+-------------+--------------+---------------+--------------------+--------+----------------+-----------------
    ↪
| 169851      | p2024        | 2             | 2024-11-05 14:14:29 | NORMAL | idp_create_time |
    ↪  [types: [DATETIMEV2]; keys: [2024-01-01 00:00:00]; ..types: [DATETIMEV2]; keys:
    ↪ [2025-01-01 00:00:00]; ) | idp_es_id       | 3       | 1              | HDD           |
    ↪ 9999-12-31 23:59:59 |                     | NULL                    | 27.396 KB | false
    ↪      | tag.location.default: 1 | true      | true               | NULL         |
+-------------+--------------+---------------+--------------------+--------+----------------+-----------------
    ↪
```

4. 展示指定 db 下指定表的最新非临时分区的信息

```
SHOW PARTITIONS FROM t_agg ORDER BY PartitionId DESC LIMIT 1;
```

```
+-------------+--------------+---------------+--------------------+--------+----------------+-----------------
    ↪
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime  | State  | PartitionKey    |
    ↪  Range
    ↪
    ↪ | DistributionKey | Buckets | ReplicationNum | StorageMedium | CooldownTime        |
    ↪ RemoteStoragePolicy | LastConsistencyCheckTime | DataSize | IsInMemory |
    ↪ ReplicaAllocation       | IsMutable | SyncWithBaseTables | UnsyncTables |
+-------------+--------------+---------------+--------------------+--------+----------------+-----------------
    ↪
| 169866      | p2025        | 1             | 2024-11-05 14:13:56 | NORMAL | idp_create_time |
    ↪  [types: [DATETIMEV2]; keys: [2025-01-01 00:00:00]; ..types: [DATETIMEV2]; keys:
    ↪ [2026-01-01 00:00:00]; ) | idp_es_id       | 3       | 1              | HDD           |
    ↪ 9999-12-31 23:59:59 |                     | NULL                    | 0.000    | false
    ↪      | tag.location.default: 1 | true      | true               | NULL         |
+-------------+--------------+---------------+--------------------+--------+----------------+-----------------
    ↪
```

7.3.8.1.23 SHOW DYNAMIC PARTITION TABLES

描述

该语句用于展示当前 db 下所有的动态分区表状态

语法：

```
SHOW DYNAMIC PARTITION TABLES [ FROM <db_name> ];
```

可选参数

1. <db_name>

指定展示动态分区表状态的 DB 名称，如果不指定，则默认展示当前 DB 下的所有动态分区表状态。

返回值

列名	类型	说明
TableName	varchar	当前 DB 或指定 DB 的表名称
Enable	varchar	是否开启了表的动态分区属性
TimeUnit	varchar	动态分区表的分区粒度，有 HOUR，DAY,WEEK,MONTH,YEAR
Start	varchar	动态分区的起始偏移，为负数。默认值为 -2147483648，即不删除历史分区。根据 time_unit 属性的不同，以当天（星期/月）为基准，分区范围在此偏移之前的分区将会被删除。
End	varchar	动态分区的结束偏移，为正数。根据 time_unit 属性的不同，以当天（星期/月）为基准，提前创建对应范围的分区。
Prefix	varchar	动态创建的分区名前缀。
Buckets	varchar	动态创建的分区所对应的分桶数量。
ReplicationNum	varchar	动态创建的分区所对应的副本数量，如果不填写，则默认为该表创建时指定的副本数量。
ReplicaAllocation	varchar	动态创建的分区所对应的副本分布策略，如果不填写，则默认为该表创建时指定的副本分布策略。
StartOf	varchar	动态分区每个分区粒度的起始点。当 time_unit 为 WEEK 时，该字段表示每周的起始点，取值为 MONDAY 到 SUNDAY；当 time_unit 为 MONTH 时，表示每月的起始日期，取值为 1rd 至 28rd；当 time_unit 为 MONTH 时，该值默认为 NULL。
LastUpdateTime	varchar	动态分区的上一次更新时间，默认为 NULL。
LastSchedulerTime	datetime	动态分区的上一次调度时间。
State	varchar	动态分区的创建状态。
LastCreatePartitionMsg	varchar	最后一次执行动态添加分区调度的错误信息。
LastDropPartitionMsg	varchar	最后一次执行动态删除分区调度的错误信息。
ReservedHistoryPeriods	varchar	动态分区保留的历史分区的分区区间，它表示在动态分区表中，哪些历史分区应该被保留，而不是被自动删除。

权限控制

1. 如果没有指定参数 db_name，展示的是当前 DB 下的所有动态分区表状态，默认用户具备当前 DB 的SHOW_PRIV权限。
2. 如果指定了参数 db_name，展示的是指定 DB 下的所有动态分区表状态，用户需要具备该 DB 的 SHOW_PRIV 权限。

示例

1. 查看当前数据库下的所有动态分区表状态：

```
SHOW DYNAMIC PARTITION TABLES;
```

```
+-----------+--------+----------+-------------+------+--------+---------+-----------+----------------+-----------
    ↪
| TableName | Enable | TimeUnit | Start       | End  | Prefix | Buckets | StartOf   |
    ↪ LastUpdateTime | LastSchedulerTime   | State  | LastCreatePartitionMsg |
    ↪ LastDropPartitionMsg | ReservedHistoryPeriods  |
```

```
+-----------+--------+----------+-------------+------+--------+---------+-----------+---------------+-----------
    ↪
| d3        | true   | WEEK     | -3          | 3    | p      | 1       | MONDAY    | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | [2021-12-01,2021-12-31] |
| d5        | true   | DAY      | -7          | 3    | p      | 32      | N/A       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
| d4        | true   | WEEK     | -3          | 3    | p      | 1       | WEDNESDAY | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
| d6        | true   | MONTH    | -2147483648 | 2    | p      | 8       | 3rd       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
| d2        | true   | DAY      | -3          | 3    | p      | 32      | N/A       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
| d7        | true   | MONTH    | -2147483648 | 5    | p      | 8       | 24th      | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
+-----------+--------+----------+-------------+------+--------+---------+-----------+---------------+-----------
    ↪
```

2. 查看指定数据库下的所有动态分区表状态：

```
SHOW DYNAMIC PARTITION TABLES FROM test;
```

```
+-----------+--------+----------+-------------+------+--------+---------+-----------+---------------+-----------
    ↪
| TableName | Enable | TimeUnit | Start       | End  | Prefix | Buckets | StartOf   |
    ↪ LastUpdateTime | LastSchedulerTime   | State  | LastCreatePartitionMsg |
    ↪ LastDropPartitionMsg | ReservedHistoryPeriods  |
+-----------+--------+----------+-------------+------+--------+---------+-----------+---------------+-----------
    ↪
| test1     | true   | WEEK     | -30          | 3    | p      | 8       | MONDAY    | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | [2021-12-01,2021-12-31] |
| test2     | true   | DAY      | -7          | 3    | p      | 32      | N/A       | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
| test3     | true   | WEEK     | -3          | 3    | p      | 1       | WEDNESDAY | N/A
    ↪            | 2020-05-25 14:29:24 | NORMAL | N/A                   | N/A
    ↪                 | NULL                    |
+-----------+--------+----------+-------------+------+--------+---------+-----------+---------------+-----------
    ↪
```

7.3.8.1.24 SHOW COLUMNS

描述

该语句用于指定表的列信息

语法

```
SHOW [ FULL ] COLUMNS FROM <tbl>;
```

必选参数

1. <tbl>

需要指定查看列信息的表名称。

可选参数

1. FULL

如果指定了 FULL 关键字，会返回列的详细信息，包括列的聚合类型、权限、注释等。

返回值

列名	类型	说明
Field	varchar	列名
Type	varchar	列类型
Collation	varchar	列的排序规则
Null	varchar	是否允许为 NULL
Key	varchar	列的主键
Default	varchar	默认值
Extra	varchar	额外信息
Privileges	varchar	列的权限
Comment	varchar	列的注释

权限控制

需要具备要查看的表的 SHOW 权限。

示例

1. 查看指定表详细的列信息

```
SHOW FULL COLUMNS FROM t_agg;
```

```
+-------+-----------------+-----------+------+------+---------+---------+------------+---------+
| Field | Type            | Collation | Null | Key  | Default | Extra   | Privileges | Comment |
+-------+-----------------+-----------+------+------+---------+---------+------------+---------+
| k1    | tinyint         |           | YES  | YES  | NULL    |         |            |         |
| k2    | decimalv3(10,2) |           | YES  | YES  | 10.5    |         |            |         |
| v1    | char(10)        |           | YES  | NO   | NULL    | REPLACE |            |         |
```

```
| v2    | int              |           | YES  | NO   | NULL    | SUM     |            |         |
+-------+------------------+-----------+------+------+---------+---------+------------+---------+
```

2. 查看指定表的普通列信息

```
SHOW COLUMNS FROM t_agg;
```

```
+-------+-----------------+------+------+---------+---------+
| Field | Type            | Null | Key  | Default | Extra   |
+-------+-----------------+------+------+---------+---------+
| k1    | tinyint         | YES  | YES  | NULL    |         |
| k2    | decimalv3(10,2) | YES  | YES  | 10.5    |         |
| v1    | char(10)        | YES  | NO   | NULL    | REPLACE |
| v2    | int             | YES  | NO   | NULL    | SUM     |
+-------+-----------------+------+------+---------+---------+
```

7.3.8.1.25 ALTER COLOCATE GROUP

描述

该语句用于修改 Colocation Group 的属性。

语法

```
ALTER COLOCATE GROUP  [ <database>. ] <group_name>
SET (
    <property_list>
    );
```

必选参数

1. <group_name>

指定要修改的 colocate group 的名称。

2.<property_list>

property_list 是 colocation group 属性，目前只支持修改 replication_num 和 replication_allocation。修改 colocation group 的这两个属性修改之后，同时把该 group 的表的属性 default.replication_allocation 、属性 dynamic.replication_allocation、以及已有分区的 replication_allocation改成跟它一样。

可选参数

1. <database>

指定要修改的 colocate group 的所属数据库。

注意：1. 如果 colocate group 是全局的，即它的名称是以 __global__ 开头的，那它不属于任何一个 Database；

返回值

无。

权限控制

需要 ADMIN 的权限。

示例

1. 修改一个全局 group 的副本数，建表时设置 "colocate_with" = "__global__foo"

```
ALTER COLOCATE GROUP __global__foo
SET (
    "replication_num"="1"
    );
```

2. 修改一个非全局 group 的副本数，建表时设置 “colocate_with” = “bar”，且表属于 Database example_db

```
ALTER COLOCATE GROUP example_db.bar
SET (
"replication_num"="1"
);
```

7.3.8.2 索引

7.3.8.2.1 CREATE INDEX

描述

为表创建新的索引，必须指定表名和索引名，可选指定索引类型、属性、注释。

语法

```
CREATE INDEX [IF NOT EXISTS] <index_name>
             ON <table_name> (<column_name> [, ...])
             [USING {INVERTED | NGRAM_BF}]
             [PROPERTIES ("<key>" = "<value>"[ , ...])]
             [COMMENT '<index_comment>']
```

必选参数

1. <index_name>

> 指定索引的标识符（即名称），在其所在的表（Table）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <table_name>

> 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

3. <column_name> [, ...]

> 指定在哪些列上创建索引（目前仅支持一个），列在其所在的（Table）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1. USING {INVERTED | NGRAM_BF}

> 指定索引类型，目前支持两种：INVERTED 倒排索引，NGRAM_BF ngram bloomfilter 索引。

2. PROPERTIES ("<key>" = "<value>"[, ...])

> 指定索引的参数，使用通用的 PROPERTIES 格式，每个索引支持的参数及语义，请参考具体类型的索引文档。

3. COMMENT '<index_comment>'

> 指定索引的注释，便于维护。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	表	CREATE INDEX 属于表 ALTER 操作

注意事项

- INVERTED 倒排索引创建后对新写入的数据立即生效，历史数据的索引需要进行 BUILD INDEX 操作。
- NGRAM_BF NGram BloomFilter 索引创建后会在后台对所用数据进行 schema change 以完成索引构建，进度可以通过 SHOW ALTER TABLE COLUMN 查看进度

示例

- 在 table1 上创建倒排索引 index1

```
CREATE INDEX index1 ON table1 USING INVERTED;
```

- 在 table1 上创建 NGram BloomFilter 索引 index2

```
CREATE INDEX index2 ON table1 USING NGRAM_BF PROPERTIES("gram_size"="3", "bf_size"="1024");
```

7.3.8.2.2 DROP INDEX

描述

该语句用于从一个表中删除指定名称的索引，目前仅支持 bitmap, inverted index 索引。

语法

```
DROP INDEX [ IF EXISTS ] <index_name> ON [ <db_name> . ] <table_name>;
```

必选参数

1. <index_name>：索引名称。

2. <table_name>：索引归属的表名。

可选参数

1. <db_name>：库名，选填，不填默认当前库。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	表（Table）	DROP INDEX 属于表 ALTER 操作

示例

- 删除索引

```
DROP INDEX IF NOT EXISTS index_name ON table1 ;
```

7.3.8.2.3 SHOW INDEX

描述

该语句用于展示一个表中索引的相关信息，目前只支持 bitmap 索引

语法

```
SHOW INDEX [ ES ] FROM [ <db_name>. ] <table_name> [ FROM <db_name> ];
```

变种语法

```
SHOW KEY[ S ] FROM [ <db_name>. ] <table_name> [ FROM <db_name> ];
```

必选参数

1. <table_name>：索引归属的表名。

可选参数

1. <db_name>：库名，选填，不填默认当前库。

返回值

列名	类型	说明
Table	string	索引所在的表的名称。
Non_unique	int	指示该索引是否为唯一索引：- 0：唯一索引 - 1：非唯一索引
Key_name	string	索引的名称。
Seq_in_index	int	索引中列的顺序。该列显示的是列在索引中的位置，多个列组成复合索引时使用。
Column_name	string	被索引的列名。
Collation	string	索引列的排序方式：- A：升序 - D：降序。
Cardinality	int	索引中独立值的数量。该值用于估计查询效率，值越大，表示索引的选择性越高，查询效率越好。
Sub_part	int	索引所使用的前缀长度。如果索引列为字符串类型，Sub_part 表示索引的前几个字符长度。
Packed	string	索引是否压缩。
Null	string	是否允许 NULL 值：- YES：允许 NULL 值 - NO：不允许 NULL 值
Index_type	string	索引的类型：- BTREE：B+ 树索引（MySQL 默认类型）- HASH：哈希索引 - RTREE：R 树索引 - INVERTED：倒排索引（如全文索引）
Comment	string	索引的注释或描述，通常为自定义的备注信息。
Properties	string	索引的附加属性。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SHOW_PRIV	库（Database）	

示例

- 展示指定 table_name 的下索引

```
    SHOW INDEX FROM example_db.table_name;
```

7.3.8.2.4 BUILD INDEX

描述

为整个表或者表的分区构建索引，必须指定表名和索引名，可选指定分区列表。

语法

```
BUILD INDEX <index_name> ON <table_name> [partition_list]
```

其中：

```
partition_list
  : PARTITION (<partition_name1>[ , parition_name2 ][ ... ])
```

必选参数

1. <index_name>

> 指定索引的标识符（即名称），在其所在的表（Table）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <table_name>

指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。

标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。

标识符不能使用保留关键字。

有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1.<partition_list>

指定分区的标识符（即名称）列表，以逗号分割，在其所在的表（Table）中必须唯一。

标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。

标识符不能使用保留关键字。

有关更多详细信息，请参阅标识符要求和保留关键字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	表（Table）	BUILD INDEX 属于表 ALTER 操作

注意事项

- 目前只对倒排索引生效，其他索引如 bloomfilter index 不生效。
- 目前只对存算一体模式生效，存算分离模式不生效。
- BUILD INDEX 的进度可以通过 SHOW BUILD INDEX 查看

示例

- 在 table1 整个表上构建索引 index1

```
BUILD INDEX index1 ON table1
```

- 在 table1 的分区 p1 和 p2 上构建索引 index1

```
BUILD INDEX index1 ON table1 PARTITION(p1, p2)
```

7.3.8.2.5 CANCEL BUILD INDEX

描述

取消索引构建的后台任务。

语法

```
CANCEL BUILD INDEX ON <table_name> [ job_list ]
```

其中：

```
job_list
  : (<job_id1>[ , job_id2 ][ ... ])
```

必选参数

1. <table_name>

> 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1. <job_list>

> 指定索引构建任务的标识符列表，以括号包围的逗号分割。
>
> 标识符必须是数字，可以通过 SHOW BUILD INDEX 查看。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	表（Table）	CANCEL BUILD INDEX 属于表 ALTER 操作

注意事项

- 目前只对倒排索引生效，其他索引如 bloomfilter index 不生效。

- 目前只对存算一体模式生效，存算分离模式不生效。
- BUILD INDEX 的进度和索引构建任务可以通过 SHOW BUILD INDEX 查看

示例

- 取消表 table1 上的所有索引构建任务

```
    CANCEL BUILD INDEX ON TABLE table1
```

- 取消表 table1 上的索引构建任务 jobid1 和 jobid2

```
    CANCEL BUILD INDEX ON TABLE table1(jobid1, jobid2)
```

7.3.8.2.6 SHOW BUILD INDEX

描述

查看索引构建任务的状态。

语法

```
SHOW BUILD INDEX [ (FROM | IN) <database_name>
[ where_clause ] [ sort_clause ] [ limit_clause ] ]
```

其中：

```
where_clause
  : WHERE <output_column_name = value>
```

其中：

```
sort_clause
  :
   ORDER BY <output_column_name>
```

其中：

```
limit_clause
  :
   LIMIT <n>
```

可选参数（Optional Parameters）

1. <database_name>

指定数据库的标识符（即名称），在其所在的集群（Cluster）中必须唯一。
标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
标识符不能使用保留关键字。
有关更多详细信息，请参阅标识符要求和保留关键字。

2. <WHERE output_column_name = value>

指定输出过滤条件，output_column_name 必须在输出的字段列表中。

3. <ORDER BY output_column_name>

指定输出排序列，output_column_name 必须在输出的字段列表中。

4. LIMIT <n>

指定输出行数限制，n 必须是数字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SHOW_PRIV	数据库（Database）	

注意事项

- 目前只对倒排索引生效，其他索引如 bloomfilter index 不生效。
- 目前只对存算一体模式生效，存算分离模式不生效。

示例（Examples）

- 查看所有索引构建任务

```
SHOW BUILD INDEX
```

- 查看数据库 database1 的索引构建任务

```
SHOW BUILD INDEX FROM database1
```

- 查看数据库 table1 的索引构建任务

```
SHOW BUILD INDEX WHERE TableName = 'table1'
```

- 查看数据库 table1 的索引构建任务，并按照输出的 JobId 排序取前 10 行

```
SHOW BUILD INDEX WHERE TableName = 'table1' ORDER BY JobId LIMIT 10
```

7.3.8.3 视图

7.3.8.3.1 CREATE VIEW

描述

该语句用于通过指定的查询语句创建一个逻辑视图。

语法

```
CREATE VIEW [IF NOT EXISTS] [<db_name>.]<view_name>
   [(<column_definition>)]
AS <query_stmt>
```

其中：

```
column_definition:
    <column_name> [COMMENT '<comment>'] [,...]
```

必选参数

1. <view_name> > 视图的标识符（即名称）；在创建视图的数据库中必须唯一。
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My View`）。
> 标识符不能使用保留关键字。
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <query_stmt> > 定义视图的 SELECT 查询语句。

可选参数

1. <db_name> > 视图所在的数据库名称。如果未指定，则默认为当前数据库。

2. <column_definition> > 视图的列定义。
> 其中：
> 1. <column_name>

> 列名。

> 2. <comment>

> 列的注释。

权限控制

权限	对象	说明
CREATE_PRIV	数据库	
SELECT_PRIV	表、视图	需要拥有被查询的表、视图、物化视图的 SELECT_PRIV 权限。

注意事项

- 视图为逻辑视图，没有物理存储。所有在视图上的查询相当于在视图对应的子查询上进行。
- 视图的创建和删除不会影响底层表的数据。

示例

1. 在 example_db 上创建视图 example_view

```
CREATE VIEW example_db.example_view (k1, k2, k3, v1)
AS
SELECT c1 as k1, k2, k3, SUM(v1) FROM example_table
WHERE k1 = 20160112 GROUP BY k1,k2,k3;
```

2. 创建一个包含列定义的视图

```
CREATE VIEW example_db.example_view
(
    k1 COMMENT "first key",
    k2 COMMENT "second key",
    k3 COMMENT "third key",
    v1 COMMENT "first value"
)
COMMENT "my first view"
AS
SELECT c1 as k1, k2, k3, SUM(v1) FROM example_table
WHERE k1 = 20160112 GROUP BY k1,k2,k3;
```

7.3.8.3.2 ALTER VIEW

描述

该语句用于修改一个逻辑视图的定义。

语法

```
ALTER VIEW [<db_name>.]<view_name>
 [(<column_definition>)]
AS <query_stmt>
```

其中：

```
column_definition:
    <column_name> [COMMENT '<comment>'] [,...]
```

必选参数

1. <view_name> > 要修改的视图的标识符（即名称）。

2. <query_stmt> > 定义视图的 SELECT 查询语句。

可选参数

1. <db_name> > 视图所在的数据库名称。如果未指定，则默认为当前数据库。

2. <column_definition> > 视图的列定义。
> 其中：
> 1. <column_name>
> 列名。
> 2. <comment>
> 列的注释。

权限控制

权限	对象	说明
ALTER_PRIV	视图	需要所修改视图的 SELECT_PRIV 权限。
SELECT_PRIV	表、视图	需要拥有被查询的表、视图、物化视图的 SELECT_PRIV 权限。

示例

1、修改 example_db 上的视图 example_view

```
ALTER VIEW example_db.example_view
(
  c1 COMMENT "column 1",
  c2 COMMENT "column 2",
  c3 COMMENT "column 3"
)
AS SELECT k1, k2, SUM(v1) FROM example_table
GROUP BY k1, k2
```

7.3.8.3.3 DROP VIEW

描述

在当前或指定的数据库中删除一个视图。

语法

```
DROP VIEW [ IF EXISTS ] <name>
```

必选参数

<name> : 要删除的视图名称。

可选参数

[IF EXISTS]

如果指定此参数，当视图不存在时不会抛出错误，而是直接跳过删除操作。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
DROP_PRIV	表（Table）	

注意事项

已删除的视图无法恢复，必须重新创建。

示例

```
CREATE VIEW vtest AS SELECT 1, 'test';
DROP VIEW IF EXISTS vtest;
```

7.3.8.3.4 SHOW CREATE VIEW

描述

显示指定视图创建时的 CREATE VIEW 语句。

语法

```
SHOW CREATE VIEW <name>
```

必选参数

<name>：要查看的视图名称。

返回结果

- View: 查询的视图名称。
- Create View: 数据库中持久化的 SQL 语句。
- character_set_client: 表示创建视图时会话中 character_set_client 系统变量的值。
- collation_connection: 表示创建视图时会话中 collation_connection 系统变量的值。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SHOW_VIEW_PRIV	表（Table）	

视图信息还可以通过 INFORMATION_SCHEMA.VIEWS 表查询。

示例

```
CREATE VIEW vtest AS SELECT 1, 'test';
SHOW CREATE VIEW vtest;
```

查询结果：

```
+--
    ↪ -----+-----------------------------------------+---------------------+----------------------+
    ↪
| View  | Create View                               | character_set_client | collation_connection
    ↪ |
+--
    ↪ -----+-----------------------------------------+---------------------+----------------------+
    ↪
| vtest | CREATE VIEW `vtest` AS SELECT 1, 'test'; | utf8mb4              | utf8mb4_0900_bin
    ↪ |
+--
    ↪ -----+-----------------------------------------+---------------------+----------------------+
    ↪
```

7.3.8.3.5 SHOW VIEW

描述

该语句用于展示基于给定表建立的所有视图

语法：

```
SHOW VIEW { FROM | IN } table [ FROM db ]
```

示例

1. 展示基于表 testTbl 建立的所有视图 view

```
  SHOW VIEW FROM testTbl;
```

关键词

SHOW, VIEW

7.3.8.4 同步物化视图

7.3.8.4.1 CREATE SYNC MATERIALIZED VIEW

描述

创建同步物化视图语句

语法

```
CREATE MATERIALIZED VIEW <materialized_view_name> AS <query>
```

其中

```
query
    :
    SELECT <select_expr> select_expr[, select_expr ...]
    FROM <base_table>
    WHERE condition
    GROUP BY <column_name>[, <column_name> ...]
    ORDER BY <column_name>[, <column_name> ...]
```

必选参数

1. <materialized_view_name>

> 指定表的标识符（即名称）；同步物化视图基于表创建，所以名称需要在相同表中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <query>

> 用于构建物化视图的查询语句，查询语句的结果既物化视图的数据。目前支持的 query 格式为：
>
> 语法和查询语句语法一致。
>
> - `select_expr`：物化视图的 schema 中所有的列。
> - 至少包含一个单列。
> - `base_table`：物化视图的原始表名，必填项。
> - 必须是单表，且非子查询
> - `where`：物化视图的过滤条件，选填项。

- 不填则数据不进行过滤。
- `group by`：物化视图的分组列，选填项。
- 不填则数据不进行分组。
- `order by`：物化视图的排序列，选填项。
- 排序列的声明顺序必须和 select_expr 中列声明顺序一致。
- 如果不声明 order by，则根据规则自动补充排序列。如果物化视图是聚合类型，则所有的分组列自动补充为排序列。如果物化视图是非聚合类型，则前 36 个字节自动补充为排序列。
- 如果自动补充的排序个数小于 3 个，则前三个作为排序列。如果 query 中包含分组列的话，则排序列必须和分组列一致。

权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	表（Table）	需要拥有当前物化视图基表的 ALTER_PRIV 权限

注意事项

- 同步物化视图只支持针对单个表的 SELECT 语句，支持 WHERE、GROUP BY、ORDER BY 等子句，但不支持 JOIN、HAVING、LIMIT 子句和 LATERAL VIEW。
- SELECT 列表中，不能包含自增列，不能包含常量，不能有重复表达式，也不支持窗口函数。
- 如果 SELECT 列表包含聚合函数，则聚合函数必须是根表达式（不支持 `sum(a)+ 1`，支持 `sum(a + 1)`），且聚合函数之后不能有其他非聚合函数表达式（例如，`SELECT x, sum(a)` 可以，而 `SELECT sum(a), x` 不行）。
- 单表上过多的物化视图会影响导入的效率：导入数据时，物化视图和 Base 表的数据是同步更新的。如果一张表的物化视图表过多，可能会导致导入速度变慢，这就像单次导入需要同时导入多张表的数据一样。
- 物化视图针对 Unique Key 数据模型时，只能改变列的顺序，不能起到聚合的作用。因此，在 Unique Key 模型上不能通过创建物化视图的方式对数据进行粗粒度的聚合操作。
- 物化视图针对 Unique Key 和 Aggregate Key 数据模型时，如果指定了 where 条件，那 where 条件只能使用 Key 列，而不能使用 Value 列。

示例

```
desc lineitem;
```

```
+-----------------+---------------+------+-------+---------+-------+
| Field           | Type          | Null | Key   | Default | Extra |
+-----------------+---------------+------+-------+---------+-------+
| l_orderkey      | int           | No   | true  | NULL    |       |
| l_partkey       | int           | No   | true  | NULL    |       |
| l_suppkey       | int           | No   | true  | NULL    |       |
| l_linenumber    | int           | No   | true  | NULL    |       |
```

```
| l_quantity      | decimal(15,2) | No   | false | NULL    | NONE   |
| l_extendedprice | decimal(15,2) | No   | false | NULL    | NONE   |
| l_discount      | decimal(15,2) | No   | false | NULL    | NONE   |
| l_tax           | decimal(15,2) | No   | false | NULL    | NONE   |
| l_returnflag    | char(1)       | No   | false | NULL    | NONE   |
| l_linestatus    | char(1)       | No   | false | NULL    | NONE   |
| l_shipdate      | date          | No   | false | NULL    | NONE   |
| l_commitdate    | date          | No   | false | NULL    | NONE   |
| l_receiptdate   | date          | No   | false | NULL    | NONE   |
| l_shipinstruct  | char(25)      | No   | false | NULL    | NONE   |
| l_shipmode      | char(10)      | No   | false | NULL    | NONE   |
| l_comment       | varchar(44)   | No   | false | NULL    | NONE   |
+-----------------+---------------+------+-------+---------+--------+
```

```
CREATE MATERIALIZED VIEW sync_agg_mv AS
SELECT
  l_shipdate,
  l_partkey,
  count(*),
  sum(l_discount)
FROM
  lineitem
GROUP BY
  l_shipdate,
  l_partkey;
```

7.3.8.4.2 DROP MATERIALIZED VIEW

描述

删除同步物化视图。

语法

```
DROP MATERIALIZED VIEW
[ IF EXISTS ] <materialized_view_name>
ON <table_name>
```

必选参数

1.

> 目标要删除的物化视图名称

2. <table_name>

物化视图所属的表

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	表	需要拥有当前被删除的物化视图所属表的 ALTER_PRIV 权限

示例

删除 lineitem 表上的 sync_agg_mv 同步物化视图

```
DROP MATERIALIZED VIEW sync_agg_mv on lineitem;
```

7.3.8.4.3 SHOW ALTER TABLE MATERIALIZED VIEW

描述

查看同步物化视图构建任务状态。

由于创建同步物化视图是一个异步操作，用户在提交创建物化视图任务后，需要异步地通过命令查看同步物化视图构建状态。

语法

```
SHOW ALTER TABLE MATERIALIZED VIEW FROM <database>
```

必选参数

1. <database>

同步物化视图的基表所属数据库

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ALTER_PRIV	表	需要拥有当前物化视图所属表的 ALTER_PRIV 权限

示例

```
SHOW ALTER TABLE MATERIALIZED VIEW FROM doc_db;
```

7.3.8.4.4 SHOW CREATE SYNC MATERIALIZED VIEW

描述

查看同步物化视图创建语句。

语法

```
SHOW CREATE MATERIALIZED VIEW <materialized_view_name> ON <table_name>
```

必选参数

1. <materialized_view_new_name>

> 物化视图名称

2. <table_name>

> 物化视图所属的表

返回值

列名	说明
TableName	表名
ViewName	物化视图名
CreateStmt	物化视图创建语句

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV/LOAD_PRIV/ALTER_PRIV/CREATE_PRIV/DROP_PRIV	表	需要拥有当前物化视图所属表的权限

示例（Examples）

1. 查看同步物化视图创建语句

```
    SHOW CREATE MATERIALIZED VIEW sync_agg_mv on lineitem;
```

7.3.8.5 异步物化视图

7.3.8.5.1 ALTER ASYNC MATERIALIZED VIEW

描述

该语句用于修改异步物化视图。

语法

```
ALTER MATERIALIZED VIEW mvName=multipartIdentifier ((RENAME newName=identifier)
       | (REFRESH (refreshMethod | refreshTrigger | refreshMethod refreshTrigger))
       | REPLACE WITH MATERIALIZED VIEW newName=identifier propertyClause?
       | (SET  LEFT_PAREN fileProperties=propertyItemList RIGHT_PAREN))
```

说明

RENAME

用来更改物化视图的名字

例如：将 mv1 **的名字改为** mv2

```
ALTER MATERIALIZED VIEW mv1 rename mv2;
```

refreshMethod

同创建异步物化视图

refreshTrigger

同创建异步物化视图

SET

修改物化视图特有的 property

例如修改 mv1 **的** grace_period **为** 3000ms

```
ALTER MATERIALIZED VIEW mv1 set("grace_period"="3000");
```

REPLACE

```
ALTER MATERIALIZED VIEW [db.]mv1 REPLACE WITH MATERIALIZED VIEW mv2
[PROPERTIES('swap' = 'true')];
```

两个物化视图进行原子的替换操作

swap **默认为** TRUE

- **如果** swap **参数为** TRUE**，相当于把物化视图** mv1 **重命名为** mv2 **,同时把** mv2 **重命名为** mv1

- 如果 swap 参数为 FALSE，相当于把 mv2 重命名为 mv1，原有的 mv1 被删除

例如想把 mv1 和 mv2 的名字互换

```
ALTER MATERIALIZED VIEW db1.mv1 REPLACE WITH MATERIALIZED VIEW mv2;
```

例如想把 mv2 重命名为 mv1，并删除原先的 mv1

```
ALTER MATERIALIZED VIEW db1.mv1 REPLACE WITH MATERIALIZED VIEW mv2
PROPERTIES('swap' = 'false');
```

关键词

```
ALTER, ASYNC, MATERIALIZED, VIEW
```

7.3.8.5.2 CANCEL MATERIALIZED VIEW TASK

描述

该语句用于取消物化视图的 task

语法

```
CANCEL MATERIALIZED VIEW TASK <task_id> ON <mv_name>
```

必选参数

1. <task_id>

> 指定物化视图创建 job 的 task id。

2. <mv_name>

> 指定物化视图的名字。
>
> 物化视图的名字必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个名字串用反引号括起来（例如`My Object`）。
>
> 物化视图的名字不能使用保留关键字。
>
> 有关更多详细信息，请参阅保留关键字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	物化视图	CANCEL 属于物化视图的 ALTER 操作

示例

- 取消物化视图 mv1 的 id 为 1 的 task

```
    CANCEL MATERIALIZED VIEW TASK 1 on mv1;
```

7.3.8.5.3 CREATE ASYNC MATERIALIZED VIEW

描述

创建异步物化视图语句，列名和列类型是通过物化视图 SQL 语句推导出来的，可以自定义列名，不可以定义列类型。

语法

```
CREATE MATERIALIZED VIEW
[ IF NOT EXISTS ] <materialized_view_name>
    [ (<columns_definition>) ]
    [ BUILD <build_mode> ]
    [ REFRESH <refresh_method> [<refresh_trigger>]]
    [ [DUPLICATE] KEY (<key_cols>) ]
    [ COMMENT '<table_comment>' ]
    [ PARTITION BY (
        { <partition_col>
            | DATE_TRUNC(<partition_col>, <partition_unit>) }
        )]
    [ DISTRIBUTED BY { HASH (<distribute_cols>) | RANDOM }
        [ BUCKETS { <bucket_count> | AUTO } ]
    ]
    [ PROPERTIES (
          -- Table property
          <table_property>
          -- Additional table properties
          [ , ... ])
    ]
    AS <query>
```

其中：

```
columns_definition
: -- Column definition
    <col_name>
      [ COMMENT '<col_comment>' ]
```

```
refresh_trigger
  : ON MANUAL
  | ON SCHEDULE EVERY <int_value> <refresh_unit> [ STARTS '<start_time>']
  | ON COMMIT
```

必选参数

1. <materialized_view_name>

> 指定表的标识符（即名称）；在创建表的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <query>

> 在创建物化视图中为必选参数。指定填充数据的 SELECT 语句。

可选参数

1. <key_cols>

> 表的 key 列。Doris 中 Key 列必须是表的前 K 个列。关于 Key 的限制，以及如何选择 Key 列，请参阅 “数据模型” 章节中的各个小节。

2. <build_mode>

> 刷新时机：物化视图创建完成是否立即刷新
>
> IMMEDIATE：立即刷新，默认 IMMEDIATE
>
> DEFERRED：延迟刷新

3. <refresh_method>

刷新方式

COMPLETE：无论分区数据是否有变更，强制刷新所有分区

AUTO：尽量增量刷新，只刷新自上次物化刷新后数据变化的分区，如果是分区物化视图建议用此刷新方式

注意如果是分区物化视图，刷新方式使用 COMPLETE，物化视图会全量刷新所有分区数据，退化成全量物化视图。

4.<refresh_trigger>

触发方式

MANUAL：手动刷新

ON SCHEDULE：定时刷新

ON COMMIT：触发式刷新，基表数据变更，触发物化视图刷新

5. <refresh_unit>

周期刷新时间单位，目前支持 MINUTE，HOUR，DAY，WEEK

6. <partition_col>

如果不指定 PARTITION BY，默认只有一个分区。

如果指定分区字段，会自动推导出字段来自哪个基表并同步基表（当前支持内表和 Hive 表），如果是内表，只允许有一个分区字段。

物化视图也可以通过分区上卷的方式减少物化视图的分区数量，目前分区上卷函数支持 date_trunc

7. <partition_unit>

分区上卷的聚合粒度，目前支持 HOUR，DAY，WEEK，QUARTER，MONTH，YEAR

8. <start_time>

调度开始时间需要比当前时间大，需要是未来的某个时间

9. <table_property>

注意 DISTRIBUTED BY 这个子句如果不写，在 Apache Doris 2.1.10 之前会报错，在 Apache Doris 2.1.10 及以后如果不写，默认是 RANDOM 分布。

内表使用的属性，物化视图基本都可以使用，还有一些是物化视图特有的属性，列举如下

属性名	作用
grace_period	查询改写时允许物化视图数据的最大延迟时间（单位：秒）。如果分区 A 和基表的数据不一致，物化视图的分区 A 上次刷新时间为 10:15:00，系统当前时间为 10:15:08，那么该分区不会被透明改写。但是如果 grace_period = 10，该分区就会被用于透明改写
excluded_trigger_tables	数据刷新时忽略的表名，逗号分割。例如table1,table2
refresh_partition_num	单次 insert 语句刷新的分区数量，默认为 1。物化视图刷新时会先计算要刷新的分区列表，然后根据该配置拆分成多个 Insert 语句顺序执行。遇到失败的 Insert 语句，整个任务将停止执行。物化视图保证单个 Insert 语句的事务性，失败的 Insert 语句不会影响到已经刷新成功的分区
workload_group	物化视图执行刷新任务时使用的 workload_group 名称。用来限制物化视图刷新数据使用的资源，避免影响到其它业务的运行。关于 workload_group 的创建及使用，可参考 WORKLOAD-GROUP 文档。
partition_sync_limit	当基表的分区字段为时间时，可以用此属性配置同步基表的分区范围，配合 partition_sync_time_unit 一起使用。例如设置为 2，partition_sync_time_unit 设置为 MONTH，代表仅同步基表近 2 个月的分区和数据。最小值为 1。随着时间的变化物化视图每次刷新时都会自动增删分区，例如物化视图现在有 2,3 两个月的数据，下个月的时候，会自动删除 2 月的数据，增加 4 月的数据。
partition_sync_time_unit	分区刷新的时间单位，支持 DAY/MONTH/YEAR（默认 DAY）
partition_date_format	当基表的分区字段为字符串时，如果想使用 partition_sync_limit的能力，可以设置日期的格式，将按照 partition_date_format的设置解析分区时间

属性名	作用
enable_nondeterministic_function	物化视图定义 SQL 是否允许包含 nondeterministic 函数，比如 current_date(), now(), random() 等，如果是 true, 允许包含，否则不允许包含，默认不允许包含。
use_for_rewrite	标识此物化视图是否参与到透明改写中，如果为 false，不参与到透明改写，默认是 true。数据建模场景中，如果物化视图只是用于直查，物化视图可以设置此属性，从而不参与透明改写，提高查询响应速度。

> 注意
>
> 在 Apache Doris 2.1.10 和 3.0.6 版本之前，excluded_trigger_tables 属性仅支持指定基础表名。自该版本起，该属性已支持指定包含 Catalog 和 Database 的全限定表名（例如：internal.db1.table1 ）。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
CREATE_PRIV	数据库	
SELECT_PRIV	表, 视图	需要拥有中被查询的表或者视图的 SELECT_PRIV 权限

注意事项

- 物化视图 DML, DDL 限制

物化视图不支持修改列类型，新增，删除列等 schema change 操作，原因是列是通过物化视图定义 SQL 推导出来的。

物化视图不支持手动 insert into 或者 insert overwrite 数据。

- 分区物化视图创建条件

> 物化视图的 SQL 定义和分区字段需要满足如下条件，才可以进行分区增量更新：
>
> 1. 物化视图使用的 Base Table 中至少有一个是分区表。
> 2. 物化视图使用的 Base Table 分区表，必须使用 List 或者 Range 分区策略。
> 3. 物化视图定义 SQL 中 Partition By 分区列只能有一个分区字段。
> 4. 物化视图的 SQL 中 Partition By 的分区列，要在 Select 后。
> 5. 物化视图定义 SQL，如果使用了 Group By，分区列的字段一定要在 Group By 后。

6. 物化视图定义 SQL，如果使用了 Window 函数，分区列的字段一定要在 Partition By 后。
7. 数据变更应发生在分区表上，如果发生在非分区表，物化视图需要全量构建。
8. 物化视图使用 Join 的 NULL 产生端的字段作为分区字段，不能分区增量更新，例如对于 LEFT OUTER JOIN 分区字段需要在左侧，在右侧则不行。

示例

1. 全量物化视图

```
CREATE MATERIALIZED VIEW complete_mv (
orderdate COMMENT '订单日期',
orderkey COMMENT '订单键',
partkey COMMENT '部件键'
)
BUILD IMMEDIATE
REFRESH AUTO
ON SCHEDULE EVERY 1 DAY STARTS '2024-12-01 20:30:00'
DISTRIBUTED BY HASH (orderkey) BUCKETS 2
PROPERTIES
("replication_num" = "1")
AS
SELECT
o_orderdate,
l_orderkey,
l_partkey
FROM
orders
LEFT JOIN lineitem ON l_orderkey = o_orderkey
LEFT JOIN partsupp ON ps_partkey = l_partkey
and l_suppkey = ps_suppkey;
```

2. 分区物化视图

如下所示，如果指定分区字段，会自动推导出字段来自哪个基表并同步基表分区。基表按照天分区，分区字段是 o_orderdate，分区类型是 RANGE。物化视图按月分区，使用 DATE_TRUNC 函数对基表分区按月进行上卷。

```
CREATE TABLE IF NOT EXISTS orders  (
o_orderkey       integer not null,
o_custkey        integer not null,
o_orderstatus    char(1) not null,
o_totalprice     decimalv3(15,2) not null,
o_orderdate      date not null,
o_orderpriority  char(15) not null,
```

```
    o_clerk          char(15) not null,
    o_shippriority   integer not null,
    o_comment        varchar(79) not null
    )
    DUPLICATE KEY(o_orderkey, o_custkey)
    PARTITION BY RANGE(o_orderdate)(
    FROM ('2023-10-16') TO ('2023-11-30') INTERVAL 1 DAY
    )
    DISTRIBUTED BY HASH(o_orderkey) BUCKETS 3
    PROPERTIES (
    "replication_num" = "3"
    );
```

```
    CREATE MATERIALIZED VIEW partition_mv
    BUILD IMMEDIATE
    REFRESH AUTO
    ON SCHEDULE EVERY 1 DAY STARTS '2024-12-01 20:30:00'
    PARTITION BY (DATE_TRUNC(o_orderdate, 'MONTH'))
    DISTRIBUTED BY HASH (l_orderkey) BUCKETS 2
    PROPERTIES
    ("replication_num" = "3")
    AS
    SELECT
    o_orderdate,
    l_orderkey,
    l_partkey
    FROM
    orders
    LEFT JOIN lineitem ON l_orderkey = o_orderkey
    LEFT JOIN partsupp ON ps_partkey = l_partkey
    and l_suppkey = ps_suppkey;
```

如下用例就不能创建分区物化视图，因为分区字段使用了非 date_trunc 函数，报错信息是

```
because column to check use invalid implicit expression, invalid expression is min(o_orderdate#4)
```

```
    CREATE MATERIALIZED VIEW partition_mv_2
    BUILD IMMEDIATE
    REFRESH AUTO
    ON SCHEDULE EVERY 1 DAY STARTS '2024-12-01 20:30:00'
    PARTITION BY (DATE_TRUNC(min_orderdate, 'MONTH'))
    DISTRIBUTED BY HASH (l_orderkey) BUCKETS 2
    PROPERTIES
    ("replication_num" = "3")
    AS
    SELECT
```

```
    min(o_orderdate) AS min_orderdate,
    l_orderkey,
    l_partkey
    FROM
    orders
    LEFT JOIN lineitem ON l_orderkey = o_orderkey
    LEFT JOIN partsupp ON ps_partkey = l_partkey
    and l_suppkey = ps_suppkey
    GROUP BY
    o_orderdate,
    l_orderkey,
    l_partkey;
```

3. 修改物化视图属性

使用 ALTER MATERIALIZED VIEW 语句。例如：

```
    ALTER MATERIALIZED VIEW partition_mv
    SET (
        "grace_period" = "10",
        "excluded_trigger_tables" = "lineitem,partsupp"
    );
```

4. 修改物化视图刷新方式使用 ALTER MATERIALIZED VIEW 语句。例如：

```
    ALTER MATERIALIZED VIEW partition_mv REFRESH COMPLETE;
```

再运行 SHOW CREATE MATERIALIZED VIEW partition_mv; 可以看到物化视图的刷新方式已经变成了 COMPLETE。

7.3.8.5.4 PAUSE MATERIALIZED VIEW

描述

该语句用于暂停物化视图的定时调度

语法

```
PAUSE MATERIALIZED VIEW JOB ON <mv_name>
```

必选参数

1. <mv_name>

指定物化视图的名字。

物化视图的名字必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个名字串用反引号括起来（例如`My Object`）。

物化视图的名字不能使用保留关键字。

有关更多详细信息，请参阅保留关键字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	物化视图	PAUSE 属于物化视图的 ALTER 操作

注意事项

- 当执行完 PAUSE MATERIALIZED VIEW 语句后，可以使用 RESUME MATERIALIZED VIEW 语句恢复暂停的任务

示例

- 暂停物化视图 mv1 的定时调度

```
PAUSE MATERIALIZED VIEW JOB ON mv1;
```

7.3.8.5.5 DROP ASYNC MATERIALIZED VIEW

描述

该语句用于删除异步物化视图。

语法：

```
DROP MATERIALIZED VIEW (IF EXISTS)? mvName=multipartIdentifier
```

1. IF EXISTS: 如果物化视图不存在，不要抛出错误。如果不声明此关键字，物化视图不存在则报错。
2. mv_name: 待删除的物化视图的名称。必填项。

示例

1. 删除表物化视图 mv1

```
DROP MATERIALIZED VIEW mv1;
```

2. 如果存在，删除指定 database 的物化视图

```
DROP MATERIALIZED VIEW IF EXISTS db1.mv1;
```

关键词

```
DROP, ASYNC, MATERIALIZED, VIEW
```

最佳实践

7.3.8.5.6 REFRESH MATERIALIZED VIEW

描述

该语句用于手动刷新指定的异步物化视图

语法

```
REFRESH MATERIALIZED VIEW <mv_name> <refresh_type>
```

其中：

```
refresh_type
  : { <partitionSpec> | COMPLETE | AUTO }
```

```
partitionSpec
  : PARTITIONS (<partition_name> [, <partition_name> [, ... ] ])
```

必选参数

1. <mv_name>

> 指定物化视图的名字。
>
> 物化视图的名字必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个名字串用反引号括起来（例如`My Object`）。
>
> 物化视图的名字不能使用保留关键字。
>
> 有关更多详细信息，请参阅保留关键字。

2. <refresh_type>

> 指定物化视图的刷新方式。
>
> 其刷新方式可以是`COMPLETE`，`AUTO`，`partitionSpec` 三种之一

可选参数

1. <partition_name> > 指定要刷新分区的分区名称 >

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	物化视图	REFRESH 属于物化视图的 ALTER 操作

注意事项

- AUTO：会计算物化视图的哪些分区和基表不同步（目前，如果基表是外表，会被认为始终和物化视图同步，因此如果基表是外表，需要指定COMPLETE或指定要刷新的分区），然后刷新对应的分区
- COMPLETE：会强制刷新物化视图的所有分区，不会判断分区是否和基表同步
- partitionSpec：会强制刷新指定的分区，不会判断分区是否和基表同步

示例

- 刷新物化视图 mv1(自动计算要刷新的分区)

```
REFRESH MATERIALIZED VIEW mv1 AUTO;
```

- 刷新名字为 p_19950801_19950901 和 p_19950901_19951001 的分区

```
REFRESH MATERIALIZED VIEW mv1 partitions(p_19950801_19950901,p_19950901_19951001);
```

- 强制刷新物化视图全部数据

```
REFRESH MATERIALIZED VIEW mv1 complete;
```

7.3.8.5.7 RESUME MATERIALIZED VIEW

描述

该语句用于暂恢复物化视图的定时调度

语法

```
RESUME MATERIALIZED VIEW JOB ON <mv_name>
```

必选参数

1. <mv_name>

指定物化视图的名字。

物化视图的名字必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个名字串用反引号括起来（例如`My Object`）。

物化视图的名字不能使用保留关键字。

有关更多详细信息，请参阅保留关键字。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	物化视图	RESUME 属于物化视图的 ALTER 操作

注意事项

- 该语句一般在 PAUSE MATERIALIZED VIEW 语句后执行

示例

- 恢复物化视图 mv1 的定时调度

```
RESUME MATERIALIZED VIEW JOB ON mv1;
```

7.3.8.5.8 SHOW CREATE ASYNC MATERIALIZED VIEW

描述

查看异步物化视图创建语句。

语法

```
SHOW CREATE MATERIALIZED VIEW <materialized_view_name>
```

必选参数

1. <materialized_view_new_name>

物化视图名称

返回值

列名	说明
Materialized View	物化视图名
Create Materialized View	物化视图创建语句

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
SELECT_PRIV/LOAD_PRIV/ALTER_PRIV/CREATE_PRIV/DROP_PRIV	表	

示例（Examples）

1. 查看异步物化视图创建语句

```
SHOW CREATE MATERIALIZED VIEW partition_mv;
```

7.3.8.6 表数据和状态管理

7.3.8.6.1 REBALANCE DISK

描述

REBALANCE DISK 语句用于优化 BE（Backend）节点上的数据分布。该语句具有以下功能：

- 可以针对指定的 BE 节点进行数据均衡
- 可以对整个集群的所有 BE 节点进行数据均衡
- 优先均衡指定节点的数据，不受集群整体均衡状态的限制

语法

```
ADMIN REBALANCE DISK [ ON ( "<host>:<port>" [, ... ] ) ];
```

可选参数

1. "<host>:<port>"

> 指定需要进行数据均衡的 BE 节点列表。
>
> 每个节点由主机名（或 IP 地址）和心跳端口组成。
>
> 如果不指定此参数，则对所有 BE 节点进行均衡。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN	系统	用户必须拥有 ADMIN 权限才能执行该命令

注意事项

- 命令的默认超时时间为 24 小时。超时后，系统将不再优先均衡指定的 BE 磁盘数据。如需继续均衡，需要重新执行该命令。
- 当指定 BE 节点的磁盘数据均衡完成后，该节点的优先均衡设置将自动失效。
- 该命令可以在集群非均衡状态下执行。

示例

- 对集群内所有 BE 节点进行数据均衡：

```
ADMIN REBALANCE DISK;
```

- 对指定的两个 BE 节点进行数据均衡：

```
ADMIN REBALANCE DISK ON ("192.168.1.1:1234", "192.168.1.2:1234");
```

7.3.8.6.2 CANCEL REBALANCE DISK

描述

CANCEL REBALANCE DISK 语句用于取消优先均衡 BE（Backend）节点的磁盘数据。该语句具有以下功能：

- 可以取消指定 BE 节点的优先磁盘均衡
- 可以取消整个集群所有 BE 节点的优先磁盘均衡
- 取消后系统仍会以默认调度方式均衡 BE 的磁盘数据

语法

```
ADMIN CANCEL REBALANCE DISK [ ON ( "<host>:<port>" [, ... ] ) ];
```

可选参数

1. "<host>:<port>"

> 指定需要取消优先磁盘均衡的 BE 节点列表。
>
> 每个节点由主机名（或 IP 地址）和心跳端口组成。
>
> 如果不指定此参数，则取消所有 BE 节点的优先磁盘均衡。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN	系统	用户必须拥有 ADMIN 权限才能执行该命令

注意事项

- 该语句仅表示系统不再优先均衡指定 BE 的磁盘数据，系统仍会以默认调度方式均衡 BE 的磁盘数据。
- 执行该命令后，之前设置的优先均衡策略将立即失效。

示例

- 取消集群所有 BE 的优先磁盘均衡：

```
    ADMIN CANCEL REBALANCE DISK;
```

- 取消指定 BE 的优先磁盘均衡：

```
ADMIN CANCEL REBALANCE DISK ON ("192.168.1.1:1234", "192.168.1.2:1234");
```

7.3.8.6.3 SHOW DATA

描述

SHOW DATA 语句用于展示数据量、副本数量以及统计行数信息。该语句具有以下功能：

- 可以展示当前数据库下所有表的数据量和副本数量
- 可以展示指定表的物化视图数据量、副本数量和统计行数
- 可以展示数据库的配额使用情况
- 支持按照数据量、副本数量等进行排序

语法

```
SHOW DATA [ FROM [<db_name>.]<table_name> ] [ ORDER BY <order_by_clause> ];
```

其中：

```
order_by_clause:
    <column_name> [ ASC | DESC ] [ , <column_name> [ ASC | DESC ] ... ]
```

可选参数

1. FROM [<db_name>.]<table_name>

指定要查看的表名。可以包含数据库名称。
如果不指定此参数，则展示当前数据库下所有表的数据信息。

2. ORDER BY <order_by_clause>

指定结果集的排序方式。
可以对任意列进行升序（ASC）或降序（DESC）排序。
支持多列组合排序。

返回值

根据不同查询场景，返回以下结果集：

- 不指定 FROM 子句时（展示数据库级别信息）：

列名	说明
DbId	数据库 ID
DbName	数据库名称
Size	数据库总数据量
RemoteSize	远程存储数据量
RecycleSize	回收站数据量
RecycleRemoteSize	回收站远程存储数据量

- 指定 FROM 子句时（展示表级别信息）：

列名	说明
TableName	表名
IndexName	索引（物化视图）名称
Size	数据大小
ReplicaCount	副本数量
RowCount	统计行数（仅在查看具体表时显示）

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT	表（Table）	需要对查看的表有 SELECT 权限

注意事项

- 数据量统计包含所有副本的总数据量
- 副本数量包含表的所有分区以及所有物化视图的副本数量
- 统计行数时，以多个副本中行数最大的那个副本为准
- 结果集中的 Total 行表示汇总数据
- 结果集中的 Quota 行表示当前数据库设置的配额
- 结果集中的 Left 行表示剩余配额
- 如果需要查看各个 Partition 的大小，请使用 SHOW PARTITIONS 命令

示例

- 展示所有数据库的数据量信息：

```
SHOW DATA;
```

```
+-------+----------------------------------+--------+------------+-------------+------------------+
    ↪
| DbId  | DbName                           | Size   | RemoteSize | RecycleSize |
    ↪ RecycleRemoteSize |
+-------+----------------------------------+--------+------------+-------------+------------------+
    ↪
| 21009 | db1                              | 0      | 0          | 0           | 0
    ↪                 |
| 22011 | regression_test_inverted_index_p0 | 72764  | 0          | 0           | 0
    ↪                 |
| Total | NULL                             | 118946 | 0          | 0           | 0
    ↪                 |
+-------+----------------------------------+--------+------------+-------------+------------------+
    ↪
```

- 展示当前数据库下所有表的数据量信息：

```
USE db1;
SHOW DATA;
```

```
+-----------+-------------+--------------+
| TableName | Size        | ReplicaCount |
+-----------+-------------+--------------+
| tbl1      | 900.000 B   | 6            |
```

```
| tbl2      | 500.000 B   | 3            |
| Total     | 1.400 KB    | 9            |
| Quota     | 1024.000 GB | 1073741824   |
| Left      | 1021.921 GB | 1073741815   |
+-----------+-------------+--------------+
```

- 展示指定表的详细数据量信息：

```
SHOW DATA FROM example_db.test;
```

```
+-----------+-----------+-----------+--------------+-----------+
| TableName | IndexName | Size      | ReplicaCount | RowCount |
+-----------+-----------+-----------+--------------+-----------+
| test      | r1        | 10.000MB  | 30           | 10000    |
|           | r2        | 20.000MB  | 30           | 20000    |
|           | test2     | 50.000MB  | 30           | 50000    |
|           | Total     | 80.000    | 90           |          |
+-----------+-----------+-----------+--------------+-----------+
```

- 按照副本数量降序、数据量升序排序：

```
SHOW DATA ORDER BY ReplicaCount DESC, Size ASC;
```

```
+-----------+-------------+--------------+
| TableName | Size        | ReplicaCount |
+-----------+-------------+--------------+
| table_c   | 3.102 KB    | 40           |
| table_d   | .000        | 20           |
| table_b   | 324.000 B   | 20           |
| table_a   | 1.266 KB    | 10           |
| Total     | 4.684 KB    | 90           |
| Quota     | 1024.000 GB | 1073741824   |
| Left      | 1024.000 GB | 1073741734   |
+-----------+-------------+--------------+
```

7.3.8.6.4 SHOW DATA SKEW

描述

SHOW DATA SKEW 语句用于查看表或分区的数据倾斜情况。该语句具有以下功能：

- 可以查看整个表的数据分布情况
- 可以查看指定分区的数据分布情况

- 展示各个分桶的数据行数、数据量及其占比
- 支持分区表和非分区表

语法

```
SHOW DATA SKEW FROM [<db_name>.]<table_name> [ PARTITION (<partition_name> [, ...]) ];
```

必选参数

1. FROM [<db_name>.]<table_name>

> 指定要查看的表名。可以包含数据库名称。
>
> 表名在其所在的数据库中必须唯一。

可选参数

1. PARTITION (<partition_name> [, ...])

> 指定要查看的分区名称列表。
>
> 如果不指定此参数，则展示表中所有分区的数据分布情况。
>
> 对于非分区表，分区名称同表名。

返回值

列名	说明
PartitionName	分区名称
BucketIdx	分桶索引号
AvgRowCount	平均行数
AvgDataSize	平均数据大小（字节）
Graph	数据分布可视化图表
Percent	该分桶数据量占总数据量的百分比

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT	表（Table）	需要对查看的表有 SELECT 权限

注意事项

- 数据分布情况按照分区和分桶两个维度展示
- Graph 列使用字符 > 直观展示数据分布比例
- 百分比精确到小数点后两位
- 对于非分区表，查询结果中分区名称同表名

示例

- 创建分区表并查看数据分布：

```
CREATE TABLE test_show_data_skew
(
    id int,
    name string,
    pdate date
)
PARTITION BY RANGE(pdate)
(
    FROM ("2023-04-16") TO ("2023-04-20") INTERVAL 1 DAY
)
DISTRIBUTED BY HASH(id) BUCKETS 5
PROPERTIES (
    "replication_num" = "1"
);
```

查看整表数据分布：

```
SHOW DATA SKEW FROM test_show_data_skew;
```

```
+---------------+-----------+-------------+-------------+-------------------------------------------------
    ↪
| PartitionName | BucketIdx | AvgRowCount | AvgDataSize | Graph
    ↪
    ↪ | Percent |
+---------------+-----------+-------------+-------------+-------------------------------------------------
    ↪
| p_20230416    | 0         | 1           | 648         |
    ↪ >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
    ↪                                                       | 49.77 % |
| p_20230416    | 1         | 2           | 654         |
    ↪ >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
    ↪                                                      | 50.23 % |
| p_20230416    | 2         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
```

```
| p_20230416    | 3         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230416    | 4         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230417    | 0         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230417    | 1         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230417    | 2         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230417    | 3         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230417    | 4         | 2           | 656         |
    ↪ >>>>
```

```
| p_20230419    | 3         | 1           | 649         |
    ↪ >>>>
```

```
| p_20230418    | 3         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
| p_20230418    | 4         | 0           | 0           |
    ↪
    ↪ | 00.00 % |
+---------------+-----------+-------------+-------------+-----------------------------------------------
    ↪
```

- 查看非分区表的数据分布：

```
CREATE TABLE test_show_data_skew2
(
    id int,
    name string,
    pdate date
)
DISTRIBUTED BY HASH(id) BUCKETS 5
PROPERTIES (
    "replication_num" = "1"
);
```

```
SHOW DATA SKEW FROM test_show_data_skew2;
```

```
+----------------------+-----------+-------------+-------------+---------------------------+---------+
    ↪
| PartitionName        | BucketIdx | AvgRowCount | AvgDataSize | Graph                     |
    ↪ Percent |
+----------------------+-----------+-------------+-------------+---------------------------+---------+
    ↪
| test_show_data_skew2 | 0         | 1           | 648         | >>>>>>>>>>>>>>>>>>>>>>>>  |
    ↪ 24.73 % |
| test_show_data_skew2 | 1         | 4           | 667         | >>>>>>>>>>>>>>>>>>>>>>>>> |
    ↪ 25.46 % |
| test_show_data_skew2 | 2         | 0           | 0           |                           |
    ↪ 00.00 % |
| test_show_data_skew2 | 3         | 1           | 649         | >>>>>>>>>>>>>>>>>>>>>>>>  |
    ↪ 24.77 % |
| test_show_data_skew2 | 4         | 2           | 656         | >>>>>>>>>>>>>>>>>>>>>>>>> |
    ↪ 25.04 % |
+----------------------+-----------+-------------+-------------+---------------------------+---------+
    ↪
```

7.3.8.6.5 COMPACT TABLE

描述

存算一体模式中，用于对指定表分区下的所有副本触发一次 Compaction。

存算分离模式不支持这个命令。

语法

```
ADMIN COMPACT TABLE <table_name>
PARTITION <partition_name>
WHERE TYPE={ BASE | CUMULATIVE }
```

必选参数

1. `<table_name>`：待触发 Compaction 的表名
2. `<partition_name>`：待触发 Compaction 的表名
3. `TYPE={ BASE | CUMULATIVE }`：其中 BASE 是指触发 Base Compaction，CUMULATIVE 是指触发 Cumulative Compaction，具体可以参考 COMPACTION 章节

权限控制

执行此 SQL 命令成功的前置条件是，拥有 ADMIN_PRIV 权限，参考权限文档。

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	整个集群管理权限	除 NODE_PRIV 以外的所有权限

示例

1. 触发表 tbl 分区 par01 的 cumulative compaction。

```
ADMIN COMPACT TABLE tbl PARTITION par01 WHERE TYPE='CUMULATIVE';
```

注意事项（Usage Note）

1. 存算分离模式不支持这个命令，在此模式下执行会报错，例如：

```
ADMIN COMPACT TABLE tbl PARTITION par01 WHERE TYPE='CUMULATIVE';
```

报错信息如下：

```
ERROR 1105 (HY000): errCode = 2, detailMessage = Unsupported operation
```

7.3.8.6.6 REPAIR TABLE

描述

`REPAIR TABLE` 语句用于优先修复指定表或分区的副本。该语句具有以下功能：

- 可以修复整个表的所有副本
- 可以修复指定分区的副本
- 以高优先级进行副本修复
- 支持设置修复超时时间

语法

```
ADMIN REPAIR TABLE <table_name> [ PARTITION (<partition_name> [, ...]) ];
```

必选参数

1. `<table_name>`

> 指定需要修复的表名。
>
> 表名在其所在的数据库中必须唯一。

可选参数

1. `PARTITION (<partition_name> [, ...])`

> 指定需要修复的分区名称列表。
>
> 如果不指定此参数，则修复整个表的所有分区。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN	系统	用户必须拥有 ADMIN 权限才能执行该命令

注意事项

- 该语句仅表示系统会尝试以高优先级修复指定的副本，不保证一定能修复成功
- 默认超时时间为 14400 秒（4 小时）
- 超时后系统将不再以高优先级修复指定的副本

- 如果修复超时，需要重新执行该命令来继续修复
- 可以通过 SHOW REPLICA STATUS 命令查看修复进度
- 该命令不会影响系统的正常副本修复机制，仅提升指定表或分区的修复优先级

示例

- 修复整个表的副本：

```
ADMIN REPAIR TABLE tbl1;
```

- 修复指定分区的副本：

```
ADMIN REPAIR TABLE tbl1 PARTITION (p1, p2);
```

- 查看修复进度：

```
SHOW REPLICA STATUS FROM tbl1;
```

7.3.8.6.7 CANCEL REPAIR TABLE

描述

CANCEL REPAIR TABLE 语句用于取消对指定表或分区的高优先级修复。该语句具有以下功能：

- 可以取消整个表的高优先级修复
- 可以取消指定分区的高优先级修复
- 不影响系统默认的副本修复机制

语法

```
ADMIN CANCEL REPAIR TABLE <table_name> [ PARTITION (<partition_name> [, ...]) ];
```

必选参数

1. <table_name>

> 指定要取消修复的表名。
>
> 表名在其所在的数据库中必须唯一。

可选参数

1. PARTITION (<partition_name> [, ...])

指定要取消修复的分区名称列表。

如果不指定此参数，则取消整个表的高优先级修复。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN	系统	用户必须拥有 ADMIN 权限才能执行该命令

注意事项

- 该语句仅取消高优先级修复，不会停止系统的默认副本修复机制
- 取消后，系统仍会以默认调度方式修复副本
- 如果需要重新设置高优先级修复，可以使用 `ADMIN REPAIR TABLE` 命令
- 该命令执行后立即生效

示例

- 取消整个表的高优先级修复：

```
ADMIN CANCEL REPAIR TABLE tbl;
```

- 取消指定分区的高优先级修复：

```
ADMIN CANCEL REPAIR TABLE tbl PARTITION(p1, p2);
```

7.3.8.6.8 SET TABLE STATUS

描述

`SET TABLE STATUS` 语句用于手动设置 OLAP 表的状态。该语句具有以下功能：

- 仅支持 OLAP 表的状态设置
- 可以将表状态修改为指定的目标状态
- 用于解除因表状态导致的任务阻塞

支持的状态：

状态	说明
NORMAL	表示表处于正常状态
ROLLUP	表示表正在进行 ROLLUP 操作
SCHEMA_CHANGE	表示表正在进行 Schema 变更
BACKUP	表示表正在进行备份
RESTORE	表示表正在进行恢复
WAITING_STABLE	表示表正在等待稳定状态

语法

```
ADMIN SET TABLE <table_name> STATUS PROPERTIES ("<key>" = "<value>" [, ...]);
```

其中：

```
<key>
  : "state"

<value>
  : "NORMAL"
  | "ROLLUP"
  | "SCHEMA_CHANGE"
  | "BACKUP"
  | "RESTORE"
  | "WAITING_STABLE"
```

必选参数

1. <table_name>

> 指定要设置状态的表名。
> 表名在其所在的数据库中必须唯一。

2. PROPERTIES ("state" = "<value>")

> 指定表的目标状态。
> 必须设置 “state” 属性，且值必须是支持的状态之一。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN	系统	用户必须拥有 ADMIN 权限才能执行该命令

注意事项

- 此命令仅用于紧急故障修复，请谨慎操作
- 仅支持 OLAP 表，不支持其他类型的表
- 如果表已经处于目标状态，该命令将被忽略
- 不当的状态设置可能会导致系统异常，建议在技术支持指导下使用
- 修改状态后，建议及时观察系统运行情况

示例

- 将表状态设置为 NORMAL：

```
ADMIN SET TABLE tbl1 STATUS PROPERTIES("state" = "NORMAL");
```

- 将表状态设置为 SCHEMA_CHANGE：

```
ADMIN SET TABLE tbl2 STATUS PROPERTIES("state" = "SCHEMA_CHANGE");
```

7.3.8.6.9 SET TABLE PARTITION VERSION

描述

存算一体模式，该语句用于手动改变指定分区的可见版本。在某些特殊情况下，元数据中分区的版本有可能和实际副本的版本不一致。该命令可手动改变元数据中分区的版本。此命令一般只用于紧急故障修复，请谨慎操作。

语法

```
ADMIN SET TABLE <table_name> PARTITION VERSION PROPERTIES ("<partition_id>" = "visible_version>")
    ↪ ;
```

必选参数

1. <table_name>: 待设置的表名
2. <partition_id>: 指定一个 Partition Id
3. <visible_version>: 指定 Version

示例

1. 设置 partition_id 为 10075 的分区在 FE 元数据上的版本为 100

```
ADMIN SET TABLE __internal_schema.audit_log PARTITION VERSION PROPERTIES("partition_id" = "
    ↪ 10075", "visible_version" = "100");
```

注意事项

1. 设置分区的版本需要先确认 BE 机器上实际副本的版本，此命令一般只用于紧急故障修复，请谨慎操作。
2. 存算分离模式不支持这个命令，设置了不会生效

7.3.8.6.10 DIAGNOSE TABLET

描述

存算一体模式中，该语句用于诊断指定 tablet。结果中将显示这个 tablet 的信息和一些潜在的问题。

存算分离模式不支持这个命令。

语法

```
SHOW TABLET DIAGNOSIS <tablet_id>;
```

必选参数

1. <tablet_id>: 待诊断 tablet 的 id

返回值

返回 tablet 相关信息

- TabletExist: Tablet 是否存在
- TabletId : Tablet ID
- Database: Tablet 所属 DB 和其 ID
- Table: Tablet 所属 Table 和其 ID
- Partition: Tablet 所属 Partition 和其 ID
- MaterializedIndex: Tablet 所属物化视图和其 ID
- Replicas: Tablet 各副本和其所在 BE
- ReplicasNum: 副本数量是否正确
- ReplicaBackendStatus: 副本所在 BE 节点是否正常
- ReplicaVersionStatus: 副本的版本号是否正常
- ReplicaStatus: 副本状态是否正常
- ReplicaCompactionStatus: 副本 Compaction 状态是否正常

示例

1. 诊断指定 tablet id 为 10078 的 tablet 信息

```
show tablet diagnosis 10078;
+--
    ↪ -------------------------------+-----------------------------------------------+------------+
    ↪
| Item                            | Info                                          | Suggestion
    ↪  |
+--
    ↪ -------------------------------+-----------------------------------------------+------------+
    ↪
| TabletExist                     | Yes                                           |
    ↪            |
| TabletId                        | 10078                                         |
    ↪            |
| Database                        | __internal_schema: 10005                      |
    ↪            |
| Table                           | audit_log: 10058                              |
    ↪            |
| Partition                       | p20241109: 10075                              |
    ↪            |
| MaterializedIndex               | audit_log: 10059                              |
    ↪            |
| Replicas(ReplicaId -> BackendId) | {"10099":10003,"10116":10002,"10079":10004} |
    ↪            |
| ReplicasNum                     | OK                                            |
    ↪            |
| ReplicaBackendStatus            | OK                                            |
    ↪            |
| ReplicaVersionStatus            | OK                                            |
    ↪            |
| ReplicaStatus                   | OK                                            |
    ↪            |
| ReplicaCompactionStatus         | OK                                            |
    ↪            |
+--
    ↪ -------------------------------+-----------------------------------------------+------------+
    ↪
```

权限控制

执行此 SQL 命令成功的前置条件是，拥有 ADMIN_PRIV 权限，参考权限文档。

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	整个集群管理权限	除 NODE_PRIV 以外的所有权限

注意事项

1. 存算分离模式不支持这个命令，在此模式下执行会报错，例如：

```
    show tablet diagnosis 15177;
```

报错信息如下：

```
    ERROR 1105 (HY000): errCode = 2, detailMessage = Unsupported operation
```

7.3.8.6.11 ADMIN COPY TABLET

描述

该语句用于为指定的 tablet 制作快照，主要用于本地加载 tablet 来复现问题。

语法

```
ADMIN COPY TABLET <tablet_id> PROPERTIES ("<key>"="<value>" [,...]).
```

必选参数

1. <tablet_id>

要复制的 tablet 的 ID。

可选参数

```
  [ PROPERTIES ("<key>"="<value>" [, ... ]) ]
```

PROPERTIES 子句允许指定附加参数：

1. <backend_id>

指定副本所在的 BE 节点 ID。如果未指定，则随机选择一个副本。

2. <version>

指定快照的版本。版本必须小于或等于副本的最大版本。如果未指定，则使用最大版本。

3. <expiration_minutes>

快照的保留时间。默认为 1 小时，超时后会自动清理。单位为分钟。

返回值

列名	类型	说明
TabletId	string	为该 tablet 创建的快照的 ID。

列名	类型	说明
BackendId	string	存储该快照的 BE 节点的 ID。
Ip	string	存储该快照的 BE 节点的 IP 地址。
Path	string	快照在 BE 节点上的存储路径。
ExpirationMinutes	string	快照将自动删除的时间（单位：分钟）。
CreateTableStmt	string	对应 tablet 的表创建语句。此语句不是原始的建表语句，而是用于后续加载该 tablet 到本地的简化建表

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 为指定 BE 节点上的副本创建快照

```
ADMIN COPY TABLET 10020 PROPERTIES("backend_id" = "10003");
```

```
          TabletId: 10020
         BackendId: 10003
                Ip: 192.168.10.1
              Path: /path/to/be/storage/snapshot/20220830101353.2.3600
ExpirationMinutes: 60
  CreateTableStmt: CREATE TABLE `tbl1` (
  `k1` int(11) NULL,
  `k2` int(11) NULL
) ENGINE=OLAP
DUPLICATE KEY(`k1`, `k2`)
DISTRIBUTED BY HASH(k1) BUCKETS 1
PROPERTIES (
"replication_num" = "1",
"version_info" = "2"
);
```

- 为指定 BE 节点上指定版本的副本创建快照

```
ADMIN COPY TABLET 10010 PROPERTIES("backend_id" = "10003", "version" = "10");
```

```
          TabletId: 10010
         BackendId: 10003
```

```
                Ip: 192.168.10.1
              Path: /path/to/be/storage/snapshot/20220830101353.2.3600
  ExpirationMinutes: 60
    CreateTableStmt: CREATE TABLE `tbl1` (
    `k1` int(11) NULL,
    `k2` int(11) NULL
  ) ENGINE=OLAP
  DUPLICATE KEY(`k1`, `k2`)
  DISTRIBUTED BY HASH(k1) BUCKETS 1
  PROPERTIES (
  "replication_num" = "1",
  "version_info" = "2"
  );
```

7.3.8.6.12 ADMIN CHECK TABLET

描述

该语句用于对一组 tablet 执行指定的检查操作

语法

```
ADMIN CHECK TABLET ( <tablet_id> [,...] ) PROPERTIES("type" = "<type_value>")
```

必选参数

1. `<tablet_id>`

需要进行执行指定的检查操作的 tablet ID。

可选参数

1. `<type_value>` 目前只支持 `consistency`

- consistency:

对 tablet 的副本数据一致性进行检查。该命令为异步命令，发送后，Doris 会开始执行对应 tablet 的一致性检查作业。

返回值

执行语句的最终的结果，将体现在`SHOW PROC "/cluster_health/tablet_health";` 结果中的 InconsistentNum 列。

列名	类型	说明
InconsistentNum	Int	不一致的的 tablet 数量

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 对指定的一组 tablet 进行副本数据一致性检查

```
admin check tablet (10000, 10001) PROPERTIES("type" = "consistency");
```

7.3.8.6.13 SHOW TABLET

描述

该语句用于显示指定 tablet id 信息（仅管理员使用）。

语法

```
SHOW TABLET <tablet_id>;
```

必选参数

1. <tablet_id>

> 要显示信息的特定 tablet 的 ID。

返回值

使用 SHOW TABLET <tablet_id> 时，将返回以下列：

列名	数据类型	说明
DbName	String	包含该 tablet 的数据库名称。
TableName	String	包含该 tablet 的表名称。
PartitionName	String	包含该 tablet 的分区名称。
IndexName	String	包含该 tablet 的索引名称。
DbId	Int	数据库的 ID。
TableId	Int	表的 ID。
PartitionId	Int	分区的 ID。
IndexId	Int	索引的 ID。
IsSync	Boolean	该 tablet 是否与其副本同步。
Order	Int	tablet 的顺序。
QueryHits	Int	该 tablet 上的查询命中次数。
DetailCmd	String	获取有关该 tablet 更详细信息的命令。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
Admin_priv	Database	需要执行数据库的管理操作，包括管理表、分区和系统级命令。

示例

显示特定 tablet 的详细信息：

```
SHOW TABLET 10145;
```

```
+--------+-----------+---------------+-----------+-------+---------+-------------+---------+--------+-------+----
    ↪
| DbName | TableName | PartitionName | IndexName | DbId  | TableId | PartitionId | IndexId |
    ↪ IsSync | Order | QueryHits | DetailCmd                                                                  |
+--------+-----------+---------------+-----------+-------+---------+-------------+---------+--------+-------+----
    ↪
| test   | sell_user | sell_user     | sell_user | 10103 | 10143   | 10142       | 10144   | true
    ↪    | 0     | 0          | SHOW PROC '/dbs/10103/10143/partitions/10142/10144/10145'; |
+--------+-----------+---------------+-----------+-------+---------+-------------+---------+--------+-------+----
    ↪
```

7.3.8.6.14 SHOW TABLETS BELONG

描述

该语句用于显示指定 tablet 及其所属表的信息。

语法

```
SHOW TABLETS BELONG <tablet_id> [, <tablet_id>]...;
```

必选参数

1. <tablet_id>

> 一个或多个 tablet ID，用逗号分隔。结果中会对重复的 ID 进行去重。

返回值

使用 SHOW TABLETS BELONG <tablet_id> [, <tablet_id>]... 时，将返回以下列：

列名	数据类型	说明
DbName	String	包含该 tablet 的数据库名称。
TableName	String	包含该 tablet 的表名称。
TableSize	String	表的大小（例如：“8.649 KB”）。
PartitionNum	Int	表中的分区数量。
BucketNum	Int	表中的分桶数量。
ReplicaCount	Int	表中的副本数量。
TabletIds	Array	属于该表的 tablet ID 列表。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
Admin_priv	Database	需要执行数据库的管理操作，包括管理表、分区和系统级命令。

示例

显示特定 tablet 的信息：

```
SHOW TABLETS BELONG 10145;
```

```
+--------+-----------+-----------+-------------+-----------+-------------+-----------+
| DbName | TableName | TableSize | PartitionNum | BucketNum | ReplicaCount | TabletIds |
+--------+-----------+-----------+-------------+-----------+-------------+-----------+
| test   | sell_user | 8.649 KB  | 1            | 4         | 4            | [10145]   |
+--------+-----------+-----------+-------------+-----------+-------------+-----------+
```

显示多个 tablet 的信息：

```
SHOW TABLETS BELONG 27028,78880,78382,27028;
```

```
+---------------------+-----------+-----------+-------------+-----------+-------------+-----------------+
    ↪
| DbName              | TableName | TableSize | PartitionNum | BucketNum | ReplicaCount |
    ↪ TabletIds      |
+---------------------+-----------+-----------+-------------+-----------+-------------+-----------------+
    ↪
| default_cluster:db1 | kec       | 613.000 B | 379          | 604       | 604          | [78880,
    ↪  78382] |
| default_cluster:db1 | test      | 1.874 KB  | 1            | 1         | 1            | [27028]
    ↪         |
+---------------------+-----------+-----------+-------------+-----------+-------------+-----------------+
    ↪
```

7.3.8.6.15 SHOW TABLET STORAGE FORMAT

描述

该语句用于显示 Backend 上的存储格式信息

语法

```
SHOW TABLET STORAGE FORMAT [VERBOSE]
```

可选参数

** 1. VERBOSE **

展示更详细的信息

返回值

列名	类型	说明
BackendId	Int	BE（Backend）节点的 ID，表示该 tablet 的副本所在的节点。
V1Count	Int	V1 版本 tablet 数量。
V2Count	Int	V2 版本 tablet 数量。
TabletId	Int	tablet 的唯一标识符，用于标识具体的 tablet。
StorageFormat	String	tablet 的版本 V1 或 V2

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 执行未添加 verbose 参数的语句

```
show tablet storage format;
```

```
+-----------+---------+---------+
| BackendId | V1Count | V2Count |
+-----------+---------+---------+
| 10002     | 0       | 2867    |
+-----------+---------+---------+
```

- 执行添加 verbose 参数的语句

```
show tablet storage format verbose;
```

```
+-----------+----------+---------------+
| BackendId | TabletId | StorageFormat |
+-----------+----------+---------------+
| 10002     | 39227    | V2            |
| 10002     | 39221    | V2            |
| 10002     | 39215    | V2            |
| 10002     | 39199    | V2            |
+-----------+----------+---------------+
```

7.3.8.6.16 SHOW TABLET DIAGNOSIS

描述

该语句用于诊断指定 tablet。结果中将显示这个 tablet 的信息和一些潜在的问题。

语法

```
SHOW TABLET DIAGNOSIS <tablet_id>
```

必选参数

1. `<tablet_id>`

需要进行执行诊断的 tablet ID。

返回值

列名	类型	说明
TabletExist	String	Tablet 是否存在
TabletId	String	Tablet ID
Database	String	Tablet 所属 DB 和其 ID
Table	String	Tablet 所属 Table 和其 ID
Partition	String	Tablet 所属 Partition 和其 ID
MaterializedIndex	String	Tablet 所属物化视图和其 ID
Replicas(ReplicaId -> BackendId)	String	Tablet 各副本和其所在 BE
ReplicasNum	String	副本数量是否正确
ReplicaBackendStatus	String	副本所在 BE 节点是否正常
ReplicaVersionStatus	String	副本的版本号是否正常
ReplicaStatus	String	副本状态是否正常
ReplicaCompactionStatus	String	副本 Compaction 状态是否正常

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

```
SHOW TABLET DIAGNOSIS 10145;
```

```
+----------------------------------+------------------+------------+
| Item                             | Info             | Suggestion |
+----------------------------------+------------------+------------+
| TabletExist                      | Yes              |            |
| TabletId                         | 10145            |            |
| Database                         | test: 10103      |            |
| Table                            | sell_user: 10143 |            |
| Partition                        | sell_user: 10142 |            |
| MaterializedIndex                | sell_user: 10144 |            |
| Replicas(ReplicaId -> BackendId) | {"10146":10009}  |            |
| ReplicasNum                      | OK               |            |
| ReplicaBackendStatus             | OK               |            |
| ReplicaVersionStatus             | OK               |            |
| ReplicaStatus                    | OK               |            |
| ReplicaCompactionStatus          | OK               |            |
+----------------------------------+------------------+------------+
```

7.3.8.6.17 ADMIN SET REPLICA STATUS

描述

该语句用于设置指定副本的状态，目前仅用于手动将某些副本状态设置为 BAD、DROP 和 OK，从而使得系统能够自动修复这些副本。

语法

```
ADMIN SET REPLICA STATUS
PROPERTIES ("tablet_id"="<tablet_id>","backend_id"="<backend_id>","status"="<status>")
```

必选参数

1. <tablet_id>

需要设置副本状态的 tablet ID。

2. <backend_id>

指定副本所在的 BE 节点 ID

3. <status>

当前仅支持“drop”、“bad”、“ok”如果指定的副本不存在，或状态已经是 bad，则会被忽略

注意：

- 设置为 Bad 状态的副本

它将不能读写。另外，设置 Bad 有时是不生效的。如果该副本实际数据是正确的，当 BE 上报该副本状态是 ok 的，fe 将把副本自动恢复回 ok 状态。操作可能立刻删除该副本，请谨慎操作。

- 设置为 Drop 状态的副本

它仍然可以读写。会在其他机器先增加一个健康副本，再删除该副本。相比设置 Bad，设置 Drop 的操作是安全的。

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 设置 tablet 10003 在 BE 10001 上的副本状态为 bad。

```
ADMIN SET REPLICA STATUS PROPERTIES("tablet_id" = "10003", "backend_id" = "10001", "status" = "
    ↪ bad");
```

- 设置 tablet 10003 在 BE 10001 上的副本状态为 drop。

```
ADMIN SET REPLICA STATUS PROPERTIES("tablet_id" = "10003", "backend_id" = "10001", "status" = "
    ↪ drop");
```

- 设置 tablet 10003 在 BE 10001 上的副本状态为 ok。

```
ADMIN SET REPLICA STATUS PROPERTIES("tablet_id" = "10003", "backend_id" = "10001", "status" = "
    ↪ ok");
```

7.3.8.6.18 ADMIN SET REPLICA VERSION

描述

该语句用于设置指定副本的版本、最大成功版本、最大失败版本，目前仅用于在程序异常情况下，手动修复副本的版本，从而使得副本从异常状态恢复过来。

语法

```
ADMIN SET REPLICA VERSION PROPERTIES ("<key>"="<value>" [,...])
```

必选参数

** 1. "<key>"="<value>"**

key	value type	Notes
tablet_id	Int	需要执行操作的 tablet ID
backend_id	Int	指定 tablet 副本所在的 BE 节点 ID

可选参数

** 1. "<key>"="<value>"**

key	value type	Notes
version	Int	设置副本的版本。
last_success_version	Int	设置副本的最大成功版本。
last_failed_version	Int	设置副本的最大失败版本。

注意

- 如果指定的副本不存在，则会被忽略。
- 修改这几个数值，可能会导致后面数据读写失败，造成数据不一致，请谨慎操作！
- 修改之前先记录原来的值。修改完毕之后，对表进行读写验证，如果读写失败，请恢复原来的值！但可能会恢复失败！
- 严禁对正在写入数据的 tablet 进行操作！

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 清除 tablet 10003 在 BE 10001 上的副本状态失败标志。

```
ADMIN SET REPLICA VERSION PROPERTIES("tablet_id" = "10003", "backend_id" = "10001", "last_
    ↪ failed_version" = "-1");
```

- 设置 tablet 10003 在 BE 10001 上的副本版本号为 1004。

```
ADMIN SET REPLICA VERSION PROPERTIES("tablet_id" = "10003", "backend_id" = "10001", "version" =
    ↪  "1004");
```

7.3.8.6.19 SHOW REPLICA STATUS

描述

该语句用于展示一个表或分区的副本状态信息。

语法

```
SHOW REPLICA STATUS FROM [ <database_name>.]<table_name> [<partition_list>]
[where_clause]
```

其中：

```
partition_list
: PARTITION (<partition_name>[ , ... ])
```

其中：

```
where_clause
: WHERE <output_column_name> = <value>
```

必选参数

1. <table_name>

> 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如 My Object）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1. <db_name>

> 指定数据库的标识符（即名称），在其所在的集群（Cluster）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如 My Object）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <partition_list>

指定分区的标识符（即名称）列表，以逗号分割，在其所在的表（Table）中必须唯一。

标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。

标识符不能使用保留关键字。

有关更多详细信息，请参阅标识符要求和保留关键字。

3. `WHERE <output_column_name> = <value>`

指定输出过滤条件，output_column_name 必须在输出的字段列表中。

当 `output_column_name` 为 `STATUS` 时

`value` 可选值如下 - OK: replica 处于健康状态 - DEAD: replica 所在 Backend 不可用 - VERSION_ERROR: replica 数据版本有缺失 - SCHEMA_ERROR: replica 的 schema hash 不正确 - MISSING: replica 不存在

返回值

列名	类型	说明
TabletId	Int	tablet 的唯一标识符。
ReplicaId	Int	副本的唯一标识符。
BackendId	Int	副本所在的 BE（Backend）节点的 ID。
Version	Int	副本的当前版本号。
LastFailedVersion	Int	副本最后失败的版本号，-1 表示没有失败。
LastSuccessVersion	Int	副本最后成功的版本号。
CommittedVersion	Int	副本的提交版本号。
SchemaHash	Int	副本的 schema 哈希值。
VersionNum	Int	副本的版本数量。
IsBad	Boolean	指示副本是否处于坏状态（true/false）。
IsUserDrop	Boolean	指示副本是否被标记为用户删除。
State	String	副本的当前状态（例如：NORMAL）。
Status	String	副本的健康状态（例如：OK）。

权限控制

执行此 SQL 命令的用户必须至少拥有以下权限：

权限	对象	说明
Admin_priv	Database	执行数据库管理操作所需的权限，包括管理表、分区以及系统级命令等操作。

示例

- 查看表全部的副本状态

```
SHOW REPLICA STATUS FROM db1.tbl1;
```

```
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+-----
    ↪
| TabletId | ReplicaId | BackendId | Version | LastFailedVersion | LastSuccessVersion |
    ↪ CommittedVersion | SchemaHash | VersionNum | IsBad | IsUserDrop | State  | Status |
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+-----
    ↪
| 10145    | 10146     | 10009     | 14      | -1                | 14                 | 14
    ↪               | 182881783  | 1          | false | false      | NORMAL | OK     |
| 10147    | 10148     | 10009     | 14      | -1                | 14                 | 14
    ↪               | 182881783  | 1          | false | false      | NORMAL | OK     |
| 10149    | 10150     | 10009     | 14      | -1                | 14                 | 14
    ↪               | 182881783  | 1          | false | false      | NORMAL | OK     |
| 10151    | 10152     | 10009     | 14      | -1                | 14                 | 14
    ↪               | 182881783  | 1          | false | false      | NORMAL | OK     |
+----------+-----------+-----------+---------+-------------------+--------------------+------------------+-----
    ↪
```

- 查看表某个分区状态为 VERSION_ERROR 的副本

```
SHOW REPLICA STATUS FROM tbl1 PARTITION (p1, p2)
WHERE STATUS = "VERSION_ERROR";
```

- 查看表所有状态不健康的副本

```
SHOW REPLICA STATUS FROM tbl1
WHERE STATUS != "OK";
```

7.3.8.6.20 SHOW REPLICA DISTRIBUTION

描述

该语句用于展示一个表或分区副本分布状态

语法

```
SHOW REPLICA DISTRIBUTION FROM [ <database_name>.]<table_name> [<partition_list>]
[where_clause]
```

其中：

```
partition_list
  : PARTITION (<partition_name>[ , ... ])
```

其中：

```
where_clause
: WHERE <output_column_name> = <value>
```

必选参数

1. <table_name>

> 指定表的标识符（即名称），在其所在的数据库（Database）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如 My Object）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

可选参数

1. <db_name>

> 指定数据库的标识符（即名称），在其所在的集群（Cluster）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如 My Object）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

2. <partition_list>

> 指定分区的标识符（即名称）列表，以逗号分割，在其所在的表（Table）中必须唯一。
>
> 标识符必须以字母字符（如果开启 unicode 名字支持，则可以是任意语言文字的字符）开头，并且不能包含空格或特殊字符，除非整个标识符字符串用反引号括起来（例如`My Object`）。
>
> 标识符不能使用保留关键字。
>
> 有关更多详细信息，请参阅标识符要求和保留关键字。

返回值

列名	类型	说明
BackendId	Int	BE（Backend）节点的 ID，表示该副本所在的节点。
ReplicaNum	Int	该 BE 节点上的副本数量。
ReplicaSize	Int	该 BE 节点上所有副本占用的存储大小（单位：bytes）。
NumGraph	String	以可视化方式（> 符号）表示副本数量的比例。
NumPercent	String	以百分比表示副本数量的占比，例如 100.00%。
SizeGraph	String	以可视化方式（> 符号）表示副本存储大小的比例。
SizePercent	String	以百分比表示副本存储大小的占比，例如 100.00%。

示例

- 查看表的副本分布

```
SHOW REPLICA DISTRIBUTION FROM sell_user;
```

```
*************************** 1. row ***************************
BackendId: 10009
 ReplicaNum: 4
ReplicaSize: 8857
   NumGraph:
       ↪ >>>>
```

指定需要清理的 backend。如果不加 ON，默认清理所有 backend。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能进行 CLEAN TRASH 操作

示例

```
-- 清理所有 be 节点的垃圾数据。
ADMIN CLEAN TRASH;
```

```
-- 清理'192.168.0.1:9050'和'192.168.0.2:9050'的垃圾数据。
ADMIN CLEAN TRASH ON ("192.168.0.1:9050", "192.168.0.2:9050");
```

7.3.8.6.22 SHOW TRASH

描述

该语句用于查看 backend 内的垃圾数据占用空间。

语法：

```
SHOW TRASH [ON ("<be_host>:<be_heartbeat_port>" [, ...])];
```

可选参数

1. [ON ("<be_host>:<be_heartbeat_port>" [, ...])]

指定需要查看的 backend。如果不加 ON，默认查看所有 backend。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV 或 NODE_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 或 NODE_PRIV 权限才能进行 SHOW TRASH 操作

示例

1. 查看所有 be 节点的垃圾数据占用空间。

```
SHOW TRASH;
```

2. 查看’192.168.0.1:9050’ 的垃圾数据占用空间 (会显示具体磁盘信息)。

```
SHOW TRASH ON "192.168.0.1:9050";
```

7.3.9 回收站

7.3.9.1 SHOW CATALOG RECYCLE BIN

7.3.9.1.1 描述

该语句用于展示回收站中可回收的库、表或分区元数据信息。

7.3.9.1.2 语法

```
SHOW CATALOG RECYCLE BIN [ WHERE NAME [ = "<name>" | LIKE "<name_matcher>"] ]
```

7.3.9.1.3 可选参数

通过名称过滤

1. <name> > 库、表或分区名称。

通过模式匹配过滤

1. <name_matcher> > 库、表或分区名称的模式匹配。

7.3.9.1.4 返回值

列名	类型	说明
Type	String	元数据类型：Database、 Table、 Partition
Name	String	元数据名称
DbId	Bigint	database 对应的 id
TableId	Bigint	table 对应的 id
PartitionId	Bigint	table 对应的 id
DropTime	DateTime	元数据放入回收站的时间
DataSize	Bigint	数据量。如果元数据类型是 database, 该值包含了 database 下在回收站中的所有 table 和 partition 的数据量
RemoteDataSize	Decimal	远端存储 (hdfs 或对象存储) 的数据量。如果元数据类型是 database, 该值包含了 database 下在回收站中的所有 table 和 partition 的 remote storage 数据量

7.3.9.1.5 权限控制

权限	对象	说明
ADMIN_PRIV		

7.3.9.1.6 示例

1. 展示所有回收站元数据

```
    SHOW CATALOG RECYCLE BIN;
```

2. 展示回收站中名称’test’ 的元数据

```
    SHOW CATALOG RECYCLE BIN WHERE NAME = 'test';
```

2. 展示回收站中名称以 ‘test’ 开头的元数据

```
    SHOW CATALOG RECYCLE BIN WHERE NAME LIKE 'test%';
```

7.3.9.2 DROP CATALOG RECYCLE BIN

7.3.9.2.1 描述

该语句用于立即删除回收站中的数据库、表或者分区。

7.3.9.2.2 语法

```
DROP CATALOG RECYCLE BIN WHERE { 'DbId' = <db_id> | 'TableId' = <table_id> | 'PartitionId' = <
    ↪ partition_id> }
```

7.3.9.2.3 必选参数

根据 DbId 删除数据库

1. <db_id> > 要立即删除的数据库的 ID。

根据 TableId 删除表

1. <table_id> > 要立即删除的表的 ID。

3. 根据 PartitionId 删除分区

1. <partition_id> > 要立即删除的分区的 ID。

7.3.9.2.4 权限控制

权限	对象	说明
ADMIN_PRIV		

7.3.9.2.5 注意事项

- 当删除数据库、表或者分区时，回收站会在 catalog_trash_expire_second秒后将其删除（在 fe.conf 中设置）。此语句将立即删除它们。
- 'DbId'、'TableId' 和 'PartitionId' 大小写不敏感且不区分单引号和双引号。
- 当删除不在回收站中的数据库时，也会删除回收站中具有相同 DbId 的所有表和分区。只有在没有删除任何内容（数据库、表或分区）的情况下，它才会报错。当删除不在回收站中的表时，处理方法类似。
- 可以通过 SHOW CATALOG RECYCLE BIN 来查询当前可删除的元信息。

7.3.9.2.6 示例

1. 删除 DbId 为 example_db_id 的数据库、表和分区

```
DROP CATALOG RECYCLE BIN WHERE 'DbId' = example_db_id;
```

2. 删除 TableId 为 example_tbl_id 的表和分区

```
DROP CATALOG RECYCLE BIN WHERE 'TableId' = example_tbl_id;
```

3. 删除 id 为 p1_id 的分区

```
DROP CATALOG RECYCLE BIN WHERE 'PartitionId' = p1_id;
```

7.3.9.3 RECOVER

7.3.9.3.1 描述

该语句用于恢复之前删除的 database、table 或者 partition。

支持通过 name、id 来恢复指定的元信息，并且支持将恢复的元信息重命名。

7.3.9.3.2 语法：

```
RECOVER { DATABASE <db_name> [<db_id>] [AS <new_db_name>]
        | TABLE [<db_name>.]<table_name> [<table_id>] [AS <new_table_name>]
        | PARTITION <partition_name> [<partition_id>] FROM [<db_name>.]<table_name> [AS <new_
            ↪ partition_name>] }
```

7.3.9.3.3 必选参数

恢复数据库

1. <db_name> > 要恢复的数据库名称。

恢复数据表

1. <table_name> > 要恢复的表名称。

恢复分区

1. <partition_name> > 要恢复的数据库名称。

2. <table_name> > 要恢复的分区所在的表名称。

7.3.9.3.4 可选参数

恢复数据库

1. <db_id> > 要恢复的数据库 ID。

2. <new_db_name> > 恢复后新的数据库名称。

恢复数据表

1. <db_name> > 要恢复的表所在的数据库名称。

2. <table_id> > 要恢复的表 ID。

3. <new_table_name> > 恢复后新的数据表名称。

恢复分区

1. <partition_id> > 要恢复的分区 ID。

2. <db_name> > 要恢复分区所在表的数据库名称。

3. <new_partition_name> > 恢复后新的分区名称。

7.3.9.3.5 权限控制

权限	对象	说明
ADMIN_PRIV		

7.3.9.3.6 注意事项

- 该操作仅能恢复之前一段时间内删除的元信息。默认为 1 天。(可通过 fe.conf 中catalog_trash_expire_
  ↪ second参数配置)
- 如果恢复元信息时没有指定 id，则默认恢复最后一个删除的同名元数据。
- 可以通过 SHOW CATALOG RECYCLE BIN 来查询当前可恢复的元信息。

7.3.9.3.7 示例

1. 恢复名为 example_db 的 database

```
RECOVER DATABASE example_db;
```

2. 恢复名为 example_tbl 的 table

```
RECOVER TABLE example_db.example_tbl;
```

3. 恢复表 example_tbl 中名为 p1 的 partition

```
RECOVER PARTITION p1 FROM example_tbl;
```

4. 恢复 example_db_id 且名为 example_db 的 database

```
RECOVER DATABASE example_db example_db_id;
```

5. 恢复 example_tbl_id 且名为 example_tbl 的 table

```
RECOVER TABLE example_db.example_tbl example_tbl_id;
```

6. 恢复表 example_tbl 中 p1_id 且名为 p1 的 partition

```
RECOVER PARTITION p1 p1_id FROM example_tbl;
```

7. 恢复 example_db_id 且名为 example_db 的 database，并设定新名字 new_example_db

```
RECOVER DATABASE example_db example_db_id AS new_example_db;
```

8. 恢复名为 example_tbl 的 table，并设定新名字 new_example_tbl

```
RECOVER TABLE example_db.example_tbl AS new_example_tbl;
```

9. 恢复表 example_tbl 中 p1_id 且名为 p1 的 partition，并设定新名字 new_p1

```
RECOVER PARTITION p1 p1_id AS new_p1 FROM example_tbl;
```

7.3.10 函数

7.3.10.1 CREATE FUNCTION

7.3.10.1.1 描述

此语句用于创建一个自定义函数。

7.3.10.1.2 语法

```
CREATE [ GLOBAL ]
    [{AGGREGATE | TABLES | ALIAS }] FUNCTION <function_name>
    (<arg_type> [, ...])
    [ RETURNS <ret_type> ]
    [ INTERMEDIATE <inter_type> ]
    [ WITH PARAMETER(<param> [,...]) AS <origin_function> ]
    [ PROPERTIES ("<key>" = "<value>" [, ...]) ]
```

7.3.10.1.3 必选参数

1. <function_name>

> 如果 function_name 中包含了数据库名字，比如：db1.my_func，那么这个自定义函数会创建在对应的数据库中，否则这个函数将会创建在当前会话所在的数据库。新函数的名字与参数不能够与当前命名空间中已存在的函数完全相同，否则会创建失败。

2. <arg_type>

> 函数的输入参数类型。变长参数时可以使用，...来表示，如果是变长类型，那么变长部分参数的类型与最后一个非变长参数类型一致。

3. <ret_type>

> 函数的返回参数类型，对创建新的函数来说，是必填项。如果是给已有函数取别名则可不用填写该参数。

7.3.10.1.4 可选参数

1. GLOBAL

如果有此项，表示的是创建的函数是全局范围内生效。

2. AGGREGATE

如果有此项，表示的是创建的函数是一个聚合函数。

3. TABLES

如果有此项，表示的是创建的函数是一个表函数。

4. ALIAS

如果有此项，表示的是创建的函数是一个别名函数。

如果没有选择上述代表函数的参数，则表示创建的函数是一个标量函数

5. <inter_type>

用于表示聚合函数中间阶段的数据类型。

6. <param>

用于表示别名函数的参数，至少包含一个。

7. <origin_function>

用于表示别名函数对应的原始函数。

8. <properties>

- file: 表示的包含用户 UDF 的 jar 包，当在多机环境时，也可以使用 http 的方式下载 jar 包。这个参数是必须设定的。
- symbol: 表示的是包含 UDF 类的类名。这个参数是必须设定的
- type: 表示的 UDF 调用类型，默认为 Native，使用 Java UDF 时传 JAVA_UDF。
- always_nullable：表示的 UDF 返回结果中是否有可能出现 NULL 值，是可选参数，默认值为 true。

7.3.10.1.5 权限控制

执行此命令需要用户拥有 ADMIN_PRIV 权限。

7.3.10.1.6 示例

1. 创建一个自定义 UDF 函数，更多详细信息可以查看JAVA-UDF

```
CREATE FUNCTION java_udf_add_one(int) RETURNS int PROPERTIES (
    "file"="file:///path/to/java-udf-demo-jar-with-dependencies.jar",
    "symbol"="org.apache.doris.udf.AddOne",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

2. 创建一个自定义 UDAF 函数

```
CREATE AGGREGATE FUNCTION simple_sum(INT) RETURNS INT PROPERTIES (
    "file"="file:///pathTo/java-udaf.jar",
    "symbol"="org.apache.doris.udf.demo.SimpleDemo",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

3. 创建一个自定义 UDTF 函数

```
CREATE TABLES FUNCTION java-utdf(string, string) RETURNS array<string> PROPERTIES (
    "file"="file:///pathTo/java-udaf.jar",
    "symbol"="org.apache.doris.udf.demo.UDTFStringTest",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

4. 创建一个自定义别名函数，更多信息可以查看别名函数

```
CREATE ALIAS FUNCTION id_masking(INT) WITH PARAMETER(id) AS CONCAT(LEFT(id, 3), '****', RIGHT
    ↪ (id, 4));
```

5. 创建一个全局自定义别名函数

```
CREATE GLOBAL ALIAS FUNCTION id_masking(INT) WITH PARAMETER(id) AS CONCAT(LEFT(id, 3), '****'
    ↪ , RIGHT(id, 4));
```

7.3.10.2 DROP FUNCTION

7.3.10.2.1 描述

删除一个自定义函数。

7.3.10.2.2 语法

```
DROP [ GLOBAL ] <function_name> ( <arg_type> )
```

7.3.10.2.3 必选参数

1. <function_name>

> 指定要删除的函数的名字。
> 该函数名称需要与建立函数时的函数名称完全一致

2. <arg_type>

> 指定要删除函数的参数列表。
> 参数列表对应位置需要填写对应位置参数的数据类型

7.3.10.2.4 可选参数

1.GLOBAL

> GLOBAL 为选填项
> 若填写 GLOBAL 则为全局搜索该函数并删除
> 若不填写 GLOABL 则只在当前数据库下搜索该函数并删除

7.3.10.2.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	函数（自定义函数）	DROP 属于管理操作

7.3.10.2.6 注意事项

- 函数的名字、参数类型完全一致才能够被删除

7.3.10.2.7 示例

```
DROP FUNCTION my_add(INT, INT)
```

```
DROP GLOBAL FUNCTION my_add(INT, INT)
```

7.3.10.3 SHOW CREATE FUNCTION

7.3.10.3.1 描述

该语句用于展示用户自定义函数的创建语句

7.3.10.3.2 语法

```
SHOW CREATE [ GLOBAL ] FUNCTION <function_name>( <arg_type> ) [ FROM <db_name> ];
```

7.3.10.3.3 必选参数

1. <function_name>

需要查询创建语句的自定义函数的名称。

2. <arg_type>

需要查询创建语句的自定义函数的参数列表。
参数列表对应位置需要填写对应位置参数的数据类型

7.3.10.3.4 可选参数

1.GLOBAL

GLOBAL 为选填项
若填写 GLOBAL 则为全局搜索该函数
若不填写 GLOABL 则只在当前数据库下搜索该函数

2.<db_name>

FROM db_name 表示从指定的数据库中查询该自定义函数

7.3.10.3.5 返回值

列名	说明
SYMBOL	函数包名
FILE	jar 包路径
ALWAYS_NULLABLE	结果是否可以为 NULL
TYPE	函数类型

7.3.10.3.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SHOW_PRIV	函数	需要对该函数有 show 权限

7.3.10.3.7 示例

```
SHOW CREATE FUNCTION add_one(INT)
```

```
| Function Signature | Create Function
+--------------------+-------------------------------------------------------
| add_one(INT)       | CREATE FUNCTION add_one(INT) RETURNS INT PROPERTIES (
  "SYMBOL"="org.apache.doris.udf.AddOne",
  "FILE"="file:///xxx.jar",
  "ALWAYS_NULLABLE"="true",
  "TYPE"="JAVA_UDF"
  ); |
+--------------------+-------------------------------------------------------
```

7.3.10.4 DESC FUNCTION

7.3.10.4.1 描述

利用 `desc function table_valued_function` 获取对应表值函数的 Schema 信息。

7.3.10.4.2 语法

```
DESC FUNCTION <table_valued_function>
```

7.3.10.4.3 必选参数

1. `<table_valued_function>`: 表值函数的名字，如 CATALOGS。支持的表值函数列表，请参阅 “表值函数” 章节

7.3.10.4.4 示例

查询表值函数 CATALOGS 的信息：

```
DESC FUNCTION catalogs();
```

结果如下：

```
+-------------+--------+------+-------+---------+-------+
| Field       | Type   | Null | Key   | Default | Extra |
+-------------+--------+------+-------+---------+-------+
| CatalogId   | bigint | No   | false | NULL    | NONE  |
```

```
| CatalogName | text    | No    | false | NULL     | NONE   |
| CatalogType | text    | No    | false | NULL     | NONE   |
| Property    | text    | No    | false | NULL     | NONE   |
| Value       | text    | No    | false | NULL     | NONE   |
+-------------+---------+-------+-------+----------+--------+
```

7.3.10.5 SHOW FUNCTIONS

7.3.10.5.1 描述

查看数据库下所有的自定义与系统提供的函数。

7.3.10.5.2 语法

```
SHOW [ FULL ] [ BUILTIN ] FUNCTIONS [ { IN | FROM } <db> ]  [ LIKE '<function_pattern>' ]
```

7.3.10.5.3 变种语法

```
SHOW GLOBAL [ FULL ] FUNCTIONS [ LIKE '<function_pattern>' ]
```

7.3.10.5.4 必选参数

1. <function_pattern>

> 用来过滤函数名称的匹配模式规则

7.3.10.5.5 可选参数

1. FULL

> FULL 为选填项
> 若填写表示显示函数的详细信息。

2. BUILTIN

> BUILTIN 为选填项
> 若填写表示需要显示系统提供的函数

3. <db>

> db 为选填项
> 若填写表示在指定的数据库下查询

7.3.10.5.6 返回值

列名	说明
Signature	函数名与参数类型
Return Type	函数返回值的数据类型
Function Type	函数的类型
Intermediate Type	中间结果类型
Properties	函数的详细属性

7.3.10.5.7 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SHOW_PRIV	函数	需要对该函数有 show 权限

7.3.10.5.8 示例

```
show full functions in testDb
```

```
*************************** 1. row ***************************
Signature: my_add(INT,INT)
Return Type: INT
Function Type: Scalar
Intermediate Type: NULL
Properties: {"symbol":"_ZN9doris_udf6AddUdfEPNS_15FunctionContextERKNS_6IntValES4_","object_file
    ↪ ":"http://host:port/libudfsample.so","md5":"cfe7a362d10f3aaf6c49974ee0f1f878"}
*************************** 2. row ***************************
Signature: my_count(BIGINT)
Return Type: BIGINT
Function Type: Aggregate
Intermediate Type: NULL
Properties: {"object_file":"http://host:port/libudasample.so","finalize_fn":"_ZN9doris_
    ↪ udf13CountFinalizeEPNS_15FunctionContextERKNS_9BigIntValE","init_fn":"_ZN9doris_
    ↪ udf9CountInitEPNS_15FunctionContextEPNS_9BigIntValE","merge_fn":"_ZN9doris_
    ↪ udf10CountMergeEPNS_15FunctionContextERKNS_9BigIntValEPS2_","md5":"37
```

```
    ↪ d185f80f95569e2676da3d5b5b9d2f","update_fn":"_ZN9doris_udf11CountUpdateEPNS_15
    ↪ FunctionContextERKNS_6IntValEPNS_9BigIntValE"}
*************************** 3. row ***************************
Signature: id_masking(BIGINT)
Return Type: VARCHAR
Function Type: Alias
Intermediate Type: NULL
Properties: {"parameter":"id","origin_function":"concat(left(`id`, 3), `****`, right(`id`, 4))"}
```

```
show builtin functions in testDb like 'year%';
```

```
+---------------+
| Function Name |
+---------------+
| year          |
| years_add     |
| years_diff    |
| years_sub     |
+---------------+
```

```
show global full functions
```

```
*************************** 1. row ***************************
        Signature: decimal(ALL, INT, INT)
      Return Type: VARCHAR
    Function Type: Alias
Intermediate Type: NULL
       Properties: {"parameter":"col, precision, scale","origin_function":"CAST(`col` AS decimal
           ↪ (`precision`, `scale`))"}
*************************** 2. row ***************************
        Signature: id_masking(BIGINT)
      Return Type: VARCHAR
    Function Type: Alias
Intermediate Type: NULL
       Properties: {"parameter":"id","origin_function":"concat(left(`id`, 3), `****`, right(`id`,
           ↪  4))"}
```

```
show global functions
```

```
+---------------+
| Function Name |
+---------------+
| decimal       |
| id_masking    |
+---------------+
```

7.3.11 统计信息

7.3.11.1 ANALYZE

7.3.11.1.1 描述

该语句用于收集统计信息。可以针对表（可以指定具体列）或整个数据库进行列统计信息的收集。

7.3.11.1.2 语法

```
ANALYZE {TABLE <table_name> [ (<column_name> [, ...]) ] | DATABASE <database_name>}
    [ [ WITH SYNC ] [ WITH SAMPLE {PERCENT | ROWS} <sample_rate> ] ];
```

7.3.11.1.3 必选参数

1. <table_name>

> 指定的目标表。该参数与<database_name>参数必须且只能指定其中之一。

2. <database_name>

> 指定的目标数据库。该参数与<table_name>参数必须且只能指定其中之一。

7.3.11.1.4 可选参数

1. <column_name>

> 指定表的目标列。必须是 table_name 中存在的列，多个列名称用逗号分隔。

2. WITH SYNC

> 指定同步执行该 ANALYZE 语句。不指定时默认后台异步执行。

3. WITH SAMPLE {PERCENT | ROWS} <sample_rate>

指定使用抽样方式收集。当不指定时，默认为全量收集。为抽样参数，在 PERCENT 采样时指定抽样百分比，ROWS 采样时指定抽样行数。

7.3.11.1.5 返回值

列名	说明
Job_Id	收集作业的唯一 ID
Catalog_Name	Catalog 名
DB_Name	数据库名
Columns	收集的列列表

7.3.11.1.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	当执行 ANALYZE 时，需要拥有被查询的表的 SELECT_PRIV 权限

7.3.11.1.7 举例

1. 对 lineitem 表按照 10% 的比例采样收集统计数据：

```
ANALYZE TABLE lineitem WITH SAMPLE PERCENT 10;
```

2. 对 lineitem 表按采样 10 万行收集统计数据

```
ANALYZE TABLE lineitem WITH SAMPLE ROWS 100000;
```

7.3.11.2 ALTER STATS

7.3.11.2.1 描述

手动修改指定表中指定列的统计信息。请参阅统计信息章节

7.3.11.2.2 语法

```
ALTER TABLE <table_name>
  [ INDEX <index_name> ]
  MODIFY COLUMN <column_name>
  SET STATS (<column_stats>)
```

其中：

```
column_stats
  : -- column stats value
  ("key1" = "value1", "key2" = "value2" [...])
```

7.3.11.2.3 必选参数

1. <table_name>: 指定表的标识符（即名称）
2. <column_name>: 指定列标识符（即名称）。在不指定 index_name 的情况下，就是基表的列名称。
3. <column_stats>:

   要设置的统计信息值，以 key = value 的形式给出，key 和 value 需要用引号包裹，kv 对之间用逗号分隔。可以设置的统计信息包括：

   - row_count，总行数
   - ndv，列的基数
   - num_nulls，列的空值数量
   - data_size，列的总大小
   - min_value，列的最小值
   - max_value，列的最大值

   其中 row_count 是必须指定的，其他属性为可选项。如果不设置，该列的对应统计信息属性值就为空。

7.3.11.2.4 可选参数

1. <index_name>: 同步物化视图（请参阅“同步物化视图”章节）标识符（即名称）。一张表可以创建 0 到多个物化视图，如果需要设置某个物化视图中某一列的统计信息，需要使用 index_name 来制定物化视图的名称。不指定的情况下，设定的是基表中列的属性。

7.3.11.2.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ALTER_PRIV	表（Table）	

7.3.11.2.6 注意事项

用户手动对某张表注入统计信息后，这张表就不再参与统计信息的自动收集（请参阅“统计信息自动收集”章节），以免覆盖用户手动注入的信息。如果不再使用注入的统计信息，可以使用 drop stats 语句删掉已经注入的信息，这样可以让该表重新开启自动收集。

7.3.11.2.7 示例

- 给 Part 表的 p_parkey 列（基表列，因为没有指定 index_name）注入统计信息。

```
alter
    table part
    modify column p_partkey
    set stats ('row_count'='2.0E7', 'ndv'='2.0252576E7', 'num_nulls'='0.0', 'data_size'='8.0
        ↪ E7', 'min_value'='1', 'max_value'='20000000');
```

- 给 Part 表的 index1 物化视图的 col1 列（物化视图列，因为指定了 index_name）注入统计信息。

```
alter
    table part index index1
    modify column col1
    set stats ('row_count'='2.0E7', 'ndv'='2.0252576E7', 'num_nulls'='0.0', 'data_size'='8.0
        ↪ E7', 'min_value'='1', 'max_value'='20000000');
```

7.3.11.3 DROP STATS

7.3.11.3.1 描述

删除指定表和列的统计信息。如果不指定列名，则删除所有列的统计信息。

7.3.11.3.2 语法

```
DROP STATS <table_name> [ <column_names> ]
```

其中：

```
column_names
  :
  (<column_name>, [ <column_name>... ])
```

7.3.11.3.3 必选参数

<table_name>: 表的标识符（即名称）

7.3.11.3.4 可选参数

<column_names>: 列标识符列表（即名称列表）

7.3.11.3.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
DROP_PRIV	表（Table）	

7.3.11.3.6 示例

- 删除 table1 中所有列的统计信息

```
DROP STATS table1
```

- 删除 table1 中 col1 和 col2 的统计信息

```
DROP STATS table1 (col1, col2)
```

7.3.11.4 SHOW TABLE STATS

7.3.11.4.1 描述

该语句用来查看表的统计信息收集概况。

7.3.11.4.2 语法

```
SHOW TABLE STATS <table_name>;
```

7.3.11.4.3 必选参数

1. <table_name>

> 目标表名

7.3.11.4.4 可选参数

无

7.3.11.4.5 返回值

列名	说明
updated_rows	表当前更新行数
query_times	表被查询次数
row_count	表当前的总行数
updated_time	表上次更新时间
columns	收集过的列列表
trigger	收集触发方式
new_partition	是否有新分区首次导入数据
user_inject	用户是否手动注入了统计信息
enable_auto_analyze	这张表是否参与统计信息自动收集
last_analyze_time	上次收集时间

7.3.11.4.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	当执行 SHOW 时，需要拥有被查询的表的 SELECT_PRIV 权限

7.3.11.4.7 举例

1. 展示表 test1 的统计信息概况

```
SHOW TABLE STATS test1;
```

```
+--------------+-------------+-----------+---------------------+-----------------------+---------+-------------
    ↪
| updated_rows | query_times | row_count | updated_time        | columns               | trigger
    ↪  | new_partition | user_inject | enable_auto_analyze | last_analyze_time   |
+--------------+-------------+-----------+---------------------+-----------------------+---------+-------------
    ↪
| 0            | 0           | 100000    | 2025-01-17 16:46:31 | [test1:name, test1:id] | MANUAL
    ↪  | false         | false       | true                | 2025-02-05 12:17:41 |
+--------------+-------------+-----------+---------------------+-----------------------+---------+-------------
    ↪
```

7.3.11.5 SHOW COLUMN STATS

7.3.11.5.1 描述

该语句用来查看表的列统计信息。

7.3.11.5.2 语法

```
SHOW COLUMN [CACHED] STATS <table_name> [ (<column_name> [, ...]) ];
```

7.3.11.5.3 必选参数

1. <table_name>

需要展示列统计信息的表名。

7.3.11.5.4 可选参数

1. CACHED

显示 FE 缓存中的统计信息。不指定的时候默认显示统计信息表中持久化的信息。

2. <column_name>

指定需要显示的列名。列名在表中必须存在，多个列名之间用逗号分隔。如果不指定，默认显示所有列的信息。

7.3.11.5.5 返回值

列名	说明
column_name	列名
index_name	列所属的索引名
count	列的行数
ndv	列的基数
num_null	列的空值数
data_size	列的总数据量
avg_size_byte	列的平均字节数
min	列的最小值
max	列的最大值
method	收集方式
type	收集类型
trigger	触发方式

列名	说明
query_times	信息被查询次数
updated_time	信息更新时间
update_rows	上次收集时数据更新行数
last_analyze_row_count	上次收集时表的总行数
last_analyze_version	上次收集时表的版本值

7.3.11.5.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	当执行 SHOW 时，需要拥有被查询的表的 SELECT_PRIV 权限

7.3.11.5.7 举例

1. 展示表 test1 所有列的统计信息

```
SHOW COLUMN STATS test1;
```

```
+-------------+------------+----------+---------+----------+-----------+---------------+--------+--------+-------
    ↪
| column_name | index_name | count    | ndv     | num_null | data_size | avg_size_byte | min    |
    ↪  max    | method | type         | trigger | query_times | updated_time        | update_
    ↪ rows | last_analyze_row_count | last_analyze_version |
+-------------+------------+----------+---------+----------+-----------+---------------+--------+--------+-------
    ↪
| name        | test1      | 87775.0  | 48824.0 | 0.0      | 351100.0  | 4.0           | '0001' |
    ↪  'ffff' | FULL   | FUNDAMENTALS | MANUAL  | 0           | 2025-02-05 12:17:08 | 0
    ↪           | 100000                 | 3                    |
| id          | test1      | 100000.0 | 8965.0  | 0.0      | 351400.0  | 3.514         | 1000   |
    ↪  9999   | SAMPLE | FUNDAMENTALS | MANUAL  | 0           | 2025-02-05 12:17:41 | 0
    ↪           | 100000                 | 3                    |
+-------------+------------+----------+---------+----------+-----------+---------------+--------+--------+-------
    ↪
```

2. 展示表 test1 所有列在当前 FE 缓存中的统计信息

```
SHOW COLUMN CACHED STATS test1;
```

```
+-------------+------------+----------+---------+----------+-----------+---------------+--------+--------+-------
    ↪
```

```
| column_name | index_name | count    | ndv     | num_null | data_size | avg_size_byte | min    |
    ↪  max    | method | type         | trigger | query_times | updated_time        | update_
    ↪ rows | last_analyze_row_count | last_analyze_version |
+-------------+------------+----------+---------+----------+-----------+---------------+--------+-------+-------
    ↪
| name        | test1      | 87775.0  | 48824.0 | 0.0      | 351100.0  | 4.0           | '0001' |
    ↪  'ffff' | FULL   | FUNDAMENTALS | MANUAL  | 0           | 2025-02-05 12:17:08 | 0
    ↪           | 100000                 | 3                    |
| id          | test1      | 100000.0 | 8965.0  | 0.0      | 351400.0  | 3.514         | 1000   |
    ↪  9999   | SAMPLE | FUNDAMENTALS | MANUAL  | 0           | 2025-02-05 12:17:41 | 0
    ↪           | 100000                 | 3                    |
+-------------+------------+----------+---------+----------+-----------+---------------+--------+-------+-------
    ↪
```

7.3.11.6 DROP ANALYZE JOB

7.3.11.6.1 描述

删除指定的统计信息收集作业的历史记录。

7.3.11.6.2 语法

```
DROP ANALYZE JOB <job_id>
```

7.3.11.6.3 必选参数

1. <job_id>：指定作业的 id。可以通过 SHOW ANALYZE 获取作业的 job_id。详细用法，请参阅 SHOW ANALYZE 章节

7.3.11.6.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	

7.3.11.6.5 示例

删除 id 为 10036 的统计信息作业记录

```
DROP ANALYZE JOB 10036
```

7.3.11.7 KILL ANALYZE JOB

7.3.11.7.1 描述

停止正在后台执行的统计信息收集作业。

7.3.11.7.2 语法

```
KILL ANALYZE <job_id>
```

7.3.11.7.3 必选参数

<job_id>: 指定作业的 id。可以通过 SHOW ANALYZE 获取作业的 job_id。详细用法，请参阅 “SHOW ANALYZE” 章节

7.3.11.7.4 可选参数

无

7.3.11.7.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	

7.3.11.7.6 注意事项

已经执行完的作业无法停止

7.3.11.7.7 示例

停止 id 为 10036 的统计信息作业记录

```
kill analyze 10036
```

7.3.11.8 SHOW ANALYZE

7.3.11.8.1 描述

该语句用来查看统计信息收集作业的状态。

7.3.11.8.2 语法

```
SHOW [AUTO] ANALYZE [ <table_name> | <job_id> ]
    [ WHERE STATE = {"PENDING" | "RUNNING" | "FINISHED" | "FAILED"} ];
```

7.3.11.8.3 必选参数

无

7.3.11.8.4 可选参数

1. AUTO

> 展示自动作业信息。如不指定，默认显示手动作业信息。

2. <table_name>

> 表名。指定后可查看该表对应的作业信息。不指定时默认返回所有表的作业信息。

3. <job_id>

> 统计信息作业 ID，执行 ANALYZE 异步收集时得到。不指定 id 时此命令返回所有作业信息。

4. WHERE STATE = {"PENDING" | "RUNNING" | "FINISHED" | "FAILED"}

> 作业状态过滤条件。如不指定，默认显示所有状态的作业信息。

7.3.11.8.5 返回值

列名	说明
job_id	收集作业的唯一 ID
catalog_name	Catalog 名
db_name	数据库名
tbl_name	表名
col_name	收集的列列表
job_type	作业类型
analysis_type	分析类型
message	错误信息
last_exec_time_in_ms	上次收集完成时间

列名	说明
state	作业状态
progress	作业进度
schedule_type	调度类型
start_time	作业开始时间
end_time	作业结束时间
priority	作业优先级
enable_partition	是否开启分区收集

7.3.11.8.6 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
SELECT_PRIV	表（Table）	当执行 SHOW 时，需要拥有被查询的表的 SELECT_PRIV 权限

7.3.11.8.7 举例

1. 通过表名展示作业

```
SHOW ANALYZE test1 WHERE STATE="FINISHED";
```

```
+---------------+-------------+---------+----------+----------------------+----------+---------------+---------
    ↪
| job_id        | catalog_name | db_name | tbl_name | col_name             | job_type | analysis
    ↪ _type | message | last_exec_time_in_ms | state    | progress
    ↪                                          | schedule_type | start_time          | end_
    ↪ time            | priority | enable_partition |
+---------------+-------------+---------+----------+----------------------+----------+---------------+---------
    ↪
| 1737454119144 | internal     | test    | test1    | [test1:name,test1:id] | MANUAL   |
    ↪ FUNDAMENTALS  |         | 2025-01-21 18:10:11  | FINISHED | 2 Finished  |  0 Failed  |  0
    ↪  In Progress  |  2 Total | ONCE          | 2025-01-21 18:10:10 | 2025-01-21 18:10:11 |
    ↪ MANUAL   | false            |
| 1738725887879 | internal     | test    | test1    | [test1:name,test1:id] | MANUAL   |
    ↪ FUNDAMENTALS  |         | 2025-02-05 11:26:15  | FINISHED | 2 Finished  |  0 Failed  |  0
    ↪  In Progress  |  2 Total | ONCE          | 2025-02-05 11:26:15 | 2025-02-05 11:26:15 |
    ↪ MANUAL   | false            |
| 1738725887890 | internal     | test    | test1    | [test1:name,test1:id] | MANUAL   |
    ↪ FUNDAMENTALS  |         | 2025-02-05 12:17:09  | FINISHED | 2 Finished  |  0 Failed  |  0
    ↪  In Progress  |  2 Total | ONCE          | 2025-02-05 12:17:08 | 2025-02-05 12:17:09 |
    ↪ MANUAL   | false            |
```

```
| 1738725887895 | internal     | test    | test1    | [test1:id]            | MANUAL    |
    ↪ FUNDAMENTALS  |          | 2025-02-05 12:17:24  | FINISHED | 1 Finished  |  0 Failed  |  0
    ↪  In Progress  |  1 Total | ONCE          | 2025-02-05 12:17:23 | 2025-02-05 12:17:24 |
    ↪ MANUAL    | false            |
| 1738725887903 | internal     | test    | test1    | [test1:id]            | MANUAL    |
    ↪ FUNDAMENTALS  |          | 2025-02-05 12:17:42  | FINISHED | 1 Finished  |  0 Failed  |  0
    ↪  In Progress  |  1 Total | ONCE          | 2025-02-05 12:17:41 | 2025-02-05 12:17:42 |
    ↪ MANUAL    | false            |
+---------------+-------------+---------+----------+----------------------+----------+---------------+---------
    ↪
```

2. 通过作业 ID 展示作业

```
show analyze 1738725887903;
```

```
+---------------+-------------+---------+----------+------------+----------+---------------+---------+---------
    ↪
| job_id        | catalog_name | db_name | tbl_name | col_name   | job_type | analysis_type |
    ↪ message | last_exec_time_in_ms | state    | progress
    ↪                                              | schedule_type | start_time          | end_
    ↪ time            | priority | enable_partition |
+---------------+-------------+---------+----------+------------+----------+---------------+---------+---------
    ↪
| 1738725887903 | internal     | test    | test1    | [test1:id] | MANUAL   | FUNDAMENTALS  |
    ↪         | 2025-02-05 12:17:42  | FINISHED | 1 Finished  |  0 Failed  |  0 In Progress  |
    ↪  1 Total | ONCE          | 2025-02-05 12:17:41 | 2025-02-05 12:17:42 | MANUAL    | false
    ↪            |
+---------------+-------------+---------+----------+------------+----------+---------------+---------+---------
    ↪
```

7.3.12 集群管理

7.3.12.1 物理实例管理

7.3.12.1.1 ADD FOLLOWER

描述

该语句是增加 FRONTEND 的 FOLLOWER 角色的节点，(仅管理员使用 !)

语法：

```
ALTER SYSTEM ADD FOLLOWER "<follower_host>:<edit_log_port>"
```

必选参数

1. <follower_host>

可以是 FE 节点的主机名或 IP 地址

2. <edit_log_port>

FE 节点的 bdbje 通信端口，默认为 9010

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 在添加新的 FOLLOWER 节点之前，确保节点已正确配置。
2. 在添加 FOLLOWER 节点之前，确保集群中 FOLLOWER 节点在新增之后数量为奇数个。
3. 添加 FOLLOWER 节点后，使用SHOW FRONTENDS命令验证它们是否已成功添加并处于正常状态。

示例

1. 添加一个 FOLLOWER 节点

```
ALTER SYSTEM ADD FOLLOWER "host_ip:9010"
```

此命令向集群添加一个 FOLLOWER 节点（IP host_ip，端口 9010）

7.3.12.1.2 DROP FOLLOWER

描述

该语句是删除 FRONTEND 的 FOLLOWER 角色的节点，(仅管理员使用！)

语法

```
ALTER SYSTEM DROP FOLLOWER "<follower_host>:<edit_log_port>"
```

必选参数

1. <follower_host>

可以是 FE 节点的主机名或 IP 地址

2. <edit_log_port>

FE 节点的 bdbje 通信端口，默认为 9010

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 在删除 FOLLOWER 节点之前，确保需要下线的节点不是 Master 节点。
2. 在删除 FOLLOWER 节点之前，确保集群中 FOLLOWER 节点在下线之后数量为奇数个。
3. 删除 FOLLOWER 节点后，使用SHOW FRONTENDS命令验证它们是否已成功删除。

示例

1. 删除一个 FOLLOWER 节点

```
ALTER SYSTEM DROP FOLLOWER "host_ip:9010"
```

此命令是删除集群中的一个 FOLLOWER 节点（IP host_ip，端口 9010）

7.3.12.1.3 ADD OBSERVER

描述

该语句是增加 FRONTEND 的 OBSERVER 角色的节点，（仅管理员使用！）

语法

```
ALTER SYSTEM ADD OBSERVER "<observer_host>:<edit_log_port>"
```

必选参数

1. <observer_host>

可以是 FE 节点的主机名或 IP 地址

2. <edit_log_port>

FE 节点的 bdbje 通信端口，默认为 9010

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 在添加新的 OBSERVER 节点之前，确保节点已正确配置。
2. 添加 OBSERVER 节点后，使用SHOW FRONTENDS命令验证它们是否已成功添加并处于正常状态。

示例

1. 添加一个 OBSERVER 节点

```
ALTER SYSTEM ADD OBSERVER "host_ip:9010"
```

此命令向集群添加一个 OBSERVER 节点（IP host_ip，端口 9010）

7.3.12.1.4 DROP OBSERVER

描述

该语句是删除 FRONTEND 的 OBSERVER 角色的节点，(仅管理员使用！)

语法

```
ALTER SYSTEM DROP OBSERVER "<observer_host>:<edit_log_port>"
```

必选参数

1. <observer_host>

可以是 FE 节点的主机名或 IP 地址

2. <edit_log_port>

FE 节点的 bdbje 通信端口，默认为 9010

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 删除 OBSERVER 节点后，使用SHOW FRONTENDS命令验证它们是否已成功删除。

示例

1. 删除一个 OBSERVER 节点

```
ALTER SYSTEM DROP OBSERVER "host_ip:9010"
```

此命令是删除集群中的一个 OBSERVER 节点（IP host_ip，端口 9010）

7.3.12.1.5 SET FRONTEND CONFIG

描述

该语句用于设置集群的配置项（当前仅支持设置 FE 的配置项）。

语法：

```
ADMIN SET \{ALL FRONTENDS | FRONTEND\} CONFIG ("<fe_config_key>" = "<fe_config_value>")
```

必选参数

{ALL FRONTENDS | FRONTEND}

ALL FRONTENDS：代表 Doris 集群中的所有 FE 节点 FRONTEND：代表当前连接的 FE 节点，即用户当前交互的 FE 节点

可选参数

需要修改的 <fe_config_key>、 <fe_config_value> 可通过 SHOW FRONTEND CONFIG 命令查看

提示

- 使用 ALL 关键字后配置参数将应用于所有 FE(除 master_only 参数外)。
- 该语法不会持久化修改的配置，FE 重启后，修改的配置失效。如需持久化，需要在 fe.conf 中同步添加配置项。

示例

1. 设置 disable_balance 为 true

```
ADMIN SET FRONTEND CONFIG ("disable_balance" = "true");
```

2. 设置所有 FE 节点的 disable_balance 为 true

```
ADMIN SET ALL FRONTENDS CONFIG ("disable_balance" = "true");
```

7.3.12.1.6 SHOW FRONTEND CONFIG

描述

该语句用于展示当前集群的配置（当前仅支持展示 FE 的配置项）

语法：

```
SHOW FRONTEND CONFIG [LIKE "<pattern>"];
```

可选参数

<pattern> > 可以包含普通字符和通配符的字符串

返回值

列名		说明
Value		配置项值
Type		配置项类型
IsMutable		是否可以通过 ADMIN SET CONFIG 命令设置
MasterOnly	3237	是否仅适用于 Master FE
Comment		配置项说明

示例

1. 查看当前 FE 节点的配置

```
SHOW FRONTEND CONFIG;
```

2. 使用 like 谓词搜索当前 Fe 节点的配置

```
SHOW FRONTEND CONFIG LIKE '%check_java_version%';
```

```
+--------------------+-------+---------+-----------+------------+---------+
| Key                | Value | Type    | IsMutable | MasterOnly | Comment |
+--------------------+-------+---------+-----------+------------+---------+
| check_java_version | true  | boolean | false     | false      |         |
+--------------------+-------+---------+-----------+------------+---------+
```

7.3.12.1.7 MODIFY FRONTEND HOSTNAME

描述

修改 FRONTEND（后续使用简称 FE）的属性。当前，此命令仅能修改 FE 的主机名（HOSTNAME）。当集群中的某一个 FE 实例运行的主机需要变更主机名时，可以使用此命令更改此 FE 在集群中注册的主机名，使其可以继续正常运行。

此命令只用于将 DORIS 集群转变为 FQDN 方式部署。有关 FQDN 部署的细节，请参阅“FQDN”章节。

语法

```
ALTER SYSTEM MODIFY FRONTEND "<frontend_hostname_port>" HOSTNAME "<frontend_new_hostname>"
```

必选参数

1. <frontend_hostname_port>: 需要变更主机名的 FE 注册的 hostname 和 edit log port。可以通过 SHOW FRONTENDS 命令查看集群中所有 FE 的相关信息。详细用法请参阅“SHOW FRONTENDS”章节。
2. <frontend_new_hostname>: FE 的新主机名。

权限控制

执行此 SQL 命令的用户必须至少具有 NOD_PRIV 权限。

示例

将集群中的一个 FE 实例的 hostname，从 10.10.10.1 变为 172.22.0.1：

```
ALTER SYSTEM
MODIFY FRONTEND "10.10.10.1:9010"
HOSTNAME "172.22.0.1"
```

7.3.12.1.8 SHOW FRONTENDS

描述

该语句用于查看 FE 节点的基本状态信息。

语法

```
SHOW FRONTENDS
```

返回值

列名	说明
Name	当前 FE 在此 Doris 中的名称，该名称通常是一个以fe为前缀的随机字符串
Host	当前 FE 的 IP 地址或主机名
EditLogPort	当前 FE 的 bdbje 通信端口
HttpPort	当前 FE 的 http 通信端口
QueryPort	当前 FE 的 MySQL 协议通信端口
RpcPort	当前 FE 的 thrift RPC 通信端口
ArrowFlightSqlPort	当前 FE 的 ArrowFlight 协议通信端口
Role	当前 FE 的角色，可能的值有 FOLLOWER 和 OBSERVER
IsMaster	当前 FE 是否被选举为 Master
ClusterId	当前 Doris 集群的 ID，通常为一个随机生成的数字
Join	用于表示当前 FE 节点是否成功加入当前 Doris 集群
Alive	当前 FE 是否存活
ReplayedJournalId	当前 FE 已经回放的最大元数据日志 ID
LastStartTime	当前 FE 启动的时间戳
LastHeartbeat	当前 FE 上一次成功发送心跳的时间戳
IsHelper	当前 FE 是否为 bdbje 中的 helper 节点
ErrMsg	当前 FE 心跳失败时的错误信息
Version	当前 FE 的版本信息
CurrentConnected	当前客户端链接是否连接了当前 FE 节点

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV		

注意事项

如果需要对查询结果进行进一步的过滤，可以使用表值函数 frontends()。SHOW FRONTENDS 与下面语句等价：

```
SELECT * FROM FRONTENDS();
```

示例

```
SHOW FRONTENDS
```

```
+-----------------------------------------+-----------+-------------+----------+-----------+---------+-----------
    ↪
| Name                                    | Host      | EditLogPort | HttpPort | QueryPort |
    ↪ RpcPort | ArrowFlightSqlPort | Role     | IsMaster | ClusterId | Join | Alive |
    ↪ ReplayedJournalId | LastStartTime       | LastHeartbeat       | IsHelper | ErrMsg |
    ↪ Version                      | CurrentConnected |
+-----------------------------------------+-----------+-------------+----------+-----------+---------+-----------
    ↪
| fe_65a0c6f0_b31f_42ac_bd20_26d851299f1a | 127.0.0.1 | 9010        | 8030     | 9030      | 9020
    ↪     | 10030               | FOLLOWER | true     | 840241689 | true | true  | 302891
    ↪            | 2025-01-20 02:11:39 | 2025-01-21 09:48:36 | true     |        | doris-2.1.7-
    ↪ rc03-443e87e203 | Yes               |
+-----------------------------------------+-----------+-------------+----------+-----------+---------+-----------
    ↪
```

7.3.12.1.9 SHOW FRONTENDS DISKS

描述

该语句用于查看 FE 节点的重要目录如：元数据、日志、审计日志、临时目录对应的磁盘信息

语法

```
SHOW FRONTENDS DISKS;
```

返回值

列名	说明
Name	该 FE 节点在 bdbje 中的名称
Host	该 FE 节点的 IP
DirType	要展示的目录类型，分别有四种类型：meta、log、audit-log、temp、deploy
Dir	要展示的目录类型的目录
FileSystem	要展示的目录类型所在的 linux 系统的文件系统
Capacity	文件系统的容量
Used	文件系统已用大小
Available	文件系统剩余容量
UseRate	文件系统使用容量占比
MountOn	文件系统挂载目录

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV 或 NODE_PRIV		

注意事项

如果需要对查询结果进行进一步的过滤，可以使用表值函数 frontends_disks()。SHOW FRONTENDS DISKS 与下面语句等价：

```
SELECT * FROM FRONTENDS_DISKS();
```

示例

```
SHOW FRONTENDS DISKS;
```

```
+-----------------------------------------+-------------+-----------+--------------------------------+----------
    ↪
| Name                                    | Host        | DirType   | Dir
    ↪                                | Filesystem | Capacity | Used | Available | UseRate |
    ↪ MountOn    |
+-----------------------------------------+-------------+-----------+--------------------------------+----------
    ↪
| fe_a1daac68_5ec0_477c_b5e8_f90a33cdc1bb | 10.xx.xx.90 | meta      | /home/disk/output/fe/doris-
    ↪ meta | /dev/sdf1  | 7T       | 2T   | 4T        | 36%     | /home/disk |
| fe_a1daac68_5ec0_477c_b5e8_f90a33cdc1bb | 10.xx.xx.90 | log       | /home/disk/output/fe/log
    ↪        | /dev/sdf1  | 7T       | 2T   | 4T        | 36%     | /home/disk |
| fe_a1daac68_5ec0_477c_b5e8_f90a33cdc1bb | 10.xx.xx.90 | audit-log | /home/disk/output/fe/log
    ↪        | /dev/sdf1  | 7T       | 2T   | 4T        | 36%     | /home/disk |
| fe_a1daac68_5ec0_477c_b5e8_f90a33cdc1bb | 10.xx.xx.90 | temp      | /home/disk/output/fe/temp_
    ↪ dir    | /dev/sdf1  | 7T       | 2T   | 4T        | 36%     | /home/disk |
| fe_a1daac68_5ec0_477c_b5e8_f90a33cdc1bb | 10.xx.xx.90 | deploy    | /home/disk/output/fe
    ↪            | /dev/sdf1  | 7T       | 2T   | 4T        | 36%     | /home/disk |
+-----------------------------------------+-------------+-----------+--------------------------------+----------
    ↪
```

7.3.12.1.10 ADD BACKEND

描述

ADD BACKEND 命令用于向 Doris 集群中添加一个或多个 BE 节点。此命令允许管理员指定新 BE 节点的主机和端口，以及可选的属性来配置它们的行为。

语法

```
ALTER SYSTEM ADD BACKEND "<host>:<heartbeat_port>"[,"<host>:<heartbeat_port>"...] [PROPERTIES ("<
    ↪ key>"="<value>" [, ...] )]
```

必选参数

1. <host>

> 可以是 BE 节点的主机名或 IP 地址

2. <heartbeat_port>

> BE 节点的心跳端口，默认为 9050

可选参数

1. PROPERTIES (“key”=“value”, …)

> 一组键值对，用于定义 BE 节点的附加属性。这些属性可用于自定义正在添加的 BE 的配置。可用属性包括：- tag.location：存算一体模式下用于指定 BE 节点所属的资源组。- tag.compute_group_name：存算分离模式下用于指定 BE 节点所属的计算组。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 在添加新的 BE 节点之前，确保节点已正确配置并运行。
2. 使用Resource Group可以帮助您更好地管理和组织集群中的 BE 节点。
3. 添加多个 BE 节点时，可以在一个命令中指定它们，以提高效率。
4. 添加 BE 节点后，使用SHOW BACKENDS命令验证它们是否已成功添加并处于正常状态。
5. 考虑在不同的物理位置或机架上添加 BE 节点，以提高集群的可用性和容错能力。
6. 定期检查和平衡集群中的负载，确保新添加的 BE 节点得到适当利用。

示例

1. 不带附加属性添加 BE

```
ALTER SYSTEM ADD BACKEND "192.168.0.1:9050,192.168.0.2:9050";
```

此命令向集群添加两个 BE 节点：

- 192.168.0.1，端口 9050
- 192.168.0.2，端口 9050 未指定附加属性，因此将应用默认设置。

2. 存算一体模式下，添加指定资源组的 BE

```
ALTER SYSTEM ADD BACKEND "doris-be01:9050" PROPERTIES ("tag.location" = "groupb");
```

此命令将单个 BE 节点（主机名 doris-be01，端口 9050）添加到集群中的资源组groupb。

3. 存算分离模式下，添加指定计算组的 BE

```
ALTER SYSTEM ADD BACKEND "192.168.0.3:9050" PROPERTIES ("tag.compute_group_name" = "cloud
    ↪ _groupc");
```

此命令将单个 BE 节点（IP 192.168.0.3，端口 9050）添加到集群中的计算组cloud_groupc。

7.3.12.1.11 DROP BACKEND

描述

该语句用于将 BE 节点从 Doris 集群中删除。

语法

```
ALTER SYSTEM DROP BACKEND "<be_identifier>" [, "<be_identifier>" ... ]
```

其中：

```
be_identifier
  : "<be_host>:<be_heartbeat_port>"
  | "<backend_id>"
```

必选参数

1.

可以是 BE 节点的主机名或 IP 地址

2.

BE 节点的心跳端口，默认为 9050

3.

> BE 节点的 ID

> <be_host>、<be_heartbeat_port>及<backend_id>均可通过 SHOW BACKENDS 语句查询获得。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 不推荐使用该命令下线 BE，该命令会直接将 BE 直接从集群中删去，当前节点的数据并不会负载均衡到其他 BE 节点，如果集群存在单副本的表，那么就有可能出现数据丢失的情况。更好的做法是使用 DECOMMISSION BACKEND 命令优雅下线 BE。
2. 由于此操作是高危操作，因此当直接运行此命令时：

```
ALTER SYSTEM DROP BACKEND "127.0.0.1:9050";
```

```
ERROR 1105 (HY000): errCode = 2, detailMessage = It is highly NOT RECOMMENDED to use DROP
    ↪ BACKEND stmt.It is not safe to directly drop a backend. All data on this backend
    ↪ will be discarded permanently. If you insist, use DROPP instead of DROP
```

会出现以上提示信息，如果您明白您当前所做的事情，可将DROP关键字替换成DROPP，并继续下去：

```
ALTER SYSTEM DROPP BACKEND "127.0.0.1:9050";
```

示例

1. 根据 BE 的 Host 和 HeartbeatPort 从集群中删除两个节点

```
ALTER SYSTEM DROPP BACKEND "192.168.0.1:9050", "192.168.0.2:9050";
```

2. 根据 BE 的 ID 从集群中删除一个节点

```
ALTER SYSTEM DROPP BACKEND "10002";
```

7.3.12.1.12 MODIFY BACKEND

描述

该语句用于修改 BE 节点属性，修改 BE 节点属性后，会影响到当前的节点的查询、写入和数据分布。以下是支持修改的属性：

属性	影响
tag.location	BE 的标签名，默认值为default。修改后，会影响到同一标签组内的 BE 数据均衡，以及在建表时，数据分布的 BE 节点。更多信息可参考Resource Group
disable_query	是否禁用查询，默认为false。设置为true后，将不会再有新的查询请求规划到这台 BE 节点上。
disable_load	是否禁用导入，默认为false。设置为true后，将不会再有新的导入请求规划到这台 BE 节点上。

> 存算分离模式暂不支持此命令。

语法

```
ALTER SYSTEM MODIFY BACKEND <be_identifier> [, <be_identifier> [...] ]
SET (
     "<key>" = "<value>"
)
```

其中：

```
be_identifier
  : "<be_host>:<be_heartbeat_port>"
  | "<backend_id>"
```

必选参数

1.

> 可以是 BE 节点的主机名或 IP 地址

2.

> BE 节点的心跳端口，默认为 9050

3.

BE 节点的 ID

<be_host>、<be_heartbeat_port>及<backend_id>均可通过 SHOW BACKENDS 语句查询获得。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

由于此操作是针对整个 BE 级别的，影响面较广，如果操作不慎，可能会影响到整个集群的正常查询、导入甚至是建表操作。请谨慎操作。

示例

1. 修改 BE 的资源标签

```
ALTER SYSTEM MODIFY BACKEND "127.0.0.1:9050" SET ("tag.location" = "group_a");
```

2. 修改 BE 的查询禁用属性

```
ALTER SYSTEM MODIFY BACKEND "10002" SET ("disable_query" = "true");
```

3. 修改 BE 的导入禁用属性

```
ALTER SYSTEM MODIFY BACKEND "127.0.0.1:9050" SET ("disable_load" = "true");
```

7.3.12.1.13 SHOW BACKENDS

描述

该语句用于查看 BE 节点的基本状态信息。

语法

```
SHOW BACKENDS
```

返回值

列名	说明
BackendId	当前 BE 的 ID
Host	当前 BE 的 IP 地址或主机名
HeartbeatPort	当前 BE 的心跳服务通信端口
BePort	当前 BE 的 thrift RPC 通信端口
HttpPort	当前 BE 的 http 通信端口
BrpcPort	当前 BE 的 bRPC 通信端口
ArrowFlightSqlPort	当前 BE 的 ArrowFlight 协议通信端口
LastStartTime	当前 BE 启动的时间戳
LastHeartbeat	当前 BE 上一次成功发送心跳的时间戳
Alive	当前 BE 是否存活
SystemDecommissioned	该值为 true 时，表示当前 BE 节点正在安全下线中
TabletNum	当前 BE 上存储的数据分片数量
DataUsedCapacity	当前 BE 数据所占用的磁盘空间
TrashUsedCapacity	当前 BE 垃圾回收站中数据所占用的磁盘空间
AvailCapacity	当前 BE 可用的磁盘空间
TotalCapacity	当前 BE 总的磁盘空间，TotalCapacity = AvailCapacity + TrashUsedCapacity + DataUsedCapacity + 其他非用户数据文件占用空间
UsedPct	当前 BE 所有磁盘总的已使用量百分比
MaxDiskUsedPct	当前 BE 所有磁盘的已使用量百分比中最大的一个
RemoteUsedCapacity	当前 BE 在使用了冷热分层功能后，上传到远端存储的数据占用空间
Tag	当前 BE 的标签信息，以 JSON 格式展示，不同部署模式下的保存的标签信息不同，存算一体模式下保存当前 BE 资源组名称，存算分离模式下保存一些额外的信息
ErrMsg	当前 BE 心跳失败时的错误信息
Version	当前 BE 的版本信息
Status	当前 BE 的一些状态信息，以 JSON 格式展示，包括：上一次成功上报 tablet 的时间、上一次 StreamLoad 的时间、是否允许查询、是否允许导入等，需要注意的是，不同版本保存的信息会有些许差异
HeartbeatFailureCounter	当前 BE 连续失败的心跳次数，如果次数超过 FE Master 配置`max_backend_heartbeat_failure_tolerance_count`（默认值为 1），则 `Alive` 字段会置为 false
NodeRole	当前 BE 的角色，有两种：`mix`是默认的角色，`computation`表示当前节点只用于联邦分析查询

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV		

注意事项

如果需要对查询结果进行进一步的过滤，可以使用表值函数 backends()。`SHOW BACKENDS` 与下面语句等价：

```
SELECT * FROM BACKENDS();
```

示例

```
SHOW BACKENDS;
```

```
+-----------+-----------+---------------+--------+----------+----------+--------------------+-----------------
    ↪
| BackendId | Host      | HeartbeatPort | BePort | HttpPort | BrpcPort | ArrowFlightSqlPort |
    ↪ LastStartTime       | LastHeartbeat       | Alive | SystemDecommissioned | TabletNum |
    ↪ DataUsedCapacity | TrashUsedCapacity | AvailCapacity | TotalCapacity | UsedPct |
    ↪ MaxDiskUsedPct | RemoteUsedCapacity | Tag                      | ErrMsg | Version
    ↪                      | Status
    ↪
    ↪ | HeartbeatFailureCounter | NodeRole |
+-----------+-----------+---------------+--------+----------+----------+--------------------+-----------------
    ↪
| 10002     | 127.0.0.1 | 9050          | 9060   | 8040     | 8060     | 10040              |
    ↪ 2025-01-20 02:11:39 | 2025-01-21 11:52:40 | true  | false                | 281       |
    ↪ 9.690 MB         | 0.000             | 10.505 GB     | 71.750 GB     | 85.36 % | 85.36 %
    ↪        | 0.000              | {"location" : "default"} |        | doris-2.1.7-rc03-443
    ↪ e87e203 | {"lastSuccessReportTabletsTime":"2025-01-21 11:51:59","lastStreamLoadTime
    ↪ ":1737460114345,"isQueryDisabled":false,"isLoadDisabled":false} | 0
    ↪                        | mix      |
+-----------+-----------+---------------+--------+----------+----------+--------------------+-----------------
    ↪
```

7.3.12.1.14 SHOW BACKEND CONFIG

描述

展示 BACKEND（即 BE）的配置项和其当前值。

语法

```
[ ADMIN ] SHOW BACKEND CONFIG [ LIKE <config_key_pattern> ] [ FROM <backend_id> ]
```

可选参数

> 提供一个通配符模式，用于匹配 BE 配置项。匹配规则与 LIKE 表达式相同。书写规则请参考“匹配表达式”章节。

BE 的 ID。用于查看指定 ID 的 BE 的配置。BE 的 ID 可以通过 SHOW BACKENDS 命令获得。具体请参阅 “SHOW BACKENDS” 命令

返回值（Return Value）

- BackendId: BE 的 ID
- Host: BE 的主机地址
- Key: 配置项的名称
- Value: 配置项对应的值
- Type: 配置值的类型

权限控制（Access Control Requirements）

执行此 SQL 命令的用户必须至少具有 ADMIN_PRIV 权限。

示例（Examples）

查询所有配置项

```
SHOW BACKEND CONFIG
```

结果为

```
+--
    ↪ ---------+--------------+--------------------------------+-------+-------------------------+-----------+
    ↪
| BackendId | Host         | Key                            | Value | Type                    |
    ↪ IsMutable |
+--
    ↪ ---------+--------------+--------------------------------+-------+-------------------------+-----------+
    ↪
| 12793     | 172.16.123.1 | LZ4_HC_compression_level       | 9     | int64_t                 |
    ↪ true      |
| 12793     | 172.16.123.1 | agent_task_trace_threshold_sec | 2     | int32_t                 |
    ↪ true      |
...
| 12794     | 172.16.123.2 | zone_map_row_num_threshold     | 20    | int32_t                 |
    ↪ true      |
+--
    ↪ ---------+--------------+--------------------------------+-------+-------------------------+-----------+
    ↪
```

查询指定 ID 的 BE 的配置项

```
SHOW BACKEND CONFIG FROM 12793
```

结果为

```
+--
    ↪ ---------+--------------+-------------------------------+-------+-------------------------+-----------+
    ↪
| BackendId | Host         | Key                           | Value | Type                    |
    ↪ IsMutable |
+--
    ↪ ---------+--------------+-------------------------------+-------+-------------------------+-----------+
    ↪
| 12793     | 172.16.123.1 | LZ4_HC_compression_level      | 9     | int64_t                 |
    ↪ true      |
| 12793     | 172.16.123.1 | agent_task_trace_threshold_sec | 2     | int32_t                 |
    ↪ true      |
...
| 12793     | 172.16.123.1 | zone_map_row_num_threshold    | 20    | int32_t                 |
    ↪ true      |
+--
    ↪ ---------+--------------+-------------------------------+-------+-------------------------+-----------+
    ↪
```

查询符合指定模式的配置项

```
SHOW BACKEND CONFIG LIKE '%compression_level%'
```

结果为

```
+-----------+--------------+--------------------------+-------+---------+-----------+
| BackendId | Host         | Key                      | Value | Type    | IsMutable |
+-----------+--------------+--------------------------+-------+---------+-----------+
| 12793     | 172.16.123.1 | LZ4_HC_compression_level | 9     | int64_t | true      |
| 12794     | 172.16.123.2 | LZ4_HC_compression_level | 9     | int64_t | true      |
+-----------+--------------+--------------------------+-------+---------+-----------+
```

7.3.12.1.15 DECOMMISSION BACKEND

描述

该语句用于将 BE 节点安全地从集群中下线。该操作为异步操作。

语法

```
ALTER SYSTEM DECOMMISSION BACKEND "<be_identifier>" [, "<be_identifier>" ... ]
```

其中：

```
be_identifier
  : "<be_host>:<be_heartbeat_port>"
  | "<backend_id>"
```

必选参数

1.

可以是 BE 节点的主机名或 IP 地址

2.

BE 节点的心跳端口，默认为 9050

3.

BE 节点的 ID

<be_host>、<be_heartbeat_port>及<backend_id>均可通过 SHOW BACKENDS 语句查询获得。

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
NODE_PRIV		

注意事项

1. 执行此命令后，可以通过 SHOW BACKENDS 语句查看下线状态（SystemDecommissioned列的值为true）和下线进度（TabletNum列的值会缓慢降到 0）
2. 通常情况下，在TabletNum列的值会降到 0 后，此 BE 节点就会被删除，如果您不希望 Doris 自动删除 BE，可以更改 FE Master 的配置drop_backend_after_decommission为 false

3. 当前 BE 如果存储的数据量比较大，那么 DECOMMISSION 操作可能持续几个小时甚至是几天
4. 如果 DECOMMISSION 操作的进度卡住不动，具体表现为 SHOW BACKENDS 语句中的TabletNum列一直固定在某个值，那么可能是以下的一些情况：

- 当前 BE 上的 tablet 找不到合适的其他 BE 去迁移，比如在一个 3 节点的集群有一张 3 副本的表，要下线其中一个节点，那么该节点找不到其他 BE 可用来迁移数据（其他两个 BE 已经各有一个副本了）
- 当前 BE 上的 tablet 还在回收站中，可以清空回收站后，再等待
- 当前 BE 上的 tablet 太大，导致在迁移单个 tablet 时，一直因为超时而无法将这个 tablet 迁移走，可以调整 FE Master 的配置max_clone_task_timeout_sec为一个更大的值（默认为 7200 秒）
- 当前 BE 上的 tablet 存在未完成的事务，可以等事务完成或手动中止事务
- 其他情况可以在 FE Master 的日志过滤replicas to decommission关键字，找出异常的 tablet，使用 SHOW TABLET 语句找到该 tablet 所属的表，然后重新建一张新的表，将数据从旧表迁移至新表，最后使用 DROP TABLE FORCE 的方式将旧表删除掉

示例

1. 根据 BE 的 Host 和 HeartbeatPort 从集群中安全下线两个节点

```
ALTER SYSTEM DECOMMISSION BACKEND "192.168.0.1:9050", "192.168.0.2:9050";
```

2. 根据 BE 的 ID 从集群中安全下线一个节点

```
ALTER SYSTEM DECOMMISSION BACKEND "10002";
```

7.3.12.1.16 ADD BROKER

描述

该语句用于添加一个或者多个 BROKER 节点。

语法

```
ALTER SYSTEM ADD BROKER <broker_name> "<host>:<ipc_port>" [,"<host>:<ipc_port>" [, ...] ];
```

必选参数

1. <broker_name>

给添加的 broker 进程起的名字。同一个集群中的 broker_name 建议保持一致。

2. <host>

需要添加的 broker 进程所在节点的 IP，如果启用了 FQDN，则使用该节点的 FQDN。

3. <ipc_port>

需要添加的 broker 进程所在节点的 PORT，该端口默认值为 8000。

输出字段

无

权限控制

执行该操作的用户需要具备 NODE_PRIV 的权限。

示例

1. 增加两个 Broker

```
ALTER SYSTEM ADD BROKER "host1:port", "host2:port";
```

2. 增加一个 Broker，使用 FQDN

```
ALTER SYSTEM ADD BROKER "broker_fqdn1:port";
```

7.3.12.1.17 DROP BROKER

描述

该语句是删除 BROKER 节点。

语法

1. 删除所有 Broker

```
ALTER SYSTEM DROP ALL BROKER <broker_name>;
```

2. 删除某一个 Broker 节点

```
ALTER SYSTEM DROP BROKER <broker_name> "<host>:<ipc_port>"[, "<host>:<ipc_port>" [, ...] ];
```

必选参数

1. <broker_name>

需要删除的 broker 进程的名字。

2. <host>

需要删除的 broker 进程所在节点的 IP，如果启用了 FQDN，则使用该节点的 FQDN。

3. <ipc_port>

需要删除的 broker 进程所在节点的 PORT，该端口默认值为 8000。

输出字段

无 ###### 权限控制执行该操作的用户需要具备 NODE_PRIV 的权限。

示例

1. 删除所有 Broker

```
ALTER SYSTEM DROP ALL BROKER broker_name;
```

2. 删除某一个 Broker 节点

```
ALTER SYSTEM DROP BROKER broker_name "10.10.10.1:8000";
```

7.3.12.1.18 SHOW BROKER

描述

该语句用于查看当前存在的 broker 进程状态。

语法：

```
SHOW BROKER;
```

输出字段

列名	类型	说明
Name	varchar	Broker 进程名称
Host	varchar	Broker 进程所在节点 IP
Port	varchar	Broker 进程所在节点 Port
Alive	varchar	Broker 进程状态
LastStartTime	varchar	Broker 进程上次启动时间
LastUpdateTime	varchar	Broker 进程上次更新时间
ErrMsg	varchar	Broker 进程上次启动失败的错误信息

权限控制

执行该语句的用户需要具备 ADMIN/OPERATOR 的权限

示例

1. 查看当前存在的 broker 进程状态

```
show broker;
```

```
+-------------+------------+------+-------+---------------------+---------------------+---------+
    ↪
| Name        | Host       | Port | Alive | LastStartTime       | LastUpdateTime      |
    ↪ ErrMsg |
+-------------+------------+------+-------+---------------------+---------------------+---------+
    ↪
| broker_test | 10.10.10.1 | 8196 | true  | 2025-01-21 11:30:10 | 2025-01-21 11:31:40 |
    ↪         |
+-------------+------------+------+-------+---------------------+---------------------+---------+
    ↪
```

7.3.12.2 计算资源管理

7.3.12.2.1 CREATE RESOURCE

描述

该语句用于创建资源。仅 root 或 admin 用户可以创建资源。目前支持 Spark, ODBC, S3, JDBC, HDFS, HMS, ES 外部资源。将来其他外部资源可能会加入到 Doris 中使用，如 Spark/GPU 用于查询，HDFS/S3 用于外部存储，MapReduce 用于 ETL 等。

语法

```
CREATE [EXTERNAL] RESOURCE "<resource_name>"
PROPERTIES (
   `<property>`
    [ , ... ]
);
```

参数

1.<property>

<property> 格式为 <key> = <value>，<key>的具体可选值如下：

参数	说明	是否必填
<type>	指定资源的类型，支持 spark/odbc_catalog/s3/jdbc/hdfs/hms/es。	是

根据<type>的不同 PROPERTIES 的参数有所不同，具体见示例。

示例

1. 创建 yarn cluster 模式，名为 spark0 的 Spark 资源。

```
  CREATE EXTERNAL RESOURCE "spark0"
  PROPERTIES
  (
    "type" = "spark",
    "spark.master" = "yarn",
    "spark.submit.deployMode" = "cluster",
    "spark.jars" = "xxx.jar,yyy.jar",
    "spark.files" = "/tmp/aaa,/tmp/bbb",
    "spark.executor.memory" = "1g",
    "spark.yarn.queue" = "queue0",
    "spark.hadoop.yarn.resourcemanager.address" = "127.0.0.1:9999",
    "spark.hadoop.fs.defaultFS" = "hdfs://127.0.0.1:10000",
    "working_dir" = "hdfs://127.0.0.1:10000/tmp/doris",
    "broker" = "broker0",
    "broker.username" = "user0",
    "broker.password" = "password0"
  );
```

Spark 相关参数如下：- spark.master: 必填，目前支持 yarn，spark://host:port。 - spark.submit.deployMode: Spark 程序的部署模式，必填，支持 cluster，client 两种。- spark.hadoop.yarn.resourcemanager.address: master 为 yarn 时必填。- spark.hadoop.fs.defaultFS: master 为 yarn 时必填。- 其他参数为可选，参考这里

Spark 用于 ETL 时需要指定 working_dir 和 broker。说明如下：

- working_dir: ETL 使用的目录。spark 作为 ETL 资源使用时必填。例如：hdfs://host:port/tmp/doris。
- broker: broker 名字。spark 作为 ETL 资源使用时必填。需要使用`ALTER SYSTEM ADD BROKER` 命令提前完成配置。
- broker.property_key: broker 读取 ETL 生成的中间文件时需要指定的认证信息等。

2. 创建 ODBC resource

```
CREATE EXTERNAL RESOURCE `oracle_odbc`
PROPERTIES (
   "type" = "odbc_catalog",
   "host" = "192.168.0.1",
   "port" = "8086",
   "user" = "test",
   "password" = "test",
   "database" = "test",
   "odbc_type" = "oracle",
   "driver" = "Oracle 19 ODBC driver"
);
```

ODBC 的相关参数如下：- hosts：外表数据库的 IP 地址 - driver：ODBC 外表的 Driver 名，该名字需要和 be/conf/odbcinst.ini 中的 Driver 名一致。- odbc_type：外表数据库的类型，当前支持 oracle, mysql, postgresql - user：外表数据库的用户名 - password：对应用户的密码信息 - charset: 数据库链接的编码信息 - 另外还支持每个 ODBC Driver 实现自定义的参数，参见对应 ODBC Driver 的说明

3. 创建 S3 resource

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
   "type" = "s3",
   "s3.endpoint" = "bj.s3.com",
   "s3.region" = "bj",
   "s3.access_key" = "bbb",
   "s3.secret_key" = "aaaa",
   -- the followings are optional
   "s3.connection.maximum" = "50",
   "s3.connection.request.timeout" = "3000",
   "s3.connection.timeout" = "1000"
);
```

如果 s3 reource 在冷热分层中使用，需要添加额外的字段。

```
CREATE RESOURCE "remote_s3"
PROPERTIES
(
```

```
    "type" = "s3",
    "s3.endpoint" = "bj.s3.com",
    "s3.region" = "bj",
    "s3.access_key" = "bbb",
    "s3.secret_key" = "aaaa",
    -- required by cooldown
    "s3.root.path" = "/path/to/root",
    "s3.bucket" = "test-bucket"
);
```

S3 相关参数如下：- 必需参数 - s3.endpoint：s3 endpoint - s3.region：s3 region - s3.root.path：s3 根目录 - s3.access_key：s3 access key - s3.secret_key：s3 secret key - s3.bucket：s3 的桶名 - 可选参数 - s3.connection.maximum：s3 最大连接数量，默认为 50 - s3.connection.request.timeout：s3 请求超时时间，单位毫秒，默认为 3000 - s3.connection.timeout：s3 连接超时时间，单位毫秒，默认为 1000

```
Doris 也支持通过 `AWS Assume Role` 的方式创建 S3 Resource, 请参考如下文档配置和使用[AWS intergration](#aws-认证和鉴权).
```

4. 创建 JDBC resource

```
CREATE RESOURCE mysql_resource PROPERTIES (
    "type"="jdbc",
    "user"="root",
    "password"="123456",
    "jdbc_url" = "jdbc:mysql://127.0.0.1:3316/doris_test?useSSL=false",
    "driver_url" = "https://doris-community-test-1308700295.cos.ap-hongkong.myqcloud.com/jdbc_driver/mysql-connector-java-8.0.25.jar",
"driver_class" = "com.mysql.cj.jdbc.Driver"
);
```

JDBC 的相关参数如下：- user：连接数据库使用的用户名 - password：连接数据库使用的密码 - jdbc_url: 连接到指定数据库的标识符 - driver_url: jdbc 驱动包的 url - driver_class: jdbc 驱动类

5. 创建 HDFS resource

```
CREATE RESOURCE hdfs_resource PROPERTIES (
    "type"="hdfs",
    "hadoop.username"="user",
    "dfs.nameservices" = "my_ha",
    "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",
    "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",
    "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",
    "dfs.client.failover.proxy.provider.my_ha" = "org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider"
);
```

HDFS 相关参数如下：- fs.defaultFS: namenode 地址和端口 - hadoop.username: hdfs 用户名 - dfs.nameservices: name service 名称，与 hdfs-site.xml 保持一致 - dfs.ha.namenodes.[nameservice ID]: namenode 的 id 列表，与 hdfs-site.xml 保持

一致 - dfs.namenode.rpc-address.[nameservice ID].[name node ID]: Name node 的 rpc 地址，数量与 namenode 数量相同，与 hdfs-site.xml 保持一致

6. 创建 HMS resource

HMS resource 用于 hms catalog

```
CREATE RESOURCE hms_resource PROPERTIES (
   'type'='hms',
   'hive.metastore.uris' = 'thrift://127.0.0.1:7004',
   'dfs.nameservices'='HANN',
   'dfs.ha.namenodes.HANN'='nn1,nn2',
   'dfs.namenode.rpc-address.HANN.nn1'='nn1_host:rpc_port',
   'dfs.namenode.rpc-address.HANN.nn2'='nn2_host:rpc_port',
   'dfs.client.failover.proxy.provider.HANN'='org.apache.hadoop.hdfs.server.namenode.ha.
      ↪ ConfiguredFailoverProxyProvider'
);
```

HMS 的相关参数如下：- hive.metastore.uris: hive metastore server 地址可选参数：- dfs.: 如果 hive 数据存放在 hdfs，需要添加类似 HDFS resource 的参数，也可以将 hive-site.xml 拷贝到 fe/conf 目录下 - s3.: 如果 hive 数据存放在 s3，需要添加类似 S3 resource 的参数。如果连接 阿里云 Data Lake Formation，可以将 hive-site.xml 拷贝到 fe/conf 目录下

7. 创建 ES resource

```
CREATE RESOURCE es_resource PROPERTIES (
   "type"="es",
   "hosts"="http://127.0.0.1:29200",
   "nodes_discovery"="false",
   "enable_keyword_sniff"="true"
);
```

ES 的相关参数如下：- hosts: ES 地址，可以是一个或多个，也可以是 ES 的负载均衡地址 - user: ES 用户名 - password: 对应用户的密码信息 - enable_docvalue_scan: 是否开启通过 ES/Lucene 列式存储获取查询字段的值，默认为 true - enable_keyword_sniff: 是否对 ES 中字符串分词类型 text.fields 进行探测，通过 keyword 进行查询 (默认为 true，设置为 false 会按照分词后的内容匹配) - nodes_discovery: 是否开启 ES 节点发现，默认为 true，在网络隔离环境下设置为 false，只连接指定节点 - http_ssl_enabled: ES 是否开启 https 访问模式，目前在 fe/be 实现方式为信任所有

7.3.12.2.2 ALTER RESOURCE

描述

该语句用于修改一个已有的资源。仅 root 或 admin 用户可以修改资源。

语法

```
ALTER RESOURCE '<resource_name>'
PROPERTIES (
  `<property>`,
  [ , ... ]
```

```
);
```

参数

1.<property>

<property> 格式为 <key> = <value>，不支持修改<key>等于 type 的<value>。

其他可修改的 properties 参数参考 CREATE-RESOURCE 章节;

示例

1. 修改名为 spark0 的 Spark 资源的工作目录：

```
ALTER RESOURCE 'spark0' PROPERTIES ("working_dir" = "hdfs://127.0.0.1:10000/tmp/doris_new");
```

2. 修改名为 remote_s3 的 S3 资源的最大连接数：

```
ALTER RESOURCE 'remote_s3' PROPERTIES ("s3.connection.maximum" = "100");
```

3. 修改冷热分层 S3 资源相关信息

- 支持修改项
- s3.access_key s3 的 ak 信息
- s3.secret_key s3 的 sk 信息
- s3.session_token s3 的 session token 信息
- s3.connection.maximum s3 最大连接数，默认 50
- s3.connection.timeout s3 连接超时时间，默认 1000ms
- s3.connection.request.timeout s3 请求超时时间，默认 3000ms
- 禁止修改项
- s3.region
- s3.bucket"
- s3.root.path
- s3.endpoint

```
ALTER RESOURCE "showPolicy_1_resource" PROPERTIES("s3.connection.maximum" = "1111");
```

7.3.12.2.3 DROP RESOURCE

描述

该语句用于删除一个已有的资源。仅 root 或 admin 用户可以删除资源。

语法

```
DROP RESOURCE '<resource_name>'
```

注意事项

正在使用的 ODBC/S3 资源无法删除。

示例

1. 删除名为 spark0 的 Spark 资源：

```
DROP RESOURCE 'spark0';
```

7.3.12.2.4 SHOW RESOURCES

描述

该语句用于展示用户有使用权限的资源。普通用户仅能展示有使用权限的资源，root 或 admin 用户会展示所有的资源。

语法

```
SHOW RESOURCES
[
  WHERE
  [NAME [ = "<your_resource_name>" | LIKE "<name_matcher>"]]
  [RESOURCETYPE = "<type>"]
] | [LIKE "<pattern>"]
[ORDER BY ...]
[LIMIT <limit>][OFFSET <offset>];
```

注意事项

1. 如果使用 NAME LIKE，则会匹配 RESOURCES 的 Name 包含 name_matcher 的 Resource
2. 如果使用 NAME = ，则精确匹配指定的 Name
3. 如果指定了 RESOURCETYPE，则匹配对应的 Resource 类型，支持的 RESOURCETYPE，可参考 CREATE-RESOURCE;
4. 可以使用 ORDER BY 对任意列组合进行排序
5. 如果指定了 LIMIT，则显示 limit 条匹配记录。否则全部显示
6. 如果指定了 OFFSET，则从偏移量 offset 开始显示查询结果。默认情况下偏移量为 0。
7. 如果使用了 LIKE，则会忽略 WHERE 语句。

示例

1. 展示当前用户拥有权限的所有 Resource

```
SHOW RESOURCES;
```

2. 展示指定 Resource，NAME 中包含字符串 “20140102”，展示 10 个属性

```
SHOW RESOURCES WHERE NAME LIKE "2014_01_02" LIMIT 10;
```

3. 展示指定 Resource，指定 NAME 为 “20140102” 并按 KEY 降序排序

```
    SHOW RESOURCES WHERE NAME = "20140102" ORDER BY `KEY` DESC;
```

4. 使用 LIKE 进行模糊匹配

```
    SHOW RESOURCES LIKE "jdbc%";
```

7.3.12.2.5 CREATE WORKLOAD GROUP

描述

该语句用于创建资源组。资源组可实现单个 be 上 cpu 资源和内存资源的隔离。

语法

```
CREATE WORKLOAD GROUP [IF NOT EXISTS] "<rg_name>"
PROPERTIES (
    `<property>`
    [ , ... ]
);
```

参数

1.<property>

<property> 格式为 <key> = <value>，<key>的具体可选值如下：

参数	说明	是否必填

参数	说明	是否必填
<cpu_share>	用于设置资源组获取cpu时间的多少，可以实现cpu资源软隔离。cpu_share是相对值，表示正在运行的资源组可	是

参数	说明	是否必填
< ↪ memory ↪ _ ↪ limit ↪ > ↪	用于设置资源组可以使用 be 内存的百分比。资源组内存限制的绝对值为：物理内存 ↪ ↪ * ↪ ↪ mem ↪ _ ↪ limit ↪ ↪ * ↪ ↪ memory ↪ _ ↪ limit ↪ ，其中	是

参数	说明	是否必填
< ↪ enable ↪ _ ↪ memory ↪ _ ↪ overcommit ↪ > ↪	用于开启资源组内存软隔离，默认为false。如果设置为false，则该资源组为内存硬隔离，系统检测到资源组内存使用超	否

参数	说明	是否必填

示例

1. 创建名为 g1 的资源组：

```
create workload group if not exists g1
properties (
    "cpu_share"="10",
    "memory_limit"="30%",
    "enable_memory_overcommit"="true"
);
```

7.3.12.2.6 ALTER WORKLOAD GROUP

描述

该语句用于修改资源组。

语法

```
ALTER WORKLOAD GROUP  "<rg_name>"
PROPERTIES (
  `<property>`
  [ , ... ]
);
```

参数

1.<property>

<property> 格式为 <key> = <value>，<key>的具体可选值如下：

参数	说明	是否必填

参数	说明	是否必填
 ↪ cpu ↪ _ ↪ share ↪ > ↪	用于设置资源组获取 cpu 时间的多少，可以实现 cpu 资源软隔离。cpu_share 是相对值，表示正在运行的资源组可	是

参数	说明	是否必填
< ↪ memory ↪ _ ↪ limit ↪ > ↪	用于设置资源组可以使用 be 内存的百分比。资源组内存限制的绝对值为：物理内存 ↪ ↪ * ↪ ↪ mem ↪ _ ↪ limit ↪ ↪ * ↪ ↪ memory ↪ _ ↪ limit ↪ ，其中	是

参数	说明	是否必填
< ↪ enable ↪ _ ↪ memory ↪ _ ↪ overcommit ↪ > ↪	用于开启资源组内存软隔离，默认为false。如果设置为false，则该资源组为内存硬隔离，系统检测到资源组内存使用超	否

参数	说明	是否必填

示例

1. 修改名为 g1 的资源组：

```
alter workload group g1
properties (
    "cpu_share"="30",
    "memory_limit"="30%"
);
```

7.3.12.2.7 DROP WORKLOAD GROUP

描述

该语句用于删除资源组。

语法

```
DROP WORKLOAD GROUP [IF EXISTS] '<rg_name>'
```

示例

1. 删除名为 g1 的资源组：

```
drop workload group if exists g1;
```

7.3.12.2.8 SHOW WORKLOAD GROUPS

描述

该语句用于展示当前用户具有 usage_priv 权限的资源组。

语法

```
SHOW WORKLOAD GROUPS [LIKE "<pattern>"];
```

注意事项

该语句仅做资源组简单展示，更复杂的展示可参考 tvf workload_groups().

示例

1. 展示所有资源组：

```
mysql> show workload groups;
+----------+--------+--------------------------+---------+
| Id       | Name   | Item                     | Value   |
+----------+--------+--------------------------+---------+
| 10343386 | normal | cpu_share                | 10      |
| 10343386 | normal | memory_limit             | 30%     |
| 10343386 | normal | enable_memory_overcommit | true    |
| 10352416 | g1     | memory_limit             | 20%     |
| 10352416 | g1     | cpu_share                | 10      |
+----------+--------+--------------------------+---------+
```

2. 使用 LIKE 模糊匹配：

```
mysql> show workload groups like "normal%"
+----------+--------+--------------------------+---------+
| Id       | Name   | Item                     | Value   |
+----------+--------+--------------------------+---------+
| 10343386 | normal | cpu_share                | 10      |
| 10343386 | normal | memory_limit             | 30%     |
| 10343386 | normal | enable_memory_overcommit | true    |
+----------+--------+--------------------------+---------+
```

7.3.12.2.9 CREATE WORKLOAD POLICY

描述

创建一个 Workload Policy，用于当一个查询满足一些条件时，就对该查询执行相应的动作。

语法

```
CREATE WORKLOAD POLICY [ IF NOT EXISTS ] <workload_policy_name>
CONDITIONS(<conditions>) ACTIONS(<actions>)
[ PROPERTIES (<properties>) ]
```

必选参数

1. <workload_policy_name>: Workload Policy 的名字
2. <conditions>
    - be_scan_rows，一个 SQL 在单个 BE 进程内 Scan 的行数，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值。
    - be_scan_bytes，一个 SQL 在单个 BE 进程内 Scan 的字节数，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值，单位是字节。
    - query_time，一个 SQL 在单个 BE 进程上的运行时间，时间单位是毫秒。

- query_be_memory_bytes，从 2.1.5 版本开始支持。一个 SQL 在单个 BE 进程内使用的内存用量，如果这个 SQL 在 BE 上是多并发执行，那么就是多个并发的累加值，单位是字节。

3. <actions>

- set_session_variable，这个 Action 可以执行一条 set_session_variable 的语句。同一个 Policy 可以有多个 set_session_variable，也就是说一个 Policy 可以执行多个修改 session 变量的语句。
- cancel_query，取消查询。

可选参数

1. <properties>

- enabled，取值为 true 或 false，默认值为 true，表示当前 Policy 处于启用状态，false 表示当前 Policy 处于禁用状态。
- priority，取值范围为 0 到 100 的正整数，默认值为 0，代表 Policy 的优先级，该值越大，优先级越高。这个属性的主要作用是，当匹配到多个 Policy 时，选择优先级最高的 Policy。
- workload_group，目前一个 Policy 可以绑定一个 Workload Group，代表这个 Policy 只对某个 Workload Group 生效。默认为空，代表对所有查询生效。

权限控制

至少具备ADMIN_PRIV权限

示例

1. 新建一个 Workload Policy，作用是杀死所有查询时间超过 3s 的查询

```
create workload policy kill_big_query conditions(query_time > 3000) actions(cancel_query)
```

2. 新建一个 Workload Policy，默认不开启

```
create workload policy kill_big_query conditions(query_time > 3000) actions(cancel_query)
    ↪ properties('enabled'='false')
```

7.3.12.2.10 ALTER WORKLOAD POLICY

描述

修改一个 Workload Group 的属性，目前只支持修改属性，不支持修改 action 和 condition。

语法

```
ALTER WORKLOAD POLICY <workload_policy_name> PROPERTIES( <properties> )
```

必选参数

1. <workload_policy_name>: Workload Policy 的 Name

2. <properties>:

    - enabled，取值为 true 或 false，默认值为 true，表示当前 Policy 处于启用状态，false 表示当前 Policy 处于禁用状态。
    - priority，取值范围为 0 到 100 的正整数，默认值为 0，代表 Policy 的优先级，该值越大，优先级越高。这个属性的主要作用是，当匹配到多个 Policy 时，选择优先级最高的 Policy。
    - workload_group，目前一个 Policy 可以绑定一个 Workload Group，代表这个 Policy 只对某个 Workload Group 生效。默认为空，代表对所有查询生效。

权限控制

至少具有ADMIN_PRIV权限

示例

1. 禁用一个 Workload Policy

```
  alter workload policy cancel_big_query properties('enabled'='false')
```

7.3.12.2.11 DROP WORKLOAD POLICY

描述

删除一个 Workload Policy

语法

```
DROP WORKLOAD POLICY [ IF EXISTS ] <workload_policy_name>
```

必选参数

1. <workload_policy_name>: Workload Policy 的 Name

权限控制

至少具有ADMIN_PRIV权限

示例

1. 删除一个名为 cancel_big_query 的 Workload Policy

```
  drop workload policy if exists cancel_big_query
```

7.3.12.2.12 SHOW COMPUTE GROUPS

描述

在存算分离模式中，显示当前用户有集群使用权限的计算集群列表

语法

```
SHOW COMPUTE GROUPS
```

返回值

返回当前拥有计算集群权限的集群列表

- Name - 计算集群 compute group 名字
- IsCurrent 当前用户是否正在使用这个 compute group
- Users 将此项 compute group 设置为 default compute group 的用户名
- BackendNum 此项 compute group 拥有的 backend 个数

示例

指定使用该计算集群 compute_cluster

```
 show compute groups;
```

结果为

```
+-----------------+-----------+-------+------------+
| Name            | IsCurrent | Users | BackendNum |
+-----------------+-----------+-------+------------+
| compute_cluster | TRUE      |       | 3          |
+-----------------+-----------+-------+------------+
```

注意事项（Usage Note）

若当前用户无任何 compute group 权限，show compute group 将返回空列表

7.3.12.3 存储资源管理

7.3.12.3.1 CREATE-STORAGE-VAULT

描述

该命令用于创建存储库。本文档的主题描述了创建 Doris 自管理存储库的语法。

语法

```
CREATE STORAGE VAULT [IF NOT EXISTS] <`vault_name`> [ <`properties`> ]
```

必选参数

参数	描述
vault_name	存储库的名称。这是您要创建的新存储库的唯一标识符。

可选参数

参数	描述
[IF NOT EXISTS]	如果指定的存储库已经存在，则不会执行创建操作，并且不会抛出错误。这可以防止重复创建相同的存储库。
PROPERTIES	一组键值对，用来设置或更新存储库的具体属性。每个属性由键（<key>）和值（<value>）组成，并用等号 (=) 分隔。多个键值对之间用逗号 (,) 分隔。

S3 Vault

参数	是否必需	描述
s3.endpoint	必需	用于对象存储的端点。对于 Azure Blob 存储，endpoint 是固定的 blob.core.windows.net。
s3.region	必需	您的存储桶的区域。(如果您使用 GCP 或 AZURE, 则可填 us-east-1)。
s3.root.path	必需	存储数据的路径。
s3.bucket	必需	您的对象存储账户的存储桶。(如果您使用 Azure，则为 StorageAccount)。
s3.access_key	必需	您的对象存储账户的访问密钥。(如果您使用 Azure，则为 AccountName)。
s3.secret_key	必需	您的对象存储账户的秘密密钥。(如果您使用 Azure，则为 AccountKey)。
provider	必需	提供对象存储服务的云供应商。支持的值有COS，OSS，S3，OBS，BOS，AZURE，GCP
use_path_style	可选	使用 path-style URL(私有化部署环境) 或者virtual-hosted-style URL(公有云环境建议), 默认值 true (path-style)

注意:

1. s3.endpoint 如果不提供http:// 或 https:// 前缀, 则默认使用 http; 如提供，则会以前缀为准;
2. Doris 也支持AWS Assume Role的方式创建 Storage Vault(仅限于 AWS S3)，配置方式请参考AWS 集成。

HDFS vault

参数	是否必需	描述
fs.defaultFS	必需	Hadoop 配置属性，指定要使用的默认文件系统。
path_prefix	可选	存储数据的路径前缀。如果没有指定则会使用 user 账户下的默认路径。
hadoop.username	可选	Hadoop 配置属性，指定访问文件系统的用户。如果没有指定则会使用启动 ha
hadoop.security.authentication	可选	用于 hadoop 的认证方式。如果希望使用 kerberos 则可以填写kerberos。
hadoop.kerberos.principal	可选	您的 kerberos 主体的路径。
hadoop.kerberos.keytab	可选	您的 kerberos keytab 的路径。

举例

1. 创建 HDFS storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS hdfs_vault_demo
PROPERTIES (
    "type" = "hdfs",                                    -- required
    "fs.defaultFS" = "hdfs://127.0.0.1:8020",           -- required
    "path_prefix" = "big/data",                         -- optional,  一般按照业务名称填写
    "hadoop.username" = "user"                          -- optional
    "hadoop.security.authentication" = "kerberos"       -- optional
    "hadoop.kerberos.principal" = "hadoop/127.0.0.1@XXX" -- optional
    "hadoop.kerberos.keytab" = "/etc/emr.keytab"        -- optional
);
```

2. 创建阿里云 OSS storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS oss_demo_vault
PROPERTIES (
    "type" = "S3",                                      -- required
    "s3.endpoint" = "oss-cn-beijing.aliyuncs.com",      -- required
    "s3.access_key" = "xxxxxx",                         -- required,  Your OSS access key
    "s3.secret_key" = "xxxxxx",                         -- required,  Your OSS secret key
    "s3.region" = "cn-beijing",                         -- required
    "s3.root.path" = "oss_demo_vault_prefix",           -- required
    "s3.bucket" = "xxxxxx",                             -- required,  Your OSS bucket name
    "provider" = "OSS",                                 -- required
    "use_path_style" = "false"                          -- optional,  OSS 建议设置 false
);
```

3. 创建腾讯云 COS storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS cos_demo_vault
PROPERTIES (
    "type" = "S3",
    "s3.endpoint" = "cos.ap-guangzhou.myqcloud.com",    -- required
    "s3.access_key" = "xxxxxx",                         -- required,  Your COS access key
    "s3.secret_key" = "xxxxxx",                         -- required,  Your COS secret key
    "s3.region" = "ap-guangzhou",                       -- required
    "s3.root.path" = "cos_demo_vault_prefix",           -- required
    "s3.bucket" = "xxxxxx",                             -- required,  Your COS bucket name
    "provider" = "COS",                                 -- required
    "use_path_style" = "false"                          -- optional,  COS 建议设置 false
);
```

4. 创建华为云 OBS storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS obs_demo_vault
```

```
PROPERTIES (
    "type" = "S3",                                      -- required
    "s3.endpoint" = "obs.cn-north-4.myhuaweicloud.com",  -- required
    "s3.access_key" = "xxxxxx",                         -- required,  Your OBS access key
    "s3.secret_key" = "xxxxxx",                         -- required,  Your OBS secret key
    "s3.region" = "cn-north-4",                         -- required
    "s3.root.path" = "obs_demo_vault_prefix",           -- required
    "s3.bucket" = "xxxxxx",                             -- required,  Your OBS bucket name
    "provider" = "OBS",                                 -- required
    "use_path_style" = "false"                          -- optional,  OBS 建议设置 false
);
```

5. 创建百度云 BOS storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS bos_demo_vault
PROPERTIES (
    "type" = "S3",                                      -- required
    "s3.endpoint" = "s3.bj.bcebos.com",                 -- required
    "s3.access_key" = "xxxxxx",                         -- required,  Your BOS access key
    "s3.secret_key" = "xxxxxx",                         -- required,  Your BOS secret key
    "s3.region" = "bj",                                 -- required
    "s3.root.path" = "bos_demo_vault_prefix",           -- required
    "s3.bucket" = "xxxxxx",                             -- required,  Your BOS bucket name
    "provider" = "BOS",                                 -- required
    "use_path_style" = "false"                          -- optional,  BOS 建议设置 false
);
```

6. 创建亚马逊云 S3 storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS s3_demo_vault
PROPERTIES (
    "type" = "S3",                                      -- required
    "s3.endpoint" = "s3.us-east-1.amazonaws.com",       -- required
    "s3.access_key" = "xxxxxx",                         -- required,  Your S3 access key
    "s3.secret_key" = "xxxxxx",                         -- required,  Your S3 secret key
    "s3.region" = "us-east-1",                          -- required
    "s3.root.path" = "s3_demo_vault_prefix",            -- required
    "s3.bucket" = "xxxxxx",                             -- required,  Your S3 bucket name
    "provider" = "S3",                                  -- required
    "use_path_style" = "false"                          -- optional,  S3 建议设置 false
);
```

注意:

Doris 也支持AWS Assume Role的方式创建 Storage Vault(仅限于 AWS S3)，配置方式请参考AWS 集成.

7. 创建 MinIO storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS minio_demo_vault
PROPERTIES (
    "type" = "S3",                                    -- required
    "s3.endpoint" = "127.0.0.1:9000",                 -- required
    "s3.access_key" = "xxxxxx",                       -- required,  Your minio access key
    "s3.secret_key" = "xxxxxx",                       -- required,  Your minio secret key
    "s3.region" = "us-east-1",                        -- required
    "s3.root.path" = "minio_demo_vault_prefix",       -- required
    "s3.bucket" = "xxxxxx",                           -- required,  Your minio bucket name
    "provider" = "S3",                                -- required
    "use_path_style" = "true"                         -- required,  minio 建议设置 true
);
```

8. 创建微软 AZURE storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS azure_demo_vault
PROPERTIES (
    "type" = "S3",                                     -- required
    "s3.endpoint" = "blob.core.windows.net",           -- required
    "s3.access_key" = "xxxxxx",                        -- required,  Your Azure AccountName
    "s3.secret_key" = "xxxxxx",                        -- required,  Your Azure AccountKey
    "s3.region" = "us-east-1",                         -- required
    "s3.root.path" = "azure_demo_vault_prefix",        -- required
    "s3.bucket" = "xxxxxx",                            -- required,  Your Azure StorageAccount
    "provider" = "AZURE"                               -- required
);
```

9. 创建谷歌 GCP storage vault。

```
CREATE STORAGE VAULT IF NOT EXISTS gcp_demo_vault
PROPERTIES (
    "type" = "S3",                                     -- required
    "s3.endpoint" = "storage.googleapis.com",          -- required
    "s3.access_key" = "xxxxxx",                        -- required
    "s3.secret_key" = "xxxxxx",                        -- required
    "s3.region" = "us-east-1",                         -- required
    "s3.root.path" = "gcp_demo_vault_prefix",          -- required
    "s3.bucket" = "xxxxxx",                            -- required
    "provider" = "GCP"                                 -- required
);
```

注意

s3.access_key 对应 GCP HMAC key 的 Access ID

s3.secret_key 对应 GCP HMAC key 的 Secret

关键词

```
CREATE, STORAGE VAULT
```

7.3.12.3.2 ALTER STORAGE VAULT

描述

更改 Storage Vault 的可修改属性值

语法

```
ALTER STORAGE VAULT <storage_vault_name>
PROPERTIES (<storage_vault_property>)
```

必选参数

- type：可选值为 s3, hdfs. 这个字段从 3.0.8 之后是选填字段.

当 type 为 s3 时，允许出现的属性字段如下：

- s3.access_key：s3 vault 的 ak
- s3.secret_key：s3 vault 的 sk
- vault_name：vault 的名字。当一个 vault 通过SET <original_vault_name> DEFAULT STORAGE VAULT语句被设为默认存储 vault 时，不能修改其名称。若需修改名称，需首先通过UNSET DEFAULT STORAGE VAULT命令取消默认存储 vault 设置，再执行重命名操作。最后，若要将重命名后的 vault 设为默认存储 vault，可通过执行SET <new_vault_name> AS DEFAULT STORAGE VAULT语句完成设置。
- use_path_style：是否允许 path style url，可选值为 true，false。默认值是 false。

当 type 为 hdfs 时，禁止出现的字段：

- path_prefix：存储路径前缀
- fs.defaultFS：hdfs name

权限控制

执行此 SQL 命令的用户必须至少具有 ADMIN_PRIV 权限。

示例

修改 s3 storage vault ak

```
ALTER STORAGE VAULT old_vault_name
PROPERTIES (
  "type"="S3",
  "VAULT_NAME" = "new_vault_name",
   "s3.access_key" = "new_ak"
```

```
);
```

修改 hdfs storage vault

```
ALTER STORAGE VAULT old_vault_name
PROPERTIES (
  "type"="hdfs",
  "VAULT_NAME" = "new_vault_name",
  "hadoop.username" = "hdfs"
);
```

7.3.12.3.3 SET DEFAULT STORAGE VAULT

描述

该语句用于在 Doris 中设置默认存储库。默认存储库用于存储内部或系统表的数据。如果未设置默认存储库，Doris 将无法正常运行。一旦设置了默认存储库，就无法移除它。

语法

```
SET <vault_name> DEFAULT AS STORAGE VAULT
```

必选参数

参数名称	描述
<vault_name>	存储库的名称。这是您要设置为默认存储库的唯一标识符。

注意事项：

1. 只有 ADMIN 用户可以设置默认存储库

示例

1. 将名为 s3_vault 的存储库设置为默认存储库

```
  SET s3_vault AS DEFAULT STORAGE VAULT;
```

7.3.12.3.4 UNSET DEFAULT STORAGE VAULT

描述

取消已指定的默认 Storage Vault

语法

```
UNSET DEFAULT STORAGE VAULT
```

权限控制

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	Storage Vault	只有 admin 用户有权限执行该语句

示例

```
UNSET DEFAULT STORAGE VAULT
```

7.3.12.3.5 SHOW STORAGE VAULTS

描述

SHOW STORAGE VAULTS 命令用于显示系统中配置的所有 storage vault 的信息。 storage vault 用于管理数据外部存储位置。

语法

```
    SHOW STORAGE VAULTS
```

Return Values

此命令返回一个结果集，包含以下列：

- `StorageVaultName`: storage vault 的名称。
- `StorageVaultId`: storage vault 的 ID。
- `Properties`: 包含 storage vault 配置属性的 JSON 字符串。
- `IsDefault`: 指示该 storage vault 是否设置为默认值（TRUE 或 FALSE）。

相关命令

- CREATE STORAGE VAULT
- GRANT
- REVOKE
- SET DEFAULT STORAGE VAULT

关键词

```
SHOW, STORAGE VAULTS
```

7.3.12.3.6 CREATE STORAGE POLICY

描述

创建一个存储策略，必须先创建存储资源，然后创建迁移策略时候关联创建的存储资源名，具体可参考 RESOURCE 章节。

语法

```
CREATE STORAGE POLICY <policy_name>
PROPERTIES(
    "storage_resource" = "<storage_resource_name>"
    [{,  "cooldown_datetime" = "<cooldown_datetime>"
     | ,  "cooldown_ttl" = "<cooldown_ttl>"}]
);
```

必选参数

1. <policy_name>: 待创建的存储策略名字
2. <storage_resource_name>: 关联的存储资源名字，具体如何创建可参考 RESOURCE 章节

可选参数

1. <cooldown_datetime>: 指定创建数据迁移策略冷却的时间
2. <cooldown_ttl>: 指定创建数据迁移策略热数据持续时间

权限控制（Access Control Requirements）

执行此 SQL 命令成功的前置条件是，拥有 ADMIN_PRIV 权限，参考权限文档。

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	整个集群管理权限	除 NODE_PRIV 以外的所有权限

示例

1. 指定数据冷却时间创建数据迁移策略。

```
CREATE STORAGE POLICY testPolicy
PROPERTIES(
"storage_resource" = "s3",
"cooldown_datetime" = "2022-06-08 00:00:00"
);
```

2. 指定热数据持续时间创建数据迁移策略

```
CREATE STORAGE POLICY testPolicy
PROPERTIES(
"storage_resource" = "s3",
"cooldown_ttl" = "1d"
);
```

7.3.12.3.7 ALTER STORAGE POLICY

描述

该语句用于修改一个已有的冷热分层迁移策略。仅 root 或 admin 用户可以修改资源。

语法

```
ALTER STORAGE POLICY  '<policy_name>' PROPERTIE ("<key>"="<value>"[, ... ]);
```

必选参数

参数名称	描述
<policy_name>	存储策略的名称。这是您想要修改的存储策略的唯一标识符，必须指定一个已经存在的存储策略名称。

可选参数

参数名称	描述
retention_days	数据保留天数。定义数据在存储中保持的时间长度，超过此期限的数据将被自动删除。
redundancy_level	冗余级别。定义数据副本的数量，以确保高可用性和容错能力。例如，值为 2 表示每个数据块有两份副本
storage_type	存储类型。指定使用的存储介质，如 SSD、HDD 或混合存储。这会影响性能和成本。
cooloff_time	冷却时间。在数据被标记为可删除后，等待实际删除之前的时间间隔。用于防止误操作导致的数据丢失。
location_policy	地理位置策略。定义数据存放的地理位置，例如跨区域复制以实现灾难恢复。

示例

1. 修改名为 cooldown_datetime 冷热分层数据迁移时间点：

```
ALTER STORAGE POLICY has_test_policy_to_alter PROPERTIES("cooldown_datetime" = "2023-06-08
    ↪ 00:00:00");
```

2. 修改名为 cooldown_ttl 的冷热分层数据迁移倒计时

```
ALTER STORAGE POLICY has_test_policy_to_alter PROPERTIES ("cooldown_ttl" = "10000");
```

```
ALTER STORAGE POLICY has_test_policy_to_alter PROPERTIES ("cooldown_ttl" = "1h");
```

```
ALTER STORAGE POLICY has_test_policy_to_alter PROPERTIES ("cooldown_ttl" = "3d");
```

7.3.12.3.8 DROP STORAGE POLICY

描述

删除存储策略。存储策略的详细说明，请参阅 “存储策略” 章节。

语法

```
DROP STORAGE POLICY <policy_name>
```

必选参数

存储策略名称

权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	全局	

示例

1. 删除名字为 policy1 的存储策略

```
DROP STORAGE POLICY policy1
```

7.3.12.3.9 SHOW STORAGE POLICY

描述

查看所有/指定存储策略关联的表和分区。

语法

```
SHOW STORAGE POLICY [ USING [ FOR <storage_policy_name> ] ]
```

必选参数（Required Parameters）

<storage_policy_name>: 要查看的存储策略的名字。

权限控制

执行此 SQL 命令成功的前置条件是，拥有 ADMIN_PRIV 权限，参考权限文档。

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	整个集群管理权限	除 NODE_PRIV 以外的所有权限

示例

1. 查看所有启用了存储策略的对象。

```
show storage policy using;
```

```
+----------------------+----------------------------------------+-----------------------------------------+--
    ↪
| PolicyName           | Database                               | Table
    ↪                                  | Partitions |
+----------------------+----------------------------------------+-----------------------------------------+--
    ↪
| test_storage_policy  | regression_test_cold_heat_separation_p2 | table_with_storage_policy_
    ↪ 1            | ALL        |
| test_storage_policy  | regression_test_cold_heat_separation_p2 | partition_with_multiple_
    ↪ storage_policy | p201701    |
| test_storage_policy_2 | regression_test_cold_heat_separation_p2 | partition_with_multiple_
    ↪ storage_policy | p201702    |
| test_storage_policy_2 | regression_test_cold_heat_separation_p2 | table_with_storage_policy_
    ↪ 2            | ALL        |
| test_policy          | db2                                    | db2_test_1
    ↪                                | ALL        |
+----------------------+----------------------------------------+-----------------------------------------+--
    ↪
```

2. 查看使用存储策略 test_storage_policy 的对象。

```
show storage policy using for test_storage_policy;
```

```
+--------------------+-----------+--------------------------------+------------+
| PolicyName         | Database  | Table                          | Partitions |
+--------------------+-----------+--------------------------------+------------+
| test_storage_policy | db_1      | partition_with_storage_policy_1 | p201701    |
| test_storage_policy | db_1      | table_with_storage_policy_1     | ALL        |
+--------------------+-----------+--------------------------------+------------+
```

3. 查看所有存储策略的属性。

```
show storage policy;
```

```
+-------------+----------+---------+---------+----------------+-----------------+-------------+
    ↪
| PolicyName  | Id       | Version | Type    | StorageResource | CooldownDatetime |
    ↪ CooldownTtl |
+-------------+----------+---------+---------+----------------+-----------------+-------------+
    ↪
| test_policy | 14589252 | 0       | STORAGE | remote_s3       | -1               | 300
    ↪         |
| dev_policy  | 14589521 | 0       | STORAGE | remote_s3       | -1               | 3000
    ↪        |
+-------------+----------+---------+---------+----------------+-----------------+-------------+
    ↪
```

7.3.12.3.10 WARM UP

描述

WARM UP COMPUTE GROUP 语句用于预热计算组中的数据，以提高查询性能。预热操作可以从另一个计算组中获取资源，也可以指定特定的表和分区进行预热。预热操作返回一个作业 ID，可以用于追踪预热作业的状态。

语法

```
WARM UP COMPUTE GROUP <destination_compute_group_name> WITH COMPUTE GROUP <source_compute_group_
    ↪ name> FORCE;
```

```
WARM UP COMPUTE GROUP <destination_compute_group_name> WITH <warm_up_list>;
```

```
warm_up_list ::= warm_up_item [AND warm_up_item...];
```

```
warm_up_item ::= TABLE <table_name> [PARTITION <partition_name>];
```

参数

参数	描说明述
destination_compute_group_name	要预热的目标计算组的名称。
source_compute_group_name	从中获取资源的源集群的名称。
warm_up_list	要预热的特定项目的列表，可以包括表和分区。
table_name	用于预热的表的名称。
partition_name	用于预热的分区的名称。

返回值

- JobId: 预热作业的 ID。

示例

1. 使用名为 source_group_name 的计算组预热名为 destination_group_name 的计算组。

```
WARM UP COMPUTE GROUP destination_group_name WITH COMPUTE GROUP source_group_name;
```

2. 使用名为 destination_group 的计算组预热表 sales_data 和 customer_info 以及表 orders 的分区 q1_2024。

```
 WARM UP COMPUTE GROUP destination_group WITH
     TABLE sales_data
     AND TABLE customer_info
     AND TABLE orders PARTITION q1_2024;
```

7.3.12.3.11 CANCEL WARM UP

描述

用于在 Doris 终止指定的预热作业。

语法

```
CANCEL WARM UP JOB WHERE id = <id>;
```

必选参数

<id>

> 想要终止的预热任务的 id，可以通过命令 SHOW WARM UP JOB 查询得到。

权限控制

执行此 SQL 命令的用户必须至少具有 ADMIN_PRIV 权限。

示例

通过 SHOW WARM UP JOB 查询得到当前系统中运行的预热任务：

```
SHOW WARM UP JOB
```

其结果为：

```
+---------------+--------------+---------+-------+-------------------------+-------------+-----------+----------
    ↪
| JobId          | ClusterName   | Status  | Type  | CreateTime              | FinishBatch |
    ↪ AllBatch | FinishTime | ErrMsg |
+---------------+--------------+---------+-------+-------------------------+-------------+-----------+----------
    ↪
| 90290165739458 | CloudCluster1 | RUNNING | TABLE | 2024-11-11 11:11:42.700 | 1           | 3
    ↪        | NULL       |         |
```

```
+---------------+--------------+---------+-------+------------------------+-------------+----------+---------
    ↪
```

然后通过 CANCEL WARM UP 进行终止：

```
CANCEL WARM UP WHERE id = 90290165739458;
```

若返回以下内容则表明指定 id 对应的预热任务不存在：

```
ERROR 1105 (HY000): errCode = 2, detailMessage = job id: 110 does not exist.
```

正确返回后再次 SHOW WARM UP JOB 可以看到任务状态从 RUNNING 变更为 CANCELLED：

```
+---------------+--------------+-----------+-------+------------------------+-------------+----------+--------
    ↪
| JobId          | ClusterName   | Status    | Type  | CreateTime              | FinishBatch |
    ↪ AllBatch | FinishTime              | ErrMsg      |
+---------------+--------------+-----------+-------+------------------------+-------------+----------+--------
    ↪
| 90290165739458 | CloudCluster1 | CANCELLED | TABLE | 2024-11-11 11:11:42.700 | 1           | 3
    ↪        | 2024-11-11 11:11:43.700 | user cancel |
+---------------+--------------+-----------+-------+------------------------+-------------+----------+--------
    ↪
```

7.3.12.3.12 SHOW WARM UP JOB

描述

这些命令用于在 Doris 中显示预热作业。

语法

```
SHOW WARM UP JOB [ WHERE id = 'id' ] ;
```

参数

参数	说明
id	预热作业的 ID

示例

1. 查看所有预热作业

```
SHOW WARM UP JOB;
```

2. 查看 id 为 13418 的预热作业

```
SHOW WARM UP JOB WHERE id = 13418;
```

参考

- 管理文件缓存
- WARMUP COMPUTE GROUP

7.3.12.3.13 SHOW CACHE HOTSPOT

描述

该语句用于显示文件缓存的热点信息。

> 备注在 3.0.4 版本之前可以使用SHOW CACHE HOTSPOT语句进行缓存热度信息统计查询，从 3.0.4 版本开始不再支持使用 SHOW CACHE HOTSPOT 语句进行缓存热度信息统计查询，请直接访问系统表 __internal_schema.cloud_cache_hotspot 进行查询。具体用法请参考 MANAGING FILE CACHE

语法

```
SHOW CACHE HOTSPOT '/[<compute_group_name>/<db.table_name>]';
```

参数

参数名称	描述
	计算组的名称。 表的名称。

示例

1. 显示整个系统的缓存热点信息

```
SHOW CACHE HOTSPOT '/';
```

2. 显示特定计算组 my_compute_group 的缓存热点信息

```
SHOW CACHE HOTSPOT '/my_compute_group/';
```

参考

- MANAGING FILE CACHE
- WARMUP CACHE

7.3.13 安全合规

7.3.13.1 CREATE FILE

7.3.13.1.1 描述

该语句用于创建并上传一个文件到 Doris 集群。该功能通常用于管理一些其他命令中需要使用到的文件，如证书、公钥私钥等等。

7.3.13.1.2 语法

```
CREATE FILE <file_name>
        [ { FROM | IN } <database_name>] PROPERTIES ("<key>"="<value>" [ , ... ]);
```

7.3.13.1.3 必选参数

1. <file_name>

> 自定义文件名。

2. <key>

> 文件的属性名。- url：必须。指定一个文件的下载路径。当前仅支持无认证的 http 下载路径。命令执行成功后，文件将被保存在 doris 中，该 url 将不再需要。- catalog：必须。对文件的分类名，可以自定义。但在某些命令中，会查找指定 catalog 中的文件。比如例行导入中的，数据源为 kafka 时，会查找 catalog 名为 kafka 下的文件。- md5: 可选。文件的 md5。如果指定，会在下载文件后进行校验。

2. <value>

> 文件的属性值。

7.3.13.1.4 可选参数

1. <database_name>

文件归属于某一个 database，如果没有指定，则使用当前 session 的 database。

7.3.13.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能执行

7.3.13.1.6 注意事项

- 文件的访问规则

某个文件都归属于某一个特定的 database，对 database 拥有访问权限的用户都可以使用该文件。

- 文件大小和数量限制

这个功能主要用于管理一些证书等小文件。因此单个文件大小限制为 1MB。一个 Doris 集群最多上传 100 个文件。

7.3.13.1.7 示例

- 创建文件 ca.pem，分类为 kafka

```
CREATE FILE "ca.pem"
PROPERTIES
(
   "url" = "https://test.bj.bcebos.com/kafka-key/ca.pem",
   "catalog" = "kafka"
);
```

- 创建文件 client.key，分类为 my_catalog

```
CREATE FILE "client.key"
IN my_database
PROPERTIES
(
   "url" = "https://test.bj.bcebos.com/kafka-key/client.key",
   "catalog" = "my_catalog",
   "md5" = "b5bb901bf10f99205b39a46ac3557dd9"
);
```

- 创建文件 client_1.key，分类为 my_catalog

```
CREATE FILE "client_1.key"
FROM my_database
PROPERTIES
(
   "url" = "https://test.bj.bcebos.com/kafka-key/client.key",
   "catalog" = "my_catalog",
   "md5" = "b5bb901bf10f99205b39a46ac3557dd9"
);
```

7.3.13.2 DROP FILE

7.3.13.2.1 描述

该语句用于删除一个已上传的文件。

7.3.13.2.2 语法

```
DROP FILE "<file_name>" [ { FROM | IN } <database>] PROPERTIES ("<key>"="<value>" [ , ... ])
```

7.3.13.2.3 必选参数

1. <file_name>

> 自定义文件名。

2. <key>

文件的属性名。- catalog：必须。文件所属分类。

2. <value>

文件的属性值。

7.3.13.2.4 可选参数

1. <database>

文件归属于某一个 db，如果没有指定，则使用当前 session 的 db。

7.3.13.2.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能执行

7.3.13.2.6 示例

- 删除文件 ca.pem

```
DROP FILE "ca.pem" properties("catalog" = "kafka");
```

- 删除文件 client.key，分类为 my_catalog

```
DROP FILE "client.key"
IN my_database
```

- 删除文件 client_1.key，分类为 my_catalog

```
DROP FILE "client_1.key"
FROM my_database
```

7.3.13.3 SHOW FILE

7.3.13.3.1 描述

该语句用于展示一个 database 内创建的文件。

7.3.13.3.2 语法

```
SHOW FILE { [ FROM | IN ] <database_name>}
```

7.3.13.3.3 可选参数

1. <database_name>

> 文件归属于的 database，如果没有指定，则使用当前 session 的 database。

7.3.13.3.4 返回值

列名	说明
FileId	文件 ID，全局唯一
DbName	所属数据库名称
Catalog	自定义分类
FileName	文件名
FileSize	文件大小，单位字节
IsContent	是否有内容
MD5	文件的 MD5

7.3.13.3.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有文件所属数据库的访问权限就能执行此指令

7.3.13.3.6 示例

- 查看 session 数据库中已上传的文件

```
SHOW FILE;
```

```
+-------+--------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
| Id    | DbName | Catalog  | FileName                | FileSize | IsContent | MD5
    ↪                               |
+-------+--------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
| 12006 | testdb | doris_be | doris_be_metadata_layout | 89349    | true      | 9
    ↪ a3f68160b4106b0e923a0aa2fc05599 |
+-------+--------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
```

- 查看数据库 example_db 中已上传的文件

```
SHOW FILE FROM example_db;
```

```
+-------+------------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
| Id    | DbName     | Catalog  | FileName                | FileSize | IsContent | MD5
    ↪                               |
+-------+------------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
| 12007 | example_db | doris_fe | doris_fe_metadata_layout | 569373   | true      | 10385505
    ↪ d3c0d03f085fea6f8d51adfa |
| 12008 | example_db | doris_be | doris_be_metadata_layout | 89349    | true      | 9
    ↪ a3f68160b4106b0e923a0aa2fc05599 |
+-------+------------+----------+-------------------------+----------+-----------+----------------------------------
    ↪
```

7.3.13.4 CREATE ENCRYPTKEY

7.3.13.4.1 描述

此语句创建一个自定义密钥。

7.3.13.4.2 语法

```
CREATE ENCRYPTKEY <key_name> AS "<key_string>"
```

7.3.13.4.3 必选参数

1. <key_name>

要创建密钥的名字，可以包含数据库的名字。比如：db1.my_key。

2. <key_string>

要创建密钥的字符串。如果 key_name 中包含了数据库名字，那么这个自定义密钥会创建在对应的数据库中，否则这个函数将会创建在当前会话所在的数据库。新密钥的名字不能够与对应数据库中已存在的密钥相同，否则会创建失败。

7.3.13.4.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	需对目标用户或角色持有 ADMIN_PRIV 权限方可创建自定义密钥

7.3.13.4.5 示例

- 创建一个自定义密钥

```
CREATE ENCRYPTKEY my_key AS "ABCD123456789";
```

- 在 testdb 数据库下创建一个自定义密钥

```
CREATE ENCRYPTKEY testdb.test_key AS "ABCD123456789";
```

- 使用自定义密钥加密

使用自定义密钥需在密钥前添加关键字 KEY/key，与 key_name 空格隔开。

```
SELECT HEX(AES_ENCRYPT("Doris is Great", KEY my_key));
```

```
+-----------------------------------------------+
| hex(aes_encrypt('Doris is Great', key my_key)) |
```

```
+-------------------------------------------------+
| D26DB38579D6A343350EDDC6F2AD47C6                |
+-------------------------------------------------+
```

- 使用自定义密钥解密

```
SELECT AES_DECRYPT(UNHEX('D26DB38579D6A343350EDDC6F2AD47C6'), KEY my_key);
```

```
+------------------------------------------------- -------------------+
| aes_decrypt(unhex('D26DB38579D6A343350EDDC6F2AD47C6'), key my_key)  |
+------------------------------------------------- -------------------+
| Doris is Great                                                      |
+------------------------------------------------- -------------------+
```

7.3.13.5 DROP ENCRYPTKEY

7.3.13.5.1 描述

此语句用于删除一个自定义密钥。密钥的名字完全一致才能够被删除。

7.3.13.5.2 语法

```
DROP ENCRYPTKEY [IF EXISTS] <key_name>
```

7.3.13.5.3 必选参数

1. <key_name>

> 要删除密钥的名字，可以包含数据库的名字。比如：db1.my_key。

7.3.13.5.4 可选参数

1. [IF EXISTS]

> 如果密钥不存在，则不执行任何操作。

7.3.13.5.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	用户或者角色拥有 ADMIN_PRIV 权限才能进行删除密钥操作

7.3.13.5.6 示例

- 删除掉一个密钥

```
DROP ENCRYPTKEY my_key;
```

- 删除掉一个密钥，如果密钥不存在则不执行任何操作

```
DROP ENCRYPTKEY IF EXISTS testdb.my_key;
```

7.3.13.6 SHOW ENCRYPTKEY

7.3.13.6.1 描述

查看数据库下所有的自定义的密钥。

7.3.13.6.2 语法

```
SHOW ENCRYPTKEYS [ { IN | FROM } <db> ] [ LIKE '<key_pattern>']
```

7.3.13.6.3 可选参数

1. <db>

> 要查询的数据库名字。比如：db1.my_key。

2. <key_pattern>

> 用来过滤密钥名称的参数。

7.3.13.6.4 返回值

列名	说明
EncryptKey Name	密钥名称
EncryptKey String	密钥的值

7.3.13.6.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	用户（User）或角色（Role）	需对目标用户或角色持有 ADMIN_PRIV 权限才能查看密钥

7.3.13.6.6 注意事项

- 如果用户指定了数据库，那么查看对应数据库的，否则直接查询当前会话所在数据库。

7.3.13.6.7 示例

- 查看当前会话所在数据库下所有的密钥

```
SHOW ENCRYPTKEYS;
```

```
+-----------------+-------------------+
| EncryptKey Name | EncryptKey String |
+-----------------+-------------------+
| testdb.test_key | ABCD123456789     |
+-----------------+-------------------+
```

- 查看指定数据库下所有的密钥

```
SHOW ENCRYPTKEYS FROM example_db ;
```

```
+---------------------+-------------------+
| EncryptKey Name     | EncryptKey String |
+---------------------+-------------------+
| example_db.my_key   | ABCD123456789     |
| example_db.test_key | ABCD123456789     |
+---------------------+-------------------+
```

- 查看指定数据库下匹配指定密钥名称的密钥

```
SHOW ENCRYPTKEYS FROM example_db LIKE "%my%";
```

```
+------------------+------------------+
| EncryptKey Name  | EncryptKey String |
+------------------+------------------+
| example_db.my_key | ABCD123456789    |
+------------------+------------------+
```

7.3.14 数据治理

7.3.14.1 CREATE ROW POLICY

7.3.14.1.1 描述

创建行安全策略，Explain 可以查看改写后的执行计划。

7.3.14.1.2 语法

```
CREATE ROW POLICY [ IF NOT EXISTS ] <policy_name>
ON <table_name>
AS { RESTRICTIVE | PERMISSIVE }
TO { <user_name> | ROLE <role_name> }
USING (<filter>);
```

7.3.14.1.3 必选参数

1. <policy_name>: 行安全策略名称
2. <table_name>: 表名称
3. <filter_type>: RESTRICTIVE 将一组策略通过 AND 连接，PERMISSIVE 将一组策略通过 OR 连接
4. <filter>: 相当于查询语句的过滤条件，例如：id=1

7.3.14.1.4 可选参数

1. <user_name>: 用户名称，不允许对 root 和 admin 用户创建
2. <role_name>: 角色名称

7.3.14.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV 或 GRANT_PRIV	全局	

7.3.14.1.6 示例

1. 创建一组行安全策略

```
CREATE ROW POLICY test_row_policy_1 ON test.table1
AS RESTRICTIVE TO test USING (c1 = 'a');
CREATE ROW POLICY test_row_policy_2 ON test.table1
AS RESTRICTIVE TO test USING (c2 = 'b');
CREATE ROW POLICY test_row_policy_3 ON test.table1
AS PERMISSIVE TO test USING (c3 = 'c');
CREATE ROW POLICY test_row_policy_3 ON test.table1
AS PERMISSIVE TO test USING (c4 = 'd');
```

当我们执行对 `table1` 的查询时被改写后的 `sql` 为

```
SELECT * FROM (SELECT * FROM table1 WHERE (c1 = 'a' AND c2 = 'b') AND (c3 = 'c' OR c4 = 'd'))
```

7.3.14.2 DROP ROW POLICY

7.3.14.2.1 描述

删除行安全策略。

7.3.14.2.2 语法

```
DROP ROW POLICY <policy_name> on <table_name>
  [ FOR { <user_name> | ROLE <role_name> } ];
```

7.3.14.2.3 必选参数

1. `<policy_name>`: 行安全策略名称
2. `<table_name>`: 表名称

7.3.14.2.4 可选参数

1. `<user_name>`: 用户名称
2. `<role_name>`: 角色名称

7.3.14.2.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV 或 GRANT_PRIV	全局	

7.3.14.2.6 示例

1. 删除 db1.table1 的 policy1 行安全策略

```
DROP ROW POLICY policy1 ON db1.table1
```

2. 删除 db1.table1 作用于 user1 的 policy1 行安全策略

```
DROP ROW POLICY policy1 ON db1.table1 FOR user1
```

3. 删除 db1.table1 作用于 role1 的 policy1 行安全策略

```
DROP ROW POLICY policy1 ON db1.table1 FOR role role1
```

7.3.14.3 SHOW ROW POLICY

7.3.14.3.1 描述

查看行安全策略。

7.3.14.3.2 语法

```
SHOW ROW POLICY [ FOR { <user_name> | ROLE <role_name> } ];
```

7.3.14.3.3 可选参数

1. <user_name>: 用户名称
2. <role_name>: 角色名称

7.3.14.3.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	全局	

7.3.14.3.5 示例

1. 查看所有安全策略

```
SHOW ROW POLICY;
```

2. 指定用户名查询

```
SHOW ROW POLICY FOR user1;
```

3. 指定角色名查询

```
SHOW ROW POLICY for role role1;
```

7.3.15 后台任务

7.3.15.1 CREATE JOB

7.3.15.1.1 描述

Doris Job 是根据既定计划运行的任务，用于在特定时间或指定时间间隔触发预定义的操作，从而帮助我们自动执行一些任务。从功能上来讲，它类似于操作系统上的定时任务（如：Linux 中的 cron、Windows 中的计划任务）。

Job 有两种类型：ONE_TIME 和 RECURRING。其中 ONE_TIME 类型的 Job 会在指定的时间点触发，它主要用于一次性任务，而 RECURRING 类型的 Job 会在指定的时间间隔内循环触发，此方式主要用于周期性执行的任务。RECURRING 类型的 Job 可指定开始时间，结束时间，即 STARTS/ENDS, 如果不指定开始时间，则默认首次执行时间为当前时间加一次调度周期。如果指定结束时间，则 task 执行完成如果达到结束时间（或超过，或下次执行周期会超过结束时间）则更新为 FINISHED 状态，此时不会再产生 Task。

JOB 共 4 种状态（RUNNING,STOPPED,PAUSED,FINISHED），初始状态为 RUNNING，RUNNING 状态的 JOB 会根据既定的调度周期去生成 TASK 执行，Job 执行完成达到结束时间则状态变更为 FINISHED.

PAUSED 状态的 JOB 可以通过 RESUME 操作来恢复运行，更改为 RUNNING 状态。

STOPPED 状态的 JOB 由用户主动触发，此时会 Cancel 正在运行中的作业，然后删除 JOB。

Finished 状态的 JOB 会保留在系统中 24 H，24H 后会被删除。

JOB 只描述作业信息，执行会生成 TASK，TASK 状态分为 PENDING，RUNNING，SUCCEESS,FAILED,CANCELED. PENDING 表示到达触发时间了但是等待资源 RUN，分配到资源后状态变更为 RUNNING，执行成功/失败即变更为 SUCCESS ↪ /FAILED. CANCELED 即取消状态，TASK 持久化最终状态，即 SUCCESS/FAILED，其他状态运行中可以查到，但是如果重启则不可见。

7.3.15.1.2 语法

```
CREATE
    JOB
    <job_name>
    ON SCHEDULE <schedule>
    [ COMMENT <string> ]
    DO <sql_body> ;
```

其中：

```
schedule:
    {  AT <at_timestamp> | EVERY <interval> [STARTS <start_timestamp> ] [ENDS <end_timestamp> ]
        ↪ }
```

其中：

```
interval:
    quantity { WEEK | DAY | HOUR | MINUTE }
```

7.3.15.1.3 必选参数

1. <job_name> > 作业名称，它在一个 db 中标识唯一事件。JOB 名称必须是全局唯一的，如果已经存在同名的 JOB，则会报错。我们保留了 inner_ 前缀在系统内部使用，因此用户不能创建以 inner_ 开头的名称。

2. <schedule> > ON SCHEDULE 子句，指定了 Job 作业的类型和触发时间以及频率，它可以指定一次性作业或者周期性作业。

3. <sql_body> > DO 子句，它指定了 Job 作业触发时需要执行的操作，即一条 SQL 语句。

7.3.15.1.4 可选参数

1. AT <at_timestamp> > 格式： 'YYYY-MM-DD HH:MM:SS'，用于一次性事件，它指定事件仅在给定的日期和时间执行一次 timestamp，当执行完成后，JOB 状态会变更为 FINISHED。

2. EVERY <interval> > 表示定期重复操作，它指定了作业的执行频率，关键字后面要指定一个时间间隔，该时间间隔可以是天、小时、分钟、秒、周。

3. STARTS <start_timestamp> > 格式： 'YYYY-MM-DD HH:MM:SS'，用于指定作业的开始时间，如果没有指定，则从当前时间的下一个时间点开始执行。开始时间必须大于当前时间。

4. ENDS <end_timestamp> > 格式： 'YYYY-MM-DD HH:MM:SS'，用于指定作业的结束时间，如果没有指定，则表示永久执行。该日期必须大于当前时间，如果指定了开始时间，即 STARTS，则结束时间必须大于开始时间。

7.3.15.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库（DB）	目前仅支持 ADMIN 权限执行此操作

7.3.15.1.6 注意事项

- TASK 只保留最新的 100 条记录。
- 目前仅支持 INSERT 内表操作，后续会支持更多的操作。
- 当下一个计划任务时间到期，即需要调度任务执行时，如果当前 JOB 仍有历史任务正在执行，则会跳过当前任务调度。因此控制一个合理的执行间隔非常重要。

7.3.15.1.7 示例

- 创建一个名为 my_job 的作业，每分钟执行一次，执行的操作是将 db2.tbl2 中的数据导入到 db1.tbl1 中。

```
CREATE JOB my_job ON SCHEDULE EVERY 1 MINUTE DO INSERT INTO db1.tbl1 SELECT * FROM db2.tbl2;
```

- 创建一个一次性的 Job，它会在 2020-01-01 00:00:00 时执行一次，执行的操作是将 db2.tbl2 中的数据导入到 db1.tbl1 中。

```
CREATE JOB my_job ON SCHEDULE AT '2020-01-01 00:00:00' DO INSERT INTO db1.tbl1 SELECT * FROM
    ↪ db2.tbl2;
```

- 创建一个周期性的 Job，它会在 2020-01-01 00:00:00 时开始执行，每天执行一次，执行的操作是将 db2.tbl2 中的数据导入到 db1.tbl1 中。

```
CREATE JOB my_job ON SCHEDULE EVERY 1 DAY STARTS '2020-01-01 00:00:00' DO INSERT INTO db1.
    ↪ tbl1 SELECT * FROM db2.tbl2 WHERE  create_time >=  days_add(now(),-1);
```

- 创建一个周期性的 Job，它会在 2020-01-01 00:00:00 时开始执行，每天执行一次，执行的操作是将 db2.tbl2 中的数据导入到 db1.tbl1 中，该 Job 在 2020-01-01 00:10:00 时结束。

```
CREATE JOB my_job ON SCHEDULE EVERY 1 DAY STARTS '2020-01-01 00:00:00' ENDS '2020-01-01
    ↪ 00:10:00' DO INSERT INTO db1.tbl1 SELECT * FROM db2.tbl2 create_time >=  days_add(now
    ↪ (),-1);
```

7.3.15.1.8 最佳实践

- 合理的进行 Job 的管理，避免大量的 Job 同时触发，导致任务堆积，从而影响系统的正常运行。
- 任务的执行间隔应该设置在一个合理的范围，至少应该大于任务执行时间。

7.3.15.1.9 相关文档

- 暂停 -JOB
- 恢复 -JOB
- 删除 -JOB
- 查询 -JOB
- 查询 -TASKS

7.3.15.1.10 CONFIG

fe.conf

- job_dispatch_timer_job_thread_num, 用于分发定时任务的线程数，默认值 2，如果含有大量周期执行任务，可以调大这个参数。
- job_dispatch_timer_job_queue_size, 任务堆积时用于存放定时任务的队列大小，默认值 1024. 如果有大量任务同一时间触发，可以调大这个参数。否则会导致队列满，提交任务会进入阻塞状态，从而导致后续任务无法提交。
- finished_job_cleanup_threshold_time_hour, 用于清理已完成的任务的时间阈值，单位为小时，默认值为 24 小时。
- job_insert_task_consumer_thread_num = 10; 用于执行 Insert 任务的线程数，值应该大于 0，否则默认为 5.

7.3.15.2 PAUSE JOB

7.3.15.2.1 描述

用户暂停一个正在 RUNNING 状态的 JOB，正在运行的 TASK 会被中断，JOB 状态变更为 PAUSED。被停止的 JOB 可以通过 RESUME 操作恢复运行。

7.3.15.2.2 语法

```
PAUSE JOB WHERE jobname = <job_name> ;
```

7.3.15.2.3 必选参数

1. <job_name> > 暂停任务的作业名称。

7.3.15.2.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库（DB）	目前仅支持 ADMIN 权限执行此操作

7.3.15.2.5 示例

- 暂停名称为 example 的 JOB。

```
      PAUSE JOB where jobname='example';
```

7.3.15.3 DROP JOB

7.3.15.3.1 描述

用户删除一个 JOB 作业。作业会被立即停止同时删除。

7.3.15.3.2 语法

```
DROP JOB where jobName = <job_name> ;
```

7.3.15.3.3 必选参数

1. <job_name> > 删除任务的作业名称。

7.3.15.3.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库（DB）	目前仅支持 ADMIN 权限执行此操作

7.3.15.3.5 示例

- 删除名称为 example 的作业。

```
  DROP JOB where jobName='example';
```

7.3.15.4 RESUME JOB

7.3.15.4.1 描述

将处于 PAUSED 状态的 JOB 恢复为 RUNNING 状态。 RUNNING 状态的 JOB 将会根据既定的调度周期去执行。

7.3.15.4.2 语法

```
RESUME JOB where jobName = <job_name> ;
```

7.3.15.4.3 必选参数

1. <job_name> > 恢复任务的作业名称。

7.3.15.4.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
ADMIN_PRIV	数据库（DB）	目前仅支持 ADMIN 权限执行此操作

7.3.15.4.5 示例

- 恢复运行名称为 example 的 JOB。

```
RESUME JOB where jobName= 'example';
```

7.3.15.5 CANCEL TASK

7.3.15.5.1 描述

取消通过CREATE JOB 语句创建生成的正在运行中任务

- 任务必须是通过 CREATE JOB 语句创建生成的任务
- 必须是正在运行中的任务
- 该函数自 2.1.0 版本支持。

7.3.15.5.2 语法

```
CANCEL TASK WHERE jobName = '<job_name>' AND taskId = '<task_id>';
```

7.3.15.5.3 必选参数

1. <job_name>: 作业名称，字符串类型。
2. <task_id>: 任务 ID，整型类型。可通过 tasks 表值函数查询。如：SELECT * FROM tasks(‘type’ = ‘insert’)。详细信息请参阅 “task 表值函数”。

7.3.15.5.4 权限控制

执行此 SQL 命令的用户必须至少具有 ADMIN_PRIV 权限。

7.3.15.5.5 示例

取消一个 jobName 是 example，taskID 是 378912 的后台任务。

```
CANCEL TASK WHERE jobName='example' AND taskId=378912
```

7.3.16 插件

7.3.16.1 INSTALL PLUGIN

7.3.16.1.1 描述

该语句用于安装一个插件

7.3.16.1.2 语法

```
INSTALL PLUGIN FROM <source> [PROPERTIES ("<key>"="<value>", ...)]
```

7.3.16.1.3 必选参数

** 1. <source>** > 待安装插件路径，支持三种类型：> 1. 指向一个 zip 文件的绝对路径。> 2. 指向一个插件目录的绝对路径。> 3. 指向一个 http 或 https 协议的 zip 文件下载路径

7.3.16.1.4 可选参数

** 1. [PROPERTIES ("<key>"="<value>", ...)]** > 用于指定安装插件时的属性或参数

7.3.16.1.5 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	整个集群	需要对整个集群具有管理权限

7.3.16.1.6 注意事项

注意需要放置一个和.zip 文件同名的 md5 文件，如 http://mywebsite.com/plugin.zip.md5。其中内容为.zip 文件的 MD5 值。

7.3.16.1.7 示例

- 安装一个本地 zip 文件插件：

```
    INSTALL PLUGIN FROM "/home/users/doris/auditdemo.zip";
```

- 安装一个本地目录中的插件：

```
INSTALL PLUGIN FROM "/home/users/doris/auditdemo/";
```

- 下载并安装一个插件：

```
INSTALL PLUGIN FROM "http://mywebsite.com/plugin.zip";
```

- 下载并安装一个插件，同时设置了 zip 文件的 md5sum 的值：

```
INSTALL PLUGIN FROM "http://mywebsite.com/plugin.zip" PROPERTIES("md5sum" = "73877
    ↪ f6029216f4314d712086a146570");
```

7.3.16.2 UNINSTALL PLUGIN

7.3.16.2.1 描述

该语句用于卸载一个插件

7.3.16.2.2 语法：

```
UNINSTALL PLUGIN <plugin_name>;
```

7.3.16.2.3 必选参数

** 1. <plugin_name>** > 卸载插件的名称

7.3.16.2.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	整个集群	需要对整个集群具有管理权限

7.3.16.2.5 注意事项

只能卸载非 builtin 的插件。

7.3.16.2.6 示例

- 卸载一个插件：

```
UNINSTALL PLUGIN auditdemo;
```

7.3.16.3 SHOW PLUGINS

7.3.16.3.1 描述

该语句用于展示已安装的插件

7.3.16.3.2 语法

```
SHOW PLUGINS
```

7.3.16.3.3 返回值

列	描述
Description	对应插件描述
Version	插件对应版本号
JavaVersion	对应 Java 版本号
ClassName	程序类名
SoName	程序共享对象名称
Sources	插件来源
Status	安装状态
Properties	插件属性

7.3.16.3.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限	对象	说明
ADMIN_PRIV	整个集群	需要对整个集群具有管理权限

7.3.16.3.5 示例

- 展示已安装的插件：

```
SHOW PLUGINS;
```

```
+------------------------------+---------+-------------------------------------------------------------------+-------
    ↪
| Name                         | Type    | Description
    ↪                                                 | Version | JavaVersion | ClassName
```

```
    ↪                                                    | SoName | Sources | Status    |
    ↪ Properties |
+------------------------------+---------+-----------------------------------------------------------+-------
    ↪
| __builtin_AuditLoader          | AUDIT   | builtin audit loader, to load audit log to
    ↪ internal table | 2.1.0   | 1.8.31      | org.apache.doris.plugin.audit.AuditLoader
    ↪                   | NULL   | Builtin | INSTALLED | {}         |
| __builtin_AuditLogBuilder     | AUDIT   | builtin audit logger
    ↪                                      | 0.12.0  | 1.8.31      | org.apache.doris.plugin
    ↪ .audit.AuditLogBuilder              | NULL   | Builtin | INSTALLED | {}         |
| __builtin_SqlDialectConverter | DIALECT | builtin sql dialect converter
    ↪                               | 2.1.0   | 1.8.31      | org.apache.doris.plugin.dialect.
    ↪ HttpDialectConverterPlugin | NULL   | Builtin | INSTALLED | {}         |
+------------------------------+---------+-----------------------------------------------------------+-------
    ↪
```

7.3.17 字符集

7.3.17.1 SHOW COLLATION

7.3.17.1.1 描述

在 Doris 中，`SHOW COLLATION` 命令用于显示数据库中可用的字符集校对。校对是一组决定数据如何排序和比较的规则。这些规则会影响字符数据的存储和检索。

7.3.17.1.2 语法

```
SHOW COLLATION
```

7.3.17.1.3 返回值

列名	说明
Collation	校对名称
Charset	字符集
Id	校对的 ID
Default	是否是该字符集的默认校对
Compiled	是否已编译
Sortlen	排序长度

7.3.17.1.4 注意事项

在 Doris 中，虽然兼容 MySQL 的设置 collation 的命令。但是实际并不会生效。执行时，永远会使用 utf8mb4_0900_bin 作为比较规则。

7.3.17.1.5 示例

```
SHOW COLLATION;
```

```
+--------------------+---------+------+---------+----------+---------+
| Collation          | Charset | Id   | Default | Compiled | Sortlen |
+--------------------+---------+------+---------+----------+---------+
| utf8mb4_0900_bin   | utf8mb4 |  309 | Yes     | Yes      |       1 |
| utf8mb3_general_ci | utf8mb3 |   33 | Yes     | Yes      |       1 |
+--------------------+---------+------+---------+----------+---------+
```

7.3.17.2 SHOW CHARSET

7.3.17.2.1 描述

“SHOW CHARSET”命令用于显示当前数据库管理系统中可用的字符集（character set）以及与每个字符集相关联的一些属性。

这些属性可能包括字符集的名称、默认排序规则、最大字节长度等。通过运行“SHOW CHARSET”命令，可以查看系统中支持的字符集列表及其详细信息。

7.3.17.2.2 语法

```
SHOW CHARSET
```

7.3.17.2.3 返回值

列名	说明
Charset	字符集
Description	描述
Default Collation	默认校对名称
Maxlen	最大字节长度

7.3.17.2.4 示例

```
SHOW CHARSET;
```

```
+---------+---------------+-------------------+--------+
| Charset | Description   | Default collation | Maxlen |
+---------+---------------+-------------------+--------+
| utf8mb4 | UTF-8 Unicode | utf8mb4_0900_bin  | 4      |
+---------+---------------+-------------------+--------+
```

7.3.18 类型

7.3.18.1 SHOW DATA TYPES

7.3.18.1.1 描述

该语句用于查看 DORIS 支持的所有数据类型。

7.3.18.1.2 语法

```
SHOW DATA TYPES;
```

7.3.18.1.3 返回值

列名	说明
TypeName	类型名称
Size	字节大小

7.3.18.1.4 权限控制

执行此 SQL 命令的用户不需要具有特定的权限

7.3.18.1.5 示例

- 查看 Doris 支持的所有数据类型

```
SHOW DATA TYPES;
```

```
+----------------+------+
| TypeName       | Size |
+----------------+------+
| AGG_STATE      | 16   |
| ARRAY          | 32   |
| BIGINT         | 8    |
| BITMAP         | 16   |
| BOOLEAN        | 1    |
| CHAR           | 16   |
| DATE           | 16   |
| DATETIME       | 16   |
| DATETIMEV2     | 8    |
| DATEV2         | 4    |
| DECIMAL128     | 16   |
| DECIMAL32      | 4    |
```

```
| DECIMAL64       | 8    |
| DECIMALV2       | 16   |
| DOUBLE          | 8    |
| FLOAT           | 4    |
| HLL             | 16   |
| INT             | 4    |
| IPV4            | 4    |
| IPV6            | 16   |
| JSON            | 16   |
| LARGEINT        | 16   |
| MAP             | 24   |
| QUANTILE_STATE  | 16   |
| SMALLINT        | 2    |
| STRING          | 16   |
| TINYINT         | 1    |
| VARCHAR         | 16   |
+-----------------+------+
```

7.3.18.2 SHOW TYPECAST

7.3.18.2.1 描述

查看数据库下所有的类型转换。

```
SHOW TYPE_CAST [ IN | FROM <db>];
```

7.3.18.2.2 必选参数

1. <db>

查询的数据库名称

7.3.18.2.3 返回值

列名	说明
Origin Type	原始类型
Cast Type	转换类型

7.3.18.2.4 权限控制

执行此 SQL 命令的用户必须至少具有以下权限：

权限（Privilege）	对象（Object）	说明（Notes）
Select_priv	库（DB）	用户或者角色对于 DB 拥护 Select_Priv 才能查看数据库下所有类型的转换

权限（Privilege）	对象（Object）	说明（Notes）

7.3.18.2.5 注意事项

- 如果用户指定了数据库，那么查看对应数据库的，否则直接查询当前会话所在数据库

7.3.18.2.6 示例

- 查看数据库 TESTDB 下所有的类型转换

```
SHOW TYPE_CAST IN TESTDB;
```

```
+----------------+----------------+
| Origin Type    | Cast Type      |
+----------------+----------------+
| DATETIMEV2     | BOOLEAN        |
| DATETIMEV2     | TINYINT        |
| DATETIMEV2     | SMALLINT       |
| DATETIMEV2     | INT            |
| DATETIMEV2     | BIGINT         |
| DATETIMEV2     | LARGEINT       |
| DATETIMEV2     | FLOAT          |
| DATETIMEV2     | DOUBLE         |
| DATETIMEV2     | DATE           |
| DATETIMEV2     | DATETIME       |
| DATETIMEV2     | DATEV2         |
| DATETIMEV2     | DATETIMEV2     |
| DATETIMEV2     | DECIMALV2      |
| DATETIMEV2     | DECIMAL32      |
| DATETIMEV2     | DECIMAL64      |
| DATETIMEV2     | DECIMAL128     |
| DATETIMEV2     | DECIMAL256     |
| DATETIMEV2     | VARCHAR        |
| DATETIMEV2     | STRING         |
| DECIMAL256     | DECIMAL128     |
| DECIMAL256     | DECIMAL256     |
| DECIMAL256     | VARCHAR        |
| DECIMAL256     | STRING         |
+----------------+----------------+
```

7.3.19 系统信息和帮助

7.3.19.1 SHOW PROC

7.3.19.1.1 描述

Proc 系统是 Doris 的一个比较有特色的功能。使用过 Linux 的同学可能比较了解这个概念。在 Linux 系统中，proc 是一个虚拟的文件系统，通常挂载在 /proc 目录下。用户可以通过这个文件系统来查看系统内部的数据结构。比如可以通过 /proc/pid 查看指定 pid 进程的详细情况。

和 Linux 中的 proc 系统类似，Doris 中的 proc 系统也被组织成一个类似目录的结构，根据用户指定的“目录路径（proc 路径）”，来查看不同的系统信息。

proc 系统被设计为主要面向系统管理人员，方便其查看系统内部的一些运行状态。如表的 tablet 状态、集群均衡状态、各种作业的状态等等。是一个非常实用的功能

Doris 中有两种方式可以查看 proc 系统。

1. 通过 Doris 提供的 WEB UI 界面查看，访问地址：`http://FE_IP:FE_HTTP_PORT`
2. 另外一种方式是通过命令

通过 `SHOW PROC "/";` 可看到 Doris PROC 支持的所有命令

通过 MySQL 客户端连接 Doris 后，可以执行 SHOW PROC 语句查看指定 proc 目录的信息。proc 目录是以“/”开头的绝对路径。

show proc 语句的结果以二维表的形式展现。而通常结果表的第一列的值为 proc 的下一级子目录。

```
mysql> show proc "/";
+---------------------------+
| name                      |
+---------------------------+
| auth                      |
| backends                  |
| bdbje                     |
| brokers                   |
| catalogs                  |
| cluster_balance           |
| cluster_health            |
| colocation_group          |
| current_backend_instances |
| current_queries           |
| current_query_stmts       |
| dbs                       |
| diagnose                  |
| frontends                 |
| jobs                      |
| load_error_hub            |
| monitor                   |
| resources                 |
| routine_loads             |
| statistic                 |
| stream_loads              |
```

```
| tasks                     |
| transactions              |
| trash                     |
+---------------------------+
23 rows in set (0.00 sec)
```

说明：

1. auth：用户名称及对应的权限信息
2. backends：显示集群中 BE 的节点列表，等同于 SHOW BACKENDS

3. bdbje：查看 bdbje 数据库列表，需要修改 `fe.conf` 文件增加 `enable_bdbje_debug_mode=true`，然后通过 `sh start_fe.sh --daemon` 启动 FE 即可进入 debug 模式。进入 debug 模式之后，仅会启动 `http server` 和 `MySQLServer` 并打开 BDBJE 实例，但不会进入任何元数据的加载及后续其他启动流程，
4. binlog: 查看 binlog 相关信息，包括 binlog 记录数、字节数、时间范围等信息。
5. brokers : 查看集群 Broker 节点信息，等同于 SHOW BROKER
6. catalogs : 查看当前已创建的数据目录，等同于 SHOW CATALOGS
7. cluster_balance：查看集群均衡情况，具体参照数据副本管理
8. cluster_health : 通过 SHOW PROC ‘/cluster_health/tablet_health’; 命令可以查看整个集群的副本状态。
9. colocation_group : 该命令可以查看集群内已存在的 Group 信息，具体可以查看Colocation Join 章节
10. current_backend_instances：显示当前正在执行作业的 be 节点列表
11. current_queries : 查看正在执行的查询列表，当前正在运行的 SQL 语句。

12. current_query_stmts : 返回当前正在执行的 query。
13. dbs：主要用于查看 Doris 集群中各个数据库以及其中的表的元数据信息。这些信息包括表结构、分区、物化视图、数据分片和副本等等。通过这个目录和其子目录，可以清楚的展示集群中的表元数据情况，以及定位一些如数据倾斜、副本故障等问题
14. diagnose : 报告和诊断集群中的常见管控问题，主要包括副本均衡和迁移、事务异常等
15. frontends：显示集群中所有的 FE 节点信息，包括 IP 地址、角色、状态、是否是 master 等，等同于 SHOW FRONTENDS
16. jobs：各类任务的统计信息，可查看指定数据库的 Job 的统计信息，如果 `dbId = -1`, 则返回所有库的汇总信息
17. load_error_hub：Doris 支持将 load 作业产生的错误信息集中存储到一个 error hub 中。然后直接通过 SHOW LOAD WARNINGS; 语句查看错误信息。这里展示的就是 error hub 的配置信息。
18. monitor : 显示的是 FE JVM 的资源使用情况
19. resources : 查看系统资源，普通账户只能看到自己有 USAGE_PRIV 使用权限的资源。只有 root 和 admin 账户可以看到所有的资源。等同于 SHOW RESOURCES
20. routine_loads：显示所有的 routine load 作业信息，包括作业名称、状态等
21. statistics：主要用于汇总查看 Doris 集群中数据库、表、分区、分片、副本的数量。以及不健康副本的数量。这个信息有助于我们总体把控集群元信息的规模。帮助我们从整体视角查看集群分片情况，能够快速查看集群分片的健康情况。从而进一步定位有问题的数据分片。
22. stream_loads : 返回当前正在执行的 stream load 任务。
23. tasks : 显示现在各种作业的任务总量，及失败的数量。
24. transactions：用于查看指定 transaction id 的事务详情，等同于 SHOW TRANSACTION
25. trash：该语句用于查看 backend 内的垃圾数据占用空间。等同于 SHOW TRASH

7.3.19.1.2 详细说明

1. /binlog

Binlog 是 Doris 中的一项重要功能，用于记录数据变更，可用于跨集群数据同步（CCR）等场景。通过此命令，管理员可以监控 Binlog 的状态，确保其正常运行并合理规划存储空间。

字段名	数据类型	描述
Name	字符串	数据库或表的名称。
Type	字符串	数据对象的类型，取值“db”（数据库）或“table”（表）
Id	数字	数据库 ID 或表 ID

字段名	数据类型	描述
Dropped	布尔值	该数据库或表是否已被删除。值为“true”表示该对象已从 Doris 中删除，但其 Binlog 记录仍然保留；值为“false”表示该对象仍然存在于系统中。即使数据库或表被删除，系统仍然会保留一段时间的 Binlog 记录，直到 TTL 到期或被手动清理
BinlogLength	数字	该数据库或表的二进制日志条目总数

字段名	数据类型	描述
BinlogSize	数字	二进制日志的总大小（字节）
FirstBinlogCommittedTime	数字	第一条二进制日志的提交时间戳（Unix 时间戳，毫秒）
ReadableFirstBinlogCommittedTime	字符串	第一条二进制日志的提交时间（可读格式）
LastBinlogCommittedTime	数字	最后一条二进制日志的提交时间戳（Unix 时间戳，毫秒）
ReadableLastBinlogCommittedTime	字符串	最后一条二进制日志的提交时间（可读格式）

字段名	数据类型	描述
BinlogTtlSeconds	数字	二进制日志的生存时间（秒），超过此时间的日志可能会被清理
BinlogMaxBytes	数字	二进制日志的最大大小（字节），超过此大小可能会触发清理
BinlogMaxHistoryNums	数字	保留的二进制日志最大历史记录数，超过此数量可能会触发清理

注意事项：- 只有启用了二进制日志功能的数据库和表才会在此命令的输出中显示 - 对于数据库级别，如果启用了数据库级的二进制日志，则显示数据库的二进制日志信息；否则，显示该数据库下启用了二进制日志的各个表的信息

7.3.19.1.3 示例

1. 如“/dbs”展示所有数据库，而“/dbs/10002”展示 id 为 10002 的数据库下的所有表

```
mysql> show proc "/dbs/10002";
+--
    ↪ -------+----------------------+----------+--------------------+-------------+--------+------+-------
    ↪
| TableId | TableName            | IndexNum | PartitionColumnName | PartitionNum | State  |
    ↪ Type | LastConsistencyCheckTime | ReplicaCount |
```

```
+--
    ↪ -------+----------------------+----------+---------------------+--------------+---------+------+-------
    ↪
| 10065   | dwd_product_live     | 1        | dt                  | 9            | NORMAL |
    ↪ OLAP | NULL                 | 18           |
| 10109   | ODS_MR_BILL_COSTS_DO | 1        | NULL                | 1            | NORMAL |
    ↪ OLAP | NULL                 | 1            |
| 10119   | test                 | 1        | NULL                | 1            | NORMAL |
    ↪ OLAP | NULL                 | 1            |
| 10124   | test_parquet_import  | 1        | NULL                | 1            | NORMAL |
    ↪ OLAP | NULL                 | 1            |
+--
    ↪ -------+----------------------+----------+---------------------+--------------+---------+------+-------
    ↪
4 rows in set (0.00 sec)
```

2. 展示集群中所有库表个数相关的信息。

```
mysql> show proc '/statistic';
+-------+----------------------+----------+--------------+----------+-----------+------------+
| DbId  | DbName               | TableNum | PartitionNum | IndexNum | TabletNum | ReplicaNum |
+-------+----------------------+----------+--------------+----------+-----------+------------+
| 10002 | default_cluster:test | 4        | 12           | 12       | 21        | 21         |
| Total | 1                    | 4        | 12           | 12       | 21        | 21         |
+-------+----------------------+----------+--------------+----------+-----------+------------+
2 rows in set (0.00 sec)
```

3. 以下命令可以查看集群内已存在的 Group 信息。

```
SHOW PROC '/colocation_group';

+--
    ↪ -----------+-------------+--------------+------------+----------------+----------+-----------+
    ↪
| GroupId     | GroupName    | TableIds     | BucketsNum | ReplicationNum | DistCols |
    ↪ IsStable |
+--
    ↪ -----------+-------------+--------------+------------+----------------+----------+-----------+
    ↪
| 10005.10008 | 10005_group1 | 10007, 10040 | 10         | 3              | int(11)  | true
    ↪     |
+--
    ↪ -----------+-------------+--------------+------------+----------------+----------+-----------+
    ↪
```

- GroupId：一个 Group 的全集群唯一标识，前半部分为 db id，后半部分为 group id。
- GroupName：Group 的全名。
- TabletIds：该 Group 包含的 Table 的 id 列表。
- BucketsNum：分桶数。
- ReplicationNum：副本数。
- DistCols：Distribution columns，即分桶列类型。
- IsStable：该 Group 是否稳定（稳定的定义，见 `Colocation 副本均衡和修复` 一节）。

4. 通过以下命令可以进一步查看一个 Group 的数据分布情况：

```
SHOW PROC '/colocation_group/10005.10008';

+-------------+---------------------+
| BucketIndex | BackendIds          |
+-------------+---------------------+
| 0           | 10004, 10002, 10001 |
| 1           | 10003, 10002, 10004 |
| 2           | 10002, 10004, 10001 |
| 3           | 10003, 10002, 10004 |
| 4           | 10002, 10004, 10003 |
| 5           | 10003, 10002, 10001 |
| 6           | 10003, 10004, 10001 |
| 7           | 10003, 10004, 10002 |
+-------------+---------------------+
```

- BucketIndex：分桶序列的下标。
- BackendIds：分桶中数据分片所在的 BE 节点 id 列表。

5. 显示现在各种作业的任务总量，及失败的数量。

```
mysql> show proc '/tasks';
+-------------------------+-----------+----------+
| TaskType                | FailedNum | TotalNum |
+-------------------------+-----------+----------+
| CREATE                  | 0         | 0        |
| DROP                    | 0         | 0        |
| PUSH                    | 0         | 0        |
| CLONE                   | 0         | 0        |
| STORAGE_MEDIUM_MIGRATE  | 0         | 0        |
| ROLLUP                  | 0         | 0        |
| SCHEMA_CHANGE           | 0         | 0        |
| CANCEL_DELETE           | 0         | 0        |
| MAKE_SNAPSHOT           | 0         | 0        |
| RELEASE_SNAPSHOT        | 0         | 0        |
```

```
| CHECK_CONSISTENCY       | 0         | 0          |
| UPLOAD                  | 0         | 0          |
| DOWNLOAD                | 0         | 0          |
| CLEAR_REMOTE_FILE       | 0         | 0          |
| MOVE                    | 0         | 0          |
| REALTIME_PUSH           | 0         | 0          |
| PUBLISH_VERSION         | 0         | 0          |
| CLEAR_ALTER_TASK        | 0         | 0          |
| CLEAR_TRANSACTION_TASK  | 0         | 0          |
| RECOVER_TABLET          | 0         | 0          |
| STREAM_LOAD             | 0         | 0          |
| UPDATE_TABLET_META_INFO | 0         | 0          |
| ALTER                   | 0         | 0          |
| INSTALL_PLUGIN          | 0         | 0          |
| UNINSTALL_PLUGIN        | 0         | 0          |
| Total                   | 0         | 0          |
+-------------------------+-----------+----------+
26 rows in set (0.01 sec)
```

6. 显示整个集群的副本状态。

```
mysql> show proc '/cluster_health/tablet_health';
+--
    ↪ --------+---------------------------+-----------+------------+-------------------+-----------------
    ↪
| DbId     | DbName                     | TabletNum | HealthyNum | ReplicaMissingNum |
    ↪ VersionIncompleteNum | ReplicaRelocatingNum | RedundantNum |
    ↪ ReplicaMissingInClusterNum | ReplicaMissingForTagNum | ForceRedundantNum |
    ↪ ColocateMismatchNum | ColocateRedundantNum | NeedFurtherRepairNum | UnrecoverableNum |
    ↪  ReplicaCompactionTooSlowNum | InconsistentNum | OversizeNum | CloningNum |
+--
    ↪ --------+---------------------------+-----------+------------+-------------------+-----------------
    ↪
| 25852112 | default_cluster:bowen     | 1920      | 1920       | 0                 | 0
    ↪                    | 0                     | 0            | 0
    ↪ | 0                       | 0                | 0                   | 0
    ↪                    | 0                     | 0                | 0
    ↪                           | 0               | 0            | 0          |
| 25342914 | default_cluster:bw        | 128       | 128        | 0                 | 0
    ↪                    | 0                     | 0            | 0
    ↪ | 0                       | 0                | 0                   | 0
    ↪                    | 0                     | 0                | 0
    ↪                           | 0               | 0            | 0          |
| 2575532  | default_cluster:cps       | 1440      | 1440       | 0                 | 0
    ↪                    | 0                     | 0            | 0
```

```
    ↪ | 0                        | 0                 | 0                   | 0
    ↪                       | 0                     | 0                 | 0
    ↪                            | 0               | 16          | 0          |
| 26150325 | default_cluster:db        | 38374     | 38374      | 0                  | 0
    ↪                       | 0                     | 0           | 0
    ↪ | 0                        | 0                 | 0                   | 0
    ↪                       | 0                     | 0                 | 0
    ↪                            | 0               | 453         | 0          |
+--
    ↪ --------+---------------------------+-----------+------------+-------------------+-------------------
    ↪
4 rows in set (0.01 sec)
```

查看某个数据库下面的副本状态，如 DbId 为 25852112 的数据库。

```
mysql> show proc '/cluster_health/tablet_health/25852112';
```

7. 报告和诊断集群管控问题

```
MySQL > show proc "/diagnose";
+-----------------+----------+------------+
| Item            | ErrorNum | WarningNum |
+-----------------+----------+------------+
| cluster_balance | 2        | 0          |
| Total           | 2        | 0          |
+-----------------+----------+------------+

2 rows in set
```

查看副本均衡迁移问题

```
MySQL > show proc "/diagnose/cluster_balance";
+--
    ↪ --------------------+--------+-----------------------------------------------------------------------
    ↪
| Item                  | Status | Content
    ↪
    ↪ | Detail Cmd                                                                | Suggestion |
+--
    ↪ --------------------+--------+-----------------------------------------------------------------------
    ↪
| Tablet Health         | ERROR  | healthy tablet num 691 < total tablet num 1014
    ↪                                                                  | show   proc "/cluster_
    ↪ health/tablet_health";                          | <null>     |
```

```
| BeLoad Balance        | ERROR  | backend load not balance for tag {"location" : "default"},
    ↪  low load backends [], high load backends  [10009] | show proc "/cluster_balance/
    ↪ cluster_load_stat/location_default/HDD" | <null>     |
| Disk Balance          | OK     | <null>
    ↪
    ↪ | <null>                                                          | <null>     |
| Colocate Group Stable | OK     | <null>
    ↪
    ↪ | <null>                                                          | <null>     |
| History Tablet Sched  | OK     | <null>
    ↪
    ↪ | <null>                                                          | <null>     |
+--
    ↪ --------------------+--------+-----------------------------------------------------------------
    ↪

5 rows in set
```

7.3.19.1.4 关键词

```
SHOW, PROC
```

最佳实践

8 版本发布

8.1 最新发布

本文列出了近一年内所有已发布的 Apache Doris 版本，按发布时间倒序呈现。

> 最新发布 3.1.2 版本已于 2025 年 10 月 27 日正式发布，详情可查看版本发布。Apache Doris 3.1 是半结构化分析领域的里程碑，引入了稀疏列和模板化 schema，显著提升了查询和索引性能。它还显著增强了湖仓一体能力，通过异步物化视图和优化连接属性，实现了数据湖和数据仓的无缝集成。新版本在存储引擎上提供了灵活列更新，并通过分区裁剪和基于数据特征的优化器，实现了查询性能的巨大飞跃。
>
> 3.0.8 版本已于 2025 年 09 月 19 日正式发布，详情可查看版本发布。从 3.X 版本开始，Apache Doris 除了支持计算存储一体模式外，还支持计算存储分离模式进行集群部署。借助将计算和存储层解耦的云原生架构，用户可以在多个计算集群之间实现查询负载的物理隔离，以及读写负载的隔离。
>
> 2.1.11 版本现已于 2025 年 08 月 15 日正式发布，详情可查看版本发布。子查询性能方面 2.1 版本开箱即用查询的性能提高了 100%；在数据湖分析场景方面，相对于 Trino 和 Spark 分别有

4-6 倍性能提升；在半结构化数据分析场景中提供了强有力的支持，包括新的 Variant 类型和一系列分析函数。此外，2.1 版本起支持异步物化视图以加速查询，优化了大规模实时写入，并通过稳定性和运行时 SQL 资源跟踪改进了工作负载管理。

- 2025-10-27, Apache Doris 3.1.2 版本发布
- 2025-09-26, Apache Doris 3.1.1 版本发布
- 2025-09-19, Apache Doris 3.0.8 版本发布
- 2025-09-04, Apache Doris 3.1.0 版本发布
- 2025-08-25, Apache Doris 3.0.7 版本发布
- 2025-08-15, Apache Doris 2.1.11 版本发布
- 2025-06-16, Apache Doris 3.0.6 版本发布
- 2025-05-17, Apache Doris 2.1.10 版本发布
- 2025-04-28, Apache Doris 3.0.5 版本发布
- 2025-04-02, Apache Doris 2.1.9 版本发布
- 2025-02-28, Apache Doris 3.0.4 版本发布
- 2025-01-24, Apache Doris 2.1.8 版本发布
- 2024-12-02, Apache Doris 3.0.3 版本发布
- 2024-11-10, Apache Doris 2.1.7 版本发布
- 2024-10-15, Apache Doris 3.0.2 版本发布
- 2024-09-30, Apache Doris 2.0.15 版本发布
- 2024-09-10, Apache Doris 2.1.6 版本发布
- 2024-08-23, Apache Doris 3.0.1 版本发布
- 2024-07-24, Apache Doris 2.1.5 版本发布
- 2024-07-17, Apache Doris 2.0.13 版本发布
- 2024-06-27, Apache Doris 2.0.12 版本发布
- 2024-06-26, Apache Doris 2.1.4 版本发布
- 2024-06-05, Apache Doris 2.0.11 版本发布
- 2024-05-21, Apache Doris 2.1.3 版本发布
- 2024-05-16, Apache Doris 2.0.10 版本发布

- 2024-04-23, Apache Doris 2.0.9 版本发布
- 2024-04-12, Apache Doris 2.1.2 版本发布
- 2024-04-09, Apache Doris 2.0.8 版本发布
- 2024-04-03, Apache Doris 2.1.1 版本发布
- 2024-03-26, Apache Doris 2.0.7 版本发布
- 2024-03-12, Apache Doris 2.1.0 版本发布
- 2024-03-11, Apache Doris 2.0.6 版本发布
- 2024-02-28, Apache Doris 2.0.5 版本发布
- 2024-01-26, Apache Doris 2.0.4 版本发布

8.2 v3.1

8.2.1 Release 3.1.3

8.2.1.1 新功能

8.2.1.1.1 存储与文件系统

- 升级 libhdfs 至 3.4.2（#57638）
- 为 S3 Reader 增加 `TotalGetRequestTime` 性能指标（#57636）

8.2.1.1.2 Catalog

- 支持 MaxCompute Catalog（project-schema-table 模式）（#57286）
- 支持 Azure Blob Storage（#57219）
- 支持 在转换方言后的 SQL 执行错误后，重试原始 SQL（#57498）

8.2.1.1.3 云模式

- 支持 balance sync 热身机制（#57666）
- 支持 同一集群内 BE 间的 peer cache 读取（#57672）

8.2.1.1.4 SQL 引擎与优化器

- 在生成执行计划前检查 SQL 正则屏蔽规则（#57706）
- EXPLODE 函数支持 struct 类型展开（#57827）

8.2.1.2 优化

8.2.1.2.1 查询执行与优化器

- 优化 variant 类型仅含 NULL 值时的 cast 性能（#57161）
- 优化 FROM_UNIXTIME 函数性能（#57573）
- 改进 优雅下线行为与查询重试逻辑（#57805）

8.2.1.2.2 存储与 Compaction

- MergeIO 读取切片大小支持配置化（#57159）
- 为冷数据 Compaction 增加评分阈值（#57217）
- 为小内存任务保护机制增加 可配置阈值（#56994）
- 优化 jemalloc 配置，减少缺页中断（#57152）

8.2.1.2.3 云模式

- 暴露 云端 rebalance 指标（#57352）
- 优化 warm-up 任务创建逻辑（#57865）
- 提升 warm-up 与 peer read 效率（#57554，#57807）

8.2.1.2.4 索引与搜索

- 支持 自定义分词器（char_filter、 basic 和 ICU tokenizer）（#57137）
- 自定义分词器支持 内置 analyzer 名称（#57727）

8.2.1.3 Bug 修复

8.2.1.3.1 存储与文件 I/O

- 修复 添加 key 列时 segcompaction 崩溃问题（#57212）
- 修复 Parquet RLE_DICTIONARY 解码性能问题（#57614）
- 修复 schema change 表达式缓存误用问题（#57517）
- 使用 ForkJoinPool 重构 tablet report 实现（#57927）

8.2.1.3.2 云模式

- 修复 pipeline 任务数量计算错误（#57261）
- 修复 rebalance 残留指标未清理问题（#57438）
- 在 rebalance 未初始化时 跳过 tablet report（#57393）
- 修复 domain 用户默认集群上报错误（#57555）
- 修复 私有 endpoint 配置错误（#57675）
- 修复 peer read 错误与线程处理问题（#57910，#57807）
- 修复 filecache 指标与 microbench 问题（#57535，#57536）

8.2.1.3.3 Catalog

- 修复 MaxCompute 谓词下推空指针错误（#57567）
- 修复 Iceberg client.region 与 REST 认证问题（#57539）
- 修复 Iceberg Catalog NPE 与查询异常（#57796，#57790）
- 修复 Paimon S3 前缀与配置不一致问题（#57526）
- 修复 JDBC Catalog `zeroDateTimeBehavior` 参数兼容性问题（#57731）
- 修复 Parquet Schema 分析错误（#57500）
- 修复 Parquet 所有 row group 被过滤问题（#57646）
- 修复 CSV 读取 escape=enclose 时的结果错误（#57762）
- 防止 Catalog 被误删出刷新队列（#57671）
- 修复 max_meta_object_cache_num 必须大于 0 的配置问题（#57793）

8.2.1.3.4 SQL 引擎与优化器

- 修复 FROM_UNIXTIME + decimal 常量折叠错误（#57606）
- 修复 MV 重写在 group sets + filter 下失败问题（#57674）
- 修复 prepare statement 仅 explain SQL 的问题（#57504）
- 在 Profile.releaseMemory() 中释放物理计划内存（#57316）
- 修复 group sets 下聚合消除错误（#57885）
- 修复 LargeInt 溢出（max_value+1）问题（#57351）
- 修复 decimal256 转 float 溢出问题（#57503）

8.2.1.3.5 网络与平台

- 修复 MySQL SSL unwrap 无限循环问题（#57599）
- 禁用 MySQL TLS renegotiation（#57748）
- 修复 uint128 构造未对齐问题（#57430）
- 修复 JNI 本地/全局引用泄漏（#57597）
- Scanner.close() 增加线程安全保护（#57644）
- 修复 Exchange 节点空指针导致的崩溃问题（#57698）

8.2.1.4 杂项

- 废弃 LakeSoul 外部 Catalog 支持（#57163）

8.2.1.5 总结

Apache Doris 3.1.3 带来了以下主要改进：

- 存储兼容性增强（支持 Azure Blob、 Hadoop 3.4.2、 S3 性能指标）
- 云端性能与可靠性提升（warm-up、 rebalance、 peer cache）
- SQL 优化器稳定性增强
- 依赖升级与安全性改进

本次版本显著提升了 Doris 在 稳定性、性能与云原生融合能力方面的整体表现

8.2.2 Release 3.1.2

8.2.2.1 新功能

存储与压缩

- 可配置的表压缩类型 —— 支持为每张表单独指定压缩算法。#56276
- 自适应压缩写缓存 —— 在基础压缩（base compaction）行集（rowset）刷写过程中动态调整写缓存策略。#56278

云与对象存储

- 云模式查询新鲜度控制 —— 新增用户自定义的数据延迟与一致性之间的容忍度配置。#56390
- 放宽对象存储端点验证 —— 支持私有或自定义存储端点。#56641

数据湖（Datalake）

- 支持通过数据湖 VPC 端点（`dlf/datalake-vpc`）访问 OSS**。#56476
- AWS Glue Catalog 现支持通过 IAM AssumeRole 访问 S3。#57036
- S3 客户端已更新为使用 `CustomAwsCredentialsProviderChain`，以改进凭证管理。#56943

功能增强

- Java UDF 现支持 IP 类型。#56346
- BE REST API 新增 `RunningTasks` 输出项，用于任务监控。#56781
- 事务监控新增 BRPC 写放大（write-amplification）指标。#56832

优化

查询执行与优化器

- `COUNT(*)` 优化 ** —— 自动选择最小的列以减少扫描负载。#56483
- 压缩过程跳过空行集，以提升吞吐量。#56768
- 预热（Warmup）统计信息新增“跳过的行集”指标，提升可观测性。#56373

存储层

- 为稀疏列新增 Variant 列缓存，以加速读取。#56730
- 段（Segment）页脚现已缓存在索引页缓存（Index Page Cache）中，以降低延迟。#56459
- 回收器（Recycler）支持并行清理任务，提高吞吐量。#56573

数据湖

- 改进 Paimon 时间旅行（Time Travel）功能，并修复了模式（schema）不匹配问题。#56338
- 优化 Iceberg 扫描错误信息，并支持嵌套命名空间。#56370, #57035
- 移除旧版 DLF Catalog 属性。#56196, #56505

- JSON 导入默认采用逐行解析模式（row-by-row parsing mode）处理基于行的数据。#56736

缺陷修复

数据湖

- 修复 Iceberg 系统表类加载器（classloader）错误。#56220
- 修复 Iceberg 分区表在无分区值时失败的问题。#57043
- 修复 S3A Catalog 未正确使用 IAM AssumeRole 配置文件的问题。#56250
- 为多配置对象存储 Catalog 禁用 Hadoop FileSystem 缓存。#57153

查询执行与 SQL 引擎

- 修复 COUNT 下推逻辑错误。#56482
- 修复 UNION 本地 shuffle 行为异常。#56556
- 修复 OLAP 存储类型中 IN 谓词导致的崩溃问题。#56834
- 修复 datetimev1 类型下 timestampdiff 计算错误。#56893
- 修复 explode() 函数导致的崩溃问题。#57002

存储与导入

- 修复源文件不存在时 S3 导入检查失败的问题。#56376
- 修复 FileCache 清理时崩溃的问题。#56584
- 修复 MOW 压缩模式下删除位图（delete bitmap）未被清除的问题。#56785
- 修复小文件使用 Outfile 的 bz2 压缩时失败的问题。#57041

云与回收机制

- 修复预热（Warmup）跳过多段（multi-segment）行集的问题。#56680
- 修复 CloudTablet 预热过程中因引用捕获导致的 core dump 问题。#56627
- 修复回收器（Recycler）清理任务中的空指针崩溃问题。#56773
- 修复云模式下未捕获的分区边界错误。#56968

系统及其他

- 修复 FE 中 Prometheus 指标格式错误的问题。#57082
- 修复 FE 重启后自增列值不正确的问题。#57118
- 修复 SHOW CREATE VIEW 语句缺失列定义的问题。#57045
- 修复 HDFS Reader 在采样 Profile 数据时崩溃的问题。#56950

8.2.3 Release 3.1.1

8.2.3.1 概述

Apache Doris 3.1.1 是一个维护版本，重点在于关键性错误修复、性能优化和稳定性提升。此版本包含大量针对数据合并（compaction）、数据导入、查询处理和云功能的修复，使其在生产环境中更加稳健可靠。

8.2.3.2 新特性

8.2.3.2.1 核心功能

- [feature](#) 支持 count_substrings 函数（#42055, #55847）

8.2.3.2.2 数据集成与存储

- [feat](#) 新增 HDFS 高可用（HA）配置验证（#55675, #55764）
- [feat](#) 为校验器增加 Tablet 统计键一致性检查（#54754, #55663）
- [feat](#) 在 outfile 和导出中支持 CSV 格式的压缩类型（#55392, #55561）
- [feat](#) 支持云环境下 Group Commit Stream Load 的 BE 转发模式（#55326, #55527）

8.2.3.2.3 性能与优化

- [support](#) 支持 ORC 文件元数据缓存（#54591, #55584）
- [Exec](#) 支持使用 SIMD 计算 KNN 距离（#55275）

8.2.3.3 改进

8.2.3.3.1 性能优化

- [opt](#) 减少元数据服务中空 Rowset 带来的压力（#54395, #55171, #55604, #55742, #55837, #55934）
- [Opt](#) 优化 MOW 导入性能和 CPU 使用率（#55073, #55733, #55771, #55767）
- [opt](#) 将 hive.recursive_directories 默认值设为 true（#55737, #55905）
- [opt](#) 避免频繁发起 Aws::Internal::GetEC2MetadataClient HTTP 请求（#55546, #55682）
- [opt](#) 不再捕获调用栈以降低 CPU 开销（#55368, #55526）
- [opt](#) 使临时 Rowset 批量转换逻辑更具自适应性（#55035, #55573）
- [opt](#) 优化将大字符串转换为复杂类型时的性能（#55476, #55521）
- [opt](#) 支持简化字符串范围（#55378, #55456）
- [opt](#) 优化窗口函数的规范化逻辑（#54947, #55046）
- [opt](#) 当 OLAP 表具有自动分区时，优化 INSERT 命令的并行度（#54983, #55030）

8.2.3.3.2 系统增强

- [enhancement](#) 将部分日志级别从 info 调整为 debug（#55808, #55841）
- [enhancement](#) 支持多个缓存实例之间并行清理缓存（#55259, #55437）
- [enhancement](#) 禁止对隐藏列执行 Schema Change（#53376, #55385）
- [enhancement](#) 在备份过程中正确处理已被删除的表和分区（#52935, #54989）
- [enhancement](#) 修复从 Doris 2.1 版本恢复到 3.1 版本时的云恢复问题（#55110）
- [enhancement](#) 支持 time 与 datetime 类型之间的相互转换（#53734, #54985）

8.2.3.3.3 基础设施改进

- [refactor](#) 使用统一架构重构临时凭证系统（#55912）
- [refactor](#) 将云恢复中创建 Tablet 的 RPC 拆分为多个批次（#55691）
- [opt](#) 在 FE 异常时增加跳过某些 editlog 异常的能力（#54090, #55204）

8.2.3.4 关键性错误修复

8.2.3.4.1 合并与存储

- [fix](#) 对于版本号 ≤ alter_version 的空 Rowset，跳过版本空洞填充（#56209, #56212）
- [fix](#) 修复合并后输入 Rowset 被过早驱逐，导致查询失败的问题（#55382, #55966）
- [fix](#) 使创建 Tablet 操作具有幂等性，从而保证合并任务的幂等性（#56061, #56108）
- [fix](#) 在段合并（segcompaction）中使用 Rowset 元数据文件系统，并增加 RPC 客户端就绪检查（#55951, #55988）
- [fix](#) 在合并过程中跳过合并分数为 0 的 Tablet（#55550, #55570）

8.2.3.4.2 查询处理与函数

- [fix](#) abs 函数的返回类型应与参数类型一致（#56190, #56210）
- [fix](#) 当 float/double 值为 NaN 时，不在 BE 端进行常量折叠（#55425, #55874）
- [Fix](#) 修复 unix_timestamp 函数返回的小数位数错误（#55013, #55962）
- [fix](#) 修复因精度丢失或空值转换导致的比较谓词简化错误（#55884, #56110）
- [fix](#) 修复在 Join 重排时抛出“eq 函数不存在”异常的执行错误（#54953, #55667）
- [fix](#) 修复窗口表达式别名复用时的表达式 ID 错误（#55286, #55486）
- [fix](#) 在与 count() 聚合函数比较时，使用 bigint 字面量而非 int（#55545, #55590）
- [fix](#) 在生成巨大表达式时停止合并投影（#55293, #55519）

8.2.3.4.3 数据加载与导入

- [fix](#) 修复 S3 导入连接检查失败的问题（#56123）
- [fix](#) 修复已完成导入任务进度显示不正确的问题（#55509, #55530）
- [fix](#) 修复特定导入错误场景导致 BE 崩溃（core dump）的问题（#55500）
- [fix](#) 修复 Routine Load 任务因 MEM_LIMIT_EXCEED 失败后无法再次调度的问题（#55481, #55616）

8.2.3.4.4 云与分布式功能

- [fix](#) 在 replayUpdateCloudReplica 中移除无用的表锁（#55579, #55955）
- [fix](#) calc_sync_versions 应考虑全量合并（full compaction）（#55630, #55710）
- [fix](#) 修复 CloudTablet::complete_rowset_segment_warmup 导致的崩溃问题（#55932）

8.2.3.4.5 数据库操作

- [fix](#) 修复重命名数据库与创建表之间的竞态条件（#55054, #55991）
- [fix](#) 并发重命名数据库会导致建表及重放失败（#54614, #56039）
- [fix](#) 将删除 editlog 操作移至表锁内执行（#55705, #55947）
- [fix](#) 启用轻量级 Schema Change 后，Tablet 列未被重建（#55909, #55939）

8.2.3.4.6 数据类型与序列化

- [fix](#) 修复将空值序列化为 JSON 字符串时的处理逻辑（#55876, #56138）
- [fix](#) 修复稀疏列为空时的兼容性错误（#55817）
- [fix](#) 增强 Variant 类型的 `max_sparse_column_statistics_size` 配置（#55124, #55752）

8.2.3.4.7 外部数据源

- [fix](#) 修复 Paimon 原生读取器未使用延迟物化（late materialization）的问题（#55894, #55917）
- [fix](#) 通过在缓存键中加入 `dlf.catalog.id` 修复 Paimon DLF Catalog 缓存问题（#55875, #55888）
- [fix](#) 修复 Paimon 到 Doris 类型映射中 CHAR/VARCHAR 字段过大的处理问题（#55051, #55531）
- [fix](#) 修复在下推 MaxCompute 谓词时因表列不存在而抛出 NereidsException 的问题（#55635, #55746）
- [fix](#) 修复国际用户无法访问 MaxCompute Catalog 的问题（#55256, #55560）
- [fix](#) 修复查询仅需分区列（无数据字段）的 Hudi JNI 表时的问题（#55466, #55662）
- [fix](#) 修复查询 `NULL DEFINED AS ''` 的 Hive Text 表时的问题（#55626, #55661）
- [fix](#) 为元数据扫描器补充缺失的 `iceberg-aws` 依赖（#55741, #55743）
- [fix](#) 使用 Iceberg 默认值刷新 OAuth2 Token（#55578, #55624）

8.2.3.4.8 内存与资源管理

- [fix](#) 内存未被 MemTracker 正确追踪的问题（#55796, #55823）
- [fix](#) 修复 `BaseTablet::get_rowset_by_ids()` 中 MOW 导致的崩溃（#55539, #55601）
- [fix](#) 修复 MOW 聚合缓存版本检查问题（#55330, #55475）
- [fix](#) 修复因错误跳过段而引起的段数量不匹配问题（#55092, #55471）
- [fix](#) 云模式下段缓存不再限制文件描述符数量（#55610, #55638）

8.2.3.4.9 安全与加密

- [fix](#) 修正加密密钥版本的显示（#56092, #56068）
- [fix](#) 修复与透明数据加密（TDE）相关的问题（#55692）

8.2.3.4.10 其他修复

- [fix](#) 修复当分区表没有分区时 MTMV 无法刷新的问题（#55468, #56085）
- [fix](#) 修复插件目录的兼容性问题（#56060）
- [fix](http stream) HTTP 流式接口在 SQL 解析失败时应抛出异常（#55863, #55891）

- [fix](#) 支持备份元数据/作业信息超过 2GB（#55608, #55867）
- [fix](#) 转发到 Master 时正确设置更多语句存在标志（#55711, #55871）
- [fix](#) 修复因超时断开连接时未清理会话相关数据的问题（#55008, #55809, #55396）
- [fix](#) 执行失败时重放 WAL 中止事务失败的问题（#55881, #55924）
- [fix](#) 清理已恢复的表/分区/资源以降低开销（#55757, #55784）
- [fix](#) 移除未使用的更新索引（#55514, #55704）
- [fix](#) 修复事务延迟提交与 Schema Change 冲突的问题（#55349, #55701）
- [fix](#) 修复 SSL 模式下的查询错误（#53134, #55628）
- [fix](#) 使用位与操作替代 `Math.abs` 以确保生成非负 ID（#55183, #55689）
- [fix](#) 修复 `array_agg_foreach` 函数结果错误的问题（#55075, #55420）

8.2.3.5 基础设施与开发

8.2.3.5.1 构建与依赖

- [chore](#) 优化构建脚本（#56027, #56028）
- [chore](#) 将 aws-sdk-cpp 从 1.11.119 升级至 1.11.219（#54780, #54971）
- [chore](#) 更新带 OpenSSL 的 libevent 依赖（#54652, #54857）
- [chore](#) 添加 `brpc::usercode_in_pthread` 配置以支持 ASAN（#54656, #54829）

8.2.3.5.2 测试与质量

- [chore](#) 修复若干失败的测试用例（#56140, #56167）
- [fix](#) 修复若干失败的测试用例（#56019, #56035）
- [fix](#) 修改回归测试以提高稳定性并调整预期日志级别（#55169, #55898）
- [fix](#) 修复若干失败的测试用例（#55739, #55769）
- [fix](#) 修复回归测试用例：cse.groovy（#53434, #55897）
- [fix](#) 修复 `test_hudi_snapshot` 测试失败问题（#55761, #55791）
- [fix](#) 修复若干失败的测试用例（#55811, #55835）
- [fix](#) 等待 MV 任务时应只关注最新任务（#55802, #55830）
- [fix](#) 修复 Variant 构建索引的测试用例（#55613, #55648）
- [Fix](#) 修复 show data p2 测试用例（#55449, #55494）
- [fix](#) 修复异步物化视图的 `show create table` 显示失败问题（#55278, #55480）
- [fix](#) 在云模式下跳过部分测试（#55448, #55535）
- [Fix](#) 修复若干测试用例（#55606, #55656）
- [test](#) 为包含表达式的导出用例增加并行度测试（#55636, #55659）
- [test](#) 新增 Polaris 测试（#55484, #55557）
- [test](#) 为 SQL 缓存/有序分区缓存增加单元测试（#55520, #55536）
- [test](#) 适配 HMS 和 GCS 上的 Paimon（#55473, #55512）
- [test](#) 修复不稳定的周期性预热测试用例（#55365, #55453）

8.2.3.5.3 安全与配置

- [chore](#) 对日志中的 `secret key` 进行加密，并隐藏 `access key`（#55241, #55619）

- [chore](#) user_files_secure_path 配置项运行时不可更改（#55395, #55504）
- [chore](#) ignore_load_tablet_failure 默认值设为 true（#55109, #55441）

8.2.3.5.4 云基础设施

- [chore](#) 更新构建和启动脚本（#56031, #56064）
- [chore](#) 支持在事务提交时上报冲突范围（#55340, #55714）
- [chore](#) 改进回收器（recycler）指标（#55455, #55479）
- [chore](#) 打印导出任务拆分 Tablet ID 的日志（#55170, #55646）

8.2.3.5.5 第三方与补丁

- [thirdparty](#) BRPC 强制所有连接使用 SSL（#55658, #55696）
- [thirdparty](#) 修复启用 SSL 时 BRPC 崩溃的问题（#55649, #55695）
- [fix](#) 将 Kafka Docker 镜像更新为内部源（#55460, #55487）

8.2.3.5.6 CI 与性能

- [ci](#) 更新目标分支的 Docker 镜像引用（#55511）

8.2.3.6 行为变更

8.2.3.6.1 配置变更

- [opt](#) hive.recursive_directories 默认值变更为 true
- [chore](#) ignore_load_tablet_failure 默认值变更为 true
- [chore](#) user_files_secure_path 配置项运行时不可再修改

8.2.3.6.2 安全增强

- [chore](#) 为提升安全性，日志中 secret key 现已加密，access key 已隐藏

8.2.3.7 兼容性说明

- 本版本与 Apache Doris 3.1.0 保持向后兼容
- 云恢复功能现已支持从 Doris 2.1 版本迁移至 3.1 版本
- 增强了 time 与 datetime 类型之间的类型转换支持

8.2.4 Release 3.1.0

亲爱的社区小伙伴们，我们很高兴地向大家宣布，近期我们迎来了 Apache Doris 3.1 版本的正式发布，欢迎大家下载使用体验。

3.1 版本是 Apache Doris 在半结构化分析上的一个里程碑版本。在 VARIANT 类型上，3.1 版本新增了稀疏列能力，使得 VARIANT 可以轻松应对数万子列的场景。同时，在 VARIANT 类型上引入了模板化 schema 能力，让 VARIANT 类型在关键路径上，查询更快、索引更稳、成本可控，同时不丢失灵活性。在倒排索引能力上，3.1 版本引入了 index v3 版本的索引格式，相比较于 v2 版本存储空间节省可达 20%。同时，支持了更为丰富的分词手段，提供了三种全新的分词器：ICU Tokenizer、IK Tokenizer 和 Basic Tokenizer。还进一步支持了自定义分词器，可以突破内置分词器的局限性，根据业务场景定制，显著提升搜索召回率。

3.1 版本同样在湖仓一体上有了显著的增强。在 3.1 版本中，Apache Doris 将异步物化视图中的分区构建和透明改写分区补偿，这两项重要能力引入数据湖中，在湖和仓中间架起一座重要的桥梁。3.1 版本还扩充了对 iceberg 和 paimon 特性的支持范围。另外，通过引入动态分区裁剪和批量分片执行在特定场景下提升了数据湖查询的性能多达 40%，并显著降低了 FE 的内存占用。同时 3.1 版本还重构了各个数据源的连接属性，不仅能够以更加清晰的方式对接各类元数据服务和数据存储系统，同时还支持了更加丰富的连接能力。

3.1 版本 Apache Doris 持续打磨存储引擎。提供了全新的数据更新方式 —— 灵活列更新。在部分列更新的基础上，进一步放开限制。在一次导入中对于每一行可以更新不同的列。另外，在存算分离场景下，优化了 MOW 表部分链路的锁获取逻辑和使用范围，提升高并发导入场景的使用体验。

在性能方面，3.1 着重优化了分区裁剪的能力和规划性能。在数万分区和复杂分区过滤表达式的场景下，能够显著提升查询性能并降低资源消耗。同时，3.1 还在优化器中全面引入了基于数据特征的优化手段，在特定场景下可以获得超过 10 倍的性能提升。

在 3.1 版本的研发过程中，有超过 90 名贡献者为 Apache Doris 提交了 1000+ 个优化与修复。 在此向所有参与版本研发、测试和需求反馈的贡献者们表示最衷心的感谢。

- GitHub 下载
- 官网下载

8.2.4.1 一、VARIANT 半结构化查询华丽变身

8.2.4.1.1 存储能力质变：稀疏列与子列 Vertical Compaction，轻松支持数万子列

传统 OLAP 面对 “超宽表/超多列”（上千到上万）常遇到元数据膨胀、合并放大与查询退化；Doris 3.1 通过 VARIANT 的稀疏子列与子列级 Vertical Compaction，将可维护的列数上限抬升到数万级。

通过对存储层的深入优化，Variant 给用户带来以下收益：

- 稳定支撑 “上千 - 数万” 子列（列式存储），查询/合并延迟更平滑。
- 元数据与索引可控，避免指数级膨胀。
- 实测可进行 10,000+ 子列提取（列式存储），Compaction 效率顺畅。

超多列的适用场景：

- 车联网/IoT 遥测：设备型号多、传感器维度动态增减。

- 营销自动化/CRM：事件/用户属性持续扩展（如自定义 event/property）。
- 广告/埋点事件：海量可选 properties，字段稀疏且不断演进。
- 安全审计/日志：不同源日志字段各异，需按模式聚合检索。
- 电商商品属性：类目跨度大，商品属性高度可变。

实现原理

- 稀疏子列（Sparse Subcolumns）：按 JSON Key 频次排序，只提取 Top-N 高频子列入“真列式”；长尾保持在稀疏列存储，避免无序扩张。
- 子列级 Vertical Compaction：对 VARIANT 子列应用 Vertical Compaction，分组合并、内存占用更小；合并时动态识别并固化热点路径，进一步降低合并开销。
- 优化值填充默认值效率，按 batch 的方式进行批量填充（减少虚函数开销）。
- 通过 LRU 机制减少内存中列存储相关元数据缓存内存开销。

如何开启与使用

新增列级别控制 Variant 参数，列属性（Properties）：

`variant_max_subcolumns_count`：默认是0，表明不开启稀疏列能力，设置成特定值后将会提取 Top-N 高频的 JSON key 作为列式存储，余下的列进入稀疏列存储。

```
-- Enable sparse subcolumns and cap hot subcolumn count
CREATE TABLE IF NOT EXISTS tbl (
  k BIGINT,
  v VARIANT<
      properties("variant_max_subcolumns_count" = "2048") -- pick top-2048 hot keys
  >
) DUPLICATE KEY(k);
```

8.2.4.1.2 模板化 Schema（Schema Template）- 变化中的不变量

一句话总结：模板化 Schema 让“常变”的 JSON 在关键路径上“变得可预期”：查询更快、索引更稳、成本可控，同时不丢失灵活性。

使用模板化的 Schema，将会给使用 Variant 数据类型带来以下收益：

- 类型稳定：关键子路径类型可在 DDL 中固定，避免类型漂移引发的查询报错、索引失效与隐式转换开销。
- 检索更快更准：为不同子路径定制倒排策略（分词/非分词、解析器、短语搜索等），常用查询延迟更低、命中更稳定。
- 索引与成本可控：不再“整列统一继承索引”（2.1 的做法易膨胀），而是“按子路径精细化配置”，显著降低索引数量、写放大与存储成本。
- 可维护/可协作：等同给 JSON 加“数据契约”，跨团队语义一致；类型与索引状态更可观测，问题更易定位。
- 演进友好：核心高频路径模板化并可选建索引，长尾字段继续保持灵活扩展，不牺牲可扩展性。

如何开启与使用

- 显式声明结构，指定类型：在VARIANT<...>中预定义常用子路径与类型（含通配），例如'a' : int, 'c
  ↪ .d' : text, 'array_int_*' : array<int>。
- 配置索引，针对同一 VARIANT 列的不同子路径配置不同索引策略（field_pattern、解析器、分词、短语搜索等），差异化提升检索效率，可用通配符批量匹配。
- 新增列级别控制Variant参数,列属性（Properties）:variant_enable_typed_paths_to_sparse:默认是false
  ↪ ，表明预定义的列不会进入稀疏列，true 开启后预定义类型路径也会进入稀疏存储（用于避免匹配过多列后导致的列数膨胀）

示例 1：schema 定义 + 单列多索引

```
-- Common properties: field_pattern (target subpath), analyzer, parser, support_phrase, etc.
CREATE TABLE IF NOT EXISTS tbl (
    k BIGINT,
    v VARIANT<'content' : STRING>, -- specify concrete type for subcolumn 'content'
    INDEX idx_tokenized(v) USING INVERTED PROPERTIES("parser" = "english", "field_pattern" = "
        ↪ content"), -- tokenized inverted index for 'content' with english parser
    INDEX idx_v(v) USING INVERTED PROPERTIES("field_pattern" = "content") -- non-tokenized
        ↪ inverted index for 'content'
);

-- v.content will have both a tokenized (english) inverted index and a non-tokenized inverted
    ↪ index

-- Use tokenized index
SELECT * FROM tbl WHERE v['content'] MATCH 'Doris';

-- Use non-tokenized index
SELECT * FROM tbl WHERE v['content'] = 'Doris';
```

示例 2：通配符批量处理符合模式的列

```
-- Use wildcard-typed subpaths with per-pattern indexes
CREATE TABLE IF NOT EXISTS tbl2 (
  k BIGINT,
  v VARIANT<
      'pattern1_*' : STRING, -- batch-typing: all subpaths matching pattern1_* are STRING
      'pattern2_*' : BIGINT, -- batch-typing: all subpaths matching pattern2_* are BIGINT
      properties("variant_max_subcolumns_count" = "2048") -- enable sparse subcolumns; keep top
          ↪ -2048 hot keys
  >,
  INDEX idx_p1 (v) USING INVERTED
    PROPERTIES("field_pattern"="pattern1_*", "parser" = "english"), -- tokenized inverted index
        ↪ for pattern1_* with english parser
  INDEX idx_p2 (v) USING INVERTED
    PROPERTIES("field_pattern"="pattern2_*") -- non-tokenized inverted index for pattern2_*
) DUPLICATE KEY(k);
```

示例 3：允许预定义类型的列进入稀疏列

```
-- Allow predefined typed paths to participate in sparse extraction
CREATE TABLE IF NOT EXISTS tbl3 (
  k BIGINT,
  v VARIANT<
    'message*' : STRING, -- batch-typing: all subpaths matching prefix 'message*' are STRING
    properties(
      "variant_max_subcolumns_count" = "2048",              -- enable sparse subcolumns; keep top
          ↪ -2048 hot keys
      "variant_enable_typed_paths_to_sparse" = "true"       -- include typed (predefined) paths
          ↪ as sparse candidates (default: false)
    )
  >
) DUPLICATE KEY(k);
```

8.2.4.2 二、索引架构全面进化

8.2.4.2.1 倒排索引存储格式 V3 - 性能和功能的双重提升

相比 V2 进一步优化存储

索引文件更小，减少磁盘占用和 I/O 开销，以 httplogs 与 logsbench 两个测试集测试结果来看，存储空间最大可以通过 V3 节省 20%，适合大规模文本数据、日志分析场景。

倒排索引存储格式 V3 - 性能和功能的双重提升

核心改进

- 引入倒排索引 ZSTD 词典压缩：采用 ZSTD 压缩算法对倒排索引内的词典文件进行压缩，通过 index properties 中的 `dict_compression` 开启。
- 新增倒排索引位置信息压缩：支持对倒排索引中为每个 term 即词元记录的位置信息进行编码压缩，进一步减少倒排索引空间占用。

使用方式

```
-- 建表时启用V3格式
CREATE TABLE example_table (
    content TEXT,
    INDEX content_idx (content) USING INVERTED
    PROPERTIES("parser" = "english", "dict_compression" = "true")
) ENGINE=OLAP
PROPERTIES ("inverted_index_storage_format" = "V3");
```

8.2.4.2.2 倒排索引 - 分词器灵活多样好用易用

新增三种常用分词器

进一步提升用户在不同场景下的分词需求：

ICU Tokenizer

- 实现：ICU（International Components for Unicode）
- 适用场景：包含复杂文字系统的国际化文本，特别适合多语言混合文档。
- 示例：

```
SELECT TOKENIZE('□□□□□□ □□□□□□□□ Hello 世界', '"parser"="icu"');
-- 结果：["□□□□□□", "□□□□□□□□", "Hello", "世界"]

SELECT TOKENIZE('□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□', '"parser"="icu"');
-- 结果：["□□", "□□□□□□", "□□", "□□□", "□□□□", "□□□□□□□"]
```

IK Tokenizer

- 实现：IK Analyzer（中文分词器），基于算法的高级中文分词，结合词典和统计模型
- 适用场景：对分词质量要求较高的中文文本处理
- 模式：
- ik_smart：智能模式，词少且更长，语义集中，适合精确搜索
- ik_max_word：最细粒度模式，更多短词，覆盖更全面，适合召回搜索
- 示例：

```
-- 智能模式
SELECT TOKENIZE('中华人民共和国国歌', '"parser"="ik","parser_mode"="ik_smart"');
-- 结果：["中华人民共和国", "国歌"]

-- 最细粒度模式
SELECT TOKENIZE('中华人民共和国国歌', '"parser"="ik","parser_mode"="ik_max_word"');
-- 结果：["中华人民共和国", "中华人民", "中华", "华人", "人民共和国", "人民", "共和国", "共和",
    ↪  "国歌"]
```

Basic Tokenizer

- 实现：简单规则的自定义分词器，基础分词，采用字符类型识别进行分词
- 适用场景：简单场景、对性能要求极高的场景
- 分词规则：
- 连续的字母数字字符作为一个词（word tokens）
- 中文字符单独分词（每个汉字一个 token）
- 忽略标点符号、空格和特殊符号
- 示例：

```
-- 英文文本分词
SELECT TOKENIZE('Hello World! This is a test.', '"parser"="basic"');
-- 结果: ["hello", "world", "this", "is", "a", "test"]

-- 中文文本分词
SELECT TOKENIZE('你好世界', '"parser"="basic"');
-- 结果: ["你", "好", "世", "界"]

-- 混合语言分词
SELECT TOKENIZE('Hello 你好 World 世界', '"parser"="basic"');
-- 结果: ["hello", "你", "好", "world", "世", "界"]

-- 包含数字和特殊字符
SELECT TOKENIZE('GET /images/hm_bg.jpg HTTP/1.0', '"parser"="basic"');
-- 结果: ["get", "images", "hm", "bg", "jpg", "http", "1", "0"]

-- 处理长数字序列
SELECT TOKENIZE('12345678901234567890', '"parser"="basic"');
-- 结果: ["12345678901234567890"]
```

自定义分词

推出自定义分词功能，方便用户根据自身分词需求，进行 DIY 组合，进一步提高文本检索召回率。自定义分词可以突破内置分词的局限，根据特定需求组合字符过滤器、分词器和词元过滤器，精细定义文本如何被切分成可搜索的词项，这直接决定了搜索结果的相关性与数据分析的准确性。

自定义分词

使用场景举例

- 问题

使用默认 unicode 分词器时，电话号码“13891972631”被当作完整 token，无法支持前缀搜索如“138”。

- 解决方案
- 创建分词器（tokenizer）
- 使用 Edge N-gram 自定义分词器：

```
CREATE INVERTED INDEX TOKENIZER IF NOT EXISTS edge_ngram_phone_tokenizer
 PROPERTIES
 (
     "type" = "edge_ngram",
     "min_gram" = "3",
     "max_gram" = "10",
     "token_chars" = "digit"
 );
```

- 创建分析器（analyzer）

```
CREATE INVERTED INDEX ANALYZER IF NOT EXISTS phone_prefix_analyzer
  PROPERTIES
  (
      "tokenizer" = "edge_ngram_phone_tokenizer"
  );
```

- 创建表指定 analyzer

```
  CREATE TABLE customer_contacts (
  id bigint NOT NULL AUTO_INCREMENT(1),
  phone text NULL,
  INDEX idx_phone (phone) USING INVERTED PROPERTIES(
      "analyzer" = "phone_prefix_analyzer"
  )
  ) ENGINE=OLAP
  DUPLICATE KEY(id)
  DISTRIBUTED BY RANDOM BUCKETS 1
  PROPERTIES ("replication_allocation" = "tag.location.default: 1");
```

- 查看分词效果

```
SELECT tokenize('13891972631', '"analyzer"="phone_prefix_analyzer"');
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| tokenize('13891972631', '"analyzer"="phone_prefix_analyzer"')
    ↪
    ↪ |
+--
    ↪ ----------------------------------------------------------------------------------------------------
    ↪
| [{
        "token": "138"
    }, {
        "token": "1389"
    }, {
        "token": "13891"
    }, {
        "token": "138919"
    }, {
        "token": "1389197"
    }, {
        "token": "13891972"
```

```
    }, {
        "token": "138919726"
    }, {
        "token": "1389197263"
    }] |
+--
    ↪ ----------------------------------------------------------------------------------------------------------------
    ↪
```

- 文本搜索效果

```
SELECT * FROM customer_contacts_optimized WHERE phone MATCH '138';
+------+-------------+
| id   | phone       |
+------+-------------+
|    1 | 13891972631 |
|    2 | 13812345678 |
+------+-------------+
SELECT * FROM customer_contacts_optimized WHERE phone MATCH '1389';
+------+-------------+
| id   | phone       |
+------+-------------+
|    1 | 13891972631 |
|    2 | 13812345678 |
+------+-------------+
2 rows in set (0.043 sec)
```

通过 Edge N-gram 分词器，一个电话号码被拆分成多个前缀 token，实现了灵活的前缀匹配搜索。

8.2.4.3 三、湖仓一体能力再跃新高

8.2.4.3.1 异步物化视图全面支持数据湖

在 3.1 版本中，异步物化视图再次进化，现在可以完整支持 Paimon / Iceberg / Hudi 的分区增量构建和分区透明改写。Doris 自 2.1 版本支持异步物化视图功能开始。经过多个版本的迭代。已经支持了非常多有价值的特性。包括：

异步物化视图全面支持数据湖

3.1 版本则重点打磨湖仓一体方向上的功能，全面支持主流的数据湖表格式 Paimon / Iceberg / Hudi 的分区刷新，和透明改写时的外部数据源分区补偿。使其成为联通湖和仓之间的高速公路。具体支持范围详见下表：

异步物化视图全面支持数据湖

8.2.4.3.2 Iceberg / Paimon 能力全面扩充

Iceberg

3.1.0 版本针对 Iceberg 表格式上做出多项优化和能力增强，紧密推进与 Iceberg 最新特性的融合。

支持 Branch / Tag 完整生命周期管理

从 3.1.0 开始，Doris 原生支持 Iceberg Branch & Tag 的创建、删除、读取与写入操作。该功能能够让用户像 Git 一样操作和管理 Iceberg 表数据。这一能力为 Iceberg 表格式的多版本并行管理、灰度测试、环境隔离等业务场景提供了原生的支持，无需额外引擎或自定义逻辑。

```
-- 创建分支
ALTER TABLE iceberg_tbl CREATE BRANCH b1;
-- 写入数据到指定分支
INSERT INTO iceberg_tbl@branch(b1) values(1, 2);
-- 查询指定分支
SELECT * FROM iceberg_tbl@branch(b1);
```

丰富的 Iceberg 系统表支持

3.1.0 新增对 Iceberg `$entries`，`$files`，`$history`，`$manifests`，`$refs`，`$snapshots` 等系统表的支持，可用 `SELECT * FROM iceberg_table$history`、…`$refs` 等语句直接查询 Iceberg 的底层 metadata、snapshot 列表、分支/标签信息等，从而深入了解数据文件的组织结构、快照的变更历史以及分支的映射情况。这种能力大大提升了 Iceberg 元数据的可观测性，使得问题定位、调优分析和治理决策更加直观、透明。

如通过系统表查看 delete file 数量：

```
SELECT
  CASE
    WHEN content = 0 THEN 'DataFile'
    WHEN content = 1 THEN 'PositionDeleteFile'
    WHEN content = 2 THEN 'EqualityDeleteFile'
    ELSE 'Unknown'
  END AS ContentType,
  COUNT(*) AS FileNum,
  SUM(file_size_in_bytes) AS SizeInBytes,
  SUM(record_count) AS Records
FROM
  iceberg_table$files
GROUP BY
  ContentType;

+--------------------+---------+-------------+---------+
| ContentType        | FileNum | SizeInBytes | Records |
+--------------------+---------+-------------+---------+
| EqualityDeleteFile |    2787 |     1432518 |   27870 |
| DataFile           |    2787 |     4062416 |   38760 |
| PositionDeleteFile |      11 |       36608 |   10890 |
+--------------------+---------+-------------+---------+
```

Iceberg 视图查询

3.1.0 版本新增对 Iceberg 逻辑视图的访问和查询。该功能进一步提升了 Doris 对 Iceberg 功能的完善程度。在后续 3 位版本迭代中，我们将进一步支持 Iceberg View 的 SQL 方言转换能力。

通过 ALTER 语句修改 Iceberg 表结构

3.1.0 支持通过 ALTER TABLE 语句对 Iceberg 表进行字段的新增、删除、重命名和重排序操作。该功能进一步完善了 Doris 对 Iceberg 表的管理能力，无需再借助 Spark 等第三方引擎进行 Iceberg 表管理。

```
ALTER TABLE iceberg_table
ADD COLUMN new_col int;
```

同时，在 3.1.0 版本中，Iceberg 的依赖版本升级到 1.9.2，以便更好的支持 Iceberg 的新的功能。在后续 3.1 的迭代版本中，我们将进一步增强 Iceberg 的表管理能力，包括数据合并、分支演进等能力。

详情参考文档

Paimon

3.1.0 版本针对 Paimon 表格式，结合用户实际场景，进行了多项功能更新和能力增强。

支持 Paimon Batch Incremental Query

3.1.0 版本支持读取 Paimon 表指定的两个快照之间的增量数据。该功能增强了用户对 Paimon 表增量数据的访问能力。尤其是在增量物化视图构建方面，基于此功能实现了 Paimon 表的增量聚合物化视图能力。详见物化视图方面的说明。

```
SELECT * FROM paimon_tbl@incr('startSnapshotId'='2', 'endSnapshotId'='5');
```

支持 Branch / Tag 读取

从 3.1.0 开始，Doris 支持对 Paimon 表的 Branch / Tag 进行读取，帮助用户更灵活的访问多版本的 Paimon 数据。

```
SELECT * FROM paimon_tbl@branch(branch1);
SELECT * FROM paimon_tbl@tag(tag1);
```

丰富的 Paimon 系统表支持

同 Iceberg 一样，3.1.0 新增对 Paimon `$files`，`$partitions`，`$manifests`，`$tags`，`$snapshots` 等系统表的支持，可用 `SELECT * FROM partition_table$files` 等语句直接查询 Paimon 的底层元数据信息。更方便用户对 Paimon 表进行探测、调试和优化。

如我们可以通过系统表统计分区新增数据文件：

```
SELECT
  partition,
  COUNT(*) AS new_file_count,
  SUM(file_size_in_bytes)/1024/1024 AS new_total_size_mb
FROM my_table$files
WHERE creation_time >= DATE_SUB(NOW(), INTERVAL 3 DAY)
GROUP BY partition
ORDER BY new_total_size_mb DESC;
```

在 3.1.0 版本中，Paimon 的依赖版本升级到 1.1.1，以便更好的支持 Paimon 的新的功能。

详情参考文档

8.2.4.3.3 数据湖查询性能更上一层楼

3.1.0 版本，针对数据湖表格式的查询性能进行了多项深度优化，旨在实际生产环境下，为用户提供更加稳定、高效的数据湖分析能力。

动态分区裁剪

动态分区裁剪功能，能够在多表关联查询场景下，根据右表数据生成分区列谓词，并对左表数据进行运行时的分区剪枝，从而减少数据 IO，提升查询性能。在 3.0 版本中，Doris 已经支持了 Hive 表的动态分区裁剪功能。在 3.1.0 版本中，这个功能进一步扩充到了 Iceberg、Paimon 和 Hudi 表上。在测试场景下，针对选择率较高的查询，可以提升 30%-40% 的性能。

批量分片执行

当湖表的数据分片较多时，如果 FE 进行规划并将所有分片信息一次性组装完成发送给 BE，那么可能造成 FE 内存消耗过大以及处理实际过长的问题。尤其是在查询大数据量表时，会导致规划部分的资源开销大耗时长。批量分片执行功能，通过分批次生产数据分片信息，并且边生产变执行，能够有效缓解 FE 的内存开销，同时能够让分片信息的生产和执行并行执行，提升整体的执行效率。在 3.0 版本中，Doris 已经支持了 Hive 表上的该功能。在 3.1.0 版本中，进一步增加了对 Iceberg 表的批量分片执行支持。在大数据量测试场景下，可以显著降低 FE 的内存开销和查询规划时间。

8.2.4.3.4 联邦分析 - 连接器更好用更多样

3.1 版本重构了各个数据源的连接属性，不仅能够以更加清晰的方式对接各类元数据服务和数据存储系统，同时还支持了更加丰富的连接能力。

Iceberg Rest Catalog

3.1 版本进一步增强了对 Iceberg Rest Catalog 的支持。不仅支持了包括 Unity、Polaris、Gravitino、Glue 等多种 Iceberg Rest Catalog 后端实现，同时支持了 vended credentials 功能，能够更加安全、灵活的管理访问凭证。目前支持 AWS 平台，后续小版本迭代中将陆续支持 GCP、Azure 等云平台的凭证管理。

详情参考文档

支持 Paimon Rest Catalog

3.1.0 版本中支持基于阿里云 DLF 的 Paimon Rest Catalog，可以直接访问新版本 DLF 管理的 Paimon 表数据。

详情参考文档

多 Kerberos 环境支持

3.1 版本允许用户在同一个 Doris 集群内访问不同的 Kerberos 认证环境。不同的 Kerberos 环境可能采用不同的 KDC 服务、Principal 以及对应的 Keytab。新版本允许针对不同的 Catalog，配置不同的 Kerberos 认证信息，并且相互之间不受干扰。该功能极大的方便了拥有多套 Kerberos 认证环境的用户，可以使用 Doris 进行统一的访问管理。

详情参考文档

多 Hadoop 环境支持

在之前的版本中，Doris 只允许用户在 conf 目录下放置一套 hadoop 集群的配置文件（hive-site.xml，hdfs-site.xml 等）。如果用户有多套不同的 Hadoop 环境和配置，则无法支持。新版本运行用户为不同的 Catalog 指定不同的 Hadoop 配置文件，帮助用户更灵活的管理外部数据源。

详情参考文档

8.2.4.4 四、存储层构持续打磨

在 3.1 版本中，我们对存储层也进行了持续的打磨，性能和稳定性都有了显著的提升。

8.2.4.4.1 灵活列更新 - 数据更新全新体验

此前 Doris 的部分列更新功能要求一次导入中每一行必须更新相同的列。在一些场景下，源端系统输出的记录往往只包含主键和被更新的列，不同行更新的列可能不同。为了解决这种需求，Doris 引入了 灵活列更新功能，使用灵活列更新可以大幅简化用户侧按列攒数据的工作以及提升写入性能。

使用方式

- 创建 Merge-on-Write Unique 表时，在表属性中开启：
- `"enable_unique_key_skip_bitmap_column" = "true"`
- 在导入时指定导入模式：
- `unique_key_update_mode: UPDATE_FLEXIBLE_COLUMNS`
- Doris 会自动完成灵活列更新与数据补齐。

示例支持在一次导入中对不同记录更新不同的列，例如：

- 删除某行（`DORIS_DELETE_SIGN`）
- 更新部分列（如 v1、v2、v5 等）
- 插入新行（仅提供主键和被更新列，其他列使用默认值或补齐历史值）

效果

- 在测试环境下（1 FE + 4 BE，16C 64GB，3 亿行 101 列数据，3 副本行存表）：
- 每次导入 20,000 行，仅更新 1 列（需补齐 99 列）
- 单并发导入性能可达 10.4k 行/s
- 单机资源占用：CPU ~ 60%，内存 ~ 30GB，读 IOPS ~ 7.5k/s，写 IOPS ~ 5k/s

8.2.4.4.2 存算分离 mow 锁优化

在存算分离场景下，MOW 表更新 Delete Bitmap 需要获取分布式锁 `delete_bitmap_update_lock`。原有实现中，导入、Compaction 和 Schema Change 会竞争该锁，容易在高并发导入场景下导致长时间等待甚至失败。

本次优化包括两方面：

减少 Compaction 持锁时间

- 通过引入新的 `mow_tablet_compaction_key`，避免多个 Compaction/Schema Change 任务在更新 `initiators` 列表时产生不必要的事务冲突。

- 在多 Tablet 高并发导入测试中，导入提交事务的 p99 平均耗时从 1.68 分钟降低到 49.4 秒，大幅降低了事务提交延迟。
- 新增配置项 `delete_bitmap_lock_v2_white_list`，支持为指定仓库开启该优化。

降低导入事务长尾延迟

- 增加 FE 配置 `mow_load_force_take_ms_lock_threshold_ms`，当导入事务等待锁超过阈值时，将强制获取分布式锁，避免长时间饥饿。
- 在高并发导入测试下，该优化显著减少了导入事务的长尾延迟。

8.2.4.5 五、查询性能提升

8.2.4.5.1 分区裁剪性能和适用范围提升

Doris 支持数据按照分区组织，这些分区可以独立存储、独立查询、独立管理。通过分区，可以提升查询性能、优化数据管理，并降低资源消耗。在查询过程中，通过使用过滤条件，提前过滤无需查询的分区，可以显著提升查询性能，降低系统资源消耗。在日志数据分析系统，风控系统等使用场景中，单表可能存在万级别甚至十万级别的分区数量，而通常单词查询，只会命中百级别以下的分区数据。对于在这类数据上的查询，能否分区裁剪，对于查询性能的影响十分显著。

在 3.1 版本中，Doris 通过引入一系列的优化，显著提升了分区裁剪的性能和适用范围，包括

- 分区裁剪二分查找。对于在时间列上的分区，通过将分区按照列值排序，将分区裁剪的计算从线性遍历，改为二分查找。在使用 DATETIME 类型作为分区字段，13.6 万分区数的场景下。实测分区裁剪的耗时，从 724ms 提升到 43ms。提升超过 16 倍。
- 增加大量单调函数参与分区裁剪。在实际的使用场景中，在时间分区列上的过滤条件，通常不是简单的逻辑比较，而是在分区列上包含时间函数计算的复杂表达式。如：`to_date(time_stamp)> '2022-12-22'`，`date_format(timestamp,'%Y-%m-%d %H:%i:%s')> '2022-12-22 11:00:00'`等。Doris 在 3.1 版本中引入了函数单调特性描述，当函数为单调函数时，可以通过计算分区边界值是否可以被裁剪得知整个分区是否可以被裁剪。在 3.1 版本中，已经支持了 CAST 和 25 个常见的时间相关的函数。可以覆盖绝大多数常见的时间类型分区列上的过滤条件。
- 此外，3.1 版本中，还对分区裁剪的全路径代码做了许多代码级别的详细优化，减少了不必要开销。

8.2.4.5.2 洞察数据特征 - 获得性能 10 倍的潜力

在 3.1 中，优化器可以更聪明的使用数据特征对查询进行优化。优化器会执行计划树种各个节点的收集唯一性（Unique）、均一性（Uniform），等值集（Equal Set）等数据特征，并推导列之间的函数依赖关系。当在特定的节点，数据符合特定特征时，可以移除不必要的连接、聚合或排序计算，显著提升查询性能。

在针对特定优化构建的测试用例下，利用数据特征可以获得超过 10 倍的性能提升，详见下表：

洞察数据特征 - 获得性能 10 倍的潜力

8.2.4.6 六、功能改进

8.2.4.6.1 半结构化

VARIANT

- 新增 `variant_type(x)`函数：返回 Variant 子 field 对应的“当前实际类型”。
- 新增 ComputeSignature/Helper，增强函数参数/返回类型推断能力。

STRUCT

- 支持使用 Schema Change 为 STRUCT 类型增加子列

8.2.4.6.2 湖仓一体

- 支持在 Catalog 级别设置元数据缓存策略，如缓存过期时间等。帮助用户根据需求灵活调整数据时效性和元数据访问性能。详情参考文档
- 支持 `FILE()` 表函数（Table Valued Function），该表函数是原有的 `S3()`，`HDFS()`，`LOCAL()`表函数的集合，方便用户使用和理解。

8.2.4.6.3 聚合算子能力增强

在 3.1 版本中，优化器重点增强了聚合算子。支持了两个使用较为广泛的能力。

非标 GROUP BY 支持

对于标准的聚合查询，要求聚合输出的标量表达式，其本身或其子树必须是聚合键。但在 MySQL 中，当设置了 SQL_MODE 不包含“ONLY_FULL_GROUP_BY”时，则没有次限制。详见 MySQL 文档

此时，此列输出的值为聚合键对应多行中的任意一行计算的值。举例如下：

```
-- 非标 GROUP BY
SELECT c1, c2 FROM t GROUP BY c1
-- 等价于
SELECT c1, any_value(c2) FROM t GROUP BY c1
```

在 3.1 版本中。Doris 在 SQL_MODE 中默认开启“ONLY_FULL_GROUP_BY”，即和之前的行为保持一致。如果需要使用非标 GROUP BY 功能。则可以通过如下设置开启：

```
set sql_mode = replace(@@sql_mode, 'ONLY_FULL_GROUP_BY', '');
```

多 distinct 聚合支持

在之前的版本中，如果聚合查询中，包含多个 distinct 聚合函数，且他们的参数不一致。同时聚合函数的 distinct 语义和非 distinct 语义不一致，且不是以下之一，则 Doris 无法执行查询：

- 单参数的 COUNT
- SUM
- AVG
- GROUP_CONCAT

在 3.1 版本中，Doris 对此方面进行了加强。现在这些查询可以正常执行并获取结果。例如：

```
SELECT count(DISTINCT c1,c2), count(DISTINCT c2,c3), count(DISTINCT c3) FROM t;
```

8.2.4.6.4 连接协议增强

- 开启 Proxy Protocol 协议后，依然可以通过非该协议的客户端连接。该改进在负载均衡 IP 透传场景下，方便用户更灵活的连接 Doris。
- 查询 VIEW 时，JDBC 的元数据接口 ResultSetMetaData#getColumnName 可以正确的返回 VIEW 中的列名

8.2.4.7 七、行为变更

8.2.4.7.1 VARIANT

- `variant_max_subcolumns_count` 约束
- 同一张表中，所有 Variant 列的 `variant_max_subcolumns_count` 必须“要么全为 0，要么全为 > 0”。混用会在建表 / Schema Change 时报错。
- 新的 Variant 读写/serde 与 Compaction 路径对旧数据兼容。老版本 Variant 升级上来查询格式会产生差异（比如多一些空格、或是。分隔符导致层级构建，产生额外的层级）
- 创建 Variant 倒排索引，如果数据中所有字段不符合索引条件也会生成空索引文件，属预期行为

8.2.4.7.2 权限

- show transcation 的权限需求从拥有 ADMIN_PRIV 权限，变更为拥有导入对应数据库的 LOAD_PRIV 权限
- 统一了 SHOW FRONTENDS / BACKENDS 和 NODE Restful API 的权限。现在这些接口的权限需求为拥有 information_schema 库的 SELECT_PRIV 权限。

8.2.4.8 立刻开启 3.1

在 3.1 版本正式发布之前，半结构化和数据湖的多个能力已经经过真实线上场景的验证，并获得了符合预期的性能提升。推荐有相应能力需求的用户下载尝鲜。

8.2.4.9 致谢

在此，再次向所有参与版本研发、测试和需求反馈的贡献者们表示最衷心的感谢：

@924060929 @airborne12 @amorynan @BePPPower @BiteTheDDDDt @bobhan1 @CalvinKirs @cambyzju @cjj2010 @csun5285 @DarvenDuan @dataroaring @deardeng @dtkavin @dwdwqfwe @eldenmoon @englefly @feifeifeimoon @feiniaofeiafei @felixwluo @freemandealer @Gabriel39 @gavinchou @ghkang98 @gnehil @gohalo @HappenLee @heguanhui @hello-stephen @HonestManXin @htyoung @hubgeter @hust-hhb @jacktengg @jeffreys-cat @Jibing-Li @JNSimba @kaijchen @kaka11chen @KeeProMise @koarz @liaoxin01 @liujiwen-up @liutang123 @luwei16 @MoanasDaddyXu @morningman @morrySnow @mrhhsg @Mryange @mymeiyi @nsivarajan @qidaye @qzsee @Ryan19929 @seawinde @shuke987 @sollhui @starocean999 @suxiaogang223 @SWJTU-ZhangLei @TangSiyang2001 @Vallishp @vinlee19 @w41ter @wangbo @wenzhenghu @wumeibanfa @wuwenchi @wyxxxcat @xiedeyantu @xinyiZzz @XLPE @XnY-wei @XueYuhai @xy720 @yagagagaga @Yao-MR @yiguolei @yoock @yujun777 @Yukang-Lian @Yulei-Yang @yx-keith @Z-SWEI @zclllyybb @zddr @zfr9527 @zgxme @zhangm365 @zhangstar333 @zhaorongsheng @zhiqiang-hhhh @zy-kkk @zzzxl1993

8.3 v3.0

8.3.1 Release 3.0.8

8.3.1.1 行为变更

- 当使用 ranger / LDAP 时，不再禁止在 Doris 中创建用户 #50139
- variant 在默认情况下会关闭 nested 属性，若需在建表时开启，需先在 session variable 中执行以下命令：`set enable_variant_flatten_nested = true`#54413

8.3.1.2 新特性

8.3.1.2.1 查询优化器

- 支持 MySQL 的 GROUP BY WITH ORDER 语法 #53037

8.3.1.3 改进

8.3.1.3.1 导入

- 优化内存不足时的下刷策略 (#52906, #53909, #42649, #54517)
- S3 Load 和 TVF 支持无 AK/SK 访问公开可读的对象 (#53592, #54040)

8.3.1.3.2 存算分离

- 当缓存空间充足时，base compaction 生成的 rowset 可以写入文件缓存 (#53801, #54693)
- 优化 `ALTER STORAGE VAULT` 命令，`type`属性可以自动推导，无需显式制定 (#54394, #54475)

8.3.1.3.3 查询优化器

- 点查查询会被规划为只有一个 fragment，以提升点查的执行速度 #53541

8.3.1.3.4 查询执行

- 提升 unique key 表在点查时的性能 #53948

8.3.1.3.5 倒排索引

- 优化不分词索引写入时常见默认分词器的额外资源消耗 #54666

8.3.1.4 缺陷修复

8.3.1.4.1 导入

- 修复在使用多字符列分隔符时，`enclose` 解析错误的问题 (#54581, #55052)
- 修复 S3 Load 进度更新不及时的问题 (#54606, #54790)
- 修复 JSON 格式布尔类型加载到 INT 列时的错误 (#54397, #54640)
- 修复 Stream Load 缺失错误 URL 返回的问题 (#54115, #54266)
- 修复在 schema change 抛出异常后 group commit 被阻塞的问题 #54312

8.3.1.4.2 Lakehouse

- 修复部分情况下使用 JDBC SQL 透传解析失败的问题 #54077
- 修复写入 decimal 分区的 iceberg 表失败的问题 #54557
- 修复某些情况下 Hudi 表 Timestamp 类型分区列查询失败的问题 #53791

8.3.1.4.3 查询优化器

- 修复在部分自关联场景中，错误使用 colocate join 的问题 #54323
- 修复 select distinct 与窗口函数一起使用时可能导致的结果错误 #54133
- 当 lambda 表达式出现在非预期位置时，提供更友好的报错 #53657

8.3.1.4.4 权限

- 修复查询外部视图时，错误检查视图中基表权限的问题 #53786

8.3.1.4.5 查询执行

- 修复 IPV6 类型不能解析 IPV4 类型数据的问题 #54391
- 修复 IPV6 类型解析时出现栈溢出的错误 #53713

8.3.1.4.6 复杂数据类型

- BE 支持启动时选择符合指令集的 simdjson parser #52732
- 修复 variant nested 数据类型在数据类型冲突情况下导致的错误类型推断 #53083
- 修复 variant nested 顶层嵌套 array 数据默认值填充问题 #54396
- 禁止 variant 类型在 cloud 上 build index #54777
- 修复 variant 创建倒排索引后写入不符合索引条件的数据时生成空索引文件的问题 #53814

8.3.1.4.7 其他

schema-change

- 修复在清理失败的 SC 任务时新 tablet 为空的问题 (#53952, #54064)
- 按原有顺序重建 bucket 列 (#54024, #54072, #54109)
- 禁止删除 bucket 列 #54037
- 网络错误时支持自动重试 (#54419, #54488)
- 避免在 `tabletInvertedIndex` 上的死锁 (#54197, #54996)

8.3.2 Release 3.0.7

8.3.2.1 行为变更

- 调整 show frontends 和 show backends 的权限需求，使其与对应的 RESTful API 保持一致，即需要 information_schema 库的 SELECT_PRIV 权限
- 指定 domain 的 admin 和 root 用户不再视为系统用户
- 存储：单库默认并发事务数调整为 10000

8.3.2.2 新特性

8.3.2.2.1 查询优化器

- 支持 MySQL 的聚合上卷语法 GROUP BY ... WITH ROLLUP

8.3.2.2.2 查询执行

- 新增数据函数：cot/sec/cosec
- Like 语句支持 escape 语法

8.3.2.2.3 半结构化数据管理

- 通过设置会话变量 enable_add_index_for_new_data=true，支持仅对新增数据构建不分词倒排索引和 NGram bloomfilter 索引

8.3.2.3 改进

8.3.2.3.1 导入

- 优化 SHOW CREATE LOAD 错误信息提示

8.3.2.3.2 主键

- 新增 segment key bounds 截断能力，避免单次大导入失败的问题

8.3.2.3.3 存储

- 增强 Compaction 和导入数据的可靠性
- 优化 balance 速度
- 优化建表速度
- 优化 compaction 默认参数及可观测性
- 优化查询报错 -230 的问题
- 增加系统表 backend_tablets
- 优化 Cloud 模式下从 follower 节点查询 information_schema.tables 的性能

8.3.2.3.4 存算分离

- 增强 Meta-service recycler 可观测性
- 支持导入 compaction 过程进行跨 compute group 增量预热
- 优化 Storage vault 连通性检查
- 支持通过 MS API 更新存储后端信息

8.3.2.3.5 Lakehouse

- 优化 x86 环境下 ORC zlib 的解压性能并修复潜在问题
- 优化外表读取的默认并发线程数
- 优化不支持 DDL 操作的 Catalog 的报错信息

8.3.2.3.6 异步物化视图

- 优化透明改写规划的性能

8.3.2.3.7 查询优化器

- `group_concat` 函数现在允许参数为非字符串类型
- `sum` 和 `avg` 函数允许参数为非数值类型
- 扩展 TOP-N 查询延迟物化的支持范围，当查询部分列时也能延迟物化
- 创建分区时，list 分区允许包含 `MAX_VALUE`
- 优化采样收集聚合模型表统计信息的性能
- 优化采样收集统计信息时 NDV 值的准确性

8.3.2.3.8 倒排索引

- 统一 `show create table` 中倒排索引展示的 properties 顺序
- 为倒排索引过滤条件新增逐条件的 profile 指标（如命中行数与执行时间），便于性能分析
- 增强 profile 中倒排索引相关信息展示

8.3.2.3.9 权限

- Ranger 支持设置 storage vault 和 compute group 的权限

8.3.2.4 缺陷修复

8.3.2.4.1 导入

- 修复导入 CSV 文件使用多字符分隔符可能导致的正确性问题
- 修复修改任务属性后显示 `ROUTINE LOAD` 任务结果不正确的问题
- 修复主节点重启或 Leader 切换后一流多表导入计划失效的问题
- 修复 `ROUTINE LOAD` 任务因找不到可用 BE 节点导致所有调度任务阻塞的问题
- 修复 `runningTxnIds` 并发读写冲突问题

8.3.2.4.2 主键

- 优化 mow 表在高频并发导入下的导入性能
- mow 表 full compaction 释放被删除数据的空间
- 修复 mow 表在极端场景下可能出现的导入失败问题
- 优化 mow 表 compaction 性能
- 修复 mow 表在有并发导入和 sc 时可能的正确性问题
- 修复 mow 空表执行 schema change 可能导致导入卡住或 schema change 失败的问题
- 修复 mow delete bitmap cache 内存泄漏问题
- 修复 mow 表在 sc 后可能的正确性问题

8.3.2.4.3 存储

- 修复 compaction 导致的 clone 过程 missing rowset 问题
- 修复 autobucket 计算 size 不准确及默认值问题
- 修复分桶列可能导致的正确性问题
- 修复单列表不能 rename 的问题
- 修复 memtable 可能的内存泄漏问题
- 修复空表事务写对不支持行为的报错不统一问题

8.3.2.4.4 存算分离

- File cache 相关修复
- 修复 schema 过程中 cumulative point 可能回滚的问题
- 修复后台任务影响自动重启的问题
- 修复 azure 环境中数据回收过程未处理的异常问题
- 修复单 rowset 做 compaction 未及时清理 file cache 的问题

8.3.2.4.5 Lakehouse

- 修复 Kerberos 环境下 Iceberg 表写入事务提交失败的问题
- 修复 kerberos 环境下查询 hudi 的问题
- 修复多 Catalog 情况下潜在的死锁问题
- 修复某些情况下并发刷新 Catalog 导致元数据不一致的问题
- 修复 ORC footer 某些情况下会被多次读取的问题
- 修复 Table Valued Function 无法读取压缩格式 json 文件的问题
- SQL Server Catalog 支持识别 IDENTITY 列信息
- SQL Convertor 支持指定多个 url 以实现高可用

8.3.2.4.6 异步物化视图

- 修复当查询被优化为空集结果时，可能错误进行分区补偿的问题

8.3.2.4.7 查询优化器

- 修复 sql_select_limit 以外的影响 DML 执行结果的问题
- 修复开始 local shuffle 时，物化的 CTE 在极端情况下可能执行报错的问题
- 修复 prepare 的 insert 语句无法在非 master 节点执行的问题
- 修复 cast ipv4 到 string 的结果错误问题

8.3.2.4.8 权限

- 当一个用户拥有多个角色时，会合并多个角色的权限后再执行鉴权

8.3.2.4.9 查询执行

- 修复部分 json 函数问题
- 修复异步线程池满时可能导致 BE Core 的问题
- 修复 hll_to_base64 结果不正确的问题
- 修复 decimal256 转换为 float 时结果错误的问题
- 修复两处内存泄漏问题
- 修复 bitmap_from_base64 导致的 be core 问题
- 修复 array_map 函数可能导致的 be core 问题
- 修复 split_by_regexp 函数可能的错误问题
- 修复超大数据量下 bitmap_union 函数可能的结果错误问题
- 修复 format round 函数在部分边界值下可能 core 的问题

8.3.2.4.10 倒排索引

- 修复倒排索引在异常情况下产生的内存泄漏问题
- 修复写入和查询空索引文件时报错的问题
- 捕获倒排索引字符串读取中的 IO 异常，避免因异常导致进程崩溃

8.3.2.4.11 复杂数据类型

- 修复 Variant Nested 嵌套数据类型冲突时可能导致的类型推断错误
- 修复 map 函数参数类型推导错误
- 修复 jsonpath 中指定 '$.' 作为 path 导致数据错误变为 NULL 的问题
- 修复 Variant 的子字段包含 . 时，序列化格式无法还原的问题

8.3.2.4.12 其他

- 修复 auditlog 表 IP 字段长度不足的问题
- 修复 SQL 解析错误时，审计日志中记录的 query id 为上一次执行查询的 query id 的问题

8.3.3 Release 3.0.6

亲爱的社区小伙伴们，Apache Doris 3.0.6 版本已于 2025 年 06 月 16 日正式发布。 该版本进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- GitHub 下载
- 官网下载

8.3.3.1 行为变更

- 禁止 Unique 表使用时序 Compaction #49905
- 存算分离场景下 Auto Bucket 单分桶容量调整为 10GB #50566

8.3.3.2 新特性

8.3.3.2.1 Lakehouse

- 支持访问 AWS S3 Table Buckets 中的 Iceberg 表格式
  - 详情请参考文档：Iceberg on S3 Tables

8.3.3.2.2 存储

- 对象存储访问支持 IAM Role 授权适用于导入/导出、备份恢复及存算分离场景 #50252 #50682 #49541 #49565 #49422
  - 详情请参考文档

8.3.3.2.3 新增函数

- `json_extract_no_quotes`
  - 详情请参考文档
- `unhex_null`
  - 详情请参考文档
- `xpath_string`
  - 详情请参考文档
- `str_to_map`
  - 详情请参考文档
- `months_between`
  - 详情请参考文档

- `next_day`
  - 详情请参考文档
- `format_round`
  - 详情请参考文档

8.3.3.3 改进

8.3.3.3.1 导入

- 引入黑名单机制：避免 Routine Load 将元信息分发至不可用 BE 节点 #50587
- 提高负载优先级阈值：`load_task_high_priority_threshold_second` 默认值增大 #50478

8.3.3.3.2 主键模型

- 减少冗余日志输出 #51093

8.3.3.3.3 存储优化

- 精简 Compaction Profile 及日志 #50950
- 优化调度策略提升 Compaction 吞吐量 #49882 #48759 #51482 #50672 #49953 #50819

8.3.3.3.4 存算分离

- 启动优化：加速 File Cache 初始化 #50726
- 查询加速：优化 File Cache 查询性能 #50275 #50387 #50555
- 元数据获取优化：解决 `get_version` 导致的性能瓶颈 #51111 #50439
- 对象回收加速：提升存算分离模式垃圾回收效率 #50037 #50766
- 稳定性提升：优化对象存储重试策略 #50957
- Profile 细化：增强 Tablet/Segment Footer 维度统计 #49945 #50564 #50326
- Schema Change 容错：默认启用 New Tablet Compaction 规避 -230 错误 #51070

8.3.3.3.5 Lakehouse

Catalog 增强

- Hive Catalog 支持分区缓存 TTL 控制（`partition.cache.ttl-second`）#50724
- 详情参考文档：元数据缓存
- 支持 Hive 表 `skip.header.line.count` 属性 #49929
- 兼容 `org.openx.data.jsonserde.JsonSerDe` 格式的 Hive 表 #49958
- 详情参考文档：文本格式
- Paimon 版本升级至 1.0.1
- Iceberg 版本升级至 1.6.1

功能扩展

- 支持阿里云 OSS-HDFS Root Policy 功能 #50678
- 方言兼容：返回 Hive 格式查询结果 #49931
  - 详情参考文档：SQL 转换器

8.3.3.3.6 异步物化视图

- 内存优化：降低透明改写内存占用 #48887

8.3.3.3.7 查询优化器

- 分桶剪枝性能提升 #49388
- Lambda 表达式增强：支持引用闭包外部 Slot #44365

8.3.3.3.8 查询执行

- TopN 查询加速：优化存算分离场景性能 #50803
- 函数扩展：`substring_index` 支持变量参数 #50149
- 地理信息函数：新增 `ST_CONTAINS/ST_INTERSECTS/ST_TOUCHES/ST_DISJOINT` #49665

8.3.3.3.9 核心组件

- 内存追踪优化：高并发场景性能提升约 10% #50462
- 审计日志增强：通过 `audit_plugin_max_insert_stmt_length` 限制 INSERT 语句长度 #51314
  - 详情请参考文档：审计插件
- SQL 转换器控制：新增会话变量 `sql_convertor_config` 和 `enable_sql_convertor_features`
  - 详情请参考文档：SQL 转换器

8.3.3.4 缺陷修复

8.3.3.4.1 导入

- 修复 BE 事务清理失败问题 #50103
- 优化 Routine Load 任务报错准确性 #51078
- 禁止向 `disable_load=true` 节点分发元信息任务 #50421
- 修复 FE 重启后消费进度回退 #50221
- 修复 Group Commit 与 Schema Change 冲突导致的 Core Dump #51144
- 解决 S3 Load 使用 HTTPS 协议报错 #51246 #51529

8.3.3.4.2 主键模型

- 修复竞争导致的主键重复问题 #50019 #50051 #50106 #50417 #50847 #50974

8.3.3.4.3 存储

- 解决 CCR 与磁盘均衡竞争 #50663
- 修复默认分区 Key 未持久化问题 #50489
- CCR 支持 Rollup 表 #50337
- 修复 `cooldown_ttl=0` 边界问题 #50830
- 解决数据 GC 与 Publish 竞争导致数据丢失 #50343
- 修复 Delete Job 分区剪枝失效 #50674

8.3.3.4.4 存算分离

- 修复 Schema Change 阻塞 Compaction #50908
- 解决 `storage_vault_prefix` 为空时对象回收失败 #50352
- 修复 Tablet Cache 导致的查询性能问题 #51193 #49420
- 消除残留 Tablet Cache 引起的性能抖动 #50200

8.3.3.4.5 Lakehouse

Export 修复

- 解决 FE 内存泄漏 #51171
- 避免 FE 死锁 #50088

Catalog 修复

- JDBC Catalog 支持组合条件下推 #50542
- 修复阿里云 OSS Paimon 表 Deletion Vector 读取 #49645
- 支持含逗号的 Hive 表分区值 #49382
- 修正 MaxCompute Timestamp 列类型解析 #49600
- Trino Catalog 支持显示 `information_schema` 系统表 #49912

文件格式

- 修复 LZO 压缩格式读取失败 #49538
- 兼容旧版 ORC 文件 #50358
- 修正 ORC 复杂类型解析错误 #50136

8.3.3.4.6 异步物化视图

- 修复同时指定 `start time` 与立即触发模式时的少刷新问题 #50624

8.3.3.4.7 查询优化器

- 修复 Lambda 表达式改写错误 #49166
- 解决 Group By 常量键规划失败 #49473
- 修正常量折叠逻辑 #50142 #50810
- 补全系统表信息 #50721
- 修复 NULL Literal 创建 View 的列类型错误 #49881

8.3.3.4.8 查询执行

- 解决 JSON 导入非法值导致 BE Core #50978
- 修复 Intersect 输入 NULL 常量结果错误 #50951
- 修正 Variant 类型谓词错误执行 #50934
- 修复 `get_json_string` JSON Path 非法时的结果错误 #50859
- 对齐 MySQL 函数行为（JSON_REPLACE/INSERT/SET/ARRAY）#50308
- 解决 `array_map` 空参数 Core #50201
- 修复 Variant 转 JSONB 异常 Core #50180
- 修复 `explode_json_array_json_outer` 函数缺失 #50164
- 对齐 `percentile` 与 `percentile_array` 结果 #49351
- 优化 UTF8 编码函数行为（url_encode/strright/append_trail_char_if_absent）#49127

8.3.3.4.9 其他

- 修复高并发下审计日志丢失 #50357
- 解决动态分区建表导致元数据回放失败 #49569
- 避免 Global UDF 重启丢失 #50279
- 对齐 MySQL View 元数据返回格式 #51058

8.3.4 Release 3.0.5

亲爱的社区小伙伴们，Apache Doris 3.0.5 版本已于 2025 年 04 月 28 日正式发布。 该版本进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- GitHub 下载
- 官网下载

8.3.4.1 新特性

8.3.4.1.1 Lakehouse

- FE Metrics 新增 Catalog/Database/Table 数量监控指标（#47891）
- MaxCompute Catalog 支持 Timestamp 类型（#48768）

8.3.4.1.2 查询执行

- 新增 URL 处理函数：`top_level_domain`、`first_significant_subdomain`、`cut_to_first_significant_subdomain` (#42488)
- 新增 `year_of_week` 函数，兼容 Trino 语法实现 (#48870)
- `percentile_array` 函数支持 Float 和 Double 数据类型 (#48094)

8.3.4.1.3 存算分离

- 支持重命名计算组 (Rename Compute Group) (#46221)

8.3.4.2 改进

8.3.4.2.1 存储

- 优化主键表 (MOW) 高频导入场景的查询性能 (#48968)
- 优化 Key Range 查询的 Profile 信息展示 (#48191)
- Stream Load 支持 JSON 压缩文件导入 (#49044)
- 优化多个导入场景的错误提示信息 (#48436 #47721 #47804 #48638 #48344 #49287 #48009)
- 新增 Routine Load 多项监控指标 (#49045 #48764)
- 优化 Routine Load 调度算法，避免单任务异常影响整体调度 (#47847)
- 新增 Routine Load 系统表 (#49284)
- 优化 Compaction 任务生成速度以提升性能 (#49547)

8.3.4.2.2 存算分离

- 修复多个 File Cache 稳定性及性能问题 (#48786 #48623 #48687 #49050 #48318)
- 优化 Storage Vault 创建校验逻辑 (#48073 #48369)

8.3.4.2.3 Lakehouse

- 优化 Trino Connector Catalog 的 BE 端 Scanner 关闭逻辑，加速内存释放 (#47857)
- ClickHouse JDBC Catalog 自动兼容新旧版本驱动 (#46026)

8.3.4.2.4 异步物化视图

- 优化透明改写 (Transparent Rewrite) 的规划性能 (#48782)
- 优化 `tvf mv_infos` 性能 (#47415)
- 基于外部表的物化视图构建时取消 Catalog 元数据刷新，减少内存占用 (#48767)

8.3.4.2.5 查询优化器

- 优化 Key 列与分区列的统计信息收集性能（#46534）
- 查询结果别名与用户输入保持严格一致（#47093）
- 优化聚合算子中公共子表达式抽取后的列裁剪逻辑（#46627）
- 增强函数绑定失败及子查询不支持的报错信息（#47919 #47985）

8.3.4.2.6 半结构化数据管理

- `json_object` 函数支持复杂类型参数（#47779）
- 支持将 UInt128 写入 IPv6 类型（#48802）
- 支持 VARIANT 类型中 ARRAY 字段的倒排索引（#47688 #48117）

8.3.4.2.7 权限

- 提升 Ranger 鉴权性能（#49352）

8.3.4.2.8 其他

- 优化 JVM Metrics 接口性能（#49380）

8.3.4.3 Bug 修复

8.3.4.3.1 存储

- 修复若干极端场景下的数据正确性问题（#48056 #48399 #48400 #48748 #48775 #48867 #49165 #49193 #49350 #49710 #49825）
- 修复已完成事务未及时清理的问题（#49564）
- 部分列更新时 JSONB 类型默认值改用 `{}`（#49066）
- 修复存算分离主键模型 Compaction 未释放 Delete Bitmap 锁导致导入卡顿的问题（#47766）
- 修复 ARM 架构下 Stream Load 数据丢失问题（#49666）
- 修复 Insert Into Select 遇到数据质量错误未返回错误 URL 的问题（#49687）
- 修复 Routine Load 多表导入时数据质量错误未返回错误 URL 的问题（#49130）
- 修复 Schema Change 期间 Insert Into Values 导入结果异常问题（#49338）
- 修复 Tablet Commit 信息上报导致的 Core Dump 问题（#48732）
- 修复 S3 Load 导入不支持 Azure 中国区域名的问题（#48642）
- 修复 K8s 环境下 FE 报 “get image failed” 错误（#49072）
- 优化动态分区调度的 CPU 消耗（#48577）
- 修复重命名物化视图（MV）导致列异常的问题（#48328）
- 修复 Schema Change 失败后未释放内存和 File Cache 的问题（#48426）
- 修复含空分区表的 Base Compaction 失败问题（#49062）
- 修复复杂类型变更导致的数据正确性问题（#49452）
- 修复 Cold Compaction 导致 Core Dump 的问题（#48329）
- 修复存在 Delete 操作时 Cumulative Point 未提升的问题（#47282）
- 修复大数据量 Full Compaction 内存不足问题（#48958）

8.3.4.3.2 存算分离

- 修复 K8s 环境下 File Cache 清除失败问题（#49199）
- 修复高频导入时读写锁导致的 FE CPU 飙升问题（#48564）

8.3.4.3.3 Lakehouse

Data Lakes

- 修复并发写入 Hive/Iceberg 表可能引发的 BE Core Dump（#49842）
- 修复 AWS S3 存储的 Hive/Iceberg 表写入失败问题（#47162）
- 修复 Iceberg Position Deletion 读取结果错误（#47977）
- 修复腾讯云 COS 无法创建 Iceberg 表的问题（#49885）
- 修复 Kerberos 认证 HDFS 访问 Paimon 数据失败问题（#47192）
- 修复 Hudi Jni Scanner 内存泄漏问题（#48955）
- 修复 MaxCompute Catalog 多分区列表读取错误（#48325）

JDBC

- 修复 JDBC Catalog 表行数查询空指针问题（#49442）
- 修复 OceanBase Oracle 模式连接测试失败（#49442）
- 修复 JDBC Catalog 并发场景下列类型长度错误（#48541）
- 修复 JDBC Catalog BE 端 Classloader 泄漏（#46912）
- 修复 PostgreSQL JDBC Catalog 连接线程泄漏（#49568）

Export

- 修复 EXPORT 作业卡在 EXPORTING 状态（#47974）
- 禁止 OUTFILE 自动重试以防止重复文件导出（#48095）

其他

- 修复 FE WebUI 执行 TVF 查询空指针问题（#49213）
- 修复 Hadoop Libhdfs Thread Local 空指针异常（#48280）
- 修复 FE 访问 Hadoop Filesystem 报“Filesystem already closed”（#48351）
- 修复 Catalog Comment 未持久化问题（#46946）
- 修复 Parquet 复杂类型读取报错（#47734）

8.3.4.3.4 异步物化视图

- 修复极端场景下物化视图构建任务卡顿问题（#48074）
- 修复嵌套物化视图透明改写失效问题（#48222）

8.3.4.3.5 查询优化器

- 修复函数常量折叠计算结果错误（#49225 #47966 #49416 #49087 #49033 #49061 #48895 #48957 #47288 #48641 #49413 #48783）
- 修复嵌套窗口函数使用 ORDER BY 子句意外报错（#48492）

8.3.4.3.6 查询执行

- 修复 Pipeline 任务调度导致的卡死/性能问题（#49976 #49007）
- 修复 FE 连接失败时的内存越界问题（#48370 #48313）
- 修复 Lambda 函数与数组函数共用导致的内存越界（#49140）
- 修复 String 与 JSONB 类型转换空值导致 BE Core（#49810）
- 规范 `parse_url` 未定义行为（#49149）
- 修复 `array_overlap` 函数空值结果异常（#49403）
- 修复非 ASCII 字符大小写转换错误（#49763）
- 修复 `percentile` 函数部分场景 BE Core（#48563）
- 修复多个内存越界问题（#48288 #49737 #48018 #47964）
- 修复 SET 算子结果错误（#48001）
- 降低 Arrow Flight 默认线程池大小以避免句柄耗尽（#48530）
- 修复窗口函数内存越界导致 BE Core（#48458）

8.3.4.3.7 半结构化数据管理

- 修复 Transfer-Encoding: chunked 的 Stream Load JSON 导入异常（#48474）
- 增强 JSONB 格式合法性校验（#48731）
- 修复 STRUCT 类型字段过多导致的 Crash（#49552）
- 支持复杂类型 VARCHAR 长度扩展（#48025）
- 修复 `array_avg` 函数在特定参数下的 Crash（#48691）
- 修复 VARIANT 类型 `ColumnObject::pop_back` Crash（#48935 #48978）
- 禁用 VARIANT 类型的索引构建操作（#49844）
- 禁用 VARIANT 类型倒排索引 V1 格式（#49890）
- 修复 VARIANT 多层 CAST 结果错误（#47954）
- 优化 VARIANT 多子列倒排索引元数据查询性能（#48153）
- 优化存算分离模式下 VARIANT Schema 内存消耗（#47629 #48463）
- 修复 PreparedStatement ID 溢出问题（#48116）
- 修复行存与 Delete 操作结合问题（#49609）

8.3.4.3.8 倒排索引

- 修复 ARRAY 类型倒排索引 Null Bitmap 错误（#48052）
- 修复 Date/Datetimev1 类型 Bloomfilter 索引比较错误（#47005）
- 修复 UTF-8 四字节字符截断问题（#48792）
- 修复新增列后立即创建倒排索引导致丢失的问题（#48547）
- 修复 ARRAY 倒排索引空数据处理异常（#48264）

- 修复倒排索引 FE 元数据升级兼容性（#49283）
- 修复 `match_phrase_prefix` 缓存错误（#46517）
- 修复 Compaction 后倒排索引 File Cache 未清理（#49738）

8.3.4.3.9 权限

- DELETE 操作不再检查 Select_Priv 权限（#49239）
- 禁止非 root 用户修改 root 权限（#48752）
- 修复 LDAP 偶发 Partial Result Exception（#47858）

8.3.4.3.10 其他

- 修复 JDK17 环境 JAVA_OPTS 识别异常（#48170）
- 修复 InterruptException 导致 BDB 元数据写入失败（#47874）
- 优化多语句请求的 SQL Hash 生成（#48242）
- 用户属性变量优先级高于 Session 变量（#48548）

8.3.5 Release 3.0.4

亲爱的社区小伙伴们，Apache Doris 3.0.4 版本已于 2025 年 02 月 28 日正式发布。 该版本进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- GitHub 下载：https://github.com/apache/doris/releases
- 官网下载：https://doris.apache.org/download

8.3.5.1 行为变更

- 在 Audit Log 中，`drop table` 和 `drop database` 语句保持 `force` 标志。#43227
- 导出数据至 Parquet/ORC 格式时，`bitmap`、`quantile_state` 和 `hll` 类型将以 Binary 格式导出。同时新增支持导出 `jsonb` 和 `variant` 类型，导出格式为 `string`。#44041
- 更多内容，参考文档：Export Overview - Apache Doris
- 当通过 External Catalog 查询表名大小写不敏感的数据源（如 Hive）时，在之前版本中，可以使用任意大小写进行表名查询，但是在 3.0.4 版本中，将严格遵循 Doris 自身的表名大小写敏感策略。
- 将 Hudi JNI Scanner 从 Spark API 替换为 Hadoop API，以增强兼容性。用户可以通过设置会话变量 `set hudi_jni_scanner=spark/hadoop` 进行切换。#44267
- 禁止在 Colocate 表中使用 `auto bucket`。#44396
- 为 Catalog 增加 Paimon 缓存，不再进行实时数据查询。#44911
- 增大 `max_broker_concurrency` 的默认值，以提升 Broker Load 在大规模数据导入时的性能。#44929

- 将 Auto Partition 分区的 `storage medium` 默认值修改为当前表的 `storage medium` 属性值，而非系统默认值。#45955
- 禁止在修改 Key 列的 Schema Change 执行期间进行列更新。#46347
- 对于包含自增列的 Key 列表，支持在列更新时不提供自增列。#44528
- FE ID 生成器策略切换为与物理时间相关的策略，ID 不再从 10000 开始。#44790
- 在存算分离模式下，Compaction 产生的 stale rowset 默认回收延迟时间减小至 1800 秒，以减少回收间隔。某些极端场景下可能会导致超大查询失败，如遇问题可按需调整。#45460
- 在存算分离模式下禁用 `show cache hotspot` 语句，需直接访问系统表。#47332
- 禁止删除系统创建的 `admin` 用户。#44751

8.3.5.2 改进优化

8.3.5.2.1 存储

- 优化 Routine Load 因 `max_match_interval` 设置过小导致任务频繁超时的问题。#46292
- 提升 Broker Load 在导入多个压缩文件时的性能。#43975
- 增大 `webserver_num_workers` 的默认值以提升 Stream Load 性能。#46593
- 优化 Routine Load 导入任务在 BE 节点扩容时负载不均衡的问题。#44798
- 优化 Routine Load 线程池使用，防止 Routine Load 超时失败影响查询。#45039

8.3.5.2.2 存算分离

- 加强 Meta-service 的稳定性和可观测性。#44036, #45617, #45255, #45068
- 优化 File Cache，增加提前淘汰策略，减小持锁时间，提升查询性能。#47473, #45678, #47472
- 优化 File Cache 初始化检查以及队列转换，提升稳定性。#44004, #44429, #45057, #47229
- 优化 HDFS 数据回收速度。#46393
- 优化超高频导入时 FE 获取计算组可能存在的性能问题。#47203
- 优化存算分离主键表的若干导入相关参数，提升实时高并发导入的稳定性。#47295, #46750, #46365

8.3.5.2.3 Lakehouse

- 支持读取 Hive Json 格式的表数据。#43469
- 更多内容，参考文档：Text/CSV/JSON - Apache Doris
- 支持会话变量 `enable_text_validate_utf8`，可忽略 CSV 格式中的 UTF-8 编码检测。#45537
- 更多内容，参考文档：Text/CSV/JSON - Apache Doris
- 将 Hudi 版本更新至 0.15，并优化 Hudi 表的查询规划性能。
- 优化 MaxCompute 分区表的读取性能。#45148

- 优化在高过滤率情况下，Parquet 文件延迟物化的性能。#46183
- 支持 Parquet 复杂类型的延迟物化。#44098
- 优化 ORC 类型的谓词下推逻辑，支持更多谓词条件用于索引过滤。#43255

8.3.5.2.4 异步物化视图

- 支持更多场景下的聚合上卷改写。#44412

8.3.5.2.5 查询优化器

- 优化分区裁剪性能。#46261
- 增加利用数据特征消除 `group by key` 的规则。#43391
- 根据目标表的数据量自适应调整 Runtime Filter 的等待时间。#42640
- 优化聚合下压连接的能力，以适应更多场景。#43856, #43380
- 优化 Limit 下压聚合，以适应更多场景。#44042

8.3.5.2.6 其他

- 优化 FE、 BE、 MS 进程启动脚本，使输出内容更明确。#45610, #45490, #45883
- `show tables` 显示的表名大小写现在与 MySQL 行为一致。#46030
- `show index` 支持任意目标表类型。#45861
- `information_schema.columns` 支持显示默认值。#44849
- `information_schema.views` 支持显示视图定义。#45857
- 支持 MySQL 协议的 `COM_RESET_CONNECTION` 命令。#44747

8.3.5.3 缺陷修复

8.3.5.3.1 存储

- 修复聚合表模型导入过程中可能出现的内存错误。#46997
- 修复存算分离模式下 FE 主节点重启时导致 Routine Load offset 丢失的问题。#46566
- 修复存算模式下 FE Observer 节点在批量导入场景中的内存泄漏问题。#47244
- 修复 Full Compaction 进行 Order Data Compaction 导致 Cumulative Point 回退的问题。#44359
- 修复 Delete 操作可能导致 Tablet Compaction 短暂无法调度的问题。#43466
- 修复多计算集群时，Schema Change 后 Tablet 状态不正确的问题。#45821
- 修复在有 `sequence_type` 的主键表上进行 Column Rename Schema Change 时可能报 NPE 错误的问题。#46906
- 数据正确性：修复主键表在部分列更新导入包含 DELETE SIGN 列时的正确性问题。#46194
- 修复主键表 Publish 任务持续卡住时，FE 可能存在内存泄漏的问题。#44846

8.3.5.3.2 存算分离

- 修复 File Cache 可能导致缓存大小大于表数据大小的问题。#46561, #46390
- 修复数据上传至 5MB 边界值时可能导致上传失败的问题。#47333
- 修复 Storage Vault 若干 `alter` 相关操作，增加更多参数检查，提升鲁棒性。#45155, #45156, #46625, #47078, #45685, #46779
- 修复因 Storage Vault 配置不当导致数据无法回收或回收缓慢的问题。#46798, #47536, #47475, #47324, #45072
- 修复回收过程中可能卡住导致数据无法及时回收的问题。#45760
- 修复存算分离下 MTTM-230 错误时未正确重试的问题。#47370, #47326
- 修复存算分离模式下 Decommission BE 时，Group Commit WAL 未回放完成的问题。#47187
- 修复超过 2GB 的 Tablet Meta 导致 MS 不可用的问题。#44780
- 数据正确性：修复存算分离主键表的两个重复 Key 问题。#46039, #44975
- 修复存算分离主键表在高频实时导入下，可能因 Delete Bitmap 过大导致 Base Compaction 持续失败的问题。#46969
- 修改 Schema Change 在存算分离主键表上的一些错误重试逻辑，提高 Schema Change 的健壮性。#46748

8.3.5.3.3 Lakehouse

Hive

- 修复无法查询 Spark 创建的 Hive 视图的问题。#43553
- 修复无法正确读取某些 Hive Transaction 表的问题。#45753
- 修复 Hive 表分区存在特殊字符时，无法进行正确分区裁剪的问题。#42906

Iceberg

- 修复在 Kerberos 认证环境下，无法创建 Iceberg 表的问题。#43445
- 修复某些情况下，Iceberg 表存在 dangling delete 情况下，`count*` 查询不准确的问题。#44039
- 修复某些情况下，Iceberg 表列名不匹配导致查询错误的问题。#44470
- 修复某些情况下，Iceberg 表分区被修改后无法读取的问题。#45367

Paimon

- 修复 Paimon Catalog 无法访问阿里云 OSS-HDFS 的问题。#42585

Hudi

- 修复某些情况下，Hudi 表分区裁剪失效的问题。#44669

JDBC

- 修复某些情况下，开启表名大小写不敏感功能后，使用 JDBC Catalog 无法获取表的问题。

MaxCompute

- 修复某些情况下，MaxCompute 表分区裁剪失效的问题。#44508

其他

- 修复某些情况下，Export 任务导致 FE 内存泄漏的问题。#44019
- 修复某些情况下，无法使用 HTTPS 协议访问 S3 对象存储的问题。#44242
- 修复某些情况下，Kerberos 认证票据无法自动刷新的问题。#44916
- 修复某些情况下，读取 Hadoop Block 压缩格式文件出错的问题。#45289
- 查询 ORC 格式的数据时，不再下推 CHAR 类型的谓词，以避免可能的结果错误。#45484

8.3.5.3.4 异步物化视图

- 修复极端场景下查询透明改写可能导致规划或结果错误的问题。#44575, #45744
- 修复极端场景下异步物化视图调度可能多产生构建任务的问题。#46020, #46280

8.3.5.3.5 查询优化器

- 修复部分表达式改写可能产生错误表达式的问题。#44770, #44920, #45922, #45596
- 修复偶现的 SQL Cache 结果错误问题。#44782, #44631, #46443, #47266
- 修复部分场景下 Limit 下压聚合算子可能导致错误结果的问题。#45369
- 修复部分场景下延迟物化优化产生错误执行计划的问题。#45693, #46551

8.3.5.3.6 查询执行

- 修复正则表达式和 `like` 函数在特殊字符时结果不正确的问题。#44547
- 修复 SQL Cache 在切换 DB 时结果可能不正确的问题。#44782
- 修复一系列 Arrow Flight 相关问题。#45023, #43929
- 修复当 HashJoin 的 Hash 表超过 4G 时，部分情况下结果错误的问题。#46461
- 修复 `convert_to` 函数在中文字符时溢出的问题。#46405
- 修复 `group by` 带 Limit 时，在极端情况下结果可能出错的问题。#47844
- 修复访问某些系统表结果可能不正确的问题。#47498
- 修复 `percentile` 函数可能导致系统崩溃的问题。#47068
- 修复单表查询带 Limit 时性能退化的问题。#46090
- 修复 `StDistanceSphere` 和 `StAngleSphere` 函数导致系统崩溃的问题。#45508
- 修复 `map_agg` 结果错误的问题。#40454

8.3.5.3.7 半结构化数据管理

BloomFilter Index

- 修复 BloomFilter Index 参数过大导致的异常。#45780
- 修复 BloomFilter Index 写入时内存占用过高的问题。#45833
- 修复删除列时 BloomFilter Index 没有正确删除的问题。#44361, #43378

Inverted Index

- 修复倒排索引构建过程中偶发崩溃的问题。#43246
- 修复倒排索引合并时，出现次数为 0 的词占用空间的问题。#43113
- 避免 Index Size 统计出现超大异常值。#46549
- 修复 VARIANT 类型字段的倒排索引异常。#43375
- 优化倒排索引的本地缓存局部性，提高缓存命中率。#46518
- 在查询 Profile 中增加倒排索引读远程存储的指标 `NumInvertedIndexRemoteIOTotal`。#45675, #44863

其他

- 修复 `ipv6_cidr_to_range` 函数在特殊 NULL 数据时崩溃的问题。#44700

8.3.5.3.8 权限

- 赋予 `CREATE_PRIV` 时，不再检查对应资源是否存在。#45125
- 修复在极端场景下，可能出现的查询有权限的视图，但报错没有视图中引用的表的权限的问题。#44621
- 修复 `use db` 时检查权限时不区分内外 Catalog 的问题。#45720

8.3.6 Release 3.0.3

亲爱的社区小伙伴们，Apache Doris 3.0.3 版本已于 2024 年 12 月 02 日正式发布。 该版本进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- GitHub 下载：https://github.com/apache/doris/releases
- 官网下载：https://doris.apache.org/download

8.3.6.1 行为变更

- 禁止在具有同步物化视图的 MOW 表上进行列更新。#40190
- 调整 RoutineLoad 的默认参数以提升导入效率。#42968
- 当 StreamLoad 失败时，LoadedRows 的返回值调整为 0。#41946 #42291
- 将 Segment cache 的默认内存限制调整为 5%。#42308 #42436

8.3.6.2 新特性

- 引入 `enable_cooldown_replica_affinity` 会话变量，用以控制冷热分层副本的亲和性。#42677

8.3.6.2.1 Lakehouse

- 新增 `table$partition` 语法，用于查询 Hive 表的分区信息。#40774
- 查看文档
- 支持创建 Text 格式的 Hive 表。#41860 #42175
- 查看文档

8.3.6.2.2 异步物化视图

- 引入新的物化视图属性 use_for_rewrite。 当 use_for_rewrite 设置为 false 时，物化视图不参与透明改写。#40332

8.3.6.2.3 查询优化器

- 支持关联非聚合子查询。#42236

8.3.6.2.4 查询执行

- 增加了 ngram_search、normal_cdf、to_iso8601、from_iso8601_date、SESSION_USER()、last_query_id 函数。#38226 #40695 #41075 #41600 #39575 #40739
- aes_encrypt 和 aes_decrypt 函数支持 GCM 模式。#40004
- Profile 中输出变更的会话变量值。#41016 #41318

8.3.6.2.5 半结构化数据管理

- 新增数组函数 array_match_all 和 array_match_any。#40605 #43514
- 数组函数 array_agg 支持在 ARRAY 中嵌套 ARRAY/MAP/STRUCT。#42009
- 新增近似聚合统计函数 approx_top_k 和 approx_top_sum。#44082

8.3.6.3 改进与优化

8.3.6.3.1 存储

- 支持将 bitmap_empty 作为默认值。#40364
- 引入 insert_timeout 会话变量，用以控制 DELETE 语句的超时时间。#41063
- 改进部分错误提示信息。#41048 #39631
- 改进副本修复的优先级调度。#41076
- 提高了建表时对时区处理的鲁棒性。#41926 #42389
- 在创建表时检查分区表达式的合法性。#40158
- 在 DELETE 操作时支持 Unicode 编码的列名。#39381

8.3.6.3.2 存算分离

- 存算分离模式支持 ARM 架构部署。#42467 #43377
- 优化文件缓存的淘汰策略和锁竞争，提高命中率及高并发点查性能。#42451 #43201 #41818 #43401
- S3 storage vault 支持 use_path_style，解决对象存储使用自定义域名的问题。#43060 #43343 #43330
- 优化存算分离配置及部署，预防不同模式下的误操作。#43381 #43522 #43434 #40764 #43891
- 优化可观测性，并提供删除指定 segment file cache 的接口。#38489 #42896 #41037 #43412
- 优化 Meta-service 运维接口：RPC 限速及修复 tablet 元数据修正。#42413 #43884 #41782 #43460

8.3.6.3.3 Lakehouse

- Paimon Catalog 支持阿里云 DLF 和 OSS-HDFS 存储。#41247 #42585
- 查看文档
- 支持读取 OpenCSV 格式的 Hive 表。#42257 #42942
- 优化了访问 External Catalog 中 `information_schema.columns` 表的性能。#41659 #41962
- 使用新的 MaxCompute 开放存储 API 访问 MaxCompute 数据源。#41614
- 优化了 Paimon 表 JNI 部分的调度策略，使得扫描任务更加均衡。#43310
- 优化了 ORC 小文件的读取性能。#42004 #43467
- 支持读取 brotli 压缩格式的 parquet 文件。#42177
- 在 `information_schema` 库下新增 `file_cache_statistics` 表，用于查看元数据缓存统计信息。#42160

8.3.6.3.4 查询优化器

- 优化：当查询仅注释不同时，可以复用同一个 SQL Cache。#40049
- 优化：提升了在数据频繁更新时统计信息的稳定性。#43865 #39788 #43009 #40457 #42409 #41894
- 优化：提升常量折叠的稳定性。#42910 #41164 #39723 #41394 #42256 #40441
- 优化：列裁剪可以生成更优的执行计划。#41719 #41548

8.3.6.3.5 查询执行

- 优化了 sort 算子的内存使用。#39306
- 优化了 ARM 下运算的性能。#38888 #38759
- 优化了一系列函数的计算性能。#40366 #40821 #40670 #41206 #40162
- 使用 SSE 指令优化 `match_ipv6_subnet` 函数的性能。#38755
- 在 insert overwrite 时支持自动创建新的分区。#38628 #42645
- 在 Profile 中增加了每个 PipelineTask 的状态。#42981
- IP 类型支持 runtime filter。#39985

8.3.6.3.6 半结构化数据管理

- 审计日志中输出 prepared statement 的真实 SQL。#43321
- filebeat doris output plugin 支持容错、进度报告等。#36355
- 倒排索引查询性能优化。#41547 #41585 #41567 #41577 #42060 #42372
- 数组函数 `array overlaps` 支持使用倒排索引加速。#41571
- IP 函数 `is_ip_address_in_range` 支持使用倒排索引加速。#41571
- 优化 VARIANT 数据类型的 CAST 性能。#41775 #42438 #43320
- 优化 Variant 数据类型的 CPU 资源消耗。#42856 #43062 #43634
- 优化 Variant 数据类型的元数据和执行内存资源消耗。#42448 #43326 #41482 #43093 #43567 #43620

8.3.6.3.7 权限

- LDAP 新增配置项 `ldap_group_filter` 用于自定义过滤 group。#43292

8.3.6.3.8 其他

- FE 监控项中的连接数信息支持按用户分别显示。#39200

8.3.6.4 问题修复

8.3.6.4.1 存储

- 修复 IPv6 hostname 使用问题。#40074
- 修复 broker/s3 load 进度展示不准确问题。#43535
- 修复查询从 FE 可能卡住的问题。#41303 #42382
- 修复异常情况下自增 id 重复的问题。#43774 #43983
- 修复 groupcommit 偶发 NPE 问题。#43635
- 修复 auto bucket 计算不准确的问题。#41675 #41835
- 修复 FE 重启时流控多表不能正确规划的问题。#41677 #42290

8.3.6.4.2 存算分离

- 修复 MOW 主键表 delete bitmap 过大可能导致 coredump 的问题。#43088 #43457 #43479 #43407 #43297 #43613 #43615 #43854 #43968 #44074 #41793 #42142
- 修复 segment 文件为 5MB 整数倍时上传对象失败的问题。#43254
- 修复 aws sdk 默认重试策略不生效的问题。#43575 #43648
- 修复 alter storage vault 时指定错误 type 也能继续执行的问题。#43489 #43352 #43495
- 修复大事务延迟提交过程中 tablet_id 可能为 0 的问题。#42043 #42905
- 修复常量折叠 RCP 以及 FE 转发 SQL 可能不在预期的计算组执行的问题。#43110 #41819 #41846
- 修复 meta-service 接收到 RPC 时不严格检查 instance_id 的问题。#43253 #43832
- 修复 FE follower information_schema version 没有及时更新的问题。#43496
- 修复 file cache rename 原子性以及指标不准确的问题。#42869 #43504 #43220

8.3.6.4.3 Lakehouse

- 禁止带有隐式转换的谓词条件下推给 JDBC 数据源，避免不一致的查询结果。#42102
- 修复 Hive 高版本事务表的一些读取问题。#42226
- 修复 Export 命令可能导致死锁的问题。#43083 #43402
- 修复无法查询 Spark 创建的 Hive 视图的问题。#43552
- 修复 Hive 分区路径中包含特殊字符导致分区裁剪有误的问题。#42906
- 修复 Iceberg Catalog 无法使用 AWS Glue 的问题。#41084

8.3.6.4.4 异步物化视图

- 修复基表重建后，异步物化视图可能无法刷新的问题。#41762

8.3.6.4.5 查询优化器

- 修复使用多列 range 分区时，分区裁剪结果可能有误的问题。#43332
- 修复部分 limit offset 场景下计算结果错误的问题。#42576

8.3.6.4.6 查询执行

- 修复 hash join 时 array 类型的大小超过 4G 导致 BE Core 的问题。#43861
- 修复 is null 谓词运算部分场景下结果不正确的问题。#43619
- 修复 bitmap 类型在 hash join 时输出结果不正确的问题。#43718
- 修复一些函数结果计算错误的问题。#40710 #39358 #40929 #40869 #40285 #39891 #40530 #41948 #43588
- 修复一些 JSON 类型解析的问题。#39937
- 修复 varchar 和 char 类型在 runtime filter 运算时的问题。#43758 #43919
- 修复一些 decimal256 在标量函数和聚合函数里使用的问题。#42136 #42356
- 修复 arrow flight 在连接时报 `Reach limit of connections` 错误的问题。#39127
- 修复 k8s 环境下，BE 可用内存统计不正确的问题。#41123

8.3.6.4.7 半结构化数据管理

- 调整 `segment_cache_fd_percentage` 和 `inverted_index_fd_number_limit_percent` 的默认值。[#42224](https://github.com/apach
- logstash 支持 group_commit。#40450
- 修复 build index 时 coredump 的问题。#43246 #43298
- 修复 variant index 的问题。#43375 #43773
- 修复后台 compaction 异常情况下可能出现的 fd 和内存泄漏。#42374
- 倒排索引 match null 正确返回 null 而不是 false。#41786
- 修复 ngram bloomfilter 索引 bf_size 设置为 65536 时 coredump 的问题。#43645
- 修复复杂数据类型 JOIN 可能出 coredump 的问题。#40398
- 修复 TVF JSON 数据 coredump 的问题。#43187
- 修复 bloom filter 计算日期和时间的精度问题。#43612
- 修复 IPv6 类型行存 coredump 的问题。#43251
- 修复关闭 light_schema_change 时使用 VARIANT 类型 coredump 的问题。#40908
- 提升高并发点查的 cache 性能。#44077
- 修复删除列时 bloom filter 索引没有同步更新的问题。#43378
- 修复 es catalog 在数组和标量混合数据等特殊情况下的不稳定问题。#40314 #40385 #43399 #40614
- 修复异常正则匹配导致的 coredump 问题。#43394

8.3.6.4.8 权限

- 修复若干权限授权之后无法正常限制的问题。#43193 #41723 #42107 #43306
- 加强若干权限校验。#40688 #40533 #41791 #42106

8.3.6.4.9 其他

- 补充了审计日志表和文件中缺失的审计日志字段。#43303
- 查看文档

8.3.7 Release 3.0.2

亲爱的社区小伙伴们，Apache Doris 3.0.2 版本已于 2024 年 10 月 15 日正式发布。 3.0.2 版本在存算分离、存储、湖仓一体、查询优化器以及执行引擎持续升级改进，欢迎大家下载使用。

- GitHub 下载：https://github.com/apache/doris/releases
- 官网下载：https://doris.apache.org/download

8.3.7.1 行为变更

8.3.7.1.1 存储

- 限制单个备份任务的 Tablet 数量，避免 FE 内存溢出。#40518
- SHOW PARTITIONS命令现在显示分区的CommittedVersion。#28274

8.3.7.1.2 其他

- fe.log的默认打印模式（异步）现在包含文件行号信息。如果遇到因行号输出导致的性能问题，请选择 BRIEF 模式。#39419
- 默认将 Session 变量ENABLE_PREPARED_STMT_AUDIT_LOG 的值从 true 更改为 false，不再打印 Prepare 语句的审计日志。#38865
- 将 Session 变量 max_allowed_packet 的默认值从 1MB 调整为 16MB，与 MySQL 8.4 保持一致。#38697
- FE 和 BE 的 JVM 默认使用 UTF-8 字符集。#39521

8.3.7.2 新特性

8.3.7.2.1 存储

- 备份和恢复现在支持清除不在备份中的表或分区。#39028

8.3.7.2.2 存算分离

- 支持并行回收多个 Tablet 上的过期数据。#37630
- 支持通过ALTER语句变更 Storage Vault。#38685 #37606
- 支持单个事务同时导入大量 Tablet（5000+）（实验性功能）。#38243
- 支持自动中止因节点重启等原因导致的未决事务，解决未决事务阻塞 Secommission 或 Schema Change 的问题。#37669

- 新增 Session 变量 enable_segment_cache 控制查询时是否使用 Segment Cache（默认为true）。#37141
- 解决存算分离模式下进行 Schema Change 时不能大量导入的问题。#39558
- 支持在存算分离模式下允许添加多个 Follower 角色的 FE。#38388
- 支持在无盘或低性能 HDD 环境下使用内存作为 File Cache 以加速查询。#38811

8.3.7.2.3 Lakehouse

- 新增 Lakesoul Catalog
- 新增系统表 catalog_meta_cache_statistics，用于查看 External Catalog 中各类元数据缓存的使用情况。#40155

8.3.7.2.4 查询优化器

- 支持 is [not] true/false表达式。#38623

8.3.7.2.5 查询执行

- 新增 CRC32 函数。#38204
- 新增聚合函数 Skew 和 Kurt。#41277
- 将 Profile 持久化到 FE 的磁盘中，以保留更多的 Profile。#33690
- 新增系统表workload_group_privileges以查看 Workload Group 相关的权限信息。#38436
- 新增系统表workload_group_resource_usage以监控 Workload Group 的资源统计信息。#39177
- Workload Group 现在支持限制本地 IO 和远程 IO 的读取。#39012
- Workload Group 现在支持 cgroupv2 以限制 CPU 使用。#39374
- 新增系统表information_schema.partitions以查看一些建表属性。#40636

8.3.7.2.6 其他

- 支持使用SHOW语句展示 BE 的配置信息，例如SHOW BACKEND CONFIG LIKE ${pattern}。#36525

8.3.7.3 改进与优化

8.3.7.3.1 导入

- 优化了 Routine Load 在遇到 Kafka 频繁 EOF 时的导入效率。#39975
- Stream Load 结果中增加了读取 HTTP 数据的耗时时间ReceiveDataTimeMs，可以快速判断网络原因导致的 Stream Load 慢问题。#40735
- 优化了 Routine Load 超时逻辑，避免了倒排和 MOW 写入频繁超时问题。#40818

8.3.7.3.2 存储

- 支持批量添加分区。#37114

8.3.7.3.3 存算分离

- 添加了 meta-service HTTP 接口`/MetaService/http/show_meta_ranges`，便于统计 FDB 中 KV 分布组成。#39208
- meta-service/recycler stop 脚本确保进程完全退出后才返回。#40218
- 支持使用 Session 变量`version_comment`（Cloud Mode）来显示当前部署模式为存算分离模式。#38269
- 修复了提交事务失败时返回的详细消息。#40584
- 支持使用一个 meta-service 进程同时提供元数据服务和数据回收服务。#40223
- 优化了 file_cache 的默认配置，避免了未设置时可能导致的无法正确运行的问题。#41421 #41507
- 通过批量获取多个 Partition 的 Version 提高了查询性能。#38949
- 延迟变更 Tablet 的分布，避免了临时网络抖动引起的查询性能问题。#40371
- 优化了 Balance 逻辑中的读写锁。#40633
- 提高了 File Cache 在重启/宕机等情况下处理 TTL 文件名的鲁棒性。#40226
- 增加了 BE HTTP 接口`/api/file_cache?op=hash`，方便计算 Segment 文件在盘上的 Hash 文件名。#40831
- 优化了统一命名，兼容使用 Compute Group 代表 BE 分组（原 Cloud Cluster）。#40767
- 优化了主键表计算 Delete Bitmap 时获取锁的等待时间。#40341
- 当主键表 Delete Bitmap 数量多时，通过提前合并多个 Delete Bitmap 来优化查询时 CPU 消耗高的问题。#40204
- 支持通过 SQL 语句管理存算分离模式下的 FE/BE 节点，隐藏部署存算分离模式时直接和 meta-service 交互的逻辑。#40264
- 增加了快速部署 FDB 脚本。#39803
- 优化了`SHOW CACHE HOTSPOT`的输出，使其和其他`SHOW`语句的列名风格统一。#41322
- 使用 Storage Vault 作为存储后端时，不允许使用`latest_fs()`以规避同个表绑定不同的存储后端。#40516
- 优化了 MOW 表导入时计算 Delete Bitmap 的超时策略。#40562 #40333
- 存算分离模式下 be.conf 的 `enable_file_cache` 默认开启。#41502

8.3.7.3.4 Lakehouse

- 读取 CSV 格式的表时，支持通过会话`keep_carriage_return`设置对`\r`符号的读取行为。#39980
- BE 的 JVM 最大内存默认调整为 2GB（仅影响新部署用户）。#41403
- Hive Catalog 新增`hive.recursive_directories_table`和`hive.ignore_absent_partitions`属性，用于指定是否递归遍历数据目录，以及是否忽略缺失的分区。#39494
- 优化了 Catalog 刷新逻辑，避免了刷新产生大量连接。#39205
- `SHOW CREATE DATABASE`和`SHOW CREATE TABLE`针对外部数据源，增加了 Location 信息显示。#39179
- 新优化器支持通过`INSERT INTO`命令将数据插入到 JDBC 外表。#41511
- MaxCompute Catalog 支持复杂类型。#39259
- 优化了外表数据分片的读取合并逻辑。#38311
- 优化了外表元数据缓存的一些刷新策略。#38506
- Paimon 表支持`IN/NOT IN`谓词下推。#38390
- 兼容 Paimon 0.9 版本创建的 Parquet 格式的表。#41020

8.3.7.3.5 异步物化视图

- 构建异步物化视图支持同时使用 Immediate 和 Starttime。#39573
- 基于外表的异步物化视图，在刷新物化视图前会刷新外表的元数据缓存，保证基于最新外表数据构建。#38212

- 分区增量构建支持按照周和季度粒度上卷。#39286

8.3.7.3.6 查询优化器

- 聚合函数GROUP_CONCAT现在支持同时使用DISTINCT和ORDER BY。#38080
- 优化了统计信息的收集、使用，以及估算行数和代价计算的逻辑，现在可以生成更高效稳定的执行计划。
- 窗口函数分区数据预过滤支持包含多个窗口函数的情况。#38393

8.3.7.3.7 查询执行

- 通过并行运行 Prepare Pipeline Task 来降低查询延时。#40874
- 在 Profile 中显示 Catalog 信息。#38283
- 优化了IN过滤条件的计算性能。#40917
- 在 K8S 中支持 cgroupv2 来限制 Doris 的内存使用。#39256
- 优化了字符串到 DATETIME 类型的转换性能。#38385
- 当字符串是一个小数时，支持将其 CAST 为 INT，这将更兼容 MySQL 的某些行为。#38847

8.3.7.3.8 半结构化数据管理

- 优化了倒排索引匹配的性能。#41122
- 暂时禁止在数组上创建带分词的倒排索引。#39062
- explode_json_array支持二进制 JSON 类型。#37278
- IP 数据类型支持 BloomFilter 索引。#39253
- IP 数据类型支持行存。#39258
- ARRAY、 MAP、 STRUCT 嵌套数据类型支持 Schema Change。#39210
- 创建 MTMV 时遇到 VARIANT 数据类型自动截断 KEY。#39988
- 查询时懒加载倒排索引提升性能。#38979
- add inverted index file size for open file。#37482
- Compaction 时减少倒排索引访问对象存储接口提升性能。#41079
- 增加了 3 个倒排索引相关的 Query Profile Metric。#36696
- 减少非 PreparedStatement SQL 的 Cache 开销提升性能。#40910
- 预热缓存支持倒排索引。#38986
- 倒排索引写入即缓存。#39076

8.3.7.3.9 兼容性

- 修复了 Thrift ID 在 Master 上与 Branch-2.1 不兼容的问题。#41057

8.3.7.3.10 其他

- BE HTTP API 支持鉴权，需要鉴权时将 config::enable_all_http_auth 设置为 true（默认为 false）。#39577
- 优化了 REFRESH 操作所需的用户权限。从 ALTER 权限放宽到 SHOW 权限。#39008
- 减少了调用 advanceNextId() 时 nextId 的范围。#40160
- 优化了 Java UDF 的缓存机制。#40404

8.3.7.4 缺陷修复

8.3.7.4.1 导入

- 修复了abortTransaction没有处理返回码的问题。#41275
- 修复了存算分离模式下提交/中止事务失败时未调用afterCommit/afterAbort的问题。#41267
- 修复了存算分离模式下 Routine Load 修改消费偏移量无法工作的问题。#39159
- 修复了获取错误日志文件路径时重复关闭文件的问题。#41320
- 修复了存算分离模式下 Routine Load 作业进度缓存不正确的问题。#39313
- 修复了存算分离模式下 Routine Load 提交事务失败导致卡住的问题。#40539
- 修复了存算分离模式下 Routine Load 一直报数据质量检查错误的问题。#39790
- 修复了存算分离模式下 Routine Load 未在提交前事务进行检查的问题。#39775
- 修复了存算分离模式下 Routine Load 未在中止事务前进行检查的问题。#40463
- 修复了 Cluster Key 不支持某些数据类型的问题。#38966
- 修复了事务被重复提交的问题。#39786
- 修复了 WAL 在 BE 退出时 Use After Free 的问题。#33131
- 修复了存算分离模式下 WAL 回放未跳过已经完成了的导入事务的问题。#41262
- 修复了存算分离模式下 Group Commit 选择 BE 的逻辑。#39986 #38644
- 修复了 Insert Into 开启 Group Commit 时 BE 可能 Coredump 的问题。#39339
- 修复了 Insert Into 开启 Group Commit 时可能会卡住的问题。#39391
- 修复了导入不打开 Group Commit 选项时可能会报找不到表的问题。#39731
- 修复了 Tablet 数量太多提交事务超时的问题。#40031
- 修复了 Auto Partition 并发 open 的问题。#38605
- 修复了导入锁粒度太大的问题。#40134
- 修复了 Varchar 长度为 0 导致 Coredump 的问题。#40940
- 修复了日志打印的 index ID 值不正确的问题。#38790
- 修复了 Memtable 前移未 Close BRPC Streaming 的问题。#40105
- 修复了 Memtable 前移 bvar 统计不准确的问题。#39075
- 修复了 Memtable 前移多副本容错的问题。#38003
- 修复了 Routine Load 一流多表错误计算消息长度的问题。#40367
- 修复了 Broker Load 进度汇报不准确的问题。#40325
- 修复了 Broker Load 扫描数据量汇报不准确的问题。#40694
- 修复了存算分离模式下 Routine Load 并发的问题。#39242
- 修复了存算分离模式下 Routine Load Job 被取消的问题。#39514
- 修复了删除 Kafka Topic 时进度未被重置的问题。#38474
- 修复了 Routine Load 事务状态转换时更新进度的问题。#39311
- 修复了 Routine Load 从暂停状态切换到暂停状态的问题。#40728
- 修复了 Stream Load 记录因数据库被删除被漏记录的问题。#39360

8.3.7.4.2 存储

- 修复了 Storage Policy 丢失的问题。#38700
- 修复了跨版本备份恢复报错的问题。#38370
- 修复了 CCR Binlog NPE 问题。#39909

- 修复了可能的 MOW 重复 Key 问题。#41309 #39791 #39958 #38369 #38331
- 修复了高频写入场景下备份恢复之后不能写入的问题。#40118 #38321
- 修复了删除空字符串和 Schema Change 交叉可能触发的数据错误问题。#41064
- 修复了列更新导致的数据统计不正确问题。#40880
- 限制了 Tablet Meta PB 的大小，防止大小过大导致 BE 宕机。#39455
- 修复了`begin; insert into values; commit`新优化器可能的列错位问题。#39295

8.3.7.4.3 存算分离

- 修复了存算分离模式下多个 FE 的 Tablet 分布可能不一致的问题。#41458
- 修复了 TVF 在多计算组环境下可能不工作的问题。#39249
- 修复了存算分离模式 BE 退出时 Compaction 使用了已经释放的资源问题。#39302
- 修复了自动启停可能导致 FE replay 卡住的问题。#40027
- 修复了 BE 状态和 Meta-Service 中存储的状态不一致的问题。#40799
- 修复了 FE->Meta-Service 连接池不能自动过期重连的问题。#41202 #40661
- 修复了 Rebalance 过程中有一些 Tablet 可能会来回进行非预期的 Balance 问题。#39792
- 修复了 FE 重启后 Storage Vault 权限丢失的问题。#40260
- 修复了 Tablet 行数等统计信息可能因为 FDB Scan Range 分页导致统计不全的问题。#40494
- 修复了同个 Label 下关联大量的 Abort 事务导致的性能问题。#40606
- 修复了 commit_txn 没有自动重入的问题，保持存算一体和存算分离行为一致。#39615
- 修复了 Drop Column 时投影列变多的问题。#40187
- 修复了 Delete 语句返回值没有正确处理导致删除之后数据仍可见的问题。#39428
- 修复了文件缓存预热时因为 Rowset 元数据竞争导致的 coredump 问题。#39361
- 修复了 TTL 缓存开启 LRU 淘汰时会用满整个缓存空间的问题。#39814
- 修复了基于 HDFS 存储后端导入 Commit Rowset 失败时临时文件不能回收的问题。#40215

8.3.7.4.4 Lakehouse

- 修复了一些 JDBC Catalog 谓词下推的问题。#39064
- 修复了当 Parquet 格式中 Struct 类型列缺失时无法读取的问题。#38718
- 修复了部分情况下 FE 侧 FileSystem 泄露的问题。#38610
- 修复了部分情况下 Hive/Iceberg 表写回导致元数据缓存信息不一致的问题。#40729
- 修复了部分情况下为外表生成分区 ID 不稳定的问题。#39325
- 修复了部分情况下外表查询会选择在黑名单中的 BE 节点的问题。#39451
- 优化了分批获取外表分区信息时的超时时间，避免了长时间占用线程。#39346
- 修复了部分情况下查询 Hudi 表导致内存泄露的问题。#41256
- 修复了部分情况下 JDBC Catalog 可能存在连接池连接泄露的问题。#39582
- 修复了部分情况下 JDBC Catalog 可能存在 BE 内存泄露的问题。#41041
- 修复了无法查询阿里云 OSS 上 Hudi 数据的问题。#41316
- 修复了无法读取 MaxCompute 空分区的问题。#40046
- 修复了通过 JDBC Catalog 查询 Oracle 表示性能差的问题。#41513
- 修复了开启文件缓存功能后，查询 Paimon 表 Deletion Vector 时 BE 宕机的问题。#39877
- 修复了无法访问开启 HA 的 HDFS 集群上 Paimon 表的问题。#39806
- 临时关闭了 Parquet 的 Page Index 过滤功能以避免一些潜在问题。#38691

- 修复了无法读取 Parquet 文件中 Unsigned 类型的问题。#39926
- 修复了部分情况下读取 Parquet 文件可能导致死循环的问题。#39523

8.3.7.4.5 异步物化视图

- 修复了分区构建时，如果两侧有相同的列名，可能选择错误的表跟踪分区的问题。#40810
- 修复了透明改写分区补偿可能导致结果错误的问题。#40803
- 修复了透明改写在外表不生效的问题。#38909
- 修复了嵌套物化视图可能不能正常刷新的问题。#40433

8.3.7.4.6 同步物化视图

- 修复了在 MOW 表上创建同步物化视图可能导致查询结果错误的问题。#39171

8.3.7.4.7 查询优化器

- 修复了升级后原有同步物化视图可能不可用的问题。#41283
- 修复了 DATETIME 字面量比较时，没有正确处理毫秒的问题。#40121
- 修复了条件函数分区裁剪可能错误的问题。#39298
- 修复了存在同步物化视图的 MOW 表无法执行 Delete 的问题。#39578
- 修复了 JDBC 外表查询谓词中的 Slot 的 Nullable 可能规划不正确，导致查询报错的问题。#41014

8.3.7.4.8 查询执行

- 修复了 Runtime Filter 在使用过程中导致的内存泄露问题。#39155
- 修复了 Window Function 在使用内存特别多的问题。#39581
- 修复了一系列滚动升级期间函数兼容性的问题。#41023 #40438 #39648
- 修复了`encryption_function` 在常量时结果错误的问题。#40201
- 修复了单表物化视图导入时报错的问题。#39061
- 修复了窗口函数分区结果计算错误的问题。#39100 #40761
- 修复了 TOPN 计算在有 Null 值时计算错误的问题。#39497
- 修复了 `map_agg` 函数计算结果错误的问题。#39743
- 修复了 Cancel 返回的消息错误的问题。#38982
- 修复了 Encrypt 和 Decrypt 函数导致 BE Core 的问题。#40726
- 修复了在高并发场景下，过多的 Scanner 导致查询卡住的问题。#40495
- Runtime Filter 中支持 TIME 类型。#38258
- 修复了 Window Funnel 函数结果错误的问题。#40960

8.3.7.4.9 半结构化数据管理

- 修复了没有索引时 Match 函数报错的问题。#38989
- 修复了 ARRAY 数据类型作为 `array_min`/`array_max` 函数参数时 Crash 的问题。#39492
- 修复了 `array_enumerate_uniq` 函数 Nullable 的问题。#38384

- 修复了添加或删除列时 BloomFilter 索引没有更新的问题。#38431
- 修复了 ES-Catalog 解析异常 Array 数据的问题。#39104
- 修复了 ES-Catalog 不合理条件下推的问题。#40111
- 修复了 `map()`/`struct()` 函数修改了输入数据导致异常的问题。#39699
- 修复了特殊情况下索引 Compaction Crash 的问题。#40294
- 修复了 ARRAY 类型倒排索引缺少 Nullbitmap 的问题。#38907
- 修复了倒排索引 `count()` 结果的问题。#41152
- 修复了 `explode_map` 使用别名时结果正确性问题。#39757
- 修复了 VARIANT 类型中异常 JSON 数据无法使用行存的问题。#39394
- 修复了 VARIANT 类型中返回 ARRAY 结果时内存泄漏的问题。#41358
- 修复了 VARIANT 类型修改列名的问题。#40320
- 修复了 VARIANT 类型转成 DECIMAL 类型可能丢失精度的问题。#39650
- 修复了 VARIANT 类型 Nullable 处理问题。#39732
- 修复了 VARIANT 类型稀疏列读取问题。#40295

8.3.7.4.10 其他

- 修复了新旧 Audit Log Plugin 兼容性问题。#41401
- 修复了某些情况下用户能看到他人进程的问题。#39747
- 修复了有权限的用户也不能导出的问题。#38365
- 修复了 CREATE TABLE LIKE 需要已有表的 CREATE 权限的问题。#37879
- 修复了一些功能没有校验权限的问题。#39726
- 修复了使用 SSL 连接时未正确关闭连接的问题。#38587
- 修复了部分情况下执行 ALTER VIEW 操作导致 FE 无法启动的问题。#40872

8.3.8 Release 3.0.1

亲爱的社区小伙伴们，Apache Doris 3.0.1 版本已于 2024 年 8 月 23 日正式发布。从 3.0 系列版本开始，Apache Doris 开始支持存算分离模式，用户可以在集群部署时选择采用存算一体模式或存算分离模式。同时在 3.0.1 版本中，Apache Doris 在存算分离、湖仓一体、半结构化数据分析、异步物化视图等方面进行了全面更新与改进，欢迎大家下载使用。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.3.8.1 行为变更

8.3.8.1.1 查询优化器

- 新增变量 `use_max_length_of_varchar_in_ctas`，用于控制在执行 `CREATE TABLE AS SELECT`（CTAS）操作时 VARCHAR 类型的长度行为。此变量默认设置为 true。当设置为 true 时，如果 VARCHAR 类型的列源自一个表，则采用推导长度；否则，使用最大长度。当设置为 false 时，VARCHAR 类型将始终使用推导出的长度。#37069
- 所有的数据类型将以小写形式展示，以保持与 MySQL 格式的兼容性。#38012

- 同一查询请求中的多条查询语句现在必须使用分号分隔。#38670

8.3.8.1.2 查询执行

- 将集群在执行 Shuffle 操作后默认的并行任务数设置为 100，这将提高大型集群中查询的稳定性和并发处理能力。#38196

8.3.8.1.3 存储

- `trash_file_expire_time_sec` 的默认值已从 86400 秒更改为 0 秒，这意味着如果误删除文件并清空了 FE 回收站，数据将无法恢复。
- 表属性 `enable_mow_delete_on_delete_predicate`（在版本 3.0.0 中引入）已更名为 `enable_mow_light_delete`。
- 显式事务现在被禁止对已写入数据的表执行 Delete 操作。
- 禁止对含有自增字段的表进行重量级的 Schema Change 操作。

8.3.8.2 新特性

8.3.8.2.1 任务调度

- 优化内部调度作业的执行逻辑，取消开始时间和立即执行参数之间的强关联。现在任务在创建时可以指定开始时间或选择立即执行，两者不再冲突，从而提高了调度的灵活性。#36805

8.3.8.2.2 存算分离

- 支持动态更改 File Cache 的使用上限。#37484
- Recycler 现在支持对象存储限速以及服务端限速重试功能。#37663 #37680

8.3.8.2.3 Lakehouse

- 新增会话变量 `serde_dialect`，可以设置复杂类型的输出格式。#37039
- SQL 拦截功能现在支持外部表
- 更多内容，参考文档SQL 拦截
- Insert Overwrite 现在支持 Iceberg 表。#37191

8.3.8.2.4 异步物化视图

- 支持按小时级别分区上卷构建。#37678
- 支持原子替换异步物化视图定义语句。#36749
- 透明改写现在支持 Insert 语句。#38115
- 透明改写现在支持 Variant 类型。#37929

8.3.8.2.5 查询执行

- Group Concat 函数现在支持 DISTINCT 和 ORDER BY 选项。#38744

8.3.8.2.6 半结构化数据管理

- ES Catalog 现在将 Elasticsearch 中的 `nested` 或 `object` 类型映射为 Doris 的 JSON 类型。#37101
- 新增 `MULTI_MATCH` 函数，支持在多个字段中匹配关键词，并能利用倒排索引加速搜索。#37722
- 新增 `explode_json_object` 函数，可以将 JSON 数据中的 Object 展开为多行。#36887
- 倒排索引现在支持 Memtable 前移，在多副本写入时只需构建一次索引，减少 CPU 消耗并提升性能。#35891
- 新增 `MATCH_PHRASE` 支持正向词距（slop），例如 `msg MATCH_PHRASE 'a b 2+'` 可以匹配包含词 a 和 b，它们之间的词距不超过两个，并且 a 在 b 的前面；而普通的词距（slop）如果没有最后的加号 `+`，则不保证 a 在 b 的前面。#36356

8.3.8.2.7 其他

- 新增加了 FE 参数 `skip_audit_user_list`，在此配置项中的用户操作将不会被记录到审计日志中。#38310
- 更多内容，参考文档审计插件

8.3.8.3 改进

8.3.8.3.1 存储

- 降低单个 BE 内磁盘间均衡导致写失败的可能性。#38000
- 降低 Memtable Limiter 的内存消耗。#37511
- 在替换分区操作时，将旧分区移动到 FE 回收站。#36361
- 优化了 Compaction 的内存消耗。#37099
- 增加了会话变量以控制 JDBC PreparedStatement 的审计日志，默认不打印。#38419

- 优化了 Group Commit 选择 BE 的逻辑。#35558
- 优化了列更新的性能。#38487
- 优化了 `delete bitmap cache` 的使用。#38761
- 添加了配置以控制冷热分层时查询的亲和性。#37492

8.3.8.3.2 存算分离

- 遇到对象存储服务端限速时，现在会自动重试。#37199
- 适应存算分离模式下 Memtable Flush 的线程数。#38789
- 将 Azure 作为编译选项，以便支持在不支持 Azure 的环境中编译。
- 优化了对象存储访问限速的可观测性。#38294
- 允许 File Cache TTL 队列进行 LRU 淘汰，增加了 TTL 队列的可用性。#37312
- 优化了存算分离模式下 Balance Writeeditlog IO 次数。#37787
- 优化了存算分离模式下建表的速度，批量发送创建 Tablet 的请求。#36786
- 通过退避重试的方式，优化了本地 File Cache 可能不一致时导致的读取失败问题。#38645

8.3.8.3.3 Lakehouse

- 优化了 Parquet/ORC 格式读写操作的内存统计。#37234
- Trino Connector Catalog 现在支持谓词下推。#37874
- 新增会话变量 `enable_count_push_down_for_external_table`，用于控制是否开启外部表的 `count(*)` 下推优化。#37046
- 优化了 Hudi 快照读的读取逻辑，当快照为空时返回空集，与 Spark 行为保持一致。#37702
- 优化了 Hive 表分区列的读取性能。#37377

8.3.8.3.4 异步物化视图

- 透明改写计划速度提升了 20%。#37197
- 如果 Group Key 满足数据唯一性，在透明改写时不再进行上卷，以更好地进行嵌套匹配。#38387
- 透明改写现在可以更好地进行聚合消除，以提高嵌套物化视图的匹配成功率。#36888

8.3.8.3.5 MySQL 兼容性

- 现在正确填充了 MySQL 协议中结果列的库名、表名和原始名称。#38126
- 支持了形如 `/*+ func(value)*/` 的 Hint 格式。#37720

8.3.8.3.6 查询优化器

- 显著提升了复杂查询的计划速度。#38317
- 根据数据分桶数量，自适应选择是否进行 Bucket Shuffle，以避免极端情况下的性能劣化。#36784
- 优化了 SEMI/ANTI JOIN 的代价估算逻辑。#37951 #37060
- 支持将 Limit 下推到第一阶段聚合，以提升性能。#34853
- 分区裁剪现在支持过滤条件中包含 `date_trunc` 或 `date` 函数。#38025 #38743
- SQL 缓存现在支持包含用户变量的查询场景。#37915
- 优化了聚合语义不合法时的错误信息。#38122

8.3.8.3.7 查询执行

- 适配了 AggState 的 2.1 到 3.x 兼容性，并修复了 Coredump 问题。#37104
- 重构了不带 Join 时 Local Shuffle 的策略选择。#37282
- 将内部表查询的 Scanner 修改为异步方式，以防止查询内部表时卡住。#38403
- 优化了 Join 算子构建 Hash 表时的 Block Merge 过程。#37471
- 优化了 MultiCast 持有锁的时间。#37462
- 优化了 gRPC 的 keepAliveTime 并增加了链接监测机制，降低了查询过程中因 RPC 错误导致查询失败的概率。#37304
- 当内存超限时，清理 Jemalloc 中的所有 Dirty Pages。#37164
- 优化了 `aes_encrypt` /`decrypt` 函数对常量类型的处理性能。#37194
- 优化了 `json_extract` 函数对常量数据的处理性能。#36927
- 优化了 `ParseUrl` 函数对常量数据的处理性能。#36882

8.3.8.3.8 半结构化数据管理

- Bitmap 索引现在默认使用反向索引，`enable_create_bitmap_index_as_inverted_index` 默认设置为 true。#36692
- 在存算分离模式下，DESC 现在可以查看 VARIANT 类型的子列。#38143
- 移除了倒排索引查询时检查文件是否存在的步骤，以降低远程存储的访问延迟。#36945
- ARRAY / MAP / STRUCT 复杂类型现在支持 AGG 表的 `replace_if_not_null`。#38304
- 现在支持 JSON 数据的转义字符。#37176 #37251
- 倒排索引查询现在在 MOW 表上与 Duplicate 表一致。#37428

- 优化了倒排索引加速 IN 查询的性能。#37395
- TOPN 查询时减少了多余的内存分配，以提升性能。#37429
- 当创建带分词的倒排索引时，现在自动开启 `support_phrase` 选项，以加速 `match_phrase` 系列短语查询。#37949

8.3.8.3.9 其他

- Audit Log 现在可以记录 SQL 类型。#37790
- 增加对 `information_schema.processlist` Show All FE 的支持。#38701
- 缓存 Ranger 的 `atamask` 和 `rowpolicy`，以加速查询效率。#37723
- 优化 Job Manager 的元数据管理，在修改元数据后立即释放锁，以减少锁持有时间。#38162

8.3.8.4 缺陷修复

8.3.8.4.1 升级

- 修复从 2.1 版本升级时 mtmv load 失败的问题。#38799
- 修复在 2.1 版本升级时找不到 `null_type` 的问题。#39373
- 修复从 2.1 版本升级到 3.0 版本时权限持久化的兼容性问题。#39288

8.3.8.4.2 导入

- 修复 CSV 格式解析中，换行符被包围符包围时解析失败的问题。#38347
- 修复 FE 在转发 Group Commit 时可能出现的异常问题。#38228 #38265
- Group Commit 现在支持新优化器。#37002
- 修复 JDBC setNull 时 Group Commit 报告数据错误的问题。#38262
- 优化 Group Commit 遇到 `delete bitmap lock` 错误时的重试逻辑。#37600
- 修复 Routine Load 不能使用 CSV 包围符和转义符的问题。#38402
- 修复 Routine Load Job 名字大小写混用时无法显示的问题。#38523
- 优化 FE 主从切换时主动恢复 Routine Load 的逻辑。#37876
- 修复 Kafka 中数据全部过期时 Routine Load 暂停的问题。#37288
- 修复 `show routine load` 返回空结果的问题。#38199
- 修复 Routine Load 多表流式导入时的内存泄露问题。#38255
- 修复 Stream Load 不返回 Error URL 的问题。#38325

- 修复 Load Channel 可能泄露的问题。#38031 #37500
- 修复导入少于预期的 Segment 时可能不报错的问题。#36753
- 修复 Load Stream 泄露的问题。#38912
- 优化下线节点对导入操作的影响。#38198
- 修复 Insert Into 空数据情况下事务不结束的问题。#38991

8.3.8.4.3 存储

01 备份与恢复

- 修复备份恢复后表无法写入的问题。#37089
- 修复备份恢复后视图中数据库名称错误的问题。#37412

02 Compaction（压缩）

- 修复有序数据压缩时 Cumu Compaction 处理 Delete 错误的的问题。#38742
- 修复顺序压缩优化导致的聚合表重复 Key 问题。#38224
- 修复大宽表下压缩操作导致 Coredump 的问题。#37960
- 修复压缩任务并发统计不准确导致的压缩饥饿问题。#37318

03 MOW Unique Key（MOW 唯一键）

- 解决累计压缩删除 Delete Sign 导致的副本间数据不一致问题。#37950
- 在新的优化器下，MOW Delete 表现在使用部分列更新。#38751
- 修复存算分离下 MOW 表可能出现的重复 Key 问题。#39018
- 修复 MOW Unique 和 Duplicate 表不能修改列顺序的问题。#37067
- 修复 Segcompaction 可能导致的数据正确性问题。#37760
- 修复列更新可能出现的内存泄露问题。#37706

04 其他

- 修复 TOPN 查询可能出现的小概率异常。#39119 #39199

修复 FE 重启时自增 ID 可能重复的问题。#37306

- 修复 Delete 操作优先级队列可能的排队问题。#37169
- 优化 Delete 重试逻辑。#37363

- 修复新优化器下建表语句中 `bucket = 0` 的问题。#38971
- 修复 FE 生成 Image 失败时错误地报告成功的问题。#37508
- 修复 FE 下线节点时使用错误 nodename 可能导致的 FE 成员不一致问题。#37987
- 修复 CCR 增加分区可能失败的问题。#37295
- 修复倒排索引文件中 `int32` 溢出的问题。#38891
- 修复 TRUNCATE TABLE 失败可能导致 BE 不能下线的问题。#37334
- 修复因空指针导致的 Publish 无法继续的问题。#37724 #37531
- 修复手动触发磁盘迁移时可能出现的 Coredump 问题。#37712

8.3.8.4.4 存算分离

- 修复 `show create table` 可能会展示两次 `file_cache_ttl_seconds` 属性的问题。#38052
- 修复设置 File Cache TTL 后，Segment Footer TTL 未正确设置的问题。#37485
- 修复 File Cache 因大量转换 Cache 类型可能会导致 Coredump 的问题。#38518
- 修复 File Cache 可能会泄漏 fd 的问题。#38051
- 修复 Schema Change Job 覆盖 Compaction Job 导致 Base Tablet Compaction 不能正常完成的问题。#38210
- 修复 Base Compaction Score 因 Data Race 可能会不准确的问题。#38006
- 修复导入返回的错误信息可能不能正确上传到对象存储的问题。#38359
- 修复存算分离模式和存算一体模式 2PC 导入返回信息不一致的问题。#38076
- 修复 File Cache 预热未正确设置 File Size 导致 Coredump 的问题。#38939
- 修复部分列更新没有正确出列 Delete 的问题。#37151
- 修复存算分离模式权限持久化兼容问题。#38136 #37708
- 修复 Observer 遇到 -230 错误没有进行正确重试的问题。#37625
- 修复 `show load` 带条件时没有正确 analyze 的问题。#37656
- 修复存算分离模式下 `show streamload` 导致 BE Coredump 的问题。#37903
- 修复 `copy into` 在严格模式下未正确校验列名的问题。#37650
- 修复一表多流导入没有权限的问题。#38878
- 修复 getVersionUpdateTimeMs 可能会越界的问题。#38074
- 修复 FE Azure Blob List 没有实现正确的问题。#37986
- 修复 Azure Blob 回收时间计算不准确导致不触发回收的问题。#37535
- 修复存算分离模式下倒排索引文件漏删的问题。#38306

8.3.8.4.5 Lakehouse

- 修复 Oracle Catalog 读取二进制数据的问题。#37078
- 修复多 FE 情况下，获取外表元数据可能导致的死锁问题。#37756
- 修复 JNI Scanner 打开失败导致 BE 节点宕机的问题。#37697
- 修复 Trino Connector Catalog 读取 Date 类型慢的问题。#37266
- 优化 Hive Catalog 的 Kerberos 认证逻辑。#37301
- 修复解析 MinIO 属性时，Region 属性可能解析错误的问题。#37249
- 修复 FE 创建过多的 FileSystem 导致内存泄漏的问题。#36954
- 修复读取 Paimon 时区信息错误的问题。#37716
- 修复 Hive 写回操作可能导致的线程泄漏问题。#36990
- 修复开启 Hive Metastore Event 同步功能导致的空指针问题。#38421
- 修复创建 Catalog 时报错信息不清晰或卡死的情况。#37551
- 修复读取 Hive Text 格式表时与 Hive 行为不一致的问题。#37638
- 修复切换 Catalog 和 Database 逻辑错误的问题。#37828

8.3.8.4.6 MySQL 兼容性

- 修复开启 SSL 后，MySQL 协议中某些 Flag 设置不正确的问题。#38086

8.3.8.4.7 异步物化视图

- 修复基表分区数量非常多时可能导致的构建失败问题。#37589
- 修复构建嵌套物化视图时，即使可以进行分区刷新，也错误地进行了全表刷新的问题。#38698
- 修复分区刷新在分析分区依赖时，不能处理同时存在合法和不合法依赖关系的问题。#38367
- 修复最终返回结果包含 NULL Type 导致异步物化视图可能构建失败的问题。#37019
- 当包含同名的同步物化视图和异步物化视图时，透明改写可能出现规划错误。#37311

8.3.8.4.8 同步物化视图

- 现在改写后的同步物化视图也可以正确地进行分区裁剪。#38527
- 同步物化视图改写时，不再选择数据未就绪的同步物化视图。#38148

8.3.8.4.9 查询优化器

- 修复查询和 Delete 等操作同时进行可能导致的死锁问题。#38660
- 修复分桶裁剪在 Decimal 列分桶上可能错误裁剪的问题。#37889
- 修复当 Mark Join 参与 Join Reorder 时，规划可能出现错误的问题。#39152
- 修复关联子查询关联条件不是简单列时，结果错误的问题。#37644
- 修复分区裁剪不能正确处理 or 表达式的问题。#38897
- 修复当进行 JOIN 和 AGG 交换执行顺序的优化时，可能导致的规划报错问题。#37343
- 修复 `str_to_date` 在 DATEV1 类型上进行常量折叠计算错误的问题。#37360
- 修复 ACOS 函数常量折叠返回非 NaN 的问题。#37932
- 修复偶尔出现的规划报错 “The children format needs to be [WhenClause+, DefaultValue?]” 的问题。#38491
- 修复当投影中包含窗口函数，且同时存在一个列的原始列和其别名时，规划可能出现错误的问题。#38166
- 修复当聚合参数中含有 Lambda 表达式，可能导致规划报错的问题。#37109
- 修复在极端情况下可能出现的 Insert 报错：“MultiCastDataSink cannot be cast to DataStreamSink” 的问题。#38526
- 修复创建表时，新优化器对于传入的 `char(0)/varchar(0)` 没有正确处理的问题。#38427
- 修复 `char(255)toSql` 行为不正确的问题。#37340
- 修复 `agg_state`类型内部的 nullable 属性可能规划错误的问题。#37489
- 修复 Mark Join 时行数统计不准确的问题。#38270

8.3.8.4.10 查询执行

- 修复多个场景下，Pipeline 执行引擎被卡住导致查询不结束的问题。#38657 #38206 #38885 #38151 #37297
- 修复 NULL 和非 NULL 列在差集计算时导致的 Coredump 问题。#38750
- 修复 Delete 语句中 DECIMAL 类型为纯小数时报错的问题。#37801
- 修复 `width_bucket` 函数结果错误的问题。#37892
- 修复当单行数据很大且返回结果集也很大时（超过 2GB）查询报错的问题。#37990
- 修复单副本导入时 rpc 链接没有正确释放导致的 Coredump 问题。#38087
- 修复 `foreach` 函数处理 NULL 导致的 Coredump 问题。#37349
- 修复 stddev 在 DecimalV2 类型下结果错误的问题。#38731
- 修复`bitmap union` 计算性能慢的问题。#37816

- 修复 Profile 中聚合算子的 RowsProduced 没有设置的问题。#38271
- 修复 Hash Join 下计算 Hash 表 Bucket 数目时溢出的问题。#37193 #37493
- 修复 `jemalloc cache memory tracker` 记录不准确的问题。#37464
- 增加配置项 `enable_stacktrace`，用户可以通过设置此选项来控制 BE 日志中是否输出异常栈。#37713
- 修复 Arrow Flight SQL 在设置 `enable_parallel_result_sink` 为 false 时不能正常工作的问题。#37779
- 修复错误地使用 Colocate Join 的问题。#37361 #37729
- 修复 `round` 函数在 DECIMAL128 类型上计算溢出的问题。#37733 #38106
- 修复 `sleep` 函数传参 const 字符串时的 Coredump 问题。#37681
- 增加审计日志的队列长度，解决了数千并发场景下审计日志不能正常记录的问题。#37786
- 修复创建 Workload Group 导致的线程数过多，导致 BE Coredump 的问题。#38096
- 修复 MULTI_MATCH_ANY 函数导致的 Coredump 问题。#37959
- 修复`insert overwrite auto partition`导致事务回滚的问题。#38103
- 修复 TimeUtils formatter 没有使用正确时区的问题。#37465
- 修复 week/yearweek 常量折叠场景下结果错误的问题。#37376
- 修复 `convert_tz` 函数结果错误的问题。#37358 #38764
- 修复 `collect_set` 函数结合窗口函数使用时 Coredump 的问题。#38234
- 修复 `percentile_approx` 在滚动升级过程中导致的 Coredump 问题。#39321
- 修复 `mod` 函数在异常输入时导致的 Coredump 问题。#37999
- 修复 Broadcast Join 在 probe 开始运行时 Hash Table 构建未完成的问题。#37643
- 修复多线程下执行相同表达式可能导致 Java UDF 结果错误的问题。#38612
- 修复 `conv` 函数返回类型错误导致的溢出问题。#38001
- 修复 `json_replace` 函数返回类型不正确的问题。#3701
- 修复 `percentile` 聚合函数 Nullable 属性设置不合理的问题。#37330
- 修复 `histogram` 函数结果不稳定的问题。#38608
- 修复 Profile 中 Task State 显示不正确的问题。#38082
- 修复系统刚启动时部分 query 被错误取消的问题。#37662

8.3.8.4.11 半结构化数据管理

- 修复时间序列压缩的一些问题。#39170 #39176
- 修复压缩过程中索引大小统计错误的问题。#37232
- 修复倒排索引对不分词的超长字符串匹配可能不正确的问题。#37679 #38218
- 修复 `array_range` 和 `array_with_const` 函数在大数据量下内存占用高的问题。#38284 #37495
- 修复选择 ARRAY / MAP / STRUCT 类型的列时可能出现的 Coredump 问题。#37936
- 修复 Stream Load 指定 jsonpath 时 simdjson 解析错误导致导入失败的问题。#38490
- 修复 JSON 数据中有重复 Key 时处理异常的问题。#38146
- 修复 DROP INDEX 后可能出现查询报错的问题。#37646
- 修复索引压缩时在合并行检查中的错误返回问题。#38732
- 倒排索引 v2 格式现在支持修改列名。#38079
- 修复没有索引时 MATCH 函数匹配空字符串时 Coredump 的问题。#37947
- 修复倒排索引对 NULL 值处理的问题。#37921 #37842 #38741
- 修复 FE 重启后 `row_store_page_size` 不正确的问题。#38240

8.3.8.4.12 其他

- 修复时区配置问题，现在默认时区不再固定为 UTC+8，而是从系统配置中获取。#37294
- 修复由于存在多个 JSR 规范实现导致使用 Ranger 时出现的类冲突问题。#37575
- 修复部分 BE 代码中字段可能未初始化的问题。#37403
- 修复 Random Distributed 表 Delete 语句报错的问题。#37985
- 修复创建同步物化视图时错误地需要基表的 `alter_priv` 权限问题。#38011
- 修复当 TVF 中使用了 Resource 时未对 Resource 鉴权的问题。#36928

8.3.8.5 致谢

@133tosakarin、@924060929、@AshinGau、@Baymine、@BePPPower、@BiteTheDDDDt、@ByteYue、@CalvinKirs、@Ceng23333、@DarvenDuan、@FreeOnePlus、@Gabriel39、@HappenLee、@JNSimba、@Jibing-Li、@KassieZ、@Lchangliang、@LiBinfeng-01、@Mryange、@SWJTU-ZhangLei、@TangSiyang2001、@Tech-Circle-48、@Vallishp、@Yukang-Lian、@Yulei-Yang、@airborne12、@amorynan、@bobhan1、@cambyzju、@cjj2010、@csun5285、@dataroaring、@deardeng、@eldenmoon、@englefly、@feiniaofeiafei、@felixwluo、@freemandealer、@gavinchou、@ghkang98、@hello-stephen、@hubgeter、@hust-hhb、@jacktengg、@kaijchen、@kaka11chen、@keanji-x、@liaoxin01、@liutang123、@luwei16、@luzhijing、@lxr599、@morningman、@morrySnow、@mrhhsg、@mymeiyi、@platoneko、@qidaye、@qzsee、@seawinde、@shuke987、@sollhui、@starocean999、@suxiaogang223、@w41ter、@wangbo、@wangshuo128、@whutpencil、@wsjz、@wuwenchi、@wyxxxcat、@xiaokang、@xiedeyantu、@xinyiZzz、@xy720、@xzj7019、@yagagagaga、@yiguolei、@yujun777、@z404289981、@zclllyybb、@zddr、@zfr9527、@zhangbutao、@zhangstar333、@zhannngchen、 @zhiqiang-hhhh、 @zjj、 @zy-kkk、 @zzzxl1993

8.3.9 Release 3.0.0

亲爱的社区小伙伴们，我们很高兴地向大家宣布，在近期我们迎来了 Apache Doris 3.0 版本的正式发布，欢迎大家下载使用体验！

从 3.0 系列版本开始，Apache Doris 开始支持存算分离模式，用户可以在集群部署时选择采用存算一体模式或存算分离模式。基于云原生存算分离的架构，用户可以通过多计算集群实现查询负载间的物理隔离以及读写负载隔离，并借助对象存储或 HDFS 等低成本的共享存储系统来大幅降低存储成本。

3.0 版本是 Apache Doris 在湖仓一体演化路线上的重要里程碑版本。在 3.0 版本中 Apache Doris 增加了数据湖写回功能，用户可以在 Apache Doris 中完成多个数据源之间的数据分析、共享、处理、存储操作。结合异步物化视图等能力，Apache Doris 可以作为企业统一的湖仓数据处理引擎，帮助用户更好的管理湖、仓、数据库中的数据。与此同时，3.0 版本引入了 Trino Connector 类型，用户可以快速使用 Trino Connector 来连接或适配更多数据源、并可以利用 Apache Doris 的高性能计算引擎提供比 Trino 更快的数据查询能力。

3.0 版本同样对 ETL 批处理场景进行了增强，对`insert into select`、`delete` 和 `update` 操作提供了显式事务支持；对查询执行过程中的可观测性进行了增强。

在性能方面，3.0 版本的查询优化器在框架能力、基础设施以及规则扩充等方面做了重要增强，针对更复杂更多样的业务场景提供更极致的优化能力，盲测性能更高。实现了自适应的 Runtime Filter 计算方式，能够在运行时根据数据大小准确估算 Runtime Filter，在大数据量和高压力场景下有更好的性能表现。对异步物化视图的构建能力、透明改写能力以及性能均进行了增强，使得物化视图在查询加速、数据建模等场景具有更好的稳定性和易用性。

在 3.0 版本的研发过程中，有超过 170 名贡献者为 Apache Doris 提交了近 5000 个优化与修复。 来自飞轮科技、百度、美团、字节跳动、腾讯、阿里、快手、华为、天翼云等企业的贡献者与社区深度共建，贡献了大量来自真实业务场景下测试 Case 来帮助我们持续打磨、共同改进，在此向所有参与版本研发、测试和需求反馈的贡献者们表示最衷心的感谢。

- GitHub 下载：https://github.com/apache/doris/releases
- 官网下载：https://doris.apache.org/download

8.3.9.1 1. 存算分离全新架构

从 Apache Doris 3.0 版本开始，Apache Doris 开始支持存算分离模式，用户可以在集群部署时选择采用存算一体模式或存算分离模式。

全新存算分离模式对计算与存储进行了解耦，计算节点不再存储主数据，而是引入共享存储层（HDFS 与对象存储）作为统一的数据主存储空间，计算资源和存储资源可以独立扩缩容，为用户带来了多方面价值：

- 负载隔离：多个计算集群共享同一份数据，用户可以使用多计算集群对不同业务或者在离线的负载进行隔离；
- 存储成本大幅降低：全量数据存储到成本更低且极其可靠的共享存储中，热数据仅在本地 Cache，相比存算一体三副本，存储成本最高下降至原先的 1/10；
- 计算资源弹性：数据不保存在计算节点，计算资源可以按照负载需求实现灵活弹性扩缩容，比如单个计算集群的扩缩容或者加减计算集群，弹性带来了资源配置的灵活性以及成本的降低；

- 系统鲁棒性的提升：数据存储到共享存储，Doris 不再有多副本一致性的逻辑，会大幅度简化分布式存储带来的复杂度，从而会提升系统的鲁棒性。
- 数据共享和克隆的灵活性：存算分离架构的灵活性不止在一个 Doris 集群内部，在跨 Doris 集群时也应该体现出灵活性，比如 Doris 集群 A 中的库表可以轻量地在 Doris 集群 B 中完成克隆，即只做元数据级别的复制，不做数据的复制。

8.3.9.1.1 1-1. 从存算一体到存算分离

在存算一体模式中，Apache Doris 整体由 Frontend（FE）和 Backend（BE）两类进程组成，其中 FE 节点主要负责用户请求接入、查询解析规划、元数据管理和集群管理等相关工作，BE 节点主要负责数据存储和查询计划的执行，多 BE 节点间采取 MPP 分布式计算架构，通过多副本一致性协议来帮助服务的高可用和数据的高可靠。

从存算一体到存算分离

新兴云计算基础设施的成熟，无论是公有云、私有云以及基于 K8s 的容器平台，云计算基础设施的革新催生了新的需求，越来越多用户期待 Apache Doris 针对云计算基础设施提供更加深度的适配，以便提供更加灵活强大的弹性能力，因此飞轮科技团队早在 2022 年基于 Apache Doris 设计并实现了云原生存算分离版本（SelectDB Cloud），在经过数百家企业近两年的大规模生产打磨后，将其贡献回 Apache Doris 社区，即当前 3.0 版本的存算分离模式。

在存算分离模式下，Apache Doris 整体架构演化成元数据层、计算层和共享存储层三层：

- 元数据层：新引入 Meta Service 模块来提供系统的元数据服务，例如库表、Schema、Rowset Meta、事务等信息，是一个可以横向扩展的无状态服务。目前 BE 的 Meta 已经全部进入了 Meta Service，FE 的部分 Meta 已进入 Meta Service，其它 Meta 在后续版本中也会全部进入 Meta Service。
- 计算层：负责执行查询规划，计算节点即无状态的 BE 节点，会缓存一部分 Tablet 元数据和数据到本地以提高查询性能。通过多个无状态的 BE 节点可以组成计算资源集合（即多计算集群），多个计算集群共享一份数据和元数据服务，计算集群支持随时弹性加减节点。
- 共享存储层：数据持久化到共享存储层，目前支持 HDFS 以及 S3、OSS、GCS、Azure Blob、COS、BOS、MinIO 等各类云上兼容 S3 协议的对象存储系统。

8.3.9.1.2 1-2 设计亮点

全新存算分离架构最大的设计亮点在于将 FE 全内存的元数据模式演变成共享的元数据服务，这一方案的优势在于，元数据服务提供了全局一致的状态视图，任何节点可以直接提交写入，不再需要经过 FE 做 Publish。写入时数据进入共享存储，元数据进入元数据服务，可以有效控制共享存储上的小文件数量，同时单表的实时写入性能和存算一体相差无几，整个系统的写入能力不再受限于单 FE 的处理能力。

Apache Doris 存算分离设计亮点

基于全局一致的状态视图，在数据 GC 时，我们采用了设计上容易证明正确性和效率更高的正向数据删除。具体而言，将共享存储中的数据纳入到在共享元数据服务提供的全局一致视图中，数据生成时绑定一个事务，元数据删除时也绑定一个事务，以此可以实现删除和写入不能一起成功，视图中记录了哪些数据需要删除，异步删除过程只需要根据事务记录正向删除数据即可，不需要反向 GC。

未来随着 FE 中 Tablet 相关的 Meta 进入共享服务，Doris 集群规模也不再受限于单 FE 内存。基于共享的元数据服务和正向的数据删除技术，在此基础上可以便捷地扩展数据共享、轻量克隆等功能。

8.3.9.1.3 1-3 业界同类方案对比

业界还有另一种存算分离架构的设计方案，将数据和 BE 节点的 Meta 存放在共享对象存储或者 HDFS 中，该方案会有如下的问题：

- 无法承载实时写：在写入数据时，数据会根据分区分桶规则映射到 Tablet 中并生成 Segment 文件以及 Rowset Meta。在写入阶段会通过 FE 进行两阶段提交（即 Publish），在 BE 节点接到 Publish 请求后再将 Rowset 设置为可见，Publish 是不允许失败的。如果将 Rowset Meta 保存在共享存储中，实时写入过程中的小文件数据是数据文件的三倍，一次写入数据结束后生成 Rowset Meta、一次 Publish 时更改 Rowset Meta 状态。Publish 是由 FE 节点单点驱动的，不论单个表或整个系统的写入能力都受限于 FE 节点。

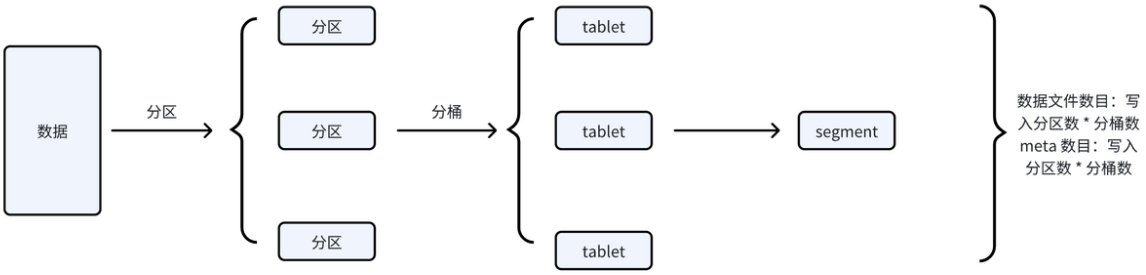


图 247: Doris 与业界同类方案对比

在 Apache Doris 3.0 版本正式发版后，我们对该方案实时数据写入性能与 Apache Doris 进行了对比。在此我们在相同计算资源下分别模拟 500 并发任务写入 10000 个 500 行的数据文件和 50 并发任务写入 250 个 20000 行的数据文件，可以看到在 50 并发下 Apache Doris 存算分离和存算一体模式的微批写入性能基本相当，而该方案写入性能与 Apache Doris 差距达到 100 倍。在 500 并发下，Apache Doris 存算分离模式性能稍有损耗，但对比该方案仍有超过 11 倍的巨大优势。为了保证测试公平性，Apache Doris 并未开启 Group Commit 服务端攒批的能力（该方案不具备此能力），而在开启 Group Commit 后实时写入能力还将进一步增强。

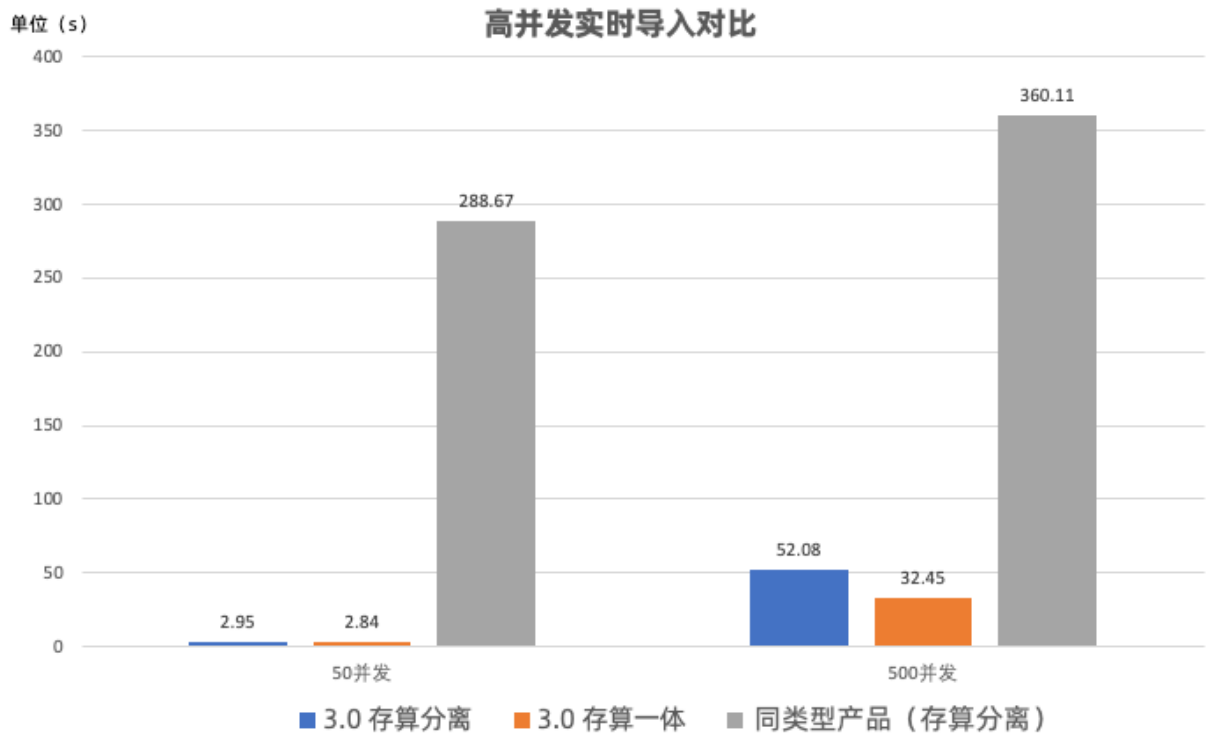


图 248: Doris 与业界同类方案对比

此外该方案在实时写入方面还存在稳定性和成本问题：

- 稳定性隐患：小文件数目多会给共享存储特别是 HDFS 带来更大的压力和不稳定隐患。
- 对象存储请求费用高：部分公有云对象存储的 Put 和 Delete 收费是 Get 的 10 倍，小文件数目多会导致对象存储请求费用大幅上升，甚至超过存储费用。
- 扩展性受限：FE 的 Meta 是全内存的，存算分离模式下往往会面对更大规模的数据存储，因此当 Tablet 数量超大时（例如超过千万级别），FE 内存的压力较大；整个系统的写入能力受到 FE 单点瓶颈的限制。
- 数据删除逻辑隐患：存算分离架构下数据只有一份，因此数据删除逻辑对于系统的可靠性至关重要。常规的跨系统数据删除做法是对比计算出差集，对于写入过程中的数据依赖超时时间，没办法从机制上 100% 避免删除和写入一起成功，删除和写入一起成功就会丢数据。另外在存储系统有异常时，用于计算差集的输入可能错误，这就可能导致误删除数据。
- 数据共享与轻量级克隆：存算分离架构未来可以借助于架构的灵活性实现数据共享和轻量级数据克隆，降低企业数据管理的负担。如若每个集群是单独的 FE，跨集群克隆数据之后，很难计算出哪些数据没有被引用可以被删除，跨多个集群计算很容易误删数据。

与之相比，Doris 3.0 版本将 FE 全内存的元数据模式演变成共享的元数据服务有效地克服了以上问题。

8.3.9.1.4　1-4 查询性能对比

由于存算分离模式下数据需要从远端共享存储系统中读取，因此数据传输的主要瓶颈，从存算一体模式下的磁盘 IO 转变为存算分离模式下的网络带宽，在一定程度上会造成性能损耗。

为了加速数据访问，Apache Doris 实现了基于本地磁盘的高速缓存机制，并提供 LRU 和 TTL 两种高效的缓存管理策略，并对索引相关的数据进行了优化，旨在最大程度上缓存用户常用数据、提升查询性能。新导入的数据将异步写入缓存中，以加速最新数据的首次访问。如果查询所需数据不在缓存中，系统将从远端存储中读取该数据进内存并同步写入缓存中，以便于后续查询。在涉及多计算集群的应用场景中，Apache Doris 提供缓存预热功能，当新计算集群建立时，用户可以选择对特定的数据（如表或分区）进行预热，以进一步提高查询效率。

在此我们分别对存算一体模式和存算分离模式进行了不同缓存下的性能测试，以 TPC-DS 1TB 测试集为例，主要结果如下：

- 完全命中缓存时（即查询的所有数据均被加载进缓存中）存算分离模式与存算一体模式查询性能完全持平；
- 部分命中缓存时（即测试开始前清空所有缓存，初始状态下缓存中无任何数据，在测试过程中数据被逐渐加载进缓存中，性能随之持续提升）存算分离模式与存算一体模式查询性能基本相当，总体性能损耗约 10%，这一测试场景也与用户实际应用中最为类似。
- 完全未命中任何缓存时（每次执行 SQL 前均清理所有缓存，模拟极端情况）性能损耗约 35%。即使在冷数据读取时存在一定性能损耗，但相较于业内其他同类系统，存算分离模式下的 Apache Doris 仍有着极为明显的性能优势。

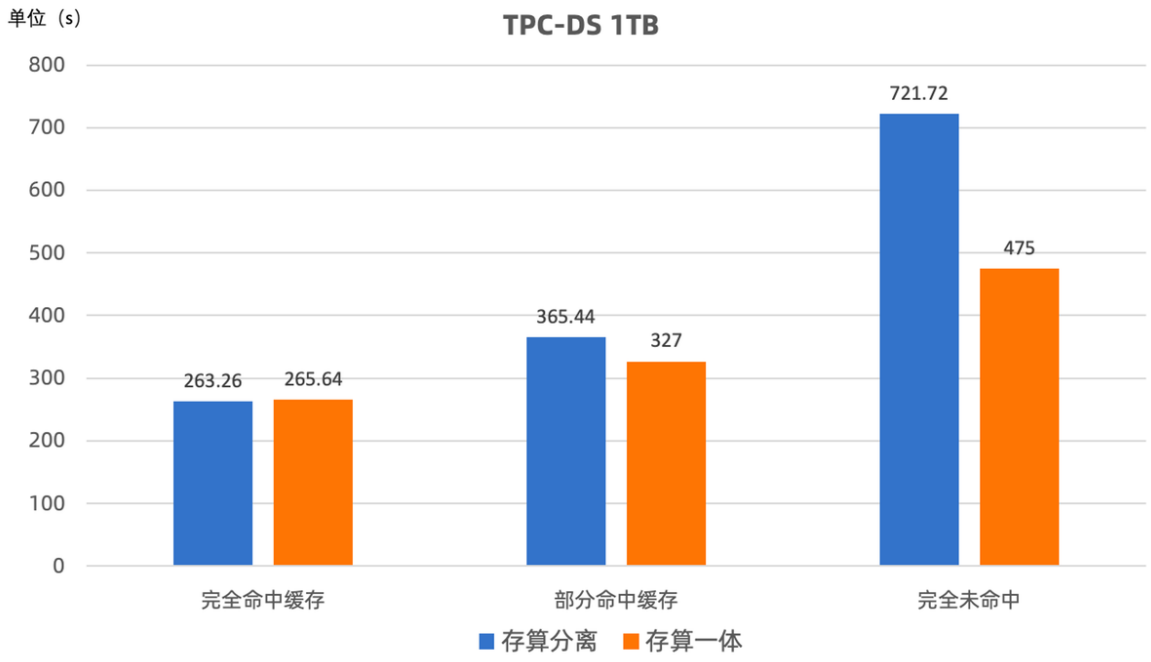


图 249: Doris 查询性能对比

8.3.9.1.5 1-5 写入性能对比

在写入性能方面，我们在相同计算资源下分别模拟了批量导入和高并发实时导入两大场景，存算一体模式和存算分离模式的写入性能对比结论如下：

- 在批量数据导入场景，导入 TPC-H 1TB 和 TPC-DS 1TB 测试数据集，在存算一体模式采用单副本的情况下，存算分离模式写入性能较存算一体模式分别提升了 20.05% 和 27.98%。批量导入时 Segment 文件大小一般会在几十 MB 到上百 MB，存算分离模式会将 Segment 文件切分成多个小文件并发上传到对象存储，因而会带来比写本地磁盘更高的吞吐。实际部署中存算一体模式一般会采用三副本，此时存算分离模式的写入性能优势会更加明显。
- 高并发实时导入场景在前文中已介绍，在此不在赘述。

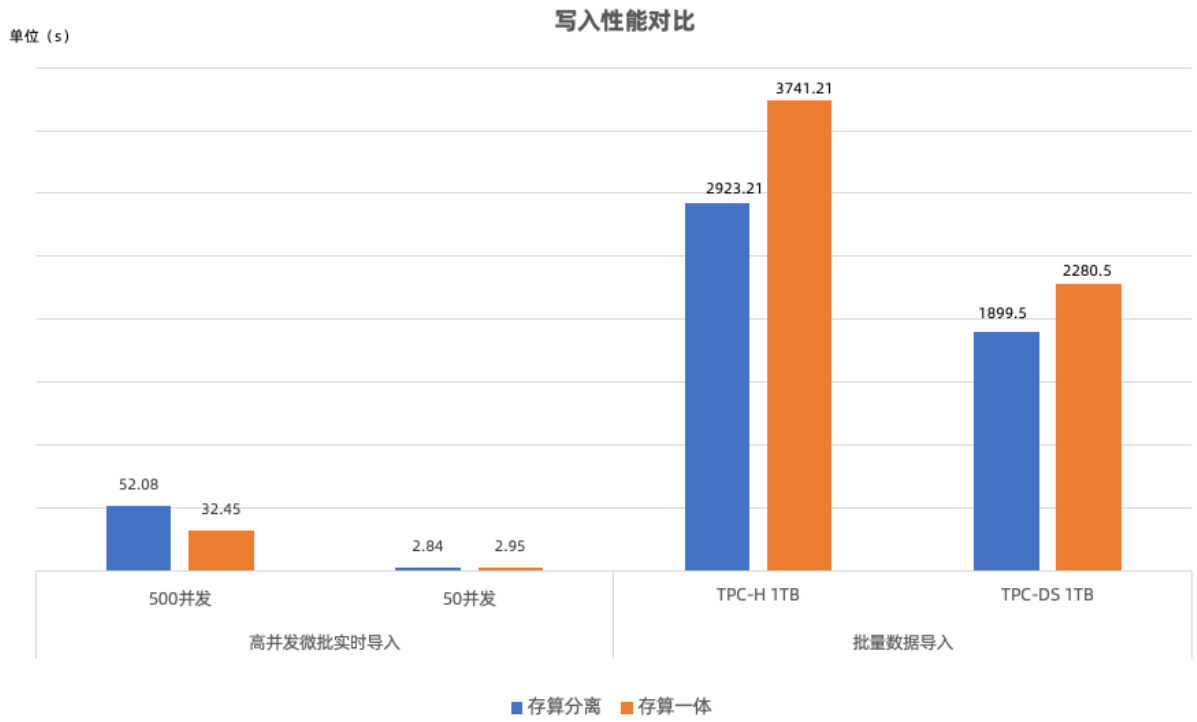


图 250: Doris 写入性能对比

8.3.9.1.6 1-6 生产运行经验

- 性能：实时数据场景下可以指定 TTL 的 Cache 以及写入时数据进入 Cache，使查询性能达到与存算一体。Compaction 和 Schema Change 后台任务生成的数据根据热度进入 Cache 可以避免查询抖动。
- 负载隔离：多计算集群提供了物理层资源隔离，比如不同业务单元适合使用多计算集群隔离。单个计算集群内依然需要使用 Workload Group 对不同的查询做资源的限制和隔离。

8.3.9.1.7　1-7 注意事项

- 当前 3.0 版本存算一体模式与存算分离模式不可共存，用户在集群部署时需要指定其中一种模式进行部署。
- 存算一体模式的部署和升级可以正常按照官网文档进行，推荐采用 Doris Manager 进行快速部署和集群升级，存算分离模式暂不支持 Doris Manager 部署和升级，后续我们将会持续迭代以实现更好支持。
- 暂不支持从 2.1 版本原地升级至 3.0 存算分离模式，需要在存算分离集群部署完成后通过工具进行数据迁移，后续也会支持通过 CCR 能力实现不停服迁移。

> 备注
>
> 参考文档：存算分离

8.3.9.2　2. 湖仓一体再进化

尽管 Apache Doris 定位于实时数据仓库，在以往版本中一直不拘于数据仓库的能力边界，在湖仓一体方向持续发力。而 3.0 版本是 Apache Doris 在湖仓一体路线上的重要里程碑版本，从 3.0 版本开始，Apache Doris 在湖仓一体场景的能力臻于完善。对于湖仓一体，我们认为其最核心的理念即数据无界、湖仓融合：

数据无界：将 Apache Doris 作为统一查询处理引擎，打破数据在不同系统间的屏障，在数据仓库、数据湖乃至数据流、本地数据文件等所有数据源端都能提供一致且极速的分析处理体验。

- 湖仓查询加速：无需将数据迁移至 Apache Doris，用户便可直接利用 Doris 高效的查询引擎，直接查询 Iceberg、 Hudi、 Paimon 等数据湖和 Hive 等离线数仓中的数据，实现查询分析的加速。
- 联邦分析：Apache Doris 通过扩展 Catalog 和存储插件，丰富其联邦分析能力，使用户无需将数据物理集中至统一的存储空间，在保持各数据源独立性和隐私性的同时，仅借助 Apache Doris 即可实现多个异构数据源的统一分析，既可以直查外部表以及存储文件、也可以执行内表和外表以及外表相互之间的关联分析，打破数据孤岛、提供全局一致性的数据洞察与分析。
- 数据湖构建：Apache Doris 增加了 Hive、Iceberg 数据写回功能，写回功能支持用户直接通过 Doris 创建 Hive、Iceberg 表，并将数据写入到表中。基于此，用户可以将 Apache Doris 中的内表数据写回离线湖仓，或者对离线湖仓中的数据利用 Apache Doris 进行数据加工后落地回离线湖仓，从而实现更简化和高效的数据湖构建。

湖仓融合：数据湖架构日益复杂，增加了用户技术选型成本与维护成本。同时实现多个系统一致的细粒度权限管控也变得非常困难，实时性也不足。为应对这一挑战，Apache Doris 融入了湖的核心特征，将其打造成一个轻量、高效的原生实时湖仓。

- 数据实时更新： 从 1.2 版本开始 Apache Doris 增强了主键模型，引入 Merge-on-Write，支持实时更新。借助这一特性，上游数据变更可以基于主键进行高频实时数据更新。
- 数据科学与 AI 计算支撑： 从 2.1 版本开始 Apache Doris 借助高效的 Arrow Flight 协议，增强了存储的开放性和对多计算负载的高效支持，这让 Apache Doris 支持数据科学和 AI 计算成为可能。

- 半结构化与非结构化数据增强： Apache Doris 先后引入 Array / Map / Struct / JSON / Variant 等数据类型，未来还会支持向量索引。
- 存算分离资源能效提升： 从 3.0 版本中支持了存算分离模式，进一步提升了资源效率和可扩展性。

8.3.9.2.1 2-1 湖仓查询加速

查询加速是湖仓一体化进程中的重要一环。借助 Apache Doris 强大的分布式查询引擎，可以帮助用户对湖仓数据进行快速分析。在 TPC-H 和 TPC-DS 标准测试集上，Apache Doris 的平均查询性能是 Trino/Presto 的 3-5 倍。

在 3.0 版本中，我们重点针对用户实际生产环境中的湖仓查询加速场景进行了优化，包括：

- 更精细的任务拆分策略： 通过对一致性哈希算法的调整以及引入任务分片权重机制，确保各个节点的查询负载均衡。
- 面向多分区、多文件场景的调度优化： 在大量文件（100 万 + ）场景下，通过异步、分批获取文件分片的方式，显著降低查询耗时（100s -> 10s ），并降低 FE 内存压力。

后续我们将进一步针对性的提升真实业务场景下的查询加速性能，提升用户实际感受，构建业界领先的湖仓查询加速引擎。

8.3.9.2.2 2-2 联邦分析 - 更丰富的数据源连接器

在之前的版本中，Apache Doris 已经支持了 10 余种主流湖、仓、关系型数据库的连接器。在 3.0 版本中，我们引入了 Trino Connector 兼容框架，极大扩展了 Apache Doris 可连接的数据源。借助该框架，仅需简单适配，用户即可通过 Doris 访问对应的数据源，并利用 Doris 的极速计算引擎进行数据分析。

目前 Doris 已完成 Delta Lake、 Kudu、 BigQuery、 Kafka、 TPCH、 TPCDS 等多种 Connector 的适配，也欢迎所有开发者参考开发指南，为 Apache Doris 适配更多数据源。

> 备注参考文档：
>
> - 接入 Trino Connector
> - TPC-H
> - TPC-DS
> - Delta Lake
> - Kudu
> - BigQuery

8.3.9.2.3 2-3 数据湖构建

在 3.0 版本中，Apache Doris 增加了 Hive、 Iceberg 数据写回功能。写回功能支持用户直接通过 Doris 创建 Hive、Iceberg 表，并将数据写入到表中。该功能使得 Apache Doris 在湖仓数据处理能力上形成闭环，用户可以在 Apache Doris 中完成多个数据源之间的数据分析、共享、处理、存储操作。

在后续的迭代版本中，Apache Doris 将进一步完善对数据湖表格式的支持以及存储 API 开放性。

> 备注
>
> 参考文档：数据湖构建

8.3.9.3 3. 半结构化分析全面增强

在过去发布的 2.0 和 2.1 版本中，Apache Doris 陆续引入了倒排索引、N-Gram Bloom Filter、Variant 数据类型等重磅特性，支持高性能的全文检索和任意维度分析，对复杂半结构化数据的存储和处理分析更加灵活高效。

在 3.0 版本中，我们继续对这一场景能力进行了全面增强，在应对半结构化数据分析和日志检索分析场景的挑战时更加得心应手。

Variant 数据类型在经过大规模生产打磨后，已具备充分的稳定性，成为 JSON 数据存储和分析的首选。在 3.0 版本中我们对 Variant 类型进行了多项优化：

- Variant 数据类型支持创建索引加速查询，包括倒排索引、 Bloom Filter 索引以及内置的 ZoneMap 索引；
- 对于包含 Variant 数据类型的 Unique 模型表，支持灵活的部分列更新；
- Variant 数据类型支持在存算分离模式上使用，并对其元数据存储进行了优化；
- 支持将 Variant 数据类型导出成 Parquet、 CSV 等格式。

倒排索引自 2.0 版本开始被引入起，经历一年多的打磨，目前已具备较高的成熟度，在数百家企业的生产环境中长期运行。在 3.0 版本中，我们对倒排索引进行了多项优化：

- 通过锁并发等一系列的性能优化，在实时报表分析场景中 Apache Doris 在查询延迟和并发等关键指标已大幅超过 Elasticsearch；
- 在存算分离模式下优化了索引文件，减少远端存储的调用，大幅优化了索引查询延迟；
- 增加了对 Array 类型的支持，加速 `array_contains` 查询；
- 对短语查询系列 `match_phrase_*` 功能进行增强，包括支持词距 slop、短语前缀匹配 `match_phrase_prefix` 等。

8.3.9.4 4. ETL 能力持续增强

8.3.9.4.1 4-1. 事务增强

数据加工在数据仓库中是一个常见的场景，通常需要多个数据变更作为一个事务。Doris 3.0 对`insert into select`、`delete` 和 `update` 操作提供了显式事务支持。具体的应用场景比如：

- 事务性要求：例如更新一个时间范围的数据，通常的做法是先删除这个时间范围的数据，然后写入。考虑到数据已经在服务，希望查询时要么看到老的数据要么看到新的数据，因此可以在一个事务中执行 `delete` 和 `insert into select`。

```
BEGIN;
DELETE FROM table WHERE date >= "2024-07-01" AND date <= "2024-07-31";
INSERT INTO table SELECT * FROM stage_table;
COMMIT;
```

- 简化任务失败的处理：例如一个数据加工包含 2 个insert into select，在一个事务中去执行，任何一步失败只需要重新开始执行即可。

```
BEGIN WITH LABEL label_etl_1;
INSERT INTO table1 SELECT * FROM stage_table1;
INSERT INTO table SELECT * FROM stage_table;
COMMIT;
```

备注

参考文档：

- 事务
- 目前 CCR 暂未支持显示事务同步。

8.3.9.4.2 4-2 可观测性增强

- Profile 实时获取：在过去，某些复杂查询可能由于 Plan 的原因或者数据的原因，导致计算量特别大，开发者只有在查询结束后才能拿到 Profile 做性能分析以发现问题，有可能对线上系统产生影响。通过 Profile 实时获取，开发者可以在查询的运行过程中实时获取查询执行的 Profile，看到每个算子的执行情况，不需要等到查询执行结束。基于实时 Profile 获取功能，开发者还可以进一步监控每一 ETL 作业的执行进度。
- 系统表 backend_active_tasks：backend_active_tasks 系统表提供了每个 Query 在每个 BE 上的实时资源消耗信息，可以通过 SQL 对系统表做分析计算，进而实时获得每个 Query 的资源使用量，有利于用户发现大查询或者不合理的工作负载。

8.3.9.5 5. 多表物化视图

在 Apache Doris 3.0 版本中，对多表物化视图的构建能力进行了增强并提高稳定性，拓展了透明改写的能力、透明改写性能提升 2 倍，重构了同步物化视图的透明改写逻辑并拓展了透明改写的能力，同时在异步物化视图的易用性上做了增强，让物化视图在查询加速，数据建模等场景更好用、更稳定。

8.3.9.5.1 5-1. 构建刷新功能

- 物化视图的支持分区增量更新，大大减少了物化视图的构建成本，并且支持物化视图分区上卷，满足不同粒度的分区刷新物化视图需求。

- 支持构建嵌套物化视图，在数据建模场景更好用。
- 允许异步物化视图创建索引和指定排序键，命中物化视图后查询速度会提升。
- 提高了物化视图 DDL 的易用性，支持物化视图原子替换，可以保证物化视图一直可用的情况下，修改物化视图定义 SQL。
- 允许物化视图使用非确定函数，在定时构建 T+1 物化数据的场景更易用。
- 新增了触发刷新物化的功能，在使用嵌套物化视图数据建模时，保证数据一致性。
- 拓展了可以构建分区物化视图的 SQL 模式，让更多的场景可以使用分区增量更新能力。

8.3.9.5.2 5-2. 构建刷新稳定性

- 支持指定物化视图构建时的 Workload Group，限制物化视图构建使用的资源，保障查询的可用资源。

8.3.9.5.3 5-3. 透明改写功能拓展

- 支持了更多 Join 类型的改写，并且支持了 Join 衍生改写。当查询和物化视图的 Join 的类型不一致时，如果物化可以提供查询所需的所有数据时，通过在 Join 的外部补偿谓词，也可以进行透明改写。
- 增强了聚合改写，支持了更多的聚合函数上卷，并且支持了多维聚合函数 GROUPING SETS、ROLLUP、CUBE 的改写，同时支持了查询包含聚合，物化不包含聚合的改写，可以节省 Join 连接和表达式计算。
- 支持了嵌套物化视图的透明改写，在复杂的查询加速场景下，可以借助嵌套物化视图来进行极致加速。
- 分区物化视图部分分区失效，支持物化视图 Union All 基表补全数据，增加了分区物化视图的适用范围。

8.3.9.5.4 5-4. 透明改写性能

- 持续优化了透明改写的性能，透明改写性能是 2.1.0 版本的两倍。

> 备注参考文档：
>
> - 异步物化视图概览
> - 查询异步物化视图

8.3.9.6 6. 性能提升

8.3.9.6.1 6-1. 优化器更智能

在 3.0 版本中，查询优化器在框架能力、分布式计划支持、优化器基础设施以及规则扩充等方面做了重要增强，支持更复杂更多样的业务场景、提供更极致的优化能力，对于复杂 SQL 有更高的盲测性能：

- 更完善的计划枚举能力：对计划枚举的关键结构 Memo 做了 Projection 规范化重构，不仅提升了 Cascades 框架枚举有效计划的效率和枚举出更优计划的可能性，同时也修复了历史版本 Join Reorder 过程中列裁剪不完全导致的 Join 算子不必要开销等遗留问题，有效提升相应场景下的执行性能。
- 更完善的分布式计划支持：对分布式查询计划做了增强，使得聚合，连接和窗口函数操作能更智能的识别中间运算结果的数据特征，避免无效的数据重分布操作，提升相应场景的性能。同时，3.0 版本中对多副本连续执行模式下的执行性能也进行了优化，使其对数据缓存更友好，相应性能也得到较大提升。
- 更完善的优化器基础设施：在代价模型和统计信息估行方面，3.0 版本也修复了若干重要估行问题，代价模型问题的修复也更加适配执行引擎迭代，使得执行计划相较历史版本更稳定。
- 更丰富的 Runtime Filter 计划支持：3.0 版本在传统 Join Runtime Filter 的基础上，拓展了 TopN Runtime Filter 的能力。典型场景如存在 TopN 算子时，可以生成 TopN 的 Runtime Filter 以获得更好的执行性能。
- 更丰富的优化规则库：持续的增强和迭代基础优化规则库，通过持续不断地业务反馈和内测增强，3.0 版本中加入了如 Intersect Reorder 等典型的优化规则支持，使得优化器的规则成熟度得到了进一步的提升。

8.3.9.6.2 6-2. 自适应 Runtime Filter

Runtime Filter 是否能够准确生成对查询性能的影响至关重要，在过去 Doris 非常依赖于用户手工设置或者优化器根据统计信息来生成，在某些情况下一旦设置不准确将会带来性能抖动。

在 3.0 版本实现了自适应的 Runtime Filter 计算方式，能够在运行时根据数据大小准确估算 Runtime Filter，在大数据量和高压力场景下有更好的性能表现。

8.3.9.6.3 6-3. 函数性能优化

- 3.0 版本对数十个函数的向量化实现做了改进，部分常用函数的性能提升 50% 以上。
- 对 Nullable 类型数据的聚合计算也做了大量优化，性能提升 30%。

8.3.9.6.4 6-4. 盲测性能进一步提升

我们对 3.0 版本与 2.1 版本分别在 TPC-DS 和 TPC-H 测试数据集上进行了盲测性能测试，查询性能分别提升了 7.3% 以及 6.2%。

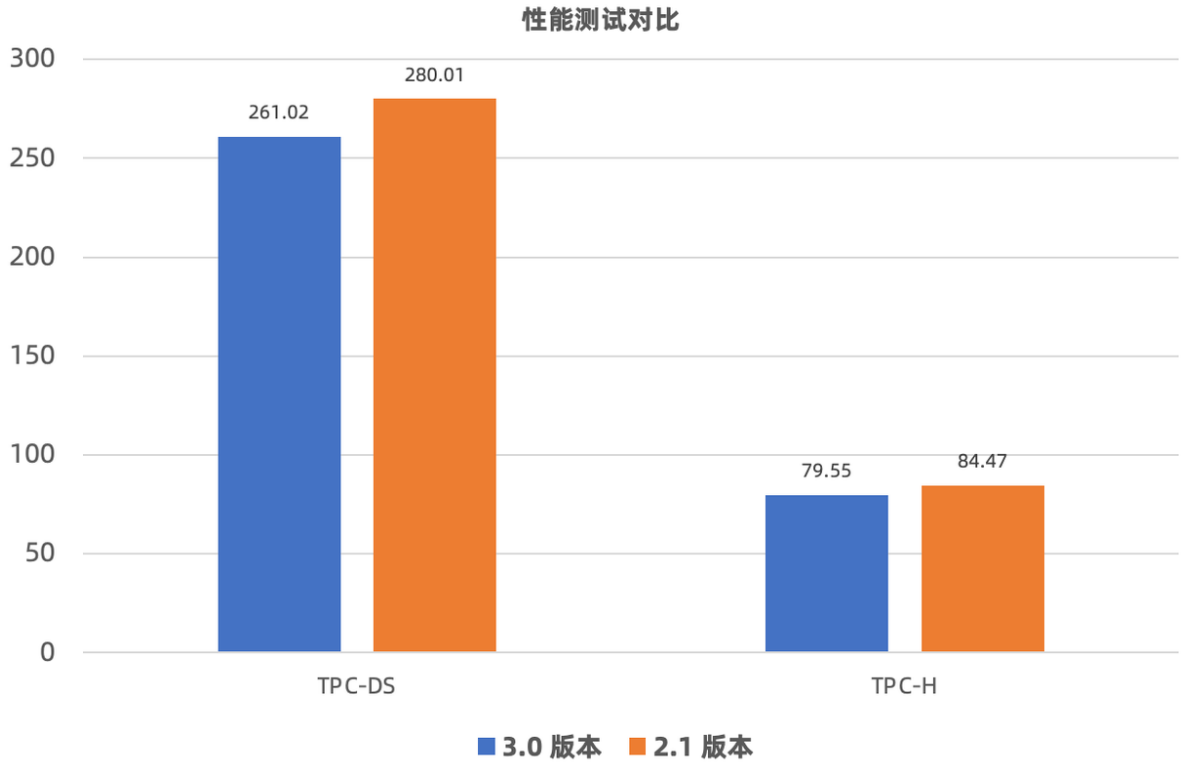


图 251: 盲测性能进一步提升

8.3.9.7 7. 新功能

8.3.9.7.1 7-1. Java UDTF

从 3.0 版本开始支持增加对 Java UDTF 的支持，主要操作如下：

- 编写 UDTF：UDTF 和 UDF 函数一样，需要用户自主实现一个 `evaluate` 方法，注意 UDTF 函数的返回值必须是 Array 类型。

```
public class UDTFStringTest {
    public ArrayList<String> evaluate(String value, String separator) {
        if (value == null || separator == null) {
            return null;
        } else {
            return new ArrayList<>(Arrays.asList(value.split(separator)));
        }
    }
}
```

- 创建 UDTF：这里会默认创建两个对应的函数 java-utdf和 java-utdf_outer，outer的后缀在表函数生成 0 行数据时添加一行Null数据。

```
CREATE TABLES FUNCTION java-utdf(string, string) RETURNS array<string> PROPERTIES (
    "file"="file:///pathTo/java-udaf.jar",
    "symbol"="org.apache.doris.udf.demo.UDTFStringTest",
    "always_nullable"="true",
    "type"="JAVA_UDF"
);
```

> 备注参考文档：Java UDF - UDTF

8.3.9.7.2 7-2 生成列

生成列是一种特殊的数据库表列，其值由其他列的值计算而来，而不是直接由用户插入或更新。该功能支持预先计算表达式的结果，并存储在数据库中，适用于需要频繁查询或进行复杂计算的场景。

生成列可以在数据导入或更新时自动根据预定义的表达式计算结果，并将这些结果持久化存储。在后续的查询过程中，可以直接访问这些已经计算好的结果，而无需在查询时再进行复杂的计算，从而显著减少查询时的计算负担，提升查询性能。

从 3.0 版本开始 Apache Doris 支持生成列功能，创建表时可以指定列为 Generated 列。Generated 列可在写入时，根据定义的表达式，自动获取计算结果。相比于 Default value，可以定义更为复杂的表达式，但不可以显式写入指定的值。

> 备注
>
> 参考文档：
>
> CREATE TABLE AND GENERATED COLUMN

8.3.9.8 8. 功能改进

8.3.9.8.1 8-1. 同步物化视图

重构同步物化视图选择逻辑，将选择逻辑从 RBO 迁移至 CBO，使其与异步物化视图保持一致。

此功能默认开启。如遇到问题，可使用开关 `set global enable_sync_mv_cost_based_rewrite = false` 回退到 RBO 模式。

8.3.9.8.2 8-2. Routine Load 导入

在之前的版本中，Routine Load 存在部分体验性问题：任务在 BE 节点之间调度不均、调度不及时，配置繁琐、需要同时改 FE 和 BE 的多个配置进行调优，稳定性不够高、重启或升级等都可能导致 Routine Load Job 暂停，需要人工介入才能恢复。我们针对这些问题对 Routine Load 做了大量的优化，使得 Routine Load 更高效、稳定且易用，为用户提供更好的体验。

- 资源调度方面，改善了任务在 BE 节点之间的调度均衡性，确保任务能够更加均匀地分配到各个节点。对于发生无法恢复错误的 Job 及时暂停，避免无效调度导致的资源浪费，并改进了调度的及时性，提升了 Routine Load 的导入性能。
- 参数配置方面，简化了配置过程，用户在大部分环境下无需修改 FE 和 BE 的配置进行调优。同时引入超时参数自动调整机制，避免集群压力增大时，任务持续超时重试。
- 稳定性方面，增强了其在各种异常场景下的稳定性，如 FE 切主、BE 滚动升级、Kafka 集群异常等都能持续稳定的工作。优化了 Auto Resume 机制，使得在故障恢复后 Routine Load 能够自动恢复运行，减少了用户的人工介入。

8.3.9.9 9 行为变更

- `cpu_resource_limit` 将不再支持，所有的资源隔离方式都依赖 Workload Group 实现。
- 从 3.0 版本开始请使用 JDK 17，推荐版本 `jdk-17.0.10_linux-x64_bin.tar.gz`。

8.3.9.10 立刻开启 3.0

在 3.0 版本正式发布之前，存算分离架构在 SelectDB 数百家企业的生产环境已经得到近两年的大规模打磨，同时来自百度、美团、字节跳动、腾讯、阿里、快手、华为、天翼云等企业多名贡献者与社区深度共建，贡献了大量来自真实业务场景下测试 Case，使得 3.0 版本在功能易用性和系统稳定性得到了有力验证，推荐有存算分离需求的用户下载 3.0 版本进行尝鲜。

后续我们也将加快发版迭代节奏，力求给所有用户带来更稳定的版本体验。

8.3.9.11 致谢

在此再次向所有参与版本研发、测试和需求反馈的贡献者们表示最衷心的感谢：

@133tosakarin、@390008457、@924060929、@AcKing-Sam、@AshinGau、@BePPPower、@BiteTheDDDDt、@ByteYue、@CSTGluigi、@CalvinKirs、@Ceng23333、@DarvenDuan、@DongLiang-0、@Doris-Extras、@Dragonliu2018、@Emor-nj、@FreeOnePlus、@Gabriel39、@GoGoWen、@HappenLee、@HowardQin、@Hyman-zhao、@INNOCENT-BOY、@JNSimba、@JackDrogon、@Jibing-Li、@KassieZ、@Lchangliang、@LemonLiTree、@LiBinfeng-01、@LompleZ、@M1saka2003、@Mryange、@Nitin-Kashyap、@On-Work-Song、@SWJTU-ZhangLei、@StarryVerse、@TangSiyang2001、@Tech-Circle-48、@Thearas、@Vallishp、@WinkerDu、@XieJiann、@XuJianxu、@XuPengfei-1020、@Yukang-Lian、@Yulei-Yang、@Z-SWEI、@ZhongJinHacker、@adonis0147、@airborne12、@allenhooo、@amorynan、@bingquanzhao、@biohazard4321、@bobhan1、@caiconghui、@cambyzju、@caoliang-web、@catpineapple、@cjj2010、@csun5285、@dataroaring、@deardeng、@dongsilun、@dutyu、@echo-hhj、@eldenmoon、@elvestar、@englefly、@feelshana、@feifeifeimoon、@feiniaofeiafei、@felixwluo、@freemandealer、@gavinchou、@ghkang98、@gnehil、@hechao-ustc、@hello-stephen、@httpshirley、@hubgeter、@hust-hhb、@iszhangpch、@iwanttobepowerful、@ixzc、@jacktengg、@jackwener、@jeffreys-cat、@kaijchen、@kaka11chen、@kindred77、@koarz、@kobe6th、@kylinmac、@larshelge、@liaoxin01、@lide-reed、@liugddx、@liujiwen-up、

@liutang123、@lsy3993、@luwei16、@luzhijing、@lxliyou001、@mongo360、@morningman、@morrySnow、@mrhhsg、@my-vegetable-has-exploded、@mymeiyi、@nanfeng1999、@nextdreamblue、@pingchunzhang、@platoneko、@py023、@qidaye、@qzsee、@raboof、@rohitrs1983、@rotkang、@ryanzryu、@seawinde、@shoothzj、@shuke987、@sjyango、@smallhibiscus、@sollhui、@sollhui、@spaces-X、@stalary、@starocean999、@superdiaodiao、@suxiaogang223、@taptao、@vhwzx、@vinlee19、@w41ter、@wangbo、@wangshuo128、@whutpencil、@wsjz、@wuwenchi、@wyxxxcat、@xiaokang、@xiedeyantu、@xiedeyantu、@xingyingone、@xinyiZzz、@xy720、@xzj7019、@yagagagaga、@yiguolei、@yongjinhou、@ytwp、@yuanyuan8983、@yujun777、@yuxuan-luo、@zclllyybb、@zddr、@zfr9527、@zgxme、@zhangbutao、@zhangstar333、@zhannngchen、@zhiqiang-hhhh、@ziyanTOP、@zxealous、@zy-kkk、@zzzxl1993、@zzzzzzzs

8.4 v2.1

8.4.1 Release 2.1.11

亲爱的社区小伙伴们，我们很高兴地向大家宣布，在 8 月 15 日我们引来了 Apache Doris 2.1.11 版本的正式发布，欢迎大家下载使用。

8.4.1.1 行为变更

- time_series_max_tablet_version_num 控制时序 compaction 策略表的最大版本数目。#51371
- 修复冷热分层时 hdfs root_path 没有生效的问题。#48441
- 在新优化器（Nereids）中，当查询时的表达式的深度或宽度超过阈值限制时，无论是否开始查询回退到老优化器，都不会回退。#52431
- 统一了开始 unicode 名字与否的名字检查规则，现在非 unicode 名字规则是 unicode 名字规则的严格子集。#53264

8.4.1.2 新功能

8.4.1.2.1 查询执行引擎

- 引入系统表 routine_load_job 查看 routine load job 信息。#48963

8.4.1.2.2 查询优化器

- 支持了 MySQL 的 GROUP BY 上卷语法 GROUP BY ⋯ WITH ROLLUP。#51978

8.4.1.3 改进提升

8.4.1.3.1 查询优化器

- 优化了在聚合模型表和主键模型 mor 表上收集统计信息的性能。#51675

8.4.1.3.2 异步物化视图

- 优化了透明改写的规划性能 #51309
- 优化了刷新的性能 #51493

8.4.1.4 Bug 修复

8.4.1.4.1 导入

- 修复 routineload alter 属性之后 show 展示结果不符合预期的问题 #53038

8.4.1.4.2 湖仓一体

- 修复某些情况读取 iceberg equality delete 数据错误的问题 #51253
- 修复 iceberg hadoop catalog 在 kerberos 环境下报错的问题 #50623 #52149
- 修复 Kerberos 环境下 Iceberg 表写入事务提交失败的问题 #51508
- 修复 Iceberg 表写入事务提交错误的问题 #52716
- 修复某些情况下访问 kerberos 环境的 Hudi 表数据报错的问题 #51713
- SQL Server Catalog 支持识别 IDENTITY 列信息 #51285
- 修复某些情况下 Jdbc Catalog 表无法获取行数信息的问题 #50901
- 优化 orc zlib 在 x86 环境下的解压性能并修复潜在问题 #51775
- 在 Profile 中增加 Parquet/ORC 条件过滤和延迟物化相关的指标 #51248
- 优化 ORC Footer 的读取性能 #51117
- 修复 Table Valued Function 无法读压缩格式的 json 文件的问题 #51983
- 修复某些情况下并发刷新 Catalog 导致元数据不一致的问题 #51787

8.4.1.4.3 索引

- 修复了倒排索引在处理包含 CAST 操作的 IN 谓词时出现的查询错误，避免返回错误的查询结果。#50860
- 修复了倒排索引在执行异常情况下的内存泄漏问题。#52747

8.4.1.4.4 半结构化数据类型

- 修复了一些 json 函数在 null 值情况下结果错误的问题。
- 修复了一些 json 函数相关的 bug。#52543 #51516

8.4.1.4.5 查询优化器

- 修复解析字符串为日期失败时，查询无法继续执行的问题 #50900
- 修复了个别场景下常量折叠结果错误的问题 #51738
- 修复个别数组函数在遇到 null literal 作为输入时，无法正常规划的问题 #50899
- 修复在极端场景下，开启 local shuffle 可能导致结果错误的问题 #51313 #52871
- 修复了 replace view 可能导致 desc view 时看不到列信息的问题 #52043
- 修复了 prepare command 在非 master FE 节点上有可能无法正确执行的问题 #52265

8.4.1.4.6 异步物化视图

- 修复当基表列的类型变更，可能导致透明改写后查询失败的问题 #50730
- 修复了个别场景下，透明改写分区补偿错误的问题 #51899 #52218

8.4.1.4.7 查询执行引擎

- 修复 TopN 计算时如果遇到 variant 列类型，可能会 core 的问题。#52573
- 修复函数 bitmap_from_base64 在输入错误数据时会 Core 的问题。#53018
- 修复了 bitmap_union 函数在超大数据量时，一些结果错误的问题。#52033
- 修复了 multi_distinct_group_concat 在窗口函数中使用时计算错误的问题。#51875
- 修复了 array_map 函数，在极端值时可能 core 的问题。#51618 #50913
- 修复了错误的时区处理的问题。#51454

8.4.1.4.8 Others

- 修复多语句在主 FE 和非主 FE 行为不一致的问题。#52632
- 修复 prepared statment 在非主 FE 报错的问题。#48689
- 修复 roolup 操作时可能导致 CCR 中断的问题。#50830

8.4.2 Release 2.1.10

亲爱的社区小伙伴们，Apache Doris 2.1.10 版本已于 2025 年 05 月 17 日正式发布。 该版本持续在查询执行引擎、湖仓一体等方面进行改进提升与问题修复，进一步加强系统的性能和稳定性，欢迎大家下载体验。

- 立即下载
- GitHub 下载

8.4.2.1 行为变更

- DELETE 不再错误的需要目标表的 SELECT_PRIV 权限 #49794
- Insert Overwrite 不再限制对同一个表并发只能为 1 #48673
- Merge on write unique 表禁止使用时序 compaction #49905
- 禁止在 VARIANT 类型上 build index。#49159

8.4.2.2 新功能

8.4.2.2.1 查询执行引擎

- 支持了更多的 GEO 类型的计算函数ST_CONTAINS , ST_INTERSECTS, ST_TOUCHES，GeometryFromText，ST_Intersects, ST_Disjoint, ST_Touches。#49665 #48695
- 支持 years_of_week 函数。#48870

8.4.2.3 湖仓一体

- Hive Catalog 支持 Catalog 级别的分区缓存开关控制 #50724
- 更多详情，可参考文档

8.4.2.4 改进提升

8.4.2.4.1 湖仓一体

- Paimon 依赖版本升级到 1.0.1
- Iceberg 依赖版本升级到 1.6.1
- 将 Parquet Footer 的内存开销纳入 Memory Tracker 管控，以避免可能的 OOM 问题。#49037
- 优化 JDBC Catalog 的谓词下推逻辑，支持 AND/OR 等连接谓词的下推#50542
- 预编译版本默认携带 Jindofs 扩展包以支持阿里云 OSS-HDFS 访问。

8.4.2.4.2 半结构化管理

- ANY 函数支持 JSON 类型 #50311
- JSON_REPLACE，JSON_INSERT，JSON_SET，JSON_ARRAY 函数支持 JSON 数据类型和复杂数据类型#50308

8.4.2.4.3 查询优化器

- 当 in 表达式的 options 多于 `Config.max_distribution_pruner_recursion_depth` 时，不执行分桶裁剪，以提升规划速度 #49387

8.4.2.4.4 存储管理

- 减少日志和改进部分日志。#47647 #48523

8.4.2.4.5 其他

- 避免 thrift rpc END_OF_FILE 异常 #49649

8.4.2.5 Bug 修复

8.4.2.5.1 湖仓一体

- 修复某些情况下，在 Hive 侧新建表，Doris 侧无法立即查看到的问题 #50188
- 修复某些 Text 格式 Hive 表访问报错 “Storage schema reading not supported” 的问题 #50038
- 查看文档 get_schema_from_table 详情
- 修复某些情况下，写入 Hive/Iceberg 表时，元数据提交并发问题 #49842
- 修复某些情况下，写入存储在 oss-hdfs 上的 Hive 表失败的问题 #49754

- 修复当 Hive 分区键值有逗号的情况下，访问失败的问题 #49382
- 修复某些情况下，Paimon 表 Split 分配不均匀的问题 #50083
- 修复读取存储在 OSS 上的 Paimon 表时，无法正确处理 Delete 文件的问题 #49645
- 修复 MaxCompute Catalog 中，读取高精度 Timestamp 列时无法访问的问题 #49600
- 修复某些情况下，删除 Catalog 可能导致部分资源泄露的问题 #49621
- 修复某些情况下，读取 LZO 压缩格式的数据失败的问题 #49538
- 修复某些情况下，ORC 延迟物化功能导致复杂类型读取错误的问题 #50136
- 修复某些情况下，读取 pyorc-0.3 版本产生的 ORC 文件报错的问题 #50358
- 修复某些情况下，EXPORT 操作导致元数据死锁的问题 #50088

8.4.2.5.2 索引

- 修复多次添加、删除和重命名列操作后构建倒排索引的错误 #50056
- 在 index compaction 中索引对应的列唯一 ID 的校验，避免潜在的数据异常和系统错误 #47562

8.4.2.5.3 半结构化数据类型

- 修复某些情况下，VARIANT 类型转 JSON 类型返回 NULL 错误的结果 #50180
- 修复某些情况下，JSONB CAST 导致 crash #49810
- 禁止在 VARIANT 类型上 build index #49159
- 修复 named_struct 函数 decimal 类型精度正确性 #48964

8.4.2.5.4 查询优化器

- 修复常量折叠中的一些问题 #49413 #50425 #49686 #49575 #50142
- 公共表达式提取在 lambda 表达式上可能工作异常 #49166
- 修复消除 group by key 中的常量可能不能正常工作的问题 #49589
- 修复在极端场景下，由于统计信息的推导错误，规划无法正常执行的问题 #49415
- 修复部分依赖 BE 中元数据的 information_schema 表，不能获取完整数据的问题 #50721

8.4.2.5.5 查询执行引擎

- 修复了找不到 explode_json_array_json_outer 函数的问题。#50164
- 修复了 substring_index 不支持动态参数的问题。#50149
- 修复了很多 st_contains 函数计算结果不对的问题。#50115
- 修复了 array_range 函数可能导致的 core 的问题。#49993
- 修复了 date_diff 函数计算结果错误的问题。#49429
- 修复了一系列字符串函数在非 ASCII 编码下的乱码或者结果错误的问题。#49231 #49846 #49127 #40710

8.4.2.5.6 存储管理

- 修复某些情况下，动态分区表（Dynamic Partition Table）回放元数据失败的问题#49569
- 修复 ARM 下 streamload 可能因为操作序列丢数据的问题 #48948

- 修复 full compaction 报错以及可能导致 mow 数据重复的问题 #49825 #48958
- 修复没有持久化分区 Storage Policy 的问题。#49721
- 修复导入之后文件极小概率不存在的问题。#50343
- 修复 CCR 和磁盘均衡并发可能导致的文件找不见问题。#50663
- 修复备份恢复大快照时可能出现的 connection reset 问题。#49649
- 修复 FE Follower 丢失本地备份快照的问题。#49550

8.4.2.5.7 Others

- 修复某些场景下，审计日志可能丢失的问题 #50357
- 修复审计日志中 isQuery 标记可能不正确的问题 #49959
- 修复审计日志中部分查询 sqlHash 不正确的问题 #49984

8.4.3 Release 2.1.9

亲爱的社区小伙伴们，Apache Doris 2.1.9 版本已于 2025 年 04 月 02 日正式发布。 该版本持续在倒排索引、查询优化器与存储管理等方面进行改进提升与问题修复，进一步加强系统的性能和稳定性，欢迎大家下载体验。

- 立即下载
- GitHub 下载

8.4.3.1 行为变更

- Audit Log 中的 SQLHash 现在通过当前执行的 SQL 精确计算，解决了同一请求中所有 SQL 使用相同 SQLHash 的问题。#48242
- 查询返回的 ColumnLabelName 与 SQL 中的输入完全一致。#47093
- 所有在用户属性中设置的变量，优先级均高于 session 级别设置的变量。#47185

8.4.3.2 新功能

8.4.3.2.1 存储管理

- 禁止 rename 分区列。#47596

8.4.3.2.2 其他

- FE 监控指标新增 Catalog、 Database、 Table 数量指标。#47891

8.4.3.3 改进提升

8.4.3.3.1 倒排索引

- VARIANT 类型中的 ARRAY 支持倒排索引。#47688
- Profile 中展示每个过滤条件的倒排索引性能指标。#47504

8.4.3.3.2 查询优化器

- 支持在聚合查询中使用 SELECT`` *，如果下层 relation 仅输出聚合 key 列。#48006

8.4.3.3.3 存储管理

- CCR 优化回收 binlog 效率、小文件传输效率，并增强了混沌环境下的健壮性。#47547 #47313#45061
- 改进了导入的错误提示，使错误提示更加具体。#47918 #47470

8.4.3.4 Bug 修复

8.4.3.4.1 湖仓一体

- 修复 BE 端无法正确配置 krb5.conf 路径的问题。#47679
- 禁止 SELECT ``OUTFILE 语句重试以避免重复导出数据。#48095
- 修复无法通过 JAVA API 访问 Paimon 表的问题。#47192
- 修复无法写入存储位置为 s3a:// 的 Hive 表的问题。#47162
- 修复 Catalog 的 Comment 字段没有被持久化的问题。#46946
- 修复某些情况下，JDBC BE 端类加载泄漏的问题。#46912
- 修复 JDBC Catalog 无法使用高版本 ClickHouse JDBC Driver 的问题。#46026
- 修复某些情况下，读取 Iceberg Position Delete 导致 BE 宕机的问题。#47977
- 修复多分区列情况下读取 MaxCompute 表数据错误的问题。#48325
- 修复某些情况下读取 Parquet 复杂列类型错误的问题。#47734

8.4.3.4.2 倒排索引

- 修复 ARRAY 类型倒排索引空值处理错误的问题。#48231
- 修复对刚刚添加的列执行 BUILD INDEX 异常的问题。#48389
- 修复特殊字符 UTF8 编码索引被截断导致结果错误的问题。#48657

8.4.3.4.3 半结构化数据类型

- 修复 array_agg 函数在特殊情况下 crash 的问题。#46927
- 修复 Stream Load 导入 JSON 类型时，chunk 参数设置错误导致 crash 的问题。#48196

8.4.3.4.4 查询优化器

- 修复时间函数内嵌套 `current_date` 等关键字函数无法的进行常量折叠的问题。#47288
- 修复非确定性函数相关的结果错误问题。#48321
- 修复当原表有 on update 列属性时，CREATE TABLE LIKE 无法执行的问题。#48007
- 修复直查聚合模型表的物化视图可能产生非预期规划报错的问题。#47658
- 修复 PrepareStatement 因为内部 ID 溢出导致异常的问题。#47950

8.4.3.4.5 查询执行引擎

- 修复了查询系统表时，可能的查询卡住或者空指针的问题。#48370
- LEAD/LAG 函数支持了 DOUBLE 类型。#47940
- 修复了 `case when` 条件超过 256 个时，查询报错的问题。#47179
- 修复了 `str_to_date` 函数在空格的时候，结果错误的问题。#48920
- 修复了`split_part` 函数在常量折叠时遇到 `||` ，结果错误的问题。#48910
- 修复了 `log` 函数结果错误的问题。#47228
- 修复了 `array` / `map` 函数在 lambda 表达式中使用时导致的 core 的问题。#49140

8.4.3.4.6 存储管理

- 修复了导入聚合表时，可能的内存写脏问题。#47523
- 修复内存紧张时 MoW 导入偶发 coredump 问题。#47715
- 修复 MoW 在 BE 重启和 Schema Change 时可能出现重复 key 的问题。#48056 #48775

- 修复 Group Commit 和全局打开列更新以及 memtable 前移时的问题。#48120 #47968

8.4.3.4.7 权限管理

- 使用 LDAP 时不再会抛出 PartialResultException 异常。#47858

8.4.4 Release 2.1.8

亲爱的社区小伙伴们，Apache Doris 2.1.8 版本已于 2025 年 01 月 24 日正式发布。 该版本持续在湖仓一体、异步物化视图、查询优化器与执行引擎、存储管理等方面进行改进提升与问题修复，进一步加强系统的性能和稳定性，欢迎大家下载体验。

- 立即下载
- GitHub 下载

8.4.4.1 行为变更

- 当通过 External Catalog 查询表名大小写不敏感的数据源（如 Hive）时，在之前版本中，可以使用任意大小写进行表名查询，但是在 2.1.8 版本中，将严格遵循 Doris 自身的表名大小写敏感策略。
- 添加环境变量 SKIP_CHECK_ULIMIT 以跳过 BE 进程内关于 ulimit 值校验检查，仅适用于 Docker 快速启动场景中应用。#45267
- 添加 enable_cooldown_replica_affinity session 变量控制冷热分层下查询选用副本亲和性
- FE 添加配置restore_job_compressed_serialization 和 backup_job_compressed_serialization 用于解决 db tablet 数量非常大情况下备份和恢复操作时 FE OOM 的问题，默认关闭，打开之后无法降级

8.4.4.2 新功能

- 查询执行引擎：Arrowflight 协议支持通过负载均衡设备访问 BE。#43281
- 其他：当前 Lambda 表达式支持捕获外部的列。#45186

8.4.4.3 改进提升

8.4.4.3.1 湖仓一体

- Hudi 版本更新至 0.15，并且优化了 Hudi 表的查询规划性能。
- 优化了 MaxCompute 分区表的读取性能。#45148
- 支持会话变量 enable_text_validate_utf8，可以忽略 CSV 格式中的 UTF8 编码检测。#45537
- 优化在高过滤率情况下，Parquet 文件延迟物化的性能。#46183

8.4.4.3.2 异步物化视图

- 现在支持手动刷新异步物化视图中不存在的分区。#45290
- 优化了透明改写规划的性能。#44786

8.4.4.3.3 查询优化器

- 提升了 Runtime Filter 的自适应能力。#42640
- 增加了在 MAX / MIN 聚合函数列上的过滤条件生成原始列过滤条件的能力。#39252
- 增加了在连接谓词上抽取单测过滤条件的能力。#38479
- 优化了谓词推导在集合算子上的能力，可以更好的生成过滤谓词。#39450
- 优化了统计信息收集和使用的异常处理能力，避免在收集异常时产生非预期的执行计划。#43009 #43776 #43865 #42104 #42399 #41729

8.4.4.3.4 查询执行引擎

- Resource group 支持在当前 group 不可用的时候，降级到别的 Group. #44255
- 优化带 limit 的查询执行使其能够更快的结束，避免多余的数据扫描。#45222

8.4.4.3.5 存储管理

- CCR 支持了更加全面的操作，比如 Rename Table，Rename Column，Modify Comment，Drop View，Drop Rollup 等。
- 提升了 Broker Load 导入进度的准确性和多个压缩文件导入时的性能。
- 改进了 Routine Load 超时策略、线程池使用以防止 Routine Load 超时失败和影响查询。

8.4.4.3.6 其他

- Docker 快速启动镜像支持不设置环境参数直接启动，添加环境变量 `SKIP_CHECK_ULIMIT` 以跳过 `start_be.sh` 脚本以及 BE 进程内关于 `swap`、`max_map_count`、`ulimit` 相关校验检查，仅适用于 Docker 快速启动场景中应用。#45269
- 新增 LDAP 配置型 `ldap_group_filter` 用于自定义 Group 过滤。#43292
- 优化了使用 Ranger 时的性能。#41207
- 修复审计日志中，`scan bytes` 统计不准的问题。#45167
- 在 COLUMNS 系统表中能够正确显示列的默认值。#44849
- 在 VIEWS 系统表中能够正确显示视图的定义。#45857
- 当前，admin 用户不能被删除。#44751

8.4.4.4 Bug 修复

8.4.4.4.1 湖仓一体

- Hive
- 修复无法查询 Spark 创建的 Hive 视图的问题。#43553
- 修复无法正确读取某些 Hive Transaction 表的问题。#45753
- 修复 Hive 表分区存在特殊字符时，无法进行正确分区裁剪的问题。#42906
- Iceberg
- 修复在 Kerberos 认证环境下，无法创建 Iceberg 表的问题。#43445
- 修复某些情况下，Iceberg 表存在 dangling delete 情况下，`count(*)` 查询不准确的问题。#44039
- 修复某些情况下，Iceberg 表列名不匹配导致查询错误的问题#44470
- 修复某些情况下，当 Iceberg 表分区被修改后，无法读取的问题#45367
- Paimon
- 修复 Paimon Catalog 无法访问阿里云 OSS-HDFS 的问题。#42585
- Hudi
- 修复某些情况下，Hudi 表分区裁剪失效的问题。#44669
- JDBC

- 修复某些情况下，开始表名大小写不敏感功能后，使用 JDBC Catalog 无法获取表的问题。
- MaxCompute
- 修复某些情况下，MaxCompute 表分区裁剪失效的问题。#44508
- 其他
- 修复某些情况下，Export 任务导致 FE 内存泄露的问题。#44019
- 修复某些情况下，无法使用 HTTPS 协议访问 S3 对象存储的问题。#44242
- 修复某些情况下，Kerberos 认证票据无法自动刷新的问题。#44916
- 修复某些情况下，读取 Hadoop Block 压缩格式文件出错的问题。#45289
- 查询 ORC 格式的数据时，不再下推 CHAR 类型的谓词，以避免可能的结果错误。#45484

8.4.4.4.2 异步物化视图

- 修复了当物化视图定义中存在 CTE 时，无法刷新的问题。#44857
- 修复了当基表增加列后，异步物化视图不能命中透明改写的问题。#44867
- 修复了当查询中在不同位置包含相同的过滤谓词时，透明改写失败的问题。#44575
- 修复了当过滤谓词或连接谓词中使用列的别名时，无法透明改写的问题。#44779

8.4.4.4.3 索引

- 修复倒排索引 Compaction 异常处理的问题 #45773
- 修复倒排索引构建因为等锁超时失败的问题 #43589
- 修复异常情况下倒排索引写入 Crash 的问题。#46075
- 修复 Match 函数特殊参数时空指针的问题 #45774
- 修复 VARIANT 倒排索引相关的问题，禁用 VARIANT 使用索引 v1 格式。#43971 #45179
- 修复 NGram Bloomfilter Index 设置 `gram_size = 65535` 时 Crash 的问题。#43654
- 修复 Bloomfilter Index 计算 DATE 和 DATETIME 不对的问题。#43622
- 修复 Drop Coloumn 没有自动 Drop Bloomfilter Index 的问题。#44478
- 减少 Bloomfilter Index 写入时的内存占用。#46047

8.4.4.4.4 半结构化数据类型

- 优化内存占用，降低 VARIANT 数据类型的内存消耗。#43349 #44585 #45734
- 优化 VARIANT Schema Copy 性能。#45731
- 自动推断 Tablet Key 时不将 VARIANT 作为 Key。#44736
- 修复 VARIANT 从 NOT NULL 改成 NULL 的问题。#45734
- 修复 Lambda 函数类型推断错误的问题。#45798
- 修复 `ipv6_cidr_to_range` 函数边界条件 Coredump。#46252

8.4.4.4.5 查询优化器

- 修复了潜在的表读锁互斥导致的死锁问题，并优化了锁的使用逻辑#45045 #43376 #44164 #44967 #45995
- 修复了 SQL Cache 功能错误的使用常量折叠导致在使用包含时间格式的函数时结果不正确的问题。#44631
- 修复了比较表达式优化，在边缘情况下可能优化错误，导致结果不正确的问题。#44054 #44725 #44922 #45735 #45868
- 修复高并发点查审计日志不正确的问题。#43345#44588
- 修复高并发点查遇到异常后持续报错的问题。#44582
- 修复部分字段 Prepared Statement 不正确的问题。#45732

8.4.4.4.6 查询执行引擎

- 修复了正则表达式和 LIKE 函数在特殊字符时结果不对的问题。#44547
- 修复 SQL Cache 在切换 DB 的时候结果可能不对的问题。#44782
- 修复`cut_ipv6` 函数结果不对的问题。#43921
- 修复数值类型到 bool 类型 cast 的问题。#46275
- 修复了一系列 Arrow Flight 相关的问题。#45661 #45023 #43960 #43929
- 修复了当 hashjoin 的 hash 表超过 4G 时，部分情况结果错误的问题。#46461
- 修复了 convert_to 函数在中文字符时溢出的问题。#46505

8.4.4.4.7 存储管理

- 修复高并发 DDL 可能导致 FE 启动失败的问题。
- 修复自增列可能出现重复值的问题。
- 修复扩容时 Routine Load 不能使用新扩容 BE 的问题。

8.4.4.4.8 权限管理

- 修复使用 Ranger 作为鉴权插件时，频繁访问 Ranger 服务的问题#45645

8.4.4.4.9 Others

- 修复 BE 端开启 `enable_jvm_monitor=true` 后可能导致的内存泄露问题。#44311

8.4.5 Release 2.1.7

亲爱的社区小伙伴们，Apache Doris 2.1.7 版本已于 2024 年 11 月 10 日正式发布。2.1.7 版本持续升级改进，同时在湖仓一体、异步物化视图、半结构化数据管理、查询优化器、执行引擎、存储管理、以及权限管理等方面完成了若干修复。欢迎大家下载使用。

- 立即下载
- GitHub 下载

8.4.5.1 行为变更

- 以下全局变量会被强制设置到下列默认值
- enable_nereids_dml: true
- enable_nereids_dml_with_pipeline: true
- enable_nereids_planner: true
- enable_fallback_to_original_planner: true
- enable_pipeline_x_engine: true
- 审计日志增加了新的列 #42262
- 更多信息，请参考管理指南

8.4.5.2 新功能

8.4.5.2.1 异步物化视图

- 异步物化视图增加了一个属性 use_for_rewrite 用于控制是否参与透明改写 #40332

8.4.5.2.2 查询执行引擎

- 在 Profile 中输出变更的 session variable 列表。#41016
- 增加了`trim_in`、`ltrim_in` 和 `rtrim_in` 函数的支持。#42641
- 增加了一些 URL 函数，包括对 `to``p_level_domain`、`first_significant_subdomain` 、`cut_to_first_significant_subdomain` 支持。#42916
- 增加了 `bit_set` 函数。#42099
- 增加了`count_substrings` 函数。#42055
- 增加 `translate` 和 `url_encode` 函数。#41051
- 增加 `normal_cdf`, `to_iso8601`, `from_iso8601_date` 函数。#40695

8.4.5.2.3 存储管理

- 增加了 `information_schema.table_options` 和 `information_schema.``table_properties` 系统表，支持查询建表时设置的一些属性。#34384
- 更多信息，请参考系统表：
  - table_options
  - table_properties
- 支持 `bitmap_empty` 作为默认值。#40364
- 增加了一个新的 Session 变量`require_sequence_in_insert` 来控制向 Unique Key 表进行`insert into select` 写入时，是否必须提供 Sequence 列。#41655

8.4.5.2.4 其他

允许在 BE WebUI 页面生成火焰图。#41044

8.4.5.3 改进提升

8.4.5.3.1 湖仓一体

- 支持写入数据到 Hive Text 格式表。#40537
- 更多信息，请参考使用 Hive 构建数据湖文档
- 使用 MaxCompute Open Storage API 访问 MaxCompute 数据。#41610
- 更多信息，请参考MaxCompute 文档
- 支持 Paimon DLF Catalog。#41694
- 更多信息，请参考 Paimon Catalog 文档
- 新增语法 `table$partitions` 语法支持直接查询 Hive 分区信息 #41230
- 更多信息，请参考通过 Hive 分析数据湖文档
- 支持 brotli 压缩格式的 Parquet 文件读取。#42162
- 支持读取 Parquet 文件中的 DECIMAL 256 类型。#42241
- 支持读取 OpenCsvSerde 格式的 Hive 表。#42939

8.4.5.3.2 异步物化视图

- 细化了异步物化视图中构建时锁持有的粒度。#40402 #41010

8.4.5.3.3 查询优化器

- 优化了极端情况下统计信息收集和使用的准确性，以提升规划稳定性。#40457
- 现在可以在更多情况下生成 Runtime Filter，以提升查询性能。#40815
- 提升数值，日期和字符串函数的常量折叠能力，以提升查询性能。#40820
- 优化了列裁剪的算法，以提升查询性能。#41548

8.4.5.3.4 查询执行引擎

- 支持并行的 Prepare 降低短查询的耗时。#40270
- 修正了 Profile 中一些 Counter 的名字，保持跟审计日志一致。#41993
- 增加了新的 Local Shuffle 规则，使得部分查询更快。#40637

8.4.5.3.5 存储管理

- Show Partitions 命令支持显示 Commit Version。#28274
- 建表时检查不合理的 Partition EXPR。#40158
- 优化 Routine Load EOF 时的调度逻辑。#40509
- Routine Load 感知 Schema 变化。#40508
- 优化 Routine Load Task 超时逻辑。#41135

8.4.5.3.6 其他

- 支持通过 BE 配置关闭 BRPC 的内置服务端口。#41047
- 修复审计日志缺失字段以及重复记录的问题。#41047

8.4.5.4 Bug 修复

8.4.5.4.1 湖仓一体

- 修复了 INSERT OVERWRITE 的行为跟 Hive 不一致的问题。#39840
- 清理临时创建的文件夹，解决 HDFS 上空文件夹太多的问题。#40424
- 修复某些情况下，使用 JDBC Catalog 导致 FE 内存泄露的问题。#40923
- 修复某些情况下，使用 JDBC Catalog 导致 BE 内存泄露的问题。#41266
- 修复某些情况下，读取 Snappy 压缩格式错误的问题。#40862
- 修复某些情况下，FE 端 FileSystem 可能泄露的问题。#41108
- 修复某些情况下，通过 EXPLAIN VERBOSE 查看外表执行计划可能导致空指针的问题。#41231
- 修复无法读取 Paimon parquet 格式表的问题。#41487
- 修复 JDBC Oracle Catalog 兼容性改动引入的性能问题。#41407
- 禁止下推隐式转换后的谓词条件已解决 JDBC Catalog 某些情况下查询结果不正确的问题。#42242
- 修复 External Catalog 中表名大小写访问异常的一些问题。#42261

8.4.5.4.2 异步物化视图

- 修复用户指定的 Start Time 不生效的问题。#39573
- 修复嵌套物化视图不刷新的问题。#40433
- 修复删除重建基表后，物化视图可能不刷新的问题。#41762
- 修复分区补偿改写可能导致结果错误的问题。#40803
- 当 `sql_select_limit` 设置时，改写结果可能错误的问题。#40106

8.4.5.4.3 半结构化管理

- 修复了索引文件句柄泄露的问题。#41915
- 修复了特殊情况下倒排索引 `count()` 不准确的问题。#41127
- 修复了未开启 Light Schema Change 时 Variant 异常的问题。#40908
- 修复了 Variant 返回数组时内存泄漏的问题。#41339

8.4.5.4.4 查询优化器

- 修正了外表查询时，可能存在过滤条件 nullable 计算错误，导致执行异常的问题。#41014
- 修复范围比较表达式优化可能发生错误的问题。#41356

8.4.5.4.5 查询执行引擎

- `match_regexp` 函数不能正确处理空字符串的问题。#39503
- 解决在高并发场景下，Scanner 线程池卡死的问题。#40495
- 修复了 `data_floor` 函数结果错误的问题。#41948
- 修复了部分场景下，Cancel 消息不正确的问题。#41798
- 修复 Arrow Flight 打印太多的 Warn 日志的问题。[#41770] (https://github.com/apache/doris/pull/41770)
- 解决部分场景下 Runtime Filter 发送失败的问题。#41698
- 修复了一些系统表查询的时候不能正常结束或者卡住的问题。#41592
- 修复了窗口函数结果不正确的问题。#40761
- 修复 ENCRYPT 和 DECRYPT 函数导致 BE Core 的问题。#40726
- 修复 CONV 函数结果错误的问题。#40530

8.4.5.4.6 存储管理

- Memtable 前移在多副本情况下，有机器宕机时导入失败的问题。#38003
- 导入过程中，Memtable 在 Flush 阶段时，统计的内存不准确。#39536
- 修复 Memtable 前移多副本容错的问题。#40477
- 修复 Memtable 前移 bvar 统计不准的问题。#40985
- 修复 s3 Load 进度汇报不准的问题。#40987

8.4.5.4.7 权限管理

- 修复了 SHOW COLUMNS, SHOW SYNC, SHOW DATA FROM DB.TABLE 相关的权限问题。#39726

8.4.5.4.8 Others

- 修复 2.0 版本的审计日志插件在 2.1 版本无法使用的问题#41400

8.4.6 Release 2.1.6

亲爱的社区小伙伴们，Apache Doris 2.1.6 版本已于 2024 年 9 月 10 日正式发布。2.1.6 版本在湖仓一体、异步物化视图、半结构化数据管理持续升级改进，同时在查询优化器、执行引擎、存储管理、数据导入与导出以及权限管理等方面完成了若干修复。欢迎大家下载使用。

- 官网下载：https://doris.apache.org/download
- GitHub 下载：https://github.com/apache/doris/releases/tag/2.1.6-rc04

8.4.6.1 行为变更

- 移除 `create repository` 命令中的 `delete_if_exists` 选项。#38192
- 新增会话变量 `enable_prepared_stmt_audit_log`，用于控制 JDBC 预编译语句是否记录审计日志，默认不记录。#38624 #39009

- 采用文件描述符限制和内存限制来管理 Segment Cache。 #39689
- 当 sys_log_mode 配置项设置为 BRIEF 时，在日志中增加文件位置信息，以提供更详细的上下文。#39571
- 将会话变量 max_allowed_packet 的默认值调整为 16MB，提高数据传输限制。#38697
- 在单次请求中，若包含多个 SQL 语句，各语句间必须使用分号进行分隔，以增强语句的清晰度和执行效率。#38670
- 现在支持 SQL 语句以分号开始，提供更灵活的语句书写方式。#39399
- 在执行如 show create table 等语句时，类型格式与 MySQL 保持一致，提升与 MySQL 的兼容性。#38012
- 当新优化器规划查询超时后，不再回退到旧优化器，以避免潜在的性能下降问题。#39499

8.4.6.2 新功能

8.4.6.2.1 Lakehouse

- 实现 Iceberg 表的写回功能。
- 更多信息，请查看文档数据湖构建 -Iceberg
- 增强 SQL 拦截规则，支持对外表的拦截处理。
- 更多信息，请查看文档查询管理 -SQL 拦截
- 新增系统表file_cache_statistics，用于查看 BE 节点的数据缓存性能指标。
- 更多信息，请查看文档系统表 -file_cache_statistics

8.4.6.2.2 异步物化视图

- 支持在 Insert 中进行透明改写。#38115
- 支持对查询中存在 VARIANT 类型时的透明改写。#37929

8.4.6.2.3 半结构化数据管理

- 支持 ARRAY MAP 类型到 JSON 类型的 CAST 转换功能。#36548
- 引入json_keys函数，用于提取 JSON 中的键名。#36411
- 支持在导入 JSON 时指定json path$ “#38213
- ARRAY / MAP / STRUCT 类型支持replace_if_not_null#38304
- 允许调整 ARRAY / MAP / STRUCT 类型的列顺序。#39210
- 新增multi_match函数，支持在多个字段中匹配关键词，并利用倒排索引加速查询。#37722

8.4.6.2.4 查询优化器

- 完善 MySQL 协议返回列的信息，包括原始数据库名、表名、列名和别名。#38126
- 增强聚合函数group_concat，支持同时使用order by和distinct进行复杂数据聚合。#38080
- 改进了 SQL 缓存机制，支持通过注释区分不同的查询以复用缓存结果。#40049
- 增强分区裁剪功能，支持在过滤条件中使用date_trunc和date函数。#38025 #38743
- 允许在表别名前使用数据库名作为限定名前缀。#38640
- 支持 Hint 格式注释。#39113

8.4.6.2.5 执行引擎

- Group concat函数现支持distinct和order by选项。#38744

8.4.6.2.6 Others

- 新增系统表table_properties，便于用户查看和管理表的各项属性。
- 更多信息，请查看文档 table_properties
- 新增 FE 中死锁和慢锁检测功能。
- 更多信息，请查看文档FE 锁管理

8.4.6.3 改进提升

8.4.6.3.1 湖仓一体

- 革新外表元数据缓存机制。
- 更多信息，请查看文档元数据缓存
- 新增会话变量keep_carriage_return，默认关闭。读取 Hive Text 格式表时，默认将\\r\\n与\n均视为换行符。#38099
- 优化 Parquet / ORC 文件读写内存统计。#37257
- Paimon 表支持 IN/ NOT IN 谓词下推。#38390
- 升级优化器，支持 Hudi 表的 Time Travel 语法。#38591
- Kerberos 认证流程优化，提升安全认证效率与稳定性。#37301
- 支持 Rename column 操作后读取 Hive 表。#38809
- 提升外表分区列读取性能。#38810

- 优化外表查询规划，优化数据分片合并策略，有效避免小分片对查询性能的影响。#38964
- SHOW CREATE DATABASE / TABLE 新增 Location 等属性展示。#39644
- MaxCompute Catalog 扩展支持复杂类型。#39822
- 优化文件缓存加载策略，通过异步加载方式避免 BE 启动时间过长的问题。#39036
- 升级文件缓存淘汰策略，有效管理长时间占用锁的资源。#39721

8.4.6.3.2 异步物化视图

- 支持小时、周及季度级别的分区上卷构建。#37678
- 基于 Hive 外表的物化视图，在刷新前自动更新元数据缓存，以保证每次刷新可以获取最新数据。#38212
- 通过批量获取元数据，优化存算分离模式下的透明改写规划性能。#39301
- 通过禁止重复枚举，进一步提升透明改写的规划性能。#39541
- 优化基于 Hive 外表分区刷新物化视图的透明改写性能。#38525

8.4.6.3.3 半结构化数据管理

- 优化 TOPN 查询内存分配，显著提升查询性能。#37429
- 优化倒排索引字符串处理性能。#37395
- 优化倒排索引在 MOW 表中的性能。#37428
- 建表时支持指定行存 `page_size`，以控制压缩效果。#37145

8.4.6.3.4 查询优化器

- 调整 Mark Join 行数估计算法，提高基数估算准确性。#38270
- 优化 Semi / Anti Join 代价估计算法，能够正确选择最佳 Join 顺序。#37951
- 调整部分列无统计信息情况下的过滤估计算法，使估算更精准。#39592
- 改进 Set Operation 算子 Instance 计算逻辑，防止在极端情况下并行度不足的问题。#39999
- 优化 Bucket Shuffle 使用策略，数据打散不充分时也能获得更好的性能。#36784
- 窗口函数数据提前过滤，支持单投影中存在多窗口函数的情况。#38393
- 过滤条件含 `NullLiteral` 时，智能折叠为 False，转换为 `EmptySet`，减少不必要的数据扫描量。#38135
- 扩大谓词推导适用范围，在特定模式的查询下能够大幅减少数据扫描量。#37314
- 在分区裁剪中支持部分短路计算逻辑，以提升分区裁剪性能。在特定场景下，性能提升超过 100%。#38191
- 在用户变量中，支持计算任意的标量函数。#39144
- 当查询中存在别名冲突时，报错信息能够保持与 MySQL 一致。#38104

8.4.6.3.5 执行引擎

- 实现 AggState 从 2.1 到 3.x 版本的兼容，并解决了 coredump 问题。#37104
- 重构无 Join 操作时的 Local Shuffle 策略选择机制。#37282
- 将内部表查询的 scanner 调整为异步模式，以防止查询内部表时出现卡顿。#38403
- 优化 Join 算子在构建 Hash 表时的 Block Merge 流程。#37471
- 缩短 MultiCast 持有锁的时间。#37462
- 优化 gRPC 的 keepAliveTime 设置并增加了链接监测机制，降低了因 RPC 错误导致的查询失败率。#37304
- 当内存超出限制时，将清理 `jemalloc` 中的所有 Dirty Pages。#37164
- 提升 `aes_encrypt/decrypt` 函数对常量类型的处理效率。#37194
- 加快 `json_extract` 函数对常量数据的处理速度。#36927
- 提高 `ParseUrl` 函数处理常量数据的性能。#36882

8.4.6.3.6 存储管理

备份恢复 / 跨集群同步

- Restore 功能现已支持删除多余的 Tablet 和分区选项。#39363
- 在创建 Repository 时，支持检查存储连通性。#39538
- Binlog 支持 Drop 表操作，使 CCR 能够支持 Drop 表的增量同步。#38541

Compaction

- 改进高优 Compaction 任务不受并发控制限制的问题。#38189
- 根据数据特性自动调整 Compaction 的内存消耗。#37486
- 修复顺序数据优化策略可能引发的聚合表或 MOR UNIQUE 表数据准确性问题。#38299
- 优化补副本期间 Compaction 选择 rowset 的策略，以避免触发 -235 错误。#39262

Merge-on-Write

- 解决了列更新和 Compaction 并发时列更新慢的问题。#38682
- 修复一次导入大量数据时，Segcompaction 可能导致 MOW 数据不正确的问题。#38992 #39707
- 解决 BE 重启后，可能导致列更新数据丢失的问题。#39035

其他

- 增加了 FE 配置，用于控制冷热分层下查询是否优先访问本地数据的副本。#38322
- 解决了过期的 BE 汇报消息未包含新创建 Tablet 的问题。#38839#39605
- 优化副本调度优先级策略，优先调度缺少数据的副本。#38884
- 对于有未完成 ALTER JOB 的 Tablet，不进行均衡调度。#39202
- List 分区方式的表现支持修改分桶数。#39688
- 优先选择在线的磁盘服务进行查询。#39654
- 改进了同步物化视图的 Base 表不支持删除时的提示信息。#39857
- 改进了单列超过 4G 时的报错信息。#39897
- 修复了 Insert 语句遇到 Plan 错误时未正确中止事务的问题。#38260
- 修复了 SSL 链接关闭时的异常问题。#38677
- 修复了使用 Label 中止事务时未持有表锁的问题。#38842
- 修复了 Gson Pretty 导致 Image 过大的问题。#39135
- 修复了 CREAT TABLE 语句在新优化器下未检查 Bucket 为 0 的问题。#38999
- 修复了 DELETE 条件谓词中包含中文列时报错的问题。#39500
- 修复了分区均衡模式下频繁均衡 Tablet 的问题。#39606
- 修复了分区丢失 Storage Policy 属性的问题。#39677
- 修复了事务内导入多个表时统计信息不正确的问题。#39548
- 修复了 Random 分桶表删除时报错的问题。#39830
- 修复了 UDF 不存在导致 FE 无法启动的问题。#39868
- 修复了 FE 主从 Last Failed Version 不一致的问题。#39947
- 修复了 Schema Change Job 被取消时，相关 Tablet 可能仍处于 Schema Change 状态的问题。#39327
- 修复了单个语句修改类型和列顺序 SC 时出现的报错问题。#39107

8.4.6.3.7 数据导入

- 改进了导入发生 -238 错误时的错误信息提示。#39182
- 实现在 Restore 分区时，其他分区可以同时进行导入。#39915
- 优化了 Group Commit FE 选择 BE 的策略。#37830 #39010
- 对于一些常见的 Stream Load 错误信息，避免了程序栈的打印，简化了错误处理。#38418
- 改进下线的 BE 可能影响导入出错的问题。#38256

8.4.6.3.8 权限管理

- 优化了开启 Ranger 鉴权插件后的访问性能。#38575
- 优化了 Refresh Catalog / Database / Table 操作的权限策略，用户仅需 SHOW 权限即可执行此操作。#39008

8.4.6.4 Bug 修复

8.4.6.4.1 湖仓一体

- 修复切换 Catalog 时可能出现的数据库找不到问题。#38114
- 解决了读取 S3 上不存在的数据时出现的异常报错。#38253
- 修正导出操作时，指定异常路径可能导致导出位置异常的问题。#38602
- 修复 Paimon 表时间列时区问题。#37716
- 临时关闭 Parquet PageIndex 功能以避免部分错误行为。
- 修复外表查询时，错误选取黑名单中 Backend 节点的问题。#38984
- 解决读取 Parquet Struct 列类型中缺失子列导致查询错误的问题。#39192
- 修复 JDBC Catalog 的谓词下推问题。#39082
- 修正 Parquet 格式读取时，历史格式导致查询结果错误的问题。#39375
- 增强了 Oracle JDBC Catalog 对 OJDBC6 驱动的兼容性。#39408
- 解决了 Refresh Catalog/Database/Table 操作可能导致的 FE 内存泄漏问题。#39186 #39871
- 修复了 JDBC Catalog 在某些情况下的线程泄漏问题。#39666#39582
- 修复开启 Hive Metastore 事件订阅后，可能出现事件处理失败的问题。#39239
- 禁止读取自定义 Escape CHAR 和 NULL Format 的 Hive Text 格式表，防止数据错误。#39869
- 修复某些情况下，无法访问通过 Iceberg API 创建的 Iceberg 表的问题。#39203
- 修复无法读取存储在开启高可用的 HDFS 集群上的 Paimon 表的问题。#39876
- 修复开启文件缓存后，读取 Paimon 表 Deletion Vector 可能导致错误的问题。#39875
- 修复某些情况下读取 Parquet 可能导致死锁的问题 #39945

8.4.6.4.2 异步物化视图

- 修复无法在 Follower FE 上使用 `show create materialized view` 命令的问题。#38794
- 统一异步物化视图在元数据中的对象类型，使其在数据工具中正常显示。#38797
- 修复嵌套异步物化视图总是进行全量刷新的问题。#38698
- 修正 Cancel 任务在重启 FE 后状态可能显示为 running 的问题。#39424
- 修复错误使用上下文，导致刷新物化视图任务可能非预期失败的问题。#39690
- 修复基于外表创建异步物化视图时，VARCHAR 类型因长度不合理导致写入失败的问题。#37668
- 修复 FE 重启或 Catalog 重建后，基于外表的异步物化视图可能失效的问题。#39355
- 禁止 List 分区的物化视图使用分区上卷，以防止生成错误数据。#38124
- 修复在聚合上卷透明改写时，SELECT List 中存在字面量导致的结果错误问题。#38958
- 修复当查询中存在形如`a = a`的过滤条件时，透明改写可能出错的问题。#39629
- 修复透明改写直查外表无法成功的问题。#39041

8.4.6.4.3 半结构化数据管理

- 删除老优化器上 `PreparedStatement` 的支持。#39465
- 修复 JSON 转义字符处理的问题。#37251
- 修复 JSON 字段重复处理的问题。#38490
- 修复部分 ARRAY MAP 函数的问题。#39307 #39699 #39757
- 修复倒排索引查询和 LIKE 查询复杂组合的问题。#36687

8.4.6.4.4 查询优化器

- 修复分区过滤条件中存在 or 时，可能导致分区裁剪错误的问题。#38897
- 修复存在复杂表达式时，可能导致的分区裁剪错误的问题。#39298
- 修复 AGG_STATE 类型中的子类型，Nullable 可能规划不正确导致执行报错的问题。#37489
- 修复 Set Operation 算子 Nullable 可能规划不正确，导致执行报错的问题。#39109
- 修复 Intersect 算子执行优先级不正确的问题。#39095
- 修复当查询中存在最大合法日期字面量时，可能出现 NPE 的问题。#39482
- 修复偶现的规划报错，导致的执行时报错 Slot 不合法的问题。#39640
- 修复重复引用 CTE 中的列，可能导致结果缺少部分列数据的问题。#39850

- 修复在查询中存在 CASE WHEN 时，偶现的规划报错问题。#38491
- 修复不能将 IP 类型隐式转换为 STRING 类型的问题。#39318
- 修复在使用多维聚合时，当 SELECT List 中存在相同列和其别名时，可能出现的规划报错问题。#38166
- 修复使用 BE 常量折叠时，处理 BOOLEAN 类型可能不正确的问题。#39019
- 修复在表达式中存在 `default_cluster:` 作为 Database 名称前缀导致的规划报错问题。#39114
- 修复 Insert Into 可能导致的死锁问题。#38660
- 修复没有在规划全过程持有表锁导致可能出现规划报错的问题。#38950
- 修复创建表时不能正确处理 CHAR(0), VARCHAR(0) 的问题。#38427
- 修复 SHOW CREAT TABLE 可能错误的显示出隐藏列的问题。#38796
- 修复创建表时没有禁止使用和隐藏列同名列的问题。#38796
- 修复在执行 INSERT INTO AS SELECT 时，如果存在 CTE，偶现的规划报错问题。#38526
- 修复 INSERT INTO VALUES 无法自动填充 NULL 默认值的问题。#39122
- 修复在 DELETE 中使用 CTE，但是没有使用 USING 时，导致的 NPE 问题。#39379
- 修复对随机分布的聚合模型表执行删除操作会失败的问题。#37985

8.4.6.4.5 执行引擎

- 修复多个场景下，Pipeline 执行引擎被卡顿，导致查询不结束的问题。#38657 #38206 #38885
- 修复了 NULL 和非 NULL 列在差集计算时导致的 Coredump 问题。#38737
- 修复了 `width_bucket` 函数结果错误的问题。#37892
- 修复了当单行数据很大且返回结果集也很大时（超过 2GB）查询报错的问题。#37990
- 修复了 `stddev` 在 `DecimalV2` 类型下结果错误的问题。#38731
- 修复了 `MULTI_MATCH_ANY` 函数导致的 Coredump 问题。#37959
- 修复了 INSERT OVERWRITE AUTO PARTITION 导致事务回滚的问题。#38103
- 修复了 `convert_tz` 函数结果错误的问题。#37358 #38764
- 修复了 `collect_set` 函数结合窗口函数使用时 Coredump 的问题。#38234
- 修复了 `mod` 函数在异常输入时导致的 Coredump 问题。#37999
- 修复了多线程下执行相同表达式可能导致 Java UDF 结果错误的问题。#38612
- 修复了 `conv` 函数返回类型错误导致的溢出问题。#38001
- 修复了 `histogram` 函数结果不稳定的问题。#38608

8.4.6.4.6 存储管理

- 修复备份恢复后，写入数据时可能出现不可读的问题。#38343
- 修复跨版本 Restore Version 使用问题。#38396
- 修复 Backup 失败时 Job 没有取消的问题。#38993
- 修复 2.1.4 升级到 2.1.5 CCR 报 NPE，导致 FE 不能启动的问题。#39910
- 修复 Restore 之后视图和物化视图不能使用的问题。#38072 #39848

8.4.6.4.7 数据导入

Routine Load

- 修复 Routine Load 一流多表可能得内存泄露的问题。#38824
- 修复 Routine Load 包围符和转义符不生效的问题。#38825
- 修复 Routine Load 任务名包含大写字母时 `show routineload` 结果不正确的问题。#38826
- 修复改变 Routine Load Topic 时没有重置 Offset Cache 的问题。#38474
- 修复并发情况下 `show routineload` 可能触发异常的问题。#39525
- 修复 Routine Load 可能重复导入数据的问题。#39526

Group Commit

- 修复 JDBC 方式下打开 Group Commit 时 setNull 导致的数据报错问题 #38276
- 修复打开 `group commit insert` 发往非 Master FE 时可能导致 NPE 问题 #38345
- 修复 Group Commit 内部写数据错误处理不正确的问题。#38997
- 修复 Group Commit 执行规划失败时可能触发的 Coredump。#39396

其它

- 修复并发导入 Auto Partition 表可能报 Tablet 不存在的问题。#38793
- 修复可能的 Load Stream 泄露问题。#39039
- 修复 INSERT INTO SELECT 没有数据时开启事务的问题。#39108
- 使用 Memtable 前移时忽略单副本导入的配置。#39154
- 修复后台导入 `stream load record` 遇见 Database 删除时异常中止的问题。#39527
- 修复 Strict Mode 模式下，出现数据错误时错误信息提示不准确的问题。#39587
- 修复 Stream Load 遇见错误数据不返回 Error URL 的问题。#38417
- 修复 Insert Overwrite 和 Auto Partition 配合使用的问题。#38442
- 修复 CSV 遇到行分隔符被包围符包围数据时解析错误的问题。#38445

8.4.6.4.8 数据导出

- 修复导出操作中指定 `delete_existing_files` 属性后，可能会重复删除导出数据的问题。#39304

8.4.6.4.9 权限管理

- 修复创建物化视图时，错误地要求拥有 ALTER TABLE 的权限的问题。#38011
- 修复 `show routine load` 时，Database 显式为空的问题。#38365
- 修复 `create table like` 错误的要求拥有对原表的创建权限的问题。#37879
- 修复赋权操作没有检查对象是否存在的问题。#39597

8.4.6.5 版本升级说明

Doris 升级请遵守不要跨两个二位版本升级的原则，依次往后升级。

比如从 0.15.x 升级到 2.0.x 版本，则建议先升级至 1.1 最新版本，然后升级到最新的 1.2 版本，最后升级到最新的 2.0 版本，以此类推。

8.4.7 Release 2.1.5

亲爱的社区小伙伴们，Apache Doris 2.1.5 版本已于 2024 年 7 月 24 日正式发布。2.1.5 版本在湖仓一体、多表物化视图、半结构化数据分析等方面进行了全面更新及改进，同时在倒排索引、查询优化器、查询引擎、存储管理等 10 余方向上完成了若干问题修复，欢迎大家下载使用。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.4.7.1 行为变更

- JDBC Catalog 的默认连接池大小从 10 调整为 30。#37023
- 创建 JDBC Catalog 时，参数 `connection_pool_max_size` 的默认值改为 30，以避免高并发场景下连接池耗尽的问题。
- 将系统的保留内存的最小值，即 `low water mark` 调整为 `min (6.4G, MemTotal * 5%)`，以更好地防止 BE 出现 OOM 问题。
- 修改了单请求多个语句的处理逻辑，当客户端未设置 `CLIENT_MULTI_STATEMENTS` 标志位时，将仅返回最后一个语句的结果，而非所有语句结果。
- 不再允许直接更改异步物化视图的数据。#37129
- 增加会话变量 `use_max_length_of_varchar_in_ctas`，用于控制 CTAS 时 VARCHAR 和 CHAR 类型长度的生成行为。默认值是 true。当设置为 false 时，使用推导出的 VARCHAR 长度，而不是使用最大长度。#37284
- 统计信息收集，默认开启了通过文件大小预估 Hive 表行数的功能。#37694

- 默认开启异步物化视图透明改写机制。#35897
- 透明改写利用分区物化视图，如果分物物化视图部分分区失效，默认行为是将所有基础表与物化视图联合，以保证查询数据的正确性。#35897

8.4.7.2 新功能

8.4.7.2.1 湖仓一体

- 会话变量 `read_csv_empty_line_as_null` 用于控制在读取 CSV 格式文件时，是否忽略空行。默认情况下忽略空行，当设置为 true 时，空行将被读取为所有列均为 Null 的行。#37153
- 更多信息，请参考文档。
- 新增兼容 Presto 的复杂类型输出格式。通过设置 `set serde_dialect="presto"`，可以控制复杂类型的输出格式与 Presto 一致，用于平滑迁移 Presto 业务。#37253

8.4.7.2.2 多表物化视图

- 支持在构建物化视图中使用非确定性函数。#37651
- 支持原子替换异步物化视图定义。#37147
- 支持通过 `show create materialized view` 查看异步物化视图创建语句。#37125
- 支持对多维聚合查询的透明改写。#37436
- 支持对非聚合物化视图的聚合查询进行透明改写。#37497
- 支持使用 Key 列，对查询中的 DISTINCT 聚合做透明改写。#37651
- 支持对物化视图进行分区，通过使用 `date_trunc` 对分区进行汇总。#31812 #35562
- 支持分区表值函数（TVF）#36479

8.4.7.2.3 半结构化数据分析

- 使用 VARIANT 类型的表支持部分列更新。#34925
- 支持默认开启 PreparedStatement。#36581
- VARIANT 类型支持导出为 CSV 格式。#37857
- 支持 `explode_json_object` 函数，用于将 JSON Object 行转列。#36887
- ES Catalog 将 ES 的 NESTED 或者 OBJECT 类型映射成 Doris JSON 类型。#37101
- 默认情况下，对于具有指定分词器的倒排索引，默认开启 `support_phrase` 以提升 `match_phrase` 系列查询性能。#37949

8.4.7.2.4 查询优化器

- 支持 `explain DELETE FROM` 语句。#37100
- 支持常量表达式参数的 Hint 形式。#37988

8.4.7.2.5 内存管理

- 增加了 HTTP API 以清除缓存。#36599

8.4.7.2.6 权限管理

- 支持对表值函数（TVF）中的资源进行鉴权。#37132

8.4.7.3 改进提升

8.4.7.3.1 湖仓一体

- 将 Paimon 升级至 0.8.1 版本。
- 修复在部分情况下，查询 Paimon 表时导致 `org.apache.commons.lang.StringUtils` 的问题。#37512
- 支持腾讯云 LakeFS。#36891
- 优化了外部表查询时获取文件列表的超时时间。#36842
- 可通过会话变量 `fetch_splits_max_wait_time_ms` 进行设置
- 改进了 SQLServer JDBC Catalog 的默认连接逻辑。#36971
- 默认情况下，不干预连接加密设置。仅当 `force_sqlserver_jdbc_encrypt_false` 设置为 true 时，才会强制在 JDBC URL 中添加 `encrypt=false` 以减少认证错误，从而提供更灵活的控制加密行为的能力。
- Hive 表的 `show create table` 语句增加序列化/反序列化。#37096
- FE 端 Hive 表列表默认缓存时间由 1 天改为 4 小时
- 数据导出（Export/Outfile）支持指定 Parquet 和 ORC 的压缩格式。
- 更多信息，请参考文档。
- 当使用 CTAS+TVF 创建表时，TVF 中的分区列将被自动映射为 Varchar（65533）而非 String，以便该分区列能够作为内表的分区列使用。#37161
- 优化 Hive 写入操作元数据的访问次数。#37127
- ES Catalog 支持将 NESTED/OBJECT 类型映射到 Doris 的 JSON 类型。#37182
- 优化使用低版本 OBJECT 驱动连接 Oracle 时的报错信息。#37634
- 当 Hudi 表 Incremental Read 返回空集时，Doris 同样返回空集而非报错。#37636
- 修复部分情况下内外表关联查询可能导致 FE 超时的问题。#37757
- 修复了在从旧版本升级到新版本时，如果开启了 Hive Metastore Even Listener 情况下，可能出现 FE 元数据回放错误的问题。#37757

8.4.7.3.2 多表物化视图

- 创建异步物化视图时，支持自动选择 Key 列。#36601
- 异步物化视图分区刷新支持定义中使用 `date_trunc` 函数。#35562
- 嵌套物化视图中，当下层命中聚合上卷改写后，上层现在依然可以继续进行透明改写。#37651
- 当 Schema Change 不影响异步物化视图数据正确性时，异步物化视图保持可用状态。#37122
- 提升了透明改写的规划速度。#37935
- 计算异步物化视图可用性时，不再考虑当前的刷新状态。#36617

8.4.7.3.3 半结构化数据管理

- 通过采样优化 DESC 查看 VARIANT 子列的性能。#37217
- 行存 `page_size` 默认从 4K 调到 16K 压缩率提升 30%，而且支持表级别可配置。
- JSON 类型支持 Key 为空的特殊 JSON 数据。#36762

8.4.7.3.4 倒排索引

- 减少倒排索引 Exists 调用避免对象存储访问延迟。#36945
- 优化倒排索引查询流程额外开销。#35357
- 在物化视图中不创建倒排索引。#36869

8.4.7.3.5 查询优化器

- 当比较表达式两侧都是 Literal 时，String Literal 会尝试向另一侧的类型转换。#36921
- 重构了 VARIANT 类型的子路径下推功能，现在可以更好地支持复杂的下推场景。#36923
- 优化了物化视图代价计算的逻辑，能够更准确的选择代价更低的物化视图。#37098
- 提升了 SQL 中使用用户变量时的 SQL 缓存规划速度。#37119
- 优化了 NOT NULL 表达式的估行逻辑，当查询中存在 NOT NULL 时可以获得更好的性能。#37498
- 优化了 LIKE 表达式的 NULL 拒绝推导逻辑。#37864
- 优化查询指定分区失败时的报错信息，可以更清楚看到是哪个表导致的问题。#37280

8.4.7.3.6 查询引擎

- 将某些场景下 BITMAP_UNION 算子的性能提升了 3 倍。
- 提升 Arrow Flight 在 ARM 环境下的读取性能。
- 优化了 `explode`、`explode_map`、`explode_json` 函数的执行性能。

8.4.7.3.7 数据导入

- 支持为 `INSERT INTO ... FROM TABLE VALUE FUNCTION` 语句设置 `max_filter_ratio` 参数。
- 更多信息，请参考文档

8.4.7.4 Bug 修复

8.4.7.4.1 湖仓一体

- 修复部分情况下查询 Parquet 格式导致 BE 宕机的问题。#37086
- 修复查询 Parquet 格式，BE 端打印大量日志的问题。#37012
- 修复部分情况下 FE 端重复创建大量 FileSystem 对象的问题。#37142
- 修复部分情况下，写入 Hive 后的事务信息未清理的问题。#37172
- 修复部分情况下，Hive 表写入操作导致线程泄露的问题。#37247
- 修复部分情况下，无法正确获取 Hive Text 格式行列分隔符的问题。#37188
- 修复部分情况下，读取 lz4 压缩块时的并发问题。#37187
- 修复部分情况下，Iceberg 表 `count(*)` 返回错误的问题。#37810。
- 修复部分情况下，创建基于 MinIO 的 Paimon Catalog 导致 FE 元数据回放错误的问题。#37249
- 修复部分情况下使用 Ranger 创建 Catalog 客户端卡死的问题。#37551

8.4.7.4.2 多表物化视图

- 修复当基表增加新的分区时，可能导致的分区聚合上卷改写后结果错误的问题。#37651
- 修复关联的基表分区删除后，物化视图分区状态没有被置为不同步的问题。#36602
- 修复异步物化视图构建偶现的死锁问题。#37133
- 修复异步物化视图单次刷新大量分区时偶现的，报错 `nereids cost too much time` 问题。#37589
- 修复创建异步物化视图时，如果最终的 Select List 中存在 Null Literal，则无法创建的问题。#37281
- 修复单表物化视图，如果构建了聚合的物化视图，虽然改写成功，但是 CBO 没有选择的问题。#35721 #36058
- 修复 Join 输入都是聚合的情况下，构建分区物化视图，分区推导失败的问题。#34781

8.4.7.4.3 半结构化数据管理

- 修复 VARIANT 在并发/异常数据等特殊情况下的问题。#37976 #37839 #37794 #37674 #36997
- 修复 VARIANT 用在不支持的 SQL 中 Coredump 的问题。#37640
- 修复 1.x 版本升级到 2.x 或者更高版本时因为 MAP 数据类型 Coredump 的问题。#36937
- 修复 ES Catalog 对 Array 的支持。#36936

8.4.7.4.4 倒排索引

- 修复倒排索引 v2 DROP INDEX 元数据没有删除的问题。#37646
- 修复字符串长度超过 “ignore above” 时查询准确性问题。#37679
- 修复索引大小统计的问题。#37232 #37564

8.4.7.4.5 查询优化器

- 修复部分因为保留关键字而导致导入无法执行的问题。#35938
- 修复了在创建表时 CHAR(255) 类型错误的记录为 CHAR(1) 的问题。#37671
- 修复了在相关子查询中的连接表达式为复杂表达式时返回错误结果的问题。#37683
- 修复了 DECIMAL 类型分桶裁剪有可能错误的问题。#38013
- 修复了部分场景下开启 Pipeline Local Shuffle 后，聚合算子计算结果错误的问题。#38061
- 修复当聚合算子中存在相等的表达式时，可能出现的规划报错问题。#36622
- 修复当聚合算子中存在 Lambda 表达式时，可能出现的规划报错问题。#37285
- 修复了由窗口函数生成的字面量在优化为字面量时类型错误导致无法执行的问题。#37283
- 修复了聚合函数 `foreach combinator` 错误输出 Null 属性问题。#37980
- 修复了 acos 函数在参数为超越范围值的字面量时不能规划的问题。#37996
- 修复当查询指定的同步物化视图时，显示指定查询分区导致规划报错的问题。#36982
- 修复了在规划过程中偶尔出现 NPE 的问题。#38024

8.4.7.4.6 查询引擎

- 修复 DELETE WHERE 语句中，在 DECIMAL 数据类型作为条件报错的问题。#37801
- 修复查询执行结束，但是 BE 内存不释放的问题。#37792 #37297
- 修复在千级别 QPS 场景下，Audit Log 占用 FE 内存太多的问题。https://github.com/apache/doris/pull/37786
- 修复 sleep 函数在输入非法值时 BE Core 的问题。#37681
- 修复执行过程中 `sync filter size meet error` 的问题。#37103
- 修复执行过程中，使用时区时结果不对的问题。#37062
- 修复 `cast string` 到 `int` 时结果不对的问题。#36788
- 修复 Arrow Flight 协议在开启 Pipelinex 时查询报错的问题。#35804
- 修复 `cast string to date/datetime` 报错的问题。#35637
- 修复使用 <=> 做大表关联查询时 BE Core 的问题。#36263

8.4.7.4.7 存储管理

- 修复列更新写入时遇到 DELETE SIGN 数据不可见问题。#36755
- 优化 Schema Change 期间 FE 的内存占用。#36756
- 修复 BE 重启时事务没有 Abort 导致的 BE 下线卡住问题。#36437
- 修复 NOT-NULL 到 NULL 类型变更的偶发报错问题。#36389
- 优化 BE 宕机时的副本修复调度。#36897
- 单个 BE 创建 Tablet 时支持 round-robin 选择磁盘。#36900
- 修复 Publish 慢导致的查询 -230 错误。#36222
- 优化 Partition Balance 的速度。#36976
- 使用 FD 数目和内存控制 Segment Cache 避免 FD 不足。#37035
- 修复 Clone 和 Alter 并发可能导致的副本丢失问题。#36858
- 修复不能调整列顺序问题。#37226
- 禁止自增列的部分 Schema Change 操作。#37331
- 修复 Delete 操作报错不准确。#37374
- BE 侧 Trash 过期时间调整为一天。#37409
- 优化 Compaction 内存占用和调度。#37491
- 检查潜在的过大 Backup 导致 FE 重启的问题。#37466
- 恢复动态分区删除策略以及交叉分区的行为到 2.1.3。#37570 #37506
- 修复 DELETE 谓词重部分 DECIMAL 报错问题。#37710

8.4.7.4.8 数据导入

- 修复导入时错误处理竞争导致的数据不可见问题。#36744
- Stream Load 导入支持 `hhl_from_base64`。#36819
- 修复潜在的单表非常多 Tablet 导入失败时可能导致 FE OOM 的问题。#36944
- 修复 FE 主从切换时自增列可能重复的问题。#36961
- 修复 INSERT INTO SELECT 自增列报错问题。#37029
- 降低数据下刷线程数，优化内存占用。#37092
- 优化 Routine Load 任务自动恢复和错误信息。#37371
- 增加 Routine Load 默认攒批大小。#37388
- 修复 Routine Load 在 Kafka EOF 过期的任务停止问题。#37983
- 修复一流多表 Coredump。#37370
- 修复 Group Commit 内存估计不准导致的提前反压问题。#37379
- 优化 Group Commit BE 侧线程占用。#37380
- 修复数据没有分区时没有错误 URL 的问题。#37401
- 修复导入时潜在的内存误操作问题。#38021

8.4.7.4.9 主键模型

- 降低主键表 Compaction 的内存占用。#36968
- 修复主键副本 Clone 失败时可能的重复数据问题。#37229

8.4.7.4.10 内存管理

- 修复 Jemalloc Cache 统计不准的问题。#37464
- 修复在 K8s / CGroup 中不能正确获取内存大小的问题。#36966

8.4.7.4.11 权限管理

- 修复 Table Valued Function 引用 Resource 时没有鉴权的问题。#37132
- 修复 Show Role 语句中没有 Workload Group 权限的问题。#36032
- 修复创建 Row Policy 时，同时执行两条语句，导致 FE 重启失败的问题。#37342
- 修复部分情况下，老版本升级后，因为 Row Policy 导致 FE 元数据回放失败的问题。#37342

8.4.7.4.12 其他

- 修复计算节点参与内部表创建的问题。#37961
- 修复 `enable_strong_read_consistency = true` 时从延迟问题。#37641

8.4.8 Release 2.1.4

Apache Doris 2.1.4 版本已于 2024 年 6 月 26 日正式发布。 在 2.1.4 版本中，我们对数据湖分析场景进行了多项功能体验优化，重点修复了旧版本中异常内存占用的问题，同时提交了若干改进项以及问题修复，进一步提升了系统的性能、稳定性及易用性，欢迎大家下载使用。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.4.8.1 行为变更

- 通过 Catalog 查询外部表（如 Hive 数据表）时，系统将忽略不存在的文件： 当从元数据缓存中获取文件列表时，由于缓存更新并非实时，因此可能存在实际的文件列表已删除、而元数据缓存中仍存在该文件的情况。为了避免由于尝试访问不存在的文件而导致的查询错误，系统会忽略这些不存在的文件。#35319
- 默认情况下，创建 Bitmap Index 不再默认变更为 Inverted Index。该行为由 FE 配置项 `enable_create_bitmap_index_as_inverted_index` 控制，默认为 FALSE。#35521
- 当使用 `--console` 启动 FE、BE 进程时，所有日志将输出到标准输出，并通过前缀区分不同类型的日志。#35679

  关于更多信息，请参考文档：

- BE 日志管理
- FE 日志管理
- 如果建表时没有填写表注释，默认注释为空，不再使用表类型作为默认表注释。#36025
- DECIMALV3 的默认精度从 (9, 0) 调整为 (38,9)，以和最初发布此功能的版本保持兼容。#36316

8.4.8.2 新增功能

8.4.8.2.1 查询优化器

- 支持 FE 火焰图工具：在 FE 部署目录 `${DORIS_FE_HOME}/bin` 中会增加`profile_fe.sh` 脚本，可以利用 async-profiler 工具生成 FE 的火焰图，用以发现性能瓶颈点。

关于更多信息，请参考文档：使用 FE Profiler 生成火焰图

- 支持 SELECT DISTINCT 与聚合函数同时使用：支持 SELECT DISTINCT 与聚合函数同时使用，在一个查询中同时去重和进行聚合操作，如 SUM、 MIN/MAX 等。
- 支持无 GROUP BY 的单表查询重写：无 GROUP BY 的单表查询重写功能允许数据库优化器在不需要分组的情况下，根据查询的复杂性和数据表的结构，自动选择最佳的执行计划来执行查询，这可以提高查询的性能，减少不必要的资源消耗，并简化查询逻辑。#35242.
- 查询优化器全面支持高并发点查询功能：在 2.1.4 版本之后，查询优化器全面支持高并发点查询功能，所有符合点查询条件的 SQL 语句会自动走短路径查询，无需用户在客户端额外设置 `set experimental_enable_nereids_planner = false`。#36205.

8.4.8.2.2 湖仓一体

- 支持 Paimon 的原生读取器来处理 Deletion Vector： Deletion Vector 主要用于标记或追踪哪些数据已被删除或标记为删除，通常应用在需要保留历史数据的场景，基于本优化可以提升大量数据更新或删除时的处理效率。#35241

关于更多信息，请参考文档：数据湖分析 - Paimon

- 支持在表值函数（TVF）中使用 Resource：TVF 功能为 Apache Doris 提供了直接将对象存储或 HDFS 上的文件作为 Table 进行查询分析的能力。通过在 TVF 中引用 Resource，可以避免重复填写连接信息，提升使用体验。#35139

  关于更多信息，请参考文档：表函数 - HDFS
- 支持通过 Ranger 插件实现数据脱敏：开启 Ranger 鉴权功能后，支持使用 Ranger 中的 Data Mask 功能进行数据脱敏。

  关于更多信息，请参考文档：基于 Apache Ranger 的鉴权管理

8.4.8.2.3 异步物化视图

- 构建支持内表触发式更新，如果物化视图使用的是内表，如果内表数据发生变化，可以触发物化视图刷新，需要在创建物化视图时指定 REFRESH ON COMMIT。
- 支持单表透明改写。

关于更多信息，请参考文档：查询异步物化视图

- 透明改写支持 agg_state, agg_union 类型的聚合上卷，物化视图可以定义为 agg_state 或者 agg_union，查询使用具体的聚合函数，或者使用 agg_merge

关于更多信息，请参考文档：AGG_STATE

8.4.8.2.4 其他

- 新增 `replace_empty` 函数：将字符串中的子字符串进行替换，当旧字符串为空时，会将新字符串插入到原有字符串的每个字符前以及最后。

关于更多信息，请参考文档：字符串函数 - REPLACE_EMPTY

- 支持 `show storage policy using` 语句：支持查看所有或指定存储策略关联的表和分区。

  关于更多信息，请参考文档：SQL 语句 - SHOW

- 支持 BE 侧的 JVM 指标： 通过在 `be.conf` 配置文件中设置`enable_jvm_monitor=true`，可以启用对 BE 节点 JVM 的监控和指标收集，有助于了解 BE JVM 的资源使用情况，以便进行故障排除和性能优化。

8.4.8.3 改进优化

- 支持为中文列名创建倒排索引。#36321
- 优化 Segment Cache 所消耗内存的估算准确度，以便能够更快地释放未使用的内存。#35751
- 在使用 Export 功能导出数据时，提前过滤空分区以提升导出效率。#35542
- 优化 Routine Load 任务分配算法以平衡 BE 节点之间的负载压力。#34778
- 在设置错误的会话变量名时，自动识别近似变量值并给出更详细的错误提示。#35775
- 支持将 Java UDF Jar 文件放到 FE 的 `custom_lib` 目录中并默认加载。#35984
- 为审计日志导入作业添加超时的全局变量`audit_plugin_load_timeout` ，以控制在加载审计插件或处理审计日志时允许的最大执行时间。
- 优化了异步物化视图透明改写规划的性能。
- 当 `INSERT` 源数据为空时，BE 将不会执行任何操作。#34418
- 支持分批获取 Hudi 和 Hive 文件列表，当存在大量数据文件时可以提升数据扫描性能。#35107
- 120 万文件场景中，获取文件列表的时间由 390 秒缩减到 46 秒。
- 创建异步物化视图时，禁止使用动态分区。
- 支持检测 Hive 外表分区数据是否和异步物化视图同步。
- 允许异步物化视图创建索引。

8.4.8.4 缺陷修复

8.4.8.4.1 查询优化器

- 修复 SQL Cache 在 `truncate paritition` 后依然返回旧结果的问题。#34698
- 修复从 JSON Cast 到其他类型 Nullable 属性不对的问题。#34707
- 修复偶现的 DATETIMEV2 Literal 化简错误。#35153
- 修复窗口函数中不能使用 `COUNT(*)` 的问题。#35220
- 修复 `UNION ALL` 下全部是无 `FROM` 的SELECT ‘时，Nullable 属性可能错误的问题。#35074
- 修复 `bitmap in join` 和子查询解嵌套无法同时使用的问题。#35435
- 修复在特定情况下过滤条件不能下推到 CTE Producer 导致的性能问题。#35463
- 修复聚合 Combinator 为大写时，无法找到函数的问题。#35540
- 修复窗口函数没有被列裁剪正确裁剪导致的性能问题。#35504
- 修复多个同名不同库的表同时出现在查询中时，可能解析错误导致结果错误的问题。#35571
- 修复对于 Schema 表扫描时，由于生成了 Runtime Filter 导致查询报错的问题。#35655
- 修复关联子查询解嵌套，关联条件被折叠为 Null Literal 导致无法执行的问题。#35811
- 修复规划时，偶现的 Decimal Literal 被错误设置精度的问题。#36055
- 修复偶现的多层聚合被合并后规划错误的问题。#36145
- 修复偶现的聚合扩展规划报错输入输出不匹配的问题。#36207
- 修复偶现的 <=> 被错误转换为 = 的问题。#36521

8.4.8.4.2 查询执行

- 修复 Pipeline 引擎上达到限定的行数且内存没有释放时查询被挂起的问题。#35746
- 修复当设置 `enable_decimal256 =true` 且查询优化器回退到旧版本时 BE 发生 Core 的问题。#35731

8.4.8.4.3 物化视图

- 修复构建异步物化视图指定 store_row_column 属性，be core 的问题。
- 修复构建异步物化视图指定 storage_medium 不生效的问题。
- 修复基表删除后，异步物化视图 show partitions 报错的问题。
- 修复异步物化视图引起备份恢复异常的问题。
- 修复分区改写可能导致错误结果的问题。

8.4.8.4.4 半结构化数据分析

- 修复带有空 Key 的 VARIANT 类型发生 Core 的问题。#35671
- Bitmap 索引和 BloomFilter 索引不应支持轻量级索引变更。#35225

8.4.8.4.5 主键模型

- 修复在有部分列更新导入的情况下发生异常重启，可能会产生重复 Key 的问题。#35678
- 修复在内存紧张时发生 Clone 时 BE 可能会发生 Core 的问题。#34702

8.4.8.4.6 湖仓一体

- 修复创建 Hive 表时无法使用完全限定名（如 `ctl.db.tbl`）的问题。#34984
- 修复 Refresh 操作时 Hive Metastore 连接未关闭的问题。#35426
- 修复从 2.0.x 升级到 2.1.x 时可能的元数据回放问题。#35532
- 修复 TVF 表函数无法读取空 Snappy 压缩文件的问题。#34926
- 修复无法读取具有无效最小/最大列统计信息的 Parquet 文件的问题。#35041
- 修复 Parquet/ORC Reader 中无法处理带有 null-aware 函数下推谓词的问题。#35335
- 修复创建 Hive 表时分区列顺序的问题。#35347
- 修复当分区值包含空格时无法将 Hive 表写入 S3 的问题。#35645
- 修复 Doris 写入 Parquet 格式 Hive 表无法被 Hive 读取的问题。#34981
- 修复 Hive 表 Schema 变更后无法读取 ORC 文件的问题。#35583
- 修复了部分情况下，启用 Hive Metastore Listener 后 FE 无法启动的问题。#36533
- 修复由 Hadoop FS 缓存引起的 FE OOM 问题。#36403
- 修复写出 Parquet 格式文件写出 Row Group 过小的问题。#36042 #36143
- 修复 Paimon 表 Schema 变更后无法通过 JNI 读取 Paimon 表的问题。#35309
- 修复 Paimon 表 Schema 变更后由于表字段长度判断错误导致无法读取的问题。#36049
- 修复了读取 Iceberg 中的时间戳列类型时的时区问题。#36435
- 修复了 Iceberg 表上的日期时间转换错误和数据路径错误的问题。#35708
- 修复阿里云 OSS Endpoint 不正确的问题。#34907
- 修复了大量文件导致的查询性能下降问题。#36431
- 允许用户定义的属性通过表函数传递给 S3 SDK。#35515

8.4.8.4.7 数据导入

- 修复 CANCEL LOAD 命令不生效的问题。#35352
- 修复导入事务 Publish 阶段空指针错误导致导入事务无法完成的问题。#35977
- 修复 bRPC 通过 HTTP 发送大数据文件序列化的问题。#36169

8.4.8.4.8 数据管控

- 修复了在将 DDL 或 DML 转发到主 FE 后，ConnectionContext 中的资源标签未设置的问题。#35618
- 修复了在启用 lower_case_table_names 时，Restore 表名不正确的问题。#35508
- 修复了清理无用数据或文件的管理命令不生效的问题。#35271
- 修复了无法从分区中删除存储策略的问题。#35874
- 修复了向多副本自动分区表导入数据时的数据丢失问题。#36586
- 修复了使用旧优化器查询或插入自动分区表时，表的分区列发生变化的问题。#36514

8.4.8.4.9 内存管理

- 修复日志中频繁报错 Cgroup meminfo 获取失败的问题。#35425
- 修复使用 BloomFilter 时 Segment 缓存大小不受控制导致进程内存异常增长的问题。#34871

8.4.8.4.10 权限

- 修复开启表名大小写不敏感后，权限设置无效的问题。#36557
- 修复通过非 Master FE 节点设置 LDAP 密码不生效的问题。#36598
- 修复了无法检查 SELECT COUNT(*) 语句授权的问题。#35465

8.4.8.4.11 其他

- 修复 MySQL 连接损坏情况下，客户端 JDBC 程序无法关闭连接的问题。#36616
- 修改 SHOW PROCEDURE STATUS 语句返回值与 MySQL 协议不兼容的问题。#35350
- libevent 库强制开启 Keepalive 以解决部分情况下连接泄露的问题。#36088

8.4.8.5 致谢

@133tosakarin、@924060929、@airborne12、@amorynan、@AshinGau、@BePPPower、@BiteTheDDDDt、@ByteYue、@caiconghui、@CalvinKirs、@cambyzju、@catpineapple、@cjj2010、@csun5285、@DarvenDuan、@dataroaring、@deardeng、@Doris-Extras、@eldenmoon、@englefly、@feiniaofeiafei、@felixwluo、@freemandealer、@Gabriel39、@gavinchou、@GoGoWen、@HappenLee、@hello-stephen、@hubgeter、@hust-hhb、@jacktengg、@jackwener、@jeffreys-cat、@Jibing-Li、@kaijchen、@kaka11chen、@Lchangliang、@liaoxin01、@LiBinfeng-01、@lide-reed、@luennng、@luwei16、@mongo360、@morningman、@morrySnow、@mrhhsg、@Mryange、@mymeiyi、@nextdreamblue、@platoneko、@qidaye、@qzsee、@seawinde、@shuke987、@sollhui、@starocean999、@suxiaogang223、@TangSiyang2001、@Thearas、@Vallishp、@w41ter、@wangbo、@whutpencil、@wsjz、@wuwenchi、@xiaokang、@xiedeyantu、@XieJiann、@xinyiZzz、@XuPengfei-1020、@xy720、@xzj7019、@yiguolei、@yongjinhou、@yujun777、@Yukang-Lian、@Yulei-Yang、@zclllyybb、@zddr、@zfr9527、@zgxme、@zhangbutao、@zhangstar333、@zhannngchen、@zhiqiang-hhhh、 @zy-kkk、 @zzzxl1993

8.4.9 Release 2.1.3

Apache Doris 2.1.3 版本已于 2024 年 5 月 21 日正式发布。该版本更新带来了若干改进项，包括支持向 Hive 回写数据、物化视图、新函数等功能，同时改善权限管理并修复若干问题，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.4.9.1 功能特性

1. 支持通过 Hive Catalog 向 Hive 表中回写数据

从 2.1.3 版本开始，Apache Doris 支持对 Hive 的 DDL 和 DML 操作。用户可以直接通过 Apache Doris 在 Hive 中创建库表，通过执行`INSERT INTO`语句来向 Hive 表中写入数据。通过该功能，用户可以通过 Apache Doris 对 Hive 进行完整的数据查询和写入操作，进一步帮助用户简化湖仓一体架构。

参考文档

2. 支持在异步物化视图之上构建新的异步物化视图

用户可以在异步物化视图之上来创建新的异步物化视图，直接复用计算好的中间结果进行数据加工处理，简化复杂的聚合和计算操作带来的资源消耗和维护成本，进一步加速查询性能、提升数据可用性。

3. 支持通过物化视图嵌套物化视图进行重写

物化视图（Materialized View，MV）是用于存储查询结果的数据库对象。现在，Apache Doris 支持通过 MV 嵌套物化视图进行重写，这有助于优化查询性能。

4. 新增 SHOW VIEWS 语句

可以使用`SHOW VIEWS`语句来查询数据库中的视图，有助于更好地管理和理解数据库中的视图对象。

5. Workload Group 支持绑定到特定的 BE 节点

Workload Group 可以绑定到特定的 BE 节点，实现查询执行的更精细化控制，以优化资源使用和提高性能。

6. Broker Load 支持压缩的 JSON 格式

Broker Load 支持导入压缩的 JSON 格式数据，可以显著减少数据传输的带宽需求、加速数据导入性能。

7. TRUNCATE 函数可以使用列作为 scale 参数

TRUNCATE 函数现在可以接受列作为 scale 参数，这使得在处理数值数据时可以更加灵活。

8. 添加新的函数 uuid_to_int 和 int_to_uuid

这两个函数允许用户在 UUID 和整数之间进行转换，对于需要处理 UUID 数据的场景有明显帮助。

9. 添加 bypass_workload_group Session Variable 以绕过查询队列

会话变量 bypass_workload_group 允许某些查询绕过 Workload Group 队列直接执行，这可以用于处理需要快速响应的关键查询。

10. 添加 strcmp 函数

strcmp 函数用于比较两个字符串并返回它们的比较结果，帮助文本数据的处理更加简易。

11. 支持 HLL 函数 hll_from_base64 和 hll_to_base64

HLL（HyperLogLog）是一种用于基数估计的算法，以上两个函数允许用户将 HLL 数据从 Base64 编码的字符串中解码，或将 HLL 数据编码为 Base64 字符串，这对于存储和传输 HLL 数据非常有用。

8.4.9.2 优化改进

1. 替换 SipHash 为 XXHash 以改善 Shuffle 性能

SipHash 和 XXHash 都是哈希函数，但 XXHash 在某些场景下可能提供更快的哈希速度和更好的性能，此优化旨在通过采用 XXHash 来提高数据 Shuffle 过程中的性能。

2. 异步物化视图支持 OLAP 表分区列为可以为 NULL：

允许异步物化视图支持 OLAP 表的分区列可以为 NULL，从而增强了数据处理的灵活性。

3. 收集列统计信息时限制最大字符串长度为 1024 以控制 BE 内存使用

在收集列统计信息时，限制字符串的长度可以防止过大的数据消耗过多的 BE 内存，有助于保持系统的稳定性和性能。

4. 支持动态删除 Bitmap Cache 以提高性能

通过支持动态删除不再需要的 Bitmap Cache，可以释放内存并改善系统性能。

5. 在 ALTER 操作中减少内存使用

减少 ALTER 操作中的内存使用，以提高系统资源的利用效率。

6. 支持复杂类型的常量折叠

支持 Array/Map/Struct 复杂类型的常量折叠；

7. 在 Aggregate Key 聚合模型中增加对 Variant 类型的支持

Variant 数据类型能够存储多种数据类型，在此优化中允许对 Variant 类型的数据进行聚合操作，从而增强了半结构化数据分析的灵活性。

8. 在 CCR 中支持新的倒排索引格式

9. 优化嵌套物化视图的重写性能

10. 支持 decimal256 类型的行存格式

在行存格式中支持 decimal 256 类型，以以扩展系统对高精度数值数据的处理能力。

8.4.9.3 行为变更

1. 授权（Authorization）

- Grant_priv 权限更改：Grant_priv不能再被任意授予。执行 GRANT 操作时，用户不仅需要具有Grant_priv，还需要具有要授予的权限。例如，如果想要授予对table1的 SELECT 权限，那么该用户不仅需要具有 GRANT 权限，还需要具有对table1的 SELECT 权限，这增加了权限管理的安全性和一致性。
- Workload Group 和 Resource 的 Usage_priv：Usage_priv 对 Workload Group 和 Resource 的权限不再是全局级别的，而是仅限于 Resource 和 Workload Group 内，权限的授予和使用将更加具体。
- 操作的授权：之前未被授权的操作现在都有了相应的授权，以实现更加细致和全面地操作权限控制。

2. LOG 目录配置

FE 和 BE 的日志目录配置现在统一使用LOG_DIR环境变量，所有其他不同类型的日志都将以LOG_DIR作为根目录进行存储。同时为了保持版本间的兼容性，以前的配置项sys_log_dir仍然可以使用。

3. S3 表函数（TVF）

由于之前的解析方式在某些情况下可能无法正确识别或处理 S3 的 URL，因此将对象存储路径的解析逻辑进行重构。对于 S3 表函数中的文件路径，需要传递force_parsing_by_standard_uri参数来确保被正确解析。

8.4.9.4 升级问题

由于许多用户将某些关键字用作列名或属性值，因此将如下关键字设置为非保留关键字，允许用户将其用作标识符使用。

8.4.9.5 问题修复

1. 修复在腾讯云 COSN 上读取 Hive 表时的无数据错误

解决了在腾讯云 COSN 存储上读取 Hive 表时可能遇到的无数据错误，增强了与腾讯云存储服务的兼容性。

2. 修复 milliseconds_diff 函数返回错误结果

修复milliseconds_diff函数在某些情况下返回错误结果的问题，确保了时间差计算的准确性。

3. 用户定义变量应转发到 Master 节点

确保用户定义的变量能够正确地传递到 Master 节点，以便在整个系统中保持一致性和正确的执行逻辑。

4. 修复添加复杂类型列时遇到的 Schema Change 问题

在添加复杂类型列时，可能会遇到 Schema Change 问题，此修复确保了 Schema Change 的正确性。

5. 修复 FE master 节点更改时 Routine Load 的数据丢失问题

Routine Load常用于订阅 Kafka 消息队列中的数据，此修复解决了在 FE Master 节点更改时可能导致的数据丢失问题。

6. 修复当找不到 Workload Group 时 Routine Load 失败的问题

修复了当Routine Load找不到指定 Workload Group 时导致的失败问题。

7. 支持 column string64，以避免在 string size 溢出 unit32 时 Join 失败的问题

在某些情况下，字符串的大小可能会超过 unit32 的限制，支持`string64`类型可以确保字符串 JOIN 操作的正确执行。

8. 允许 Hadoop 用户创建 Paimon Catalog

允许具有权限的对应 Hadoop 用户来创建 Paimon Catalog。

9. 修复 function_ipxx_cidr 函数与常量参数的问题

修复了`function_ipxx_cidr`函数在处理常量参数时可能出现的问题，保证函数执行的正确性。

10. 修复使用 HDFS 进行还原时的文件下载错误

解决了在使用 HDFS 进行数据还原时遇到的 “failed to download” 错误，确保了数据恢复的正确性和可靠性。

11. 修复隐藏列相关的列权限问题

在某些情况下，隐藏列的权限设置可能不正确，此修复确保了列权限设置的正确性和安全性。

12. 修复在 K8s 部署中 Arrow Flight 无法获取正确 IP 的问题

此修复解决了在 Kubernetes 部署环境中 Arrow Flight 无法正确获取 IP 地址的问题。

8.4.10 Release 2.1.2

亲爱的社区小伙伴们，Apache Doris 2.1.2 版本已于 2024 年 4 月 12 日正式发布。该版本提交了若干改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.4.10.1 1 行为变更

1. 将 EXPORT 命令中 `data_consistence` 属性的默认值调整为 Partition，这可以使得并发导入的同时做 EXPORT 操作更容易成功。

   - https://github.com/apache/doris/pull/32830

2. 兼容部分 MySQL Connector（如 MySQL.Data for .NET）将 SELECT @``@autocommit 的返回值类型变更为 BIGINT。

   - https://github.com/apache/doris/pull/33282

3. Auto Partition 语法变化，详见文档

   - https://github.com/apache/doris/pull/32737

4. Auto Partition 禁止和 Dynamic Partition 同时作用在一张表上

   - https://github.com/apache/doris/pull/33736

8.4.10.2 2 升级问题

1. 修复正常 Workload Group 从 2.0 或者更早版本升级到 2.1 时没有默认创建的问题。

- https://github.com/apache/doris/pull/33197

8.4.10.3 3 新功能

1. 增加 processlist 系统表功能，用户可以通过查询系统表获得活跃的链接信息。

- https://github.com/apache/doris/pull/32511

2. 增加新的表函数 LOCAL 以访问部分共享存储上的文件。

- https://github.com/apache/doris-website/pull/494

8.4.10.4 4 改进与优化

1. 跳过部分不必要检查，加速在 K8s 环境下优雅退出的速度。

- https://github.com/apache/doris/pull/33212

2. 在 Profile 中增加已命中的物化视图信息，能够方便地定位物化视图是否命中。

- https://github.com/apache/doris/pull/33137

3. 针对 DB2 Catalog，增加测试链接是否通畅的功能，能够在建立 Catalog 时做部分链接检查。

- https://github.com/apache/doris/pull/33335

4. 增加 DNS Cache，解决 K8s 环境下域名解析较慢，从而影响查询的问题。

- https://github.com/apache/doris/pull/32869

5. 增加异步刷新 Catalog 中表的行数信息，避免查询抖动。

- https://github.com/apache/doris/pull/32997

8.4.10.5 5 Bug 修复

1. 修复 Iceberg Catalog 中，不支持 Iceberg 自定义属性的问题，例如 “io.manifest.cache-enabled”。

- https://github.com/apache/doris/pull/33113

2. LEAD/LAG 函数的 Offset 起始位置可以设置为 0。

- https://github.com/apache/doris/pull/33174

3. 修复部分导入过程中可能出现的 Timeout 的问题。

- https://github.com/apache/doris/pull/33077
- https://github.com/apache/doris/pull/33260

4. 修复部分 ARRAY / MAP / STRUCT 类型在 Compaction 中引起 Core 的问题。

- https://github.com/apache/doris/pull/33130 https://github.com/apache/doris/pull/33295

5. 修复查询过程中 Runtime Filter 部分等待超时的问题。

- https://github.com/apache/doris/pull/33369

6. 修复 `unix_timestamp` 函数在 Auto Partition 中可能导致 Core 的问题。

- https://github.com/apache/doris/pull/32871

8.4.11 Release 2.1.1

亲爱的社区小伙伴们，Apache Doris 2.1.1 版本已于 2024 年 4 月 3 日正式发布。该版本针对 2.1.0 版本出现的问题进行较为全面的优化，提交了若干改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- 立即下载： https://doris.apache.org/download/
- GitHub Release: https://github.com/apache/doris/releases

8.4.11.1 1 行为变更

1. 改变了 Float 类型字段返回值序列化的方式，可以提升大数据量下 Float 返回的性能。

- https://github.com/apache/doris/pull/32049

2. 将部分 Table Valued Function 变更为系统表 `active_queries()`, `workload_groups()`。

- https://github.com/apache/doris/pull/32314

3. 由于 `show query``/l``oad profile stmt` 语句在实际用户场景中使用较少，该语句将不再支持与维护。同时该功能在 Pipeline 与 PipelineX 引擎中不支持。

- https://github.com/apache/doris/pull/32467

4. 升级 Arrow Flight 版本至 15.0.2，同时用户需要使用 ADBC 15.0.2 版本访问 Doris。

- https://github.com/apache/doris/pull/32827.

8.4.11.2 2 升级问题

1. 修复了从 2.0.x 滚动升级至 2.1.x 的过程中，部分 BE 节点升级出现 Core 的问题。

- https://github.com/apache/doris/pull/32672
- https://github.com/apache/doris/pull/32444
- https://github.com/apache/doris/pull/32162

2. 修复了在 2.0.x 滚动升级至 2.1.x 过程中，使用 JDBC Catalog 会出现 Query 报错的问题。

- https://github.com/apache/doris/pull/32618

8.4.11.3 3 新功能

1. 默认开启列级权限。

- https://github.com/apache/doris/pull/32659

2. Pipeline 和 PipelineX 引擎能够在 K8S 下准确获取 CPU 核数。

- https://github.com/apache/doris/pull/32370

3. 支持读取 Parquet INT96 类型

- https://github.com/apache/doris/pull/32394

4. 支持 IP 透传的协议，以方便在 FE 之前启用代理的同时还能获取客户端准确的 IP 地址，实现白名单权限控制。

- https://github.com/apache/doris/pull/32338/files

5. 增加对 Workload Queue 检测指标。

- https://github.com/apache/doris/pull/32259

6. 增加系统表 `backend_active_tasks`，以实时监测每个 BE 上活跃任务以及消耗的资源信息。

- https://github.com/apache/doris/pull/31945

7. 在 Spark Doris Connector 中增加 IPV4 和 IPV6 的支持。

- https://github.com/apache/doris/pull/32240

8. CCR 支持倒排索引。

- https://github.com/apache/doris/pull/32101

9. 支持查询 Experimental 的 Session Variable。

- https://github.com/apache/doris/pull/31837

10. 支持建立 `bitmap_union(bitmap_from_array())` 函数的物化视图。

-https://github.com/apache/doris/pull/31962

11. 支持对 Hive 中 `HIVE_DEFAULT_PARTITION` 分区进行列裁剪。

- https://github.com/apache/doris/pull/31736

12. 支持 `set variable` 语句中使用函数。

- https://github.com/apache/doris/pull/32492

13. Arrow 序列化方式增加对 Variant 类型的支持。

- https://github.com/apache/doris/pull/32809

8.4.11.4 4 改进与优化

1. 当系统自动重启或者滚动升级之后，自动启动 Routine Load 导入任务。

- https://github.com/apache/doris/pull/32239

2. 优化了 Routine Load 任务在各个 BE 上的分布方式，让各个 BE 负载更加均衡。

- https://github.com/apache/doris/pull/32021

3. 升级 Spark 的版本，解决部分 Spark Load 的安全问题。

- https://github.com/apache/doris/pull/30368

4. 在冷热分离过程中，自动跳过被删除的 Tablet.

- https://github.com/apache/doris/pull/32079

5. Workload Group 支持对 Routine Load 的资源进行限制。

- https://github.com/apache/doris/pull/31671

6. 大幅度优化多表物化视图查询改写性能。

- https://github.com/apache/doris/pull/31886

7. 优化 Broker Load 任务对 FE 的内存使用

- https://github.com/apache/doris/pull/31985

8. 优化 Partition 的裁剪逻辑。

- https://github.com/apache/doris/pull/31970

9. 优化 Tablet Schema Cache 对 BE 内存使用。

- https://github.com/apache/doris/pull/31141

10. 多表物化视图增加更多对 JOIN 类型的支持，包括 INNER JOIN、LEFT OUTER JOIN、RIGHT OUTER JOIN、FULL OUTER JOIN、 LEFT SEMI JOIN、 RIGHT SEMI JOIN、 LEFT ANTI JOIN、 RIGHT ANTI JOIN

- https://github.com/apache/doris/pull/32909

8.4.11.5 5 Bugs 修复

1. 修复 TopN 下推导致的问题。

- https://github.com/apache/doris/pull/326332.

2. 修复 JAVA UDF 带来的内存泄露问题。

- https://github.com/apache/doris/pull/32630

3. 修复 ODBC 表备份恢复问题。

- https://github.com/apache/doris/pull/31989

4. 修复对 Variant 类型进行运算时常量折叠会导致 BE 出错的问题

- https://github.com/apache/doris/pull/32265

5. 修复了部分导入任务失败时 Routine Load 卡住的问题。

- https://github.com/apache/doris/pull/32638

6. 修复 SEMI JOIN 结果不正确的问题。

- https://github.com/apache/doris/pull/32477

7. 当列的数据为空时，修复建立倒排索引会出错的问题。

- https://github.com/apache/doris/pull/32669

8. 修复`<=> join` 操作会出现 Core 的问题。

- https://github.com/apache/doris/pull/32623

9. 修复部分列更新在有 Sequence 列结果准确性的问题。

- https://github.com/apache/doris/pull/32574

10. 修复 Select Outfile 导出到 Parquet 或者 ORC 格式的列类型映射问题。

- https://github.com/apache/doris/pull/32281

11. 修复在 Restore 过程中 BE 有时候会 Core 的问题。

- https://github.com/apache/doris/pull/32489

12. 修复 `array_agg`函数结果不对的问题。

- https://github.com/apache/doris/pull/32387

13. 使 Variant 类型应当一直是 nullable.

- https://github.com/apache/doris/pull/32248

14. 修复 Schema Change 没有正确处理空 Block 的问题。

- https://github.com/apache/doris/pull/32396

15. 修复使用 `json_length()` 函数时部分场景会出错的问题。

- https://github.com/apache/doris/pull/32145

16. 修复 Iceberg 表没有正确处理 Date Cast 转换的问题。

- https://github.com/apache/doris/pull/32194

17. 修复 Variant 类型建立 Index 时出现的部分 Bug。

- https://github.com/apache/doris/pull/31992

18. 修复当多个 `map_agg` 函数同时使用时结果不正确的问题。

- https://github.com/apache/doris/pull/31928

19. 修复 `money_format` 函数的返回结果不正确的问题。

- https://github.com/apache/doris/pull/31883

20. 修复在高并发的建立链接时部分请求会卡住的问题。

- https://github.com/apache/doris/pull/31594

8.4.12 Release 2.1.0

亲爱的社区小伙伴们，我们很高兴地向大家宣布，在 3 月 8 日我们引来了 Apache Doris 2.1.0 版本的正式发布，欢迎大家下载使用。

- 在查询性能方面，2.1 系列版本我们着重提升了开箱盲测性能，力争不做调优的情况下取得较好的性能表现，包含了对复杂 SQL 查询性能的进一步提升，在 TPC-DS 1TB 测试数据集上获得超过 100% 的性能提升，查询性能居于业界领先地位。
- 在数据湖分析场景，我们进行了大量性能方面的改进、相对于 Trino 和 Spark 分别有 4-6 倍的性能提升，并引入了多 SQL 方言兼容、便于用户可以从原有系统无缝切换至 Apache Doris。在面向数据科学以及其他形式的大规模数据读取场景，我们引入了基于 Arrow Flight 的高速读取接口，数据传输效率提升 100 倍。
- 在半结构化数据分析场景，我们引入了全新的 Variant 和 IP 数据类型，完善了一系列分析函数，面向复杂半结构化数据的存储和分析处理更加得心应手。
- 在 2.1.0 版本中我们也引入了基于多表的异步物化视图以提升查询性能，支持透明改写加速、自动刷新、外表到内表的物化视图以及物化视图直查，基于这一能力物化视图也可用于数据仓库分层建模、作业调度和数据加工。
- 在存储方面，我们引入了自增列、自动分区、MemTable 前移以及服务端攒批的能力，使得大规模数据实时写入的效率更高。
- 在负载管理方面，我们进一步完善了 Workload Group 资源组的隔离能力，并增加了运行时查看 SQL 资源用量的能力，进一步提升了多负载场景下的稳定性。

在 2.1.0 版本的研发过程中，有 237 位贡献者为 Apache Doris 带来了接近 6000 个 Commits。 同时 2.1.0 版本也同样经过了近百家社区用户的大规模打磨，在测试过程中向我们反馈了许多有价值的优化项，在此向所有参与版本研发、测试和需求反馈的贡献者们表示最衷心的感谢。后续我们将会持续敏捷发版来响应所有用户对功能和稳定性的更高追求，欢迎大家在使用过程中给予我们更多反馈。

- GitHub 下载：https://github.com/apache/doris/releases
- 官网下载：https://doris.apache.org/download

8.4.12.1 复杂查询性能提升 100%，TPC-DS 业界领先

在 2.1 系列版本中，我们着重提升了开箱盲测性能，力争不做调优的情况下取得较好的性能表现，包含了对复杂 SQL 查询性能的进一步提升。在此我们以 TPC-DS 1TB 作为性能测试对比的基准，重点对比最新 2.1.0 版本与 2.0.5 版本的性能提升。集群规模均为 1FE、3BE，其中 BE 节点的服务器配置为 48C 192G。从以下测试结果中可以看到：

- 2.1.0 版本的总查询耗时为 245.7 秒，相较于 2.0.5 版本的 489.6 秒，性能提升达到 100 %；
- 在全部 99 个 SQL 中，有近三分之一的 SQL 查询性能提升达到 2 倍以上，超过 80 个 SQL 都获得显著性能提升；
- 不论是基础的过滤、排序、聚合，或者复杂的多表关联查询、子查询以及窗口函数计算，2.1.0 版本都有更为明显的性能优势；

- 2.0.5 版本或 2.1.0 版本，都可以完整执行 TPC-DS 的 99 个查询。

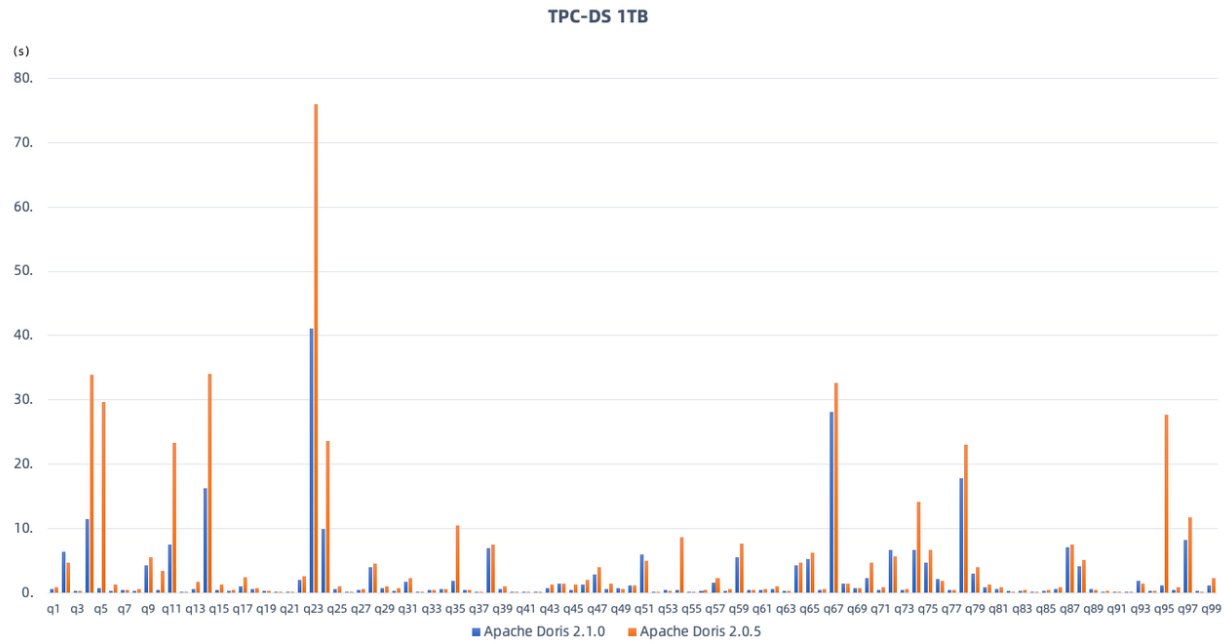


图 252: 复杂查询性能提升 100%，TPC-DS 业界领先

以上详细测试结果我们将在后续提交到 Apache Doris 官网文档中，也欢迎所有用户在完成最新版本的部署后进行测试复现。

与此同时，我们也对业内多个 OLAP 系统在同等硬件资源和多个测试数据规模下进行了性能测试，不论大宽表场景或多表关联场景，Apache Doris 都具备着极为明显的性能优势。毫无疑问，Apache Doris 已在业界同类产品中性能居于最领先地位！

8.4.12.1.1 优化器更智能

在 Apache Doris 2.0 版本中我们引入了全新查询优化器，在绝大多数场景无需任何调优即可实现极致的查询性能。而在最新发布的 Apache Doris 2.1 版本中，查询优化器在整体代际更新的基础上，进行了优化规则的扩展和枚举框架的完善，面向复杂分析场景更加得心应手：

- 优化器基础设施完善：在多种优化器基础设施方面进行了补充和增强，例如对统计信息推导和代价模型方面的持续改进，使之能够收集更多的特征信息为复杂优化提供基础；
- 优化规则持续扩展：得益于丰富的实际场景反馈，新版本中查询优化器增强了包括算子下压在内的许多经典规则，结合上述基础设施扩充而引入的新优化规则，使得新版本的查询优化器能覆盖更广泛的使用场景；
- 枚举框架进一步优化：在查询优化器 Cascades 和 DPhyper 两大融合框架的基础上，继续深耕框架能力、优化框架性能，确立了更为清晰的枚举策略，兼顾计划质量和枚举效率，为高性能引擎提供坚实基础。例如将 Cascades 默认枚举表上限从 5 提升到了 8、有效扩大了高质量计划的覆盖范围，同时进一步优化 DPhyper 枚举效率、使之能够枚举出更优计划。

8.4.12.1.2 无统计信息优化

针对海量数据规模以及数据湖分析场景下，针对统计信息收集难度高、收集时间久的问题，在 2.1 版本中查询优化器利用多种启发式技术，大大提升了无统计信息场景下的计划质量，使得在没有统计信息的场景下也可获得较好的查询计划。同时扩展了 Runtime Filter 的下压场景和自适应能力，在执行过程中能够自适应地动态调整部分表达式谓词，使得 Apache Doris 在不依赖统计信息的情况下也具有优异的性能表现。

8.4.12.1.3 Parallel Adaptive Scan 并行自适应扫描

在复杂数据分析场景下，每次查询都需要扫描大量的数据进行计算，因此 IO 瓶颈很大程度上决定了查询性能的上限。为了提升 Scan IO 的性能，Apache Doris 采取了并行读取的技术，每个扫描线程读取 1 个或者多个 Tablet（即用户建表时指定的 Bucket），但如若用户建表时指定的 Bucket 数目不合理、那么磁盘扫描线程就无法并行工作，直接影响查询性能。为此，在 2.1 版本中我们引入了 Tablet 内的并行扫描技术，可以将多个 Tablet 进行池化，在磁盘扫描端可以根据行数来拆分多个线程并行扫描（最多支持 48 个线程），从而有效避免分桶数不合理导致的查询性能问题。

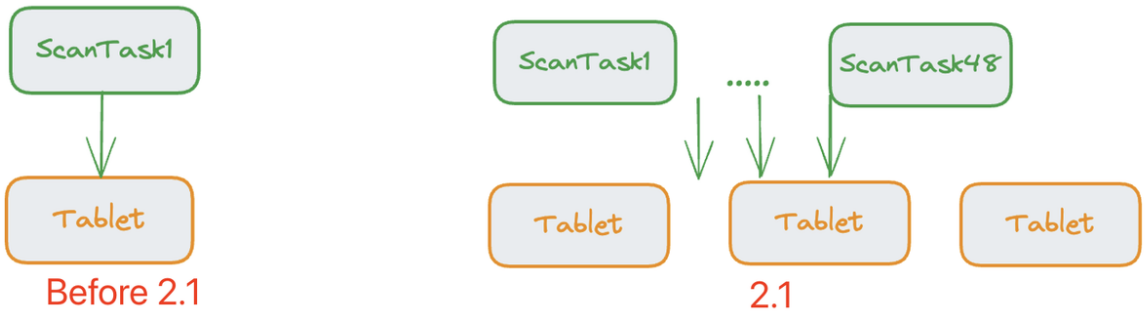


图 253: Parallel Adaptive Scan 并行自适应扫描

因此在 2.1 版本以后，我们建议用户在建表时设置的分桶数 = 整个集群磁盘的数量，在 IO 层面能将整个集群所有的 IO 资源全部利用起来。

> 当前 2.1.0 版本的 Parallel Adaptive Scan 只能针对 Unqiue Key 模型的 Merge-on-Write 表以及 Duplicate Key 模型生效，预计在 2.1.1 版本中会增加对 Unique Key 模型 Merge-on-Read 表和 Aggregate Key 模型的支持。

8.4.12.1.4 Local Shuffle

在部分场景下，数据分布不均会导致多个 Instance 的查询执行出现长尾。而为了解决单个 BE 节点上多个 Instance 之间的数据倾斜问题，在 Apache Doris 2.1 版本中我们引入了 Local Shuffle 技术，尽可能将数据打散从而加速查询。例如在某一典型的聚合查询中，数据在经过聚合之前将会通过一个 Local Shuffle 节点被均匀分布在不同的 Pipeline Task 中，如下图所示：

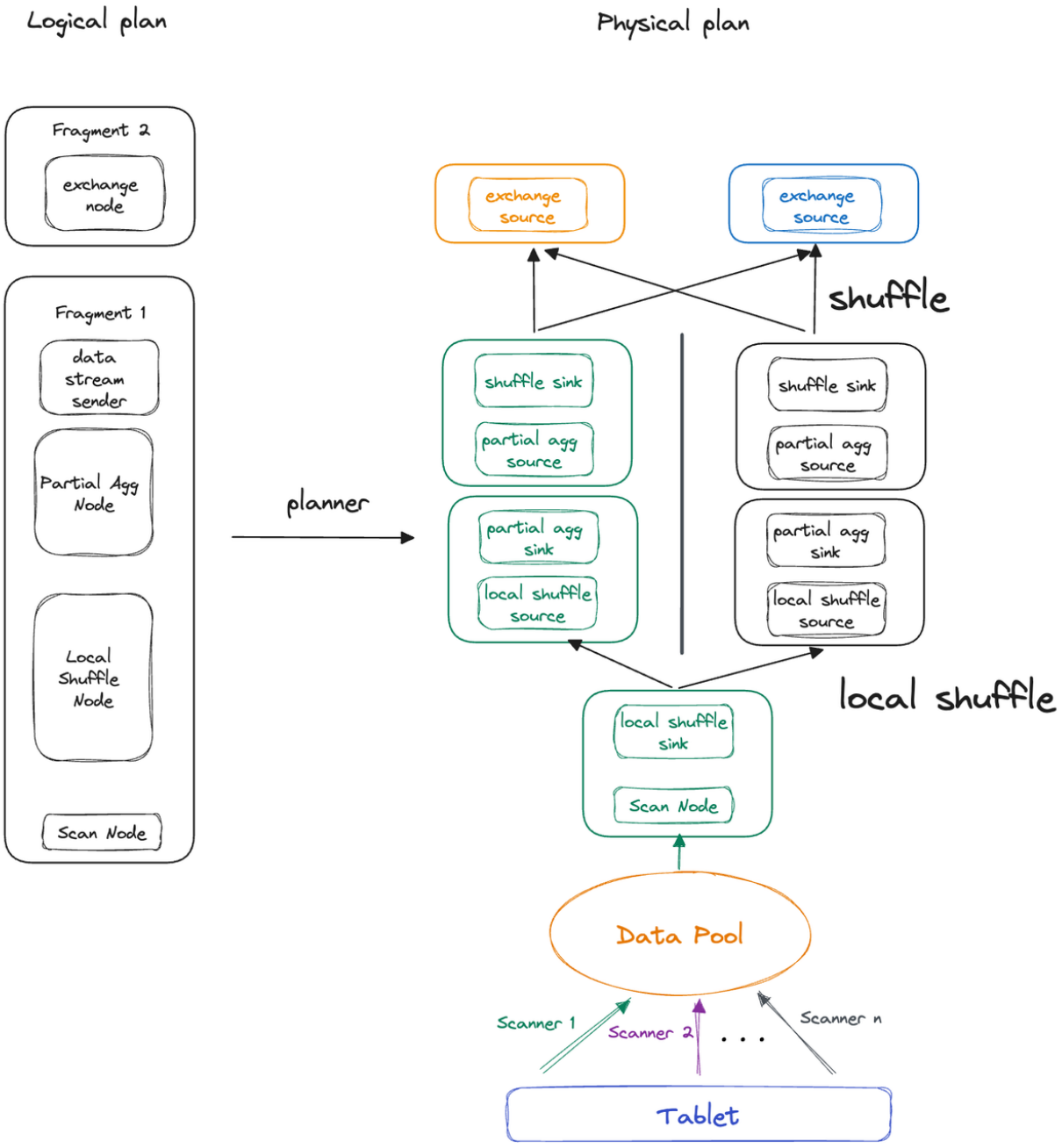


图 254: Local Shuffle

在具备了 Parallel Adaptive Scan 和 Local Shuffle 能力之后，Apache Doris 能够规避由于分桶数不合理、数据分布不均带来的性能问题。

在此我们分别使用 Clickbench（大宽表场景）和 TPC-H（多表 Join 的复杂分析场景）数据集模拟建表分桶不合理的情况，在 Clickbench 数据集中我们建表 Bucket 数量分别设为 1 和 16，在 TPC-H 100G 数据集下我们建表时每个 Partition 的 Bucket 数目分别设为 1 和 16。在开启 Parallel Adaptive Scan 和 Local Shuffle 之后，整体查询性能表现比较平稳，即使不合理的数据分布也能取得优异的性能表现。

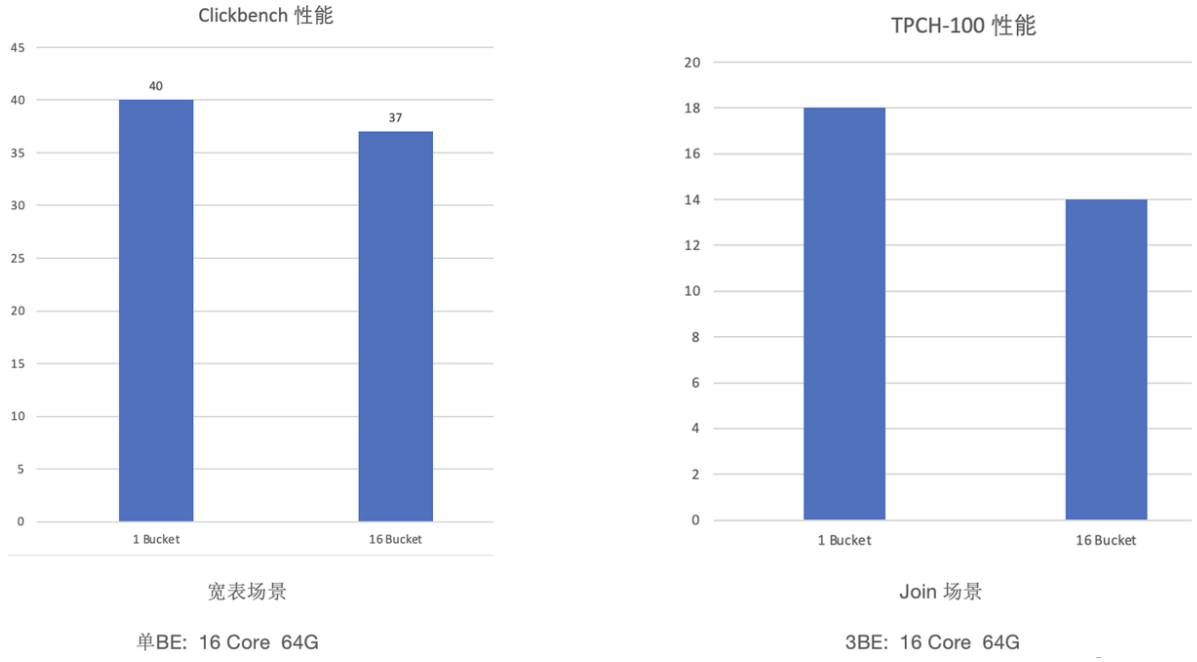


图 255: Local Shuffle Clickbench and TPCH-100

备注参考文档：Pipeline X 执行引擎

8.4.12.2 ARM 架构深度适配，性能提升 230%

在 Apache Doris 2.1 版本中我们针对 ARM 架构进行了深度的适配和指令集优化，可以在 ARM 架构上充分发挥 Apache Doris 的性能优势。相较于 2.0 版本，2.1 版本在 ClickBench、SSB 100G、TPC-H 100G 以及 TPC-DS 1TB 等多个测试数据集中取得了超过 100% 的性能提升。在此我们以大宽表场景的 ClickBench 以及多表关联场景的 TPC-H 为例，集群配置均为 1FE 3BE、BE 节点的服务器配置为 16C 64G 的 ARM 服务器，测试结论如下：

- 在大宽表场景中，ClickBench 测试数据集 43 个 SQL 的总查询耗时从 102.36 秒降低至 30.73 秒，性能提升超过 230%；
- 在多表关联场景中，TPC-H 22 个 SQL 的总查询耗时从 174.8 秒降低至 90.4 秒，性能提升 93%；

8.4.12.3 数据湖分析

8.4.12.3.1 性能提升

在 2.1 版本中，我们对数据湖分析方面做了大量改进，包括对 HDFS 和对象存储的 IO 优化、Parquet/ORC 文件格式的读取反序列优化、浮点类型解压优化、谓词下推执行优化、缓存策略以及扫描任务调度策略的优化，以

及针对不同数据源的统计信息准确性的提升及更精准的优化器代价模型。基于以上优化，Apache Doris 在数据湖分析场景下的性能得到大幅度提升。

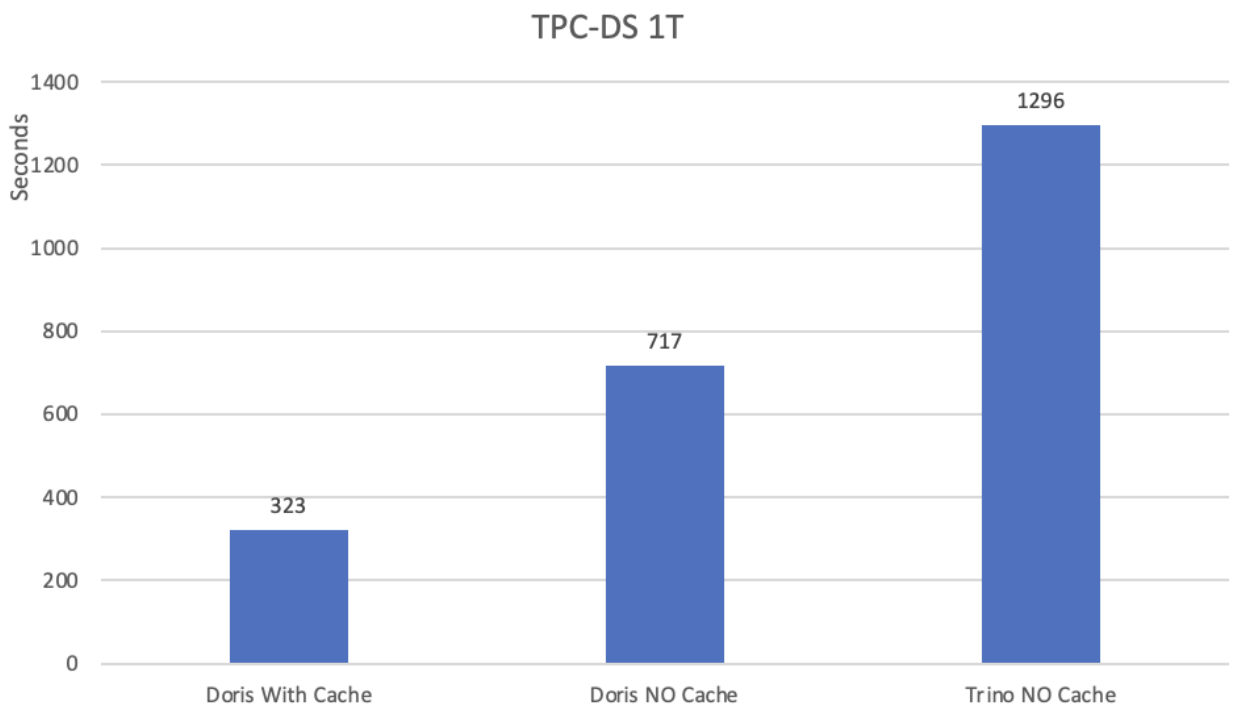


图 256: Doris 数据湖分析 - 性能提升

在此我们以 TPC-DS 1TB 场景下进行测试，Apache Doris 2.1 版本和 Trino 435 版本的性能测试结果如下：

- 在无缓存情况下，Apache Doris 的总体运行耗时间为 717s、Trino 为 1296s，查询耗时降低了 45%，全部 99 条 SQL 中有 80% 比 Trino 更快；
- 在开启文件缓存功能并命中的情况下，Apache Doris 的总体性能可以进一步提升 2.2 倍以上，较 Trino 有 4 倍以上的性能提升，全部 99 条 SQL 性能均优于 Trino。

与此同时也在 TPC-DS 10TB 场景下对 Apache Doris 2.1 版本与 Spark 3.5.0 以及 3.3.1 版本进行了性能测试，查询性能分别提升 4.2 倍和 6.1 倍。

8.4.12.3.2 多 SQL 方言兼容

当用户从原有 OLAP 系统（如 Clickhouse、Trino、Presto、Hive 等）迁移至 Apache Doris 时，一方面因为 SQL 方言存在差异，需要同步修改大量的业务查询逻辑进行适配，无法进行平滑迁移。另一方面，当使用 Apache Doris 作为统一数据分析网关时，需要对接原先的 Hive、Spark 等系统、以满足不同数据源的查询需求。

因此在 Apache Doris 2.1 版本中我们引入了多 SQL 方言转换功能，用户可以直接使用原先系统的 SQL 方言在 Doris 中进行数据查询而无需修改业务逻辑。在部署好 SQL 转换服务后，用户只需通过会话变量 `sql_dialect`设置当前会话的 SQL 方言类型，即可使用对应的 SQL 方言进行查询。

该功能目前为实验性质功能，当前已经支持 ClickHouse、Presto、Trino、Hive、Spark。在此我们以 Trino 为例，部署完 SQL 转换服务后，在会话变量中设置 `set sql_dialect = trino`，即可直接采取 Trino SQL 语法执行查询。在某些社区用户的实际线上业务 SQL 兼容性测试中，在全部 3w 多条查询语句中与 Trino SQL 兼容度高达 99% 以上。也欢迎所有用户在使用过程中向我们反馈不兼容的 Case，帮助 Apache Doris 更加完善。

- 演示 Demo
- 参考文档：SQL 方言兼容

8.4.12.3.3 高速数据读取，数据传输效率提升 100 倍

如今许多大数据系统都采取列式内存数据格式，以 MySQL/JDBC/ODBC 作为与数据库系统交互的主流协议与标准。在数据输出至外部系统的过程中，需要将数据从系统特定的列存格式序列化为 MySQL/JDBC/ODBC 协议的行存格式，再反序列化回客户端的列存格式，这会使数据传输速度大幅降低，在面向数据科学或其他形式的大规模数据读写时，数据传输的效率缺陷愈发明显。

作为用于大规模数据处理的列式内存格式，Apache Arrow 提供了高效的数据结构、允许不同系统间更快共享数据。如果源数据库和目标客户端都支持 Apache Arrow 作为列式内存格式，使用 Arrow Flight SQL 协议传输将无需序列化和反序列化数据，消除数据传输中的开销。同时 Arrow Flight 还可以利用多节点和多核架构，通过完全并行化优化吞吐能力。

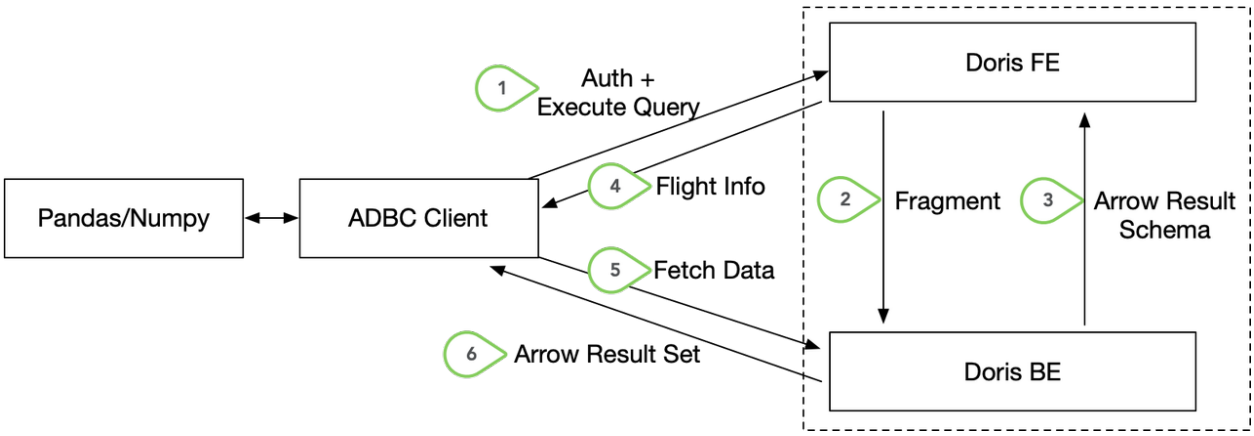


图 257: 高速数据读取，数据传输效率提升 100 倍

在过去如果需要采取 Python 读取 Apache Doris 中的数据，需要将 Apache Doris 中列存的 Block 序列化为 MySQL 协议的行存 Bytes，然后在 Python 客户端再反序列化到 Pandas 中，传输过程带来的性能损耗非常大。

在 Apache Doris 2.1 版本中，我们提供了基于 Arrow Flight 的 HTTP Data API 高吞吐数据读写接口。相比于过去的 MySQL 协议，使用 Arrow Flight SQL 后，我们在 Apache Doris 中先将列存的 Block 转为同样列存的 Arrow RecordBatch，这一步转换效率非常高、且传输过程中无需再次序列化和反序列化，而后在 Python 客户端再将 Arrow RecordBatch 转到同样列存的 Pandas DataFrame 中，这一步转换同样非常快。通过 Arrow Flight 提供的 Python 客户端 Pandas/Numpy 等数据科学工具，可以快速从 Apache Doris 中读取数据并在本地进行分析。

基于此，Apache Doris 可以与整个 AI 和数据科学生态进行良好的整合，这也是未来的重要发展方向。

```
conn = flight_sql.connect(uri="grpc://{FE_HOST}:{fe.conf:arrow_flight_sql_port}", db_kwargs={
            adbc_driver_manager.DatabaseOptions.USERNAME.value: "user",
            adbc_driver_manager.DatabaseOptions.PASSWORD.value: "pass",
        })
cursor = conn.cursor()
cursor.execute("select * from arrow_flight_sql_test order by k0;")
print(cursor.fetchallarrow().to_pandas())
```

针对常见的数据类型，我们通过不同的 MySQL 客户端进行了对比测试，基于 Arrow Flight SQL 数据传输性能相较于 MySQL 协议提升了近百倍。

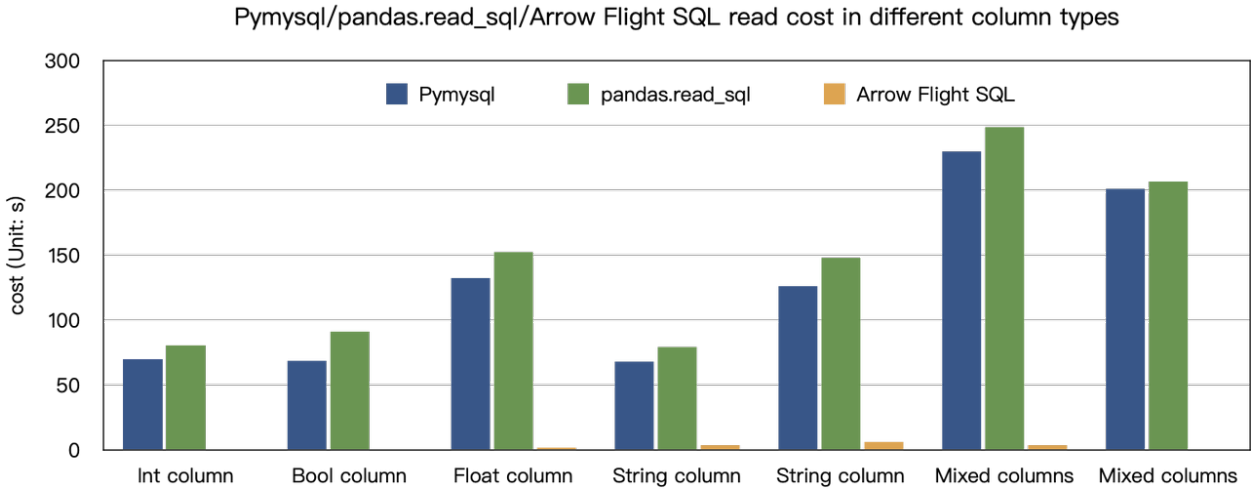


图 258: Arrow Flight SQL

演示 Demo：https://www.bilibili.com/video/BV1mj421Z7b7/?spm_id_from=333.999.0.0

8.4.12.3.4 其他

- Paimon Catalog：Paimon 版本升级至 0.6.0，优化了 Read Optimized 表的读取，在 Paimon 数据充分合并的场景下，可以有 10 倍的性能提升；
- Iceberg Catalog：Iceberg 版本升级至 1.4.3，同时解决了 AWS S3 认证的若干兼容性问题；
- Hudi Catalog：Hudi 版本升级至 0.14.1，同时解决了 Hudi Flink Catalog 的若干兼容性问题。

8.4.12.4 多表物化视图

作为一种典型的“空间换时间”策略，物化视图通过预先计算和存储 SQL 查询结果，当执行相同查询时可以直接从物化视图表中获取结果，在大幅提升查询性能的同时、更是减少重复计算带来的系统资源消耗。

在过去版本中 Apache Doris 提供了强一致的单表物化视图、保证基表和物化视图表的原子性，并支持了查询语句在物化视图上的智能路由。

在 Apache Doris 2.1 版本中，我们引入了全新的异步物化视图，可以基于多表来构建。 异步物化视图可以全量或者分区增量构建，也可以手动或者周期性地构建刷新数据。在多表关联查询且表数据量较大的场景下，优化器会根据代价模型进行透明改写、并自动寻找最优物化视图来响应查询，以大幅提升查询性能。与此同时，也提供了从外表到内表的物化视图以及直查物化视图的能力，基于此特性，异步物化视图也可用于数据仓库分层建模、作业调度和数据加工。异步物化视图使用方式如下：

表定义：

```
use tpch;

CREATE TABLE IF NOT EXISTS orders  (
    o_orderkey       integer not null,
    o_custkey        integer not null,
    o_orderstatus    char(1) not null,
    o_totalprice     decimalv3(15,2) not null,
    o_orderdate      date not null,
    o_orderpriority  char(15) not null,
    o_clerk          char(15) not null,
    o_shippriority   integer not null,
    o_comment        varchar(79) not null
    )
    DUPLICATE KEY(o_orderkey, o_custkey)
    PARTITION BY RANGE(o_orderdate)(
    FROM ('2023-10-17') TO ('2023-10-20') INTERVAL 1 DAY)
    DISTRIBUTED BY HASH(o_orderkey) BUCKETS 3
    PROPERTIES ("replication_num" = "1");

insert into orders values
   (1, 1, 'ok', 99.5, '2023-10-17', 'a', 'b', 1, 'yy'),
   (2, 2, 'ok', 109.2, '2023-10-18', 'c','d',2, 'mm'),
   (3, 3, 'ok', 99.5, '2023-10-19', 'a', 'b', 1, 'yy');

CREATE TABLE IF NOT EXISTS lineitem (
    l_orderkey    integer not null,
    l_partkey     integer not null,
    l_suppkey     integer not null,
    l_linenumber  integer not null,
    l_quantity    decimalv3(15,2) not null,
    l_extendedprice  decimalv3(15,2) not null,
    l_discount    decimalv3(15,2) not null,
    l_tax         decimalv3(15,2) not null,
    l_returnflag  char(1) not null,
    l_linestatus  char(1) not null,
    l_shipdate    date not null,
```

```
    l_commitdate  date not null,
    l_receiptdate date not null,
    l_shipinstruct char(25) not null,
    l_shipmode     char(10) not null,
    l_comment      varchar(44) not null
    )
    DUPLICATE KEY(l_orderkey, l_partkey, l_suppkey, l_linenumber)
    PARTITION BY RANGE(l_shipdate)
    (FROM ('2023-10-17') TO ('2023-10-20') INTERVAL 1 DAY)
    DISTRIBUTED BY HASH(l_orderkey) BUCKETS 3
    PROPERTIES ("replication_num" = "1");

insert into lineitem values
 (1, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-17', '2023-10-17', '2023-10-17', 'a', 'b', '
    ↪ yyyyyyyyy'),
 (2, 2, 3, 4, 5.5, 6.5, 7.5, 8.5, 'o', 'k', '2023-10-18', '2023-10-18', '2023-10-18', 'a', 'b', '
    ↪ yyyyyyyyy'),
 (3, 2, 3, 6, 7.5, 8.5, 9.5, 10.5, 'k', 'o', '2023-10-19', '2023-10-19', '2023-10-19', 'c', 'd',
    ↪ 'xxxxxxxxx');


 CREATE TABLE IF NOT EXISTS partsupp (
  ps_partkey     INTEGER NOT NULL,
  ps_suppkey     INTEGER NOT NULL,
  ps_availqty    INTEGER NOT NULL,
  ps_supplycost  DECIMALV3(15,2)  NOT NULL,
  ps_comment     VARCHAR(199) NOT NULL
)
DUPLICATE KEY(ps_partkey, ps_suppkey)
DISTRIBUTED BY HASH(ps_partkey) BUCKETS 3
PROPERTIES (
  "replication_num" = "1"
)
```

创建物化视图：

```
CREATE MATERIALIZED VIEW mv1
        BUILD DEFERRED REFRESH AUTO ON MANUAL
        partition by(l_shipdate)
        DISTRIBUTED BY RANDOM BUCKETS 2
        PROPERTIES ('replication_num' = '1')
        AS
        select l_shipdate, o_orderdate, l_partkey,
            l_suppkey, sum(o_totalprice) as sum_total
            from lineitem
            left join orders on lineitem.l_orderkey = orders.o_orderkey
```

```
                    and l_shipdate = o_orderdate
        group by
        l_shipdate,
        o_orderdate,
        l_partkey,
        l_suppkey;
```

目前异步物化视图已经具备以下功能：

- 透明改写加速：支持常见算子的透明改写，如 Select、Where、Join、Group by、Aggregation 等，可以直接通过建立物化视图，对现有的查询进行加速。例如在 BI 报表场景，某些报表查询延时比较高，就可以通过建立合适的物化视图进行加速。
- 自动刷新：物化视图支持不同刷新策略，如定时刷新和手动刷新，也支持不同的刷新粒度，如全量刷新、分区粒度的增量刷新等。
- 外表到内表的物化视图：可以对存放在 Hive、Hudi、Iceberg 等数据湖系统上的数据建立物化视图，加速对数据湖的访问，也可以通过物化视图的方式将数据湖中的数据同步到 Apache Doris 内表中。
- 物化视图直查：用户也可以将物化视图的构建看做 ETL 的过程，把物化视图看做是 ETL 加工后的结果数据，由于物化视图本身也是一个表，所以用户可以直接查询物化视图。

> - 演示 Demo: https://www.bilibili.com/video/BV1s2421T71z/?spm_id_from=333.999.0.0
> - 参考文档：异步物化视图

8.4.12.5 存储能力增强

8.4.12.5.1 自增列 AUTO_INCREMENT

自增列 AUTO_INCREMENT 是 OLTP 数据库中常见的一项功能，提供了一种方便高效的方式来为新插入的数据行自动分配唯一标识符。由于自增列的可用值分配涉及到全局事务，因此在分布式 OLAP 数据库中并不常见。在 Apache Doris 2.1 版本中，我们通过创新性的自增序列预分配策略，提供了高效的自增列实现。基于自增列的唯一性保证，用户可以利用自增列实现高效的字典编码和查询分页。

字典编码： 在进行 PV/UV 统计或人群圈选等需要精确去重的查询时，可以使用自增列对 UserID 或订单 ID 等字符串值创建字典表，将用户数据批量或者实时写入字典表即可生成字典，根据各种维度的条件对对应的 Bitmap 进行聚合运算；

```
CREATE TABLE `demo`.`dictionary_tbl` (
    `user_id` varchar(50) NOT NULL,
    `aid` BIGINT NOT NULL AUTO_INCREMENT
) ENGINE=OLAP
UNIQUE KEY(`user_id`)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 32
PROPERTIES (
```

```
"replication_allocation" = "tag.location.default: 3",
"enable_unique_key_merge_on_write" = "true"
);
```

查询分页：在页面展示数据时，往往需要做分页展示。传统的分页通常使用 SQL 中的 limit, offset + order by 进行查询。在进行深分页查询时，即使查询数据量较少、数据库仍需将全部数据读取至内存进行全量排序，查询效率比较低下。采取自增列可以为每一行生成唯一标识、查询时记住上一页最大唯一标识并用于下一页的查询条件，实现更高效的分页查询。

以下表为例，unique_value 是一个唯一值：

```
CREATE TABLE `demo`.`records_tbl2` (
    `key` int(11) NOT NULL COMMENT "",
    `name` varchar(26) NOT NULL COMMENT "",
    `address` varchar(41) NOT NULL COMMENT "",
    `city` varchar(11) NOT NULL COMMENT "",
    `nation` varchar(16) NOT NULL COMMENT "",
    `region` varchar(13) NOT NULL COMMENT "",
    `phone` varchar(16) NOT NULL COMMENT "",
    `mktsegment` varchar(11) NOT NULL COMMENT "",
    `unique_value` BIGINT NOT NULL AUTO_INCREMENT
) DUPLICATE KEY (`key`, `name`)
DISTRIBUTED BY HASH(`key`) BUCKETS 10
PROPERTIES (
    "replication_num" = "3"
);
```

在分页展示中，每页展示 100 条数据，使用如下方式获取第一页的数据：

```
select * from records_tbl2 order by unique_value limit 100;
```

通过程序记录下返回结果中unique_value中的最大值，假设为 99，则可用如下方式查询第二页的数据：

```
select * from records_tbl2 where unique_value > 99 order by unique_value limit 100;
```

如果要直接查询一个靠后页面的内容，此时不方便直接获取之前页面数据中unique_value的最大值时，例如要直接获取第 101 页的内容，则可以使用如下方式进行查询

```
select key, name, address, city, nation, region, phone, mktsegment
from records_tbl2, (select unique_value as max_value from records_tbl2 order by uniuqe_value
    ↪ limit 1 offset 9999) as previous_data
where records_tbl2.uniuqe_value > previous_data.max_value
order by unique_value limit 100;
```

> 演示 Demo：https://www.bilibili.com/video/BV1VC411h7Gr/?spm_id_from=333.999.0.0

8.4.12.5.2 自动分区 Auto Partition

在 Apache Doris 2.1 版本之前一直采取手动分区的形式，用户需要提前把分区建立好，否则在导入数据过程中会因为找不到对应分区而出错。而自动分区功能支持了在导入数据过程中自动检测分区列的数据对应的分区是否存在。如果不存在，则会自动创建分区并正常进行导入。

自动分区功能使用方式如下：

```
CREATE TABLE `DAILY_TRADE_VALUE`
(
    `TRADE_DATE`              datev2 NULL COMMENT '交易日期',
    `TRADE_ID`                varchar(40) NULL COMMENT '交易编号',
    ......
)
UNIQUE KEY(`TRADE_DATE`, `TRADE_ID`)
AUTO PARTITION BY RANGE date_trunc(`TRADE_DATE`, 'year')
(
)
DISTRIBUTED BY HASH(`TRADE_DATE`) BUCKETS 10
PROPERTIES (
  "replication_num" = "1"
);
```

> 注意事项
>
> 1. 当前自动分区功能仅支持一个分区列，并且分区列必须为 NOT NULL 列；
> 2. 自动分区当前已支持 Range 分区和 List 分区，其中 Range 分区函数仅支持 date_trunc ↪ 、分区列仅支持 DATE 或者 DATETIME 格式；List 分区不支持函数调用，分区列支持 BOOLEAN、TINYINT、SMALLINT、INT、BIGINT、LARGEINT、DATE、DATETIME、CHAR、↪ VARCHAR 数据类型，分区值为枚举值；
> 3. 使用 List 分区时，一旦分区列的值当前不存在，自动分区功能都会为其创建一个独立的新分区。

> 参考文档：数据划分

8.4.12.5.3 INSERT INTO SELECT 导入性能提升 100%

INSERT INTO…SELECT 语句是 ETL 中最高频使用的操作之一，可以快速完成数据在库表之间的迁移、转换以及清洗合并，提升 INSERT INTO…SELECT 性能可以更好满足用户对数据快速提取和分析的需求。在 Apache Doris 2.0 版本中，我们引入了单副本导入功能（Single Replica Load）来减少多副本的重复写入和 Compaction 工作，但是导入性能还存在优化的空间。

在 Apache Doris 2.1 版本中，为了进一步提升INSERT INTO…SELECT 性能，我们实现了 MemTable 前移以进一步减少导入过程中的开销，能在大多数场景中能在 2.0 版本的基础上取得 100% 的导入性能提升。

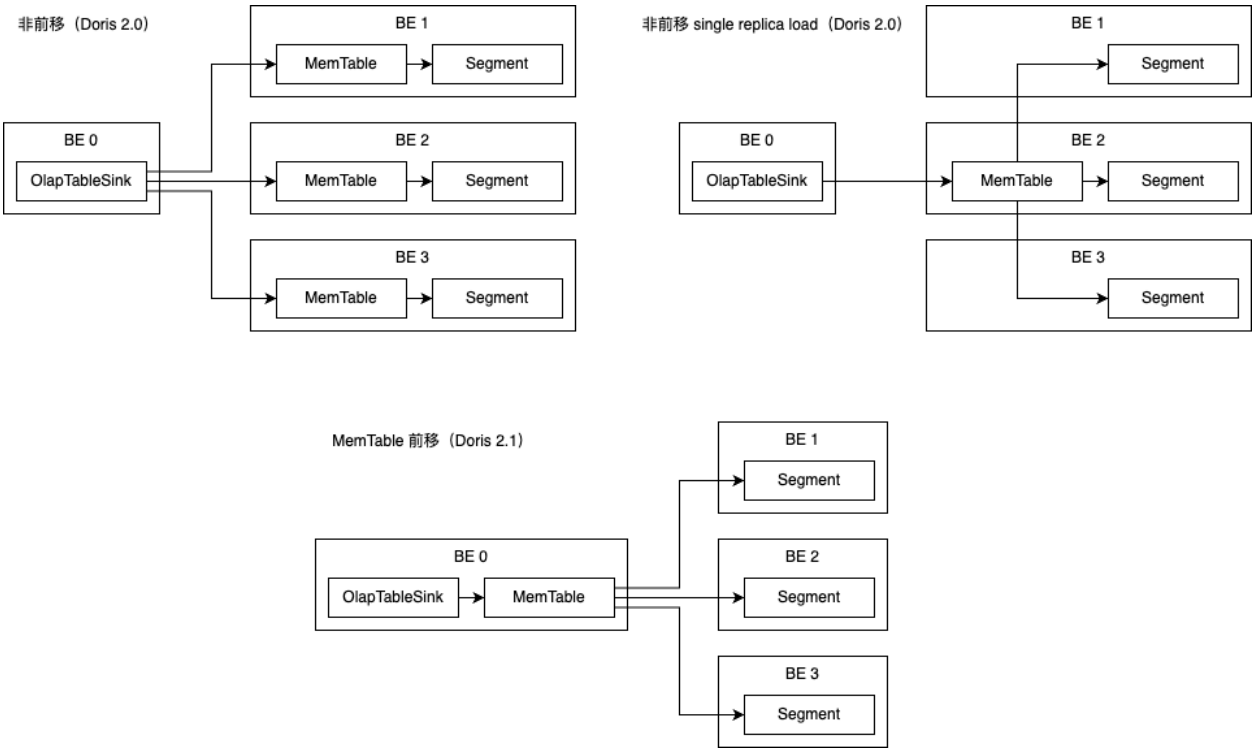


图 259: INSERT INTO SELECT 导入性能提升 100%

MemTable 前移和非前移的流程对比如上图所示，Sink 节点不再发送编码后的 Block，而是在本地处理完 MemTable 将生成的 Segment 数据发给下游节点，减少了数据多次编码的开销，同时使内存反压更准确和及时。此外，我们使用了 Streaming RPC 来替代了 Ping-pong RPC，减少了数据传输过程中的等待。

在此我们对 2.1 版本开启 MemTable 前移后的导入性能进行了测试，测试环境如下：1 FE+3 BE、每个节点 16C 64G、3 块高性能云盘（保证磁盘 I/O 不成为瓶颈）

可以看到在单副本场景下，2.1 版本开启 MemTable 前移后、导入耗时降低至 2.0 版本的 36%，三副本场景下导入耗时降低至 2.0 版本的 54%，整体导入性能提升超过 100%。

INSERT INTO 表	Doris 2.0 非前移默认	Doris 2.1MemTable 前移
linitem 1 副本 (38G)	30.2 s	11.1 s
linitem 3 副本 (38G)	47.4 s	25.4 s

INSERT INTO 表	Doris 2.0 非前移 默认	Doris 2.1 MemTable 前移
linitem 1 副本 (38G)	30.2 s	11.1 s
linitem 3 副本 (38G)	47.4 s	25.4 s

图 260: INSERT INTO SELECT 导入性能提升 100%

MemTable 前移在 2.1 版本中默认开启，用户无需修改原有的导入命令即可获得大幅性能提升。如果在使用过程中遇到问题、希望回退到原有的导入方式，可以在 MySQL 连接中设置环境变量 `enable_memtable_on_sink_node=false` 来关闭 MemTable 前移。

8.4.12.5.4 高频实时导入/服务端攒批 Group Commit

在数据导入过程中，不同批次导入的数据都会写入内存表中，随后在磁盘中上形成一个个 RowSet 文件，每个 Rowset 文件对应一次数据导入版本。后台 Compaction 进程会自动对多个版本的 RowSet 文件进行合并，将多个 RowSet 小文件合并成 RowSet 大文件以优化查询性能以及存储空间，而每一次的 Compaction 进程都会产生对 CPU、内存以及磁盘 IO 资源的消耗。在实际数据写入场景中，写入越实时高频、生成 RowSet 版本数越高、Compaction 所消耗的资源就越大。为了避免高频写入带来的过多资源消耗甚至 OOM，Apache Doris 引入了反压机制，即在版本过多的情况下会返回 -235，并对数据的版本数量进行控制。

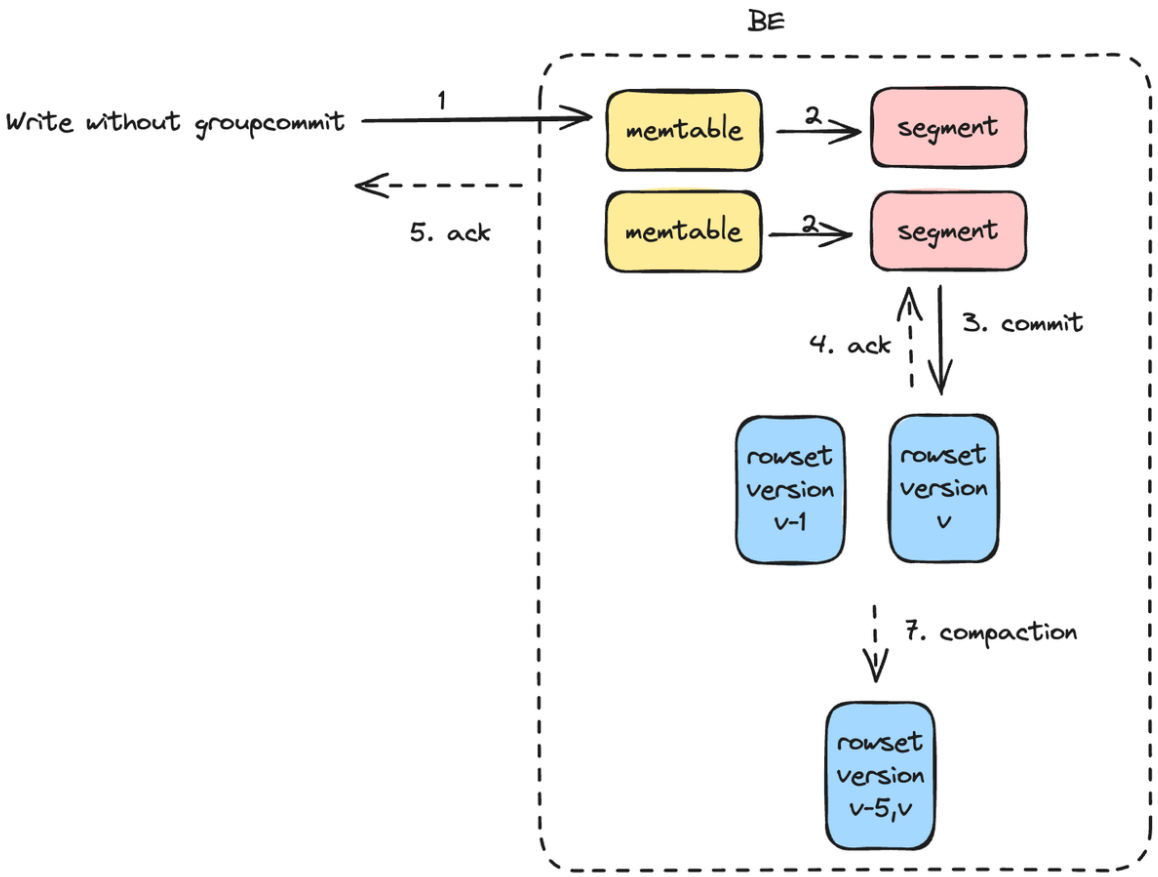


图 261: 高频实时导入/服务端攒批 Group Commit

从 Apache Doris 2.1 版本开始，我们引入了服务端攒批 Group Commit，大幅强化了高并发、高频实时写入的能力。

顾名思义，Group Commit 会把用户侧的多次写入在 BE 端进行积攒后批量提交。对于用户而言，无需控制写入程序的频率，Doris 会自动把用户提交的多次写入在内部合并为一个版本，从而可以大幅提升用户侧的写入频次。

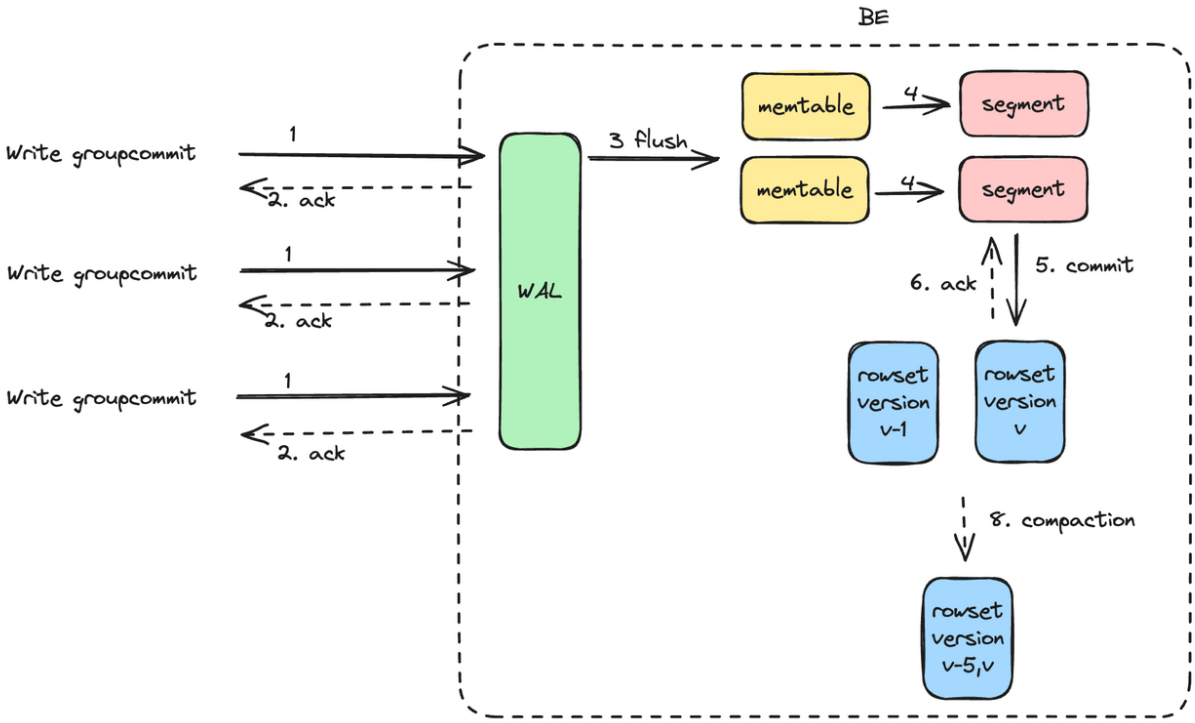


图 262: 高频实时导入/服务端攒批 Group Commit

当前 Group Commit 已经支持同步模式 sync_mode 和异步模式 async_mode。同步模式下会将多个导入在一个事务提交，事务提交后导入返回，在导入完成后数据立即可见。异步模式下数据会先写入 WAL，Apache Doris 会根据负载和表的group_commit_interval属性异步提交数据，提交之后数据可见。为了防止 WAL 占用较大的磁盘空间，单次导入数据量较大时，会自动切换为sync_mode。

我们分别采取 JDBC 和 Stream Load 两种方式对高并发写入场景下 Group Commit（异步模式 async_mode）的写入性能进行了测试，测试报告如下：

- JDBC 写入：
- 集群配置为 1FE 1BE，数据集为 TPC-H SF10 Lineitem 表，总共约 22GB、1.8 亿行；
- 经测试，在并发数 20、单次 Insert 数据行数 100 行下，导入效率达到 10.69w 行/秒、导入吞吐达 11.46 MB/秒，BE 节点的 CPU 使用率稳定保持在 10%-20%；
- Stream Load 写入：
- 集群配置为 1FE 3BE，数据集为 httplogs、总共 31GB、2.47 亿行。在未开启 Group Commit 和开启 Group Commit 的异步模式时，通过设置不同的单并发数据量和并发数，对比数据的写入性能。
- 经测试，在并发数 10、单次导入数据量 1 MB 下，未开启 Group Commit 时会提示 -235 错误，开启后可稳定运行且导入效率达 81w 行/秒、导入吞吐达 104 MB/秒；在并发数 10、单次导入数据量 10MB 下，开启 Group Commit 后耗时降低至原先的 55%、导入吞吐提升 79%；

- 演示 Demo：https://www.bilibili.com/video/BV1um411o7Ha/?spm_id_from=333.999.0.0
- 参考文档和完整测试报告：Group Commit

8.4.12.6 半结构化数据分析

8.4.12.6.1 Variant 数据类型

过去 Apache Doris 在应对复杂半结构化数据的存储和分析处理时，一般有两种方式：

1. 一种方式是用户提前预定好表结构，加工成宽表，在数据进入前将数据解析好，这种方案的优点是写入性能好，查询也不需要解析，但是使用不够灵活、对表结构发起变更增加运维、研发的成本。
2. 使用 Doris 中的 JSON 类型、或是存成 JSON String，将原始 JSON 数据不经过加工直接入库，查询的时候，用解析函数处理。优点是不需要额外的数据加工、预定义表结构拍平嵌套结构，运维、研发方便，但存在解析性能以及数据读取效率低下的问题。

为了解决上述半结构化数据的挑战，在 Apache Doris 2.1 版本中我们引入全新的数据类型VARIANT，支持存储半结构化数据、允许存储包含不同数据类型（如整数、字符串、布尔值等）的复杂数据结构，无需在表结构中提前定义具体的列，其存储和查询与传统的 String、JSONB 等行存类型发生了本质的改变，期望其作为半结构化数据首选数据类型，给用户带来更加高效的数据处理机制。

Variant 类型特别适用于处理结构可能随时会发生变化的复杂嵌套结构。在写入过程中，Variant 类型可以自动根据列的结构和类型推断列信息，并将其合并到现有表的 Schema 中，将 JSON 键及其对应的值灵活存储为动态子列。同时，一个表可以同时包含灵活的 Variant 对象列和预先定义类型的更严格的静态列，从而在数据存储、查询上提供了更大的灵活性。除此之外，Variant 类型能够与 Doris 核心特性融合，利用列式存储、向量化引擎、优化器等技术，为用户带来极高性价比的查询性能及存储性能。

使用方式如下：

```
-- 无索引
CREATE TABLE IF NOT EXISTS ${table_name} (
    k BIGINT,
    v VARIANT
)
table_properties;

-- 在v列创建索引，可选指定分词方式，默认不分词
CREATE TABLE IF NOT EXISTS ${table_name} (
    k BIGINT,
    v VARIANT,
    INDEX idx_var(v) USING INVERTED [PROPERTIES("parser" = "english|unicode|chinese")] [COMMENT '
        ↪ your comment']
)
table_properties;
```

```
-- 查询，使用`[]`形式访问子列
SELECT v["properties"]["title"] from ${table_name}
```

相比 JSON 类型的优势

在 Apache Doris 中 JSON 类型是以二进制 JSONB 格式进行存储，整行 JSON 以行存的形式存储到 Segment 文件中。而 VARIANT 类型在写入的时候进行类型推断，将写入的 JSON 列存化，查询不需要进行解析。此外 Variant 类型针对稀疏场景的 JSON 进行优化，只提取频繁出现的列，稀疏的列会以单独的格式进行存储。

为了验证引入 Variant 数据类型后在存储以及查询上所带来的优势，我们基于 ClickBench 测试数据集进行了存储空间和查询性能的测试。

在存储空间方面，相同数据采取 Variant 类型，所占用的存储空间跟预定义的静态列的存储空间持平，相比于 JSON 类型则减少了约 65%。在一些低基数场景，由于列存的优势，存储资源的成本效应会更加明显。

	存储空间
预定义静态列	12.618 GB
Variant 类型	12.718 GB
JSON 类型	35.711 GB

图 263: 相比 JSON 类型的优势

在查询性能方面，如下表可知，Variant 类型与预定义静态列的查询性能差异在 10% 左右；而对于 JSON 类型来说，Variant 类型的热查询速度相比于 JSON 类型提升了 8 倍以上，冷查询有着数量级的提升。（由于 I/O 原因，JSONB 类型的冷查询大部分超时）。

	第一次查询 (Cold Run)	第二次查询 (Hot Run)	第三次查询 (Hot Run)
预定义静态列	233.79s	86.02s	83.03s
Variant 类型	248.66s	94.82s	92.29s
JSON 类型	大部分查询超时	789.24s	743.69s

图 264: 相比 JSON 类型的优势

注意事项：

- 目前 Variant 暂不支持 Aggregate 模型，也不支持将 Variant 类型作为 Unique 或 Duplicate 模型的主键及排序键；
- 推荐使用 RANDOM 模式或者开启 Group Commit 导入，写入性能更高效；
- 日期、 Decimal 等非标准 JSON 类型尽可能提取出来作为静态字段，性能更好；
- 二维及其以上的数组以及数组嵌套对象，列存化会被存成 JSONB 编码，性能不如原生数组；
- 查询过滤、聚合需要带 Cast，存储层会根据存储类型和 Cast 目标类型来提示（hint）存储引擎谓词下推，加速查询。

- 演示 Demo: https://www.bilibili.com/video/BV13u4m1g7ra/?spm_id_from=333.999.0.0
- 参考文档：VARIANT

8.4.12.6.2 IP 数据类型

在网络流量监控的场景中，IP 地址是一个常见的字段，大量的统计分析基于 IP 地址进行。在 Apache Doris 2.1 版本中，将原生支持 IPv4 和 IPv6 数据类型，用高效的二进制形式存储 IP 数据。相比于使用明文的 IP String，内存和存储空间可节省 60% 左右。

同时基于 IP 类型，我们增加了常用的 20 多个 IP 处理函数，如：

- IPV4_NUM_TO_STRING：将类型为 Int16、 Int32、 Int64 且大端表示的 IPv4 的地址，返回相应 IPv4 的字符串表现形式；

- IPV4_CIDR_TO_RANGE：接收一个 IPv4 和一个包含 CIDR 的 Int16 值，返回一个结构体，其中包含两个 IPv4 字段分别表示子网的较低范围（min）和较高范围（max）;
- INET_ATON：获取包含 IPv4 地址的字符串，格式为 A.B.C.D（点分隔的十进制数字）

> 参考文档：IPV6

8.4.12.6.3 复杂数据类型分析函数完善

在 Apache Doris 2.1 版本中我们丰富了行转列和 IN 能支持的数据类型。如：

- explode_map：支持 MAP 类型数据行转列（仅在新优化器中实现）

支持 Map 类型 Explode 行转列，将 Map 字段的 N 个 Key Value 对展开成 N 行，每行的 Map 字段替换成 Key 和 Value 两个字段。explode_map 需要和 Lateral View 一起使用，可以接多个 Lateral View, 结果则是每个 explode_map 之后的行数以笛卡尔积的形式展示。

具体使用如下：

```
-- 建表语句
 CREATE TABLE `sdu` (
  `id` INT NULL,
  `name` TEXT NULL,
  `score` MAP<TEXT,INT> NULL
) ENGINE=OLAP
DUPLICATE KEY(`id`)
COMMENT 'OLAP'
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
"replication_allocation" = "tag.location.default: 1"
);

-- insert 数据
insert into sdu values (0, "zhangsan", {"Chinese":"80","Math":"60","English":"90"});
insert into sdu values (1, "lisi", {"null":null});
insert into sdu values (2, "wangwu", {"Chinese":"88","Math":"90","English":"96"});
insert into sdu values (3, "lisi2", {null:null});
insert into sdu values (4, "amory", NULL);

mysql> select name, course_0, score_0 from sdu lateral view explode_map(score) tmp as course_0,
    ↪ score_0;
+----------+----------+---------+
| name     | course_0 | score_0 |
+----------+----------+---------+
| zhangsan | Chinese  |      80 |
```

```
| zhangsan | Math     |      60 |
| zhangsan | English  |      90 |
| lisi     | null     |    NULL |
| wangwu   | Chinese  |      88 |
| wangwu   | Math     |      90 |
| wangwu   | English  |      96 |
| lisi2    | NULL     |    NULL |
+----------+----------+---------+

mysql> select name, course_0, score_0, course_1, score_1 from sdu lateral view explode_map(score)
    ↪  tmp as course_0,score_0 lateral view explode_map(score) tmp1 as course_1,score_1;
+----------+----------+---------+----------+---------+
| name     | course_0 | score_0 | course_1 | score_1 |
+----------+----------+---------+----------+---------+
| zhangsan | Chinese  |      80 | Chinese  |      80 |
| zhangsan | Chinese  |      80 | Math     |      60 |
| zhangsan | Chinese  |      80 | English  |      90 |
| zhangsan | Math     |      60 | Chinese  |      80 |
| zhangsan | Math     |      60 | Math     |      60 |
| zhangsan | Math     |      60 | English  |      90 |
| zhangsan | English  |      90 | Chinese  |      80 |
| zhangsan | English  |      90 | Math     |      60 |
| zhangsan | English  |      90 | English  |      90 |
| lisi     | null     |    NULL | null     |    NULL |
| wangwu   | Chinese  |      88 | Chinese  |      88 |
| wangwu   | Chinese  |      88 | Math     |      90 |
| wangwu   | Chinese  |      88 | English  |      96 |
| wangwu   | Math     |      90 | Chinese  |      88 |
| wangwu   | Math     |      90 | Math     |      90 |
| wangwu   | Math     |      90 | English  |      96 |
| wangwu   | English  |      96 | Chinese  |      88 |
| wangwu   | English  |      96 | Math     |      90 |
| wangwu   | English  |      96 | English  |      96 |
| lisi2    | NULL     |    NULL | NULL     |    NULL |
+----------+----------+---------+----------+---------+
```

explode_map_outer 和 explode_outer 的目的一致，可以将当前 MAP 类型的列中是 NULL 的数据行展示出来。

```
mysql> select name, course_0, score_0 from sdu lateral view explode_map_outer(score) tmp as
    ↪ course_0,score_0;
+----------+----------+---------+
| name     | course_0 | score_0 |
+----------+----------+---------+
| zhangsan | Chinese  |      80 |
| zhangsan | Math     |      60 |
| zhangsan | English  |      90 |
```

```
| lisi     | null     |    NULL |
| wangwu   | Chinese  |      88 |
| wangwu   | Math     |      90 |
| wangwu   | English  |      96 |
| lisi2    | NULL     |    NULL |
| amory    | NULL     |    NULL |
+----------+----------+---------+
```

- 增加 IN 谓词支持 Struct 类型数据的能力（仅在新优化器中实现）

IN 谓词的左参数支持 `struct()function` 构建的 Struct 类型的数据，也支持 Select 某表中某一列是 Struct 类型的数据，右边的参数支持一个由 `struct()function` 构建的 Struct 类型的数据的数组。IN 谓词支持 Struct 类型可以有效替换 Where 条件中如果需要大量的 or 连词连接的表达式，如：`(a = 1 and b = '2')or (a = 1 and b = '3')or (...)` 可以通过 IN 实现为 `struct(a,b)in (struct(1, '2'), struct(1, '3'), ...)`

```
mysql> select struct(1,"2")  in (struct(1,3), struct(1,"2"), struct(1,1), null);
+--
    ↪ ----------------------------------------------------------------------------------------------------------
    ↪
| cast(struct(1, '2') as STRUCT<col1:TINYINT,col2:TEXT>) IN (NULL, cast(struct(1, '2') as STRUCT<
    ↪ col1:TINYINT,col2:TEXT>), cast(struct(1, 1) as STRUCT<col1:TINYINT,col2:TEXT>), cast(
    ↪ struct(1, 3) as STRUCT<col1:TINYINT,col2:TEXT>)) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 1 |
+--
    ↪ ----------------------------------------------------------------------------------------------------------
    ↪
mysql> select struct(1,"2") not in (struct(1,3), struct(1,"2"), struct(1,1), null);
+--
    ↪ ----------------------------------------------------------------------------------------------------------
    ↪
| ( not cast(struct(1, '2') as STRUCT<col1:TINYINT,col2:TEXT>) IN (NULL, cast(struct(1, '2') as
    ↪ STRUCT<col1:TINYINT,col2:TEXT>), cast(struct(1, 1) as STRUCT<col1:TINYINT,col2:TEXT>),
    ↪ cast(struct(1, 3) as STRUCT<col1:TINYINT,col2:TEXT>))) |
+--
    ↪ ----------------------------------------------------------------------------------------------------------
    ↪
|
    ↪
    ↪ 0 |
```

```
+--
    ↪ ----------------------
```

8.4.12.7.2 TopSQL

> 自 2.1.1 版本之后，active_queries() 已经废弃，TopSQl 主要通过 Doris 内置的系统表实现，参考文档工作负载诊断与分析

当集群出现预期外的大查询导致集群整体负载上升、查询可用性下降时，用户难以快速找到这些大查询并进行相应的降级操作。因此在 Apache Doris 2.1 版本中我们支持了运行时查看 SQL 资源用量的功能，具体指标如下：

```
mysql [(none)]>desc function active_queries();
+------------------------+--------+------+-------+---------+-------+
| Field                  | Type   | Null | Key   | Default | Extra |
+------------------------+--------+------+-------+---------+-------+
| BeHost                 | TEXT   | No   | false | NULL    | NONE  |
| BePort                 | BIGINT | No   | false | NULL    | NONE  |
| QueryId                | TEXT   | No   | false | NULL    | NONE  |
| StartTime              | TEXT   | No   | false | NULL    | NONE  |
| QueryTimeMs            | BIGINT | No   | false | NULL    | NONE  |
| WorkloadGroupId        | BIGINT | No   | false | NULL    | NONE  |
| QueryCpuTimeMs         | BIGINT | No   | false | NULL    | NONE  |
| ScanRows               | BIGINT | No   | false | NULL    | NONE  |
| ScanBytes              | BIGINT | No   | false | NULL    | NONE  |
| BePeakMemoryBytes      | BIGINT | No   | false | NULL    | NONE  |
| CurrentUsedMemoryBytes | BIGINT | No   | false | NULL    | NONE  |
| ShuffleSendBytes       | BIGINT | No   | false | NULL    | NONE  |
| ShuffleSendRows        | BIGINT | No   | false | NULL    | NONE  |
| Database               | TEXT   | No   | false | NULL    | NONE  |
| FrontendInstance       | TEXT   | No   | false | NULL    | NONE  |
| Sql                    | TEXT   | No   | false | NULL    | NONE  |
+------------------------+--------+------+-------+---------+-------+
```

active_queries() 函数记录了查询在各个 BE 上运行时的审计信息，该函数可以当做普通的 Doris 表来看待，支持查询、谓词过滤、排序和 Join 等操作。常用的指标包括 SQL 的运行时间、CPU 时间、单 BE 的峰值内存、Scan 的数据量以及 Shuffle 的数据量，也可以从 BE 的粒度做上卷，查看 SQL 全局的资源用量。

需要注意的是这里只显示运行时的 SQL，查询结束的 SQL 不会在这里显示，而是写入审计日志中（目前主要是 fe.audit.log ）。常用的 SQL 如下：

查看集群中目前运行时间最久的n个sql

```
select QueryId,max(QueryTimeMs) as query_time from active_queries() group by QueryId order by
    ↪ query_time desc limit 10;
```

查看目前集群中CPU耗时最长的n个sql

```
select QueryId, sum(QueryCpuTimeMs) as cpu_time from active_queries() group by QueryId order by
    ↪ cpu_time desc limit 10
```

查看目前集群中scan行数最多的n个sql以及他们的运行时间

```
select t1.QueryId,t1.scan_rows, t2.query_time from
    (select QueryId, sum(ScanRows) as scan_rows from active_queries()  group by QueryId order by
        ↪ scan_rows desc limit 10) t1
    left join (select QueryId,max(QueryTimeMs) as query_time from active_queries() group by
        ↪ QueryId) t2 on t1.QueryId = t2.QueryId
```

查看目前各个BE的负载情况，按照CPU时间/scan行数/shuffle字节数降序排列

```
select BeHost,sum(QueryCpuTimeMs) as query_cpu_time, sum(ScanRows) as scan_rows,sum(
    ↪ ShuffleSendBytes) as shuffle_bytes from active_queries() group by BeHost order by query_
    ↪ cpu_time desc,scan_rows desc ,shuffle_bytes desc limit 10
```

查看单BE峰值内存最高的N个sql

```
select QueryId,max(BePeakMemoryBytes) as be_peak_mem from active_queries() group by QueryId order
    ↪  by be_peak_mem desc limit 10;
```

目前主要展示的负载类型包括 Select 和Insert Into……Select，预计在 2.1 版本之上的三位迭代版本中会支持 Stream Load 和 Broker Load 的资源用量展示。

> 参考文档：ACTIVE_QUERIES

8.4.12.8 其他

8.4.12.8.1 Decimal256

为了更好的满足金融类或者财务类客户以及一些高端制造业客户对于数字类型进行精确计算的需求，2.1 新版本中提供了更高精度的 Decimal 数据类型，最高支持 76 位有效数字（该类型处于 Experimental 状态，需要手工开启配置项 set enable_decimal256=true 才能使用）。

示例：

```
CREATE TABLE `test_arithmetic_expressions_256` (
      k1 decimal(76, 30),
      k2 decimal(76, 30)
    )
    DISTRIBUTED BY HASH(k1)
    PROPERTIES (
    "replication_num" = "1"
    );

insert into test_arithmetic_expressions_256 values
  (1.000000000000000000000000000001,
      ↪ 99999999999999999999999999999999999999999999998.999999999999999999999999999998),
```

```
  (2.10000000000000000000000000001,
      ↪ 49999999999999999999999999999999999999999999999.89999999999999999999999999998),
  (3.66666666666666666666666666666,
      ↪ 33333333333333333333333333333333333333333333333.33333333333333333333333333333);
```

查询语句及结果：

```
select k1, k2, k1 + k2 a from test_arithmetic_expressions_256 order by 1, 2;
+--
    ↪ --------------------------------+-----------------------------------------------------------------------------------
    ↪
| k1                                 | k2
    ↪                                                                                                 | a
    ↪                                                                                                  |
+--
    ↪ --------------------------------+-----------------------------------------------------------------------------------
    ↪
| 1.00000000000000000000000000001 |
    ↪ 99999999999999999999999999999999999999999999998.99999999999999999999999999998 |
    ↪ 99999999999999999999999999999999999999999999999.99999999999999999999999999999 |
| 2.10000000000000000000000000001 |
    ↪ 49999999999999999999999999999999999999999999999.89999999999999999999999999998 |
    ↪ 50000000000000000000000000000000000000000000001.99999999999999999999999999999 |
| 3.66666666666666666666666666666 |
    ↪ 33333333333333333333333333333333333333333333333.33333333333333333333333333333 |
    ↪ 33333333333333333333333333333333333333333333336.99999999999999999999999999999 |
+--
    ↪ --------------------------------+-----------------------------------------------------------------------------------
    ↪
3 rows in set (0.09 sec)
```

> 注意事项 - Decimal256 类型对于计算 CPU 的消耗更高，因此在性能上会有一些损耗。

8.4.12.8.2 任务调度 Job Scheduler

同社区用户多次交流中，我们发现许多场景下用户使用 Apache Doris 时都存在定时调度的需求，例如：

- 周期性的 Backup；
- 过期数据定时清理；
- 周期性的导入任务，如定时通过 Catalog 的方式去进行增量或全量数据同步；

- 定期 ETL，如部分用户定期从宽表中 Load 数据至指定表、从明细表中定时拉取数据存至聚合表、ODS 层表定时打宽并写入原有宽表更新；

尽管诸如 Airflow、DolphinScheduler 等可供选择的外部调度系统非常多，但仍面临一致性的问题 ——在极端情况下，外部调度系统触发 Doris 导入任务并执行成功，因意外情况忽然宕机时，外部调度系统无法正确获取执行结果，会认为此次调度失败，导致触发调度系统的容错机制，通常是重试或者直接失败。而无论采用哪种策略，最终都会导致以下几个情况发生：

- 资源浪费：由于调度系统误认为任务失败，可能会重新调度执行已经成功的任务，导致不必要的资源消耗。
- 数据重复或丢失：如果调度系统选择重试导入任务，可能导致数据重复导入，造成数据冗余和不一致。另一方面，如果调度系统直接标记任务为失败，可能导致实际已成功导入的数据被忽略或丢失。
- 时间延误：由于调度系统的容错机制被触发，可能需要进行额外的任务调度和重试，导致整体数据处理时间延长，影响业务效率和响应速度。
- 系统稳定性下降：频繁的重试或直接失败可能导致调度系统和 Doris 的负载增加，进而影响系统的稳定性和性能。

因此我们在 Apache Doris 2.1 版本中引入了 Job Scheduler 功能并具备了自行任务调度的能力。Doris Job Scheduler 是根据既定计划运行的任务，用于在特定时间或指定时间间隔触发预定义的操作，从而帮助我们自动执行一些任务。从功能上来讲，它类似于操作系统上的定时任务（如：Linux 中的 cron、Windows 中的计划任务），但 Doris 的 Job 调度可以精确到秒级。对于导入场景，我们能够做到完全的一致性保障。除此之外，Doris 内置的 Jon Scheduler 还具有以下特点：

1. 高效调度：Job Scheduler 可以在指定的时间间隔内安排任务和事件，确保数据处理的高效性。采用时间轮算法保证事件能够精准做到秒级触发。
2. 灵活调度：Job Scheduler 提供了多种调度选项，如按分、小时、天或周的间隔进行调度，同时支持一次性调度以及循环（周期）事件调度，并且周期调度也可以指定开始时间、结束时间。
3. 事件池和高性能处理队列：Job Scheduler 采用 Disruptor 实现高性能的生产消费者模型，最大可能的避免任务执行过载。
4. 调度记录可追溯：Job Scheduler 会存储最新的 Task 执行记录（可配置），通过简单的命令即可查看任务执行记录，确保过程可追溯。
5. 高可用：依托于 Doris 自身的高可用机制，Job 可以很轻松的做到自恢复，高可用。

在此我们创建一个定时调度任务作为示例：

```
// 从  2023-11-17 起每天定时执行 insert语句直到 2038 年结束
CREATE
JOB e_daily
    ON SCHEDULE
      EVERY 1 DAY
      STARTS '2023-11-17 23:59:00'
      ENDS '2038-01-19 03:14:07'
```

```
COMMENT 'Saves total number of sessions'
DO
    INSERT INTO site_activity.totals (time, total)
    SELECT CURRENT_TIMESTAMP, COUNT(*)
    FROM site_activity.sessions where create_time >=  days_add(now(),-1) ;
```

> 注意事项
>
> 当前 Job Scheduler 仅支持 Insert 内表，参考文档：CREATE-JOB

8.4.12.9 Behavior Changed

- Unique Key 模型默认开启 Merge On Write 写时合并，新创建的 Unique Key 模型的表将自动设置 enable_unique_key_merge_on_write=true。
- 倒排索引 Invert Index 经过一年多时间的打磨，已实现了对原本的位图索引 Bitmap Index 功能和场景的全覆盖，且功能上和性能上都大幅优于原本的位图索引 Bitmap Index，因此从 Apache Doris 2.1 版本起，我们将默认停止对位图索引 Bitmap Index 的支持，已经创建的位图索引保持不变将继续生效，不允许创建新的位图索引，在未来我们将会移除位图索引的相关代码。
- cpu_resource_limit不再支持，其本身是限制 BE 上 Scanner 线程数目的功能，而 Workload Group 也能支持设置 BE Scanner 线程数目，所以已设置的 cpu_resource_limit将失效。
- Segment Compaction 主要应对单批次大数据量的导入，可以在同一批次数据中进行多个 Segment 的 Compaction 操作，在 2.1 版本开始 Segment Compaction 将默认开启，enable_segcompaction 默认值设置为 True。
- Audit Log 插件
- 从 2.1 版本开始，Apache Doris 开始内置 Audit Log 审计日志插件，用户只需通过设置全局变量 enable_audit_plugin 开启或关闭审计日志功能。
- 对于之前已经安装过审计日志插件的用户，升级后可以继续使用原有插件，也可以通过 uninstall 命令卸载原有插件后，使用新的插件。但注意，切换插件后，审计日志表也将切换到新的表中。
- 具体可参阅：审计日志插件

8.4.12.10 致谢

467887319, 924060929, acnot, airborne12, AKIRA, alan_rodriguez, AlexYue, allenhooo, amory, amory, AshinGau, beat4ocean, BePPPower, bigben0204, bingquanzhao, BirdAmosBird, BiteTheDDDDt, bobhan1, caiconghui, camby, camby, CanGuan, caoliang-web, catpineapple, Centurybbx, chen, ChengDaqi2023, ChenyangSunChenyang, Chester, ChinaYiGuan, ChouGavinChou, chunping, colagy, CSTGluigi, czzmmc, daidai, dalong, dataroaring, DeadlineFen, DeadlineFen, deadlinefen, deardeng, didiaode18, DongLiang-0, dong-shuai, Doris-Extras, Dragonliu2018, DrogonJackDrogon, DuanXujianDuan, DuRipeng, dutyu, echo-dundun, ElvinWei, englefly, Euporia, feelshana, feifeifeimoon, feiniaofeiafei, felixwluo, figurant, flynn, fornaix, FreeOnePlus, Gabriel39, gitccl, gnehil, GoGoWen,

gohalo, guardcrystal, hammer, HappenLee, HB, hechao, HelgeLarsHelge, herry2038, HeZhangJianHe, HHoflittlefish777, Honest-ManXin, hongkun-Shao, HowardQin, hqx871, httpshirley, htyoung, huanghaibin, HuJerryHu, HuZhiyuHu, Hyman-zhao, i78086, irenesrl, ixzc, jacktengg, jacktengg, jackwener, jayhua, Jeffrey, jiafeng.zhang, Jibing-Li, JingDas, julic20s, kaijchen, kaka11chen, KassieZ, kindred77, KirsCalvinKirs, KirsCalvinKirs, kkop, koarz, LemonLiTree, LHG41278, liaoxin01, LiBinfeng-01, LiChuangLi, LiDongyangLi, Lightman, lihangyu, lihuigang, LingAdonisLing, liugddx, LiuGuangdongLiu, LiuHongLiu, liuJiwenliu, LiuLijiaLiu, lsy3993, LuGuangmingLu, LuoMetaLuo, luozenglin, Luwei, Luzhijing, lxliyou001, Ma1oneZhang, mch_ucchi, Miaohongkai, morn-ingman, morrySnow, Mryange, mymeiyi, nanfeng, nanfeng, Nitin-Kashyap, PaiVallishPai, Petrichor, plat1ko, py023, q763562998, qidaye, QiHouliangQi, ranxiang327, realize096, rohitrs1983, sdhzwc, seawinde, seuhezhiqiang, seuhezhiqiang, shee, shuke987, shysnow, songguangfan, Stalary, starocean999, SunChenyangSun, sunny, SWJTU-ZhangLei, TangSiyang2001, Tanya-W, taoxutao, Uniqueyou, vhwzIs, walter, walter, wangbo, Wanghuan, wangqt, wangtao, wangtianyi2004, wenluowen, whuxingying, wsjz, wudi, wudongliang, wuwenchihdu, wyx123654, xiangran0327, Xiaocc, XiaoChangmingXiao, xiaokang, XieJiann, Xinxing, xiongjx, xuefengze, xueweizhang, XueYuhai, XuJianxu, xuke-hat, xy, xy720, xyfsjq, xzj7019, yagagagaga, yangshijie, YangYAN, yiguolei, yiguolei, yimeng, YinShaowenYin, Yoko, yongjinhou, ytwp, yuanyuan8983, yujian, yujun777, Yukang-Lian, Yulei-Yang, yuxuan-luo, zclllyybb, ZenoYang, zfr95, zgxme, zhangdong, zhangguoqiang, zhangstar333, zhangstar333, zhangy5, ZhangYu0123, zhannngchen, ZhaoLongZhao, zhaoshuo, zhengyu, zhiqqqq, ZhongJinHacker, ZhuArmandoZhu, zlw5307, ZouXinyiZou, zxealous, zy-kkk, zzwwhh, zzzxl1993, zzzzzzzs

8.5 v2.0

8.5.1 Release 2.0.15

亲爱的社区小伙伴们，Apache Doris 2.0.15 版本已于 2024 年 9 月 30 日正式与大家见面，该版本提交了 157 个改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

- 立即下载：https://doris.apache.org/download
- GitHub 下载：https://github.com/apache/doris/releases/tag/2.0.15

8.5.1.1 行为变更

无

8.5.1.2 新功能

- 恢复功能现在支持删除冗余的表块和分区选项。#39028
- 支持 JSON 函数 `json_search`。#40948

8.5.1.3 改进与优化

8.5.1.3.1 稳定性

- 添加了 FE 配置 `abort_txn_after_lost_heartbeat_time_second`，用于设置事务中止时间。#28662
- BE 失去心跳信号超过 1 分钟后中止事务，而不是 5 秒，以避免事务中止过于敏感。#22781

- 延迟调度例行加载的 EOF 任务，以避免过多的小事务。#39975
- 优先从在线磁盘服务进行查询，以提高稳健性。#39467
- 在非严格模式的部分更新中，如果行的删除标志已标记，则跳过检查新插入的行。#40322
- 为防止 FE 内存不足，限制备份任务中的表块数量，默认值为 300,000。#39987
- ARRAY MAP STRUCT 类型支持 REPLACE_IF_NOT_NULL。#38304
- 对非 DELETE_INVALID_XXX失败的删除作业进行重试。#37834

8.5.1.3.2 查询性能

- 优化由并发列更新和 compaction 引起的慢速列更新问题。#38487
- 当过滤条件中存在 NullLiteral 时，可以将其折叠为 false 并进一步转换为 EmptySet，以减少不必要的数据扫描和计算。#38135
- 提高 ORDER BY 全排序的性能。#38985
- 提高倒排索引中字符串处理的性能。#37395

8.5.1.3.3 查询优化器

- 增加了对以分号开头的语句的支持以兼容老优化器。#39399
- 完善了一些聚合函数签名匹配。#39352
- 在 Schema 变更后删除列统计信息并触发自动分析。#39101
- 支持使用 DROP CACHED STATS table_name 删除缓存的统计信息。#39367

8.5.1.3.4 Multi Catalog

- 优化 JDBC Catalog 刷新，减少客户端创建频率。#40261
- 修复 JDBC Catalog 在某些条件下存在的线程泄漏问题。#39423

致谢

@924060929、@BePPPower、@BiteTheDDDDt、@CalvinKirs、@GoGoWen、@HappenLee、@Jibing-Li、@Johnnyssc、@LiBinfeng-01、@Mryange、@SWJTU-ZhangLei、@TangSiyang2001、@Toms1999、@Vallishp、@Yukang-Lian、@airborne12、@amorynan、@bobhan1、@cambyzju、@csun5285、@dataroaring、@eldenmoon、@englefly、@feiniaofeiafei、@hello-stephen、@htyoung、@hubgeter、@justfortaste、@liaoxin01、@liugddx、@liutang123、@luwei16、@mongo360、@morrySnow、@qidaye、@smallx、@sollhui、@starocean999、@w41ter、@xiaokang、@xzj7019、@yujun777、@zclllyybb、@zddr、@zhangstar333、@zhannngchen、@zy-kkk、 @zzzxl1993

8.5.2 Release 2.0.14

亲爱的社区小伙伴们，Apache Doris 2.0.14 版本已于 2024 年 8 月 6 日正式与大家见面，该版本提交了 110 个改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

8.5.2.1 1 新功能

- 增加获取最近一个查询 Profile 的 REST 接口 `curl http://user:password@127.0.0.1:8030/api/profile/text` 。 #38268

8.5.2.2 2 改进和优化

- 优化 MOW 表带有 Sequence 列的主键点查性能。 #38287
- 优化倒排索引在查询条件很多时的性能。 #35346
- 创建带分词的倒排索引时，自动开启 `support_phrase` 选项加速 `match_phrase` 系列短语查询。 #37949
- 支持简化的 SQL Hint，例如 `SELECT /*+ query_timeout(3000)*/ * FROM t;`。 #37720
- 读对象存储遇到 429 错误时自动重试提升稳定性。 #35396
- LEFT SEMI / ANTI JOIN 在匹配到符合的数据行时，终止后续的匹配执行提升性能。 #34703
- 避免非法数据返回 MySQL 结果时出发 coredump。 #28069
- 输出类型名字时统一使用小写，保持跟 MySQL 兼容对 BI 工具更加友好。 #38521

8.5.2.3 致谢

@924060929、@BiteTheDDDDt、@ByteYue、@CalvinKirs、@GoGoWen、@HappenLee、@Jibing-Li、@Lchangliang、@LiBinfeng-01、@Mryange、@XieJiann、@Yukang-Lian、@Yulei-Yang、@airborne12、@amorynan、@biohazard4321、@cambyzju、@csun5285、@eldenmoon、@englefly、@freemandealer、@hello-stephen、@hubgeter、@kaijchen、@liaoxin01、@luwei16、@morningman、@morrySnow、@mymeiyi、@qidaye、@sollhui、@starocean999、@w41ter、@wuwenchi、@xiaokang、@xy720、@yujun777、@zclllyybb、 @zddr、 @zhangstar333、 @zhiqiang-hhhh、 @zy-kkk、 @zzzxl1993

8.5.3 Release 2.0.13

亲爱的社区小伙伴们，Apache Doris 2.0.13 版本已于 2024 年 7 月 16 日正式与大家见面，该版本提交了 112 个改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

快速下载

8.5.3.1 行为变更

仅在客户端启用了 `CLIENT_MULTI_STATEMENTS` 设置时，SQL 输入才会被视为多条语句，从而增强了与 MySQL 的兼容性。 #36759

8.5.3.2 新增功能

- 新增了 BE 配置 `allow_zero_date`，允许使用全零的日期。设置为 `false` 时，`0000-00-00` 会被解析为 `NULL`；设置为 `true` 时，会被解析为 `0000-01-01`。默认值为 `false`，以保持与之前行为的一致性。#34961
- `LogicalWindow` 和 `LogicalPartitionTopN` 现在支持多字段谓词下推，以提升性能。#36828
- ES Catalog 现在将 ES 的 `nested` 或 `object` 类型映射到 Doris 的 JSON 类型。#37101

8.5.3.3 改进和优化

- `LIMIT` 查询现在会更早地停止读取数据，以减少资源消耗并提升性能。#36535
- 现在支持具有空键的特殊 JSON 数据。#36762
- 改进了 Routine Load 的稳定性和可用性，包括负载均衡、自动恢复、异常处理以及更友好的错误消息。#36450 #35376 #35266 #33372 #32282 #32046 #32021 #31846 #31273
- 对 BE 的硬盘选择策略和速度进行了优化。#36826 #36795 #36509
- 改进了 JDBC Catalog 的稳定性和可用性，包括加密、线程池连接数配置以及更友好的错误消息。#36940 #36720 #30880 #35692

8.5.3.4 致谢

@DarvenDuan、@Gabriel39、@Jibing-Li、@Johnnyssc、@Lchangliang、@LiBinfeng-01、@SWJTU-ZhangLei、@Thearas、@Yukang-Lian、@Yulei-Yang、@airborne12、@amorynan、@bobhan1、@cambyzju、@csun5285、@dataroaring、@deardeng、@eldenmoon、@englefly、@feiniaofeiafei、@hello-stephen、@jacktengg、@kaijchen、@liutang123、@luwei16、@morningman、@morrySnow、@mrhhsg、@mymeiyi、@platoneko、@qidaye、@sollhui、@starocean999、@w41ter、@xiaokang、@xy720、@yujun777、@zclllyybb

8.5.4 Release 2.0.12

亲爱的社区小伙伴们，Apache Doris 2.0.12 版本已于 2024 年 6 月 27 日正式与大家见面，该版本提交了 99 个改进项以及问题修复，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.4.1 行为变更

- 不再将建表的默认注释设置为表的类型，而是改成默认为空，比如 COMMENT ‘OLAP’ 变成 COMMENT ”，这样对于依赖注释的 BI 软件更加友好。#35855
- 将 `@@autocommit` 变量的类型从 `BOOLEAN` 改成 `BIGINT`，以免有些 MySQL 客户端（比如.NET MySQL.Data）报错。#33282

8.5.4.2 改进优化

- 删除 `disable_nested_complex_type` 参数，默认允许创建嵌套的 ARRAY MAP STRUCT 类型。#36255
- HMS Catalog 支持 `SHOW CREATE DATABASE` 命令。#28145
- 在 Query Profile 中增加更多倒排索引的指标。#36545
- 跨集群数据复制（CCR）支持倒排索引 #31743

8.5.4.3 致谢

@amorynan、@BiteTheDDDDt、@cambyzju、@caoliang-web、@dataroaring、@eldenmoon、@feiniaofeiafei、@felixwluo、@gavinchou、@HappenLee、@hello-stephen、@jacktengg、@Jibing-Li、@Johnnyssc、@liaoxin01、@LiBinfeng-01、@luwei16、@mongo360、@morningman、@morrySnow、@mrhhsg、@Mryange、@mymeiyi、@qidaye、@qzsee、@starocean999、@w41ter、@wangbo、@wsjz、@wuwenchi、@xiaokang、@XuPengfei-1020、@xy720、@yongjinhou、@yujun777、@Yukang-Lian、@Yulei-Yang、@zclllyybb、@zddr、 @zhannngchen、 @zhiqiang-hhhh、 @zy-kkk、 @zzzxl1993

8.5.5 Release 2.0.11

亲爱的社区小伙伴们，Apache Doris 2.0.11 版本已于 2024 年 6 月 5 日正式与大家见面，该版本提交了 123 个改进项以及问题修复，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.5.1 1 行为变更

由于倒排索引已经成熟稳定，可以替换老的 `BITMAP INDEX`，因此后续新建 `BITMAP INDEX` 会自动切换成 `INVERTED INDEX`，而已经创建的 `BITMAP INDEX` 保持不变。整个切换过程对用户无感知，写入和查询没有变化，此外用户可以修改 FE 配置 `enable_create_bitmap_index_as_inverted_index = false` 来关闭该自动切换。#35528

8.5.5.2 2 改进和优化

- 为 JSON 和 TIME 添加 Trino JDBC Catalog 类型映射。
- 在无法转移到（非）主节点时，FE 退出以防止未知状态和过多日志。
- 在删除统计表时写入审计日志。
- 如果表只进行了部分分析，忽略最小/最大列统计以避免低效的查询计划。
- 支持集合操作减法，例如 `set1 - set2`。
- 使用 concat(col, pattern_str) 改进 LIKE 和 REGEXP 子句的性能，例如：`col1 LIKE concat('%', col2, '%')`。
- 添加查询选项以支持短路查询，保证升级兼容性。

8.5.5.3 3 致谢

@924060929、@airborne12、@AshinGau、@BePPPower、@BiteTheDDDDt、@ByteYue、@CalvinKirs、@cambyzju、@csun5285、@dataroaring、@eldenmoon、@englefly、@feiniaofeiafei、@Gabriel39、@GoGoWen、@HHoflittlefish777、@hubgeter、@jacktengg、@jackwener、@jeffreys-cat、@Jibing-Li、@kaka11chen、@kobe6th、@LiBinfeng-01、@mongo360、@morningman、@morrySnow、@mrhhsg、@Mryange、@nextdreamblue、@qidaye、@sjyango、@starocean999、@SWJTU-ZhangLei、@w41ter、@wangbo、@wsjz、@wuwenchi、@xiaokang、@XieJiann、@xy720、@yujun777、@Yukang-Lian、@Yulei-Yang、@zclllyybb、@zddr、@zhangstar333、@zhiqiang-hhhh、 @zy-kkk、 @zzzxl1993

8.5.6 Release 2.0.10

亲爱的社区小伙伴们，Apache Doris 2.0.10 版本已于 2024 年 5 月 16 日正式与大家见面，该版本提交了 83 个改进项以及问题修复，进一步提升了系统的性能及稳定性，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.6.1 改进和优化

- 增加了`read_only`和`super_read_only`变量以保持和 MySQL 兼容
- 仅在 IO_ERROR 的错误才把数据目录加入 Broken List，防止 fd 超限等错误导致误加入
- 基于外表 CTAS 创建新表时，把 VARCHAR 类型转成 STRING 类型
- 支持把 Paimon 的 ROW 类型映射成 Doris 的 STRUCT 类型
- 在创建 Tablet 选择数据盘时，允许存在少量的倾斜
- 对 `set replica drop` 命令记录 Editlog，以防止在 Follower 节点执行命令后，其状态显示不正确
- Schema Change 内存自适应避免内存超限
- 倒排索引中 Unicode 分词器可以配置不使用停用词

8.5.6.2 致谢

@airborne12, @BePPPower, @ByteYue, @CalvinKirs, @cambyzju, @csun5285, @dataroaring, @deardeng, @DongLiang-0, @eldenmoon, @felixwluo, @HappenLee, @hubgeter, @jackwener, @kaijchen, @kaka11chen, @Lchangliang, @liaoxin01, @LiBinfeng-01, @luennng, @morningman, @morrySnow, @Mryange, @nextdreamblue, @qidaye, @starocean999, @suxiaogang223, @SWJTU-ZhangLei, @w41ter, @xiaokang, @xy720, @yujun777, @Yukang-Lian, @zhangstar333, @zxealous, @zy-kkk, @zzzxl1993

8.5.7 Release 2.0.9

亲爱的社区小伙伴们，Apache Doris 2.0.9 版本已正式发布。在本次版本中，有 34 位贡献者提交了约 68 个功能改进以及问题修复，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.7.1 1 行为变更

无

8.5.7.2 2 新功能

- 物化视图的 Key 和 Value 列都允许出现谓词
- 物化视图支持 `bitmap_union(bitmap_from_array())`
- 增加一个 FE 配置强制集群中所有表的 Replicate Allocation
- 新优化器支持日期字面量指定时区
- MATCH_PHRASE 全文检索支持 slop 参数指定搜索词之间的距离

8.5.7.3 3 改进和优化

- `first_value` / `last_value` 函数增加第二个参数指定忽略 NULL 值
- LEAD/ LAG 函数的 Offset 参数可以为 0
- 调整物化视图匹配的顺序优先利用索引和预聚合加速查询
- 优化 TopN 查询 `ORDER BY k LIMIT n` 的性能
- 优化 Meta Cache 的性能
- 为`delete_bitmap get_agg` 函数增加 Profile 便于性能分析
- 增加 FE 参数设置 Autobucket 的最大 Bucket 数

8.5.7.4 4 致谢

adonis0147, airborne12, amorynan, AshinGau, BePPPower, BiteTheDDDDt, CalvinKirs, cambyzju, csun5285, eldenmoon, englefly, feiniaofeiafei, HHoflittlefish777, htyoung, hust-hhb, jackwener, Jibing-Li, kaijchen, kylinmac, liaoxin01, luwei16, morningman, mrhhsg, qidaye, starocean999, SWJTU-ZhangLei, w41ter, xiaokang, xiedeyantu, xy720, zclllyybb, zhangstar333, zhannngchen, zykkk, zzzxl1993

8.5.8 Release 2.0.8

亲爱的社区小伙伴们，Apache Doris 2.0.8 版本已于 2024 年 04 月 09 日正式与大家见面。本次版本中，有 35 位贡献者提交了约 65 个功能改进以及问题修复，进一步提升了系统的稳定性和性能，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.8.1 1 行为变更

由于 ADMIN SHOW xx 语句在 MySQL 8.x jdbc driver 不能执行，所以将名字改成 SHOW xx

- https://github.com/apache/doris/pull/29492

```
ADMIN SHOW CONFIG -> SHOW CONFIG
ADMIN SHOW REPLICA -> SHOW REPLICA
ADMIN DIAGNOSE TABLET -> SHOW TABLET DIAGNOSIS
ADMIN SHOW TABLET -> SHOW TABLET
```

8.5.8.2 2 新功能

N/A

8.5.8.3 3 改进和优化

- 新优化器支持 TopN 优化中使用倒排索引
- 限制统计信息 STRING 长度为 1024 以控制 BE 内存消耗
- 修复未创建 JDBC Client 时意外关闭的情况
- 接受所有 Iceberg Database，不再做额外的名字检查
- 异步更新外表行数统计，避免同步更新带来的 Cache miss 和 Plan 不稳定
- 简化 Hive 外表的 isSplitable 方法，避免过多的 Hadoop metric

8.5.8.4 4 致谢

924060929, AcKing-Sam, amorynan, AshinGau, BePPPower, BiteTheDDDDt, ByteYue, cambyzju, dongsilun, eldenmoon, feiniaofeiafei, gnehil, Jibing-Li, liaoxin01, luwei16, morningman, morrySnow, mrhhsg, Mryange, nextdreamblue, platoneko, starocean999, SWJTU-ZhangLei, wuwenchi, xiaokang, xinyiZzz, Yukang-Lian, Yulei-Yang, zclllyybb, zddr, zhangstar333, zhiqiang-hhhh, ziyanTOP, zy-kkk, zzzxl1993

8.5.9 Release 2.0.7

8.5.9.1 1 行为变更

- round 函数行为跟 MySQL 保持一致，例如 round(5/2) 返回 3 而不是 2.
- https://github.com/apache/doris/pull/31583
- 时间精度转换行为跟 MySQL 保持一致，例如 ‘2023-10-12 14:31:49.666’ 四舍五入到 ‘2023-10-12 14:31:50’.
- https://github.com/apache/doris/pull/27965

8.5.9.2 2 新功能

- 在更多的情况下可以将 OUTER JOIN 转换成 ANTI JOIN 来加速查询
- https://github.com/apache/doris/pull/31854
- 支持通过 Nginx, HAProxy 等代理连接的 IP 透传
- https://github.com/apache/doris/pull/32338

8.5.9.3 3 改进和优化

- 通过在 `information_schema` 中增加 DEFAULT_ENCRYPTION 列、增加 `processlist` 表，提升 BI 工具的兼容性
- 创建 JDBC Catalog 时默认自动检测连通性
- 增强自动恢复提升 Kafka Routine Load 的稳定性
- 倒排索引中文分词对英文默认做小写转换
- Repeat 函数的重复次数超过限制时报错
- 自动跳过 Hive 外表中的隐藏文件和目录
- 在某些极端情况下减少 File Meta Cache 避免 OOM
- 减少 Broker Load 的 jvm 内存占用
- 加速带排序的 INSERT INTO SELECT 比如 `INSERT INTO t1 SELECT * FROM t2 ORDER BY k`

8.5.9.4 4 致谢

924060929,airborne12,amorynan,ByteYue,dataroaring,deardeng,feiniaofeiafei,felixwluo,freemandealer,gavinchou,hello-stephen,HHoflittlefish777,jack
cat,Jibing-Li,KassieZ,LiBinfeng-01,luwei16,morningman,mrhhsg,Mryange,nextdreamblue,platoneko,qidaye,rohitrs1983,seawinde,shuke987,starocean99
ZhangLei,w41ter,wsjz,wuwenchi,xiaokang,XieJiann,XuJianxu,yujun777,Yulei-Yang,zhangstar333,zhiqiang-hhhh,zy-kkk,zzzxl1993

8.5.10 Release 2.0.6

亲爱的社区小伙伴们，Apache Doris 2.0.6 版本已于 2024 年 3 月 12 日正式与大家见面。本次版本中，有 51 位贡献者提交了约 114 个功能改进以及问题修复，进一步提升了系统的稳定性和性能，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.10.1 行为变更

- 无

8.5.10.2 新功能

- 自动选择物化视图时支持匹配带别名的函数
- 增加安全下线一个 tablet 副本的命令
- 外表统计信息增加行数统计缓存
- 统计信息收集支持 Rollup

8.5.10.3 改进和优化

- 使用 protobuf 稳定序列化减少 Tablet Schema 缓存内存占用
- 提升 `show column stats` 的性能
- 统计信息收集和优化器支持 Iceberg 和 Paimon 的行数估计
- JDBC Catalog 支持读取 SQL Server 的 Timestamp 类型

8.5.10.4 致谢

最后，衷心感谢 51 位开发者为 Apache Doris 2.0.6 版本做出了重要贡献：

924060929, AshinGau, BePPPower, BiteTheDDDDt, CalvinKirs, cambyzju, deardeng, DongLiang-0, eldenmoon, englefly, feelshana, feiniaofeiafei, felixwluo, HappenLee, hust-hhb, iwanttobepowerful, ixzc, JackDrogon, Jibing-Li, KassieZ, larshelge, liaoxin01, LiBinfeng-01, liutang123, luennng, morningman, morrySnow, mrhhsg, qidaye, starocean999, TangSiyang2001, wangbo, wsjz, wuwenchi, xiaokang, XieJiann, xuwei0912, xy720, xzj7019, yiguolei, yujun777, Yukang-Lian, Yulei-Yang, zclllyybb, zddr, zhangstar333, zhannngchen, zhiqiang-hhhh, zy-kkk, zzzxl1993

8.5.11 Release 2.0.5

亲爱的社区小伙伴们，Apache Doris 2.0.5 版本已于 2024 年 2 月 27 日正式与大家见面。这次更新带来一系列行为变更和功能更新，并进行了若干的改进与优化，旨在为用户提供更为稳定高效的数据查询与分析体验。新版本已经上线，欢迎大家下载体验！

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.11.1 行为变更

- `select char(0)= '\0'` 返回 true，跟 MySQL 的行为保持一致
- https://github.com/apache/doris/pull/30034
- Export 导出数据支持空表
- https://github.com/apache/doris/pull/30703

8.5.11.2 新功能

- 利用过滤条件中的 `is null` 谓词，将 OUTER JOIN 转换为 ANTI JOIN
- 增加 `SHOW TABLETS BELONG` 语法用于获取 tablet 属于哪个 table
- InferPredicates 支持 `IN`，例如：`a = b & a in [1, 2] -> b in [1, 2]`

- 支持对物化视图收集统计信息
- SHOW PROCESSLIST 支持输出连接对应的 FE
- Export 导出 CSV 文件支持通过 with_bom 参数控制是否带有 Windows BOM

8.5.11.3 改进和优化

- 在无统计信息时优化 Query Plan
- 基于 Rollup 的统计信息优化 Query Plan
- 用户停止 Auto Analyze 后尽快停止统计信息收集任务
- 缓存统计信息收集异常，避免大约太多异常栈
- 支持在 SQL 中自定使用某个物化视图
- JDBC Catalog 谓词下推列名字符转义
- 修复 MySQL Catalog 中 to_date 函数下推的问题
- 优化 JDBC 客户端连接关闭的逻辑，在异常时正常取消查询
- 优化 JDBC 连接池的参数
- 通过 HMS API 获取 Hudi 外表的分区信息
- 优化 Routine Load 的内存占用和错误信息
- 如果 max_backup_restore_job_num_per_db 参数为 0，跳过所有备份恢复任务

8.5.11.4 致谢

最后，衷心感谢 59 位开发者为 Apache Doris 2.0.5 版本做出了重要贡献：

airborne12, alexxing662, amorynan, AshinGau, BePPPower, bingquanzhao, BiteTheDDDDt, ByteYue, caiconghui, cambyzju, catpineapple, dataroaring, eldenmoon, Emor-nj, englefly, felixwluo, GoGoWen, HappenLee, hello-stephen, HHoflittlefish777, HowardQin, JackDrogon, jacktengg, jackwener, Jibing-Li, KassieZ, LemonLiTree, liaoxin01, liugddx, LuGuangming, morningman, morrySnow, mrhhsg, Mryange, mymeiyi, nextdreamblue, qidaye, ryanzryu, seawinde,starocean999, TangSiyang2001, vinlee19, w41ter, wangbo, wsjz, wuwenchi, xiaokang, XieJiann, xingyingone, xy720,xzj7019, yujun777, zclllyybb, zhangstar333, zhannngchen, zhiqiang-hhhh, zxealous, zy-kkk, zzzxl1993

8.5.12 Release 2.0.4

亲爱的社区小伙伴们，Apache Doris 2.0.4 版本已于 2024 年 1 月 26 日正式发布，该版本在新优化器、倒排索引、数据湖等功能上有了进一步的完善与更新，使 Apache Doris 能够适配更广泛的场景。此外，该版本进行了若干的改进与优化，以提供更加稳定高效的性能体验。新版本已经上线，欢迎大家下载使用！

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.5.12.1 行为变更

- 提供了更精确的 Precision 和 Scale 推导，可满足金融场景计算的高要求
- https://github.com/apache/doris/pull/28034
- Drop Policy 支持了 User 和 Role
- https://github.com/apache/doris/pull/29488

8.5.12.2 新功能

- 新优化器支持了 datev1，datetimev1 及 decimalv2 数据类型
- 新优化器支持了 ODBC 外表
- 倒排索引支持了 `lower_case` 和 `ignore_above` 选项
- 倒排索引支持了 `match_regexp` 和 `match_phrase_prefix` 查询加速
- 数据湖支持了 Paimon Native Reader
- 数据湖支持读取 LZO 压缩的 Parquet 文件
- 审计日志支持 `insert into`

8.5.12.3 改进和优化

- 对数据均衡、迁移等存储管控进行了改进
- 对数据冷却策略进行了改进，以节省本地硬盘存储空间
- 对 ASCII 字符串 substr 进行了优化
- 针对使用 date 函数查询时的分区裁剪进行了优化
- 针对优化器自动统计信息收集的可观测性和性能进行了优化

8.5.12.4 致谢

感谢 73 位开发者为 Apache Doris 2.0.4 版本做出了重要贡献，正是由于他们的努力，Apache Doris 在性能和稳定性方面取得了显著的进步。

airborne12、amorynan、AshinGau、BePPPower、bingquanzhao、BiteTheDDDDt、bobhan1、ByteYue、caiconghui、CalvinKirs、cambyzju、caoliang-web、catpineapple、csun5285、dataroaring、deardeng、dutyu、eldenmoon、englefly、feifeifeimoon、fornaix、Gabriel39、gnehil、HappenLee、hello-stephen、HHoflittlefish777、hubgeter、hust-hhb、ixzc、jacktengg、jackwener、Jibing-Li、kaka11chen、KassieZ、LemonLiTree、liaoxin01、LiBinfeng-01、lihuigang、liugddx、luwei16、morningman、morrySnow、mrhhsg、Mryange、nextdreamblue、Nitin-Kashyap、platoneko、py023、qidaye、shuke987、starocean999、SWJTU-ZhangLei、w41ter、wangbo、wsjz、wuwenchi、Xiaoccer、xiaokang、XieJiann、xingyingone、xinyiZzz、xuwei0912、xy720、xzj7019、yujun777、zclllyybb、zddr、zhangguoqiang666、zhangstar333、zhannngchen、zhiqiang-hhhh、zy-kkk、zzzxl1993

8.5.13 Release 2.0.3

亲爱的社区小伙伴们，Apache Doris 2.0.3 版本已于 2023 年 12 月 14 日正式发布，该版本对复杂数据类型、统计信息收集、倒排索引、数据湖分析、分布式副本管理等多个功能进行了优化，有 104 位贡献者为 Apache Doris 2.0.3 版本提交了超过 1000 个功能优化项以及问题修复，进一步提升了系统的稳定性和性能，欢迎大家下载体验。

GitHub 下载：https://github.com/apache/doris/releases

官网下载页：https://doris.apache.org/download/

8.5.13.1 新增特性

8.5.13.1.1 自动统计信息收集

统计信息是 CBO 优化器进行代价估算时的依赖，通过收集统计信息有助于优化器了解数据分布特性、估算每个执行计划的成本并选择更优的执行计划，以此大幅提升查询效率。从 2.0.3 版本开始，Apache Doris 开始支持自动统计信息收集，默认为开启状态。

在每次导入事务提交后，Apache Doris 将记录导入事务更新的表信息并估算表统计信息的健康度，对于健康度低于配置参数的表会认为统计信息已过时并自动触发表的统计信息收集作业。同时为了降低统计信息作业的资源开销，Apache Doris 会自动采取采样的方式收集统计信息，用户也可以调整参数来采样更多行以获得更准确的数据分布信息。

更多信息请参考：Statistics

8.5.13.1.2 数据湖框架支持复杂数据类型

- Java UDF、JDBC catalog、 Hudi MOR 表等功能支持复杂数据类型
- https://github.com/apache/doris/pull/24810
- https://github.com/apache/doris/pull/26236
- Paimon catalog 支持复杂数据类型
- https://github.com/apache/doris/pull/25364
- Paimon catalog 支持 Paimon 0.5 版本
- https://github.com/apache/doris/pull/24985

8.5.13.1.3 增加更多内置函数

- 新优化器支持 BitmapAgg 函数
- https://github.com/apache/doris/pull/25508
- 支持 SHA 系列摘要函数
- https://github.com/apache/doris/pull/24342
- 聚合函数 min_by 和 max_by 支持 bitmap 数据类型
- https://github.com/apache/doris/pull/25430
- 增加 milliseconds/microseconds_add/sub/diff 函数
- https://github.com/apache/doris/pull/24114
- 增加 json_insert, json_replace, json_set JSON 函数
- https://github.com/apache/doris/pull/24384

8.5.13.2 改进优化

8.5.13.2.1 性能优化

- 在过滤率高的倒排索引 match where 条件和过滤率低的普通 where 条件组合时，大幅降低索引列的 IO
- 优化经过 where 条件过滤后随机读数据的效率
- 优化在 JSON 数据类型上使用老的 get_json_xx 函数的性能，提升 2-4 倍
- 支持配置降低读数据线程的优先级，保证写入的 CPU 资源和实时性
- 增加返回 largeint 的 uuid-numeric 函数，性能比返回 string 的 uuid 函数快 20 倍
- Case when 的性能提升 3 倍
- 在存储引擎执行中裁剪不必要的谓词计算
- 支持 count 算子下推到存储层
- 优化支持 and or 表达式中包含 nullable 类型的计算性能
- 支持更多场景下 limit 算子提前到 join 前执行的改写，以提升执行效率
- 增加消除 inline view 中的无用的 order by 算子，以提升执行效率
- 优化了部分情况下的基数估计和代价模型的准确性，以提升执行效率
- 优化了 JDBC catalog 的谓词下推逻辑和大小写逻辑
- 优化了 file cache 的第一次开启后的读取效率
- 优化 Hive 表 SQL cache 策略，使用 HMS 中存储的分区更新时间作为 cache 是否失效的判断，提高 cache 命中率
- 优化了 Merge-on-Write compaction 效率
- 优化了外表查询的线程分配逻辑，降低内存使用
- 优化 column reader 的内存使用

8.5.13.2.2 分布式副本管理改进

优化跳过删除分区、 colocate group、 持续写时均衡失败、冷热分层表不能均衡等；

8.5.13.2.3 安全性提升

- 审计日志插件的配置使用 token 代替明文密码以增强安全性
- https://github.com/apache/doris/pull/26278
- log4j 配置安全性增强
- https://github.com/apache/doris/pull/24861
- 日志中不显示用户敏感信息
- https://github.com/apache/doris/pull/26912

8.5.13.3 Bugfix 和稳定性提升

8.5.13.3.1 复杂数据类型

- 修复了 map/struct 对定长 CHAR(n) 没有正确截断的问题
- https://github.com/apache/doris/pull/25725

- 修复了 struct 嵌套 map/array 写入失败的问题
- https://github.com/apache/doris/pull/26973
- 修复了 count distinct 不支持 array/map/struct 的问题
- https://github.com/apache/doris/pull/25483
- 解决 query 中出现 delete 复杂类型之后，升级过程中出现 BE crash 的问题
- https://github.com/apache/doris/pull/26006
- 修复了 jsonb 在 where 条件中 BE crash 问题
- https://github.com/apache/doris/pull/27325
- 修复了 outer join 中有 array 类型时 BE crash 的问题
- https://github.com/apache/doris/pull/25669
- 修复 orc 格式 decimal 类型读取错误的问题
- https://github.com/apache/doris/pull/26548
- https://github.com/apache/doris/pull/25977
- https://github.com/apache/doris/pull/26633

8.5.13.3.2 倒排索引

- 修复了关闭倒排索引查询时 OR NOT 组合 where 条件结果错误的问题
- https://github.com/apache/doris/pull/26327
- 修复了空数组的倒排索引写入时 BE crash 的问题
- https://github.com/apache/doris/pull/25984
- 修复输出为空的情况下 index compaction BE crash 的问题
- https://github.com/apache/doris/pull/25486
- 修复新增列没有写入数据时，增加倒排索引 BE crash 的问题
- https://github.com/apache/doris/pull/27276
- 修复 1.2 版本误建倒排索引后升级 2.0 等情况下倒排索引硬链缺失和泄露的问题
- https://github.com/apache/doris/pull/26903

8.5.13.3.3 物化视图

- 修复 group by 语句中包括重复表达式导致 BE crash 的问题
- https://github.com/apache/doris/pull/27523
- 禁止视图创建时 group by 子句中使用 float/doubld 类型
- https://github.com/apache/doris/pull/25823
- 增强支持了 select 查询命中物化视图的功能
- https://github.com/apache/doris/pull/24691
- 修复当使用了表的 alias 时物化视图不能命中的问题
- https://github.com/apache/doris/pull/25321
- 修复了创建物化视图中使用 percentile_approx 的问题
- https://github.com/apache/doris/pull/26528

8.5.13.3.4 采样查询

- 修复 table sample 功能在 partition table 上无法正常工作的问题
- https://github.com/apache/doris/pull/25912
- 修复 table sample 指定 tablet 无法工作的问题
- https://github.com/apache/doris/pull/25378

8.5.13.3.5 主键表

- 修复基于主键条件更新的空指针异常
- https://github.com/apache/doris/pull/26881
- 修复部分列更新字段名大小写问题
- https://github.com/apache/doris/pull/27223
- 修复 schema change 时 mow 会出现重复 key 的问题
- https://github.com/apache/doris/pull/25705

8.5.13.3.6 导入和 Compaction

- 修复 routine load 一流多表时 unknown slot descriptor 错误
- https://github.com/apache/doris/pull/25762
- 修复内存统计并发访问导致 BE crash 问题
- https://github.com/apache/doris/pull/27101
- 修复重复取消导入导致 BE crash 的问题
- https://github.com/apache/doris/pull/27111
- 修复 broker load 时 broker 连接报错问题
- https://github.com/apache/doris/pull/26050
- 修复 compaction 和 scan 并发下 delete 谓词可能导致查询结果不对的问题
- https://github.com/apache/doris/pull/24638
- 修复 compaction task 存在时打印大量 stacktrace 日志的问题
- https://github.com/apache/doris/pull/25597

8.5.13.3.7 数据湖兼容性

- 解决 iceberg 表中包含特殊字符导致查询失败的问题
- https://github.com/apache/doris/pull/27108
- 修复 Hive metastore 不同版本的兼容性问题
- https://github.com/apache/doris/pull/27327
- 修复读取 MaxCompute 分区表错误的问题
- https://github.com/apache/doris/pull/24911
- 修复备份到对象存储失败的问题
- https://github.com/apache/doris/pull/25496
- https://github.com/apache/doris/pull/25803

8.5.13.3.8 JDBC 外表兼容性

- 修复 JDBC catalog 处理 Oracle 日期类型格式错误的问题
- https://github.com/apache/doris/pull/25487
- 修复 JDBC catalog 读取 MySQL 0000-00-00 日期异常的问题
- https://github.com/apache/doris/pull/26569
- 修复从 MariaDB 读取数据时间类型默认值为 current_timestamp 时空指针异常问题
- https://github.com/apache/doris/pull/25016
- 修复 JDBC catalog 处理 bitmap 类型时 BE crash 的问题
- https://github.com/apache/doris/pull/25034
- https://github.com/apache/doris/pull/26933

8.5.13.3.9 SQL 规划和优化

- 修复了部分场景下分区裁剪错误的问题
- https://github.com/apache/doris/pull/27047
- https://github.com/apache/doris/pull/26873
- https://github.com/apache/doris/pull/25769
- https://github.com/apache/doris/pull/27636
- 修复了部分场景下子查询处理不正确的问题
- https://github.com/apache/doris/pull/26034
- https://github.com/apache/doris/pull/25492
- https://github.com/apache/doris/pull/25955
- https://github.com/apache/doris/pull/27177
- 修复了部分语义解析的错误
- https://github.com/apache/doris/pull/24928
- https://github.com/apache/doris/pull/25627
- 修复 right outer/anti join 时，有可能丢失数据的问题
- https://github.com/apache/doris/pull/26529
- 修复了谓词被错误的下推穿过聚合算子的问题
- https://github.com/apache/doris/pull/25525

- 修正了部分情况下返回的结果 header 不正确的问题
- https://github.com/apache/doris/pull/25372
- 包含有 nullsafeEquals 表达式 (<=>) 作为连接条件时，可以正确对规划出 hash join
- https://github.com/apache/doris/pull/27127
- 修复了 set operation 算子中无法正确列裁剪的问题
- https://github.com/apache/doris/pull/26884

8.5.13.4 行为变更

- 复杂数据类型 array/map/struct 的输出格式改成跟输入格式以及 JSON 规范保持一致，跟之前版本的主要变化是日期和字符串用双引号括起来，array/map 内部的空值显示为 null 而不是 NULL。
- https://github.com/apache/doris/pull/25946
- 默认情况下，当用户属性 `resource_tags.location` 没有设置时，只能使用 default 资源组的节点，而之前版本中可以访问任意节点。
- https://github.com/apache/doris/pull/25331
- 支持 SHOW_VIEW 权限，拥有 SELECT 或 LOAD 权限的用户将不再能够执行 `SHOW CREATE VIEW` 语句，必须单独授予 SHOW_VIEW 权限。
- https://github.com/apache/doris/pull/25370

8.5.14 Release 2.0.2

亲爱的社区小伙伴们，Apache Doris 2.0.2 版本已于 2023 年 10 月 6 日正式发布，该版本对多个功能进行了更新优化，旨在更好地满足用户的需求。有 92 位贡献者为 Apache Doris 2.0.2 版本提交了功能优化项以及问题修复，进一步提升了系统的稳定性和性能，欢迎大家下载体验。

GitHub 下载：https://github.com/apache/doris/releases/tag/2.0.2-rc05

官网下载页：https://doris.apache.org/download/

8.5.14.1 Behavior Changes

- https://github.com/apache/doris/pull/24679

删除与 lambda 函数语法冲突的 json“->”运算符，可以使用函数 json_extract 代替。

- https://github.com/apache/doris/pull/24308

将 `metadata_failure_recovery` 从 fe.conf 移动到 start_fe.sh 参数，以避免异常操作。

- https://github.com/apache/doris/pull/24207

对于普通类型中的 null 值使用 \n 来表示，对于复杂类型或嵌套类型的 null 值，跟 JSON 类型保持一致、采取 null 来表示。

- https://github.com/apache/doris/pull/23795
- https://github.com/apache/doris/pull/23784

优化 BE 节点 priority_network 配置项的绑定策略，如果用户配置了错误的 priority_network 则直接启动失败，以避免用户错误地认为配置是正确的。如果用户没有配置 priority_network，则仅从 IPv4 列表中选择第一个 IP，而不是从所有 IP 中选择，以避免用户的服务器不支持 IPv4。

- https://github.com/apache/doris/pull/17730

支持取消正在重试的导入任务，修复取消加载失败的问题。

8.5.14.2 功能优化

8.5.14.2.1 易用性提升

- https://github.com/apache/doris/pull/23887

某些场景下，用户需要向集群中添加一些自定义的库，如 lzo.jar、 orai18n.jar 等。在过去的版本中，这些 lib 文件位于 fe/lib 或 be/lib 中，但在升级集群时，lib 库将被新的 lib 库替换，导致所有自定义的 lib 库都会丢失。

在新版本中，为 FE 和 BE 添加了新的自定义目录 custom_lib，用户可以在其中放置自定义 lib 文件。

- https://github.com/apache/doris/pull/23022

支持基于用户角色的权限访问控制，实现了行级细粒度的权限控制策略。

8.5.14.2.2 改进查询优化器 Nereids 统计信息收集

- https://github.com/apache/doris/pull/23663

在运行 Analysis 任务时禁用 File Cache，Analysis 任务是后台任务，不应影响用户本地 File Cache 数据。

- https://github.com/apache/doris/pull/23703

在过去版本中，查看列的统计信息时将忽略出现错误的列。

在新版本中，当 min 或 max 值未能反序列化时，查看列的统计信息时将使用 N/A 作为 min 或 max 的值并仍显示其余的统计信息，包括 count、 null_count、 ndv 等。

- https://github.com/apache/doris/pull/23965

支持 JDBC 外部表的统计信息收集。

- https://github.com/apache/doris/pull/24625

跳过 `__internal_schema` 和 `information_schema` 上未知列的统计信息检查。

8.5.14.2.3 Multi-Catalog 功能优化

- https://github.com/apache/doris/pull/24168

支持 Hadoop viewfs；

- https://github.com/apache/doris/pull/22369

优化 JDBC Catalog Checksum Replay 和 Range 相关问题；

- https://github.com/apache/doris/pull/23868

优化了 JDBC Catalog 的 Property 检查和错误消息提示。

- https://github.com/apache/doris/pull/24242

修复了 MaxCompute Catalog Decimal 类型解析问题以及使用对象存储地址错误的问题。

- https://github.com/apache/doris/pull/23391

支持 Hive Metastore Catalog 的 SQL Cache。

- https://github.com/apache/doris/pull/22869

提高了 Hive Metastore Catalog 的元数据同步性能。

- https://github.com/apache/doris/pull/22702

添加 metadata_name_ids 以快速获取 Catalogs、DB、Table，在创建或删除 Catalog 和 Table 时无需 Refresh Catalog，并添加 Profiling 表从而与 MySQL 兼容。

8.5.14.2.4 倒排索引性能优化

- https://github.com/apache/doris/pull/23952

增加 bkd 索引的查询缓存，通过缓存可以加速在命中 bkd 索引时的查询性能，在高并发场景中效果更为明显；

- https://github.com/apache/doris/pull/24678

提升倒排索引在 Count 算子上的查询性能；

- https://github.com/apache/doris/pull/24751

提升了 Match 算子在未命中索引时的效率，在测试表现中性能最高提升 60 倍；

- https://github.com/apache/doris/pull/23871
- https://github.com/apache/doris/pull/24389

提升了 MATCH 和 MATCH_ALL 在倒排索引上的查询性能；

8.5.14.2.5 Array 函数优化

- https://github.com/apache/doris/pull/23630

优化了老版本查询优化器 Array 函数无法处理 Decimal 类型的问题；

- https://github.com/apache/doris/pull/24327

优化了 `array_union` 数组函数对多个参数的支持；

- https://github.com/apache/doris/pull/24455

支持通过 explode 函数来处理数组嵌套复杂类型；

8.5.14.3 Bug 修复

修复了之前版本存在的部分 Bug，使系统整体稳定性表现得到大幅提升，完整 BugFix 列表请参考 GitHub Commits 记录；

- https://github.com/apache/doris/pull/23601
- https://github.com/apache/doris/pull/23630
- https://github.com/apache/doris/pull/23555
- https://github.com/apache/doris/pull/17644
- https://github.com/apache/doris/pull/23779
- https://github.com/apache/doris/pull/23940
- https://github.com/apache/doris/pull/23860
- https://github.com/apache/doris/pull/23973
- https://github.com/apache/doris/pull/24020
- https://github.com/apache/doris/pull/24039
- https://github.com/apache/doris/pull/23958
- https://github.com/apache/doris/pull/24104
- https://github.com/apache/doris/pull/24097
- https://github.com/apache/doris/pull/23852
- https://github.com/apache/doris/pull/24139
- https://github.com/apache/doris/pull/24165
- https://github.com/apache/doris/pull/24164
- https://github.com/apache/doris/pull/24369
- https://github.com/apache/doris/pull/24372
- https://github.com/apache/doris/pull/24381
- https://github.com/apache/doris/pull/24385
- https://github.com/apache/doris/pull/24290
- https://github.com/apache/doris/pull/24207
- https://github.com/apache/doris/pull/24521
- https://github.com/apache/doris/pull/24460
- https://github.com/apache/doris/pull/24568

- https://github.com/apache/doris/pull/24610
- https://github.com/apache/doris/pull/24595
- https://github.com/apache/doris/pull/24616
- https://github.com/apache/doris/pull/24635
- https://github.com/apache/doris/pull/24625
- https://github.com/apache/doris/pull/24572
- https://github.com/apache/doris/pull/24578
- https://github.com/apache/doris/pull/23943
- https://github.com/apache/doris/pull/24697
- https://github.com/apache/doris/pull/24681
- https://github.com/apache/doris/pull/24617
- https://github.com/apache/doris/pull/24692
- https://github.com/apache/doris/pull/24700
- https://github.com/apache/doris/pull/24389
- https://github.com/apache/doris/pull/24698
- https://github.com/apache/doris/pull/24778
- https://github.com/apache/doris/pull/24782
- https://github.com/apache/doris/pull/24800
- https://github.com/apache/doris/pull/24808
- https://github.com/apache/doris/pull/24636
- https://github.com/apache/doris/pull/24981
- https://github.com/apache/doris/pull/24949

8.5.14.4 致谢

感谢所有在 2.0.2 版本中参与功能开发与优化以及问题修复的所有贡献者，他们分别是：

[@adonis0147](https://github.com/adonis0147) [@airborne12](https://github.com/airborne12) [@amorynan](https://github.com/amorynan) [@AshinGau](https://github.com/AshinGau) [@BePPPower](https://github.com/BePPPower) [@BiteTheDDDDt](https://github.com/BiteTheDDDDt) [@bobhan1](https://github.com/bobhan1) [@ByteYue](https://github.com/ByteYue) [@caiconghui](https://github.com/caiconghui) [@CalvinKirs](https://github.com/CalvinKirs) [@cambyzju](https://github.com/cambyzju) [@ChengDaqi2023](https://github.com/ChengDaqi2023) [@ChinaYiGuan](https://github.com/ChinaYiGuan) [@CodeCooker17](https://github.com/CodeCooker17) [@csun5285](https://github.com/csun5285) [@dataroaring](https://github.com/dataroaring) [@deadlinefen](https://github.com/deadlinefen) [@DongLiang-0](https://github.com/DongLiang-0) [@Doris-Extras](https://github.com/Doris-Extras) [@dutyu](https://github.com/dutyu) [@eldenmoon](https://github.com/eldenmoon) [@englefly](https://github.com/englefly) [@freemandealer](https://github.com/freemandealer) [@Gabriel39](https://github.com/Gabriel39) [@gnehil](https://github.com/gnehil) [@GoGoWen](https://github.com/GoGoWen) [@gohalo](https://github.com/gohalo) [@HappenLee](https://github.com/HappenLee) [@hello-stephen](https://github.com/hello-stephen) [@HHoflittlefish777](https://github.com/HHoflittlefish777) [@hubgeter](https://github.com/hubgeter) [@hust-hhb](https://github.com/hust-hhb) [@ixzc](https://github.com/ixzc) [@JackDrogon](https://github.com/JackDrogon) [@jacktengg](https://github.com/jacktengg) [@jackwener](https://github.com/jackwener) [@Jibing-Li](https://github.com/Jibing-Li) [@JNSimba](https://github.com/JNSimba) [@kaijchen](https://github.com/kaijchen) [@kaka11chen](https://github.com/kaka11chen) [@Kikyou1997](https://github.com/Kikyou1997) [@Lchangliang](https://github.com/Lchangliang) [@LemonLiTree](https://github.com/LemonLiTree) [@liaoxin01](https://github.com/liaoxin01) [@LiBinfeng-01](https://github.com/LiBinfeng-01) [@liugddx](https://github.com/liugddx) [@luwei16](https://github.com/luwei16) [@mongo360](https://github.com/mongo360) [@morningman](https://github.com/morningman) [@morrySnow](https://github.com/morrySnow) @mrhhsg @Mryange @mymeiyi @neuyilan @pingchunzhang @platoneko @qidaye @realize096 @RYH61 @shuke987 @sohardforaname @starocean999 @SWJTU-ZhangLei @TangSiyang2001 @Tech-Circle-48 @w41ter @wangbo @wsjz @wuwenchi @wyx123654 @xiaokang @XieJiann @xinyiZzz

@XuJianxu @xutaoustc @xy720 @xyfsjq @xzj7019 @yiguolei @yujun777 @Yukang-Lian @Yulei-Yang @zclllyybb @zddr @zhangguo-qiang666 @zhangstar333 @ZhangYu0123 @zhannngchen @zxealous @zy-kkk @zzzxl1993 @zzzzzzzs

8.5.15 Release 2.0.1

亲爱的社区小伙伴们，我们很高兴地向大家宣布，Apache Doris 2.0.1 Release 版本已于 2023 年 9 月 4 日正式发布，有超过 71 位贡献者为 Apache Doris 提交了超过 380 个优化与修复。

- 将 varchar 默认长度 1 修改为 65533

8.5.16 功能改进

8.5.16.0.1 Array 和 Map 数据类型的功能优化及稳定性改进

- https://github.com/apache/doris/pull/22793
- https://github.com/apache/doris/pull/22927
- https://github.com/apache/doris/pull/22738
- https://github.com/apache/doris/pull/22347
- https://github.com/apache/doris/pull/23250
- https://github.com/apache/doris/pull/22300

8.5.16.0.2 倒排索引的查询性能优化

- https://github.com/apache/doris/pull/22836
- https://github.com/apache/doris/pull/23381
- https://github.com/apache/doris/pull/23389
- https://github.com/apache/doris/pull/22570

8.5.16.0.3 bitmap、 like、 scan、 agg 等执行性能优化

- https://github.com/apache/doris/pull/23172
- https://github.com/apache/doris/pull/23495
- https://github.com/apache/doris/pull/23476
- https://github.com/apache/doris/pull/23396
- https://github.com/apache/doris/pull/23182
- https://github.com/apache/doris/pull/22216

8.5.16.0.4 CCR 的功能优化与稳定性提升

- https://github.com/apache/doris/pull/22447
- https://github.com/apache/doris/pull/22559
- https://github.com/apache/doris/pull/22173
- https://github.com/apache/doris/pull/22678

8.5.16.0.5 Merge-on-Write 主键表的能力增强

- https://github.com/apache/doris/pull/22282
- https://github.com/apache/doris/pull/22984
- https://github.com/apache/doris/pull/21933
- https://github.com/apache/doris/pull/22874

8.5.16.0.6 表状态和统计信息的功能优化

- https://github.com/apache/doris/pull/22658
- https://github.com/apache/doris/pull/22211
- https://github.com/apache/doris/pull/22775
- https://github.com/apache/doris/pull/22896
- https://github.com/apache/doris/pull/22788
- https://github.com/apache/doris/pull/22882

8.5.16.0.7 Multi-Catalog 的功能优化及稳定性改进

- https://github.com/apache/doris/pull/22949
- https://github.com/apache/doris/pull/22923
- https://github.com/apache/doris/pull/22336
- https://github.com/apache/doris/pull/22915
- https://github.com/apache/doris/pull/23056
- https://github.com/apache/doris/pull/23297
- https://github.com/apache/doris/pull/23279

8.5.17 问题修复

修复了若干个 2.0.0 版本中的问题，使系统稳定性得到进一步提升

- https://github.com/apache/doris/pull/22673
- https://github.com/apache/doris/pull/22656
- https://github.com/apache/doris/pull/22892
- https://github.com/apache/doris/pull/22959
- https://github.com/apache/doris/pull/22902
- https://github.com/apache/doris/pull/22976
- https://github.com/apache/doris/pull/22734
- https://github.com/apache/doris/pull/22840
- https://github.com/apache/doris/pull/23008
- https://github.com/apache/doris/pull/23003
- https://github.com/apache/doris/pull/22966
- https://github.com/apache/doris/pull/22965
- https://github.com/apache/doris/pull/22784
- https://github.com/apache/doris/pull/23049

- https://github.com/apache/doris/pull/23084
- https://github.com/apache/doris/pull/22947
- https://github.com/apache/doris/pull/22919
- https://github.com/apache/doris/pull/22979
- https://github.com/apache/doris/pull/23096
- https://github.com/apache/doris/pull/23113
- https://github.com/apache/doris/pull/23062
- https://github.com/apache/doris/pull/22918
- https://github.com/apache/doris/pull/23026
- https://github.com/apache/doris/pull/23175
- https://github.com/apache/doris/pull/23167
- https://github.com/apache/doris/pull/23015
- https://github.com/apache/doris/pull/23165
- https://github.com/apache/doris/pull/23264
- https://github.com/apache/doris/pull/23246
- https://github.com/apache/doris/pull/23198
- https://github.com/apache/doris/pull/23221
- https://github.com/apache/doris/pull/23277
- https://github.com/apache/doris/pull/23249
- https://github.com/apache/doris/pull/23272
- https://github.com/apache/doris/pull/23383
- https://github.com/apache/doris/pull/23372
- https://github.com/apache/doris/pull/23399
- https://github.com/apache/doris/pull/23295
- https://github.com/apache/doris/pull/23446
- https://github.com/apache/doris/pull/23406
- https://github.com/apache/doris/pull/23387
- https://github.com/apache/doris/pull/23421
- https://github.com/apache/doris/pull/23456
- https://github.com/apache/doris/pull/23361
- https://github.com/apache/doris/pull/23402
- https://github.com/apache/doris/pull/23369
- https://github.com/apache/doris/pull/23245
- https://github.com/apache/doris/pull/23532
- https://github.com/apache/doris/pull/23529
- https://github.com/apache/doris/pull/23601

优化改进及修复问题的完整列表请在 GitHub 按照标签 dev/2.0.1-merged 进行筛选即可。

8.5.18 致谢

向所有参与 Apache Doris 2.0.1 版本开发和测试的贡献者们表示最衷心的感谢，他们分别是：

adonis0147、airborne12、amorynan、AshinGau、BePPPower、BiteTheDDDDt、bobhan1、ByteYue、caiconghui、CalvinKirs、csun5285、DarvenDuan、deadlinefen、DongLiang-0、Doris-Extras、dutyu、englefly、freemandealer、Gabriel39、GoGoWen、HappenLee、hello-stephen、HHoflittlefish777、hubgeter、hust-hhb、JackDrogon、jacktengg、jackwener、Jibing-Li、kaijchen、

kaka11chen、Kikyou1997、Lchangliang、LemonLiTree、liaoxin01、LiBinfeng-01、lsy3993、luozenglin、morningman、morrySnow、mrhhsg、Mryange、mymeiyi、shuke987、sohardforaname、starocean999、TangSiyang2001、Tanya-W、ucasfl、vinlee19、wangbo wsjz、wuwenchi、xiaokang、XieJiann、xinyiZzz、yujun777、Yukang-Lian、Yulei-Yang、zclllyybb、zddr、zenoyang、zgxme、zhangguoqiang666、zhangstar333、zhannngchen、zhiqiang-hhhh、zxealous、zy-kkk、zzzxl1993、zzzzzzzs

8.5.19 Release 2.0.0

亲爱的社区小伙伴们，我们很高兴地向大家宣布，Apache Doris 2.0.0 Release 版本已于 2023 年 8 月 11 日正式发布，有超过 275 位贡献者为 Apache Doris 提交了超过 4100 个优化与修复。

在 2.0.0 版本中，Apache Doris 在标准 Benchmark 数据集上盲测查询性能得到超过 10 倍的提升、在日志分析和湖仓一体场景能力得到全面加强、数据更新效率和写入效率都更加高效稳定、支持了更加完善的多租户和资源隔离机制、在资源弹性与存算分离方向踏上了新的台阶、增加了一系列面向企业用户的易用性特性。在经过近半年的开发、测试与稳定性调优后，这一版本已经正式稳定可用，欢迎大家下载使用！

> 下载链接：https://doris.apache.org/download
>
> GitHub 源码：https://github.com/apache/doris/tree/2.0.0-rc04

8.5.19.1 盲测性能 10 倍以上提升！

在 Apache Doris 2.0.0 版本中，我们引入了全新查询优化器和自适应的并行执行模型，结合存储层、执行层以及执行算子上的一系列性能优化手段，实现了盲测性能 10 倍以上的提升。以 SSB-Flat 和 TPC-H 标准测试数据集为例，在相同的集群和机器配置下，新版本宽表场景盲测较之前版本性能提升 10 倍、多表关联场景盲测提升了 13 倍，实现了巨大的性能飞跃。

8.5.19.1.1 更智能的全新查询优化器

全新查询优化器采取了更先进的 Cascades 框架、使用了更丰富的统计信息、实现了更智能化的自适应调优，在绝大多数场景无需任何调优和 SQL 改写即可实现极致的查询性能，同时对复杂 SQL 支持得更加完备、可完整支持 TPC-DS 全部 99 个 SQL。通过全新查询优化器，我们可以胜任更多真实业务场景的挑战，减少因人工调优带来的人力消耗，真正助力业务提效。

以 TPC-H 为例，全新优化器在未进行任何手工调优和 SQL 改写的情况下，绝大多数 SQL 仍领先于旧优化器手工调优后的性能表现！而在超过百家 2.0 版本提前体验用户的真实业务场景中，绝大多数原始 SQL 执行效率得以极大提升！

参考文档：更智能的全新查询优化器

如何开启：`SET enable_nereids_planner=true` 在 Apache Doris 2.0-beta 版本中全新查询优化器已经默认开启

8.5.19.1.2 倒排索引支持

在 2.0.0 版本中我们对现有的索引结构进行了丰富，引入了倒排索引来应对多维度快速检索的需求，在关键字模糊查询、等值查询和范围查询等场景中均取得了显著的查询性能和并发能力提升。

在此以某头部手机厂商的用户行为分析场景为例，在之前的版本中，随着并发量的上升、查询耗时逐步提升，性能下降趋势比较明显。而在 2.0.0 版本开启倒排索引后，随着并发量的提升查询性能始终保持在毫秒级。在同等查询并发量的情况下，2.0.0 版本在该用户行为分析场景中并发查询性能提升了 5-90 倍！

8.5.19.1.3 点查询并发能力提升 20 倍

在银行交易流水单号查询、保险代理人保单查询、电商历史订单查询、快递运单号查询等 Data Serving 场景，会面临大量一线业务人员及 C 端用户基于主键 ID 检索整行数据的需求，同时在用户画像、实时风控等场景中还会面对机器大规模的程序化查询，在过去此类需求往往需要引入 Apache HBase 等 KV 系统来应对点查询、Redis 作为缓存层来分担高并发带来的系统压力。对于基于列式存储引擎构建的 Apache Doris 而言，此类的点查询在数百列宽表上将会放大随机读取 IO，并且查询优化器和执行引擎对于此类简单 SQL 的解析、分发也将带来不必要的额外开销，负责 SQL 解析的 FE 模块往往会成为限制并发的瓶颈，因此需要更高效简洁的执行方式。

在 Apache Doris 2.0.0 版本，我们引入了全新的行列混合存储以及行级 Cache，使得单次读取整行数据时效率更高、大大减少磁盘访问次数，同时引入了点查询短路径优化、跳过执行引擎并直接使用快速高效的读路径来检索所需的数据，并引入了预处理语句复用执行 SQL 解析来减少 FE 开销。

通过以上一系列优化，Apache Doris 2.0.0 版本在并发能力上实现了数量级的提升，实现了单节点 30000 QPS 的并发表现，较过去版本点查询并发能力提升超 20 倍！

基于以上能力，Apache Doris 可以更好应对高并发数据服务场景的需求，替代 HBase 在此类场景中的能力，减少复杂技术栈带来的维护成本以及数据的冗余存储。

8.5.19.1.4 自适应的并行执行模型

在实现极速分析体验的同时，为了保证多个混合分析负载的执行效率以及查询的稳定性，在 2.0.0 版本中我们引入了 Pipeline 执行模型作为查询执行引擎。在 Pipeline 执行引擎中，查询的执行是由数据来驱动控制流变化的，各个查询执行过程之中的阻塞算子被拆分成不同 Pipeline，各个 Pipeline 能否获取执行线程调度执行取决于前置数据是否就绪，实现了阻塞操作的异步化、可以更加灵活地管理系统资源，同时减少了线程频繁创建和销毁带来的开销，并提升了 Apache Doris 对于 CPU 的利用效率。因此 Apache Doris 在混合负载场景中的查询性能和稳定性都得到了全面提升。

参考文档：查询执行引擎

如何开启：`Set enable_pipeline_engine = true` - 该功能在 Apache Doris 2.0 版本中将默认开启，BE 在进行查询执行时默认将 SQL 的执行模型转变 Pipeline 的执行方式。- `parallel_pipeline_task_num`代表了 SQL 查询进行查询并发的 Pipeline Task 数目。Apache Doris 默认配置为0，此时 Apache Doris 会自动感知每个 BE 的 CPU 核数并把并发度设置为 CPU 核数的一半，用户也可以根据自己的实际情况进行调整。- 对于从老版本升级的用户，系统自动将该参数设置成老版本中`parallel_fragment_exec_instance_num`的值。

8.5.19.2 更统一多样的分析场景

作为最初诞生于报表分析场景的 OLAP 系统，Apache Doris 在这一擅长领域中做到了极致，凭借自身优异的分析性能和极简的使用体验收获到了众多用户的认可，在诸如实时看板（Dashboard）、实时大屏、业务报表、管理驾驶舱等实时报表场景以及自助 BI 平台、用户行为分析等即席查询场景获得了极为广泛的运用。

而随着用户规模的极速扩张，越来越多用户开始希望通过 Apache Doris 来简化现有的繁重大数据技术栈，减少多套系统带来的使用及运维成本。因此 Apache Doris 也在不断拓展应用场景的边界，从过去的实时报表和 Ad-hoc 等典型 OLAP 场景到湖仓一体、ELT/ETL、日志检索与分析、高并发 Data Serving 等更多业务场景，而日志检索分析、湖仓一体也是我们在 Apache Doris 最新版本中的重要突破。

8.5.19.2.1 10 倍以上性价比的日志检索分析平台

在 Apache Doris 2.0.0 版本中，我们提供了原生的半结构化数据支持，在已有的 JSON、Array 基础之上增加了复杂类型 Map，并基于 Light Schema Change 功能实现了 Schema Evolution。与此同时，2.0.0 版本新引入的倒排索引和高性能文本分析算法全面加强了 Apache Doris 在日志检索分析场景的能力，可以支持更高效的任意维度分析和全文检索。结合过去在大规模数据写入和低成本存储等方面的优势，相对于业内常见的日志分析解决方案，基于 Apache Doris 构建的新一代日志检索分析平台实现了 10 倍以上的性价比提升。

8.5.19.2.2 湖仓一体

在 Apache Doris 1.2 版本中，我们引入了 Multi-Catalog 功能，支持了多种异构数据源的元数据自动映射与同步，实现了便捷的元数据和数据打通。在 2.0.0 版本中，我们进一步对湖仓一体进行了加强，引入了更多数据源，并针对用户的实际生产环境做了诸多性能优化，在真实工作负载情况下查询性能得到大幅提升。

在数据源方面，Apache Doris 2.0.0 版本支持了 Hudi Copy-on-Write 表的 Snapshot Query 以及 Merge-on-Read 表的 Read Optimized Query，截止目前已经支持了 Hive、Hudi、Iceberg、Paimon、MaxCompute、Elasticsearch、Trino、ClickHouse 等数十种数据源，几乎支持了所有开放湖仓格式和 Metastore。同时还支持通过 Apache Range 对 Hive Catalog 进行鉴权，可以无缝对接用户现有的权限系统。同时还支持可扩展的鉴权插件，为任意 Catalog 实现自定义的鉴权方式。

在性能方面，利用 Apache Doris 自身高效的分布式执行框架、向量化执行引擎以及查询优化器，结合 2.0 版本中对于小文件和宽表的读取优化、本地文件 Cache、ORC/Parquet 文件读取效率优化、弹性计算节点以及外表的统计信息收集，Apaceh Doris 在 TPC-H 场景下查询 Hive 外部表相较于 Presto/Trino 性能提升 3-5 倍。

通过这一系列优化，Apache Doris 湖仓一体的能力得到极大拓展，在如下场景可以更好发挥其优异的分析能力：

- 湖仓查询加速：为数据湖、Elasticsearch 以及各类关系型数据库提供优秀的查询加速能力，相比 Hive、Presto、 Spark 等查询引擎实现数倍的性能提升。
- 数据导入与集成：基于可扩展的连接框架，增强 Apache Doris 在数据集成方面的能力，让数据更便捷的被消费和处理。用户可以通过 Apache Doris 对上游的多种数据源进行统一的增量、全量同步，并利用 Apache Doris 的数据处理能力对数据进行加工和展示，也可以将加工后的数据写回到数据源，或提供给下游系统进行消费。
- 统一数据分析网关：利用 Apache Doris 构建完善可扩展的数据源连接框架，支持用户将这些外部数据源统一到 Doris 的元数据映射结构上，当用户通过 Doris 查询这些外部数据源时，能够提供一致的查询体验。

8.5.19.3 高效的数据更新

在实时分析场景中，数据更新是非常普遍的需求。用户不仅希望能够实时查询最新数据，也希望能够对数据进行灵活的实时更新。典型场景如电商订单分析、物流运单分析、用户画像等，需要支持数据更新类型包括整行更新、部分列更新、按条件进行批量更新或删除以及整表或者整个分区的重写（inser overwrite）。

高效的数据更新一直是大数据分析领域的痛点，离线数据仓库 hive 通常只支持分区级别的数据更新，而 Hudi 和 Iceberg 等数据湖，虽然支持 Record 级别更新，但是通常采用 Merge-on-Read 或 Copy-on-Write 的方式，仅适合低频批量更新而不适合实时高频更新。

在 Apache Doris 1.2 版本，我们在 Unique Key 主键模型实现了 Merge-on-Write 的数据更新模式，数据在写入阶段就能完成所有的数据合并工作，因此查询性能得到 5-10 倍的提升。在 Apache Doris 2.0 版本我们进一步加强了数据更新能力，主要包括：

- 对写入性能进行了大幅优化，高并发写入和混合负载写入场景的稳定性也显著提升。例如在单 Tablet 7GB 的重复导入测试中，数据导入的耗时从约 30 分钟缩短到了 90s，写入效率提升 20 倍；以某头部支付产品的场景压测为例，在 20 个并行写入任务下可以达到 30 万条每秒的写入吞吐，并且持续写入十几个小时后仍然表现非常稳定。
- 支持部分列更新功能。在 2.0.0 版本之前 Apache Doris 仅支持通过 Aggregate Key 聚合模型的 Replace_if_not_null 进行部分列更新，在 2.0.0 版本中我们增加了 Unique Key 主键模型的部分列更新，在多张上游源表同时写入一张宽表时，无需由 Flink 进行多流 Join 打宽，直接写入宽表即可，减少了计算资源的消耗并大幅降低了数据处理链路的复杂性。同时在面对画像场景的实时标签列更新、订单场景的状态更新时，直接更新指定的列即可，较过去更为便捷。
- 支持复杂条件更新和条件删除。在 2.0.0 版本之前 Unique Key 主键模型仅支持简单 Update 和 Delete 操作，在 2.0.0 版本中我们基于 Merge-on-Write 实现了复杂条件的数据更新和删除，并且执行效率更加高效。基于以上优化，Apache Doris 对于各类数据更新需求都有完备的能力支持！

8.5.19.4 更加高效稳定的数据写入

8.5.19.4.1 导入性能进一步提升

聚焦于实时分析，我们在过去的几个版本中在不断增强实时分析能力，其中端到端的数据实时写入能力是优化的重要方向，在 Apache Doris 2.0 版本中，我们进一步强化了这一能力。通过 Memtable 不使用 Skiplist、并行下刷、单副本导入等优化，使得导入性能有了大幅提升：

- 使用 Stream Load 对 TPC-H 144G lineitem 表原始数据进行三副本导入 48 buckets Duplicate 表，吞吐量提升 100%。
- 使用 Stream Load 对 TPC-H 144G lineitem 表原始数据进行三副本导入 48 buckets Unique Key 表，吞吐量提升 200%。
- 使用 insert into select 对 TPC-H 144G lineitem 表进行导入 48 buckets Duplicate 表，吞吐量提升 50%。
- 使用 insert into select 对 TPC-H 144G lineitem 表进行导入 48 buckets Unique Key 表，吞吐提升 150%。

8.5.19.4.2 数据高频写入更稳定

在高频数据写入过程中，小文件合并和写放大问题以及随之而来的磁盘 I/O 和 CPU 资源开销是制约系统稳定性的关键，因此在 2.0 版本中我们引入了 Vertical Compaction 以及 Segment Compaction，用以彻底解决 Compaction 内存问题以及写入过程中的 Segment 文件过多问题，资源消耗降低 90%，速度提升 50%，内存占用仅为原先的 10%。

8.5.19.4.3 数据表结构自动同步

在过去版本中我们引入了毫秒级别的 Schema Change，而在最新版本 Flink-Doris-Connector 中，我们实现了从 MySQL 等关系型数据库到 Apache Doris 的一键整库同步。在实际测试中单个同步任务可以承载数千张表的实时并行写入，从此彻底告别过去繁琐复杂的同步流程，通过简单命令即可实现上游业务数据库的表结构及数据同步。同时当上游数据结构发生变更时，也可以自动捕获 Schema 变更并将 DDL 动态同步到 Doris 中，保证业务的无缝运行。

8.5.19.5 更加完善的多租户资源隔离

多租户与资源隔离的主要目的是为了保证高负载时避免相互发生资源抢占，Apache Doris 在过去版本中推出了资源组（Resource Group）的硬隔离方案，通过对同一个集群内部的 BE 打上标签，标签相同的 BE 会组成一个资源组。数据入库时会按照资源组配置将数据副本写入到不同的资源组中，查询时按照资源组的划分使用对应资源组上的计算资源进行计算，例如将读、写流量放在不同的副本上从而实现读写分离，或者将在线与离线业务划分在不同的资源组、避免在离线分析任务之间的资源抢占。

资源组这一硬隔离方案可以有效避免多业务间的资源抢占，但在实际业务场景中可能会存在某些资源组紧张而某些资源组空闲的情况发生，这时需要有更加灵活的方式进行空闲资源的共享，以降低资源空置率。因此在 2.0.0 版本中我们增加了 Workload Group 资源软限制的方案，通过对 Workload 进行分组管理，以保证内存和 CPU 资源的灵活调配和管控。

通过将 Query 与 Workload Group 相关联，可以限制单个 Query 在 BE 节点上的 CPU 和内存资源的百分比，并可以配置开启资源组的内存软限制。当集群资源紧张时，将自动 Kill 组内占用内存最大的若干个查询任务以减缓集群压力。当集群资源空闲时，一旦 Workload Group 使用资源超过预设值时，多个 Workload 将共享集群可用空闲资源并自动突破阈值，继续使用系统内存以保证查询任务的稳定执行。Workload Group 还支持设置优先级，通过预先设置的优先级进行资源分配管理，来确定哪些任务可正常获得资源，哪些任务只能获取少量或没有资源。

与此同时，在 Workload Group 中我们还引入了查询排队的功能，在创建 Workload Group 时可以设置最大查询数，超出最大并发的查询将会进行队列中等待执行，以此来缓解高负载下系统的压力。

8.5.19.6 极致弹性与存算分离

过去 Apache Doris 凭借在易用性方面的诸多设计帮助用户大幅节约了计算与存储资源成本，而面向未来的云原生架构，我们已经走出了坚实的一步。

从降本增效的趋势出发，用户对于计算和存储资源的需求可以概括为以下几方面：

- 计算资源弹性：面对业务计算高峰时可以快速进行资源扩展提升效率，在计算低谷时可以快速缩容以降低成本；
- 存储成本更低：面对海量数据可以引入更为廉价的存储介质以降低成本，同时存储与计算单独设置、相互不干预；
- 业务负载隔离：不同的业务负载可以使用独立的计算资源，避免相互资源抢占；
- 数据管控统一：统一 Catalog、统一管理数据，可以更加便捷地分析数据。

存算一体的架构在弹性需求不强的场景具有简单和易于维护的优势，但是在弹性需求较强的场景有一定的局限。而存算分离的架构本质是解决资源弹性的技术手段，在资源弹性方面有着更为明显的优势，但对于存储具有更高的稳定性要求，而存储的稳定性又会进一步影响到 OLAP 的稳定性以及业务的存续性，因此也引入了 Cache 管理、计算资源管理、垃圾数据回收等一系列机制。

而在与 Apache Doris 社区广大用户的交流中，我们发现用户对于存算分离的需求可以分为以下三类：

- 目前选择简单易用的存算一体架构，暂时没有资源弹性的需求；
- 欠缺稳定的大规模存储，要求在 Apache Doris 原有基础上提供弹性、负载隔离以及低成本；

- 有稳定的大规模存储，要求极致弹性架构、解决资源快速伸缩的问题，因此也需要更为彻底的存算分离架构；

为了满足前两类用户的需求，Apache Doris 2.0 版本中提供了可以兼容升级的存算分离方案：第一种，计算节点。2.0 版本中我们引入了无状态的计算节点 Compute Node，专门用于数据湖分析。相对于原本存储计算一体的混合节点，Compute Node 不保存任何数据，在集群扩缩容时无需进行数据分片的负载均衡，因此在数据湖分析这种具有明显高峰的场景中可以灵活扩容、快速加入集群分摊计算压力。同时由于用户数据往往存储在 HDFS/S3 等远端存储中，执行查询时查询任务会优先调度到 Compute Node 执行，以避免内表与外表查询之间的计算资源抢占。

第二种，冷热数据分层。在存储方面，冷热数据往往面临不同频次的查询和响应速度要求，因此通常可以将冷数据存储在成本更低的存储介质中。在过去版本中 Apache Doris 支持对表分区进行生命周期管理，通过后台任务将热数据从 SSD 自动冷却到 HDD，但 HDD 上的数据是以多副本的方式存储的，并没有做到最大程度的成本节约，因此对于冷数据存储成本仍然有较大的优化空间。在 Apache Doris 2.0 版本中推出了冷热数据分层功能，冷热数据分层功能使 Apache Doris 可以将冷数据下沉到存储成本更加低廉的对象存储中，同时冷数据在对象存储上的保存方式也从多副本变为单副本，存储成本进一步降至原先的三分之一，同时也减少了因存储附加的计算资源成本和网络开销成本。通过实际测算，存储成本最高可以降低超过 70%！

面对更加彻底的存储计算分离需求，飞轮科技（SelectDB）技术团队设计并实现了全新的云原生存算分离架构（SelectDB Cloud），近一年来经历了大量企业客户的大规模使用，在性能、功能成熟度、系统稳定性等方面经受了真实生产环境的考验。在 Apache Doris 2.0.0 版本发布之际，飞轮科技宣布将这一经过大规模打磨后的成熟架构贡献至 Apache Doris 社区。这一工作预计将于 2023 年 10 月前后完成，届时全部存算分离的代码都将会提交到 Apache Doris 社区主干分支中，预计在 9 月广大社区用户就可以提前体验到基于存算分离架构的预览版本。

8.5.19.7 易用性进一步提升

除了以上功能需求外，在 Apache Doris 还增加了许多面向企业级特性的体验改进：

8.5.19.7.1 支持 Kubernetes 容器化部署

在过去 Apache Doris 是基于 IP 通信的，在 K8s 环境部署时由于宿主机故障发生 Pod IP 漂移将导致集群不可用，在 2.0 版本中我们支持了 FQDN，使得 Apache Doris 可以在无需人工干预的情况下实现节点自愈，因此可以更好应对 K8s 环境部署以及灵活扩缩容。

8.5.19.7.2 跨集群数据复制

在 Apache Doris 2.0.0 版本中，我们可以通过 CCR 的功能在库/表级别将源集群的数据变更同步到目标集群，可根据场景精细控制同步范围；用户也可以根据需求灵活选择全量或者增量同步，有效提升了数据同步的灵活性和效率；此外 Dors CCR 还支持 DDL 同步，源集群执行的 DDL 语句可以自动同步到目标集群，从而保证了数据的一致性。Doris CCR 配置和使用也非常简单，简单操作即可快速完成跨集群数据复制。基于 Doris CCR 优异的能力，可以更好实现读写负载分离以及多机房备份，并可以更好支持不同场景的跨集群复制需求。

8.5.19.8 其他升级注意事项

- 1.2-lts 需要停机升级到 2.0.0，2.0-alpha 需要停机升级到 2.0.0
- 查询优化器开关默认开启 `enable_nereids_planner=true`；
- 系统中移除了非向量化代码，所以 `enable_vectorized_engine` 参数将不再生效；

- 新增参数 enable_single_replica_compaction；
- 默认使用 datev2, datetimev2, decimalv3 来创建表，不支持 datev1，datetimev1，decimalv2 创建表；
- 在 JDBC 和 Iceberg Catalog 中默认使用 decimalv3；
- date type 新增 AGG_STATE；
- backend 表去掉 cluster 列；
- 为了与 BI 工具更好兼容，在 show create table 时，将 datev2 和 datetimev2 显示为 date 和 datetime。
- 在 BE 启动脚本中增加了 max_openfiles 和 swap 的检查，所以如果系统配置不合理，be 有可能会启动失败；
- 禁止在 localhost 访问 FE 时无密码登录；
- 当系统中存在 Multi-Catalog 时，查询 information schema 的数据默认只显示 internal catalog 的数据；
- 限制了表达式树的深度，默认为 200；
- array string 返回值单引号变双引号；
- 对 Doris 的进程名重命名为 DorisFE 和 DorisBE；
- AES 和 SM4 加解密函数的两参数版本行为变化，详见对应函数文档

8.5.19.9 正式踏上 2.0 之旅

在 Apache Doris 2.0.0 版本发布过程中，我们邀请了数百家企业参与新版本的打磨，力求为所有用户提供性能更佳、稳定性更高、易用性更好的数据分析体验。后续我们将会持续敏捷发版来响应所有用户对功能和稳定性的更高追求，预计 2.0 系列的第一个迭代版本 2.0.1 将于 8 月下旬发布，9 月会进一步发布 2.0.2 版本。在快速 Bugfix 的同时，也会不断将一些最新特性加入到新版本中。9 月份我们还将发布 2.1 版本的尝鲜版本，会增加一系列呼声已久的新能力，包括 Variant 可变数据类型以更好满足半结构化数据 Schema Free 的分析需求，多表物化视图，在导入性能方面持续优化、增加新的更加简洁的数据导入方式，通过自动攒批实现更加实时的数据写入，复合数据类型的嵌套能力等。

期待 Apache Doris 2.0 版本的正式发布为更多社区用户提供实时统一的分析体验，我们也相信 Apache Doris 2.0 版本会成为您在实时分析场景中的最理想选择。

8.5.19.10 致谢

再次向所有参与 Apache Doris 2.0.0 版本开发和测试的贡献者们表示最衷心的感谢，他们分别是：

0xflotus、1330571、15767714253、924060929、ArmandoZ、AshinGau、BBB-source、BePPPower、Bears0haunt、BiteTheDDDDt、ByteYue、Cai-Yao、CalvinKirs、Centurybbx、ChaseHuangxu、CodeCooker17、DarvenDua、Dazhuwei、DongLiang-0、EvanTheBoy、FreeOnePlus、Gabriel39、GoGoWen、HHoflittlefish777、HackToday、HappenLee、Henry2SS、HonestManXin、JNSimba、JackDrogon、Jake-00、Jenson97Jibing-Li、Johnnyssc、JoverZhang、KassieZ、Kikyou1997、Larborator、Lchangliang、LemonLiTree、LiBinfeng-01、MRYOG、Mellorssss、Moonm3n、Mryange、Myasuka、NetShrimp06、Reminiscent、SWJTU-ZhangLei、SaintBacchus、ShaoStaticTiger、Shoothzj、SilasKenneth、TangSiyang2001、Tanya-W、TeslaCN、TsukiokaKogane、UnicornLee、WinkerDu、WuWQ98、Xiaoccer、XieJiann、Yanko-7、Yukang-Lian、Yulei-Yang、ZI-MA、ZashJie、ZhangYu0123、Zhiyu-h、adonis0147、airborne12、alissa-tung、amorynan、beijita、bigben0204、bin41215、bingquanzhao、bobhan1、bowenliang123、brody715、caiconghui、cambyzju、caoliang-web、catpineapple、chenlinzhong、cjq9458、cnissnzg、colagy、csun5285、czzmmc、dataroaring、davidshtian、deadlinefen、deardeng、didiaode18、dong-shuai、dujl、dutyu、echo-hhj、eldenmoon、englefly、figurant、fornaix、fracly、freemandealer、fsilent、fuchanghai、gavinchou、git-hulk、gitccl、gnehil、guoxiaolongzte、gwxog、hailin0、hanyisong、haochengxia、haohuaijin、hechao-ustc、hello-stephen、herry2038、hey-hoho、hf200012、hqx871、httpshirley、htyoung、hubgeter、hufengkai、hust-hhb、isHuangXin、ixzc、jacktengg、jackwener、jeffreys-cat、jiugem、jixxiong、kaijchen、kaka11chen、levy5307、lexluo09、liangjiawei1110、liaoxin01、liugddx、liujinhui1994、liujiwen-up、liutang123、liuxinzero07、liwei9902、lljqy、lsy3993、luozenglin、luwei16、luzhijing、lvshaokang、maochongxin、meredith620、mklzl、mongo360、morningman、morrySnow、mrhhsg、myfjdthink、mymeiyi、nanfeng1999、neuyilan、nextdreamblue、niebayes、nikam14、pengxiangyu、pingchunzhang、platoneko、q763562998、qidaye、

qzsee、reswqa、sepastian、shenxingwuying、shuke987、shysnow、siriume、sjyago、skyhitnow、smallhibiscus、sohardforaname、spaces-X、stalary、starocean999、superspeedone、taomengen、tarepanda1024、timyuer、ucasfl、vinlee19、wangbo、wanghuan2054、wangshuo128、wangtianyi2004、wangyf0555、wangyujia2023、web-flow、weizhengte、weizuo93、whutpencil、wsjz、wuwenchi、wzymumon、xiaojunjie、xiaokang、xiedeyantu、xinyiZzz、xuqinghuang、xutaoustc、xy720、xzj7019、ya-dao、yagagagaga、yangzhg、yiguolei、yimeng、yinzhijian、yixiutt、yongjinhou、youtNa、yuanyuan8983、yujian225、yujun777、yuxuan-luo、yz-jayhua、zbtzbtzbt、zclllyybb、zddr、zenoyang、zgxme、zhangguoqiang666、zhangstar333、zhangy5、zhannngchen、zhbinbin、zhengshengjun、zhengshiJ、zwuis、zxealous、zy-kkk、zzzxl1993、zzzzzzzs

8.6 v1.2

8.6.1 Release 1.2.8

亲爱的社区小伙伴们，Apache Doris 1.2.8 版本已于 2024 年 3 月 09 日正式与大家见面。该版本对多个功能进行了更新优化，旨在更好地满足用户的需求，欢迎大家下载体验。

官网下载： https://doris.apache.org/download/

GitHub 下载： https://github.com/apache/doris/releases

8.6.1.1 改进和优化

- 修复若干查询执行的问题
- 修复若干 Spark Load 相关的问题
- 修复若干 Parquet/ORC 文件读取的问题。
- 修复 Broker 进行因为 “FileSystem closed” 错误导致运行失败的问题。
- 修复若干 Broker Load 相关的问题。
- 修复若干 CTAS 操作相关的问题。
- 修复若干备份恢复功能相关的问题。
- 修复若干导出（Export/Outfile）相关的问题。
- 修复 `replayEraseTable` 方法导致 FE 无法启动的问题。
- 优化 Iceberg Catalog 元数据缓存的性能。
- Audit Log 中新增 Catalog 列。

8.6.2 Release 1.2.7

- 修复了一些查询问题。
- 修复了一些存储问题。
- 修复一些小数精度问题。
- 修复由无效的 sql_select_limit 会话变量值引起的查询错误。
- 修复了无法使用 hdfs 短路读取的问题。
- 修复了腾讯云 cosn 无法访问的问题。
- 修复了一些 Hive Catalog kerberos 访问的问题。
- 修复 Stream load Profile 无法使用的问题。
- 修复 Promethus 监控参数格式问题。
- 修复了创建大量 Tablet 时建表超时的问题。

8.6.3 最新特性

- Unique Key 模型支持将数组类型作为 Key 列；-添加了 have_query_cache 变量以保证与 MySQL 生态系统兼容。-添加 enable_strong _consistency_read 以支持会话之间的强一致性读取。-FE 指标支持用户级的查询计数器。

8.6.4 Release 1.2.6

- 新增 BE 配置项 `allow_invalid_decimalv2_triteral` 以控制是否可以导入超过小数精度的 Decimal 类型数据，用于兼容之前的逻辑。

8.6.5 Bug Fixes

8.6.5.1 查询

- 修复了部分查询计划的问题；
- 支持会话变量 `sql_select_limit` 和 `have_query_cache` 用于与老版本的 MySQL 客户端兼容；
- 优化 Cold Run 查询性能；
- 修复 Expr Context 类内存泄漏的问题；
- 修复 `explode_split` 函数在某些情况下执行错误的问题。

8.6.5.2 Multi Catalog

- 修复了同步 Hive 元数据时 FE 回放元数据日志失败的问题；
- 修复了 `refresh catalog` 操作可能导致 FE OOM 的问题；
- 修复了 JDBC Catalog 无法正确处理 `0000-00-00` 日期格式的问题；
- 修复了 kerberos ticket 无法自动刷新的问题；
- 优化了 Hive Partition 裁剪性能；
- 修复 JDBC Catalog 中 Trino 和 Presto 不一致的行为；
- 修复了在某些环境中无法使用 HDFS 短路读取来提高查询效率的问题；
- 修复无法读取 CHDFS Iceberg 表的问题。

8.6.5.3 存储

- 修复 Merge-on-Write 表中删除 bitmap 逻辑计算错误的问题；
- 修复了若干 BE 内存问题；
- 修复了表数据 Snappy 压缩的问题；
- 修复 jemalloc 在某些情况下可能导致 BE 崩溃的问题。

8.6.5.4 其他

- 修复了部分 Java UDF 相关问题；
- 修复了 `recover table` 操作错误地触发动态分区创建的问题；
- 修复了通过 Broker Load 导入 orc 文件时的时区问题；

- 修复新添加的 PERCENT 关键字导致 Routine Load 作业的回放元数据失败的问题；
- 修复了 `truncate` 操作无法作用于非分区表的问题；
- 修复了由于 `show snapshot` 操作导致 MySQL 连接丢失的问题；
- 优化锁逻辑以降低创建表时发生锁超时错误的概率；
- 优化了导入发生错误时的报错信息。

8.6.6 致谢

感谢以下开发者在 Apache Doris 1.2.6 版本中所做的贡献；

@amorynan

@BiteTheDDDDt

@caoliang-web

@dataroaring

@Doris-Extras

@dutyu

@Gabriel39

@HHoflittlefish777

@htyoung

@jacktengg

@jeffreys-cat

@kaijchen

@kaka11chen

@Kikyou1997

@KnightLiJunLong

@liaoxin01

@LiBinfeng-01

@morningman

@mrhhsg

@sohardforaname

@starocean999

@vinlee19

@wangbo

@wsjz

@xiaokang

@xinyiZzz

@yiguolei

@yujun777

@Yulei-Yang

@zhangstar333

@zy-kkk

8.6.7 Release 1.2.5

在 1.2.5 版本中，Doris 团队已经修复了自 1.2.4 版本发布以来近 210 个问题或性能改进项。同时，1.2.5 版本也作为 1.2.4 的迭代版本，具备更高的稳定性，建议所有用户升级到这个版本。

- BE 启动脚本会检查系统的最大文件句柄数需大于等于 65536，否则启动失败。
- BE 配置项 `enable_quick_compaction` 默认设为 true。即默认开启 Quick Compaction 功能。该功能用于优化大批量导入情况下的小文件问题。
- 修改表的动态分区属性后，将不再立即生效，而是统一等待下一次动态分区表的任务调度，以避免一些死锁问题。

8.6.8 Improvement

- 优化 bthread 和 pthread 的使用，减少查询过程中的 RPC 阻塞问题。
- FE 前端页面的 Profile 页面增加下载 Profile 的按钮。
- 新增 FE 配置 `recover_with_skip_missing_version`，用于在某些故障情况下，查询跳过有问题的数据副本。
- 行级权限功能支持 Catalog 外表。
- Hive Catalog 支持 BE 端自动刷新 kerberos 票据，无需手动刷新。
- JDBC Catalog 支持通过 MySQL/ClickHouse 系统库（`information_schema`）下的表。

8.6.9 Bug Fixes

- 修复低基数列优化导致的查询结果不正确的问题
- 修复若干访问 HDFS 的认证和兼容性问题。
- 修复若干浮点和 decimal 类型的问题。
- 修复若干 date/datetimev2 类型的问题。
- 修复若干查询执行和规划的问题。
- 修复 JDBC Catalog 的若干问题。

- 修复 Hive Catalog 的若干查询相关问题，以及 Hive Metastore 元数据同步的问题。
- 修复 `show load profile` 结果不正确的问题。
- 修复若干内存相关问题。
- 修复 `CREATE TABLE AS SELECT` 功能的若干问题。
- 修复 JSONB 类型在不支持 avx2 的机型上导致 BE 宕机的问题。
- 修复动态分区的若干问题。
- 修复 TopN 查询优化的若干问题。
- 修复 Unique Key Merge-on-Write 表模型的若干问题。

8.6.10 致谢

有 58 贡献者参与到 1.2.5 的完善和发布中，感谢他们的辛劳付出：

@adonis0147

@airborne12

@AshinGau

@BePPPower

@BiteTheDDDDt

@caiconghui

@CalvinKirs

@cambyzju

@caoliang-web

@dataroaring

@Doris-Extras

@dujl

@dutyu

@fsilent

@Gabriel39

@gitccl

@gnehil

@GoGoWen

@gongzexin

@HappenLee

@herry2038

@jacktengg

@Jibing-Li

@kaka11chen

@Kikyou1997

@LemonLiTree

@liaoxin01

@LiBinfeng-01

@luwei16

@Moonm3n

@morningman

@mrhhsg

@Mryange

@nextdreamblue

@nsnhuang

@qidaye

@Shoothzj

@sohardforaname

@stalary

@starocean999

@SWJTU-ZhangLei

@wsjz

@xiaokang

@xinyiZzz

@yangzhg

@yiguolei

@yixiutt

@yujun777

@Yulei-Yang

@yuxuan-luo

@zclllyybb

@zddr

@zenoyang

@zhangstar333

@zhannngchen

@zxealous

@zy-kkk

@zzzzzzzs

8.6.11 Release 1.2.4

在 1.2.4 版本中，Doris 团队已经修复了自 1.2.3 版本发布以来近 150 个问题或性能改进项。同时，1.2.4 版本也作为 1.2.3 的迭代版本，具备更高的稳定性，建议所有用户升级到这个版本。

- 针对 Date/DatetimeV2 和 DecimalV3 类型，在 DESCRIBLE 和 SHOW CREATE TABLE 语句的结果中，将不再显示为 Date/DatetimeV2 或 DecimalV3，而直接显示为 Date/Datetime 或 Decimal。
- 这个改动用于兼容部分 BI 系统。如果想查看列的实际类型，可以通过 DESCRIBE ALL 语句查看。
- 查询 information_schema 库中的表时，默认不再返回 External Catalog 中的元信息。
- 这个改动避免了因 External Catalog 的连接问题导致的 information_schema 库不可查的问题，从而解决部分 BI 系统与 Doris 配合使用的问题。可以通过 FE 的配置项 infodb_support_ext_catalog控制，默认为 false，即不返回 External Catalog 中的元信息。

8.6.12 Improvement

8.6.12.0.1 JDBC Catalog

- 支持通过 JDBC Catalog 连接其他 Trino/Presto 集群

参考文档：https://doris.apache.org/zh-CN/docs/dev/lakehouse/multi-catalog/jdbc#trino

- JDBC Catalog 连接 Clickhouse 数据源支持 Array 类型映射

参考文档：https://doris.apache.org/zh-CN/docs/dev/lakehouse/multi-catalog/jdbc#clickhouse

8.6.12.0.2 Spark Load

- Spark Load 支持 Resource Manager HA 相关配置

参考 PR：https://github.com/apache/doris/pull/15000

8.6.13 Bug Fixes

- 修复 Hive Catalog 的若干连通性问题。
- 修复 Hudi Catalog 的若干问题。
- 优化 JDBC Catalog 的连接池，避免过多的连接。
- 修复通过 JDBC Catalog 从另一个 Doris 集群导入数据是会发生 OOM 的问题。
- 修复若干查询和导入的规划问题。
- 修复 Unique Key Merge-On-Write 表的若干问题。
- 修复若干 BDBJE 问题，解决某些情况下 FE 元数据异常的问题。
- 修复 `CREATE VIEW` 语句不支持 Table Valued Function 的问题。
- 修复若干内存统计的问题。
- 修复读取 Parquet/ORC 表的若干问题。
- 修复 DecimalV3 的若干问题。
- 修复 `SHOW QUERY/LOAD PROFILE` 的若干问题。

8.6.14 致谢

有 47 位贡献者参与到 1.2.4 的完善和发布中，感谢他们的辛劳付出：

@zy-kkk

@zhannngchen

@zhangstar333

@yixiutt

@yiguolei

@xinyiZzz

@xiaokang

@wsjz

@wangbo

@starocean999

@sohardforaname

@siriume

@pingchunzhang

@nextdreamblue

@mymeiyi

@mrhhsg

@morrySnow

@morningman

@luwei16

@luozenglin

@liujinhui1994

@liaoxin01

@kaka11chen

@jeffreys-cat

@jacktengg

@gavinchou

@dutyu

@dataroaring

@chenlinzhong

@caoliang-web

@cambyzju

@adonis0147

@Yulei-Yang

@Yukang-Lian

@SWJTU-ZhangLei

@Kikyou1997

@Jibing-Li

@JackDrogon

@HappenLee

@GoGoWen

@Gabriel39

@Doris-Extras

@CalvinKirs

@Cai-Yao

@ByteYue

@BiteTheDDDDt

@BePPPower

8.6.15 Release 1.2.3

在 1.2.3 版本中，Doris 团队已经修复了自 1.2.2 版本发布以来超过 200 个问题或性能改进项。同时，1.2.3 版本也作为 1.2.2 的迭代版本，具备更高的稳定性，建议所有用户升级到这个版本。

8.6.15.0.1 JDBC Catalog

- 支持通过 JDBC Catalog 连接到另一个 Doris 数据库。

目前 JDBC Catalog 连接 Doris 只支持用 5.x 版本的 JDBC jar 包。如果使用 8.x JDBC jar 包可能会出现列类型无法匹配问题。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/jdbc/#doris

- 支持通过参数 `only_specified_database` 来同步指定的数据库。
- 支持通过 `lower_case_table_names` 参数控制是否以小写形式同步表名，解决表名区分大小写的问题。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/jdbc

- 优化 JDBC Catalog 的读取性能。

8.6.15.0.2 Elasticsearch Catalog

- 支持 Array 类型映射。
- 支持通过 `like_push_down` 属性下推 like 表达式来控制 ES 集群的 CPU 开销。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/es

8.6.15.0.3 Hive Catalog

- 支持 Hive 表默认分区 `__Hive_default_partition__`。
- Hive Metastore 元数据自动同步支持压缩格式的通知事件。

8.6.15.0.4 动态分区优化

- 支持通过 storage_medium 参数来控制创建动态分区的默认存储介质。

参考文档：https://doris.apache.org/docs/dev/advanced/partition/dynamic-partition

8.6.15.0.5 优化 BE 的线程模型

- 优化 BE 的线程模型，以避免频繁创建和销毁线程所带来的稳定性问题。

8.6.16 Bug 修复

- 修复了部分 Unique Key 模型 Merge-on-Write 表的问题；
- 修复了部分 Compaction 相关问题；
- 修复了部分 Delete 语句导致的数据问题；
- 修复了部分 Query 执行问题；
- 修复了在某些操作系统上使用 JDBC Catalog 导致 BE 宕机的问题；
- 修复了部分 Multi-Catalog 的问题；
- 修复了部分内存统计和优化问题；
- 修复了部分 DecimalV3 和 date/datetimev2 的相关问题。
- 修复了部分导入过程中的稳定性问题；
- 修复了部分 Light Schema Change 的问题；
- 修复了使用 `datetime` 类型创建批处理分区的问题；
- 修复了 Broker Load 大数据量导入失败而导致的 FE 内存使用过高的问题；
- 修复了删除表后无法取消 Stream Load 的问题；
- 修复了某些情况下查询 `information_schema` 超时的问题；
- 修复了使用 `select outfile` 并发数据导出导致 BE 宕机的问题；
- 修复了事务性插入操作导致内存泄漏的问题；
- 修复了部分查询和导入 Profile 的问题，并支持通过 FE web ui 直接下载 Profile 文件；
- 修复了 BE Tablet GC 线程导致 IO 负载过高的问题；
- 修复了 Kafka Routine Load 中提交 Offset 不准确的问题。

8.6.17 Release 1.2.2

在 1.2.2 版本中，Doris 团队已经修复了自 1.2.1 版本发布以来超过 200 个问题或性能改进项。同时，1.2.2 版本也作为 1.2.1 的迭代版本，具备更高的稳定性，建议所有用户升级到这个版本。

8.6.17.0.1 数据湖分析

- 支持自动同步 Hive Metastore 元数据信息。 默认情况下外部数据源的元数据变更，如创建或删除表、加减列等操作不会同步给 Doris，用户需要使用 `REFRESH CATALOG` 命令手动刷新元数据。在 1.2.2 版本中支持自动刷新 Hive Metastore 元数据信息，通过让 FE 节点定时读取 HMS 的 notification event 来感知 Hive 表元数据的变更情况。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/

- 支持读取 Iceberg Snapshot 以及查询 Snapshot 历史。在执行 Iceberg 数据写入时，每一次写操作都会产生一个新的快照。默认情况下通过 Apache Doris 读取 Iceberg 表仅会读取最新版本的快照。在 1.2.2 版本中可以使用 `FOR TIME AS OF` 和 `FOR VERSION AS OF` 语句，根据快照 ID 或者快照产生的时间读取历史版本的数据，也可以使用 iceberg_meta 表函数查询指定表的快照信息。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/iceberg

- JDBC Catalog 支持 PostgreSQL、 Clickhouse、 Oracle、 SQLServer。
- JDBC Catalog 支持 insert into 操作。 在 Doris 中建立 JDBC Catalog 后，可以通过 insert into 语句直接写入数据，也可以将 Doris 执行完查询之后的结果写入 JDBC Catalog，或者是从一个 JDBC 外表将数据导入另一个 JDBC 外表。

参考文档：https://doris.apache.org/docs/dev/lakehouse/multi-catalog/jdbc/

8.6.17.0.2 自动分桶推算

支持通过 `DISTRIBUTED BY HASH(……)BUCKETS AUTO` 语句设置自动分桶，系统帮助用户设定以及伸缩不同分区的分桶数，使分桶数保持在一个相对合适的范围内。

参考文档：https://mp.weixin.qq.com/s/DSyZGJtjQZUYUsvfK0IcCg

8.6.17.0.3 新增函数

增加归类分析函数 `width_bucket` 。

参考文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-functions/width-bucket/#description

8.6.18 Behavior Changes

8.6.18.0.1 默认情况下禁用 BE 的 Page Cache

关闭此配置以优化内存使用并降低内存 OOM 的风险，但有可能增加一些小查询的查询延迟。如果您对查询延迟敏感，或者具有高并发小查询场景，可以配置 `disable_storage_page_cache=false` 以再次启用 Page Cache。

8.6.18.0.2 增加新 Session 变量 `group_by_and_having_use_alias_first`

用于控制 group by 和 having 语句是否优先使用列的别名，而非从 From 语句里寻找列的名字，默认为 false。

8.6.19 Improvement

8.6.19.0.1 Compaction 优化

- 支持 Vetical Compaction。在过去版本中，宽列场景 Compaction 往往会带来大量的内存开销。在 1.2.2 版本中，Vertical Compaction 采用了按列组的方式进行数据合并，单次合并只需要加载部分列的数据，能够极大减少合并过程中的内存占用。在实际测试中，Vertical compaction 使用内存仅为原有 compaction 算法的 1/10，同时 Compaction 速率提升 15%。

- 支持 Segment Compaction。在过去版本中，当用户大数据量高频导入时可能会遇到 -238 以及 -235 问题，Segment Compaction 允许在导入数据的同时进行数据的合并，以有效控制 Segment 文件的数量，提升高频导入的系统稳定性。

参考文档：https://doris.apache.org/docs/dev/advanced/best-practice/compaction

8.6.19.0.2 数据湖分析

- Hive Catalog 支持访问 Hive 1/2/3 版本。
- Hive Catalog 可以使用 Broker 访问数据存储在 JuiceFS 的 Hive。
- Iceberg Catalog 支持 Hive Metastore 和 Rest 作为元数据服务。
- ES Catalog 支持元数据字段 _id 列映射。

参考文档：https://doris.apache.org/zh-CN/docs/dev/lakehouse/multi-catalog/hive

- 优化 Iceberg V2 表有大量删除行诗时的读取性能。
- 支持读取 Schema Evolution 后 Iceberg 表。
- Parquet Reader 正确处理列名大小写。

8.6.19.0.3 其他

- 支持访问 Hadoop KMS 加密的 HDFS。
- 支持取消正在执行的导出任务。

参考文档：https://doris.apache.org/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Manipulation/CANCEL-EXPORT

- 将`explode_split` 函数执行效率优化 1 倍。
- 将 nullable 列的读取性能优化 3 倍。
- 优化 Memtracker 的部分问题，提高内存管理精度，优化内存应用。

8.6.20 BugFix

- 修复了使用 Doris-Flink-Connector 导入数据时的内存泄漏问题；#16430
- 修复了 BE 可能的线程调度问题，并减少了 BE 线程耗尽导致的 Fragment_sent_timeout。
- 修复了 datetimev2/decivalv3 的部分正确性和精度问题。
- 修复了 Light Schema Change 功能的各种已知问题。

- 修复了 bitmap 类型 Runtime Filter 的各种数据正确性问题。
- 修复了 1.2.1 版本中引入的 CSV 读取性能差的问题。
- 修复了 Spark Load 数据下载阶段导致的 BE OOM 问题。
- 修复了从 1.1.x 版升级到 1.2.x 版时可能出现的元数据兼容性问题。
- 修复了创建 JDBC Catalog 时的元数据问题。
- 修复了由于导入操作导致的 CPU 使用率高的问题。
- 修复了大量失败 Broker Load 作业导致的 FE OOM 问题。
- 修复了加载浮点类型时精度丢失的问题。
- 修复了 Stream Load 使用两阶段提交时出现的内存泄漏问题。

8.6.21 其他

添加指标以查看 BE 上的 Rowset 和 Segment 总数字 `doris_be_all_rowsets_num` 和 `doris_be_all_segments_num`

8.6.22 致谢

有 53 位贡献者参与到 1.2.2 版本的开发与完善中，感谢他们的付出，他们分别是：

@adonis0147

@AshinGau

@BePPPower

@BiteTheDDDDt

@ByteYue

@caiconghui

@cambyzju

@chenlinzhong

@DarvenDuan

@dataroaring

@Doris-Extras

@dutyu

@englefly

@freemandealer

@Gabriel39

@HappenLee

@Henry2SS

@htyoung

@isHuangXin

@JackDrogon

@jacktengg

@Jibing-Li

@kaka11chen

@Kikyou1997

@Lchangliang

@LemonLiTree

@liaoxin01

@liqing-coder

@luozenglin

@morningman

@morrySnow

@mrhhsg

@nextdreamblue

@qidaye

@qzsee

@spaces-X

@stalary

@starocean999

@weizuo93

@wsjz

@xiaokang

@xinyiZzz

@xy720

@yangzhg

@yiguolei

@yixiutt

@Yukang-Lian

@Yulei-Yang

@zclllyybb

@zddr

@zhangstar333

@zhannngchen

@zy-kkk

8.6.23 Release 1.2.1

在 1.2.1 版本中，Doris 团队已经修复了自 1.2.0 版本发布以来约 200 个问题或性能改进项。同时，1.2.1 版本也作为 1.2 的第一个迭代版本，具备更高的稳定性，建议所有用户升级到这个版本。

8.6.23.0.1 支持高精度小数 DecimalV3

支持精度更高和性能更好的 DecimalV3，相较于过去版本具有以下优势：

- 可表示范围更大，取值范围都进行了明显扩充，有效数字范围 [1,38]。
- 性能更高，根据不同精度，占用存储空间可自适应调整。
- 支持更完备的精度推演，对于不同的表达式，应用不同的精度推演规则对结果的精度进行推演。

DecimalV3

8.6.23.0.2 支持 Iceberg V2

支持 Iceberg V2 (仅支持 Position Delete，Equality Delete 会在后续版本支持)，可以通过 Multi-Catalog 功能访问 Iceberg V2 格式的表。

8.6.23.0.3 支持 OR 条件转 IN

支持将 where 条件表达式后的 or 条件转换成 in 条件，在部分场景中可以提升执行效率。#15437 #12872

8.6.23.0.4 优化 JSONB 类型的导入和查询性能

优化 JSONB 类型的导入和查询性能，在测试数据上约有 70% 的性能提升。#15219 #15219

8.6.23.0.5 Stream load 支持带引号的 CSV 数据

通过导入任务参数 `trim_double_quotes` 来控制，默认值为 false，为 true 时表示裁剪掉 CSV 文件每个字段最外层的双引号。#15241

8.6.23.0.6 Broker 支持腾讯云 CHDFS 和百度云 BOS、AFS

可以通过 Broker 访问存储在腾讯云 CHDFS 和百度智能云 BOS、AFS 上的数据。#15297 #15448

8.6.23.0.7 新增函数

新增函数 substring_index。#15373

8.6.24 问题修复

- 修复部分情况下，从 1.1.x 版本升级到 1.2.0 版本后，用户权限信息丢失的问题。#15144
- 修复使用 date/datetimev2 类型进行分区时，分区值错误的问题。#15094
- 修复部分已发布功能的 Bug，具体列表可参阅：PR List

8.6.25 升级注意事项

8.6.25.0.1 已知问题

- 请勿使用 JDK11 作为 BE 的运行时 JDK，会导致 BE Crash。
- 该版本对 csv 格式的读取性能有下降，会影响 csv 格式的导入和读取效率，我们会在下一个三位版本尽快修复

8.6.25.0.2 行为改变

- BE 配置项 high_priority_flush_thread_num_per_store 默认值由 1 改成 6，以提升 Routine Load 的写入效率。#14775
- FE 配置项 enable_new_load_scan_node 默认值改为 true，将使用新的 File Scan Node 执行导入任务，对用户无影响。#14808
- 删除 FE 配置项 enable_multi_catalog，默认开启 Multi-Catalog 功能。
- 默认强制开启向量化执行引擎。会话变量 enable_vectorized_engine 将不再生效，如需重新生效，需将 FE 配置项 disable_enable_vectorized_engine 设为 false，并重启 FE。#15213

8.6.26 致谢

有 45 位贡献者参与到 1.2.1 版本的开发与完善中，感谢他们的付出，他们分别是：

@adonis0147

@AshinGau

@BePPPower

@BiteTheDDDDt

@ByteYue

@caiconghui

@cambyzju

@chenlinzhong

@dataroaring

@Doris-Extras

@dutyu

@eldenmoon

@englefly

@freemandealer

@Gabriel39

@HappenLee

@Henry2SS

@hf200012

@jacktengg

@Jibing-Li

@Kikyou1997

@liaoxin01

@luozenglin

@morningman

@morrySnow

@mrhhsg

@nextdreamblue

@qidaye

@spaces-X

@starocean999

@wangshuo128

@weizuo93

@wsjz

@xiaokang

@xinyiZzz

@xutaoustc

@yangzhg

@yiguolei

@yixiutt

@Yulei-Yang

@yuxuan-luo

@zenoyang

@zhangstar333

@zhannngchen

@zhengshengjun

8.6.27 Release 1.2.0

亲爱的社区小伙伴们，再一次经历数月的等候后，我们很高兴地宣布，Apache Doris 于 2022 年 12 月 7 日迎来 1.2.0 Release 版本的正式发布！有近 118 位 Contributor 为 Apache Doris 提交了超 2400 项优化和修复，感谢每一位让 Apache Doris 更好的你！

自从社区正式确立 LTS 版本管理机制后，在 1.1.x 系列版本中不再合入大的功能，仅提供问题修复和稳定性改进，力求满足更多社区用户在稳定性方面的高要求。而在综合考虑版本迭代节奏和用户需求后，我们决定将众多新特性在 1.2 版本中发布，这无疑承载了众多社区用户和开发者的深切期盼，同时这也是一场厚积而薄发后的全面进化！

在 1.2 版本中，我们实现了全面的向量化、实现多场景查询性能 3-11 倍的提升，在 Unique Key 模型上实现了 Merge-on-Write 的数据更新模式、数据高频更新时查询性能提升达 3-6 倍，增加了 Multi-Catalog 多源数据目录、提供了无缝接入 Hive、ES、Hudi、Iceberg 等外部数据源的能力，引入了 Light Schema Change 轻量表结构变更、实现毫秒级的 Schema Change 操作并可以借助 Flink CDC 自动同步上游数据库的 DML 和 DDL 操作，以 JDBC 外部表替换了过去的 ODBC 外部表，支持了 Java UDF 和 Romote UDF 以及 Array 数组类型和 JSONB 类型，修复了诸多之前版本的性能和稳定性问题，推荐大家下载和使用！

GitHub 下载：https://github.com/apache/doris/releases

官网下载页：https://doris.apache.org/download

源码地址：https://github.com/apache/doris/releases/tag/1.2.0-rc04

8.6.27.0.1 下载说明：

由于 Apache 服务器文件大小限制，官网下载页的 1.2.0 版本的二进制程序分为三个包：

1. apache-doris-fe
2. apache-doris-be
3. apache-doris-java-udf-jar-with-dependencies

其中新增的 `apache-doris-java-udf-jar-with-dependencies` 包用于支持 1.2.0 版本中的 JDBC 外表和 JAVA UDF。下载后，需要将其中的 `java-udf-jar-with-dependencies.jar` 文件放到 `be/lib` 目录下，方可启动 BE，否则无法启动成功。

8.6.27.0.2 部署说明：

从历史版本升级到 1.2.0 版本，需完整更新 fe、 be 下的 bin 和 lib 目录。

其他升级注意事项，请完整阅读本发版通告最后一节“升级注意事项”以及安装部署文档 https://doris.apache.org/zh-CN/docs/dev/install/install-deploy 和集群升级文档 https://doris.apache.org/zh-CN/docs/dev/admin-manual/cluster-management/upgrade

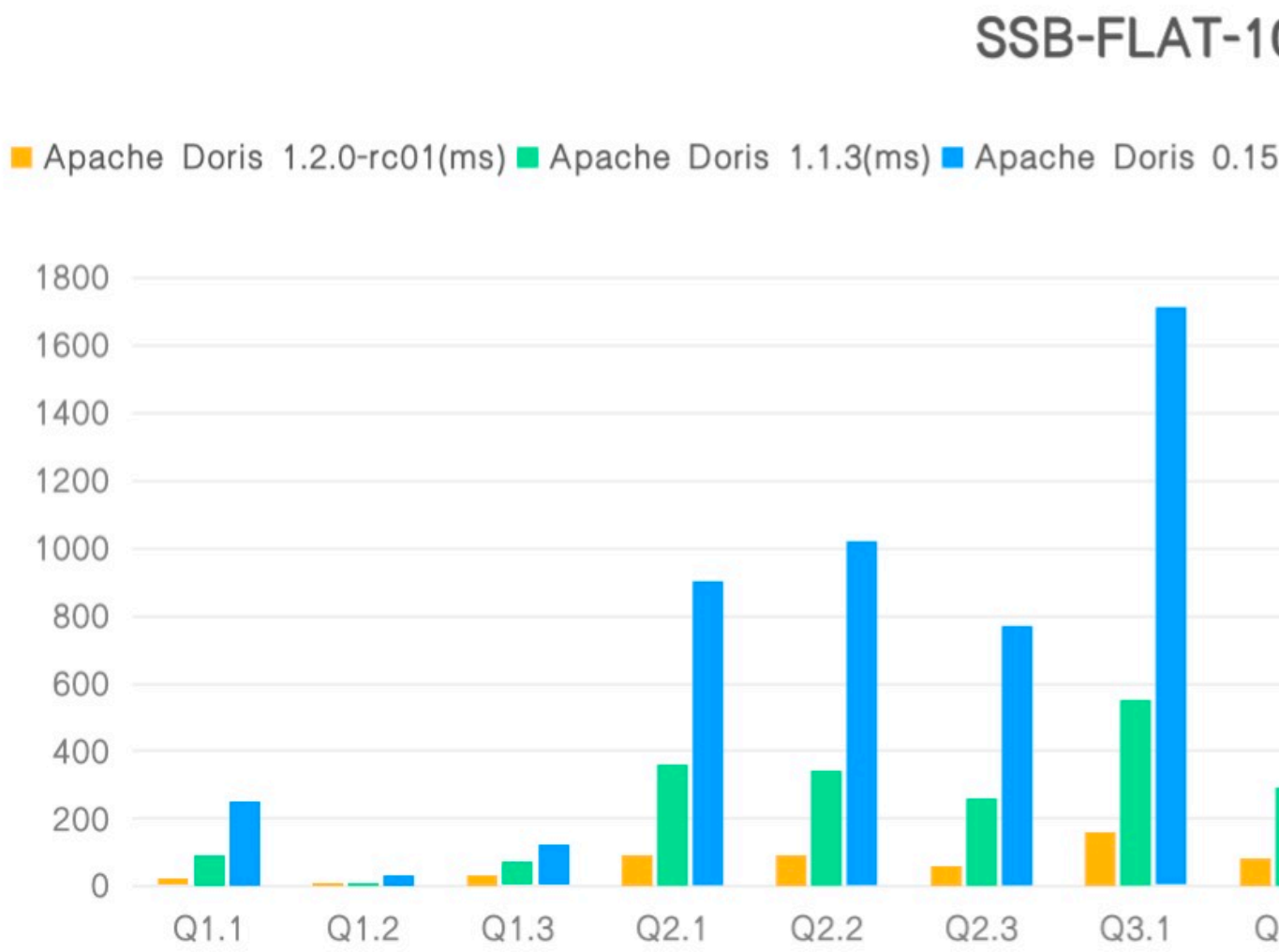


图 265: ssb_flat

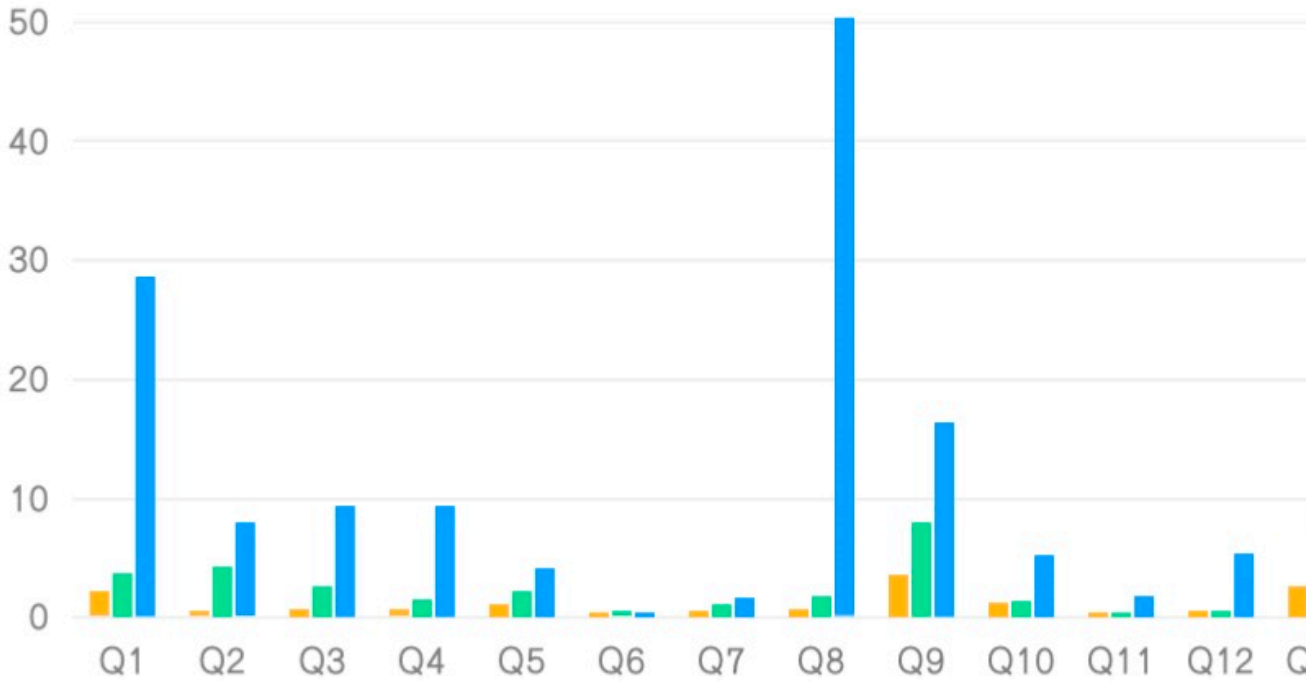


图 266: tpch

同时，我们将 1.2.0 版本的测试数据提交到了全球知名的数据库测试排行榜 ClickBench，在最新的排行榜中，Apache Doris 1.2.0 新版本取得了通用机型（ c6a.4xlarge, 500gb gp2 ）下查询性能 Cold Run 第二和 Hot Run 第三的醒目成绩，共有 8 个 SQL 刷新榜单最佳成绩、成为新的性能标杆。导入性能方面，1.2.0 新版本数据写入效率在同机型所有产品中位列第一，压缩前 70G 数据写入仅耗时 415s、单节点写入速度超过 170 MB/s，在实现极致查询性能的同时也保证了高效的写入效率！

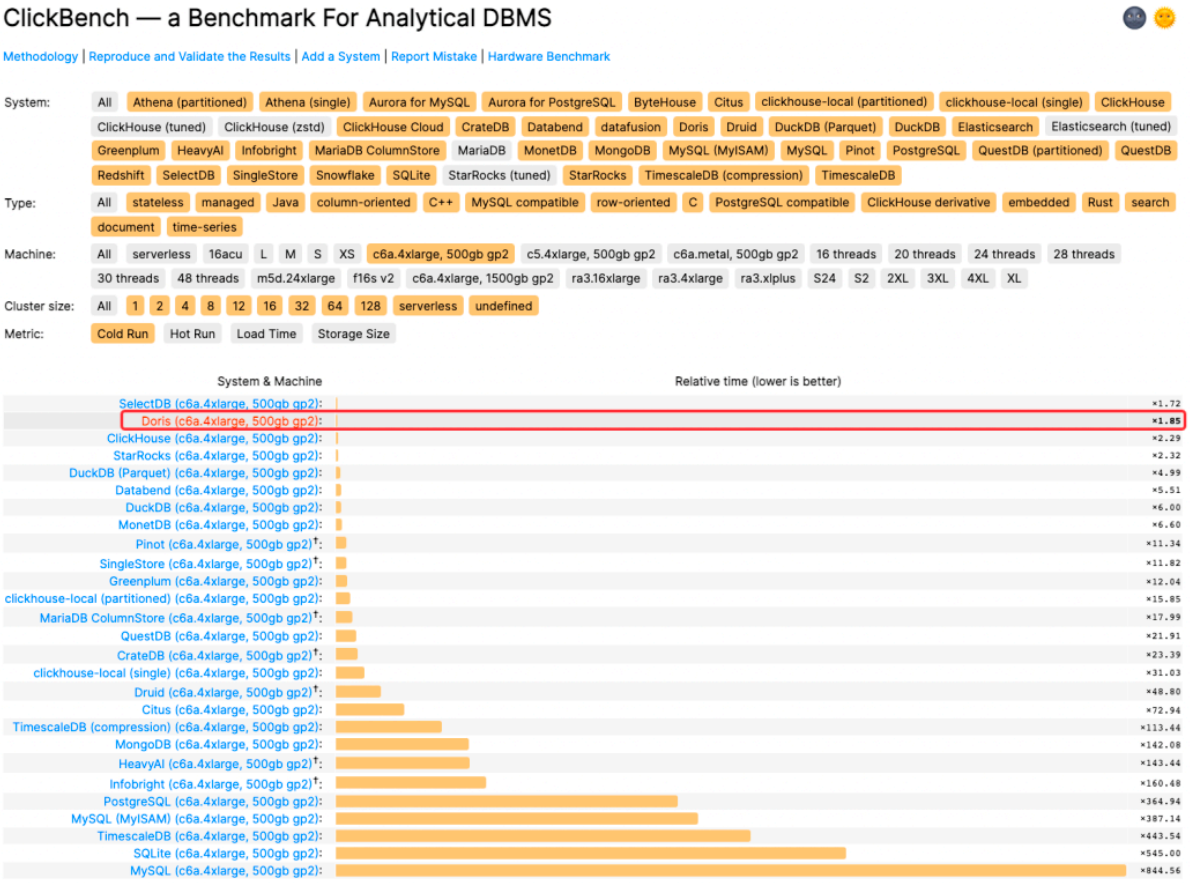


图 267: coldrun

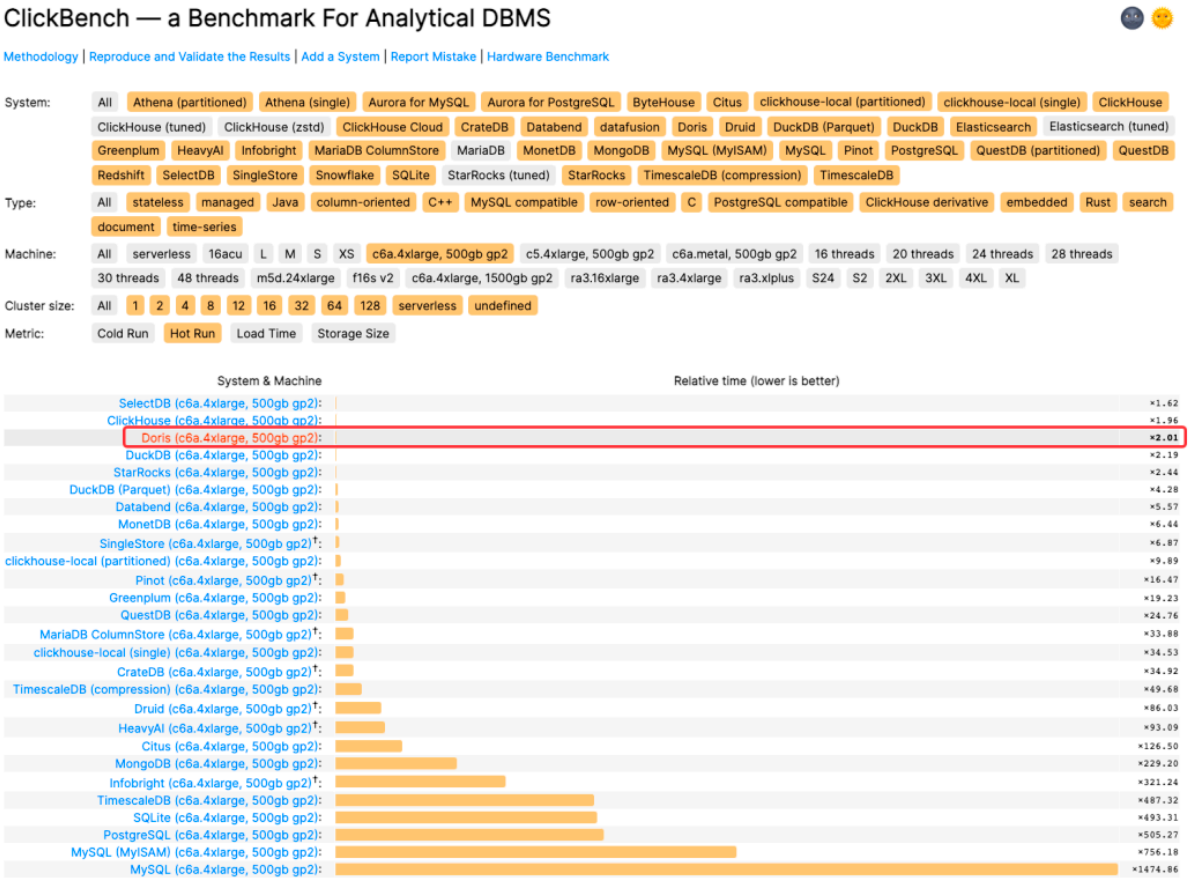


图 268: hotrun

8.6.28.0.2 2. 在 Unique Key 模型上实现了 Merge-on-Write 的数据更新模式

在过去版本中，Apache Doris 主要是通过 Unique Key 数据模型来实现数据实时更新的。但由于采用的是 Merge-on-Read 的实现方式，查询存在着效率瓶颈，有大量非必要的 CPU 计算资源消耗和 IO 开销，且可能将出现查询性能抖动等问题。

在 1.2.0 版本中，我们在原有的 Unique Key 数据模型上，增加了 Merge-on-Write 的数据更新模式。该模式在数据写入时即对需要删除或更新的数据进行标记，始终保证有效的主键只出现在一个文件中（即在写入的时候保证了主键的唯一性），不需要在读取的时候通过归并排序来对主键进行去重，这对于高频写入的场景来说，大大减少了查询执行时的额外消耗。此外还能够支持谓词下推，并能够很好利用 Doris 丰富的索引，在数据 IO 层面就能够进行充分的数据裁剪，大大减少数据的读取量和计算量，因此在很多场景的查询中都有非常明显的性能提升。

在比较有代表性的 SSB-Flat 数据集上，通过模拟多个持续导入场景，新版本的大部分查询取得了 3-6 倍的性能提升。

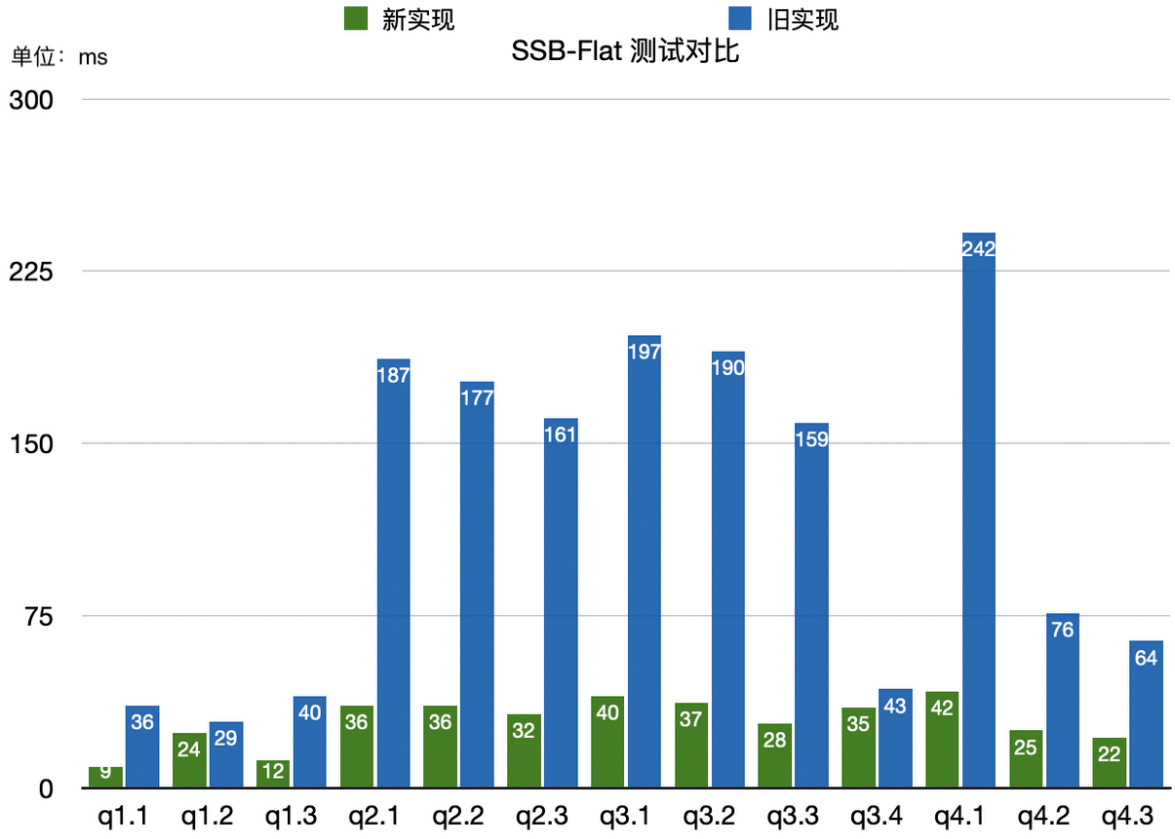


图 269: mergeonwrite_ssb

使用场景：所有对主键唯一性有需求，需要频繁进行实时 Upsert 更新的用户建议打开。

使用说明：作为新的 Feature 默认关闭，用户可以通过在建表时添加下面的 Property 来开启：

```
“enable_unique_key_merge_on_write” = “true”
```

另外新版本 Merge-on-Write 数据更新模式与旧版本 Merge-on-Read 实现方式存在差异，因此已经创建的 Unique Key 表无法直接通过 Alter Table 添加 Property 来支持，只能在新建表的时候指定。如果用户需要将旧表转换到新表，可以使用 `insert into new_table select * from old_table` 的方式来实现。

8.6.28.0.3 3. Multi Catalog 多源数据目录

Multi-Catalog 多源数据目录功能的目标在于能够帮助用户更方便对接外部数据目录，以增强 Apache Doris 的数据湖分析和联邦数据查询能力。

在过去版本中，当我们需要对接外部数据源时，只能在 Database 或 Table 层级对接。当外部数据目录 Schema 发生变化、或者外部数据目录的 Database 或 Table 非常多时，需要用户手工进行一一映射，维护量非常大。1.2.0 版本新增的多源数据目录功能为 Apache Doris 提供了快速接入外部数据源进行访问的能力，用户可以通过 CREATE CATALOG 命令连接到外部数据源，Doris 会自动映射外部数据源的库、表信息。之后，用户就可以像访问普通表一样，对这些外部数据源中的数据进行访问，避免了之前用户需要对每张表手动建立外表映射的复杂操作。

目前能支持以下数据源：

1. Hive Metastore：可以访问包括 Hive、Iceberg、Hudi 在内的数据表，也可对接兼容 Hive Metastore 的数据源，如阿里云的 DataLake Formation，同时支持 HDFS 和对象存储上的数据访问。
2. Elasticsearch：访问 ES 数据源。
3. JDBC：支持通过 JDBC 访问 MySQL 数据源。

注：相应的权限层级也会自动变更，详见 “升级注意事项” 部分

文档：https://doris.apache.org/zh-CN/docs/dev/lakehouse/multi-catalog

8.6.28.0.4 4. 轻量表结构变更 Light Schema Change

在过去版本中，Schema Change 是一项相对消耗比较大的工作，需要对数据文件进行修改，在集群规模和表数据量较大时执行效率会明显降低。同时由于是异步作业，当上游 Schema 发生变更时，需要停止数据同步任务并手动执行 Schema Change，增加开发和运维成本的同时还可能造成消费数据的积压。

在 1.2.0 新版本中，对数据表的加减列操作，不再需要同步更改数据文件，仅需在 FE 中更新元数据即可，从而实现毫秒级的 Schema Change 操作，且存在导入任务时效率的提升更为显著。与此同时，使得 Apache Doris 在面对上游数据表维度变化时，可以更加快速稳定实现表结构同步，保证系统的高效且平稳运转。如用户可以通过 Flink CDC，可实现上游数据库到 Doris 的 DML 和 DDL 同步，进一步提升了实时数仓数据处理和分析链路的时效性与便捷性。

性能对比	无导入任务		有导入任务
	加列	减列	加列
Hard Linked Schema Change	1310 ms	1438 ms	13 minutes
Light Schema Change	7 ms	3 ms	3 ms

图 270: lightschemachange_compare.png

使用说明：作为新的 Feature 默认关闭，用户可以通过在建表时添加下面的 Property 来开启：

```
"light_schema_change" = "true"
```

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Create/CREATE-TABLE

8.6.28.0.5 5. JDBC 外部表

在过去版本中，Apache Doris 提供了 ODBC 外部表的方式来访问 MySQL、Oracle、SQL Server、PostgreSQL 等数据源，但由于 ODBC 驱动版本问题可能造成系统的不稳定。相对于 ODBC，JDBC 接口更为统一且支持数据库众多，因此在 1.2.0 版本中我们实现了 JDBC 外部表以替换原有的 ODBC 外部表。在新版本中，用户可以通过 JDBC 连接支持 JDBC 协议的外部数据源，

当前已适配的数据源包括：

- MySQL
- PostgreSQL
- Oracle
- SQLServer
- ClickHouse

更多数据源的适配已经在规划之中，原则上任何支持 JDBC 协议访问的数据库均能通过 JDBC 外部表的方式来访问。而之前的 ODBC 外部表功能将会在后续的某个版本中移除，还请尽量切换到 JDBC 外表功能。

文档：https://doris.apache.org/zh-CN/docs/dev/lakehouse/multi-catalog/jdbc

8.6.28.0.6 6. JAVA UDF

在过去版本中，Apache Doris 提供了 C++ 语言的原生 UDF，便于用户通过自己编写自定义函数来满足特定场景的分析需求。但由于原生 UDF 与 Doris 代码耦合度高、当 UDF 出现错误时可能会影响集群稳定性，且只支持 C++ 语言，对于熟悉 Hive、Spark 等大数据技术栈的用户而言存在较高门槛，因此在 1.2.0 新版本我们增加了 Java 语言的自定义函数，支持通过 Java 编写 UDF/UDAF，方便用户在 Java 生态中使用。同时，通过堆外内存、Zero Copy 等技术，使得跨语言的数据访问效率大幅提升。

文档：https://doris.apache.org/zh-CN/docs/dev/ecosystem/udf/java-user-defined-function

示例：https://github.com/apache/doris/tree/master/samples/doris-demo

8.6.28.0.7 7. Remote UDF

远程 UDF 支持通过 RPC 的方式访问远程用户自定义函数服务，从而彻底消除用户编写 UDF 的语言限制，用户可以使用任意编程语言实现自定义函数，完成复杂的数据分析工作。

文档：https://doris.apache.org/zh-CN/docs/ecosystem/udf/remote-user-defined-function

示例：https://github.com/apache/doris/tree/master/samples/doris-demo

8.6.28.0.8 8. Array/JSONB 复合数据类型

- Array 类型

支持了数组类型，同时也支持多级嵌套的数组类型。在一些用户画像，标签等场景，可以利用 Array 类型更好的适配业务场景。同时在新版本中，我们也实现了大量数组相关的函数，以更好的支持该数据类型在实际场景中的应用。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Types/ARRAY

相关函数：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-functions/array-functions/array

- JSONB 类型

支持二进制的 JSON 数据类型 JSONB。该类型提供更紧凑的 JSONB 编码格式，同时提供在编码格式上的数据访问，相比于使用字符串存储的 JSON 数据，有数倍的性能提升。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Types/JSONB

相关函数：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-functions/json-functions/jsonb_parse

8.6.28.0.9 9. DateV2/DatatimeV2 新版日期/日期时间数据类型

支持 DateV2 日期类型和 DatetimeV2 日期时间类型，相较于原有的 Date 和 Datetime 效率更高且支持最多到微秒的时间精度，建议使用新版日期类型。

文档：https://doris.apache.org/zh-CN/docs/1.2/sql-manual/sql-reference/Data-Types/DATETIMEV2

https://doris.apache.org/zh-CN/docs/1.2/sql-manual/sql-reference/Data-Types/DATEV2

影响范围：1. 用户需要在建表时指定 DateV2 和 DatetimeV2，原有表的 Date 以及 Datetime 不受影响。2. Datev2 和 Datetimev2 在与原来的 Date 和 Datetime 做计算时（例如等值连接），原有类型会被 cast 成新类型做计算 3. Example 参考文档中说明

8.6.28.0.10 10. 全新内存管理框架

在 Apache Doris 1.2.0 版本中我们增加了全新的内存跟踪器（Memory Tracker），用以记录 Doris BE 进程内存使用，包括查询、导入、Compaction、Schema Change 等任务生命周期中使用的内存以及各项缓存。通过 Memory Tracker 实现了更加精细的内存监控和控制，大大减少了因内存超限导致的 OOM 问题，使系统稳定性进一步得到提升。

文档：https://doris.apache.org/zh-CN/docs/dev/admin-manual/maint-monitor/memory-management/memory-tracker

8.6.28.0.11 11. Table Valued Function 表函数

增加了 Table Valued Function（TVF，表函数），TVF 可以视作一张普通的表，可以出现在 SQL 中所有“表”可以出现的位置，让用户像访问关系表格式数据一样，读取或访问来自 HDFS 或 S3 上的文件内容，

例如使用 S3 TVF 实现对象存储上的数据导入：

```
insert into tbl select * from s3("s3://bucket/file.*", "ak" = "xx", "sk" = "xxx") where c1 > 2;
```

或者直接查询 HDFS 上的数据文件：

```
insert into tbl select * from hdfs("hdfs://bucket/file.*") where c1 > 2;
```

TVF 可以帮助用户充分利用 SQL 丰富的表达能力，灵活处理各类数据。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-functions/table-functions/s3

https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-functions/table-functions/hdfs

8.6.29 更多功能

8.6.29.0.1 1. 更便捷的分区创建方式

支持通过 FROM TO 命令创建一个时间范围内的多个分区。

文档搜索 “MULTI RANGE”：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Create/CREATE-TABLE

示例：

```
// 根据时间date 创建分区，支持多个批量逻辑和单独创建分区的混合使用

PARTITION BY RANGE(event_day)(
        FROM ("2000-11-14") TO ("2021-11-14") INTERVAL 1 YEAR,
        FROM ("2021-11-14") TO ("2022-11-14") INTERVAL 1 MONTH,
        FROM ("2022-11-14") TO ("2023-01-03") INTERVAL 1 WEEK,
        FROM ("2023-01-03") TO ("2023-01-14") INTERVAL 1 DAY,
        PARTITION p_20230114 VALUES [('2023-01-14'), ('2023-01-15'))
)
```

```
// 根据时间datetime 创建分区
PARTITION BY RANGE(event_time)(
        FROM ("2023-01-03 12") TO ("2023-01-14 22") INTERVAL 1 HOUR
)
```

8.6.29.0.2 2. 列重命名

对于开启了 Light Schema Change 的表，支持对列进行重命名。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Alter/ALTER-TABLE-RENAME

8.6.29.0.3 3. 更丰富权限管理

- 支持行级权限

可以通过 CREATE ROW POLICY 命令创建行级权限。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Create/CREATE-POLICY

- 支持指定密码强度、过期时间等。
- 支持在多次失败登录后锁定账户。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Account-Management-Statements/ALTER-USER

8.6.29.0.4 4. 导入相关

- CSV 导入支持带 header 的 CSV 文件。

在文档中搜索 `csv_with_names`：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Load/STREAM-LOAD/

- Stream Load 新增 `hidden_columns`，可以显式指定 delete flag 列和 sequence 列。

在文档中搜索 `hidden_columns`：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Load/STREAM-LOAD

- Spark Load 支持 Parquet 和 ORC 文件导入。
- 支持清理已完成的导入的 Label 文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Load/CLEAN-LABEL
- 支持通过状态批量取消导入作业文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Load/CANCEL-LOAD
- Broker Load 新增支持阿里云 OSS，腾讯 CHDFS 和华为云 OBS。

文档：https://doris.apache.org/zh-CN/docs/dev/advanced/broker

- 支持通过 hive-site.xml 文件配置访问 HDFS。

文档：https://doris.apache.org/zh-CN/docs/dev/admin-manual/config/config-dir

8.6.29.0.5 5. 支持通过 `SHOW CATALOG RECYCLE BIN` 功能查看回收站中的内容。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Show-Statements/SHOW-CATALOG-RECYCLE-BIN

8.6.29.0.6 6. 支持 `SELECT * EXCEPT` 语法。

文档：https://doris.apache.org/zh-CN/docs/dev/data-table/basic-usage

8.6.29.0.7 7. OUTFILE 支持 ORC 格式导出，并且支持多字节分隔符。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/OUTFILE

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/OUTFILE

8.6.29.0.8 8. 支持通过配置修改可保存的 Query Profile 的数量。

文档搜索 FE 配置项：`max_query_profile_num`

8.6.29.0.9 9. DELETE 语句支持 IN 谓词条件。并且支持分区裁剪。

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Manipulation-Statements/Manipulation/DELETE

8.6.29.0.10　10. 时间列的默认值支持使用 CURRENT_TIMESTAMP

文档中搜索“CURRENT_TIMESTAMP”：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Create/CREATE-TABLE

8.6.29.0.11　11. 添加两张系统表：backends、 rowsets

backends 是 Doris 中内置系统表，存放在 information_schema 数据库下，通过该系统表可以查看当前 Doris 集群中的 BE 节点信息。

rowsets 是 Doris 中内置系统表，存放在 information_schema 数据库下，通过该系统表可以查看 Doris 集群中各个 BE 节点当前 rowsets 情况。

文档：

https://doris.apache.org/zh-CN/docs/dev/admin-manual/system-table/backends

https://doris.apache.org/zh-CN/docs/dev/admin-manual/system-table/rowsets

8.6.29.0.12　12. 备份恢复

- Restore 作业支持 reserve_replica 参数，使得恢复后的表的副本数和备份时一致。
- Restore 作业支持 reserve_dynamic_partition_enable 参数，使得恢复后的表保持动态分区开启状态。

文　档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Backup-and-Restore/RESTORE

- 支持通过内置的 libhdfs 进行备份恢复操作，不再依赖 broker。

文　档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Backup-and-Restore/CREATE-REPOSITORY

8.6.29.0.13　13. 支持同机多磁盘之间的数据均衡

文档：

https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Database-Administration-Statements/ADMIN-REBALANCE-DISK

https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Database-Administration-Statements/ADMIN-CANCEL-REBALANCE-DISK

8.6.29.0.14　14. Routine Load 支持订阅 Kerberos 认证的 Kafka 服务。

文档中搜索 kerberos：https://doris.apache.org/zh-CN/docs/dev/data-operate/import/import-way/routine-load-manual

8.6.29.0.15　15. New built-in-function 新增内置函数

新增以下内置函数：

- cbrt

- sequence_match/sequence_count
- mask/mask_first_n/mask_last_n
- elt
- any/any_value
- group_bitmap_xor
- ntile
- nvl
- uuid
- initcap
- regexp_replace_one/regexp_extract_all
- multi_search_all_positions/multi_match_any
- domain/domain_without_www/protocol
- running_difference
- bitmap_hash64
- murmur_hash3_64
- to_monday
- not_null_or_empty
- window_funnel
- outer combine 以及所有 Array 函数

8.6.30 升级注意事项

8.6.30.0.1 FE 元数据版本变更【重要】

FE Meta Version 由 107 变更为 114，因此从 1.1.x 以及更早版本升级至 1.2.0 版本后，不可回滚到之前版本。升级过程中，建议通过灰度升级的方式，先升级部分节点并观察业务运行情况，以降低升级风险，若执行非法的回滚操作将可能导致数据丢失与损坏。

8.6.30.0.2 行为改变

- 权限层级变更。

  因为引入了 Catalog 层级，所以相应的用户权限层级也会自动变更。规则如下：

- GlobalPrivs 和 ResourcePrivs 保持不变
- 新增 CatalogPrivs 层级。
- 原 DatabasePrivs 层级增加 internal 前缀（表示 internal catalog 中的 db）
- 原 TablePrivs 层级增加 internal 前缀（表示 internal catalog 中的 tbl）
- GroupBy 和 Having 子句中，优先使用列名而不是别名进行匹配。
- 不再支持创建以“mv_”开头的列。“mv_”是物化视图中的保留关键词
- 移除了 order by 语句默认添加的 65535 行的 Limit 限制，并增加 Session 变量 `default_order_by_limit` 可以自定配置这个限制。
- “Create Table As Select”生成的表，所有字符串列统一使用 String 类型，不再区分 varchar/char/string

- audit log 中，移除 db 和 user 名称前的 `default_cluster` 字样。
- audit log 中增加 sql digest 字段
- union 子句总 order by 逻辑变动。新版本中，order by 子句将在 union 执行完成后执行，除非通过括号进行显式的关联。
- 进行 decommission 操作时，会忽略回收站中的 tablet，确保 decomission 能够完成。
- Decimal 的返回结果将按照原始列中声明的精度进行显示，或者按照显式指定的 cast 函数中的精度进行展示。
- 列名的长度限制由 64 变更为 256
- FE 配置项变动
- 默认开启 `enable_vectorized_load` 参数。
- 增大了 `create_table_timeout` 值。建表操作的默认超时时间将增大。
- 修改 `stream_load_default_timeout_second` 默认值为 3 天。
- 修改`alter_table_timeout_second` 的默认值为一个月。
- 增加参数 `max_replica_count_when_schema_change` 用于限制 alter 作业中涉及的 replica 数量，默认为 100000。
- 添加 `disable_iceberg_hudi_table`。默认禁用了 iceberg 和 hudi 外表，推荐使用 multi catalog 功能。
- BE 配置项变动
- 移除了 `disable_stream_load_2pc` 参数。2PC 的 stream load 可直接使用。
- 修改`tablet_rowset_stale_sweep_time_sec` ，从 1800 秒修改为 300 秒。
- Session 变量变动
- 修改变量 `enable_insert_strict` 默认为 true。这会导致一些之前可以执行，但是插入了非法值的 insert 操作，不再能够执行。
- 修改变量 `enable_local_exchange` 默认为 true
- 默认通过 lz4 压缩进行数据传输，通过变量 `fragment_transmission_compression_codec` 控制
- 增加 `skip_storage_engine_merge` 变量，用于调试 unique 或 agg 模型的数据文档：https://doris.apache.org/zh-CN/docs/dev/advanced/variables
- BE 启动脚本会通过 `/proc/sys/vm/max_map_count` 检查数值是否大于 200W，否则启动失败。
- 移除了 mini load 接口

8.6.30.0.3 升级过程中需注意

1. 升级准备

- 需替换：lib, bin 目录（start/stop 脚本均有修改）
- BE 也需要配置 JAVA_HOME，已支持 JDBC Table 和 Java UDF。
- fe.conf 中默认 JVM Xmx 参数修改为 8GB。

2. 升级过程中可能的错误

- repeat 函数不可使用并报错：`vectorized repeat function cannot be executed`，可以在升级前先关闭向量化执行引擎。
- schema change 失败并报错：`desc_tbl is not set. Maybe the FE version is not equal to the BE`
- 向量化 hash join 不可使用并报错。`vectorized hash join cannot be executed`。可以在升级前先关闭向量化执行引擎。

以上错误在完全升级后会恢复正常。

8.6.30.0.4 性能影响

- 默认使用 JeMalloc 作为新版本 BE 的内存分配器，替换 TcMalloc。

JeMalloc 相比 TcMalloc 使用的内存更少、高并发场景性能更高，但在内存充足的性能测试时，TcMalloc 比 JeMalloc 性能高 5%-10%，详细测试见：https://github.com/apache/doris/pull/12496

- tablet sink 中的 batch size 修改为至少 8K。
- 默认关闭 Page Cache 和减少 Chunk Allocator 预留内存大小

Page Cache 和 Chunk Allocator 分别缓存用户数据块和内存预分配，这两个功能会占用一定比例的内存并且不会释放。由于这部分内存占用无法灵活调配，导致在某些场景下可能因这部分内存占用而导致其他任务内存不足，影响系统稳定性和可用性，因此新版本中默认关闭了这两个功能。

但在某些延迟敏感的报表场景下，关闭该功能可能会导致查询延迟增加。如用户担心升级后该功能对业务造成影响，可以通过在 be.conf 中增加以下参数以保持和之前版本行为一致。

```
disable_storage_page_cache=false
chunk_reserved_bytes_limit=10%
```

8.6.30.0.5 API 变化

- BE 的 http api 错误返回信息，由 `{"status": "Fail", "msg": "xxx"}` 变更为更具体的 `{"status": "Not found", "msg": "Tablet not found. tablet_id=1202"}`
- `SHOW CREATE TABLE` 中，comment 的内容由双引号包裹变为单引号包裹

- 支持普通用户通过 http 命令获取 query profile。

文档：https://doris.apache.org/zh-CN/docs/dev/admin-manual/http-actions/fe/manager/query-profile-action

- 优化了 sequence 列的指定方式，可以直接指定列名。

文档：https://doris.apache.org/zh-CN/docs/dev/data-operate/update-delete/sequence-column-manual

- show backends 和 show tablets 返回结果中，增加远端存储的空间使用情况 (#11450)
- 移除了 Num-Based Compaction 相关代码 (#13409)
- 重构了 BE 的错误码机制，部分返回的错误信息会发生变化 (#8855)

8.6.31 其他

- 支持 Docker 官方镜像。
- 支持在 MacOS(x86/M1) 和 ubuntu-22.04 上编译 Doris
- 支持进行 image 文件的校验。

文档搜索 “–image”：https://doris.apache.org/zh-CN/docs/dev/admin-manual/maint-monitor/metadata-operation - 脚本相关 - FE、BE 的 stop 脚本支持通过 --grace 参数退出 FE、BE（使用 kill -15 信号代替 kill -9）- FE start 脚本支持通过 –version 查看当前 FE 版本 (#11563) - 支持通过 ADMIN COPY TABLET 命令获取某个 tablet 的数据和相关建表语句，用于本地问题调试

文档：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Database-Administration-Statements/ADMIN-COPY-TABLET

- 支持通过 http api，获取一个 SQL 语句相关的建表语句，用于本地问题复现

文档：https://doris.apache.org/zh-CN/docs/dev/admin-manual/http-actions/fe/query-schema-action

- 支持建表时关闭这个表的 compaction 功能，用于测试

文档中搜索 “disble_auto_compaction”：https://doris.apache.org/zh-CN/docs/dev/sql-manual/sql-reference/Data-Definition-Statements/Create/CREATE-TABLE

8.6.32 致谢

Apache Doris 1.2.0 版本的发布离不开所有社区用户的支持，在此向所有参与版本设计、开发、测试、讨论的社区贡献者们表示感谢，他们分别是（首字母排序）：

```
@924060929
@a19920714liou
@adonis0147
@Aiden-Dong
@aiwenmo
@AshinGau
```

@b19mud
@BePPPower
@BiteTheDDDDt
@bridgeDream
@ByteYue
@caiconghui
@CalvinKirs
@cambyzju
@caoliang-web
@carlvinhust2012
@catpineapple
@ccoffline
@chenlinzhong
@chovy-3012
@coderjiang
@cxzl25
@dataalive
@dataroaring
@dependabot
@dinggege1024
@DongLiang-0
@Doris-Extras
@eldenmoon
@EmmyMiao87
@englefly
@FreeOnePlus
@Gabriel39
@gaodayue
@geniusjoe
@gj-zhang
@gnehil
@GoGoWen
@HappenLee
@hello-stephen
@Henry2SS
@hf200012
@huyuanfeng2018
@jacktengg
@jackwener
@jeffreys-cat
@Jibing-Li
@JNSimba
@Kikyou1997
@Lchangliang
@LemonLiTree

@lexoning
@liaoxin01
@lide-reed
@link3280
@liutang123
@liuyaolin
@LOVEGISER
@lsy3993
@luozenglin
@luzhijing
@madongz
@morningman
@morningman-cmy
@morrySnow
@mrhhsg
@Myasuka
@myfjdthink
@nextdreamblue
@pan3793
@pangzhili
@pengxiangyu
@platoneko
@qidaye
@qzsee
@SaintBacchus
@SeekingYang
@smallhibiscus
@sohardforaname
@song7788q
@spaces-X
@ssusieee
@stalary
@starocean999
@SWJTU-ZhangLei
@TaoZex
@timelxy
@Wahno
@wangbo
@wangshuo128
@wangyf0555
@weizhengte
@weizuo93
@wsjz
@wunan1210
@xhmz

@xiaokang
@xiaokangguo
@xinyiZzz
@xy720
@yangzhg
@Yankee24
@yeyudefeng
@yiguolei
@yinzhijian
@yixiutt
@yuanyuan8983
@zbtzbtzbt
@zenoyang
@zhangboya1
@zhangstar333
@zhannngchen
@ZHbamboo
@zhengshiJ
@zhenhb
@zhqu1148980644
@zuochunwei
@zy-kkk

8.7 v1.1

8.7.1 Release 1.1.5

在 1.1.5 版本中，Doris 团队已经修复了自 1.1.4 版本发布以来约 36 个问题或性能改进项。同时，1.1.5 版本也是作为 1.1 LTS 版本的错误修复版本，建议所有用户升级到这个版本。

当别名与原始列名相同时，例如 “select year(birthday) as birthday”，在 group by、 order by、 having 子句中使用别名时将与 MySQL 中保持一致，Group by 和 having 将首先使用原始列，order by 将首先使用别名。这里可能会对用户带来疑惑，因此建议最好不要使用与原始列名相同的别名。

8.7.2 Features

支持 Hash 函数 murmur_hash3_64。 #14636

8.7.3 Improvements

为日期函数 convert_tz 添加时区缓存以提高性能。 #14616

当调用 show 子句时，按 tablename 对结果进行排序。 #14492

8.7.4 Bug Fix

修复 if 语句中带有常量时导致 BE 可能 Coredump 的问题。#14858

修复 ColumnVector::insert_date_column 可能崩溃的问题 #14839

更新 high_priority_flush_thread_num_per_store 默认值为 6，将提高负载性能。#14775

优化 quick compaction core。#14731

修复分区列非 duplicate key 时 Spark Load 抛出 IndexOutOfBounds 错误的问题。#14661

修正 VCollectorIterator 中的内存泄漏问题。#14549

修复了存在 Sequence 列时可能存在的建表问题。#14511

使用 avg rowset 来计算批量大小，而不是使用 total_bytes，因为它要花费大量的 Cpu。#14273

修复了 right outer join 可能导致 core 的问题。#14821

优化了 TCMalloc gc 的策略。#14777 #14738 #14374

8.7.5 Release 1.1.4

作为 1.1 LTS（Long-term Support，长周期支持）版本基础之上的 Bugfix 版本，在 Apache Doris 1.1.4 版本中，Doris 团队修复了自 1.1.3 版本以来的约 60 个 Issue 或性能优化项。改进了 Spark Load 的使用体验，优化了诸多内存以及 BE 异常宕机的问题，系统稳定性和性能得以进一步加强，推荐所有用户下载和使用。

- Broker Load 支持华为云 OBS 对象存储。#13523
- Spark Load 支持 Parquet 和 Orc 文件。#13438

8.7.6 优化改进

- 禁用 Metric Hook 中的互斥量，其将影响数据导入过程中的查询性能。#10941

8.7.7 Bug 修复

- 修复了当 Spark Load 加载文件时 Where 条件不生效的问题。#13804
- 修复了 If 函数存在 Nullable 列时开启向量化返回错误结果的问题。#13779
- 修复了在使用 Anti Join 和其他 Join 谓词时产生错误结果的问题。#13743
- 修复了当调用函数 concat(ifnull) 时 BE 宕机的问题。#13693
- 修复了 group by 语句中存在函数时 planner 错误的问题。#13613
- 修复了 lateral view 语句不能正确识别表名和列名的问题。#13600
- 修复了使用物化视图和表别名时出现未知列的问题。#13605
- 修复了 JSONReader 无法释放值和解析 allocator 内存的问题。#13513

- 修复了当 enable_vectorized_alter_table 为 true 时允许使用 to_bitmap() 对负值列创建物化视图的问题。#13448
- 修复了函数 from_date_format_str 中微秒数丢失的问题。#13446
- 修复了排序 exprs 的 nullability 属性在使用子 smap 信息进行替换后可能不正确的问题。#13328
- 修复了 case when 有 1000 个条件时出现 Core 的问题。#13315
- 修复了 Stream Load 导入数据时最后一行数据丢失的问题。#13066
- 恢复表或分区的副本数与备份前相同。#11942

8.7.8 Release 1.1.3

作为 1.1.2 LTS（Long-term Support，长周期支持）版本基础之上的 Bugfix 版本，在 Apache Doris 1.1.3 版本中，有超过 80 个 Issue 或性能优化项被合入，优化了在导入或查询过程中的内存控制，修复了许多导致 BE Core 以及产生错误查询结果的问题，系统稳定性和性能得以进一步加强，推荐所有用户下载和使用。

- 在 ODBC 表中支持 SQLServer 和 PostgreSQL 的转义标识符。
- 支持使用 Parquet 作为导出文件格式。

8.7.9 优化改进

- 优化了 Flush 策略以及避免过多 Segment 小文件。#12706 #12716
- 重构 Runtime Filter 以减少初始准备时间。#13127
- 修复了若干个在查询或导入过程中的内存控制问题。#12682 #12688 #12708 #12776 #12782 #12791 #12794 #12820 #12932 #12954 #12951

8.7.10 Bug 修复

- 修复了 largeint 类型在 Compaction 过程中导致 Core 的问题。#10094
- 修复了 Grouping set 导致 BE Core 或者返回错误结果的问题。#12313
- 修复了使用 orthogonal_bitmap_union_count 函数时执行计划 PREAGGREGATION 显示错误的问题。#12581
- 修复了 Level1Iterator 未被释放导致的内存泄漏问题。#12592
- 修复了当 2 BE 且存在 Colocation 表时通过 Decommission 下线节点失败的问题。#12644
- 修复了 TBrokerOpenReaderResponse 过大时导致堆栈缓冲区溢出而导致的 BE Core 问题。#12658
- 修复了出现 -238 错误时 BE 节点可能 OOM 的问题。#12666
- 修复了 LEAD() 函数错误子表达式的问题。#12587
- 修复了行存代码中相关查询失败的问题。#12712
- 修复了 curdate()/current_date() 函数产生错误结果的问题。#12720

- 修复了 lateral View explode_split 函数出现错误结果的问题。#13643
- 修复了两张相同表中 Bucket Shuffle Join 计划错误的问题。#12930
- 修复了更新或导入过程中 Tablet 版本可能错误的问题。#13070
- 修复了在加密函数下使用 Broker 导入数据时 BE 可能发生 Core 的问题。#13009

8.7.11 升级说明

默认情况下禁用 PageCache 和 ChunkAllocator 以减少内存使用，用户可以通过修改配置项 `disable_storage_page_cache` 和 `chunk_reserved_bytes_limit` 来重新启用。

Storage Page Cache 和 Chunk Allocator 分别缓存用户数据块和内存预分配。

这两个功能会占用一定比例的内存，并且不会释放。这部分内存占用无法灵活调配，导致在某些场景下，因这部分内存占用而导致其他任务内存不足，影响系统稳定性和可用性。因此我们在 1.1.3 版本中默认关闭了这两个功能。

但在某些延迟敏感的报表场景下，关闭该功能可能会导致查询延迟增加。如用户担心升级后该功能对业务造成影响，可以通过在 be.conf 中增加以下参数以保持和之前版本行为一致。

```
disable_storage_page_cache=false
chunk_reserved_bytes_limit=10%
```

- `disable_storage_page_cache`：是否关闭 Storage Page Cache。1.1.2（含）之前的版本，默认是 false，即打开。1.1.3 版本默认为 true，即关闭。
- `chunk_reserved_bytes_limit`：Chunk allocator 预留内存大小。1.1.2（含）之前的版本，默认是整体内存的 10%。1.1.3 版本默认为 209715200（200MB）。

8.7.12 Release 1.1.2

在 Apache Doris 1.1.2 版本中，我们引入了新的 Memtracker、极大程度上避免 OOM 类问题的发生，提升了向量化执行引擎在多数查询场景的性能表现，修复了诸多导致 BE 和 FE 发生异常的问题，优化了在湖仓联邦查询场景的部分体验问题并提升访问外部数据的性能。

相较于 1.1.1 版本，在 1.1.2 版本中有超过 170 个 Issue 和性能优化项被合入，系统稳定性和性能都得到进一步加强。与此同时，1.1.2 版本还将作为 Apache Doris 首个 LTS（Long-term Support）长周期支持版本，后续长期维护和支持，推荐所有用户下载和升级。

8.7.12.0.1 MemTracker

MemTracker 是一个用于分析内存使用情况的统计工具，在 1.1.1 版本中我们引入了简易版 Memtracker 用以控制 BE 侧内存。在 1.1.2 版本中，我们引入了新的 MemTracker，在向量化执行引擎和非向量化执行引擎中都更为准确。

8.7.12.0.2 增加展示和取消正在执行 Query 的 API

`GET /rest/v2/manager/query/current_queries`

`GET /rest/v2/manager/query/kill/{query_id}`

具体使用参考文档 Query Profile Action

8.7.12.0.3 支持读写 Emoji 表情通过 ODBC 外表

8.7.13 优化改进

8.7.13.0.1 数据湖相关改进

- 扫描 HDFS ORC 文件时性能提升约 300%。#11501
- 查询 Iceberg 表支持 HDFS 的 HA 模式。
- 支持查询由 Apache Tez 创建的 Hive 数据
- 添加阿里云 OSS 作为 Hive 外部支持

8.7.13.0.2 在 Spark Load 中增加对 String 字符串类型和 Text 文本类型的支持

8.7.13.0.3 在非向量化引擎支持复用 Block，在某些场景中有 50% 性能提升。#11392

8.7.13.0.4 提升 Like 和正则表达式的性能

8.7.13.0.5 禁用 TCMalloc 的 aggressive_memory_decommit。

在查询或导入时将会有 40% 性能提升，也可以在配置文件中通过 `tc_enable_aggressive_memory_decommit`来修改

8.7.14 Bug Fix

8.7.14.0.1 修复部分可能导致 FE 失败或者数据损坏的问题

- 在 HA 环境中，BDBJE 将保留尽可能多的文件，通过增加配置 `bdbje_reserved_disk_bytes`以避免产生太多的 BDBJE 文件，BDBJE 日志只有在接近磁盘限制时才会删除。
- 修复了 BDBJE 中的重要错误，该错误将导致 FE 副本无法正确启动或数据损坏。

8.7.14.0.2 修复 FE 在查询过程中会在 waitFor_rpc 上 Hang 住以及 BE 在高并发情况下会 Hang 住的问题。

#12459 #12458 #12392

8.7.14.0.3 修复向量化执行引擎查询时得到错误结果的问题。

#11754 #11694

8.7.14.0.4 修复许多 Planner 导致 BE Core 或者处于不正常状态的问题。

#12080 #12075 #12040 #12003 #12007 #11971 #11933 #11861 #11859 #11855 #11837 #11834 #11821 #11782 #11723 #11569

8.7.15 Release 1.1.1

8.7.15.1 新增功能

8.7.15.1.1 向量化执行引擎支持 ODBC Sink。

在 1.1.0 版本的向量化执行引擎中 ODBC Sink 是不支持的，而这一功能在之前版本的行存引擎是支持的，因此在 1.1.1 版本中我们重新完善了这一功能。

8.7.15.1.2 增加简易版 MemTracker

MemTracker 是一个用于分析内存使用情况的统计工具，在 1.1.0 版本的向量化执行引擎中，由于 BE 侧没有 MemTracker，可能出现因内存失控导致的 OOM 问题。在 1.1.1 版本中，BE 侧增加了一个简易版 MemTracker，可以帮助控制内存，并在内存超出时取消查询。

完整版 MemTracker 将在 1.1.2 版本中正式发布。

8.7.15.2 改进

8.7.15.2.1 支持在 Page Cache 中缓存解压后数据。

在 Page Cache 中有些数据是用 bitshuffle 编码方式压缩的，在查询过程中需要花费大量的时间来解压。在 1.1.1 版本中，Doris 将缓存解压由 bitshuffle 编码的数据以加速查询，我们发现在 ssb-flat 的一些查询中，可以减少 30% 的延时。

8.7.15.3 Bug 修复

8.7.15.3.1 修复无法从 1.0 版本进行滚动升级的问题。

这个问题是在 1.1.0 版本中出现的，当升级 BE 而不升级 FE 时，可能会导致 BE Core。

如果你遇到这个问题，你可以尝试用 #10833 来修复它。

8.7.15.3.2 修复某些查询不能回退到非向量化引擎的问题，并导致 BE Core。

目前，向量化执行引擎不能处理所有的 SQL 查询，一些查询（如 left outer join）将使用非向量化引擎来运行。但部分场景在 1.1.0 版本中未被覆盖到，这可能导致 BE 挂掉。

8.7.15.3.3 修复 Compaction 不能正常工作导致的 -235 错误。

在 Unique Key 模型中，当一个 Rowset 有多个 Segment 时，在做 Compaction 过程中由于没有正确的统计行数，会导致 Compaction 失败并且产生 Tablet 版本过多而导致的 -235 错误。

8.7.15.3.4 修复查询过程中出现的部分 Segment fault。

#10961 #10954 #10962

感谢所有参与贡献 1.1.1 版本的开发者：

```
@jacktengg
@mrhhsg
@xinyiZzz
@yixiutt
@starocean999
@morrySnow
@morningman
@HappenLee
```

8.7.16 Release 1.1.0

在 1.1 版本中，我们实现了计算层和存储层的全面向量化、正式将向量化执行引擎作为稳定功能进行全面启用，所有查询默认通过向量化执行引擎来执行，性能较之前版本有 3-5 倍的巨大提升；增加了直接访问 Apache Iceberg 外部表的能力，支持对 Doris 和 Iceberg 中的数据进行联邦查询，扩展了 Apache Doris 在数据湖上的分析能力；在原有的 LZ4 基础上增加了 ZSTD 压缩算法，进一步提升了数据压缩率；修复了诸多之前版本存在的性能与稳定性问题，使系统稳定性得到大幅提升。欢迎大家下载使用。

8.7.16.1 升级说明

8.7.16.1.1 向量化执行引擎默认开启

在 Apache Doris 1.0 版本中，我们引入了向量化执行引擎作为实验性功能。用户需要在执行 SQL 查询手工开启，通过 `set batch_size = 4096` 和 `set enable_vectorized_engine = true`配置 session 变量来开启向量化执行引擎。

在 1.1 版本中，我们正式将向量化执行引擎作为稳定功能进行了全面启用，session 变量`enable_vectorized_engine` 默认设置为 true，无需用户手工开启，所有查询默认通过向量化执行引擎来执行。

8.7.16.1.2 BE 二进制文件更名

BE 二进制文件从原有的 palo_be 更名为 doris_be，如果您以前依赖进程名称进行集群管理和其他操作，请注意修改相关脚本。

8.7.16.1.3 Segment 存储格式升级

Apache Doris 早期版本的存储格式为 Segment V1，在 0.12 版本中我们实现了新的存储格式 Segment V2，引入了 Bitmap 索引、内存表、Page Cache、字典压缩以及延迟物化等诸多特性。从 0.13 版本开始，新建表的默认存储格式为 Segment V2，与此同时也保留了对 Segment V1 格式的兼容。

为了保证代码结构的可维护性、降低冗余历史代码带来的额外学习及开发成本，我们决定从下一个版本起不再支持 Segment v1 存储格式，预计在 Apache Doris 1.2 版本中将删除这部分代码。

8.7.16.1.4 正常升级

正常升级操作请按照官网上的集群升级文档进行滚动升级即可。

https://doris.apache.org/zh-CN/docs/admin-manual/cluster-management/upgrade

8.7.16.2 重要功能

8.7.16.2.1 支持数据随机分布 [实验性功能] [#8259](https://github.com/apache/doris/pull/8259) #8041

在某些场景中（例如日志分析类场景），用户可能无法找到一个合适的分桶键来避免数据倾斜，因此需要由系统提供额外的分布方式来解决数据倾斜的问题。

因此通过在建表时可以不指定具体分桶键，选择使用随机分布对数据进行分桶`DISTRIBUTED BY random BUCKETS number`，数据导入时将会随机写入单个 Tablet，以减少加载过程中的数据扇出，并减少资源开销、提升系统稳定性。

8.7.16.2.2 支持创建 Iceberg 外部表 [实验性功能] [#7391](https://github.com/apache/doris/pull/7391) #7981 #8179

Iceberg 外部表为 Apache Doris 提供了直接访问存储在 Iceberg 数据的能力。通过 Iceberg 外部表可以实现对本地存储和 Iceberg 存储的数据进行联邦查询，省去繁琐的数据加载工作、简化数据分析的系统架构，并进行更复杂的分析操作。

在 1.1 版本中，Apache Doris 支持了创建 Iceberg 外部表并查询数据，并支持通过 REFRESH 命令实现 Iceberg 数据库中所有表 Schema 的自动同步。

8.7.16.2.3 增加 ZSTD 压缩算法 #8923 #9747

目前 Apache Doris 中数据压缩方法是系统统一指定的，默认为 LZ4。针对部分对数据存储成本敏感的场景，例如日志类场景，原有的数据压缩率需求无法得到满足。

在 1.1 版本中，用户建表时可以在表属性中设置`"compression"="zstd"`将压缩方法指定为 ZSTD。在 25GB 1.1 亿行的文本日志测试数据中，最高获得了近 10 倍的压缩率、较原有压缩率提升了 53%，从磁盘读取数据并进行解压缩的速度提升了 30%。

8.7.16.3 功能优化

8.7.16.3.1 更全面的向量化支持

在 1.1 版本中，我们实现了计算层和存储层的全面向量化，包括：

- 实现了所有内置函数的向量化
- 存储层实现向量化，并支持了低基数字符串列的字典优化
- 优化并解决了向量化引擎的大量性能和稳定性问题。

我们对 Apache Doris 1.1 版本与 0.15 版本分别在 SSB 和 TPC-H 标准测试数据集上进行了性能测试：

- 在 SSB 测试数据集的全部 13 个 SQL 上，1.1 版本均优于 0.15 版本，整体性能约提升了 3 倍，解决了 1.0 版本中存在的部分场景性能劣化问题；
- 在 TPC-H 测试数据集的全部 22 个 SQL 上，1.1 版本均优于 0.15 版本，整体性能约提升了 4.5 倍，部分场景性能达到了十余倍的提升；

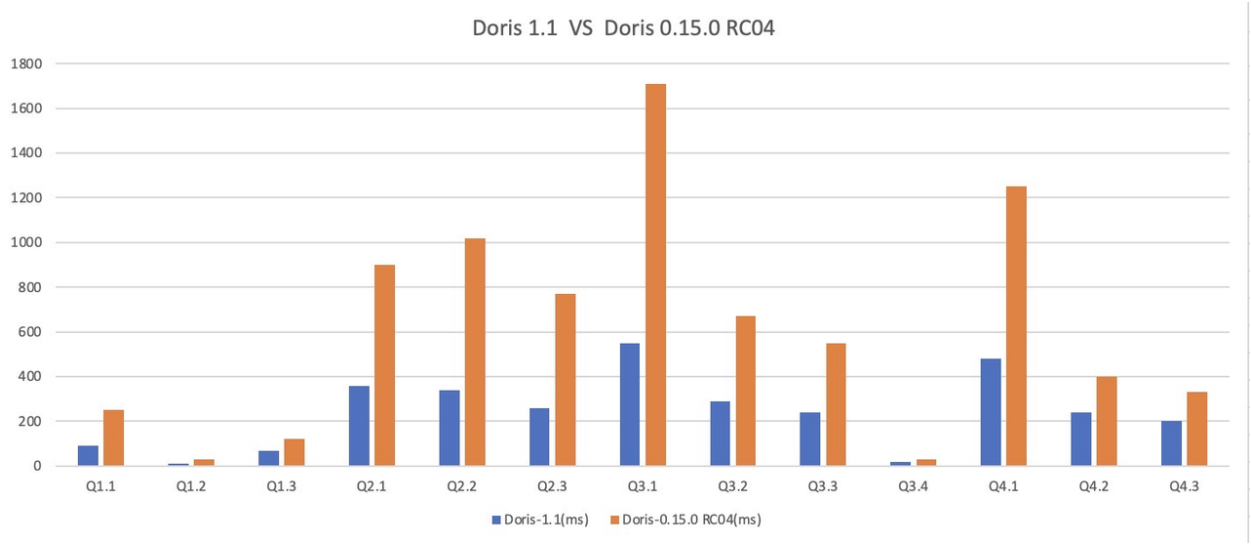


图 271: release-note-1.1.0-SSB

SSB 测试数据集

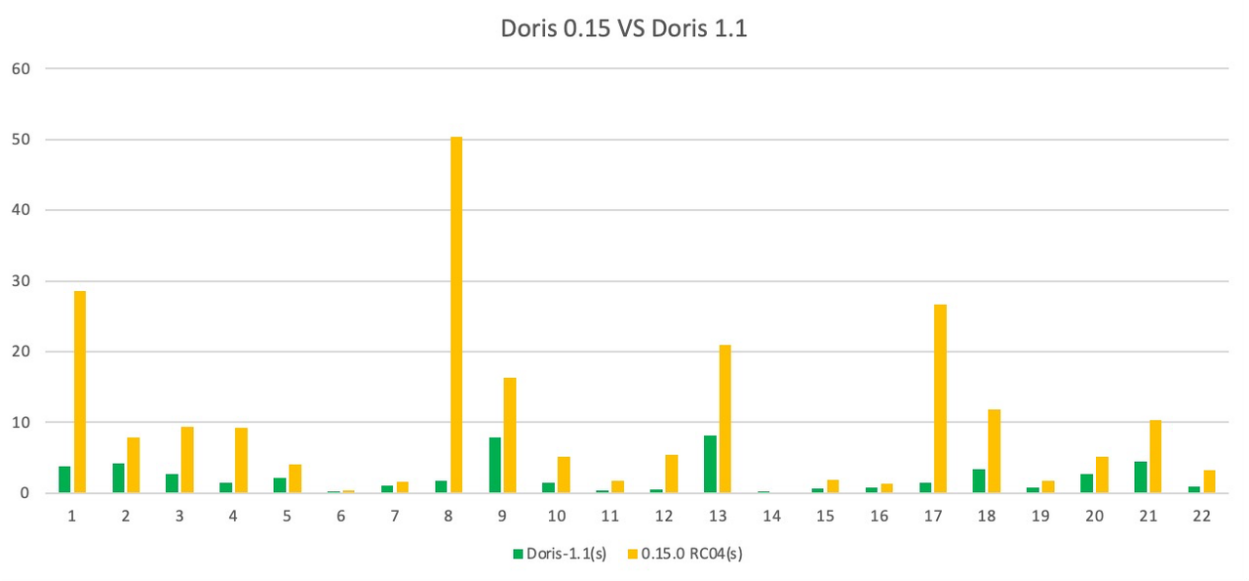


图 272: release-note-1.1.0-TPC-H

TPC-H 测试数据集

性能测试报告：

https://doris.apache.org/zh-CN/docs/benchmark/ssb

https://doris.apache.org/zh-CN/docs/benchmark/tpch

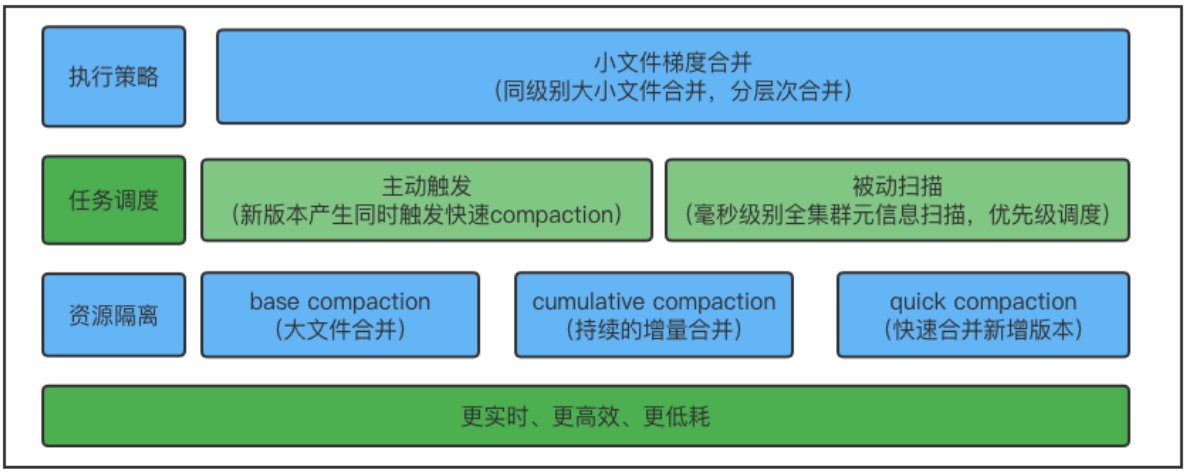
8.7.16.3.2 Compaction 逻辑优化与实时性保证 #10153

在 Apache Doris 中每次 Commit 都会产生一个数据版本，在高并发写入场景下，容易出现因数据版本过多且 Compaction 不及时而导致的 -235 错误，同时查询性能也会随之下降。

在 1.1 版本中我们引入了 QuickCompaction，增加了主动触发式的 Compaction 检查，在数据版本增加的时候主动触发 Compaction，同时通过提升分片元信息扫描的能力，快速发现数据版本过多的分片并触发 Compaction。通过主动式触发加被动式扫描的方式，彻底解决数据合并的实时性问题。

同时，针对高频的小文件 Cumulative Compaction，实现了 Compaction 任务的调度隔离，防止重量级的 Base Compaction 对新增数据的合并造成影响。

最后，针对小文件合并，优化了小文件合并的策略，采用梯度合并的方式，每次参与合并的文件都属于同一个数据量级，防止大小差别很大的版本进行合并，逐渐有层次的合并，减少单个文件参与合并的次数，能够大幅地节省系统的 CPU 消耗。



compaction优化

图 273:

在数据上游维持每秒 10w 的写入频率时（20 个并发写入任务、每个作业 5000 行、Checkpoint 间隔 1s），1.1 版本表现如下：

- 数据快速合并：Tablet 数据版本维持在 50 以下，Compaction Score 稳定。相较于之前版本高并发写入时频繁出现的 -235 问题，Compaction 合并效率有 10 倍以上的提升。
- CPU 资源消耗显著降低：针对小文件 Compaction 进行了策略优化，在上述高并发写入场景下，CPU 资源消耗降低 25%；
- 查询耗时稳定：提升了数据整体有序性，大幅降低查询耗时的波动性，高并发写入时的查询耗时与仅查询时持平，查询性能较之前版本有 3-4 倍提升。

图 274:

8.7.16.3.3 Parquet 和 ORC 文件的读取效率优化 #9472

通过调整 Arrow 参数，利用 Arrow 的多线程读取能力来加速 Arrow 对每个 row_group 的读取，并修改成 SPSC 模型，通过预取来降低等待网络的代价。优化前后对 Parquet 文件导入的性能有 4 ~ 5 倍的提升。

8.7.16.3.4 更安全的元数据 Checkpoint #9180 #9192

通过对元数据检查点后生成的 image 文件进行双重检查和保留历史 image 文件的功能，解决了 image 文件错误导致的元数据损坏问题。

8.7.16.4 Bug 修复

8.7.16.4.1 修复由于缺少数据版本而无法查询数据的问题。（严重）#9267 #9266

问题描述：`failed to initialize storage reader. tablet=924991.xxxx, res=-214, backend=xxxx`

该问题是在版本 1.0 中引入的，可能会导致多个副本的数据版本丢失。

8.7.16.4.2 解决了资源隔离对加载任务的资源使用限制无效的问题（中等）#9492

在 1.1 版本中，Broker Load 和 Routine Load 将使用具有指定资源标记的 BE 节点进行加载。

8.7.16.4.3 修复使用 HTTP BRPC 超过 2GB 传输网络数据包导致数据传输错误的问题（中等）#9770

在以前的版本中，当通过 BRPC 在后端之间传输的数据超过 2GB 时，可能会导致数据传输错误。

8.7.16.5 其他

8.7.16.5.1 禁用 Mini Load

Mini Load 与 Stream Load 的导入实现方式完全一致，都是通过 HTTP 协议提交和传输数据，在导入功能支持上 Stream Load 更加完备。

在 1.1 版本中，默认情况下 Mini Load 接口 `/_load` 将处于禁用状态，请统一使用 Stream Load 来替换 Mini Load。您也可以通过关闭 FE 配置项 `disable_mini_load` 来重新启用 Mini Load 接口。在版本 1.2 中，将彻底删除 Mini Load。

8.7.16.5.2 完全禁用 SegmentV1 存储格式

在 1.1 版本中将不再允许新创建 SegmentV1 存储格式的数据，现有数据仍可以继续正常访问。

您可以使用 ADMIN SHOW TABLET STORAGE FORMAT 语句检查集群中是否仍然存在 SegmentV1 格式的数据，如果存在请务必通过数据转换命令转换为 SegmentV2。

在 Apache Doris 1.2 版本中不再支持对 Segment V1 数据的访问，同时 Segment V1 代码将被彻底删除。

8.7.16.5.3 限制 String 类型的最大长度 #8567

String 类型是 Apache Doris 在 0.15 版本中引入的新数据类型，在过去 String 类型的最大长度允许为 2GB。

在 1.1 版本中，我们将 String 类型的最大长度限制为 1 MB，超过此长度的字符串无法再写入，同时不再支持将 String 类型用作表的 Key 列、分区列以及分桶列。

已写入的字符串类型可以正常访问。

8.7.16.5.4 修复 fastjson 相关漏洞 #9763

对 Canal 版本进行更新以修复 fastjson 安全漏洞

8.7.16.5.5 添加了 ADMIN DIAGNOSE TABLET 命令 #8839

通过 ADMIN DIAGNOSE TABLET tablet_id 命令可以快速诊断指定 Tablet 的问题。

8.7.16.6 下载使用

8.7.16.6.1 下载链接

https://doris.apache.org/zh-CN/download

8.7.16.6.2 升级说明

您可以从 Apache Doris 1.0 Release 版本和 1.0.x 发行版本升级到 1.1 Release 版本，升级过程请官网参考文档。如果您当前是 0.15 Release 版本或 0.15.x 发行版本，可跳过 1.0 版本直接升级至 1.1。

[https://doris.apache.org/zh-CN/docs/admin-manual/cluster-management/upgrade](https://doris.apache.org/zh-CN/docs/admin-manual/cluster-management/upgrade

8.7.16.6.3 意见反馈

如果您遇到任何使用上的问题，欢迎随时通过 GitHub Discussion 论坛或者 Dev 邮件组与我们取得联系。

GitHub 论坛：https://github.com/apache/incubator-doris/discussions

Dev 邮件组：dev@doris.apache.org

8.7.16.7 致谢

Apache Doris 1.1 Release 版本的发布离不开所有社区用户的支持，在此向所有参与版本设计、开发、测试、讨论的社区贡献者们表示感谢，他们分别是：

```
@adonis0147

@airborne12

@amosbird
```

@aopangzi

@arthuryangcs

@awakeljw

@BePPPower

@BiteTheDDDDt

@bridgeDream

@caiconghui

@cambyzju

@ccoffline

@chenlinzhong

@daikon12

@DarvenDuan

@dataalive

@dataroaring

@deardeng

@Doris-Extras

@emerkfu

@EmmyMiao87

@englefly

@Gabriel39

@GoGoWen

@gtchaos

@HappenLee

@hello-stephen

@Henry2SS

@hewei-nju

@hf200012

@jacktengg

@jackwener

@Jibing-Li

@JNSimba

@kangshisen

@Kikyou1997

@kylinmac

@Lchangliang

@leo65535

@liaoxin01

@liutang123

@lovingfeel

@luozenglin

@luwei16

@luzhijing

@mklzl

@morningman

@morrySnow

@nextdreamblue

@Nivane

@pengxiangyu

@qidaye

@qzsee

@SaintBacchus

@SleepyBear96

@smallhibiscus

@spaces-X

@stalary

@starocean999

@steadyBoy

@SWJTU-ZhangLei

@Tanya-W

@tarepanda1024

@tianhui5

@Userwhite

@wangbo

@wangyf0555

@weizuo93

@whutpencil

@wsjz

@wunan1210

@xiaokang

@xinyiZzz

@xlwh

@xy720

@yangzhg

@Yankee24

@yiguolei

@yinzhijian

@yixiutt

@zbtzbtzbt

@zenoyang

@zhangstar333

@zhangyifan27

@zhannngchen

@zhengshengjun

@zhengshiJ

@zingdle

@zuochunwei

@zy-kkk

Copyright © 2024 The Apache Software Foundation,Licensed under the Apache License, Version 2.0. Apache, Doris, Apache Doris, the Apache feather logo and the Apache Doris logo are trademarks of The Apache Software Foundation.